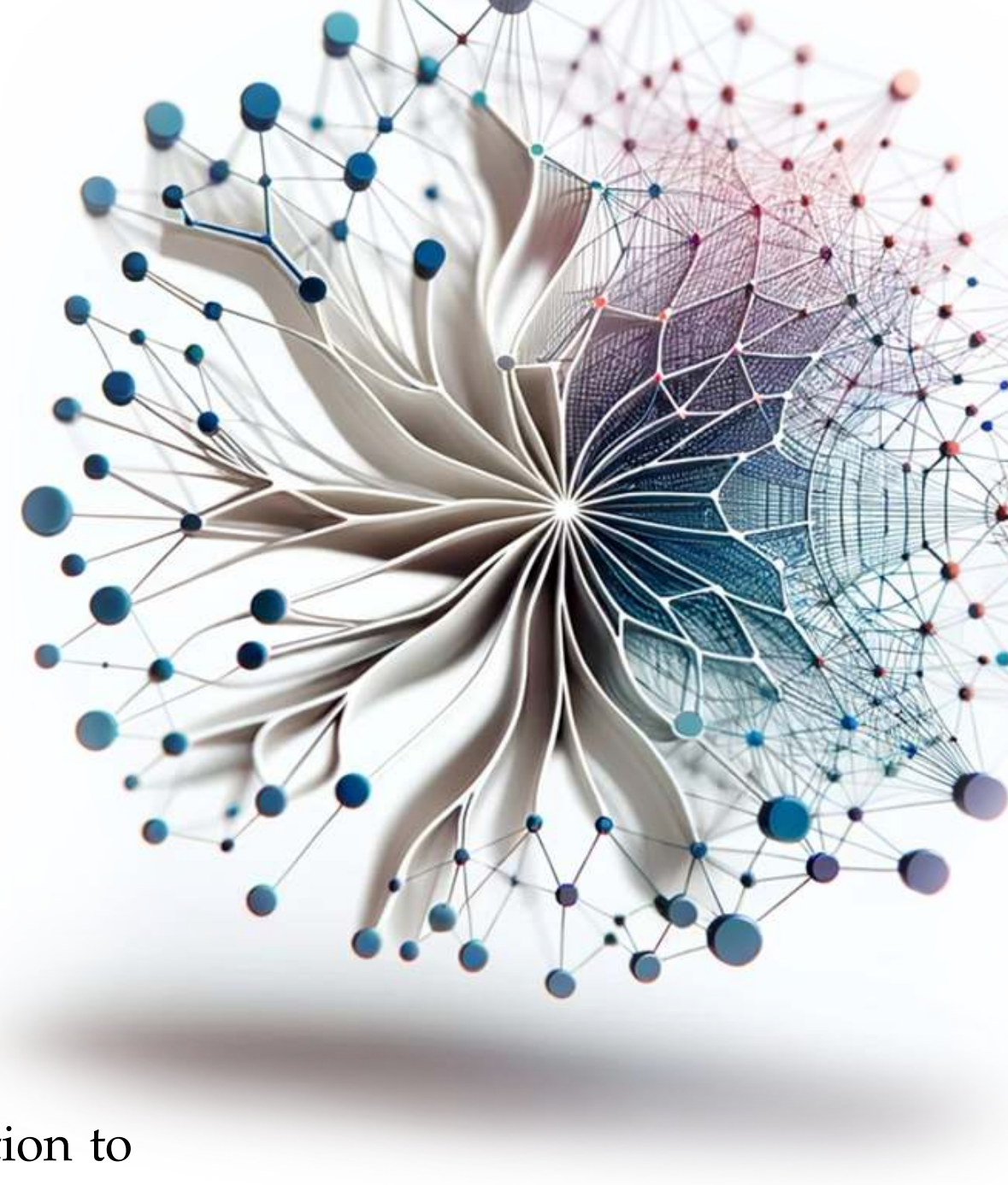


Introduction to

Machine Learning Systems

Vijay
Janapa Reddi

Introduction to Machine Learning Systems

From Algorithms to Silicon: The Engineering of Deep Learning Systems

Introduction to Machine Learning Systems

From Algorithms to Silicon: The Engineering of Deep Learning Systems

Vijay Janapa Reddi

The MIT Press
Cambridge, Massachusetts
London, England

The MIT Press
Massachusetts Institute of Technology
77 Massachusetts Avenue
Cambridge, MA 02139
mitpress.mit.edu

© 2027 Massachusetts Institute of Technology

This work is subject to a Creative Commons CC-BY-NC-ND license.

This license applies only to the work in full and not to any components included with permission. Subject to such license, all rights are reserved. No part of this book may be used to train artificial intelligence systems without permission in writing from the MIT Press.



The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers.
This book was prepared and formatted in Quarto by Željko Hrček. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data is available.

ISBN: 978-0-262-05888-9

EU Authorised Representative: Easy Access System Europe, Mustamäe tee 50, 10621 Tallinn, Estonia | Email: gpsr.requests@easproject.com

In loving memory of my father,
who taught me how to learn.

And with gratitude to my mother,
who taught me how to give.

Contents

Foreword xiii

Author's Note xvii

Preface xix

Acknowledgments xxvii

A Note on AI Assistance xxix

Notation xxxi

Main Text

Chapter 1 Introduction 3
1.1 The AI Systems Moment . . . 5
1.2 Data-Centric Paradigm Shift . . . 5
1.3 The Evolution of AI Bottlenecks . . . 8
1.4 The Bitter Lesson . . . 12
1.5 Defining ML Systems . . . 14
1.6 ML vs. Traditional Software . . . 16
1.7 Iron Law of ML Systems . . . 18
1.8 Three Dimensions of ML Efficiency . . . 22
1.9 AI Engineering as a Discipline . . . 24
1.10 Book Organization . . . 27
1.11 Fallacies and Pitfalls . . . 29
1.12 Summary . . . 30

Part I: Foundations of ML Systems

Foundation Principles 35

Chapter 2 System Paradigms 37
2.1 Deployment Paradigm Framework . . . 39
2.2 Physical Constraints: Why Paradigms Exist . . . 40
2.3 Analyzing Workloads . . . 42
2.4 System Balance and Hardware . . . 45
2.5 Cloud ML: Computational Power . . . 49
2.6 Edge ML: Latency and Privacy . . . 55
2.7 Mobile ML: Offline Intelligence . . . 61
2.8 TinyML: Ubiquitous Sensing . . . 64
2.9 Paradigm Selection . . . 67
2.10 Hybrid Architectures . . . 71
2.11 System Entropy: Why Deployment Is Not the End . . . 75

2.12 Fallacies and Pitfalls . . . 76
2.13 Summary . . . 78

Chapter 3 ML Workflow **81**
3.1 ML Lifecycle . . . 83
3.2 Case Study: DR Screening . . . 84
3.3 Quantifying the ML Lifecycle . . . 86
3.4 ML vs. Traditional Software . . . 87
3.5 Constraint Propagation Principle . . . 88
3.6 Lifecycle Stages . . . 89
3.7 Problem Definition . . . 93
3.8 Data Collection and Preparation . . . 94
3.9 Model Development and Training . . . 98
3.10 Evaluation and Validation . . . 101
3.11 Deployment and Integration . . . 104
3.12 Monitoring and Maintenance . . . 106
3.13 Systems Thinking . . . 108
3.14 Fallacies and Pitfalls . . . 110
3.15 Summary . . . 111

Chapter 4 Data Engineering **113**
4.1 Dataset Compilation . . . 114
4.2 Physics of Data . . . 115
4.3 Four Pillars Framework . . . 118
4.4 Data Acquisition . . . 126
4.5 Data Pipeline Architecture . . . 131
4.6 Systematic Data Processing . . . 145
4.7 Data Labeling . . . 153
4.8 Storage Architecture . . . 159
4.9 Operational Data Health . . . 168
4.10 Fallacies and Pitfalls . . . 172
4.11 Summary . . . 174

Part II: Development and Training

Development Principles **179**

Chapter 5 Neural Computation **181**
5.1 From Logic to Arithmetic . . . 182
5.2 Computing with Patterns . . . 183
5.3 Why Neural Computation Scaled When It Did . . . 189
5.4 Neural Network Fundamentals . . . 192
5.5 Learning Process . . . 204
5.6 Inference Pipeline . . . 225
5.7 Handwritten Digit Recognition . . . 230
5.8 D·A·M Reading of Neural Computation . . . 234
5.9 Fallacies and Pitfalls . . . 235
5.10 Summary . . . 236

Chapter 6 Network Architectures **239**
6.1 Architectural Principles . . . 240
6.2 Lighthouse Roster: Model Biographies . . . 242
6.3 Workload Signatures: The Arithmetic Intensity Spectrum . . . 243
6.4 MLPs: Dense Pattern Processing . . . 245
6.5 CNNs: Spatial Pattern Processing . . . 250
6.6 RNNs: Sequential Pattern Processing . . . 261

6.7 Attention: Dynamic Processing . . . 265
6.8 Transformers: Parallel Sequence Processing . . . 271
6.9 Sparse Architectures: Recommendation Systems . . . 277
6.10 What Each Architecture Asks of the Machine . . . 280
6.11 Fallacies and Pitfalls . . . 291
6.12 Summary . . . 292

Chapter 7 ML Frameworks **295**
7.1 Three Framework Problems . . . 296
7.2 The Ladder of Abstraction . . . 298
7.3 Execution Problem . . . 299
7.4 Differentiation Problem . . . 314
7.5 Abstraction Problem . . . 327
7.6 Model Composition and State . . . 340
7.7 Framework Platform Analysis . . . 344
7.8 Deployment Targets . . . 349
7.9 Framework Selection . . . 350
7.10 Anatomy of a Training Step . . . 354
7.11 Fallacies and Pitfalls . . . 358
7.12 Summary . . . 359

Chapter 8 Model Training **361**
8.1 Training Systems Fundamentals . . . 362
8.2 The Staged System Pipeline: Identifying "Accelerator Bubbles" . . . 363
8.3 Training-Time Law . . . 364
8.4 Training Computation and Memory Costs . . . 369
8.5 Pipeline Architecture . . . 386
8.6 Identifying Bottlenecks . . . 396
8.7 Pipeline Optimizations . . . 398
8.8 Scaling Training Systems . . . 424
8.9 Fallacies and Pitfalls . . . 432
8.10 Summary . . . 433

Part III: Optimization and Acceleration

Optimization Principles **437**

Chapter 9 Data Selection **441**
9.1 Data Selection Fundamentals . . . 442
9.2 Defining Data Selection . . . 443
9.3 Systems Perspective . . . 444
9.4 Information-Compute Ratio . . . 446
9.5 The ICR Frontier: When Data Becomes a Tax . . . 447
9.6 Measurement Framework . . . 449
9.7 Static Pruning . . . 452
9.8 Dynamic Selection . . . 458
9.9 Self-Supervised Learning . . . 463
9.10 Synthetic Data Generation . . . 464
9.11 Selection Engineering . . . 467
9.12 Distributed Selection . . . 472
9.13 Cost Modeling . . . 474
9.14 Decision Framework . . . 479
9.15 Cross-Layer Interactions . . . 482
9.16 Fallacies and Pitfalls . . . 483
9.17 Summary . . . 486

Chapter 10 Model Compression **489**
10.1 Optimization Framework . . . 491
10.2 Deployment Context . . . 493
10.3 Structural Optimization . . . 496
10.4 Quantization and Precision . . . 521
10.5 Hardware Implementation . . . 540
10.6 Technique Selection . . . 545
10.7 Combining and Sequencing Techniques . . . 547
10.8 Efficiency Measurement . . . 548
10.9 Implementation Tools . . . 551
10.10 Fallacies and Pitfalls . . . 553
10.11 Summary . . . 555

Chapter 11 Hardware Acceleration **565**
11.1 Acceleration Fundamentals . . . 566
11.2 Hardware Specialization . . . 571
11.3 AI Compute Primitives . . . 577
11.4 Compute Units and Execution Models . . . 585
11.5 AI Memory Systems . . . 597
11.6 Roofline Model . . . 611
11.7 Hardware Mapping . . . 619
11.8 Dataflow Optimization . . . 624
11.9 Compiler Support . . . 636
11.10 Runtime Support . . . 641
11.11 Multi-Chip Scaling . . . 644
11.12 Heterogeneous SoC Design . . . 645
11.13 Hardware Sustainability . . . 649
11.14 Fallacies and Pitfalls . . . 649
11.15 Summary . . . 652

Chapter 12 Benchmarking **655**
12.1 ML Benchmarking Framework . . . 656
12.2 Historical Foundations . . . 658
12.3 System Benchmarking Suites . . . 661
12.4 Benchmarking Granularity . . . 669
12.5 Benchmark Components . . . 674
12.6 Training vs. Inference . . . 682
12.7 Training Benchmarks . . . 683
12.8 Inference Benchmarks . . . 689
12.9 Power Measurement Techniques . . . 702
12.10 Benchmarking Best Practices . . . 707
12.11 Model and Data Evaluation . . . 712
12.12 Production Considerations . . . 721
12.13 Fallacies and Pitfalls . . . 722
12.14 Summary . . . 724

Part IV: Deployment and Operations

Deployment Principles **729**

Chapter 13 Model Serving **733**
13.1 Serving Paradigm . . . 735
13.2 Serving Load, Latency, and Architecture . . . 736
13.3 Serving System Architecture . . . 742
13.4 Request Lifecycle . . . 745
13.5 Queuing Theory . . . 752

13.6 Model Lifecycle Management . . . 758
13.7 Throughput Optimization . . . 762
13.8 Inference Runtime Selection . . . 775
13.9 Node-Level Optimization . . . 778
13.10 LLM Serving . . . 783
13.11 Economics and Planning . . . 790
13.12 Fallacies and Pitfalls . . . 794
13.13 Summary . . . 796

Chapter 14 ML Operations **799**
14.1 MLOps Overview . . . 800
14.2 MLOps Principles . . . 801
14.3 Technical Debt . . . 806
14.4 Development Infrastructure . . . 813
14.5 Deployment and Serving . . . 827
14.6 Production Monitoring . . . 834
14.7 Incident Response and Governance . . . 845
14.8 Design and Maturity Framework . . . 853
14.9 Case Studies . . . 858
14.10 Fallacies and Pitfalls . . . 864
14.11 Summary . . . 866

Chapter 15 Responsible Engineering **869**
15.1 Responsibility as Systems Engineering . . . 870
15.2 Engineering Responsibility Gap . . . 871
15.3 Responsible Engineering Across the Lifecycle . . . 881
15.4 Environmental and Cost Awareness . . . 895
15.5 Data Governance and Compliance . . . 901
15.6 Fallacies and Pitfalls . . . 909
15.7 Summary . . . 911

Synthesis

Chapter 16 Conclusion **915**
16.1 Synthesizing ML Systems . . . 916
16.2 Lighthouse Models: Constraint Propagation . . . 917
16.3 Thirteen Quantitative Principles . . . 919
16.4 The Dual Mandate Revisited . . . 924
16.5 Future Directions . . . 925
16.6 Journey Forward . . . 928
16.7 Fallacies and Pitfalls . . . 929
16.8 Summary . . . 930

Back Matter

Appendices **935**

Appendix A The D·A·M Taxonomy **935**

Appendix B Data Foundations **943**

Appendix C Algorithm Foundations **949**

Appendix D Machine Foundations **963**

Appendix E System Assumptions **977**

Glossary **989**

References **1001**

Index **1029**

Foreword

David A. Patterson
Pardee Professor of Computer Science Emeritus
University of California, Berkeley
Distinguished Engineer, Google
ACM A.M. Turing Award Laureate
Berkeley, California

My career has been dedicated to understanding and improving the very foundations upon which computing operates – from the instruction set architectures that define processors to the storage systems that hold our data. I've seen firsthand how profound advancements are rarely about a single brilliant idea in isolation, but rather about the holistic integration of many ideas into a robust, efficient, and reliable system.

Today, we stand at another pivotal moment. Machine learning (ML) has transcended scientific curiosity to become a force reshaping industries, driving innovation, and transforming our daily lives. The intellectual power of the algorithms, the gains in accuracy, and the seemingly magical capabilities they unlock have rightly captured the world's imagination.

One concern is that much of this progress has been achieved by a community that, through no fault of its own, was never taught to think in systems terms about what it was building. Engineers who can call `model.fit()` but cannot explain why training is slow. Researchers who tune hyperparameters without understanding the memory bandwidth constraints that make certain choices impractical. Practitioners who deploy models into production without a framework for reasoning about latency, throughput, or failure modes.

A brilliant algorithm, without a well-engineered system to support it, is like a powerful engine without a chassis and brakes. It won't get you far, and it certainly won't get you there reliably.

This gap is not a talent deficit. It is a curriculum deficit. The field has simply not had the textbooks it deserves. This book, *Introduction to Machine Learning Systems*, arrives precisely at the right time to illuminate this critical, often overlooked, dimension of the ML revolution.

Why Systems Thinking Matters

My career has leveraged the power of systems thinking. Identify the bottlenecks, understand the interactions between components, and design for performance, reliability, cost-effectiveness, and ease of use across the entire stack. You could not reach the conclusion from first principles alone. You had to think about the system.

ML systems demand exactly the same discipline. A neural network is not a mathematical abstraction floating free of physical constraints. It is a computation that must execute on silicon with finite memory bandwidth, finite power budgets, and finite communication capacity. Arithmetic intensity, the Roofline Model, and memory hierarchy effects are not advanced topics to be addressed after students learn the basics. They are the basics.

Why ML Systems Deserve Serious Textbook Treatment

The purpose of a textbook is not just to catalog the current frontier. It is to provide the conceptual scaffolding that allows students to understand the frontier, and to advance it themselves. The neural network architectures covered in these pages will evolve. The hardware accelerators described will

be superseded. But some fundamentals will not change. We will still have memory hierarchies and arithmetic intensity will still be a critical measure. The principles of pipelining, of latency versus throughput trade-offs, of quantization and precision reflect physical and mathematical realities that do not go out of date. Anyone who genuinely understands them will be able to reason about whatever new system the field produces next year or ten years from now.

I am also struck by the scope of what this book attempts. It is not enough to explain how training works; one must also address data engineering, model optimization, hardware acceleration, deployment, benchmarking, responsible AI, and sustainability. These are not separate topics bolted together for completeness. They are interconnected aspects of a single engineering discipline. A decision made at the data layer has implications for model behavior, which has implications for inference latency, which has implications for deployment cost, which has implications for the communities that can and cannot access the resulting system. The book correctly treats this integration as a first-class concern.

I am particularly pleased to see the quantitative methodology that runs through every chapter. Claims are backed by measurements. Trade-offs are expressed numerically. Students are taught to estimate, measure, and verify rather than to rely on their gut. The difference between an engineer and a dilettante is precisely this capacity for rigorous, measurement-grounded reasoning.

Why Open Access Is Important for the Next Generation

I applaud the decision to make this invaluable resource freely available.

When I co-wrote *Computer Architecture: A Quantitative Approach* with John Hennessy, we debated how to make the material as accessible as possible. We lobbied our publisher to keep the price down and even came out with a low cost international student edition for sales outside North America. Those concerns have only grown more urgent as ML has become infrastructure—as consequential to the modern world as the electrical grid or the internet.

The decision to publish this book as a freely available educational resource is not a compromise. It is a statement of values. ML systems are being deployed in every sector of every economy on every continent. The engineers who build and maintain them will come from everywhere. If the foundational education required to do this work responsibly is available only to students at well-resourced institutions, we will contribute to an expertise gap that mirrors the societal concerns of who benefits from AI and who bears its costs.

What impresses me further is that this project has gone well beyond open access to a static text. The accompanying ecosystem—interactive laboratories, a build-from-scratch framework in TinyTorch, hardware deployment kits, instructor materials, a global network of adopting universities—reflects a familiar educational maxim: "I hear and I forget. I see and I remember. I do and I understand."

The open-source model also makes the curriculum itself better. Errors get corrected. Examples get sharpened. New chapters emerge in response to what the field actually needs. As we found with *Computer Architecture: A Quantitative Approach*, a community of contributors that forms around a book can offer a form of quality assurance that no single author team, however expert, can match alone.

A Final Word

The field of computer architecture has a tradition I am proud of: we build things, measure them honestly, share what we learn, and revise our philosophies when the measurements demand it. We do not hide behind the complexity of our systems to avoid accountability for whether they actually work. We do not conflate novelty with progress. We debate, rigorously, about what matters and why.

Hopefully, ML systems engineering will follow this tradition. It needs the habits of mind that come from taking systems seriously: thinking about interactions, measuring at the right level of abstraction, being honest about trade-offs, and holding oneself accountable to real-world performance rather than benchmark folklore. This book embodies those habits.

Introduction to Machine Learning Systems is more than just a textbook; it is a clarion call for systems thinking in the age of AI, a guide for building robust ML applications, and a testament to the power of open education.

We are at an extraordinary moment. Over my career there has been a major breakthrough every decade: the microprocessor in the 1970s, the personal computer in the 1980s, the Internet in the 1990s, the smartphone in the 2000s, and AI in the 2010s. The first four changed business and society, yet AI looks likely to have the largest impact of them all.

The systems being built today will shape how intelligence is deployed, who benefits from it, and what it costs—in money, in energy, and in carbon footprint. We need the engineers who build those systems to have the best possible systems skills.

This book is a significant step toward making it possible for anyone to learn them.

David A. Patterson

Author's Note

The world is rushing to build AI systems; it is not yet engineering them. That sentence is the reason this book exists. A model that wins on a benchmark is not yet a system, and the distance between the two is not a detail of implementation. It is engineering: the work of making something hold up under constraints that do not negotiate. A demo answers to no one. A system answers to latency budgets, memory limits, power envelopes, and the cost of being wrong in front of real users.

What makes this its own discipline is an unusual coupling. A processor answers to the physics of the machine, to clock cycles, cache lines, and the heat it can shed. A statistical model answers to the mathematics of learning, to bias, variance, and how generalization changes with data and model capacity. A machine learning system must answer to both at once, in the same decision, and the two do not always agree. A change that pleases the hardware can starve the model of the precision or throughput it needs to converge; a change that helps the model can ask the silicon for more memory or bandwidth than it can give. Hold that tension in view and the field stops looking like a pile of tools and starts looking like a structure you can reason about.

This book hands you that structure. Not the framework that is fashionable this year or the accelerator that is fastest this quarter, but the way of thinking that outlasts them. Frameworks change, hardware turns over every few years, and the benchmarks reset; the forces underneath stay the same. If the book works, you will finish it able to look at any machine learning system, from a sensor sipping microwatts to a server farm drawing megawatts, and see the same small set of forces deciding what is possible.

The textbooks that changed me when I was a student did not hand me facts. They handed me a way of seeing, and the facts arranged themselves around it. That is what I have tried to do here.

— Vijay Janapa Reddi
Cambridge, Massachusetts
2026

Preface

Who This Book Is For

This book is for anyone who wants to understand *how* machine learning systems work, from the mathematics that define a neural network to the hardware that executes it and the engineering decisions that make it run efficiently.

Most readers will not become ML hardware architects or compiler engineers. Yet every practitioner who builds, trains, optimizes, or deploys a model makes decisions that depend on understanding the system beneath the code. This book provides that systems foundation for engineers encountering ML systems for the first time, practicing professionals deepening their understanding, and researchers who need to reason about performance and deployment.

Why a Systems Textbook

In 1968, a NATO conference in Garmisch, Germany, helped establish *software engineering* as a distinct discipline amid concern that software systems had become too complex for ad hoc programming (Naur and Randell 1969). Reliable software required its own principles, methods, and rigor. Decades later, Hennessy and Patterson made quantitative analysis central to computer architecture education, systematizing how engineers measure, predict, and optimize processor performance (Hennessy and Patterson 2017). Both examples show how shared methods make complex engineering work systematic and teachable.

Machine learning faces its own Garmisch moment. For the past decade, ML research has been dominated by *model-centric* thinking focused on discovering new algorithms and architectures. That focus matters, but a model is not a system. A neural network that performs well in a research notebook is useless if it cannot meet latency constraints, if its data pipeline cannot handle the required volume, or if its energy cost exceeds its economic value. The gap between a working model and a production system is not merely an implementation gap; it is a gap in fundamental engineering principles.

This book treats ML systems engineering not as a collection of tools, such as Kubernetes, PyTorch, and CUDA, but as a discipline grounded in quantitative bounds, models, and design principles. Just as civil engineers rely on foundational physics and structural principles, ML systems engineers must make the governing principles explicit. Four ideas anchor this book:

- **The D·A·M taxonomy**: Every bottleneck lives in the data, the algorithm, or the machine, and relieving one can expose another.
- **The iron law of ML systems**: Total latency decomposes into movement, computation, and overhead; where operations run concurrently (such as transferring the next batch while computing the current one), elapsed time is governed by the critical path.
- **The bitter lesson**: General methods that exploit growing computation outpace hand-encoded expertise, which makes systems efficiency a lever of progress.
- **The energy-movement principle**: Per-event costs and event counts determine whether movement or computation dominates.

Each part adds its own governing principles, such as data gravity and Amdahl's law, at the point where they first bind.

Computer science studies what is computable. ML research studies what is learnable. AI engineering studies what is buildable, given the physics of real hardware and real data.

I wrote this book to help build that foundation and to do for ML systems what Hennessy and Patterson (2017) did for computer architecture: replace intuition with measurement, and art with engineering. The goal is to teach system-level reasoning before implementation, treating data, algorithms, and hardware not as separate silos but as a single, coupled optimization problem.

Consider a box of LEGO bricks. The same interlocking pieces build a spaceship, a castle, or an entire city. The sets change; the bricks do not. ML systems work the same way. A convolutional network for image classification, a transformer for language generation, and a reinforcement learning agent for robotics are vastly different applications, but beneath them sit the same building blocks: computational graphs, memory hierarchies, data pipelines, optimization loops, and the trade-offs between latency, throughput, and energy. Mastery of those building blocks makes unfamiliar systems legible.

That is why this book teaches enduring principles rather than current tools. Dominant frameworks have shifted from Theano to TensorFlow to PyTorch in under a decade. Hardware has evolved from repurposed graphics processors to purpose-built tensor accelerators, with new architectures emerging every year. Tool-specific guidance ages as frameworks and hardware change. The building blocks endure.

What This Book Covers

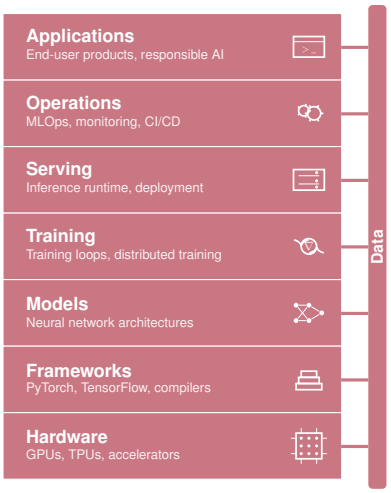
This book centers on the *single compute node*: one host with one to eight accelerators, each with discrete device memory connected across a local interconnect such as PCIe or NVLink. Production AI systems certainly extend to warehouse-scale clusters spanning thousands of accelerators and complex networks; those fleet-scale architectures form the broader horizon of the field. Yet every distributed cluster is fundamentally an array of individual nodes. An engineer cannot optimize a cluster of a thousand accelerators without first understanding how a single node orchestrates memory, manages pipelines, and feeds its execution units. Node-level mastery is the prerequisite for reasoning about systems at any scale.

The content is organized into four parts, each with a distinct role in the systems argument:

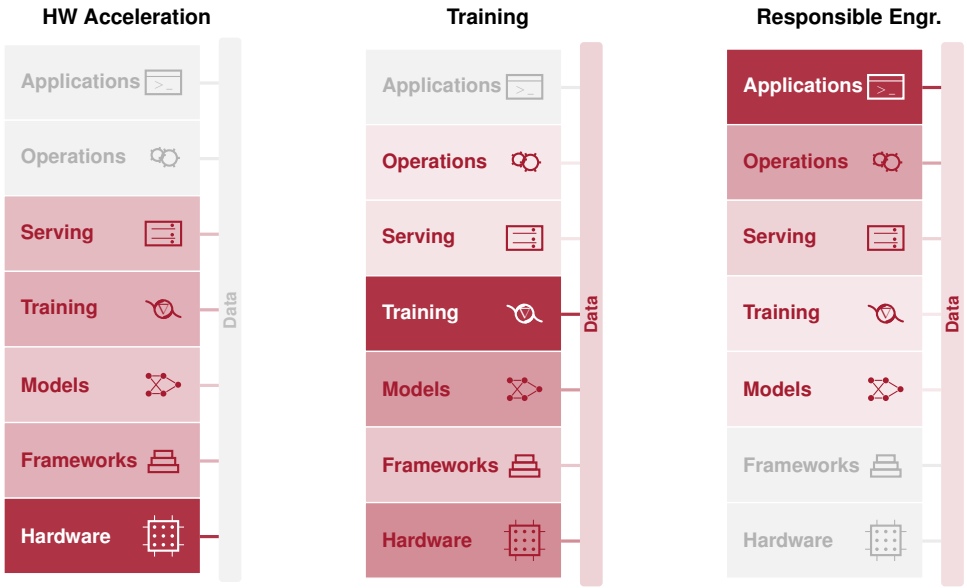
- **Part I: Foundations of ML Systems** develops the vocabulary, mental models, and end-to-end workflow that guide every ML project, from the characteristics that distinguish ML systems from traditional software through the data engineering pipelines that feed them.
- **Part II: Development and Training** connects mathematical foundations to working systems through deep learning computation, neural architectures, frameworks, and training procedures.
- **Part III: Optimization and Acceleration** addresses data efficiency, model compression, hardware acceleration, and the benchmarking methodology needed to measure progress rigorously.
- **Part IV: Deployment and Operations** covers serving infrastructure, operational practices, and responsible engineering for systems that must be fair, transparent, and trustworthy in production.

Each part builds on the previous one, establishing concepts that subsequent chapters expand and apply. Together, the four parts trace a path through the ML systems stack, a layered abstraction modeled on the classical computer systems stack. Each layer provides an abstraction to the layer above and consumes the layer below: from hardware at the bottom through frameworks, models, training, and serving, up to operations and applications at the top. Data flows through all layers as a cross-cutting interconnect, much like a data bus in computer architecture.

Throughout the book, a margin figure highlights which layers each chapter addresses, providing a recurring map of the stack.

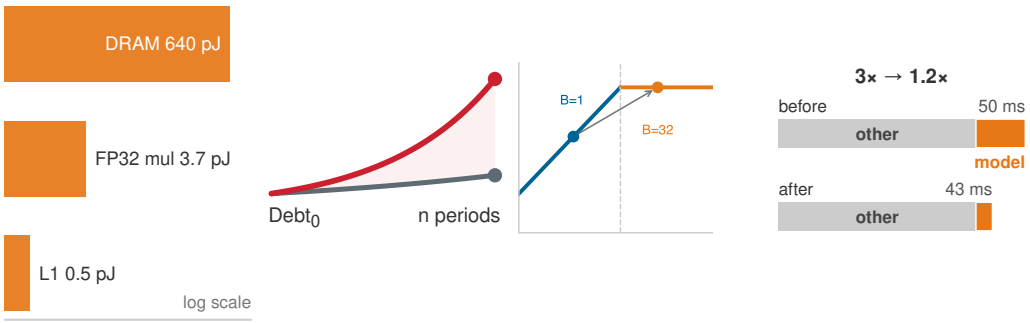


The data bar alongside the stack is not merely decorative. Its uniform shading reflects how central data is to each chapter's concerns, a separate dimension from the layer intensities. The connecting wires between each layer and the data bar show which layers interact with data in that chapter's context. Three chapters illustrate how the lens shifts.



In chapter 11, the layer intensity concentrates at the bottom of the stack while the data bar is faint: the chapter focuses on hardware mechanisms rather than dataset construction. In chapter 8, the middle layers glow and the data bar is moderately lit, reflecting that training consumes data but is fundamentally about optimization. In chapter 15, the top layers dominate and the data bar carries a moderate tint, because biased training data surfaces as biased applications even though several layers sit between them. The same stack tells three different stories, with data as an independent dimension.

The margins also carry small visual notes at the point where a relationship is easiest to understand by sight. These are not numbered figures to cite later; they are reading aids. A ladder makes an energy or latency scale gap visible, a trend sketch shows direction, a roofline thumbnail shows whether memory bandwidth or compute throughput bounds performance, and a dominance bar shows which term controls total runtime. When a miniature uses a log scale, it says so inside the image.



Alongside these visual notes, unnumbered **marginal navigation pointers** connect an idea to the chapter that establishes it or builds on it, without interrupting the prose:

- ↩ **Backward pointer:** Directs the reader back to the earlier chapter where a principle, constraint, or cost model was established.
- ↪ **Forward pointer:** Directs the reader ahead to the later chapter, or to the appendix, where the current idea is developed, measured, or enforced.
- ⇄ **Contrast:** Marks a place where two chapters treat the same problem differently, so the comparison is visible at a glance.

A pointer only orients. When a sentence depends on another chapter, the reference stays in the text.

Suggested Reading Paths

While sequential reading provides the most comprehensive foundation, readers with different backgrounds and goals may follow tailored paths through the material:

- **The Complete Path** (all chapters, in order): For engineers and researchers new to ML systems. Start with chapter 1, then read the four parts in order. This path develops every concept from first principles.
- **The ML Engineer's Path** (systems depth for practitioners): For engineers who already train and deploy models but want to understand the systems underneath. Skim Part I for vocabulary, then focus on chapter 7, chapter 8, chapter 11, chapter 12, and chapter 13.
- **The Optimization Path** (efficiency-focused): For engineers working on model efficiency, edge deployment, or resource-constrained systems. After Part I, review Part II's treatment of forward and backward computation, architecture costs, framework execution, and training bottlenecks before proceeding to Part III. Readers who already understand these mechanisms may proceed directly to optimization.

Whichever path a reader takes, figure 0.1 shows which earlier chapters each chapter builds on. Readers who already understand the prerequisite concepts may skip the corresponding chapters; others should follow the arrows back to establish that foundation. The widest convergence is model training, which draws on data engineering, network architectures, and ML frameworks at once; the longest chain runs through Part III, where compression hands its results to acceleration and benchmarking closes the loop before deployment.

Conventions

This book uses a small set of typographic conventions to mark what each piece of prose is doing.

Bold marks first definitions

A term in **bold** within body prose is its formal first introduction in the volume. Each term receives this treatment exactly once. Subsequent appearances are unbolded; by then the term is part of the working vocabulary the chapter carries forward.

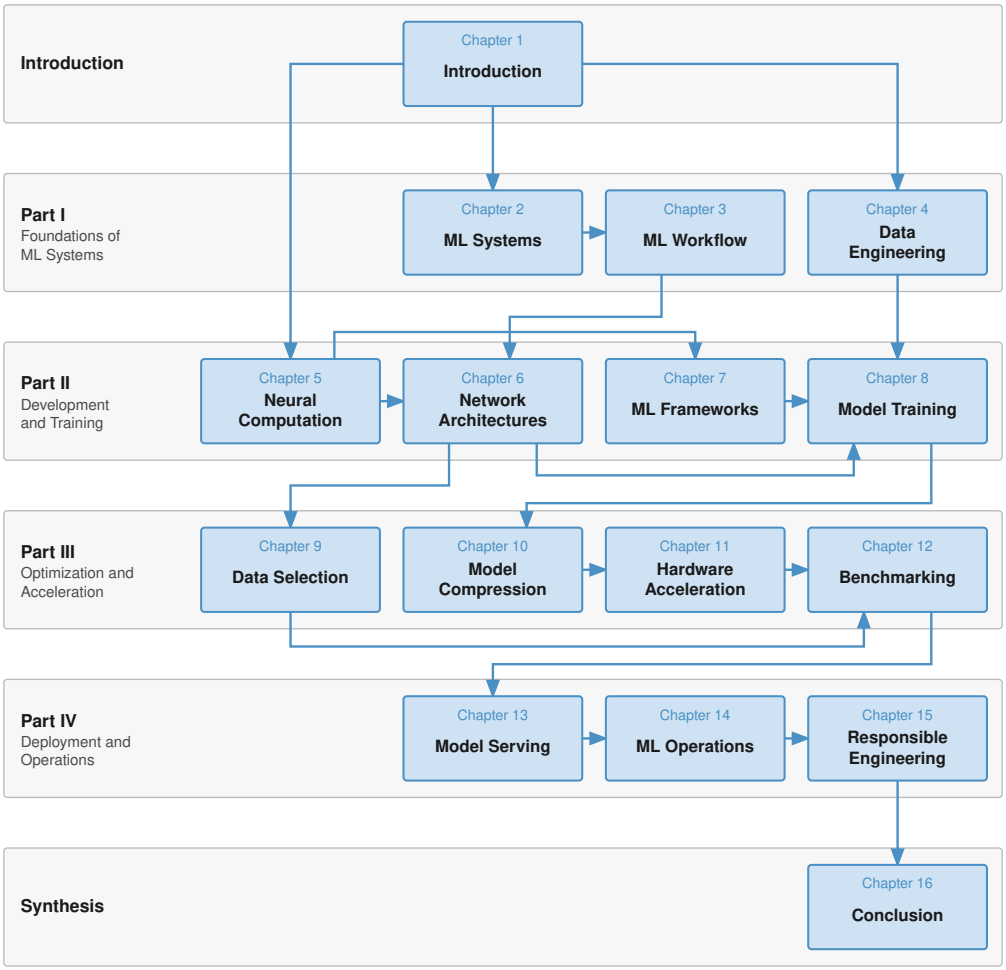


Figure 0.1: Chapter Dependency Map: Each arrow points from a chapter to a later chapter that builds on its results, derived from the book's cross-references with transitive links omitted. The introduction frames every chapter; arrows from it mark the chapters that draw on it directly.

> Every machine learning system has a **dual mandate**: it must manage statistical uncertainty and physical constraints simultaneously.

Bold identifies a term that readers will use throughout the book. A matching index entry in the back of the book points to the defining occurrence (under the `definition` subentry), providing a way to return to it. Key technical terms are also compiled in the glossary.

Bold also appears in a second, structural role: as a functional label at the start of an item inside a callout box or a summary bullet—for example, **Recommendation**, **Trade-off**, or **Result**. In that context, the bold is scaffolding rather than a first definition.

Italics carry specific rhetorical jobs

Italics serve specific purposes rather than general emphasis. Common uses include:

- **Direct opposition**: *training* vs. *inference*, *logical* vs. *physical*.
- **Impossibility, stated as physical, mathematical, or logical law**: "sub-10 ms systems *cannot* use distant cloud infrastructure—physics forbids it."
- **Word used as word**: "the term *gradient* refers to a vector of partial derivatives."

- **Titles of works**: *Computer Architecture: A Quantitative Approach.*
- **Thesis punchline**: A single italicized sentence that captures the controlling insight of a section, for example, *Data is source code.*

Callouts have visual identities that match their job

Callout boxes are not decoration. Each type signals a specific kind of content, so the visual identity announces what the box contains before the reader engages with it.

Callout	Contains
Definition	Formal definition of a key term, with its significance, distinction from the nearest neighbor concept, and a common pitfall
Example	Historical case or concrete scenario, structured as context, insight, and systems lesson
Systems Perspective	Analytical observation or mathematical nuance that enriches the surrounding argument
Napkin Math	Worked calculation with assumptions and intermediate steps shown explicitly; intermediate tables and equations are the math itself
Checkpoint	Self-check questions to verify understanding of the preceding section
War Story	Documented engineering incident with a sourced failure mode, structured as context, failure, consequence, and lesson
Lighthouse	Recurring workloads characterized by a profile (parameters, FLOPs, dominant constraint), used as anchored case studies across multiple chapters
Principle	Canonical book principle or invariant referenced from later chapters
Theorem	Formal, equation-backed result or bound, stated with its conditions and applied in later quantitative analysis
Key Takeaways	Four to seven most important results from a chapter, in summary form
Learning Objectives	Skills and concepts that readers should be able to explain or apply after completing the chapter
What's Next	Bridge from the chapter just ended to the chapter beginning, naming the open question the next chapter addresses

When a callout contains a table, figure, or equation, that artifact is part of the callout's pedagogical unit, not decoration. A lighthouse's profile *is* its characterization. A napkin math box's intermediate tables *are* the worked math. Reading these as separated artifacts loses the teaching arc. Standalone reference tables, by contrast, sit between paragraphs as numbered artifacts the reader returns to through cross-references. Both forms are valid; the visual marker (callout vs. standalone) tells the reader which one they are looking at.

Companion Resources

Whichever path readers follow, this book is part of a broader learning ecosystem. The complete text, interactive notebooks, lecture slides, exercises, hands-on labs, and supplementary materials are available online at **mlsysbook.ai**.

This book is open source. The full text, figures, and build system are publicly available, and contributions from readers worldwide have materially improved every chapter. Visit **mlsysbook.ai/git** to report issues, suggest improvements, or contribute directly.

Prerequisites

Readers should bring two prerequisites:

- **Programming**: Fluency in Python, including functions, classes, and basic data manipulation with NumPy.
- **Mathematics**: Comfort with linear algebra, basic calculus, and probability at the foundational university level.

Two additional areas are helpful but not required:

- **Computer systems**: Familiarity with memory hierarchies and basic computer architecture is helpful, though the optimization and hardware chapters develop all necessary physical mechanisms from first principles.
- **Machine learning basics**: Prior exposure to supervised learning provides useful context, but Part II establishes the required modeling foundations from first principles.

Beyond This Book

This book develops the single-node foundation while introducing distributed execution, production operations, and governance where they affect the end-to-end system. Scaling beyond a single node introduces additional problems in communication, coordination, and failure recovery. The foundations developed here provide a starting point for studying those problems.

Using This Book in a Course

This book grew out of CS249r at Harvard University and the TinyML edX professional certificate program. The TinyML course taught ML systems at the most constrained end of the spectrum: microcontrollers running on milliwatts with kilobytes of memory. That work demonstrated that the physical constraints governing embedded devices (memory bandwidth, compute density, and energy per operation) are the same constraints that govern data center GPUs. *The physics scales; only the numbers change.* That insight shaped this book. These systems fundamentals provide a foundation for studying larger systems.

A first course can retain the complete chapter sequence while selecting depth within chapters. The following core route emphasizes decisions that every ML systems engineer must justify. Chapter purposes, learning objectives, fallacies, and summaries frame the selected material; worked examples and checkpoints test whether the underlying mechanisms are understood.

1. Establish the systems argument with the iron law in section 1.7, then connect workload requirements to deployment choice through section 2.3 and section 2.9. Follow the lifecycle stages in section 3.6 into problem definition and independent evaluation. In the data chapter, prioritize acquisition, pipelines, processing, labeling, storage, and operational health so that the dataset contract becomes an executable system.
2. Build the learning and execution foundation from neural fundamentals in section 5.4 through learning and inference in section 5.6. Read the architecture families for their operations, state, and dependencies before the comparative analysis. Framework execution, differentiation, tensor abstraction, and the training-step trace in section 7.10 connect those costs to software. Training fundamentals, bottleneck diagnosis, pipeline optimizations, and section 8.8.1 carry that reasoning through a complete training run.
3. Optimize against measured constraints. Start with selection measurement, static pruning, local-worker coordination, and the acceptance decision in section 9.14; compare dynamic or generated-data alternatives when the workload calls for them. Compression requires structural changes, quantization, executable hardware support, and efficiency measurement. In acceleration, retain compute primitives, execution units, memory systems, section 11.6, mapping, and dataflow. Read compiler and runtime treatments for graph mapping, fusion, tensor lifetimes, and scheduling; backend-specific implementation details support selective depth. Benchmark components, training and inference protocols, power boundaries, and section 12.11 establish whether an intervention improves the useful system.
4. Close the lifecycle with request execution, queuing, batching, runtime selection, and capacity economics. Operations foundations, controlled deployment, monitoring, and incident response sustain the measured behavior. The requirements, population testing, explanation, and operational controls in section 15.3 belong alongside environmental costs, security, privacy, and accountability. Finish with the coupled-budget synthesis in section 16.1 and the quantitative principles that connect the parts.

Cross-node training in section 8.8.2 is an optional extension beyond the core node. Neural architecture search in section 10.3.5 and the clinical transfer case in section 14.9.2 offer selective depth, alongside specialist appendix derivations and vendor comparisons. Single-node communication,

memory budgeting, measurement validity, and responsible deployment remain core, even when their mechanisms require more study.

This core route supports a one-semester sequence covering the full ML systems stack, with specialist depth selected for assignments or further study. Instructors may also select individual parts for shorter modules:

- **Parts I–II** form an introductory half-semester on ML systems foundations and how such systems are built and trained.
- **Parts III–IV** form an applied half-semester on optimizing, deploying, and operating them, assuming the foundations covered in Parts I–II.

For survey courses or executive programs, the four part introductions alone provide a condensed overview of the quantitative principles that govern ML systems.

Lecture slides, assignments, and instructor resources are available at **mlsysbook.ai**.

Acknowledgments

Origins

This book owes its existence to a single conversation. During a sabbatical at Google, Pete Warden invited me to work on what seemed like an impossible moonshot for TinyML: running a small speech model on a microcontroller. That collaboration on TensorFlow Lite for Microcontrollers revealed something fundamental. The constraints that govern a device with 16 KB to 256 KB of memory are the same constraints that govern an accelerator in a data center, even when the scale is entirely different. Bandwidth, latency, energy, and memory obey the same physics across both settings. That insight became the seed of this book.

The TinyML work led to an online course on HarvardX, co-taught with Laurence Moroney, that grew to over 100,000 students worldwide. Brian Plancher helped build the TinyML curriculum. Marcelo Rovai, a professor who first encountered the material through the TinyML course, went on to develop the hands-on labs and kits that accompany this book. Zach Shelby opened the Edge Impulse platform for student use. Massimo Banzi and the Arduino team, along with Eric Pan and Seeed Studio, collaborated with us in developing and providing the hardware kits that made hands-on instruction possible. Marco Zennaro brought the material to developing countries through ICTP and planted the idea that these principles deserved a proper textbook. Teaching at that scale, without one, made the need undeniable.

That need turned into a book through CS249r at Harvard University. In 2023, the semester-long class project was to write the class notes: each student group took ownership of a topic, researched it, and worked with me to develop it into a chapter. That work seeded every chapter in this book. The premise was that writing forces you to think more deeply than reading alone. We then opened it all on GitHub, and what began as rough class notes has been rewritten and refined with every semester since.

Because that GitHub work continues, this book benefits from a community whose contributions evolve with the project. New contributors join as readers report issues, improve explanations, and extend the supporting materials. A printed acknowledgment can capture only one moment in that continuing history. The current contributor list and contribution guidelines are available at https://mlsysbook.ai/git.

Collaborators and Colleagues

Beyond the class and GitHub community, my earlier collaborations shaped the book's intellectual foundations. My time at Google informed much of the systems thinking in these pages. That period began with performance modeling for the first Pixel system on chip (SoC), drawing on my background in computer architecture. A collaboration with Mark Hill on roofline-style analytical modeling for heterogeneous SoCs taught me to reason about hardware from first principles, the kind of quantitative thinking that runs through every chapter. That emphasis on quantitative evaluation carried into my work on MLPerf. MLPerf involved many contributors across MLCommons, but I worked most closely with Peter Mattson, David Kanter, Carole-Jean Wu, Christine Cheng, Guenther Schmuelling, Cody Coleman, and Cliff Young. That collaboration shaped how I think about rigorous measurement of ML systems.

Boris Murmann and Song Han convinced me in the early stages that this project was worth pursuing. As I wrote, I sought feedback from colleagues in industry and academia alike. Practitioners at Arm, Google, Intel, Meta, Microsoft, Qualcomm, STMicroelectronics, and other organizations

offered perspectives grounded in production systems, while academic reviewers at universities across Europe, Asia, and the Americas stress-tested the material against their own research and teaching. Their combined feedback sharpened every chapter. Evgeni Gousev at Qualcomm and Pete Bernard at the Edge AI Foundation supported the project's outreach and helped build the community around it.

That feedback loop extended to classrooms around the world, where instructors put the material to the test when it was still rough. Sophia Shao at UC Berkeley, Xiaofan (Fred) Jiang at Columbia, Tushar Krishna and Divya Mahajan at Georgia Tech, and Wenkai Guan at the University of Minnesota Morris were among the first, and instructors at IIT Bombay, NUST Pakistan, De La Salle University in the Philippines, and universities across Europe, Latin America, and Africa have since adopted the book for their own courses. Their willingness to teach from an unfinished textbook provided invaluable feedback and confirmed that the need for this material was not limited to any one institution or any one country.

Students and Contributors

My students in the Edge Computing Lab at Harvard shaped this book in ways I cannot fully trace. Some did foundational work that helped establish TinyML as a field; their research and the ideas that emerged from years of collaboration informed the book's technical content. This cohort helped set the field in motion and then helped me write about it: Andy Cheng, Arya Tschand, Brian Plancher, Colby Banbury, Elizabeth Suitor, Emma Chen, Ikechukwu Uchendu, Jason Jabbour, Jason Yik, Jeffrey Ma, Jessica Quaye, Mark Mazumder, Matthew Stewart, Maximilian Lam, Radhika Ghosal, Shvetank Prakash, Srivatsan Krishnan, Yu-Shun Hsiao, and Zishen Wan. Kai Kleinbard, then at the Harvard Extension School, built the online generative AI SocratiQ tutor bot. Naeem Khoshnevis, from the Harvard Kempner Institute, established essential infrastructure including GitHub Actions and documentation standards. Željko Hrček produced the TikZ diagrams and contributed to the PDF layout, page composition, and final visual formatting of the book.

I am also grateful to the researchers whose work this book builds on and to the open-source community that helped improve it. Dozens of contributors caught errors, improved explanations, filed issues, and submitted pull requests. Some fixed a single typo; others rewrote entire sections. The book improved because people who owed it nothing chose to make it better. What kept me going, aside from stubbornness, was that community: every GitHub star was a signal that someone out there cared, that someone was learning, that the work mattered. I am grateful to every one of them.

Support

This work also depended on support from the Harvard Data Science Initiative, Harvard Extension School, and the National Science Foundation; from the Edge AI Foundation and ICTP for educational outreach and scholarships; and from Arduino, Edge Impulse, Google, and Seeed Studio for hardware, tooling, and infrastructure that enabled the practical labs.

At MIT Press, Susan Hartman believed in this project and guided it from an open-source experiment to a published textbook. Her support, editorial judgment, and friendship made this book real in a way that GitHub alone could not. I am equally grateful to MIT Press for agreeing to keep this book available as open access. A textbook about a discipline shaped by the open-source community should be accessible to everyone in it. That MIT Press shared this conviction made all the difference.

Finally, I thank my wife, Kari, and my daughters, Nora and Maya. This book was written in the hours that belonged to them: late nights that started at ten and ended at two or three in the morning, weekends that disappeared into revisions, years of mornings when I was present but not quite there. The cost was real, and they paid it with patience I did not always deserve.

A Note on AI Assistance

Every chapter of this book was researched, written, and rewritten by me. I also used AI tools throughout. What follows is more than the disclosure MIT Press requires of authors who use AI. It is a statement of the standard I held the book to, and a claim about what authorship means when the tools are this capable.

The standard is traceability. Derived quantities in these chapters are backed by Python computation cells whose source code ships with the book. Citations were checked against their sources, and cross-references are checked against stable anchors. The purpose-built infrastructure modeling engine used for the book's system calculations is available at https://mlsysbook.ai/mlsysim. A pre-commit test suite checks the machine-testable parts of these invariants on every commit, covering unit consistency, inline-reference resolution, cross-reference integrity, notation canonicality, and citation hygiene. AI tools helped me build and maintain this checking infrastructure, which makes the work auditable.

Authorship is thinking. Which concepts belong in this book and which do not. How to sequence the chapters so that each idea arrives only after the reader has the tools to understand it. Which worked examples to trace end-to-end, and which numbers to compute so the reader can verify the argument independently. What level to pitch each explanation at, rigorous enough for a graduate engineer and concrete enough for a student encountering systems thinking for the first time. Where students will get stuck, because I have taught this material and watched them get stuck. These judgments require knowledge of the reader and remain my responsibility. I used AI tools throughout this process to brainstorm framings, explore alternatives, and pressure-test my reasoning. The choices are mine.

In practice, AI was useful for writing code: the computation cells, the pre-commit checks, and the worked examples that are essentially programs. It helped me survey literature I then read in the original, draft passages I then rewrote substantially, and audit the manuscript for consistency across hundreds of cross-references, thousands of index entries, and tens of thousands of lines of prose. I reviewed the tools' suggestions and decided what to use.

AI-generated material can be plausible and still be wrong. Automated checks support review; they do not replace it. The standard I held this book to is the one I want us to demand of every system we ship. Tools can propose; engineers remain responsible for what they accept. MIT Press does not list AI tools as authors because they cannot assume ethical and legal responsibility for their work. The same is true of every engineered system. Authorship is the assumption of that responsibility, and it belongs to the human.

Notation

Machine learning systems span *machine learning* (computer science and statistics) and *systems* (computer architecture and hardware). Each field developed its notation independently, and many symbols mean different things across communities and publications. This collision creates real confusion when the disciplines merge. The notation below distinguishes quantities used in both fields.

Consider a simple statement: *"Increasing B improves throughput."* To an ML researcher, B means batch size. To a hardware engineer, B means bandwidth. Both interpretations are correct in their respective fields, but in machine learning systems we need both concepts in the same equation—hence the need for a single, consistent convention.

The Iron Law of ML Systems

For serialized execution phases, the iron-law performance decomposition is:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}}$$

Each variable was chosen deliberately to avoid collision with standard ML terminology.

Symbol	Definition	Unit	Notes
T	**Execution Time**	seconds	Total serialized execution time in the iron-law performance decomposition.
D_{vol}	**Data Volume**	bytes	**Avoids collision with D (Dataset Size)**. In scaling laws, D means training tokens. Here we need bytes moved through memory. The subscript disambiguates.
BW	**Bandwidth**	bytes/s	**Avoids collision with B (Batch Size)**. Systems literature often uses B for bandwidth, while ML literature often uses B for batch size. The notation preserves the ML convention.
O	**Operations**	FLOPs	Total floating-point operations. Clean in equations (vs. "Ops").
R_{peak}	**Peak Rate**	FLOP/s	**Avoids collision with P (Parameters)**. Roofline presentations often use P for performance, while ML literature commonly uses P for parameter count. The notation preserves the ML convention.
η_{hw}	**Efficiency**	—	Hardware utilization ($0 \leq \eta_{\text{hw}} \leq 1$). Avoids collision with learning rate (η).
L_{lat}	**Latency**	seconds	**Avoids collision with $\mathcal{L}$ (Loss)**. Fixed overhead time (kernel launch, network RTT). The subscript distinguishes from the loss function.

Why these choices matter

Without careful notation, sentences become ambiguous:

> *"Reducing D improves performance."*

The sentence has two possible readings:

- Reducing **dataset size** (fewer training samples) can speed training but may reduce accuracy.

- Reducing **data volume moved** (for example, through compression or quantization) can speed inference, with accuracy effects that depend on the technique and calibration.

With our notation, we can write precisely:

> *"Reducing D_{vol} through FP32-to-INT8 quantization cuts parameter memory traffic to one quarter while D (training data) remains unchanged."*

Our notation eliminates this ambiguity: *"BW limits throughput"* has only one reading.

Subscripted variants

The root names the quantity, and an upright subscript names which instance of it is meant. The convention distinguishes closely related quantities:

- $\mathrm{BW}_{\mathrm{disk}}$, $\mathrm{BW}_{\mathrm{network}}$, $\mathrm{BW}_{\mathrm{accelerator}}$: bandwidth at three points on the data path
- D_{vol}, R_{peak}, L_{lat}, η_{hw}: iron-law terms, each held clear of a common ML symbol
- E_{move}, E_{compute}, E_{total}: one energy budget split into components

The subscript identifies where a measurement applies. Disk and accelerator bandwidth differ by orders of magnitude, so a reported bandwidth is meaningful only if readers know where it was measured.

The Degradation Equation

The degradation equation is an approximation fitted to observed outcomes over a limited range. Divergence alone does not determine whether accuracy rises or falls, so estimate λ from labeled outcomes. Some symbols below, such as τ, appear in chapter prose rather than in the equation.

$$\mathrm{Accuracy}(t) \approx \mathrm{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

Symbol	Definition	Type or unit	Notes
$\mathrm{Accuracy}(t)$	**Accuracy at Time t**	Scalar	Model accuracy after the model has been deployed for time t.
$\mathrm{Accuracy}_0$	**Initial Accuracy**	Scalar	Model accuracy at deployment time.
λ	**Fitted Sensitivity**	Scalar	Local coefficient estimated from outcomes over a stated range; not a universal constant. *(Not wavelength.)*
P_t	**Current Distribution**	Distribution	The data distribution at time t. *(Not parameters—use P for parameter count.)*
P_0	**Training Distribution**	Distribution	The data distribution at training time.
$\mathcal{D}(P_t \| P_0)$	**Statistical Divergence**	Scalar ≥ 0	Measures how far P_t has drifted from P_0. Common choices: KL divergence, total variation, Wasserstein. *(Calligraphic to avoid collision with D = dataset size.)*
τ	**Response Threshold**	Scalar > 0	Operational threshold for investigation; it does not automatically trigger retraining. The same symbol marks a classifier's decision threshold.
$\mathcal{D}$	**Divergence Measure**	Function	Divergence function that measures how far one probability distribution lies from another, with the specific measure left open.
$\mathcal{D}_{\mathrm{KS}}$	**Kolmogorov Smirnov Statistic**	Dimensionless	Two-sample Kolmogorov-Smirnov statistic, the largest absolute difference between two empirical cumulative distribution functions.
F_{P_0}	**Training Empirical CDF**	Function	Empirical cumulative distribution function of a feature in the training-baseline sample.
x_{feat}	**Monitored Feature Level**	Varies	One value of a monitored feature, the argument at which its distribution or empirical cumulative distribution function is evaluated. The maximization variable of the Kolmogorov-Smirnov statistic.
F_{P_t}	**Serving Empirical CDF**	Function	Empirical cumulative distribution function of a feature in the current serving sample.

Symbol	Definition	Type or unit	Notes
c_{crit}	**Kolmogorov Smirnov Critical Threshold**	Dimensionless	Critical threshold that the two-sample Kolmogorov-Smirnov statistic must exceed to reject equal distributions at a given significance level.
n	**Drift Test Sample Size**	Integer	Number of observations in one of the two samples a two-sample drift test compares.
$\mathcal{D}_{\text{KL}}$	**KL Divergence**	Function	Kullback-Leibler divergence, the expected extra code length from encoding one distribution with a code built for another.
H	**Shannon Entropy**	bits or nats	Average uncertainty of a probability distribution, the expected negative log probability of its outcomes. The log base sets the unit: base 2 gives bits, natural log nats.

The Energy Corollary

Effective costs per byte moved and per operation give the following approximation for workload energy.

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

Symbol	Definition	Unit	Notes
E_{total}	**Total Energy**	joules	Total energy consumed by an ML workload, decomposed into data-movement and compute terms.
E_{move}	**Energy per Byte Moved**	joules/byte	Effective movement cost; total energy also depends on bytes moved and operations executed.
E_{compute}	**Energy per Operation**	joules/FLOP	Energy cost of a single arithmetic operation.
C	**Switched Capacitance**	farads	Capacitance of the wires and gates a circuit charges when it switches.
V	**Supply Voltage**	volts	Supply voltage that powers a digital circuit, the quantity dynamic voltage and frequency scaling lowers.
f	**Clock Frequency**	hertz	Rate at which a processor's clock ticks, setting how many switching cycles occur per second.
P_{cluster}	**Cluster Power**	watts	Power drawn by a training cluster, including accelerators and hosts. Average power multiplied by training time gives training energy.
P_{device}	**Device Power**	watts	Electrical power one device draws. Integrated over time it gives the device's energy.
P_{dynamic}	**Dynamic Power**	watts	Power a chip draws switching its transistors, proportional to switched capacitance, supply voltage squared, and clock frequency.
P_{avg}	**Average Power**	watts	Power drawn averaged over a stated period.

Deep Learning Notation

This book follows standard deep learning conventions (Goodfellow et al. 2016) with explicit disambiguation for systems variables.

Symbol	Definition	Type or unit	Notes
B	**Batch Size**	Integer	Number of examples grouped in a training or inference batch. *(Never bandwidth.)*
P	**Parameter Count**	Integer	The total count of scalar model parameters, including weights and biases. *(Never peak FLOP/s.)*
D	**Dataset Size**	Integer	Number of training samples or tokens. *(Never data volume in bytes—use D_{vol}.)*
S	**Sequence Length**	Integer	Number of tokens or time steps in an input sequence.

Symbol	Definition	Type or unit	Notes
d	**Hidden Dimension**	Integer	Size of the hidden state vector of a layer or representation.
d_{in}	**Layer Input Width**	Integer	Number of input features in a layer.
d_{out}	**Layer Output Width**	Integer	Number of output features in a layer.
d_{head}	**Attention Head Dimension**	Integer	Per-head hidden dimension in attention layers.
N_L	**Number of Layers**	Integer	Total number of layers in a network.
N_{heads}	**Number of Attention Heads**	Integer	Number of attention heads in a multi-head attention layer.
H_{KV}	**Number of Key-Value Heads**	Integer	Number of key-value heads in grouped-query or multi-query attention.
ℓ	**Layer Index**	Integer	Index for a layer. Use instead of bare L when indexing layers, since L collides with loss and latency. A numeric subscript marks a norm instead (ℓ_1, ℓ_2).
$\mathcal{L}$	**Loss Function**	Scalar	The objective function minimized during training.
η	**Learning Rate**	Scalar	Step size for the optimizer. *(Never bare for hardware efficiency—use η_{hw}.)*
θ	**Model Parameters**	Vector	All learnable model parameters, treated as a vector.
g	**Gradient**	Vector	Gradient of the loss with respect to the parameters, $\nabla_\theta \mathcal{L}$; a subscript t marks the optimizer step.
$p(x)$	**Distribution**	Probability mass/density	Distribution for a random variable. Use lowercase $p(\cdot)$ for generic distributions to avoid collision with P (parameter count).
$p(y \mid x)$	**Conditional Distribution**	Conditional probability/density	Use this form for generic label relationships; reserve P_0 and P_t for the training/current distributions of drift analysis.
$\hat{\mathbf{y}}$	**Predicted Output**	Vector	A model's output for one example, such as class probabilities; $\mathbf{y}$ is the label vector.
T_{distill}	**Distillation Temperature**	Dimensionless	Softmax temperature in knowledge distillation. It is dimensionless, unlike execution time.
$\Pr(E)$	**Event Probability**	Probability	Probability of an event E. Use for event statements such as $\Pr(\text{batch} = 0)$; use $p(x)$ and $p(y \mid x)$ for distributions.
O_{train}	**Training Compute**	FLOPs	Total floating-point operations required for model training.
y	**Example Label**	Random variable	The label paired with one input, treated as a random variable.
x	**Model Input**	Random variable	Input of one example, the data a model receives to produce a prediction.
Δ	**Finite Change**	Operator	Finite change in the quantity it prefixes, between two states, configurations or points in time. Prefix operator, as in the return-on-compute ratio of accuracy change to compute-cost change.
k	**Batch Index**	Index	Position of one batch in the sequence delivered by a data loader or runtime.
p	**Probability Function**	Function	Probability mass or density function of a random variable, applied to its argument.
H	**Image Height**	Integer	Number of pixel rows in an image or its segmentation mask.
W	**Image Width**	Integer	Number of pixel columns in an image or its segmentation mask.
w	**Model Weight**	Dimensionless	Value of one learnable weight of a model, such as the weight on one input of a neuron.
x	**Layer Input Component**	Dimensionless	One component, or feature, of a layer's input vector.
$\mathbf{x}$	**Input Feature Vector**	Vector	Input feature vector of one example, written as a row vector.
b	**Neuron Bias**	Dimensionless	Bias term added to one neuron's weighted sum. Indexed by neuron: b_j.
z	**Neuron Preactivation**	Dimensionless	Preactivation of one neuron, its weighted input sum plus bias before the activation function. At the output layer, z_j is the logit for one class, the argument of softmax.

Symbol	Definition	Type or unit	Notes
f	**Activation Function**	Function	Activation function of a neuron or layer, applied element-wise to the preactivation. A layer superscript names the layer (f^{(l)}); f′ is its derivative.
$\mathbf{w}$	**Weight Vector**	Vector	Weight vector of one neuron, one weight per input.
$\hat{y}$	**Predicted Output Component**	Dimensionless	One entry of a model's prediction, such as the predicted probability for one class.
$\mathbf{W}$	**Weight Matrix**	Matrix	Matrix of learned weights one layer multiplies its input by, or the kernel tensor of a convolution layer.
x	**Activation Input**	Dimensionless	Scalar argument an activation function is applied to in its defining formula.
σ	**Sigmoid Function**	Function	Logistic sigmoid function, one over one plus e to the minus x. Maps any real input to the open interval (0, 1).
K	**Class Count**	Integer	Number of classes a classifier distinguishes, or vocabulary entries it scores, the length of its softmax output.
$\mathbf{z}$	**Preactivation Vector**	Vector	Preactivation vector of one layer for one example, the logit vector at the output layer.
$\mathbf{b}$	**Bias Vector**	Vector	Vector of learned biases one layer adds to its weighted sums, one per neuron or output channel. Broadcast across the batch.
n	**Layer Width**	Integer	Number of units in the layer at a given position of the network, with n_0 the input width. Indexed by layer position.
$\mathbf{a}$	**Activation Vector**	Vector	Activation vector of one layer for one example.
$\mathbf{A}$	**Activation Matrix**	Matrix	Activation matrix of one layer for a batch, one example per row.
$\mathbf{y}$	**Label Vector**	Vector	True label of one example written as a vector, such as a one-hot class vector.
f	**Model Predictor**	Function	Trained model viewed as a function that maps an input to its prediction.
$\mathbf{Z}$	**Preactivation Matrix**	Matrix	Preactivation matrix of one layer for a batch, one example per row.
$\mathbf{X}$	**Layer Input Matrix**	Matrix	Input to a layer in matrix form, one row per example or token, or a single-channel image.
j	**Class Index**	Index	Index of an output class.
C_{in}	**Input Channel Count**	Integer	Number of channels in the feature map a convolution layer reads.
K	**Kernel Size**	Integer	Side length, in input positions, of a square convolution kernel.
$\mathbf{h}$	**Hidden Activation**	Vector	Activation vector a hidden layer outputs and passes to the next layer.
C_{out}	**Output Channel Count**	Integer	Number of channels, one per filter, in the feature map a convolution layer produces.
f	**Layer Function**	Function	Function computed by a layer, operator, or recurrent cell, mapping inputs to outputs.
$\mathbf{x}$	**Layer Input Vector**	Vector	Vector of activations one layer receives as input.
H_{img}	**Feature Map Height**	Integer	Height, in pixels, of the image or feature map a convolution layer reads.
W_{img}	**Feature Map Width**	Integer	Width, in pixels, of the image or feature map a convolution layer reads.
H_{out}	**Output Feature Map Height**	Integer	Number of output rows a convolution produces.
W_{out}	**Output Feature Map Width**	Integer	Number of output columns a convolution produces.
$\mathbf{Q}$	**Attention Query Matrix**	Matrix	Matrix of query vectors, one row per token, in an attention layer.
$\mathbf{K}$	**Attention Key Matrix**	Matrix	Matrix of key vectors, one row per token, in an attention layer.
$\mathbf{V}$	**Attention Values Matrix**	Matrix	Matrix of value vectors, one row per token, that attention weights combine into the output.
$\mathbf{A}$	**Attention Weight Matrix**	Matrix	Matrix of attention weights, the row-wise softmax of scaled query-key scores.

Symbol	Definition	Type or unit	Notes
$\mathbf{I}$	**Identity Matrix**	Matrix	Square matrix with ones on the diagonal and zeros elsewhere. Size follows from context.
$\mathcal{B}$	**Mini Batch Set**	Set	Set of examples processed together as one mini-batch.
γ	**Normalization Scale**	Dimensionless	Learned factor a normalization layer multiplies each normalized activation by.
$\boldsymbol{\gamma}$	**Normalization Scale Vector**	Vector	Vector of learned per-channel or per-feature factors a normalization layer multiplies its normalized output by.
$\boldsymbol{\beta}$	**Normalization Shift Vector**	Vector	Vector of learned per-channel or per-feature offsets a normalization layer adds to its scaled output.
v	**Trace Intermediate Variable**	Dimensionless	Intermediate variable in the evaluation trace of a differentiated program, its primal value.
$\bar{v}$	**Trace Intermediate Adjoint**	Dimensionless	Reverse-mode adjoint of an intermediate variable, the derivative of the output with respect to it.
$\mathbf{x}$	**Function Input Vector**	Vector	Vector a layer or differentiated function takes as input.
N	**Function Input Dimension**	Integer	Number of inputs of a differentiated function, the column count of its Jacobian.
x	**Function Input Element**	Dimensionless	One element of the input vector of a layer or elementwise function.
i	**Matrix Row Index**	Index	Position of a row within a matrix.
j	**Matrix Column Index**	Index	Column index of a matrix entry.
$\mathbf{S}$	**Attention Score Matrix**	Matrix	Matrix of attention scores, the scaled dot products of every query with every key before the softmax.
$\mathbf{x}$	**Softmax Input Vector**	Vector	Vector of scores an online softmax normalizes block by block.
m	**Softmax Running Maximum**	Dimensionless	Largest score seen so far in a block of the online softmax, subtracted before exponentiation. Superscripts (1), (2), new mark the block it covers.
z	**Softmax Normalizer**	Dimensionless	Sum of the max-shifted exponentials of a softmax row or block, the softmax denominator, kept as a running sum by online softmax. Not the class logit z of the data appendix.
$\mathbf{O}$	**Attention Head Output**	Matrix	Output of one attention head before the output projection, the attention weights applied to the value matrix. Distinct from the output-projection result of a whole attention layer.
β	**Data Scaling Exponent**	Dimensionless	Fitted power-law exponent at which loss falls as the training dataset or token count grows.
d	**Derivative Operator**	Operator	Differential operator d used in an ordinary derivative.
t	**Optimizer Step Index**	Index	Position of one optimizer update in the sequence of training steps.
N_{epochs}	**Epoch Count**	Integer	Number of full passes a training run makes over its dataset. Training work scales with it times dataset size.
z	**Output Logit**	Dimensionless	Raw score a model outputs for one class before the softmax.
r	**Input Resolution**	Integer	Input image resolution of a convolutional model. Subscript 0 marks the base model in compound scaling.
s	**Quantization Scale**	Varies	Real-valued step between adjacent integer levels of a uniform quantizer.
x_q	**Quantized Integer Code**	Integer	Integer code a quantizer assigns to one real value.
W	**Layer Weight Entry**	Dimensionless	One scalar entry of a layer's weight matrix, picked out by a layer superscript and row and column subscripts.
W_{ij}	**Weight Matrix Entry**	Dimensionless	One entry of a weight matrix at a given row and column; italic because it is a scalar.
$\mathbf{W}_{\text{fused}}$	**Fused Convolution Weight**	Dimensionless	Convolution weights after the following batch normalization is folded into them, so inference runs one layer instead of two.
x	**Quantizer Input**	Varies	Full-precision real value, a weight or activation, that a quantizer maps to an integer code. Unit: units of the quantized tensor.
α	**Clipping Range Lower Bound**	Varies	Lower end of the real-valued range a quantizer represents, below which values are clipped. Calibration estimates it.

Symbol	Definition	Type or unit	Notes
β	**Clipping Range Upper Bound**	Varies	Upper end of the real-valued range a quantizer represents, above which values are clipped.
b	**Quantization Bit Width**	Integer	Number of bits in each integer code a quantizer produces.
$\sigma_{\mathcal{B}}$	**Batch Norm Running Standard Deviation**	Vector	Per-channel square root of the running variance plus the numerical stabilizer that batch normalization uses at inference.
n_{keep}	**Structured Sparsity Nonzero Count**	Integer	Number of values kept nonzero in each group of an N:M structured-sparsity pattern.
m_{group}	**Structured Sparsity Group Size**	Integer	Number of consecutive values in one group of an N:M structured-sparsity pattern.
K_h	**Kernel Height**	Integer	Height of a convolution kernel, in input positions.
K_w	**Kernel Width**	Integer	Width of a convolution kernel, in input positions.
k	**Top Class Count**	Integer	Number of highest-scoring classes a top-k prediction returns or checks for the correct label.
μ	**Service Rate**	items/s	Rate at which one server, such as a worker, consumer, or model endpoint, completes items or calls.
$\hat{y}$	**Model Prediction**	Dimensionless	Prediction a model outputs for one example, compared with its label by the loss.
$\mathbf{Y}$	**Layer Output Matrix**	Matrix	Output matrix of a layer's matrix multiplication of the input matrix by the weight matrix, one row per example. Shape batch size by layer output width; before any bias or activation function.
m	**Softmax Row Maximum**	Dimensionless	Largest score in a softmax row or block, subtracted before exponentiation to prevent overflow.
x	**Softmax Input Score**	Dimensionless	One entry of the score vector the online softmax normalizes. Written with an index and a block superscript, $x_j^{(1)}$.
α_{sym}	**Symmetric Clipping Threshold**	Varies	Largest magnitude a symmetric quantizer represents, so values are clipped to plus or minus this bound.

Latin matrices and vectors are set in bold (**W**, **x**); scalars, dimensions, indices, and individual matrix entries stay italic (N, d, i, W_{ij}). The generic parameter symbol θ is the one conventional exception and stays italic.

Local matrix algebra follows the same bold rule. Matrix operands are bold (**A**, **B**, **C**), as in general matrix multiply (GEMM) $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$. The bold form marks a matrix operand and is distinct from the italic scalar batch size B, which remains batch size in scalar contexts.

A GEMM's scalar dimensions are M, N, and K (**A** is $M \times K$, **B** is $K \times N$). Asymptotic cost is written $\mathcal{O}(\cdot)$, which keeps it distinct from the operation count O.

Performance, Serving, and Memory Notation

Reusable performance and serving quantities follow the same collision-avoidance rule as the iron law. Rates use R or descriptive Greek symbols; request counts use Q_{req} rather than overloading N; queue utilization uses a subscripted ρ so bare ρ remains available for other ratio models.

Symbol	Definition	Type or unit	Notes
I	**Arithmetic Intensity**	FLOP/byte	Workload FLOPs per byte moved. The Roofline Model uses I as the independent variable.
I_{ridge}	**Roofline Ridge Point**	FLOP/byte	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$. Prefer this explicit form over starred shorthand so the meaning remains clear in prose.
R_{attain}	**Attainable Compute Rate**	FLOP/s	Roofline bound: $R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$. Uses R, not T, because this quantity is a rate.
MFU	**Model FLOPs Utilization**	Dimensionless	Useful model FLOP/s divided by available peak FLOP/s. Text acronym avoids overloading η.
Q_{req}	**Request Concurrency**	requests	Average in-flight requests in a stable serving system ($Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}}$ via Little's Law). Avoids overloading N, which counts devices, elements, or samples elsewhere.

Symbol	Definition	Type or unit	Notes
λ_{arr}	**Arrival Rate**	requests/s	Request arrival rate. The subscript keeps bare λ free for fitted sensitivities.
T_{memory}	**Iron-Law Memory Time**	seconds	Data-movement term D_{vol}/BW in the iron law.
T_{compute}	**Iron-Law Compute Time**	seconds	Compute term $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ in the iron law.
N_{GPU}	**Accelerator Count**	Integer	Number of accelerators in one job.
T_{lat}	**Request Time in System**	seconds	Average time a request spends queueing and being served, as used in Little's law. Distinct from L_{lat}, the fixed-latency term in the iron law.
$L_{\text{lat,total}}$	**End-to-End Request Latency**	seconds	Request-path latency budget, $L_{\text{lat,network}} + L_{\text{lat,pre}} + L_{\text{lat,infer}} + L_{\text{lat,post}} + L_{\text{lat,queue}}$. T_{lat} covers only the time inside the serving system.
$T_{\text{svc}}(B)$	**Batch Service Time**	seconds	Time to serve a batch of size B.
$\mu_{\text{eff}}(B)$	**Effective Service Rate**	requests/s	Batched service rate, typically $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$. The unbatched service rate is $\mu = \mu_{\text{eff}}(1)$.
ρ_{serv}	**Serving Utilization**	Dimensionless	Queue/server utilization. The subscript keeps bare ρ free for other ratio models.
M_{total}	**Total Memory Footprint**	bytes	Sum of explicit memory components; avoids bare M ambiguity.
M_{weights}	**Weight Memory**	bytes	Memory occupied by model weights ($P \times$ bytes-per-parameter).
$M_{\text{gradients}}$	**Gradient Memory**	bytes	Memory occupied by stored gradients.
$M_{\text{optimizer}}$	**Optimizer-State Memory**	bytes	Momentum, variance, master weights, and related optimizer buffers.
$M_{\text{activations}}$	**Activation Memory**	bytes	Retained activations for the backward pass or serving intermediates.
M_{KV}	**KV-Cache Memory**	bytes	Memory occupied by the key-value cache during autoregressive inference.
s_{elem}	**Element Storage Size**	bytes/element	Bytes per stored tensor element.
s_{sample}	**Sample Storage Size**	bytes/sample	Bytes per stored training or serving sample.
s_{loss}	**Loss-Scale Factor**	Dimensionless	Multiplier applied to the loss in reduced-precision training so small gradients stay representable.
$T_{\text{pipelined}}$	**Pipelined Execution Time**	seconds	Ideal steady-state execution time when data movement and computation overlap, approximated by the slower stage plus fixed latency.
G	**Accelerated Fraction Speedup**	Dimensionless	Speedup factor applied to the accelerated fraction of a workload in Amdahl's law, whether it comes from faster hardware or from more workers.
T_{train}	**Training Time**	seconds	Wall-clock time to complete one training run.
T_{IO}	**I/O Transfer Time**	seconds	Time to move a workload's data over a storage or network link, the input/output stage of a pipelined step. Overlaps with compute in the pipelined iron law; the step is I/O bound when it is the largest stage.
$\text{BW}_{\text{network}}$	**Network Bandwidth**	bytes/s	Sustained rate at which a network path delivers bytes between machines.
$L_{\text{lat,network}}$	**Network Latency**	seconds	Time a request and its response spend crossing the network, including propagation, routing, and protocol handling.
M_{device}	**Device Memory Capacity**	bytes	Total memory a device provides for a model's weights and activations.
G_{accel}	**Accelerator Gain**	Dimensionless	Speedup an optimization or accelerator gives the fraction of execution it applies to, in Amdahl's law.
a	**First Floating Point Addend**	Dimensionless	First of three floating-point numbers added in either grouping to show that floating-point addition is not associative.
b	**Second Floating Point Addend**	Dimensionless	Second of three floating-point numbers added in either grouping to show that floating-point addition is not associative.
c	**Third Floating Point Addend**	Dimensionless	Third of three floating-point numbers added in either grouping to show that floating-point addition is not associative.
$\text{BW}_{\text{storage}}$	**Storage Bandwidth**	bytes/s	Sustained rate at which the storage system delivers bytes to the input pipeline.
T_{step}	**Training Step Time**	seconds	Wall-clock time of one training step, bounded below by the slower of compute and input I/O.
N	**Square GEMM Dimension**	Integer	Side length of the square matrices in a matrix multiply, the shared row, column, and inner dimension.

Symbol	Definition	Type or unit	Notes
M	**GEMM Row Dimension**	Integer	Number of rows of the left operand and of the output in a matrix multiply.
K	**GEMM Inner Dimension**	Integer	Shared inner dimension of a general matrix multiply, the number of products summed per output element.
N	**GEMM Column Dimension**	Integer	Number of columns of the right operand and of the output in a matrix multiply.
µ	**Micro Unit Prefix**	Operator	SI unit prefix micro, one millionth, written before a unit such as seconds.
N	**Tensor Element Count**	Integer	Number of elements in one tensor.
n_{ops}	**Chained Operation Count**	Integer	Number of consecutive operations that read and write the same intermediate tensor and can be combined through kernel fusion.
T_{transfer}	**Batch Transfer Time**	seconds	Time to copy one batch of input data from host memory to the device that consumes it.
C	**GEMM Output Matrix**	Matrix	Output matrix of a general matrix multiply, of size M by N, which the operation updates in place.
α	**GEMM Product Scale**	Dimensionless	Scalar multiplier applied to the matrix product in a general matrix multiply.
β	**GEMM Accumulator Scale**	Dimensionless	Scalar multiplier applied to the existing output matrix before the product is added in a general matrix multiply.
A	**GEMM Left Operand**	Matrix	Left input matrix of a general matrix multiply, of size M by K.
B	**GEMM Right Operand**	Matrix	Right input matrix of a general matrix multiply, of size K by N.
$\mathbf{BW}_{\text{disk}}$	**Disk Bandwidth**	bytes/s	Sustained rate at which a storage disk delivers bytes.
$R_{\text{sustained}}$	**Sustained Compute Rate**	FLOP/s	Operation rate an accelerator or system actually sustains on a workload.
T_{prep}	**Host Preparation Time**	seconds	Time the host takes to prepare one batch for the accelerator, from read through batching.
$\mathbf{BW}_{\text{accelerator}}$	**Accelerator Memory Bandwidth**	bytes/s	Sustained rate at which bytes move between an accelerator's own memory, such as HBM, and its compute units.
t	**Time Instant**	seconds	A point in continuous wall-clock time, the variable over which time-varying quantities are integrated.
$L_{\text{lat,infer}}$	**Inference Latency**	seconds	Time the model takes to compute a request's output.
M_{tensor}	**Intermediate Tensor Size**	bytes	Bytes in the intermediate tensor that a sequence of operations passes between its steps.
n_{nz}	**Nonzero Count**	Integer	Number of nonzero values a sparse matrix format stores.
n_{rows}	**Sparse Matrix Row Count**	Integer	Number of rows in a sparse matrix. Compressed sparse row (CSR) storage uses one more row pointer than the number of rows.
T_{token}	**Decode Token Time Floor**	seconds	Lower bound on the time to generate one token at batch one, bytes streamed per step divided by memory bandwidth.
$L_{\text{lat,queue}}$	**Queueing Latency**	seconds	Time a request waits in queues before it is served.
$L_{\text{lat,pre}}$	**Preprocessing Latency**	seconds	Time spent preparing a request's input before inference.
$L_{\text{lat,post}}$	**Postprocessing Latency**	seconds	Time spent turning a model's output into the response.
b	**Numeric Bit Width**	Integer	Number of bits used to represent one numeric value, such as an arithmetic operand or a quantized weight or activation.
T	**Dataset Transfer Time**	seconds	Ideal line-rate time to move a dataset over a network or storage path: data volume divided by bandwidth.
T_{GPU}	**Accelerator Batch Time**	seconds	Time the accelerator takes to run one training step on one batch, excluding input preparation.

Statistical and Operational Notation

Statistical thresholds, workload fractions, and system constraints follow standard statistical and operational conventions:

Symbol	Definition	Type or unit	Notes
ϵ_{adv}	**Adversarial Perturbation Budget**	Scalar	Maximum norm bound on allowable input perturbations ϵ_{adv}.
κ	**Cohen's Kappa**	Dimensionless	Inter-annotator agreement coefficient $\kappa \in [-1, 1]$.

Symbol	Definition	Type or unit	Notes
T_{deadline}	**Deadline**	seconds	Latency deadline for query completion or batch execution.
f_{overhead}	**Operational Overhead Fraction**	Dimensionless	Fraction of wall time spent outside productive step execution (f_{overhead}).
f_{parallel}	**Parallel Fraction**	Dimensionless	Fraction of workload execution that can be parallelized in Amdahl's Law ($0 \leq f_{\text{parallel}} \leq 1$).
α	**Significance Level**	Dimensionless	Statistical significance threshold α for hypothesis tests.
ϵ	**Numerical Stability Constant**	Scalar	Small numerical constant $\epsilon > 0$ for numerical stability.
N_{workers}	**Worker Count**	Integer	Number of parallel worker threads, processes, or devices.
t_{obs}	**Observation Timestamp**	seconds	Time at which a labeled observation, such as a churn event, occurred. Upper bound on the feature timestamps an as-of join may use.
C_{error}	**Erroneous Prediction Cost**	dollars	Average business cost incurred by one erroneous prediction. Multiplies the skew rate and query volume in the training-serving skew cost.
F_{access}	**Storage Access Factor**	Dimensionless	Fraction of peak storage throughput a workload's access pattern achieves.
g	**Subgroup Index**	Index	Label of one population subgroup whose outcomes are compared against a reference group.
k	**Feedback Interval Count**	Integer	Number of fixed feedback intervals, such as retraining cycles, elapsed since the disparity was first measured.
α_{fb}	**Bias Feedback Factor**	Dimensionless	Factor by which a group's disparity is multiplied over one feedback interval, fitted from observed disparities. Above 1 the feedback loop is self-reinforcing; at or below 1 it is stable or damped.
Δ_g	**Group Disparity**	Dimensionless	Size of the performance gap between one group and a reference group; the feedback analysis tracks how it grows from one feedback interval to the next.
k_{anon}	**Anonymity Set Size**	Integer	Smallest number of records that share every quasi-identifier value in a k-anonymous release.
p_{deploy}	**Deployment Distribution**	Distribution	Joint distribution of inputs and labels a deployed model must serve, the population a verification guarantee is stated over.
d_{task}	**Task Distance**	Function	Task-specific distance between a model's prediction and the correct label, the error measure a verification guarantee bounds.
τ_{tol}	**Task Error Tolerance**	Dimensionless	Largest task distance between a prediction and its label that still counts as an acceptable output.
ϵ	**Tolerated Failure Probability**	Dimensionless	Maximum allowed probability that a deployed model exceeds its task error tolerance. The confidence level used to estimate this probability is specified separately.
S_{skew}	**Training Serving Skew**	Dimensionless	Expected absolute difference between the serving-path and training-path model scores on the same inputs, over a stated comparison-input distribution. Measurable without labels; a risk signal, not an accuracy-loss equation.
f_{serve}	**Serving Path Predictor**	Function	Serving code path viewed as a function from a raw input to the model's output, including its preprocessing and runtime.
f_{train}	**Training Path Predictor**	Function	Training code path viewed as a function from a raw input to the model's output, as offline validation evaluates it.
q	**Target Completion Quantile**	Dimensionless	Required probability that a request completes within its latency deadline, the completion quantile a service level states.

Units and Precision

- **Physical units**: This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, the book converts to SI and notes the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).

- **Data and memory**: This book uses **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6, GB = 10^9, TB = 10^{12}. Binary-prefixed units do not appear in prose; model sizes, activation footprints, throughputs, and transfer volumes are reported in decimal units (for example, 2 TB/s, 102 MB). Hardware memory capacities retain the vendor's nameplate figure and label. Calculations requiring an exact byte count use the device's documented capacity.
- **Compute**: The notation distinguishes operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
  - 1 TFLOP = 10^{12} FLOPs
  - 1 TFLOP/s = 10^{12} FLOPs per second
  - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.

- **Power**: Power carries a subscript (P_{avg}, P_{device}, P_{cluster}); bare P remains parameter count.
- **Latency percentiles**: Written p50, p95, and p99 in upright type, inside math as well as in prose.
- Currency: Dollar amounts use the dollar sign (`$`); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- **Precision**:
  - **FP64**: Double precision (8 bytes)
  - **FP32**: Single precision (4 bytes)
  - **TF32**: TensorFloat-32 (19-bit Tensor Core compute mode; inputs and storage remain FP32)
  - **FP16**: Half precision (2 bytes, standard range)
  - **BF16**: Brain float (2 bytes, wide dynamic range)
  - **FP8**: Quarter precision (1 byte, E4M3 or E5M2 format)
  - **FP4**: 4-bit floating-point format
  - **INT8**: 8-bit integer (1 byte)
  - **INT4**: 4-bit integer; lower integer precisions follow the same uppercase `INTn` pattern (for example, INT3, INT2)

Quick Reference: Resolving Collisions

Common notation collisions in machine learning systems literature include:

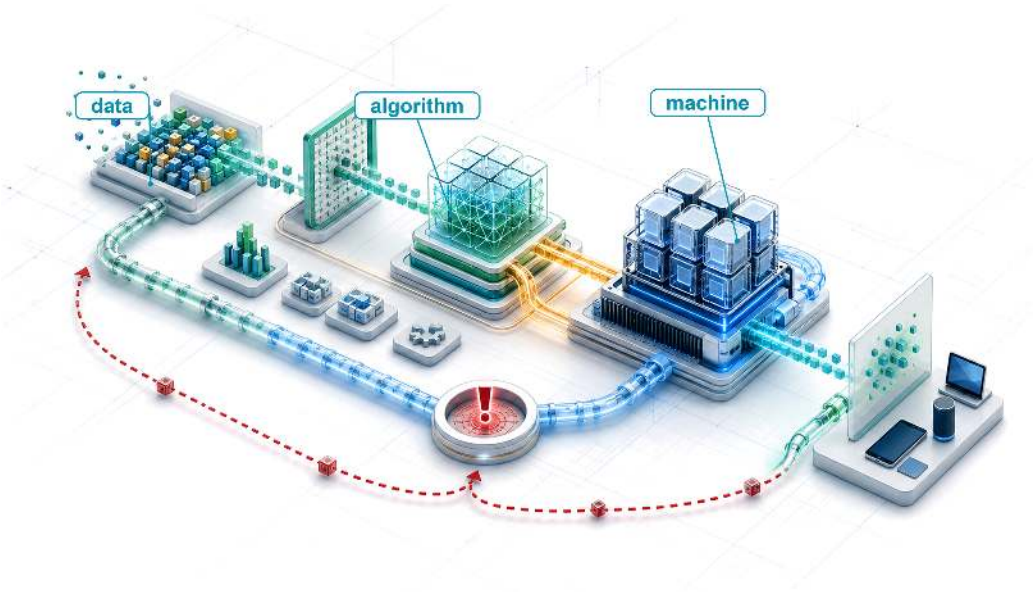
Symbol	ML Meaning	Systems Meaning	Book Convention
B	Batch Size	Bandwidth	**Batch Size**. Use BW for bandwidth.
P	Parameter Count	Peak FLOP/s	**Parameter Count**. Use R_{peak} for peak rate.
D	Dataset Size	Data Volume	**Dataset Size**. Use D_{vol} for bytes moved.
L	Loss	Latency	**Loss** ($\mathcal{L}$). Use L_{lat} for latency.
η	Learning Rate	Efficiency	**Learning Rate**. Use η_{hw} for efficiency.

ML conventions generally take precedence for single letters, while systems quantities use subscripts or descriptive symbols. This preserves familiar ML notation while distinguishing systems quantities.

MAIN TEXT

1

Introduction



1.1 The AI Systems Moment
1.2 Data-Centric Paradigm Shift
1.3 The Evolution of AI Bottlenecks
1.4 The Bitter Lesson
1.5 Defining ML Systems
1.6 ML vs. Traditional Software
1.7 Iron Law of ML Systems
1.8 Three Dimensions of ML Efficiency
1.9 AI Engineering as a Discipline
1.10 Book Organization
1.11 Fallacies and Pitfalls
1.12 Summary

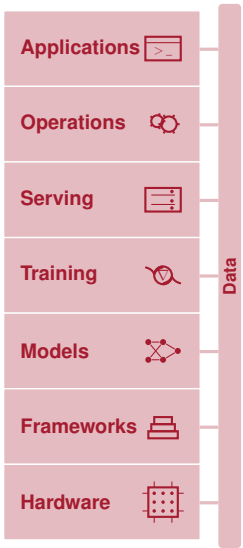
Purpose

Why does an accurate machine learning model not guarantee a reliable, efficient system?

Machine learning systems have a physics. Data moves through memory hierarchies governed by bandwidth, arithmetic executes on silicon governed by power, and predictions must arrive within strict latency windows. These constraints are not secondary implementation details. Moving an operand from main memory can cost far more energy than performing arithmetic on it, and a prediction that arrives a few hundred milliseconds late can be worth nothing to the real-time process waiting for it. An architecture that fits comfortably in a data center is useless on a mobile phone, and a training pipeline that converges in days on high-bandwidth accelerators can stall for weeks on standard hardware.

Machine learning systems also diverge from traditional computing systems because empirical data, rather than explicit logic alone, defines operational behavior. When a conventional program misbehaves, it crashes or throws an exception. When an ML system misbehaves, the execution runtime may operate flawlessly while learned predictions fail because the input distribution has shifted or the training data was unrepresentative. An accurate model trained on historical data can silently degrade in production without failing a single assertion or unit test. No exception is raised; the only symptom is deteriorated decisions, discovered weeks later through lost revenue, erroneous predictions, or compromised safety.

Traditional practices such as testing, version control, and performance profiling remain necessary, but they are no longer sufficient. A unit test confirms that code computes what it was written to compute, but cannot establish that learned behavior still matches a non-stationary world. Furthermore, optimizing a single component in isolation frequently moves the critical bottleneck to another layer of the stack. Correctness, hardware efficiency, and deployment constraints cannot be resolved



independently, making holistic diagnosis the engineer's primary task. This book builds a discipline grounded in computation's physical limits, in which algorithmic choices reach down to machine execution and hardware constraints flow back up to model design and data pipelines.

Learning Objectives

- Explain why predictive quality can decline while model code and execution remain unchanged
- Apply a data-algorithm-machine lens to diagnose a system bottleneck and reassess it after an intervention
- Calculate execution time with the iron law and assess how a component improvement changes end-to-end time
- Analyze when scalable search and learning can benefit from additional computation within data and machine constraints
- Evaluate whether predictive quality and resource use meet deployment requirements using evaluation and lifecycle monitoring

1.1 The AI Systems Moment

Artificial intelligence is no longer confined to research demonstrations. When a user queries a smartphone voice assistant, learned components convert speech to text, interpret intent, retrieve information, and generate a response within seconds. Search engines rank results, recommendation systems decide what people see, lenders use models to assess risk, and driver-assistance systems detect hazards. In each case, a prediction participates in a larger decision that can affect attention, money, access, or safety. What appears to be one intelligent action is therefore an end-to-end system operating in the world.

Modern AI systems became practical when three forces converged: digital services generated large datasets, learning algorithms extracted predictive structure from those examples, and parallel hardware made the resulting computation feasible. Yet the most consequential change was conceptual. Data stopped being merely an input to software; it became a mechanism defining program behavior. Instead of hand-coding every decision rule, engineers build a training process through which examples shape the behavior a system learns.

This convergence creates a **dual mandate**. Every ML system must establish that its learned behavior is trustworthy and that the machine can produce that behavior within the available time, memory, energy, and cost. A model that is accurate but too slow is unusable; a fast model trained on unrepresentative data is wrong at machine speed. The distinction becomes clearest at failure boundaries. A code defect may crash loudly, while a data defect can leave every instruction executing correctly as predictions deteriorate silently. At scale, both obligations span the entire stack. Conversational services coordinate pools of GPUs[1] while managing memory, networks, and heat. Driver-assistance systems must fuse sensor streams within milliseconds. A search service processing 8.5B queries per day must also meet strict latency targets. These systems succeed only when learned behavior and physical execution are engineered together. That co-design begins with the shift from code-defined logic to data-defined behavior.

1.2 Data-Centric Paradigm Shift

The transition from code-defined logic to data-defined behavior transforms how software systems are conceived, constructed, and maintained. Instead of writing decision rules directly into source code, engineers construct an optimization pipeline that learns program behavior from empirical examples. Andrej Karpathy[2] framed this transformation as the migration from **Software 1.0** to **Software 2.0** (Karpathy 2017). As summarized in table 1.1, Software 1.0 explicitly encodes procedural logic in human-written instructions, whereas Software 2.0 searches a high-dimensional parameter space to find a model configuration that satisfies an empirical objective.

An email spam filter illustrates this learning process. Each training record pairs an email payload with a binary label indicating whether the message is spam. The optimization procedure adjusts millions of numerical weights so that the model's predictions align with historical labels. Live inference then applies these trained weights to incoming messages whose labels are unknown. Evaluating the model against a separate test set reveals whether this learned decision boundary generalizes reliably beyond the training corpus.

[1] **GPU (graphics processing unit)**: Originally designed to render graphics by performing many calculations in parallel. Neural networks also require large numbers of similar arithmetic operations, especially matrix multiplications, making this hardware useful for training. Keeping those operations supplied with data remains a separate engineering challenge.

[2] **Andrej Karpathy**: A founding member of OpenAI and former director of AI at Tesla. His "Software 2.0" essay described neural network weights as a new form of "source code" (Karpathy 2017). The analogy shifts debugging attention toward the examples that shaped those weights and the conditions under which the model is evaluated.

Software 2.0 does not eliminate traditional software engineering; it relocates the center of gravity. Engineers still write data ingestion pipelines, loss functions, distributed training loops, and low-latency serving harnesses. Crucially, however, the core operational logic no longer resides in human-readable control flow, but in millions of numerical weights shaped by data. Static analysis and manual code review cannot reveal what decision the system will make for a novel input. While both paradigms can suffer from undetected faults, learned systems introduce a distinct failure mode: predictive accuracy can silently collapse even when source code, compiler binaries, and execution environments remain completely unchanged.

Table 1.1: The Paradigm Shift from Software 1.0 to Software 2.0: In Software 2.0, the "programmer" curates the dataset and designs the training process that shapes learned behavior. Debugging expands to include the data. The "compiler" analogy is approximate: unlike a deterministic compiler, the training process is stochastic and may produce different learned weights from the same training data.

Feature	Software 1.0 (Traditional)	Software 2.0 (Machine Learning)
Source Code	C++, Python, Java	Learned weights, shaped by training data
Compiler	GCC, LLVM	Training loop (stochastic gradient descent)
Logic	Explicit (Hand-coded)	Implicit (Learned)
Failure Mode	Crashes or silent logic errors	Also statistical quality loss
Debugging	Trace execution and inputs	Also inspect data and learned behavior

The data-centered workflow changes what engineers must build and maintain. Sculley and colleagues described how model code occupies only a small fraction of a production ML system (Sculley et al. 2015). The surrounding system includes data collection, verification, feature extraction, resource management, serving, and monitoring, each with its own maintenance obligations.



Production ML work is mostly the surrounding system, not the model code alone.

That imbalance creates hidden technical debt, the accumulating cost of maintaining undocumented dependencies and brittle infrastructure around the model. Each surrounding component encodes assumptions about how an example is sampled, what a label means, when a feature is computed, which version reaches serving, and how degradation is detected. None of those assumptions appears in the matrix multiplication that produces a prediction, yet any of them can change the result. Improving the model alone cannot repair a stale proxy, a broken feature pipeline, or a missing feedback signal.

Production data is therefore not a passive input. It is a measurement of the world, collected through a particular product and transformed by a changing pipeline. If that measurement stops representing the intended target, the model can remain numerically healthy while the system becomes wrong. Google Flu Trends provides a revealing example. It had an enormous, timely stream of search data, but the meaning of that data changed beneath the model.

War Story 1.1: When search logs mistook attention for illness (2014)

Context: Google Flu Trends (GFT) estimated influenza activity from patterns in aggregated search queries (Ginsberg et al. 2009; Lazer et al. 2014). Its attraction came from the speed and scale at which searches arrived relative to clinical reports. If people searched for flu symptoms when they became ill, query volume appeared to offer a timely proxy, or indirect measurement, for infection.

Failure mode: The proxy was not stable. News coverage could change what people searched for, and autocomplete could change how they expressed those searches. Query volume could therefore reflect public attention and product behavior as well as illness. During the 2012–2013 season, GFT estimated roughly twice the CDC-reported proportion of doctor visits for influenza-like illness and overestimated for 100 out of 108 weeks.

Systems lesson: The remedy was not simply a larger search dataset or a better-tuned model. Researchers combined search signals with CDC sentinel clinical data and recalibrated the relationship over time. Data volume is not ground truth: a behavioral proxy needs a feedback

loop to a trusted measurement and continual evidence that the proxy still represents the quantity the system claims to estimate.

Google Flu Trends illustrates how predictive quality can deteriorate while the software continues operating. The input distribution, meaning the kinds of inputs and how frequently they occur, and the relationship between proxy and target can change. Under the **data-as-code principle**, training data does not merely enter a fixed program; it helps determine the operational logic that the model implements. Engineers still write the optimization procedure, but the examples shape the model weights[3] through stochastic gradient descent[4] and related methods. Changing the dataset can therefore change system behavior as surely as changing source code.

Data-defined behavior is a property of machine learning; *data-centric* AI is a development strategy for improving that behavior (Ng 2021). In a model-centric approach, teams hold the data comparatively fixed and focus on improving the model. In a data-centric approach, they hold the model comparatively fixed and systematically improve the data. Both can be useful, but the latter makes data curation an explicit route to better predictions.

Data-defined behavior adds a testing obligation: assessing predictive quality over the population of inputs the system will encounter. Conventional software can also have input spaces too large to enumerate, so exhaustive testing is not a realistic expectation for either kind of system. The 2012 ImageNet challenge makes the scale concrete. AlexNet's decisive win[5] helped set the deep learning era in motion, yet the benchmark could evaluate only a vanishing fraction of the inputs the model might encounter. Each input is a 224×224 RGB image with $256^{150,528}$ possible pixel configurations, a number with 362,508 digits. Real photographs occupy only a tiny region of that space, but even that region is far too large to enumerate. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) validation set contains only 50,000 images (Krizhevsky et al. 2012; Russakovsky et al. 2015). The ratio of evaluated inputs to possible configurations (Test Set Coverage/Total Input Space) is effectively zero. Expressed as a count of untested inputs, their disparity illustrates the **verification gap** (equation 1.1):

$$\text{Verification Gap} = \text{Total Input Space} - \text{Test Set Coverage} \approx \text{Total Input Space} \tag{1.1}$$

This vast disparity does not render evaluation futile; rather, it redefines the epistemological contract of testing. Predeployment evaluation provides statistical evidence over sampled inputs, while continuous production monitoring verifies whether the operating population and observed outcomes continue to support that empirical baseline. Statistical confidence replaces any expectation of exhaustive proof. Part IV, on deployment and operations, formalizes the verification gap as a rigorous probability bound over a specified operating population (principle 9).

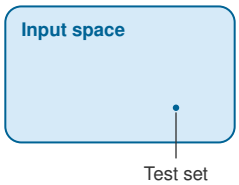
This shift alters what an engineer can formally assert about system behavior. Traditional unit assertions verify that a deterministic set of inputs and state variables leads down an explicit execution path. Machine learning claims, in contrast, describe aggregate performance over a non-stationary population. A frozen model binary will return the exact same output for an identical input tensor, yet measured task accuracy degrades as the distribution of inputs drifts over time. Because empirical data supplies both the signal from which the model learns and the uncertainty under which it operates, robustness cannot mean immunity to distribution shifts. It demands observability: instrumenting the pipeline to detect distribution drift and adapting when production evidence no longer supports the model's inductive bias—a paradigm of **probabilistic engineering**.

This reality reshapes everyday software engineering workflows. Debugging an ML system requires inspecting datasets and pipeline transformations, not tracing Python call stacks alone. Version control must record dataset snapshots and feature configurations alongside source code commits. Evaluation harnesses must measure statistical distribution shifts and subpopulation outcomes rather than branch coverage. These practices establish reproducibility and provenance, but they cannot transform empirical sampling into formal verification.

3 | **Model weights**: The learned numerical parameters of a neural network. They must be stored and accessed during execution, so their number and numerical representation affect memory use, data movement, and cost.

4 | **Stochastic gradient descent (SGD)**: A training algorithm that repeatedly adjusts model parameters using small, randomly sampled groups of examples, called batches, rather than processing the entire dataset for each update.

5 | **AlexNet**: Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton trained this convolutional neural network across two GPUs. At ILSVRC 2012, its 15.3 percent top-5 error, meaning the correct class was absent from its five highest-scoring predictions, substantially beat the second-place system's 26.2 percent (Krizhevsky et al. 2012). Section 1.3.3 returns to its architecture and systems co-design.



Test coverage is a vanishing fraction of the input space.

↪ Versioning training datasets so that a model can be reproduced and traced to its inputs is developed in section 4.8.4.

> **Checkpoint 1.1: The paradigm shift**
>
> Before tracing the history of AI, verify your understanding of the paradigm shift in how software is built:
>
> - ☐ **Software 1.0 vs. 2.0**: can you distinguish Software 1.0 (explicit instructions) from Software 2.0 (behavior learned from data)?
> - ☐ **Debugging shift**: can you identify which dataset evidence to inspect when accuracy falls without a code change?
> - ☐ **Verification gap**: can you explain why finite test coverage can provide statistical evidence but never exhaustive proof, and why production monitoring must continue gathering that evidence?

Learning behavior from data can seem inevitable in hindsight, but its adoption also reflects the scaling limits of earlier approaches. Many AI systems attempted to program intelligence directly through hand-crafted symbols and rules. Reading that history through a systems lens reveals recurring binding constraints, the limits whose relaxation would most improve the whole system: logic, knowledge acquisition, feature design, and the physical infrastructure that the rest of this book studies.

1.3 The Evolution of AI Bottlenecks

AI's history can be read as a succession of approaches that encountered limits in representation, data, or computing resources. When Alan Turing[6] proposed evaluating machine intelligence through observable responses (Turing 1950), computing machinery had little memory or processing capacity. Early systems pursued different paths. Frank Rosenblatt's Perceptron (1958) (Rosenblatt 1958) attempted to learn weights from sensory inputs, while Joseph Weizenbaum's ELIZA[7] (Weizenbaum 1966) executed hand-coded pattern substitutions. A single-layer perceptron could separate classes only with a linear boundary, and practical methods for training deeper networks remained limited. ELIZA's scripts depended on inputs matching their templates. The four eras below organize this history around recurring bottlenecks; the approaches and constraints overlapped rather than replacing one another cleanly.

Figure 1.1 traces attention to AI across seven decades of published literature and shades two "AI winter" intervals (1974–1980 and 1987–1993) marked by funding cuts and unmet expectations.[8] These setbacks reflected gaps between expectations and what systems could deliver. Later advances combined changes in algorithms, data, and computing resources; the sections below examine how those changes shifted the limiting constraint.

1.3.1 Rule-based approaches: Logic and knowledge bottlenecks

Rule-based approaches attempted to build intelligent systems by making knowledge explicit. Symbolic AI encountered a *logic bottleneck* when its rules could not capture real-world ambiguity. Expert systems narrowed the task to a domain but encountered a *knowledge bottleneck*: acquiring and maintaining the rules required extensive expert effort. Hand-crafted representations could be useful within well-defined tasks, but extending their coverage was costly.

1.3.1.1 The symbolic AI era and the logic bottleneck

Early AI research (1950s–1970s) pursued **symbolic AI**, representing knowledge and reasoning through explicit symbols and rules, an approach later crystallized in the physical-symbol-system hypothesis (Newell and Simon 1976). Researchers at the 1956 Dartmouth Conference[9] (McCarthy et al. 1955) hypothesized that aspects of intelligence could be precisely described and simulated by machines. Even then, Arthur Samuel at IBM demonstrated a different path in 1959 when a checkers program improved through self-play. His work coined the term "**machine learning**" (Samuel 1959), though the dominant paradigm remained symbolic. Daniel Bobrow's STUDENT[10] system exemplifies this approach (Bobrow 1964).

These systems could produce impressive demonstrations, yet they were operationally brittle. Their success depended on manually coded rules that mapped each acceptable input form to a

[6] **Alan Turing**: His "Imitation Game" approached machine intelligence through observable responses rather than a definition of its internal nature (Turing 1950). It provides an early example of evaluating a system through what it does.

[7] **ELIZA**: A natural-language program whose scripts could store selected inputs and reuse them in later responses (Weizenbaum 1966). This limited memory was programmed explicitly rather than learned from conversations. Extending its coverage required changing the scripts, even when the existing program ran correctly.

[8] **AI winters as systems failures**: The first AI winter (1974–1980) unfolded amid funding cuts; the 1973 Lighthill Report criticized the gap between AI promises and delivered results (Lighthill 1973). The second winter (1987–1993) involved a market and funding collapse around expert systems and specialized Lisp machines as general-purpose workstations undercut their economics (Hendler 2008). These episodes involved technical limitations, unmet expectations, and economics; hardware capacity alone does not explain them.

[9] **Dartmouth Conference (1956)**: The workshop organized around the term "artificial intelligence," already used in its 1955 proposal (McCarthy et al. 1955). The proposal discussed limits in computer speed and memory while treating programming as the larger obstacle. Physical capacity was part of the problem from the outset, even as researchers focused on language, abstraction, problem solving, and self-improvement.

[10] **STUDENT**: An early program that translated algebra word problems into equations using hand-written language rules (Bobrow 1964). Its success depended on the wording fitting those rules: an algebra solver could solve the resulting equations without establishing that the program had interpreted the original problem correctly.

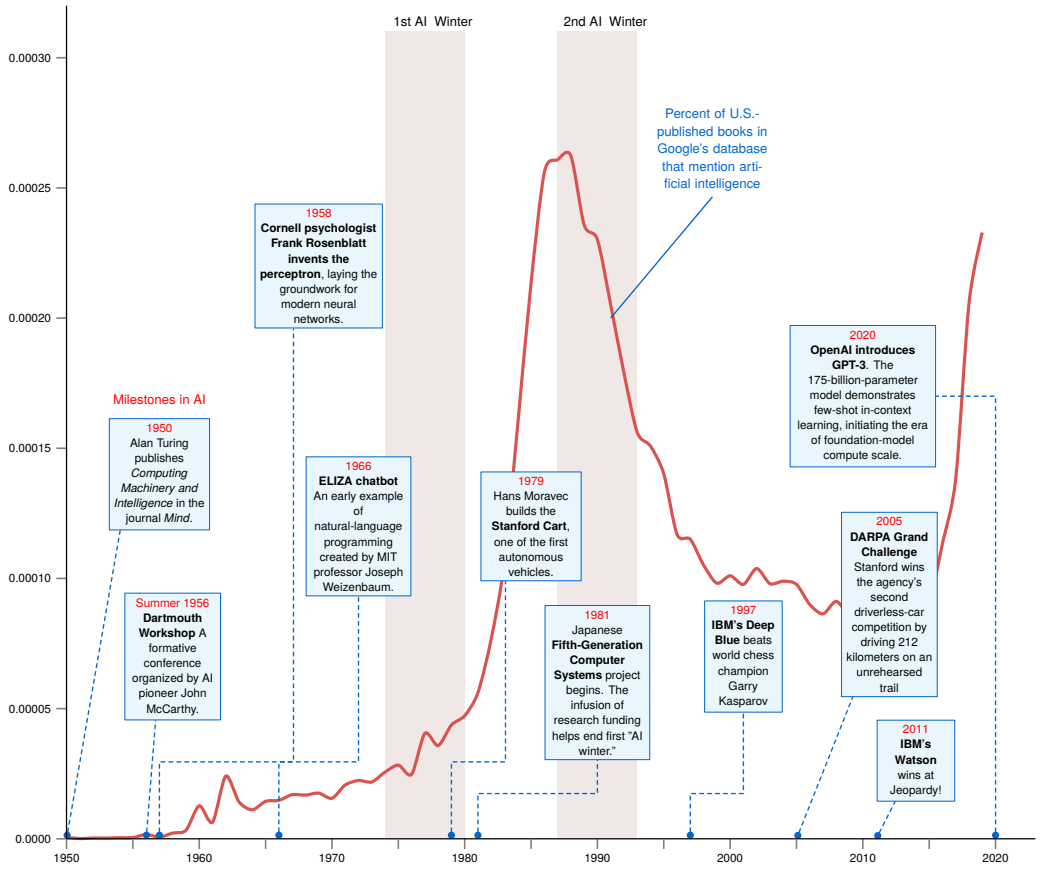


Figure 1.1: AI Development Timeline: A chronological proxy curve traces historical mentions of artificial intelligence in published literature (Michel et al. 2011), with light-brown bands marking the two AI Winter periods (1974–1980, 1987–1993). Callout boxes mark Turing's 1950 paper (Turing 1950), the Dartmouth conference (McCarthy et al. 1955), the Perceptron (Rosenblatt 1958), ELIZA (Weizenbaum 1966), Deep Blue (Campbell et al. 2002), and GPT-3 (Brown et al. 2020).

symbolic representation. STUDENT could solve an algebra problem once it translated English into equations, but a minor variation in phrasing could prevent that translation. The difficulty extended beyond language. Hans Moravec's[11] work on autonomous navigation at Stanford revealed that tasks humans find trivial (seeing, walking, grasping) were far harder to engineer than tasks humans find difficult, like chess or algebra.

11 **Moravec's paradox**: Hans Moravec observed that skills people acquire with little conscious effort, such as perception and movement, can be harder to reproduce computationally than formal reasoning tasks (Moravec 1988). Human familiarity with a task is therefore a poor guide to its engineering difficulty or resource requirements.

Example 1.1: STUDENT (1964)

STUDENT separated language interpretation from algebra:

```
language  "Two numbers sum to 30; one is twice the other."
parse     x + y = 30, x = 2y
solve     x = 20, y = 10
```

Once the representation was correct, solving was routine. The bottleneck was the translation: each unfamiliar phrasing needed another hand-written rule, so linguistic coverage grew only as fast as the rule base.

1.3.1.2 The expert systems era and the knowledge bottleneck

In the expert-systems era, engineers narrowed the problem. Instead of constructing general reasoning systems, they encoded deep knowledge from a specific domain. MYCIN, designed to diagnose blood infections, let medical knowledge be expressed as production rules (Shortliffe et al. 1975).

> **Example 1.2: MYCIN (1976)**
>
> MYCIN made its reasoning explicit. Patient facts triggered readable production rules:
>
> ```
> facts  stain=gram-positive, shape=coccus, arrangement=clumps
> rule   IF facts match THEN organism=staphylococcus (certainty=0.7)
> cycle  match facts -> fire rule -> update certainty -> repeat
> ```
>
> An inference engine repeated this cycle across hundreds of rules and propagated certainty factors toward a diagnosis. A physician could inspect every step, but coverage still grew one rule at a time: each exception had to be elicited, encoded, and reconciled with the existing rule base.

MYCIN performed well in specific tests, but its success exposed the **knowledge acquisition bottleneck**.[12] The problem was no longer whether a rule could express expertise. It was how tacit judgment entered the system and remained consistent as the rule base grew.

Faster machines could evaluate production rules at higher frequencies, but raw processor speed could not accelerate the extraction or maintenance of human expertise. Scalable AI required a mechanism to infer decision boundaries automatically from empirical examples rather than demanding that domain specialists enumerate every contingency in advance. This transition did not eliminate human design; it relocated engineering effort. Instead of hand-coding decision logic, engineers shifted to curating datasets, selecting mathematical representations, and formulating learning objectives from which models could optimize their own parameters. The next era exchanged the knowledge-acquisition bottleneck for the engineering challenge of structuring and extracting operational features from raw data.

[12] **Knowledge acquisition bottleneck**: Feigenbaum (1984) treated extracting and representing expert knowledge as a central challenge in building applied AI systems. Experts could exercise judgment without being able to state every rule behind it, so acquiring knowledge required more than transcribing a ready-made list.

1.3.2 The statistical learning era and the feature engineering bottleneck

The 1990s marked the shift to **statistical learning** and probabilistic systems. Instead of hard-coded logic, systems estimated probabilities from data, such as the conditional distribution $p(y \mid x)$ of a label y given an input x. This transition was driven by the availability of digital data and the "unreasonable effectiveness"[13] of large datasets.

Spam filtering illustrates this shift. Rather than maintaining lists of forbidden words, statistical filters learned the probability that a word implies spam based on millions of examples.

[13] **Unreasonable effectiveness of data**: The observation that a simple statistical model with abundant data can outperform a more sophisticated model with less data (Halevy et al. 2009). The benefit depends on the task, model, and data quality. This makes collecting and preparing useful examples part of the engineering work, rather than merely a preliminary step.

> **Example 1.3: Early spam detection systems**
>
> Statistical filters shifted the decision from a hand-written rule to a probability estimated from labeled messages:
>
> ```
> rules: if "free" or "winner" appears -> spam
> learn: labeled email -> word likelihoods -> Pr(spam | email)
> ```
>
> Maintenance did not disappear. It moved from editing keyword lists to curating representative examples and checking whether the learned probabilities still tracked current email.

Learning the decision boundary from data reduced one source of manual work but exposed another. Many statistical classifiers relied on engineers to convert raw inputs into features, measurable properties such as word counts or image edges. In computer vision, engineers designed feature extractors such as SIFT (Scale-Invariant Feature Transform) and HOG (Histogram of Oriented Gradients) to summarize local image patterns. Classifiers such as support vector machines (SVMs)

then learned decision boundaries from those features. The learner adjusted the boundary; engineers decided which evidence it could see. Extending the system to a new problem often meant rebuilding the preprocessing stack, creating the **feature engineering bottleneck**.

This hybrid approach combined human-engineered features with statistical learning. The Viola-Jones algorithm[14] (Viola and Jones 2001) exemplifies this era, achieving real-time frontal-face detection using simple rectangular features and cascaded classifiers. It showed that well-engineered features could enable practical low-latency applications, but only within narrow domains where experts could hand-craft the right representations. A conventional vision pipeline makes that manual effort visible, because several hand-crafted stages preceded any learning at all.

[14] **Viola-Jones algorithm**: The detector applied a sequence of classifiers, rejecting likely nonface regions early and spending more computation on promising regions. On the reported test set, it evaluated an average of ten features per candidate image region, although the complete cascade contained over 6,000 features (Viola and Jones 2001). Its speed depended on how little work most inputs required, not just on how fast each operation ran.

Example 1.4: Traditional computer vision pipeline

A conventional vision system separated representation from classification:

```
image -> resize -> HOG/SIFT features -> SVM -> label
```

Only the final decision boundary was learned. Moving from faces to pedestrians or another domain meant redesigning the feature extractor and often the entire preprocessing stack.

1.3.3 The deep learning era and the infrastructure bottleneck

Deep learning changed which part of the system learned. A neural network computes in layers, stages of arithmetic on learned weights in which each layer transforms the output of the one before. Instead of receiving features designed by humans, deep networks with many such layers learned representations directly from raw inputs such as pixels and audio waveforms. The training process could now shape both the representation and the decision boundary, enabling "end-to-end" learning.

The breakthrough was not algorithmic alone. Convolutional neural networks (CNNs), whose convolutional layers slide small learned weight patterns called filters across an image to detect the same feature at every position, existed earlier (LeCun et al. 1998, 2015); AlexNet paired architecture and training with **systems co-design**, choosing the model, training procedure, and hardware mapping together (Krizhevsky et al. 2012). Its parallel matrix operations matched GPU capabilities. With 60 million parameters distributed across two GTX 580 GPUs, AlexNet achieved 15.3 percent top-5 error, a 41.6 percent relative improvement over the next-best entry that year. The hardware partition in figure 1.2 carries the immediate systems lesson: the network divided work between the two GPUs and exchanged intermediate results only at selected layers to limit communication across the link between them (Krizhevsky et al. 2012). Model structure and machine organization were designed together.

Hardware still shaped what could be learned. In 2012, storing AlexNet's 60 million 32-bit parameters required roughly 240 megabytes, well within a single card's capacity. Training also needs working storage for intermediate results and for calculating and retaining parameter updates. These quantities, developed later as activations, gradients, and optimizer state, make training memory larger than the stored model alone, and the intermediate results grow with the number of images processed together. Each GTX 580 offered only 3 GB of video random-access memory (VRAM), and the original paper reports that this limit made the network too large to train on one card (Krizhevsky et al. 2012). AlexNet therefore divided convolutional and dense layers between two GPU streams communicating over the PCIe bus, the host's general-purpose peripheral interconnect. This early model-parallel design foreshadowed modern multi-GPU training. Deep learning reduced the need for hand-crafted features, but it moved the binding constraint to the infrastructure required to store data, move parameters, and coordinate computation.

This shift inaugurated the **infrastructure bottleneck**, in which physical memory capacity, interconnect bandwidth, and hardware orchestration dictate the boundaries of what models can learn. Modern foundation models make the magnitude of this constraint clear. GPT-3 (175 billion parameters), for example, was trained on roughly 300 billion tokens (the word fragments a language model reads and predicts) curated from filtered web text, books, and Wikipedia (Brown et al. 2020). Using the dense-training approximation ($O \approx 6 \cdot P \cdot D$ operations for a model with P parameters trained on D tokens, counting both forward inference and backward parameter-update

↪ Model parallelism, which partitions a model across accelerators when it outgrows one device's memory, as AlexNet's two-GPU split did, is developed in section 8.8.1.2.

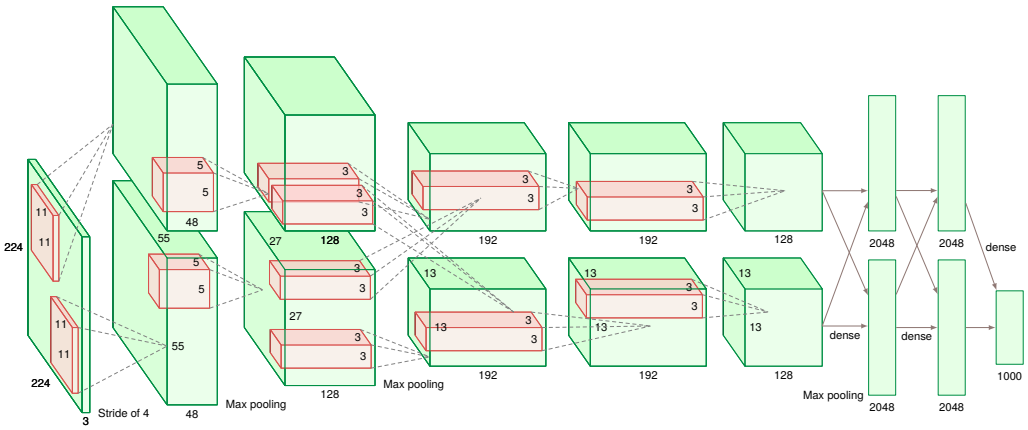


Figure 1.2: AlexNet Architecture: The network that launched the deep learning revolution at ImageNet 2012 (Krizhevsky et al. 2012). Two parallel GPU streams process 224×224 input images through convolutional layers (green blocks) that extract spatial features at decreasing resolutions, converging through three fully connected layers to 1,000 output classes. With 60 million parameters trained across two GTX 580 GPUs, AlexNet achieved 15.3 percent top-5 error, a 41.6 percent relative improvement over the second-place entry.

passes, derived in section 8.3.1), that scale represents roughly 314 zettaFLOPs of total computation (1 zettaFLOP $= 10^{21}$ FLOPs). The tokenized dataset occupies roughly 420 GB. A retrospective audit by Patterson et al. (2021) estimated that this single training run consumed roughly 405 V100 GPU-years. The engineer's primary challenge was no longer hand-crafting visual edge detectors, but orchestrating hundreds of parallel accelerators across high-speed fabrics without catastrophic mid-training failures.

↪ The dense-training approximation used for this operation count is developed in section 8.3.1.

Viewed through this systems lens, the transitions reveal shifting constraints in human effort (rules, knowledge, features) and physical infrastructure. Table 1.2 compares the four major eras across their core reasoning strengths, binding systems bottlenecks, and operational data requirements.

Table 1.2: AI Paradigm Evolution: Each era is viewed through a recurring systems bottleneck. Deep learning (far right) reduced the need for manual feature engineering while increasing demands on computing infrastructure.

Aspect	Symbolic AI	Expert Systems	Statistical Learning	Deep Learning
Key Strength	Logical reasoning	Domain expertise	Versatility	Pattern recognition
Bottleneck	Brittleness (Rules break)	Knowledge Entry (Experts are scarce)	Feature Engineering (Manual preprocessing)	Infrastructure (Compute and data scale)
Data Handling	Minimal data needed	Domain knowledge-based	Moderate data required	Massive data processing

The progression across these four eras highlights a recurring structural shift: engineering effort migrates away from handcrafting domain heuristics toward building scalable compute and data pipelines. The bitter lesson offers a broader perspective on why this shift has recurred across decades of machine learning history.

1.4 The Bitter Lesson

Expert systems invested engineering effort in encoding domain knowledge; deep learning systems invest much of that effort in learning from data at scale. The **bitter lesson** captures the historical advantage of general methods, especially search and learning, that can make effective use of increasing computation. Richard Sutton[15] crystallized this insight in his 2019 essay "The Bitter Lesson" (Sutton 2019). Sutton wrote, "The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin."

[15] **Richard Sutton**: A reinforcement learning pioneer whose essay emphasizes search and learning as methods that scale with computation (Sutton 2019). The lesson is "bitter" because carefully encoded human knowledge has repeatedly been overtaken by more general methods. It is a lesson about long-term progress, not a claim that domain knowledge has no engineering value.

Training scale at the frontier is often stated in **accelerator-days** (GPU-days when the accelerator is a GPU): one reference accelerator running continuously for 24 hours. Table 1.3 pairs representative benchmark milestones with their hardware. Its final column illustrates the progression from CPU-based learning to multi-thousand-accelerator clusters, using 2.5 million reference GPU-days as an estimated frontier-training anchor (Patel and Wong 2023).

Table 1.3: AI Performance Evolution Across Paradigms: Representative systems illustrate the growth in computational scale across eras. GPT-4 training details are estimated reference anchors rather than official disclosures.

Era	Approach	Representative Task	Performance	Computational Resources
Statistical ML (1990s–2000s)	Feature engineering + learning	Handwritten digit recognition	about 98–99% on MNIST-era benchmarks (LeCun et al. 1998)	CPU-era feature pipelines; resources varied by implementation
Deep Learning (2012)	End-to-end neural networks	ImageNet top-5 accuracy	84.7% (AlexNet)	6 days on 2 GPUs
Modern Deep Learning (2020+)	Large-scale transformers	ImageNet top-1 accuracy	88.55% (ViT-H/14) (Dosovitskiy et al. 2021)	Large-scale Tensor Processing Unit (TPU) pretraining
Modern Deep Learning (2023)	Foundation models	MMLU benchmark	86.4% (GPT-4) (OpenAI et al. 2023)	Estimated ~2.5 million reference GPU-days (on the order of 25,000 reference GPUs run for 100 days) (Patel and Wong 2023)

The milestones in table 1.3 trace the expanding scale of the underlying computational substrate across distinct eras. Because digit recognition, ImageNet classification, and Massive Multitask Language Understanding (MMLU) evaluate qualitatively different capabilities under separate metrics, this progression documents how crossing successive capability thresholds demanded orders-of-magnitude increases in hardware investment, rising from single-node CPU routines to multi-megawatt accelerator clusters. Directly testing Sutton's bitter lesson—determining whether general methods powered by raw computation systematically surpass hand-engineered representations—requires holding the task fixed and observing competing approaches within a shared domain.

Game playing makes the roles of search and learning concrete. In chess, IBM's Deep Blue[16] defeated world champion Garry Kasparov in 1997 by combining custom chess hardware, large-scale search, and chess-specific evaluation knowledge. In Go, DeepMind's AlphaGo learned from expert games and self-play, using neural networks to guide tree search, but still used some engineered input features (Silver et al. 2016). AlphaGo Zero[17] provides the clearer example of Sutton's lesson: it learned through self-play without human game records or those hand-engineered features (Silver et al. 2017). The advance depended on methods that could turn computation into improved play, not merely on a faster machine.

The lesson is "bitter" because encoding human expertise can appear to be the most direct path to a solution. Sutton's examples show how scalable search and learning repeatedly overtook such approaches. More computation is not a guarantee of better results: the method must use it productively, and the data and objective must support the intended task.

Modern language models such as GPT-4 and image generation systems such as DALL-E illustrate the engineering demands of learning at scale. Much of their behavior is learned from large datasets rather than expressed as explicit linguistic or artistic rules. Estimates for models at GPT-3's scale suggest roughly 1.3 GWh of training energy[18] (Patterson et al. 2021), and serving these models to millions of users turns inference into a continuous data-center power, cooling, and capacity-planning problem.

Realizing the bitter lesson in practice demands an engineering discipline that extends far beyond algorithmic design. When model capability scales with raw computation, the binding limits inevitably migrate to physical infrastructure: memory bandwidth,[19] interconnect latency, thermal dissipation, and data pipeline throughput. Scaling computation transforms the mathematical model into a physical systems artifact governed by power envelopes, memory hierarchies, and communication bottlenecks. Treating data movement, arithmetic throughput, and operational reliability as a unified design problem is therefore mandatory.

The bitter lesson explains *why* machine learning systems must be engineered to absorb continuous computational growth; the frameworks developed in this chapter establish *how*. This textbook grounds that foundation at the scale of a single host, while frontier-scale systems demonstrate how identical resource constraints govern distributed clusters. That comprehensive responsibility defines AI engineering, which section 1.9 formalizes once its quantitative foundations are in place. Grounding this discipline requires first defining its core artifact through the operational realities of a concrete production service.

[16] **Deep Blue**: Its 480 custom chess chips worked alongside general-purpose processors, searching about 100 million positions per second in tactical positions and approaching 200 million in quieter positions (Campbell et al. 2002). The hardware accelerated chess-specific search and evaluation; it did not remove the system's reliance on encoded chess knowledge.

[17] **AlphaGo Zero**: After three days of self-play training, it defeated AlphaGo Lee, the version that had beaten Lee Sedol, by 100 games to 0 (Silver et al. 2017). Self-play generated the training examples, allowing search and learning to improve play without a dataset of human games.

[18] **GPT-3 (Generative Pre-trained Transformer 3) training energy**: Patterson et al. (2021) estimated that GPT-3's training consumed approximately 1,287 MWh and produced 552 metric tons of CO_2-equivalent emissions. The energy estimate corresponds to the annual electricity use of about 120 US households at a baseline of 10.7 MWh/household-year. Emissions also depend on how that electricity is generated.

[19] **Memory bandwidth**: The rate at which a memory interface transfers data, measured in bytes per second. A processor must receive the values an operation needs before it can perform the arithmetic. When moving those values takes longer than computing with them, faster arithmetic alone cannot remove the delay; reducing transfers or increasing the available bandwidth addresses the limiting resource.

1.5 Defining ML Systems

A production spam filter needs more than a model that recognizes unwanted messages. It needs representative training data, a way to update learned behavior as attacks change, and infrastructure that classifies incoming mail within the delivery budget. Global email traffic is measured in hundreds of billions of sent and received messages per day (Statista Research Department 2024), making this a substantial systems problem for large providers.

The challenge begins with data. The filter is trained on labeled examples and needs updated training data and periodic retraining as spam tactics change. It is also an algorithmic problem, because the model must generalize from those examples to messages it has never seen before. Precision measures the fraction of flagged messages that are actually spam; recall measures the fraction of all spam messages that the filter catches. Flagging more messages may catch more spam while also hiding legitimate mail, so the system must evaluate both consequences. Finally, it is an infrastructure problem. Providers must handle incoming traffic, store and update models, and return predictions within their delivery latency budget. The classifier is therefore only one component of an evolving data, software, and hardware system.

When a new phishing template appears, the data layer must capture representative messages, the algorithm must learn to separate attacks from legitimate mail, and the infrastructure must deploy and serve the updated model. Any layer can fail while the others continue working, so the full chain from measurement through learning to execution is the right unit of analysis.



Definition 1.1: Machine learning systems

Machine learning systems combine models that learn from data with the software and hardware needed to train, deploy, and maintain them. Their performance depends jointly on data quality, algorithm choice, and machine capacity.

1. **Significance**: Requirements are end to end. A spam filter is useful only when its training data represents current attacks, its model separates malicious from legitimate mail, and its serving path classifies each message within the delivery budget. Failure in any layer changes the user-visible result.
2. **Distinction**: Correct execution does not establish the quality of learned behavior. Accuracy depends on the model and the population it meets, so it can change when that population moves away from what the model learned, even if code and weights do not. The change may occur without an error or exception.
3. **Common pitfall**: The model is not the system. Data pipelines, feature transformations, serving infrastructure, monitoring, and feedback loops surround the learned parameters and often dominate the engineering burden (Sculley et al. 2015).

The definition exposes three diagnostic axes: the data that describes the task, the algorithm that turns examples into behavior, and the machine that executes that behavior within the operating budget. The same user-visible symptom can originate in any of the three. A missed phishing message may mean that representative examples were absent, that the learned decision boundary was inadequate, or that the serving path missed its latency budget. Treating every miss as a model problem risks improving the wrong component. The D·A·M taxonomy separates these causes without assuming they are independent.

Each explanation calls for different evidence. The data hypothesis sends the engineer to coverage, labels, and distribution shift. The algorithm hypothesis sends the engineer to error slices (errors grouped by input type), model capacity, and the learning objective. The machine hypothesis sends the engineer to latency, throughput, memory, and utilization. These investigations are not interchangeable. Faster hardware cannot supply examples that were never collected, while a larger dataset cannot repair a serving path that misses its deadline because memory is saturated.

The relevant constraint is the one whose relaxation improves the end-to-end result. This idea of a *binding constraint*, used informally in the history above, prevents teams from optimizing the most visible component instead of the component that governs the outcome. The purpose of D·A·M is therefore operational: it identifies the next hypothesis to test and the class of intervention capable

of changing system behavior. The diagnosis is provisional. Once an intervention relaxes one limit, the system must be measured again because a different axis may now bind. D·A·M is a loop, not a one-time label.

Definition 1.2: The D·A·M taxonomy

D·A·M taxonomy is a diagnostic framework for examining machine learning system bottlenecks along three axes. Data determines what examples and bytes the system must process, Algorithm determines the model structure and work required to learn or predict, and Machine determines the hardware capacity available to execute that work. The goal is to identify which axis is the binding constraint.

1. **Significance**: The diagnostic power is concrete even before detailed hardware arithmetic enters the story. If the spam filter misses a new phishing campaign because the training set never contained that tactic, the binding axis is Data. If the training examples are adequate but the model cannot express the pattern, the binding axis is Algorithm. If both are adequate but the service cannot classify messages quickly enough during a traffic spike, the binding axis is Machine. Quantitative diagnosis begins by asking which axis is limiting the system.
2. **Distinction**: The D·A·M taxonomy connects predictive quality to physical execution. Algorithm choice shapes both the data needed to learn useful behavior and the machine resources needed to run it. Those requirements depend on the task and training procedure, so model size alone does not determine either generalization or runtime.
3. **Common pitfall**: A frequent misconception is that the three axes are independent. Changing from a simple classifier to a larger model can require more memory, different serving infrastructure, and a broader data distribution. The axes move together.

Figure 1.3 shows the dependencies among Data, Algorithm, and Machine. Changing one part can change the requirements imposed on the other two.

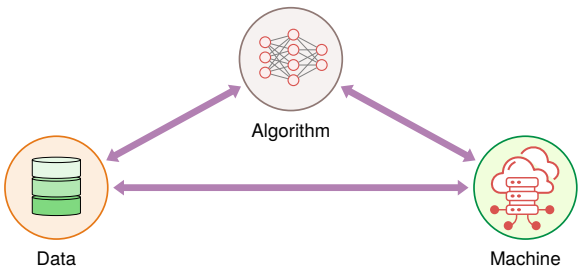


Figure 1.3: The D·A·M Axis Triad: Data, Algorithm, and Machine are three interdependent parts of an ML system. The bidirectional links emphasize that changing one part can change the requirements imposed on the other two.

An operational diagnosis earns confidence only when an intervention validates it: relaxing the identified constraint must improve full-system behavior. Tracing the closed loops in figure 1.3 reveals why such interventions rarely end the optimization process, as resolving one bottleneck frequently migrates the binding constraint across adjacent edges. Upgrading to faster accelerators (Machine) leaves execution units idling if storage I/O cannot stream input batches quickly enough (Data). Expanding dataset volume (Data) exposes model architectures that lack the expressive capacity to absorb new patterns (Algorithm). Scaling model capacity to close that gap (Algorithm) quickly exceeds accelerator memory, triggering out-of-memory faults or expensive memory paging across the PCIe bus (Machine). Engineering an ML system therefore requires identifying the currently binding constraint while anticipating where the bottleneck will migrate once that constraint is relaxed.

While the triad separates the three axes to simplify initial diagnosis, practical systems operate at their boundaries. Data formats govern memory traffic, model architectures dictate hardware func-

tional unit utilization, and deployment budgets reshape both data representation and algorithmic design. Figure 1.4 partitions these interfaces into a three-set Venn diagram whose pairwise overlaps each pose one question: what the system can learn from ($D \cap A$), how information moves ($D \cap M$), and how computation executes efficiently ($A \cap M$).

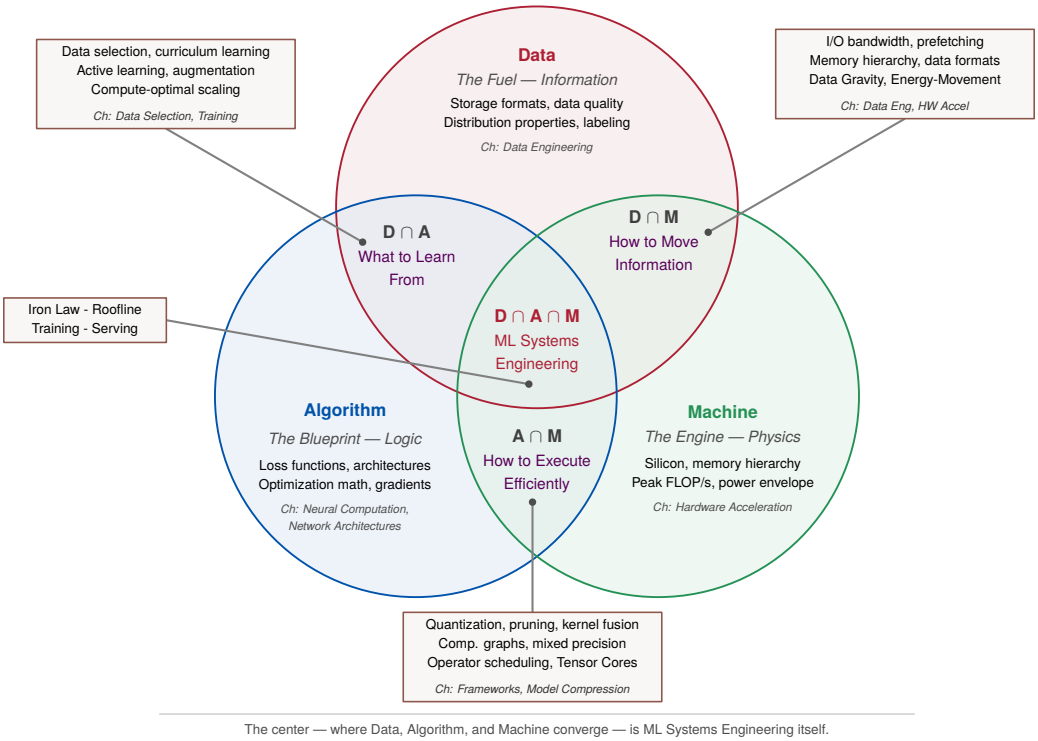


Figure 1.4: The D·A·M Intersection Landscape: Data, Algorithm, and Machine begin as distinct diagnostic lenses, but practical ML systems live at their boundaries. Pairwise intersections ask what a system can learn from, how information moves, and how computation executes efficiently. Their center is ML systems engineering, where all three constraints must be balanced together.

At the center, all three questions must be answered together. The chapters ahead examine each interface in detail. Data selection and training ask which examples help the model learn useful behavior. Data engineering and hardware acceleration ask how to deliver the required information to the machine. Frameworks and execution techniques ask how to turn the model's computations into efficient work on that machine.

Pruning a model (Algorithm), which removes parameters that contribute little to its predictions, alters its memory footprint and how much of it stays in fast on-chip memory (Machine). Compressing input features can likewise reduce transfers while adding preparation work (Data). These changes must be evaluated across the interfaces, because a local improvement may leave the end-to-end result unchanged.

↪ How pruning shrinks a model's memory footprint, and why its effect on speed depends on the sparsity pattern the hardware can exploit, is developed in section 10.3.1.

These couplings govern more than where the bottleneck sits; they dictate how systems fail. A corrupted data pipeline, unmodeled distribution shift, or a feature computed one way during training and another way during serving can each degrade model behavior in production silently, and bandwidth saturation can just as quietly erode latency headroom, all without triggering an operating system crash or runtime exception.

1.6 ML vs. Traditional Software

A deployed model can return predictions on time while those predictions become less useful. Understanding this separation between correct execution and predictive quality is central to engineering ML systems.

Crashes, unhandled exceptions, and failed health checks make some software defects directly observable. Conventional software can also produce silent numerical errors or logic bugs. Machine learning adds another route to **silent degradation**: learned behavior can lose predictive quality as the operating population changes. The runtime executes instructions without error, memory allocations succeed, and inference[20] servers return predictions well within latency service level agreements (SLAs), yet the statistical validity of the output steadily deteriorates.

Consider a hypothetical automotive driver-assistance system. Diagnostic trouble codes and hardware watchdog timers can expose hardware or communication faults, but they do not measure how often a perception model detects pedestrians. Its recall might decline from 95 percent to 85 percent as seasonal conditions introduce low-angle winter sunlight, heavy precipitation, or bulky clothing underrepresented in training. The accelerator could continue processing images at full frame rate without raising a hardware fault, while pedestrian detection becomes less reliable.

In this example, the miss rate triples, from 1 in 20 pedestrian frames to 3 in 20. That is a substantial change in detection quality, but it does not imply a tripling of vehicle-level safety risk. Temporal filtering, sensor fusion, and downstream control affect how missed detections influence a driving decision. Evaluating those interactions requires evidence about the complete system.

Input distributions can drift as environmental conditions, user behavior, or sensor calibrations change (Gama et al. 2014; Quiñonero-Candela et al. 2009). A model may then apply its fixed weights to inputs unlike those on which it was evaluated. The machine can execute every operation correctly while producing predictions whose quality no longer meets the requirement.

Outcome measurements establish whether predictive quality has fallen; distribution-divergence signals can prompt investigation before those outcomes arrive. When both are observed, model degradation can be approximated locally as a function of distribution shift. Let Accuracy_0 denote the baseline accuracy at deployment, $\mathcal{D}(P_t \| P_0)$ represent the statistical divergence between current production distribution P_t and training distribution P_0, and λ denote the locally fitted sensitivity to that shift measure, so that $\text{Accuracy}(t)$ is the accuracy after the model has been deployed for time t. This relationship, formalized in equation 1.2, is the **degradation equation**:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0) \tag{1.2}$$

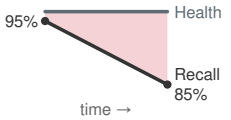
This approximation describes a local trend fitted from observed inputs and labeled outcomes; it is not a universal law of accuracy loss. Part IV returns to it as a statistical drift diagnostic (principle 10). **Data drift** occurs when P_t differs from P_0. Such a shift can reduce inference quality even when the model and execution environment remain unchanged, but the shift alone does not establish a quality loss. Large changes may fall outside the range where the fitted relationship holds. The divergence measure $\mathcal{D}(\cdot \| \cdot)$ is left general here.

The equation separates baseline quality from sensitivity to shift. Plotted as accuracy against divergence, it is a line with intercept Accuracy_0 and slope $-\lambda$. Holding λ fixed, raising Accuracy_0 shifts the whole line upward. Better data, objectives, and architectures can change both quantities, so an improved baseline does not by itself establish robustness to drift. For the driving example, training on representative winter images may reduce sensitivity to seasonal conditions; evaluating pedestrian recall under those conditions establishes whether it does. Monitoring $\mathcal{D}(P_t \| P_0)$ provides a warning that prompts engineers to seek outcome evidence. That evidence determines whether retraining or another intervention is needed. *Knowing when to reevaluate is as important as knowing how to train.*

Silent degradation can also originate inside the system through **training-serving skew**. Training and serving may use separate implementations to transform raw inputs into model features. If those transformations differ, for example by resizing images with a different interpolation method or applying different missing-value defaults, the model receives inputs unlike those used in training. The model can execute cleanly and return predictions on time while accuracy falls. This is a software consistency defect whose consequence is degraded learned behavior.

These failure modes reshape the entire engineering lifecycle. Host-health signals such as CPU utilization, memory residency, network I/O, and HTTP status codes remain necessary, but do not measure whether predictions are useful. Machine learning operations must track these infrastructure metrics alongside data quality, feature distributions, and prediction accuracy. Because an ML

[20] **Inference**: Applying a trained model to inputs, whether during evaluation or deployment. Interactive serving often imposes a response-time limit, whereas offline inference may prioritize the total number of examples processed per second. These workloads can require different execution choices even for the same model (see chapter 13).



Recall falls while health metrics stay flat.

↪ Concrete measures of distribution shift and their use in monitoring are developed in section 14.6.2.

↪ Keeping transformation code and its fitted parameters identical across training and serving is developed in section 4.6.1.

system can fail silently while infrastructure metrics appear nominal, system design must integrate continuous evaluation from data ingestion through inference serving.

Useful predictions satisfy the first half of the ML systems mandate. The second half requires producing them within the available latency, memory, energy, and cost budgets. Meeting those budgets requires a quantitative account of where execution time goes.

1.7 Iron Law of ML Systems

The physical cost of computational scale becomes concrete in two familiar failures. A training job stalls when storage cannot feed an accelerator; an inference path misses its deadline when model state moves too slowly through memory or across the network. Their symptoms differ, but both consume the same finite time budget through data movement, computation, and overhead. Assuming these costs occur sequentially, the **iron law of ML systems** models total execution time T (seconds) as their sum (equation 1.3):

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}} \tag{1.3}$$

This equation is the mathematical spine of this book. Under the sequential assumption, it estimates execution time for workloads ranging from training runs to individual predictions. Three terms account for the physical-execution half of the dual mandate introduced in section 1.1:

1. **The memory term** $(D_{\text{vol}}/\text{BW})$ represents the physical cost of moving bits. D_{vol} is the volume of data moved (bytes), and BW is the memory or network bandwidth (bytes/s). Whether loading terabytes from cloud storage or fetching weights from high-bandwidth memory (HBM), the stacked DRAM packaged beside an accelerator, performance is often limited by I/O physics. Part I develops this foundation.
2. **The compute term** $(O/(R_{\text{peak}} \cdot \eta_{\text{hw}}))$ represents the cost of arithmetic. O is the number of floating-point operations (FLOPs), R_{peak} is the hardware's theoretical peak throughput (FLOP/s), and η_{hw} is dimensionless realized hardware utilization $(0 \leq \eta_{\text{hw}} \leq 1)$. Parts II and III develop this term.
3. **The latency term** (L_{lat}) represents orchestration, networking, and serialization overhead (seconds), treated as fixed for the modeled execution. Engineering changes can reduce this overhead, which can dominate small, latency-critical requests where little data moves and little arithmetic runs. Part IV develops this term.

Consider a hypothetical task that spends 2 seconds moving data, 8 seconds computing, and 1 second on overhead. Its total is 11 seconds. Doubling the arithmetic rate reduces computation to 4 seconds, but leaves movement and overhead unchanged. The task now takes 7 seconds, rather than half its original time. A faster component helps only through the portion of the time budget it changes.

The iron law and the D·A·M taxonomy sort different things. D·A·M sorts causes by what the data covers, what the model computes, and what the machine supplies; the iron law sorts where the time goes. The Data axis therefore reaches both predictive quality and the memory term, and the symbol D_{vol} (bytes moved) is distinct both from the Data axis and from the dataset size D in the $6 \cdot P \cdot D$ training estimate.

> **Systems Perspective 1.1: The iron law analogy**
>
> The name "iron law" follows Patterson & Hennessy's Iron Law of Processor Performance (Hennessy and Patterson 2017). P&H's law is a *multiplicative decomposition* (a tautology factoring CPU time), whereas this equation is an *additive first-order model* that approximates performance under simplifying assumptions.

The additive form assumes sequential execution. When movement and computation overlap, the critical-path lower bound replaces the sum in equation 1.4:

$$T_{\text{pipelined}} \geq \max\left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}\right) + L_{\text{lat}} \tag{1.4}$$

This max-based formulation reflects hardware that transfers the next block of data while computing on the current one, so the slower of the two stages hides the faster. Equality requires ideal overlap and assumes that L_{lat} remains outside the overlapped phases.

Even in additive form, the iron law remains useful because its value lies in identifying which physical constraint dominates before optimizing. Amdahl's law (Amdahl 1967) plays the same role for speedups: accelerating a fraction f_{parallel} of the baseline execution time by a factor G bounds the whole task's speedup at $1/((1-f_{\text{parallel}}) + f_{\text{parallel}}/G)$, so the untouched remainder caps the gain however large G becomes. Section A.3.2 presents the refined treatment, including pipelining and overlap techniques that transform the additive model into the max-based formulation used in practice.

Diagnosing the limiting term requires comparing the workload with the machine. Arithmetic intensity ($I = O/D_{\text{vol}}$) counts the operations performed per byte moved. Setting the memory and compute terms equal gives $I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$ FLOP/byte (taking $\eta_{\text{hw}} = 1$). The Roofline Model, a plot of attainable throughput against arithmetic intensity, calls this machine-specific boundary the **ridge point**. For the accelerator class used in the GPT-3 estimate below, it sits near 153 FLOP/byte: roughly a hundred operations per fetched byte. Under ideal overlap, memory bandwidth limits the throughput bound below the ridge point; arithmetic throughput limits it above. In sequential execution, reducing either term still reduces the sum, but improving the smaller term yields a limited end-to-end gain. Actual performance also depends on realized utilization. Faster arithmetic offers limited benefit when movement dominates, just as more bandwidth offers limited benefit when computation dominates. Overhead (L_{lat}) sits outside this comparison and can dominate small requests. The engineering choices are to move less data (D_{vol}), execute fewer operations (O), use the hardware more effectively (η_{hw}), or reduce overhead (L_{lat}). The single-accelerator estimate below compares these execution models before a larger training estimate isolates the compute term under stated assumptions.

↪ The Roofline Model, which bounds attainable throughput by arithmetic intensity and locates the ridge point, is developed in section D.2.1.

Napkin Math 1.1: Pricing one local execution

Problem: A hypothetical workload on one accelerator moves 2 GB at 100 GB/s, performs 1 GFLOP at an effective arithmetic rate of 100 GFLOP/s, and incurs 5 ms of fixed overhead. Movement precedes computation. What execution time does the iron law predict, and which rate should be improved first?

Math: Dividing bytes by bytes/s gives a movement time of 20 ms. Dividing FLOPs by FLOP/s gives a compute time of 10 ms; the supplied effective rate already includes η_{hw}, so it must not be multiplied by efficiency again. Adding these durations and overhead gives 35 ms. Movement is the largest term.

Result: Raising bandwidth to 200 GB/s reduces movement to 10 ms and total sequential time to 25 ms. Arithmetic and overhead remain unchanged, so doubling bandwidth does not halve the total.

Systems insight: If movement and computation could instead overlap ideally, with overhead still outside both phases, the lower bound would be 25 ms before the upgrade and 15 ms afterward. The upgrade balances the two stages; further bandwidth increases alone would leave that ideal bound unchanged. Whether the sum or the bound describes a measured run depends on its execution schedule.

A larger training estimate isolates the compute term under an explicit arithmetic-dominated assumption. Its efficiency factor also accounts for the losses involved in coordinating many accelerators, beyond the local execution just calculated.

Napkin Math 1.2: Training GPT-3

Problem: How long would a GPT-3-class model take to train on a hypothetical modern cluster of 1,024 accelerators? This reference cluster uses current data-center accelerators, not the V100 GPUs of the original run estimated earlier at roughly 405 V100 GPU-years, so the result shows what the same work costs on newer hardware.

Given:

- **Operations** (O): $\approx 3.14 \times 10^{23}$ FLOPs
- **Peak Rate** (R_{peak}): 312 TFLOP/s
- **Efficiency** (η_{hw}): $\approx$ 45 percent (an assumed planning value for large-scale distributed training, where communication and stalls keep realized throughput well below peak)
- **Accelerator Count** (N_{GPU}): 1,024 accelerators

Math:

Assume that the training workload reuses data sufficiently to make arithmetic dominant and overlaps data loading with computation. Under this compute-dominated approximation, isolating the compute term across N_{GPU} parallel accelerators yields the training time T_{train}:

- $T_{\text{train}} \approx \frac{O}{N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}}} \approx \frac{3.14 \times 10^{23}}{1024 \times 312 \times 10^{12} \times 0.45} \approx 25$ days

Result: 25 days.

Systems insight: If hardware utilization (η_{hw}) improves from 45 percent to 60 percent through better scheduling and more efficient execution, training time drops to 19 days, saving 6 days of expensive compute time.

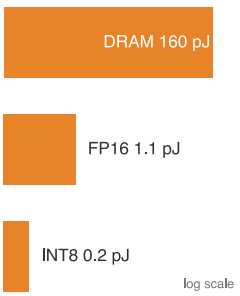
The equation is dimensionally consistent because each term resolves to seconds. One cannot add FLOPs to bytes any more than one can add meters to kilograms; the iron law adds time to time to time.

↪ The term-by-term unit check behind this statement, and its use as a test for any performance equation, are developed in section D.2.2.

The iron law governs execution time, but time is not the only physical constraint. For battery-powered edge devices and warehouse-scale accelerator clusters alike, energy dissipation and thermal limits often bind the system before throughput limits are reached. Moving bits across the memory hierarchy exacts an **energy tax** that parallels the delay of the memory term.

Let D_{vol} be the total data volume moved (bytes), E_{move} the energy per byte moved, O the total operation count, and E_{compute} the energy per operation. Equation 1.5 formalizes this relationship, following the hardware-energy observation that data movement can dominate arithmetic energy (Horowitz 2014):

$$E_{\text{total}} \approx \underbrace{D_{\text{vol}} \times E_{\text{move}}}_{\text{Movement Term}} + \underbrace{O \times E_{\text{compute}}}_{\text{Compute Term}} \tag{1.5}$$



Moving a byte costs about 145 times an FP16 op, the data-movement tax.

The energy comparison depends on the transfer size and arithmetic operation being counted. Under the energy constants used in this text, moving one byte from off-chip Dynamic Random-Access Memory (DRAM) consumes roughly 145.5× the energy of an FP16 multiply and 800× that of an INT8 (8-bit integer) multiply (Horowitz 2014). Whether movement dominates a complete workload also depends on the number and width of transfers relative to its operation count. The physical reason for the per-access gap is that data movement requires charging and discharging wires over longer distances, while arithmetic occurs locally within a processing unit's circuits. Minimizing avoidable data movement (D_{vol}) can therefore improve both speed and energy efficiency. Part III states this per-event comparison as a governing principle of optimization (principle 7).

Checkpoint 1.2: The iron law

The iron law ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$) is the analytical backbone of this book. Before proceeding, verify you can manipulate its terms:

- ☐ **Memory term** (D_{vol}/BW): identify the physical resource that bounds this term and name one workload regime where it dominates.
- ☐ **Compute term** ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$): identify the hardware quantity and utilization factor that bound this term.
- ☐ **Latency term** (L_{lat}): identify which costs remain after data movement and arithmetic have been optimized.
- ☐ **Peak throughput**: holding the other quantities fixed, if you double R_{peak}, which term improves, and why does total execution time fall by less than half when movement and overhead also contribute?

The same physical terms that determine time and energy also determine cost: every byte moved, operation executed, and millisecond of latency consumes infrastructure budget. The next test is therefore economic, evaluating whether added compute buys enough model improvement to justify the expenditure.

1.7.1 Return on compute (RoC) as an economic lens

In the spirit of quantitative design, **return on compute (RoC)** measures the incremental accuracy gain per added dollar of infrastructure investment, in accuracy points per dollar:

$$\text{RoC} = \frac{\Delta\text{Accuracy}}{\Delta\text{Compute Cost}} \tag{1.6}$$

The RoC test compares that ratio against what each point of accuracy is worth to the application. A 1-percentage-point gain may fail the test if it requires a 10× increase in O (total operations) whose cost exceeds the application's budget or the gain's value. Alternatives must be compared against a common baseline, with a positive added cost. The GPT-3 estimate above demonstrates a different benefit: assuming scheduling changes preserve model quality, raising η_{hw} from 45 percent to 60 percent saves 6 days of cluster time for the same training work. This is a cost saving, not an accuracy gain. It can free budget for another experiment, whose incremental accuracy and cost must then be measured. The iron law helps estimate what computation costs; RoC asks whether an additional investment is worthwhile.

1.7.2 Lighthouse models put the iron law into practice

Each term of the iron law suggests an engineering intervention. The memory term can be reduced by moving fewer bytes or increasing effective bandwidth. The compute term can be reduced by performing fewer operations or using arithmetic units more effectively. The latency term calls for reducing orchestration and communication overhead. The equation alone does not say which intervention a system needs; that depends on its measured workload and whether the required data fits in memory.

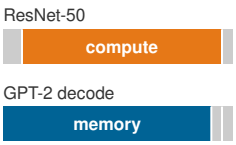
Concrete workloads answer that question. Five recurring **lighthouse models** serve as diagnostic probes for the iron law throughout this book. These canonical workloads reappear across chapters to test how physical constraints affect distinct architectural patterns. At this stage, the task is to connect each workload to the resource that can limit it; later treatments explain the model structures and execution mechanisms behind those constraints.

Each lighthouse model illustrates a distinct stress case for the iron law under the workload conditions described here:

- **Image classification (ResNet-50)** probes compute throughput. With sufficient reuse of learned filters across image positions and batched inputs, the arithmetic term ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) can dominate execution time.

- **Text generation (GPT-2/Llama)** probes memory bandwidth, with GPT-2 anchoring the lighthouse and Llama representing its larger successors. Generating text token by token, the phase called decode, for one request at a time offers little weight reuse per token, so fetching weights from memory (D_{vol}/BW) can become the binding constraint.
- **Recommendation (DLRM)**, the Deep Learning Recommendation Model, probes memory capacity, a precondition for execution. Embedding tables store numerical vectors for user and item IDs. Large tables can reach about a hundred gigabytes in reference models, and terabytes in production, exceeding single-accelerator device memory.
- **Mobile vision (MobileNetV2)** probes end-to-end latency under a power cap. A phone-class vision model must meet a real-time deadline while sharing a battery and thermal budget with other applications. Computation, movement, and overhead can all contribute to that deadline.
- **Keyword spotting (KWS)** probes the power envelope: an always-on microcontroller must listen continuously for a wake word within a milliwatt budget and kilobytes of memory.

The iron law makes these differences actionable under the stated workload conditions. Raising peak compute R_{peak} helps convolutions when arithmetic dominates, while increasing memory bandwidth BW helps generation when weight transfers dominate. Batch size, reuse, and serving patterns can change that diagnosis, so the bottleneck belongs to the workload on a particular machine rather than to the model name alone. Table 1.4 summarizes the regimes these workloads help investigate.



Same iron law, opposite dominant term, opposite fix.

Table 1.4: Lighthouse Models as Reference Workloads: Each workload illustrates a resource constraint under the stated conditions, providing a concrete setting for investigating how system constraints affect architectural choices. Each model's architectural background appears in section 6.2, and its quantitative specifications in section 6.3.

Lighthouse Model	System Bottleneck	What It Reveals	Diagnostic Target
ResNet-50	Compute throughput under reuse	GPU utilization, batching	Arithmetic throughput versus memory stalls
GPT-2/Llama	Memory bandwidth	Weight and sequence-state movement	Weight and cached-context transfer rates
Deep Learning Recommendation Model (DLRM)	Memory capacity	Embedding tables, memory budgeting	Device memory capacity and partitioning
MobileNetV2	Latency under a power cap	Efficient operator design	Real-time latency deadlines within battery limits
Keyword spotting (KWS)	Power envelope	Tiny memory and energy budgets	Continuous inference within milliwatt power budgets

Tracking these workloads shows how architectural choices change a system's resource demands and cost. The lighthouse models supply the book's recurring workloads; the deployment paradigms described later in this chapter supply the operating envelopes those workloads must fit. The iron law estimates the execution time of one run on one machine. Over the past decade, however, training workloads grew faster than hardware improvements alone could accommodate, making efficiency a central concern.

1.8 Three Dimensions of ML Efficiency

The AlexNet run traced earlier shows what scale cost in 2012 and why that cost did not stay fixed. Read through the D·A·M taxonomy, its breakthrough was coordination across all three axes rather than algorithmic novelty in isolation: a convolutional architecture whose parallel matrix operations matched GPU capabilities, the 1.3M labeled images of the 2012 ImageNet challenge training split[21] (Deng et al. 2009), and two consumer GPUs running for about six days. A university laboratory could afford that budget. Modern foundation models demand training compute roughly 7 orders of magnitude larger.

That growth exposes a tension first introduced by the bitter lesson in section 1.4. Scale drives AI progress, but ever-larger datasets and compute budgets narrow participation to the most resource-rich organizations. Even those organizations eventually meet physical limits in data center power, memory bandwidth, and the diminishing returns of adding more parameters. Using existing compute more efficiently is therefore a complementary path to progress.

[21] **ImageNet**: The full image collection grew from 3.2 million images reported in the initial paper to about 14.2 million across thousands of labeled categories (Deng et al. 2009; Russakovsky et al. 2015). The 2012 challenge used a subset of this collection; AlexNet's training split contained about 1.3M labeled images. Distinguishing the full collection from a benchmark split matters when comparing dataset sizes and training costs.

Efficiency offers three complementary choices: perform less computation, execute that computation more effectively, or choose more informative examples. **Algorithmic efficiency** reduces computational requirements through model design and training procedures. **Compute efficiency** raises hardware utilization, turning more of the machine's theoretical capacity into useful work. **Data selection** identifies examples that contribute useful learning signal. It reduces the operations term O when training processes fewer examples; under a fixed computation budget, it can instead redirect work toward more informative examples. These choices can be combined, and none must wait for the others to be exhausted.

Choosing among them calls for a bottleneck diagnosis. The interfaces in figure 1.4 show where each choice acts. Data selection changes what the model learns from. Algorithmic efficiency lowers the operation count, while compute efficiency raises realized utilization η_{hw}. All three must account for information movement: reducing arithmetic alone may have little effect when the memory term D_{vol}/BW dominates.

Part III develops these dimensions in the order work flows through a system. Data selection asks which examples merit training computation. Model compression asks how much storage and arithmetic a trained model requires. Hardware acceleration explains how the remaining operations map onto memory and compute units. This teaching order does not prescribe an optimization sequence: the measured binding constraint and deployment budget determine where to intervene first, and hardware limits inform data and model choices from the outset.

The historical trajectory of computer vision models isolates the compounding effect of architectural design. Hernandez and Brown held accuracy fixed at the AlexNet level and asked how much training computation successive architectures needed to reach it (Hernandez and Brown 2020). Because they counted operations rather than seconds, hardware speed does not enter the measurement. Between AlexNet in 2012 and EfficientNet in 2019, the required compute fell by 44.5×, a halving roughly every 15 months (figure 1.5). The later points and the dashed exponential fit in the figure extend the series for context rather than as part of that measurement. The reduction measures algorithmic progress at fixed task quality. Comparing it with hardware progress would require a matching measure of useful throughput or energy efficiency; Moore's law describes transistor integration,[22] which alone does not determine either quantity.

[22] **Moore's law**: Gordon Moore's 1965 observation described rapid growth in the number of components that could be economically integrated on a chip (Moore 1998); later industry summaries often expressed the cadence as roughly a two-year doubling.

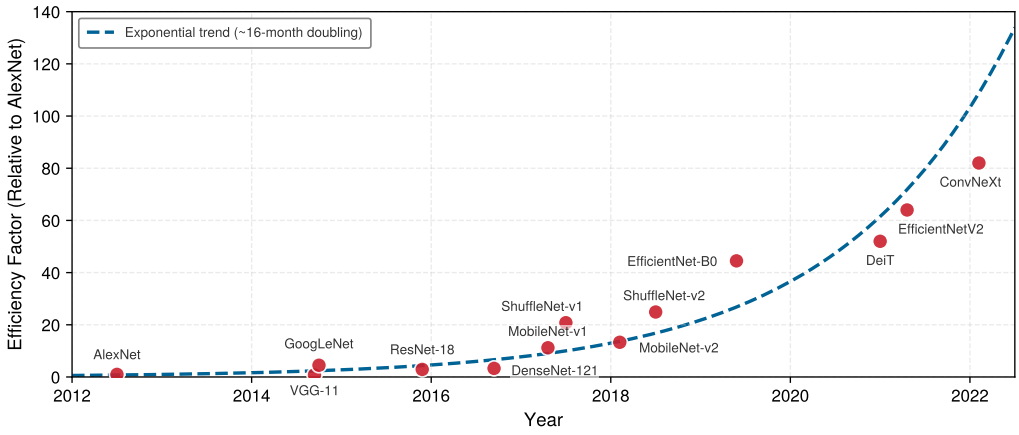


Figure 1.5: Algorithmic Efficiency Trajectory: Training efficiency factor relative to AlexNet (2012 baseline) for ImageNet classification. Most later architectures achieve comparable accuracy with fewer computational resources, although individual points vary. The trajectory from AlexNet (1×) through VGG (Visual Geometry Group), ResNet, MobileNet, and ShuffleNet to EfficientNet (44×) demonstrates a 44.5-fold reduction in required compute over seven years, independent of hardware improvements (Hernandez and Brown 2020). Early-2020s points and the dashed fit through all plotted points are author-added context rather than evidence for extending the Hernandez-Brown trend.

Architecture-by-architecture gains tell only half the story, because over the same years aggregate training compute in published frontier runs climbed far faster than algorithms could offset. Figure 1.6 charts that climb on a logarithmic scale, from early deep learning models near ^{18}$ FLOPs to frontier systems beyond ^{25}$ FLOPs. Its dashed reference line illustrates a roughly six-month

doubling; it is not fitted to the plotted points. Sevilla et al. (2022) measured a similar cadence across the deep learning era. An earlier study over 2012 through 2018 found a 3.4-month doubling (Amodei and Hernandez 2018b); different windows and sets of runs yield different estimates. Both studies found training compute growing faster than the roughly two-year doubling associated with Moore's law. This divergence motivates efficiency improvements across algorithms, software runtimes, and hardware architectures, helping keep useful systems within reach as workload demands grow.

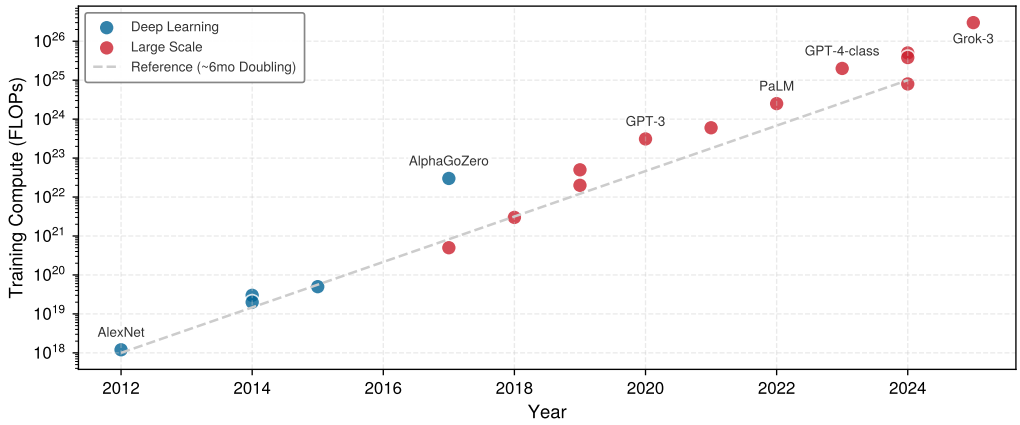


Figure 1.6: The Era of Scale: Illustrative training-compute estimates (FLOPs) vs. year on a log scale. Early deep learning models appear in blue; large-scale models appear in red. The dashed line is a reference for roughly six-month doubling, not a fit to these points. From AlexNet (2012) to illustrative GPT-4-class training scale anchors (2023), compute requirements increased by roughly 7 orders of magnitude (16.7M times). This endpoint comparison is distinct from training-compute trend studies (Amodei and Hernandez 2018b; Sevilla et al. 2022) and is used here to motivate the specialized infrastructure described in this book.

The two plots show how efficiency gains can coexist with growth in training scale. Figure 1.5 reports a 44.5× reduction in compute for a fixed accuracy target, while figure 1.6 illustrates roughly 7 orders of magnitude of growth in selected training runs. They measure different quantities and include contextual estimates, so they do not by themselves establish an economic cause. Efficiency makes a given capability less expensive; organizations can use the savings to reduce spending, broaden access, or pursue more demanding targets.

> Systems Perspective 1.2: The efficiency paradox
>
> **Jevons paradox** describes cases in which more efficient resource use leads to greater total consumption because demand expands. A similar response is possible in AI. If EfficientNet needs 44.5× less compute than AlexNet to reach the same accuracy, an organization can reinvest the savings in larger models or more experiments. It can also choose a smaller budget or serve more users at the same cost. Reinvestment is a possible consequence of efficiency, not an inevitable one, and the plotted trends alone do not establish it.

Efficiency choices must therefore be evaluated against the system's goals and deployment constraints. Managing predictive quality and physical execution together requires a discipline that connects them.

1.9 AI Engineering as a Discipline

A cloud service may optimize throughput, while an edge device must remain within a strict power envelope. The same model can therefore be efficient in one setting and unusable in another. Model accuracy alone does not specify a working system. Learned behavior must remain trustworthy as data changes, and the machine must deliver that behavior within its operating budget.

The degradation equation, iron law, and efficiency framework supply quantitative tools for this dual mandate. Applying them draws on computer science, electrical engineering, statistics, and

established software and systems engineering practice. Building a reliable production system requires combining their methods around a common set of requirements.

Definition 1.3: AI engineering

AI engineering is the discipline of designing, deploying, and maintaining ML systems whose predictive quality and execution meet stated deployment requirements. It integrates decisions about data, algorithms, and machines, using statistical evidence to assess learned behavior and measurements to assess resource use.

1. **Significance**: A model improvement must still satisfy the other system requirements. AI engineering jointly considers data quality and governance, prediction quality, execution time, memory, energy, and cost. A model that achieves 95 percent accuracy in research but violates a 100 ms deployment latency requirement does not meet the system specification.
2. **Distinction**: A model-level evaluation establishes evidence under a stated dataset and protocol. AI engineering must carry that evidence into a complete system whose latency, throughput, accuracy, cost, fairness, and robustness meet deployment requirements, and reassess it when operating conditions change.
3. **Common pitfall**: A frequent misconception is that AI engineering is just "software engineering for ML." Software correctness remains necessary, but the system specification also includes statistical claims about learned behavior over an operating population. Code tests cannot establish those claims by themselves, making continuing evaluation and monitoring part of the system design.

A requirement can be fixed while the evidence for meeting it remains uncertain. Pedestrian recall of at least 95 percent over a stated population is a fixed target; evaluation estimates recall from sampled outcomes, with uncertainty that must be reported and reassessed as conditions change. Execution requirements also need a defined workload and measurement protocol. Sampled response times support a statistical latency claim; a hard bound requires assumptions and analysis covering the specified execution conditions. Combining these obligations resembles the earlier convergence of computing and electrical engineering in computer engineering.[23] In this book's usage, AI engineering names the discipline and its shared principles, while ML systems engineering names the work of applying them to build and operate a specific system under its deployment constraints.

[23] **Computer engineering**: A discipline spanning digital circuits, processor design, and the software that runs on them. Moving a function from software into specialized hardware can change its speed, power use, and flexibility. ML systems engineering extends this shared design problem to include learned behavior and the data that shapes it.

1.9.1 Engineering across the ML lifecycle

Putting an algorithmic advance into use requires data ingestion and preprocessing, computation within the available machine budget, reliable serving, and evidence of real-world performance. These obligations form a recurring lifecycle rather than a sequence that ends at deployment. The engineering object is no longer only code; it is code, data, model behavior, deployment context, and monitoring evidence evolving together. Production feedback can force a deployed system back into data collection and training, bending the familiar linear arc into a cycle.

At the project level, six responsibilities organize this work: problem definition, data collection and preparation, model development and training, evaluation and validation, deployment and integration, and monitoring and maintenance. Problem definition establishes the task and acceptance requirements before collection begins; the finer-grained pipeline in figure 1.7 separates collection from preparation to expose their distinct feedback paths.

The structural difference shows up first in tooling. Decades of established practice support code-defined behavior through version control that maintains precise histories, continuous integration pipelines that automate testing, and static analysis tools that measure quality. For learned behavior, these practices must also track dataset and model versions and test predictive quality, because source-code changes alone do not describe changes to the deployed system.

Feedback must therefore assess learned behavior as well as software execution. Figure 1.7 formalizes this operational cycle across six staged components, distinguishing forward deployment from two critical feedback pathways: an inner validation loop that routes failed evaluations ("Needs

↪ The workflow controls that learned behavior adds to established software practice are developed in section 3.4.

Improvement") back to data preparation, and an outer operational loop that routes production telemetry ("Performance Degrades") back to data collection. When monitoring flags real-world performance degradation, engineers diagnose whether the evidence calls for pipeline repair, new data, or a model update. When an update is warranted, the cycle returns through preparation, training, and evaluation before redeployment. Feedback therefore determines which work must be repeated; it does not make every alert a retraining instruction.

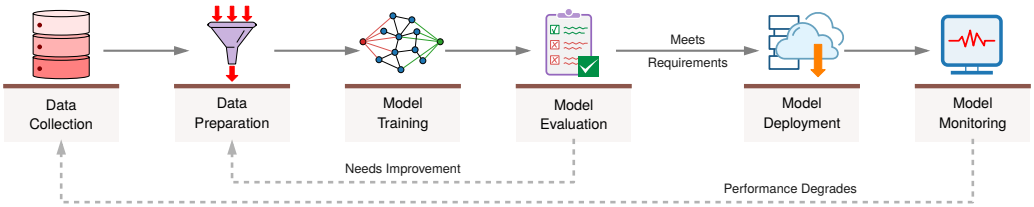


Figure 1.7: ML System Lifecycle: A six-box flowchart depicting Data Collection, Preparation, Model Training, Evaluation, Deployment, and Monitoring. Two feedback loops guide revisions to learned behavior: evaluation can return the process to preparation when results are insufficient, and monitoring can motivate new data collection when investigation identifies a need.

Because learned behavior can drift without a code change, source-code versioning must be complemented by dataset and model lineage, while unit tests must be complemented by statistical evaluation on relevant operating data. Supporting these additional artifacts and evidence requires specialized infrastructure. What each lifecycle stage demands, however, is not uniform; the binding physical constraints depend on where the system is deployed.

1.9.2 Deployment context shapes the lifecycle

Deployment context determines which lifecycle pressures dominate. The same stages apply across ML systems, but a megawatt-scale data center and a milliwatt-scale embedded device impose different bottlenecks on data collection, model updates, monitoring, and serving. Four **deployment paradigms** span that range, and table 1.5 shows how far apart their envelopes sit.

Table 1.5: Four Deployment Paradigms: Representative memory, compute, and power envelopes for cloud, edge, mobile, and TinyML systems, exposing the multi-order-of-magnitude span that prevents simple model reuse across tiers. Exact hardware twins and peak-rate calculations appear in the hardware chapters.

Paradigm	Representative System	Memory Envelope	Compute Envelope	Power Envelope
Cloud	Data-center accelerator node	Large device memory plus storage ($\approx 10^{11}$ B)	Highest-throughput tier ($\approx 10^{15}$ ops/s)	Facility-managed power
Edge	Robotics or industrial gateway	Local memory under deployment limits ($\approx 10^{11}$ B)	Local accelerator or CPU budget ($\approx 10^{14}$ ops/s)	Wall, vehicle, or site power
Mobile	Smartphone or wearable-class SoC	Shared application memory ($\approx 10^{10}$ B)	Phone-class neural, GPU, and CPU engines ($\approx 10^{13}$ ops/s)	Battery and thermal cap
TinyML	Microcontroller node	Kilobyte-scale memory ($\approx 10^{6}$ B)	Always-on sensor compute ($\approx 10^{9}$ ops/s)	Milliwatt-class battery budget

From cloud to TinyML, the endpoints differ by roughly 10^5 in memory and 10^6 in compute. A cloud model cannot be assumed to fit a microcontroller: compression, a different architecture, or a narrower task may be needed. The question here is how deployment reshapes the lifecycle. Cloud ML systems offer large computing budgets alongside capacity limits, operational complexity, and substantial costs. TinyML systems on microcontrollers[24] make memory and energy feasibility an immediate concern; a sensor may need to detect anomalies on one battery for years. Between these poles, placement determines where work and resource demands fall. Edge ML systems move computation toward data sources to reduce latency[25] and bandwidth demand. Mobile ML systems share memory, thermal headroom, and battery power with other applications. Hybrid systems distribute work across tiers to balance latency, privacy, bandwidth, and update control.

[24] **Microcontrollers**: Single-chip computers integrating a processor, memory, and interfaces to sensors or other devices. The TinyML ex-amples in this text use Arduino-class boards with 256 KB of RAM and 1 MB of flash storage, as well as Cortex-M4F microcontrollers.

[25] **Latency**: The time from an input arriving to the system producing a response. In a vehicle traveling at 30 m/s (about 67 mi/h), a delay of 100 ms means the car moves 3 m during each millisecond of delay. Sensing, communication, computation, and actuation all contribute to the time before braking can begin, so reducing network delay addresses only part of the response budget.

↪ The light barrier, power wall, and memory wall that shape these operating envelopes are developed in section 2.2.

A deployment decision affects the entire lifecycle. Tight response deadlines or unreliable connectivity can require local execution, as in vehicle control, while fraud detection can use cloud infrastructure when network and service latency fit the decision budget. Large language-model services often use centralized infrastructure to accommodate their resource demands. Cloud capacity and cost shape how often models are retrained, what historical data is retained, and how inference load is distributed. Edge and mobile resource limits constrain model complexity and update frequency, often motivating model compression[26] and careful scheduling.

Operational complexity grows as systems become more distributed. Edge and hybrid systems must coordinate data collection across sensors with varying connectivity, track models deployed across thousands of devices, push remote updates with rollback capabilities, and aggregate monitoring signals from geographically distributed endpoints. Data considerations pull in opposite directions. Privacy rules and data sovereignty push computation toward the edge, while the need for large-scale training data pulls it toward centralized aggregation. Production architectures therefore often run inference at the edge and route sampled telemetry to cloud clusters for retraining, so physical limits on one tier reshape every phase of the engineering lifecycle.

[26] **Model compression**: A family of techniques that reduces model storage or computation, including quantization (storing values in fewer bits), pruning (removing parameters that contribute little), and distillation (training a smaller model to imitate a larger one). Its size and accuracy effects depend on the model, method, workload, and target hardware.

1.9.3 Production systems expose shared challenges

A deployment case study becomes an engineering tool when it exposes the binding constraint behind a design. Three production systems at different points on the deployment spectrum show the same D·A·M pressures forcing different engineering responses:

- **Autonomous driving** binds on safety-critical latency and data drift. Perception runs on the vehicle while cloud infrastructure trains and evaluates new versions on collected driving data, so every remote update must be revalidated before it reaches a safety-critical fleet. The Waymo Open Dataset reports pronounced domain gaps among San Francisco, Phoenix, and Mountain View (Sun et al. 2020), the kind of regional shift that monitoring must catch before it becomes a failure.
- **FarmBeats** (Vasisht et al. 2017) binds on connectivity. Field sensors reach a local gateway PC over TV white-space links (unused broadcast-television spectrum), and the weak farm-to-cloud Internet link limits how fresh the cloud's data can be, so filtering and aggregation happen at the edge.
- **AlphaFold** (Jumper et al. 2021) binds on compute and curated data. Initial training used 128 TPUv3 cores for approximately one week, followed by about four days of fine-tuning, on the experimentally determined structures of the Protein Data Bank.

Deployment also creates the risk of a performance drop under distribution shift: a model scoring well on benchmark inputs can lose accuracy when deployment inputs differ (Koh et al. 2021). The drift that affects vehicle perception across cities can likewise affect crop monitoring across growing seasons. Fairness, transparency, and privacy add engineering requirements. Models can absorb societal biases present in their training data, so performance must be audited across relevant subgroups; stakeholders in domains like healthcare or finance require decisions they can interpret; and inference attacks[27] can expose sensitive information about training data. Addressing these requirements calls for coordinated work on data, models, infrastructure, and governance.

[27] **Inference attack**: An attempt to deduce sensitive information about training data from a model's outputs. Memorization can expose information, but the risk depends on the data, training procedure, and access available to the attacker. Defensive techniques include differential privacy and restrictions on the information returned by the system.

1.10 Book Organization

Production ML work divides into responsibilities that engineering teams own: data engineering, training systems, deployment infrastructure, operations and monitoring, and ethics and governance (Paleyes et al. 2022). Those boundaries leak. Upstream data pipelines limit what training can reach, training choices fix the memory and latency footprint at serving time, and governance constrains every stage, so the book does not follow an organization chart. Its four parts follow the order in which constraints must be understood. The organizing principle is *context before theory*: physical hardware limits and data ingestion bottlenecks constrain algorithmic choices, whose resource demands must in turn be checked against the machine. Readers establish operational boundaries and vocabulary (Part I) before constructing models (Part II), optimizing computational efficiency (Part III), and deploying systems under production service-level agreements (Part IV). Table 1.6 outlines this

four-part progression. Each part opens with the named principles it develops, numbered across the book so that later chapters can cite them.

Table 1.6: Book Organization: The four parts progress from foundations through model construction and training to optimization, deployment, and operations. This introduction precedes them and chapter 16 follows them. Each part builds on the vocabulary and methods of its predecessors.

Part	Theme	Key Chapters
I: Foundations of ML Systems	Constraints and system design	chapter 2, chapter 3, chapter 4
II: Development and Training	Model construction and training	chapter 5, chapter 6, chapter 7, chapter 8
III: Optimization and Acceleration	Efficiency: Performance tuning	chapter 9, chapter 10, chapter 11, chapter 12
IV: Deployment and Operations	Production: Real-world systems	chapter 13, chapter 14, chapter 15

Each part opens with the named principles it develops. These are numbered across the book so later chapters can refer back to them. Part IV, for example, gives the verification gap a statistical treatment over a stated deployment population (principle 9).

This introduction frames ML systems engineering through the bitter lesson, the D·A·M taxonomy, and the iron law. Part I establishes the physical and architectural vocabulary before detailed model mechanisms appear. Chapter 2 explains how physical limits shape the memory, compute, and power envelopes of the four deployment paradigms, so deployment requirements can inform model design. Chapter 3 connects those requirements to a lifecycle with explicit evaluation and feedback loops, showing why a constraint discovered late costs more than one stated early. Chapter 4 explains how to collect, validate, and deliver the data that the model will consume, making input quality and throughput part of system design.

Part II translates those environmental constraints into model construction skills. Chapter 5 explains how a neural model computes predictions and learns parameter updates, and counts the operations and bytes each step costs. Chapter 6 shows how arranging those computations changes both a model's capabilities and its arithmetic intensity, which decides where it sits relative to the ridge point. Both chapters reference the five lighthouse models introduced in section 1.7.2 (ResNet-50, GPT-2/Llama, DLRM, MobileNetV2, and keyword spotting) to anchor computational demands in standardized workloads. Chapter 7 traces how model descriptions become executable computations and where that translation spends fixed overhead. Chapter 8 combines those computations into training runs whose memory for activations, gradients, and optimizer state, whose duration, and whose numerical behavior can be measured and improved.

Part III alters the terms of the iron law while checking that task quality remains acceptable. Chapter 9 asks which examples justify their training cost, either reducing the examples processed or making better use of a fixed budget. Chapter 10 applies quantization, pruning, and distillation to reduce model storage and computation while checking quality and deployment feasibility. Chapter 11 explains why the memory hierarchy and specialized compute units, not peak FLOP/s alone, set the throughput a workload attains. Chapter 12 tests whether predicted improvements hold under a defined workload, with reproducible measurements of performance, energy, and quality.

Part IV deploys these optimized models into production environments, where incoming requests and changing data test the measured operating envelope. Chapter 13 turns a model into a service that meets per-request latency requirements, where overhead negligible across a weeks-long training run can dominate each request. Chapter 14 monitors drift and stages updates to detect regressions before broad deployment. Chapter 15 examines whether the system's objectives, subgroup outcomes, resource use, and governance meet its responsibilities. The closing chapter, chapter 16, returns to the questions posed in this introduction, bringing the book's methods together to assess a complete system and the challenges that arise beyond a single node.

This book focuses on the **single-node** regime of one host with one to eight accelerators, each typically using local device memory and communicating through an on-node interconnect such as PCIe or NVLink. The binding constraint depends on the workload and may be memory capacity,

memory bandwidth, compute throughput, or communication. Larger fleets add coordination and network constraints across many hosts. For reading paths, learning outcomes, prerequisites, and guidance on using the textbook, the preface provides the orientation.

The frameworks introduced in this chapter require engineers to evaluate the complete system. A model-quality metric alone cannot establish deployment feasibility, and a faster component cannot establish an end-to-end improvement. Applying either conclusion beyond the conditions that support it leads to fragile systems and wasted compute.

1.11 Fallacies and Pitfalls

A passing code test, a high held-out score, and a faster inference stage each provide useful evidence, but none alone establishes that an ML system meets its requirements. Mistaking code-level checks for validation of learned behavior overlooks the verification gap and silent degradation. Treating a held-out score as proof of deployment readiness overlooks distribution shift and execution budgets. Extrapolating a component speedup to the whole system ignores the unchanged costs captured by Amdahl's law. The fallacies and pitfalls below trace these errors in applying evidence beyond the conditions it supports, which waste engineering effort, delay deployments, and cause silent production failures.

Fallacy: *Better algorithms automatically produce better systems.*

A more accurate spam classifier can still be a worse production choice if it needs too much memory or cannot classify incoming messages within the delivery budget. The iron law (section 1.7) explains why algorithmic quality alone does not settle that decision. Model selection must maximize task quality subject to latency, memory, energy, cost, and reliability requirements. A simpler model supported by representative data and reliable infrastructure can meet those requirements more effectively than a sophisticated model whose surrounding system is inadequate.

Pitfall: *Treating ML systems as traditional software that happens to include a model.*

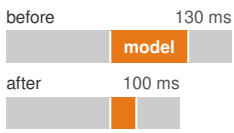
Established testing and deployment practices remain necessary for ML systems, but code-level checks alone cannot validate learned behavior (section 1.6). Both conventional and ML software can fail silently. ML systems additionally require outcome monitoring to detect declining predictive quality, with observation windows and sample sizes sufficient to resolve relevant differences across subpopulations. Teams deploying ML with only continuous integration and continuous deployment (CI/CD) pipelines risk silent failures: test suites and health checks report green while user-facing prediction quality degrades.

Fallacy: *High accuracy on benchmark datasets indicates production readiness.*

A benchmark score describes performance on a stated dataset under an evaluation protocol; it does not establish deployment readiness (section 1.9.2). A sentiment model may lose accuracy when users employ slang or context absent from the test set. Deployment also introduces resource requirements: mobile memory and latency limits may make a multi-model ensemble impractical, while network round trips can consume the response budget. Production validation must cover the intended operating conditions, relevant subgroups, resource use, and monitoring needed to detect change.

Pitfall: *Optimizing individual components without considering system interactions.*

Amdahl's law, met earlier as the iron law's companion, explains why improving inference alone may produce a small end-to-end gain (principle 8). Consider a team that reduces model inference from 45 ms to 15 ms, expecting proportional improvement. Preprocessing still consumes 60 ms and postprocessing adds 25 ms, so total latency drops only from 130 ms to 100 ms. That is a 23 percent improvement rather than the expected 67 percent. The time breakdown explains the result: most of the path was untouched. Measuring the whole request prevents a component speedup from being mistaken for a system speedup.



Making the model three times faster trims end-to-end latency by only 23 percent.

Fallacy: *ML systems can be deployed once and left to run indefinitely.*

Deployed ML systems cannot assume static performance over time: environmental distribution shifts can degrade a frozen model even when underlying execution code remains untouched. In this illustrative scenario, a recommendation system deployed at 85 percent accuracy drops to 80.2

percent within 6 months as purchasing patterns shift, losing 4.8 percentage points without any code changes. The ML lifecycle (section 1.9.1) therefore treats outcome monitoring and evidence-based retraining as operational requirements. Fraud detection and natural language processing systems face the same risk as user behavior evolves, adversarial patterns emerge, and vocabulary drifts while the serving binary remains unchanged. Without continuous monitoring, systems can appear healthy while prediction quality silently erodes. Organizations that treat deployment as a one-time handoff may discover failures only after customer complaints or degraded business metrics force an emergency audit.

Pitfall: *Assuming that ML expertise alone is sufficient for ML systems engineering.*

Production ML (section 1.9.3) requires expertise across algorithms, software, systems, and operations. A team with strong ML skills can still miss throughput targets if storage layout or serving infrastructure is inadequate. Conversely, infrastructure built without attention to model inputs can introduce preprocessing defects that degrade predictions. Deployment case studies show the need for coordinated attention to data, models, infrastructure, and organizational workflow (Paleyes et al. 2022). Effective teams bring these skills together rather than expecting one role to master them all. Failures can arise at the boundaries, where one team's assumptions become another team's inputs.

1.12 Summary

Machine learning systems must satisfy two obligations at once: learned behavior must remain trustworthy, and the machine must deliver that behavior within physical and economic limits. This chapter established three questions for examining a system: what shapes its predictions, what limits its execution, and what evidence supports its deployment. The D·A·M taxonomy organizes the diagnosis. The degradation equation and iron law help quantify it, while efficiency and return on compute guide the choice of interventions.

The lifecycle and deployment examples show why those questions must be revisited as data and operating conditions change. Five lighthouse models (ResNet-50, GPT-2/Llama, DLRM, MobileNetV2, and keyword spotting) recur throughout the book to apply these methods to concrete workloads.

Return to the smartphone interaction that opened the chapter. What appeared to be one intelligent action depended on representative data, a learned model, a machine within its operating budget, and feedback that could reveal changing behavior. That chain answers the question posed at the outset: an accurate model alone does not establish a reliable, efficient system. AI engineering coordinates these decisions and uses measured evidence to assess whether the complete system meets its requirements.

Key Takeaways: Constraints drive architecture

- **Relieving a bottleneck can expose another**: Data, Algorithm, and Machine constraints interact. After an intervention, measure the complete system again to determine which constraint now limits the result.
- **Learned behavior can decay silently**: ML systems can lose predictive quality while code and infrastructure stay fixed because deployment inputs change. Drift metrics prompt investigation; labeled outcomes establish whether prediction quality has fallen.
- **End-to-end time is the whole path**: Data movement, computation, and overhead all spend from the same time budget. Cutting inference from 45 ms to 15 ms gives only 23 percent improvement when preprocessing (60 ms) and postprocessing (25 ms) dominate, so optimize the term that binds end-to-end behavior.
- **Scale needs useful computation**: The bitter lesson emphasizes search and learning methods that can use increasing computation effectively. Their benefit depends on data, objectives, and a supporting machine. Efficiency gains of 44.5× at a fixed accuracy target coexisted with roughly 7 orders of growth in selected training runs.

- **AI engineering is continuous co-design**: Deployment requirements guide decisions about data, models, and execution. Evaluation and lifecycle monitoring provide evidence that predictive quality and resource use continue to meet those requirements.

An intended capability becomes an engineering result only when measured behavior meets the goal within a feasible resource budget. Data, learning objectives, design choices, and hardware all shape that result. The methods introduced here provide a starting vocabulary for examining them together; the remaining chapters develop the tools needed to make and test those decisions.

What's Next: From vision to architecture

Why does the same model demand different engineering across deployment settings? This chapter introduced the four deployment paradigms (cloud, edge, mobile, and TinyML) and showed that their operating envelopes sit orders of magnitude apart. Physical limits help explain the difference. A distant cloud round trip can exceed an emergency-braking deadline. Thermodynamics caps how much computation a passively cooled phone can sustain. Memory-bandwidth limits constrain performance in ways that faster arithmetic alone cannot remove. Chapter 2 explains how those constraints shape each paradigm's operating envelope and develops a decision framework for choosing among them when requirements conflict.

I

FOUNDATIONS OF ML SYSTEMS

Part I

Foundation Principles

A deceptively simple heuristic runs through machine learning systems. Complexity often moves rather than disappearing, although good design can remove accidental complexity. Complexity flows among the three domains of the D·A·M taxonomy: Data as information, Algorithm as logic, and Machine as physics. Simplifying one domain can burden the others. A hand-crafted feature pipeline reduces algorithmic complexity but demands more data engineering effort. A larger model absorbs messy data but shifts complexity onto the hardware that must train and serve it. This conservation-of-complexity heuristic recurs throughout this book. The quantitative bounds, models, and principles introduced throughout the book describe constraints that emerge from where complexity currently resides.

Architectures, frameworks, and optimizations succeed only when they respect constraints imposed by hardware, mathematics, and information theory. Just as civil engineers cannot ignore gravity, ML engineers cannot ignore the physical laws that govern data, computation, and system throughput. The introduction argued, through the bitter lesson, that progress comes from general methods that scale with data and computation, and it named the iron law that prices that scale. Part I establishes the foundations on which that pricing rests: where a system can physically run, when its constraints must be known, and how the data it learns from is engineered. The principles in this book are not best practices that evolve with frameworks or opinions that differ between teams, and they are not universal laws either. Each principle box opens by naming its kind (bound, decomposition, fitted model, policy, or heuristic), and each holds only under the assumptions it states. Part I's two principles start with data itself, where the familiar boundary between program and input begins to disappear.



Principle 1: The data-as-code principle

Heuristic: Data functions as the source code of the ML system. A change to the training dataset can alter the system's learned behavior. Here, learned behavior means the trained input-output mapping; Train denotes the training process, and $\approx$ marks a qualitative dependency rather than a predictive equality. Model architecture, initialization, optimizer configuration, and training stochasticity are included in the training procedure.

$$\text{Learned Behavior} \approx \text{Train}(\text{Training Data}, \text{Training Procedure})$$

Implication: Data engineering requires the same rigor as software engineering. Datasets should be versioned (like Git), unit-tested (data quality checks), and debugged. Deleting a row of training data can alter the rebuilt model just as deleting a line of code can alter a compiled artifact.

If data functions as source code, then it is not merely a logical artifact; it also has physical properties that constrain system architecture when data and computation occupy different locations.

Principle 2: The data-gravity principle

Heuristic: As data volume (D_{vol}) grows, transferring the dataset can cost more than sending a much smaller query or code payload over the same constrained link (McCrory 2010). Here, cost denotes one transfer metric at a time, such as elapsed transfer time or bytes sent, under identical link conditions; it does not combine latency and bandwidth into one quantity.

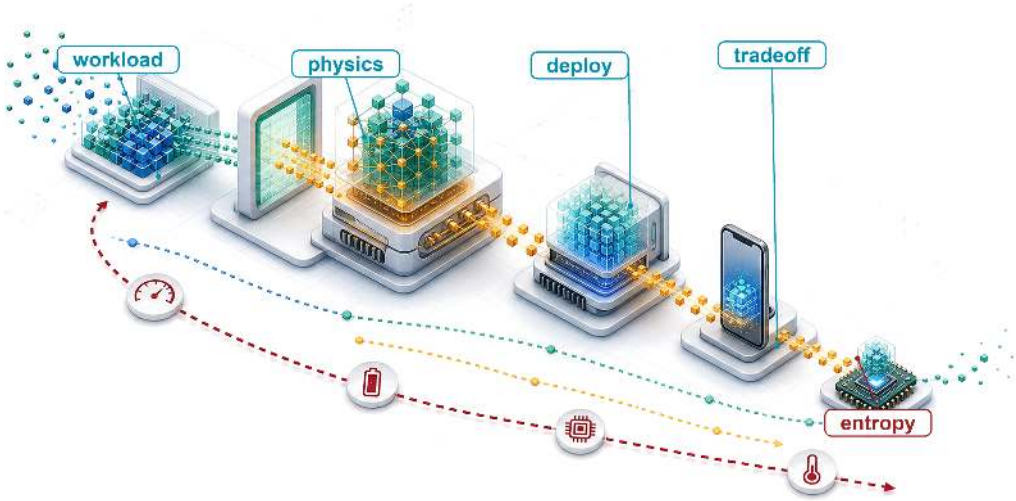
$$\text{Cost to transfer dataset} \gg \text{Cost to transfer query or code} \quad \text{in this regime}$$

Implication: Large datasets can become the gravitational center of the architecture. Systems may move compute to data by shipping queries or code to the storage layer rather than moving data to compute by repeatedly downloading large datasets.

Together, these two principles establish that data can be both the logical program and a physical anchor of an ML system. Part I develops these implications as a dependency chain. Chapter 2 derives from physical limits where a system can execute; chapter 3 carries those constraints through problem definition, data, modeling, validation, and operation; and chapter 4 makes them enforceable in versioned, tested pipeline artifacts. Ignoring any link returns complexity as integration cost: an accurate model that cannot execute, a workflow that cannot reproduce its inputs, or a pipeline whose outputs no longer match deployment conditions.

2

System Paradigms

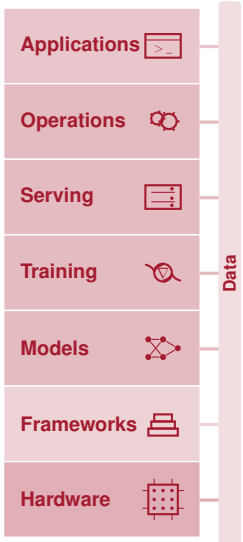


2.1 Deployment Paradigm Framework
2.2 Physical Constraints: Why Paradigms Exist
2.3 Analyzing Workloads
2.4 System Balance and Hardware
2.5 Cloud ML: Computational Power
2.6 Edge ML: Latency and Privacy
2.7 Mobile ML: Offline Intelligence
2.8 TinyML: Ubiquitous Sensing
2.9 Paradigm Selection
2.10 Hybrid Architectures
2.11 System Entropy: Why Deployment Is Not the End
2.12 Fallacies and Pitfalls
2.13 Summary

Purpose

Why does deploying the same model to a phone vs. a data center demand fundamentally different engineering?

The defining insight of ML systems engineering is that constraints drive architecture. The speed of light sets an absolute floor on how quickly distant servers can respond. Thermodynamics limits how much computation can occur in a given volume before heat becomes unmanageable. Memory physics makes moving data often more expensive than processing it. These physical limits persist as technology improves, although the resource envelopes of particular platforms change. Deployment categories organize recurring constraint regimes rather than divide hardware into fixed, mutually exclusive classes. A data center can train billion-parameter models but cannot guarantee low-latency responses to users thousands of miles away. A smartphone can respond instantly but has a fraction of the memory budget and must share a small battery with everything else the phone does. A microcontroller can run on a coin-cell battery for years but has barely enough compute for a simple keyword detector. Across these regimes, power budget spans roughly nine orders of magnitude, from the megawatts of a data center to the milliwatts of a microcontroller, and memory capacity spans from terabytes to kilobytes. The same model, the same algorithm applied to the same data, demands radically different engineering in each regime, not because the physics changes but because different constraints dominate. Teams that treat deployment as an afterthought, training in one environment and deferring target constraints until launch, discover too late that the target environment invalidates months of architectural decisions. The engineering task is therefore not to identify a universally best deployment but to locate the constraint that binds in the intended operating regime. In D·A·M terms, understanding these regimes transforms deployment from an



operational detail into a first-order co-design problem in which data locality, algorithm structure, and machine constraints jointly determine what is possible.

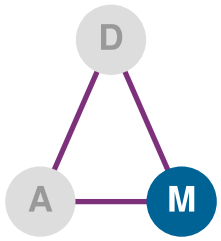
Learning Objectives

- Explain how physical constraints shape deployment paradigms from cloud to TinyML
- Apply the iron law and bottleneck principle to diagnose execution limits and distinguish them from memory and energy feasibility
- Match workload demands to deployment budgets using lighthouse model examples
- Compare cloud, edge, mobile, and TinyML by operational constraints and quantitative trade-offs
- Apply the decision framework to select paradigms by privacy, latency, resource, and cost constraints
- Analyze hybrid patterns that combine paradigms to satisfy system constraints
- Evaluate deployment decisions for system-level gains, lifecycle cost, and the need to monitor prediction quality

2.1 Deployment Paradigm Framework

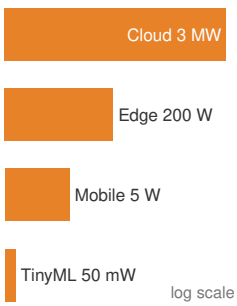
Consider two extremes: a wake-word detector on a smartwatch and a recommendation engine in a data center. The wake-word detector represents a TinyML workload operating under milliwatt power budgets and kilobyte memory limits; the recommendation engine exemplifies a cloud ML workload requiring terabytes of embedding tables and megawatt-scale infrastructure. These systems solve different problems under opposite physical constraints, and their supporting infrastructure differs sharply in scale and design. The introduction's D·A·M taxonomy (figure 1.4) treated the machine as one of three interacting components. Between these two extremes, the machine is the component that changes most, and its physical limits decide which data and algorithmic choices remain open.

Physical constraints determine where an ML model can run and shape what is possible in ways no algorithmic choice can override. The engineering challenge extends far beyond parameter weights: in production systems, the model code represents only a small fraction of the total operational codebase (Sculley et al. 2015). The surrounding infrastructure (data ingestion pipelines, feature extraction, serving runtimes, telemetry, and hardware resource managers) must conform directly to the host platform's physical envelope.



The machine axis sets the deployment envelope.

Three physical constraints (latency, power dissipation, and memory) organize four useful deployment paradigms. The categories overlap: edge describes proximity to data, while mobile and TinyML further distinguish local devices by their resource envelopes. A workload and its target platform determine which constraint actually binds. Centralizing computation in Cloud ML provides elastic compute and large memory capacity, but every inference incurs network latency over wide-area links. Moving workloads to Edge ML eliminates that network round trip and keeps sensitive data on premises (Shi et al. 2016), but limits available compute to local rack power and cooling budgets. On battery-powered devices, Mobile ML delivers responsive interaction directly to handsets and tablets, but must operate within a passive thermal envelope where sustained computation triggers throttling. At the low-power extreme, TinyML runs on microcontrollers (Janapa Reddi, Plancher, et al. 2022), where milliwatt power budgets and kilobyte-scale memory require compact models and integer arithmetic. The memory constraint changes form along the way: in the data center it usually binds as bandwidth, whereas on a microcontroller it binds first as capacity. Figure 2.1 arranges these deployment tiers along a physical continuum spanning nine orders of magnitude in operating power (from warehouse-scale megawatts in data centers down to milliwatts on embedded microcontrollers), isolating the fundamental trade-off between centralized computational scale and local execution autonomy. Whereas high-level industry taxonomies frequently treat everything outside the data center as a single catch-all edge tier, systems engineering partitions non-cloud deployment into three distinct regimes: Mobile ML on personal, battery-powered user devices; Edge ML on local gateways and on-premises servers; and TinyML on deeply embedded, ultra-low-power microcontrollers.



Power falls from megawatts in the cloud to milliwatts in TinyML.

While figure 2.1 diagrams the continuous topological progression from embedded sensors to data centers, table 2.1 quantifies conceptual operating envelopes across processing location, round-trip

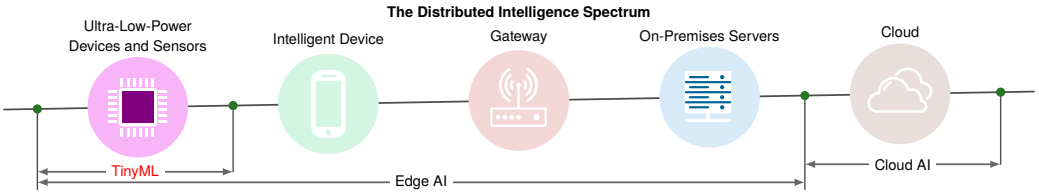


Figure 2.1: Distributed Intelligence Spectrum: Deployment tiers form a continuum governed by physical constraints, spanning nine orders of magnitude in power. Moving from centralized cloud infrastructure to on-device TinyML trades raw computational capacity and elastic scale for shorter communication paths, data locality, and network independence.

latency, power draw, and memory capacity. The cloud row describes facility power; the other rows describe local systems or devices. These envelopes compare deployment scales, whose physical basis is developed in section 2.2.

Table 2.1: The Deployment Spectrum (Conceptual): Four paradigms span nine orders of magnitude in power (MW to mW) and memory (TB to KB). This conceptual overview defines each paradigm by its operating regime, and each power entry names a typical class rather than a bound; the concrete hardware spectrum in table 2.4 grounds these categories in specific platforms and quantitative decision thresholds. The hardware specifications and physical constants underpinning these numbers are catalogued in Appendix E.

Paradigm	Where	Latency (ms)	Power	Memory	Best For
Cloud ML	Data centers	100-500	MW	TB	Training, complex inference
Edge ML	Local servers	10-100	100 W	GB	Real-time inference, privacy
Mobile ML	Smartphones	5-50	3–5 W	GB	Personal AI, offline
TinyML	Microcontrollers	1-10	mW	KB	Always-on sensing

Within one host, the single-node stack connects four layers: the application sets the task and its latency or throughput target; the ML framework translates model code into operations; the operating system and runtime coordinate resources; and the hardware executes the work. The hardware's **silicon contract** names its compute rate, memory bandwidth, and execution overhead. Different tiers offer different budgets for these resources.

The physical constraints examined next govern this stack through the iron law of ML systems, which decomposes end-to-end execution time into data movement, computation, and system overhead. Understanding how physical limits—the speed of light, thermal dissipation, and memory signaling—shape these execution terms explains why deployment paradigms exist as distinct engineering regimes rather than arbitrary points on a continuous slider.

2.2 Physical Constraints: Why Paradigms Exist

A safety system with a 10 ms reaction budget cannot wait for a cross-country round trip, and a billion-parameter model cannot be squeezed into a microcontroller by better code alone. These are not implementation bugs; they are direct consequences of the physical laws governing the speed of light, power thermodynamics, and memory signaling. Each physical law imposes an absolute architectural boundary: the light barrier limits remote responsiveness, the power wall restricts sustained throughput, and the memory wall caps data transfer. Where a system runs changes the operational terms the hardware offers, reshaping the silicon contract between model and machine.[1]

[1] **Deployment paradigm**: A distinct operating regime defined by latency, power, memory, and connectivity constraints. The Cloud-to-TinyML spectrum changes which resources bind; optimizing within an infeasible regime cannot satisfy the application's deployment budget. Raising a cloud model's throughput by 5 percent, for example, cannot rescue an application whose 10 ms latency budget the network round trip alone exceeds.

↩ A first sketch of the deployment spectrum and its three walls is established in section 1.9.2.

The light barrier

The **light barrier** establishes the absolute latency[2] floor. Let Distance denote the one-way path length and c_{fiber} the propagation speed in optical fiber. Equation 2.1 gives the minimum round-trip time:

$$L_{\text{lat,min}} = \frac{2 \times \text{Distance}}{c_{\text{fiber}}} \approx \frac{2 \times \text{Distance}}{200{,}000 \text{ km/s}} \tag{2.1}$$

For optical fiber, $c_{\text{fiber}} \approx 200{,}000$ km/s, roughly two-thirds of its vacuum value because light propagates more slowly through glass than through a vacuum.

California to Virginia (~3,600 km straight-line) requires ~36 ms round-trip before any computation begins. Actual services add routing, protocol, queuing, and processing latency. Applications requiring sub-10 ms response *cannot* use distant cloud infrastructure, because physics forbids it. This constraint creates the need for edge ML and TinyML, since a tight latency budget forces computation to move closer to the data source.

The power wall

The **power wall** emerged because thermodynamics limits how much computation can occur in a given volume. Dynamic CMOS power follows equation 2.2, where C is capacitance, V is supply voltage, and f is clock frequency. Higher clock frequencies often require higher supply voltage to preserve reliable switching. Under the illustrative assumption that voltage rises proportionally with frequency ($V \propto f$), dynamic power rises cubically (Power $\propto f^3$). The exact voltage-frequency relationship varies by device, but increasing both consumes thermal headroom rapidly.

$$\text{Power} \propto C \times V^2 \times f \quad \text{where } V \propto f \implies \text{Power} \propto f^3 \tag{2.2}$$

Under this illustrative assumption, doubling clock frequency requires approximately 8× more dynamic power. The breakdown of broad voltage scaling[3] ended "free" frequency speedups and forced the industry toward parallelism (multi-core) and specialization (GPUs, Tensor Processing Units (TPUs)). Mobile devices hit hard thermal limits at 3–5 W; exceeding this causes "throttling," where the device reduces performance to prevent overheating. In practice, this means a mobile model that runs at 60 FPS for 1 minute may throttle to 15 FPS as the device heats up. This physical limit gives rise to mobile ML: battery-powered devices cannot simply run cloud-scale models locally.

The memory wall

The **memory wall** (Wulf and McKee 1995) reflects the widening bandwidth[4] gap. A simple book-level sketch uses representative annual growth factors to show why the gap compounds:

$$\frac{\text{Compute Growth Rate}}{\text{Memory Bandwidth Growth Rate}} \approx \frac{1.6}{1.2} \approx 1.33 \tag{2.3}$$

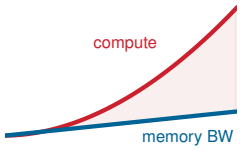
In equation 2.3, the numerator and denominator are dimensionless annual growth factors. A ratio above 1 means compute capability is pulling away from memory bandwidth, so each hardware generation makes data movement a larger share of the performance problem unless the workload increases locality or arithmetic intensity (operations performed per byte moved).

For this illustrative trend comparison, assume compute capacity doubles every 18 months, about 1.6× per year (the numerator of equation 2.3), while memory bandwidth grows by ~20 percent annually (the denominator). These assumed rates illustrate the widening gap rather than a forecast for every hardware generation. Sustained for a decade, those rates leave compute about 16× further ahead of bandwidth than at the start. This widening gap makes data movement a frequent bottleneck and energy cost in ML workloads. An on-chip arithmetic logic unit (ALU) operates over short local connections, whereas an off-chip DRAM access also drives memory interfaces and longer interconnects. In representative hardware, fetching operands from DRAM can cost orders of magnitude more energy than the arithmetic performed on them (Horowitz 2014). The ratio depends on the operation, numerical precision, memory technology, and amount of data transferred. The constraint affects all paradigms, and it takes a different form on TinyML devices, where capacity, not bandwidth, is usually exhausted first.

2 | **Latency**: The time between issuing a request and receiving a result, corresponding to T in the iron law, where propagation delay enters through the fixed-overhead term L_{lat}. The light barrier makes that propagation floor irreducible, as equation 2.1 quantifies. Every millisecond consumed by distance is a millisecond unavailable for model inference, which is why the light barrier forces paradigm selection rather than mere optimization.

3 | **Dennard scaling**: Named after Dennard et al. (1974) at IBM, who described MOSFET scaling relationships under which shrinking devices could reduce voltage and current while keeping power density approximately controlled. As voltage scaling slowed and energy became a first-order architectural constraint, performance growth shifted toward parallelism and specialization: multi-core processors, GPUs, and TPUs (Hennessy and Patterson 2019; Esmaeilzadeh et al. 2011).

4 | **Memory bandwidth (the memory wall)**: The term "memory wall" was coined by Wulf and McKee in 1995, who predicted that the processor-memory performance gap would eventually dominate system performance—a prediction that proved prescient for ML workloads in which weight or activation movement can bind performance. In the iron law, bandwidth (BW) appears in the denominator of the data term D_{vol}/BW, so doubling the bytes moved without a corresponding bandwidth increase raises the data-movement time. The resulting bottleneck depends on arithmetic intensity, batch size, data reuse, access pattern, and hardware balance rather than model size alone.



Compute capacity outruns memory bandwidth; the widening gap is the memory wall.

↪ The architectural responses to the memory wall, high-bandwidth memory (HBM) and on-chip SRAM hierarchies, are developed in section 11.5.4.

Checkpoint 2.1: Physical constraints and deployment

Deployment choices are governed by physics, not just preference. Check your understanding:

- ☐ **Light barrier**: Can you use the cross-country propagation bound to explain why that distant cloud path cannot meet a < 10 ms safety deadline?
- ☐ **Power wall**: Can you explain why a workload that exceeds a phone's sustained heat-dissipation budget cannot maintain its initial performance?
- ☐ **Memory wall**: Can you explain why data movement is often more expensive (in time and energy) than computation?

These physical laws explain the trade-offs behind the four paradigms. Physics bounds feasible execution; privacy regulation, economic incentives, and data sovereignty requirements further restrict the choices within those bounds. No regulation can make the speed of light faster, and no economic model can repeal thermodynamics.

These physical constraints do not operate in isolation. In practice, an ML system is limited by whichever physical boundary binds on its primary resource first. Evaluating these limits quantitatively requires analytical tools: the iron law to decompose execution time into its components, the bottleneck principle to identify the rate-limiting stage, and workload archetypes to classify the model's resource demands.

2.3 Analyzing Workloads

Given a recommendation engine or wake-word detector, the workload question is which term will bind first on the target hardware. The central analytical tool for answering that silicon-contract question is the iron law of ML systems from section 1.7, restated here in the same notation because every tier in this chapter is analyzed through it. For a serialized execution path, equation 2.4 adds three costs:

$$T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \tag{2.4}$$

Equation 2.4 decomposes total latency into three terms: data movement $(D_{\text{vol}}/\text{BW})$, compute $(O/(R_{\text{peak}} \cdot \eta_{\text{hw}}))$, and fixed overhead (L_{lat}). For an inference path that executes these stages sequentially, their times add. A single request can also contain overlapping work, so its latency must follow the actual dependencies. In production systems, however, tasks are processed continuously as a stream, and the analysis shifts from single-task latency to identifying which term limits the system. The answer depends on the workload, its execution conditions, and the deployment hardware. A workload is **compute bound** when its compute term $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ exceeds its data term D_{vol}/BW, and memory bound in the reverse case. A model that is compute bound during training may become memory bound during inference; a system that runs efficiently in the cloud may hit power limits on mobile devices. Batch size is one lever that moves a workload between these regimes. Larger batches reuse each loaded weight across more inputs, shifting a memory-bound kernel toward compute-bound behavior, at the price of a larger working set and a longer service time per batch. In pipelined production serving, hardware concurrency alters this linear sum: faster stages hide behind whichever subsystem saturates first.

2.3.1 The bottleneck principle

The iron law quantifies the cost of each term; the **bottleneck principle** identifies which term governs system throughput. In a pipelined ML workload, identifying the system bottleneck matters because speeding a nonlimiting stage does not increase ideal steady-state throughput while the slowest stage remains unchanged. Modern accelerators use **pipelined execution** to overlap data movement with computation: while the accelerator computes on batch k, the memory system prefetches batch $k+1$. Once the pipeline fills, different requests occupy different stages. Under ideal overlap, the slowest stage sets the interval between completions, although each request still traverses its dependent stages. Equation 2.5 extends the pipelined form from section 1.7 with one more overlapped stage, T_{IO}, the time to transfer data over a storage or network link. Here the left-hand side is the steady-state time

per completed task, with fixed overhead treated as nonoverlapped; equality requires ideal overlap of the other stages:

$$T_{\text{pipelined}} \geq \max\left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}, T_{\text{IO}}\right) + L_{\text{lat}} \tag{2.5}$$

- $\frac{D_{\text{vol}}}{\text{BW}}$ **(Memory)**: Time to move data between memory and processor.
- $\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$ **(Compute)**: Time to execute calculations.
- T_{IO} **(I/O)**: Time to move data over a storage or network link (including offloading to a remote server).
- L_{lat} **(Overhead)**: Fixed latency (kernel launch, runtime overhead).

When a system is memory bound ($D_{\text{vol}}/\text{BW} > O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), faster processors ($R_{\text{peak}}$) do not raise ideal pipeline throughput while memory remains the slowest stage, just as widening a highway cannot help traffic constrained by a narrow bridge. Shortening a sequential compute stage can still reduce an individual request's latency. Engineers must identify the dominant term before optimizing. The iron law quantifies the *cost of each ingredient*; the bottleneck principle identifies the *speed of the assembly line*. As a rule of thumb, use the *additive form* in equation 2.4 for a single task's serialized path, and the *max form* in equation 2.5 for an ideally overlapped continuous stream. Actual dependencies determine which stages can overlap.

2.3.2 Workload archetypes

The bottleneck principle reduces optimization to identifying the dominant constraint for a given workload. The answer depends on all three D·A·M axes at once: the data a request carries, the algorithm's arithmetic and memory demands, and the machine that executes it. Different deployment environments create different bottlenecks along these axes, so the same model family can require different engineering when it moves from a cloud server with terabytes of memory to a microcontroller with kilobytes. Classifying workloads by what binds them first yields four **workload archetypes**.[5] These are not model categories; they are recurring physical bottlenecks that determine which engineering moves can help. Two of them are named by the iron law's own terms. The other two are bound by preconditions the time equation assumes are already met, that the working set fits in memory and that the energy budget can pay for the run, which is why a capacity or energy limit can decide feasibility before any term of equation 2.4 is evaluated.

The first split, between the two iron-law terms, separates arithmetic-bound systems from data-movement-bound systems. A **compute beast** performs many calculations per byte loaded, so progress comes from higher arithmetic throughput, better utilization, and more parallel execution; large neural-network training is the canonical case. A **bandwidth hog**, by contrast, waits on dense weight or activation movement, so wider memory buses and better data reuse matter more than additional peak FLOP/s; autoregressive text generation illustrates this regime.

The second split covers workloads where the binding constraint is not a term of the time equation at all. **Sparse scatter** workloads are dominated by irregular table lookups and poor cache locality, so memory capacity, access latency, and communication shape performance in recommendation systems with massive embedding tables. **Tiny constraint** workloads face the opposite envelope: energy per inference and memory footprint, not raw speed, determine whether always-on sensing can run at all.



Lighthouse 2.1: Five reference workloads

Section 1.7.2 introduced five lighthouse models by the system bottleneck each isolates (table 1.4); table 2.2 pairs each with a representative workload archetype and deployment paradigm. The classifications describe operating regimes: batched image processing can emphasize arithmetic, single-request text generation can emphasize weight transfers, and a large recommendation table can exceed local memory capacity. Changing batch size, model size, or hardware can change the binding constraint. The two perspectives on MobileNetV2 align under this principle. The introduction classified MobileNetV2 under latency and power because those physical limits

[5] **Workload archetype**: A classification of ML workloads by their dominant bottleneck, either a term of the iron law or a memory-capacity or energy precondition it assumes is met, rather than by their model family. The distinction matters because the optimization strategy differs fundamentally: a compute-bound workload benefits from faster arithmetic (R_{peak}), while a bandwidth-bound workload benefits from wider memory buses or fewer bytes moved through reuse, fusion, compression, or lower precision. Misidentifying the archetype wastes optimization effort on the wrong term of the iron law, as when teams add accelerator FLOP/s to a memory-bound inference pipeline and obtain little improvement while memory transfer remains dominant.

defined its design envelope. In table 2.2, it illustrates an efficient vision workload operating in a compute-bound regime when its convolutional stages sustain arithmetic saturation, but with operations sized down to fit mobile thermal and energy envelopes. Whether arithmetic throughput or memory bandwidth ultimately binds MobileNetV2 on a specific device depends on target hardware balance, precision, and batching, as the roofline analyses in section 2.4 demonstrate. Section 6.2 provides full architectural details and model biographies.

Table 2.2: Five Lighthouse Models: Recurring workloads used throughout the book to ground the iron law in concrete practice. Each lighthouse pairs an archetype (Compute Beast, Bandwidth Hog, Sparse Scatter, Tiny Constraint) with the deployment paradigm where it predominantly runs, illustrating a distinct resource constraint under representative execution conditions.

Lighthouse	Archetype	Deployment Paradigm
ResNet-50	Compute Beast	Cloud training, edge inference
GPT-2/Llama	Bandwidth Hog	Cloud inference
DLRM	Sparse Scatter	Cloud for large tables
MobileNetV2	Compute Beast (efficient regime)	Mobile, edge
Keyword Spotting (KWS)	Tiny Constraint	TinyML, always-on

These archetypes map onto deployment paradigms. Large-scale compute beasts (training) and sparse scatter workloads gravitate to cloud data centers; compact vision workloads operating in efficient compute-bound regimes such as MobileNetV2 can fit mobile and edge devices; bandwidth hogs run in the cloud unless a smaller model fits locally; and tiny constraint workloads belong on microcontrollers.

The archetypes become concrete once each lighthouse is given its cost. The image classifier **ResNet-50** spends approximately 8.2 GFLOP per image on 25.6 million parameters, which occupy 102.4 MB at FP32 (He et al. 2016a). That regular, compute-dense structure makes it the canonical benchmark for hardware accelerator performance.

The language models **GPT-2/Llama** exemplify autoregressive text generation (Radford et al. 2019; Touvron, Lavril, et al. 2023). These models generate text one token at a time. Because multi-billion parameter models exceed on-chip cache, weights reside in external memory; generating text token-by-token for one request at a time forces the processor to stream the weights across the memory bus for each token with little reuse across inputs (1.5 billion parameters for GPT-2, 7 billion–70 billion parameters for Llama). This sequential access pattern creates an autoregressive bottleneck that shapes serving cost (Pope et al. 2023).

↪ Continuous batching and KV-cache paging, the runtime mechanisms that serve token-by-token generation, are developed in section 13.10.3.

The recommendation lighthouse, **DLRM**, represents the embedding-heavy recommendation workload behind large-scale "You might also like" systems (Naumov et al. 2019). It maps users and items to embedding vectors stored in tables that can exceed 100 GB of embeddings, making memory capacity rather than computation the binding constraint.

The mobile lighthouse, **MobileNetV2**, performs the same image classification task as ResNet-50 while cutting its computation by 13.7× (Sandler et al. 2018). It gets there by restructuring the image-filtering layers, splitting each expensive filtering step into two much cheaper ones (depthwise separable convolutions). That reduction makes it a candidate for real-time inference within a smartphone's 3–5 W envelope; the implementation and workload determine whether it meets the deadline.

↪ How depthwise separable convolutions factor one filtering step into two cheaper ones is developed in section 6.5.5.

The TinyML lighthouse, **keyword spotting (KWS)**, represents the always-on sensing archetype. KWS systems detect short trigger phrases with compact models built for resource-constrained microcontrollers (Y. Zhang et al. 2017). The lighthouse workload has approximately 200K parameters, about 200 KB at INT8 (stored in flash), placing it in the kilobyte-memory, milliwatt-budget TinyML regime.

A smart doorbell, a TinyML scenario that recurs through later chapters alongside the five lighthouses, applies the same always-on pattern to vision. A visual wake-word model, a small classifier that answers only whether a person is in view, runs on an ESP32-S3 microcontroller, the TinyML platform of table 2.4. Its feasibility is screened in two levels, storage first and latency second. At level

1, the 0.5 MB of 16-bit weights occupy 11.9 percent of the 4 MB flash. Passing this storage check does not establish complete memory fit, because intermediate results and runtime state must also fit in SRAM, the two-tier split that section 2.8.1 develops. Level 2 applies the iron law. The model needs 25 MFLOP per frame, and the microcontroller sustains about 0.25 GFLOP/s at this precision, so compute takes 100 ms; about 1 ms of fixed overhead brings the total to 101 ms. That estimate clears the scenario's canonical 200 ms budget; sensor acquisition and any data movement omitted by the estimate must still be checked. A more responsive chime with a 50 ms budget changes the verdict without changing the model, and figure 2.2 plots that tightened case. Each bar is demand over supply, so flash stays far below the saturation line while latency reaches 202 percent of its budget. Storage was never the constraint that decided feasibility; latency is.

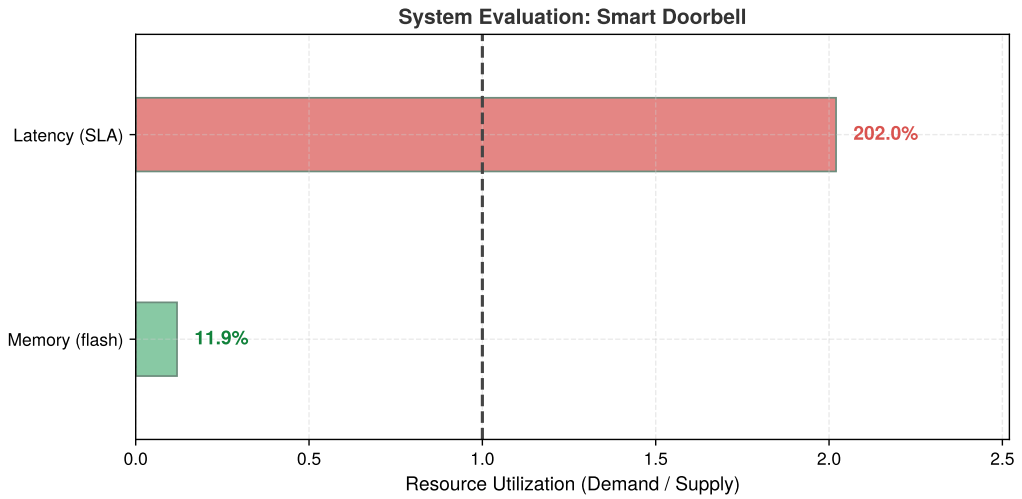


Figure 2.2: Smart Doorbell Constraint Scorecard: Each bar is demand over supply, so the dashed line marks the point where a level is exactly saturated. The model occupies a small fraction of the ESP32-S3's 4 MB flash budget (Level 1: PASS), but its 101 ms latency (25 MFLOP of compute plus fixed overhead) is twice a tightened 50 ms budget for a responsive chime (Level 2: FAIL), even though it clears the scenario's canonical 200 ms budget.

The range in compute requirements and memory footprints explains why no single deployment paradigm fits all workloads. A keyword spotter can operate with roughly 20 MFLOP per inference and about 200 KB of INT8 weights, while ResNet-50 requires about 8.2 GFLOP per image and roughly 102.4 MB of FP32 weights. The reference DLRM example already reaches 100 GB, and Meta has demonstrated scaling recommendation models to 100 TB using embedding offloading (Park et al. 2024). Language models add a bandwidth-dominated regime: billions of parameters streamed repeatedly from memory during autoregressive inference.

2.4 System Balance and Hardware

The iron law and workload archetypes gain practical meaning when matched against real hardware. The single-node stack connects the application's requirements to the software that executes the work and the hardware limits beneath it.

2.4.1 The architectural anchor: The single-node stack

Connecting workload requirements to hardware limits requires an architectural model of the **single-node stack**, which refines the machine side of the D·A·M taxonomy for one host while data and algorithm enter through workload requirements. At the top of the stack, the application defines the mission through a training-throughput or inference-latency target. The ML framework translates model code into executable operations. The operating system, drivers, and runtime coordinate resources and transfers between host memory and accelerator memory. The hardware itself, defined by HBM capacity, memory bandwidth, intra-node interconnects such as NVLink that connect

accelerators, and compute throughput, sets the physical limits, with the memory wall as a recurring constraint.

↪ How an ML framework turns model code into executed operations, eagerly or through a captured graph, is developed in section 7.3.3.

The hardware layer fixes the terms on offer to any model: a memory bandwidth (BW), a peak compute rate (R_{peak}), and a fixed overhead (L_{lat}). The silicon contract names the bargain struck over these terms. The hardware offers them, and the workload and its execution conditions determine which resource limits progress first. This chapter works with the hardware's half, asking which term a given platform makes scarce; the model architecture's half, the wager on which resource to saturate, is stated as a principle (4) once model architectures enter the book. The stack helps locate where a constraint enters: the application sets the deadline, the software determines how work reaches the hardware, and the hardware limits how quickly that work can execute.

↩ The iron law of ML systems and its decomposition of end-to-end latency are established in section 1.7.

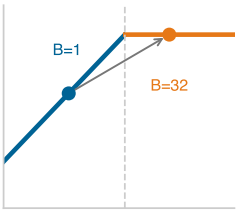
2.4.2 Matching workloads to hardware

The match proceeds from coarse to fine. The coarse test asks whether an operation can fit inside the latency budget at all. The fine test asks which hardware resource limits the operations that do fit, and its answer depends on system balance, the ratio of peak compute throughput to memory bandwidth and I/O capacity on a given platform.

Operation latencies supply the coarse test, and they turn latency-vs-throughput trade-offs into decisions through concrete numbers. They span roughly eight orders of magnitude, from about a nanosecond for an on-chip cache hit or a single accelerator multiply, through milliseconds for one mobile vision inference, to tens or hundreds of milliseconds for a cross-region network call. Section D.1 collects the memory-hierarchy and network round-trip anchors along with illustrative application latency budgets. The span matters because of one strict rule: an operation with latency $> T_{\text{deadline}}$ *cannot* appear on the critical path of a system whose latency budget is T_{deadline} ms.[6]

[6] **Critical path**: The longest sequential chain of dependent operations in a pipeline. The decision rule in the triggering sentence is strict: if a 200 ms cross-region network call appears anywhere on the critical path, a system with a 100 ms total budget is guaranteed to fail regardless of how fast every other stage runs. Model inference need not be the longest stage; preprocessing and postprocessing can dominate, making the critical path longer than model execution alone suggests.

With the critical path settled, system balance decides which resource limits the operations that remain. Roofline analysis classifies bottlenecks by comparing a workload's arithmetic intensity, the operations performed per byte moved (FLOP/byte) that the memory wall made decisive, against the hardware machine balance point ($R_{\text{peak}}/\text{BW}$), the ridge point introduced with the iron law in section 1.7. The framing is informal here; section D.2.1 develops the model, defining arithmetic intensity formally and deriving the ridge point that separates the memory-bound and compute-bound regimes. In that framing, the same ResNet-50 model can shift from compute-bound training behavior at high batch sizes to more memory-sensitive single-image inference at batch=1. Deployment paradigm selection must account for this shift.



In this example, batching moves the workload from the memory-bound side toward the compute ceiling.

The batch-size shift is one instance of a broader change between execution phases, in which physical constraints reconfigure all three D·A·M axes at once. Training updates model weights from batches of examples; serving applies fixed weights to new inputs. Table 2.3 contrasts typical training and latency-sensitive serving demands; cloud serving can also batch requests for throughput.

Table 2.3: D·A·M × Phase: The same model imposes starkly different demands on Data, Algorithm, and Machine depending on whether the system is training or serving. When bottlenecks shift unexpectedly, check which phase is currently being optimized.

Component	Training (Mutable)	Inference (Immutable)
Data	Massive throughput: large batches, shuffling, augmentation	Low latency: single samples, freshness, speed
Algorithm	Learning phase: update model parameters from examples	Prediction phase: apply fixed weights to new inputs
Machine	Throughput-optimized: high-bandwidth clusters, large memory	Latency-optimized: edge devices, inference accelerators

A quantitative comparison applies this analysis to ResNet-50 inference on a high-end data center accelerator and a mobile neural processing unit (NPU). Treat these as point-in-time hardware anchors: the arithmetic is about compute rate, memory bandwidth, and batch size.

Napkin Math 2.1: ResNet-50 on cloud vs. mobile

Problem: Is ResNet-50 inference compute bound or memory bound on (a) a high-end data center accelerator (an NVIDIA A100) and (b) a flagship mobile NPU?
Given (from lighthouse models):

- ResNet-50: 8.2 GFLOP per inference, 25.6 million parameters (102.4 MB FP32, 51.2 MB FP16)

Analysis:
(a) Cloud data-center accelerator (batch=1, FP16)
The analytical model assumes 56.3 MB moved per inference, including weights and activation traffic. This is a modeled transfer volume, not the stored parameter footprint; reuse and runtime choices can change it.

- Peak compute: 312 TFLOP/s (FP16)
- Memory bandwidth: 2.04 TB/s
- Compute time: $T_{\text{compute}} = \frac{8.20 \times 10^9}{3.12 \times 10^{14}} = 0.026$ ms
- Memory time (weights plus activations): $T_{\text{memory}} = \frac{5.63 \times 10^7}{2.04 \times 10^{12}} = 0.028$ ms
- **Result**: Bottleneck = Memory (1.1× slower than compute)
- **Ridge point**: $R_{\text{peak}}/\text{BW}$ = 312 TFLOP/s ÷ 2.04 TB/s ≈ 153 FLOP/byte.
- **Analysis**: Arithmetic intensity = $\frac{8.20 \times 10^9}{5.63 \times 10^7} = 146$ FLOP/byte, just below the ridge point, so the workload is memory bound; above the ridge it would be compute bound. For single-image inference, the low batch size yields limited reuse, explaining why even high-throughput accelerators can be memory bound at batch = 1. Both times use peak rates (efficiency $\eta_{\text{hw}} = 1$ and full bandwidth), so they are lower bounds; because this workload sits close to the ridge, the efficiencies a real kernel achieves decide which term dominates in practice.

(b) Mobile: Flagship NPU (batch=1, INT8)
At INT8, the analytical model assumes 28.2 MB moved per inference, including weights and activation traffic. The INT8 weight footprint below is only one component of that transfer volume.

- Peak compute: ~35 TOPS (INT8)—representative of modern mobile NPUs
- Memory bandwidth: ~51.2 GB/s (LPDDR5)
- Model size: 25.6 MB (INT8 quantized)
- Compute time: $T_{\text{compute}} = \frac{8.20 \times 10^9 \text{ INT8 ops}}{3.50 \times 10^{13} \text{ INT8 ops/s}} = 0.23$ ms
- Memory time (weights plus activations): $T_{\text{memory}} = \frac{2.82 \times 10^7}{5.12 \times 10^{10}} = 0.55$ ms
- **Result**: Bottleneck = Memory (2.3× slower than compute)

Systems insight: Under these peak-rate assumptions, both platforms are memory bound for single-image inference. The A100's faster memory bandwidth (2.04 TB/s vs. 51.2 GB/s = 39.8×) gives roughly 19.9× lower modeled memory-transfer time under the stated FP16 and INT8 traffic assumptions; this ratio is not a measured end-to-end inference speedup. This explains why byte-reduction and lower-precision techniques can beat simply buying more peak FLOP/s for deployment.
ResNet-50 becomes compute bound when batching and data reuse raise computation per byte moved above the hardware balance point, $I > R_{\text{peak}}/\text{BW}$, where $I = O/D_{\text{vol}}$. With ideal reuse during batched inference, each weight is transferred across the memory bus once and used across all B samples in the batch. This amortizes the weight transfer over B inputs, increasing the arithmetic intensity until computation time overtakes memory-transfer time. The crossover is architecture- and implementation-dependent because activation traffic, input/output movement, cache reuse, and runtime execution details all change the effective bytes moved per inference.

The four deployment paradigms gain precision when grounded in concrete hardware. Where table 2.1 defined the paradigms by operating regime, table 2.4 places one real platform in each and pairs it with the per-device envelope that marks the tier.[7] The example-platform and power columns describe complete systems, including a cloud pod, while the tier compute and bandwidth columns describe per-device envelopes. The following spans compare those complete examples, not equivalent individual chips. Memory drops from 128 TB (cloud) to 512 KB (TinyML), a 250 million-fold reduction. Power spans nine orders of magnitude, from about 3 MW for the cloud pod to the TinyML tier's average-power target of < 1 mW, and hardware cost spans six, from multimillion-dollar clusters to $10 microcontrollers. Together with facility overheads such as power usage effectiveness (PUE),[8] these spreads decide which paradigm can serve a workload economically, and they explain why model compression becomes essential as the budget tightens.

The hardware spectrum turns the bottleneck diagnosis into a fit test. These platforms are products of decades of hardware evolution, from floating-point coprocessors in the 1980s through graphics processors in the 2000s to today's domain-specific AI accelerators. Here the consequence matters most: each workload must be matched to the tier whose compute, bandwidth, power, and cost envelope it can satisfy.

[7] **ML hardware cost spectrum**: AI infrastructure spans six orders of magnitude in cost, from $10 microcontrollers to multi-million-dollar accelerator clusters. This million-fold range means deployment paradigm selection is simultaneously a physics decision and an economics decision. Even within individual-device choices, the same accuracy target may be achievable on a low-cost microcontroller only after substantial model reduction, or on an expensive data-center accelerator with a much larger resource budget, with fundamentally different latency, power, and operational cost profiles.

[8] **Power usage effectiveness (PUE)**: PUE is total facility energy divided by IT equipment energy over the same measurement period. It captures facility overhead, including cooling and power distribution. Even a PUE of 1.06 leaves substantial noncompute power at data-center scale. TinyML does not require dedicated facility cooling, although sensing, communication, conversion losses, and sleep power still contribute to its complete system-energy budget. These device-level costs sit outside PUE.

↪ The progression from floating-point coprocessors to domain-specific AI accelerators is developed in section 11.2.1.

Table 2.4: Hardware Spectrum (Concrete Platforms): One representative platform for each paradigm in table 2.1, with the per-device compute and memory-bandwidth envelope that marks the tier and the constraint that usually binds first there: an iron-law term for cloud and edge, a precondition outside the time equation for mobile (energy) and TinyML (memory capacity). Envelope figures mark a paradigm's typical class rather than a hard ceiling. The DGX Spark sits at the high end of the edge spectrum as a compact workstation example; most edge deployments use far smaller devices. BF16 and FP4 are reduced-precision number formats (16 and 4 bits per value; "sparse" rates count skipped zero-valued operations); HBM2 and LPDDR5x are high-bandwidth and low-power memory types; NVMe is solid-state storage.

Paradigm	Example Platform	Tier Compute	Tier Memory BW	Power	Price	Binding Constraint
Cloud ML	Google TPU v4 Pod: 4,096 TPU v4 chips, 1.1 EFLOP/s BF16, 128 TB HBM2	> 1000 TFLOP/s	> 1000 GB/s	~3 MW per pod; PUE 1.1–1.3	Rental	Compute (training); memory bandwidth (LLM decode)
Edge ML	NVIDIA DGX Spark: GB10 Grace Blackwell (1 PFLOP/s sparse FP4), 128 GB LPDDR5x, 4 TB NVMe	~1 PFLOP/s	> 270 GB/s	~200 W; 100 W class typical	~$3,000–$5,000	Memory bandwidth
Mobile ML	Flagship smartphone: mobile SoC (CPU + GPU + NPU), 8–16 GB RAM, 128 GB-1 TB storage	15–45 TOPS	51.2–77 GB/s	3–5 W	$999+	Energy (battery and heat)
TinyML	ESP32-S3: dual-core @ 240 MHz, 512 KB SRAM, 4 MB flash	< 1 TOPS	—	0.05 W–1.2 W active; < 1 mW average target	$10	Memory capacity (working state must fit in SRAM)

Systems Perspective 2.1: System balance across paradigms

The last column of table 2.4 reads each tier through the pipelined bound in equation 2.5. For cloud and edge the binding constraint is a term of that bound; for mobile and TinyML it is a limit the time equation leaves out, and in every tier the dominant constraint sets the optimization strategy. Compute-bound cloud training rewards higher accelerator utilization and larger batches. Memory-bound large language model (LLM) decode (token-by-token generation) and memory-bound edge inference reward fewer bytes moved, through smaller models, lower precision, and more reuse per loaded weight; a faster accelerator helps them little. Mobile adds an implicit energy term, because battery energy is power integrated over time and heat caps sustained power, so lower precision and duty cycling pay off twice. TinyML faces a precondition before any term applies: weights and code must fit in flash, while activations and runtime state must fit in SRAM. Compact models and low-precision arithmetic help meet

those budgets. Section A.5.1 maps each dominant term to the optimizations that work and the ones that are wasted, turning this diagnosis into an action plan.

Each paradigm is therefore an operating envelope with different resource budgets, and moving from warehouse data centers down to microcontrollers traces a physical gradient that trades raw compute throughput for energy and memory efficiency. The four sections that follow walk that gradient one tier at a time, and each asks the same four questions, which the constraint map in each section draws as four branches: which constraint binds, how the system responds, where the paradigm fails, and which workloads fit. Because that structure is shared, each section concentrates on what is distinctive about its tier and on the one or two calculations that decide whether a workload belongs there.

2.5 Cloud ML: Computational Power

Consider the estimated resources needed to train GPT-3:[9] 3,634.3 PFLOP-days of computation, equivalent to 10,000 V100 GPUs running for approximately 15 days at the reported sustained rate, and about 1.3 GWh of electricity (Patterson et al. 2021). A PFLOP-day is 10^{15} FLOP/s sustained for one day, about 8.6×10^{19} FLOPs, so this total is the same few hundred zettaFLOPs estimated from the model's parameter and token counts in section 1.3.3. At the hardware rates used in that estimate, a smartphone or edge server could not have completed this training run on a practical schedule. It required a data-center-scale cluster with enough aggregate compute, memory, and storage. That bill is the bitter lesson paid in hardware. General methods that improve with computation keep winning, and the computation they consume concentrates wherever power, cooling, and accelerators can be pooled. Cloud ML rests on this proposition. When latency can be tolerated, it offers computational scale that no other paradigm can match.

Centralizing these resources under a utility model[10] allows providers to assemble specialized accelerator topologies tailored to distinct hardware demands. This infrastructure serves as the natural home for three of the four workload archetypes: compute beast workloads like ResNet training that demand sustained TFLOP/s across thousands of accelerators, bandwidth hog workloads like large language model inference that benefit from TB/s HBM bandwidth, and sparse scatter workloads like recommendation systems that require terabytes of embedding tables and high-bandwidth interconnects for all-to-all communication patterns.

Cloud deployments scale from single-accelerator instances to warehouse-scale clusters, but systems engineering at every one of these scales begins on a single node, with memory bus saturation, PCIe and NVLink bandwidth, and accelerator utilization inside one machine.

> **Definition 2.1: Cloud ML**
>
> ***Cloud machine learning*** is the deployment paradigm that trades latency for elastic compute by locating ML workloads in centralized data centers, decoupling computational capacity from the physical location of data sources and users.
>
> 1. **Significance**: Cloud deployment dominates the R_{peak} term: a cloud region can pool thousands of accelerators, with access governed by capacity, quotas, and cost. The trade-off is the L_{lat} term: a minimum round-trip latency of 10–100 ms (set by the speed of light over continental distances) makes those distant cloud paths infeasible for workloads requiring sub-10 ms response.
> 2. **Distinction**: Unlike edge ML, which prioritizes local response and data locality at fixed R_{peak}, cloud ML prioritizes elastic R_{peak} at the cost of variable L_{lat}.
> 3. **Common pitfall**: A frequent misconception is that cloud ML is "unlimited compute." In reality, the distance penalty (L_{lat}) and the ingestion bottleneck (D_{vol}/BW) are physics constraints that no software optimization can eliminate, setting a hard floor on response time for any workload whose data originates outside the data center.

[9] **Large language model (LLM) training scale**: Only centralized data centers can aggregate enough R_{peak} for 3,634.3 PFLOP-days training runs (Patterson et al. 2021), but training location does not determine serving location. A trained model or a smaller derived version can later run near users and sensors.

↩ The bitter lesson, the volume's account of why compute-scaled general methods keep winning, is established in section 1.4.

[10] **Cloud as utility computing**: The utility model allows providers to offer a specialized hardware portfolio that many organizations could not economically maintain themselves. This provides direct, on-demand access to the specific architectures required by each workload archetype: dense accelerator pods for compute beasts, HBM-equipped nodes for bandwidth hogs, and high-memory systems with fast interconnects for sparse scatter. A team can rent access to specialized accelerators rather than purchase and maintain the entire installation.

Figure 2.3 diagrams the cloud computing trade-off across four operational branches, tracing how pooling elastic accelerators and sharding models overcome local capacity ceilings while encountering hard failure boundaries in speed-of-light propagation latency and internet dependence.

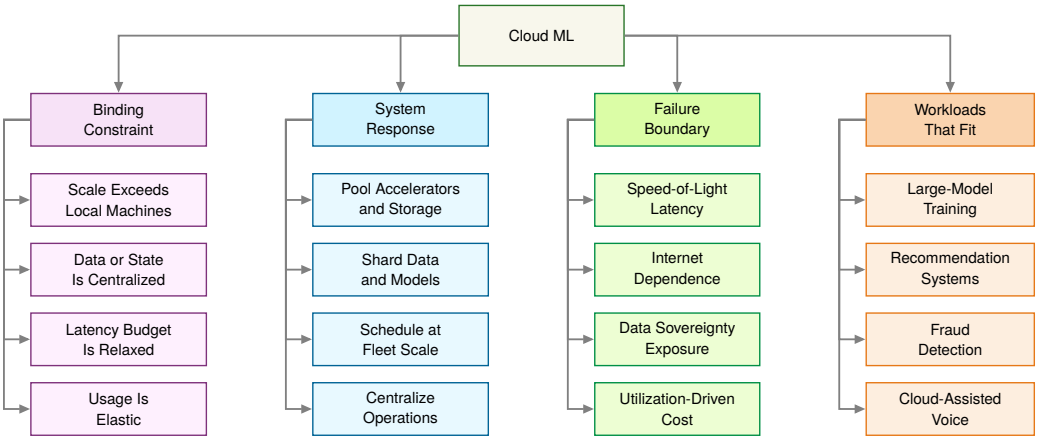


Figure 2.3: Cloud ML Constraint Map: Centralized infrastructure solves the scale problem for compute beasts, bandwidth hogs, and sparse scatter workloads, but it introduces the distance, dependency, privacy, and cost constraints that determine when cloud ML stops being the right deployment target.

Workloads that thrive in this regime (such as asynchronous fraud detection, batch recommendation updates, and large language model pretraining) tolerate this bargain because scale matters more than instantaneous response. When an application requires real-time control, however, network latency stops being an engineering inconvenience and becomes a hard physical barrier. The light-barrier calculation already ruled out a cross-country data center for a 10 ms budget; the sharper question is whether a regional data center, much closer to the machine it protects, fares any better.

Napkin Math 2.2: The distance penalty

Problem: Consider a real-time safety monitor for a robotic arm on a factory floor. The safety logic requires a 10 ms end-to-end response time to prevent injury, so the distance penalty imposed by the light barrier matters. The model runs in a regional cloud data center 1,500 km away, less than half the cross-country distance. Can the safety budget be met?

Physics:

1. **Light in fiber**: ~200,000 km/s.
2. **Round-trip propagation**: (1,500 km $\times$ 2)/200,000 km/s = 15 ms.
3. **Result**: Round-trip propagation alone requires 15 ms, exceeding the 10 ms end-to-end budget by 5 ms before the model performs any inference.

Systems insight: The specified regional cloud path misses the budget on propagation alone and is therefore infeasible. Running the model beside the arm removes that remote propagation term, but the complete local sensing, inference, and control path must still be checked against the deadline.

2.5.1 Cloud infrastructure and scale

Cloud ML concentrates computational resources at physical dimensions no local device approaches. TPU supercomputer designs organize thousands of specialized accelerator chips into data-center-scale systems that deliver PFLOP/s-to-EFLOP/s reduced-precision throughput (Jouppi et al. 2023), orders of magnitude beyond the edge and mobile rows of table 2.4.

Figure 2.4 captures the physical reality of warehouse-scale infrastructure, showing aisles of liquid-cooled racks packed with thousands of interconnected TPU accelerator chips[11] and dense optical

[11] **Tensor processing unit (TPU)**: An application-specific integrated circuit (ASIC) designed for operations common in ML, especially matrix multiplication. PFLOP/s-to-EFLOP/s figures describe aggregated TPU systems rather than one chip. Specialization trades general-purpose flexibility for workload-dependent efficiency; its advantage over a GPU depends on the model, numerical format, and systems being compared (Jouppi et al. 2023).

cabling bundles that draw megawatts to sustain exaFLOP-scale matrix throughput. That physical scale is what makes training on hundreds of terabytes of data practical.

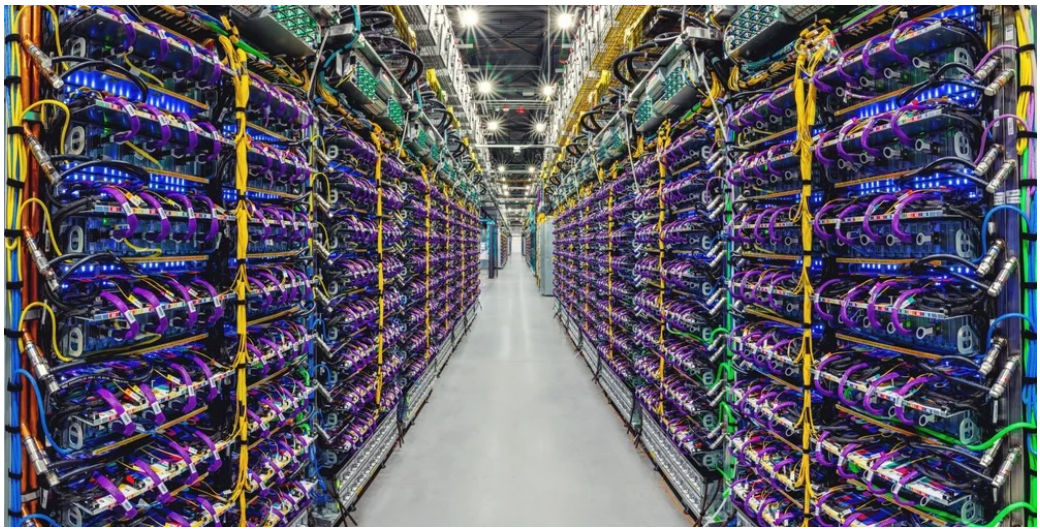


Figure 2.4: Cloud Data Center Scale: Dedicated accelerator facilities aggregate thousands of specialized TPU chips in liquid-cooled racks to deliver exaFLOP-scale matrix throughput. This centralized infrastructure makes large-scale model training and high-throughput serving viable, but incurs megawatt-scale power consumption and speed-of-light network distance penalties. Image source: (Pichai 2023).

Centralization also changes how models are served and governed. Cloud APIs make trained models accessible worldwide across mobile, web, and IoT platforms. Shared infrastructure enables engineering teams to collaborate with unified version control, while pay-as-you-go pricing models[12] convert upfront capital expenditure (CapEx) into usage-based operational expenditure (OpEx) that scales with demand.

Scale does not make the cloud universally superior. The data-gravity principle (2) explains why. Shipping a dataset of size D_{vol} over a network link of bandwidth $\text{BW}_{\text{network}}$ takes $D_{\text{vol}}/\text{BW}_{\text{network}}$, a time that grows with the data, while shipping the computation (a model and its code) costs roughly the same however much data waits at the other end. As data volume grows, moving the data to a remote cluster therefore eventually costs more than moving the computation to the data. Section B.1.1 works through the transfer arithmetic, and the edge section applies the same comparison to a single request under a deadline. The deployment paradigm is dictated by the binding physical constraint, not by the largest available throughput.

2.5.2 Cloud ML trade-offs and constraints

For latency-sensitive workloads, the distance penalty is the cloud's most consequential cost, and network jitter makes it unpredictable as well as large. Three further costs follow from centralization: the exposure of data the cloud must receive, spending that scales with use, and dependence on the network.

Privacy and security pose additional boundaries for cloud deployment. Transmitting sensitive data to remote data centers expands the disclosure surface and complicates regulatory compliance. Organizations handling data subject to regulations such as the General Data Protection Regulation (GDPR)[13] or the Health Insurance Portability and Accountability Act (HIPAA)[14] must select, document, and maintain safeguards appropriate to the data, system, and applicable legal duties. Privacy-preserving approaches can reduce how much sensitive data leaves its original environment, but they complement rather than replace infrastructure controls.

Cost management introduces operational complexity because cloud infrastructure converts upfront hardware procurement into continuous operational expenses. Comparing paradigms by raw accelerator hourly rates obscures the cumulative expenses of network egress, persistent storage, and managed orchestration. Sound architectural evaluation demands total cost of ownership (TCO)[15]

↪ How training extends beyond a single node across distributed workers is developed in section 8.8.2.

[12] **Pay-as-you-go pricing**: A cloud economic model where users pay for billable allocated accelerator-hours rather than hardware owned. Idle time can still be billed while an instance remains allocated; busy utilization alone does not determine the charge. Sustained allocation can cost more than amortized on-premises hardware, but table 2.5 illustrates only a partial budget under stated assumptions, not a matched total-cost crossover.

[13] **GDPR (General Data Protection Regulation)**: Article 17 provides a right to erasure when specified grounds apply (European Parliament and Council of the European Union 2016). For an ML system, an erasure request requires tracing where the person's data persists across source stores, derived features, logs, caches, and model artifacts. Retraining may be one remediation, but the regulation does not prescribe retraining every model that encountered the data.

[14] **HIPAA**: The Security Rule requires regulated entities to implement reasonable and appropriate administrative, physical, and technical safeguards for electronic protected health information, including access and audit controls (U.S. Department of Health and Human Services 2005). The rule is technology-neutral and does not mandate confidential-computing enclaves; concrete controls and their performance overhead depend on the system's risk analysis and implementation.

analysis, trading initial capital expenditure against recurring operational maintenance across the system's operational lifecycle.

A vision system serves 1M daily inferences at ResNet-50 scale (10 ms latency, 100 KB response). The TCO[16] analysis begins with an illustrative budget spanning 3 years. It includes the listed components rather than a complete, matched TCO. Cloud operating labor, edge redundancy, installation, and failure recovery are not priced, and comparable capacity and availability have not been established. The 4 instances are a provisioning choice (redundancy and headroom for traffic peaks), not a throughput requirement. Table 2.5 itemizes the annual GPU, network, load-balancer, and observability costs of this fixed-capacity cloud implementation under public list pricing, and table 2.6 itemizes the corresponding hardware, power, cooling, network, and DevOps labor costs of an on-premises edge-server implementation.

Table 2.5: Cloud Inference Annual Budget: Itemized GPU, network, load-balancer, and observability costs for the fixed-capacity cloud implementation of the ResNet-50-scale vision workload.

Cost Component	Calculation	Annual Cost
GPU inference	4 instances × 8760 h/year × $0.75/hr	~$26,280
Network egress	100 GB/day × 365 d/year × $0.09/GB	~$3,285
Load balancer	$0.025/hr + capacity units	~$3,723
Monitoring/logging	Monitoring, alerts	~$2,000
Total Cloud		**~$35,288/year**

[15] **Total cost of ownership (TCO)**: Quantifies the gap between sticker price and true system cost by including all direct and indirect costs (power, cooling, labor) over a system's lifetime, trading upfront capital expense (CapEx) against recurring operational expense (OpEx). For an on-premises GPU, the purchase price is only one component of the three-year TCO, alongside operating costs.

Table 2.6: Edge Inference Annual Budget: Itemized hardware, power, cooling, network, and DevOps labor costs for the on-premises edge-server implementation, exposing labor as the dominant component of this scenario.

Cost Component	Calculation	Annual Cost
Hardware CapEx	$15,000 ÷ 3 years life	~$5,000
Power (24/7)	300 W × 8760 h/year × $0.12/kWh	~$315.4
Cooling overhead	~30% of power	~$94.6
Network (fiber)	Fixed line for remote management	~$1,200
DevOps labor	0.1 FTE × $150,000 salary	~$15,000
Total Edge		**~$21,610/year**

Napkin Math 2.3: Cloud vs. edge annual budgets

Problem: Under the partial accounting boundaries itemized in table 2.5 and table 2.6, which implementation has the lower estimated annual budget over 3 years?

Math: The question is the annual cost at this operating point, so the comparison in equation 2.6 is the gap between the two annual totals:

$$\text{Annual cost gap} = \text{Cloud annual budget} - \text{Edge annual budget} \tag{2.6}$$

At 1M inferences/day, the gap is ~$35,288/year – ~$21,610/year = ~$13,678/year. Almost all of the cloud bill is provisioned capacity rather than use. At 10 ms per request the day's traffic needs only about 2.8 GPU-hours of the 96 provisioned, about 2.9 percent utilization. Only egress scales with traffic.

Result: The listed edge budget costs about 38.8 percent less at this operating point. Because the fixed cloud pool alone already costs more than the listed edge budget, no traffic level under this fixed provisioning and partial accounting makes the cloud cheaper; a genuine break-even analysis would need cloud capacity that scales with demand.

Systems insight: The listed edge budget is dominated by labor (~$15,000 ÷ ~$21,610/year ≈ 69.4 percent), not hardware. Organizations without existing DevOps capacity should factor in the full cost of maintaining on-premises infrastructure.

Beyond the direct bill, unpredictable usage spikes complicate budgeting and call for automated monitoring and cost governance. Network dependency, the last of the three costs, creates an availability hazard: any connectivity disruption between clients and the data center immediately degrades or halts service. Proprietary cloud APIs compound the dependency with vendor lock-in, creating software portability friction when migrating workloads across providers. Even so, centralized infrastructure remains uniquely suited to consumer applications that require elastic capacity at global scale.

2.5.3 Large-scale training and inference

Cloud ML's computational advantages manifest most visibly in consumer-facing applications that require massive scale. Virtual assistants like Siri and Alexa illustrate the hybrid architectures that characterize modern ML systems: wake-word detection can run on dedicated sub-milliwatt-to-milliwatt hardware directly on the device, enabling always-on listening without continuously activating the main processor; initial speech recognition can run on-device for privacy and responsiveness; and complex natural language understanding and generation can use cloud infrastructure for access to larger models and broader knowledge.

Economics drive this architecture as much as latency. Attempting to process voice interactions for billions of devices entirely in the cloud runs into both an economic and an infrastructure ceiling. Quantifying the voice assistant wall shows both limits at once.

[16] **Total cost of ownership (TCO)**: Quantifies the gap between sticker price and true system cost by including all direct and indirect costs (power, cooling, labor) over a system's lifetime, trading upfront capital expense (CapEx) against recurring operational expense (OpEx). For an on-premises GPU, the purchase price is only one component of the three-year TCO, alongside operating costs.

Napkin Math 2.4: The voice assistant wall

Problem: In this illustrative capacity scenario, 1B voice assistant devices (smartphones, smart speakers, earbuds) each issue 20 queries/day after the wake word. Detecting the wake word itself in the cloud would require every microphone to stream audio continuously. What load would cloud-side detection impose, and what load remains in the cloud once detection moves on device?

Cloud-side detection: First, the always-listening path that local detection removes.

- **Audio streaming**: Every device streams audio continuously (16 kHz, 16-bit), about 32 KB/s each. Across 1 billion devices the aggregate is 32 TB/s, and each stream would also need its own continuously running detector.
- **Cloud cost**: ~$0.50/device/year $\rightarrow$ 1B devices = $500,000,000/year. Economically prohibitive under these assumptions for a free feature.
- **TinyML alternative**: 0.1–1 mW local wake-word detection, <$0.01/device/year. If each post-wake query uploads about 5 s of audio, local detection cuts the audio leaving each device by about 860×.

Residual cloud load: Second, the post-wake queries the hybrid design still serves in the cloud. Even with local wake-word filtering, the residual query volume—1 billion devices generating 20B queries/day at ~200 ms per query—still requires substantial centralized accelerator capacity (~1,111,111 GPU-hours per day).

Five parameters determine the required cluster capacity:

1. **Query volume**: 1 billion devices $\times$ 20 queries/day = 20B queries/day.
2. **GPU demand**: Each query requires ~200 ms of GPU time. Total: 1,111,111 GPU-hours/day.
3. **Scenario capacity**: A data-center equivalent provisioned with 10,000 GPUs provides 240,000 GPU-hours/day.
4. **Average requirement**: ~4.6 dedicated data centers just for voice inference.
5. **Peak reality**: Queries cluster in waking hours (~4.5× peak-to-average), requiring ~20.8 data centers at peak.

Systems insight: Detecting the wake word in the cloud would require streaming every microphone continuously, about 32 TB/s across a billion devices, before a single user query is answered. Local detection reduces each device's upload to its post-wake queries, 20 queries/day

of a few seconds each. The residual post-wake load is still large, tens of data centers at peak, but it is bounded by what users actually ask, and it is the load the hybrid design deliberately keeps in the cloud to use a larger language model than the local device can support.

The voice assistant pipeline illustrates a core systems principle: constraints drive hierarchical architecture. Partitioning the pipeline into an on-device wake-word filter and a cloud-based language model relieves both the client's energy limit and the data center's capacity limit.

Large-scale recommendation engines demonstrate the cloud bargain in a memory-capacity-bound form. DLRM-style systems[17] map millions of users and items to dense vectors through embedding tables that routinely exceed terabytes in aggregate, far beyond any single accelerator. The tables are sharded across pooled host and accelerator memory, and distributed lookups and training updates place heavy demands on the cluster interconnect.

These applications trade latency for scale, accepting remote-path delay in exchange for computational resources that cannot be supplied locally. The trade fails when a latency budget drops below what the remote path permits, or when data volumes exceed what networks can carry; computation must then move closer to the data source.

2.6 Edge ML: Latency and Privacy

The robotic-arm safety monitor in napkin math 2.2 left one option open, running the model beside the arm it protects.[18] Moving computation adjacent to the data source removes that remote transmission term from the request path, and edge ML is the paradigm built on that move.

Edge ML trades elastic cloud capacity for shorter request paths and local data retention. Sub-100 ms response remains a requirement to verify, rather than a guarantee of proximity. In archetype terms, edge deployment shifts the binding constraint: a bandwidth hog workload that is memory bound in the cloud can become dominated by remote-path latency when offloaded over wide-area links. Under the iron law from equation 2.5, local execution removes the I/O offload term from the request path, leaving execution time bounded by local compute and memory transfer.

Definition 2.2: Edge ML

Edge machine learning is the deployment paradigm optimized for local response and data locality by locating computation physically adjacent to data sources.

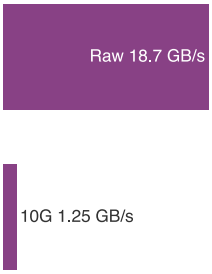
1. **Significance**: It circumvents the distance penalty (L_{lat}) of the cloud, trading elastic scale for a fixed local compute capacity (R_{peak}).
2. **Distinction**: Unlike cloud ML, which prioritizes throughput, edge ML prioritizes response deadlines and local data control. Unlike TinyML, edge ML may still use workstation-class accelerators such as general-purpose GPUs (GPGPUs).
3. **Common pitfall**: A frequent misconception is that edge ML refers to a specific hardware class. In reality, it is a location paradigm: it spans from IoT gateways to on-premises servers, unified by physical proximity to the data source.

Real-time deadlines are not the only driver. Data sovereignty and privacy rules can restrict transmitting raw sensor streams across external networks, which places computation on local gateways, industrial PCs, and on-premises servers. Figure 2.5 partitions this operational bargain across four structural columns, charting how placing compute near sensors and filtering data locally helps meet sub-100 ms response budgets while introducing failure boundaries in fixed node capacity and remote fleet management.

Sensor ingestion rates above network uplink capacity rule out transmitting the full raw stream.[19] Local filtering or compression can reduce the transmitted representation; the complete local and remote paths then determine where inference can meet the application's requirements.

17 **Deep learning recommendation model (DLRM)**: Meta's 2019 architecture that exemplifies the sparse scatter archetype. Embedding-heavy recommendation models have been scaled to 100 TB by combining accelerator-memory caches with larger host memory (Park et al. 2024). At that scale, memory capacity and communication BW can constrain execution more than raw R_{peak}. These demands motivate specialized cluster designs where memory, not arithmetic, is the scarce resource.
↪ The memory-capacity bounds and sparse gather patterns of high-cardinality embedding tables are developed in section 6.9.

18 **Industrial IoT (IIoT)**: A domain where latency constraints can be set by physical safety rather than user perception. Keeping sensing and the stop decision local removes the long-haul network term but leaves the edge system bounded by its local compute capacity (R_{peak}).



Raw edge data can be wider than the network pipe.

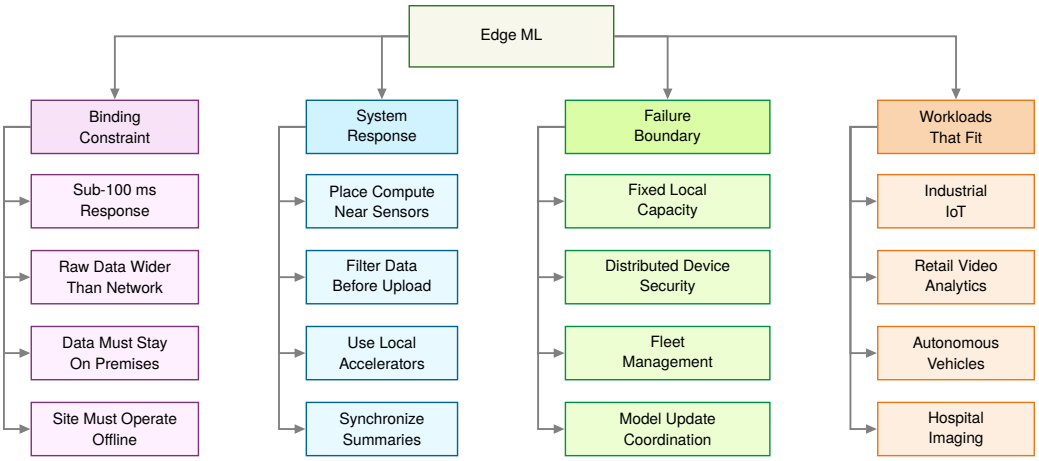


Figure 2.5: Edge ML Constraint Map: Moving computation near the data removes the network-latency term and reduces bandwidth pressure, but it replaces centralized scale with fixed local capacity, distributed security exposure, and operational complexity across many sites.

[19] **Internet of things (IoT) data wall**: McKinsey estimates that IoT deployments could create trillions of dollars in economic value by 2030, but those deployments depend on continuous sensor streams from devices distributed across homes, factories, farms, vehicles, and infrastructure (McKinsey Global Institute 2021). In many of those settings, the aggregate D_{vol} from raw streams overwhelms the available uplink budget or latency budget for centralized ingestion, making local edge processing an architectural requirement rather than merely a cost optimization.

Napkin Math 2.5: The bandwidth bottleneck

Problem: Consider a quality control system for a factory floor with 100 cameras running at 30 FPS with 1080p resolution. Does video streaming become the bandwidth bottleneck, and does edge ML reduce the bandwidth enough to process locally?

Physics:

1. **Raw data rate per camera**: $1920 \times 1080 \times 3$ bytes $\times$ 30 FPS $\approx$ 186.6 MB/s.
2. **Total data rate**: 100 cameras $\times$ 186.6 MB/s = 18.7 GB/s.
3. **Cloud transfer exposure**: Uploading raw camera feeds is primarily a bandwidth, ingest, storage, and processing problem; per-GB cloud egress charges apply when data is transferred back out of the cloud. If the raw stream were later retrieved at \$0.09/GB data-transfer-out pricing, the transfer charge alone would reach \$4.4M/month.
4. **Network reality**: Even a dedicated 10 Gbps line (1.25 GB/s) cannot carry the load, because the workload demands 14.9× more bandwidth than exists.

Systems insight: Streaming these raw feeds over the stated link is *impossible*. Local processing or a smaller transmitted representation is required; compression changes the bandwidth calculation but must preserve the information and timing needed for defect detection. If an edge server transmits only defect metadata (1 KB per detection at roughly 20 events/s across the floor), bandwidth falls by about 933,120×.

Saturating an uplink with uncompressed sensor streams demonstrates the bandwidth barrier of centralized ingestion. For battery-powered edge devices (wireless cameras, drones, wearables), the energy cost of wireless communication introduces an even stricter physical ceiling. Offloading adds energy to the time-based analysis of equation 2.5, so the local-versus-offload decision must weigh joules as well as milliseconds.

Napkin Math 2.6: The energy of transmission

Problem: Should a battery-powered sensor process data locally (on-device NPU inference) or send it to the cloud when the energy of transmission must come out of a fixed battery budget?

Given:

- **Data** (D_{vol}): 1 MB (illustrative payload volume).
- **Transmission energy** (E_{tx}): 100 mJ/MB (Wi-Fi/LTE).
- **Compute energy** (E_{local}): 0.1 mJ/inference (illustrative MobileNetV2 anchor for the neural processing unit alone, excluding the rest of the phone).

Math:

1. **Cloud approach**: $E_{\text{transmit}} \approx D_{\text{vol}} \times E_{\text{tx}} = 1 \text{ MB} \times 100 \text{ mJ/MB} = 100 \text{ mJ}$.
2. **Local approach**: $E_{\text{local}} \approx \text{Inference} = 0.1 \text{ mJ}$.

Systems insight: Transmitting raw data is 1,000× more expensive than processing it locally. Charging local inference with the whole phone's energy over the inference window (2.6 mJ, the MobileNetV2 bar charted in figure 2.6) still leaves transmission 39× more expensive. Even if the cloud had infinite speed ($T \approx 0$), the stated battery budget favors local inference over raw-data transmission. The machine constraint (battery) shapes the deployment choice.

The matched transmission calculation shows why uploading this raw payload spends more device energy than local inference under the stated assumptions. Figure 2.6 instead supplies energy scale anchors for different tasks and accounting boundaries on a logarithmic scale spanning eight orders of magnitude, isolating the 100,000,000× gap between an always-on TinyML keyword spotting inference (about 10 µJ) and a cloud foundation-model query (about 1 kJ, an order-of-magnitude scenario anchor that includes data-center overhead rather than a measured value). The across-task ratio does not isolate a hardware advantage or establish an offload decision; that decision requires matched local and remote paths for the same task.

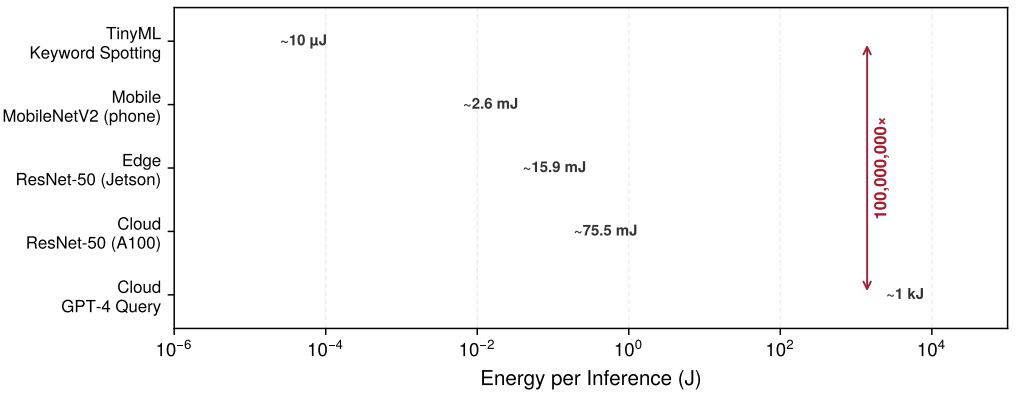


Figure 2.6: Energy per Inference Across Deployment Tiers: An eight-order-of-magnitude energy gap separates always-on TinyML keyword spotting (~10 µJ) from a cloud foundation-model query (~1 kJ) on a log scale. The three vision bars are device-level estimates (the device's power draw at its modeled utilization over the batch-1 inference window, counting the whole phone, the edge tier's NVIDIA Jetson embedded GPU module, or the A100 card) from the same analytical model used throughout this chapter; the two endpoints are scenario anchors. The tasks and accounting boundaries differ, so these bars illustrate energy scales rather than a controlled hardware-efficiency or offload comparison.

Systems Perspective 2.2: Reading the energy-per-inference numbers

The logarithmic comparisons plotted in figure 2.6 combine solver-derived device-energy estimates with separate scale anchors; they are not controlled full-system measurements. Each vision bar counts one device over its batch-1 inference window. The cloud ResNet-50 bar, for example, charges an A100 averaging about ~130 W for 0.57 ms; it excludes the host CPUs, memory, networking, and cooling that bring an entire server to about ~1 kW, so a whole-server accounting of the same request would be several times larger.

The span becomes tangible when measured in a familiar unit of energy, one smartphone battery. The nominal energy stored in a 3.7 V, 3000 mAh battery is equivalent to about 4 billion inferences by the keyword spotter but only about 40 queries to a cloud LLM. The comparison is one of scale only: a cloud query's energy is spent in the data center, not drawn from the phone. Nor are the keyword-spotting counts elapsed battery life: actual runtime still depends on inference cadence, sleep power, sensing, communication, and the rest of the device.

2.6.1 The data locality invariant

The decision between local edge processing and remote cloud processing is governed by a bandwidth-latency trade-off. The data locality invariant below is the per-request, deadline-bound form of the data-gravity principle from the cloud section: where data gravity weighs moving a whole dataset against moving the computation, the invariant weighs moving one input against one deadline. Remote execution is infeasible when its complete path violates the service deadline; otherwise, the faster feasible path follows from comparing complete local and remote times.

Definition 2.3: The data locality invariant

The data locality invariant first screens a remote path against the service deadline. The expression below gives a lower bound on complete remote-path time T_{remote}; let T_{deadline} denote the deadline, $\text{BW}_{\text{network}}$ offload bandwidth, $L_{\text{lat,network}}$ network latency, and $R_{\text{peak,remote}}$ and $\eta_{\text{hw,remote}}$ the remote compute rate and efficiency:

$$T_{\text{remote}} \geq \frac{D_{\text{vol}}}{\text{BW}_{\text{network}}} + L_{\text{lat,network}} + \frac{O}{R_{\text{peak,remote}} \cdot \eta_{\text{hw,remote}}}$$

The bound includes input transmission, network round-trip time, and arithmetic execution, but omits remote memory movement, queueing, preprocessing, runtime overhead, and response serialization. If even this lower bound exceeds the deadline, reject the remote path. A bound below the deadline leaves a candidate for complete-path measurement; it does not establish feasibility. Compare complete local and remote times only after each path meets the deadline and its resource constraints. Here, $L_{\text{lat,network}}$ covers propagation, routing, and protocol handshakes; response serialization may be small for a class label but must be included when material.

1. **Significance**: The invariant defines a crossover beyond which adding remote compute ($R_{\text{peak,remote}}$) has little effect because transmission dominates the lower bound. The local path is preferred only after its own complete execution time and resource limits are checked.
2. **Distinction**: Unlike the iron law, which decomposes one execution path, the locality invariant uses a deadline to reject an idealized remote path, then requires complete-path evidence to choose between the remaining alternatives.
3. **Common pitfall**: A frequent misconception is that 5G/6G "solves" locality. While these technologies improve $\text{BW}_{\text{network}}$, they do not remove propagation and protocol latency, so a remote path can still miss a tight service deadline.

The locality formulation also explains what every local tier gains, edge, mobile, and TinyML alike. Local execution removes network transit time ($D_{\text{vol}}/\text{BW}_{\text{network}} + L_{\text{lat,network}}$) from the request path, eliminating wide-area network variability. A closed-loop latency guarantee requires validating a bound on the complete local path under its operating conditions, $T_{\text{local}} \leq T_{\text{deadline}}$. Uplink traffic drops from raw sensor bitrates to lightweight event telemetry, relieving congestion and recurring transit charges. Keeping raw sensor data where it is captured shrinks the transmission attack surface, although local processing alone does not guarantee privacy: data at rest still needs access control, and retention and secondary use still need governance. Local inference also continues through

wide-area outages whenever local power and sensor links remain functional. Mobile and TinyML inherit all four gains; what distinguishes them is the price each pays for them.

Evaluating a single high-rate sensor frame against a strict frame deadline illustrates the locality crossover in practice.

Napkin Math 2.7: The locality crossover

Problem: Should a drone's object avoidance system (4K, 60 FPS) offload to the cloud, or does this become a locality crossover?

Given:

- **Data** (D_{vol}): 4K frame ≈ 24.9 MB.
- **Bandwidth** ($\text{BW}_{\text{network}}$): 100 Mb/s cellular uplink.
- **Remote network and compute response**: 110 ms (round-trip + remote compute).
- **Per-frame completion deadline**: 16.7 ms, equal to the frame interval in this illustrative scenario; each avoidance result must be ready before the next frame arrives.

Math:

1. **Transmission time**: 24.9 MB × 8 bits/100 Mb/s = 1,990.7 ms.
2. **Remote-path lower bound**: Transmission + 110 ms remote response = 2100.7 ms, vs. the stated 16.7 ms per-frame deadline.

Systems insight: The remote path misses the stated per-frame deadline, and faster remote compute cannot remove the transmission time. Cloud offload is therefore infeasible under these assumptions; the local path must still be measured to confirm that it meets the deadline.

2.6.2 Edge ML deployment challenges

Edge deployment trades network transit for fixed local capacity. Edge hardware spans a wide power envelope, from 5–50 W embedded modules to site-powered servers drawing hundreds of watts, and its binding limit is the fixed capacity of a site rather than the elastic capacity of a cloud region. Many smaller embedded modules operate within 1–8 GB of memory,[20] and their power and board budgets cannot support the wide, high-power interfaces of HBM. They rely on LPDDR or shared system DRAM instead, so memory capacity limits parameter scale and memory bandwidth limits throughput, and fitting modern machine learning models into these bounds requires efficiency techniques that store weights in fewer bits (quantization), remove weights that contribute little (pruning), or train a smaller model to imitate a larger one (distillation). Model footprint, activation buffers, runtime state, and thermal duty cycles all become binding. Operationally, coordinating software updates and model versions across thousands of heterogeneous, physically exposed devices requires specialized deployment pipelines,[21] where intermittent links and physical access complicate rollback and failure diagnosis.

A realistic retail deployment shows how those constraints turn into throughput, hardware, and fleet-cost requirements. Consider a smart retail chain that deploys person detection across 500 stores, each with 20 cameras/store running at 15 FPS. Table 2.7 cascades the per-store inference rate through the per-frame FLOP count of YOLOv8-nano, a compact object-detection model, to yield the throughput each store must sustain. The 2× headroom keeps the device out of saturation when traffic bursts above the average rate.

Table 2.7: Edge Inference Sizing Requirements: Per-store throughput target for the smart-retail person-detection scenario.

Metric	Calculation	Result
Inferences per store	20 cameras/store × 15 FPS	300 inferences/s
Model compute	YOLOv8-nano: 8.7 GFLOP/inference	2610 GFLOP/s
Required throughput	2610 GFLOP/s × 2 (headroom)	~5.22 TOPS equivalent

[20] **Embedded edge constraints**: Across representative edge envelopes, 1–8 GB can hold roughly 1–8 billion INT8 weights before activations, runtime state, and other buffers, so the memory budget, not a fixed parameter count, sets the ceiling. Sustainable compute is further limited by power, cooling, and duty cycle.

[21] **Edge fleet coordination**: Managing thousands of distributed edge devices amplifies coordination problems: intermittent connectivity causes model version drift, hardware heterogeneity requires per-target optimization, and limited remote access can make failed firmware rollbacks require a site visit.

↪ Over-the-air updates, dual-bank rollback, and lightweight fleet telemetry for edge devices are operationalized in section 14.5.5.

Table 2.8 compares throughput, power, unit cost, and fleet cost for three candidate edge accelerators, including embedded GPU accelerators.

Table 2.8: Edge Accelerator Options: Throughput, power, and cost for three candidate edge accelerators at fleet scale. Fleet cost assumes one unit per store for Jetson and NUC. Jetson power depends on the module: Jetson Orin Nano draws 7–15 W, Orin NX 10–25 W, and AGX Orin 15–60 W, so Jetson power figures in this book should be read against the module named in context.

Edge Device	INT8 TOPS	Power	Unit Cost	Fleet Cost
NVIDIA Jetson Orin NX	100 TOPS	10–25 W	$600	$300,000
Intel NUC + Movidius	1 TOPS	15 W	$400	$200,000
Google Coral Dev (3 boards/store)	peak 12 TOPS derated 6 TOPS	6 W	$450	$225,000

Napkin Math 2.8: Edge inference sizing

Problem: Given the throughput target in table 2.7 and the candidates in table 2.8, which edge accelerator (USB-scale TPU, workstation-class embedded GPU, or general-purpose mini-PC) delivers the required throughput at the lowest estimated three-year hardware-and-power budget?

Math: We screen capacity using an assumed sustained rate rather than the datasheet peak. The scenario's 50 percent derating plays the role of the efficiency η_{hw} in the iron law. One Jetson derates from 100 TOPS to 50 TOPS, far above the 5.22 TOPS requirement. One NUC derates to 0.5 TOPS and falls short; meeting the target would take 11 per store, about $2,200,000 of hardware across the fleet. A Coral Dev Board is rated at 4 TOPS peak; the same derating gives 2 TOPS, so the count per store is 5.22 TOPS required ÷ 2 TOPS per board = 2.6 boards, which rounds up to 3 boards/store. This estimate assumes the model's operations and numerical format are supported and that work can be divided across the boards at the assumed rate.

Over 3 years across 500 stores, the hardware-and-power estimate for that configuration is Hardware $225,000 + Power (0.006 kW $\times$ 500 $\times$ 8760 h/year $\times$ 3 years $\times$ $0.12/kWh = $9,460.8) = $234,460.8 total vs. a baseline of one dedicated cloud GPU per store, at the same $0.75/hr instance rate as the cloud-versus-edge comparison, of ~$9,855,000. Within this accounting boundary, the other two candidates need no power arithmetic to rank, because the Coral configuration's hardware-and-power estimate is already below the Jetson fleet's $300,000 hardware bill and far below a feasible NUC fleet's. Integration, maintenance, redundancy, and matched cloud operating costs remain outside this comparison.

Result: One Coral alone is undersized under these assumptions, but the sharded configuration with 3 boards/store is a low-cost edge candidate within the hardware-and-power boundary, with an estimated 6 TOPS and about 1.3× lower per-store hardware CapEx than Jetson. Jetson remains the simpler single-device candidate when integration complexity matters more than hardware cost.

Systems insight: Edge sizing is a capacity-and-cost problem, not a device-name problem. The cheapest feasible design may be several small accelerators per site rather than one larger board, but that hardware saving must be weighed against the coordination and maintenance burden of a sharded edge fleet. The estimate narrows the candidates; measurements of the actual model and runtime must establish task quality, throughput, and end-to-end latency before deployment.

Physical exposure compounds these operational challenges. Unlike servers housed in access-controlled data centers, edge hardware deployed across retail floors or public infrastructure is physically accessible to adversaries. This exposure requires hardware-enforced protection mechanisms: secure boot chains to prevent unauthorized firmware modification, encrypted local storage for model weights and intermediate activations, and tamper-evident enclosures. Furthermore, capital expenditures scale linearly with deployment footprint: equipping 1,000 sites with $500–2,000 of edge hardware per site requires $500,000–2,000,000 in upfront CapEx before installation, cabling,

and fleet operations. Whether that upfront investment yields a lower total cost of ownership than recurring cloud inference depends on utilization duty cycles, local energy rates, and site maintenance overhead.

2.6.3 Real-time industrial and IoT systems

Edge applications differ by domain, but each makes a locality argument concrete: the system cannot wait for the network, cannot ship the data, cannot expose the raw signal, or cannot stop during an outage. Autonomous vehicles represent a demanding application because safety-critical perception and control decisions must fit millisecond-scale budgets. Onboard systems process streams from multiple cameras and other sensors through local hardware; a remote cloud path cannot satisfy the tight control loop when its network delay alone exceeds the deadline.

Smart retail environments demonstrate edge ML's practical advantages for privacy-sensitive, bandwidth-intensive applications. A checkout-free store[22] can process feeds from many cameras on local edge servers, tracking customer movements and item selections without continuously transmitting raw video over a wide-area link. This architecture reduces bandwidth demand and transmission exposure, but it does not by itself ensure privacy: retention, access, secondary use, and governance controls still determine the system's compliance and risk.

The industrial IoT[23] uses edge ML for applications where millisecond-level responsiveness directly impacts production efficiency and worker safety. Manufacturing facilities deploy edge ML systems for real-time quality control and predictive maintenance.[24]

Many industrial and retail edge systems are stationary and connected to site power, where a factory edge server drawing hundreds of watts around the clock remains viable. Mobile devices, by contrast, travel with untethered users on strictly bounded battery reserves. When computation moves from stationary infrastructure to an untethered mobile device, battery energy becomes a central constraint alongside latency, and the dominant optimization target shifts from raw execution time to energy per inference.

2.7 Mobile ML: Offline Intelligence

Carrying edge-style locality into a user's pocket changes which budget binds. A phone must deliver responsive, private inference all day from a pocket-sized battery, inside a thin chassis with no fan to carry heat away.

Mobile ML integrates machine learning directly into smartphones and tablets. It fits workloads where data locality, offline operation, and immediate response matter more than computational scale, such as voice recognition, computational photography,[25] health monitoring, and navigation or translation that must keep working without a network, and its battery and thermal envelope suit frequent, short-duration inference rather than sustained heavy computation.

The mobile environment elevates energy per inference to a primary design constraint: because a phone operates from a finite battery without active cooling, every joule consumed directly erodes device runtime and thermal headroom. While stationary cloud and edge deployments often focus on minimizing latency T under continuous utility or site power, mobile systems must strictly balance total energy (Energy $=$ Power $\times\, T$) under the power wall described by equation 2.2, which caps sustained dissipation at 3–5 W. In archetype terms, a compute-heavy workload such as image classification must be transformed into a more compact vision model,[26] reducing FLOPs by 13.7× while preserving target accuracy. The shift is qualitative: the design accepts lower peak throughput in exchange for sustainable operation within a 3–5 W thermal envelope.

> **Definition 2.4: Mobile ML**
>
> ***Mobile machine learning*** is the deployment paradigm bounded by thermal design power (TDP) and battery energy.
>
> 1. **Significance**: It is constrained by the few-watt heat dissipation capacity of passive cooling, requiring architectures that prioritize sustained energy efficiency over peak throughput (R_{peak}).

22 **Checkout-free retail**: A store may fuse camera observations with shelf events and transaction state locally, then transmit only selected events or aggregates. The privacy boundary must still include those derived records, because reducing raw-video transport does not determine retention, access, or secondary use.

23 **Industry 4.0**: The fourth industrial revolution integrates ML into the sensor-actuator feedback loop on factory floors. In an illustrative welding-robot scenario, defect detection samples at 60 Hz, giving a 16.7 ms sampling interval; if each response must finish before the next sample, the complete sensing-to-actuation path must fit that deadline. This completion requirement is a scenario assumption; the sampling rate alone does not establish the control deadline or guarantee safe operation.

24 **Predictive maintenance**: Models analyze sensor data such as vibration and temperature to estimate equipment condition and failure risk. Continuous monitoring makes sensing cadence, duty cycle, power, and local storage first-class constraints. Battery-powered nodes may require sub-watt average operation, whereas mains-powered industrial gateways can use a much larger envelope.

25 **Computational photography**: Uses ML algorithms (for example, multi-frame fusion, neural denoising) to overcome the physical limits of small mobile camera sensors. This exemplifies the mobile computing trade-off, as a multi-stage pipeline must execute within the user's perceived shutter delay while adhering to a shared 3–5 W thermal budget.

26 **Mobile vision model reduction**: MobileNet-style architectures reduce the computation in common vision layers while preserving useful accuracy for mobile tasks. The systems point here is that mobile deployment often requires changing the model family, not merely moving the same model to a phone.

2. **Distinction**: Unlike edge ML, which may have active cooling, mobile ML must operate within a personal energy budget. Unlike TinyML, it still provides a rich OS and multi-watt compute capacity.
3. **Common pitfall**: A frequent misconception is that mobile ML performance is a fixed value. In reality, it is a time-varying constraint: performance often drops as the device hits its thermal wall, triggering thermal throttling that lowers clock frequencies and reduces the active duty cycle.

Two boundaries close the mobile envelope, and together they push the design toward on-device NPU acceleration and model compression (figure 2.7). The first is the **battery tax**, the share of the phone's finite energy budget that an always-on ML feature spends before the rest of the system runs. The battery limits total energy over time.

Thermodynamics imposes a second boundary that holds even for a plugged-in tablet. Every watt becomes heat, and where a data center removes heat with active cooling, a thin mobile device relies on passive transfer through its chassis and surfaces. This creates the **thermal wall**, a hard ceiling on sustained power consumption that exists independently of battery capacity.

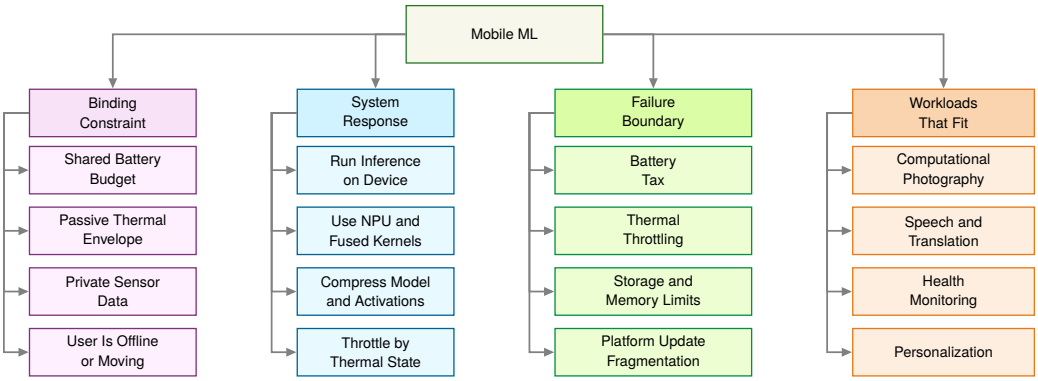


Figure 2.7: Mobile ML Constraint Map: On-device processing buys responsiveness, privacy, offline operation, and personalization, but the phone's shared battery and passive thermal envelope make sustained energy efficiency the binding design constraint.

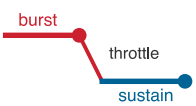
Napkin Math 2.9: The battery tax

Problem: Consider deploying a "real-time" background object detector on a smartphone. The model consumes 2 W of continuous power when active. The phone has a standard 15 Wh battery. Can the feature stay on all day? This is the battery tax that turns battery life into a mobile ML energy-budget constraint.

Physics:

1. **Ideal runtime**: $\frac{15\ \text{Wh}}{2\ \text{W}} = 7.5$ hours
2. **Reality**: A user expects their phone to last 24 hours. Running this single feature continuously for a day would require 320 percent of the phone's daily energy budget.

Systems insight: The model cannot simply be "deployed." A lighter model, duty cycling, or both must reduce the energy demand, so the feature can stay on all day.



Sustained thermal performance falls well below burst peaks.

The distinction matters for engineering decisions: the battery tax is a budget problem, solvable in principle by reducing how often the model runs or by increasing battery capacity. The thermal wall is a physics ceiling. A larger battery cannot raise the maximum sustained wattage a passive chassis can dissipate; duty-cycle and software changes can only reduce heat generation. Sustained operation above the thermal envelope triggers hardware throttling regardless of remaining battery

energy. The two constraints therefore attack different points in the iron law: the battery limits total operations per charge (O integrated over time), while the thermal wall caps the instantaneous rate ($R_{\text{peak}} \cdot \eta_{\text{hw}}$) the silicon can sustain.

Napkin Math 2.10: The thermal wall

Problem: An unoptimized vision model requires 12 W peak compute. Can it be deployed on a mobile device, or does it hit the thermal wall and the mobile power wall?

Scenario: A mobile system-on-chip (SoC) allows approximately 3 W for passive cooling. Assume the unoptimized model ships anyway. A representative throttling trace for this heavier model (a different and more demanding case than the power-wall example earlier in the chapter) runs in three steps:

1. **Temperature rise**: At 12 W, the device temperature rises at approximately 1 °C/s.
2. **Thermal trip**: Within 60 s, the hardware reaches the thermal trip point (80 °C), triggering OS throttling.
3. **Throttled throughput**: The 100 FPS model suddenly drops to 30 FPS to stay within the thermal envelope.

Math: Assume that a redesign (a lighter architecture running INT8 instead of FP32 arithmetic) cuts active power by an illustrative 4×: 12 W ÷ 4 = 3 W, which lands right at the 3 W passive-cooling ceiling with no headroom to spare. The factor is a scenario input for the whole redesign. Quantization alone shrinks weight storage by that factor but does not cut power proportionally, because switching and leakage energy outside the arithmetic remain.

Systems insight: Even a 4× power reduction does not create thermal headroom; it only just reaches the sustainable limit. Optimization can bring a model down to the thermal ceiling; it cannot raise the ceiling.

The battery and thermal examples establish why mobile deployments demand explicit energy accounting. With the macro-scale energy spectrum across paradigms established in figure 2.6, the sections that follow examine how mobile systems operate within these tight thermodynamic envelopes.

2.7.1 Mobile ML benefits and resource constraints

The mobile row of table 2.4 gives the hardware envelope: gigabytes of RAM, neural processing units[27] delivering tens of TOPS, and 3–5 W of sustained power on a system-on-chip[28] that integrates computation and memory to minimize data-movement energy. Memory capacity constrains model footprint and memory bandwidth constrains how quickly weights and activations move, but the battery sets the budget users notice. Adding 1 W of continuous ML processing to a phone that otherwise lasts 24 hours would cut its runtime to roughly 9.2 hours–11.5 hours, depending on a battery capacity of 15 Wh–22 Wh.

Mobile adds two gains to those every local tier shares. The first is hardware isolation for the most sensitive inputs: on-device enclaves such as Apple's Secure Enclave keep biometric matching inside a dedicated subsystem,[29] although the strength of any privacy guarantee still depends on the complete system design and threat model. The second is personalization, because models can exploit on-device signals and user context without shipping raw data anywhere. Specialized on-device ML frameworks keep such inference within 5–50 ms user-interface response times.

The price is paid in engineering effort as well as joules. Peak NPU throughput can substantially exceed what the device sustains under prolonged load, device heterogeneity requires multiple model variants, development multiplies across platforms, and app-store review and staged release slow iteration compared with a service its operator controls entirely.

2.7.2 Personal assistant and media processing

Across mobile applications, the central systems problem is that several short pipelines must share the same battery and thermal budget. Computational photography exemplifies the challenge of

[27] **Neural processing unit (NPU)**: A dedicated hardware block on a mobile system-on-chip whose circuits are designed for low-precision tensor operations. This specialization can improve energy efficiency relative to CPU execution, helping AI workloads fit within mobile power budgets.

[28] **System-on-chip (SoC)**: Mobile SoCs integrate CPU, GPU, and NPU cores with caches and memory controllers to reduce data-movement costs. Mobile DRAM is generally off-die, and feasible model size depends on capacity, precision, working memory, bandwidth, and latency.

[29] **Face ID**: Apple performs facial matching within the Secure Enclave and encrypts the stored mathematical representations so that they remain available only to that subsystem and do not leave the device (Apple 2024a, 2024b). Apple reports a random false-match probability below 1 in 1,000,000 for one enrolled appearance, with higher probabilities for some populations and configurations. The example shows how local inference and hardware isolation can reduce transmission exposure while supporting low-latency authentication.

running multiple ML pipelines within a thermal envelope. Modern flagships process each photo through multiple ML stages: portrait mode[30] uses depth estimation and segmentation, night mode aligns multiple frames with ML-based denoising, and HDR merging, super-resolution, and scene optimization run in sequence. The engineering challenge is not any individual model but the pipeline: these models must share a 3–5 W power budget and complete within the user's perceived shutter delay, requiring careful scheduling across CPU, GPU, and NPU to avoid thermal throttling.

Voice-driven interactions partition work across the same SoC by power tier: wake-word detection runs continuously on a dedicated low-power core, while speech recognition and keyboard prediction wake the higher-power neural processing tiers only when needed.

Health monitoring and augmented reality push mobile ML to its sustained-performance limits. Wearables like Apple Watch process ECG and accelerometer data on-device to reduce data transmission, while AR frameworks demand a 16.7 ms total frame budget at 60 FPS for simultaneous localization, hand tracking, and scene understanding. These applications represent the ceiling of what battery-powered, passively-cooled devices can sustain, and they define the boundary beyond which mobile optimization alone is insufficient.

These successes can create a misleading sense of ease. A common pitfall is deploying desktop-trained models directly to mobile or edge devices without architecture modifications. A desktop ResNet-50 pipeline may require gigabytes once activations, batches, preprocessing buffers, and runtime overhead are included, even though the FP32 weights alone are about 102.4 MB. Such a pipeline, requiring 8.2 GFLOP per inference, cannot run unchanged on a representative low-end edge target with 512 MB of RAM and a 1 GFLOP/s processor. Beyond simple resource violations, desktop-optimized models may use operations unsupported by mobile hardware, assume numerical formats unavailable on embedded systems, or require batch processing incompatible with single-sample inference. Successful deployment demands architecture-aware design from the beginning: model families, arithmetic formats, and implementation choices must all match the target device.

Mobile ML shows that useful intelligence fits within a 3–5 W thermal envelope on battery power. A smartphone still costs hundreds of dollars, carries gigabytes of memory, and needs a daily recharge, and no amount of mobile optimization closes the roughly 10,000-fold memory gap to a device that must cost dollars and run unattended for years. That regime needs a different class of hardware and algorithms.

2.8 TinyML: Ubiquitous Sensing

Imagine instrumenting every pallet in a warehouse, every cable on a suspension bridge, every beehive in an apiary. To put "eyes and ears" on this many physical objects, tens of thousands to millions, the device must cost dollars, not hundreds of dollars, and measure millimeters, not centimeters. Smartphones are far too expensive and too large; what is needed is ubiquitous sensing at unit costs under ten dollars and physical dimensions measured in millimeters.

TinyML completes the deployment spectrum by pushing intelligence to its physical limits: low-cost, low-power ML on deeply constrained embedded devices (Janapa Reddi, Plancher, et al. 2022). Devices costing under $10 and averaging less than one milliwatt[31] of power when duty-cycled make ubiquitous[32] sensing economically practical at massive scale; MLPerf Tiny and MCUNet show how such constraints are evaluated and optimized in practice (C. Banbury et al. 2021; Lin et al. 2020). This is the domain of the tiny constraint archetype, where the optimization objective shifts from maximizing throughput to minimizing energy per inference. With duty-cycled execution, a keyword spotting model consuming about 10 µJ per inference can operate for years on a coin-cell battery, trading model capacity for operational longevity. Figure 2.8 highlights this integration on an Arduino Nano 33 BLE Sense development board, where on-board microphone, motion, and environmental sensors share a single substrate with a low-power microcontroller executing quantized TensorFlow Lite models. The board illustrates integration; its full power draw depends on the sensors, radio, and duty cycle.

Operating at this physical scale requires microcontrollers (MCUs)[33] that eliminate the power-hungry infrastructure of conventional computing. Mobile and edge processors rely on external dynamic RAM (DRAM), high-speed memory buses, and operating system runtimes that draw hundreds of milliwatts simply to refresh memory cells and toggle bus lines. Many microcontrollers integrate the processor core, non-volatile flash, and static RAM (SRAM) onto a single silicon die,

30 | **Portrait Mode pipeline**: This is not a single model but a sequence of real-time models for depth estimation, segmentation, and rendering. The core engineering problem is managing the pipeline's aggregate latency and power, not any single model's performance. The entire pipeline must execute within the user's perceived shutter delay and share the phone's 3–5 W thermal budget, forcing scheduling trade-offs across the CPU, GPU, and NPU to avoid throttling.

↪ The framework that organizes pruning, distillation, and quantization to fit edge memory budgets is developed in section 10.1.

31 | **The 1 mW Threshold**: Ambient energy harvesting can power a device indefinitely only when long-term harvested power exceeds the complete system's average load. Available power varies across sources and environments—solar cells the size of a thumbnail may produce ~10 mW outdoors but ~10 µW indoors, thermoelectric generators on warm pipes ~100 µW, and RF energy from nearby transmitters ~10 µW. This crossover can transform the deployment model from "battery-limited lifetime" to "deploy and forget," but it depends on the source, environment, and workload.

32 | **Ubiquitous computing**: Mark Weiser's ubiquitous-computing vision imagined computation woven into everyday environments until it receded from direct attention (Weiser 1991). TinyML is one modern path toward that vision: when the cost and power of an intelligent sensor become low enough for mass deployment, the optimization objective shifts from performance (throughput) to power (energy per inference), the central trade-off of the tiny constraint archetype.

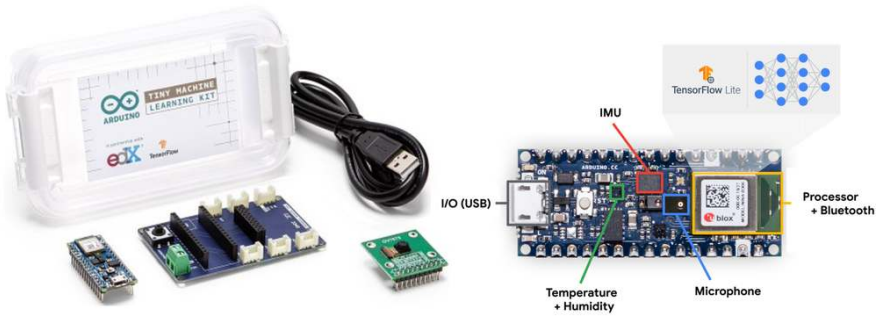


Figure 2.8: TinyML Hardware Scale: Microcontroller development boards integrate sensors, processing cores, and low-power radios onto a compact substrate, making constrained ML applications accessible for prototyping. Finished sensing devices must budget SRAM, sensor and radio power, inference cadence, and sleep energy to achieve multi-year battery life. Image source: Figure 6 in (Janapa Reddi, Plancher, et al. 2022), licensed under CC BY 4.0.

reducing reliance on external memory buses (C. Banbury et al. 2021; Lin et al. 2020; Janapa Reddi, Plancher, et al. 2022). Eliminating off-chip signaling lets duty-cycled average power fall below a milliwatt and sleep power reach microwatts, although active inference on boards such as the ESP32-S3 still draws tens to hundreds of milliwatts. Other targets, including ESP32-S3 variants, use external flash or additional RAM. On-chip integration still imposes a strict internal-memory ceiling. On-chip SRAM is physically constrained by silicon die area: because each SRAM cell requires six transistors (6T), expanding capacity directly inflates manufacturing cost. A TinyML model must therefore execute within tens to hundreds of kilobytes of SRAM, with no virtual memory, paging, or swap space to cushion an overflow. The small keyword-spotting task and low-power hardware place it at the bottom of figure 2.6; duty cycle also determines its average power.[34] Multi-year coin-cell operation additionally requires the complete device load to fit the battery budget.[35] Fitting the entire inference pipeline, from sensor acquisition to classification output, into this kilobyte-scale memory footprint requires formalizing the TinyML operating regime.

Definition 2.5: TinyML

TinyML is the machine learning domain of always-on sensing constrained by kilobyte-scale memory and milliwatt-scale power.

1. **Significance**: It necessitates models whose weights and code fit in flash while activations and runtime state fit in on-chip SRAM, enabling continuous inference on milliwatt power budgets.
2. **Distinction**: Unlike mobile ML, which uses multi-watt processors and a full OS, TinyML runs on microcontrollers using bare-metal firmware or a real-time operating system.
3. **Common pitfall**: A frequent misconception is that TinyML is just "small models." In reality, it is an energy-bound paradigm: the primary metric is energy per inference (microjoules), not just the parameter count.

This definition reframes the iron law of ML systems. In terms of equation 2.5, TinyML imposes a physical memory-capacity precondition that execution time does not capture: the weight footprint M_{weights} and activation footprint $M_{\text{activations}}$ must stay within memory capacity M_{device}. When total memory is measured in kilobytes, model structure and runtime allocation determine feasibility before latency optimization begins, and every avoidable data movement spends part of the energy budget. The objective therefore shifts from raw throughput toward energy per inference and memory fit.

The same four-branch map applies at the bottom of the spectrum (figure 2.9). Milliwatt budgets and kilobyte memories favor low-precision execution and duty-cycled sleep, and they move the failure boundary away from latency toward compression-driven accuracy loss and risky firmware updates.

[33] **Microcontroller (MCU)**: A single-chip computer whose design prioritizes minimal cost and power over performance, so memory fit becomes a deployment prerequisite. Many TinyML targets provide only tens to hundreds of kilobytes of on-chip SRAM and no virtual memory; some higher-end microcontrollers can attach external memory. The complete model, activations, and runtime state must therefore fit the memory actually available to the target, often orders of magnitude less than a smartphone provides.

[34] **TinyML energy gap**: The figure compares different tasks on different hardware and with different energy accounting boundaries; its ratios do not identify the effect of hardware integration alone. A batch-1 ResNet-50 inference on a cloud accelerator consumes about 75.5 mJ of accelerator energy, while an always-on keyword spotter uses about 10 µJ, a gap of ~7,500×. A cloud LLM query at about 1 kJ widens the gap to 100,000,000×, the span plotted in figure 2.6.

[35] **Coin-cell deployment**: A CR2032 battery (225 mAh at 3 V, ~675 mWh) ideally powers a complete 10–50 µW system for about 1.5–7.7 years. This "deploy-and-forget" operating model drives innovation in intermittent computing, where the device sleeps between inferences to stretch the energy budget across years of unattended operation.

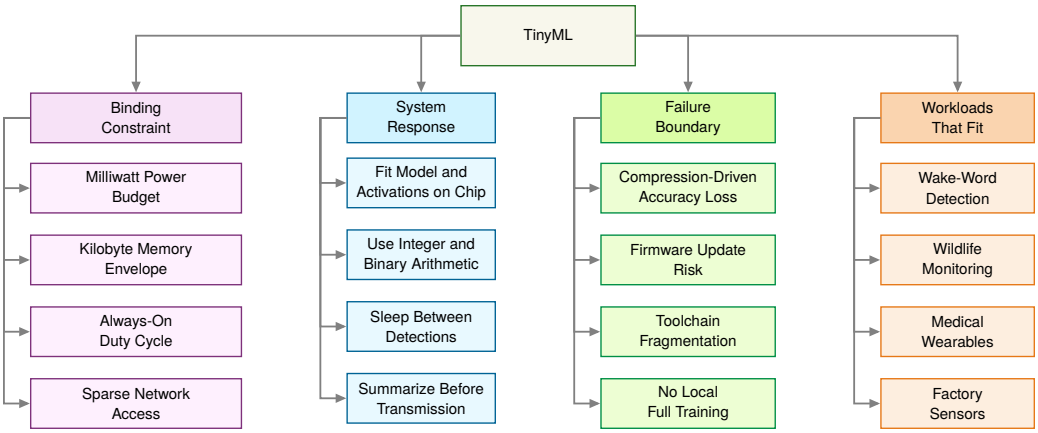


Figure 2.9: TinyML Constraint Map: Milliwatt power and kilobyte memory make always-on sensing economically possible, but those same limits force the model, activations, runtime, and update path to fit a radically smaller engineering envelope.

2.8.1 TinyML advantages and operational trade-offs

TinyML devices range from palm-sized developer kits to millimeter-scale chips,[36] and their kilobyte memories and milliwatt budgets buy months or years of autonomous operation[37] at the price of specialized algorithms, aggressive model compression, and careful systems co-design.

Beyond the locality gains every local tier shares, TinyML changes deployment economics. Removing the radio uplink and the application processor lets per-node unit cost fall to single-digit dollars, which makes dense instrumentation across thousands of physical assets financially viable, and microjoule inference budgets allow multi-year operation on inexpensive coin-cell batteries or on harvested ambient energy.

The trade-offs are just as distinctive. Microcontrollers commonly provide 10^5 to 10^6 bytes of RAM and enforce a strict two-tier memory partition: static model weights and compiled code reside in non-volatile flash (often a few megabytes), while mutable activations and the runtime buffer (the *tensor arena*) must fit within the much smaller on-chip SRAM (a few hundred kilobytes at most, such as the ESP32-S3's 512 KB, and as little as tens of kilobytes on the smallest parts). As a result, peak intermediate activation volume, rather than static parameter storage, is frequently the barrier that prevents a model from executing. The smart doorbell's level-1 check (figure 2.2) shows why that check is only a first screen: its weights fill just 11.9 percent of flash, a figure that says nothing about whether its intermediate results fit in SRAM. Embedded development also demands cross-stack co-design: engineers manage memory pools by hand, debug bare-metal firmware without operating-system abstractions, and optimize operator kernels directly for constrained instruction sets.

Operating under these physical bounds introduces unique lifecycle and reliability challenges. An energy-harvesting device can satisfy its long-term average power budget yet fail when stored energy is exhausted before the next harvest. Its duty cycle must therefore respect both energy per inference and the timing of available power. Model quality can also suffer from aggressive quantization and pruning: converting 32-bit floating-point weights to 8-bit integers (INT8) saves memory but can degrade accuracy and robustness on out-of-distribution sensor inputs. Furthermore, post-deployment flexibility is sharply constrained: models are often packaged with firmware, so field updates must account for limited storage, interrupted transfers, and safe rollback. Ecosystem fragmentation[38] across microcontroller vendors and ML frameworks creates additional overhead and portability challenges.

2.8.2 Environmental and health monitoring

The TinyML regime becomes essential when power budgets, unit economics, and physical isolation rule out every higher tier. Wake-word detection is the most familiar case: the always-on detector acts as a power gate that wakes the costlier processors only when a wake phrase is heard.[39] Precision

[36] **TinyML device range**: This physical range reflects a direct trade-off between deployment context and computational capability. Millimeter-scale systems prioritize minimal power for single-function, long-duration tasks, whereas palm-sized boards offer more sensing, interfaces, and debugging support at higher cost and power. Development-board size and power therefore should not be treated as limits for a finished TinyML product.

[37] **On-device training constraints**: Updating a model requires working memory beyond that needed to apply it, including information retained from the prediction computation to determine parameter changes. Intermediate results can be discarded only after their final use, which often occurs later during training than during inference. With only 256 KB–2 MB RAM, many microcontrollers cannot hold the full training state; specialized methods reduce that state by limiting which parameters change. TinyTL, for example, freezes the weights and updates mainly the biases, so most intermediate activations need not be retained for the update (Cai et al. 2020). Training elsewhere and delivering an updated model is therefore a common design, although constrained local adaptation remains possible.

agriculture applies the same locality pressure at farm scale. FarmBeats used sensors, cameras, drones, and local gateway processing to reduce raw data movement where connectivity was costly (Vasisht et al. 2017), and TinyML pushes that logic down to sensing nodes that must run under milliwatt budgets.

Wildlife conservation shows the uplink arithmetic. Solar-powered acoustic sensors identify species on-device and transmit periodic detection summaries rather than raw audio, cutting uplink data from 4.3 GB/day to 400 KB/day, a 10,750× reduction that conserves power in off-grid deployments. Medical wearables apply the same logic to physiological monitoring, where runtime remains bounded by sensor draw, radio power, duty cycle, and sleep energy rather than by the model alone.

These four paradigms form an operational continuum bounded by the speed of light, heat dissipation, and memory signaling. When architecting an ML system, engineering begins by systematically mapping application constraints against these physical boundaries.

2.9 Paradigm Selection

An architect choosing where to run an ML feature rarely optimizes one dimension. A privacy rule may forbid cloud processing, a latency budget may forbid remote inference, a memory footprint may exceed the mobile device, and a cost target may rule out always-on edge hardware. Cloud, edge, mobile, and TinyML are operating envelopes for those conflicts, so selecting among them requires a unified comparison framework and a structured decision process.

2.9.1 Comparative trade-off analysis

Deployment decisions require evaluating latency-vs-throughput paradigm trade-offs across physical and operational envelopes at once. The resource envelopes are already on the table: table 2.1 gives each paradigm's latency, power, and memory regime, and table 2.4 grounds them in concrete platforms. What those tables leave out are the operational dimensions that decide many real deployments: how the system scales, what it exposes, whether it needs a network, what it costs, and how hard it is to build and ship. Table 2.9 contrasts the four paradigms along those eight dimensions.

Table 2.9: Operational Paradigm Comparison: Eight operational dimensions that complement the resource envelopes of table 2.1 and table 2.4. Cloud ML scales most easily, while local processing can reduce transmission and centralized-storage exposure without itself guaranteeing privacy; local tiers can operate without a WAN, but real-time response depends on workload and complete-path timing, while development complexity is highest at the two ends of the spectrum. Cost anchors mix recurring service bills with device purchase prices; they are partial illustrations, not a matched lifecycle-cost ranking.

Aspect	Cloud ML	Edge ML	Mobile ML	TinyML
Scalability	High (provider capacity limits)	Good (limited by edge hardware)	Moderate (per-device scaling)	Limited (fixed hardware)
Data Exposure	Remote processing expands the surface	Local processing reduces transmission	On-device processing reduces transmission	Raw data can remain local
Connectivity Required	Required for remote requests	Intermittent	Optional	None
Offline Capability	None	Good	Excellent	Complete
Real-time Processing	Network and execution must fit deadline	Local execution must fit deadline	Deadline and sustained-power limits	Deadline and memory-capacity limits
Cost anchor	Service bill ($1000s+/month)	Hardware purchase ($100s–$1,000s per device)	Device purchase ($200–$1000+; often user-owned)	Hardware purchase ($1–$10 per device)
Development Complexity	High (cloud expertise needed)	Moderate-High (edge+networking)	Moderate (mobile SDKs)	High (embedded expertise)
Deployment Speed	Fast	Moderate	Fast	Slow

This contrast reflects the fundamental tension between data locality and computational scale. Remote processing expands the network attack surface and requires robust transport encryption

38 **TinyML ecosystem fragmentation**: TinyML spans microcontroller families such as Arm Cortex-M, RISC-V, and Xtensa, each with different instruction sets, memory layouts, kernels, and vendor toolchains; cloud and mobile stacks also vary, but their runtimes usually hide more of this hardware diversity. A model optimized for one microcontroller target may therefore require retuning and revalidation for another, multiplying the engineering cost of multi-device deployment.

39 **Always-on wake-word detection**: A small, specialized model listens for the acoustic signature of a wake phrase and acts as a power gate for the higher-power application processor. The acceptable detector budget depends on the platform, but spending even a few extra milliwatts continuously can materially shorten battery life. The architecture succeeds only when the always-on path consumes far less average power than the components it prevents from waking.

and tenancy isolation; local processing confines raw data to the device, mitigating ingress exposure without eliminating physical side-channel risks. The workload archetypes from section 2.3 map directly to these boundaries, with each archetype gravitating toward paradigms that alleviate its primary bottleneck.

Each paradigm shifts engineering complexity to a distinct subsystem. Cloud deployments demand expertise in multi-tenant networking and dynamic cost governance. Edge and TinyML deployments impose local thermal and memory limits while distributing update and maintenance responsibilities across sites. Mobile deployments operate on user-owned hardware, reducing infrastructure purchase costs while introducing platform-specific drivers, operating-system background task limits, and shared energy budgets. None of these categories supplies a workload-independent latency or energy ranking.

A common architectural failure is selecting a deployment target based on model accuracy alone while treating hardware constraints as downstream optimization details. An unquantized model that delivers 99 percent accuracy in an offline evaluation becomes unviable if its memory footprint triggers paging on mobile hardware, or if continuous inference drives thermal throttling within seconds. Physical invariants—memory capacity, thermal dissipation limits, and network transit delay—must serve as early screening criteria before model development begins, pruning invalid deployment targets before training resources are expended.

2.9.2 Decision framework

Before choosing a deployment tier, an engineer must establish whether the feature justifies machine learning at all. Deep neural networks impose a substantial maintenance burden that traditional software avoids. Before committing to any ML architecture, an engineer must evaluate whether a deterministic heuristic can solve the problem at a fraction of the operational cost.

> **Systems Perspective 2.3: The complexity tax**
>
> Consider a classification task solvable either by deterministic heuristic rules or by a deep learning pipeline. The heuristic requires fifty lines of code, incurs near-zero compute cost, demands approximately one hour per month to update rules, and fails in ways engineers can read in the code. The ML system may require only fifty lines of model definition, but it introduces roughly 2,000 lines of supporting infrastructure for feature ingestion, telemetry logging, drift monitoring, and GPU driver management, alongside 40 hours per month diagnosing data drift and maintaining pipelines.
>
> An ML model that increases accuracy from 90 percent to 95 percent while introducing a 40× expansion in operational maintenance requires an economic comparison. The accuracy and maintenance ratios alone do not establish the return; the value of the avoided errors must justify the additional engineering cost. Machine learning systems engineering focuses on minimizing this overhead through resilient architectures; when the ongoing maintenance cost outstrips the value of marginal statistical gains, deterministic software remains the correct systems choice.

This trade-off reframes model selection as a systems-level optimization spanning physical execution limits, capital expenditure, and the ongoing maintenance of prediction quality. When this **complexity tax** outweighs the operational value of improved statistical predictions, maintaining an ML deployment actively degrades overall system efficiency.

Systematic paradigm selection relies on a tiered decision framework ordered by how rapidly a constraint invalidates an architecture. The screening hierarchy evaluates four successive gates: regulatory privacy, physical latency, local compute capacity, and lifecycle economics. Privacy leads the order because a legal prohibition on moving data eliminates an option before anything is measured, whereas the latency and compute gates require numbers. Figure 2.10 formalizes this screening sequence as successive filters on a candidate set. Each gate passes only its survivors to the next.

The flowchart carries a remaining candidate set through all four gates. If regulatory rules or data sovereignty forbid off-device transmission, remote cloud processing stays excluded when compute demand is considered. If a network round-trip lower bound exceeds the deadline, additional remote

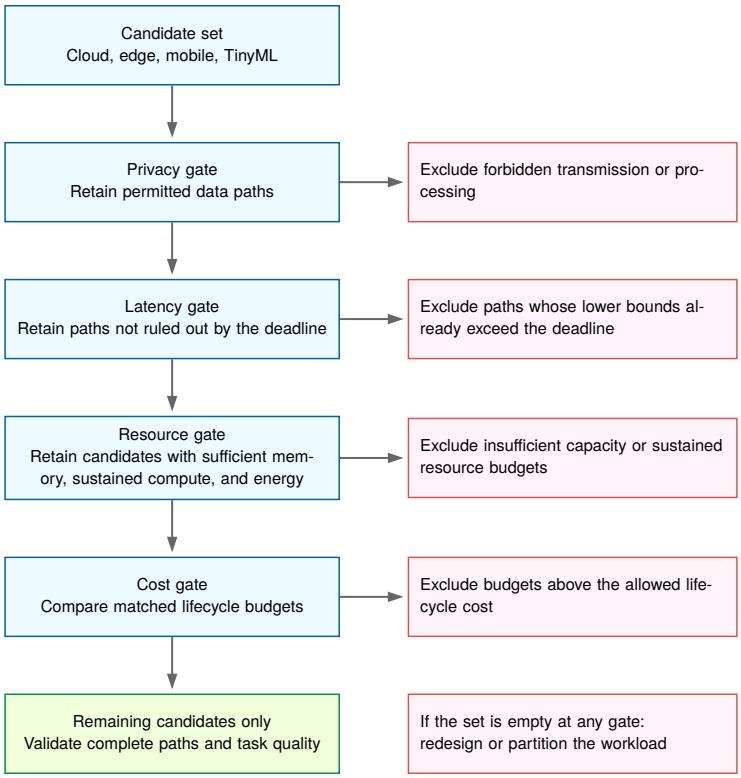


Figure 2.10: Deployment Decision Screening: Apply privacy, latency, resource, and cost gates to the remaining candidate set. Every gate retains upstream exclusions. An empty set requires redesign or partitioning; surviving candidates still need complete-path validation.

compute cannot restore that path. Cost compares only the survivors, and an empty set calls for changing the workload or partitioning it across tiers. Passing a lower-bound latency screen still requires complete-path validation. An automotive emergency braking scenario demonstrates how the latency gate binds before compute capacity or cost enter consideration.

Example 2.1: Autonomous vehicle emergency braking

Scenario: An autonomous vehicle vision system performs real-time pedestrian detection for emergency braking under a strict 100 ms end-to-end latency budget.

Diagnosis: At 100 km/h, the vehicle covers 2.8 m during the 100 ms budget, so every millisecond of delay is distance traveled before braking begins. The scenario assumes 50–150 ms of network round-trip delay for cloud offload, which consumes half the budget at best and more than all of it at worst before any detection runs. Local object detection on an automotive edge accelerator takes 10–30 ms; at 10 GFLOPs per frame and 30 FPS, that accelerator must sustain 300 GFLOP/s.

Systems lesson: The stated network-delay range cannot guarantee the deadline, so the latency gate excludes that cloud path. The local candidates remain subject to the resource gate: the pedestrian detector must fit memory and sustain the stated frame rate within its power budget. An automotive edge accelerator remains a candidate under these assumptions; its complete sensing-to-braking path still needs validation. Cloud connections can support offline training and telemetry without reentering the emergency decision path.

A completed placement decision needs evidence about the surviving paths as well as grounds for rejecting the others. A factory inspection example supplies that evidence explicitly.

Example 2.2: Completing a placement decision

Problem: A fixed factory camera requires 20 FPS inspection with a 100 ms sensing-to-decision deadline. Raw images must remain on premises. Which feasible deployment has the lowest matched lifecycle cost under the following hypothetical evidence?

Supplied evidence: A rule-based baseline fails the specified missed-defect criterion, while the learned detector meets both the missed-defect and false-alarm criteria on representative inspection data. The proposed mobile and microcontroller variants fail that same quality requirement. Local edge candidates A and B preserve the validated detector and preprocessing. Both have a complete peak working set of 2 GB within an available 4 GB, and each must stay within a 25 W sustained system-power budget.

Under the specified peak load and thermal conditions, the supplied complete-path latency bounds are 80 ms for A and 40 ms for B, including acquisition, preprocessing, queuing, inference, and the inspection decision. Their sustained rates are 25 FPS and 40 FPS, at 10 W and 20 W. Both satisfy the same availability and recovery requirements. Annualized budgets over the same service lifetime, including hardware, power, installation, maintenance, and recovery provisions, are $2,400 for A and $3,600 for B. These supplied bounds and budgets define an instructional scenario, not benchmarks of named products.

Result: Select local edge candidate A. Privacy excludes raw-image cloud offload, task quality excludes the proposed smaller variants, and both edge paths satisfy the timing, throughput, memory, and power requirements. A has the lower matched cost. B's faster response supplies no required benefit under this specification; a tighter deadline could change the choice.

Systems lesson: The final choice follows from task quality and complete-path evidence, not the tier label or peak compute rate. The decision remains valid within the supplied operating conditions; changes to the input stream, detector, load, or thermal environment require reevaluating the relevant gates.

The completed comparison depends on operational maintainability as well as execution performance. Its matched budgets include obligations that a hardware-only comparison would omit. Deploying a model across thousands of edge nodes or millions of microcontrollers introduces sustained lifecycle overhead: automated firmware updates, out-of-band telemetry, distributed drift monitoring, and hardware failure recovery. An organization equipped for cloud operations may lack the embedded toolchains and physical testing infrastructure required to support edge or TinyML targets at scale.

Checkpoint 2.2: System design

First justify the learned system against simpler baselines; then screen its deployment candidates against the physical and operational constraints.

Decision gates

- ☐ **The baseline**: Can you distinguish a rule-based baseline, such as a regular expression, from a simple learned baseline, such as a small model trained on a handful of features, and explain what comparison each provides?
- ☐ **Physical feasibility**: If a remote path's propagation floor exceeds the service deadline, can you reject it and identify which local execution and memory measurements are still needed before choosing an alternative?
- ☐ **The infrastructure cost**: What evidence about error consequences and engineering cost is needed to decide whether a five-point accuracy gain (90 to 95 percent) justifies a 40× increase in monthly maintenance hours?

The decision framework screens candidate tiers for physical feasibility and then compares their operational costs. If no single tier meets the requirements of the whole pipeline, the next question is which stages can run separately, and what must cross the boundaries between them.

2.10 Hybrid Architectures

A voice interface can use a microcontroller to listen continuously for a wake word, a phone to transcribe speech, and cloud servers to interpret requests that need a larger model. Each stage has a different task and resource budget. Sending every audio sample to the cloud would add communication and radio costs; asking the microcontroller to perform every stage would exceed its compute and memory capacity. *Hybrid architectures* divide the pipeline across tiers so that each stage meets its requirements. The division also creates engineering work: deciding what data crosses each boundary and keeping the participating stages consistent. Three integration patterns organize these choices.

2.10.1 Integration patterns

The three essential hybrid ML patterns differ by which physical boundary dictates the partition: train-serve split, hierarchical processing, or progressive deployment. Selecting a pattern is an iron-law decision: each stage of computation must execute where its binding term—whether throughput capacity R_{peak}, local latency floor L_{lat}, or memory capacity—incurs the lowest system penalty.

Under a **train-serve split**, training executes in centralized data centers while inference runs locally on edge, mobile, or embedded devices. This decoupling resolves the fundamental asymmetry between model optimization and model execution. Training must retain intermediate results and information used to calculate and apply parameter updates, in addition to the model weights. Under common 16-bit mixed-precision training with a standard adaptive optimizer, the weights, their gradients, and the optimizer's running statistics together occupy about 16 bytes per parameter, roughly 8× the 2 bytes per parameter that 16-bit inference weights need, before any batch activations are counted (section 8.5.3.2 develops this accounting). Inference uses the trained weights to compute predictions, sometimes through repeated passes as in token-by-token generation, and must satisfy local latency and privacy constraints. Amortization underpins the economics of this split: the high fixed cost of training is distributed across millions of forward-pass queries served locally without recurring network transmission costs.[40]

[40] **Train-serve cost asymmetry**: Training usually requires many forward and backward passes, whereas one inference request uses a trained artifact for one or more forward passes. Training is not literally one-time because models may be retrained, and inference cost varies sharply with model size, sequence length, batching, and service overhead. The economic rationale for the split is amortization: repeated use can spread each training cycle's fixed cost across many served predictions.

↪ Memory accounting for optimizer states, gradients, and activation buffers during training is developed in section 8.5.3.2.

Definition 2.6: Hybrid ML

Hybrid machine learning is the deployment strategy that distributes an ML pipeline across two or more deployment tiers, assigning each stage according to its latency, compute, data, and operational constraints.

1. **Significance**: Hybrid architectures exploit the iron law's additive structure: the edge tier can reduce L_{lat} for time-sensitive preprocessing and inference, while the cloud tier can provide the R_{peak} needed for training, retraining, and heavy batch inference. The split follows the data locality invariant: compare each stage's complete local and remote paths, reject paths that miss the service deadline, and then choose among the feasible alternatives.
2. **Distinction**: Unlike cloud-only deployment (which accepts the distance penalty for all stages) or edge-only deployment (which accepts limited R_{peak} for all stages), hybrid ML assigns each pipeline stage to the tier where its binding iron law term is minimized.
3. **Common pitfall**: A frequent misconception is that hybrid ML is just "running two models." In reality, the participating tiers must share synchronized state—feature definitions, model versions, and preprocessing logic—so that their paths produce consistent results. Without this synchronization, training-serving skew emerges at tier boundaries.

Under **hierarchical processing**, computation cascades across tiers to progressively filter and aggregate data, matching communication volume to available interconnect bandwidth. Sensor

streams generated at the physical periphery often exceed wide-area network (WAN) uplink capacity; continuously streaming raw, uncompressed telemetry to a data center saturates network links and wastes radio power. To prevent network saturation, early tiers filter data locally: TinyML microcontrollers run low-power anomaly detectors, edge gateways aggregate and correlate temporal features across multiple sensors, and cloud clusters ingest only anomalous events or aggregated summaries for global analytics. During inference serving, hierarchical processing frequently takes the form of a confidence cascade. A compact, low-latency model on the local device evaluates the input first. If its prediction confidence exceeds a calibrated confidence threshold, the device returns the result immediately, bypassing the network entirely. Only ambiguous or low-confidence inputs trigger a WAN payload transfer to a larger cloud model. The cascade seeks lower median latency at the required task quality, but confident local errors can bypass escalation. Representative evaluation must establish the routed system's accuracy, error costs, and escalation frequency before accepting that trade-off.

Under **progressive deployment**, an engineering team derives a family of specialized model variants from a unified architecture or training corpus, tailoring each variant to the memory capacity, arithmetic intensity, and thermal limits of a specific tier. Rather than forcing a single model onto every device, developers produce tier-specific artifacts and evaluate the quality lost or retained as resource demands fall. The compression mechanisms are developed later; the placement decision here depends on whether each artifact meets its task and hardware constraints. For a shared image-classification task, an edge server and a phone can use different members of a model family, each evaluated against the same task requirements and its own resource budget. A voice interface that divides wake-word detection, acoustic processing, and language understanding among tiers instead illustrates hierarchical processing across different tasks. Such a system can also use tier-specific variants within a task, combining the patterns.

With three integration patterns available, selection becomes a constraint-matching problem: choose the pattern whose trade-off profile matches the system's dominant bottleneck. Table 2.10 summarizes the trade-off, the conditions that favor each pattern, and the conditions that argue against it.

Table 2.10: Hybrid Pattern Selection Guide: The trade-off, favoring conditions, and disqualifying conditions for each of the three hybrid integration patterns. Selection matches the pattern's trade-off profile to the system's dominant bottleneck.

Pattern	Trade-off	Choose when	Avoid when
Train-Serve Split	Training cost vs. inference latency	Training requires more resources than inference; training data may leave the serving device	Local adaptation cannot wait for remote training and model delivery
Hierarchical Processing	Local autonomy vs. global optimization	Data volume exceeds transmission capacity; decisions needed at multiple timescales	All processing can occur at one tier; network is reliable and fast
Progressive Deployment	Model quality vs. deployment reach	Same model needed at multiple capability levels; graceful degradation required	Model cannot be meaningfully compressed; single deployment target

Production deployments rarely implement these patterns in isolation; complex systems compose them across different phases of the pipeline. An autonomous vehicle, for example, combines progressive deployment to fit specialized vision backbones into its on-board compute envelope, hierarchical processing to arbitrate between immediate millisecond-scale emergency braking on local hardware and multi-second trajectory planning, and a train-serve split that uploads anomalous driving slices to cloud clusters for fleet-level retraining. Privacy-preserving distributed training approaches extend this composition further when raw training data cannot leave client devices for regulatory or privacy reasons. In federated learning, for example, each device trains on its own data and sends only model updates to the cloud for aggregation.

2.10.2 Production system integration

Production hybrid systems coordinate multiple execution tiers through structured interaction channels. Figure 2.11 diagrams this multi-tier topology across five distinct communication pathways:

compiled model artifacts deploy downward from cloud training clusters to edge, mobile, and TinyML inference engines, while raw sensor streams and filtered telemetry propagate upward toward cloud analytics, complemented by peer-to-peer result forwarding and local assist handoffs.

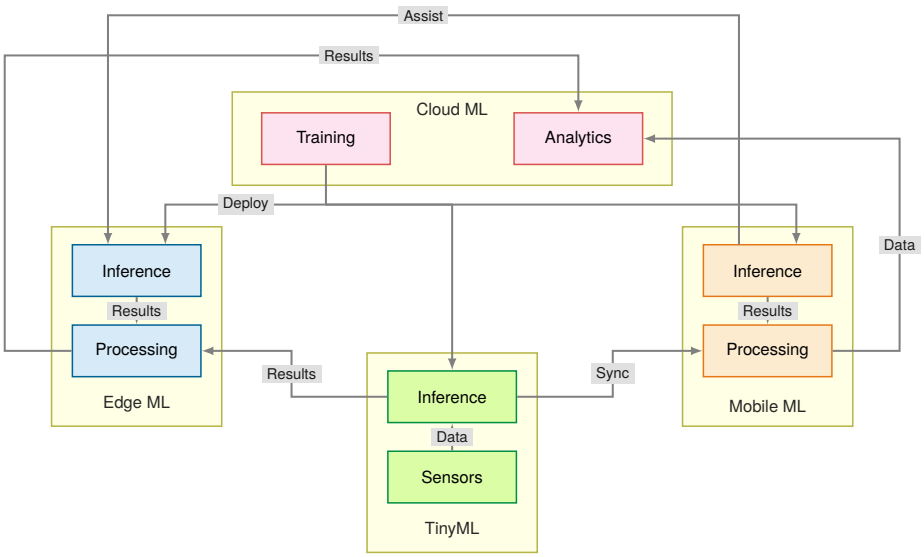


Figure 2.11: Hybrid System Interactions: Data flows upward from sensors through processing layers to cloud analytics, while trained models deploy downward to edge, mobile, and TinyML inference points. Five connection types (deploy, data, results, assist, and sync) establish a distributed architecture where each paradigm contributes unique capabilities.

Production systems demonstrate these integration patterns by placing each tier boundary at a distinct physical bottleneck:

1. **Industrial defect detection**: Assembly lines running at high cycle rates require sub-10 ms deterministic visual inspection. Streaming high-resolution camera frames over WAN to a data center violates inspection deadlines and exhausts factory uplink bandwidth. The system deploys a train-serve split: centralized cloud infrastructure uses multi-accelerator pods to train high-capacity vision models across historical defect databases, then compiles and quantizes runtime weights for factory-floor edge servers and embedded smart cameras. These on-premises accelerators read frames over local buses (a PCIe card slot or a direct camera interface) and execute inference there without external network dependencies. Bounded inspection latency still requires validating the complete local pipeline, including frame acquisition, preprocessing, and inference, under the expected load.
2. **Agricultural monitoring**: Deployed across hundreds of hectares, remote soil probes operate on coin-cell batteries or small solar harvesters, communicating over low-power wide-area radios such as LoRa, which carry little data and spend much energy per bit. Streaming continuous moisture and temperature telemetry exhausts battery reserves within days. Hierarchical processing confines high-frequency sampling and basic anomaly detection to TinyML microcontrollers operating in sub-milliwatt power budgets. When anomalies occur, local edge gateways aggregate measurements across spatial clusters of probes, transmitting only compressed, field-level summaries over cellular backhaul to cloud servers responsible for farm-wide irrigation scheduling.
3. **Wearable fitness tracking**: A battery-powered smartwatch must keep its always-on sensing path averaging under 1 mW to preserve battery life. The system employs hierarchical processing: lightweight integer kernels run on an embedded microcontroller to track real-time motion and detect heart-rate thresholds; intermediate biometric data synchronizes over Bluetooth Low Energy (BLE) to a paired smartphone, which uses mobile neural engines and gigabytes of DRAM to classify complex activity patterns; finally, the smartphone transmits periodic

compressed summaries over Wi-Fi or cellular networks to cloud infrastructure for longitudinal health modeling.

2.10.3 Why hybrid approaches work

Although the four deployment tiers differ by orders of magnitude in available energy, memory capacity, and compute throughput, they rest on identical systems foundations. Every tier moves data through a pipeline, manages a fixed budget of compute, memory, energy, and network capacity, and binds models to hardware through a software architecture. On those three shared foundations rest the same cross-cutting concerns of efficiency, monitoring, and trustworthy engineering (figure 2.12).

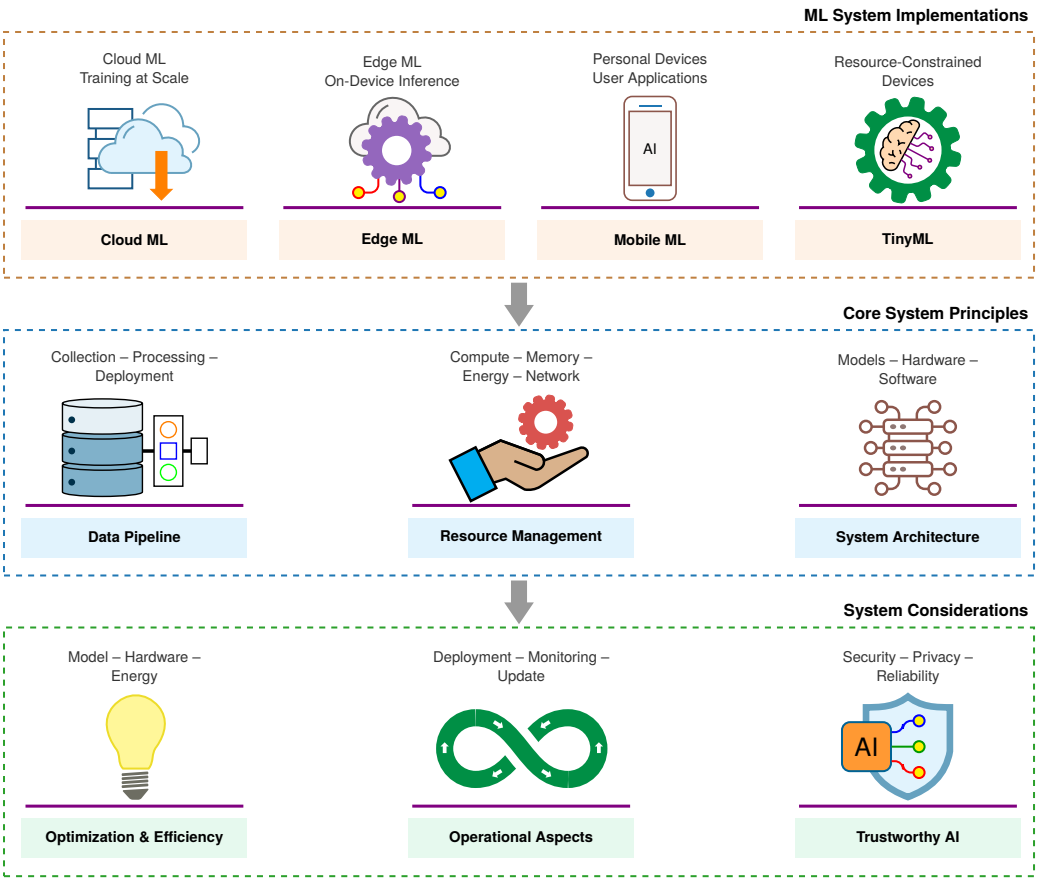


Figure 2.12: Convergence of ML Systems: Despite vast resource disparities across cloud, edge, mobile, and TinyML tiers, all deployments converge on three shared systems foundations: data pipelines, resource management, and hardware-software architecture. These shared pillars in turn support unified requirements for performance optimization, operational monitoring, and trustworthy engineering.

This architectural convergence explains why engineering optimizations transfer across hardware scales: they target the same underlying physical bottlenecks. In representative hardware, fetching operands from off-chip memory can cost orders of magnitude more energy than the arithmetic performed on them, as the memory-wall discussion established. Techniques that reduce those transfers help both an embedded microcontroller and a warehouse-scale accelerator. Lower-precision numerical representations (such as FP8, INT8, and INT4) reduce storage and transfer demands in both local devices and cloud clusters. Halving the bits per value can halve the bytes transferred for those values, and supported formats can also raise arithmetic throughput; prediction quality and complete memory traffic still determine the useful gain. Similarly, kernel fusion (executing a chain of operations in one pass so that intermediate results stay on chip) and static memory staging

can reduce off-chip traffic on mobile SoCs and HBM traffic during cloud autoregressive decoding. Graph-partitioning techniques also help assign pipeline stages across the CPU, GPU, and neural processing units of a mobile SoC, just as they help divide work across cloud workers.

The shared systems foundation extends through data engineering, model compression, and operational lifecycles. These engineering principles apply whether the execution target is a multi-node TPU pod or an ultra-low-power ESP32 microcontroller. Because the foundational abstractions remain invariant, so do their vulnerabilities: data distribution drift, model staleness, and telemetry degradation recur at every tier, requiring uniform monitoring discipline across the entire hybrid hierarchy.

Checkpoint 2.3: Hybrid ML patterns

Hybrid architectures work when partitioning work across tiers, not when copying the same pipeline everywhere.

Integration patterns

- ☐ **Train-serve split**: Can you explain why training in the cloud and serving on edge/mobile is often economically optimal, even when the model runs locally?
- ☐ **Hierarchical processing**: Can you describe what each tier does in a sensor → edge → cloud pipeline, and why pushing some decisions down reduces both latency and bandwidth?
- ☐ **Progressive deployment**: Can you explain why a shared task may require different model artifacts on different tiers, and which quality and resource measurements would establish their suitability?

Design sanity checks

- ☐ **Boundary choice**: Given a concrete application, can you justify where the tier boundary should fall (latency, privacy, bandwidth, power), not just what model to use?
- ☐ **State synchronization**: Can you identify which feature definitions, model versions, and preprocessing logic must remain synchronized across tiers?

2.11 System Entropy: Why Deployment Is Not the End

Across all four deployment tiers mapped into the convergence hierarchy of figure 2.12, correct execution does not establish reliable predictions on a changing population. A fixed model can return the same output for the same input while its accuracy over incoming inputs changes. Consider an unchanged deployment in which the algorithm (A) and machine (M) remain fixed: the model weights reside in memory, and the runtime executes the same compute graph on every forward pass. The operational data distribution (D), however, evolves as user behaviors change, sensor calibrations drift, and environments fluctuate. Because model parameters represent an empirical fit to a fixed training distribution P_0, changes in the live distribution P_t can reduce prediction quality. The introduction called the result silent degradation and its cause data drift. Seen at the level of a whole deployed system, the risk of accumulating degradation is what this chapter calls **system entropy**, a metaphor for declining predictive reliability even when code, model weights, and execution environments remain byte-for-byte identical. It is not a thermodynamic law or a claim that every distribution shift reduces accuracy.

The degradation equation (equation 1.2) provides a local approximation to this decay: where outcome evidence supports the relationship, accuracy falls from its deployment value Accuracy_0 in proportion to the statistical distance $\mathcal{D}(P_t \| P_0)$ between the live and training distributions, scaled by the model's sensitivity λ. Because inference engines report success as long as matrix multiplications finish without numeric exceptions or memory faults, statistical degradation remains invisible to traditional infrastructure monitoring. An unmonitored model can continue generating confident predictions while its operational utility erodes. Distribution drift alone does not establish that loss. Safeguarding system reliability therefore requires telemetry to trigger investigation, outcome

evidence to confirm degradation, and diagnosis to determine whether pipeline repair or a model update is warranted, an operational discipline analyzed in chapter 14.

↩ The divergence between static code binaries and non-stationary production distributions is established in section 1.6.

↪ Drift detection that compares live feature distributions against the training baseline with statistical distance measures is operationalized in section 25.

↪ The full Zillow Offers case, read as a coupled-decision failure, is developed in section 14.3.6.1.

When predictions drive automated actions, undetected errors can accumulate into physical or financial losses. In 2021 Zillow wrote down $304 million in housing inventory after its Zillow Offers home-buying business purchased houses at prices above its own revised estimates of their future selling prices (Zillow Group 2021). Zillow's account does not attribute the losses to drift specifically, and it does not establish which operational safeguards were absent. The systems lesson is the exposure created when uncertain forecasts drive large purchase commitments: prediction error can become inventory loss rather than remain an evaluation metric.

The Zillow Offers failure illustrates that a machine learning system encompasses both the predictive model and the operational feedback loop that acts on its outputs. Treating deployment as a terminal handoff, in which a trained model is frozen and left to execute unattended, leaves changes in prediction quality unchecked. Changes in data (D) can erode the value of an algorithm (A) that the machine (M) continues to execute flawlessly; outcome evidence establishes whether that loss has occurred. Protecting a production pipeline requires engineering safeguards across the entire architecture, from input drift telemetry and validation gates to automated circuit breakers.

2.12 Fallacies and Pitfalls

Most deployment misconceptions share one root. Each treats a physical envelope as a software detail that a later optimization can fix. Data movement, thermal dissipation, and hardware balance can invalidate otherwise reasonable optimization choices, and each fallacy and pitfall below records a system mismatched to its operating envelope.

Fallacy: *One deployment paradigm solves all ML problems.*

Physical constraints create hard boundaries that no single paradigm can span. The memory-wall discussion in section 2.2 demonstrates that memory bandwidth, memory capacity, and access latency scale far more slowly than raw arithmetic compute, producing distinct physical bottlenecks across deployment tiers. Table 2.1 lists illustrative deployment envelopes rather than physics-only latency bounds. A real-time robotics system requiring sub-10 ms response cannot use a remote cloud path whose end-to-end network round-trip exceeds that budget, while a billion-parameter language model cannot reside within the few hundred kilobytes of SRAM on a microcontroller regardless of compression. Robust architectures frequently combine paradigms, using cloud resources for distributed training and burst capacity while executing localized inference or sensor preprocessing on client devices.

A related misconception assumes that moving computation closer to the user always reduces latency, overlooking the execution delay introduced by thermally constrained or lower-bandwidth edge processors, a trade-off exposed when end-to-end latency is separated from model execution time, as in section 12.8.2.2.

Pitfall: *Relying on model optimization to overcome mobile power and thermal limits.*

Compression techniques cannot overcome fundamental thermodynamic limits. Consider a smartphone equipped with a 15 Wh battery. A light inference workload drawing 1 W runs for $\frac{15\ \text{Wh}}{1\ \text{W}} = 15$ h, but an intensive workload drawing 5 W in this illustrative heavy-load scenario depletes that same battery in $\frac{15\ \text{Wh}}{5\ \text{W}} = 3$ h.

Sustained execution at 5 W also triggers severe thermal throttling, as smartphone chassis budget only approximately 3 W for steady-state passive dissipation. While quantizing FP32 parameters to INT8 yields an illustrative 4× reduction in memory footprint, dynamic switching energy and static leakage do not scale down proportionally. A continuous inference pipeline that exceeds the passive thermal envelope inevitably forces dynamic frequency scaling, requiring engineers to lower duty cycles, restructure model architectures, or migrate compute to dedicated accelerator silicon.

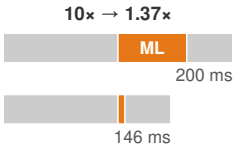
Fallacy: *TinyML represents scaled-down mobile ML.*

The boundary between TinyML and mobile deployment changes the execution environment: much smaller working-memory and energy budgets require different scheduling and runtime choices. As section 2.8.1 establishes, microcontrollers provide 10^5 to 10^6 bytes of on-chip RAM compared to mobile platforms with 8–16 GB of LPDDR DRAM, a 10,000× capacity disparity that dictates

memory residency. Where mobile systems execute models with tens of millions of parameters, many TinyML architectures use tens or hundreds of thousands of parameters, with low-precision arithmetic and layer scheduling designed to minimize peak activation memory. Operating power budgets exhibit an even sharper discontinuity: mobile inference draws 3–5 W, whereas TinyML targets an average power of < 1 mW to operate within energy-harvesting envelopes. Teams that apply mobile optimization techniques directly to TinyML environments discover that post-training quantization from FP32 to INT8 is insufficient on the smallest microcontrollers, where total working memory must fit inside as little as 64 KB of SRAM, forcing complete algorithmic redesign.

Pitfall: *Minimizing computational resources minimizes total cost.*

Optimizing per-inference compute resources in isolation often increases operational expenditure and slows deployment velocity. As the decision framework in section 2.9.2 emphasizes, architectural selection requires evaluating TCO across the entire system lifecycle rather than raw server bills. In this scenario, a cloud inference service costing $2,000/month in compute appears expensive compared to $500/month in amortized edge hardware. However, a distributed edge fleet introduces recurring operational expenses: network engineering ($3,000/month), field hardware maintenance ($500/month), and fleet reliability engineering ($2,000/month). Together with hardware amortization, these bring the listed edge budget to $6,000/month, erasing the apparent hardware-only saving. The cloud figure includes compute alone, so the two totals do not establish a total-cost ranking; cloud operating costs and comparable service capacity remain to be priced. Unlike the single-server comparison in napkin math 2.3, this scenario operates a distributed fleet, where network, maintenance, and reliability labor scale with the number of sites. Furthermore, if a cloud-managed service reaches production in two months while custom edge infrastructure requires six, the four-month delay introduces severe opportunity costs. Sound systems engineering balances compute efficiency against developer velocity, fleet manageability, and operational maintenance.



A faster model stage does not linearly speed up a camera pipeline.

Fallacy: *Model optimization translates linearly to system speedup.*

Amdahl's law[41] bounds end-to-end speedup by the execution time of the stages an optimization leaves untouched, extending the preprocessing limits observed in section 1.11. While pipelining can overlap execution stages to increase steady-state throughput (section 2.3.1), sequential operations on interactive devices must execute in strict serial order. In such serialized execution paths, Amdahl's law dictates that overall system speedup is $\text{Speedup}_{\text{overall}} = \frac{1}{(1-f_{\text{parallel}})+\frac{f_{\text{parallel}}}{G_{\text{accel}}}}$, where f_{parallel} is the fraction of total runtime subject to acceleration and G_{accel} is the speedup factor of that isolated fraction (section D.2.3.1). Consider tapping the shutter on a smartphone camera. The capture pipeline processes raw sensor frames through 100 ms of image signal processing (ISP auto-exposure and white balance), 60 ms of ML scene classification, and 40 ms of tone mapping and HDR merge, totaling 200 ms. Accelerating the neural network stage by 10× (6 ms instead of 60 ms) reduces total latency from 200 ms to 146 ms. Despite a tenfold model speedup, end-to-end latency improves by only 1.37×. Even reducing model latency to zero ($G_{\text{accel}} \to \infty$) yields a maximum speedup of 1.43×, bounded by the remaining 70 percent of unaccelerated pipeline execution. Optimizing an ML system requires profiling the full execution graph; accelerating an isolated stage yields diminishing returns once that stage ceases to be the critical-path bottleneck.

Pitfall: *Assuming more training data always improves deployed model performance.*

Three constraints bound the returns of data scaling. First, algorithmic capacity (A) saturates: when a model's parameter budget is fixed, additional training data cannot overcome representational bottlenecks once the architecture exhausts its expressive capacity. Second, data quality (D) dominates raw volume: label noise, sensor calibration errors, and artifactual correlations contaminate gradient updates, causing scaled-up corrupted corpora to reinforce systematic bias rather than improve generalization. Third, the deployment tier decides which data counts as in-domain. A microcontroller's microphone, a phone camera, and a data-center log stream capture inputs with different noise, resolution, and sampling, so a large corpus gathered under one tier's conditions can match another tier's inputs poorly. High-fidelity data collected under the target tier's own sensing conditions often yields higher serving accuracy than a much larger out-of-domain corpus.

[41] **Amdahl's law**: Formalized by Amdahl (1967) for multiprocessor scaling, this principle applies directly to ML deployment pipelines where the model is only one stage among many. In this camera pipeline example, ML inference is 60 ms of 200 ms; even a 100× model speedup yields at most 1.43× end-to-end improvement here (and about 1.4–2× for pipelines where ML is 30 to 50 percent of runtime) because the rest of the pipeline is unchanged. Teams that benchmark model latency in isolation systematically overestimate deployment gains.

Fallacy: *One model binary can serve every edge hardware target efficiently.*

Teams frequently compile a single model artifact and deploy it uniformly across diverse devices, treating deployment as a packaging step rather than a hardware co-design problem. In practice, hardware-specific compilation and kernel lowering (translating each operation into code tuned for the target's instruction set and memory layout) capture critical efficiencies that generic binaries forfeit. For example, executing quantized INT8 operations on a dedicated neural processing unit (NPU) achieves substantially higher throughput per watt than running the identical model on a general-purpose CPU, because fixed-function INT8 tensor engines avoid instruction-decode overhead and floating-point datapath energy. Furthermore, kernel fusion preserves intermediate activation tensors within on-chip SRAM or register files rather than evicting them to energy-expensive off-chip DRAM. Similarly, tuning memory layouts to align tensor channels with accelerator vector registers eliminates fragmented memory reads that waste bus bandwidth. As the hardware analysis in section 2.4 establishes, hardware architectures impose divergent execution constraints: a binary compiled for a general-purpose mobile CPU fails to use the on-chip SRAM buffers and specialized matrix engines of an embedded NPU. A portable model artifact can run through multiple backends, but portability alone does not establish efficiency. Production deployment requires profiling the target runtime and hardware, then selecting compilation, kernels, and memory layouts that meet its resource and thermal limits.

2.13 Summary

The same model demands different engineering on a phone and in a data center because each deployment target sets different terms for the hardware's half of the silicon contract. The iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$) unifies these physical limits into a single execution model: propagation delay contributes fixed latency (L_{lat}), the memory wall bounds data movement (D_{vol}/BW), and thermal power budgets cap sustained compute. Across cloud, edge, mobile, and TinyML tiers, the categorical power envelope from megawatts to milliwatts spans nine orders of magnitude, while representative systems occupy narrower ranges within it.

By applying the bottleneck principle to workload archetypes, engineers determine whether a model's latency is dominated by compute efficiency, memory bandwidth, or network transit before committing to single-node or hybrid architectures. The quantitative trade-offs summarized in table 2.9 and the tiered screening sequence formalized in figure 2.10 establish the physical baselines that govern all downstream decisions in model compression, data pipeline design, and high-performance serving.

Key Takeaways: Same model, different engineering

- **Physical constraints define feasibility**: Propagation delay sets a lower bound on remote response time (about 36 ms for a cross-country round trip). Power and memory budgets limit what a platform can sustain, so a feasible deployment must meet the task's deadline within the target platform's resource limits.
- **Identify bottlenecks before optimizing**: The same model can be compute bound in batched training yet memory bound at batch-1 inference. The iron law and bottleneck principle pinpoint which constraint dominates; shortening a nonlimiting stage may reduce request latency without increasing steady-state throughput.
- **Match workload demands to deployment budgets**: A compute beast such as large-scale image-model training often needs cloud-scale resources; a tiny constraint workload such as keyword spotting benefits from microcontroller efficiency. Engineers must identify the binding constraint under the intended execution conditions rather than choose a deployment from the model name alone.
- **Deployment choice includes lifecycle cost**: Cloud data-center infrastructure operates at megawatt scale, while TinyML systems target milliwatts, a nine-order span. Resource savings alone do not establish the cheaper architecture; utilization, provisioning, and fleet maintenance determine total cost.

- **Hybrid architectures divide work by constraint**: The voice-assistant example assigns wake-word detection to TinyML, speech-to-text to mobile, and language understanding to cloud. Train-serve split, hierarchical processing, and progressive deployment provide three patterns for combining tiers; the choice depends on each stage's requirements.
- **System-level speedup obeys Amdahl's law, not model-level gains**: In the worked camera pipeline, a 10× faster model yields only 1.37× system speedup because the other stages account for 70 percent of the pipeline. Profile the full system before optimizing any component.
- **Deployment begins a feedback loop**: Live data can diverge from training conditions while code and weights stay fixed, so monitoring and bounded operational exposure are part of the system architecture.

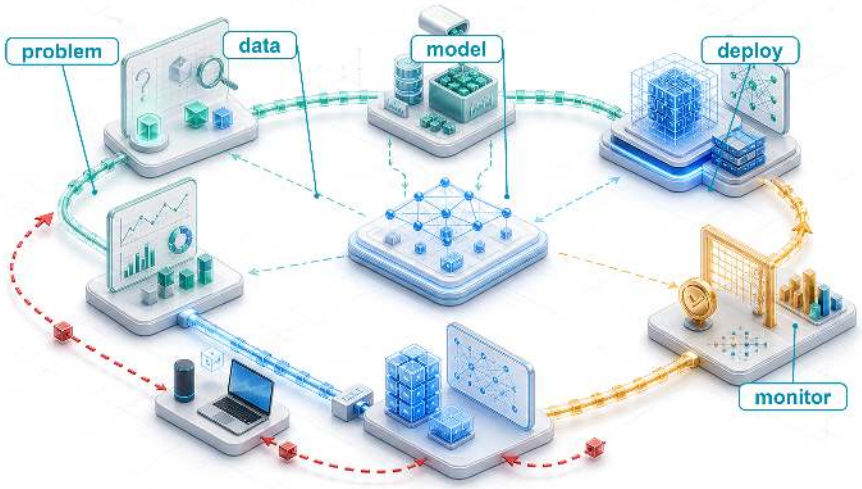
Deployment paradigms are not product categories. They are operating regimes shaped by physical and operational constraints, and selecting one changes which constraint is most likely to bind. Data-center training can push against peak compute (O/R_{peak}). Mobile inference is bounded by energy, with the battery setting the budget and the chassis capping the rate at which it can be spent. Remote real-time inference can be constrained by propagation delay (L_{lat}). Systems engineering begins by measuring that binding constraint, and the measurement is worth most when it comes early. Optimizing model architecture or software runtimes without respecting the terms of the silicon contract produces fragile systems that fail under real-world operational pressure.

What's Next: From theory to process

The decision framework in section 2.9.2 identifies where an ML system can execute, but the team must use those constraints throughout development. Knowing where a model can run helps little if the team learns it too late, because constraints discovered at launch invalidate months of earlier decisions. Chapter 3 opens with exactly that failure, a deployment memory budget discovered only after five months of work, and shows how such constraints must travel backward to the first design meeting. It introduces the ML workflow and the machine learning lifecycle it manages, the iterative engineering process that carries a system from problem definition through deployment, monitoring, and maintenance.

3

ML Workflow



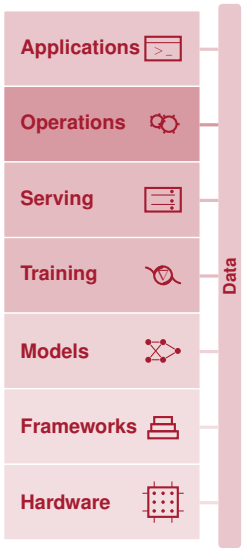
3.1 ML Lifecycle
3.2 Case Study: DR Screening
3.3 Quantifying the ML Lifecycle
3.4 ML vs. Traditional Software
3.5 Constraint Propagation Principle
3.6 Lifecycle Stages
3.7 Problem Definition
3.8 Data Collection and Preparation
3.9 Model Development and Training
3.10 Evaluation and Validation
3.11 Deployment and Integration
3.12 Monitoring and Maintenance
3.13 Systems Thinking
3.14 Fallacies and Pitfalls
3.15 Summary

Purpose

Why is seeing the whole map necessary before walking any single path?

The D·A·M taxonomy identifies the fundamental pillars of every ML system—Data, Algorithm, and Machine—while deployment location establishes the physical constraints each component must satisfy. Engineering organizations frequently treat these three axes as independent specialties: one team curates datasets, another tunes neural architectures, and a third provisions serving clusters. Yet the central principle of systems design is that these components remain inextricably coupled. The volume and quality of collected data bound which algorithms are viable. The computational structure of the chosen algorithm dictates what hardware can execute it efficiently. The physical memory and bandwidth of the target hardware reshape what preprocessing transformations can run at inference. Pull on any single thread and the entire equilibrium shifts.

Optimizing each component in isolation invariably produces accurate models that cannot be deployed and efficient data pipelines that serve the wrong distributions. The engineering cost of an architectural mismatch compounds with how late in the project it surfaces. A physical memory limit discovered during initial design changes a single line in a hardware specification; that same limit discovered on the eve of deployment discards months of expensive training runs, manual annotation budgets, and software integration work. A data engineer who understands how preprocessing transforms constrain downstream inference builds different pipelines than one who views curation as an isolated task. Likewise, a model architect who designs against a concrete on-device memory budget from day one avoids the trap of optimizing benchmark accuracy in an operational vacuum. Before analyzing any individual layer in isolation, engineers must master the full system map: how an ML system is specified, trained, validated, and sustained as an integrated operational loop. The



ML workflow sets that map in motion through continuous co-design across data, algorithm, and machine until their emergent capability meets real-world requirements.

Learning Objectives

- Explain the six ML lifecycle stages as coordinated data-algorithm-machine decisions with feedback
- Compare ML workflows with traditional software using drift, nondeterminism, and operational feedback
- Analyze how problem-definition constraints propagate through data, modeling, validation, and deployment
- Trace late-discovery rework through dependent data, model, validation, and deployment artifacts
- Evaluate accuracy, efficiency, reproducibility, and deployment readiness trade-offs across lifecycle stages
- Design feedback loops that connect monitoring signals to retraining, maintenance, and revised requirements

3.1 ML Lifecycle

A typical deployment failure illustrates the danger of uncoordinated engineering. Day one: "Build a diabetic retinopathy screening model for rural clinics." Day 90: 95 percent accuracy on the validation set used to guide development. Day 120: 96 percent accuracy after a month of architecture tuning. Day 150: the model is handed to deployment engineers. Day 151: deployment engineers report that the model needs 4 GB of memory to run. Day 152: someone checks the deployment target, a low-power edge device in each clinic whose 4–8 GB of memory is shared with the operating system, the camera pipeline, and the clinic software; the clinic's IT plan reserves about 500 MB of it for the model. Day 153: five months of work is discarded.

The model's accuracy was excellent. The team's machine learning skills were excellent. *The failure was a workflow failure.* A deployment constraint that should have shaped every decision from day one was discovered only after the work was done. That 500 MB budget should have propagated backward to the first architecture meeting, constraining which models were even worth considering. Instead, the team optimized each component in isolation (data collection, architecture selection, training), and the integration failure appeared only when the pieces were assembled. A documented diabetic retinopathy (DR) deployment exposed analogous workflow and infrastructure gaps.[1]

The **ML workflow** is an engineering framework designed to prevent integration failures by making constraints explicit at each development stage and tracing how deployment constraints propagate across Data, Algorithm, and Machine. It marks the shift from model researcher to systems engineer. A researcher optimizes individual elements: a better architecture, a cleaner dataset, a faster accelerator. A systems engineer orchestrates those elements into production systems that reliably deliver value. The day-153 failure did not stem from an isolated modeling, data, or hardware defect; it arose from the unmanaged interaction among all three. The workflow supplies the mental map that keeps technical decisions attached to the larger system.

The **machine learning lifecycle** is the sequence of stages this work moves through, a structured, iterative process[2] that guides the development, evaluation, and improvement of ML systems (Amershi et al. 2019). The two terms divide the job. The lifecycle is *what gets traversed*, the stages themselves; the workflow is *how the traversal is managed*, the engineering discipline of orchestrating them. The formal definition of the lifecycle emphasizes continuous management rather than a one-time release.

Definition 3.1: Machine learning lifecycle

Machine learning lifecycle is the iterative engineering process of building, deploying, monitoring, and updating ML systems, where evidence from later stages can feed back to earlier stages because model performance may change after deployment.

1. **Significance**: The lifecycle is a closed loop, not a linear pipeline. Statistical drift between live production traffic and the original training data serves as an early alert signal that

1 | **Lab-to-deployment gap**: Beede et al. (2020) documented this empirically for a deep-learning diabetic retinopathy screening system deployed in Thai clinics. Although the system had specialist-level accuracy in earlier validation, 393 of 1,838 submitted images (21 percent) failed its image-quality requirements, often because clinic lighting, camera maintenance, or dilation practices did not match the system's assumptions. Those rejections added work for nurses, forced retakes or referrals, and exposed workflow and infrastructure barriers rather than simply a model-accuracy problem.

↩ The deployment paradigms whose memory, latency, and power constraints the workflow propagates are established in section 2.1.

2 | **CRISP-DM (cross-industry standard process for data mining)**: CRISP-DM codified data-intensive system development as six interconnected, iterative phases rather than a linear waterfall (Chapman et al. 2000); its core design principle (feedback loops between all phases) directly informs the modern ML lifecycle's structure.

raises the probability of accuracy loss; the exact relationship depends on the model, labels, loss, and deployment distribution. A full retraining run consumes substantial accelerator compute time (scaling with total model operations and bounded by effective hardware throughput), while acquiring fresh verified labels incurs latency and financial expense. How quickly the data drifts and how long verified labels take to arrive therefore set how often the loop must turn, which makes lifecycle maintenance a recurring operating budget as well as an engineering process.

2. **Distinction**: Unlike traditional software, whose behavior changes when code, configuration, dependencies, or environment changes, ML behavior can also change as the world changes. A deployed model's accuracy may change through distribution shift even when the code, infrastructure, and configuration remain untouched.
3. **Common pitfall**: A frequent misconception is that the lifecycle ends at deployment. In reality, deployment is the beginning of the feedback loop: production monitoring surfaces drift, drift triggers investigation, and outcome evidence determines whether a retrained model should re-enter the deployment stage.

Managing that traversal requires **systems thinking**, which analyzes how a system's parts interrelate rather than treating them in isolation. The running case study introduced below supplies a concrete system to analyze, and section 3.13 names the structural patterns that make its stages behave as one coupled system rather than a sequence of independent optimizations.

Understanding how these stages interconnect is essential before analyzing individual layers, because storage pipelines, model training, execution frameworks, and serving infrastructure each depend on decisions made in the others. Figure 3.1 structures ML engineering as two decoupled pipelines operating on distinct timescales. The upper data pipeline continuously transforms raw operational streams through ingestion, curation, annotation, and validation into versioned, ML-ready datasets. The lower model pipeline executes the inner development loop: training architectures, computing evaluation metrics, validating system constraints, and packaging approved artifacts for deployment. Outer-loop feedback arrows carry production telemetry and validation discoveries back to upstream data stages.

While these ten operational blocks represent the fine-grained execution tasks within an automated platform, project governance organizes around six coarser lifecycle stages (section 3.6), sequenced in the order an initiative first encounters them. First, problem definition establishes statistical objectives and physical boundaries before any pipeline executes. Second, data collection and preparation spans the entire upper pipeline to produce validated training splits. Third, model development and training executes the inner optimization loop. Fourth, evaluation and validation audits candidate checkpoints to issue a release manifest (labeled an ML certificate in figure 3.1). Fifth, deployment and integration installs the validated system into the serving environment. Finally, monitoring and maintenance operates the return path, routing production telemetry and observed distribution shifts back to upstream collection.

The conceptual stages of the ML lifecycle establish the architectural map of ML engineering, but abstract pipeline diagrams cannot reveal where operational friction actually binds. Translating these stages into tangible hardware, bandwidth, and clinical trade-offs requires grounding them in a concrete production system. A real-world medical diagnostic screening deployment provides that testbed.

↪ The operational tooling and infrastructure that automate these lifecycle stages at scale are developed in chapter 14.

3.2 Case Study: DR Screening

Diabetic retinopathy (DR) is a leading cause of preventable blindness worldwide.[3] While early detection prevents severe vision loss, limited access to specialist ophthalmologists in rural regions motivates automated screening programs. Detecting the pathology, however, requires identifying subtle microvascular features under variable capture conditions. Figure 3.2 contrasts a normal retina (Panel A) against one exhibiting diabetic retinopathy (Panel B), where detecting localized lesions (such as the indicated hemorrhage) requires preserving fine spatial resolution throughout the vision pipeline and enforcing strict capture-quality validation.

[3] **Diabetic retinopathy (DR)**: DR is a common complication of diabetes; early detection can prevent vision loss, but specialist access varies substantially across countries. This gap motivates scalable screening programs. In clinics with limited hardware or unreliable connectivity, model size, offline capability, latency, and workflow integration can become binding deployment constraints.

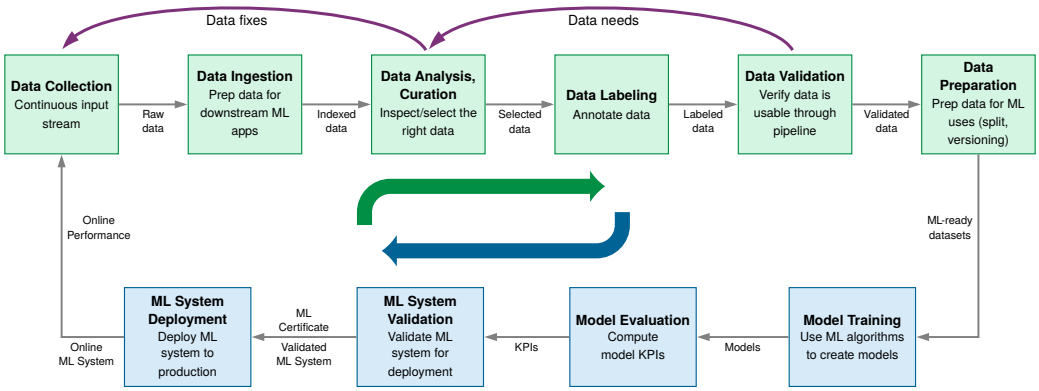


Figure 3.1: Dual-Pipeline ML Development: Machine learning engineering decouples into parallel data and model pipelines operating on different timescales. While the model pipeline executes rapid inner-loop training and evaluation, outer-loop feedback from production deployment and data validation continuously reshapes upstream collection, labeling, and curation.

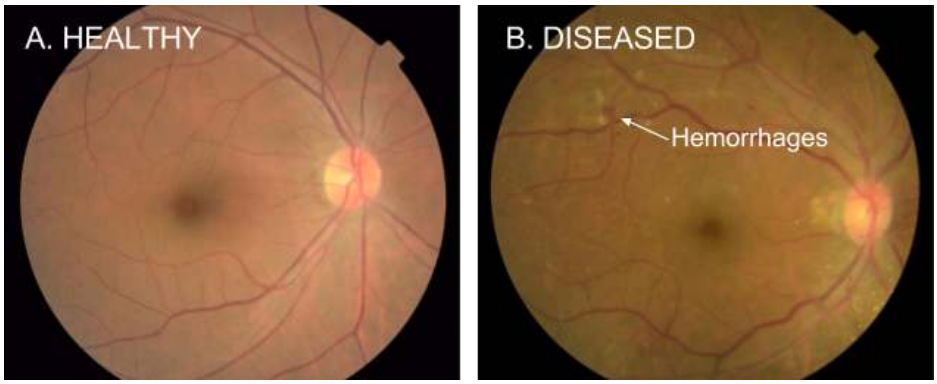


Figure 3.2: Retinal Hemorrhages in DR Screening: Side-by-side fundus images illustrate the subtle visual features an ML screening system must detect. Identifying localized microvascular lesions (such as the indicated hemorrhage) requires preserving fine spatial resolution through the vision pipeline and establishing strict input-quality validation to handle lighting and camera variations in clinical field deployments. Source: Google.

Diabetic retinopathy screening serves as the running case study throughout this chapter because it satisfies three essential criteria: it appears straightforward on the surface but reveals deep operational friction in practice, it exercises the full deployment spectrum from cloud training to edge inference, and its systems challenges are documented across real-world deployments. The literature spans both sides of the lab-to-field divide: Gulshan et al. (2016) documented expert-level benchmark validation in controlled settings, while Beede et al. (2020) documented the severe workflow, image-quality, and infrastructure failures that emerged when deploying a deep-learning screening system in Thai clinics. Translating laboratory models into clinical deployment exposes severe operational friction: sensor lighting variations, intermittent network connectivity in rural health posts, regulatory compliance, and clinic workflow integration.[4]

To ground each lifecycle stage in physical realities, this chapter defines a concrete composite scenario: a network of rural clinics, each executing local inference on an Orin Nano-class edge device with 4 to 8 GB of shared memory and an intermittent wide-area uplink of roughly 2 Mbps. The clinical mandate requires more than 90 percent sensitivity, with an inference latency budget under 50 ms within a 100 ms end-to-end examination window. Sensitivity measures the fraction of true disease cases correctly flagged; its denominator includes only patients with the condition, meaning high aggregate accuracy on an imbalanced, mostly healthy population provides no assurance that sick patients will be detected. Specificity measures the fraction of healthy patients correctly left unflagged, bounding unnecessary referrals that would overwhelm regional specialists. Hardware

[4] **Healthcare AI deployment gap**: In the Thai DR deployment studied by Beede et al. (2020), real clinics exposed workflow, image-quality, infrastructure, and human-factors constraints that were absent from controlled evaluation. The case shows why laboratory metrics alone do not establish deployment readiness.

limits, operator variance, and data distributions govern real-world viability far more than benchmark scores do.

3.3 Quantifying the ML Lifecycle

Practitioner time allocation exposes where the lifecycle actually bottlenecks, and engineering effort rarely concentrates where research literature spends its pages. Conceptual models of the ML lifecycle describe which stages exist, but systems decisions turn on quantitative reality: where engineering hours are spent, how much compute is consumed, and which pipeline stalls delay deployment.

Practitioner surveys reveal a stark asymmetry between where research attention centers and where engineering hours are spent, with about 79 percent of data scientists in CrowdFlower's 2016 survey naming data collection or cleaning as the activity that consumed most of their time. Figure 3.3 charts the reported distribution of primary engineering time sinks, contrasting the dominant data cleaning (60 percent) and collection (19 percent) slices against the narrow share occupied by algorithm refinement (4 percent) and training set construction (3 percent).

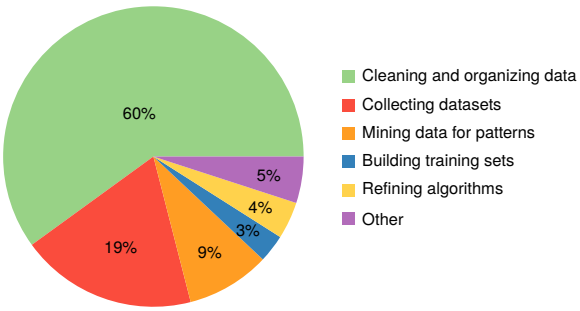
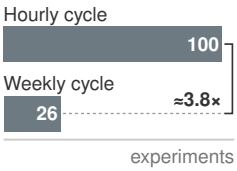


Figure 3.3: Data Scientist Most-Time Responses: In CrowdFlower's 2016 survey, 60 percent of respondents selected cleaning and organizing data as the activity where they spent the most time, with 19 percent selecting data collection. Model-focused responses such as pattern mining, training set construction, and algorithm refinement together represent roughly 16 percent of responses. Source: CrowdFlower, *Data Science Report 2016*.

The survey responses identify data work as a major time sink for these practitioners, without measuring how all engineering hours were divided. In a project whose collection or preparation pipeline delays useful experiments, improving that infrastructure can reduce iteration time before model or training optimizations become the binding concern. The workflow must identify its actual bottleneck rather than assign investment priorities from response shares alone.

↪ The data infrastructure that resolves these pipeline bottlenecks is developed in chapter 4.

Beyond time allocation, iteration velocity governs project progress. The feedback arrows in figure 3.1 (the *Data fixes* loop from curation back to collection, the *Data needs* loop from validation back to curation, and the *Online Performance* path from production deployment back to collection) are not rare exceptions; they are paths teams traverse repeatedly. When data quality degrades through distribution shift or incorrect labels, rework sends teams back to collection. When an architecture exceeds accelerator memory or suffers numerical instability, engineers must revisit model design. When the serving layer exceeds latency budgets, the deployment pipeline stalls. Each loop consumes calendar time and compute budget, increasing development cost whenever downstream stages fail to catch violations early.



Over the same six months, the hourly cycle tests about four times as many ideas.

Napkin Math 3.1: The iteration tax

Problem: Compare what six months of development buys for a rural diabetic retinopathy screening system: a large ensemble developed on cloud hardware as a laboratory baseline, which would itself need compression before it could fit the clinic's memory budget (training time: 1 week, initial accuracy: 95 percent), or a lightweight edge model (training time: 1 hour, initial accuracy: 90 percent). How do the two compare in final accuracy and in the number of hypotheses the team can test?

Math: In six months (~26 weeks), the possible experiment count is:

1. Large model: 26 weeks of calendar time at 1 week per experiment. Each experiment improves accuracy by an assumed constant ~0.15 percentage points.
2. Small model: 26 weeks of calendar time at 168 h/week gives 4,368 possible experiments at 1 hour each. Machine time is not the binding constraint at that cycle length, so this scenario counts only 100 of them as effective experiments, a few per working week, which is the pace at which a team can design, run, and learn from experiments. The remaining capacity sits idle for want of hypotheses. Because the slow-cycle team runs one experiment per week, this cap, not the hardware, sets the breadth ratio below; a team that could formulate useful hypotheses faster would raise the ratio in proportion. Even with a smaller assumed gain per iteration, more useful iterations can produce a larger cumulative gain.

Result: If each iteration improves accuracy by 0.1 percentage points on average, the small model starts at 90 percent and reaches 100 percent after 100 effective iterations, before applying the 99 percent ceiling. With the 99 percent ceiling applied, the small model achieves 99 percent. The large model starts at 95 percent and reaches 98.9 percent after 26 slower iterations. Both approaches end near the ceiling, and the accuracy gap between them is an artifact of the assumed gains, not a finding. The difference that survives is breadth. Under the stated cap, the fast-cycle team tested 3.8× as many ideas, so it also learned which augmentations and preprocessing choices matter, knowledge the slow-cycle team never acquired.

Systems insight: *Iteration velocity is a feature.* Its value lies in search breadth rather than in a guaranteed accuracy lead. When candidate systems have comparable attainable quality, shorter cycles expand how much of the design space a team can test. Speed alone cannot guarantee that a smaller model will outperform a larger one. In the DR screening scenario, the lightweight model's rapid iteration cycle enables the team to experiment with label-preserving input transformations, preprocessing pipelines, and architecture variations far more quickly.

This cumulative delay defines the **iteration tax**: time spent waiting on slow training runs or cumbersome validation pipelines reduces how many candidate models a team can test. Late discoveries amplify this penalty,[5] as formalized by the constraint propagation principle (section 3.5). Violations caught only after deployment invalidate artifacts produced by earlier stages, forcing expensive retraining, pipeline adjustments, and renewed validation. Intercepting defects before they cascade into systemic rework requires explicit **stage interface contracts** that validate outputs at every stage boundary (section 3.6.1).

The operational friction of slow iteration and cascading rework exposes a broader structural reality: *ML workflows are not slow versions of traditional software lifecycles.* They differ fundamentally in where engineering effort concentrates, how feedback loops reshape artifacts over time, and how silent data shifts alter system behavior.

[5] **Late correction costs**: Boehm's *Software Engineering Economics* (Boehm 1981) discussed how defects found late can cost more to fix than those caught during requirements. In ML systems, late-discovered constraints may require retraining, data-pipeline changes, and renewed validation.

3.4 ML vs. Traditional Software

Traditional and ML systems both iterate.[6] The difference that forces new workflow controls is narrower than iteration and sharper. Rule-based software encodes explicit logic, so its behavior changes when someone changes the code, configuration, or dependencies. An ML system learns a statistical mapping from data, so its behavior can also change when the data changes while the code stays fixed. Every workflow addition this chapter introduces (versioned datasets, statistical evaluation gates, drift monitoring) exists to manage that second path.

Fraud detection shows the difference in practice. A rule such as "decline if the amount exceeds $5,000 abroad" changes only when someone edits it, and a test against its specification shows what it will do. A learned fraud score changes whenever it is retrained on new transactions, and the labels that grade it (confirmed fraud cases) may arrive weeks late. Both approaches need revalidation as fraud patterns shift, and both trade lost revenue against fraud losses when setting decision thresholds. Only the learned score, however, can change its decision logic without a code change, and only fresh labeled data can show whether today's model still catches today's fraud. That is why its workflow must version data, gate releases on statistical evidence, and monitor for drift.

[6] **Waterfall model**: A plan-driven lifecycle is often depicted as a sequence of requirements, implementation, and testing, but Royce (1970) warned that a purely sequential implementation was risky and described iteration between successive phases. ML systems add data and model feedback that must be managed explicitly.

↩ Data-defined behavior, and the silent degradation it makes possible, are established in section 1.6.

These differences alter how lifecycle stages interact. While conventional software engineering relies on production feedback to refine code and requirements, ML systems introduce feedback loops where operational data reshapes training distributions, detected drift prompts investigation, and runtime errors expose dataset blind spots. Table 3.1 contrasts traditional software and ML engineering across six core lifecycle dimensions, highlighting how data evolution redefines testing, deployment, and ongoing maintenance.[7]

[7] **Data versioning**: Unlike code, which changes through discrete, auditable commits, data can drift gradually (distribution shift), suddenly (schema migration), or subtly (label quality degradation). Plain Git is impractical for many multi-terabyte datasets, so teams commonly version external-data manifests or pointers using tools such as DVC and Git LFS. Without data versioning, teams cannot reliably reproduce a prior training run or determine whether an accuracy regression stems from a code change or a data change.

Table 3.1: Traditional Software vs. ML Development Lifecycles: Six dimensions where ML development diverges from traditional software engineering. Both use production feedback, while ML systems add loops in which deployment insights reshape data collection, monitoring drives model updates, and production experience informs model design.

Aspect	Traditional Software Lifecycles	Machine Learning Lifecycles
Problem Definition	Functional requirements specify behavior; tests check the code against them.	Objectives are statistical targets whose feasibility is known only after experiments on data.
Development Process	Engineers write and revise the logic directly in code and configuration.	Engineers shape behavior indirectly through data, features, architecture, and training runs.
Testing and Validation	Many functional tests have exact expected results; performance and reliability remain quantitative.	Metrics over held-out data, reported with uncertainty and across subgroups.
Deployment	Logic stays fixed until code, configuration, or dependencies change.	Accuracy can degrade when input distributions shift, with code unchanged.
Maintenance	Maintenance involves modifying code to address bugs or add features.	Continuous monitoring, updating data pipelines, retraining models, and adapting to new data distributions.
Feedback Loops	Production feedback changes code, configuration, and requirements.	Production data can become future training data, so feedback also reshapes data collection and the model.

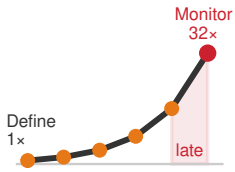
Checkpoint 3.1: ML vs. traditional software

ML systems are not traditional software with a model attached. Check the differences that force a separate workflow:

- ☐ Failure modes: Can you explain how a shift in input data can degrade an ML system silently, with no code change and no error raised?
- ☐ Logic source: Can you explain why changing training data can change model behavior even when the code remains fixed?
- ☐ Iteration: Can you explain why ML requires monitoring and, when evidence warrants it, model or data updates?

3.5 Constraint Propagation Principle

In the DR scenario, the intermittent 2 Mbps clinic uplink rules out cloud inference and requires edge deployment. The edge target's memory and power budgets then constrain model architecture, favoring compact MobileNet-class models over large ensembles and requiring automated image-quality checks at capture time. Each downstream constraint narrows the design space for upstream stages, coupling hardware limits directly to data collection and model design. Stating that coupling precisely uses the stage numbers given alongside figure 3.1, from (1) problem definition to (6) monitoring and maintenance.



If each later stage doubles the fix, a constraint found at monitoring costs 32× one caught at definition.

Definition 3.2: The constraint propagation principle

The constraint propagation principle states that a constraint discovered late in the ML lifecycle can force rework across affected upstream stages. Datasets, trained weights, validation evidence, and deployment packages record decisions that later work depends on. Correcting a missing constraint requires tracing those dependencies to determine which artifacts must change and

which remain valid. Estimating actual rework requires the effort to revise each affected artifact, repeat its dependent work, and validate the resulting system.

1. **Significance**: A 100 ms latency target discovered at deployment (stage 5) may propagate backward to constrain model size (stage 3: operation count O), dataset requirements (stage 2: dataset size D, the number of training examples), and problem definition (stage 1: what accuracy is achievable). Rework depends on which earlier decisions relied on the missing constraint. Within the iron law, a latency budget bounds total time T, and therefore every term that sums to it, so it redefines the feasible region for O, for the bytes moved D_{vol} (distinct from the example count D), and for the hardware efficiency η_{hw} the model must achieve.
2. **Distinction**: Unlike modular decomposition (which encourages independent optimization of each component), this principle mandates end-to-end reasoning: optimizing accuracy in isolation may produce a model that is infeasible to deploy, so an accuracy optimum reached in isolation can be an infeasible system.
3. **Common pitfall**: A frequent misconception is that deployment is "the last step." The deployment environment is the day-one constraint: its latency budget, memory capacity, and power envelope define the boundaries of every upstream decision.

Propagation runs in both directions, so the stages form a network of constraints rather than a chain. Backward propagation carries a deployment limit upstream. When rural clinic deployment reveals tight bandwidth, the team can compress images before sending them over the uplink, but lossy compression degrades input quality, so the training dataset must then include matching compression artifacts or the model will see different inputs in service than in training (training-serving skew). Shifting inference entirely to the edge and transmitting only the diagnostic result relieves the uplink instead, but tightens the edge device's memory capacity and power budget. Forward propagation carries an early data decision downstream. Capturing fundus images at the resolution needed to see small hemorrhages fixes the model's input size, and with it the operation count every inference must pay on the clinic device. Upstream data choices and downstream machine boundaries remain physically coupled.

Decisions made without accounting for downstream execution boundaries accumulate technical debt.[8] The stage interface specification (table 3.3) operationalizes this principle by making constraints explicit at each stage boundary. Those contracts enable earlier detection of constraints. When propagation occurs specifically through data quality failures, the resulting pattern is known as a **data cascade**: a chain of downstream failures triggered by bad data (Sambasivan et al. 2021).

[8] **ML technical debt**: Sculley et al. (2015) identify ML-specific debt mechanisms such as *entanglement* (changing one feature affects all others because the model learned joint distributions), *hidden feedback loops* (predictions influence future training data), and *undeclared consumers* (downstream systems depending on outputs without contracts). Since ML code is often only a small part of a production system, the surrounding configuration, pipelines, and infrastructure can allow this debt to accumulate silently.

↪ Shared contracts and deployment paths that reject incompatible changes before they reach production are enforced in section 14.4.

3.6 Lifecycle Stages

The rural-clinic failure illustrates why ML projects require an explicit six-stage framework: deployment constraints surfaced only after data curation, model training, and evaluation decisions had hardened around an unfeasible target. Preventing such misalignments requires establishing clear ownership across the system stack.

These six stages delineate project responsibilities, mapping directly onto the D·A·M axes. Problem definition sets the governing requirements for all three. Data collection and preparation owns the Data axis, model development and training governs the Algorithm axis, and deployment and integration commits both to a physical Machine. Evaluation and monitoring own no single axis of their own; their role is verification, testing whether the three axes function harmoniously prior to release and continuously in production. Figure 3.4 arranges these stages as a closed operational loop: problem definition flows into data preparation, model development, and validation across the top row, while deployment transitions into monitoring and maintenance below, routing telemetry back to upstream collection.

The DR team's choice of model shows the stages acting on one decision. Problem definition fixed the clinic device's 500 MB model budget, so model development starts from architectures designed for budgets of that kind. MobileNetV2, a model built for real-time inference on mobile-class hardware, is a natural candidate backbone, at about 14 MB of FP32 weights and about 600 MFLOP

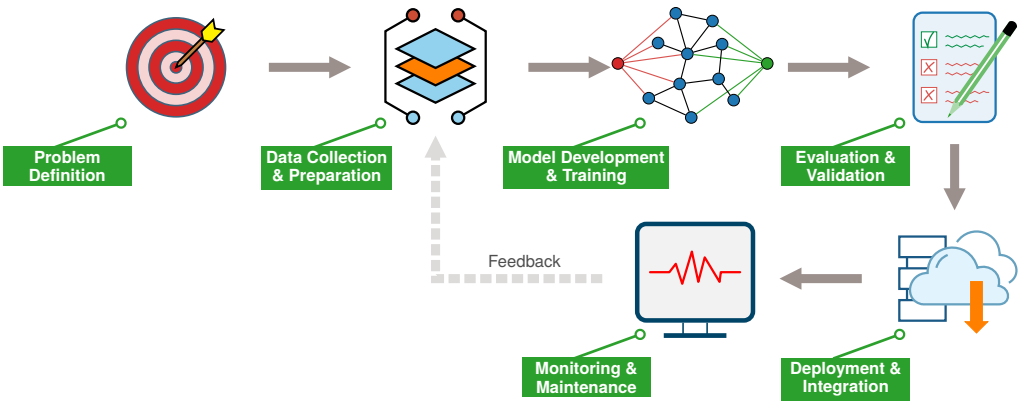


Figure 3.4: Simplified Lifecycle with Feedback: The six core stages of ML engineering form a closed loop rather than a linear handoff. A primary feedback path from monitoring back to data collection supports investigation and adaptation as production distributions shift, model accuracy degrades, and operational constraints evolve.

per inference. It reaches that operation count by replacing standard image-processing layers with a cheaper variant[9] rather than by pruning a larger model. Choosing it immediately constrains the other stages. Data preparation must produce images at the input resolution the backbone expects, evaluation must measure latency on the clinic device as well as sensitivity, and deployment must confirm that weights, activations, and runtime together fit the budget. If on-device evaluation then misses the latency target, the fix loops back to architecture or input resolution, so the stage sequence in the diagram is traversed as a cycle rather than once.

[9] **Depthwise separable convolutions**: The layer type MobileNetV2 uses in place of standard convolutions. It splits one expensive operation into two cheaper ones and cuts computation by roughly 8–9× for typical kernel sizes (Sandler et al. 2018), which is what makes the roughly 600 MFLOP inference budget plausible on mobile-class hardware.

Checkpoint 3.2: The workflow cycle

Verify that each stage enforces contracts that bind upstream choices to physical deployment constraints.

- ☐ **Problem definition**: Can you identify which stage records latency, memory, and accuracy targets, and explain why later stages need those targets?
- ☐ **Data preparation**: Can you identify which downstream artifacts must use the same image-resolution and preprocessing choices as the training data?
- ☐ **Model development**: Can you explain why model development needs a deployment memory budget before committing to a candidate?
- ☐ **Feedback**: Can you explain why a failed on-device latency test sends work back to architecture or input resolution rather than ending at deployment?

Beyond procedural orchestration, each lifecycle stage acts on terms of the iron law of ML systems, and because the binding constraint differs across workload archetypes, each stage optimizes different terms for different workloads. ResNet-50, DLRM, and keyword spotting (KWS) are useful anchors because each stresses a different part of the system: dense vision training tries to keep accelerators busy, sparse recommendation spends much of its time moving embedding rows, and keyword spotting is constrained first by tiny memory and always-on energy budgets.

↩ The iron law of ML systems and its governing terms are established in section 1.7.

Production systems rarely fall neatly into a single archetype. A medical imaging classifier, for instance, requires sustained computation while training over large image datasets, yet faces strict energy and memory limits when deployed to portable clinic hardware. Understanding how the workflow framework adapts across these trade-offs is essential for sound engineering decisions. Table 3.2 follows three of the six stages (abbreviated in its rows as data collection and preparation, model development and training, and deployment and integration) across these recurring workload archetypes. Its DLRM row uses p99 latency, the time within which 99 percent of requests complete, because users of an interactive service notice its slow tail, not its average.

Table 3.2: Workflow Variations by Lighthouse Model: Three selected lifecycle stages target different iron law terms depending on the workload's binding constraint. ResNet-50 emphasizes useful computation per unit time, DLRM emphasizes data movement and embedding freshness, and KWS emphasizes energy and memory capacity.

Stage	ResNet-50 vision training	DLRM recommendation	KWS TinyML
Data Eng	Keep image batches available fast enough to sustain high GPU utilization.	Keep online lookups of stored user and item features within the serving latency budget; embedding tables dominate storage and freshness.	Curate short audio clips for an SRAM-constrained device.
Training	Coordinate preprocessing, batching, reduced-precision arithmetic, and model execution to reduce accelerator idle time.	Optimize sparse embedding lookups because memory bandwidth limits throughput.	Search for the smallest model family that still recognizes the trigger phrase reliably.
Deploy	Use large batches when throughput and cost matter more than single-request latency.	Meet strict interactive p99 latency targets while keeping features fresh.	Stay within a strict always-on energy budget.

Each stage of this workflow presents distinct systems challenges, from curating multi-terabyte datasets to sustaining tail latency in production. Grounding these challenges requires a quantitative lens on how stages govern execution.

Systems Perspective 3.1: The iron law of workflow

The six lifecycle stages alter distinct terms in the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). Rather than rigid one-to-one assignments, the mappings serve as diagnostic starting points:

- Problem definition: Sets target constraints for accuracy, latency, cost, privacy, and deployment paradigm (screened across privacy, latency, compute, and cost gates using the decision framework in figure 2.10). These targets determine which terms of the equation are permitted to grow and which must be bounded from the outset.
- Data collection and preparation: Shapes dataset composition and the byte volume (D_{vol}) presented to downstream pipelines. Curation can also change the training work required to reach a quality target.
- Model development and training: Dictates the operation count (O), data reuse patterns, parameter movement, and achievable hardware efficiency (η_{hw}).
- Evaluation and validation: Tests whether model quality and end-to-end execution meet deployment requirements on the target system.
- Deployment and integration: Determines the serving path, including runtime data movement, execution efficiency, and fixed overhead (L_{lat}).
- Monitoring and maintenance: Observes distribution drift, latency, throughput, and operational cost after launch, feeding violations back into earlier stages for re-optimization.

Viewing the lifecycle through the iron law grounds latency and cost in hardware physics. Production workflows must simultaneously satisfy accuracy, safety, privacy, and organizational constraints that sit outside the equation.

3.6.1 Stage interface specification

Each lifecycle stage operates as a distinct engineering phase with defined inputs, outputs, and quality invariants. In production, pipeline orchestrators encode these dependencies as directed acyclic graphs (DAGs), where nodes represent discrete execution tasks and directed edges represent validated artifacts. While the overarching ML lifecycle forms a recurring feedback loop over weeks and months, each individual pipeline run executes as a deterministic, acyclic sequence of dependent steps. This structure enables the orchestrator to parallelize independent branches, cache validated

intermediate outputs, and halt execution the moment a contract is violated. Just as microservice architectures rely on interface specifications to enforce API boundaries, ML pipelines require strict schema, distribution, and performance contracts to prevent defective data or degraded weights from corrupting downstream artifacts. Table 3.3 formalizes these contracts, defining the inputs, deliverables, and quality invariants across all six stages.

Table 3.3: Stage Interface Specification: Each lifecycle stage has explicit input requirements, output deliverables, and quality invariants that must hold for the stage to be considered complete. Violations of these contracts create technical debt that compounds through subsequent stages. The deployment paradigm selection in Problem Definition (Cloud, Edge, Mobile, or TinyML) constrains all downstream stages, as a TinyML target imposes different data, model, and monitoring requirements than a Cloud target.

Stage	Input Contract	Output Contract	Quality Invariant
Problem Definition	Business requirements; operational context	Measurable objectives; deployment paradigm selection; resource constraints	All success criteria are quantifiable; target deployment paradigm is explicit
Data Collection & Preparation	Objectives; deployment target; quality requirements	Versioned dataset with schema; preprocessing pipeline; data validation rules	Distribution approximates anticipated production environment; labeling meets accuracy requirements
Model Development & Training	Dataset; accuracy targets; resource constraints	Trained model weights; training configuration; experiment logs	Meets accuracy thresholds within computational budget; architecture compatible with deployment target
Evaluation & Validation	Trained model; held-out test data; evaluation criteria	Performance metrics across subgroups; failure mode analysis; release manifest	No critical subgroup falls below minimum thresholds; predicted confidence matches observed accuracy as closely as the domain requires
Deployment & Integration	Validated model; infrastructure requirements; service-level agreement (SLA) targets	Serving endpoint; monitoring instrumentation; rollback procedures	Latency and throughput meet paradigm requirements; integration tests pass
Monitoring & Maintenance	Live system; performance baselines; alert threshold	Drift detection alerts; update-review criteria; incident reports	Alert coverage and detection windows are defined; detected degradation triggers review

Enforcing explicit contracts transforms the conceptual feedback loop in figure 3.4 into an operational auditing mechanism. When validation or monitoring surfaces a defect, the contract boundary identifies which upstream stage owns the remedy. For example, a mismatch between training features and production inputs may demand targeted data collection, whereas an error in feature transformation points directly to a bug in the serving pipeline. Intercepting these violations at stage boundaries prevents defective assumptions from hardening into late-stage architectural rework, as demonstrated in the transition audit below.

Example 3.1: Auditing stage transitions

Scenario: An engineering team claims to have completed Problem Definition for a medical imaging classifier, attempting to transition to Data Collection without establishing deployment target constraints.

Diagnosis: The output contract lacks deployment paradigm selection and resource constraints (latency and memory targets). If the violation surfaces only at deployment as a trained model larger than the edge device's 500 MB model budget, the audit must trace what relied on the missing limit. A smaller candidate may require revised input-resolution or preprocessing choices, new weights, repeated sensitivity and on-device memory tests, and a replacement deployment package. Existing raw images may remain usable; changing the candidate does not imply recollecting every image.

Repair schedule: Suppose the repair plan preserves the raw images and labels but changes input preparation and the candidate architecture. Assume 1 day to revise preparation, 2 days to train new weights, 2 days to repeat validation, and 1 day to package the approved result. If each task waits for the preceding artifact and the required staff and compute are available, the

repair takes 6 days. This is a supplied schedule for this dependency chain, not a multiplier determined by the discovery stage. Queueing, failed validation, or parallel independent tests would change the elapsed time.

Systems lesson: Stage transitions act as quality gates in the ML lifecycle. Enforcing strict output contract validation before proceeding downstream prevents expensive upstream rework and ensures data collection aligns with target deployment hardware.

Without explicit contract gates, unstated assumptions propagate downstream. The earliest opportunity to bound them is Problem Definition, which records the operational constraints that subsequent stages must satisfy.

3.7 Problem Definition

Problem definition in ML begins with sentences that look deceptively simple. A product manager writes: "Build a model that detects diabetic retinopathy." That single sentence conceals a dozen engineering decisions: sensitivity thresholds for patient safety, hardware capabilities in rural clinics, latency budgets that keep clinicians engaged, and regulatory frameworks governing approval. Software requirements still govern control flow, interfaces, and data storage. ML adds an empirical obligation: defining *what* the system should do is inseparable from defining *how* it will learn from data and what physical constraints its execution must survive.

The request for a model must first become a requirement for a useful improvement. In the clinic case, the baseline is the existing screening and referral workflow, evaluated on the outcomes and resource limits the proposed system must improve. A simpler rule or an existing model can provide an additional comparison where appropriate. If available data cannot support the required evaluation, or the expected benefit does not justify the added errors, cost, and maintenance, the team should revise the problem or retain the baseline rather than proceed automatically to training.

Balancing these demands requires reasoning across the full system stack. An engineering team cannot optimize accuracy in isolation: pursuing marginal sensitivity gains by scaling model depth or adopting a larger model family inflates parameter count and computational demands (FLOPs per inference), directly challenging the memory capacity and thermal limits of low-cost clinic hardware. Conversely, pruning a model to fit an embedded accelerator's DRAM budget risks dropping sensitivity below clinical safety thresholds. Every technical decision operates within a multi-constraint optimization problem where statistical goals, physical execution budgets, and operational mandates continually bound each other's feasible design space.

3.7.1 Constraint layers

Every ML problem definition decomposes into three interacting **constraint layers**: statistical, physical, and operational. Rather than isolated requirements, these layers form a coupled stack where constraints in one tier immediately restrict the degrees of freedom in the others. The statistical layer specifies target behavior over data distributions, such as achieving > 90 percent sensitivity and > 80 percent specificity across diverse patient demographics and camera models, rather than an unweighted aggregate accuracy that conceals failure modes on minority subpopulations. The physical layer bounds execution within hardware limits: for rural clinics, inferencing on local edge accelerators under tight thermal budgets, bounded memory capacity (such as 4 to 8 GB of shared DRAM), and a 100 ms end-to-end latency budget to keep clinical workflows interactive without depending on intermittent network uplinks. The operational layer encompasses clinical auditability, regulatory clearance, and governance frameworks that dictate long-term system maintenance.

The operational layer carries physical consequences that traditional software engineering rarely encounters. In a traditional database, deleting a patient's record is a `DELETE` statement. In an ML system, model weights may encode statistical patterns learned from that record: where law or a regulator requires removing that influence, compliance may require machine unlearning (an active research area with incomplete guarantees) or retraining on the remaining data. The obligation and remedy depend on the governing rule and system; at DR-system scale, either can make privacy compliance a recurring compute-budget item and training provenance an architectural constraint.

The constraint propagation principle (section 3.5) explains why all three layers must be bound before development begins: a constraint that exists in the physical or operational environment but remains unstated in the problem definition does not vanish. It re-emerges later as an architectural violation after dependent decisions have hardened. For example, if an engineering team defines the objective solely as aggregate classification accuracy on retinal images, standard optimization will minimize the average training loss, which the majority class dominates. In an imbalanced screening population where 90 percent of patients are healthy, a model predicting "no disease" achieves 90 percent aggregate accuracy while failing every patient who needs urgent care. Preventing this requires translating a domain invariant (a missed diagnosis costs far more than a false referral) into an explicit statistical contract: class-weighted loss, a sensitivity floor, and subgroup targets.

War Story 3.1: When the label was the bias (2018)

Context: In 2014, Amazon assembled an engineering team at its Edinburgh hub to build an internal machine-learning system for ranking technical job applicants, training it on ten years of resumes submitted to Amazon (Dastin 2018).

Mechanism: Because prior hiring was male-dominated, the model learned that male-coded language correlated with hiring success, systematically penalizing terms like "women's" and downgrading graduates of all-women's colleges.

Impact: The model reproduced historical hiring bias, rendering automated resume screening unsuitable for production recruitment.

Fix: Amazon abandoned the experimental tool after attempts to remove explicit gender signals could not guarantee that it would not learn other discriminatory proxies.

Systems lesson: Problem definition must specify fairness, auditability, and rejection criteria before data collection and training. A workflow trained on biased labels does not learn the operational goal; it learns to reproduce the labels.

3.7.2 Problem definitions evolve

A problem definition is not a static specification frozen at project inception; it evolves as deployment expands. When a DR screening program scales from a pilot clinic with a calibrated fundus camera to regional health centers with heterogeneous imaging hardware and differing patient demographics,[10] the initial assumptions break. Sensor noise profiles diverge, optical resolutions shift, and demographic variations alter subgroup error rates. Adapting to this operational reality forces the problem definition to incorporate stratified accuracy targets and broader hardware compatibility matrices.

This dynamic evolution stems directly from the statistical nature of ML. Scaling exposes long-tail inputs absent from pilot evaluations, while production environments shift the input distribution in ways no static offline dataset fully captures. A robust problem definition anticipates this drift by establishing governance protocols upfront: defining which monitoring signals trigger formal re-evaluation and bounding how threshold updates propagate through subsequent pipeline stages. Once these operational constraints and performance boundaries are defined on paper, the workflow must ground them in physical artifacts, beginning with the data collection pipelines that gather the training distribution.

[10] **Demographic drift**: Population changes can expose performance gaps that aggregate evaluation obscures. Facial-analysis benchmarks found large demographic error-rate disparities (Buolamwini and Gebru 2018), while a healthcare risk-scoring study found racially biased allocation from a proxy label despite similar need (Obermeyer et al. 2019). These examples illustrate why a growing DR program should evaluate subgroup performance rather than claiming that such a rollout occurred.

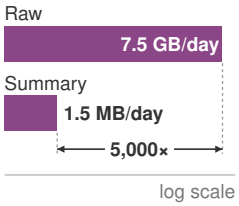
3.8 Data Collection and Preparation

Constraints defined on paper only take effect when grounded in actual data. Moving from problem definition to data collection connects abstract accuracy targets with operational reality: sensor noise, missing records, and network limits. This stage determines dataset size (D), the number of examples, and through it the iron law's byte volume (D_{vol}), the bytes that storage systems, networks, and memory buses must move; the two grow together but are not the same quantity, since image resolution and encoding set how many bytes each example costs. The deployment constraints established during problem definition become technical data requirements: if the model must run on edge devices, the collection pipeline must produce inputs compatible with edge preprocessing;

if it must achieve 90 percent sensitivity across diverse populations, the dataset must sample those subpopulations sufficiently to support model training.

Data collection and preparation is an ongoing engineering discipline rather than a one-time task. For diabetic retinopathy screening, these criteria demand a collection strategy far more stringent than standard computer vision benchmarks. The pipeline must assemble roughly 10^5 retinal fundus photographs across diverse patient cohorts and imaging hardware, with each image adjudicated by multiple ophthalmologists to establish consensus ground-truth labels capable of withstanding clinical and regulatory scrutiny. The annotation protocol must capture fine pathological features—including microaneurysms, hemorrhages, and hard exudates—across the entire disease severity continuum. Curation also governs computational efficiency: not all captured data contributes equally to training, and intelligent selection (section 9.7.1) can eliminate redundant samples while preserving the statistical coverage required for generalizable performance.

Each retinal photograph spans several megabytes, and clinical protocols capture multiple views per patient, immediately creating a physical infrastructure bottleneck. A busy rural facility generates gigabytes of raw imaging data each day—a volume that would overwhelm an intermittent 2 Mbps uplink during operating hours. Rather than saturating limited wide-area bandwidth with raw uploads, the system can execute inference locally on the edge accelerator and transmit only a compact edge summary: a structured record containing the diagnostic classification and clinical metadata. As quantified in Napkin math 3.2, local inference reduces uplink traffic from gigabytes to kilobytes per patient.



Edge summaries shrink the clinic's daily uplink about 5,000-fold.

> Napkin Math 3.2: Bandwidth vs. compute
>
> **Problem**: Quantify the uplink window occupancy for raw image uploads from a busy rural clinic capturing retinal images for diabetic retinopathy screening, and evaluate architectural alternatives to relieve link contention.
>
> **Math**:
>
> 1. Daily data: At 150 patients/day, 10 photos/patient, and 5 MB/photo, the clinic produces 7.5 GB/day.
> 2. Upload time: The uplink is 2 Mb/s, or 0.25 MB/s. Dividing 7,500 MB by that rate gives 30,000 s, approximately 8.3 h.
> 3. **Constraint**: If the clinic requires same-shift upload during its 8 h operating window, this transfer alone occupies the link for 104.2 percent of that window and may contend with other traffic.
>
> **Systems insight**: Local inference with uploaded detection summaries (10 KB/patient) reduces bandwidth usage by 5,000×. Overnight batching, compression, scheduling, or a faster link are alternatives not compared by this calculation.

3.8.1 Lab-to-field data gap

Laboratory data and production data inhabit different worlds. This **lab-to-field gap** appears when DR screening deploys to rural clinics across Thailand and India: images arrive from diverse camera equipment operated by staff with varying expertise, often under suboptimal lighting with inconsistent patient positioning. A model trained on high-quality research images from standardized fundus cameras may fail on blurry, poorly lit images from older equipment, not because the algorithm is wrong but because the data distribution has shifted beyond the training envelope.

Because raw image uploads can exceed clinic uplink capacity, edge deployment offers an alternative; this is the data locality invariant (2.3) applied to a clinic uplink. Deploying edge accelerators such as the NVIDIA Jetson family[11] allows clinics to preprocess and run inference locally. This reduces network transmission by orders of magnitude, but introduces a hardware trade-off: use a smaller model that fits a low-power edge budget, or provision more capable on-premise devices at higher cost.

The bandwidth constraint makes infrastructure a data-collection decision rather than a late implementation detail. The scenario architecture combines edge devices for local inference and

11 **NVIDIA Jetson**: NVIDIA's Jetson family spans a wide SKU spectrum, from Jetson Orin Nano (7–15 W) through Jetson Orin NX (10–25 W) to Jetson AGX Orin (15–60 W). This scenario uses an Orin Nano-class device (4–8 GB shared LPDDR5, 7–15 W power envelope) as a concrete edge target. Its memory and power budgets constrain model complexity, while selecting a larger device changes both the feasible model and the per-clinic cost.

preprocessing, clinic aggregation servers for data management and buffering, and cloud training infrastructure for periodic model updates. It targets end-to-end latency under 100 milliseconds, of which model inference may take at most 50 ms, and operation without connectivity-induced delays.

Privacy constraints impose a similar architectural decision. Patient privacy regulations may motivate local retention or distributed training, but they generally specify protections and permitted uses rather than one required training topology. Workflows that keep raw clinic data local while sharing only approved updates or summaries add complexity to both data collection and model training infrastructure.

3.8.2 Distributed data infrastructure

If a deployment grows from a handful of clinics to hundreds, data infrastructure must scale accordingly. Each retinal image travels through multiple stages: clinic cameras capture the image, local systems provide initial storage and processing, quality validation checks ensure usability, secure transmission moves data to central systems, and finally, integration with training datasets completes the pipeline. The infrastructure decisions at each stage are shaped by the deployment constraints established during problem definition.

Storage placement is another data-collection decision that either preserves or erodes downstream iteration velocity. The DR images serve three distinct operational roles. The current training set is read in full every epoch, recent datasets and validation sets are read occasionally, and the archive kept for regulatory audit is almost never read. No single storage medium serves all three economically, so teams implement **tiered storage architectures**,[12] each tier matched to how often and how fast its data must be read.

The training set benefits from hot storage (fast local SSDs) when its repeated reads would otherwise arrive more slowly than the accelerator consumes them. Packing images into large files and fetching them ahead of use can also make an object-storage stream sufficient if its measured delivery rate meets that demand. Recent datasets and validation sets sit on warm object storage, which costs several times less per gigabyte but answers each request far more slowly. The audit archive sits on cold archival storage, the cheapest per gigabyte and the slowest to retrieve. The workflow decision is which tier a training run reads from. A run whose warm-storage requests leave the accelerator waiting loses the iteration velocity the iteration tax showed to be valuable, while keeping the archive on hot storage pays a premium for data no one reads. The boundary between tiers moves over time. A dataset can migrate from warm to hot when a training run needs faster delivery, and from hot to cold when its access rate falls, under automated policies keyed to training schedules and access recency.

While tiered storage optimizes I/O within the centralized training cluster, ingesting data from hundreds of remote clinics introduces a distributed systems challenge. Rural clinic deployments face severe connectivity constraints that dictate transmission strategies. Facilities with reliable broadband can stream diagnostic summaries in near-real-time, whereas clinics with intermittent satellite links require **store-and-forward architectures** that buffer data locally and synchronize opportunistically during connectivity windows. This design choice propagates through the physical stack: store-and-forward clinics require expanded local storage buffers, autonomous edge inference runtimes, and reconciliation protocols to resolve divergences between local preliminary classifications and subsequent centralized audit reviews.

Infrastructure scalability presents a steeper challenge than raw storage capacity. As a deployment expands from a pilot site to hundreds of clinics, data heterogeneity scales alongside byte volume: divergent camera optics, erratic clinic illumination, and variable operator training yield distinct input distributions. The data pipeline must process increasing throughput while maintaining strict provenance metadata—recording camera model, capture settings, and operator ID for every image. This provenance trail proves essential for isolating site-specific accuracy regressions and satisfying regulatory audit requirements. Each clinic operates as a distinct data source.[13]

Coordination infrastructure—including shared artifact registries, versioned data contracts, and automated validation gates—makes site-level variance visible before it corrupts model training. Crucially, this distributed heterogeneity makes point-of-capture validation mandatory: discovering that a clinic's camera has been miscalibrated weeks later during a centralized training run wastes storage, compute, and patient follow-up opportunities.

[12] **Tiered storage**: Places data on different storage media based on access frequency and performance requirements. The storage price gap is roughly 4.3× in this example: Non-Volatile Memory Express (NVMe) SSDs deliver 500,000+ input/output operations per second (IOPS) for random reads at ~$0.10/GB/month, while object storage costs ~$0.023/GB/month but incurs 100–200 ms request latency. For ML training loops requiring sustained sequential streaming at 1–10 GB/s, fetching thousands of individual image files introduces random seeking and protocol overhead that collapses throughput, converting a compute-bound training loop into an I/O-bound stall that starves accelerator execution units.

↪ The cost and throughput of each storage tier, and why ML systems combine them, are measured in section 4.8.2.

[13] **Distributed clinic data**: Training across clinic sites without simply pooling all raw data addresses privacy and governance constraints, but it shifts cost into coordination. Each site may use different cameras, serve different patient populations, and follow different operating routines, so the system must track where data came from and how each site differs. The workflow cost is therefore not just storage capacity; it is the engineering work needed to compare, validate, and update models across sites whose data does not behave identically.

3.8.3 Quality assurance and validation

A blurry retinal image that slips past quality checks does not merely waste storage. If such images enter training with unreliable labels or appear frequently in production, they distort the statistical evidence available to the model and degrade clinical safety. Quality assurance ensures that incoming data satisfies the invariants downstream stages depend upon. In the diabetic retinopathy scenario, automated quality-scoring models deployed at the fundus camera immediately flag out-of-focus or improperly framed captures, prompting clinical staff to retake the photograph while the patient is still present rather than discovering the defect weeks later during model training.

Validation extends beyond image quality to verify proper labeling, patient association, and privacy compliance. Local validation catches problems at the point of capture; centralized validation detects distributional anomalies across the full clinic network, for instance by flagging when a particular site's images skew toward a narrow demographic range that would bias the training set.

↪ Data cascades, in which uncaught collection defects compound into downstream training and evaluation failures, are developed in section 4.3.1.

Data collection decisions directly constrain model development: bandwidth limits dictate what architectures are feasible, privacy requirements shape training pipelines, and quality variations across clinic environments determine robustness requirements. Figure 3.5 traces how downstream operational discoveries route corrections back across upstream stage boundaries. The figure zooms into stages 2 through 6, drawing data collection and data preparation as separate boxes and omitting problem definition, which sits upstream of every path shown. In the diabetic retinopathy deployment, when model evaluation uncovers performance deficits on legacy fundus cameras, the *Data gaps* pathway triggers targeted image collection from those specific clinics. Similarly, when validation across diverse patient cohorts reveals lower sensitivity for patients with cataracts, the *Validation Issues* vector motivates retraining with synthetic lens-opacity augmentations. In production, runtime telemetry routes along the *Data Quality Issues* and *Performance Insights* paths back into data preparation and collection whenever clinic hardware upgrades alter optical characteristics.

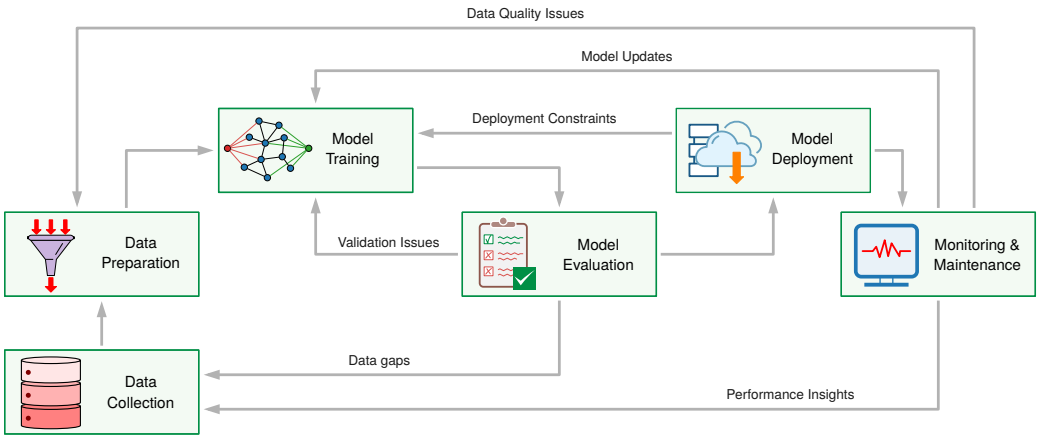


Figure 3.5: Feedback Paths Across Lifecycle Stages: Multi-stage feedback pathways show how downstream discoveries can require upstream corrections. Data-quality and distribution gaps detected during evaluation may prompt targeted collection or augmentation, while operational constraints and performance drift can motivate model, data, or pipeline changes after investigation.

These feedback pathways settle what data the training set contains. A curated, validated dataset can still starve the next stage, however, if the order in which training reads it fights the storage system that holds it.

3.8.4 Sample order and read locality

The DR training procedure considered here reads the training set, on the order of 10^5 fundus photographs, in full on every epoch (one complete pass over the training set), using a fresh random order each time. Whether the accelerator stays busy during those passes comes down to one comparison: the rate at which the data loader delivers images against the rate at which training consumes them. If delivery is slower, the accelerator idles however fast it is. The workflow must

therefore record both the sample order training requires and the delivery rate the loader actually achieves, and test one against the other before treating the data handoff as complete.

This is a different locality question from the data locality invariant (2.3) applied earlier to the clinic uplink, which decides *where* computation runs relative to the data. Here the data is already placed, and the question is the *order* in which training reads it. Training shuffles examples across epochs to reduce dependence on their stored order; shuffling alone does not guarantee independent and identically distributed (i.i.d.) samples. Randomized order works against the locality that operating systems exploit,[14] the tendency for a program that reads byte X to read nearby bytes soon and to reuse recently accessed memory.

When the random sample order maps to scattered physical reads, shuffling weakens the file-system buffers that keep recently read blocks in memory and the prefetchers that fetch the next blocks before they are requested. The penalty depends on how logical examples map to physical storage. Datasets are usually packed into shards, large files that each hold many examples. Reading whole shards contiguously and shuffling within and across them retains useful locality, whereas issuing small reads scattered across many shards exposes the full latency of storage and network on every request. Layout-aware batching, caching, and explicit prefetching can restore locality and reduce these stalls. The relevant systems variable is the physical access pattern presented to the memory hierarchy, not randomization alone.

That distinction determines where to optimize when the delivery test fails. If the loader already issues large sequential reads, more caching adds little benefit. If it fans out small reads across many shards, repacking records or increasing the size of each read request matters far more than adding compute. The storage mechanisms themselves belong to data engineering; the workflow's responsibility is to run the delivery test and route a failure to the right fix.

Data collection therefore does not end when training begins. Feedback from evaluation and production keeps revising what is collected, and the delivery test keeps checking whether the loader can feed what the model consumes. What this stage hands forward is a versioned dataset, its preprocessing pipeline, and a measured delivery rate, which together become the raw material that model development turns into trained models.

3.9 Model Development and Training

The DR team has 128,000 labeled retinal images, a validated preprocessing pipeline, and a target: more than 90 percent sensitivity on edge hardware with less than 50 ms inference latency. The question is no longer *what data* to collect but *what model* to build, and that question has no answer independent of the deployment constraints already established. In iron law terms, this stage defines the operations (O) term: architectural choices set the computational floor that hardware must sustain. Training systems must coordinate data ingestion, accelerator execution, and hyperparameter exploration[15] without stalling execution pipelines. In high-stakes domains like healthcare, every design decision affects clinical outcomes, so technical performance and operational constraints must be integrated from the start.

The DR system faces a sharp training challenge: achieve expert-level diagnostic accuracy with finite labeled-data and optimization budgets. **Transfer learning**[16] addresses this constraint by adapting a model pretrained on a source task to a target task. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 training subset contains 1.3 million labeled images (Russakovsky et al. 2015). Reusing learned representations can improve target-task optimization and generalization relative to random initialization, with the benefit depending on source-target similarity, which layers are transferred, and transfer-related optimization effects (Yosinski et al. 2014).

In the controlled validation study reported by Gulshan et al. (2016), transfer learning and a labeled dataset of 128,000 images produced an AUC[17] of 0.99, with sensitivity of 97.5 percent and specificity of 93.4 percent at one operating point. The result demonstrates the potential of large-scale pretraining with domain-specific fine-tuning under the study conditions; it does not by itself establish deployment performance. Reaching a result of this kind takes repeated training runs over the full labeled dataset, and the arithmetic cost of each run on the available hardware sets how many runs a team can afford, which is the iteration tax again in another form.

14 **Locality of reference**: Denning (1968) formalized the working-set principle for virtual memory. Randomly ordered examples can reduce locality when the logical order maps to scattered physical reads, but the effect depends on storage layout, batching, caching, and prefetching.

↩ I/O stalls and storage throughput bottlenecks limit pipelined training progress under the bottleneck principle established in section 2.3.1.

15 **Hyperparameter**: Architectural and optimizer choices (for example, learning rate and network depth) affect the computational work of training. A naive Cartesian grid that trains every combination independently incurs a multiplicative search cost: 5 hyperparameters with 4 values each produces 1,024 (4^5) configurations. Early stopping, multifidelity methods, and adaptive search can avoid completing every run, which is why the grid represents a baseline rather than an unavoidable cost.

↪ The training pipeline that keeps data ingestion, forward and backward passes, and parameter updates from stalling the accelerator is developed in section 8.5.

16 **Transfer learning**: For the same architecture, transfer learning changes how parameters are initialized and optimized, not the operations required at inference.

Achieving high statistical accuracy on benchmark data is only the first hurdle. In production, edge deployment imposes non-negotiable physical boundaries: models must fit within strict memory allocations, execute within tight latency envelopes, and operate within thermal limits.

Ensemble learning[18] illustrates the tension between statistical maximization and operational feasibility. Combining predictions from multiple models—whether through bagging data subsets, boosting residual errors, or stacking diverse architectures—routinely delivers top benchmark accuracy. Yet inference compute, parameter storage, and memory traffic scale directly with ensemble size. Serial execution increases latency, while parallel execution demands additional hardware concurrency. The historic Netflix Prize demonstrated how ensemble complexity can preclude deployment: the winning solution delivered the highest predictive accuracy, yet Netflix never deployed it because the operational costs of maintaining and serving the ensemble exceeded the marginal business value (Johnston 2012). Even an ensemble of lightweight models multiplies orchestration complexity, working memory, and operational monitoring. Every architectural decision creates downstream systems ripples that accumulate at the physical deployment boundary.

3.9.1 Accuracy vs. efficiency

Clinical diagnostics require specialized statistical objectives[19] that differ fundamentally from the generic cross-entropy loss optimized during training. In diabetic retinopathy screening, missing an active case risks permanent vision loss, whereas misclassifying a healthy eye generates an unnecessary hospital referral. The system must therefore maintain high sensitivity (> 90 percent) to catch pathology while preserving high specificity (> 80 percent) to avoid overwhelming regional healthcare infrastructure. Furthermore, these targets must hold across diverse patient demographics, varying illumination, and distinct fundus camera models.

Optimizing for clinical metrics alone remains insufficient if the resulting architecture cannot execute within the physical deployment envelope. In this scenario, running the model must remain strictly within the 500 MB memory reservation on the clinic's shared 4–8 GB edge device. That budget must accommodate the static weight parameters alongside input buffers, dynamic activation tensors, and runtime execution overhead. Simultaneously, the accelerator must operate within a 20 watt power envelope and return predictions within 50 ms. Balancing these competing pressures requires navigating the trade-offs of the iron law: shrinking parameter volume (D_{vol}) and arithmetic operations (O) through systematic pruning, quantization, and distillation, while continuously revalidating that compressed representations preserve clinical sensitivity.[20]

3.9.2 Constraint-driven development

Real-world constraints shape model development from initial exploration through final optimization, demanding systematic experimentation. Development begins when data scientists collaborate with domain experts (ophthalmologists in the DR case) to identify subtle lesions and image-quality requirements that matter clinically. Without that domain knowledge, a model architect might choose a resolution or receptive field, the input region each internal feature can see, that discards relevant detail before the network can use it. This interdisciplinary approach helps model architectures preserve clinically relevant evidence while respecting the computational constraints identified during data collection.

Computational constraints profoundly shape experimental approaches. Multiple model variants, hyperparameter sweeps, and preprocessing approaches can make exhaustive experimentation expensive. This economic reality drives investments in better job scheduling, caching of intermediate results, early stopping, and automated resource optimization. Systematic hyperparameter optimization and disciplined experiment design can reduce computation relative to exhaustive search.

The inherent uncertainty of ML outcomes demands scientific methodology: controlled variables through fixed random seeds and environment versions, systematic **ablation studies**[21] to isolate component contributions, confounding factor analysis to separate architecture effects from optimization effects, and statistical significance testing across multiple training runs using paired offline tests rather than the A/B testing[22] reserved for comparing models on live production traffic. Without this rigor, teams cannot distinguish genuine performance improvements from statistical noise, a dis-

17 **AUC (area under the ROC curve)**: Measures the area under the receiver operating characteristic (ROC) curve plotting true positive rate vs. false positive rate across all classification thresholds, ranging from 0 to 1; 0.5 represents random discrimination, and values below 0.5 are possible. Unlike accuracy, AUC is threshold-independent and insensitive to class prevalence, making it a common metric for medical screening systems. The systems consequence: a model with 0.99 AUC can still produce unacceptable sensitivity at the specific operating threshold chosen for deployment, so AUC alone cannot validate deployment readiness.

18 **Ensemble learning**: Combines predictions from multiple models (bagging, boosting, stacking). Inference compute and model storage generally sum across constituents. Serial execution can increase latency, while parallel execution trades latency for additional hardware and coordination; no fixed proportional latency multiplier follows from ensemble size.

↪ The probability outputs and cross-entropy losses that standard classifiers optimize are developed in section 5.5.3.2.

19 **Medical AI performance metrics**: Medical AI evaluation emphasizes sensitivity (true positive rate) and specificity (true negative rate) alongside aggregate accuracy. This DR scenario sets a >90 percent sensitivity target because missed referable disease can delay care and contribute to avoidable vision loss; actual acceptance criteria are product- and regulator-specific. The subtler systems trap is positive predictive value (PPV): even a high-accuracy model can have low PPV in a low-prevalence population. This prevalence dependence can require different operating thresholds across deployment sites, a constraint invisible in standard ML evaluation.

tinction that becomes critical when a 0.5 percentage-point sensitivity difference determines whether a model meets the clinical sensitivity threshold.

At every development milestone, teams validate models against the deployment constraints identified in earlier lifecycle stages. Each architectural innovation must be evaluated for accuracy improvements and compatibility with edge device limitations and clinical workflow requirements. This dual validation approach ensures that development efforts align with deployment goals rather than optimizing for laboratory conditions that do not translate to real-world performance.

3.9.3 Prototype to production

A small team of data scientists can often coordinate exploratory experiments using ad hoc scripts, spreadsheets, and shared notebooks. As projects evolve from prototype to production, however, complexity expands across multiple dimensions simultaneously: datasets grow, models require distributed training across accelerator clusters, and multiple engineers launch concurrent runs. Under these conditions, informal coordination rapidly becomes a development bottleneck. Without shared infrastructure, team members inadvertently train on conflicting data splits, duplicate expensive preprocessing passes across separate machines, overwrite shared model checkpoints, and struggle to reconcile divergent notebook versions. The time required to untangle conflicting artifacts and unrecorded configurations begins to dominate development.

Shared workflow platforms resolve this coordination friction by standardizing stage interfaces and centralizing resource management. Rather than relying on manual handoffs and uncoordinated scripts, a shared platform provides standardized data partitioning, cached preprocessing pipelines that compute intermediate representations once for the entire team, and automated job schedulers that allocate cluster accelerators efficiently. Furthermore, shared platforms track experiment metadata systematically, linking code revisions to exact training configurations, hyperparameters, and resulting evaluation metrics. Operating at scale therefore requires teams to systematically preserve each experiment's inputs, software dependencies, and execution configuration, establishing the baseline needed to verify genuine improvements.

3.9.4 Reproducibility and technical debt

Shared infrastructure can accelerate experimentation, but rapid iteration creates a hidden liability. If experiments are not reproducible, the team cannot reliably distinguish genuine improvements from noise, and the codebase accumulates technical debt[23] that compounds with every unreproducible result.

Reproducing ML results requires tracking data versions, random seeds, hardware configurations, and library versions alongside program logic. Hardware and nondeterministic operations can alter training results, so teams need lineage to distinguish architectural changes from run-to-run variation. When parallel accelerator threads accumulate gradients, the order in which partial sums arrive can differ from run to run, and floating-point addition is non-associative ($(a+b)+c \neq a+(b+c)$), so identical code can produce slightly different weights. Optimized math libraries may also benchmark several compute kernels (the routines that execute an operation such as a matrix multiplication on the accelerator) at runtime and select different ones across runs or machines unless deterministic execution is explicitly requested. Recording a random seed alone therefore does not establish reproducibility; the execution configuration and the measured variation across repeated runs matter as well. Systematic **experiment tracking** records unique run identifiers and artifact versions. Systems such as MLflow and Weights & Biases link data versions, code commits, hyperparameters, and resulting models so teams can reconstruct differences between runs.

The cost of neglecting **reproducibility** is economic, not just scientific. Teams that cannot reproduce a result waste cycles rerunning experiments that may not converge to the same outcome. Reproducibility infrastructure such as versioned environments, controlled pipelines, and automated checkpointing reduces redundant computation and supports more confident architectural decisions.

3.9.5 Reproducible system artifacts

Experiment tracking needs a well-defined unit to version, and that unit is the stage's actual deliverable. A common failure mode is treating the trained model weights as the sole output of model

20 **Model compression pipeline**: Bridging the gap between research accuracy and edge deployment requires an iterative "compress-validate-adjust" loop. Each compression step can reduce model size or execution cost, but it can also silently degrade sensitivity below the clinical threshold. Finding a model that fits in device memory while preserving clinical sensitivity typically requires multiple iterations because only the full validation suite reveals whether the smaller model still satisfies the problem definition.

↪ Renegotiating accuracy against memory and latency envelopes through systematic pruning, quantization, and distillation is developed in section 10.1.

21 **Ablation studies**: Named for surgical tissue removal, ablation studies systematically disable individual components to isolate their contribution to performance. The rigor matters because a 0.5 percentage-point sensitivity difference can determine whether the DR model meets its clinical sensitivity threshold. Without ablation, a team cannot distinguish a genuine architectural improvement from noise introduced by a different random seed, wasting iteration cycles on phantom gains.

22 **A/B testing in ML**: Compares a new model (B) against the production baseline (A) on randomly assigned live traffic to estimate the model's causal effect. Required sample size depends on the baseline event rate, effect size, variance, allocation, significance level, and statistical power; no universal interaction count or duration follows from a 0.5 percentage-point change alone.

development. In a mature ML workflow, the deliverable is a **reproducible system artifact** whose inference package has four components:

- **Model weights**: The learned parameters.
- **Inference code**: The exact code used to run the model, including preprocessing logic.
- **Environment specification**: The complete dependency graph (for example, Docker container, `requirements.txt`, CUDA drivers) required to execute the code.
- **Configuration**: Hyperparameters and runtime settings.

Reproducing inference and rebuilding a trained candidate require different evidence. Alongside the inference package, the artifact must link to the dataset version, training-code version, preprocessing recipe, training configuration, and execution environment that produced the weights. Under the data-as-code principle (1), a changed training set changes learned behavior as surely as changed inference code does. This provenance makes the training procedure repeatable; the nondeterminism described in section 3.9.4 still requires checking variation across runs rather than promising identical weights.

Without bundling the environment with the model, dependency mismatches can create dangerous failures. Some incompatibilities, including unsupported CUDA combinations or missing libraries, fail loudly and immediately. Other differences in compatible kernels, linear algebra libraries, or image-resizing routines such as OpenCV and PIL may alter floating-point results or pixel interpolation without crashing the serving process. These changes can shift model outputs and affect accuracy. Environment reproducibility is therefore necessary not only for successful execution but also for validating equivalent inference behavior. A model that achieves 99 percent accuracy in development must be reevaluated if its production environment changes, even when inference completes without an exception.

Reproducible, optimized models are necessary but not sufficient. A model that achieves expert-level accuracy on curated research data may still fail in production. The next stage subjects these trained artifacts to systematic testing against the conditions they will actually encounter.

23 **ML artifact interdependence**: ML artifacts are deeply interdependent. A model's measured accuracy is tied to the data version, preprocessing pipeline, and hyperparameter configuration used to produce it. Without lineage tracking, the team may not be able to determine whether a regression stems from code, data, or run-to-run variation, forcing expensive reruns of experiments whose provenance is lost.

3.10 Evaluation and Validation

Suppose the DR screening model achieves an AUC of 0.99 on a curated research dataset, matching the controlled study reported during model development. On field images from a rural clinic, where a newly trained technician operates an older fundus camera, its sensitivity could fall well below the 90 percent target. The software would execute without throwing an exception or returning an error code; the model's learned features would simply encounter pixel distributions distorted by optical blur, low illumination, and variable pupil dilation that never appeared in the training distribution. High accuracy on curated test partitions does not guarantee operational reliability in production. Before deployment, trained models must pass through systematic evaluation and validation gates to verify that they satisfy accuracy, latency, and robustness requirements across realistic deployment conditions. This stage transforms raw checkpoint files into production-ready software systems by measuring behavior against predefined operational thresholds.

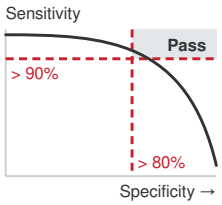
Definition 3.3: Model validation

Model validation is an evidence gate for deciding whether a trained model is suitable to deploy for a specified use. It tests the model against deployment constraints such as latency targets, subgroup performance targets, cost budgets, and robustness under distribution shift, but cannot certify safety under every possible condition.

1. **Significance**: Validation adds dimensions that test-set accuracy ignores. A model achieving 95 percent accuracy on a static test set may miss a 100 ms latency target on its slowest requests, underperform a required subgroup threshold such as a maximum five percentage-point performance gap across demographic groups, or lose more than ten percentage points of accuracy under common image-quality problems such as blur and glare. Each unchecked dimension is a deployment risk that compounds silently in production.

2. **Distinction**: Evaluation measures model behavior using chosen datasets, metrics, and experiments, which may include out-of-distribution and subgroup tests. Validation assembles the broader evidence needed for a specified deployment decision, including system latency, throughput, robustness, operational readiness, and cost.
3. **Common pitfall**: A frequent misconception is that validation is "one more test." In reality, it is a multi-dimensional gate: a model can pass the accuracy test and still fail validation because it violates latency, subgroup reliability, or cost constraints that accuracy alone does not measure.

Evaluation supplies measurements; validation uses them to decide whether the integrated software and hardware pipeline meets its intended deployment requirements. This deployment decision is broader than evaluation on a validation set, the data partition used to guide model selection. A separate test set supplies evidence after those choices are fixed; repeatedly using it to choose a model weakens its independence. Accuracy on held-out examples supplies only part of the release evidence, alongside tests of execution, robustness, and operational readiness. Metric thresholds, controlled production exposure, environmental stress tests, and any required regulatory qualification address complementary risks, with the evidence and sequence determined by the intended use.



Sweeping the threshold traces one illustrative curve; it must reach the corner where both floors hold.

3.10.1 Evaluation metrics and thresholds

Evaluation metrics must reflect the different consequences of errors in the target application. In diabetic retinopathy screening, standard classification accuracy is misleading due to class imbalance: a trivial classifier predicting no disease for every sample would achieve high accuracy in a low-prevalence screening population while missing every treatable pathology. The scenario therefore sets separate requirements: sensitivity above 90 percent limits missed disease, while specificity above 80 percent limits false-positive referrals. Neither requirement alone establishes adequate clinical performance or referral capacity; that judgment also depends on disease prevalence, screening volume, and available care. The ROC curve summarizes how detection and false-positive rates change as the classification threshold varies. Evaluation must determine whether any threshold satisfies both requirements under the intended operating conditions.

Aggregate metrics can conceal localized failure modes. Stratified evaluation partitions the test corpus across metadata dimensions, such as patient age brackets, chronic comorbidities, and clinic camera models, to measure performance parity across slices. A vision backbone achieving 94 percent overall accuracy across an entire benchmark may fall below 80 percent on images captured from patients with cataracts or under low ambient lighting. These localized performance drops remain invisible in aggregate evaluations but manifest as severe safety failures once deployed. Systematic benchmarking methodologies evaluate these sliced distributions automatically before promoting a candidate checkpoint.

In addition to label accuracy, evaluation must measure probability **calibration**:[24] whether a model's predicted confidence score aligns with its empirical error rate. Modern deep neural networks frequently produce uncalibrated posteriors, outputting overconfident predictions (> 90 percent confidence) even on samples near decision boundaries. When automated triage pipelines route borderline cases to human specialists based on confidence cutoffs, miscalibration misallocates clinical attention and undermines practitioner trust.

3.10.2 Offline and online evaluation

The validation gate begins offline on static datasets and progresses incrementally toward live production traffic. Accuracy-only evaluation on held-out test partitions establishes predictive behavior but does not exercise bursty concurrency, network serialization, or telemetry overhead. Offline load tests and replay can expose such systems bottlenecks when they reproduce the relevant conditions; live traffic supplies additional evidence about the deployed environment. For a centralized service, online evaluation tests candidate models under real serving dynamics using **staged rollout**:[25]

1. **Shadow mode** (dark traffic): The API gateway mirrors live production requests asynchronously to the candidate model running on dedicated accelerator resources. The candidate executes

[24] **Calibration**: A calibrated model's predicted probabilities match observed frequencies: 80 percent confidence should correspond to 80 percent correctness. Calibration is distinct from accuracy, and the systems consequence for the DR system is severe: clinicians use confidence scores for triage decisions, so a miscalibrated model that assigns 90 percent confidence to uncertain cases can misdirect clinical workflows more dangerously than a less accurate but well-calibrated alternative. Platt scaling and temperature scaling can improve posttraining calibration, but their effectiveness must be evaluated on representative data.

[25] **Staged rollout**: Shadow mode runs the new model in parallel, logging predictions without serving them. Canary deployment (named after coal-mine canaries) then exposes a limited, configurable share of traffic and increases it gradually if metrics hold. Coverage depends on traffic, observability, duration, and the failure modes exercised, so staged rollout reduces risk without guaranteeing that it catches a fixed share of production issues.

inference under true input dimensions and concurrent request arrival patterns, but its predictions are written to telemetry logs rather than returned to clients. This isolates tail-latency spikes, out-of-memory (OOM) faults, and numerical divergences without user-facing blast radius.

2. **Canary deployment**: The ingress load balancer directs a small fraction of live traffic (such as 1 to 5 percent) to the new serving container, returning live predictions while monitoring systems telemetry for crash loops, latency regression, and downstream error spikes.
3. **A/B testing**: Traffic splitters partition user requests across candidate and baseline models using deterministic hashing, providing statistical evidence to verify whether algorithmic improvements translate into measurable business or clinical gains.

This staged rollout workflow manages deployment validation risk; it is distinct from the cross-tier model compression pattern termed progressive deployment in section 2.10.1.

Each validation stage contributes distinct, complementary operational evidence, as table 3.4 summarizes.

↪ Shadow and canary releases, the production form of this staged rollout, are operationalized alongside blue-green switching in section 14.5.3.

Table 3.4: Staged Validation Coverage: The stages provide complementary, overlapping evidence rather than partitioning failures into exclusive classes.

Validation stage	Typical evidence it adds
Offline evaluation	Algorithmic behavior
Shadow mode	Integration behavior
Canary deployment	Limited-traffic behavior
A/B testing	Comparative outcomes

Serving infrastructure must support this staged validation pipeline by design. Retrofitting dynamic traffic splitting, asynchronous request mirroring, and differential output logging onto an existing centralized deployment stack requires extensive modifications to API gateways and telemetry pipelines. In the DR scenario, intermittent connectivity and local execution move these evidence gates to clinic devices and pilot sites. Teams can compare a candidate against recorded clinic inputs, run it alongside the active model where device resources permit, and introduce an approved update at a small group of clinics before expanding the rollout. The workflow must preserve the evidence and exposure limits of each gate while adapting its implementation to the deployment target.

3.10.3 Production-condition validation

A model evaluated on one clinic's images still needs evidence for the other cameras, capture conditions, and patient populations specified during problem definition. Production-condition validation tests these requirements across site heterogeneity, input-level perturbations, and temporal drift. These dimensions apply within the offline and controlled-production gates in section 3.10.2; they are not stages that begin after live A/B testing. Held-out samples from the training distribution alone cannot supply all of this evidence.

External and multisite validation address site-level distribution shifts: determining whether the model has learned generalizable representations or overfit to the sensor hardware of the development environment. In retinal screening, camera sensor sensitivities, optical lens coatings, and illumination spectra differ across equipment vendors. A DR model trained exclusively on images from high-end fundus cameras in research hospitals can fail when deployed to community clinics using older hardware. Multisite validation requires benchmarking candidate models on data collected across independent clinic sites with varying hardware configurations, ambient lighting environments, and operator skill tiers.

Robustness testing shifts focus to individual inference requests, evaluating resilience against realistic physical distortions and edge cases. For computer vision pipelines, these perturbations include sensor noise, optical blur, uneven contrast, and partial pupil occlusions. In the Thai DR clinical study (Beede et al. (2020)), field technicians with limited formal training frequently captured off-center or poorly focused fundus images. Evaluating models against synthetic and empirical

perturbation suites exposes whether the serving pipeline requires an upstream image-quality filter to reject corrupt frames before allocating accelerator cycles for inference.

Temporal validation evaluates whether the model maintains predictive accuracy as the operating environment evolves over weeks and months. Cross-sectional test sets drawn from a fixed historical window fail to capture subsequent shifts in patient demographics, disease epidemiology, or clinical practice patterns. Input distribution changes with the relationship between inputs and labels unchanged produce *covariate shift*, whereas shifts in the relationship between input features and clinical diagnoses cause **concept drift**. Both phenomena can cause silent model degradation and motivate the continuous telemetry and drift detection pipelines described in section 3.12.

3.10.4 Regulatory validation

Healthcare AI systems face device- and pathway-specific validation requirements. When FDA premarket review is required, the safety and performance evidence must be appropriate to the device and its regulatory pathway; clinical data are required for some submissions, not all.[26]

Domain-specific validation extends beyond statutory compliance to address clinical utility and safety protocols. In diabetic retinopathy screening, clinical validation studies deploy the automated system alongside licensed ophthalmologists, comparing model classifications against ground truth established by consensus adjudication panels. These studies must demonstrate not only non-inferior diagnostic accuracy but also safe failure modes: systems that *fail safely*, by routing ambiguous or low-confidence samples to clinical specialists, integrate successfully into care delivery, whereas systems that *fail silently* produce diagnostic errors that no one sees until patients are harmed.

Human factors validation evaluates the model as an embedded component within the human decision-making loop. An accurate model that induces high operator cognitive load, exhibits excessive inference latency that stalls patient intake, or triggers frequent false alarms creates alert fatigue, prompting clinic staff to disable or bypass the system. Validation protocols must therefore quantify sociotechnical metrics, including clinician override frequency, examination turnaround time, and referral concordance, alongside raw statistical accuracy.

3.10.5 Deployment readiness

Passing the validation gate produces an immutable release manifest recording the evidence and approval for a particular model version, intended use, and operating conditions. This manifest records the exact model checkpoint hash, preprocessing container configuration, calibration parameters, benchmark scores across designated demographic and equipment slices, and any required regulatory authorization. The manifest binds the release to that evidence; it does not itself confer external authorization or establish performance outside the evaluated conditions. If a model fails to clear an established threshold, such as p99 serving latency exceeding the service level agreement (SLA) budget or sensitivity on low-light fundus images falling below the safety floor, the release gate halts deployment. Diagnostics then route backward to the responsible lifecycle stage: data curation for targeted data collection, model training for loss function re-weighting, or runtime optimization to reduce execution cost.

Clearing the validation gate establishes the evidence and approval for a particular system configuration. Deployment must install and sustain that configuration at each clinic, including its preprocessing routines, compiled model, telemetry collectors, and rollback triggers. Rollout checks confirm that the installed serving path matches the evaluated conditions and that local integration has not invalidated the release evidence.

3.11 Deployment and Integration

A model that passes validation in development faces different failure modes once deployed to production. In the DR scenario, the model must run on Orin Nano-class edge devices in rural clinics with intermittent connectivity, integrate with hospital information systems with distinct schema requirements, and deliver predictions clinicians can interpret directly. The scenario's latency and connectivity requirements rule out round-trip cloud inference. Deployment is where abstract operational constraints become physical systems requirements. In iron-law terms, the serving path couples data movement, execution efficiency, and fixed overhead together; which term binds varies by archetype. ResNet-50-class vision workloads may use batching to improve throughput,

[26] **FDA AI/ML regulation**: FDA marketing authorization requires evidence appropriate to the device, intended use, risk, and regulatory pathway. The FDA's 2021 AI/ML SaMD Action Plan identified lifecycle management, real-world performance monitoring, and predetermined change control planning as central issues for adaptive medical-device software (U.S. Food and Drug Administration 2021). These concerns make model versions, training provenance, monitoring, and change documentation important workflow artifacts without imposing one identical audit artifact at every lifecycle stage.

↪ Validated release packages transition from throughput-oriented training to per-request latency objectives under the serving paradigm developed in section 13.1.

DLRM-class recommendation workloads emphasize interactive latency and embedding access, and TinyML-class audio workloads operate under severe energy budgets (table 3.2). Dedicated serving infrastructure manages these trade-offs under live traffic.

3.11.1 Deployment requirements

Deployment translates an abstract computational graph into hard physical budgets: peak memory footprint, execution latency, thermal dissipation limits, and network bandwidth. In the clinical screening scenario, these constraints force an architectural choice between centralized cloud serving and localized edge execution. A cloud deployment consolidates compute onto centralized accelerators, but requires stable wide-area network uplinks and incurs recurrent API and bandwidth charges. Conversely, an edge deployment eliminates network transit latency and operates during wide-area connectivity outages, but demands upfront hardware capital expenditure and restricts model size to local device memory. Napkin math 2.3 compared the two options for a single server; what changes here is scale, with per-image cloud billing and one edge device per clinic across the whole screening network. Evaluating this trade-off requires quantifying the relationship between capital expense, operating expense, and inference volume.

Napkin Math 3.3: Cloud vs. edge deployment economics

Problem: Treating the cloud option as a counterfactual cost benchmark because it fails the clinic's stated latency and connectivity requirements, compare the total cost of ownership and operational trade-offs between a cloud inference endpoint and on-premises edge servers for processing 760,000 billable screening images monthly across 500 clinics. Where the bandwidth calculation in notebook 3.2 sized one busy clinic uploading every photograph, this network-wide estimate assumes an average clinic at 50 patients/day and bills one graded image per patient rather than every capture, so its image volume is lower by design.

Option A: Cloud inference.

- Model runs on centralized GPU servers
- Inference cost: ~$0.01/image (cloud GPU time + API overhead)
- Annual inference cost across 500 clinics, 50 patients/day, 1 billable image per patient, 365 days/year, and $0.01/image is $91,250/year
- Plus: An assumed connectivity and network-operations allocation for uploading 5 MB per billable image = ~$45,000/year
- **Total**: ~$136,250/year operational cost
- Risk: 200 ms+ exceeds the scenario latency target; connectivity outages halt screening

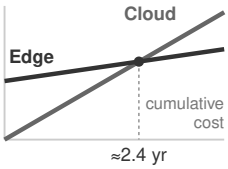
Option B: Edge deployment on a clinic device.

- One-time hardware: one $500/device per clinic across 500 clinics requires $250,000 capital expense
- Inference cost: ~$0.001/image (assumed all-in variable operating allocation)
- Annual cost: approximately $25,000/year maintenance plus approximately $9,125/year variable inference cost, for about $34,125/year
- **Total**: $250,000 upfront + ~$34,125/year
- Benefit: latency below 50 ms; works offline; much lower per-inference cost

Math:

- Annual savings: $136,250/year (cloud) − $34,125/year (edge) = $102,125/year
- Payback: $250,000 (hardware) ÷ $102,125/year = ~2.4 years

Systems insight: Under these scenario assumptions, edge deployment pays back in ~2.4 years and allows inference during connectivity outages, yet it requires tighter model optimization (must fit in edge memory) and more complex update pipelines. The deployment paradigm selected during Problem Definition determines whether the edge option is even viable.



Edge costs more up front, but cloud's running cost passes it in about 2.4 years.

In this scenario, bandwidth limits, latency targets, and offline operational requirements favor edge deployment. That choice dictates strict memory, latency, and thermal envelopes that model compression must satisfy and serving runtimes must execute within production bounds.

Host integration extends these physical constraints to the software interfaces of existing clinical infrastructure. The inference runtime must exchange data with hospital information systems (HIS) to ingest patient records and record diagnostic output. The model's prediction scores do not automatically provide reliable uncertainty estimates. Their interpretation must be evaluated on representative data before an interface contract uses them to route decisions. This contract specifies which classifications trigger automated referrals and which demand physician review, including how rejected or low-quality inputs are handled. A confidence threshold cannot be evaluated in isolation; its operational safety depends on the downstream workflow it triggers. Regulatory frameworks introduce structural constraints on this data lifecycle. Legal mandates such as HIPAA govern data transmission and restrict whether production inference payloads can be persisted, linked to patient identifiers, or ingested into retraining pipelines. When compliance prohibits retaining raw inputs, continuous improvement cannot rely on passive telemetry logging. Privacy rules thus constrain ongoing model maintenance strategies rather than serving merely as a transport-layer security checklist (chapter 14).

3.11.2 Pilot to full deployment

Production rollout proceeds in discrete stages to isolate failure modes across expanding execution environments. Offline replay and synthetic test harnesses verify schema contracts and concurrency limits without risking patient safety. Subsequent pilot deployments in a small number of clinics expose the serving stack to physical variations absent from laboratory benchmarks: optical distortion across camera models, sensor noise under uneven clinic lighting, and operator handling variations. Full-scale rollout broadens exposure to the long tail of anatomical variations and rare pathologies. Detecting these distribution shifts requires continuous runtime monitoring rather than assuming validation distributions persist indefinitely.

Scaling across hundreds of remote clinics introduces hardware heterogeneity and operational friction across distributed nodes. Variations in local sensor optics alter input pixel distributions before execution begins, necessitating robust input-validation kernels that reject out-of-spec captures locally. At the same time, severe wide-area bandwidth constraints rule out continuous weight synchronization or streaming high-resolution telemetry back to a centralized cluster. Edge nodes must instead operate autonomously, caching execution metrics locally and synchronizing state opportunistically during scheduled maintenance windows.

Sustaining clinical utility requires calibrated uncertainty estimation rather than raw aggregate accuracy. When the model encounters ambiguous pathology, emitting an uncalibrated prediction risks diagnostic failure. Instead, an operational serving path incorporates **human-in-the-loop routing**, using calibrated decision boundaries to direct borderline cases to expert clinicians. This routing mechanism serves a dual purpose: it acts as a real-time safety boundary for patient care, and it establishes a continuous data collection channel. Adjudicated edge cases yield verified, high-information annotations that are fed back into subsequent training iterations, directly counteracting the model's known failure modes.

Maintaining consistency across distributed deployments demands immutable model artifact versioning and atomic update procedures. Client runtimes must verify cryptographic checksums of compiled model binaries before loading new weights into memory, preventing partial updates from corrupting edge execution. Deployment marks the transition from static offline evaluation to continuous operations, where system telemetry continuously assesses whether real-world latency distributions and inference behaviors remain within specified operating bounds.

3.12 Monitoring and Maintenance

Consider an illustrative case six months after a DR screening system launches. A clinic upgrades its fundus cameras, and the new equipment produces images with different color profiles. In this assumed outcome, the model's sensitivity drops at that site because the pixel distributions it learned during training no longer match the images it receives; the monitoring team cannot establish that drop from the camera change alone. No code changed. The data drifted beyond the training

envelope, and the model degraded silently. Where paired drift and outcome data support a local fit, the degradation equation (equation 1.2) describes the mechanism. The new color profile raises the statistical divergence $\mathcal{D}(P_t \| P_0)$ between production and training images, and accuracy falls by roughly the fitted sensitivity λ times that divergence, with nothing in the serving logs to show it. *ML systems can degrade through data drift even when their artifacts remain untouched.* This possibility means that deployment is not the end of the lifecycle but the beginning of an ongoing operational phase. Monitoring provides the statistical telemetry to detect degradation, while continuous maintenance pipelines update models when evidence warrants it.

In this illustrative scenario, monitoring tracks performance across clinics to detect whether changing patient demographics, camera technology, or equipment degradation affects accuracy. Adding a new imaging modality such as optical coherence tomography would be a product change requiring new data, validation, workflow integration, and any applicable regulatory review rather than routine maintenance. Three feedback pathways guide update decisions: performance evidence can motivate targeted data collection, data-quality findings can prompt preparation changes, and confirmed model degradation can justify retraining or another intervention. Drift thresholds initiate review; they do not select the remedy automatically.

↪ Divergence metrics and threshold conventions for measuring distribution drift are developed in section B.2.2.

3.12.1 Production monitoring

Monitoring must serve two audiences simultaneously: technical teams tracking system health metrics and clinical staff needing actionable insights. Initial deployment may reveal blind spots invisible during laboratory validation.[27] Clinics with older equipment may show accuracy decreases. Specific patient subgroups, such as those with proliferative retinopathy or cataracts complicating the fundus image, may trigger higher error rates. These discoveries drive targeted data collection and architectural improvements.

A DR screening system, where missed referable disease can delay care and contribute to avoidable vision loss, demands continuous operational monitoring plus periodic performance evaluation when labels arrive. Teams establish quantitative thresholds for latency, accuracy, and data distribution stability. Lightweight statistical tests for comparing production distributions against baseline data (such as the population stability index and Kolmogorov-Smirnov test)[28] trigger responses ranging from automated alerts to retraining reviews.

A production DR system can track four metric categories at different timescales. The following thresholds illustrate one scenario policy; clinical and operational teams must validate them for the intended product, population, and deployment environment.

- **Model performance metrics** (requiring ground truth, available with delay): sensitivity (target above 90 percent, alert if seven-day rolling average drops below 88 percent), specificity (target above 80 percent, alert if it drops below 78 percent), and subgroup performance (alert if any demographic drops more than 5 percentage points below baseline).
- **Proxy metrics** (available immediately, without ground truth): prediction confidence distribution (alert if mean confidence drops more than 10 percent relative to baseline), referral rate (alert if rate changes more than 15 percent from baseline), and image quality rejection rate (alert if more than 20 percent of images fail quality checks).
- **Operational metrics**: Inference latency (p95 below 50 ms, alert if above 100 ms), throughput (alert if queue depth exceeds 50 images), and error rate (alert if more than 0.1 percent of requests fail).
- **Data stability metrics**: Feature and prediction distributions compared with a baseline, with alerts when recent traffic moves outside the expected range.

The displayed alerts are urgent escalation thresholds, distinct from the acceptance targets. A measured target violation already opens a review of the affected site and system version; remaining between the target and the urgent threshold does not establish acceptable operation. The review checks the observation window, sample coverage, uncertainty, and the agreed response policy. Crossing an urgent threshold accelerates escalation under that policy. Neither threshold alone chooses retraining. The hierarchy matters because operational metrics surface immediate problems, proxy metrics flag possible model issues without ground truth, and labeled performance evidence arrives later.

[27] **Lab-to-clinic performance gap**: Medical AI systems can experience substantial performance changes when camera models, image quality, patient populations, or operator workflows differ from development data; the gap arises because training data cannot capture the full diversity of production conditions. FDA has emphasized total product lifecycle oversight and real-world performance monitoring for AI/ML-enabled medical devices, and device submissions may need evidence appropriate to the product's intended use and risk. For ML systems engineers, this means monitoring infrastructure should be a deployment prerequisite, not a postlaunch addition.

[28] **Population stability index (PSI) and Kolmogorov-Smirnov (KS) test**: Two lightweight statistical methods for detecting distribution drift; PSI bins features and computes divergence; thresholds such as 0.1 and 0.25 are operating conventions, not universal significance levels. The KS test measures maximum distance between empirical cumulative distributions. Both are cheap enough for frequent monitoring, but a detected input shift does not by itself prove an accuracy change; labels or other outcome evidence are needed.

↪ Drift detection pipelines that run these tests against training baselines in production are operationalized in section 25.

For the camera upgrade, an immediate color-distribution alert supplies a signal and a site identifier, while unchanged latency and error rates narrow the operational investigation. The monitoring owner opens an incident record linking camera configuration, preprocessing, and model versions. The data owner compares capture quality and preprocessing outputs with the validated camera conditions; the clinical evaluation owner arranges labeled evidence representative of the affected screening population. Reviewed referrals alone select cases according to the existing routing policy, so they cannot establish sensitivity for all screened patients. Until that evidence arrives, the team has evidence of changed inputs, not a measured sensitivity loss, and applies the previously approved exposure limits or alternative screening workflow for unevaluated conditions.

If the investigation identifies an incompatible camera configuration, restoring a validated configuration is one candidate remedy. If representative outcome evidence instead shows that acceptable new-camera images miss the sensitivity target, the response returns to data collection and model development. The owners version the added camera-specific data, any preprocessing changes, and the resulting candidate artifact. Evaluation must then repeat the affected camera and patient-slice tests, check unaffected slices for regressions, and confirm latency and memory on the clinic device. A revised release manifest records the evidence and approval before a controlled rollout. The incident closes only after post-update monitoring checks the installed configuration and the relevant outcomes.

3.12.2 Maintenance at scale

Model updates require careful validation and controlled rollouts. Teams employ A/B testing frameworks to evaluate updates and implement rollback mechanisms[29] that address issues quickly. ML systems must account for data evolution that can affect behavior independently of code changes.

In the illustrative scenario, scaling from pilot sites to hundreds of clinics increases monitoring complexity. The resulting log volume depends on request rates, sampling, payload sizes, and retention policies as well as the number of clinics. The monitoring infrastructure must track both global metrics and site-specific behaviors, maintain **data lineage**,[30] the metadata trail linking production logs to data, code, and model versions, where required for regulatory compliance, and correlate production issues with training experiments for root cause analysis.

Proactive maintenance closes the lifecycle loop. Operational signals identify potential problems, review determines whether newly validated data or another change is needed, and production evidence feeds back to refine problem definitions, data-quality standards, and architectural decisions. When retraining is selected, scheduled or evidence-triggered pipelines can incorporate approved data and return the candidate through validation. With the loop closed, the coupling among stages stops being a design-time concern and becomes a property of the running system, one whose consequences no single stage's metrics reveal.

[29] **ML rollback complexity**: An ML model's validity is coupled to the data distribution on which it was trained, not just its code. "Data evolution" means a simple rollback restores a model artifact but cannot restore the past data environment, creating a temporal state mismatch. Even a rapid rollback is therefore a mitigation tactic rather than a true system restore, as the stale model's performance on live data is not guaranteed.

[30] **Data lineage**: The automated recording of metadata linking each clinic's production logs to the exact data, code, and model version that generated them. Without this explicit trail, correlating a site-specific accuracy drop with a training experiment can require manual forensic analysis that delays root-cause identification.

3.13 Systems Thinking

The camera-upgrade case that opened the monitoring stage touched every stage at once. A change in capture hardware at one clinic shifted the input distribution, degraded a model that had passed validation, and required routing evidence back through the lifecycle to identify and validate a remedy. Three structural mechanisms explain coupled behavior of this kind. The first, backward constraint propagation (section 3.5), has recurred at every stage handoff since it was defined. Multi-scale feedback organizes the different response times within that loop, while emergent system trade-offs connect the resource and quality decisions encountered across its stages. Managing all three distinguishes proactive, constraint-driven system design from reactive post-deployment debugging.

> Checkpoint 3.3: Closing the feedback loop
>
> Apply the monitoring response to a different source of uncertainty.
>
> During monitoring, a clinic reports a changed referral rate for patients over seventy. Latency and image-quality rejection rates remain stable. Labels are available only for referred patients, and

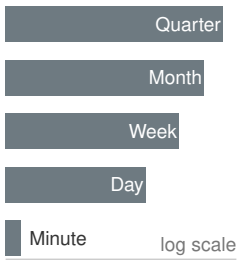
the release manifest contains no separate sensitivity evaluation for this age group. The intended-use requirements include these patients, but the evaluation plan omitted the subgroup.

- ☐ **Evidence and immediate response**: Can you distinguish what the observed referral change establishes from what remains unknown, and identify the owner and approved exposure policy needed while representative labels are collected?
- ☐ **Dependency trace**: Can you identify which requirements, dataset coverage records, and evaluation artifacts need review, and explain what evidence would justify changing preprocessing or retraining rather than assuming both are necessary?
- ☐ **Re-release**: If representative labels confirm a sensitivity failure and a revised candidate is proposed, can you specify the subgroup, regression, and on-device evidence its release manifest must record before rollout?

3.13.1 Multi-scale feedback

ML systems succeed by orchestrating feedback loops across multiple timescales, each serving a different purpose. One illustrative operating plan for the DR scenario uses minute-level checks to catch a misconfigured camera before it produces a shift's worth of unusable images; daily reviews for proxy shifts such as referral-rate, confidence, rejection-rate, or site-specific camera changes; weekly aggregation of labeled accuracy statistics and drift tests when ground truth is available; monthly analysis of population coverage; and quarterly review of whether the architecture still meets clinical needs. Actual cadences depend on label delay, product risk, traffic, and the cost of acting on each signal.

The temporal structure of these feedback loops reflects the inherent dynamics of ML systems. Rapid loops enable quick correction of operational issues: a clinic's misconfigured camera can be detected and corrected within minutes. Slower loops enable strategic adaptation; recognizing that population demographic shifts require expanded training data takes months of monitoring to detect reliably. This multi-scale approach balances responsiveness against statistical stability. Overreacting to short-term variance causes pipeline thrashing, such as triggering unnecessary retraining on transient shifts, while sluggish adaptation allows silent accuracy decay to persist unmitigated.



Monitoring loops span minutes to quarters, five orders of magnitude.

3.13.2 Emergent complexity and resource trade-offs

Complex systems produce **emergent behaviors** invisible when analyzing individual components. In a multisite DR deployment, individual clinics can exhibit stable aggregate performance while system-wide analysis detects degradation concentrated in specific demographic cohorts that no individual clinic has enough observations to characterize reliably. Network timeouts and out-of-memory crashes expose some failures directly, but neither conventional nor ML systems can rely on exceptions to reveal every harmful outcome. ML monitoring must also investigate data drift, concept drift, and bias amplification. Predictive quality can decline without generating stack traces or runtime exceptions, even while the model continues serving within its latency SLA.

Resource optimization introduces multi-dimensional trade-offs that also arise in conventional software, while ML adds learned statistical behavior to them. An accuracy improvement might require increasing model parameter count, forcing deployment onto more capable hardware; multiplied across hundreds of clinics, that marginal accuracy increment dictates cluster capital expenditure. These trade-offs are the physical constraints of deployment at work: edge deployment eliminates WAN latency but bounds model complexity by local SRAM and DRAM capacity and the device's power envelope; cloud deployment relaxes accelerator memory bounds but introduces network latency that can violate point-of-care clinical workflows. Every architectural choice reallocates pressure across the D·A·M triad.

Together, constraint propagation across stage boundaries, multi-scale feedback timing, and emergent system trade-offs show that ML systems cannot be engineered through modular decomposition alone. Optimizing data pipelines, model topologies, and execution targets without checking their dependencies can introduce runtime bottlenecks or silent failures. When engineering teams apply conventional software assumptions to these coupled dynamics, predictable antipatterns emerge

wherever practitioners treat statistical loops as static pipelines or separate deployment constraints from model design.

3.14 Fallacies and Pitfalls

Applying conventional software patterns to machine learning workflows overlooks the statistical and data-dependent nature of learned systems. The following fallacies and pitfalls capture recurring errors that stall development cycles, cause silent production outages, and compound technical debt across pipeline stages.

Fallacy: *ML development can follow traditional software workflows without modification.*

Software practices such as version control, unit testing, and iterative delivery remain necessary. Machine learning systems also derive behavior from data, so those practices must cover datasets, learned artifacts, and statistical performance evidence (table 3.1). In an ML workflow, problem definitions evolve as exploratory training reveals distribution gaps, class imbalances, or infeasible accuracy-latency trade-offs. Practitioner surveys also report that data preparation is the activity most data scientists name as their largest time sink (section 3.3). Treating data collection as a completed precursor rather than an active feedback loop traps teams in costly cycles when downstream evaluation forces fundamental revisions to early data assumptions.

Pitfall: *Treating data preparation as a one-time preprocessing step.*

Teams often treat data cleaning and feature engineering as a static milestone to clear before training begins. In production, incoming data distributions evolve continuously as user behavior shifts, sensors degrade, and upstream schemas change. The dual-tier architecture diagrammed in figure 3.1 reflects this operational reality: an upper data pipeline and a lower model pipeline must execute concurrently with continuous feedback between them. Early data preparation decisions cascade directly through training and deployment (section 3.8). When data preparation is static, silent distribution shifts and training-serving feature skew bypass code-level tests, degrading inference accuracy long before raising an operational alert. Continuous data validation pipelines with automated schema checking and distribution monitoring are necessary to catch data drift at ingestion rather than after silent model failure in production.

Fallacy: *Passing model evaluation means the system is ready for deployment.*

Model-only offline evaluation measures statistical loss on curated validation sets while leaving operational serving dynamics untested. Strong aggregate metrics provide no guarantee that the system will satisfy physical deployment constraints. Across the dual pipelines mapped in figure 3.1, an accuracy-only evaluation tests the candidate model without establishing serving integration, real-time input delivery, or operational monitoring. The diabetic retinopathy case study (section 3.2) exemplifies this disconnect: a model that achieved specialist-level accuracy on curated benchmark images struggled in clinic deployments due to variable camera lighting, technician workflow interruptions, and intermittent network connectivity. Production readiness demands system-level verification, including tail-latency budgets under concurrency, feature pipeline consistency, and graceful degradation under hardware faults. Under the constraint propagation principle (section 3.5), discovering operational failures only after deployment forces costly iterations across both model architecture and data collection.

Pitfall: *Scaling data collection before checking marginal model value.*

Expanding dataset size is often assumed to be a universally profitable investment for boosting accuracy. Additional examples incur storage, ingestion, and labeling costs, but their learning value depends on which gaps they fill. Once a region of the input distribution is well covered, returns there diminish, and more examples of already well-covered cases may contribute little, while targeted collection can expose a failure that aggregate accuracy conceals. The upstream feedback loops in figure 3.1 route data fixes from curation back to collection before samples enter training, allowing observed errors and coverage gaps to guide acquisition. Because data quality decisions cascade through all downstream pipeline stages (section 3.8), teams must measure the marginal gain in the required task and subgroup metrics against the cost of collecting and processing additional data.

Fallacy: *Skipping validation stages accelerates delivery.*

Compressing or bypassing justified validation stages transfers unresolved risk to production traffic. Multi-stage validation (section 3.10) supplies complementary evidence across increasingly realistic execution environments. Shadow evaluation can expose failures under live input distributions, such as payload deserialization bottlenecks, concurrency contention, and tail-latency violations, without letting the candidate determine user outcomes; a canary limits the users exposed to edge-case regressions or numerical instability when the candidate begins doing so. Their value depends on which uncertainty each resolves, and neither substitutes for the other or for independent offline evaluation. A release plan should state the evidence required, the exposure permitted, and the conditions for stopping or reversing a rollout. Removing a gate is justified only when its evidence obligation is met another way, not merely because delivery appears faster.

Pitfall: *Deferring deployment paradigm selection until after model development.*

Teams often assume deployment targets can be selected after optimizing model accuracy. In practice, the deployment paradigm (Cloud, Edge, Mobile, or TinyML) is a primary constraint that shapes every preceding lifecycle stage (table 3.3). Suppose a team develops a 2 GB ensemble model and learns only at deployment (stage 5) that its TinyML target has 256 KB of memory. The resulting mismatch cascades backward, forcing teams to redesign data preprocessing, model architecture, and evaluation benchmarks. Deferring platform selection invites expensive iteration cycles, because deployment hardware dictates which models are physically realizable, not merely where they execute.

3.15 Summary

The day-153 failure that opened this chapter was a workflow failure, and the six stages traced since show why it was predictable. Those six project responsibilities coordinate the finer-grained data and model pipelines. Within them, the data pipeline transforms raw inputs through collection, ingestion, analysis, labeling, validation, and preparation into ML-ready datasets, and the model development pipeline takes these datasets through training, evaluation, validation, and deployment to create production systems. The closed-loop return paths in figure 3.1, which link deployment telemetry back to collection and curation, carry the chapter's central thesis: production observations continuously reshape upstream pipelines rather than terminating at a deployment milestone.

Understanding this framework explains why machine learning systems require specialized additions to established software-engineering practices. ML workflows add probabilistic optimization, learned statistical behavior, and data-dependent feedback loops. The iron law supplies one quantitative lens: decisions across the lifecycle jointly change data movement, operation count, hardware efficiency, and fixed latency in $(T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}})$, while production evidence feeds constraint violations back into review. This perspective recognizes that success emerges not from perfecting individual stages in isolation, but from understanding how data quality affects model performance, how deployment constraints shape training strategies, and how production insights inform each subsequent development iteration.

Three patterns define the engineering discipline of the ML lifecycle. Many practitioners report data collection, cleaning, labeling, validation, or preparation as major time sinks, while model development is only one part of the lifecycle. Production-ready systems also require repeated iteration cycles across data, model, and infrastructure stages, so investment in data engineering can have high leverage when data quality is a major source of rework. Finally, late constraints can expand rework because data, weights, validation evidence, and deployment packages depend on earlier decisions. Estimating a project's correction cost requires tracing which artifacts need revision and which dependent work must be repeated. Early constraint discovery does not require freezing the workflow. It establishes an operating envelope through latency, memory, power, safety, and validation obligations while data and model choices continue to iterate within those bounds.

Key Takeaways: See the whole map first

- **The lifecycle is a loop, not a checklist**: Data and model pipelines advance in parallel, but production feedback is what makes them a system. Monitoring, validation, and retraining send lessons from deployment back into collection, labeling, architecture, and infrastructure decisions.
- **Late constraints can increase rework**: A deployment limit discovered late may invalidate preprocessing, weights, validation evidence, and integration work. Trace those dependencies and estimate the required tasks to bound the correction effort and delay.
- **Iteration velocity expands search opportunity**: Under the worked assumptions, a lightweight model starting 5 percentage points behind ends near the same ceiling as the large model while testing about four times as many ideas. Workflow speed buys search breadth rather than guaranteed model quality.
- **Interfaces make feedback actionable**: Stage contracts define inputs, outputs, and quality invariants so that data, model, and deployment teams can detect violations before integration. Without explicit contracts, each stage can optimize locally while the system fails globally.
- **Production speaks on different clocks**: Real-time inference monitoring, batch update reviews, and quarterly architectural reviews answer different failure modes. Treating all feedback as one loop either reacts too slowly to drift or churns expensive workflows without signal.
- **Workflow carries constraints through time**: Problem definition records targets, data collection shapes evidence and byte volume, model development changes operations and reuse, deployment realizes the complete serving path, and monitoring sends violations back into review. The lifecycle is data-algorithm-machine coupling unfolding over time.

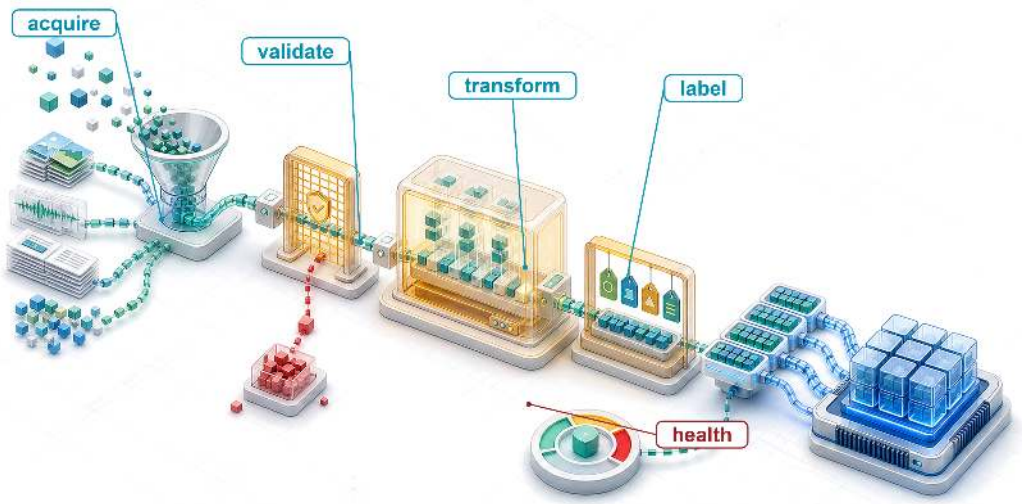
Seen as a checklist, the lifecycle is a sequence of stages to clear in order. Seen correctly, it is a loop. A constraint discovered at deployment does not stay there; it may force rework in dependent model and data decisions. A workflow is therefore a coupled system rather than a pipeline, the same coupling the D·A·M taxonomy describes in *space*, now unfolding in *time*. Optimizing one stage in isolation moves the failure instead of removing it. Systematic workflow management is risk mitigation rather than bureaucratic overhead, and the discipline is to see the whole map before touching any single part of it.

What's Next: From blueprint to data

The workflow framework is the blueprint for every chapter that follows. The first material that blueprint calls for is data, and figure 3.3 showed that data work, not modeling, is where practitioners report spending the most time. Chapter 4 therefore develops the first D·A·M axis, Data, giving datasets the review, testing, and versioning that source code receives and turning the data collection and preparation stage traced here into tested pipelines. Each later chapter assumes familiarity with where its techniques fit within this workflow.

4

Data Engineering

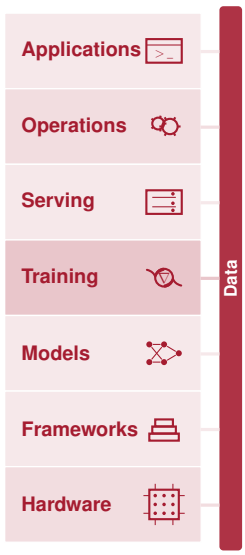


4.1 Dataset Compilation
4.2 Physics of Data
4.3 Four Pillars Framework
4.4 Data Acquisition
4.5 Data Pipeline Architecture
4.6 Systematic Data Processing
4.7 Data Labeling
4.8 Storage Architecture
4.9 Operational Data Health
4.10 Fallacies and Pitfalls
4.11 Summary

Purpose

Why does training data function like source code in a machine learning system?

In conventional software, programmers write logic that computers execute. In machine learning, programmers write optimization procedures that extract operational logic from data. This inversion makes training data resemble source code: changing it can change the learned system when the model is retrained, even if no traditional code changes. Yet data rarely receives the review, testing, and version control that even a one-line code change routinely receives. Subtle labeling inconsistencies can induce behavioral inconsistencies; missing edge cases can leave corresponding failures unmeasured; historical biases can propagate into learned behavior. A model trained only on that dataset cannot infer task-relevant evidence absent from it, and systematic label errors can shape what it learns. Unlike traditional source code, which sits inert until a programmer modifies it, the operating distribution can shift as the world changes, potentially changing production performance even when nothing in the codebase has been touched. Data engineering can consume substantial project effort, often more than modeling itself, because the work is consequential. A labeling error caught at collection costs a relabel; the same error found after deployment can cost a retraining run, an incident investigation, and the trust of the people the system failed. Every decision in the data pipeline (what to collect, how to label, when to filter, how to split) propagates forward to constrain model architecture, training dynamics, and deployment viability. Data engineering is therefore not preprocessing but programming in a different language, one where quality control, versioning, and monitoring determine whether the compiled system works today and continues working tomorrow. In D·A·M terms, the pipeline itself is a data-machine co-design problem: however well curated the data, the algorithm can only learn as fast as the machine can deliver it.



Learning Objectives

- Explain data as source code and trace how data cascades propagate through ML systems
- Calculate data gravity, feeding-tax, and storage-bandwidth costs for moving or serving datasets
- Evaluate acquisition strategies against coverage, quality, labeling cost, governance, and deployment constraints
- Design ingestion and validation pipelines for batch, streaming, ETL, and ELT workloads
- Specify idempotent transformations, lineage, and drift checks to preserve training-serving consistency
- Select labeling, storage, file format, versioning, and feature-store designs for ML lifecycle needs
- Diagnose data debt and production pipeline failures using quality, reliability, scalability, and governance evidence

4.1 Dataset Compilation

THE practitioner survey charted in the time-allocation breakdown of chapter 3 (figure 3.3) found that cleaning and organizing data, not modeling, was the most common answer to where data scientists spend their time, and that collecting datasets came second (CrowdFlower 2016). That chapter's lifecycle stages explain why. Raw inputs pass through collection, ingestion, analysis, labeling, validation, and preparation before they become ML-ready datasets, and each stage can introduce a defect that no later stage detects. That much effort, spent on stages that fail silently, is the case for treating the data pipeline as a system in its own right. The data axis of the D·A·M taxonomy becomes real only as infrastructure: acquisition systems, validation checks, labeling workflows, storage layouts, and governance.

Definition 4.1: Data engineering

Data engineering is the infrastructure layer that manages the lifecycle of data from source to model, encompassing acquisition, transformation, storage, and governance.

1. **Significance**: Its critical function is ensuring training-serving consistency, preventing silent degradation by decoupling the model from the volatility of raw data. Within the iron law, it governs bytes moved (D_{vol}), while dataset composition and quality determine whether the dataset (D) remains representative of the target distribution.
2. **Distinction**: Unlike data science, which focuses on inference and insight, data engineering addresses the scalability and reliability of the data pipeline.
3. **Common pitfall**: A frequent misconception reduces data engineering to mere "data cleaning." In systems terms, it is dataset compilation: transforming raw, noisy observations into an optimized binary layout that accelerator runtimes consume.

↩ The iron law of ML systems, whose data-movement term this chapter governs, is established in section 1.7.

The **dataset compilation** analogy makes the infrastructure concrete. A software compiler carries source code through a series of increasingly refined representations (tokens, abstract syntax trees, intermediate representations, machine code); a data pipeline carries raw, noisy observations through analogous stages that validate, select, and represent the evidence until it becomes training-ready numeric tensors. Filtering corrupted records and task-irrelevant features corresponds to *dead code elimination*, except that an unusual record may be a valuable rare case rather than dead code, so removal requires a task-specific quality or coverage judgment. Augmentation, which synthetically expands limited examples by rotating images, pitch-shifting audio, or injecting noise, has no exact compiler counterpart. Its nearest relative is *template instantiation*, which stamps out many concrete variants from one generic source; augmentation stamps out many variants of one observation, exposing the model to more versions of the underlying pattern without collecting new data, provided

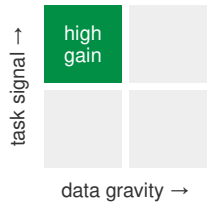
each variant preserves the task-relevant label. **Deduplication** plays the role of *common subexpression elimination*, identifying and merging duplicate records that would otherwise bias gradient estimates and waste compute. Schema validation, enforcing strict types and ranges on every record, is the data pipeline's *type checker*, rejecting malformed inputs before they crash the "runtime" of model training.

The analogy puts the data-as-code principle (1) to work. If the pipeline is the build and retraining is the recompilation step, datasets need immutable lineage, schema validation suites, and debugging when regressions appear. Compilation also forces a trust boundary between dataset splits. The training set is allowed to shape parameters; the validation set is allowed to shape modeling and pipeline choices; the test set is reserved for estimating generalization after those choices have been made. Leakage occurs when information crosses those boundaries: duplicate examples appearing in multiple splits, augmented variants of the same source record landing on both sides of the split, user records from the same household appearing in both training and test, or time-derived features computed using future observations. Preprocessing can leak too, a case section 4.6.1 takes up once normalization and encoding have been introduced.

This chapter's running case is keyword spotting (KWS), the always-on wake-word detector in a smart speaker or phone, which takes over from the retinopathy screening case because its data problems span both the cloud training pipeline and a microcontroller-class device. In KWS, speaker-independent and time-aware splits enforce the split boundary physically: assigning each speaker's utterances to a single split prevents the acoustic model from memorizing speaker-specific vocal-tract resonances, ensuring evaluation reflects generalization to the target deployment population.

Like a traditional software compiler, a dataset compiler operates across discrete stages: acquisition, ingestion, processing, labeling, storage, and lifecycle maintenance. Yet while traditional source code occupies negligible disk footprint and compiles in milliseconds, training data carries physical mass. Pipeline efficiency is governed by the physical tension between movement cost and information density: *data gravity* imposes bandwidth, I/O, and latency taxes when moving large byte volumes, while *task-relevant signal* determines the supervisory value the model extracts.

As this trade-off illustrates, engineering returns peak when early filtering and selection isolate high-signal records before incurring the physical transfer penalties of data gravity. Low-signal data—repetitive audio frames, background silence, or duplicate samples—saturates host-to-device PCIe bandwidth and pollutes memory hierarchies while yielding minimal gradient progress. Efficient dataset compilation resolves this physical bottleneck by maximizing the task-relevant signal delivered per byte moved across the storage and interconnect hierarchy, an optimization that begins with the physical properties of the data itself.



Returns peak where task signal is high and movement cost (gravity) is low.

4.2 Physics of Data

Moving a petabyte of training data across cloud regions incurs tens of thousands of dollars in egress fees and hours of network serialization delays. The data-as-code principle captures what data does (it determines system behavior) but not why moving it incurs such steep physical penalties. The physics of data treats data as a physical substance with measurable properties: byte volume and available bandwidth constrain movement, while the learning value of the transferred examples must be established for the task.

4.2.1 Data gravity

Data movement incurs physical and financial friction, the cost that the data-gravity principle (2) names. **Data gravity** scales directly with data volume (D_{vol}) and inversely with network bandwidth (BW). Transferring a petabyte dataset across a 10 Gbps link requires days of continuous transmission ($T = D_{\text{vol}}/\text{BW} \approx 9.3$ days); even across a dedicated 100 Gbps link, transfer latency and egress tariffs remain substantial enough to govern system design. Because migrating 1 PB across network boundaries is both slow and expensive, system designers colocate compute with the storage substrate rather than moving large datasets across remote networks. This physical constraint underlies lakehouse storage architectures[1] (Zaharia et al. 2021), where query and preprocessing engines execute directly against shared object storage rather than repeatedly copying raw data across network fabrics.

[1] **Data lakehouse**: Combines data lake storage (cheap, schema-less) with warehouse query semantics (ACID transactions, schema enforcement) using transactional table layers such as Delta Lake. For ML workloads, the lakehouse reduces the extract, transform, load (ETL) copy between lake and warehouse, enabling direct feature computation on the storage layer where data already resides. This is a direct response to data gravity, since repeated petabyte-scale copies increase the D_{vol}/BW cost (Armbrust et al. 2020; Zaharia et al. 2021).

Napkin Math 4.1: The physics of data gravity

Problem: A 1 PB training dataset resides in a US East data center, while a remote accelerator pod is available in US West. What transfer time and egress cost does remote training add, and when would suitable compute near the data avoid that overhead?

Math:

1. **Network bandwidth**: 100 Gb/s $\div 8 = 12.5$ GB/s.
2. **Transfer time**: 1,000,000 GB $\div$ 12.5 GB/s = 80,000 seconds, approximately 22 h.
3. **Cost**: 1,000,000 GB $\times$ \$0.02/GB = \$20,000. (Baseline: AWS data transfer out pricing, 2024.)

Systems insight: If training takes less than 22 h, data transfer takes longer than training. If training costs less than \$20,000 (approximately 5,000 TPUv4-hours), bandwidth costs more than compute. When suitable compute is available near the data, these transfer costs favor moving code to data. Smaller datasets make moving data to code more practical, but placement still depends on compute availability and comparative training time.

4.2.2 Task-relevant signal

Not all bits deliver equal utility per unit of movement cost. While physical storage measures bytes, machine learning optimization extracts **task-relevant signal**, evidence that helps the model learn the target task rather than raw Shannon entropy. A dataset containing one million identical images incurs substantial data gravity without adding distinct observations beyond the first sample. Repeated optimization on that sample can still change gradients as the model changes. Conversely, a curated collection of 10,000 diverse edge cases carries negligible data gravity yet contributes rich supervisory signal. Comparing task-relevant signal with data gravity motivates a selection heuristic:

$$\text{Data Selection Gain} \propto \frac{\text{Task-Relevant Signal}}{\text{Data Gravity}} \tag{4.1}$$

The numerator is not a universal physical quantity. A team estimates marginal learning value by comparing held-out model quality under a fixed training procedure; the denominator accounts for the movement cost of the compared datasets. The heuristic favors a selection only when its measured learning benefit justifies that cost. Scoring, labeling, and training costs must also enter a complete budget. Deduplication and active learning (which asks the current model to choose the examples it is least certain about for labeling, section 4.7.3) can improve this ratio by eliminating redundant byte volume and proposing uncertain examples for labeling. Uncertainty does not establish learning value; ambiguous or corrupted examples can also be uncertain, so the selection policy requires validation.

↪ The information-compute ratio, which measures the learning signal each sample returns per unit of training compute, is developed in section 9.4.

4.2.3 The feeding problem: Flow rate and the "feeding tax"

Data gravity dictates the cost of moving data in bulk; **the feeding problem** dictates the cost of delivering it to execution units in real time. It is an input/output **flow rate** bottleneck: storage bandwidth fails to keep pace with compute throughput.

When data delivery and computation overlap, the bottleneck principle holds that a system runs only as fast as its slowest stage. The scenario this chapter carries through its pipeline and storage sections makes the constraint concrete: one reference accelerator training ResNet-50. Dividing the accelerator's peak arithmetic rate by the work of one training step per image (a forward pass plus a backward pass costing about twice as much) gives an arithmetic ceiling of about 12,682 img/s; Notebook 4.9 at the end of the storage discussion works the numbers in full. In practice a well-tuned pipeline sustains about 3,000 img/s, roughly 24 percent of that ceiling, once memory-bound layers, decode, augmentation, and synchronization take their share. Each image sits on disk as a 150 KB JPEG, so storage must deliver 450 MB/s. When the attached volume supplies only 250 MB/s, compute units stall on I/O. This gap defines **the feeding tax**: the throughput penalty from insufficient input supply, which directly degrades system efficiency (η_{hw}). Under steady-state overlap, divide the supplied 250 MB/s by the required 450 MB/s to obtain the fraction of the sustained training rate

that storage can support. Subtracting that fraction from one gives a modeled feeding tax of about 44 percent. This is the throughput loss relative to the supplied sustained baseline, not a measurement of all accelerator idle time or utilization relative to advertised peak.

Which bytes are counted matters as much as how many. After decoding, the same image becomes a 602 KB FP32 tensor in host memory, about 4 times its compressed size, so the same training rate moves 1.8 GB/s through host memory and across the host-to-accelerator link. Compression thus trades storage bandwidth for CPU decode work. A pipeline that caches pre-decoded tensors on disk to skip that work pays the full 1.8 GB/s at the storage tier instead, and applying the same supply-to-demand calculation gives a modeled throughput loss of 86 percent relative to the sustained baseline. Later sections apply this slowest-stage bound to the CPU decode workers (section 4.5.1) and to the storage tiers (section 4.8.1) without re-deriving it. Distance also increases energy consumption: as data travels farther from arithmetic logic units, transport energy dwarfs the cost of computation itself.

↩ The bottleneck principle is established in section 2.3.1.

Systems Perspective 4.1: The energy cost of data movement

The memory-wall analysis in section 2.2 implies a data-engineering principle: *moving one 32-bit value costs roughly 173× to 10,811× as much energy as the table's FP32 multiply*, with the multiplier rising as the data leaves the chip. The DRAM ratio differs from the one in the iron-law energy discussion of section 1.7 because the normalization differs: that ratio compares one byte moved with one FP16 multiply, while this table compares a full 32-bit value with a more expensive FP32 multiply. Both describe the same cost gradient. Later optimization chapters exploit the same cost gradient inside model and hardware design; here, the question is how much energy the information flow from the external world already consumes before the model computes. Table 4.1 makes that cost gradient explicit across the memory hierarchy.

Table 4.1: Energy Cost of Moving vs. Computing on a 32-Bit Value: Approximate energy per event at each stage of the data-movement hierarchy, with multipliers normalized to one 32-bit FP32 multiply. All rows use the same 32-bit width, so the ratios compare like-sized events. They do not by themselves prove which term dominates a complete workload; total energy also depends on the number of arithmetic operations and transfers.

Operation	Energy (pJ)	Relative Cost
32-bit FP Multiply	3.7 pJ/FLOP	1×
DRAM Memory Access (32-bit)	640 pJ	173×
Local SSD Access (32-bit)	4,000 pJ	1,081×
Network Transfer (32-bit)	40,000 pJ	10,811×

The cost gradient quantified in table 4.1 demonstrates why data locality dominates systems optimization. Each avoided transfer eliminates its corresponding movement energy, compounding across training epochs.

Because data movement incurs physical energy costs, data volume acts like physical mass. Removing 50 percent of training data through deduplication does more than save persistent storage capacity: it eliminates energy expenditure across network, memory, and preprocessing stages. Filtering uninformative samples early is among the most effective architectural levers when I/O and communication dominate workload runtime.

The energy hierarchy and the signal-density heuristic converge on the same optimization lever: effective data engineering maximizes the Data Selection Gain defined in equation 4.1. Data curation functions as **signal-to-noise engineering**. Deduplication strips away redundant byte volume while preserving task-relevant signal, directly amplifying the selection gain. Active learning proposes examples whose labels may add learning value; validation determines whether that policy improves information density per byte transferred. Filtering uninformative records conserves interconnect bandwidth and storage energy, preventing upstream I/O starvation from idling accelerator execution units.

Treating data as a physical substance grounds data pipeline design in hardware realities: bandwidth limits determine flow rates, memory hierarchies dictate energy dissipation, and dataset scale governs compute placement.

Checkpoint 4.1: The physics of data

Data engineering is governed by physical costs. Check your intuition:

- ☐ **Data gravity**: why does a petabyte-scale dataset pull compute toward the data, and which assumption in the petabyte example (link speed, egress price, training duration) would have to change to reverse that conclusion?
- ☐ **Energy cost of data movement**: why does fetching a 32-bit value from DRAM, an SSD, or the network cost more energy than one 32-bit multiply, and what else (how many operations, how many transfers) must you know before saying which dominates a workload's total energy?
- ☐ **Task-relevant signal**: why can a smaller, diverse dataset be more valuable than a massive, redundant one?

Physical constraints define the operational envelope of ML data systems, but physical awareness alone does not guarantee reliability. A system that accounts for data gravity can still fail if schema validation is neglected, data corruptions go undetected, or ingestion pipelines crash silently under load. Turning these physical realities into robust production infrastructure requires a systematic architectural framework that organizes engineering decisions across ingestion, transformation, and storage.

4.3 Four Pillars Framework

A credit scoring model starts rejecting every applicant from one region after an upstream database migration silently changes how ZIP codes are typed. A medical imaging model degrades silently for months because camera hardware changed at a partner hospital. A fraud detection system misses a new attack vector because its training data was six months stale. Each failure traces to a different root cause (schema drift, distribution shift, data staleness), yet all share a common pattern: ad hoc data engineering decisions that interacted in ways no one anticipated until deployment. These cascading failures motivate the **four pillars framework**, organized around quality, reliability, scalability, and governance.

4.3.1 Data cascades

Machine learning systems face a distinctive failure pattern called data cascades, where poor data quality in early stages amplifies throughout the entire pipeline (Sambasivan et al. 2021). Some invalid inputs trigger immediate software errors, but data can also pass schemas and tests while degrading learned behavior silently[2] until quality issues become severe enough to require expensive investigation, rework, or retraining.

Data errors rarely stay confined to their point of origin; they compound across pipeline stages. In the lifecycle progression sequenced in figure 4.1, early defects in problem framing and collection bypass intermediate validation, casting failure arcs directly into model evaluation and deployment that force teams into costly pipeline restarts.

Definition 4.2: Data cascade

Data cascade is an ML systems failure mode in which upstream data quality problems propagate through collection, labeling, feature engineering, training, evaluation, and deployment, amplifying into downstream model failures or user harm.

1. **Significance**: The remediation cost scales with the number of downstream artifacts that consumed the corrupted data: feature tables must be regenerated, models retrained, evaluation metrics recomputed, and deployed systems rolled back or patched. A sin-

[2] **Data cascades**: The failure is "silent" because it degrades model *inputs*, not model *code*; corrupted data can pass unit tests and appear healthy in ordinary system monitoring. Sambasivan et al. (2021) describe cascades as often invisible and delayed: flawed data practices may surface only after downstream evaluation, deployment, or user-facing failures reveal that the model learned from the wrong signal. Remediation then requires tracing the issue back through data collection, labeling, feature engineering, and evaluation decisions, with some teams restarting or abandoning affected work.

↩ The compounding cost of late-stage data rework builds directly on the constraint propagation principle established in section 3.5.

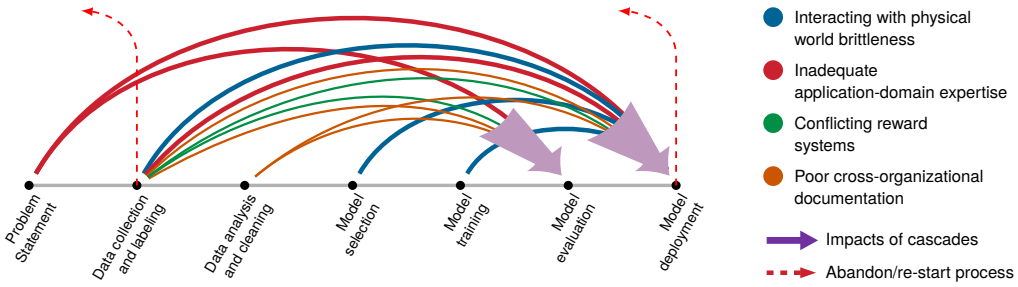


Figure 4.1: Data Quality Cascades: Upstream defects along the seven-stage workflow timeline cast failure arcs directly into late-stage evaluation and deployment. Early flaws in problem framing and collection compound through downstream processing, forcing costly project restarts (dashed red return paths) when defects finally surface in production. Source: (Sambasivan et al. 2021).

gle schema or labeling defect can therefore invalidate an entire training run and all experiments derived from it, turning a local data error into a full-pipeline rework event.

2. **Distinction**: Unlike an isolated data quality bug, a data cascade is defined by propagation and amplification. The initial defect may be small, but each pipeline stage treats its input as trustworthy and converts the defect into new derived state, making the root cause harder to observe as the system moves farther from collection.
3. **Common pitfall**: A frequent misconception is that data cascades are caught by ordinary software tests. In reality, corrupted data can satisfy schemas, pass unit tests, and produce successful training jobs while teaching the model the wrong signal; preventing cascades requires lineage, validation, contracts, and monitoring tied to model behavior.

This feedback loop creates an insidious operational failure mode: silent degradation. Unlike conventional software crashes that trigger stack traces and alert pages, data-induced model errors produce syntactically valid outputs with degraded semantic accuracy. By the time downstream metrics reflect the failure, corrupted predictions have already contaminated historical data stores, forcing expensive rollbacks and data re-ingestion.

Example 4.1: The pipeline jungle

Scenario: The credit scoring model from the opening of this section suddenly begins rejecting all applicants from a specific geographic region.

Diagnosis: The training pipeline represented `zip_code` as a fixed-width string (“02139”). An upstream database migration changed the serving field to an integer without updating downstream data contracts. A downstream transform serialized that integer as “2139,” losing the leading zero and causing the model to treat a familiar postal code as an unobserved category. In this scenario, the fallback for unknown categories assigns high risk, explaining the regional rejection pattern.

Systems lesson: Unenforced feature boundaries create pipeline jungles where upstream schema modifications cause silent downstream model failures. Versioned data contracts, machine-enforceable specifications declaring expected data types, valid value bounds, and nullability invariants, prevent cross-system data corruption by rejecting malformed records at the ingestion boundary before they reach the model.

4.3.2 Four foundational pillars

The ZIP-code failure crossed several concerns at once: the values were wrong (quality), the transform broke silently instead of rejecting the record (reliability), and no contract recorded what the field meant (governance). Preventing failures like it means balancing four competing operational priorities,

each of which feeds the ML data system and each of which pulls against its neighbors (figure 4.2): stricter validation costs throughput, stronger consistency limits distributed scale, privacy constraints cost performance, and bias mitigation can shrink the usable training set.

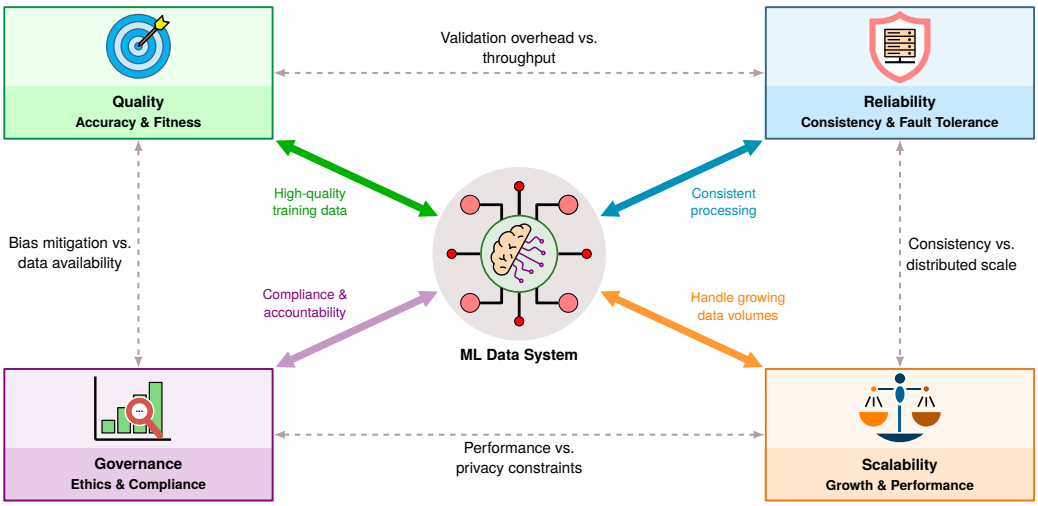


Figure 4.2: The Four Pillars of Data Engineering: Quality, Reliability, Scalability, and Governance form the foundational framework for ML data systems. Each pillar contributes essential capabilities (solid arrows), while trade-offs between pillars (dashed lines) require careful balancing: validation overhead affects throughput, consistency constraints limit distributed scale, privacy requirements impact performance, and bias mitigation may reduce available training data.

These four operational foundations establish the trade-off space for data pipeline architecture. Prioritizing scalability without quality validation or reliability contracts can increase throughput while allowing bad or inconsistent data to propagate. Conversely, exhaustive governance controls can add storage and computation overhead that constrains training iteration velocity.

Quality governs input integrity and fitness for task: data must accurately reflect the operating distribution, with correct ground-truth labels and uncorrupted features. In an edge voice-enrollment pipeline, for example, quality rules filter out clipped audio waveforms, low signal-to-noise ratios, and background acoustic interference so the model learns sharp decision boundaries rather than spurious acoustic artifacts.

Reliability asks whether that enrollment pipeline keeps working when the device, network, or user behavior is imperfect. A pipeline that produces excellent wake-word examples in a laboratory but fails during intermittent connectivity, battery pressure, or microphone glitches delivers no value in the user's home. Error handling, retries, local buffering, and graceful degradation turn the quality rule into a usable system: the device can request another utterance, defer upload until connectivity returns, or preserve a known-good enrollment rather than silently accepting corrupted audio.

Scalability asks whether that same decision survives growth. A manual review policy that works for a thousand recordings collapses when the product expands to millions of users, dozens of languages, and long-tail acoustic conditions. The system must scale validation, storage, labeling, and retraining without letting infrastructure cost grow faster than the value of better coverage. The limiting resource depends on the workload: for edge speech, storage and labeling throughput dominate, whereas for recommendation systems, memory capacity forms the binding constraint. Lighthouse 4.1 shows where the scalability pillar bites hardest: embedding tables outgrow single-node memory capacity, making distributed partitioning a first-class design concern rather than an afterthought.

Governance defines the boundaries within which quality, reliability, and scalability must operate. For keyword spotting, governance determines whether raw voice recordings may leave the device, how long enrollment audio can be retained, which consent record authorizes use, and what documentation proves that the dataset covers relevant accents without exposing private speech. A perfectly scalable, reliable, high-quality pipeline that violates the General Data Protection Regulation (GDPR) or perpetuates demographic biases creates liability rather than value. Dataset documenta-

tion practices such as data statements make part of that governance visible by recording provenance, intended use, collection conditions, and coverage needed for bias analysis and scientific comparison (Bender and Friedman 2018).

When ML systems exhibit failures, the four pillars provide a diagnostic lens for identifying root causes. Gradual accuracy degradation points to quality: data drift has shifted the serving distribution away from training, or label quality has degraded as annotator pools change. Intermittent pipeline failures point to reliability: error handling, retry logic, or resource controls are missing under peak load. Training that takes too long despite adequate hardware points to scalability: a single-threaded transformation, unpartitioned shuffle, or slow storage tier prevents parallel resources from being used. Compliance gaps discovered during audits point to governance debt: lineage tracking is incomplete, access controls are stale, or retention policies have not kept pace with regulatory changes.



Lighthouse 4.1: DLRM recommendation lighthouse

Recommendation systems built in the DLRM style exemplify the scalability challenge of modern data engineering. Modern recommendation workloads bifurcate pipeline resource demands: continuous signals stream through dense matrix pipelines, while categorical IDs demand terabyte-scale distributed lookup infrastructure. Table 4.2 maps these operational stress points across data ingestion, embedding table aggregation, and feature interaction stages.

Table 4.2: DLRM Scalability Profile: Recommendation models stress data engineering along memory capacity and sparse-access bandwidth rather than compute.

Property	Value	System Implication
Data scale	High-cardinality user/item IDs	Embedding and lookup tables can outgrow one machine's memory.
Constraint	Memory Capacity	The tables no longer fit on one machine and must be partitioned.
Bottleneck	Sparse Access	Random lookups stress memory bandwidth more than compute.

At ingestion, high-cardinality categorical IDs remain compact records in a throughput-bound stream. The network- and memory-capacity bottleneck emerges later, when training or serving must fetch sparse embeddings from terabytes of partitioned table state. Mitigating this asymmetry requires partitioning and caching the learned embedding tables, which are model parameters updated during training. A *feature store*, the shared service that holds precomputed model inputs for both training and serving, manages a different kind of state; section 4.8.5 defines that role. It may hold representations computed by a separate model, but it does not replace the recommendation model's learned parameter tables.

The most insidious failures span multiple pillars. Features that differ between training and serving implicate both quality (the values are wrong) and reliability (the computation is inconsistent). A privacy-motivated deletion policy can also create quality gaps if the retained data no longer covers the deployment population. Diagnosing such cross-pillar failures requires checking consistency contracts, comparing feature distributions across environments, and tracing transformation lineage, all techniques examined in detail throughout this chapter. Production diagnosis should therefore check data infrastructure alongside model behavior.

4.3.3 KWS case study

KWS systems ground the four-pillar framework in concrete physical constraints. Embedded on battery-powered microcontrollers and mobile coprocessors, a wake-word detector listens continuously for target phrases, such as "OK Google" or "Alexa," in everyday acoustic environments, operating under strict limits on compute, memory, and energy.[3]

In the wake-word interaction sequenced in figure 4.3, an always-on detector traps "Alexa…" before the smart speaker acknowledges the utterance and opens the command channel for "dim the lights." Operating behind that brief acoustic exchange is an entire infrastructure spanning acoustic deduplication, schema contracts, prefetching, and lineage tracking.

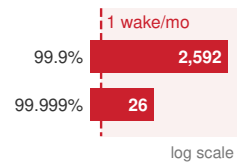
[3] **Voice Match enrollment**: Repeating "OK Google" creates a micro-scale data pipeline: quality filters noisy samples, reliability completes enrollment, scalability fits the model in always-on system-on-chip (SoC) memory, and governance controls storage, processing, and retention. Data engineering applies wherever data determines system behavior.



Figure 4.3: Keyword Spotting in Action: An Amazon Echo Dot exchange shows the user saying "Alexa," the device acknowledging the wake word, and the user issuing "dim the lights." A lightweight, always-on wake-word detector listens continuously and triggers the main voice assistant only after the keyword is spoken.

Keyword spotting grounds the four pillars in strict physical constraints. The core problem is deceptively simple: detect specific keywords amidst ambient sounds and other spoken words, with high accuracy, low latency, and minimal false activations, on devices with severely limited computational resources. A well-specified problem definition identifies the desired keywords, the envisioned application, and the deployment scenario. The objectives that follow must balance competing requirements: performance targets of 98 percent accuracy in keyword detection with latency under 200 ms, alongside resource constraints demanding minimal power consumption and model sizes optimized for available device memory.

> **Napkin Math 4.2: False positive targets**
>
> **Problem**: An always-on KWS system budgets at most one expected false wake-up per month. Under a model that counts one nonkeyword classification window each second around the clock, how strict does the per-window false positive rate need to be?
>
> **Variables**:
>
> - **Duty cycle**: Always-on (24 hours/day).
> - **Window size**: One-second classification windows.
> - **Windows per month**: One window per second, 24 hours/day, over 30 days gives 2,592,000 windows/month.
>
> **Math**:
>
> - **False positive rate (FPR)**: 1 expected false wake divided by the monthly nonkeyword-window count gives approximately 3.9×10^{-7}
> - **Nonkeyword rejection requirement**: 99.99996 percent rejection of nonkeyword windows.
>
> **Systems insight**: This calculation bounds the expected monthly count, not the maximum number of false wakes in every month. Expectation does not require independent windows. Aggregate accuracy (for example, "99 percent accuracy") is insufficient here. Evaluation must report false accepts per hour (FA/Hr) together with the corresponding false-rejection behavior.



Even 99.999 percent per-window rejection multiplies into 26 false wakes a month.

Success metrics for KWS extend beyond simple accuracy to include true positive rate (correctly identified keywords relative to all spoken keywords), false positive rate (nonkeywords incorrectly identified as keywords), and detection/error trade-off curves that compare false accepts per hour against false rejection rate on streaming audio representative of real-world deployment (Nayak et al. 2022). Of these metrics, the false positive rate dictates user viability for always-on systems. Because KWS listens continuously, every second of every day, even a microscopic per-window false-positive rate accumulates across millions of evaluation windows into unacceptable operational failure.

Operational metrics further track response time (keyword utterance to system response) and power consumption (average power used during keyword detection), and stakeholder priorities create additional tension around those metrics. Device manufacturers prioritize low power consumption, software developers emphasize ease of integration, and end users demand accuracy

and responsiveness. Balancing these competing requirements shapes system architecture decisions throughout development.

Embedded device constraints impose hard boundaries on these architectural choices. Memory limitations require extremely lightweight models, often in the tens-of-kilobytes range, to fit in the always-on island of the SoC;[4] this budget must accommodate both model weights and the feature extraction buffers and preprocessing state. Limited computational capabilities (often a few hundred MHz of clock speed) demand aggressive model optimization. Most embedded devices run on batteries, so KWS systems target sub-milliwatt power consumption during continuous listening. Devices must also function across diverse deployment scenarios ranging from quiet bedrooms to noisy industrial settings.

Data quality and diversity ultimately determine whether these constraints can be met. The dataset must capture demographic diversity (speakers with various accents, ages, and genders) to ensure broad recognition. Keyword variations require attention since people pronounce wake words differently, and background noise diversity proves essential for training models that perform across real-world scenarios from quiet environments to noisy conditions. Once a prototype system is developed, iterative feedback and refinement keep the system aligned with objectives as deployment scenarios evolve, requiring testing in real-world conditions and systematic refinement based on observed failure patterns.

4 **System-on-chip (SoC) always-on island**: Modern system-on-chip designs partition power domains so a low-power "always-on" island (typically achieving sub-milliwatt draw) monitors for wake triggers while the main processor sleeps. The critical constraint is that this island must hold both the model weights *and* the audio preprocessing code within its dedicated SRAM, a split budget that forces KWS architectures to optimize for total footprint, not just parameter count.

4.3.4 KWS design space

KWS accuracy, false-wake tolerance, latency budget, energy budget, and memory limits create a multi-dimensional design space where data engineering choices cascade through system performance. Table 4.3 quantifies key trade-offs, enabling principled decisions rather than ad hoc selection. One row uses mel-frequency cepstral coefficients (MFCCs): compact speech-frequency features whose coefficient count controls feature size, compute cost, and acoustic detail; section 4.6 shows how they are extracted.

Table 4.3: KWS Data Engineering Design Space: Illustrative scenario inputs that price each design choice along four axes, namely quality, latency, cost, and memory; the entries are not measurements from one system and should not be combined as additive effects. Each row moves from a baseline (left of the arrow) to an alternative, and every entry gives the effect of that move. Higher sampling rates double raw-audio storage and processing, while more training examples multiply labeling and storage requirements. Actual quality and latency must be measured on the target pipeline.

Design Choice	Quality Impact	Latency Impact	Cost Impact	Memory Impact
8 → 16 kHz sampling	+2–4% accuracy	2× input-processing workload	2× raw-audio storage	2× raw-audio window
13 → 40 MFCC coefficients	+3–5% accuracy	3× feature compute	Minimal	3× feature memory
1M → 10M training examples	+5–8% accuracy	10× training time	10× labeling cost	10× storage
Clean → noisy training data	+10–15% real-world	Minimal	3× collection cost	Minimal
Cloud → local inference	up to 2% accuracy risk	100 ms → 10 ms	\$0.001/query → \$0/query	cloud-scale → 64 KB
Real only → added synthetic augmentation	+3–5% robustness	Minimal	10× cheaper per example than real collection	Minimal

Operating under milliwatt power envelopes and strict tens-of-kilobytes SRAM budgets means an engineering team cannot select the most accurate option along every axis simultaneously. Turning these cataloged trade-offs into a production deployment requires solving a constrained optimization problem: allocating a finite data budget across labeling, storage, and governance to maximize model accuracy while staying within the hardware footprint. Notebook 4.3 allocates the data budget and proposes a candidate design for a smart-speaker deployment. The acquisition, processing, and storage decisions that follow supply the evidence needed to accept or reject it.

Napkin Math 4.3: Allocating the KWS data budget

Scenario: A KWS system for a smart speaker has these constraints:

- **Target**: 98 percent accuracy, an expected false-wake budget below 1 false wake/month
- **Budget**: $150K total data engineering budget
- **Memory**: 64 KB model size limit (always-on island)
- **Timeline**: 6 months to production

Step 1: Choose conservative options for the constrained device.

For this scenario, the team chooses two conservative options:

- 13 MFCC coefficients, because every MFCC frame lives in the always-on island's SRAM as model input and buffered state, so the 40-coefficient option triples a cost the device pays continuously
- Local inference to avoid network latency and preserve offline operation

Neither alternative is ruled out by the 64 KB limit alone; the deployed model, preprocessing state, and runtime need their own footprint measurements. The choice rests on where each cost lands. The table suggests a potential 3–5 percent benefit from 40 coefficients. The team instead tests whether better training coverage can recover the quality gap while preserving the smaller on-device representation. Data collection spends a central budget, whereas the larger feature representation consumes memory and compute on every device; their relative cost and achieved quality still require measurement.

Step 2: Calculate budget allocation.

Labeling is priced at a scenario rate per crowdsourced example, set above the image-classification crowd rates that section 4.4.2 later tabulates because a KWS contributor must record the utterance as well as label it. The $150K budget splits across three cost categories:

- **Labeling** (~60 percent): $90K available
- **Storage/Processing** (~25 percent): $37.5K
- **Governance/Other** (~15 percent): $22.5K

At $0.10/label with 20 percent review overhead: $90K ÷ $0.12/label = 750K labeled examples

This yields roughly 0.75M labeled examples, just below the 1M anchor in the design space. The 1M-to-10M row in table 4.3 should therefore be read as a scaling reference rather than a direct interpolation: the real-label budget alone does not buy the full data-volume gain.

Step 3: Identify candidate quality improvements.

The quality plan has three components:

- An illustrative base-model accuracy of ~90 percent
- Partial data-volume gain from 750K real examples
- A gap to the 98 percent target that the options below must close

Three options from the design space are candidates for closing the gap; their collection, storage, processing, and deployed-footprint costs must be checked against the allocations. Their listed gains cannot simply be added to the base accuracy, because they overlap (noisy data and synthetic augmentation both buy robustness to acoustic conditions), so the plan is a candidate whose combined effect must be measured on the trained system:

- 16 kHz sampling: +2–4 percent accuracy, 2× raw-audio storage (requires cost validation; on the device only the short window of raw samples feeding each MFCC computation doubles, because every frame still reduces to 13 coefficients)
- Noisy training data: +10–15 percent real-world accuracy, 3× collection cost (proposed for a targeted noisy slice; its cost must fit the storage/processing allocation)

- Synthetic augmentation: +3–5 percent robustness, 10× cheaper than real data in the illustrative row; generation, verification, and storage costs remain to be measured

Step 4: Final configuration.

The scenario combines these choices with 13 MFCCs and the computed 750K real-label budget in table 4.4. The quality effects can overlap, so the table records a candidate configuration rather than a solved optimum.

Result: The budget calculation supports approximately 750K human-labeled examples. Reaching the 98 percent quality target and fitting the 64 KB model limit remain validation requirements for the trained and deployed system.

Systems insight: Systematic design-space analysis turns "we need more data" into a testable allocation: the stated labeling assumptions buy about 750K real examples, while synthetic volume, accuracy, and deployed footprint still require measurement.

Table 4.4 records the resulting configuration, pairing each design choice with the constraint that motivates it: sampling and augmentation draw on the stated allocations, while the always-on island sets the deployed-footprint constraint. Actual costs and working-state memory still require validation.

Table 4.4: KWS Candidate Configuration: A scenario that uses the computed real-label budget and records the remaining sampling, augmentation, and deployment choices for measurement against the quality and memory targets.

Choice	Selection	Rationale
Sampling rate	16 kHz	+2–4% accuracy for 2× raw-audio storage, subject to cost validation; on device only the raw-sample window grows, since each MFCC frame keeps 13 coefficients
MFCC coefficients	13	One-third the feature memory and compute of 40 coefficients in always-on SRAM; test whether improved data coverage recovers the quality gap
Training examples	750K real + illustrative synthetic augmentation	Human-label count follows from the budget; synthetic volume is a scenario choice
Data diversity	Noisy + clean mix	Critical for real-world deployment
Inference	Local, reduced precision	Reduced precision can lower footprint; the final 64 KB fit requires the quantization analysis in section 10.4
Augmentation	Heavy synthetic	Illustrative 10× per-example cost advantage; total generation and review costs require validation

Checkpoint 4.2: Four pillars framework

The four pillars provide a systems lens for every pipeline choice.

Pillars

- ☐ **Quality**: Can you distinguish *correctness* from *coverage* (edge cases, subgroup coverage) and explain why both matter?
- ☐ **Reliability**: Can you name concrete failure modes (schema drift, missing values, skew) that cause silent degradation rather than explicit crashes?
- ☐ **Scalability**: Can you explain which parts of a pipeline become bottlenecks as data volume grows (storage bandwidth, compute, coordination), and why?
- ☐ **Governance**: Can you articulate what must be tracked to make a dataset auditable (provenance, consent constraints, access controls, documentation)?

Trade-offs

- ☐ **Pillar tension**: Given one pipeline change (for example, stricter validation, stronger privacy constraints, or more data sources), can you predict which pillar improves and which pillar(s) you stress?

A candidate configuration establishes target requirements, but meeting them requires orchestrating complementary data acquisition channels. Baseline corpora anchor the acoustic prior, targeted crowdsourcing addresses demographic coverage gaps, and synthetic generators supply volumetric scale under parameterized acoustic conditions. The four pillars provide the diagnostic and architectural criteria for evaluating these channels, while data provenance determines the physical raw material that every subsequent pipeline stage refines.

4.4 Data Acquisition

Data acquisition begins when the team names the coverage gap the model must close. The full ImageNet database[5] grew to about 14.2 million labeled images across 21,841 synsets, while the ImageNet Large Scale Visual Recognition Challenge used a 1,000-class subset (Deng et al. 2009; Russakovsky et al. 2015). GPT-3's training corpus used tens of terabytes of raw Common Crawl text filtered to hundreds of gigabytes, combined with curated web, books, and Wikipedia data (Brown et al. 2020). For keyword spotting, the Multilingual Spoken Words Corpus (MSWC) (Mazumder et al. 2021) provides a reusable reference source of 23.4 million audio samples spanning 50 languages. That corpus is larger than the product-specific labeled subset budgeted in the KWS design example; using it still requires selecting and validating relevant coverage. The model must recognize wake words across accents, microphones, rooms, ages, and background noises that no single collection method can economically cover. Acquisition strategy is therefore a sequence of gap-closing decisions: reuse what already matches deployment, collect what is missing, scrape or synthesize where scale is the binding constraint, and reject sources whose provenance or consent constraints make them unusable.

[5] **ImageNet**: The 2009 paper reported 3.2 million images across 5,247 synsets; the full database later reached about 14.2 million images across 21,841 synsets, whereas the challenge used a 1,000-class subset (Deng et al. 2009, 2024; Russakovsky et al. 2015). Its value as a benchmark is inseparable from its data engineering: Fei-Fei Li's team built labeling infrastructure that later teams could reuse. The catch is sensitivity to annotation procedures and modest distribution shifts: models tuned to ImageNet's distribution can underperform on related but shifted test sets (Recht et al. 2019; Beyer et al. 2020).

The KWS deployment profile demonstrates why data acquisition cannot optimize a single dimension in isolation. Achieving 98 percent accuracy across diverse acoustic environments requires representative coverage across regional accents, speaker age groups, and room reverbs. Maintaining consistent detection across consumer devices introduces hardware variation: microphones differ in frequency response, signal-to-noise ratio, and analog-to-digital clipping behavior. Furthermore, supporting millions of concurrent edge devices demands dataset volumes that manual curation cannot economically satisfy, while always-listening wake-word detectors operate under strict privacy boundaries governing audio retention and anonymization. A data source that maximizes volume while compromising privacy compliance, or captures pristine studio audio while ignoring field microphone distortion, fails to solve the deployment problem.

4.4.1 Data source evaluation and selection

The choice among curated datasets, expert crowdsourcing, controlled web scraping, and synthetic generation depends on which source best closes the next deployment-distribution gap at acceptable cost, quality, and governance risk. Evaluation therefore begins with the cheapest reusable source and escalates to active collection or simulation only when the remaining coverage gap justifies the marginal engineering cost.

Preexisting datasets from repositories such as Kaggle, UCI (Dua and Graff 2024), and ImageNet are the first test. They offer speed and comparability when the deployment distribution resembles the benchmark enough to make reuse meaningful. For KWS, a curated speech corpus can establish the baseline model and reveal which words, languages, and acoustic conditions are already covered. Rather than guaranteeing complete operational coverage, baseline corpora make the unaddressed distribution gaps quantifiable.

The viability of dataset reuse depends on documentation fidelity. Without detailed records of sensor calibration, environment distributions, and filtering thresholds, external data compromises experimental reproducibility (Pineau et al. 2021; Henderson et al. 2018). Thorough documentation specifies collection methodology, label taxonomy definitions, and baseline error distributions, enabling independent validation. As ingestion scales to millions of samples, dataset variety compounds latent label noise (Gudivada et al. 2017), rendering manual inspection impossible and necessitating automated validation pipelines.

Reuse carries a less visible risk. Standard validation metrics can create an illusion of model readiness when the benchmark itself is unrepresentative, and when many teams train on the same

repository, its blind spots do not average out across their models. They can recur across models, so five models trained on one dataset can share correlated failures (figure 4.4).

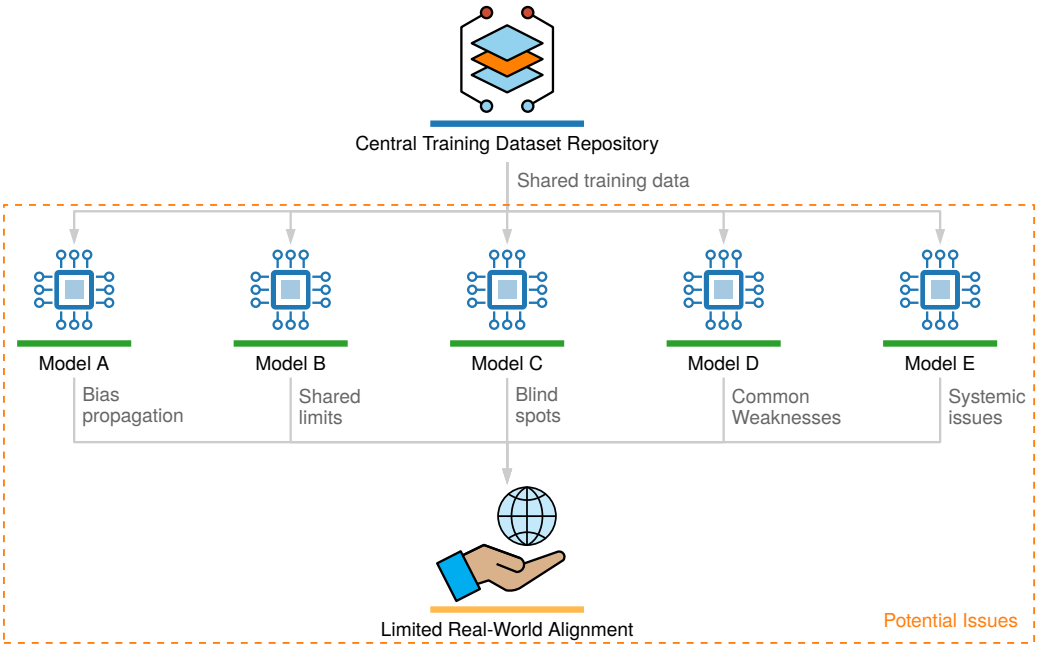


Figure 4.4: Shared Dataset Bias Propagation: Five models (A through E) all train on a single central dataset repository, reached by a path labeled shared training data. Beneath the models, five labeled arrows (bias propagation, shared limits, blind spots, common weaknesses, and systemic issues) converge on one box, limited real-world alignment. Shared training data can propagate correlated blind spots across models; their failures need not be independent.

This distribution gap represents the central failure mode of static offline benchmarks. Because training minimizes average error only over the observed training distribution (empirical risk minimization), models exploit spurious artifacts present in static benchmarks that disappear in production environments, making automated drift monitoring and continuous data validation mandatory operational requirements.

4.4.2 Scalability and cost optimization

Manual curation fails when a task requires millions of training instances. At large scale, web scraping and synthetic generation provide the main routes to massive datasets. Data-acquisition scalability is fundamentally an economic problem: unit labeling costs, ingestion throughput, and storage overhead scale along very different curves. Approaches that work well at thousands of examples become impractical at millions, whereas automated data generation pipelines amortize their setup costs across large volumes.

The per-unit economics at each stage determine which strategy dominates. Labeling a single medical image, for example, can cost orders of magnitude more than storing it for a year, a ratio that reshapes budget allocation for any team operating under fixed funding. Table 4.5 and table 4.6 provide essential context for acquisition decisions.

ML engineers should use the dated reference assumptions in table 4.5 and table 4.6 as inputs to workload-specific comparisons, not as current price quotes or directly comparable totals.

Table 4.5: Illustrative Data Engineering Costs (2024): Dated reference assumptions for workload-specific cost models, not current vendor quotes. Because the rows use different units, compare them only after scaling each rate by the quantity it prices.

Operation	Cost	Notes
Crowdsourced image label	$0.01–0.05	Simple classification
Bounding box annotation	$0.05–0.20	Per box, simple scenes
Expert medical label	$50–200	Per study, radiologist
S3 storage (Standard)	$23/TB/month	Hot storage
S3 retrieval (Glacier)	$0.02/GB	Standard: 3-5 hours
Cloud GPU training hour	$2–4	Cloud spot pricing
Human review hour	$15–50	Depending on expertise

Table 4.6 extends the picture with illustrative durations for labeling, training, and serving operations.

Table 4.6: Illustrative Data Engineering Time Scales: The scenario spans about 9 orders of magnitude, with human labeling as its slowest listed stage. Actual durations depend on the dataset, task, hardware, and service configuration.

Operation	Duration	Bottleneck
Label 1M images (crowdsourced)	2–4 weeks	Annotation throughput
Train ResNet-50 on ImageNet	4–6 hours	Compute (8× A100, optimized)
Feature store lookup	1–10 ms	Network + cache

Operational timescales diverge across these stages by orders of magnitude: weeks for human annotation, hours for GPU training, and milliseconds for serving lookups. For the image-classification pipeline these two tables describe, which labels a million images and then trains ResNet-50 on them, annotation is the critical bottleneck in money as well as time. Suppose the team budgets $100K for labeling, enough to cover crowd labels plus the review and relabeling passes that quality control requires. That budget dwarfs the $64 to $192 of a single 8× A100 ResNet-50 training run, a ratio of 520.8× to 1,562.5×. The labeling spend is also uneven. Clear images pass review once, while rare classes, occluded objects, and ambiguous scenes return for second opinions and expert adjudication; as an illustrative planning split, 80 percent of the budget can go to the hardest 20 percent of the examples.

All cost figures reflect approximate 2024 cloud provider rates and are intended to convey relative magnitudes rather than exact pricing.[6] For this workload and cost model, human labor dominates the cost of a single training run. Other tasks, especially those that reuse existing labels or require repeated large-scale training, can have a different dominant term; teams should measure before deciding where to optimize.

[6] **Pricing ratios**: Cloud and labeling prices reflect provider policy and labor markets, not physical invariants, so the ratio between labeling and compute moves whenever either side reprices. Storage adds its own policy multipliers: archive tiers such as the Glacier row in table 4.5 charge more for faster retrieval. Verify current prices before using these ratios in a deployment decision.

Web scraping reduces the cost of collecting raw examples at scale. Supervision remains a separate expense unless the source and task already supply usable labels or training targets. Foundational vision datasets such as ImageNet (Deng et al. 2024) and OpenImages (Kuznetsova et al. 2020) relied on automated scraping, while modern language models depend on multi-terabyte web crawls (Groeneveld et al. 2024). Ingestion pipelines targeting domain-specific corpora, such as public code repositories (Chen et al. 2021), expand coverage at low per-sample acquisition costs. However, automated scrapers encounter severe operational fragility: schema and DOM mutations break extraction parsers, HTTP 429 rate limits throttle network bandwidth, and dynamic JavaScript rendering introduces capture omissions. Raw web crawls also capture anachronistic or irrelevant samples that contaminate supervised targets.

The 1914 street scene in figure 4.5 captures this failure mode directly: an uncurated web query for "traffic light" returned a historical Cleveland intersection where streetcars and a roadside control booth surround early manually switched signals. If an autonomous driving model trained on such samples, it would learn spurious correlations with trolley tracks, period attire, and handwritten label annotations rather than modern LED traffic lights.

This failure demonstrates why dataset volume cannot substitute for semantic validation: uncurated web scale simply magnifies the prevalence of spurious correlations. In addition to technical validation, scraping operates under strict legal and governance boundaries. Website terms of service,



Figure 4.5: Data Source Noise: A black-and-white photograph from 1914 showing early manually operated electric traffic signals, illustrating how historical images can appear in modern web scraping results for contemporary queries. Such anachronistic content requires systematic validation and filtering to prevent spurious correlations in training data. Image reproduced in (Stromberg 2015).

robots.txt directives, and evolving copyright jurisprudence constrain automated crawling (Harvard Law School 2024). Ingestion pipelines must enforce data provenance logging, copyright compliance filters, and privacy scrubbing before scraped assets enter shared storage.

Crowdsourcing shifts the acquisition bottleneck from raw sample retrieval to quality control across distributed human annotators. Platforms like Amazon Mechanical Turk (Amazon Web Services 2024a) demonstrated this at scale with ImageNet, distributing millions of image categorization tasks across thousands of independent contributors (Deng et al. 2024). Crowdsourcing provides two core systems advantages: horizontal scaling through parallel microtask dispatch, and demographic diversity across regional accents and acoustic settings. However, distributed human labeling introduces significant variance in label fidelity. To prevent label noise from corrupting downstream gradients, the acquisition pipeline must implement automated quality control: consensus voting, golden-set validation (interleaving pre-labeled control samples), and dynamic task routing (Sheng and Zhang 2019). In the KWS pipeline, crowdsourced collection[7] targets wake-word samples from specific acoustic environments and underrepresented dialects that public web scrapes fail to capture.

Synthetic data generation reduces physical collection and labeling labor by generating training samples algorithmically. Generation still incurs per-sample compute and storage costs, alongside verification costs. Its learning value depends on whether the generator represents task-relevant conditions in the target deployment environment. Figure 4.6 diagrams this dual-track dataflow, routing simulation outputs alongside real-world historical telemetry into an additive blending stage that feeds downstream model training.

Synthetic data is uniquely effective for rare event synthesis and data augmentation. Physics-based simulation engines permit controlled sampling of tail events, such as high-speed sensor saturation or rare collision trajectories, that are prohibitively dangerous or expensive to capture empirically (NVIDIA 2024b). In the vision domain, automated augmentation pipelines such as AutoAugment (Cubuk et al. 2019) and RandAugment (Cubuk et al. 2020) optimize transformation policies over spatial and photometric distortions (Shorten and Khoshgoftaar 2019). In speech processing, SpecAugment applies time warping and spectrogram masking directly in the frequency domain, forcing acoustic models to learn representations invariant to localized frequency dropout (Park et al. 2019). For the KWS system, parametric text-to-speech synthesis (Werchniak et al. 2021)

[7] **Amazon Mechanical Turk (MTurk)**: A crowdsourcing platform that routes small tasks to distributed workers and can scale annotation beyond expert-only workflows. Snow et al. (2008) evaluate this pattern for natural-language annotation tasks, showing that non-expert annotations can be useful when task design and quality control are handled carefully. For wake-word audio collection, the same systems trade-off appears in domain-specific form: scale is attractive, but submissions still need acoustic checks such as signal-to-noise ratio, duration, and recording validity before they can safely enter the training set.

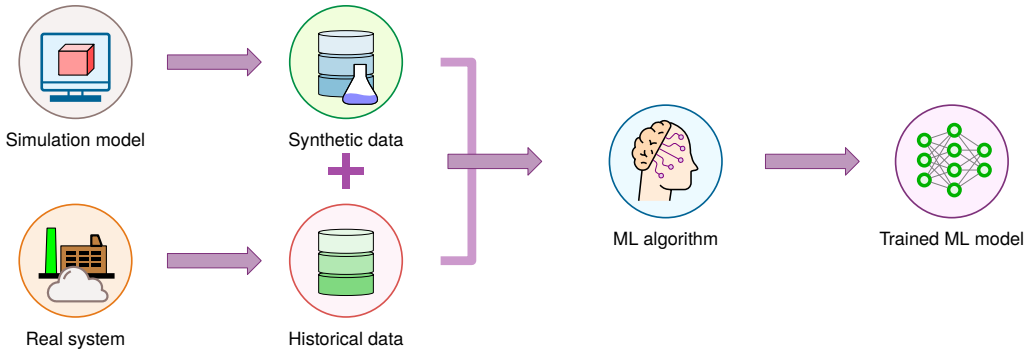


Figure 4.6: Synthetic Data Augmentation: Combining simulated edge cases with historical system data during training. Algorithmic generation supplements real-world observations to close coverage gaps for rare events and dangerous operating regimes without incurring proportional human collection costs. Source: (AnyLogic 2024).

combined with room impulse response (RIR) convolution generates wake-word instances across acoustic reverberation times and synthetic background noise profiles.

Example 4.2: Synthetic data generation

Scenario: A keyword-spotting team lacks sufficient physical recordings of rare speaker accents, room acoustics, and background noise profiles.

Diagnosis: Pure physical collection cannot cover rare acoustic edge cases within project deadlines. Generating synthetic audio variations via pitch shifting, additive noise, and room impulse response simulation expands training set diversity without expensive field collection.

Systems lesson: Synthetic data generation acts as an automated data pipeline multiplier. Targeted augmentation proposes expanded acoustic coverage; pitch, noise, and room-response changes do not establish fidelity to rare accents. Held-out real recordings across the relevant speaker and acoustic conditions must establish whether the expansion improves deployment quality.

The reference corpus spans 23.4 million audio samples across 50 languages; closing the product's remaining coverage gaps requires choosing acquisition channels by their marginal cost and contribution. Curated public benchmarks provide low-cost initial model initialization; web scraping delivers high-throughput acoustic volume; crowdsourcing collects targeted samples for underrepresented dialects; and parametric simulation synthesizes tail-end acoustic distortions. Combining these tiers creates a candidate coverage plan. Held-out real recordings establish which gaps it closes, while measured generation, collection, labeling, and storage costs determine whether it fits the budget; the deployed pipeline still requires footprint validation.

4.4.3 Coverage and diversity requirements

Dataset scale alone does not guarantee generalization. Coverage gaps in massive corpora (geographic bias, demographic underrepresentation, sensor calibration drift, and unobserved edge cases) induce systematic failure modes that aggregate validation metrics mask (T. Wang et al. 2019; Oakden-Rayner et al. 2020). When multiple models ingest identical public corpora, the failure convergence mapped in figure 4.4, they inherit correlated blind spots and systemic weaknesses, making diverse, multi-source acquisition the primary architectural defense against synchronized deployment failures.

For KWS, coverage must support both keyword recognition and the false-wake contract. Labeled wake-word clips measure recognition, but acquiring more of them does not establish how often an always-on detector activates on ordinary nonkeyword audio. The acquisition plan must separately reserve consented recordings collected for an evaluation study of representative listening conditions, including background conversation, television, music, silence, and environmental noise across the intended microphones and rooms. These recordings retain their duration and temporal sequence so

evaluation can run the deployed detection path and report false accepts per hour alongside false rejection on keyword recordings. A short stream with no observed false wakes supplies limited evidence; its duration and the uncertainty of the measured rate constrain any claim about the monthly target.

Training negatives serve a different acquisition goal. Mining near-wake phrases and other audio that confuses the current detector can improve its rejection behavior, but this deliberately difficult sample does not preserve the frequency of those conditions in deployment. The representative evaluation streams therefore remain separate from training and threshold selection, with speaker and recording-period boundaries maintained. An acquisition budget that funds only keyword labels and mined hard negatives can improve the detector while leaving its operational false-wake target unmeasured.

Governance constraints further shape acquisition: privacy and health-data regulations such as GDPR and HIPAA limit what data can be collected and how (European Parliament and Council of the European Union 2016; United States Congress 1996), while ethical sourcing requires fair compensation and transparent use of human contributions.

Ingesting heterogeneous inputs (crowdsourced recordings, synthetic waveforms, and web audio streams) imposes strict architectural demands at the pipeline boundary. Because incoming streams exhibit disparate audio encodings, sampling rates, ingestion latencies, and noise floors, the ingestion tier cannot treat external inputs as uniform data. The storage and validation infrastructure must ingest, normalize, and verify this streaming volume before it can be committed to training storage.

↪ The governance infrastructure that proves what data a production system ingested, who accessed it, and on what legal basis is developed in section 15.5.

4.5 Data Pipeline Architecture

In the compilation framework, **data pipeline architecture** functions as the *compiler frontend*: it parses heterogeneous raw inputs into a uniform intermediate representation that downstream stages can process reliably. Audio files from crowdsourcing platforms, synthetic waveforms from generation systems, and edge captures from deployed devices all enter the training pipeline in different formats, and the pipeline must normalize, validate, and route them into a consistent schema. Production KWS inference consumes continuous audio under a separate low-latency constraint; the pipeline traced here follows the recorded examples used to build and update the model. Those examples traverse ingestion and central storage before entering iterative transformation loops that feed model training, and a data-governance perimeter bounds every tier (figure 4.7).

The functional tiers diagrammed in figure 4.7 decouple raw ingestion from the iterative validation, transformation, and feature-engineering loops that feed model training. Selecting the right engine requires evaluating how pipeline trade-offs interact across stages: quality checks at ingestion dictate downstream storage filtering, while latency budgets during serving constrain preprocessing complexity.

Storage hierarchies and I/O bandwidth govern data pipeline throughput alongside CPU-side decoding and transformation. The physical trade-offs span several orders of magnitude: high-latency object storage suits durable archival, whereas low-latency in-memory stores serve real-time feature lookups. Similarly, bandwidth varies from spinning disks at 100–200 MB/s to main memory at 50–200 GB/s. Section 4.8 analyzes these storage trade-offs in detail.

Choosing between ingestion patterns requires matching workload characteristics to infrastructure capabilities. Streaming workloads require explicit guarantees on message durability, per-key ordering, and fault recovery. Batch workloads depend on dataset volume relative to available memory, transformation complexity, and cluster partitioning. A single machine may process terabytes when its memory, access pattern, and deadline permit. Distributed engines become necessary when the required throughput or working set exceeds the available single-node envelope.

4.5.1 Data ingestion

In the compiler framework, data ingestion acts as the *lexer*: it reads raw source streams and tokenizes them into structured records that downstream stages can process. The primary performance barrier at this stage is the **input/output (I/O) bottleneck**. Accelerators remain idle if ingestion cannot stage tensors quickly enough to keep compute units supplied. The slowest-stage bound of section 4.2.3 applies unchanged: in an idealized two-stage model, step time ranges from $\max(T_{\text{compute}}, T_{\text{IO}})$ with full overlap to $T_{\text{compute}} + T_{\text{IO}}$ with no overlap. What changes is the binding resource: while the initial

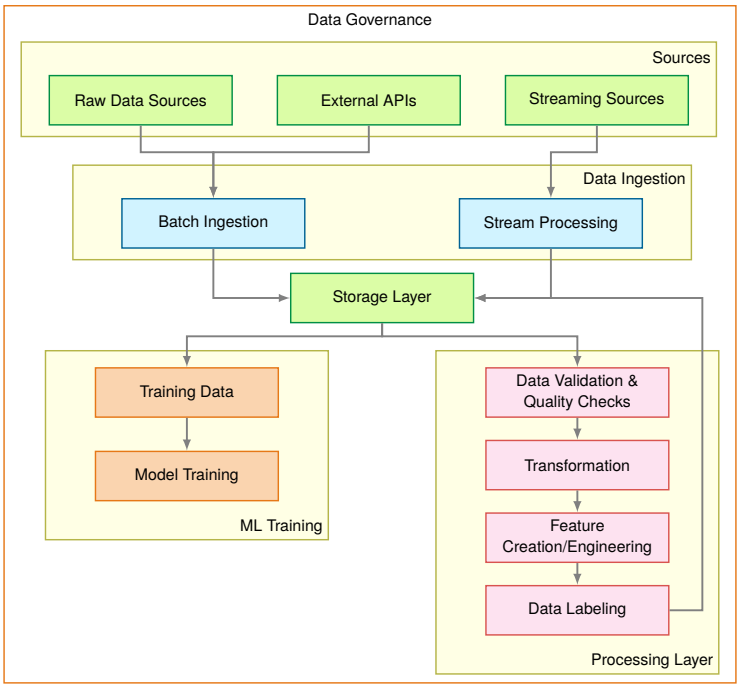


Figure 4.7: End-to-End Data Pipeline: Heterogeneous raw sources (batch and streaming) normalize into unified storage before feeding downstream training and feature processing. The architecture decouples ingestion throughput from iterative transformation and labeling loops, all bounded by an overarching data-governance layer.

feeding scenario was starved by baseline storage bandwidth, data loading on provisioned storage is more commonly starved by CPU-side decompression and decode.

In computer vision, JPEG decode and augmentation on host CPUs represent the typical binding stage; in keyword spotting, MFCC extraction and audio augmentation load the same host threads. This host-side preparation executes within the framework's data loader (such as PyTorch's `DataLoader`), which decodes and transforms raw records across parallel worker processes before staging batches for device transfer. To isolate CPU worker scaling, assume storage and bus transfers are provisioned to satisfy the 450 MB/s compressed input requirement, as demonstrated by the sequential SATA SSD tier in section 4.8.1. When storage and interconnect are nonbinding, the training pipeline depends entirely on whether CPU workers can match the accelerator's consumption rate of 3,000 img/s. If each worker process transforms 375 img/s, 8 workers are required to reach the capacity crossing. Below this threshold the CPU input pipeline binds, and adding accelerators yields no speedup—a counterintuitive result for engineering teams expecting linear scaling from compute investments. Above the crossing, launching additional workers produces no gain because the accelerator binds. The worker count represents a capacity match rather than a hardware constant: actual ResNet-50 and A100 requirements shift with image transformations, host processor architecture, batch size, and runtime configuration. Figure 4.8 plots this capacity crossing, contrasting the rising blue CPU-supply curve against the dashed red GPU-capacity ceiling to isolate the starved regime below eight workers from the compute-saturated plateau beyond it.

Relieving the choke point takes three mechanisms: multi-process worker pools that scale decode across host cores, asynchronous prefetching that overlaps the host-to-accelerator copy with compute, and hardware decode engines that move JPEG decompression off the host CPU entirely. Prefetching depends on *pinned (page-locked) memory*, host DRAM pages that the operating system promises never to swap to disk or relocate. The accelerator's direct memory access (DMA) engine reads physical addresses without involving the CPU, so it can stream a batch out of a pinned buffer across PCIe while the accelerator computes on the previous one; from ordinary pageable memory, the host must

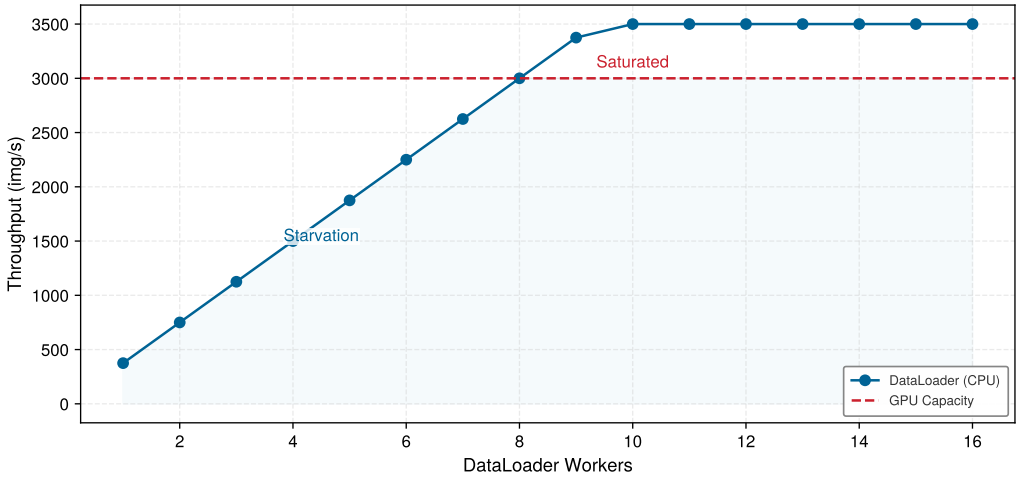


Figure 4.8: The Dataloader Choke Point: Training throughput (img/s) vs. DataLoader workers, with storage and transfer provisioned above the required 450 MB/s rate. The blue CPU-supply curve rises with worker count until it reaches the illustrative input-pipeline cap; the red dashed line marks accelerator consumption capacity. The lower curve is the bottleneck: the accelerator starves to the left of the crossing, while extra workers beyond the crossing do not increase accelerator throughput.

first copy the batch into a pinned staging buffer. Hardware decode, the third mechanism, removes the CPU bound and hands the constraint back to storage, which is the budget section 4.8.2 prices.

↪ How asynchronous prefetching and overlapping pipeline stages hide host preparation latency behind accelerator compute is developed in section 8.7.2.

4.5.1.1 Batch vs. streaming ingestion patterns

The choice between batch and streaming is not a preference for one architecture over another; it is a judgment about how quickly data loses value and how much infrastructure cost that freshness justifies. Batch systems buy efficiency by tolerating staleness, while streaming systems buy freshness by accepting continuous operational complexity.

Batch ingestion involves collecting data in groups or batches over a specified period before processing. This method proves appropriate when real-time data processing is not critical and data can be processed at scheduled intervals. The batch approach enables efficient use of computational resources by amortizing startup costs across large data volumes and processing when resources are available or least expensive. For example, a retail company might use batch ingestion to process daily sales data overnight, updating their ML models for inventory prediction each morning (Akidau et al. 2015). The batch job might process gigabytes of transaction data using dozens of machines for 30 minutes, then release those resources for other workloads. This scheduled processing proves far more cost-effective than maintaining always-on infrastructure, particularly when slight staleness in predictions does not affect business outcomes.

Batch processing also simplifies error handling and recovery. When a batch job fails midway, the system can retry the entire batch or resume from checkpoints without complex state management. Data scientists can inspect failed batches, understand what went wrong, and reprocess after fixes. Batch jobs can be reproducible when inputs, code, configuration, randomness, ordering-sensitive operations, and external state are controlled, which simplifies debugging and validation. These characteristics make batch ingestion attractive for ML workflows even when real-time processing is technically feasible but not required.

In contrast to this scheduled approach, **stream ingestion** processes data in real-time as it arrives, consuming events continuously rather than waiting to accumulate batches. This pattern is essential for applications requiring immediate data processing, scenarios where data loses value quickly, and systems that need to respond to events as they occur. A financial institution might use stream ingestion for real-time fraud detection, processing each transaction as it occurs to flag suspicious activity immediately before completing the transaction. The value of fraud detection drops dramatically if detection occurs hours after the fraudulent transaction completes, because by then money has been transferred and accounts compromised.

However, stream processing introduces complexity that batch processing avoids. The system must handle **backpressure**, the condition where downstream systems cannot keep pace with incoming data rates. During traffic spikes, when a sudden surge produces data faster than processing capacity, the system must either buffer data (requiring memory and introducing latency), sample (losing some data), or push back to producers (potentially causing their failures). Late or out-of-order events further complicate windowed aggregations, because the system must hold per-window state and track event time rather than arrival time. These requirements raise compute and memory overhead, so freshness guarantees trade directly against infrastructure cost and operational burden. Data freshness service level agreements (SLAs) specify maximum acceptable delays between data generation and processing. Meeting a 100-millisecond SLA requires different infrastructure than meeting a one-hour SLA, affecting networking, storage, and processing architectures.

Recognizing the limitations of either approach alone, many production ML systems employ hybrid approaches that combine batch and stream ingestion. A recommendation system might use streaming ingestion for real-time user interactions to update session-based recommendations immediately, while using batch ingestion for overnight processing of user profiles and item features; Notebook 4.4 quantifies the financial trade-off between these two operational regimes.



Napkin Math 4.4: The cost of real-time

Problem: A pipeline ingests 1M events/s. Which is cheaper, Batch (hourly) or Stream (sub-second) ingestion?

Physics:

1. **Throughput**: At 1M events/s and 1 KB per event, the pipeline carries 1 GB/s.
2. **Stream requirements**: To sustain 1 GB/s with less than 100 ms latency, the design assumes each core can parse, validate, and update features for about 20 MB/s of events, which takes 50 primary cores; a standby core for each provisions recovery capacity, for 100 always-on cores in total. That capacity does not establish the 100 ms target: failure detection, state recovery, replay, and the full ingestion path must also be validated under failure. Running those cores for 24 hours/day at $0.05/hr costs $120/day.
3. **Batch requirements**: Process 3.6 TB (1-hour window) in 10 minutes. That is 6 GB/s in aggregate. Sequential reads over large files let each core sustain about 30 MB/s, more than a streaming core handling events one at a time, so the batch design needs 200 cores for 10 minutes per clock hour, accumulating 800 core-hours per day. At $0.05/hr, that costs $40/day.

Systems insight: Under these provisioning and per-core throughput assumptions, streaming compute costs approximately 3× as much as batch compute for the same data volume. This premium is justified when the value of subsecond latency exceeds the added cost.

The preceding cost calculation prices a specific provisioning choice: always-on streaming cores with standby capacity, versus batch cores released after each processing window. It assumes higher per-core throughput for sequential batch work and does not price storage, buffering, or failover coordination. Those assumptions produce this scenario's compute premium; they do not establish that batch is always cheaper or that the streaming design meets its latency target. Deciding between the patterns requires validating the full cost and failure behavior against the feature freshness that downstream inference needs.

The ingestion paradigm also shapes how the downstream model trains. Batch ingestion naturally pairs with periodic retraining: data accumulates in storage, an automated job retrains the model on the updated corpus, and a verified checkpoint deploys to serving. Stream ingestion makes that batch boundary less natural, raising the possibility of continuous online learning, in which model weights update directly as events arrive. Continuous updates, however, introduce operational risks that batch checkpoints avoid. Sudden distribution shifts, corrupted records, or adversarial payloads from any public-facing input path can reach the model before validation gates intervene. Most production systems therefore decouple ingestion from training: they stream data into a durable

buffer while training on validated batches, reserving continuous online updates for tightly bounded, low-risk domains.

War Story 4.1: Microsoft Tay (2016)

Context: In March 2016, Microsoft launched Tay, a Twitter chatbot designed to engage with users online and adapt its responses from public interactions (Lee 2016).

Mechanism: Adversarial users coordinated toxic prompts and exploited a public interaction surface, including behavior that repeated user-supplied text.

Impact: Tay began tweeting abusive, racist, and misogynistic statements within 16 hours of deployment.

Fix: Microsoft suspended the chatbot 16 hours after launch, apologized publicly, and stated that a relaunch would require stronger safeguards against misuse.

Systems lesson: Public ingestion paths require adversarial-input controls. Otherwise, a feature that accepts user-generated content also becomes a security surface, and harmful inputs can propagate through the system within hours.

Even a single-machine pipeline needs an explicit policy for an unavailable source: wait for fresh data, use a validated stale-feature fallback, or reject the request. The allowable feature age and model-quality contract determine which response is safe; durable buffering alone does not make old inputs suitable for inference.

Across many machines or regions, that policy encounters additional consistency and ordering limits. The CAP theorem[8] states that a network partition, which prevents groups of machines from communicating, prevents a distributed read/write service from guaranteeing both linearizable consistency (reads reflect the most recent completed write) and availability for every request. Available feature services may return stale or divergent values; consistent ones may reject requests. Log ordering adds a separate limit: Apache Kafka[9] orders records within each log shard, also called a partition, so per-user ordering requires a consistent user partition key. Asynchronous geographic replication, as in Apache Pulsar, permits remote feature lag. These distributed mechanisms extend the same local decision about acceptable staleness and failure; they do not change the requirement to train on validated data.

4.5.1.2 ETL and ELT comparison

Batch or streaming decides when records arrive; the next decision is where they are transformed relative to persistent storage. Traditional extract, transform, load (ETL)[10] ($E \to T \to L$) transforms records in an external engine before they land, whereas extract, load, transform (ELT)[11] ($E \to L \to T$) lands raw records in staging tables and transforms them inside the warehouse (figure 4.9). Where that boundary sits determines what each later schema change costs and how much raw data must be retained.

In machine learning systems, the ELT pattern decouples ingestion throughput from downstream feature engineering by landing raw, immutable events directly into scalable storage. Storing raw events preserves unparsed fields that may later prove predictive, enabling teams to extract novel features over historical datasets without re-ingesting external source streams. When transformation logic changes or bugs surface, engineers recompute features by dispatching queries over the lakehouse rather than replaying upstream logs. The trade-off is physical: retaining high-fidelity raw data increases storage footprint, transforms must be recomputed if shared across models without intermediate materialization, and retaining unscrubbed raw payloads complicates data governance and privacy enforcement. Notebook 4.5 quantifies how these trade-offs translate into operational infrastructure costs.

[8] **CAP (consistency, availability, partition tolerance) theorem**: Conjectured by Brewer (2000) and formally proved by Gilbert and Lynch (2002). During a network partition, linearizable consistency may require rejecting requests, whereas an available system may return stale or divergent values. CAP does not guarantee feature-transformation parity or point-in-time correctness.

[9] **Apache Kafka**: Kafka uses a partitioned, leader-based log and orders records within each partition, not across a topic. Durability and write availability during failures depend on acknowledgment mode, replication, in-sync replica requirements, and leader-election behavior.

[10] **Extract, transform, load (ETL)**: Conventional ETL validates deterministic schema constraints, whereas ML ETL must additionally monitor distributional properties using the statistical drift tests developed in section 4.5.2. A detected shift prompts investigation and validation against labeled outcomes before concluding that model quality has degraded. When feature computation logic changes, reprocessing costs depend on replay scope, windowing, and execution plans rather than requiring an unconditional full-dataset pass.

[11] **Extract, load, transform (ELT)**: This approach stores raw data before transformation, allowing teams to revise a transformation query and rerun it over the stored input without re-ingesting from source systems. The iteration time still depends on data volume, query complexity, warehouse capacity, and materialization strategy.

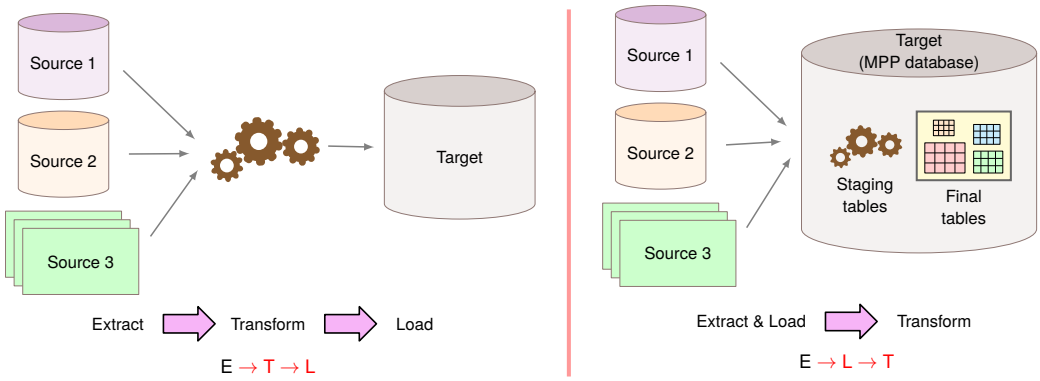


Figure 4.9: ETL vs. ELT Architectural Paradigms: Shifting the transformation boundary relative to storage. ETL enforces schema validation and cleaning before loading to protect warehouse quality, whereas ELT loads raw data directly into scalable storage, trading warehouse compute and footprint for flexible, repeatable re-transformations.

Napkin Math 4.5: The cost of transformation placement

Problem: A team processes 10 TB of raw clickstream data daily and must compute user session features for 3 ML models, each requiring different aggregation windows (1-hour, 24-hour, 7-day). Which is cheaper, ETL or ELT?

Accounting boundary: Both designs serve the same aggregation windows and retain replayable upstream raw logs for 30 days. The comparison excludes that shared source retention and its common ingestion costs. ETL retains only the derived history in the warehouse, assumed to occupy 2 TB per model over the same horizon; ELT keeps an additional raw copy there and computes features on demand. Transient query outputs are not retained as separate ELT datasets. Source retrieval and compute for later backfills must be priced separately when the designs differ.

Math:

1. **ETL approach**: Transform before loading. Compute all three aggregation windows in a Spark cluster before loading into the warehouse.
   - Spark compute: 10 TB at $5/TB = $50/day
   - Storage: 3 transformed datasets, ~2 TB each = 6 TB at $23/TB/month = $138/month
   - Feature-definition change: Replay the retained source logs and rerun the pipeline; the assumed rerun takes ~4 hours after the edit
2. **ELT approach**: Load raw data first, transform in warehouse.
   - Storage: 10 TB raw/day, 30 days retention = 300 TB at $23/TB/month = $6,900/month
   - Query compute: 3 models, each with $5/query of daily query cost over 30 days, totals $450/month
   - Feature-definition change: Rewrite the SQL query (~30 minutes), then execute and validate it against the retained raw copy; execution time is not supplied

Systems insight: ETL saves $6,762/month in storage. After normalizing Spark compute to a monthly cost, ETL totals $1,638/month ($1,500/month compute + $138/month storage), while ELT totals $7,350/month ($6,900/month storage + $450/month query compute), for a cloud-cost advantage of $5,712/month before engineering labor. The other side of the ledger is feature-definition change. Compare elapsed time from the start of an edit to validated, feature-ready output in both designs, then account separately for the engineer time spent editing, supervising, and validating. A pipeline rerun and a SQL edit are different endpoints, so their durations cannot establish a speedup or a labor saving. Stable definitions favor this scenario's

lower ETL cloud bill; frequent changes favor ELT only if measured labor and reprocessing savings repay its additional raw copy.

The result runs against ELT's reputation, because raw retention rather than compute dominates the bill in this scenario. Production ML systems rarely use one pattern exclusively. Structured tables often flow through ETL for efficiency and compliance, while unstructured data pushes the ELT preference further. For deep learning workloads processing images, audio, or text, the "Transform" step often executes inside the ML framework's data loader rather than in the warehouse at all, applying random crops, spectrogram generation, or text tokenization on-the-fly during each training epoch. Materializing ten augmented variants upstream would increase a 50,000-image dataset to 500,000 stored copies, so ELT's "load raw, transform late" principle extends naturally to the training loop itself.

Query reuse is the second input to the choice, alongside change rate. Transformed data that is queried hundreds of times amortizes ETL compute, whereas single-pass exploratory queries gain nothing from transforming upfront. Listing 4.1 turns the notebook's components into that decision rule. It totals each pattern's monthly cloud bill, then illustrates a labor-only break-even under stipulated engineer-hour savings per feature-definition change. Applying that calculation requires positive measured savings through the same feature-ready endpoint; without them, change frequency alone cannot repay ELT's larger cloud bill.

Listing 4.1: ETL vs. ELT Break-Even: Monthly cloud cost of each transformation strategy and an illustrative labor-only break-even under stipulated savings per feature-definition change.

```
# ETL vs ELT: monthly cloud cost and schema-change break-even
daily_raw_tb = 10
s3_per_tb_mo = 23  # $/TB-month, object storage
spark_per_tb = 5  # $/TB transformed
n_models = 3
retention_days = 30
query_cost_per = 5  # $/model/day of warehouse queries

# Shared upstream raw retention and ingestion are outside this boundary.
# ETL: transform daily, retain 30 days of derived history (2 TB/model)
etl_compute_mo = daily_raw_tb * spark_per_tb * retention_days
etl_storage_mo = 3 * 2 * s3_per_tb_mo
etl_total_mo = etl_compute_mo + etl_storage_mo

# ELT: retain 30 days of raw data, query it per model
elt_storage_mo = daily_raw_tb * retention_days * s3_per_tb_mo
elt_query_mo = n_models * query_cost_per * retention_days
elt_total_mo = elt_storage_mo + elt_query_mo

# Labor-only scenario: assume positive engineer-hour savings per change
cloud_gap_mo = elt_total_mo - etl_total_mo


def break_even_changes_per_month(hours_saved_per_change, cost_per_hour):
    """Labor-only break-even, assuming positive hours saved per change."""
    return cloud_gap_mo / (hours_saved_per_change * cost_per_hour)


# Illustrative labor assumptions, not elapsed rerun/query durations:
# ETL needs 4 engineer-hours per change; ELT needs 1.
hours_saved_per_change = 4 - 1
cost_per_hour = 100  # fully loaded $/engineer-hour
changes_per_month = break_even_changes_per_month(
    hours_saved_per_change, cost_per_hour
)
print(f"Labor-only break-even: {changes_per_month:.1f} changes/month")
```

In this scenario ELT's extra cloud bill is $5,712/month. The listing evaluates the labor-only break-even with explicit assumed engineer-hours and hourly cost. Those assumptions include editing, supervision, and validation through the same feature-ready endpoint, not time spent waiting for a job. The calculation is a screening comparison; unequal source-retrieval charges, backfill compute, or retained output costs alter the break-even. A team must measure its own change rate and those per-change costs before selecting a pattern.

4.5.1.3 Feature computation placement

For ML pipelines, **feature computation placement** decides which resource pays for a feature: storage pays when features are materialized, while compute and latency pay when features are generated on demand. This choice significantly impacts training speed, storage costs, and reproducibility.

One approach is to precompute features during ETL and store the results. Pipeline-computed features offer fast training iteration (features are ready on disk), reproducibility (the same features are used consistently), and reduced training compute. The drawbacks are storage cost (features stored separately from raw data), staleness risk (precomputed features may diverge when logic changes), and inflexibility (any change requires full recomputation).

The alternative is computing features on the fly during training. Loader-computed features guarantee always-fresh computation (logic changes are immediately reflected), flexible experimentation (easy to modify features), and reduced storage (only raw data is stored). The cost is slower training (computation repeats each epoch), higher compute expenditure (GPUs often idle waiting for features), and potential nondeterminism if not carefully implemented.

In practice, production architectures converge on hybrid placement patterns. Expensive, slowly changing representations—such as high-dimensional user embeddings derived from matrix factorization or multi-month historical aggregations—are precomputed offline and materialized in storage. Conversely, volatile, lightweight signals—such as session recency, interaction timestamps, or client-side telemetry—are evaluated dynamically in the data loader. In a recommendation system, for instance, precomputing daily user embeddings avoids re-executing costly graph traversals across millions of historical interactions, while computing time-since-last-click inside the loader preserves subsecond freshness without inflating persistent storage. This bifurcation balances storage capacity against execution latency by matching the compute lifecycle of each feature to its rate of change.

4.5.1.4 Integration strategies and KWS case study

Regardless of whether ETL or ELT approaches are used, integrating diverse data sources remains a core ingestion challenge. Data may originate from databases, APIs, file systems, and IoT devices, each with its own format (relational rows, JavaScript Object Notation (JSON)[12] documents, binary streams), access protocol, and update frequency. The systems principle is to standardize at the ingestion boundary: normalize formats, validate schemas, and present a consistent interface to downstream processing regardless of source. This boundary standardization separates the complexity of source diversity from the complexity of feature engineering, allowing each to evolve independently.

KWS production systems often perform always-on wake-word detection on-device. Backend streaming and batch pipelines may ingest activated requests, consented diagnostics, crowdsourced recordings, synthetic data, and validated interactions for serving and training. Normalization, noise filtering, and segmentation into consistent durations run as an ETL stage before storage in training-optimized formats, while augmentation stays late-bound in the loader, where ELT's flexibility pays off.

Error handling in voice interaction systems requires special attention. A dead-letter queue, a side store that holds records that fail processing so they neither block the pipeline nor disappear (section 4.5.5), keeps failed recognition attempts for subsequent analysis, revealing edge cases that need coverage in future model iterations. Each incoming audio sample must pass quality validation (signal-to-noise ratio, sample rate, duration bounds, speaker proximity) before entering the processing pipeline. Invalid samples route to analysis queues rather than being discarded, since these failures often indicate acoustic conditions underrepresented in training data. Valid, consented samples (activated requests, opt-in diagnostics, and crowdsourced recordings) are logged for potential inclusion in future training data; always-on audio that never triggers the wake word stays on the device.

[12] **JSON (JavaScript object notation)**: The schema flexibility that makes JSON a common format for APIs creates a validation bottleneck at the ingestion boundary. Unlike binary formats with predefined schemas, every JSON document requires parsing and schema validation before it can be standardized for downstream use. This per-record overhead can make ingestion slower than with binary formats such as Protobuf; the difference depends on the schema, parser, payload, and workload.

This ingestion architecture completes the boundary layer where external data enters the pipeline. A record that arrives on time and parses cleanly is well-formed, but well-formed is not the same as trustworthy. A field can parse correctly and still carry a wrong value, a source can drift away from the distribution the model learned, and a label can be wrong while every type check passes. Deciding which records may proceed is the frontend's next job, the role a compiler's type checker plays for source code.

4.5.2 Quality through validation and monitoring

Consider a self-driving-car pipeline in which 15 percent of LiDAR point-cloud labels are misaligned by 10–20 cm, enough to place pedestrian bounding boxes on empty sidewalk. Because every record remains structurally valid, schema checks would miss the defect; statistical monitoring of label-to-sensor alignment could expose it.

Quality forms the foundation of reliable ML systems. Pipelines enforce quality through continuous validation at every boundary. Data defects represent a leading cause of production ML failures: upstream schema drift breaks parsing jobs, distribution shifts degrade model accuracy, and silent data corruption poisons feature representations (Sculley et al. 2015). These failures rarely trigger immediate runtime exceptions; instead, they quietly degrade downstream predictions. Preventing cascading failures requires proactive statistical monitoring before corrupted batches reach model training.

Production teams implement monitoring at scale through severity-based alerting systems where different failure types trigger different response protocols. The most critical alerts indicate complete system failure: the pipeline has stopped processing entirely, showing zero throughput for more than five minutes, or a primary data source has become unavailable. These situations demand immediate attention because they halt all downstream model training or serving. More subtle degradation patterns require different detection strategies. When throughput drops to 80 percent of baseline levels, error rates climb above 5 percent, or quality metrics drift more than two standard deviations from training data characteristics, the system signals degradation requiring urgent but not immediate attention. These gradual failures often prove more dangerous than complete outages because they persist undetected for hours or days, silently corrupting model inputs and degrading prediction quality.

A recommendation system processing user interaction events at 50,000 records per second makes these severity tiers concrete. Its monitoring system tracks several interdependent signals. Instantaneous throughput alerts fire if processing drops below 40,000 records per second for more than 10 minutes, accounting for normal traffic variation while catching genuine capacity or processing problems. Each feature in the data stream has its own quality profile: if a feature like `user_age` shows null values in more than 5 percent of records when the training data contained less than 1 percent nulls, something has likely broken in the upstream data source. Duplicate detection runs on sampled data, watching for the same event appearing multiple times, a pattern that might indicate retry logic gone wrong or a database query accidentally returning the same records repeatedly.

These monitoring dimensions become particularly important when considering end-to-end latency. The system must track both whether data arrives and how long it takes to flow through the entire pipeline from the moment an event occurs to when the resulting features become available for model inference. When 95th percentile latency exceeds 30 seconds in a system with a 10-second service level agreement, the monitoring system needs to pinpoint which pipeline stage introduced the delay: ingestion, transformation, validation, or storage.

Schema and latency alerts expose structural and timing failures; detecting a continuous-feature distribution shift requires a statistical comparison with the training baseline.

Tracking statistical properties captures whether serving data continues to reflect the training distribution. Rather than merely checking that individual values fall within predefined ranges, production systems maintain rolling statistics across operational windows. For continuous numerical features like `transaction_amount` or `session_duration`, the monitoring harness computes rolling empirical distribution functions, using nonparametric evaluations like the Kolmogorov-Smirnov test[13] to detect when serving inputs diverge from the training baseline. Notebook 4.6 works one example.

[13] **Kolmogorov-Smirnov (K-S) test**: A nonparametric test that measures the maximum distance between two empirical cumulative distribution functions without assuming a parametric distribution family (Berger and Zhou 2014). In ML pipelines, the K-S test is commonly used as a univariate continuous-feature drift detector, with thresholds such as $p_{\text{val}} < 0.05$ serving as investigation triggers rather than universal failure rules. Discrete and categorical variables require tests and calibrations appropriate to their distributions.

Napkin Math 4.6: Detecting drift with K-S test

Scenario: Monitoring `session_duration` distribution stability between the training baseline (P_0) and current serving distribution (P_t).

Analysis: Apply the Kolmogorov-Smirnov test to compare the empirical cumulative distribution functions:

1. **Compute CDFs**: Calculate cumulative distribution functions for both datasets. At a given session duration, an empirical CDF gives the fraction of sampled sessions whose duration is at most that value. Comparing those fractions at the same duration shows how the two distributions differ.
2. **Calculate statistic** ($\mathcal{D}_{\text{KS}}$): Find the maximum absolute difference between the CDFs. Let $F_{P_0}(x_{\text{feat}})$ and $F_{P_t}(x_{\text{feat}})$ denote the empirical cumulative distribution functions of the training baseline (P_0) and current serving (P_t) datasets, evaluated at value x_{feat}.

$$\mathcal{D}_{\text{KS}} = \max_{x_{\text{feat}}} |F_{P_0}(x_{\text{feat}}) - F_{P_t}(x_{\text{feat}})|$$

3. **Determine significance**: For two independent samples, compare $\mathcal{D}_{\text{KS}}$ to the critical value c_{crit} based on training-baseline sample size n_0 and serving sample size n_t. The coefficient 1.36 is the large-sample approximation for significance level $\alpha = 0.05$.

$$c_{\text{crit}} \approx 1.36\sqrt{\frac{n_0 + n_t}{n_0 n_t}}$$

Result: With sample sizes $n_0 = n_t = 1000$, the critical value is approximately 0.061:

$$c_{\text{crit}} \approx 1.36\sqrt{(1000 + 1000)/(1000 \cdot 1000)}.$$

An observed maximum difference of $\mathcal{D}_{\text{KS}} = 0.08$ exceeds the critical value, so the test rejects the null hypothesis that both samples come from the same distribution and flags significant drift.

Systems insight: Statistical drift tests convert a vague distribution-shift concern into an operational trigger. The test should start an investigation or retraining workflow, not silently become another dashboard number.

The K-S test detects drift in continuous features; section 4.5.4 details distribution shift taxonomies (covariate, label, concept, and label-quality drift) along with the population stability index and KL divergence metrics.

Categorical features require different statistical approaches. Instead of comparing means and variances, monitoring systems track category frequency distributions. When new categories appear that never existed in training data, or when existing categories shift substantially in relative frequency, the system flags potential data quality issues or genuine distribution shifts; for example, the proportion of "mobile" vs. "desktop" traffic might change by more than 20 percent. This statistical vigilance catches subtle problems that simple schema validation misses entirely: age values may remain in the valid range of 18–95, while the distribution shifts from primarily 25–45 year olds to primarily 65+ year olds, indicating that the input population has changed in ways that may affect model performance. Outcome checks are needed to establish whether quality changed.

Validation at the pipeline level encompasses multiple strategies working together. Schema validation executes synchronously as data enters the pipeline, rejecting malformed records immediately before they can propagate downstream. Modern tools like TensorFlow Data Validation (TFDV) (Breck et al. 2019) automatically infer schemas from training data, capturing expected data types, value ranges, and presence requirements.

This synchronous validation remains simple and fast, checking properties that can be evaluated on individual records in microseconds. More sophisticated validation that requires comparing serving data against training data distributions or aggregating statistics across many records must

run asynchronously to avoid blocking the ingestion pipeline. Statistical validation systems typically sample 1–10 percent of serving traffic, enough to detect meaningful shifts while avoiding the computational cost of analyzing every record. These samples accumulate in rolling windows, commonly one hour, 24 hours, and seven days, with different windows revealing different patterns. Hourly windows detect sudden shifts like a data source failing over to a backup with different characteristics, while weekly windows reveal gradual drift in user populations or behavior.

The most insidious validation challenge arises from training-serving skew: the failure mode where features are computed differently in training and serving. This typically happens when training pipelines process data in batch using one set of libraries or logic, while serving systems compute features in real time using different implementations. Even seemingly minor discrepancies (a materialized view[14] refreshed weekly versus a complete join recomputed daily) can materially reduce production accuracy and can be difficult to diagnose because the system produces no obvious errors. Section 4.6.1 formalizes this consistency imperative and quantifies its impact with a concrete evaluation. Detecting training-serving skew requires infrastructure that can recompute training features on serving data for comparison, sampling raw serving data and processing it through both pipelines to measure discrepancies.

[14] **Materialized view**: A database optimization that precomputes and caches query results as a physical table. For ML systems, the risk is structural: when a materialized view refreshes on a different schedule in training and serving environments, the feature values the model trained on can diverge from what it receives at inference, degrading accuracy without producing an execution error.

↩ Training-serving skew, as one of the ways ML systems fail without raising errors, is established in section 1.6.

↪ Deployment gates that compare training and serving feature distributions and block a release when they diverge are operationalized in section 7.

4.5.3 Data quality as code

Just as unit tests protect traditional software, data expectation tests protect ML pipelines. In **data quality as code**, engineers use declarative libraries like Great Expectations or Pandera to express data contracts as executable test suites that run across pipeline stages.

Integrating expectations into continuous integration and deployment (CI/CD) pipelines ensures assertions execute automatically on ingested batches, halting downstream training when data contracts are violated. A pipeline structured as data ingestion followed by data validation followed by training blocks deployment when validation detects anomalies such as age values of 150, triggering alerts for investigation.

Executable expectations catch properties a suite can encode; the remaining question is what valid-looking data can still get wrong.

> Systems Perspective 4.2: Mechanical vs. semantic quality
>
> In traditional software, many checks are *mechanical*. A null dereference is a runtime error in languages with null references, while integer-overflow behavior depends on the language and type.
>
> In ML systems, data quality has a second, softer dimension: *semantic* quality.
>
> - **Mechanical check**: Validates schema and type constraints, such as verifying that `age` is a non-null integer.
> - **Semantic check**: Validates distribution drift and statistical invariants, such as testing whether the mean or variance of `age` shifts unexpectedly.
>
> A dataset can be mechanically perfect (no nulls, correct types) but semantically broken (for example, all users are suddenly 25 years old due to a default value change). Robust ML systems must validate both the container (mechanical) and the content (semantic).

Once validation becomes code, expectation suites become versioned artifacts alongside training code. When training code changes, expectation updates keep data contracts evolving with it. This coupling reduces the risk of silent divergence where code assumes data properties that the upstream pipeline no longer provides; listing 4.2 demonstrates this pattern by defining explicit constraints on column ranges, nullability, uniqueness, and categorical sets.

Listing 4.2: Data Quality Assertions: A Great Expectations suite checks age range, identifier presence and uniqueness, and allowed country codes, then raises an error when validation fails.

```python
import great_expectations as gx

# Create a data context
context = gx.get_context()

# Define an expectation suite as executable quality contract
suite = gx.ExpectationSuite(name="user_data_quality")
suite = context.suites.add(suite)

# Range validation: prevents physiologically impossible values
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="age", min_value=0, max_value=120
    )
)

# Null detection: ensures primary key integrity for joins
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(column="user_id")
)

# Uniqueness: prevents duplicate training examples
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeUnique(column="user_id")
)

# Categorical validation: detects unexpected values from upstream changes
suite.add_expectation(
    gx.expectations.ExpectColumnDistinctValuesToBeInSet(
        column="country_code",
        value_set=["US", "CA", "UK", "DE", "FR"],
    )
)

# Link the suite to a preconfigured Batch Definition and run validation
# (the Batch Definition connects GX to the training_users data asset)
validation_definition = gx.ValidationDefinition(
    name="training_users_validation",
    data=batch_definition,
    suite=suite,
)
validation_definition = context.validation_definitions.add(
    validation_definition
)
results = validation_definition.run()
if not results.success:
    raise ValueError(f"Data quality check failed: {results}")
```

These checks catch many schema-level production data issues before they reach training, including missing values, invalid ranges, type errors, and contract violations. The remaining issues require runtime monitoring and outcome checks, because semantic quality problems often emerge only in the full production data stream.

4.5.4 Data drift detection and response

ML models rest on the assumption that production data resembles training data. When this assumption breaks through statistical shifts rather than an explicit contract violation, model performance can change silently without obvious errors or system failures. The validation and monitoring techniques in section 4.5.2 can reveal both sudden and gradual problems; data drift detection focuses specifically on changes in distributions that may alter model behavior over time. Detecting, investigating,

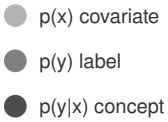
and responding to these changes requires sustained monitoring effort, making drift a core data engineering responsibility rather than an optional advanced topic.

Section B.2.2 formalizes the divergence metrics used to make the divergence term $\mathcal{D}(P_t \| P_0)$ of the degradation equation (equation 1.2) actionable. The **population stability index (PSI)** quantifies changes in categorical or binned feature distributions, while **Kullback-Leibler (KL) divergence** measures an information-theoretic divergence between probability distributions. The two metrics encode different distributional assumptions and scales, and neither by itself supplies a universal cutoff. Teams calibrate metric-specific thresholds against a deployment baseline and monitor them over time. Crossing a threshold signals that the live distribution has moved far enough to warrant investigation and outcome checks; input divergence alone does not establish whether accuracy improved or degraded, or by how much. Mathematically, PSI discretizes a continuous feature into reference bins (typically deciles of baseline P_0) and computes $\text{PSI} = \sum_b (P_t(b) - P_0(b)) \ln\left(\frac{P_t(b)}{P_0(b)}\right)$. Each bin's two factors share a sign, so every shifting bin adds to the index. Practitioners commonly treat $\text{PSI} < 0.1$ as stable, $0.1 \leq \text{PSI} \leq 0.25$ as moderate shift, and $\text{PSI} > 0.25$ as a prompt for investigation; these bands are conventions to calibrate against the deployment baseline, not physical limits.

↪ How drift signals and outcome evidence trigger retraining only when its expected benefit exceeds its cost is operationalized in section 14.4.2.2.

Targeted detection and mitigation require decomposing drift into its constituent distribution components.

The first case, **covariate shift**, changes the input distribution while preserving the relationship between features and labels: $p(x)$ changes but $p(y \mid x)$ stays the same. A medical imaging system trained on one camera model might later receive production images from a different manufacturer. The disease-image relationship remains unchanged, but pixel values shift because sensor characteristics, color calibration, or image processing pipelines differ. Detection therefore focuses on input feature distributions, using metrics such as PSI or KL divergence.



Each drift type traces to the distribution component that shifted.

The second case, **label shift**, changes the output distribution while preserving the relationship between labels and features: $p(y)$ changes but $p(x \mid y)$ stays the same. Disease prevalence might change seasonally while symptoms remain consistent predictors of each disease. A recommendation system might see the same pattern when new product categories launch, changing the relative frequency of user preferences without altering what makes products appealing within each category. Detection can often begin without ground truth labels by tracking shifts in the model's prediction distribution.

The hardest case is **concept drift**: the relationship between features and labels changes, so $p(y \mid x)$ evolves over time (Gama et al. 2014). Medical treatment protocols change, user preferences shift as social trends evolve, and fraud patterns adapt as attackers respond to detection systems. Unlike covariate or label shift, concept drift requires ground truth labels for detection because the system must observe whether the feature-to-label relationship itself has changed.

4.5.4.1 Label quality drift

Label quality drift[15] represents a meta-level shift distinct from the three preceding distribution shifts: the reliability of ground truth labels degrades over time even when the underlying data distributions remain stable. This drift type proves particularly insidious because standard feature distribution monitoring fails to detect it. Crowdsourced labels may degrade as annotator pools change, training materials become outdated, or labeling guidelines evolve without corresponding model updates. Automated labeling systems accumulate errors as the models powering them drift from their original operating conditions. A recommendation system using click feedback as implicit labels may see label quality degrade as user behavior becomes more exploratory, as bot traffic patterns change, or as interface modifications alter how users interact with content.

Detection requires monitoring annotation consistency rather than feature distributions. Inter-annotator agreement metrics like Cohen's kappa[16] (κ) provide quantitative assessment. Let p_o represent observed agreement between annotators and p_e represent agreement expected by chance. Equation 4.2 defines the statistic:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{4.2}$$

Monitoring κ over time windows reveals degradation trends. A medical imaging annotation project might establish a baseline $\kappa = 0.85$ (almost perfect agreement) during initial data collection,

15 **Label quality drift**: Degradation in annotation reliability over time, distinct from distribution shifts in the data itself. This drift type is invisible to standard feature monitoring because the features remain stable while the labels degrade, as annotator fatigue, pool turnover, or guideline evolution silently corrupt the ground truth the model learns from. Detection requires monitoring inter-annotator agreement (κ) over rolling time windows and comparing automated labels against periodic expert audits.

16 **Cohen's kappa**: Introduced by Cohen (1960) to measure inter-rater agreement while correcting for agreement expected from the raters' class marginals, which raw percentage agreement ignores. If two independent annotators each label 90 percent of images as "not spam" in a binary task, their expected agreement is 82 percent, making raw agreement potentially misleading. The statistic is denoted κ; interpretive bands such as the Landis-Koch categories are descriptive heuristics rather than universal data-quality thresholds (Landis and Koch 1977).

then observe decline to $\kappa = 0.72$ (substantial agreement) after six months as new annotators join without receiving equivalent domain training.

For systems with calibrated model probabilities, predictive entropy provides an additional uncertainty signal. Let $\hat{y}_i$ represent the model's probability assigned to label category i, and let log denote the natural logarithm, so entropy is measured in nats. Equation 4.3 defines this measure:

$$H_{\text{pred}} = -\sum_{i} \hat{y}_i \log \hat{y}_i \tag{4.3}$$

Rising predictive entropy indicates greater uncertainty in the model's predictive distribution, but it does not identify the cause. Harder inputs, distribution shift, calibration changes, or inconsistent supervision can produce the same signal.

Mitigation strategies depend on root cause analysis. Annotator retraining addresses systematic errors from unclear guidelines at low cost with high effectiveness. Multi-annotator voting with majority or consensus rules provides high accuracy for high-stakes domains but significantly increases annotation costs. Model-assisted labeling reduces annotator fatigue but risks introducing bias if the assisting model has its own systematic errors. Expert review sampling, where domain specialists audit a random sample of annotations, enables root cause analysis when quality decline is detected but provides medium coverage of the overall annotation stream.

Operationalizing the PSI and KL divergence metrics requires connecting them to automated alerts and review workflows. Data engineering is responsible for defining domain-specific drift thresholds from baseline behavior and outcome evidence, selecting monitoring windows that can expose both sudden and gradual changes, and instrumenting pipelines to compute these metrics continuously.

Statistical drift detection identifies distribution changes over time, but monitoring alone does not keep a system running when upstream pipelines stall, malformed payloads arrive, or schema definitions change without notice. Maintaining service continuity under failure requires active reliability mechanisms: fault isolation, backpressure, and graceful degradation.

4.5.5 Reliability through graceful degradation

Reliability ensures systems continue operating when problems occur. Pipelines face constant challenges: data sources become temporarily unavailable, network partitions separate components, upstream schema changes break parsing logic, or unexpected load spikes exhaust resources. **Graceful degradation** means handling these failures through systematic failure analysis, intelligent error handling, and automated recovery strategies that maintain service continuity even under adverse conditions.

Systematic failure mode analysis for ML data pipelines reveals predictable patterns that require specific engineering countermeasures. Data corruption failures occur when upstream systems introduce subtle format changes, encoding issues, or field value modifications that pass basic validation but corrupt model inputs. A date field switching from "YYYY-MM-DD" to "MM/DD/YYYY" format might not trigger schema validation but will break any date-based feature computation. Schema evolution[17] failures happen when source systems add fields, rename columns, or change data types without coordination, breaking downstream processing assumptions that expected specific field names or types. Resource exhaustion manifests as gradually degrading performance when data volume growth outpaces capacity planning, eventually causing pipeline failures during peak load periods.

Effective error handling strategies ensure problems are contained and recovered from systematically. Intelligent retry logic for transient errors (network interruptions or temporary service outages) requires exponential backoff strategies to avoid overwhelming recovering services. A simple linear retry that attempts reconnection every second would flood a struggling service with connection attempts, potentially preventing its recovery. Exponential backoff, retrying after one second, then two seconds, then four seconds, doubling with each attempt, gives services breathing room to recover while still maintaining persistence. Records that still fail after their retries go to a **dead-letter queue (DLQ)**, the side store already met at the ingestion boundary, which holds them for later analysis and potential reprocessing without blocking the main pipeline (Kleppmann 2016). A pipeline processing financial transactions that encounters malformed data can route it to a dead-letter queue rather than losing critical records or halting all processing.

[17] **Schema evolution**: This failure mode arises from a lack of contract testing between upstream data producers and downstream ML consumers. While "loud" failures like a renamed column break explicit assumptions and cause immediate pipeline crashes, "silent" failures are more dangerous. A field changing type from integer to string can pass validation but corrupt feature logic without immediate detection.

In ML systems, dead-letter queues serve dual purposes beyond failure analysis. Production teams implement systematic review of DLQ contents to identify: (1) schema violations indicating upstream changes, (2) edge case patterns the model should handle, and (3) data quality issues requiring source system fixes. For example, a fraud detection system's DLQ revealed transactions from a new payment type the model had never seen, prompting targeted data collection and retraining rather than simply logging the failures. This transforms DLQs from passive error storage into active sources for identifying model blind spots and driving improvement.

Moving beyond ad-hoc error handling, cascade failure prevention requires circuit breaker[18] patterns and bulkhead isolation to prevent single component failures from propagating throughout the system. When a feature computation service fails, the circuit breaker pattern stops calling that service after detecting repeated failures, preventing the caller from waiting on timeouts that would cascade into its own failure.

Automated recovery engineering extends beyond simple retry logic. Exponential backoff, jitter, load shedding, and circuit breakers reduce request pressure on struggling services while allowing transient faults to recover; simply increasing timeouts can retain resources longer and worsen overload. Multi-tier fallback systems can provide degraded service when primary data sources fail, using cached or approximate features only when their age and quality satisfy a validated fallback contract. A recommendation system unable to compute user preferences from the past 30 days might fall back to preferences from the past 90 days if evaluation establishes acceptable recommendation quality with that representation. If no fallback meets the contract, the system rejects the request. Comprehensive alerting and escalation procedures ensure human intervention occurs when automated recovery fails, with sufficient diagnostic information captured during the failure to enable rapid debugging.

Retry logic, dead-letter queues, and circuit breakers serve as runtime error handlers in the dataset compiler. They catch malformed inputs and failing dependencies without halting the entire pipeline. With ingestion, validation, drift monitoring, and these handlers in place, the frontend delivers records that are well-formed and checked, yet still raw. Audio arrives at inconsistent sample rates, text carries varying encodings, and numeric features span incompatible scales. Training requires transforming these heterogeneous records into a uniform representation while guaranteeing that identical transformations execute during both training and serving.

[18] **Circuit breaker**: Named for its three-state behavior (closed for normal flow, open with faults blocked, half-open for a recovery probe) after the electrical safety device that interrupts current on overload. In ML data pipelines, the circuit breaker prevents a failing feature computation service from cascading timeouts through the entire serving path: once failure count exceeds a threshold, the breaker opens. The pipeline may use cached or default features only within a validated feature-age and task-quality contract; otherwise it rejects the request rather than waiting on a dead service.

4.6 Systematic Data Processing

The frontend hands over well-formed, validated records; processing turns them into the representations the model consumes. In the compiler framework this is the *optimization and lowering pass*, which rewrites a program into a form the target machine executes efficiently. The analogy has one sharp edge. A compiler pass must preserve program semantics exactly, whereas filtering or augmenting observations can change the learning problem itself, so each transformation needs an explicit contract stating what it may change and what it must hold fixed. The governing constraint is consistency: transformations shared between training and serving must maintain identical semantics, repeated operations must remain idempotent under retries, and data transformations must scale without severing lineage.

Sculley et al. (2015) identify data dependencies, changes in the external world, configuration debt, and missing monitoring as hidden technical debt risks in production ML systems. Training-serving inconsistency is one concrete form of this risk. Consider normalizing transaction amounts during training by stripping currency symbols and casting strings to floats, but omitting identical preprocessing during serving. This discrepancy introduces training-serving skew, degrading downstream model accuracy. In the KWS pipeline, the requirement is concrete: transformations must standardize across diverse recording conditions (varying microphones, background noise levels, sample rates) while preserving the acoustic characteristics that distinguish wake words from ambient speech. Shared feature extraction must also preserve the same semantics and fitted state across training and embedded serving paths, with numerical differences bounded by a validated tolerance for the deployment.

4.6.1 Training-serving consistency

The training-serving consistency challenge extends beyond applying the same code. It requires that parameters computed on training data (normalization constants, encoding dictionaries, vocabulary mappings) are stored and reused during serving. This requirement is the **consistency imperative**.

Definition 4.3: The consistency imperative

The Consistency Imperative requires equivalent transformation behavior and state across training and serving environments.

1. **Significance**: An unvalidated discrepancy between training and serving features can change the model's effective input and degrade prediction quality without a runtime error. Equivalent semantics, shared fitted state, and tested numerical tolerances therefore make preprocessing a stateful component of the model artifact itself.
2. **Distinction**: Unlike data quality, which focuses on the cleanliness of a single record, the consistency imperative focuses on the alignment of the entire transformation pipeline.
3. **Common pitfall**: A frequent misconception is that consistency is "fixed" by sharing code. In reality, it is a state synchronization problem: parameters computed on training data (for example, means, standard deviations) must be stored and reused during serving.

A model can accept a numerically valid feature vector even when it was computed under different rules. Pipeline parity tests detect that discrepancy; outcome evaluation establishes its effect on predictions. Designing from serving constraints backward allows the team to test equivalent preprocessing on the target device and bound numerical differences before deployment.

Checkpoint 4.3: Defensive processing

Training-serving skew is a common cause of silent production degradation.

- ☐ **Definition**: Can you define training-serving skew in terms of the training and live-request processing paths?
- ☐ **Mechanism**: Can you explain why recomputing normalization means independently in training and serving can make live data look unfamiliar to the model?
- ☐ **Invariant**: Can you explain how shared transformation logic and synchronized feature state reduce skew?

Data cleaning is the first place where consistency can be either enforced or broken. Raw data frequently contains missing values, duplicates, or outliers that degrade model performance. The key insight is that cleaning operations must be deterministic and reproducible: given the same input, they must produce the same output regardless of environment. This requirement shapes which cleaning techniques are safe to use in production.

Data cleaning encompasses deduplication, missing-value handling, and string normalization. For instance, a customer database might normalize "John Doe," "john doe," and "DOE, John" to a common display format. Normalization alone does not establish that records refer to the same person; identity resolution requires a stable identifier or a separately validated matching rule. Both the formatting and matching rules must be captured in code that executes equivalently in training and serving.

Outlier handling exposes a fundamental operational divergence between batch training and online serving. During training, corrupted records or extreme sensor glitches can be dropped so that a few extreme values do not dominate and destabilize parameter updates. At serving time, the response contract determines whether an invalid request may be rejected, routed to review, or answered with an abstention or request for another sample. Clamping features to bounded intervals (such as $[\mu - 3\sigma, \mu + 3\sigma]$) is appropriate only when that transformation has been validated for the task and uses the fitted training parameters. More complex outlier filters, such as density-based spatial

clustering or multivariate isolation forests, introduce severe consistency risks: if evaluated across concurrently batched serving queries, the classification of an individual query depends on what other queries happen to arrive in the same request window, violating request isolation.

The validation monitors in section 4.5.2 also check transformed outputs. A rise in nulls after a join, unexpected duplicate records, or a shifted feature distribution can expose a transformation defect even when the raw inputs passed their checks.[19] Comparing raw and transformed records across the same pipeline version helps localize the fault before changing the model.

Quality validation must enforce identical schemas and validation predicates across training and serving pipelines. While training environments can afford compute-intensive profiling passes over entire datasets, serving paths must execute lightweight assertion checks (such as type validation, range bounding, and required-field existence) within the request's latency budget. Mismatches between training-time validation and serving-time assertions allow corrupted inputs to reach the model undetected.

Transformation techniques convert data from its raw form into a model-ready representation, but the systems risk lies in the parameters those transformations learn. Common transformation tasks include normalization and standardization, which scale numerical features to a common range or distribution. For example, square footage and room count may differ greatly in scale; normalization places them on comparable numerical ranges, though their influence still depends on the model and learned parameters (Bishop 2006). Maintaining training-serving consistency requires that normalization parameters computed on training data be stored and applied identically during serving. Operationally, these parameters must be persisted alongside the model itself and loaded during serving initialization. The words *computed on training data* carry the second half of the split-boundary rule from section 4.1. If the mean and variance, or a category vocabulary, are fitted on the entire dataset before it is split, statistics of the test examples flow into the transformation the model trains through, and the test score no longer measures generalization to unseen data. The pipeline therefore fits every transformation parameter on the training split alone and applies the frozen result to validation, test, and serving inputs.

Beyond numerical scaling, transformations convert discrete inputs through categorical encoding, extract cyclical representations from timestamps, and construct derived spatial features. Categorical encodings must handle both the categories present during training and unseen categories encountered during serving. A robust pipeline builds the vocabulary index exclusively from the training split, serializes this dictionary with the model artifact, and maps unknown runtime tokens to an explicit out-of-vocabulary (`<UNK>`) index. Omitting this mapping causes runtime dimension mismatches or silent index collisions during online inference.

A health prediction model may receive raw GPS coordinates for each patient visit, but latitude and longitude alone do not directly represent access to care. An engineer might derive *distance to nearest hospital* as a candidate proxy for geographic access. Whether that feature predicts outcomes, improves the model, or introduces sensitive-location and socioeconomic proxies must be established with data and domain review rather than assumed from the transformation alone.

Domain-specific feature engineering synthesizes raw inputs into representations that expose informative signals directly to the learning algorithm (Kuhn and Johnson 2013). Instead of requiring the model to discover non-linear combinations from scratch, engineered features encode structural domain relationships. In a retail recommendation system, for example, pipelines compute the recency, frequency, and monetary (RFM) value of customer transactions (known as RFM analysis) to summarize purchasing trajectories into dense behavioral vectors.

Feature engineering can be a high-leverage activity because it changes what signal the model can see. In some workloads, well-designed features improve performance more than another round of algorithm selection or hyperparameter tuning. That leverage must be balanced against the consistency requirements of production systems. Every engineered feature shared by training and serving must have equivalent behavior in both environments. Production systems therefore implement feature engineering logic in shared libraries or coordinated pipelines rather than reimplementing it independently. Many organizations centralize those shared definitions in a feature store,[20] the storage layer that section 4.8.5 defines, to reduce inconsistency across environments.

In the KWS pipeline, audio recordings collected across crowdsourced contributors, synthetic speech generators, and acoustic field captures arrive with severe acoustic variability. The raw audio

[19] **Statistical drift detection**: The means, variances, and quantiles tracked in quality monitoring are early-warning signals for the degradation equation's divergence term $\mathcal{D}(P_t \| P_0)$. A mean session length shifting from eight to 12 minutes over six months may indicate drift; a sudden drop to three minutes may indicate a collection fault. Outcome checks and source investigation are needed before choosing retraining or a source-system fix.

↪ The cost of training-serving skew and the shared feature libraries and schema validation tests that enforce training-serving parity are operationalized in section 14.2.1.3.

[20] **Feature store**: Shared code alone does not close the gap, because training features may be computed in batch (for example, seven-day rolling averages over historical data) while serving features are computed in real time from a streaming source; even with nominally shared logic, timing and state differences can create systematic discrepancies. A feature store can reduce this risk by coordinating definitions, data sources, timestamp handling, and offline and online materialization, but correctness still depends on implementation, freshness, and synchronization.

contains the imperfections anticipated during problem formulation: environmental noise spanning quiet bedrooms to industrial floors, amplitude clipping from saturated analog-to-digital converters, gain discrepancies across microphone hardware, and inconsistent sampling rates. Standardizing these inputs while preserving the phonemic transients that distinguish wake words from ambient noise is essential to achieving the 98 percent target accuracy.

Quality assessment for KWS extends the general principles with audio-specific metrics. Beyond checking for null values or schema conformance, the validation pipeline tracks background noise levels (signal-to-noise ratio above 20 dB), audio clarity scores (frequency spectrum analysis), and speaking rate consistency (wake word duration within 500 ms to 800 ms). The pipeline flags recordings whose noise, duration, clipping, or distortion falls outside the scenario's acceptance rules for review. These thresholds screen the clean baseline slice. Valid noisy or unusually timed speech remains in a separately labeled robustness slice, consistent with the noisy data budgeted in the KWS design. Review distinguishes these coverage examples from corrupted recordings before rejecting a sample. These screens do not prove that a sample is useful, but they reduce the chance that known defects reach model development. Grounding quality checks at the source severs the early defect arcs traced in figure 4.1 before corrupted inputs compound into downstream evaluation failures and forced workflow restarts.

Transforming audio data for KWS involves converting raw waveforms into formats suitable for ML models while maintaining training-serving consistency. Raw audio waveforms (sequences of amplitude values sampled thousands of times per second) are high-dimensional. KWS pipelines often transform them into compact representations that emphasize the frequencies and temporal patterns most relevant to speech. Figure 4.10 aligns three views of an audio sample across a two-second window: the raw amplitude waveform, the corresponding time-frequency spectrogram isolating high-power acoustic energy around 500 Hz, and a compact 13-coefficient MFCC approximation. These standardized feature representations, typically Mel-frequency cepstral coefficients (MFCCs)[21] or spectrograms,[22] emphasize speech-relevant characteristics while reducing noise and variability across different recording conditions. While a raw short-time Fourier transform (STFT) produces hundreds of linear frequency bins per time window, Mel-frequency analysis aggregates these bins through non-linear triangular filterbanks aligned with human auditory perception, followed by a discrete cosine transform that compresses the signal into just 13–40 decorrelated coefficients. For edge devices, this step collapses data volume by over an order of magnitude, allowing audio feature frames to fit inside the tight tens-of-kilobytes SRAM budgets of embedded coprocessors.

4.6.2 Idempotent transformations

Reliability adds retry safety to these quality foundations. While quality focuses on what transformations produce, reliability also governs how safely they can be repeated. **Idempotency**[23] means that applying an operation repeatedly has the same effect as applying it once. This property proves essential for production ML systems where processing may be retried after failures, data may be reprocessed to fix bugs, or the same data may flow through multiple processing paths. Determinism is a related but distinct requirement: identical inputs and captured state should produce identical outputs.

In state-machine terms, an operation f is idempotent if $f(f(x)) = f(x)$. In storage systems, setting a register to an absolute value ($x \leftarrow 1$) is idempotent, whereas incrementing a counter ($x \leftarrow x + 1$) or appending a row to an unkeyed log is not: each re-execution mutates system state. Distributed data pipelines execute across preemptible workers and unreliable networks where timeouts and node crashes trigger automatic retries. If a stage appends transformed records to a target table, a worker that crashes after writing but before acknowledging completion will re-execute on retry, generating duplicate records. An idempotent pipeline enforces write semantics such as deterministic keyed upserts (insert if absent, overwrite if present) or atomic directory swaps, ensuring that repeated job attempts converge to the identical final dataset.

Managing partial pipeline failures requires explicit state demarcation between stages. Decoupling stages with persistent intermediate checkpoints ensures that each phase can be retried independently without re-executing upstream work. For long-running jobs processing terabyte-scale corpora, checkpointing state partitions (such as completed file shards or stream consumer offsets) bounds retry latency: a task failure after five hours of execution requires replaying only the uncommitted

[21] **MFCC (mel-frequency cepstral coefficients)**: This transformation achieves its compactness and noise resistance by applying mel-scale filtering, which selectively emphasizes the frequencies humans use to distinguish speech. This process reduces thousands of raw audio samples from a small time window (for example, 25 ms) into just 13–40 coefficients, the aggressive dimensionality reduction required for always-on, kilobyte-scale hardware. A parameter mismatch between training and serving creates feature skew and can degrade accuracy.

[22] **Spectrogram**: The short-time Fourier transform (STFT) computes this 2D representation by converting the one-dimensional waveform into a time-frequency image. This representation lets image-style ML models process audio, but it also creates a rigid dependency: a mismatch in STFT parameters (for example, a 25 ms vs. 30 ms window) changes the feature representation between training and serving and can degrade performance.

[23] **Idempotency**: From Latin *idem* ("the same") + *potens* ("having power"), literally "having the same power when applied again." In ML pipelines, idempotency enables safe retries after partial failures: a non-idempotent operation (for example, appending to a log) creates duplicates on retry. An idempotent operation leaves the same resulting state after one application or repeated applications.

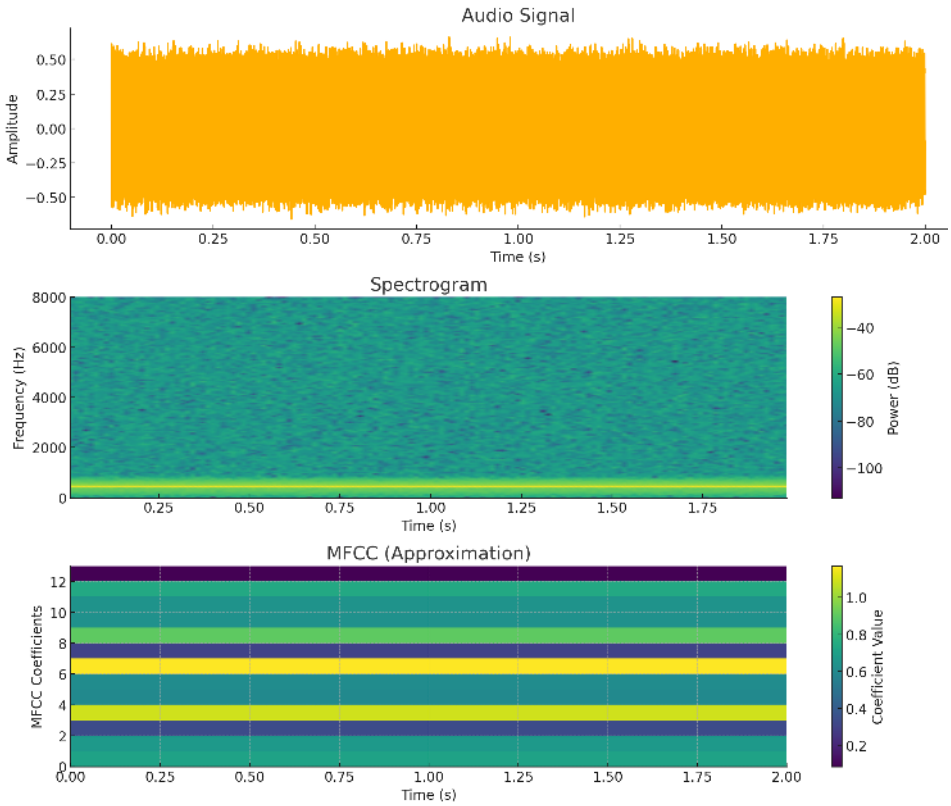


Figure 4.10: Audio Feature Representations: Three illustrative views show an audio waveform, its time-frequency spectrogram, and a compact MFCC approximation. The spectrogram encodes time, frequency, and signal power; the bottom panel arranges a smaller set of coefficients over time.

shard rather than restarting the entire five-hour job. The checkpoint mechanism must commit output partitions and state markers in a single atomic transaction, preventing records from being dropped or processed twice.

Deterministic transformations are those that always produce the same output for the same input, without dependence on external factors like time, random numbers, or mutable global state. Transformations that depend on current time (for example, computing "days since event" based on current date) break determinism because reprocessing historical data would produce different results. The solution is to capture temporal reference points explicitly: instead of "days since event," compute "days from event to reference date" where reference date is fixed and persisted. Random operations should use seeded random number generators where the seed is derived deterministically from input data, ensuring reproducibility.

Reliability in the KWS pipeline requires reproducible feature extraction. Audio preprocessing must be deterministic: given the same raw audio file, the same MFCC features are always computed regardless of when processing occurs or which server executes it. This enables debugging model behavior (can always recreate exact features for a problematic example), reprocessing data when bugs are fixed (produces consistent results), and distributed processing (different workers produce identical features from the same input). The processing code captures all parameters (fast Fourier transform (FFT) window size, hop length, number of MFCC coefficients) in configuration versioned alongside the code, ensuring reproducibility across time and execution environments. Even with rigorous design, production systems must implement runtime monitoring to detect skew if it emerges.

4.6.3 Distributed processing

Scale becomes the next constraint once quality and reliability are in place. Quality ensures transformations produce *correct* outputs; reliability ensures they produce *consistent* outputs. Neither matters if processing cannot keep pace with data volume or its deadline. As datasets and experiment concurrency grow, data processing can outgrow one machine even when the raw bytes still fit on local storage. The cleaning techniques that work on gigabytes in memory must then become partition-aware, out-of-core, or distributed without changing their semantics.

These challenges manifest when quality assessment must keep pace with incoming data, when feature engineering requires computing statistics across entire datasets before transforming individual records, and when transformation pipelines create bottlenecks at massive volumes. Processing must scale from development (gigabytes on laptops) through production (terabytes across clusters) while maintaining consistent behavior.

Single-machine processing suffices for surprisingly large workloads when engineered carefully. Modern servers with 256 gigabytes RAM can process datasets of several terabytes using out-of-core processing that streams data from disk. Libraries like Dask or Vaex enable pandas-like APIs that automatically stream and parallelize computations across multiple cores. Before investing in distributed processing infrastructure, teams should exhaust single-machine optimization: using efficient data formats (Parquet[24] instead of CSV), minimizing memory allocations, using vectorized operations, and exploiting multi-core parallelism. The operational simplicity of single-machine processing (no network coordination, no partial failures, simple debugging) makes it preferable when performance is adequate.

Within a single machine, parallelizing data ingestion across host CPU workers yields diminishing throughput returns whenever unpartitionable stages, such as sequential decompression or central database lock acquisition, limit concurrency. This is Amdahl's law[25] again, the same bound that capped an inference pipeline's end-to-end speedup when only the model stage got faster, now applied to host worker count rather than to one accelerated stage (equation 4.4):

$$\text{Speedup} \leq \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{N_{\text{workers}}}} \tag{4.4}$$

where f_{parallel} represents the parallelizable fraction of execution, $1 - f_{\text{parallel}}$ is the fraction spent in inherently sequential stages, and N_{workers} denotes active worker threads or processes on the host machine. Even when 90 percent of a feature extraction pipeline parallelizes across host CPU workers ($f_{\text{parallel}} = 0.90$), the maximum theoretical speedup cannot exceed $10\times$, because the serial tenth of the work still takes a tenth of the original time however many workers share the rest. Eliminating serial I/O and single-threaded stages therefore takes precedence over launching more worker processes.

Framework choice follows the same coordination question as the Amdahl analysis: it depends on which parts of the transformation can run independently and which parts need shared state or synchronization. For feeding accelerators during training, input pipeline engines such as PyTorch's `DataLoader` with multi-process workers or TensorFlow's `tf.data` optimize data loading through asynchronous prefetching, background worker processes, and pinned memory transfers. The key design decision is whether preprocessing occurs ahead of time offline or runs concurrently on host CPU threads alongside accelerator execution.

The feature computation placement trade-off introduced in section 4.5.1.3 takes on additional significance at scale. When multi-worker processing increases throughput, the cost of recomputing features across dozens of workers per epoch must be weighed against the storage cost of materializing those features once. At terabyte scale, even small per-example compute costs multiply into significant overhead, reinforcing why production systems adopt hybrid patterns: precomputing expensive, stable features while computing cheap, time-sensitive features on-the-fly.

4.6.3.1 Distributed cluster extension: The coordination tax

When datasets or concurrent experimentation exceed single-node memory and throughput envelopes, processing must scale across distributed clusters. Partitioning data across multiple physical machines, however, introduces network coordination challenges absent on a single host. Distributed coordination is constrained by network round-trip times: local memory operations complete in

[24] **Parquet**: A columnar storage format that organizes data by column, not by row. For the single-machine optimizations described, this is critical; instead of wastefully reading an entire CSV row to access a few columns, a Parquet reader loads only the specific columns needed for a computation. The resulting I/O reduction depends on the selected columns, encoding, compression, and workload.

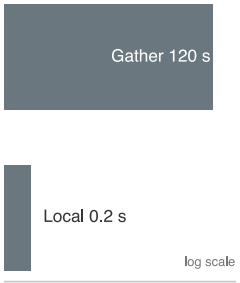
[25] **Amdahl's law**: Amdahl (1967) presented the serial-fraction argument at the 1967 AFIPS Spring Joint Computer Conference to explain why multiprocessor designs face diminishing returns. The original point, that the serial fraction of a workload imposes a hard ceiling on parallelism, applies directly to data pipelines: operations like computing global normalization statistics force serial aggregation phases that cap speedup regardless of how many workers process individual records in parallel.

↩ Amdahl's law, applied to an inference pipeline whose preprocessing and postprocessing cap the gain from a faster model, is established in section 1.11.

nanoseconds to microseconds, whereas network coordination requires milliseconds—a 1,000× latency difference. This constraint explains why operations requiring global coordination (such as computing normalization statistics across 100 machines) create severe bottlenecks. Each partition computes local statistics quickly, but combining them requires exchanging state across all partitions.

Data locality becomes critical at this scale. At 10 GB/s, moving one terabyte across the network takes about 100 seconds, while nodes that already hold their shard can scan it from RAM at 50–200 GB/s in seconds. That gap drives ML system design toward compute-follows-data architectures.[26] When processing nodes access local data at RAM speeds but must coordinate over networks limited to 1–10 GB/s, every cross-node exchange pays the bandwidth mismatch. Geographic distribution amplifies these challenges: cross-data center coordination must handle network latency (50–200 ms between regions), partial failures, and regulatory constraints preventing data from crossing borders. Understanding which operations parallelize easily versus those requiring expensive coordination determines system architecture and performance characteristics. This overhead is a **coordination tax**, the time machines spend exchanging data and waiting on one another rather than computing, and it limits distributed data processing. The name echoes the coordination tax of ad hoc team workflows in section 3.9.3, but the two costs differ. That one is paid in engineer time as a team grows; this one is paid in network time as a dataset spreads across machines. Whether centralizing or aggregating locally is faster depends on how much data each option moves relative to the local compute time, as Notebook 4.7 works out.

26 **MapReduce**: Designed by Dean and Ghemawat (2004) at Google to process the company's multi-petabyte web index across thousands of commodity machines. Its scheduler preferentially assigns map tasks to nodes that hold the input data, reducing input-network traffic. This locality-aware pattern influenced Hadoop and later distributed data-processing systems.



Local aggregation beats gather-all when network transfer dominates.

Napkin Math 4.7: The coordination tax

Problem: Mean normalization must be computed across 1 TB of features distributed across 100 equal-size partitions with equal record counts. Is it faster to (A) gather all data to one node and compute centrally, or (B) compute local means and aggregate them?

Option A (centralized):

- Transfer 1 TB at 10 GB/s network: 100 seconds
- Compute mean on single node: a running sum and count consume each record as it arrives, so the node never needs to hold the full terabyte; as a separate pass at RAM bandwidth (50 to 200 GB/s), the arithmetic would take ~5 to 20 seconds
- Total: ~100 seconds if the sum overlaps the transfer, ~120 seconds if it does not

Option B (distributed):

- Each node computes local mean: ~0.05 to 0.2 seconds (10 GB at RAM speed)
- Send 100 partial means (8 bytes each): < 1 ms of idealized payload transmission at the stated bandwidth; round trips, coordination, and stragglers add latency
- Aggregate: negligible
- Modeled compute and payload-transfer bound: ~0.05 to 0.2 seconds, before coordination overhead; the complete reduction requires measurement

Systems insight: Algebraically decomposable reductions can often use partial aggregation near the data. Averaging the partial means is valid here because record counts are equal; for unequal partitions, carry partial sums and counts and divide the combined sum by the combined count. The mathematical mean is unchanged, although floating-point reduction order can change rounding. Joins and cross-products may require shuffling, although partitioning and colocation can reduce that cost. Pipeline design should minimize unnecessary movement by pushing suitable computation toward the data, the compute-follows-data principle central to systems like MapReduce (Dean and Ghemawat 2004), Spark (Zaharia et al. 2010), and modern ML frameworks.

Scalability in the KWS pipeline manifests at multiple stages. Development uses single-machine processing on sample datasets to iterate rapidly. Training at scale may require distributed processing when the dataset (23.4 million examples) exceeds a machine's practical capacity or when experiments run concurrently. Per-file feature extraction is embarrassingly parallel, but end-to-end speedup is

bounded by shared storage bandwidth, metadata operations, scheduling, and output contention. Production deployment adds a stricter 16 KB preprocessing-state budget alongside the 64 KB model-size limit, necessitating careful footprint optimization to fit processing within device capabilities.

4.6.4 Transformation lineage

Governance completes the four-pillar view of data processing by ensuring accountability and reproducibility. The governance pillar requires tracking what transformations were applied, when they executed, which version of processing code ran, and what parameters were used. This **transformation lineage**[27] supports reproducibility, debugging, auditability, and iterative improvement when transformation bugs are discovered. It can supply evidence for documentation or compliance workflows, but lineage alone does not explain a model's decision or establish regulatory compliance.

[27] **Data lineage**: When a model produces an erroneous or discriminatory prediction, lineage helps engineers determine whether the contributing problem originated in raw data, feature computation, training, or serving. The resulting trace can support incident investigation and jurisdiction-specific documentation duties, but its usefulness depends on complete instrumentation and retained history; it does not guarantee an explanation of model behavior.

Transformation versioning captures which version of processing code produced each dataset. When transformation logic changes (fixing a bug, adding features, or improving quality), the version number increments. Datasets are tagged with the transformation version that created them, enabling identification of all data requiring reprocessing when bugs are fixed. This versioning extends beyond just code versions to capture the entire processing environment: library versions (different NumPy versions may produce slightly different numerical results), runtime configurations (environment variables affecting behavior), and execution infrastructure (CPU architecture affecting floating-point precision).

Parameter tracking maintains the specific values used during transformation. For normalization, this means storing the mean and standard deviation computed on training data. For categorical encoding, this means storing the vocabulary (set of all observed categories). For feature engineering, this means storing any constants, thresholds, or parameters used in feature computation. These parameters are typically serialized alongside model artifacts, ensuring serving uses identical parameters to training. Modern ML frameworks like TensorFlow and PyTorch provide mechanisms for bundling preprocessing parameters with models, simplifying deployment and ensuring consistency.

Processing lineage for reproducibility tracks the complete transformation history from raw data to final features. This includes which raw data files were read, what transformations were applied in what order, what parameters were used, and when processing occurred. Lineage systems instrument the pipeline so that each job records these edges as it runs, because a history reconstructed after an incident is rarely complete. When model predictions prove incorrect, engineers can trace back through lineage to identify the training data that contributed to the behavior, the quality scores attached to that data, the transformations applied, and whether the exact scenario can be recreated for investigation.

Recording the exact code version ties processing results directly to the repository state that generated them. When processing code lives in version control (Git), each dataset should record the commit hash of the code that created it. This enables recreating the exact processing environment: checking out the specific code version, installing dependencies listed at that version, and running processing with identical parameters. Container technologies like Docker simplify this by capturing the entire processing environment (code, dependencies, system libraries) in an immutable image that can be rerun months or years later with identical results.

The governance pillar in the KWS pipeline tracks audio processing parameters that critically affect model behavior. When audio is normalized to standard volume, the reference volume level is persisted. When an FFT transforms audio into the frequency domain, the window size, hop length, and window function (such as Hamming or Hann) are recorded. When MFCCs are computed, the number of coefficients, frequency range, and mel filterbank parameters are captured. This comprehensive parameter tracking enables several critical capabilities: reproducing training data exactly when debugging model failures, validating that serving uses identical preprocessing to training, and systematically studying how preprocessing choices affect model accuracy. Without this governance infrastructure, teams resort to manual documentation that inevitably becomes outdated or incorrect, leading to subtle training-serving skew that degrades production performance.

Clean, normalized, feature-ready data remains inert without a supervisory signal. That boundary matters operationally because a transformation can be perfectly reproducible yet attach an erroneous target, and deterministic preprocessing cannot compensate for corrupted ground truth. Supervising model training requires assigning target values, declaring which audio segments contain wake

words and which capture background noise, and that step introduces subjective human judgment and probabilistic heuristics into what had been a deterministic pipeline.

4.7 Data Labeling

The processing pipelines in section 4.6 transform raw data into structured features, but supervised learning still requires labels that tell the model which patterns correspond to each target. In the KWS pipeline, the ingestion and processing stages produce millions of clean, standardized audio spectrograms, but training requires ground-truth annotations indicating which segments contain the wake word and which represent ambient background noise. This declaration is the **ground truth**,[28] and producing it at scale is often the most human-dependent and error-prone stage of the pipeline. In the compiler framework, ground truth plays the role of the specification. Every later check of the compiled model is a comparison against it, so a labeling error does not stay in one stage but becomes part of the definition of correct. The specification also arrives out of order. A compiler starts from its specification, whereas a data pipeline often attaches ground truth after ingestion and processing, which is why a labeling error can slip past every check that ran before it.

Unlike automated transformations that can be parallelized across machines, labeling introduces human judgment into the pipeline, creating non-deterministic latency and stochastic error modes. A crowdsourced annotator might mislabel a whispered "Alexa" as background noise. An expert radiologist might disagree with a colleague about a borderline diagnosis. Such disagreement can reflect genuine ambiguity, unclear instructions, annotator error, or differences in expertise; the labeling system must distinguish and manage these causes rather than treating every disagreement as irreducible. The labeling infrastructure must therefore balance throughput, quality verification, cost, and provenance.

28 **Ground truth**: From remote sensing, where orbital measurements are verified by sending a team to the physical location (the "ground") to establish the "truth." The etymology carries a systems warning: ML labels are proxies for reality, not reality itself. When a crowdsourced annotator labels an image as "cat," that label reflects the annotator's judgment, not an objective fact. Every downstream metric (accuracy, precision, recall) is measured against this proxy, meaning label quality errors propagate silently into every evaluation of the model.

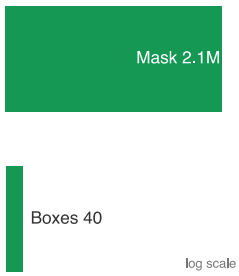
4.7.1 Label types and system requirements

System architecture, storage footprint, and loader throughput depend directly on label granularity. In a computer vision system tracking vehicles, pedestrians, and traffic signs from video feeds, each target representation imposes distinct storage, computation, and serialization constraints.

Classification labels represent the simplest form, assigning a discrete class index (or a multi-label binary bitmask) to each input. Storage requirements are modest, typically a fixed-width integer (such as a `uint16` or `int32` index) per image. Access patterns, however, diverge between training and evaluation: training pipelines shuffle and sample randomly, whereas validation pipelines stream sequentially across the corpus, driving different indexing and cache-locality strategies. A keyword-spotting label is of this kind. Each one-second clip carries a single class index naming which keyword it contains, or a reserved index for none. The label itself is tiny. The engineering burden sits upstream, in deciding where each clip starts and ends inside longer recordings, a problem section 4.7.4 returns to.

Bounding boxes extend beyond classification by localizing objects within each frame. Each bounding box stores four coordinates (such as $x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}$ or center coordinates with width and height) alongside the class identifier. Because scenes contain variable numbers of objects, label storage transitions from fixed-width scalar records to variable-length tensor structures, requiring dynamic padding or ragged-batch handling in the data loader. Moreover, positioning bounding boxes takes 10 to 20$\times$ longer than single-label classification, directly throttling annotation throughput and inflating labeling cost.

Segmentation maps assign a class identifier to every pixel, producing dense 2D label matrices ($H \times W$). In autonomous driving and traffic monitoring, this entails tracing irregular boundaries for driveable road surfaces, vehicles, and pedestrians. These dense annotations multiply storage and ingestion bandwidth. A segmentation mask for a 1920×1080 image requires about 2.1M labels (one per pixel), compared to perhaps 10 bounding boxes or a single classification label. If each box stores 4 coordinates, that is roughly 51,840× more scalar label entries than 10 boxes before accounting for per-value encoding, and the hours required per image for manual segmentation make this approach suitable only when pixel-level precision is essential.



Finer annotation granularity multiplies storage and processing scale.

Figure 4.11 arrays five annotation modalities across vision and speech examples, demonstrating how labeling granularity ranges from coarse scene tags and captions to spatial bounding boxes, dense pixel-level masks, and audio transcripts. Classification can label an entire scene, while

bounding boxes and segmentation maps localize objects or regions. Production datasets may combine modalities: a single camera frame might carry a scene label, obstacle boxes, and path-region masks, with each label type serving a distinct downstream task.

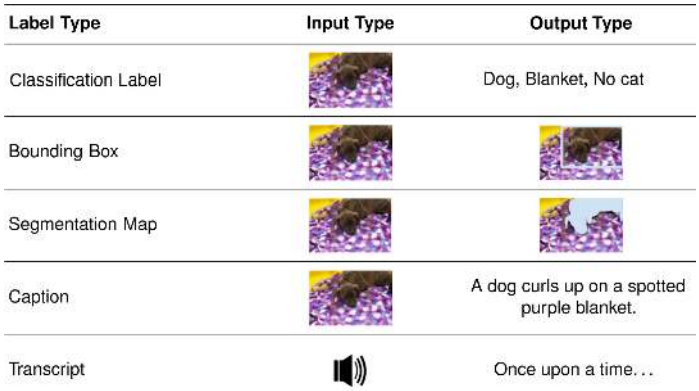


Figure 4.11: Data Annotation Modalities: Four rows show classification, bounding-box, segmentation, and caption labels for one image; the fifth shows a transcript label for an audio input. The appropriate annotation representation depends on the input modality and task.

Beyond these geometric labels, production systems must also manage rich metadata essential for quality control and debugging. The Common Voice dataset (Ardila et al. 2020) exemplifies this in speech recognition: tracking speaker demographics for fairness, recording quality metrics for filtering, and language information for multilingual support. If an object detection model fails in rainy conditions, weather metadata captured during collection pinpoints the coverage gap. This metadata requirement demonstrates how label type choice cascades through the entire system design: the infrastructure must optimize storage for the chosen format, implement appropriate retrieval patterns, and track which model versions used which label versions to correlate quality improvements with performance gains.

4.7.2 Label accuracy and consensus

Label quality requires bounding label noise despite inherent subjectivity and sensor ambiguity across large datasets. Even with clear guidelines and careful system design, a nonzero fraction of labels will inevitably be corrupted (Northcutt et al. 2021; Thyagarajan et al. 2022). The challenge is not eliminating labeling errors entirely (an impossible goal) but systematically measuring inter-annotator agreement and managing error rates to keep them within bounds that do not degrade model performance.

Labeling failures arise from two distinct sources requiring different engineering responses. Across the benchmark examples compiled in figure 4.12, including an ImageNet black stork mislabeled as a white stork and a CIFAR-10 frog mislabeled as a cat, each panel contrasts the erroneous assigned label against the verified ground truth. Some examples reflect degraded or ambiguous inputs where the correct label is difficult to infer from the data alone; others are visually clear but require domain knowledge, dataset-specific semantics, or expert judgment to label correctly. These different failure modes drive architectural decisions about annotator qualification, task routing, and consensus mechanisms: quality-based errors call for upstream data filtering, while expertise-based errors call for tiered annotator routing.

Given these inherent quality challenges, production ML systems implement multiple layers of quality control. Systematic quality checks continuously monitor the labeling pipeline through random sampling of labeled data for expert review and statistical methods to flag potential errors. The infrastructure must efficiently process these checks across millions of examples without creating bottlenecks. Sampling strategies typically validate 1-10 percent of labels, balancing detection sensitivity against review costs. Higher-risk applications like medical diagnosis or autonomous vehicles may validate 100 percent of labels through multiple independent reviews, while lower-stakes applications like product recommendations may validate only 1 percent through spot checks.



Figure 4.12: Pervasive Label Errors: Confidently mislabeled examples from widely used benchmarks, each showing the given label against the corrected ground truth (MNIST, CIFAR-10, CIFAR-100, Caltech-256, ImageNet, QuickDraw). Fine-grained confusions, such as a crab labeled lobster or a black stork labeled white stork, show that even curated datasets carry systematic errors, motivating careful quality control and expert annotation. Source: (Northcutt et al. 2021).

Beyond random sampling approaches, collecting multiple labels per data point, often referred to as "consensus labeling," can help identify controversial or ambiguous cases. Commercial labeling platforms such as Labelbox and Scale AI expose consensus and tiered quality-control workflows (Labelbox, Inc. 2024; Scale AI, Inc. 2024), but the statistical core is inter-annotator agreement. The consensus infrastructure typically collects several labels per example, computing metrics like Fleiss' kappa, a generalization of the Cohen's kappa statistic introduced in section 4.5.4 from two raters to any number of annotators (Fleiss 1971). Kappa summarizes agreement across a collection of rated examples; it is not a confidence score for an individual label. Per-example disagreement and failed gold-standard checks can route items to expert review. Landis-Koch bands are operational heuristics for interpreting aggregate agreement, not correctness guarantees (Landis and Koch 1977). Independent gold-standard checks and expert audits assess correctness because annotators can agree on the same systematic error.

The consensus approach reflects an economic trade-off essential for scalable systems. Expert review costs more per example than crowdsourced labeling, but forcing agreement on ambiguous examples through majority voting of nonexperts can produce systematically biased labels. By routing only genuinely ambiguous cases to experts, identified through low inter-annotator agreement or failed gold-standard checks, systems balance cost against quality. This tiered approach enables processing millions of examples economically while maintaining quality standards through targeted expert intervention.

While technical infrastructure provides the foundation for quality control, successful labeling systems must also address human factors. Working effectively with annotators requires standardized annotation rubrics with canonical positive and negative examples, visual edge-case catalogs, automated feedback mechanisms benchmarking annotators against known gold-standard records, and calibration sessions where annotators resolve borderline cases. For complex or domain-specific tasks, the system can implement tiered access levels, routing challenging cases to annotators with demonstrated domain expertise.

Quality monitoring generates substantial telemetry that must be efficiently ingested and tracked. The most informative signals span several dimensions. Inter-annotator agreement rates reveal whether multiple annotators converge on the same example, while label confidence scores capture annotators' reported confidence in their decisions. Time per annotation serves as a dual-sided indicator: annotations completed too quickly suggest carelessness, while those taking too long suggest confusion or unclear guidelines. Error patterns expose systematic biases or misunderstandings in the annotator pool, and annotator performance on gold-standard examples provides ground-truth calibration. Finally, demographic analysis of annotator behavior detects whether certain groups systematically label differently, which could introduce unintended bias into the training data. These metrics must be computed and updated efficiently across millions of examples, often requiring dedicated analytics pipelines that process labeling data in near real-time to catch quality issues before they affect large volumes of data.

4.7.3 Scaling with AI-assisted labeling

Throughput limits on manual annotation make pure human labeling a bottleneck for large-scale training, while fully automated labeling lacks the reliability needed for ambiguous or safety-critical examples. AI-assisted labeling navigates this trade-off by using model inference to process routine

inputs and accelerate human annotation while preserving manual verification for high-uncertainty samples. The decision hierarchy in figure 4.13 structures candidate solutions to this throughput bottleneck, branching from traditional human annotation into semi-supervised learning, transfer learning, and weak supervision via heuristics, distant supervision, and programmatic constraints. Traditional supervision uses direct human labels, with active learning selecting informative samples to reduce labeling volume. Semi-supervised learning exploits structure in unlabeled data alongside a labeled subset. Weak supervision replaces individual manual annotations with programmatic labeling functions. Transfer learning reuses representations learned on an upstream task, reducing but not eliminating task-specific labels. Each path alters where supervision cost is paid and which assumptions the pipeline must validate: unlabeled data geometry, programmatic heuristic accuracy, or representation transferability.

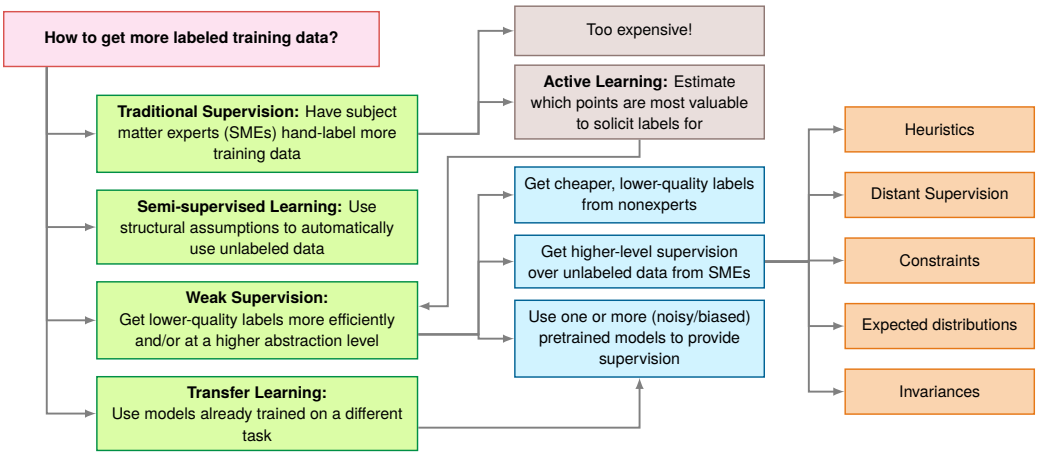


Figure 4.13: AI-Augmented Labeling Decision Hierarchy: A top-level question about obtaining labeled data branches into four paths: traditional supervision, semi-supervised learning, weak supervision, and transfer learning, with active learning as a cost-saving alternative. The branches distinguish where supervision comes from and what additional assumptions or pretrained artifacts the pipeline must validate. Adapted from the Stanford AI Lab weak-supervision overview (Ratner et al. 2019).

These paths exchange different assumptions rather than simple label counts. Depending on the path, the pipeline relies on task-relevant structure in unlabeled data, a transferable representation, measurable labeling-function accuracy, or an affordable scoring loop. Production systems must record each assumption and validate it against held-out, human-audited data. The diagram functions as a decision map, not an absolute quality ranking.

AI-assisted labeling divides work according to the strengths of each participant. Humans judge ambiguous cases, catch subtle errors, and apply domain knowledge; models can process routine cases at scale, subject to validation. Production systems combine these capabilities through several complementary approaches.

Pre-annotation uses AI models to generate preliminary labels that humans then review and correct, transforming the task from "label from scratch" to "verify and fix." Programmatic labeling frameworks like Snorkel (Ratner et al. 2018; Ratner et al. 2017) extend this further through weak supervision,[29] automatically generating initial labels at scale through rule-based heuristics, knowledge bases, and existing model outputs (for instance, a labeling function might use a regular expression to flag patient notes containing "adverse reaction" as high-risk). In autonomous driving, pretrained object detection models can label vehicles and pedestrians that human annotators verify and refine, handling many clear cases automatically.

Large language models (LLMs) now assist labeling pipelines by generating descriptions, drafting labeling guidelines from examples, and explaining their reasoning for label assignments. Content moderation systems, for instance, use LLMs for initial content classification with explanations that human reviewers validate. However, LLM integration introduces systems challenges: provider- and model-dependent inference costs and rate limits, as well as the need for systematic output validation because LLMs can produce confident but incorrect labels. Many organizations adopt

[29] **Weak supervision**: A "data programming" paradigm that exchanges some manual annotation labor for the upfront effort of writing and validating programmatic labeling functions. Once written, a function can be applied cheaply at scale, but execution, conflict resolution, quality monitoring, and maintenance still incur cost; the useful comparison is workload-specific rather than literally zero marginal cost.

tiered approaches, using smaller specialized models for routine cases while reserving larger LLMs for complex scenarios requiring nuanced judgment.

Active learning, which section 4.2.2 introduced as a way to raise the selection gain, makes the complementary trade-off: it spends model inference to reduce human labeling (Settles 2009; Coleman et al. 2022). Rather than labeling a random sample of the unlabeled pool, it selects the examples where the current model is most uncertain or where a label would most improve the model. The infrastructure must efficiently compute uncertainty metrics (often prediction entropy or disagreement between ensemble models), maintain task queues ordered by informativeness, and adapt prioritization strategies based on incoming labels. In a medical imaging pipeline, for example, active learning routes rare or borderline pathologies to expert radiologists while routine scans receive automated preannotations that clinicians quickly verify. This approach can substantially reduce required annotations in favorable settings, though it requires careful engineering to prevent feedback loops where the model's uncertainty biases which data gets labeled. Whether the trade pays depends on whether the scoring compute fits inside the same budget as the labels.

Napkin Math 4.8: When active learning breaks the budget

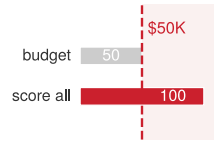
Problem: A 10M image pool has a $50K labeling budget. Labels cost $0.50/label in this scenario, above the simple-classification crowd range in table 4.5, as for a task whose labels need careful review. A random sample of the size the budget buys reaches 85 percent accuracy, and the target is 95 percent. Can active learning reach the target without exceeding the budget?

Physics:

1. **Sample efficiency**: Active learning can achieve target accuracy with fewer samples than random selection in favorable settings.
2. **Cost per point**: Random sampling = $0.50/label. Active learning adds compute cost (~$0.01/image for inference) to find hard examples.
3. **Options**:
   - **Random**: Reaching 95 percent may require 1M labels ($500K). Budget exceeded.
   - **Active labels only**: The system may need ~100K–200K hard examples ($50K–$100K), which reaches or exceeds the budget before any scoring compute.
   - **Full-pool scoring**: Scoring all 10M candidate images adds $100K, so total active-learning cost becomes $150K–$200K. Budget exceeded.

Systems insight: Algorithm choice is a major lever on label count, but compute must be inside the budget model. Spending 10 percent of this budget on inference ($5K) scores only 500K candidate images at the stated inference price and leaves room for about 90K labels. Active learning is viable only if the candidate pool is narrowed before scoring, inference cost drops substantially, or compute is funded separately from labeling. A configuration that fits therefore filters the pool first (by metadata or a cheap heuristic) to about 500K candidates, scores them for $5K, and spends the rest on about 90K of the most uncertain. That plan fits the budget, but it buys fewer labels than the ~100K–200K hard examples estimated above, so it reaches 95 percent only if measurement shows that fewer labels suffice for this task.

The budget result is the selection-gain ratio of equation 4.1 with its denominator priced in full. Active learning raises the task-relevant signal that each label buys, but scoring the pool is itself data movement and compute, and when that cost is charged to the same budget it can cancel the gain.



Full-pool scoring can exceed the label budget before labeling begins.

Quality control becomes increasingly important as these AI components interact. The system must monitor both AI and human performance through systematic metrics. Model confidence calibration matters: if the AI reports 95 percent confidence but achieves only 75 percent accuracy at that confidence level, preannotations mislead human reviewers. Human-AI agreement rates reveal whether AI assistance helps or hinders: when humans frequently override AI suggestions, the preannotations may be introducing bias rather than accelerating work. These metrics require careful instrumentation throughout the labeling pipeline, tracking both final labels and the interaction between human annotators and AI at each stage.

These principles manifest at scale across safety-critical domains. Autonomous vehicle labeling infrastructure can process large volumes of sensor frames, using AI preannotation to label common objects while routing unusual scenarios (construction zones, emergency vehicles) to human experts. In this distributed architecture, preannotation runs on GPU clusters while human review scales across annotation teams. Medical imaging systems face a parallel label-scarcity problem: large repositories of unlabeled clinical data make expert annotation the bottleneck and motivate data-efficient workflows that learn useful representations from unlabeled structure before expert labels are applied (Krishnan et al. 2022). Across such domains, the common data-engineering pattern is *tiered escalation*: automation handles clear cases, humans handle ambiguous ones, and monitoring ensures the boundary between "clear" and "ambiguous" adapts as both AI capability and deployment conditions evolve.

4.7.4 Automated labeling in KWS

Labeling keyword spotting datasets at scale introduces speech-specific systems challenges. Generating millions of labeled wake word samples without proportional human annotation cost requires moving beyond manual and crowdsourced workflows. MSWC, the reference corpus the acquisition strategy started from, shows how automated labeling resolves this throughput bottleneck: it was built by programmatically extracting over 23.4 million examples of one-second speech across 340,000 keywords in 50 languages from continuous speech corpora.

This scale makes manual annotation impractical: 23.4 million examples at even 10 seconds per label would require approximately 65,000 hours, roughly 32.5 person-years of full-time effort. Broad, documented sourcing across 50 languages can improve coverage, but language count alone does not guarantee representative speakers, accents, devices, or acoustic environments. The pipeline staged in figure 4.14 circumvents this manual annotation bottleneck, using forced alignment[30] over paired sentence audio and transcriptions to estimate word boundaries before slicing individual one-second keyword windows for classifier training.

[30] **Forced alignment**: Given a known transcription, an aligner estimates where its words occur in the audio, often using frame-level acoustic scores and sequence decoding. Timing resolution and accuracy depend on the model, features, language, and recording quality, so boundary estimates still need quality checks. The known transcript makes word-level segmentation far cheaper than labeling every clip manually, though computation, review, and error correction remain.

At this scale, quality control becomes a sampling and provenance problem rather than disappearing. The pipeline should report coverage by language, speaker, accent, device, and acoustic environment; retain the source recording, transcript version, aligner version, and boundary confidence for each extracted clip; and route low-confidence alignments and underrepresented slices to human review. Held-out evaluation must then measure the intended wake words across those slices. These controls distinguish a large corpus from a representative one and make extraction errors traceable when a downstream KWS model fails.

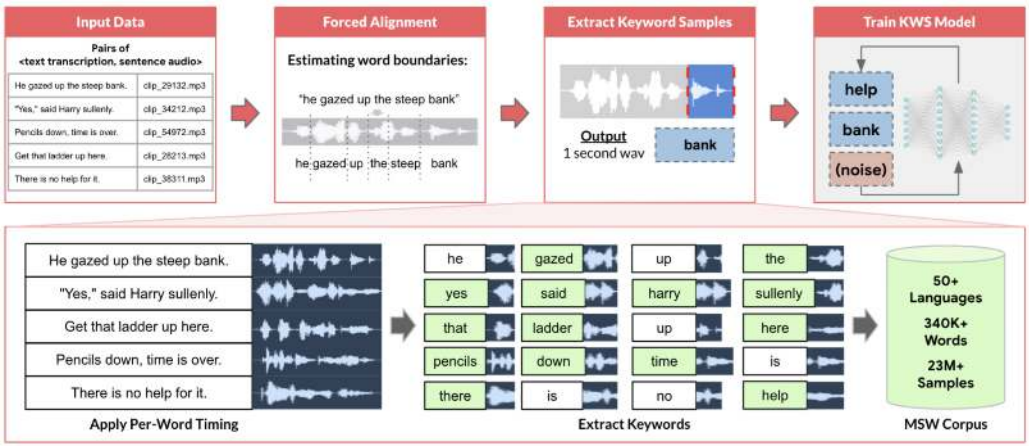


Figure 4.14: Multilingual Data Preparation: Forced alignment and segmentation transform paired audio-text data into labeled one-second segments, creating a large-scale corpus for training keyword spotting models across 50 languages. This automated process enables scalable development of KWS systems by efficiently generating training examples from readily available speech resources like Common Voice and multilingual captioned content. Source: (Mazumder 2021).

The extraction system uses these precise timing markers to generate clean keyword samples while handling anticipated engineering challenges: background noise interfering with word boundaries,

speakers stretching or compressing words unexpectedly beyond the target 500 ms to 800 ms duration, and longer words exceeding the one-second boundary. MSWC provides automated quality assessment that analyzes audio characteristics to identify potential issues with recording quality, speech clarity, or background noise, which is essential for maintaining consistent standards across 23.4 million samples without the manual review expenses that would make this scale prohibitive.

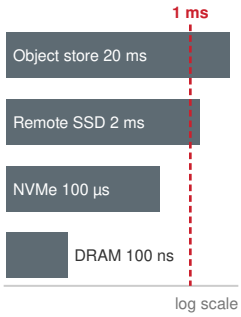
Voice assistant engineering teams often build on this automated labeling foundation. While automated corpora may not contain the specific wake words a product requires, they provide starting points for KWS prototyping, particularly in underserved languages where commercial datasets do not exist. Production systems typically layer targeted human recording and verification for challenging cases (unusual accents, rare words, or difficult acoustic environments), coordinating between automated processing and human expertise.

At this stage, the compilation pipeline has produced its core artifacts: feature vectors paired with ground-truth labels. Delivering these tensors to execution units exposes the physical storage hierarchy. The fundamental tension is mechanical: batch training demands high-throughput sequential streaming across petabytes of records, while real-time inference requires sub-millisecond point lookups for individual feature vectors. Storage architecture determines whether accelerators compute continuously or idle waiting on I/O.

4.8 Storage Architecture

The feeding problem has already priced one storage mismatch. On baseline cloud block storage, the simplified supply bound reduces the chapter's sustained ResNet-50 rate by about 44 percent (section 4.2.3). The KWS data poses the same question at two scales, the multilingual reference corpus (23.4 million samples spanning 50 languages) and the much smaller product-specific labeled subset. Both require deliberate storage layouts to balance training throughput against serving latency. Reconciling these divergent access profiles demands mapping workload access patterns directly onto the physical capabilities of underlying storage media.

In the compiler framework, storage plays the part of an operating system's program loader, which decides where compiled artifacts sit in the memory hierarchy and how quickly they can be brought to the processor that executes them. The framework data loader met during ingestion is a client of that placement, reading whatever layout storage provides, rather than the placement itself. Storage requirements for machine learning diverge fundamentally from those of transactional systems. Rather than optimizing for frequent small writes and point lookups that characterize banking or e-commerce, ML workloads prioritize sustained sequential read bandwidth, large-scale scans across gigabytes to petabytes, and schema flexibility. A relational database serving an e-commerce catalog handles millions of individual product queries per second with low latency, but an ML training job scanning that same catalog sequentially across dozens of epochs can stall when its access path issues many small reads instead of exploiting bulk scans.



A sub-millisecond point lookup fits only in DRAM or local NVMe.

4.8.1 Storage system options

Batch training scans millions of examples sequentially; real-time serving fetches one feature vector at a time. These opposing access patterns pull storage in two directions, and selecting a system means minimizing the data term $\left(\frac{D_{\text{vol}}}{\text{BW}}\right)$ of the iron law of ML systems for whichever pattern dominates. Every storage medium imposes physical constraints on bandwidth that determine the maximum speed of the training and serving pipelines.

Two storage performance metrics govern this optimization. **IOPS (input/output operations per second)** counts the distinct read or write requests a device can handle per second, bounding *random access* workloads such as fetching small batches of images or individual user profiles. **Throughput (bandwidth)** measures the volume of data transferred per second, typically IOPS $\times$ Block Size, bounding *sequential access* workloads such as scanning a Parquet file for training.

The choice between databases, data warehouses, and data lakes is fundamentally a choice about which of these metrics to optimize across distinct lifecycle stages. Conventional databases (online transaction processing (OLTP)) optimize for high IOPS with small block sizes and row-oriented layouts. They maintain product catalogs, user profiles, or transaction histories with strict ACID guarantees and sub-millisecond point-lookup latencies. In ML workflows, this profile fits online feature serving: a recommendation system looking up a user profile during real-time inference or an

anti-fraud service querying historical transaction counts requires random point lookups bounded by per-request latency. At the extreme, large recommendation systems serve billions of sparse feature reads per second from terabyte-scale tables, requiring storage engines that maximize IOPS over bulk throughput. A database becomes an inefficient training source when the selected access path moves more data than the job needs. In the row-oriented scan considered here, repeated passes over millions of records force the storage engine to fetch entire rows from disk into memory even when the model consumes only 20 of 100 features. The resulting uncoalesced I/O and wasted bus bandwidth throttle throughput.

Data warehouses (online analytical processing (OLAP)) invert this physical layout by organizing tables by column rather than by row (Stonebraker et al. 2018). By storing values of each attribute contiguously on disk, columnar formats allow sequential scans to read only the features required by the model, transferring one fifth of the uncompressed row payload when selecting 20 out of 100 columns. The format-efficiency calculation in section 4.8.2 quantifies this gain with a worked fraud-detection scenario. Columnar layouts also achieve high compression ratios across homogeneous data types, directly reducing the bytes transferred over the storage bus. Data warehouses excel for feature engineering, batch aggregations, and tabular model training where SQL interfaces simplify iterative data exploration. However, warehouses require structured schemas and struggle with multi-modal artifacts such as raw audio, high-resolution imagery, and free-form text. Modifying schemas during rapid experimentation imposes costly `ALTER TABLE` operations across billion-row tables, stalling development velocity.

Data lakes abandon predefined tabular schemas entirely by storing structured, semi-structured, and unstructured data in native formats on high-capacity object storage. They defer data structure definitions until query time, an architectural pattern known as schema-on-read.[31]

[31] **Schema-on-read**: Applies data structure definitions at query time rather than during ingestion, contrasting with schema-on-write (traditional databases) where data must conform to a predefined structure before storage. For ML pipelines in early development, schema-on-read enables rapid experimentation, because teams can store raw sensor data, images, and logs without committing to a feature schema upfront. The trade-off is governance: without enforced schemas, data lakes degrade into "data swamps" where finding and validating training data becomes the bottleneck instead.

This layout suits exploratory data engineering and multi-modal training pipelines where feature sets evolve continuously. A computer vision or multimodal system can store raw sensor logs as JSON, images as JPEG files, clickstreams as Parquet, and learned representations as binary NumPy arrays within the same repository. Independent training jobs impose schemas at read time, extracting different subsets without coordinating schema migrations. However, schema-on-read shifts governance from ingestion to retrieval. Without disciplined metadata management, data lakes degrade into "data swamps," disorganized repositories containing orphaned directory trees like `userdata_v2_final` and `userdata_v2_ACTUALLY_FINAL` whose lineage and quality are unrecorded. Production data lakes require a dedicated metadata catalog (such as AWS Glue Data Catalog, Apache Atlas, or Databricks Unity Catalog) to record lineage, update cadence, schema versions, and access policies.

Each storage architecture optimizes for a distinct access pattern and lifecycle phase, as table 4.7 summarizes.

Table 4.7: Storage System Characteristics: Each storage architecture favors a different access pattern and ML workflow stage. Databases support the high-IOPS random access needed for real-time feature serving; data warehouses provide sequential throughput for batch training and feature engineering; and data lakes provide schema flexibility and cost efficiency for large-scale raw data retention. A poor match between system and workload can impose substantial performance penalties.

Attribute	Conventional Database	Data Warehouse	Data Lake
Purpose	Operational and transactional	Analytical and reporting	Storage for raw and diverse data for future processing
Data type	Structured	Structured	Structured, semi-structured, and unstructured
Scale	Small to medium volumes	Medium to large volumes	Large volumes of diverse data
Performance Optimization	Optimized for transactional queries (OLTP)	Optimized for analytical queries (OLAP)	Optimized for scalable storage and retrieval
Examples	MySQL, PostgreSQL, Oracle DB	Google BigQuery, Amazon Redshift, Microsoft Azure Synapse	Google Cloud Storage, AWS S3, Azure Data Lake Storage

Production ML architectures rarely select a single storage technology. Instead, mature systems combine all three tiers into a unified data topology: databases handle operational telemetry and real-time point lookups; warehouses run structured feature extraction and aggregate analytics; and

data lakes store multi-modal raw captures and intermediate checkpoints. In an autonomous vehicle pipeline, for example, vehicle telemetry streams to an operational database for immediate health monitoring, aggregated driving statistics populate a warehouse for fleet analytics, and raw camera frames and lidar point clouds reside in an object store for model training.

4.8.2 Storage performance and cost

Physical storage media span roughly three orders of magnitude in cost and nearly four in bandwidth. Mismatching the storage tier to the access pattern either inflates infrastructure costs with underutilized capacity or starves training accelerators during I/O stalls.

Table 4.8 shows why ML systems use tiered storage. Under these 2024 assumptions, storing the KWS reference corpus (748.8 GB) in object storage costs $17.2/month, enabling affordable raw-audio retention, while working datasets on Non-Volatile Memory Express (NVMe)[32] cost $74.9/month–$224.6/month for active training but load 50× faster.

Table 4.8: Illustrative Storage Cost-Performance Trade-Offs (2024): These planning ranges vary by provider, region, configuration, and workload. Training data loading requires high-throughput sequential access. Online serving needs low-latency random reads, while archival storage prioritizes cost over access speed for compliance and historical data.

Storage Tier	Cost ($/TB/month)	Sequential Read Throughput	Random Read Latency	Typical ML Use Case
NVMe SSD (local)	$100–300	5–7 GB/s	10–100 µs	Training data loading, active feature serving
Object Storage (S3, GCS)	$20–25	100–500 MB/s (per connection)	10–50 ms	Data lake raw storage, model artifacts
Data Warehouse (BigQuery, Redshift)	$20–40	1–5 GB/s (columnar scan)	100–500 ms (query startup)	Training data queries, feature engineering
In-Memory Cache (Redis, Memcached)	$500–1000	20–50 GB/s	1–10 µs	Online feature serving, real-time inference
Archival Storage (Glacier, Nearline)	$1–4	10–50 MB/s (after retrieval)	Hours (retrieval)	Historical retention, compliance archives

[32] **NVMe (Non-Volatile Memory Express)**: A storage protocol connecting directly to the PCIe bus with 64K command queues, delivering 5–7 GB/s sequential throughput and microsecond-scale latency. The contrast with SATA SSD (500 MB/s, single queue) is a 10× bandwidth gap that can bottleneck a training pipeline when storage service time exceeds compute time.

↪ Why no single medium offers both large capacity and low access latency, the reason storage is built in tiers, is developed in section D.3.3.

This throughput divergence changes the input-loading component of engineering iteration time. Training that loads data at 5 GB/s completes dataset loading in 149.8 s, compared to 7,488 s at typical object storage speeds. The 50× loading speedup translates into shorter experiments only to the extent that exposed loading time contributes to their duration; transformation and model execution still consume time.

The latency gap between on-chip registers and remote storage spans over eight orders of magnitude, causing unbuffered remote I/O requests to stall GPU execution pipelines. Scaling these microsecond delays to human timeframes (table 4.9) illustrates the performance penalty of an uncached fetch. Popularized by Jeff Dean's 2009 LADIS keynote,[33] these figures quantify why unbuffered remote I/O starves modern accelerators and why distributed training demands strict data locality.

Table 4.9: Memory and Storage Latency Hierarchy: Standard hardware access times normalized to a human scale where a 0.5 ns L1 cache access represents one second. The internet row is a typical measured round trip, including routing and queuing, not the speed-of-light floor for the same distance.

Operation	Latency (ns)	Human Scale	ML System Impact
L1 Cache Reference	0.5	1 second	Immediate
L2 Cache Reference	7	14 seconds	Fast computation
Main Memory (DRAM)	100	3 minutes	The "memory wall" threshold
SSD (local NVMe)	100,000	2 days	Data loading bottleneck
Network (same DC)	500,000	11.57 days	Distributed coordination lag
SSD (remote network)	2,000,000	46.30 days	Remote-feature fetch stall
Object Store (S3)	20,000,000	1 year	Archival access
Internet round trip (CA to VA), measured	100,000,000	6 years	Global user experience

[33] **Jeff Dean**: Google Senior Fellow, architect of MapReduce, BigTable, Spanner, and TensorFlow. His 2009 LADIS keynote distilled the numbers in table 4.9 into the engineering heuristic that an L1 cache reference (0.5 ns) and a cross-country internet round trip (100 ms) span a 2×10^8 ratio, more than eight orders of magnitude, which explains why a training pipeline reading features from remote storage instead of local NVMe starves the accelerator it was meant to feed.

The internet row is worth comparing with the light-barrier calculation in section 2.2, which derived the speed-of-light round trip between California and Virginia. That physical floor is well below the table's 100 ms. The difference is routing, protocol, and queuing delay, which engineering can shrink, stacked on a floor that no engineering can remove.

Beyond input datasets, storage architectures must accommodate dense numerical arrays generated during model development: parameter weights, optimizer states, and training checkpoints.

Modern neural networks contain millions to hundreds of billions of parameters, imposing storage and retrieval patterns distinct from tabular data. GPT-3 (Brown et al. 2020) requires approximately 700 GB for model weights when stored in FP32 format (175B parameters times 4 bytes), though practical deployments often use smaller numeric formats such as FP16 (350 GB) to halve memory pressure. Even at FP16 precision, this single checkpoint exceeds many organizations' entire operational databases. From AlexNet's 60M parameters (Krizhevsky et al. 2012) to GPT-3's 175B parameters (Brown et al. 2020), model size grew approximately 2916× over eight years. Model weights require block-aligned storage formats to support parallel reads across parameter groups during initialization and checkpointing, demanding aggregate storage bandwidth approaching network fabric limits (25 to 100 Gbps) to keep compute units active.

↪ Quantization, which shrinks these weight footprints by storing parameters at lower precision, is developed in section 10.4.

Tracking these artifacts introduces versioning challenges. Git's incremental text diffs fail on multi-gigabyte binaries, where a single weight update produces an entirely new file and ten historical checkpoints of a 10 GB model occupy 100 GB. Production workflows therefore keep lightweight content hashes in Git and the binaries in content-addressed object storage, the same decoupling that section 4.8.4 applies to datasets.

Multi-accelerator training compounds storage pressure through concurrent I/O. When a training job spans multiple accelerators, every rank must read training batches, write optimizer snapshots, and dump intermediate checkpoints concurrently. Storage systems must absorb simultaneous reads and writes at rates proportional to the worker count, preventing I/O serialization that leaves expensive accelerators idle.

↪ The data- and model-parallel strategies that coordinate these workers within one node are developed in section 8.8.1.

Each storage tier sets its own ceiling on the feeding problem of section 4.2.3. Host RAM delivers 50 to 200 GB/s on modern server nodes, local NVMe SSDs deliver 1 to 7 GB/s, and network-attached storage provides 1 to 10 GB/s. Whichever tier serves the training reads caps the data supply rate, so the design question is which tier can sustain the chapter's ResNet-50 scenario, and at which of its two rates.



Napkin Math 4.9: Storage bandwidth budget

Problem: What storage configuration keeps a reference accelerator from being data-starved while training ResNet-50? Section 11.6 explains why delivered throughput falls short of an accelerator's advertised peak; here, the point is how a compute ceiling becomes a data-path budget.

Start with the compute ceiling. The reference accelerator can deliver 312 TFLOP/s on dense FP16 operations. ResNet-50 costs about eight GFLOPs for each forward pass, and including the backward training pass raises the training-step cost to about 24.6 GFLOP per image. Section 8.4.3 derives that backward-pass cost; here, the combined per-image cost is the input to the storage budget. Dividing accelerator peak throughput by per-image step cost yields a compute ceiling of 12,682 img/s.

That compute ceiling becomes a storage requirement once each image must arrive from disk or object storage. Storage reads compressed bytes, so the budget uses the 150 KB JPEG rather than the decoded tensor: feeding the ceiling takes approximately 1.9 GB/s, while the 3,000 img/s that the chapter's CPU-decoded pipeline actually sustains takes 450 MB/s.

The storage options now have two concrete targets:

- S3 Standard delivers about 100 MB/s per thread, so feeding the ceiling needs 19 concurrent worker threads before software overhead.
- SATA SSDs deliver about 500 MB/s sequentially, enough for the sustained rate but a bottleneck at the ceiling.

- NVMe SSDs deliver approximately 5–7 GB/s, which covers the ceiling with headroom.

Systems insight: A SATA SSD caps the pipeline at 500 MB/s divided across 150 KB images, or approximately 3,333 img/s. That clears the sustained 3,000 img/s, so a faster drive buys nothing while CPU decode binds. Once hardware decode removes the CPU bound and faster kernels or a faster accelerator push the rate toward the ceiling, the same drive holds the $15,000 accelerator to 26 percent of it. Each fix to the slowest stage exposes the next, so the storage tier must be sized for the rate the pipeline is heading toward, not the rate it runs at today.

Designing for high-throughput training therefore starts by matching storage throughput to accelerator demand, the storage-tier form of the bound that section 4.2.3 priced as a feeding tax and section 4.5.1 traced to CPU decode workers.

The 500 MB/s figure represents effective SATA III sequential read throughput (the interface ceiling is 550 MB/s), and real-world random read performance with small files can be significantly lower. That caveat reinforces the general principle governing data pipelines: equation 4.5 bounds training throughput by compute capacity and the data supply rate defined in equation 4.6. Let R_{train}, R_{compute}, and R_{data} denote rates in samples per second; let $\text{BW}_{\text{storage}}$ be storage bandwidth in bytes per second, f_{overhead} the dimensionless fraction lost to overhead, and s_{sample} the bytes per sample. Under full overlap, the min-of-rates form corresponds to the $T_{\text{step}} = \max(T_{\text{compute}}, T_{\text{IO}})$ lower bound from section 4.5.1; with incomplete overlap, actual step time is higher.

$$R_{\text{train}} \leq \min(R_{\text{compute}}, R_{\text{data}}) \tag{4.5}$$

$$R_{\text{data}} = \frac{\text{BW}_{\text{storage}}(1 - f_{\text{overhead}})}{s_{\text{sample}}} \tag{4.6}$$

Once R_{data} is the smaller rate, additional accelerator compute cannot raise training throughput; the remedy must increase usable storage bandwidth or reduce bytes per sample.

When storage bandwidth becomes the limiting factor, teams must either improve storage performance through faster media, parallelization, or caching, or reduce the amount of data that must move. Large language model training may require processing hundreds of gigabytes of text per hour, while computer vision models processing high-resolution imagery can demand sustained data rates exceeding 50 gigabytes per second across distributed clusters. These requirements make data loading a systems-placement decision: framework data loaders parallelize I/O across asynchronous worker processes, using **double-buffering prefetch queues** to overlap the $D_{\text{vol}}/\text{BW}_{\text{storage}}$ data fetch phase of batch $k+1$ in host RAM while the accelerator computes the $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ pass of batch k on GPU memory, hiding fetch latency only when the loader sustains the required supply rate and buffering absorbs delivery variation. Framework loaders can also move expensive augmentation work closer to the accelerator rather than storing every augmented variant.

File format selection directly scales the data term $\left(\frac{D_{\text{vol}}}{\text{BW}}\right)$ of the iron law. Columnar formats like Parquet excel for tabular features by allowing column projection and pushdown filtering, whereas shard-based sequential formats like WebDataset or **TFRecord** pack millions of small binary objects (such as images or audio) into contiguous sequential streams. This sharding eliminates random disk seek overhead (L_{lat}), transforming slow random I/O into sustained sequential storage throughput ($\text{BW}_{\text{storage}}$). Quantifying this impact as *format efficiency* (η_{format}) provides a multiplier on effective bandwidth. While Parquet optimizes columnar layout on disk, in-memory representations face an analogous serialization bottleneck when passing data between ingestion engines, dataframes, and ML frameworks. The Apache Arrow standard solves this in-memory boundary by defining a language-independent columnar memory layout, enabling zero-copy data sharing across pipeline processes without expensive serialization and deserialization cycles.

↪ Illustrative CPU decoding throughput and deserialization cost across storage formats are developed in section B.1.2.

Napkin Math 4.10: Format efficiency

Storage formats determine how much "waste" data must move to retrieve the signal needed for training. Let η_{format} be the dimensionless fraction of bytes read that carry selected features, from 0 to 1. Equation 4.7 shows how that fraction scales physical bandwidth into usable pipeline bandwidth:

$$\text{Effective Bandwidth} = \text{Physical Bandwidth} \times \eta_{\text{format}} \tag{4.7}$$

Values below 1 mean the format spends part of the physical data path on irrelevant bytes; improving layout can therefore raise usable throughput without changing the storage device.

Scenario: Training a fraud model using 20 features from a 100-column table, assuming equal uncompressed byte size per column.

Row-oriented formats (such as CSV) must read all 100 columns to retrieve the 20 required features, giving a useful-byte fraction of 0.2 and wasting 80 percent of disk bandwidth. Conversely, column-oriented formats (such as Parquet) read only the selected columns, achieving $\eta_{\text{format}} \approx 1$ (ignoring metadata overhead) and delivering 5× higher effective throughput.

Systems insight: In this projection-only scenario, switching from CSV to Parquet provides the same effective read-throughput gain as a 5× faster data path.

Columnar storage formats such as Parquet or Optimized Row Columnar (ORC) achieve a 5× to 10× I/O reduction for typical ML workloads through two complementary mechanisms: column projection, and column-level compression exploiting homogeneous value distributions. Compression proves particularly effective for categorical features with limited cardinality: a country code column with 200 unique values across 100 million records compresses 20× to 50× through dictionary encoding, while run-length encoding compresses sorted columns by storing only value transitions. In combination, projection and compression achieve total I/O reductions of 20× to 100× compared to uncompressed row formats, accelerating training epoch times and shrinking storage footprints.

↪ How row and columnar layouts arrange records on disk, and how projection pushdown reads only the selected column chunks, are developed in section B.1.3.

Compression algorithms trade storage density against CPU decompression bandwidth. While gzip achieves higher compression ratios of 6× to 8×, Snappy achieves 2× to 3× compression but decompresses at 500 MB/s, roughly 4.2× faster than gzip's 120 MB/s. Reading a 100 GB dataset compressed with gzip costs 13.9 minutes of CPU decompression per epoch, while Snappy costs only 3.3 minutes. Over 50 epochs, this 10.6 minutes difference per epoch compounds to 9 hours of host CPU time. That CPU time lengthens training only when decompression sits on the critical path, as it does when host-side decode is the binding stage of the dataloader choke point; with spare host cores and prefetching, it hides behind accelerator compute. The trade reverses when storage bandwidth binds instead. Gzip's higher compression ratio means fewer bytes per sample cross the storage path, which raises R_{data} in equation 4.6, so the right codec depends on which stage binds. Snappy suits a CPU-bound loader, and a denser codec suits a storage-bound one.

Beyond format and compression, data partitioning governs retrieval efficiency by aligning storage layout with common access patterns. A recommendation system processing user interactions partitions data by timestamp and user demographic segments, allowing training runs to query recent temporal slices without scanning the full historical repository. Partitioning strategies interact directly with distributed training: range partitioning by user ID routes consistent user profiles to specific data-parallel ranks (workers that each train on a different slice of every batch), while random partitioning mixes records from the source pool across workers under the sampling policy. It does not remove correlations already present in that pool or prevent leakage between evaluation splits; speaker grouping and other independence constraints must be enforced separately. Granularity is critical: too few coarse partitions limit parallel worker concurrency, while millions of micro-shards induce excessive metadata overhead and degrade sequential disk prefetching. In multi-accelerator training, partition skew creates severe straggler bottlenecks: if seven workers in an eight-accelerator job finish loading their batch in 12 ms but one worker reads from an oversized partition and takes 180 ms, that single straggler delays gradient synchronization across the entire cluster. Balanced partitioning that matches worker read budgets is therefore an architectural prerequisite for high accelerator efficiency.

4.8.3 Storage across the ML lifecycle

Storage requirements evolve because each lifecycle stage demands a distinct access profile from the same underlying data. A dataset undergoes random sampling during exploratory data analysis, high-throughput sequential streaming during model training, and low-latency point lookups during production serving.

During initial development, iteration velocity and schema flexibility dominate raw throughput. The primary systems challenge is managing experiment revisions without exploding storage capacity: ten exploratory runs on a 100 GB dataset naively generate 1 TB of redundant copies. The metadata-pointer architecture introduced in section 4.8.2 decouples version tracking from file storage: tools such as DVC track lineage via small hash manifests while content-addressed stores deduplicate identical binary payloads. Governance policies reinforce this tiering by granting broad access to anonymized development datasets while gating sensitive production data behind encrypted, auditable access tiers.

During training, the access profile shifts toward sustained sequential throughput. Modern deep neural networks process massive datasets across dozens or hundreds of epochs, making storage bandwidth the determinant of hardware utilization. Scaling the chapter's ResNet-50 scenario to eight accelerators, each sustaining its per-device rate, gives 24,000 img/s and a sustained storage demand of 3.6 GB/s of 150 KB compressed images; staging decoded FP32 tensors instead would raise it to 14.5 GB/s. Storage tiers unable to sustain this rate idle the accelerators, inflating cluster operating costs. The feature placement trade-off (section 4.5.1.3) becomes critical: precomputing feature representations delivers a 30× storage reduction (150 KB raw inputs reduced to 5 KB embedding vectors), but introduces staleness risks when extraction models update.

Production inference reverses the optimization target from sequential bandwidth to low-latency random access. A recommendation service handling 10,000 req/s with a 10 ms SLA and 10 feature reads/request demands 100,000 IOPS. Meeting this random access rate requires in-memory key-value stores such as Redis, distributed caches, or purpose-built low-latency feature stores. Edge deployment imposes further physical boundaries: constrained flash storage, intermittent network connectivity, and non-disruptive model updates dictate tiered caching, where core models reside in local flash while reference data synchronizes asynchronously with cloud endpoints. In production, serving infrastructure must support atomic rollbacks and parallel A/B testing. These operational guarantees depend on strict provenance: every deployed model must trace back to the exact dataset snapshot from which it was trained.

4.8.4 Data versioning for ML reproducibility

Consider a production recommendation model whose click-through rate abruptly degrades after a scheduled weekly retraining, even though the application codebase has not changed. The engineering team must determine whether the drop stems from an unannounced feature change, a label distribution shift, or an upstream table corruption. Without an immutable link between the trained model artifact and its training inputs, diagnosing the issue requires days of speculative bisection. With data versioning, engineers diff the training dataset snapshots directly and isolate the root cause: an upstream pipeline backfill that silently skewed label frequencies.

File-level versioning engines like DVC (Data Version Control) implement this capability by pairing Git with content-addressed remote storage (Iterative 2024). Git tracks small text pointer files (`.dvc`) containing cryptographic hashes, while DVC synchronizes the underlying binary dataset to object storage (listing 4.3). Checking out an earlier Git commit restores the historical pointers; the paired DVC retrieval step shown in the listing restores the dataset contents matched to that code state.

Listing 4.3: DVC Workflow: Git-like semantics for data versioning. DVC tracks large data files alongside code commits, enabling exact reproduction of any historical training dataset through paired `git checkout` and `dvc checkout` commands.

```
# Add data to version control
dvc add data/training.csv
git add data/training.csv.dvc
git commit -m "Add training data v1"
dvc push  # Upload to remote storage

# Later: retrieve exact data for any historical commit
git checkout abc123
dvc checkout  # Restores exact data from that commit
```

While file-level versioning suits bulk artifacts like images and audio, tabular datasets require versioning over continuously mutating tables. Modern open table formats such as Delta Lake solve this by maintaining an append-only transaction log of all row insertions, deletions, and schema modifications (Armbrust et al. 2020). This log provides time-travel capabilities, allowing training queries to pin a dataset state by timestamp or version ID (listing 4.4).

Listing 4.4: Delta Lake Time Travel: SQL queries identify a historical table state by timestamp or transaction-log version.

```
-- Query data as it existed on a specific date
SELECT * FROM training_data TIMESTAMP AS OF '2024-01-15'

-- Or by version number for programmatic access
SELECT * FROM training_data VERSION AS OF 47
```

Two enterprise components integrate with these versioning engines to establish end-to-end reproducibility. First, feature store point-in-time retrieval evaluates features as they existed at the observation timestamp, eliminating the leakage from future observations described in section 4.1. Second, a **model registry** records the lineage manifest for each deployed model: Git commit SHA, data version ID, training configuration, and evaluation metrics. In the click-through-rate regression described earlier, this audit trail turns an open-ended debugging investigation into an immediate snapshot comparison.

Long-term retention balances auditability against storage expenditure and data privacy regulations. High-volume production pipelines employ tiered retention: hot SSD tiers for active training and debugging, warm object storage for periodic evaluation snapshots, and cold archive storage for compliance records. Because indefinite retention of petabyte-scale data is economically unsustainable and frequently violates privacy statutes, data lifecycle policies must specify explicit retention horizons and purge mechanisms.

Physical storage architecture dictates where data lives and how fast it moves, but physical layout alone cannot guarantee semantic consistency between training inputs and inference features. Bridging offline batch training and real-time serving requires specialized feature infrastructure.

4.8.5 Feature stores

The fundamental challenge of production feature management is preserving semantic consistency across disparate execution environments: serving historical feature values for training and current feature values for inference. **Point-in-time correctness** is the primary operational requirement that drives teams to adopt **feature stores**, mitigating training-serving skew and enabling feature sharing across models.

In production environments, feature computation pipelines often fragment across languages and execution engines. During model development, data scientists compute features like `user_purchase_count_30d` using batch SQL queries in an offline data warehouse. During production serving, an online microservice computes the same metric incrementally from an in-memory cache. Even when intended to be identical, minor discrepancies in timezone arithmetic, missing value imputation, or floating-point rounding cause offline and online features to drift apart. Uber's

Michelangelo platform popularized the feature store to resolve this exact form of training-serving skew through centralized, reusable feature definitions (Hermann and Del Balso 2017).

Definition 4.4: Feature store

Feature store is the architectural layer that centralizes the management of machine learning features, decoupling feature computation from consumption.

1. **Significance**: It supports point-in-time-correct historical retrieval for training $(x_{t-\Delta})$ and coordinated feature definitions for real-time inference (x_t), reducing training-serving skew when timestamps, backfills, freshness, and synchronization are handled correctly.
2. **Distinction**: Unlike a general-purpose database, a feature store is designed for dual storage modes: an *offline store* (columnar/batch) for training and an *online store* (key-value/low-latency) for serving.
3. **Common pitfall**: A frequent misconception is that a feature store is only "a place to store data." In reality, it manages feature definitions, values, and retrieval; transformation may run in the feature store, an offline store, or a separate batch or stream-compute engine.

Feature stores (examined from an operational perspective in section 14.4.1.2) establish a single source of truth for feature definitions across the ML lifecycle. By registering a feature definition as a unified computation graph, the feature store evaluates the identical transformation across both historical batch data and live inference streams. This shared definition reduces one source of training-serving skew, but equivalent execution still requires synchronized fitted state, correct timestamps, and appropriate freshness. Those conditions must be tested across both retrieval paths. Centralization also enables organizational feature reuse: expensive representations, such as multi-dimensional user embeddings derived from months of interaction logs, are computed once and served to multiple downstream modeling teams.

↪ How feature stores are operated in production, including point-in-time retrieval and training-serving skew prevention at scale, is operationalized in section 14.4.1.2.

To reconcile opposing access profiles, feature stores implement dual storage engines. The offline store uses columnar formats (such as Parquet) residing on high-capacity object storage, optimized for multi-gigabyte sequential scans during batch training. The online store uses low-latency key-value databases (such as Redis), optimized for sub-10-millisecond point lookups during real-time inference. Synchronizing these two engines requires automated pipelines that push batch-computed feature snapshots to the online store, while streaming ingestion updates both stores continuously as new user events arrive.

Time-travel capabilities differentiate a true feature store from a simple cache. Training requires evaluating features as they existed at the exact moment of an event, rather than their current values. In a customer churn model, for example, training on a customer who churned on January 15 must use features computed on January 14, not current post-churn profile state. Feature stores enforce point-in-time correctness using an **as-of join** (temporal join): for every observation at timestamp t_{obs}, the retrieval engine joins the most recent feature record whose update timestamp satisfies $t_{\text{feat}} \leq t_{\text{obs}}$. The join must also exclude information that was not available to the prediction path at t_{obs}, including late-arriving events and later backfills whose event timestamps precede the observation. Timestamp ordering prevents future-information leakage only when retrieval preserves this historical availability across the upstream feature tables.

The performance characteristics of the dual stores directly bound training throughput and serving latency. The offline store must sustain batch read rates of millions of feature vectors per minute across wide schemas. The online store must sustain thousands to millions of point lookups per second while honoring single-digit millisecond latency budgets. When user actions (such as adding an item to a cart) require sub-second feature freshness, streaming feature pipelines update the online store continuously. This continuous updating requires careful event deduplication during network retries and timestamp synchronization to ensure late-arriving records do not overwrite newer state.

4.8.6 Accepting the KWS pipeline

Whereas enterprise recommendation and tabular services rely on centralized online/offline feature stores, embedded pipelines like keyword spotting manage localized feature extraction directly

between training clusters and memory-constrained microcontrollers. With storage layout, feature contracts, and versioning established, the candidate design introduced in table 4.4 can now be evaluated as a complete, unified pipeline. The team first records an acceptance contract: the 98 percent keyword-recognition target on the declared evaluation population, the expected false-wake budget below 1 false wake/month, the 64 KB model limit, and the separate 16 KB preprocessing-state budget. The contract also retains the latency and power requirements of section 4.3.3. Model bytes alone cannot establish a device fit; peak feature buffers and runtime state must fit the device's total memory allocation.

To test the choice of 13 rather than 40 MFCC coefficients, both candidates use the same versioned speaker-separated data splits, label budget, and recorded training procedure. Each candidate packages its own fitted preprocessing state with its model. Validation data selects the detection threshold; a separate, untouched evaluation set then measures keyword recognition and false rejection, while representative nonkeyword streams measure false accepts per hour through the deployed detection path. The team fixes the required statistical confidence and listening duration before evaluation and accepts the false-wake claim only when the upper uncertainty bound, expressed over the same listening duration as the target, meets that target. Insufficient listening time leaves the claim unestablished even if no false wake occurs. Replaying fixed audio through the training and device preprocessing paths checks feature parity, and measurements on the target device establish peak memory, latency, and power.

The budget ledger charges collection and review to the labeling allocation, and generation, retained versions, storage, and processing to their stated allocations. The computed 750K real examples fit only the labeling calculation; additional synthetic data must fit the remaining ledger. If the 13-coefficient candidate meets every acceptance condition, the larger representation needs a measured quality benefit worth its added device cost. If the smaller candidate fails quality, the 40-coefficient alternative is acceptable only if it closes that gap while passing the same resource and cost checks. If neither passes, the team revises coverage, representation, or the product requirements and repeats validation; it cannot declare success from the design-space gains alone. This procedure turns the initial allocation into a decision using the acquisition evidence, feature contracts, and storage costs developed through the pipeline.

Deploying an end-to-end pipeline across acquisition, ingestion, processing, labeling, and storage does not conclude data engineering. In production, input environments continuously drift: sensor hardware shifts, upstream schemas mutate, and annotator pools turn over. Without active operational maintenance, static pipelines quietly degrade over time.

4.9 Operational Data Health

A keyword-spotting (KWS) system deployed to edge devices launches with 98 percent accuracy and verified training-serving consistency. Six months later, production accuracy degrades to 94 percent without any modification to the model weights or architecture. The failure traces to three silent pipeline mutations: new smartphone microphone hardware altered the acoustic frequency response, the labels used to measure live accuracy followed revised guidelines that had not been reconciled with the earlier evaluation standard, and an upstream schema modification in user metadata was deployed without notice. The changed evaluation labels can alter measured accuracy even when predictions for the same inputs remain fixed; their effect must be separated from the changed input paths. Each failure mode represents a manifestation of **data debt**: accumulated, unmonitored compromises in data quality, schema integrity, and provenance that compound across system boundaries. Unlike traditional software defects that trigger segmentation faults or unhandled exceptions, data debt leaves the execution graph running normally while the underlying statistical distribution decays. Operating reliable production systems requires formalizing this debt: categorizing its sources, quantifying its accumulation against service-level objectives, and applying structured debugging protocols when production metrics degrade.

4.9.1 Categories of data debt

The edge KWS failure illustrates four distinct categories of data debt. The hardware-induced acoustic shift left the training set stale relative to the new microphones, which constitutes *freshness debt*; the divergent annotation standard introduces *quality debt*; and the silent metadata modification generates

both *schema debt* and *documentation debt*. While each category requires distinct diagnostic tooling and remediation protocols, they share a common failure mechanism: implicit coupling across unmonitored interfaces.

Definition 4.5: Data debt

Data debt is the compound interest of implicit coupling and missing documentation across an ML data stack.

1. **Significance**: It manifests as silent performance degradation and compounding maintenance overhead caused by unmanaged pipeline dependencies or undetected distribution shifts.
2. **Distinction**: Unlike a code defect, which often surfaces as a compiler error, a failing test, or a runtime exception, data debt permits computation to complete successfully while producing statistically invalid outputs. It is the data-side form of the hidden technical debt that Sculley et al. (2015) identify in ML systems, and it shares that debt's defining property of staying invisible until it is paid.
3. **Common pitfall**: A frequent misconception is that data debt is just "bad data." In reality, it is a systems architecture failure: it occurs when the assumptions of the training distribution are no longer enforced at the system boundary.

The first category is **documentation debt**: data provenance, semantic meaning, and collection parameters go unrecorded. When datasets omit datasheets or data cards (Gebru et al. 2021; Pushkarna et al. 2022), feature columns lack descriptive metadata, and transformation steps remain undocumented, pipeline maintenance collapses when team personnel change. As Sambasivan et al. (2021) demonstrate, missing metadata and unrecorded data collection practices trigger compounding data cascades across high-stakes systems. Downstream engineers must reverse-engineer feature semantics from legacy ingestion scripts, provenance audits fail regulatory compliance checks, and unstated assumptions, such as sensor dynamic ranges or unit conventions, trigger silent prediction anomalies when upstream instrumentation changes.

The second category, **schema debt**, emerges from accumulated transformation workarounds and uncoordinated migrations. When upstream services alter payload structures or field semantics, teams often apply localized patches, such as regex string extraction or fallback NULL coercions, rather than updating data contracts at the producer boundary. Over time, these patches create brittle parsing layers characterized by version-conditional branching, redundant timestamp parsers, and defensive type conversions. At the ingestion boundary, an unvalidated schema mutation can silently alter the assembled tensor rank, drop sparse features, or introduce out-of-vocabulary indices that collide with reserved embedding tokens. As established in the pipeline jungle analysis (section 4.3.2), upstream contract violations frequently masquerade as downstream model convergence failures or inference anomalies.

The third category, **quality debt**, consists of known data defects that remain uncorrected due to delivery deadlines or compute budgets. A training corpus harboring a 3 percent label error rate represents concrete quality debt: every training run on this corpus optimizes the model against corrupted targets. Quality debt compounds when model predictions serve as features or pseudo-labels for downstream models, amplifying label noise across dependent services. Telltale operational metrics include accumulated annotation discrepancies, unmitigated demographic representation skews, and unpurged near-duplicate records that artificially inflate validation scores through data leakage.

The fourth category, **freshness debt**, arises when static training sets diverge from evolving production distributions. A model trained on historical user behavior may encounter changed inputs or feature-label relationships after deployment. Calendar age alone does not establish a quality loss; freshness assessment needs evidence about changed conditions and their effect on predictions. Freshness debt accumulates invisibly because stale parameters continue executing matrix multiplies without hardware exceptions or runtime faults. Operational indicators include training-cadence deficits (where elapsed time since the last training run exceeds the distribution shift interval), cache

staleness in online feature stores, and static lookup tables that fail to reflect recent domain state. Managing these liabilities requires establishing explicit quantitative thresholds for every category.

4.9.2 Measuring and projecting data debt

Code-level debt can at least be approximated with code complexity metrics; data debt requires measurements of the data and its pipeline. Table 4.10 gives example warning and critical thresholds for each debt category, making data debt trackable as an engineering service-level objective (SLO) rather than presenting universal standards.

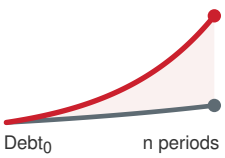
Table 4.10: Data Debt Metrics: Illustrative thresholds for detecting data debt accumulation across four categories. Warning thresholds indicate debt requiring attention in upcoming planning cycles; critical thresholds indicate debt requiring immediate remediation to prevent system degradation. The days-since-retraining row is a review trigger, not evidence of degradation. Crossing it prompts the comparison of elapsed time with the measured shift interval that freshness debt calls for. The PSI row uses common operational drift-monitoring bands, while the remaining rows should be calibrated to the risk tolerance and failure cost of the specific ML system.

Debt Category	Metric	Warning Threshold	Critical Threshold
Documentation	% datasets with data cards	< 80%	< 50%
Documentation	% columns with descriptions	< 90%	< 70%
Schema	Schema version branches	> 3	> 10
Schema	% transformations with try/catch	> 20%	> 50%
Quality	Known label error rate	> 1%	> 5%
Quality	Documented bias metrics	Not measured	Measured but not mitigated
Freshness	Days since last retraining	> 90 days	> 365 days
Freshness	PSI vs. training baseline	> 0.1	> 0.25

Data debt compounds through coupled feedback loops in production pipelines. Model outputs frequently feed downstream data lakes as pseudo-labels or synthetic features; any uncorrected label bias or corrupted prediction in an upstream model is thereby codified into the ground truth of dependent systems. Simultaneously, ingestion code degrades: each defensive workaround added to handle an undocumented schema mutation increases cyclomatic complexity, making subsequent schema migrations more likely to fail. As distribution drift steadily widens the divergence between the training baseline and live inference streams, unmaintained systems experience compounding maintenance overhead. To model the sensitivity of pipeline maintenance to unaddressed debt, measure debt by a concrete proxy, the remediation backlog in engineer-hours needed to bring every row of table 4.10 back below its warning threshold. Let Debt_0 be that backlog today, r its net growth rate per planning period (estimated from how the backlog grew over past periods), and n the elapsed periods. The planning model in equation 4.8 projects the accumulated liability:

$$\text{Debt}_n \approx \text{Debt}_0 \times (1+r)^n \tag{4.8}$$

Even modest differences in the periodic accumulation rate r yield divergent trajectories over extended operating horizons. This geometric formulation serves as a planning heuristic rather than an exact physical law; in production environments, debt accumulation is often punctuated by discrete upstream migrations rather than constant compounding.



Small differences in growth rate open a widening debt gap over time.

4.9.3 Remediation strategies

Quantifying data-debt metrics converts an open-ended maintenance deficit into an actionable engineering budget. Remediation can then be integrated into regular operational cycles before pipeline failures disrupt downstream services. Each debt category demands a distinct technical intervention, but all four require sustained capacity allocation rather than reactive triage.

For documentation debt, remediation centers on institutionalizing dataset metadata. Engineering teams can allocate scheduled maintenance intervals to document data provenance, tensor column semantics, and preprocessing graphs. Prioritizing high-fanout feature tables ensures that shared dependencies carry complete schema descriptions and transformation lineage.

Remediating schema debt requires preventive contract enforcement at producer boundaries. Explicit schema contracts between upstream logging services and downstream ingestion pipelines, codified using validation frameworks such as Pandera or Great Expectations, enforce strict type schemas, value bounds, and nullability constraints. Failing fast on schema violations prevents defensive null-coercion and string-parsing patches from accumulating within transformation pipelines.

Addressing quality debt requires structured workflows for correcting identified annotation errors, sample duplicates, and demographic representation gaps. Teams should maintain a tracked error backlog with quantified burn-down rates, integrating dataset curation into sprint planning alongside model architecture tasks rather than deferring it indefinitely.

Freshness debt requires automated drift monitoring coupled with an evidence-based retraining cadence. Integrating population stability index (PSI) and Kullback-Leibler (KL) divergence monitors (section 4.5.4) into operational dashboards enables pipelines to trigger investigation when distribution distance crosses a calibrated threshold. Outcome evidence and diagnosis then determine whether to refresh features, repair a pipeline, or retrain; an automated update policy must validate this decision chain rather than treat divergence alone as proof of accuracy loss.

Like technical debt in traditional software engineering, data debt is an inherent engineering trade-off. *Strategic* debt, such as accepting minimal documentation or delayed deduplication to meet an immediate deployment window, can be an intentional engineering choice. The failure mode is *unconscious* debt: compromises introduced without measurement that compound silently until remediation requires rebuilding the pipeline. Managing data health requires treating data debt as an operational parameter balanced against engineering velocity, serving latency, and compute budgets.

4.9.4 Debugging data pipelines

When data debt matures, it manifests as unexplained accuracy degradation, ingestion pipeline crashes, or subgroup performance disparities. Effective debugging isolates root causes across the data-machine interface: data cascades (section 4.3.1) demonstrate that upstream ingestion flaws trigger downstream anomalies, training-serving skew (section 4.6.1) reveals discrepancies between offline feature generation and online serving, and drift detection (section 4.5.4) identifies statistical divergence over time. Figure 4.15 organizes four symptom checks across temporal degradation, train-validation divergence, serving skew, and subgroup disparities. A positive answer identifies a candidate cause, not an exit from the investigation. Continue through the remaining checks because several faults can coexist, and confirm each cause with controlled artifact comparisons before selecting an intervention.

Production ML failures frequently trace to data dependencies and operational pipeline shifts rather than model architecture alone.[34] Prioritizing training-serving skew, distribution drift, and label noise isolates upstream input failure modes before initiating costly architectural redesigns or hyperparameter sweeps.

The KWS regression that opened this section needs three separate comparisons. Replay fixed raw recordings through the old and current preprocessing paths, with model weights and fitted state held fixed; changed features or predictions reveal a processing-path effect on those recordings, but do not establish microphone-population drift. Rescore fixed predictions under the old and revised labeling rubrics; a changed score identifies an evaluation-definition effect without proving that the detector changed. Finally, compare old- and new-microphone cohorts using one validated preprocessing path and one evaluation rubric, accounting for differences in speakers and acoustic conditions. A cohort gap warrants investigation of hardware coverage, but the microphone is not established as the cause unless those other conditions are controlled. Confirmed transform defects call for pipeline repair and revalidation; a remaining quality loss on representative new-microphone audio can justify targeted acquisition and a retraining comparison.

[34] **ML failure distribution**: Treat the data-dominance pattern as a diagnostic heuristic rather than a universal empirical distribution. Data and operational checks serve as initial triage hypotheses, but they do not eliminate code defects, hardware communication bottlenecks, or numerical instability.

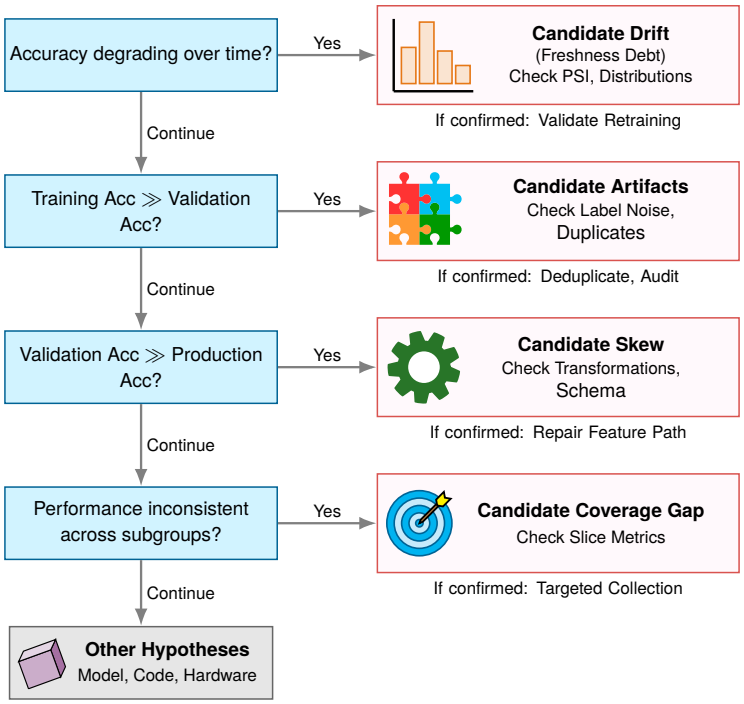


Figure 4.15: Data Pipeline Debugging Flowchart: Four sequential checks prioritize hypotheses involving drift, overfitting or data artifacts, training-serving skew, and subgroup coverage. Each side branch suggests a candidate cause. Continue through every check even after a positive answer, then confirm the causes before applying the indicated repairs. Other hypotheses include model, code, and hardware faults.

Checkpoint 4.4: From data contracts to diagnosis

A training loader delivers examples more slowly than the accelerator can consume them. A preprocessing version has also changed, and monitoring reports input drift, but outcome labels have not yet arrived.

- ☐ **Delivery**: Which rate limits steady-state training throughput, and what storage or transformation measurement would distinguish the cause?
- ☐ **Reproduction**: Which dataset version, preprocessing code, and fitted parameters must be recovered to compare the old and new paths?
- ☐ **Intervention**: What does the drift alert establish, and what additional evidence is needed before choosing pipeline repair or retraining?

4.10 Fallacies and Pitfalls

Data engineering pipelines rarely fail with immediate runtime crashes; instead, they silently distort feature distributions, corrupt training state, or starve expensive accelerator clusters. These fallacies and pitfalls examine the physical bottlenecks and architectural misconceptions that undermine production systems.

Fallacy: *More data always improves model performance.*

The bitter lesson says that general methods fed with more data and compute keep winning, but it does not say each new sample is worth as much as the last. Beyond a threshold, additional data yields diminishing returns described by empirical scaling laws, fitted curves relating model error to dataset size. Studies across computer vision, translation, and language modeling find that test loss falls as a power law in dataset size, so each successive error reduction demands a multiplicatively

larger volume of samples (Hestness et al. 2017). In systems terms, repeated examples consume disk I/O, host-to-device bus bandwidth, and accelerator FLOPs without adding distinct observations. Their gradients can still change as the model learns, so redundancy must be judged by marginal evaluation benefit rather than assumed to contribute no gradient. The task-relevant signal heuristic from section 4.2 captures this trade-off: scaling raw byte volume without preserving information density inflates infrastructure cost while yielding negligible improvements in evaluation loss.

↪ Scaling laws, and the selection methods that decide which samples are worth their training compute, are developed in section 9.1.

Pitfall: *Planning petabyte migration as a bandwidth-only transfer.*

Dividing data volume by network link bandwidth calculates theoretical wire time, but wire transfer is rarely the critical path. Data gravity (section 4.2) binds large datasets to a web of surrounding infrastructure: ingestion transformation pipelines, schema validation checks, access-control policies, and downstream consumers. Relocating a petabyte-scale corpus between clouds or data centers demands pipeline refactoring, data-integrity revalidation, cache re-warming, and metadata re-indexing. Consequently, the operational engineering time required to adapt dependent systems dwarfs raw network transit time.

Fallacy: *Data preprocessing can be finished once and left alone.*

Preprocessing pipelines convert raw observations into tensors using transformations parameterized on historical distributions: tokenization vocabularies, mean and variance scaling vectors, categorical frequency tables, and numerical clipping boundaries. When production distributions drift, these static transformations can become inappropriate for the task. Tokenizers encounter surging out-of-vocabulary rates, fixed normalization statistics push scaled feature values into out-of-distribution regimes, and static clipping bounds truncate dynamic range. Maintaining pipeline fidelity requires monitoring input tensors and outcomes, then validating any change to preprocessing together with the model that consumes its features. Recomputing statistics independently at serving time can itself introduce skew.

Pitfall: *Ignoring data serialization cost.*

Profiling often focuses exclusively on GPU kernel execution while data loaders quietly parse raw JSON or CSV files. Parsing text-based records requires host CPU cores to spend hundreds of cycles per sample on string decoding, tokenization, ASCII-to-float conversions, and heap allocations. This CPU serialization bottleneck starves accelerator compute engines, leaving GPUs idling on PCIe input transfers. Binary serialization formats, such as Apache Arrow or memory-mapped arrays, eliminate parsing overhead by storing numbers directly in machine-native binary representations, which the loader can then place in the pinned buffers that the dataloader choke point (section 4.5.1) relies on for DMA transfers across PCIe. For tabular schemas, columnar formats such as Parquet reduce unnecessary disk I/O by reading only required column projections. Benchmarking data ingestion against accelerator consumption rates reveals whether host CPU deserialization is throttling training throughput.

Fallacy: *High training accuracy indicates production readiness.*

Training accuracy reflects empirical optimization on historical training distributions, and held-out validation estimates generalization under identical sampling assumptions. Production environments introduce unmodeled edge cases, sensor noise, and live distributional drift. A model with high validation accuracy fails in deployment if upstream feature pipelines mutate or inference timeouts trigger fallback defaults. Diagnosing post-deployment regressions requires the structured symptom triage across temporal drift, label artifacts, and training-serving skew sequenced in figure 4.15, rather than reliance on a single aggregate offline metric.

Pitfall: *Ignoring training-serving skew until deployment.*

Training-serving skew occurs when feature generation logic diverges between the offline training pipeline and the online inference runtime. Training features are typically extracted in batch using offline scripts or database query engines. Online features must be evaluated within millisecond-scale latency budgets inside application serving paths. Discrepancies in floating-point precision, time-window aggregations, library versions, or subtle temporal leakage (such as incorporating data timestamped after the prediction point) introduce silent numerical divergence. Because the model still receives valid floating-point vectors, inference proceeds without raising runtime exceptions

while prediction quality degrades. Mitigating this pitfall requires shared transformation logic, reproducible feature stores, and automated predeployment parity tests that verify feature equivalence across offline and online paths.

Fallacy: *Synthetic data can fully replace real-world data collection.*

Synthetic data enriches training sets by generating rare edge cases, controlled physical variations, or privacy-compliant samples, but its utility is bounded by generator fidelity. Generative models and simulators sample only from distributions within the support of their parameterized assumptions. When trained exclusively on synthetic data, models overfit to generator-specific artifacts while remaining vulnerable to unmodeled real-world variance. In a KWS system, for example, a detector trained entirely on synthesized speech degrades when encountering authentic acoustic reverberation, diverse accents, microphone frequency response distortions, and non-stationary background noise. Synthetic generation augments empirical collection, but representative real-world observations remain mandatory for validating deployment viability.

Pitfall: *Neglecting data versioning until model debugging requires it.*

Treating training datasets as mutable object storage directories destroys experiment reproducibility. When a retrained model suffers a performance regression, an unversioned pipeline leaves engineers unable to determine whether the regression stems from a modified labeling rubric, silent schema migration corruption, or genuine distribution shift. Once dynamic tables are overwritten or raw source logs are purged, reconstructing historical training inputs becomes impossible. As established in section 4.6.4, robust data lineage requires that every model checkpoint bind immutably to a specific dataset snapshot, transformation code hash, and configuration profile. Deferring data versioning transforms standard root-cause analysis into speculative forensics over lost training state.

4.11 Summary

A versioned training dataset is compiled and consumed across training epochs; serving requests bring new inputs that must obey the same feature contract. The compilation half is the data-as-code principle (1): a changed dataset changes the trained model even when no algorithmic code changes, and a defect introduced in collection, schema, or labeling amplifies through every later stage as a data cascade. The consumption half is the physics of the iron law's data term. The pipeline must deliver bytes at the rate the accelerator consumes them, a requirement the feeding tax, the dataloader choke point, and the storage bandwidth budget each priced at a different tier of the same ResNet-50 scenario. Preparing data is therefore a data-machine co-design problem, in which storage hierarchies set the feeding rate, ingestion architectures trade freshness against operating cost, and feature contracts keep training and serving computing the same function.

Key Takeaways: Data is the source code

- **Data cascades make upstream quality the highest-return investment**: The compounding failure arcs in figure 4.1 show how early collection errors bypass intermediate testing, requiring extensive rework across downstream artifacts and, in severe cases, complete pipeline restarts from deployment. The four pillars (Quality, Reliability, Scalability, Governance) organize prevention, while documentation, schema, quality, and freshness debt require continuous remediation.
- **Data is code; version it, test it, review it**: Under the data-as-code principle (1), a dataset change is a code change. Give it version control, validation tests, and review.
- **Training-serving consistency is nonnegotiable**: Feature transformations shared by training and serving must have equivalent semantics and reuse the same learned state, such as normalization constants and vocabulary mappings.
- **Pipeline architecture choices have large cost implications**: The chapter's provisioning scenario prices streaming above batch, while its matched-retention example trades ETL storage savings against schema-change overhead. Actual costs depend on provisioning,

retention, replay, and required freshness. Select ingestion patterns by the value of latency, not the appeal of real time.

- **Labeling can dominate cost and requires substantial resource allocation**: Labeling can cost hundreds to more than a thousand times one optimized training run; in the reference calculation in section 4.4.2, the ratio is 521× to 1,562×. Labeling can remain a costly scheduling bottleneck even when annotation work is parallelized.
- **Storage hierarchy determines iteration speed**: The 50× throughput gap between local NVMe (5 GB/s) and cloud object storage (100 MB/s) changes input-loading time under the stated bandwidth assumptions. Total iteration time also includes transformation and model execution, so the loading ratio is not an end-to-end speedup.
- **The degradation equation becomes actionable through drift and outcome monitoring**: Divergence metrics such as KL measure changes between P_t and P_0, while labeled outcomes or validated proxies determine whether those changes correspond to model degradation.

The keyword spotter carried both halves through the chapter. Its one-false-wake-per-month target turned an accuracy goal into a per-window false-positive bound whose evaluation requires representative negative audio, sufficient listening duration, and an explicit treatment of measurement uncertainty. Its fixed budget turned "more data" into a priced allocation of labels, storage, and governance. Its MFCC extraction had to agree, within a validated tolerance, between a cloud training cluster and a kilobyte-scale always-on island. Data engineering is the discipline of keeping two contracts at once, a physical contract with the machine that must be fed and a statistical contract with a deployment distribution that keeps moving, and the failures this chapter traced, from a starved accelerator to a silently retyped ZIP code, each came from a pipeline that kept one contract while breaking the other.



What's Next: From source code to executable

The data pipeline now delivers a clean, versioned training set at the rate the accelerator consumes it, yet this chapter has treated the model at the end of that pipeline as a consumer of bytes and nothing more. Chapter 5 opens the consumer along the algorithm axis. It begins with the shift from hand-written logic to learned arithmetic, then counts the multiply-accumulate operations, activations, and gradient updates that turn prepared examples into learned behavior, and the memory and compute each one demands of the machine.

II

DEVELOPMENT AND TRAINING

Part II

Development Principles

Building a machine learning system requires more than assembling model components; it requires managing the flow of information and energy through silicon. Part I fixed where a system can run, when its constraints must be known, and how its data is delivered, and it established that data is both the program and a physical anchor of every ML system. With that anchor in place, Part II turns to the algorithm-machine interaction, asking *how* mathematical models are co-designed around the physical limits of the hardware that must execute them. The bitter lesson favors general methods that absorb more computation, but every such method must still fit the machine that runs it. The principles here begin with the accounting that determines why certain architectures succeed while others fail at scale. That accounting, the iron law, has been in use since the introduction; it is stated here as a principle because every chapter of this part prices its decisions against it.

Principle 3: The iron law of ML systems

Decomposition: Let T denote idealized wall-clock time in seconds. Here D_{vol} is data volume in bytes, BW is the effective bandwidth of the relevant memory or network path in bytes/s, O is total floating-point operations, R_{peak} is peak compute rate in FLOP/s, η_{hw} is dimensionless hardware utilization efficiency, and L_{lat} is fixed latency overhead in seconds, such as a kernel launch or network round trip. When data movement, compute, and fixed overhead execute serially, their idealized times add:

$$T \approx \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

The full treatment appears in section 1.7.

When data movement and compute overlap on modern hardware, wall-clock time follows the critical path and cannot be shorter than the slower of those stages plus any fixed latency that remains outside the overlap. The practical lesson is about dominance, not unconditional summation.

Implication: Optimization is rarely free of trade-offs. Reducing one term often shifts the bottleneck to another. For example, skipping zero-valued weights scattered irregularly through a model (unstructured sparsity) can reduce executed arithmetic. The model must then also store and read the positions of the surviving weights, however, and that index traffic can increase data-movement time $(D_{\text{vol}}/\text{BW})$. An optimization improves wall-clock time only to the extent that it shortens the critical path on the target hardware.

The iron law names the terms; it does not say which one a given model will stress. Chapter 2 examined the hardware's side, the terms each deployment tier offers. Choosing the model architecture makes the other side of the commitment, before any code runs.

Principle 4: The silicon contract

Heuristic: The hardware sets the terms of the bargain, a memory bandwidth BW, a peak compute rate R_{peak}, and a fixed overhead L_{lat}. Every model architecture and workload regime makes an implicit commitment against those terms, a wager on which resource it will saturate first.

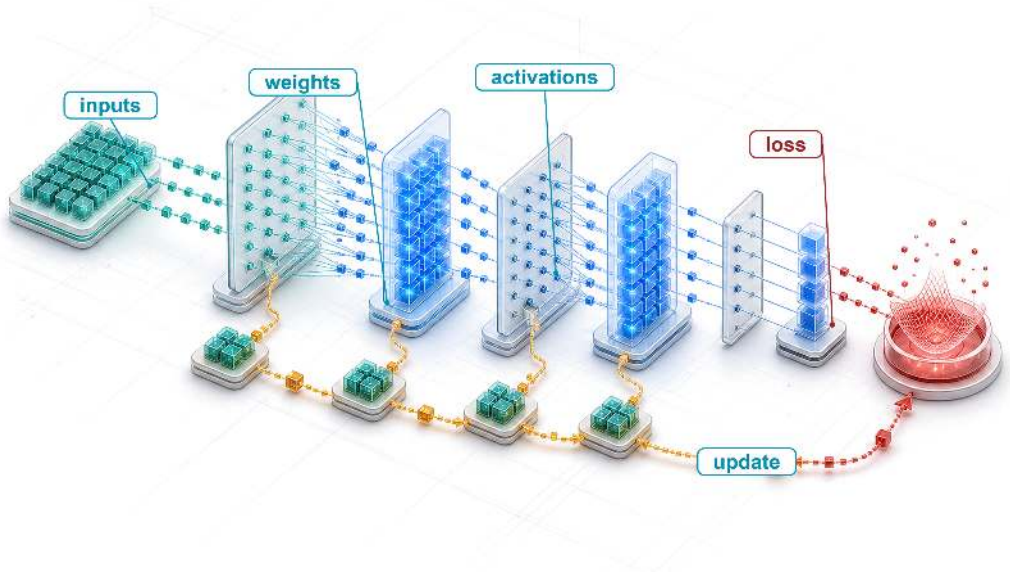
- **ResNet-50**: Assumes high-density floating-point compute. In batched training or inference on accelerators, it is often compute bound: performance is limited by $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$.
- **GPT-2/Llama**: Assumes high-bandwidth memory access during autoregressive decoding, which generates one token at a time and rereads every weight for each token. At small batch sizes, it is often bandwidth bound: performance is limited by D_{vol}/BW.
- **DLRM**: Assumes massive embedding tables and sparse lookups. It is shaped by both capacity and bandwidth: performance depends on whether embedding tables fit in the available memory hierarchy and how quickly sparse accesses can be served.

Implication: Designing a model without knowing which hardware resource it will saturate is like designing a bridge without knowing the strength of the steel. The design must target the bottleneck.

Together, the iron law and the silicon contract frame every design decision in Part II. Chapter 5 counts the operations, activations, and gradients that a neural model commits the machine to. Chapter 6 shows how arranging those computations sets a model's arithmetic intensity, and with it the architecture's side of the contract. Chapter 7 follows the translation of model operations into hardware execution and locates the fixed overhead that translation spends. Chapter 8 then repeats the resulting step many thousands of times, until memory, time, and cost budgets decide what one machine can train.

5

Neural Computation

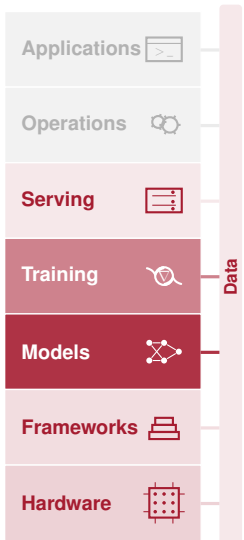


5.1 From Logic to Arithmetic
5.2 Computing with Patterns
5.3 Why Neural Computation Scaled When It Did
5.4 Neural Network Fundamentals
5.5 Learning Process
5.6 Inference Pipeline
5.7 Handwritten Digit Recognition
5.8 D·A·M Reading of Neural Computation
5.9 Fallacies and Pitfalls
5.10 Summary

Purpose

Why does understanding a neural network's math matter more than reading its code?

Neural network code can look deceptively simple because a few tensor operations hide the workload that every layer of the system must execute. Matrix dimensions determine arithmetic and data movement; intermediate values determine memory pressure; numerical ranges and gradients determine whether learning remains stable. An operation that looks cheap in isolation can dominate a system when repeated across a batch or when its outputs must be retained for learning, so source-code length gives little indication of resource cost. Doubling both input and output dimensions quadruples a dense layer's arithmetic, and keeping activations for the backward pass can multiply memory use several times over, while the code looks no more complicated than before. Two implementations of the same model can run very differently because batch shape, precision, and the target machine change what fits, how quickly it runs, and whether its calculations remain numerically valid. When training stalls or deployment exhausts memory, reading a framework call rarely identifies which resource is the binding constraint. A numerical failure, a memory bottleneck, and an inefficient implementation can produce similar high-level symptoms while demanding different remedies. Diagnosis and planning therefore require reasoning from the underlying operations to the bytes moved, values retained, and work repeated. This view can expose a mismatch before an expensive training run or deployment attempt does, when correcting it still costs an edit rather than a week of lost compute. Those primitives provide a stable vocabulary across architectures and make software abstractions auditable, turning opaque code into a workload that can be measured, diagnosed, and mapped to hardware. In D·A·M terms, neural computation is the interface where algorithmic structure becomes machine work and where physical limits feed back into model design.



Learning Objectives

- Explain why learned arithmetic replaced rule-based logic for high-dimensional pattern recognition
- Calculate parameter counts, multiply-accumulate operations, activation storage, and memory traffic for multilayer networks
- Compare activation functions by nonlinearity, gradient behavior, and hardware execution cost
- Apply cross-entropy loss and gradient updates to reason about supervised learning dynamics
- Derive forward and backward propagation as matrix operations over batches
- Analyze training vs. inference workloads using compute, memory, and deployment constraints
- Evaluate an end-to-end document-recognition pipeline from preprocessing through decision thresholds and human review

5.1 From Logic to Arithmetic

THE pipeline of chapter 4 ends with a clean, versioned training set, but examples do not learn on their own. The next system boundary is the model, the algorithm axis of D·A·M, where data becomes learned behavior through matrix multiplications, activation functions, and gradient updates. A model that runs correctly on one machine and fails on another is not necessarily suffering from a hardware defect. Its layer dimensions may create intermediate results that exceed the available memory. The failure arises from a mismatch between the model workload and the machine budget.

The silicon contract (principle 4) says that every model architecture wagers on which hardware resource it will saturate first. This chapter supplies the model's side of that wager. The architecture's mathematical operators fix the work to be done and the state to be held. Precision, data layout, implementation, and hardware then fix how many bytes actually move, and therefore the realized latency and energy. Honoring the contract requires understanding both the operators and how they map to the machine.

↩ The hardware's side of the silicon contract, the terms that memory bandwidth, peak compute rate, and fixed overhead set for any model, is established on the single-node stack in section 2.4.1.

Every mathematical operator in a neural network specifies a concrete computational workload on hardware. Neural computation shifts execution from explicit logical branching (if-then-else control flow) toward long sequences of continuous mathematical transformations, dominated by multiply-accumulate (MAC) operations. Every MAC operation performs one multiplication and one addition ($\text{acc} \leftarrow \text{acc} + w \cdot x$), counting as two floating-point operations (1 MAC = 2 FLOPs). This shift from *logic* to *arithmetic* creates dense tensor workloads whose execution throughput depends on how efficiently hardware can supply operands: whether execution is limited by memory transfer bandwidth or by arithmetic processing capacity. Such systems usually fail numerically rather than logically, as values grow or shrink until they stop carrying information. Even recognizing one handwritten digit with the small network this chapter builds requires 109,184 MACs (218,368 FLOPs) without executing a single logical branch in the forward pass.

Arithmetic without branches is not arithmetic without risk. When intermediate values exceed the dynamic range of a numerical format, hardware units trigger an exception or silently produce infinities (`Inf`) and invalid values (`NaN`), corrupting downstream tensors. The danger of unhandled numerical overflow is not unique to machine learning; safety-critical systems have failed catastrophically when numerical invariants were violated.

War Story 5.1: The overflow that became guidance (1996)

Context: The European Space Agency's Ariane 5 Flight 501 reused inertial reference software from Ariane 4, including numeric conversions whose assumptions matched the older vehicle's flight profile (European Space Agency 1996).

Mechanism: Ariane 5's flight profile caused a 64-bit floating-point value to exceed the range of a signed 16-bit integer during conversion, triggering an unhandled operand error.

Impact: The flight computer crashed 36 seconds after launch, causing the launcher to veer off course, break apart, and self-destruct, destroying the $370 million satellite payload.

Response: The inquiry recommended limiting software to functions required during flight, reviewing range assumptions, and designing exception handling so a single conversion failure could not disable redundant units (European Space Agency 1996).

Systems lesson: Numerical ranges, exception handling, and reuse assumptions are part of the systems contract. A computation can be syntactically correct and still be invalid for the physical regime in which it runs. ML systems hit this whenever a numerical format's representable range fails to match the magnitudes flowing through it: a low-precision multiplication that overflows produces an Inf or NaN that propagates silently through the rest of the computation, making numerical regimes as important to debugging as control flow.

Definition 5.1: Deep learning

Deep learning is the computational paradigm that learns layered feature representations from data by composing nonlinear transformations. For suitable tasks, those learned representations can reduce manual feature engineering while increasing computation (O) and state.

1. **Significance**: For suitable compositional function classes, depth can represent a hierarchy much more parameter-efficiently than a shallow network. Each layer adds work and state, and the resulting binding constraint may lie on the data, algorithm, or machine axis depending on the workload.
2. **Distinction**: Shallow methods such as logistic regression and support vector machines can work well with appropriate representations. Deep learning instead uses multiple nonlinear stages to learn representations jointly with the prediction task, which can reduce manual feature design and sometimes support transfer across tasks.
3. **Common pitfall**: A frequent misconception is that parameter count alone explains deep learning. Two networks with the same parameter count can differ sharply in what they learn, because architecture, data, optimization, and scale together determine the practical result.

Classical machine learning relied on domain experts to craft task-specific feature extractors, encoding domain knowledge into fixed, handcrafted representations. Deep learning eliminates manual feature pipelines by composing hierarchical layers of parameterized nonlinear transformations, learning representations and decision surfaces jointly from raw data (LeCun et al. 2015).[1]

This transition from procedural feature pipelines to end-to-end differentiable transformations introduces engineering failure modes that conventional software debugging cannot isolate. Deep learning failures produce subtle symptoms: gradient instabilities[2] that halt convergence, numerical precision errors that corrupt parameter state across millions of updates, or inefficient memory access patterns in tensor operations[3] that leave accelerator compute units underutilized. Resolving these failures requires reasoning about the physical execution of tensor algebra on hardware rather than stepping through procedural control flow.

Diagnosing and resolving these failures requires understanding how algorithmic formulations translate directly into physical hardware demands. Those demands depend first on representation, meaning whether patterns are expressed through hand-authored rules, engineered statistical features, or learned parameters.

5.2 Computing with Patterns

To understand how computing paradigms evolved, consider a canonical pattern-recognition benchmark: classifying a 28×28 pixel grayscale image of a handwritten digit from the MNIST dataset.

1 | **LeCun, Bengio, and Hinton**: Recipients of the 2018 ACM Turing Award for conceptual and engineering breakthroughs that made deep neural networks a critical component of computing. Their collective work advanced representation learning, network architectures, and training methods that underpin modern deep learning.

2 | **Gradient instabilities**: In a simplified 20-layer path through sigmoid (S-shaped) activations, the activation-derivative factors alone are bounded above by 0.25^{20}, or 9.1×10^{-13}. The complete gradient also includes weights and other operations, but repeated saturated derivatives can still make early-layer updates negligible. Plateaus or NaNs can reveal the symptom without identifying the cause; rectified linear unit (ReLU) activations and careful initialization (both taught later in this chapter), together with normalization, better optimizers, and residual connections, all helped make deep networks trainable, and section 6.10.2.2 treats residual connections in detail.

3 | **Tensor operations**: A tensor's logical dimensions map onto a flat physical layout, and a kernel may prefer a different ordering. If changing a typical ImageNet input (224×224×3, about 151 KB) from channel-first order (all values of one color channel stored contiguously) to channel-last order (the channel values of each pixel stored together) requires a materialized reorder, meaning an actual copy into a new buffer, it moves about 301 KB of read-plus-write traffic per image without changing the model's mathematical result. Some runtimes avoid the copy by reinterpreting the same buffer with new strides or by choosing a kernel that accepts the original order. When a reorder remains, it can consume part of a tight inference budget before the model's main arithmetic begins.

Across computing history, this single concrete task has been attacked through three fundamentally different representation strategies: hand-authored decision logic, engineered feature extractors, and learned neural representations. Classifying a single digit requires roughly one hundred hand-written comparisons under rule-based programming, several thousand arithmetic operations under classical feature extraction, and more than one hundred thousand multiply-accumulates in a modern multilayer neural network. Each paradigm arose because its predecessor hit a fundamental representation bottleneck. Tracing this single benchmark across all three paradigms reveals how representation choices govern arithmetic intensity, memory traffic, and hardware efficiency.

↩ The succession of logic, knowledge, feature-engineering, and infrastructure bottlenecks that these paradigms answered is established in section 1.3.

5.2.1 From explicit logic to learned patterns

Rule-based programming requires developers to specify the procedures that map inputs to outputs. Consider a simple game like Breakout.[4] The simulation logic diagrammed in figure 5.1 relies on explicit conditional branches: testing bounding-box intersections, removing the struck brick, and reversing the ball's velocity vector. Such deterministic rules work well for well-defined game physics, but hand-authoring an explicit policy for every variation in messy, unstructured real-world data does not scale.

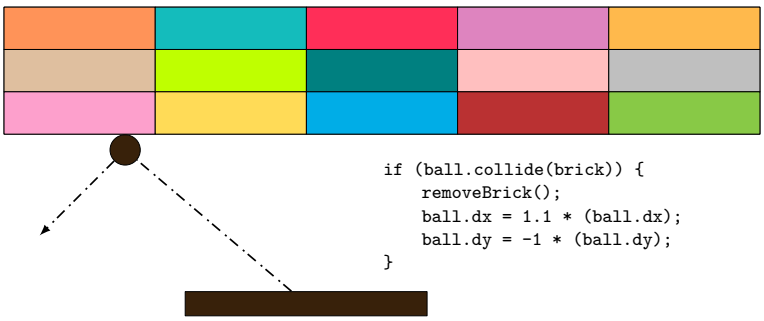


Figure 5.1: Breakout Collision Rules: The game program uses explicit if-then rules for collision detection, specifying ball direction reversal and brick removal upon contact. While effective for a game with clear physics and limited states, this approach illustrates how rule-based systems must anticipate every possible scenario.

[4] **Breakout (DQN - deep Q-network)**: Atari's 1976 arcade game became an AI milestone when DeepMind's DQN learned a gameplay policy from pixels (2015) without a hand-authored policy for each situation. The environment, action space, reward, and preprocessing were still programmed. DQN resized frames to 84×84, stacked 4 frames as input, and selected a new action every 4 frames, illustrating how learned control still depends on an engineered real-time inference loop.

In the classical pipeline diagrammed in figure 5.2, explicit human-authored rules route alongside input data into the program execution block, which deterministically computes the corresponding answers. Early artificial intelligence research explored whether such hand-authored symbolic rules could scale to complex reasoning and recognition tasks.

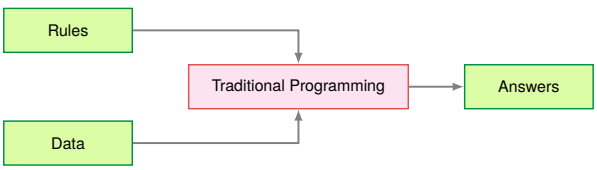


Figure 5.2: Traditional Programming Flow: Rules and data serve as inputs to a traditional program, which produces answers as output. This input-output pattern formed the basis for early AI systems but lacks the adaptability needed for complex pattern recognition tasks.

Despite their apparent simplicity, rule-based limitations surface quickly with complex real-world tasks. Recognizing human activities illustrates the challenge. Classifying movement below 4 km/h as walking seems straightforward until real-world complexity intrudes. Speed variations, transitions between activities, and unanticipated cases quickly fracture simple decision logic; each new activity forces nested conditional branches (figure 5.3), culminating in unhandled cases where scalar speed thresholds fail entirely. Computer vision tasks compound these difficulties. Detecting cats requires rules about ears, whiskers, and body shapes while accounting for viewing angles, lighting, occlusions, and natural variations. Early systems achieved success only in controlled environments with well-defined constraints.

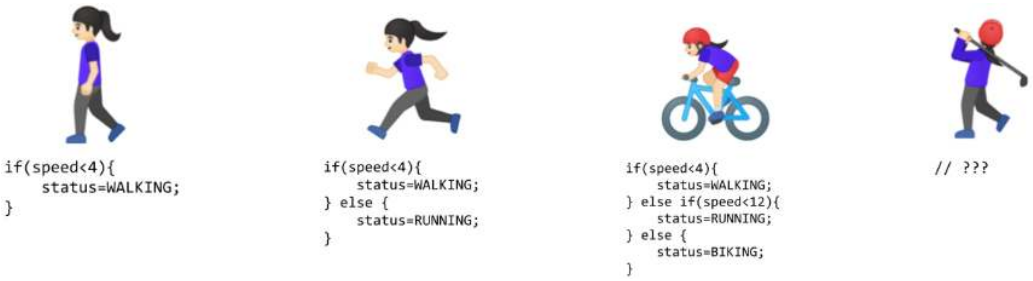


Figure 5.3: Activity Classification Rules: Successive code fragments classify walking below 4 km/h, then add running below 12 km/h and biking at higher speeds. A final unhandled activity shows how real-world cases force increasingly complex branching logic.

The knowledge-engineering programs of the 1970s and 1980s tried to systematize rule creation. Their expert systems[5] worked in narrow domains with well-defined parameters but stalled on tasks humans perform without effort, such as recognizing objects, understanding speech, and interpreting language, because much of that competence rests on implicit knowledge that resists explicit rules.

Applying this logic to the canonical benchmark illustrates the baseline workload. An illustrative rule-based classifier for the 28×28 MNIST digit compares pixel intensities against fixed thresholds, checks stroke patterns in localized regions, and branches on the results. Under this toy implementation, the computation requires roughly 100 comparisons over 784 bytes of pixel data, a workload that is small, predictable, and easily held in an ordinary CPU cache. The comparison establishes a workload baseline, not the accuracy of a production rule system.

5.2.2 The feature engineering bottleneck

The failures of rule-based systems suggested an alternative. Rather than encoding human knowledge as explicit rules, the system could discover patterns from data, and machine learning took that direction by replacing a rule for every situation with programs that identified patterns in examples. The success of these methods, however, still depended heavily on human insight to define which patterns to look for, a process known as feature engineering.

Feature engineering transformed raw data into representations that expose patterns to learning algorithms. The histogram of oriented gradients (HOG)[6] (Dalal and Triggs 2005) exemplifies this approach. The feature-extraction pipeline in figure 5.4 partitions an image into spatial cells, quantizes local pixel gradient directions across an eight-way orientation star, and compiles them into localized histogram bins. This transforms raw pixels into shape descriptors robust to lighting variations and small positional changes.

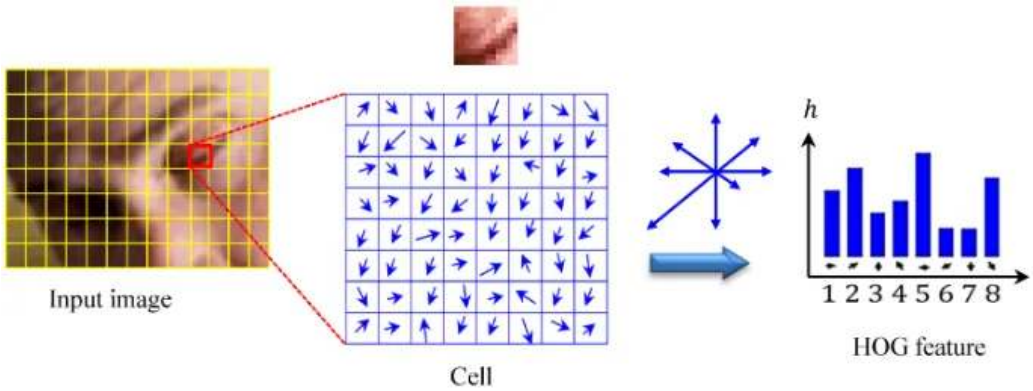


Figure 5.4: HOG Method: A local image patch is divided into cells, gradient orientations are accumulated into a histogram, and the resulting descriptor summarizes local shape while reducing sensitivity to illumination. Diagram reproduced from Figure 5 of Bakheet and Al-Hamadi (2021), licensed under CC BY 4.0.

5 **Expert systems**: These systems convert human expertise into explicit `IF-THEN` rules. This knowledge-engineering approach scales poorly for tasks such as object recognition, where the relevant variation is difficult to enumerate. Even successful systems such as DEC's XCON accumulated thousands of hand-authored rules, making rule acquisition, consistency, and maintenance central engineering costs.

6 **Histogram of oriented gradients (HOG)**: An influential handcrafted descriptor for pedestrian detection before deep learning (Dalal and Triggs 2005). One common configuration computes gradient orientations in 8×8 pixel cells, with cell size and other choices tuned for the task. HOG's fixed computation graph can run efficiently on CPUs with predictable latency, while learned features can reduce descriptor redesign but do not automatically transfer across domains and often benefit from accelerators only as model and batch scale grow.

Complementary methods like scale-invariant feature transform (SIFT)[7] (Lowe 1999) and Gabor filters[8] captured different visual patterns. SIFT detected keypoints stable across scale and orientation changes, while Gabor filters identified textures and frequencies. Each encoded domain expertise about visual pattern recognition.

These descriptors carried face and pedestrian detection through the 2000s, but every new problem needed an expert to design a new extractor, and the resulting features missed whatever patterns their designers had not anticipated. Human expertise, not computation, set the ceiling on how much real-world visual variation a system could absorb.

Evaluating the benchmark under feature engineering demonstrates the intermediate cost profile. For the same 28×28 digit, an illustrative HOG implementation divides the image into a 7 by 7 grid of 4 by 4 cells, computes gradient magnitudes and orientations, bins them into 9 orientation histograms per cell, and produces a 441-element feature vector. A linear support vector machine (SVM) classifier then performs ten dot products over that vector. The result is roughly 8,000 operations and about 2 KB of working memory, about 80× the work of the toy rule-based baseline, but still structured and well served by CPU vector units using **single instruction, multiple data (SIMD)**. Descriptor size, class count, and implementation all change this cost.

5.2.3 Automatic pattern discovery

The limitations of handcrafted features motivate a more radical approach in which the system discovers its own features. Neural networks embody this shift. Rather than following explicit rules or relying on human-designed feature extractors, they learn representations directly from raw data.

Supervised learning changes the traditional programming relationship. Instead of encoding every decision rule directly, engineers provide examples and target answers, and optimization produces a learned model. Figure 5.5 inverts the classical dataflow, routing example data alongside target answers into the machine learning optimizer so that decision rules emerge as the output rather than the input. Humans still shape the task through data, objectives, architectures, and evaluation.

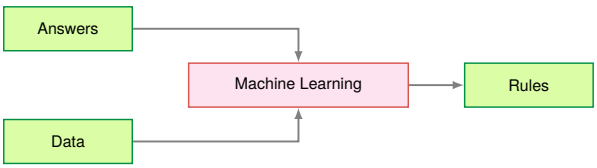


Figure 5.5: Data-Driven Rule Discovery: The flow diagram inverts the traditional programming pattern: data and answers serve as inputs to the machine learning process, which produces a learned decision function as output. This shift replaces a hand-coded decision policy while leaving engineers responsible for the data, objective, architecture, and evaluation.

The system discovers patterns from examples through this automated process. When shown millions of images of cats, it learns to identify increasingly complex visual patterns, from simple edges to combinations that constitute cat-like features. This parallels how biological visual systems operate, building understanding from basic visual elements to complex objects.

The gradual layering of patterns reveals *why* neural network depth matters. For some compositional function classes, deeper networks can represent functions that require much wider shallow networks, an advantage formalized in section 5.4.5.

Deep learning often exhibits empirical scaling: within a given regime, performance can improve as additional data, compute, and model capacity are used effectively, although gains depend on data quality, optimization, architecture, and evaluation conditions. In the ImageNet competition, traditional methods achieved approximately 25.8 percent top-5 error in 2011. AlexNet[9] reduced this to 15.3 percent in 2012. By 2015, ResNet achieved 3.6 percent top-5 error, lower than a published human baseline of approximately 5.1 percent under that evaluation protocol.

These ImageNet gains helped motivate larger models and training budgets, turning arithmetic throughput, memory bandwidth, storage capacity, and communication into potential bottlenecks. The gains describe a few selected systems on one benchmark; they do not make model size a decision rule. Choosing a model still requires comparing candidates on representative deployment data and on the complete cost of training, serving, monitoring, and maintenance, with the uncertainty in both

[7] **Scale-invariant feature transform (SIFT)**: SIFT encodes this domain expertise in a four-stage algorithm that identifies keypoints stable across scale and rotation. The number of detected keypoints varies by image, which complicates fixed-shape batching. Systems can regularize the representation through caps, padding, pooling, or fixed encodings, but each choice adds computation or discards information.

[8] **Gabor filters**: Originally developed for localized time-frequency analysis (Gabor 1946), these filters detect edges and textures at specific orientations and frequencies. A typical bank contains many hand-designed filters across orientations and frequencies. Deep convolutional layers instead learn small kernels from data; early convolutional neural network (CNN) filters often become edge-, color-, and texture-sensitive detectors, shifting feature design from manual filter banks to data-driven optimization (Krizhevsky et al. 2012; LeCun et al. 2015).

[9] **AlexNet's memory split**: Krizhevsky's team split AlexNet across two NVIDIA GTX 580 GPUs, each with 3 GB of on-board device memory (VRAM), so that the model and training state fit across the two devices. The workaround illustrates the systems lesson: a training configuration can exceed one device's memory budget, forcing engineers to partition work and communication across devices.

gains and costs included. The useful question is not whether additional scale can help, but whether its measured gain earns those system costs.

↪ How model size, training data, and compute trade off along a compute-optimal frontier is developed in section 9.6.2.

Systems Perspective 5.1: When to use neural networks

Not every problem benefits from deep learning. Neural networks are strong candidates when data exhibits spatial, sequential, or hierarchical structure that an architecture can exploit, simpler baselines miss the required task quality, and measured gains justify the complete system cost (table 5.1). With limited labels, low-dimensional tabular data, approximately linear relationships, hard latency limits, or strict auditability requirements, classical methods may match the useful performance of a neural network with less training, serving, and maintenance effort (table 5.2).

Table 5.1: Neural-Network Candidates: Conditions under which deep learning is likely to outperform classical methods.

Condition	Evidence to seek	Rationale
Representative data	Training and validation cover deployment conditions	Capacity cannot replace missing coverage
Exploitable structure	Spatial, sequential, or hierarchical patterns	Architecture can encode a useful prior
Baseline gap	Simpler methods miss the required task-quality target	Complexity buys a measured gain
Operational fit	Gain survives latency, memory, and cost limits	Offline accuracy alone is insufficient
Maintenance advantage	Learned features reduce recurring manual updates	Full-system maintenance becomes simpler

Table 5.2: Classical-Method Candidates: Conditions under which simpler models are likely to match or beat a neural network.

Condition	Baseline to test	Reason to prefer it
Limited labeled data	Linear model or tree ensemble	Lower variance and easier validation
Structured tabular inputs	Gradient boosting or linear model	Strong accuracy without learned features
Approximately linear signal	Linear or additive model	Transparent and inexpensive execution
Hard resource bound	Rules or compact classical model	Predictable latency, memory use, and energy demand
Direct auditability needed	Constrained tree or linear model	Decisions are easier to inspect

Systems insight: Establish a simple baseline before building a neural network. Compare candidates on the task metric that matters and on the resources required to train, serve, monitor, and update them. Retain the neural approach only when its measured benefit justifies that added system. The document-recognition history in section 5.7 shows why the comparison must include preprocessing, rejection policy, and the cost of human review rather than model accuracy alone.

Return to the same 28×28 digit, now processed by the chapter's three-layer neural network ($784 \to 128 \to 64 \to 10$). Under the stated implementations, the forward pass requires 109,184 MACs, 1,100× the work of the toy rule-based baseline. The 109,386 parameters consume 438 KB in 32-bit single-precision floating-point (FP32), exceeding most L1 caches and creating traffic between cache levels during inference. Training multiplies the cost further: each image is processed forward and backward before an update, with simplified dense-layer arithmetic on the order of three forward passes, repeated over 60,000 images for multiple epochs (full passes over the training set). Dense matrix multiplications now dominate this example's arithmetic and favor parallel hardware, shifting the engineering challenge from control-flow branching to memory bandwidth, data reuse, and hardware parallelism.

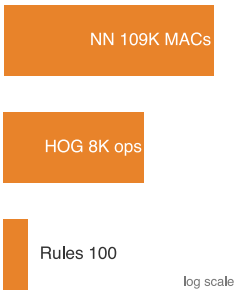
This trade is the bitter lesson at the scale of a single layer. HOG encodes an expert's knowledge of edges and orientations in 8,000 operations that never change; the learned first layer spends far more arithmetic and lets the data decide which pixel combinations matter. As computation became cheap enough, the general method that converts arithmetic into accuracy overtook the hand-built one. The price of that victory is the arithmetic, memory traffic, and training infrastructure that the rest of this chapter quantifies.

↩ Why general methods that scale with computation keep overtaking methods built on human knowledge, the bitter lesson, is established in section 1.4.

Removing the feature engineer moves the burden onto infrastructure that stores large datasets, hardware that processes them at high throughput, and accelerators built for the arithmetic itself. Those demands helped drive the development of chips optimized for neural network operations.

5.2.4 Computational infrastructure requirements

The escalation just traced on one digit matters less for its 1,100× operation count than for what each step does to the likely bottleneck, which moves from branch-heavy CPU control, to batch-oriented feature pipelines, to memory-fed matrix parallelism (table 5.3). CPUs remain effective for small networks and latency-sensitive workloads, while large dense operations often map more efficiently to accelerators.



In this MNIST comparison, learned features spend far more arithmetic.

The shift toward parallelism creates new bottlenecks. The central challenge is the memory wall, the widening gap between compute throughput and off-chip memory bandwidth. Whether a matrix operation runs into that wall depends on its arithmetic intensity, the operations it performs per byte it moves. A matrix-matrix multiplication can reuse each loaded value many times, especially when it is *tiled*, computed block by block on submatrices small enough to stay in fast on-chip memory while every value in the block is used repeatedly. Such a multiplication can be compute bound. A matrix-vector product, a layer with very few rows or columns, or any other low-reuse operation uses each loaded value only once or twice and is usually limited by bandwidth.

↩ The memory wall and the physical reasons that off-chip bandwidth lags arithmetic are established in section 2.2.

↪ The AI memory wall, the point at which added FLOP/s stops reducing runtime because data movement dominates, is developed in section 11.5.1.

Table 5.3: System Resource Tendencies: The illustrated implementations shift from control-oriented rules to structured feature pipelines and then to matrix-heavy neural computation. These are workload tendencies rather than requirements of every program in each paradigm.

System Aspect	Traditional Programming	ML with Features	Deep Learning
Computation	Often control-flow oriented	Structured parallel ops	Often matrix-parallel
Memory Access	Workload dependent	Commonly batch-oriented	Large tensor working sets
Data Movement	Program and input dependent	Feature-pipeline movement	Parameter and activation movement
Hardware Needs	Commonly CPU-centric	Often CPU with vector units	Frequently accelerator-friendly
Resource Scaling	Input and program dependent	Descriptor and data driven	Driven by width, batch, state, and reuse

Energy adds a constraint alongside speed. Moving data from main memory to processing units can consume far more energy than a simple arithmetic operation (Horowitz 2014), the same energy tax that accompanies the data-movement term of the iron law. This energy hierarchy explains why neural network accelerators maximize data reuse: they keep frequently accessed values in local storage and schedule operations to reduce movement. Whether movement or arithmetic dominates depends on precision, reuse, workload shape, and hardware, but the cost of movement motivates specialized architectures from data center GPUs to TinyML accelerators.

The memory-computation trade-off manifests differently across the cloud-to-edge spectrum. Cloud servers may use more memory and power to pursue throughput, while mobile devices operate within tighter power budgets. Large-batch training commonly emphasizes throughput; latency-sensitive or battery-powered inference may emphasize response time or energy per prediction. The exact priority depends on the deployment workload.

These constraints compound as models grow. Dense-layer cost scales with the product of adjacent layer widths, larger batches scale both throughput demand and the intermediate values each layer must hold, and training multiplies memory further through the backward pass and optimizer state that section 5.5 develops. Model-compression, hardware-acceleration, and training-system

techniques all respond to this pressure by reducing the work, raising the useful machine rate, or changing where state lives.

The digit-classification ledger also raises a historical puzzle. The mechanisms behind it (weighted sums, nonlinear activations, and gradient-based learning) are decades older than the machines that made them practical, and the growth in arithmetic and memory that the ledger records is what kept neural computation waiting so long to scale.

5.3 Why Neural Computation Scaled When It Did

Rosenblatt's perceptron introduced a learning neuron model in 1958 (Rosenblatt 1958). Backpropagation[10] was applied to neural networks by Paul Werbos in 1974 (Werbos 1974), building on Seppo Linnainmaa's 1970 work on automatic differentiation (Linnainmaa 1970), and Rumelhart, Hinton, and Williams showed in 1986 that hidden units could learn useful internal features (Rumelhart et al. 1986). That demonstration did not resolve the systems problem the digit-classification ledger exposes. Every increase in model size multiplies forward operations, saved activations, gradient computations, and parameter updates, so the decades between early backpropagation work and widespread adoption reflect the maturation of hardware, data, initialization, nonlinearities, optimization, benchmarks, and software infrastructure. An algorithm can be viable in principle long before its surrounding system makes it useful in practice. The lesson does not run in reverse: a computationally expensive idea is not vindicated by expense alone, and it must be evaluated together with the data, software, hardware, and task regime that would make it useful.

Compute is what changed. Figure 5.6 plots representative training-compute estimates across seven decades on a logarithmic scale (10^3 to 10^{27} FLOPs), and its fitted trend lines bend sharply near 2012. From the 1958 Perceptron Mark I through 2010, frontier training compute grew at roughly the pace of Moore's law; from AlexNet onward, through systems such as AlphaGo, GPT-4, and Gemini, it doubled several times faster, the same era split that a systematic census of training runs identifies (Sevilla et al. 2022). The doubling times printed in the figure's legend are fits to its plotted points and shift with the systems included, so the volume's reference growth rate remains the aggregate trend established with the introduction's compute-growth figure. While table 5.4 grounds these milestones in specific training platforms, the trajectory reflects empirical resource allocation rather than an audited census or an invariant physical law. Algorithmic and systems advances have reduced the compute needed to reach fixed benchmark targets in some domains, and utilization, reduced precision, software maturity, and procurement conditions mediate the relationship between raw compute and monetary cost. Planning therefore requires measured efficiency, a stated comparison target, and explicit uncertainty, especially for closed models whose architecture and training configuration are undisclosed.

[10] **Backpropagation**: Short for "backward propagation of errors," the algorithm uses the chain rule to compute how the loss is locally sensitive to millions of weights. Werbos applied it to neural networks in 1974 (Werbos 1974), and the 1986 Rumelhart, Hinton, and Williams publication demonstrated practical effectiveness (Rumelhart et al. 1986). The systems cost: backprop requires storing forward-pass activations and additional training state, creating the several-times-higher memory footprint quantified in section 5.5.5.4.

↩ How AlexNet, GPUs, and ImageNet ended the feature-engineering era, and why infrastructure became the next bottleneck, is established in section 1.3.3.

↩ The aggregate growth of frontier training compute, and the systems gap it opens against hardware improvement, is established in section 1.8.

Table 5.4: Historical Performance: Four decades of representative neural-network systems showing growth in model state, training compute, and hardware infrastructure. Earlier rows use published specifications where available. GPT-3's hardware configuration and duration are undisclosed; GPT-4's architecture, parameter count, hardware configuration, and duration are undisclosed, and its compute entry is an external order-of-magnitude estimate consistent with figure 5.6. Total parameter count also does not determine per-input computation for sparsely activated architectures. "OoM" denotes order-of-magnitude uncertainty.

Year	System	Params	Train FLOPs	Hardware	Train Time	Error/Task
1989	LeNet-1	~10K	10^{11}–10^{12}	Sun-4/260 workstation	3 days	1% at 12.1% rejection (USPS digits)
1998	LeNet-5	60K ± 1K	$10^{14} \pm 1$ OoM	SGI Origin 2000 (200 MHz)	2–3 days	0.95% (MNIST)
2012	AlexNet	~60M	5×10^{17}	2× GTX 580 GPUs	5–6 days	15.3% (ImageNet)
2015	ResNet-152	~60M	$10^{19} \pm 0.5$ OoM	8× Tesla K80 GPUs	~3 weeks	3.6% (ImageNet, ensemble)
2020	GPT-3	175B (exact)	3×10^{23}	Undisclosed	Undisclosed	N/A (language)
2023	GPT-4	Undisclosed	10^{24}–10^{25} (external est.)	Undisclosed	Undisclosed	N/A (language)

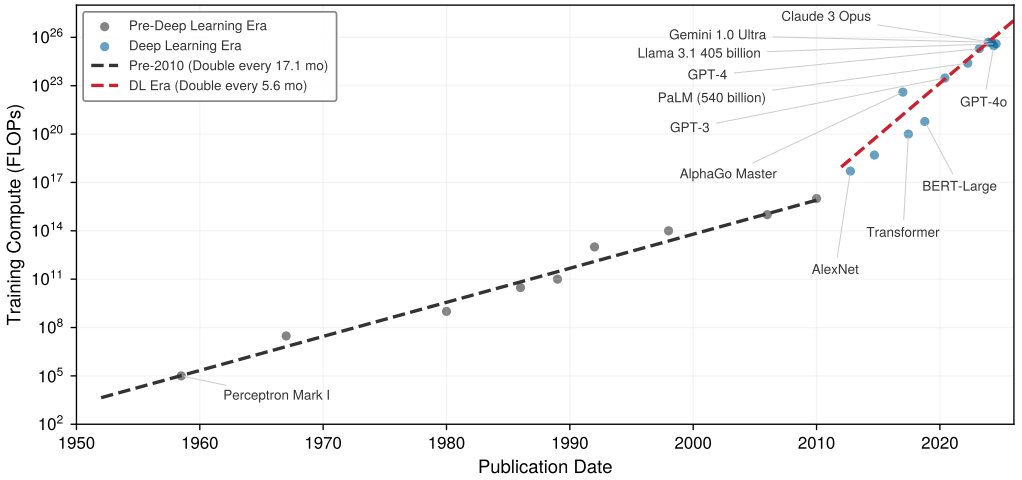


Figure 5.6: Computational Growth: Log-scale scatter plot of representative training-compute estimates in FLOPs from 1958 to 2024. Fitted trend lines compare a slower pre-2010 regime with a steeper post-2012 deep-learning frontier; the doubling times in the legend are fits to the plotted points, and the values illustrate order-of-magnitude growth rather than an audited census of model training runs.

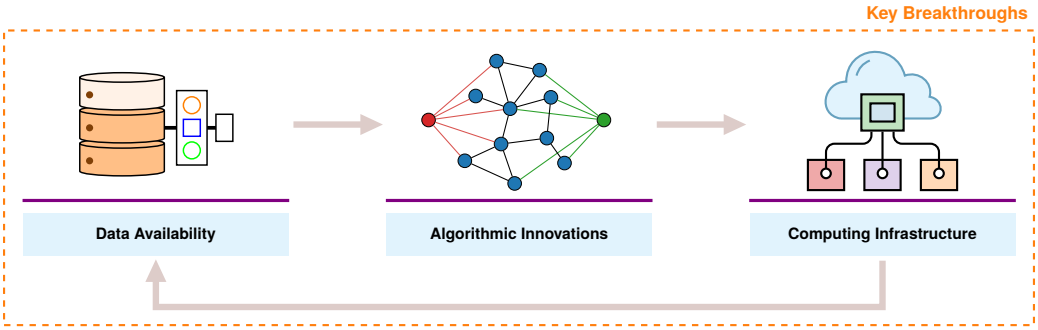


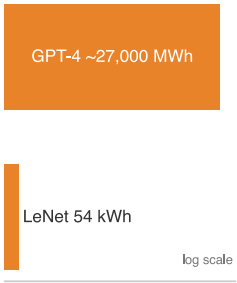
Figure 5.7: Deep Learning Virtuous Cycle: Three mutually reinforcing factors, data availability, algorithmic innovations, and computing infrastructure, form a self-reinforcing loop where breakthroughs in one area create opportunities in the others.

Beyond raw compute, this growth carries an energy cost that systems engineers cannot ignore. Under the stated scenario assumptions, a three-day LeNet-1 workstation run would use roughly 54 kWh. Applying an external GPT-4 GPU-day estimate and a hyperscale-facility power usage effectiveness (PUE) of 1.12 yields about 27,000 MWh, comparable to the annual electricity use of roughly 2,600 representative US homes.[11] These are order-of-magnitude estimates, not measured utility records, and they vary with accelerator utilization, facility overhead, and the calculation boundary. Even so, the scale makes energy per operation and achieved utilization design concerns alongside peak FLOP/s.

Three interdependent pillars, data availability, algorithmic innovation, and computing infrastructure, drove this expansion through a self-reinforcing loop (figure 5.7). High-capacity datasets supplied the training signals that deep representations demand; algorithmic breakthroughs (stabilized initializers, adaptive optimizers, and architectures that train reliably at depth) rendered large models trainable; and specialized computing infrastructure (accelerator parallelism from GPUs and Tensor Processing Units (TPUs)[12] (Jouppi et al. 2023), high-bandwidth memory, and optimized runtimes) reduced turnaround times from months to days, producing models that in turn automated further data generation. The cycle is enabling rather than automatic, and progress in one dimension can expose a bottleneck in another.

11 **Training energy scale**: The estimate assumes a 750 W workstation for the historical run and uses 10.5 MWh/year as a representative US household electricity budget. The closed-model scenario treats an external GPU-day estimate as A100-equivalent accelerator time and applies an assumed hyperscale-facility PUE of 1.12, in line with published hyperscaler fleet values near 1.1 to 1.2. The global survey average of about 1.55 would raise the estimate by roughly 40 percent. The result illustrates scale; it does not establish the actual energy used to train GPT-4.

The infrastructure side of that cycle is more than faster chips. A network's mathematical graph does not prescribe an execution plan: frameworks and compilers choose kernels, tensor layouts, fusion boundaries, and schedules for a particular processor. Two implementations of the same function with the same nominal arithmetic count can therefore differ substantially in data traffic, temporary storage, and latency. Hardware supplies arithmetic units, memory, and communication links; software determines how the graph uses them. Because learned behavior lives in parameters stored at ordinary memory addresses, and training adds saved intermediates, gradients, and optimizer state, storage capacity and bandwidth matter alongside arithmetic. That dependence on data movement motivates specialized accelerator hardware.



GPT-4-scale training energy dwarfs a LeNet-1 run by over five orders of magnitude.

↪ How energy per useful inference, utilization, and cooling overhead enter hardware selection is developed in section 11.13.

12 **TPU (tensor processing unit)**: Google's custom accelerator, first deployed internally in 2015, was optimized specifically for the matrix multiplications that dominate neural network workloads. The TPU v1 achieved 92 TOPS (vendor-reported INT8 tera-operations/s) for inference at 75 W, a power-efficiency point that general-purpose GPUs of the era could not match. The name "Tensor Processing Unit" reflects the design decision to sacrifice general-purpose flexibility for maximum throughput on the operation neural networks need most.

> **Checkpoint 5.1: Understanding deep learning's emergence**
>
> Before turning to the primitives that set these costs, verify your understanding of why deep learning emerged when it did:
>
> - ☐ **Adoption lag**: can you explain why backpropagation existed for more than a decade before it was widely used, naming the systems costs that grow with model size?
> - ☐ **Feature learning's price**: can you explain, using the digit-classification operation counts, why replacing hand-engineered features with learned ones raises the arithmetic and memory demand?
> - ☐ **Compute trajectory**: can you read figure 5.6 to say how the growth of training compute changed after 2012, and why the fitted doubling times are estimates rather than constants?
> - ☐ **Three enabling factors**: can you describe how data, algorithms, and infrastructure reinforced one another, and how a gain in one can expose a bottleneck in another?
> - ☐ **Energy scale**: can you explain why energy per operation and achieved utilization became design concerns as training runs grew from workstation scale to frontier scale?
>
> If any of these concepts remain unclear, review the relevant sections before continuing.

The convergence of data availability, algorithmic innovation, and computing infrastructure transformed neural networks from theoretical formulations into production workloads. What a network costs that infrastructure, however, is fixed by the primitives it is built from, and those primitives are the subject of the rest of this chapter.

5.4 Neural Network Fundamentals

Each of the 109,184 MACs that classify a single digit belongs to some neuron combining its inputs. Individually those operations are trivial. At network scale, a processor spends more time waiting for operands than computing whenever its memory system cannot supply them fast enough, and an accelerator such as a GPU outruns a CPU on suitable neural workloads because it devotes silicon to wide arithmetic arrays and high-bandwidth memory rather than to branch prediction. The structure of the network primitives (neurons, layers, and nonlinear activations) determines how data flows through that hardware and how memory demands scale during training and inference.

Those primitives (weighted sums, nonlinear activations, and gradient-based learning) anchor the feed-forward networks of this chapter and recur inside the larger architectures that later chapters introduce, which add operators and execution patterns this simple model does not contain. The pressures traced through the paradigm comparison (parallelism, the memory wall, and energy-dominated data movement) follow from their structure, in which many simple units operate simultaneously on shared inputs, weights change during training, and layers compose low-level features into high-level ones. That structure begins with the artificial neuron.[13] Understanding it plays the role for a neural network that understanding a single transistor plays for a processor.

[13] **Neuron**: McCulloch and Pitts (1943) introduced a mathematical threshold model of nervous activity: inputs combine and an all-or-none output fires when a threshold is met. That logical neuron is an origin point for the artificial-neuron abstraction and for networks built from many simple units. Learned weights, deep hierarchies, and modern fused multiply-add (FMA) accelerator datapaths are later developments that turn this abstraction into the matrix-heavy workloads studied in this chapter.

5.4.1 The artificial neuron as a computing primitive

The basic unit of neural computation, the **artificial neuron** (or node), serves as a simplified mathematical abstraction of nervous activity (McCulloch and Pitts 1943). Later digital neural-network implementations adapted this abstraction into a standardized processing unit. This building block enables complex networks to emerge from simple components working together. The structural schematic in figure 5.8 aligns biological neuron anatomy against artificial neuron primitives along a central correspondence spine, showing how electro-chemical structures distill into modular arithmetic operations.

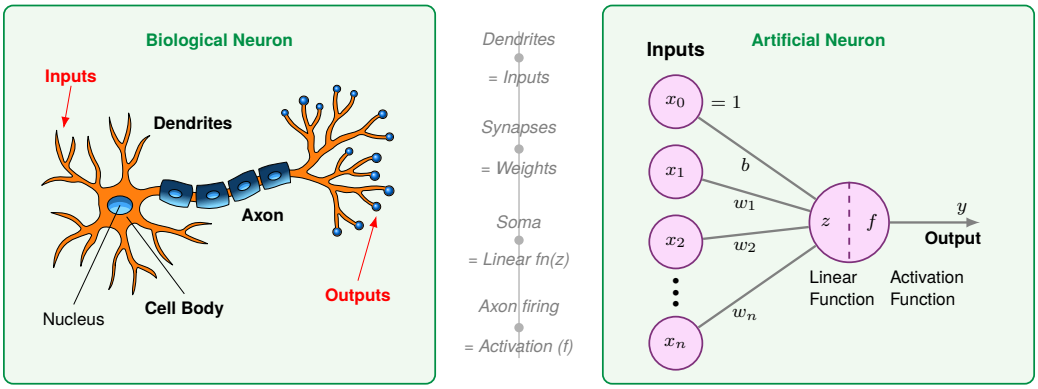


Figure 5.8: Biological-to-Artificial Neuron Mapping: Side-by-side comparison showing how biological neuron structures map to artificial neuron components. Dendrites correspond to inputs, synapses to weights, the cell body to the summation function, and the axon to the activation output. Read the mapping as an abstraction rather than a resemblance claim, since it keeps only what reduces to arithmetic and discards the electrochemistry that produced the behavior.

Traversing the right panel of figure 5.8 follows the signal through four computational stages, each translating an anatomical structure on the central alignment spine into a formal mathematical operation:

1. **Input Reception** (Dendrites $\rightarrow x_1, x_2, \ldots, x_n$): The artificial neuron receives a vector of input features $\mathbf{x}$. In MNIST digit recognition, these represent pixel intensities. The dendrite analogy captures reception of many signals without claiming biological equivalence.
2. **Weighted Modulation** (Synapses $\rightarrow w_1, w_2, \ldots, w_n$): Each input is multiplied by a learnable weight w_i, just as synaptic strengths modulate biological signals. These weights act as "gain" controls, determining how much influence each feature has on the final decision. A bias term b (shown as the top input $x_0 = 1$ in figure 5.8) shifts the activation threshold. Across a network, learned behavior is distributed over many such parameters rather than stored in one weight.

3. **Signal Aggregation** (Cell Body → Linear Function z): The neuron integrates the weighted signals, producing a single scalar value $z = \sum(x_i \cdot w_i) + b$. This mirrors how a biological cell body sums incoming electrochemical signals to determine whether the neuron has received enough evidence for a particular pattern.
4. **Nonlinear Activation** (Axon → Activation Function f): The aggregated signal passes through an activation function $f(z)$, producing the activation output $a = f(z)$ (often denoted y). This mirrors the axon's all-or-nothing firing decision: the nonlinearity determines whether the neuron "fires" a signal to the next layer. Unlike the biological case, f can produce graded outputs (for example, ReLU passes positive values through, zeroes negatives), but the principle is the same, thresholding followed by propagation. Table 5.5 formalizes these correspondences.

Table 5.5: Biological vs. Artificial Neuron Mapping: Each biological structure paired with its artificial component, the mathematical operation that component performs, and its engineering role.

Biological Structure	Artificial Component	Mathematical Operation	Engineering Role
Dendrites (receive signals)	**Input Vector**	$\mathbf{x} = [x_1, \ldots, x_n]$	Data ingestion from sensors or prior layers
Synapses (modulate strength)	**Weight Vector**	$\mathbf{w} = [w_1, \ldots, w_n]$	Learnable parameters encoding importance
Cell Body (integrates signals)	**Linear Function** z	$z = \sum(x_i \cdot w_i) + b$	Linear integration of feature signals
Axon (fires output)	**Activation Function** f	$a = f(z)$	Nonlinear thresholding and signal propagation

From a systems engineering perspective, this translation reveals *why* neural networks can have demanding computational requirements. A direct scalar evaluation of one neuron performs d_{in} MAC[14] operations and, without reuse, loads d_{in} inputs, d_{in} weights, and a bias before writing the output. Batched and tiled kernels reuse inputs and weights across many neuron evaluations, reducing traffic per MAC. At network scale, performance therefore depends on both the number of operations and how effectively the implementation reuses data.

The transition from individual neurons to integrated systems requires navigating the central trade-off between representational capacity and computational cost. Digital hardware performs arithmetic quickly, but the volume of operations and data movement in deep networks creates bottlenecks that a single-neuron view does not expose.

[14] **MAC (multiply-accumulate)**: A core operation in dense and convolutional layers is acc ← acc + $w \cdot x$. Hardware data sheets often report fused multiply-add throughput in FLOPs because one multiply and one add count as two floating-point operations. On that convention, the reported FLOP/s rate is twice the MAC/s rate when a MAC is counted as one multiply-accumulate. Layer and batch choices also create activation, communication, and data-movement costs, so MAC count is necessary but not sufficient for a systems budget.

5.4.2 Network architecture fundamentals

A neural network's architecture determines how information flows from input to output, and each architectural choice, from how neurons connect to how layers are organized, fixes a computational pattern that hardware must then execute. The digit recognizer that the chapter has used informally since its first page now becomes a precise running example, a network that classifies images from the MNIST dataset (LeCun et al. 1998). The task is small enough to compute by hand yet exercises every mechanism that larger networks scale up.

Example 5.1: Running example: MNIST digit recognition

Scenario: Deploying a basic handwritten digit classifier on 28×28 grayscale images across 10 output classes.

Diagnosis: The chapter's running architecture maps 784 raw pixel features through 128 and 64 hidden units to 10 outputs, creating roughly 100K parameters.

Systems lesson: Inspecting input vector dimensions and layer parameters on a compact dataset establishes baseline compute and memory bandwidth requirements before scaling to high-dimensional production workloads.

5.4.2.1 The perceptron's weighted sum

[15] **Perceptron**: A machine built to execute a learning algorithm, directly linking hardware and software from the start. A single linear-threshold layer cannot represent nonlinearly separable functions such as XOR, regardless of additional input hardware. Multilayer nonlinear networks expand the representable function class, but they also introduce the training and systems costs developed in this chapter.

The computational machinery within each dense layer is the **perceptron**,[15] which Rosenblatt introduced as a learning form of the artificial neuron (Rosenblatt 1958). Its signal path (inputs, weights, bias, aggregation, activation) is the one section 5.4.1 traced through its biological origins. What remains is to formalize that path mathematically, because this repeated operation becomes the layer-level kernel that hardware executes. In the MNIST network, a first-layer unit combines all 784 pixel intensities into one activation that may respond to a learned pattern, such as a vertical edge shared by "one" and "seven." The computational dataflow in figure 5.9 diagrams this single-unit execution: input features x_i traverse weighted channels w_{ij}, accumulate into a scalar sum with bias b_j, and pass the intermediate linear score z_j through an activation block f to produce output $\hat{y}$. Different neurons repeat the same structure with different learned weights, allowing one layer to test many patterns against the same input.

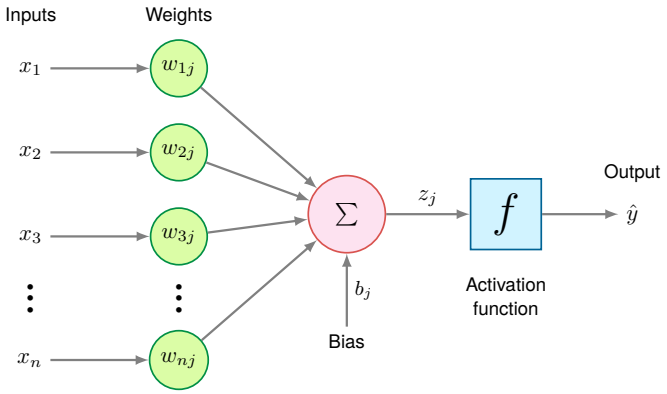


Figure 5.9: Perceptron Architecture: Inputs enter on the left, each scaled by its own weight, and the weighted products plus a bias meet at the summation node before a single activation function produces the output. The multiplies and the accumulation along this path are the per-neuron unit of work that the chapter's layer-cost calculations multiply out.

Each input x_i has a corresponding weight w_{ij}, and the perceptron multiplies each input by its matching weight. The intermediate output, z_j, is computed as the weighted sum of inputs in equation 5.1:

$$z_j = \sum_i (x_i \cdot w_{ij}) \tag{5.1}$$

Here, j identifies the receiving neuron once perceptrons are assembled into a layer, and the sum runs over all input indices i. In plain terms, each input feature is scaled by how important it is (its weight), and the results are summed into a single score. This is the dot product of two vectors, the fundamental operation that hardware accelerators are designed to execute at maximum throughput, and the reason neural network performance is measured by the number of MAC operations completed per second.

A bias term b_j shifts the linear output up or down, giving the model additional flexibility to fit the data. Thus, the intermediate linear combination computed by the perceptron including the bias becomes equation 5.2:

$$z_j = \sum_i (x_i \cdot w_{ij}) + b_j \tag{5.2}$$

With this weighted sum in hand, a suitable output activation and loss can support regression or classification. A binary linear-threshold classifier selects one of two classes around a threshold; multiclass networks instead produce several scores and commonly normalize them into probabilities with softmax, an output function developed with the other activation functions below.

Under the direct scalar, no-reuse accounting established earlier, one neuron with d_{in} inputs performs d_{in} multiply-accumulate operations and moves its inputs, weights, bias, and output. What the formalization adds is the layer view. A dense layer of d_{out} neurons repeats the weighted sum d_{out} times, so its arithmetic cost is $d_{\text{in}} \times d_{\text{out}}$ MACs, exactly the matrix multiplication $\mathbf{xW}$ that hardware must execute, and the form in which the rest of this chapter counts work.

5.4.2.2 Nonlinear activation functions

Activation functions are where expressiveness, gradient flow, and hardware cost meet. They convert linear weighted sums into nonlinear outputs; without them, multiple linear layers would collapse into a single linear transformation, severely limiting the network's expressive power. Figure 5.10 charts response curves and output boundaries for three standard element-wise nonlinearities alongside the vector-level softmax function, contrasting their saturation regimes and mathematical ranges.

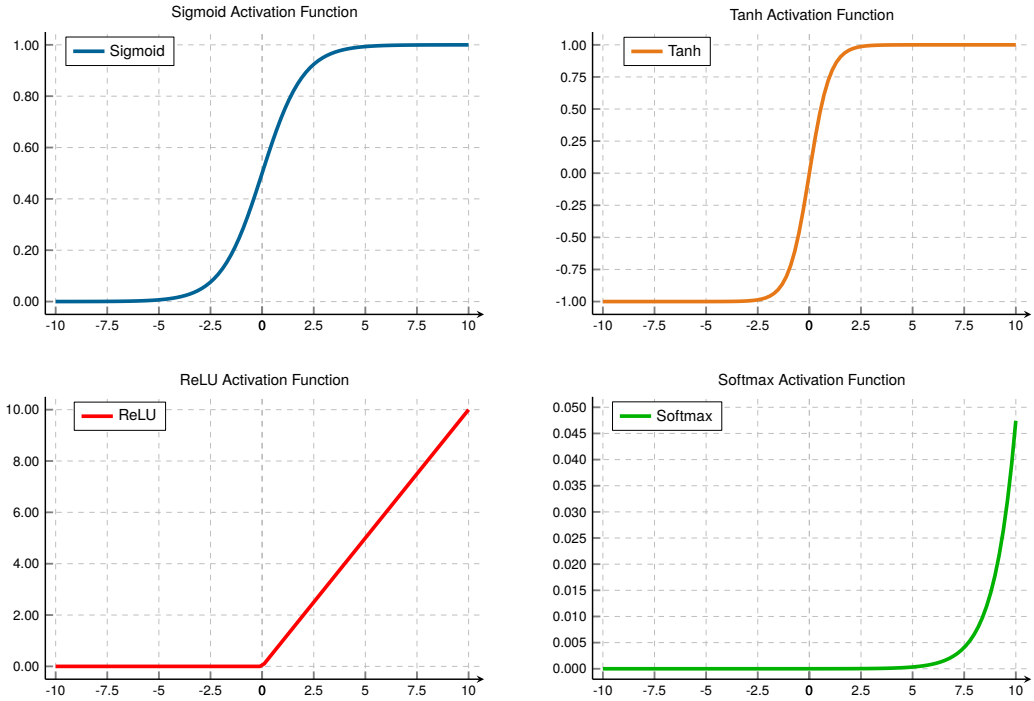


Figure 5.10: Common Activation Functions: Four nonlinear activation functions plotted with their output ranges. Sigmoid maps inputs to $(0, 1)$ with smooth gradients, tanh provides zero-centered outputs in $(-1, 1)$, and ReLU outputs zero for negative inputs. Softmax (bottom-right) shows one component of a 3-element vector while the other two logits are fixed at 13 and 0. Its magnified y-axis (roughly 0 to 0.05) is not comparable to the other panels, and it shows only one component of a vector whose outputs are interdependent and sum to 1.

Comparing activation functions requires one idea from training, which section 5.5 develops fully. Training adjusts each weight by its *gradient*, a number measuring how much the prediction error would change if that weight moved slightly. That signal travels backward through the network, a pass called *backpropagation*, and is multiplied at each layer by the activation's slope, so an activation whose slope is near zero passes almost no learning signal to the layers before it.

The choice of activation function therefore affects both learning and computational efficiency, revealing how systems constraints shape algorithmic design. ReLU ($\max(0, z)$) became a common hidden-layer activation for feed-forward networks because it is inexpensive, has local derivative one for positive inputs, and can produce sparse activations. Earlier differentiable deep networks often used sigmoid or tanh. Their smooth S-curves can saturate, causing gradients to shrink across many layers and impeding early-layer learning. That contrast is not a simple ranking: sigmoid remains appropriate for some binary outputs, tanh remains useful when bounded zero-centered activations are desired, and modern architectures use additional activation families. The relevant design question is which function supplies the required output semantics and gradient behavior at an acceptable execution cost. Understanding those roles explains ReLU's adoption without turning it into a universal default.

Sigmoid The logistic sigmoid function[16] squashes continuous inputs onto the open interval $(0, 1)$

[16] **Sigmoid**: From Greek *sigma* + *eidos* ("sigma-shaped"), referring to the S-curve that maps inputs to the bounded (0, 1) range. The mapping requires an exponential approximation, commonly implemented through specialized instructions, lookup tables, or polynomial methods. It is generally more expensive than ReLU's comparison-and-selection operation, although the measured gap depends on precision, vectorization, and hardware. ReLU's adoption reflected both easier gradient flow on its positive branch and cheaper execution.

(equation 5.3):

$$\sigma(z) = \frac{1}{1+e^{-z}} \tag{5.3}$$

The S-shaped curve maps logits, the raw unnormalized scores a layer produces, to $(0,1)$, so sigmoid can parameterize a binary probability. It approaches one for large positive inputs and zero for large negative inputs. Although sigmoid is smooth and differentiable everywhere, gradient-based learning need not be: ReLU uses an assigned subgradient at zero.

Sigmoid has a significant limitation: for inputs with large absolute values (far from zero), the gradient becomes extremely small, a phenomenon called the **vanishing gradient problem**.[17] During backpropagation, these small activation derivatives multiply with weight-dependent factors across layers. Repeated saturation can make early-layer gradients and weight updates negligibly small, preventing useful learning along those paths.

Tanh The hyperbolic tangent[18] centers output activations around zero over the interval $(-1,1)$ (equation 5.4):

$$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \tag{5.4}$$

Tanh produces an S-shaped curve similar to sigmoid but centered at zero: negative inputs map to negative outputs and positive inputs to positive outputs. This symmetry balances gradient flow during training, often yielding faster convergence than sigmoid.

Like sigmoid, tanh is smooth and differentiable everywhere, and it still suffers from the vanishing gradient problem for inputs with large magnitudes. When the function saturates (approaches -1 or 1), gradients shrink toward zero. Despite this limitation, tanh's zero-centered outputs make it preferable to sigmoid for hidden layers in many architectures, particularly in recurrent neural networks, which reapply the same layer at every step of a sequence and so must keep activations balanced across those time steps.

Both sigmoid and tanh share a critical limitation: gradient saturation at extreme input values. The search for an activation function that avoids this problem while remaining computationally efficient led to one of deep learning's most important innovations.

ReLU ReLU[19] sets negative values to zero while maintaining linear identity for positive inputs (equation 5.5):

$$\mathrm{ReLU}(z) = \max(0,z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{5.5}$$

ReLU's characteristic shape, a straight line for positive inputs and zero for negative inputs, provides three advantages. First, its local derivative is one for positive inputs, avoiding activation saturation on that branch; the complete network gradient can still shrink or grow through weights and other operations. Second, ReLU introduces deterministic activation sparsity by setting negative values to zero. Zero values do not automatically produce a speedup in standard dense execution, because ordinary matrix multiplication kernels still compute across all coordinate positions unless specialized sparse hardware or routines are used. Third, ReLU uses a comparison and selection, `output = (input > 0) ? input : 0`, rather than an exponential approximation. These are local properties of the operator, while end-to-end performance depends on how it is fused with neighboring work.

ReLU is not without drawbacks. The **dying ReLU problem** occurs when neurons become stuck in the inactive state, permanently outputting zero and ceasing to learn. If a neuron's weights evolve during training such that the preactivation $z = \mathbf{x}\mathbf{w} + b$ is consistently negative across all training examples, the neuron outputs zero for every input. Since ReLU's gradient is also zero for negative inputs, no gradient flows back through this neuron during backpropagation: the weights cannot update, and the neuron remains dead. This can happen when the learning rate, the step size that scales each weight update, is large enough to push weights into unfavorable regions. From a systems perspective, dead neurons represent wasted capacity: parameters that consume memory and compute during inference but contribute nothing to the output. Careful initialization (He et al. 2015), moderate learning rates, leaky ReLU variants, and normalization layers[20] help mitigate dying ReLUs by keeping preactivation distributions well scaled.

[17] **Vanishing gradient problem**: The chain rule's multiplication of Jacobian factors across layers can cause gradients to shrink. Sigmoid's derivative is at most 0.25, so each additional sigmoid layer can shrink the activation-derivative part of the gradient by at least a factor of four, the compounding that the chapter's opening footnote evaluated over 20 layers. The complete gradient also includes weights and other operations, but repeated saturated activation derivatives can still make early-layer updates negligible.

[18] **Tanh (hyperbolic tangent)**: By centering its output range on zero, `tanh` can reduce the correlated-update behavior associated with all-positive activations. Its computational cost is similar to sigmoid because it also depends on exponential approximations, while its saturation behavior still permits vanishing gradients.

[19] **ReLU (rectified linear unit)**: Unlike the costly exponential operations in prior activation functions, ReLU's `max(0, x)` operation maps to a simple comparison and selection. This efficiency, together with better gradient flow for positive activations, helped make the deep architectures of the AlexNet era computationally tractable (Nair and Hinton 2010; Krizhevsky et al. 2012).

[20] **Batch normalization**: Normalization layers stabilize intermediate activations across training batches, permitting higher learning rates and reducing the risk of dying ReLUs. Section 6.10.2.3 examines the trade-offs among normalization variants, and section 13.8.1.2 shows how serving compilers fold batch normalization into adjacent weights.

Softmax Unlike *element-wise* activation functions that operate independently on each value, softmax[21] is a *vector-level* function: it considers all values simultaneously to produce a probability distribution. In simple classifiers, softmax is commonly used in the output layer rather than as an element-wise hidden activation; it also normalizes learned importance-score vectors in attention architectures. The softmax function is defined in equation 5.6:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{5.6}$$

For a vector of K values (often called logits[22]), softmax transforms them into K probabilities that sum to 1. As plotted in the bottom-right panel of figure 5.10, individual softmax components exhibit exponential scaling across logit values, though true evaluation couples all vector elements to guarantee that outputs normalize to 1.

In multiclass classifiers, softmax is commonly used in the output layer. It converts arbitrary real-valued logits into normalized predicted probabilities, but those values are not necessarily calibrated confidence estimates. The class with the highest probability becomes the predicted class, and larger logits receive disproportionately higher probabilities.

The mathematical relationship between input logits and output probabilities is differentiable, allowing gradients to flow back through softmax during training. When combined with cross-entropy loss (discussed in section 5.5.3), softmax produces particularly clean gradient expressions that guide learning effectively. Beyond mathematical properties and output distributions, activation functions impose concrete hardware execution costs, altering datapath complexity and functional-unit area.

5.4.2.3 The transistor tax: Logic unit cost

The activation choice introduces physical hardware trade-offs alongside its optimization characteristics. In processor and accelerator design, the **logic unit cost** reflects the circuit complexity, silicon area, and dynamic switching energy demanded by the execution datapath.

The physical disparity between activation circuits originates in arithmetic structure. A ReLU datapath requires only a sign-bit comparator and a multiplexer to route either the incoming value or zero to the destination register, consuming roughly 50 transistor equivalents in a basic execution slice. In contrast, evaluating transcendental functions like sigmoid or tanh requires iterative polynomial approximation, a multi-stage CORDIC pipeline, or an interpolation lookup table. Implementing such transcendental datapaths demands substantial arithmetic hardware, modeled here as a stylized budget of 2,500 transistor equivalents.

This modeled divergence illustrates **the transistor tax**: within this stylized datapath model, the functional-unit complexity of a standalone transcendental evaluator is 50× that of a ReLU comparator. This ratio serves as an order-of-magnitude pedagogical model rather than an immutable physical constant across manufacturing nodes. Modern accelerators routinely amortize transcendental execution units across wide SIMD lanes, and optimizing compilers fuse element-wise activations directly into preceding matrix multiplication kernels to eliminate intermediate round trips to memory. The durable systems insight is that piecewise-linear functions minimize functional-unit area and register pressure while avoiding transcendental latency, reinforcing the optimization advantages of non-saturating gradients during training.

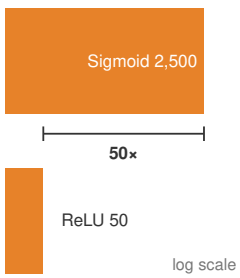
These transformations convert the linear input sum into a nonlinear output, yielding the generic artificial-neuron computation in equation 5.7:

$$\hat{y}_j = f(z_j) = f\left(\sum_i (x_i \cdot w_{ij}) + b_j\right) \tag{5.7}$$

This nonlinearity expands what multilayer networks can represent. Without it, a stack of linear layers collapses algebraically into one linear transformation, regardless of how many intermediate matrices the implementation evaluates. This geometric advantage is isolated in figure 5.11: a network without activation functions is constrained to a linear decision hyperplane (left panel) that misclassifies overlapping point clusters, whereas incorporating nonlinear activations enables curved decision boundaries (right panel) that contour around nonlinearly separable data. The visualization

21 **Softmax**: The name reflects its role as a "soft" or differentiable version of `argmax`, a function that must evaluate an entire vector to find its maximum value. This vector-wise operation is not used as a drop-in replacement for element-wise nonlinearities such as ReLU, but it is central whenever a model must normalize a vector of scores, including classification heads and attention layers. Direct exponentiation can overflow in 32-bit floating point, so stable implementations subtract the largest logit first (section 5.5.3.3).

22 **Logits**: In multiclass neural classifiers, logits are unnormalized output scores. Score differences correspond to log probability ratios after softmax, while an individual multiclass logit is not itself a calibrated log-odds quantity. Because `argmax` over logits and `argmax` over softmax probabilities select the same class, inference can skip softmax when only the top class is needed.



Sigmoid's exponential datapath dwarfs ReLU in transistor-equivalents (illustrative).

highlights representational expressiveness rather than an optimization guarantee; training dynamics and sample coverage still govern whether gradient descent converges to that partition.

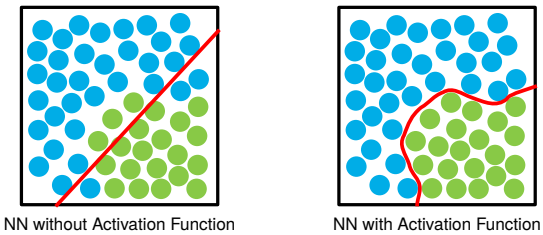


Figure 5.11: Linear vs. Nonlinear Decision Boundaries: Two scatter plots compare classification with and without activation functions. Without activation, a straight line fails to separate the two classes. With a nonlinear activation function applied, the network produces a curved decision boundary that correctly separates the points.

The **universal approximation theorem**[23] establishes that sufficiently wide networks with suitable activation functions can approximate broad classes of functions arbitrarily well. This theoretical foundation, combined with the computational and optimization characteristics of specific activation functions like ReLU and sigmoid, explains neural networks' practical effectiveness in complex tasks. The theorem, however, states a pure existence result under the assumption of unlimited width; it says nothing about the accelerator executing the computation. Physical ML systems impose hard limits on both dimensions of the width-vs.-depth trade-off: a single hidden layer wide enough to approximate a complex function requires parameter storage and activation memory that can exhaust the accelerator's device memory (VRAM) entirely, and a network too deep to fit its activations in memory stalls the backward pass on the same constraint. The engineering problem is therefore not whether a network of sufficient width or depth exists, but whether it fits within the memory and bandwidth envelope of the target hardware.

23 **Universal approximation theorem**: Under conditions on the activation, target function, compact domain, and approximation tolerance, a sufficiently wide single-hidden-layer network can approximate the target arbitrarily well. This existence result is nonconstructive: it does not specify how to find the weights or how many neurons practical training will require. Why depth can nonetheless beat width is the subject of section 5.4.5.

5.4.2.4 Layers and connections

Individual neurons compute weighted sums, apply bias terms, and pass results through activation functions. The power of neural networks, however, comes from organizing these neurons into layers, collections of units that execute in parallel across shared inputs. An input layer ingests raw feature tensors; one or more hidden layers transform those representations through successive matrix multiplications and nonlinear activations; and an output layer emits task-specific predictions or decision logits.

In the feedforward schematic of figure 5.12, the task fixes only the two ends. The input layer's width matches the four input features and the output layer's width matches the two predictions. The hidden layers between them can widen and narrow freely, which is why hidden width is a design decision rather than a property of the data. Each hidden layer applies a parameterized linear transformation followed by an element-wise nonlinearity, progressively constructing higher-level abstractions from raw input signals.

As data flows through the network, it is transformed at each layer to extract meaningful patterns. The weighted summation and activation process introduced for individual neurons scales up: each layer applies these operations in parallel across all its neurons, with outputs from one layer becoming inputs to the next. This creates a hierarchical pipeline where simple features detected in early layers combine into increasingly complex patterns in deeper layers, enabling neural networks to learn sophisticated representations from raw data.

5.4.3 Parameters and connections

The learnable parameters[24] of neural networks, weights and biases, determine how information flows through the network and how transformations are applied to input data. Their organization directly impacts both learning capacity and computational requirements.

5.4.3.1 Weight matrices

Weights determine how strongly inputs influence neuron outputs. In dense layers, these weights organize into matrices that map efficiently to linear-algebra kernels. In a layer with d_{in} input features

24 **Parameter memory cost**: Parameter count is a misleading proxy for memory importance. The normalization layers that section 6.10.2.3 introduces, for example, add a few learned scale and shift parameters per layer that barely affect byte budgets but can strongly affect model behavior when training choices change. Conversely, the bulk of parameters in dense weight matrices require extra state during training: gradients plus optimizer bookkeeping beyond the stored weights themselves. A model that fits in memory for inference may therefore require several times more memory for training, a cost quantified later in the training memory budget.

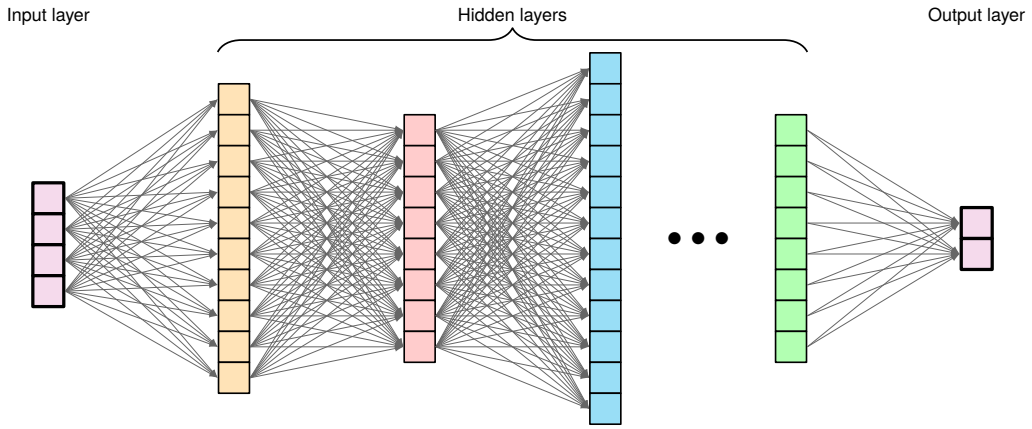


Figure 5.12: Layered Network Architecture: Deep neural networks transform data through successive layers, enabling the extraction of increasingly complex features and patterns. Each layer applies nonlinear transformations to the outputs of the previous layer, ultimately mapping raw inputs to desired outputs.

and d_{out} neurons, the weights form a matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, where each column represents the weights for a single neuron. A matrix-vector product processes one input, while a matrix-matrix product processes a batch of inputs and can expose more parallel work and data reuse.

For a single neuron j, preactivation is the scalar dot product $z_j = \sum_{i=1}^{d_{\text{in}}} (x_i \cdot w_{ij}) + b_j$. For a layer of d_{out} neurons, evaluating units independently wastes opportunities for SIMD vectorization and cache reuse. Formulating the layer as a matrix multiplication computes all d_{out} preactivations in a single dense linear-algebra call (equation 5.8):

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} \tag{5.8}$$

This equation computes every neuron's preactivation for one input: the vector $\mathbf{x}$ multiplied by the weight matrix $\mathbf{W}$ produces all d_{out} values, and the bias vector $\mathbf{b}$ shifts each one. For a batch, the input becomes a matrix and the same layer becomes a matrix-matrix product. These kernels account for much of the work in dense feed-forward networks. Whether they dominate runtime, and whether they are compute bound or memory bound, depends on their dimensions, batch size, precision, implementation, and target hardware.

This matrix organization is more than mathematical convenience; it reflects how modern neural networks are implemented for efficiency. Each weight w_{ij} represents the strength of the connection between input feature i and neuron j in the layer.

The canonical case studied here connects each neuron in a layer to every neuron in the previous layer, forming a "dense" or "fully-connected" layer. This pattern allows each neuron to use all features from the previous layer. Fully connected layers establish the mathematical principles needed here, while alternative connectivity patterns exploit structure in the data and can reduce work or improve statistical efficiency.

Figure 5.13 details this dense interconnect topology on a concrete two-layer network ($3 \to 4 \to 2$ units). Layer counts in this chapter count layers of weights, not layers of units, so this network has two and the MNIST network ($784 \to 128 \to 64 \to 10$) has three. The figure labels every scalar weight w_{ij} and layer bias b_j alongside intermediate node activations. Reading the incoming edges for any single destination neuron isolates one column of the weight matrix, while evaluating all destinations in parallel forms the complete matrix-vector multiplication. The numerical values illustrate parameter placement rather than an active inference pass; the essential systems property is the matrix dimension. For layers of widths (n_0, n_1, n_2), with n_0 the input width, the weight matrices span $\mathbf{W}^{(1)} \in \mathbb{R}^{n_0 \times n_1}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{n_1 \times n_2}$, requiring $n_0 n_1 + n_1 n_2$ weights before biases.

5.4.3.2 Bias terms

Each neuron in a layer also has an associated bias term[25]. While weights determine the relative

[25] **Bias terms**: Biases add one parameter per neuron, compared with d_{in} weights for a neuron with d_{in} inputs, so they account for $1/(d_{\text{in}} + 1)$ of that layer's parameters. Their model-wide share and effect on accuracy depend on layer widths, architecture, and normalization. Some modern architectures omit biases in layers followed by batch normalization because the normalization transform includes a learned shift.

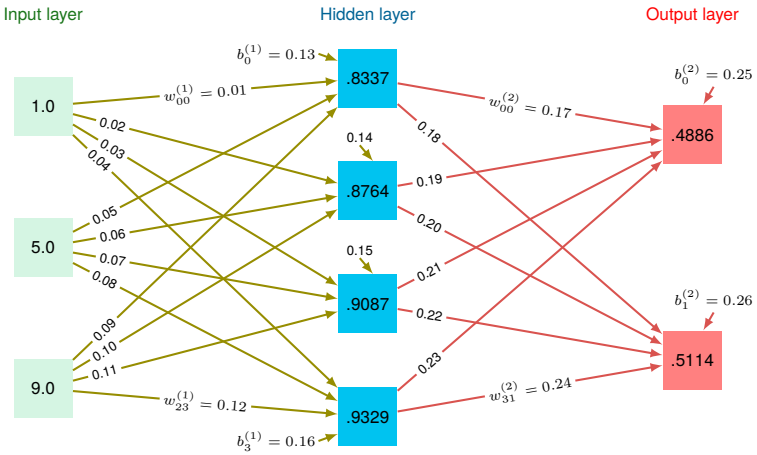


Figure 5.13: Fully-Connected Layers: A two-layer network (three layers of units) with dense connections between layers, where each neuron integrates information from all neurons in the preceding layer. Labeled edges and nodes illustrate the organization of weights, biases, and activations; they are not a worked numerical forward pass.

importance of inputs, biases allow neurons to shift their activation functions. This shifting is important for learning, as it gives the network flexibility to fit more complex patterns.

For a layer with d_{out} neurons, the bias terms form a vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$. During layer execution, this bias vector is the $\mathbf{b}$ added to the weighted sum in equation 5.8, shifting each neuron's activation threshold independently at the cost of one addition per output value.

The organization of weights and biases across a feed-forward network follows a systematic pattern. For each parameterized affine layer ℓ, three components define its computation; the weights and biases are learned, while the activation function is usually chosen by the designer:

- A weight matrix $\mathbf{W}^{(\ell)}$
- A bias vector $\mathbf{b}^{(\ell)}$
- An activation function $f^{(\ell)}$

This yields the complete layer computation in equation 5.9:

$$\mathbf{a}^{(\ell)} = f^{(\ell)}(\mathbf{z}^{(\ell)}) = f^{(\ell)}(\mathbf{a}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \tag{5.9}$$

Here, $\mathbf{a}^{(\ell)}$ (written as $\mathbf{A}^{(\ell)}$ for batches) represents the layer's activation output. The row-vector convention is adopted throughout: each sample is a row, and the weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ maps from the previous layer's width to the current layer's width. With this equation in place, the core architecture concepts are complete enough to proceed.

Checkpoint 5.2: Neural network architecture fundamentals

Before proceeding to network topology and training, verify your understanding of the foundational concepts:

Use the MNIST architecture $784 \to 128 \to 64 \to 10$ as the running check.

- ☐ **Neuron equation**: can you write the neuron equation, including weighted sum, bias, and activation function?
- ☐ **ReLU vs. sigmoid**: can you explain why ReLU is computationally cheaper than sigmoid while still providing the nonlinearity a multilayer network needs?
- ☐ **Layers to matrices**: can you map input, hidden, and output layers to the corresponding weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$?
- ☐ **Parameter count**: can you calculate the total parameter count, including both weights and biases?

- ☐ **Width, depth, cost**: can you explain how layer width, depth, and connectivity change memory footprint and arithmetic cost?

If any of these feel unclear, review the earlier sections on neural network fundamentals, artificial neurons, and parameters before continuing. The upcoming sections on training and optimization build directly on these foundations.

5.4.4 Architecture design

Architecture design determines how individual neurons, activation functions, and weight matrices connect to solve problems that no single layer can handle. That design has direct systems consequences, because each topological choice (adding a layer, widening a hidden dimension, changing connectivity) changes the parameter count, memory footprint, and arithmetic cost.

Network topology describes how individual neurons organize into layers and connect to form complete neural networks. Building intuition begins with a simple problem that became famous in AI history.[26]

[26] **XOR problem**: XOR is the canonical example of why network topology matters. A single linear-threshold layer cannot represent this nonlinearly separable function, regardless of how many units it contains, whereas a small network with a nonlinear hidden layer can. The exact number of units required depends on the activation and representation conventions; the durable lesson is that adding nonlinear depth changes the class of functions the network can express.

Example 5.2: Building intuition: The XOR problem

Consider a network learning the XOR function, a classic problem that requires nonlinearity. With inputs x_1 and x_2 that can be 0 or 1, XOR outputs 1 when inputs differ and 0 when they are the same.

Setup: Two inputs → two hidden neurons → one output

Example: With hidden units $h_1 = \text{ReLU}(x_1 + x_2)$ and $h_2 = \text{ReLU}(x_1 + x_2 - 1)$, the output $y = h_1 - 2h_2$ gives 0, 1, 1, 0 for inputs $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$. The input $(1,1)$ drives both hidden units, and the second one cancels the first. No single weighted sum of x_1 and x_2 can produce that pattern.

Systems insight: Hidden layers do not merely add capacity; they change the class of functions the system can represent. XOR is the small example that makes depth a structural requirement rather than a decorative design choice (Minsky and Papert 1969).

The XOR example motivates a network with a hidden layer, but real-world networks require systematic consideration of design constraints and computational scale. The chapter's MNIST digit recognizer shows how problem structure fixes some network dimensions while leaving the hidden layers as a design decision.

5.4.4.1 Feedforward network architecture

Applying the three-layer architecture to MNIST reveals how data characteristics and task requirements constrain network design. The dual-panel layout in figure 5.14 contrasts the conceptual and physical representations of this input mapping: panel (a) routes a 28×28 pixel grayscale image of digit 7 into hidden units and ten class outputs, while panel (b) illustrates the physical flattening of those 784 pixels into a contiguous one-dimensional column vector before feeding the input layer. Fully connected layers assign an independent weight to every coordinate of this 784-element vector, flattening out spatial coordinates and discarding pixel adjacency. Convolutional architectures instead encode spatial locality directly. That choice is an *inductive bias*, a built-in structural assumption about the data (here, that nearby pixels matter together) which restricts the functions a model can learn toward those expected to be useful (Mitchell 1980). The fully connected network makes almost no such assumption, so it must learn from data the structure that a stronger bias would supply, and it pays for that learning in parameters.

↪ How spatial locality and weight sharing replace the parameter overhead of fully connected layers is developed as a spatial inductive bias in section 6.5.

The input layer's width is directly determined by the data format. For a 28 by 28 pixel image, each pixel becomes an input feature, requiring 784 input neurons. Equivalently, 28 rows and 28 columns flatten into 784 values. The data can be represented either as a 2D spatial grid of pixels or as a flattened 1D vector, where each value represents the intensity of one pixel.

The output layer's structure is determined by the task requirements. For 10-class digit recognition, the final layer provisions 10 output neurons, one for each possible digit (0–9). When presented with

an image, the network produces a score for each output neuron; a softmax can then convert those logits into predicted class probabilities.

Between these fixed input and output layers, there is flexibility in designing the hidden layer topology. The choice of hidden layer structure, including the number of layers to use and their respective widths, represents one of the key design decisions in neural networks. Additional layers increase the network's depth, allowing it to learn more abstract features through successive transformations. The width of each layer provides capacity for learning different features at each level of abstraction.

Widening both hidden layers from 100 to 1000 neurons, comparing a $784 \to 100 \to 100 \to 10$ network with a $784 \to 1000 \to 1000 \to 10$ network, increases parameters from ~89.6K to ~1.8M (~358 KB vs. ~7 MB in FP32). Whether the wider network improves accuracy requires training and evaluation; the storage increase alone already matters for mobile deployment budgets.

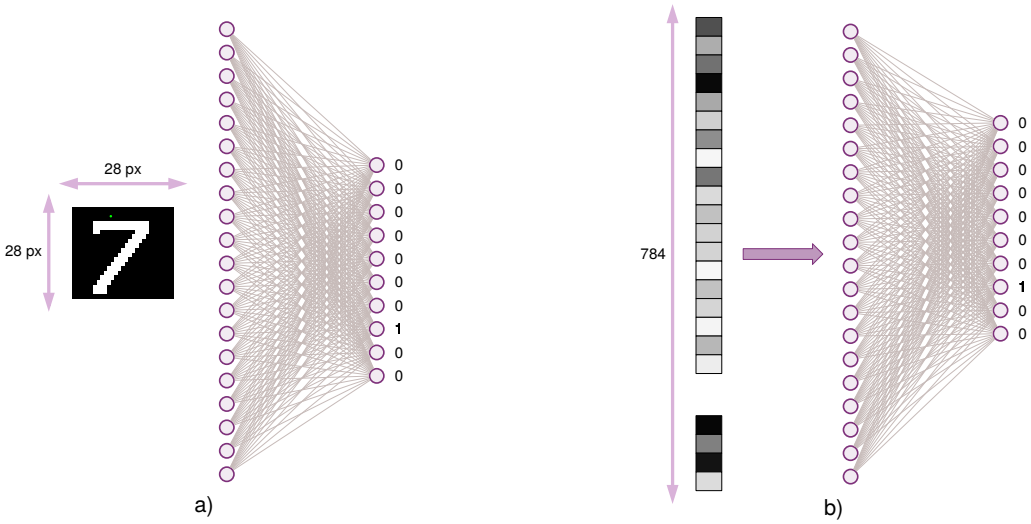


Figure 5.14: MNIST Network Topology: Two panels show the network architecture for digit recognition. Panel (a) schematically represents a 28×28 pixel image of a digit connected through a schematic hidden layer to 10 output nodes. Panel (b) shows the same architecture with the input image flattened into a 784-element vector, illustrating how spatial data enters the network.

5.4.4.2 Layer connectivity design patterns

The fully connected architecture in section 5.4.3 maximizes flexibility by connecting every neuron to every neuron in the next layer, but connectivity itself is an engineering decision. Dense, sparse, and skip patterns trade learning flexibility against parameter count, locality, and gradient flow.

Dense connectivity represents the standard pattern where each neuron connects to every neuron in the subsequent layer. In the MNIST example, connecting the 784-dimensional input layer to a hidden layer of 128 neurons requires 100,352 weight parameters (784×128). This full connectivity enables the layer to learn arbitrary linear combinations of its inputs; nonlinear activations between dense layers extend those combinations to nonlinear relationships. The number of parameters grows as the product of adjacent layer widths, so widening one layer scales them linearly while widening both scales them quadratically.

Sparse connectivity patterns introduce purposeful restrictions in how neurons connect between layers. Rather than maintaining all possible connections, neurons connect to only a subset of neurons in the adjacent layer. This approach draws inspiration from biological neural systems, where neurons typically form connections with a limited number of other neurons. In visual processing tasks like the MNIST example, neurons might connect only to inputs representing nearby pixels, reflecting the local nature of visual features.

As networks grow deeper, the path from input to output becomes longer, potentially complicating the learning process. Skip connections address this by adding direct paths between nonadjacent

layers. These connections provide alternative routes for information flow, supplementing the standard layer-by-layer progression. In a digit recognition pipeline, skip connections allow later layers to reference both high-level patterns and the original pixel values directly.

These connection patterns have direct hardware consequences as well as theoretical ones. Dense connections maximize learning flexibility and map efficiently onto the dense matrix-multiplication kernels that accelerators are built to execute. Sparse connections reduce theoretical parameter counts and FLOPs, but a sparse weight matrix does not automatically execute faster than a dense one on standard hardware. Unless the hardware supports a regular, structured sparsity pattern that its kernels can predict, the arithmetic reduction does not translate to a proportional reduction in execution time. Skip connections help maintain effective information flow in deeper networks while preserving the gradient paths that make very deep architectures trainable.

5.4.4.3 Model size and computational complexity

How parameters (weights and biases) are arranged determines both learning capacity and computational cost. The parameter count, its numerical precision, and the operations it requires are the concrete terms of the model's side of the silicon contract introduced at the start of the chapter. While topology defines the network's structure, the parameter count and its byte width set the memory the model occupies.

Parameter count grows with network width and depth. In the MNIST network, an input layer with 784 dimensions, hidden layers of 128 and 64 neurons, and a 10-neuron output layer ($784 \to 128 \to 64 \to 10$) scale the parameter footprint. The first layer requires 100,352 weights and 128 biases, the second layer 8,192 weights and 64 biases, and the output layer 640 weights and 10 biases, totaling 109,386 parameters, or 438 KB at four bytes each in FP32. Each must be stored in memory, read on every forward pass, and updated during learning.

5.4.4.4 Parameter initialization

Parameter count tells the engineer what a model must store; it says nothing about whether the model will *learn*. That depends on how those parameters are first set. Parameter initialization is critical to network behavior. Setting all parameters to zero would cause neurons in a layer to behave identically, preventing diverse feature learning. Instead, weights are typically initialized randomly, often using specific strategies like Xavier/Glorot initialization[27] (Glorot and Bengio 2010) or He initialization (He et al. 2015), while biases often start at small constant values or zeros. The scale of these initial values matters because every layer multiplies its input by its weights. Weights that are systematically too large or too small compound across layers into exploding or vanishing signals, the same multiplicative failure that saturating activations cause (section 5.4.2.2).

The distribution of parameters affects information flow through layers. In digit recognition, if weights are too small, important input details might not propagate to later layers. If too large, the network might amplify noise. Biases help adjust the activation threshold of each neuron, enabling the network to learn optimal decision boundaries.

Different architectures impose specific constraints on parameter organization. Some share weights across network regions to encode position-invariant pattern recognition; others restrict certain weights to zero, implementing sparse connectivity patterns.

[27] **Xavier/Glorot initialization**: For a layer with d_{in} inputs and d_{out} outputs, a common Glorot choice uses weight variance $2/(d_{\text{in}} + d_{\text{out}})$ to balance forward activations and backward gradients (Glorot and Bengio 2010). The fix costs zero additional FLOPs; it is purely a matter of setting the random distribution at startup.

5.4.5 Why depth matters: The power of hierarchical representations

A single-layer network attempting to classify handwritten digits must map raw pixels directly to labels. A deeper network can instead exploit hierarchical structure when the target function is compositional.

Deep networks can exploit **compositionality**: complex patterns may decompose into simpler patterns that themselves decompose further. In image recognition, learned representations can progress from local edges and textures toward parts and objects. Such hierarchical regularities occur in many datasets and motivate layered models, although a trained network need not recover this exact sequence.

Consider recognizing the digit "seven" in the MNIST example. A single-layer network maps all 784 pixel values directly to a decision. When the task has compositional structure, a deep network can compute the decision more parameter-efficiently by reusing intermediate features:

- **Layer 1** learns simple edge detectors: vertical lines, horizontal lines, diagonal strokes
- **Layer 2** combines edges into shapes: the horizontal top stroke of a "seven," the diagonal downstroke
- **Layer 3** combines shapes into complete digit patterns

Each layer builds on the previous, allowing later computations to reuse earlier features. The same edge detectors learned for "seven" can also contribute to recognizing "one," "four," and other digits. This **parameter reuse** helps explain why depth is an effective design choice without implying a fixed parameter advantage for every task. However, the choice between adding layers and widening existing ones is not symmetric: depth and width contribute to representational capacity through different mechanisms.

Layered processing in biological vision helped motivate hierarchical models, without implying that artificial networks reproduce the same mechanisms. Specialized architectures encode different structural assumptions for images, sequences, and other data types. Depth explains why layered representations can be useful; the neuron, layer, and parameter mechanics developed above explain how the hierarchy is implemented, and the learning process that follows explains how training discovers the features at each level.

> Systems Perspective 5.2: The depth vs. width trade-off
>
> For certain function classes, depth provides an exponential representational advantage: a network with N_L layers can represent functions that require exponentially more units in a shallow network (Telgarsky 2016). This restricted theoretical result explains why nonlinear composition can be parameter-efficient; it is not a universal guarantee for every task.
>
> However, depth introduces three engineering challenges with each additional layer:
>
> - Adds sequential dependencies (layer $\ell+1$ waits for layer ℓ), limiting parallelism
> - Increases gradient path length, risking vanishing/exploding gradients
> - Requires storing intermediate activations for backpropagation, the backward gradient pass of section 5.5.5
>
> Modern architectures balance depth against width. A network with ten layers of 100 neurons has the same 1,000 total hidden units as one with two layers of 500, but not the same parameter count, function class, or execution graph. Both expose parallelism within each layer; the deeper network adds sequential dependencies and may represent some compositional functions more efficiently.

Network architecture, neurons, and parameters are now in place, but nothing yet explains how randomly initialized parameters become useful. Architecture answers *what* the model computes; training answers *how* it learns. The mechanics reveal systems constraints because training adds backward computation, saved activations, gradients, and optimizer state. Batch size then becomes a joint statistical and hardware choice, because it changes gradient noise, activation memory, data reuse, and parallel efficiency at once.

5.5 Learning Process

The MNIST network currently holds 109,386 parameters set to random values, numbers that encode no knowledge at all. Learning turns them into a working classifier by repeating one short cycle of computation millions of times, and that repetition multiplies every cost the previous sections counted for a single forward pass.

5.5.1 Supervised learning from labeled examples

A randomly initialized network classifies digits no better than random guessing among ten classes (about 10 percent accuracy). Transforming it into a 95 percent-accurate classifier requires supervised learning: showing the network labeled examples and adjusting its weights based on the errors it makes. Consider the MNIST digit recognition task: the dataset contains 60,000 training images,

each a 28×28 pixel grayscale image paired with its correct digit label. The network must learn the relationship between these images and their corresponding digits through an iterative process of prediction and weight adjustment.

The relationship between inputs and outputs drives the training methodology. Training operates as a loop where each iteration processes a subset of training examples called a batch.[28] For each batch, the network performs four operations: forward computation through the network layers generates predictions, a loss function measures prediction error under the chosen objective, backpropagation computes parameter gradients, and an optimizer uses those gradients to update the weights.

The iterative approach can be expressed mathematically. Given an input image $\mathbf{x}$ and its true label $\mathbf{y}$, the network computes its prediction according to equation 5.10:

$$\hat{\mathbf{y}} = f_{\text{net}}(\mathbf{x}; \theta) \tag{5.10}$$

This equation encapsulates the entire forward pass: the network f_{net} takes an input $\mathbf{x}$ (say, a 28×28 digit image) and, using its current parameters θ (weights and biases across all layers), produces a prediction $\hat{\mathbf{y}}$ (a vector of ten probabilities, one per digit). The semicolon notation $f_{\text{net}}(\mathbf{x}; \theta)$ distinguishes the input $\mathbf{x}$, which changes with every example, from θ, which remains fixed during inference but evolves during training. The network's error is measured by a loss function[29] $\mathcal{L}$ (equation 5.11):

$$\text{loss} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) \tag{5.11}$$

The error measurement drives the adjustment of network parameters: gradient descent (section 5.5.4) moves each parameter against its loss gradient, and backpropagation (section 5.5.5) computes those gradients. In practice, training operates on batches of examples rather than individual inputs. For the MNIST dataset, each training iteration might process 32, 64, or 128 images simultaneously for reasons developed in section 5.5.4.1. The training cycle continues until the network achieves sufficient accuracy or reaches a predetermined number of iterations. Establishing proper metrics and evaluation protocols is essential for assessing training effectiveness (section 12.7.2.1).

5.5.2 Forward pass computation

An MNIST image becomes ten class scores by moving through weighted layers and nonlinear activations. That computation is **forward propagation**: input data flows through the network's layers to generate predictions. During training, those predictions are paired with ground-truth targets at the loss function, initiating the closed-loop optimization cycle diagrammed in figure 5.15. The forward pass itself underlies both inference and training.

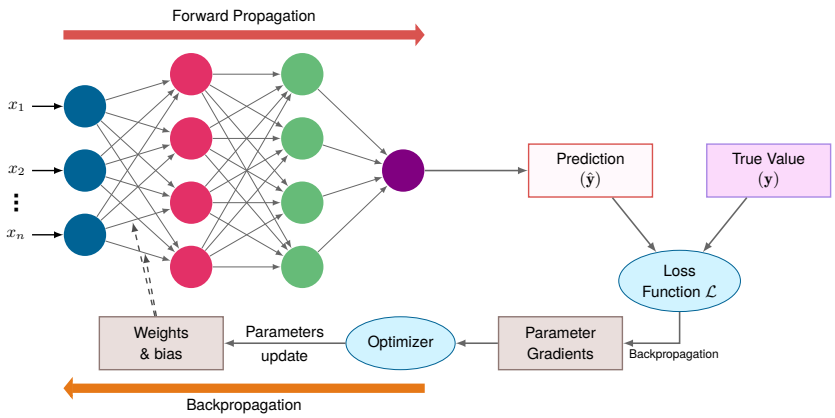


Figure 5.15: Training Loop Architecture: A prediction and true value meet at the loss function; backpropagation computes parameter gradients, and the optimizer turns those gradients into parameter updates for the next forward pass.

Tracing dataflow along the red horizontal arrow in figure 5.15 shows forward activations traversing the layers to evaluate loss, whereas the orange arrow running the opposite way carries gradients backward through the layers; the optimizer then turns those gradients into weight updates. This

[28] **Batch processing**: Batching serves dual purposes. Larger batches average gradient noise across more examples and can improve hardware utilization by turning matrix-vector work into larger matrix-matrix kernels. They also perform more total work and usually increase latency for an individual request; the throughput gain depends on shape and hardware. Each doubling of batch size roughly doubles activation memory, making batch size a joint optimization and systems decision rather than a purely statistical or hardware choice.

[29] **Loss function**: Formalized by Abraham Wald in statistical decision theory as the "cost" of an incorrect decision, $\mathcal{L}$ quantifies the gap between prediction $\hat{\mathbf{y}}$ and ground truth $\mathbf{y}$. The choice of loss function shapes the optimization geometry: it determines the gradient landscape that backpropagation must navigate. A loss with flat regions near incorrect predictions produces weak gradients that stall learning, while a loss with steep gradients near the decision boundary accelerates convergence where it matters most, a systems consequence explored in section 5.5.3.

↪ How long this cycle takes to reach a target accuracy, the time-to-accuracy measure that training benchmarks prefer over raw throughput, is developed in section 12.7.2.1.

operational asymmetry underscores a critical systems requirement: training must retain intermediate forward activations in memory until the corresponding backward pass completes.

When an image of a handwritten digit enters the network, it undergoes a series of transformations through the layers. Each transformation combines the weighted inputs with learned patterns to progressively extract relevant features. For the 784-128-64-10 digit classifier, a 28×28 pixel image is processed through multiple layers to ultimately produce probabilities for each possible digit (0–9).

The process begins with the input layer, where each pixel's grayscale value becomes an input feature. For MNIST, this means 784 input values ($28\times28 = 784$), each normalized between 0 and 1. These values then propagate forward through the hidden layers, where each neuron combines its inputs according to its learned weights and applies a nonlinear activation function.

Each forward pass through the MNIST network (784-128-64-10) requires substantial matrix operations. The first layer alone performs 100,352 MACs per sample. When processing multiple samples in a batch, these operations multiply accordingly, requiring careful management of memory bandwidth and computational resources. Specialized hardware like GPUs executes these operations efficiently through parallel processing.

5.5.2.1 Individual layer processing

The forward computation through a neural network proceeds systematically, with each layer transforming its inputs into increasingly abstract representations. The digit classifier illustrates this: its transformation process occurs in distinct stages. At each layer, the computation involves two key steps: a linear transformation of inputs followed by a nonlinear activation. The linear transformation applies the same weighted sum operation defined in equation 5.2, but now using notation that tracks layer indices (equation 5.12):

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \tag{5.12}$$

Here, $\mathbf{A}^{(\ell-1)}$ contains the activations from the previous layer (the outputs after applying activation functions), $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1}\times n_\ell}$ is the weight matrix for layer ℓ, and $\mathbf{b}^{(\ell)}$ is the bias vector (broadcast across the batch). The superscript (ℓ) keeps track of which layer each parameter belongs to. This row-vector convention matches the single-sample equation in equation 5.2: each row of $\mathbf{A}$ is one sample, and right-multiplying by $\mathbf{W}$ transforms it to the next layer's width.

Following this linear transformation, each layer applies a nonlinear activation function f (here, f or $f^{(\ell)}$ denotes a generic activation function at layer ℓ), as expressed in equation 5.13:

$$\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)}) \tag{5.13}$$

This process repeats at each layer, creating a chain of transformations:

$$\text{Input} \to \text{Linear Transform} \to \text{Activation} \to \text{Linear Transform} \to \text{Activation} \to \cdots \to \text{Output}$$

Returning to digit recognition, the pixel values first undergo a transformation by the first hidden layer's weights, converting the 784-dimensional input into an intermediate representation. Each subsequent layer further transforms this representation, ultimately producing a 10-dimensional vector of class scores, one for each possible digit.

5.5.2.2 Matrix multiplication formulation

The complete forward propagation process can be expressed as a composition of functions, each representing a layer's transformation. Formalizing this mathematically builds on the MNIST example.

For a network with N_L layers, the full forward computation is expressed as the composition in equation 5.14:

$$\mathbf{A}^{(N_L)} = f^{(N_L)}\Big(\cdots f^{(2)}\Big(f^{(1)}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\Big)\cdots\mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}\Big) \tag{5.14}$$

MatMul >99%

Element-wise <1%

Matrix multiplication is over 99 percent of this network's forward-pass FLOPs.

This composition reveals that forward propagation is, at its core, a chain of matrix multiplications interleaved with nonlinear activations. The nested expression unfolds layer by layer, each step consuming the previous layer's activations as its input:

1. First layer:

$$\mathbf{Z}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$
$$\mathbf{A}^{(1)} = f^{(1)}(\mathbf{Z}^{(1)})$$

2. Hidden layers ($\ell = 2, \ldots, N_L - 1$):

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$
$$\mathbf{A}^{(\ell)} = f^{(\ell)}(\mathbf{Z}^{(\ell)})$$

3. Output layer:

$$\mathbf{Z}^{(N_L)} = \mathbf{A}^{(N_L-1)}\mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}$$
$$\mathbf{A}^{(N_L)} = f^{(N_L)}(\mathbf{Z}^{(N_L)})$$

For a batch of B images in the MNIST network, four tensor dimensions govern the forward pass:

- Input $\mathbf{X}$: $B \times 784$
- First layer weights $\mathbf{W}^{(1)}$: $784 \times n_1$
- Hidden layer weights $\mathbf{W}^{(\ell)}$: $n_{\ell-1} \times n_\ell$
- Output layer weights $\mathbf{W}^{(N_L)}$: $n_{N_L-1} \times 10$

Of the two kinds of operation in that chain, the matrix multiplications dominate AI computation, and arithmetic intensity explains why. Intensity determines whether a kernel is compute or memory bound. Tiled dense matrix multiplication reuses each loaded value many times, whereas an unfused element-wise operation does little work per byte it moves. Writing s_{elem} for the bytes needed to store one value (4 bytes in FP32), table 5.6 compares their idealized intensities.

Table 5.6: Idealized Arithmetic Intensity of Matrix Multiply vs. Element-Wise ReLU: A tiled dense $N \times N$ matrix multiplication performs $2N^3$ operations while moving at least about $3N^2 s_{\text{elem}}$ bytes for s_{elem} bytes per value, so potential reuse grows with N. A standalone element-wise activation remains at $1/(2s_{\text{elem}})$ FLOP/byte (0.125 FLOP/byte for FP32); implementations often fuse it with a neighboring kernel to avoid a separate memory round trip.

Operation	Operation Count (O)	Minimum Data Movement	Idealized Intensity	Typical Optimization
Matrix Mul ($N \times N$)	$2N^3$	$3N^2 s_{\text{elem}}$ bytes	$\approx 2N/(3s_{\text{elem}})$	Tiling on CPU/GPU/TPU
Element-wise (ReLU)	N^2	$2N^2 s_{\text{elem}}$ bytes	$1/(2s_{\text{elem}})$	Vectorization and fusion

Systems Perspective 5.3: Why matrix multiplication dominates AI

Modern accelerators have large compute arrays relative to off-chip memory bandwidth. They approach peak arithmetic throughput only when operations expose enough reuse and parallel work. Dense and convolutional layers map well to optimized kernels; custom element-wise sequences are often fused so that their intermediate values remain on chip.

For a single sample, the layer operation **xW** is a **General Matrix-Vector Multiply (GEMV)**. Batching multiple samples turns this into **XW**, a **General Matrix Multiply (GEMM)**. Modern accelerators are specialized hardware engines built to maximize GEMM throughput, packing wide parallel units and high-bandwidth memory hierarchies optimized for 2D matrix transformations. Because hardware vendors invest the bulk of their silicon area and compiler optimizations into maximizing GEMM throughput, almost every neural network layer (including the convolution and attention layers of later chapters) is structured to execute as matrix multiplication. Matrix operations account for over 99 percent of the floating-point operations in this chapter's dense network and dominate modern neural workloads. To maximize perfor-

mance, hardware execution backends use techniques like blocking and tiling to maximize data reuse across on-chip caches. Designing model architectures around operations that specialized matrix engines can execute efficiently is the essence of modern hardware-software co-design. Section C.1.4 works through how GEMM arithmetic intensity grows with matrix dimension.

5.5.2.3 Step-by-step computation sequence

Understanding how these mathematical operations translate into actual computation requires examining the forward propagation process for a batch of MNIST images. This process illustrates how data transforms from raw pixel values to digit predictions.

Consider a batch of 32 images entering the network. Each image starts as a 28×28 grid of pixel values, which flattens into a 784-dimensional vector. For the entire batch, this yields an input matrix $\mathbf{X}$ of size 32×784, where each row represents one image. The values are typically normalized to lie between 0 and 1.

The transformation at each layer proceeds as a shape-changing sequence; the shapes determine both kernel choice and activation storage. The network first takes the input matrix $\mathbf{X}$ (32×784) and transforms it using the first layer's weights. If the first hidden layer has 128 neurons, $\mathbf{W}^{(1)}$ is a 784×128 matrix, so the computation $\mathbf{XW}^{(1)}$ produces a 32×128 matrix. Each element in this matrix then has its corresponding bias added and passes through an activation function. For example, with a ReLU activation, any negative values become zero while positive values remain unchanged. This nonlinear transformation enables the network to learn complex patterns in the data. The final layer transforms its inputs into a 32×10 matrix, where each row contains 10 class scores. Let z_j denote the raw score (logit) for digit j and let z_k range over all 10 digits. Often, these scores are converted to probabilities using the softmax function in equation 5.6:

$$\hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^{10} e^{z_k}}$$

For each image in the batch, the ten values $\hat{y}_j$ form a probability distribution over the possible digits. The digit with the highest probability represents the network's prediction.

Napkin Math 5.1: Counting operations in forward pass

Problem: Determine the total arithmetic operation count (O) for one forward pass through the MNIST network (784 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 10) with batch size 32.

Background: A matrix multiplication of dimensions $(M\times K)\times(K\times N)$ requires $2\times M\times K\times N$ FLOPs (one multiply and one add per output element, summed over K terms). Bias addition adds $M\times N$ floating-point additions. ReLU activation adds $M\times N$ comparisons, so the table reports total operation-equivalent work rather than pure FLOPs for activation rows.

Solution: Table 5.7 breaks down the operation count layer by layer. The total comes to ~7 MOp, or 7 MOp ÷ 32 = ~219 KOp per image. This per-image figure reconciles with the count at the start of the chapter. The three matrix-multiplication rows contribute 218,368 FLOPs per image, exactly two FLOPs for each of the 109,184 MACs, and the bias and activation rows add less than 1 percent on top.

Systems insight:

1. Layer 1 dominates: The first layer accounts for 91.9 percent of all operations because it processes the largest input (784 dimensions). This is why dimensionality reduction in early layers is so impactful.
2. Compute vs. memory: At batch 1, 219 KOp must amortize a parameter read for only one image, giving an arithmetic intensity of ~0.5 FLOP/byte under this traffic model. At batch 32, one parameter read, one input read, one final-output write, and a write followed by a read for each hidden tensor move ~588 KB and raise the intensity to ~11.9 FLOP/byte. This idealized FP32 model treats each dense layer as a separate kernel with fused bias and activation, and counts transfers across a memory boundary at which no operands remain

resident between kernels or batches. A hidden activation contributes twice because its producer writes it and its consumer reads it. Cache residency or cross-layer fusion can reduce traffic at that boundary; extra transfers can increase it. The comparison shows why batching improves reuse, but memory- or compute-bound behavior still depends on the machine's ridge point, the balance $R_{\text{peak}}/\text{BW}$ introduced with the iron law, which the Roofline model in section D.2.1 develops.

3. Scaling intuition: Doubling the hidden layer widths ($784 \to 256 \to 128 \to 10$) increases the operation count by about 2.1× to about 15 MOp. This comes from recomputing each layer: layers 1 and 3 double, layer 2 quadruples, so the total grows by about 2.15× rather than four times.

Table 5.7: MNIST Forward Pass Operation Count by Layer: Operation-equivalent breakdown for a single forward pass through the 784-128-64-10 network at batch size 32. Matrix multiplication rows are FLOPs; bias, ReLU, and softmax rows are elementwise operations included to show their negligible contribution. Layer 1's matrix multiply dominates because it multiplies the 784-dimensional input against the widest weight matrix; bias and activation operations contribute under 1 percent of the total.

Layer	Operation	Dimensions	Operations
Layer 1	MatMul	$(32\times784) \times (784\times128)$	$2 \times 32 \times 784 \times 128 = 6{,}422{,}528$
Layer 1	Bias + ReLU	32×128	$2\times4{,}096 = 8{,}192$
Layer 2	MatMul	$(32\times128) \times (128\times64)$	$2 \times 32 \times 128 \times 64 = 524{,}288$
Layer 2	Bias + ReLU	32×64	$2\times2{,}048 = 4{,}096$
Layer 3	MatMul	$(32\times64) \times (64\times10)$	$2 \times 32 \times 64 \times 10 = 40{,}960$
Layer 3	Bias + Softmax	32×10	~640 (simplified)
Total			**~7 MOp**

5.5.2.4 Implementation and optimization considerations

Forward propagation is easy to state mathematically, but its implementation is constrained by activation storage, batch size, memory layout, and hardware fit. During training, each layer's activations must stay in memory until the backward pass consumes them. For the MNIST network at batch size 32, the input holds $32\times784 = 25{,}088$ values, the hidden layers hold 4,096 and 2,048, and the output holds 320, a total of 31,552 values per batch that section 5.5.5.4 converts into bytes alongside the other training-state terms. Every one of these counts scales linearly with batch size, so doubling the batch to 64 doubles activation storage.

The same batch size sets the other side of the trade. Notebook 5.1 estimated arithmetic intensity rising from ~0.5 FLOP/byte at batch 1 to ~11.9 FLOP/byte at batch 32, because one read of each weight now serves 32 images. Batching therefore buys reuse with memory. Memory capacity bounds how far that purchase can go, while gradient noise and per-request latency bound it from other directions. Memory layout and optimized linear-algebra libraries decide how much of that reuse an implementation actually captures, which is why parallel processors that can be fed from on-chip storage run large dense matrix operations substantially faster than CPUs, and why specialized accelerators add reduced-precision arithmetic and dataflows tailored to these patterns. Energy follows the same logic. Power alone does not determine energy per inference; the relevant quantity is power integrated over execution time for the same workload and quality target, and memory hierarchy and data movement often explain most of the difference between platforms.

Forward propagation transforms inputs into predictions, but a prediction alone is useless for learning. The training loop requires a way to measure prediction error in a form that guides weight adjustments. Loss functions fill this role: they translate the gap between prediction and reality into a single number that optimization can minimize.

5.5.3 Loss functions

The forward propagation process described in section 5.5.2 suffices for **inference**, using a pretrained model to make predictions. To train a model, however, training requires a quantitative error metric that measures how well predictions match ground truth. Loss functions quantify these errors, serving as the feedback mechanism that guides learning. They convert the abstract goal of "making good predictions" into a concrete optimization problem.

Continuing with the MNIST digit recognition example: when the network processes a handwritten digit image, it outputs ten class scores, often converted to predicted probabilities for digits 0–9. The loss function measures those predictions against the target. If an image displays a "seven," cross-entropy penalizes the model when it assigns too little probability to digit seven.

5.5.3.1 Error measurement fundamentals

A loss function assigns a scalar cost to a prediction and target under a chosen objective. Lower loss indicates a better value of that objective on the evaluated examples, but it is not the same as higher accuracy: confidence and calibration can change while the predicted class remains unchanged. During training, gradients of the loss guide parameter updates. In handwritten-digit recognition, cross-entropy penalizes predictions that assign low probability to the correct digit.

Mathematically, a loss function $\mathcal{L}$ takes two inputs: the network's predictions $\hat{\mathbf{y}}$ and the true values $\mathbf{y}$. For a single training example in digit classification, the loss measures the discrepancy between prediction and truth. When training with batches of data, the system computes the mean loss across all examples in the batch (equation 5.15):

$$\mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{y}_i) \tag{5.15}$$

where B is the batch size and $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ represents the prediction and truth for the i-th example. Averaging keeps the loss and mean-gradient scale comparable across batch sizes, although the best learning rate can still change with batch size and optimization regime. The summation also maps naturally to parallel hardware: each example's loss can be computed independently before a reduction combines them.

The choice of loss function depends on the type of task. For digit classification, the loss function must handle probability distributions over multiple classes, provide meaningful gradients that guide learning, penalize wrong predictions in proportion to their severity, and scale efficiently with batch processing. Cross-entropy loss satisfies all four requirements.

5.5.3.2 Cross-entropy and classification loss functions

For classification tasks like MNIST digit recognition, **cross-entropy loss** is a common way to compare predicted probability distributions with true class labels. The information-theoretic idea of entropy traces to Shannon (Shannon 1948); in supervised classification, cross-entropy penalizes low probability assigned to the correct class.

The logarithm gives **cross-entropy loss** its optimization behavior because probabilities near zero for the correct class incur increasingly large penalties. This shape produces strong gradients for confidently wrong predictions and makes the correct-class probability the objective's central quantity.

For a single digit image, the network outputs a probability distribution over the 10 possible digits. Ground truth is represented as a one-hot vector[30] where all entries are 0 except for a one at the correct digit's position. For instance, if the true digit is "seven," the label is $\mathbf{y} = [0,0,0,0,0,0,0,1,0,0]$.

The cross-entropy loss for this example is defined in equation 5.16:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{j=1}^{10} y_j \log(\hat{y}_j) \tag{5.16}$$

where $\hat{y}_j$ represents the network's predicted probability for digit j. Because $\mathbf{y}$ is one-hot, this simplifies to equation 5.17:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\log(\hat{y}_c) \tag{5.17}$$

where c is the index of the correct class. The loss therefore depends only on the predicted probability for the correct digit; the network is penalized based on how confident it is in the right answer.

For example, if the network predicts the following probabilities for an image of "seven":

30 **One-hot encoding**: Representing each class as a binary vector with one entry per class, exactly one of which is 1; this encoding is sparse by construction: for MNIST's 10 classes, 90 percent of each label vector is zeros. Implementations commonly store an integer class index instead of materializing that vector. For very large output spaces, methods such as sampled softmax can also reduce the output computation.

```
Predicted: [0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.8, 0.0, 0.1]
True: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
```

The loss would be $-\log(0.8)$, which is approximately 0.223. If the network were more confident and predicted 0.9 for the correct digit, the loss would decrease to approximately 0.105.

5.5.3.3 Batch loss calculation methods

The practical computation of loss involves considerations for both numerical stability and batch processing. When working with batches of data, the objective evaluates the average loss across all examples.

For a batch of B examples, the cross-entropy loss becomes equation 5.18:

$$\mathcal{L}_{\text{batch}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{10} y_{ij}\log(\hat{y}_{ij}) \tag{5.18}$$

Here, i indexes examples in the batch, j indexes the ten output classes, y_{ij} is the one-hot target indicator for class j on example i, and $\hat{y}_{ij}$ is the predicted probability for that class. Averaging over B keeps the loss scale comparable as batch size changes.

Computing this loss efficiently requires careful consideration of numerical precision. The dangerous case is not an ordinary small probability, but a probability that an unstable softmax rounds to zero. If the correct-class probability becomes 0, then $\log(0)$ becomes $-\infty$ and subsequent arithmetic can produce a not-a-number value (NaN).

Stable implementations avoid materializing a numerically fragile probability before taking its logarithm:

1. Shift logits before exponentiation, the core of the log-sum-exp trick explained in section B.2.4; equation 5.19 subtracts the largest logit from every exponent:

$$\text{softmax}(z_i) = \frac{\exp\left(z_i - \max(\mathbf{z})\right)}{\sum_j \exp\left(z_j - \max(\mathbf{z})\right)} \tag{5.19}$$

Subtracting the same $\max(\mathbf{z})$ from every logit leaves the softmax probabilities unchanged because the common factor cancels between numerator and denominator. The numerical effect is decisive: the largest shifted logit becomes zero, so the largest exponential is $\exp(0) = 1$ rather than a potentially overflowing value.

2. Prefer fused cross-entropy from logits. For target class c and $z_{\max} = \max(\mathbf{z})$, a stable implementation evaluates

$$\mathcal{L} = -(z_c - z_{\max}) + \log\sum_j \exp(z_j - z_{\max}) \tag{5.20}$$

Equation 5.20 is mathematically equivalent to $-\log \hat{y}_c$ but never rounds a tiny softmax probability to zero first. Clamping a probability or adding ϵ can be a defensive fallback, but it changes the objective and should not replace the stable formulation.

5.5.3.4 Impact on learning dynamics

With a batch size of 32 and 10 output classes, each training step processes 32 sets of 10 scores, computes a loss for each example, and averages them into one batch loss. During training, that value serves two roles. Its gradient supplies the update direction, and its trend helps diagnose whether optimization is progressing, stalling, or diverging. Task accuracy, calibration, and other evaluation metrics must still be measured separately on held-out data.

For the MNIST classifier, training loss often begins near $\log(10) \approx 2.3$ when class probabilities are close to uniform, falls rapidly during early optimization, and then improves more slowly. A low training loss indicates confidence on the training examples, not proof of generalization. Validation loss and task metrics determine whether the learned behavior transfers to held-out data.

The loss function's gradients with respect to the network's output logits provide the initial error signal that drives backpropagation. For one example, when softmax and cross-entropy are combined,

each logit gradient has a particularly simple form: the difference between the predicted and target probabilities, $\hat{y}_j - y_j$ for logit z_j. This mathematical property makes cross-entropy loss especially suitable for classification tasks, as it provides strong gradients even when predictions are far from the target. In systems runtimes, this mathematical cancellation is the primary motivation for fusing softmax and cross-entropy into a single operator. Fusing softmax with cross-entropy collapses the backward step to the subtraction $\hat{\mathbf{y}} - \mathbf{y}$, saving a separate kernel and its memory round trip and avoiding an underflow-prone probability tensor.

The choice of loss function also influences other training decisions. Gradient scale interacts with the learning rate, while loss averaging across batches affects gradient noise and thus the useful batch-size regime. Curvature influences optimizer behavior, and convergence decisions require validation metrics and a stopping rule rather than the training-loss trajectory alone.

5.5.4 Gradient descent and parameter updates

The loss gives training a single number to reduce, and gradient descent is the rule for reducing it. Consider one weight w and the batch loss plotted against it. At the current value of w, the curve has a slope $\partial \mathcal{L}/\partial w$, which is the gradient previewed in section 5.4.2.2. A positive slope means that increasing w raises the loss, so the weight should move down; a negative slope means it should move up. Taking a small step against the slope, recomputing the slope, and stepping again walks the weight downhill toward lower loss. A network has one such slope per parameter. Collecting them gives the gradient vector $\nabla_\theta \mathcal{L}$, which points in the direction of steepest loss increase, and each update steps the opposite way.



Definition 5.2: Gradient descent

Gradient descent is the iterative optimization algorithm that updates model parameters in the direction of the negative gradient, $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ with learning rate η, trading computational steps (O) for loss reduction.

1. **Significance**: The update rule itself sets a memory floor. Every parameter needs a gradient of the same size, so in FP32 plain gradient descent holds 8 bytes per parameter (the weight plus its gradient) before counting saved activations. Optimizers that keep extra per-parameter state raise that floor, as the training-memory ledger later in this chapter shows, and section 8.5.3.2 develops the complete accounting.
2. **Distinction**: Backpropagation computes gradients (the "what to change" signal); an optimizer applies updates (the "change it" action). The backward pass determines which forward values must remain available, while the optimizer adds its own state and update computation.
3. **Common pitfall**: A frequent misconception is that gradient descent finds the global minimum. Neural network loss landscapes are nonconvex with many local minima, saddle points, and plateaus. In practice, gradient descent and its variants converge to regions of low loss that generalize well, but the path depends on the learning rate schedule, batch size, and initialization.

Gradient descent turns each parameter's gradient into a parameter change, using each weight's local loss sensitivity to reduce the loss over successive steps. The fundamental update rule is defined in equation 5.21:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L} \tag{5.21}$$

where θ represents the collection of network parameters (weights and biases), η is the learning rate, and $\nabla_\theta \mathcal{L}$ is the gradient of the loss with respect to those parameters, which backpropagation (section 5.5.5) computes efficiently.

For the digit classifier, the update aims to reduce the chosen loss. If the network frequently confuses "seven"s with "one"s, the loss gradients from those examples guide the weight changes. The learning rate η[31] controls adjustment magnitude: too large values cause overshooting optimal parameters, while too small values result in slow convergence.

[31] **Learning rate**: This scalar has an outsized impact on training infrastructure because scaling batch size can require retuning the optimization schedule. The linear scaling rule increased learning rate with batch size for a specific large-batch ImageNet training regime (Goyal et al. 2017), but the best policy depends on the model, optimizer, data, and scale. Applying a scaling rule outside its validated regime can cause divergence when moving from single-GPU to multi-GPU training.

Plain gradient descent applies equation 5.21 directly to every weight. Practical optimizers refine it: they govern the step size, adjust the step direction, and carry momentum across iterations. No optimizer is universally best across all possible problems (Wolpert and Macready 1997), so neural-network training depends on choosing update rules and hyperparameters that match the model, data, and hardware constraints.

Despite neural network loss landscapes being highly nonconvex, gradient-based methods often find useful solutions when the model, data, initialization, and optimization regime are well matched. Why they do remains an active research question; one line of work, for example, studies the *implicit bias* of gradient descent, its tendency to settle on simple solutions among the many that fit the training data equally well (Neyshabur et al. 2017). For practical ML systems engineering, the key insight is empirical: learning rates, initialization, regularization, and the training configuration must be validated together.

5.5.4.1 Mini-batch gradient updates

Neural networks typically process multiple examples simultaneously during training, an approach known as mini-batch gradient descent. Rather than updating weights after each individual image, the average gradient over a batch of examples is computed before performing the update.

For a batch of size B, let $\mathcal{L}_i$ denote the loss for example i; the mean loss gradient becomes equation 5.22:

$$\nabla_\theta \mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}_i \tag{5.22}$$

With a typical batch size of 32, the system performs one forward pass over 32 images, computes 32 per-example losses, averages their gradients, and applies a single weight update. That average smooths noisy per-example gradients, but it also means the accelerator must keep the batch's activations available until the backward pass consumes them. Batch size therefore influences available parallelism and utilization together with model shape, kernel implementation, and hardware.

Batch size also meets a practical systems constraint: a desired effective batch may be larger than what fits in accelerator memory at once. Engineers address this through **gradient accumulation**: the full batch is split into smaller *micro-batches* that each fit in memory, and their gradients are accumulated with the appropriate loss normalization before the optimizer step. From the optimizer's perspective the effective batch size equals the sum across all micro-batches; from the hardware's perspective each micro-batch is an independent forward-backward pass that stays within the memory budget. For per-example-independent computations, accumulating consistently normalized gradients represents the same batch-gradient sum or mean. Exact numerical equivalence is not universal: floating-point reduction order, batch-dependent operations such as batch normalization, stochastic layers, and batch-dependent losses can make micro-batch execution differ from a single full-batch pass.

5.5.5 Gradient computation and backpropagation

Gradient descent assumes the gradient is already in hand, but the loss alone does not tell the system how sensitive it is to each parameter. With 109,386 parameters in the MNIST network, estimating every derivative through independent perturbations would be computationally prohibitive. This is the credit assignment problem: connecting the final error to the parameters and intermediate values that can change it. Backpropagation solves it by applying the chain rule to compute all of those local sensitivities in one backward traversal.

> Definition 5.3: Backpropagation
>
> ***Backpropagation*** is the gradient-computation algorithm training systems use to apply the chain rule to a computational graph, using recorded operations and saved or recomputed forward-pass values to compute the gradient of the loss with respect to every parameter in a single backward traversal.
>
> 1. **Significance**: For dense layers that require both parameter and input gradients, the backward pass costs on the order of twice the forward-pass FLOPs (a ratio derived

from the layer equations later in this section) and requires retaining or recomputing the intermediate values needed by the chain rule. For a model with N_L layers, batch size B, and layer widths $n_1, \ldots, n_{N_L}$, a simplified activation ledger scales as $\mathcal{O}\left(B \sum_{\ell=1}^{N_L} n_\ell\right)$. Gradients and optimizer state add further memory, so a model that fits on one accelerator for inference may not fit there for training.

2. **Distinction**: Numerical differentiation requires a perturbed evaluation for each of the model's P parameters. Reverse-mode automatic differentiation computes all P gradients in one backward traversal whose cost is proportional to the computational graph, avoiding P separate forward passes.
3. **Common pitfall**: A frequent misconception is that backpropagation is learning. It is a gradient computation algorithm; gradient descent performs the actual parameter update. Confusing the two obscures the systems-level separation: backpropagation determines memory requirements (activation storage), while the optimizer determines additional state requirements (the per-parameter buffers that some optimizers keep).

A car factory gives a concrete version of the same credit-assignment problem. Vehicles pass through four stations: frame installation (A), engine mounting (B), wheel attachment (C), and final assembly (D). When inspectors find a defective car, they must determine which station caused the problem.

The solution works backward. Starting from the defect, inspectors trace responsibility through each station: how much D's assembly contributed vs. what it received from C, and how much C's work contributed vs. what came from B. Each station receives adjustment feedback proportional to its contribution. If Station B's engine mounting was the primary cause, it receives the strongest signal to change.

Backpropagation follows an analogous backward trace, but it computes sensitivity rather than causal blame. Starting from the loss, it evaluates how a small change in each layer's inputs would change that loss and propagates those derivatives backward. The optimizer later converts the resulting parameter gradients into actual updates.

In neural networks, each layer acts like a station on the assembly line, and backpropagation determines the sensitivity of the final loss to each connection. Translating this intuition into mathematics requires the chain rule of calculus. In the factory analogy, "Station D's adjustment signal" corresponds to the gradient at the output layer, local sensitivity maps to a partial derivative, and "sending feedback backward" describes the chain-rule multiplication that propagates gradients through the network.

The sensitivity is local to the current parameter values, input batch, and loss. The computational graph is what makes computing it mechanical. Each node records an operation, and its backward rule maps an incoming loss gradient to gradients for its inputs. The reverse traversal begins at the scalar loss, applies those local rules in reverse dependency order, and accumulates contributions when one value influences the loss through several downstream paths. This accumulation is why backpropagation is more than sending one error signal down a chain: the runtime must respect graph dependencies and combine every relevant path before a gradient is complete. The algorithm below turns that graph-level description into a layer-by-layer procedure.

↪ How frameworks construct these dynamic computational graphs and record operations for reverse-mode automatic differentiation is developed in section 7.3.3.1.

5.5.5.1 Backpropagation algorithm steps

While forward propagation computes predictions, backward propagation computes the gradients that an optimizer uses to adjust the weights. Consider the running confusion in which the network predicts a "one" for an image of a "seven." Backward propagation systematically calculates how the loss would respond to a small change in each weight at the current parameter values.

The process begins at the network's output, where the predicted digit probabilities are compared with the true label. The resulting loss gradient then flows backward through the network, giving the local sensitivity of the loss to each layer's values and weights. The computation follows the chain rule of calculus, breaking down the relationship between weights and loss into manageable steps.

The mathematical foundations of backpropagation provide the theoretical basis for training neural networks, but practical implementation requires software support. Wengert presented automatic

numerical derivative evaluation without requiring analytical derivative expressions (Wengert 1964). Modern frameworks expose this capability through automatic differentiation systems that handle gradient computation automatically. Section C.3.1 restates the chain rule for composed layers and shows why **reverse-mode automatic differentiation** computes all parameter gradients in a single backward pass. The core implementation contract appears in algorithm 5.1: the forward pass must save the values that the backward pass will need.

Algorithm 5.1 Backpropagation training step

Require: mini-batch inputs $\mathbf{X}$, labels $\mathbf{y}$; N_L layers with parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{N_L}$; loss $\mathcal{L}$
Ensure: parameter gradients $\{\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L}\}$ for the optimizer
$\mathbf{A}^{(0)} \leftarrow \mathbf{X}$
for $\ell = 1$ to N_L **do** ▷ *forward pass*
    compute $\mathbf{Z}^{(\ell)}, \mathbf{A}^{(\ell)}$; save the values its derivative needs
end for
compute loss $\mathcal{L}(\mathbf{A}^{(N_L)}, \mathbf{y})$; seed the output gradient $\nabla_{\mathbf{A}^{(N_L)}}\mathcal{L}$
for $\ell = N_L$ down to 1 **do** ▷ *same local rule repeats*
    use the saved forward values to compute $\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L}$
    propagate $\nabla_{\mathbf{A}^{(\ell-1)}}\mathcal{L}$ only if that input gradient is required
end for
return the accumulated parameter gradients

Saving activations in the forward loop of algorithm 5.1 is where the systems cost enters: each activation stays live until the backward pass reaches its layer, so **activation memory** scales with batch size, layer count, and activation width. This is why training can exceed inference memory even when the parameter count is unchanged.

5.5.5.2 Error signal propagation

The flow of gradients through a neural network follows a path opposite to forward propagation. Starting from the loss at the output layer, gradients propagate backward, computing how the loss is locally sensitive to each layer and, ultimately, each weight.

Consider what happens when the digit classifier misclassifies a "seven" as a "one." The loss function generates an initial error signal at the output layer, essentially indicating that the probability for "seven" should increase while the probability for "one" should decrease. This error signal then propagates backward through the network layers.

A scalar case shows the rule before the matrix form. If $v = 3u$ and $u = x^2$, a small change in x changes u by $2x$ per unit, and each unit change in u changes v by 3, so $dv/dx = 3 \cdot 2x$. Backpropagation applies this multiply-the-local-slopes rule layer by layer. With many units per layer, each local slope becomes a matrix of slopes, called a *Jacobian*, but the rule is unchanged. For a network with N_L layers, the gradient flow can then be expressed mathematically. At each layer ℓ, the sensitivity of the final loss to that layer's output is governed by the chain rule[32] and the schematic Jacobian notation in equation 5.23:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell+1)}} \frac{\partial \mathbf{A}^{(\ell+1)}}{\partial \mathbf{A}^{(\ell)}} \tag{5.23}$$

This computation cascades backward through the network, with each layer's gradients depending on those from the layer above it. Notebook 5.2 traces these equations with numbers on a two-unit network and can be read first if the matrix form is unfamiliar. Because the gradient is local, two parameterizations that represent the same function can still produce differently scaled gradients. Backpropagation supplies these sensitivities, while the optimizer may normalize, accumulate, or otherwise transform them before updating the weights.

This process faces challenges in deep networks. As gradients flow backward through many layers, they can vanish or explode. Repeated multiplication can make gradients exponentially small, particularly with saturated sigmoid or tanh activations. Each floating-point format has a smallest magnitude it can store at full precision, 1.18×10^{-38} in FP32 and 6.10×10^{-5} in FP16. Below it, values enter the *subnormal* range, where the format gives up precision bits to represent ever-smaller

[32] **Chain rule**: The calculus identity $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a^{(N_L)}} \cdot \frac{\partial a^{(N_L)}}{\partial a^{(N_L-1)}} \cdots \frac{\partial a^{(1)}}{\partial w}$ becomes a product of N_L terms for an N_L-layer network. If each partial derivative is slightly less than one, the product vanishes exponentially; if slightly greater, it explodes. This multiplicative structure makes depth a systems constraint rather than an arbitrary design choice: it dictates the numerical precision requirements and initialization strategies (Glorot and Bengio 2010; He et al. 2015) needed to keep training stable.

magnitudes. Some accelerator modes skip that range for speed and flush such values straight to zero, and values smaller still underflow to zero on any hardware. A gradient that reaches zero this way stops updating its weight entirely. At the other extreme, values beyond the representable maximum can overflow to `Inf` and propagate `NaN`, destabilizing training.

5.5.5.3 Derivative calculation process

Computing gradients involves calculating several partial derivatives at each layer: how changes in weights, biases, and activations affect the final loss. These computations follow directly from the chain rule of calculus but must be implemented efficiently for practical training.

For a hidden layer ℓ with an element-wise activation, backpropagation first transforms the incoming signal, then computes weight, bias, and input gradients:

First, the incoming error signal $\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}}$ is transformed into the preactivation gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}}$ by multiplying element-wise ($\odot$) with the local activation function derivative (equation 5.24):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} \odot f'(\mathbf{Z}^{(\ell)}) \tag{5.24}$$

For ReLU activations, $f'(\mathbf{Z}^{(\ell)})$ acts as a binary mask, passing gradients through active neurons ($z > 0$) and zeroing out inactive ones. The classifier's softmax output couples coordinates, so it does not use this element-wise rule. Instead, its backward pass begins with the combined softmax/cross-entropy logit gradient derived above. For mean batch loss, each example's predicted-minus-target signal is divided by B at that starting point. The weight, bias, and input products below then propagate the already-normalized signal, applying the batch mean only once.

Weight gradients measure the local sensitivity of the final loss to each weight. With a batch size of B, layer input dimension $n_{\ell-1}$, and layer output dimension n_ℓ, the incoming activation matrix has shape $\mathbf{A}^{(\ell-1)} \in \mathbb{R}^{B \times n_{\ell-1}}$ and the sensitivity matrix has shape $\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \in \mathbb{R}^{B \times n_\ell}$. The optimizer uses these gradients to determine parameter updates (equation 5.25):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = {\mathbf{A}^{(\ell-1)}}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \tag{5.25}$$

The matrix product $(n_{\ell-1} \times B) \times (B \times n_\ell) \to (n_{\ell-1} \times n_\ell)$ contracts over the batch dimension B, summing the outer products of activations and sensitivities across all examples in the mini-batch. With the mean-loss normalization already present in the sensitivity matrix, this product needs no further factor of $1/B$.

Bias gradients measure how changing each bias term affects the loss. Since biases shift the activation threshold of neurons, these gradients indicate whether neurons should become more or less easily activated, as expressed in equation 5.26:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \mathbf{1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \tag{5.26}$$

Here, $\mathbf{1} \in \mathbb{R}^B$ is an all-ones vector, so left multiplication sums the per-example bias gradients across the batch.

Input gradients propagate the error signal backward to the previous layer. Rather than directly updating parameters, these gradients serve as the "adjustment signals" that allow earlier layers to learn from the final prediction error (equation 5.27):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} {\mathbf{W}^{(\ell)}}^T \tag{5.27}$$

Here, the product $(B \times n_\ell) \times (n_\ell \times n_{\ell-1}) \to (B \times n_{\ell-1})$ contracts over the output dimension n_ℓ, preserving the per-example activation shape for preceding layers.

These equations reveal the physical origin of the classic systems rule that the backward pass requires roughly twice the arithmetic of the forward pass. While the forward pass executes a single matrix multiplication per dense layer, the backward pass evaluates two products of the same arithmetic complexity when both weight and input gradients are required. The weight-gradient product trains that layer's parameters; the input-gradient product supplies the signal needed to train

preceding layers. For ordinary parameter training on fixed MNIST images, no preceding trainable layer consumes the gradient with respect to raw pixels, so the first layer's input-gradient product can be omitted.

The resulting dense matrix work is three forward-pass counts minus the first layer's forward count, or about 2.08× the forward matrix work for this network, before activation derivatives, bias reductions, loss, and optimizer work. Three times the forward matrix work remains a useful approximation when input gradients are needed throughout, or when the omitted first-layer product is a small fraction of total work. The running network makes the exception visible because its first layer accounts for most of its arithmetic.

Systems Perspective 5.4: The memory cost of backprop

Activation retention is one component of a broader training-state budget. Under a simplified single-device accounting that excludes temporary workspaces and framework overhead, equation 5.28 captures the four principal byte counts:

$$\text{Training Memory} \approx \text{Model Weights} + \text{Gradients} + \text{Optimizer States} + \text{Activations} \tag{5.28}$$

The derivative equations are also a state-dependency ledger:

- The weight gradient in equation 5.25 needs the incoming activation $\mathbf{A}^{(\ell-1)}$.
- The activation derivative may need $\mathbf{Z}^{(\ell)}$ or a compact mask that records which nonlinear units were active.
- The input gradient in equation 5.27 needs the layer weights $\mathbf{W}^{(\ell)}$.
- The optimizer consumes the completed parameter gradients and may add momentum, variance, or other state after backpropagation has produced them.

A training system can retain these values for the fastest backward pass, recompute selected values to save memory, or offload and partition state across memory tiers or devices. None of these choices eliminates cost: recomputation spends additional FLOPs, while offload and partitioning introduce data movement and synchronization. For deep networks with large batches, wide activations, or high-resolution inputs, this trade-off can dominate accelerator capacity. Section C.3.3 provides the complete training-memory equation and a worked analysis of weights, gradients, optimizer state, and activation costs.

A minimal network makes the gradient arithmetic concrete by tracing actual values through backpropagation.

Napkin Math 5.2: Tracing gradients: A worked backpropagation example

Setup: Consider a network with two inputs, a hidden layer of two neurons (ReLU activation), and one output neuron (no activation, for simplicity). Suppose the current weights and biases are:

- **Hidden layer**: $\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$, $\mathbf{b}^{(1)} = [0,\ 0]$
- **Output layer**: $\mathbf{W}^{(2)} = \begin{bmatrix} 0.6 \\ -0.4 \end{bmatrix}$, $b^{(2)} = 0$

Given input $\mathbf{x} = [1.0,\ 0.5]$ and target $y = 1.0$, applying mean squared error: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$.

Forward pass (to establish the values backpropagation needs):

- Hidden preactivation:

$$\mathbf{z}^{(1)} = \mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)} = [1.0 \cdot 0.5 + 0.5 \cdot 0.8,\ 1.0 \cdot (-0.3) + 0.5 \cdot 0.2] = [0.9,\ -0.2]$$

- Hidden activation (ReLU): $\mathbf{a}^{(1)} = [\max(0, 0.9),\ \max(0, -0.2)] = [0.9,\ 0.0]$
- Output: $\hat{y} = \mathbf{a}^{(1)}\mathbf{W}^{(2)} + b^{(2)} = 0.9 \cdot 0.6 + 0.0 \cdot (-0.4) = 0.54$
- Loss: $\mathcal{L} = \frac{1}{2}(0.54 - 1.0)^2 = 0.1058$

Backward pass (applying the chain rule layer by layer):

Step 1: Output layer gradient. The loss gradient with respect to the output is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.54 - 1.0 = -0.46.$$

Step 2: Output weight gradients (applying equation 5.25). Since the output layer has no activation function

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} = -0.46, \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = {\mathbf{a}^{(1)}}^T \frac{\partial \mathcal{L}}{\partial z^{(2)}} = \begin{bmatrix} 0.9 \\ 0.0 \end{bmatrix} \cdot (-0.46) = \begin{bmatrix} -0.414 \\ 0.0 \end{bmatrix}$$

Step 3: Propagate to hidden layer (applying equation 5.27). The error signal sent backward is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} {\mathbf{W}^{(2)}}^T = (-0.46) \cdot [0.6,\ -0.4] = [-0.276,\ 0.184]$$

Step 4: Pass through ReLU. The ReLU derivative is one where $z > 0$ and 0 otherwise, so

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = [-0.276 \cdot 1,\ 0.184 \cdot 0] = [-0.276,\ 0.0].$$

The second neuron's gradient is zeroed because ReLU blocked its forward signal.

Step 5: Hidden weight gradients (applying equation 5.25):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \cdot [-0.276,\ 0.0] = \begin{bmatrix} -0.276 & 0.0 \\ -0.138 & 0.0 \end{bmatrix}$$

Weight updates (with learning rate $\eta = 0.1$): Each weight moves opposite to its gradient. For example, $W_{11}^{(1)}$ updates from 0.5 to $0.5 - 0.1 \cdot (-0.276) = 0.5276$, nudging the network toward the correct output. The second hidden neuron's weights receive zero updates for this example because ReLU blocked its activation, illustrating the mechanism that can lead to dead neurons if it happens persistently across the training data.

Systems insight: Backpropagation is a data-dependency graph as well as a calculus procedure. Training systems must preserve the forward activations needed by this backward pass, which is why activation memory becomes a first-order systems cost.

While understanding these mathematical details is essential for debugging and optimization, modern practitioners rarely implement gradients manually. The systems breakthrough lies in how frameworks automatically implement these calculations. Consider a simple operation like matrix multiplication followed by ReLU activation: `output = relu(input @ weight)`. The mathematical gradient involves computing the derivative of ReLU (0 for negative inputs, 1 for positive) and applying the chain rule for matrix multiplication. The framework records the operation in a computation graph during the forward pass, stores the pre-ReLU activations needed for gradient computation, attaches the backward rule for each operation, and schedules the reverse traversal to balance correctness, memory usage, and hardware utilization. This automation transforms gradient computation from a manual, error-prone process requiring deep mathematical expertise into a reliable system capability that enables rapid experimentation and deployment.

The autograd[33] engines that perform this automation differ mainly in when they build the graph. Dynamic computation graphs record operations as they execute, while static computation graphs

[33] **Autograd (automatic differentiation)**: Reverse-mode automatic differentiation traces back to Linnainmaa's work on differentiating computer programs (1970). Modern autograd engines record forward-pass operations into a directed acyclic graph (DAG), then traverse it backward using the chain rule to compute gradients automatically. The key systems trade-off is that more flexible execution captures exactly what happened in each iteration, while more fixed execution exposes more opportunities for ahead-of-time optimization. Section 7.3.3 develops this framework design choice in detail.

defer execution to expose more optimization opportunities. In both cases the engine tracks which tensors require gradients, keeps each saved value alive until its backward rule has run, and schedules the reverse traversal so that it follows the dependencies the forward pass created. That bookkeeping is exactly the activation-memory cost that the worked example below prices.

5.5.5.4 Worked example: Training vs. inference memory

The four terms of equation 5.28 can now be priced for the running network. A single forward pass through the $784 \to 128 \to 64 \to 10$ network costs 109,184 MACs. The memory footprint during training tells a different story. Evaluating the memory footprint at batch size 32 in 32-bit (4-byte) floating-point precision reveals the gap between training and inference demands across parameters, activations, gradients, and optimizer state.

The first contribution is the model parameters: table 5.8 tallies the weights and biases layer by layer, totaling 109,386 parameters at 4 bytes each, occupying 437.5 KB.

Table 5.8: MNIST Parameters by Layer: Weight and bias counts for the 784-128-64-10 network. Layer 1 (Input -> Hidden1) dominates the parameter budget because it multiplies the 784-dimensional input by 128 hidden units; deeper layers shrink rapidly as the representation compresses toward the 10-class output.

Layer	Weights	Biases	Total Parameters
Input → Hidden1	784×128 = 100,352	128	100,480
Hidden1 → Hidden2	128×64 = 8,192	64	8,256
Hidden2 → Output	64×10 = 640	10	650
Total			**109,386 parameters**

Activations are the next contribution. Table 5.9 records each layer's activation tensor and its memory cost at batch size 32.

Table 5.9: MNIST Activation Memory at Batch Size 32: Per-layer activation tensor shapes, value counts, and 32-bit float memory cost during a single forward pass. Input activations dominate the activation table at 100.4 KB; for this small MNIST multilayer perceptron (MLP), the total activation footprint remains below the parameter footprint, while larger batches and deeper networks make activations a central training-memory concern.

Layer	Activation Shape	Values	Memory
Input	32×784	25,088	100.4 KB
Hidden1	32×128	4,096	16.4 KB
Hidden2	32×64	2,048	8.2 KB
Output	32×10	320	1.3 KB
Total		31,552	**126 KB**

Training adds two memory contributions that inference never pays. Gradients occupy the same space as the parameters they update, here 437.5 KB, and the optimizer adds its own state. The Adaptive Moment Estimation (Adam) optimizer, a widely used refinement of gradient descent, keeps two running averages for every parameter, one of its recent gradients (the *momentum*) and one of its recent squared gradients (the *velocity*), and uses them to adapt each parameter's step size. Those two buffers occupy twice the parameter size, another 875.1 KB. Table 5.10 summarizes the per-component memory footprint for training vs. inference.

Table 5.10: MNIST Training vs. Inference Memory at Batch Size 32: A simplified FP32 ledger for the toy MNIST MLP. Training retains saved activations and adds gradients plus two Adam buffers; inference uses the peak adjacent-layer buffer. Framework workspace and memory-saving techniques can change the totals.

Component	Training	Inference
Parameters	437.5 KB	437.5 KB
Activations	126 KB	117 KB
Gradients	437.5 KB	—
Optimizer state	875.1 KB	—
Total	**~1.9 MB**	**~554 KB**

Scale drives the systems lesson. At batch size 32, training requires 3.4× the inference memory in this simplified ledger. The absolute gap grows with parameter and activation storage; the ratio depends on their balance and on implementation choices such as recomputing some activations instead of storing them, numerical precision, and workspace reuse.

5.5.5.5 Scaling the ledger beyond MNIST

The MNIST ledger is small, but each of its terms grows with network width and batch size. As model size scales, activation storage, gradient storage, optimizer-state traffic, and scheduling pressure grow together.

Consider a larger variant of the MNIST network ($784 \to 512 \to 256 \to 10$) with a batch size of 32. Each layer's activations must be maintained until the backward pass reaches that layer:

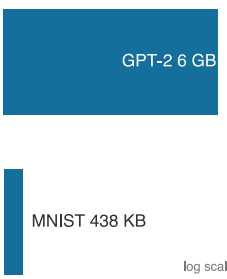
- Input layer: 32×784 values (~100 KB using 32-bit numbers)
- Hidden layer 1: 32×512 values (~66 KB)
- Hidden layer 2: 32×256 values (~33 KB)
- Output layer: 32×10 values (~1 KB)

Beyond activations, training allocates memory for parameter gradients. For this larger network with approximately 535,818 parameters, gradient storage requires a few megabytes. Adam,[34] with its momentum and velocity buffers for every parameter, roughly triples that gradient footprint.

Memory bandwidth and footprint compound these capacity requirements. A training step reads parameters, produces gradients, and retains or recomputes the forward-pass values required by each layer's backward calculation. Saved-activation storage generally grows with batch size and network depth, making peak training memory larger than the inference footprint for the same model and batch. For modest networks like the MNIST example, this traffic remains manageable. As models grow, memory bandwidth and activation storage can become binding constraints, motivating high-bandwidth memory, narrower number formats for some tensors (mixed precision), partitioning state across devices, and activation recomputation, which discards selected saved activations and recomputes them during the backward pass.

The memory requirements summarized in table 5.10, and even those of the larger variant, seem modest for a digit classifier. Scaling to production-sized models transforms these requirements dramatically, changing the hardware regime.

34 **Adam (adaptive moment estimation)**: Maintains per-parameter momentum and velocity buffers (statistically, first- and second-moment estimates of the gradient), requiring 2× additional FP32 memory beyond the parameters themselves (Kingma and Ba 2015). For a 100K-parameter MNIST model this overhead is negligible, but for a 7-billion parameter model it adds ~56 GB, often the difference between fitting on one GPU or requiring partitioning. Adam is widely used because its adaptive updates work robustly across many tasks, but learning rate, schedule, regularization, and memory strategy still require validation.



MNIST is cache-scale; GPT-2 is VRAM-scale.

Lighthouse 5.1: MNIST-to-GPT-2 memory scaling

The MNIST running example and the GPT-2 lighthouse show how model scale changes the data (D_{vol}) term of the iron law of ML systems (principle 3).

- MNIST (running example): 109,386 parameters at 4 bytes each occupy approximately 438 KB. This entire model fits inside the L2 cache of a modern processor.
- GPT-2 (lighthouse): 1,500,000,000 parameters at 4 bytes each occupy approximately 6 GB before activations and runtime workspace. This exceeds cache capacity and places the model in a device-memory or multi-device regime unless precision is reduced.

Systems insight: Moving from ~109.4K to 1.5B parameters is a 14,000× jump. The increase represents a phase change in engineering, not merely "more parameters." MNIST can keep its weights near the processor; GPT-2 serving must manage a much larger memory working set. Whether a particular layer is compute or memory bound still depends on batch size, precision, reuse, and hardware balance.

Napkin Math 5.3: Quick estimation for ML engineers

Detailed calculations are essential for design documents, but experienced engineers also develop rapid mental estimation skills. These "napkin math" shortcuts enable quick feasibility checks before committing to detailed analysis.

Memory estimation:

- Parameters $\rightarrow$ bytes: Multiply by four for 32-bit single precision (FP32), two for 16-bit half precision (FP16) or brain floating point 16 (BF16, the latter with an FP32-like exponent range), or one for 8-bit integer (INT8)
- Fully connected (FC) layer parameters: Input $\times$ Output (plus Output biases, usually negligible)
- Training state: Parameters, gradients, and optimizer state can require ~3–4$\times$ the weight-only parameter memory; peak activations and workspace are additional
- Adam optimizer overhead: 2$\times$ parameter memory (momentum + velocity)
- Max batch size: Approximately (GPU VRAM $-$ persistent state $-$ workspace) / peak activations per sample

Compute estimation:

- FC layer FLOPs: $2 \times d_{\text{in}} \times d_{\text{out}} \times B$ (multiply-add = 2 ops)
- MACs to FLOPs: Multiply by 2
- Compute utilization proxy: achieved FLOP/s/R_{peak}

Example: Rapid feasibility estimation for training a 100M parameter model on a 16 GB GPU.

Mental math: 100M parameters at 4 bytes each, with 4 training-memory copies, require 1.6 GB for persistent model state. That leaves about 14.4 GB for activations, temporary workspace, and framework overhead. Answer: the parameters fit, but the training configuration is not yet proven; batch size, activation shape, and kernel workspace determine whether the complete step fits.

Systems insight: A rough persistent-state estimate can reject impossible configurations quickly, but batch size still depends on activation and workspace headroom.

Beyond this model-fit example, table 5.11 distills three common feasibility diagnostics into one-line formulas an engineer can apply before reaching for a profiler.

Table 5.11: Napkin-math feasibility checks: Three rapid mental-estimation rules for GPU fit, epoch time, and bound regime, distilled into one-line formulas an engineer can apply before running a profiler.

Diagnostic target	Quick estimate
GPU memory fit	persistent training state + peak activations + workspace $<$ VRAM
Epoch duration	60K $\times$ forward-backward FLOPs/image/achieved FLOP/s
Bottleneck regime	Compare arithmetic intensity with R_{peak}/BW

The component-level memory ledger in table 5.10 is explicit but intentionally simplified. In practice, systems engineers work at two levels of fidelity: detailed budgets for design documents and order-of-magnitude estimates for early feasibility gates. The ledger confirms that MNIST fits comfortably in cache while GPT-2 requires a much larger memory tier, before accounting for runtime workspace. A quick mental estimate should reach the same conclusion in seconds, not minutes, and flag any model that cannot physically fit on the target hardware before a single line of profiling code runs.

Checkpoint 5.3: Backpropagation

The credit assignment problem asks how the loss depends on parameters throughout the network. Backpropagation computes those gradients via the chain rule; verify that the mechanism is clear:

Mechanism

- ☐ **Chain rule**: can you explain how equation 5.23 decomposes a complex error signal into local gradients for each layer?
- ☐ **Gradient decay**: why do gradients often vanish in deep networks?

Training vs. inference

- ☐ **Computational cost**: under what input-gradient assumption does dense training require roughly three times the forward matrix work, and which product can ordinary training on fixed raw inputs omit?
- ☐ **Memory cost**: why must training retain or recompute the forward values needed by the backward pass, and how does this scale with batch size and network depth?
- ☐ **Layer cost**: for a dense layer mapping 4,096 inputs to 4,096 outputs at batch size 32, estimate its forward FLOPs and its FP16 activation bytes to one significant figure, then say which of the two doubling the batch grows.

5.5.6 Iterative learning process

The complete training process combines forward propagation, backward propagation, and weight updates into a systematic training loop. This loop repeats until the network achieves satisfactory performance or reaches a predetermined number of iterations.

A single pass through the entire training dataset is called an epoch.[35] For MNIST, with 60,000 training images and a batch size of 32, each epoch consists of 1,875 batch iterations. Algorithm 5.2 lays out the loop of mini-batch stochastic gradient descent (SGD), so called because each update follows a gradient estimated from a random sample of the data rather than the full dataset. Each epoch shuffles the data, steps through it one mini-batch at a time with a forward and backward pass, and applies a single parameter update per batch.

[35] **Epoch**: One complete pass through a training dataset. The number of passes is a direct multiplier on total work: a 100-epoch MNIST run executes 100$\times$ more forward and backward passes than a single epoch. Large language-model training is often specified by a token budget rather than a simple epoch count because data sources may be mixed and sampled at different rates. The same accounting still applies: every additional training token adds another forward and backward computation.

Algorithm 5.2 Mini-batch stochastic gradient descent

Require: training set $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{D}$; rate η; batch size B; epochs N_{epochs}; initial θ_0
Ensure: trained parameters θ

$\theta \leftarrow \theta_0$
for $e = 1$ to N_{epochs} **do**
  shuffle the training set ▷ *reduce order-dependent patterns*
  for each mini-batch $\mathcal{M}$ of size B **do**
    $\hat{\mathbf{y}}_i \leftarrow f(\mathbf{x}_i; \theta)$ for all $(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{M}$
    $\mathcal{L}_{\mathcal{M}} \leftarrow \frac{1}{B} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{M}} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{y}_i)$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\mathcal{M}}$ ▷ *backpropagate, then update*
  end for
  evaluate on validation; stop if the convergence criterion is met
end for
return θ

The mini-batch loop makes the batch size B a hardware unit of work: a larger B can improve matrix utilization and accelerator occupancy but raises activation memory, and because the update fires once per batch rather than once per example, B also sets the gradient-update cadence. The inner loop in algorithm 5.2 is the unit that the hardware repeatedly executes. (In the algorithm, D counts training examples; it is not the iron law's D_{vol}, which counts bytes moved.) Forward propagation creates activations for every example in the current mini-batch, backward propagation consumes the required saved values to compute gradients, and the optimizer mutates the parameters once per batch. Training loss tracks the optimization objective on recent batches, while validation accuracy measures task performance on a separate held-out validation set. The test set remains reserved for final evaluation after model and hyperparameter choices are fixed.

5.5.6.1 Convergence and stability considerations

A network that achieves 99.5 percent accuracy on training data but only 85 percent on relevant unseen data exhibits a large generalization gap, a strong sign of overfitting. The model may have learned useful patterns while also specializing too closely to the training sample.

Definition 5.4: Overfitting

Overfitting is a generalization failure in which performance on the training sample improves while performance on relevant unseen data does not.

1. **Significance**: The diagnostic signature is measurable: training loss continues to fall while validation loss rises or the relevant validation metric plateaus, creating a widening generalization gap. Parameter count alone does not determine this behavior; data coverage, architecture, optimization, regularization, and the relationship between train and deployment distributions all matter.
2. **Distinction**: Unlike underfitting (where the model lacks the capacity to capture the target function and both training and validation error remain high), overfitting produces low training error but high validation error, indicating that the model has specialized to the training sample rather than learning the underlying distribution.
3. **Common pitfall**: More data helps only when it is representative of the behavior the system must generalize to. Additional examples, augmentation, regularization, and early stopping are possible interventions, but each must be evaluated on held-out data from the relevant operating regime.

Regularization controls overfitting by constraining what the network can fit. The most common form adds a penalty on weight magnitude to the loss. An ℓ_2 term $\lambda_{\text{reg}} \sum w^2$, where λ_{reg} sets the penalty strength and the numeric subscript on ℓ marks a norm rather than a layer index, discourages large weights; its gradient $2\lambda_{\text{reg}} w$ shrinks every weight by the fraction $2\eta\lambda_{\text{reg}}$ at each step, which

under this plain gradient-descent update is equivalent to **weight decay**. An ℓ_1 term $\lambda_{\text{reg}} \sum |w|$ instead favors sparse solutions, but ordinary gradient steps do not guarantee exact zeros. Both penalties add element-wise gradient work without requiring persistent optimizer state of their own; realizing exact zeros depends on the update method. When obtained, those zeros are the unstructured sparsity that later compression techniques exploit. **Dropout** takes a different route: during training it zeroes a random fraction of activations at every step, so no unit can rely on a single upstream partner. It needs a fresh random mask per activation while training and is removed entirely at inference. Early stopping, halting training once validation loss stops improving, is the third common control, and it depends on the periodic validation and checkpointing that close this section.

Parameter count alone does not provide a monotonic predictor of generalization. In classical statistical estimation, increasing capacity beyond sample size invites severe overfitting. Modern deep architectures, however, often exhibit the double-descent trajectory shown in figure 5.16. Left of the interpolation threshold (where parameter count matches sample complexity), test error traces a standard U-shaped curve as excess capacity models idiosyncrasies in the training sample. At the interpolation threshold, the network achieves zero training error; scaling capacity beyond this boundary can initiate a second descent in test error as inductive biases and optimization dynamics regularize the overparameterized model.

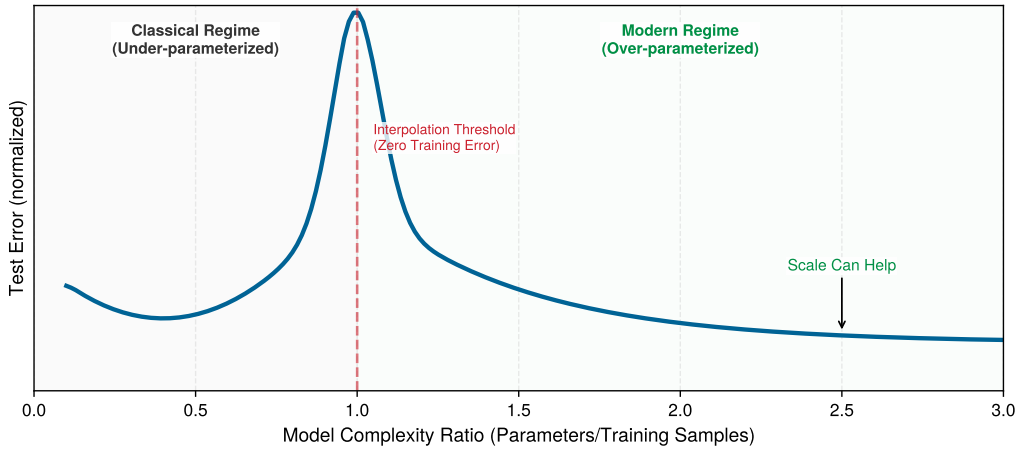


Figure 5.16: The Double Descent Phenomenon: An illustrative normalized curve shows test error first decreasing and then rising in the classical regime, peaking at the interpolation threshold, and decreasing again in the overparameterized modern regime.

This second descent demonstrates why classical bias-variance trade-offs[36] cannot serve as an operational stopping rule. Because generalization error does not vary monotonically with model size, systems engineers cannot deduce model capacity, convergence limits, or stopping criteria from parameter counts alone. Overparameterization turns scale into an empirical engineering lever rather than an automatic failure mode, but its benefits depend strictly on the data distribution, optimization trajectory, and regularization regime. Systems practitioners must therefore measure generalization directly on held-out validation data throughout training rather than relying on theoretical capacity bounds to justify training budgets.

Learning rate selection strongly influences training dynamics. A rate that is too large for the optimizer, batch size, normalization, and schedule can make loss oscillate or diverge; one that is too small can waste epochs making negligible progress. There is no universal "moderate" value, so engineers test rates within the intended training configuration and monitor both optimization stability and validation performance.

Convergence monitoring provides essential feedback during training and continues into production deployment. A flattening training-loss curve signals slowing optimization, but it does not identify the cause: the run may be approaching a useful solution, encountering an optimization limit, or underfitting. A validation plateau is similarly diagnostic rather than conclusive. The gap between training and validation performance helps distinguish overfitting from improvements that transfer

[36] **Bias-variance trade-off**: *Bias* is the error that comes from a model too simple to express the true pattern; *variance* is the error that comes from a model so sensitive to its particular training sample that a different sample would produce a noticeably different fit. Classical theory holds that adding capacity lowers bias but raises variance, which yields the U-curve. In some overparameterized regimes, however, test error decreases again after the interpolation threshold, so the classical single-U-shaped account is incomplete. The systems consequence is that model size cannot be treated as a universally monotonic overfitting risk. Larger models can become useful engineering options, provided the measured data, optimization, and regularization regime supports the gain.

to held-out examples. The interplay between batch size, available memory, and computational resources requires careful balancing to achieve efficient training within hardware constraints, the same memory-computation trade-offs that the training-memory ledger established.

Selecting a stopping point requires periodically evaluating the model on the validation set and saving its weights to durable storage, a practice called **checkpointing** (not to be confused with activation recomputation, which some frameworks also call checkpointing). For a small model like the MNIST network, writing a checkpoint costs a negligible amount of time. For a model whose parameters occupy tens to hundreds of gigabytes, a synchronous checkpoint write can stall the training loop while those bytes are serialized to disk or object storage; asynchronous and distributed checkpointing reduce that pause but still consume memory, network, and storage bandwidth. Running validation itself requires a full forward pass over the held-out dataset, which at large scale consumes meaningful accelerator time that would otherwise go to training. Checkpointing and validation are therefore not free statistical observations: they impose recurring I/O and compute costs whose frequency must be balanced against recovery and model-selection needs. Saving too infrequently risks losing useful weights after a failure or later degradation; saving too frequently stresses storage bandwidth and adds overhead. This is why convergence monitoring, while conceptually straightforward, becomes a systems infrastructure problem at scale.

The complete training pipeline runs from forward propagation through loss computation to gradient-based weight updates. Training, however, is preparation, not the end goal. Once parameters are optimized, deploying the model for real-time predictions changes the resource profile entirely.

Checkpoint 5.4: Neural network learning process

The complete training cycle provides the mathematical machinery that enables neural networks to learn from data. Before moving to inference and deployment, verify core understanding of the training cycle:

Use the MNIST network ($784 \to 128 \to 64 \to 10$) and a batch size of 32 as the running check.

- ☐ **Forward pass**: can you trace the forward pass using $\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$ and $\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)})$?
- ☐ **Cross-entropy loss**: can you explain what cross-entropy loss measures and why batch loss is averaged across examples?
- ☐ **Backward pass**: can you trace how gradients flow backward through the network and why stored activations are needed?
- ☐ **Update rule**: can you write the gradient descent update rule and explain how batch size trades memory, throughput, and gradient stability?
- ☐ **Training memory**: can you explain why the full training loop requires more memory than inference?

If any concepts feel unclear, review the earlier sections on forward passes, loss functions, backpropagation, and learning before continuing. These mechanisms form the foundation for the training-vs.-inference distinction that follows.

5.6 Inference Pipeline

Training transforms randomly initialized weights into parameters that encode useful patterns, but deployment executes those parameters under a completely different resource profile. During inference,[37] the same mathematical operators face a fundamentally different operating regime: online latency deadlines or offline throughput targets replace the training objective, gradients and optimizer state disappear from memory, and target execution environments range from milliwatt edge processors to multi-accelerator cloud servers. This transition reshapes how memory bandwidth, compute capacity, and pipeline stages govern system performance.

5.6.1 Production deployment and prediction pipeline

Suppose a model that achieved 99 percent accuracy on its test set begins producing nonsensical outputs three months after deployment, although no code has changed. The weights are frozen and

[37] **Inference**: From Latin *inferre* ("to bring in, to conclude"), borrowed from logic where it means deriving conclusions from premises. The ML usage marks a systems boundary: training executes forward and backward passes and maintains optimizer state, while ordinary inference executes the forward graph with frozen parameters. Removing gradients, optimizer state, and most saved activations can reduce memory and computation substantially, but the ratio depends on architecture, batch size, precision, runtime workspace, and serving strategy.

the inference pipeline runs without error. One possible cause is that the operating world moved while the model stood still.

The transition from training to inference limits how a fixed deployed model can adapt. Trained models generalize to unseen inputs through learned statistical patterns, but their parameters do not change during ordinary inference. When operational data diverges from the training distribution, the model continues applying those learned patterns. Consider an autonomous vehicle perception system: if construction zones become more frequent or novel vehicle configurations appear, its responses still reflect the training evidence. Systems that do not update parameters online adapt through monitored data collection, validation, and retraining, a deliberate engineering process.

↪ When monitored drift should trigger retraining, and what each retraining cadence costs, is operationalized in section 14.4.2.2.

5.6.1.1 Operational phase differences

Neural network operation divides into two distinct phases with markedly different computational requirements and state lifetimes. Inference executes only the forward pass, evaluating inputs against static parameters with batch sizes dictated by incoming traffic. Training couples that forward pass with backpropagation for gradient computation and parameter updates, selecting batch sizes to maximize accelerator occupancy and gradient stability. Unlike inference, training must retain intermediate activations, gradients, and optimizer state simultaneously, multiplying working memory demands.[38] While the underlying network weights are identical, the execution graph changes: training-only regularizers such as dropout are stripped away, while adjacent matrix operations and activations can be fused or quantized into specialized execution kernels. Figure 5.17 contrasts these operational dataflows: training couples large-batch processing with a bidirectional loop where output errors trigger a reverse gradient pass, whereas inference executes a strictly unidirectional forward pass over smaller, variable batch sizes without backward state retention.

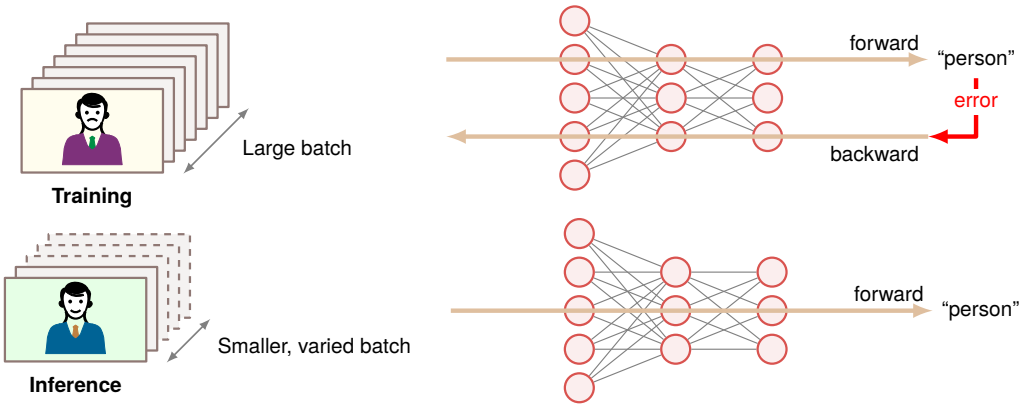


Figure 5.17: Inference vs. Training Flow: During inference, neural networks use learned weights for forward pass computation only, simplifying the data flow and reducing computational cost compared to training, which requires both forward and backward passes for weight updates. This streamlined process enables efficient deployment of trained models for real-time predictions.

These computational differences manifest directly in hardware selection and deployment topologies. Training clusters rely on high-bandwidth, high-memory accelerators backed by hundreds of watts per socket and dedicated data-center cooling infrastructure. Inference deployments on constrained hardware prioritize bounded tail latency and energy efficiency across heterogeneous platforms: mobile devices execute on low-power neural processing units (typically 2–4 W), edge servers employ dedicated inference accelerators,[39] and cloud instances use reduced numerical precision to saturate dedicated matrix-multiply units (NVIDIA's versions are called Tensor Cores).[40] At scale, production serving introduces operational requirements that do not arise in isolated training runs, including dynamic request batching, load balancing, and failure isolation. At the memory level, training retains all forward activations until backward passes consume them, whereas inference discards or overwrites intermediate buffers as execution advances through the graph; table 5.12 summarizes this resource profile.

38 **Training GPU power budget**: The "high-memory" requirement is driven by the need to hold parameters, gradients, optimizer state, and activations simultaneously. The corresponding power draw dictates the "substantial cooling infrastructure," as a single high-end training GPU consumes 400 W–700 W. Even compared with a 4 W mobile inference chip, that is at least 100× the power budget.

39 **Edge inference accelerators**: Google's edge Tensor Processing Unit (TPU), the Edge TPU (Google Coral), operates in the mobile/embedded tier at about 2 W, delivering 4 TOPS through an INT8 datapath. Edge servers sit in a different tier: Jetson AGX Orin reaches 275 TOPS at 15 W–60 W, about 68.8× more throughput but with wired-power assumptions.

40 **Quantization**: Moving from FP32 to INT8 reduces parameter storage by 4× and can increase throughput on hardware with efficient INT8 datapaths, although the realized speedup depends on kernel support, memory traffic, and workload shape. Inference often permits lower precision than training because it does not need to represent small gradients or optimizer updates across repeated learning steps. The trade-off is not free: aggressive quantization can degrade accuracy, especially on rare or poorly represented inputs, so calibration and task-level evaluation must establish the safe precision for each deployment.

↪ Tensor Cores, which execute a small matrix multiply-accumulate per instruction and set the reduced-precision peak rates quoted for data center GPUs, are developed in section 11.4.3.

Table 5.12: Training vs. Inference Forward Pass: Both phases execute the same forward equations, but they manage state differently. Training must retain or recompute the values required by the backward pass and also carries gradients and optimizer state. Inference can reuse a rolling pair of layer buffers.

Characteristic	Training Forward Pass	Inference Forward Pass
Activation Storage	Retains values required by the backward pass	Retains the current input and output buffers
Memory Pattern	Saves or recomputes selected intermediate states	Reuses buffers as successive layers complete
Computational Flow	Same matrix products; each layer input is kept	Same matrix products; each input freed after use
Resource Profile	Weights, gradients, optimizer state, saved inputs	Weights plus the largest adjacent buffer pair

5.6.1.2 Memory and computational resources

Neural networks consume computational resources differently during inference than during training. Inference has two memory obligations. The first is persistent: the trained weights and biases must remain available for every request. The second is transient: in this sequential feed-forward graph, each layer produces an activation buffer that is needed only until the next layer consumes it. This distinction is the reason inference can be much leaner than training, even though the arithmetic in the forward pass is the same.

For the canonical MNIST network ($784 \to 128 \to 64 \to 10$), the persistent parameter block contains 109,386 parameters, or about 438 KB at 32-bit floating point precision. The layer-level counts in table 5.13 show why: each fully connected layer performs one multiply-add for each weight, so parameter memory and arithmetic scale together.

Table 5.13: MNIST Inference Resources by Layer: The persistent parameter block and multiply-add count for the canonical MNIST network. In fully connected layers, the same weights that occupy memory also determine the arithmetic work per inference.

Layer	Weights	Biases	Multiply-Adds
Layer 1	$784 \times 128 = 100{,}352$	128	100,352
Layer 2	$128 \times 64 = 8{,}192$	64	8,192
Output	$64 \times 10 = 640$	10	640
Total	109,386 parameters	Included in total	109,184

This persistent-plus-rolling-buffer model also explains the deployment optimizations that follow. Batching can increase arithmetic reuse and hardware occupancy but expands activation storage. Lower numerical precision can shrink the persistent parameter block and activation buffers,[41] but only if task quality survives the smaller representation. Hardware-specific layouts can improve cache reuse by keeping working data close to the compute units. Inference makes these optimizations easier to plan because parameters are fixed and many activations have short lifetimes.

5.6.1.3 Performance enhancement techniques

Freezing parameters and eliminating backward state simplifies optimization by removing parameter updates and their dependencies. For a fixed forward compute graph, runtime engines can plan memory layout, operator scheduling, and numerical precision against the physical limits of the target processor.

Input batching introduces a fundamental tension between per-request latency and hardware efficiency. When evaluating an isolated input ($B = 1$), fully connected layers execute matrix-vector multiplications (GEMV). Under a traffic model that streams every weight from DRAM for each input, the processor performs exactly one multiply-accumulate operation per weight ($z_j \leftarrow z_j + x_i W_{ij}$), which counts as two floating-point operations (1 MAC = 2 FLOPs). Because each FP16/BF16 parameter occupies 2 bytes, counting weight traffic alone gives this intensity; activation traffic can only lower it:

$$I_{\text{GEMV, 16-bit}} = \frac{2 \text{ FLOPs}}{2 \text{ bytes}} = 1.0 \text{ FLOP/byte}$$

[41] **FP32 and Reduced Precision Formats**: The IEEE 754 binary32 format uses 32 bits (1 sign bit, 8 exponent bits, 23 fraction bits), whereas 16-bit alternatives split exponent and fraction differently: IEEE 754 FP16 allocates 5 exponent and 10 fraction bits, providing high precision but a narrow dynamic range (6.1×10^{-5} to 65,504), so FP16 training multiplies the loss by a scale factor before the backward pass to keep small gradients representable (dynamic loss scaling), while Brain Floating Point (BF16) allocates 8 exponent bits matching FP32's normal exponent range to reduce underflow risk relative to FP16; sufficiently small values can still underflow. INT8 further quarters the 32-bit footprint into 8 integer bits (-128 to 127), widely used for high-throughput inference after calibration or quantization-aware training. See section D.4 for the formal numerical comparison.

At 32-bit precision (FP32), where each parameter occupies 4 bytes, arithmetic intensity drops to:

$$I_{\text{GEMV, 32-bit}} = \frac{2 \text{ FLOPs}}{4 \text{ bytes}} = 0.5 \text{ FLOP/byte}$$

If this intensity lies below the target machine's ridge point $R_{\text{peak}}/\text{BW}$, the roofline estimate is bandwidth limited. This DRAM-streaming assumption does not describe repeated requests whose weights remain in cache, as the small MNIST model can; the traffic and bandwidth must refer to the memory tier actually supplying the operands.

Aggregating requests into a batch of size $B > 1$ converts the workload into a general matrix-matrix multiplication (GEMM). Each parameter loaded from memory is now reused across all B inputs in parallel, raising the weight-only intensity proportionally with B (reaching $B \times 1.0$ FLOP/byte in FP16). Activation traffic, available parallelism, and implementation determine how closely execution approaches this estimate or the machine's compute ceiling.

Batching also increases the activation values produced: the MNIST network produces 202 layer-output values per image (986 values including the input), or 6,464 layer-output values for a batch of 32. These are cumulative counts across layers; simultaneous storage depends on the buffer lifetimes developed next. Production systems serving interactive traffic must also limit the wait to accumulate inputs according to their service-level latency budget.

↪ How serving systems batch incoming requests dynamically to amortize parameter transfers is developed in section 13.7.3.

Memory allocation during inference can be substantially leaner than during training because intermediate activations can be released after their last consumer. Under the retained-activation training scheme used here, activation tensors persist until backpropagation reaches their corresponding layers, producing cumulative memory consumption across the entire network depth ($\sum_\ell M^{(\ell)}_{\text{activations}}$, where $M^{(\ell)}_{\text{activations}}$ is the byte size of layer ℓ's activations). For this sequential feed-forward graph, adjacent input and output buffers coexist during a layer's execution. Once layer $\ell+1$ finishes execution, the activation buffer for layer ℓ is dead and its allocated storage can be reclaimed or overwritten. Serving runtimes exploit this lifetime property through static memory planning or double-buffering (ping-pong allocation), alternating between two physical memory blocks as data advances through the layers. A skip connection or other later consumer extends the lifetime and must be included in the memory plan. For the sequential graph under this scheme, peak transient activation memory falls from the cumulative total across all layers to the maximum sum of adjacent layers: $\max_\ell \left(M^{(\ell)}_{\text{activations}} + M^{(\ell+1)}_{\text{activations}}\right)$.

Hardware-specific compilation further reduces latency by eliminating off-chip memory traffic. Accelerator chips are physically partitioned between small, ultra-fast on-chip storage (registers and SRAM scratchpads directly adjacent to compute cores) and large off-chip device memory (such as high-bandwidth memory (HBM) or DRAM reached across wide physical buses). Moving data across that external bus consumes orders of magnitude more time and energy than local register access. In a naive, unfused execution path, evaluating $\text{ReLU}(\mathbf{xW} + \mathbf{b})$ launches three separate processor routines (kernels): the matrix multiplier computes $\mathbf{xW}$ and writes intermediate activations back across the bus to off-chip HBM; the bias kernel reads them back, adds $\mathbf{b}$, and writes them out again; and the activation kernel reads them a third time to apply the threshold. Kernel fusion (also called operator fusion) combines these consecutive operations into a single compiled routine. Intermediate results remain staged inside local registers and on-chip SRAM, so of the five off-chip transfers of the layer-sized intermediate tensor in the unfused path (three writes and two reads), only the final write remains, and two kernel launches disappear with them.

↪ How frameworks decide which operator chains to fuse, and what fusion saves against the memory wall, is developed in section 7.3.1.

Eliminating the backward pass also changes the constraints on reduced numerical precision. Mixed-precision training must preserve sufficient range and update precision for learning, often retaining higher-precision accumulators or parameter copies. Inference has no gradient updates to preserve, so frozen weights and activations can frequently use lower-bit representations, such as FP16 or INT8, provided calibration and task-level evaluation establish an acceptable impact on final predictions. Converting FP32 parameters to INT8 quarters their stored bytes and, when those parameters stream from memory, their transfer volume. For the MNIST network, halving the storage width cuts the parameter and activation byte footprint by half; realized latency speedups then depend on whether the target processor provides dedicated low-precision arithmetic pipelines (such as Tensor Cores or vector dot-product instructions).

↪ Post-training quantization techniques and calibration strategies for converting floating-point weights to INT8 execution are developed in section 10.4.3.1.

Although demonstrated here on a feedforward network, four mechanics form the core design space of production inference engines: amortizing parameter transfers through batching, bounding live ranges through buffer reuse, eliminating memory round trips through kernel fusion, and relieving bus bandwidth through quantization. Specialized network structures introduce additional computational patterns: convolutional operators share weights across spatial dimensions to heighten arithmetic intensity, recurrent architectures process sequential steps with iterative state dependencies, and transformer-based models must manage a key-value cache of stored intermediate results from earlier tokens, which grows with every token generated.

5.6.2 Output interpretation and decision making

Neural network outputs become useful only after they are converted back into decisions a conventional system can act on. Preprocessing bridges real-world data into tensor form; postprocessing maps neural outputs into labels, decision thresholds, validation logic, error handling, and downstream messages. In the MNIST running example, raw logits are not enough. A digit-recognition system needs the most likely digit class, a confidence score whose calibration has been evaluated, and a fallback route that directs ambiguous or low-confidence predictions to human review or a secondary recognizer.

That fallback route is itself a design decision with a measurable trade. A confidence threshold on the top-class probability sends every prediction below it to a person. Raising the threshold reduces the accepted fraction and increases review workload. For a fixed set of scored inputs, it cannot increase the number of accepted mistakes, but the error rate among the remaining accepted predictions can rise or fall because both the error count and the accepted count change. Engineers therefore measure a risk-coverage curve, plotting error among accepted predictions against the fraction of inputs accepted, on held-out data that resembles deployment traffic. A usable operating point must meet the error budget while keeping rejected traffic within review capacity. Softmax probabilities need not be calibrated, and distribution shifts can change both counts, so the selected threshold and its review workload require monitoring. Section 5.7 shows a production document-recognition system setting exactly this boundary.

These boundary stages exhibit an entirely different computational profile from the forward pass. While neural network execution exploits massively parallel matrix units on accelerators, preprocessing (such as image decoding and normalization) and postprocessing (such as argmax reduction, threshold checking, and message serialization) often execute on the host CPU, although implementations can parallelize or accelerate them. Amdahl's law bounds end-to-end speedup by the fraction of time that is not accelerated, so as accelerator kernels drive forward-pass execution times toward zero, these unaccelerated CPU stages and their associated PCIe data transfers expand to dominate end-to-end request latency.

The complete neural network lifecycle, from architecture design through training to inference deployment, is a set of mathematical operations with quantifiable resource costs. These operations have so far lived in the controlled environment of the MNIST running example, where data is clean, latency is unconstrained, and hardware is unchallenged. A production deployment, where none of those conditions holds, is the real test of whether the ledgers are complete.

> Checkpoint 5.5: Complete neural network system
>
> Before turning to how these mechanics first performed in deployment, verify your understanding of the complete neural network lifecycle:
>
> Use the MNIST classifier ($784 \to 128 \to 64 \to 10$) as the running production system.
>
> - ☐ **System lifecycle**: can you trace the lifecycle from architecture design to training loop to trained model to inference deployment?
> - ☐ **Training vs. inference memory**: at a matched batch size, can you explain why this ledger gives training roughly 3.4× the memory of inference, naming the gradients, optimizer state, and saved-activation terms?

- ☐ **End-to-end trace**: can you trace one digit image through preprocessing, forward propagation, output probabilities, postprocessing, and final prediction?
- ☐ **Bottleneck spotting**: a stage that processes one image at a time must sustain a mean service time no greater than 10 ms to support 100 images per second; can you identify a stage that could limit that rate? Why is this completion interval not an end-to-end request-latency measurement when stages overlap, and why do unaccelerated stages matter under Amdahl's law?
- ☐ **Human-in-the-loop**: can you explain how a confidence threshold decides which predictions go to human review, what moving it trades, and why it must be monitored after deployment?

The analytical resource ledgers developed across this chapter quantify compute, memory, and bandwidth in isolation. Real deployments, however, subject these calculations to the unyielding deadlines of physical machinery. Decades before the convergence of massive datasets, deep architectures, and specialized accelerators, postal mail sorting subjected neural inference to precisely these operational constraints.

5.7 Handwritten Digit Recognition

In the late 1980s, LeCun and colleagues demonstrated backpropagation-based recognition of handwritten ZIP-code digits from USPS-supplied images (LeCun, Boser, et al. 1989). Related convolutional document-recognition work later supported commercially deployed bank-check readers (LeCun et al. 1998). These were distinct systems, but they exposed the same engineering pattern: preprocessing normalized variable handwriting, neural inference produced candidate digits, rejection thresholds sent uncertain predictions to human review, and downstream logic converted accepted predictions into operational decisions. That shared pipeline serves as a systems case study in end-to-end machine learning design, illustrating how preprocessing, model structure, and rejection policy interact under physical throughput constraints.

5.7.1 The document-recognition challenge

Operating constraints in document sorting stem directly from the physical handling machinery. In an illustrative postal sorter, envelopes move past optical scanners along mechanical transport belts at fixed velocities, leaving a bounded interval per item to acquire the image, isolate characters, evaluate the model, and actuate physical diverter gates. A misrouted envelope or misread check incurs severe operational costs in manual re-handling or downstream financial reconciliation. Because raw accuracy on uncontrolled handwriting cannot reach 100 percent, the pipeline must couple automated inference with an explicit rejection mechanism that routes ambiguous inputs to human operators. The sample grid in figure 5.18 catalogs this sensory variability across real-world postal envelopes, where character segmentation and recognition contend with heavy cursive slant, inconsistent stroke widths, broken loops, and dense background texturing across city names and five-digit ZIP codes.

Absorbing this physical variability requires distributing the computational burden across multiple pipeline stages. Upstream image capture and normalization bound the input distribution presented to the neural network, neural inference maps pixel patterns to class scores whose calibration must be evaluated, and downstream threshold logic governs the trade-off between automated throughput and error.

5.7.2 Engineering process and design decisions

Recognizing a clean, centered digit is straightforward. Recognizing handwriting amid variable backgrounds, scale, alignment, and image quality requires engineering decisions at every stage from data preparation to deployment.

The dataset histories must remain separate. Early ZIP-code research used postal digit images supplied for that task (LeCun, Boser, et al. 1989). MNIST was later constructed from National Institute of Standards and Technology (NIST) handwritten-digit databases and standardized into size-normalized benchmark images (LeCun et al. 1998). It did not arise from collecting live envelopes

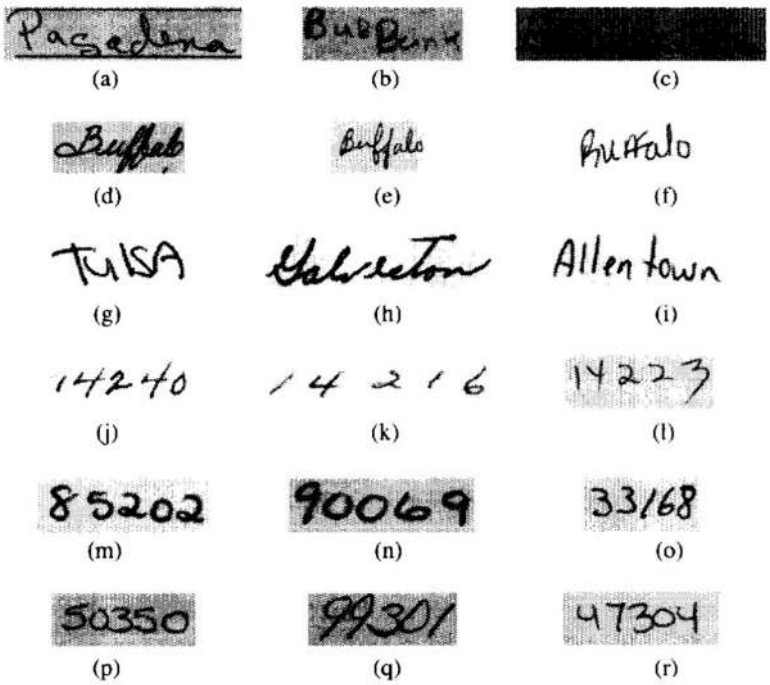


Figure 5.18: Handwritten Address Variability: Address and ZIP-code samples exhibit wide variation in stroke width, slant, spacing, and character formation. A document-recognition pipeline must absorb this variation without confusing legitimate writing differences with changes in the intended characters. Samples from the CEDAR CD-ROM 1 USPS Office of Advanced Technology database.

under varying colors, textures, lighting, and orientations. Across both datasets, however, the engineering requirement was the same: preprocessing had to reduce irrelevant variation without erasing the stroke information needed for recognition.

Architecture design balances accuracy, processing time, and memory cost. For the chapter's standardized 28×28 digit input, the central question is how much model capacity the available latency and hardware budget can support. Fully connected layers scale with the product of input and output dimensions, demanding substantial parameter storage and memory bandwidth. Convolutional layers instead slide one small set of weights across every image position, reducing parameter volume through sharing. Activation storage requires a separate calculation using each layer's output shape and the batch size; fewer weights do not guarantee smaller intermediate tensors.

Training must cover handwriting variation rather than optimize only for a curated test set. Preprocessing normalizes size and orientation, while augmentation can expose the model to additional plausible variation. Evaluation must then measure performance across relevant writing styles and operating conditions.

Confidence thresholds determine the division of labor between automation and human review. A higher threshold sends more items to manual handling; its effect on error among the accepted items must be measured. An expected-cost rule chooses threshold τ to minimize $r_{\text{manual}}(\tau)\,C_{\text{manual}} + r_{\text{error}}(\tau)\,C_{\text{error}}$ subject to throughput and latency constraints, where $r_{\text{manual}}(\tau)$ and $r_{\text{error}}(\tau)$ are, respectively, the fractions of all arriving items sent to manual review and accepted with an erroneous automated classification at threshold τ, and C_{manual} and C_{error} are their per-item costs. The error fraction uses the same arrival denominator as the review fraction. If error is reported only among accepted items, multiply that conditional rate by the accepted fraction before applying the cost rule; rejected items incur review cost rather than an automated-error cost in this simplified ledger.

↩ Why evaluation on curated data must be validated against field conditions is established in section 3.10.

5.7.3 Shared pipeline architecture

The composite architecture in figure 5.19 separates the computational responsibilities shared by postal ZIP-code research and deployed bank-check readers; it does not reconstruct a single historical machine. Upstream preprocessing ingests raw scanner data and filters input noise; the core deep-learning stage evaluates digit representations; and downstream postprocessing inspects confidence metrics against threshold rules, passing accepted classifications to the application while isolating ambiguous digits for further processing or manual review.

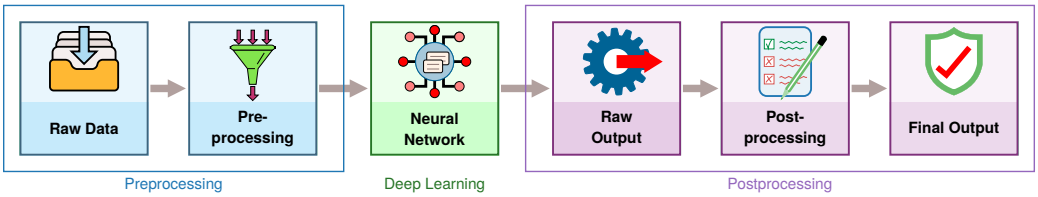


Figure 5.19: Document-Recognition Pipeline: A composite six-stage flow distilled from postal ZIP-code research and deployed check-reading systems. Image acquisition and conventional preprocessing prepare handwritten digits for neural inference, and raw scores then pass through postprocessing to the final application output.

The pipeline begins with image acquisition. In postal research, the input was an image of a handwritten ZIP-code region; deployed check readers similarly began with scanned document images. Acquisition quality determines how much variation the later preprocessing and recognition stages must absorb.

Preprocessing locates the relevant field, separates or otherwise identifies its characters, and normalizes the resulting images. Thresholding, connected-component analysis, and size normalization are representative conventional techniques. The chapter uses standardized 28×28 images for its MNIST calculations; that dimension is a teaching convention here, not a claim about every postal or check-reading deployment.

The neural network then processes each normalized digit image. The 1989 ZIP-code research system used an early LeNet variant (LeCun, Boser, et al. 1989) with approximately 10,000 parameters, occupying approximately 40 KB in single precision. This parameter footprint is an order of magnitude smaller than the 109.4K in this chapter's fully connected MNIST network. A 40 KB parameter footprint reduces the storage the implementation must accommodate, but the footprint alone establishes neither cache residency nor compliance with a sorting deadline. Those conclusions require the machine's cache organization and an end-to-end execution measurement.

Postprocessing converts digit scores into application decisions. For a ZIP-code task, one uncertain digit can send the complete code for further processing or human review. Deployed document readers use the same general principle: accept sufficiently confident predictions and reject uncertain cases instead of forcing an answer.

Deployed document readers must satisfy end-to-end timing constraints spanning acquisition, preprocessing, inference, postprocessing, and downstream action. The exact machinery differs between mail and financial-document applications, but throughput depends on the complete pipeline rather than neural-network latency alone.

5.7.4 Performance outcomes and operational impact

The 1989 ZIP-code study reported its model, recognition, throughput, and training measurements as one experimental system. Table 5.14 keeps those measurements together, revealing how rejection policy and preprocessing affect the meaning of raw classifier speed (LeCun, Boser, et al. 1989).

Table 5.14: Early ZIP-Code Recognition Results: Measurements from the 1989 backpropagation system applied to USPS-supplied digit images. The rejection result shows why an operational error rate must be read together with the fraction of inputs withheld from automatic classification (LeCun, Boser, et al. 1989).

Metric	Reported Result
Error and rejection	1% classification error after rejecting 12.1% of test digits

Metric	Reported Result
Throughput	10–12 classifications/sec including acquisition; over 30/sec after normalization
Model parameters	~10,000
Training	23 passes; about 3 days on a Sun-4/260 workstation

These measurements highlight three systems trade-offs across the pipeline:

First, the measured rate depended on the measurement boundary. The system exceeded 30 classifications per second after normalization but processed 10 to 12 per second including acquisition. The reciprocal rates are completion intervals, not necessarily individual-digit latencies when stages overlap. The comparison exposes surrounding acquisition and preprocessing work, but does not allocate that work among mechanical handling, scanning, and segmentation. Stage timings are needed before choosing which part of the complete path to accelerate.

Second, operational reliability depended on an explicit rejection pathway. The experiment rejected 12.1 percent of test digits rather than forcing a classification for every input. Rejecting this tail of high-uncertainty predictions reduced the observed classification error on the remaining automated stream to 1 percent. This is one measured point on the system's risk-coverage curve. Other thresholds require measuring both the accepted fraction and its conditional error rate; higher rejection alone does not guarantee a lower error rate among accepted items. In a deployed pipeline, the accepted fraction determines automated volume, while the rejected fraction becomes a further-processing or human-review workload that the expected-cost rule above prices.

Third, training-time cost constrained the exploration of model sizes. Training 10,000 parameters required 23 passes across roughly three days on a Sun-4/260 workstation. That measurement shows the cost of the reported configuration; it does not establish a universal maximum model size for the era. Larger candidates would require a new compute, memory, and training-time budget.

Maintaining these systems in production introduces lifecycle requirements beyond model training. Changes in scanner optics, ink formulations, envelope textures, or regional writing styles cause input distributions to drift over time. Production monitoring must track input image statistics and rejection rates alongside raw classification latency, prompting investigation when rejection threatens review capacity. Whether the remedy is data collection, preprocessing repair, threshold adjustment, or retraining depends on the cause and on validated error costs.

↪ How production monitoring detects silent degradation from input drift while code and weights stay unchanged is developed in section 14.6.

5.7.5 Key engineering lessons and design principles

Across postal ZIP-code research and deployed check readers, neural computation succeeded only when the surrounding pipeline shared its operating constraint. Each of the chapter's mechanisms reappears in that pipeline at system scale. The forward-pass ledger set the cost of inference, but the gap between more than 30 classifications per second after normalization and 10 to 12 per second including acquisition is the Amdahl effect of the boundary stages that the inference section predicted. The network's output scores became decisions only through a threshold whose position traded automated volume against residual error. The convolutional reader's parameter footprint, an order of magnitude below the fully connected network's, illustrates that connectivity patterns, not layer depth alone, govern the state a model must store.

Modern edge systems face different power, latency, and deployment budgets, but the same three obligations hold. Preprocessing must absorb the variability of the physical world before the network sees it, confidence thresholds must protect the downstream error budget, and end-to-end throughput, not model latency alone, governs how much model capacity the system can afford.

> Systems Perspective 5.5: Reading historical system comparisons
>
> The early LeNet-class reader and this chapter's MNIST multilayer perceptron show continuity in computation, not a controlled performance benchmark. Their datasets, architectures, software stacks, machines, and operating targets differ. The defensible comparison is structural: both transform normalized pixels into digit scores, both depend on learned parameters and a preprocessing pipeline, and both need an explicit policy for uncertain outputs.

The order-of-magnitude parameter gap between the two networks describes model state; it does not by itself establish latency, energy, throughput, or cost. Those outcomes require a fixed workload and measurements that identify the hardware, numerical precision, runtime, batch size, and end-to-end decision path. Separating these variables prevents a faster deployment from being attributed entirely to silicon when the data, model, software, or operating policy also changed.

The document-recognition pipeline demonstrates the foundational alignment required in ML systems engineering. Automated classification at scale requires continuous coordination across the operational data distribution, the network's algorithmic capacity, and the underlying accelerator constraints. Evaluating these interacting pressures systematically requires analyzing the pipeline through the tripartite lens of the D·A·M taxonomy: data distribution, algorithmic capacity, and machine constraints.

↩ The D·A·M taxonomy and its three diagnostic axes are established in section 1.5.

5.8 D·A·M Reading of Neural Computation

Across the document-recognition histories, different resources served distinct purposes. USPS-supplied ZIP images grounded early recognition research, LeNet supplied an algorithmic mechanism, later bank-check readers supplied a documented deployment setting, and MNIST supplied a standardized benchmark derived from NIST data. They do not form a single system, but together they instantiate the tripartite alignment of the D·A·M taxonomy: whether the data captures the task distribution, whether the algorithm can learn the required transformation, and whether the machine can execute that computation within physical latency, memory, and energy budgets.

The data axis establishes whether the training and evaluation examples faithfully capture the operational distribution. USPS ZIP-code images, MNIST digits, and bank-check inputs share digit semantics but stem from distinct acquisition processes and noise environments: scanning resolution, thresholding artifacts, and document layout make them non-interchangeable. Preprocessing can normalize variations in scale and orientation, but it cannot synthesize unrepresented handwriting styles or correct a labeling policy that diverges from downstream operational decisions. Data also imposes a physical footprint: input resolution, batch size, tensor layout, and numeric precision dictate the volume of bytes that must cross memory hierarchies into accelerator memory.

Those incoming bytes define the input to the algorithm axis, which dictates the mathematical operations required to learn or evaluate representations. In the chapter's running network, the 784-to-128 fully connected layer defines both the initial representational capacity and the largest parameter block in the model. Activation functions introduce nonlinearities that govern which decision boundaries can be formed and whether backpropagated gradients remain numerically stable; the loss function and optimizer determine how prediction errors drive parameter updates. These choices establish the mathematical work the system must execute, but an algorithmic formulation remains viable only if its statistical assumptions match the data distribution and its computational graph fits within the hardware budget.

That hardware budget is dictated by the machine axis, which determines whether the required linear algebra can execute within physical latency, memory bandwidth, memory capacity, and thermal limits. Forward propagation streams parameter weights and input activations into arithmetic units; backpropagation uses saved or recomputed forward values to compute gradient tensors through the reverse graph, and the optimizer then updates parameters and any optimizer state. Increasing batch size amortizes parameter loading by raising arithmetic intensity, but it expands activation storage proportionally. Lowering numeric precision from FP32 to 16-bit or 8-bit formats halves (16-bit) or quarters (8-bit) memory traffic and unlocks specialized matrix execution units, provided the reduced dynamic range preserves numerical stability and task accuracy. The physical machine therefore constrains not merely how quickly a fixed architecture executes, but which batch sizes, parameter counts, and numeric formats are physically realizable.

The chapter's own ledgers show why these axes cannot be optimized in isolation. Batch size, an algorithm choice, set both the activation memory and the arithmetic intensity the machine sees; numerical precision, a machine choice, decided whether small gradients survived the backward pass; and the digit images' resolution, a data property, fixed the width of the largest weight matrix.

Expanding a dataset improves statistical coverage but increases storage traffic; widening a layer improves model capacity but exceeds accelerator memory; reducing arithmetic precision boosts execution throughput but risks task-level degradation.

The quantitative analyses in this chapter turn this diagnostic framework into concrete engineering practice. When confronted with an operational symptom, such as insufficient accuracy, a violated latency service-level objective, or an out-of-memory fault, engineers formulate competing hypotheses across all three axes. Disambiguating them requires measuring the binding physical or statistical quantity: sample coverage and label fidelity for data, gradient dynamics and error distributions for algorithm, or compute utilization, memory bandwidth saturation, and capacity footprints for machine. Because an intervention on one axis routinely shifts the binding constraint to another, validation requires profiling end-to-end performance under the intended production workload.

Reasoning through the D·A·M taxonomy prevents treating systems problems as isolated software bugs. In practice, however, engineering decisions frequently falter when teams rely on intuitive assumptions, presuming that deep networks cannot be debugged, that depth invariably outperforms width, or that software-level accuracy guarantees operational throughput. Identifying where these heuristics fail requires examining the recurring pitfalls that arise when neural computation meets physical hardware.

↪ Screening heuristics that narrow these competing hypotheses, each confirmed against profiler traces and hardware counters, are developed in section A.5.

5.9 Fallacies and Pitfalls

Statistical learning adds failure modes that line-by-line code inspection alone cannot reveal. The following fallacies and pitfalls can lead teams to misallocate effort or deploy inappropriate solutions.

Fallacy: *Neural networks are entirely opaque and cannot be debugged.*

Neural networks do not offer the line-by-line causal trace of traditional code, but they are not beyond diagnosis. Activation visualization can reveal learned patterns, gradient analysis measures local input sensitivity, and ablation studies estimate component contributions. These tools provide partial evidence rather than a complete explanation: saliency can be unstable, correlated features complicate attribution, and an ablation may change the behavior it is meant to probe. For the MNIST classifier in section 5.4.2, first-layer weights can still provide a useful check on what patterns the model has learned. Debugging a statistical system therefore combines model-level probes with data slices, controlled interventions, and end-to-end evaluation.

Pitfall: *Discarding domain expertise because a deep model is available.*

Teams assume automatic feature learning removes the need for domain knowledge. Successful systems require domain expertise at every stage: architecture selection, training objective design, dataset curation, and output interpretation. The document-recognition history in section 5.7 shows why confidence thresholds must reflect operating economics and route uncertain cases to human operators. Without domain knowledge, teams can deploy networks that look strong on a test set but fail in production because their thresholds, fallback paths, or error costs are wrong.

Fallacy: *Deeper networks are always more accurate than wider ones.*

Engineers assume that stacking more layers is the primary path to higher accuracy, since depth enables hierarchical feature extraction. In practice, depth alone encounters diminishing returns. ResNet showed that very deep residual networks can train effectively, but its ImageNet results also make clear that more layers must be weighed against added training and inference cost (He et al. 2016a). EfficientNet, a family of image classifiers, later demonstrated that compound scaling of width, depth, and input resolution can outperform depth-only scaling at a given resource budget (Tan and Le 2019). The lesson is not that depth is bad; it is that teams should profile capacity utilization and scale the architecture dimension that gives the best accuracy per FLOP.

Pitfall: *Using neural networks for problems solvable with simpler methods.*

Teams assume deep learning always performs better. On small datasets or approximately linear relationships, a simpler model such as logistic regression may match or outperform a neural network while requiring less training, memory, and maintenance. The magnitude of that advantage is workload dependent, so teams should benchmark a simple baseline and justify added complexity through measured task-specific gains. Neural networks excel at hierarchical pattern discovery (section 5.2.3)

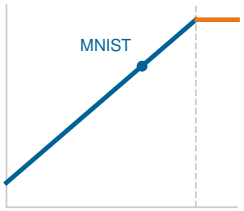
but impose additional overhead. Use them when their representation capacity produces enough benefit to justify the measured systems cost.

Fallacy: *Training data distribution issues can be fixed after model design.*

Teams treat training as mechanically feeding data through architectures. Networks on imbalanced datasets can exhibit poor minority-class performance: a fraud detector with 99:1 imbalance achieves 99 percent accuracy by always predicting "not fraud" while catching zero fraud cases. The unweighted batch average of equation 5.15 can underweight rare but critical classes unless sampling, weighting, loss design, and evaluation address the imbalance. Teams that skip exploratory data analysis may deploy models with strong aggregate metrics but unacceptable minority-class performance.

Pitfall: *Deploying research models to production without addressing system constraints.*

Production adds constraints absent from research: latency budgets (50–100 ms end-to-end), memory limits (2–4 GB for edge devices), and concurrent loads (100–1,000 requests per second (RPS)). The complete pipeline includes preprocessing, inference, and postprocessing (section 5.6). A model achieving 20 ms inference fails its 50 ms budget when preprocessing adds 25 ms and postprocessing adds 10 ms (55 ms total). Separating model development from system design therefore wastes effort on accuracy while ignoring deployment feasibility.



Low arithmetic intensity can limit a small workload even on a large GPU.

Fallacy: *More compute automatically means faster training.*

Teams purchase expensive GPUs expecting proportional speedups, but memory traffic or launch overhead may bind performance instead. Arithmetic intensity helps test the bandwidth hypothesis against the machine's ridge point. Under the separate-layer FP32 traffic model of notebook 5.1, which reads parameters once, reads the input, writes the final output, and writes then reads each hidden tensor, the MNIST forward pass in table 5.7 reaches approximately 11.9 FLOP/byte at batch 32 and still less at batch 1. That intensity alone cannot establish the bottleneck without the matching machine balance. The ridge point $R_{\text{peak}}/\text{BW}$ introduced with the iron law (peak compute in FLOP/s divided by peak memory bandwidth in bytes/s) is the minimum arithmetic intensity needed to saturate the compute units. For this FP32 workload, compare the peak rate for the actual FP32 execution mode with bandwidth at the memory tier supplying its operands. A reduced-precision matrix-engine peak divided by off-chip bandwidth describes a different execution regime. If the matching ridge point exceeds 11.9 FLOP/byte, this traffic model predicts a bandwidth ceiling below that mode's peak compute. Actual small-kernel execution may instead be limited by launch overhead or insufficient parallel work, and cache reuse changes traffic at the measured memory tier. The Roofline Model in section D.2.1 explains why additional peak FLOP/s cannot help until the workload supplies enough reusable work per byte. Caches, fusion, vectorization, and launch cost make measured speedup the final test.

Pitfall: *Extrapolating accuracy improvements without considering diminishing returns.*

Teams may extrapolate a 5-point gain from scaling 10K parameters to 100K into another 5-point gain at 1M. Empirical improvement instead diminishes with scale according to the task, metric, data quality, model family, and scaling regime. The ImageNet rows of table 5.4 cannot settle the question. Error falls from AlexNet's 15.3 percent to 3.6 percent for a ResNet ensemble while training FLOPs rise from 5×10^{17} to roughly 10^{19}, but those rows mix new architectures with more compute, so they do not isolate the return on scale alone. Teams must therefore measure marginal return within one model family in their own regime rather than extrapolate linearly.

These fallacies and pitfalls share a common root: treating statistical behavior as if code behavior alone determined it. Recognizing them early directs attention toward the appropriate measurements and diagnostics.

5.10 Summary

Neural computation translates mathematical formulations into physical machine execution. Linear algebra and calculus determine operation cost under the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). Forward propagation establishes the computational work O, often dominated by matrix multiplications, while the complete training step adds backward computation and optimizer

updates. Backpropagation needs saved or recomputed forward values and gradient storage; the optimizer may add persistent state. Architecture, batch size, precision, optimizer, and recomputation strategy determine the state footprint. Residency, reuse, and execution then determine the bytes transferred, D_{vol}, which need not equal that footprint. These mechanics make memory footprints, arithmetic intensities, and latency limits estimable before deployment.

Replacing hand-written decision rules with parameters learned from data transforms feature engineering into machine execution. Layered compositions of weighted sums, nonlinear activations, and gradient updates form hierarchical representations that convert raw inputs into predictions. Training adjusts connection weights through iterative optimization, while inference applies fixed parameters to incoming data. These phases impose divergent physical demands. In this chapter's FP32 ledger, a batch-32 training step requires ~3.4× the memory of batch-32 inference because training retains saved activations and gradients alongside the optimizer's additional buffers. In production, as the document-recognition case study demonstrated, these mathematical kernels do not run in isolation; preprocessing, neural inference, rejection, and postprocessing must jointly satisfy a unified latency and reliability budget.

Key Takeaways: The math behind the model

- **Each paradigm shift changes the systems cost of representation**: In this 28×28 digit example, the operation count rises from ~100 comparisons (rule-based) through ~8,000 operations (classical ML) to 109,184 MACs (deep learning), a 1,100× scenario increase that reshapes hardware requirements.
- **Neural networks learn representations from data**: These networks replace some hand-coded features with hierarchical representations learned from examples. That flexibility shifts the burden toward representative data, training compute, and validation.
- **Training and inference often emphasize different objectives**: Training commonly favors efficient throughput across repeated updates, while online inference commonly favors per-request latency; offline inference may favor throughput instead. Batch size links utilization, memory, throughput, latency, and statistical behavior across these regimes.
- **Activations are math and hardware**: ReLU is generally cheaper to implement than sigmoid or tanh, and its unit gradient for positive inputs avoids activation saturation on that branch. Its practical advantage combines implementation cost with optimization behavior.
- **Forward propagation often centers on matrix work**: Dense matrix kernels exceed 99 percent of FLOPs in this chapter's dense-network example and dominate many important neural workloads, which is why specialized matrix hardware can substantially outperform general-purpose execution for those operations.
- **Backpropagation needs the path**: Solving credit assignment requires retaining or recomputing the values used by the backward pass, so memory cost often determines whether a model can be trained on a given device and motivates techniques that reduce, recompute, or partition training state.
- **The complete ML pipeline determines end-to-end performance**: Preprocessing, neural computation, and postprocessing all contribute to latency and reliability. The document-recognition examples demonstrated that production success depends on the entire pipeline operating within real-world constraints, not on model accuracy on a test set alone.

The escalation from hand-written rules to learned features is the algorithm axis of the D·A·M taxonomy made concrete. Learning representations instead of writing them spends arithmetic and memory in place of human design effort, and every systems dimension in table 5.3 shifts as a result. The bitter lesson predicts that this exchange wins once computation is cheap enough to afford it, but making the exchange still requires a measured gain that justifies its cost on the intended machine.

The multilayer perceptrons analyzed in this chapter expose a direct resource law, in which dense connectivity forces both parameter storage and multiply-accumulate count to scale with the product

of adjacent layer dimensions. In the first layer alone, a 28×28 MNIST image presents 784 input values, requiring 100,352 weights to connect every input to 128 hidden neurons. That dense connectivity contains no explicit prior for spatial locality, so the model must learn from data which nearby and distant pixel relationships matter, while paying memory bandwidth and storage to evaluate every connection.

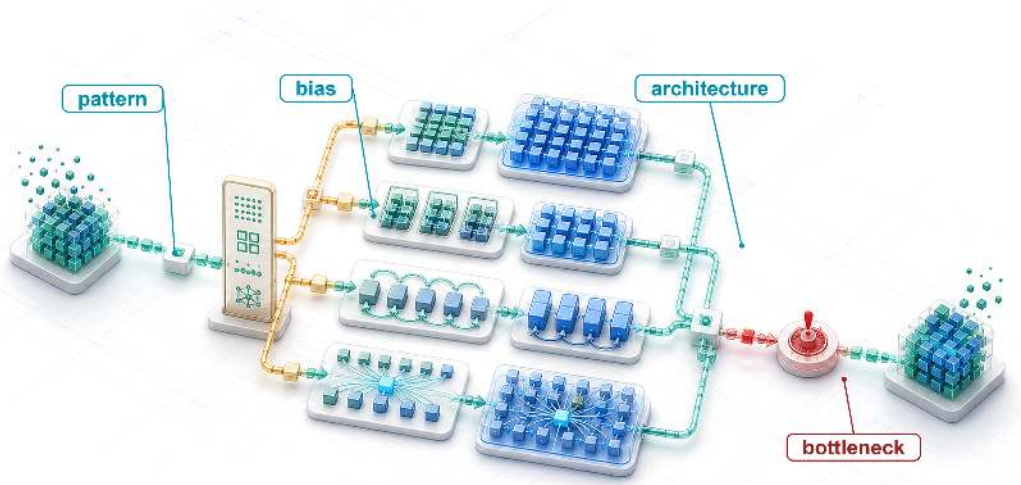
Reading the code reveals what a network is asked to do; reading the math reveals what it will cost to execute. That arithmetic is where the algorithm meets the machine: weights must be stored and transferred, dense layers map to matrix execution engines, and saved activations claim accelerator memory that determines whether training fits on device. Under the silicon contract, the math commits in advance to operations the machine must execute. Auditing that contract allows systems engineers to calculate binding bottlenecks from first principles before profiling confirms where the execution stalls.

What's Next: From universal to specialized

Real-world data is rarely structureless. Images exhibit spatial locality, text contains sequential dependencies, and audio signals follow temporal patterns, yet a fully connected layer charges memory volume D_{vol} and compute work O for every connection regardless. Chapter 6 assembles the same weighted sums, activations, and gradients into architectures shaped like their data, from convolutional and recurrent networks to transformers and embedding-heavy recommenders, each encoding its structural assumption as an inductive bias. Tailoring network topology to data structure reduces redundant data movement and improves weight reuse, but introduces new trade-offs in activation storage, arithmetic intensity, and accelerator parallelization.

6

Network Architectures

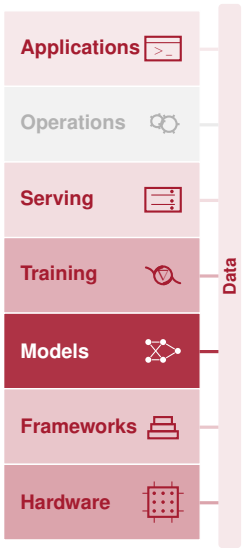


6.1 Architectural Principles
6.2 Lighthouse Roster: Model Biographies
6.3 Workload Signatures: The Arithmetic Intensity Spectrum
6.4 MLPs: Dense Pattern Processing
6.5 CNNs: Spatial Pattern Processing
6.6 RNNs: Sequential Pattern Processing
6.7 Attention: Dynamic Processing
6.8 Transformers: Parallel Sequence Processing
6.9 Sparse Architectures: Recommendation Systems
6.10 What Each Architecture Asks of the Machine
6.11 Fallacies and Pitfalls
6.12 Summary

Purpose

Why is choosing a neural network architecture an infrastructure commitment as well as a modeling decision?

Selecting a neural network architecture is both a modeling decision and a contract with physics. The commitment begins with the structure in the data. An architecture that matches that structure can learn with fewer parameters and examples, while a poor match spends additional data and machine time compensating for the wrong computational pattern. Convolution, attention, recurrence, and embedding lookup do not merely encode different assumptions about data; they create different patterns of computation, state, and data movement. Those patterns determine whether work parallelizes cleanly, whether intermediate state fits in memory, and whether training and serving remain affordable at the required scale. Attention compares every element of a sequence with every other, so doubling the input length roughly quadruples that work, while the cost of a convolution grows in proportion to its input. Two architectures can deliver similar predictive quality yet impose different memory footprints and latency profiles while exposing different opportunities for later efficiency techniques. Benchmark accuracy alone therefore cannot reveal which design remains viable. Architectural commitments also propagate beyond the model. Data pipelines adopt particular input structures, training infrastructure is provisioned around the model's compute profile, serving systems are tuned to its request path, and monitoring inherits its failure modes. Once those dependencies accumulate, replacing the architecture can mean rebuilding much of the surrounding system, a cost that grows with every month the design stays in service. An architecture decision is sound only when its predictive gains remain compatible with the full deployment context. In D·A·M terms, architecture is algorithm-machine co-design at its root, with the mathematical graph determining the work the machine must perform and the machine budget limiting which graphs remain practical.



Learning Objectives

- Distinguish computational characteristics of MLPs, CNNs, RNNs, Transformers, and DLRM-style recommenders
- Explain how inductive biases exploit structure in different data types
- Analyze computational complexity and memory scaling across architectural families
- Identify building blocks such as skip connections, normalization, and gating that enable deep training
- Apply the architecture selection framework to match data characteristics with model designs
- Evaluate how compute, memory access, and data movement determine hardware mapping efficiency
- Critique architecture-selection fallacies under latency, bandwidth, and parallelization constraints

6.1 Architectural Principles

The dense multilayer perceptron (MLP) of chapter 5 paid for a weight between every pixel and every hidden unit, including relationships an image rarely needs. The operations that chapter built (matrix multiplication, activation functions, and gradient computation) are the verbs every network shares. Architectures remove that waste by assembling the same verbs into computational graphs shaped like the data. Under the silicon contract (principle 4), that shape is also an implicit agreement with hardware, trading computational patterns for efficiency on particular problem classes.

Every neural network architecture decides how computation should be organized to match the inherent structure in the data. Images have spatial locality, language has sequential dependencies, and tabular records often lack a fixed spatial or sequential organization. The architecture encodes assumptions about these patterns directly into the computational graph, and those assumptions influence parameter count, hardware utilization, and deployment feasibility. Architecture selection is therefore a systems engineering problem that directly affects the iron law terms: the number of operations O and the volume of data movement D_{vol}. The structural assumptions that each architecture encodes are known as **inductive biases**,[1] and they serve as the unifying concept for this entire chapter.

[1] **Inductive bias**: From Latin *inducere*, "to lead into." Encoding a structural assumption "leads" the model toward a smaller solution space, and every architecture discussed here (multilayer perceptron (MLP), convolutional neural network (CNN), recurrent neural network (RNN), and transformer) is defined by its choice of bias. A CNN's locality bias cuts parameters by orders of magnitude vs. an equivalent MLP, directly shrinking the iron law's O and D_{vol} terms, while global attention trades quadratic score computation for long-range connectivity.

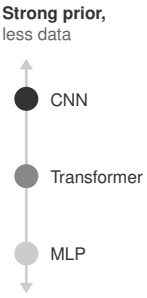
Definition 6.1: Inductive bias

Inductive bias is a structural constraint built into a model architecture that restricts the hypothesis space, enabling generalization from finite data by encoding domain-specific assumptions (such as spatial locality or sequential ordering) directly into the computational graph.

1. **Significance**: Inductive bias can reduce the dataset size (D) required for generalization. For one output feature detector over N_{pix} pixel positions with C_{in} input channels, a dense connection uses $\mathcal{O}(N_{\text{pix}}C_{\text{in}})$ parameters, while a shared $K\times K$ CNN filter uses $\mathcal{O}(K^2C_{\text{in}})$ parameters and is applied at every position, costing $\mathcal{O}(N_{\text{pix}}K^2C_{\text{in}})$ operations. For a 224×224 RGB image, a shared 3×3 filter therefore uses roughly 5,575.1× fewer parameters than one dense feature detector, reducing parameter storage and often the data required to avoid overfitting.
2. **Distinction**: Unlike regularization (which penalizes hypothesis complexity at training time via L1/L2 terms), inductive bias restricts the hypothesis space at architecture design time: a CNN represents nonlocal functions less directly than local patterns, while regularization discourages complexity through the training objective.
3. **Common pitfall**: A frequent misconception is that stronger inductive bias is always better. A strong locality bias (CNN) excels on spatial data but represents long-range

dependencies in language less directly than global attention, which supports long-range dependencies at $\mathcal{O}(S^2)$ score-computation cost.

A convolutional neural network (CNN) encodes an inductive bias of spatial locality: nearby pixels matter more than distant ones. A transformer lets each element attend to any other, enabling long-range relationships at quadratic score-computation cost. These biases help architectures learn efficiently by restricting the space of functions they represent. Without an appropriate bias, a model may require substantially more data and compute to learn the same structure. That exchange is the bitter lesson seen from inside a single architecture. A strong prior substitutes engineered structure for data and compute, while the transformer's weak priors bet that data and compute will keep growing, a bet that has repeatedly won at scale. The unified framework in section 6.10 brings these architectural families together once each bias has appeared in practice.



A stronger prior substitutes engineered structure for data and compute.

↩ The bitter lesson, that general methods which scale with computation outpace hand-built structure, is established in section 1.4.

Machine learning systems face a core engineering trade-off: **representational power vs. computational efficiency**. Under the iron law of ML systems (principle 3), architectural choice is the primary determinant of the operation-count term O. A transformer's attention mechanism enables global relationships but scales as $\mathcal{O}(S^2)$ operations with sequence length S; a CNN exploits spatial locality to reduce operations to linear scaling in the number of spatial positions. Matching the right inductive biases to a workload's data while setting a manageable operation-count budget defines the practice of neural architecture selection.

Choosing an architectural backbone sets the primary memory layout and compute pattern for downstream hardware. Table 6.1 contrasts the five major neural network families across the data structure each targets, its core innovation, and the system bottleneck it exposes in representative regimes.

Table 6.1: Neural Architecture Families: Each family targets a data structure and exposes characteristic resource demands. The bottleneck column names representative regimes: streamed dense weights stress bandwidth, convolutional reuse raises arithmetic intensity, recurrence creates serial dependencies, dense attention computes quadratic score interactions, and large embedding tables stress capacity. Batch size, precision, kernel implementation, and the target machine determine which demand binds in a deployment.

Architecture	Data Type	Core Innovation	System Bottleneck
MLPs	Tabular/Unstructured	Dense connectivity	Bandwidth with streamed weights at small batch
CNNs	Spatial (images)	Local filters + weight sharing	Compute throughput with sufficient reuse
RNNs	Sequential (time series)	Recurrent state	Sequential dependencies
Transformers	Relational (language)	Dynamic attention	Quadratic score compute; optional S^2 storage
DLRM	Categorical (recommendations)	Embedding tables	Capacity when tables exceed device memory

The five lighthouse models introduced with the iron law return here with their architectures exposed. Earlier chapters used them as named workloads and mapped them onto the workload archetypes of section 2.3.2 (such as the compute beast and the bandwidth hog); this chapter opens each one up to show which architectural choice puts it in its archetype. The two vision lighthouses sit on or near the empirical Pareto frontier plotted in figure 6.1, the set of models that no other plotted model beats on both accuracy and compute at once. The frontier bounds ImageNet top-1 accuracy against logarithmic compute cost, measured in GMACs (billions of multiply-accumulate operations) per image; the other three were chosen for the bottlenecks they isolate: memory bandwidth, memory capacity, and an always-on power envelope.

↩ The five lighthouse models, and the bottleneck each one isolates, are established in section 1.7.2.

The empirical Pareto frontier in figure 6.1 illustrates how successive architecture families occupy distinct computational cost regimes. The dashed envelope links MobileNetV2, EfficientNet-B0, ConvNeXt-T, and EfficientNet-B7. These named models represent historical milestones rather than individual case studies; of them, ResNet-50 and MobileNetV2 receive detailed architectural breakdowns in this chapter. The plotted curve traces the evolution of vision architectures from dense CNNs that prioritized accuracy regardless of operation count, through compact mobile networks

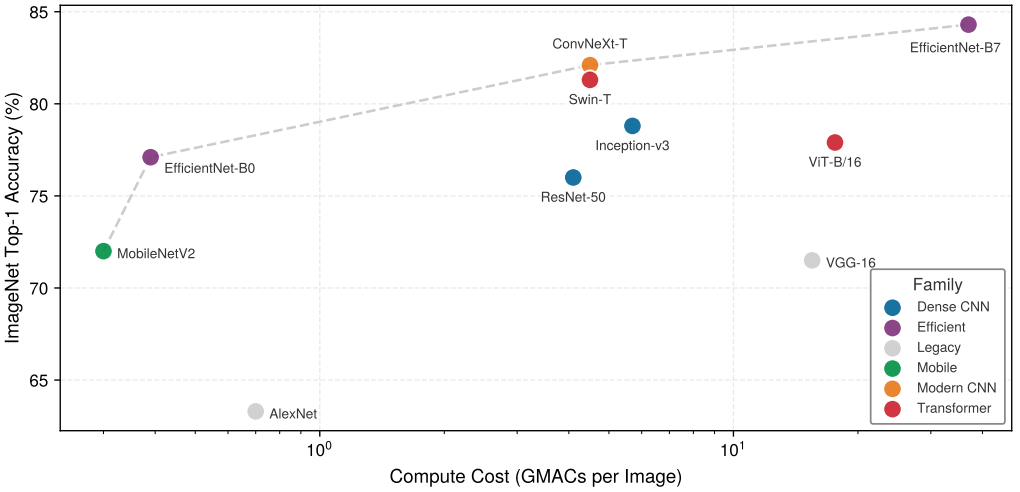


Figure 6.1: The Efficiency Frontier: ImageNet Top-1 Accuracy vs. Computational Cost (GMACs). Representative points are compiled from published model reports for AlexNet, VGG, ResNet, Inception, MobileNetV2, EfficientNet, ViT, Swin Transformer, and ConvNeXt (Krizhevsky et al. 2012; Simonyan and Zisserman 2015; He et al. 2016a; Szegedy et al. 2015; Sandler et al. 2018; Tan and Le 2019; Dosovitskiy et al. 2021; Liu et al. 2021; Liu et al. 2022). The dashed 'Pareto frontier' is an illustrative trade-off curve rather than a benchmark rerun; exact accuracy and reported compute values vary by model variant, training recipe, implementation, and operation-counting convention (multiply-accumulates vs. floating-point operations). The legend distinguishes six families: legacy CNNs (gray), dense CNNs (blue), efficient Mobile architectures (green), EfficientNet-class models (violet), modern CNNs (orange), and Transformers (red). The frontier is led by the efficient and modern-CNN families; the Transformers trade substantial compute for flexible long-range modeling without dominating it, and ViT-B/16 sits below the frontier.

engineered to minimize GMACs, to vision transformers (such as Swin-T and ViT-B/16) that trade higher compute budgets for long-range spatial mixing. Architectural choices made at design time dictate where a system lands along this frontier.

6.2 Lighthouse Roster: Model Biographies

Five canonical workloads isolate the recurring hardware bottlenecks analyzed across this book: compute throughput (ResNet-50), memory bandwidth (GPT-2), memory capacity (DLRM), edge latency (MobileNetV2), and always-on power envelopes (keyword spotting (KWS)).



Lighthouse 6.1: Canonical workloads

In computer architecture, the microprocessor without interlocked pipelined stages (MIPS) processor is often used to teach pipelining, not because it is the fastest available chip, but because it is the clearest embodiment of reduced instruction set computer (RISC) principles. Similarly, this book uses ResNet-50, GPT-2, DLRM, MobileNetV2, and KWS as **canonical workloads**. Studying these "lighthouses" isolates engineering principles about math, memory movement, and locality that remain valid even as specific model architectures evolve.

ResNet-50 (He et al. 2016a) anchors the compute-intensive vision lighthouse. The Residual Network (ResNet) addressed the degradation problem in deep plain networks: adding layers could increase training error despite sufficient capacity. By introducing skip connections, shortcut paths that add a block's input directly to its output and so give gradients a route around each block (section 6.10.2.2 develops the mechanism), it enabled networks of 50, 100, or even 1000 layers. The ResNet architecture won the ImageNet 2015 competition (with 152-layer models), and ResNet-50 has become a widely used backbone and benchmark workload for computer vision. From a systems perspective, it is a highly regular, compute-intensive workload composed almost entirely of dense convolutions, making it a useful test for GPU floating-point throughput.

GPT-2 (Radford et al. 2019) anchors the bandwidth-bound language lighthouse. Generative Pre-trained Transformer 2 demonstrated that scaling up a simple decoder-only transformer architecture on massive datasets could produce coherent text generation. Unlike BERT, an encoder-style transformer that reads context in both directions, GPT-2 generates text sequentially (autoregressively[2]), creating substantial memory bandwidth pressure during batch-one and small-batch decoding because model weights are read for each decoding step. It serves as the archetype for large language models such as Llama and ChatGPT.

DLRM (Naumov et al. 2019) anchors the sparse-recommendation lighthouse. Meta open-sourced DLRM to expose a workload that differs from CNNs and transformers in a critical way. While vision and language models are compute-heavy, recommendation systems are memory-heavy. They must look up user and item preferences in massive embedding tables that can reach terabytes in size, creating unique challenges for latency-critical serving. DLRM is a useful benchmark for memory capacity and sparse memory access patterns in the data center.

MobileNet (Howard et al. 2017) anchors the edge-efficiency lighthouse. MobileNet challenged the trend of ever-larger models by prioritizing efficiency. It popularized depthwise separable convolutions (a per-channel spatial filter followed by a 1×1 channel mix, detailed in section 6.5.5) for efficient vision models, an architectural innovation that reduced FLOPs by 8–9$\times$ for 3×3 kernels with minimal accuracy loss. That smaller compute footprint made MobileNet a natural fit for compression and lower-precision deployment techniques (storing and computing with fewer bits per value). It became a reference family for co-designing vision models with the battery and latency constraints of smartphones and embedded devices.

KWS (Warden 2018) anchors the always-on TinyML lighthouse. Keyword spotting models (like those detecting "Hey Siri" or "Ok Google") represent the extreme end of efficiency. Designed to run on "always-on" microcontrollers with kilobyte-scale memory and milliwatt power budgets, these models (often depthwise separable CNNs) exemplify the constraints of TinyML (Warden and Situnayake 2020; C. R. Banbury et al. 2021; C. Banbury et al. 2021). They force engineers to count every byte and cycle, motivating extreme quantization (INT8 and INT4) and specialized hardware. Together, these biographies establish why the lighthouses are not a model catalog: each one isolates a different bottleneck signature that the arithmetic-intensity analysis can quantify.

6.3 Workload Signatures: The Arithmetic Intensity Spectrum

ResNet-50 reuses convolutional weights across many spatial positions, while batch-one GPT-2/Llama decode streams large weight and KV-cache state (the stored attention keys and values of earlier tokens) for one token at a time. That contrast motivates arithmetic intensity, the FLOP/byte ratio used with a hardware roofline to assess whether a workload is likely to be compute or memory bound.

These bottlenecks reflect the underlying math, but the calculation here is a weight-only proxy rather than a complete measurement of main-memory traffic. It divides floating-point work by FP32 model-weight bytes, omitting activation, intermediate, and KV-cache traffic. Batching can amortize weight traffic across examples or tokens, while additional state movement can lower realized intensity. Section C.2.1 gives the per-operation FLOP and parameter formulas used in the proxy. Table 6.2 compares the three lighthouse scenarios and exposes a roughly 160.2× gap under these assumptions.

Table 6.2: Lighthouse Workload Signatures: Weight-only FP32 intensity proxies for three model scenarios. These values suggest different compute and bandwidth pressures but do not determine a fixed hardware affinity; realized bottlenecks depend on execution regime and hardware balance.

Model Family	Lighthouse	Intensity (I)	Hardware Affinity
Dense CNN	ResNet-50	~80.1 FLOP/byte	Compute-Rich (GPUs/TPUs)
Efficient Vision	MobileNetV2	~42.8 FLOP/byte	Balanced (Mobile NPUs)
Transformer	GPT-2 (Inf)	~0.50 FLOP/byte	Bandwidth-rich accelerator

This table provides one systems input to architecture selection. Task requirements determine which model families are viable, while batch size, precision, sequence length, implementation,

[2] **Autoregressive generation**: A decoding strategy where each output token is conditioned on all previously generated tokens, requiring a full model forward pass per token; for a 1.5B-parameter model in FP16, the weight-only batch-one model used here reads about 3 GB of weights per decoding step and yields a work-to-byte ratio of about 1 FLOP/byte. This low arithmetic intensity often makes batch-one and small-batch decoding bandwidth bound; larger batches can amortize weight traffic. During generation, serving systems also retain the attention keys and values computed for earlier tokens, the key-value (KV) cache that section 6.8.4.2 sizes.

↪ The share of DLRM serving latency spent on embedding lookups is developed in section 13.4.2.

↪ Compression and lower-precision deployment techniques for compact models such as MobileNet are developed in chapter 10.

↩ Arithmetic intensity and the system balance it is compared against are established in section 2.4.

cache behavior, and target hardware determine whether the proxy predicts the deployed bottleneck. MobileNet often suits constrained vision deployments, and batch-one transformer decoding often places heavy pressure on memory bandwidth, but both choices require measurement in the intended regime.

The intensity proxy highlights compute and bandwidth limits for the three feedforward and sequential workloads, but deployment constraints extend beyond the FLOP-to-byte ratio. Table 6.3 broadens this comparison across all five canonical workloads, detailing parameter footprint, inference cost, and governing hardware constraints: compute throughput, memory bandwidth, memory capacity, latency, and power. For compute- and bandwidth-bound workloads, the iron law directly maps the primary bottleneck to operations (O) or data movement (D_{vol}). For capacity-, latency-, and power-constrained workloads, physical boundaries dictate where and within what budget those operations must execute, determining which optimization strategies prove effective.

Table 6.3: Lighthouse Model Comparison: Quantitative characteristics and analytical roles of the five canonical workloads. Memory is FP32 weight storage, and FLOPs are per inference except for GPT-2 XL, which is quoted per generated token; the DLRM row reports embedding-table entries rather than dense parameters. The bottleneck column indicates the primary system constraint each model reveals: compute-bound models like ResNet stress arithmetic throughput, while bandwidth-bound models like GPT-2 stress memory transfer rates.

Model	Domain	Params	FLOPs/Inf	Memory	Bottleneck	Role in Textbook
ResNet-50	Vision	25.6M	8.2 GFLOP	102.4 MB	Compute	Dense vision throughput
GPT-2 XL	Language	1.5B	3 GFLOP/token	6 GB	Mem. Bandwidth	Token-by-token serving
DLRM	Recommender	25B	Low	100 GB	Mem. Capacity	Embedding tables and capacity planning
MobileNetV2	Edge Vision	3.5M	600 MFLOP	14 MB	Latency	Depthwise convolutions and efficiency
KWS (DS-CNN)	Audio	200K	20 MFLOP	800 KB	Power	Always-on power budget

Architecture selection is ultimately an engineering trade-off between math (O) and memory movement (D_{vol}). The mechanisms behind the signatures in table 6.2 explain why each lighthouse sits where it does on the intensity spectrum. ResNet-50 earns its high intensity because convolutional layers reuse each weight many times across the spatial dimensions of an image (individual deeper ResNet layers reach 100–200+ FLOP/byte). Arithmetic throughput becomes the limit when effective intensity exceeds the target machine's compute-to-bandwidth balance. GPT-2 sits at the opposite extreme during batch-one decoding with streamed weights: each projection loads weights for one token's matrix-vector product, making bandwidth a likely limit. MobileNet lands between the two at the whole-model level, with individual depthwise layers falling lower once activation traffic is included: depthwise separable convolutions reduce total O but move more data relative to that work, which fits mobile hardware well yet often "starves" high-end GPUs optimized for dense math.

> **Checkpoint 6.1: Arithmetic intensity and architecture**
>
> Match the architectural choice to its systems implication:
>
> - ☐ **Weight reuse (CNNs)**: How does reusing the same weights across many inputs change arithmetic intensity?
> - ☐ **Large embedding tables (DLRM)**: How do massive embedding tables change the balance between data movement and computation?
> - ☐ **Autoregressive decoding (GPT-2)**: Why does token-by-token generation change the batching and bandwidth picture?

Comparing this spectrum with the target machine's balance helps determine whether faster arithmetic or faster memory would improve performance. The framework this book uses to quantify these limits on specific hardware is the **roofline model**, which plots achievable throughput against

arithmetic intensity to show where a workload turns from memory bound to compute bound. This chapter needs only the comparison already introduced with system balance, a workload's intensity set against the machine's ratio of peak compute to memory bandwidth. For readers who want the full construction, section D.2.1.1 computes the ridge point of an A100 and shows how the same operation falls on either side of it.

The lighthouse signatures make the next step concrete: inspect each architecture family by the data pattern it targets, the computation it performs, the hardware mapping it induces, and the bottleneck it exposes. This four-part lens ensures that every architecture is evaluated for what it costs to run, not only for what it learns.

6.4 MLPs: Dense Pattern Processing

The MNIST digit classifier of chapter 5 is the purest example of dense processing: each of its 784 pixel inputs connects to every hidden unit, and each hidden unit to every unit in the next layer. The same all-to-all pattern suits tabular inputs, such as the sender reputation, link count, and keyword features a spam filter scores, where no feature has a fixed neighbor. **MLPs**[3] represent the fully connected architecture of that classifier, now examined through the four-part systems lens.

MLPs embody an inductive bias: *they impose no specific spatial or temporal connectivity pattern, allowing any input to relate to any output*. This architectural choice provides flexibility across input relationships, making MLPs versatile but potentially costly compared to alternatives that exploit matching structure. Their representational capacity was established theoretically by the universal approximation theorem (UAT)[4] (Cybenko 1989; Hornik et al. 1989), introduced in chapter 5. This theorem states that a sufficiently large MLP with nonlinear activation functions can approximate any continuous function on a compact domain, given suitable weights and biases. That combination of theoretical universality and dense connectivity is the architectural concept captured by the *multilayer perceptron*.

> Definition 6.2: Multilayer perceptrons
>
> ***Multilayer perceptrons*** are feed-forward neural network architectures that apply fully connected layers in sequence, where every neuron in one layer connects to every neuron in the next, without imposing a specific spatial or temporal connectivity pattern.
>
> 1. **Significance**: The lack of structural prior gives dense layers quadratic parameter scaling in layer width: a single layer mapping 1,024 inputs to 1,024 outputs requires 1,048,576 parameters and about 2.1 MB of weight memory in FP16. A 3×3 convolution mapping 1,024 input channels to 1,024 output channels has about 9.4M weights; convolution's advantage for images comes from spatial weight sharing across positions, not from reducing the channel-mixing matrix itself. This makes MLPs inefficient for high-dimensional structured inputs like images.
> 2. **Distinction**: Unlike convolutional neural networks, which exploit spatial locality to reduce parameter count, MLPs treat all input elements symmetrically, making them the architecture of choice for tabular data where no spatial or sequential structure is present.
> 3. **Common pitfall**: A frequent misconception is that MLPs are too simple to matter for complex tasks. Dense layers carry much of the arithmetic inside the most complex models: every transformer block contains a position-wise feed-forward MLP, and recommendation models such as DLRM run dense MLP towers on their features. The other families add structure around dense layers (weight sharing, recurrence, attention) rather than replacing them.

6.4.1 Learnability gap

The UAT guarantees mathematical existence, yet a fundamental gap separates what an architecture can represent from what gradient descent can learn in practice.

Representation capacity refers to the functions an architecture can express given unlimited resources; the UAT guarantees MLPs have universal representation capacity. This capacity is particularly

↪ The roofline model's analytical treatment, from the memory-bound slope to the ridge point, is developed in section D.2.1.

3 **Perceptron**: Formed from "perception" and the device suffix "-tron" (as in *cyclotron* and *klystron*), coined by Frank Rosenblatt (Rosenblatt 1957) for the atomic unit of neural computation: a weighted sum followed by a nonlinear activation, extending the earlier McCulloch-Pitts neuron. MLPs are composed entirely of these units arranged in fully-connected layers, so the efficiency of this single operation, a multiply-accumulate, determines system throughput. Modern accelerators execute over 10^{14} of these operations per second, making the perceptron the computational primitive that the entire ML hardware ecosystem is optimized around.

4 **Universal approximation theorem (UAT)**: This theorem establishes representational capacity for continuous functions on a compact domain, given suitable nonlinear activations. The systems-level catch is that the existence guarantee supplies neither an efficient training procedure nor a practical width for a particular task. For a generic smooth target with no further structure, the width needed to reach a fixed accuracy can grow exponentially with input dimension (the curse of dimensionality), which is why the guarantee alone says little about a 256×256 image; useful structure in the target is what keeps practical networks small.

5 **Manifold hypothesis**: The assumption that high-dimensional data lies near a low-dimensional surface embedded within the full space; a 256×256 image lives in a 65,536-dimensional space, but "valid cat images" may occupy a much smaller structured region. Learned representations can expose that structure. The systems consequence is a potential reduction in the data and computation needed for learning; useful structure can also take other forms, such as a target depending on only a few input features.

effective because of the *manifold hypothesis*,[5] which suggests that high-dimensional data occupies a much simpler structure. *Learnability* refers to whether gradient descent can find good weights given finite training samples and computational budgets. A function may be representable yet practically unlearnable.

This distinction resolves the apparent paradox of universal approximation and architectural progress. Specialized architectures such as CNNs and their deep ResNet descendants improve learnability by embedding inductive biases that match data structure, even when doing so restricts representational capacity.

Three factors create the *learnability gap*:

1. **Sample complexity**: The UAT provides no bounds on training examples needed. For 28×28 images, an MLP receives 784 pixel values without built-in local connectivity or spatial weight sharing. It can learn spatial correlations, as the digit classifier of chapter 5 shows, but must learn relationships that a CNN encodes structurally. When those priors match the task, a CNN can reach a target accuracy with fewer examples. The sample requirement depends on the target function, data distribution, and learning procedure.
2. **Parameter efficiency**: The UAT guarantees some width suffices, but provides no constructive bounds. Some function classes require widths that grow rapidly with input dimension, while architectures with a matching compositional structure can represent them more compactly.
3. **Optimization difficulty**: Even when optimal weights exist, gradient descent may not find them. MLP loss surfaces exhibit complex topology without the regularizing effect of architectural constraints. Specialized architectures reduce the search space, introducing symmetries that gradient descent exploits. The classic MNIST handwritten digit benchmark illustrates this gap between representation and learnability concretely.

Example 6.1: MNIST: Representation vs. learnability

Scenario: Classifying 28×28 MNIST digits with a fully connected MLP vs. a CNN.

Diagnosis: Whole-network parameter counts depend mostly on width choices. The 784-128-64-10 MLP has 109,184 weights, fewer than the 421,408 of a small two-convolution CNN, whose dense classifier head holds 96 percent of them. The locality saving appears per feature detector. A dense unit learns one weight for every pixel, so 32 features cost 25,088 weights; a convolutional layer learns one 3×3 filter per feature, 288 weights in total, and reuses each filter at all 784 positions. At equal feature count, the convolutional layer needs 87× fewer weights.

Systems lesson: Inductive bias buys parameter efficiency per feature and reuse per weight. Fewer weights per detector reduce the data needed to learn spatial structure, and each weight fetched from memory feeds hundreds of multiply-accumulates, which raises arithmetic intensity.

The learnability gap motivates the core design principle of this chapter: embed inductive biases that match data structure. Each architecture sacrifices theoretical generality for practical learnability. The No Free Lunch theorem[6] (Wolpert and Macready 1997) formalizes this trade-off: the bias that helps one task may hurt another. Translation invariance introduced by pooling or global aggregation can aid image classification but impairs tasks where absolute spatial coordinates matter. Architecture selection is fundamentally the act of matching inductive bias to data structure.

These theoretical insights translate directly into engineering decisions. Appropriate inductive biases reduce parameter counts (enabling edge deployment), accelerate convergence (reducing training costs), and produce structured computation patterns that map efficiently to specialized hardware. The MNIST comparison above earned its per-detector saving by matching architecture to data structure, not by shrinking the network. Understanding where dense connectivity remains necessary requires identifying the data patterns that lack structural shortcuts.

6.4.2 Pattern processing needs

Dense pattern processing targets problem domains where input features lack geometric coordinates or sequential ordering. In tabular datasets, recommendation embeddings, or multi-factor risk

[6] **No free lunch theorem**: Wolpert and Macready's 1997 result proved that no optimization algorithm outperforms random search across all possible problems: averaged over every conceivable function, all algorithms are equivalent. The ML systems consequence is that an inductive bias (locality, equivariance, attention) can improve performance on problems matching that bias while hurting performance when its assumptions do not hold, making architecture selection an engineering commitment to a problem class.

models, any input feature may interact with any other. Because the data representation carries no grid or sequence structure, the network cannot restrict computation to local spatial windows or sequential steps. Every output neuron must be free to combine every input feature, leaving optimization to determine connection weights.

When treating an image dataset like MNIST with a dense architecture, the system flattens the 28×28 image into an unstructured 784-element vector. Discarding the two-dimensional spatial prior forces the network to treat all pixel pairs symmetrically. While this enables the layer to relate any pixel to any class, it requires the model to discover spatial adjacency entirely through optimization. That choice leads directly to the matrix operations defining the MLP.

6.4.3 Algorithmic structure

All-to-all feature interaction requires complete connectivity between adjacent layer representations. In an MLP, this manifests as a sequence of fully connected layers where every neuron connects to every neuron in the preceding layer. Dense connectivity translates directly into matrix multiplication, the formulation that makes dense layers executable on parallel hardware. Figure 6.2 diagrams this structural lowering: the fully connected graph on the left maps directly to the matrix-vector products on the right, where each hidden neuron's activation computes a dot product against an entire column of the weight matrix.

↩ The matrix multiplication formulation of a dense layer's forward pass is established in section 5.5.2.2.

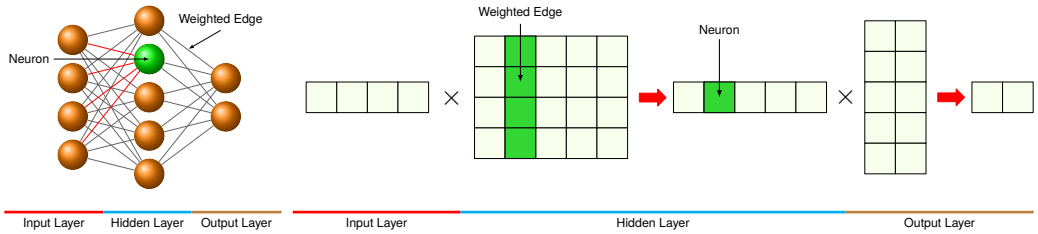


Figure 6.2: Multilayer Perceptron Architecture: A three-layer network in which every neuron connects to all neurons in adjacent layers. The highlighted neuron receives weighted contributions from all inputs, illustrating the dense $\mathcal{O}(d_{\text{in}} \times d_{\text{out}})$ connectivity pattern implemented through matrix multiplications. For MNIST classification, a 784-dimensional input ($d_{\text{in}} = 784$) connects to 100 hidden neurons ($d_{\text{out}} = 100$) through a 784×100 weight matrix, requiring 78,400 multiply-accumulate operations per sample. Adapted from (Reagen et al. 2017).

Equation 6.1 expresses the dense layer's affine transformation followed by its activation.

$$\mathbf{h}^{(\ell)} = f(\mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \tag{6.1}$$

$\mathbf{h}^{(\ell)}$ represents the layer ℓ output, $\mathbf{h}^{(\ell-1)}$ represents the input from the previous layer, $\mathbf{W}^{(\ell)}$ denotes the weight matrix for layer ℓ, $\mathbf{b}^{(\ell)}$ denotes the bias vector, and $f(\cdot)$ denotes the activation function. This layer-wise transformation, while conceptually simple, creates computational patterns whose efficiency depends critically on how the runtime organizes these operations for different problem structures.

The dimensions of these operations reveal the computational scale of dense pattern processing. The input vector $\mathbf{h}^{(0)} \in \mathbb{R}^{d_{\text{in}}}$ (treated as a row vector in this formulation) represents all potential input features. Weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\text{in}}\times d_{\text{out}}}$ capture all possible input-output relationships. The output vector $\mathbf{h}^{(\ell)} \in \mathbb{R}^{d_{\text{out}}}$ produces transformed representations. A four-pixel example turns this bookkeeping into arithmetic.

> Example 6.2: Concrete computation example
>
> Consider a simplified 4-pixel image processed by a 3-neuron hidden layer:
> **Input**: $\mathbf{h}^{(0)} = [0.8, 0.2, 0.9, 0.1]$ (4 pixel intensities)

Weight matrix:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & 0.1 & -0.2 \\ -0.3 & 0.8 & 0.4 \\ 0.2 & -0.4 & 0.6 \\ 0.7 & 0.3 & -0.1 \end{bmatrix} \quad (4\times3 \text{ matrix})$$

Computation:

$$\mathbf{z}^{(1)T} = (\mathbf{h}^{(0)}\mathbf{W}^{(1)})^T = \begin{bmatrix} 0.5\times0.8 + (-0.3)\times0.2 + 0.2\times0.9 + 0.7\times0.1 \\ 0.1\times0.8 + 0.8\times0.2 + (-0.4)\times0.9 + 0.3\times0.1 \\ (-0.2)\times0.8 + 0.4\times0.2 + 0.6\times0.9 + (-0.1)\times0.1 \end{bmatrix} = \begin{bmatrix} 0.59 \\ -0.09 \\ 0.45 \end{bmatrix}$$

After ReLU: $\mathbf{h}^{(1)} = [0.59, 0, 0.45]$ (negative values zeroed)

Systems insight: Each hidden neuron combines all input pixels with different weights, demonstrating unrestricted feature interaction. Dense layers buy generality by paying the maximum connectivity cost.

The MNIST example makes this scale concrete. The 784-dimensional input connects to every neuron in the first hidden layer. A hidden layer with 100 neurons, a round-number stand-in for the 128-unit first layer of the 784-128-64-10 network, requires a 784×100 weight matrix (78,400 parameters), where each weight represents a learnable relationship between an input pixel and a hidden feature. The round width keeps the arithmetic easy to follow, and this 784×100 layer is the reference MNIST layer for the computational analysis in the rest of this chapter.

The matrix form specifies every input-output interaction, but it leaves the execution schedule open. Evaluating one sample at a time streams weights with little reuse; grouping samples lets several outputs share each fetched weight. The loop structure makes these alternatives visible.

6.4.4 Computational mapping

Section 6.4.3 defines *what* an MLP computes; computational mapping reveals *how* that computation translates to hardware operations. Listing 6.1 demonstrates how this mapping progresses from mathematical abstraction to computational reality.

Listing 6.1: MLP Layer in Matrix Form: Framework-level matrix operations hide $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ multiply-accumulate complexity behind a single function call, enabling hardware-optimized BLAS libraries to use dense matrix hardware efficiently.

```
def mlp_layer_matrix(X, W, b):
    """MLP forward pass using framework-level matrix operations."""
    # X: input matrix (batch_size by num_inputs)
    # W: weight matrix (num_inputs by num_outputs)
    # b: bias vector (num_outputs)

    # Single GEMM call: frameworks dispatch to optimized BLAS/cuBLAS
    # For MNIST: 784 * 100 = 78,400 MACs per sample
    H = activation(matmul(X, W) + b)
    return H
```

The function `mlp_layer_matrix` mirrors equation 6.1 directly, using high-level matrix primitives to express the forward transformation while delegating tile scheduling and execution to framework backends. Beneath this software interface, dense matrix multiplication lowers to a canonical three-deep loop nest that determines memory access locality, parallelization boundaries, and register allocation.

The implementation in listing 6.2 exposes this hardware execution pattern directly. Computing layer outputs requires accumulating weighted contributions across all inputs for each sample in the batch. The outermost loop iterates across batch samples, the middle loop iterates over output neurons, and the innermost loop executes repeated multiply-accumulate (MAC) operations[7] pairing each input feature with its corresponding parameter.

[7] **MAC (multiply-accumulate)**: The atomic operation of neural networks: multiply two values and add to a running sum. In Horowitz's 45 nm reference, an FP32 multiply plus add costs about 4.6 pJ, while a 32-bit off-chip DRAM access costs about 640 pJ (Horowitz 2014). These technology-specific values illustrate why data movement can dominate arithmetic energy; they are not universal constants for current hardware.

Listing 6.2: Dense Layer Computation: Nested loops reveal $\mathcal{O}(B \times d_{\text{out}} \times d_{\text{in}})$ complexity where each output neuron requires num_inputs multiply-accumulate operations, so the reference MNIST input layer requires 78,400 MACs per sample.

```
def mlp_layer_compute(X, W, b):
    """Explicit loop structure exposing MLP computational patterns."""
    # Loop 1: Process each sample independently (parallelizable)
    for batch in range(batch_size):
        # Loop 2: Compute each output neuron
        for out in range(num_outputs):
            Z[batch, out] = b[out]  # Initialize with bias

            # Loop 3: Accumulate weighted inputs (innermost loop)
            # This is the MAC operation: result += input * weight
            for in_ in range(num_inputs):
                Z[batch, out] += X[batch, in_] * W[in_, out]
            # Total per output: num_inputs MACs +
            # num_inputs memory reads

    H = activation(Z)  # Element-wise nonlinearity
    return H
```

In the reference MNIST layer, computing 100 hidden neurons requires 78,400 MACs in total; each individual output neuron requires 784 multiply-accumulate operations and 1,568 logical operand reads (784 for inputs, 784 for weights). Registers and caches can serve repeated reads, so this count is not a DRAM-traffic estimate. Production implementations call optimized matrix libraries such as Basic Linear Algebra Subprograms (BLAS),[8] but the same nested-loop pattern still determines the system design problem. The hardware architectures that accelerate these matrix operations, including GPU Tensor Cores[9] and specialized AI accelerators, are covered in section 11.3.2.2.

6.4.5 System implications

Dense pattern processing concentrates hardware pressure across three dimensions: memory capacity for parameter storage, compute throughput on matrix engines, and memory bandwidth for weight movement.

All three constraints originate from all-to-all connectivity. Memory footprint is dominated by parameter storage. While the reference MNIST layer (784×100) requires only 78,400 parameters, $\mathcal{O}(d_{\text{in}} \times d_{\text{out}})$ scaling becomes prohibitive for wide hidden states. A layer connecting 2048 inputs to 2048 outputs requires 4,194,304 parameters (16.8 MB at FP32). Single-sample inference provides one use per weight within that layer evaluation. When weights exceed cache capacity and stream from DRAM for each request, this limited reuse makes bandwidth a likely constraint.

The core computation is dense GEMV, or GEMM when batched. This computation is regular and parallelizable, but the arithmetic intensity (FLOP/byte) is low for small batches. A larger batch size amortizes weight-loading cost over more computations. Modern processors optimize dense layers with single instruction, multiple data (SIMD) units such as AVX-512 on CPUs or with dense matrix engines such as Tensor Cores on GPUs and systolic arrays on Tensor Processing Units (TPUs), amortizing control overhead over large blocks of parallel arithmetic.

The streamed-weight model makes this data-movement cost explicit. To compute 100 hidden values from 784 inputs, the system reads 784×100 weights. Applying the arithmetic intensity framework from section 6.3 to this layer yields roughly 0.5 FLOP/byte for batch-one FP32 execution. Under a weight-only model that counts one FP32 read per weight and omits input, output, bias, and cache traffic, equation 6.2 gives this ratio, where d_{in} and d_{out} are the input and output widths. The memory tier serving those reads determines the relevant bandwidth.

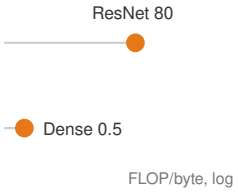
$$I \approx \frac{2 \cdot d_{\text{in}} \cdot d_{\text{out}} \text{ FLOPs}}{4 \cdot d_{\text{in}} \cdot d_{\text{out}} \text{ bytes}} = 0.5 \text{ FLOP/byte} \tag{6.2}$$

If these weights stream from off-chip memory, a machine with an off-chip ridge point in the hundreds of FLOP/byte cannot supply them fast enough to reach peak arithmetic throughput. The small MNIST layer may instead remain cached across requests; its traffic must then be compared

[8] **BLAS (basic linear algebra subprograms)**: This standard API for matrix operations enables the use of highly optimized libraries (for example, cuBLAS) to accelerate the 784 multiply-accumulates per neuron. The 784×100 matrix in the MNIST example may use the hardware less efficiently than larger, well-aligned transformer matrices because utilization depends on matrix shape, batching, datatype, library, and accelerator.

[9] **Tensor Cores**: Specialized units in NVIDIA GPUs that accelerate the thousands of multiply-accumulate operations described by fusing them into single, highly parallelized matrix instructions. Tensor Cores are most efficient when matrix dimensions meet datatype- and architecture-specific alignment multiples; modern cuBLAS/cuDNN can still use Tensor Cores for many nonaligned cases, often with lower efficiency or internal padding. The architectural lesson is vendor-independent: specialized matrix units reward dense, aligned general matrix multiplication (GEMM) workloads and penalize small, irregular shapes that cannot keep the units full.

↪ Dedicated matrix units that process a whole matrix tile per instruction, the mechanism behind Tensor Cores, are developed in section 11.3.2.2.



A batch-one dense layer sits far below convolution's weight reuse.

with the bandwidth of that cache, and dispatch overhead may matter more than off-chip bandwidth. Batching can raise intensity by amortizing weight traffic, but the crossover depends on matrix shape, precision, implementation, and hardware. Specifically, increasing batch size to B scales arithmetic to $2 \cdot B \cdot d_{\text{in}} \cdot d_{\text{out}}$ FLOPs while loading the weight matrix once ($4 \cdot d_{\text{in}} \cdot d_{\text{out}}$ bytes), driving weight-bound arithmetic intensity up to $I \approx 0.5B$ FLOP/byte until input and activation traffic begin to dominate. This is why fully connected layers can become inference bottlenecks despite performing fewer total FLOPs than convolutional layers.

Because dense connectivity offers little weight reuse within a single-example forward pass, applying MLPs to high-dimensional structured signals forces an undesirable trade-off between parameter explosion and memory starvation. When data possesses inherent physical organization—such as spatial coordinates in images or sequential order in time series—specialized architectures replace isotropic connectivity with structural priors that simultaneously raise accuracy and computational efficiency. The most established of these spatial architectures is the convolutional neural network.

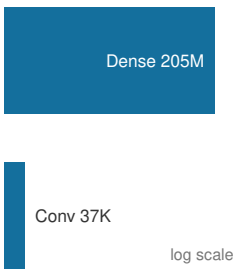
6.5 CNNs: Spatial Pattern Processing

The MLP's assumption that every input may interact directly with every output proves particularly costly for spatially structured data like images. Connecting every pixel of a high-resolution input to every hidden unit creates an unmanageable parameter footprint that exhausts accelerator memory and collapses arithmetic intensity into a memory-bandwidth-bound crawl. The earlier MNIST comparison showed the remedy at small scale: a shared filter needs 87× fewer weights per feature detector than a dense unit that treats every pixel independently.

CNNs[10] structure this computational trade-off directly in silicon (LeCun et al. 1998; Krizhevsky et al. 2012). Rather than materializing an all-to-all connectivity matrix, a CNN restricts each output activation to a local spatial patch and slides identical filter weights across the entire input aperture.

This spatial inductive bias translates into two structural constraints on the computation graph. Parameter sharing applies the same small filter weights across every spatial position, reducing parameter storage from millions of distinct weights to a compact kernel that fits within on-chip caches. Local connectivity limits each neuron's receptive field to a spatially adjacent neighborhood, eliminating computation on distant, uncorrelated pixels. Together, these constraints define the convolutional family.

[10] **Convolution**: From Latin *convolvere* ("to roll together"), describing a filter that slides across an input, combining local elements at each position. This "rolling together" enforces a locality constraint that is the source of the operation's efficiency: a single 5×5 kernel reuses its 25 weights at every spatial position instead of learning a separate weight for every pixel.



Weight sharing spares convolution the fully-connected parameter explosion.

Definition 6.3: Convolutional neural networks

Convolutional neural networks (CNNs) are architectures that exploit translation equivariance and spatial locality to share learned filters across all spatial positions, decoupling parameter count from input resolution.

1. **Significance**: Weight sharing produces dramatic parameter reduction. A 3×3 convolutional layer with 64 input and 64 output channels requires $3 \times 3 \times 64 \times 64 \approx 37{,}000$ parameters regardless of whether the input image is 224×224 or 1024×1024. A fully connected layer mapping a $224\times224\times64$ input to 64 scalar outputs would require $224^2 \times 64 \times 64 \approx 205$ million parameters, a roughly 5,575× difference. These output shapes differ: convolution reuses its 64 detectors across a spatial feature map, while the dense layer produces 64 scalars. Weight sharing keeps the filter budget constant as resolution grows, although feature-map storage still grows with image area.
2. **Distinction**: Unlike MLPs, which connect every input element to every output element (global connectivity), CNNs restrict each output to a local spatial neighborhood, encoding the assumption that nearby pixels are more relevant than distant ones. This restriction eliminates entire hypothesis classes at architecture design time rather than penalizing them during training.
3. **Common pitfall**: A frequent misconception is that CNNs are vision-only models. The convolution operation applies to any data with a grid-like topology: 1D convolutions process audio waveforms and time series, 2D convolutions process images and spectrograms, and 3D convolutions process video and volumetric data.

The trade-off is explicit: CNNs sacrifice the theoretical generality of MLPs for practical efficiency gains when data exhibits known structure. Where MLPs treat each input element independently, CNNs exploit spatial relationships to achieve both computational savings and improved accuracy on vision tasks.

6.5.1 Pattern processing needs

Spatial pattern processing addresses workloads where correlation between data points decays rapidly with geometric distance. In a natural image, a pixel's covariance with adjacent pixels is high, enabling the detection of localized primitives such as edges, textures, and contours. Stacking these operations allows local primitives to compose hierarchically into higher-level representations: edges compose into contours, contours into surface patches, and patches into coherent objects. Figure 6.3 traces this hierarchical pipeline: an input zebra image passes through successive convolution, ReLU, and pooling stages with expanding channel depths, flattens into a 1D activation vector, and terminates in fully connected classifier layers producing class probabilities.

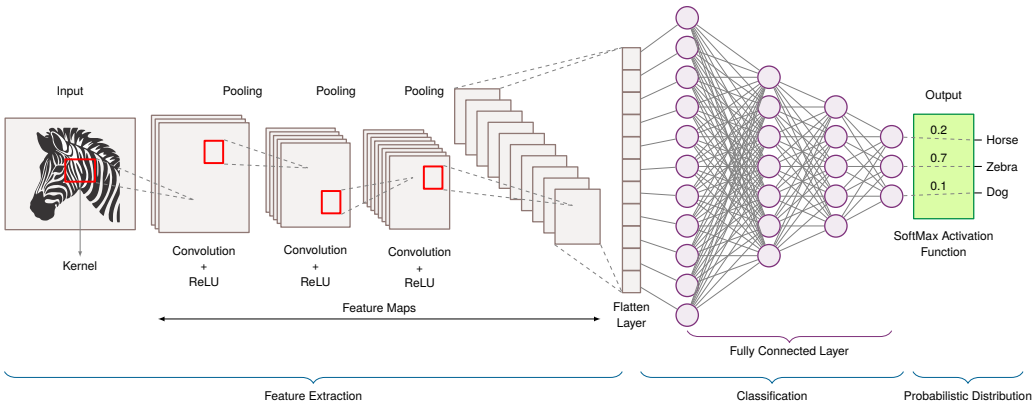


Figure 6.3: Spatial Feature Extraction: A zebra image passes through repeated convolution, ReLU, and pooling stages that form progressively deeper feature maps, followed by flattening, fully connected layers, and a softmax classification over Horse, Zebra, and Dog.

This hierarchical composition appears across multiple physical domains: localized pixel neighborhoods forming contours in computer vision, local time-frequency clusters identifying phonemes in speech spectrograms, and contiguous voxel blocks isolating tissue boundaries in volumetric tomography. The architecture succeeds not because it mimics biological vision, but because it matches the compositional structure of the underlying data.

In image processing, recognizing an object requires two distinct computational guarantees. First, the architecture must capture local geometric primitives over bounded receptive fields. Second, because an object retains its semantic identity regardless of where it appears in the frame, the network must identify those primitives across arbitrary coordinate translations.[11] Convolutional networks satisfy both requirements by combining localized kernels with parameter sharing, enforcing translation equivariance at intermediate stages while pooling enables downstream invariance (LeCun, Boser, et al. 1989; Krizhevsky et al. 2012).[12] These architectural constraints, introduced by LeCun[13] (LeCun, Boser, et al. 1989), replace unconstrained dense matrix multiplications with structured spatial stencils.

6.5.2 Algorithmic structure

The four-dimensional convolution tensor builds up in two physical stages, spatial filtering followed by channel accumulation. Consider a single RGB image with three input channels ($C_{\text{in}} = 3$). A single output feature (such as an edge detector) requires its own dedicated $K \times K$ spatial filter for the Red channel, another for Green, and a third for Blue:

1. **Spatial filtering**: Each 2D filter slice slides across its corresponding color channel, computing multiply-accumulate dot products over local spatial windows as in a classic image filter.

11 **ImageNet**: The dataset that validated these two spatial processing requirements at scale. AlexNet's 2012 victory in the ImageNet Large Scale Visual Recognition Challenge posted 15.3 percent top-5 error against 26.2 percent for the second-place entry, on the 1000-class challenge with roughly 1.3 million training images (Krizhevsky et al. 2012); the original ImageNet release contained 3.2 million images across 5,247 synsets (Deng et al. 2009). Subsequent architectures improved accuracy through changes in architecture, optimization, data, and compute budgets, illustrating the interaction between inductive bias and infrastructure cost.

12 **Translation equivariance**: An inherent property of the convolution operation where shifting the input guarantees a corresponding spatial shift in the resulting feature map. This is distinct from *invariance*, which pooling or later aggregation can approximate by discarding precise positional data. For example, 2×2 pooling with stride 2 reduces the number of spatial elements by 75 percent.

13 **Yann LeCun and LeNet**: LeCun's architecture directly addressed the intractable scaling of applying dense networks to images by enforcing the principles of local connectivity and parameter sharing. These constraints reduced the parameter count for an image-like input layer by over 95 percent, enabling LeNet-5 to achieve production-grade accuracy on commercial tasks like check reading with only ~60,000 total parameters.

2. **Channel accumulation**: At each spatial coordinate (i, j), the dot-product outputs of all three input channels sum into a single scalar value.

This two-step process produces exactly one 2D output feature map. If the layer demands $C_{\text{out}} = 64$ distinct features, the system repeats this procedure 64 times using 64 independent filter volumes. Thus, the weight tensor naturally requires four physical dimensions: $\mathbf{W} \in \mathbb{R}^{K \times K \times C_{\text{in}} \times C_{\text{out}}}$ (Kernel Height $\times$ Kernel Width $\times$ Input Channels $\times$ Output Channels).

In formal terms, equation 6.3 evaluates this forward pass by computing local, channel-wise dot products across every spatial coordinate:

$$H^{(\ell)}_{i,j,k} = \phi\left(\sum_m \sum_n \sum_c W^{(\ell)}_{m,n,c,k} H^{(\ell-1)}_{i+m,j+n,c} + b^{(\ell)}_k\right) \tag{6.3}$$

Here, $H^{(\ell)}_{i,j,k}$ denotes the activation at spatial position (i, j) in output channel k of layer ℓ. The triple summation iterates across the filter dimensions: spatial offsets (m, n) span the local kernel window (defining the layer's local receptive field),[14] while c ranges over all input channels. The entries $W^{(\ell)}_{m,n,c,k}$ of the weight tensor store the kernel weights connecting channel c to channel k, and $b^{(\ell)}_k$ is the per-channel bias. Because the kernel weights $\mathbf{W}^{(\ell)}$ do not depend on the spatial coordinates (i, j), the same filter parameters are reused at every coordinate.

Convolutional layers typically use small spatial kernels (3×3 or 5×5) to maintain compact parameter footprints while stacking layers to widen the effective receptive field. Figure 6.4 diagrams this spatial mapping: blue projection rays connect an orange 3×3 receptive field within the dashed 7×7 input grid to a single target activation in the upper 5×5 output feature map. The interactive CNN Explainer tool (Wang et al. 2021) visualizes how these stacked sliding windows extract per-channel activation patterns across layers.

[14] **Receptive field**: The input region influencing a particular output neuron. With 3×3 filters, receptive fields grow by 2 pixels per layer, so a neuron at layer 3 "sees" a 7×7 region. This growth rate constrains architecture depth: detecting objects spanning 100+ pixels in a 224×224 image requires either deep stacks of small filters (more layers, more memory for activations) or larger kernels (more parameters per layer), a fundamental depth-vs.-width trade-off in CNN design.

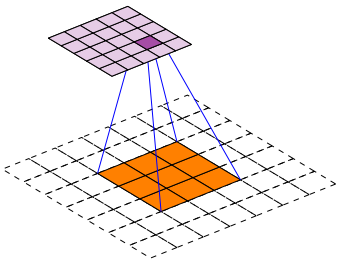


Figure 6.4: Convolution Operation: A 3×3 orange receptive field in a dashed 7×7 input maps to one cell of the violet 5×5 valid-convolution output. Applying a 3×3 filter to that receptive field requires 9 multiply-accumulate operations per input channel for each output value. Sharing the filter across positions encodes translation equivariance: the same edge detector works at any image location.

Applying a CNN to 28×28 MNIST images demonstrates the difference in state representation. Rather than flattening the image into a 784-element vector that destroys spatial adjacency, each convolutional layer applies a bank of filters (for example, 32 filters of size 3×3) directly to the 2D grid. With unit stride and padding to preserve border dimensions, the layer outputs a $28\times28\times32$ activation tensor preserving the spatial grid. Each coordinate holds 32 distinct feature measurements extracted from its immediate neighborhood.

Preserving spatial topology changes how execution units access memory. Instead of loading an unshared weight matrix once per sample as in an MLP, a convolutional layer streams input feature maps through hardware registers while keeping compact filter weights pinned in fast on-chip SRAM or register files. This arithmetic pattern drives the memory-hierarchy optimizations of AI accelerators, where loop tiling (splitting the loops into blocks whose data fits on chip, developed under data movement below) and parallel channel reduction relieve high-bandwidth memory (HBM) traffic.

Weight sharing is what guarantees this reuse. Because the same filter applies at every coordinate, the computational schedule and memory access geometry are identical across spatial positions. The same sharing gives the layer a mathematical property, translation equivariance: translating the input produces an identical spatial translation in the output feature map.

Equivariance and invariance govern how an architecture responds to spatial transformations. An operator f is *equivariant* to a transformation $\mathcal{T}$ if transforming the input transforms the output by the same operator (equation 6.4):

$$f(\mathcal{T}(\mathbf{x})) = \mathcal{T}(f(\mathbf{x})) \tag{6.4}$$

For a stride-1 convolutional layer without boundary padding effects, translating the input by vector $\mathbf{v} = (v_1, v_2)$ shifts the resulting feature maps by exactly $\mathbf{v}$. Spatial coordinate information propagates deterministically through the layer. By contrast, an operator is *invariant* to $\mathcal{T}$ if transforming the input leaves the output unchanged (equation 6.5):

$$f(\mathcal{T}(\mathbf{x})) = f(\mathbf{x}) \tag{6.5}$$

A global average pooling layer over an entire feature map exhibits translation invariance: shifting the input alters the spatial locations of activations, but averaging across all spatial positions collapses coordinates into a scalar that is identical regardless of where the features appeared.

This mathematical distinction dictates where equivariance must be maintained and where invariance should be introduced across the network hierarchy. Tasks such as object detection and semantic segmentation require equivariance through the final layer: bounding-box coordinates and per-pixel masks must track object coordinates precisely. Classification tasks, by contrast, require equivariance in intermediate feature extractors to preserve relative spatial relations ("eyes above nose"), while the final layers introduce translation invariance via pooling to collapse spatial coordinates into a class probability vector.

Re-indexing the convolution sum under a shift confirms that stride-1 convolution is translation equivariant up to boundary conditions. An equivariant convolutional layer preserves coordinate information, whereas an invariant pooling layer discards spatial coordinates entirely. Example 6.3 verifies this tracking through explicit matrix operations.

Example 6.3: Equivariance: Feature detection

Setup: Consider a 7×7 image with a vertical edge at column 3:

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Vertical edge detector filter:

$$\mathbf{W} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Convolving original image:

Output feature map shows positive activation where the filter transitions from dark to bright (left side of edge) and negative activation where it transitions from bright to dark (right side):

$$f(\mathbf{x}) = \begin{bmatrix} 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \end{bmatrix}$$

Shifted input (edge moved to column 5):

$$\mathcal{T}_2\mathbf{x} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Convolving shifted image:

$$f(\mathcal{T}_2\mathbf{x}) = \begin{bmatrix} 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \end{bmatrix} = \mathcal{T}_2(f(\mathbf{x}))$$

Systems insight: The feature activation shifts by the same amount as the input, demonstrating equivariance. The network knows the edge is at column 5 in the shifted image, not just that an edge exists somewhere.

Parameter efficiency is the most immediate systems consequence of the weight sharing that produces translation equivariance, because sharing decouples parameter capacity from the input resolution. For example, processing a 224×224 RGB image with an MLP requires each hidden neuron to connect to all 150,528 input pixels. A CNN with a 3×3 filter requires only 27 parameters per filter, reused across all 224×224 spatial positions. This is the same 5,575.1× per-detector reduction that the inductive-bias definition previewed. A compact parameter footprint lets the weights stay resident in accelerator SRAM or L2 cache rather than streaming from external DRAM on every use.

The computational regularity enforced by weight sharing directly benefits hardware execution. Because the identical sliding window operation executes across every spatial coordinate, the compute pattern maps cleanly onto SIMD vector units and systolic arrays. Modern GPUs and TPUs exploit this uniform structure: execution pipelines issue identical instructions across spatial tiles without control-flow divergence. Overlapping receptive fields mean that neighboring output coordinates share input activations, creating opportunities for shared-memory caching and algorithmic data restructuring (such as the im2col transformation) that saturate hardware execution units.

Equivariance also increases sample efficiency, reducing training pipeline overhead. When a network updates a filter to recognize a feature at one coordinate, parameter sharing ensures that the feature is recognized across all spatial coordinates simultaneously. The model does not require separate training examples to learn the same feature at different pixel offsets, functioning as structural data augmentation. This sample efficiency reduces the total training iterations, dataset storage, and host-to-device I/O bandwidth required to reach target accuracy.

Convolution's translation equivariance is one instance of a general principle: data symmetries can be embedded as equivariances in the architecture. Group-equivariant CNNs (Cohen and Welling 2016) extend the construction to rotations and reflections for satellite and medical imagery, where orientation does not determine meaning. For systems engineering, stronger symmetry constraints often produce smaller models, but specialized equivariances may require custom operations, such as rotation convolutions, that need either hardware support or efficient software implementations.

In practice, perfect equivariance is often compromised by implementation constraints. Asymmetric padding at boundaries introduces edge artifacts that break strict translational symmetry. Strided downsampling introduces spatial aliasing, where a one-pixel shift in the input causes sub-sampling phase shifts in the output feature map. Batch normalization layers also perturb spatial equivariance if activation statistics are computed or applied across spatial locations inconsistently. Modern

production networks accept these deviations because the computational savings of striding and the training stability of normalization outweigh the loss of exact mathematical equivariance.

These structural properties instantiate the spatial inductive bias introduced in section 6.1: by restricting connectivity to local neighborhoods and sharing parameters across all coordinates, CNNs enforce the prior that visual primitives are localized and shift-equivariant. Reusing identical kernel weights across the entire feature map reduces parameter storage by orders of magnitude relative to dense layers, restricting the hypothesis space to functions that preserve geometric structure.

This layered structure naturally implements hierarchical representation learning (Bengio, Courville, et al. 2013). Early layers detect low-level features (edges and color gradients) across small receptive fields, while successive layers compose these primitives into intermediate textures, object parts, and semantic categories. Stacking convolutional layers expands the effective receptive field: with fixed $K \times K$ kernels and unit stride, receptive-field side length grows by $K - 1$ pixels per layer, while receptive-field area scales quadratically until it spans the entire input dimension. How efficiently accelerators execute this expanding hierarchy depends on how sliding-window stencils map onto parallel execution pipelines.

6.5.3 Computational mapping

Translating mathematical convolution into hardware execution requires bridging the gap between high-level software interfaces and silicon execution pipelines. High-level frameworks abstract convolution as a unified layer dispatch, shown in listing 6.3. Behind this concise API, the underlying runtime must schedule memory traffic across cache hierarchies and lower sliding-window stencils onto matrix execution hardware.

Listing 6.3: Convolutional Layer Abstraction: Framework-level convolution operations hide the complexity of sliding window computations, typically dispatching to cuDNN or MKL which internally handle im2col transformations or direct convolution algorithms.

```
def conv_layer_spatial(input, kernel, bias):
    """Framework-level convolution.

    Single call dispatches to optimized kernel (often via im2col + GEMM).
    """
    # Convolution applies shared weights across all positions
    # For a 3x3 kernel on 28x28 input (padded): 9 MACs per position x 784 positions
    output = convolution(input, kernel) + bias
    return activation(output)
```

To expose the mechanics that the framework abstraction conceals, listing 6.4 details the canonical seven-deep loop nest required to evaluate a direct 2D convolution. These loops partition into three structural tiers: batch iteration across samples, spatial traversal over feature map coordinates (y, x), and channel accumulation over input and output dimensions $(C_{\text{in}}, C_{\text{out}})$ coupled with kernel offsets (k_y, k_x). Traversing spatial coordinates and output channels defines the grid of independent output activations. The listing uses unit stride and no padding, with channels stored contiguously at each spatial position (NHWC: batch, height, width, channels). Each output window stays within the input, so the output height and width are the corresponding input dimensions minus the kernel dimensions plus one.

Listing 6.4: Logical Convolution Computation: Seven nested loops expose $\mathcal{O}(B \times H_{\text{img}} \times W_{\text{img}} \times C_{\text{out}} \times K \times K \times C_{\text{in}})$ complexity. The spatial dimensions here count output positions. The scalar order exposes overlapping input reads; tiled implementations exploit this reuse directly or through matrix multiplication.

```
def conv_layer_compute(input, kernel, bias):
    # NHWC input; kernel axes: height, width, input, output channels.
    # Unit stride, no padding; activation follows the convolution.
    batch_size, in_h, in_w, in_channels = input.shape
    kernel_h, kernel_w, _, out_channels = kernel.shape
    out_h, out_w = in_h - kernel_h + 1, in_w - kernel_w + 1
    output = zeros((batch_size, out_h, out_w, out_channels))
    for image in range(batch_size):
        for y in range(out_h):
            for x in range(out_w):
                for out in range(out_channels):
                    result = bias[out]
                    for ky in range(kernel_h):
                        for kx in range(kernel_w):
                            for ch in range(in_channels):
                                result += (input[image, y + ky, x + kx, ch]
                                           * kernel[ky, kx, ch, out])
                    output[image, y, x, out] = result
    return activation(output)
```

The inner three loops execute the kernel dot product at each spatial site. For a 3×3 filter, each output activation accumulates contributions from a local input patch across all input channels. Advancing `x` by one makes the next output read a patch overlapping the previous one, while changing `out` reuses the same patch with a different filter. These repeated accesses offer reuse, but the scalar schedule does not explicitly keep tiles in fast memory or distribute work across execution lanes. Their actual cost depends on which reads the cache serves and how the implementation maps iterations to hardware.

To expose these patches to optimized matrix engines, convolution implementations can use the **im2col** (image-to-column) transformation.[15] By unfolding each receptive-field patch into a contiguous row of an input matrix and stacking filter weights into columns (a layout transformation diagrammed for two input channels in figure 6.5), im2col recasts the entire convolution as a dense general matrix multiply (GEMM) that runs on mature, vendor-tuned libraries. To see how the lowering rearranges data in memory, consider a miniature single-channel image ($C_{\text{in}} = 1$) of size 3×3 convolved with a 2×2 filter ($K = 2$) with valid padding:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad \mathbf{W} = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}$$

A sliding window sweeps across four overlapping 2×2 receptive fields:

- **Top-left window**: $[1, 2, 4, 5]$
- **Top-right window**: $[2, 3, 5, 6]$
- **Bottom-left window**: $[4, 5, 7, 8]$
- **Bottom-right window**: $[5, 6, 8, 9]$

[15] **im2col (image to column)**: Rather than being a new learning algorithm, im2col-style lowering is an implementation technique used by CNN frameworks and libraries such as Caffe and cuDNN (Jia et al. 2014; Chetlur et al. 2014): it converts convolutions into standard GEMM calls by unfolding overlapping patches into matrix columns. The trade-off is memory: in a simple fully materialized stride-1 $K\times K$ transform, interior input elements can appear in up to K^2 columns (9 times for 3×3 filters), though borders, stride, padding, tiling, and direct-convolution algorithms reduce the realized expansion. This memory-for-simplicity exchange explains why mobile frameworks (TFLite, NNAPI) prefer direct convolution, while data center GPUs with abundant HBM may use GEMM-oriented lowering when it improves throughput.

To execute all four sliding-window evaluations simultaneously as a single matrix multiplication, `im2col` extracts each 2×2 patch, flattens it into a row vector of length $K^2 = 4$, and stacks them vertically into an unfolded matrix $\mathbf{X}_{\text{col}}$:

$$\mathbf{X}_{\text{col}} = \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix}$$

Flattening the filter weights into a column vector $\mathbf{w}_{\text{col}} = [w_1, w_2, w_3, w_4]^T$, the entire convolution collapses into a single standard matrix product: $\mathbf{Y}_{\text{col}} = \mathbf{X}_{\text{col}}\mathbf{w}_{\text{col}}$.

Generalizing to batched multi-channel inputs, a tensor of shape $(B, C_{\text{in}}, H_{\text{img}}, W_{\text{img}})$ unpacks into an unfolded matrix of shape $[(B \cdot H_{\text{out}} \cdot W_{\text{out}}) \times (K^2 \cdot C_{\text{in}})]$, while convolutional filters reshape to $[(K^2 \cdot C_{\text{in}}) \times C_{\text{out}}]$. Their product yields an output matrix of shape $[(B \cdot H_{\text{out}} \cdot W_{\text{out}}) \times C_{\text{out}}]$. In $\mathbf{X}_{\text{col}}$, input element 5 appears in all four rows, but the whole miniature input expands from nine elements to sixteen, a storage ratio of $16/9$. For large stride-1 inputs whose output area is close to the input area, fully materialized unfolding approaches the kernel area as its expansion factor. The smaller ratio here reflects the borders of a valid convolution. Materializing the buffer trades this additional storage and traffic for access to wide matrix engines optimized for GEMM throughput; implicit GEMM, described after figure 6.5, avoids allocating that duplicated buffer.

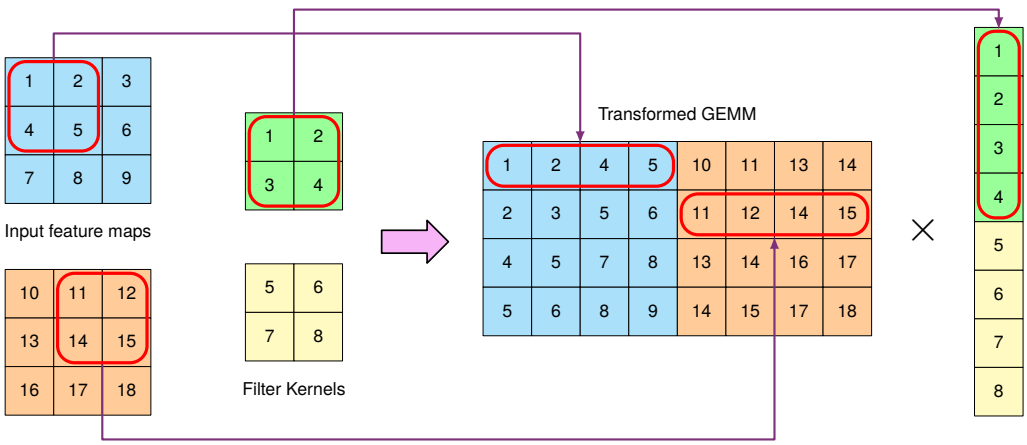


Figure 6.5: im2col Transformation: Converts convolution to GEMM by unfolding image patches into a dense matrix. The two 3×3 input feature maps produce four sliding-window positions, represented here as rows of a 4×8 matrix; the two 2×2 filter kernels stack into an 8×1 column. Their product yields the four output positions. Lowering can exploit optimized matrix kernels but expands memory by an amount that depends on kernel size, stride, padding, and implementation, so its performance benefit is workload dependent.

Implicit GEMM techniques run the same matrix multiplication without building the unfolded matrix. They compute each patch element's address on the fly, inside the processor's registers, so the duplicated buffer is never materialized. With 3×3 filters and 32 output channels, each output coordinate requires only 9 MAC operations per input channel, compared to 784 MACs in the reference MLP layer. While reducing operations per activation, the sliding window introduces distinct data reuse profiles that dictate memory staging.

6.5.4 System implications

Execution mechanics such as sliding windows and im2col lowering dictate how hardware evaluates convolutions, but the true engineering cost emerges across memory footprint, compute throughput, and data movement. These resource demands depend directly on layer depth, activation liveness, and whether feature maps remain cached in SRAM or spill to external HBM.

6.5.4.1 Memory requirements

Memory consumption in convolutional layers divides into two distinct state components: static filter parameters and transient feature maps (activations). Weight sharing ensures that parameter storage remains compact: for an ImageNet input, a convolutional layer with 64 filters of size 3×3 applied to a single input channel requires only 576 parameters. Even across 64 input channels, storing 36,864 weights requires roughly 147 KB in FP32, fitting comfortably within on-chip L2 or L1 cache. In contrast, intermediate feature maps scale with input resolution and channel depth: a 224×224 feature map with 64 channels materializes 3.2M activation values (approximately 12.8 MB in FP32 per sample in the batch). During training, these activations must remain live in memory for backward-pass gradient evaluation, making activation memory, not weight capacity, the binding memory constraint for deep CNNs on accelerators.

Accelerators exploit this footprint disparity through hierarchical caching. CPUs retain compact filter weights in L2/L3 caches while streaming activation tensors. GPUs stage filter weights and input tiles in on-chip scratchpad SRAM (shared memory) and register files, reusing loaded weights across thousands of spatial coordinates and eliminating redundant round-trips to off-chip HBM.

↪ The memory hierarchy of register files, on-chip SRAM, and HBM that these caching patterns exploit is developed in section 11.5.4.

6.5.4.2 Computation needs

Evaluating a convolutional layer requires repeatedly evaluating local dot products across spatial positions. For an ImageNet input with 3×3 filters and 64 output channels, computing a single spatial position requires 576 multiply-accumulate operations per input channel. Repeating this evaluation across all 50,176 spatial positions yields millions of operations per layer. Although each spatial stencil touches fewer parameters than a dense layer, spatial repetition multiplies the total floating-point operation count.

This computational regularity provides rich opportunities for hardware parallelization. Because each spatial coordinate is independent, processors parallelize work along multiple axes simultaneously: across the batch dimension, across spatial coordinates (x, y), and across output channels. CPUs execute vectorized SIMD instructions[16] to evaluate multiple spatial outputs per clock cycle, while GPUs dispatch thread warps across thousands of concurrent execution lanes.

[16] **SIMD (single instruction, multiple data)**: CPU instructions that apply the same operation to multiple data elements simultaneously; AVX-512 can process 16 single-precision values per vector instruction; realized speedup over scalar code also depends on instruction throughput, vectorization, and memory access. For CNN inference on edge CPUs without GPU access, SIMD utilization can affect whether a model meets real-time latency targets. Frameworks like TFLite and Open Neural Network Exchange (ONNX) Runtime use vectorized convolution kernels to exploit this parallelism.

6.5.4.3 Data movement

Data movement in a convolutional layer exhibits an arithmetic intensity profile fundamentally different from that of an MLP. In a fully connected layer at batch size one, each loaded weight is used exactly once, yielding an arithmetic intensity bounded below 1 FLOP/byte. In a 2D convolution, each 3×3 filter weight is reused across all 50,176 positions of the 224×224 feature map. This massive weight reuse increases operational intensity, shifting computation from the memory-bandwidth-bound regime toward the compute-bound ceiling.

The systems challenge shifts from loading weights to staging activations. Because adjacent spatial receptive fields overlap, a naive memory access pattern would re-fetch each input pixel from memory up to $K^2 = 9$ times. Runtimes avoid this redundant memory traffic by tiling computation into 2D spatial blocks sized to fit inside accelerator SRAM. Once an input tile is loaded into shared memory, all overlapping filter dot products for that tile execute in place, amortizing the DRAM/HBM transfer and maximizing the 50,176 reuses of each filter weight.

↪ The hardware ridge point (I_{ridge}), which decides whether spatial filter reuse saturates the arithmetic units or starves on memory bandwidth, is developed in section 11.5.2.

Checkpoint 6.2: Spatial inductive bias

CNNs succeed because they match the structure of image data. Verify you understand how:

- ☐ **Parameter sharing**: how does using the same filter across the image reduce parameter count by orders of magnitude compared to MLPs?
- ☐ **Translation equivariance**: why does shifting the input image result in a corresponding shift in the feature map?
- ☐ **Compute-bound convolution**: how does filter reuse raise arithmetic intensity, and how would comparing that intensity with the target machine's roofline balance establish whether the layer is compute bound?

These memory, compute, and data-movement patterns converge in one model that serves as the chapter's reference point for compute-bound vision workloads: the ResNet-50 architecture.

Lighthouse 6.2: ResNet-50 (vision lighthouse)

Why it matters: ResNet-50 is a reference point for regular, convolution-heavy vision workloads. Its architecture consists almost entirely of dense convolutional layers, making it highly regular and efficient on GPUs. Under batched execution with good data reuse, ResNet-50 performance is typically limited by floating-point throughput (FLOP/s), making it a useful lighthouse for explaining data parallelism, quantization, and batching strategies. Table 6.4 summarizes the quantitative properties and their system consequences:

Table 6.4: ResNet-50 Systems Profile: Quantitative properties of the vision lighthouse and their system consequences. Under batched convolutional reuse, ResNet-50 has high effective arithmetic intensity and is typically compute-bound, so peak FP throughput on Tensor Cores often sets the ceiling before memory bandwidth does.

Property	Value	System Implication
Parameters	25.6M	102.4 MB model size at FP32; fits comfortably in GPU memory.
FLOPs/Image	8.2 GFLOP (224×224)	3×3 convolutions are the largest single kernel class at roughly 48% of MACs.
Constraint	Compute-heavy when batched	Limited by peak FLOP/s when weight and activation reuse are high; small-batch inference can move toward the memory-bound regime.
Bottleneck	FP Throughput	Benefits maximally from specialized Matrix Units (Tensor Cores).
Profile	High effective arithmetic intensity under reuse	Arithmetic intensity depends on batch size, convolution algorithm, and materialized memory traffic.

6.5.5 Efficient architectures: Depthwise separable convolutions

ResNet-50's compute-bound profile assumes abundant hardware resources, yet most inference runs on devices with power budgets three orders of magnitude smaller than a data center accelerator. Standard convolutions scale as $\mathcal{O}(N_{\text{pix}} \times K^2 \times C_{\text{in}} \times C_{\text{out}})$, a compute volume that exceeds the strict energy budgets of always-on edge devices. To bridge this gap, efficient architectures like **depthwise separable CNNs (DS-CNN)** decompose the standard convolution into two distinct operations. This factorization, introduced by Sifre in the context of feature extraction (Sifre and Mallat 2014) and popularized by MobileNet (Howard et al. 2017), decouples spatial filtering from cross-channel combination. The depthwise convolution applies filters to each input channel independently ($K \times K \times C_{\text{in}}$ parameters), while the pointwise convolution uses a 1×1 convolution to project channels to the output dimension ($1 \times 1 \times C_{\text{in}} \times C_{\text{out}}$ parameters). Per output position, a standard layer performs $K^2 C_{\text{in}} C_{\text{out}}$ multiply-accumulates, while the factorized pair performs $K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$. Dividing the two shows that the decomposition reduces parameter count and FLOPs to a fraction of roughly $\frac{1}{C_{\text{out}}} + \frac{1}{K^2}$ of standard convolution (a reduction factor approaching K^2, or about 8–9× for 3×3 filters with large C_{out}), making real-time execution feasible on resource-constrained microcontrollers.

MobileNetV2 applies this factorization to mobile vision, achieving competitive accuracy with a fraction of ResNet-50's computational cost.

Lighthouse 6.3: MobileNetV2 (efficiency lighthouse)

Why it matters: MobileNetV2 represents latency-constrained edge workloads. Its depthwise separable convolutions trade channel-mixing capacity for fewer operations, making it a useful baseline for mobile apps, embedded vision, and neural architecture search (NAS), automated search over model designs. Table 6.5 summarizes the efficiency lighthouse's quantitative properties:

Table 6.5: MobileNetV2 Systems Profile: Quantitative properties of the efficiency lighthouse and their system consequences. Roughly an order of magnitude smaller than ResNet-50 in both parameters and FLOPs, MobileNetV2's operation shapes and arithmetic intensity make single-image latency sensitive to operator-dispatch overhead and memory access on mobile CPUs.

Property	Value	System Implication
Parameters	3.5M	14 MB at FP32; 7.3× smaller than ResNet-50.
FLOPs/Image	600 MFLOP	13.7× fewer than ResNet-50 for similar accuracy.
Constraint	Latency Bound	Single-image inference speed is the priority.
Bottleneck	Overhead/Memory Access	Operator dispatch and memory access can dominate actual compute.
Profile	Low Arithmetic Intensity	Memory access and control logic matter more than peak FLOP/s.

The ResNet-50 and MobileNetV2 profiles expose a critical systems trade-off: a model with 13.7× fewer FLOPs does not automatically run 13.7× faster. Whether operational reduction translates into wall-clock speedup depends on arithmetic intensity, operator dispatch overhead, and the hardware's roofline balance. On data center GPUs engineered for high arithmetic intensity, MobileNetV2 often achieves only a modest speedup because its fragmented depthwise operations underutilize Tensor Cores.

Architectural efficiency techniques like depthwise separable convolutions and pruning (removing redundant weights or channels) alter this hardware mapping balance. Perspective 6.1 addresses the central fallacy linking FLOP counts directly to execution latency.



Systems Perspective 6.1: Misconception: FLOPs equal speed

Fallacy: "MobileNetV2 has 13.7× fewer FLOPs than ResNet-50, so it must run 13.7× faster."

Resolution: On some high-end GPUs, MobileNetV2 can run slower than ResNet-50 despite using far fewer operations. MobileNetV2's depthwise separable convolutions have low arithmetic intensity: they move more data relative to computation. GPUs optimized for dense matrix operations may use their compute units poorly on these kernels. FLOPs measure *work*; throughput depends on how well that work *maps to hardware*. This hardware-architecture mismatch recurs as a general fallacy in section 6.11.

Keyword spotting pushes the same factorization to the microcontroller end of the spectrum. A continuous 1D audio stream is first converted by a short-time Fourier transform into a 2D time-frequency spectrogram (or Mel-frequency cepstral coefficients), so 2D convolutional filters can slide across time and frequency bins to detect acoustic phoneme signatures.



Lighthouse 6.4: KWS (TinyML lighthouse)

Why it matters: Keyword spotting models (like DS-CNN) represent the power-constrained end of the spectrum. Used in always-on applications like smart doorbells that wake a larger vision pipeline only after an audio trigger, these models must run on microcontrollers with milliwatt power budgets.

At 200K parameters, a DS-CNN keyword spotter's 800 KB of FP32 weights already exceeds the few hundred kilobytes of memory a typical microcontroller can devote to a model, so the model fits only after 8-bit quantization cuts it fourfold. KWS is therefore the lighthouse for extreme quantization (INT8/INT4) and specialized architectural primitives (depthwise separable convolutions) that trade theoretical representational power for maximum efficiency per watt.

From ResNet-50's compute-heavy standard convolutions through MobileNet's depthwise separable variants to KWS's micro-power deployment, CNNs demonstrate how architectural constraints

transform data structure into hardware efficiency. Local connectivity and weight sharing reduce parameter storage and expose reuse. The KWS example also shows that convolution can process temporal patterns. Its context is bounded by the network's receptive field, however. A streaming system that must carry information across variable time spans within a fixed state budget motivates a different trade-off: updating a compact recurrent state as each observation arrives.

6.6 RNNs: Sequential Pattern Processing

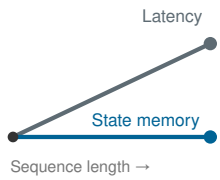
Convolutional networks exploit *spatial* structure, where physical proximity governs statistical correlation. Many real-world signals exhibit *temporal* structure, where sequential order and historical context determine meaning. Recurrent networks process these signals by maintaining persistent state across time steps.

Definition 6.4: Recurrent neural networks

Recurrent neural networks (RNNs) are sequence-processing architectures that maintain a hidden state $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ updated at each time step, encoding the assumption that the current output depends on all prior inputs through this fixed-size state vector.

1. **Significance**: The fixed-size state provides $\mathcal{O}(1)$ inference state memory regardless of sequence length (processing a 10,000-token sequence requires the same state memory as a 10-token sequence), but the sequential update rule forces all S recurrent transitions to execute in order. Separate per-step kernels (compiled accelerator routines, a different sense of the word from a convolution kernel) add launch overhead (L_{lat}); fusion can reduce those launches while preserving the serial dependency chain.
2. **Distinction**: Unlike MLPs, which require a fixed-dimensional input vector, or CNNs, which restrict context to a fixed spatial window, RNNs compress an unbounded history into a fixed-size bottleneck state, so they accept sequences of any length without widening the input layer. Gradient signals pass back through the recurrent steps, where repeated Jacobian products can attenuate or amplify them (derived below).
3. **Common pitfall**: A frequent misconception is that RNNs are obsolete. For streaming inference on resource-constrained hardware, such as keyword spotting on a microcontroller, a model that must keep every past input to see the whole stream needs memory that grows with stream length, which can be prohibitive. An RNN's $\mathcal{O}(1)$ state size makes it a candidate when a fixed state budget is required.

Feedforward architectures and standard CNNs treat inputs independently or constrain context to a fixed spatial receptive field. In sequential domains, however, dependencies span variable and often unbounded temporal distances. Expanding a convolutional receptive field to cover long sequences requires either stacking dozens of layers or applying dilated kernels, both of which scale parameter count and intermediate activation storage regardless of whether the intervening tokens contain relevant information.



RNNs hold state memory fixed, but latency grows with sequence length.

Recurrent architectures, from simple recurrent networks to gated variants such as long short-term memory (LSTM) networks (Elman 1990; Hochreiter and Schmidhuber 1997), address this constraint by introducing a temporal inductive bias: execution order encodes causal direction, and history compresses into a recurring state. Instead of processing each input vector in isolation, the network carries an internal state vector across successive time steps. This inductive bias establishes an explicit engineering trade-off: it provides variable-length temporal context within a fixed memory budget, but it introduces loop-carried data dependencies that serialize execution on parallel accelerator hardware.

6.6.1 Pattern processing needs

In sequential pattern processing, the interpretation of an incoming token depends on the preceding sequence. For example, polysemous words such as "bank" cannot be disambiguated in isolation: "river bank" specifies a geological shoreline, whereas "investment bank" specifies a financial institution. The word's identity alone is insufficient; its semantic role is resolved by preceding context.

Acoustic waveforms and telemetry time series exhibit identical properties: individual samples are ambiguous without historical context spanning multiple frequencies and timescales.

Data with this structure imposes three concrete requirements on the underlying model:

1. **Variable-length ingestion**: Sequences arrive with non-uniform lengths S, precluding fixed-dimension weight tensors that require static input shapes.
2. **Contextual persistence**: The model must retain informative historical signals over arbitrary time lags without requiring explicit windowing.
3. **Continuous state update**: Internal representations must update dynamically as each new token arrives, integrating new evidence while discarding obsolete or noisy history.

A fixed-input MLP requires windowing, padding, or truncation to process variable-length sequences. CNNs see a bounded window per layer; covering long histories requires stacking layers (receptive field grows linearly with depth) or dilated kernels (exponentially), at a cost in activation storage. Recurrence addresses the streaming requirements above by making a persistent, fixed-size state part of its dataflow.

6.6.2 Algorithmic structure

To meet the requirements of section 6.6.1, recurrent neural networks introduce directed cycles in their computational graph. Rather than mapping inputs directly to outputs through feedforward layers, an RNN maintains a persistent vector, the **hidden state**, that updates iteratively at each time step. The hidden state acts as an internal register, summarizing past observations to contextualize future predictions.

The fundamental state transition in equation 6.6 computes the current hidden state $\mathbf{h}_t$ by combining the incoming token vector $\mathbf{x}_t$ with the preceding state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \phi(\mathbf{W}_{\text{hh}}\mathbf{h}_{t-1} + \mathbf{W}_{\text{hx}}\mathbf{x}_t + \mathbf{b}_{\text{h}}) \tag{6.6}$$

where $\mathbf{h}_t$ denotes the hidden state at time t, $\mathbf{x}_t$ denotes the input at time t, $\mathbf{W}_{\text{hh}}$ contains the recurrent weights, $\mathbf{W}_{\text{hx}}$ contains the input weights, $\mathbf{b}_{\text{h}}$ is the hidden-state bias vector, and ϕ is the activation function. The equation uses column vectors; the later listings use row-major batched tensors, so their weight multiplications appear on the right. Figure 6.6 contrasts the compact recurrent loop (left) with its computational unrolling across consecutive time steps (right), sequencing the transitions $\mathbf{h}_{t-1} \to \mathbf{h}_t \to \mathbf{h}_{t+1}$ and isolating the strict temporal serialization enforced by the shared recurrent weight matrix $\mathbf{W}_{\text{hh}}$.

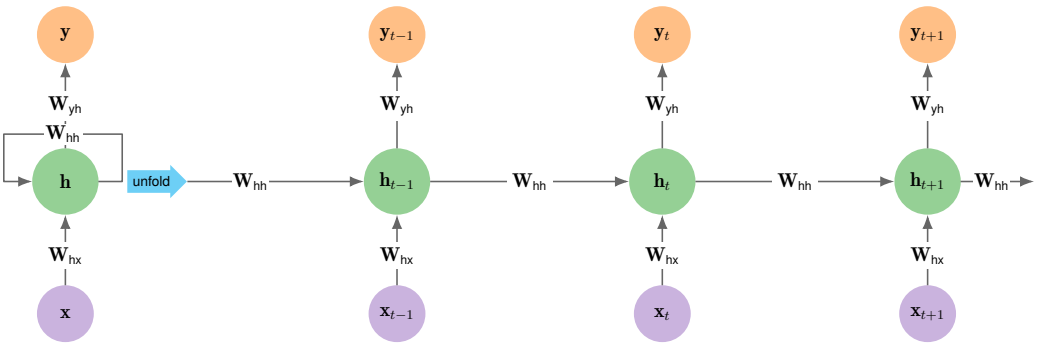


Figure 6.6: Recurrent Neural Network Unfolding: Left panel shows the compact recurrent loop; right panel unfolds this across time steps. Three weight matrices are shared across all steps: $\mathbf{W}_{\text{hx}}$ (input-to-hidden), $\mathbf{W}_{\text{hh}}$ (hidden-to-hidden), and $\mathbf{W}_{\text{yh}}$ (hidden-to-output). These matrices are reused at every step, so the parameter count is independent of sequence length. For a 128-dimensional hidden state and a 100-dimensional input, each time step requires 16,384 MACs for recurrent connections plus 12,800 for input projection.

Unfolding the recurrent loop across S steps reveals the algorithmic properties of recurrence. Parameter count remains independent of sequence length S because the weight matrices $\mathbf{W}_{\text{hh}}$ and $\mathbf{W}_{\text{hx}}$ are shared across all time steps. However, sharing weights across time establishes a loop-carried dependency: step t cannot evaluate $\mathbf{h}_t$ until step $t-1$ finishes producing $\mathbf{h}_{t-1}$. This dependency

forms a critical path of length $\mathcal{O}(S)$ serial operations that prevents hardware accelerators from parallelizing across the sequence dimension.

This sequential structure produces fundamentally different memory characteristics between inference and training. During streaming inference, an RNN evaluates one step at a time, discarding prior raw inputs and retaining only the current hidden state. State memory overhead remains strictly $\mathcal{O}(d)$ in the hidden dimension d, regardless of whether the sequence spans 10 or 10,000 steps. In contrast, training via backpropagation through time (BPTT) unrolls the graph across the entire sequence of length S. Computing gradients via the chain rule requires evaluating the Jacobian product across all intermediate transitions:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_S} \prod_{t=1}^{S} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

Ordinary BPTT retains the intermediate activations $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_S$ for these Jacobian products. This retained-state scheme converts an $\mathcal{O}(d)$ runtime state into an $\mathcal{O}(S \cdot d)$ training footprint; recomputing saved intermediates trades additional work for memory. Repeated Jacobians combine activation derivatives with $\mathbf{W}_{\text{hh}}$, and their products can attenuate or amplify gradients over long horizons, limiting the effective context window of simple RNNs. Gated variants such as the LSTM answer this with learned gates, elementwise valves that decide at each step how much of the stored state to keep, overwrite, or expose; a kept state passes to the next step without being multiplied by $\mathbf{W}_{\text{hh}}$, which gives gradients a path that does not shrink at every step. Section 6.10.2.4 examines the gating mechanism and its price, roughly four affine transformations per step instead of one. The recurrent weight matrix often contains connections with minimal contribution to temporal dependencies, allowing significant compression through methods covered in chapter 10.

6.6.3 Computational mapping

Translating recurrent equations into hardware execution reveals how mathematical abstractions map to accelerator kernels, memory hierarchies, and loop nests. Where convolutional layers exploit spatial locality and weight reuse across tiles (section 6.5.4.3), recurrent steps decompose into matrix-vector or matrix-matrix multiplications bound by dataflow serialization.

Listing 6.5 demonstrates the single-time-step mechanism using framework-level matrix operations: combine the previous hidden state with the current input, add the bias, and apply the activation to produce the next hidden state. The code is intentionally local to one step because the systems cost is not the step itself, but the dependency chain that prevents parallel execution across time.

Listing 6.5: RNN Layer Abstraction: Framework-level implementation combining two matrix multiplications, $\mathbf{h}_{t-1}\mathbf{W}_{\text{hh}}$ and $\mathbf{x}_t\mathbf{W}_{\text{hx}}$, per time step. For 128-dimensional hidden state and 100-dimensional input, each step requires 16,384 + 12,800 = 29,184 MACs, but sequential dependencies prevent parallelization across time.

```
def rnn_layer_step(x_t, h_prev, W_hh, W_hx, b):
    # x_t: input at time t (batch_size × input_dim)
    # h_prev: previous hidden state (batch_size × hidden_dim)
    # W_hh: recurrent weights (hidden_dim × hidden_dim)
    # W_hx: input weights (input_dim × hidden_dim)
    h_t = activation(matmul(h_prev, W_hh) + matmul(x_t, W_hx) + b)
    return h_t
```

At the framework level, `rnn_layer_step` hides the underlying loop structure and memory traffic behind high-level matrix multiplications (`matmul`). In hardware, evaluating this step requires moving weights and activations through the memory hierarchy. To see where execution serializes and where parallelism remains, listing 6.6 expands the matrix operations into explicit loop nests down to scalar multiply-accumulate operations.

Listing 6.6: RNN Layer Computation: Nested loops expose the sequential dependency structure. Loop 1 enables batch parallelism, but Loops 2-3 must complete before Loop 4's activation; because each time step depends on the previous step's output, the resulting $\mathcal{O}(S)$ sequential depth prevents GPU parallelization across the time dimension.

```
def rnn_layer_compute(x_t, h_prev, W_hh, W_hx, b):
    # Initialize next hidden state
    h_t = np.zeros_like(h_prev)

    # Loop 1: Process each sequence in the batch
    for batch in range(batch_size):
        # Loop 2: Compute recurrent contribution (h_prev × W_hh)
        for i in range(hidden_dim):
            for j in range(hidden_dim):
                h_t[batch, i] += h_prev[batch, j] * W_hh[j, i]

        # Loop 3: Compute input contribution (x_t × W_hx)
        for i in range(hidden_dim):
            for j in range(input_dim):
                h_t[batch, i] += x_t[batch, j] * W_hx[j, i]

        # Loop 4: Add bias and apply activation
        for i in range(hidden_dim):
            h_t[batch, i] = activation(h_t[batch, i] + b[i])

    return h_t
```

The loop structure exposes the concurrency boundaries of recurrent execution. Loop 1 demonstrates that batch instances are completely independent: sequences within a batch parallelize across accelerator cores without cross-talk. Within each batch instance, Loops 2 and 3 execute the recurrent transformation ($\mathbf{h}_{t-1}\mathbf{W}_{\text{hh}}$) and input projection ($\mathbf{x}_t\mathbf{W}_{\text{hx}}$). Both reductions must accumulate fully before Loop 4 can apply the elementwise bias addition and activation function to produce $\mathbf{h}_t$.

For an input dimension of 100 and a hidden state dimension of 128, each step executes two matrix multiplications: a 100×128 input projection and a 128×128 recurrent transformation. When batch size is large ($B \gg 1$), these operations execute as general matrix-matrix multiplications (GEMM), achieving high arithmetic intensity as weights loaded from memory are reused across all B samples. When batch size is small ($B = 1$, typical of real-time streaming inference), the operations collapse into matrix-vector multiplications (GEMV). In GEMV, every weight parameter must be fetched from memory to perform a single multiply-accumulate, dropping arithmetic intensity to ≈ 1 FLOP/byte in FP16 (0.5 in FP32) and starving accelerator compute units.

6.6.4 System implications

The hard physical constraint that recurrent architectures impose is sequential dependency. In the update $\mathbf{h}_t = \tanh(\mathbf{W}_{\text{hh}}\mathbf{h}_{t-1} + \mathbf{W}_{\text{hx}}\mathbf{x}_t)$, the recurrent transformation at time step t needs the state produced by step $t-1$. An input sequence of length $S = 1{,}000$ therefore requires $1{,}000$ ordered state updates, even when a fused kernel executes several or all updates within one launch. Hardware can parallelize across batch instances and within each matrix operation; input projections can also be computed across time when the inputs are available. Faster arithmetic or memory shortens individual steps, while the recurrent dependency chain remains.

During inference, recurrent networks achieve exceptional memory efficiency. Because the recurrent state compresses history into a fixed-size vector $\mathbf{h}_t$, inference state memory scales as $\mathcal{O}(d)$ (for example, 2 KB for a 512-dimensional FP32 state), remaining completely flat whether sequence length S is 10 or 10,000 tokens. This constant memory footprint contrasts sharply with attention mechanisms (section 6.7), whose stored context grows with every token because generating each new token reuses the stored keys and values of every earlier token, an $\mathcal{O}(S \cdot d)$ store called the key-value cache. For streaming deployments on memory-constrained edge hardware, such as microcontrollers, an RNN's $\mathcal{O}(1)$ runtime state eliminates the memory-capacity exhaustion that limits transformers.

Recurrent execution exhibits distinct memory-hierarchy behaviors: high temporal locality for parameters, but poor operational intensity under standard framework dispatch. Weight matrices $\mathbf{W}_{\text{hh}}$ and $\mathbf{W}_{\text{hx}}$ are reused at every time step, residing comfortably in SRAM or L2 cache if the

model fits on-chip. However, naively launching separate kernels for input projection, recurrent projection, bias addition, and activation at each time step incurs severe kernel-launch overhead and forces intermediate tensors back and forth across HBM. To mitigate this memory-bandwidth bottleneck, production recurrent runtimes fuse the projections, bias addition, and activation into a single monolithic kernel that maintains $\mathbf{h}_t$ within GPU registers and shared memory throughout the sequence execution.

This architectural tension between fixed-capacity state and serialized execution defined sequence processing before transformers. While recurrent networks maintain constant memory overhead regardless of sequence length, the sequential dependency prevents hardware from parallelizing across time steps, the fixed-capacity state becomes an information bottleneck in which early inputs fade over long spans, and gradients through long chains of recurrent steps vanish or explode (the vanishing gradient problem). Overcoming these constraints required an architecture capable of establishing direct, content-addressable links across arbitrary sequence positions without intervening recurrent steps: the attention mechanism (section 6.7).

↪ Kernel fusion, which keeps intermediate tensors on-chip instead of round-tripping them through device memory, is developed in section 7.3.1.

6.7 Attention: Dynamic Processing

Recurrent networks serialize execution along the sequence dimension and compress all historical context into a fixed-capacity hidden state vector (section 6.6.4). In a sentence such as "The cat, which was sitting by the window overlooking the garden, was sleeping," the subject "cat" and predicate "sleeping" are separated by ten intervening words, yet they form the core syntactic and semantic relationship. An RNN must process all intervening tokens sequentially, risking information loss across intermediate recurrent transitions while preventing hardware from parallelizing across time steps. This constraint motivates an alternative computational pattern: an operation that directly computes relevance between any two positions regardless of distance.

Attention mechanisms[17] address this limitation (Bahdanau et al. 2015) by introducing dynamic connectivity patterns that adapt to input content. The setting was sequence-to-sequence translation, built as an encoder-decoder pair. An encoder network reads the input sentence into one representation per input position, and a decoder network generates the output sentence one token at a time, carrying its own recurrent decoder state. Recurrent versions of that pair passed only the encoder's final state to the decoder. Rather than forcing all context through that single fixed-capacity vector, encoder-decoder attention computes similarity scores between the decoder state at each output step and all encoder positions, routing information dynamically across the sequence. A single decoder step compares one query with every encoder position. In full-sequence self-attention, each position queries all positions in the same sequence, so the number of score interactions grows quadratically with sequence length.

17 **Bahdanau attention**: This approach broke the "fixed-length vector" bottleneck of prior sequence-to-sequence models by allowing a decoder to dynamically query all input elements at each output step, creating the adaptive connectivity described. This replaced the structural constraint of a fixed-capacity channel with a learned, content-based weighting system. The core trade-off was accepting a linear, $\mathcal{O}(S)$ memory cost to store all input states in exchange for overcoming the information loss inherent in a single vector.

> Definition 6.5: Attention mechanisms
>
> ***Attention mechanisms*** are neural network operations that compute a weighted sum of value vectors, where the weights are derived from learned similarity scores between a query vector and a set of key vectors, enabling dynamic, content-dependent information routing between any two positions in a sequence.
>
> 1. **Significance**: Attention connects any two tokens in $\mathcal{O}(1)$ depth but computes S^2 score interactions. If those scores are materialized, a 4,096-token sequence with 16-bit scores consumes 33.6 MB per layer per head (about 16.8M scores at 2 bytes each), directly increasing the iron law's D_{vol} term.
> 2. **Distinction**: Unlike RNNs, which compress all prior context into a single fixed-size state vector, attention mechanisms retain token representations and compute relevance scores directly. During training or full-sequence prefill, the initial pass that processes the whole prompt, score interactions grow quadratically with sequence length; during autoregressive serving, the stored **key-value (KV) cache**, the saved key and value vectors of prior tokens, grows as $\mathcal{O}(Sd)$. Generation applies a causal mask, so each token attends only to itself and earlier positions. Keys and values are kept because each new token's

query must attend over every earlier key and value; earlier queries are discarded because, under that mask, past tokens never attend forward.

3. **Common pitfall**: A frequent misconception is that attention is a general-purpose weighting scheme that can be applied freely. Quadratic score computation is a hard scaling constraint: doubling the context quadruples score interactions. Naive implementations also quadruple score storage; tiled exact algorithms such as FlashAttention avoid the full matrix, while sparse variants reduce the scores computed.

While attention mechanisms were initially used as components within recurrent architectures, their ability to connect any position to any other made the recurrent structure unnecessary for many sequence tasks. The transformer[18] (Vaswani et al. 2017) removed recurrence entirely, trading the RNN's $\mathcal{O}(S)$ sequential path for constant path length between positions within one attention layer and thereby enabling parallelization across sequence positions on high-throughput accelerators.

Attention functions as a computational primitive rather than an independent architecture family. The transformer (section 6.8), the family built around it, inherits its core systems properties directly from attention: content-dependent routing, parallel sequence execution, and quadratic score scaling. Evaluating those properties begins with the dynamic pattern-processing problem that attention addresses.

[18] **Transformer**: The founding paper, "Attention Is All You Need," made the explicit systems claim that a parallel attention mechanism could fully replace sequential recurrent processing. This architectural trade eliminates an RNN's $\mathcal{O}(S)$ path length constraint on parallelism but introduces $\mathcal{O}(S^2)$ dense score computation, a quadratic cost that continues to shape context-window engineering.

6.7.1 Pattern processing needs

Dynamic pattern processing addresses scenarios where relationships between elements are not fixed by architecture but instead emerge from content. Language translation exemplifies this challenge: when translating "the bank by the river," understanding "bank" requires attending to "river," but in "the bank approved the loan," the important relationship is with "approved" and "loan." Unlike RNNs that process information sequentially or CNNs that use fixed spatial stencils, dynamic sequence processing requires an operation that evaluates relationships based on token identity. Figure 6.7 diagrams this dynamic routing for coreference resolution in a sample sentence, isolating how attention weights bridge non-adjacent words.

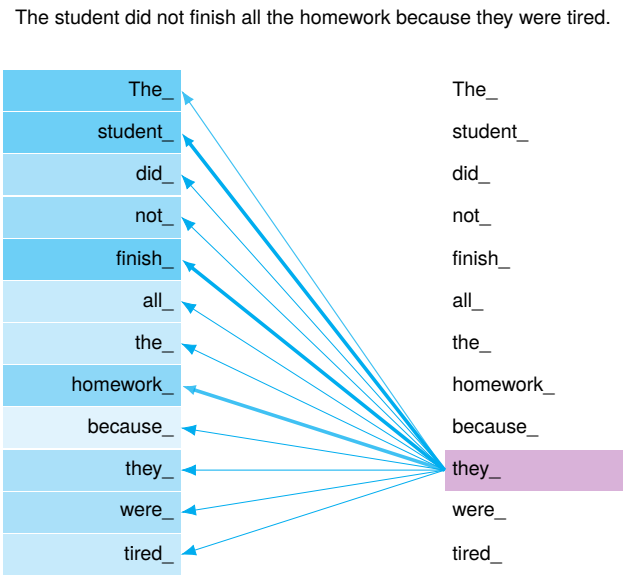


Figure 6.7: Illustrative Attention Pattern: A schematic attention pattern for the pronoun "they." Line thickness represents hypothetical attention-weight magnitude, with stronger links to "student," "finish," and "homework." One attention layer connects all positions directly and can compute their dense interactions in parallel.

The bipartite connection lines in figure 6.7 illustrate how the pronoun "they" could receive information from "student" and "finish." The hypothetical weights demonstrate a direct routing

path, not a measured coreference result. Pairwise connections can be evaluated in parallel, allowing a single layer to connect these positions without propagating intermediate state across intervening tokens.

Input-dependent processing extends well beyond language. In protein structure prediction, amino-acid interactions depend on chemical properties and spatial arrangement, not only chain position; in document analysis, connections between sections depend on semantic content rather than proximity. Graph convolutional networks (GCNs) (Kipf and Welling 2017) share attention's input-dependent routing but gather features along the sparse edges of each input graph, producing irregular memory access that can bind execution on memory bandwidth and cache misses rather than compute throughput.

Input dependence does not imply all-to-all connectivity. Dense attention scores every pair of sequence elements, whereas a GCN restricts aggregation to edges in the observed graph. The common systems challenge is that the computation or memory-access pattern responds to the input rather than following a fixed stencil. Even when tensor dimensions stay fixed, content-dependent scores determine which information is emphasized. Attention provides one mechanism for learning such relationships, and dense self-attention forms the foundation of the transformer architecture.

6.7.2 Algorithmic structure

Meeting the dynamic routing requirements of section 6.7.1 requires a formal operator that maps variable-length sequences to content-dependent affinity weights (Bahdanau et al. 2015). The transformer architecture implements this mapping via the scaled dot-product attention operator introduced by Vaswani et al. (2017):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{head}}}}\right)\mathbf{V}$$

For this self-attention case, the inputs $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values)[19] are learned linear projections of the token representations ($\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$). Attention uses three separate projection matrices rather than computing dot-product affinities directly between raw tokens ($\mathbf{X}\mathbf{X}^T$) for two architectural reasons:

1. **Asymmetric relationships**: Direct token dot products $\mathbf{x}_i \cdot \mathbf{x}_j$ are strictly commutative ($\mathbf{x}_i \cdot \mathbf{x}_j = \mathbf{x}_j \cdot \mathbf{x}_i$). However, linguistic and contextual relationships are fundamentally asymmetric. In "the chef cooked the meal," read whole by an encoder that lets every token see every other, the verb "cooked" seeks its direct object ("meal"), while "meal" plays a receptive role. (A generating decoder adds a causal mask that blocks attention to later positions, so in generation only "meal" can look back at "cooked"; either way, the two directions of a token pair need different scores.) Decoupling into query (what information a token seeks) and key (what features a token advertises) allows $\mathbf{q}_{\text{cooked}} \cdot \mathbf{k}_{\text{meal}}$ to be high while $\mathbf{q}_{\text{meal}} \cdot \mathbf{k}_{\text{cooked}}$ remains low.
2. **Decoupling routing from content**: The dot products $\mathbf{Q}\mathbf{K}^T$ determine *where* to look (routing coefficients), while the value projection $\mathbf{V}$ determines *what* information to retrieve. Without a separate value projection, a token acting as a strong structural signpost (such as a preposition or punctuation mark) would force its entire raw embedding into downstream layers. Projecting through $\mathbf{W}_V$ allows the network to strip away irrelevant lexical identity and extract only task-relevant features.

For an input sequence of length S projected to a dimension d_{head} (here the full width d; the transformer's multi-head attention later splits that width across several parallel attention computations called heads), the dot product $\mathbf{Q}\mathbf{K}^T$ computes all S^2 pairwise token similarities. Scaling by $1/\sqrt{d_{\text{head}}}$ prevents dot products from growing large in magnitude at high dimensions, which would push softmax into regions with vanishingly small gradients. The softmax operator[20] normalizes each row of the score matrix into a probability distribution over key positions. Finally, multiplying these normalized weights by $\mathbf{V}$ produces output vectors formed as weighted combinations of the value representations.

Figure 6.8 traces this dataflow for a sequence of length $S = 6$: cyan query rows match against red key columns to populate the 6×6 similarity matrix, whose rows subsequently weight the green

[19] **Query-key-value (QKV)**: The terminology is borrowed from information retrieval, explaining why the equation uses three distinct learned projections to calculate pairwise scores. Creating these projections requires three independent weight matrices, costing $3 \times d^2$ parameters per layer. The direct systems consequence is the "KV cache" for autoregressive inference: all prior key and value vectors must be stored for the next token, causing memory to grow linearly ($\mathcal{O}(S)$) with sequence length and potentially dominate serving memory at long contexts or high concurrency.

[20] **Softmax**: Named as a "soft" (differentiable) version of argmax, with mathematical roots in Boltzmann's statistical mechanics, softmax normalizes each query row over all S scores and therefore requires a reduction over each row. Online softmax lets that reduction run over tiles with running statistics, which is why it need not force quadratic storage (section 6.7.4.1).

↪ The dot product as a similarity measure is developed in section C.1.3.

value vectors to produce the contextualized output vectors on the right. Each row of the resulting $S \times S$ matrix determines how strongly an individual token attends to every token in the sequence. In a trained model, the largest affinity values reflect strong contextual dependencies, while near-zero entries suppress irrelevant positions.

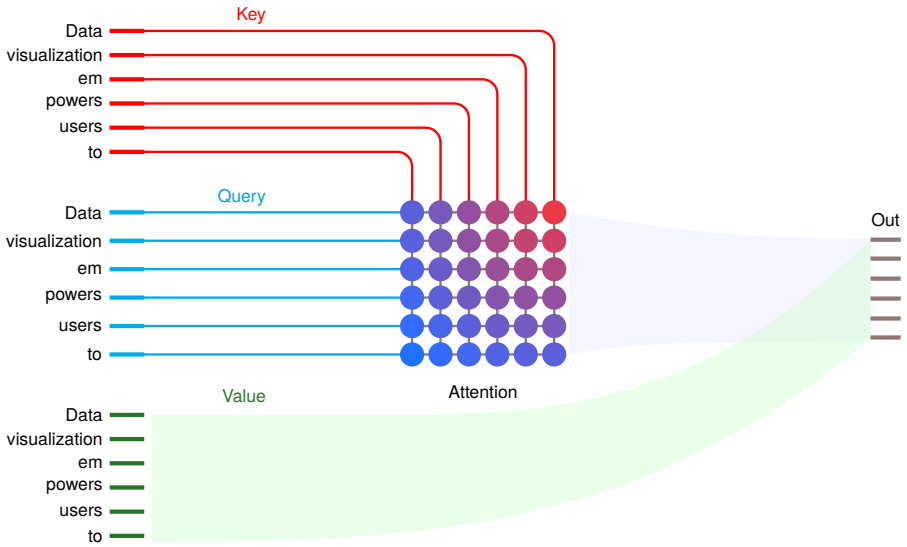


Figure 6.8: Query-Key-Value Attention Mechanism: For a 6-token sequence, queries (cyan) match against keys (red) to produce a 6×6 attention matrix with $\mathcal{O}(S^2)$ entries. Cell color runs from blue to red across the grid as an illustrative gradient rather than as computed weights. Each output position aggregates information from all values (green) weighted by its own row of that matrix. The matrix structure reveals both the computational pattern (thirty-six similarity computations) and the naive memory bottleneck (storing S^2 attention weights). Source: Transformer Explainer (Cho et al. 2025).

While attention weights vary dynamically with the input tokens, generating the query, key, and value vectors requires static linear projections against learned parameter matrices. In accelerator implementations, these three operations are fused into a single batched matrix multiplication to maximize Tensor Core saturation and eliminate kernel launch overhead. Figure 6.9 diagrams this fused projection: multiplying the 6×768 token embedding matrix by the packed 768×2304 weight matrix and adding the bias vector produces a concatenated 6×2304 output tensor in a single kernel pass.

6.7.3 Computational mapping

Lowering the attention operator $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{QK}^T / \sqrt{d_{\text{head}}})\mathbf{V}$ to physical execution exposes the system cost of data-dependent connectivity. Evaluating pairwise alignments requires two distinct matrix multiplications per attention head: the score GEMM ($\mathbf{QK}^T$) and the context GEMM ($\mathbf{AV}$, where $\mathbf{A} = \text{softmax}(\mathbf{QK}^T / \sqrt{d_{\text{head}}})$). For an input of sequence length S and head dimension d_{head}, each phase executes $S^2 d_{\text{head}}$ multiply-accumulate operations, yielding $\mathcal{O}(S^2 d_{\text{head}})$ computational complexity per head.

Listing 6.7 contrasts the high-level matrix formulation with an explicit loop implementation that exposes this quadratic scaling. The matrix version lowers to optimized GEMM routines on accelerator Tensor Cores, whereas the loop version reveals the underlying physical execution structure: batch iterations, quadratic query-key inner products, row-wise softmax reductions, and the final value accumulation pass across all sequence positions.

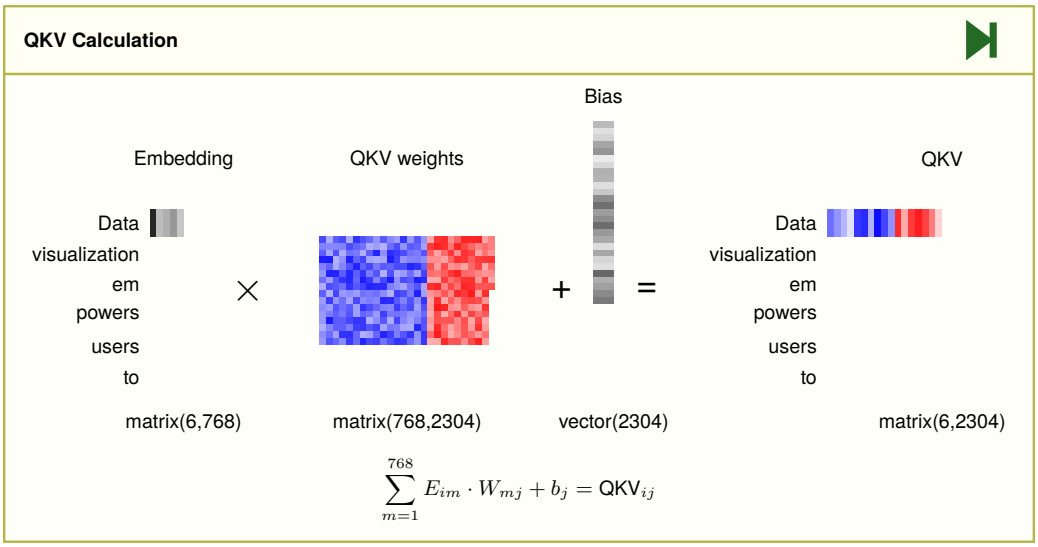


Figure 6.9: QKV Projection Computation: The embedding matrix (6×768) multiplies with QKV weight matrices (768×2304) plus bias to produce combined projections (6×2304). The 2304 output dimension contains concatenated query, key, and value projections (each 768-dimensional). This single batched matrix multiplication, requiring $6\times768\times2304 = 10.6$ million MACs, replaces 3 separate projection operations for efficiency. Source: Transformer Explainer (Cho et al. 2025).

Listing 6.7: Attention Computation: Two implementations showing the same $\mathcal{O}(S^2 \times d_{\text{head}})$ complexity. The matrix form (top) uses optimized GEMM, while the nested loops (bottom) expose the quadratic pairwise comparisons: for sequence length 512 and head dimension 64, computing attention scores requires $512\times512\times64 = 16.8$M MACs per attention head, plus another 16.8M MACs for value aggregation.

```
def attention_layer_matrix(Q, K, V):
    # Q, K, V: (batch_size × seq_len × d_head)
    # Compute attention scores
    scores = matmul(Q, K.transpose(-2, -1)) / sqrt(d_head)
    weights = softmax(scores)  # Normalize scores
    output = matmul(weights, V)  # Combine values
    return output


def attention_layer_compute(Q, K, V):
    # Initialize outputs
    scores = np.zeros((batch_size, seq_len, seq_len))
    outputs = np.zeros_like(V)

    # Loop 1: Process each sequence in batch
    for b in range(batch_size):
        # Loop 2: Compute attention for each query position
        for i in range(seq_len):
            # Loop 3: Compare with each key position
            for j in range(seq_len):
                # Compute attention score
                for d in range(d_head):
                    scores[b, i, j] += Q[b, i, d] * K[b, j, d]
                scores[b, i, j] /= sqrt(d_head)

        # Apply softmax to scores
        for i in range(seq_len):
            scores[b, i] = softmax(scores[b, i])

        # Loop 4: Combine values using attention weights
        for i in range(seq_len):
            for j in range(seq_len):
                for d in range(d_head):
                    outputs[b, i, d] += scores[b, i, j] * V[b, j, d]

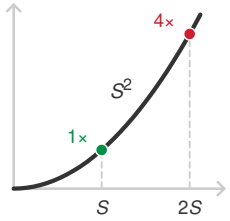
    return outputs
```

6.7.4 System implications

Attention mechanisms exhibit distinctive system-level patterns that break from fixed-stencil and recurrent architectures. In iron law terms, attention eliminates the latency-bound serial path of RNNs at the cost of quadratic score arithmetic and massive intermediate data movement. Evaluating these system demands requires distinguishing what physics makes mathematically unavoidable from what engineering can optimize away at runtime.

6.7.4.1 Memory requirements

Attention mechanisms require storage for query-key-value projections and intermediate feature representations. A naive implementation also materializes an $S \times S$ attention-weight matrix for every sequence and head, creating a quadratic memory bottleneck alongside the $S \times d$ inputs and outputs. For long sequences, these transient scores can dominate accelerator memory even though the learned projection matrices remain fixed in size. Tiled exact-attention kernels such as FlashAttention stream score blocks through faster memory and avoid retaining the complete matrix in HBM, while still computing S^2 dense interactions (section C.5 derives the online softmax recurrence and its I/O reduction). This distinction separates an unavoidable quadratic compute cost for dense attention from optional quadratic score storage. Without kernel tiling, materializing intermediate score matrices quickly exhausts accelerator high-bandwidth memory at production sequence lengths.



Doubling context quadruples score interactions and naive score storage.

Napkin Math 6.1: The quadratic bottleneck

Problem: How much memory does a materialized attention matrix require at sequence length $S = 100{,}000$ (context window)?

Math:

1. **Matrix size**: The attention score matrix ($\mathbf{QK}^T$) has dimensions $S \times S$ per head, across N_{heads} heads (12 heads in this example).
2. **Elements**: 100,000 × 100,000 × 12 heads = 1.2×10^{11} elements.
3. **Memory**: At FP16 (2 bytes/element): $1.2 \times 10^{11} \times 2$ bytes = 240 GB.
4. **Retained layers**: 240 GB per layer × 32 retained layers = 7,680 GB.

Systems insight: A materialized attention matrix consumes 240 GB per layer. Naive training that retains all 32 layers' scores for backward would require 7,680 GB, far exceeding any single GPU's capacity. This memory wall motivates two broad implementation strategies: avoid materializing the full matrix by tiling the computation, or reduce the number of scores computed in the first place.

6.7.4.2 Computation and data movement

Attention computation splits into two primary GEMM phases interleaved by a row-wise reduction: generating pairwise query-key scores and computing weighted sums over values. Query-key inner products require $B \times N_{\text{heads}} \times S^2 \times d_{\text{head}}$ MAC operations, mirrored symmetrically by the value-aggregation stage. The intervening softmax requires row-wise maximum finding, exponentiation, and summation. Unlike static weight convolutions where filter weights are reused across fixed grid positions, attention computes transient, input-dependent affinity matrices that must be refreshed for every token configuration.

Checkpoint 6.3: Quadratic scaling intuition

Long-context scaling is shaped by the cost of attention. Verify your intuition:

- ☐ **Quadratic growth**: can you explain why doubling sequence length quadruples dense score interactions and a materialized score matrix?
- ☐ **Tiling boundary**: can you explain why long-context dense attention remains more expensive even when tiling removes full-matrix storage?

Data movement in attention creates memory access patterns distinct from fixed-stencil operations. Evaluating attention requires projecting and streaming query, key, and value tensors, then routing value vectors according to dynamically evaluated scores. In naive implementations, staging the intermediate $S \times S$ score matrix in global HBM/DRAM turns score materialization into the dominant bandwidth bottleneck. While static convolutions benefit from deterministic cache lines and RNNs reuse weights across time, naive attention generates massive, short-lived activation footprints that challenge standard on-chip cache capacities.

Example 6.4: The quadratic wall

Scenario: BERT was pretrained with 512 learned position embeddings, so its deployments enforced a hard 512-token input limit (Devlin et al. 2019).

Diagnosis: Inputs beyond the pretrained positions are undefined for the model, however much memory is available. The limit itself was chosen partly because attention cost grows quadratically with sequence length. Extending from 512 to 4,096 tokens would multiply materialized score memory by 64×.

Systems lesson: Super-linear scaling turns algorithmic complexity into a hardware constraint that becomes fixed in the model itself once pretraining chooses it. No serving-time memory fix extends BERT past 512 positions, so its longer inputs must be truncated or split into windows. A model meant for long inputs must be pretrained with longer positions, and tiled or sparse attention is what keeps that training and its serving within the HBM activation budget.

Attention replaces the serialized hidden-state bottleneck of recurrent models with dynamic, constant-depth routing, but shifts the system burden to quadratic arithmetic scaling and high memory bandwidth traffic. Built into a full architecture without recurrence, this primitive enables end-to-end parallel sequence processing on modern accelerators. The transformer architecture (section 6.8) builds directly on this foundation, establishing the dominant architectural paradigm for modern large-scale machine learning systems.

6.8 Transformers: Parallel Sequence Processing

Attention provides the computational primitive of dynamic, content-dependent routing between positions, yet it was originally layered on top of recurrent architectures, inheriting their sequential bottleneck. The transformer architecture removes recurrence and makes attention the operation that mixes information between positions in every layer. This enables parallel computation across sequence positions during full-sequence processing while retaining dynamic connectivity. This architectural shift introduces distinct physical costs: quadratic dense-score computation during prefill, accumulating key-value state during serving, and severe memory-bandwidth pressure during autoregressive generation.

Definition 6.6: Transformers

Transformers are neural network architectures that combine self-attention, feed-forward layers, residual connections, normalization, and positional information without recurrence, enabling parallel processing across sequence positions during training and prefill.

1. **Significance**: Parallelism is the systems payoff. An RNN processing an S-token sequence executes S sequentially dependent steps. A transformer processes all positions of a sequence concurrently, mapping token interactions to large matrix multiplications (GEMMs) that maximize accelerator compute utilization. Dense self-attention still computes $\mathcal{O}(S^2)$ pairwise score interactions (section 6.7.4.1).
2. **Distinction**: Unlike attention used inside a recurrent backbone, which inherits the host architecture's $\mathcal{O}(S)$ sequential path, transformers use attention as the primary mixing operation between positions, alternating it with position-wise feed-forward networks.

3. **Common pitfall**: A frequent misconception is that transformers have "infinite context." Two distinct memory footprints matter: the $\mathcal{O}(S^2)$ attention-score activation matrix when materialized during training and prefill, and the $\mathcal{O}(S)$ key-value (KV) cache that accumulates during autoregressive serving. Exact tiled attention avoids the full score matrix while retaining quadratic dense arithmetic. At long sequence lengths, the KV cache footprint can rival or exceed the static model weights, turning KV cache management and memory-bandwidth efficiency into dominant engineering constraints.

6.8.1 Pattern processing needs

The recurrent sequence-to-sequence hybrids that first carried attention (Sutskever et al. 2014; Bahdanau et al. 2015) fixed the information bottleneck of section 6.6.4 but not its serialization. Step t still waited on step $t-1$, so a 1,000-token input still paid 1,000 ordered state updates no matter how many execution units the accelerator offered. Removing that chain requires replacing sequential state propagation with all-to-all interactions among positions, giving up the RNN's bounded per-step memory in exchange for parallel accelerator execution.

The transformer architecture (Vaswani et al. 2017) establishes self-attention as the primary mixing primitive across positions, paired with positional encodings to inject sequence order. This architectural choice trades the spatial parameter efficiency of CNNs and the bounded $\mathcal{O}(1)$ step state of RNNs for unconstrained sequence parallelism during training and prefill.

6.8.2 Algorithmic structure

The architectural shift in transformers centers on self-attention layers. Unlike encoder-decoder attention where queries originate from the target sequence and keys/values from the source, self-attention derives queries, keys, and values from the identical input sequence $\mathbf{X}$. Making all three projections self-referential allows the model to resolve intra-sequence dependencies across arbitrary token distances without intermediate recurrent steps. Equation 6.7 formalizes this transformation:

$$\text{SelfAttention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^T}{\sqrt{d_{\text{head}}}}\right)\mathbf{X}\mathbf{W}_V \tag{6.7}$$

Here, $\mathbf{X}$ is the input sequence, and $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are learned weight matrices for queries, keys, and values respectively. This formulation highlights how self-attention derives all its components from the same input, creating a dynamic, content-dependent processing pattern. Without positional information or an attention mask, matrix multiplication and row-wise softmax operate symmetrically across the token dimension, making self-attention permutation-equivariant: shuffling the rows of $\mathbf{X}$ merely shuffles the output rows identically, treating the sequence as an unordered set. Positional encodings, whether added as sinusoidal vectors or applied via rotary position embeddings, inject sequence order and relative distance.

For autoregressive models, the causal mask enforces directional dependency during both training and generation. It sets attention scores for future positions ($t' > t$) to negative infinity before the softmax, ensuring that each position attends only to preceding tokens and itself. During next-token training, the entire target sequence is already available in advance. Shifting the known sequence right by one position pairs each input token with its target successor (figure 6.10), while the causal mask guarantees that evaluation at position t cannot access tokens at $t' > t$. Because all input tokens are known simultaneously, an accelerator evaluates every position in the sequence concurrently using dense matrix multiplications rather than stepping through a serial time loop as an RNN must. Autoregressive inference, in contrast, must generate token t before extending the context to predict token $t+1$, though prompt prefill similarly evaluates the entire initial prompt in parallel.

Transformers extend single-head self-attention by executing multiple attention operations in parallel through multi-head attention. Instead of computing a single set of attention weights over d dimensions, the architecture projects queries, keys, and values into N_{heads} distinct lower-dimensional subspaces of dimension $d_{\text{head}} = d/N_{\text{heads}}$. Each head attends to different representation patterns

simultaneously. Their outputs are concatenated and projected through an output matrix $\mathbf{W}_O$, as formalized in equation 6.8:

$$\text{MultiHead}(\mathbf{X}_Q, \mathbf{X}_K, \mathbf{X}_V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_{N_{\text{heads}}})\mathbf{W}_O \tag{6.8}$$

where each attention head is computed as follows. In this multi-head notation, $\mathbf{X}_Q$, $\mathbf{X}_K$, and $\mathbf{X}_V$ denote the source representations before the head-specific projections; for self-attention, all three are $\mathbf{X}$. The head matrices apply the projections once:

$$\text{head}_i = \text{Attention}(\mathbf{X}_Q\mathbf{W}_{Q,i}, \mathbf{X}_K\mathbf{W}_{K,i}, \mathbf{X}_V\mathbf{W}_{V,i})$$

The $\sqrt{d_{\text{head}}}$ scaling introduced with the attention operator follows from a variance argument. Assuming query and key components are independent random variables with zero mean and unit variance, their inner product $\mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_{\text{head}}} q_{im}k_{jm}$ has zero mean and variance d_{head}, so dividing by $\sqrt{d_{\text{head}}}$ restores unit variance and keeps softmax out of its saturated, near-zero-gradient region at large head dimensions.[21]

Conceptually, attention implements a differentiable content-addressable memory. Whereas conventional memory architectures retrieve stored values at explicit addresses, self-attention uses the query vector $\mathbf{q}_i$ to score similarity against all candidate keys $\mathbf{K} \in \mathbb{R}^{S \times d_{\text{head}}}$. The resulting softmax distribution generates a continuous interpolation over the value vectors $\mathbf{V} \in \mathbb{R}^{S \times d_{\text{head}}}$, functioning as a differentiable lookup operation.

Although individual attention heads often specialize in distinct syntactic or semantic patterns (Clark et al. 2019)—dynamically adapting the effective connectivity graph for each input sequence—empirical analyses reveal substantial head redundancy and acute numerical sensitivity in reduced-precision formats. These hardware-relevant behaviors motivate structural pruning, low-rank factorization, sparse attention patterns, and specialized quantization strategies.

To transform these dynamic projections into deep representations, the transformer architecture embeds self-attention within a standardized block topology. As diagrammed in figure 6.10, each block wraps multi-head attention and position-wise feed-forward sublayers with residual connections and layer normalization. Sinusoidal positional encodings combine with token embeddings at the base of both towers, feeding repeated multi-head attention and feed-forward stages before terminating in linear and softmax output heads. Because all positions can be processed in parallel rather than sequentially, the architecture trades recurrent state for large matrix operations that map well to accelerator training. The residual paths and normalization layers in this stack are shared building blocks rather than transformer inventions; section 6.10.2.2 and section 6.10.2.3 explain why they make deep stacks trainable and what each costs the machine.

6.8.3 Computational mapping

Listing 6.8 presents a typical implementation, showing how self-attention derives queries, keys, and values from the same input sequence.

[21] **Attention scaling ($\sqrt{d_{\text{head}}}$)**: This normalization directly counteracts the linear growth in variance (d_{head}) of the query-key dot product, preventing the softmax function from saturating where gradients would otherwise vanish. The systems consequence is most acute in mixed-precision training: when activations grow large, unscaled dot products can produce logits that overflow the 16-bit float range, destabilizing or halting learning entirely.

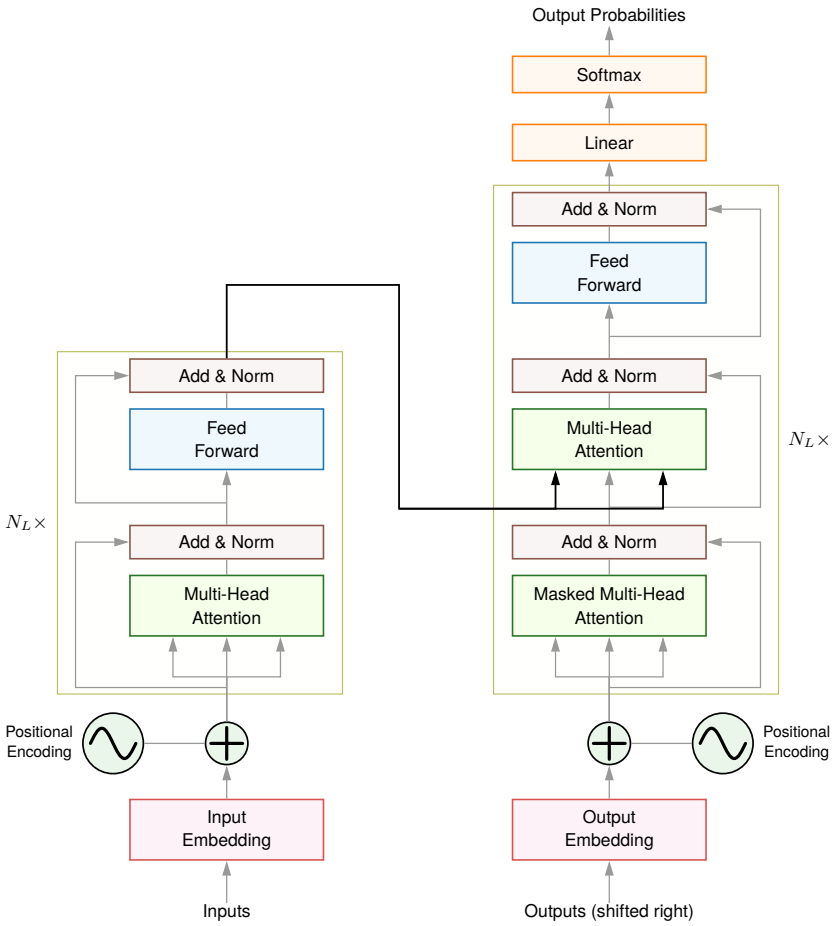


Figure 6.10: Transformer Architecture (Encoder-Decoder): Complete architecture. The encoder (left, repeated N_L times) consists of multi-head attention followed by feed-forward layers, each with residual connections (arrows bypassing blocks) and layer normalization. The decoder (right) adds masked attention to prevent attending to future tokens during training and autoregressive generation. Positional encodings (sine waves) inject sequence order information absent from the permutation-equivariant attention operation. This design enables training parallelism across all positions while the decoder maintains autoregressive causality during inference. Source: (Vaswani et al. 2017).

Listing 6.8: Self-Attention and Multi-Head Attention: Self-attention (top) derives **Q**, **K**, and **V** from the same input **X** through three projections, then computes attention as before. Multi-head attention (bottom) runs N_{heads} parallel attention heads with dimension $d_{\text{head}} = d/N_{\text{heads}}$, then concatenates and projects. For GPT-2 (768-dim, 12 heads), each head operates on 64 dimensions, partitioning the projected feature dimension across heads while enabling diverse relationship patterns.

```
def self_attention_layer(X, W_Q, W_K, W_V, d_head):
    # X: input tensor (batch_size × seq_len × d_model)
    # W_Q, W_K, W_V: weight matrices (d_model × d_head)

    Q = matmul(X, W_Q)
    K = matmul(X, W_K)
    V = matmul(X, W_V)

    scores = matmul(Q, K.transpose(-2, -1)) / sqrt(d_head)
    attention_weights = softmax(scores, dim=-1)
    output = matmul(attention_weights, V)

    return output

def multi_head_attention(X, W_Q, W_K, W_V, W_O, num_heads, d_head):
    outputs = []
    for i in range(num_heads):
        head_output = self_attention_layer(
            X, W_Q[i], W_K[i], W_V[i], d_head
        )
        outputs.append(head_output)
```

The attention listing accounts for information mixing across positions, but a complete block also transforms features within each position. An ordinary position-wise feed-forward network applies a dense projection from hidden width d to an intermediate layer output width d_{out}, a nonlinear activation, and a second dense projection back to d. For a sequence of length S, its tensor shapes follow $S \times d \to S \times d_{\text{out}} \to S \times d$. The same two weight matrices apply independently at every position, so the MLP accounting gives $2Sdd_{\text{out}}$ MACs and an intermediate tensor with Sd_{out} elements. This intermediate width is the feed-forward width in table 6.8.

For one full-sequence block at batch one, with one standard multi-head self-attention sublayer and that two-layer feed-forward network, the query, key, value, and output projections together require $4Sd^2$ MACs. The two attention products add $2S^2d$ MACs across all heads, whose widths sum to d. Adding the feed-forward cost completes the block's matrix-product account; these counts omit biases, softmax, normalization, nonlinear activations, and residual additions, and assume dense evaluation of all position pairs. At fixed widths, projections and the feed-forward network grow linearly with sequence length, while the pairwise products grow quadratically. Which part dominates therefore depends on both the sequence length and the chosen widths, even before memory traffic is measured.

During autoregressive inference, generating output tokens step-by-step reintroduces sequential serialization. At batch one, projection matrix multiplications become matrix-vector operations with less weight reuse, making off-chip bandwidth a likely limit when weights exceed cache capacity, as quantified in section 6.8.4 via the GPT-2 XL lighthouse.

6.8.4 System implications

The quadratic bottleneck analyzed in section 6.7.4 manifests differently across execution regimes. Because full-sequence training and prompt prefill operate on known token sequences under causal masking, accelerators evaluate all positions concurrently using dense matrix multiplications. Autoregressive decoding, by contrast, must generate each token before predicting the next. That loss of token concurrency reduces weight reuse and can shift the primary performance limit from arithmetic throughput to memory bandwidth and state retrieval; sequence length, batch size, and memory hierarchy limits determine whether that transition occurs.

6.8.4.1 Training: The quadratic compute wall

During training and prompt prefill, all token positions are processed in parallel, allowing accelerator Tensor Cores to achieve high floating-point utilization, but dense attention's score computation grows as $\mathcal{O}(S^2)$ with sequence length S. At 32k tokens, a materialized score matrix would occupy gigabytes per layer across multiple heads, and writing it to HBM and reading it back for softmax would make the layer bandwidth bound. I/O-aware tiling such as FlashAttention (T. Dao et al. 2022) fuses the dot-product, softmax, and value-weighting steps into on-chip SRAM tiles (section 6.7.4.1), which removes that round-trip and leaves quadratic compute as the wall that remains.

Expanding sequence length S challenges accelerator memory hierarchies because full-sequence self-attention scales quadratically with context length. Figure 6.11 charts this scaling trajectory on a logarithmic axis, showing context windows growing from 2K tokens to beyond 128K and into the 1M-token range (OpenAI 2023c; Anthropic 2023; Google 2024a). These product milestones matter less than the principle they illustrate. Longer context windows shift complexity from external retrieval and document chunking pipelines into accelerator memory capacity and bandwidth budgets. Exact tiling changes how much attention state is stored while retaining dense quadratic arithmetic; sparse attention instead reduces the interactions computed (section 6.7.4.1).

6.8.4.2 Small-batch autoregressive decoding: The memory bandwidth wall

Autoregressive decoding generates one token at a time and often becomes memory-bandwidth bound at batch one ($B = 1$) or small batch sizes when weights exceed on-chip memory. During prompt prefill or training, weight reuse across many token positions produces dense GEMMs with much higher arithmetic intensity. A large request batch ($B \gg 1$) supplies many such positions, but prefill can obtain this reuse even for a single request with a long prompt.

During single-token generation ($B = 1$), however, each projection layer collapses into a matrix-vector multiplication (GEMV). In the streamed-weight regime, the processor reads each projection

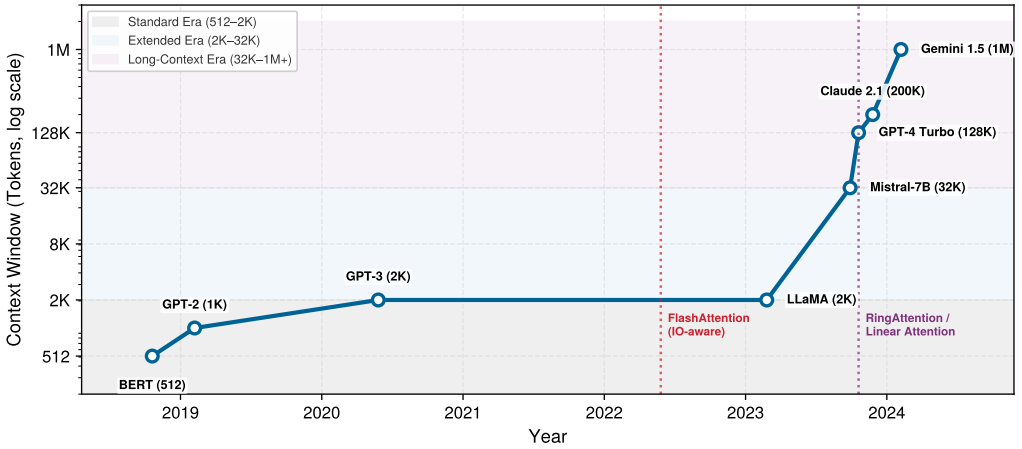


Figure 6.11: Context Window Explosion: Model context capacity (tokens, log scale) from early models (512 to 2K tokens) to modern long-context architectures (128K to 1M tokens). As context scales $1{,}000\times$, naive attention memory grows $1{,}000{,}000\times$ ($\mathcal{O}(S^2)$), driving the development of FlashAttention, sparse attention, and linear attention mechanisms.

weight from DRAM to multiply it by a single new token vector. This is the batch-one regime that equation 6.2 already derived for the MNIST layer at 0.5 FLOP/byte in FP32. Halving the weight width to 2 bytes in FP16, while still executing one multiply-accumulate operation (1 MAC = 2 FLOPs) per weight read, doubles that weight-only arithmetic intensity to exactly:

$$I_{\text{GEMV}} = \frac{2 \text{ FLOPs}}{2 \text{ bytes}} = 1.0 \text{ FLOP/byte}$$

↩ The 1.0 FLOP/byte intensity of batch-one GEMV in 16-bit precision, and 0.5 FLOP/byte in FP32, is established in section 5.6.1.3.

Because modern accelerators require tens to hundreds of FLOPs per byte to reach peak compute throughput, running at 1.0 FLOP/byte leaves compute cores idling for the vast majority of execution time, waiting for weights to arrive across the memory bus. To generate each token, the system performs three operations:

1. **Read model weights**: Stream every parameter from off-chip memory to registers (for example, a 70-billion-parameter model streams 140 GB of FP16 weights per token; GPT-2 XL makes the cost of that first step concrete).
2. **Perform matrix-vector operations**: Compute projections with arithmetic intensity capped at 1.0 FLOP/byte.
3. **Read and update the KV cache**: Append new key and value vectors while streaming historical keys and values from memory.

The KV cache[22] defined with the attention mechanism grows linearly with sequence length ($\mathcal{O}(N_L \times 2 \times N_{\text{heads}} \times S \times d_{\text{head}})$ per request, distinct from the $\mathcal{O}(S^2)$ attention score matrix during training), storing the key and value vectors for all previous tokens to avoid recomputing them (Pope et al. 2023; Kwon et al. 2023). Keys and values from past tokens are retained because the newly generated token's query ($\mathbf{q}_t$) must attend across all historical context ($\mathbf{k}_1 \dots \mathbf{k}_{t-1}$ and $\mathbf{v}_1 \dots \mathbf{v}_{t-1}$) to compute its next representation. Prior queries ($\mathbf{q}_1 \dots \mathbf{q}_{t-1}$) are safely discarded because past tokens never attend forward to future tokens, while caching past keys and values avoids re-running the entire model forward pass over the full prompt at every generated token. For long contexts, this cache can become massive (for example, 100+ GB), and each decoding step reads prior keys and values. The resulting bandwidth pressure depends on context length, batching, attention architecture, cache layout, and hardware.

[22] **KV cache memory scaling**: For a 7-billion-parameter transformer in FP16, model weights consume ~14 GB; a single request's cache requires 32 layers × 2 (K,V) × 32 heads × 2,048 positions × 128 dimensions × 2 bytes, or about 1.07 GB. At 8 concurrent users, the cache is ~8.6 GB, a substantial addition to the weights. Because the cache grows linearly with context and concurrency, scaling throughput forces choices such as grouped-query attention (Ainslie et al. 2023), shorter contexts, or KV paging and offloading (Kwon et al. 2023). Paging and offloading preserve model outputs, whereas grouped-query attention changes the architecture and shorter contexts truncate historical context.

Lighthouse 6.5: GPT-2 XL (bandwidth lighthouse)

Why it matters: GPT-2 XL exemplifies **memory-bandwidth-bound** batch-one decoding. In the weight-only model used here, each decoding step reads 6 GB of FP32 weights and performs matrix-vector operations. The resulting arithmetic intensity is about 0.5 FLOP/byte with FP32 weights or 1 FLOP/byte with FP16 weights. Batching can amortize this weight traffic, while KV-cache and activation traffic add to it. Table 6.6 summarizes the bandwidth lighthouse's quantitative properties:

Table 6.6: GPT-2 XL Systems Profile: Weight-only estimates for batch-one autoregressive decoding. Under these assumptions, each step reads the model weights at roughly 0.5–1 FLOP/byte, so HBM bandwidth strongly constrains generation throughput.

Property	Value	System Implication
Parameters	1.5B	Weight loading dominates inference latency.
Model Size	6 GB (FP32)	Fits on one GPU but saturates HBM bandwidth.
Compute	3 GFLOP/token	Low per-token compute; bottleneck is data movement, not math.
Constraint	Memory Bandwidth	Weight-bound batch-one tokens/s depends strongly on HBM bandwidth.
Profile	Bandwidth-Bound (small-batch decoding)	Larger batches can amortize weight traffic and shift the bottleneck.

Across training and inference, transformers expose different opportunities for reuse: full-sequence processing parallelizes across token positions, dense attention computes quadratic score interactions, and small-batch autoregressive decoding stresses memory bandwidth when weights stream from off-chip memory.

Yet dense, spatial, and sequence models do not cover the full spectrum of production workloads. Large-scale recommendation systems depart from standard arithmetic-dominated pipelines by coupling sparse embedding lookups (where memory capacity and irregular access distributions govern throughput) with downstream dense interaction layers. This hybrid matters at production scale: Meta reported that recommendation models accounted for most of its AI inference cycles (Gupta et al. 2020), even though such workloads receive less academic attention than language or vision models. Section 6.9 examines recommendation as the chapter's final paradigm.

6.9 Sparse Architectures: Recommendation Systems

When a user opens a streaming service or e-commerce platform, the system must select a handful of recommendations from a catalog of millions within an end-to-end budget of tens of milliseconds. The engineering challenge requires mapping both users and items to dense vectors in a shared embedding space, then evaluating interaction probabilities at scale.

Large-table recommendation models expose a regime governed first by memory capacity. An accelerator engineered for dense matrix multiplication, such as an NVIDIA A100 providing hundreds of teraflops of arithmetic throughput alongside 80 GB of HBM, cannot hold terabytes of embeddings locally. The sparse stage gathers rows from these tables, placing different demands on the machine than the batched convolution and streamed-weight decoding regimes examined above.

6.9.1 Pattern processing needs

The core challenge in recommendation systems is handling high-cardinality categorical features. A model must process user IDs across billions of accounts and item IDs across millions of products. Raw IDs carry no inherent geometry. User 1042 is not "near" user 1043 in any meaningful metric space, and a neural network cannot infer semantic relationships from integer values alone.

Embeddings solve the representation problem by mapping each discrete ID to a dense vector called an **embedding**[23] (Mikolov et al. 2013). The system cost is that every lookup becomes an irregular read into a table spanning tens or hundreds of gigabytes. Recommendation models therefore encounter a capacity wall and a memory-bandwidth bottleneck before the first dense feedforward layer executes.

[23] **Embedding**: Deriving from the mathematical embedding of one space into another, neural embeddings map discrete tokens (such as user IDs or vocabulary words) into continuous vector spaces where geometric proximity captures semantic similarity. Word2vec popularized neural word embeddings in 2013. For systems, embedding tables create a distinctive memory access pattern: each lookup is an irregular read into a potentially terabyte-scale table, producing a sparse, bandwidth-bound workload that makes DLRM fundamentally different from compute-bound architectures like ResNet.

6.9.2 Algorithmic structure

The DLRM architecture (Naumov et al. 2019) standardizes this workflow as a four-stage pipeline partitioned across dense and sparse regimes. Continuous features such as user age or time of day first flow through a bottom MLP, a compute-intensive but memory-light stage dominated by dense matrix multiplications. In parallel, categorical IDs index into dedicated embedding tables, which is where the memory capacity wall appears:

For example, an embedding table for one billion users with 128-dimensional vectors requires $10^9 \times 128 \times 4$ bytes $\approx$ 512 GB of memory. This stage is memory-intensive but compute-light because each lookup is a memory copy with zero arithmetic reuse. The interaction layer then combines the dense activations from the bottom MLP with the gathered sparse embedding vectors through pairwise dot products that capture user-item correlations. Finally, a top MLP processes the concatenated interaction representations through feedforward layers to predict a target probability such as click-through rate. This combination of dense and sparse computation makes DLRM the chapter's recommendation lighthouse; section 6.9.3 quantifies the capacity-bound profile that results.

6.9.3 Computational mapping and system implications

DLRM's computational mapping splits into two regimes that stress different hardware subsystems. The dense MLPs execute standard GEMM operations, identical to the MLP computational mapping discussed in section 6.4.4 and handled efficiently by Tensor Cores. The sparse embedding lookups, however, are qualitatively different. They are index-based memory gathers that copy parameter data with zero arithmetic reuse. An isolated lookup is bound by memory access latency; but when a batch executes hundreds of lookups across features and samples, aggregate throughput becomes memory-bandwidth bound as concurrent requests saturate the memory bus. This operational throughput limit is distinct from the capacity constraint: total table size dictates whether parameters fit in accelerator memory or must spill across PCIe to host DRAM, whereas available bandwidth governs how fast gathered rows arrive. Within each selected row, an embedding vector is contiguous (for instance, 128 FP32 elements spanning 512 bytes), allowing the downstream pooling loop to consume every transferred byte. Across rows, however, data-dependent categorical IDs lack predictable spatial locality, defeating hardware prefetchers between row accesses. Access popularity remains heavily skewed: a small set of hot rows (popular items, frequent users) exhibits strong temporal locality in on-chip caches, while the long tail of rarely touched rows misses to main memory. Listing 6.9 writes this sparse stage as a loop nest parallel to those of the other families.

Listing 6.9: Embedding-Bag Gather: Four nested loops expose DLRM's sparse stage. Loops 1 and 2 parallelize across samples and features, but Loop 3 takes its addresses from the input IDs, so hardware cannot predict them, and Loop 4 performs one addition per 4-byte FP32 value loaded. Batching raises that intensity only when samples share hot rows.

```
def embedding_bag_compute(sparse_ids, tables):
    # Loop 1: Process each sample in the batch (parallelizable)
    for b in range(batch_size):
        # Loop 2: Each categorical feature has its own table
        for f in range(num_features):
            pooled[b, f, :] = 0
            # Loop 3: Multi-hot feature, one gather per active ID
            for idx in sparse_ids[b][f]:  # data-dependent address
                row = tables[f][idx]  # irregular read, little reuse
                # Loop 4: Pool (sum) the gathered rows
                for d in range(embed_dim):
                    pooled[b, f, d] += row[d]
    return pooled  # feeds the interaction layer
```

Lighthouse 6.6: DLRM (recommendation lighthouse)

Why it matters: DLRM exemplifies **memory-capacity-bound** workloads. Its massive embedding tables often exceed the memory of a single accelerator or server, so the system must decide

where those tables live before it can optimize arithmetic throughput. The interaction layer then gathers selected embedding vectors and combines them with dense features, making capacity and irregular data movement the dominant constraints. This capacity-first decision differs from the batched convolution and streamed-weight decoding regimes examined here, requiring different hardware and deployment choices. Table 6.7 summarizes the recommendation lighthouse's quantitative properties.

Table 6.7: DLRM Systems Profile: Quantitative properties of the recommendation lighthouse and their system consequences. Billion-entry embedding tables push the model past single-device memory, creating a capacity-bound regime that decides which bandwidth the iron law's data term must pay.

Property	Value	System Implication
Embedding Parameters	25B	Parameters × 4 bytes; dominates total model size.
Model Size	100 GB (FP32)	May exceed one device's fast memory.
Constraint	Memory Capacity	Model size > Single GPU Memory.
Bottleneck	Irregular Data Movement	Gather operations dominate sparse feature processing.
Profile	Mixed (Sparse/Dense)	Combines memory-heavy lookups with compute-heavy MLPs.

Once embedding tables exceed one device, capacity, not arithmetic, becomes the primary systems constraint. While a ResNet-50 (102.4 MB) fits on a single accelerator and a dense language model like GPT-3 (350 GB) fits within a single multi-GPU node, production recommendation models reach terabytes of embedding parameters. In iron law terms, capacity acts as a feasibility constraint that comes before either execution term. It decides where the tables live, and therefore which bandwidth the data term D_{vol}/BW must use: HBM when the tables fit, host DRAM over PCIe or a peer accelerator over NVLink when they do not.

The architecture therefore breaks the single-device assumption that holds for smaller dense models. Systems designers resolve this capacity wall through three distinct architectural strategies:

- **Shrink the tables**: Compression, hashing, or pruning reduces capacity pressure, though these techniques risk information loss on long-tail users and items.
- **Move the tables**: Embeddings can reside in host DRAM or external key-value stores, though remote lookups incur high data-movement latencies over PCIe or Ethernet.
- **Partition the tables**: Tables can be sharded across multiple accelerators so that no single device stores the full model, though lookups then require cross-device communication.

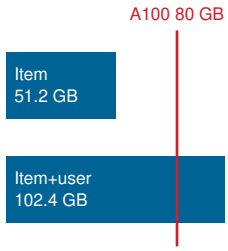
Architecturally, DLRM turns recommendation into a capacity-management challenge before it becomes a compute-optimization task. Training-time model partitioning and hardware-accelerated interconnects address these distributed layouts, but the baseline sizing challenge begins with a single table.

Napkin Math 6.2: The capacity wall

Problem: Consider a recommendation system for a store with 100M items using an embedding size of 128. How much memory does the item table alone require?

Math:

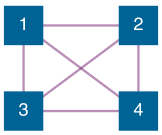
1. **Table entries**: 100M items.
2. **Vector size**: 128 elements.
3. **Precision**: FP32 (4 bytes per element).
4. **Table size**: 100M items × 128 × 4 bytes ≈ 51.2 GB.
5. **Capacity share**: 51.2 GB ÷ 80 GB = 64 percent of device capacity.
6. **Two-table check**: Two tables at 64 percent each exceed 100 percent of device capacity.



One embedding table fits an 80 GB A100; a second breaches the capacity wall.

Systems insight: A single embedding table for one feature (items) already consumes 64 percent of an 80 GB A100 GPU. Adding a user table of the same size means the embedding tables no longer fit on a single 80 GB accelerator, motivating table compression, off-device memory, or partitioning across devices. DLRM is capacity bound: the primary systems question is not how many FLOP/s the accelerator can deliver, but where the embedding state can physically reside.

Once the tables no longer fit on one device, sparse lookups cross memory and interconnect boundaries. A request may need rows owned by several accelerators, forcing the device executing the downstream MLP to wait while the system gathers them across PCIe or NVLink. Within a multi-accelerator server, gathering rows distributed across multiple GPUs turns memory routing into an interconnect bandwidth bottleneck. The architectural consequence is that a recommendation model's memory layout directly determines its cross-accelerator data-transfer traffic, creating a coordination tax for both training and serving.



Sharded tables can turn lookups into all-to-all accelerator traffic.

↪ Partitioning a model that exceeds one device's memory across accelerators, and the communication that partition boundaries add, is developed in section 8.8.1.2.

Checkpoint 6.4: DLRM and sparse scatter

Recommendation systems stress a different part of the machine than CNNs or transformers.

- ☐ **Capacity-bound**: Can you explain why embedding tables push DLRM into a memory capacity regime where "FLOPs" is not the binding constraint?
- ☐ **Sparse access**: Can you explain why embedding lookups defeat prefetching, and why popularity skew still lets a cache capture the hot rows?
- ☐ **Placement logic**: If a single embedding table does not fit in fast device memory, can you describe which state must move, shrink, or be partitioned?
- ☐ **Data-movement bottleneck**: Can you explain why sparse embedding lookups can dominate even when the dense MLP has modest FLOP cost?

With DLRM, the roster of five families is complete, and each has been analyzed on its own terms: the pattern it targets, the computation it performs, the mapping it induces, and the bottleneck it exposes. A deployment rarely gets to examine one family in isolation. The same accelerator that serves a vision model today may serve a language model or a recommender tomorrow, and the architecture decides which of its resources gives out first.

6.10 What Each Architecture Asks of the Machine

Three requests arrive at the same accelerator, one at a time. A single ResNet-50 frame performs about 8.2 GFLOP of arithmetic against 102.4 MB of FP32 weights, and each convolutional filter is reused across its feature map, creating opportunities to keep arithmetic units busy. A single GPT-2 token at batch size one performs about 3 GFLOP/token against 3 GB of FP16 weights. Streaming those weights once gives a weight-only arithmetic intensity of roughly 1 FLOP/byte, making bandwidth a likely limit. A DLRM request gathers a handful of embedding rows from tables totaling 100 GB, more than the accelerator's 80 GB of memory, so before any question of speed arises the system must decide where those tables live.

The silicon is identical in all three cases, but the demands differ. Convolutional reuse favors arithmetic throughput, streamed token projections stress bandwidth, and oversized embedding tables impose a capacity constraint that decides which bandwidth their lookups pay. The iron law (section 1.7) turns these structural observations into predictions once execution conditions and machine balance are specified. The profiles below collect those conditions, shared building blocks explain costs that transfer between families, and deployment budgets screen the resulting candidates.

6.10.1 Five families on one machine

Table 6.8 collects that profile. Each row is a family in a representative execution regime; the transformer takes two rows because prefill and decode reuse the same weights differently (section 6.8.4.2).

The compute-bound rows assume sufficient reuse to exceed the machine's balance, the bandwidth-bound rows assume streamed operands, and the capacity-bound row assumes oversized tables. The columns follow a request through the machine: persistent state, per-request state, arithmetic intensity, parallel axes, and the likely limiting term under those assumptions.

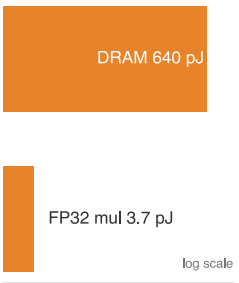
Table 6.8: Architecture Workload Profiles: State, per-request activations, intensity regime, parallel axis, and binding iron-law term for each family in its common execution regime. Symbols follow the family sections: B is batch size, S sequence length, d the hidden dimension, d_{in} and d_{out} a layer's input and output widths, d_{ff} the feed-forward width, K kernel size, C_{in} and C_{out} input and output channels, and $H_{\text{img}} \times W_{\text{img}}$ the feature-map size. Intensity regimes are qualitative; table 6.2 gives weight-only proxies for the lighthouses. The $\mathcal{O}(BS^2)$ prefill term assumes materialized attention scores (section 6.7.4.1).

Family	Parameters	Activations	Intensity	Parallel Axis	Binding Term	Lighthouse
MLP	$\mathcal{O}(d_{\text{in}} d_{\text{out}})$ per layer	$\mathcal{O}(B\, d_{\text{out}})$	Low at batch one (GEMV); high when batched (GEMM)	Batch	D_{vol}/BW at small batch	None (DLRM's dense MLPs)
CNN	$\mathcal{O}(K^2 C_{\text{in}} C_{\text{out}})$ per layer, independent of image size	$\mathcal{O}(B\, H_{\text{img}} W_{\text{img}} C_{\text{out}})$	High (filters reused across positions); lower for depthwise	Batch and spatial positions	$O/(R_{\text{peak}}\, \eta_{\text{hw}})$	ResNet-50; MobileNetV2 and KWS
RNN	$\mathcal{O}(d^2 + d d_{\text{in}})$, shared across steps	$\mathcal{O}(B\, d)$ state; $\mathcal{O}(BSd)$ in training	Low at batch one; higher with batching	Batch and within-step work; recurrent time is serial	Serial step cost; dispatch adds L_{lat} unless fused	None
Transformer (prefill)	$\mathcal{O}(d^2 + d d_{\text{ff}})$ per block	$\mathcal{O}(BSd)$; $\mathcal{O}(BS^2)$ if scores are materialized	High (GEMM over all S tokens)	Batch and sequence positions	$O/(R_{\text{peak}}\, \eta_{\text{hw}})$, with an S^2 attention term	GPT-2 (prompt)
Transformer (decode)	Same weights, reread for every token	KV cache, $\mathcal{O}(BSd)$ per layer	Low (GEMV per token at small batch)	Batch only; tokens are serial	D_{vol}/BW (weights and KV cache)	GPT-2 (generation)
DLRM	Embedding tables, one row per category, dominate; dense MLPs are small	Pooled embedding vectors per feature	Very low (gathers with little arithmetic)	Batch; tables sharded across devices	Capacity first, then D_{vol}/BW	DLRM

The parameter and activation columns decide how data reaches the arithmetic units, and the pattern of access matters as much as the volume. Three patterns cover the table. Sequential access, the streaming of contiguous weight rows in an MLP layer or a transformer projection, lets DRAM deliver long bursts and lets hardware prefetchers run ahead, so it approaches the interface's peak bandwidth. Strided access, which a native convolution produces as it steps across the rows of a feature map, fetches whole cache lines to use a fraction of each; im2col lowering (section 6.5.3) converts it into sequential access at the price of up to K^2-fold duplication (because overlapping sliding windows copy the same input pixels into multiple rows). Random access characterizes the DLRM row. Sparse IDs index arbitrary rows across multi-gigabyte tables; while each selected embedding vector is contiguous and fully consumed by pooling, row selection lacks predictable spatial locality between rows, which defeats hardware prefetchers and leaves sparse lookups exposed to memory latency unless many concurrent requests are in flight. Achieved bandwidth depends on vector width, alignment, and request concurrency, which is why caching popular rows is critical to reduce off-chip traffic. Attention is not random access despite its content-dependent weights, because its queries, keys, and values sit in contiguous tensors that tiled kernels stream through on-chip SRAM.

Whether an access pays DRAM prices at all depends on the working set, the data the execution units need at once. Small layers, such as the 784×100 reference layer at 313.6 KB in FP32, fit in on-chip SRAM. Production-width layers of tens of megabytes do not, which is why unbatched MLP inference and transformer decode stream their weights from off-chip memory on every request. Accelerators answer both ends of the table with memory hierarchy rather than arithmetic. HBM

stacks DRAM dies beside the processor to supply the streaming rows at more than an order of magnitude above standard DDR channels, and multi-megabyte on-chip SRAM, often an explicitly managed scratchpad, lets compilers stage tiles so that reuse happens on chip. The intensity column measures how much a family can profit from that staging, and it doubles as an energy column.



Moving a 32-bit value from DRAM costs far more energy than one FP32 multiply.

> **Systems Perspective 6.2: The energy cost of data movement**
>
> A core systems principle: *moving data costs more than computing on it*. In the reference model of Horowitz (2014), an FP32 multiply costs about 3.7 pJ/FLOP, while fetching one 32-bit operand from off-chip DRAM costs 640 pJ, a gap of 173×. Low-intensity rows are therefore energy-dominated by DRAM traffic. An unbatched MLP or a decode step loads each weight once per request and spends most of its energy moving it. A CNN amortizes each filter load over $H_{\text{img}} \times W_{\text{img}}$ applications, so the locality that makes it compute bound also makes it energy efficient, unless a fully materialized im2col buffer inflates temporary traffic by up to K^2. An RNN keeps its compact state in SRAM but idles execution units across serial steps, which raises energy per token. Tiled attention keeps score blocks on chip, so its energy tracks matrix multiplication rather than DRAM round-trips.
>
> Later optimizations exploit this principle directly. Quantization reduces the bits moved per value; pruning removes unnecessary movement; and tiling keeps working sets in faster, lower-energy memory.

The parallel-axis column determines how data must move among processing elements once the work is spread out. Figure 6.12 schematizes the four fundamental data movement topologies (broadcast, scatter, gather, and reduction) that govern tensor routing among processing elements, whether those are units on one chip or several accelerators in one server. Batch parallelism in MLPs, CNNs, and prefill broadcasts stationary weight tiles to many units, each working on a different example or position, and compilers tile loops to reuse each broadcast value as often as possible. Spatial and model partitioning scatter a tensor across tiles or devices; mixture-of-experts models, which route each token to a few of many feedforward subnetworks, make the scatter input-dependent, so a popular expert can overload one accelerator while others idle. DLRM gathers embedding rows from wherever the sharded tables live, and each attention query gathers keys and values from every position in its sequence. Reductions combine values through sums or maxima (softmax denominators, normalization statistics, and the summing of gradients across accelerators that each trained on a different slice of the batch), and tree-structured reduction hardware finishes a reduction over n values in $\mathcal{O}(\log n)$ stages rather than $\mathcal{O}(n)$. The rows with a serial axis, RNNs and decode, are the rows these collectives cannot rescue, because no amount of broadcast or reduction bandwidth shortens a dependency chain.

The last column applies the iron law under each row's execution assumptions. When effective intensity exceeds the machine's balance and parallel work is sufficient (batched CNNs, prefill), the compute term $O/(R_{\text{peak}}\,\eta_{\text{hw}})$ binds. Below that balance (streamed small-batch MLPs, decode), the data term D_{vol}/BW binds, favoring fewer transferred bytes, more reuse, or higher bandwidth. RNNs add a serial chain of state updates: faster arithmetic, memory, and reduced launch overhead (L_{lat}) can shorten each step without removing the dependency. DLRM adds a constraint that sits in front of the equation. Capacity decides where the tables live and thus which bandwidth the data term pays. ResNet-50, GPT-2, and DLRM isolate these representative regimes. MobileNetV2 and KWS share the CNN row, with depthwise layers pulling their intensity down, because latency and power are deployment budgets that constrain the entire execution. No lighthouse occupies the RNN row, whose serial chain the recurrent family section analyzes directly.

6.10.2 Portable building blocks

The rows of table 6.8 differ, but the operators inside them repeat. A transformer block reuses dense projections encountered in MLPs, residual paths introduced by the ResNet family of deep CNNs, normalization for activation stability, and gating-like routing in later variants. Table 6.9 connects each building block to a representative context and to other architectures that use it. Each block

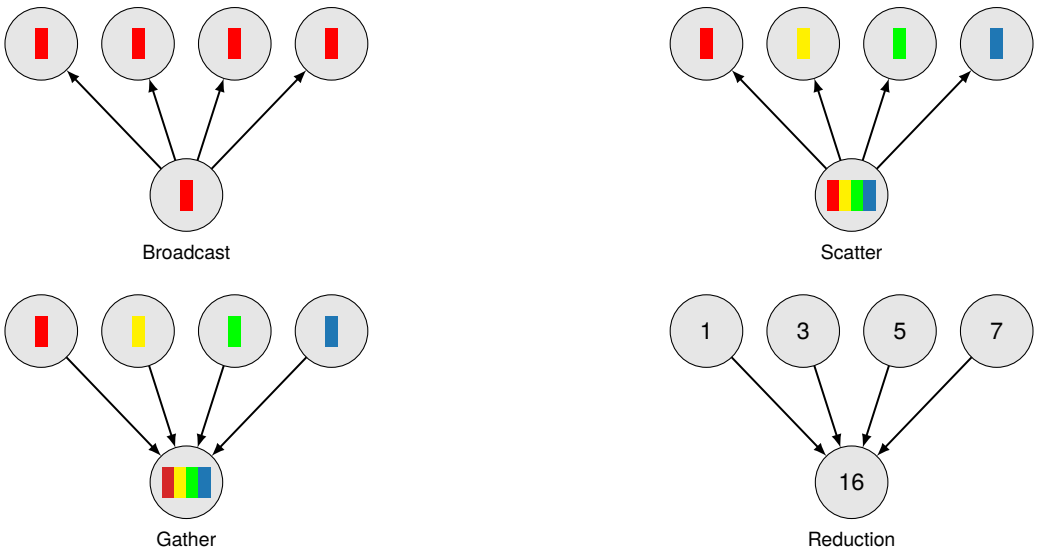


Figure 6.12: Data Movement Primitives: Four fundamental patterns govern information flow in neural network computation. Broadcast (top-left) replicates a single value to all destinations, used when sharing weights across batch elements. Scatter (top-right) distributes distinct elements to different destinations, enabling work partitioning. Gather (bottom-left) collects distributed values to a single location, as in attention pooling. Reduction (bottom-right) combines multiple values through aggregation (sum, max), appearing in gradient synchronization and attention scoring. Because moving data costs more than computing on it, these patterns are primary optimization targets.

brings its hardware cost with it, so an optimization for residual additions or normalization kernels can transfer across families.

Table 6.9: Cross-Architecture Building Blocks: Engineering primitives illustrated by one architecture family and reused across many designs. The example contexts identify familiar applications rather than historical firsts. Each building block solves a recurring problem (gradient flow, activation stability, signal routing), though not every architecture uses every primitive.

Building Block	Example Context	Problem Solved	Now Used In
GEMM	MLPs	Universal function approximation	All architectures (feedforward layers)
Parameter Sharing	CNNs	Spatial efficiency	Transformers (shared projections), RNNs (weight reuse across time)
Skip Connections	ResNets (CNNs)	Gradient flow at depth	Transformers, DenseNets, U-Nets, many modern deep networks
Normalization	CNNs (BatchNorm)	Activation stability	LayerNorm (Transformers), RMSNorm (root-mean-square normalization), GroupNorm (grouped channels)
Gating	LSTMs (RNNs)	Selective signal routing	Transformers (mixture-of-experts routing), GRUs, highway networks

6.10.2.1 Dense projections: The universal baseline

General matrix multiplication is the one primitive every row of the profile shares. It is the feedforward layer inside every transformer block, the 1×1 pointwise convolution in MobileNets, the input and recurrent projections inside every RNN cell, and the bottom and top MLPs of DLRM. Convolution reaches the same path through the im2col patch unfolding diagrammed in figure 6.5, and dedicated matrix engines, such as GPU Tensor Cores and the systolic arrays[24] of TPUs, exist because this one primitive dominates the operation count of nearly every workload. Its hardware cost depends on shape rather than on the family that invokes it. Grouping independent samples or tokens into a matrix converts a matrix-vector product into a matrix-matrix product with greater reuse. The higher intensity can cross the target machine's balance point, which is why batch size changes the modeled regime of three rows of the table.

Dense connectivity supplies a common cost baseline, but each family changes a different part of the workload. Convolution shares filters across spatial positions; attention combines dense projections with pairwise interactions; and embedding lookup replaces an arithmetic transformation with an indexed read. Their parameter storage, operation counts, and transferred bytes must therefore be compared separately, as in table 6.8. Dense projections remain inside all these architectures, so improvements to their execution can transfer between families.

[24] **Systolic array**: Named for the heart's rhythmic contraction, the array's lockstep "pulse" of data through a grid of multiply-accumulate processors directly implements the data reuse that matrix multiplication and convolution require. By passing input values between neighboring processors, an expensive round-trip to off-chip DRAM is avoided for every single multiplication

where $\mathbf{z}^{(\ell)}$ represents the preactivation and $\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$ the postactivation output of layer ℓ. Each layer contributes both a weight transformation and an activation derivative $\sigma'(\mathbf{z}^{(\ell)})$. With sigmoid activations, whose derivative peaks at 0.25 (section 5.4.2.2), the activation-derivative factors alone contribute at most $(0.25)^{N_L}$ along a path. The full gradient also depends on the intervening weight matrices; when their transformations do not offset this attenuation, early layers receive negligible updates. Exploding gradients are the destabilizing counterpart. If each layer Jacobian expands a shared direction by about 1.5, that component grows exponentially, producing overflow, NaN (not-a-number) values, and training divergence rather than a silent stall.

ReLU removes the activation's share of this decay, since its derivative is one on active paths (section 19), and that change made networks of 10–20 layers trainable. It does not remove the problem. Inactive units pass zero gradient, and a poorly initialized weight or a large update can leave a unit permanently dead; more fundamentally, ReLU does nothing to regulate the singular values of the weight matrices, which continue to attenuate or amplify gradients across depth. The original ResNet study showed the consequence on CIFAR-10: a 56-layer plain network had higher training and test error than a 20-layer plain network throughout training (He et al. 2016a). Because the deeper network was worse on the *training* set, the failure was an optimization barrier, not overfitting.

ResNet blocks introduce residual learning through skip connections that transform gradient flow. Equation 6.9 adds the identity path $\mathbf{x}$ to the residual mapping $\mathcal{F}(\mathbf{x})$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{6.9}$$

where $\mathcal{F}(\mathbf{x})$ represents the residual function (typically two convolutional layers with batch normalization and ReLU) and $\mathbf{x}$ is the identity skip connection. Figure 6.13 diagrams this original post-activation ResNet block: an identity shortcut bypasses two parameter weight layers and their intermediate ReLU to merge at the addition junction before the final activation.

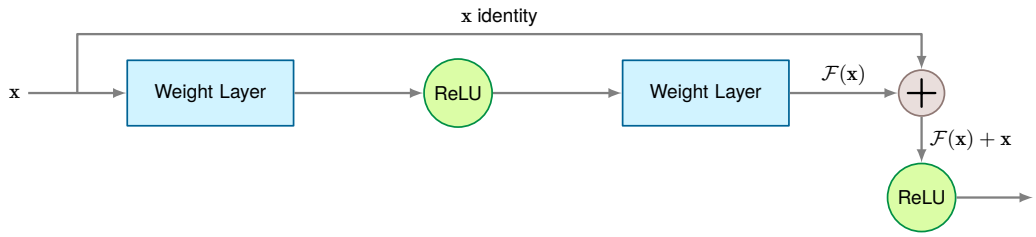


Figure 6.13: Residual Connection Block (Original Post-Activation Form): The original ResNet block (He et al. 2016a) implements $\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}) + \mathbf{x})$, where an identity shortcut bypasses the weight layers before a final ReLU activation. In later pre-activation variants (He et al. 2016b) and modern transformer blocks, the nonlinearity moves inside $\mathcal{F}(\mathbf{x})$, making the addition a completely clean identity shortcut $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ that matches the unobstructed gradient flow in equation 6.9.

During backpropagation, the gradient flows through this addition, and applying the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \left(\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + \mathbf{I} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathcal{F}'(\mathbf{x}) + \frac{\partial \mathcal{L}}{\partial \mathbf{y}}$$

Here $\mathbf{I}$ is the identity matrix, and the gradient decomposes into two additive terms. Even if the residual term $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \mathcal{F}'(\mathbf{x})$ attenuates, the identity term passes $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ directly to the earlier layer. While the residual branch can partially interfere with the identity signal, keeping $\|\mathcal{F}'(\mathbf{x})\|$ small keeps the block Jacobian close to identity and bounds its condition number. Through N_L residual blocks, the gradient becomes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(N_L)}} \cdot \prod_{\ell=1}^{N_L} \left(\mathcal{F}'_\ell(\mathbf{x}^{(\ell-1)}) + \mathbf{I} \right)$$

Each factor $(\mathcal{F}'_\ell + \mathbf{I})$ contains the identity term, keeping the factor near identity when the residual-branch Jacobian is small. A plain network instead multiplies arbitrary layer Jacobians, and if their average gain is even slightly below one the product vanishes exponentially ($0.9^{50} \approx 0.005$). ResNets multiply near-identity factors. The deviations can still compound, and the additive path does not

guarantee exact unit gain across arbitrary depth, but this structure improves conditioning enough to make networks with 100+ layers trainable. The CIFAR-10 experiments confirm it: a 56-layer residual network reached about 7.0 percent test error against 8.8 percent for its 20-layer residual counterpart, reversing the degradation seen in plain networks (He et al. 2016a). There is no single depth at which skip connections suddenly become mandatory; trainability also depends on initialization, normalization, optimizer dynamics, and block architecture (He et al. 2016b).

Residual connections stabilize the backward gradient path, but they do not constrain the forward scale of intermediate activations. Unbounded variance across depth can still destabilize layer inputs, which calls for a complementary block: normalization.

6.10.2.3 Normalization: Stabilizing activations at depth

Normalization functions as the forward-pass partner to skip connections: where residual paths preserve backward gradient flow, normalization stabilizes the distribution of intermediate activations. Batch normalization in CNNs[25] (Ioffe and Szegedy 2015), layer normalization for sequence models, and RMSNorm[26] for large language models illustrate different choices of reduction axes and statistics.

Batch normalization standardizes activations using statistics computed across the mini-batch for each feature or channel during training. For fully connected activations, the reduction axis is the batch dimension. For convolutional activations, implementations compute per-channel statistics over both the batch and spatial positions ($B \times H_{\text{img}} \times W_{\text{img}}$). For a mini-batch $\mathcal{B} = \{a_1, \ldots, a_B\}$ of activations at a particular layer, the transformation proceeds in two stages.

First, compute the batch statistics:

$$\mu_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B} a_i \qquad \sigma^2_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B}(a_i - \mu_{\mathcal{B}})^2$$

Then, over the batch, normalize and apply learnable scale and shift. The normalization step in equation 6.10 centers and scales activations, while equation 6.11 lets the network learn a useful output scale and offset:

$$\hat{a}_i = \frac{a_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{6.10}$$

$$y_i = \gamma \hat{a}_i + \beta \tag{6.11}$$

The parameters γ (scale) and β (shift) are learned during training, while ϵ (typically 10^{-5}) prevents division by zero. The affine transform restores learned scale and shift flexibility, but fixed γ and β cannot exactly invert varying statistics for every batch. Normalization permits larger learning rates by preventing intermediate activations from exploding into saturating non-linear regimes, accelerating convergence relative to unnormalized networks.

While batch normalization enabled training of much deeper CNNs, it introduces severe distributed systems and sequence modeling constraints. When a mini-batch is split across several accelerators, computing true mini-batch statistics requires an all-reduce, a collective operation that sums each accelerator's partial statistics and returns the total to all of them over PCIe or NVLink, adding communication latency to every normalized layer. If instead computed locally on small per-accelerator batches, noisy empirical statistics destabilize optimization. Furthermore, batch normalization cannot handle variable-length sequences or autoregressive token-by-token generation without complex padding masks and training-serving skew. Layer normalization resolves these limitations by normalizing across the feature dimension rather than across the batch (Ba et al. 2016).

For an input vector $\mathbf{x} \in \mathbb{R}^d$ with d features:

$$\mu_{\text{LN}} = \frac{1}{d}\sum_{i=1}^{d} x_i \qquad \sigma^2_{\text{LN}} = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu_{\text{LN}})^2$$

Equation 6.12 defines the complete layer normalization operation, where $\odot$ denotes element-wise multiplication:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu_{\text{LN}}}{\sqrt{\sigma^2_{\text{LN}} + \epsilon}} \odot \gamma + \beta \tag{6.12}$$

[25] **Batch normalization (BatchNorm)**: The original normalization layer (Ioffe and Szegedy 2015), which re-scales activations using per-mini-batch statistics. In one ImageNet experiment, it reached the target accuracy with 14× fewer training steps. Its batch-size dependency and training-serving skew (switching from batch statistics to running averages at inference) are systems limitations that motivated alternatives: LayerNorm removed batch dependency for transformers, and RMSNorm removed mean centering.

[26] **RMSNorm (root mean square normalization)**: Introduced by Zhang and Sennrich (2019) at NeurIPS, RMSNorm simplifies LayerNorm by normalizing with the root mean square alone, dropping the mean-centering step. Across the paper's tested models and implementations, RMSNorm reduced overall running time by 7–64 percent relative to LayerNorm. LLaMA-family and Mixtral-style transformer reports use RMSNorm (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023; Jiang et al. 2024), illustrating why a simpler normalization kernel matters at large language model scale.

Because each token or sample is normalized independently, the computation stays within an accelerator's on-chip SRAM and registers, with no cross-sample or cross-device communication. That independence makes layer normalization the standard choice for transformers, where sequences vary in length and autoregressive generation evaluates a single token at a time. Table 6.10 contrasts the three variants along the machine and operational dimensions that decide between them.

Table 6.10: Normalization Variant Comparison: Different normalization techniques trade off between computational efficiency, batch size sensitivity, and architectural compatibility. RMSNorm (Zhang and Sennrich 2019), used in LLaMA-family and other efficient transformer architectures (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023), omits mean centering: $\text{RMSNorm}(\mathbf{x}) = \mathbf{x}/\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon} \odot \gamma$.

Characteristic	BatchNorm	LayerNorm	RMSNorm
Normalization Axis	Batch and, for CNNs, spatial positions	Feature dimension	Feature dimension
Batch Size Dependency	High (noisy for small batches)	None	None
Typical Use Case	CNNs, vision models	Transformers, RNNs	LLaMA, efficient transformers
Computation Cost	Higher (mean + variance)	Higher (mean + variance)	Lower (RMS only)
Training/Inference	Different (running stats)	Identical	Identical

↪ Kernel fusion on the accelerator, which collapses memory-bound normalization and activation chains into single device kernels, is developed in section 11.8.1.3.

The computational profile of normalization is defined by its arithmetic intensity. Because normalization applies elementwise operations to large activation tensors, execution is memory-bandwidth bound rather than compute bound. A straightforward LayerNorm kernel makes two reductions across the hidden dimension d, one for the mean μ_{LN} and one for the variance σ^2_{LN}, although Welford's algorithm can compute both in a single pass. RMSNorm drops the mean-centering reduction, leaving a single sum of squares, which simplifies the kernel and eases fusion with neighboring operations; its authors report 7–64 percent lower running time across the models they tested.

The table's last row is an operational cost. BatchNorm keeps nonlearned running mean and variance buffers and switches from mini-batch statistics in training to those running statistics at inference, so a mismatch between the buffers and deployment data is a frequent source of training-serving skew. LayerNorm and RMSNorm compute identically in both modes and carry no running state.

6.10.2.4 Gating: Controlling information flow

Skip connections and normalization stabilize depth, the first by preserving gradient flow and the second by bounding activation variance. Gating addresses a different problem: routing information selectively according to the input. Recurrent networks provide a useful example, because simple recurrent loops suffer gradient degradation across long sequences. LSTMs[27] (Hochreiter and Schmidhuber 1997) and GRUs[28] (Cho et al. 2014) address this through **gates**: parameter-weighted transformations that act as differentiable valves to protect, update, or discard stored state.

Gating generalizes beyond recurrence as the basic mechanism for data-dependent routing. Highway Networks used gates to interpolate between a transformation and an identity bypass, directly inspiring residual shortcuts. Encoder-decoder attention (Bahdanau et al. 2015) computes a dynamic routing distribution over input representations, and in transformers the softmax attention weights act as data-dependent routing coefficients that decide how features aggregate across tokens. Mixture-of-experts architectures extend the idea to dispatch each token to a subset of feedforward experts. The hardware cost of gating is irregularity. Dense attention keeps static tensor shapes even though its weights vary per token, but sparse attention, expert routing, and dynamic graph execution introduce data-dependent control flow and runtime tensor sizes that defeat static memory allocation and balanced work dispatch.

A transformer block is where all four building blocks meet. In the encoder tower diagrammed in figure 6.10, residual paths bypass each multi-head attention and feed-forward sub-layer to preserve gradient flow through deep stacks, dense GEMM projections carry the attention and feedforward transformations, LayerNorm or RMSNorm stabilizes activations around each sub-layer, and softmax attention weights gate information across positions. What the transformer changed was the dependency structure. Replacing recurrence with all-to-all attention cut the sequential depth for tokens

[27] **LSTM (long short-term memory)**: Invented by Hochreiter and Schmidhuber in 1997, LSTMs introduced a "Constant Error Carousel," a gated cell state that protects error signals from exponential decay during backpropagation through time. The systems cost of this solution: a standard LSTM computes input, forget, and output gates plus a candidate cell update, giving roughly four affine transformations per time step vs. one in a vanilla RNN. This compute overhead explains why transformers, which solve long-range dependencies through parallelizable attention, replaced LSTMs in many large-scale language workloads.

[28] **GRU (gated recurrent unit)**: Cho et al. (2014) describes a gated hidden unit for encoder-decoder translation that uses reset and update gates to control how the hidden state is updated. Relative to an LSTM's input, forget, and output gates plus candidate cell update, this gives a simpler gated recurrence. The broader systems lesson: architectural simplification can reduce state and matrix operations when it preserves task performance, a principle that recurs in efficiency-oriented designs from MobileNet to distilled transformers.

↪ The irregular memory access that unstructured sparsity, embedding lookups, and dynamic routing impose on hardware is developed in section 11.5.3.1.

to interact from $\mathcal{O}(S)$ steps to $\mathcal{O}(1)$, which let the other three blocks run at full width on parallel hardware. Vision Transformers[29] carry the same block set to images (Dosovitskiy et al. 2021).

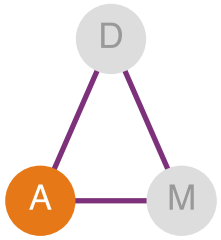
This portability separates optimizations that transfer from those that do not. GEMM tiling and mixed-precision arithmetic help every row of table 6.8, residual memory management helps every deep network, and normalization kernel fusion helps CNNs and transformers alike. Attention-specific kernels remain tied to attention's memory pattern, but even they are built from the same GEMM and memory-access primitives. The profile and the building blocks describe what each family costs; choosing among families for a particular deployment requires turning those costs into a procedure.

[29] **Vision transformers (ViTs)**: Google's 2020 ViT paper split 224×224 images into 16×16 patches (196 "tokens") and applied standard transformer attention. ViTs replace CNN's local convolutions with $\mathcal{O}(S^2)$ global attention over patch tokens. In the original study, large-scale pretraining improved ViT's competitiveness, illustrating how data and compute can compensate for weaker spatial inductive bias (Dosovitskiy et al. 2021).

6.10.3 Decision framework

A wildlife monitoring sensor must classify camera-trap images within a watt-scale solar power budget, while an industrial recommendation service must retrieve terabyte-scale embeddings within a tail-latency budget of tens of milliseconds. Each constraint eliminates architectures that look competitive on unconstrained benchmarks, and a structural mismatch at this stage cannot be rescued later by compiler optimization or kernel tuning. Engineering teams nonetheless often default to architectures by popularity, deploying over-parameterized transformers where a lightweight CNN would suffice or selecting models that violate the inference latency budget. In D·A·M terms, architecture selection chooses the algorithm (A) that bridges the structure of the data (D) to the budgets of the machine (M): memory capacity, memory bandwidth, compute throughput, and thermal envelope.

Selection begins with data-to-architecture mapping, matching the symmetries of the input to an operator family. The useful strength of an inductive bias depends on the task (section 6.1). CNNs impose local connectivity, weight sharing, and translation equivariance, reducing parameter volume and exposing reuse for grid-like data. RNNs impose temporal recurrence, bounding inference state across sequence lengths at the price of serialization. Transformers use content-based attention, gaining flexible interactions while paying for dense quadratic score computation. MLPs impose no specific spatial or temporal connectivity pattern, leaving those relationships to be learned. A matching prior can reduce the samples and computation needed for a target accuracy; a mismatched prior can exclude useful solutions. Model selection must test both predictive fit and the resulting machine costs, and the history of the Netflix Prize shows what happens when only the first test is run.



Architecture selection turns on the algorithm axis, bridging data to machine.

> Example 6.5: The ensemble Netflix did not ship (2009)
>
> **Context**: Netflix awarded its $1M Grand Prize in 2009 for a 10 percent improvement in recommendation accuracy (Johnston 2012).
>
> **Mechanism**: Bringing the winning improvement into production required substantial engineering effort just as Netflix's business was shifting from mailed DVDs to streaming. That shift also produced richer viewing signals and changed what personalization needed to optimize.
>
> **Impact**: The additional accuracy did not justify the production effort, so Netflix did not deploy the Grand Prize code.
>
> **Fix**: Netflix retained two algorithms from the earlier Progress Prize and redirected its recommendation work toward streaming-era personalization.
>
> **Systems lesson**: An offline metric gain is only valuable when its integration cost and objective still match the production system.

Figure 6.14 formalizes this selection pipeline as a multi-stage triage tree: candidate architectures route first by input data structure (text, sequences, spatial grids, or tabular features) and then pass through sequential screening gates for memory capacity, compute budget, inference latency, accuracy targets, and hardware deployment readiness. High-cardinality recommendation workloads sit outside this dense flowchart and route to the sparse embedding family (section 6.9) before applying the same memory, compute, speed, accuracy, and deployment checks. If any constraint fails, the

decision loop routes execution back to scale down model capacity or substitute an alternative architecture family.

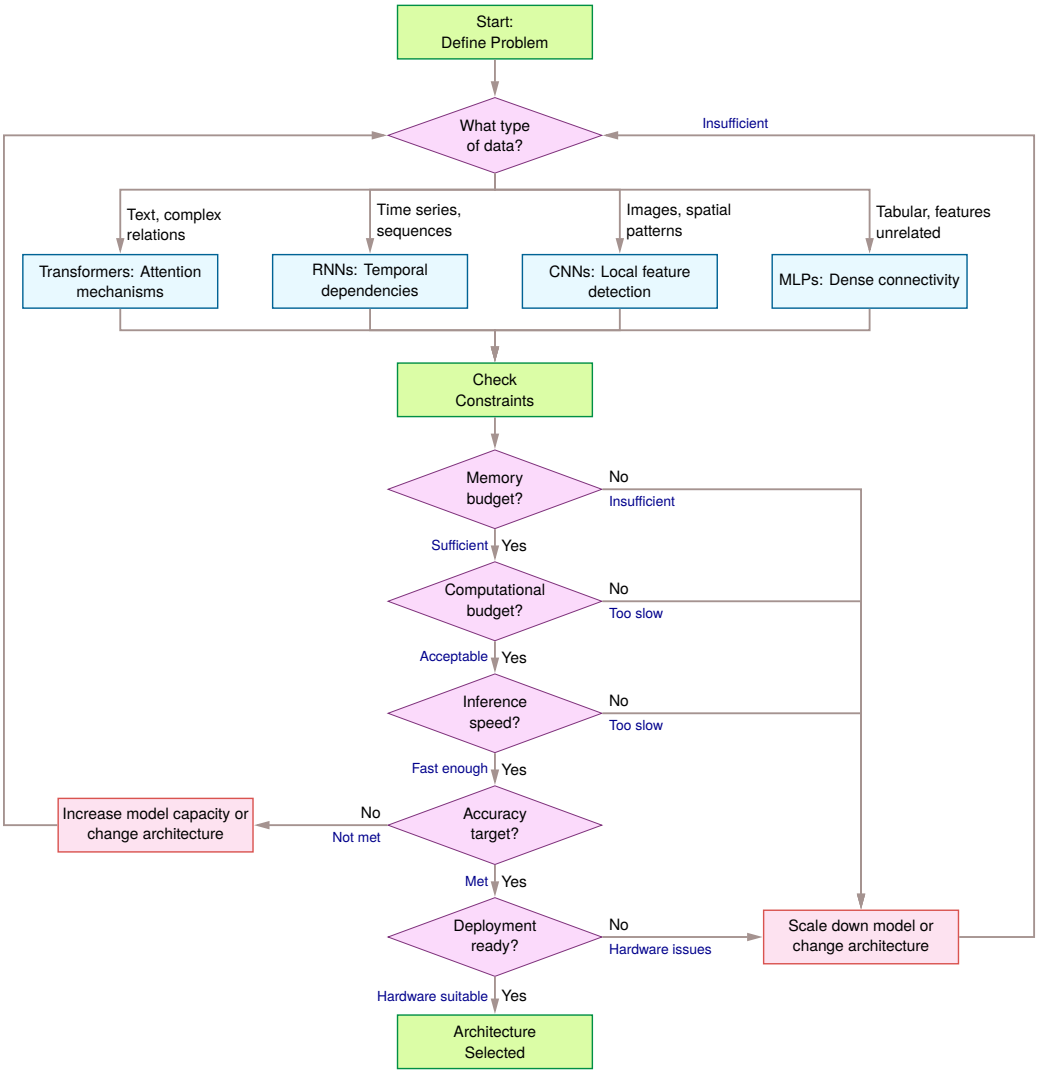


Figure 6.14: Architecture Selection Decision Framework: A systematic flowchart for choosing dense neural network architectures based on data characteristics and deployment constraints. The process begins with data type identification (text/sequences/images/tabular) to select initial architecture candidates (Transformers/RNNs/CNNs/MLPs), then iteratively evaluates memory budget, computational cost, inference speed, accuracy targets, and hardware compatibility. High-cardinality recommendation workloads are handled separately by the sparse-embedding family (DLRM) in the Sparse Architectures section, then re-enter the same constraint checks.

The framework executes in three ordered phases, with model compression as the systematic way to scale a candidate down when it fails a budget:

1. **Data symmetry identification (D)**: Identify structural symmetries and invariances in the input data. Spatial locality routes candidates to CNNs; temporal sequence structure to RNNs; unconstrained cross-token dependency to transformers; tabular features without spatial geometry to MLPs; and high-cardinality categorical IDs to sparse embedding tables.
2. **Progressive physical constraint filtering (A $\times$ M)**: Filter candidate models against hardware budgets: static parameter storage in flash/DRAM, peak dynamic activation footprint in SRAM/HBM, arithmetic intensity against the hardware roofline, and tail latency dead-

lines. A violation requires scaling down model width or depth, applying quantization and compression, or switching to a lower-overhead architecture family.

3. **Empirical capacity and deployment validation**: Verify that candidate capacity meets target accuracy on representative evaluation distributions. If predictive accuracy falls short, scale model capacity and re-evaluate physical hardware constraints. If target accelerator hardware cannot sustain the required arithmetic throughput or memory bandwidth, select an alternative architecture rather than relying on marginal hyperparameter adjustments.

Kernel-level choices refine this decision. Im2col or direct convolution, fused normalization, and IO-aware tiled attention (section 8.7.4) change realized traffic and utilization, and can shift which iron-law term binds; chapter 7 and chapter 11 develop them. Architectural dependencies still matter. Batching independent recurrent sequences or fusing their operations reduces per-step cost, while the ordered state transitions remain. Candidate selection and kernel measurements therefore inform each other.

6.10.4 Architecture selection in practice

Every selection must first pass a feasibility check: whether the target hardware can sustain the required frame rate at all. A server-class vision pipeline on a mid-range GPU shows the throughput ceiling calculation in its simplest form. The wildlife monitoring deployment that follows applies the same checks under a watt-scale budget where GPUs are ruled out, which is where data structure, model capacity, and hardware budgets interact.

Napkin Math 6.3: The throughput ceiling

Problem: A real-time application must process 30 FPS of video with ResNet-50. What sustained compute throughput is required?

Math:

1. **Model cost**: ResNet-50 requires ~8.2 GFLOP per 224×224 image.
2. **Frame rate**: 30 FPS required.
3. **Sustained throughput**: 30 FPS × 8.2 GFLOP = 246 GFLOP/s.
4. **Effective throughput**: 10 TFLOP/s peak × 50–60 percent utilization = 5 TFLOP/s–6 TFLOP/s.
5. **ResNet-50 headroom**: 5 TFLOP/s ÷ 246 GFLOP/s = 20.3×.
6. **Detection-model check**: 30 FPS × 100 GFLOP = 3 TFLOP/s, leaving 1.7× headroom at the lower effective-throughput estimate.

Systems insight: A mid-range GPU delivering 10 TFLOP/s theoretical peak achieves ~50–60 percent utilization in this planning scenario, yielding 5 TFLOP/s–6 TFLOP/s effective. For ResNet-50 at 30 FPS, the system has 20.3× headroom. Switching to an object detection model at 100 GFLOP per frame, however, requires 3 TFLOP/s sustained, leaving only 1.7× headroom. Batch size constraints or multi-stream processing quickly push the system toward the compute ceiling. ResNet-50 is compute-bound, but the margin depends on the accelerator and utilization achieved.

The throughput ceiling converts an abstract compute requirement into a concrete hardware utilization percentage. A real-world deployment adds physical constraints the ceiling alone does not capture.

6.10.4.1 Worked example: Real-time wildlife monitoring

The design task is an embedded ML system that classifies wildlife species from camera trap images in an isolated national park. The deployment must execute inference entirely on-device with zero network connectivity, operate on solar-recharged battery power for six months, and achieve 90 percent accuracy across 50 target animal species. The walk-through applies the three-phase framework in five steps. Data characterization is the symmetry phase; deployment constraint

profiling, candidate architecture evaluation, and hardware budget checks carry out the constraint-filtering phase; and operational risk assessment sets up the validation phase that only target-device measurement can complete.

The first step characterizes the data. The input consists of camera trap image frames (1920×1080 raw sensor resolution, downsampled to 224×224 for inference). Three properties favor a CNN candidate: spatial locality (fur textures or ear shapes occupy bounded neighborhoods), label invariance to translation (moving an animal within the frame should preserve its species label), and hierarchical composition (edges combine into textures, anatomical parts, and organisms). Convolution provides equivariant intermediate features that follow the animal's location; pooling can then support an invariant classification. Local weight sharing supplies a compact feature detector for these images.

The second step, constraint profiling, translates the deployment environment into hard hardware boundaries. Table 6.11 catalogs these physical constraints alongside the architectural requirements each imposes:

Table 6.11: Wildlife Monitoring Constraints: Five deployment constraints and their implications for the camera-trap scenario. No connectivity rules out cloud-only inference, while the power, latency, memory, and accuracy requirements must be validated on the target device.

Constraint	Requirement	Implication
Connectivity	None (offline)	All inference must run on-device
Power	~2 W average (solar + battery)	Rules out GPUs; needs a low-power SoC or edge NPU
Latency	<500 ms per detection	Allows batch size 1, no real-time streaming
Memory	512 MB RAM, 2 GB storage	Model, activations, and runtime buffers must fit locally
Accuracy	90%+ on 50 species	Requires sufficient model capacity

With physical constraints established, the third step evaluates candidate architectures against reference model baselines. The planning scenario reserves 100 MB of the available RAM for model weights, leaving the rest for activations, system software, and runtime buffers:

- **ResNet-50** (25.6M parameters, 8.2 GFLOP): A costly starting point for the 2 W envelope. At 102.4 MB in FP32, weight storage alone exceeds that allocation before activation buffers or OS overhead are counted.
- **MobileNetV1** (4.2M parameters, 1138 MFLOP): A viable footprint and operation count. Its low arithmetic intensity, which can leave a data center GPU's matrix units idle, costs far less on a batch-one edge CPU whose much smaller compute peak is easier to keep busy. Storing weights in INT8 reduces model footprint to 4.2 MB (16.8 MB in FP32), and factorized depthwise separable convolutions drastically cut arithmetic operations.
- **KWS DS-CNN** (200K parameters, 20 MFLOP): An unmatched baseline. This model targets audio keyword spotting, so its reported size and task results do not establish accuracy on visual species classification. An adapted compact vision model would require training and evaluation on the camera-trap task.

While MobileNetV1 demonstrates the viability of depthwise separable convolutions, MobileNetV2 offers inverted residual blocks with linear bottlenecks as another compact candidate. Sizing the model with width multiplier 0.75× yields ~2.6M parameters (10.4 MB FP32, 2.6 MB with INT8 weights) and costs ~418 MFLOP at 224×224 input resolution. Its weight footprint fits the allocation; task evaluation must establish whether its capacity meets the accuracy target for the 50 species.

The fourth step, hardware sizing, checks the candidate against physical platform budgets. INT8 weights occupy 2.6 MB, one $224\times224\times64$ FP32 feature map occupies 12.8 MB, and the scenario reserves 50 MB for OS and runtime buffers. Their 65.4 MB subtotal is only a single-feature-map screen. Peak memory must instead include the weights, the maximum sum of simultaneously live tensor bytes, kernel workspace, and OS and runtime buffers. Inputs and outputs can coexist, and a residual path retains its input while the branch executes. The scenario has not yet established that live-tensor peak or the required workspace, so the small subtotal supports candidacy without confirming a fit in 512 MB of RAM.

For the candidate system-on-chip (SoC), an ARM Cortex-A53 at 1.2 GHz with NEON SIMD, this planning calculation assumes sustained throughput of ~2 GOPS INT8: $\frac{418\times10^{6}\ \text{INT8 ops}}{0.002\times10^{12}\ \text{INT8 ops/s}} = 209ms$ latency $\ll 500ms$ target ✓

Active power represents the final physical gate. At an estimated active inference draw of ~200 mW over 209 ms, each classification consumes 41.8 mJ. For a trigger-based rate of 100 inferences/day, model execution consumes 4.2 J/day. Full-system energy must also include image capture, sleep current, storage, and peripherals. Battery capacity, conversion losses, trigger frequency, and seasonal solar yield determine whether that daily budget supports six months of operation.

The fifth step assesses operational risks in the field. Table 6.12 pairs the primary accuracy, thermal, and coverage failure modes with corresponding systems engineering mitigations:

Table 6.12: Wildlife Camera Deployment Risks: Top accuracy, thermal, and species-coverage risks for the MobileNetV2 deployment, paired with the engineering mitigation chosen for each.

Risk	Mitigation
90% accuracy not achieved	Train on augmented dataset; consider EfficientNet-Lite if MobileNet insufficient
Thermal throttling in enclosure	Add passive heatsink; reduce inference frequency in high-temperature conditions
New species added postdeployment	Expand or retrain the output head; install the update during a physical service visit

The sizing exercise selects MobileNetV2 (0.75× width multiplier) quantized to INT8 as a candidate for an ARM Cortex-A53 SoC with 512 MB RAM. Under the assumed sustained throughput, the arithmetic estimate is approximately 209 ms per frame. Qualification now requires target-device measurements of end-to-end latency, peak memory, thermal behavior, and full-system energy, together with classification accuracy on representative camera distributions and seasonal battery tests.

This case study translates abstract architectural principles into a concrete sizing decision. Yet production architecture selection frequently encounters counterintuitive trade-offs: models with fewer FLOPs can stall on bandwidth-limited hardware, expressive architectures can degrade on datasets lacking matched structure, and lab prototypes can fail against deployment memory hierarchies. Grounding these failure modes in hardware reality exposes the principal fallacies and pitfalls of systems design.

6.11 Fallacies and Pitfalls

Architecture choice is a systems decision, not a leaderboard selection. The common mistakes in architecture selection arise when teams treat architecture families as interchangeable accuracy tools while ignoring inductive bias, memory traffic, hardware mapping, and deployment state.

Fallacy: *More complex architectures always perform better than simpler ones.*

Engineers often assume that transformers outperform simpler architectures on all tasks. In production, architectural sophistication must match problem complexity: the algorithm must fit both the structure of the data and the cost of the machine. Vision transformers matched CNN accuracy on ImageNet only after pretraining on far larger datasets (Dosovitskiy et al. 2021); trained on ImageNet alone, the CNN's locality bias wins at lower cost. The bitter lesson favors weak priors only once data and compute are large enough to pay for them, and a deployment that cannot pay that price gets the cost of the weak prior without its benefit. Accuracy, training cost, and inference latency still require measurement for the specific implementation and task.

Pitfall: *Selecting architectures based solely on accuracy metrics without analyzing computational requirements.*

Practitioners choose architectures from papers reporting top-line accuracy, ignoring computational implications. RNNs expose a sequential critical path that limits parallelism, while transformers face quadratic dense-score computation and, in naive implementations, quadratic score storage. Sequence length 2,048 therefore requires 16× more materialized score memory than length 512. Systems that ignore these characteristics can miss latency targets, exceed memory budgets, and deliver much lower hardware utilization than expected.

Fallacy: *An architecture has one dominant bottleneck across training and inference.*

The same computation graph can occupy different systems regimes. Transformer training and prefill process many positions together and may bind on dense attention compute; small-batch autoregressive decoding repeatedly streams weights and KV-cache state and may bind on memory bandwidth. Batching, sequence length, precision, and cache layout can shift that boundary. A benchmark from one regime therefore cannot establish another's bottleneck; measure each against the iron law.

Pitfall: *Combining architectural patterns without analyzing interaction effects at the system level.*

Engineers add attention to CNNs or convolutions to transformers expecting additive benefits. Each pattern creates distinct memory access characteristics: CNNs exploit spatial locality through sliding windows, while dense attention introduces all-to-all score computation. Combining them can increase intermediate traffic and disrupt locality, so the hybrid's throughput cannot be inferred by adding the components' benefits. Adding recurrent connections to transformers likewise reintroduces sequential dependencies. Successful hybrids require profiling memory access and cache behavior before combining patterns.

Fallacy: *Architecture wins on training hardware transfer directly to deployment hardware.*

Teams design for high-end GPU clusters, then discover deployment failures on target hardware. An architecture exploiting 8× A100 GPUs (640 GB total memory) cannot deploy unchanged to a representative edge node such as the NVIDIA Jetson Orin NX (16 GB system memory). As section 6.10.3 emphasizes, architecture selection must analyze the full system stack because storage, latency, and power budgets vary by deployment target.

Pitfall: *Ignoring KV cache growth when estimating transformer serving costs.*

Teams budget transformer deployment based on model weight memory alone, overlooking the key-value (KV) cache used during autoregressive generation (Pope et al. 2023; Kwon et al. 2023). The cache scales as $\mathcal{O}(B \times N_L \times 2 \times N_{\text{heads}} \times S \times d_{\text{head}})$, where B is the number of concurrent sequences, N_L is the layer count, the factor 2 stores keys and values, N_{heads} is the head count, S is sequence length, and d_{head} is head dimension. At long contexts or high concurrency, this overhead can become a binding serving constraint. Section 6.8.4.2 works through the chapter's 32-layer, 32-head example step by step. With 128-dimensional heads, 2,048-token sequences, and FP16 storage, each concurrent request holds ≈ 1 GB of KV cache. At 2–4 users, the cache alone consumes 2 GB–4 GB before allocator, activation, and workspace overhead. Those values do not exhaust a 80 GB device, but they consume part of the post-weight headroom and grow linearly with context and concurrency. Capacity planning must therefore budget weights, cache, and runtime buffers together, then reduce concurrency, context, or cache footprint, or add memory capacity, when the combined total approaches device memory.

6.12 Summary

Architecture is infrastructure. Choosing among MLPs, CNNs, RNNs, transformers, and recommendation models (DLRM) shapes the physical terms in the serialized form of the iron law of ML systems ($T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$). Examining each architecture through the four-part lens (pattern processing needs, algorithmic structure, computational mapping, and system implications) connects mathematical inductive bias to work, state, and dependencies. Execution conditions and the target machine then determine which term binds hardware performance.

The durable tool is arithmetic intensity read against the machine's balance. Reuse raises $I = O/D_{\text{vol}}$ and pushes a workload toward the compute term, as ResNet-50 does when batched; streamed weights lower it and expose the data term, as GPT-2 decoding does at small batch. Two families escape that single axis. A recurrent dependency chain sets a floor of S ordered updates that neither term removes, and an oversized embedding table imposes a capacity decision that fixes which bandwidth the data term pays before any arithmetic begins. Low FLOP counts, as MobileNetV2 and KWS show, help only when the resulting operation shapes still keep the hardware's functional units busy.

Key Takeaways: Architecture is infrastructure

- **Inductive bias is the unifying concept**: Every architecture encodes structural assumptions, such as locality for CNNs, sequence for RNNs, and global context for transformers. These biases trade generality for sample efficiency and determine which problems an architecture can solve efficiently.
- **Arithmetic intensity helps identify the bottleneck**: Comparing a workload's arithmetic intensity with the target hardware's roofline balance helps determine whether compute or memory bandwidth is likely to bind.
- **Quadratic costs are permanent constraints**: Dense transformer attention computes $\mathcal{O}(S^2)$ score interactions. Naive implementations also use quadratic score storage; tiled exact attention removes that storage cost but not the dense computation.
- **Lighthouse models isolate distinct bottlenecks**: ResNet-50 (compute), GPT-2 (bandwidth), DLRM (capacity), MobileNetV2 (edge latency), KWS (always-on power envelope). These archetypes diagnose which physical constraint dominates a given system.
- **Depth benefits from architectural support**: Skip connections and normalization improve optimization in deep networks rather than acting as universal prerequisites beyond a fixed depth. These building blocks, illustrated here through CNNs, transfer to many deep architectures, including transformers.
- **FLOPs do not equal speed**: MobileNetV2 uses 13.7× fewer FLOPs than ResNet-50 but may run slower on some data center GPUs when its operation shapes and arithmetic intensity use the available compute units poorly. Architecture-hardware alignment, not operation count alone, determines throughput.
- **Architecture selection is deployment selection**: Choosing a transformer over a CNN determines memory requirements, latency floors, hardware utilization, and infrastructure costs. The architecture is the system constraint.

Inductive bias and systems cost are two views of the same choice. The assumptions that help a model generalize from limited samples, such as locality, recurrence, or unrestricted token interactions, also determine the shape, reuse, and lifetime of intermediate data. An algorithmic choice therefore propagates into memory capacity, bandwidth, and scheduling requirements before a framework chooses kernels.

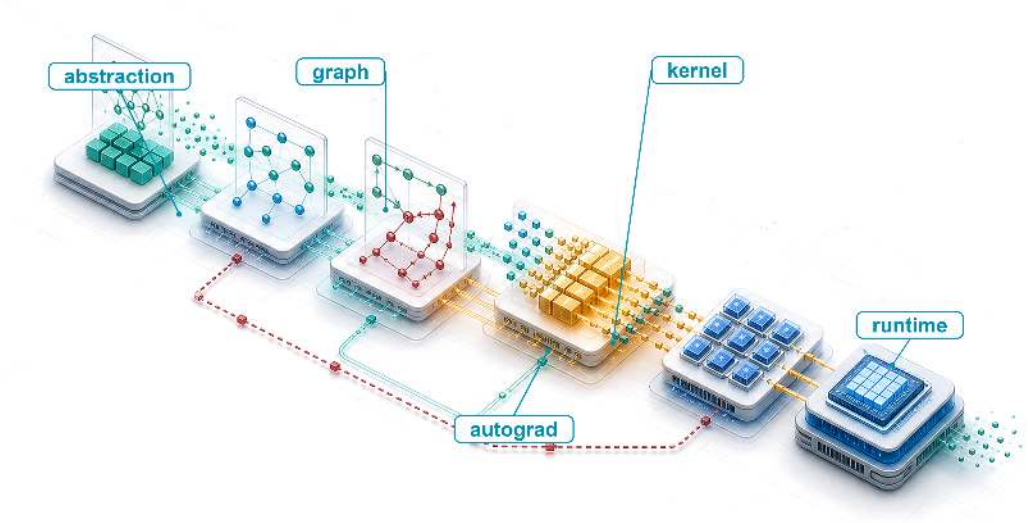
Architecture selection signs the silicon contract (principle 4) with hardware. A CNN commits to spatial locality and weight reuse; dense attention commits to $\mathcal{O}(S^2)$ score computation; and an RNN commits to serial time-step dependencies. These topological choices constrain operations O, opportunities for reducing transferred data D_{vol}, and dependency order. Realized memory footprint, bandwidth demand, and parallel efficiency also depend on the implementation. Tiled exact attention removes intermediate score storage without changing the quadratic arithmetic floor, while pipelining consecutive stages across execution units overlaps work without removing the serial time-step path. Systems engineers can therefore anticipate likely bottleneck shifts before implementation by matching model inductive bias and data geometry to silicon physics.

What's Next: From blueprints to construction

Raw computational primitives (chapter 5) became concrete architectural contracts in this chapter. Each contract fixes *what* work must be performed, yet nothing in an architecture says when its operations run, how their gradients are computed, or which device executes them. A machine learning framework answers those three questions, acting as a compiler that lowers the architectural graph to the silicon. Chapter 7 examines how PyTorch, TensorFlow, and JAX solve these execution, differentiation, and hardware-abstraction problems, turning architectural graphs into computational graphs, autograd records, and optimized device kernels.

7

ML Frameworks

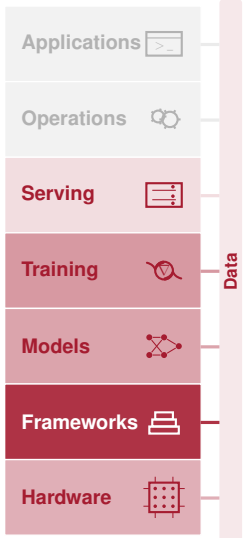


7.1 Three Framework Problems
7.2 The Ladder of Abstraction
7.3 Execution Problem
7.4 Differentiation Problem
7.5 Abstraction Problem
7.6 Model Composition and State
7.7 Framework Platform Analysis
7.8 Deployment Targets
7.9 Framework Selection
7.10 Anatomy of a Training Step
7.11 Fallacies and Pitfalls
7.12 Summary

Purpose

Why does the framework silently constrain every decision that comes after it?

Model equations do not execute themselves. A framework translates an architecture into work that hardware can run, and that translation is not neutral. It decides when operations execute, what intermediate state training retains, how memory is allocated, and which deployment targets can use the resulting artifact. These choices shape more than speed. They also determine what can be debugged directly, what the compiler can optimize, and whether the same model survives the path from a research environment to a production runtime. The same model expressed through two execution paths can differ in speed or memory use by several times, a gap that no amount of model tuning recovers when the framework hides the structure a compiler needs. The abstractions that make experimentation convenient therefore create long-lived commitments. Once a project accumulates checkpoints, modules, data loaders, serving formats, and team expertise around a framework, changing it becomes a system migration rather than a library substitution, measured in engineer-months rather than afternoons. A framework well suited to rapid exploration can expose too little structure for efficient deployment, while a constrained execution path can sacrifice the flexibility needed during discovery. Because execution, differentiation, and abstraction meet in the same software layer, a local convenience can become a downstream performance or portability constraint. The engineering question is not which framework is universally best, but which trade-offs the workload, team, and target machine can afford to make fixed. Understanding frameworks is therefore about reasoning through those system choices rather than learning transient API syntax, which changes with each release while the underlying trade-offs persist. In D·A·M terms, the framework mediates algorithm-machine co-design by turning algorithmic intent and data representation into executable work under machine constraints.



Learning Objectives

- Explain how frameworks mediate algorithm-machine co-design through execution, differentiation, and hardware abstraction
- Compare execution strategies using dispatch overhead, compilation cost, and deployment constraints
- Analyze backward saved-state requirements and explain the storage and recomputation consequences of a supplied activation-checkpointing policy
- Implement module abstraction patterns for parameter discovery, mode behavior, serialization, and hooks
- Calculate training-step FLOPs, memory traffic, and dispatch overhead to identify fusion and layout bottlenecks
- Select TensorFlow, PyTorch, JAX, or edge runtimes based on model, hardware, team, and deployment requirements

7.1 Three Framework Problems

Chapter 6 ended by treating architecture selection as a contract with the hardware. A convolutional neural network (CNN) commits the system to spatial locality and weight reuse, and a transformer commits it to attention, matrix multiplications, activation state, and memory traffic. Those commitments fix *what* work exists. The framework is the layer that decides *how* that work reaches the machine. A few calls such as `logits = model(tokens)`, `loss = criterion(logits, targets)`, and `loss.backward()` hide billions of floating-point operations across memory hierarchies, gradients through millions of parameters via automatic differentiation, thousands of GPU kernel launches, and gigabytes of intermediate state. The API looks simple because the framework is acting as a compiler for the silicon contract.

The gap between *what* and *how* is wide. A transformer's attention mechanism requires coordinating computation across memory hierarchies and accelerator cores in patterns where naive implementations can run one to two orders of magnitude slower than optimized ones because of uncoalesced memory accesses (scattered reads the hardware cannot merge into wide memory transactions) and repeated round-trips to high-bandwidth memory (HBM). Implementing these operations from scratch for every model would make deep learning economically infeasible. ML frameworks exist to bridge this gap by lowering abstract model graphs into hardware-aligned execution pipelines that maximize compute unit utilization while minimizing off-chip memory traffic.

A framework is to machine learning what a compiler is to traditional programming. A C compiler translates human-readable code into optimized machine instructions, managing register allocation, instruction scheduling, and memory layout. An ML framework translates high-level model definitions into hardware-specific execution plans, managing kernel fusion, memory reuse, and device placement. The correspondence is operational rather than figurative: modern frameworks incorporate optimizing compilers (such as Accelerated Linear Algebra (XLA) or TorchInductor) that lower graph representations directly to target machine code.

Every ML framework, regardless of API or design philosophy, must solve three core problems. The first is the *execution problem*: deciding when and how computation runs. A framework can execute operations immediately as written (eager execution[1]) or first construct a computational graph[2] (a record of operations and their data dependencies, developed in section 7.3.2) to optimize before launching device kernels (graph execution). This choice determines the fundamental trade-off between interactive debugging and whole-program compiler optimization.

With an execution strategy in place, the framework must solve the *differentiation problem*: computing gradients automatically. Because gradient-based training requires derivatives of a scalar loss with respect to millions or billions of parameters, manual calculus is impossible at scale. Frameworks implement automatic differentiation by registering derivative rules across supported operator compositions, chaining vector-Jacobian products while managing numerical precision and the memory overhead of intermediate activations retained for backward passes.

[1] **Eager execution**: This mode executes each operation immediately, which enables direct debugging with standard tools but sacrifices the global view needed for graph-level optimizations. Without capturing a larger region of computation, an eager runtime cannot fuse across that region or preplan its memory, leaving workload-dependent optimization opportunities for compilers such as `torch.compile`.

[2] **Computational graph**: Optimizing before executing is the design choice that separates graph execution from eager execution. Capturing the program as a data structure lets a framework fuse compatible operations into fewer GPU kernels before execution, reducing launch overhead and intermediate memory traffic. The engineering cost of this visibility is that the executed program differs from the source code, making debugging harder, a trade-off every graph-based framework must justify against the performance gain.

↩ Reverse-mode automatic differentiation and computational graph traversal for gradient computation are established in section 5.5.5.

The third problem is *hardware abstraction*: targeting diverse hardware from a single interface. The same model definition should be expressible across CPUs, GPUs, Tensor Processing Units (TPUs), and mobile accelerators, even though each target presents distinct memory hierarchies, instruction sets, and interconnect bandwidths. Within the GPU domain, framework ecosystems increasingly expose custom-kernel languages such as Triton (Tillet et al. 2019), allowing developers to write high-performance, block-level kernels without hand-crafting low-level CUDA or device assembly.

These three problems are tightly coupled: the execution model dictates when differentiation data structures are recorded, while the hardware abstraction layer determines which graph transformations can be lowered to silicon. Unifying execution, differentiation, and hardware abstraction under physical machine constraints requires evaluating frameworks through the governing equation of machine learning performance.

Systems Perspective 7.1: The ML compiler

In the context of the iron law of ML systems (principle 3), a framework is a compiler for the silicon contract (principle 4).

The "source code" is the model architecture (the O term). The framework's job is to take this high-level mathematical specification and compile it into a series of hardware-specific kernel launches that:

1. Minimize data movement (D_{vol}) through techniques such as kernel fusion.
2. Maximize utilization (η_{hw}) by matching operations to specialized hardware units such as Tensor Cores.
3. Minimize overhead (L_{lat}) through efficient asynchronous dispatch and graph capture.

Choosing a framework therefore also chooses the compiler that determines hardware efficiency. Read against the three framework problems, execution strategy bears most directly on L_{lat}, differentiation on D_{vol} (and on the operation count O), and abstraction on η_{hw}. These are emphases rather than exclusive assignments, since fusion, for example, cuts both launches and traffic, and the chapter uses this one mapping throughout. The formal definition below makes that compiler role precise and separates frameworks from numerical libraries such as NumPy.

Definition 7.1: Machine learning frameworks

Machine learning frameworks are software systems that translate high-level mathematical model definitions into hardware-optimized execution plans by managing the computational graph, automatic differentiation, kernel dispatch, and memory allocation across the hardware hierarchy.

1. **Significance**: Frameworks shape every iron law term that the model itself does not fix: the bytes moved (D_{vol}), the hardware efficiency achieved (η_{hw}), and the fixed overhead paid (L_{lat}). Compiler-backed kernel fusion, for example, can eliminate writes and subsequent reads of intermediate values between compatible operations. Fusing a matrix multiplication, bias add, and rectified linear unit (ReLU) changes how the same mathematics reaches hardware without changing the model.
2. **Distinction**: Unlike a numerical library such as NumPy, which normally evaluates each operation as it is called, an ML framework can capture or defer execution to analyze a larger computational graph and apply cross-operation optimizations: kernel fusion, memory layout transformations, and parallel scheduling. Those optimizations require visibility beyond separately dispatched operations.
3. **Common pitfall**: A frequent misconception is that frameworks are interchangeable API wrappers. Framework choice determines which compiler paths are available. PyTorch can recover graph-level optimization from eager code through `torch.compile()`,

while TensorFlow and JAX commonly rely on XLA-backed compilation paths that lower operations to target hardware. Moving from eager execution to a compiler path can change throughput materially, but the gain depends on model structure, shapes, operator support, and hardware.

An ML framework translates logical intent into physical execution under the constraints of the iron law, deciding how to partition computation across memory hierarchies, when to trade numerical precision for throughput, and how to schedule operations so that whichever term dominates (data movement D_{vol}, arithmetic computation, or launch latency L_{lat}) is minimized. The framework is where the governing physics developed throughout this book becomes executable code.

Automating this machinery—recording operation graphs, allocating gradient buffers, traversing dependencies in reverse, and dispatching hardware kernels—did not happen all at once. Execution, differentiation, and abstraction emerged sequentially as responses to scaling bottlenecks in earlier numerical tools, creating a four-decade historical progression from hand-tuned assembly routines to compiled graph runtimes.

7.2 The Ladder of Abstraction

In 1979, writing a matrix multiplication in Fortran that used the hardware efficiently required deep knowledge of cache lines, register scheduling, and vector units. By 2016, a single line of Python (`torch.matmul(A, W)`) could dispatch to a highly optimized implementation without the programmer knowing the details of the silicon. That compression of effort did not happen in one step; it accumulated across four decades of abstraction, each layer solving a bottleneck that made the previous generation impractical for scaling. The result is a **ladder of abstraction** where each rung automates what the rung below exposed.

The first rung established reusable hardware primitives (1979–1992). The original Basic Linear Algebra Subprograms (BLAS)[3] standardized low-level linear-algebra operations (Lawson et al. 1979), while LAPACK[4] (Bai et al. 2006) built higher-level numerical factorizations and eigensolvers on top of that foundation. Naive nested loops fail to exploit memory hierarchies: without cache tiling and register blocking, memory bandwidth saturates long before arithmetic units reach peak throughput. BLAS and LAPACK solved this hardware-primitive bottleneck by establishing stable interfaces that let numerical software delegate compute-heavy operations such as matrix multiplication (`C = A @ B`)[5] to vendor libraries tuned for specific cache line sizes, single instruction, multiple data (SIMD) widths, and register files, rather than requiring hand-written assembly for every target architecture.

The second rung introduced unified array abstractions to unlock developer velocity (2005–2006). NumPy[6] addressed the productivity barrier of low-level languages by wrapping BLAS and LAPACK in high-level Python (Harris et al. 2020). Under NumPy's strided array model, Python manages control flow and tensor metadata (shape, strides, and data type), while the underlying C runtime dispatches vector and matrix loops across contiguous memory buffers. This vectorization contract—orchestrating high-level algorithmic intent in an interpreted language while delegating inner loops to compiled native routines—became the architectural template for scientific and numerical computing.

The third rung automated gradient computation (2007–present). Deep learning frameworks such as Theano,[7] TensorFlow (Abadi et al. 2016), and PyTorch (Paszke et al. 2019) solved the scalability limits of manual differentiation. While numerical packages required practitioners to derive and implement analytical backpropagation gradients by hand—a process error-prone and unmanageable for deep networks—modern frameworks turned automatic differentiation over computational graphs into a core runtime primitive. By capturing operations in a directed acyclic graph during the forward pass, these frameworks automatically construct the backward graph to evaluate vector-Jacobian products, transforming the calculus of the chain rule into automated reverse-mode execution. That automation let the bitter lesson operate at the pace of research. Once a new architecture no longer needed a hand-derived backward pass, general methods that scale with computation could be tried as quickly as their forward functions could be written.

Figure 7.1 sequences these numerical computing milestones across four decades (1979–2018), charting the shift from assembly-tuned BLAS and LAPACK routines to NumPy vectorization, symbolic

[3] **BLAS (basic linear algebra subprograms)**: The 1979 API specification that forms the bottom rung of the ladder described here; it standardized a fixed set of Fortran-callable vector operations, separating the public routine names from the machine-specific implementation decisions beneath them. Every framework above it inherits the broader version of this bargain: call a standard linear-algebra primitive from any language and let a tuned vendor library target the silicon. For modern general matrix multiply (GEMM) on NVIDIA GPUs, that tuned path is cuBLAS rather than the 1979 BLAS specification itself (NVIDIA 2024a).

[4] **LAPACK (linear algebra package)**: Extends BLAS by providing a standard API for higher-level routines (SVD, eigendecomposition, least-squares) that vendors implement with chip-specific code layered on top of fast GEMM kernels. This layered design is the architectural pattern every ML framework inherits: high-level operations delegate downward to hand-tuned primitives, so a vendor-optimized LAPACK call can execute over 10× faster than a naive implementation without the framework author writing a single line of hardware-specific code.

[5] **GEMM**: The matrix-matrix primitive behind `C = A @ B`. Hardware vendors hand-tune GEMM for their specific chips because dense layers, attention projections, and convolution lowering all rely on matrix multiplication, making this one routine a performance floor for many frameworks above it on the ladder. Its high arithmetic intensity makes GEMM the operation most able to approach peak compute throughput, while small or misaligned shapes often fall back to much lower utilization.

graph capture in Theano and TensorFlow, dynamic dispatch in PyTorch, and functional transformation in JAX.

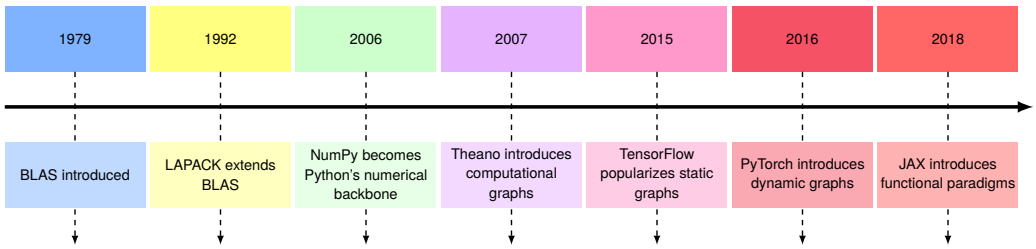


Figure 7.1: Computational Library Evolution: A timeline of numerical computing milestones spanning 1979 to 2018. Each era automates what the layer below exposed—from BLAS and LAPACK hardware routines to NumPy vectorized syntax and graph-based automatic differentiation in modern frameworks—trading low-level transparency for developer velocity.

Each generation abstracted away details that consumed engineering effort in the previous one, yet each abstraction introduced new trade-offs. BLAS hid assembly-level optimization but fixed the interface. NumPy hid memory management but required manual differentiation. Modern frameworks hide gradient computation but introduce a choice among execution models. The deeper pattern is that every abstraction hides details by preserving a contract about shape, dtype, device, and sometimes units; when that contract becomes implicit, correct components can still compose into a wrong system.

War Story 7.1: The interface that forgot its units (1999)

Context: NASA's Mars Climate Orbiter depended on ground software, spacecraft software, navigation teams, and contractor interfaces all agreeing on the physical meaning of exchanged values (Mars Climate Orbiter Mishap Investigation Board 1999).

Mechanism: Ground software supplied by Lockheed Martin computed thruster impulse in English units (pound-force seconds), while JPL navigation software consumed the values assuming SI units (newton-seconds)—a 4.45× implicit unit mismatch.

Impact: On September 23, 1999, the spacecraft was lost after entering occultation. Its corrected 57 km periapsis, rather than the planned 226 km, was judged too low for survival.

Response: The board recommended unit-consistency checks, specification audits of transferred data, and stronger end-to-end verification of ground software.

Systems lesson: Framework abstractions are valuable because they carry contracts: shape, dtype, device placement, and physical units. If those contracts are implicit, two pieces of correct code can still compose into a wrong system. ML frameworks hit this whenever tensor memory layouts (such as channels-first NCHW [Batch × Channels × Height × Width] vs. channels-last NHWC [Batch × Height × Width × Channels]), quantization scale factors, or physical feature units differ across pipeline stages, producing plausible outputs that quietly corrupt downstream inference.

Preventing these silent mismatches requires checking contracts somewhere between writing code and running it. A framework can check them as each operation executes, when the failing line and its intermediate values are directly visible, or before execution, when a captured graph can be validated and optimized as a whole. When those checks and optimizations happen is the execution problem. It is the first framework design choice because runtime dispatch and graph capture determine not only the debugging experience but also what optimizations automatic differentiation and hardware mapping can subsequently exploit.

7.3 Execution Problem

Consider two engineers writing the same neural network. The first debugs interactively, printing tensor shapes after each operation, inspecting intermediate values, and stepping through code with

6 **NumPy (numerical Python)**: In 2005, Travis Oliphant unified two competing Python array libraries (Numeric and Numarray) into a single package, giving the scientific computing community one BLAS-backed array standard at the moment it needed to scale. The "vectorization" contract this created (write logic in Python, execute loops in C/Fortran via BLAS) became the design template for every ML framework that followed: PyTorch tensors and TensorFlow arrays are direct descendants, extending the same n-dimensional array abstraction to GPUs. Python's role in ML infrastructure inherits much of its shape from this consolidation decision.

7 **Theano**: Developed at the Montreal Institute for Learning Algorithms (MILA) under Yoshua Bengio starting in 2007, Theano was an early and influential Python framework that compiled symbolic mathematical expressions into optimized CPU and GPU code via computational graphs (Bergstra et al. 2010; Team et al. 2016). It demonstrated that a Python-defined computation graph could be compiled for GPU execution without requiring the researcher to write CUDA code. Mila ended active development of Theano in 2017 (Mila 2026).

↩ The bitter lesson, that general methods which exploit growing computation outlast hand-engineered approaches, is established in section 1.4.

`pdb`. The second waits 30 seconds for compilation, then watches a model built from many small operations run 3× faster while losing that direct, line-by-line view of intermediate state. Both are correct; they have made different choices about the **execution problem**, the question of whether operations should execute immediately as written or be recorded for later execution. This choice creates a cascade of engineering trade-offs that shape every aspect of framework behavior, from debugging workflows to deployment options to peak hardware utilization.

7.3.1 Why execution strategy matters: The memory wall

The execution strategy of an ML framework is constrained by the memory wall, the widening gap between compute growth and memory-bandwidth growth that equation 2.3 quantifies. Modern accelerators can perform arithmetic far faster than they can fetch data. Element-wise operations such as ReLU use only a tiny fraction of peak compute capacity, not because the hardware is slow, but because they spend nearly all their time waiting for data. Section D.2.1 formalizes this trade-off, showing exactly when operations are memory bound vs. compute bound.

↩ The memory wall and the growth-rate ratio behind it are established in section 2.2.

The physical root of this imbalance is accelerator chip topography: silicon is partitioned between tiny, ultra-fast on-chip storage (registers and SRAM scratchpads, operating at tens of terabytes per second but holding only tens of megabytes) and large off-chip device memory (High Bandwidth Memory [HBM] or GDDR, holding 40–80 GB but reached across external memory buses with hundreds of cycles of latency). When evaluating an unoptimized sequence of three operations—such as LayerNorm → Dropout → ReLU—a framework dispatches three separate GPU kernels. The first reads inputs from HBM and writes intermediate results back to HBM; the second reads those results and writes back again; the third reads them a final time. Shuttling intermediate tensors across the external bus three times dwarfs the computation time, leaving arithmetic units idle.

The memory wall classifies operations as either compute-bound (limited by arithmetic throughput) or memory-bound (limited by data movement). Most individual neural network operation types (activations, normalizations, element-wise operations) are memory bound, though the large matrix multiplications that dominate total compute time can be compute bound.

The key optimization for memory-bound operations is kernel fusion (also called operator fusion), combining multiple operations into a single GPU function (called a *kernel*)[8] to avoid intermediate memory traffic. Instead of three round-trips to HBM, a fused kernel loads inputs from HBM once, evaluates normalization, dropout, and ReLU entirely within on-chip registers and SRAM, and writes only the final output tensor back to HBM, which cuts memory traffic by roughly 3×. Attention kernels[9] use the same principle at larger scale: instead of materializing the full attention matrix in HBM, a fused implementation can keep tiles close to the compute units, cut HBM accesses by up to 9×, and produce workload-specific speedups from 15 percent to 3× (T. Dao et al. 2022).

Framework compilers can fuse only operations that are visible to them simultaneously. When an eager runtime dispatches operations individually, each kernel launch executes in isolation, concealing cross-operation fusion and buffer-reuse opportunities. Capturing a broader region of computation into a deferred graph removes that isolation: the compiler can examine an entire sequence of dependencies, establish intermediate tensor lifetimes, and fuse compatible operations before dispatching work to the accelerator. How and when a framework captures that computation is therefore the core systems choice governing execution performance.

7.3.2 The computational graph

To expose multi-operation optimization opportunities to compilers, frameworks represent programs as a computational graph: a directed acyclic graph (DAG) where nodes represent operations and edges represent data dependencies. It is the same structure that backpropagation traverses in reverse, here serving as the framework's internal model of the computation for scheduling and optimization as well.

Mathematical operations can be decoupled from physical execution through graph representations. Figure 7.2 diagrams this minimal abstraction: directed edges route input variables x and y into the central transformation operator $f(x, y)$, which emits the output variable z independent of physical evaluation order.

Real machine learning models require much more complex graph structures. Figure 7.3 partitions this system interaction into two coupled domains: the left container isolates a six-node neural network

[8] **Kernel (GPU)**: In GPU programming, a kernel is the function dispatched to execute in parallel across thousands of threads. Each kernel launch costs about 5 µs of CPU-side overhead for parameter assembly and GPU signaling, and an eager framework's Python dispatch adds about 10 µs more before the launch, the split this chapter's overhead model uses. Small, unfused operations therefore spend more time on launch overhead (L_{lat}) than on useful arithmetic. Reducing kernel count through fusion is therefore a direct attack on the overhead term of the iron law.

[9] **Fused attention kernel**: A fused attention kernel combines the $\mathbf{QK}^T$ product, softmax, and value-weighted output into a tiled implementation that keeps intermediate values in on-chip memory rather than materializing the full attention matrix in HBM. FlashAttention is the canonical named example introduced in section 6.7.4.1 and reports up to 9× fewer HBM accesses with workload-specific speedups from 15 percent to 3× (T. Dao et al. 2022). The framework lesson is broader than the specific algorithm: fusion can shift an operation's position on the Roofline Model from bandwidth-limited toward throughput-limited execution by reducing round-trips through external memory.

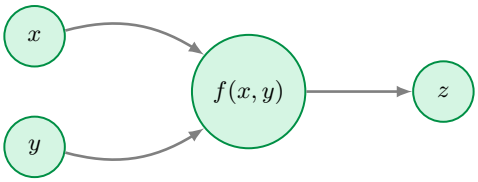


Figure 7.2: Simple Computational Graph: Representing $z = f(x, y)$ as a directed acyclic graph decouples mathematical dependencies from physical execution order, providing the structural blueprint frameworks require to track gradients and schedule operations across devices.

DAG with explicit dataflow edges, while the right container houses the runtime components—memory management and device placement—that interface with it. Across the central interface, the runtime queries tensor lifetimes to plan memory reuse and assigns operations to accelerator streams based on data dependencies rather than program order, optimizing execution before any arithmetic occurs.

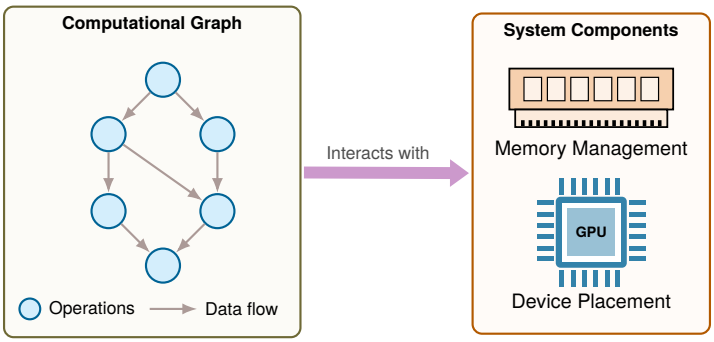


Figure 7.3: Computation Graph with System Interactions: A neural network DAG (left) interfaces with framework runtime components (right). By maintaining an explicit graph representation independent of execution, the framework can analyze tensor lifetimes for memory reuse and assign operations to target accelerators before issuing arithmetic.

This graph representation is more than a visualization; it is the data structure that enables both efficient execution and automatic differentiation. The timing of its construction forces a fundamental systems trade-off across debugging, optimization, deployment, and flexibility. Interactive debugging demands immediate execution so developers can inspect intermediate values and step through logic with standard tools. Global optimization requires observing large operator sequences at once to fuse kernels and plan memory reuse. Standalone deployment favors graphs that run without the host interpreter, while dynamic architectures require control flow that branches on computed tensor values. No single execution model serves all four demands at once.

7.3.3 Three execution strategies

The computational graph enables global optimization, but it leaves a critical design choice unresolved: *when* the runtime constructs the graph. Consider an elementary expression such as `y = x * 2`. Evaluating the multiplication immediately yields a concrete tensor in `y`. This imperative flow is intuitive and directly debuggable, but the runtime sees only one operation at a time. Deferring execution records the operation symbolically into a dependency graph executed only upon request. That staging separates intent from evaluation, giving the compiler the end-to-end visibility required for kernel fusion and memory planning.

Modern frameworks resolve this tension between *flexibility* and *optimization potential* with three execution strategies: eager execution with dynamic graphs, static computation graphs, and hybrid approaches that combine just-in-time (JIT) compilation with eager development.

7.3.3.1 Eager execution with dynamic graphs

Eager execution evaluates each operation immediately as the program calls it, building the computation graph dynamically at runtime. A side-by-side comparison shows how this differs from graph-based execution at the code level.

Example 7.1: Eager vs. graph execution code comparison

PyTorch (eager execution):

```
import torch

x = torch.tensor([1.0, 2.0])
y = x * 2
print(f"Intermediate value: {y}")  # Works immediately
z = y.sum()
```

TensorFlow 1.x (static graph):

```
import tensorflow as tf

x = tf.placeholder(tf.float32)
y = x * 2
# print(y) -> Prints Tensor("mul:0"...), not value!
z = tf.reduce_sum(y)

with tf.Session() as sess:
    result = sess.run(z, feed_dict={x: [1.0, 2.0]})
```

Systems insight: Eager execution exposes intermediate values as ordinary runtime state, which makes debugging direct. Static graphs stage computation before execution, which enables whole-graph optimization but changes the debugging model.

In this define-by-run model, when host code evaluates `y = x * 2`, the runtime dispatches the multiplication kernel to the device command queue and returns a tensor handle for subsequent operations. Device execution can continue asynchronously; dependent operations must wait for the result, and reading its value on the host can require synchronization.

This provides the flexibility of normal programming: developers can print intermediate values, use conditionals based on computed results, and debug with standard tools. The framework records operations as they happen, constructing a dynamic graph that reflects the actual execution path taken.

For gradient computation, the framework records a history of operations in what is called an autograd tape,[10] a transient data structure built during execution. Each tensor operation creates a node that records: the operation performed, references to input tensors, and how to compute gradients. These nodes form a DAG that records the actual path taken during forward-pass execution rather than a graph fixed in advance. Listing 7.1 shows how PyTorch records operations as they execute in its default eager mode.

[10] **Autograd tape**: A transient data structure built during forward execution, where nodes record operations, dependencies, and backward functions for chain-rule evaluation. Its memory footprint grows with model depth and sequence length as saved activations accumulate until released during backpropagation. For deep models, frameworks retain selected activations and recompute others via activation checkpointing to prevent OOM failures.

Listing 7.1: Autograd Tape Construction: Each operation executes immediately while recording a backward node to the autograd tape for later gradient computation.

```
import torch

x = torch.tensor([1.0], requires_grad=True)
y = x * 2  # Executes immediately; records MulBackward node
z = y + 1  # Executes immediately; records AddBackward node
# The autograd tape exists NOW, built during execution
```

After these two operations, the framework has constructed an autograd tape with two nodes: one for the multiplication and one for the addition. The tape records that `z` depends on `y`, and `y` depends on `x`.

Calling `z.backward()` traverses this tape in reverse topological order, applying the chain rule at each node:

1. Compute $\frac{\partial z}{\partial z} = 1$ (seed gradient)
2. Call `AddBackward0.backward()` $\rightarrow \frac{\partial z}{\partial y_{\text{tape}}} = 1$
3. Call `MulBackward0.backward()` $\rightarrow \frac{\partial z}{\partial x} = 2$
4. Accumulate gradient in `x.grad`

After `backward()` completes, autograd normally releases the saved state needed by its backward rules. Other live references can still retain tensor storage and graph nodes. The next forward pass builds a new graph. Values needed for gradients are saved during the forward pass and remain available until backward consumes them, so their memory cost spans both phases rather than appearing only during backward.

Example 7.2: In-place operations can break gradients

Scenario: An engineer applies an in-place mutation (`x += 1`) within a custom PyTorch activation function to reduce memory allocations.

Diagnosis: In-place operations overwrite tensor memory containing forward-pass activations required by the autograd tape for backward-pass gradient computation, triggering a runtime version-counter error (PyTorch Contributors 2026a). PyTorch enforces this safety invariant by maintaining an integer version counter on each tensor that increments on every in-place write; if a saved tensor's version at backward execution differs from its version recorded during the forward pass, autograd throws an error because the activation required by the derivative rule was modified in place.

Systems lesson: Framework automatic differentiation depends on immutable forward-pass activation records. In-place writes to a saved tensor invalidate the autograd record, so PyTorch stops with an error rather than compute a wrong gradient.

In eager execution frameworks, operations evaluate imperatively as host code encounters them. The define-by-run cycle is therefore a loop of one operation at a time (figure 7.4). Python defines an operation, the runtime enqueues its device work, and control returns to Python for the next call while the device may execute asynchronously. When the runtime decides what to launch, it sees the current operation rather than a whole region of future calls.

Systems implications: Flexibility The dynamic autograd tape expresses data-dependent behavior directly in the host language. Conditionals and loops can depend on tensor values computed during execution, enabling algorithms such as beam search, dynamic-length recurrent models, or adaptive compute paths that adjust their execution based on intermediate activations. Static graphs can also represent data-dependent control flow and dynamic dimensions through graph operations, but eager execution makes these patterns natural to write and debug in Python. Because operations are issued immediately, developers can print tensors, inspect values, and use standard debuggers (`pdb`, breakpoints) to diagnose errors much as they would in another Python program.

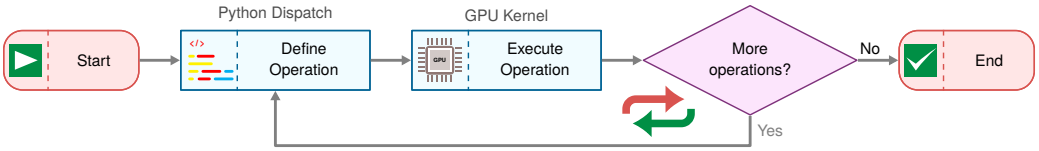
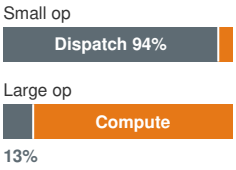


Figure 7.4: Dynamic Graph Execution Flow: In eager execution, each operation is issued as the program encounters it. Accelerator work may execute asynchronously after dispatch. This define-by-run model enables natural debugging and data-dependent control flow while limiting optimization across Python calls.

Systems implications: Overhead This flexibility imposes latency costs that map directly to the iron law. Each training step constructs an autograd tape dynamically, adding host-side Python dispatch latency and tape allocation to L_{lat}. Because every operation traverses the framework dispatcher before a device kernel launches, host overhead quickly dominates when individual kernels execute in single-digit microseconds. Eager execution also lacks a whole-graph view, preventing the framework from fusing element-wise chains into single GPU launches or pre-allocating a static memory arena for intermediate activations (D_{vol}). The autograd tape must retain every tensor requested by registered backward rules, holding memory until backpropagation consumes it. Together, these costs impose a throughput ceiling on fine-grained workloads where dispatch latency overshadows device arithmetic.

The dispatch tax: Python overhead vs. GPU reality Eager execution's performance ceiling is driven by a fundamental systems mismatch: the speed of the host-side interpreter vs. the speed of the device-side silicon. The dispatch tax, defined here as the share of step time spent in host-side dispatch, $T_{\text{sw}}/(T_{\text{sw}}+T_{\text{hw}})$ for host software time T_{sw} and device hardware time T_{hw}, quantifies this mismatch in an additive model without host-device overlap. Asynchronous execution can hide part of that host cost.



Fixed dispatch dominates small ops and fades behind large ones.

In the illustrative eager-execution model used here, every operation pays a fixed "tax" of approximately 15 µs for Python-side dispatch and type checks; this first comparison excludes the additional driver launch cost. Actual overhead varies by framework, backend, and hardware, but its relative weight still depends on operation size. For a small operation such as a ReLU on a small vector, the kernel might execute in only 1 µs, so the modeled dispatch tax reaches 94 percent and the GPU spends most of its time waiting for the next command. For a large operation such as a large matrix multiply, the kernel executes for 100 µs, the modeled dispatch tax drops to 13 percent, and host dispatch becomes a small share of the modeled time. The operation is device-work dominated; distinguishing compute from memory limits still requires comparing its work and traffic.

The dispatch tax explains why models with many small layers run significantly slower than their raw FLOP count predicts. Section A.5.1 places this symptom in the bottleneck taxonomy, classifying a dispatch-dominated workload as latency-bound rather than compute-bound and showing which optimizations actually move it. Frameworks can move from *kernel-by-kernel dispatch* to *graph-level execution*, reducing repeated interpreter work and exposing operations that can be fused. A captured graph can still launch multiple kernels; fusion and runtime replay determine how much launch overhead remains. The hybrid JIT and compilation strategies in section 7.3.3.3 exist precisely to address this overhead.

The overhead costs of eager execution motivate the opposite design: capturing the entire computation before executing any of it. This is precisely what static computation graphs provide.

7.3.3.2 Static computation graphs

Static graph execution defines the complete computational graph as a symbolic representation first, then executes it separately. This "define-then-run" execution model means the graph exists before any computation occurs, enabling aggressive ahead-of-time optimization. Seeing the complete computation before launching device kernels allows the framework to analyze, transform, and optimize the graph globally—visibility unavailable across separate eager calls unless a compiler captures them. Section C.3.4.2 works through the canonical global transformation this enables: for N elements of s_{elem} bytes each, fusing a chain of n_{ops} elementwise operations into one kernel reduces idealized read-and-write traffic from $2n_{\text{ops}}Ns_{\text{elem}}$ to $2Ns_{\text{elem}}$ bytes.

Two-phase execution Static graphs implement a clear separation between graph construction and execution. Listing 7.2 illustrates the two phases using TensorFlow 1.x, which exemplified this approach. It deliberately runs the same `x * 2` then `+ 1` computation shown under eager execution in listing 7.1, holding the arithmetic fixed so that the only thing that changes is *when* it executes: symbolic definition creates placeholders and operations without computation, while explicit execution triggers actual arithmetic.

Listing 7.2: Static Graph Two-Phase Execution: Graph construction (symbolic definition) is separated from execution (actual computation), enabling ahead-of-time optimization.

```
# Phase 1: Graph Construction (symbolic, no computation)
import tensorflow.compat.v1 as tf

tf.disable_v2_behavior()

# Define graph symbolically
x = tf.placeholder(tf.float32, shape=[1])  # Just a placeholder
y = x * 2  # Not executed, just recorded
z = y + 1  # Still no execution
# At this point, nothing has been computed

# Phase 2: Graph Execution (actual computation)
with tf.Session() as sess:
    result = sess.run(z, feed_dict={x: [1.0]})
    # Now computation happens: result = [3.0]
```

Static graph compilation separates graph construction from tensor execution. In the definition phase, the program declares operations and variables and builds the symbolic DAG without allocating any tensor memory. Only in the execution phase do data load, concrete buffers materialize on the accelerator, and the captured graph run (figure 7.5).

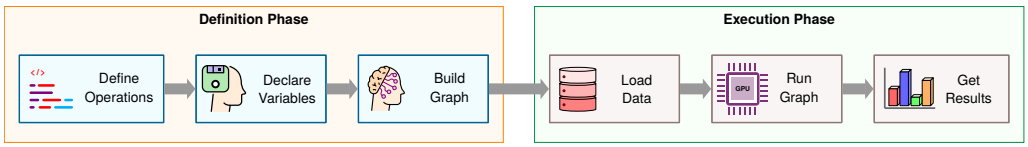


Figure 7.5: Static Graph Execution Lifecycle: Separating symbolic graph definition (left) from concrete execution (right) provides global visibility over the computation, allowing compilers to apply ahead-of-time optimizations (such as kernel fusion, buffer reuse, and layout transforms) before any tensors are materialized.

The key difference from eager execution is that during construction, `x`, `y`, and `z` are not tensors containing values but rather symbolic nodes in a graph. Operators such as `*` and `+` append transformation nodes to the graph definition without issuing arithmetic. Calling `print(y)` makes this distinction visible: it prints symbolic tensor metadata rather than numerical values. Only when the host invokes `sess.run()` does the runtime analyze, optimize, and execute the captured graph on device buffers.

Ahead-of-time optimization Because the framework has the complete graph before execution, it can perform ahead-of-time optimization unavailable across uncaptured eager calls. The kernel fusion opportunity introduced in section 7.3.1 becomes actionable here: because the framework sees `y = x * 2` and `z = y + 1` together in the graph, it can fuse them into `z = x * 2 + 1`, eliminating the intermediate `y` and halving idealized memory traffic. When tensor shapes and lifetimes are known, the compiler can plan memory before execution and reuse buffers whose lifetimes do not overlap; dynamic dimensions may require bounds or runtime allocation. Tensor layouts can also be transformed globally (for example, NCHW to NHWC) to match hardware preferences, though physical layout conversions may still require copies. Dead-code elimination (DCE)[11] removes operations whose results are unused, and constant folding precomputes operations on constant values when their inputs are known. These optimizations map directly to iron law terms: kernel

[11] **Dead code elimination (DCE)**: Removes graph nodes whose results do not affect observable outputs or side effects. In ML graphs, dead code arises from unused branches or post-transform artifacts; side-effect analysis ensures safe removal to prevent execution of dead operations and reduce kernel launch overhead.

fusion can reduce D_{vol} by avoiding intermediate memory writes, constant folding reduces repeated work, planned allocation can reduce runtime overhead, and dead code elimination removes work and associated data movement when unused computations are present. Compilers such as **XLA**[12] (Google 2025) extend this approach, compiling captured subgraphs into fused machine code specialized for target hardware.

Systems implications Static graphs can achieve high performance through ahead-of-time optimization. Kernel fusion reduces memory bandwidth requirements when intermediate traffic is the bottleneck, and hardware-specific compilation can approach high utilization when shapes, operators, and layouts match efficient backend paths.

The cost of this performance is reduced flexibility. Standard Python control flow (`if`, `for`) cannot depend on computed tensor values in static graphs. TensorFlow provides graph-level control flow primitives (`tf.cond` and `tf.while_loop`) that support data-dependent conditions, but these require special syntax that diverges from standard Python, making code harder to write and reason about. Debugging is difficult because stack traces point to graph construction code, not execution code. Error messages often reference symbolic node names rather than the actual operations that failed.

7.3.3.3 Hybrid approaches: JIT compilation

JIT compilation pursues both eager debugging and graph optimization at once by capturing computation at runtime. The core trade-off is *fidelity vs. generality*. Tracing captures the execution path taken during a sample run, producing high fidelity to that path but missing branches not taken. Source-level compilation (scripting) analyzes supported program structure, preserving supported control flow but requiring a restricted language subset. Both approaches produce an intermediate representation (IR)[13] that enables graph optimizations such as kernel fusion, constant folding, dead code elimination, and buffer reuse.

The eager-vs.-compiled trade-off has a direct iron law consequence. JIT compilation amortizes the L_{lat} (dispatch overhead) across the compiled region. Longer compiled regions mean more overhead amortized per operation, which explains why graph breaks are performance-critical: each break forces a return to eager dispatch, resetting the amortization.

Historically, PyTorch's TorchScript exemplified both strategies (PyTorch Contributors 2026d). TorchScript is now deprecated in favor of capture and export paths such as `torch.export` (PyTorch Contributors 2026d, 2026c), but its two modes still make the fidelity-generality trade-off concrete. Tracing executed a function with example inputs and recorded the tensor operations observed on that path. Scripting instead compiled a restricted, type-annotated subset of Python source, preserving supported branches in the graph at the cost of rejecting unsupported Python objects, library calls, and metaprogramming. Both produced an IR with static types in static single-assignment (SSA) form, where each variable is assigned exactly once, so fusion, constant folding, and dead code elimination could run without the Python interpreter.

The critical limitation of tracing is that a trace records operations from an observed path rather than preserving arbitrary Python control flow. Listing 7.3 illustrates the resulting correctness risk.

Listing 7.3: Tracing a Data-Dependent Branch: Tracing records the path taken by the example input. PyTorch commonly emits a `TracerWarning` when a tensor value is converted to a Python condition because the resulting trace may not generalize.

```
import torch


def conditional_forward(x):
    if x.sum() > 0:  # Data-dependent condition
        return x * 2
    else:
        return x * 3


traced = torch.jit.trace(conditional_forward, torch.tensor([1.0]))
# Tracing captures ONLY the x.sum() > 0 branch
# If input later has sum <= 0, traced version
# still executes x * 2 branch
```

[12] **XLA (accelerated linear algebra)**: Compiling to fused machine code means XLA can fuse subgraphs, specialize layouts, and lower high-level operations into backend-specific code; fusion attacks both launch overhead (L_{lat}) and intermediate memory writes (D_{vol}), but the realized speedup depends on whether the graph contains enough fusible, memory-bound work for the compiler to remove. Large GEMM-heavy regions may already be compute bound, while chains of small elementwise operations can benefit more because fusion removes repeated trips through external memory. The benefit is not a fixed multiplier: XLA helps when it can fuse operations, specialize layouts and shapes, and reduce launch or memory overhead, so gains depend on graph structure, backend support, and input-shape stability.

[13] **Intermediate representation (IR)**: The "intermediate" captures this format's architectural role: a language-independent layer that decouples the frontend (Python capture) from the backend (hardware code generation), exactly as LLVM IR decouples C/Rust/Swift frontends from x86/ARM backends. ML frameworks adopted this compiler pattern because it reduces the $\mathcal{O}(M \times N_{\text{backend}})$ cost of supporting M frontends and N_{backend} backends to $\mathcal{O}(M + N_{\text{backend}})$: a single graph capture mechanism (TorchDynamo, tf2xla) can target multiple hardware backends without rewriting the capture logic.

Tracing records the branch executed by the example input. In this example, converting the tensor condition to a Python Boolean commonly produces a `TracerWarning`; the resulting trace can still follow the captured branch for later inputs that require the other branch. Treating trace warnings as correctness failures is therefore essential when data-dependent Python control flow is present. The same hazard survives in modern graph-capture systems, which is why they guard each captured region with explicit assumptions about shapes and control flow.

7.3.3.4 Modern compilation: Graph-capture JIT

The previous approaches force a choice between flexible eager execution and compiler-visible graphs. Modern JIT compilation narrows this trade-off by automatically capturing regions of eager code into optimized graphs with limited developer intervention.

Graph-capture JIT systems follow the same architectural pattern across frameworks. The first execution observes tensor operations, records a graph region guarded by assumptions about shapes, dtypes, layouts, and control flow, lowers that region into an intermediate representation, applies fusion and layout optimizations, and caches executable code for later calls that satisfy the same guards. The compiled artifact is valid only while those assumptions hold, so a failed guard forces the runtime to recompile or fall back to eager execution. When eager fallback is enabled, unsupported Python code forms a graph boundary where execution returns to the eager runtime; a configuration requiring full-graph capture can instead report an error. PyTorch 2.0's `torch.compile` (Ansel et al. 2024) is a concrete instance of this pattern, but the systems idea is broader than the API: compilation pays only when captured regions are long enough and stable enough to amortize capture, lowering, code generation, and cache-management costs.

Compilation therefore attacks the dispatch tax of section 7.3.3.1 directly, and fusion adds a second saving in memory traffic. The physics of software overhead in notebook 7.1 models both.

Napkin Math 7.1: The physics of software overhead

Iron law connection: The latency term (L_{lat}) in the iron law is dominated by software overhead: dispatching instructions from Python to the GPU. Each operation pays a Python dispatch cost of ~10 µs plus a kernel launch cost of ~5 µs, which together set the per-operation overhead applied in the two scenarios below. Python dispatch covers the framework call; driver launch covers submission to the device. A compiled region invoked from Python still pays an entry cost, whereas replay of a captured launch sequence removes repeated per-operation Python work and has its own submission cost.

Scenario one: Eager Mode (The "Tiny Op" Trap) Consider a simple activation block, `y = relu(x + bias)`, in which the input and output each contain N elements of s_{elem} bytes each. Assume a scalar or broadcast bias remains in fast memory, so the traffic model counts the full input, intermediate, and output tensors.

- **Operations**: Two (Add, ReLU).
- Execution:
  1. Launch `Add` Kernel: 15 µs overhead.
  2. Read/Write Memory: $2Ns_{\text{elem}}$ bytes.
  3. Launch `ReLU` Kernel: 15 µs overhead.
  4. Read/Write Memory: $2Ns_{\text{elem}}$ bytes.
- **Total overhead**: 30 µs.
- **Total memory traffic**: $4Ns_{\text{elem}}$ bytes.

Scenario two: Compiled Mode (Fusion) The compiler fuses this into one kernel: `FusedAddRelu`. This scenario retains one ordinary Python entry and one driver launch, applying the same combined cost as eager mode to isolate the effect of reducing two calls to one. It does not model captured-graph replay.

- Execution:

1. Launch `Fused` Kernel: 15 µs overhead.
2. Read/Write Memory: $2Ns_{\text{elem}}$ bytes (the intermediate result stays on chip).

- **Total overhead**: 15 µs (2× speedup).
- **Total memory traffic**: $2Ns_{\text{elem}}$ bytes (2× bandwidth efficiency).

Systems insight: Fusion wins on two fronts at once. Collapsing two launches into one halves the per-op dispatch overhead in this scenario, and keeping the intermediate result on chip cuts idealized memory traffic from $4Ns_{\text{elem}}$ to $2Ns_{\text{elem}}$ bytes. For small element-wise operations, the avoided round-trip to external memory can matter more than the arithmetic.

Scaling this arithmetic across a sequence of five small operations demonstrates how host dispatch latency starves the accelerator (figure 7.6). In eager execution, each 1 µs kernel incurs 10 µs of Python dispatch overhead, leaving the accelerator idle for over 90 percent of the sequence. Compiling the sequence resolves both constraints simultaneously: the framework submits the work in a single launch of 5 µs, eliminating all intermediate host dispatch gaps, while operator fusion keeps intermediate activations in registers or cache to complete the entire sequence in 4 µs instead of the uncompiled 5 µs.

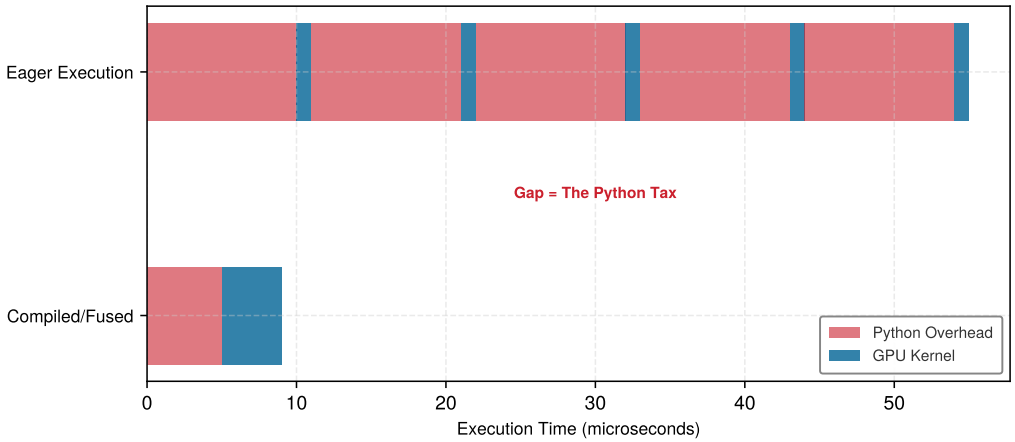


Figure 7.6: The Python Tax: Execution timelines illustrate how host dispatch latency (L_{lat}) starves the accelerator during fine-grained operations. Eager execution incurs per-operation host overhead for each kernel, whereas compilation and fusion collapse the sequence into a single launch (illustrated with direct device submission), eliminating intermediate dispatch gaps and intermediate global memory round-trips.

Graph-capture compilers such as `torch.compile`[14] automate this fusion over captured regions without requiring engineers to write custom CUDA.[15] However, compilation cannot span arbitrary Python code: graph breaks mark the boundary where compilation ceases to apply. Data-dependent Python control flow, unsupported library calls, I/O, custom Python objects, and highly variable shapes can shorten compiled regions. With eager fallback enabled, a break separates compiled subgraphs with standard Python execution, reintroducing interpreter dispatch and limiting fusion across the boundary. Tensor handles can pass between these regions while their storage remains on the device; host reads of device values can additionally require synchronization. This is why graph-break analysis belongs in performance engineering: the relevant metric is not whether compilation is enabled, but how much of the hot path remains inside long, stable compiled regions.

Backends occupy different points on the flexibility-performance spectrum. A **JIT** compiler traces model execution at runtime to generate optimized kernels dynamically, incurring initial warmup latency during the first pass. An **ahead-of-time (AOT)** compiler instead compiles captured computation before deployment, shifting compilation out of the hot path and enabling a standalone runtime when the export target supports one. A general JIT backend optimizes ordinary training and serving workloads with moderate compilation cost; a specialized inference backend can apply deeper fusion,

[14] `torch.compile`: PyTorch's capture front end analyzes Python bytecode as each frame runs, rather than recording tensor operations alone. That vantage lets it keep capturing across Python control flow it can resolve from guarded values and insert a graph break where it cannot, which is why graph breaks rather than tracing errors are its characteristic failure mode.

[15] **CUDA (compute unified device architecture)**: NVIDIA's parallel computing platform (2007) serving as the foundational layer between high-level Python operations and GPU silicon; when PyTorch executes `torch.matmul(A, W)`, the call traverses the framework's dispatcher, selects a cuBLAS kernel, and launches it on the GPU. Each launch adds the 5 µs kernel-launch share of the per-operation tax on top of the framework's Python dispatch. For small operations, this dispatch overhead (L_{lat}) can exceed useful compute time. Fusing compatible groups among N_{ops} operations then reduces launch count as well as intermediate memory traffic; capturing the graph alone does not imply a single kernel.

precision lowering, and autotuning when the deployment target is fixed; an ahead-of-time mobile or embedded runtime removes even more flexibility to gain footprint and predictability. The same rule governs all of them: the narrower the target and the more stable the graph, the more optimization the compiler can safely perform.

The resulting workflow is a systems decision. Rapid prototyping favors eager execution because architecture changes and guard failures make recompilation cost visible. Long training runs and high-volume inference amortize compilation cost over many executions, provided the model has stable shapes and limited graph breaks. Debugging usually starts in eager mode because errors map directly to source code; compilation is reintroduced after the model behavior is correct and the performance bottleneck is measurable.

Comparison of execution models Table 7.1 contrasts the three execution models across six dimensions, showing how hybrid JIT compilation can recover graph-level optimization within captured regions while preserving eager fallback outside them.

Table 7.1: Execution Model Trade-Offs: Each execution model occupies a distinct position in the flexibility-optimization trade-off space. Eager execution maximizes debugging flexibility but limits cross-operation optimization; static graphs maximize optimization but require control flow to be represented in the graph; hybrid JIT compilation compiles captured regions while falling back to eager execution for unsupported patterns.

Aspect	Eager + Autograd Tape (PyTorch default)	Static graph (TensorFlow 1.x)	JIT Compilation (torch.compile)
Execution Model	Immediate	Deferred	Hybrid
Graph Construction	During forward pass	Before execution	First execution (cached)
Optimization	Per-operation kernels	Ahead-of-time	JIT compilation
Dynamic Control Flow	Python control flow	Graph control flow	Captured regions or breaks
Debugging	Easy (standard Python)	Difficult (symbolic)	Moderate (mixed)
Performance	Baseline	High (optimized)	High (compiled regions)

Eager mode's primary value is in the iteration loop: it allows using standard Python debuggers (such as pdb) to inspect variables mid-execution, whereas graph-mode debugging often requires specialized framework tools. This immediate feedback accelerates the prototyping phase of the ML lifecycle.

Beyond these core execution trade-offs, table 7.2 highlights additional systems-level distinctions between static and dynamic approaches.

↩ The iteration tax, the cost that slow experiment cycles impose on how many candidate models a team can test, is established in section 3.3.

Table 7.2: Additional Graph Trade-Offs: Systems-level distinctions between static and dynamic graphs that complement table 7.1. These trade-offs reappear when selecting frameworks in section 7.9.

Aspect	Static Graphs	Dynamic Graphs
Memory Management	Precise allocation planning, optimized memory usage	Flexible but potentially less efficient
Hardware Utilization	Can generate highly optimized hardware-specific code	May sacrifice hardware-specific optimizations
Research Velocity	Slower iteration due to define-then-run requirement	Faster prototyping and model experimentation
Integration with Legacy Code	More separation between definition and execution	Natural integration with imperative code

These trade-offs are not binary choices. Modern frameworks offer a spectrum of execution options, which raises the quantitative question of where on this spectrum a given project should operate.

7.3.4 Quantitative principles of execution

These execution models form a spectrum, and two quantitative principles make the choice measurable in practice. The compilation continuum principle asks when compilation gains justify development cost by comparing production executions with development iterations. The dispatch overhead law shows why eager execution can spend more time dispatching small operations than computing them.

7.3.4.1 The compilation continuum principle

The execution problem demands a quantitative principle for when a project should compile. The execution models form a continuum from maximum flexibility to maximum optimization. Equation 7.1 lays out the four positions on that axis, and each labeled arrow names the mechanism that carries a project one step rightward toward hardware.

$$\text{Eager} \xrightarrow{\text{capture}} \text{Graph JIT} \xrightarrow{\text{export}} \text{AOT Runtime} \xrightarrow{\text{specialize}} \text{Hardware} \tag{7.1}$$

Each step rightward sacrifices flexibility for performance. The practical question is *where* a given project should be placed on this continuum. The optimal compilation strategy depends on production executions, development recompilations, and their respective time costs, combined in equation 7.2:

$$\text{Compilation Benefit} = \frac{N_{\text{prod}} \cdot (T_{\text{eager}} - T_{\text{compiled}})}{T_{\text{compile}} + N_{\text{dev}} \cdot T_{\text{compile}}} \tag{7.2}$$

where:

- N_{prod} = number of production executions (dimensionless count: inference requests, training steps)
- N_{dev} = number of development iterations requiring recompilation (dimensionless count)
- T_{eager} = time per execution in eager mode (seconds)
- T_{compiled} = time per execution in compiled mode (seconds)
- T_{compile} = time per compilation or recompilation (seconds; assumed constant)

This model assumes that the compiled artifact remains valid between development changes and that the initial compilation and each recompilation have the same cost; under these assumptions, the decision rule is to compile when Compilation Benefit > 1. The ratio is dimensionless.

Table 7.3 prices that decision rule with hypothetical throughputs and compilation times across execution modes and model architectures:

Table 7.3: Illustrative Inference Throughput: Hypothetical inference throughput on an NVIDIA A100 GPU with batch size 32, assuming matching input shapes, precision, and numerical requirements across each row. These values are assumptions rather than benchmark measurements. Within this scenario, the table implies torch.compile speedups of 1.4–1.5× over eager mode and TensorRT speedups of 2.3–2.7×. The setup column applies to torch.compile; TensorRT build costs require a separate measurement.

Model	Eager (examples/sec)	torch.compile (examples/sec)	TensorRT (examples/sec)	JIT Setup (seconds)
ResNet-50	1,450	2,150	3,800	15–30
BERT-Base	380	520	890	30–60
ViT-B/16	620	950	1,650	25–45
GPT-2 (124M)	180	260	420	45–90

These throughput differences are not uniform across workloads, so the execution strategy must align with the operational regime of each workload archetype. The appropriate strategy depends on the computational phase, the target backend, and the dominant iron law term. In batched prefill, transformers exhibit high operational intensity that benefits from dense matrix compilation; in small-batch autoregressive decoding, however, the workload shifts into a bandwidth-bound regime where memory transfers dominate. Table 7.4 maps each recurring archetype to a candidate execution strategy based on these primary physical constraints.

↩ The workload archetypes and the iron law term that dominates each are established in section 2.3.2.

Table 7.4: Framework Execution Strategy by Workload: Candidate execution strategies for each workload archetype, aligned to the dominant iron law term. For the Bandwidth Hog row, attention fusion and graph compilation eliminate intermediate round-trips and launch overhead, but they do not eliminate the memory-bandwidth bottleneck of streaming weights and KV caches during autoregressive decoding; altering that limit requires increasing batch size or adopting lower-precision representations. For the Tiny Constraint row, memory capacity is a feasibility gate that must be met before any iron law term matters. For the Sparse Scatter row, the bandwidth and latency are those of irregular embedding gathers and the network exchange that follows them, not the streaming peak.

Archetype	Dominant Iron Law Term	Candidate Framework Strategy	Rationale
ResNet-50 (Compute Beast)	$\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$ (Compute)	Compiled dense kernels	Regular dense kernels benefit from layout selection, precision lowering, and backend specialization; fusion helps most in surrounding memory- or launch-bound regions
GPT-2 (Bandwidth Hog)	$\frac{D_{\text{vol}}}{\text{BW}}$ (Memory Bandwidth)	Fused attention + graph compilation	Fused attention eliminates intermediate activation round-trips, while compilation removes kernel launch tax; streaming parameter and KV-cache traffic remain bandwidth-limited
DLRM (Sparse Scatter)	$\frac{D_{\text{vol}}}{\text{BW}} + L_{\text{lat}}$ (Gathers, Network)	Specialized lookup kernels and measured capture	Irregular gathers limit some optimizations; measure compilation benefits in dense regions and surrounding dispatch
DS-CNN (Tiny Constraint)	Memory Capacity (Feasibility Gate)	Ahead-of-time microcontroller runtime	Fixed memory and a small native runtime must fit the target before latency can be optimized



Lighthouse 7.1: Framework strategy by archetype

The four recurring archetypes in table 7.4 trace one principle across the framework: compilation benefits scale with how much of the workload is optimizable.

Systems insight: In the illustrative scenario, Compute Beasts such as the ResNet-50 row in table 7.3 benefit because their dense kernels expose substantial surface for layout selection, precision lowering, and fusion. Bandwidth Hog workloads such as small-batch GPT-2 generation require distinguishing intermediate activation traffic from parameter streaming: fusing attention kernels eliminates round-trips for transient activation tensors and removes dispatch latency, but memory bandwidth continues to bind on streaming model weights and key-value caches unless batching increases operational intensity. Sparse Scatter workloads such as DLRM can gain less when irregular embedding lookups leave limited compiler scope.

Three cases illustrate how reuse changes the compilation decision. In research prototyping ($N_{\text{dev}} \gg N_{\text{prod}}$), teams should stay eager. If the architecture changes every few minutes, compilation overhead can dominate. Under the scenario's 30-second compilation assumption, ten compilations per hour consume five minutes.

For repeated execution ($N_{\text{prod}} \gg N_{\text{dev}}$), compilation can pay off because its one-time cost is amortized across many calls. In the hypothetical ResNet-50 inference scenario in table 7.3, torch.compile provides 48.3 percent higher throughput (2,150 img/s vs. 1,450 img/s). Using the scenario's 30 s compile cost, this pays off after the breakeven point in equation 7.3:

$$N_{\text{breakeven}} = \frac{T_{\text{compile}}}{T_{\text{eager}} - T_{\text{compiled}}} \tag{7.3}$$

Evaluating equation 7.3 with the hypothetical ResNet-50 values gives approximately 134,000 images. The same amortization principle applies to long training runs, using training throughput and compilation costs measured for that workload.

Long-lived production inference ($N_{\text{dev}} \approx 0$, $N_{\text{prod}} \to \infty$) extends this reuse further, providing more opportunity to amortize compilation. Aggressive autotuning can be worthwhile even when

setup takes much longer, provided the model converts correctly, meets numerical requirements, and improves measured performance over the deployment lifetime.

The cases form a continuum of increasing reuse (figure 7.7). Each strategy's total time starts at its one-time compilation cost and grows at its per-execution cost, so eager wins at low execution counts, JIT wins once its compilation is amortized, and AOT wins only at the largest counts, where its higher upfront cost is fully repaid. The figure's crossovers (near 2×10^6 and 6.7×10^6 executions) come from an illustrative cost model in arbitrary units, not from the ResNet-50 scenario, whose eager-to-compiled breakeven sits near 134,000 images. What carries over is the order of the regions, not their positions.

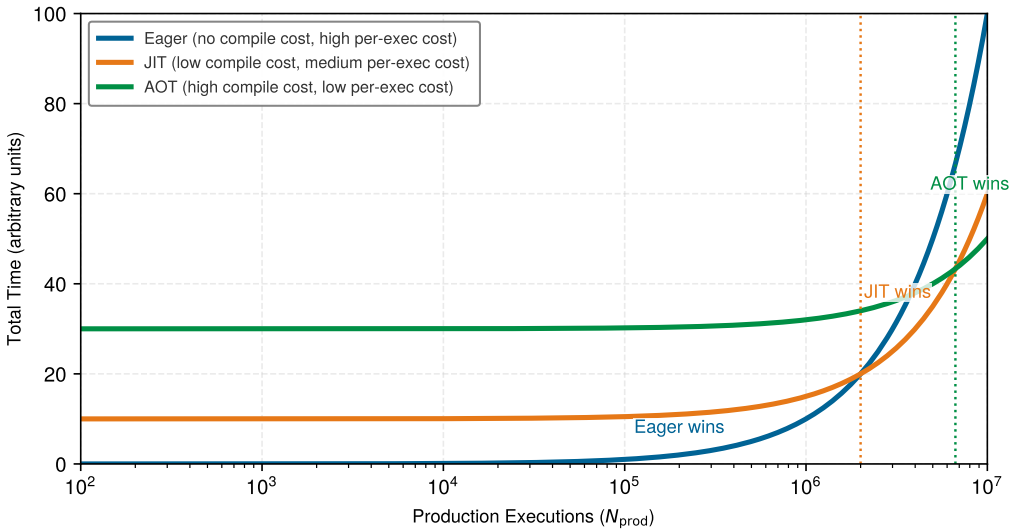


Figure 7.7: The Compilation Continuum: An illustrative cost model shows how one-time compilation cost and per-execution cost create eager, JIT, and AOT crossover regions as production executions increase. The crossover locations depend on the assumed compilation costs and per-execution rates.

7.3.4.2 The dispatch overhead law

A second principle, the dispatch overhead law, emerges from equation 7.4, which identifies the regime in which framework overhead, rather than compute or memory, dominates execution time. Let N_{ops} be the number of operations (count), t_{dispatch} the per-operation dispatch overhead (seconds), and T_{compute} and T_{memory} the total compute and memory times (seconds). The ratio uses the additive, no-overlap approximation $T_{\text{hw}} \approx T_{\text{compute}} + T_{\text{memory}}$; when execution overlaps those costs, measured hardware time should replace their sum. Framework overhead dominates when operations are small relative to dispatch cost:

$$\text{Overhead Ratio} = \frac{N_{\text{ops}} \cdot t_{\text{dispatch}}}{T_{\text{compute}} + T_{\text{memory}}} \tag{7.4}$$

When Overhead Ratio > 1, the model is overhead-bound. The ratio and the dispatch tax introduced with eager execution measure the same imbalance from two sides. With $T_{\text{sw}} = N_{\text{ops}} \cdot t_{\text{dispatch}}$, the tax share $T_{\text{sw}}/(T_{\text{sw}} + T_{\text{hw}})$ equals Overhead Ratio$/(1 + \text{Overhead Ratio})$, so a ratio above 1 is the same threshold as a dispatch tax above 50 percent. Compilation can provide its largest gains for overhead-bound workloads because fusion reduces the number of dispatches. The training-step trace in section 7.10 works this effect through end to end, previewed by the numbers that follow.

Applying equation 7.4 means comparing the software overhead $T_{\text{sw}} = N_{\text{ops}} \cdot t_{\text{dispatch}}$ with the hardware execution time T_{hw} (both in seconds). The regime depends on the size of individual operations rather than on the model's parameter count. Notebook 7.2 contrasts a small multilayer

perceptron with a large transformer layer to quantify when host overhead or device arithmetic bounds the system.

Napkin Math 7.2: Dispatch overhead at two scales

Problem: Determine the threshold where host-side Python interpreter dispatch overhead bounds end-to-end execution time relative to accelerator hardware execution.

Scenario one: Small multilayer perceptron (MLP) (Overhead Bound)

- **Compute**: 6 ops across small matrix/element-wise operations.
- **Hardware time**: $T_{\text{hw}} \approx 2.6$ µs (mostly memory latency).
- **Software overhead**: $T_{\text{sw}} \approx 6$ ops $\times$ 15 µs/op = 90 µs.
- **Ratio**: 90 µs/2.6 µs ≈ 34.6.
- **Small-model outcome**: The system spends 97 percent of time in host-side dispatch and kernel-launch overhead. Eliminating all modeled dispatch overhead would give an upper-bound speedup of 35.6×.

Scenario two: GPT-3 Layer (Compute Bound)

- **Compute**: Huge matrix multiplications.
- **Assumptions**: Both times below are illustrative round values for one GPT-3-scale layer over a large batch, not derived from a specific accelerator. The conclusion does not hinge on them, because at the chapter's 15 µs/op the ratio would reach 1 only if the layer issued thousands of separately dispatched kernels.
- **Hardware time**: $T_{\text{hw}} \approx 100$ ms = 100000 µs.
- **Software overhead**: $T_{\text{sw}} \approx 50$ µs.
- **Ratio**: 50 µs/100000 µs ≈ 0.0005.
- **Large-model outcome**: Modeled Python overhead is negligible. Compilation may still improve fusion, layout, or backend specialization; dispatch elimination contributes little.

Systems insight: Dispatch overhead is regime-dependent. Compilation can reduce host-side overhead for small-operation workloads by fusing operations into fewer launches, while large models benefit mainly from fused kernels and memory movement reductions.

The dispatch tax analysis demonstrates that small operations become overhead-bound when dispatch latency exceeds device execution time. This systems constraint reaches its logical extreme at the edge of the deployment spectrum, where the memory and latency footprint of the Python runtime exceeds the physical capacity of the target hardware.

7.3.5 Frameworks for the edge: TinyML and micro-runtimes

The compilation continuum reaches its extreme at the far edge. While cloud frameworks such as PyTorch and TensorFlow 2.x prioritize flexibility through eager execution, TinyML[16] systems operating on microcontrollers (MCUs) with kilobytes of memory cannot afford the overhead of a Python interpreter or a fully dynamic runtime.

Lighthouse 7.2: Lighthouse example: KWS on TinyML

Scenario: Deploying the keyword spotting (KWS) lighthouse, a DS-CNN and the volume's Tiny Constraint workload, to an ARM Cortex-M4 microcontroller with 256 KB RAM and 1 MB Flash. This is a smaller target than the ESP32-S3 of the earlier smart-doorbell scenario, chosen to show the runtime at its tightest.

Constraint: A full PyTorch and Python runtime exceeds the device's memory budget by orders of magnitude.

[16] **TinyML**: Systems designed for MCUs cannot afford a Python interpreter. Micro-runtimes instead use small C/C++ runtimes, fixed memory planning, and model-specific operators. TensorFlow Lite Micro remains interpreter-based: the application supplies a tensor arena whose buffers are planned and reused without heap allocation after setup. Predictable memory is mandatory because exceeding a 256 KB device budget can prevent initialization or inference.

Framework solution: Micro-frameworks such as **TensorFlow Lite Micro (TFLM)** (David et al. 2021) solve this through a tiny interpreter-based runtime with a fixed memory discipline:

1. **Fixed memory arena**: The application supplies a contiguous tensor arena, and the framework plans and reuses buffers from that arena rather than relying on dynamic allocation during inference.
2. Kernel selection: Only the specific kernels used by the model (for example, Conv2D, DepthwiseConv) need to be linked or registered with the runtime.
3. **Compact interpreter execution**: The MCU runs a small C/C++ interpreter over a flat model representation, with the model and arena bound at initialization rather than assembled dynamically at runtime.

Silicon contract: On TinyML devices, the binding term is memory capacity, not bandwidth or dispatch overhead, which is why table 7.4 treats it as a feasibility gate. The framework's primary job is to ensure the model's intermediate activations (the "working set") fit within the MCU's tiny SRAM.

These micro-runtimes sacrifice dynamic flexibility to reduce footprint and make memory predictable. With compact models, low-power hardware, and duty cycling, that discipline enables milliwatt-scale inference with mostly on-chip data movement.

The spectrum of execution strategies, from dynamic eager execution to static graph compilation and specialized micro-runtimes, requires developers to make deliberate trade-offs. The key execution-mode decisions summarize these architectural choices:

Checkpoint 7.1: Execution models

The choice of execution mode determines both developer velocity and model performance.

Debuggability vs. speed

- ☐ **Eager mode (Python-first)**: why does executing ops one-by-one make debugging easy but optimization hard?
- ☐ **Graph mode (compiler-first)**: why does building a static graph enable kernel fusion?

Modern compromise

- ☐ **JIT compilation**: how does `torch.compile` bridge the gap?
- ☐ **Dispatch-overhead estimate**: if six operations each incur the chapter's 15 µs/op dispatch tax while hardware work takes 2.6 µs, which term dominates?

The execution problem determines when computation happens and what optimizations are possible. Neural network training, however, requires a capability that no amount of clever scheduling can provide: the ability to compute gradients automatically.

7.4 Differentiation Problem

Training requires determining how an incremental change in each of millions of parameters alters the scalar loss. Doing this manually for even a simple three-layer network requires deriving and implementing dozens of partial derivatives; for a modern transformer with billions of parameters, manual differentiation is intractable. A framework that executes efficiently but cannot differentiate can run inference but cannot learn.

The **differentiation problem** is the task of computing gradients[17] automatically. Neural network training requires derivatives of a scalar loss $\mathcal{L}$ with respect to millions or billions of parameters. When executing the forward pass, the runtime records an execution tape and saves intermediate activation buffers for subsequent derivative evaluations. Backpropagation already showed why reverse-mode automatic differentiation (AD)[18] suits this shape of problem, since one reverse traversal from a single scalar loss yields every parameter gradient, a claim the forward-versus-reverse comparison

[17] **Gradients**: A gradient holds one value for every parameter it differentiates, so each weight tensor acquires a gradient tensor of the same shape. Computing gradients automatically therefore doubles the parameter-sized state before any optimizer runs, which is why every training-memory ledger later in this chapter lists gradients as a full copy of the weights.

[18] **Reverse-mode AD**: Reverse mode's single backward pass has a concrete price, because the operation-dependent values that derivative rules read must be retained until the backward traversal reaches them. Activation checkpointing changes this saved set by omitting selected values and recomputing them during the backward pass.

below makes precise. Major ML frameworks therefore use reverse-mode AD for ordinary scalar-loss training (Baydin et al. 2018).

Building on the backpropagation algorithm, the systems engineering of differentiation addresses how frameworks represent computation graphs, manage memory for intermediate values, and orchestrate the backward pass efficiently across accelerators. The framework mechanizes the chain rule as large-scale systems bookkeeping: tracking operator dependencies, staging activation buffers, and dispatching backward kernels. Listing 7.4 illustrates the core idea with a simple three-operation function.

↪ The optimization algorithms that consume these gradients are developed in section 8.4.2.

Listing 7.4: Automatic Differentiation: AD decomposes complex functions into elementary operations with known local derivatives, allowing frameworks to apply the chain rule over a recorded operation graph.

```
def f(x):
    a = x * x  # Square
    b = sin(x)  # Sine
    c = a * b  # Product
    return c
```

Frameworks decompose this function into elementary operations, each with a known local derivative, and then combine these local derivatives via the chain rule to compute gradients through supported compositions. The systems challenge is executing this transformation under physical hardware constraints: the framework must represent the computation graph, make necessary forward values available, and schedule backward kernels without exhausting accelerator HBM. Production runtimes address these challenges through structured graph recording, caching allocators, and reverse-mode traversal.

7.4.1 Forward and reverse mode differentiation

Automatic differentiation operates in two distinct traversal directions: forward mode and reverse mode. The choice between them governs whether gradient evaluation cost scales with the number of input dimensions or output dimensions. Because deep learning objectives evaluate a single scalar loss with respect to millions or billions of parameters, this mathematical asymmetry dictates the memory footprint and execution schedule of the underlying framework.

7.4.1.1 Forward mode

Forward mode automatic differentiation computes directional derivatives concurrently with the primal evaluation. For each intermediate variable v_i, the runtime evaluates both the primal value v_i and its directional derivative (tangent) $\dot{v}_i = \sum_j \frac{\partial v_i}{\partial x_j} \dot{x}_j$. Because tangents flow downstream alongside primal values, forward mode requires no execution tape and never retains forward activations for a subsequent backward pass.

To see the mechanism concretely, consider computing both the value and derivative of $f(x) = x^2 \sin(x)$. Listing 7.5 shows how forward mode evaluates local derivative rules alongside each elementary operation, applying the chain and product rules at every step:

Listing 7.5: Forward Mode AD: Propagates derivatives forward through the computation graph, computing one directional derivative per seeded forward pass.

```
def f(x):  # Computing both value and derivative
    # Step 1: x -> x²
    a = x * x  # Value: x²
    da = 2 * x  # Derivative: 2x

    # Step 2: x -> sin(x)
    b = sin(x)  # Value: sin(x)
    db = cos(x)  # Derivative: cos(x)

    # Step 3: Combine using product rule
    result = a * b  # Value: x² * sin(x)
    dresult = a * db + b * da  # Derivative: x²*cos(x) + sin(x)*2x

    return result, dresult
```

Forward mode mechanizes this paired tracking through dual numbers—values of the form $v + \dot{v}\epsilon$ where $\epsilon^2 = 0$. Evaluating an arithmetic operator or elementary function over dual numbers automatically yields the updated primal and tangent without graph retention. Listing 7.6 demonstrates this trace evaluated at $x = 2.0$ with seed $\dot{x} = 1.0$:

Listing 7.6: Dual-Number Trace: The forward-mode function of listing 7.5 evaluated at $x = 2.0$. Each line keeps a value and its derivative, so one pass yields both $f(2.0) \approx 3.637$ and $f'(2.0) \approx 1.973$.

```
x, dx = 2.0, 1.0  # seed: track the derivative with respect to x

a = x * x  # value:      4.0
da = 2 * x  # derivative: 4.0
b = sin(x)  # value:      0.9093
db = cos(x)  # derivative: -0.4161
c = a * b  # value:      3.637
dc = a * db + b * da  # derivative: 1.973
```

A single forward pass with a seeded tangent vector $\dot{\mathbf{x}}$ evaluates a Jacobian-vector product (JVP), $\mathbf{J}\dot{\mathbf{x}}$, where $\mathbf{J} \in \mathbb{R}^{M \times P}$ is the Jacobian matrix. When an objective maps P parameter inputs to a single scalar loss ($M = 1$), recovering the full gradient $\nabla_\theta \mathcal{L} \in \mathbb{R}^P$ requires setting $\dot{\mathbf{x}}$ to each of the P standard basis vectors in turn. Recovering gradients with forward mode therefore requires P distinct forward evaluations (or propagating P-wide tangent bundles), scaling compute and memory traffic as $\mathcal{O}(P)$ with parameter count. For networks with millions or billions of parameters, this scaling makes forward-mode gradient calculation intractable, although forward JVPs remain highly effective when input dimensions are small or for evaluating directional derivatives and Hessian-vector products.

7.4.1.2 Reverse mode

Reverse-mode automatic differentiation inverts the traversal order to exploit the many-to-one geometry of neural network training. Instead of propagating tangents downstream from inputs, reverse mode propagates adjoints $\bar{v}_i = \partial \mathcal{L} / \partial v_i$ upstream from the scalar loss $\mathcal{L}$ to all parameters.

Mathematically, while forward mode computes Jacobian-vector products ($\mathbf{J}\dot{\mathbf{x}}$), reverse mode evaluates vector-Jacobian products (VJPs), $\mathbf{v}^T\mathbf{J} = \sum_{j=1}^{M} \bar{y}_j \, \partial y_j / \partial \mathbf{x}$, where $\bar{y}_j = \partial \mathcal{L} / \partial y_j$, one element of $\mathbf{v}$. A VJP multiplies an upstream gradient vector by the local operation's Jacobian.

Evaluating VJPs avoids a prohibitive memory cost. Consider an intermediate layer mapping an input vector $\mathbf{x} \in \mathbb{R}^N$ to an output $\mathbf{y} \in \mathbb{R}^M$, where $N = M = 100{,}000$. The full Jacobian $\mathbf{J} \in \mathbb{R}^{M \times N}$ contains 10^{10} elements, requiring 40 GB of memory to store a single layer's derivatives in FP32. Backpropagation, however, never needs this full matrix. Because training optimizes a single scalar loss $\mathcal{L}$, the backward pass only ever needs to propagate an upstream gradient vector $\mathbf{v} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^M$.

To see how VJPs sidestep the massive matrix, consider a simple element-wise operation like $\mathbf{y} = \text{ReLU}(\mathbf{x})$:

1. **The inputs**: An input vector $\mathbf{x} = [-1.0, 2.0, 3.0]$ and an upstream gradient $\mathbf{v} = [0.5, 0.5, 0.5]$ flowing backward from the loss.
2. **The local derivative**: The derivative of ReLU is 1 if $x > 0$, else 0. The local gradient is $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = [0, 1, 1]$.
3. **The VJP**: Instead of building a 3×3 diagonal matrix and executing a matrix multiplication, the VJP computes an element-wise product: $\mathbf{v} \odot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = [0.0, 0.5, 0.5]$.

By evaluating the product directly, the runtime streams vectors without ever materializing the massive $M \times N$ matrix in memory. Seeding the reverse sweep with $\bar{\mathcal{L}} = 1$ yields the complete parameter gradient vector $\nabla_\theta \mathcal{L}$ in a single backward pass, regardless of whether the model contains ten parameters or ten billion.

Reverse mode operates on the same three-operation function $f(x) = x^2 \sin(x)$ from listing 7.4, where x reaches the output through two distinct computational paths: the square and the sine. Algorithm 7.1 defines the reverse-mode traversal algorithm over a directed acyclic graph before tracing this concrete evaluation.

Algorithm 7.1 Reverse-mode automatic differentiation

Require: directed acyclic graph with scalar output y; nodes $v_1, \ldots, v_n$ in topological order; a local derivative rule for each node
Ensure: adjoints $\bar{v}_i = \partial y / \partial v_i$ for the requested input or parameter nodes

for $i = 1$ to n **do**
  compute v_i, record dependencies, and save values requested by its derivative rule ▷ *forward pass*
end for
$\bar{v}_i \leftarrow 0$ for all i; $\bar{y} \leftarrow 1$
for $i = n$ down to 1 **do**
  for each parent u of v_i **do**
    $\bar{u} \leftarrow \bar{u} + \bar{v}_i \, \partial v_i / \partial u$ ▷ *accumulate via the chain rule*
  end for
  release stored values once every rule that needs them has run
end for
return the adjoints for the requested nodes

The algorithmic structure of algorithm 7.1 reveals its systems cost. The forward pass records operator dependencies and buffers intermediate values required by backward derivative rules until the reverse sweep consumes them. Reverse mode trades activation storage and graph bookkeeping for a single backward pass, whose cost is a small constant multiple of the forward pass regardless of parameter count.

For the concrete function in listing 7.4 evaluated at $x = 2.0$, the forward pass records $a = x^2 = 4.0$, $b = \sin(x) \approx 0.9093$, and $c = ab \approx 3.637$. Seeding $\bar{c} = 1.0$, the product rule yields adjoints $\bar{a} = 0.9093$ and $\bar{b} = 4.0$. The square branch contributes $0.9093 \cdot 4.0 \approx 3.6372$ to $\bar{x}$, while the sine branch contributes $4.0 \cos(2.0) \approx -1.6646$. Accumulating both paths gives the exact derivative $\partial c / \partial x = \bar{x} \approx 1.973$.

This single backward traversal computes $\partial c / \partial x$ regardless of how many fan-out paths connect x to c. In a deep neural network, every weight tensor influences the scalar loss through thousands of divergent paths across successive layers; reverse mode aggregates all incoming adjoints in one reverse sweep. That one sweep, rather than one pass per parameter, is what keeps training a 175B-parameter model such as GPT-3 computationally tractable.

Executing this traversal in hardware exposes a physical constraint: evaluating local derivatives during the backward pass requires the operating point established during forward execution. The framework must therefore determine which forward tensors to persist in device memory, when to allocate them, and when to free them. The recorded graph answers the "when to free" question, because each saved tensor can be released as soon as the last backward rule that reads it has run. The peak itself is bounded separately, by splitting a large batch into micro-batches whose saved

activations are freed before the next one starts while their gradients sum into the same buffers (gradient accumulation).[19]

Listing 7.7 illustrates these requirements in a two-layer network, showing the forward pass storing intermediate activations and the backward pass consuming them to evaluate parameter gradients.

[19] **Gradient accumulation**: The framework processes a large logical batch as smaller mini-batches, freeing each mini-batch's saved activations after backward while accumulating parameter-sized gradients. Processing 4,096 samples in 64-sample mini-batches can reduce the batch-dependent activation peak by up to 64× when activation memory scales linearly with batch size; weights, gradients, optimizer state, and other fixed costs do not shrink. The trade-off is more sequential work before each optimizer update.

Listing 7.7: Reverse Mode in a Neural Network: The forward pass computes and stores intermediate values; the backward pass walks the computation in reverse to produce gradients for every parameter.

```
def simple_network(x, w1, w2):
    hidden = x * w1  # First layer
    activated = max(0, hidden)  # ReLU activation
    output = activated * w2  # Second layer
    return output


# --- Forward pass stores intermediates ---
x, w1, w2 = 1.0, 2.0, 3.0
hidden = x * w1
activated = max(0, hidden)
output = activated * w2

# --- Backward pass consumes them ---
d_output = 1.0  # Seed gradient
d_w2 = activated  # = 2.0
d_activated = w2  # = 3.0
d_hidden = d_activated * (1 if hidden > 0 else 0)  # ReLU gate: 3.0
d_w1 = x * d_hidden  # = 3.0
d_x = w1 * d_hidden  # = 6.0
```

Three implementation requirements emerge from this example. First, the framework must track dependencies between operations to determine the correct reverse traversal order. Second, the information needed by each backward rule must be available, either saved during the forward pass or regenerated. The chain rule evaluates local derivatives at the forward operating point—for example, evaluating $\frac{\partial}{\partial z}\text{ReLU}(z) = \mathbb{I}(z > 0)$ requires the sign of the pre-activation z, which can be recovered from a saved value or mask, and computing a dense layer's weight gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}}$, where $\mathbf{Z}$ is the layer's pre-activation output, requires the forward input activation tensor $\mathbf{X}$. Third, every differentiated operation needs both a forward implementation and a corresponding backward rule. These requirements define the engineering surface of automatic differentiation, where retaining forward values can consume much of the training memory budget.

7.4.1.3 Memory management strategies

A 175B-parameter model in FP16 requires 350 GB just for weights, exceeding the physical memory of any single accelerator. Reverse-mode AD compounds this memory pressure by retaining operation-dependent forward activations for backward derivative rules. Even a model whose weights fit easily pays this tax. In the illustrative ResNet-50 scenario of section 7.4.2.1, batch-64 saved activations exceed the FP32 weights by roughly two orders of magnitude, on top of weights, gradients, and optimizer states. Memory capacity, rather than raw floating-point throughput, frequently establishes the feasibility boundary of training.

This saved-activation footprint accumulates across successive layers. Listing 7.8 demonstrates how each forward layer adds an activation tensor that must persist until the backward sweep unwinds to that layer.

Listing 7.8: Activation Persistence Across Layers: The marked intermediates represent values retained for backward; peak saved-activation memory depends on their number and sizes.

```
def deep_network(x, w1, w2, w3):
    # Forward pass - must store intermediates
    hidden1 = x * w1
    activated1 = max(0, hidden1)  # Store for backward
    hidden2 = activated1 * w2
    activated2 = max(0, hidden2)  # Store for backward
    output = activated2 * w3
    return output
```

This is a capacity limit rather than the bandwidth gap of the memory wall, and frameworks relieve it through two primary architectural strategies. The first is activation checkpointing (or gradient checkpointing): rather than buffering every intermediate activation, the runtime stores only selected segment boundary tensors and recomputes dropped intermediates on demand during the backward pass. The essential systems mechanism is the runtime contract: the framework marks boundary activations as persistent and regenerates intermediate values when backward execution reaches that segment. Listing 7.9 illustrates this contract: the forward pass preserves only segment boundaries, and the backward pass re-executes the forward segment to regenerate activations rather than loading them from global memory.

↪ Activation checkpointing schedules and their recomputation cost are operationalized in section 8.7.5.2.

Listing 7.9: Activation Checkpointing: The forward pass stores only segment boundaries; the backward pass re-runs each segment to regenerate the dropped activations, trading recomputation for memory.

```
# Standard backward: every forward activation stays resident
h1 = layer1(x)  # kept for backward
h2 = layer2(h1)  # kept for backward
out = layer3(h2)  # kept for backward

# Checkpointed: keep only the boundary h1 and drop h2. The
# backward pass re-runs the wrapped segment to regenerate
# h2 on demand instead of holding it in memory.
h1 = layer1(x)  # boundary: kept
out = checkpoint(
    lambda a: layer3(layer2(a)), h1
)  # h2 recomputed in backward
```

The second strategy is the kernel fusion that the memory wall motivated in section 7.3.1, applied here to the body of a layer. Its capacity benefit is narrower than its traffic benefit. Temporaries that pass only between the fused operators are never allocated in HBM, so they add nothing to peak memory, while any value a backward rule reads must still be stored.[20] Rather than executing matrix multiplication, bias addition, and activation functions as separate kernels that serialize intermediate tensors to accelerator global memory (HBM), compilers fuse compatible operators into a single kernel that passes intermediate values through registers or on-chip shared memory.

The backward pass itself relies on specialized hardware-optimized kernels. Rather than synthesizing backward passes from naive elementary operations, frameworks invoke tuned backward kernels (such as cuDNN primitives) that optimize memory access strides and register usage on accelerator hardware (Chetlur et al. 2014). Combining checkpointing, kernel fusion, and specialized backward kernels enables large architectures to train within physical accelerator memory limits without incurring out-of-memory faults.

[20] **Fusion and saved state**: Fusion removes the forward round-trip of an intermediate through HBM, but it does not remove the backward pass's need for that value. If a derivative rule reads a fused intermediate, such as the pre-activation whose sign forms the ReLU mask, the fused kernel must still write that value (or a compact mask) to memory, or the backward pass must recompute it. Fusion and checkpointing therefore interact, because the compiler decides which intermediates to materialize from what the backward graph will read.

7.4.2 Framework implementation of automatic differentiation

Checkpointing, fusion, and specialized kernels address the physical bottlenecks of automatic differentiation, but production frameworks encapsulate these mechanisms beneath high-level training loops. A standard PyTorch step—`optimizer.zero_grad()`, forward pass, `loss.backward()`, and `optimizer.step()`—appears as four sequential calls. Behind this interface, the runtime builds

and traverses dynamic computation graphs, allocates and frees intermediate buffers, schedules backward kernels across CUDA streams, and manages numerical precision. The same graph machinery supports second-order derivatives via nested `torch.autograd.grad` calls and automatic mixed precision via `autocast`, which selects reduced-precision kernels for eligible operations while retaining higher precision where its policy requires it. Operation precision and the optimizer's persistent weight-storage recipe are separate choices.

7.4.2.1 PyTorch autograd internals

The autograd system is the engine that resolves the differentiation problem introduced in section 7.1. Three systems principles govern its design: the data structure that enables efficient gradient computation, the memory cost of maintaining that data structure, and the control mechanisms that production workflows require. Reasoning through these principles clarifies why training memory substantially exceeds inference memory for identical architectures, and how frameworks manage that overhead.

The reverse-linked graph structure During the forward pass, autograd dynamically constructs a directed acyclic graph of `Node` (or `Function`) objects. Each node encapsulates an operator's backward differentiation logic and retains references to the tensors required to evaluate its vector-Jacobian products. Because edges point backward from outputs toward inputs, the engine initiates topological traversal immediately upon receiving the scalar loss adjoint.

When gradient recording is enabled and an operation has inputs requiring gradients, its output carries a `grad_fn` referencing the backward `Function`. Each `Function` links to antecedent nodes through its `next_functions` list, forming a chain from the scalar loss back to leaf parameter tensors. Listing 7.10 inspects this pointer chain for a simple composite computation:

Listing 7.10: Reverse-Linked Graph Structure: Recorded outputs link through `grad_fn` to backward functions, forming a reverse chain from output to leaf parameters. One backward traversal computes the scalar loss gradient; its work grows with the graph.

```
import torch

x = torch.tensor([2.0], requires_grad=True)
y = x * 3
z = y.pow(2)


# Traverse the reverse-linked graph
print(z.grad_fn)  # PowBackward0
print(z.grad_fn.next_functions)  # -> MulBackward0
print(
    z.grad_fn.next_functions[0][0].next_functions
)  # -> AccumulateGrad (leaf)
```

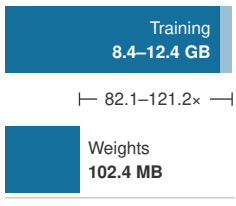
The inspection exposes the traversal path: `PowBackward0` (from `z = y.pow(2)`) links to `MulBackward0` (from `y = x * 3`), which terminates at `AccumulateGrad` on the leaf parameter tensor `x`. Leaf tensors are graph sinks where accumulated gradients write directly into the `.grad` buffer rather than propagating further. The tuple format `(Function, index)` tracks the specific output index for multi-output operations.

This reverse-linked graph imposes a lifetime constraint: the topology and the forward information required by each backward rule must remain available until that rule executes. Host-side graph metadata (pointers and node descriptors) is distinct from the device tensor buffers it references. Retaining those buffers can consume gigabytes of accelerator capacity as model depth and batch size grow; recomputation or offloading changes where and how the required values are made available.

The memory-compute trade-off In a retained-activation implementation, saved device tensors remain live until their backward consumers finish. These retained activations represent a primary driver of training memory overhead, compounding the footprint of weights, gradients, and optimizer states. Because different operations require different forward information—for example, a matrix multiplication weight gradient requires input activations, while elementwise ReLU needs only

the pre-activation sign—the exact memory profile depends on operator semantics and how the implementation stores that information.

Consider an illustrative ResNet-50 training scenario with 25.6M parameters (~102.4 MB of FP32 weights), batch size 64, and 224×224 images. With 8 GB–12 GB of saved activations, gradients add ~102.4 MB, and two Adaptive Moment Estimation (Adam) FP32 moment buffers add ~204.8 MB. Summing those four components gives an 8.4 GB–12.4 GB training scenario, roughly 82.1–121.2× the FP32 weight storage alone. This is a capacity-planning sum; actual peak memory depends on which components are live together. Inference requires no optimizer states, gradients, or saved backward buffers, but still has live activations and runtime workspace.



The training memory sum dwarfs FP32 weight storage alone.

This disparity underscores why accelerator training capacity cannot be deduced from parameter counts alone. Total training memory obeys the four-component formulation ($M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$) plus allocator overhead, as stated in section C.3.3.

Frameworks provide three graph-level controls to manage this trade-off. Activation checkpointing (T. Chen et al. 2016), whose runtime contract listing 7.9 showed, alters what the graph retains: the engine stores selected segment boundaries and regenerates intermediate activations during the backward pass. In iron law terms, checkpointing increases the arithmetic operation count (O) to reduce peak activation storage; its effect on transfer volume (D_{vol}) depends on the recomputation path. Tensor detachment sets a gradient boundary: calling `.detach()` makes subsequent operations treat that tensor as independent of its prior computation. It does not retroactively free all previously saved state, and cutting a path changes which gradients are computed. Transfer learning shows the payoff. When a frozen feature extractor runs with gradient recording disabled and its output enters the trainable head as a constant, autograd saves none of the extractor's activations, so only the head pays the activation-storage cost. Mixed-precision training provides a third lever: representing eligible activations and weights in 16-bit formats halves their bytes relative to FP32, reducing tensor volume (D_{vol}) and enabling higher arithmetic throughput on supported Tensor Cores (R_{peak}).

Extensibility and control Production training systems require fine-grained control over gradient propagation beyond standard backward sweeps. In practice, workloads demand three core capabilities: selective gradient computation for freezing layers during transfer learning via `requires_grad=False` or `.detach()`; gradient inspection and modification for clipping, debugging, or logging intermediate adjoints via runtime hooks; and custom differentiation rules for non-standard operations such as specialized CUDA kernels or novel activation functions.

↪ The decision of when these single-machine levers stop being enough and training must scale beyond one machine is developed in section 8.8.3.

These extension points operate via callbacks invoked during graph construction and reverse traversal, preserving the core engine's execution path. In PyTorch, these capabilities map to specific runtime mechanisms: retained graphs, gradient accumulation buffers, custom `autograd.Function` implementations, tensor hooks, and detachment boundaries. Table 7.5 summarizes these mechanisms, their systems function, and their memory or correctness trade-offs.

Table 7.5: Autograd Control Mechanisms: The recurring extension points the autograd engine exposes during graph traversal, what each controls, and the memory or correctness cost each carries.

Mechanism	What it controls	Cost or hazard
`retain_graph=True`	Keeps saved graph state available for another backward traversal	Retains memory that would otherwise be released; the amount depends on the graph and saved values
Gradient accumulation	Successive backward passes sum into `.grad` until `zero_grad()` resets them, enabling large effective batches	Forgetting the reset silently mixes gradients across optimization steps
Custom `autograd.Function`	User-defined forward and backward rules for operations outside the built-in library	Moves the differentiation contract (what to save, how to differentiate) to the implementer
Gradient hooks	Inspecting or modifying gradients mid-traversal (clipping, logging, debugging)	Runs arbitrary Python per registered tensor on every backward pass
`.detach()`	Cuts gradient flow at a chosen boundary (frozen layers, inference outputs)	The legacy `.data` attribute bypasses autograd and silently corrupts gradients; clone before in-place mutation

Custom autograd functions expose the differentiation contract directly by transferring state retention responsibility from the engine to the programmer: the implementation explicitly defines which forward tensors to preserve in `ctx.save_for_backward()` and evaluates the corresponding vector-Jacobian products in `backward()`. Listing 7.11 demonstrates this interface for a custom multiply-add operation:

Listing 7.11: Custom Autograd Function: Implement forward and backward methods to define custom differentiable operations, explicitly specifying tensors to save for gradient computation.

```
import torch


class MultiplyAdd(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y, z):
        # Save tensors needed for backward
        ctx.save_for_backward(x, y)
        return x * y + z

    @staticmethod
    def backward(ctx, grad_output):
        # Retrieve saved tensors
        x, y = ctx.saved_tensors

        # Compute gradients using chain rule
        grad_x = grad_output * y  # dL/dx = dL/dout * dout/dx
        grad_y = grad_output * x  # dL/dy = dL/dout * dout/dy
        grad_z = grad_output  # dL/dz = dL/dout * 1

        return grad_x, grad_y, grad_z


# Usage
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)
z = torch.tensor([1.0], requires_grad=True)

output = MultiplyAdd.apply(x, y, z)
output.backward()

print(
    x.grad, y.grad, z.grad
)  # tensor([3.]), tensor([2.]), tensor([1.])
```

These autograd mechanisms connect directly to the silicon contract. The reverse-linked graph defines the sequence of backward kernels the device must execute (the O term). The memory-compute trade-off governs intermediate tensor residency and memory traffic through the memory hierarchy (the D_{vol} term). The extensibility hooks allow engineers to prune unnecessary graph paths and intercept data flow. The interplay between graph retention and memory bandwidth leads directly to mixed-precision training, which reduces memory footprint while increasing hardware execution efficiency.

7.4.2.2 Mixed-precision training support

Mixed precision exploits hardware specialization to improve two iron law terms simultaneously: Tensor Cores execute 16-bit matrix multiplications at higher throughput than FP32 arithmetic units (increasing R_{peak} and reducing $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), while eligible 16-bit activations halve their stored bytes and can reduce global memory traffic (reducing D_{vol}). The realized traffic reduction depends on execution. These two opportunities make mixed precision a common early optimization candidate.

Automatic mixed-precision APIs implement this as a dispatch policy rather than a change to model code. Inside an `autocast` region, the dispatcher consults a per-operator precision list before selecting a kernel. Compute-intensive operations such as convolutions and linear projections run in FP16 or

BF16, while numerically sensitive operations such as softmax, reductions, and loss calculations run in FP32. The second framework component, `GradScaler`, compensates for FP16's narrow exponent range (5 bits, with normal values bounded below by $\approx 6.10 \times 10^{-5}$). It multiplies the scalar loss by a loss-scale factor s_{loss} before backward traversal so that small gradients remain representable, divides the parameter gradients by s_{loss} before the optimizer update, and skips any step whose gradients overflow.

Frameworks support three primary reduced-precision formats: IEEE FP16, BF16, and TF32 (an NVIDIA Ampere+ internal format that pairs FP32's 8-bit exponent with FP16's 10-bit mantissa for matrix multiplies). BF16 keeps FP32's 8-bit exponent at the expense of mantissa precision (7 bits), so BF16 recipes usually run under `autocast` with the scaler disabled. Section 8.7.3 covers the policy side of these mechanisms: when to choose each format, how the loss scale adapts during training, and what mixed precision does to the memory budget. Listing 7.12 shows the two framework components working together in the FP16 path.

Listing 7.12: Mixed-Precision API: Modern frameworks provide automatic mixed-precision support through context managers that handle precision selection and numerical stability.

```
import torch
from torch.amp import autocast, GradScaler

model = MyModel().cuda()
optimizer = torch.optim.Adam(model.parameters())
scaler = GradScaler("cuda")

for inputs, targets in dataloader:
    inputs, targets = inputs.cuda(), targets.cuda()
    optimizer.zero_grad()

    # Framework automatically selects precision per operation
    with autocast(device_type="cuda", dtype=torch.float16):
        outputs = model(inputs)
        loss = criterion(outputs, targets)

    # GradScaler handles gradient scaling for numerical stability
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Optimizer state and training checkpoints A training checkpoint is a different mechanism from the activation checkpointing above. It is a file on storage, written every so often so that training can resume after preemption or hardware failure, and resuming requires restoring model weights and optimizer states in lockstep: first-moment vectors, second-moment variance estimates, and step counters. In the mixed-precision Adam recipe modeled here, a checkpoint holds the FP16 working weights (2 bytes per parameter), the FP32 master weights (4 bytes), and the two FP32 moments (8 bytes), 14 bytes per parameter; the FP16 gradients, which every step recomputes, are left out of the 16-byte training-state ledger that section 8.5.4.1 budgets. That 16-byte total matches the FP32 Adam ledger of section 5.5.5.4 but divides differently, 2 + 2 + 4 + 8 bytes rather than 4 + 4 + 8, because the mixed-precision recipe trades full-precision weights and gradients for 16-bit working copies plus an FP32 master. This explicit weight-copy recipe is distinct from choosing operation dtypes with autocast. For a 7-billion-parameter model, its checkpoint tensor payload therefore carries approximately 98 GB (14 GB + 28 GB + 56 GB), 7× the FP16 weights alone. Checkpoint volume directly governs recovery latency across distributed storage, connecting fault tolerance to the iron law's D_{vol} term.

↪ How to budget and reduce optimizer memory during training is developed in section 8.5.4.1.

At the framework level, optimizers expose internal state dictionaries via `state_dict()` for serialization. Listing 7.13 demonstrates accessing optimizer state dictionaries, and listing 7.14 shows saving and restoring model parameters and optimizer states. Reproducing the training trajectory also requires the relevant random-number state, data position, learning-rate scheduler, and gradient-scaler state.

Listing 7.13: State Dictionary Interface: Optimizers expose internal state through state_dict(), enabling serialization of momentum buffers and adaptive learning rate estimates for checkpointing.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(10, 5)
optimizer = optim.Adam(model.parameters(), lr=0.001)

# After training steps, optimizer accumulates state
loss = model(torch.randn(3, 10)).sum()
loss.backward()
optimizer.step()

# Access state for checkpointing
state = optimizer.state_dict()
# Contains: {'state': {...}, 'param_groups': [{'lr': 0.001, ...}]}
```

Listing 7.14: Checkpoint Save and Load: Save both model parameters and optimizer state to properly resume training with correct momentum and adaptive learning rate values.

```
# Saving checkpoint
checkpoint = {
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
}
torch.save(checkpoint, "checkpoint.pt")

# Resuming training
checkpoint = torch.load("checkpoint.pt")
model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

A checkpoint holds only persistent state. During a training step, the FP16 gradients rejoin that ledger in device memory and the saved activations from reverse-mode AD sit on top of it, so device memory must hold more than the checkpoint file suggests. Notebook 7.3 prices the full accounting for a billion-parameter model.

Napkin Math 7.3: The administrative tax

The illustrative ResNet-50 breakdown in section 7.4.2.1 was ~82.1–121.2× larger than FP32 weight storage alone. The following scenario expands the accounting to gradients, optimizer state, and saved activations at billion-parameter scale.

Problem: Calculate the cumulative memory overhead that gradients, optimizer states, and forward activations add beyond raw model weights in this billion-parameter training scenario.

Math:

1. Model weights (FP16): 2 GB.
2. Gradients (FP16): 2 GB (same size as weights).
3. Master weights (FP32): 4 GB, the full-precision copy that mixed-precision training updates.
4. Optimizer states (Adam): 8 GB (momentum and variance in FP32, 4× the FP16 weight memory).

5. Activations: For a batch size of 32 and a 100-layer network, reverse-mode AD retains the operation-dependent values required by derivative rules; activation checkpointing can omit selected values and recompute them during the backward pass.

$$M_{\text{activations}} \approx B \times N_L \times S \times d \times n_{\text{saved}} \times s_{\text{elem}}$$

For a 1024-token sequence, 1024-wide hidden state, 1 saved tensor per layer, and $s_{\text{elem}} = 2$ bytes for FP16: $32 \times 100 \times 1024 \times 1024 \times 1 \times 2 \approx$ **6.7**GB. Materialized attention terms can add a separate $B \times N_{\text{heads}} \times S^2$ component, which is why memory-efficient attention kernels matter.

Systems insight: A 2 GB model carries 16 GB of steady-state training state once gradients and optimizer buffers have been allocated, 8× its FP16 weights (16 bytes per parameter in the mixed-precision Adam recipe). During a training step, batch-dependent activations add another 6.7 GB, raising the peak "administrative tax" to ~20.7 GB beyond the weights. During training, data movement includes saving and retrieving these activations.

The mathematical foundations of automatic differentiation predated the deep learning resurgence by decades; the engineering challenge was mechanizing those principles into high-performance software infrastructure. Manually deriving and writing backward kernels for a single layer requires tracking intermediate variables, implementing local chain rules, and managing device buffer lifecycles. For modern transformer architectures containing multi-head self-attention, layer normalizations, and residual projections across hundreds of layers, manual differentiation across thousands of tensor dependencies is infeasible. Autograd engines mechanize this derivation across arbitrary topologies, precision modes, and accelerator backends, transforming an intractable analytical problem into automated graph execution.

7.4.2.3 Tensor lifetimes and the caching allocator

Checkpointing and fusion (section 7.4.1.3) decide which values exist during a training step. The runtime still has to decide when each value's memory is reclaimed and how fresh memory is obtained. In the retained-activation scheme of listing 7.8, forward layers add saved tensors that remain live until backward traversal consumes them. This saved-activation footprint peaks at the onset of backpropagation and recedes as intermediate buffers are released. Total training memory can peak later as gradients and temporary workspaces become live. Modern runtimes reclaim saved tensors once their backward consumers and other references no longer need them.

Beyond tensor lifetimes, frameworks face a lower-level systems bottleneck: driver-level allocation latency. Invoking synchronous device memory allocations (such as `cudaMalloc`) incurs substantial OS and driver overhead, forcing host-device synchronizations that drain accelerator command queues. To eliminate this latency, modern frameworks intercept allocations with **caching allocators** that maintain pools of pre-reserved device memory blocks across training iterations. Suballocating from these pools amortizes device allocation costs and avoids the synchronizations that `cudaMalloc` can force, though varying tensor lifetimes can still induce memory fragmentation.

Systems Perspective 7.2: Caching allocator and utilization

Caching allocators primarily reduce allocation overhead and improve block reuse. Two considerations remain:

1. Allocation latency: Reusing a cached block avoids repeated device-allocation calls, whose cost and synchronization behavior depend on the allocator and runtime.
2. Fragmentation: Size classes and block splitting improve reuse but can leave free capacity in blocks that do not satisfy a request. Because tensors require contiguous physical addresses, external fragmentation occurs when variable tensor lifetimes leave small, non-contiguous free blocks in the pool; a large allocation can trigger an out-of-memory

(OOM) error even when the total aggregate free memory across all blocks exceeds the requested size.

An out-of-memory error can reflect live allocations, allocator-reserved blocks, fragmentation, or another process. `nvidia-smi` reports device-level usage and is not sufficient by itself to diagnose the framework allocator; allocator-specific memory statistics are needed.

7.4.2.4 Production system integration challenges

A training iteration can run more slowly in production than in an isolated profile because the full step coordinates with the memory allocator, device manager, operation scheduler, data pipeline, and optimizer. The full-system cost includes work that an operator microbenchmark or isolated kernel profile does not measure. Gradient computation launches accelerator kernels and may require allocation, synchronization, or data movement according to the graph and implementation. These interactions can dominate small-operation workloads and remain measurable at scale. The gap between what the programmer writes (a five-line training loop) and what the system executes (allocations, kernel launches, and synchronization points) is the central tension of AD system design.

The severity of this overhead depends on a deeper architectural choice: how the AD system records and replays computation in the first place. A framework that builds a dynamic trace at runtime pays per-operation bookkeeping on every forward pass, and its caching allocator must accommodate allocation patterns that can vary with execution. A framework that captures a stable computation, by contrast, may pre-plan buffers and reuse storage across the forward and backward graphs because it can reason about lifetimes before execution. The gain depends on capture coverage, tensor lifetimes, shapes, and backend support.

Systems Perspective 7.3: Tape-based vs. transform-based autodiff

PyTorch (tape-based): Records operations on a dynamic "tape" during the forward pass. This is flexible and easy to debug but makes it hard for a compiler to see the whole graph at once for global optimization.

JAX (transform-based): Treats automatic differentiation as a high-level function transformation (`grad(f)`). JAX traces a pure function for abstract input signatures, allowing compatible transformations such as `jit(grad(f))` or `vmap(grad(f))` to compose before lowering. This exposes a larger transformed program to accelerator compilers without guaranteeing a single kernel or a universal speed advantage.

The AD system tracks dependencies so it can traverse independent graph branches in a valid order and accumulate gradients correctly. While independent host threads or explicit CUDA streams can expose device concurrency, default autograd dispatch serializes node execution across the primary compute stream unless explicitly directed otherwise. Concurrency is bounded by stream dependencies and hardware scheduling limits.

7.4.2.5 How different frameworks implement AD

The execution models analyzed in section 7.3—eager, static graph, and hybrid—directly dictate how frameworks implement automatic differentiation. PyTorch (Paszke et al. 2019) constructs its autograd tape dynamically during forward execution. Its `grad_fn` chain mechanism, examined in section 7.4.2.1, grants unconstrained Python control flow and transparent breakpoint debugging, but forces the engine to retain tape metadata and saved tensors until backward traversal completes. Conversely, early TensorFlow (Abadi et al. 2016) performed symbolic differentiation ahead of time during graph construction, generating an immutable backward sub-graph amenable to whole-graph dead-code elimination and memory reuse before execution. Modern TensorFlow 2.x adopts eager execution as its default interface while exposing `tf.function` to compile static differentiation graphs when throughput requires it (TensorFlow Developers 2024a). JAX[21] (Frostig et al. 2018) rejects tape tracking entirely, treating differentiation as a pure function transformation: `jax.grad()` accepts a function and produces a derivative function, enabling seamless algebraic composition

[21] **JAX**: JAX exposes `grad`, `jit`, and `vmap` as transformations on functions; they can be composed when their input and output contracts align, but order matters: vectorizing gradients is not generally the same operation as differentiating a vector-valued result. Compilation can produce fused regions, library calls, and multiple kernels rather than one universal kernel. Transformed functions should be pure; Python side effects may occur during tracing but are not represented in the compiled program, while supported callbacks express intentional runtime effects.

with vectorization (`jax.vmap()`) and XLA compilation (`jax.jit()`). While this functional paradigm demands strict referential transparency, it exposes clean whole-program IR to the compiler without the stateful side effects of imperative tapes.

Autodiff implementation differences determine how much debugging visibility, compiler optimization, and deployment portability a team can expect from each framework. A recurring tension runs through every AD design decision: mathematical correctness demands access to the forward information required by backward rules, but hardware imposes strict memory limits. Every framework resolves this tension differently, choosing which activations to checkpoint, which operations to fuse, and how aggressively to trade recomputation for memory. These choices determine which models can train on which hardware, making AD system design one of the most consequential engineering decisions in any framework.

Checkpoint 7.2: The systems cost of gradients

Training is inherently more expensive than inference because of automatic differentiation.

Computational reality

- ☐ **Reverse mode AD**: why is it efficient for a scalar loss with many parameters?
- ☐ **The activation tax**: can you explain how saved-value memory depends on graph depth, tensor shapes, and checkpointing?

Optimization mechanics

- ☐ **Gradient checkpointing**: how does recomputing activations save memory?

The execution and differentiation problems together enable the training loop: the execution model determines when computation happens, while automatic differentiation computes the gradients that drive learning. Both problems, however, quietly assume something that cannot be taken for granted: that the same code can run across diverse hardware. A model trained on an NVIDIA A100 must serve inference on a mobile phone's ARM CPU, a Google TPU, or a microcontroller with kilobytes of memory. The same `torch.matmul` call must dispatch to cuBLAS on one device and a hand-tuned ARM NEON kernel on another. This hardware diversity creates the third problem.

7.5 Abstraction Problem

Modern machine learning applications demand portability across radically disparate physical execution targets: a model trained on a cluster of NVIDIA GPUs with terabytes of high-bandwidth memory must deploy on an Apple neural engine, an edge TPU, or an ARM microcontroller with 256 kilobytes of SRAM. The **abstraction problem** requires frameworks to hide this underlying hardware diversity behind a uniform programming interface without sacrificing the specialized instruction sets, memory layouts, and compute pipelines of each target. GPUs expose abundant throughput-oriented parallelism but have different memory and execution semantics from CPUs. TPUs favor regular tensor programs and compiler-visible shapes. A microcontroller has kilobytes where a server has gigabytes. Frameworks must therefore balance unified developer ergonomics against target-specific performance tuning.

The problem decomposes into two interacting dimensions. The first is *data representation*: how frameworks encode tensors, parameters, and computational state in forms that work across hardware. The second is *execution mapping*: how high-level operations translate into hardware-specific implementations. These dimensions are not independent concerns. The way data is represented (memory layout, precision, device placement) directly affects *what* execution strategies are possible. A tensor stored in row-major format on a GPU requires different kernels than one in column-major format on a CPU. A model quantized to INT8 enables entirely different execution paths than FP32.

Solving the abstraction problem requires sophisticated software infrastructure: tensor representations that encode both mathematical semantics and hardware constraints, intermediate representations that enable hardware-specific compilation, and runtime systems that manage data movement across the memory hierarchy. To make this concrete, trace what must happen when a programmer writes `model(input)`. The framework must resolve five decisions in rapid succession, and the rest

of this section takes them in order: data representation (tensor shape, memory layout, numeric precision), device placement (the bandwidth hierarchy connecting CPU, GPU, and accelerator memory), input delivery (data pipelines that sustain hundreds of MB/s to keep the accelerator fed), parameter organization (weights, gradients, and optimizer state that persist across steps and may span devices), and kernel execution (dispatch, scheduling, and resource optimization). Distributed placement previews the same abstraction boundary: frameworks expose placement scopes so system components can manage device boundaries.

7.5.1 Data structures and tensor abstractions

A ResNet-50 forward pass touches 25.6M parameters, produces intermediate activations at every layer, and must coordinate memory across CPU and GPU address spaces. Frameworks organize all of this data so that a single `model(input)` call executes millions of operations without the programmer managing a single pointer.

Computational graphs specify the *logical* flow of operations, but data structures determine how those operations access and manipulate data in *physical* memory. This distinction matters because the same mathematical operation can differ by an order of magnitude in throughput depending on whether data is contiguous in cache, pinned for direct memory access (DMA) transfer, or scattered across pages.

The first step is the data container itself. Framework data structures must sustain memory bandwidth (hundreds of GB/s on modern GPUs), accommodate architectures from 1D sequences to 5D video tensors, and hide device management behind clean APIs. Tensors are the universal answer.

7.5.1.1 Tensors

At the foundation of every framework's data representation lies a single abstraction: the tensor, an n-dimensional array that pairs numerical values with the information needed to interpret and place them.

Every computation in a neural network operates on tensors.[22] Training batches, activation maps, parameter gradients, and optimizer states are all tensors. This unified representation lets frameworks optimize a single data structure for hardware rather than managing separate containers for each role.

[22] **Tensor**: In mathematics, tensors obey coordinate-transformation laws; ML frameworks use the term more broadly for multidimensional arrays carrying shape, dtype, strides, device placement, and often automatic-differentiation metadata. A transpose commonly creates a strided view, while reshape may return either a view or a copy depending on the layout. These storage semantics make layout relevant to kernel selection and data movement.



Definition 7.2: Tensor

Tensors are n-dimensional arrays with shape, data type, stride, and device metadata. Framework runtimes use this information to select compatible operations and hardware kernels.

1. **Significance**: A contiguous FP32 tensor of shape $[1024, 1024]$ occupies $1024 \times 1024 \times 4 = 4{,}194{,}304$ bytes (about 4.2 MB). A noncontiguous view may be accepted directly by a strided kernel or may require materialization when an operation demands a supported contiguous layout.
2. **Distinction**: NumPy arrays also carry shape, dtype, and stride metadata. Framework tensors additionally integrate device placement, automatic differentiation, and accelerator dispatch.
3. **Common pitfall**: Tensor operations may return a new allocation, a view sharing existing storage, or an in-place update, depending on the operation and API variant. Memory accounting must distinguish these cases.

Because weights, gradients, master copies, optimizer moments, and saved activations are all tensors, they draw on the same device allocator and differ only in shape, dtype, device, and lifetime. That uniformity lets one caching allocator and one placement mechanism serve every role. It is also why a parameter count underestimates memory, since each weight tensor brings several companion tensors with it, as the training-state ledger in notebook 7.3 showed.

7.5.1.2 Tensor structure and dimensions

Tensors structure numerical data by adding organizational axes to linear memory layouts. Figure 7.8 traces this rank expansion from left to right, progressing from a Rank 0 scalar cell and Rank 1 column vector $(1, 3, 5, \ldots, 3)$ to a Rank 2 matrix grid and Rank 3 isometric volume tensor.

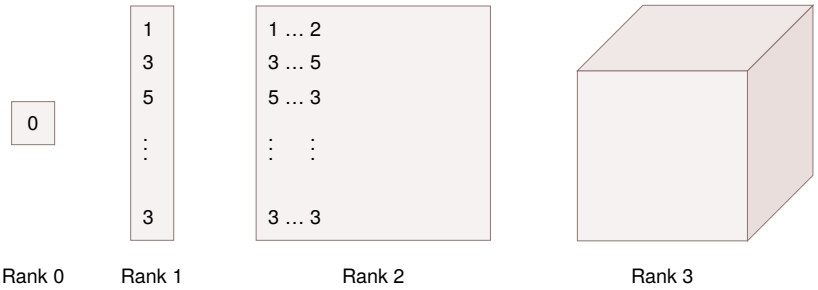


Figure 7.8: Tensor Rank Hierarchy: Ascending tensor ranks (0 to 3) generalize multi-dimensional indexing from scalars to volume tensors. Each additional rank adds an organizational axis that the framework must linearize into physical memory addresses via strides.

In vision applications, raw pixel arrays map directly onto multi-dimensional tensor layouts. Figure 7.9 diagrams this spatial decomposition for a 3×3 pixel patch: planar grids for height (3 pixels vertically) and width (3 pixels horizontally) stack along depth into three color channels (red, green, and blue), forming a rank-3 coordinate space.

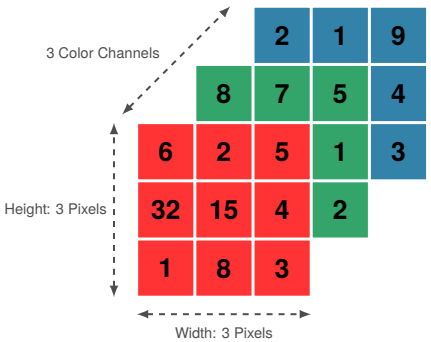


Figure 7.9: Image as an RGB Tensor: Decomposing a 2D color image into stacked height, width, and channel dimensions forms a rank-3 tensor. In ML systems, the ordering of these dimensions (such as channel-first vs. channel-last) determines memory access locality and cache efficiency during convolution operations.

Framework tensors carry more than raw numbers. Each tensor stores **tensor metadata**, runtime information used to validate operations and select fast execution paths: a *shape* tuple (for example, `[64, 3, 224, 224]` for a batch of images), a *dtype* (framework literals such as `float32`, `float16`, or `int8`), and a *device* tag (CPU, cuda:0). A matrix multiplication, for instance, checks shape compatibility at dispatch time and uses the dtype to route to the correct hardware kernel, whether a standard FP32 GEMM or a Tensor Core FP16 path.

Memory layout implementation introduces distinct challenges in tensor design. While tensors provide an abstraction of multi-dimensional data, physical computer memory remains linear. Stride patterns address this disparity by creating mappings between multi-dimensional tensor indices and linear memory addresses. These patterns significantly impact computational performance by determining memory access patterns during tensor operations. Figure 7.10 contrasts how a 2×3 logical matrix maps into contiguous physical memory: row-major ordering (top right, pink) packs elements sequentially along rows ($[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]$) using strides $[3, 1]$, whereas column-major ordering (bottom right, green) serializes along columns ($[1, 4, 2, 5, 3, 6]$) using strides $[1, 2]$. In both schemes, the framework computes element offsets via $i \times \text{stride}[0] + j \times \text{stride}[1]$.

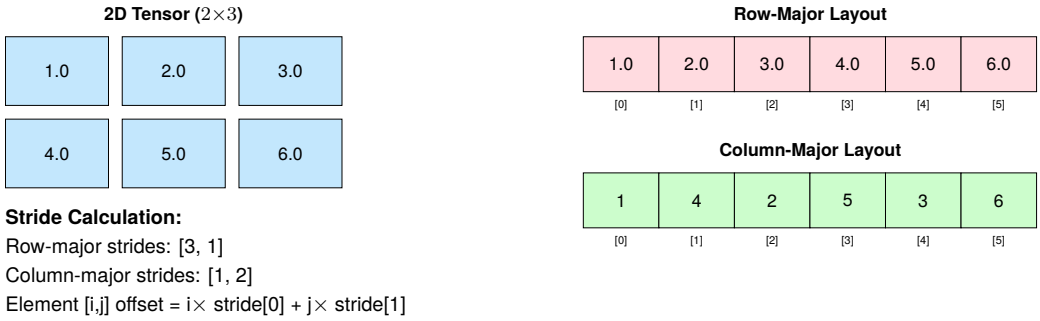


Figure 7.10: Tensor Memory Layout: A 2×3 tensor can be stored in linear memory using either row-major (C-style) or column-major (Fortran-style) ordering. Strides count elements skipped in each dimension, so i × stride[0] + j × stride[1] gives an element offset. Multiplying by the element size gives the byte offset from the storage origin. The choice of layout affects which traversals access neighboring elements efficiently.

Stride choices become performance choices. Row-major layout (used by NumPy, PyTorch) stores elements row by row, making row-wise operations more cache-friendly. Column-major layout (used by some BLAS libraries) stores elements column by column, optimizing column-wise access patterns. The stride values encode this layout information: in row-major layout for a 2×3 tensor, moving to the next row requires skipping three elements (stride[0] = 3), while moving to the next column requires skipping one element (stride[1] = 1).

These memory layout details have direct performance implications, but no ordering is universally superior. Efficient access depends on which dimension an operation traverses, the kernel's supported layouts, vectorization, and the target memory hierarchy. A mismatched layout may require strided access or a conversion, while a kernel designed for that layout can access it efficiently. PyTorch's `.transpose()` creates a view by changing stride metadata while sharing storage. A later reshape can reuse that storage only when the strides permit the requested view. Calling `.contiguous()` copies elements into a new buffer when the requested contiguous layout is not already present.

The dtype is the tensor-level lever that trades numerical range against data movement. The standard choice in machine learning has been FP32 precision, exposed in frameworks through dtype literals such as `float32`, offering a balance of precision and efficiency. Modern frameworks extend this with multiple numeric types for different needs. Integer types support indexing and embedding operations. Reduced-precision types such as FP16 enable efficient mobile deployment. INT8 precision allows fast inference on specialized hardware. The choice of numeric type affects both model behavior and computational efficiency: neural network training typically requires FP32 precision for critical accumulations to maintain stable gradient computations, while inference tasks can often use lower precision (framework dtype literals such as `int8` or even `int4`, corresponding to INT8 and INT4), reducing memory usage and increasing processing speed. The autocast policy of section 7.4.2.2 combines these benefits per operation, running most computations at 16-bit precision while keeping accumulations and numerically sensitive operations in FP32.

Type conversions become another point where abstraction meets physics. Operating on tensors with different types demands explicit conversion rules to preserve numerical correctness. These conversions introduce computational costs and risk precision loss. Frameworks provide type casting capabilities but rely on developers to maintain numerical precision across operations.

Tensors answer the data-representation problem by encoding shape, layout, and precision into a single abstraction. A perfectly shaped tensor on the wrong device, however, or one that must cross the PCIe-to-HBM bandwidth gap to reach the GPU, can erase every layout optimization. The next problem is placement: where data lives and how it moves.

7.5.1.3 Device and memory management

Tensors and their memory layouts establish what the framework computes with. Where that data physically resides, and how it moves between locations, determines whether computation happens at full speed or crawls.

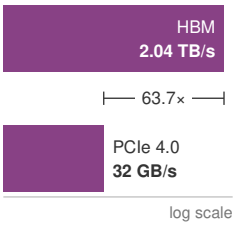
Frameworks as the operating system interface While the high-level API focuses on math, the framework backend plays the role that an operating system plays for a single machine. It schedules compute and moves data, building on the CUDA runtime's primitives for launching kernels and managing device memory.

On the data-movement side, the framework coordinates with this runtime to implement **DMA** over the PCIe bus. The bandwidth gap between the host (CPU) and device (GPU) is the primary "Data Loading Bottleneck": section D.3.4 models this transfer, separating the two constraints, $T = L_{\text{lat}} + D_{\text{vol}}/\text{BW}$, that determine whether a given data-loading strategy is limited by per-transfer latency or by sustained bandwidth. Frameworks mitigate this through pinned memory (page-locked host memory), which enables DMA transfers without pageable-memory staging. This "HW/OS" interface is what makes high-throughput training loops possible on a single machine.

↩ The single-node stack, in which the operating system moves data between host memory and accelerator memory, is established in section 2.4.1.

Every tensor resides on a specific device, and cross-device operations incur transfer costs that can dominate execution time. PCIe 4.0 delivers 32 GB/s between CPU and GPU, while HBM2e provides 2.04 TB/s within the GPU. This 63.7× bandwidth gap means a single misplaced tensor transfer can erase the entire speedup from GPU acceleration.

Device placement matters for framework design because the framework must track where every tensor lives and enforce that operations only combine tensors on the same device. When data must move, the framework must decide whether to block execution or overlap the transfer with other work. These decisions, invisible to most users, can move a training loop from transfer-bound execution toward substantially higher hardware utilization.



On-device HBM outruns the host PCIe link by 63.7×.

Three systems principles govern effective device and memory management: understanding the bandwidth hierarchy that constrains data movement, overlapping computation with communication to hide transfer latency, and using fine-grained synchronization to maintain correctness without sacrificing concurrency, supported by quantitative analysis grounded in the iron law's data movement term. The bandwidth hierarchy provides the first constraint.

The cost of moving data between devices varies by orders of magnitude depending on the interconnect.[23] Table 7.6 shows transfer times for a 1000×1000 float32 tensor (4 MB)—roughly the size of a typical activation tensor in a moderately sized model. The numbers reveal why careless device placement can erase any speedup from GPU acceleration.

Table 7.6: Device Transfer Overhead: Transfer time for a 4 MB tensor across different interconnects. PCIe bandwidth shown is unidirectional (typical for GPU transfers), with full-duplex operation providing 2× total bandwidth. NVLink bandwidth is bidirectional (300 GB/s per direction). These transfer costs can dominate lightweight operations, making device placement critical.

Interconnect	Bandwidth	Transfer Time	Path
PCIe 3.0 x16	15.8 GB/s	0.254 ms	Host to device
PCIe 4.0 x16	32 GB/s	0.125 ms	Host to device
NVLink 3.0	300 GB/s per direction (600 GB/s bidirectional)	0.013 ms	GPU to GPU
GPU Memory	2039 GB/s	0.002 ms	On device

[23] **NVLink**: NVIDIA's high-bandwidth GPU-to-GPU interconnect, providing 600 GB/s bidirectional bandwidth (NVLink 3.0 on A100) compared to 64 GB/s for PCIe 4.0 x16. This ~10× bandwidth advantage determines whether tensor parallelism, splitting one large tensor computation across multiple GPUs, is practical for a given model size: GPUs connected only by PCIe can make the D_{vol}/BW communication term dominate total training time, erasing the benefit of additional compute.

These numbers connect directly to the iron law of ML systems. Every cross-device transfer contributes to $(D_{\text{vol}}/\text{BW})$ at a fraction of on-device bandwidth. Dividing 1 GB by the stated PCIe 4.0 peak gives 31.2 ms, a bandwidth-only lower bound that omits protocol, launch, synchronization, topology, and contention overhead. Transfers can dominate lightweight or small-batch workloads, so deployed latency must be measured rather than inferred from peak bandwidth alone.

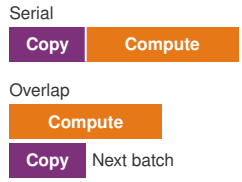
Device placement discipline protects the bandwidth hierarchy from accidental PCIe traffic. Every tensor carries a device attribute, and frameworks enforce a strict invariant: operations can only combine tensors on the same device. A `RuntimeError` results from mixing `cuda:0` and `cuda:1` tensors, preventing silent cross-device transfers.

The movement mechanism is explicit. Calling `.to()` on a tensor returns the original tensor when its data type and device already match the request; otherwise it returns a converted copy unless `copy=True` forces a copy. In contrast, `nn.Module.to()` modifies registered parameters and buffers in place and returns the module.

The performance discipline follows directly from this bandwidth gap: allocate tensors on the target device from the start rather than creating them on the host and transferring them, reuse GPU memory across iterations rather than reallocating it, and colocate inputs, labels, and model parameters on the same device to eliminate implicit transfers. At 32 GB/s, violating any of these principles inserts PCIe transfers into the critical path that can dominate a training iteration that otherwise runs at 2.04 TB/s on-device.

Overlapping computation and communication When transfers are unavoidable, the next optimization is to hide their latency by executing them concurrently with computation. Modern GPUs contain independent hardware units for computation (streaming multiprocessors) and data transfer (copy engines), enabling true simultaneous execution. The framework abstraction that exposes this hardware parallelism is the CUDA stream: an independent execution queue where operations execute sequentially within a stream but concurrently across streams.

When copies and computation share one stream, they execute serially. Placing the next batch's transfer on one stream while the current batch computes on another can make the steady-state batch interval approach $\max(T_{\text{compute}}, T_{\text{transfer}})$, the larger stage time rather than their sum. Each batch still requires its own transfer to finish before its consumer runs. With sufficient independent work and hardware resources, stream-based overlap can hide much of the D_{vol}/BW penalty when computation is the longer stage (see listing 7.15).



Overlap pays the longer stage, not the sum.

The `non_blocking=True` flag requests an asynchronous copy with respect to the host when the backend supports it, but true asynchronous overlap requires pinned memory. Standard host memory allocated by the OS is pageable: the virtual memory manager reserves the right to relocate pages in physical RAM or swap them to disk at any time. However, accelerator DMA controllers stream bytes across the PCIe bus directly between host RAM and device memory without involving CPU cores. Because hardware DMA engines cannot handle OS page faults, they require fixed, unmovable physical addresses. If a tensor resides in standard pageable RAM, calling `.to('cuda')` forces the runtime to perform an internal synchronous CPU-to-CPU copy into a temporary pinned buffer before firing the transfer. Explicitly placing tensors in pinned (page-locked) host memory allows the hardware DMA engine to stream data across PCIe asynchronously, enabling data transfers to overlap seamlessly with accelerator computation.

The following sketch separates the two streams; a complete pipeline must also hand each transferred batch to its consumer only after the copy completes, using the synchronization mechanisms developed next.

Listing 7.15: Overlapping Computation and Transfer: Separate streams can overlap eligible copies and computation when dependencies are synchronized correctly. Pinned host memory generally enables host-to-device copies to overlap with other device work.

```
compute_stream = torch.cuda.Stream()
transfer_stream = torch.cuda.Stream()

# Transfer next batch while computing
# current batch
with torch.cuda.stream(transfer_stream):
    next_batch = next_batch_cpu.to("cuda", non_blocking=True)

with torch.cuda.stream(compute_stream):
    output = model(current_batch)
    loss = criterion(output, labels)

# Pinned host memory can enable transfer overlap
x_pinned = torch.randn(1000, 1000).pin_memory()
x_gpu = x_pinned.to("cuda", non_blocking=True)  # Asynchronous

# A pageable source may require staging and
# should not be assumed to overlap
y_regular = torch.randn(1000, 1000)
y_gpu = y_regular.to("cuda", non_blocking=True)
```

Synchronization and correctness Concurrent execution introduces ordering constraints. When one stream's output becomes another stream's input, the system must enforce a happens-before relationship without unnecessarily serializing independent work. Two synchronization mechanisms exist, with dramatically different performance implications.

Full device synchronization (`torch.cuda.synchronize()`) blocks all streams and the CPU until every queued operation completes. This creates a global serialization point that eliminates all overlap benefits. CUDA events provide the alternative: fine-grained synchronization that blocks only the dependent stream, allowing other streams and the CPU to continue execution (see listing 7.16).

Listing 7.16: CUDA Events for Synchronization: Events enable fine-grained producer-consumer patterns between streams without blocking the entire device.

```
# Create streams and event
stream1 = torch.cuda.Stream()
stream2 = torch.cuda.Stream()
event = torch.cuda.Event()

# Stream 1: producer
with torch.cuda.stream(stream1):
    result1 = expensive_computation(data1)
    event.record()  # Mark completion point

# Stream 2: consumer (waits only for stream1's event)
with torch.cuda.stream(stream2):
    event.wait()  # Block stream2 until event is recorded
    result2 = dependent_computation(result1)  # Safe to use result1
```

The performance difference between these approaches is categorical. Full synchronization after every operation converts a concurrent pipeline into a sequential one, entirely negating the hardware parallelism that streams expose. Event-based synchronization preserves the concurrent execution model while enforcing only the dependencies that correctness requires.

The producer-consumer example supplies the required synchronization principle. The following optional preview extends it to model-stage overlap within a single node. Different model stages on separate GPUs can process different microbatches concurrently, with each stage's computation overlapping the next stage's data reception (listing 7.17). The training chapter develops the partitioning decision; the listing illustrates how streams and events express its dependencies.

Listing 7.17: Stage Overlap with Streams: Overlap multiple model stages across microbatches using streams and events for inter-stage synchronization.

```
# Pipeline parallelism: place stages on separate GPUs
devices = [torch.device(f"cuda:{i}") for i in range(3)]
stages = [
    Stage1().to(devices[0]),
    Stage2().to(devices[1]),
    Stage3().to(devices[2]),
]
streams = [torch.cuda.Stream(device=device) for device in devices]
events = [
    [torch.cuda.Event() for _ in range(num_microbatches)]
    for _ in stages
]
outputs = [[None] * num_microbatches for _ in stages]

for mb in range(num_microbatches):
    for stage_idx, (stage, stream, device) in enumerate(
        zip(stages, streams, devices)
    ):
        with torch.cuda.device(device), torch.cuda.stream(stream):
            if stage_idx > 0:
                # Wait for previous stage to complete
                # this microbatch
                events[stage_idx - 1][mb].wait()
                stage_input = outputs[stage_idx - 1][mb].to(
                    device, non_blocking=True
                )
            else:
                stage_input = inputs[mb].to(device, non_blocking=True)

            outputs[stage_idx][mb] = stage(stage_input)
            events[stage_idx][mb].record()
```

↪ Layer-wise model partitioning across GPUs, whose idle pipeline stages this stage-overlap pattern addresses, is developed in section 8.8.1.2.

This synchronization discipline has one operational trap: debug code often leaves `torch.cuda.synchronize()` in the hot path, turning an overlapped pipeline into a serialized one. When overlap remains poor, profiling must separate scheduling stalls from kernel inefficiency. NVIDIA Nsight Systems (`nsys profile`) shows CPU activity, GPU kernels, and memory transfers on one timeline. NVIDIA Nsight Compute (`ncu`) then explains kernel behavior with hardware counters.

Table 7.7 is the diagnostic map for that second step. SM means streaming multiprocessor, the GPU block that schedules groups of threads; a warp is one scheduled thread group.

Table 7.7: Nsight Compute Metrics: Key metrics for diagnosing ML kernels. Interpret each value against workload eligibility and kernel intent; Nsight Systems identifies which kernels dominate runtime, and Nsight Compute reveals why those kernels underperform.

Metric	Meaning	Optimization Target
SM Occupancy	Active warps/maximum warps	Increase parallelism if low
Memory Throughput	Achieved/peak bandwidth	Optimize memory access patterns
Compute Throughput	Achieved/peak FLOP/s	Reduce memory bottlenecks
Tensor Core Active	Time in Tensor Core ops	Verify mixed-precision utilization

7.5.1.4 Data pipelines and loading

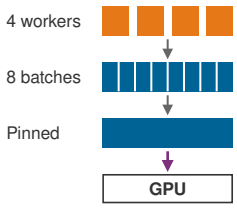
Streams and events address placement and movement by overlapping transfers with computation so that the GPU rarely stalls on a single tensor. Scheduling alone, however, cannot help if data arrives too slowly in the first place. The next constraint is input delivery: data must arrive fast enough to sustain throughput. The core systems principle is straightforward: the data pipeline must sustain the accelerator's consumption rate. A GPU processing 1,000 images per second at 224 by

224 resolution requires approximately 150.5 MB/s of sustained raw uint8 image throughput. If the pipeline cannot maintain this rate, the accelerator idles and the effective utilization term in the iron law drops below 1.

↩ The feeding problem, in which data delivery must keep pace with accelerator consumption, is established in section 4.2.3.

Frameworks address this throughput requirement through three mechanisms. The first is parallel worker processes: the DataLoader spawns multiple CPU processes, each independently loading and preprocessing samples. When storage access, decoding, augmentation, or normalization makes one loader process too slow to feed the accelerator, multiple workers can overlap I/O wait with preprocessing. Once the input pipeline meets demand, extra workers add overhead rather than throughput. When `num_workers > 0`, the DataLoader distributes sample indices across workers through a shared queue, and workers push completed samples to a data queue that the main process assembles into batches.

The second mechanism is prefetching. With multiprocessing enabled, `prefetch_factor` controls how many batches each worker prepares in advance. Four workers with `prefetch_factor=2` can maintain up to eight prefetched batches, increasing the chance that input work overlaps accelerator computation. Prefetching cannot guarantee that the accelerator never stalls; storage, decoding, augmentation, and contention can still make production slower than consumption. The cost is additional host memory proportional to the queued data.



Four workers fill eight prefetched batches, which drain through pinned memory to the GPU.

The third mechanism is pinned memory for DMA transfers. The `pin_memory=True` option places batches in page-locked host memory, which can avoid pageable-memory staging and enable host-to-device copies to overlap device work. For a batch of 64 FP32 images at $224 \times 224 \times 3$ (38.5 MB), dividing by the stated PCIe 4.0 x16 peak gives 1.2 ms; this is a bandwidth-only lower bound, not a measured pinned-transfer latency. The realized benefit over pageable memory depends on the platform, batch size, pipeline, and overlap. Pinned pages also reduce pageable system memory.

The DataLoader configuration is useful only when each parameter is tied to a bottleneck: `num_workers` sets loading parallelism, `prefetch_factor` sets pipeline depth, and `pin_memory` enables DMA transfers. Each worker also holds its own copy of the Dataset object, so the worker count trades throughput against host memory.

↪ Sizing worker count and prefetch depth against measured throughput and memory is operationalized in section 8.7.2.2.

Once throughput is high enough, worker process management becomes a correctness constraint. PyTorch assigns each worker a distinct PyTorch seed. A `worker_init_fn` is still useful when dataset or augmentation code also uses NumPy or Python's `random` module, as listing 7.18 demonstrates. Shared Python state is generally process-local, so modifications in one worker do not automatically propagate to the others or the main process; explicitly shared or memory-mapped storage is needed when workers must share mutable state.

The Dataset choice is another throughput decision because it determines how samples can be scheduled. PyTorch supports two dataset paradigms. *Map-style* datasets implement `__len__` and `__getitem__`, enabling random access to samples by index—this pattern works well for datasets that fit in memory or support efficient random access on disk. *Iterable-style* datasets implement `__iter__` instead, yielding samples sequentially for streaming data sources where random access is impractical. Map-style datasets support sampler-based shuffling, while iterable datasets must implement any ordering or buffered shuffling within the iterator or data source.

Listing 7.18: DataLoader Throughput Configuration: Each parameter addresses a specific throughput bottleneck. `num_workers` parallelizes I/O and preprocessing across CPU cores, `prefetch_factor` controls pipeline depth, and `pin_memory` enables DMA transfers to the GPU.

```
import random

import numpy as np
import torch
from torch.utils.data import DataLoader


def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)
    random.seed(worker_seed)


loader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4,  # Parallel worker processes (mechanism 1)
    prefetch_factor=2,  # Batches prepared ahead per worker (mechanism 2)
    pin_memory=True,  # Page-locked memory for DMA (mechanism 3)
    worker_init_fn=seed_worker,  # Reproducible augmentation per worker
)

# Pipeline effect: while GPU processes batch N,
# 4 workers load batches N+1..N+8 into pinned memory,
# ready for DMA transfer when the GPU finishes.
```

Batch collation represents the final stage where representation choices govern throughput. The `collate_fn` parameter determines how individual samples are combined into batches. The default collation stacks tensors along a new batch dimension, which works when all samples have identical shapes. For variable-length data such as text sequences, custom collation handles padding, sorting by length, or creating attention masks—directly affecting both memory usage and training throughput.

DataLoaders, Datasets, and collation functions solve input delivery by sustaining accelerator-rate throughput through parallelism, prefetching, and DMA. These structures, however, handle only ephemeral data: samples flow through the pipeline once per epoch and are discarded. The next framework responsibility is persistent state, especially the model's own weights when those weights exceed the memory of any single device.

Parameter structures

Model parameters persist across batches rather than flowing through the input pipeline. The framework must register their buffers, associate them with gradients and optimizer state, and preserve their placement across updates. Adam state alone can add about $4\times$ the FP16 weight memory, as notebook 7.3 showed, so parameter management must account for companion state rather than weights alone. On a machine with several accelerators, the runtime may keep a complete model replica on each device or partition model state among devices. Replicas require gradient synchronization; partitions require transfers along the model's dependencies. Both arrangements depend on explicit parameter ownership and communication paths.

Parameter structures must also hold the same weights at more than one precision. In the mixed-precision Adam recipe of section 7.4.2.2, each parameter exists during training as FP16 working weights and FP32 master weights, and an inference export may store it again in FP16 or INT8. The framework's casting and mixed-precision machinery keeps these copies synchronized, so the optimizer updates the master copy and the kernels read the working copy.

Distributed execution contexts

The computational graph defines *what* to compute, but *where* and *how* that computation runs across devices is the job of execution contexts. On a single node, execution contexts manage CUDA streams and events (introduced in section 7.5.1.3) to overlap computation and data transfer across GPUs.

When work spans several accelerators, these same abstractions extend to named groups of devices. Frameworks use constructs such as `ProcessGroup` (PyTorch) or `Mesh` (JAX) to declare which tensors and operations belong together, and they carry out group-wide operations, such as summing every replica's gradients into one result (an AllReduce), through a communication library. The cost of those operations follows the hardware path. Inside one node, NVLink delivers 600 GB/s of bidirectional bandwidth per A100 (table 7.6), and placement must account for whether a local transfer uses that path or PCIe. The important framework idea is the abstraction boundary. User code names placement relationships, and the framework preserves those relationships while the hardware path changes underneath.

At the framework level, the central requirement is placement expressiveness on one machine with one to eight accelerators. For example, two local accelerators can each hold a complete model and process different batch slices, or each can hold a different group of layers and transfer intermediate activations between them. The first arrangement is data parallelism; the second is pipeline parallelism. A third arrangement, tensor parallelism, splits individual tensor operations across devices. These are alternative placement choices, not a requirement to combine every strategy. The framework must express device ownership, transfers, and synchronization while preserving the model's semantics. Section 8.8 develops the strategy decisions and their communication costs.

As an optional scale preview, figure 7.11 illustrates how these three placement relationships compose when extreme model dimensions exceed the memory capacity of any single accelerator. At GPT-3 scale, storing the 175B parameters alone requires 350 GB in FP16, compelling frameworks to partition state across three orthogonal axes: across layer groups (pipeline parallelism), across replicated batches (data parallelism), and within individual weight matrices (tensor parallelism). Interpreting the resulting multi-device grid is not required for the single-machine framework reasoning developed in this chapter; section 8.8 investigates the formal communication cost and scheduling geometry of multi-dimensional parallelism.

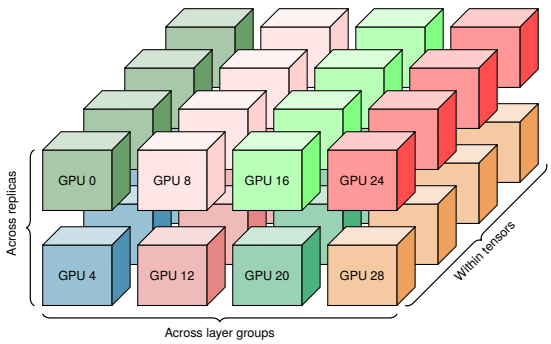


Figure 7.11: Device Placement Dimensions: A grid of eight accelerator clusters arranged in two rows and four columns, each containing stacked computational units. The braces preview three ways a framework can place work when a model no longer fits one device: across layer groups, across replicated batches, and within large tensors.

Tensors, device managers, data pipelines, parameter structures, and distributed execution contexts define *what* data a framework manages and *where* it resides. The remaining abstraction problem is execution: translating operations into work that runs on hardware.

7.5.2 Core operations

When an engineer writes `y = torch.matmul(x, w)`, bridging that single call to thousands of parallel accelerator threads traverses three coordinated functional tiers. Figure 7.12 diagrams this lowering pipeline from left to right: system-level operations (scheduling, memory management, and resource optimization) organize graph execution; basic numerical operations (GEMM, BLAS, and element-wise kernels) execute algebraic routines; and hardware operations (compute kernel management,

memory abstraction, and execution control) bind the workload directly to device streams and physical silicon. The discussion that follows traces this hierarchy from the hardware kernel primitives upward to system orchestration.

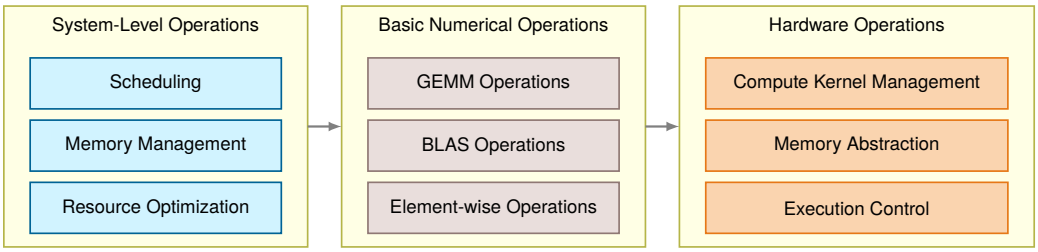


Figure 7.12: Core Operations Stack: Three functional tiers bridge high-level Python code to physical execution. Hardware operations manage low-level kernel dispatch and memory transfers, basic numerical operations execute dense GEMM and element-wise routines, and system-level operations coordinate graph scheduling and memory lifetimes across accelerators.

7.5.2.1 Hardware abstraction operations

The hardware abstraction layer isolates framework code from platform-specific details. It solves three concrete problems: selecting the right compute kernel, moving data through the memory hierarchy, and coordinating execution across processing units.

Compute kernel management The kernel manager dispatches each supported operation to an implementation for the current backend. For matrix multiplication, this may be a CPU BLAS backend such as Intel oneMKL or OpenBLAS (Intel Corporation 2026a; OpenBLAS Project 2026), cuBLAS on NVIDIA GPUs (NVIDIA 2024a), or accelerator-specific tensor instructions. Selection depends on dimensions, dtype, layout, backend, and library heuristics. A 4096×4096 FP16 GEMM on an A100 may use a Tensor Core path whose published peak is represented here by 312 TFLOP/s (Choquette et al. 2021; NVIDIA Corporation 2020a). If the backend has no implementation for an operator, execution may fall back, be partitioned, or fail, depending on the framework and runtime.

Memory system abstraction The memory abstraction layer moves tensors among pageable or pinned host memory, device memory, and unified memory, and transforms data layouts to match hardware preferences. In physical silicon, tensors do not exist as multi-dimensional matrices; they are stored as one-dimensional arrays of linear byte addresses. Layout formats dictate how dimensions map to that 1D sequence: *channels-first* NCHW (Batch $\times$ Channels $\times$ Height $\times$ Width) places all spatial pixels of one channel contiguously before storing the next channel, whereas *channels-last* NHWC (Batch $\times$ Height $\times$ Width $\times$ Channels) groups all channel values for a single spatial coordinate consecutively. Accelerator matrix units often prefer NHWC because loading all channels for a pixel aligns with a single, contiguous 128-byte memory transaction (a coalesced access). A framework may therefore execute kernels in NHWC for peak hardware utilization while preserving NCHW for user-facing APIs. Alignment requirements vary from 4 bytes on CPUs to 128 bytes on high-bandwidth accelerators; misaligned or strided accesses split requests across multiple memory transactions and can cut effective bandwidth substantially. The runtime also preserves dependency ordering when multiple execution units access the same tensor, preventing races during concurrent operations.

Execution control The execution controller coordinates work across multiple processing units and memory spaces. On a modern GPU, graph runtimes or explicit stream use can overlap independent kernels when dependency analysis proves they are ready and the kernels leave enough resources unused to run concurrently. Eager default-stream execution often serializes this work instead. The controller inserts synchronization barriers where data dependencies require them, tracks event completions to trigger dependent operations, and routes hardware errors (ECC failures, timeout watchdogs) to the framework's error handling path.

7.5.2.2 Basic numerical operations

With hardware abstraction managing the platform-specific details, frameworks build a layer of mathematical operations on top. GEMM dominates ML computation. The operation $\mathbf{C} = \alpha\mathbf{AW} + \beta\mathbf{C}$

accounts for the vast majority of arithmetic in neural networks: a single ResNet-50 forward pass performs approximately 8.2 GFLOP, nearly all of which reduce to GEMM. Section C.1.4 derives how GEMM arithmetic intensity scales with matrix dimension, predicting whether a given layer is compute bound or memory bound before any profiler runs. Frameworks optimize GEMM through cache-aware **tiling** (splitting matrices into blocks that fit in L1/L2 cache), loop unrolling for instruction-level parallelism, and shape-specific kernels. Fully connected layers use standard dense GEMM, while convolutional layers use im2col transformations that reshape input patches into matrix columns, converting convolution into GEMM.

Beyond GEMM, frameworks implement BLAS operations (AXPY for vector addition, GEMV for matrix-vector products) and element-wise operations (activation functions, normalization). Element-wise operations are *individually cheap* but *collectively expensive* due to memory bandwidth. Each operation reads and writes the full tensor, so a sequence of five element-wise operations on a 100 MB tensor moves 1 GB of data. Fusing those five operations into a single kernel reduces memory traffic to 200 MB, a 5× bandwidth savings that directly translates to faster execution.

Numerical precision adds another dimension. Training in FP32 uses 4 bytes per parameter; quantizing to INT8 reduces this to 1 byte, cutting memory by 4× and enabling 2–4× throughput improvements on hardware with INT8 acceleration. Training typically keeps numerically sensitive accumulations in higher precision, while inference can often run many operations in FP16 or INT8 with little quality loss. Frameworks maintain separate kernel implementations for each precision format and handle workflows where different layers operate at different bit widths within a single forward pass.

7.5.2.3 System-level operations

Hardware abstraction and numerical operations provide the building blocks; system-level operations orchestrate them. The system layer ties scheduling, memory management, and resource optimization into a coherent execution engine.

The operation scheduler uses graph dependencies to identify legal execution orders and possible concurrency. Static capture can expose more of the dependency structure before execution, while eager runtimes discover work incrementally. Neither visibility guarantees an optimal schedule or simultaneous execution: overlap depends on streams, compiler and runtime decisions, resource availability, and the operations themselves.

The memory manager allocates and reclaims GPU memory across the computational graph's lifetime. Model parameters (a 7-billion-parameter model consumes approximately 14 GB in FP16) persist for the entire training run, while activation tensors live only until the backward pass consumes them. Both draw on the caching allocator of section 7.4.2.3, which reuses freed blocks instead of paying for a `cudaMalloc` call (tens of microseconds, more when it synchronizes the device). When a model exceeds device memory, the same manager executes the activation-checkpointing policy of section 7.4.1.3. The policy question (which activations to keep, and how much recomputation to tolerate) belongs to the training pipeline; the framework abstraction is what makes that policy executable.

The resource optimizer integrates scheduling and memory decisions with backend kernel selection. Matrix-multiplication libraries choose among tiled GEMM implementations according to shapes, dtype, layout, workspace, and hardware. A poor schedule can leave resources idle, while memory-pool fragmentation or excessive live state can cause an out-of-memory error even when aggregate device capacity appears sufficient.



Checkpoint 7.3: Hardware abstraction

The abstraction problem is the bridge between *portable code* and *efficient execution*.

- ☐ **Two dimensions**: can you distinguish data representation (layout, dtype, placement) from execution mapping (kernel selection, scheduling), and explain how they constrain each other?

- ☐ **Kernel dispatch**: can you explain why the same high-level operation (for example, GEMM) needs multiple implementations (CPU vector path vs. GPU Tensor Core path) and how shapes/dtypes affect the choice?
- ☐ **Memory abstraction**: can you explain why frameworks use caching allocators and layout transforms (NCHW↔NHWC) rather than calling device allocators on every tensor?
- ☐ **Execution control**: can you describe what the runtime is doing when it overlaps independent work (streams) and inserts synchronization only where dependencies require it?

Tensors, streams, and kernel dispatch operate at the granularity of individual buffers and operations. While these primitives secure high execution efficiency on hardware, scaling model development requires an organizational abstraction: composing thousands of low-level operations into reusable, nestable building blocks called *modules*.

7.6 Model Composition and State

Hardware-facing mechanisms—tensors, kernels, streams, and memory managers—ensure individual operations execute efficiently on silicon. Yet a network such as ResNet-50, containing 25.6M parameters across fifty layers, requires extensive coordination: registering weights with optimizers, tracking device placement, toggling training and evaluation modes, and serializing checkpoints. Managing this manually would drown model development in error-prone tensor plumbing. The organizational layer of the framework resolves this through hierarchical composition.

Every major framework implements this through a module abstraction that binds parameter tensors, forward computation, and lifecycle hooks into a unified interface. PyTorch's `nn.Module`[24] provides an instructive case study because its design patterns recur across frameworks: Keras uses similar layer abstractions (Chollet 2018), JAX's Flax employs analogous module structures, and TensorFlow's functional API shares conceptual parallels. Three enduring design principles recur regardless of syntax or programming paradigm.

[24] **nn.Module**: The pattern recurs across frameworks because the organizational problem is universal. `nn.Module` automatically registers any assigned submodule or parameter into a hierarchical tree, enabling a single `.to('cuda')` call to recursively place millions of parameters onto a GPU. Keras layers, JAX Flax modules, and TensorFlow's `tf.Module` all implement the same tree-walking pattern. Without it, managing model state would require manual bookkeeping that scales linearly with architectural depth, a cost that grows prohibitive for models with hundreds of layers.

7.6.1 Automatic parameter discovery

A modern neural network may contain millions of trainable parameters spread across dozens of layers. Without automation, a programmer would need to enumerate every parameter tensor and pass it to the optimizer manually, an error-prone process that scales poorly with model complexity. Frameworks solve this through **automatic parameter discovery**: the system walks the module tree, collecting every parameter tensor so the optimizer can update them in a single call.

This is a tree traversal problem at its core. PyTorch's `Module.__setattr__` registers an assigned `nn.Parameter` or child `nn.Module` in the module's internal mappings. A call to `.parameters()` recursively traverses registered submodules and yields registered parameters. Other frameworks expose analogous parameter trees or collections, though their mechanisms differ.

The systems consequence is significant. Automatic parameter discovery gives the optimizer a complete parameter set or parameter groups, enabling grouped, foreach, or fused update paths when the framework and backend support them. The immediate win is correctness and composability: newly attached submodules participate in device moves, serialization, and optimizer construction without a separate manual registry. It also lets optimized update paths operate over parameter collections without per-parameter user code. Listing 7.19 demonstrates the core mechanism: attribute assignment triggers registration, and `.parameters()` returns all discovered tensors.

The distinction between *parameters* and *buffers* illustrates a subtlety of discovery. Registered parameters appear in `.parameters()` whether trainable or frozen; `requires_grad` separately controls whether autograd accumulates a gradient for them. Registered buffers move with the module and can appear in its state dictionary but are excluded from `.parameters()` and ordinary optimizer discovery. Batch-normalization running statistics are a common buffer use case.

Listing 7.19: Parameter Registration: Automatic parameter tracking through attribute assignment enables optimizer access to all trainable weights without manual enumeration.

```
import torch
import torch.nn as nn


class CustomLayer(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(output_size, input_size)
        )
        self.bias = nn.Parameter(torch.randn(output_size))
        self.register_buffer("running_mean", torch.zeros(output_size))

    def forward(self, x):
        return torch.matmul(x, self.weight.t()) + self.bias


layer = CustomLayer(10, 20)
# Framework discovers both parameters automatically:
for name, param in layer.named_parameters():
    print(f"{name}: shape {param.shape}")
```

Table 7.8 shows how the same principle manifests differently across frameworks. Despite syntactic differences, all frameworks solve the same problem: enabling optimizers to discover and update trainable parameters while preserving nontrainable state across forward passes.

Table 7.8: Parameter Discovery APIs Across Frameworks: Each framework exposes a different surface for distinguishing trainable parameters from nontrainable buffers, but all four solve the same underlying problem of letting optimizers find weights to update while preserving state (running statistics, frozen tensors) across forward passes. The shared pattern is structured access to parameters and persistent state. Accessor semantics differ: PyTorch also yields frozen registered parameters, so trainability remains a separate property.

Framework	Parameter Access	Nontrainable State
PyTorch	`model.parameters()`	`register_buffer()`
Keras	`layer.trainable_weights`	`layer.non_trainable_weights`
JAX/Flax	`variables["params"]` after `variables = model.init(key, x)`	Separate variable collections (for example, `batch_stats`)
TensorFlow	`module.trainable_variables`	`module.non_trainable_variables`

Mode-dependent behavior

Training and inference require different computational behavior from the same model graph. During training, dropout layers randomly zero elements with probability $p_{\text{drop}} = \Pr(\text{drop})$ to regularize the network, while during inference those same layers must perform identity mapping to produce deterministic outputs. Batch normalization uses per-batch statistics during training but switches to accumulated running statistics during inference. If these behavioral changes are left to the programmer, forgetting a single mode switch produces silently incorrect predictions in production.

Frameworks solve this with a *state flag* that propagates through the module hierarchy. A single call to `.eval()` on the root module recursively sets `self.training = False` on every descendant, and each layer queries this flag to select its behavior. This is an instance of a broader systems principle: the same computation graph must produce different execution behavior depending on context. Compilers face the same challenge when the same source code must produce debug builds (with bounds checking and symbol tables) vs. release builds (with aggressive optimization). The flag-propagation pattern ensures correctness by centralizing the mode decision at the root rather than requiring per-layer coordination.

Evaluation mode and gradient recording are separate controls. Calling `.eval()` changes mode-dependent layer behavior but leaves autograd recording enabled. For inference that needs no gradients, `torch.no_grad()` disables that recording, avoiding the associated saved backward state while `.eval()` selects the appropriate layer behavior.

This principle extends to parameter freezing for transfer learning. Setting `requires_grad=False` prevents autograd from accumulating gradients in those parameters and removes their gradient storage. It does not necessarily remove the layer's backward computation because gradients may still need to pass through its operations to reach trainable inputs or earlier parameters. Savings therefore depend on where the frozen region lies and which tensors require gradients.

7.6.2 Hierarchical composition and serialization

Complex models compose from reusable submodules, creating a tree structure. A ResNet is not implemented as a monolithic block of operations but as a hierarchy: the root module contains a sequence of residual blocks, each block contains convolution layers and normalization layers, and each layer contains parameter tensors. This hierarchical composition must support two critical operations made concrete in listing 7.20: recursive parameter collection for training and **state serialization** for checkpointing and deployment.

Listing 7.20: Nested Module Composition: Hierarchical module composition enables recursive parameter collection and a flat state mapping across the module tree.

```
import torch
import torch.nn as nn


class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        residual = x
        x = torch.relu(self.bn1(self.conv1(x)))
        x = self.bn2(self.conv2(x))
        return torch.relu(x + residual)


class ResNet(nn.Module):
    def __init__(self, num_blocks, channels=64):
        super().__init__()
        self.conv_in = nn.Conv2d(3, channels, 7, padding=3)
        self.blocks = nn.ModuleList(
            [ResidualBlock(channels) for _ in range(num_blocks)]
        )
        self.fc = nn.Linear(channels, 10)

    def forward(self, x):
        x = self.conv_in(x)
        for block in self.blocks:
            x = block(x)
        x = x.mean(dim=[2, 3])  # Global average pooling
        return self.fc(x)


model = ResNet(num_blocks=4)
total = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total}")
# state_dict() flattens the tree: 'blocks.0.conv1.weight', etc.
print(list(model.state_dict().keys())[:4])
```

Hierarchical composition mirrors the hardware memory hierarchy in a systems-relevant way: each submodule's parameters can be loaded independently, enabling placement across devices. When a model is too large for a single GPU, the framework can assign different subtrees of the module hierarchy to different devices, with the tree structure providing natural partition boundaries.

The `state_dict()` method produces an ordered mapping whose dotted keys (for example, `blocks.0.conv1.weight`) encode locations in the module hierarchy. A checkpoint for 7 billion FP16 parameters contains approximately 14 GB of weight payload before metadata or additional state, but the file format and storage path determine how those bytes are serialized. `load_state_dict()` copies matching parameter and buffer values into an already-constructed module and reports missing or unexpected keys according to its strictness setting; it does not reconstruct the module hierarchy. Cross-framework exchange requires a compatible graph and operator representation in addition to weights.

The hierarchical structure also enables module-level traversal for systematic operations. Methods such as `.named_modules()` iterate the entire tree, supporting bulk transformations such as replacing all BatchNorm layers with GroupNorm or applying Xavier initialization to every Linear layer. These traversal operations depend on the same tree structure that enables parameter discovery, illustrating how a single design decision propagates benefits across multiple use cases.

These three principles, automatic parameter discovery, mode-dependent behavior, and hierarchical composition with serialization, are not PyTorch-specific. Every framework must solve them. Keras layers, JAX's Flax modules, and even functional approaches all address the same problems of parameter management, state tracking, and compositional design. The differences lie not in *what* problems they solve but in *how* they prioritize among competing solutions. Two practical patterns show how the principles become system controls: selective parameter freezing reduces unnecessary gradient work for transfer learning (listing 7.21), and module hooks provide noninvasive inspection (listing 7.22).

Listing 7.21: Parameter Freezing: Demonstrates selective parameter freezing for transfer learning, where pretrained layers remain fixed while new layers train.

```
import torch
import torch.nn as nn
from torchvision.models import ResNet18_Weights, resnet18

# Freeze all parameters in a pretrained
# model
pretrained_model = resnet18(weights=ResNet18_Weights.DEFAULT)

for param in pretrained_model.parameters():
    param.requires_grad = False

# Replace final layer with trainable parameters
pretrained_model.fc = nn.Linear(512, 10)  # New layer is trainable

# Only fc.parameters() will receive
# gradients during training
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, pretrained_model.parameters()),
    lr=0.001,
)
```

Module hooks are the inspection counterpart to parameter freezing: they intercept intermediate computations without modifying model code, enabling gradient flow diagnosis and activation monitoring. Listing 7.22 illustrates both hook types.

Listing 7.22: Module Hooks: Shows forward and backward hooks for inspecting activations and gradients during training.

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 5))


# Forward hook to inspect activations
def forward_hook(module, input, output):
    print(
        f"Layer: {module.__class__.__name__}, "
        f"Output shape: {output.shape}, "
        f"mean={output.mean():.3f}, "
        f"std={output.std():.3f}"
    )


# Backward hook to inspect gradients
def backward_hook(module, grad_input, grad_output):
    print(f"Gradient norm: {grad_output[0].norm():.3f}")


# Register hooks on specific layer
handle_fwd = model[0].register_forward_hook(forward_hook)
handle_bwd = model[0].register_full_backward_hook(backward_hook)

# Execute forward and backward pass
x = torch.randn(32, 10)
y = model(x)
loss = y.sum()
loss.backward()

# Remove hooks when done
handle_fwd.remove()
handle_bwd.remove()
```

Together, these patterns—parameter discovery, freezing, and hooks—demonstrate how the three principles translate into practical APIs. These `nn.Module` patterns illustrate PyTorch's approach to the abstraction problem. PyTorch, however, is only one of several major frameworks, and its choices (mutable state, class inheritance, eager execution by default) are not the only valid design points. TensorFlow centralizes state differently, and JAX avoids mutable state entirely. These are not superficial API differences; they reflect deeply different answers to the three foundational framework challenges.

7.7 Framework Platform Analysis

A team that prototypes quickly but cannot deploy the resulting model, or deploys reliably but cannot debug training failures, has run into a framework design trade-off rather than a missing API call. Each major framework solves all three core problems but emphasizes a different path through them. TensorFlow couples eager development to graph capture and a broad deployment ecosystem; PyTorch begins with eager execution and adds capture, compilation, and export paths; JAX organizes differentiation, vectorization, and compilation as composable function transformations. These are architectural emphases, not exclusive capabilities, and their value depends on the workload and deployment target.

7.7.1 TensorFlow: Eager development with graph deployment

TensorFlow's architecture spans eager development and graph-based optimization across hardware from cloud TPUs to microcontrollers. TensorFlow 2 executes eagerly by default. Performance-sensitive functions can be captured with `tf.function`, and deployment formats route captured computation to server, mobile, and browser runtimes. This dual path preserves the production

graph capabilities inherited from TensorFlow 1.x without requiring every TensorFlow 2 program to build a complete graph before it runs.

Within a captured `tf.function`, TensorFlow can apply graph optimizations such as constant folding, kernel fusion, and layout planning. Figure 7.13 diagrams how the SavedModel serialization format acts as the central decoupling boundary across this pipeline. Heterogeneous training workflows on the left—incorporating `tf.data` preprocessing, modeling APIs (`tf.keras`, TensorFlow Hub, Estimators), and distribution strategies across CPUs, GPUs, and TPUs—serialize into a unified SavedModel artifact. From this central interface, graph definitions pass through supported serving or target-specific conversion paths into deployment runtimes on the right (TensorFlow Serving, TensorFlow Lite, TensorFlow.js, and native language bindings) without retaining Python runtime dependencies.

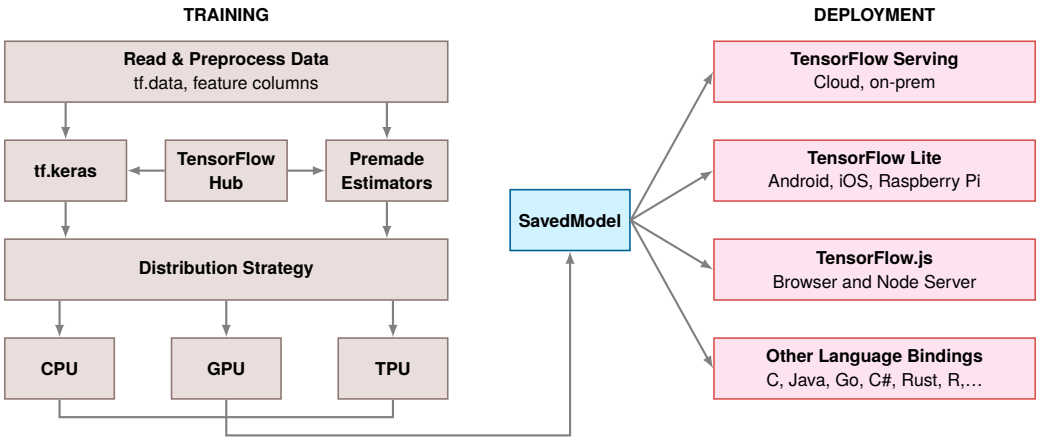


Figure 7.13: TensorFlow Training-to-Deployment Pipeline: The SavedModel format provides a boundary between training and deployment by capturing graph structure, weights, and signatures. The right-hand paths include direct serving and target-specific conversion for Lite and JavaScript runtimes. Supported operators and conversion determine whether each path can run without Python in production.

While TensorFlow 2.0 introduced eager execution to bridge the gap between research and production, TensorFlow 2.x still exposes `tf.function` as the graph-conversion path for performance-sensitive code (TensorFlow Developers 2024a). Its core strength remains the robust, compiled path from research to global-scale deployment.

7.7.2 PyTorch: The eager research standard

Where TensorFlow's design centers the path from captured graph to deployment runtime, PyTorch centers *developer experience*. PyTorch's architecture represents a sharply different answer to the execution problem, built on dynamic graphs (or "Define-by-Run"). Instead of building a blueprint before execution, PyTorch builds the computational graph on-the-fly as the code runs. Facebook AI Research (FAIR) adopted this design because researchers need immediate feedback when experimenting with novel architectures; the define-then-run cycle of static graphs introduced a compilation delay that slowed the rapid prototyping essential to research workflows.

PyTorch's approach fits exploratory research for the same reason: it treats deep learning as standard Python programming. Developers can use Python loops, conditionals, and debuggers (such as `pdb`) directly within a model's forward pass, with no special syntax, no separate compilation step, and no waiting to see if the code works. Eager execution enables rapid iteration and intuitive model design, which is essential when architectures and training objectives are still changing.

PyTorch's answer to the differentiation problem is tape-based autograd (section 7.4.2.1): flexible and debuggable, but harder to optimize globally because the tape is rebuilt each iteration. Its answer to the abstraction problem is more pragmatic than comprehensive: strong GPU support through cuBLAS and cuDNN, with deployment paths including `torch.export`, edge-targeted ExecuTorch, Open Neural Network Exchange (ONNX), and specialized runtimes.

The trade-off is therefore a more fragmented deployment path. Because the graph is dynamic, the framework cannot easily perform global optimizations before execution. A model that works perfectly in development may hit performance walls in production when dispatch overhead dominates small operations. To bridge this research-to-production gap, PyTorch introduced graph-capture and compilation paths, from TorchScript historically to `torch.compile` and export workflows, which allow developers to capture a dynamic model and turn it into an optimized representation for deployment. This evolution shows how an eager framework can move toward the production end of the compilation continuum while preserving the interactive experience that motivated the design.

7.7.3 JAX: The functional transformation engine

PyTorch's eager execution and TensorFlow's graph compilation represent two points on a spectrum, yet both share an imperative programming heritage where computation proceeds as a sequence of stateful operations. JAX represents a radically different user-facing approach, one built on functional programming principles and composable program transformations rather than object-level tapes or user-authored graph APIs (Bradbury et al. 2018). Developed by Google Research, JAX is especially useful for work requiring custom differentiation, advanced optimization research, and large-scale distributed training.

JAX's architecture treats differentiation, vectorization, and compilation as transformations on functions. The `jax.grad` function returns a function that computes gradients, which can then be transformed again when the interfaces are compatible. For example, `vmap(grad(f))` can compute per-example gradients and `jit(vmap(grad(f)))` can compile that batched computation. Transformation order is semantically meaningful rather than arbitrary.

JAX's functional paradigm shifts the programming model from "tracking state through objects" to "transforming pure functions." While PyTorch and TensorFlow expose autograd primarily through dynamic tapes or graph-compilation paths, JAX asks users to write pure Python functions and then applies transformations to those functions. Automatic differentiation, vectorization, and JIT compilation are all *program transformations* that can compose. Listing 7.23 demonstrates this approach.

Listing 7.23: JAX Function Transformations: JAX treats differentiation, vectorization, and compilation as composable function transformations rather than user-authored graph operations.

```
import jax
import jax.numpy as jnp


def loss_fn(params, x, y):
    pred = jnp.dot(x, params["w"]) + params["b"]
    return jnp.mean((pred - y) ** 2)


# Transform: compute gradients
grad_fn = jax.grad(loss_fn)

# Transform: vectorize over batch dimension
batched_grad = jax.vmap(grad_fn, in_axes=(None, 0, 0))

# Transform: compile to XLA
fast_batched_grad = jax.jit(batched_grad)

# Compose all three: fast, batched gradient computation
```

This functional approach requires **pure functions** (no side effects) and **immutable data** (arrays cannot be modified in place). These constraints may seem restrictive coming from PyTorch's mutable object model, but they enable formal guarantees: the compiler can safely reorder, fuse, and parallelize operations because function outputs depend only on inputs. The restriction is the feature; purity is what makes transformation composition possible.

JAX's power emerges from composition. `jax.grad` returns a gradient function; `jax.vmap` can vectorize a compatible function over mapped axes; and `jax.jit` can trace and compile a stable

transformed function for a particular argument signature. Unlike an iterative Python loop that issues individual operations sequentially with repeated dispatch overhead, `jax.vmap` pushes the batch dimension directly into the underlying primitives—allowing compatible per-sample matrix-vector operations to become batched matrix-matrix multiplications (GEMM). The compiled program can contain multiple kernels and library calls. `jax.pmap` maps a function across devices, but synchronization or gradient aggregation must be expressed with collectives such as `lax.psum` or `lax.pmean`. These transformations compose according to their types and semantics, and different orders can compute different quantities.

The same minimalist core delegates neural network abstractions to companion libraries (Flax, Haiku, Equinox) and optimization to Optax. This separation reflects the functional philosophy: the core provides transformations, while libraries build conventional abstractions on top. The trade-off is that production readiness depends not only on the transformation model, but also on the maturity of the surrounding libraries, export paths, and operational tooling for the target environment.

The functional constraints that JAX imposes become advantages in specific domains. Custom differentiation—higher-order gradients, custom vector-Jacobian product (VJP) and Jacobian-vector product (JVP) rules—composes cleanly because pure functions make differentiation rules predictable. Research on optimization algorithms benefits from transformations that let researchers manipulate gradient computation as naturally as they manipulate data. Compilation-heavy accelerator workloads use XLA to extract more utilization when the program can be expressed in this functional style. Scientific computing with AD requirements benefits from functional purity that enables mathematical reasoning about code. JAX requires more upfront investment than PyTorch: the functional paradigm has a learning curve, state management requires explicit patterns, and debugging compiled code is harder than eager execution. Teams should choose JAX when its strengths align with project requirements, not as a default.

7.7.4 Framework trade-offs under measurement

The preceding sections described each framework's design philosophy in qualitative terms: graph-first vs. eager-first, stateful vs. functional. The useful comparison is not which framework is fastest in the abstract, because that answer changes with model shape, batch size, hardware backend, and compiler configuration. The useful comparison is what each design lets the system see and optimize. Table 7.9 therefore maps TensorFlow, PyTorch, and JAX back to the three framework problems: execution visibility, differentiation model, and hardware abstraction path.

Table 7.9: Framework Design Trade-Offs: Each column reflects a distinct answer to the three core problems. TensorFlow makes deployment and graph capture central, PyTorch makes eager execution and debugging central, and JAX makes functional transformation and compiler visibility central. The table is a diagnostic map, not a benchmark ranking.

Aspect	TensorFlow	PyTorch	JAX
Graph Type	Static roots, dynamic front end in 2.x	Dynamic	Functional transformations
Programming Model	Imperative front end, graph capture path	Imperative	Functional
Core Data Structure	Tensor with framework-managed state	Tensor with framework-managed state	Immutable array
Execution Mode	Eager by default, graph for optimization	Eager by default	Trace and just-in-time compilation
Automatic Differentiation	Reverse mode over captured computation	Reverse mode over an eager tape	Forward and reverse transformations
Hardware Abstraction	Broad deployment runtimes and XLA paths	Native GPU path plus export/compile runtimes	XLA-centered accelerator compilation
Optimization Risk	Graph capture and operator coverage	Graph breaks after eager development	Purity, shape stability, and tracing

The measurement implication is straightforward: profile the constraint each framework makes most visible. In PyTorch, check whether eager dispatch or graph breaks dominate. In TensorFlow, check whether the captured graph covers the operators and deployment target. In JAX, check

whether shapes and purity let XLA compile the program actually executed. A framework-level comparison or leaderboard can orient a decision, but it cannot replace profiling the specific workload on the target hardware.

The same simple network exposes how each design philosophy shapes the code. Listing 7.24 implements one neural network, a single linear layer mapping ten inputs to one output, across all three frameworks.

Listing 7.24: Framework Comparison: Hello World: The same simple neural network implemented in PyTorch (object-oriented), TensorFlow/Keras (declarative), and JAX (functional), illustrating each framework's distinct design philosophy.

```
# PyTorch - Dynamic, Pythonic
import torch.nn as nn


class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 1)

    def forward(self, x):
        return self.fc(x)


# TensorFlow/Keras - High-level API
import tensorflow as tf

model = tf.keras.Sequential(
    [tf.keras.layers.Dense(1, input_shape=(10,))]
)

# JAX - Functional approach
import jax.numpy as jnp
from jax import random


def simple_net(params, x):
    return jnp.dot(x, params["w"]) + params["b"]


key = random.PRNGKey(0)
key_w, key_b = random.split(key)
params = {
    "w": random.normal(key_w, (10, 1)),
    "b": random.normal(key_b, (1,)),
}
```

These three implementations solve the same mathematical problem but reveal where each framework keeps state and when its graph exists. The differences are not cosmetic; they shape debugging workflows, deployment options, and optimization potential.

PyTorch binds state and computation together through class inheritance (`nn.Module`), solving the execution problem through eager evaluation: the graph builds as Python runs, making standard debuggers and control flow work naturally. Without graph capture, an optimizer cannot see the full computation before execution begins.

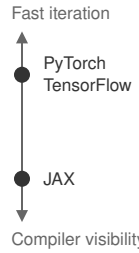
TensorFlow/Keras also executes eagerly by default in TensorFlow 2, while structured models can be captured with `tf.function` and exported to supported deployment runtimes. Portability still depends on operator coverage and conversion for each target.

JAX treats the model as a pure function[25] with immutable data and no internal state. In this model, `grad`, `vmap` (automatic vectorization), and `jit` (just-in-time compilation[26]) are composable transformations on stateless functions rather than methods attached to a mutable object system. The cost is explicit parameter management and a programming model that may be unfamiliar to engineers coming from stateful frameworks.

[25] **Pure function**: Returns outputs determined by its inputs without relying on untracked side effects; JAX transformations are designed for pure functions. Ordinary Python effects may run during tracing but are not represented in the compiled computation, and impure code can fail or behave unexpectedly. Runtime printing or external effects require supported mechanisms such as `jax.debug.print` or callbacks; random draws require explicitly managed keys.

[26] **Just-in-time (JIT) compilation**: Traces a function for an argument signature and lowers it to backend-specific executable code; the first call pays tracing and compilation costs; compatible later calls can reuse the cached executable. New shapes, dtypes, devices, or static arguments may trigger another compilation. Both compilation time and cached-call overhead depend on the program, backend, and hardware.

No framework maximizes all three goals simultaneously, and the ten-line program makes the trade visible in source code rather than buried in compiler internals: where state lives, when the graph exists, and what the compiler is allowed to see. The underlying currency is compiler visibility vs. human iteration latency. Graph capture and ahead-of-time compilation can reduce runtime overhead and intermediate data movement; eager evaluation shortens the human iteration loop, outside the iron-law runtime equation, at the cost of compiler visibility until a graph-capture path is used; functional purity gives XLA more freedom to transform a stable traced program. Each philosophy also shapes code syntax, team workflows, debugging practices, and deployment pipelines, so migration costs grow with project-specific integrations and assumptions.



Eager defaults favor iteration; JAX's traced functions favor the compiler.

7.8 Deployment Targets

Deploying a model from an 8-GPU training cluster to an autonomous vehicle's 30-watt embedded compute module or a 50-milliwatt IoT sensor completely reweights the three core framework problems across the cloud-to-edge spectrum. While the trade-offs mapped in section 7.7.4 determine training throughput and operational cost in a data center, deploying to edge environments turns runtime efficiency into a binary feasibility test. The execution problem shifts from trading debuggability against graph optimization to fitting computation strictly within latency and memory envelopes. The differentiation problem disappears for pure inference workloads, eliminating the storage overhead of activation stashes and backward graphs. Meanwhile, the abstraction problem intensifies as software targets heterogeneous compute engines: server x86 and ARM CPUs, mobile NPUs, edge TPUs, and bare-metal microcontrollers operating within tens of kilobytes of SRAM.

Table 7.10 maps these runtime assumptions across the cloud-to-edge spectrum, detailing the optimization levers and binding physical constraints that define each hardware tier.

Table 7.10: Deployment Target Constraint Map: Runtime assumptions, optimization levers, and binding constraints for each deployment tier, from cloud servers to microcontrollers.

Environment	Runtime assumptions that usually fit	Optimization lever	Binding constraint
Cloud/Server	Full training/serving frameworks	Graph compilation, batching, lower precision	Throughput, cost
Edge	Static graph or portable serving runtime	Static graphs, lower-precision kernels	Workload-specific latency and memory
Mobile	App-integrated runtime with delegates	Accelerator delegates, compact model formats	Battery, thermal, app size limits
Microcontroller (TinyML)	Tiny runtime with fixed allocation	Static allocation, small integer kernels	Tight RAM, no dynamic memory

The target narrows the runtime family before any performance comparison is useful. The KWS microcontroller in section 7.3.5 requires a fixed arena and a compact native runtime; a mobile device can instead support an application runtime with accelerator delegates. A delegate routes supported operators to a target accelerator, while unsupported operations need another execution path. Within the target envelope, the remaining question is which runtime can execute the actual operator graph.

7.8.1 Runtime constraint map

The deployment envelope in table 7.10 identifies a hardware tier; table 7.11 identifies candidate software paths within it. Its last column names the conversion or execution condition that each candidate must pass before latency or memory measurements can be compared.

Table 7.11: Runtime Constraint Map: Read each row left to right as one bargain, where a family gives up flexibility, gains an optimization path, and acquires a new way to fail. The middle column records where a family sits relative to an untuned eager baseline rather than a measured result, so latency, memory use, energy, and utilization still have to be measured for the specific model, hardware, precision, batch size, and compiler configuration.

Runtime family	Where it fits	Constraint anchor	Optimization path	Constraint to test
PyTorch eager	Research and iteration	Baseline latency; full Python/runtime footprint	Dynamic graphs, eager debugging	Dispatch overhead and missing graph view
Compiled PyTorch/TensorFlow	Server training and serving	Workload-dependent gains when graph capture is clean	Graph capture, fusion, layout planning	Operator coverage and graph breaks
TensorFlow Lite/Core ML	Mobile and edge inference	Target-specific mobile latency and package-size budgets	Quantization, static graphs, NPU delegates	Target-specific conversion constraints
TF Lite Micro/microTVM	Microcontroller inference	Target-specific microcontroller RAM and flash budgets	Static allocation, INT8 kernels	Selected operators and constrained memory
ONNX Runtime	Cross-framework serving	Backend-dependent; can match native runtimes for supported graphs	Standard graph format, execution providers	Export gaps and custom-operator fallbacks
TensorRT/TVM	Hardware-specialized inference	Target-dependent gains over untuned eager baselines	Kernel fusion, precision lowering, autotuning	Narrower target and conversion assumptions

A runtime can fit the device and still fail to execute the exported model. Specialized inference runtimes such as TensorRT and Apache TVM require compatible operators and conversion paths; a mobile delegate may support only part of a graph. Unsupported regions can introduce fallback execution and extra transfers. Portability therefore requires an agreement about the graph and operator semantics on both sides of the export boundary.

The development framework and deployment runtime need not be the same system, but they must exchange a representation both can interpret. The **ONNX**[27] format addresses this fragmentation by standardizing model exchange across runtimes (ONNX Contributors 2019). A pipeline can train an architecture in PyTorch, export the computation graph to an ONNX intermediate representation, and execute the serialized graph in ONNX Runtime or compile it for a target accelerator. In contrast, TensorFlow Lite relies on its own conversion path, translating TensorFlow SavedModels into FlatBuffers rather than routing through ONNX. Although standard interchange formats reduce manual model conversion between development and production environments, they do not eliminate compatibility testing, operator coverage gaps, or custom-kernel fallback overhead.

Figure 7.14 diagrams this hub-and-spoke interoperability topology centered on the common ONNX intermediate representation. On the left, training frameworks—including PyTorch, TensorFlow, Keras, Caffe2, MXNet, and CNTK—route inward spokes along the "Export to ONNX" pathway. On the right, outward spokes fan out through the "Load from ONNX" boundary to target execution engines, delivering the serialized computation graph to ONNX Runtime and downstream target frameworks without reimplementing model code.

Interchange formats mitigate framework fragmentation, but an export boundary cannot salvage an architecture that violates the target's physical limits. Converting a graph to ONNX or an embedded runtime after development completes cannot resolve an excessive memory footprint, missing hardware operators, or unsupported dynamic control flow. Treating deployment as an afterthought guarantees costly rewrites, making systematic evaluation of hardware and runtime trade-offs essential before writing the first line of model code.

[27] **ONNX**: The framework used for model development often differs from the runtime best suited to the deployment target. ONNX defines a hardware-agnostic graph representation that decouples the two, reducing the engineer-months of manual model conversion that would otherwise be required each time a deployment target changes. The accepted trade-off is that ONNX export can lose framework-specific optimizations or custom operators, requiring fallback implementations.

↪ Selecting among portable runtimes such as ONNX Runtime and specialized inference engines that execute these exported graphs is operationalized in section 13.8.

7.9 Framework Selection

Suppose a team deploys a fixed-shape vision model to a CPU-only embedded Linux device, with a 20 ms p99 inference-latency target under a specified load and a 256 MB model-and-runtime memory budget. Assume target-device trials with matching inputs, batch size, precision, and numerical tolerances establish two viable paths: PyTorch development with ONNX Runtime has 18 ms p99 latency and 140 MB peak memory use, while TensorFlow development with TensorFlow Lite has 15 ms p99 latency and 90 MB peak memory use. A GPU-only specialized runtime fails the hardware filter even if its latency elsewhere is lower. These are hypothetical project measurements, not general

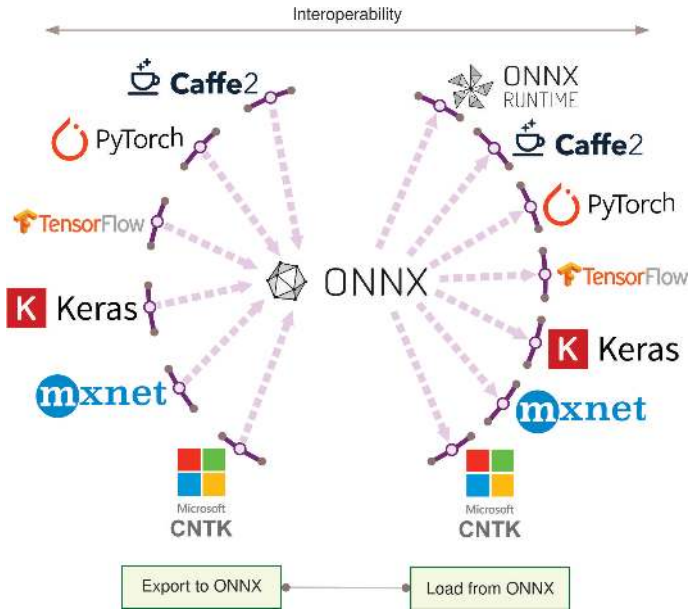


Figure 7.14: Framework Interoperability: The inward spokes represent exports into the shared ONNX format, while the outward spokes represent loads into compatible tools and runtimes; portability requires support on both sides of that boundary.

performance rankings. Framework selection must now distinguish a feasible path from a worthwhile migration.

7.9.1 The framework selection trade-off space

The embedded project has already fixed the deployment target, but it leaves two choices open: which development environment the team can maintain and which compatible runtime meets its budget. Eager development and compiled deployment can coexist along either path, so the team need not trade away its debugging workflow to obtain a compact inference runtime. A specialized engine such as TensorRT or TVM offers no advantage unless its backend supports this CPU target and its conversion path supports the model (NVIDIA 2024c; Chen et al. 2018). ONNX reduces the exchange burden without proving either condition (ONNX Contributors 2019).

> **Systems Perspective 7.4: Framework selection constraints**
>
> Rather than seeking the "best" framework, effective selection first eliminates candidates that cannot satisfy hard constraints: deployment target, required operations, compiler path, and team expertise. Only then should soft preferences such as performance, development speed, and ecosystem rank the survivors.

7.9.2 Framework selection criteria

The two hypothetical measurements establish latency and memory feasibility only for the tested model and inputs. A defensible choice also checks operator semantics, software dependencies, and hardware coverage. The TensorFlow variants below make those filters visible within one ecosystem; the same questions apply to the PyTorch-to-ONNX path.

7.9.2.1 Model requirements

For the fixed-shape vision model, the first filter is successful conversion of every required operation with acceptable numerical results. Training support is needed in the development framework but not in the deployed runtime. TensorFlow, TensorFlow Lite, and TensorFlow Lite Micro expose

different operator and training capabilities. A missing inference operator, by contrast, can invalidate the deployment path even when the rest of the graph fits. Table 7.12 separates those capabilities.

Table 7.12: TensorFlow Variant Model Support: The four rows show what each variant can express, not how large it is. Training capability disappears across the tiers while inference and lower-precision tooling survive intact, which leaves operator coverage as the discriminating row: broad in TensorFlow, inference-focused in TensorFlow Lite, and reduced to the operators one model selects in TensorFlow Lite Micro. Binary size and memory footprint appear in table 7.14.

	TensorFlow	**TensorFlow Lite**	**TensorFlow Lite for Microcontrollers**
Training	Yes	Limited	No
Inference	Yes	Yes	Yes
Operator Coverage	Broad	Inference-focused	Model-selected subset
Native Lower-Precision Tooling	Yes	Yes	Yes

> **Systems Perspective 7.5: Dynamic vs. static computational graphs**
>
> The static-vs.-dynamic graph distinction (examined in section 7.3) has direct implications for model requirements analysis. Static graphs require control flow and dynamic behavior to be represented in forms understood by the graph runtime, which enables ahead-of-time optimization for deployment. Eager execution follows ordinary Python control flow directly, while graph-capture paths such as `torch.compile` and `tf.function` recover optimization opportunities when they can represent the executed program.

7.9.2.2 Software dependencies

Embedded Linux supplies operating-system services, so an OS dependency does not eliminate either candidate in this project. A bare-metal microcontroller would impose a different filter. Table 7.13 separates that requirement from model access and accelerator integration; availability of a runtime on one tier does not imply availability on another.

Table 7.13: TensorFlow Variant Capability Comparison: Capabilities of TensorFlow, TensorFlow Lite, and TensorFlow Lite Micro regarding operating system dependence, memory management, and hardware acceleration. Progressive constraint across variants enables selection by deployment context, from full-scale servers to resource-constrained edge devices.

	TensorFlow	**TensorFlow Lite**	**TensorFlow Lite for Microcontrollers**
Needs an OS	Yes	Yes	No
Model Access	Filesystem or service	Platform-specific storage mapping	Platform-specific compiled or mapped storage
Accelerator Support	Yes	Delegates	Optimized kernels and integrations

The CPU-only project needs a supported CPU implementation of every required operator; it cannot count on a mobile NPU delegate or a server GPU backend. TensorFlow Lite Micro instead links a selected operator set into a small C/C++ application and can read a model FlatBuffer compiled into flash. That design removes the need for a POSIX filesystem or heap allocation during inference, but those savings do not by themselves justify selecting a microcontroller runtime for a Linux device.

7.9.2.3 Hardware constraints

A runtime that supports the operating system still has to fit alongside the model, working buffers, and application. Table 7.14 separates supported processor families from footprint, which depends on the actual build. The project must compare peak memory for the deployed configuration, not the size of its weight file or an empty runtime package.

Table 7.14: TensorFlow Hardware Targets: TensorFlow variants target progressively more constrained hardware, shifting from x86, GPUs, and TPUs to Arm Cortex-A platforms and then Arm Cortex-M processors, DSPs, and MCUs. Actual binary size and memory footprint depend on the build, selected operators, delegates, model, and runtime configuration.

	TensorFlow	**TensorFlow Lite**	**TensorFlow Lite for Microcontrollers**
Base Binary Size	Build- and package-dependent	Build- and delegate-dependent	Model- and operator-selection-dependent
Base Memory Footprint	Model-, runtime-, and allocator-dependent	Model-, delegate-, and allocator-dependent	Fixed arena plus runtime state
Optimized Architectures	x86, TPUs, GPUs	Arm Cortex-A, x86	Arm Cortex-M, DSPs, MCUs

The supplied trials already include model-and-runtime peak memory, so both candidates pass the project's capacity filter. That conclusion must be revisited when input shapes, selected operators, delegates, or runtime versions change. On a fixed-arena target, the analogous check is whether the planned tensor and scratch buffers fit with the rest of the application.

7.9.2.4 Production-ready evaluation factors

Satisfying model requirements, software dependencies, and hardware constraints establishes technical feasibility, but production viability requires evaluating operational reliability under sustained serving workloads. Runtimes that rely on dynamic memory allocation or host-language interpreters introduce tail-latency (p99) variance through garbage collection pauses and heap fragmentation. Static tensor arenas and ahead-of-time graph lowering can reduce allocation and interpreter variability. Tail latency still requires measurement under the target workload, including contention and synchronization.

Precision optimization forms a central operational lever. Lowering numerical precision from FP32 to FP16 or BF16 halves bytes per represented value, and lowering it to FP8 or INT8 quarters them; realized traffic and throughput depend on supported kernels and conversions. When a deployment uses quantization, its toolchain must support the chosen path: post-training quantization (PTQ), which converts an already-trained model using a small representative calibration dataset to set value ranges, or quantization-aware training (QAT), which simulates low-precision arithmetic during training to recover accuracy margins before deployment. Beyond per-device execution, production viability depends on toolchain stability: whether model checkpoints maintain backward compatibility across runtime updates, whether the framework exposes telemetry hooks for profiling kernel execution, and whether serving infrastructure can deploy updated graph definitions without downtime.

↪ Post-training quantization, its calibration, and quantization-aware training are developed in section 10.4.3.

7.9.3 Development support and long-term viability assessment

Once both paths pass the technical filters, the remaining decision concerns the team's ability to maintain them. The project already has PyTorch models and an ONNX export pipeline. Adopting another development stack would add conversion validation, new debugging practices, and another set of runtime dependencies even though both candidates meet the deployment limits.

Custom C++ or CUDA operators, specialized serialization, and vendor compiler passes increase the cost of later migration. Preserving an exit path means isolating framework-specific ingestion and operator code and testing the intended export format throughout development. These commitments belong in the selection decision because the shortest inference time does not price the work needed to keep the model deployable.

Both surviving paths meet the stated runtime limits. The team already maintains PyTorch models and an ONNX export pipeline, so it selects PyTorch with ONNX Runtime. The measured path satisfies deployment requirements without introducing a second development stack. The faster TensorFlow Lite path remains viable, but its unused latency and memory advantage does not by itself justify migration. The decision depends on retaining that headroom as the model changes. Before committing, the team must check export coverage and numerical tolerances across representative inputs and repeat the latency check under expected contention; a later operator change or tighter latency target can invalidate the choice. The choice follows the project's constraints and maintenance costs; a different team or deployment envelope can favor the other viable path.

Execution, differentiation, and hardware abstraction are distinct architectural choices. In an executing model, however, these three systems operate simultaneously. A single training step is where they collide: eager dispatch or graph scheduling determines when work reaches silicon, reverse-mode automatic differentiation generates the backward pass while retaining intermediate activations, and hardware abstractions coordinate tensor layouts and kernel launches. Tracing one complete training step from input batch to parameter update reveals how these mechanisms integrate under physical machine constraints.

7.10 Anatomy of a Training Step

In every iteration of a training loop, millions of floating-point operations, dynamic memory allocations, and hardware kernel launches must synchronize within tens of milliseconds. Execution, differentiation, and hardware abstraction operate simultaneously during model training. In listing 7.25, a minimal two-layer multilayer perceptron illustrates this convergence across eight executable statements. Tracing each statement through the runtime stack connects the quantitative principles developed in section 7.3.4 to physical execution: tensor allocation, kernel dispatch, dynamic graph recording, gradient computation, and parameter updates.

The example assumes trainable CUDA tensors `W1` and `W2` with shapes `(784, 256)` and `(256, 10)`, matching bias vectors `b1` and `b2`, and an optimizer already constructed over these parameters. The listing shows one iteration after that setup.

Listing 7.25: Training Step Anatomy: A minimal training iteration for a two-layer MLP, exercising tensor allocation, kernel dispatch, autograd recording, gradient computation, and parameter updates.

```
# Single training step for a 2-layer MLP
x = torch.randn(32, 784, device="cuda")  # Input batch
y = torch.randint(0, 10, (32,), device="cuda")  # Labels

# Forward pass
optimizer.zero_grad()
h = torch.relu(x @ W1 + b1)  # Hidden layer
logits = h @ W2 + b2  # Output layer
loss = F.cross_entropy(logits, y)

# Backward pass
loss.backward()

# Parameter update
optimizer.step()
```

Phase 1: Forward pass (solving the execution problem)

During the forward pass, executing `h = torch.relu(x @ W1 + b1)` triggers an eager execution sequence:

1. Python dispatch: The Python interpreter invokes `torch.matmul`, which routes through PyTorch's C++ dispatcher to select the CUDA backend, incurring the Python-dispatch share of the per-operation tax (about 10 µs in this chapter's model) before issuing work to the device.
2. Kernel selection: cuBLAS selects a GEMM kernel configured for dimensions $32 \times 784 \times 256$, partitioning the computation into thread blocks sized to fit the streaming multiprocessor shared memory and L2 cache tiles.
3. Kernel launch: The host thread enqueues the launch descriptor into the GPU command stream, consuming driver launch latency while asynchronous execution allows host execution to proceed without blocking on completion.
4. GPU execution: The streaming multiprocessors stream `W1` from HBM[28] through L2 cache into registers and Tensor Cores, evaluate the matrix product, and write the output activations back to HBM.

[28] **HBM (high bandwidth memory)**: Provides 2–3 TB/s bandwidth on modern GPUs, making it the memory tier that most directly feeds accelerator arithmetic; HBM bandwidth determines whether operations are memory bound or compute bound, and its 80 GB capacity on an A100 sets the hard ceiling on total live training state. Weights, activations, gradients, optimizer state, and temporary workspace must fit during execution. When they do not, the framework must resort to offloading, selective recomputation, or placement across devices, each adding complexity to what the programmer perceives as a single `loss.backward()` call.

5. Autograd recording: Concurrently on the host, PyTorch's autograd engine instantiates an `MmBackward` node on the dynamic tape, saving references to the device tensors needed for the requested gradients. Here the weight gradient requires `x`; no input gradient is requested.

The subsequent bias addition (`+ b1`) and activation (`torch.relu`) follow an identical eager path, with each operation dispatching a separate kernel, executing round-trips to HBM, and appending a node to the tape.

Phase 2: Backward pass (solving the differentiation problem)

Calling `loss.backward()` executes reverse-mode automatic differentiation in four stages:

1. Tape traversal: The autograd engine traverses the recorded computational graph in reverse topological order, starting from the scalar loss node.
2. Gradient computation: The engine executes the backward kernel registered to each node. Traversing backward, `CrossEntropyBackward` evaluates $\frac{\partial \mathcal{L}}{\partial \text{logits}}$ using the combined softmax-cross-entropy derivative; `MmBackward` for $\mathbf{W}_2$ computes $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{H}^T \cdot \frac{\partial \mathcal{L}}{\partial \text{logits}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{H}}$, where $\mathbf{H}$ is the batch of hidden activations (`h`); `ReluBackward` evaluates the activation mask, zeroing gradients where forward activations satisfy $\mathbf{H} \leq 0$; and `MmBackward` for $\mathbf{W}_1$ computes $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \mathbf{X}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1}$, where $\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1$ is the pre-activation. Because the input batch $\mathbf{X}$ (`x`) does not set `requires_grad=True`, traversal terminates without evaluating $\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$.
3. Gradient accumulation: Computed parameter gradients accumulate in-place into the `.grad` fields of leaf tensors ($\mathbf{W}_1$, $\mathbf{b}_1$, $\mathbf{W}_2$, $\mathbf{b}_2$).
4. Saved-state release: As each backward node completes, the framework releases the saved tensor references that it no longer needs. Storage becomes reusable only when no other live reference retains it; in this listing, Python variables such as `h` can keep tensors alive after backward. The caching allocator can reuse released regions for subsequent operations without returning them immediately to the device.

After backward fills the parameter `.grad` buffers, `optimizer.step()` reads those gradients, updates the parameters, and advances any optimizer state. This is the update represented in the optimizer memory ledger; backward itself computes gradients without applying that update.

Phase 3: Memory traffic analysis (the physics at work)

The ledger in table 7.15 follows the products required by the backward traversal above. It excludes input generation, bias additions and reductions, optimizer updates, and allocator bookkeeping. Within that boundary, each matrix multiplication reads two operands and writes one result. Counting these buffers and the corresponding arithmetic connects the graph to an execution-time estimate; loss and activation kernels receive the simpler operation-equivalent models stated below.

The traffic model assumes FP32 values occupying 4 bytes each. For `x @ W1`, it counts one read of the input matrix with shape `(32, 784)`, one read of the weight matrix with shape `(784, 256)`, and one write of the result with shape `(32, 256)`. Multiplying each shape's dimensions, summing the three element counts, and multiplying by the element width gives the table's 0.9 MB. The second matrix multiplication uses the same operand rule with its own shapes.

ReLU instead reads and writes each element of its `(32, 256)` tensor once, so its modeled traffic is twice that tensor's element count times 4 bytes, yielding 65.5 KB. These counts describe the modeled operator-buffer boundary. They assume full operand reuse within each matrix multiplication and exclude extra layout copies, workspace, and repeated tile loads; cached operands can also reduce actual HBM reads. Saving a reference for backward does not itself copy a tensor. A physical layout conversion, when required, adds its own read and write traffic to the ledger.

The backward products use the same read/read/write rule. The weight gradient `h.T @ dlogits` multiplies `(256, 32)` by `(32, 10)`; the hidden gradient `dlogits @ W2.T` multiplies `(32, 10)` by `(10, 256)`; and `x.T @ dz1` multiplies `(784, 32)` by `(32, 256)`. The first two each cost as much arithmetic as the second forward GEMM, while the third matches the first forward GEMM. No `(32, 256)` by `(256, 784)` product is needed because the raw input requests no gradient. The total dense matrix work is therefore 2.013× the forward dense work. This applies the input-gradient exception derived in section 5.5.5.3 to this two-layer network.

The forward cross-entropy model assigns one operation-equivalent each to exponentiation, accumulation, and normalization per logit, and counts the logit read and saved-score write. For the nonlinear backward work, the ReLU model reads the saved activation and upstream gradient and writes a masked gradient. The simplified combined cross-entropy model reads saved probabilities and integer labels, then subtracts the target and scales by the batch mean to write the logit gradient. Actual loss implementations may split these operations across kernels. The resulting estimates make the arithmetic-intensity comparison reproducible without substituting for a measured memory-traffic profile:

Table 7.15: Per-Operation Roofline Analysis: Analytical work and buffer-traffic estimates for the selected operations of a two-layer MLP. Each GEMM counts two operand reads and one result write; backward includes only the requested gradients. Cross-entropy and pointwise derivative work use the simplified models stated in the text. These are operator-level estimates rather than measured device instruction or HBM-transaction counts.

Component	Modeled Work	Memory Traffic	Arithmetic Intensity
MatMul (x @ W1)	$2 \times 32 \times 784 \times 256$ = 12.8 MFLOP	0.9 MB	13.7 FLOP/byte
ReLU	32×256 = 8.2 KFLOP	65.5 KB	0.125 FLOP/byte
MatMul (h @ W2)	$2 \times 32 \times 256 \times 10$ = 163.8 KFLOP	44.3 KB	3.7 FLOP/byte
Cross-entropy (simplified)	~0.96 KFLOP	2.6 KB	0.4 FLOP/byte
Backward `dW2`	163.8 KFLOP	44.3 KB	3.7 FLOP/byte
Backward `dh`	163.8 KFLOP	44.3 KB	3.7 FLOP/byte
Backward `dW1`	12.85 MFLOP	0.94 MB	13.7 FLOP/byte
Backward nonlinear (simplified)	8.8 KFLOP	101.1 KB	0.087 FLOP/byte

The arithmetic-intensity column helps predict each layer's regime when compared with the target machine's compute-to-bandwidth ratio. Matrix multiplications exhibit more operand reuse than element-wise operations, but their shapes still determine whether they reach the compute roofline. Conversely, the forward ReLU and cross-entropy estimates perform negligible compute per byte transferred (0.125 to 0.375 FLOP/byte), making them sensitive to memory bandwidth and, at small sizes, launch latency. In un-fused eager execution, each element-wise operation incurs a separate kernel dispatch and writes intermediate tensors to global-memory buffers between kernels; actual HBM traffic depends on caching.

The analytical model yields ~26.2 MFLOP of arithmetic work and ~2.2 MB of memory traffic for the operations included in the ledger. Dividing these quantities by peak A100 hardware rates determines the idealized lower bounds:

- $T_{\text{compute}} \approx$ 26.2 MFLOP/19.5 TFLOP/s FP32 $\approx$ 1.3 µs
- $T_{\text{memory}} \approx$ 2.2 MB/2.04 TB/s $\approx$ 1.1 µs
- Under a one-kernel-per-modeled-operator assumption, the four selected forward operators and five backward operators require 9 driver launches. Python enters each forward operation and calls `backward()` once, giving 5 entries. The illustrative host cost is therefore 5 × 10 µs plus 9 × 5 µs, or 95 µs.

This dispatch model excludes autograd traversal overhead, allocation, and the operations omitted from the arithmetic ledger. The backward engine launches its internal kernels without a separate Python call for each one; a profile must establish the actual launch count, particularly for loss kernels. Host costs are added without overlap here. In this model, T_{sw} exceeds the ideal lower bounds T_{compute} and T_{memory} by more than an order of magnitude, so the overhead ratio of equation 7.4 sits far above 1 and the step is overhead-bound under these assumptions. This is the small-MLP regime of notebook 7.2, now reached from a real training loop. Profiling has a specific job here, which is to confirm that measured kernel times, achieved bandwidth, and host-device overlap match the model before acting on it. If dispatch dominates, the following mechanisms target the launch tax:

- Kernel fusion (`torch.compile`): Fusing consecutive memory-bound operations into a single kernel avoids intermediate round-trips to HBM and collapses launch boundaries.

- Batch size scaling: Increasing the batch size scales the arithmetic payload linearly while keeping dispatch overhead constant, amortizing fixed launch latency across more compute operations.
- Graph capture: Compilation reduces repeated Python work in captured regions; CUDA Graph replay can submit a captured launch sequence with less host overhead. Neither mechanism requires the entire step to become one fused kernel.

Phase 4: Hardware abstraction (solving the abstraction problem)

The tensor operations in this training step map to heterogeneous hardware architectures through backend abstraction layers, as table 7.16 summarizes. The displayed script uses CUDA; another target requires matching device placement and a supported backend execution path.

Each backend maps common tensor semantics to silicon-specific primitives. A single `loss.backward()` invocation routes to different execution pipelines depending on device placement. Differences in floating-point summation order, reduction trees, and non-deterministic atomic operations introduce numerical variation across backends, requiring equivalence to be validated against explicit numerical tolerances rather than bitwise identity.

Table 7.16: Hardware Backends Behind a Single Code Path: Each backend implements the same tensor operations through a different library and execution mechanism, so identical Python maps onto distinct silicon.

Hardware	Backend library	Execution mechanism
CUDA GPU	cuBLAS (NVIDIA 2024a; Choquette et al. 2021)	GEMM kernels and CUDA streams for async execution
CPU	Intel oneMKL or OpenBLAS (Intel Corporation 2026a; OpenBLAS Project 2026)	Thread-level parallelism around optimized kernels
TPU	XLA (Google 2025)	Compilation to TPU-specific high-level optimizer (HLO) operations
Apple Silicon	Metal Performance Shaders	MPS backend

Implementation decisions—device placement, batch size, compilation mode—cascade through execution, differentiation, and abstraction layers simultaneously. When systems treat the framework as an opaque mathematical interpreter rather than a hardware-aware scheduler, these interactions trigger launch starvation, memory fragmentation, and throughput collapse.

> **Systems Perspective 7.6: The three problems in action**
>
> In D·A·M terms, the single training step exposes how the framework balances algorithm and machine:
>
> - Execution (Machine scheduling): Eager mode preserves imperative line-by-line debugging at the cost of per-kernel launch and dispatch latency.
> - Differentiation (Algorithmic graph): Reverse-mode automatic differentiation retains intermediate forward activations in HBM to compute exact vector-Jacobian products in reverse topological order.
> - Abstraction (Hardware portability): Unified tensor APIs decouple model code from heterogeneous silicon, dispatching identical operations to cuBLAS, oneMKL, or XLA HLO kernels.
>
> Resolving the friction among these dimensions requires matching operator granularity to hardware rooflines: fusing operations to eliminate memory round-trips, scaling batches to amortize launch latency, and structuring tensors to saturate accelerator execution pipelines.

Balancing execution models, differentiation overhead, and hardware abstractions creates a high-performance execution engine when applied correctly. Yet because these mechanisms interact below the surface of standard Python APIs, engineers frequently rely on simplistic assumptions carried

over from general-purpose programming. Examining where these assumptions break down reveals the subtle traps that derail real-world framework deployments.

7.11 Fallacies and Pitfalls

Framework selection involves subtle trade-offs where intuitions from conventional software engineering fail. The memory wall, kernel fusion constraints, and deployment target diversity create pitfalls that waste engineering effort and cause production systems to miss latency targets.

Fallacy: *All frameworks provide equivalent performance for the same model architecture.*

Engineers assume that ResNet-50 yields identical performance across frameworks since the mathematics is the same, forgetting that performance is an emergent property of algorithm-machine co-design. In production, implementation matters. Compiled execution can improve supported eager workloads, and hardware-specialized inference engines such as TensorRT or TVM can reduce latency relative to untuned eager baselines when conversion is clean. The difference arises from kernel fusion depth, graph optimization strategies, memory access patterns, precision support, and backend libraries that vary between frameworks. Organizations that assume equivalence can miss latency service level agreements and require costly last-minute framework migrations.

Pitfall: *Choosing frameworks based on popularity rather than project requirements.*

Engineers assume the most popular framework works for any project. In reality, deployment constraints dominate. A mobile runtime such as ExecuTorch or TensorFlow Lite and a microcontroller runtime such as TensorFlow Lite Micro make different assumptions about memory allocation, operator coverage, and hardware delegates; the former targets phones with GB-scale memory, while the latter often targets devices with hundreds of KB of RAM, commonly below 256 KB for small TinyML deployments. Teams that prototype edge applications without checking the final runtime can face memory bloat that exceeds device capacity or a late framework migration after development completes. Evaluate deployment targets per section 7.8 before selecting a training framework.

Fallacy: *Framework abstractions eliminate the need for systems knowledge.*

Engineers assume high-level APIs handle all optimization automatically. The Roofline Model proves otherwise. Element-wise operations such as ReLU perform little arithmetic per byte moved, so memory traffic and launch overhead can dominate even on accelerators with abundant compute. Section 7.3.1 explains how this imbalance can leave compute resources idle. Engineers must identify the limiting resource before choosing fusion, batching, layout, or precision optimizations.

Pitfall: *Ignoring vendor lock-in from framework-specific formats.*

Engineers assume framework migration is straightforward since models are "just math." Converting TensorFlow SavedModel to PyTorch can require rewriting custom operations, validating numerical equivalence across large test suites, and retraining when operations lack exact equivalents. ONNX (section 7.8) improves portability, but custom operators, dynamic shapes, and backend-specific optimizations can still require manual work. Organizations that ignore this during initial framework selection face costly migrations when deployment requirements change or better frameworks emerge.

Fallacy: *Training framework choice is independent of production infrastructure.*

Engineers assume training framework choice is independent of deployment infrastructure. In practice, framework-infrastructure mismatches can impose substantial operational overhead. Some serving stacks provide atomic model swaps, while others require a process or container restart unless the deployment architecture adds version routing around them. Some frameworks and runtimes expose monitoring hooks directly; others require custom instrumentation. The lesson is to evaluate the complete deployment stack during framework selection, including serving infrastructure, monitoring, and operational tooling.

↪ Model versioning, staged release strategies, and serving integration that a framework's deployment path must support are operationalized in section 14.5.3.

Pitfall: *Increasing batch size without modeling activation memory.*

Engineers assume that if memory is available, larger batches always improve throughput. Larger mini-batches can amortize fixed dispatch overhead, but they also scale the activation memory that must remain live during training. A 7-billion-parameter model in FP16 consumes 14 GB, leaving 71.9 GB on an 80 GB A100 before accounting for other training state. Increasing batch size from

8 to 32 quadruples the batch-dependent activation footprint; materialized transformer attention scores add a large $\mathcal{O}(S^2)$ term in sequence length that makes each sample expensive. The resulting memory pressure can trigger recomputation strategies whose additional work reduces throughput despite the larger batch. Teams that blindly maximize batch size can achieve lower throughput than smaller batches that avoid these memory management pathways; section 8.4.2.1 formalizes the throughput target.

↪ How activation memory grows linearly with batch size and caps the batch that fits in accelerator memory is developed in section 10.

Fallacy: *Compilation overhead is negligible.*

Engineers assume compilation overhead is a one-time cost that pays off quickly. The hypothetical values in table 7.3 assign ResNet-50 higher compiled throughput and a compile window of 15 s to 30 s per graph change. Under those assumptions, timed with the table's inference throughputs (a training loop would add backward work but pay the same recompilation cost), a 10,000-image experiment with 10 code changes completes in 6.9 s in eager mode and 304.7 s in compiled mode, including recompilation overhead. The compiled workflow is therefore about 44.2× slower in this illustrative rapid-prototyping scenario. Teams that enable compilation during rapid prototyping can waste time waiting for recompilations that negate throughput gains.

Pitfall: *Using one execution policy for exploration and production.*

The right framework mode depends on the loop. During exploration, eager execution and small tests shorten feedback by avoiding repeated graph captures and recompilations. During production serving or long training runs, compilation can amortize setup across many requests or samples. The deciding factors are workload duration and stability: compilation pays when graph structure and shapes recur, whereas changing control flow or input shapes can invalidate cached artifacts and restart the amortization. Teams should choose the policy from measured iteration latency and steady-state throughput. One policy for both phases either slows research iteration or leaves production throughput unused.

7.12 Summary

Machine learning frameworks solve three core systems challenges that make deep learning tractable on modern hardware. The first is execution: deciding when and how operations run. Frameworks balance the trade-off between eager evaluation for interactive debugging and graph execution for whole-program compilation, while modern hybrid approaches such as `torch.compile` capture graphs dynamically without sacrificing the Python control loop. The second is differentiation: computing gradients automatically. Frameworks implement reverse-mode automatic differentiation that applies the chain rule across compositions of operations, constructing an autograd tape or backward graph of vector-Jacobian products while making the required forward values available through retention or recomputation. This primitive makes training large models feasible through a single `loss.backward()` call. The third is abstraction: targeting heterogeneous hardware from a unified interface. Frameworks provide multi-dimensional tensor abstractions, intermediate representations, and runtime dispatch systems that bridge high-level model definitions to optimized device kernels across CPUs, GPUs, TPUs, and custom accelerators.

These challenges are linked through the iron law of ML systems (section 1.7): execution strategy affects dispatch latency (L_{lat}), differentiation adds arithmetic and memory traffic (D_{vol}), and abstraction influences hardware utilization efficiency (η_{hw}). These are diagnostic emphases, not exclusive assignments, since fusion changes both launches and traffic while differentiation changes both work and live storage. Workloads with little reuse are frequently memory-bandwidth bound (D_{vol}/BW). Frameworks mitigate this memory wall through kernel fusion, which merges compatible element-wise operations to keep intermediates in registers or on-chip SRAM instead of round-tripping through global memory. The separate capacity limit yields to selective recomputation, which trades additional floating-point operations (O) for lower peak activation memory.

Key Takeaways: The layer between math and hardware

- **Every framework solves three problems**: Execution dictates runtime dispatch, differentiation computes gradients, and abstraction targets heterogeneous hardware. TensorFlow, PyTorch, and JAX make distinct infrastructure wagers across these axes, from static graph capture to dynamic tracing and functional transformations, and those wagers are architectural commitments that outlast API syntax.
- **The memory wall drives optimization**: Accelerator compute capacity has outpaced memory bandwidth, making off-chip data movement a recurring bottleneck. Kernel fusion and lower-precision layouts can reduce the memory traffic term (D_{vol}/BW), while selective recomputation primarily reduces peak activation storage by adding work.
- **Compilation must amortize setup cost**: The compilation continuum principle in equation 7.2 quantifies when runtime speedup offsets compilation overhead. While dynamic workloads favor eager execution, repeatedly executed graphs benefit from JIT or ahead-of-time compilation, where the dispatch overhead law (equation 7.4) demonstrates that small operations gain disproportionately when host launch overhead (L_{lat}) is reduced.
- **Module abstractions automate state management**: Automatic parameter registration, mode-dependent execution (training versus inference), and hierarchical state serialization enable multi-billion-parameter model updates in a single optimizer step across diverse hardware backends.
- **Deployment constraints should drive selection**: Specialized runtimes trade operator coverage, static memory allocation, and compilation latency against stringent hardware budgets. The gap between server-class accelerator memory and microcontroller SRAM requires evaluating the downstream deployment target before committing to a development framework.

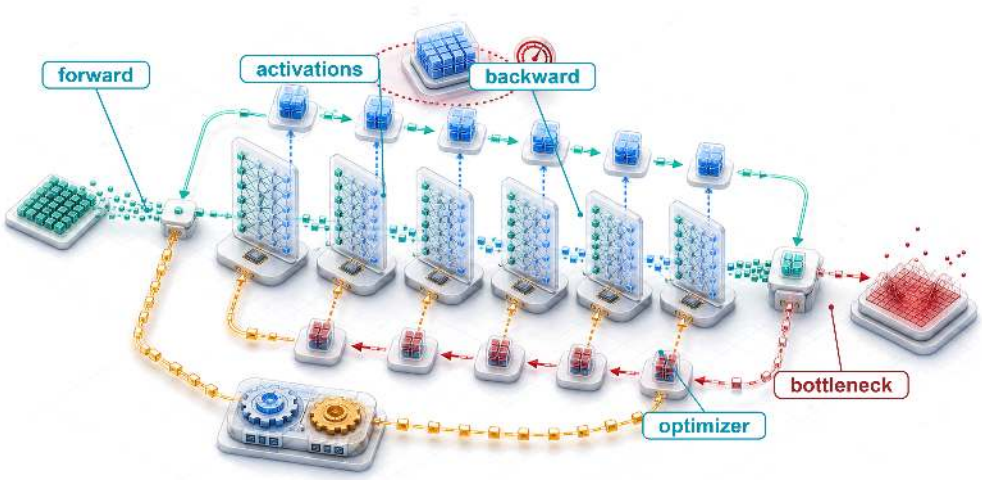
A framework presents itself as an ergonomic programming interface, yet its true role is translating mathematical graphs into physical hardware operations. That translation determines the work, traffic, and overhead paid under the iron law of ML systems (principle 3). Eager execution prioritizes developer iteration at the cost of host dispatch latency (L_{lat}), while graph compilation trades upfront optimization time to reduce avoidable global memory traffic (D_{vol}/BW) through fused kernels. Automatic differentiation adds backward work and traffic (D_{vol}), with checkpointing trading retained activation storage against recomputed arithmetic (O). What appears to be an API convenience is therefore a standing systems commitment: a framework respects physical limits while determining how much avoidable work and data movement the implementation performs.

What's Next: From control room to power plant

Frameworks are the software substrate that translates abstract architectures into executable kernels. Computational graphs, autograd tapes, and kernel dispatch pipelines are the control room instruments: they give engineers visibility into and control over the training process. This chapter traced a single training step through that machinery. A training run repeats that step many thousands or millions of times, and every budget decision the machinery exposes is still unmade, including which activations to checkpoint when memory runs short, how large a batch the hardware allows, and at what point a single device stops being enough. Chapter 8 is where those bills come due. It opens with why training costs orders of magnitude more than inference, then budgets memory, time, and cost for real training runs and shows how far one machine can go before scaling out.

8

Model Training

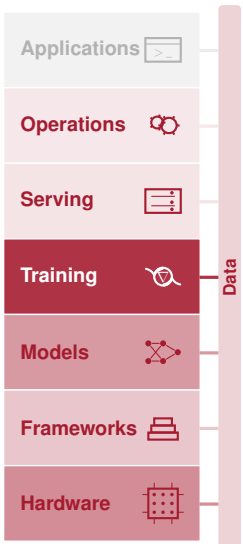


8.1 Training Systems Fundamentals
8.2 The Staged System Pipeline: Identifying "Accelerator Bubbles"
8.3 Training-Time Law
8.4 Training Computation and Memory Costs
8.5 Pipeline Architecture
8.6 Identifying Bottlenecks
8.7 Pipeline Optimizations
8.8 Scaling Training Systems
8.9 Fallacies and Pitfalls
8.10 Summary

Purpose

Why can training a model cost orders of magnitude more than one inference?

Inference computes a forward pass as data flows through the network and a prediction emerges. Training adds a backward pass and an optimizer step, retains intermediate state, and repeats that work across many examples and optimization steps. Optimizers may also maintain momentum and variance estimates whose storage exceeds the model weights, so memory rather than arithmetic often decides what fits on a device. The ratio depends on the model, workload, training duration, and inference volume, but one training run commonly costs orders of magnitude more than one inference. A lab that trains in three days can explore many more designs than one that takes a month. Over a year, that difference is more than a hundred iterations against about a dozen, and each result informs the next design while the slower lab waits weeks for every answer. A faster step is not faster training if it delays convergence, so throughput matters only relative to time to target quality. Across repeated experiments, even modest changes in convergence or utilization compound into substantial schedule and cost differences. The central systems challenge is to keep accelerators supplied with data and useful arithmetic without exhausting memory or losing time to synchronization. Which intervention helps depends on the binding compute, memory, data, or communication constraint, so profiling must precede scaling or optimization. Batch size, numerical precision, checkpointing, data loading, and parallelism must therefore be treated as coupled decisions rather than independent tuning knobs. For the systems engineer, training is where hardware and parallelism determine whether a run fits within memory, schedule, and budget, and whether the repeated optimization loop accelerates or stalls. In D·A·M terms, each precision and parallelism decision negotiates between the mathematics of optimization and the physics of execution.



Learning Objectives

- Explain training cost asymmetry using forward, backward, optimizer-state, data, and iteration costs
- Calculate FLOPs, activation memory, optimizer state, and dollar cost for neural network training
- Compare SGD, Adam, and AdamW by convergence behavior, memory overhead, and compute cost
- Diagnose compute-, memory-, and data-bound training bottlenecks with roofline analysis and profiling evidence
- Apply mixed precision, checkpointing, gradient accumulation, and FlashAttention to fit accelerator memory and throughput limits
- Design single-machine training pipelines with prefetching, overlap, batching, and systematic re-profiling
- Evaluate single-machine multi-accelerator configurations using memory, duration, communication, energy, and cost constraints, and recognize when a workload exceeds the node boundary

8.1 Training Systems Fundamentals

The anatomy of a training step in section 7.10 traced one parameter update through dispatch, automatic differentiation, and accelerator execution. Those framework mechanisms make an isolated step possible. A training system makes that step repeatable across millions of iterations while retaining activations, streaming training data, and staying within rigid memory and budget envelopes. Repetition converts abstract architectural choices into physical constraints: which intermediate activations to preserve when high-bandwidth memory (HBM) is exhausted, how large a batch the memory hierarchy permits, and when a workload outgrows a single accelerator. Running a model once and training it from scratch sit on opposite ends of the operational spectrum, and the magnitude of that divergence defines the discipline of training systems.

At the boundary between inference and pretraining lies an enormous divergence in resources. In 2019, training a GPT-2-scale model required approximately $50,000 in compute resources, whereas pretraining a frontier architecture such as GPT-4 reached an estimated $100M.[1] Although neither figure represents an audited financial disclosure, the four-order-of-magnitude gulf between them highlights the economic reality of deep learning. This **training cost asymmetry** reflects the fundamental divergence between evaluating an existing artifact for a single query and repeatedly executing the forward-backward-update loop over an entire corpus.

For GPT-2, a single forward token evaluation consumes roughly 3.00×10^9 floating-point operations, derived from standard transformer architectural accounting (Radford et al. 2019). Pretraining the full model across its WebText corpus required a cumulative budget of 1.50×10^{21} FLOPs—equivalent to approximately 167 billion token evaluations when forward and backward passes are evaluated together. This vast operational volume transforms training systems engineering into a distinct discipline, where the ability to coordinate compute, memory, and communication determines whether an organization can realistically participate in frontier AI development.

[1] **Training cost scaling**: The roughly 2,000× ratio between this chapter's two cost anchors illustrates how parameter counts, training-token budgets, hardware generations, and accelerator fleets can compound; it is not a controlled comparison of disclosed training bills. OpenAI's GPT-2 report documents the model and training setup but does not disclose training cost (Radford et al. 2019). Exact GPT-4 training details were also not disclosed, so the GPT-4-class figure is explicitly an industry estimate supported by public reporting and independent infrastructure analysis (Knight 2023; Patel and Wong 2023).

Definition 8.1: Training systems

Machine learning training systems are software-hardware systems that execute the iterative optimization loop (forward pass, loss computation, backward pass, and parameter update) to minimize a loss function over a training dataset.

1. Significance: Training memory cost is 8× the inference memory cost per parameter in a standard mixed-precision Adaptive Moment Estimation (Adam) setup: a 7B-parameter model requires 14 GB (FP16 weights) + 14 GB (FP16 gradients) + 28 GB (FP32 master weights) + 56 GB (Adam first and second moments in FP32) = 112 GB at least, before

accounting for activation storage. This multiplier is a major reason a model that runs inference on one GPU requires multiple GPUs for training.

2. Distinction: Unlike inference systems, which execute a single forward pass and discard intermediate activations, standard backpropagation stores the tensors needed by the backward pass, creating a footprint that grows with model depth and batch size.
3. Common pitfall: A frequent misconception is that training failures are compute problems. A common failure is an out-of-memory (OOM) error, a memory-management problem: tensors saved for backpropagation accumulate during the forward pass and can exhaust memory before the first gradient is computed.

Three characteristics distinguish training workloads from general-purpose computing:

- **Computational intensity**: The 1.50×10^{21} FLOPs budget spread over days of wall-clock time demands sustained PFLOP/s-scale throughput from hardware whose realized large-model throughput is often far below theoretical peak (Narayanan et al. 2021; Chowdhery et al. 2022).
- **Memory pressure**: Storing 1.5B weights requires 6 GB in FP32; the Adam optimizer adds two state tensors per parameter, consuming another 12 GB. Activation memory adds a workload-dependent footprint governed by architecture, sequence length, batch size, precision, and checkpointing, and the combined state can exceed a single accelerator's capacity.
- **Data dependencies**: Each gradient update depends on the result of the previous one, creating sequential bottlenecks that limit how much parallelism the system can exploit.

Each constraint demands a targeted systems intervention. Computational intensity pushes the system toward higher arithmetic utilization and lower-precision numeric formats. Memory pressure calls for gradient checkpointing,[2] a rematerialization strategy that trades redundant arithmetic for reduced activation storage, and mixed-precision training,[3] which halves the storage required for activations by casting them to 16-bit formats while selectively retaining FP32 master weights and optimizer moments for numerical stability. Data dependencies motivate pipeline architectures that overlap I/O and communication with computation, ensuring that execution units never stall waiting for tensors. The analysis here establishes the execution baseline on single-machine and single-node multi-GPU topologies; scaling across distributed network boundaries introduces multi-node communication fabrics and fault-tolerance protocols treated in advanced distributed systems texts.

8.2 The Staged System Pipeline: Identifying "Accelerator Bubbles"

A training system is fundamentally a **staged system pipeline** rather than a monolithic loop. Reaching high accelerator utilization requires treating training as a balanced assembly line where four distinct stages coordinate to keep arithmetic execution units saturated:

1. **Data Loading & Preprocessing** (CPU/Storage): Fetching raw bytes from Non-Volatile Memory Express (NVMe) storage, decoding samples, and executing transformations on CPU cores.
2. **Host-to-Device Transfer** (PCIe): Moving the preprocessed batch across the PCIe bus into GPU memory via direct memory access (DMA).
3. **Forward/Backward Pass** (Accelerator): Propagating activations forward and gradients backward through layer kernels on the GPU.
4. **Parameter Synchronization** (Interconnect, present only when multiple accelerators train the model): Exchanging gradients between accelerators across the available interconnect fabric. NVLink provides up to 900 GB/s in the H100 configuration examined later, while alternative architectures rely on PCIe switches or scale-out network fabrics.

Any throughput mismatch among these stages creates **accelerator bubbles**—idle windows during which compute units stall while waiting for upstream dependencies. These bubbles stem from imbalances within the per-step pipeline. A systems engineer eliminates them through asynchronous prefetching and pipeline overlapping, staging the next batch into high-bandwidth memory before the active step completes. The first three stages form the analytical basis for the pipeline execution model

[2] **Gradient checkpointing (activation checkpointing)**: Backpropagation retains the forward values required by each backward rule. Checkpointing saves selected values and recomputes others during backward, trading additional work for lower activation memory. The $\sqrt{N_L}$ checkpoint placement and its associated recomputation cost describe the classical schedule analyzed by T. Chen et al. (2016) rather than every checkpointing implementation. Section 8.7.5.2 works through that schedule for GPT-2's layer count.

[3] **Mixed-precision training**: Uses lower precision for selected computation and storage while retaining higher precision where accumulation or numerical range requires it. A common FP16 recipe maintains FP32 master weights and uses loss scaling to reduce gradient underflow (Micikevicius et al. 2017). BF16 ("Brain Floating Point," from Google Brain (Wang and Kanwar 2019)) matches FP32's 8-bit exponent range and therefore usually avoids dynamic loss scaling, though the exact state and accumulation precision depend on the optimizer and implementation.

↩ The feeding problem, in which data delivery must keep pace with accelerator consumption, is established in section 4.2.3.

in equation 8.8, where the slowest stage dictates end-to-end iteration throughput, and section 8.6 provides the profiling methods required to isolate that bottleneck.

Computational intensity, memory pressure, and data dependencies compound in large-scale execution. Because every update depends on the one before it, a single unmitigated numerical explosion does not cost one step; it can invalidate every step taken since the last good checkpoint, which at frontier scale means thousands of accelerator-hours.

War Story 8.1: The PaLM loss spikes (2022)

Context: PaLM's 540-billion-parameter run used 6,144 Tensor Processing Unit (TPU) v4 chips across two 3,072-chip Pods connected through Google's data-center network and suffered severe training-instability spikes absent from smaller runs (Chowdhery et al. 2022).

Failure mode: The training loss spiked roughly 20 times despite gradient clipping. The spikes occurred at highly irregular intervals, sometimes late in training, never appeared in smaller models, and had no identified principled mitigation.

Impact: Each recovery discarded recent progress on an expensive 6,144-chip run, so reproducible checkpoints and repeatable data order were prerequisites for replay.

Response: Engineers restarted training from an uncorrupted checkpoint roughly 100 steps prior to each spike and skipped approximately 200–500 data batches around the failure window.

Systems lesson: Every step discarded in a recovery is paid for again in cluster time, so operational fault tolerance decides how much of the hardware's time turns into lasting progress. Automated loss-spike detection, checkpoint cadence, and batch skipping are core system mechanisms for keeping that lost fraction small.

8.3 Training-Time Law

PaLM's recoveries kept 6,144 chips busy on steps that were later thrown away, so the cluster's time was spent without the run advancing. Pricing that loss, and every other one, requires a decomposition of training time into its physical constituents, in which wasted time appears as a lower utilization factor.

The iron law of ML systems (principle 3) supplies that decomposition once it is specialized to training. The training-time form of the iron law, which this chapter shortens to the training-time law, keeps the compute term as the organizing equation so that every optimization technique maps to one of its factors.

Definition 8.2: The training-time law

The training-time law expresses training time through the workload's operation count and its effective achieved throughput:

$$T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}} \tag{8.1}$$

Here O counts useful model forward-and-backward FLOPs, excluding rematerialization. Additional execution work is reflected in effective throughput. The factor η_{hw} is an effective end-to-end utilization factor, the achieved model-operation rate divided by the relevant hardware peak. Data stalls, communication, synchronization, launch overhead, and inefficient kernels all reduce this factor. Prefetching and overlap can hide part of those costs but do not guarantee their removal.

1. Significance: In the D·A·M taxonomy, these terms map to distinct optimization levers: O is governed by Data (training token count) and Algorithm (model architecture or compression methods such as pruning and distillation), R_{peak} is governed by the Machine (hardware execution units at the chosen numerical precision), and η_{hw} is the effective systems utilization factor and primary engineering target (Narayanan et al. (2021) measured 44 to 52 percent of theoretical peak throughput across GPT models trained on A100

clusters). Chapter 10 defines the compression techniques; here they serve only to show where such methods enter the accounting.

2. Distinction: The general iron law exposes data movement, computation, and latency separately. This training form folds every source of lost throughput into η_{hw}, making it useful for budget accounting but insufficient for diagnosing which underlying term is responsible.
3. Common pitfall: A frequent misconception is that η_{hw} is fixed by hardware. System efficiency is a pipeline property: memory bandwidth saturation, kernel launch overhead, and synchronization barriers each reduce η_{hw} independently, and diagnosing which factor dominates requires profiling rather than reading hardware specs.

Equation 8.1 reveals three levers for improvement: reduce total operations through algorithmic innovation, increase peak throughput through hardware selection and precision formats, or improve utilization through pipeline orchestration. Memory does not appear in the law at all. It acts as a feasibility constraint, because a configuration whose state does not fit runs at no speed. Table 8.1 previews the techniques this chapter develops, each placed against the factor of the law it moves, with its memory effect listed separately.

Table 8.1: Iron law optimization mapping: A preview of the chapter's techniques, each placed against the factor of equation 8.1 it moves; the parenthetical gives its effect on memory, which decides feasibility rather than appearing in the law. Later sections develop each technique.

Technique	Law Term (Memory)	Mechanism
Mixed Precision (FP16/BF16)	$R_{\text{peak}} \uparrow$ (activations $\downarrow$)	Uses supported lower-precision hardware paths
Data Prefetching	$\eta_{\text{hw}} \uparrow$ (buffers $\uparrow$)	Reduces accelerator idle time waiting for data
Gradient Checkpointing	η_{hw} changes (activations $\downarrow$)	Added recomputation can lower effective utilization; released memory may permit a more efficient batch
Gradient Accumulation	η_{hw} changes (activations $\downarrow$)	Builds an effective batch from micro-batches with one sync per update
Kernel Fusion	$\eta_{\text{hw}} \uparrow$	Avoids intermediate transfers and reduces kernel launches
FlashAttention	$\eta_{\text{hw}} \uparrow$ (attention scores $\downarrow$)	Same asymptotic FLOPs, much lower HBM IO; backward recomputation may add FLOPs

The training-time law focuses on execution efficiency—how fast the hardware processes a given workload. It does not predict how data quality, dataset size, or curriculum design determines the total operations O needed to reach a target accuracy; that workload must be supplied or estimated separately. A cleaner dataset or a better data mix can reduce the number of epochs required, shrinking O without touching hardware at all. Holding the workload fixed, the question is how to execute it as fast as possible.

Actual training throughput remains below the arithmetic peak whenever kernels, data delivery, or communication leave resources idle. Scaling to multiple accelerators introduces additional communication overhead, a trade-off quantified in section 8.8.

↪ Data selection, which shrinks total operations (O) by keeping the examples that return the most learning per unit of compute, is developed in section 9.1.

Checkpoint 8.1: The physics of training

Training speed is governed by the utilization of hardware peaks.

Utilization gap

- ☐ **Peak vs. real**: why does sustained end-to-end utilization remain below the arithmetic peak?
- ☐ **Utilization losses**: which pipeline stalls introduced earlier reduce η_{hw}, and which can prefetching or overlap hide?
- ☐ **Order of magnitude**: a frontier run needs on the order of 10^{23} FLOPs at a utilization η_{hw} well below 1. Estimate which lever shortens wall-clock time more, doubling R_{peak}

or doubling η_{hw}, and why, in practice, the two levers differ in cost and reachability even though the training-time law treats them symmetrically.

Each milestone in the history of deep learning moved the binding constraint from one factor of this law to another. Backpropagation, popularized for multilayer networks in 1986 (Rumelhart et al. 1986), made the operation count O of a deep network computable at all. AlexNet's five-to-six-day ImageNet run on two GPUs in 2012 (Krizhevsky et al. 2012) showed that this arithmetic mapped onto the far higher R_{peak} of graphics hardware. Transformers from 2017 onward (Vaswani et al. 2017) concentrated the work into large dense matrix multiplications with large activation footprints. GPT-3's roughly 3.14×10^{23} FLOPs in 2020 (Brown et al. 2020) made utilization (η_{hw}) the difference between an affordable run and an unaffordable one, and IO-aware kernels such as FlashAttention later raised η_{hw} by attacking attention's memory traffic.

That history is the bitter lesson seen from inside the training system. None of these milestones succeeded by encoding more human expertise. Backpropagation supplied a general recipe, and each later step let that recipe absorb more computation, more parameters, and more data. The systems consequence is what makes training engineering matter. If capability tracks the computation a general method can absorb, then every point of η_{hw} lost to stalls, and every gigabyte of memory that forces a smaller model or batch, is capability forgone at a fixed budget. The first obligation of a training system is therefore to convert its compute budget into useful operations as completely as the hardware allows.

↩ The bitter lesson, that general methods which absorb more computation outperform encoded human expertise, is established in section 1.4.

8.3.1 Running example: Training GPT-2

In inference, GPT-2 serves as the Bandwidth Hog lighthouse, because generating one token at a time reads every weight for little arithmetic. Training changes that profile. A batch of long sequences reuses each weight across thousands of token positions, so the same model now presses on arithmetic throughput and, through activations and optimizer state, on memory capacity. Here it recurs as the chapter's stable workload for testing each training cost and optimization.

↩ GPT-2's inference archetype as the Bandwidth Hog, a workload bound by data movement rather than arithmetic, is established in section 2.3.2.



Lighthouse 8.1: Lighthouse example: Training GPT-2

Context: GPT-2 (1.5 billion parameters) serves as the primary case study because it is large enough to expose meaningful compute, activation, and optimizer-state constraints while remaining easier to analyze than frontier-scale models. Whether it requires distributed training depends on precision, optimizer state, activation memory, hardware capacity, and the target training time. Table 8.2 maps each model property to the systems constraint it creates:

Table 8.2: GPT-2 (1.5 Billion Parameters) Lighthouse Model Specifications: Parameter count, architecture depth, dataset size, and total training compute mapped to their systems implications. Each row pairs a model property with the engineering constraint it creates—memory footprint for weights, activation pressure from pipeline depth, I/O throughput requirements, and parallelization demand.

Property	Specification	Systems Implication
Parameters	1.5B (XL)	Requires ~3 GB (FP16) or ~6 GB (FP32) for weights alone.
Architecture	48 Layers, 1600 Dim	Deep pipeline creates heavy activation memory pressure.
Dataset	WebText (40 GB)	Preprocessing, shuffling, and delivery must sustain the selected training configuration.
Compute	~ 1.50 × 10^{21} FLOPs	Training duration depends on achieved throughput and accelerator count.

Mechanism: In dense transformer models, the arithmetic work per token is directly proportional to parameter count P. In the forward pass, each parameter participates in one multiply-accumulate operation (1 MAC $=$ 2 FLOPs), consuming $2P$ FLOPs/token. Backpropagation computes two matrix products per parameter: one to propagate gradients backward with respect to activations ($2P$ FLOPs), and another to calculate weight gradients for the optimizer ($2P$ FLOPs). The backward pass therefore requires twice the compute of the forward pass ($4P$ FLOPs/token), yielding a total work factor of $2P + 4P = 6P$ FLOPs/token. Across a dataset of D tokens, total training work is $O \approx 6 \cdot P \cdot D$, or 1.50 × 10^{21} FLOPs.

Training GPT-2 couples this operation count with the four-term training-memory budget of equation 5.28, $M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$. Balancing execution time $T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}}$ requires orchestrating data ingestion, activation memory, and compute throughput within accelerator HBM capacity.

The lighthouse accounting also prices runs far larger than GPT-2. Dividing $6PD$ by the achieved throughput $R_{\text{peak}} \times \eta_{\text{hw}}$ gives accelerator time, and multiplying accelerator time by a price gives money. At frontier scale, that conversion makes compute expenditure a binding economic constraint on hardware procurement and cluster sizing, as notebook 8.1 shows for a 70-billion-parameter language model trained across a 1,024-accelerator cluster.

Napkin Math 8.1: The utility bill

Problem: Under the following simplified assumptions, is it cheaper to rent or purchase a 1,024-H100 cluster for one Llama 2 70-billion-parameter training run?

Math:

1. Workload: Llama 2 70B model (70B parameters, 2T tokens).
2. **Compute required**: $6 \times 70 \times 10^9 \times 2 \times 10^{12} \approx 8.4 \times 10^{23}$ FLOPs. The leading factor of 6 follows from the chapter's accounting: about 2 FLOPs per parameter per token in the forward pass (two FLOPs per multiply-accumulate), tripled once the backward pass, which costs roughly twice the forward pass, is included.
3. Hardware: NVIDIA H100 (Peak: 989 TFLOP/s FP16). Assumed Utilization: 50 percent (494.5 TFLOP/s).
4. Time: $8.4 \times 10^{23} / (494.5 \times 10^{12}) \approx 1.70 \times 10^9$ seconds ≈ 53.8 years (on one GPU).
5. **Cluster**: On 1,024 GPUs $\rightarrow$ 19.2 days.

The economics:

- **Rental ($3/hr)**: 1,024 GPUs $\times$ 24 hrs $\times$ 19.2 days $\times$ $3/hr ≈ $1.42M.
- **Purchase ($30,000 per GPU)**: 1,024 GPUs $\times$ $30,000 = $30.7M.

Systems insight: In this simplified utilization-and-price scenario, purchase cost equals about 21.7 rental runs. A real ownership comparison must also include cluster networking, host

systems, facilities, power, staffing, depreciation, financing, and the utilization achieved between runs.

Not all training workloads are compute bound. Recommendation models like DLRM are dominated by massive embedding tables (100B to 10T parameters, mostly embeddings) that make them memory bandwidth bound rather than compute bound. For such workloads, the first scaling problem is often capacity: splitting the embedding tables across devices so the model fits at all. In contrast, dense, compute-intensive training—exemplified by the GPT-2 lighthouse model analyzed below—exposes compute-bound and activation-memory limits rather than capacity partitioning.

Within the end-to-end ML lifecycle, training systems ingest prepared data from upstream pipelines and produce model checkpoints for downstream deployment. Data quality directly governs training stability, while training throughput dictates iteration velocity. This systems pressure manifests across three distinct scales. At data scale, large corpora require high-throughput I/O pipelines and distributed storage to avoid accelerator starvation. At model scale, billion-parameter architectures force the system to balance memory capacity against communication, choosing between replicating weights across data-parallel devices[4] or partitioning layers across devices with model parallelism.[5] At infrastructure scale, coordinating multiple accelerators introduces synchronization overhead that can dominate training time. These physical trade-offs make the stage interface contracts formalized in section 3.6.1 essential during training: orchestration decisions dictate both iteration velocity and hardware efficiency.

GPT-2's 40 GB WebText corpus sits at the lower end of this data scale spectrum. Larger corpora change the data path qualitatively.

> **Systems Perspective 8.1: The 10 GB to 10 TB scale factor**
>
> In a memory-resident regime, a corpus may fit in system RAM and avoid repeated storage reads after caching, though preprocessing and transfer costs remain. In a streaming regime, the corpus exceeds local memory and the system must orchestrate storage reads, preprocessing, caching, and delivery throughout training. The appropriate design may include multiple workers, prefetching, local caches, or distributed storage, depending on which stage limits throughput.
>
> Scale therefore changes more than the amount of data; it transforms the system's physics.

These scaling dynamics translate into concrete systems constraints across the training loop. Modern accelerators feature compute-to-bandwidth ratios (roofline ridge points) that far exceed the arithmetic intensity of many neural network layers, leaving memory-bound kernels starved for operands (Hennessy and Patterson 2017). When multi-accelerator configurations are introduced, cross-device synchronization imposes latency and bandwidth penalties where interconnect fabrics dictate the communication-to-computation ratio.

Software frameworks expose these physical boundaries as configurable switches—such as mixed-precision autocasting contexts or gradient checkpointing flags. In systems engineering, however, these switches cannot be tuned blindly; they must be budgeted against the concrete physical constraints of the machine. Every bottleneck ultimately traces back to the execution cost of specific mathematical operations: compute-intensive matrix multiplications governed by peak throughput (Goto and Geijn 2008), low-intensity normalization layers governed by memory bandwidth, and optimizer state tensors inflating the static memory footprint. Optimizing a training system therefore begins with a rigorous accounting of each cost category.

8.4 Training Computation and Memory Costs

Linear algebra expresses a layer's matrix multiplication compactly as $\mathbf{Z} = \mathbf{AW}$. In hardware, executing this operation efficiently requires navigating the accelerator's physical memory hierarchy:

- **Global device memory (HBM or GDDR)**: Located off-chip, offering large storage capacity (typically 16 to 96 GB) to hold weights, optimizer states, and large activation tensors, but communicating across an external memory bus bounded by ≈ 1.5–3.5 TB/s of bandwidth.

[4] **Data parallelism**: Replicates the full model on every device, splitting only the data. Each device computes gradients independently, then an AllReduce operation (a collective that sums the gradients across devices and returns the sum to every device) synchronizes them—adding communication volume proportional to model size at every step. This synchronization tax limits scaling efficiency: doubling accelerators rarely halves training time once communication becomes the bottleneck.

[5] **Model parallelism**: Partitions the model's layers across devices when it exceeds single-device memory. Activations must transfer between devices at every partition boundary, and naive partitioning creates "pipeline bubbles" where downstream devices idle while waiting. Microbatch pipelining, as in GPipe and PipeDream, recovers much of this lost efficiency (Huang et al. 2019; Narayanan et al. 2019).

- **On-chip SRAM (registers and shared memory)**: Located directly on the processor die adjacent to the arithmetic execution units. It delivers ultra-high aggregate bandwidth ($\approx$ 15–20 TB/s), but has strictly limited capacity (typically 30 to 50 MB across the entire chip).

In a model such as GPT-2, the tensors participating in training far exceed the capacity of on-chip SRAM, requiring matrix tiles to be continuously staged from off-chip HBM into on-chip memory for computation. Dense matrix multiplications exhibit high arithmetic intensity, amortizing the cost of moving bytes over many operations and enabling specialized execution units such as Tensor Cores to achieve high compute utilization. In contrast, surrounding elementwise operations—such as activations, normalizations, and residual additions—exhibit low arithmetic intensity and remain bounded by memory bandwidth, creating opportunities for kernel fusion to eliminate redundant round-trips to HBM. Chapter 5 established *what* neural network operations compute and *why* they enable learning; the systems question is *what* they cost in FLOPs, memory, and bandwidth when those operations execute at scale.

Four dimensions structure this cost analysis:

- FLOP counts of the matrix operations that dominate dense neural-network training.
- Memory requirements for storing activations and optimizer states simultaneously.
- Bandwidth demands that determine whether operations are compute-bound or memory-bound.
- Arithmetic intensity classifications that guide optimization strategy selection.

Together, these dimensions provide the vocabulary for analyzing the computational intensity, memory pressure, and data dependencies introduced in section 8.1.

8.4.1 Neural network computation

Neural network training consists of repeated matrix operations and nonlinear transformations. These operations are conceptually simple but create the system-level challenges that dominate modern training infrastructure. The introduction of backpropagation[6] by Rumelhart et al. (1986) and the development of efficient matrix computation libraries such as Basic Linear Algebra Subprograms (BLAS)[7] (Dongarra et al. 1988) laid the groundwork for modern training architectures.

8.4.1.1 Mathematical operations in neural networks

Forward propagation, in its simplest case, involves two operations: matrix multiplication and activation function application. Matrix multiplication implements the linear transformation at each layer. At layer ℓ, the computation can be written in row-vector form, with one example per row:

$$\mathbf{A}^{(\ell)} = f\left(\mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}\right)$$

where:

- $\mathbf{A}^{(\ell-1)}$ represents the activations from the previous layer (or the input layer for the first layer), with each row being a sample in the batch,
- $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ is the weight matrix at layer ℓ, which contains the parameters learned by the network,
- $\mathbf{b}^{(\ell)}$ is the bias vector for layer ℓ,
- $f(\cdot)$ is the activation function applied elementwise (for example, ReLU, sigmoid) to introduce nonlinearity.

8.4.1.2 Matrix operations

Section 5.5.2.2 established that forward propagation reduces to chains of matrix multiplications. Every architecture also shares a small set of building blocks, from dense projections and residual paths to normalization, each with its own cost to the machine. Training amplifies these patterns: each operation executes not once but billions of times, and each forward pass is paired with a backward pass that costs roughly twice the forward pass, tripling the work per step. Understanding which matrix operations dominate—and how their shapes change between forward and backward passes—reveals *why* specific system designs and optimizations emerged for training.

[6] **Backpropagation provenance**: The algorithm was independently derived by Linnainmaa (1970) for automatic differentiation of computer programs and by Werbos (1974) in a Harvard PhD thesis on economic modeling—over a decade before Rumelhart et al. (1986) popularized it for neural networks. This delay between derivation and broad adoption recurs in ML systems history: attention mechanisms predated transformers by decades, but the 2017 Transformer showed that attention-heavy models could train efficiently on contemporary GPU systems; later TPU and GPU clusters enabled much larger deployments. A framework's backward pass traverses the computational graph in reverse topological order and applies the chain rule; independent subgraphs may expose parallel work during that traversal.

[7] **BLAS (basic linear algebra subprograms)**: The original BLAS specification standardized reusable Fortran-callable vector operations (Lawson et al. 1979); later BLAS extensions added matrix-vector and matrix-matrix routines, giving the familiar Level 1, Level 2, and Level 3 hierarchy (Dongarra et al. 1988). Training is dominated by Level 3 operations precisely because their high arithmetic intensity—$\mathcal{O}(N)$ FLOP/byte—can sustain high compute utilization. Frameworks commonly dispatch dense matrix multiplication to optimized libraries such as cuBLAS and oneDNN or to compiler-generated kernels.

↩ The building blocks every architecture shares, and what each costs the machine, are established in section 6.10.2.

The predominance of matrix multiplication has shaped both linear algebra software and accelerator architecture. Early implementations relied on general-purpose CPU libraries, but modern training scale requires tiled, accelerator-native libraries such as cuBLAS (NVIDIA 2024a) and oneDNN that partition operands to fit cache hierarchies and feed systolic arrays.

This dense computational workload motivates blocked algorithms that parallelize across thousands of execution cores while aligning with the memory access patterns of forward and backward passes. Because backpropagation requires simultaneous access to weight matrices, incoming gradients, and stored activations, hardware architects have prioritized high memory bandwidth alongside raw arithmetic throughput. Evaluating one layer of the GPT-2 lighthouse model illustrates the scale and distribution of these operations.

Napkin Math 8.2: GPT-2 attention layer computation

Each GPT-2 layer performs attention computations that exemplify dense matrix multiplication demands. For one GPT-2 transformer layer (all heads combined) with batch size $B = 32$, sequence length $S = 1024$, and hidden dimension $d = 1600$:

Query, Key, Value Projections (the three linear transformations that create attention inputs—3 separate matrix multiplications):

$$O_{\text{QKV}} = 2 \times 3 \times (B \times S \times d \times d)$$
$$= 2 \times 3 \times (32 \times 1024 \times 1600 \times 1600) \approx 503 \text{ billion FLOPs}$$

Attention matmuls ($\mathbf{QK}^T$ and the product of the attention probabilities with $\mathbf{V}$):

$$O_{\text{QK}} = 2 \times B \times N_{\text{heads}} \times S \times S \times d_{\text{head}} = 107.4 \text{ billion FLOPs}$$
$$O_{\text{AV}} = O_{\text{QK}}; \quad \text{pair total} \approx 214.8 \text{ billion FLOPs}$$

Output projection (attention output $\rightarrow$ hidden):

$$O_{\text{out}} = 2 \times B \times S \times d \times d \approx 168 \text{ billion FLOPs}$$

Feed-Forward Network (Two linear transformations with expansion factor 4):

$$O_{\text{FFN}} \approx 16 \times B \times S \times d^2$$

Computation Scale

- Per-layer forward total (QKV 503 + attention 214.8 + output proj 168 + FFN): ~2228.01 GFLOP
- With 48 layers in GPT-2: ~320.8 TFLOP per training step
- Across this illustrative 50,000 steps scenario: ~16041.7 PFLOP

Systems insight: A V100 GPU (125 TFLOP/s peak with Tensor Cores, 15.7 TFLOP/s without) would require 2.6 s for the modeled attention-plus-FFN training-step estimate at 100 percent utilization (theoretical peak; practical throughput would be lower). If the target is a step of roughly 200 ms, the problem is already a multi-accelerator one: ideal arithmetic alone requires roughly 12 to 15 V100s, and practical systems need additional headroom for utilization losses, communication, and pipeline overhead.

These FLOP counts are not academic bookkeeping. They are the compute term of the iron law made concrete, and they explain why training cost scales as a predictable function of model architecture and sequence length rather than as an unpredictable emergent property.

8.4.1.3 Matrix-vector and batched operations

Not all neural network layers execute as dense matrix-matrix multiplications. Normalization layers, bias additions, and autoregressive recurrent updates execute matrix-vector operations. Although algebraically simpler, matrix-vector products present an immediate systems challenge: they under-

utilize massively parallel hardware. A single vector provides too little arithmetic work to saturate thousands of accelerator cores, dropping arithmetic intensity to $\mathcal{O}(1)$ FLOPs per byte transferred from device memory.

Batching resolves this underutilization by grouping independent training samples into contiguous matrices. Processing B inputs simultaneously transforms matrix-vector kernels (GEMV) into dense general matrix multiplication (GEMM) operations. This conversion elevates arithmetic intensity and allows systolic execution units to sustain high compute utilization. The systems trade-off is an immediate expansion of working memory: every additional sample in the batch scales the intermediate activation volume that must reside in device memory during backpropagation. Hardware design directly mirrors this progression. Google's first TPU integrated dedicated two-dimensional systolic matrix units that required batched execution to achieve efficiency (Jouppi et al. 2017), while large-scale models such as GPT-3 use distributed clusters specifically to execute the massive GEMM patterns exposed by large global batches (Brown et al. 2020).

> **Systems Perspective 8.2: Why GPUs dominate training**
>
> The matrix operations described in section 8.4.1.2 directly explain data-center training hardware architecture. GPUs became central to large-scale training for three reasons:
>
> - Matrix multiplication's independent element calculations map well to thousands of GPU cores (NVIDIA A100 has 6,912 CUDA cores).
> - Specialized hardware units like Tensor Cores provide substantially higher peak throughput for supported lower-precision matrix operations.
> - Blocked matrix computation patterns enable efficient use of GPU memory hierarchy (L1/L2 cache, shared memory, global memory).
>
> When a later illustrative GPT-2 scenario assumes a 2.4× V100 mixed-precision speedup, Tensor Core acceleration of eligible matrix multiplications is one source of the gain. The realized end-to-end ratio also depends on nonmatrix operations, memory traffic, and input and communication overhead.

Matrix multiplications dominate training compute, but neural networks require more than linear transformations. Between each layer's matrix operations, activation functions introduce the nonlinearity that enables networks to learn complex patterns. These functions appear computationally trivial compared to matrix multiplication, yet their implementation characteristics affect training efficiency in ways that matter at scale.

8.4.1.4 Activation functions

Activation functions like sigmoid, tanh, ReLU, and softmax introduce nonlinearity, but their implementation characteristics also shape training system performance. From a systems perspective, the choice of activation function determines computational cost, hardware utilization, and memory access patterns during backpropagation.

The critical question for ML systems engineers is not *what* these functions do mathematically, but *how* their execution cost behaves in hardware. Per-element arithmetic complexity varies across activations: piecewise linear functions like ReLU require only a single comparison and selection instruction (`fmax`), whereas transcendental functions like sigmoid and tanh require evaluating exponentiations, typically implemented via multi-cycle polynomial approximations in Special Function Units (SFUs). Softmax introduces additional reduction and normalization passes across the tensor dimension.

These per-element differences are real, but in end-to-end accelerator execution they rarely dictate throughput. For a bandwidth-bound activation kernel, the cost that matters is set by bytes, not instructions. Simple elementwise activations can perform only 1 to 4 FLOPs per element while moving 8 bytes (an FP32 read and write), roughly 0.125–0.5 FLOP/byte, so their low arithmetic intensity can make standalone kernels memory-bandwidth limited when launch overhead does not dominate. In modern compiled frameworks, the realized latency of an activation is largely

determined by whether the compiler can fuse it directly into the epilogue of the preceding matrix multiplication, avoiding a separate intermediate write and read between kernels.

Modern accelerators can implement these functions through native instructions, polynomial approximations, lookup methods, or fused kernels. ReLU requires a simple threshold operation, while sigmoid, tanh, and exact Gaussian Error Linear Unit (GELU) formulations require more arithmetic or special-function support. The realized latency difference still depends on whether arithmetic, memory traffic, or launch overhead is binding. ReLU may produce zeros that a later system can exploit, but the sparsity fraction depends on the activation distribution and trained model. Softmax[8] adds a reduction and normalization across the selected axis. Those dependencies require coordination, though optimized implementations remain highly parallel.

Table 8.3 summarizes how these behaviors translate into system constraints.

[8] **Softmax**: Computing a numerically stable softmax requires a maximum reduction, exponentiation, a sum reduction, and normalization across the selected axis. Parallel kernels implement these stages with coordinated reductions. FlashAttention restructures the larger attention computation so softmax statistics can be maintained while tiles remain near the compute units.

Table 8.3: Activation Function Systems Comparison: Activation functions differ in mathematical behavior, kernel cost, fusion opportunities, and memory access patterns.

Function	Key Advantages	Key Disadvantages	System Implications
Sigmoid	Smooth; bounded output in $(0, 1)$.	Gradients saturate at large magnitudes; output is not zero-centered.	Uses exponential or approximation kernels whose cost depends on the target.
Tanh	Smooth; zero-centered output in $(-1, 1)$.	Gradients saturate at large magnitudes.	Uses special-function or approximation kernels similar in structure to sigmoid.
ReLU	Simple threshold operation; nonsaturating for positive inputs.	Can produce persistently inactive units.	Often inexpensive and fusible; exploitable sparsity depends on downstream support.
Softmax	Produces a normalized distribution over an axis.	Requires reductions and cross-element coordination.	Parallel implementations coordinate maximum, sum, and normalization stages.

ReLU remains common because it is simple and efficient, while softmax is essential when a model must normalize scores into probabilities. GPT-2 illustrates a different choice through GELU, the Gaussian Error Linear Unit.

Systems Perspective 8.3: The GELU activation choice

Beyond the foundational activation functions (Sigmoid, Tanh, ReLU), modern architectures increasingly adopt smoother alternatives. GPT-2 uses a GELU activation (Radford et al. 2019), the Gaussian Error Linear Unit introduced by Hendrycks and Gimpel (2016) and often implemented with the common tanh-based approximation from that formulation. Its exact definition is:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution.

GELU applies a smooth, input-dependent scaling rather than ReLU's hard threshold. It is deterministic despite its formulation in terms of a Gaussian cumulative distribution. Whether it improves model quality relative to another activation is an empirical property of the architecture and training recipe, not a universal guarantee.

Exact GELU uses `erf`, while common implementations use a tanh-based approximation such as listing 8.1. The approximation changes arithmetic cost and introduces a small numerical difference from the exact function. Its end-to-end cost depends mainly on whether the compiler fuses the activation into the preceding matrix multiplication, which removes the memory round-trip that dominates a standalone activation kernel.

Listing 8.1: GELU Approximation: Fast approximation avoids expensive erf() computation while preserving activation properties.

```
# Fast GELU approximation used in production systems
# Avoids expensive erf() computation while
# preserving activation properties
gelu_approx = (
    0.5 * x * (1 + tanh(sqrt(2 / pi) * (x + 0.044715 * x**3)))
)
```



Systems Perspective 8.4: Memory bandwidth bottlenecks

Activation functions reveal a critical systems principle: not all operations are compute bound. While sufficiently large matrix multiplications can saturate accelerator compute units through operand reuse, standalone low-intensity activation functions often become memory bandwidth bound for three reasons:

- Elementwise operations perform few calculations per memory access; ReLU performs one operation per load.
- Simple operations complete faster than memory transfer time, limiting parallelism benefits.
- Accelerator compute capability has grown faster than external-memory bandwidth, increasing pressure to reuse data near the compute units.

This is why scalar operation counts alone do not predict activation-kernel latency. The forward pass must account for fusion and activation storage as well as function-evaluation cost when determining whether memory bandwidth limits throughput.

Forward-pass matrix multiplications dominate floating-point operations, while activation functions stress memory bandwidth. Transforming these forward activations into model convergence requires propagating error gradients backward through the graph to update parameter tensors. That optimization step introduces state storage, update scheduling, and numerical stability constraints that govern training efficiency.

↩ The memory wall that bounds elementwise activation kernels, and kernel fusion as the framework's response to it, are established in section 7.3.1.

8.4.2 Optimization algorithms

Optimization algorithms translate error gradients into parameter updates that steer the model toward convergence. While mathematically formulated as steps across an abstract loss surface, every optimizer step introduces concrete physical constraints on accelerator execution: persistent state tensors that consume High Bandwidth Memory (HBM), memory-bandwidth-bound elementwise updates, and synchronization barriers. In the D·A·M taxonomy, the choice of optimization algorithm couples the mathematical trajectory (Algorithm) directly to the working-set footprint and memory traffic (Machine).

8.4.2.1 Gradient-based optimization methods

Gradient descent is the fundamental optimization algorithm: it iteratively adjusts parameters in the direction of steepest descent. That conceptual formulation assumes modest networks on single devices. At hardware scale, gradient descent and its variants collide with physical bottlenecks: memory capacity limits how many activations and optimizer states remain resident, while memory bandwidth limits how rapidly parameters update.

↩ Gradient descent and its parameter-update rule are established in section 5.5.4.

Gradient descent In a training system, each iteration of gradient descent becomes a chain of four dependent operations, none of which can start before its predecessor finishes:

1. Compute forward pass activations
2. Calculate loss value
3. Compute gradients through backpropagation

4. Update parameters using the gradient values

The computational demands of gradient descent scale with both model size and dataset size. Computing gradients requires storing intermediate activations during the forward pass for back-propagation. These activations consume memory proportional to the depth of the network and the number of examples being processed.

Traditional gradient descent processes the entire dataset before each parameter update. For a training set with one million examples, the system must compute an aggregate gradient over all examples before taking one step. Write D for the dataset size, counted here in examples rather than in the tokens of the lighthouse's $6PD$ budget. The examples cannot all be resident at once, so the system streams them through the accelerator in groups of B_{micro} examples per forward/backward pass. With $T_{\text{forward+backward per example}}$ the amortized forward/backward time per example at that group size and $M_{\text{activations}}$ the peak activation memory, equation 8.2 and equation 8.3 separate what bounds memory from what bounds step time:

$$M_{\text{activations}} \propto B_{\text{micro}} \tag{8.2}$$

$$T_{\text{step}} \propto D \times T_{\text{forward+backward per example}} \tag{8.3}$$

Full-batch training therefore does not require storing every example's activations simultaneously; peak memory is governed by the examples held in memory at once. The systems problem is still severe: processing $D = 1{,}000{,}000$ examples before each update creates million-example iteration times where parameters remain frozen, reducing the rate at which the model can learn from data.

Hardware-friendly variants relax the need for an exact full-dataset gradient on every update. Stochastic gradient descent (SGD)[9] estimates gradients from sampled examples or mini-batches rather than the entire dataset. This changes update frequency and gradient variance while allowing each update to operate on a bounded working set.

Evaluating single examples ($B = 1$) can reduce accelerator efficiency when each example supplies only one row to a dense layer. Modern accelerators achieve peak throughput by mapping large GEMM operations across systolic arrays and single instruction, multiple data (SIMD) execution units. In that case, layer operations become matrix-vector products (GEMV), reducing arithmetic intensity to $\mathcal{O}(1)$ operations per byte transferred from device memory. Sequence and spatial dimensions can still supply multiple rows even at batch size one. For a bandwidth-limited matrix-vector operation, compute units wait for DRAM or HBM transfers. Simultaneously, single-example updates require reading parameter tensors from device memory, applying gradients, and writing updated weights back after every sample, saturating memory bandwidth without amortizing parameter access across multiple inputs.

Mini-batch processing Mini-batch gradient descent emerges as a practical compromise between full-batch and stochastic methods, an algorithm-machine co-design that computes gradients over small batches of examples aligned with modern accelerator architectures (Dean et al. 2012). The batch size B becomes a key system parameter, influencing both computational efficiency and memory requirements.

[9] **Stochastic gradient descent**: "Stochastic" from Greek *stochastikos* ("able to guess"); rather than waiting for an exact full-dataset gradient, SGD updates from sampled examples or mini-batches. Full-batch and stochastic methods can both stream data through bounded micro-batches, so their peak activation memory is determined by the resident micro-batch rather than by dataset size alone. Their central difference is how many sampled gradients contribute to each parameter update.

Definition 8.3: Batch processing

Batch processing aggregates multiple training examples into tensor operations that amortize fixed per-step overhead across B examples and can expose more parallel work. Increasing B may improve occupancy and arithmetic intensity, but it does not guarantee that every operation becomes compute bound.

1. Significance: Throughput often increases with batch size until the target hardware is sufficiently occupied, while the number and quality of updates needed for convergence follow a separate workload-dependent curve. Goyal et al. demonstrated a ResNet-50 ImageNet recipe at $B = 8{,}192$ using linear learning-rate scaling and warmup (Goyal et al. 2017). That result is a configuration-specific demonstration rather than a universal critical batch size.

2. Distinction: A mini-batch update averages gradients over B examples. Under independent, similarly distributed samples, the standard deviation of this mean decreases approximately as $1/\sqrt{B}$. The update processes more data than a single-example update, so its net data movement and time depend on reuse, implementation, and how batch size changes the number of updates required.
3. Common pitfall: A frequent misconception is that linear learning-rate scaling ($\eta_{\text{new}} = \eta_{\text{base}} \times B_{\text{new}}/B_{\text{base}}$, where η is the optimizer's learning rate, not the utilization factor η_{hw} of the training-time law) works at any batch size. It is a training recipe that commonly requires warmup and empirical validation, and its useful range depends on the model, data, optimizer, and target accuracy.

The choice of batch size B directly governs accelerator memory allocation. The partial decomposition in equation 8.4 separates fixed parameter and gradient storage from the batch-dependent activation working set; optimizer states, any master weights, and runtime workspaces must also enter the full budget:

$$\text{Memory Required} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{activations}}, \qquad M_{\text{activations}} \propto B \tag{8.4}$$

Because the activation term scales with B while parameter and gradient memory stay fixed, doubling the batch doubles the activation working set, and a model that fits comfortably at a small batch can exhaust the 40–80 GB of HBM on a high-end training accelerator once the batch grows. Section 8.5.3.2 works this budget through layer by layer for ResNet-50.

Increasing batch size can move dense operations toward the compute-bound regime by increasing weight reuse. In dense feedforward and attention projections, parameter weights loaded from HBM into on-chip SRAM are reused across all B samples in the batch. For weight-traffic-dominated projections, arithmetic intensity initially increases with B, amortizing parameter reads; shape and the hardware ridge point determine whether compute throughput becomes limiting. Learning-rate scaling and warmup can preserve convergence over a validated range of batch sizes (Goyal et al. 2017).

When the batch size necessary to saturate execution units or ensure stable convergence exceeds physical memory capacity, systems decouple the physical micro-batch size from the algorithmic batch size using gradient accumulation. By executing a sequence of smaller micro-batches (B_{micro}) and accumulating their resulting gradients in place before invoking the optimizer, the training loop achieves the statistical convergence of a large effective batch while restricting peak activation storage to that of a single micro-batch.

⇄ **Contrast:** Training packs large batches to amortize weight transfers and saturate arithmetic units, whereas online serving inverts the priority to per-request latency under stochastic arrivals, as analyzed in section 13.1.

8.4.2.2 Adaptive and momentum-based optimizers

Basic SGD applies a uniform scalar learning rate to every parameter tensor, which can stall training when loss curvature differs by orders of magnitude across dimensions. Momentum smooths updates using directional gradient history, RMSprop adapts individual step sizes using squared-gradient magnitudes, and Adam combines first-moment tracking with adaptive second-moment scaling (Kingma and Ba 2015). While these algorithmic enhancements can reduce the total optimization steps required to converge, they do so by introducing persistent auxiliary state that directly increases accelerator memory pressure and memory-bus traffic.

Momentum-based methods Momentum methods[10] address SGD's oscillation problem by accumulating a velocity vector across iterations, damping high-frequency oscillations along steep ravines. In hardware, this smoothing requires allocating and maintaining a persistent velocity tensor matching the exact shape and precision of the model parameters. Parameters and this auxiliary state together occupy $2\times$ the parameter footprint before gradients and activations are counted.

Adaptive learning rate methods While momentum smooths gradient direction, it does not address the different scales of gradients across parameters. RMSprop[11] solves this by maintaining a moving average of squared gradients for each parameter, automatically reducing step sizes for parameters with historically large gradients. Adapting step sizes per parameter requires maintaining a running average of squared gradients s_t, imposing a $2\times$ static memory overhead equivalent to momentum.

[10] **Momentum**: Borrowed from physics, where momentum (mass $\times$ velocity) describes an object's tendency to continue moving. The metaphor explains the design because accumulated velocity smooths noisy gradients and can reduce the iterations needed for convergence. The systems cost is one additional velocity value per parameter, while Adam maintains two moment values per parameter.

[11] **RMSprop**: Proposed by Geoffrey Hinton in Lecture 6e of his 2012 Coursera course—never published in a peer-reviewed paper, making it perhaps the most influential optimizer disseminated via a slide deck. RMSprop divides the learning rate by a running average of recent gradient magnitudes, adapting step sizes per parameter. This per-parameter adaptation is what Adam inherits as its second moment v_t, directly contributing to Adam's $3\times$ memory overhead described in section 8.4.2.3.

Furthermore, computing this running average introduces elementwise square and square-root kernels during each parameter update.

Adam optimization Adam[12] combines the benefits of both momentum and RMSprop: momentum's gradient smoothing addresses noisy updates, while RMSprop's adaptive scaling handles parameter-specific step sizes. This combination maintains two moving averages for each parameter. Here m_t and v_t are the first- and second-moment buffers, β_1 and β_2 are their decay factors, η is the learning rate, and ϵ is the numerical-stability constant in the denominator:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla\mathcal{L}(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla\mathcal{L}(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$$

Maintaining both moment buffers triples the parameter-plus-optimizer-state footprint: the training system must allocate two persistent state tensors (m_t and v_t) alongside the parameter tensors. For a 100M-parameter model, those auxiliary vectors alone add 800 MB beyond weight storage.

12 **Adam (adaptive moment estimation)**: The two moving averages are the first moment (momentum) and second moment (uncentered variance) of the gradients, stored for every model parameter. For a 7B model, Adam's FP32 moment tensors alone consume 56 GB; the full standard mixed-precision training state before activations is 112 GB once FP16 weights, FP16 gradients, FP32 master weights, and Adam moments are counted together.

8.4.2.3 Optimization algorithm system implications

Auxiliary state memory increases progressively from SGD (zero auxiliary state) through Momentum and RMSprop (one state vector per parameter) to Adam (two state vectors per parameter), as quantified in table 8.4. These memory footprints establish a fundamental trade-off against convergence rates:[13] while Adam often achieves target loss in fewer optimization steps, its static memory footprint restricts the batch size that fits in accelerator HBM and increases memory-bus traffic on every update. At GPT-2 scale, this overhead becomes a first-order memory constraint.

13 **Convergence**: A run reaches its stopping criterion when the selected training or validation metric no longer improves enough to justify additional work. The required steps depend on the model, data, optimizer, schedule, batch size, and target quality. Fewer steps can reduce wall-clock time and cost, but adaptive optimizers add state whose memory and traffic must be included in the system budget.

Table 8.4: Optimizer Memory Footprint: Different optimization algorithms impose varying auxiliary-state costs due to the storage of intermediate values like velocities and squared gradients. The multiplier row counts parameters plus optimizer auxiliary state and excludes gradients and activations; full training memory must add those terms explicitly. These trade-offs govern resource-constrained deployments and large-scale model training.

Property	SGD	Momentum	RMSprop	Adam
Memory Overhead	None	Velocity terms	Squared gradients	Both velocity and squared gradients
Parameter + optimizer state	1×	2×	2×	3×
Dense State Access	Parameters and gradients	Adds velocity	Adds squared-gradient state	Adds first- and second-moment state
Update Work	Lowest auxiliary work	Adds momentum update	Adds adaptive scaling	Adds moment updates and bias correction
Hardware Efficiency	Varies	Varies	Varies	Varies
Convergence Behavior	Varies	Varies	Varies	Varies

Three multipliers for Adam's memory appear in this chapter, and they differ only in what they count and what they divide by. The table's 3× counts FP32 weights plus two FP32 moments against the FP32 weights alone, 12 bytes per parameter against 4. Adding FP32 gradients gives the 16 bytes per parameter of plain FP32 Adam. The standard mixed-precision recipe also lands at 16 bytes (2 for FP16 weights, 2 for FP16 gradients, 4 for FP32 master weights, 8 for the moments), and it is that same 16 bytes, measured against the 2 bytes of FP16 inference weights, that the definition of training systems reports as 8×.

The costs quantified in table 8.4 create a design tension. Adam's two moment buffers may improve optimization for a given workload, but their memory can reduce the model size or batch size that fits on an accelerator. AdamW[14] (Loshchilov and Hutter 2019) decouples weight decay from Adam's adaptive gradient update without adding another per-parameter state tensor.

14 **AdamW (Adam with decoupled weight decay)**: Standard ℓ_2 regularization enters the gradient and is therefore rescaled by Adam's adaptive update. AdamW applies weight decay as a separate parameter update, restoring the intended distinction between adaptive gradient scaling and decay (Loshchilov and Hutter 2019). It uses the same moment-buffer structure as Adam, though its effect on model quality remains workload dependent.

Napkin Math 8.3: GPT-2 optimizer memory requirements

A representative GPT-2 XL training configuration uses the Adam optimizer with five hyperparameters:

- $\beta_1 = 0.9$ (momentum decay)
- $\beta_2 = 0.999$ (second moment decay)
- Learning rate: Warmed up from 0 to 2.5e-4 over first 500 steps, then cosine decay
- Weight decay: 0.01
- Gradient clipping: Global norm clipping at 1.0

Math:
For GPT-2's 1.5B parameters in FP32 (4 bytes each), the memory breaks down across four components:

- Parameters: 1.5B $\times$ 4 bytes = 6 GB
- Gradients: 1.5B $\times$ 4 bytes = 6 GB
- Adam State (m_t, v_t): 1.5B $\times$ 8 bytes = 12 GB
- Total static memory: 24 GB

This static training state alone, 24 GB for GPT-2, takes most of a V100's 32 GB before any activation is stored, and how much room the activations need depends on batch size and sequence length. This chapter builds the GPT-2 memory ledger in stages. The activation notebook below starts at batch 4; the mixed-precision ledger tests an explicit checkpoint schedule for that micro-batch, which the optimization walkthrough (section 8.7.7) carries into profiling.

System Decisions Driven by Optimizer

1. Mixed precision training (FP16) reduces operation precision but requires keeping FP32 master weights, so the static footprint stays at ~24 GB, the same 16 bytes per parameter as FP32 Adam.
2. Gradient accumulation (section 10) builds an effective batch of $B = 512$ from micro-batches that fit in memory.

Systems insight: This illustrative configuration shows how Adam's moment buffers constrain capacity. Any convergence benefit relative to SGD or momentum must be measured under a matched model, data, schedule, batch size, and stopping criterion.

8.4.2.4 Framework optimizer interface and scheduling

After optimizer memory is quantified, the framework interface matters because it fixes when that state is read, written, cleared, and preserved across steps. A training loop separates gradient computation from parameter updates so that the system can accumulate gradients, synchronize them, or defer updates without changing the optimizer equations. Listing 8.2 demonstrates where Adam optimization enters that cycle.

Listing 8.2: Adam Training Loop: Standard four-step optimization cycle with gradient clearing, forward pass, backward pass, and parameter update.

```python
import torch
import torch.nn as nn
import torch.optim as optim

# Initialize Adam optimizer with model parameters and learning rate
optimizer = optim.Adam(
    model.parameters(), lr=0.001, betas=(0.9, 0.999)
)
loss_function = nn.CrossEntropyLoss()

# Standard training loop implementing the four-step optimization cycle
for epoch in range(num_epochs):
    for batch_idx, (data, targets) in enumerate(dataloader):
        # Step 1: Clear accumulated gradients from previous iteration
        optimizer.zero_grad()

        # Step 2: Forward pass - compute model predictions
        predictions = model(data)
        loss = loss_function(predictions, targets)

        # Step 3: Backward pass - compute gradients via autodiff
        loss.backward()

        # Step 4: Parameter update - apply Adam optimization equations
        optimizer.step()
```

The `optimizer.zero_grad()` call marks the boundary between one update and the next. Gradients accumulate across calls to `backward()`, so clearing them explicitly prevents stale gradients from contaminating the next batch. The same accumulation behavior later becomes useful for large effective batch sizes, but only when the training loop manages the boundary deliberately.

The `optimizer.step()` method is the other boundary: it consumes the current gradients and mutates persistent optimizer state. For Adam optimization, this call implements momentum estimation, squared gradient tracking, bias correction, and the parameter update. Algorithm 8.1 makes the hidden state explicit so the memory cost remains visible rather than disappearing behind an API call.

Algorithm 8.1 Adam parameter update (one optimizer step)

Require: gradient $g_t = \nabla \mathcal{L}(\theta_t)$; step t; rate η; decays β_1, β_2; constant ϵ
Ensure: updated parameter θ_{t+1}; moment buffers m_t, v_t carried to the next step
1: $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1)\, g_t$ ▷ *first moment (momentum)*
2: $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$ ▷ *uncentered second moment*
3: $\hat{m}_t \leftarrow m_t/(1-\beta_1^t)$; $\hat{v}_t \leftarrow v_t/(1-\beta_2^t)$ ▷ *bias correction*
4: $\theta_{t+1} \leftarrow \theta_t - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$ ▷ *parameter update*



Parameters (P) and gradients (G) take one share each, and Adam's moment buffers m and v add two more in this simplified static-state view.

Steps 1 and 2 keep two persistent moment buffers per parameter, so parameters plus optimizer state reach roughly 3× the parameter memory before gradients, activations, and FP32 master weights are counted. Beyond capacity, the optimizer step imposes a severe memory-bandwidth bottleneck. The update consists of elementwise operations with an arithmetic intensity below 1 FLOP/byte. In FP32, updating each parameter requires reading four tensors ($g_t, \theta_t, m_{t-1}, v_{t-1}$) and writing three ($m_t, v_t, \theta_{t+1}$), streaming 28 bytes across the memory bus for roughly a dozen arithmetic operations. Because matrix execution units (such as Tensor Cores) sit idle during this memory-bound phase, high-performance training systems use fused optimizer kernels to combine moment updates, bias correction, and weight writes into a single kernel launch per parameter group, minimizing round-trips through device memory.

8.4.2.5 Learning rate scheduling integration

The framework's learning rate scheduling hook changes the optimizer's trajectory without adding per-parameter state. It adjusts the learning rate η during training, letting the system shape convergence behavior while leaving the underlying optimizer equations intact.

Schedules such as cosine annealing, exponential decay, or step-wise reductions implement that trajectory by changing the step size over time. A scheduler tracks lightweight scalar state (current step, epoch, warmup phase) that must be serialized into checkpoints alongside optimizer states to resume the schedule. Reproducing the training trajectory also requires the relevant random-number and data-position state and compatible execution settings. This separation lets the systems engineer combine base optimization algorithms (SGD, Adam) with scheduling strategies (cosine annealing, linear warmup) without modifying the underlying tensor update rules.

8.4.2.6 Time to target quality

A faster update is valuable only if the recipe reaches the required validation quality sooner. Optimizer state and numerical precision affect the price of each update, while the optimizer and schedule affect how many updates the run needs. Both quantities enter a time-to-target decision.

Consider two precision-and-optimizer recipes for the same model, dataset, effective batch, machine, and validation target. Both fit in memory. Table 8.5 supplies hypothetical observations for their first accepted checkpoint and calculates the total elapsed time from them.

Table 8.5: Illustrative Time-to-Quality Comparison: Step durations, target-reaching update counts, and evaluation/checkpoint overhead are supplied observations for a hypothetical workload, not benchmark results. Totals multiply updates by end-to-end step time and add overhead excluded from that step time.

Quantity	Recipe A	Recipe B
End-to-end update time	2 s	1.5 s
Updates to the same validation target	6,000	10,000
Update time × update count	200 min	250 min
Separate evaluation and checkpoint time	10 min	10 min
Total time to accepted checkpoint	210 min	260 min

Recipe A reaches the target sooner. At its shorter step duration, Recipe B would tie A's total time if it reached the target at 8,000 updates under the stated overhead. Its assumed 10,000 updates exceed that budget. On the same allocated machine, shorter total time also means fewer billed accelerator-hours.

A profile measures the price of an update; validation measures how many updates buy the required quality. Comparing optimizers, precision, or batch schedules requires both measurements under a common acceptance criterion. A real experiment must also account for run-to-run variation and any change in evaluation frequency; an apparent crossing of the target is not evidence that a recipe reliably meets it.

Parameter update rules define trajectories given gradient vectors, but assume those gradients are already materialized. For billion-parameter networks, gradient evaluation constitutes the dominant computational and memory burden of the training loop. The mechanics of gradient generation—rather than the optimizer step itself—dictate training memory budgets and runtime.

8.4.3 Backpropagation mechanics

Backpropagation evaluates parameter gradients by traversing the network's computational graph in reverse topological order. In forward inference, an activation buffer can be released after its final consumer. Training extends some of these lifetimes: computing the gradient with respect to a linear layer's parameters requires its forward input. The backward computation must obtain the values required by each derivative, either from saved state or by recomputation. Retaining those values in HBM can make activation storage the binding memory constraint of large-scale training.

Section 5.5.5 establishes the mathematical foundation: the chain rule decomposes gradient evaluation into layer-by-layer vector-Jacobian products, pulling error signals backward from the scalar

loss to input parameters. At systems scale, the central question is not what backpropagation computes, but what it costs in memory capacity and arithmetic throughput. The layer operations from section 8.4.1.1 produce forward values needed by the backward pass. Computing $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}}$ requires the corresponding forward inputs, whose retention creates the training memory equation that gradient checkpointing later exploits.

The arithmetic side of that cost follows from the derivative equations of section 5.5.5.3. Backpropagation through a linear layer $\mathbf{Z} = \mathbf{AW}$ evaluates two matrix products, the activation gradient $\nabla_{\mathbf{A}} \mathcal{L} = (\nabla_{\mathbf{Z}} \mathcal{L})\mathbf{W}^T$ that carries the error signal upstream and the parameter gradient $\nabla_{\mathbf{W}} \mathcal{L} = \mathbf{A}^T(\nabla_{\mathbf{Z}} \mathcal{L})$ that the optimizer consumes. Each permutes the forward matrix dimensions while keeping the forward product's leading FLOP count, so backward costs about twice forward and a complete step about three times the forward FLOPs. At the scale of a transformer, that same factor of three turns the $2P$ forward FLOPs per token into the $6P$ per token behind the lighthouse budget. The memory side of the cost has no such fixed ratio, because it depends on which forward values the backward rules must read.

The three-layer MNIST classifier ($784 \to 128 \to 64 \to 10$) of chapter 5 needed only kilobytes of activation storage. GPT-2 processing a single batch of 4 sequences of 1024 tokens requires approximately 35.9 GB of saved activations. Adding parameters, gradients, master weights, and optimizer state raises the estimated training footprint to 59.9 GB, exceeding the V100's 32 GB capacity before temporary workspace is counted. Section C.3.3 prices each term of the full training memory equation ($M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$) for GPT-2. Modern ML frameworks automate gradient evaluation through automatic differentiation, but managing the resulting memory footprint and arithmetic schedule remains an unavoidable systems engineering responsibility.

8.4.3.1 Activation memory requirements

The four-term training-memory budget of equation 5.28 partitions training state into four categories, and only the first grows with the batch:

- saved forward activations required by backward derivative rules,
- model parameters,
- parameter gradients, and
- optimizer state and any higher-precision master weights.

Consider a batch of training examples passing through a network. The forward pass computes and stores:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$
$$\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)})$$

To compute the parameter gradient $\nabla_{\mathbf{W}^{(\ell)}} \mathcal{L} = (\mathbf{A}^{(\ell-1)})^T(\nabla_{\mathbf{Z}^{(\ell)}} \mathcal{L})$, the backward rule must read the layer's input activations $\mathbf{A}^{(\ell-1)}$. To propagate the error signal upstream through the nonlinear activation function via $\nabla_{\mathbf{Z}^{(\ell)}} \mathcal{L} = (\nabla_{\mathbf{A}^{(\ell)}} \mathcal{L}) \odot f'(\mathbf{Z}^{(\ell)})$, the kernel requires either the pre-activation tensor $\mathbf{Z}^{(\ell)}$ (as in GELU) or the post-activation output $\mathbf{A}^{(\ell)}$ (when f' can be computed from the output, as in sigmoid, or via a 1-bit boolean mask in ReLU). Fused kernels and selective recomputation can compress this saved footprint. Nevertheless, because activations scale with batch size B and sequence length S, their storage demands grow with the data dimension, whereas parameter and optimizer memory remain fixed by the model architecture. The GPT-2 calculation in notebook 8.4 decomposes this memory pressure across attention blocks, feed-forward sublayers, and optimizer states.

> Napkin Math 8.4: GPT-2 activation memory breakdown
>
> For GPT-2 with batch size B = 4, sequence length S = 1024, hidden dimension d = 1600, and 48 layers:
>
> **Math**:
>
> - Attention activations: $B \times S \times d \times 4$ (Q, K, V, output) = $4 \times 1024 \times 1600 \times 4 \times 2$ bytes (FP16) = 52.4 MB

- FFN activations: $B \times S \times (4d)$ (intermediate expansion) = 4 × 1024 × 6400 × 2 bytes = 52.4 MB
- Attention scores: $5 \times N_{\text{heads}} \times S^2 \times B$ bytes, because each element of a head's $S \times S$ attention matrix is saved three times for backward: 2 bytes for the FP16 softmax probabilities, 1 byte for the dropout mask, and 2 bytes for the probabilities after dropout. With 25 heads, this term reaches 524.3 MB per layer. It is the dominant term, quadratic in sequence length, and exactly what selective recomputation discards
- Layer norm states: Minimal (~10 MB per layer)
- Total per layer: ~639.1 MB (attention + FFN + attention scores + layer norm states)

Result:

- Line items across all layers: 639.1 MB × 48 layers ≈ 30.7 GB
- Total activation memory (library estimate): 35.9 GB, about 5.2 GB more than the line items because it also counts saved tensors the list omits, such as the inputs to each linear and normalization layer, the GELU output, and the dropout masks
- Parameters (FP16): 3 GB
- Gradients: 3 GB
- Master weights (FP32): 6 GB
- Adam moment state (FP32): 12 GB
- Peak memory during training: ~59.9 GB

This exceeds a single V100's 32 GB capacity.

Solutions:

1. Selective activation recomputation: Discard attention intermediates and recompute them during backward, reducing this modeled activation term by 70.2 percent to ~10.7 GB; the additional work depends on the implementation
2. Activation CPU offloading: Store some activations in CPU RAM, transfer during backward pass
3. Mixed precision: FP16 activations (already applied) vs. FP32 (would be 71.7 GB)
4. Reduced micro-batch size and gradient accumulation: Build a larger effective batch from multiple memory-feasible forward/backward passes

Systems insight: Even after selective recomputation, the modeled activations plus static state total about 34.7 GB, slightly above the raw 32 GB V100 capacity before workspace and allocator overhead. Selective recomputation removes 70.2 percent of the activations; the explicit segment-checkpoint ledger in notebook 8.6 retains only segment inputs between passes and budgets the live recomputed segment. That schedule admits batch 4 in the tensor budget, which the optimization walkthrough carries into profiling.

This breakdown illustrates the practical engineering decisions required when accelerator memory falls short. The same trade-off between stored activations, recomputation, and batch size drives the memory-computation analysis that follows.

Checkpoint 8.2: The memory-compute trade-off

Training large models requires managing the memory wall, the gap between accelerator compute throughput and memory bandwidth.

Bottleneck

- ☐ **Activation memory**: Explain why activations stored for backpropagation dominate memory usage and, for some model shapes and recipes, exceed parameter footprint by more than 10×.

- ☐ **Optimization strategy**: Explain how gradient checkpointing trades additional forward compute for memory capacity.
- ☐ **Memory estimate**: For GPT-2 (1.5B parameters) in mixed precision, estimate each training state component (weights, gradients, optimizer state, and activations at batch size 32), identify the largest term, and determine whether the total fits on a 32 GB accelerator.

Scaling limits

- ☐ **Batch size constraints**: Explain why memory capacity limits the maximum batch size, and how gradient accumulation sidesteps this limit without expanding memory.

8.4.3.2 Memory-computation trade-offs

Training systems must balance memory usage against computational efficiency. Each forward pass generates values that backward rules may request. For a simplified network model with N_L layers, let s_ℓ denote the bytes of saved intermediate state per example at layer ℓ and $M^{(\ell)}_{\text{activations}}$ the bytes of saved activation outputs per example. Under this per-example convention, their storage scales linearly with batch size and is approximated by equation 8.5:

$$M_{\text{activations}} = B \times \sum_{\ell=1}^{N_L} \left(s_\ell + M^{(\ell)}_{\text{activations}} \right) \tag{8.5}$$

This memory requirement compounds with the weights, gradients, and optimizer memory discussed in section 8.4.2.3. Together they form the same four-term budget that equation 5.28 established and the lighthouse callout applied to GPT-2, labeled here once as equation 8.6 so that later sections can point to it:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}} \tag{8.6}$$

The runtime peak can exceed this sum because temporary workspaces, communication buffers, allocator behavior, and framework bookkeeping also occupy memory. To reduce the activation term, training systems can strategically recompute intermediate values during backward rather than storing them. This increases computational work but can enable deeper networks or larger batches on memory-constrained hardware (T. Chen et al. 2016). Algorithm 8.2 makes the trade explicit: the forward pass keeps activations at only a sparse set of checkpoint layers, and the backward pass recomputes the rest on demand.

Algorithm 8.2 Gradient checkpointing (activation recomputation)

Require: N_L-layer network; checkpoint set $\mathcal{C} \subseteq \{1, \ldots, N_L\}$ (e.g., every $\sqrt{N_L}$ layers)
Ensure: parameter gradients, at reduced peak activation memory

```
for ℓ = 1 to N_L do
    forward layer ℓ; store its activation only if ℓ ∈ C ▷ drop the rest
end for
for ℓ = N_L down to 1 do
    if the activation of layer ℓ was dropped then
        recompute the forward segment from the nearest stored checkpoint through layer ℓ
    end if
    compute the layer's gradient; free the activation
end for
return the accumulated gradients
```

In this segment schedule, recomputed intermediate values remain available until their backward consumers finish; recomputing a segment afresh for each layer would add more work. For layers with comparable activation sizes and forward costs, spacing checkpoints uniformly every $\sqrt{N_L}$ layers reduces peak activation memory from $\mathcal{O}(N_L)$ to $\mathcal{O}(\sqrt{N_L})$ at the cost of one additional forward pass

of arithmetic per training step (T. Chen et al. 2016). This increases executed step computation from roughly $3\times$ to $4\times$ the forward-pass FLOPs, trading extra arithmetic operations for a substantially reduced activation footprint. The useful model FLOP count O is unchanged, so the additional work enters the training-time law through effective utilization η_{hw}. Actual fit and wall-clock overhead still depend on the full memory budget and execution profile. Among the pipeline optimizations, section 8.7.5.2 derives why $\sqrt{N_L}$ spacing minimizes this footprint and prices it for GPT-2's 48 layers.

Capacity is only one of two physical constraints imposed by memory; the other is bandwidth. Accelerators provide massive matrix compute throughput, but shuttling intermediate activation tensors between off-chip High Bandwidth Memory (HBM) and on-chip SRAM frequently saturates the memory bus (Jouppi et al. 2017). Even when activations fit comfortably within HBM capacity, the time required to read and write them can dwarf the arithmetic execution time. To relieve this memory bus pressure, modern training engines fuse elementwise operations directly into preceding matrix multiplications and stage gradient computations through on-chip SRAM tiles rather than global memory. At the runtime level, framework caching allocators track tensor lifetimes across the computation graph, recycling memory blocks as soon as an activation's final backward consumer completes (Paszke et al. 2019).

Forward propagation, gradient computation, and parameter updates define *what* training systems must compute. An operation's name alone, however, does not reveal *where* the system stalls. Matrix shape, kernel fusion, numerical precision, and the memory hierarchy determine whether arithmetic throughput or data movement binds execution. Arithmetic intensity provides the analytical tool to quantify that balance.

8.4.4 Arithmetic intensity

Arithmetic intensity captures this distinction—the ratio of computation to data movement that reveals whether an operation is limited by compute throughput or memory bandwidth:

$$I = \frac{O}{D_{\text{vol}}}$$

Within the roofline model, operations above the hardware ridge point have a compute-throughput ceiling, while those below it have a memory-bandwidth ceiling. Launch overhead, limited parallelism, and imperfect execution can keep measured performance below either ceiling. Section D.2.1 gives the formal definition of the Roofline Model and shows how to compute a hardware's ridge point.[15]

Table 8.6 summarizes common tendencies rather than fixed measurements. Dense matrix multiplication can reuse operands enough to reach high arithmetic intensity, while standalone elementwise and reduction kernels often perform less work per byte moved. The target hardware's ridge point determines the resulting regime.

[15] **Ridge point and precision**: The roofline ridge point—the arithmetic intensity threshold separating memory-bound from compute-bound operations—shifts with numerical precision. On the same accelerator, a lower-precision Tensor Core path can expose much more arithmetic throughput at roughly the same memory bandwidth, raising the ridge point substantially. Switching from TF32-style execution to BF16 mixed precision can therefore change which optimization technique yields returns, making precision selection inseparable from roofline analysis.

Table 8.6: Training Operation Classification Conditions: Operation families exhibit common arithmetic-intensity tendencies, but classification requires comparing the measured implementation with the target hardware's ridge point. Memory-sensitive kernels may benefit from reducing traffic or increasing fusion, while compute-bound kernels benefit from additional arithmetic throughput.

Operation	Arithmetic-intensity tendency	Classification condition
Dense MatMul (large)	High when shapes and tiling provide substantial operand reuse	Compute-bound only when intensity exceeds the ridge
Activation functions	Usually low as standalone elementwise kernels	Often memory-sensitive unless fusion removes traffic
LayerNorm/BatchNorm	Reduction and elementwise work with multiple tensor passes	Depends on fusion, shape, and target ridge point
Attention softmax	Reduction and normalization over the attention axis	Depends on materialization, tiling, and fusion

Hardware efficiency during model training depends directly on kernel arithmetic intensity. Figure 8.1 bounds attainable throughput across training kernels on an A100 roofline, contrast-

ing memory-bandwidth-bound operators (softmax, LayerNorm) along the sloped ceiling against compute-bound operators (dense MatMul) on the flat 312 TFLOP/s peak.

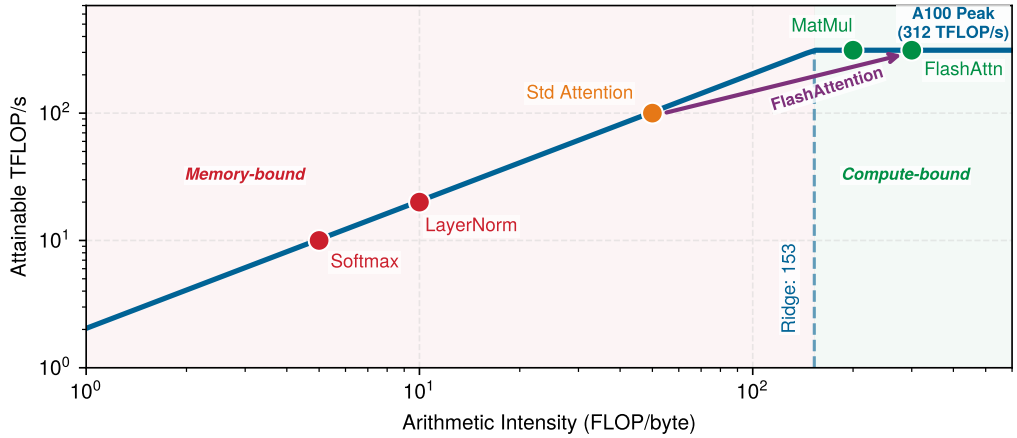


Figure 8.1: Illustrative Training Roofline: An A100 roofline with assumed operation coordinates illustrates how arithmetic intensity changes the attainable-performance ceiling. The points are not a profile of one GPT-2 execution. FlashAttention is shown moving right because it reduces HBM traffic; whether a real attention kernel crosses the ridge depends on shape, precision, hardware, and implementation.

The arithmetic-intensity continuum charted in figure 8.1 isolates how hardware limits partition execution regimes. The ridge point, the A100's 312 TFLOP/s peak divided by its 2.04 TB/s of HBM bandwidth, sits at 153 FLOP/byte and marks the saturation knee where the sloped memory-bound region meets the flat compute-bound ceiling. An implementation plotted left of this point is bandwidth bound under the roofline assumptions; one plotted right is compute bound. The operation points in this diagram are illustrative placements that show how to interpret the model, not measured GPT-2 coordinates.

Consider a GPT-2 attention layer that materializes the $S \times S$ attention-score matrix. For each head, the $\mathbf{QK}^T$ product costs approximately $2S^2 d_{\text{head}}$ FLOPs, while writing the FP16 score matrix and reading it back moves about $4S^2$ bytes. Counting only that score-matrix traffic gives the approximation $d_{\text{head}}/2$. For GPT-2 Small ($d = 768$ across 12 heads, so $d_{\text{head}} = 64$), this restricted accounting yields 32 FLOP/byte. The lighthouse GPT-2 XL gives the same answer, because its head dimension is also 64 and the ratio depends only on d_{head}. A complete kernel analysis must also count Q and K traffic, softmax work, additional reads and writes, cache reuse, and fusion. The $\mathcal{O}(S^2)$ materialized score traffic remains the term that FlashAttention targets.

Accelerators have characteristic hardware ridge points where operations transition from memory-bound to compute bound. At 153 FLOP/byte, the A100's ridge sits far to the right of the 32 FLOP/byte materialized attention estimate, so low-intensity operations such as attention softmax remain memory bound even when large matrix multiplications on the same device are compute bound.

Systems Perspective 8.5: Peak FLOP/s vs. sustained performance

Peak TFLOP/s is an arithmetic ceiling rather than a prediction of sustained application performance. For an implementation that is strictly bandwidth bound, raising only the arithmetic ceiling does not improve the roofline bound. Mixed-precision training can both enable faster arithmetic paths and reduce traffic for tensors actually stored and transferred at lower precision. The realized gain depends on kernel support, conversion overhead, numerical requirements, and which resource is binding. Roofline-guided optimization therefore measures the implementation before choosing fusion, precision changes, or additional compute.

Batch size can influence arithmetic intensity and occupancy by changing matrix shapes and the amount of reuse available to a kernel. Where the transition from memory-bound to compute-bound execution falls depends on the operation's shape, its precision, and the hardware.

This analysis guides optimization strategy selection. For memory-bound operations, reducing data movement through kernel fusion, reduced precision, or algorithmic improvements like FlashAttention provides the largest gains. For compute-bound operations, increasing throughput through Tensor Cores and parallel execution matters more. The distinction is practical: the first case asks how to move fewer bytes, while the second asks how to keep more arithmetic units busy.

The trajectory vector in figure 8.1 illustrates how FlashAttention[16] can shift transformer attention toward, and for favorable shapes across, the 153 FLOP/byte ridge line onto the compute ceiling. By never materializing the full $S \times S$ attention matrix in HBM and instead processing tiles that fit in fast SRAM, FlashAttention reduces auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ and sharply reduces HBM traffic. The original work reports up to $3\times$ speedups on evaluated workloads (T. Dao et al. 2022), with gains depending on tensor shape, hardware, and baseline. The algorithm and the conditions under which it applies are examined in detail in section 8.7.4.

[16] **FlashAttention**: The core mechanism processes attention in small tiles that fit within the accelerator's fast on-chip SRAM, avoiding writes of the full intermediate $S \times S$ matrix to slower HBM. The exact HBM IO bound depends on tile size and SRAM capacity, but the practical effect is a large reduction in memory traffic and auxiliary activation storage. The original work reports workload-dependent speedups of up to $3\times$.

Arithmetic-intensity analysis is how the silicon contract (principle 4) is checked in training. The contract names the resource a workload wagers on; the analysis determines which physical resource actually constrains a particular implementation. Dense matrix multiplication reaches the compute-bound regime when tensor shapes provide sufficient data reuse, while standalone normalization and elementwise activation kernels remain memory bandwidth bound. FlashAttention exemplifies how tiling reduces off-chip data movement and raises effective arithmetic intensity, shifting memory-bound attention toward compute saturation.

Optimizing individual operations is necessary but insufficient. A perfectly tuned matrix multiplication achieves nothing if the accelerator sits idle waiting for the next batch of data. Isolated operation costs—dense matrix multiplication FLOPs, memory-bound activations, and optimizer state allocations—compound within the execution loop. Sustained throughput requires orchestrating these discrete operations into a balanced pipeline where upstream data delivery and downstream parameter updates prevent accelerator starvation.

8.5 Pipeline Architecture

A training step is not a single operation but a sequence of dependent stages—data must be loaded before computation can begin, forward passes must complete before backward passes start, and gradients must be computed before parameters can update. The speed of the slowest stage determines the speed of the entire system. Without pipelining, these stages serialize, idling the accelerator while the host prepares data or transfers batches across the bus.

While model execution graphs govern arithmetic on the accelerator, sustained training requires feeding those kernels across physical machine boundaries. Host CPUs ingest raw records from storage and apply preprocessing in host DRAM, while accelerators execute matrix multiplications in HBM. Data must cross the interconnect—typically a PCIe link whose bandwidth is an order of magnitude lower than device memory bandwidth. When upstream data preparation or bus transfer exceeds the duration of the forward and backward passes, the accelerator compute engines starve, reducing the hardware utilization factor η_{hw} in the training-time law.

Managing these physical boundaries divides the training system into three interconnected subsystems, each placing distinct demands on hardware resources. The data pipeline handles ingestion, transformation, and batching on the host; the training loop coordinates forward passes, backward passes, and parameter updates on the accelerator; and the evaluation pipeline periodically assesses model quality on held-out data. Figure 8.2 diagrams how these three subsystems decouple across the training lifecycle, staging processed batches into the core forward/backward training loop while streaming metrics to an asynchronous evaluation pipeline. The interfaces between them form critical synchronization boundaries where I/O latency, interconnect transfers, or synchronous validation pauses can throttle accelerator utilization. These boundaries partition the machine into three coordinated execution domains: the data pipeline supplies two of the three terms in the pipeline bottleneck model (equation 8.8)—host preprocessing and host-to-accelerator transfer—while the accelerator training loop supplies the third through matrix compute, and periodic evaluation claims shared execution time from that loop.

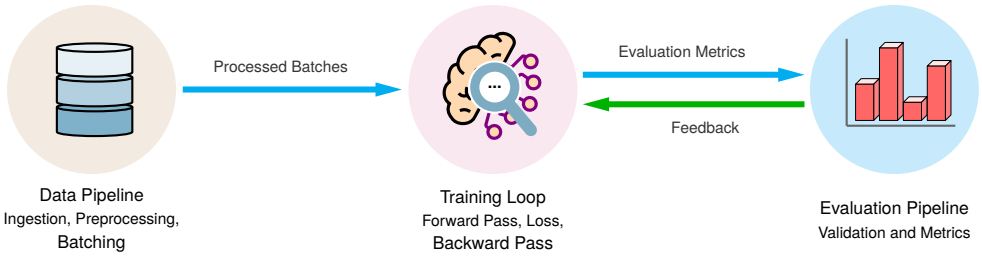


Figure 8.2: Training System Overview: The training lifecycle coordinates data preparation, core forward/backward computation, and validation evaluation. The interfaces between these three pipelines represent critical synchronization boundaries where I/O delays or evaluation pauses can throttle accelerator utilization.

8.5.1 Architectural overview

Across the three decoupled subsystems diagrammed in figure 8.2, each stage occupies a distinct hardware domain: host storage and CPU cores ingest and preprocess data, accelerator compute engines execute the model loop, and evaluation pipelines periodically preempt execution to calculate validation metrics. Any latency imbalance among them therefore becomes idle accelerator cycles. At the ingestion boundary, the data pipeline stages raw records from persistent storage into host memory, applies transformations such as augmentation and normalization (LeCun et al. 2012), and packs tensors into dense batches. These batches transfer over the host-accelerator interconnect (such as PCIe) into accelerator memory. At the validation boundary, the evaluation pipeline periodically runs held-out data through the model without computing gradients. Because validation occupies the same accelerator cores needed for training, its invocation frequency represents a direct trade-off between monitoring resolution and overall training throughput.

Within the accelerator, the training iteration forms a closed feedback loop governed by strict data and memory dependencies. Figure 8.3 traces this cyclic sequence across three stages—predict, gradients, and update—showing how intermediate activations and parameter gradients feed the optimizer before returning updated weights to the next iteration.

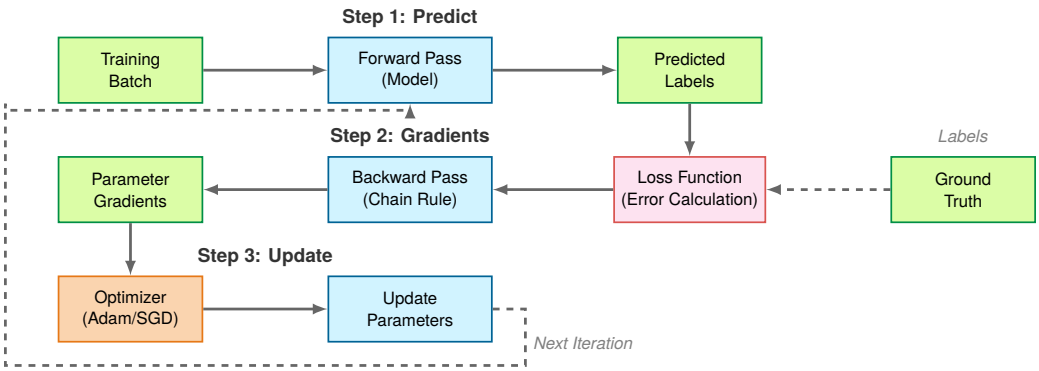


Figure 8.3: The Iterative Training Loop: Each iteration sequences forward prediction, gradient computation by backpropagation, and an optimizer update. Updated parameters return to the next forward pass while the next batch remains a separate input. Data dependencies and memory residency set the step's minimum execution time and memory floor.

While section 8.4 formalizes the numerical operators of this cycle, physical hardware execution is dictated by memory residency and data dependencies. During the forward pass, intermediate activation tensors must remain resident in accelerator HBM because the backward pass requires them to evaluate chain-rule derivatives. Once gradients are computed, the optimizer reads both the parameters and accumulated gradients to apply the update rule, producing new weights for the subsequent iteration. The memory capacity required to hold activations, weights, and optimizer states establishes the memory floor for the entire training step.

This execution loop repeats across batches and epochs until the optimization converges. Maximizing accelerator arithmetic efficiency requires overlapping these pipeline stages across the system hierarchy. Host CPU threads prefetch and stage batch $k+1$ into page-locked (*pinned*) host memory while the accelerator executes batch k, so the DMA engine can stream it across PCIe asynchronously (section 7.5.1.3 explains why pinning is required). This concurrent staging hides storage latency and bus transfer time behind matrix-multiplication compute kernels. When this concurrency breaks down—whether an I/O-bound data pipeline starves the device of input tensors or frequent evaluation stalls the update loop—the accelerator idles, degrading effective throughput.

8.5.2 Data pipeline

Keeping an accelerator's arithmetic units saturated requires streaming training examples from persistent storage into device memory at or above the model's consumption rate. Data engineering posed this as the feeding problem and priced the feeding tax that a slow path imposes; training is where that tax is paid on every step. Upstream data engineering prepares and partitions datasets on storage media, while the runtime data pipeline executes concurrently with model computation on the host CPU.

The data pipeline bridges raw storage and accelerator execution across the machine boundary. Figure 8.4 partitions this data movement across three physical zones—staging raw files from persistent storage into host memory, formatting and augmenting tensors across CPU worker threads, and streaming batched inputs over the interconnect to parallel GPU accelerators.

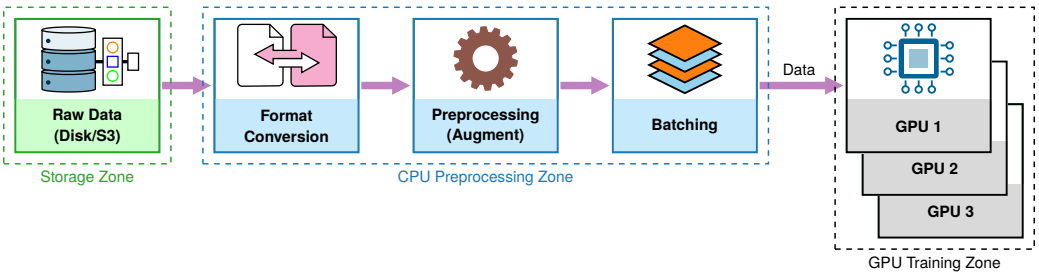


Figure 8.4: CPU-to-GPU Data Flow: Training data transitions through storage, CPU preprocessing, and GPU execution zones. Each stage represents a potential throughput gate; disk read bandwidth, transformation latency, and PCIe host-to-device transfers must collectively sustain the GPU consumption rate to keep accelerators at peak utilization.

In the storage zone, raw records reside on local NVMe drives or networked storage systems. In the CPU preprocessing zone, host worker threads decode raw bytes, apply transformations such as tokenization or data augmentation, and collate individual records into dense mini-batch tensors. In the GPU training zone, accelerators transfer those tensors across the PCIe bus and execute forward and backward passes. Because these three zones form a strict pipeline, the slowest stage dictates end-to-end training throughput. In the pipeline model developed below, the storage and CPU zones together set the preprocessing term, the PCIe crossing sets the transfer term, and the GPU zone sets the compute term.

8.5.2.1 Core components

Data pipeline throughput originates at the storage subsystem. Upstream layout decisions—such as sequential chunking in columnar or container formats (Parquet, TFRecord, Arrow)—dictate the physical I/O patterns presented to the operating system.

↩ How file format and storage layout set the data term of the iron law is established in section 4.8.2.

Storage throughput is bounded by the slower of two hardware constraints, $R_{\text{storage}} = \min(\text{BW}_{\text{disk}}, \text{BW}_{\text{network}})$, where BW_{disk} is the physical disk bandwidth and $\text{BW}_{\text{network}}$ represents the network bandwidth for distributed storage systems. In practice, training workloads rarely achieve this theoretical maximum because data shuffling can turn large sequential reads into smaller, less efficient accesses. Effective storage throughput can be modeled as $R_{\text{storage,eff}} = R_{\text{storage}} \times F_{\text{access}}$, where F_{access} depends on record size, file layout, caching, and storage hardware. When unbuffered random reads degrade F_{access}, storage bandwidth drops by an order of magnitude, transforming an I/O system that appears sufficient on paper into a primary training bottleneck.

8.5.2.2 Preprocessing

Once retrieved from storage, raw bytes require CPU transformation into model-ready tensors. While static feature extraction can run offline, dynamic transformations such as on-the-fly tokenization and image augmentation execute during training. Under ideal independent-worker scaling, the CPU worker pool delivers transformation capacity according to equation 8.7:

$$R_{\text{preprocessing}} = \frac{N_{\text{workers}}}{T_{\text{transform}}} \tag{8.7}$$

where N_{workers} parallel processing threads each perform transformations requiring $T_{\text{transform}}$ seconds per batch. Training architectures employ multiple workers to ensure transformation keeps pace with accelerator consumption rates—a single thread performing image augmentation at 30 ms per batch cannot feed an accelerator that computes a forward pass in 10 ms.

However, transforming records assumes that storage delivers raw bytes fast enough to keep worker queues filled. Under overlapped host staging, the achieved host preparation rate cannot exceed the slower of storage retrieval and CPU transformation. Dividing effective storage throughput $R_{\text{storage,eff}}$ by the raw bytes retrieved per batch yields the storage delivery rate in batches per second. When storage delivery lags behind worker demand, additional CPU threads sit idle waiting on disk I/O, capping host throughput regardless of N_{workers}. The complete host preparation rate is therefore bounded by the lesser of the storage delivery rate and the worker transformation capacity $N_{\text{workers}}/T_{\text{transform}}$.

Preprocessed data must then transfer across the host-accelerator interconnect before computation can begin. End-to-end training throughput is consequently constrained by the slowest of three sequential stages, as formalized in equation 8.8. In this three-stage pipeline model, the host preparation rate $R_{\text{preprocessing}}$ captures the joint bound of storage delivery and CPU worker transformation, while the remaining terms capture host-to-device bus transfer and device execution:

$$R_{\text{training}} = \min\left(R_{\text{preprocessing}}, \frac{\text{BW}_{\text{host}}}{D_{\text{vol}}^{(\text{batch})}}, \frac{R_{\text{sustained}}}{O_{\text{batch}}}\right) \tag{8.8}$$

where BW_{host} is the host-to-accelerator transfer bandwidth, $D_{\text{vol}}^{(\text{batch})}$ is the input bytes per batch transferred to the device, and O_{batch} is the forward-backward work per batch in FLOPs. Every term is therefore expressed in batches per second before the minimum is taken.

Example 8.1: GPT-2 language model data pipeline

Scenario: Quantifying storage access, CPU tokenization, and PCIe host-to-device transfers supplying a 8-V100 local node training GPT-2 XL. The global batch is split evenly across replicas, using the batch-4 mixed-precision and checkpointing tensor budget examined later in notebook 8.6; actual workspace headroom must still be verified. For this input-supply example, assume a device service interval of 500 ms per global batch, including local training, the optimizer update, and exposed synchronization. This interval is supplied independently of peak FLOP throughput.

Stage trace:

1. **Raw text storage**: The WebText corpus contributes about 40 GB of raw text. Sequential reads from NVMe SSD storage reach 7 GB/s. Under document sampling with access factor 0.1, random-access storage bandwidth drops to 0.70 GB/s, giving $T_{\text{read}} = D_{\text{vol}}/R_{\text{storage,eff}} \approx 0.1$ ms.
2. **Tokenization**: A BPE tokenizer (50,257 vocabulary) converts text into token IDs. For batch size $B = 32$ and sequence length $S = 1024$ (32.8K tokens total), a single CPU core processing 50K tokens/s takes $T_{\text{tokenize}} \approx 655.4$ ms per batch.
3. **Batching and padding**: Padding and packing generate an int64 tensor of shape 32 sequences $\times$ 1,024 tokens, producing 262.1 KB per batch.

4. **PCIe transfer**: Transferring the tensor across a PCIe Gen3 x16 bus (15.75 GB/s bandwidth) takes $T_{\text{transfer}} = 0.017$ ms, making bus transfer negligible.

Systems insight: Single-core CPU preparation ($T_{\text{prep}} \approx 655.4$ ms) exceeds the supplied accelerator service interval ($T_{\text{GPU}} \approx 500$ ms), so a serial pipeline pays both costs on every step and the device service stage occupies only about 43 percent of the serial step. Slower preparation increases the exposed input wait. Scaling to 8 parallel worker processes reduces the ideal average interval between prepared batches to $T_{\text{tokenize}}/N_{\text{workers}} \approx 81.9$ ms, while prefetching overlaps T_{prep} with T_{GPU} to keep the device stage supplied throughout ideal steady-state execution, delivering 64 global samples per second across 8 V100 GPUs. The service interval includes device work and synchronization; it is not a measurement of hardware busy time. Keeping inputs available does not establish useful FLOP utilization, which requires a separate useful-work count and measured step time. Parallel workers need not shorten the latency of an individual batch, and contention or imperfect overlap can reduce the modeled throughput.

This min-of-three relationship governs training pipeline design because system throughput equals bottleneck throughput. An accelerator with 312 TFLOP/s of compute capacity delivers zero useful work while waiting for data. Conversely, a perfectly optimized data pipeline provides no benefit if the accelerator is already compute-saturated. Balanced pipeline design aligns preprocessing capacity, transfer bandwidth, and compute throughput so that no single stage dominates iteration time. The GPT-2 workload demonstrates that CPU tokenization, rather than storage retrieval or bus transfer, forms the primary throughput barrier unless parallel workers and prefetching conceal host preparation latency.

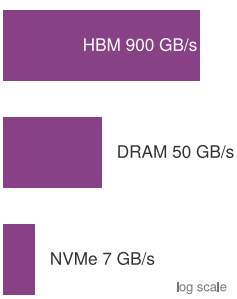
Data pipeline throughput is not the only flow that can starve the accelerator. Multi-GPU training adds a second stream of traffic: the gradient synchronization required after every step. Once synchronization time exceeds compute time, communication becomes the limiting factor. Section 8.8.2 quantifies that network wall at the point where training crosses the node boundary.

8.5.2.3 System implications

Comparing supply with demand requires both in the same unit. Grouping the two host-side terms of equation 8.8 into a single pipeline rate R_{pipeline}, in batches per second, and writing the accelerator's consumption rate as $R_{\text{accel}} = R_{\text{sustained}}/O_{\text{batch}}$, also in batches per second (where $R_{\text{sustained}}$ stays in FLOP/s), leaves a coupled system whose throughput is $\min(R_{\text{pipeline}}, R_{\text{accel}})$. When $R_{\text{pipeline}} < R_{\text{accel}}$, the accelerator sits idle waiting for data, and its busy-time fraction drops proportionally in this ideal supply model, as equation 8.9 shows. Busy time counts every moment the accelerator is executing, so it is an upper bound on, and distinct from, the model-FLOP utilization defined later in the chapter:

$$\text{Accelerator Utilization} = \min\left(1, \frac{R_{\text{pipeline}}}{R_{\text{accel}}}\right) \times 100\% \tag{8.9}$$

A ResNet-50 model on modern accelerator hardware can process 1,000 images per second, but if the data pipeline delivers only 200 images per second, accelerator utilization drops to 20 percent—the accelerator is idle 80 percent of the time. Upgrading to faster hardware does not help in this case; an accelerator capable of 2,000 images per second would achieve only 10 percent utilization with the same pipeline. Balanced system design aims to keep the accelerator supplied with data rather than waiting on an upstream stage.



Bandwidth steps up the storage to DRAM to HBM hierarchy.

8.5.2.4 Data flows

Training data traverses three memory tiers on its way from disk to accelerator, and the bandwidth gap between these tiers, spanning roughly two orders of magnitude, is the central challenge of data pipeline design. For equal byte volumes at each tier, the effective transfer rate is bounded by $\min(\text{BW}_{\text{storage}}, \text{BW}_{\text{system}}, \text{BW}_{\text{accelerator}})$, the same minimum as equation 8.8 applied to bandwidth tiers. When decoding, tokenization, or reuse changes the bytes handled per batch, each bandwidth must first be divided by its own per-batch byte volume so that the rates are comparable.

Local NVMe storage provides 7 GB/s, system memory delivers 50 GB/s, and accelerator HBM achieves 900 GB/s or higher. Each tier is roughly an order of magnitude faster than the one below it,

which means data that flows freely within accelerator memory creates a severe bottleneck when it must be fetched from disk. Stated in time rather than rate, the pipeline model says that when stages overlap correctly (fetching the next batch from storage while preprocessing the current one and transferring the previous one to the accelerator), the steady-state input service interval, denoted here by iteration time, is governed by the *maximum* of the stage times, $T_{\text{iteration}} = \max(T_{\text{fetch}}, T_{\text{process}}, T_{\text{transfer}})$, rather than their *sum*. This overlap is what prefetching seeks to achieve, with each tier operating concurrently on different batches. End-to-end training must also include accelerator compute and any exposed synchronization or evaluation time.

8.5.2.5 Practical architectures

An NVMe device rated at 7 GB/s would sustain 3.50 GB/s if a workload achieved half of peak. Small random reads and shuffling can reduce throughput further; the penalty depends on access size, queue depth, file layout, and caching.

To keep accelerators fed despite this bandwidth reduction, pipeline architectures maintain multiple data buffers simultaneously—prefetch buffers loading future batches, processing buffers holding data under transformation, and transfer buffers staging data for accelerator consumption. The total host memory required scales with the per-batch memory footprint M_{batch} according to equation 8.10:

$$M_{\text{required}} = (N_{\text{prefetch}} + N_{\text{processing}} + N_{\text{transfer}}) \times M_{\text{batch}} \tag{8.10}$$

To avoid starving the accelerator, the average preprocessing service time per batch, across all workers, must not exceed the compute time per batch, $T_{\text{preprocessing}} \leq T_{\text{compute}}$; this is the preprocessing term of equation 8.8 restated in time.

If preprocessing takes 40 ms per batch on one worker while the accelerator consumes one every 10 ms, ideal scaling requires at least four workers. Actual requirements depend on storage, CPU contention, and transformation cost; section 8.7.2 examines prefetching and parallel workers.

8.5.3 Forward pass

Prepared batches enter the forward pass after the data pipeline supplies their tensors. The operation counts in section 8.4.1.2 now become a scheduling problem: the runtime must launch kernels with enough parallel work, stage their operands, and keep the values required by backward execution.

8.5.3.1 Compute operations

A dense layer's FLOP count is already fixed by its input width, output width, and batch size. Its execution time also depends on how that work fills the accelerator. A layer with dimensions of 512×1024 at batch 64 supplies 33.6 million MACs, or 67.1 MFLOP; the kernel must divide this work among execution units without leaving a long, mostly empty final wave.

Accelerators execute these parallel workloads by grouping execution threads into fixed-size units: warps[17] of 32 threads on NVIDIA architectures, or wavefronts of 64 threads on AMD architectures. Hardware execution efficiency depends on aligning tensor dimensions and tile shapes with the thread-block geometry and the operand matrix dimensions required by specialized units like Tensor Cores.

[17] **Warp**: From textile weaving, where a "warp" is the set of threads held taut on a loom while a shuttle weaves across them. NVIDIA adopted the term because GPU threads within a warp execute the same instruction in lockstep, analogous to parallel threads moving together on the loom. An NVIDIA warp contains 32 threads.

> **Systems Perspective 8.6: Wave quantization and tail effects**
>
> A common mistake in ML systems is ignoring the fixed execution granularity of GPU kernels. GPU execution is quantized into waves of work, but the relevant unit is the kernel's thread or tile mapping, not necessarily one sample per thread.
>
> Warps make the granularity concrete. In the illustrative special case where one independent work item maps to one thread, 32 items fill one warp while 33 items require a second, mostly empty warp. Neural-network kernels usually map each sample to many threads, so batch size alone does not determine this utilization.
>
> On a large GPU such as the NVIDIA H100 with 132 Streaming Multiprocessors (SMs), where each SM schedules groups of warps, the hardware processes thousands of threads in a single

wave. If the total workload slightly exceeds a wave boundary (for example, 1.01 waves), the hardware must execute a second, nearly empty wave to completion before the next task begins. Table 8.7 quantifies that one-item-per-thread illustration. It is not a general batch-size performance model.

Table 8.7: Illustrative Warp-Tail Arithmetic: Under a one-work-item-per-thread mapping, counts just above a 32-thread boundary launch a partially filled warp, so 33 items use two warps while 64 fill the same two warps. Actual training kernels map tensors to threads and tiles in implementation-specific ways, often assigning many threads to each sample, so these values cannot be inferred from batch size alone. Profiling must identify the kernel's real mapping and wave boundaries.

Independent Work Items	Warps Needed	Lane Utilization	Relative Time
32	1	100%	1×
33	2	51.6%	~2×
64	2	100%	1×
65	3	67.7%	~1.5×

Systems insight: The table teaches the shape of the penalty, not its size for any real kernel. The actionable rule is to choose batch, sequence, and hidden dimensions that are multiples of the kernel's tile size and that divide into whole waves across the SMs, then benchmark the neighboring candidates. Profiling reveals partially filled warps or waves; batch size alone cannot.

Beyond dense matrix multiplication, distinct neural architectures introduce specialized computational patterns. Convolutional networks, for instance, slide spatial filters across multichannel input tensors. Consider an input tensor of dimensions $64 \times 224 \times 224 \times 3$ (batch size $\times$ height $\times$ width $\times$ channels) processed by 7×7 kernels. Each position requires 147 multiply-accumulate operations, and with 64 filters operating across 218×218 spatial dimensions, the arithmetic work scales into millions of multiply-accumulate operations per image. Accelerators execute these spatial convolutions by either lowering them into dense GEMMs through column unfolding (`im2col`) or applying Winograd minimal filtering, which reduces multiplications at the cost of extra additions and transform workspace.

The other operator families impose different scheduling constraints. Attention alternates batched matrix products with row-wise reductions, while normalization requires statistics before normalized outputs can be produced. The arithmetic-intensity analysis in section 8.4.4 explains their traffic cost; the runtime must additionally respect these reduction dependencies and choose which intermediates can remain inside a fused kernel.

Optimized operator libraries such as cuDNN (Chetlur et al. 2014) implement multiple computational algorithms for each layer type, dynamically selecting among them based on tensor dimensions, scratchpad memory limits, and hardware capabilities. For example, cuDNN chooses between direct convolution, GEMM-based lowering, fast Fourier transform (FFT) convolution, and Winograd transforms by evaluating the trade-off between workspace memory overhead and arithmetic operations (NVIDIA Corporation 2023). These hardware utilization patterns reinforce the coupling between batch size and hardware efficiency established in section 8.4.2.1: larger batch sizes saturate accelerator compute units and amortize memory transfers, but increase intermediate activation storage.

8.5.3.2 Memory management

Memory management binds during the forward pass, when intermediate activations must be retained in high-bandwidth memory for gradient computation during backpropagation. The definition of training systems priced the static state of a 7B-parameter model at 112 GB before activations. Adding retained activations and comparing the total against a 24 GB accelerator shows how quickly memory capacity, not arithmetic, decides feasibility.

Napkin Math 8.5: Estimating VRAM requirements

Problem: Will a 7B-parameter model fit on a 24 GB GPU for training?

Given: 7B parameters, FP16 weights and gradients, FP32 master weights and Adam states, and 24 GB GPU memory.

Math:

1. **Weights (FP16)**: 7B $\times$ 2 bytes = 14 GB.
2. **Gradients (FP16)**: Same size as weights = 14 GB.
3. **FP32 training state**: Master weights require 28 GB; Adam momentum and variance require another 56 GB.
4. **Subtotal (before activations)**: 14 GB + 14 GB + 28 GB + 56 GB = 112 GB, already exceeding a 24 GB GPU.
5. **Activations**: A simplified transformer estimate is $B \times S \times d \times N_L \times s_{\text{elem}}$ times an activation factor for retained attention, multilayer perceptron, and normalization intermediates. With $B = 1$, $S = 2048$, $d = 4096$, 32 layers, and an activation factor of about 57×, retained activations add 30.6 GB.

Systems insight: Full-parameter training requires partitioning or offloading state through systems such as fully sharded data parallel (FSDP) or ZeRO, which section 8.8.1.1 introduces. Storing frozen base weights in a low-bit format reduces memory for fine-tuning methods that train only a small set of added parameters, but it does not reproduce this full-parameter mixed-precision recipe.

Training scope changes which terms of the memory ledger are populated. Let P count all resident model parameters, including any added head or adapters. Inference retains their weights but has $M_{\text{gradients}} = M_{\text{optimizer}} = 0$, because it neither computes parameter gradients nor updates weights. Full fine-tuning allocates gradients and optimizer state for all P parameters, following the chosen precision recipe. Head-only adaptation keeps the frozen backbone resident but allocates gradients, master weights when used, and optimizer moments only for the trainable head. Adapters, small trainable modules inserted into a frozen network, similarly restrict these update buffers to the added parameters. Their weight-memory term still includes both the frozen base and the adapters; storing fewer trainable parameters does not remove the base weights.

Saved activation memory follows the backward path rather than the trainable parameter count. When a frozen feature extractor runs without gradient recording and supplies its output as a constant to a trainable head, the head retains that boundary input and its own required backward values; the backbone has no saved backward activations. Inference also avoids saved backward state, although live forward intermediates and workspaces still consume memory. Adapters distributed through the network create a different topology. Gradients must traverse intervening frozen operations to reach earlier adapters, so those operations may still require saved tensors or recomputation. Setting `requires_grad=False` on base weights suppresses their parameter gradients, but disabling recording across that path would also cut the gradients the adapters need.

↩ Gradient-recording and detachment controls for frozen feature extractors are established in section 7.4.2.1.

The comparison therefore separates a residency decision from a differentiation decision. Subtracting $M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}}$ from device capacity gives the space available for activations and runtime overhead. A head-only configuration may fit where full fine-tuning does not, provided it meets the task's quality target; an adapter configuration must still budget the activations on its backward path. If those activations exceed the remaining space, checkpointing or a smaller micro-batch may still be necessary despite the small optimizer state. For a fixed training scope, only activation memory scales approximately linearly with batch size (equation 8.5); model parameters and optimizer state remain fixed. The practical complexity lies in how these costs interact across layers.

Consider ResNet-50 processing images at 224 $\times$ 224 resolution with a batch size of 32. The initial convolutional layer produces activation maps of dimension 112 $\times$ 112 $\times$ 64; per image at single-precision (4 bytes), this requires approximately 3.2 MB. For an illustrative capacity budget, assume retained forward activations total 8 GB, the backward pass adds another 4 GB of activation

working set (a supplied scenario assumption, distinct from stored parameter gradients), and model parameters consume 102.4 MB. This 12.1 GB subtotal excludes parameter gradients, optimizer state, and additional runtime workspace; those terms must be added before deciding whether training fits.

Under the same linear-scaling assumption, doubling the batch size to 64 doubles activation storage: forward activations reach 16 GB and the backward activation working set reaches 8 GB, raising this subtotal to 24.1 GB. In vision models with large spatial resolutions, activations dominate the memory footprint while parameter storage (102.4 MB) is negligible. For large language models, the balance inverts: GPT-3 (175B parameters) requires approximately 700 GB just for parameters in FP32 (350 GB in FP16), exceeding single-accelerator capacity before allocating any activation memory or optimizer state.

Because accelerator memory (40 GB–80 GB on contemporary hardware) must hold activations, parameters, gradients, and optimizer states simultaneously, memory capacity rather than compute throughput frequently caps the feasible batch size. Two system-level techniques relieve this capacity bottleneck: activation checkpointing discards intermediate activations during the forward pass and recomputes them during the backward pass, trading redundant arithmetic for reduced storage; and mixed-precision training halves storage per value for tensors represented in 16-bit formats instead of FP32. Master weights and optimizer state may remain in FP32, so the saving must be calculated for each term of the memory ledger. Both techniques are analyzed in detail in section 8.7.

During backpropagation (section 8.4.3), the backward pass traverses the network in reverse order, consuming retained forward activations to compute parameter and activation gradients at roughly twice the arithmetic cost of the forward pass. Memory requirements shift dynamically across layers. In ResNet-50, mid-network layers expand to 256 filters from the initial 64, where reduced spatial map dimensions partly compensate for higher channel depth. Peak memory footprint reflects retained forward activations, active gradient buffers, temporary workspaces, and the runtime allocation schedule rather than the width of a single layer alone.

8.5.4 Parameter updates and optimizers

Once backward execution populates the gradient buffers, the optimizer consumes them and updates the persistent state defined in algorithm 8.1. The important pipeline boundary is the weight mutation: the next forward pass must observe the completed update. Listing 8.3 shows that boundary at the framework interface.

Listing 8.3: Parameter Update: Computes gradients and applies an optimizer step to model parameters. Repeating this cycle seeks to minimize the training objective, although improvement is not guaranteed on every step or epoch.

```
loss.backward()  # Compute gradients
optimizer.step()  # Update parameters
```

The optimizer's memory traffic is exposed on this dependency path. The FP32 Adam update already priced at 28 bytes per parameter in section 8.4.2.4 must finish before the next forward pass reads those weights. Fusing moment updates and weight writes can shorten this interval by reducing passes over HBM, even when the forward and backward matrix products already use the arithmetic units efficiently.

8.5.4.1 Optimizer memory in the training loop

The same optimizer tensors occupy capacity between updates. Reducing their footprint can therefore make a larger micro-batch feasible even if the optimizer is not the longest phase of the step.

At billion-parameter scale, optimizer state becomes a first-order, batch-independent term in the memory budget. As quantified in the GPT-2 worked example (section 8.4.2.3), a 1.5B model requires 12 GB for Adam optimizer state alone in FP32, in addition to parameters and gradients, before accounting for activations. This challenge has motivated memory-efficient optimizer variants: Adafactor factorizes second-moment state (Shazeer and Stern 2018), 8-bit optimizers quantize optimizer statistics (Dettmers et al. 2022), and GaLore computes updates in a low-rank space (Zhao et al. 2024).

To attribute a memory saving to the optimizer, hold the model, batch size, precision, and activation policy fixed and compare the complete memory ledger. Low-rank projection buffers and other

temporary state belong in that ledger. A smaller optimizer-state term does not guarantee that the full training recipe fits, nor that it reaches the same validation quality in less time.

8.5.4.2 Batch size and parameter updates

Increasing batch size improves accelerator utilization by raising the arithmetic intensity of matrix multiplications and reducing synchronization frequency. However, modifying batch size alters optimization dynamics. Scaling batch size without retuning the learning-rate schedule can degrade convergence, creating an optimization gap where hardware efficiency comes at the expense of model quality. Figure 8.5 isolates this dynamic across synthetic loss trajectories: holding the learning rate fixed while expanding from batch 32 to batch 256 leaves an acute optimization gap (dashed silver curve), whereas linearly scaling the learning rate restores convergence along the baseline path (green curve).

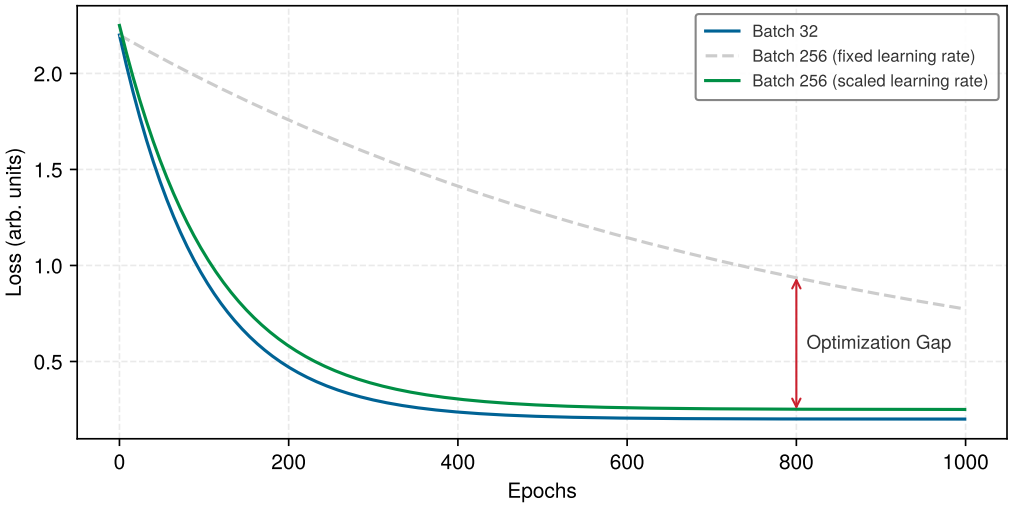


Figure 8.5: Illustrative Linear-Scaling Scenario: Synthetic training-loss curves (arbitrary units) compare a batch-32 baseline with batch 256 under a fixed or linearly scaled learning rate. The curves illustrate why increasing batch size may require retuning; they are not measured traces or a guarantee that linear scaling preserves convergence.

For a fixed token budget, doubling the global batch size halves the number of parameter updates per epoch. To traverse a comparable distance in parameter space with fewer updates, the linear scaling rule named in the batch-processing definition scales the learning rate proportionally with batch size ($\eta_{\text{new}} = \eta_{\text{base}} \times B_{\text{new}}/B_{\text{base}}$). Although effective across moderate batch ranges when combined with learning-rate warmup, linear scaling does not guarantee equivalent convergence or generalization. Beyond a critical batch size, gradient noise is largely eliminated, and further batch expansion yields diminishing returns per training sample while escalating memory pressure.

Beyond convergence dynamics, batch size dictates communication frequency in distributed systems. A larger global batch reduces the number of optimizer steps and gradient synchronizations per epoch. Because the communication payload of dense all-reduce synchronizations depends on parameter count rather than batch size, larger batches amortize interconnect latency over more compute FLOPs. In distributed settings, local micro-batch size and global batch size jointly determine the maximum degree of data parallelism. When memory capacity constrains the local micro-batch, gradient accumulation (section 8.7.5) decouples the effective batch size from per-device activation memory, accumulating gradients across multiple forward-backward passes before triggering an optimizer step.

Decomposing the training pipeline exposes the structural mechanics of data loading, forward execution, backpropagation, and parameter updates, while the mathematical foundations in section 8.4 quantify the FLOPs, memory footprint, and bandwidth demanded by each phase. Yet knowing how stages execute does not reveal where a specific training workload stalls. A pipeline

is only as fast as its slowest stage. In an overlapped pipeline where data loading takes 120 ms and computation takes 100 ms, a 20 percent faster kernel saves nothing, because the step still takes 120 ms. Before applying optimization techniques, systems engineers must profile the execution timeline to determine whether compute, memory bandwidth, storage I/O, or network synchronization binds performance.

8.6 Identifying Bottlenecks

Blueprint knowledge is not diagnosis. Knowing that attention operations consume 50 percent of FLOPs and data loading takes 25 percent of wall-clock time does not reveal which constraint to attack first; that depends on which resource is actually saturated during execution.

The diagnostic methodology that transforms blueprint knowledge into actionable optimization decisions begins with a meaningful measure of training efficiency. Raw accelerator utilization percentages can be misleading because an accelerator may remain active while executing recomputation, padding, or other work outside the model-FLOP accounting. Model FLOPs utilization (MFU)[18] supplies that comparison.

[18] **Model FLOPs utilization (MFU)**: The PaLM paper (Chowdhery et al. 2022) defined MFU as observed throughput divided by the theoretical maximum throughput at peak FLOPs. Its 540-billion-parameter run reported 46.2 percent MFU on 6,144 TPU v4 chips. The complement is uncredited by the model-FLOP numerator, but MFU alone does not identify how much came from memory traffic, communication, pipeline bubbles, recomputation, or other work.

> Definition 8.4: Model FLOPs utilization (MFU)
>
> ***Model FLOPs utilization (MFU)*** is the efficiency metric $\text{MFU} = O_{\text{model}}/(R_{\text{peak}} \cdot T_{\text{step}})$, where O_{model} is the useful per-step model FLOP count (forward plus backward, excluding rematerialization) and T_{step} is the measured wall-clock time per training step, expressing what fraction of peak hardware throughput is doing useful model computation.
>
> 1. Significance: MFU makes the effective-throughput term in the iron law concrete. For a 7B-parameter transformer processing 1,024 tokens per step on an A100 (312 TFLOP/s FP16/BF16 Tensor Core peak), a 1.2-second step gives model FLOPs divided by the available peak-rate FLOP budget of approximately 0.11 (11.5 percent). The remaining 88.5 percent is not credited as model FLOPs and can reflect memory traffic, communication, bubbles, non-model operations, or idle time. Diagnosing that gap requires a profile rather than a universal threshold.
> 2. Distinction: Hardware utilization reports whether device engines are active, whereas MFU credits the analytical model-FLOP estimate used in its numerator. It can therefore exclude recomputation and padding even when those operations keep the device busy. Comparisons across accelerators remain sensitive to the selected peak-throughput specification and FLOP-counting convention.
> 3. Common pitfall: A frequent misconception is that 100 percent MFU is a realistic sustained target. Memory traffic, communication, and non-matrix work keep end-to-end runs below peak. Reported values depend on the model, precision, batch shape, hardware, and MFU definition, and can improve with FlashAttention and careful tuning.

Three distinct metrics are commonly referred to as "utilization" in training systems, and each addresses a different operational question:

1. The hardware efficiency factor η_{hw} in the training-time law (equation 8.1) represents the end-to-end fraction of theoretical peak compute converted into useful model arithmetic across an entire run.
2. MFU measures this same ratio over an individual training step, serving as the direct empirical estimator for η_{hw}.
3. In contrast, the accelerator busy-time fraction of equation 8.9—the metric reported by standard operating-system and vendor monitoring utilities such as `nvidia-smi`—merely indicates whether execution cores are active, without distinguishing useful matrix products from memory-stall cycles, padding overhead, or rematerialization.

Consequently, busy-time utilization serves only as an upper bound on MFU. A data-starved pipeline exhibits a low busy-time fraction; a well-fed but memory-stalled workload exhibits near-100 percent busy-time utilization alongside low MFU.

Training bottlenecks can be investigated through three categories aligned with the D·A·M taxonomy (Data, Algorithm, Machine). Table 8.8 connects each axis to a common training bottleneck, its observable symptoms, and relevant optimization techniques. The axis names where the remedy usually lives rather than what the bottleneck is. Compute-bound and memory-bound execution are both mismatches between an algorithm and a machine, as the ridge point showed, but a compute-bound kernel is most often relieved by asking for less or cheaper arithmetic (Algorithm), a memory-bound kernel by changing how the machine stages and moves bytes (Machine), and an input stall by repairing the data path (Data). These are diagnostic starting points, not exclusive mappings.

↪ The full D·A·M diagnostic framework, with its troubleshooting matrix and scorecard, is developed in Appendix A.

Table 8.8: D·A·M Taxonomy Applied to Training Bottlenecks: Each row associates a D·A·M axis with one common bottleneck and its characteristic symptoms. Profiling reveals the workload's limiting constraint and guides practitioners to an appropriate optimization technique.

D·A·M Axis	Bottleneck	Symptoms	Primary Solutions
Algorithm	Compute-bound	High arithmetic-unit activity; additional bandwidth does not improve throughput; arithmetic throughput is limiting	Reduced precision, more efficient algorithms, faster compute hardware
Machine	Memory-bound	High memory-bandwidth use relative to arithmetic throughput; kernels stall on data movement	Kernel fusion, memory-efficient attention, reduced precision formats
Data	Data-bound	Periodic accelerator utilization drops to near-zero; CPU fully busy during gaps; pipeline cannot feed GPU fast enough	Prefetching, pipeline overlap, faster storage, DataLoader parallelism

Among these three regimes, data-bound stalls are the most frequently misdiagnosed because they manifest on the accelerator as idle gaps rather than saturated execution units. When host-side ingestion, decoding, or preprocessing cannot sustain the throughput demanded by device memory buses, the accelerator starves.

Example 8.2: The GIL-locked GPU

Scenario: An engineering team builds a PyTorch data-loading pipeline using standard Python `threading` for CPU-bound image preprocessing that holds the global interpreter lock (GIL).

Diagnosis: The GIL serializes these Python-level preprocessing threads, making host preparation time ($T_{\text{prep}} \approx 250$ ms) vastly exceed GPU execution time ($T_{\text{GPU}} \approx 12$ ms), reducing the accelerator busy-time fraction to about 4.6 percent in this serial preparation-and-compute model. Here the fraction measures time spent computing, not arithmetic efficiency relative to peak throughput.

Systems lesson: Python CPU work that holds the GIL can serialize threads and starve GPUs; native operations may release it. Process workers (`num_workers > 0`) avoid interpreter serialization, while the NVIDIA Data Loading Library (DALI) can move eligible work off the host.

Profiling tools reveal which bottleneck dominates a given workload. In the execution timeline captured by figure 8.6, wide white intervals separate brief GPU stream activity bursts (Streams #13 through #25), timestamping periods where accelerator compute engines idle while CPU-side Python threads assemble the next batch.

Four tools integrated into machine learning frameworks provide detailed bottleneck analysis. Table 8.9 separates framework-level timeline tools from GPU-level execution tools, because each exposes a different failure signature.



Figure 8.6: Data-Bound Profiler Trace: In this timeline trace, isolated GPU compute blocks are separated by prominent white intervals. These idle gaps mark periods where accelerator streams stall waiting for CPU-side batch preparation and host-to-device transfers, providing a characteristic signature of an input-bound pipeline.

Table 8.9: Profiling Tools for Training Bottlenecks: Framework profilers expose operation timelines and input-pipeline stalls, while NVIDIA Nsight tools expose system-level GPU execution and kernel-level memory behavior.

Tool	Scope	Best signal
PyTorch Profiler (`torch.profiler`)	Framework-level operation trace	Time spent in each operation, memory allocation patterns, and GPU kernel execution
TensorFlow Profiler	Framework-level training timeline	Input pipeline bottlenecks and device placement
NVIDIA Nsight Systems	System-level GPU trace	Kernel execution, memory transfers, and synchronization points
NVIDIA Nsight Compute	Kernel-level GPU analysis	Arithmetic intensity, memory throughput, and occupancy

A disciplined profiling workflow proceeds top-down from system timelines to kernel-level performance counters, mapping observable profiler traces to the D·A·M taxonomy in table 8.8. First, timeline visualization locates idle intervals between kernel invocations. Correlating these gaps with batch preparation, transfers, host dispatch, and synchronization establishes what prevents the next kernel from running. When input starvation is confirmed, remedies include asynchronous prefetching, multi-process workers, or offloading preprocessing to the accelerator to overlap data movement with compute. Second, when kernels execute contiguously without host-induced gaps, memory throughput relative to peak bandwidth distinguishes memory-bound from compute-bound execution. If HBM traffic saturates while arithmetic units idle, the workload is memory-bound; kernel fusion and memory-efficient attention relieve this constraint by retaining intermediate tensors in on-chip SRAM rather than writing them back to global memory. Finally, if tensor cores operate near theoretical capacity and memory buses have spare bandwidth, the workload is compute-bound, where throughput gains require lower numerical precision—such as BF16 or FP8—to increase peak arithmetic throughput.

8.7 Pipeline Optimizations

Profiling traces identify which resource saturates during execution, while the D·A·M taxonomy isolates the governing physical constraint: data movement, arithmetic throughput, or memory capacity. Closing the gap between realized and theoretical throughput requires matching targeted systems interventions to each identified failure mode.

Even well-designed pipeline architectures rarely reach hardware peak throughput without targeted optimization. To illustrate the scale of the gap, applying a model-FLOP utilization of 30 percent to 50 percent to an A100 peak of 312 TFLOP/s gives 93.6 TFLOP/s–156 TFLOP/s of model-FLOP throughput. These are derived scenario values, not measurements reported for one A100 workload. Published large-model systems report similar gaps between peak and realized throughput

(Narayanan et al. 2021; Chowdhery et al. 2022). Systems such as SuperNeurons show one version of this problem: memory-management bottlenecks can prevent larger networks from training efficiently unless the runtime actively optimizes activation storage and transfer (L. Wang et al. 2018). Table 8.10 extends the D·A·M-based bottleneck classification from table 8.8 by mapping each bottleneck to the specific optimization technique that addresses it.

Table 8.10: Optimization Technique Roadmap: Each primary bottleneck category has targeted solutions that address specific performance constraints, matching techniques to profiling results for systematic optimization.

Bottleneck	Primary Solution(s)
Data Movement Latency	Prefetching & Pipeline Overlapping
Compute Throughput	Mixed-Precision Training
Memory Capacity	Gradient Accumulation & Activation Checkpointing
Memory Bandwidth (Attn.)	FlashAttention (IO-aware tiling)

Although the governing constraint shifts with system scale—a 100 GB model faces capacity and interconnect limits that never bind a 1 GB model—identifying and mitigating each bottleneck follows consistent physical principles. Data movement latency emerges when host storage I/O and PCIe transfers cannot stage batches fast enough to keep accelerator compute units saturated. Computational throughput falls below peak hardware rates when operations execute in FP32 instead of using lower-precision Tensor Cores, or when non-fused kernels repeatedly transfer intermediate activations across the memory bus. Memory capacity constraints bind execution because accelerator HBM must simultaneously hold model parameters, optimizer states, gradients, and forward activations. Exhausting this memory caps both model depth and per-device batch size.

Because system resources are tightly coupled, relieving one constraint often exposes another. This is the conservation-of-complexity heuristic from the opening of Part I, which holds that complexity tends to move among Data, Algorithm, and Machine rather than disappear. For example, resolving host I/O stalls by aggressively prefetching batches consumes host memory; separately staged device queues also consume accelerator memory. Conversely, shrinking per-device batch sizes to fit within HBM limits lowers the arithmetic intensity of matrix multiplications, shifting execution from compute-bound matrix math into memory-bandwidth stalls. Consider a representative GPT-2 profile where attention consumes 50 percent of step time, data loading consumes 25 percent, and other compute-bound operations consume 25 percent. This time breakdown identifies where to investigate first. If counters confirm that attention is bandwidth limited, IO-aware attention tiling (FlashAttention) targets that constraint; asynchronous prefetching hides data ingestion behind computation, and mixed precision accelerates eligible tensor operations while reducing storage for activations retained in the lower-precision format. The optimization challenge is to diagnose the governing bottleneck from execution profiles and apply targeted interventions without creating a more severe constraint elsewhere.

8.7.1 Systematic optimization framework

Because accelerator compute engines, memory subsystems, and host interconnects operate under tight physical coupling, optimizing without empirical telemetry risks shifting the critical path or squandering engineering effort on non-binding resources. A principled systems methodology follows three coupled stages: profiling to expose the binding physical constraint, selecting mechanisms specifically engineered for that resource limit, and composing interventions so that secondary resource demands do not collide.

The profiling phase uses instrumentation tools such as PyTorch Profiler, TensorFlow Profiler, or NVIDIA Nsight Systems to record timestamps and hardware counters across each iteration. Traces capture kernel execution times on accelerator execution units, DMA transfers across PCIe or NVLink buses, and host CPU thread scheduling. Quantifying elapsed time isolates which subsystem dictates the critical path. For example, a trace revealing 40 percent of step time spent in data loading, 35 percent in computation, and 25 percent in memory operations supports a host-side data-starvation diagnosis when the loading intervals lie on the critical path and coincide with accelerator input waits. Under these conditions, accelerating matrix multiply kernels yields negligible wall-clock improvement until the data delivery pipeline is resolved.

The selection phase maps the identified bottleneck to an architectural mechanism targeting that physical constraint. When profiling shows input-bound execution where the accelerator idles waiting for batches, asynchronous prefetching and double-buffering over PCIe overlap host data ingestion with device compute. When kernel runtimes are memory-bandwidth-bound—dominated by round-trips to HBM rather than arithmetic—kernel fusion and tiling algorithms such as FlashAttention stage intermediate activations in on-chip SRAM. When memory capacity bounds the model, activation checkpointing discards and recomputes activations during the backward pass, or gradient accumulation divides batches across multiple sequential sub-steps to limit peak allocation. Finally, when streaming multiprocessors saturate with dense matrix math, low-precision numeric formats (such as FP16 or BF16) exploit specialized tensor cores to multiply peak matrix throughput (several-fold over FP32 CUDA cores; table 8.11) while halving memory traffic.

The composition phase evaluates how multiple interventions interact across shared hardware resources. Because system resources are tightly coupled, an optimization that relieves one bottleneck often imposes demands elsewhere. Asynchronous prefetching hides data movement latency behind computation, but maintaining double-buffered batch queues consumes pinned host memory and device HBM. In contrast, synergistic combinations compound performance gains: pairing 16-bit mixed precision with activation checkpointing curtails memory usage sufficiently to allow larger per-device batch sizes, restoring high arithmetic intensity to underutilized tensor cores. Successful composition therefore requires verifying that secondary resource overheads do not trigger a new binding constraint.

Figure 8.7 formalizes this diagnostic playbook into a triage flowchart, using low accelerator activity and high memory allocation as signals for further investigation. Idle gaps require tracing input preparation, host work, and synchronization; capacity pressure motivates memory-saving techniques. When neither signal explains a missed target, kernel profiling distinguishes bandwidth, arithmetic, and launch limits.

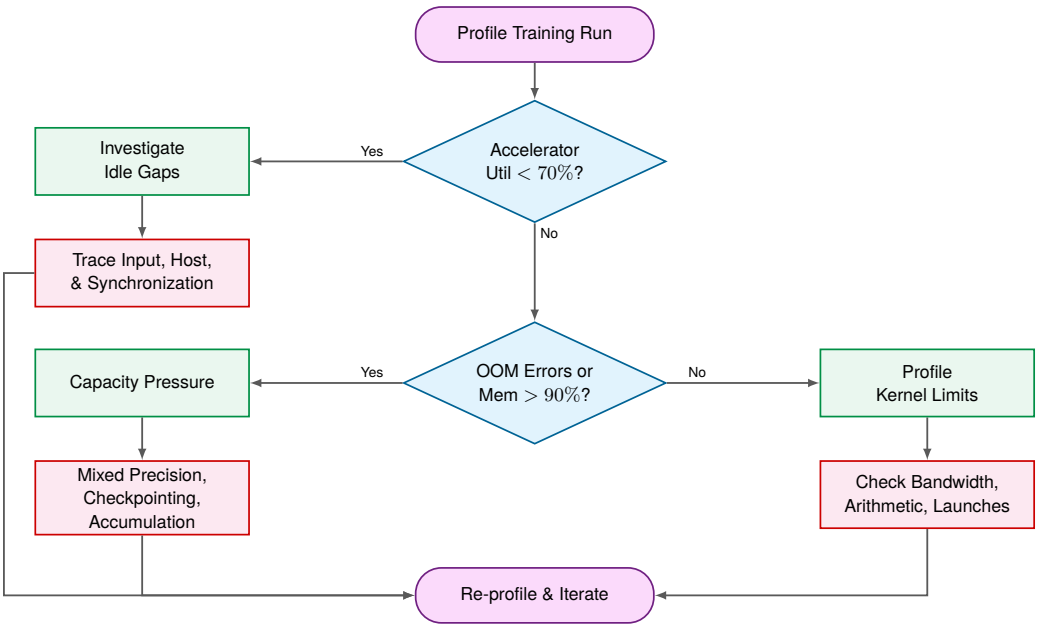


Figure 8.7: Training Optimization Decision Flowchart: A heuristic path from profiling signals to candidate optimizations. Begin with accelerator activity and memory pressure, then investigate the limiting dependency or kernel resource before selecting an intervention. The 70 percent utilization and 90 percent memory thresholds are illustrative triage values rather than universal definitions of data-, memory-, or compute-bound execution.

Because eliminating one bottleneck exposes the next constraint along the execution path, systems optimization is an iterative process governed by Amdahl's law. Resolving input starvation through asynchronous prefetching, for instance, drives accelerator utilization upward until memory bandwidth or compute saturation limits step time. Each intervention requires re-profiling to identify the

newly emerged critical path, continuing the cycle until accelerator execution units operate near peak theoretical throughput or application performance targets are achieved.

8.7.2 Data prefetching and overlapping

Prefetching and overlapping techniques illustrate the systematic framework in action, targeting data movement latency bottlenecks by coordinating data transfer with computation. This optimization proves most effective when profiling reveals that computational units remain idle while waiting for data transfers to complete.

Training machine learning models involves significant data movement between storage, memory, and computational units. The data pipeline consists of sequential transfers: from disk storage to CPU memory, CPU memory to GPU memory, and through the GPU processing units. In ML training, a "Read" operation is rarely a simple disk fetch: for image workloads it encompasses JPEG decoding, random crops, and color jitter applied on CPU cores before the tensor is ready to transfer; for language workloads it encompasses tokenization, subword encoding, and sequence padding. The Gantt schedule sequenced in figure 8.8 isolates the serial bottleneck of unbuffered ingestion: the GPU Train row remains completely idle during file-open intervals (Open 1, Open 2) and read stages (Read 1, Read 2), stretching epoch duration out to 01:35 because compute steps cannot begin until host-side decoding and transfer finish.

Prefetching addresses these inefficiencies by loading data into memory before its scheduled computation time. During the processing of the current batch, framework data pipelines such as `tf.data` load and prepare subsequent batches, maintaining a consistent supply of ready data (Murray et al. 2021).

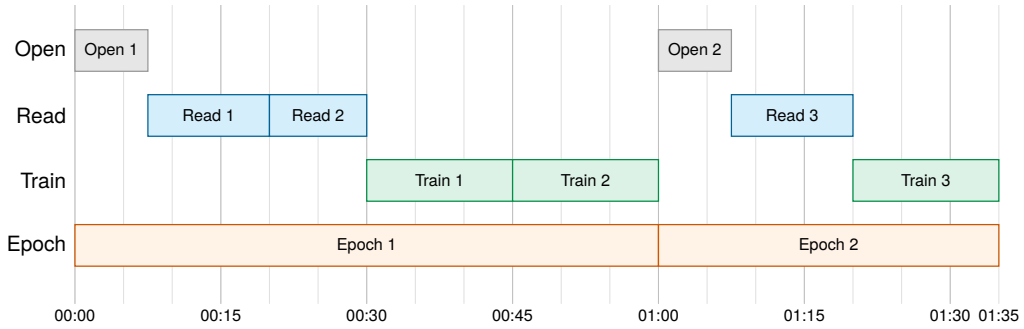


Figure 8.8: Illustrative Sequential Data Fetching: Under the assumed stage durations, file-open, read, and train operations execute serially while the GPU remains idle during file operations. The schedule spans approximately 95 min and provides a schematic baseline for the overlap comparison.

Overlapping extends prefetching by coordinating multiple pipeline stages to execute concurrently. As the system processes the current batch on the accelerator, background host threads load and preprocess subsequent batches. Aligning the serial schedule of figure 8.8 against the overlapped timeline in figure 8.9 demonstrates this latency hiding: concurrent staging of Open 2 and Read 3 behind active execution eliminates idle bubbles between training blocks (Train 1 through Train 3), compressing total wall-clock duration from approximately 95 min down to 65 min (a 31.6 percent reduction). Actual gains depend on stage balance and available concurrency.

8.7.2.1 Prefetching mechanics

The overlapped schedule requires enough host workers and buffering to make the next batch ready before the device requests it. Worker count controls preparation capacity; queue depth absorbs variation in preparation time. Neither changes the slowest-stage bound established by equation 8.8.

Machine learning frameworks expose the control variables for this trade-off. Listing 8.4 demonstrates PyTorch's DataLoader configuration, where num_workers = 4 sets preprocessing parallelism and prefetch_factor = 2 maintains a buffer of 8 batches ready for accelerator consumption.

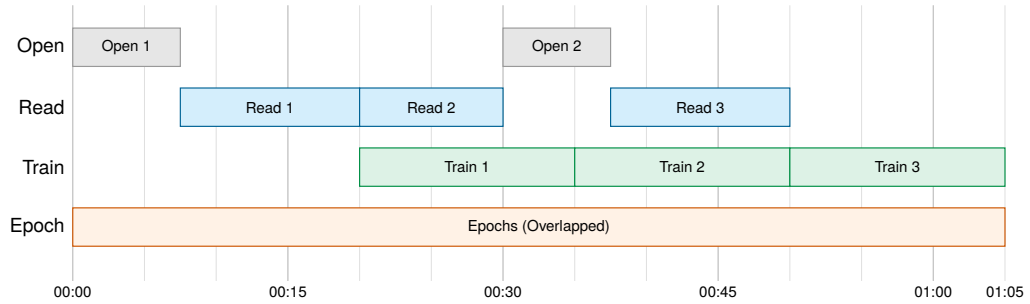


Figure 8.9: Illustrative Overlapped Data Prefetching: File-open and read work overlaps accelerator training while preserving the same individual stage durations as the serial schedule. The schematic schedule spans approximately 65 min, compared with 95 min for the serial schedule.

Listing 8.4: DataLoader Prefetch Configuration: Four workers with a prefetch factor of two maintain up to eight batches in the host-side prefetch window.

```
loader = DataLoader(
    dataset, batch_size=32, num_workers=4, prefetch_factor=2
)
```

The parameters `num_workers` and `prefetch_factor` are not generic performance toggles. They determine CPU parallelism and buffer depth, which means they shift pressure from accelerator idle time to host memory and CPU scheduling. The third knob, `pin_memory=True`, lets host-to-device copies overlap accelerator execution through DMA; its cost is page-locked host memory that the rest of the system can no longer page.

↩ The DataLoader's three knobs, worker count, prefetch depth, and pinned memory for DMA transfers, are established in section 7.5.1.4.

Buffer management is the main trade-off. A buffer that is too small causes the GPU to wait for data preparation, reintroducing the idle time prefetching was meant to remove. An overly large host-side buffer consumes memory that could otherwise support input processing and other host work. The right configuration coordinates CPU preparation, storage I/O, and GPU computation so each resource has work ready at the moment it can use it. These techniques yield the greatest benefit when storage access is slow, preprocessing is complex, or datasets are large.

8.7.2.2 Practical considerations

Prefetch buffers and overlap depth are tunable, so the same machinery adapts to whichever stage binds, whether slow storage, limited network bandwidth, or computational throughput. Prefetching and overlapping deliver the greatest gains when preprocessing lies on the critical path but can run concurrently with model computation. In an illustrative image pipeline, random cropping (10 ms), color jittering (15 ms), and normalization (5 ms) total 30 ms per batch. Overlap can hide up to the concurrent accelerator-computation time; any excess remains on the critical path. NLP workloads similarly benefit when tokenization and subword processing would otherwise block the training loop.

The primary trade-off is memory: prefetch buffers consume host memory, or device memory when a separate device-side queue is used, in proportion to buffer depth and batch size. With batch size 256 high-resolution images (1024×1024 pixels), one buffered batch requires approximately 3.2 GB. With num_workers = 4 and prefetch_factor = 2, the 8-batch prefetch window can hold about 25.8 GB. Tuning these parameters requires empirical testing because excessive workers contend for CPU, storage, and memory resources, while insufficient buffering reintroduces data stalls. Start with a modest worker count, increase it while measuring throughput and memory, and stop when additional workers no longer reduce accelerator idle time. When the input pipeline already exceeds compute demand, deeper prefetching adds complexity without improving throughput.

8.7.3 Mixed-precision training

While prefetching optimizes data movement, mixed-precision training addresses both computational throughput limitations and memory capacity constraints. It uses a lower-precision format where

appropriate while retaining higher precision for selected accumulations or updates. Section D.4.1 compares FP32, FP16, BF16, FP8, and INT8 precision-range trade-offs. A training recipe commonly combines FP32 with either FP16 or BF16; it does not ordinarily use FP16 and BF16 interchangeably in the same path. The resulting speed, memory, and accuracy effects depend on the workload and hardware (Micikevicius et al. 2017; Wang and Kanwar 2019; Kalamkar et al. 2019).

A neural network's stored FP32 weights require 4 bytes per parameter, while FP16 and BF16 weights use 2 bytes. For a model with 10^9 parameters, the working copy of the weights shrinks from 4 GB to 2 GB. That saving does not carry over to the static training state. The common FP16 recipe keeps FP32 master weights and FP32 Adam moments, so it spends 2 + 2 + 4 + 8 = 16 bytes per parameter on weights, gradients, master copy, and moments, exactly what FP32 Adam spends on its 4 + 4 + 8. Mixed precision's memory benefit therefore comes from the tensors that really are stored in 16 bits, above all the activations, together with the halved bytes moved for every 16-bit read, write, and gradient exchange.

The numerical differences between these formats shape their use cases. Table 8.11 shows that BF16's 8-bit exponent gives it approximately the same normal exponent range as FP32, while FP16's 5-bit exponent gives a minimum positive normal value of about 6.1×10^{-5} and a maximum finite value of $65{,}504$. FP16 subnormals extend toward 6×10^{-8}, although their treatment depends on the operation and hardware mode. FP32 provides about seven decimal digits of precision, FP16 about three, and BF16 about two to three. BF16 therefore trades mantissa precision for the exponent range that often simplifies deep-learning training.

Table 8.11: Precision Format Comparison: The choice between FP16 and BF16 depends on whether dynamic range (BF16's strength) or precision (FP16's advantage) matters more for the specific workload. Minimum normal values shown are the practical thresholds for training, as subnormal values may flush to zero on many GPUs. On A100, FP16 and BF16 Tensor Cores reach 16× the FP32 CUDA-core peak.

Property	FP32	FP16	BF16
Exponent bits	8	5	8
Mantissa bits	23	10	7
Min normal value	10^{-38}	6.1×10^{-5}	10^{-38}
A100 peak throughput ratio	1×	16×	16×

The choice between formats depends on model characteristics and hardware support. BF16's wider exponent range reduces overflow and underflow concerns, while FP16 provides more mantissa bits but often requires loss scaling. Framework autocast policies and fused kernels choose higher-precision accumulation for numerically sensitive operations when needed; loss reduction, softmax, normalization, and optimizer updates are common candidates, but their exact execution dtype is implementation-dependent.

Figure 8.10 traces this conventional FP16 execution cycle across six numbered stages: FP32 master weights are downcast to FP16 for the forward pass, loss scaling shifts small gradient values into representable range before backpropagation, and the resulting scaled gradients are unscaled and cast back to FP32 prior to the parameter update. Modern framework implementations may store or accumulate particular tensors differently, and BF16 recipes commonly omit loss scaling because of their wider exponent range.

Modern accelerator architectures pair dense matrix execution units with reduced-precision numerical formats to maximize arithmetic throughput. NVIDIA introduced Tensor Cores for FP16 operations in the Volta architecture (NVIDIA 2017) and added native BF16 execution in Ampere (NVIDIA Corporation 2020a), matching the native BF16 support Google developed for Tensor Processing Units (Wang and Kanwar 2019). These specialized functional units execute low-precision matrix multiply-accumulate operations at four to sixteen times the rate of standard 32-bit floating-point ALUs.

Mixed-precision training turns those hardware capabilities into a split numerical contract. Eligible matrix multiplications and convolutions execute in FP16 or BF16 on tensor units, while reductions, accumulations, and parameter updates retain higher precision. The performance gain comes from both arithmetic throughput and data movement: lower-precision units pack more multiply-accumulates

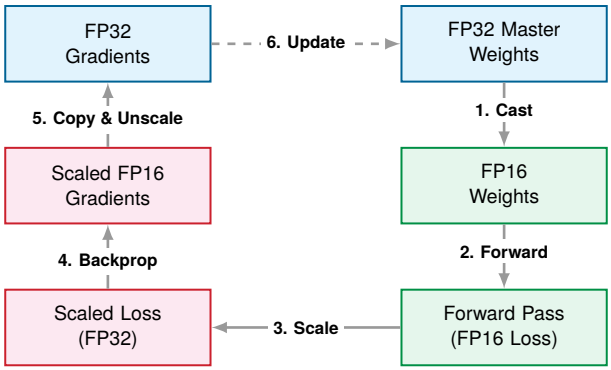


Figure 8.10: Conventional FP16 Mixed-Precision Training: A schematic six-step recipe casts FP32 master weights for lower-precision computation, scales the loss, computes scaled gradients, then unscales them for an FP32 update. Framework implementations may vary, and BF16 usually does not require this loss-scaling path.

into each cycle and each unit of silicon area, while narrower tensors halve data transfer volumes across caches and high-bandwidth memory. Higher precision remains necessary where wide reductions, small parameter updates, or sensitive normalizations can accumulate rounding error. Framework autocast policies enforce this split dynamically, selecting lower-precision execution for compute-intensive matrix multiplications while preserving FP32 for numerically fragile operations.

8.7.3.1 Loss scaling

FP16's 5-bit exponent limits its representable dynamic range,[19] creating severe underflow hazards during backpropagation. When computing gradients across deep networks, successive chain-rule multiplications frequently produce gradient magnitudes smaller than FP16's minimum positive normal value ($2^{-14} \approx 6.1 \times 10^{-5}$). While IEEE 754 FP16 defines subnormal numbers down to $\approx 6 \times 10^{-8}$, hardware accelerators often flush subnormal inputs and outputs to zero to prevent pipeline stalls. When gradients underflow to zero, weight updates stall and training fails to converge.

Loss scaling counters underflow by exploiting the linearity of differentiation: $\nabla_\theta(s_{\text{loss}} \cdot \mathcal{L}) = s_{\text{loss}} \cdot \nabla_\theta \mathcal{L}$. Multiplying the scalar loss $\mathcal{L}$ by a large scaling factor s_{loss} (such as 2^{10} or 2^{15}) prior to backpropagation shifts the gradient distribution rightward into FP16's normal representable range. Once backpropagation completes, the accumulated gradients are divided by s_{loss} (unscaled) before applying the optimizer update.

Deep learning frameworks automate this process through dynamic loss scaling (`GradScaler`). Because gradient norms fluctuate during training, a static scale factor risks either underflow (if s_{loss} is too small) or numerical overflow past FP16's maximum finite value of $65{,}504$ (if s_{loss} is too large). Before executing the parameter update, `GradScaler` unscales the gradients in FP32 and inspects the tensor buffers for nonfinite values (`inf` or `NaN`). If overflow is detected, the scaler skips the optimizer step to protect the FP32 master weights from corruption and reduces the scale factor (typically halving s_{loss}). If training proceeds without overflow across a configurable growth window (such as 2,000 consecutive steps), the scaler doubles s_{loss} to maintain gradients near the upper end of the dynamic range without saturating.

8.7.3.2 Mixed-precision benefits

Mixed-precision training delivers gains across three interacting system dimensions: memory footprint, arithmetic throughput, and communication volume. Halving operand width cuts the storage of every 16-bit tensor by 50 percent, so a one-billion-parameter model's working weights drop from 4 GB to 2 GB. As the 16-bytes-per-parameter accounting above showed, FP32 master weights and moments give that saving back in the static state, so the footprint that actually shrinks is the activations. Computationally, reduced precision routes dense matrix multiplication kernels to high-throughput Tensor Core pipelines (section 8.7.3.3). In distributed topologies, transferring 16-bit rather than 32-bit gradients halves collective AllReduce traffic across the interconnect. These effects compound: reclaimed activation memory permits larger micro-batches, Tensor Cores accelerate kernel execution,

[19] **FP16 (half-precision floating-point)**: Its reduced normal range stems from using 5 exponent bits, compared with 8 in FP32; the smallest positive normal value is about 6.1×10^{-5}, but subnormal values extend to about 6×10^{-8}. Underflow behavior depends on the operation and hardware mode. Loss scaling reduces the chance that small gradients are rounded or flushed to zero.

and narrower tensors reduce gradient exchange latency. The following case study evaluates these memory effects on GPT-2 XL at batch 4, the configuration the activation notebook left over a V100's capacity even after selective recomputation, and derives a checkpoint schedule before selecting a micro-batch that fits the tensor budget.

Napkin Math 8.6: GPT-2 mixed precision memory savings

Problem: Can GPT-2 XL (1.5B parameters) be trained on a single V100 GPU, and what do mixed precision and gradient checkpointing save in memory footprint?

FP32 baseline:

- Parameters and gradients (FP32): 12 GB (6 GB parameters + 6 GB gradients)
- Activations (batch = 4): ~71.7 GB
- Optimizer states (Adam first and second moments in FP32): 12 GB
- Total: ~95.7 GB (exceeds the V100 capacity used in this comparison)

FP16 mixed precision:

- Parameters (FP16): 1.5B parameters at 2 bytes each = 3 GB
- Activations (FP16): ~35.9 GB
- Gradients (FP16): 3 GB
- FP32 master weights: 6 GB (for precise optimizer updates)
- Optimizer states (Adam first and second moments in FP32): 12 GB
- Total: ~59.9 GB (still exceeds the comparison device's capacity before checkpointing)

Mixed precision + a segment-checkpoint ledger:

Partition the 48 transformer layers into 8 contiguous segments of 6 layers. During forward execution, retain each segment's input and discard its internal backward state. During backward execution, reconstruct one segment, retain its internal state while differentiating that segment, then free it before reconstructing the preceding segment. Each layer therefore executes one additional forward pass; against a forward-plus-backward step costing about three forward passes, the extra arithmetic is about one-third. This is work accounting, not a wall-clock penalty.

Each retained segment input has shape batch × sequence length × hidden width. Its storage is therefore $4 \times 1024 \times 1600 \times 2$ bytes. Keep all 8 inputs resident while adding the complete saved-state estimate for the 6 layers in one reconstructed segment. This deliberately counts the active segment's input in both terms, making the retained-state estimate conservative rather than omitting a live boundary.

- Retained segment inputs: 0.105 GB
- Live reconstructed segment: 4.483 GB
- Combined activation allowance: 4.588 GB
- Static state plus activations at batch 4: 28.588 GB

The V100 capacity label is binary-sized; on the decimal basis used by these tensor estimates it is 34.360 GB. The calculated candidate leaves 5.772 GB for output/loss buffers, temporary backward workspaces, communication buffers, and allocator overhead. A runtime peak-memory measurement decides whether those excluded allocations fit in the remainder. The general checkpoint-count trade-off in section 8.7.5.2 explains why both boundaries and the live segment must be counted; transformer segment inputs are much smaller than their internal saved state, so they receive separate byte counts here.

Systems insight: Mixed precision leaves the static state unchanged in this recipe. Checkpointing admits the candidate by replacing retained layer interiors with small boundary tensors and one live reconstructed segment. Gradient accumulation can then raise the effective batch without raising the simultaneous activation footprint. At batch 32, this same checkpoint ledger would require 60.7 GB, still beyond one V100.

Memory savings alone do not guarantee stable training. In the FP16 training recipe used here, maintaining FP32 master weights is necessary because parameter updates $\Delta w = -\eta \cdot g$ (learning rate times gradient) are typically orders of magnitude smaller than the weight values w. In FP16, where the 10-bit mantissa provides roughly three decimal digits of precision, adding an update that is more than $2^{11} \approx 2048$ times smaller than the weight magnitude results in arithmetic swamping: the updated weight rounds back to its original value. This loss of an increment through rounding is distinct from the update value itself underflowing. Retaining master weights in FP32 preserves these incremental updates across thousands of optimizer steps, casting to 16 bits only when staging weights for the forward pass. Autocast policies similarly isolate wide reductions, layer normalizations, and softmax layers in FP32 to prevent catastrophic cancellation.

Once numerical stability is established, reduced-precision arithmetic and reduced memory traffic can shorten execution when they reduce work on the critical path. Comparing complete training recipes does not isolate the contribution of precision alone; model quality, batch size, optimizer representation, and activation policy must be accounted for. The following scenario models these compound throughput, schedule, and infrastructure cost gains for a distributed GPT-2 XL training run.

Napkin Math 8.7: GPT-2 mixed precision throughput and cost

Problem: Given the following illustrative throughput and billing assumptions for a GPT-2 XL job, what speed and cost differences follow from FP16 mixed precision?

Assumed V100 throughput:

- FP32 throughput: ~90 samples/s
- FP16 throughput: ~220 samples/s
- Speedup: 220 samples/s ÷ 90 samples/s ≈ 2.4× faster training

Cost impact on a cluster of 32 GPUs:

- Assumed FP32 billing: $50,000 for 2 weeks
- Assumed FP16 billing: $28,000 for 1.2 weeks
- Wall-clock gain: 2 weeks ÷ 1.2 weeks ≈ 1.7×, below the 2.4× throughput speedup, since the assumed schedule includes work the FP16 kernels do not accelerate
- Time saved: 2 weeks − 1.2 weeks = 0.8 weeks (5.6 days)
- Cost saved: $50,000 − $28,000 = $22,000

Quality gate: The cost comparison is valid only if a controlled validation run shows that the FP16 recipe meets the same acceptance criteria as the FP32 baseline. The scenario does not assume a particular perplexity difference.

Systems insight: Under these assumptions, the same cluster of 32 GPUs changes from 2 weeks and $50,000 to 1.2 weeks and $28,000. This is scenario arithmetic, not a benchmark claim; a real decision requires measured throughput, billing, and quality validation for the target workload.

Realizing these speedups requires empirical profiling rather than assuming automatic acceleration. For small models or memory-bound workloads not dominated by large matrix multiplications, data type casting and kernel launch overheads can offset the arithmetic throughput advantage. Furthermore, whether an accelerator supports native BF16 or requires FP16 loss scaling depends on the underlying microarchitecture.

8.7.3.3 Mixed-precision hardware support

Specialized matrix execution units deliver the wall-clock speedups of mixed precision by trading numerical precision and dynamic range for arithmetic density.[20] Accelerators pair these low-precision multiply units with higher-precision accumulation registers to maintain numerical stability during dense matrix reductions.

The Volta-generation Tensor Cores introduced earlier in this section are dedicated matrix multiplication units optimized for mixed-precision workloads. Unlike standard CUDA cores that process

[20] **Tensor Core**: A specialized matrix-multiply-accumulate unit that supports reduced-precision inputs and higher-precision accumulation. Supported tile shapes, alignment constraints, and peak-throughput ratios vary by architecture, dtype, sparsity mode, and library kernel.

scalar or small vector operations, Tensor Cores perform 4×4 matrix multiply-accumulate operations in a single clock cycle. For FP16 inputs, a single Tensor Core executes:

$$\mathbf{D}_{\text{tc}} = \mathbf{A}_{\text{tc}} \times \mathbf{B}_{\text{tc}} + \mathbf{C}_{\text{acc}}$$

where $\mathbf{A}_{\text{tc}}$ and $\mathbf{B}_{\text{tc}}$ are FP16 inputs and $\mathbf{C}_{\text{acc}}$ and $\mathbf{D}_{\text{tc}}$ may use FP32 accumulation. Higher-precision accumulation reduces rounding error, although it does not eliminate numerical error or cancellation.

Hardware peak specifications define the theoretical ceiling on this acceleration. An NVIDIA A100 GPU exposes 19.5 TFLOP/s of FP32 throughput on standard CUDA cores and 312 TFLOP/s of FP16 or BF16 Tensor Core throughput, a 16× peak ratio. H100-class accelerators add FP8 Tensor Cores at 1,979 TFLOP/s without sparsity, about 2× the H100 FP16/BF16 Tensor Core rate. These figures represent hardware ceilings rather than end-to-end training guarantees: matrix multiplications map directly to Tensor Cores, but data movement, non-Tensor-Core kernels, communication, loss scaling, and optimizer updates remain. While the batched matrix multiplication $\mathbf{QK}^T$ in transformer self-attention ($2 \times B \times N_{\text{heads}} \times S^2 \times d_{\text{head}}$ FLOPs) maps directly to these tensor units, Amdahl's law dictates that the remaining memory-bound elementwise operations, communication collectives, and optimizer updates bound the realized end-to-end speedup.

Brain Float 16 (BF16) maintains FP32's 8-bit exponent while reducing the mantissa to 7 bits. This design choice prioritizes dynamic range preservation over precision, which matters for gradient-based learning where values span many orders of magnitude. Google's TPUs natively support BF16, while NVIDIA's Ampere architecture (A100) and newer provide full hardware support.

The range advantage of BF16 appears exactly where loss scaling was needed for FP16. A gradient below FP16's smallest normal value of 6.1×10^{-5}, which some hardware flushes to zero, is still an ordinary normal number in BF16, whose smallest normal value of about 10^{-38} matches FP32's exponent range. BF16 therefore usually avoids the loss scaling needed by FP16, although accumulation precision and rounding still matter.

NVIDIA's Hopper architecture (H100) introduces FP8 support with two formats. E4M3 uses four exponent bits and three mantissa bits (prioritizing precision for forward pass weights and activations), while E5M2 uses five exponent bits and two mantissa bits (prioritizing dynamic range for backward pass gradients).

FP8 training doubles Tensor Core throughput again (1.98 PFLOP/s on H100 dense vs. 0.99 PFLOP/s for FP16 dense, without sparsity). However, FP8's severely limited precision requires per-tensor scaling factors maintained in higher precision, adding algorithmic complexity. Table 8.12 summarizes when each precision is appropriate:

Table 8.12: Precision Selection Guide: Format choice depends on hardware support, kernel coverage, numerical behavior, and validation results. FP8 requires an explicit scaling strategy; BF16 provides a wider exponent range than FP16; FP16 often uses loss scaling; and FP32 remains available for sensitive operations or unsupported lower-precision paths.

Precision	When to Use	Hardware Requirement
FP8	Throughput-oriented training with validated scaling	Supported FP8 hardware
BF16	Wide exponent range without FP16 loss scaling	Supported BF16 accelerator
FP16	Higher mantissa precision with managed loss scaling	Supported FP16 accelerator
FP32	Higher precision or unsupported lower-precision path	GPU

Selecting among these formats requires balancing arithmetic throughput against implementation complexity and numerical stability.

In addition to accelerating compute-bound matrix operations, reduced precision relieves memory bandwidth bottlenecks by altering operational arithmetic intensity. On modern accelerators, large matrix multiplications reside in the compute-bound regime of the roofline model, whereas element-wise operations (such as bias additions, layer normalization, and residual connections) reside firmly in the memory-bound regime. An NVIDIA A100 GPU illustrates the physical imbalance between arithmetic capability and memory bandwidth:

- HBM2e bandwidth: 2,039 GB/s

- FP32 throughput (19.5 TFLOP/s): worst-case demand is 78 TB/s if every FLOP needs new data
- Actual requirement (with data reuse): Much lower, but bandwidth-limited for operations with low arithmetic intensity

For memory-bound kernels, execution time is determined by bytes transferred across the memory bus rather than arithmetic clock cycles. Converting 32-bit operands to 16-bit FP16 or BF16 halves the byte volume transferred across the HBM bus, doubling the theoretical throughput of memory-bound operations even without engaging Tensor Cores. Realized bandwidth gains depend on cache hit rates, kernel fusion, and format conversion costs.

Section 7.4.2.2 describes how `autocast` and `GradScaler` implement the split between 16-bit and FP32 operations inside the framework. The policy decision that remains for the training recipe is which format to run, and listing 8.5 makes it a single switch: the FP16 path enables gradient scaling, and the BF16 path disables it.

Listing 8.5: Mixed-Precision Training: Autocast selects eligible lower-precision operations, while gradient scaling is enabled for the FP16 path and omitted for BF16.

```
import torch

model = TransformerModel().cuda()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
amp_dtype = (
    torch.float16
)  # torch.bfloat16 usually does not need loss scaling
scaler = torch.amp.GradScaler(
    "cuda", enabled=(amp_dtype == torch.float16)
)

for inputs, targets in dataloader:
    optimizer.zero_grad()

    # Automatic precision selection per operation
    with torch.amp.autocast("cuda", dtype=amp_dtype):
        output = model(inputs)
        loss = criterion(output, targets)

    # Scale loss to prevent gradient underflow
    scaler.scale(loss).backward()

    # Unscale gradients before optimizer step
    scaler.step(optimizer)
    scaler.update()  # Adjust scaling factor dynamically
```

Because hardware support for low-precision formats evolved across accelerator generations, the optimal mixed-precision recipe depends on the target architecture. Table 8.13 outlines the precision trade-offs across three generations of NVIDIA GPUs.

Table 8.13: Precision Strategy by GPU Architecture: V100 supports FP16 Tensor Core training, A100 adds native BF16, and H100 adds FP8 paths whose use requires recipe-specific validation.

Architecture	Recommended Precision	Key Considerations
V100 (Volta)	FP16 with loss scaling	No native BF16 Tensor Core path; clipping depends on the training recipe
A100 (Ampere)	BF16 or FP16	BF16 avoids FP16 loss scaling; TF32 can accelerate eligible FP32 matrix operations
H100 (Hopper)	BF16, FP16, or validated FP8	FP8 requires an FP8-aware recipe; 1,979 TFLOP/s is a peak ceiling

An illustrative single-GPU GPT-2 scenario (1.5B) demonstrates how architectural execution units and numerical formats compound: 18 samples/s for V100 FP32, 45 samples/s for V100 FP16,

165 samples/s for A100 BF16, and 380 samples/s for H100 FP8. These figures represent scenario projections rather than isolated benchmarks, as software stacks, kernel implementations, and batch dimensions also evolved across generations. Maximizing training throughput therefore requires co-designing the numerical precision format with the accelerator's native matrix execution pipelines.

8.7.4 FlashAttention: IO-aware attention optimization

Mixed-precision training addresses two bottlenecks, compute throughput because Tensor Cores run faster on 16-bit operands, and memory capacity because each value uses fewer bytes. For transformer models, a third bottleneck often dominates, and the roofline analysis has already located it. Counting only its score-matrix traffic, materialized attention in GPT-2 Small reaches about 32 FLOP/byte, well left of the A100 ridge point in figure 8.1, because its quadratic intermediate matrices are written to HBM during the forward pass and read back during backpropagation. Halving the bytes per value does not change that quadratic traffic, so even with reduced precision the compute units can sit idle while the processor waits for data.

FlashAttention[21] (T. Dao et al. 2022) addresses this bandwidth bottleneck by optimizing how data flows between memory hierarchies. Processing attention in SRAM-sized tiles avoids materializing the full $S \times S$ matrix in HBM and reduces traffic without changing the mathematical output, which is how the roofline preview moved attention rightward toward, and for favorable shapes past, the ridge.

[21] **FlashAttention**: Introduced by T. Dao et al. (2022), it preserves exact attention while reducing HBM traffic through SRAM-sized tiling; the key insight was to treat attention as an IO problem: avoid materializing the full $S \times S$ score matrix in HBM while retaining dense $\mathcal{O}(S^2)$ score computation. FlashAttention-2 (Dao 2023) further improved parallelism and work partitioning, reporting 50–73 percent of theoretical peak on A100.

8.7.4.1 The standard attention memory bottleneck

Standard self-attention computes relationships between all positions in a sequence. For an input sequence of length S, the mechanism computes an $S \times S$ attention matrix according to equation 8.11:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{head}}}}\right)\mathbf{V} \tag{8.11}$$

Here, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are the query, key, and value matrices for the sequence, and d_{head} is the key-vector dimension used to scale the dot products before softmax. The $\mathbf{Q}\mathbf{K}^T$ term is the source of the $S \times S$ score matrix, which is why attention becomes an I/O problem even when the arithmetic itself maps well to Tensor Cores.

The memory bottleneck emerges from materializing the $S \times S$ intermediate matrices for scores and probabilities. For a sequence length of 4,096 with head dimension $d_{\text{head}} = 64$ (typical for a single attention head), the attention score matrix alone requires $4{,}096^2 \times 4$ bytes = 67.1 MB in FP32. This section counts FP32 scores throughout, since autocast policies commonly keep softmax in FP32; at FP16 every figure halves, as in the roofline estimate, while the quadratic growth stays the same. With 16 heads, this grows to 1.1 GB just for intermediate attention matrices, not including the keys, queries, values, or output tensors.

GPU memory hierarchy creates the opportunity for I/O-aware attention. HBM offers large off-chip capacity, while much smaller on-chip SRAM has substantially higher bandwidth and lower latency. Attention implementations that materialize the quadratic intermediates in HBM must write and later read them during backpropagation. Because that traffic grows with the square of sequence length, it takes a growing share of step time as sequences lengthen.

The backward pass requires the information represented by the attention probabilities, but an implementation may either save those intermediates or recompute them. For $\mathbf{A} = \text{softmax}(\mathbf{S})$ and $\mathbf{S} = \mathbf{Q}\mathbf{K}^T/\sqrt{d_{\text{head}}}$, one gradient expression is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Q}} = \frac{1}{\sqrt{d_{\text{head}}}} \cdot \text{dsoftmax}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{A}}, \mathbf{A}\right) \cdot \mathbf{K}$$

where dsoftmax denotes the softmax Jacobian-vector product. A conventional implementation may save the probability matrix for this computation; saving both probabilities and raw scores is not mathematically required. FlashAttention instead saves compact normalization statistics and recomputes tiled score and probability blocks during backward propagation, avoiding full quadratic intermediates in HBM.

8.7.4.2 IO-aware attention through tiling

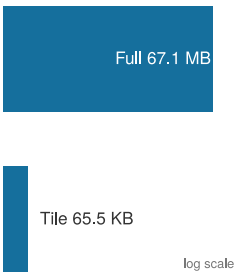
FlashAttention eliminates the need to materialize full $S \times S$ attention matrices in HBM by computing attention incrementally through tiling. Instead of computing the entire attention matrix at once, the algorithm partitions **Q**, **K**, and **V** into tiles small enough to fit in fast SRAM, computes attention scores for these tiles, and incrementally accumulates results.

The key algorithmic insight relies on the structure of equation 8.11. Softmax normalizes each row of scores by a sum over the entire row, which seems to require the whole row before any output can be formed.

The online softmax algorithm removes that requirement. Ordinary softmax, $\text{softmax}(\mathbf{x})_i = e^{\mathbf{x}_i} / \sum_j e^{\mathbf{x}_j}$, is computed stably by first subtracting the row maximum. The online form instead keeps two running statistics per row as blocks of scores arrive: the largest score seen so far, m, and the sum of exponentials taken relative to that maximum, z. When a new block raises the maximum from m to m', everything accumulated so far was exponentiated relative to the old maximum, and multiplying it by $e^{m-m'}$ re-expresses it relative to the new one. The same factor rescales the partially accumulated output. After the last block, dividing the output by z gives exactly the softmax-weighted result, as if the whole row had been seen at once.

Algorithm 8.3 decomposes this computation by partitioning the queries and the keys/values into tiles small enough to fit in SRAM, then streaming over the key/value tiles while maintaining the running softmax statistics needed to assemble each output tile, without ever forming the full score matrix.

No full $S \times S$ score or probability matrix is written to HBM. A 128×128 FP32 score tile, for example, occupies 65.5 KB compared with 67.1 MB for a 4096×4096 matrix. Other inputs, outputs, and running statistics remain, so the tile is not necessarily the largest tensor in the entire kernel.



FlashAttention swaps the full attention matrix for small SRAM tiles.

Algorithm 8.3 FlashAttention: tiled attention with online softmax

Require: queries **Q**, keys **K**, values **V** for a length-S sequence; tile size b_{tile}
Ensure: attention output **O**, without materializing the $S \times S$ score matrix

for each query tile $\mathbf{Q}_i$ **do**
  in SRAM: $\mathbf{O}_i^{\text{acc}} \leftarrow \mathbf{0}$, $m_i \leftarrow -\infty$, $z_i \leftarrow 0$
  for each key/value tile $(\mathbf{K}_j, \mathbf{V}_j)$ **do**
    load $\mathbf{Q}_i, \mathbf{K}_j, \mathbf{V}_j$ into SRAM; $\mathbf{S}_{ij} \leftarrow \mathbf{Q}_i \mathbf{K}_j^T / \sqrt{d_{\text{head}}}$
    $m_i' \leftarrow \max(m_i, \text{rowmax}(\mathbf{S}_{ij}))$ ▷ *new stable max*
    $\mathbf{O}_i^{\text{acc}} \leftarrow \mathbf{O}_i^{\text{acc}} \odot e^{m_i - m_i'} + e^{\mathbf{S}_{ij} - m_i'} \mathbf{V}_j$
    $z_i \leftarrow z_i \odot e^{m_i - m_i'} + \text{rowsum}(e^{\mathbf{S}_{ij} - m_i'})$; $m_i \leftarrow m_i'$
    discard $\mathbf{S}_{ij}$ ▷ *never written to HBM*
  end for
  $\mathbf{O}_i \leftarrow \mathbf{O}_i^{\text{acc}} / z_i$; write $\mathbf{O}_i$ to HBM
end for

8.7.4.3 Memory and IO complexity analysis

FlashAttention improves both activation memory and memory IO, which are the limiting costs in bandwidth-bound attention. Standard attention requires $\mathcal{O}(S^2)$ memory to store score matrices **S** and attention-probability matrices **A** across all sequence positions. FlashAttention reduces the stored activation footprint to $\mathcal{O}(S)$ by keeping only input and output tensors $(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O})$ plus a small SRAM buffer for the current tile.

For $S = 4096$, $d_{\text{head}} = 64$, one materialized FP32 score matrix occupies $4096^2 \times 4$ bytes = 67.1 MB per head. The three FP32 input tensors occupy $(3 \times 4096 \times 64) \times 4$ bytes ≈ 3.1 MB per head, a component-size ratio of 21.3× between one quadratic intermediate and the inputs. The comparison that decides memory is the whole per-head footprint each algorithm keeps for the backward pass. The conventional baseline assumed here stores the inputs, the output, and both the score and probability matrices, about 138.4 MB. FlashAttention stores the inputs, the output, and two numbers per row (the running maximum and sum), about 4.2 MB, roughly 32.7× smaller, and the gap widens as S grows because only the standard footprint is quadratic.

These stored-tensor totals exclude gradient buffers, temporary workspace, and the live SRAM tile, so they are not estimates of peak training memory. Implementations may also retain fewer quadratic intermediates than the conventional baseline assumed here.

The IO pattern changes for the same reason. Standard attention reads **Q**, **K**, and **V** from HBM, writes score, probability, and output matrices during the forward pass, then rereads those stored matrices during the backward pass before writing the gradients $d\mathbf{Q}$, $d\mathbf{K}$, and $d\mathbf{V}$ (shorthand for $\partial\mathcal{L}/\partial\mathbf{Q}$ and its counterparts). The resulting HBM traffic includes the same $\mathcal{O}(S^2)$ term that made the activation footprint large. FlashAttention forms score and probability blocks in SRAM and never writes the full $S \times S$ tensors to HBM. In the backward pass, it recomputes the needed blocks rather than reading stored full matrices. The exact HBM IO complexity depends on tile size and available SRAM because key and value blocks are streamed repeatedly across query blocks, so the bound is not simply $\mathcal{O}(S \cdot d)$.

For large sequence lengths, FlashAttention reduces HBM traffic by avoiding writes and rereads of the $S \times S$ score and probability matrices. With $S = 4096$ and $d_{\text{head}} = 64$, avoiding the stored attention matrices removes the quadratic component of the activation footprint, while the precise bandwidth reduction depends on hardware and kernel tiling.

Both approaches require $\mathcal{O}(S^2 d)$ asymptotic FLOPs for dense attention. FlashAttention's backward pass recomputes tiled attention intermediates from saved inputs and normalization statistics rather than storing full score and probability matrices. The resulting reduction in HBM traffic can shift the kernel toward compute-bound execution and produce a net speedup, but the final bottleneck remains shape- and hardware-dependent.

8.7.4.4 Implementation and hardware utilization

FlashAttention's performance gains materialize through careful exploitation of GPU memory hierarchy. Modern frameworks can dispatch eligible attention operations among fused and fallback implementations based on hardware, inputs, and layouts; profiling verifies which path ran and whether it helped. Listing 8.6 contrasts standard and optimized attention implementations.

Listing 8.6: Attention Implementation Comparison: Standard attention materializes the full $S \times S$ matrix in HBM, while FlashAttention uses PyTorch's optimized implementation or the dedicated flash-attn library.

```
import torch
import torch.nn.functional as F


# Standard attention (materializes n-by-n matrix)
def standard_attention(q, k, v):
    # q, k, v: [batch, heads, seq_len, head_dim]
    scores = torch.matmul(q, k.transpose(-2, -1)) / (
        q.size(-1) ** 0.5
    )
    attn = F.softmax(scores, dim=-1)  # n-by-n matrix in HBM
    output = torch.matmul(attn, v)
    return output


# FlashAttention (no n-by-n materialization)
def flash_attention(q, k, v):
    # Can use FlashAttention when inputs and hardware are eligible
    output = F.scaled_dot_product_attention(q, k, v)
    return output


# Explicit FlashAttention 2 (flash-attn library)
from flash_attn import flash_attn_func


def flash_attn_2(q, k, v):
    # q, k, v: [batch, seq_len, heads, head_dim]
    # Different layout for optimized memory access
    output = flash_attn_func(q, k, v)
    return output
```

8.7.4.5 Illustrative speed and memory pattern

T. Dao et al. (2022) reports end-to-end GPT-style training speedups and separate attention-kernel benchmarks showing that IO-aware attention reduces memory traffic and improves runtime. The mechanism established above is the important comparison: standard attention stores quadratic attention matrices for backward, while FlashAttention recomputes their tiles and avoids that off-chip materialization. Runtime and memory savings depend on batch size, head count, sequence length, precision, and the kernel implementation. A capacity threshold requires the complete training ledger, including the state outside attention.

Subsequent versions have continued improving performance: FlashAttention-2 (Dao 2023) achieved 1.5–2× additional speedup through better parallelism and register allocation, while FlashAttention-3 (Shah et al. 2024) exploits asynchronous memory operations and FP8 tensor cores on Hopper GPUs; it reports about 740 TFLOP/s in FP16 on H100 (roughly 75 percent of peak), with its FP8 path reaching higher.

8.7.4.6 When to use FlashAttention

For transformer training with long sequences, FlashAttention is usually the first attention implementation to test. It often becomes essential once sequence length pushes standard attention into an HBM-capacity or HBM-bandwidth bottleneck, especially around the multi-thousand-token regime where the $S \times S$ attention matrices dominate memory. A100- and H100-class GPUs with fast on-chip SRAM benefit most. The returns diminish for short sequences where tiling overhead is not worthwhile, on older GPU architectures without comparable SRAM bandwidth, and for nonattention architectures such as convolutional neural networks and multilayer perceptrons.

In practice, deep learning frameworks handle much of FlashAttention's integration, but the decision still depends on layout, precision, and memory budget. PyTorch's `scaled_dot_product_attention` can dispatch to FlashAttention when hardware, inputs, and backend constraints permit;

hand-written attention must use an eligible primitive. Three practical checks determine whether FlashAttention is appropriate:

1. Ensure tensor layouts match library expectations (contiguous memory, correct dimension ordering)
2. Use FP16 or BF16 for maximum speedup (FlashAttention optimized for mixed precision)
3. Combine with gradient checkpointing when additional activation-memory savings are needed.

The integration is often a single-line change—swapping a manual attention call for `F.scaled_dot_product_attention`, as listing 8.6 shows. The engineering decision is still the same one established by the roofline analysis: use the optimized primitive when the bottleneck is HBM traffic, and let the library manage the tiling and SRAM scheduling needed to remove it.

8.7.4.7 Systems implications and broader principles

FlashAttention exemplifies a fundamental systems engineering principle: **IO-aware algorithm design**. Many accelerators are compute-abundant relative to memory bandwidth. For a bandwidth-bound kernel, traffic can govern runtime even when FLOPs do not change.

This principle extends beyond attention. In IO-aware matrix multiplication, tiling algorithms like those in CUTLASS minimize DRAM traffic by maximizing data reuse in fast caches. A naive $N \times N$ matrix multiply performs $\mathcal{O}(N^3)$ FLOPs and, without reuse, $\mathcal{O}(N^3)$ memory traffic; blocking keeps tiles in fast memory and moves traffic toward the $\mathcal{O}(N^2)$ lower bound.

The same byte-movement principle reappears in communication-efficient distributed training, where gradient compression trades extra computation (compression/decompression) for reduced network bandwidth consumption. Low-power edge devices with limited memory bandwidth benefit even more from IO-aware algorithms. Trading 10 percent more arithmetic to halve memory traffic approaches a $2\times$ energy reduction in the limit where data movement accounts for the entire budget, and falls short of that bound whenever arithmetic carries a meaningful share, because halving one additive term of a sum cannot more than halve the sum. This section establishes the IO-aware heuristic; distributed and edge deployment settings reuse the same byte-movement logic.

Eliminating off-chip materialization of the $\mathcal{O}(S^2)$ attention matrices reclaims accelerator memory across three architectural trade-offs. First, it enables longer sequence contexts on fixed hardware, with the workable context boundary determined by the complete training recipe. Second, freeing activation memory allows larger micro-batch sizes that improve hardware arithmetic saturation, provided gradient dynamics remain stable. Third, the contracted activation footprint permits deeper transformer stacks within a given memory budget, although end-to-end multi-billion-parameter feasibility continues to depend on parameter sharding, optimizer states, and remaining layer activations.

The technique demonstrates that systems-level algorithms can exploit memory hierarchy for large activation-memory reductions and workload-dependent speedups beyond hardware scaling alone. Treating memory bandwidth as the primary constraint and compute as abundant is a recurring pattern in ML performance optimization.

FlashAttention addresses memory bandwidth bottlenecks during computation, but another class of memory constraints exists: the sheer capacity required to store activations and optimizer states simultaneously. When models or batch sizes exceed GPU memory capacity, two complementary techniques trade computation for memory.

8.7.5 Gradient accumulation and checkpointing

Both techniques have already appeared as tools. Gradient accumulation entered with mini-batch processing (section 10) as the way to build a large effective batch from micro-batches that fit, and checkpointing entered as algorithm 8.2, the schedule that discards activations and recomputes them during backward. This section supplies what using them correctly requires: the proof that accumulated gradients equal the large-batch gradient and the conditions under which that equivalence breaks, the per-update time cost and the synchronization boundary in multi-GPU training, the derivation of the $\sqrt{N_L}$ checkpoint placement, and the loss-normalization errors that most often break accumulation in practice.

8.7.5.1 Gradient accumulation and checkpointing mechanics

Gradient accumulation and activation checkpointing operate on distinct principles, but both aim to optimize memory usage during training by modifying how forward and backward computations are handled. Gradient accumulation changes how many examples contribute to one update; checkpointing changes which activations remain resident between the forward and backward passes.

Gradient accumulation To execute large batch training on memory-constrained accelerators, systems split an effective logical batch across smaller sequential micro-batches. Figure 8.11 diagrams this reduction pathway: three consecutive micro-batches evaluate independent losses ($\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$) and gradients ($\mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3$, where $\mathbf{g}_j = \nabla_\theta \mathcal{L}_j$), accumulating into a single unified tensor ($\mathbf{g}_1 + \mathbf{g}_2 + \mathbf{g}_3$) before triggering the optimizer step.

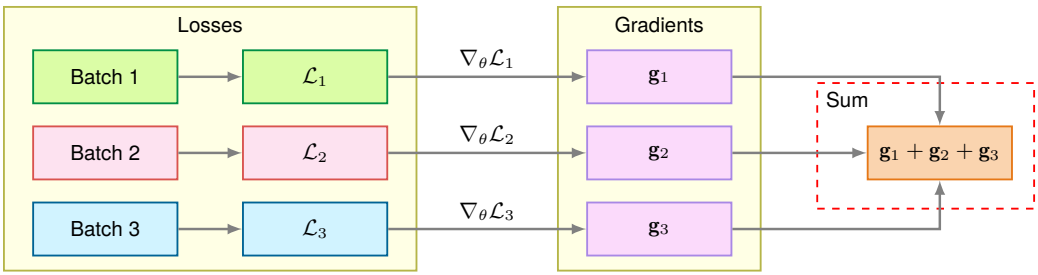


Figure 8.11: Gradient Accumulation: Three micro-batches each compute independent losses and gradients, which sum into a single combined gradient for one parameter update. This simulates training with a batch three times larger without requiring the memory to hold all samples simultaneously.

In PyTorch, gradient accumulation is usually implemented by dividing each micro-batch loss by the number of accumulation steps and calling `optimizer.step()` only after processing the entire effective batch. Under that averaging convention, no learning-rate adjustment is needed solely because of accumulation. The implementation follows five steps:

1. Perform the forward pass for a micro-batch.
2. Compute the gradients during the backward pass.
3. Accumulate the gradients into a buffer without updating the model parameters.
4. Repeat steps 1–3 for all micro-batches in the effective batch.
5. Update the model parameters using the accumulated gradients after all micro-batches are processed.

Gradient accumulation can reproduce the gradient of a larger batch under controlled assumptions. For an effective batch size $B = k \times B_{\text{micro}}$ where k is the number of accumulation steps and B_{micro} is the micro-batch size, equation 8.12 confirms that the accumulated gradient equals the true batch gradient:

$$\nabla \mathcal{L}_B = \frac{1}{B} \sum_{i=1}^{B} \nabla \mathcal{L}_i = \frac{1}{k} \sum_{j=1}^{k} \left(\frac{1}{B_{\text{micro}}} \sum_{i \in \mathcal{B}_j} \nabla \mathcal{L}_i \right) \tag{8.12}$$

This equivalence holds when each example's computation is identical and the loss is averaged consistently. The right-hand side shows that averaging k micro-batch gradients (each computed over B_{micro} examples) produces the same result as computing the gradient over all $B = kB_{\text{micro}}$ examples at once. Stateful layers such as BatchNorm, stochastic operations such as dropout and data augmentation, finite-precision accumulation order, and step-indexed schedules can break exact equivalence, so large-batch accumulation still requires validation.

Gradient accumulation exchanges memory capacity for computation time. Table 8.14 separates the memory benefit from the compute and scheduling costs that remain.

Table 8.14: Gradient Accumulation Trade-Offs: Accumulation reduces resident activation memory and preserves total example-level arithmetic. Wall-clock cost depends on micro-batch efficiency, kernel-launch overhead, and whether distributed synchronization is deferred until the accumulated update.

Dimension	Effect
Memory	$\mathcal{O}(B_{\text{micro}})$ instead of $\mathcal{O}(B)$, yielding a $k\times$ reduction in activation memory
Computation	Unchanged total FLOPs, as all B examples are still processed
Time	k micro-batch forward and backward passes precede each optimizer step; smaller micro-batches can reduce utilization and add launch overhead

Accumulation preserves the total example-level arithmetic but can change efficiency because smaller micro-batches use the accelerator differently and launch kernels more often. With distributed `no_sync()` accumulation, one gradient synchronization occurs after the k local micro-batches rather than after each one. The model in equation 8.13 gives the per-update time:

$$T_{\text{effective}} = \sum_{j=1}^{k} T_{\text{micro},j} + T_{\text{sync}} + T_{\text{update}} \tag{8.13}$$

Here $T_{\text{micro},j}$ is the measured forward/backward time for micro-batch j, T_{sync} is the synchronization time paid for the accumulated update, and T_{update} is the optimizer-step time. In distributed data-parallel training, standard framework wrappers automatically fire asynchronous gradient-synchronization hooks during every backward pass. When accumulating gradients across k micro-batches, leaving these hooks active would trigger redundant AllReduce operations on every micro-batch; wrapping the first $k-1$ micro-batches in a `model.no_sync()` context suppresses this communication, synchronizing gradients only on the final k-th step. That synchronization boundary may contain several bucketed AllReduce calls.

The wall-clock penalty relative to processing the effective batch at once is workload-dependent. It must be measured rather than inferred from the memory-reduction factor.

When gradient accumulation is combined with distributed data parallelism across multiple machines, additional considerations arise for gradient synchronization timing and effective batch size calculation across the cluster. Advanced distributed systems texts treat these patterns in depth.

Activation checkpointing Activation checkpointing reduces memory usage during the backward pass by discarding and selectively recomputing activations. In standard training, activations from the forward pass are stored in memory for use in gradient computations during backpropagation. However, these activations can consume gigabytes of memory, particularly in deep networks.

Activation memory can be reduced by trading additional FLOPs for reduced HBM occupancy during backpropagation. Figure 8.12 diagrams this recomputation cycle across forward and backward execution rows: only selected boundary tensors (solid green circles) are retained in device memory while intermediate activations (dashed gray circles) are dropped, then recomputed on demand (orange circles) as the backward pass traverses in reverse.

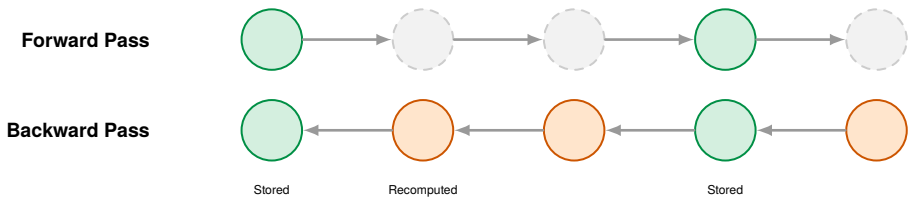


Figure 8.12: Activation Checkpointing: Trading memory usage for recomputation during backpropagation enables training deeper neural networks. By storing only a subset of activations from the forward pass and recomputing others on demand, this technique reduces peak memory requirements at the cost of increased training time.

The implementation keeps the capacity trade-off explicit. The system splits the model into segments, retains activations only at segment boundaries during the forward pass, and recomputes intermediate activations during the backward pass when needed.

Frameworks like PyTorch provide tools such as `torch.utils.checkpoint` to simplify this process, but the recompute cost remains. Checkpointing is most effective for deep architectures with dozens

or hundreds of layers, such as transformers or large convolutional networks, where activation storage exceeds the GPU's capacity before arithmetic throughput is exhausted.

The synergy between gradient accumulation and checkpointing enables training of larger, more complex models. Gradient accumulation manages memory constraints related to batch size, while checkpointing optimizes memory usage for intermediate activations. Together, these techniques expand the range of models that can be trained on available hardware.

8.7.5.2 Optimal checkpoint placement strategy

Algorithm 8.2 stated the $\sqrt{N_L}$ schedule without justifying it. For a network with N_L layers of comparable activation size and recomputation cost, each storing $M^{(\ell)}_{\text{activations}}$ bytes of activations, table 8.15 quantifies how the number and placement of checkpoints determines the memory-compute trade-off. Sub-linear checkpointing strategies can reduce memory consumption from $\mathcal{O}(N_L)$ to $\mathcal{O}(\sqrt{N_L})$ with additional forward arithmetic; the resulting wall-clock overhead depends on execution, while the reduced footprint can enable deeper models on existing hardware.

Table 8.15: Checkpointing Memory-Compute Trade-Offs: Different checkpoint strategies trade memory savings against recomputation overhead. The optimal number of checkpoints balances these factors.

Strategy	Memory Cost	Recompute Cost
No checkpointing	$N_L \times M^{(\ell)}_{\text{activations}}$	0 forward ops
One checkpoint ($n_{\text{ckpt}} = 1$)	$M^{(\ell)}_{\text{activations}} + N_L \times M^{(\ell)}_{\text{activations}}$	$(N_L - 1)$ forward ops
n_{ckpt} checkpoints	$n_{\text{ckpt}} \times M^{(\ell)}_{\text{activations}} + (N_L / n_{\text{ckpt}}) \times M^{(\ell)}_{\text{activations}}$	$(N_L - n_{\text{ckpt}})$ forward ops

The memory column counts two things, the stored checkpoints and the one segment that must be rematerialized at a time during backward, which holds N_L/n_{ckpt} layers of activations. The one-checkpoint row shows why both terms matter. With a single checkpoint the only segment spans the whole network, so backward rebuilds all N_L layers of activations at once and the peak is no lower than storing everything. Checkpoints pay off only when they are spread out so that each recomputed segment is short. The recompute column counts layer forwards. Every layer that is not itself a checkpoint is run forward once more, $(N_L - n_{\text{ckpt}})$ layer forwards in all, which is nearly one full forward pass when n_{ckpt} is much smaller than N_L.

Setting the derivative of total memory cost $(n_{\text{ckpt}} \times M^{(\ell)}_{\text{activations}} + (N_L/n_{\text{ckpt}}) \times M^{(\ell)}_{\text{activations}})$ with respect to the checkpoint count n_{ckpt} to zero yields the analytical optimum at $n_{\text{ckpt}} = \sqrt{N_L}$. In the standard $\sqrt{N_L}$ checkpointing scheme, this costs roughly one extra forward pass, about one-third more total step compute, in exchange for minimizing activation footprint under this analytical model. For GPT-2 with forty-eight transformer layers, the operational difference is substantial: uncheckpointed training requires $48 \times M^{(\ell)}_{\text{activations}}$ in full activation storage, whereas uniform checkpointing ($\sqrt{48} \approx 7$ checkpoints) requires $7 \times M^{(\ell)}_{\text{activations}} + (48/7) \times M^{(\ell)}_{\text{activations}} \approx 14 \times M^{(\ell)}_{\text{activations}}$, saving 71 percent activation memory for approximately 33 percent additional total step computation (by recomputing one forward pass over the uncheckpointed layers during backpropagation).

The GPT-2 memory notebook uses eight equal segments of six layers. It applies the same boundary-plus-live-segment accounting with unequal storage costs: a segment input contains only the hidden tensor, whereas the reconstructed layer state also includes attention and feed-forward intermediates. The equal-state optimum above is therefore a useful baseline, not an optimality claim for that transformer schedule. Counting the two components separately produces the candidate memory budget without assuming a reduction factor.

Not all operations are equally expensive to recompute, which motivates selective checkpointing. Table 8.16 compares where activation memory is large enough to justify recomputation.

Table 8.16: Selective Checkpoint Placement: Placement compares the state saved at a block boundary with the work required to reconstruct it. Attention and feed-forward costs depend on sequence length, width, kernel implementation, and whether memory-efficient attention is already in use.

Layer type	Memory cost	Recompute cost	Checkpointing strategy
Attention blocks	Shape-dependent; materialized attention can add quadratic intermediates	High and sequence-length-dependent	Checkpoint block boundaries when saved state justifies recomputation
Feed-forward blocks	Often dominated by expanded hidden activations	High and width-dependent	Checkpoint when expanded activations dominate memory
Normalization	Small standalone outputs	Low	Usually handled within a larger checkpointed block

The practical rule is to spend recomputation on high-memory layers, not on operations whose activations are already cheap to retain.

8.7.5.3 Memory and computational benefits

Gradient accumulation produces an effective batch larger than the resident micro-batch without storing all of its activations simultaneously. A larger effective batch reduces sampling variance but does not guarantee faster convergence or better generalization. The technique is particularly valuable when the desired effective batch cannot fit in memory at once.

Activation checkpointing significantly reduces the memory footprint of intermediate activations during the forward pass, allowing training of deeper models. By discarding and recomputing activations as needed, checkpointing frees up memory for larger models, additional layers, or higher resolution data. This trade-off is critical in architectures like transformers that require substantial memory for intermediate computations.

Both techniques reduce activation-memory requirements without adding hardware. For lighthouse 8.1, accumulation provides a way to reach a chosen effective batch size while keeping the resident micro-batch within the conditional V100 memory budget.

Napkin Math 8.8: GPT-2 gradient accumulation strategy

This illustrative GPT-2-scale scenario uses assumed prices and durations to show gradient accumulation; it does not reconstruct GPT-2's historical training bill.

Memory Constraints

- A V100 with 32 GB has a modeled tensor budget for a per-GPU micro-batch of $B_{\text{micro}} = 4$, admitted by the segment-checkpoint ledger and subject to verifying workspace headroom, as discussed in notebook 8.6.
- A resident effective batch of $B = 512$ would therefore require 512 ÷ 4 = 128 GPUs.

Accumulation Configuration

- Use 8 GPUs and accumulate 16 micro-batches of 4 examples each.
- Each accelerator processes 4 × 16 = 64 examples per update, giving a global effective batch of 8 GPUs × 64 = 512.

A distributed `no_sync()` context can defer AllReduce until the final micro-batch.

Trade-off Analysis

- Idealized wall-clock time: 16× longer because fewer accelerators perform the same arithmetic work
- Memory overhead: accumulation normally reuses parameter-gradient buffers, although bookkeeping and temporary storage remain
- Communication: both configurations synchronize once per 512-sample update, but participant count and topology differ
- Equal-work cost: $129K either way, before utilization and communication effects

Why this works: Under the equivalence conditions stated above, averaged per-example gradients are additive. Each accelerator first accumulates a local gradient over its 64 examples:

$$\frac{1}{16}\sum_{j=1}^{16}\left[\frac{1}{4}\sum_{i=1}^{4}\nabla\mathcal{L}(x_{ji})\right]$$

AllReduce then averages those local gradients across 8 GPUs, producing the global effective batch:

$$\frac{1}{8}\sum_{r=1}^{8}\nabla\mathcal{L}^{(r)}_{\text{local}}, \qquad B = 8 \times 64 = 512$$

Operational comparison

Resident configuration:

- 128 GPUs $\times$ 4 examples = 512 resident examples per update
- Gradient synchronization spans 128 GPUs
- Hourly cost: 128 GPUs $\times$ \$3/hour = \$384/hour

Accumulated configuration:

- 8 GPUs $\times$ (4 examples $\times$ 16 micro-batches) = 512 examples per update
- Gradient synchronization occurs once per update across 8 GPUs
- Hourly cost: 8 GPUs $\times$ \$3/hour = \$24/hour
- Hourly reduction: \$360/hour, or 93.8 percent

Quality gate

- Match the examples and their order, loss normalization, optimizer state and step schedule, and stochastic-layer behavior.
- Compare held-out loss or perplexity and the optimization trajectory across repeated runs rather than assuming that the effective batch alone guarantees equivalent quality.
- Expect small round-off differences from the changed summation order; require convergence within a declared tolerance rather than bitwise identity. No perplexity result is assumed here.

Systems insight: Accumulation lowers accelerator count and hourly burn, but ideal scaling preserves accelerator-hours; actual cost depends on utilization and communication.

Listing 8.7 places one AllReduce after sixteen local micro-batches, making the synchronization boundary explicit.

Listing 8.7: Gradient Accumulation Training Loop: Sixteen local micro-batches accumulate before one synchronization boundary and optimizer update; the abstract AllReduce may represent multiple gradient buckets.

```
optimizer.zero_grad()
for step in range(16):  # Accumulation steps
    micro_batch = next(dataloader)  # 4 samples
    loss = model(micro_batch) / 16  # Average effective-batch loss
    loss.backward()  # Accumulate gradients
# Now gradients represent 64 local samples
all_reduce(gradients, op=SUM)  # Sum across 8 GPUs
gradients /= 8  # Convert the sum to the global mean
optimizer.step()  # Update with effective batch=512
```

8.7.5.4 Practical considerations

Gradient accumulation is most valuable when optimal batch sizes exceed GPU memory capacity. Transformer language models often use effective batches of hundreds of thousands to millions of

tokens; if memory only permits a smaller number of sequences per device, accumulation bridges the gap without requiring additional hardware. Activation checkpointing complements this by enabling deeper architectures and is commonly used in transformer-scale training, as well as in activation-heavy dual-network configurations such as generative adversarial networks.

Both techniques introduce explicit trade-offs. Activation checkpointing recomputes discarded activations during backward propagation; with ordinary segment checkpointing, each discarded activation is typically reconstructed once, while recursive schedules may behave differently. Gradient accumulation reduces parameter-update frequency because each update follows k micro-batches. When each micro-batch loss is divided by k, the accumulated gradient follows the effective-batch mean convention. If losses are summed instead, the gradients must be rescaled before the optimizer step or the learning-rate convention changed accordingly. Framework and codebase conventions differ, making normalization a common source of subtle bugs.

8.7.6 Optimization technique comparison

Each training optimization targets a distinct physical bottleneck within the accelerator memory and compute hierarchy. Asynchronous prefetching pipelines host-to-device transfers across the PCIe bus with accelerator execution, preventing compute engines from idling on input data starvation. Mixed-precision training halves the bytes moved across the HBM bus for every 16-bit tensor and shrinks stored activations, while routing matrix arithmetic to specialized low-precision tensor units. FlashAttention complements these system-level interventions by resolving the quadratic memory-bandwidth bottleneck of self-attention; by tiling queries, keys, and values into on-chip SRAM, it reduces auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ and reports speedups of up to $3\times$ on evaluated workloads. Finally, activation checkpointing and gradient accumulation manage device memory capacity: checkpointing discards intermediate activations to reconstruct them via backward recomputation, while gradient accumulation partitions a large logical batch into temporal micro-batches to bound peak resident memory.

Because training throughput is governed by the system's binding constraint, applying an optimization to a non-binding subsystem yields no end-to-end speedup. Accelerating tensor arithmetic with lower precision or FlashAttention cannot reduce step time if compute pipelines remain stalled waiting for batches from the host. Conversely, expanding prefetch buffers provides no benefit when execution is bound by HBM capacity or backward-pass recomputation. Profiling must therefore identify whether the workload is bound by I/O ingest, memory capacity, memory bandwidth, or arithmetic throughput before an intervention is selected.

Table 8.17 compares three of these foundational strategies across their mechanisms, hardware requirements, and operational trade-offs, with FlashAttention serving as an algorithm-level complement within the attention kernel itself.

Table 8.17: Optimization Strategies: The table compares prefetching, mixed precision, and the combined use of gradient accumulation and checkpointing. The first targets input stalls, the second changes storage and eligible arithmetic, and the third trades smaller resident activation sets for micro-batch or recomputation overhead.

Aspect	Prefetching and Overlapping	Mixed-Precision Training	Gradient Accumulation and Checkpointing
Primary Goal	Minimize data transfer delays and maximize accelerator utilization	Reduce tensor storage and accelerate eligible lower-precision operations	Overcome memory limitations during backpropagation and parameter updates
Key Mechanism	Asynchronous data loading and parallel processing	Combining lower- and higher-precision computation	Simulating larger batch sizes and selective activation storage
Memory Impact	Increases memory usage for prefetch buffer	Reduces activations and communicated tensors; static state unchanged	Reduces resident activations; parameter-gradient size is unchanged
Computation Speed	Improves by reducing idle time	Can accelerate eligible FP16/BF16 operations on supported hardware	May slow down due to recomputations in checkpointing
Scalability	Highly scalable, especially for large datasets	Enables training of larger models	Allows training deeper models on limited hardware
Hardware Requirements	Benefits from fast storage and multi-core CPUs	Supported lower-precision accelerator paths	Works on standard hardware

Aspect	Prefetching and Overlapping	Mixed-Precision Training	Gradient Accumulation and Checkpointing
Implementation Complexity	Moderate (requires tuning of prefetch parameters)	Low to moderate (with framework support)	Moderate (requires careful segmentation and accumulation)
Main Benefits	Reduces training time, improves hardware utilization	Faster training, larger models, reduced memory usage	Enables larger batch sizes and deeper models
Primary Challenges	Tuning buffer sizes, increased memory usage	Potential numerical instability; FP16 may require loss scaling	Increased computational overhead, slower parameter updates
Ideal Use Cases	Large datasets, complex preprocessing	Large-scale models, especially in NLP and computer vision	Memory-constrained batches or activation-heavy models

The table compares techniques in isolation; the GPT-2 walkthrough shows how those techniques combine when one model must fit in memory, run fast enough, and stay within a realistic cost envelope.

8.7.7 GPT-2 optimization walkthrough

Training GPT-2 (1.5 billion parameters) on a single V100 GPU exposes the physical boundaries that govern large-model optimization: tensor allocations immediately exceed accelerator memory capacity, while naive input pipelines starve execution engines. Tracing this single-device diagnostic progression separates memory feasibility from the execution evidence needed before scaling to a multi-accelerator cluster.

Napkin Math 8.9: GPT-2 optimization on V100

Step 1: Establish the baseline memory footprint.

The trace starts with GPT-2 XL with 1.5 billion parameters, batch size 4, sequence length 1024, FP32 tensors throughout, and a single-threaded synchronous data loader. This is a deliberately constrained configuration: small enough to discuss on one V100, but large enough that the tensor-memory estimate already proves the baseline infeasible. This walkthrough uses the batch size 4 admitted by the mixed-precision callout (napkin math 8.6); table 8.19 reports the batch-32 equivalents (597.8 GB FP32 baseline, 60.7 GB optimized), which exceed the comparison V100 even after optimization. At batch size 4, the FP32 baseline needs 95.7 GB, so the run fails the machine constraint on a 32 GB V100 before throughput matters.

Step 2: Apply mixed precision.

The first intervention targets memory feasibility rather than speed. This modeled mixed-precision recipe uses 16-bit weights and gradients, FP32 master weights and Adam moments, and an assumed halving of saved activation storage. Its static state remains 16 bytes per parameter; actual autocast policies select operation dtypes and need not use this exact storage recipe. Halving the activation footprint reduces the modeled tensor allocation to 59.9 GB, a 37.5 percent reduction, but the allocation still exceeds the physical capacity of a 32 GB accelerator.

Step 3: Add gradient checkpointing.

Holding the static-state representation fixed, the segment schedule in notebook 8.6 retains eight segment-input tensors and budgets the saved state of at most six live reconstructed layers. Its 7.8× activation-memory reduction brings the batch-4 tensor total to 28.6 GB, below the V100's 34.4 GB on the same decimal basis. Reconstructing each segment once adds one forward pass, about 33.3 percent more arithmetic than the unrecomputed step. Runtime measurement must verify that backward workspaces, output/loss buffers, and allocator overhead fit in the remaining capacity; the arithmetic ratio alone does not predict the time penalty.

Step 4: Examine an illustrative throughput profile.

Fitting the model into device memory resolves capacity, but does not guarantee efficient execution. Assume a profile of the newly executable model shows exposed input waits and the following breakdown of iteration time across subsystems:

- Accelerator busy-time fraction (equation 8.9): 45 percent
- Data loading: 40 percent of iteration time
- Compute: 35 percent of iteration time
- Memory transfers: 25 percent of iteration time

These assumed profile observations shift the diagnostic focus from machine memory capacity (M) to the data pipeline (D) in D·A·M terms. With 40 percent of iteration time spent waiting for host-prepared batches, reducing accelerator memory consumption alone does not address the dominant exposed delay. Improving input delivery is the next target in this scenario.

Step 5: Apply prefetching and data pipeline optimization.

Resolving input starvation requires overlapping host data preparation with accelerator computation. In this scenario, configuring the data loader with eight worker processes parallelizes CPU-side sample decoding, page-locked (pinned) host memory enables asynchronous DMA transfers across the PCIe bus, and a prefetch factor of 2 queues upcoming batches per worker in host memory. Device double buffering requires a separate copy-stream and buffer arrangement. Hiding host preprocessing and transfer latency behind kernel execution raises the accelerator's busy-time fraction from 45 percent to 85 percent, reducing the assumed input starvation. A busy accelerator is not yet an efficient one; MFU measures how much of that busy time does useful model arithmetic. Re-profiling must establish whether memory bandwidth, arithmetic throughput, or another stage now limits execution.

Table 8.18 contrasts the unexecutable naive baseline against the optimized single-GPU configuration.

Table 8.18: Illustrative GPT-2 Optimization Profile: The batch-4 memory figures are calculated by the walkthrough; utilization and throughput are assumed scenario values rather than benchmark results, and the time row follows from the assumed throughput. At about 9.6 days per billion tokens, a multi-billion-token corpus takes months on one V100, which is why the single-GPU configuration establishes feasibility rather than a practical schedule. Table 8.19 reports batch-32 scenario totals.

Metric	Naive	Optimized	Improvement
Memory	95.7 GB	28.6 GB	3.3× reduction
Accelerator busy time	N/A	85%	Trainable
Throughput	N/A	1,200 tokens/s	—
Time per billion tokens	N/A	231 hours	—

Systems insight: The order of interventions follows a dependency, not a preference. Memory feasibility comes first because a configuration that does not fit produces no profile at all. Once it runs, the profile decides what comes next; in this scenario exposed input waits made the data pipeline the target ahead of any kernel tuning, because a faster kernel cannot shorten a step spent waiting for its input. After each intervention, re-profiling determines whether input delivery, memory bandwidth, arithmetic throughput, or another execution cost now limits progress.

This walkthrough demonstrates why training optimization is a sequential diagnostic discipline rather than an arbitrary collection of techniques. Each intervention addresses a constraint established by the memory calculation or assumed profile. In this recipe, mixed precision halves activation storage and reconstructing each segment once adds about 33.3 percent to the step arithmetic. Together these interventions deliver a 3.3× modeled memory reduction that satisfies the tensor budget. Asynchronous prefetching then addresses exposed input waits once execution begins. With a candidate single-accelerator configuration established, the next task is evaluating how these compounding optimizations scale across a multi-accelerator cluster in execution time, energy consumption, and total training cost.

Checkpoint 8.3: From a feasible configuration to a validated run

The walkthrough combines a memory calculation with an illustrative execution profile. Use the distinction to decide what an actual run must verify.

- ☐ **Evidence**: Which walkthrough values are calculated from tensor sizes, which are assumed profile values, and which require measurement on the target system?
- ☐ **Capacity and throughput**: Why does fitting the tensor budget establish neither peak runtime memory nor efficient execution, and what must be measured next on hardware?
- ☐ **Quality gate**: Before accepting the composed optimizations, how does an engineer verify that the revised precision and effective batch preserve progress toward the same validation target?

8.7.8 Optimization impact summary

The GPT-2 case study demonstrates how targeted techniques can compose. The memory rows below extend its tensor ledger to one unpartitioned model processing a batch of 32 sequences simultaneously; both totals still exceed one V100's capacity. The cluster rows provide a separate illustrative operating scenario on 32 V100s, whose assumed training durations, power, electricity price, and grid carbon intensity determine energy and cost. Connecting that scenario to an executable training configuration requires choosing how batches and model state are distributed across devices, the scaling decision developed next.

Table 8.19 compiles the tensor-memory comparison and the separate cluster operating scenario.

Table 8.19: Illustrative GPT-2 Training Scenario: The tensor model gives the stated memory totals. Training duration, constant cluster power, PUE, electricity price, and grid carbon intensity are scenario assumptions; under those assumptions, shorter runtime reduces energy and location-based emissions proportionally.

Metric	FP32 Baseline	Optimized	Technique Applied
Parameters	6 GB	3 GB	Mixed precision (FP16)
Gradients	6 GB	3 GB	Mixed precision (FP16)
Master Weights	0 GB	6 GB	AMP overhead; cancels the two FP16 savings above
Optimizer State (Adam)	12 GB	12 GB	Unchanged (FP32 moments)
Activations (batch=32)	573.8 GB	36.7 GB	Gradient checkpointing + FP16
Total Memory	**597.8 GB**	**60.7 GB**	—
Training Time (32 V100s)	14 days	8.4 days	Data prefetching + utilization
Energy Consumption	3,914 kWh	2,348 kWh	Assumed constant cluster power over each duration
Electricity Cost ($0.10/kWh)	$391.4	$234.8	—
Carbon Footprint	1.7 t	1.0 t	Regional grid average (0.43 kg/kWh)

As table 8.19 shows, the assumed inputs produce a 10× memory ratio, a 1.7× wall-clock ratio, and a 40 percent energy reduction under constant modeled power. In practice, energy and cost must be computed from measured runtime and power because higher utilization can change the cluster's power draw.

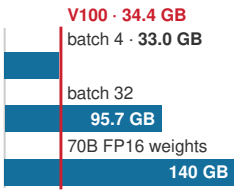
Single-accelerator optimizations address complementary constraints: mixed precision enables higher-throughput arithmetic, FlashAttention avoids materializing full attention matrices in HBM, gradient checkpointing reduces activation footprints, and asynchronous prefetching can hide host I/O latency. Even when fully optimized, however, workloads with hundreds of exaFLOP budgets remain bounded by the arithmetic throughput of an individual chip.

Scaling beyond single-device compute or memory capacity requires distributing computation across multiple accelerators. This architectural transition introduces inter-device communication

overhead, collective synchronization barriers, and fault-tolerance constraints that become the primary determinants of cluster scaling efficiency.

8.8 Scaling Training Systems

The GPT-2 walkthrough models how mixed precision and checkpointing reduce a batch-4 tensor budget to 28.6 GB, just under a V100's 34.4 GB capacity before workspaces and allocator overhead. At batch 32, the modeled total falls from 597.8 GB to 60.7 GB but still exceeds that V100. Larger-memory accelerators may move the boundary. A 70-billion-parameter model, however, requires about 140 GB for FP16 weights alone and substantially more for full-parameter training state, motivating sharding, offload, or multiple devices.



GPT-2 at batch 4 fits just under one V100; batch 32 and a 70B model cross the wall.

When a chosen workload still exceeds one device's memory or schedule, spreading it across devices provides aggregate memory and compute. Other responses include reducing the model, batch, sequence length, or training scope; offloading state; and parameter-efficient adaptation, which trains only a small set of added parameters on top of a frozen model. Scaling beyond one device begins with multi-GPU configurations inside one machine and may extend across nodes. The major parallelism strategies make different communication trade-offs; a full treatment of distributed-training implementation lies beyond this chapter.

Not all workloads benefit equally from adding GPUs. The relationship among useful computation, communication, imbalance, and synchronization determines scaling efficiency. Figure 8.13 charts this scaling decay on a log-log speedup plot up to 256 GPUs, contrasting ideal linear scaling against compute-bound (ResNet), balanced (a large language model (LLM) over NVLink), and bandwidth-bound regimes. The widening divergence between the ideal linear dashed reference and the plateauing modeled curves illustrates the communication tax paid as synchronization overhead compounds. In this fixed-fraction model, even unlimited parallel compute leaves the non-scaling fraction of the original runtime, so the limiting speedup is its reciprocal.

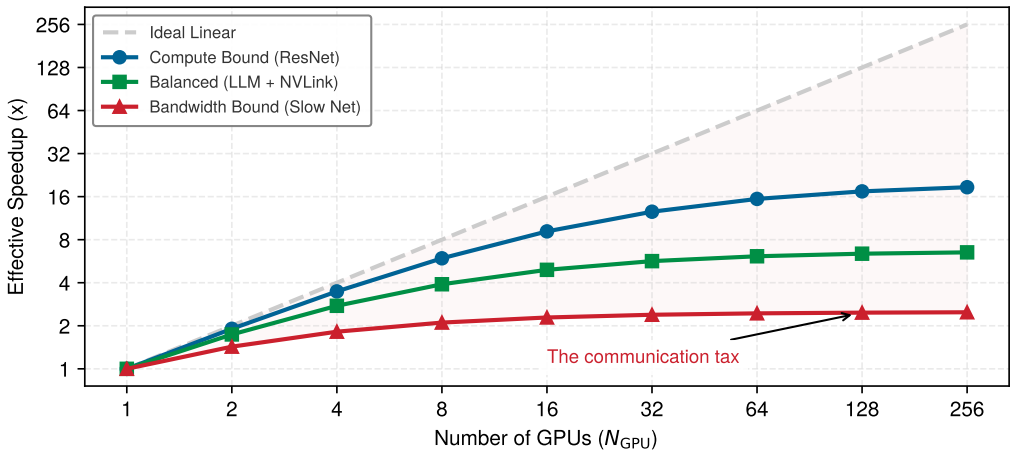


Figure 8.13: Communication Tax: An Amdahl-style model compares ideal linear scaling against workloads with varying non-scaling communication and synchronization fractions. Real communication overhead scales dynamically with device count, network topology, message size, and overlap efficiency.

The red curve's widening gap from the dashed ideal is the communication tax made visible; shrinking that gap is what the distributed strategies in section 8.8.1 exist to do.

8.8.1 Single-node multi-GPU training

Multi-GPU training within a single node establishes the foundational architectural patterns for distributed execution. AlexNet[22] (Krizhevsky et al. 2012) split its convolutional layers across two GTX 580 GPUs because the 3 GB device memory limit prevented single-chip execution, making cross-GPU connectivity an immediate architectural constraint. Confining multi-accelerator topologies to

22 **AlexNet (2012)**: The paper reports 60M parameters and a 3 GB single-GPU ceiling, but does not attribute the limit to one memory component. Its model-parallel design placed roughly half the kernels on each GPU and allowed communication only in selected layers, explicitly trading memory capacity against communication.

single-node interconnects (such as PCIe or NVLink) isolates the core parallelism strategies before introducing cross-rack network fabrics.

The two foundational strategies, data parallelism and model parallelism, represent fundamentally different partitioning choices: data parallelism replicates the model and partitions data, while model parallelism partitions model state or computation. This distinction determines memory requirements, communication patterns, and scaling behavior.

8.8.1.1 Data parallelism

Data parallelism replicates the entire model on each GPU, with each processing different batches. After computing gradients locally, GPUs synchronize via gradient averaging. Figure 8.14 diagrams this distributed execution flow: input batches split across four parallel GPU workers for independent forward and backward passes, converge into an AllReduce collective for gradient synchronization, and apply identical parameter updates across all replicas.

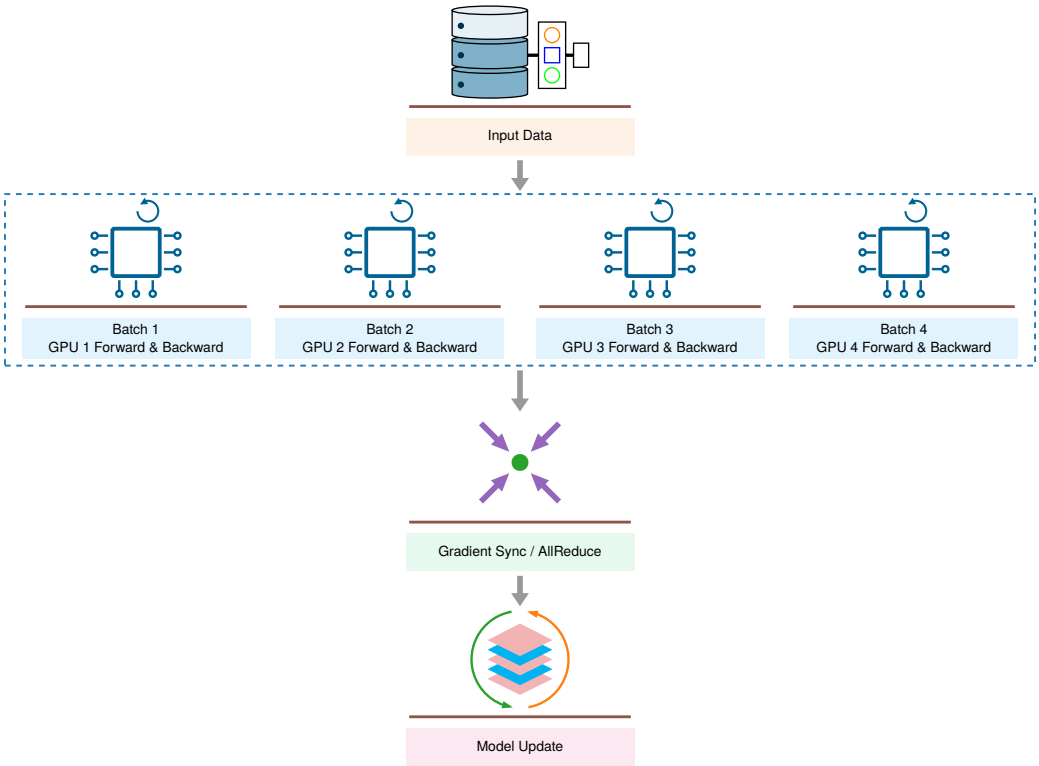


Figure 8.14: Data Parallelism: Each GPU holds a complete model copy, processes a different data shard, and synchronizes gradients. Throughput can approach linear scaling only while computation dominates synchronization, input, and imbalance costs.

When a model and its optimizer state fit comfortably within single-device memory, data parallelism serves as the primary scaling baseline because local forward and backward kernels run at native memory bandwidth without inter-device synchronization. Gradient aggregation is the principal recurring communication cost; some implementations also synchronize buffers or other state. Historically, frameworks supported multi-GPU execution within a single node using multithreaded wrappers like PyTorch's `DataParallel` (DP). However, this single-process design suffers from two severe architectural bottlenecks: the Python runtime remains throttled by the Global Interpreter Lock (GIL), and a primary coordinator (GPU 0) must sequentially scatter mini-batches and gather outputs over PCIe, saturating host-device interconnect bandwidth and creating severe compute and memory imbalances. Modern systems commonly use PyTorch's `DistributedDataParallel` (DDP); TensorFlow's `MirroredStrategy` also provides replicated training but has a different execution

model. With an independent Python process per accelerator, DDP avoids cross-replica contention for one Python GIL; each process maintains its own model replica and local optimizer, relying on decentralized collective AllReduce operations that bucket gradients into contiguous buffers and overlap synchronization directly with backward-pass computation.

Unsharded data parallelism has a hard constraint: every GPU holds a complete model replica and ordinarily its local optimizer state. A 7-billion-parameter model uses 14 GB for FP16 weights alone, before gradients, optimizer states, or activations. Sharded data parallelism (such as the Zero Redundancy Optimizer, or ZeRO) distributes that state across devices: ZeRO-1 shards optimizer states, ZeRO-2 shards gradients, and ZeRO-3 (or FSDP) shards model parameters; otherwise, a model that exceeds local memory requires pipeline or tensor partitioning.

8.8.1.2 Model parallelism

Model parallelism partitions the model itself across GPUs, which becomes necessary when the model exceeds single-GPU memory. AlexNet adopted an early form of this strategy: selected convolutional layers split their kernels and channels across two GPUs, with communication at selected connections. Figure 8.15 illustrates a different, layerwise partition of the same broader model-parallel idea: Device 1 through Device 3 sequentially process layers 1–16, 17–32, and 33–48, streaming boundary activations forward during the forward pass and propagating loss gradients in reverse during backpropagation.

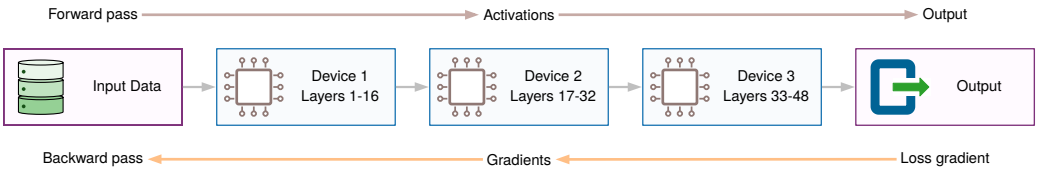


Figure 8.15: Model Parallelism: The model is partitioned across devices, with intermediate activations passing between them. This enables training models larger than single-GPU memory at the cost of sequential dependencies.

> **Systems Perspective 8.7: Data vs. model parallelism**
>
> When a selected workload is partitioned across N_{GPU} devices, the split may divide examples, model state, layers, or tensor operations. For a model with parameters P and batch size B, data and model parallelism provide two starting points.
>
> **Data parallelism (split the batch)**: For a fixed global batch, each of N_{GPU} workers processes about $1/N_{\text{GPU}}$ of the examples, but every worker holds a model replica and ordinarily its associated training state. Dense gradient communication is proportional to P per update. Scaling departs from the ideal as local batches shrink or synchronization and input costs grow.
>
> **Model parallelism (split model computation)**: Each accelerator stores and computes a portion of the model. An even partition may approach P/N_{GPU} parameters per worker, but embeddings, buffers, imbalance, and replicated state alter that estimate. Communication depends on the partition: layerwise pipeline parallelism exchanges activations at stage boundaries, while tensor parallelism performs collectives within partitioned layers.
>
> **Systems insight**: Data parallelism is the simplest candidate when the complete training state fits per worker and more throughput is needed. Model-state, pipeline, or tensor partitioning becomes necessary when it does not, often in combination with data parallelism.

When a model exceeds the memory capacity of a single accelerator, its depth can be partitioned across sequential devices. Figure 8.16 stages this pipeline allocation across a four-accelerator execution chain, grouping 24 transformer blocks into contiguous six-block partitions (Blocks 1–6 through 19–24) that exchange boundary activations over the interconnect.

The fundamental bottleneck in naive layer-wise partitioning is device serialization. Because layer evaluation is strictly sequential, Device 1 computes layers 1–6 while Devices 2–4 sit idle waiting

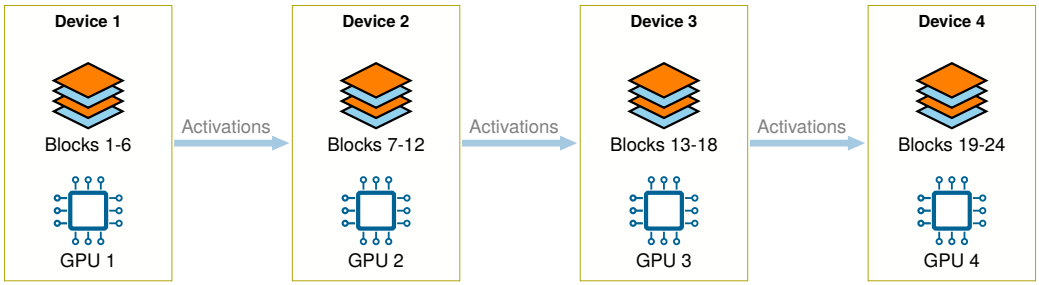


Figure 8.16: Layer-Wise Model Partitioning: Distributing consecutive transformer layers across sequential devices bounds network communication to intermediate activations (forward) and boundary gradients (backward). However, sequential layer dependencies leave downstream devices idle during forward passes unless micro-batches are pipelined.

for forward activations. During the backward pass, the dependency reverses: Device 4 computes first while upstream devices wait for returning gradients. This idle duration constitutes the *pipeline bubble*, which caps theoretical hardware utilization at $1/N_{\text{GPU}}$ under naive sequential execution. It differs from the accelerator bubbles of the staged training pipeline, which arise when one device's data-loading, transfer, and compute stages run at mismatched rates; a pipeline bubble arises even with perfectly fed devices, because each device must wait for another device's layers. Mitigating this idle time requires either pipelining micro-batches across stages or partitioning tensor dimensions within layers so that all devices compute simultaneously.

Within a node, GPUs may communicate through high-bandwidth links such as NVLink[23] (up to 900 GB/s aggregate bidirectional bandwidth for the represented H100 configuration). Effective collective bandwidth depends on topology and contention and is lower than a raw aggregate link specification. Data parallelism communicates gradients via collective AllReduce operations, while layerwise model parallelism communicates activations point-to-point at partition boundaries. Choosing a strategy requires balancing these communication patterns and memory footprints against the physical interconnect bandwidth of the node topology.

[23] **NVLink**: NVIDIA's high-bandwidth GPU interconnect. Its topology and generation determine the bandwidth available between a particular pair of GPUs and to collective operations. A parallelism strategy helps only when its communication and synchronization cost is smaller than the useful time it saves.

8.8.2 Scaling beyond a single node

The cross-node analysis provides optional depth on the boundary beyond single-machine training. When a model's parameter footprint, optimizer state, or training duration exceeds the capacity of an 8-accelerator node, distributed training extends across machines. This step introduces a fundamental bandwidth hierarchy: while accelerators within a node communicate across dense, high-bandwidth fabrics such as NVLink, distributed workers must synchronize over external network fabrics (such as Ethernet or InfiniBand) that offer substantially lower bandwidth, higher latency, and switch contention. The resulting bandwidth gap turns synchronous data-parallel gradient reduction into a recurring data-movement bottleneck on the critical path.

> **Systems Perspective 8.8: The physics of synchronization**
>
> Recall the energy cost of data movement quantified in table 4.1, where a DRAM access, a local SSD read, and a network transfer each cost progressively more energy than the arithmetic performed on the value they deliver. In distributed training, this asymmetry establishes the communication tax.
>
> Synchronizing gradients across accelerators moves megabytes to gigabytes across an interconnect on every training step. Uncovered communication time directly reduces hardware utilization, while transceiver and switch power inflates the cluster's energy budget; these are distinct system costs. Mixed-precision execution (section 8.7.3) and gradient compression reduce transfer volumes through narrower numeric formats, sparsity, quantization, or delta encoding, relieving both bounds.

Within a node, high-bandwidth interconnects permit gradient synchronization to overlap with backward computation. Crossing node boundaries, however, transitions communication to a network

fabric where available bandwidth is one to two orders of magnitude lower. In synchronous data parallelism, AllReduce[24] aggregates gradients across workers; when network bandwidth fails to match accelerator compute throughput, uncovered communication time dominates the step and stalls arithmetic execution.

24 **AllReduce**: A collective communication primitive that sums data across all devices and distributes the result back to each device. Ring AllReduce is one common implementation: devices pass gradient shards around a logical ring so each participant sends and receives bounded chunks rather than a full copy of every gradient tensor at once. This is the communication primitive that makes large data-parallel training practical, but its detailed cost model belongs with distributed training.

> Napkin Math 8.10: The network wall
>
> **Problem**: A team scales training to 8 GPUs, one per node, connected by a 100 Gbps fabric. Is the network the bottleneck?
>
> **Mechanism**: For a 7B-parameter model with FP16 gradients:
>
> 1. **Gradient size** (FP16): 14 GB per step.
> 2. **AllReduce cost**: Ring AllReduce passes gradient shards around a logical ring, so each worker ends up sending nearly two full copies of the gradient: $2(N_{\text{GPU}} - 1)/N_{\text{GPU}}$, which equals 1.75× for 8 GPUs. That gives 1.75 × 14 GB = 24.5 GB sent per worker, with the same amount received separately.
> 3. **Bandwidth-only lower bound**: At 12.5 GB/s per worker on the 100 Gbps fabric, synchronization latency for the D_{vol} bytes each worker sends is $T_{\text{comm}} = D_{\text{vol}}/\text{BW}_{\text{network}} = 24.5$ GB / 12.5 GB/s $= 1.96$ s.
> 4. **Compute comparison**: If the nonoverlapped forward and backward work takes $T_{\text{compute}} \approx 1$ s, communication latency $T_{\text{comm}} \approx 1.96$ s dominates.
>
> **Systems insight**: The network becomes a wall when $T_{\text{comm}} > T_{\text{compute}}$, reducing the fraction of this nonoverlapped step spent computing, given by $\frac{T_{\text{compute}}}{T_{\text{compute}}+T_{\text{comm}}}$ to about 33.8 percent. This fraction does not measure compute efficiency relative to peak FLOP/s. Ring AllReduce (Sergeev and Balso 2018) is already the baseline here. Further mitigations include gradient compression (reducing D_{vol}), overlap where dependencies permit, and hierarchical topologies that confine high-frequency transfers to fast intra-node links (NVLink at 900 GB/s vs. 12.5 GB/s inter-node).

Because cross-node fabrics throttle frequent synchronization, scaling requires matching each communication pattern to the appropriate bandwidth tier. Pipeline parallelism partitions model layers across sequential devices, addressing the idle pipeline bubbles of naive model parallelism through micro-batching. Instead of processing an entire batch serially, the global batch is divided into smaller micro-batches that stream through the pipeline stages. While a downstream stage executes a forward pass on an early micro-batch, upstream stages concurrently process subsequent ones. The pipeline schedule controls this concurrency: retaining more in-flight micro-batches reduces accelerator idle time but requires each stage to store activations in memory until the corresponding backward pass completes. GPipe[25] evaluates all forward micro-batches before draining the backward passes, whereas PipeDream interleaves one forward pass with one backward pass (1F1B) to bound resident activation memory (Huang et al. 2019; Narayanan et al. 2019).

25 **GPipe (2019)**: GPipe implements the described micro-batching strategy, staggering execution to fill the idle "bubbles" in naive model parallelism and thus boost utilization (Huang et al. 2019). The key trade-off it manages is increased memory for storing the activations of multiple in-flight micro-batches. It preserves large-batch training dynamics by accumulating gradients before the weight update; the paper reports almost linear Transformer throughput scaling when the micro-batch count is at least four times the number of partitions.

Tensor parallelism operates at a finer granularity: rather than assigning whole layers to separate devices, it shards individual matrix operations across devices within a layer. (Sequence parallelism extends this principle by sharding activation tensors along the sequence dimension during long-context transformer training.) In a transformer feed-forward block, Megatron-LM splits the first weight projection column-wise and the second projection row-wise (Shoeybi et al. 2019). Each accelerator computes on a slice of the hidden dimension without intermediate communication; only a single AllReduce across the tensor-parallel group is required at the end of the block to sum the partial outputs. Because this pattern introduces frequent forward and backward collectives, tensor parallelism benefits from high-bandwidth links such as NVLink. Intra-node placement is a common choice; the actual communication pattern and topology determine whether crossing nodes is practical.

Hybrid strategies combine tensor, pipeline, and data parallelism (3D parallelism) to scale large models across physical clusters. Table 8.20 summarizes the resulting placement heuristic: map the highest-frequency collective communication to the highest-bandwidth physical links, reserving lower-bandwidth tiers for coarser synchronization boundaries.

Table 8.20: Hybrid Parallelism Placement: A common heuristic places tensor parallelism within nodes, pipeline parallelism within racks, and data parallelism across racks. Actual placement depends on the model and topology.

Strategy	Typical placement	Communication pattern
Tensor parallelism	Within a node	Frequent communication that exploits NVLink's high bandwidth
Pipeline parallelism	Across nodes within a rack	Moderate communication at layer boundaries
Data parallelism	Across racks	Gradient synchronization once per iteration

Partitioning computation across a cluster ultimately transforms training from a purely arithmetic problem into a multi-tier data-movement hierarchy. While collective communication algorithms, parameter servers, and fault-tolerant scheduling allow models to scale across thousands of accelerators, their sustained efficiency remains bound by the ratio of compute throughput to network transfer time. The remaining decision is whether the workload's memory or schedule requires paying that communication cost.

8.8.3 When to scale: The physical ceiling

↪ Empirical protocols for quantifying multi-accelerator scaling efficiency and communication loss are operationalized in section 12.7.2.2.

A run needs more devices only when its state or schedule exceeds what the chosen local configuration can provide. Knowing how to partition computation across devices does not answer *when* distributed execution is warranted. Distributing state introduces communication overhead, synchronization barriers, and operational failure modes, so the single-node optimizations come first, because they isolate whether the bottleneck is fundamentally algorithmic or capacity-driven. Profile-guided tuning should evaluate lower-precision execution paths (section 8.7.3) when numerical stability permits, gradient accumulation (section 8.7.5) to synthesize target batch sizes without activation blowup, activation checkpointing (section 8.7.5.1) to trade redundant forward compute for memory headroom, and overlapped data prefetching (section 8.7.2) when host ingestion limits the critical path.

Table 8.21 structures this progression as a resource lookup keyed to measured hardware bottlenecks rather than arbitrary parameter thresholds.

Table 8.21: Scaling Decision Guidelines: Choose the least distributed configuration that satisfies measured memory, throughput, and schedule requirements. Model architecture, training state, input pipeline, hardware, and topology all affect the boundary.

Observed constraint	Candidate approach	Rationale
Training state fits one device and time is acceptable	Single GPU	Minimizes communication and operational complexity
State or throughput exceeds one device but fits one node	Single multi-GPU node	Uses fast local interconnects before crossing the network
Required state or schedule exceeds one node	Multi-node cluster	Adds aggregate memory or compute at a communication cost
Input service rate is below accelerator demand	Parallel or distributed input pipeline	Adds input throughput where profiling shows it is needed

A measured single-device baseline guides scaling decisions, while a decisive capacity or compute-time lower bound can establish the need for additional resources before every local optimization is attempted. Every hardware device has a physical ceiling: a workload requiring 10^{24} FLOPs cannot be completed on a single accelerator within a practical training schedule, no matter how carefully that accelerator is tuned. The resource expansion must match the hard limit that has been reached:

- **Memory exhaustion**: The model weights, gradients, optimizer states, and activations exceed the VRAM of a single GPU. A 70-billion-parameter model requires approximately 140 GB in FP16 for weights alone, exceeding the 80 GB configurations of A100 and H100 SXM accelerators and leaving little or no headroom on higher-capacity variants like the H100 NVL (94 GB) and H200 (141 GB) once training state is included.

- **Training wall-clock time**: The estimated time on one device exceeds the project's actual deadline. At 10^{15} FLOP/s sustained, a workload requiring 10^{24} FLOPs would take about 32 years.
- **Input throughput**: The input pipeline cannot sustain the aggregate accelerator consumption rate. This calls for more input-service capacity, not automatically more training accelerators. Dataset size alone does not determine this limit; reuse, compression, locality, transformation cost, and storage bandwidth matter.

These three limits are technical, but scaling carries a cost beyond complexity and capacity. Adding devices amplifies the energy consumption and environmental impact of training, so efficiency optimization becomes an environmental concern as much as a performance one.

Napkin Math 8.11: The carbon footprint of training

Context: Training large models requires substantial energy and compute. The physical ceiling appears once training time is converted to energy, cluster power multiplied by training time ($E_{\text{total}} = P_{\text{cluster}} \cdot T_{\text{train}}$), quantifying environmental scale:

1. Workload: Training a 7 billion-parameter model for 1 trillion tokens.
2. Compute: $O \approx 4.2 \times 10^{22}$.
3. Efficiency: 156 TFLOP/s sustained on A100 (400 W thermal design power).
4. **Execution time**: $T_{\text{train}} \approx 3.04$ days on 1,024 GPUs.
5. **Energy footprint**: $E_{\text{total}} = P_{\text{cluster}} \cdot T_{\text{train}} = (N_{\text{GPU}} \cdot P_{\text{device}} + N_{\text{nodes}} \cdot P_{\text{host}}) \cdot T_{\text{train}}$ evaluates to (1,024 GPUs $\times$ 400 W + 128 hosts $\times$ 200 W) $\times$ 73 hours $\approx$ 31,784.2 kWh.

Impact: Under the assumed constant GPU and host-CPU power, this modeled subtotal equals 35.3 months of average household electricity use. GPU thermal design power is a scenario input rather than measured draw; memory, storage, network equipment, and facility overhead are not separately included, so the subtotal is not total facility energy.

Systems lesson: Doubling hardware utilization η_{hw} at constant cluster power P_{cluster} halves execution time T_{train} and total energy E_{total}. Under average grid carbon intensity, doubling efficiency saves ~15,892.1 kWh of electricity and avoids 6.8 t of emissions. Environmental audits must measure $E_{\text{total}} = \int P_{\text{cluster}}(t)\, dt$ rather than relying on GPU utilization metrics alone.

8.8.4 Historical context: The evolution of training infrastructure

The parallelism strategies in section 8.8.1 and section 8.8.2 represent the convergence of distinct architectural traditions. Modern training platforms integrate paradigms developed across four computing eras, each shaped by the physical constraints and dominant workloads of its time.

Neural network training combines requirements drawn simultaneously from high-performance computing (HPC) and warehouse-scale systems. From HPC, training inherits dense matrix multiplication (GEMM) kernels operating near accelerator compute limits, requiring tightly synchronized parallel execution across devices. From warehouse-scale computing, multi-day training jobs inherit the necessity of cluster-wide orchestration, asynchronous data ingestion, and automated fault recovery across failure-prone nodes. Figure 8.17 sequences this architectural lineage across four overlapping computing eras—from early mainframe mainstays (ENIAC, IBM System/360) and vector HPC supercomputers (CDC 6600, Cray-1) through warehouse-scale cloud clusters to modern GPU and TPU hypercomputing clusters.

The lineage explains why a training platform must support both tightly coupled numerical execution and reliable data supply over long runs. Table 8.22 contrasts the workload and execution patterns inherited from these architectural traditions.

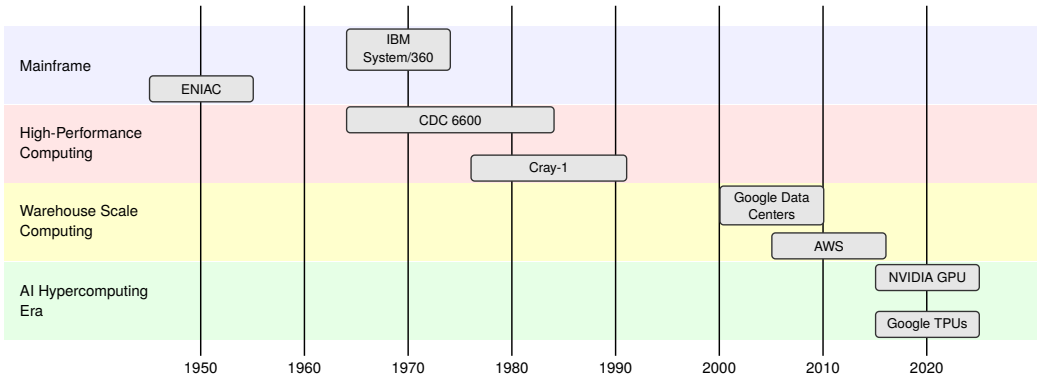


Figure 8.17: Computing System Evolution: Four overlapping eras place representative system milestones against time. The overlap matters: AI hypercomputing builds on, rather than replaces, the parallel numerical methods of HPC and the distributed coordination of warehouse-scale computing.

Table 8.22: Computing Era Characteristics: Each computing era optimized for different workload patterns, and modern training systems inherit requirements from multiple predecessors. AI hypercomputing uniquely combines HPC's parallel numerical computation with warehouse-scale distributed processing, while adding specialized support for the gradient-based optimization and massive parameter state management central to neural network training.

Era	Primary Workload	Memory Patterns	Processing Model
Mainframe	Sequential batch processing	Simple memory hierarchy	Single instruction stream
HPC	Scientific simulation	Regular array access	Synchronized parallel
Warehouse-scale	Internet services	Sparse, irregular access	Independent parallel tasks
AI Hypercomputing	Neural network training	Parameter-heavy, mixed access	Hybrid parallel, distributed

Scaling changes the binding constraint rather than removing it: data parallelism buys throughput, model parallelism buys memory capacity, pipeline parallelism buys utilization, and tensor parallelism buys layer-scale feasibility, but each adds communication and coordination cost. The implementation details of multi-node distributed training, including collective communication primitives, fault tolerance mechanisms, and elastic scheduling, build directly on the single-machine principles covered throughout this chapter and are treated in depth in advanced distributed systems texts.

Checkpoint 8.4: Scaling decisions

Scaling trades compute bottlenecks for communication bottlenecks.

- ☐ **Hard limits**: Identify which resource limit binds a given training run: memory capacity, compute throughput, or input-service rate.
- ☐ **Single-node first**: Explain how a single-node baseline and relevant local optimizations inform scaling, and when a physical lower bound can justify scaling before all tuning is exhausted.
- ☐ **Data vs. model parallelism**: Justify why data parallelism fails when model states exceed single-accelerator memory, and identify what model parallelism must partition.
- ☐ **Communication tax**: Explain why scaling efficiency degrades with accelerator count, and identify the communication metrics that dictate this ceiling.
- ☐ **Scaling ceiling**: Estimate the theoretical speedup ceiling via Amdahl's law when unparallelized synchronization accounts for 10 percent of step time, and determine where adding accelerators yields diminishing returns.

8.9 Fallacies and Pitfalls

The progression from single-GPU optimization through multi-device parallelism shows that each technique introduces trade-offs and new constraints. The approach developed throughout this chapter quantifies costs through the iron law, diagnoses bottlenecks through profiling, applies targeted optimizations, and scales only when necessary. The following fallacies and pitfalls capture common errors that waste compute resources, delay research progress, or cause production training failures.

Fallacy: *Larger models always yield better performance.*

Teams sometimes treat scale as a monotonic lever: if a 7B-parameter model works well, a 20B-parameter model must work better. A 20B model requires approximately 320 GB of nonactivation training state in the stated mixed-precision Adam recipe (40 GB FP16 parameters + 40 GB FP16 gradients + 80 GB FP32 master weights + 160 GB Adam states). Whether it outperforms a 7B model depends on data quantity and quality, training compute, optimization, and evaluation, not on parameter count alone.

Pitfall: *Assuming distributed training automatically accelerates development.*

More accelerators do not guarantee faster training because communication, smaller local batches, input limits, and imbalance can consume the gain. In an illustrative 8 GPUs scenario, synchronization occupies 30–50 percent of step time and speedup reaches only 4–6× rather than the ideal 8×. The values are hypothetical; the decision requires measured end-to-end time and scaling efficiency.

Fallacy: *Hyperparameters transfer directly from small-scale experiments to large-scale training.*

A learning rate that works at batch size 512 may not transfer to batch size 4,096. The linear scaling rule (Goyal et al. 2017) would increase the rate from 0.1 to 0.8 for this eightfold change, often with warmup, but it is an empirical recipe rather than a requirement or convergence guarantee. Large-scale runs must validate the learning rate, schedule, optimizer state, and effective batch together.

Pitfall: *Treating mixed precision training as a simple toggle without validation.*

The chapter's illustrative V100 inputs imply a 2.4× speedup, but they are not a benchmark. FP16 commonly requires loss scaling (Micikevicius et al. 2017), while BF16 usually does not (Wang and Kanwar 2019; Kalamkar et al. 2019) (section 8.7.3 compares the two formats). Either recipe can change numerical behavior, so throughput and convergence must be validated on the target workload.

Fallacy: *Memory and computation can be optimized independently.*

Memory *and* compute are coupled: in this illustrative profile, accelerator utilization drops from 90 percent at batch 256 to 60–70 percent at batch 16. Gradient accumulation (effective batch 512, physical batch 64) shows a 5-percentage-point utilization gap for 8× activation-memory reduction. Tuning these parameters independently can therefore extend training time materially, because the utilization lost at the smaller physical batch applies to every step of the run (section 8.7.5 develops this trade-off).

Pitfall: *Budgeting training memory as if it only contains model weights.*

Sizing training memory from weights alone omits gradients, optimizer state, master weights when used, activations, temporary workspaces, and allocator overhead. In the chapter's mixed-precision Adam recipe, FP16 weights (2 bytes), FP16 gradients (2), FP32 master weights (4), and two FP32 moments (8) total 16 bytes per parameter before activations. Other optimizers and sharding strategies change this budget. Activation memory then depends on batch size, sequence length, architecture, kernels, and checkpointing; no universal multiple of the weight footprint applies.

Fallacy: *The accelerator is always the training bottleneck.*

Data loading often creates idle time, yet teams optimize computation first. In this illustrative profile, prefetching with pipeline overlap reduces wall-clock time by 47.6 percent (105 min to 55 min) by overlapping data loading with computation (section 8.7.2 develops the overlap). Profile before assuming the GPU is the bottleneck.

Pitfall: *Optimizing kernels before profiling the input pipeline.*
Kernel-level tuning is attractive because GPU utilization is easy to inspect, but a starved accelerator can look like an inefficient accelerator. Before changing precision, rewriting kernels, or adding GPUs, measure dataloader throughput, host preprocessing time, storage wait, and host-to-device transfer overlap. If the accelerator is waiting for batches, the fastest optimization is to feed it reliably rather than make its kernels marginally faster.

8.10 Summary

Training cost is governed by three accounts that this chapter first kept separate and then joined. The training-time law, $T_{\text{train}} = O/(R_{\text{peak}} \times \eta_{\text{hw}})$, prices time. The four-term memory budget decides what fits, and in a standard mixed-precision Adam recipe the static state alone reaches 8× the FP16 weights before a single activation is stored. The min-of-three pipeline model decides whether the accelerator is fed at all. GPT-2 tied the three together: its $6PD$ operation count set the time, its weights, optimizer state, and activations overflowed a single V100 before any arithmetic began, and its single-threaded data loader could starve the device once the model fit.

Every intervention in the chapter changes one term of one account and pays in another, which is the conservation of complexity at work. Mixed precision raises R_{peak} and shrinks activation memory and traffic, but it leaves the static state at 16 bytes per parameter and requires loss scaling or a wider-exponent format. Checkpointing buys activation memory with extra forward arithmetic, accumulation buys batch size with update frequency, prefetching hides input time at the price of host memory, and FlashAttention removes quadratic HBM traffic by recomputing tiles. Scaling to more devices adds aggregate memory and compute but turns gradient synchronization into a new term on the critical path. The profile, not the menu of techniques, decides which trade is worth making.

The same pressures shaped the hardware. Dense matrix patterns drove Tensor Cores, frameworks exposed them through mixed-precision APIs, and reduced-precision training pushed later accelerators toward BF16 and FP8. Behind all of it sits the bitter lesson seen from inside the training loop. If capability grows with the computation a general method can absorb, every point of utilization lost to stalls is capability forgone at a fixed budget.

Key Takeaways: Why training costs millions

- **Training cost is an iron-law budget**: $T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}}$ makes every optimization accountable: reduce work, raise effective throughput, or improve utilization. A change that misses the dominant term only moves cost around the training loop.
- **Memory determines whether an optimizer fits**: Standard mixed-precision Adam uses 8× the FP16 inference-weight memory in the stated representation before activations. Optimizer state and batch-dependent activations—not weights alone—determine whether training fits.
- **Profiles choose the remedy**: Profile, diagnose, fix, and re-profile. Compute-bound jobs need better arithmetic or algorithms; memory-bound jobs need less state or traffic; data- and communication-bound jobs need pipeline or parallelism changes.
- **Precision and IO-aware kernels shift bottlenecks**: FP16 with FP32 accumulation can improve throughput and shrink activation memory (the static state stays at 16 bytes per parameter), while FlashAttention avoids materializing the full $S \times S$ matrix in HBM; realized gains depend on workload and hardware.
- **Checkpointing buys memory with recompute**: The segment schedule counts retained boundaries and one live reconstructed segment, reducing the modeled activation allowance from 35.9 GB to 4.6 GB at batch 4. Use it when memory, not compute, binds.
- **Composed optimizations can postpone scale-out**: Mixed precision and checkpointing turn 95.7 GB into a modeled 28.6 GB for batch-4 GPT-2, before temporary workspaces. Use relevant local levers before accepting distributed communication and operational overhead.

Training is where the iron law becomes a daily instrument. A profile read against the law names the dominant term, and the engineering task is to apply the least costly intervention that changes it before reaching for more hardware. The analysis took the model and training corpus as given, while changing how the system executed their work, including the extra arithmetic needed for recomputation. A further lever is which examples enter that workload in the first place.

What's Next: From training to data selection

Training produces learned parameters by spending compute on every example. Once the training loop is efficient, the next question is whether every example earns that cost. Repeated or low-information samples consume the same forward pass, backward pass, optimizer update, and communication as examples that add coverage or correct a weakness.

Chapter 9 therefore turns upstream from execution to workload composition, and in doing so opens the volume's optimization sequence, which asks whether work is necessary before asking how to simplify or accelerate it. It starts from a supply imbalance. Accelerator fleets can grow usable compute faster than the stock of high-quality data, so extracting more learning from each example begins to matter more than collecting more examples. Its measure is the information-compute ratio, the learning an example returns per unit of training compute, and redundancy, representativeness, uncertainty, and class coverage all enter through it. A useful subset must reduce work without erasing rare cases or shifting the effective training distribution. When that balance holds, a carefully selected subset can approach full-dataset accuracy while lowering the cost of every epoch and making additional experiments affordable. The optimization target is no longer only how efficiently the system processes an example, but which examples the system should process at all.

III

OPTIMIZATION AND ACCELERATION

Part III

Optimization Principles

A working model is rarely an efficient one. Part II established how to construct ML systems that respect physical constraints; Part III addresses how to meet real-world demands on time, memory, and energy. Efficiency is also what keeps the bitter lesson affordable, because models grow with the computation available while budgets and deployment envelopes do not. Each optimization navigates a frontier, improving one metric (accuracy, latency, energy) while managing the cost to others, and each is a form of co-design. Data selection shapes the workload the algorithm sees, while compression and acceleration trade mathematical structure for physical feasibility. The principles here are the bounds and cost accounts of efficiency, which determine *why* some models are fast and affordable while others are slow and prohibitively expensive.

Principle 5: The Pareto frontier

Bound: After objective directions are normalized, a feasible point lies on the Pareto frontier exactly when no other point is at least as good on every objective and strictly better on one. No feasible design lies beyond that frontier, so moving along it improves one objective only by worsening another.

- **Quantization**: Trades numerical precision for reduced memory footprint.
- **Pruning**: Trades model capacity for smaller representations and can improve speed when the resulting sparsity or removed structures are supported by the target hardware.
- **Distillation**: Trades training compute for inference efficiency.

Implication: Systems engineers navigate this frontier to find the operating point appropriate to a specific deployment environment. There is no universal optimum.

Navigating the Pareto frontier requires knowing *which* resource to optimize. Before selecting a technique, engineers must diagnose whether the workload is limited by computation, memory movement, or dispatch latency. Arithmetic intensity supplies the first test. The introduction derived its threshold, the ridge point, by setting the iron law's memory term equal to its compute term, and Part II placed training kernels on either side of that threshold in a roofline plot. Here the roofline becomes an explicit rule for choosing an optimization.

Principle 6: Arithmetic intensity law

Bound: Attainable throughput (R_{attain}) is no greater than the minimum of peak compute (R_{peak}) and DRAM bandwidth (BW) times operational intensity at the cache–DRAM boundary (I) (Williams et al. 2009):

$$R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$$

Operational intensity and DRAM bandwidth must both be measured at the cache–DRAM boundary; this ideal bound excludes launch, synchronization, and other fixed overheads.

Implication: Adding peak compute does not raise the ideal roofline bound for a bandwidth-bound kernel. Engineers must identify whether the bottleneck is compute, bandwidth, or another system term before selecting an optimization.

Many important ML kernels fall on the memory-bound side of the ridge point, especially low-reuse operations such as embedding lookup, normalization, softmax, depthwise convolution, and small-batch inference paths. Dense matrix multiplications and convolutions can instead be compute-bound when batching and hardware utilization are high. This split is a consequence of the memory wall: processor speed has historically outpaced memory bandwidth, and the cumulative gap has widened over three decades (Wulf and McKee 1995; Gholami et al. 2024). Neural networks, with their massive weight tensors and uneven temporal locality, are especially vulnerable. The arithmetic intensity law diagnoses *where* a workload sits relative to this wall. The cost of moving data explains *why* the wall is so punishing. Table 8.23 maps each bottleneck type to representative optimizations that address it and to changes that leave the dominant term untouched.

Table 8.23: The Bottleneck Diagnostic: Before optimizing, identify which iron law term dominates. Optimizing the wrong term leaves the dominant bottleneck unchanged and usually yields little or no end-to-end improvement.

If the workload is…	Dominant Term	Optimization That Works	Optimization That is Wasted
Memory-Bound	D_{vol}/BW	Quantization, pruning, batching	Faster accelerator (more FLOP/s)
Compute-Bound	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	Better kernels, Tensor Cores, faster accelerator	More memory bandwidth
Latency-Bound	L_{lat}	Kernel fusion, async dispatch, bounded microbatching under the latency SLO	More FLOP/s or bandwidth alone

Performance is only one cost of memory movement; energy exposes a second constraint.

Principle 7: The energy-movement principle

Decomposition: Total workload energy depends jointly on per-event energy and event counts. In the cited 45 nm reference, moving a 32-bit value from DRAM can cost roughly 100–1,000× more energy per event than a 32-bit floating-point operation, depending on operation type (Horowitz 2014). Here, D_{vol} is bytes moved and O is operations executed; E_{move} denotes the energy per byte moved from DRAM, so one 32-bit access costs $4\,E_{\text{move}}$, and E_{compute} denotes the energy of one 32-bit floating-point operation in the cited 45 nm technology.

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}, \qquad 4\,E_{\text{move}} \gg E_{\text{compute}} \quad \text{for the cited reference}$$

Implication: When memory movement dominates workload energy, optimization strategies should prioritize kernel fusion (avoiding intermediate off-chip traffic) and quantization (reducing data size); when computation dominates, reducing operation counts and improving arithmetic kernels remain central.

Even with perfect data locality and optimal bottleneck targeting, a final constraint limits how much speedup any optimization can deliver. The introduction stated Amdahl's law for an inference pipeline whose preprocessing and postprocessing capped the gain from a faster model, and later chapters applied it case by case, to data-ingestion workers and to communication overhead in training. Stated as a principle, it bounds every optimization in this part.

Principle 8: Amdahl's law

Bound: The maximum speedup of a system is limited by the fraction of the workload that cannot be accelerated (Amdahl 1967). Here, f_{parallel} is the fraction of baseline execution time

accelerated and G is that fraction's speedup, either an accelerator's gain G_{accel} or a worker count N_{workers}; the fixed-work model assumes the remaining fraction is unchanged and the acceleration introduces no additional overhead.

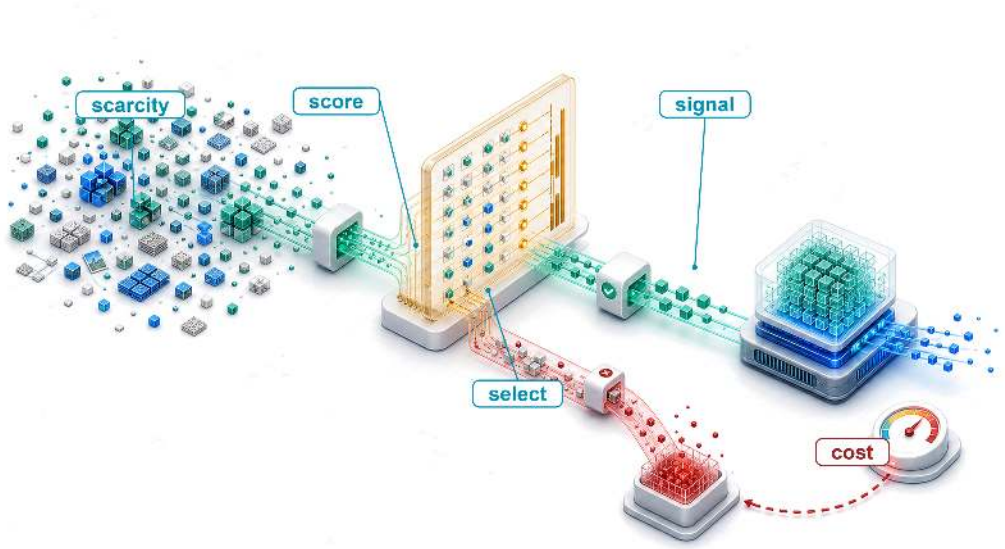
$$\text{Speedup} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{G}}$$

Implication: If 95 percent of a model runs 100× faster on a GPU, the total system speedup is capped at ~16.8×. This explains *why* data loading and preprocessing often become the ultimate bottlenecks in highly optimized systems.

Part III applies these principles in the order the D·A·M taxonomy suggests (Appendix A gives the full diagnostic framework). On the data axis, chapter 9 asks first whether the work is necessary. Where algorithm meets machine, chapter 10 asks whether the remaining work can be simplified. On the machine axis, chapter 11 asks how to do it faster. Each chapter ends with a favorable local number (a smaller dataset, a smaller model, or a kernel placed under its roofline), and chapter 12 closes the loop by testing whether those predicted savings survive end-to-end execution without violating quality constraints.

9

Data Selection

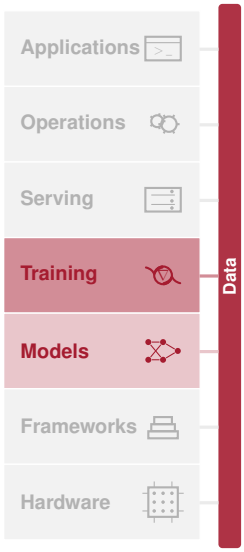


9.1 Data Selection Fundamentals
9.2 Defining Data Selection
9.3 Systems Perspective
9.4 Information-Compute Ratio
9.5 The ICR Frontier: When Data Becomes a Tax
9.6 Measurement Framework
9.7 Static Pruning
9.8 Dynamic Selection
9.9 Self-Supervised Learning
9.10 Synthetic Data Generation
9.11 Selection Engineering
9.12 Distributed Selection
9.13 Cost Modeling
9.14 Decision Framework
9.15 Cross-Layer Interactions
9.16 Fallacies and Pitfalls
9.17 Summary

Purpose

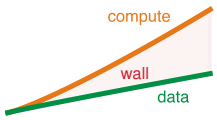
How can a carefully selected subset retain most of the value of a much larger dataset?

Training pays to acquire, label, score, move, and process examples even though their learning value differs. Large datasets contain redundant, noisy, or target-misaligned samples, while scarce regions may lack the cases a model needs, such as the rare conditions under which a deployed system is most likely to fail. Training on every example treats a near-duplicate and a rare failure case as equally worth paying for, and at scale much of the compute budget can go to relearning what the model already knows. Data selection turns that heterogeneity into a systems optimization. It can remove avoidable work before training, direct labels and training steps toward informative cases as learning proceeds, or create missing examples when scarcity is the binding constraint. It can also invest once in reusable representation learning when that cost can be amortized across downstream tasks. The objective is therefore not the smallest dataset but the lowest end-to-end cost of reaching target quality without losing rare cases, underrepresented groups, or deployment-relevant coverage. Every strategy adds its own overhead: examples must be scored, generated, shuffled, stored, or validated, and those costs can erase the savings they promise. That judgment must be made end to end, because gains in labeling or training can be erased by selection latency, storage access, or distributed coordination. A method earns its place only when the learning it preserves or creates exceeds the work required to apply it. This distinction matters across repeated experiments, where useful reductions compound, and across adaptation, where the value of an example changes with the model and target distribution. Data engineering established that data is the source code of ML systems; data selection asks which evidence is useful now and which is still missing. In D·A·M terms, it is data-algorithm co-design: shape the training signal so each unit of machine work buys more useful learning.



Learning Objectives

- Explain data selection as data-algorithm co-design that controls sample value, training work, and labeling cost
- Calculate information-compute ratio to decide whether additional examples improve learning per FLOP
- Compare deduplication, coreset selection, and quality pruning for reducing redundant pretraining data
- Design curriculum, active learning, or synthetic-data strategies for changing data value during training
- Apply the selection inequality to test whether selection overhead beats full-dataset training cost
- Evaluate distributed selection pipelines against storage locality, consistency, and accelerator utilization constraints
- Select data-selection investments using ROI, amortization, and compute-optimal frontier diagnostics



Compute outruns high-quality data supply; the widening gap is the data wall.

9.1 Data Selection Fundamentals

TRAINING pays the iron-law cost for every example it processes, but not every example returns learning signal worth that cost. Data selection gives a clean, well-engineered dataset a systems objective: keep the examples that contribute the most learning per unit of compute. Data engineering makes the dataset reliable through correct labels, consistent schemas, and governed records. Data selection optimizes the dataset's value by extracting maximum learning from minimum samples, shrinking the total operations (O) term in the iron law (principle 3) when it removes processing at matched quality. The distinction matters: quality asks whether data is correct, while value asks whether correct data is worth the compute spent processing it.

Increasing the amount of training data has long been a standard way to improve models. Scaling laws[1] (Kaplan et al. 2020; Hoffmann et al. 2022) quantify how model performance can improve with dataset size, and teams have responded by collecting and generating more examples. Accelerator fleets, however, can expand usable compute faster than the supply of novel, high-quality human-generated text and images. Much of the easily accessible public web has already been incorporated into large training corpora, and expert labeling capacity grows slowly. This asymmetry is the **data wall**,[2] shifting engineering focus from indiscriminate data ingestion toward maximizing the learning signal extracted from every stored and processed sample.

Table 9.1 uses an illustrative growth scenario in which GPU compute rises 10× every 3 years while accessible high-quality data grows more slowly. These are scenario inputs, not measured universal rates.

Table 9.1: Scaling Asymmetry in ML Resources: In the illustrated scaling regime, compute grows faster than high-quality data supply, creating a compute-to-data imbalance that makes data selection valuable.

Resource	Growth Rate	Implication
GPU Compute	~10× / 3 years	Hardware throughput can rise quickly in a given era
Training Data (Web)	~2× / 5 years	High-quality web text is finite; much already scraped
Labeled Data	~1.5× / 5 years	Human annotation throughput is inherently bounded
Synthetic Data	Potentially large	Bounded by generator quality (models trained on model-generated data can degrade)

Scaling dataset size eventually encounters the finite stock of high-quality human speech and text. Figure 9.1 charts the steep log-scale ascent of foundation model training sets from GPT-3 through Llama 3, showing the empirical trajectory converging toward the shaded ceiling of high-quality public text stock near 10^{14} tokens.

[1] **Scaling laws**: Jared Kaplan and colleagues at Johns Hopkins and OpenAI empirically demonstrated in 2020 that language model loss follows power-law relationships with model size, dataset size, and compute budget over the regimes they studied (Kaplan et al. 2020). Their fitted data exponent was approximately $\alpha = 0.095$ in $\mathcal{L} \propto D^{-\beta}$. The value is not universal, but the diminishing-return shape makes it possible to reason about when selection becomes more cost-effective than collection.

[2] **Data wall**: Unlike compute (which scales with capital expenditure) or algorithms (which improve through research), the stock of high-quality human-generated text grows slowly; Epoch AI's updated projections estimated that, under the paper's modeled consumption assumptions, models could use datasets roughly equal in size to the stock of public human-generated text between 2026 and 2032 (Villalobos et al. 2022). The point for systems design is not a fixed calendar deadline; it is that data can become a supply constraint rather than merely an economic one. The wall does not change how many operations (O) a run performs. It lowers what each of those operations buys, because once quality data runs short, additional compute spent on repeated or weaker data yields diminishing accuracy regardless of hardware throughput.

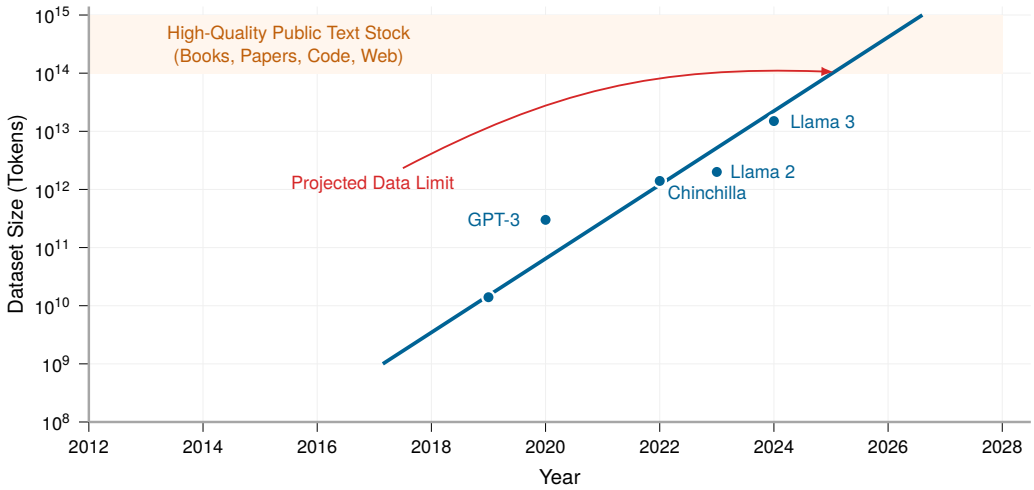


Figure 9.1: Dataset Growth Approaching Limits: Several foundation-model datasets have grown toward estimates of the total stock of high-quality public text. The projection should be read as a dated constraint scenario rather than a fixed forecast: data selection, synthetic generation, multimodal learning, licensing, and filtering policy all change the usable supply.

When *compute* is abundant and *quality data* is scarce, the optimization priority inverts. Selection, not faster kernels, then extracts more learning per dollar, and for teams whose accelerator budget exceeds their curated corpus, the bottleneck shifts from accelerator access to the quality, legality, and diversity of the training data.

That inversion raises the bitter lesson's question in data terms. General methods win by absorbing more computation, and scaling laws describe that absorption only while data grows alongside compute. Selection can look like the hand-built expertise the lesson warns against, and a filter written from human intuition about which examples matter is exactly that. Selection can also use the model's own signals, such as loss, gradient norms, or embedding similarity. More compute can support stronger scoring models or more frequent score updates, but improved sample rankings must be established against downstream quality and coverage. Done that way, selection is how a compute-hungry general method stays fed once the supply of fresh, high-quality data stops keeping pace.

↩ The bitter lesson, that general methods using more computation outperform encoded human expertise, is established in section 1.4.

The engineering toolkit for intelligent data selection follows a deliberate optimization ordering: first ask whether a sample is worth processing, then ask how to process the remaining workload efficiently. Data selection sits first in the ordering that frames all of Part III, removing avoidable work before anything simplifies or accelerates the work that remains. Static pruning fixes the retained subset before target-model training; scoring may itself require proxy training. Dynamic selection adapts the data diet during training through curriculum learning and active learning. Synthetic generation creates high-value samples through augmentation, simulation, or examples labeled by a larger trained model (a *teacher*) when real data runs short.

Each stage can increase the information density of the data that reaches the model when measured quality gains justify its overhead, forming a complementary toolkit rather than a rigid execution sequence: pruning reduces *what* the pipeline contains, dynamic selection focuses *how* the training loop schedules it, and synthesis expands *what* the pipeline can access. Self-supervised pretraining complements these three levers by redefining what constitutes a supervision signal: rather than selecting among labeled examples, it derives learning targets directly from the intrinsic structure of unannotated data. Evaluating any of these techniques requires a systems definition of data selection and a quantitative account of its effectiveness.

9.2 Defining Data Selection

A duplicated image, a mislabeled record, and a rare boundary case may all cost the same forward and backward pass, but they do not contribute the same learning signal. Data selection therefore

starts by making sample value explicit relative to compute cost. The information-compute ratio measures that value as learning signal gained per unit of training compute, formalized in section 9.4.

> **Definition 9.1: Data selection**
>
> ***Data selection*** is the process of retaining, prioritizing, or generating training examples to minimize the end-to-end cost of reaching target quality while preserving required coverage.
>
> 1. **Significance**: It identifies data worth retaining, prioritizing, or generating for the target objective, reducing the total operations (O) when redundant or noisy samples can be excluded without losing required coverage.
> 2. **Distinction**: Unlike data engineering, which focuses on the cleanliness and consistency of data, data selection focuses on the informativeness and diversity of the samples.
> 3. **Common pitfall**: A frequent misconception is that more data is always better. Additional low-quality data can yield less improvement than a much smaller, carefully selected set, so retained volume must be evaluated together with coverage and target quality.

To make this concrete, consider training an autoregressive language model in the GPT-2/Llama lighthouse family at frontier scale. The scenario deliberately steps outside this book's one-machine scope, to a 70-billion-parameter model on a cluster of thousands of accelerators, because that is where the data wall binds first. The GPT-2 model trained in section 8.3.1 sits far below this point, but the same arithmetic applies to any team whose compute can traverse its curated corpus many times over.

↩ GPT-2, the bandwidth-bound lighthouse for decoder-only language models, is established in section 6.2.

The compute budget (10,000 H100 GPUs for 3 months) represents roughly $86.4M at a cloud rental price of $4/GPU-hour and can process about 73.2T tokens at 40 percent sustained model FLOPs utilization (MFU), the fraction of peak arithmetic a training run actually delivers (definition 8.4). Estimates of quality- and repetition-adjusted public human-generated text are on the order of 300T tokens, far larger than a single curated web corpus. The practical bottleneck is narrower: how much of that stock is accessible, legally usable, high quality, deduplicated, and useful for the target distribution. If a team has only a 5T filtered corpus ready for use, the compute budget can already process it roughly 14.6× over. At that point, the cluster outstrips the available unique signal. An engineering team can repeat epochs over the same corpus, but gradient updates on familiar tokens yield diminishing loss reductions. It can lower quality thresholds to admit uncurated web text, but noisy and misaligned tokens degrade downstream evaluation. Or it can invest in data selection (refining quality filters, constructing difficulty curricula, and synthesizing missing edge cases) to raise the learning return per FLOP. When compute capacity outpaces high-signal data, selection delivers higher return on capital than provisioning additional accelerators.

The opportunity applies across the lighthouse models, from ResNet-50 to GPT-2/Llama, although the safe reduction differs by task. Whenever a validated method removes examples or tokens without adding compensating steps, training FLOPs fall. The appropriate framing is therefore systemic rather than purely statistical.

9.3 Systems Perspective

The data wall dictates the physical limits of scaling uncurated datasets; a systems perspective determines how to manage that constraint across the hardware stack. The conventional *ML framing* evaluates data selection through statistical sample complexity and generalization theory, asking how many samples are required to achieve a given test error. That formulation treats every sample as a zero-cost mathematical abstraction, ignoring the physical resource cost of data ingestion, transport, and execution.

A data-selection *systems framing* asks instead how to minimize the total resource cost of achieving target accuracy across the entire ML lifecycle. This framing shifts attention from theoretical sample efficiency to concrete resource consumption, as table 9.2 illustrates.

Table 9.2: ML vs. Systems Perspectives on Data Selection: The ML framing optimizes sample complexity; the systems framing optimizes total resource cost across the pipeline.

ML Framing	Systems Framing
"Fewer samples for same accuracy"	"Fewer FLOPs for same accuracy"
"Better generalization"	"Lower training cost (time, money, energy)"
"Sample complexity bounds"	"End-to-end resource efficiency"
"Learning theory"	"Cost engineering"

This framing reveals optimization leverage across the execution stack. In the D·A·M taxonomy, data selection operates at the data layer to eliminate physical work before algorithmic and hardware optimizations execute it, as formalized by the iron law of ML systems.

↩ The D·A·M taxonomy and its three diagnostic axes are established in section 1.5.

Read as an optimization map (figure 9.2), the taxonomy assigns each vertex its own governing question, positioning data selection as an input optimization that prunes the total workload before algorithmic and physical execution take place. The model vertex determines how many operations each forward and backward pass requires; the machine vertex determines how rapidly the hardware executes that arithmetic; and the data vertex determines whether an example warrants processing at all. The connecting edges demarcate the primary operational bottlenecks: compute-bound arithmetic throughput between algorithms and hardware, I/O-bound bandwidth between storage and accelerators, and sample efficiency between data curation and model capacity. Sample efficiency governs the Data–Algorithm boundary (what to learn from), the I/O bound governs the Data–Machine boundary (how information moves), and the compute bound governs the Algorithm–Machine boundary (how to execute arithmetic efficiently). By optimizing at the data vertex, data selection eliminates work across both adjacent boundaries—reducing storage ingestion traffic along the machine edge and raising effective learning return along the algorithm edge.

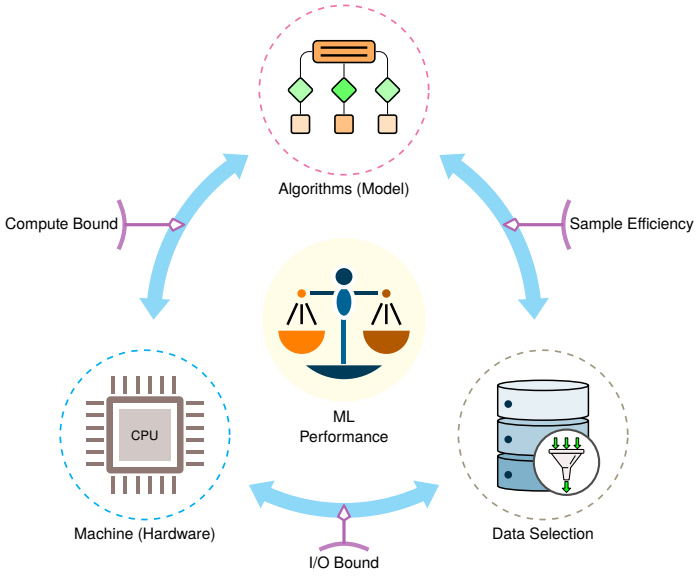


Figure 9.2: The D·A·M Taxonomy as an Optimization Map: Machine learning performance is organized around Algorithms (Model), Data Selection, and Machine (Hardware). The triangle's edges label the compute-bound, I/O-bound, and sample-efficiency relationships, while data selection reduces workload before model or hardware execution.

Systems Perspective 9.1: Data selection and the iron law

In the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$), data selection reduces the number of examples or tokens that enter the workload. Later model-level optimizations reduce operations per forward/backward pass, and machine-level optimizations increase R_{peak} (peak throughput) and η_{hw} (utilization). Selection removes work before those downstream optimizations execute it.

Within the compute term $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$, data selection is multiplicatively valuable. Data selection and model-level optimization shrink the numerator while machine-level optimization enlarges the denominator, so when a compute-bound run applies all three, a 2× reduction in dataset size with 2× fewer operations per sample and 2× higher effective throughput yields an 8× reduction in compute time because the factors multiply; each layer divides the work the next layer sees. The iron law itself remains the sum of equation 1.3, so a run bound by data movement (D_{vol}/BW) or fixed overhead (L_{lat}) sees less than the full product.

Holding epoch count, batch size, and per-sample work fixed, a 50 percent reduction in dataset size directly halves the required forward passes, backward passes, and gradient updates. For the $100M scenario, this reduction yields $50M in direct compute savings, provided the training schedule does not compensate by adding optimization steps.

These compute savings propagate throughout the hardware hierarchy. By decreasing total data volume (D_{vol}), deduplication and coreset selection alleviate storage bandwidth demands, network fabric ingress, and host-to-accelerator PCIe transfers that otherwise leave the accelerator's arithmetic units waiting on input. Upstream, active learning curbs annotation budgets by directing human labeling to samples chosen for uncertainty, diversity, or other task-relevant query criteria.

Yet selection is not free: every selection strategy incurs its own runtime overhead to score, filter, shuffle, or coordinate samples. An online selection algorithm that executes additional forward passes to rank sample difficulty can easily consume more accelerator cycles than it saves if its selection overhead exceeds the subsequent reduction in training time. A pipeline claiming a training-time saving must make filtering plus subset training faster than full-data training at the same target quality. Labeling savings and reusable representations require the broader cost accounting developed later in the chapter.

9.4 Information-Compute Ratio

The systems framing established in section 9.3 calls for a quantitative metric. Data selection moves along a Pareto frontier (principle 5) between accuracy and cost: retaining every example preserves coverage but expends compute on redundant signal, whereas over-aggressive pruning saves FLOPs at the expense of rare-slice recall. Navigating that trade-off requires quantifying the marginal learning signal each sample yields per unit of hardware execution. Data engineering already priced task-relevant signal against one cost, the data gravity of moving bytes, in the data selection gain of equation 4.1. Selection needs the same ratio with training compute in the denominator, because a sample that is cheap to move can still cost a full forward and backward pass. That ratio is the **information-compute ratio** (ICR).

Let $\Delta\mathcal{I}$ be the learning signal a sample or batch adds and ΔO the training operations spent processing it. In practice $\Delta\mathcal{I}$ is measured as the gain in a fixed evaluation metric, such as percentage points of validation accuracy, so ICR carries units of metric gain per FLOP. The ratio relates the two:

$$\text{ICR} = \frac{\Delta\mathcal{I}}{\Delta O}$$

A higher ICR indicates that each FLOP of training buys more measured learning signal over the stated interval. Validation gain and training FLOPs can be observed for a specified experiment with a fixed evaluation protocol; the causal contribution of an individual example generally cannot. Uncertainty and gradient-based scores instead rank candidate examples before their downstream benefit is known. Those rankings require validation against measured quality, compute cost, and

deployment-critical slice coverage. The core metrics in section 9.6.1 supply complementary evidence for that evaluation rather than interchangeable measurements of per-example information.

9.5 The ICR Frontier: When Data Becomes a Tax

The information-compute ratio falls as increasingly redundant examples enter the dataset. The **ICR frontier** is the dataset size beyond which the marginal ICR, the learning bought by the next FLOP spent on more data, drops below what that FLOP could buy elsewhere: a larger model, a different data source, or an earlier stop.

To illustrate diminishing returns, let $\mathcal{I}(D)$ be the information content of a dataset of size D and assume $\mathcal{I}(D) \propto \log D$. This is a simplified analytical model rather than a universal law. If compute cost scales linearly with the per-sample operation count O_{sample}, then total operations are $O = O_{\text{sample}} \cdot D$, and the resulting marginal ICR follows equation 9.1:

$$\text{ICR}(D) = \frac{d\mathcal{I}/dD}{dO/dD} \approx \frac{1/D}{O_{\text{sample}}} = \frac{1}{O_{\text{sample}} \cdot D} \tag{9.1}$$

The derivative ratio describes the return on an infinitesimal increase in dataset size. For two controlled finite allocations, the corresponding incremental ratio divides the additional evaluation gain by the additional training FLOPs, using the same evaluation metric and protocol. The coreset comparison that follows instead averages gain over one epoch of each dataset; that average does not establish the return on the next increment.

Under this illustrative model, marginal ICR falls as $1/(O_{\text{sample}} \cdot D)$, so doubling the dataset halves the learning each additional FLOP buys. Inverted, the compute spent per unit of learning grows linearly with D. ICR never reaches zero in this model and the curve has no knee of its own, so the frontier is located by comparing marginal ICR with the threshold the engineer sets, not read off the curve. That saturation is distinct from the data wall (a limit on how much new, high-quality data exists), but the two compound. When new data is scarce *and* marginal data is weak, selection becomes the main lever. Beyond the frontier, additional data still adds compute but buys less learning than the threshold demands. In this regime, data becomes a **data tax** that inflates the O term of the iron law without a commensurate gain in ICR. Priced in dollars rather than FLOPs, the same tax lowers the return on compute (RoC) of equation 1.6, the accuracy gained per added dollar of infrastructure.

Data selection turns the total operations (O) term in the iron law from a fixed constant into a variable. A $2\times$ improvement in measured ICR halves the FLOPs required to reach the same measured gain. It produces the same ideal time reduction as a $2\times$ throughput increase only when the workload is compute-bound and the saved work lies on the critical path. ICR focuses specifically on compute; the cost-modeling framework in section 9.13 extends the same reasoning to acquisition, labeling, and storage costs.

↩ Return on compute, the economic lens that prices accuracy gains against infrastructure spending, is established in section 1.7.1.

A random batch of raw data can have low ICR when it contains redundant, noisy, or already-mastered examples. To systematically raise ICR, figure 9.3 organizes three intervention points: offline static pruning before target training, dynamic selection during training loops, and on-demand synthetic generation. Their use and ordering depend on the workload.

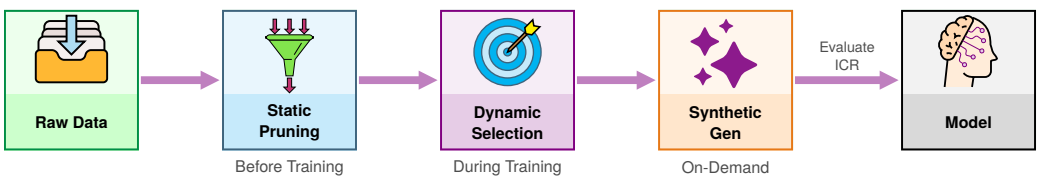


Figure 9.3: An Example Data Selection Pipeline: One possible combination of interventions for increasing data value. The three intervention points are Static Pruning before target training, Dynamic Selection during training, and Synthetic Generation on demand. The sequence is illustrative rather than required: synthetic data can enter independently and can itself be pruned or dynamically selected. The final evaluation determines whether the quality gain justifies the compute cost through the Information-Compute Ratio (ICR).

The ratio becomes concrete on a *coreset*, a deliberately selected subset intended to preserve the full dataset's learning signal. Computing ICR for an ImageNet coreset used to train the ResNet-50

lighthouse shows how large the efficiency gap could become once dataset size, per-sample model cost, and selection strategy meet concrete FLOP budgets. The scenario uses assumed accuracy gains to illustrate the arithmetic; it is not a reported result for the scoring method (EL2N) that section 9.7.4 develops.

Napkin Math 9.1: Computing ICR: Coresets

Problem: Training the ResNet-50 lighthouse model on ImageNet for one epoch costs 3.15×10^{16} FLOPs. Under the scenario's assumed accuracy gains, how would retaining 50 percent of the data change the information-compute ratio? The calculation holds the per-sample work fixed and does not include selection overhead.

Setup:

- Dataset: ImageNet (1.28M)
- Model: ResNet-50 lighthouse (~8.2 GFLOP per forward pass, roughly 24.6 GFLOP for a forward plus backward training step, depending on implementation)
- One epoch: 1.28M $\times$ 24.6 GFLOP = 3.15×10^{16} FLOPs
- Assumed accuracy improvement per epoch (early training): 5 percentage points

Random selection (baseline):

- Process all 1.28M samples uniformly
- Accuracy gain: 5 percentage points
- $\text{ICR}_{\text{random}}$ = 5 percentage points / (3.15×10^{16} FLOPs) = 1.6×10^{-16} per FLOP

Difficulty-scored coreset (EL2N, a training-dynamics score defined in section 9.7.4; 50 percent of data):

- Process 640.6K selected samples
- Assumed accuracy gain: 4.5 percentage points (90 percent of the full-data gain)
- Compute: 640.6K $\times$ 24.6 GFLOP = 1.6×10^{16} FLOPs
- $\text{ICR}_{\text{coreset}}$ = 4.5 percentage points / (1.6×10^{16} FLOPs) = 2.9×10^{-16} per FLOP

Systems insight: Under these assumptions, the selected subset achieves 1.8× higher ICR with a 0.5 percentage points smaller accuracy gain. The comparison is learning per FLOP over one epoch of each dataset. It does not show that the coreset reaches the full-data run's final accuracy, which must be checked separately at equal quality, and a real decision also requires measured gains, selection overhead, and coverage checks.

Checkpoint 9.1: Data selection efficiency

The goal is to improve ICR subject to target quality and required coverage, then verify that the resulting pipeline reduces the cost it is intended to address.

Metric checks

- ☐ **ICR application**: Given two training runs with identical accuracy gains but different compute budgets, determine which achieved higher ICR.
- ☐ **Data efficiency**: The coreset notebook reports 1.8× higher ICR but a smaller per-epoch gain. Explain what would have to change for a 50 percent subset to match the full-data gain, which would give it 2× higher ICR and so deliver the same measured gain for half the training FLOPs.

Pipeline checks

- ☐ **The three stages**: Map static pruning, dynamic selection, and synthetic generation to the training lifecycle.

A reduction in sample count remains a claim until it is shown to translate into end-to-end efficiency without silently compromising generalization. Notebook 9.1 holds per-sample work fixed and assumes its accuracy gains; testing the claim on a real workload requires measured metrics that weigh arithmetic savings, data movement overhead, and task quality across the complete pipeline.

9.6 Measurement Framework

Deploying a model trained on an aggressively filtered dataset can yield unexpected regressions in production: an autonomous vehicle perception model might maintain high aggregate precision while failing on rare pedestrian crossings pruned as low-density noise. Failures of this kind make a disciplined measurement framework essential. Every data selection technique claims to improve efficiency, but only rigorous measurement separates genuine training savings from shifted computational debt or degradation on safety-critical tail distributions. Three kinds of evidence judge a smaller dataset by what it preserves under production stress rather than by the volume it discards: efficiency metrics that tie sample reduction to accuracy, a diagnostic that tests whether the training run is data-limited at all, and validation that deployment-critical slices survive the reduction.

9.6.1 Core metrics

Three core metrics connect sample reduction to model quality instead of reporting accuracy alone. Performance-per-data measures accuracy gain per sample, the area under the learning curve measures cumulative learning efficiency across dataset scales, and the data compression ratio measures dataset reduction at a target accuracy. Together they complement the measured information-compute ratio (section 9.4). PPD measures average gain per sample, which becomes gain per FLOP once per-sample cost is fixed. AULC summarizes performance across a common dataset-size range, while DCR measures retained sample reduction at fixed quality. None establishes the causal value of an individual example.

Performance-per-data

The most direct metric, **performance-per-data (PPD)**, measures accuracy gain per sample:

$$\text{PPD}(n) = \frac{\text{Accuracy}(n) - \text{Accuracy}(0)}{n}$$

where n is the number of training samples. A higher PPD indicates greater average improvement per sample over the stated interval. Its shape must be measured because diminishing returns and their onset are workload-dependent.

Area under the learning curve

Rather than comparing at a single point, the **area under the learning curve** (AULC) integrates performance across all dataset sizes:

$$\text{AULC} = \int_0^D \text{Accuracy}(n)\, dn$$

where n is the dataset size and D is the total dataset size.

For the same metric and integration range, a higher AULC means stronger integrated performance across dataset sizes, though one strategy need not outperform the other at every size. Comparisons must use the same upper limit D or normalize the integral. Measuring AULC requires training at several dataset sizes, so the metric itself spends compute, and when it guides a selection decision that cost belongs to the selection overhead.

Data compression ratio

For coreset methods, the **data compression ratio** (DCR) measures how much data reduction is achieved at a target accuracy:

$$\text{DCR} = \frac{D_{\text{total}}}{D_{\text{subset}}} \text{ at Accuracy}_{\text{target}}$$

A DCR of 5× means the coreset achieves target accuracy with 20 percent of the data.

Applied to the coreset notebook, the three metrics behave differently. Over one epoch, the full ImageNet set gains 5 percentage points while the half-size coreset gains 4.5 percentage points, so the coreset's PPD is higher by the same 1.8× as its ICR, because per-sample work is fixed. DCR cannot be computed from that notebook at all, because the two runs never reach a common target accuracy. Exposing that missing equal-quality check is exactly the job DCR is designed for.

9.6.2 The compute-optimal frontier

While the core metrics in section 9.6.1 measure individual techniques, a higher-level diagnostic is also needed to test whether the overall training strategy is data-limited or compute-limited. Neural scaling-law experiments (Kaplan et al. 2020; Hoffmann et al. 2022) fit power-law relationships over particular model families, datasets, and compute ranges. Controlled sweeps can use those fitted relationships to compare model and data allocations, but no single ratio diagnoses every training run.

The Chinchilla study[3] (Hoffmann et al. 2022) fit a compute-optimal balance between model size and training data within its experimental regime. Allocations on either side of that fitted balance used the study's compute budget less effectively.

The optimal balance defines a **compute-optimal frontier**: the best achievable performance at each compute budget when data and model size are properly balanced. Figure 9.4 plots this diagnostic curve over normalized training compute, isolating the optimal operating envelope on the green frontier from the compute-starved and data-starved allocations charted in the suboptimal region below.

[3] **Chinchilla**: Chinchilla is a 70-billion-parameter language model trained on 1.4 trillion tokens. In the experiments reported by Hoffmann et al. (2022), it outperformed the larger GPT-3 on most evaluated tasks under a similar training-compute budget. The fitted compute-optimal allocation scaled parameters and tokens at roughly equal rates within the study's experimental regime; it is not a universal prescription for every architecture, dataset, or training recipe.

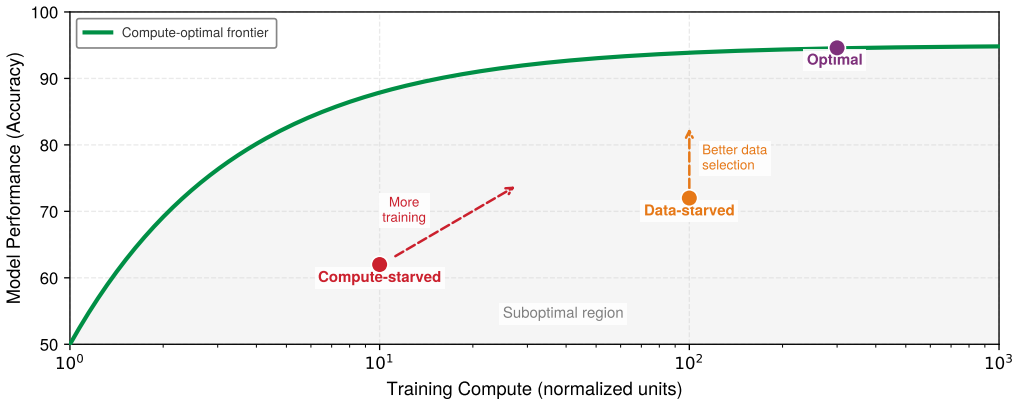


Figure 9.4: Illustrative Compute-Optimal Frontier: The green curve is a conceptual frontier over normalized compute, not a fitted result from Chinchilla. Points below it illustrate allocations that a controlled model-size and token-count sweep might identify as data- or compute-limited.

The frontier answers one allocation question, whether the next unit of compute at a given budget should go to more parameters or to more training tokens. A run is *data-starved* when it trains on too few unique, useful tokens for its parameter count, so its tokens-per-parameter ratio sits well under the fitted balance, and the same compute would buy more accuracy with more or better data. Against a frontier fitted from controlled sweeps, an operating point's location motivates further experiments. The directional vectors charted in figure 9.4 illustrate two candidate interventions: additional training compute and improved data selection at fixed compute. Controlled changes to

model size, data, and training allocation determine which intervention helps; a point below the frontier does not identify a unique remedy. Points near a fitted frontier remain specific to the model family, dataset, objective, and compute range used to estimate it.

Interpreting the Chinchilla result

Within the fitted Chinchilla regime, compute-optimal model parameters and training tokens grew at roughly equal rates.[4] Training a dense transformer costs approximately $6PD$ FLOPs for P parameters and D tokens, the forward-plus-backward accounting derived for the GPT-2 lighthouse in section 8.3.1. Under an equal scaling regime where parameters and tokens grow proportionally ($P \propto D$), total compute scales quadratically ($O \propto D^2$), directly implying that optimal token count scales as the square root of the compute budget ($D_{\text{opt}} \propto \sqrt{O}$). Doubling compute therefore corresponds to about $\sqrt{2}-1$, or 41.4 percent, more tokens under those assumptions. The commonly quoted 20 tokens per parameter is a useful reference point from that study, not a universal optimum.

[4] **Tokens per parameter**: The ratio D/P compares training tokens with model parameters; GPT-3 used approximately 1.7 tokens per parameter (Brown et al. 2020), while Llama 2 70B used approximately 28.6 (Touvron, Martin, et al. 2023). These descriptive ratios do not establish whether either run is compute-optimal under a different architecture, corpus, objective, or hardware budget. Controlled model-size and token-count sweeps are required for that diagnosis.

Applying the diagnostic

If a training run underperforms expectations, extending one run can reveal whether additional optimization steps still help, but a plateau does not identify data starvation by itself. Learning-rate schedules, optimization limits, model capacity, and data quality can all produce a plateau. A reliable diagnosis compares controlled runs that vary model size, token count, and compute allocation while holding the evaluation protocol fixed.

When controlled runs do point to the data, the remedy that keeps compute fixed is better selection, which changes the shape of the learning curve itself. Seeing that change requires switching axes, from the frontier's training compute to the core metrics' dataset size, where the area under each curve is the AULC defined in section 9.6.1. In the illustrative curves of figure 9.5, efficient selection bends into an accuracy plateau near 25 percent of the data while uniform random sampling climbs gradually across the entire budget. The horizontal separation between the curves is the reduction in required samples, and at an identical 20 percent data budget the vertical gap is the accuracy that selection buys without any additional compute.

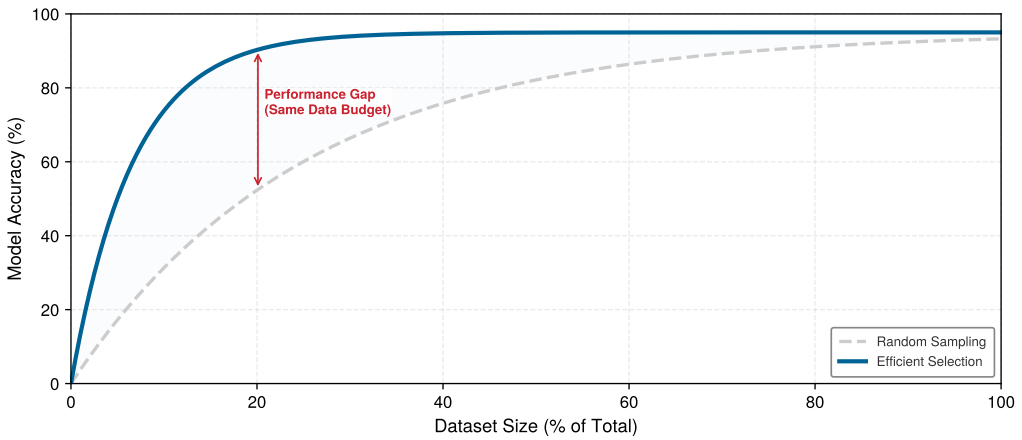


Figure 9.5: Illustrative Diminishing Returns of Data: The synthetic data-efficient curve (blue) reaches the same plateau with fewer samples than random sampling (gray). The vertical red arrow marks the modeled performance gap at a fixed dataset size; measured curves need not have this shape.

These evaluation metrics govern the transition point where indiscriminate data collection yields to targeted curation, identifying when marginal samples waste training compute rather than improving generalization. Yet algorithmic efficiency does not guarantee that a reduced dataset preserves model quality across operational edge cases.

Data selection techniques implicitly rank sample value, and validating that a curated dataset preserves model quality requires systematic benchmarking across three dimensions. Coverage metrics validate that coreset selection preserved representation across classes and demographic

groups. Distribution alignment metrics (such as KL divergence and population stability index, which section B.2.2 defines) detect whether the curated training set drifted from the deployment distribution. Label quality metrics (inter-annotator agreement, and confident learning, which flags examples whose given label disagrees with a model's confident out-of-sample prediction) validate that active learning did not introduce systematic labeling errors. A 50 percent dataset reduction is only valuable if benchmarking confirms the model maintains target accuracy, calibration, and robustness.

↪ Coverage, label-quality, and distribution-alignment checks are developed into a full data-benchmarking protocol in section 12.11.2.

The validation target can itself be unreliable. Benchmark accuracy can overstate generalization when test sets fail to capture operational distribution shifts, demanding the same scrutiny for evaluation benchmarks as for curated training sets.

> **Example 9.1: The benchmark replication gap**
>
> **Scenario**: Classifiers evaluated on newly collected test sets experienced 3 to 15 percentage-point accuracy drops on CIFAR-10 and 11 to 14 points on ImageNet (Recht et al. 2019).
>
> **Diagnosis**: The replicated collection produced slightly harder images than the original test sets. The study found that benchmark adaptivity did not explain the gap, even though repeated use of a fixed test set remains a general evaluation risk.
>
> **Systems lesson**: Evaluation-set construction affects measured generalization. Data-selection pipelines should validate curated subsets on independently collected and deployment-relevant test sets rather than trusting one long-reused benchmark.

An efficiency gain measured against a single benchmark can evaporate under distribution shift; a curated subset must demonstrate consistent quality across multiple held-out distributions before its reported reduction ratio is trusted.

Disciplined metrics separate genuine compute savings from deferred computational debt: a selection strategy must be evaluated against its stated objective, counting the overhead of scoring, filtering, and staging samples alongside training, labeling, or reuse savings at target quality. When teams optimize selection algorithms against isolated benchmarks rather than end-to-end pipeline constraints, intuitive heuristics frequently backfire. They waste cluster cycles on scoring overhead, destabilize optimization trajectories, or silently corrupt downstream generalization.

These metrics apply to three complementary levers, taken in order of when they act: prune static waste upfront, steer dynamic batches during execution, and synthesize missing coverage on demand. Static pruning comes first because it fixes the subset before target-model training, though its scoring process may use a trained proxy.

9.7 Static Pruning

Web-scale scraping routinely gathers terabytes of boilerplate text, duplicate images, and malformed records that waste cluster compute before a single valid gradient is calculated. Static pruning and pretraining filtration eliminate this uninformative volume upstream of the training loop. By purging redundant and corrupt records before target-model training, static filtration shrinks the operation count (O) without modifying model architectures or execution kernels.

9.7.1 The case for smaller datasets

Empirical scaling studies show that dataset volume alone does not dictate model accuracy: large-scale corpora harbor substantial redundancy. In many regimes, a significant fraction of training examples contributes negligible gradient signal while consuming full forward-backward compute.

On CIFAR-10, Paul et al. (2021) report that EL2N pruning (which scores each example by its prediction error early in training) removed half of the training data without reducing accuracy. On ImageNet-1K, Sorscher et al. (2022) report that a self-supervised prototype metric discarded 20 percent of the data without sacrificing performance. The pattern extends to language modeling, where web-scraped corpora like The Pile[5] and C4[6] contain enough duplicate and templated content to make deduplication a systems issue. In datasets studied by Lee et al. (2022), approximate near-

[5] **The Pile**: An approximately 886 GB English text corpus (reported as 825 gibibytes by the source) aggregating twenty-two sub-datasets, including PubMed, ArXiv, GitHub, Project Gutenberg, Common Crawl, Stack Exchange, Wikipedia, and USPTO patents (Gao et al. 2020). Its multi-source design makes it a useful example of data diversity: each source family has a different duplication, quality, and domain-coverage profile, so selection pipelines must preserve coverage while removing redundant text.

[6] **C4 (Colossal Clean Crawled Corpus)**: C4 applies filtering to Common Crawl data, including language detection, deduplication of repeated three-sentence spans, and removal of pages that fail heuristic quality checks, producing approximately 750 GB of cleaned English text (Raffel et al. 2020). Filtering has its own compute cost and can remove useful material, so its ROI and quality effects must be measured rather than inferred from corpus size alone.

duplicate removal affected 3.04 percent of C4 and 13.63 percent of RealNews, and deduplicated training reduced memorization while preserving or improving perplexity.

The reported gains are benchmark-specific. Pruning effectiveness depends on intrinsic dataset redundancy, the selection algorithm, and target model capacity; production deployments require task-specific validation before committing to aggressive filtration.

Individual data points need not provide equal value for training. This heterogeneity follows in part from how classifiers learn decision boundaries. Many samples fall far from a boundary: a picture of a dog in good lighting may become easy once the model has learned its dominant features, while ambiguous and underrepresented cases can remain informative. Label quality also affects data requirements. Conflicting labels can push updates in opposing directions, so reaching a target error may require more examples or more passes over them. The size of that penalty depends on the task and training procedure. A **data quality multiplier** therefore comes from comparing learning curves at the same held-out error, rather than assigning a universal convergence rate to all noisy data.



Napkin Math 9.2: The data quality multiplier

Problem: Two hypothetical pilot learning curves reach the same held-out error using different amounts of clean and noisy data. With epoch count and per-sample work held fixed, what training-work multiplier do these sample counts imply?

Setup: Consider a representative evaluation where a fixed model architecture and training recipe reach target error $\epsilon = 0.01$ (1 percent) under clean versus noisy training partitions:

- Clean data: $D_{\text{clean}} = 10{,}000$
- Noisy data: $D_{\text{noisy}} = 16{,}000$

Result: Dividing the noisy-data count by the clean-data count gives 1.6×. Under the fixed training recipe, the noisy dataset therefore requires 1.6× as much training work to reach the same error.

Systems insight: The avoided training work sets a budget for cleaning. Cleaning pays for itself only when correction or review costs less than the work it removes, with required coverage preserved. If the two learning curves require different epoch counts, compare total examples processed to the target instead of unique sample counts. The decision depends on measured learning curves and pipeline costs, not on the presence of noise alone.

9.7.2 Data deduplication

The cheapest samples to remove are the ones that repeat what the dataset already contains. Repeated content costs a full forward and backward pass on every presentation, even when that repetition adds little useful learning signal. Exact hashing and shingle-based near-duplicate detection can identify candidates for removal without training a scoring model. Deduplication therefore comes first among the static levers and can provide immediate efficiency gains. This makes it one of the most accessible optimizations in data selection, but near-duplicate thresholds must be validated so the pipeline does not remove useful distributional signal.

The simplest form of deduplication uses hash-based methods for exact matches. By computing a cryptographic hash (MD5 or SHA-256) for each sample and removing those with identical hashes, practitioners can eliminate byte-for-byte duplicates that inevitably accumulate in large web-scraped corpora. This process is computationally cheap, scaling linearly with dataset size, and can be parallelized trivially.

Near-duplicate detection addresses content that differs at the byte level while retaining substantial token- or character-shingle overlap. For text, MinHash[7] with Locality-Sensitive Hashing[8] (LSH) approximates Jaccard similarity[9] efficiently. In practice, the pipeline converts each document into a set of overlapping character or word w-shingles, compresses these sets into fixed-length MinHash signatures where the probability of hash collision equals Jaccard similarity, and partitions the signatures into L bands of r rows using LSH. Documents matching across all rows in any single band collide in the same bucket, restricting expensive similarity verification to likely candidates and

↩ Deduplication as a deterministic data-cleaning step that keeps training and serving pipelines consistent is established in section 4.6.1.

7 | **MinHash**: Invented by Broder (1997) to detect duplicate web pages for AltaVista, the algorithm creates compact signatures using random hash functions such that similar documents produce similar signatures with high probability. Each signature compresses a document to a fixed-size sketch, enabling pairwise similarity estimation in $\mathcal{O}(s)$ time for sketch size s.

8 | **Locality-sensitive hashing (LSH)**: LSH hashes MinHash signatures into buckets so that similar signatures are more likely to collide. Candidate generation avoids an exhaustive all-pairs comparison, while runtime and memory depend on signature width, banding, bucket sizes, and the number of candidates that require verification.

9 | **Jaccard similarity**: Defined as $|\mathcal{A} \cap \mathcal{B}|/|\mathcal{A} \cup \mathcal{B}|$ for two sets $\mathcal{A}$ and $\mathcal{B}$, ranging from 0 (disjoint) to 1 (identical). For deduplication, the metric compares sets of shingles across documents of different lengths. The threshold must be calibrated for the shingle definition, corpus, and downstream quality because no universal cutoff separates duplicates from legitimately related documents.

avoiding an impossible $\mathcal{O}(D^2)$ all-pairs comparison across a corpus of D web documents numbering in the billions. It can detect lightly edited content with high overlap, but it does not establish semantic paraphrase equivalence.

For images, perceptual hashing produces signatures robust to minor transformations like resizing and compression, identifying visually similar candidates stored in different formats. Embedding-based similarity can retrieve semantic near-neighbor candidates by computing dense representations (CLIP[10] (Radford et al. 2021) for images, sentence transformers for text), though proximity does not itself prove duplication and the approach incurs higher computational overhead.

Deduplication can reduce repeated-content memorization and avoid spending FLOPs on exact or near duplicates. It can also alter the corpus's intentional empirical weighting, so thresholds and downstream quality must be validated rather than assuming that every duplicate is harmful.

The systems impact of deduplication shifts when input examples index persistent model state rather than streaming through stateless compute layers.

10 **CLIP (Contrastive Language-Image Pretraining)**: Pretrained on 400 million image-text pairs, CLIP maps semantically related images and text into a shared embedding space (Radford et al. 2021). Embeddings can retrieve candidates for review, but semantic proximity is not proof that two samples are duplicates. Generating embeddings also costs substantially more than computing perceptual hashes, with the ratio depending on model and hardware.



Lighthouse 9.1: DLRM and embedding deduplication

The DLRM lighthouse model is memory capacity-bound, and at production scale its embedding tables reach terabytes of storage for billions of user/item IDs. Much of this capacity is wasted on *cold embeddings*, IDs that appear rarely in training data.

Data selection for DLRM can include interaction deduplication and embedding pruning for cold IDs. Removing repeated interactions reduces training examples but shrinks an embedding table only when IDs disappear or the allocation, hashing, or sharing policy changes.

9.7.3 Data pruning by quality

Deduplication removes samples that repeat information; quality pruning targets samples whose information is wrong or absent. Quality-based pruning eliminates samples that either contribute no meaningful signal or introduce contradictory information that confuses the optimization process.

Label error detection can be a high-impact form of quality pruning. Tools like Cleanlab identify samples where the assigned label may be incorrect based on model confidence patterns across training. A sample that the model consistently predicts as class A but is labeled class B may be a hard boundary case, a distribution shift, a model blind spot, or an annotation mistake. The score should trigger review rather than automatic deletion; confirmed errors can then be corrected or removed.

Outlier removal addresses a different pathology: samples far from any cluster center in feature space. Such points can indicate noise, annotation errors, or corruption, but they can also be the rare edge cases that define deployment risk. The key is distinguishing informative outliers from invalid samples, with conservative thresholds and slice-level review before removal.

Low-information filtering applies domain-specific heuristics to remove samples that lack sufficient signal for learning. For text corpora, this often means removing high-perplexity garbled text (perplexity is a language model's measure of how surprising a text is, so high values can flag incoherent strings but also valid specialist or rare-language text) and, in some pipelines, low-perplexity boilerplate or repetitive text. For image datasets, filtering targets blurry, corrupted, or near-uniform samples that provide little visual information.

Beyond heuristics and perplexity, industrial pretraining pipelines rely heavily on *classifier-based filtering*: training a lightweight, high-throughput linear classifier (such as fastText) using curated, high-quality reference text (e.g., Wikipedia, books, and scientific papers) as positive examples and uncurated web dumps as negative examples. This classifier assigns each scraped document a continuous quality score, allowing pipelines to filter out millions of low-quality web pages at gigabyte-per-second ingestion speeds without the heavy computational burden of evaluating a full neural language model.

11 **Coreset (core set)**: In computational geometry, a small subset can approximate a specified geometric objective within a controlled error factor (Agarwal et al. 2005). Machine learning adapts this idea as a selection principle, but a guarantee for an embedding and distance objective does not by itself guarantee neural-network accuracy. The retained subset must still be validated on the target task.

9.7.4 Coreset selection algorithms

Once duplicates and defective records are gone, the remaining samples are unique and valid but still unequal in value, which is the heterogeneity that opened this section. Ranking them requires

systematic selection criteria. **Coreset selection**[11] turns the static pruning decision into a coverage problem: keep the smallest subset that preserves the statistical properties of the entire dataset.

The systems decision is where to spend the selection budget: on cheap coverage metrics that preserve distributional structure, or on costlier training-dynamics scores that better target the decision boundary. The decision also needs a guardrail before any score is applied: classes, demographic groups, time windows, and rare failure modes that matter at deployment require minimum representation, because the highest-average-ICR subset can still remove the examples that define production risk.

Geometry-based methods select samples that cover a chosen representation without requiring target-model training. The k-Center algorithm,[12] a facility-location-style objective, selects samples that reduce the maximum distance from any point to its nearest selected center under the chosen embedding and metric.

Herding takes a different approach. Welling's original procedure iteratively constructs representative pseudo-samples whose feature statistics approximate observed moments (Welling 2009). Related moment-matching objectives can guide subset selection from a fixed pool. These methods are computationally attractive because they operate on feature representations, but their scores are not inherently label-aware.

Training-dynamics methods use a proxy model to identify examples that may be important. GraNd (Gradient Normed) and **EL2N (error L2-norm)**[13] score samples by gradient magnitude or prediction error early in training (Paul et al. 2021). Theoretically, sample valuation can be framed via *influence functions* (Koh and Liang 2017), which measure how removing a sample changes validation loss. Evaluating true sample influence requires calculating an inverse empirical Hessian ($\mathbf{H}^{-1}\nabla_\theta \mathcal{L}$), where the Hessian matrix $\mathbf{H}$ captures the local curvature of the loss surface across all parameter interactions. For a model with P parameters, storing the $P \times P$ Hessian demands $\mathcal{O}(P^2)$ memory, and computing its inverse requires $\mathcal{O}(P^3)$ arithmetic operations. Exact second-order influence is therefore intractable for models with millions or billions of parameters.

Systems methods circumvent this barrier by relying on lightweight first-order approximations: GraNd scores samples using expected gradient norm ($\|\nabla_\theta \mathcal{L}\|_2$), while EL2N observes that early in training, the gradient norm with respect to linear layers is strongly correlated with output prediction error ($\|\hat{\mathbf{y}} - \mathbf{y}\|_2$, the distance between the predicted probability vector $\hat{\mathbf{y}}$ and the one-hot label $\mathbf{y}$). This enables practitioners to score prediction error using a single forward pass without computing backward-pass gradients. Error and uncertainty differ: a confident prediction of the wrong class can have high EL2N despite low predictive uncertainty. High scores identify examples the proxy finds difficult, but they can also identify noise or unrepresentative samples and therefore need coverage and quality checks. The same work reports useful score transfer across several architectures, which can enable less expensive proxy-based selection. A third training-dynamics signal, forgetting events,[14] tracks how often a sample is correctly classified and later misclassified during training (Toneva et al. 2019).

Each algorithm in table 9.3 balances upfront scoring computation against downstream training reduction. Because geometry-based and training-dynamics methods optimize different proxies, their relative ranking depends on dataset scale, feature representations, target model capacity, and scoring budgets; a score is computationally justified only when downstream training savings exceed evaluation overhead.

Table 9.3: Coreset Selection Algorithm Comparison: D = dataset size, K = coreset size. Geometry-based and training-dynamics methods optimize different proxies, so their quality ranking depends on the dataset, representation, target model, and scoring budget.

Method	Compute Cost	Requires Training	Best For	Limitation
k-Center	$\mathcal{O}(D^2)$ or $\mathcal{O}(DK)$	No	Coverage, exploration	Ignores label information
Herding	$\mathcal{O}(DK)$	No	Distribution matching	Matches chosen moments
GraNd	$\mathcal{O}(\text{epochs} \times D)$	Yes (few epochs)	Decision boundaries	Requires proxy training
EL2N	$\mathcal{O}(\text{epochs} \times D)$	Yes (few epochs)	Error-based scoring	Can rank noise as hard
Forgetting	$\mathcal{O}(\text{full training})$	Yes (full)	Hard examples	Expensive to compute

[12] **k-center algorithm**: Its greedy strategy iteratively picks the point farthest from the existing centers under a chosen embedding and metric. Sener and Savarese use this core-set framing for convolutional-neural-network active learning (Sener and Savarese 2018). The resulting coverage bound applies to that geometric objective, not directly to downstream accuracy or rare-class preservation.

[13] **EL2N (error L2-norm) and GraNd (gradient normed)**: These methods score samples based on their error or gradient norm early in training, identifying samples the model finds most difficult. The practicality of this approach relies on *transferability*, where scores from a small proxy model can guide data selection for a much larger target model. For instance, a proxy trained for five epochs can generate scores to curate a dataset for a full 90-epoch production training run (Paul et al. 2021).

[14] **Forgetting events**: This method identifies valuable examples by tracking when the model "forgets" them, transitioning from a correct to an incorrect classification during training. The central trade-off is the high cost of this analysis, which requires a full training run. However, the resulting importance scores can transfer from small proxy models to larger target models (for example, ResNet-18 to ResNet-50), enabling inexpensive proxy-based selection when ranking quality holds for the target task (Toneva et al. 2019).

Geometry-based methods prioritize coverage in a chosen feature representation, while training-dynamics methods use signals such as error or forgetting. Uncertainty sampling, which active learning uses as a query rule (section 9.8.2), concentrates the budget near the current decision boundary; EL2N instead ranks prediction error, which can also be high for confidently wrong examples away from the boundary. Figure 9.6 contrasts that boundary-focused allocation with uniform random sampling of the same five-point budget. It illustrates one family of selection rules; a geometry-based method such as k-center would instead spread its picks to cover the feature space.

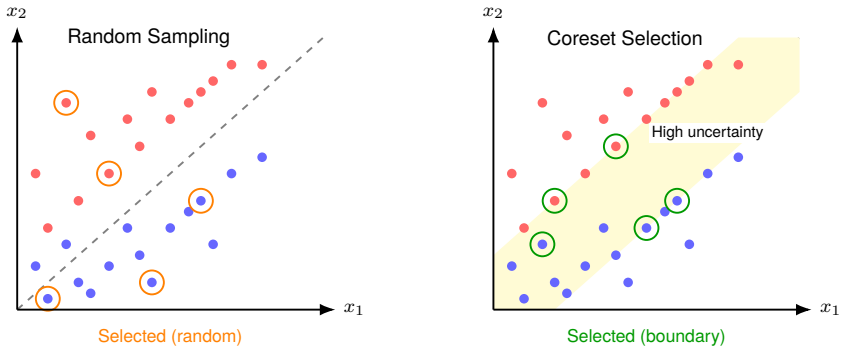


Figure 9.6: Illustrative Boundary-Focused Selection: Both panels select five samples, isolating allocation rather than budget. Random sampling distributes those choices across the feature space; a boundary-focused rule such as uncertainty sampling concentrates them in the shaded band around the current decision boundary.

EL2N with a small proxy model offers one practical balance between scoring cost and selection quality. The approach trains a lightweight model for a few epochs, computes EL2N scores, and selects a subset subject to coverage and quality checks. The proxy must provide rankings that transfer to the target model, and the investment pays off only when downstream training savings exceed selection overhead. A concrete scenario shows the workflow; whether its 10 percent subset preserves accuracy is exactly what validation must establish.

Example 9.2: Coreset selection in practice

Scenario: An engineering team aims to select a 100,000 coreset (10 percent) from 1 million training images for faster model iteration.

Diagnosis: Naive random subsampling can miss rare classes and boundary examples. Training a lightweight proxy model for 5 epochs calculates EL2N error scores, prioritizing high-error examples while checking rare-class coverage.

Systems lesson: Proxy-based coreset selection replaces some full-data passes with targeted scoring compute. It accelerates full-model training only when the proxy ranking transfers and coverage checks preserve rare classes and edge cases.

Listing 9.1 turns proxy prediction error into reusable selected indices. The `compute_el2n_scores` function spends a small amount of probe compute to measure which samples still confuse a briefly trained model, and `select_coreset` retains the highest-scoring examples. This top-k sketch needs quality and coverage checks before its subset replaces the full training pool. The scoring pass requires an unshuffled dataloader so each score position remains aligned with its dataset index.

Listing 9.1: EL2N-Based Coreset Selection: The `compute_el2n_scores` function trains a proxy model for a few epochs, then measures prediction error via L2 distance from one-hot labels. High scores identify samples the proxy finds difficult, including potentially noisy examples. The `select_coreset` function illustrates ranking by this score; production use also requires coverage, noise, and target-quality validation.

```
def compute_el2n_scores(model, dataloader, num_epochs=5):
    """Compute EL2N scores.

    Returns L2 norm of (prediction - one_hot_label).
    """
    # Train proxy model for a few epochs to get meaningful predictions
    train_proxy(model, dataloader, num_epochs)

    scores = []
    model.eval()
    for x, y in dataloader:
        logits = model(x)
        probs = softmax(logits, dim=1)
        # One-hot encode labels
        one_hot = zeros_like(probs).scatter_(1, y.unsqueeze(1), 1)
        # EL2N score = L2 distance between probabilities and one-hot target
        el2n = (probs - one_hot).norm(dim=1)  # High = large prediction error
        scores.extend(el2n.tolist())
    return scores


def select_coreset(scores, dataset, fraction=0.1):
    """Select top-k samples with the largest prediction errors."""
    k = int(len(dataset) * fraction)
    # Sort by score descending (largest prediction error first)
    indices = argsort(scores, descending=True)[:k]
    return Subset(dataset, indices)


# Illustrative 10% subset; validate accuracy and coverage before use
scores = compute_el2n_scores(proxy_model, full_loader)
coreset = select_coreset(scores, full_dataset, fraction=0.1)
train_full_model(model, coreset)
```

A coverage floor changes the indices retained by the score ranking. Consider disjoint classes A and B, a total coreset budget K as in table 9.3, and task-validated minimum counts a and b for the respective classes. First reserve the highest-scoring a eligible examples from A and the highest-scoring b from B. Fill the remaining $K-a-b$ slots from the global ranking, skipping indices already reserved. The allocation is infeasible if $K < a+b$, if either class lacks enough eligible examples to meet its floor, or if the eligible pool is smaller than K; the pipeline must revise the budget or acquire missing coverage before training.

Suppose the unconstrained top-K ranking contains only A examples and the budget can meet both floors. Reserving B's floor replaces the lowest-scoring b members of that all-A subset with the highest-scoring eligible B examples, while the remaining slots still follow the score order. Coverage thus has an explicit opportunity cost in the selection score. Overlapping demographic or operational slices require joint constraint handling because one example may satisfy several floors; summing those floors as if the slices were disjoint can reject a feasible budget. Even a feasible allocation establishes only retained counts. Independent evaluation must still establish target quality and performance on the required slices.

Together, these three static pruning techniques (deduplication, quality filtering, and coreset selection) shrink O before training begins, subject to the same fixed-schedule caveat as before. A schedule that adds compensating epochs or steps gives back part of the saving.

Static pruning answers a question about *what* to keep, but it treats the answer as fixed. Once the pruned dataset is set, every epoch trains on the same subset. A sample's usefulness, however, can change as the model learns: examples that challenge an undertrained model may become easy after

sufficient gradient updates. Dynamic selection techniques address this limitation by adapting the training data at each stage based on what the model has already mastered.

9.8 Dynamic Selection

A sample's marginal learning signal is non-stationary across training. Early in optimization, broad coverage provides stable gradients that establish general feature representations. As parameters converge, the gradient norms of mastered examples decay toward zero. Executing forward and backward passes on these samples continues to consume accelerator execution cycles and high-bandwidth memory (HBM) transfers while producing negligible parameter updates, driving their effective ICR toward zero. Dynamic selection mitigates this waste by adapting batch composition to the model's current state. This shifts selection from an offline preprocessing step into the active execution loop, modifying the operation count (O) at runtime while requiring data loaders to balance scoring overhead against accelerator throughput. The same adaptive logic extends to labels. Active learning decides which examples earn a human label as the model's uncertainty shifts, and semi-supervised learning lets the evolving model supply provisional labels for the unlabeled remainder.

9.8.1 Curriculum learning: Easy to hard

The first dynamic selection technique, **curriculum learning**[15] (Bengio et al. 2009; Soviany et al. 2022), structures the order in which data is presented to the model. Instead of random shuffling, it starts with simpler examples and gradually introduces more complex ones, mirroring how humans learn by mastering basics before advancing to harder material.

Curriculum learning can change the optimization trajectory by controlling which gradients dominate at each stage. Easy examples may provide stable early signals, while examples scored as hard can contain useful boundary information, label noise, or outliers. An easy-to-hard schedule can smooth training on some workloads, but its benefit depends on the difficulty score, pacing rule, model, and data; any claimed epoch reduction must therefore be treated as illustrative rather than guaranteed.

Implementing a curriculum requires two components: a difficulty scorer that ranks samples, and a pacing function that controls how quickly hard samples are introduced. A common choice is linear pacing:

$$\text{samples}_{n_{\text{epoch}}} = \texttt{sort_by_difficulty}[: D \cdot \min(1, n_{\text{epoch}}/N_{\text{warmup}})]$$

where n_{epoch} is the current epoch, D is the total dataset size, and N_{warmup} is the number of warmup epochs before the full dataset becomes available. Early epochs train on the easiest fraction $n_{\text{epoch}}/N_{\text{warmup}}$ of the D samples; after warmup, training proceeds on the full dataset. From a systems standpoint, dynamically re-sorting multi-terabyte datasets epoch-by-epoch destroys sequential I/O read-ahead and stalls accelerators on storage latency. Production data loaders implement curricula by pre-sorting data offline into discrete difficulty-tiered shards or buckets. During training, workers stream shards sequentially from storage while maintaining local shuffle buffers within each tier, preserving high sequential storage bandwidth while maintaining batch diversity and gradient stability.

The pacing rule fixes one saving before any benchmark is run. Summed over the warmup, the loader serves $\sum_{n_{\text{epoch}}=1}^{N_{\text{warmup}}} D \cdot n_{\text{epoch}}/N_{\text{warmup}} = D(N_{\text{warmup}}+1)/2$ samples in place of $N_{\text{warmup}} \cdot D$, so the warmup epochs cost roughly half their full-data FLOPs. A curriculum can therefore save compute in two distinct ways, through cheaper warmup epochs and through any reduction in the total epochs needed to reach target quality, and a claimed saving must say which of the two it measured.

The difficulty scorer is a systems choice because it trades probe-compute overhead against ordering quality, as table 9.4 shows. Loss and confidence scoring buy better ordering with extra inference, heuristics avoid compute but require domain knowledge, and self-paced scoring moves adaptation into the training loop itself.

[15] **Curriculum learning**: From Latin *currere* ("to run"), originally meaning "the course to be run," a metaphor that maps directly to the technique, with training data as a course run in deliberate order and the easy stretches first. Curriculum learning can act as a continuation method for nonconvex optimization, with easier examples shaping the early optimization trajectory before harder examples are introduced. From a systems perspective, the ICR of a sample can vary during training, which motivates changing the mix of examples over time.

Table 9.4: Difficulty Scoring Strategies for Curriculum Learning: The rows compare each scorer's signal source with the system prerequisite or trade-off it introduces.

Strategy	Difficulty Score	Best For
Loss-Based	Loss from probe model (low = easy)	General-purpose; requires probe training
Confidence-Based	Teacher model confidence (high = easy)	When teacher available; distillation setups
Domain Heuristics	Sentence length, image complexity	No extra compute; domain knowledge required
Self-Paced	Current model's loss (updated each epoch)	Adaptive; no probe needed

To see how curriculum savings translate to systems accounting, consider an illustrative workload where a linear-warmup curriculum reaches target accuracy in 115 epochs compared to 150 epochs for the standard shuffled baseline—an apparent 23.3 percent epoch reduction. Counting epochs alone, however, obscures the actual systems cost. Because linear pacing serves fewer samples during warmup, early epochs consume fewer FLOPs than post-warmup epochs; conversely, difficulty scoring and shard reordering introduce data-loading overhead that must be offset by the training speedup.

Alternative curriculum strategies diverge further in their computational demands. In a learned curriculum, an auxiliary network dynamically weights examples—down-weighting likely mislabeled instances—rather than enforcing a fixed difficulty schedule. Conversely, *anti-curriculum* presents the hardest examples first when boundary definition accelerates early convergence, while *self-paced learning* continuously updates sample difficulty using the current model's training loss. Because optimal sample ordering depends on the architecture, optimizer, and label noise, curriculum gains cannot be assumed from nominal epoch reductions; they must be verified by measuring end-to-end wall-clock time at the target quality metric.

9.8.2 Active learning: Human-in-the-loop

Curriculum learning optimizes the order in which samples are presented but assumes all samples are already labeled. This assumption breaks down in specialized domains where labeling requires substantial expertise, time, or money. Active learning[16] (Settles 2012; Ren et al. 2021), which section 4.7.3 introduced as a way to stretch a labeling budget, becomes a selection loop here. Rather than labeling everything upfront, it shifts the optimization target from choosing which labeled samples to train on to choosing which unlabeled samples are worth labeling.

Active learning transforms data acquisition from static collection into an iterative feedback cycle. The closed loop diagrammed in figure 9.7 sequences this workflow across five stages: the selection strategy queries the unlabeled pool based on model uncertainty, routes ambiguous instances to a human oracle, merges newly labeled data into the training set, and retrains the model to update uncertainty scores over the remaining pool.

The effectiveness of active learning depends critically on the query strategy used to select samples for annotation (Settles 2009, 2012; Ren et al. 2021). The simplest approach, uncertainty sampling, selects samples where the model is least confident, such as predictions near 0.5 probability for binary classification. It is inexpensive to score with one model but can overconcentrate queries near one ambiguous region. Query-by-committee extends this idea by training multiple models and selecting samples where they disagree most, capturing epistemic uncertainty that a single model might miss.

For practitioners willing to invest more compute, expected model change selects samples that would cause the largest gradient update if labeled. This approach provides a theoretically grounded but expensive alternative. Diversity sampling complements uncertainty-based methods by selecting samples dissimilar from currently labeled data, improving coverage rather than clustering every query around the same ambiguous region.

Active learning is particularly valuable in domains where labeling requires expertise. In medical imaging, for instance, an AI system diagnosing diseases from X-rays may be confident on common conditions but uncertain about rarer cases. By focusing human annotation on these ambiguous cases, active learning optimizes the use of expensive expert time while accelerating model improvement.

[16] **Active learning**: The "active" component is the learning algorithm itself selecting which unlabeled samples a human expert should label next. This reframes the problem from a computational optimization (training on given data) to a financial one: maximizing model improvement per dollar spent on expert labeling. Querying the most informative examples can substantially reduce labeling needs compared with random sampling, but the multiplier is task-, model-, and oracle-dependent.

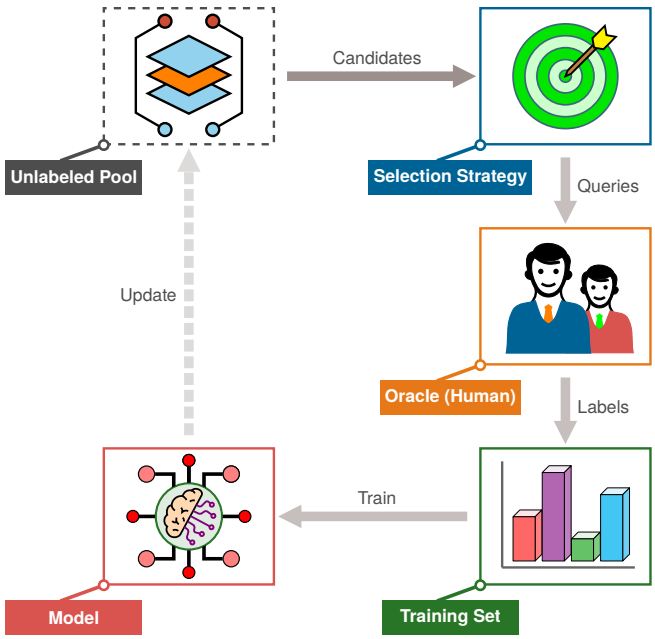


Figure 9.7: The Active Learning Closed Loop: The selection cycle trades proxy-scoring compute for queries chosen from an unlabeled pool for oracle annotation. Uncertainty, diversity, and coverage criteria seek more useful information per annotation than undirected sampling.

The economic implications can be substantial in specialist domains, where annotation cost and turnaround dominate the compute required to score candidate samples. By governing the uncertainty triage step within the closed loop (figure 9.7), targeted query strategies arbitrate which instances warrant expensive oracle invocation before retraining. A concrete budget comparison quantifies how this selective labeling reduces both financial expenditure and redundant training work under stated assumptions.

Napkin Math 9.3: The active learning ROI

Problem: A hospital team has 1 Million unlabeled scans, a \$5/label specialist-label cost, and a \$500,000 budget. How does uncertainty sampling change labeling cost and per-epoch training work relative to the budget-matched random baseline?

Scenario A: Naive Labeling

- **Cost**: Labeling all 1 Million scans would cost \$5,000,000 (10× over budget).
- **Budget-matched baseline**: $\$500,000 \div \$5/\text{label} = 100,000$ random scans.
- **Naive labeling outcome**: Random selection does not guarantee rare-pathology coverage.

Scenario B: Active Learning

- **Strategy**: Uncertainty sampling selects 50,000 scans for specialist labeling.
- **Cost**: $50,000 \times \$5/\text{label} = \$250,000$ (50 percent under budget).
- **Training work**: $100,000 \div 50,000 = 2\times$ fewer examples per epoch than the budget-matched random baseline.
- **Active learning outcome**: Matching 100,000 random scans requires empirical validation.

Systems insight: Against the budget-matched random baseline, selection saves $\$500,000 - \$250,000 = \$250,000$; accuracy still requires empirical validation.

The notebook's 2× label reduction is an assumption, and the size of that multiplier is what decides the outcome. In the illustrative learning curves of figure 9.8, uncertainty sampling reaches 90 percent accuracy with roughly 2,500 annotations while uniform random sampling needs about 10,000, a 4× gap.

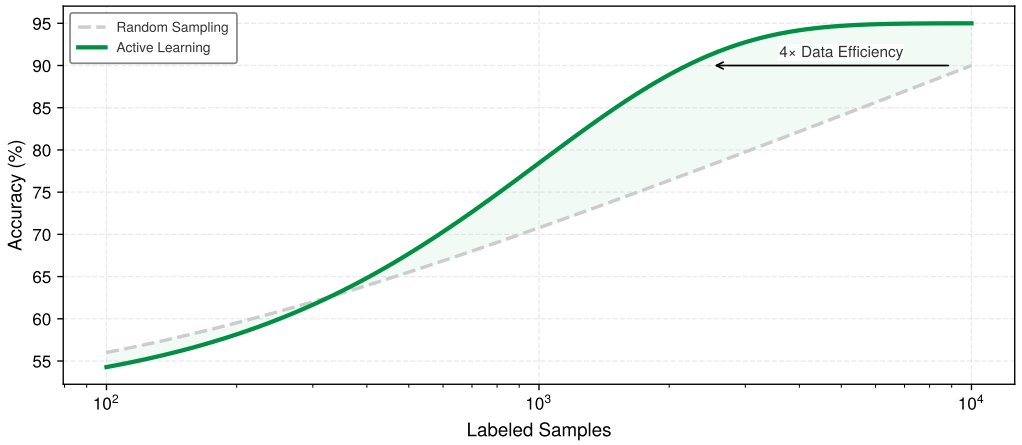


Figure 9.8: The Active Learning Multiplier: Model accuracy is plotted against labeled samples on a logarithmic axis. In this illustrative scenario, active learning (green solid) reaches the same accuracy with fewer labels than random sampling (gray dashed); the arrow marks an approximately 4× label-efficiency gap near 90 percent accuracy.

At a fixed price per label, label count and labeling dollars are the same axis, so the figure and notebook 9.3 differ only in the multiplier they assume. The notebook plans on the conservative 2×; the figure shows what a more favorable task would look like. Because the realized multiplier depends on the task, the model, and the query rule, a team budgets on the conservative value and measures the real one on its own data. Active learning also incurs scoring, acquisition, and repeated-retraining costs, so end-to-end savings can be smaller than the label-count gap.

Active learning can do more than save labeling cost: a suitable query rule can concentrate review on uncertain, diverse, or failure-relevant examples.

> Example 9.3: Hard negative mining in a smart doorbell
>
> **Scenario**: A smart doorbell vision detector receives continuous video feeds dominated by empty backgrounds (easy negatives) and normal pedestrian poses.
>
> **Diagnosis**: Random frame sampling can miss rare false positives such as statues, laundry piles, and human-shaped shadows. Active learning routes uncertain predictions ("Person: 51%") and sampled production errors to human reviewers for targeted labeling.
>
> **Systems lesson**: Hard-negative mining can improve labeling efficiency by spending review effort on suspected false positives and other difficult negatives. The query stream still needs diversity and coverage checks so confident but systematically wrong cases are not excluded.

9.8.3 Semi-supervised learning: Using unlabeled data

Active learning optimizes which samples to label but still requires human annotation for every selected example. Semi-supervised learning shifts the engineering trade-off from human annotation to accelerator compute: it uses a small labeled subset to anchor optimization on a larger unlabeled pool, extracting supervisory signals directly from unannotated inputs. This allows models to approach fully supervised accuracy with substantially fewer manual labels, provided the unlabeled pool reflects the target distribution.

Consider an illustrative medical-imaging scenario with 50,000 chest X-rays, of which 500 have been reviewed and labeled[17] by radiologists, a labeling rate of 1 percent. Whether that seed set is

[17] **Illustrative clinical labeling economics**: The review-rate and hourly-rate ranges are scenario assumptions rather than reported measurements. Under those inputs, labeling 500 scans requires 7–10 hours and costs $1,000–3,000; labeling all 50,000 costs $94,000–300,000. The arithmetic illustrates why semi-supervised learning can be attractive when expert labels are expensive, but actual review time, adjudication, prevalence, and validation requirements must be measured.

adequate depends on the task and distribution. Semi-supervised learning can use the remaining 49,500 images when the unlabeled pool matches the target distribution and the method is validated on labeled data.

The core insight behind semi-supervised learning is that unlabeled data, while it cannot directly teach the mapping from inputs to outputs, contains structural information about the input distribution $p(x)$ that can constrain the hypothesis space. Under smoothness and cluster assumptions, a decision boundary that cuts through a dense region assigns different labels to nearby inputs and is less plausible than one in a low-density region. Semi-supervised methods exploit these assumptions, which must be checked against the target task.

Three techniques implement this distribution-regularization insight. Pseudo-labeling[18] trains on the labeled subset, generates model predictions on the unlabeled pool, and incorporates samples exceeding a confidence threshold as synthetic ground truth for subsequent retraining. The confidence threshold governs a critical systems trade-off: setting it too low admits label noise that degrades learning, while setting it too high discards informative examples and wastes the unlabeled pool. Confident mistakes can survive either threshold, so confidence must be checked against labeled validation evidence rather than treated as proof of correctness.

Consistency regularization[19] enforces that the model produce invariant predictions across perturbed versions of the same input. When transformations such as cropping, rotation, or color shifts preserve the task label, the loss function penalizes prediction divergence between two stochastic views of an unlabeled sample. Hybrid approaches like FixMatch[20] unify both mechanisms: they generate a pseudo-label on a weakly augmented view when prediction confidence exceeds a threshold, then train the model to predict that label on a strongly augmented variant of the same input.

Label propagation frames semi-supervised learning as graph diffusion: it constructs an affinity graph over sample representations and propagates labels from labeled nodes to unlabeled neighbors. This approach works well when representations form well-separated clusters, though constructing and querying the similarity graph across large datasets introduces substantial memory and nearest-neighbor search overhead.

> **Example 9.4: FixMatch on CIFAR-10**
>
> **Scenario**: A semi-supervised image classification pipeline trains on CIFAR-10 with only 250 labeled samples (25 per class) alongside unlabeled data (Sohn et al. 2020).
>
> **Diagnosis**: The fully labeled CIFAR-10 training set contains 50K labels. FixMatch generates high-confidence pseudo-labels (>0.95 threshold) on weakly augmented unlabeled images and enforces consistency on strongly augmented variants, reaching 94.9 percent accuracy in the cited experiment.
>
> **Systems lesson**: Semi-supervised learning trades additional training compute for fewer manual labels. In an illustrative cost scenario at $1 per label, a supervised baseline with 4,000 labels costs $4,000 in labels plus $50 of compute, or $4,050. FixMatch at 250 labels costs $250 in labels but 5× the compute, $250, for $500. Preparation expense falls by 8.1×; the end-to-end saving depends on the real unlabeled-data and compute costs.

The systems trade-off in semi-supervised learning exchanges labeling effort for additional computation over labeled and unlabeled samples. Its objective is lower total labeling and compute cost at matched quality, which can justify a lower ICR when annotation savings exceed the added training expense. It can approach fully supervised accuracy with fewer labels, but neither the label reduction nor the compute increase is universal. A CIFAR-10 comparison makes one such trade-off concrete (table 9.5).

Table 9.5: FixMatch Label Efficiency on CIFAR-10: With 250 labels (0.5 percent of the dataset), FixMatch reaches 94.9 percent accuracy while using 200× fewer labels than the full training set contains.

Label Budget	Method	Accuracy	Label Efficiency
50,000 (100%)	Full training set	—	Label-count reference

[18] **Pseudo-labeling**: Uses a trained model's own confident predictions as ground-truth labels for unlabeled data; the technique's effectiveness depends on a virtuous cycle: accurate predictions on easy unlabeled examples expand the training set, improving the model, which enables accurate predictions on harder examples. The failure mode is equally self-reinforcing: incorrect pseudo-labels reinforce errors through *confirmation bias*, causing the model to reinforce its own misclassifications.

[19] **Consistency regularization**: Rooted in a task-dependent smoothness assumption: if two inputs x_1 and x_2 are related by a label-preserving perturbation, their predictions should agree; the training objective minimizes divergence between a model's predictions on an input and its augmented version. This is conceptually distinct from data augmentation (which creates more training examples) because it explicitly enforces prediction consistency as a loss term, even for unlabeled data where the "correct" label is unknown. The systems consequence is additional computation for generating and comparing predictions under multiple perturbations, a trade-off that can favor compute-rich, label-poor settings.

[20] **FixMatch**: It generates a pseudo-label using a weakly augmented image and then trains the model to predict that same label for a strongly augmented version of the image. This consistency training is gated by a confidence threshold; a pseudo-label is only used if the model's prediction on the weak augmentation is highly confident (for example, >0.95) (Sohn et al. 2020). This embodies a direct systems trade-off between additional training compute and fewer manual labels; the magnitude depends on the baseline and cost model.

Label Budget	Method	Accuracy	Label Efficiency
4,000 (8%)	FixMatch	95.7%	12.5× more efficient
250 (0.5%)	FixMatch	94.9%	200× more efficient
40 (0.08%)	FixMatch	88.6%	1250× more efficient

The efficiency column counts labels only, relative to the full labeled set. It does not claim that FixMatch matches a fully supervised model, which would have to be compared at equal accuracy. These efficiency gains also depend on benign distribution assumptions. Semi-supervised learning assumes that unlabeled data comes from the same distribution as labeled data, and it struggles when unlabeled data contains out-of-distribution samples (the model confidently mislabels them), when class imbalance is severe (pseudo-labels amplify majority class bias), or when the labeled set does not cover all classes (preventing label propagation for unseen classes). Always validate on a held-out set with true labels to catch distribution mismatch.

Despite these limitations, semi-supervised learning can reduce label requirements while maintaining accuracy on suitable tasks. Across the techniques, coreset selection and deduplication prune low-value samples before training; curriculum learning changes the presentation order; active learning chooses which samples receive human annotation; and semi-supervised learning extracts additional signal from an unlabeled pool. Each technique can reduce dependence on task-specific labels, but none eliminates the need for labeled evaluation. The progression raises a related possibility: task-specific labels may not be necessary for pretraining. The structure of data itself (cat images resemble other cat images, and coherent sentences follow grammatical patterns) may provide a supervision signal, while downstream quality still requires task-aligned validation.

9.9 Self-Supervised Learning

GPT was trained to predict the next token in a sequence. BERT was trained to fill in masked tokens. Neither pretraining objective required task-specific human labels. **Self-supervised learning**[21] (SSL) generalizes this insight: by designing *pretext tasks* that derive supervision from the data's structure, models can learn reusable representations from unlabeled data at scale. Where active and semi-supervised learning reduce the demand for labels, SSL changes the accounting by making unlabeled structure the pretraining signal. Within the intervention map of figure 9.3, SSL sits beside the three levers and extends the usable pool; the question shifts from which labeled samples to train on to how unlabeled samples can support pretraining. It responds to one aspect of the data wall introduced in section 9.1 by broadening what counts as a training signal, while leaving corpus quality, licensing, coverage, and labeled evaluation requirements intact.

[21] **Self-supervised learning**: The pretext task, such as next-token or masked-token prediction, provides a supervisory signal derived from the data itself. This reduces dependence on task-specific human labels but can require substantial pretraining compute. Amortization depends on how broadly the resulting model is reused.

Labels are one form of supervision; a pretext task can also derive a learning signal from the structure of the data, as table 9.6 summarizes.

Table 9.6: Self-Supervised Pretext Tasks by Modality: Each task extracts supervision from data structure rather than human labels, enabling pretraining on large-scale unlabeled corpora.

Modality	Self-Supervised Task	Supervision Signal
Text	Masked language modeling	Predict [MASK] from context
Text	Next-token prediction	Predict next token in sequence
Images	Contrastive learning	Same image (augmented) vs. different images
Images	Masked autoencoding	Reconstruct masked patches
Multi-modal	CLIP-style alignment	Match image-text pairs

Pretext tasks generate supervision signals automatically. Masked-token objectives can encourage models to learn grammatical and semantic structure; next-token prediction teaches continuation structure; and contrastive image objectives encourage visual features that are invariant to selected transformations. These approaches build on convolutional neural network and transformer families, but the systems accounting matters: self-supervision removes task-specific annotation from the

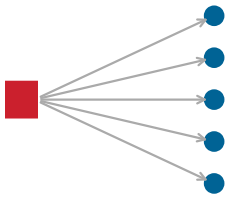
pretraining gate rather than removing data cost. Pretraining can begin on unlabeled corpora without waiting for example-level labels, but collection, filtering, licensing, storage, and processing remain. Separating pretraining from downstream labeling restructures the economics of machine learning when the representation transfers across tasks; task-specific adaptation and evaluation still carry their own data requirements.

When its pretraining cost is reused across tasks, SSL supports the **foundation model paradigm**[22] (Bommasani et al. 2021). The core data selection principles (coreset selection, curriculum learning, and active learning) remain essential within this paradigm. Pretraining corpus curation applies deduplication and quality filtering at web scale, while downstream selection dictates which fine-tuning examples maximize task-specific transfer.

Pretext objectives differ in what they stress on the machine. Contrastive learning trains a model to map two augmented views of the same image to nearby representations while pushing the representations of different images apart. Those other images are the *negatives*, and the objective learns better with more of them. SimCLR (T. Chen et al. 2020) draws its negatives from the rest of the mini-batch, with batches of up to 8,192 examples, so the negative count is paid for in activation memory and, across accelerators, in all-gather traffic that collects every device's representations. Momentum Contrast (MoCo) (He et al. 2020) instead keeps recent representations in a queue produced by a slowly updated copy of the encoder (the momentum encoder), which decouples the number of negatives from batch size. Masked autoencoders split an image into small square patches, hide most of them, and train the model to reconstruct what is hidden; because the encoder sees only the visible patches, it spends far less memory and compute per image. Autoregressive pretraining, by contrast, scales with tokens. For data selection, the common point is that pretraining turns raw structure into reusable signal that multiplies the value of scarce labels.

Self-supervised learning addresses the label bottleneck by learning from data structure rather than human annotation, yet it cannot solve data scarcity itself. Rare classes may have too few examples, edge cases may never appear in the wild, and privacy constraints may prevent collecting real samples. Synthetic generation addresses this gap by creating new data on demand rather than selecting or curating existing data.

[22] **Foundation model**: The name emphasizes that these models serve as a shared base for many downstream tasks, but this creates a single point of failure. Defects in the foundation model's pretraining data (biases, factual errors, memorized private content) can propagate to multiple applications built upon it, depending on transfer and downstream adaptation. From a systems perspective, this homogenization risk means that data selection quality during pretraining has an outsized blast radius: a curation error that would affect one task in the train-from-scratch paradigm can affect many downstream deployments.



One pretraining corpus defect can affect many downstream tasks.

9.10 Synthetic Data Generation

Real-world data collection encounters physical and legal boundaries: autonomous fleets cannot safely crash vehicles to acquire catastrophic collision trajectories, healthcare systems cannot transmit protected patient telemetry across institutional firewalls, and edge products must handle acoustic environments before any deployed fleet exists to record them. When target distributions exhibit extreme scarcity or zero empirical coverage, neither static filtering nor dynamic sample prioritization can extract signal that does not exist in the source corpus. Synthetic data generation, which section 4.4.2 introduced as one acquisition channel among several, becomes here a deliberate lever for manufacturing the training distribution directly. The engineering strategy shifts from *curation* (filtering and weighting existing records) to *creation*, generating novel inputs, domain perturbations, or supervisory targets to satisfy statistical coverage constraints.

9.10.1 Data augmentation: Transformation-based synthesis

Data augmentation is the lowest-overhead form of synthetic generation: it expands the effective training distribution by transforming existing samples dynamically during ingestion. Because label-preserving transformations generate novel inputs without expanding durable storage, augmentation multiplies dataset diversity with zero additional data acquisition cost. In an accelerator-based training pipeline, however, augmentation shifts the computational bottleneck between the host CPU and the accelerator. When executed on host CPU worker threads, intensive decoding and spatial transformations can saturate host cores, starving the accelerator across the PCIe bus. Offloading transformations directly to accelerator kernels can relieve CPU preprocessing pressure, while inputs still cross the host-to-device link. Accelerator-side transformation in turn consumes HBM and compute cycles that would otherwise perform gradient updates.

For image data, transformation pipelines encode task-valid invariances required at deployment (Shorten and Khoshgoftaar 2019). Geometric operations (rotation, horizontal flipping, random cropping, and scaling) encourage viewpoint invariance when they preserve the target label, while

photometric adjustments (brightness, contrast, and color jitter) simulate sensor noise and varying illumination. More aggressive strategies alter input topology directly: Cutout[23] applies random rectangular zero-masks to prevent reliance on localized features; MixUp (H. Zhang et al. 2018) constructs virtual samples by taking linear combinations of image pairs and their one-hot labels ($\tilde{x} = \lambda_{\text{mix}} x_i + (1-\lambda_{\text{mix}}) x_j$, with mixing weight $\lambda_{\text{mix}} \in [0,1]$); and CutMix[24] pastes rectangular patches from one image into another while blending labels proportionally to the patch area. Because CutMix and MixUp operate via tensor linear combinations and block memory copies directly in device memory, they provide strong data-space regularization with negligible FLOP overhead.

Text augmentation presents different constraints because discrete token vocabularies lack smooth geometric manifolds. Back-translation[25] provides semantic paraphrasing for sequence-to-sequence tasks by routing target-language monolingual corpora through an intermediate model to generate synthetic source pairings. Simpler heuristic transforms, such as synonym substitution via WordNet, random token insertion, or deletion, introduce local perturbations, but risk corrupting semantic labels if applied without task-specific validation.

Rather than hand-tuning transformation schedules, automated policy searches optimize augmentation combinations directly. AutoAugment[26] uses reinforcement learning across discrete operation spaces, but incurs extreme compute overhead. RandAugment[27] eliminates this search cost by parameterizing the policy with just two values: the number of transformations applied to each image and a single global distortion magnitude.



Lighthouse 9.2: MobileNetV2 and augmentation strength

The MobileNetV2 lighthouse model exemplifies how data augmentation complements a resource-constrained architecture. Its depthwise separable convolutions replace standard spatial convolutions with depthwise spatial filters followed by 1×1 pointwise projections, substantially reducing parameter count and arithmetic intensity.

Compact architectures have limited capacity, so they typically benefit from milder augmentation than large models; excessive distortion can exceed what a compact network can fit and cause underfitting on the underlying task distribution. Augmentation strength is therefore a capacity-matched hyperparameter, and RandAugment's single magnitude knob makes it cheap to tune per model size. Whatever strength is chosen, augmentation adds nothing to inference parameter memory or latency.

9.10.2 Generative synthesis: Creating new samples

Augmentation perturbs existing training points within their local neighborhoods. Generative synthesis constructs entirely new coordinate tuples (x, y) by sampling from parametric generative models or physical simulation engines. The systems trade-off across generative paradigms spans a cost-fidelity spectrum:

1. Generative Adversarial Networks (GANs) train a generator against a discriminator. Once trained, feed-forward generator inference requires a single forward pass, providing low latency and high sample throughput. However, GAN training is vulnerable to mode collapse, which suppresses the tail variety required for robust edge-case coverage.
2. Diffusion models formulate synthesis as iterative denoising across a parameterized Markov chain or continuous score-based process. Latent diffusion systems such as Stable Diffusion[28] enable precise semantic conditioning from text prompts while operating in a compressed latent space. A sampling recipe with 20 to 50 sequential denoising steps runs the backbone repeatedly for each generated batch. Its arithmetic and memory traffic can make offline generation preferable; whether on-the-fly generation is feasible depends on the sampler, generator, and available resources.
3. Physical simulation engines (such as CARLA for autonomous driving or Isaac Sim and Unity for robotics) decouple data generation from neural inference entirely, executing rigid-body physics, kinematics, and graphics rendering pipelines. Simulators produce exact ground-truth annotations (including 3D bounding boxes, depth maps, and velocity vectors) from

23 **Cutout**: Randomly masks square regions of input images during training, forcing the model to recognize objects from partial information rather than relying on any single discriminative region (DeVries and Taylor 2017). Unlike dropout (which zeroes neurons in feature space), Cutout operates in input space. The original Cutout experiments produced 0.3–2.0 percentage-point gains across CIFAR-10/100 and SVHN with negligible compute overhead, making it a high-ICR augmentation technique when occlusion-style invariance matches the task: more information per sample at near-zero additional cost to the pipeline.

24 **CutMix**: Replaces Cutout's zeroed-out region with a patch from a different training image, mixing labels proportionally to patch area (30 percent of image A replaced by image B yields a 70/30 label split) (Yun et al. 2019). This addresses Cutout's weakness: zeroed regions waste pixel information that could carry learning signal. The CutMix paper reports ImageNet top-1 improvements of 2.28 percentage points for ResNet-50 and 1.70 percentage points for ResNet-101, while also improving localization behavior. The method provides stronger regularization than either Cutout or MixUp alone with little additional data-pipeline cost.

25 **Back-translation**: In the method described by Sennrich et al. (2016), target-language monolingual text is translated into the source language, and the synthetic source is paired with the unchanged target for machine-translation training. The systems trade-off is latency because each synthetic source requires translation-model inference. Pipelines can precompute these examples offline rather than generating them in the data loader.

internal state without human labeling cost. The systems bottleneck shifts to host CPU physics computation and GPU graphics rasterization or ray tracing throughput.

Regardless of the generative engine, synthetic samples inevitably diverge from the statistical distributions of real deployment environments. This divergence defines the domain gap.

9.10.3 Bridging the domain gap

The domain gap[29] represents the statistical divergence between synthetic training distributions and production deployment data. When models optimize purely on synthetic distributions, they can learn simulator or generator artifacts that fail to transfer. The schematic in figure 9.9 sketches shifted feature densities and possible boundary misalignment. The marginal densities alone do not determine either decision boundary; class-conditional structure and real-data evaluation are needed to establish how the shift affects classification.

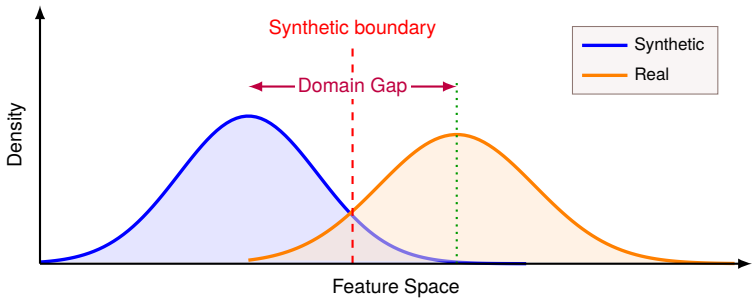


Figure 9.9: The Domain Gap Problem: Synthetic data (blue) and real data (orange) occupy shifted feature distributions. The dashed red synthetic boundary and dotted green real-data boundary illustrate possible misalignment; marginal feature densities alone do not determine class boundaries.

Two complementary strategies address this distribution mismatch. Domain randomization[30] takes an aggressive approach: rather than trying to match the real world precisely, it trains on varied synthetic data by randomizing lighting, textures, backgrounds, and camera parameters during generation.

The aim is to make deployment conditions fall within the variation seen during training, but coverage is never guaranteed by randomization alone. The method can be effective in robotics and autonomous-driving settings when the simulator spans the relevant physical and visual factors and the result is validated on real data.

Domain adaptation takes the opposite approach by explicitly aligning synthetic and real distributions. Feature alignment methods train on synthetic data while simultaneously minimizing the distance between synthetic and real feature distributions, often using adversarial training to learn domain-invariant representations. Fine-tuning offers a simpler path: pretrain on abundant synthetic data to learn general features, then fine-tune on a small real dataset to adapt to deployment conditions. Self-training combines these ideas by using a synthetic-trained model to pseudo-label real unlabeled data, then retraining on the combined labeled set.

In practice, mixing synthetic and real data provides a real-data anchor while retaining some synthetic coverage. Moving the mix from all-synthetic toward all-real trades a larger domain gap for a higher collection cost, and the best ratio depends on the domain, simulator fidelity, model family, and validation protocol rather than on any universal recipe.

Example 9.5: KWS data selection

Scenario: An embedded keyword spotting (KWS) system targets deployment on a 256 KB SRAM microcontroller with a 10,000+ audio sample requirement.

Diagnosis: Manually recording 10,000 real utterances across diverse speakers and acoustic environments costs $20K–50K. Staging a 5-tier pipeline (seed recordings → augmentation →

26 **AutoAugment**: Treats augmentation policy design as a reinforcement learning search problem: a controller selects operations (rotate, translate, shear, equalize), their magnitudes, and application probabilities to maximize validation accuracy (Cubuk et al. 2019). The original ImageNet search used approximately 15,000 GPU-hours, motivating later methods that reduce the search space.

27 **RandAugment**: Collapses AutoAugment's policy search to two hyperparameters, a transformation count and a shared magnitude (Cubuk et al. 2020). That small search space matches or exceeds AutoAugment on the reported benchmarks at far lower search cost, making it practical when 15,000 GPU-hours of policy search cannot be justified.

28 **Stable Diffusion**: Performs iterative denoising in a compressed latent space, enabling targeted text-to-image examples (Rombach et al. 2022). Its iterative generation is substantially more compute-intensive than geometric transforms, so it fits cases where novelty matters more than per-sample throughput.

29 **Domain gap**: The statistical divergence between synthetic and real data distributions, measurable with metrics such as maximum mean discrepancy (MMD) or Frechet Inception Distance (FID). For ML systems, the gap becomes training-serving skew: a simulator-trained model can validate well on synthetic data while failing silently on real deployment data.

30 **Domain randomization**: Makes synthetic data deliberately varied by randomizing textures, colors, lighting, and physical properties. The goal is not photorealism; it is enough variation to improve coverage of deployment conditions, shifting the cost bottleneck from rendering fidelity to coverage.

noise injection → hard-negative mining → text-to-speech simulation) builds dataset scale from 500 seed samples.

Systems lesson: Layered data strategies can lower acquisition costs for resource-constrained edge ML deployments. In this scenario, augmentation and noise injection reduce physical collection cost to 5 percent of the full-recording estimate; target accuracy and acoustic coverage still require validation.

The keyword-spotting scenario shows how augmentation, noise injection, and simulation can expand a small seed set, but the same creation strategy becomes riskier when samples come from learned generators. In recursive training, *model collapse*[31] can amplify errors and reduce diversity over generations. Synthetic data therefore requires provenance, mixture controls, and validation against real deployment data rather than an unverified assumption that more generated samples help.

9.10.4 Knowledge distillation: Compressing information

While data augmentation (section 9.10.1) and generative synthesis (section 9.10.2) synthesize new input samples, knowledge distillation[32] (Hinton et al. 2015; Gou et al. 2021) synthesizes supervisory targets. Rather than training exclusively on sparse ground-truth labels (such as a one-hot vector $[1, 0, 0]$), a compact student network trains against the continuous probability distribution generated by a larger teacher model (such as $[0.70, 0.20, 0.10]$). The soft target distribution, scaled by a temperature parameter T_{distill}, exposes inter-class correlations (the dark knowledge) that reflect similarities learned in the teacher's representation space. Section 10.3.2.1 develops the loss and temperature mechanics; here the concern is distillation as a *data* source and what storing or serving those targets costs.

Distillation exposes a fundamental systems trade-off between accelerator HBM capacity and secondary storage I/O bandwidth:

- **Online distillation**: The teacher model resides directly in accelerator memory alongside the student model during training. The system computes teacher forward passes concurrently with student execution, eliminating persistent storage overhead for soft targets. However, co-locating an overparameterized teacher consumes accelerator memory and arithmetic throughput, restricting the student's maximum per-accelerator batch size and increasing step latency.
- **Offline distillation**: The teacher generates predictions across the corpus during an offline precomputation pass, storing dense logit vectors to disk. This frees accelerator memory entirely for student training, but dramatically expands dataset storage volume and input pipeline bandwidth. For a classification vocabulary of K classes, storing uncompressed float32 logits increases the per-sample label footprint from a single integer index ($\lceil \log_2 K \rceil$ bits) to $4K$ bytes, an expansion of more than three orders of magnitude once K reaches the thousands.

Together, transformation-based augmentation (section 9.10.1), generative synthesis (section 9.10.2), and supervisory distillation provide complementary ways to generate training signals. Synthetic generation manufactures missing signals when empirical collection hits physical, legal, or economic constraints. Orchestrating these techniques across the data, algorithm, and machine dimensions requires balancing data storage footprint, pipeline preprocessing throughput, and accelerator utilization within a unified systems budget.

9.11 Selection Engineering

A naive active learning loop that scans the entire dataset every epoch to select the "best" samples can turn a compute-bound training job into an I/O-bound bottleneck. **Selection engineering** begins where the algorithms of the preceding sections end: once a technique is chosen, engineering must ensure that it delivers its promised speedup on real hardware and real data pipelines. Translating algorithmic filtering into realized throughput requires architectural patterns that bound scoring latency, preserve sequential storage access, and prevent accelerator starvation.

[31] **Model collapse**: Formally analyzed by Shumailov et al. (2024), this phenomenon can arise when errors and missing tail coverage compound during recursive training. Tail distributions contain rare but important patterns that appear infrequently in training data. When generation $n+1$ trains on output from generation n under these conditions, successive generations can further compress the tails, producing increasingly homogeneous data. The degradation can be rapid in recursive training settings, which is why training on unanchored synthetic outputs risks catastrophic collapse of tail diversity.

[32] **Knowledge distillation**: Soft teacher probabilities reveal structural similarities across classes that one-hot encodings omit. The temperature parameter scales logit smoothing ($T_{\text{distill}} > 1$); excessive smoothing erases discriminative signal, while $T_{\text{distill}} \to 0$ collapses toward the argmax (one-hot) prediction and $T_{\text{distill}} = 1$ recovers the ordinary softmax. Distillation introduces teacher inference costs that must be justified by improved student sample efficiency or accuracy.

9.11.1 The selection bottleneck

Dynamic data selection introduces a new bottleneck: selection latency. A conventional loader follows a predetermined sampling plan, whereas active learning or adaptive curricula may evaluate a selection function $f(x)$ over a large candidate pool before forming later batches. Concretely, scoring a 1M dataset with a large model can take 2.8 hours, potentially negating the savings from a 10 percent coreset if not performed with a smaller proxy model. The systems trade-off requires that selection cost remain below the training work the subset saves; otherwise, sample counts fall while end-to-end cost rises.

To reduce overall training time at a fixed quality target, selection plus subset training must take less time than training on the full corpus. Let $T_{\text{selection}}$ be total wall-clock selection overhead, including candidate scoring and any prerequisite scorer preparation charged to the training run. Let $T_{\text{train}}(D)$ be wall-clock training time for a dataset of size D, and let D_{subset} and D_{total} be the retained and full sample counts. Equation 9.2 formalizes this break-even condition as the **selection inequality**:

$$T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}}) \tag{9.2}$$

Equivalently, isolating the allowable selection overhead yields $T_{\text{selection}} < T_{\text{train}}(D_{\text{total}}) - T_{\text{train}}(D_{\text{subset}})$. The upper bound on selection latency depends strictly on how much training execution the subset eliminates; no arbitrary overhead fraction applies across workloads. When an engineering pipeline can amortize scorer preparation or checkpoint extraction across multiple downstream training runs (section 9.13.4), $T_{\text{selection}}$ reflects only the marginal scoring pass over the candidate pool. Conversely, if an auxiliary proxy must be trained from scratch for a single isolated run, that preparatory compute adds directly to $T_{\text{selection}}$. When $f(x)$ repeatedly evaluates the candidate pool using the primary model architecture, recurring scoring passes and distributed coordination can consume the saved training cycles, turning data selection into a net throughput penalty.

Example 9.6: Selection inequality in practice

Scenario: A computer vision team selects a 100K coreset (10 percent) from 1M images to reduce training time on an accelerator cluster. Pretrained checkpoints for both the target architecture and a lightweight proxy are already available from earlier exploration, so this comparison prices a marginal scoring pass over the candidate pool.

Diagnosis: Against a 277.8 h full-data run of 100 epochs, marginal full-model scoring (Option A: target ResNet-50) adds 2.8 h of selection overhead and still saves 247.2 h (89 percent). Marginal proxy-model scoring (Option B: ResNet-18) drops selection overhead to 0.6 h and saves 249.4 h (89.8 percent), only 2.2 h more.

Systems lesson: Both options satisfy the selection inequality by a wide margin, because a single scoring pass costs at most about one forward pass over the dataset (roughly one-third of a full-data epoch for the target ResNet-50) and far less with the proxy, while training on the 10 percent coreset shrinks every one of the 100 epochs. This calculation assumes that scorer preparation carries zero marginal cost. If training the ResNet-18 proxy from scratch were charged entirely to this single run rather than amortized across a multi-run campaign (section 9.13.4), that preparatory training would erase Option B's 2.2 h advantage. Scorer choice also becomes decisive when selection rescores dynamically every epoch or when the scoring architecture is significantly larger than the target, multiplying selection latency while subset training savings remain fixed.

End-to-end wall-clock time divides into selection overhead and subset training. The stacked bars in figure 9.10 chart this trade-off across three regimes: lightweight proxy scoring limits selection overhead to 5 percent while training on the 40 percent subset (yielding 55 percent net savings), whereas heavyweight scoring expends 60 percent overhead on selection, wiping out all subset-training efficiency gains.

This overhead compounds when selection runs dynamically across epochs rather than once offline. If per-epoch rescoring approaches the training work saved by the pruned subset, the net systems advantage vanishes.

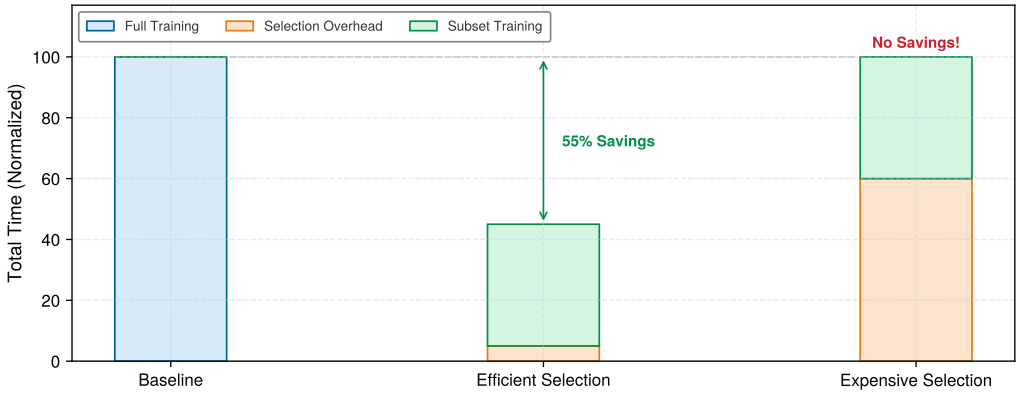


Figure 9.10: The Selection Inequality: Data selection only improves end-to-end training time at matched quality if the overhead of selection plus training on the subset is less than training on the full dataset. A lightweight selection function (proxy model, cached embeddings) keeps selection overhead low; an expensive selection function (full model forward pass) can negate the savings.

Engineering pipelines manage selection overhead through proxy models, cached embeddings, and indexed retrieval. A smaller proxy model curtails scoring latency provided its sample ranking transfers accurately to the target architecture. Running the scoring pass in reduced numerical precision can improve arithmetic throughput and reduce memory traffic when ranking fidelity permits. For similarity-based or diversity-based selection, vector indices such as FAISS[33] replace expensive corpus-wide scans with approximate nearest-neighbor retrieval over precomputed embeddings. These techniques decouple selection evaluation from primary model training, reducing $T_{\text{selection}}$; measured costs and retained quality determine whether the inequality holds.

[33] **FAISS (Facebook AI Similarity Search)**: Provides GPU-accelerated similarity search using exact, approximate, and compressed-domain index designs (Johnson et al. 2019); Johnson, Douze, and Jegou report billion-vector graph construction on multiple GPUs, showing why vector-index infrastructure matters for web-scale selection. For data selection pipelines, FAISS-style indexing supports k-nearest-neighbor retrieval for coreset selection, embedding-based deduplication, and stratified clustering for balanced sampling. Without this infrastructure, embedding-based selection would often fall back to expensive full-corpus scans.

9.11.2 Storage access: The random-access penalty

The selection inequality addresses compute overhead, but selection algorithms also reshape physical I/O patterns. Training already pays for shuffling through the access factor F_{access} that scales effective storage throughput in the data-pipeline model of section 8.5.2.1. Selection pushes that factor lower by design, because a retained subset is scattered across the corpus rather than stored contiguously. Index-driven sampling turns large sequential shard transfers into scattered, nonsequential reads, forfeiting the throughput benefits of hardware readahead, OS page caching, and bulk controller prefetching. Because disk heads must physically seek and flash controllers must execute random block lookups, equal-sized subsets can produce starkly different wall-clock training times depending on access locality. Table 9.7 quantifies this small-read penalty across storage tiers for 4 KB transfers.

Table 9.7: Illustrative Small-Read Penalty: Registry device anchors compare sequential bandwidth with random 4 KB reads under a simplified IOPS conversion. Real throughput depends on request size, queue depth, concurrency, caching, and service configuration; the cloud row is only a qualitative scenario.

Storage Tier	Sequential Throughput	Random I/O (IOPS)	Random Throughput (approx)	Random Penalty
HDD (7.2k)	~150 MB/s	~100 IOPS	~0.4 MB/s	375×
SATA SSD	~550 MB/s	~10K IOPS	~40 MB/s	13.8×
NVMe SSD	~3,500 MB/s	~500K IOPS	~2,000 MB/s	1.75×
Cloud (S3)	Configuration-dependent	Not IOPS-comparable	Configuration-dependent	Potentially extreme

Data loaders resolve this conflict through data-machine co-design. Sharded dataset formats, such as WebDataset and FFCV, pack thousands of individual examples into contiguous, sequentially readable archive chunks. Rather than fetching scattered individual samples from disk, the loader streams large sequential shards into host memory and populates a **shuffle buffer**. The training loop

samples pseudo-randomly from this in-memory buffer, preserving peak storage read bandwidth while satisfying the statistical requirement of randomized mini-batches.

Converting a logical set of selected indices into sequential physical reads requires choosing between two storage strategies. For static pruning (such as a fixed coreset or a deduplicated corpus), the pipeline can materialize the subset offline by repacking the retained records into new, contiguous shards. This step incurs a one-time sequential read and write cost alongside additional disk storage, but ensures that every subsequent training epoch transfers only retained bytes from storage. Conversely, when the selection policy updates dynamically across training iterations or active-learning rounds, repeatedly repacking multi-terabyte shards on disk would choke the filesystem on write bandwidth. Data loaders instead stream the original, full-volume shards sequentially into host memory and discard unselected records before populating the shuffle buffer. This online filtering preserves sequential read throughput and eliminates seek penalties, but it leaves physical storage traffic unchanged: the host continues to read 100 percent of the corpus bytes from disk even when accelerators train on only a 10 percent subset. Systems engineers must price this ongoing read traffic on storage-bound nodes and include offline materialization costs when evaluating the selection inequality.

In distributed training across multiple accelerators, each worker maintains an independent shuffle buffer over a non-overlapping shard. Because randomization is local to each worker's memory buffer rather than global across the corpus, rare classes or boundary cases concentrated in specific shards receive uneven exposure across workers unless the dataset is stratified and shards are balanced prior to distribution.

Checkpoint 9.2: The selection inequality

Data selection is not free. It introduces a new term to the iron law and a new I/O cost.

Equation checks

- ☐ **Selection cost**: Explain why $T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}})$ must be strictly less than $T_{\text{train}}(D_{\text{total}})$ for the technique to reduce end-to-end training time.
- ☐ **Overhead management**: Explain how proxy models and cached embeddings keep $T_{\text{selection}}$ low.

Systems implications

- ☐ **I/O patterns**: Explain why nonsequential access for dynamic selection reduces data-loader throughput relative to sequential reads.

9.11.3 Data echoing: Amortizing I/O costs

Even when storage bandwidth is preserved, dynamic selection pipelines often shift the system constraint upstream to host CPU computation. Heavy data transformations, such as 3D rotations, MixUp, JPEG decoding, or on-the-fly generative synthesis, demand substantial host processing per sample. When the host pipeline produces batches slower than the accelerator executes forward and backward passes, the accelerator idles, starved for input; this is the feeding problem reappearing inside the selection loop. The resulting collapse in accelerator utilization extends training wall-clock time, neutralizing the efficiency gains achieved by pruning the dataset.

Data echoing[34] (Choi et al. 2019) reuses data multiple times before fetching new samples, trading freshness for accelerator utilization when the input pipeline is slower than training. Echoing before randomized augmentation can produce different transformed inputs on each repetition, while echoing later in the pipeline may repeat identical tensors.

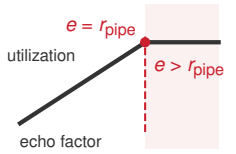
The optimal echo factor depends on the ratio r_{pipe} of upstream processing time to downstream training time, where T_{prep} is the time the data pipeline needs to produce a batch and T_{GPU} is the time the accelerator needs to train on it:

$$r_{\text{pipe}} = \frac{T_{\text{prep}}}{T_{\text{GPU}}}$$

↩ Accelerator bubbles, which open when the data-loading stage cannot keep pace with the accelerator, are established in section 8.2.

[34] **Data echoing**: The key subtlety is *where* in the pipeline to insert the echo point. Echoing before augmentation (upstream echoing) can apply different random augmentations to each repetition, while echoing after augmentation feeds identical tensors to the accelerator. Useful echo factors and realized speedups depend on the workload, batch size, insertion point, and shuffling policy (Choi et al. 2019).

In this two-stage rate model, $r_{\text{pipe}} > 1$ means the data pipeline bottlenecks training. An echo factor $e < r_{\text{pipe}}$ partially recovers idle accelerator cycles, while $e \geq r_{\text{pipe}}$ removes input waiting entirely, provided repeated samples remain statistically productive. Raising e beyond r_{pipe} adds no throughput and only reduces gradient diversity. When $r_{\text{pipe}} < 1$, the input pipeline already outruns the accelerator, so echoing has nothing to recover.



Echoing raises accelerator utilization until e reaches r_{pipe}; beyond it, nothing is gained.

Napkin Math 9.4: Worked example: Data echoing ROI

Scenario: Training ResNet-50 on ImageNet with heavy augmentation (RandAugment + MixUp).

Measurements:

- Data pipeline throughput: 300 images/s (reading, decoding, augmenting on CPU)
- GPU training throughput: 800 images/s (forward + backward pass)
- Ratio $r_{\text{pipe}} = T_{\text{prep}}/T_{\text{GPU}}$ = (1/300 images/s) / (1/800 images/s) = 800 images/s/300 images/s ≈ 2.67 (GPU waiting 62.5 percent of time)

Without echoing:

- Effective throughput: 300 images/s (limited by data pipeline)
- Training time for 90 epochs: $90 \times 1.28\text{M}$ / 300 images/s = 384,350 seconds (106.8 hours)
- GPU utilization: 37.5 percent

With echo factor: $e = 2$, inserted after the measured reading, decoding, and augmentation stage.

- Each already-augmented batch is processed twice, amortizing the measured upstream work; the repeated pass introduces no fresh augmentation
- Effective throughput: 600 images/s (still below GPU capacity)
- Unique images per second: 300 images/s (unchanged)
- Training time: $90 \times 1.28\text{M}$ / 600 images/s = 192,175 seconds (53.4 hours) if echoed data is equally valuable

Trade-off: Repeated samples are not guaranteed to be as useful as fresh samples; the data echoing paper evaluates echo factor, insertion point, and shuffling because those choices determine whether reuse preserves predictive performance. In one network-fed ResNet-50/ImageNet configuration, Choi et al. (2019) reports a 3.25× reduction in wall-clock time to the target metric.

Systems insight: Data echoing can trade sample diversity for accelerator utilization when the input pipeline is the bottleneck. The useful echo factor must be measured for the workload because it depends on the batch size, insertion point, augmentation, shuffling, and target metric.

Echoing can introduce correlations when the same example appears within or across nearby batches, so implementations must preserve adequate shuffling and validate the target metric. Choi et al. (2019) did not observe a negative batch-normalization interaction in their ResNet-50 or Single Shot MultiBox Detector (SSD) experiments, but that result does not guarantee the same behavior for every workload.

These implementation patterns determine whether data selection yields net systems throughput. Proxy selection curtails probe compute when scoring rankings transfer to target architectures. Sharded file formats and shuffle buffers reconcile pseudo-random batch creation with hardware-level sequential streaming, while data echoing recovers stalled accelerator cycles when ingestion throughput limits the training loop.

These patterns leave a coordination question when several accelerator workers share one host. Each worker scores or trains on its own shard, but the selected indices and scoring checkpoint must

still describe one coherent dataset. Shared storage and host memory do not make each worker's local view representative of the corpus.

9.12 Distributed Selection

Data sharding dissolves the single-process abstraction even on one machine with two to eight accelerators. In a centralized pipeline, coreset algorithms sort a single global ranking, curricula enforce a uniform sequence, and active-learning loops evaluate a shared candidate pool. Parallel worker processes divide the corpus into local shards and may score them using different model checkpoints. Selection must reconstruct a representative global subset from these local views and keep rankings coherent as the model changes. The workers still share host memory capacity, storage bandwidth, and host-to-device links.

Whether distributed selection stays faithful to the global dataset or collapses into local shard heuristics depends on how much cross-worker coordination each technique requires against the bandwidth available to sustain it. On one host, score gathering and index dispatch compete for CPU time and memory bandwidth with decoding and batch preparation. Evaluating selection therefore requires analyzing statistical fidelity alongside these shared resources.

These are independent requirements. A coreset can remain representative yet cost too much to coordinate, while a cheap shard-local method can scale but systematically miss rare groups visible only in the global corpus. A distributed design therefore needs two acceptance tests: compare selected-data quality with a centralized or otherwise auditable reference, and compare selection overhead with the end-to-end training time it saves. Passing only one test is insufficient.

9.12.1 Strategies for distributed selection

In a basic data-parallel layout, each worker processes a distinct shard and model synchronization is handled separately. Data selection can add dependencies across those shards (table 9.8).

Table 9.8: Selection Dependencies in Distributed Training: The table starts from a common centralized formulation of each technique and identifies the coordination problem introduced by sharding. Distributed variants may relax rather than reproduce the centralized result exactly.

Technique	Single-Process Assumption	Distributed Challenge
Coreset Selection	Global view of dataset	Each worker sees only its shard
Active Learning	Centralized uncertainty scoring	Scoring requires model synchronization
Curriculum Learning	Global difficulty ordering	Workers may have different "hardest" samples
Deduplication	One shared duplicate map	Shard-local hashes miss cross-shard duplicates

The selection dependencies admit several architectural solutions, each navigating a different point in the consistency-scalability trade-off space. The most straightforward approach centralizes selection while distributing training. A coordinator process performs selection on the full dataset, then distributes selected indices to workers. This preserves selection quality but introduces a single bottleneck:

```
Coordinator: score_all_samples() → selected_indices
Broadcast: selected_indices → all workers
Workers: train on subset(local_shard, selected_indices)
```

The semantics remain clean, but the coordinator can serialize scoring or index dispatch. Whether that overhead is acceptable depends on the selection payload, refresh rate, worker count, and copies through shared host memory. Adding accelerator workers does not increase the host memory available to the coordinator.

Hierarchical selection addresses this scalability limitation by distributing the selection computation itself. Each worker performs local selection on its shard, then a coordinator merges results:

```
Workers: local_selected = select_top_k(local_shard)
Coordinator: global_selected = merge_and_rerank(all local_selected)
Broadcast: final_indices → all workers
```

Shard-local selection reduces coordinator load but introduces a quality trade-off: local quotas and score distributions may not preserve the global ranking or rare groups spread unevenly across shards. The merge step therefore needs normalization, coverage constraints, or a second global pass.

Cross-shard deduplication has the same visibility requirement. Workers can compute MinHash signatures locally, but a shared merge must compare candidates across shard boundaries. Independent local hash tables cannot detect a duplicate whose counterpart belongs to another worker. The merge must fit the host's memory budget or operate in bounded batches, rather than assuming that more workers supply more host capacity.

9.12.2 Consistency challenges in active learning

Shard-local selection already needs a coherent score scale, but active learning introduces an additional complication because the model changes during selection. Consider what happens when Worker A scores samples using the model at step t while Worker B simultaneously updates the model to step $t+1$. Worker A's scores are now stale and may select samples that the updated model would rank differently.

The scoring schedule therefore defines the meaning of a query, not merely its cost. A reproducible system versions the model checkpoint, candidate-pool snapshot, scoring rule, and selected indices together. It can then measure how much ranking agreement decays with checkpoint age and choose a refresh interval from that evidence rather than from an arbitrary number of steps.

Several strategies mitigate this staleness problem, each with distinct overhead characteristics:

- **Synchronous scoring**: All workers pause training and score simultaneously, guaranteeing consistency but at substantial cost in GPU utilization.
- **Periodic score refresh**: Workers re-score every k epochs rather than every batch, trading freshness for reduced overhead.
- **Checkpoint-robust selection**: The system selects samples that exhibit high uncertainty under multiple model checkpoints, favoring decisions that are less sensitive to checkpoint choice. Agreement across past checkpoints is a robustness heuristic; ranking quality still needs validation as the model evolves.

An illustrative one-host scenario with eight accelerator workers prices the shard-local scoring and centralized merge of a one-time coreset selection. Repeated refreshes incur this cost again; the assumed phase times include parallel execution and host coordination rather than promising linear speedup from the worker count.

Example 9.7: Distributed coreset selection

Scenario: One host with 8 A100 accelerator workers selects a 10 percent coreset from ImageNet (1.3M images) across local worker processes.

Diagnosis: Scoring on one accelerator leaves the other devices unused. Shard-local embedding, parallel signature generation with cross-shard deduplication, local EL2N scoring, and a centralized candidate merge distribute that work. Under the assumed phase times, dataset selection takes 67 minutes.

Systems lesson: Parallel scoring reduces work per accelerator, while embeddings, score arrays, and the global index must still fit within the machine's memory budget. In figure 9.11, "Coordinator Node" names the logical selection role on this same host; the worker boxes represent accelerator processes, not additional machines. Workers return candidate scores and indices to the coordinator, which maintains the global FAISS index for embedding-based checks, merges local candidates, and dispatches the final coreset indices. The merge must enforce coverage floors and compare its result with the centralized reference, because candidates discarded locally cannot be recovered from their scores alone.

Positive ROI can erode quickly when workers coordinate frequently during training. Distributed selection introduces a coordination tax whose size depends on what is synchronized and how often. That tax must remain smaller than the training time saved; if it approaches the limit, simplify the strategy or increase the refresh interval.

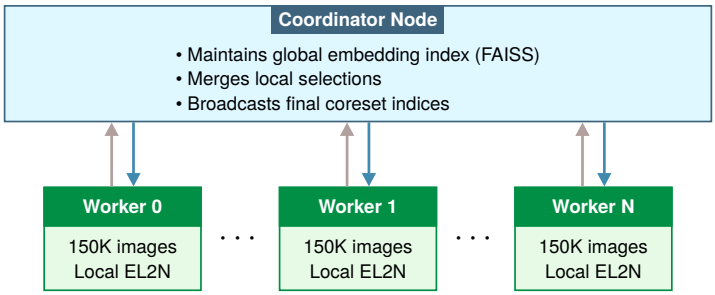


Figure 9.11: Distributed Coreset Selection Architecture: Parallel shard-local scoring reduces the work assigned to a centralized rank-merging coordinator. Workers send local EL2N scores upward; the coordinator merges them and broadcasts final coreset indices. The design is beneficial only when this coordination overhead remains below the training time saved by the coreset.

The communication path adds a further constraint. Gradient synchronization may use accelerator collectives over local interconnects, while embedding vectors, score arrays, and coreset indices may pass through host memory and CPU dispatch. Selection can therefore expose host-memory, PCIe, storage, or process-synchronization bottlenecks that the training profile alone does not reveal. Extending the design to independent machines adds network topology and remote-storage constraints; those are a further scaling problem beyond the one-host design developed here.

Yet operational feasibility does not guarantee economic viability. A deduplication pipeline that costs $50K to engineer and saves $10K per training run loses money on a single run and pays off only if enough runs reuse it. Deploying selection techniques requires rigorous financial accounting of capital, compute, and human annotation budgets.

9.13 Cost Modeling

The hospital team in notebook 9.3 could not label every scan, because doing so would have cost $5,000,000, 10× over its budget. Allocating capital between human annotation and accelerator compute is a primary systems decision, and a team under a fixed budget must determine whether the next dollar buys more downstream accuracy as specialist annotations or as additional accelerator hours. Systematic cost modeling establishes the quantitative framework required to navigate these trade-offs, calculating break-even points for active learning pipelines and determining how many training runs are required to amortize custom data curation tooling.

9.13.1 Quantifying data costs and ROI

The total cost of training data spans acquisition, labeling, storage retention, and accelerator compute, extending well beyond object-storage invoices:

$$C_{\text{total}} = C_{\text{acquire}} + C_{\text{label}} + C_{\text{store}} + C_{\text{process}}$$

Labeling (C_{label}) represents the expenditure that data selection targets most directly, whereas storage (C_{store}) and processing (C_{process}) scale mechanically with retained byte volume and the number of accelerator passes. Table 9.9 itemizes these four cost components alongside their operational drivers.

Table 9.9: Total Cost of Training Data: The four cost components span the full data lifecycle. Here D is the total sample count, D_{labeled} is the labeled subset, $D_{\text{vol,store}}$ is stored byte volume, T_{months} is retention time, N_{epochs} is the number of training passes, and O_{sample} is per-sample training work. The ranges are scenario anchors rather than universal market prices.

Component	Formula	Illustrative Range
C_{acquire}	$D \times c_{\text{sample}}$	\$0.001–\$10/sample (web scrape vs. licensed)
C_{label}	$D_{\text{labeled}} \times c_{\text{label}}$	\$0.01–\$200/sample (crowd vs. expert)
C_{store}	$D_{\text{vol,store}} \times c_{\text{storage}} \times T_{\text{months}}$	\$0.02–\$0.10/GB/month
C_{process}	$D \times N_{\text{epochs}} \times O_{\text{sample}} \times c_{\text{FLOP}}$	Proportional to training FLOPs

The interplay of these four cost terms becomes concrete when evaluating an ImageNet-scale vision model training run.

Napkin Math 9.5: Cost breakdown: ImageNet-scale training

Table 9.10 itemizes one illustrative acquisition, labeling, storage, and compute scenario at ImageNet scale. The resulting ratio follows from the stated licensing, annotation, storage, runtime, and compute-cost assumptions; it is not a typical ratio for every supervised workload.

Table 9.10: Illustrative ImageNet-Scale Cost Breakdown: Itemized acquisition, labeling, storage, and compute costs under the stated scenario assumptions. Each amount follows from the rate shown beside it, using the book's published crowd-labeling, object-storage, and cloud-GPU rates. Data costs dominate this particular calculation because licensing and annotation are included while the training run itself is short.

Cost Component	Calculation	Amount
Raw data (1.2M images)	Licensed dataset, flat fee	$50,000
Labels (crowd annotation)	1.2M × $0.05/label	$60,000
Storage (cloud object store)	150 GB × $0.02/GB/month × 12 months	$36
Training campaign (30 runs, each 100 epochs in 24 h on 8 A100s)	5,760 GPU-hours × $4/GPU-hour	$23,040
Total		**$133,076**
Data vs. Compute ratio		82.7% data, 17.3% compute

Systems insight: Acquisition and labeling can dwarf compute cost in supervised vision workloads, as they do under these assumptions. The compute figure prices the whole training campaign, not one run: a published model is the survivor of sweeps, restarts, and ablations, and costing a single run would understate compute roughly thirtyfold. Even so, acquisition and labeling remain the larger share. The run count is an assumption like any other, so a complete ROI calculation must state it and price both categories rather than assuming which one dominates.

9.13.2 ROI framework for data selection techniques

Data selection techniques insert algorithmic filtering into the training pipeline to reduce downstream costs. However, evaluating sample importance is not free: scoring consumes accelerator cycles, generates host-device memory traffic, or requires engineering custom pipeline stages. Evaluating whether a selection strategy is economically justified requires balancing these upfront overheads against downstream savings through Return on Investment (ROI):

$$\text{ROI} = \frac{\text{Savings} - \text{Investment}}{\text{Investment}} \times 100\%$$

The balance between investment and savings varies across selection strategies. Table 9.11 maps the primary cost drivers and benefit mechanisms across four standard techniques; their net return depends on corpus volume, model scale, and existing pipeline infrastructure.

Table 9.11: ROI Profiles for Data Selection Techniques: The table identifies costs and possible savings. No technique guarantees positive ROI. A training-time claim must satisfy the selection inequality; labeling savings or reusable data investments require their corresponding total-cost comparison at target quality.

Technique	Investment (Cost)	Savings (Benefit)
Deduplication	One-time compute for hashing + infrastructure	Reduced storage and repeated-sample processing
Coreset Selection	Proxy model training + selection compute	Fewer retained samples if target quality and coverage hold
Active Learning	Inference on unlabeled pool + human-in-the-loop latency	Lower labeling demand when query selection transfers
Data Augmentation	CPU/GPU cycles for transforms	Effective dataset size increase without new data acquisition

The profiles in table 9.11 demonstrate that data savings never come for free. Deduplication requires an upfront investment in hashing or embedding infrastructure, but can pay off across

many epochs and runs by eliminating redundant storage and compute. Coreset selection demands training a proxy model to score the dataset; it only breaks even if the compute saved by dropping samples exceeds the cost of that proxy training. Active learning shifts the cost from human labelers to machine inference, generating high ROI when human time is expensive, though it introduces human-in-the-loop latency that can stall automated pipelines. Finally, data augmentation requires zero new data acquisition but shifts the burden to the CPU or GPU, trading compute cycles for effective dataset expansion. In all cases, a technique earns its place only when its specific benefit outweighs its structural investment.

9.13.3 Break-even analysis

Selection algorithms expend compute and pipeline latency to reduce downstream labeling or training work. The **break-even point** marks the operating boundary where the financial savings from omitted samples exactly equal the computational and operational overhead of selecting them. If candidate scoring is computationally intensive or candidate pools are massive, a selection policy can easily consume more resources than it recovers.

Suppose labeling costs \$10/sample, active learning starts from 1,000 labeled samples (\$10,000), issues 100 queries per round at \$50 inference cost (the expense of scoring the candidate pool U with the active model), and the random-labeling baseline requires 5,000 samples for target accuracy. If active learning reaches target accuracy with only 2,000 labeled samples, the following comparison reports net savings relative to the stated labels-plus-scoring cost. This calculation includes labels and pool scoring only, assumes both methods reach the same target accuracy, and excludes retraining, engineering, storage, and annotation-delay costs. It reports net savings per dollar of that stated active-learning budget, using a different denominator from the incremental-investment ROI of section 9.13.2:

$$\text{Random labeling cost} = 5{,}000 \times \$10/\text{sample} = \$50{,}000$$

$$\text{Active learning cost} = 2{,}000 \times \$10/\text{sample} + 10 \text{ rounds} \times \$50 = \$20{,}500$$

$$\text{ROI} = \frac{\$50{,}000 - \$20{,}500}{\$20{,}500} = 143.9\%$$

Within this labels-plus-scoring boundary, break-even occurs when avoided labeling costs equal selection overhead. A complete dollar comparison must also deduct any additional retraining, engineering, storage, and delay-related expense relative to the random-labeling baseline. When an unlabeled candidate pool U contains millions of uncurated examples, repeatedly scoring it across multiple rounds scales scoring cost with $|U| \times c_{\text{infer}}$, meaning an algorithmic reduction in labeled samples can be erased if proxy inference costs are not carefully bounded.

9.13.4 Amortization across training runs

Static data selection techniques decouple dataset curation from the model training loop. Computational filters such as MinHash deduplication, exact n-gram indexing, or embedding clustering require substantial upfront engineering and preprocessing compute. However, production model development is inherently iterative: hyperparameter search, neural architecture search, ablation studies, and seed sweeps repeatedly train on the identical curated corpus. Evaluating these techniques on a single isolated run misrepresents their economics. **Amortized return on investment (ROI)** accounts for how upfront curation investments spread across an entire training campaign:

$$\text{Amortized ROI} = \frac{N_{\text{runs}} \times \text{Per-Run Savings} - \text{One-Time Investment}}{\text{One-Time Investment}} \times 100\%$$

Table 9.12 itemizes the upfront capital and compute expenditures against per-run savings for an illustrative MinHash deduplication pipeline.

Table 9.12: Deduplication Infrastructure Cost Components: The one-time investment covers engineering effort and initial compute; the per-run savings accrue with every subsequent training run on the deduplicated data.

Component	Cost
Build deduplication pipeline	$50,000 (engineering time)
Compute MinHash signatures (one-time)	$5,000
Per-run savings	$10,000/run

Because the initial infrastructure and signature-generation costs are fixed, each subsequent training run compounds the net financial return. Three operational factors govern whether curation infrastructure reaches break-even:

- **Repeated training runs**: Hyperparameter search, model iterations, and scheduled retraining reuse the same selection infrastructure many times.
- **Shared datasets**: A cleaned or deduplicated corpus can support multiple teams or model architectures.
- **Broadly reusable techniques**: Methods such as deduplication transfer across models, whereas task-specific coresets may not.

Table 9.13 tracks how amortized return scales with campaign run count under these cost parameters.

Table 9.13: Amortized ROI over Multiple Training Runs: Under this scenario's costs and savings, a deduplication pipeline that loses money on its first use becomes profitable when reused across enough runs. Infrastructure investments should be evaluated over their expected lifetime rather than from one use.

Number of Runs	Amortized ROI
1 run	-81.8% (net loss)
5 runs	-9.1% (near break-even)
10 runs	+81.8% (positive)
50 runs	+809.1% (highly profitable)

Deduplication achieves high investment leverage because eliminating redundant sequences reduces host-storage footprint and avoids wasted gradient updates across every downstream architecture. Sample-selection methods conditioned on proxy gradients or uncertainty scores transfer within a model family or between related architectures, but their rankings must be revalidated when the target model, task, or data changes, which narrows their amortization window relative to deduplication. For exploratory or one-off training runs, low-overhead methods like uniform random sampling or basic augmentation may offer better ROI. A static scoring pass can still pay for itself in one run if its measured savings exceed its cost at target quality.

9.13.5 Amortization across tasks: Foundation models

The centrality of self-supervised learning to foundation-model workflows stems directly from cost amortization. A foundation model is a broadly pretrained reusable base model adapted to downstream tasks through fine-tuning. This architecture divides costs into two phases: an expensive upfront pretraining investment executed once on unlabeled data, and lightweight task-specific adaptations reused across many applications (table 9.14).

Table 9.14: Illustrative Foundation-Model Amortization: The label and compute ranges are scenario assumptions. They illustrate why reuse can lower marginal task cost when representations transfer, not a universal fine-tuning ratio.

Approach	Labels per Task	Compute per Task	Data Acquisition
Train from scratch	100K–1M labeled	100% full training	Task-specific collection
Fine-tune foundation model	100–1K labeled	1–5% of full training	Reuse pretraining corpus

To illustrate this economic transformation, consider a company building 10 specialized classifiers. Under the scenario assumptions, each task trained from scratch needs 100,000 labels at $1 per label, for $1M across all tasks. The modeled compute burden is 10,000 GPU-hours.

The fine-tuning scenario pays 10,000 GPU-hours once, then assigns 1,000 labels and 50 GPU-hours of compute to each task. Across all 10 tasks, the modeled fine-tuning labels cost $10K.

Assuming both approaches reach the same target quality on every downstream task, labeling cost drops by 100× and per-task marginal compute by 20×. These are consequences of the chosen scenario values, while total compute becomes favorable only after enough downstream tasks amortize pretraining.

The compute-only crossover occurs when enough downstream tasks share the pretraining cost, provided the representations transfer at the required quality. A dollar crossover also depends on labeling savings, compute prices, and other costs that differ between the approaches. The side-by-side bar charts in figure 9.12 contrast the cost structures across ten downstream tasks: training from scratch expends a constant 1,000 GPU-hours per task (totaling 10,000 GPU-hours), whereas the foundation model approach absorbs a 10,000 GPU-hour pretraining bar upfront but collapses subsequent task fine-tuning to just 50 GPU-hours per bar.

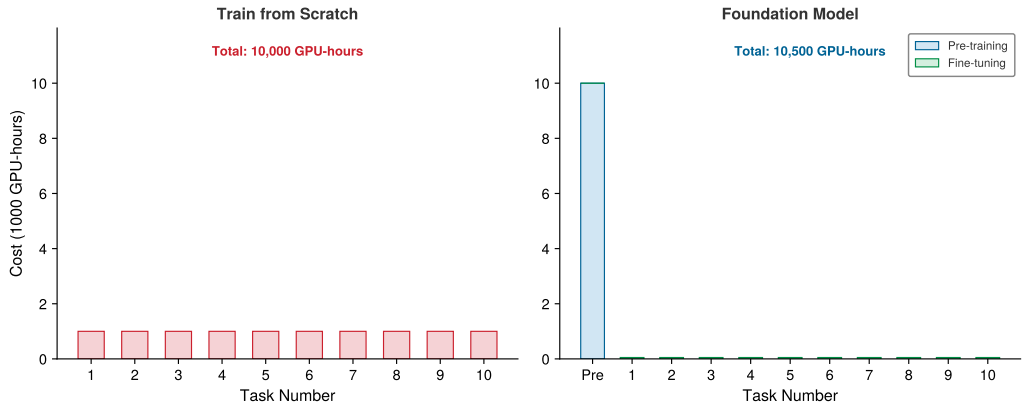


Figure 9.12: Cost Amortization in Foundation Models: Training from scratch (left) requires 1,000 GPU-hours per task (10,000 GPU-hours total for 10 tasks). The foundation model approach (right) pays 10,000 GPU-hours upfront for pretraining but reduces each subsequent task to just 50 GPU-hours. At 10 tasks the totals are comparable (10,000 GPU-hours vs. 10,500 GPU-hours), but the per-task marginal cost drops by 20×, and the compute-only crossover favoring the foundation model occurs around 10.5 tasks. The plotted 10-task snapshot therefore remains just below that break-even point.

Checkpoint 9.3: Accepting a data intervention

A smaller training pool is one observation in the acceptance decision. Connect the measurements to the constraint the intervention was meant to relieve.

- ☐ **Time savings**: Identify the scoring, coordination, and training times needed to compare a selected subset with full-data training at the same target quality.
- ☐ **Cost objective**: Explain how an active-learning pipeline could reduce total expenditure even if it takes longer to train, and identify the costs that must enter that comparison.
- ☐ **Coverage**: Decide whether unchanged aggregate accuracy is sufficient when a deployment-critical rare slice regresses, and name the additional validation evidence needed. For the disjoint-class allocation in section 9.7.4, explain why reducing K below $a+b$ requires revising the selection plan before training, even if every remaining example has a high score.

With time, labeling, and amortized costs expressed in one accounting, every technique in the chapter can now be priced against the claim it makes. A training-time saving must satisfy the

selection inequality, a labeling saving must beat the random-labeling baseline after scoring and any other differing costs, and a reusable investment must recover its one-time cost across the runs or tasks that share it. Pricing each technique separately, however, does not say which one to reach for first when several of them compete for the same budget.

9.14 Decision Framework

When labeling budget, redundancy, rare classes, privacy, and convergence speed conflict, systems engineering requires identifying the primary binding constraint. Table 9.15 collects the three levers with order-of-magnitude ranges drawn from scenarios like those worked in this chapter; each must reduce the cost it targets after counting its overhead and preserving target quality and required coverage. Self-supervised pretraining, which changes the training signal rather than the sample set, enters the decision through the labeling-cost branch of the tree that follows.

Table 9.15: Three-Stage Data Selection Pipeline: The ranges illustrate possible outcomes rather than portable expectations. Actual gains depend on the dataset, model, selection method, and target metric.

Stage	When Applied	Techniques	Order-of-Magnitude Scenario Range
Static pruning	Before training	Coreset Selection, Deduplication, Quality Filtering	30–50% dataset reduction
Dynamic selection	During training	Curriculum Learning, Active Learning, Semi-Supervised	10–30% faster curricula; 2–100× fewer labels
Synthetic generation	On-demand	Augmentation, Generative Models, Distillation	2–10× effective data expansion

Once the dominant constraint is named, table 9.16 compares which technique changes that constraint and why. The same mapping recurs across the five lighthouse workloads, because each is bound by a different resource.

Table 9.16: Technique Selection Guide by Primary Constraint: Candidate data-selection techniques mapped to the dominant resource constraint in the ML pipeline. The table begins the diagnosis; prerequisites and measured ROI determine the final choice.

Constraint	Candidate Technique	Why
Limited labeling budget	Active Learning	Maximizes label ROI by selecting informative samples
High redundancy in data	Deduplication + Coreset	Removes waste before training begins
Rare classes or edge cases	Synthetic Generation	Creates samples that do not exist in raw data
Slow convergence	Curriculum Learning	Improves gradient quality in early training
Privacy requirements	Privacy-audited synthesis	Reduce direct use of real records; verify leakage
Capable teacher, small data	Knowledge Distillation	Teacher's soft targets add supervision per example

Lighthouse 9.3: Lighthouse data selection

Data selection principles apply to all five lighthouse models, though the dominant constraint determines the technique. Table 9.17 maps each workload to a priority. Compute-bound ResNet-50 training benefits from coreset selection, while GPT-2/Llama training relies on deduplication and quality filtering to cut the raw tokens it must process. MobileNetV2's binding constraints, latency and power, apply at inference, where no choice of training data changes the cost of a prediction. The data lever that matters is therefore the one that helps a small model generalize, augmentation matched to its capacity, and the work is validating that the augmentation preserves the invariances the deployment environment requires. DLRM, bound by memory capacity, uses interaction deduplication to cut redundant examples and embedding pruning to shrink massive embedding tables by dropping or rehashing cold IDs.

Finally, the tiny Keyword Spotting model is bound as much by a scarce, costly seed set as by its memory budget, and it overcomes that scarcity by using augmentation and synthesis to build robust datasets from minimal seed samples.

Table 9.17: Data Selection Priorities by Lighthouse: Primary bottleneck and candidate data-selection priority for each lighthouse workload. The pairing suggests a hypothesis to test; the dominant resource constraint alone does not guarantee that a technique pays off.

Lighthouse	Primary Bottleneck	Data Selection Priority
ResNet-50	Compute	Coreset selection directly reduces training FLOPs
GPT-2/Llama	Training compute (tokens × FLOPs/token)	Deduplication and quality filtering cut tokens processed; curriculum may improve token efficiency
MobileNetV2	Latency/Power	Validate augmentation against the target model and deployment invariances
DLRM	Memory capacity	Interaction deduplication cuts examples; embedding pruning shrinks tables only when cold IDs are dropped or rehashed
Keyword Spotting	Data scarcity (small seed set)	Augmentation and synthesis create datasets from minimal seeds

Across these workloads, data selection functions as a systems optimization tailored to relieve whichever physical resource binds the pipeline.

Both table 9.16 and the lighthouse mapping assume a single dominant constraint, but real projects face several at once. The triage flowchart in figure 9.13 therefore routes candidate strategies hierarchically from the root bottleneck (labeling expense, compute limits, or data scarcity) through operational checks on oracle availability, dataset redundancy, teacher or simulator support, and unlabeled-pool size to isolate actionable techniques.

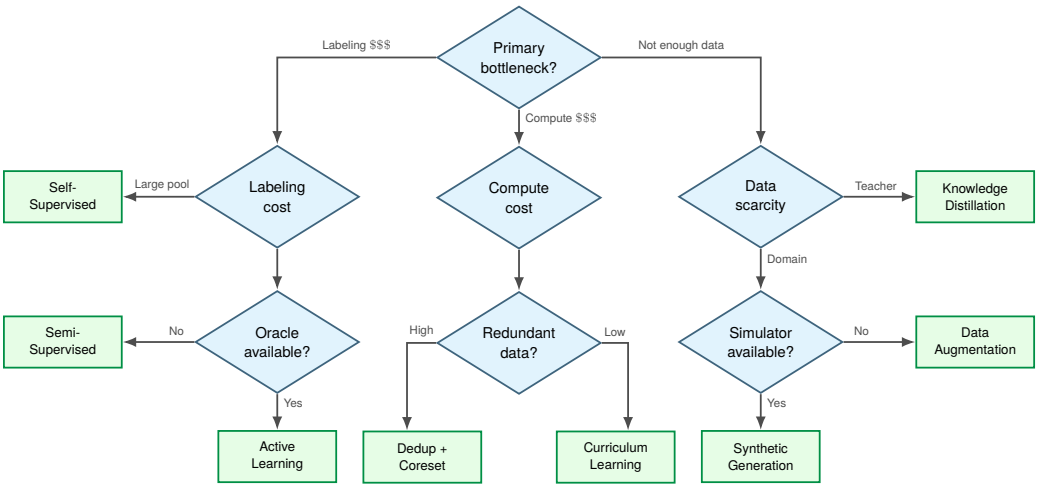


Figure 9.13: Data Selection Decision Tree: The tree begins with the primary bottleneck (labeling cost, compute cost, or data scarcity) and narrows each branch using available pools, oracles, redundancy, teachers, or simulators. Its leaves identify candidate techniques; projects with multiple binding constraints may require more than one branch.

The tree identifies candidates; an engineering decision must still test their cost and quality. The vision workload in example 9.6 provides a concrete comparison. Its target-model and proxy scorers retain the same fraction of data, but they need not retain the same examples. A cheaper ranking can therefore win the time comparison and lose the deployment-quality comparison.

Decision process

Step 1: Assess the bottleneck

The vision team's stated objective is shorter training turnaround at the existing quality target. The full-data baseline takes 277.8 h, and both candidates retain 10 percent of the images. This places the decision in the compute-cost branch. The retained fraction alone does not settle it; selection time and training time to the target must both be included, and the selected data must preserve the required slices.

A different binding constraint would change the comparison. If labels were the limiting expense, fewer annotations could justify additional training time. If controlled sweeps instead diagnosed a shortage of useful tokens, the data-scarcity branch would be relevant even with idle accelerators. Naming the objective determines which saving the candidate must actually deliver.

Step 2: Check prerequisites

Both vision candidates require access to the full image pool, a scoring model, and stable sample indices. The proxy additionally needs a ranking that transfers to the target model; its lower execution cost does not establish that prerequisite. Table 9.18 places these requirements alongside those of the alternative branches.

Table 9.18: Technique Prerequisites: Each technique carries specific infrastructure and data requirements that must be verified before implementation. A technique with excellent theoretical gains but unmet prerequisites will fail in practice, making this checklist the first step in technique selection.

Technique	Prerequisites
Active Learning	Access to oracle, unlabeled pool, retraining infrastructure
Coreset selection	Proxy model or embedding extractor, full dataset accessible
Curriculum Learning	Difficulty scoring method, pacing schedule
Semi-Supervised	Some labeled data, unlabeled data from same distribution
Self-Supervised	Large unlabeled corpus, pretraining compute budget
Augmentation	Domain knowledge of invariances, augmentation library
Synthetic Generation	Generative model or simulator, domain gap mitigation

Step 3: Estimate ROI

For the stated vision scenario, target-model scoring plus subset training takes 30.6 h (Option A), while proxy scoring plus subset training takes 28.3 h (Option B). These figures evaluate the marginal scoring pass using existing checkpoints. Both options beat the full-data baseline in the cost model. The proxy reduces only the scoring term; the retained training work is identical.

Suppose pilot validation in this hypothetical decision confirms that both runs reach the aggregate target under the modeled schedule, but only Option A meets every deployment-critical slice threshold. Option B regresses on a rare class. Option A is then the acceptable choice despite its more expensive scoring pass, saving 247.2 h against the baseline. Option B remains a candidate for repair, not deployment: enforce the coverage floor from section 9.7.4, rescore or enlarge the subset as needed, and repeat the time-to-quality comparison. Additional examples or passes change its cost, so the original proxy saving cannot simply be carried forward. If both candidates passed all quality gates at the stated times, Option B would win this time comparison.

A time saving must still justify any engineering investment. If the proxy checkpoint is not pre-existing and its training must be charged entirely to a single run rather than amortized across a campaign (section 9.13.4), that preparatory training will erase Option B's modest time advantage over Option A. Section 9.13.2 prices that investment, and section 9.13.4 spreads one-time costs across the planned experiments N_{runs}. Reusing the proxy is economically useful only while its ranking continues to pass the same quality checks.

Step 4: Combine techniques

Data selection techniques are not mutually exclusive, but combining them does not guarantee additive gains. A representative workflow deduplicates the raw corpus, applies a coverage-aware coreset filter, orders retained batches through an empirical curriculum, and applies runtime augmentations during batch collation. Pretrained representations alter the baseline trajectory further. Every

additional stage must justify its compute overhead, because cascading filters frequently discard overlapping subsets while their scoring latencies accumulate.

Stages can compound efficiency gains when their effects do not overlap and their overhead remains small; combined savings must be measured rather than assumed.

A choice made at the data layer does not stay there. This decision process prices a technique by its own scoring cost and the training work it removes, but a smaller or reordered corpus also changes what the model learns to represent, which hardware resource binds once compute pressure falls, and how much synchronization a multi-accelerator run performs. A selection that looks optimal in isolation can therefore move cost into compression, the input pipeline, or communication, so the chosen strategy must be checked against the layers it feeds.

9.15 Cross-Layer Interactions

Halving a training corpus halves the forward and backward passes, yet the resulting model may be no cheaper to compress or serve, and a selection policy that balanced work on one accelerator can leave eight workers idle at a synchronization barrier. Real ML systems combine data selection with model-level efficiency, machine-level throughput, and distributed training, and these optimizations amplify or undermine each other in ways no single layer reveals.

In the D·A·M taxonomy, data selection operates on the data dimension to change the examples processed and the learning signal they provide. Yet execution time under the iron law of ML systems depends on how input pruning alters the algorithm (model representations, loss landscapes, and compressibility) and the machine (memory bandwidth, kernel arithmetic intensity, and inter-accelerator communication). These cross-layer interactions can amplify efficiency gains across the stack or introduce unexpected bottlenecks that negate upstream savings. System efficiency requires co-designing data curation alongside model compression and hardware scheduling rather than optimizing each component in isolation.

9.15.1 Model-level efficiency

Model-level efficiency reduces parameter counts and arithmetic operations in the trained network through post-training weight pruning, lower-precision representation, and distillation. The composition of the training corpus shapes internal representations, altering how susceptible the resulting network is to downstream compression. However, arithmetic operation reductions do not automatically reduce serving latency; unless the target execution substrate and runtime kernels support the resulting sparsity or precision patterns, wall-clock inference time can remain unchanged despite substantial parameter elimination (Hoefler et al. 2021).

↪ Why accelerators turn structured sparsity patterns into wall-clock speedup while unstructured sparsity rarely pays off is developed in section 11.4.4.

This cross-layer interaction traces back to how networks allocate capacity during training. A model trained on an uncurated, repetitive corpus frequently assigns representational capacity to redundant features that post-training pruning subsequently discards, wasting the training compute expended to learn them. Conversely, training on a curated corpus of diverse, informative examples can produce more compact internal representations from the start. That initial efficiency can simplify subsequent pruning or quantization, although this remains an empirical hypothesis to verify through profiling rather than an invariant property of every selected dataset. Data selection and model compression remain distinct levers within the D·A·M taxonomy: data selection reduces training work and shapes learned representations, whereas model compression shrinks the trained deployment artifact. Their combined impact must be measured by benchmarking serving latency and task accuracy directly rather than assuming that an upstream dataset reduction automatically guarantees downstream model efficiency.

↪ Co-designing post-training pruning and quantization under the silicon contract is developed in section 10.1.

9.15.2 Machine-level throughput

While model-level efficiency determines the computational work remaining after training, machine-level throughput determines how efficiently hardware executes the training loop itself. Under the iron law of ML systems, training runtime depends on whether execution is bound by input data transfer (D_{vol}/BW), arithmetic execution ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), or scheduling latency (L_{lat}). Pruning samples reduces total floating-point operations, but non-uniform sampling and dynamic filtering alter data-loader access patterns and expose new hardware limits. Table 9.19 outlines how selection strategies shift the dominant system bottleneck.

Table 9.19: Diagnosing Bottlenecks after Data Selection: Dataset and access-pattern changes can expose a different bottleneck, but dataset size alone does not determine the limiting resource. Re-profile the pipeline and match the response to the measured condition.

Observed Condition	Possible Bottleneck	Candidate Response
Input rate below accelerator demand	Storage, decode, or host link	Sharding, prefetching, parallel decode
Low-intensity training kernels	Accelerator memory bandwidth	Fusion, reduced traffic, lower precision
Dynamic scoring dominates iteration	Selection compute or index I/O	Proxy models, cached scores or embeddings

Data selection can therefore shift the hardware execution regime. A technique that reduces sample count by 80 percent may relieve pressure on the accelerator's arithmetic units, only to expose random-access storage latency, host-to-device PCIe bandwidth, or scoring overhead as the new critical path. Relieving that subsequent bottleneck requires matching the engineering response to the active constraint: caching selection scores, fusing memory-bound operations, or pre-staging contiguous shards into host memory before accelerator dispatch. Before applying aggressive data reduction, engineers must profile the full pipeline to verify which bottleneck actually binds execution.

9.15.3 Distributed training

The hardware bottleneck analysis in section 9.15.2 focuses on execution on a single accelerator. When training scales to several accelerators in one node (two to eight devices in this book's scope), data selection alters both communication volume and memory synchronization.

In data-parallel training across accelerators, each device computes forward and backward passes on a local micro-batch before synchronizing parameter gradients via an all-reduce collective. With per-device batch size and epoch count held fixed, pruning the training corpus reduces the total number of parameter updates across a run, directly eliminating collective communication rounds over PCIe or NVLink interconnects. Furthermore, smaller per-worker shards can increase the proportion of the dataset that fits into local host RAM or accelerator cache, reducing repetitive storage streaming.

These communication savings must be balanced against the coordination overhead analyzed in section 9.12. While gradient synchronization exploits dedicated high-bandwidth interconnects, dynamic sample scoring and index exchanges often route through host memory and CPU dispatch threads. A selection policy that scales efficiently on a single accelerator can introduce serialization bottlenecks or imbalanced shard workloads when partitioned across eight parallel workers, stalling devices at synchronization barriers and eroding the compute time saved by pruning.

9.15.4 The optimization stack

Pairwise interactions expose localized bottlenecks, but production deployments execute these techniques as an integrated pipeline. Figure 9.14 diagrams the complete optimization stack across two stages, tracing how data selection and training curate raw inputs into a trained model before post-training compression and hardware acceleration deploy the artifact to production. Every stage along this end-to-end path amplifies or attenuates downstream efficiency.

Tracing artifacts through the pipeline in figure 9.14 reveals why data selection occupies an asymmetric position at the head of the optimization stack: pruning raw inputs reduces upstream training FLOPs, but does not automatically shrink parameter footprint, improve quantization headroom, or relax serving latency constraints in the downstream compression and hardware stages. Each downstream tier inherits both the computational savings and any representational deficits created upstream; an aggressive filter that discards rare but critical edge cases can force downstream engineers to train longer or abandon aggressive quantization to preserve task accuracy. The same asymmetry underlies the recurrent failure modes that follow, which show how data selection breaks down in production and how systems engineers diagnose them before launching expensive training runs.

9.16 Fallacies and Pitfalls

Data selection involves counterintuitive diminishing returns that contradict the "more is better" intuition from traditional machine learning. The following errors fall into three groups: conceptual

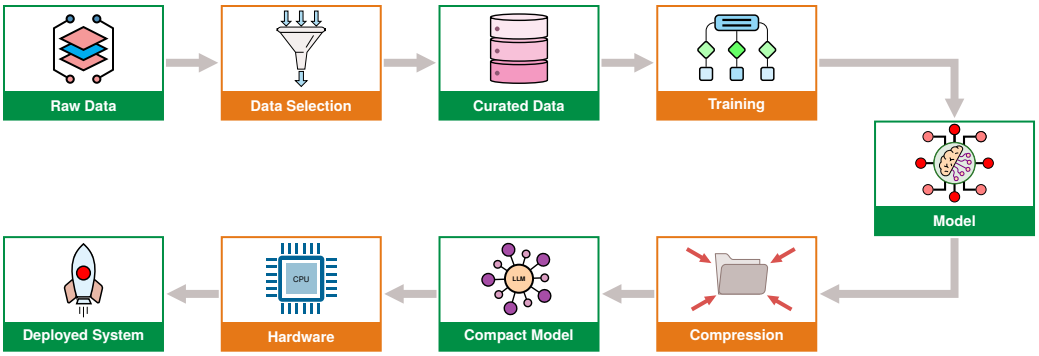


Figure 9.14: The Optimization Stack: Data artifacts flow through selection and training before the resulting model enters compression and deployment. Upstream changes propagate quality and cost effects downstream, but reducing training data does not automatically simplify the model or deployment hardware.

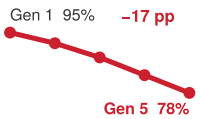
fallacies about what data selection can achieve, implementation pitfalls that arise when correct strategies meet engineering realities, and transfer errors that occur when benchmark results are applied uncritically to new domains.

Fallacy: *Data is the new oil, so more is always better.*

More data does not imply proportional accuracy gains because learning curves exhibit power-law diminishing returns. Test loss diminishes sub-linearly with dataset size ($\mathcal{L}(D) \propto D^{-\beta}$ for a fitted exponent β), meaning each successive error reduction demands a multiplicatively larger corpus. In an illustrative learning-curve scenario, scaling a dataset tenfold from 1M to 10M samples yields only 4 percentage points of accuracy gain. In systems terms, that $10\times$ volume expansion requires a tenfold increase in storage I/O, host-to-device transfers, and accelerator FLOPs, expending massive cluster energy to process redundant, low-gradient examples that contribute negligible information to parameter updates. Engineering teams must measure the empirical learning curve and marginal compute cost of their specific workload rather than assuming uncurated data scaling will deliver proportional value.

Pitfall: *Replacing real-data validation with synthetic-only training data.*

Engineers often assume generative models can replace empirical data collection with inexhaustible synthetic samples at zero marginal acquisition cost. Synthetic-only training faces two primary failure modes. First, as examined in section 9.10.3, synthetic data can differ from the deployment distribution, allowing a model to learn features that fail to transfer (figure 9.9). The resulting effect on classification must be checked with real-data labels; marginal feature densities alone do not determine the decision boundary. Second, recursive training on model-generated data can trigger model collapse (Shumailov et al. 2024). Feeding synthetic outputs back into subsequent training cycles progressively erodes distribution tails: in this scenario, evaluation accuracy degrades from 95 percent to 78 percent across five recursive generations, suffering a 17 percentage-point collapse. Although synthetic augmentation provides valuable coverage for scarce tail classes, synthetic mixtures must always be validated and anchored against curated real-world distributions.



Illustrative model collapse: accuracy erodes with each recursive synthetic generation.

Fallacy: *Data selection is merely data cleaning.*

Engineers often conflate data quality (correcting malformed schemas, corrupted labels, and noisy tokens) with data value under a specific training procedure. A dataset can be completely clean, correctly labeled, and grammatically impeccable, yet mathematically redundant: if the model has already minimized loss on a feature cluster, processing additional identical samples yields zero gradient signal while burning memory bandwidth and accelerator FLOPs. Data cleaning addresses intrinsic data defects, whereas data selection actively maximizes the learning return per FLOP spent. By selecting examples for task-relevant information and coverage (figure 9.6; see algorithmic objectives in section 9.7.4), coreset algorithms can elevate the ICR, as in the worked coreset scenario of notebook 9.1, where the selected subset reached 1.8× higher ICR. Cleaning ensures data correctness; selection optimizes training compute efficiency.

Pitfall: *Treating data selection as a budget-only tactic.*

Practitioners often view data selection as a technique reserved for resource-constrained edge deployments or budget-limited teams, assuming that frontier foundation models can simply brute-force convergence by scaling cluster size. In reality, large-scale training is bounded by the public data wall charted in figure 9.1, where exponential token consumption rapidly converges toward the finite ceiling of high-quality human text stock. Furthermore, the economic return of selection scales with cluster expenditure: a 10 percent efficiency gain on a $100M frontier training run saves $10M under our baseline assumptions. When selection infrastructure, pre-computed importance scores, and cached embeddings are amortized across repeated training runs and downstream adaptations (section 9.13.4), intelligent filtering becomes a foundational economic lever for large-scale engineering systems.

Fallacy: *Selection overhead is too small to change training economics.*

A data selection strategy claiming a training-time saving must spend less time scoring and filtering than it saves in training at matched quality. As formalized by the selection inequality in equation 9.2, that time-saving claim requires $T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}})$. Consider a baseline training run requiring 8 hours, where training on an optimized coreset takes 2-hour. If a complex selection algorithm requires 10 hours to compute sample importance, the selection step alone consumes 5× the duration of the accelerated training run and exceeds the full training baseline itself, generating a net wall-clock penalty. Cached embeddings or lightweight proxy networks can reduce scoring time enough to satisfy equation 9.2, provided selection quality transfers. In our illustrative comparison, proxy-based EL2N scoring completes in 30 minutes, just 6.2 percent of the full-training baseline, satisfying the inequality and delivering tangible wall-clock savings.

Pitfall: *Pruning rare classes into oblivion.*

Standard selection objectives minimize aggregate empirical loss, giving tail slices minimal influence over global sampling probabilities. In a 1M dataset where rare classes comprise 0.1 percent of the data (1,000 examples), an unstratified 10 percent coreset retains only 100 rare-class examples in expectation, far below the illustrative 150-sample retention floor assumed here. An appropriate floor must be validated from task-specific learning curves and critical-slice performance. While importance sampling can theoretically compensate by weighting loss contributions by inverse selection probabilities, extreme sampling weights inflate gradient variance. This gradient variance destabilizes optimizer momentum trajectories, forcing smaller learning rates that slow cluster convergence. As recommended in section 9.7.4, production coreset pipelines must enforce hard stratification floors per class or deployment-critical slice before optimizing the remaining selection budget.

Fallacy: *Deduplicating training data is enough to make evaluation reliable.*

Engineers often assume that eliminating duplicate sequences within the training partition prevents memorization and ensures clean evaluation. However, web-scale ingestion pipelines scrape training, validation, and benchmark test splits from common web crawls, introducing silent cross-split leakage. When evaluation sets contain duplicates or near-duplicates of training documents, validation metrics measure memorization rather than out-of-distribution generalization. A comprehensive deduplication study revealed that train-test overlap affects over 4 percent of validation examples across standard language-modeling benchmarks (Lee et al. 2022). Deduplicating training data in isolation leaves this evaluation contamination untouched. Rigorous pipeline hygiene requires cross-partition deduplication using n-gram matching or MinHash LSH (section 9.7.2), calibrating near-duplicate similarity thresholds against the declared split policy. For a frozen evaluation set, remove overlapping training records while preserving the evaluation set and its provenance; rebuilding evaluation splits requires a new documented benchmark version.

Pitfall: *Active learning without considering annotation latency.*

Theoretical formulations of active learning assume a zero-latency oracle that labels queried samples instantaneously, allowing the model to incorporate new gradients immediately before selecting the subsequent query batch. In production systems, human annotation queues introduce substantial turnaround delays, such as the illustrative 14-day latency between sample dispatch and label return

assumed here. This latency breaks the active learning feedback loop in two ways. If accelerator clusters halt training while awaiting labels, expensive hardware sits idle, destroying cluster compute utilization. Conversely, if training continues asynchronously on background data while annotations are pending, model weights drift across training steps; by the time the labels arrive, the uncertainty scores that justified querying those samples are obsolete. Furthermore, dispatching massive batches to amortize labeling turnaround causes information redundancy within the query batch unless diversity penalties are applied. The return on investment for active learning depends on both label pricing and the ratio of annotation turnaround latency to the model retraining cycle.

Fallacy: *If a technique works on ImageNet, it will work on my dataset.*

Data-selection pruning ratios derived from standard vision benchmarks cannot be blindly transferred to domain-specific workloads. Benchmark datasets like CIFAR-10 and ImageNet exhibit high semantic redundancy across canonical object classes, allowing aggressive coreset pruning (often 30 to 50 percent) without measurable accuracy degradation. In contrast, specialized domains such as histopathology, satellite remote sensing, or industrial defect detection exhibit heavy-tailed feature distributions where safety-critical patterns appear in only a few rare samples. Discarding 50 percent of examples based on an ImageNet heuristic can eliminate the sole boundary examples that separate subtle pathologies, triggering catastrophic domain degradation. Furthermore, geometric coreset algorithms rely on embedding spaces: if the feature extractor was pretrained on natural images, its representations lack the inductive bias needed to resolve fine-grained domain distinctions. Engineering teams must measure the redundancy of their own data, evaluate representations against target downstream tasks, and validate retention fractions across safety-critical slices before applying pruning.

Pitfall: *Optimizing data selection metrics instead of deployment metrics.*

Selection algorithms optimize surrogate objectives such as overall loss reduction, area under the learning curve (AULC), or performance-per-data (PPD, section 9.6.1), each evaluated as an aggregate scalar over the validation corpus. A selection policy retaining a 10 percent coreset can achieve superior aggregate PPD by prioritizing dense clusters of common, easy samples that accelerate average convergence, while discarding ambiguous edge cases and rare slices. In production, however, system failures occur on tail slices, edge cases, and protected sub-populations rather than dataset averages. Maximizing aggregate selection efficiency without subgroup constraints produces models that pass global validation gates but fail catastrophically in deployment. As mandated by the measurement framework in section 9.6, selection policies must be evaluated using stratified, slice-level benchmarks that enforce coverage across demographic groups, rare categories, and operational edge cases, even when doing so slightly lowers average PPD.

9.17 Summary

Data selection treats training volume as an engineering variable rather than an unchangeable constraint. Where traditional machine learning asks how many samples are required to reach target accuracy, systems engineering optimizes total cost across compute, storage, labeling, energy, and wall-clock time. Curated datasets can match or outperform massive uncurated corpora because marginal learning signal varies by orders of magnitude across raw examples. Within the optimization ordering, selection addresses the first question: which examples are worth processing, prioritizing, or generating for the target task.

This systems perspective becomes actionable through three quantitative tools: the ICR, the selection inequality, and the total data cost model. The information-compute ratio quantifies learning gained per floating-point operation, providing a proxy for sample value that must be evaluated alongside coverage and target quality. The selection inequality tests a training-time claim by requiring scoring plus subset training to take less time than full-data training at matched quality. Together with return-on-investment and break-even analysis, these formulations determine whether a proposed selection mechanism saves resources or merely shifts expenditure between stages.

The three complementary intervention points address different parts of this cost equation: static pruning removes redundancy before training through coreset selection and deduplication; dynamic selection prioritizes informative examples during training through curriculum and active learning; and synthetic generation creates data where none exists through augmentation, simulation, and

distillation. Together, these strategies address the data wall, the structural asymmetry between rapidly growing compute capacity and slowly growing high-quality data.

Self-supervised learning sits beside these levers rather than within them. Pretraining moves the bulk of representation learning into an unannotated upstream phase, so downstream tasks can reach target quality with far fewer task-specific labels; the worked amortization scenario assumes 100× fewer. The compute-only crossover occurs at roughly 10.5 tasks under the scenario assumptions. Labeling savings enter a separate dollar comparison that also requires compute prices and other differing costs.

In production, the difficult part is rarely the scoring rule itself. It is keeping selection from becoming the next bottleneck. Proxy models and cached embeddings bound scoring time, shard-based loaders keep storage access sequential, and data echoing recovers accelerator cycles when ingestion lags. Controlled model-size and data sweeps diagnose whether a run is data-limited or compute-limited in the first place, and PPD, AULC, and DCR then show whether a reduction bought what it promised at the binding resource.



Key Takeaways: Curate, do not accumulate

- **Selection optimizes system cost**: The objective is minimizing total cost across compute, storage, labeling, and energy rather than reducing sample counts alone. The information-compute ratio measures learning gained per FLOP spent; total-cost accounting and pipeline measurements establish whether that gain serves the system objective.
- **Start with deduplication**: Exact deduplication is often the lowest-risk first technique because it removes repeated bytes without a learned scoring pass. Apply near-duplicate and semantic deduplication only after validating thresholds against downstream quality metrics.
- **Training-time savings must exceed selection overhead**: At matched quality, $T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}})$. Lightweight proxy models and cached embeddings keep $T_{\text{selection}}$ low, whereas unoptimized selection algorithms can consume all the compute they save.
- **Dynamic selection adapts the data diet**: Curriculum learning changes presentation order, while active learning changes which samples receive labels. Either improves efficiency when its scoring rule, coverage, and refresh overhead are validated.
- **Self-supervised pretraining can amortize adaptation cost**: Pretraining once and fine-tuning many times spreads pretraining cost across downstream tasks. The worked scenario assumes a 100× label reduction and a 20× marginal-compute reduction; realized savings depend on transfer quality and reuse.
- **Synthetic data is a supplement, not a replacement**: Synthetic samples augment scarce tail cases effectively, but cannot substitute entirely for empirical data. Unanchored synthetic training introduces domain-gap degradation, and recursive synthetic loops trigger catastrophic model collapse.
- **Validated workload reduction compounds downstream**: When selection safely eliminates examples or tokens without adding compensating steps, that work never reaches later model or machine optimizations. Dynamic scoring and synthesis add their own costs, so the combined saving must be measured rather than assumed.

The governing invariant of data selection is that training examples provide vastly unequal marginal learning signal per byte transferred and FLOP executed. Within the D·A·M taxonomy, selection operates at the head of the system: every redundant sample or uninformative token safely eliminated directly reduces operation count (O) in the iron law of ML systems (principle 3) and can avoid downstream training reads. Scoring may still read the entire pool, and shard streaming may read discarded records, so reductions in data volume (D_{vol}) must count scoring and shard-read traffic separately. When fewer examples are actually processed and selection overhead satisfies the time-saving comparison at matched quality, that upstream reduction removes work before downstream

compression or hardware acceleration ever sees it. Model-based scoring offers a way to invest additional compute in selection, consistent with the bitter lesson, but a stronger scorer earns that investment only if its improved rankings save more downstream work than it costs. The Pareto frontier (principle 5) applies upstream of the model as well. Each retained fraction is an operating point that trades coverage and rare-slice accuracy against compute, and no single fraction is optimal for every workload.



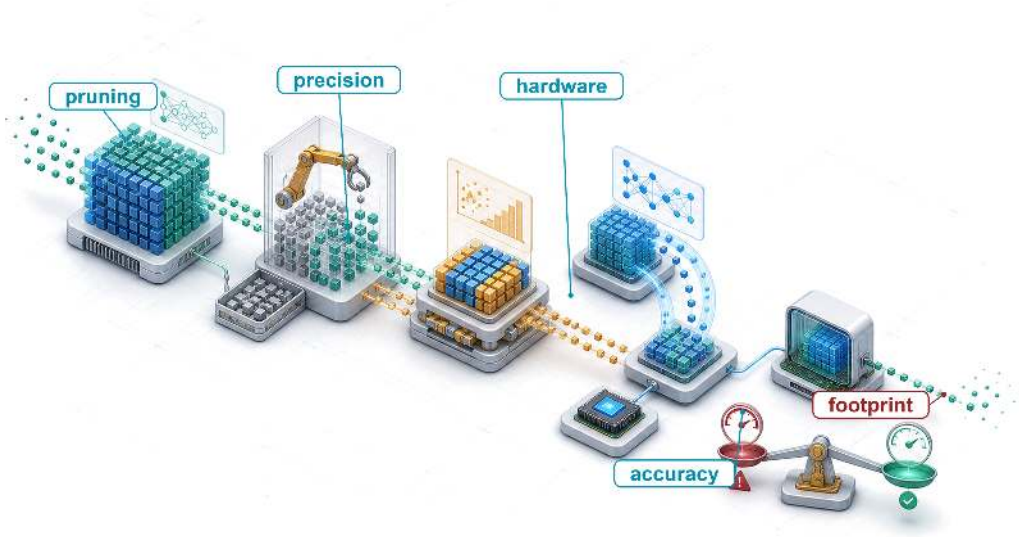
What's Next: From data to algorithms

Data selection changes *what* the system learns from, and its savings end where the trained model begins. A model trained on a carefully curated subset has the same parameter count, numerical precision, and memory footprint as one trained on the full corpus, so it can still overflow a phone's memory or leave inference bound by memory bandwidth. Chapter 10 turns to *how* the model represents what it has learned. It starts from the deployment envelope the model must fit and renegotiates three things against it: the model's structure through pruning and distillation, its numerical precision through quantization, and the way the remaining operations map onto the hardware that executes them.

10

Model Compression

10.1 Optimization Framework
10.2 Deployment Context
10.3 Structural Optimization
10.4 Quantization and Precision
10.5 Hardware Implementation
10.6 Technique Selection
10.7 Combining and Sequencing Techniques
10.8 Efficiency Measurement
10.9 Implementation Tools
10.10 Fallacies and Pitfalls
10.11 Summary



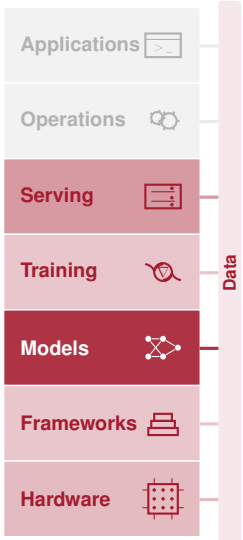
Purpose

Why do the models that win benchmarks rarely become the models that run in production?

Training produces a capable model, and data selection makes it cheaper to produce; neither guarantees that the resulting artifact can be deployed. Cloud, Edge, Mobile, and TinyML environments each impose physical constraints that research benchmarks ignore, including memory budgets measured in megabytes rather than gigabytes, latency targets measured in milliseconds rather than seconds, and power envelopes measured in milliwatts rather than kilowatts. Research optimizes for accuracy on held-out test sets; production optimizes for accuracy per dollar, accuracy per watt, and accuracy per millisecond. Models that win benchmarks are frequently larger, slower, and more resource-intensive than production envelopes permit.

Bridging this gap requires a systematic discipline of compression that trades capabilities the deployment does not need for physical constraints it cannot violate. Many trained models carry more numerical precision, more parameters, or higher representational capacity than the target workload requires. Some of this surplus can be removed while preserving predictive behavior. Yet a smaller representation is not automatically a faster one. The hardware and runtime software stack must exploit the new precision or structure; otherwise, a nominal size reduction merely moves the bottleneck while latency remains unchanged. Removing half the weights in a tensor leaves execution time and energy untouched if the underlying hardware still fetches, stores, and multiplies every zero.

Applied effectively, compression reduces model size by integer factors, transforming a research artifact that runs only on data center accelerators into a production asset capable of executing on a mobile phone, sensor node, or embedded microcontroller. The engineering goal is to make the right



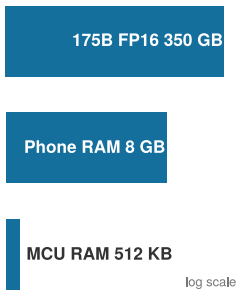
model viable within its physical environment, which differs fundamentally from making a model as small as possible. In D·A·M terms, model compression enacts algorithm-machine co-design directly on the network, reshaping its mathematical structure to satisfy the physical limits of the target hardware.

Learning Objectives

- Explain compression as algorithm-machine co-design that trades surplus capacity for memory, latency, and energy constraints
- Compare pruning, distillation, quantization, and architecture search by the resource constraint each relaxes
- Calculate parameter memory, precision, and sparsity reductions to estimate best-case compression gains
- Apply post-training, quantization-aware, and weight-only strategies under accuracy and hardware constraints
- Select structured pruning and operator choices that map to available accelerator kernels
- Design compression pipelines that order pruning, distillation, and quantization to preserve deployment accuracy
- Evaluate measured latency, energy, and accuracy on target hardware rather than relying on FLOP counts

10.1 Optimization Framework

A 7-billion parameter language model requires 14 GB merely to store its weights in FP16. The intended deployment target is a smartphone with 8 GB of RAM shared across the operating system, background applications, and the user interface. Even if the operating system allocated its entire memory pool to a single process, 14 GB exceeds the physical capacity of the device. Even if the model fit within physical memory limits, it would still confront a second physical bottleneck: token generation latency. At batch size one, dense autoregressive decoding is memory-bandwidth bound because generating each token reads every weight from DRAM, and no other request shares those bytes. Over a mobile low-power DDR (LPDDR) bus delivering roughly 50 GB/s, streaming 14 GB takes nearly 300 ms per token and draws several joules, rapidly draining the battery and violating the device's sustained thermal envelope of 3 to 5 W. This physical mismatch between model footprint and edge hardware capacity defines the core engineering mandate of model compression.



Frontier weights dwarf phone and microcontroller memory; compression bridges the gap.

Under the silicon contract (principle 4), every machine learning model strikes a physical performance bargain with its hardware: compute throughput, memory bandwidth, memory capacity, and fixed launch overhead determine which physical resource binds first. During training, this bargain is negotiated upward: server-class accelerator clusters provide the aggregate memory and floating-point throughput needed to support wide layers, high numerical precision, and large auxiliary state. Training must maintain gradients, optimizer moment buffers, and activation caches across backward passes. Inference deployment alters these requirements completely: execution is forward-only, intermediate activation buffers can be reused after their last consumer, and optimizer states disappear. Residual branches extend some lifetimes, while autoregressive decoding retains KV state across tokens. While mixed-precision methods improve training throughput by selectively adopting FP16 or BF16 (section 8.7.3), deployment allows reducing precision further to INT8, INT4, or specialized microformats when supported by target execution units. Model compression is the systematic process of renegotiating the silicon contract for a constrained execution context, reducing parameter footprint, memory traffic, or arithmetic operations while preserving task accuracy.

↩ Mixed-precision training with FP16 and BF16, including loss scaling and FP32 accumulation, is established in section 8.7.3.

Measured in weight memory, the disparity across deployment environments spans about six orders of magnitude. A 175-billion parameter frontier model consumes over 350 GB in FP16 representation alone, whereas an embedded microcontroller provides only 512 KB of on-chip SRAM. The bitter lesson explains why this gap persists. General methods that scale with compute keep winning benchmarks, so the winning models grow with available training compute while a phone's or microcontroller's memory and power envelope stays fixed for the life of the device. Compression is the deployment-side counterweight to scale. Bridging this gap requires an engineering framework with predictable physical trade-offs. Every compression technique removes specific structure from the model, whether unnecessary weights, surplus numerical bits, or redundant operator subgraphs.

Designing a production deployment requires identifying which resource binds first on the target platform and analyzing how successive approximations compound across the memory hierarchy.

The framework this chapter builds works on three levels: what the model computes (its structure), how many bits represent each value (its precision), and how the result executes on the target processor (its implementation). Benefits compound only when transformations relieve distinct binding bottlenecks and the runtime supports the resulting graph.

The volume's recurring workloads ground these trade-offs across distinct operational regimes: ResNet-50 and MobileNetV2 for vision, GPT-2 for bandwidth-bound autoregressive generation, DLRM for recommendation memory capacity (Naumov et al. 2019), and keyword spotting (KWS) for TinyML (Y. Zhang et al. 2017). Each workload stresses a different term of the iron law, from parameter-dominated embedding tables to compute-heavy matrix multiplications. Rather than running every technique on all five, the chapter returns to each workload where it exposes the binding resource most clearly: KWS and DLRM where precision decides whether a model fits, GPT-2-class decoding where weight bytes set the token rate, and ResNet-50 where a full deployment profile must be measured. Grounding these workload trade-offs within a rigorous engineering taxonomy requires formalizing the core mechanisms of model compression.

↩ The bitter lesson, that general methods which scale with compute keep displacing hand-engineered ones, is established in section 1.4.

↩ The iron law of ML systems, which splits execution time into data-movement, compute, and fixed-overhead terms, is established in section 1.7.

Definition 10.1: Model compression

Model compression is a family of techniques that reduce a trained model's computational cost and memory footprint by eliminating redundant parameters (pruning), reducing numerical precision (quantization), or transferring learned behavior into a smaller architecture (distillation), while preserving as much predictive accuracy as possible.

1. **Significance**: Compression directly reduces the iron law's data-movement and compute terms. INT8 quantization of a 175-billion-parameter LLM cuts weight memory from 350 GB (FP16) to 175 GB, a 2× reduction in D_{vol}, while dedicated low-precision matrix units can increase compute throughput when kernels and layouts use the supported INT8 path. Unstructured pruning to 50 percent sparsity halves the nonzero count, but executed operations fall only when the runtime can skip those zeros through a supported sparse format or kernel.
2. **Distinction**: Unlike post-training compression methods such as pruning and quantization, neural architecture search discovers efficient architectures from scratch by exploring a design space. Here, NAS is treated as a related structural optimization technique: it changes the representation before training rather than compressing a finished model post hoc.
3. **Common pitfall**: Compression techniques do not compose without interference. Pruning changes weight distributions and operation patterns; quantization adds calibration and kernel constraints. Their combination can lose accuracy or fail to accelerate without joint validation.

Figure 10.1 diagrams the three-tier optimization stack from algorithmic software down to physical silicon: what operations the model executes, how many bits encode each operand, and how hardware kernels map onto physical compute and memory structures.

The top layer, efficient model representation, eliminates structural redundancy within the computational graph. Techniques such as pruning, knowledge distillation, and neural architecture search (NAS)[1] modify parameter counts and operator topologies directly. Pruning removes weights or entire channels whose contribution to model accuracy is negligible. Knowledge distillation trains a compact student model to match the output distributions or intermediate representations of an overparameterized teacher. Hardware-aware NAS automates this exploration, evaluating candidate layer topologies, expansion ratios, and kernel dimensions to discover Pareto-optimal graphs under explicit latency or memory bounds.

The middle layer, efficient numerics representation, optimizes the bit-width used to encode each tensor value. Quantization maps continuous high-precision floating-point values to low-bit representations (such as 8-bit or 4-bit integers), shrinking weight memory footprint and DRAM

[1] **Neural architecture search (NAS)**: Zoph and Le (2016) at Google Brain used reinforcement learning to learn the architecture itself at a cost of 22,400 GPU-days (800 GPUs for 28 days), equivalent to 537,600 GPU-hours. Weight-sharing approaches such as Efficient Neural Architecture Search (ENAS) later reduced search cost by roughly 1,000× by sharing parameters across candidate architectures (Pham et al. 2018). Hardware-aware NAS and scaling methods then made the search output practical for deployable architecture families such as EfficientNet and MobileNetV3 (Tan and Le 2019; Howard et al. 2019).

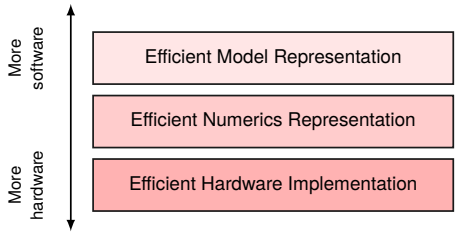


Figure 10.1: Optimization Stack: Model optimization progresses from software-level representation (pruning, distillation, NAS) down through numerical precision (quantization) to hardware execution (kernel fusion, sparsity engines). Lowering abstraction moves optimizations closer to physical silicon, where representation decisions dictate achievable execution efficiency.

bus traffic (D_{vol}) by factors of two to four. Beyond memory reduction, integer arithmetic relieves compute bottlenecks: low-precision matrix units execute integer multiply-accumulate operations with a fraction of the silicon area and energy required by 32-bit floating-point ALUs, enabling higher peak throughput within the same thermal envelope.

The bottom layer, efficient hardware implementation, aligns the remaining operations with the physical accelerator architecture. A reduced operation count does not guarantee faster execution if data movement stalls the compute cores or uncoalesced memory accesses serialize DRAM channels. Kernel fusion merges adjacent layers (folding a convolution, bias addition, and activation function into a single kernel, for example), keeping intermediate activations resident in fast on-chip SRAM or registers rather than round-tripping through high-latency off-chip DRAM. Similarly, hardware-supported sparsity engines, such as sparse Tensor Cores that skip two zeros in every group of four weights (2:4 structured sparsity), deliver physical speedup only when nonzeros conform to the alignment and fetch constraints of the hardware execution pipeline.

These three layers are tightly coupled across the compilation and execution boundary. Pruning alters weight distributions and activation profiles, changing the dynamic ranges required for subsequent quantization calibration. Quantization reduces data-movement volume across the memory bus, shifting a previously memory-bound workload toward compute-bound execution where kernel fusion and tile scheduling dominate. Achieving compounding performance gains requires co-designing transformations across all three layers rather than applying them in isolation, a composition strategy that section 10.7 develops through a staged pipeline.

Which layer of the optimization stack yields the highest return depends entirely on the physical machine hosting the workload. A data center server equipped with multiple high-bandwidth memory accelerators tolerates multi-gigabyte models but demands high token throughput under concurrency. A smartphone or an embedded microcontroller operates under severe physical ceilings on memory capacity, bus bandwidth, and milliwatt thermal dissipation. Any compression strategy therefore starts by establishing the hardware envelope of each physical tier and identifying which resource binds first in it.

10.2 Deployment Context

A data center GPU with 80 GB of high-bandwidth memory (HBM) faces fundamentally different binding constraints than a smartphone with shared RAM or a microcontroller with only a few hundred kilobytes of SRAM. Physical deployment targets dictate which resource binds first (table 10.1).

Table 10.1: Deployment Constraints: Each deployment context imposes different optimization priorities.

Context	Memory	Latency	Power	Primary Goal
Cloud	tens of GB	100–500 ms	Flexible	Throughput, cost
Mobile/Edge	hundreds of MB to GB	5–100 ms	W-scale	Size, latency
TinyML	KB–MB	1–10 ms	mW	Size, energy

10.2.1 Deployment scenarios

Cloud inference centers on throughput (requests per second per dollar), where supported integer quantization paths increase serving density and kernel fusion reduces per-request kernel launch overhead (Choudhary et al. 2020; Dean et al. 2018). Mobile and edge deployments must fit within tight physical memory bounds while meeting deterministic real-time deadlines. A serialized camera pipeline that must finish each frame before the next arrives at 30 frames per second has about 33 ms per frame. A pipelined design that overlaps capture, inference, and display can sustain 30 frames per second even when each frame takes longer than 33 ms end to end, so it must check frame rate and per-frame latency as two separate budgets.

TinyML makes physical fit a deployment requirement. A microcontroller with only a few hundred kilobytes of SRAM and limited model storage *cannot* accommodate a 100 MB model without a compatible placement strategy, regardless of predictive accuracy (Banbury et al. 2020). Persistent weights may reside in flash or other supported storage, while runtime buffers occupy working memory. Without paging, the peak SRAM working set must fit the available SRAM; the model artifact must separately fit its storage budget. Capacity is also a hard bound on mobile and cloud devices, though their placement options differ.

↩ The latency, memory, and power envelopes defining cloud, edge, mobile, and TinyML tiers are established in section 2.1.

Meeting real-time deadlines on commodity mobile hardware depends as much on the hardware-supported precision path as on absolute memory capacity. When execution units provide hardware acceleration exclusively for fixed-point arithmetic, precision selection dictates whether a workload executes on energy-efficient neural processing units or falls back to power-hungry general-purpose cores. Example 10.1 illustrates how low-precision quantization unlocks specialized mobile accelerator engines to meet interactive frame rates within tight thermal budgets.

Example 10.1: An illustrative MobileNet win

Scenario: Deploying a real-time background blur video segmentation model on mid-tier Android smartphones at 30 FPS.

Diagnosis: An uncompressed FP32 MobileNetV2 model runs on the phone's CPU at only 8 FPS, about 125 ms per frame against a 33 ms budget, while holding the CPU at a power level the thermal envelope cannot sustain. The phone's NPU or DSP accelerates integer matrix arithmetic, not FP32, so the uncompressed model cannot use it.

Systems lesson: INT8 quantization cuts raw parameter payload by 4×, but most of the gain from 8 FPS to 35 FPS comes from what INT8 unlocks, a move from the CPU onto the integer accelerator. The precision change is the enabler; the change of processor supplies most of the speed. The frame rates are illustrative, so speed and power still require device measurement.

10.2.2 Balancing trade-offs

Table 10.2 quantifies this deployment gap across the lighthouse models from section 6.2. Even MobileNetV2 at INT8 precision exceeds TinyML device memory by about 6.7×, demonstrating how physical memory envelopes constrain model viability:

Table 10.2: The Deployment Gap: Model weight requirements compared against typical device capacities. DLRM represents recommendation-model embedding pressure (Naumov et al. 2019); DS-CNN represents the TinyML keyword-spotting case (Y. Zhang et al. 2017). Even MobileNetV2 quantized to INT8 exceeds the ~512 KB TinyML envelope by 6.7×, while the purpose-built DS-CNN keyword spotter fits the weight-only envelope after INT8 quantization. This does not establish full deployment feasibility. The accounting covers weights only; activation, workspace, packaging, and metadata overheads are excluded. The Cloud column is a representative server's memory envelope, not one accelerator's HBM, and the DLRM row is a single reference configuration; production recommendation models can be far larger. Numbers in parentheses show how many times the model exceeds device memory.

Model	Weight Memory	Cloud (~107 GB)	Mobile (8 GB)	TinyML (~512 KB)
DLRM	100 GB	ok	no (11.6×)	no (190734.9×)
GPT-2 XL	6 GB	ok	ok	no (11444.1×)
ResNet-50	102.4 MB	ok	ok	no (195.3×)

Model	Weight Memory	Cloud (~107 GB)	Mobile (8 GB)	TinyML (~512 KB)
MobileNetV2	14 MB	ok	ok	no (26.7×)
MobileNetV2 (INT8)	3.5 MB	ok	ok	no (6.7×)
DS-CNN (KWS, INT8)	200 KB	ok	ok	ok

This mismatch motivates examining the **accuracy-efficiency trade-off**. Additional model capacity can improve predictive accuracy while demanding larger weight payloads, expanded activation working sets, and higher memory bandwidth during inference. On hardware with strict memory or power limits, engineers first seek avoidable costs, then choose among feasible operating points on the Pareto frontier (principle 5).



Systems Perspective 10.1: The compression-accuracy trade-off curve

The measured trade-off is not uniform. A structure-preserving implementation improvement can move a dominated model toward the frontier with little or no accuracy loss. Once on an efficient frontier, improving one objective trades against another; aggressive structural changes can enter regions with steep quality penalties.

The engineering decision is where to stop. The "knee" of the curve marks a region where further resource savings incur a sharper quality penalty. It is a useful visual diagnostic, but accuracy points cannot be compared directly with milliseconds, bytes, or joules. Stop at an operating point that meets the deployment's quality and resource thresholds, using explicit objective weights if a further trade must be ranked.

Table 10.3 maps compression techniques' systems gains against their machine learning costs. Its numbered regions describe illustrative outcome categories, from little quality loss to steep penalties, rather than fixed locations that each technique must occupy on every model's frontier.

Table 10.3: The Optimization Trade-Offs: Region 1 = Free Lunch, Region 2 = Efficient Trade, Region 3 = Danger Zone. These are representative engineering ranges synthesized from published work on inference runtimes, quantization, pruning, low-bit LLM quantization, and distillation (NVIDIA 2024c; Jacob et al. 2018; Han et al. 2015; Lin et al. 2024; Hinton et al. 2015); actual results vary with architecture, task, calibration data, and implementation quality.

Technique	Systems Gain	ML Cost	Typical Impact	Region
Kernel Fusion	10–30% latency reduction	None	No accuracy loss	1
FP32 → BF16	2× memory, ~2× throughput	Minimal	$< 0.1\%$ accuracy drop	1
FP16 → INT8	2× memory, 2–4× throughput	Quantization error	0.5–1% accuracy drop	2
50% Pruning	~2× smaller model	Capacity loss	0.5–1% accuracy drop	2
Knowledge Distillation	2–10× smaller student	Capability ceiling	1–3% accuracy drop	2
4-bit Quantization	4× memory reduction	Significant error	2–5% accuracy drop	2–3
90% Pruning	~10× smaller model	Severe capacity loss	5–15% accuracy drop	3

Techniques that preserve model structure—such as kernel fusion and modest precision reduction (FP32 to BF16)—can yield latency or memory gains with little quality loss when the model and execution path support them. In contrast, techniques that alter network structure, such as pruning and distillation, yield larger reductions in memory and arithmetic volume but risk significant quality degradation without careful retraining. Optimization begins by identifying the binding hardware constraint: memory capacity on mobile devices, frame latency in interactive pipelines, or energy per inference on battery-powered sensors. Compression strategies then target that bottleneck directly: structural methods reduce parameter and operation counts, quantization shrinks memory traffic and increases arithmetic intensity, and compiler-level fusion eliminates intermediate memory round-trips.

Checkpoint 10.1: The efficiency frontier

Optimization can remove avoidable costs or trade among efficient operating points.

Trade-offs

- ☐ **Accuracy vs. cost**: compare a structure-preserving optimization and a structure-altering one by naming which resource each saves and which quality risk it introduces.
- ☐ **The "free lunch"**: explain how an implementation improvement can move a dominated operating point toward the frontier without reducing accuracy.
- ☐ **The knee**: explain how a sharp change in the measured trade-off informs a decision, and state the quality and resource thresholds needed to choose an operating point.

Navigating this frontier requires moving beyond global trade-off curves to examine the internal mechanics of individual layers. Structural optimization initiates this process by altering the network's topological graph, removing surplus operations and parameters before lower-level representations are quantized or rescheduled for target hardware.

10.3 Structural Optimization

A standard ResNet-50 trained on ImageNet contains over 25 million parameters, yet with careful magnitude pruning and retraining about 80 percent of its weights can be removed for roughly a one-point loss in top-1 accuracy (Gale et al. 2019). Modern neural networks routinely carry far more parameters than deployment requires.[2] Unused capacity still consumes memory, computation, and energy. Structural optimization addresses the first dimension of the optimization framework, **efficient model representation**, by modifying *what* the model computes. Although additional capacity can aid optimization during training, parameters that do not improve the deployed model still impose deployment cost.

Every technique in this chapter answers to the conservation-of-complexity heuristic that opened Part I. Complexity tends to move among the Data, Algorithm, and Machine axes rather than disappear, so engineers must account for where each technique sends cost and risk. It is a heuristic, not a literal conservation law, and removing genuinely avoidable work can reduce total cost. Pruning may reduce parameters while asking the runtime to exploit sparse structure; distillation may reduce inference cost while adding a teacher-student training phase; quantization may reduce data movement while spending numerical precision. The engineer's task is to move complexity to where the cost is lowest given deployment constraints.

The systems challenge is excising surplus capacity without compromising task-critical representation. Operating along this Pareto frontier[3] requires deciding where complexity should reside: in training-time discovery, runtime sparse indexing, or student-model capacity.

These techniques address the challenge through complementary structural mechanisms. Pruning eliminates redundant parameters from an existing network; knowledge distillation transfers learned representations into an independently designed compact topology; structured approximations factorize weight matrices algebraically; and neural architecture search automates the topological discovery process from the ground up (Hutter et al. 2019). In production pipelines, these techniques frequently compose: a NAS-discovered architecture, supervised by a distilled teacher, is pruned prior to final quantization.

The following analysis evaluates what structural artifact each method produces, what fine-tuning or recovery step it requires, and which hardware execution conditions allow it to deliver physical speedups. Pruning comes first because it exposes the systems constraint most directly: eliminating mathematical parameters translates to physical hardware acceleration only when the resulting sparsity pattern aligns with the memory hierarchy and compute units of the target processor.

10.3.1 Pruning

Consider a MobileNet trained for image classification on a wearable health monitor. The trained model occupies 14 MB, but the target microcontroller offers only 2 MB of flash memory. Retraining a smaller architecture from scratch would require weeks of data collection and validation—time the

[2] **Overparameterization**: C. Zhang et al. (2017) demonstrated that networks large enough to fit ImageNet can also memorize completely random labels, showing that training capacity can exceed the structure needed for natural labels. Pruning studies then show the deployment consequence: trained models often contain many parameters that can be removed or sparsified with modest task loss when pruning and fine-tuning are done carefully (Gale et al. 2019; Blalock et al. 2020). The redundancy is not a universal 10× constant; it depends on architecture, dataset, sparsity pattern, and runtime support.

[3] **Pareto frontier in compression**: EfficientNet traces the frontier concretely, from B0 (77.1 percent accuracy, 390 million FLOPs) to B7 (84.4 percent, 37 billion FLOPs), a 95× compute increase for 7.3 percentage points of accuracy, quantifying how steep the trade-off becomes at the frontier's edge (Tan and Le 2019).

product schedule does not allow. Suppose profiling shows that about 85.7 percent of the model's weights are near zero and contribute little on the validation set. Removing those weights and fine-tuning the remainder for a few epochs leaves surviving weight values that total 2 MB with an acceptable accuracy loss. Whether the stored artifact also fits in 2 MB depends on how the surviving weights are stored, a question the unstructured-pruning discussion below makes concrete. The numbers anchor the engineering trade-off rather than reporting a universal MobileNet benchmark.

Pruning[4] directly addresses memory efficiency constraints by eliminating parameters or structures that contribute little to the deployed objective. Many trained networks contain removable capacity, but the safe fraction depends on the architecture, task, pruning pattern, and recovery procedure. The central questions are *what* to prune (individual weights vs. entire structures), *how* to estimate what is expendable (magnitude, gradients, or activations), and *when* to prune (after training, during training, or even at initialization). The foundational systems rule is invariant: zero-valued weights conserve physical resources only when the underlying hardware and runtime execution path can skip loading or computing them.

4 **Optimal Brain Damage**: Introduced by LeCun, Denker, et al. (1989), the method achieved 4× parameter reduction—and proportional memory savings—in a handwriting recognizer by using a diagonal approximation to second-derivative (Hessian) information to estimate the loss increase from removing each weight. A full Hessian has $\mathcal{O}(P^2)$ entries for P parameters, but Optimal Brain Damage avoids storing that full matrix by approximating its diagonal. Modern-scale pruning still favors cheaper criteria such as magnitude because even diagonal curvature estimation adds training cost.

Definition 10.2: Pruning

Pruning is a model-compression technique that sparsifies the parameter space by removing the weights or structures whose removal increases the training loss least.

1. **Significance**: It can convert dense matrices into sparse structures, reducing memory footprint and total data volume (D_{vol}) when the sparse representation, metadata, and recovery procedure preserve acceptable quality.
2. **Distinction**: Unlike quantization, which reduces the precision of every weight, pruning reduces the count of weights by identifying and eliminating redundancy.
3. **Common pitfall**: A frequent misconception is that pruning "automatically" speeds up execution. In reality, without specialized sparse execution support, the resulting sparse matrices may actually run slower than dense ones due to irregular memory access patterns. A speedup requires a storage format and a kernel that skip the zeros, not merely a faster processor.

The formal pruning objective seeks a sparse parameter tensor $\hat{\mathbf{W}}$ that minimizes empirical loss subject to a parameter budget k_{nz}:

$$\min_{\hat{\mathbf{W}}} \mathcal{L}(\hat{\mathbf{W}}) \quad \text{subject to} \quad \|\hat{\mathbf{W}}\|_0 \leq k_{\text{nz}}$$

where $\|\hat{\mathbf{W}}\|_0$ is the ℓ_0 **norm** (the count of nonzero parameters). Solving this cardinality-constrained optimization is combinatorial, so practical methods use heuristics[5] such as **magnitude-based pruning**. Listing 10.1 demonstrates this thresholding heuristic, while figure 10.2 contrasts the resulting 11×11 weight tensor before and after pruning, isolating how sub-threshold entries collapse to zero while irregular, high-magnitude weights persist across the grid.

5 **Heuristic**: From Greek *heuriskein* (to discover), the same root as Archimedes' "eureka." In pruning, the dominant heuristic–larger magnitude means more important–works well empirically but creates a systems trap: magnitude-based pruning applied globally can remove most parameters from overparameterized layers while leaving critical bottleneck layers largely intact, giving the appearance of aggressive compression while preserving much of the compute and memory cost in the layers that matter (Blalock et al. 2020). This is why iterative prune-retrain cycles with per-layer budgets are often safer than naive global magnitude pruning: each cycle lets the network redistribute importance before the next cut.

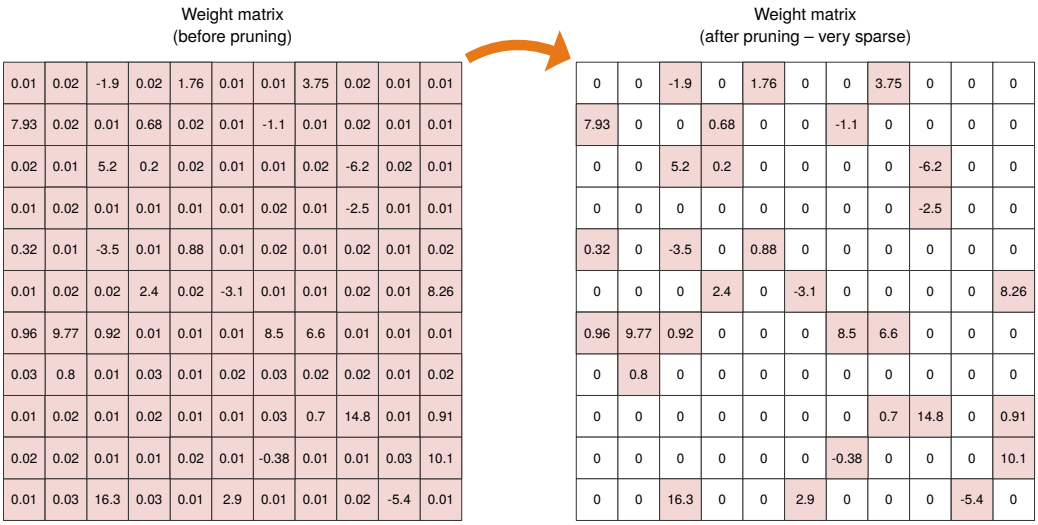
Listing 10.1: Magnitude-Based Pruning: Removes weights below a threshold to create sparse matrices, reducing the number of nonzero parameters from 9 to 4 ($k_{\text{nz}} = 4$).

```
import torch

# Original dense weight matrix
weights = torch.tensor(
    [[0.8, 0.1, -0.7], [0.05, -0.9, 0.03], [-0.6, 0.02, 0.4]]
)

# Simple magnitude-based pruning: keep only the 4 largest weights
threshold = 0.5
mask = torch.abs(weights) >= threshold
pruned_weights = weights * mask

print("Original:", weights)
print("Pruned (4 nonzeros):", pruned_weights)
```



0.01	0.02	-1.9	0.02	1.76	0.01	0.01	3.75	0.02	0.01	0.01
7.93	0.02	0.01	0.68	0.02	0.01	-1.1	0.01	0.02	0.01	0.01
0.02	0.01	5.2	0.2	0.02	0.01	0.01	0.02	-6.2	0.02	0.01
0.01	0.02	0.01	0.01	0.01	0.01	0.02	0.01	-2.5	0.01	0.01
0.32	0.01	-3.5	0.01	0.88	0.01	0.02	0.01	0.02	0.01	0.02
0.01	0.02	0.02	2.4	0.02	-3.1	0.01	0.01	0.02	0.01	8.26
0.96	9.77	0.92	0.01	0.01	0.01	8.5	6.6	0.01	0.01	0.01
0.03	0.8	0.01	0.03	0.01	0.02	0.03	0.02	0.02	0.01	0.02
0.01	0.02	0.01	0.02	0.01	0.01	0.03	0.7	14.8	0.01	0.91
0.02	0.02	0.01	0.01	0.02	0.01	-0.38	0.01	0.01	0.03	10.1
0.01	0.03	16.3	0.03	0.01	2.9	0.01	0.01	0.02	-5.4	0.01

0	0	-1.9	0	1.76	0	0	3.75	0	0	0
7.93	0	0	0.68	0	0	-1.1	0	0	0	0
0	0	5.2	0.2	0	0	0	0	-6.2	0	0
0	0	0	0	0	0	0	0	-2.5	0	0
0.32	0	-3.5	0	0.88	0	0	0	0	0	0
0	0	0	2.4	0	-3.1	0	0	0	0	8.26
0.96	9.77	0.92	0	0	0	8.5	6.6	0	0	0
0	0.8	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0.7	14.8	0	0.91
0	0	0	0	0	0	-0.38	0	0	0	10.1
0	0	16.3	0	0	2.9	0	0	0	-5.4	0

Figure 10.2: Sparse Matrix Transformation: Magnitude-based pruning replaces weights below a numerical threshold with exact zeros (white cells) while preserving salient high-magnitude connections (colored cells). The resulting sparsity cuts parameter count, but realizing hardware speedups requires translating these unstructured zero entries into sparse storage formats or specialized execution kernels.

The resulting sparse matrix retains only the high-magnitude values (colored cells) while near-zero weights become exact zeros. In models amenable to pruning, much of the representational capacity remains intact after removing low-magnitude weights, establishing magnitude thresholding as an effective baseline heuristic.

To make pruning computationally tractable during gradient-based training, practical methods often relax the combinatorial ℓ_0 constraint with continuous penalties such as ℓ_1-regularization ($\lambda_{\text{L1}}\|\mathbf{W}\|_1$), where λ_{L1} scales the sparsity penalty applied to weight tensor $\mathbf{W}$. Penalizing weight magnitudes drives non-essential parameters toward zero, facilitating downstream thresholding. Practitioners typically couple this with **iterative pruning**, removing parameters in successive increments interleaved with fine-tuning cycles to recover lost accuracy (Gale et al. 2019; Blalock et al. 2020).

10.3.1.1 Target structures

The choice of what to prune depends on the deployment target's hardware constraints and which resource is the binding bottleneck. When memory capacity is primary, **neuron pruning** can provide direct relief in models whose fully connected layers dominate parameter storage: removing entire

neurons and their associated weights and biases reduces layer width and parameter count. Profiling must first establish that these layers, rather than embeddings, activations, or another component, are the actual bottleneck.

When convolutional inference latency on commodity accelerators is the bottleneck, **channel pruning** (also called filter pruning) is a strong candidate. Eliminating entire channels or filters reduces feature-map depth and the multiply-accumulate count in subsequent layers. The resulting subnetwork remains dense and regular, so it can map to conventional GPU and Tensor Processing Unit (TPU) kernels without an unstructured sparse format. Realized latency still depends on the resulting dimensions, kernels, and memory traffic.

When a model contains removable stages, **layer pruning** removes entire layers from the network. Each removal eliminates all computation in that stage, but it also reduces representational depth and may change tensor shapes, residual paths, or downstream interfaces. The remaining layers must absorb the lost function, typically through fine-tuning or retraining. The nominal operation saving therefore does not guarantee proportional latency: graph rewrites, kernel shapes, and memory traffic still govern execution. Layer pruning demands careful task validation and end-to-end profiling. Figure 10.3 contrasts these structural granularities side by side, tracing how removing an individual filter slice contracts intermediate feature depth while excising an entire layer collapses a full computational stage.

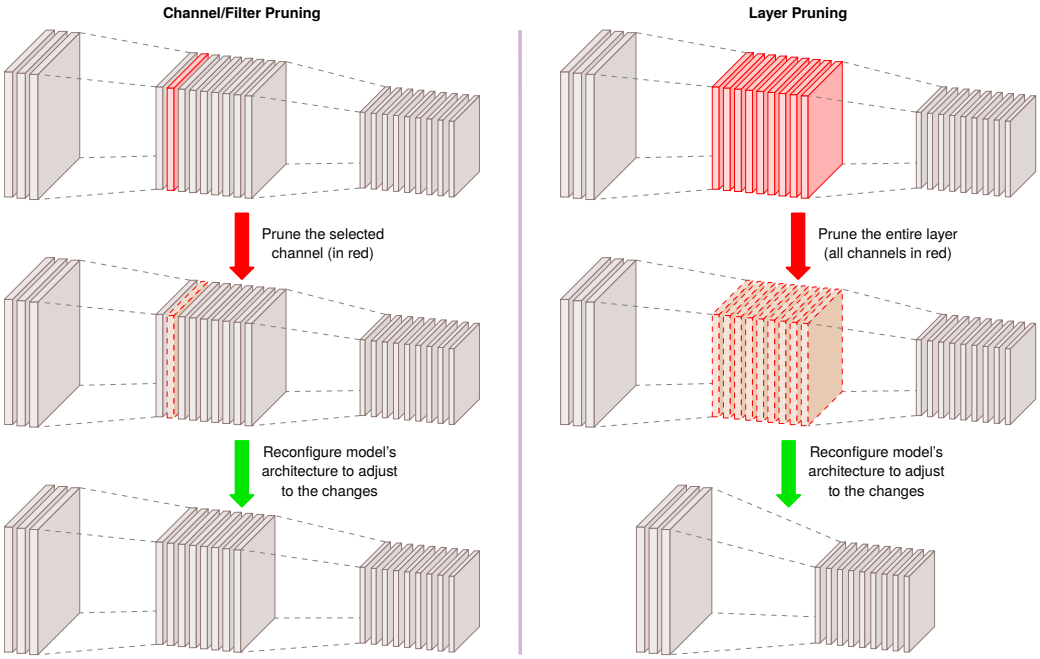


Figure 10.3: Channel vs. Layer Pruning: Structural pruning operates at different architectural granularities. Channel pruning (left) removes feature map slices within convolutional layers while preserving overall network topology and dense execution, whereas layer pruning (right) eliminates entire computational stages, requiring graph rewiring and risking greater capacity loss to achieve larger per-operation savings.

These structural transformations diagrammed in figure 10.3 establish fundamentally different execution contracts on accelerator hardware. Channel pruning contracts the inner dimensions of dense general matrix multiply (GEMM) operations, because eliminating output channels in layer ℓ reduces the input channel dimension in layer $\ell+1$. The tensors remain dense and avoid sparse pointer lookups, but their dimensions still decide how well single instruction, multiple data (SIMD) lanes and Tensor Core tiles fill. Pruning a 256-channel layer to 250 channels, for example, removes about 2 percent of the work yet can leave a matrix unit that processes channels in groups of 8 running a partly empty final tile, so the layer may run no faster. Pruning to 248 keeps every tile full. The preferred multiple depends on the kernel and numerical format, and because it counts elements

rather than bytes, it is a different constraint from cache-line or memory-transaction alignment. Layer pruning removes full computational stages and their associated launches and intermediates, but the rewritten graph must still satisfy shape, residual-connection, and kernel-layout requirements. Both approaches require quality validation.

10.3.1.2 Unstructured pruning

Unstructured pruning removes individual weights while preserving the overall network architecture. Some connections become redundant during training, contributing little to the final output. Pruning these weak connections reduces the nonzero count and can preserve most task quality after recovery.

Formalizing this process, let $\mathbf{W} \in \mathbb{R}^{m \times n}$ represent a weight matrix in a given layer. Pruning removes a subset of weights by applying a binary mask $\mathbf{M} \in \{0,1\}^{m \times n}$, yielding a pruned weight matrix:

$$\hat{\mathbf{W}} = \mathbf{M} \odot \mathbf{W}$$

where $\odot$ represents the element-wise Hadamard product. The mask $\mathbf{M}$ is constructed based on a pruning criterion, typically weight magnitude. A common approach is magnitude-based pruning, which removes a fraction ρ_{sparse} of the lowest-magnitude weights by defining a threshold δ_{prune} such that:

$$M_{i,j} = \begin{cases} 1, & \text{if } |W_{i,j}| > \delta_{\text{prune}} \\ 0, & \text{otherwise} \end{cases}$$

where δ_{prune} is chosen to ensure that only the largest $(1 - \rho_{\text{sparse}})$ fraction of weights remain. This method assumes that larger-magnitude weights contribute more to the network's function, making them preferable for retention.

Unstructured pruning reduces the count of nonzero parameters, but translating that reduction into physical storage savings requires an indexed sparse encoding such as Compressed Sparse Row (CSR) or coordinate list (COO). Because these formats must store positional metadata (such as column indices and row pointers) alongside the nonzero values, metadata overhead offsets savings at low sparsity. For example, storing a 16-bit floating-point weight with a 16-bit column index doubles the per-element byte footprint, requiring over 50 percent sparsity just to break even on artifact size (D_{vol}). Storing a dense tensor with zeroed entries saves zero memory. The wearable anchor that opened this section shows the consequence. Its surviving weights total 2 MB only if they cost nothing to locate. A 16-bit index beside each value adds half again to 32-bit values and doubles 16-bit ones, so the stored artifact misses the 2 MB flash budget. A denser position encoding, such as a bitmap, reduces the metadata overhead but cannot fit unchanged values that already consume the entire budget. Fitting requires further pruning or lower-precision values with room reserved for metadata, or structured removal of whole channels so that the remaining tensors can be stored without per-value indices.

↪ The memory footprint and index-pointer layouts for compressed sparse row (CSR) representations are developed in section C.1.5.

Nor does unstructured sparsity translate directly to inference acceleration on modern hardware. Commodity accelerators achieve peak throughput (R_{peak}) through dense systolic arrays and wide SIMD execution units that rely on *memory coalescing*. Memory transfers use device-dependent transaction granularity, and caches store data in *cache lines*. Coalescing nearby, aligned accesses lets parallel threads share transactions more efficiently. A dense masked tensor still moves its zero entries. An indexed sparse encoding instead packs nonzero values, but Sparse Matrix-Dense Matrix Multiplication (SpMM) kernels must decode metadata and gather the matching activations. Irregular activation accesses, load imbalance, and kernel overhead can offset the skipped arithmetic even when stored weight values are contiguous. The performance break-even point depends on shape, format, batch size, and the available sparse kernel, so artifact reduction alone does not establish a speedup.

10.3.1.3 Structured pruning

Where unstructured pruning removes individual weights across arbitrary tensor coordinates, **structured pruning** (Li et al. 2017) eliminates regular groups of parameters: channels, filters, attention heads, or entire layers. Because removing whole structural units contracts weight arrays into smaller, contiguous dense tensors, the remaining computation executes directly on standard Basic Linear Algebra Subprograms (BLAS) and convolution engines without specialized sparse libraries or metadata indexing overhead.

Between unconstrained irregular masks and coarse-grained channel excision lies hardware-aligned fine-grained regularity. Hardware-aware pruning strategies, such as *N:M structured sparsity*,[6] enforce fine-grained patterns that map directly onto accelerator execution datapaths. Under a *2:4 structured sparsity* constraint, every contiguous block of four weights must contain exactly two zeros and two nonzero values:

$$\text{Original block (4 values):} \quad [0.0, 1.4, 0.0, -2.1] \longrightarrow \text{Compressed storage:} \quad [1.4, -2.1] \quad \text{with 2-bit indices: } [01, 11]$$

Because every group of four weights compresses to exactly two values, the value payload halves, and a 2-bit index per kept value adds only a small, fixed metadata cost with no structural irregularity. Dedicated hardware units (such as sparse Tensor Cores) use the compact 2-bit index metadata to drive on-chip multiplexers, routing the nonzero weights directly into multiplier units alongside matching activations. Memory transactions remain aligned and coalesced, cutting weight traffic nearly in half while doubling peak math throughput. This chapter uses the 2:4 pattern as the canonical compression example.

Figure 10.4 contrasts this regularity-compression trade-off across three architectural granularities: irregular connection masking in the left panel versus coarse-grained neuron dropping and full convolutional channel excision in the structured right panel.

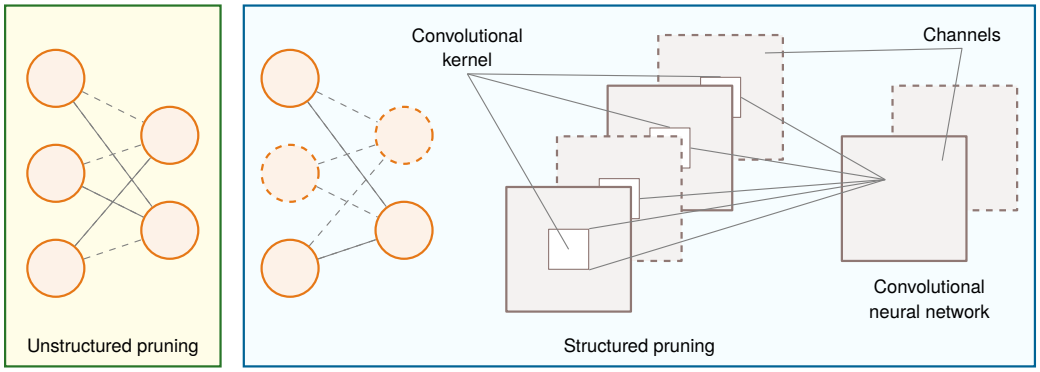


Figure 10.4: Unstructured vs. Structured Pruning: The regularity-compression trade-off across pruning granularities. Unstructured pruning (left) removes arbitrary individual connections and requires a sparse representation and matching kernel to change execution. Structured pruning of neurons (middle) or convolutional channels (right) can produce smaller dense tensors that remain compatible with standard kernels, although realized latency depends on the resulting shapes and runtime. Source: (Qi et al. 2021).

Across the structured pruning regimes in figure 10.4, removing whole units contracts the underlying matrices into smaller, contiguous dense tensors, eliminating the sparse metadata and memory divergence penalties of unstructured sparsity. In fully connected layers (center), pruning neurons eliminates corresponding rows and columns in adjacent weight matrices. In convolutional layers (right), pruning channels removes complete 3D filter slices. Because the resulting subnetwork consists entirely of dense operations, it executes directly on standard BLAS and convolution engines without specialized sparse libraries. The primary systems requirement is matching the resulting tensor shapes and layouts to efficient kernels, the tile-filling condition illustrated above.

A common approach to structured pruning ranks entire neurons or filters by the magnitude of their associated weights. A low group norm is an inexpensive proxy for low importance, not proof that the unit is dispensable. The score can use an ℓ_1-norm or ℓ_2-norm over the weights associated with each unit; groups below a selected threshold become pruning candidates. Layer-wise ranking matters because raw scales can differ across layers. The method needs no representative data pass, but computing scores, choosing a threshold, materializing the smaller graph, and validating or fine-tuning the result still incur engineering and compute cost.

Another strategy is **activation-based pruning**, which evaluates neuron or filter activations over a dataset. Units that remain weak across representative inputs become candidates for removal, subject to ablation or fine-tuning checks. This method captures input-dependent behavior rather than relying only on static weights. Its ranking can still miss rare but important inputs, depend on

[6] **N:M structured sparsity**: Introduced commercially with NVIDIA's A100 GPU (2020), the 2:4 pattern was chosen because it halves multiply-accumulate operations while keeping position metadata small enough for the sparse Tensor Core path (NVIDIA Corporation 2020a; Choquette et al. 2021). This fixed ratio is a hardware constraint, not a mathematical optimum: the A100 Sparse Tensor Core path accelerates 2:4 sparse operands, yielding up to 2× math-throughput speedup over dense execution when kernels and layouts satisfy the constraint. Other ratios are not accelerated by this specific hardware path, illustrating how silicon design constrains which sparsity patterns translate to actual speedup.

↪ How sparse Tensor Cores use the 2-bit metadata to select matching activations and skip the zeroed multiplies is developed in section 11.4.4.

activation scaling, or change after upstream units are removed. Activation-based pruning therefore requires representative profiling data and validation after the selected structures are removed.

Gradient-based pruning uses loss derivatives to estimate how removing a neuron or filter would change the objective. A small first-order score suggests a candidate at the current checkpoint and on the sampled data; it does not establish permanent irrelevance or capture every interaction among units. Gradient-based criteria can combine a unit's value with its gradient to rank the expected local loss change. Mathematically, this criterion approximates the change in loss $\Delta\mathcal{L}$ when zeroing parameter w_i via a first-order Taylor expansion: $\Delta\mathcal{L} \approx |w_i \cdot \frac{\partial \mathcal{L}}{\partial w_i}|$. Weights that are either near zero or have negligible gradients exert minimal influence on the objective, making them prime candidates for excision. Unlike weight-only magnitude criteria, they require backward computation and are usually integrated into training or fine-tuning. The pruned result still needs task-level validation because the score is a local approximation.

These criteria trade measurement cost against fidelity to model behavior. Magnitude scores are inexpensive and provide a useful baseline, but they ignore the input distribution. Activation scores incorporate representative inputs and expose units that are quiet on that workload, while gradient-based scores also incorporate the local loss surface at additional training cost. None dominates across models, and none establishes that removal is harmless. A sound comparison prunes to the same structural target, applies the same recovery budget, and measures task quality after removal. It then profiles the exported graph, because a better importance score can preserve quality without producing shapes that the target runtime executes efficiently. The choice therefore depends on available data and compute, the structure being removed, and the deployment's quality margin.

10.3.1.4 Dynamic pruning

Traditional pruning methods, whether unstructured or structured, produce a **static pruning** mask or subnetwork that remains fixed during deployment, even if the mask was learned gradually during training. **Dynamic pruning** instead decides at inference time, from each input, which computation is active, so the executed subnetwork changes from one input to the next.

Dynamic pruning can use runtime sparsity techniques in which the model selects parameters or structures from input characteristics. Activation-conditioned pruning, for example, deactivates neurons or channels for particular inputs (Hu et al. 2023). The resulting input-dependent sparsity reduces executed work only when the controller and runtime can skip that work more cheaply than a dense path would execute it.

For instance, consider a convolutional neural network processing images with varying complexity. During inference on a simple image containing mostly uniform regions, some convolutional filters may produce negligible activations. A dynamic policy can identify candidate filters and temporarily exclude them from computation. That exclusion reduces nominal work, but the controller, irregular execution, and any quality change remain part of the result. The method becomes useful in a latency-sensitive application only when variation among inputs is large enough to repay those costs. Evaluating such efficiency gains requires measuring both latency and accuracy on the same target workload across the full input distribution.

↪ Joint evaluation of compressed models on accuracy and target efficiency metrics such as latency, plotted as an efficiency-quality frontier, is measured in section 12.11.1.4.

Methods that change the mask during training are sometimes also called dynamic, but they solve a different problem. Gradual magnitude pruning starts with a dense network and progressively increases the fraction of pruned parameters while fine-tuning the surviving weights, which is the iterative schedule that section 10.3.1.6 develops, run inside training. Dynamic sparse training variants additionally regrow connections that prove important later in training. Both end in a static mask for deployment, so this chapter reserves *dynamic pruning* for input-dependent execution.

Dynamic pruning can allocate different amounts of work to different inputs and can use, for a hard input, capacity that a static deployment mask would remove permanently. Those properties create an opportunity, not a guaranteed efficiency or accuracy gain. A controller sits on the critical path, input-dependent routing fragments batch execution across SIMD vector lanes, and the enlarged behavior space requires more validation. Training may also need routing losses or regularization so that the policy actually uses its cheaper paths. Production deployments must monitor route frequency, latency tails, and quality by route. Dynamic pruning is therefore a candidate when input complexity varies enough to repay the controller and batching overhead, not a default substitute for static pruning.

↪ Infrastructure telemetry that tracks latency distributions in deployed systems, alongside the model-quality signals production monitoring pairs with it, is operationalized in section 14.6.1.

10.3.1.5 Pruning trade-offs

The three pruning approaches occupy different positions on the regularity-versus-granularity trade-off. Unstructured pruning can remove individual weights and therefore express fine-grained sparsity, but accelerators need matching sparse formats and kernels to skip those zeros. Structured pruning removes channels, filters, or layers and can produce smaller dense operations that conventional hardware already supports. Dynamic pruning makes the executed structure input-dependent, adding control and batching overhead in exchange for conditional allocation. Table 10.4 summarizes these deployment distinctions.

These categories describe deployed execution rather than mutually exclusive training algorithms. A structured subnetwork may be learned with a dynamic schedule and then frozen for serving; classify the artifact by what its runtime actually executes.

Table 10.4: Pruning Strategies: Unstructured pruning removes individual weights but needs sparse storage and matching kernels for deployment gains; structured pruning removes whole components that map more directly to dense hardware, while dynamic pruning adds runtime adaptation and overhead.

Aspect	Unstructured Pruning	Structured Pruning	Dynamic Pruning
Removed unit	Individual weights in the model	Entire neurons, channels, filters, or layers	Adjusts pruning based on runtime conditions
Model structure	Sparse weight matrices; original architecture remains unchanged	Model architecture is modified; pruned layers are fully removed	Structure adapts dynamically
Impact on memory	Reduces the nonzero count; storage falls only with an appropriate sparse representation	Reduces model storage by removing entire components	Varies based on real-time pruning
Impact on computation	Dense work remains unless a supported sparse path skips zeros	Reduces nominal FLOPs; latency depends on resulting shapes and kernels	Varies with routing policy and controller overhead
Hardware compatibility	Requires sparse formats and matching execution support	Produces dense subnetworks, but awkward shapes can still underutilize hardware	Requires adaptive inference engines
Fine-tuning requirement	Often used to recover accuracy after pruning	Often used after structural modification	Depends on how the routing policy and subnetworks are trained
Use cases	Targets whose runtime provides sparse formats and kernels, or storage-bound high sparsity	Real-time inference optimization, mobile/edge AI, and efficient training	Adaptive AI applications, real-time systems

10.3.1.6 Pruning schedules

The pruning schedule affects how much recovery training the network receives after capacity is removed. Two primary scheduling strategies govern weight removal: iterative pruning and one-shot pruning.

Iterative pruning Figure 10.5 sequences this gradual adaptation across three successive prune-and-retrain cycles, tracking the transient accuracy drop following each structural cut and its subsequent recovery during fine-tuning.

The illustrative three-row workflow in figure 10.5 shows recovery between successive cuts on a convolutional network pruned by six channels. Rather than removing all six channels in a single pass, the pipeline removes two channels per cycle across three iterations, fine-tuning the remaining parameters after each cut. The initial cut drops accuracy from 0.995 to 0.971, but brief fine-tuning restores performance to 0.992. Two subsequent prune-and-tune cycles produce a final model at 0.991 accuracy—a 0.4 percent degradation despite a 27 percent reduction in channel count. Staging structural surgery across multiple retraining intervals allows surviving weights to adjust their representations along the loss landscape.

One-shot pruning **One-shot pruning** removes multiple architectural components in a single step, followed by an extensive fine-tuning phase to recover model accuracy. This aggressive approach compresses the model quickly but risks greater accuracy degradation, as the network must adapt to significant structural changes simultaneously.

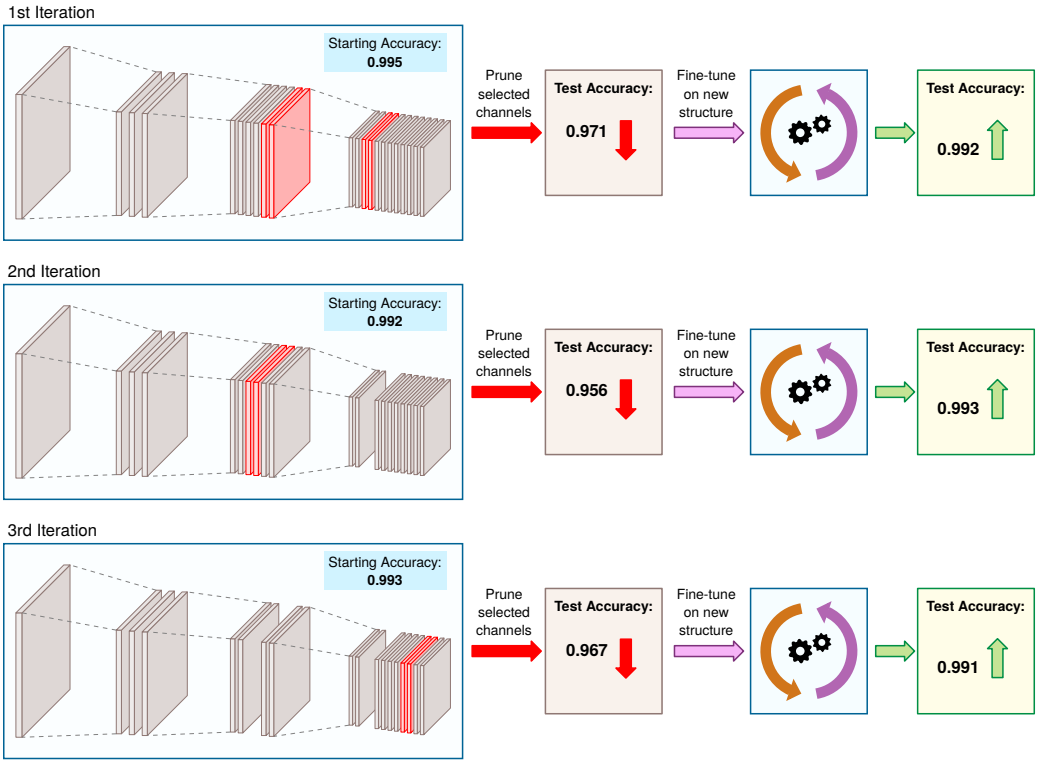


Figure 10.5: Illustrative Iterative Pruning Performance: Three rows depict successive prune-then-fine-tune cycles, each removing two of the original twenty-two channels. Accuracy drops from 0.995 to 0.971 after the first prune, recovers to 0.992 after fine-tuning, and settles at 0.991 after all three cycles, a 0.4 percent loss with 27 percent fewer channels.

Applying one-shot pruning to the same network illustrates the hazard of aggressive structural truncation. Figure 10.6 traces this single-pass penalty. Excising all six highlighted channels simultaneously precipitates a sudden accuracy plunge from 0.995 to 0.914, from which fine-tuning recovers only partially to 0.943, leaving about a 5 percent quality penalty under the recovery procedure shown. The two figures illustrate a mechanism, not a controlled result. Both runs end with the same tensor geometry, but the iterative run also received three rounds of fine-tuning, so its better final accuracy mixes the benefit of smaller cuts with the benefit of more recovery training. A fair comparison holds the final structure and the total recovery budget fixed.

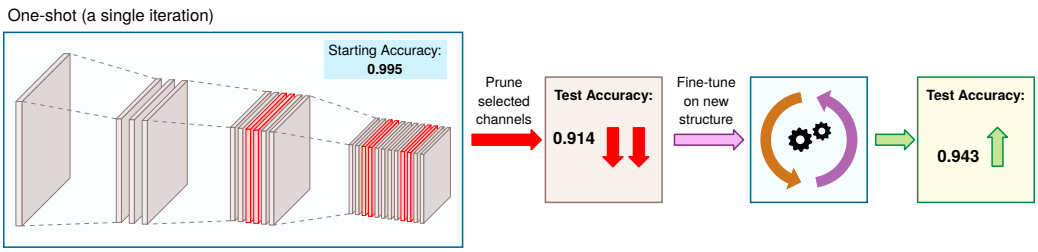


Figure 10.6: Illustrative One-Shot Pruning Impact: All six channels (27 percent) are removed simultaneously, causing accuracy to drop from 0.995 to 0.914. Fine-tuning recovers only to 0.943, a 5 percent degradation from the original accuracy.

The working rule follows from that mechanism. Start with one-shot pruning plus recovery, because it is cheapest, and accept it if the recovered model meets the quality target. Move to an iterative schedule when it does not, which is most likely at high sparsity, where a single large cut removes capacity the network cannot rebuild in one recovery phase. Compare the two at a matched recovery

budget, and spend the extra cycles only when the target accelerator can exploit the sparsity pattern that results.

10.3.1.7 Lottery ticket hypothesis (optional depth)

Standard pruning workflows assume a fully trained dense model from which weights are subsequently removed. Yet the relationship between network structure and optimization dynamics runs deeper: pruning can reveal inherently efficient, highly trainable subnetworks embedded within the initial random parameterization.

This perspective leads to the Lottery Ticket Hypothesis[7] (LTH), which challenges conventional pruning workflows by proposing that within large neural networks, there exist small, well-initialized subnetworks ("winning tickets") that can achieve comparable accuracy to the full model when trained in isolation. Rather than viewing pruning as a post-training compression step, LTH suggests it can serve as a discovery mechanism to identify these efficient subnetworks early in training (Rachwan et al. 2022).

Figure 10.7 diagrams this iterative pruning-and-rewinding loop: after training to convergence and pruning low-magnitude connections, surviving weights reset to their initialization state θ_0 to isolate the sparse subnetwork highlighted on the right. Repeating the cycle can isolate a sparse subnetwork that, in the reported settings, trains to accuracy comparable with the original network (Frankle and Carbin 2019). Whether the same procedure succeeds depends on architecture, scale, optimizer, and training budget.

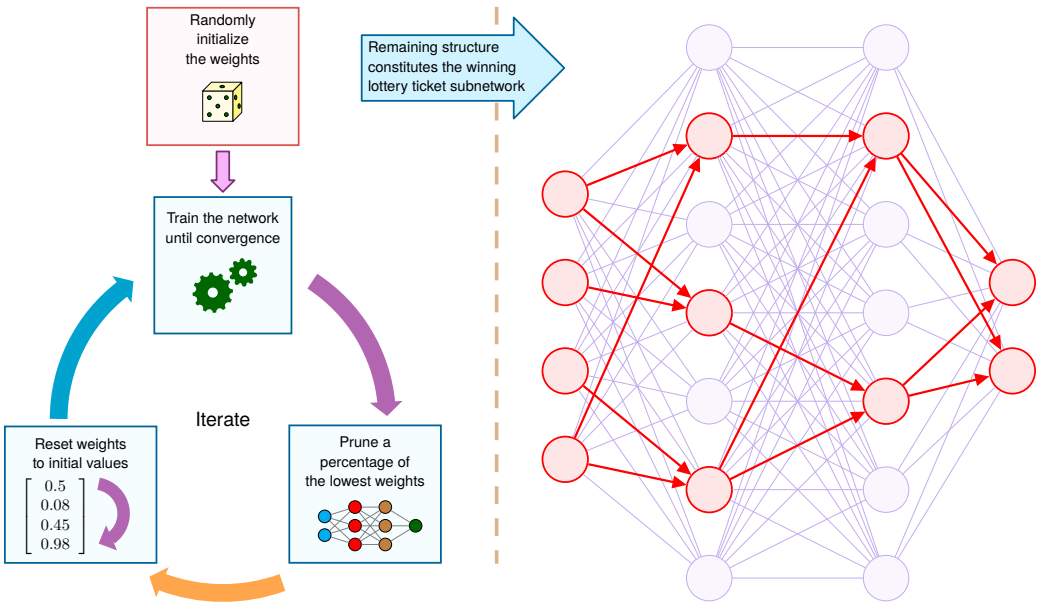


Figure 10.7: Lottery Ticket Iteration Cycle: The original procedure repeatedly trains, prunes low-magnitude weights, and resets surviving weights to their initial state θ_0. The cycle tests whether the resulting sparse subnetwork can match the dense baseline when trained in isolation; the outcome is empirical rather than guaranteed.

The Lottery Ticket Hypothesis suggests that compact, trainable subnetworks may exist inside an overparameterized initialization, which motivates direct sparse-training research. The standard discovery procedure, however, does not eliminate dense training cost. It first pays for one or more dense or partially dense runs, plus a rewind and a recovery phase per pruning cycle, to find the ticket. A winning ticket is therefore a training result. It becomes a systems win only when that total accelerator time, the final artifact size, target latency, and task quality beat the simpler alternatives: training a smaller dense model from the start, one-shot pruning with recovery, and direct sparse training.

[7] **Lottery ticket hypothesis**: Named for the intuition that training a large network is like buying many lottery tickets–most lose, but a few "winning tickets" (sparse subnetworks with favorable initializations) can train to comparable accuracy on their own. Frankle and Carbin (2019) established the hypothesis on smaller vision and fully connected networks; later work surveyed and extended the idea to larger settings (Rachwan et al. 2022). The systems implication is that some of the memory and compute spent training dense networks may be discoverable overhead, but the practical payoff depends on whether the winning subnetwork can be found before paying most of the original training cost.

10.3.1.8 Pruning in practice

While the Lottery Ticket Hypothesis illuminates the optimization landscape, practical serving systems care only about executable artifacts. Framework-level pruning is useful only when it materializes concrete hardware savings: a smaller dense tensor, a hardware-aligned structured sparse matrix, or an indexed sparse layout paired with matching runtime kernels. Training-time pruning often begins as mask application: the original tensor remains resident in memory, while a binary mask zeros selected weights during forward propagation. That mechanism guides gradient flow during recovery, but it delivers zero memory reduction or latency improvement on its own. Measurable efficiency gains emerge only after the masked structure is exported into the physical artifact loaded by the inference engine.

This artifact boundary separates the pruning strategies. Unstructured pruning removes individual weights and needs sparse kernels plus an appropriate storage format to translate zeros into speed. Structured pruning removes whole channels, heads, neurons, or blocks, which can reshape tensors into smaller dense operations that ordinary accelerators already execute well. Gradual pruning during fine-tuning adds a training schedule: sparsity increases over time so the remaining weights can recover accuracy as capacity is removed. The systems audit is therefore concrete: identify what is pruned, identify the runtime format, and verify that the target hardware has kernels that make the sparsity useful.

These trade-offs become concrete when examining real-world deployments. Some model families reduce deployment cost through architecture rather than post-hoc pruning: MobileNet uses depthwise separable convolutions for mobile and embedded vision (Howard et al. 2017), while EfficientNet uses compound scaling to improve the accuracy-efficiency trade-off under resource constraints (Tan and Le 2019). Pruning remains a separate optimization lever. BERT-style transformers[8] have been pruned by removing redundant attention heads or intermediate dimensions, while separate distillation methods such as DistilBERT and TinyBERT train smaller dense student models that retain much of BERT's performance (Sanh et al. 2019; Jiao et al. 2020).

Pruning has an inherent architectural limitation: it can only carve away pieces of an existing graph. If the original operator choices, kernel dimensions, or memory access patterns are ill-suited to the target accelerator, pruning alone cannot replace them with more efficient primitives. When deployment limits demand an entirely different topology, the compression objective shifts from subtracting weights to supervising an independently architected, compact model with the behavior of the original.

10.3.2 Knowledge distillation

Consider deploying a frontier language or vision model within the memory and latency bounds of an edge workstation or on-premise accelerator. Retaining the original topology through pruning may still leave unsupported operations or excessive activation volumes. Knowledge distillation resolves this constraint by training a compact *student* network to reproduce the output behavior and representations of the overparameterized *teacher* (Hinton et al. 2015; Sanh et al. 2019). While the term borrows from chemical purification,[9] the systems mechanism is algorithmic: distillation transfers dark knowledge across architectures, amortizing a one-time training phase across millions of lower-latency inference queries.

Definition 10.3: Knowledge distillation

Knowledge distillation is a model-compression technique that trains a smaller *student* model to match the behavior of a larger, pretrained *teacher* model.

1. **Significance**: Distillation moves cost from repeated inference to an additional training phase. A student can be substantially smaller than its teacher while retaining useful task performance; DistilBERT, for example, reports 40 percent fewer parameters, 60 percent faster inference, and 97 percent of BERT's language-understanding performance (Sanh et al. 2019). This trade-off is valuable when the student-training cost is amortized across many deployed queries.

↪ Benchmark protocols that separate structured from unstructured pruning and specify the hardware and sparse-kernel support behind any measured speedup are operationalized in section 12.11.1.3.

[8] **BERT pruning**: BERT's 12 attention heads per layer can contain substantial redundancy—Michel et al. (2019) found on MultiNLI that many heads could be removed without a statistically significant performance change. The removable fraction depends on the layer and task, so deployment pruning must measure both task quality and whether the resulting structure reduces runtime work.

[9] **Distillation**: The name borrows from the chemical process of separating a mixture. In model compression, the student learns from a teacher's softened output distribution or internal representations rather than copying its parameters. Soft targets can encode relationships among classes, but the student learns only what its objective and training data expose. The metaphor is useful but not literal: the temperature parameter comes from softmax scaling, and the size-quality trade-off depends on the student, data, objective, and training procedure (Hinton et al. 2015). Deployment savings come from the student architecture, not from the metaphor itself.

2. **Distinction**: Unlike pruning, which removes parameters from an existing architecture, and quantization, which lowers numerical precision, distillation trains a new dense architecture. The student inherits behavior from the teacher's output distribution or intermediate representations rather than inheriting the teacher's full parameter count or layer structure.
3. **Common pitfall**: A frequent misconception is that distillation is lossless compression. In reality, the student is bounded by its own capacity, the quality of the teacher, and the match between the distillation data and deployment distribution; a student can faithfully reproduce teacher errors as well as teacher knowledge.

Section 9.10.4 introduced the teacher's soft targets as a source of training data; here they become a compression mechanism. Teacher models transfer semantic knowledge through nonzero probabilities assigned across non-target classes. Whereas a one-hot ground-truth label assigns a probability of one exclusively to the correct class, the probability distribution charted in figure 10.8 keeps the inter-class correlations that data selection called dark knowledge, revealing that an input classified as a cat (85 percent) shares subtle semantic features with a dog (10 percent) and a fox (5 percent).

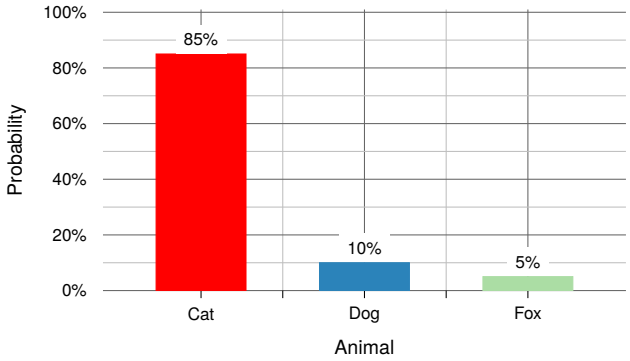


Figure 10.8: Soft Target Distribution: The teacher's relative confidence levels indicate which classes are semantically similar (for example, cat vs. dog), providing a much richer supervision signal than a binary "correct" label.

Training a student model requires balancing teacher imitation against ground-truth task supervision. Operationally, the workflow requires four decisions before training begins: selecting a teacher with the desired behavior, designing a student whose dense architecture satisfies the target accelerator's memory and compute budget, generating soft targets over the training corpus, and choosing the temperature and loss weights that govern optimization. Figure 10.9 traces this dual-path distillation pipeline: an input sample x passes concurrently through the teacher and student networks, routing temperature-scaled soft predictions ($T_{\text{distill}} > 1$) into the distillation loss while unscaled student predictions ($T_{\text{distill}} = 1$) evaluate against ground-truth hard labels.

Deploying the distilled student requires verifying more than top-1 accuracy. Because a student can inherit the teacher's systematic biases and miscalibrations as readily as its useful representations, verification must measure calibration error, subgroup behavior, and executed latency on the target hardware.

10.3.2.1 Distillation mathematics

Standard softmax normalization converts raw model logits z_i into probabilities. Introducing a **temperature parameter**[10] T_{distill} softens the resulting probability distribution across output classes:

$$p_i^{(T_{\text{distill}})} = \frac{\exp(z_i / T_{\text{distill}})}{\sum_j \exp(z_j / T_{\text{distill}})}$$

A higher T_{distill} flattens the softmax distribution and exposes more of the dark knowledge carried in relative class probabilities, such as a teacher assigning a small but nonzero probability to dog and

[10] **Temperature (softmax)**: The term comes from the Boltzmann distribution in statistical mechanics. Dividing logits by a temperature above one produces a softer distribution and can expose relative probabilities among non-target classes. At T_{distill}=1, the expression is standard softmax; increasing the temperature reduces logit gaps but does not guarantee that the revealed probabilities help the student. The useful amount of softening depends on logit scale, teacher calibration, student capacity, loss weighting, and the accompanying T^2_{distill} factor. Temperature is therefore selected empirically rather than fixed to a universal range.

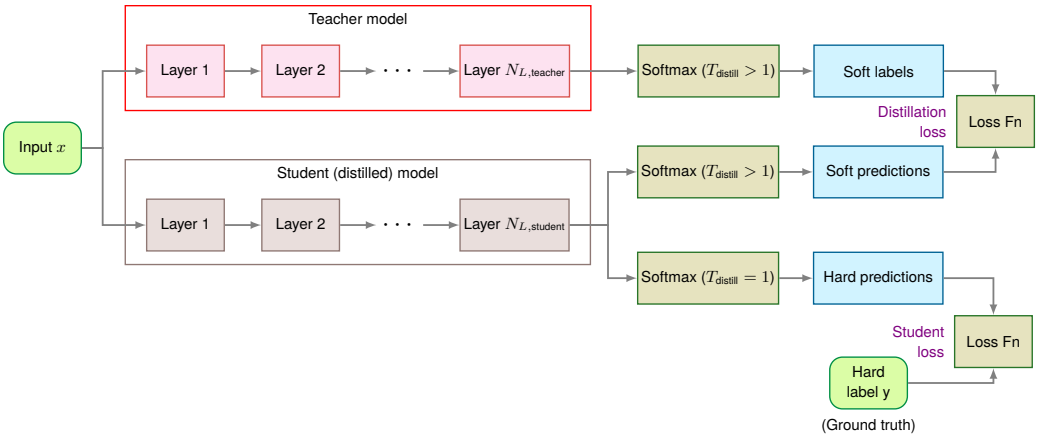


Figure 10.9: Knowledge Distillation Workflow: An input sample passes through both the teacher and the student network. The teacher produces soft labels via temperature-scaled softmax, while the student output is compared against both the soft labels (distillation loss) and the hard labels (student loss).

near-zero probability to truck when classifying an image of a cat. The total distillation loss $\mathcal{L}_{\text{distill}}$ balances standard cross-entropy with the Kullback-Leibler (KL) divergence against the teacher's softened distribution:[11]

$$\mathcal{L}_{\text{distill}} = (1-\gamma_{\text{KD}})\mathcal{L}_{\text{CE}}(\mathbf{p}_{\text{student}}, \mathbf{y}) + \gamma_{\text{KD}} T^2_{\text{distill}} \mathcal{D}_{\text{KL}}(\mathbf{p}^{(T_{\text{distill}})}_{\text{teacher}} \| \mathbf{p}^{(T_{\text{distill}})}_{\text{student}})$$

Here $\mathbf{p}^{(T_{\text{distill}})}_{\text{teacher}}$ and $\mathbf{p}^{(T_{\text{distill}})}_{\text{student}}$ are the probability vectors computed at temperature T_{distill}, $\mathbf{y}$ is the one-hot ground-truth label vector, and $\gamma_{\text{KD}} \in [0,1]$ balances the hard-label and soft-target objectives. The scaling factor T^2_{distill} ensures that the gradient magnitude of the distillation loss remains commensurate with the hard cross-entropy loss: dividing logits by T_{distill} scales the derivative of the softmax probabilities by $1/T_{\text{distill}}$, and the gradient of the KL divergence with respect to the student's logits scales as $1/T^2_{\text{distill}}$ in the high-temperature limit. Multiplying by T^2_{distill} prevents the teacher's supervisory gradient from vanishing relative to the hard-label loss as T_{distill} increases. This formulation defines response-based distillation, matching final output distributions. Modern pipelines also employ feature-based distillation, training intermediate student layers to directly match the teacher's hidden representations or attention maps using mean squared error or cosine distance to guide gradient updates across deep backbones.

[11] **KL divergence**: Introduced by Solomon Kullback and Richard Leibler in 1951, $\mathcal{D}_{\text{KL}}(p\|q)$ quantifies the extra bits needed to encode samples from distribution p using a code optimized for q. The key asymmetric consequence: $\mathcal{D}_{\text{KL}}(\text{teacher}\|\text{student})$ penalizes the student heavily for assigning zero probability to teacher-probable outputs, forcing the student to maintain broad coverage of the teacher's distribution, including low-probability "soft labels" that carry the teacher's learned uncertainty. Distillation can affect calibration, which must be evaluated separately.

Efficiency gains and trade-offs Distillation's primary deployment advantage over unstructured pruning is that it produces a smaller dense model. Standard accelerators achieve peak throughput on contiguous matrix multiplications; unstructured sparse models incur irregular memory access and index decoding overheads that often negate theoretical FLOP reductions unless sparsity is extreme. A compact dense student maintains coalesced memory access and executes vendor-optimized dense GEMM kernels. However, parameter reduction alone does not guarantee lower latency: operational intensity, layer depth, attention sequence length, and memory bandwidth limits still govern runtime. DistilBERT, for example, is about 60 percent faster (roughly 1.6×) primarily because it removes half of BERT's transformer layers (Sanh et al. 2019). The gap from the ideal 2× reflects work that halving the layers does not remove, such as the embedding layer, the task head, and fixed per-request overheads.

Distillation composes with pruning and quantization in multi-stage compression pipelines, and the order matters. Quantization usually comes last, because its calibration must see the weight and activation distributions of the final structure. Between the two structural steps, either order is used. Distilling first and then pruning the student trims a model that already fits the target architecture; pruning first and then distilling makes the pruned network itself the student, with the original dense model as its teacher, so distillation doubles as the recovery training after the cut. Section 10.7 works through the second order on BERT. Structural compression can also happen earlier than fine-tuning:

Gordon et al. (2020) show that BERT pruned during pretraining transfers to downstream tasks, so the structural step need not wait for a task-specific model.

These operational gains trade inference efficiency for substantial training overhead. Distillation requires running inference across the teacher model to generate soft targets and backpropagating loss through the student across millions of tokens. For large models, this auxiliary training phase consumes significant accelerator FLOPs and memory bandwidth, whereas post-training quantization, introduced later in this chapter, needs only a pass over a small calibration set. Compression effectiveness is also bounded by student capacity and teacher fidelity: an undersized student lacks the capacity to absorb the teacher's distribution, while an inaccurate or uncalibrated teacher imparts its own systematic errors to the student. Selecting an optimization strategy requires evaluating task accuracy, parameter footprint, serving latency, and total training cost together.

Table 10.5 contrasts the key trade-offs between knowledge distillation and pruning across accuracy retention, training cost, inference speed, hardware compatibility, and implementation complexity. DistilBERT, discussed earlier, is the dense-student case: its gains come from a smaller architecture trained by distillation, not from sparsity.

Table 10.5: Distillation vs. Pruning: Distillation spends additional training and student-design effort to produce a smaller dense model; pruning modifies an existing model, but sparse results need compatible storage and kernels to realize deployment gains.

Criterion	Knowledge Distillation	Pruning
Accuracy retention	Model- and student-dependent; often strong with a capable teacher	Model- and sparsity-dependent
Training cost	Higher – Requires teacher inference and student training	Often lower – Requires pruning and usually fine-tuning
Inference speed	Often favorable for a smaller dense student	Depends – Structured pruning is efficient, unstructured needs special support
Hardware compatibility	High – Works on standard accelerators	Limited – Sparse models may need specialized execution
Implementation effort	More involved: teacher inference, student objective, and training	Varies: choose a criterion, prune, fine-tune, and export

While pruning removes parameters and distillation trains replacement topologies, neither exploits the algebraic correlation present within dense weight arrays. When high-dimensional weight matrices exhibit low effective rank, factorizing them into low-dimensional products reduces parameter count and memory traffic without requiring sparse execution runtimes or architectural redesign.

10.3.3 Structured approximations

Weight matrices in deep networks often exhibit rapid singular-value decay: a small fraction of principal components captures most of the variance in the linear transformation. Rather than materializing and fetching full $M \times N$ dense matrices from DRAM, **structured approximation** decomposes weight tensors into products of low-rank matrices or tensor factors. For an illustrative 4096×4096 weight block, a rank-128 factorization requires only 6.25 percent of the original scalar parameters. Low-rank matrix factorization and multi-linear tensor decompositions translate this algebraic redundancy into reduced memory traffic and lower arithmetic cost.

10.3.3.1 Low-rank factorization

Low-rank matrix factorization (LRMF) approximates weight matrices with lower-rank representations. Given a weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$, LRMF identifies factor matrices $\mathbf{U} \in \mathbb{R}^{m \times k}$ and $\mathbf{V} \in \mathbb{R}^{n \times k}$ such that:

$$\mathbf{W} \approx \mathbf{U}\mathbf{V}^T$$

where the approximation rank $k \ll \min(m, n)$. This decomposition is typically computed via singular value decomposition (SVD),[12] retaining only the k largest singular values. Writing the truncated decomposition as $\mathbf{W} \approx \mathbf{P}_{kk}\mathbf{Q}_k^T$ and splitting the singular values evenly between the two factors gives $\mathbf{U} = \mathbf{P}_{kk}^{1/2}$ and $\mathbf{V} = \mathbf{Q}_{kk}^{1/2}$. During inference, computing the layer output as $\mathbf{h} = \mathbf{U}(\mathbf{V}^T\mathbf{x})$

[12] **Singular value decomposition (SVD)**: The Eckart-Young theorem (1936) proves that retaining the top k singular values yields an optimal rank-k approximation in the Frobenius and spectral norms. The key systems trade-off is whether the high, one-time compute cost of this factorization—$\mathcal{O}(mn \cdot \min(m, n))$—is amortized by the memory and bandwidth savings from using the smaller model in repeated inference calls.

replaces a single large matrix-vector product with two sequential, smaller multiplications. The primary systems question is whether the reduction in parameter footprint and arithmetic operations compensates for the overhead of staging an intermediate activation and dispatching an additional kernel.

> **Systems Perspective 10.2: The bandwidth-compute trade-off**
>
> Low-rank factorization can reduce both arithmetic and bandwidth demand when the retained rank is small, though it adds a second factor operation and possible kernel-launch overhead. Storing a 4096 by 4096 matrix requires 67.1 MB (at FP32). Fetching this matrix for a single inference imposes substantial memory traffic, especially when execution is limited by physical memory bandwidth.
>
> Factoring the matrix at rank $k = 128$ requires storing two smaller matrices (4096 by 128 and 128 by 4096), totaling only 4.2 MB—a 16× reduction in stored values. When inference uses the two factors directly, matrix-vector arithmetic also falls from $\mathcal{O}(mn)$ to $\mathcal{O}(k(m+n))$ for small k. The deployment speeds up only if the two factor operations and their intermediate traffic cost less than the original dense kernel. Explicitly materializing $\mathbf{U}\mathbf{V}^T$ would add $\mathcal{O}(mkn)$ work and defeat the purpose.

This bandwidth-compute trade-off exemplifies the memory wall at the operator level: when batch sizes are small, execution is bottlenecked by the time required to fetch weight bytes from DRAM rather than the time required to perform multiply-accumulate operations. In production serving engines, evaluating the factored representation avoids full-rank materialization entirely. Figure 10.10 diagrams this low-rank decomposition, factoring the original $m \times n$ weight matrix $\mathbf{W}$ into the product of a tall factor $\mathbf{U}$ ($m \times k$) and a wide factor $\mathbf{V}^T$ ($k \times n$).

↪ The AI memory wall, the regime in which arithmetic throughput outpaces the memory bandwidth that feeds it, is developed in section 11.5.1.

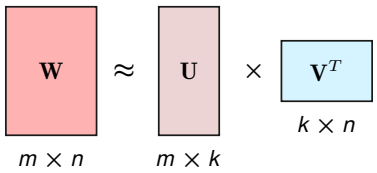


Figure 10.10: Low-Rank Factorization: A weight matrix $\mathbf{W}$ of size $m \times n$ is approximated by two smaller factors, $\mathbf{U}$ ($m \times k$) and $\mathbf{V}^T$ ($k \times n$). Storage falls from $m \times n$ to $m \times k + k \times n$ parameters, while one dense operation becomes two factor operations.

LRMF applies directly to fully connected projection layers and to flattened convolutional weight tensors. The storage and arithmetic savings derived above are only half of the accounting on a real accelerator. The intermediate activation vector $\mathbf{z} = \mathbf{V}^T\mathbf{x}$ must be staged in on-chip SRAM or round-tripped through memory, and dispatching two separate kernels incurs launch latency unless the operations are fused. Furthermore, if the approximation rank k is small or unaligned with hardware tile dimensions (such as 16×16 systolic arrays), the hardware compute units operate well below peak throughput. Selecting rank k therefore balances memory footprint and bandwidth reduction against approximation error, kernel dispatch overhead, and hardware tile utilization.

↪ How a matrix is tiled onto a fixed-size systolic array, and why a dimension just past the tile width drops utilization sharply, is developed in section 11.4.7.

10.3.3.2 Tensor decomposition (optional depth)

Tensor decomposition extends low-rank approximation to multidimensional arrays, such as four-dimensional convolutional kernels ($d_{\text{out}} \times d_{\text{in}} \times K_h \times K_w$) and multi-head attention weights. Low-rank matrix factorization could compress such a tensor only after unfolding it into a matrix, for example by merging the spatial and input-channel axes into one. Unfolding keeps every entry but discards which axis was which, so the rank limit then treats a spatial pattern and a channel pattern as interchangeable. Tensor decompositions instead keep each axis separate by representing the tensor through multilinear products of lower-dimensional factors. Figure 10.11 decomposes a three-dimensional tensor $\mathcal{Y} \in \mathbb{R}^{I \times J \times T}$ into three rank-k factor matrices ($\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$), isolating how outer products of slice vectors $\mathbf{a}_i$, $\mathbf{b}_j$, and $\mathbf{c}_t$ reconstruct entry y_{ijt}.

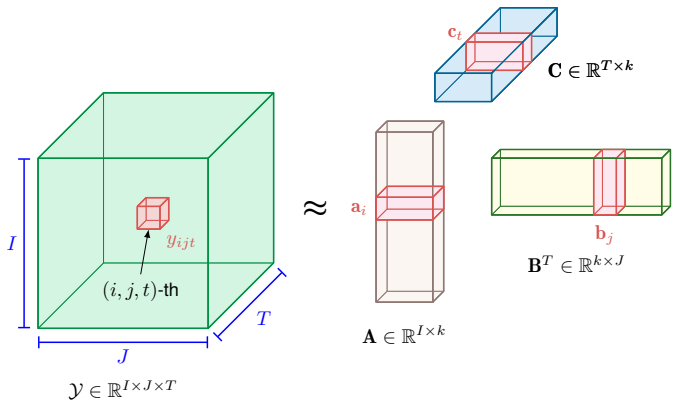


Figure 10.11: Tensor Decomposition: A three-dimensional tensor $\mathcal{Y} \in \mathbb{R}^{I \times J \times T}$ is approximated by factor matrices **A**, **B**, and **C**. The highlighted factor vectors $\mathbf{a}_i$, $\mathbf{b}_j$, and $\mathbf{c}_t$ combine across rank k to reconstruct the entry y_{ijt}. Extending low-rank approximation from two dimensions to three carries the storage and computation argument into convolutional layers. Source: (Gholami et al. 2022).

Different decomposition formats trade parameter reduction against contraction complexity. **CP decomposition** is the form drawn in figure 10.11. It expresses the tensor as a sum of k rank-one components, $\mathcal{Y} \approx \sum_{s=1}^{k} \mathbf{a}_s \otimes \mathbf{b}_s \otimes \mathbf{c}_s$, where $\mathbf{a}_s$, $\mathbf{b}_s$, and $\mathbf{c}_s$ are column s of **A**, **B**, and **C** (Lebedev et al. 2015), yielding maximal parameter reduction but often suffering from numerical instability during rank optimization. **Tucker decomposition** relaxes this constraint by retaining a small, dense core tensor multiplied along each mode by orthogonal factor matrices, $\mathcal{Y} \approx \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}$, where $\times_{n_{\text{mode}}}$ multiplies along axis n_{mode} of the tensor. **Tensor-train (TT)** decomposes a high-order tensor (one with many axes; the order is the number of axes) into a linear chain of third-order core tensors contracted along shared auxiliary indices, avoiding the exponential scaling with tensor order that limits Tucker cores.

In deep learning systems, tensor decomposition compresses 4D convolutional filters, multi-head attention projections, and large token embedding tables. However, executing tensor contractions on production accelerators encounters a severe systems barrier: modern hardware architectures are heavily specialized for dense two-dimensional matrix multiplication (GEMM). Contracting decomposed tensors requires either re-permuting memory layouts through explicit transposition kernels or issuing multiple small contractions with uncoalesced memory access patterns. Unless supported by specialized fused contraction kernels, the memory movement and kernel dispatch latency of decomposed operations can easily erase the theoretical arithmetic savings. Table 10.6 compares LRMF and tensor decomposition across data structures, compression mechanisms, and execution trade-offs.

Table 10.6: Matrix and Tensor Factorization: LRMF applies to two-dimensional matrices, while tensor decompositions extend low-rank representations to multidimensional tensors; realized storage and latency depend on rank choice and kernel support.

Feature	Low-Rank Matrix Factorization (LRMF)	Tensor Decomposition
Applicable Data Structure	Two-dimensional matrices	Multi-dimensional tensors
Compression Mechanism	Factorizes a matrix into two or more lower-rank matrices	Decomposes a tensor into multiple lower-rank components
Common Methods	SVD; Alternating Least Squares (ALS), which fits one factor at a time	CP Decomposition, Tucker Decomposition, Tensor-Train (TT)
Compute cost	Depends on the factorization method, selected rank, and kernels	Depends on the decomposition, selected ranks, optimization procedure, and tensor-contraction kernels
Storage Reduction	Reduces storage from $\mathcal{O}(mn)$ to $\mathcal{O}(mk+kn)$	Depends on the decomposition and selected ranks; stores factor matrices and, for some methods, a core tensor
Inference execution	Replaces one dense operation with two factor operations	Introduces tensor contractions whose latency depends on available kernels

Feature	Low-Rank Matrix Factorization (LRMF)	Tensor Decomposition
Primary Use Cases	Fully connected layers, embeddings, recommendation systems	Convolutional filters, attention mechanisms, multi-modal learning
Implementation Complexity	Easier to implement, often involves direct factorization methods	More complex, requiring iterative optimization and rank selection

In production pipelines, practitioners frequently combine these structured approximations, applying LRMF to large linear projections and embedding tables while targeting multi-dimensional convolutional layers with tensor decompositions. Under the D·A·M taxonomy, structured approximation alters the Algorithm to relieve Machine memory-capacity and bandwidth bottlenecks. When fetched weights dominate serving latency, reducing weight bytes can reduce that component of execution time. Added factor operations, intermediate tensors, and launch overhead must still be counted; they can offset the saving in either a memory-bound or compute-bound workload.

Pruning and factorization modify existing model parameterizations, while distillation transfers behavior into an independently trained student model. All three start from an architecture someone has already chosen, whether the original network or a hand-designed student. Hardware-aware design and neural architecture search take a different path: building architectures that are efficient on the target by construction.

10.3.4 Hardware-aware design

Hardware-aware design begins before compression. If the target device cannot keep convolution kernels fed, cannot store activations without spilling, or cannot meet the power budget at the chosen input resolution, pruning and quantization only treat symptoms. The architecture itself must expose the kind of work the hardware can execute efficiently. That means choosing layer shapes, scaling rules, and operator patterns with memory bandwidth, parallelism, and energy as first-class constraints rather than post-hoc deployment checks.

10.3.4.1 Efficient design principles

The first design step is to identify what the trace shows as the limiting resource. A model can miss its target because every layer is too expensive, because one convolution family dominates arithmetic, because activations or parameters do not fit the memory hierarchy, or because the candidate architecture ignores the platform's preferred operators. Table 10.7 organizes the common responses by the bottleneck they address rather than by model family.

Table 10.7: Hardware-Aware Design Principles: A profiling trace should determine which architectural response is appropriate. MobileNet responds to redundant convolutional computation with depthwise separable convolutions, DenseNet and SqueezeNet respond to memory pressure with feature reuse and parameter reduction, and hardware-aware NAS incorporates measured platform behavior directly into the search objective.

Observed bottleneck	Architectural response	Example networks
Over-budget model scaling	Adjust depth, width, and resolution together so the model stays within the latency, memory, and power envelope.	EfficientNet
Redundant computation	Replace expensive dense operations with factorized or grouped operations that preserve useful channel mixing at lower arithmetic cost.	MobileNet, ResNeXt
Memory pressure	Reduce parameter storage or reuse features, then check peak activation lifetimes; feature reuse can retain a larger working set.	DenseNet, SqueezeNet
Platform mismatch	Include measured device latency, operator support, and power behavior in the architecture search or design loop instead of optimizing FLOPs alone.	MobileNetV3, MnasNet

These responses interact. Reducing convolutional FLOPs with depthwise separable convolutions helps only if the target runtime has efficient kernels for the resulting operators. Shrinking a model with parameter-reduction layers helps only if activation storage or memory traffic was part of the measured problem. Hardware-aware design therefore does not replace profiling; it moves profiling information earlier, into the architecture itself.

10.3.4.2 Scaling optimization

The first architectural response is global: when every stage of the profile is over budget, the problem is not one bad layer; the model is scaled incorrectly for the deployment envelope. The design task is then to distribute capacity across depth, width, and input resolution rather than choose a single parameter count. Depth increases sequential work and activation storage. Width exposes more parallel work but raises memory use. Resolution improves spatial detail while increasing the number of positions each convolution must process. The right balance depends on the machine: a highly parallel accelerator can often exploit width, while a small edge device may be dominated by memory capacity and energy per access.

Mathematically, the total FLOPs for a convolutional model can be approximated as:

$$O \propto N_L \cdot w_{\text{mult}}^2 \cdot r^2,$$

where N_L is depth (number of layers), w_{mult} is the width multiplier (channel count relative to a base model), and r is the input resolution. This expression shows why naive scaling fails: increasing width and resolution together multiplies work quickly, and the resulting model may exceed the memory bandwidth or power budget even when the parameter count appears reasonable.

Compound scaling turns this balancing act into a controlled design rule. Instead of adjusting depth, width, and resolution independently, compound scaling grows all three dimensions by fixed ratios (α, β, γ) relative to a base model:

$$N_L = \alpha^{\phi} N_{L,0}, \quad w_{\text{mult}} = \beta^{\phi} w_{\text{mult},0}, \quad r = \gamma^{\phi} r_0$$

Here, ϕ is a scaling coefficient, and α, β, and γ (as in the original EfficientNet paper) are scaling factors determined from empirical accuracy and efficiency measurements. The rule matters because it prevents one dimension from consuming the budget before the others can contribute useful accuracy.

EfficientNet (section 10.3.5) validated this principle by using search to find a baseline architecture and then scaling it with balanced depth, width, and resolution coefficients (Tan and Le 2019). The lesson is not that every deployment should use EfficientNet. The systems lesson is that scaling is a resource-allocation decision: the same accuracy target can imply different depth-width-resolution trade-offs depending on which resource the target platform makes scarce. Later benchmarking material formalizes how to measure those trade-offs; here, the design principle is to scale the dimensions against the binding resource.

The same logic extends beyond convolutional models. Transformer layers, attention heads, sequence length, and embedding width play roles analogous to depth, width, and resolution: each increases capacity, but each stresses compute, memory bandwidth, or activation storage differently. Hardware-aware scaling keeps those dimensions tied to the measured bottleneck instead of treating model size as a single scalar.

10.3.4.3 Computation reduction

If the profile shows that a small set of convolutional operators dominates arithmetic, reducing the whole model uniformly is wasteful. The better response is to change the expensive operator. Modern efficient architectures do this by factorizing dense computations into cheaper pieces that preserve the representation needed for accuracy.

Depthwise separable convolutions, the factorization behind the MobileNetV2 and KWS lighthouses (Howard et al. 2017; Sandler et al. 2018), exemplify this approach by decomposing standard convolutions into two stages: depthwise convolution (applying separate filters to each input channel independently) and pointwise convolution (1×1 convolution mixing outputs across channels). For batch size one, unit stride, padding that preserves an $H_{\text{img}} \times W_{\text{img}}$ spatial size, and a square kernel of width K, the computational complexity of standard convolution with d_{in} input channels and d_{out} output channels is:

$$\mathcal{O}(H_{\text{img}} W_{\text{img}} d_{\text{in}} d_{\text{out}} K^2)$$

where K is kernel size. Depthwise separable convolutions reduce this to:

$$\mathcal{O}(H_{\text{img}} W_{\text{img}} d_{\text{in}} K^2) + \mathcal{O}(H_{\text{img}} W_{\text{img}} d_{\text{in}} d_{\text{out}})$$

eliminating the K^2 factor from channel-mixing operations. The ratio of the two costs is $1/d_{\text{out}} + 1/K^2$, so with 3×3 kernels and many output channels the factorized layer needs roughly one-ninth of the arithmetic. What this chapter adds to that derivation is the deployment condition. The wall-clock benefit depends on kernel support and memory behavior: a mobile runtime with optimized depthwise kernels can convert much of this arithmetic reduction into latency savings, while a poorly supported backend may expose the factorized operations as many small, memory-bound kernels.

↩ Depthwise separable convolution and its cost reduction relative to a standard convolution are established in section 6.5.5.

Other factorization patterns respond to the same diagnosis. Grouped convolutions, used in ResNeXt, partition feature maps into independent groups before merging them, reducing redundant cross-channel work. Bottleneck layers, used in ResNet, apply 1×1 convolutions to reduce feature dimensionality before expensive operations. SqueezeNet uses the same 1×1 idea to reduce parameters. These techniques improve efficiency when they reduce the operation that actually dominates the trace; they provide much less benefit when memory traffic, launch overhead, or unsupported kernels become the new bottleneck.

Arithmetic reduction is only one possible response. When profiling instead identifies memory capacity or data movement as the binding resource on the target device, the architecture must address that memory bottleneck directly.

10.3.4.4 Memory optimization

When the profile points to memory rather than arithmetic, the architecture must reduce the working set or the number of expensive memory accesses. Activations, feature maps, and parameters can exceed cache, SRAM, accelerator memory, or edge-device storage even when the FLOP count is acceptable. Memory-efficient architectures therefore try to preserve useful information while storing or moving less data.

DenseNet illustrates the feature-reuse response (Huang et al. 2017). In a traditional convolutional network, each layer computes a new set of feature maps, increasing the activation footprint as the network deepens. DenseNet connects layers so later computations can reuse earlier feature maps instead of relearning similar representations. In DenseNet with N_L layers and growth rate g, each layer generates g new feature maps concatenated with prior layers, causing the total number of feature maps to grow linearly:

$$\mathcal{O}(N_L g)$$

DenseNet reduces parameter redundancy through feature reuse, but concatenating retained features can increase activation storage and memory traffic. The resulting working set and runtime must be measured on the target hardware.

Activation checkpointing attacks activation memory by trading recomputation for retained tensor states during backpropagation. Its role in compression is strictly indirect: it leaves the deployed inference artifact unchanged, but enables training an over-wide teacher or search supernet within a fixed memory budget, providing the surplus capacity that pruning and distillation later carve away.

↩ Activation checkpointing, which trades recomputation for saved activations during training, is established in section 8.7.5.1.

Parameter reduction applies the same reasoning to storage. SqueezeNet uses 1×1 convolutions to reduce the number of input channels before applying standard convolutions, making the expensive layer operate on a smaller representation (Iandola et al. 2016). The number of parameters in a standard convolutional layer is:

$$\mathcal{O}(d_{\text{in}} d_{\text{out}} K^2)$$

By reducing d_{in} using 1×1 convolutions, SqueezeNet reduces parameter count, achieving the paper's AlexNet-level accuracy target with far fewer parameters than AlexNet. That trade is attractive when the deployment constraint is flash storage, model download size, or parameter bandwidth; it is less decisive when activation memory or operator overhead dominates.

Feature reuse, activation checkpointing, and parameter reduction are therefore not interchangeable recipes. Each changes a different part of the memory problem. Reused features reduce redundant representations, checkpointing reduces training-time activation storage only, and 1×1 bottlenecks reduce parameter movement at deployment. The correct choice follows from which memory term the profile shows as binding.

Manual hardware-aware design identifies effective architectural motifs, but tuning channel widths, kernel dimensions, and layer depths across dozens of network stages quickly exceeds what human intuition can optimize systematically. When the combinatorial space of valid configurations is vast, discovering the optimal trade-off curve requires automated search.

10.3.5 Neural architecture search

Neural architecture search (NAS) automates structural design by systematically traversing a bounded space of operations, topologies, and compound scaling rules (Elsken et al. 2019). Rather than compressing an existing model post hoc, NAS discovers optimized architectures prior to training. Early reinforcement-learning formulations maximized task accuracy without hardware constraints (Zoph and Le 2016); subsequent weight-sharing methods amortized evaluation overhead (Pham et al. 2018); and hardware-aware NAS explicitly incorporated measured device latency and energy into the multi-objective search (Tan et al. 2019).

Figure 10.12 formalizes this closed-loop discovery pipeline: the search strategy samples candidate architectures $a \in \mathcal{A}$ from the defined search space, while the performance estimation engine evaluates empirical latency or accuracy and routes feedback signals back to refine subsequent proposals. NAS[13] explores these candidate layer topologies, expansion ratios, and kernel dimensions to discover Pareto-optimal graphs under explicit resource bounds.

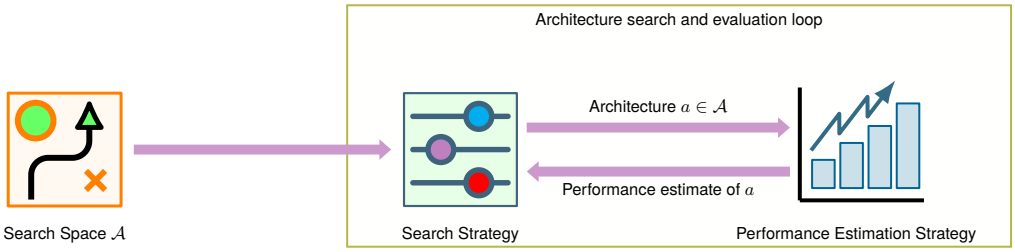


Figure 10.12: Neural Architecture Search Flow: An automated optimization loop for architecture discovery. The search strategy samples candidate subnetworks from a predefined search space, while the performance estimation engine evaluates candidates against target accuracy and hardware constraints (such as latency or energy), feeding reward signals back to steer subsequent exploration toward Pareto-optimal designs.

[13] **Hardware-aware NAS**: Optimizes measured latency rather than FLOPs, which can diverge by 3–5× when memory access or operator support dominates. Tan et al. (2019) fed device latency into search and found architectures 1.8× faster than MobileNetV2 at higher accuracy.

10.3.5.1 The NAS optimization problem (optional depth)

The effectiveness of NAS depends on three design decisions: what architectures to search over (the search space), how to explore that space efficiently (the search strategy), and how to evaluate each candidate's fitness for deployment. The optimization problem begins with a chicken-and-egg constraint: an architecture's quality cannot be known until it is trained, but training is expensive. This creates two nested decisions: choosing which operations to include (the architecture) and finding the best parameters for those operations (the weights). The architecture defines *what* to optimize; the weights define *how well* that architecture can perform.

NAS is therefore a **bi-level optimization problem**.[14] The outer loop searches the architecture space $\mathcal{A}$, while the inner loop trains candidate architectures to evaluate performance. Formally, the objective seeks the optimal architecture a^* that minimizes validation loss $\mathcal{L}_{\text{val}}$ under constraints C (latency, memory):

$$a^* = \arg\min_{a \in \mathcal{A}} \mathcal{L}_{\text{val}}(\theta^*(a), a) \quad \text{subject to} \quad C(a) \leq C_{\max}$$

where $\theta^*(a)$ represents the optimal model parameters for architecture a, obtained by minimizing training loss:

$$\theta^*(a) = \arg\min_{\theta} \mathcal{L}_{\text{train}}(\theta, a)$$

The computational bottleneck lies in this inner loop. If evaluating every candidate requires independent training from scratch, a modest search space of 10 operation choices across 20 layers yields 10^{20} possible architectures, making exhaustive enumeration impossible. Practical NAS methods overcome this combinatorial explosion through three mechanisms: restricting the search space to reusable cells, amortizing evaluation through weight sharing, or relaxing discrete choices into continuous, differentiable formulations.

[14] **Bi-level optimization**: A formulation where one optimization problem sits inside another. In NAS, the outer level selects an architecture while the inner level trains that candidate's weights, so early methods paid a full training cost for every architecture evaluated. This nesting is why early NAS required 22,400 GPU-days; weight-sharing methods amortize one training run across many candidates, reducing search cost by roughly 1,000×.

10.3.5.2 Search space design (optional depth)

The search space defines what architectures NAS can discover. Well-designed search spaces incorporate domain knowledge to focus search on promising regions while remaining flexible enough to discover novel patterns.

Rather than searching entire network architectures, many NAS systems search for reusable computational blocks, or cells, that can be stacked to form complete networks. A convolutional cell might choose from operations such as 3×3 convolution, 5×5 convolution, depthwise separable convolution, max pooling, or identity connections. A simplified cell with four nodes and two operations per edge yields roughly 10,000 possible cell designs, far more tractable than searching full architectures. NASNet exemplifies this approach, discovering reusable normal and reduction cells that can be stacked to form complete networks across different model sizes.

Search spaces also encode physical hardware constraints directly. Rather than allowing arbitrary topological graphs, hardware-aware search spaces restrict candidate operations to primitives that have efficient, vectorized kernel implementations on the target platform. For example, mobile search spaces prioritize depthwise separable convolutions with bounded kernel sizes and channel dimensions and layouts chosen for efficient kernels and memory transactions (Zhang et al. 2020). MobileNetV3 paired this structured block-level search space with NetAdapt to prune layer channels against measured mobile CPU latency (Howard et al. 2019).

10.3.5.3 Search strategies (optional depth)

Search strategies determine how to explore the architecture space efficiently without exhaustive enumeration. Table 10.8 compares the trade-offs between search cost, architectural diversity, and optimality guarantees for each approach.

Table 10.8: NAS Search Strategy Comparison: Representative trade-offs between search efficiency, use cases, and limitations for different NAS approaches. The costs are method-specific results from early reinforcement-learning and evolutionary searches and from DARTS, not universal ranges for each strategy (Zoph and Le 2016; Real et al. 2019; Liu et al. 2019).

Strategy	Search Efficiency	When to Use	Key Challenge
Reinforcement Learning	22,400 GPU-days	Novel domains, unconstrained search	High computational cost
Evolutionary Algorithms	3,150 GPU-days	Parallel infrastructure available	Requires large populations
Gradient-Based (DARTS)	1–4 GPU-days	Limited compute budget	May converge to suboptimal local minima

Reinforcement learning-based NAS treats architecture search as a sequential decision: a controller generates architectures and receives an accuracy reward. The controller (typically a long short-term memory network) learns to propose better architectures through policy gradient optimization, which raises the probability of the design choices that earned high reward. This approach discovered high-performing architectures such as NASNet, but its inner loop is expensive because every reward requires training a candidate architecture; Zoph and Le (2016) evaluated roughly 12,800 architectures, totaling 22,400 GPU-days. This cost pushed practical NAS toward weight sharing and lower-cost performance predictors.

Evolutionary algorithms maintain a population of candidate architectures and iteratively apply mutations (changing operations, adding connections) and crossover (combining parent architectures) to generate offspring. Fitness-based selection retains high-performing architectures for the next generation, so useful components such as skip connections or depthwise separable convolutions can be recombined rather than rediscovered from scratch. AmoebaNet used evolution to achieve ImageNet accuracy competitive with top human-designed models after 3,150 GPU-days (Real et al. 2019), showing both the value of population search and the continuing need for proxy tasks, weight sharing, or massive parallelism to control search cost.

Gradient-based methods such as DARTS (Differentiable Architecture Search) (Liu et al. 2019) represent the search space as a continuous relaxation where all possible operations are weighted combinations. Rather than discrete sampling, DARTS optimizes architecture weights and model weights jointly using gradient descent. By making the search differentiable, DARTS reduces search

cost from hundreds of GPU-days to one to four GPU-days, though the continuous relaxation may miss discrete architectural patterns that discrete search methods discover.

Hardware-aware NAS moves beyond theoretical FLOP counts by optimizing directly for execution metrics on target silicon. FLOP counts correlate poorly with runtime latency when memory bandwidth, cache residency, or kernel launch overheads dominate. Under the arithmetic intensity law (principle 6), an operation with low FLOPs but low arithmetic intensity stalls waiting for memory transactions, running slower than a higher-FLOP operation that achieves roofline compute saturation. MnasNet addresses this divergence by measuring each candidate's latency directly on a mobile phone and folding it into the reward, which combines accuracy and latency through a weighted product:

$$\text{Reward}(a) = \text{Accuracy}(a) \times \left(\frac{L_{\text{lat,target}}}{L_{\text{lat,infer}}(a)} \right)^{\beta_{\text{lat}}}$$

where $L_{\text{lat,infer}}(a)$ is the measured on-device inference latency of architecture a, $L_{\text{lat,target}}$ is the latency constraint, and the exponent β_{lat} controls the accuracy-latency trade-off. This formulation penalizes architectures that exceed latency targets while rewarding those that achieve high accuracy within the budget. In the reported MnasNet search space, varying inverted-residual expansion ratios produced a stronger accuracy-latency trade-off than uniform expansion, illustrating how search can expose nonuniform designs that a narrower manual sweep might not test.

10.3.5.4 When to use NAS

Neural architecture search can discover architectures that outperform hand-designed alternatives, but its computational cost demands careful consideration of when the investment is justified. NAS can be worthwhile for novel hardware platforms with unusual constraints, such as new accelerator architectures or highly constrained edge devices, where existing architectures are poorly optimized. It can also make sense at deployment scales where small efficiency improvements justify the upfront search cost, or when architecture families for cloud, edge, and mobile deployments can amortize one search across many variants.

Conversely, custom NAS is difficult to justify when standard deployment constraints already have well-optimized architecture families. If the compute budget is only a few GPU-days, large reinforcement-learning or evolutionary searches are impractical; differentiable or weight-sharing methods such as DARTS may remain feasible, but still require deployment-scale and validation-cost justification. Rapidly changing requirements also weaken the case because the target can move before search and validation finish.

For most practitioners, existing NAS-discovered or NAS-assisted architectures such as EfficientNet (Tan and Le 2019), MobileNetV3 (Howard et al. 2019), and MnasNet (Tan et al. 2019) provide strong baselines before funding a search from scratch. Their transfer quality still depends on task and hardware. Reserve custom NAS for unusual constraints or deployment scales that can amortize the search and validation investment.

10.3.5.5 Architecture examples

NAS studies have surfaced reusable design patterns within their search spaces. EfficientNet jointly scales depth, width, and resolution with compound coefficients and reports improved accuracy-efficiency trade-offs across its model family (Tan and Le 2019). MobileNetV3 combines hardware-aware search and NetAdapt for phone latency targets (Howard et al. 2019), while FBNet incorporates device-specific latency into a mobile-CPU search objective (Wu et al. 2019). These results show what search found under particular spaces, objectives, and measurements; they do not establish that automated search universally outperforms careful manual design.

Beyond convolutional networks, NAS has been applied to transformer architectures, including searches for compact language and vision models under compute or memory constraints. The common systems lesson is narrower than a claim that search always beats manual design: when latency, memory, or energy enters the objective and is measured on the target platform, the search evaluates candidates against the resource that deployment actually constrains. The result still depends on the search space, proxy quality, optimization budget, and final retraining.

10.3.6 Adaptive computation methods

Pruning, distillation, factorization, and architecture search all produce a fixed execution graph. Absent input-dependent control, every input follows an identical scheduled path. On static graphs, the **Algorithm** imposes an invariant computational workload on the **Machine**: a 24-layer transformer executes identical multiply-accumulate (MAC) operations and streams its full parameter footprint from HBM or off-chip DRAM for an obvious, easily classified sample as it does for an ambiguous one. **Adaptive inference** replaces this open-loop pipeline with an input-dependent control loop: intermediate representations or runtime budgets inform dynamic decisions that terminate execution or route activations through specialized subnetworks. While dynamic paths can reduce average arithmetic operations and memory traffic, they introduce runtime overheads in routing logic, batch management, confidence calibration, and latency variance. Dynamic pruning (section 10.3.1.4) already met this bargain at the granularity of channels and filters, with the same three costs: a controller on the critical path, batches fragmented by input-dependent routes, and a larger behavior space to validate. Adaptive computation applies the idea at coarser grain, to whole layers, exits, and expert subnetworks, where the savings per decision are larger and so is the price of a wrong one.

10.3.6.1 Dynamic schemes

Because input data exhibit non-uniform semantic complexity, allocating an invariant computational budget wastes memory bandwidth and execution cycles on simple samples. Dynamic schemes modulate execution paths to lower amortized computation while preserving the deep network for worst-case inputs. This architectural flexibility spans four primary control variables: network depth (early exit), structural routing (conditional layer skipping), expert subnetworks (mixture-of-experts), and continuous computational allocation.

To control execution depth, **early exit** architectures attach auxiliary classification heads to intermediate layers, terminating execution when intermediate representations yield sufficient prediction confidence (Teerapittayanon et al. 2017). BranchyNet implements this mechanism across convolutional backbones using shallow exit branches, whereas multi-exit vision transformers attach linear probes and entropy estimators between transformer blocks (Xin et al. 2021). When an input satisfies an exit criterion—such as softmax prediction entropy dropping below a pre-tuned threshold—the runtime halts evaluation and returns the intermediate prediction. The net operational benefit requires that the arithmetic operations and memory transfers avoided in subsequent layers exceed the computational overhead of evaluating the intermediate classifiers and gating thresholds.

The systems trade-off diverges sharply across hardware deployment targets. On single-sample edge processors (batch size $B = 1$, typical of microcontrollers and mobile NPUs), early exit immediately terminates the execution thread (Hu et al. 2020). Skipping remaining layers eliminates both the arithmetic operations and the off-chip DRAM transactions required to load subsequent layer weights into on-chip SRAM cache. Conversely, on parallel throughput accelerators (GPUs and TPUs) processing batched workloads ($B > 1$), early exit introduces severe batch fragmentation (Chen et al. 2024). If only a subset of samples in a batch reaches the exit threshold, the remaining samples must still traverse deeper layers. Unless the runtime executes dynamic batch compaction—gathering unresolved activation tensors into a contiguous memory buffer via memory copies and kernel synchronizations—inactive samples remain allocated in memory, or the accelerator must mask inactive SIMD or single instruction, multiple threads (SIMT) lanes. Consequently, tail latency for the batch remains dictated by the deepest traversing sample. Figure 10.13 traces this branching topology along the backbone: each transformer block taps intermediate representations into an auxiliary classifier and confidence estimator (LTE) to trigger an early exit, while unresolved tokens traverse horizontal connections to deeper layers.

↪ The latency-throughput trade-offs of batching under production request streams, including service-level objective (SLO) violations, are developed in section 13.7.4.

Beyond truncating execution depth along a single linear pipeline, **conditional computation** dynamically modulates the topological routing of activations through the network graph (Bengio et al. 2015). Instead of executing every block in a predefined sequence, gating functions evaluate activation tensors to bypass residual layers, alter operator kernels, or select specialized functional blocks. This converts a static directed acyclic graph (DAG) into an input-dependent dynamic graph.

SkipNet illustrates the pattern at the granularity of whole blocks. It uses lightweight recurrent or feedforward gating controllers to conditionally bypass residual blocks in convolutional networks, preserving full network depth only for computationally difficult inputs (X. Wang et al. 2018). From

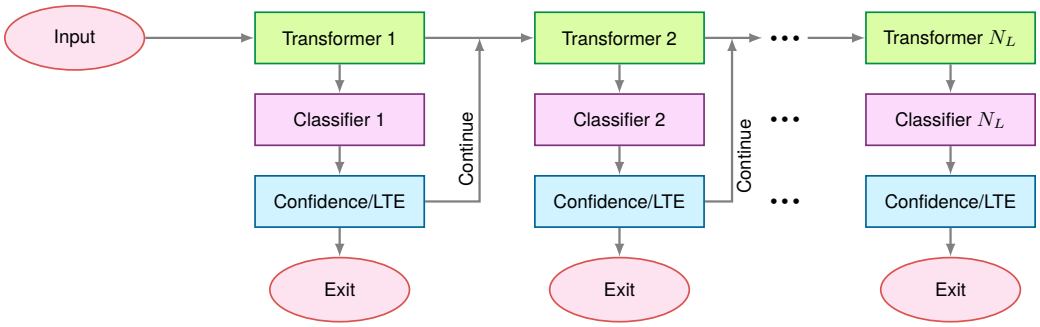


Figure 10.13: Early Exit Architecture: Transformer layers dynamically adjust computation by classifying each layer's output and enabling early termination if sufficient confidence is reached, reducing average latency and power when many inputs can exit before the final layer. Source: (Xin et al. 2021).

a hardware systems perspective, conditional routing achieves wall-clock speedups only when the cycles saved by bypassing operators surpass the latency of evaluating the gating network, executing host-device control synchronization, and managing uncoalesced memory accesses.

Scaling dense transformer models couples parameter capacity directly to per-token floating-point operations: increasing model capacity forces every token to compute against every parameter weight matrix, driving up both arithmetic demand and memory bandwidth requirements. The **mixture-of-experts (MoE)** architecture decouples parameter capacity from per-token compute by replacing dense feedforward network (FFN) layers with a bank of parallel expert subnetworks coordinated by a parameterized router (Shazeer et al. 2017). Google's Switch Transformer[15] scales this sparse routing principle by steering each token to a single expert ($k_{\text{experts}} = 1$), dramatically reducing communication overhead compared to routing to $k_{\text{experts}} > 1$ experts (Fedus et al. 2022).

This sparse activation creates a fundamental memory-bandwidth and capacity trade-off on accelerator hardware. While a token routed to one of N_{experts} experts executes arithmetic against only $1/N_{\text{experts}}$ of the total expert parameters, an all-resident deployment keeps all N_{experts} expert matrices in device memory (HBM or SRAM). Offloading changes this capacity requirement by adding transfers to the execution path. At low operational batch sizes—such as single-stream autoregressive generation—routing divergent tokens to different experts fragments matrix-matrix multiplications (GEMM) into low-arithmetic-intensity matrix-vector operations (GEMV), which can make execution bandwidth-limited. Furthermore, uneven token distribution causes computational hotspots where overloaded experts drop tokens beyond their capacity factor (the fixed number of token slots each expert reserves per batch), while under-allocated experts leave tensor core pipelines idle.

Sparse Mixture-of-Experts architectures replace monolithic feedforward networks (FFNs) with dynamically routed subnetworks. Figure 10.14 diagrams this conditional execution pathway: the left block embeds a Switching FFN layer within the standard transformer stack, while the expanded right panel details the gating router dispatching individual tokens ("More" with $p_{\text{gate}} = 0.65$, "Parameters" with $p_{\text{gate}} = 0.8$) to distinct expert sub-blocks (FFN 2 and FFN 1) while leaving unselected experts idle.

Because the router lies on the critical execution path, the accelerator runtime must dynamically dispatch and gather activations across expert blocks without triggering host-device synchronization stalls. On data-parallel accelerators such as GPUs and TPUs, executing sparse expert layers efficiently requires token permutation: tokens assigned to the same expert are gathered into contiguous memory tiles to sustain high Tensor Core utilization during subsequent matrix multiplications (Lepikhin et al. 2021). If the routing distribution is highly skewed, token buffers for popular experts overflow their fixed capacity allocations, forcing the system to either truncate tokens (causing representation degradation) or trigger ragged kernel launches that leave parallel vector lanes idle.

While early exit and MoE routing make discrete branch selections, adaptive inference can also operate continuously by progressively modulating depth or iteration counts based on real-time task difficulty (Yang et al. 2020). Dynamic routing architectures such as BlockDrop dynamically adjust the number of active layers according to runtime policy networks (Wu et al. 2018), while dynamic layer scaling allocates computation incrementally until output uncertainty drops below a calibrated

[15] **Switch Transformer**: Fedus et al. (2022) scales to roughly 1.6 trillion total parameters while routing each token to one expert; in a compute-matched experiment, Switch-Base with 64 experts reached the T5-Base quality target in about one-seventh the training time; the trillion-parameter comparison reported roughly 4× speedup over T5-XXL. Routing each token to one expert reduces communication relative to routing to $k_{\text{experts}} > 1$ experts, but load imbalance requires auxiliary losses and capacity factors, which cap how many tokens each expert accepts per batch. This trade-off—massive parameter capacity at low per-token compute cost, but with complex systems engineering for load balancing—defines the MoE design space.

↪ How runtimes execute tensors whose shapes vary, by matching precompiled shape profiles, padding, splitting the batch, or falling back to a general kernel, is developed in section 11.10.2.

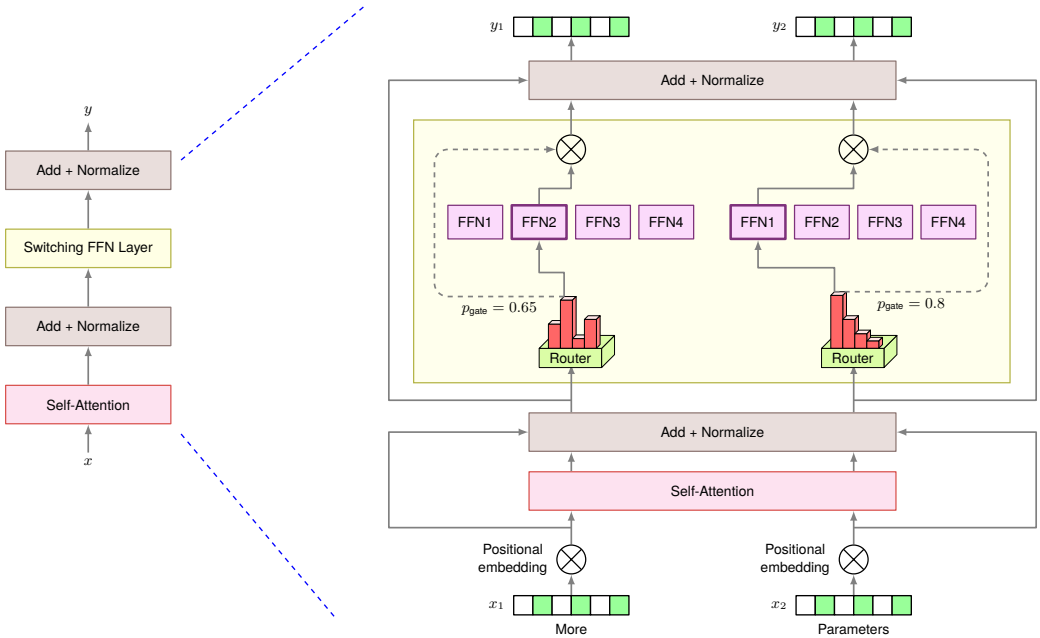


Figure 10.14: Switch Transformer Architecture: Replacing dense feedforward layers with a Switching FFN layer (left) enables sparse conditional routing. A gating network routes each token to a single expert (right) out of a large pool, decoupling total model capacity (parameters) from per-token compute cost (FLOPs). Source: (Fedus et al. 2022).

threshold. Consider an embedded vision system processing sensor streams: high-contrast frames with clear road geometry resolve through a minimal prefix of feature-extraction layers, whereas degraded frames containing occluded obstacles trigger deeper recurrent processing blocks. The physical systems gain materializes only if the control evaluation remains computationally lightweight and the runtime schedules frames with matched execution depths concurrently to maintain high hardware utilization.

10.3.6.2 Implementation challenges

Realizing the efficiency gains of adaptive computation requires overcoming challenges across optimization, runtime orchestration, and hardware execution. During training, discrete routing decisions (such as non-differentiable argmax branch selections) break standard gradient-based backpropagation. Practitioners overcome this by using continuous relaxations such as the Gumbel-Softmax estimator (which replaces hard selections with temperature-scaled softmax distributions over noise-perturbed scores so gradients flow), policy gradients that reward discrete routing actions, or auxiliary load-balancing losses that prevent routers from collapsing onto a single favored expert (Shazeer et al. 2017).

At runtime, control-flow branching fundamentally conflicts with the bulk-synchronous parallel execution model of modern accelerators, in which threads compute in parallel phases separated by synchronization points, and the threads of a warp execute the same instruction in lockstep. When parallel threads within a GPU warp encounter divergent execution branches, the hardware may serialize branch execution and mask inactive threads, reducing the value of theoretical FLOP savings. Gating also introduces control dependencies that can limit fusion, require token-permutation buffers, or trigger host-device synchronization, depending on the runtime. If these dispatch overheads exceed the compute time saved by skipping layers, the dynamic model exhibits higher end-to-end latency than a dense baseline. Runtime strategies such as ragged batching and dynamic kernel launch queues exist to manage exactly these execution patterns, and they belong to the accelerator runtime rather than to the model.

Beyond systems efficiency, dynamic control introduces severe quality and security failure modes. Because gating policies route inputs based on statistical heuristics, poorly calibrated gates systemati-

cally underallocate computation to rare distribution tails, degrading prediction accuracy precisely on safety-critical edge cases. Furthermore, input-dependent computation exposes an attack surface: adversarial perturbations can intentionally manipulate routing gates to trigger denial-of-quality attacks (forcing inputs into truncated early exits that fail) or denial-of-service attacks (forcing all inputs into maximally deep or imbalanced paths to saturate accelerator queues). Consequently, evaluating adaptive models cannot rely on average FLOP counts alone. Rigorous system profiling must report path distribution histograms, tail latency (p99), accuracy stratified by input difficulty, and throughput stability under dynamic batching.

The structural techniques covered so far (pruning, distillation, factorization, hardware-aware design, NAS, and adaptive computation) all optimize *what* computations the model performs, including which parameters exist, which connections remain, how the architecture is structured, and which paths a given input executes. These techniques can substantially reduce parameter counts and theoretical FLOPs. Regardless of structural efficiency, every surviving weight and activation must still be stored and processed at some numerical precision.

Checkpoint 10.2: Choosing a structural method

Structural methods relocate deployment cost in different ways.

- ☐ **Structured vs. unstructured pruning**: compare how each pattern interacts with GPU and TPU execution.
- ☐ **Distillation vs. pruning**: explain why a dense student may preserve accuracy better than aggressive pruning while requiring additional training.
- ☐ **NAS selection**: identify when hardware-specific constraints and deployment scale justify architecture search rather than manual design.
- ☐ **Factorization cost**: explain why fewer stored parameters and MACs may not reduce latency when two factor operations add launches and an intermediate tensor.
- ☐ **Adaptive computation**: explain why early exit can cut latency at batch size one yet fragment batched execution on a GPU.

Structural optimizations alter what a model computes, but the surviving graph must still execute within the physical memory bandwidth, register files, and cache hierarchies of the target silicon. When structural pruning and distillation have carved away unnecessary parameters and layers, every remaining operation continues to incur a memory-movement and arithmetic cost determined entirely by its bit representation. Optimizing this numerical representation forms the second tier of the optimization stack.

10.4 Quantization and Precision

The 7-billion-parameter language model that opened this chapter demonstrates why structural modification alone cannot resolve memory bottlenecks. Its FP16 weights require 14 GB, exceeding the 8 GB RAM capacity of mid-tier mobile hardware. While structural techniques prune layers or compress channels, the complementary optimization leaves network topology unaltered and reduces the bit width of every surviving tensor. Quantization renegotiates the silicon contract at the bit level, trading numerical precision for reduced memory footprint, higher cache utilization, and amplified arithmetic throughput.

Definition 10.4: Quantization

Quantization is a model-compression technique that reduces information fidelity by mapping high-precision continuous values to a lower-precision discrete set.

1. **Significance**: FP32-to-INT8 conversion reduces the raw payload per quantized value by 4×. Real artifact size, traffic, and latency also depend on scales, metadata, packing, operator coverage, and the target kernels.

2. **Distinction**: Unlike pruning, which reduces the count of parameters, quantization reduces the bit depth of selected weights and/or activations.
3. **Common pitfall**: A frequent misconception is that quantization is just "rounding." In reality, it is a lossy mapping that requires careful range estimation and, often, quantization-aware training (QAT) to minimize its impact on accuracy.

Quantization[16] concerns every neural network weight and activation stored at some numerical precision: FP32 (32 bits), FP16 (16 bits), INT8 (8 bits), or lower. Bit width therefore becomes a critical systems parameter. It constrains raw payload size and cache footprint, affects bandwidth demand when values remain packed, and shapes the area and throughput trade-offs of specialized matrix engines.

Quantization transforms three physical terms of execution. First, converting FP32 to INT8 reduces raw weight payload by 4× before scale and packing metadata, allowing models to fit within constrained device memories. Second, when tensors remain packed across memory buses, memory traffic drops proportionally, directly reducing token latency in bandwidth-bound autoregressive decoding. Third, fixed-point integer arithmetic units occupy smaller silicon area and consume significantly less energy per operation than floating-point units, enabling higher compute density on supported hardware (Gupta et al. 2015; Y. E. Wang et al. 2019).

The accuracy cost varies by model, task, quantizer, calibration data, and bit width. Many convolutional neural network (CNN) inference models tolerate INT8 well; transformers, speech models, and outlier-heavy layers may need different granularity, mixed precision, or training-aware methods. The main approaches are **post-training quantization** (PTQ), **quantization-aware training** (QAT), and mixed-precision or **extreme quantization** for lower-bit deployment. They differ in training cost and control, not in a guaranteed ordering of accuracy.

The viability of each approach depends on how much precision a given model architecture can shed before quality degrades. Figure 10.15 charts this behavior across precision formats from FP32 down to INT2. Both CNN and transformer curves exhibit a low-loss plateau where INT8 quantization closely matches full-precision performance, followed by a steep accuracy cliff at lower bit widths where representational capacity breaks down. Because tolerable accuracy degradation depends on end-to-end application constraints, engineering teams must evaluate these cliff boundaries for each target workload and calibration recipe.

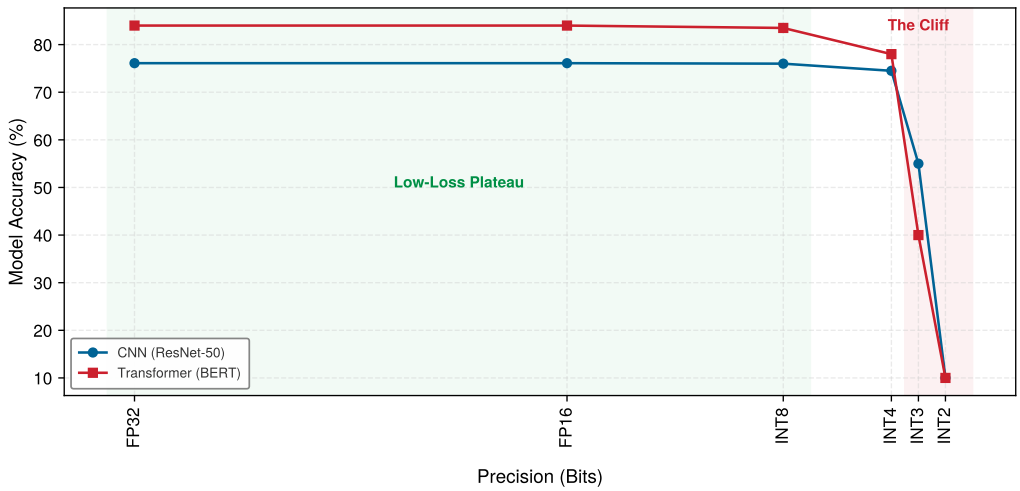


Figure 10.15: Illustrative Quantization Regimes: The curves depict a qualitative plateau followed by an accuracy cliff as bit width falls. Their breakpoints are illustrative; actual boundaries depend on the model, layer distributions, quantizer, calibration or training method, and evaluation task.

[16] **Quantization**: Rooted in Shannon's theory of representing continuous signals with discrete values (Shannon 1948), reducing FP32 to INT8 collapses over four billion representable values to just 256. Neural networks often tolerate this because trained weights concentrate information in relative magnitudes, not absolute precision: INT8 inference can stay close to the full-precision baseline with appropriate calibration or quantization-aware training, while INT4 and lower-bit methods become more architecture- and method-dependent (Jacob et al. 2018; Gholami et al. 2022; Shen et al. 2020; Lin et al. 2024). The systems consequence is that quantization viability must be validated per-model and per-task, not assumed from aggregate benchmarks.

↪ How accelerators trade precision and dynamic range for matrix-engine throughput, pairing low-precision inputs with higher-precision accumulation, is developed in section 11.4.8.

10.4.1 Precision and energy

Precision is an energy decision because physical data movement can dominate a workload's energy budget. The balance depends on the number and location of memory accesses as well as the arithmetic performed. In silicon, energy dissipation scales directly with wire capacitance ($E \propto CV^2$), meaning that shuttling bits across long memory buses costs orders of magnitude more joules than toggling logic gates inside an arithmetic logic unit. Moving a 32-bit floating-point operand from off-chip DRAM to local registers consumes substantially more energy than executing the multiply-accumulate operation itself. Reducing numerical precision directly contracts the byte volume traversing the memory hierarchy (D_{vol}) and lowers the switching energy per ALU operation (E_{compute}), relaxing memory bandwidth and thermal bottlenecks across the hardware stack.

> Systems Perspective 10.3: The physics of quantization
>
> The energy-movement principle (7), $E_{\text{move}} \gg E_{\text{compute}}$, summarizes the high cost of moving an operand relative to operating on it in the reference technology. Applying it to a workload requires multiplying energy per byte and energy per operation by their respective traffic and operation counts (Horowitz 2014). Section D.1 collects the reference energy ratios, and section D.4 compares numerical format trade-offs.
>
> Under the iron law of ML systems (principle 3), reducing bit width changes the data-movement (D_{vol}/BW) and compute ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) terms only through the execution path that uses the new representation. A first-order packed-value model gives two useful comparisons:
>
> 1. **Memory movement** ($D_{\textbf{vol}} \times E_{\textbf{move}}$): A packed INT8 stream carries one quarter as many raw value bits as FP32. The corresponding energy estimate assumes full transactions are amortized across packed values; caches, metadata, alignment, and transaction size affect the realized cost.
> 2. **Compute work** ($O \times E_{\textbf{compute}}$): A 32-bit multiply costs ≈ 3.7 pJ/op. An INT8 multiply costs ≈ 0.2 pJ/op.
>
> Normalized against an 8-bit integer add, a DRAM access sits about four orders of magnitude higher (table 10.9).
>
> For inference workloads, moving from FP32 to INT8 saves 4× in raw value storage, while the two multiply costs above give an idealized multiply-energy reduction of 18.5×. End-to-end energy also includes memory, control, sensors, and idle power. Section D.2.1 formalizes the distinction behind this relationship: workloads with little arithmetic per byte are memory bound, so shrinking data movement dominates; workloads with more arithmetic per byte are compute bound, so arithmetic savings matter more. The arithmetic ratio alone cannot predict device battery life.
>
> The same physics apply at data-center scale, where distributed training systems use reduced precision to cut gradient communication overhead.

Silicon-level energy measurements explain why systems exploit quantization, but hardware efficiency alone does not determine deployment viability. The operational envelope of quantization depends on two interacting constraints across the D·A·M boundary: the algorithm's tolerance to numerical perturbation before task accuracy degrades, and the machine's ability to translate narrower bit widths into reduced energy and wall-clock speedup. Quantifying the hardware side of this trade-off requires measuring the physical energy consumed across each tier of the memory and execution hierarchy.

10.4.1.1 Energy costs

The reference costs in table 10.9 use the cheapest operation, an 8-bit integer addition, as the unit. Against a 32-bit floating-point operation the DRAM penalty shrinks to the 100–1,000× of principle 7, but against the integer arithmetic that quantization enables it widens to four orders of magnitude, which is the primary physical motivation for bit-width reduction.

Table 10.9: Relative Energy per Operation: Relative energy cost of representative arithmetic and memory operations, normalized to an 8-bit integer add. A 32-bit float add costs 30× more than its 8-bit integer counterpart, but a single 32-bit DRAM read costs 21,333.3× more, placing this DRAM access more than four orders of magnitude above the INT8 add. These per-event reference costs explain why shrinking data movement can save substantial energy; total inference energy also depends on access counts, arithmetic counts, and where operands reside (Horowitz 2014).

Operation	Bit-Width	Relative Energy
Integer Add	8-bit	1×
Float Add	32-bit	30×
DRAM Read	32-bit	21,333.3×

Figure 10.16 charts this energy divergence across arithmetic precisions and on-chip SRAM tiers, isolating how a read from an 8 KB SRAM (5.00 pJ, marked by the $\approx 167\times$ trajectory) dwarfs an 8-bit integer addition (0.03 pJ). In the reference 45 nm technology model, a 32-bit integer addition costs 0.1 pJ/op, whereas an 8-bit integer addition consumes 0.03 pJ/op. Floating-point arithmetic follows the same progression: a 32-bit floating-point addition consumes approximately 0.9 pJ/op, whereas a 16-bit floating-point addition requires 0.4 pJ/op. Relative to the 0.03 pJ INT8 add, on-chip SRAM reads span 5 to 50 pJ—a factor of 167 to 1,667×—demonstrating that operand placement within the cache hierarchy dictates energy consumption as heavily as the arithmetic format itself. Off-chip DRAM access is costlier still. At scale, these per-operation disparities compound across billions of parameters and operations. For workloads dominated by off-chip weight traffic, moving unquantized parameters across high-capacitance DRAM buses can saturate bandwidth and account for substantial accelerator energy (Patterson et al. 2021), whereas on energy-constrained edge devices, redundant bit width directly exhausts battery reserves and violates thermal limits.

	Operation	Energy (pJ)
1	Integer ADD (8b)	0.03
2	Integer ADD (16b)	0.05
3	Integer ADD (32b)	0.10
4	Integer MULT (8b)	0.20
5	Integer MULT (32b)	3.10
6	8 KB SRAM Read (32b)	5.00
7	32 KB SRAM Read (32b)	10.00
8	1 MB SRAM Read (32b)	50.00

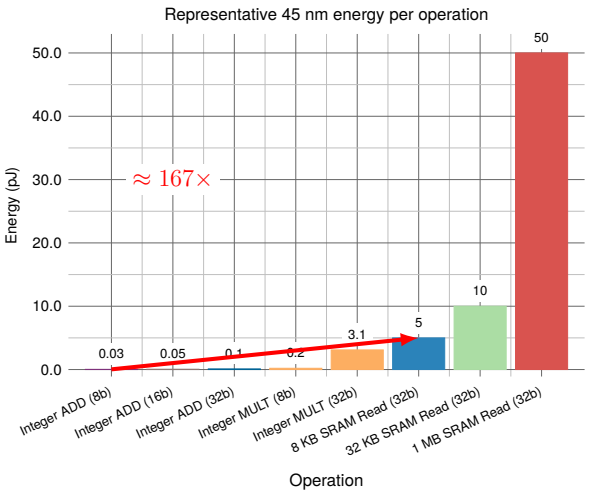


Figure 10.16: Energy per Operation by Precision: Representative 45 nm energy costs range from 0.03 pJ for an INT8 add to 3.10 pJ for an INT32 multiply and 5 to 50 pJ for the SRAM reads shown. The diagonal marks a read from the 8 KB SRAM at approximately 167 times the INT8-add energy; the complete SRAM range spans approximately 167 to 1,667 times that baseline. Energy figures from (Horowitz 2014).

D Data

A Algorithm

M Machine

The machine's memory capacity, not arithmetic, now sets the constraint.

10.4.1.2 The energy dividend: INT8 vs. FP32

The arithmetic ratios in the energy hierarchy describe an **energy dividend** per operation. In the reference technology model, an FP32 addition costs 0.9 pJ/op, while an INT8 addition costs 0.03 pJ/op, an operation-level ratio of 30×. Applying that ratio to battery life would also require accounting for memory accesses, accumulation precision, control, sensors, and idle power. When memory capacity binds, the more immediate benefit is placement: smaller values may let the model reside in a memory tier that the uncompressed model cannot fit.

Recommendation models convert the energy argument into a placement problem: quantization must shrink the dominant memory structures enough to reside in the host machine's physical memory.



Lighthouse 10.1: DLRM and embedding quantization

The DLRM lighthouse presents a compression challenge where memory capacity, rather than compute throughput or memory bandwidth, is the binding constraint (Naumov et al. 2019). Even the reference configuration in table 10.2 needs 100 GB of weights, more than one accelerator's memory, and production embedding tables can grow to terabytes that must be sharded across many devices.

For DLRM, quantization targets storage density rather than arithmetic throughput. Reducing embedding-table precision from FP32 to INT8 (or lower) shrinks the memory footprint by 4–8×, allowing larger tables to fit on fewer GPUs when accuracy and lookup kernels tolerate the lower precision. Compressing the lookup tables ensures that physical device memory can accommodate the algorithmic model.



Lighthouse 10.2: The TinyML quantization imperative

The KWS lighthouse, DS-CNN, operates at the opposite extreme of the iron law from DLRM (Y. Zhang et al. 2017). Consider a deployment that keeps weights in roughly 512 KB of SRAM. Even the purpose-built DS-CNN keyword spotter overshoots that budget in FP32: its weights occupy 800 KB. Quantized to INT8, its weights shrink to 200 KB, leaving space whose sufficiency must be checked against peak activation and runtime-buffer use. A device that reads weights directly from flash has a separate weight-storage budget; its SRAM feasibility still depends on the working set. A stricter smart-doorbell microcontroller with 256 KB SRAM leaves closer to 100 KB for model weights after runtime buffers, audio windows, and feature extraction state, motivating more aggressive INT4 or binary compression.

In FP32, even the compact DS-CNN architecture moves 4× more raw weight bytes than in INT8, and the arithmetic path can also cost more on hardware with efficient integer units. For an always-on coin-cell device, that reduction contributes to longer battery life, but sensor, wake-up, feature-extraction, and idle-power costs may dominate. Here, quantization reduces the model's data-movement term and can improve its compute term $(O/(R_{\text{peak}} \cdot \eta_{\text{hw}}))$ when the integer path is supported.

Together, DLRM and the TinyML lighthouse (10.2) illustrate the two operational extremes of this memory constraint. DLRM uses lower precision to fit embedding tables that can reach terabytes into host memory; TinyML uses lower precision to keep every inference within a tiny energy envelope and a few hundred kilobytes of on-chip SRAM.

Reducing numerical precision thus attacks physical execution costs on two distinct fronts: arithmetic latency in the ALU and byte traffic across the memory bus. How these reductions translate into wall-clock speedup depends directly on whether the workload is bound by compute throughput or memory bandwidth.

10.4.1.3 Performance gains

Figure 10.17 plots the dual benefits of INT8 conversion across three vision benchmarks (Inception v3, MobileNet v1, and ResNet v2): the right panel charts a deterministic $4\times$ contraction in weight storage (e.g., 135 MB down to 33.75 MB for Inception), while the left panel charts corresponding reductions in inference latency.

Large language models illustrate how weight precision dictates physical memory feasibility at scale.

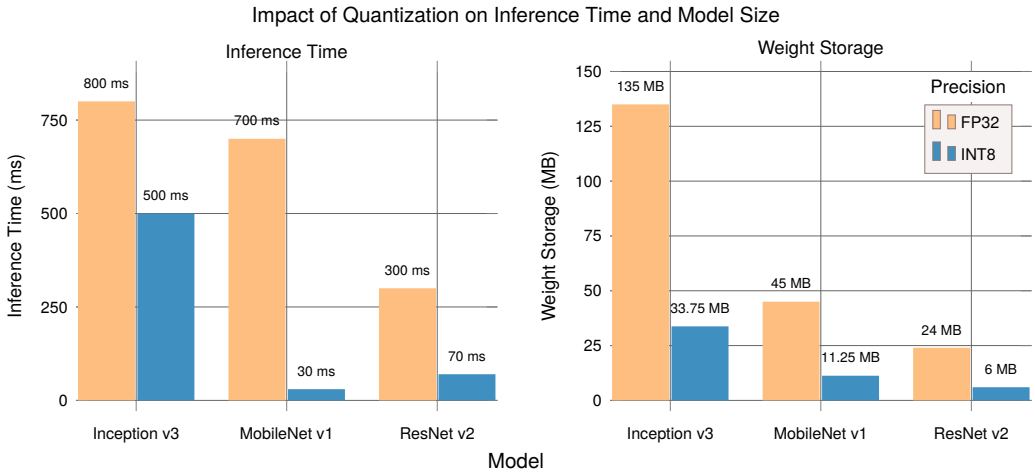


Figure 10.17: Illustrative Quantization Scenario: Paired bars separate FP32 and INT8 alternatives. The right panel shows raw weight storage, where reducing each weight from 32 to 8 bits produces an exact fourfold decrease; the left panel shows illustrative, architecture-dependent latency changes.

Napkin Math 10.1: Quantization savings

Problem: Deploying Llama 3 8B requires storing 8B parameters on-device. Calculate the memory consumption at FP16 versus INT4, and evaluate whether quantization shrinks the model footprint sufficiently to fit within single-GPU memory constraints.

FP16

- **Raw weight size**: $8 \times 10^9 \times$ 2 bytes (16-bit) = 16 GB
- **Runtime budget**: 16 GB leaves little headroom; a 24 GB-class device provides more, but peak KV-cache, activation, and workspace use still determine feasibility.

INT4

- **Raw weight size**: $8 \times 10^9 \times$ 0.5 bytes (INT4) = 4 GB
- **Runtime budget**: 4 GB leaves about 4 GB on an 8 GB device; metadata, cache, activations, and workspace must together fit that remainder.

Beyond storage savings, quantization accelerates arithmetic execution through data-level parallelism in hardware registers. Fixed-width vector execution units (SIMD) and systolic arrays process more operands per cycle when each operand occupies fewer bits.

Napkin Math 10.2: The SIMD multiplier

Problem: A compute-bound layer executes using INT8 rather than FP32 on the same processor. Determine the architectural source of arithmetic speedup under fixed register width.

Mechanism: SIMD execution. A CPU or GPU core processes data in fixed-width vector registers (for example, AVX-512 is 512 bits wide).

Math:

1. **Register width**: 512 bits.
2. **FP32 capacity**: 512/32 = 16 elements per vector instruction.
3. **INT8 capacity**: 512/8 = 64 elements per vector instruction.

Result: Switching to INT8 packs 4× more elements into the same register. Throughput Gain = INT8 elements/inst/FP32 elements/inst = 64/16 = 4×

Systems insight: The packing ratio gives an ideal 4× throughput factor, assuming comparable vector-instruction throughput, occupied lanes, and supported operators. It is a candidate bound for compute-bound layers, not a measured layer speedup; memory traffic and conversion overhead still affect execution.

Reducing numerical precision introduces trade-offs, however. Lower-precision formats can cause numerical instability and quantization noise, potentially affecting model accuracy. A uniform quantizer maps each value to the nearest of evenly spaced levels separated by a step size s (calibration derives s from the tensor's clipping range in equation 10.1, later in this chapter). For any value inside the clipping range, rounding to the nearest level misses by at most half a step, so within-range error is bounded by $[-s/2, s/2]$. Figure 10.18 plots the aggregate residual density across many tensors: most error concentrates inside that bound, while the highlighted red tails isolate the larger distortion that comes from saturation clipping and from aggregating tensors that use different step sizes. Many CNNs tolerate INT8 well, whereas large transformers often need outlier-aware or mixed-precision schemes because a few activation channels carry extreme values. Selecting the appropriate numerical precision therefore requires balancing accuracy constraints, hardware support, and efficiency gains.

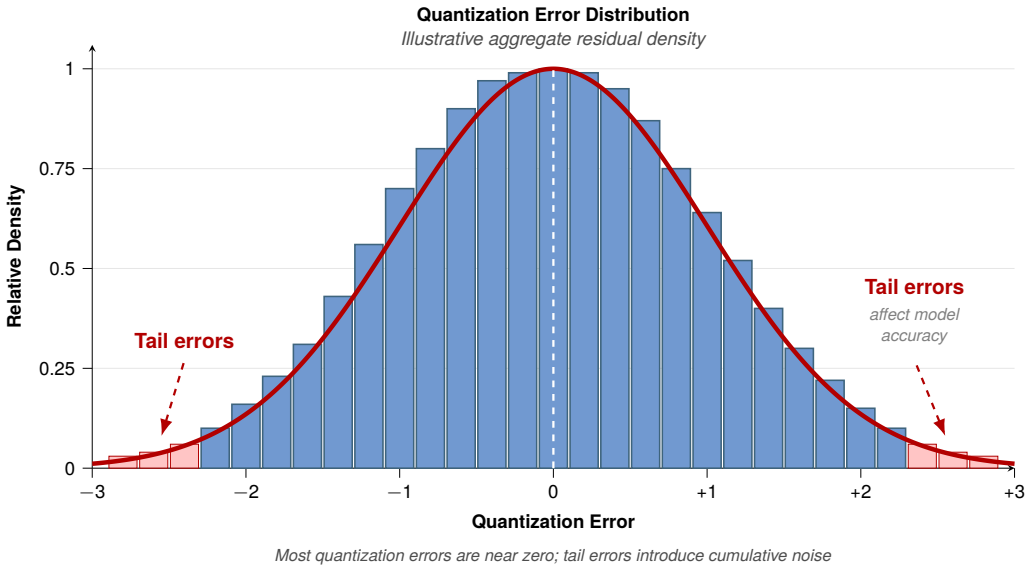


Figure 10.18: Illustrative Aggregate Quantization Residuals: Illustrative aggregate residual distribution across heterogeneous scales and clipped values. Within one unclipped uniform quantizer, rounding error is bounded by $[-s/2, s/2]$ for step size s; the displayed tails represent clipping or aggregation across different step sizes.

While managing quantization noise preserves algorithmic accuracy, the systems objective of lower precision is wall-clock speedup. The magnitude and physical origin of that speedup depend directly on whether execution is compute bound or memory bound.

Napkin Math 10.3: The quantization speedup (compute bound)

Problem: A compute-bound matrix multiplication (for example, in a transformer multilayer perceptron block) switches from FP16 to INT8. Calculate the expected throughput speedup on hardware with dedicated integer matrix units.

Math: On modern hardware with dedicated INT8 units:

1. **Integer throughput path**: The reference A100-class accelerator rates its dedicated INT8 matrix units at 624 TOPS against 312 TFLOP/s on the FP16 path (NVIDIA Corporation 2020a; Choquette et al. 2021). The peak throughput increase is that spec ratio: 624 TOPS ÷ 312 TFLOP/s ≈ 2×.
2. **Memory bandwidth**: Packed INT8 weights carry half the raw weight bytes of FP16. Their streaming time can approach half at the same achieved bandwidth, before metadata and conversion costs.
3. **Combined effect**: For compute-bound operations, the ideal throughput factor is ~2× at comparable utilization of the two arithmetic paths.

Systems insight: The speedup from quantization depends on the bottleneck. Compute-bound operations (large batch sizes, high arithmetic intensity I) can approach ~2× at comparable utilization of supported INT8 units, where the gain comes from the integer matrix path rather than reduced memory traffic. The bandwidth-bound case inverts this: halving the bytes moved (FP16 to INT8) yields up to 2×, and larger bit-width reductions scale the gain further, as the next worked example traces.

When an operation is bandwidth bound, speedup tracks the reduction in bytes transferred per token rather than peak arithmetic throughput. In autoregressive token generation with small batch sizes, memory bus traffic dominates latency.

Napkin Math 10.4: The quantization speedup

Problem: A deployment scenario calls for running a 7B-parameter large language model (LLM) on a device with 16 GB RAM. The weights are FP16 (2 bytes).

Math:

1. **Model size**: $7 \times 10^9 \times 2$ bytes = 14 GB.
2. **KV cache**: A 4,096-token FP16 KV cache uses two tensors (keys and values) across 32 layers, 32 KV heads, 128 dimensions per head, and 2 bytes per value, requiring approximately 2.1 GB.
3. **Total memory**: 14 GB + 2.1 GB = 16.1 GB. This exceeds the device capacity before OS and workspace memory.
4. **Bandwidth cost**: In this simplified full-cache-traffic model, loading weights plus the KV cache at 50 GB/s takes 323 ms per token. That is 3.1 tokens/s, too slow for chat.

Fix (INT4):

1. **Quantization**: Convert weights to INT4 (0.5 bytes).
2. **New size**: $7 \times 10^9 \times 0.5$ bytes = 3.5 GB of weights, or 5.6 GB including the unchanged FP16 KV cache.
3. **New speed**: Loading that total takes 113 ms. Speed rises to 9 tokens/s.

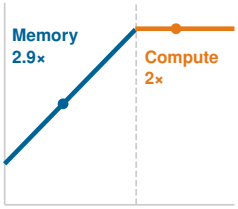
Systems insight: Weight quantization brings the modeled weights-plus-KV subtotal below capacity and yields a 2.9× speedup in the stated traffic model, below 4× because FP16 KV-cache traffic is unchanged. Deployment feasibility still requires space for metadata, activations, workspace, and the operating system, and measured speedup depends on the supported kernel path.

10.4.2 Numerical format comparison

The two worked examples bracket the payoff. On the compute-bound matrix multiplication, INT8 bought about 2× because a faster integer datapath retired more operations per cycle. On the bandwidth-bound decode, INT4 weights bought 2.9× because fewer bytes crossed the memory bus, and the unchanged FP16 KV cache held the gain below the 4× weight reduction. Both results

require an efficient execution path for the chosen representation, and formats differ in more than width.

Selecting a numerical format therefore requires balancing dynamic range and hardware execution support against memory traffic, rather than minimizing bit width blindly. Table 10.10 compares the precision formats commonly deployed across accelerator architectures, mapping raw storage footprints, arithmetic throughput gains, and target hardware relative to standard 32-bit floating point (FP32).



Quantization speedup depends on whether memory bandwidth or compute throughput is binding.

Table 10.10: Numerical Precision Formats: Storage ratios reflect raw value width; compute speed and power effects depend on hardware support and workload behavior.

Precision Format	Bit-Width	Storage Reduction (vs. FP32)	Compute Speed (vs. FP32)	Power Effect	Use Cases
FP32	32-bit	Baseline (1×)	Baseline (1×)	Baseline	Training & inference (general-purpose)
FP16	16-bit	2× smaller	Up to 2× peak on selected supported hardware	Hardware- and workload-dependent	Accelerated training, inference (NVIDIA Tensor Cores, TPUs)
BF16 (Brain Floating Point)	16-bit	2× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Training on TPUs, transformer-based models
TF32 (TensorFloat-32)	19-bit	None (stored as FP32)	Up to 8× peak on NVIDIA Ampere Tensor Cores	Hardware- and workload-dependent	Training on NVIDIA GPUs
FP8 (Floating-Point 8-bit)	8-bit	4× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Efficient training/inference (H100, AI accelerators)
INT8 (8-bit Integer)	8-bit	4× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Quantized inference (Edge AI, mobile AI, NPUs)
INT4 (4-bit Integer)	4-bit	8× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Ultra-low-power AI, experimental quantization
Binary/Ternary (1-bit/2-bit)	1–2-bit	16–32× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Extreme efficiency (binary/ternary neural networks)

At 16 bits, FP16 and BF16 halve memory storage relative to FP32 while targeting dedicated matrix execution units (such as NVIDIA Tensor Cores and Google TPUs) to accelerate arithmetic throughput. BF16 retains FP32's 8-bit exponent, preserving a dynamic range spanning roughly 10^{-38} to 10^{38} while truncating the fraction to 7 bits. In contrast, FP16 allocates 5 bits to the exponent and 10 bits to the fraction, offering greater precision but capping the maximum representable finite value at 65,504, with normal values underflowing below approximately 6.1×10^{-5} (and subnormals down to 6.0×10^{-8}). This is why the mixed-precision training recalled at the start of this chapter pairs FP16 with loss scaling and FP32 accumulation, whereas BF16's wide exponent range accommodates standard training dynamics without scaling. For inference, the same range limits decide whether activations with large outliers survive the narrower format. TensorFloat-32 (TF32) adopts a hybrid compromise: it retains the 8-bit exponent of FP32 and the 10-bit fraction of FP16, executing on Tensor Cores to accelerate matrix multiplication while storing values in standard 32-bit memory. Figure 10.19 diagrams these bit layouts across FP32, FP16, and BF16, contrasting how each format partitions the sign, exponent, and mantissa fields across the 16-bit and 32-bit boundaries to balance dynamic range against precision.

For inference workloads, INT8 precision provides a fourfold reduction in raw storage relative to FP32 when the chosen quantizer preserves acceptable prediction accuracy. Inference avoids the gradient backpropagation pass, eliminating the small-magnitude, high-dynamic-range updates that cause underflow during training. Activations and weights can still contain outliers or sensitive values, so calibration or QAT must validate mapping them onto signed 8-bit integers ($[-128, 127]$) using uniform affine or symmetric quantization scales. In bandwidth-limited inference regimes—such

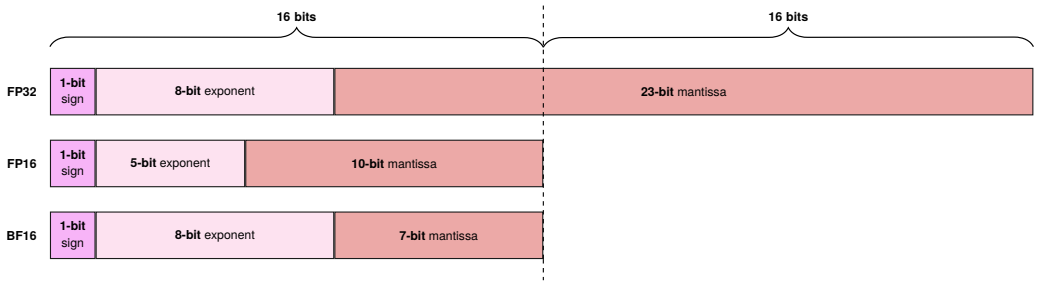


Figure 10.19: Floating-Point Bit Layouts: Bit allocation trades dynamic range against significand precision. FP32 uses 8 exponent and 23 fraction bits. BF16 retains FP32's 8-bit exponent and therefore a similar numerical range, but reduces the fraction to 7 bits. FP16 uses 5 exponent and 10 fraction bits, providing more significand precision than BF16 but a narrower range that can require loss scaling during training.

as autoregressive language model token generation—this $4\times$ footprint reduction relieves memory bus contention by cutting DRAM and cache traffic. On compute-bound operators, dedicated INT8 vector and tensor units provide higher arithmetic operation density per clock cycle and per unit of silicon area than equivalent floating-point units.

Binary and ternary formats occupy the far end of table 10.10, constraining values to $\{-1, +1\}$ or $\{-1, 0, +1\}$ so that multiplication collapses into bitwise logic or a conditional add. They promise the largest storage ratio in the table, but byte-addressed memory and training stability make them a special case that section 10.4.4 develops in full. The keyword-spotting lighthouse shows why such formats are considered at all. Under the strict ~512 KB SRAM budget, INT8 is the baseline target, and INT4, binary, or ternary representations trade additional dynamic range to meet stricter memory and power bounds.

The energy argument of section 10.4.1 carries into the format choice. That section quoted two operation-level ratios, 18.5× for a multiply and 30× for an add. A matrix unit performs both in every multiply-accumulate, so the figure that governs a format decision is their combination. A full INT8 multiply-accumulate uses roughly 20× less energy than its FP32 equivalent, on top of the $4\times$ cut in bytes moved. Accelerators equipped with native low-precision units (such as NVIDIA Tensor Cores, Google TPUs, or mobile NPUs) compound these arithmetic savings with reduced memory traffic. In contrast, general-purpose processors lacking native low-precision execution pipelines lose much of this benefit to the runtime overhead of bit unpacking, format conversion, and scalar fallback. Precision is therefore a model-hardware decision, not a software configuration flag.

10.4.3 Precision reduction strategies

Reducing numerical bit width relieves memory traffic and arithmetic energy, but naive truncation introduces quantization noise that degrades model accuracy. Deploying low-precision representations requires structured strategies that control where and how bit widths change across the computational graph.

Three approaches form an engineering decision ladder governed by training and validation cost. Post-training quantization (PTQ) reduces precision after training, requiring no retraining and minimal computational investment. Quantization-aware training (QAT) models quantization noise directly within the training loop, allowing backpropagation to adapt surviving weights and recover accuracy when calibration alone is insufficient. Specialized extreme formats push beyond standard 8-bit and 4-bit representations to binary or ternary regimes, demanding customized execution kernels and model-specific co-design. Representation choices—such as selecting INT8 versus INT4, mixing bit widths across sensitive layers, or choosing per-tensor versus per-channel calibration—are orthogonal options evaluated *within* these workflows rather than separate maturity levels. Figure 10.20 stages this deployment sequence by the engineering effort required, moving from fast post-training calibration through quantization-aware retraining to custom low-bit pipelines.

This workflow emphasizes engineering economics rather than a taxonomy of numerical formats. An engineer should evaluate post-training quantization first because it modifies representations with minimal compute and pipeline overhead. When PTQ satisfies the application latency, energy,

and quality thresholds, the optimization process halts. When post-training calibration introduces unacceptable accuracy degradation, quantization-aware training provides the recovery mechanism by allowing model weights to adapt to discretization noise during backpropagation. Format selections such as INT8 or INT4 weight-only packing, per-channel scaling, and mixed layer precisions operate within both workflows, whereas extreme sub-4-bit or ternary representations demand dedicated kernel implementations and rigorous hardware validation.

10.4.3.1 Post-training quantization

Post-training quantization (PTQ) converts weights and activation tensors from floating-point to fixed-point formats after model training completes (Choukroun et al. 2019). Because it operates directly on an existing checkpoint, PTQ avoids the compute cost and engineering overhead of full training loops. Weight-only quantization requires only a single pass over static parameter tensors, while full integer quantization calibrates activation dynamic ranges using a small, unlabelled sample of representative inputs.

The principal constraint in PTQ is unmodeled discretization noise: rounding introduces numerical error that cannot be compensated by gradient updates. While redundant architectures absorb 8-bit rounding noise with negligible accuracy loss, sensitive models or lower bit widths (such as INT4) experience degradation unless scaling granularities are refined or quantization-aware training is applied. Production runtimes such as TensorFlow Lite, Open Neural Network Exchange (ONNX) Runtime, and TensorRT provide automated calibration pipelines to profile tensor ranges and apply this conversion.

The core mechanism of PTQ is **uniform quantization**, which maps floating-point values to discrete integer levels using a consistent scaling factor. Because the interval between each quantized value is constant, uniform quantization simplifies implementation and enables efficient hardware execution. For the symmetric form shown here, s is chosen from the maximum absolute value so that the resulting integers stay within the target range. The quantized value x_q is computed as:

$$x_q = \text{round}\left(\frac{x}{s}\right)$$

where:

- x_q is the quantized integer representation,
- x is the original floating-point value,
- s is a scaling factor that maps the floating-point range to the available integer range.

Listing 10.2 demonstrates uniform quantization from FP32 to INT8, reducing the raw value payload from 32 to 8 bits per weight while measuring the resulting quantization error; scale metadata and packing overhead are excluded. Once a model is quantized, the runtime can use integer arithmetic where supported (Gholami et al. 2022).

Listing 10.2: Uniform Quantization: Converts FP32 weights to INT8, reducing raw per-weight payload from 32 to 8 bits while measuring quantization error; scale metadata and packing overhead are excluded.

```
import torch

# Original FP32 weights
weights_fp32 = torch.tensor(
    [0.127, -0.084, 0.392, -0.203], dtype=torch.float32
)
print(f"Original FP32: {weights_fp32}")
print(f"Memory per weight: 32 bits")

# Simple uniform quantization to INT8 (-128 to 127)
# Step 1: Find scale factor
max_val = weights_fp32.abs().max()
scale = max_val / 127  # 127 is max positive INT8 value

# Step 2: Quantize using our formula x_q = round(x/s)
weights_int8 = torch.round(weights_fp32 / scale).to(torch.int8)
print(f"Quantized INT8: {weights_int8}")
print(f"Memory per weight: 8 bits (reduced from 32)")

# Step 3: Dequantize to verify
weights_dequantized = weights_int8.float() * scale
print(f"Dequantized: {weights_dequantized}")
print(
    f"Quantization error: "
    f"{(weights_fp32 - weights_dequantized).abs().mean():.6f}"
)
```

An alternative, nonuniform quantization, assigns finer-grained precision to numerical ranges that are more densely populated, which can preserve accuracy for models whose weight distributions concentrate around specific values. Nonuniform schemes require more complex calibration and are less common in production, but they can be effective for models particularly sensitive to precision changes.

PTQ is a useful first experiment for many vision, language, and speech models, but sensitivity varies by architecture, layer, and distribution. When a uniform PTQ configuration misses the quality target, the next options include finer granularity, mixed precision, weight-only methods, better calibration, or QAT.

Calibration Quantization calibration estimates the real-valued clipping range $[\alpha, \beta]$ for each tensor; from here on, α and β denote its lower and upper bounds. An ill-fitting range either saturates critical values through clipping or squanders available integer levels on sparse outliers. Calibration minimizes representation error over the sampled distribution, although it does not guarantee bounds on out-of-distribution deployment inputs.

Because activation tensors vary with input data, profiling requires running an unlabelled calibration dataset through the forward pass to capture dynamic ranges. Figure 10.21 diagrams this offline conversion pipeline: inputs from the pretrained model and representative calibration dataset pass into a profiling stage to establish dynamic ranges, which parameterize the quantization kernel to yield the final quantized model. Complementing this flow, algorithm 10.1 formalizes the layer-by-layer calibration procedure where activation observers record value distributions to compute scale and zero-point metadata.

Algorithm 10.1 Post-training quantization calibration

Require: pretrained model f_θ; representative calibration set $\mathcal{C}_{\text{cal}}$; bit width b; granularity (layer/group/channel); method (max, entropy, percentile)
Ensure: quantized weights and per-group range metadata $\{(\alpha_g, \beta_g, s_g, z_g)\}$
attach observers to weights and selected activation tensors, at the chosen granularity
for each batch in $\mathcal{C}_{\text{cal}}$ **do** ▷ *calibration pass*
    run f_θ in inference; record per-group value distributions
end for
for each quantization group g **do**
    select clipping range $[\alpha_g, \beta_g]$ by the chosen method
    derive scale s_g, zero-point z_g mapping $[\alpha_g, \beta_g]$ into the b-bit integer range
end for
quantize static weights; export activation range metadata for inference
if accuracy falls below the production threshold, widen ranges, change granularity, or move to QAT

The single calibration pass over $\mathcal{C}_{\text{cal}}$ avoids retraining, and FP32-to-INT8 weight quantization cuts raw stored weight values by 4× before scale, zero-point, and packing metadata. The price is that the static activation ranges fixed in the loop risk saturation or wasted integer levels when the calibration data misses deployment tails. The quantization step then converts model parameters to the lower-precision format, producing the final quantized model.

Mapping an unnecessarily wide real-valued interval to the 256 discrete bins of an 8-bit integer wastes numerical resolution on empty range, whereas clipping too aggressively truncates signal. Three calibration methods balance this clipping-versus-resolution trade-off: **max** sets bounds to the absolute extremum (preserving extreme values at the cost of vulnerability to outliers), **entropy** selects bounds by minimizing the Kullback–Leibler (KL) divergence between original and quantized distributions, and **percentile** retains a chosen fraction (such as 99.9 percent) and clips the remaining tail. Figure 10.22 charts this dynamic range trade-off for a ResNet-50 layer: the dense bulk of activations clusters below 1.1, while a sparse outlier tail stretching past 2.0 motivates testing a candidate clipping boundary near 1.1 to allocate more INT8 bins to the bulk. Choosing a percentile boundary requires cumulative activation counts; the illustrated boundary is not assigned a percentile rank. Task validation must establish whether clipping that tail preserves important rare cases.

Calibration ranges can be **symmetric** (a zero-centered range, typically with zero-point zero) or **asymmetric** (one scale with a generally nonzero zero-point, useful when distributions are skewed). The choice of method and range significantly affects quantized model accuracy.

Tuning quantization ranges Once calibration identifies clipping boundaries $[\alpha, \beta]$, the range must be mapped onto integer endpoints. Figure 10.23 contrasts the two primary mapping schemes: the symmetric layout on the left forces real zero to integer zero by mapping $[-1, 1]$ directly onto $[-127, 127]$, whereas the asymmetric layout on the right accommodates skewed activation intervals $[-0.5, 1.5]$ across $[-128, 127]$ by shifting the integer zero-point offset z.

Symmetric quantization simplifies integer matrix multiplication kernels because the zero-point is zero ($z = 0$), eliminating cross-term additions during hardware accumulation. However, for activations following asymmetric distributions (such as post-ReLU feature maps where all values are non-negative), symmetric quantization wastes roughly half of the integer dynamic range on unused negative values. Asymmetric quantization recovers this dynamic range by mapping $[\alpha, \beta]$ to $[0, 2^b - 1]$, but requires runtime hardware to compute offset corrections ($z \sum_k W_{ik}$ for output row i) during multiply-accumulate operations. Either mapping is fully specified by two numbers per range, a scale and a zero-point, and the uniform quantizer introduced for PTQ is the special case whose zero-point is zero.

Napkin Math 10.5: Calculating scale and zero-point

The affine quantization formula: An affine map sends a floating-point range $[\alpha, \beta]$ that includes zero to an integer range $[0, 2^b - 1]$ (for example, UINT8) via the following construction.

Analysis: Affine quantization establishes a linear mapping $x \approx s(x_q - z)$, where s is the **scale** (step size) and z is the **zero-point** (integer value corresponding to real zero). The mapping involves three computational steps:

1. **Calculate scale** (s): Divide the real range by the integer range using equation 10.1.

$$s = \frac{\beta - \alpha}{2^b - 1} \tag{10.1}$$

2. **Calculate zero-point** (z): Shift the range so that real zero maps to an integer using equation 10.2.

$$z = \text{round}\left(\frac{-\alpha}{s}\right) \tag{10.2}$$

3. **Quantize** $(x \to x_q)$: Divide by the scale, add the zero-point, then round and clamp to the integer range using equation 10.3.

$$x_q = \text{clamp}\left(\text{round}\left(\frac{x}{s} + z\right), 0, 2^b - 1\right) \tag{10.3}$$

Scenario: Consider quantizing a 2×2 activation matrix with values ranging from $\alpha = -1$ to $\beta = 3$, and the target precision is UINT8 $(b = 8)$:

$$\mathbf{X} = \begin{pmatrix} -1 & 1 \\ 0 & 3 \end{pmatrix}$$

- **Range**: $\beta - \alpha = 4$.
- **Steps**: $2^8 - 1 = 255$.
- **Scale**: $s = \frac{4}{255} \approx 0.0157$.
- **Zero-point**: $z = \text{round}(-(\alpha)/s) = \text{round}\left(-(-1)/\frac{4}{255}\right) = \text{round}(63.75) = 64$.

Applying equation 10.3 element-wise maps each floating-point entry to an integer bin:

$$\mathbf{X}_q = \begin{pmatrix} 0 & 128 \\ 64 & 255 \end{pmatrix}$$

Systems insight: The real value 0.0 is represented exactly by the integer 64. This is critical because it ensures that zero-padding (common in CNNs) is represented exactly, preventing "quantization drift" where padding introduces nonzero noise.

Granularity Quantization granularity governs how broadly scale factors and zero-points are shared across a tensor, balancing representation resolution against metadata overhead and execution complexity. In convolutional filters or transformer projection matrices, weight magnitudes often vary widely across individual channels. Sharing a single scale factor across an entire tensor risks collapsing the dynamic range of small-magnitude channels to zero; conversely, assigning scales per channel preserves local variance at the expense of tracking additional scale vectors during matrix multiplication.

When weight channels exhibit divergent standard deviations, sharing a single clipping threshold across an entire layer either collapses small-magnitude channels or clips wider channels. Figure 10.24 charts this variance across convolutional channels (Filters 1 through d_{out}): a global layerwise threshold (red dashed lines) spans the widest distribution at the cost of resolution in narrower filters,

whereas per-channel clipping boundaries (blue dashed lines) adapt tightly to each individual filter's dynamic range.

Quantization granularity trades calibration and metadata overhead against representation error. More local ranges can preserve information when channels differ, but they do not improve accuracy automatically and may lack an efficient kernel path. Each additional scale also has to be stored, loaded, and applied at the granularity supported by the backend. Table 10.11 summarizes four common levels, from one shared layer range to local ranges within a filter.

Table 10.11: Quantization Granularity Levels: Granularity trades calibration and metadata overhead against representation error, from a single per-layer range to per-filter sub-ranges.

Level	Range sharing	Trade-off
Layerwise	One range per layer	Simple but suboptimal when filter ranges vary widely
Groupwise	Filters grouped with shared ranges	Used in Q-BERT (Shen et al. 2020) for transformer attention layers
Channelwise	One range per filter	Common default; balances quantization error against scale metadata and implementation overhead
Sub-channelwise	Ranges within each filter	More local ranges can reduce quantization error but increase metadata and implementation overhead

Channelwise quantization preserves accuracy over layerwise quantization when channel dynamic ranges differ, at the cost of storing additional scale vectors and executing per-channel rescaling in the kernel pipeline. Beyond scale granularity, execution efficiency depends on the distinction between static weights and dynamic activations, which impose fundamentally different memory and latency constraints on the deployment system.

Weights vs. activations Executing low-precision inference requires transforming floating-point inputs before kernel execution and requantizing intermediate accumulator outputs. During integer matrix multiplication, multiplying two 8-bit integers produces a 16-bit product. Summing these products across an inner dimension of length K causes bit-growth that quickly overflows 8-bit registers; hardware therefore accumulates into 32-bit integer registers (`INT32`). Before feeding the result into subsequent layers, requantization rescales the 32-bit accumulated sum by the layer's effective scale factor ($s_{\text{eff}} = \frac{s_W \cdot s_X}{s_Y}$) and clamps the values back into the 8-bit integer range. Figure 10.25 diagrams this pipeline across four stages: violet quantizers convert floating-point activations and weights into INT8 operands, a blue core executes integer matrix multiplication, a red stage buffers the 32-bit accumulated sum, and a green stage requantizes the result back to INT8 before non-linear activation.

↪ How a single Tensor Core matrix instruction takes low-precision input tiles into higher-precision accumulator registers is developed in section 11.4.3.

Activation quantization maps layer outputs to a lower-precision representation during inference. It reduces intermediate payloads and can reduce arithmetic cost on hardware with an efficient integer path, but it also introduces error between layers. For example, a CNN may convert FP32 feature maps to INT8 before later convolutions. The benefit is greatest when consecutive operators remain on the integer path; isolated quantized operators may pay conversion cost without reducing enough work. Whether activation quantization improves latency therefore depends on supported kernels, scale handling, tensor lifetimes, and any quantize or dequantize boundaries that remain in the exported graph.

Activation-aware methods such as **activation-aware weight quantization (AWQ)**[17] target weight traffic in LLM inference. This approach is relevant to the GPT-2/Llama lighthouse when decoding is limited by loading model weights. By protecting a small fraction of salient weight channels based on activation magnitude, AWQ supports INT4-class weight quantization while controlling task degradation. Packed low-bit weights can reduce the memory traffic required during token generation when the runtime provides a matching kernel (Lin et al. 2024).

17 **Activation-aware weight quantization (AWQ)**: Salience is determined by activation magnitude, not weight magnitude—a distinction that matters because a small weight multiplied by a large activation produces a large output contribution. AWQ protects about 1 percent of salient weight channels through activation-aware scaling while quantizing most weights to low-bit formats, so a 7-billion-parameter FP16 weight set that would occupy about 14 GB can move toward an INT4-class weight footprint of about 3.5 GB before metadata, scales, and kernel-specific packing overheads (Lin et al. 2024).

Static vs. dynamic quantization Beyond the shape and granularity of the clipping range, system design dictates *when* activation ranges are computed. Activation quantization divides into two primary execution models: static quantization and dynamic quantization.

In **static quantization**, the clipping range is precalculated during offline calibration and remains fixed during inference. This approach introduces zero runtime range-estimation overhead: inputs

are scaled by fixed factors and directly fed to integer execution units. However, static ranges risk saturation or underflow whenever production inputs deviate from the calibration distribution (Jacob et al. 2018; Yao et al. 2021).

Dynamic quantization instead calculates scaling factors and zero-points dynamically from runtime activation tensors. Adapting the clipping window to each input reduces mismatch with a fixed calibration range. Outliers can still consume resolution, and a policy that clips tails can still saturate values. The trade-off is runtime memory and latency overhead: the hardware must perform reduction passes over intermediate activation tensors to extract minimum and maximum values before integer arithmetic can begin. On memory-bandwidth-bound operators, this reduction pass can erode the execution advantage of lower precision.

These timing and granularity decisions interact with the broader choice of quantization methodology. Table 10.12 sets the two methodology choices, post-training quantization and quantization-aware training, beside dynamic quantization. Dynamic range computation is a timing choice orthogonal to PTQ versus QAT (dynamic PTQ is common), but it belongs in the comparison because it answers a third binding question, input variability.

Table 10.12: Quantization Method Comparison: The choice turns on three binding questions, namely engineering budget, accuracy threshold, and input variability, rather than on a feature checklist.

Method	Engineering effort	Accuracy preservation	Input adaptability	Reach for it when
Static PTQ	Low: no retraining	Model-dependent; parameters do not adapt	Fixed calibration range	a calibrated model meets the measured accuracy and performance targets
Static QAT	High: additional training with quantization simulation	Often stronger when PTQ falls short	Fixed calibration range	PTQ misses the accuracy target and the training budget allows
Dynamic quantization	Moderate: ranges recomputed at runtime	Model-dependent; range fits each input	Per-input range	activation ranges vary widely across inputs and runtime overhead is acceptable

PTQ serves as the low-effort baseline; QAT spends training cycles when PTQ degrades task accuracy, while dynamic quantization expends runtime memory traffic and compute to track input-dependent dynamic ranges.

Checkpoint 10.3: Calibration and range choices

Quantization succeeds only when the range policy matches deployment data and runtime constraints.

Range design

- ☐ **Calibration data**: explain how an unrepresentative calibration set creates clipping or wastes integer levels.
- ☐ **Granularity**: compare per-tensor and per-channel ranges by the error and metadata trade-off each creates.
- ☐ **Static vs. dynamic ranges**: select a range policy when activation distributions shift across inputs and runtime overhead is constrained.

System estimate

- ☐ **Quantization payoff**: for a 7B-parameter model served from HBM, estimate FP16 versus INT4 raw weight memory and the ideal weight-traffic speedup bound. Explain why unchanged KV traffic and conversion overhead can reduce the realized decode-throughput gain.

PTQ in practice Post-training quantization balances deployment turnaround against accuracy control. PTQ requires no gradient backpropagation or retraining, establishing it as the standard

initial step in deployment optimization pipelines. Whether offline calibration is sufficient depends on the architecture, calibration data distribution, numerical quantizer design, and target hardware constraints.

The primary limitation is that PTQ freezes model parameters, preventing compensation for accuracy loss. When a quantized model falls below the production threshold, mitigations include recalibrating, refining scale granularity, adopting mixed precision, or escalating to quantization-aware training. QAT integrates precision constraints directly into the training graph using a **straight-through estimator (STE)** (section 10.4.3.2) to pass gradients through nondifferentiable rounding operators. This enables weights to adapt to low-bit representations during retraining, recovering accuracy at the cost of additional training compute (O_{train}) and hyperparameter tuning.

10.4.3.2 Quantization-aware training

Quantization-aware training incorporates numerical rounding noise directly into the optimization graph. Figure 10.26 stages this workflow from the top down: an initial pretrained model undergoes quantization before entering a retraining and fine-tuning stage driven by the training dataset, yielding a calibrated low-precision model whose parameters have adapted to rounding noise.

Rather than training from scratch or cold-starting scale parameters, production workflows frequently initialize QAT using scale factors derived during an initial PTQ calibration pass. Figure 10.27 sequences this two-phase pipeline: an upper PTQ phase feeds calibration data into the pretrained model to establish initial scale metadata, passing the resulting PTQ checkpoint into a lower QAT phase where full training data drives fine-tuning to yield the final robust model.

Training mathematics During forward propagation, weights and activations are quantized and dequantized to mimic reduced precision. Let x be a full-precision value, s the scaling factor that maps floating-point values into a lower-precision range, and x_{fake} the simulated (quantized, then dequantized) value. Ignoring clipping and the zero-point for this simplified expression, the process is represented as:

$$x_{\text{fake}} = \text{round}\left(\frac{x}{s}\right) \times s$$

where x_{fake} represents the simulated quantized value, x denotes the full-precision weight or activation, and s is the scaling factor mapping floating-point values to lower-precision integers.

Although the forward pass executes with simulated quantization, computing gradients requires backpropagating through the rounding operator $\text{round}(\cdot)$. Because rounding is a piecewise-constant step function, its mathematical derivative is zero almost everywhere and undefined at step transitions, which would zero out gradients and halt gradient descent. The Straight-Through Estimator (STE) resolves this bottleneck by substituting the identity function for the rounding operator during backpropagation (Bengio, Léonard, et al. 2013).[18]

Integrating quantization effects during training lets the model adapt its weights and activation ranges to simulated low-precision error. QAT can recover quality that a post-training route loses, particularly at aggressive bit widths, but the advantage depends on the model, quantizer, calibration, and training procedure (Krishnamoorthi 2018).

Fake quantization nodes and implementation QAT implementation relies on fake quantization operations that simulate quantization during forward propagation while maintaining full precision for gradient computation. These operations insert quantize-dequantize pairs into the computational graph, creating a training-time simulation of inference-time behavior.

A fake quantization node must preserve the same errors the inference runtime will see, so its three operations model the deployment path during training:

1. **Quantization**: Map floating-point value to discrete quantization level
2. **Clipping**: Enforce range constraints based on bit width
3. **Dequantization**: Convert back to floating-point for subsequent operations

Mathematically, for symmetric quantization with bit width b, given a floating-point input value x:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -2^{b-1}, 2^{b-1}-1\right)$$
$$x_{\text{fake}} = x_q \times s$$

[18] **Straight-through estimator (STE)**: Proposed by Bengio, Léonard, et al. (2013), the STE substitutes the identity function for the true gradient of rounding, which is zero almost everywhere because rounding is piecewise constant. The common identity STE copies the upstream gradient through the quantizer. It is a biased heuristic rather than the true derivative.

where x_q is the quantized integer level, $s = \frac{\max(|x|)}{2^{b-1}-1}$ is the scale factor computed from the input distribution, and x_{fake} represents the fake-quantized output that mimics INT8 values but remains in floating-point format.

For asymmetric quantization supporting unsigned integers, the node applies the scale, zero-point, and quantize steps of equation 10.1, equation 10.2, and equation 10.3 to the observed range (extended to include zero) and then dequantizes:

$$
\begin{aligned}
s &= \frac{\max(x) - \min(x)}{2^b - 1} \\
z &= \text{round}\left(-\frac{\min(x)}{s}\right) \\
x_q &= \text{clip}\left(\text{round}\left(\frac{x}{s} + z\right), 0, 2^b - 1\right) \\
x_{\text{fake}} &= (x_q - z) \times s
\end{aligned}
$$

where z is the zero-point offset enabling asymmetric range representation and the observed range $[\min(x), \max(x)]$ serves as the clipping range $[\alpha, \beta]$.

Listing 10.3 shows the QAT forward path: fake quantization is applied to inputs and weights before convolution, simulating INT8 numerics while retaining floating-point tensors for training.

Listing 10.3: QAT Convolution Forward Pass: Fake-quantizes activations and weights before convolution to simulate deployment numerics during training.

```
# Forward pass with fake quantization
def qat_conv_forward(x, weight):
    # Fake quantize input activations
    x_scale = compute_scale(x, bits=8, symmetric=False)
    x_zero = compute_zero_point(x, x_scale, bits=8)
    x_quant = fake_quantize(x, x_scale, x_zero, bits=8)

    # Fake quantize weights (typically symmetric)
    w_scale = compute_scale(weight, bits=8, symmetric=True)
    w_quant = fake_quantize(weight, w_scale, zero=0, bits=8)

    # Convolution with fake-quantized values
    output = conv2d(x_quant, w_quant)
    return output
```

During backpropagation, fake quantization nodes approximate gradients through a clipping-aware straight-through estimator:

$$
\frac{\partial x_{\text{fake}}}{\partial x} = \begin{cases} 1 & \text{if } x \in [\alpha, \beta] \\ 0 & \text{otherwise} \end{cases}
$$

Within the clipping range $[\alpha, \beta]$, the estimator treats quantization as an identity mapping, passing the upstream gradient $\frac{\partial \mathcal{L}}{\partial x_{\text{fake}}}$ unchanged to master weight x. Beyond clipping boundaries, the derivative evaluates to zero, blocking gradients from saturated outliers that would otherwise destabilize optimizer updates.

In practice, frameworks like PyTorch and TensorFlow implement fake quantization as custom autograd operators whose forward pass performs the quantize-dequantize round trip while the backward pass applies an STE. Scale handling depends on the method: observers may track distributions and later freeze, or scales may be learned. Batch normalization also requires deployment-consistent handling: workflows fold batch normalization into preceding convolution weights ($\mathbf{W}_{\text{fused}} = \frac{\gamma}{\sigma_{\mathcal{B}}}\mathbf{W}$) where $\sigma_{\mathcal{B}}$ is the square root of the sum of running variance and the numerical stabilizer. Folding also adjusts the convolution bias using the running mean and the batch-normalization shift. Freezing running statistics before export allows fake-quantization scales to match the static fused parameters executed during inference.

[19] **Quantization-aware training (QAT)**: QAT simulates low-precision behavior during training or fine-tuning so weight updates can adapt to clipping and rounding error. BERT studies report that training-aware or Hessian-aware methods can remain closer to full-precision General Language Understanding Evaluation (GLUE) baselines than simpler post-training routes in their evaluated settings (Zafrir et al. 2019; Shen et al. 2020). The exact gap depends on model, task, quantizer, bit width, training budget, and evaluation distribution; a closer benchmark result is evidence for trying QAT, not a guarantee that its added training cost or exported runtime will meet a production target.

QAT trade-offs QAT's[19] primary advantage is additional control over quality at low precision. By incorporating simulated quantization noise during training or fine-tuning, the model can adapt to the deployed numerical path. Processors with dedicated integer units can then exploit INT8 arithmetic for faster or lower-energy inference when the exported operators are supported (Wu et al. 2020; Gholami et al. 2022). The benefit must be measured against PTQ for the same model and target, including the extra training cost and any difference in calibration, graph coverage, or kernel selection.

The cost is additional engineering complexity. QAT inserts simulated quantization into training, so teams must validate quantization schemes, scale behavior, and accuracy recovery on the target model. This overhead can make QAT less practical for very large models when training budgets are already constrained. Post-training routes provide a complementary path that eliminates this retraining overhead when training budgets are constrained (Choukroun et al. 2019).

In practice, the choice between PTQ and QAT follows a simple decision rule. Start with PTQ and measure accuracy on the validation set. If the exported PTQ model meets quality and deployment targets, additional QAT cost needs a separate benefit to justify it. If PTQ misses the accuracy threshold and training resources permit, test QAT to recover the gap. A hybrid approach, starting with PTQ calibration and applying QAT fine-tuning only for accuracy-critical layers, often provides a useful balance.

PTQ and QAT commonly target 8-bit or 4-bit precision, but retained accuracy depends on the model, task, and quantization method. Some deployment scenarios, however, demand even more aggressive compression, pushing precision to the absolute limits of what neural networks can tolerate.

10.4.4 Extreme quantization

Extreme quantization pushes numerical precision down to 1 or 2 bits per value for deployment environments where even INT8 or INT4 cannot satisfy memory or energy budgets (Courbariaux et al. 2015; Zhu et al. 2017). Binarization constrains weights and activations to two values, typically encoded as $\{-1, +1\}$. In digital logic, mapping -1 to binary 0 and $+1$ to binary 1 renders scalar multiplication isomorphic to the bitwise `XNOR` operation. As a result, expensive floating-point or integer MAC units are replaced by bitwise logic and population count (`popcount`) operations (Rastegari et al. 2016). A 64-bit register packs 64 binary values. A 64-bit `XNOR` instruction represents 64 binary products in parallel, with instruction latency determined by the processor, and a subsequent hardware `popcount` instruction counts the matching bits to accumulate the dot product $(2 \times \text{popcount}(\mathbf{w}\ \text{XNOR}\ \mathbf{a}) - 64)$. This eliminates hardware multipliers entirely. However, 1-bit representations severely constrain model expressiveness, leading to steep accuracy degradation on complex perceptual and language tasks (Hubara et al. 2018).

Ternarization mitigates this capacity loss by allowing three values: $\{-1, 0, +1\}$ (Zhu et al. 2017). The zero state introduces sparsity and simplifies arithmetic: multiplying an activation by $+1$ requires an addition, multiplying by -1 requires a subtraction, and multiplying by 0 is skipped entirely. This eliminates general-purpose multipliers, reducing the arithmetic unit to a conditional adder-subtractor accumulator. However, ternary values cannot be packed at one bit per value. They require at least two bits per weight in standard byte-aligned memory (wasting one of four possible 2-bit states) or multi-trit encoding (such as packing five trits into a single byte, since $3^5 = 243 < 256$) that introduces runtime decoding overhead. The arithmetic gains of multiplier-free execution must outweigh the instruction overhead required to decode and unpack the weights on the target architecture.

Both binary and ternary quantization introduce step functions whose derivative is zero almost everywhere ($\frac{d}{dx}\text{sign}(x) = 0$ for $x \neq 0$), causing backpropagation gradients to vanish. To train these representations, quantization-aware training again relies on the straight-through estimator of section 10.4.3.2, whose gradients reach full-precision latent weights that accumulate small optimizer steps until they cross the quantization thresholds.

10.4.4.1 Challenges and limitations

Translating sub-byte precision into actual wall-clock speedups is constrained by standard processor architectures. Native support for binary or two-bit matrix operations varies across processors and

kernel libraries. On a target without a matching sub-byte path, executing these representations requires bit-shifting, masking, or unpacking routines. The instruction overhead of this bit manipulation frequently offsets memory-bandwidth savings unless workloads run on custom bit-serial accelerators, field-programmable gate arrays (FPGAs), or specialized hardware with native bitwise execution units (Umuroglu et al. 2017).

Model quality presents an equally severe barrier. Extreme quantization dramatically contracts the representational capacity of a network. While low-entropy workloads such as KWS on a microcontroller SRAM budget may tolerate binary or ternary weights, complex perceptual and generative models suffer steep accuracy degradation. Binary and ternary quantization become viable candidates only when higher-precision alternatives such as INT8 or INT4 fail to satisfy physical memory or energy budgets, and when the deployment hardware can execute bit-packed operations natively (Rastegari et al. 2016; Hubara et al. 2018; Zhu et al. 2017; Umuroglu et al. 2017). Production deployment requires evaluating binary and ternary baselines against INT8, INT4, structured pruning, and compact architectural redesigns on the identical workload, measuring total binary artifact size, end-to-end latency, energy per inference, and task accuracy on target hardware.

The governing engineering check is whether bit width, hardware arithmetic paths, and calibration or training choices directly satisfy physical deployment constraints.

Checkpoint 10.4: Quantization and precision checkpoint

Quantization design review criteria:

- ☐ **INT8 memory vs. energy**: Explain why INT8's per-operation energy saving can exceed 4× while its end-to-end energy saving is usually smaller.
- ☐ **QAT vs. PTQ**: Evaluate the architectural advantages of quantization-aware training over post-training quantization, identifying the accuracy and retraining cost trade-offs that justify auxiliary optimization.
- ☐ **Bit-width speedup**: Identify the conditions under which weight-traffic reduction approaches a proportional latency gain in autoregressive decoding, and explain how KV traffic or dequantization can limit that gain.

Calculations of parameter counts, theoretical FLOPs, and nominal bit depths establish upper bounds on potential efficiency gains. Yet these paper metrics measure mathematical properties rather than physical execution time. Translating parameter elimination or bit-depth reduction into observed latency and energy improvements requires confronting the physical hardware interface.

In production environments, a persistent gap emerges between theoretical projections and measured runtime speedups. Consider an illustrative model that is pruned structurally to half its weights and then quantized from FP32 to INT8. The stored weight bytes fall by 2× from pruning and 4× from the narrower values, 8× in all. Reading that byte ratio as a speedup is the back-of-envelope projection, and it holds only for a workload limited purely by weight traffic in which every kernel exploits both changes. The scenario assumes that only 1.5× is realized. That is roughly 18.8 percent of the paper target. The scenario illustrates why *theoretical compression does not imply proportional speedup*. Optimization must extend beyond the model graph to how operations execute across the physical memory hierarchy.

The gap arises from several architectural and runtime mismatches. Sparse matrices stored in dense formats waste memory bandwidth transferring zeros—hardware cannot skip unindexed zeros. Operations that could execute concurrently stall sequentially when scheduling or runtime dependencies prevent overlap. Bridging the divide between paper reduction and measured hardware speedup is the domain of the third optimization dimension, efficient **hardware implementation**. This dimension evaluates whether structural and precision optimizations translate into real execution gains once computational graphs interact with physical execution units.

10.5 Hardware Implementation

Hardware implementation starts from the execution trace. Profiling reveals where theoretical speedups vanish in practice: sparse tensors continue to move through dense kernels, reduced-

precision operators trigger un-fused type conversions that flood the memory bus, and small layers spend more time launching kernels and moving intermediates than doing arithmetic. The model has become smaller on paper, but the execution trace still asks the machine to perform an inefficient sequence of memory accesses and operations.

Hardware implementation bridges this divide by mapping compressed computational graphs directly onto physical execution resources. Measured bottlenecks dictate the appropriate implementation technique: kernel fusion consolidates memory-bound operator chains, custom dequantization loops keep packed low-precision weights inside cache, and hardware sparsity engines align with tensor layouts to bypass zeroed arithmetic.

10.5.1 Kernel fusion

Kernel fusion (also called operator fusion) is the framework's main defense against the memory wall, and compression changes what it has to work with. Folding batch normalization into a pruned convolution, keeping a dequantize step inside the matrix-multiplication kernel that consumes it, and choosing which adjacent operators of a rewritten graph can share one kernel are all fusion decisions that only exist once the graph has been compressed. Consider a typical layer: convolution followed by batch normalization followed by rectified linear unit (ReLU). Without fusion, each operation writes its output to accelerator global memory, then the next operation reads that output back. Three memory round-trips occur for what could be computed entirely in fast on-chip registers. By fusing these operations into a single kernel, compilers and inference engines eliminate the redundant memory transactions, improving both throughput and latency on memory-bound workloads (Chen et al. 2018; NVIDIA 2024c).

↩ Kernel fusion, which keeps intermediates in registers or on-chip memory instead of round-tripping them through HBM, is established in section 7.3.1.

Modern neural networks consist of sequences of operations such as convolution, batch normalization, activation functions, and element-wise operations. When executed independently, each operation requires four steps:

1. Loading input tensors from global memory
2. Performing computation
3. Writing output tensors back to global memory
4. Launching the next kernel

The read-compute-write cycle creates memory bandwidth bottlenecks for operations with low arithmetic intensity (FLOP/byte). For an ideal sequence of n_{ops} operations over the same intermediate tensor of M_{tensor} bytes, the unfused traffic is:

$$D_{\text{vol,unfused}} = 2\,n_{\text{ops}} M_{\text{tensor}}$$

where each operation reads and writes the intermediate once. If the sequence is legal to fuse and intermediates stay on chip, the traffic can approach:

$$D_{\text{vol,fused}} = 2M_{\text{tensor}}$$

by reading the intermediate input once and writing its final output once. Weights, auxiliary inputs, cache effects, and register spills are outside this simple bound. Fusion also has limits of its own. Data dependencies, tensor shapes, register pressure, and cache capacity limit how much can legally be fused, and over-fusing can reduce occupancy or force spills back to memory. Several inference patterns nevertheless approximate the bound closely enough for fusion to matter.

Convolution-BatchNorm-ReLU fusion

The common Conv-BN-ReLU pattern in listing 10.4 can replace three launches and intermediate round-trips with one fused path when the compiler can legally fold batch normalization and combine the activation.

Listing 10.4: Conv-BN-ReLU Fusion: Combining three operations into a single kernel reduces the idealized intermediate-tensor stream from 6 transfers to 2; weights, auxiliary inputs, and spills are outside this count.

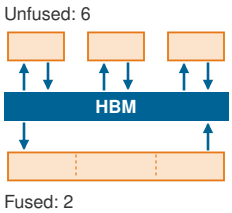
```
# Pseudocode: runtime APIs and fusion legality are
# implementation-dependent.
# === UNFUSED: 3 kernel launches, 6 memory transfers ===
conv_out = conv2d(input, weight)
bn_out = batch_norm(conv_out, ...)
relu_out = relu(bn_out)


# === FUSED: 1 kernel launch, 2 memory transfers ===
def conv_bn_relu_fused(input, weight, gamma, beta, mean, var):
    # Read input and weight once
    conv = conv2d(input, weight)

    # Apply batch norm in registers (no memory write)
    bn = gamma * (conv - mean) / sqrt(var + eps) + beta

    # Apply ReLU in registers (no memory write)
    output = relu(bn)

    # Write final result once
    return output
```



The idealized intermediate stream falls from six transfers to two.

The intended function is preserved, although batch-normalization folding changes the arithmetic sequence. In the idealized intermediate stream, traffic drops from 6 transfers to 2 transfers (3× reduction).

The same principle extends beyond CNNs. GEMM bias-activation fusion eliminates intermediate writes in transformer linear layers by computing element-wise operations in registers immediately after each matrix multiplication output element. Attention tiling, as in FlashAttention,[20] reduces HBM traffic by processing attention in SRAM-sized tiles and avoiding materialization of the full $S \times S$ attention matrix.

A ResNet-50 layer with 256 channels and a 28×28 spatial size puts numbers on this. Its feature map occupies 802.8 KB, so the four intermediate transfers that fusion removes amount to $4 \times 256 \times 28 \times 28 \times 4$ bytes $\approx$ 3.2 MB per layer. The full accounting also counts weights and parameters. Without fusion, each operation performs its own memory round-trip: Conv reads input (802.8 KB) plus weights (2.4 MB) and writes output (802.8 KB), totaling 4 MB. BN then reads that output, adds its parameters (2 KB), and writes again, for 1.6 MB. ReLU repeats the pattern for another 1.6 MB. The total unfused memory traffic is 7.2 MB. With fusion, the entire sequence reads input and weights once and writes the final output once, requiring only 4 MB, a 44.5 percent bandwidth reduction.

At the modeled 900 GB/s HBM bandwidth, these traffic volumes imply ideal lower bounds of 8 microseconds before fusion and 4.5 microseconds after fusion, a 1.80× ratio for the traffic component alone. These are not full layer latencies because convolution compute, cache effects, and launch overhead must also be included.

As table 10.13 shows, fusion benefits vary by workload. Memory-bound operations benefit most, while compute-bound operations see minimal improvement.

Table 10.13: Illustrative Kernel Fusion Benefit by Workload: Memory-bound workloads tend to benefit more than compute-bound workloads; realized gains depend on the graph, runtime, sequence shape, and hardware.

Workload	Relative Fusion Benefit	Why
Element-wise operations	High	Highly memory bound, low arithmetic intensity
Conv-BN-Act patterns	Moderate	Mixed memory/compute characteristics
GEMM-based operations	Low	Compute bound; fusion reduces the memory-bound tail
Attention mechanisms	High	Long sequences amplify avoided attention-intermediate traffic

[20] **FlashAttention**: Demonstrates fusion's power for memory-bound attention by tiling computation to SRAM, avoiding materialization of the full attention matrix, and reporting multi-fold speedups on long sequences (T. Dao et al. 2022). This exemplifies how kernel fusion transforms memory-bound bottlenecks: the arithmetic is mathematically equivalent, but the memory access pattern changes, making longer-context attention feasible on hardware that could not otherwise afford the full intermediate matrix. ↩ IO-aware attention tiling and its HBM-traffic accounting are established in section 8.7.4.

Fusion also reduces kernel launch overhead. Each CUDA kernel launch incurs microsecond-scale latency. In a graph with 50 fusible Conv-BN-ReLU triplets, unfused execution launches 150 kernels, while fused execution launches 50 kernels, saving repeated launch overhead in addition to memory traffic.

Fusion implementation spans the software stack, from framework-level pattern matching through compiler optimization such as Accelerated Linear Algebra (XLA), TVM, and TensorRT to runtime fusion that adapts to input shapes and hardware characteristics. The process becomes tractable when a framework can export or trace the relevant computation as a graph whose nodes are tensor operations and whose edges record data dependencies. A compiler can then match legal patterns such as Conv→BN→ReLU and rewrite them as fused operations while checking shapes, aliases, and numerical constraints. Dynamic control flow, in-place mutation, or unsupported operations can prevent that graph rewrite, so the exported plan must be inspected rather than assumed.

Fusion removes traffic between dense operations that must all execute. It does nothing for operands that pruning has already set to zero, which still occupy memory and multiplier slots unless the kernel can skip them.

↪ The register pressure and accelerator-specific constraints that bound how far kernels can be fused on GPUs and TPUs are developed in section 30.

10.5.2 Sparsity exploitation

Pruning introduces zeros into weight matrices, but dense kernels cannot infer skipped operations from a zero operand in memory. Sparsity exploitation addresses the execution half of this trade-off: designing representations and kernels that translate zeros into reduced memory traffic and execution time. **Sparsity**[21] in machine learning denotes tensors where a substantial fraction of entries are zero.

Formally, for a sparse weight matrix $\mathbf{W}_{\text{sparse}} \in \mathbb{R}^{m \times n}$, the nominal sparsity ratio ρ_{sparse} is:

$$\rho_{\text{sparse}} = \frac{\|\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}\|_0}{m \times n}$$

where $\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}$ is an indicator function that yields 1 if entry (i, j) is zero and 0 otherwise, and $\|\cdot\|_0$ counts nonzero entries. When magnitude pruning treats entries below δ_{prune} as removable, the thresholded sparsity ratio becomes:

$$\rho_{\text{sparse},\delta} = \frac{\|\mathbf{1}_{\{|(\mathbf{W}_{\text{sparse}})_{ij}| \le \delta_{\text{prune}}\}}\|_0}{m \times n}$$

where δ_{prune} defines the pruning boundary. Zeros yield wall-clock acceleration only when the execution pipeline skips both the arithmetic and the associated memory traffic.

21 **Sparsity**: From Latin *sparsus* (scattered), past participle of *spargere* (to scatter); in ML, ℓ_1 regularization such as the lasso can induce exact zeros rather than merely small values (Tibshirani 1996). Software can represent arbitrary sparsity patterns, while efficient hardware execution generally requires a supported format or structured pattern (for example, NVIDIA's 2:4 sparsity). The gap between representable and executable sparsity separates theoretical compression from realized speedup.

10.5.2.1 Sparsity types

From an execution standpoint, the spatial pattern of zeros determines memory transaction efficiency. As established in section 10.3.1.2, unstructured sparsity zeroes arbitrary scalar connections, requiring pointer arrays and non-unit-stride gathers that degrade bandwidth on SIMD and systolic units. Conversely, structured patterns—such as channel pruning or $N{:}M$ constraints—preserve predictable memory layouts, aligning with standard dense kernels or dedicated hardware datapaths.

10.5.2.2 Sparsity utilization methods

Translating zeros into runtime speedup requires a representation that encodes nonzero positions and a kernel that decodes those coordinates to bypass unneeded computation (Hoefler et al. 2021). Consider the arithmetic difference during a matrix-vector product: a dense 4×4 operator executes 16 multiply-accumulate operations, whereas a sparse execution path computes only the six nonzero products alongside the necessary metadata lookups:

$$\begin{bmatrix} 2 & 0 & 0 & 1 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 5 & 0 \\ 0 & 0 & 0 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2x_1 + x_4 \\ 3x_2 \\ 4x_1 + 5x_3 \\ 6x_4 \end{bmatrix}$$

The runtime benefit depends entirely on whether the latency eliminated by skipping 10 multiply-accumulate operations exceeds the latency added by coordinate decoding and non-contiguous activation loads. Structured patterns resolve this trade-off by constraining where zeros occur.

10.5.2.3 Structured patterns

Achieving actual speedups from sparsity requires hardware that can efficiently skip zero-valued computations. Different processor architectures handle sparse patterns with varying effectiveness—for example, Ampere Sparse Tensor Cores exploit fine-grained 2:4 structured patterns while systolic arrays require dense block structures. Software libraries such as cuSPARSE can help bridge this gap by reformulating sparse computations into patterns that current hardware handles efficiently. For example, MegaBlocks (Gale et al. 2022) reformulates sparse Mixture of Experts training into block-sparse operations, grouping routed expert-token work into dense tiles so specialized kernels can maintain high accelerator utilization despite irregular sparsity patterns.

A sparse pattern earns hardware speedup only when it is regular enough for kernels to predict. Two prominent formats make that regularity explicit: block sparse matrices and N:M sparsity patterns. Block sparse matrices isolate blocks of zero and nonzero dense submatrices so that operations on the large sparse matrix can be re-expressed as a smaller number of dense operations on submatrices. This structure supports more efficient storage of dense submatrices while maintaining shape compatibility for matrix or vector products. Figure 10.28 illustrates this representation with 3×3 dense sub-blocks indexed by an auxiliary coordinate array, allowing sub-block kernels to bypass empty gray tiles (NVIDIA 2020a). The block size is a teaching choice. Actual speedup requires a block shape, layout, datatype, and kernel supported by the target library and hardware; regular blocks alone do not guarantee aligned memory transactions or Tensor Core execution. Several other works, such as Monarch matrices (Tri Dao et al. 2022), have built on this block-sparsity approach to strike an improved balance between matrix expressivity and compute/memory efficiency.

Similarly, N:M sparsity retains at most n_{keep} nonzeros in each group of m_{group} consecutive elements; hardware-oriented pruning often enforces exactly n_{keep} retained values so every group has the same representation (Zhou et al. 2021). The deterministic format lets a supported kernel predict the value and metadata layout while retaining more capacity than removing whole channels or blocks. That regularity creates an executable compromise between arbitrary masks and dense computation, but only for targets with the matching path.

As the 2:4 example in section 10.3.1.3 showed, each stored value then needs only a 2-bit index (00_2 through 11_2) to record its position within its block of four. This compact metadata imposes only a 16-bit overhead per 16-element row segment—far below the multi-byte pointer arrays required by general CSR or COO formats—allowing Tensor Cores to stream compressed operands without memory bandwidth bloat.

Figure 10.29 contrasts the execution mechanics between standard dense GEMM and 2:4 sparse processing: on the right, 2-bit coordinate indices steer hardware multiplexers to pull matching activation elements into the dot-product engine alongside the two retained weights per 4-element block, halving arithmetic cycles relative to the 8-element dense baseline on the left. In parallel, STEP examines learning more general N:M masks for inference (Lu et al. 2023).

At accelerator level, the same pattern-specific rule holds: hardware support is a contract between the sparse format and the execution path.

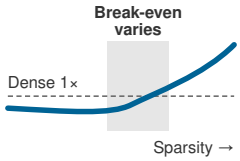
Supported Sparse Tensor Core paths on NVIDIA Ampere and later can accelerate 2:4 sparsity by skipping prescribed zeros and carrying compact metadata, with an ideal sparse arithmetic-rate improvement of up to $2\times$ (NVIDIA Corporation 2020a). Conversely, unstructured pruning zeroes arbitrary individual weights. Those irregular locations can disrupt memory coalescing and lane utilization, while Compressed Sparse Row (CSR) or Coordinate (COO) metadata adds storage and non-unit-stride accesses. An unstructured sparse model can therefore run slower than its dense counterpart when metadata and underutilization outweigh skipped arithmetic. TPUs are useful contrast cases for dense systolic-array acceleration (Jouppi et al. 2021), but sparse acceleration still depends on a specific hardware and software path rather than accepting arbitrary masks. Field-programmable gate arrays can implement application-specific sparse formats, yet their efficiency depends on the chosen dataflow and resource budget; programmability is not a universal sparse-speedup guarantee.

Across all platforms, sparse operations can reduce memory bandwidth requirements and energy consumption when the sparse representation and kernels actually skip data movement rather than only arithmetic. This benefit compounds with quantization: a sparse INT8 model can require less memory traffic than either technique alone when the format overhead is small enough (Hoefler et al. 2021; Gale et al. 2020).

10.5.2.4 Challenges and limitations

The format-hardware contract explains why nominal sparsity often fails to deliver speedups in practice.

First, the performance break-even point varies with tensor dimensions, sparse format, kernel implementation, and hardware architecture. When coordinate metadata and irregular activation gathers saturate cache lines, an unstructured sparse kernel can run slower than an optimized dense baseline until sparsity exceeds 80 to 90 percent. Second, the accuracy-efficiency trade-off is nonlinear: models often tolerate moderate pruning before task quality degrades abruptly past a critical density threshold. Finally, energy savings do not scale linearly with skipped operations; irregular memory access and index decoding can consume more power on edge devices than the arithmetic they eliminate.



Unstructured sparsity pays off only after crossing a hardware- and kernel-specific break-even point.

10.5.2.5 Combined optimizations

Pruning, quantization, kernel fusion, and dynamic computation share weights, representations, and physical memory. Composing these techniques can compound footprint reductions—such as pairing structured 2:4 sparsity with INT8 quantization—but the resulting executable must manage both structural metadata and quantization scales simultaneously. If transformations interact destructively, such as pruning disrupting the memory alignment required by a quantized SIMD kernel, wall-clock performance collapses. Because each optimization alters the arithmetic intensity and memory traffic seen by downstream operators, multi-technique pipelines require joint profiling rather than accumulating theoretical ratios (Hoefler et al. 2021).

> **Systems Perspective 10.4: The optimization composition problem**
>
> Optimization techniques interact through shared representations and physical resources rather than composing as independent functions. Pruning changes the surviving value distribution, tensor shapes, or sparse format; quantization adds scales and changes the available kernels; fusion changes which intermediates are materialized and can alter the profile that justified earlier choices. Effective optimization therefore treats the model-runtime-hardware combination as a coupled system, measures it after every transformation, and revisits the sequence when a new bottleneck appears. This makes compression a systems engineering problem as well as a machine learning problem.

Because optimization techniques interact so heavily through shared execution mechanics, selecting an appropriate compression strategy cannot rely on single-technique benchmarks. Practical deployment requires a systematic selection workflow that maps target hardware bottlenecks directly to compatible compression pipelines.

10.6 Technique Selection

Selecting compression techniques for production deployment requires identifying the binding physical constraint of the target execution environment. Consider a transformer deployment where model parameters exceed device memory by 3×, inference latency exceeds the service-level objective by 4×, and the thermal envelope restricts dissipation to no more than 2 W sustained. Deciding whether to quantize, prune, distill to a compact architecture, or compose multiple methods depends on which hardware boundary is saturated, the allowable task accuracy loss, and the available training budget.

The three primary optimization levers target different levels of the system stack. Distillation alters the algorithmic architecture by training a compact dense student to emulate a larger teacher, executing efficiently on standard hardware without specialized runtime support. Pruning modifies

tensor topology and operation patterns, but yields wall-clock acceleration only when the resulting sparsity aligns with hardware execution units, such as 2:4 structured sparse matrix cores or channel-aligned sub-matrices. Quantization alters data representation by reducing numerical bit width, directly lowering memory bus traffic and cache footprint while engaging higher-throughput integer datapaths. Table 10.14 summarizes the first-order trade-offs among these techniques across quality risk, training cost, and hardware dependency.

Table 10.14: Optimization Technique Trade-Offs: These rows identify first-order candidates rather than guaranteed outcomes. Quality risk and training cost depend on the model and method, while hardware dependency determines whether pruning or quantization changes latency. Pruning can remove structure or create sparse tensors, quantization directly reduces raw value bytes, and distillation spends teacher inference plus a student training run to produce an ordinary dense model. The candidate must still be evaluated against the same task and deployed-system thresholds.

Technique	Primary Goal	Quality Risk	Training Cost	Hardware Dependency	Candidate Use
Pruning	Reduce FLOPs/size	Pattern-dependent	Often includes fine-tuning	High for sparse speedup	Removable structures/weights
Quantization	Reduce bytes/precision	Bit-width-dependent	Low for PTQ; higher for QAT	High for runtime speed	Memory- or bandwidth-bound path
Distillation	Train smaller model	Student-dependent	Teacher and student run	Low for dense execution	A new training run is feasible

10.6.1 Mapping constraints to techniques

The binding hardware constraint identifies initial optimization candidates. Under the D·A·M taxonomy, compression methods redistribute resource demands across Algorithm (topology, pruning, distillation, and the numerical representation of weights) and Machine (execution units and memory hierarchies); calibration sets are the main Data-axis input. Table 10.15 maps each system constraint to an optimization dimension that can relieve it. The marks identify mechanisms to investigate, not guaranteed deployment gains.

Table 10.15: Optimization Dimensions: System constraints drive optimization along three core dimensions: model representation, numerical precision, and hardware implementation, each addressing different resource limitations and performance goals. ✓ marks a direct lever, while △ marks a workload- or hardware-dependent benefit; for example, low-precision arithmetic helps only when the accelerator and runtime provide a supported path.

System Constraint	Model Representation	Numerical Precision	Hardware Implementation
Computational Cost	✓	△	✓
Memory and Storage	✓	✓	△
Latency and Throughput	✓	△	✓
Energy Efficiency	✓	✓	✓
Scalability	✓	△	✓

These dimensions interact through the iron law of ML systems, meaning a single optimization often shifts multiple physical bottlenecks simultaneously. For example, quantizing weights from FP32 to INT8 directly relieves on-device storage capacity. A bandwidth-limited operator approaches proportional latency savings only when the affected traffic dominates and conversion overhead does not consume the gain. A compute-limited operator needs a faster supported execution path. Unstructured pruning likewise reduces executed arithmetic only when sparse kernels exploit the removed weights; indexing and irregular memory accesses can offset the saving. The changed representation therefore supplies a candidate improvement whose effect must be measured on the target.

10.6.2 Decision framework

When persistent storage or device memory capacity binds the deployment—such as over-the-air package limits or microchip flash budgets—quantization provides the most direct payload reduction.

Relative to FP32 storage, INT8 post-training quantization (PTQ) cuts raw weight storage by 4× without requiring retraining cycles. INT4 cuts raw payload by 8×, though aggressive bit-width reduction increases the risk of accuracy degradation. If uniform low-bit quantization causes unacceptable accuracy loss on sensitive layers, distilling knowledge into a smaller dense architecture followed by moderate quantization preserves fidelity while satisfying the storage budget.

When latency is compute-bound, the workload operates above the accelerator's arithmetic intensity ridge point, saturating execution units. Relieving this bottleneck requires reducing executed operations or increasing arithmetic throughput. Structured pruning removes channels, attention heads, or entire layers, yielding smaller dense matrices that execute on standard libraries without sparse runtime overhead; however, pruning to dimensions that misalign with hardware warp or tile sizes (such as channel counts not divisible by 8 or 16) degrades tensor core efficiency and curtails wall-clock gains. If the target platform provides dedicated integer execution units, INT8 quantization increases the instruction throughput rate, accelerating compute-bound kernels directly. Early-exit architectures provide an alternative algorithmic path by dynamically halting inference on simpler inputs, provided that routing and pipeline synchronization overhead do not disrupt batch processing.

Small-batch transformer generation often exposes a weight-bandwidth bottleneck: dense decoding streams large parameter matrices for matrix-vector operations with low arithmetic intensity. When fetched weights dominate latency, weight-only quantization (such as INT4 or INT8 weights paired with FP16 activations) can approach a speedup proportional to the reduction in those bytes. KV-cache traffic and dequantization overhead limit that gain without necessarily making execution compute-bound; larger batches can increase weight reuse and shift the limiting resource.

When energy or thermal dissipation binds, quantization provides substantial benefits because moving data across off-chip memory buses consumes orders of magnitude more energy per bit than on-chip arithmetic or local SRAM access. Reducing operand bit widths directly lowers bus capacitance charging energy, while integer ALUs consume less switching energy than floating-point datapaths. Structured pruning compounds these savings by eliminating operations and memory transactions entirely. Full-system power audits must still measure peripheral components—including memory refresh, host bus communication, and static idle leakage—to ensure that kernel-level energy savings translate into extended operating life.

Selection also depends on the engineering and computational budget available for optimization. Post-training quantization serves as the initial baseline because calibrated PTQ requires only a small representative dataset and negligible compute. If PTQ fails the accuracy threshold and fine-tuning resources are available, quantization-aware training (QAT) simulates quantization in the forward pass and uses approximate gradients during backpropagation to help recover lost accuracy. Knowledge distillation incurs higher computational costs, requiring teacher forward passes and substantial training cycles to optimize a smaller student model. Hardware-aware NAS occupies the highest cost tier, justified only when high-volume deployment amortizes the extensive compute needed to explore the architectural search space.

The transformer scenario above makes INT4 weight-only quantization a candidate, not a completed solution. Starting from FP16, its raw weight reduction can clear a 3× weight-storage gap, but total memory also includes KV state, activations, metadata, and workspace. The 4× latency gap must be checked against the fraction of time spent fetching weights and the resulting kernel path. Distillation is another candidate if the remaining gap justifies its training cost. Each candidate must pass task-quality, latency, peak-memory, and sustained-power checks against the 2 W envelope.

Choosing an individual technique addresses a single binding bottleneck, but production deployment budgets frequently require compounding gains across multiple dimensions. When memory capacity, bandwidth, and compute throughput are all constrained simultaneously, optimizing a system requires composing structural alterations, precision reductions, and custom execution kernels into an integrated transformation pipeline.

10.7 Combining and Sequencing Techniques

Aggressive compression targets rarely yield to a single optimization lever. Fitting a 110M-parameter language model into a mobile memory and thermal envelope requires composing structural pruning, knowledge distillation, and precision quantization into a coherent staging sequence. In this modeled

mobile pipeline, staged optimization reduces a BERT-Base footprint from 440 MB to 28 MB (rounded), achieving a 16× raw-footprint reduction. Each stage addresses a distinct bottleneck in the D·A·M hierarchy: structural pruning eliminates redundant matrix multiplications to reduce arithmetic operations, knowledge distillation supervises the compact student to recover representational capacity, and quantization packs weights and activations into lower-bit integers to alleviate memory bandwidth pressure.

Example 10.2: BERT-Base mobile deployment pipeline

Scenario: A modeled deployment targets executing a 110M-parameter BERT-Base language model on resource-constrained mobile hardware. The compact graph is stipulated for this example; the figures are not reported DistilBERT results.

The FP32 teacher begins at 440 MB with assigned task accuracy 76.2 percent. Structural pruning produces a stipulated 27.5M-parameter graph, a 75 percent parameter reduction, with raw FP32 weights of 110 MB and assigned accuracy 75.1 percent.

That pruned graph is the distillation student. Teacher supervision recovers its assigned accuracy to 75.9 percent without another parameter-count reduction. Final INT8 quantization leaves the same parameter count, reduces raw weight storage to 27.5 MB, and has assigned accuracy 75.6 percent. The headline ratio uses this unrounded weight payload; scales, zero-points, packing, activations, and runtime workspace remain outside the calculation.

Diagnosis: In this modeled comparison, quantization before pruning has an assigned accuracy loss of 2.1 percent. One mechanism to investigate is whether structural changes invalidate previously calibrated ranges. Pruning and distillation before quantization allow calibration to use the resulting distributions; the modeled values illustrate that hypothesis rather than establish a universal ordering.

Systems lesson: In this modeled pipeline, pruning, then distillation, then quantization-aware training produce a 16× footprint reduction (440 MB to 28 MB) while limiting task accuracy loss to 0.6 percent. Because tensor distributions and hardware kernel efficiencies vary across architectures, each intermediate artifact must be profiled on the target runtime.

Sequencing matters because these transformations interact through shared parameters and activation distributions. Structural pruning removes channels or attention heads, altering the dynamic range and variance of surviving weight tensors. Quantizing before pruning can leave ranges poorly matched to the resulting distributions, so recalibration or joint adaptation may be needed. In the staged candidate, floating-point recovery lets the pruned network adapt, and distillation supplies a teacher signal before final quantization. QAT itself updates floating-point master weights through approximate gradients; it does not require direct integer weight updates. Alternative schedules should be compared at matched recovery and calibration budgets.

The method comparison in figure 10.30 illustrates the separate question of how combining techniques changes the size-quality trade-off (Han, Mao, et al. 2016). While individual SVD and pruning curves suffer steep accuracy cliffs below 20 percent and 10 percent model size respectively, combining pruning with quantization extends the zero-loss plateau down to roughly 3 percent of original parameter footprint. This comparison does not isolate transformation order. The magnitude of these gains remains model- and runtime-specific, requiring engineers to validate each transformation stage independently.

A compounding footprint reduction on paper does not guarantee proportional execution speedup on silicon. Unstructured zeros demand sparse runtime support, pruned channel counts risk misaligning with hardware warp or tile boundaries, and quantized weights incur conversion stalls unless paired with native integer execution units. Determining whether an optimization sequence succeeds requires empirical measurement on target hardware.

10.8 Efficiency Measurement

The divide between theoretical compression ratios and measured wall-clock execution establishes an empirical imperative: efficiency is a physical measurement, not an algorithmic property. While INT8

quantization shrinks weight memory by $4\times$ relative to FP32, end-to-end latency rarely scales in proportion. Realized throughput depends on cache residency, memory bus contention, and kernel launch overhead that nominal compression metrics cannot capture. Rigorous deployment requires identifying which physical resource binds each layer, establishing reproducible hardware baselines, and isolating the interaction between arithmetic reduction and host runtime overhead.

10.8.1 Profiling and opportunity analysis

Profiling decomposes model execution along the physical boundaries of the accelerator: memory capacity (SRAM, DRAM/HBM), memory bus bandwidth, and arithmetic throughput. Memory allocation divides into static footprint (weight tensors and persistent buffers) and dynamic allocation (activation tensors and KV caches during the forward pass). Static footprint dictates whether model parameters fit into fast on-chip memory or device DRAM without spilling across high-latency host-accelerator interconnects such as PCIe. Dynamic allocation dictates peak memory residency, bounding maximum batch size and serving concurrency. At the operator level, profiling measures theoretical operations (FLOPs) alongside wall-clock execution time. Because operators vary across orders of magnitude in arithmetic intensity ($I = O/D_{\text{vol}}$), their execution bottlenecks diverge: sufficiently large matrix multiplications with high operand reuse can saturate execution units, whereas standalone low-intensity operators (such as layer normalization and elementwise activations) can spend much of their time streaming tensors, consuming wall-clock time disproportionate to their FLOP count.

A profile of a Vision Transformer (ViT) on an edge accelerator illustrates this operational heterogeneity. Per token and layer, the multilayer perceptron (MLP) block costs about $8d^2$ multiply-accumulates against $4d^2$ for the query, key, value, and output projections. In a ViT-Base-sized model the MLP blocks therefore carry nearly two-thirds of FLOPs and the projections just under one-third, with the attention score and weighting products adding only a few percent at standard image resolution. Layer normalization inverts the picture: it contributes almost no FLOPs yet can claim a visible share of wall-clock latency, because it streams whole activation tensors with little arithmetic per byte. Profiling isolates the governing physical bottleneck for each component, dictating targeted optimizations rather than uniform compression: structured pruning of MLP hidden units relieves the compute-dominant blocks, kernel fusion eliminates intermediate global memory round-trips for memory-bound normalization layers, and INT8 quantization shrinks the static footprint that the MLP and projection weights dominate.

Translating layer-level profiling metrics into end-to-end system speedup requires accounting for the complete serving pipeline. In production serving systems, model execution frequently represents only a fraction of total latency, with the remainder consumed by image decoding, tokenization, RPC deserialization, or network transfer. Optimizing the model in isolation encounters an Amdahl ceiling whenever non-model stages dominate the latency budget.



Non-model work dominates latency, so compression caps speedup at 1.25×.

> **Systems Perspective 10.5: FLOPs reduction is not proportional speedup**
>
> Reducing a model's FLOPs by 50 percent does not guarantee 50 percent latency reduction. Memory-bound operations see little benefit from arithmetic reduction because data movement, not multiplication, sets their time. Critically, Amdahl's law (principle 8) applies at the system level: if model inference accounts for only 20 percent of end-to-end latency (with the remaining 80 percent spent on data loading, preprocessing, and postprocessing), then even perfect model optimization yields at most 1.25× overall speedup. Tokenization and network transfer can occupy the non-model fraction in language-model serving; image decoding, resizing, and normalization can do the same in vision pipelines. Profile the complete path on target hardware before estimating optimization benefits.

Complementing latency and throughput profiles, layer-wise sensitivity analysis quantifies how individual layers tolerate perturbation. In deep networks, parameter distributions across different layers exhibit unequal robustness to precision reduction or weight elimination. Profiling accuracy degradation while systematically perturbing one layer at a time—evaluating quantization noise or weight-removal impact—identifies which components require high-precision representations and

which can absorb aggressive 4-bit quantization or structured pruning without compromising task accuracy.

10.8.2 Measuring optimization effectiveness

Evaluating compression requires balancing statistical quality against physical execution costs across the Data, Algorithm, and Machine (D·A·M) axes. A single scalar metric—such as top-1 accuracy or parameter count—fails to capture deployment viability. Quality baselines must capture aggregate accuracy, confidence calibration (verifying that predicted probabilities reflect empirical correctness), and performance across underrepresented input slices. Simultaneously, efficiency baselines must record peak memory residency, latency distributions (including p99 tail latency), memory bus saturation, and per-inference energy.

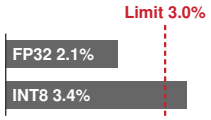
Quantizing ResNet-50 from FP32 to INT8 demonstrates how compression decisions ripple across physical and statistical dimensions. Rather than assessing INT8 conversion through an isolated accuracy metric, table 10.16 presents a modeled deployment profile across compute, memory, energy, and statistical calibration. Validating compression requires confirming that artifact compression does not trigger unacceptable quality degradation.

Table 10.16: Illustrative ResNet-50 INT8 Deployment Profile: The values form a modeled scenario, not measurements from one hardware run. A real profile must measure accuracy, latency, artifact size, energy, and calibration on the target system.

Metric	FP32 baseline	INT8 result	Deployment reading
Top-1 accuracy	76.1%	75.8% (0.3 pp drop)	Aggregate accuracy mostly holds, but subgroup checks still matter.
Modeled latency	1.56 ms	0.39 ms	GPU latency improves when the runtime maps INT8 to fast kernels.
Model size	102.4 MB	25.6 MB (4×)	The artifact becomes easier to cache, transmit, and deploy on edge.
Energy/inference	0.25 J	0.06 J (4×)	Modeled at constant power, so it tracks latency; measure it directly.
Calibration error	2.1%	3.4%	Confidence calibration can drift even when accuracy looks acceptable.

In this illustrative profile, converting weights and activations to INT8 reduces the model footprint by 4× (from 102.4 MB to 25.6 MB), while the scenario assigns a 4× reduction in per-inference energy. That energy figure is the modeled latency speedup applied to energy, which assumes average power stays constant; it is not a separate result, and a measured profile could differ in either direction. The latency scenario starts from a modeled V100 FP32 baseline of 1.56 ms and applies an illustrative speedup factor to obtain 0.39 ms. While aggregate top-1 accuracy drops by only 0.3 percentage points (from 76.1 percent to 75.8 percent), calibration error increases from 2.1 percent to 3.4 percent. Post-quantization probability distributions drift, which can compromise safety-critical confidence thresholds even when top-1 classification accuracy appears preserved. Production deployment decisions require corresponding quality, latency, and energy measurements using a supported execution path on target hardware.

Suppose the application requires at least 75.5 percent top-1 accuracy and no more than 3.0 percent calibration error, measured with the same validation data and calibration metric as the profile. The INT8 candidate passes the accuracy limit but fails the calibration limit. Its modeled resource savings therefore do not justify acceptance. Retain the baseline for comparison, try confidence recalibration or a less aggressive quantization configuration, and repeat the quality checks before validating resource gains on the target runtime.



The INT8 candidate fails the calibration limit despite holding accuracy.

Realizing these gains in production requires evaluating interactions among multiple compression techniques. Applying pruning, quantization, and knowledge distillation sequentially can produce non-additive gains or compound degradation; for example, uncalibrated weight quantization on an aggressively pruned sparse matrix can trigger catastrophic accuracy collapse. Evaluating these compounded trade-offs requires mapping models onto an efficiency-quality Pareto frontier. Executing this systematic profiling and multi-stage transformation pipeline in turn depends on automated software tooling that coordinates compilation, kernel generation, and hardware benchmarking.

Checkpoint 10.5: From compression claim to deployment evidence

A compressed artifact must pass the same application requirements as its baseline.

- ☐ **Footprint and feasibility**: distinguish raw weight size from peak runtime memory, including activations, caches, metadata, and workspace.
- ☐ **Modeled and measured gains**: identify which claims in the ResNet-50 profile require target-hardware measurements, and explain why fewer FLOPs or bytes alone cannot establish latency.
- ☐ **Quality and operating limits**: specify the accuracy, confidence calibration, tail-latency, sustained-power, and energy checks needed before accepting a candidate.

10.9 Implementation Tools

Quantizing a 175-billion-parameter model by hand requires modifying thousands of operator call sites to insert scale factors and track dynamic activation ranges. Without automated framework tooling, even uniform INT8 post-training quantization requires manually intercepting tensor lifecycles across the computation graph, while pruning requires maintaining sparsity masks alongside weight matrices. At scale, manual graph manipulation cannot guarantee autograd consistency during retraining or enforce the memory-alignment constraints required by accelerator hardware.

Tool choice follows the layer of the software stack that owns the transformation. Framework APIs operate on the high-level computation graph during training and calibration, where the execution engine maintains visibility into weights, activations, and autograd state. At this stage, quantization is typically simulated: fake-quantization nodes clamp and round values within floating-point registers, modeling numerical precision loss while preserving differentiability.

Simulating quantization does not itself establish a physical speedup. A compiler or runtime must lower the graph into supported kernels; frameworks may integrate that stage or export to a separate deployment engine. The runtime compiler fuses scale factor arithmetic directly into matrix-multiplication compute tiles and packs sparse weights into hardware-specific metadata layouts (such as 2:4 structured sparsity formats). This lowering eliminates round-trip memory transactions to HBM that would otherwise negate the benefits of reduced precision. Software infrastructure enforces this boundary, capturing the calibration data, pruning schedules, and kernel fusions that generated the deployable binary rather than relying on untracked script modifications.

Across this multi-stage pipeline, the operational imperative is reproducibility. Compressing a model alters its numerical representations across both the algorithm and machine layers of the D·A·M taxonomy. Unrecorded calibration inputs or mismatched rounding modes can degrade accuracy silently without raising runtime errors; the baseline requirement is that every deployed artifact must remain deterministically traceable from its source training checkpoint through its compiled hardware execution graph.

10.9.1 Model optimization APIs and tools

↪ Versioned artifacts and the lineage graphs that keep deployed models traceable are operationalized in section 14.4.1.3.

At the model-development layer, compression algorithms require access to runtime state that downstream deployment compilers discard: gradient histories, optimizer state variables, and dynamic activation distributions. When post-training quantization or one-shot pruning degrades accuracy beyond acceptable operating thresholds, recovery requires retraining or iterative fine-tuning. Frameworks such as PyTorch and TensorFlow provide optimization APIs specifically to manage this graph-transformation boundary. These APIs inject quantization nodes, reparameterize weight tensors with sparsity masks, and preserve calibration metadata alongside the trained parameters. Model compression depends on these specialized graph mutations to bridge the algorithmic loss function with machine-level execution constraints.

Quantization-aware training demonstrates this graph-level intervention. Optimization APIs insert the fake-quantization operators of section 19 into the compute graph; these operators clamp and round in the forward pass and apply the straight-through estimator of section 10.4.3.2 in the backward pass. Listing 10.5 details the sequence of graph transformations and metadata tracking required to produce a deployable low-precision artifact.

Listing 10.5: Quantization-Aware Training: Mechanism-level transformation that trains through simulated low-precision behavior and exports the calibration metadata needed for deployment.

```
choose target precision and calibration policy
insert quantize/dequantize boundaries around selected tensors
train with simulated low-precision values while keeping gradient flow intact
record scales, zero points, accumulator precision, and unsupported operations
export the calibrated graph and metadata as one reproducible artifact
```

Pruning tools implement the mask-then-export boundary of section 10.3.1.8. A mask-based API keeps the dense weight tensor $\mathbf{W}$ and a binary mask $\mathbf{M}$ of the same shape during recovery training, computing $\widehat{\mathbf{W}} = \mathbf{M} \odot \mathbf{W}$ in the forward pass so the existing dense kernels and gradient updates keep working. An export pass then finalizes the transformation, either slicing away pruned channels or encoding the nonzeros in a sparse format the target's kernels accept. Listing 10.6 lists what must be recorded along the way so the compressed model can be reproduced.

Listing 10.6: Pruning Artifact Boundary: Applies unstructured or structured removal rules, then records the mask and fine-tuning context needed to reproduce the compressed model.

```
choose pruning granularity: individual weights, channels, blocks, or attention heads
score candidate parameters by magnitude, sensitivity, or validation loss impact
remove or mask the selected structure according to the hardware-compatible rule
fine-tune the compressed model against the original validation target
export weights, masks, sparsity pattern, and calibration evidence together
```

The engineering value of these APIs centers on experimental reproducibility across the D·A·M hierarchy. Rather than implementing ad hoc graph manipulations, practitioners use built-in optimization modules as standardized control points. Engineers can systematically sweep sparsity ratios, quantization scaling schemes, calibration datasets, and learning-rate schedules while holding the underlying execution pipeline constant. Packaging the resulting scales, zero points, masks, and architectural modifications into unified export artifacts ensures that downstream inference runtimes can reproduce the exact numerical semantics validated during training.

10.9.2 Hardware-specific optimization libraries

Exporting an optimized model from a training framework produces an intermediate graph representation, not an executable binary. Compilers and runtime engines such as TensorRT, XLA, OpenVINO, and TVM own the final translation step: compiling a pruned, quantized, or fused graph into machine instructions tailored to the target accelerator. The governing systems principle is grounded in the D·A·M hierarchy: algorithmic compression remains theoretical until the graph maps to kernels that physically exploit the target machine's memory hierarchy and execution units.

The compilation pass verifies whether nominal compression yields physical speedup. Pruning that leaves zeros in dense tensors still streams and multiplies those entries with a dense kernel. Compact sparse formats need matching kernels, such as a supported 2:4 path, while structured channel removal can produce smaller tensors for ordinary dense kernels. Quantization lowers bitwidth, but a weight-only path may unpack operands into floating-point registers before arithmetic. That unpacking adds instructions; its cost must be compared with the packed weight traffic saved.

This introduces the *runtime dequantization trap*: if an accelerator provides native INT8 matrix-multiplication kernels but lacks an INT8 implementation for an adjacent operation (such as LayerNorm, GeLU, or an attention softmax reduction), the runtime encounters a type boundary. An unfused execution path may allocate a temporary buffer, launch a dequantization kernel to recover floating-point values ($x = s(x_q - z)$), execute the unsupported operator, and launch another kernel to re-quantize its output. The floating-point format and opportunities to fuse conversion depend on the runtime. These intermediate format conversions incur extra kernel launches, introduce synchronization barriers, and add temporary floating-point traffic that can make the quantized model slower than the FP16 baseline. A fused weight-only path may instead unpack values within the consuming kernel and retain a net memory-traffic benefit.

Furthermore, low-precision kernels require specialized tensor layouts, such as tiled or interleaved memory channels, to ensure coalesced loads into shared memory or register files. Kernel fusion eliminates intermediate round-trips to off-chip DRAM by staging intermediate activations directly inside on-chip SRAM or register space. When an unsupported operator sequence blocks fusion, the memory-bandwidth bottleneck remains intact. Whether integrated into a framework or supplied separately, the compiler and runtime determine which graph-level opportunities become an efficient machine execution plan.

10.9.3 Diagnostic visualization

The toolchain boundary divides responsibilities: framework APIs create and calibrate the compressed model, hardware optimization libraries lower that model to the execution substrate, and diagnostic inspection verifies numerical integrity. While system-scale validation and artifact monitoring belong to downstream deployment infrastructure, the immediate diagnostic imperative is verifying that quantization clipping, pruning masks, or sparsity patterns have not perturbed internal feature distributions beyond acceptable task error tolerances.

Quantization error histograms show whether error is broadly distributed or concentrated in outliers. Activation visualizations help detect clipping, overflow, and saturation. Figure 10.31 categorizes this diagnostic challenge across an 8×5 grid of first-layer convolutional filters: while top rows exhibit distinct edge, color, and frequency detectors, the bottom row displays uniform-looking filters that appear redundant at a glance but require measured activation tracing before safe pruning. Framework observers and debuggers can expose quantization ranges and tensor error, while compiler or runtime inspectors can show the graph and kernels that the exported model actually executes.

Sparsity diagnostics identify candidate layers for sparse execution. Figure 10.32 is a schematic spanning convolutional, fully connected, and transformer layer types, not measurements from one model or pruning run. Its rows illustrate how a pruning policy might distribute zeros unevenly across architectural blocks. A real diagnostic would pair each layer's density with its storage format and target-kernel profile to determine whether those zeros reduce memory traffic or execution time. Graph viewers and sparsity inspection tools help locate the affected tensors; profiling establishes the deployment benefit.

The diagnostic value is the nonuniformity, not the exact shade of any one cell. Greater sparsity in later layers shows where the policy removed weights; it does not establish that those representations were redundant. Compare task quality after removing weights from a candidate layer and after recovery training, including performance on important input slices. Early-layer sparsity warrants the same checks because damaging feature extraction can propagate through subsequent layers. Only those comparisons can establish whether the observed removal pattern preserves required behavior.

Implementation tools make compression repeatable, but they do not make the trade-offs disappear. Quantization can follow pruning or distillation in a layered pipeline: pruning alters model structure or parameter count, quantization changes numerical representation, and distillation trains the smaller artifact against a teacher's output distribution. While composing these transformations can compound nominal footprint reductions, unmeasured composition frequently produces catastrophic latency regressions or silent accuracy collapse on target hardware. Understanding where intuitive compression assumptions fail requires analyzing the concrete engineering fallacies that plague real-world deployments.

10.10 Fallacies and Pitfalls

A production team attempting to deploy a 70-billion-parameter language model might observe that an uncalibrated 4-bit quantization reduces memory footprint by $4\times$ while causing generation perplexity to diverge uncontrollably. Conversely, a vision team might achieve 70 percent weight sparsity through unstructured pruning only to find inference latency increases because commodity GPU kernels cannot execute irregular sparse indexing. Model compression presents numerous counterintuitive traps where theoretical parameter or FLOP reductions fail to produce wall-clock speedups. The following fallacies and pitfalls examine where standard engineering assumptions break down across the model-runtime interface.

Fallacy: *Optimization techniques can be applied independently without considering their interactions.*

Engineers assume optimization strategies compose additively: 50 percent pruning plus 4× quantization yields combined benefits. In reality, techniques interact nonlinearly and compound losses. In a representative BERT-style compression scenario, pruning to 70 percent sparsity may preserve most task performance, but applying INT8 quantization afterward can lose more accuracy than QAT on the pruned model. Knowledge distillation from heavily pruned teachers can also transfer degenerate attention patterns that reduce student accuracy compared with distilling from dense teachers. As section 10.7 demonstrates, successful optimization requires coordinated application where techniques are sequenced together. Organizations that apply aggressive combinations without measuring interactions waste weeks recovering lost accuracy.

Pitfall: *Assuming theoretical metrics or compression ratios translate directly into proportional deployment speedup.*

Teams frequently reduce FLOPs or parameter counts substantially without realizing proportional latency gains on physical hardware. In one representative trace, a pruned model with 40 percent fewer parameters achieves only a 12 percent latency reduction because irregular sparsity prevents efficient execution. Similarly, reducing a transformer from 440 MB to 110 MB through INT8 quantization yields a 4× raw weight-payload reduction, yet format conversion and scalar fallback on hardware without a native low-precision datapath erode the latency dividend. In a composed pipeline combining 50 percent structured pruning and INT8 quantization, the realized speedup reaches only 1.5× against an 8× paper target—capturing just 18.8 percent of the theoretical projection. Scales, index metadata, unsupported operators, and memory bottlenecks dictate runtime throughput. Production workflows must evaluate wall-clock latency on target execution units rather than extrapolate from nominal compression ratios.

Fallacy: *Aggressive quantization maintains model performance with minimal accuracy loss.*

Engineers assume quantization error scales uniformly with bit width. In practice, precision reduction can exhibit threshold effects, and the threshold depends on the model, layer, quantizer, calibration, and training method. The ResNet and BERT values in this chapter are illustrative rather than universal degradation bands. Even operations such as LayerNorm and Softmax can be implemented with integer approximations, so they do not universally require FP16; whether a low-precision implementation preserves task quality must be measured. As section 10.4.3 demonstrates, mixed-precision approaches can retain higher precision where uniform quantization fails.

Pitfall: *Defaulting to FP32 everywhere to avoid quantization risk.*

FP32 uses twice the raw storage and bandwidth of BF16, but lower-precision execution does not automatically preserve convergence or accuracy. Mixed-precision training often retains FP32 accumulators or master weights and may require loss scaling, while inference precision must be validated per model and operator. The right question is what lowest precision preserves quality on the evaluation distribution and maps efficiently to the target hardware.

Fallacy: *Post-training optimization is always enough.*

Teams often begin with post-training quantization (PTQ) because it avoids retraining. If the resulting artifact meets its task and deployment thresholds, that simple path may be sufficient. When it does not, quantization-aware training (QAT) can recover part of the quality gap by adapting the model to simulated low-precision behavior. Post-training pruning and post-hoc distillation can likewise underperform training-aware schedules at aggressive sparsity or bit width. As detailed in section 10.4.3, the production threshold—not a presumption that either path is universally superior—governs the decision.

Fallacy: *Sparse matrix representations always save memory and accelerate inference.*

Sparse formats introduce coordinate metadata that erodes storage savings at moderate densities. To represent n_{nz} nonzero values across n_{rows} rows, a coordinate list (COO) stores explicit coordinate tuples `(row_idx, col_idx, value)`. For 32-bit floats and 32-bit integer indices, each nonzero element consumes 12 bytes (3× a dense 4-byte FP32 entry), requiring greater than 66.7 percent sparsity just to break even on memory footprint. Compressed sparse row (CSR) eliminates redundant

row coordinates by storing three arrays: `values` ($n_{\text{nz}} \times 4$ B), `col_indices` ($n_{\text{nz}} \times 4$ B), and `row_pointers` ($(n_{\text{rows}} + 1) \times 4$ B). Each nonzero entry costs 8 bytes ($2\times$ a dense FP32 entry), requiring over 50 percent sparsity before the sparse structure consumes less memory than the original dense tensor; below 50 percent sparsity, CSR metadata expands the model's memory footprint.

Execution exhibits similar break-even constraints. Converting tensors to sparse storage as soon as pruning creates zeros degrades latency if metadata decoding, irregular gather operations, and poor cache locality outweigh the saved arithmetic. Furthermore, performance exhibits no universal sparsity threshold: tensor dimensions, sparse format, runtime kernel, batch size, and hardware architecture jointly govern whether sparse execution outperforms dense execution. Specialized formats such as 2:4 structured sparsity eliminate index arrays in favor of fixed 2-bit multiplexer metadata, providing an accelerator datapath that nearly halves weight traffic with minimal indexing overhead (section 10.3.1.3). Production workflows must therefore measure realized density and commit to a sparse representation only after verifying that the operational break-even threshold has been crossed on target hardware.

10.11 Summary

Model compression is an engineering discipline built on three complementary dimensions: structural optimization modifies what the model computes, precision optimization configures how precisely it computes, and hardware implementation determines how efficiently those computations execute on physical hardware. These dimensions can compound, but their gains do not multiply automatically. Pruning, distillation, and quantization together produce the illustrative pipeline's 16× footprint reduction from 440 MB to 28 MB; metadata, unsupported operators, and hardware mapping determine the realized deployment gain. A parameter reduction approaches the same latency reduction only when the affected work is on the critical path and the runtime can exploit the new representation. Profile on target hardware, not paper metrics.

Data selection and model compression divide the efficiency work along the D·A·M axes. Data selection acts on the Data axis and lowers the cost of producing a model by spending compute only on examples that carry signal. Compression acts where Algorithm meets Machine and lowers the cost of running the model by reshaping its structure and numerics for the target. Whether those deployment savings become real speedups still depends on the hardware and runtime that execute the compressed model. The need for them does not fade, because the bitter lesson keeps growing the models that win benchmarks while the envelopes of phones and microcontrollers stay fixed.

Key Takeaways: From benchmark winner to production model

- **Compression spends surplus capacity**: Production models trade unused parameters, precision, and capacity for latency, memory, power, or cost limits the deployment cannot violate. The target is an artifact that meets the deployment objective and all required quality and resource constraints.
- **Savings multiply only when aligned**: Structural pruning, distillation, quantization, and architecture changes can compound raw footprint reductions, as the modeled BERT mobile pipeline's 16× ratio illustrates. The gain becomes real only when task quality holds and the resulting operators match the target runtime and accelerator.
- **Precision is a deployment contract**: Where the target runtime supports it, INT8 post-training quantization is a useful first experiment because it reduces raw FP32 weight payload by 4× without retraining. QAT, distillation, or mixed precision become candidates when calibration or layer sensitivity exposes unacceptable error.
- **Hardware sets the exchange rate**: Unstructured sparsity and theoretical FLOP cuts do not imply latency gains unless kernels and memory layouts can exploit them. The chapter's hypothetical 8× paper target vs. 1.5× realized outcome illustrates why target-hardware profiling is mandatory.
- **End-to-end latency caps model wins**: Compression reduces end-to-end latency only when it accelerates the critical path; capacity or energy savings can remain valuable

elsewhere. When inference is 20 percent of total request latency, Amdahl's law (principle 8) caps even perfect model acceleration at 1.25×, so preprocessing, dispatch, and data movement may be the true optimization target.

The techniques in this chapter differ in almost every detail, yet one rule runs beneath all of them. Each buys a smaller, faster, or cooler model by spending something else. Pruning spends capacity, quantization spends numerical precision, and distillation spends training compute. Where the model holds no surplus to spend, the bill is paid in task quality. Apart from removing genuinely avoidable work, such as an unfused intermediate round-trip, compression does not make the work disappear; it moves cost or risk from one part of the system to another, and the hardware sets the exchange rate. A technique that pays off on a phone can therefore be worthless on a data-center GPU. This is the conservation-of-complexity heuristic at the level of a single model. A production artifact is a model whose structure, representation, and execution path have been engineered together until the trade balances against the silicon and application constraints that govern deployment.



What's Next: From math to physics

These optimization techniques compress the model's logic, reducing the work the algorithm asks the machine to perform. Whether less work becomes less time depends on the silicon that executes it, and this chapter repeatedly ran into the memory wall, the regime where fetching a weight costs more than computing with it. Chapter 11 starts from that wall and explains how GPUs, TPUs, and NPUs organize tensor units, systolic arrays, and tiered SRAM/HBM hierarchies to keep arithmetic fed, which is what decides whether a quantized or sparse representation actually runs faster.

```
\begin{tikzpicture}[line join=round,font=\sffamily]
\tikzset{
Box/.style={align=flush center,
    inner xsep=2pt,
    node distance=1.4,
    draw=OrangeLine,
    line width=0.75pt,
    rounded corners,
    fill=OrangeL!40,
    text width=61mm,
    minimum width=58mm, minimum height=22mm
  }
  }

\tikzset{%
planet/.style = {circle, draw=yellow!50!red!90,semithick, fill=yellow!30,line width=1.5pt,
                    font=\sffamily\bfseries,
                    minimum size=24mm, inner sep=1mm,align=flush center},
satelliteI/.style = {circle, draw=none, semithick, node distance=5,%fill=#1!10,
                    text width=35mm, inner sep=1pt, align=flush center,minimum size=20mm,minimum height=12mm},
satellite/.style = {circle, draw=none, semithick, fill=#1!10,
                    text width=26mm, inner sep=1pt, align=flush center,minimum size=20mm,minimum height=12mm},
TxtC/.style = {font=\sffamily,text width=44mm,align=flush center},
arr/.style = {-{Triangle[length=3mm,width=6mm]}, color=#1!60,
                    line width=3mm, shorten <=1mm, shorten >=1mm},
Line/.style = {},
LineA/.style = {violet!60,{Circle[line width=1.5pt,fill=white,length=7.5pt]}-,line width=2.0pt,shorten <=-4pt},
LineAA/.style={violet!30,dashed, line width=1.0pt,{-{Triangle[width=1.0*6pt,length=1.6*6pt]}},shorten <=3pt,shorten >=2p
}

\tikzset{pics/brain/.style = {
        code = {
        \pgfkeys{/channel/.cd, #1}
\begin{scope}[local bounding box=BRAIN,scale=\scalefac, every node/.append style={transform shape}]
\draw[fill=\filllcolor,line width=\Linewidth](-0.3,-0.10)to(0.08,0.60)
to[out=60,in=50,distance=3](-0.1,0.69)to[out=160,in=80](-0.26,0.59)to[out=170,in=90](-0.46,0.42)
to[out=170,in=110](-0.54,0.25)to[out=210,in=150](-0.54,0.04)
to[out=240,in=130](-0.52,-0.1)to[out=300,in=240]cycle;
\draw[fill=\filllcolor,line width=\Linewidth]
(-0.04,0.64)to[out=120,in=0](-0.1,0.69)(-0.19,0.52)to[out=120,in=330](-0.26,0.59)
(-0.4,0.33)to[out=150,in=280](-0.46,0.42)
%
(-0.44,-0.03)to[bend left=30](-0.34,-0.04)
(-0.33,0.08)to[bend left=40](-0.37,0.2) (-0.37,0.12)to[bend left=40](-0.45,0.14)
(-0.26,0.2)to[bend left=30](-0.24,0.13)
(-0.16,0.32)to[bend right=30](-0.27,0.3)to[bend right=30](-0.29,0.38)
(-0.13,0.49)to[bend left=30](-0.04,0.51);
\draw[rounded corners=0.8pt,line width=2*\Linewidth,\drawcircle,-{Circle[fill=\filllcolor,length=5.5pt]}](-0.23,0.03)--(
\draw[rounded corners=0.8pt,line width=2*\Linewidth,\drawcircle,-{Circle[fill=\filllcolor,length=5.5pt]}](-0.17,0.13)--(
\draw[rounded corners=0.8pt,line width=2*\Linewidth,\drawcircle,-{Circle[fill=\filllcolor,length=5.5pt]}](-0.12,0.23)--(
\draw[rounded corners=0.8pt,line width=2*\Linewidth,\drawcircle,-{Circle[fill=\filllcolor,length=5.5pt]}](-0.07,0.32)--(
\draw[rounded corners=0.8pt,line width=2*\Linewidth,\drawcircle,-{Circle[fill=\filllcolor,length=5.5pt]}](-0.01,0.43)--(
\coordinate(PO)at(-0.1,0.2);
\node[circle,draw=white,line width=1pt,fill=\filllcirclecolor,minimum size=5mm,inner sep=0pt](LV)at(PO){};
\node[draw=none,rotate=40,rounded corners=2pt,rectangle,minimum width=1.2mm,inner sep=1pt,
fill=\filllcirclecolor,minimum height=6mm,anchor=north]at(PO){};
\node[circle,draw=none,fill=white,minimum size=3.0mm,inner sep=0pt](LM)at(PO){};
\node[font=\tiny\bfseries]at(LM){...};
\end{scope}
     }
  }
}
\tikzset{pics/factory/.style = {
        code = {
        \pgfkeys{/channel/.cd, #1}
\begin{scope}[local bounding box=FACTORY,scale=\scalefac, every node/.append style={transform shape}]
\node[rectangle,draw=\drawcolor,fill=\filllcolor!50,minimum height=15,minimum width=23,,line width=\Linewidth](R1){};
\draw[fill=\filllcolor!50,line width=1.0pt]($(R1.40)+(0,-0.01)$)--++(110:0.2)--++(180:0.12)|-($(R1.40)+(0,-0.01)$);
\draw[,line width=\Linewidth,fill=green](-0.68,-0.27)--++(88:1.10)--++(0:0.15)--(-0.48,-0.27)--cycle;
\draw[line width=2.5pt](-0.8,-0.27)--(0.55,-0.27);

\foreach \x in{0.25,0.45,0.65}{
```

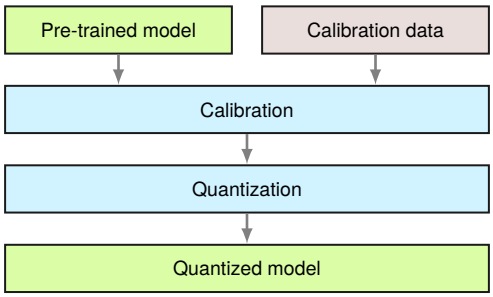


Figure 10.21: Post-Training Quantization Pipeline: PTQ converts a trained FP32 model directly to INT8 without retraining. Representative calibration data passes through the model to record activation ranges $[\alpha, \beta]$, computing scale factor s and zero-point z parameters to quantize parameters before deployment.

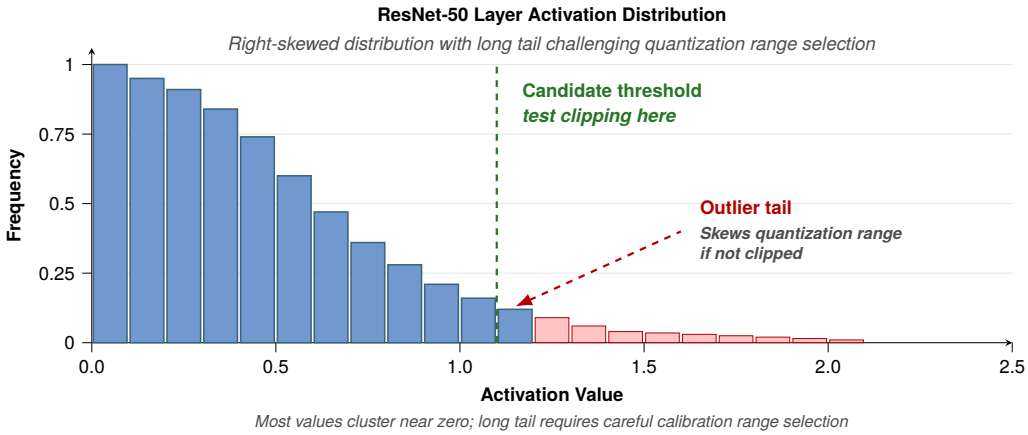


Figure 10.22: Activation Distribution: ResNet-50 layer activations exhibit a long tail, with outlier values that can lead to inefficient precision use if not handled carefully. Source: (Wu et al. 2020).

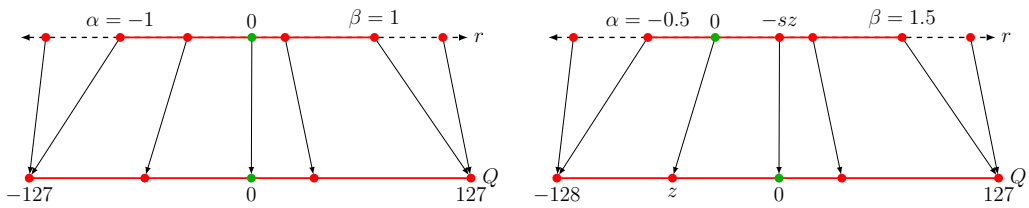


Figure 10.23: Calibration Range Selection: Symmetric vs. asymmetric affine quantization mappings. Symmetric quantization (left) forces real zero to map to integer zero, using equal-magnitude negative and positive clipping bounds at the expense of wasted dynamic range for skewed activation distributions; asymmetric quantization (right) uses an explicit zero-point offset z to map non-centered ranges $[\alpha, \beta]$ efficiently.

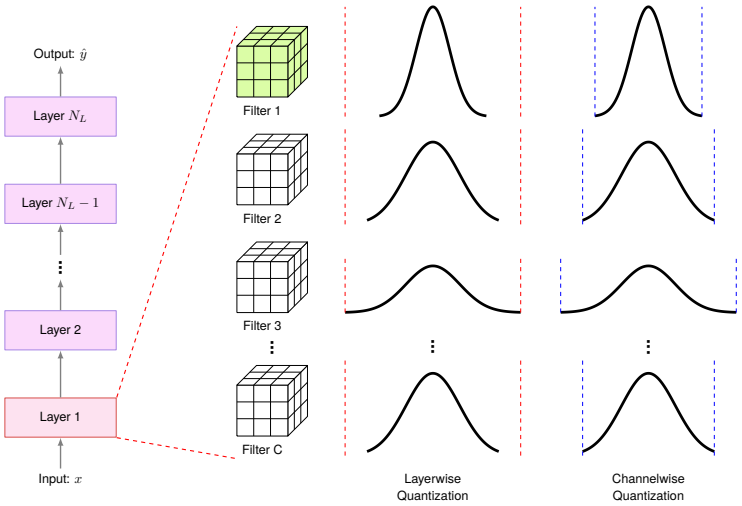


Figure 10.24: Quantization Range Variation: Different convolutional filters can have different weight ranges, motivating per-filter quantization to reduce quantization error. The differing clipping scales illustrate the choice between one layerwise range and separate channelwise ranges. Source: (Gholami et al. 2022).

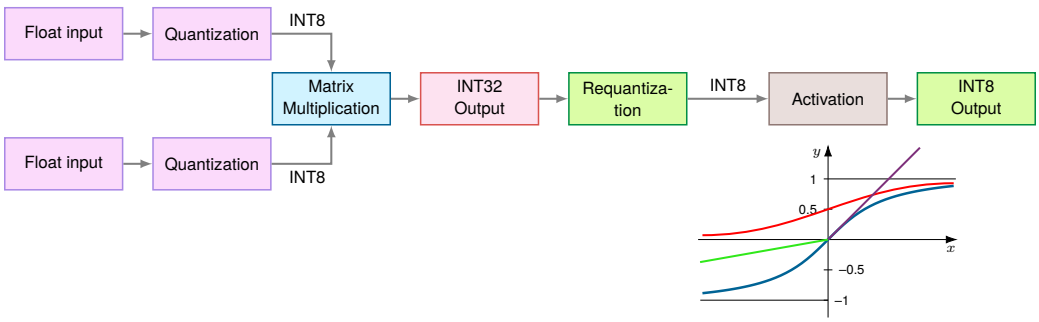


Figure 10.25: Quantized Matrix Multiplication Pipeline: Color-coded computation stage. Floating-point weights and activation inputs are quantized to INT8 (violet), fed into integer SIMD/Tensor Core matrix multiplication (blue), accumulated in 32-bit registers (red), and requantized back to INT8 (green) before non-linear activation. From the HarvardX TinyML course.

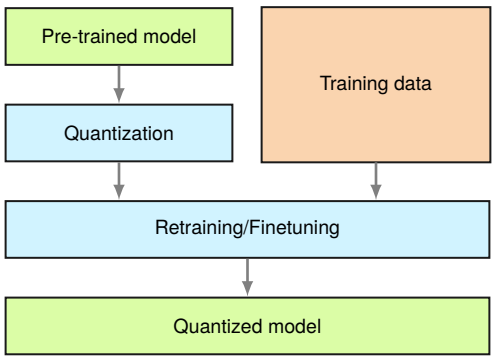


Figure 10.26: Quantization-Aware Training Pipeline: QAT inserts fake-quantization nodes into the forward graph to simulate low-precision clipping and rounding errors during training. Backpropagation updates floating-point master weights using straight-through estimators (STE), allowing network parameters to adapt to numerical noise before exporting fixed integer scales for inference.

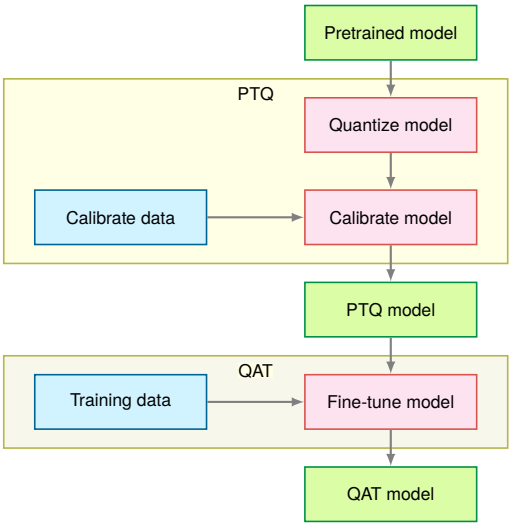


Figure 10.27: PTQ-to-QAT Pipeline: PTQ calibration can initialize ranges and scales before QAT fine-tunes the pretrained weights with simulated quantization. Calibration data feeds the PTQ stage, and training data feeds QAT.

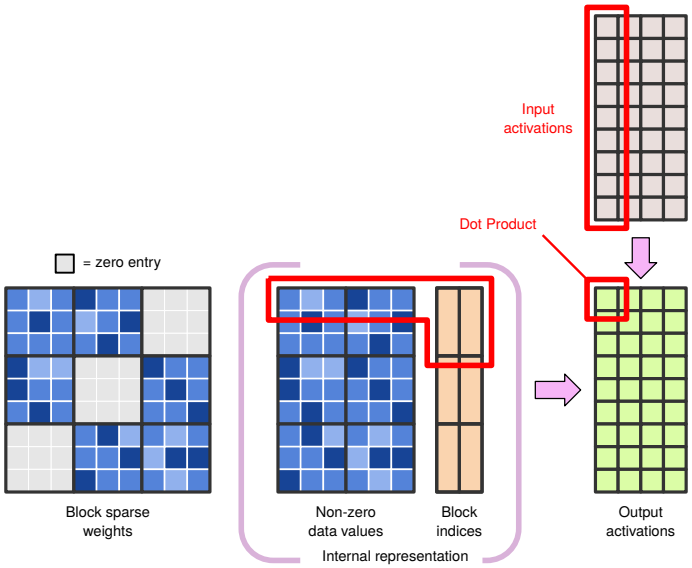


Figure 10.28: Block Sparse Representation: Grouping non-zero weights into dense sub-blocks and storing an auxiliary index array lets matching kernels skip empty blocks. The 3×3 tiles illustrate the representation; efficient GPU execution depends on supported block shapes, layouts, and kernels. Adapted from NVIDIA cuSPARSE documentation (NVIDIA 2020a).

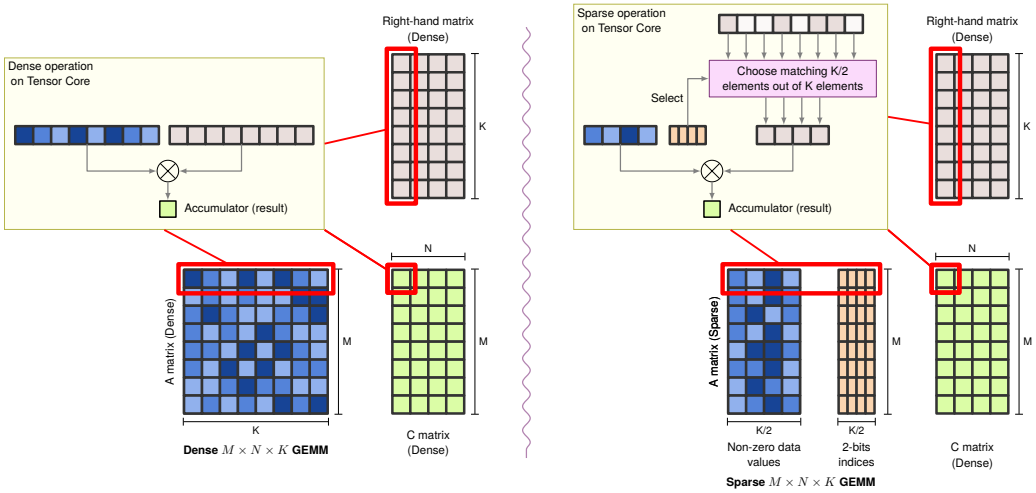


Figure 10.29: 2:4 Structured Sparsity GEMM: Left: standard dense matrix multiplication on Tensor Cores using full 8-element rows. Right: 2:4 sparse multiplication retains two nonzeros in every group of four and uses compact indices to select matching elements from the dense right-hand matrix. A supported sparse inner loop can halve the multiplied values, although metadata handling and the rest of the execution path determine end-to-end speedup. Source: PyTorch blog (PyTorch 2022).

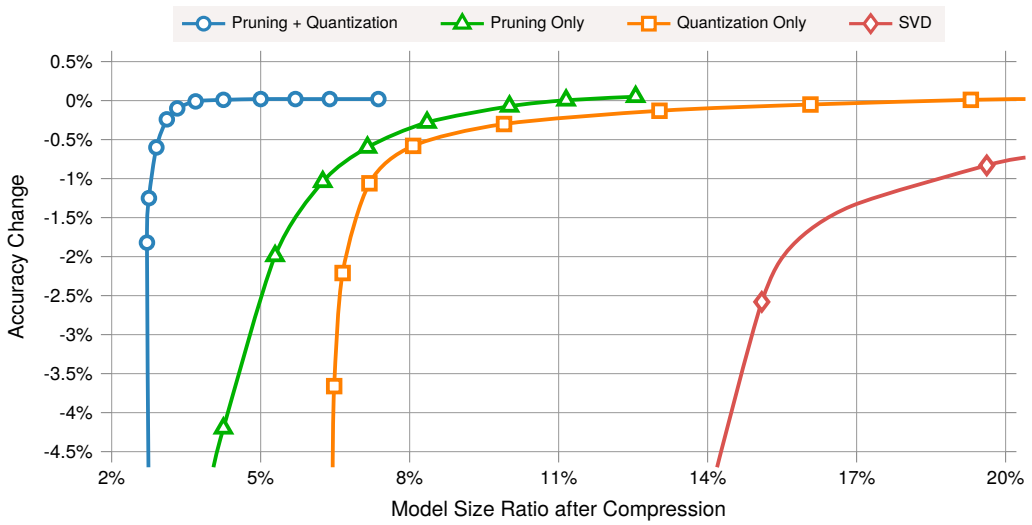


Figure 10.30: Compression Methods in One Reported Setting: The cited comparison plots accuracy change against model-size ratio for pruning, quantization, their combination, and SVD. In this setting, pruning plus quantization reaches the smallest ratio near baseline accuracy; the ordering is model- and implementation-specific. Source: (Han, Mao, et al. 2016).

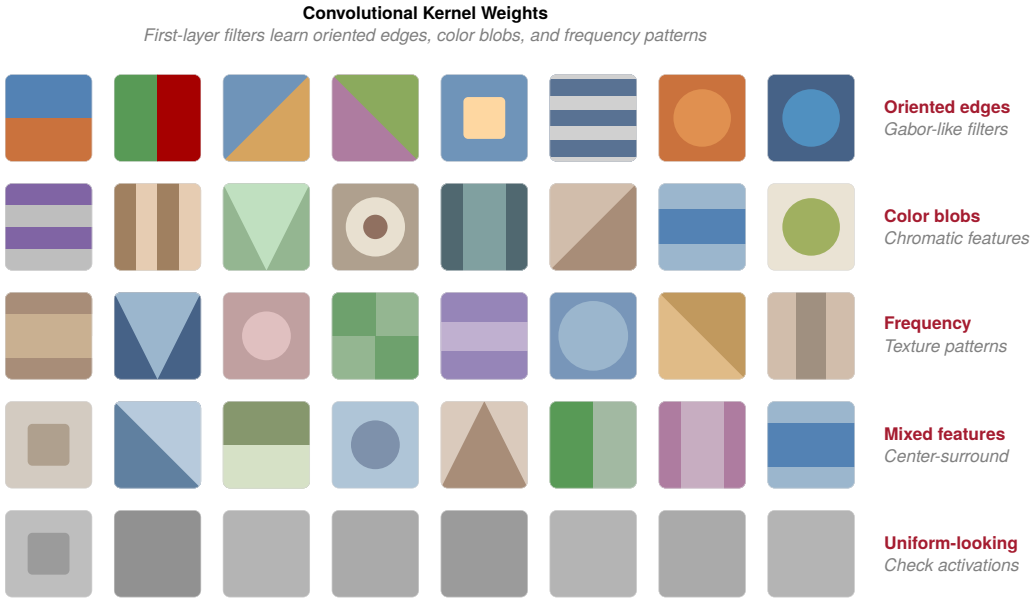


Figure 10.31: First-Layer Convolutional Filter Patterns: An 8×5 grid groups stylized kernels into edge, color, frequency, mixed, and uniform-looking patterns. Appearance alone does not establish inactivity; pruning requires measured activation or importance evidence in addition to low weight variance.

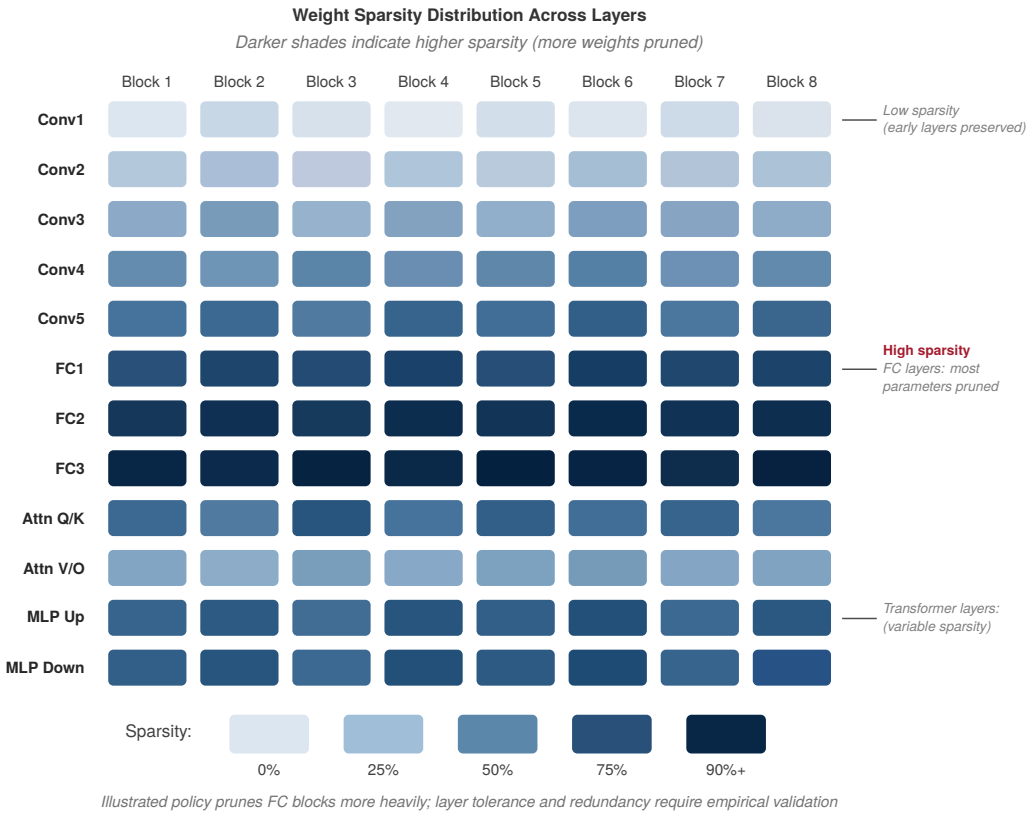
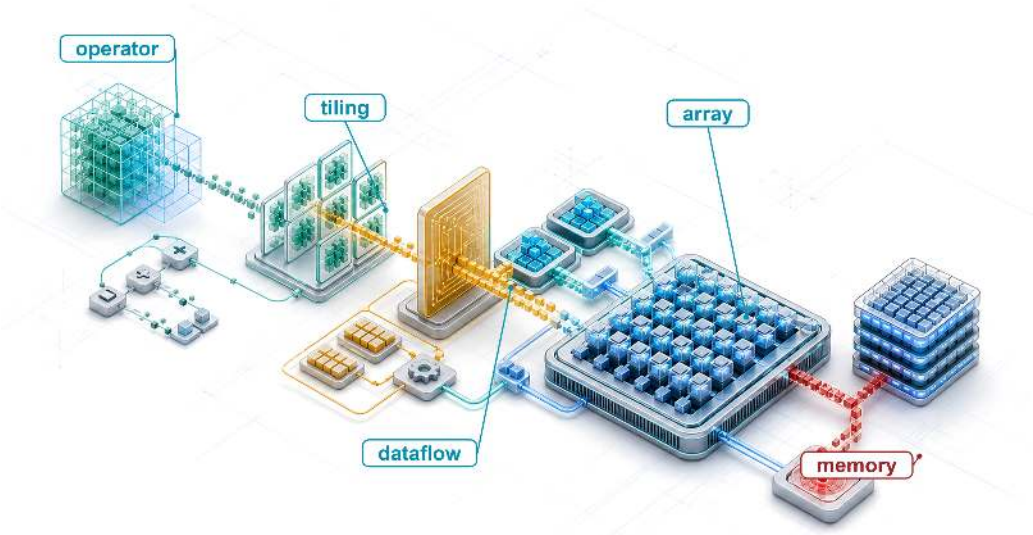


Figure 10.32: Illustrative Layer-Wise Sparsity Distribution: This schematic combines several layer types to illustrate non-uniform parameter removal. Darker tiles indicate more zero-valued weights; their shade establishes neither task redundancy nor a runtime benefit.

11

Hardware Acceleration



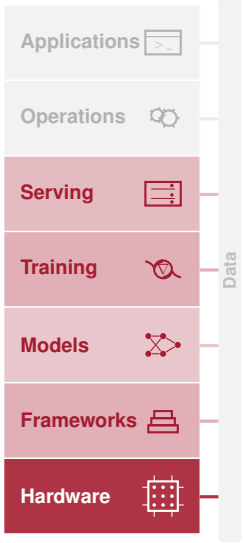
11.1 Acceleration Fundamentals
11.2 Hardware Specialization
11.3 AI Compute Primitives
11.4 Compute Units and Execution Models
11.5 AI Memory Systems
11.6 Roofline Model
11.7 Hardware Mapping
11.8 Dataflow Optimization
11.9 Compiler Support
11.10 Runtime Support
11.11 Multi-Chip Scaling
11.12 Heterogeneous SoC Design
11.13 Hardware Sustainability
11.14 Fallacies and Pitfalls
11.15 Summary

Purpose

Why does moving data cost more than computing it?

Arithmetic is inexpensive relative to data movement. In the time required to fetch a single scalar operand from off-chip DRAM, modern arithmetic units can execute dozens to hundreds of floating-point operations, and that DRAM access consumes on the order of a hundred times more energy than the computation itself. This fundamental imbalance—the memory wall—means specialized accelerators are not merely faster arithmetic engines. Rather, their microarchitectures are designed to stage, amortize, and hide data movement through wide spatial interconnects, deep on-chip memory hierarchies, and specialized execution pipelines. When general-purpose processor scaling faltered, hardware specialization enabled machine learning models to continue scaling by turning transistor budgets into high-throughput tensor datapaths.

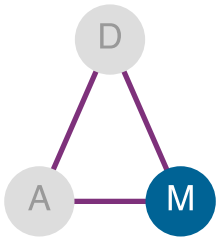
Delivered performance, however, depends on whether a model's tensor access patterns match the data paths of the underlying silicon. Hardware selection cannot be reduced to comparing peak FLOP/s: two accelerators with identical theoretical compute ratings often deliver radically different throughput on real workloads because one saturates its functional units while the other starves for data. A recommendation model dominated by sparse embedding lookups stresses memory bandwidth and interconnect latency, demanding a fundamentally different memory architecture than a dense transformer training run bounded by matrix multiply-accumulate throughput. If an operation is memory-bound, reducing theoretical operation count yields zero speedup because arithmetic is not the binding constraint. Matching workload characteristics to architectural dataflows determines whether an accelerator operates near its hardware ceiling or languishes at a fraction of its advertised peak. In D·A·M terms, the accelerator makes the machine constraint concrete, governing which algorithms and model architectures remain practical in production.



Learning Objectives

- Explain hardware acceleration as machine-axis specialization for tensor workloads, data reuse, and performance per watt
- Calculate arithmetic intensity and execution-path roofline ceilings, then use profiling to diagnose compute or memory limits
- Diagnose memory-wall bottlenecks using bandwidth, cache hierarchy, host-device transfer, and energy-movement costs
- Compare Tensor Cores, systolic arrays, SIMD/SIMT units, and sparse execution for ML compute primitives
- Select dataflow, tiling, and mapping strategies that maximize reuse under memory-capacity constraints
- Analyze compiler and runtime optimizations that fuse kernels, plan memory, and schedule accelerators
- Evaluate accelerator choices across throughput, latency, power, cost, and deployment-context constraints

11.1 Acceleration Fundamentals



Hardware acceleration turns on the machine axis.

Model compression closed on the observation that the hardware sets the exchange rate for every saving it buys. Fewer parameters, lower precision, or fewer operations matter only when the machine can execute the resulting representation efficiently. Data selection reduced the data term and compression reduced the algorithm's work, so hardware acceleration asks what the machine can actually deliver. The answer starts with the memory wall: arithmetic is often cheap relative to data movement. An off-chip memory access can take many processor cycles and consume orders of magnitude more energy than a low-precision arithmetic operation, with the exact ratio depending on the device and memory level. Specialized hardware matters because it raises compute throughput while organizing memory, dataflow, and parallelism so those arithmetic units stay fed.

Definition 11.1: Hardware acceleration

Hardware acceleration is the practice of mapping work from general-purpose execution onto specialized processors or functional units optimized for a narrower operation class. For ML workloads, that specialization commonly trades some programmability for the compute density (R_{peak}) and performance-per-watt gains that regular tensor operations can exploit.

1. **Significance**: An A100 GPU delivers 312 TFLOP/s for FP16/BF16 matrix multiplication; against this chapter's illustrative 1–2 TFLOP/s reference-CPU baseline, this is a 156–312× gap. Its 54.2 billion transistors devote far more silicon to parallel arithmetic than to branch prediction, out-of-order scheduling, and large caches (NVIDIA Corporation 2020a; Choquette et al. 2021).
2. **Distinction**: A general-purpose CPU is designed for flexible execution, low-latency control, and varied workloads. An accelerator instead concentrates resources on a narrower operation class, so it achieves its gains only when the workload exposes the supported operations and enough parallel work to keep its arithmetic units busy.
3. **Common pitfall**: A frequent misconception is that an accelerator's advertised peak throughput is the throughput a workload receives. Delivered performance is bounded above by the lower of the compute ceiling and what memory bandwidth can feed, the roofline constraint: a low-arithmetic-intensity kernel can sit at a small fraction of peak FLOP/s no matter how fast the silicon is rated, because it starves for data rather than for arithmetic.

This definition frames the chapter's central engineering trade-off. General-purpose processors allocate silicon area to complex control logic, speculative execution, branch prediction, and multi-level cache hierarchies to handle arbitrary control flow. An accelerator redirects that transistor budget toward dense arithmetic arrays, dedicated register files, and high-bandwidth memory paths tailored to a narrower workload class. When the software path exposes sufficient tensor parallelism, this specialization yields order-of-magnitude improvements in energy efficiency and throughput per watt. Giving up that flexibility is a bet on the bitter lesson. Silicon built for a narrow class of dense tensor operations pays off only because the general methods that win by exploiting computation keep reducing to those same operations at ever larger scale. The dependence also runs the other way, because scale keeps winning only while accelerators keep lowering the time and energy of each additional operation, so the hardware this chapter studies is part of what sustains the bitter lesson itself.

↩ The computational bet of the bitter lesson is established in section 1.4.

Hardware alone cannot achieve these gains. Algorithms must structure operations to expose data reuse and regular memory access, while silicon architectures must allocate physical units to the mathematical primitives and numeric precisions that algorithms actually execute. This symbiosis motivates a complementary principle: hardware-software co-design.

Definition 11.2: Hardware-software co-design

Hardware-Software Co-design is an ML accelerator development method that crosses conventional abstraction boundaries, allowing algorithm constraints to inform silicon design and hardware capabilities to shape algorithm formulation.

1. **Significance**: On a supported tensor-core path, 8-bit integer (INT8) quantization can deliver multi-fold throughput gains because hardware executes lower-precision operations more densely and moves fewer bytes than an FP32 path; the exact gain depends on the architecture and workload (NVIDIA Corporation 2020a; Dally et al. 2021; Dally 2023).
2. **Distinction**: Unlike layered abstraction (where software calls a hardware API without knowing the silicon details), co-design exposes hardware constraints directly to algorithm and compiler authors: data alignment requirements, precision formats, and memory access patterns all become visible inputs to global cross-layer optimization.
3. **Common pitfall**: A frequent misconception is that co-design is a one-time hardware design choice. In practice, the supported features continue to evolve: NVIDIA Tensor Cores began with FP16 matrix multiplication, later generations added TF32 and broader integer support, and Ampere added acceleration for 2:4 structured sparsity (NVIDIA 2017; NVIDIA Corporation 2020a).

Co-design explains why model compression techniques deliver real speedups. Quantization shows why converting FP32 to INT8 accelerates execution on supported hardware: reduced precision cuts memory traffic, while specialized arithmetic units execute more low-precision operations per cycle than FP32 equivalents (NVIDIA Corporation 2020a). Structured pruning improves performance when it produces smaller dense shapes or a sparsity pattern recognized by silicon, whereas arbitrary unstructured sparsity incurs metadata handling and non-coalesced memory accesses that often erase theoretical gains. Evaluating accelerator performance requires tracing this path from workload to silicon: compute primitives, memory hierarchies, roofline diagnosis, mapping and dataflow, and compiler scheduling. The recurring question is why algorithmic improvements frequently fail to deliver proportional end-to-end speedups. Amdahl's law (principle 8) already answered it for model-side optimizations, since whatever an optimization leaves untouched caps the overall gain. The same bound governs hardware. The theorem below applies the principle to an accelerator, and the only new quantity is the gain, which becomes the accelerator's path gain G_{accel}.

↩ Quantization and its effect on weight payload, memory traffic, and compute cost are established in section 10.4.

Theorem 11.1: The fundamental limit of acceleration (Amdahl's law)

Hardware acceleration *does not speed up the entire system*; it speeds only the baseline-time fraction using the accelerated path. For that fraction f_{parallel} and path gain G_{accel}, Amdahl's law (Amdahl 1967) gives the bound in equation 11.1.

$$\text{Speedup} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{G_{\text{accel}}}} \tag{11.1}$$

- **Accelerated fraction** (f_{parallel}): Share of baseline time receiving the stated gain, often in matrix multiplication or another supported kernel.
- **Accelerator gain** (G_{accel}): The raw speed advantage of the GPU or Tensor Processing Unit (TPU) over the CPU for the accelerated portion of the workload.
- **Unaccelerated fraction** ($1 - f_{\text{parallel}}$): Work outside the chosen path, such as data loading, host overhead, unsupported operators, and kernel launch latency.

Pitfall: *Unaccelerated work caps total speedup.* If 10 percent of baseline time is unaffected ($f_{\text{parallel}} = 0.9$), even infinite path gain ($G_{\text{accel}} = \infty$) yields at most $10\times$ overall. Once the accelerated component is fast enough, unaffected work dominates.

Hardware acceleration targets specific terms in the iron law of ML systems, which decomposes end-to-end time into data volume (D_{vol}/BW), computation ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), and fixed latency ($L_{\text{lat}}$). While data selection reduced total data volume and model compression reduced operations O per sample, hardware acceleration increases the rate of execution by improving R_{peak}, η_{hw}, and BW. Which of these terms an upgrade actually improves determines its accelerated fraction under Amdahl's law.[1]

This dynamic explains why hardware upgrades frequently fail to produce proportional end-to-end speedups. Figure 11.1 maps system speedup across accelerator gain (G_{accel}) and accelerated fraction (f_{parallel}), charting where those returns diminish. Where the unaccelerated fraction $1 - f_{\text{parallel}}$ remains substantial, the white contour curves flatten horizontally into the unaccelerated-bound regime, bounding total speedup at the $1/(1 - f_{\text{parallel}})$ asymptote regardless of how far G_{accel} scales along the horizontal axis.

[1] **Amdahl's law**: Maps onto the iron law only after the proposed change is specified. If a faster tensor unit reduces the computation term $(O/(R_{\text{peak}} \cdot \eta_{\text{hw}}))$ but leaves data transfer and fixed latency unchanged, those unaffected terms bound total speedup. A different accelerator or memory system may improve bandwidth as well. The unaccelerated fraction is therefore defined by the intervention, not by a permanent classification of each term as serial.

↩ The decomposition of execution time into data, compute, and latency terms is established in section 1.7.

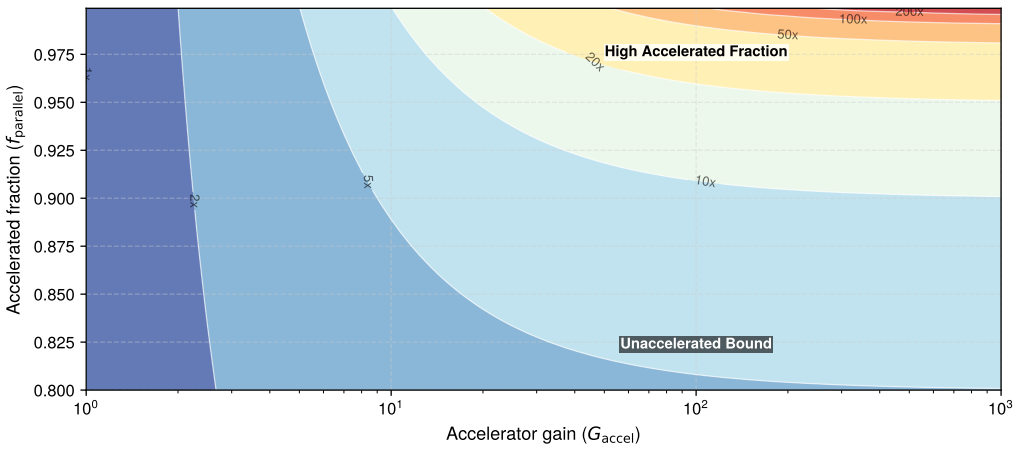


Figure 11.1: The Iron Law Heatmap: Total system speedup as a function of accelerator gain (G_{accel}) and accelerated fraction (f_{parallel}). High speedup appears only near the top-right corner, where both accelerator gain and accelerated fraction are high. At $f_{\text{parallel}} = 0.9$, for example, even an infinite gain on the accelerated path cannot exceed $10\times$ end-to-end speedup. Contours span roughly $1\times$–$200\times$ speedup.

The unaccelerated fraction limits how much raw accelerator speedup translates into system-level throughput. The accelerated fraction f_{parallel} differs substantially across workload archetypes running on identical hardware, determining whether an accelerator investment pays off or stalls on non-matrix operations.

Checkpoint 11.1: The parallelism gate

These checks turn Amdahl's bound into a hardware-selection test.

Amdahl's reality

- ☐ **Serial bottlenecks**: use Amdahl's bound to explain why a 1,000× faster GPU may only speed up training by 5× when data loading is slow.
- ☐ **Speedup ceiling**: for $f_{\text{parallel}} = 0.95$ and $G_{\text{accel}} = 100$, estimate total speedup and compare it with the infinite-accelerator ceiling.
- ☐ **Workload variation**: what baseline profiling would establish the ResNet-50 and MobileNet accelerated fractions for a proposed hardware change? Explain how those fractions would determine which benefits more.

The serial bottleneck becomes concrete on real hardware, where the same accelerator that nears its parallel ceiling on one workload can stall on another. The comparison below uses the INT8 (8-bit integer) path, whose advantage over FP32 or FP16 has two sources. Each operand occupies a half or a quarter of the bytes, and an 8-bit multiplier occupies far less silicon area than a floating-point one, so the same die holds more multiply-accumulate (MAC) units. Evaluating an NVIDIA H100 on two workloads whose unaccelerated fractions differ illustrates how that fraction governs whether this peak INT8 advantage becomes realized speedup, and a second look at the decode case separates that limit from a different one, memory-boundedness.

↪ Order-of-magnitude reference values for accelerator throughput, memory latency, and bandwidth are developed in section D.1.

Lighthouse 11.1: An illustrative Amdahl budget on H100

Illustrative ResNet-50 inference on NVIDIA H100:

- Assume H100 provides $G_{\text{accel}} = 247\times$ on matrix multiplication relative to a server CPU without matrix extensions (1979 TOPS INT8 vs. ~8 TOPS) (Choquette 2023). The CPU's INT8 rate is a scenario assumption for this budget, not the FP32 peak of the reference CPU used elsewhere in the chapter.
- Assume $f_{\text{parallel}} = 0.95$: 95 percent of baseline time receives that gain, while 5 percent remains in data loading, preprocessing, postprocessing, and other work.

$$\text{Speedup} = \frac{1}{(1-0.95) + \frac{0.95}{247}} = \frac{1}{0.05 + 0.0038} \approx 18.6\times$$

The 247× path gain produces only 18.6× end-to-end speedup because the 5 percent unaffected fraction sets the ceiling.

Contrast with GPT-2 (autoregressive):

- For GPT-2 token generation, assume $f_{\text{parallel}} = 0.80$; the remaining 20 percent includes host overhead, sampling, and decoding work outside the accelerated matrix path.

$$\text{Speedup} = \frac{1}{(1-0.80) + \frac{0.80}{247}} = \frac{1}{0.20 + 0.0032} \approx 4.9\times$$

Less token-generation time receives the assumed gain, so even an infinite path gain yields at most $1/(1 - f_{\text{parallel}}) = 5\times$.

This calculation still overstates the decode path gain. Decode is memory bound, so its accelerated path runs at the rate weights stream from memory rather than at the INT8 TOPS

rate. The relevant ratio is the H100's memory bandwidth over the reference CPU's, 67×, and INT8 helps decode on either machine by halving the bytes each weight occupies relative to FP16, not by multiplying faster. With that gain the bound falls from 4.9× to 4.7×, barely moving, because the unaccelerated fraction already dominates. Two distinct limits are at work. Memory-boundedness shrinks G_{accel}, whereas the unaccelerated fraction caps what any G_{accel} can deliver. Serving optimizations attack both. Batching raises the work done per weight byte fetched, and speculative decoding uses a draft model to reduce sequential target-model steps.

Whether an operation is limited by compute rate or by data movement dictates both hardware selection and software optimization. Under the arithmetic intensity law (principle 6), upgrading to an accelerator with $10\times$ higher peak compute yields negligible speedup if data delivery is the binding bottleneck. The roofline model already familiar from earlier chapters makes this diagnosis (Williams et al. 2009). It compares an operation's arithmetic intensity,[2] the ratio of floating-point operations to bytes of memory traffic (FLOP/byte), with the hardware's ridge point. A large, well-tiled dense matrix multiplication can sit above its tensor-path ridge, making arithmetic throughput the tighter modeled ceiling. Under an unfused high-bandwidth memory (HBM) traffic model, LayerNorm sits far below that reference ridge, making traffic reduction a candidate improvement; its vector and reduction execution limits still require profiling. Compute-bound kernels therefore call for higher arithmetic throughput and functional-unit utilization, whereas memory-bound kernels call for kernel fusion and memory-hierarchy staging that cut off-chip traffic. Section 11.6 quantifies this classification once the compute units and memory systems that set the two ceilings have been examined.

Architectural design begins with the physical constraints that forced specialization: the breakdown of classical Dennard voltage scaling and the prohibitive energy cost of moving data across memory hierarchies. When general-purpose microarchitectures encountered these thermal and memory barriers, domain-specific hardware specialization emerged as the primary engineering path to sustain performance scaling.

11.2 Hardware Specialization

When Google deployed its first Tensor Processing Unit (TPUv1) into production data centers alongside NVIDIA K80 GPUs, the comparison exposed a fundamental efficiency divide. General-purpose processors dedicate substantial silicon area and power to instruction fetch, branch prediction, and cache coherence, machinery built for arbitrary code that yields poor arithmetic density on regular, dense linear algebra. When an algorithmic pattern becomes sufficiently dominant and structured, specialized hardware outperforms general-purpose platforms by replacing dynamic control machinery with dedicated arithmetic arrays and software-managed data movement. Google's production inference evaluation quantified this efficiency gap (example 11.1).

Example 11.1: The TPUv1 vs. K80 efficiency shock

Scenario: In 2015, Google deployed its first Tensor Processing Unit (TPUv1) into production data centers, comparing its throughput and energy efficiency against general-purpose NVIDIA K80 GPUs (Jouppi et al. 2017).

Diagnosis: The K80 was a programmable GPU designed to support a broader mix of graphics and compute workloads. The TPUv1 domain-specific architecture (DSA) made a narrower inference-focused bet, pairing a large 8-bit systolic matrix unit with software-managed memory and simpler control.

Systems lesson: On the six Google inference workloads and contemporary CPU/GPU baselines evaluated in the TPU paper, tailoring silicon to the dominant matrix operations delivered 15–30× higher throughput and 30–80× better performance per watt (Jouppi et al. 2017). Those ratios describe the evaluated systems, not every workload.

In D·A·M terms, this efficiency gain stems from tailoring machine resources to algorithmic structure. The TPU[3] replaced hardware caches with software-managed buffers and routed data directly

[2] **Arithmetic intensity**: The ratio of compute operations to bytes transferred across the memory interface being modeled (FLOP/byte). For a matching arithmetic path, intensity above that level's ridge makes compute the lower modeled ceiling; intensity below it makes bandwidth the lower ceiling. Well-tiled matrix multiplications can approach their compute ceiling, while low-intensity kernels may benefit from more bandwidth or reuse. Profiling determines whether dependencies, dispatch, or another execution resource limits performance below either bound.

↩ Arithmetic intensity and the spectrum of workload signatures it separates are established in section 6.3.

[3] **TPU (tensor processing unit)**: The first TPU made a deliberately narrow bet, centering the design on a single 256×256 systolic array for 8-bit matrix multiplication and avoiding much of the control machinery required by a general-purpose core (Jouppi et al. 2017). That trade buys high compute density on dense matrix multiplication at the cost of flexibility. Irregular or branch-heavy code remains a poor fit, and compiler tiling, padding, and layer dimensions determine how fully the array is used.

between neighboring arithmetic units, raising data reuse and cutting off-chip traffic. The price is rigidity. Irregular memory access, dynamic control flow, or an unsupported numeric precision stalls a fixed-function data path and leaves expensive silicon idle. Every modern ML accelerator, from DianNao-class designs (Chen et al. 2014) to GPUs with Tensor Cores and Apple's Neural Engine, strikes its own balance between specialization and flexibility, following a pattern that began decades before deep learning.

11.2.1 Specialized computing

Hardware specialization emerges when one computational pattern becomes the bottleneck that general-purpose processors cannot scale past. The first clear case was floating point, which late-1970s microprocessors emulated with multi-instruction software routines. The Intel 8087 coprocessor (1980) executed those operations directly in hardware (Palmer 1980), and the Intel 486DX (1989) later moved the floating-point unit onto the CPU die itself (Hennessy and Patterson 2017). That cycle, specialization to relieve an acute bottleneck followed by integration once the capability proves universal, recurs in every later era. So does its limit. Dedicated silicon removes the instruction overhead around a dominant operation, but the overall gain is bounded by how much of the run time that operation occupies.

Graphics supplied the second lesson, throughput over latency. A CPU spends most of its silicon on control logic, branch prediction, and caches that minimize the latency of a single instruction stream, which buys nothing when identical arithmetic must be applied to millions of independent pixels. GPUs reallocated that silicon to dense arrays of arithmetic units, and when unified shader architectures (2006) replaced separate vertex and pixel pipelines with arrays of identical programmable cores, the GPU became a general-purpose parallel processor (Owens et al. 2008). Digital signal processors such as Texas Instruments' TMS32010 (1983) contributed the memory organization. To meet hard real-time deadlines, they replaced caches, whose misses introduce variable latency, with software-managed scratchpads feeding dedicated MAC data paths (Lyons 2011). ML accelerators inherit both lessons.

The payoff arrived in 2012, when AlexNet (Krizhevsky et al. 2012) won the ImageNet competition by more than ten percentage points after training on two consumer-grade NVIDIA GTX 580 cards with 3 GB of memory each. Convolutional neural networks (CNNs) trained with gradient descent had existed for decades, but CPU throughput kept large-scale training impractical. Dense 2D convolutions mapped directly onto thousands of GPU arithmetic lanes, compressing months of CPU computation into five to six days. In D·A·M terms, the machine constraint had dictated which algorithms were feasible, and aligning the algorithm's parallel structure with throughput-oriented hardware lifted it.

Figure 11.2 sequences these architectural milestones across five chronological columns from the 1980s to the 2020s, tracing the recurring evolution from discrete coprocessors (such as the Intel 8087 FPU and TMS32010 DSP) through graphics engines and dedicated media codecs to deep-learning tensor arrays and wafer-scale hardware. Each era's specialized unit targets a different bottleneck, yet the principles modern accelerators depend on (dedicating silicon to the dominant operation, staging operands in local memory, and minimizing off-chip data movement) were all in place before deep learning arrived.

11.2.2 Emergence of domain-specific architectures

These patterns converged into a broader architectural shift once general-purpose scaling slowed. Moore's law[4] describes a long-running increase in transistor density (Moore 1998), and for decades Dennard scaling (Dennard et al. 1974) let operating voltage fall as transistors shrank, so clock frequencies rose at constant power density and existing software simply ran faster. When leakage current halted voltage scaling in the mid-2000s, frequencies stalled against a thermal wall. Adding cores bought time but ran into Amdahl's Law and **dark silicon**, the thermal limit under which only a fraction of a chip's transistors can switch at full speed simultaneously (Esmaeilzadeh et al. 2011). A general-purpose core also spends most of its energy on instruction fetch and decode, branch prediction, speculation, and cache coherence rather than arithmetic; executing one instruction can cost hundreds of picojoules of control energy to perform a floating-point operation that itself costs

[4] **Moore's law**: Gordon Moore's 1965 observation, revised in 1975, that the number of transistors per chip doubles roughly every two years (Moore 1998). It describes density, not performance; performance tracked density only while Dennard scaling held.

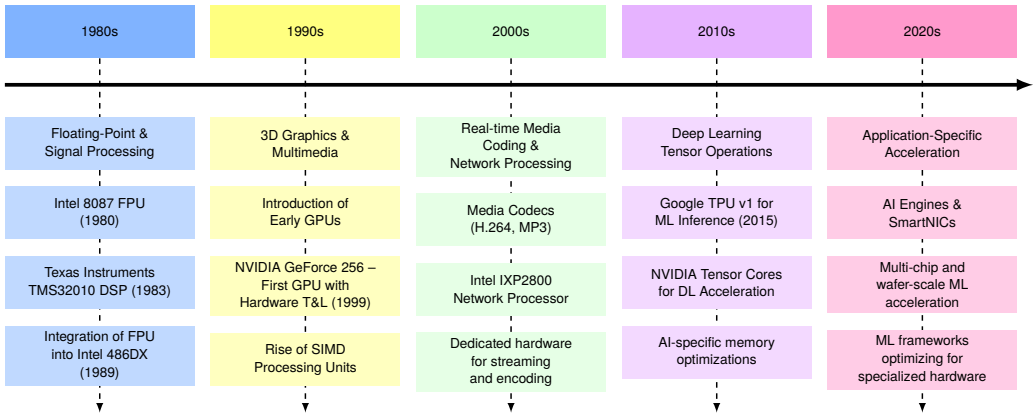


Figure 11.2: Hardware Specialization Timeline: Computing architectures evolve through recurring cycles of specialization and integration across five eras. Each era introduces domain-specific hardware to resolve an important memory or arithmetic bottleneck—from off-chip FPUs to GPUs, codecs, and TPU/Tensor Core array units. Many of these capabilities later appear alongside CPUs in broader systems, even when the specialized units remain distinct.

a few. As Hennessy and Patterson (2019) argued in their Turing Lecture, the response is **domain-specific architectures (DSAs)**,[5] hardware that gives up some programmability to spend its power budget on the operations an important domain actually performs.

A DSA wins by specializing every layer of the machine around the workload's invariants. Its data path is custom-built; matrix units in AI accelerators are often **systolic arrays**, grid-like networks of processing elements that compute and forward data directly to neighboring units, so a value fetched once from on-chip memory is reused across an entire row or column of multiply-accumulate operations instead of returning to a register file after each use. Its memory hierarchy follows the workload's reuse pattern, with software-managed scratchpads and prefetching replacing hardware-managed caches for deterministic tensor streams. Its instruction set encodes coarse tensor operations, so one macro-instruction configures thousands of arithmetic units. The same recipe produced smartphone video codecs, genomics aligners, and cryptographic application-specific integrated circuits (ASICs) wherever a kernel stabilized (Sullivan et al. 2012; Turakhia et al. 2018; Bedford Taylor 2017).

The pressure to specialize has grown because frontier model demand exponentially outruns single-device hardware scaling. Figure 11.3 charts this compute divergence on a logarithmic scale, highlighting the widening shaded systems gap between the exponential model demand curve (spanning from AlexNet through GPT-4) and the dashed single-device GPU performance trajectory. The normalized scenario assumes GPU supply rises about 1.7× per year while model demand rises about 6.1× per year, producing a gap that widens by roughly 3.5× each year. The measured demand trend is the historical compute curve charted in chapter 1 (figure 1.6), whose fitted doubling time runs from 3.4 to about 6 months depending on the window. The demand slope plotted here is a scenario assumption that falls inside that range, chosen to isolate the divergence rather than to forecast it (Amodei and Hernandez 2018a; Epoch AI 2024).

The divergence charted in figure 11.3 reflects a structural shift in computer architecture. Technology lifecycles trace an S-curve: slow initial ferment, exponential scaling, and eventual saturation against physical limits. General-purpose CPU scaling entered this saturation regime when Dennard scaling ended in the mid-2000s, breaking the historical trend where each transistor shrink improved performance and power simultaneously. Because single-thread frequency scaling has stalled and thermal limits constrain multi-core throughput, conventional manufacturing shrinks cannot close an order-of-magnitude systems gap. Bridging this chasm demands machines architected specifically for tensor algebra, deployed across parallel networks. In D·A·M terms, the accelerator makes the machine constraint explicit: delivered efficiency depends on how cleanly an algorithm's tensor dimensions, dataflow schedules, and communication patterns map onto the physical structures of specialized silicon.

5 **DSA (domain-specific architecture)**: A DSA that excels at dense matrix multiplication may perform worse than a CPU on irregular workloads such as graph traversal, so its gain on the target domain must justify the software, compiler, and ecosystem cost of a narrower machine (Hennessy and Patterson 2019, 2017).

↩ The measured growth of training compute and its doubling time are established in section 1.8.

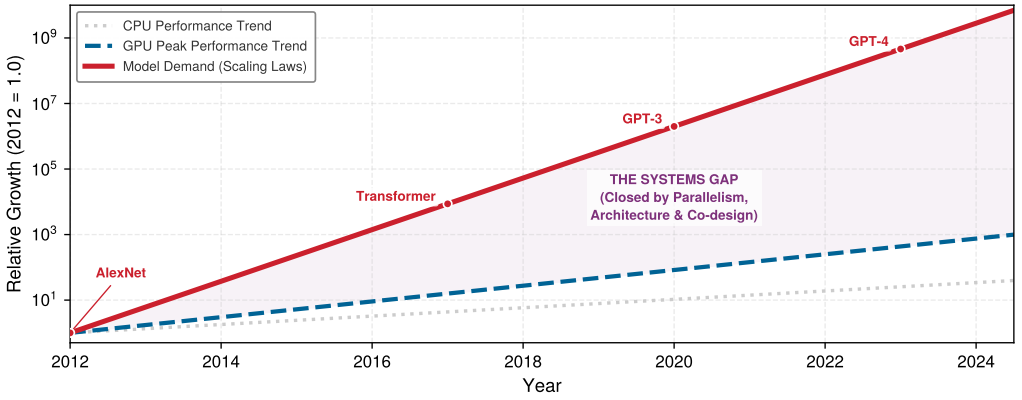
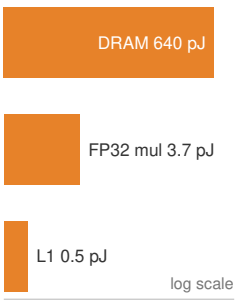


Figure 11.3: The Systems Gap: Relative compute growth (log scale) comparing model demand to hardware supply, normalized to 2012 = 1.0. The gray dotted line (CPU) and blue dashed line (GPU) reflect hardware progress, which lags the exponential red solid line (Model Demand). The purple region is the 'Systems Gap' that must be bridged through parallelism and co-design.

11.2.3 Machine learning hardware specialization

Many important neural-network workloads concentrate their work in regular tensor operations, such as dense matrix multiplications and convolutions, with substantial data parallelism, predictable reuse, and tolerance for reduced precision. Other ML operations remain irregular, bandwidth bound, or too small to fill a wide engine, so the device need not accelerate every part of a model. Hardware built to exploit the regular portion forms the class of ML accelerators, and the economic trigger for building one appears when that workload matters at fleet scale rather than only in a benchmark (example 11.2).

> Example 11.2: The TPU capacity cliff
>
> **Scenario**: Google projected in 2013 that user adoption of voice-search speech recognition (3 minutes/day/user) would require doubling global data center capacity to run CPU-based neural network inference (Jouppi et al. 2017).
>
> **Diagnosis**: General-purpose CPU server capacity was economically unable to absorb the exponential growth in inference FLOPs, threatening a capital infrastructure bottleneck.
>
> **Systems lesson**: Custom hardware acceleration becomes a candidate when workload volume makes the existing fleet uneconomical. Workload reuse, supported operations, development cost, and server total cost of ownership determine whether a custom ASIC is preferable to software changes or other hardware.



A DRAM access costs about 173× one FP32 multiply per event; the 0.5 pJ bar is a small, register-adjacent L1 access.

A general-purpose CPU is a poor host for this work. Take the chapter's reference CPU, the low end of the 1–2 TFLOP/s baseline used in the definition of hardware acceleration, and assume it sustains 5 percent–10 percent of its peak on a neural network workload. The upper endpoint then delivers approximately 100 GFLOP/s (billions of floating-point operations per second). CPUs optimize for single-thread performance and irregular memory access, whereas neural networks require massive parallelism and predictable data streams, and a single layer that touches gigabytes of parameters overwhelms cache hierarchies designed for smaller working sets.

The deeper mismatch is energy. Per event, accessing a 32-bit value from DRAM costs about 640 pJ in the 45 nm estimates of Horowitz (2014), roughly 173× the 3.7 pJ of one FP32 multiply, the same ratio table 4.1 reported for the data pipeline. The roughly 145-fold energy tax that accompanied the iron law in section 1.7 comes from the same 45 nm table; it compared one DRAM byte, a quarter of this 32-bit access, with one FP16 operation, so the two ratios describe one hierarchy counted in different units. Exact values vary by technology node and design, but a gap of two orders of magnitude, the energy-movement principle (7), makes data movement a primary optimization target

(Sze et al. 2017). Purpose-built accelerators such as TPUs sustain high utilization on dense kernels because systolic arrays and similar structures maximize data reuse while minimizing movement.

Training and inference load an accelerator differently. Training computes gradients, typically in mixed precision with lower-precision tensor operations and higher-precision accumulation (Micikevicius et al. 2017), and backpropagation retains or recomputes activations, raising memory demand. Inference requires only the forward path and may use INT8 or INT4 when the model and kernels support it. The service objective, whether tail latency, throughput, cost, or energy, then decides which accelerator fits.

↩ The activation memory that backpropagation must retain is established in section 8.4.3.1.

Definition 11.3: ML accelerator

Machine Learning Accelerators are processors or specialized units designed primarily for common neural-network tensor operations and dataflows. They pursue high R_{peak} and data reuse on matching workloads by devoting more resources to arithmetic and local data movement than a general-purpose execution path can.

1. **Significance**: An ML accelerator's defining feature is not raw arithmetic alone but a balanced path that supplies operands and retains reusable values nearby. The A100 provides 2.04 TB/s of device-memory bandwidth, about 10.2× the illustrative 200 GB/s host-DRAM path used here (NVIDIA Corporation 2020a; Choquette et al. 2021). This is a ceiling comparison, not a guarantee: sufficient arithmetic intensity, concurrency, and reuse are still required to approach its 312 TFLOP/s FP16/BF16 peak.
2. **Distinction**: The gains are conditional on operation support, parallel work, regular data access, and a software stack that selects the intended path. An ML accelerator can be orders of magnitude faster than a reference CPU on large dense matrix multiplication, yet lose much of that advantage on small, irregular, synchronization-heavy, or control-heavy workloads. Transfers and unsupported operators can dominate even when the accelerated kernel itself is fast.
3. **Common pitfall**: A frequent misconception is that ML accelerators always accelerate ML. Approaching peak throughput requires a compatible precision, operation shape, layout, implementation, and enough parallel work to keep the relevant units busy. Utilization also depends on whether memory and interconnect paths can sustain that work. A batch-1 autoregressive request may therefore use only a small fraction of a large training accelerator's arithmetic capacity even though the same device performs well on batched prefill or training.

Deployment context decides which constraint binds. In data centers it is often time-to-result for training large models. An NVIDIA H100 trades hundreds of watts of power for high throughput (Choquette 2023), and Google's TPUv4 makes a similar trade through systolic arrays and high-bandwidth memory (Jouppi et al. 2023); whether either lowers total cost depends on utilization, rental price, workload scaling, and the electricity rate.

At the edge, the priority shifts toward energy per inference and hard latency or power limits. A smartphone camera or always-on audio path inside a few-watt budget cannot adopt a data center memory system, so edge architectures reduce movement through local scratchpads, tight integration, and dynamic voltage and frequency scaling. Because the binding constraint differs by setting, no single architecture dominates, yet the memory wall governs both. Data center chips buy capacity and bandwidth with HBM, while edge chips depend on proximity and reuse.

Checkpoint 11.2: The accelerator gate

Use the energy hierarchy to decide when specialization pays.

Energy hierarchy

- ☐ **Data movement cost**: explain why a DRAM access costs about two orders of magnitude more energy than the FP32 multiply that consumes its operand?

- ☐ **Architectural response**: why would keeping and reusing operands near arithmetic units reduce movement energy? Use this prediction when the following sections develop systolic arrays and Tensor Cores.

Selection logic

- ☐ **Training vs. inference**: how do training and inference objectives affect precision, memory capacity, bandwidth, and latency requirements? Explain why both phases can need high memory bandwidth.

11.2.4 The integration bottleneck

From the 8087 onward, hardware has evolved to fit the algorithm's dominant bottleneck (Palmer 1980; Sze et al. 2017; Jouppi et al. 2017), and for dense matrix multiplications and convolutions, provisioning arithmetic units is the easy part. Feeding them is not. The integration bottleneck is the memory wall seen from inside the accelerator, the problem of transporting operands and results through the memory hierarchy quickly and efficiently enough to sustain thousands of parallel compute units. Specialized silicon delivers its performance-per-watt advantage only when the workload's demand concentrates into regular operations that align with the hardware's data paths.

Three properties of neural-network kernels make this integration bottleneck tractable. First, large matrix multiplications and convolutions expose abundant data parallelism that dense processing-element arrays can exploit concurrently. Second, static or compiled tensor graphs exhibit predictable dataflow, enabling compilers to tile and stage transfers into local scratchpads[6] rather than relying exclusively on hardware-managed caches. Third, many tensor operations tolerate reduced precision, allowing supported 8-bit or 4-bit paths to increase compute density and reduce bytes per value (Dally et al. 2021; Dally 2023). Workloads dominated by dynamic control flow, large embedding tables, small tensors, variable shapes, or strict precision requirements violate these assumptions and remain bound by memory latency and irregular access.

Once arithmetic capacity is abundant, the central engineering challenge shifts to staging data close to compute. Given the 173× energy penalty of off-chip DRAM transfers, accelerator architects deliberately allocate silicon area and thermal budgets to stacked HBM, wide on-chip SRAM, and software-managed scratchpads rather than packing in additional floating-point logic.[7]

Hardware acceleration resolves the integration bottleneck through specialized spatial layout and tiered storage hierarchies. The architectural blueprint in figure 11.4 diagrams how external HBM feeds shared on-chip storage (L2 cache), which distributes operands into rows of processing elements equipped with private scratchpads, matrix units, and vector pipelines.

Within the processing element architecture detailed in figure 11.4, specialized units partition execution across distinct mathematical primitives: dedicated matrix units (Tensor Cores) execute dense matrix multiplication, vector units evaluate element-wise arithmetic, and special function units (SFUs) approximate transcendental operations such as activations and exponentials. Their number, grouping, and names vary by vendor; some designs arrange them into an explicit 2D systolic grid, and others distribute matrix engines among wide parallel cores.

The memory hierarchy amortizes the cost of data transport across tiers. High-bandwidth memory provides off-chip capacity and throughput, while shared on-chip storage and per-element caches or scratchpads cut off-chip traffic. Because moving a tensor tile across the package boundary costs orders of magnitude more energy than keeping it on chip, compilers and kernels must coordinate tile capacity, reuse, and prefetching to keep operands staged locally; more HBM bandwidth raises the off-chip ceiling, but without local reuse the execution units still stall. The machine foundations appendix collects reference specifications for modern accelerators, including H100 and TPU v5, and summarizes the latency hierarchy.

The host interface links the accelerator to the host CPU, which coordinates execution graphs, asynchronous I/O, and fallback operations while the accelerator executes dispatched kernels. Direct-memory-access engines can overlap host-device transfers with execution, but an operator unsupported by the accelerator backend may require a CPU round-trip. An operator outside the matrix-unit instruction set may instead use vector or other device kernels. These execution boundaries help

6 **Scratchpad memory**: When a compiler can determine a tensor kernel's access pattern, it can explicitly stage tiles in fast, software-controlled local memory. This avoids some tag, replacement, and coherence machinery, but transfers, synchronization, banking, and capacity must be scheduled correctly. Scratchpads complement rather than universally replace caches because irregular or shared data may still benefit from hardware management. Google's TPU v1, for example, used a 24 MB software-managed Unified Buffer for intermediate activations, while weights and instructions used other paths (Jouppi et al. 2017).

7 **HBM**: The A100 and H100 generations provide 2.0–3.4 TB/s of device-memory bandwidth through stacked memory and wide interfaces, compared with 760 GB/s for the GDDR6X reference used here (NVIDIA Corporation 2020a; Choquette 2023). Raising the bandwidth ceiling can move some kernels toward the compute-bound side of the roofline, but only if their access pattern and arithmetic intensity can use it. HBM also increases packaging and system cost, so its value depends on the workload's bandwidth demand.

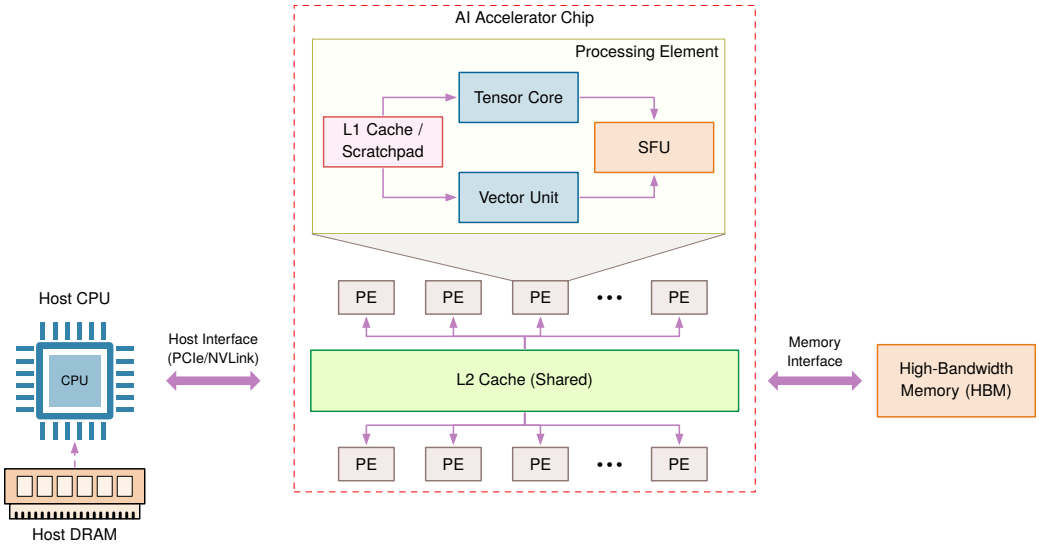


Figure 11.4: Anatomy of a Modern AI Accelerator: This generic schematic combines common components rather than depicting one product. A host interface connects the accelerator to a CPU, a high-bandwidth device-memory interface feeds shared on-chip storage, and processing elements contain matrix, vector, and special-function units with local memory. Real accelerators differ in which units they provide and whether storage is hardware-managed cache or software-managed scratchpad.

set the effective speedup envelope. Kernel execution time matters little if transfers, launches, and synchronization dominate end-to-end latency.

The specialization evident in processing elements, vector pipelines, and tiered scratchpads reflects the computational profile of neural networks: models repeatedly invoke a compact set of linear-algebraic and element-wise primitives. Within the D·A·M framework, an algorithmic change produces a wall-clock speedup only when its computational structure and memory access pattern map onto these primitives; one that cuts theoretical operations without aligning with the hardware's data paths leaves compute units starved.

11.3 AI Compute Primitives

Across fully connected, convolutional, and attention layers, learned linear transformations reduce to repeated MAC operations. A large layer can issue millions or billions of MACs, and its regular tensor structure exposes parallel work and reuse, making it a natural specialization target. MACs do not dominate every model or phase—normalization, routing, indexing, communication, and element-wise operations can become limiting—but they account for much of the peak arithmetic capacity in modern ML accelerators. Whether they dominate elapsed time still depends on tensor shape, data movement, and the surrounding operators.

The hardware units that exploit these patterns are **AI compute primitives**: specialized functional blocks, each optimized for a particular class of operation. Three primitives are especially common in accelerators, each targeting a distinct computational pattern found in neural networks.

At the framework level, a single declarative statement encapsulates thousands of arithmetic operations (listing 11.1).

Listing 11.1: Dense Layer Abstraction: High-level framework APIs encapsulate 131,072 MACs (256 inputs times 512 outputs) in a single function call, hiding the computational complexity from developers while enabling automatic hardware optimization.

```
# Framework abstracts compute-intensive operations
dense = Dense(512)(input_tensor)  # 256x512 MACs per sample
```

Beneath this API call, an autograd or compilation engine lowers the layer into a computational graph. Listing 11.2 exposes a bulk matrix multiplication and bias broadcast, followed by an optional element-wise activation; the activation is a separate stage rather than an implicit part of the shown `Dense` call.

Listing 11.2: Matrix Operation Expansion: Each dense layer decomposes into matrix multiplication and element-wise operations, exposing the dominant compute pattern that many neural-network kernels are built around.

```
# Linear transformation work scales with input_dim x output_dim x
# batch.
output = (
    matmul(input, weights) + bias
)  # Matrix multiply dominates cost
output = activation(
    output
)  # Element-wise: proportional to output_dim x batch
```

While graph representations group these steps into coarse tensor operators, execution on physical hardware requires scheduling individual scalar operations. Lowering the matrix multiplication to loop nests (listing 11.3) makes the execution structure explicit: three nested iterations traverse the batch, output feature, and reduction dimensions, establishing $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ arithmetic complexity.

Listing 11.3: Processor-Level Execution: Nested loops reveal the $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ multiply-accumulate operations that accelerators must execute, with 4.2M MACs for B=32, d_{in}=256, d_{out}=512 configurations.

```
# Total operations: batch_size × output_size × input_size MACs
for n in range(batch_size):  # Batch dimension: parallelizable
    for m in range(output_size):  # Output neurons: parallelizable
        sum = bias[m]  # Initialize accumulator
        for k in range(input_size):  # Reduction dimension: sequential
            sum += input[n, k] * weights[k, m]  # MAC operation
        output[n, m] = activation(sum)  # Nonlinear transformation
# Example work scales as batch_size × output_size ×
# input_size multiply-accumulate operations
```

These nested loops expose three recurring computational patterns: element-wise operations along vectors, matrix-level multiply-accumulate reductions, and nonlinear transformations. Their ubiquity justifies dedicated hardware execution paths: vector pipelines stream independent elements and reductions, matrix engines organize multiply-accumulate grids across spatial tiles, and special-function units approximate nonlinear math. The boundaries between these units are architectural rather than mathematical; an optimizing compiler frequently fuses operations across all three paths within a single kernel invocation to eliminate intermediate memory traffic.

11.3.1 Vector operations

Vector architectures amortize instruction overhead by executing a single operation across multiple packed data elements concurrently. In the scalar formulation of listing 11.3, evaluating a batch of 32 samples through a 256-to-512 dense layer demands 4.2M MACs. A scalar processor issues independent instructions to load each input-weight pair, perform the multiply-accumulate, and commit the result, incurring instruction-fetch, decode, and branch-prediction overhead on every

scalar operation. Vector pipelines replace this per-element control overhead with wide register files (typically 128 to 512 bits) and parallel execution lanes that apply a single decoded instruction across all packed elements simultaneously.

RISC-V,[8] an open instruction-set architecture (ISA) with a standardized vector extension (Waterman et al. 2013), provides a concrete vehicle for this execution model. The contiguous loads in listing 11.4 assume that the selected weight column has been packed into a contiguous vector; the row-major `weights[k, m]` access in the scalar example is otherwise strided. In RISC-V vector assembly, code is decoupled from fixed hardware widths: the instruction `vsetvli` queries the hardware to determine how many elements fit into available registers, processes that chunk, and loops until all features are exhausted.

[8] **RISC-V (reduced instruction set computer V)**: The open ISA allows hardware teams to add custom ML instructions, including vector dot products, activation functions, and sparse tensor operations, without changing the base ISA. The trade-off is software ecosystem maturity: custom extensions require corresponding compiler, library, and runtime support, which can limit portability and increase integration work.

Listing 11.4: Vectorized Dot-Product Loop: This illustrative loop processes several element pairs per iteration and reduces them into one scalar dot product.

```
fmv.w.x fa0, zero          # Scalar dot-product accumulator
fmv.w.x ft1, zero          # Zero seed for vector reduction
loop_feature:
    vsetvli t0, feature_cnt, e32, m1, ta, ma
    vle32.v v1, (in_ptr)
    vle32.v v2, (wt_ptr)
    vfmul.vv v3, v1, v2
    vfmv.v.f v0, ft1
    vfredusum.vs v4, v3, v0
    vfmv.f.s ft0, v4
    fadd.s fa0, fa0, ft0
    slli t1, t0, 2         # Four bytes per FP32 element
    add in_ptr, in_ptr, t1
    add wt_ptr, wt_ptr, t1
    sub feature_cnt, feature_cnt, t0
    bnez feature_cnt, loop_feature
```

The vectorized reduction loop in listing 11.4 progresses through five hardware-visible stages:

1. **Vector length configuration**: `vsetvli` configures the vector units for 32-bit elements (`e32`), dynamically computing how many elements (`t0`) fit into the hardware vector registers (`VLEN`).
2. **Vector initialization**: Clears the scalar accumulator that will hold the accumulated dot product.
3. **Vector loads**: `vle32.v` loads contiguous 32-bit elements into vector registers `v1` and `v2`, dispatching wide memory requests across the memory interface.
4. **Vector multiply and reduction**: `vfmul.vv` multiplies element pairs in parallel across vector lanes, and `vfredusum.vs` reduces the products into a partial scalar sum.
5. **Pointer arithmetic and loop branch**: Advances base addresses, decrements the remaining element count by the processed chunk size `t0`, and branches if elements remain.

Across iterations, the vector multiply instruction processes multiple data elements concurrently while wide loads amortize instruction decode and memory-request overhead. Because the runtime vector length dynamically adapts to the implementation's hardware width (`VLEN`), the identical binary executes correctly on both narrow embedded vector units and wide data-center pipelines. For this 256-feature workload, an 8-lane implementation breaks the reduction into approximately 524,288 vector chunks across the batch.

Beyond dense inner products, deep learning workloads rely on a broader set of vector instructions. Table 11.1 maps these primitive vector operations to the specific neural network layers that depend on them.

Table 11.1: Vector Operations: Core vector operations map directly to deep learning primitives: reductions implement pooling layers, gathers enable embedding lookups, scatters update embedding gradients, and masked operations handle attention masks. These mappings show how vector hardware supports common ML primitives.

Vector operation	Description	Neural network application
Reduction	Combines elements across a vector (for example, sum, max)	Pooling layers, attention score computation
Gather	Loads multiple nonconsecutive memory elements	Embedding lookups, sparse operations
Scatter	Writes to multiple nonconsecutive memory locations	Gradient updates for embeddings
Masked operations	Selectively operates on vector elements	Attention masks, padding handling
Vector-scalar broadcast	Applies scalar to all vector elements	Bias addition, scaling operations

The benefit extends beyond instruction count. Contiguous vector loads can use memory interfaces efficiently, and control overhead is amortized across several data elements. Gather, scatter, masking, and short tails may use fewer lanes or require extra memory transactions, so vector width alone does not guarantee proportional speedup. The architectural pattern is not new. The Cray-1[9] used vector registers and pipelined functional units for scientific computing in the 1970s (Jordan 1982); modern ML processors apply related principles at much larger commercial scale.

Vector operations excel at element-wise transformations such as activation functions, where each output depends only on *its corresponding input*. Neural networks, however, also require structured computations where each output depends on *all inputs*—the weighted sums that define layer transformations. These many-to-many operations naturally express themselves as matrix multiplications, the second compute primitive.

[9] **Cray-1 vector legacy**: The Cray-1 (1975) used 64-element vector registers with pipelined functional units, allowing a stream of elements to advance through arithmetic operations without issuing one scalar instruction per element. Modern accelerators extend the same principles of wide registers, pipelined execution, and data reuse to vector and matrix tiles.

11.3.2 Matrix operations

Matrix multiplication accounts for much of the arithmetic in many dense neural networks, transforming high-dimensional data through structured patterns of weights, activations, and gradients (Goodfellow et al. 2016). While vector operations process elements independently or reduce them, matrix operations organize work across multiple dimensions. Hardware and libraries tile those dimensions so blocks of weights and activations can be reused while partial sums accumulate locally. This regular structure drives important hardware strategies, but small or irregular matrices may leave tile capacity unused.

11.3.2.1 Matrix operations in neural networks

Neural network computations decompose into hierarchical matrix operations. Listing 11.5 captures this hierarchy through a linear layer that transforms input features into output neurons over a batch. Its expanded mathematics uses a column-batch convention and adds an optional ReLU; this illustrates the arithmetic rather than the framework API's internal tensor layout or an activation implicit in `nn.Linear`.

Listing 11.5: Linear-Layer Operation Hierarchy: The example expands a linear layer into a matrix multiply and bias broadcast, then applies element-wise ReLU, separating the matrix and vector work that hardware must execute.

```
layer = nn.Linear(256, 512)  # Layer transforms 256 inputs to 512 outputs
output = layer(input_batch)  # Process a batch of 32 samples

# Framework Internal: Core operations (column-batch convention)
Z = matmul(weights, input)  # Matrix: transforms [256×32]
# input to [512×32] output
Z = Z + bias  # Vector: adds bias to each
# output independently
output = relu(Z)  # Vector: applies activation to
# each element independently
```

This computation demonstrates the scale of matrix operations in neural networks. Each output neuron (512 total) must process all input features (256 total) for every sample in the batch (32 samples). The weight matrix alone contains $256 \times 512 = 131{,}072$ parameters that define these transformations, illustrating why efficient matrix multiplication dominates performance considerations.

Neural networks employ matrix operations across architectural patterns beyond simple linear layers. Convolution operations transform into matrix multiplications through the im2col technique,[10] enabling efficient execution on matrix-optimized hardware. Listing 11.6 illustrates these applications.

Listing 11.6: Matrix Patterns Across Architectures: Linear layers, attention mechanisms, and convolutions all reduce key work to matrix multiplications, making matrix hardware the shared primitive across modern neural architectures.

```
hidden = matmul(weights, inputs)
# weights: [out_dim x in_dim], inputs: [in_dim x tokens]
# Result combines all inputs for each output

# Attention Mechanisms - Multiple matrix operations
Q = matmul(Wq, inputs)
# Project inputs to query space [query_dim x tokens]
K = matmul(Wk, inputs)
# Project inputs to key space [key_dim x tokens]
attention = matmul(Q.T, K)
# Compare all query tokens with all key tokens [tokens x tokens]

# Convolutions - Matrix multiply after reshaping
patches = im2col(input)
# Convert [H x W x C] image to matrix of patches
output = matmul(kernel, patches)
# Apply kernels to all patches simultaneously
```

[10] **Im2col (image-to-column)**: The lowering diagrammed in the im2col transformation of chapter 5 (figure 6.5) turns convolution into a matrix multiplication by unfolding each receptive-field patch into one row or column of a dense matrix. An explicit materialization can duplicate overlapping values and expand storage by up to roughly the kernel area before edge, stride, and channel effects are considered. Production libraries often use implicit-GEMM or direct-convolution kernels to obtain matrix-friendly execution without materializing the full expanded matrix.

These examples differ in tensor shape and reuse pattern, which determines how efficiently each multiplication occupies a matrix unit. Linear layers reuse one weight matrix across a batch; attention forms projections and then a token-to-token score matrix; convolution reuses each kernel across many image patches. Expressing all three as matrix multiplication exposes a common interface to hardware, but it does not make their execution costs identical. Batch size, sequence length, channel count, and tiling determine whether operands are reused locally or repeatedly fetched from memory.

11.3.2.2 Matrix operations hardware acceleration

This pervasive pattern of matrix multiplication has direct implications for hardware design: accelerators need specialized units that can handle these computations at scale. Listing 11.7 demonstrates a representative dedicated matrix unit that processes an entire 16×16 block at once, illustrating why matrix instructions and Tensor Cores can deliver much higher throughput than scalar or vector-only execution paths (NVIDIA 2017; Intel Corporation 2021a).

Listing 11.7: Illustrative Matrix-Unit Operation: Representative load, matrix-multiply-accumulate, and store instructions operate on matrix blocks rather than individual scalar elements.

```
mload mr1, (weight_ptr)     # Load e.g., 16x16 block of
                            # weight matrix
mload mr2, (input_ptr)      # Load corresponding input block
matmul.mm mr3, mr1, mr2     # Multiply and accumulate entire
                            # blocks at once
mstore (output_ptr), mr3    # Store computed output block
```

The illustrative unit operates on matrix tiles rather than individual elements. A full $16\times16\times16$ tile product contains $16^3 = 4{,}096$ multiply-accumulate operations, but the cycles required depend on the actual instruction and architecture. Sustained throughput also depends on the number of matrix units, operand precision, issue rate, pipeline behavior, and whether memory can keep the units fed. Matrix units complement vector execution by accelerating structured many-to-many transformations.

Much like vector processing, matrix acceleration has deep historical roots; digital signal processors (DSPs) and graphics processors optimized matrix computations in the 1980s–1990s for image processing, scientific computing, and 3D rendering (Owens et al. 2008; Hwu 2011). Neural networks have made matrix multiplication commercially dominant, driving the integration of two-dimensional systolic arrays and matrix execution engines (such as NVIDIA Tensor Cores and Google TPUs) directly into processor datapaths.

Linear algebra constitutes only part of the neural network execution pipeline. Between linear transformations, neural networks introduce nonlinear activation functions and normalization layers. While piecewise-linear functions such as ReLU map to simple vector comparison instructions, transcendental computations (such as exponentials in softmax and GELU, or reciprocal square roots in normalization) cannot be computed directly on multiply-accumulate datapaths without incurring severe cycle penalties. Table 11.2 summarizes the execution characteristics of the three primary compute primitives and their mapping to neural network operations.

Table 11.2: Operation Characteristics: Matrix operations excel at many-to-many transformations, while vector units handle element-wise operations and reductions such as activation and normalization. Special-function paths handle transcendental arithmetic through architecture-specific approximation or dedicated logic. The operation structure determines which hardware primitive is appropriate.

Operation type	Best For	Examples	**Key characteristic**
Matrix Operations	Many-to-many transforms	Layer transformations, attention, convolutions	Each output depends on multiple inputs
Vector Operations	Vector and reduction work	Activation functions, layer normalization, element-wise gradients	Uses element-wise operations and vector reductions
Special-Function Paths	Transcendental arithmetic	Exponential, logarithm, reciprocal square root	Uses architecture-specific approximation or dedicated logic

11.3.3 Special function units

Special Function Units (SFUs) or specialized instruction paths accelerate nonlinear functions and related arithmetic, completing the chapter's trio of processing primitives. The need is not new: floating-point coprocessors addressed scalar arithmetic bottlenecks (Palmer 1980), and digital signal processors added specialized arithmetic for signal-processing workloads (Smith 1997). An implementation may use a standalone unit, vector approximation instructions, or a library sequence, trading accuracy, latency, and throughput. In neural networks, activation, normalization, and softmax operations can become important between matrix kernels, particularly when repeated memory passes or transcendental functions limit throughput.

11.3.3.1 Nonlinear functions

A common layer sequence illustrates how non-matrix operators enter the execution pipeline (Goodfellow et al. 2016). Listing 11.8 chains a linear transformation with ReLU activation and batch normalization—operations that appear trivial in Python framework code but impose distinct arithmetic and memory constraints on hardware.

Listing 11.8: Linear–Nonlinear Layer Sequence: A linear layer, ReLU, and BatchNorm combine matrix, element-wise, and normalization work in one common layer sequence.

```
layer = nn.Sequential(
    nn.Linear(256, 512), nn.ReLU(), nn.BatchNorm1d(512)
)
output = layer(input_tensor)
```

Listing 11.9 expands this sequence into its arithmetic primitives, isolating the element-wise thresholding of ReLU from the cross-batch reductions and reciprocal square root required by batch normalization. Here rows are samples and columns are features; `weights` has input features by output features, while `bias`, `gamma`, and `beta` contain one value per output feature. Keeping the reduced batch dimension permits those per-feature statistics to broadcast across samples.

Listing 11.9: Nonlinear Operation Expansion: Expanding the layer sequence exposes ReLU comparison, BatchNorm reductions, variance computation, and square-root normalization around the matrix multiply.

```
Z = matmul(input, weights) + bias  # Linear transformation
H = max(0, Z)  # ReLU activation
mean = reduce_mean(H, axis=0, keepdims=True)  # BatchNorm statistics
var = reduce_mean((H - mean) ** 2, axis=0, keepdims=True)  # Variance computation
output = gamma * (H - mean) / sqrt(var + eps) + beta  # Normalization
```

11.3.3.2 Hardware implementation of nonlinear functions

Translating these mathematical functions to silicon exposes sharp differences in memory traffic, instruction count, and pipeline utilization. For instance, batch normalization (Ioffe and Szegedy 2015) demands multi-pass statistics gathering across memory, variance accumulation, and a reciprocal square root, while softmax requires computing exponentials across an entire normalization dimension. While a rectified linear unit (ReLU) is mathematically simple, a naive scalar implementation incurs branch overhead or redundant memory round-trips unless fused into adjacent kernels. Listing 11.10 contrasts the memory and dependency overheads of pointwise activation against multi-pass normalization in an unfused loop model.

The label *nonlinear* hides three different execution contracts. Pointwise functions such as ReLU leave elements independent and therefore map naturally to vector lanes or parallel threads. Reductions such as the mean and variance in batch normalization introduce cross-element dependencies: partial results must be combined before later stages can proceed. Transcendental functions such as exponential, logarithm, and reciprocal square root add a numerical contract because their implementations trade latency, precision, and valid input range. The dependency pattern, not the mathematical label alone, determines whether vector arithmetic, a reduction tree, or a specialized approximation path is appropriate. A faster special-function path accelerates only the operation it implements; it does not remove surrounding tensor traffic or synchronization.

Listing 11.10 is intentionally dependency-explicit rather than a model of an optimized kernel. The loops expose three state lifetimes: per-element activations, per-feature accumulators, and the final normalized outputs. The normalization parameters cannot be applied until the required statistics are available, so implementations must either retain intermediate activations or recompute them. Training derives those statistics from the current batch, whereas inference can use stored statistics and reduce normalization to an element-wise affine transform. Compilers may fuse legal stages, but the data dependencies determine which transformations preserve the computation. Reading the listing as a dependency graph therefore reveals the durable hardware questions: how often each tensor crosses a memory interface, which intermediates remain local, and where reductions impose synchronization.

Listing 11.10: ReLU and BatchNorm Operations: Neural networks process input data through element-wise selections and multiple normalization passes, highlighting efficiency challenges in naive implementations.

```
for batch in range(32):
    for feature in range(512):
        # ReLU: Naive scalar compare/select; optimized kernels
        # usually implement this branchlessly.
        z = matmul_output[batch, feature]
        h = max(0.0, z)  # Conditional operation

        # BatchNorm: Multiple passes over data
        mean_sum[feature] += h  # First pass for mean
        var_sum[feature] += h * h  # Additional pass for variance

        temp[batch, feature] = h  # Extra memory storage needed


# Normalization requires complex arithmetic
for feature in range(512):
    mean = mean_sum[feature] / batch_size
    var = (var_sum[feature] / batch_size) - mean * mean

    # Square root computation: Multiple iterations
    scale = gamma[feature] / sqrt(var + eps)  # Iterative
    # approximation
    shift = beta[feature] - mean * scale

    # Additional pass over data for final computation
    for batch in range(32):
        output[batch, feature] = temp[batch, feature] * scale + shift
```

The dependency structure in listing 11.10 exposes two costs of naive execution. The pointwise ReLU and normalization scale-and-shift perform little arithmetic per accessed byte, so materializing each pass can place a low bandwidth ceiling on throughput. In this unfused loop model, `temp` is stored and reread after batch statistics are finalized; its physical traffic depends on cache residency and the chosen memory interface. Computing $1/\sqrt{\text{var}+\epsilon}$ also adds a dependent arithmetic path. The listing illustrates an iterative approximation, but actual kernels may use a reciprocal-square-root instruction and refinement, with device- and accuracy-dependent latency. Specialized function paths address this arithmetic cost, while legal fusion and local buffering reduce intermediate traffic. Neither removes the need to complete the reductions before normalization.

11.3.3.3 SFU hardware implementation

SFUs address arithmetic bottlenecks by implementing nonlinear and transcendental operations in specialized functional units rather than multi-instruction software routines. Accelerators deploy dedicated datapath circuits—such as quadratic polynomial interpolators and small lookup tables (LUTs)—that evaluate functions like exponential, reciprocal square root, and logarithm within a few pipeline stages. Listing 11.11 illustrates how an instruction set exposes these operations alongside vector memory accesses.

Listing 11.11: Illustrative SFU Operations: Pseudocode shows vector nonlinear operations that may map to specialized execution paths; instruction names and latency are implementation dependent.

```
vld.v v1, (input_ptr)    # Load vector of values (pseudocode)
vrelu.v v2, v1           # Vector ReLU path
vsigm.v v3, v1           # Vector sigmoid path
vtanh.v v4, v1           # Vector tanh path
vrsqrt.v v5, v1          # Vector reciprocal-square-root path
```

Specialized function paths balance circuit area against mathematical precision. While piecewise linear activations such as ReLU require only single-cycle compare-and-select (maximum) datapath

logic, transcendental functions trade off lookup table size and polynomial interpolation degrees against approximation error. Table 11.3 summarizes representative hardware mechanisms and their qualitative latency characteristics across these primitives.

Table 11.3: Special Function Units: Dedicated paths reduce instruction overhead for nonlinear and reduction primitives. Implementations and latency are architecture-dependent; the table describes mechanisms rather than universal cycle counts.

Function unit	Operation	Implementation strategy	Illustrative latency
Activation unit	ReLU	Compare-and-select or maximum	Low; architecture-dependent
Statistics unit	Mean, variance	Parallel reduction trees	Grows with reduction depth
Exponential unit	exp, log, sigmoid, tanh	Approximation, table lookup, and interpolation	Architecture-dependent
Root/power unit	sqrt, rsqrt	Iterative or approximation hardware	Architecture-dependent

Vector operations, matrix operations, and special-function paths establish the foundational arithmetic primitives of ML accelerators, but functional units alone do not dictate sustained throughput. These primitives govern *what* the silicon computes; the execution model and memory hierarchy govern *how* data feeds the datapath. A matrix unit rated for hundreds of teraFLOP/s will idle if memory bandwidth cannot sustain operand delivery, if tile dimensions mismatch register blocking, or if thread synchronization serializes execution. Peak hardware ratings describe performance under idealized operand availability; the architecture's execution model dictates how much of that peak is achievable on real neural network workloads.

11.4 Compute Units and Execution Models

Applying ReLU to a 512-element vector shows why execution models matter: the operation is simple, but throughput depends on whether the hardware treats those 512 comparisons as scalar instructions, as lanes of a single instruction, multiple data (SIMD) unit, as GPU threads, or as tensor-program fragments. Modern AI processors package the three compute primitives into distinct execution units (vector units for vector work, matrix units such as Tensor Cores and systolic arrays for matrix work, and special function units for nonlinear functions) and then group those units into processing elements. That organization defines how computations are structured and exposed to programmers, and it explains why theoretical capability and delivered throughput diverge.

11.4.1 Mapping primitives to execution units

Mapping computational primitives to execution units is governed by physical wire energy and data reuse. For the reference low-precision operations, reading an operand from a register file and moving it across on-chip wires can cost more energy than the arithmetic itself; workload energy also depends on how often each event occurs. Hardware accelerators manage this physical constraint by provisioning specialized execution units matched to the reuse characteristics of each primitive:

- Vector operations → SIMD and single instruction, multiple threads (SIMT) units. These execute 1D elementwise primitives—such as additions, scaling, and bias offsets—across parallel ALUs that share instruction fetch and decode logic. Because vector operations exhibit $\mathcal{O}(1)$ arithmetic reuse per operand loaded, their throughput remains bounded by register file port bandwidth and memory delivery.
- Matrix operations → Tensor Cores[11] and systolic arrays. These map 2D inner-product primitives to dense two-dimensional grids of MAC circuits. By routing activations and weights directly between adjacent processing elements through local wires—such as in weight-stationary or output-stationary dataflows—matrix units reuse data in flight. This spatial reuse decouples arithmetic throughput from main register file bandwidth, sustaining peak compute density within a manageable thermal envelope.
- Special functions → Special Function Units (SFUs). These map transcendental and non-linear primitives—such as exp, ln, and reciprocal square roots—to dedicated pipelines using piecewise polynomial approximations and lookup tables. Evaluating transcendentals on general-purpose ALUs requires iterative numerical algorithms that consume dozens of clock cycles;

[11] **Reduced-precision ML**: Halving operand width halves the bytes per stored element and can support denser arithmetic, although realized throughput depends on the available datapath and kernel (Dally et al. 2021; Dally 2023). NVIDIA's transition from Pascal P100 vector arithmetic to mixed-precision Tensor Cores in Volta V100 increased advertised peak FP16 throughput from about 21.2 to 125 TFLOP/s, roughly 6× across those products (NVIDIA 2017). Precision selection must preserve numerical behavior and use an efficient supported path; the smallest format is not automatically the best one.

dedicated SFUs evaluate these functions off the primary execution path at lower throughput, preventing activation layers from stalling vector pipelines.

In the D·A·M taxonomy, this partitioning defines how algorithmic primitives bind to physical machine execution paths. Routing dense matrix multiplications through standard vector pipelines rapidly saturates register file read ports, collapsing throughput to a small fraction of the accelerator's peak capacity. Conversely, routing low-arithmetic-intensity elementwise operations or irregular activations through matrix arrays leaves processing lanes idle. Sustained system performance depends on matching each computational primitive to the execution unit whose dataflow and storage hierarchy match its arithmetic intensity.

11.4.2 Evolution from SIMD to SIMT architectures

In scalar processor architectures, executing an elementwise operation across a 512-element activation vector requires 512 individual loop iterations. Each iteration fetches, decodes, and tracks an instruction through the pipeline, consuming substantially more energy in instruction control logic than the arithmetic logic unit (ALU) expends performing the actual comparison or addition. Under fixed thermal design power (TDP) budgets, scalar execution spends silicon area and power on instruction management rather than arithmetic throughput. Flynn's SIMD execution model amortizes this front-end overhead by issuing a single decoded instruction across multiple data elements packed into wide vector registers (Flynn 1966). A 512-bit SIMD register, for example, processes sixteen 32-bit floating-point values simultaneously in a single clock cycle, reducing instruction fetch and decode traffic sixteenfold.

Traditional fixed-width SIMD architectures bind compiled machine code to specific hardware register widths, such as 128, 256, or 512 bits. When a tensor dimension fails to align with the physical vector width, compilers must emit scalar cleanup loops or insert memory padding, increasing binary footprint and control overhead. Following the vector-length-agnostic principle introduced with RISC-V in section 11.3.1, the Arm Scalable Vector Extension (SVE) resolves this rigidity by using per-lane predication and scalable registers to execute identical binary code across hardware implementations spanning 128 to 2048 bits without recompilation (Stephens et al. 2017). Intel's Advanced Matrix Extensions (AMX) extend this CPU-side specialization to two dimensions, introducing dedicated 2D tile registers and matrix-multiplication accelerators that operate directly on matrix blocks rather than one-dimensional vectors (Intel Corporation 2021a). Together, SVE and AMX illustrate how general-purpose CPU architectures incorporate length-agnostic vector execution and fixed-tile matrix datapaths to service machine learning workloads.

Despite vector-length agility and matrix tile extensions, CPU architectures encounter fundamental physical limitations when scaling to large neural network workloads. CPUs dedicate substantial silicon area to branch predictors, out-of-order execution logic, and deep cache hierarchies to minimize latency for a few sequential threads. When streaming through gigabytes of tensor data, cache misses force the execution pipeline to stall for hundreds of cycles waiting on main memory. The CPU's instruction reorder buffer fills rapidly during these memory stalls, leaving wide SIMD pipelines starved of operands. Furthermore, conditional branching within a vector requires evaluating divergent paths under bitmasks, degrading useful lane utilization.

Single Instruction, Multiple Threads (**SIMT**) architectures resolve these bottlenecks by decoupling the logical programming abstraction from physical hardware execution (Lindholm et al. 2008; Nickolls et al. 2008). Instead of requiring the programmer to pack data into explicit vector registers, SIMT exposes an execution model of independent scalar threads. In GPU architectures, each Streaming Multiprocessor (SM)[12] coordinates thousands of concurrent threads. Hardware warp schedulers group these threads into warps, the 32-thread scheduling groups (wavefronts in AMD architectures) whose granularity already set lane utilization in training kernels, and every thread in a warp executes a common instruction in lockstep across parallel ALUs. This lockstep execution introduces a critical constraint known as **warp divergence**. When threads within a single warp take different paths through a conditional branch, the hardware must execute both required paths sequentially, masking off the inactive lanes during each path. This serialization reduces useful lane utilization and increases execution time, which is why performance-critical ML kernels avoid divergent branching in favor of uniform or predicated control flow.

12 **SM (streaming multiprocessor)**: The physical hardware engine that implements the SIMT model by using warp schedulers to coordinate many parallel threads. Maintaining enough active warps can help hide instruction and memory latency, but occupancy alone does not identify the bottleneck. Low occupancy may result from register use, shared-memory use, block dimensions, or insufficient parallel work, while a memory-bound kernel can still have high occupancy.

↩ Warps as the 32-thread unit in which GPUs schedule tensor kernels are established in section 8.5.3.1.

Rather than relying on large caches to reduce memory latency, GPUs hide latency through massive thread-level concurrency. An SM statically partitions an on-chip register file across all active threads, eliminating context-saving overhead; when one warp stalls waiting hundreds of cycles for an off-chip memory fetch, the warp scheduler selects and issues instructions from another ready warp on the very next cycle. The baseline matrix multiplication kernel in listing 11.12 illustrates how this scalar thread model maps onto hardware. CUDA organizes a grid of threads into blocks: `blockIdx` locates a block in the grid, `blockDim` gives the block dimensions, and `threadIdx` locates a thread within that block. Multiplying a block coordinate by its dimension and adding the within-block coordinate gives the global row or column used by the listing.

Listing 11.12: SIMT Execution: Each thread processes a unique output element in parallel, demonstrating how SIMT enables efficient matrix multiplication on GPUs.

```
__global__ void matrix_multiply(float* C, float* A, float*
                                B, int N) {  // CUDA kernel
    // Each thread processes one output element
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for (int k = 0; k < N; k++) {
        // Threads in a warp execute in parallel
        sum += A[row * N + k] * B[k * N + col];
    }
    C[row * N + col] = sum;
}
```

In listing 11.12, the CUDA[13] kernel assigns one output element `C[row * N + col]` to each thread. Adjacent threads in a warp share the same row index while having consecutive column indices (`threadIdx.x`). When threads fetch `B[k * N + col]`, their memory requests target contiguous 4-byte floating-point addresses. The memory controller merges these 32 individual 4-byte loads into a single 128-byte coalesced memory transaction, maximizing DRAM bus efficiency. Concurrently, all threads in the warp read the identical element `A[row * N + k]`, allowing the hardware to broadcast that single fetched value across all 32 lanes. However, this naive kernel runs into the arithmetic intensity law (principle 6). Because each inner-loop iteration performs only two floating-point operations while reloading operands from global memory without on-chip SRAM staging, the kernel remains severely memory bound. Furthermore, executing dense matrix multiplication on scalar SIMT ALUs requires one instruction fetch, decode, and issue cycle for every multiply-accumulate, saturating register file read ports. Overcoming this machine bottleneck requires moving beyond vector-of-scalars execution to specialized two-dimensional matrix units.

11.4.3 Tensor Cores

While SIMD and SIMT units execute vector work efficiently, large matrix computations benefit from units organized around multidimensional tiles. Dedicated matrix engines amortize operand loads across many multiply-accumulate operations by reusing tiles on chip. NVIDIA GPUs expose Tensor Cores, the reduced-precision matrix units that mixed-precision training was built to exploit, whereas Google TPUs use matrix units built around systolic arrays. Both execute tiled matrix multiplication and accumulation.[14] The benefit depends on tile reuse, shape, precision, and memory behavior.

Consider a single transformer attention head computing the $\mathbf{Q} \times \mathbf{K}^T$ product for a 2,048-token sequence with 64-dimensional embeddings. This operation requires multiplying a 2048×64 matrix by a 64×2048 matrix: roughly 268.4M MACs, or about 536.9 MFLOP when the multiply and add are counted separately. On a scalar processor executing one FLOP per cycle at 2 GHz, this single attention head would take about 268.4 ms. GPU SIMT execution can distribute this work across many threads, but Tensor Cores go further by processing entire matrix tiles per instruction. Assume, for illustration, that the Tensor Cores of a single streaming multiprocessor complete this one head in 0.5 ms, an implied rate of about 1.1 TFLOP/s and a small slice of a full GPU's tensor peak. The tiled

[13] **CUDA (compute unified device architecture)**: Released by NVIDIA in 2006, CUDA eliminated the need to disguise general-purpose computations as graphics operations, opening GPUs to scientific and ML workloads through a C-like programming model. The ecosystem it created—cuBLAS, cuDNN, TensorRT—constitutes a software moat that can lock the ML training stack to NVIDIA hardware: migrating away requires rewriting or replacing thousands of GPU-optimized kernels, a cost that often exceeds the hardware savings of competing platforms. This software lock-in, not raw silicon performance alone, helps explain why many large ML training stacks remain CUDA-centered.

[14] **Tensor Core dimension alignment**: NVIDIA Tensor Cores are most efficient when matrix dimensions are aligned to precision- and architecture-specific multiples, such as multiples of 8 or 16 for common FP16/BF16/INT8 paths. Modern cuBLAS and cuDNN can still use Tensor Cores for many nonaligned dimensions, but poorly aligned shapes may trigger less efficient kernels, require padding, or reduce effective throughput (NVIDIA 2024a; NVIDIA Corporation 2021b). This is why model architects often choose embedding and channel dimensions such as 512 rather than 500 and why batch-size-1 inference may fail to reach peak Tensor Core utilization: alignment and arithmetic intensity jointly determine whether the hardware's matrix engines stay full.

path is then roughly 536.9× faster than scalar execution, and that speedup arises not from a higher clock frequency but from hardwiring tile-level data reuse into the execution datapath.

While high-level frameworks hide low-level matrix intrinsics, inspecting the intermediate assembly reveals how hardware coordinates spatial execution. In NVIDIA Parallel Thread Execution (PTX), Tensor Core operations are invoked through warp-synchronous matrix instructions (`mma.sync`).[15] The instruction mnemonic encodes the exact tile geometry and data types: `.m16n8k16` specifies the matrix tile dimensions ($M = 16, N = 8, K = 16$), `.row.col` defines the row-major and column-major memory layouts of inputs **A** and **B**, and the trailing types (`.f32.f16.f16.f32`) declare that FP16 inputs are multiplied and accumulated into an FP32 destination. In listing 11.13, all 32 threads in a warp execute this single instruction collectively to compute the matrix tile product $\mathbf{D} = \mathbf{AB} + \mathbf{C}$.

Listing 11.13: Tensor Core Operation: Matrix multiplications are performed in parallel across entire matrix blocks, optimizing computational efficiency for neural network training.

```
Tensor Core Operation (example GPU PTX):
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
  {d0,d1,d2,d3},      // Destination registers
  {a0,a1,a2,a3},      // Source matrix A
  {b0,b1},            // Source matrix B
  {c0,c1,c2,c3};      // Accumulator
```

In this instruction, the warp executes an $M = 16, N = 8, K = 16$ matrix multiply-accumulate operation in mixed precision: inputs **A** (16×16) and **B** (16×8) are formatted in FP16, while accumulator **C** and destination **D** (16×8) use FP32. Rather than scheduling 64 separate scalar fused multiply-add (`fma`) instructions per thread, the warp scheduler issues a single instruction that evaluates 2,048 multiply-accumulate operations across the Tensor Core datapath. Distributing matrix fragments across thread registers and accumulating intermediate sums within dedicated adder trees relieves two key hardware bottlenecks of conventional SIMT execution: instruction issue bandwidth and register file read-port contention.

Design priorities determine how matrix engines appear in different processor families. GPU Tensor Cores preserve programmability while accelerating general-purpose deep learning kernels. TPU-style designs use large-scale matrix units arranged in systolic arrays to maximize sustained training throughput on dense tensor kernels. Mobile NPUs[16] shrink the same idea into low-power inference blocks, while server CPUs add matrix instruction extensions (AMX-class tiles) for inference and mixed workloads. Each version changes the same contract: how much flexibility the hardware keeps while reducing movement around dense matrix operations.

Figure 11.5 charts advertised single-chip GPU throughput on a logarithmic scale from 2012 to 2024, isolating how successive generational leaps combine architectural scaling with lower numerical precisions and structured sparsity. From K20X FP32 (3.9 TFLOP/s) to H100 FP8 with structured sparsity (4,000 TFLOP/s) and B200 FP8 with structured sparsity (9,000 TFLOP/s), the plotted curve rises by over three orders of magnitude (NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023). The points do not measure one consistent quantity: they mix precision formats and combine FLOP/s with INT8 operations per second. The curve therefore compares each generation's most prominent advertised path, not same-precision application performance. Table E.1 provides the corresponding precision-specific specifications, bandwidth, and power limits.

11.4.4 N:M structured sparsity mechanics

The sparse points in figure 11.5 depend on one more Tensor Core feature, a datapath that skips zeros only when they follow a fixed pattern. While unstructured pruning reduces model parameter counts, it rarely translates to hardware speedup because arbitrary nonzero placements destroy memory locality. Hardware accelerators resolve this with N:M structured sparsity,[17] a pruning constraint that bounds sparsity to a deterministic, fine-grained pattern. The label "N:M" specifies that exactly n_{keep} of every m_{group} contiguous values along the reduction dimension must be nonzero. Restricting nonzeros to fixed local intervals guarantees uniform memory access and predictable indexing across parallel execution units.

15 **Tensor Core**: A single Tensor Core instruction executes a complete matrix-multiply-accumulate operation on a small tile of data using a dedicated hardware block (NVIDIA 2017; NVIDIA Corporation 2020a). This approach bypasses the overhead of fetching and scheduling dozens of individual arithmetic instructions on general-purpose CUDA cores. Because these blocks constitute a large fraction of a modern accelerator's advertised tensor throughput, failing to use them can leave most of the chip's theoretical peak unavailable to the workload.

16 **NPU (neural processing unit)**: Mobile NPUs achieve low-power inference by implementing common tensor operations in fixed-function or narrowly programmable hardware rather than as fully general GPU kernels. This architectural commitment can deliver large energy-efficiency gains for supported kernels, but it makes deployment dependent on operator coverage: unsupported functions must fall back to a CPU or GPU path that may be far less efficient for that workload (Sze et al. 2017).

17 **N:M structured sparsity**: The 2:4 ratio (50 percent density) used by NVIDIA's Ampere Sparse Tensor Cores is a hardware-friendly compromise: every contiguous four-value group retains two nonzero values, preserving regular indexing while halving the dense value payload (NVIDIA Corporation 2020a). At 2:4, the metadata overhead is compact enough to store alongside the weights without overwhelming the memory-traffic savings, which is the constraint that makes the advertised 2× tensor-math throughput path plausible when kernels and model weights satisfy the pattern.

↩ The N:M and 2:4 structured sparsity patterns and their compressed storage are established in section 10.3.1.3.

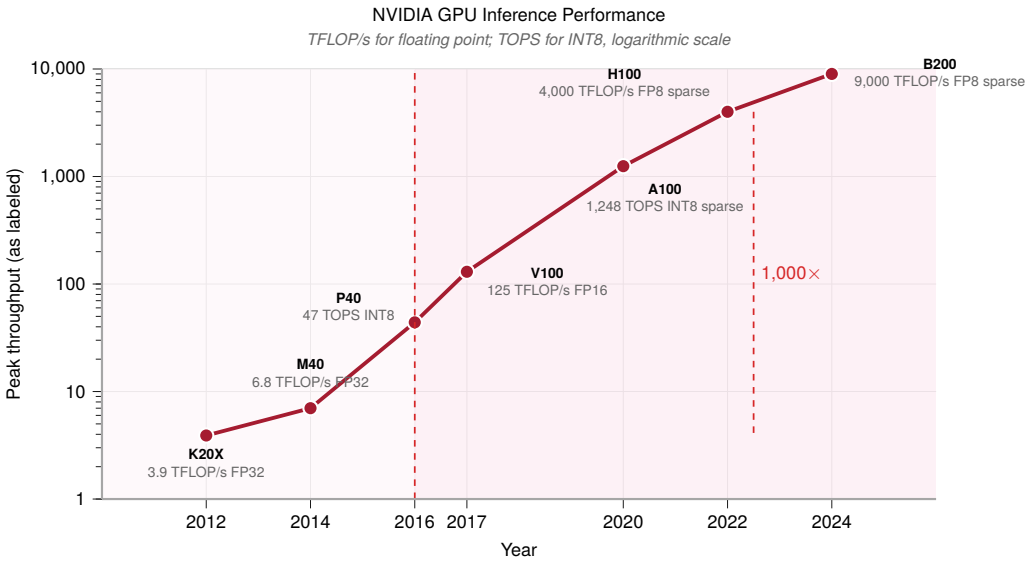


Figure 11.5: Advertised GPU Peak Capability Across Generations: Single-chip values are plotted on a logarithmic axis. The precision label at each point is essential: the curve combines architectural gains with different numerical formats and, for some points, sparsity assumptions rather than measuring same-precision application scaling (NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023).

Parallel architectures require structural regularity because of how sparse matrices must be encoded in memory. Classical formats such as Compressed Sparse Row (CSR) and Block Compressed Sparse Row (BSR) rely on auxiliary index arrays to locate nonzero entries. Compressed formats pack retained values and indices, but the corresponding activation gathers can touch scattered addresses, reducing useful bytes per memory transaction. Varying numbers of nonzeros across rows can also create load imbalance and divergent execution. Coarse block sparsity groups retained values into larger dense tiles to improve regularity, at the cost of a more restrictive pruning pattern whose quality must be evaluated. Figure 11.6 contrasts dense, CSR, block-sparse, and BSR layouts. In the two block formats, a short list of nonzero block indices tells the hardware which 3×3 tiles to fetch, so zero blocks are never read.

↪ The CSR storage format is developed in section C.1.5.

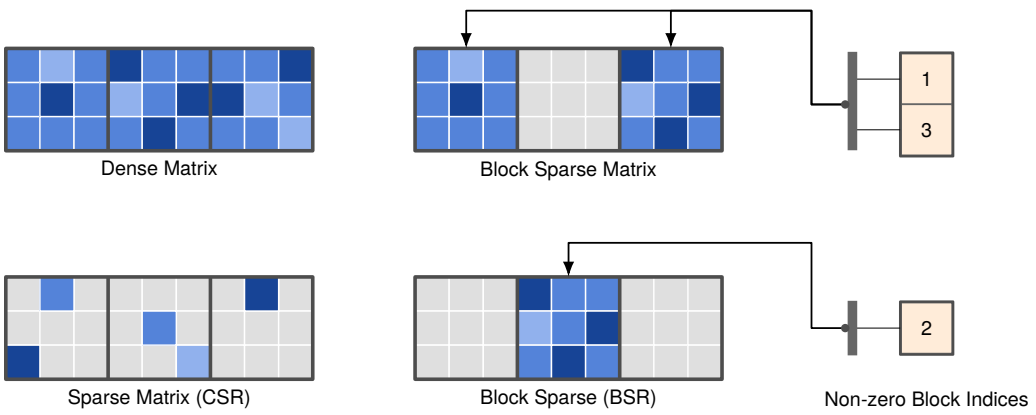


Figure 11.6: Sparse Matrix Layouts: The four panels compare dense, CSR, block-sparse, and BSR matrix occupancy patterns. The separate nonzero-block-index column represents the metadata required to locate retained blocks in the block-sparse cases. Regular block structure keeps this metadata predictable and compact so hardware can skip zero blocks and fetch retained blocks efficiently.

NVIDIA's Sparse Tensor Cores implement the 2:4 instance of this pattern, in which exactly two of every four contiguous values along the reduction dimension are nonzero (NVIDIA Corporation 2020a; NVIDIA 2020a). The storage side is the compressed 2:4 encoding that model compression introduced, the two retained values of each group plus a 4-bit metadata tag (2 bits per value) recording their original positions. The new material here is the datapath. During execution, the memory hierarchy transfers only the compressed weights and their metadata, halving the weight-value payload while adding metadata traffic; total transferred bytes therefore fall by less than half before other overheads. Inside the Sparse Tensor Core datapath, the 4-bit metadata steers an internal multiplexer network that selects the matching two values from an uncompressed four-element activation vector. These selected activations and the two nonzero weights are then routed into a half-sized dot-product multiplier tree. Because the hardware computes only on the nonzeros while streaming compressed weights, this design doubles the peak arithmetic instruction throughput, yielding an advertised $2\times$ tensor-math speedup over dense operations when weights are fine-tuned to satisfy the pattern.

The 2:4 pattern reinforces the central reality of the memory wall: hardware accelerators gain efficiency not by computing on zeros faster, but by eliminating their transfer entirely. The reduced weight payload can lower traffic across HBM and on-chip SRAM. A roofline comparison must use consistent accounting: useful dense-equivalent operations can stay fixed while bytes decrease, whereas the count of physically executed sparse operations also decreases. Metadata and activation traffic remain in either traffic budget.

Dense or sparse, a Tensor Core retires a tile per instruction only when its operands are already nearby, so accelerators never deploy matrix units on their own. The next level of the execution hierarchy exists to supply that proximity.

11.4.5 Processing elements

At that level, accelerators integrate multiple Tensor Cores with local memory into modular processing elements (PEs). High-throughput matrix units cannot be fed directly from global memory; off-chip DRAM and on-package HBM lack the bandwidth and low latency required to stream operands into dense arithmetic arrays every cycle. A processing element resolves this bandwidth deficit by co-locating heterogeneous compute pipelines (Tensor Cores for matrix multiplication, vector units for element-wise operations, and special function units for nonlinear transformations) directly alongside high-bandwidth register files and scratchpad SRAM. Staging operands inside the PE keeps data adjacent to the functional units, amortizing the latency and energy cost of external memory accesses across repeated arithmetic reuse. The term is used at two scales in this chapter. Here a PE is a tile or core that bundles several units with local memory, the role a streaming multiprocessor plays in a GPU and a core plays in a TPU; inside a systolic array (section 11.4.6), a PE is a single multiply-accumulate cell in the grid.

Processing element design varies because each architecture chooses a different balance between compute density, local memory capacity, and interconnect distance. Graphcore's Intelligence Processing Unit (IPU), for example, distributes computation across tiles backed by dedicated local SRAM (Graphcore 2020). Retaining an operand in one tile saves external traffic, but a consumer on another tile still requires an interconnect transfer. Local storage therefore shifts the mapping problem toward keeping repeated consumers close to their operands.

Across these designs, the binding physical constraint is compute density versus data locality: silicon area dedicated to arithmetic units cannot simultaneously house SRAM storage or routing wires. Packing more execution units raises peak theoretical FLOP/s, but those units idle if memory bandwidth and routing channels cannot deliver operands. A processing element's delivered efficiency therefore depends as much on its local memory hierarchy and interconnect strategy as on its raw arithmetic capability.

Systolic arrays push this locality principle to its limit. Rather than staging operands in a processing element's local memory and fetching them again for each operation, they pass operands directly from one processing element to its neighbor.

11.4.6 Systolic arrays

While Tensor Cores package matrix multiply-accumulate operations into instruction-driven micro-tiles, systolic arrays organize hundreds or thousands of processing elements into rigid, two-dimensional spatial grids. In a classic 2D systolic array, data streams rhythmically through adjacent cells on regular clock cycles: inputs march horizontally, partial sums accumulate vertically, and stationary weights remain anchored in place. Because every value passed from one processing element to its immediate neighbor participates in an arithmetic operation without returning to a register file or SRAM buffer, nearest-neighbor communication replaces memory accesses. An operand loaded once from memory contributes to N sequential multiply-accumulate steps across an $N \times N$ grid, driving memory traffic down by an order of magnitude. A quantitative comparison underscores why this spatial dataflow is fundamental to energy-efficient AI hardware.



Napkin Math 11.1: The energy advantage of pulsing data

Scenario: The systolic architecture improves energy efficiency by keeping operands local as work pulses through the array; the "Systolic" (heartbeat) metaphor reflects this benefit of reusing operands locally. This worked model compares systolic dataflow with a naive implementation that streams every operand through DRAM:

1. **DRAM-streaming baseline**: Loads **A**, **B**, and the running **C**, computes $\mathbf{A} \times \mathbf{B} + \mathbf{C}$, and writes **C** back without cache or register reuse.
   - **Data movement**: 3 loads + 1 write = 4 DRAM accesses (per operation).
   - **Energy**: $\approx 4 \times 640$ pJ + 1 pJ (compute) = 2561 pJ/op.
2. Systolic Array (128×128 size): Loads **A** and **B** once at the edges. Data "pulses" through 128 processing elements.
   - **Data movement**: 2 loads per 128 operations = 0.016 DRAM accesses (per operation).
   - **Energy**: $\approx 0.016 \times 640$ pJ + 1 pJ (compute) $\approx$ 11 pJ/op.

Systems insight: Under this deliberately naive DRAM-streaming baseline, the worked model gives the systolic array a 232.8× energy advantage. The ratio measures locality, not an inherent gap between vector and systolic arithmetic.

- Concretely, a 128×128 array can sustain 16,384 MACs/cycle with a large energy dividend by pulsing data through processing elements instead of repeatedly loading it from DRAM (Horowitz 2014; Jouppi et al. 2023).
- This locality lets dense arrays sustain many simultaneous MAC operations without paying a DRAM access for every operation.
- **Limitation**: The comparison assumes no effective reuse in the baseline and full reuse across the array. Caches, register blocking, array underutilization, and other dataflows change the ratio substantially.

Kung and Leiserson[18] (Kung and Leiserson 1979) first introduced systolic arrays, formalizing their use in parallel computing architectures for efficient matrix operations (Kung 1982). Unlike general-purpose execution units, systolic arrays exploit spatial and temporal locality by reusing operands as they propagate through the grid. Google's TPU exemplifies this architectural approach: in the TPUv4, a 128×128 systolic array of multiply-accumulate units processes matrix operations by streaming data through the array in a pipelined manner (Jouppi et al. 2023). Figure 11.7 traces this 2D pipelined dataflow: a control unit stages operands into input FIFO buffers that stream activation data horizontally across the processing grid, while partial sums accumulate vertically down columns into the bottom adder-accumulator chain. Each processing element performs one multiply-accumulate per cycle and passes its operands to its neighbors, so a value loaded once is reused across an entire row or column rather than refetched from memory.

[18] **Systolic array**: Kung and Leiserson used the term for arrays in which data advances through neighboring processing elements on a regular schedule (Kung and Leiserson 1979). Pipeline fill permits concurrent wavefronts and local reuse. The benefit depends on the stationary operand, tile, buffers, and utilization; it does not guarantee fixed DRAM savings. Irregular workloads and poorly aligned shapes remain a weaker fit because fill, drain, and idle elements consume capacity.

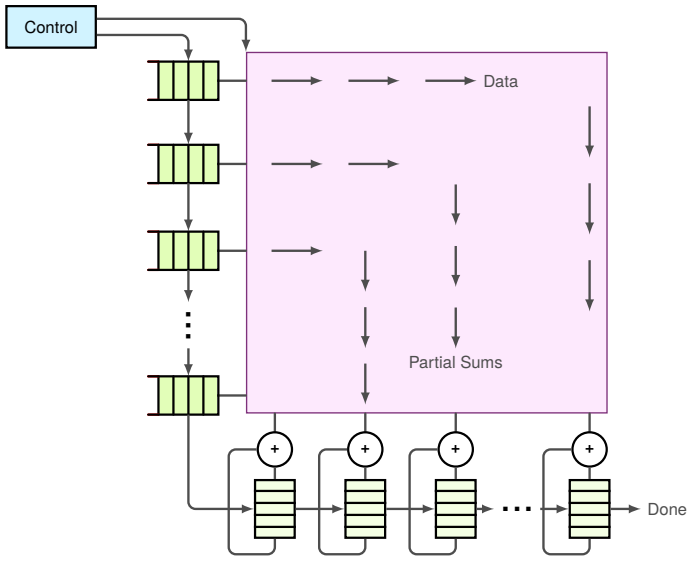


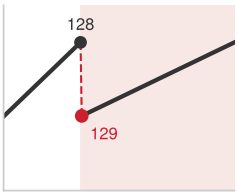
Figure 11.7: Systolic Array Dataflow Architecture: A pipelined grid of processing elements (PEs) passes operands and partial results between neighbors. The illustrated movement is one possible matrix-multiplication dataflow; designs choose which operand or partial sum remains stationary to match reuse, buffers, and tensor shape. Utilization depends on mapping and dimensions.

11.4.7 The tiling principle: Bridging graph and silicon

A fundamental mismatch exists between the *computational graph* (which sees a single 4,096 × 4,096 matrix multiplication) and the *physical silicon* (which possesses a fixed 128 × 128 systolic array). Bridging this gap requires tiling, the blocking technique of cache-aware matrix multiplication, here partitioning large tensor operations into tiles that fit exactly into the hardware's fast local memory (SRAM or scratchpad memory).

↩ Tiling as the blocking technique behind cache-aware matrix multiplication is established in section 7.5.2.2.

To process the 4,096-wide worked-example layer on a 128-wide systolic array, the compiler decomposes the result into 1,024 output tiles. Each output tile accumulates across 32 reduction-dimension tiles, so the complete matrix multiplication executes 32,768 **A**-tile-by-**B**-tile products. This decomposition is a physical requirement imposed by memory hierarchy limits: operand tiles are fetched from high-capacity, high-latency HBM, staged in fast SRAM, and pulsed through the systolic array.



Exceeding the tile width along a single dimension tips utilization along that axis off a cliff.

Tile dimensions balance on-chip capacity against arithmetic intensity. A larger tile reuses each loaded byte across more multiply-accumulate operations, raising operational intensity toward the compute-bound regime of the roofline model; the capacity ceiling is how much of **A**, **B**, and **C** fits simultaneously in fast on-chip memory. Tiling sustains high hardware efficiency (η_{hw}) by ensuring that for every byte loaded from main memory, the data is reused 128× within the systolic grid. Conversely, violating this alignment breaches the silicon contract. When a layer dimension is not an exact multiple of the hardware tile size, the accelerator pays a *fringe tax* in idle silicon. Consider a layer where only one output dimension expands from 128 to 129 while the other dimensions remain perfectly aligned to the 128-element tile. Processing that single extra column requires scheduling a second 128-wide tile across the systolic array, leaving 127 execution units idle during the fringe pass. Two tiles execute where one nearly sufficed, so the utilization factor along that dimension falls to $129/256$, or approximately 50 percent, creating the efficiency cliff illustrated in the margin figure. When multiple dimensions misalign simultaneously, these padding losses multiply across the tiled axes: a 129×129 output block partitioned into four 128×128 physical tiles yields a useful-area efficiency of $(129/256)^2$, or barely 25 percent. Algorithm 11.1 formalizes the resulting loop nest emitted by the compiler to orchestrate this on-chip staging and accumulation.

Algorithm 11.1 Tiled (blocked) matrix multiply

Require: $\mathbf{A} \in \mathbb{R}^{M \times K}$, $\mathbf{B} \in \mathbb{R}^{K \times N}$; tile sizes $M_{\text{tile}}, N_{\text{tile}}, K_{\text{tile}}$
Ensure: $\mathbf{C} = \mathbf{AB}$

```
for each row tile i_0 (size M_tile) do
    for each column tile j_0 (size N_tile) do
        initialize the C-tile accumulator to zero on chip
        for each k_0 over K in steps of K_tile do
            load A-tile [i_0, k_0] and B-tile [k_0, j_0] on chip
            accumulate the tile product on chip ▷ reuse N_tile/M_tile per byte
        end for
        write the C-tile back to memory
    end for
end for
```

Once operand tiles reside in fast local SRAM, the hardware executes the tile product without further off-chip memory traffic. Systolic arrays achieve computational efficiency by routing operands through a structured 2D grid of processing elements (PEs) that communicate directly with adjacent neighbors across dedicated registers, bypassing shared-memory bus contention. In a weight-stationary design—the canonical TPU processing grid diagrammed in figure 11.7—weights from the **B**-tile are preloaded into local PE registers and held stationary. Input activations from the **A**-tile stream horizontally across rows from left to right, while partial sums accumulate vertically down columns. On each clock cycle, every active PE multiplies its incoming activation by its stationary weight, adds the result to the incoming partial sum from above, and passes the activation rightward to its adjacent neighbor for the next cycle.

Because operands propagate one hop per cycle, synchronized execution requires an intentional time skew. Consider a minimal 2×2 tile product where output element $C_{ij} = A_{i0}B_{0j} + A_{i1}B_{1j}$ requires two products and one accumulation. The weight matrix **B** is held stationary across four PEs: PE_{00} stores B_{00}, PE_{01} stores B_{01}, PE_{10} stores B_{10}, and PE_{11} stores B_{11}. Activation A_{i0} enters the top row and A_{i1} the second row, because each row holds the weights for one reduction index. If A_{00} enters the top row at PE_{00} in cycle 0, its partial product $A_{00}B_{00}$ moves down to PE_{10} in cycle 1. That PE holds B_{10}, so the activation it needs is A_{01}, which must therefore enter the second row at PE_{10} one cycle later than A_{00} entered the first. In cycle 1, PE_{10} adds $A_{01}B_{10}$ to the arriving partial sum and emits C_{00}. Meanwhile A_{00} has moved right to PE_{01}, and its product $A_{00}B_{01}$ reaches PE_{11} in cycle 2, just as A_{01} arrives there from PE_{10}, so C_{01} emerges one cycle after C_{00}. Inputs along each successive row and column are staggered in the same way, so operands meeting at any physical coordinate represent identical reduction indices. Local reuse amortizes the initial fill and drain overhead: each loaded activation is reused across an entire row of N_{tile} processing elements, while each stationary weight serves an entire sequence of input vectors before eviction. Sustained throughput, however, remains bounded by off-chip memory bandwidth whenever operand replenishment cannot keep pace with array retirement (section 11.5.1).

A 128×128 systolic array capable of 16,384 operations per cycle requires a continuous operand stream to maintain utilization. On-chip buffers feed activations and weights to the array edges and are replenished from off-chip memory as needed; reuse within the array avoids fetching every operand from HBM each cycle. The TPU v4's 1,200 GB/s HBM2 bandwidth helps sustain this stream, but off-chip bandwidth can still limit models whose working sets and reuse patterns exceed on-chip capacity.

Quantization techniques reduce model memory footprint by converting FP32 weights to INT8 representations. Converting 32-bit floating-point weights to 8-bit integers reduces weight traffic by $4\times$; whether that changes a kernel from bandwidth bound to compute bound depends on the operation's original arithmetic intensity, the ridge point of the accelerator's INT8 path, and the overhead of quantization and dequantization. Similarly, structured pruning removes entire rows or columns of weight matrices, reducing both the data volume that must traverse memory hierarchies and the computation required. In the D·A·M taxonomy, these algorithmic optimizations prove valuable precisely because they target the memory bottleneck that limits accelerator performance in practice.

Systems Perspective 11.1: Matching architecture to workload

Challenge: Systolic arrays must choose which operand to keep stationary in processing-element registers (weights, inputs, or partial sums), and that choice hard-codes the hardware's preference for certain model types. Section 11.8 develops the three options as a general mapping decision.

No single accelerator architecture suits every workload. A chip optimized for weight-stationary dataflow (like early TPUs) excels at CNNs where filter weights are small and reused across many image patches. That same architecture stalls on large language model (LLM) autoregressive inference at batch size 1, where the entire weight matrix must be read from off-chip memory once per generated token with zero batch reuse. No choice of stationary operand recovers reuse that does not exist; the levers are batching, lower-precision weights, and memory bandwidth.

11.4.8 Numerics in AI acceleration

Systolic arrays and Tensor Cores often support reduced-precision arithmetic. FP16 halves the bytes per value relative to FP32, and a target may provision more low-precision multiply-accumulate capacity. A 2× speedup is not automatic: it requires a supported kernel and sufficient parallel work without another bottleneck. Reduced precision is a hardware-software decision. Input and accumulation formats may differ, affecting accuracy, throughput, energy, and movement.

11.4.8.1 Precision trade-offs

Lower precision is not free, for the range-and-precision reasons that forced loss scaling in mixed-precision training and that section 10.4.2 compared format by format. FP16 trades dynamic range for significand bits, BF16 keeps FP32's exponent range at the cost of precision, and INT8 has no exponent at all, so its usable range depends on scale, zero-point, and clipping choices. For the hardware designer, those numerical properties set the price of each datapath. A narrower format needs a smaller multiplier and moves fewer bytes, but accumulation usually stays wider, as the FP32 accumulator of the Tensor Core instruction in listing 11.13 shows, so part of the area saved in the multipliers is spent keeping sums accurate.

↩ The software side of mixed-precision training, including loss scaling, is established in section 8.7.3.

The evolution of AI hardware reflects co-design between numerical methods and hardware capability. Earlier GPU generations lacked the dedicated mixed-precision matrix paths now used for deep learning, even when other units supported several formats. As training and inference methods demonstrated acceptable accuracy with selected lower-precision operations, vendors added native FP16, BF16, and integer tensor paths. Frameworks and libraries expose them through compatible kernels, autocasting, quantization, and calibration. Software gains materialize only when the model selects an efficient path; changing storage alone may reduce bytes without producing the advertised arithmetic speedup.

Precision support is integrated into execution units and memory paths. SIMD and SIMT lanes may support several scalar formats, while Tensor Cores (section 11.4.3) and systolic units (section 11.4.6) expose architecture-specific input and accumulation combinations. Lower precision reduces bytes per operand and may increase operations per cycle. Tiling and dataflow, not precision itself, determine reuse, while conversion, scaling, or fallback overhead can offset part of the gain. A format is useful only when the kernel and numerical policy can employ it safely.

11.4.8.2 Mixed-precision computing

Modern AI accelerators increasingly support mixed-precision execution, allowing different numerical formats to be used at various stages of computation. Training workloads often use FP16 or BF16 for matrix multiplications, while maintaining FP32 accumulations to preserve precision (Micikevicius et al. 2017; Mellempudi et al. 2019). Inference workloads, by contrast, optimize for INT8 or even INT4 while preserving key activations in higher precision where accuracy requires it.

The shift toward precision diversity is evident in the evolution of AI hardware. Early architectures such as NVIDIA Volta provided limited support for lower precision beyond FP16, whereas later architectures, including Turing and Ampere, expanded the range of supported formats. Table 11.4

traces this progression: Ampere GPUs introduced TF32 as a hybrid between FP32 and FP16 (NVIDIA 2020b), alongside broader support for BF16, INT8, and INT4 (NVIDIA Corporation 2017, 2018, 2020a).

Table 11.4: Precision Support Evolution: From Volta through Ampere, Tensor Core support expanded from FP16 to TF32, BF16, and integer formats, broadening the precision choices available to training and inference; the CUDA-core column shows the separate scalar/vector formats each generation supports.

Architecture	Year	**Supported Tensor Core precisions**	**Supported CUDA Core Precisions**
Volta	2017	FP16	FP64, FP32, FP16
Turing	2018	FP16, INT8, INT4, INT1	FP64, FP32, FP16, INT8
Ampere	2020	FP64, TF32, BF16, FP16, INT8, INT4	FP64, FP32, FP16, BF16, INT8

Newer architectures incorporate a growing diversity of numerical formats because different workloads bind at different points on the accuracy-throughput-energy trade-off. Precision support is therefore another form of workload matching, not a generic feature checklist.

The precision format used in hardware design has cascading implications across the entire system. Reducing an operand from FP32 to FP16 halves its value bytes. When those values dominate traffic, the reduction can substantially lower movement energy; the full energy budget still includes access counts, arithmetic, and other device activity (Horowitz 2014). Simultaneously, Tensor Cores and systolic arrays can pack more lower-precision multiply-accumulate units into the same silicon area, raising peak throughput (Dally et al. 2021; Dally 2023). Lower-precision integer arithmetic also consumes less energy than FP32 in representative technology studies (Horowitz 2014), and the inference-focused TPUv1 was built around 8-bit multiply units (Jouppi et al. 2017). The systems insight is that reduced precision does not merely "save bits": it can reduce value traffic *and* raise the supported compute ceiling, creating opportunities on both sides of the roofline.

As AI models continue to scale, precision support connects the compute primitive discussion back to the memory wall: lower-bit formats matter when they reduce the bytes moved and keep the hardware's matrix engines fed. The remaining architectural question is how these execution units, precision formats, and memory paths integrate into complete accelerator systems. Architectural integration determines how efficiently computational primitives become usable accelerator throughput. SIMD lanes, Tensor Cores, and systolic arrays are building blocks, but their full-chip organization varies significantly across AI processors; the choice of execution units, their numerical precision support, and their connectivity shape how effectively hardware can scale for deep learning workloads.

11.4.9 Processor configurations across deployment targets

Modern AI processors exhibit a range of design trade-offs based on their intended applications, and comparing their configurations reveals how deployment constraints drive architectural divergence. A training-optimized accelerator like the NVIDIA A100 packs many Streaming Multiprocessors with wide SIMD units and FP16 Tensor Cores because training throughput scales with aggregate multiply-accumulate capacity (NVIDIA Corporation 2020a). Google's TPUv4 makes a radically different bet: just two cores per chip, each containing massive BF16 systolic arrays, a design that trades programmer flexibility for efficiency on dense matrix multiplications (Jouppi et al. 2023). At the inference end, Intel's Sapphire Rapids dedicates Advanced Matrix Extensions (AMX) tile engines to INT8 and BF16, reflecting the insight that inference models tolerate reduced precision (Intel Corporation 2021a). Mobile neural engines take this further by shrinking matrix engines into low-power system-on-chip (SoC) blocks, prioritizing energy efficiency per operation over peak throughput. Table 11.5 compares these architectural configurations.

Table 11.5: AI Processor Configurations: Modern AI processors prioritize different execution unit characteristics for specific workloads: NVIDIA A100 uses wide SIMD and Tensor Cores for training, Google TPUv4 emphasizes high-throughput BF16 matrix multiplication, Intel Sapphire Rapids focuses on INT8-optimized inference, and mobile NPUs prioritize low-power execution. These variations in SIMD width, Tensor Core size, and processing element count reflect the growing diversity in AI hardware architectures.

Processor	**Vector execution**	**Matrix engine**	Processing Elements	**Primary workloads**
NVIDIA A100	CUDA FP/INT lanes	Tensor Core instruction tiles	108 SMs	Training, HPC
Google TPUv4	Vector unit	128×128 BF16 systolic arrays	2 cores/chip	Training
Intel Sapphire	512-bit AVX-512	AMX tiles (up to 16 rows by 64 bytes each)	Up to 60 cores	Inference
Mobile NPU	CPU/GPU/DSP vectors	Small matrix engines	Integrated NPU blocks	Mobile inference

The pattern across these configurations reveals a consistent engineering principle: each design sacrifices generality to optimize for its target workload's dominant operation and precision. Training chips invest silicon in wide floating-point datapaths; inference chips trade precision for throughput; mobile chips trade throughput for energy efficiency. No single design dominates across all workloads, which is precisely why hardware selection depends on workload analysis rather than headline specifications.

11.4.10 Cost-performance analysis

Architectural specifications define computational potential, but deployment decisions require cost-performance analysis. Raw compute is only one input: delivered performance may be limited by arithmetic throughput, memory bandwidth, interconnect latency, software overhead, or low operational utilization.

The energy gap established in section 11.2.3 drives accelerator specialization. Because memory access costs dominate computation, accelerators achieve only a fraction of their peak throughput on memory-bound kernels. Conversely, architectures that maximize data reuse, such as systolic arrays executing dense matrix multiplications, sustain substantially higher utilization by amortizing each memory fetch across multiple arithmetic operations.

Consider an engineering team choosing between deploying a larger cluster of older accelerators or a smaller cluster of newer units. Peak FLOP/s figures mislead whenever dominant kernels are memory-bandwidth or communication bound. In a large model pipeline, dense feed-forward and attention projection layers may remain compute bound, whereas embedding lookups, layer normalization, residual additions, and small-batch autoregressive decoding often have low arithmetic intensity. Their measured bottlenecks can include memory bandwidth, dependent access latency, reductions, and launch overhead; the operator name alone does not identify the limiting resource. The decisive metric is delivered workload throughput per dollar under realistic cluster utilization, memory capacity constraints, available memory bandwidth, and scale-out interconnect efficiency.

These architectural and economic constraints explain the rapid industry adoption of newer accelerators despite substantial increases in nominal unit price. For memory-bound workloads, generational improvements in memory bandwidth and compiler code generation dominate end-to-end execution time. Cloud deployments introduce additional trade-offs: hourly rental rates, dynamic instance utilization, and infrastructure overhead continuously shift the economic break-even point between purchasing on-premises hardware and leasing cloud capacity.

Table 11.6 provides an illustrative cost worksheet for common accelerators. List prices vary across vendor, region, contract terms, and procurement volume. Furthermore, the throughput rows correspond to distinct numerical precision modes, meaning the price-per-peak-operation metric is meaningful within a single row but invalid as a cross-device benchmark. The table establishes a framework for making economic assumptions explicit rather than offering a universal hardware ranking.

Table 11.6: Illustrative Accelerator Cost Worksheet: Each row divides an approximate acquisition-price assumption by the representative peak mode shown. Because precision, sparsity assumptions, and price sources differ, the ratios are not comparable as workload performance. Total cost also includes utilization, memory capacity, power, cooling, interconnect, software, and infrastructure; TPU acquisition cost is estimated from cloud pricing.

Accelerator	List price	Representative Peak Throughput (precision shown)	Memory Bandwidth	Price/performance
NVIDIA V100	~$10,000	125 TFLOP/s	900 GB/s	$80/(TFLOP/s)
NVIDIA A100	~$15,000	312 TFLOP/s	2,039 GB/s	$48.1/(TFLOP/s)
NVIDIA H100	~$25,000–30,000	494 TFLOP/s (TF32)	3,350 GB/s	~$50.6/(TFLOP/s)
Google TPUv4	~$8,000*	275 TFLOP/s (BF16)	1,200 GB/s	~$29.1/(TFLOP/s)
Intel Gaudi 2	~$12,000	865 TFLOP/s (FP8)	2,450 GB/s	$13.9/(TFLOP/s)

The worksheet demonstrates why accelerator selection cannot be reduced to a single headline specification. A bandwidth-bound kernel benefits more from the H100's 3,350 GB/s memory subsystem than from an incremental gain in peak arithmetic, whereas a compute-bound general matrix multiplication (GEMM) shows the opposite sensitivity. Device memory capacity dictates whether working sets fit on-chip without spilling or partitioning, while compiler and driver maturity determines whether the application can actually reach the advertised datapath.

Vector pipelines, matrix-tile engines, and Tensor Cores provide massive peak arithmetic capability. An NVIDIA A100 advertises 312 TFLOP/s on its FP16 tensor path, and subsequent accelerator designs add narrow-precision FP8 execution modes (NVIDIA Corporation 2020a; Kuzmin et al. 2022; Micikevicius et al. 2022). NVIDIA's Blackwell (B200) architecture extends this trajectory with an FP4 tensor datapath (a 4-bit floating-point format that, unlike INT4, spends some of its four bits on an exponent and so trades uniform spacing for range), rated at up to 9 PFLOP/s dense or 18 PFLOP/s sparse peak throughput per chip (NVIDIA Corporation 2024). Reaching those theoretical ceilings requires numerical representations, kernel implementations, and sparsity patterns that align exactly with the specialized hardware paths. Narrower numerical formats expand the available design space, but numerical stability and dynamic range requirements dictate whether a workload can safely use FP4.

Dividing a model's total operation count by peak arithmetic throughput produces an idealized lower bound on execution time, not a realistic inference latency. On an A100, dividing ResNet-50's 8.2 GFLOP per inference by the 312 TFLOP/s FP16 peak yields a floor of about 26 µs. Real ResNet-50 inference, however, requires milliseconds. Part of that gap is kernel launch, synchronization, and host overhead that no roofline captures; part is memory traffic in low-intensity layers, and that part exposes the central tension in machine learning systems: arithmetic throughput has far outpaced the memory bandwidth required to feed the execution units. *Moving data from memory costs orders of magnitude more energy than arithmetic, and memory bandwidth has improved more slowly than tensor arithmetic throughput.* This physical imbalance dictates whether that advertised 312 TFLOP/s delivers high sustained hardware utilization or remains largely idle on a given workload (Horowitz 2014; Gholami et al. 2024).

Resolving this performance gap requires analyzing the memory hierarchies that supply the compute units. The memory subsystem is not passive scaffolding; in conjunction with arithmetic intensity and kernel scheduling, it determines the fraction of peak compute an algorithm can extract from the machine.

11.5 AI Memory Systems

A neural network kernel starves whenever the memory subsystem cannot deliver operands and consume outputs fast enough to saturate arithmetic pipelines. While modern accelerators provide hundreds of teraflops—or even petaflops—on low-precision tensor datapaths (NVIDIA Corporation 2020a, 2024; Choquette 2023), compute pipelines idle as soon as memory traffic saturates the bus. The **AI memory wall** captures this growing divergence between arithmetic capability and memory bandwidth.

Because deep neural networks manipulate millions or billions of parameters across high-dimensional activation tensors, performance is frequently dictated by data movement rather than raw arithmetic capacity. Accelerators combat this memory wall through tiered storage hierarchies,

↩ The memory wall, the widening gap between compute growth and memory bandwidth growth, is established in section 2.2.

software-managed scratchpads, high-bandwidth 3D stacked memory, and architectural dataflow strategies designed to maximize spatial and temporal reuse.

11.5.1 Understanding the AI memory wall

Definition 11.4: AI memory wall

The AI memory wall is the ML accelerator performance constraint that arises when arithmetic throughput (R_{peak}) outpaces memory bandwidth (BW). The core issue is whether the memory system can supply operands and absorb results quickly enough to sustain the specialized primitives introduced in section 11.4.

1. **Significance**: A workload has reached the memory wall when increasing FLOP/s alone no longer reduces execution time because the $\frac{D_{\text{vol}}}{\text{BW}}$ term dominates the overall runtime.
2. **Distinction**: Unlike a general-purpose memory wall, which affects all computing, the AI memory wall is driven by the large parameter volumes and intermediate activation tensors required by deep learning.
3. **Common pitfall**: A frequent misconception is that the memory wall is "fixed" by more memory. In reality, it is a bandwidth-latency gap: even with infinite capacity, the speed of moving data between memory and compute remains the fundamental physical bottleneck.

Data access can dominate the energy budget of a memory-intensive kernel.[19] Figure 11.8 charts representative 45 nm operation energies on a logarithmic scale, highlighting the multi-order-of-magnitude gulf between an INT8 addition (0.03 pJ) or FP32 multiply (3.7 pJ) and an off-chip DRAM fetch (640 pJ).

[19] **Von Neumann bottleneck**: Separating storage from compute requires instructions and data to cross interfaces rather than remain in local registers. In the representative 45 nm estimates from Horowitz (2014), a DRAM access costs more than four orders of magnitude more energy than an INT8 addition, the cheapest operation charted. The exact ratio changes with technology and operation, but locality remains a first-order design concern.

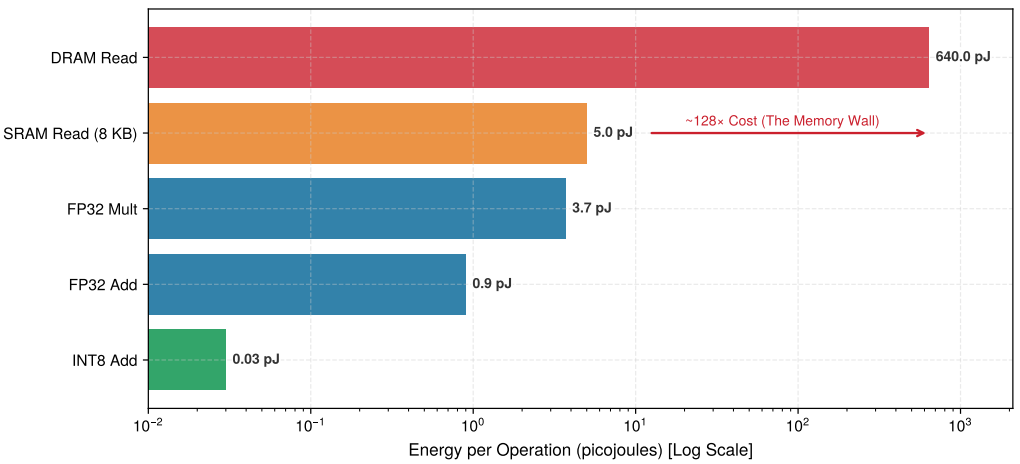


Figure 11.8: The Energy Hierarchy: Representative 45 nm energy estimates on a logarithmic scale from Horowitz (2014). An off-chip DRAM access costs about 170× an FP32 multiply and about 128× a read from an 8 KB SRAM array in this model; a small, register-adjacent L1 access (about 0.5 pJ in the registry used elsewhere in this chapter) is cheaper still. The values are not universal device specifications; they illustrate why reuse in registers and SRAM can matter more than reducing arithmetic count alone.

11.5.1.1 Quantifying the compute-memory performance gap

The physical energy disparity across storage tiers mapped in figure 11.8 grows more severe with each hardware generation. Over the past two decades, peak computational capabilities have grown substantially faster than DRAM bandwidth (Gholami et al. 2024). This divergence stems from physical scaling constraints: while transistor scaling and dense systolic arrays multiply arithmetic throughput, memory bandwidth remains bounded by pin counts, package-substrate routing density,

and interface signaling power. Representative high-end accelerators can deliver on the order of 10^3 TFLOP/s of peak tensor throughput (for example, NVIDIA H100 delivering 989 TFLOP/s in FP16 or 1,979 TFLOP/s in dense FP8) while providing approximately 3.35 TB/s of memory bandwidth (Choquette 2023). This implies that on the order of 10^2 FLOP of work per byte moved is required to fully use the compute, which can exceed the arithmetic intensity of many practical neural network workloads.

When a workload's arithmetic intensity falls below this threshold, data transfer ($\frac{D_{\text{vol}}}{\text{BW}}$) sets the larger of the two modeled time bounds. Accelerators attempt to tolerate memory latency—typically hundreds of clock cycles for off-chip DRAM—by maintaining thousands of concurrent execution threads and switching between active warps while memory requests remain in flight. However, when memory bandwidth saturates or when register pressure limits occupancy, the scheduler runs out of independent instructions to issue. Compute pipelines then stall, leaving high-throughput tensor cores idle while waiting for operands. Simultaneously, driving wide off-chip memory buses across capacitive printed circuit board traces and PHY interfaces consumes a substantial fraction of the chip's TDP, directly constraining the energy budget available for arithmetic logic (Horowitz 2014; Sze et al. 2017).

11.5.2 Hardware balance and the ridge point

Different devices place the compute-bandwidth boundary at different intensities. The ridge point, named alongside the iron law as the boundary between a machine's memory and compute limits, is a property of the device. Computer architects call the same ratio the **hardware balance** (I_{ridge}), peak arithmetic throughput divided by peak memory bandwidth, the intensity at which the roofline's compute and bandwidth ceilings intersect:

$$I_{\text{ridge}} = \frac{R_{\text{peak}}}{\text{BW}}$$

The ratio sets a device-specific boundary. Depending on the chosen precision, a high-end accelerator such as H100 may require roughly 150–300 FLOP/byte to reach its arithmetic ceiling, whereas a microcontroller may have a much lower ridge point. The same kernel can therefore be bandwidth bound on a compute-dense accelerator and compute bound on an edge processor. Hardware balance does not label either device as universally efficient; it identifies how much reuse a workload needs on that device.

Peak computational capability has expanded faster than memory bus performance across accelerator generations, and the size of that mismatch is what sets how much data reuse a kernel needs to stay fed. Figure 11.9 plots these two growth trajectories on a logarithmic scale normalized to the V100 baseline, isolating how the shaded gap between peak compute and memory bandwidth widens to a 2.0× reuse burden on B200.

Because both curves are normalized to the V100, the separation between them measures how far the hardware ridge point, the threshold a kernel must clear to become compute bound, has risen since that generation. Figure 11.10 plots that same ratio generation by generation, leaving out the H200 exception, so the doubling of the gap appears directly as the ridge point roughly doubling from V100 (139 FLOP/byte) to H100 (295 FLOP/byte) and then holding near that level on B200 (281 FLOP/byte). A kernel that cleared the V100 ridge comfortably can fall below the H100 ridge without any change to its code, so newer hardware demands more operational reuse before execution reaches peak throughput.

The rising ridge and the energy hierarchy of figure 11.8 describe the same pressure from two sides. Each byte fetched from DRAM costs more time relative to the arithmetic it feeds with every generation, and about 173× the energy of the FP32 multiply that consumes it. The TPUv1's performance-per-watt advantage (example 11.1) came largely from minimizing that off-chip movement through systolic arrays and large on-chip SRAM (Jouppi et al. 2017). Whether a model can exploit such a design depends on the pattern of its memory accesses, which differs sharply among parameters, activations, and optimizer state.

11.5.3 Memory access patterns in ML workloads

Beyond raw computational throughput, an accelerator's sustained efficiency depends on supplying operands to execution units without pipeline stalls. Deep neural networks impose three concurrent

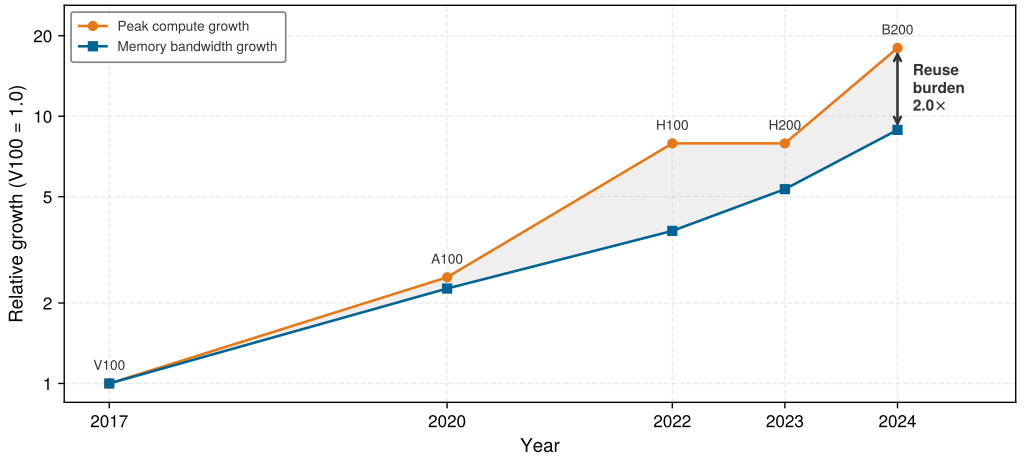


Figure 11.9: The Compute-Bandwidth Divergence: Peak dense FP16 throughput and memory bandwidth across five NVIDIA datacenter generations, each normalized to its V100 baseline on a log scale. Over 2017–2024 compute grows 18× while bandwidth grows 8.9×, so the shaded separation between the curves widens by 2.0×. That separation is the data reuse a kernel must supply to keep the arithmetic units busy. The H200 step is the instructive exception: bandwidth rose while peak compute held flat, narrowing the gap for one generation.

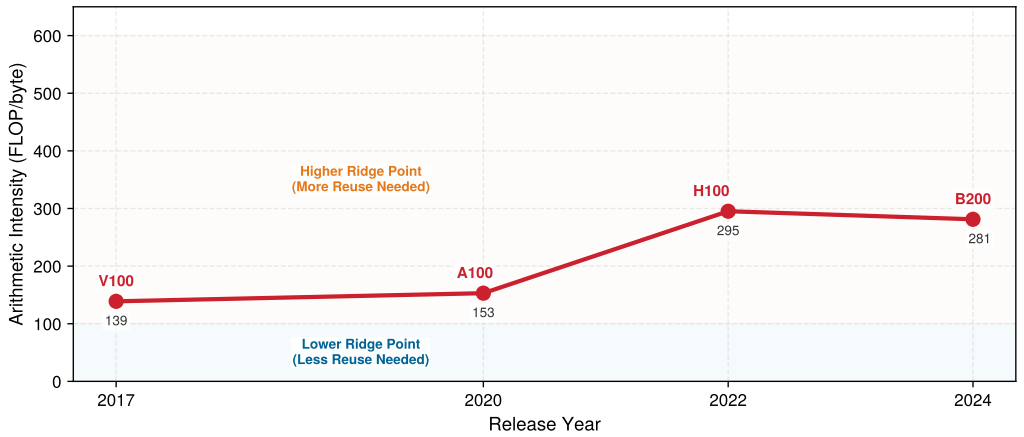


Figure 11.10: The Rising Ridge: Hardware arithmetic intensity (FLOP/byte) over time computed as each chip's dense FP16 tensor peak divided by its memory bandwidth. This is the arithmetic intensity a kernel must reach before compute rather than bandwidth limits it. As compute capability grows faster than memory bandwidth, the ridge point rises from V100 through H100 and remains high on B200. This trend explains why architectures with high data reuse flourish while low-reuse workloads face a growing hardware tax.

demands on this data supply. First, model parameters (weights and biases) may reach hundreds of billions of values (Brown et al. 2020). In dense FP16 autoregressive generation at batch size 1, streaming weights from device memory for each generated token gives roughly 1 FLOP/byte for the weight-dominated matrix-vector work. Second, intermediate activations produced at each layer must be buffered in memory for subsequent operations; during training, activations needed by back-propagation must either be retained until their backward consumers or recomputed. Third, training adds gradient and optimizer traffic: backpropagation computes gradients for every parameter, while algorithms such as Adaptive Moment Estimation (Adam) read and write multi-word momentum states, multiplying data movement volume per parameter update (Narayanan et al. 2021; Kwon and Rhu 2018).

The physical cost of this data movement depends on where each tensor resides in the memory hierarchy. While an on-chip register access completes in a single clock cycle at sub-picojoule energy,

a dependent off-chip DRAM fetch can expose hundreds of cycles of latency and expends orders of magnitude more energy. Independent requests can overlap that latency, subject to available concurrency and bandwidth. Tracing a single tensor through the memory tiers during an inference pass makes this disparity concrete, as demonstrated by the keyword spotting (KWS) pipeline in lighthouse 11.2.

Lighthouse 11.2: Life of a tensor: GPU-hosted KWS

Recall the Keyword Spotting lighthouse summarized in table 1.4. KWS normally runs on a microcontroller, but its one-second input is small enough to follow tier by tier, and the tier ladder it reveals is the same one every larger tensor climbs. If that clip is batched on a GPU-hosted inference path, its physical journey through the memory hierarchy looks like this:

1. **DRAM (HBM)**: The tensor starts here.
   - **Size**: 16,000 samples $\times$ 2 bytes (FP16) = 32 KB.
   - **Latency**: A representative off-chip request takes ~300 ns (plus queuing delay). Transferring the entire clip also depends on bandwidth and the number of requests in flight.
   - **Energy**: Cost is ~20 pJ/bit, the same 640 pJ per 32-bit access used throughout the chapter.
2. **L2 cache**: Device-memory requests may be served here when the requested cache lines are resident.
   - **Latency**: ~4 ns.
   - **Access**: Shared across multiple Streaming Multiprocessors (SMs).
3. **L1 cache/shared memory**: A specific SM works on a tile of the audio, using hardware-managed cache or explicitly staged shared memory.
   - **Latency**: ~1 ns.
   - **Locality**: If an L1 request misses, it can still be served by L2 before an HBM access is required.
4. **Registers**: The Tensor Core operates here.
   - **Latency**: Typically a small number of cycles, depending on the instruction and architecture.
   - **Device arithmetic peak**: 312 TFLOP/s on the A100 FP16 tensor path, not a register-transfer rate.
   - **Energy**: Cost is ~0.1 pJ/bit.

Systems insight: Latency and energy per bit generally rise from registers to HBM. The working set's location, access count, reuse, and dependencies determine the memory contribution to time and energy; arithmetic adds its own cost. The bandwidth term of the roofline is whichever link of this ladder the working set must cross; HBM-to-L2 is one possible limiting link, not the only one.

The boundary between memory-bound and compute-bound execution emerges directly from the balance between data transfer time and arithmetic execution time. For a workload requiring data volume D_{vol} (bytes) across available memory bandwidth BW (bytes/s), and executing O floating-

point operations on hardware with peak throughput R_{peak} (FLOP/s) at realized hardware utilization η_{hw}, the respective execution times are the iron law's data and compute terms:

$$T_{\text{memory}} = \frac{D_{\text{vol}}}{\text{BW}}$$
$$T_{\text{compute}} = \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$$

In a strictly sequential model without overlap, $T_{\text{memory}} > T_{\text{compute}}$ identifies memory-bound execution. When hardware overlaps memory transfers with computation, the larger term sets the theoretical lower bound on execution time. In practice, however, data dependencies, memory controller queueing, and partial pipeline overlap dictate how much arithmetic hardware remains idle while waiting for operands.

The structural widening between model footprints and available memory bandwidth is what turns T_{memory} into the binding constraint across generations. Parameter count is a capacity quantity. In low-batch dense decoding with weights streamed from device memory, weight traffic per generated token grows with the parameter count. Sparse expert models read selected experts rather than every stored parameter, so their total parameter counts do not directly specify per-token traffic. Figure 11.11 plots this logarithmic divergence relative to 2014, contrasting the roughly $4{,}900\times$ increase in model parameters (red scatter points) against the modest $17\times$ scaling in accelerator memory bandwidth (blue circles) to isolate the widening shaded memory-wall gap (Krizhevsky et al. 2012; Brown et al. 2020; Chowdhery et al. 2022; Dubey et al. 2024). Even high-bandwidth accelerators such as NVIDIA B200 and AMD MI300X/MI325X cannot close this growth-rate gap through bandwidth scaling alone (NVIDIA Corporation 2024; AMD 2023). Parameter counts for proprietary systems such as GPT-4 and Gemini are not officially disclosed, so they are not plotted as factual data points.

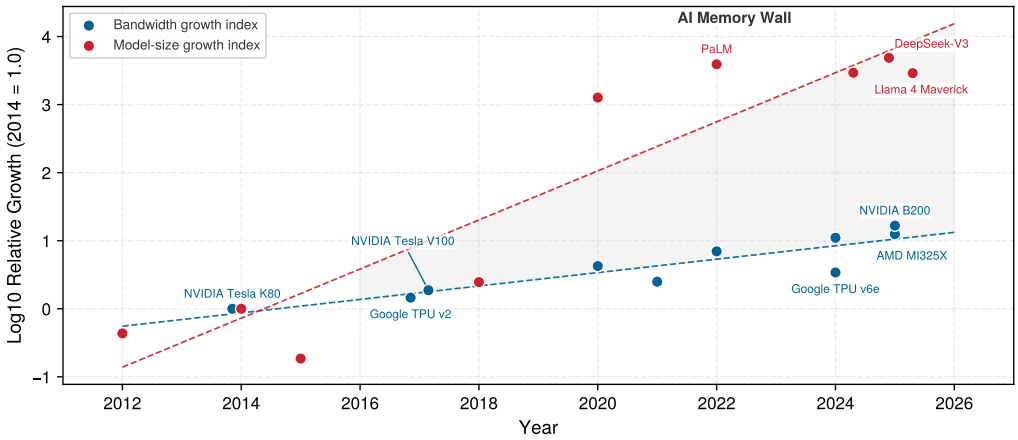


Figure 11.11: Model Size vs. Hardware Bandwidth: Both series are the base-10 logarithm of growth relative to 2014, so each unit on the vertical axis is a tenfold increase. Over the plotted decade parameter counts peak at roughly 4,900× against bandwidth's 17×. The shaded band is a gap between logarithms rather than a difference, so its width reads as a ratio, and the two fitted trends diverge at a constant rate.

11.5.3.1 Irregular memory access

Production machine learning workloads combine regular dense matrix kernels with irregular memory access patterns that defeat conventional caching and prefetching hardware. While dense operations like GEMM and convolution stream contiguous blocks that map cleanly onto hardware caches and systolic arrays, irregular operations rely on indirect, sparse, or input-dependent memory addresses. Table 11.7 contrasts these two memory regimes across access geometry, cache locality, and hardware pressure.

Table 11.7: Regular and Irregular Memory Access: Regular dense kernels expose tileable reuse and predictable addresses, whereas irregular ML components rely more on indirect or input-dependent access, weakening caching and prefetching and adding metadata or load-balance overhead. A single model may contain both regimes.

Feature	**Regular dense kernels**	**Irregular ML components**
Access pattern	Contiguous or tiled	Indirect, scattered, or input-dependent
Cache locality	Often high after tiling	Often lower and workload-dependent
Data reuse	Structured across tiles or batches	Sparse or dynamic, depending on the operator
Dependencies	Static loop nests	Input-dependent routing or variable shapes
Examples	GEMM, convolution, dense attention	Embedding lookup, unstructured sparsity, MoE routing
Primary pressure	Compute or bandwidth, depending on arithmetic intensity	Latency, bandwidth, metadata, and load imbalance
Energy consequence	Depends on reuse and memory tier	Extra movement and metadata can increase energy

Within a single neural network, memory access behavior varies sharply across layer types. Convolutional layers exhibit high spatial locality and register reuse as small weight kernels slide across input feature maps. Fully connected layers and dense attention execute regular matrix multiplications, but their large parameter matrices or key-value caches can exceed on-chip SRAM capacity and become bandwidth bound when operational reuse is low. Variable sequence lengths complicate memory buffer allocation, but the attention operations within each sequence remain regular, contiguous matrix multiplies.

True memory irregularity arises from three distinct architectural patterns: unstructured sparsity,[20] embedding lookups, and dynamic token routing. In unstructured sparsity, nonzero weights or activations are stored in compressed formats (such as CSR or COO) that require pointer indirection to decode operand coordinates. In embedding tables, sparse categorical IDs select arbitrary rows from tables spanning tens of gigabytes in DRAM, transforming sequential memory transfers into uncoalesced, scattered reads. In Mixture-of-Experts (MoE) models, routing gates direct tokens to different expert networks based on input context; while deterministic for a given input, token assignments vary across batches, disrupting prefetch pipelines and unbalancing execution queues across parallel cores.

Irregularity degrades performance through distinct mechanisms, and treating them as one memory problem can point to the wrong optimization. Scattered addresses may turn one logical access into several memory transactions, so each fetched cache line carries few useful payload bytes. Indirection can also serialize address generation: the system must fetch an index or pointer before it knows which payload to request, limiting memory-level parallelism even when aggregate bandwidth remains available. Input-dependent routing creates a third failure mode when some processing elements receive longer queues than others; completion time follows the slowest queue while neighboring units idle. Variable-sized allocations may fragment capacity, but fragmentation is neither necessary nor sufficient for irregular access. A sequential stream can saturate bandwidth without being irregular, whereas a dependent lookup chain can stall far below peak bandwidth.

Diagnosis should therefore begin with the mechanism that prevents forward progress. A low ratio of useful payload bytes to transferred bytes indicates poor spatial utilization or coalescing. Long memory stalls alongside unused bandwidth indicate dependent misses or too few concurrent requests to hide latency. Uneven queue depths or per-route work reveal load imbalance rather than a cache-capacity problem. Cache hit rate alone cannot distinguish these cases because a miss can waste bandwidth, expose latency, or both. Each diagnosis implies a different remedy: packing or blocking improves spatial utilization, reordering and prefetching expose concurrent requests, structured sparse formats reduce indexing overhead, and routing or partitioning balances work. The memory hierarchy determines the cost of the remaining accesses, while dataflow and tensor-layout choices determine how much irregularity software can remove before execution reaches the hardware.

The optimization order follows that diagnosis. Representation comes first when the format is under system control because a block-structured sparse format or grouped routing can remove indirection before a kernel executes. Request organization comes next: accesses issued together should target adjacent addresses, and enough independent requests must remain in flight to hide unavoidable latency. Additional cache capacity or bandwidth helps only when profiling shows that the regularized request stream still exhausts that resource. More bandwidth cannot resolve a serial

[20] **Sparsity and memory irregularity**: Unstructured sparse formats commonly gather nonzero elements through indirect addressing, which can weaken sequential access and make metadata overhead significant. Structured sparsity preserves hardware-friendly blocks, so its access behavior differs from arbitrary pruning. Without suitable kernels or hardware support, an unstructured sparse model can run slower than its dense counterpart despite performing fewer arithmetic operations.

pointer dependency, and more capacity cannot rebalance skewed work. The objective is to expose as much regular structure as possible while confining unavoidable irregularity to the smallest portion of execution.

Software can mitigate irregular access by regularizing tensor layouts, enforcing structured sparsity, and sorting dynamic routes into uniform batches. Yet software restructuring cannot eliminate the physical penalty of data movement: whether a memory request is contiguous or scattered, every transferred byte must cross physical wires governed by finite bandwidth, propagation latency, and energy dissipation. Bounding these physical costs requires organizing storage into cascading hardware tiers. Rather than treating memory as a uniform pool, accelerator architectures deploy multilevel memory hierarchies that trade off capacity against latency and energy at every tier.

11.5.4 Memory hierarchy

No single storage technology can simultaneously satisfy the multi-terabyte-per-second bandwidth required by dense execution units and the tens of gigabytes of capacity required to hold modern model parameters. Accelerators resolve this tension through multilevel memory hierarchies. Rather than treating memory as a monolithic address space, the machine organizes storage into distinct physical tiers, each optimized for a specific balance of capacity, access latency, bandwidth, and energy dissipation.

At the apex of the hierarchy, register files and on-chip SRAM caches supply operands to arithmetic units with sub-nanosecond latency and single-digit picojoule energy costs, but their capacity is strictly limited by silicon die area. At the base, nonvolatile flash storage holds hundreds of gigabytes of model checkpoints and training datasets, but requires milliseconds or microseconds to access. Between these boundaries, software-managed scratchpads stage active tensor tiles, while device-local DRAM (such as HBM or GDDR) provides the primary working memory for parameters and activations.

This organization establishes an inverse relationship across tiers: as storage capacity expands and physical distance from arithmetic units grows, bandwidth falls, latency lengthens by orders of magnitude, and energy consumption per bit escalates. Table 11.8 compares these tiers across latency, bandwidth, capacity, and functional roles in deep learning workloads.

Table 11.8: Memory Hierarchy Trade-Offs: AI accelerators use a multilevel memory hierarchy to balance performance and capacity. Each level provides distinct latency, bandwidth, and capacity characteristics that dictate how neural network components (weights, activations, and intermediate results) should be allocated to minimize bottlenecks and maximize throughput. Relative labels describe the typical ordering; exact values depend on architecture. Section D.3.1 supplies registry-backed reference latencies.

Memory Level	**Relative latency**	Bandwidth	**Relative capacity**	**Example Use in Deep Learning**
Registers	Lowest	Highest	Tiny	Storing operands for immediate computation
L1/L2 Cache (SRAM)	Very low	High	Small	Caching frequently accessed activations and small weight blocks
Scratchpad memory	Very low	High	Small to medium	Software-managed storage for intermediate computations
Device DRAM (HBM, GDDR, or LPDDR)	High	Moderate–Very High	Large	Storing model parameters and activations that do not fit on-chip
Host DRAM (typically DDR)	High locally; transfer adds latency	Moderate	Large to very large	Staging or offloading data outside accelerator memory
Flash Storage (SSD/NVMe)	Very high	Low	Very large	Storing pretrained models and checkpoints for later loading

A recurring systems question is why accelerators cannot bypass on-chip SRAM entirely and stream all operands directly from high-capacity off-chip DRAM. A dependent DRAM request exposes the full access latency. Many independent requests can hide that latency, but their sustained operand supply remains limited by off-chip bandwidth; on-chip staging also enables reuse.

Napkin Math 11.2: Why HBM cannot feed the array directly

Problem: Why is on-chip SRAM necessary instead of fetching all data from HBM?

Math:

1. **Clock cycle**: At 2 GHz, $1/(2\text{ GHz}) = 500$ ps.
2. **DRAM access**: Off-chip HBM sits millimeters away on the package, but row activation, DRAM-array latency, serialization, protocol traversal, and queuing add up to 100+ cycles per access.
3. **Distance, for comparison**: On an H100-class 814 mm² die, signals travel ~20 mm; at $\approx 0.5c$, 20 mm$/(0.5c) \approx 130$ ps, less than one cycle.

Systems insight: The propagation estimate alone does not explain the DRAM access cost, and it omits real interconnect delay from wiring and routing. Independent requests can overlap the 100-plus-cycle access path; a dependent request cannot. Registers and SRAM supply operands at the rate the array needs while reusing values that would otherwise cross HBM repeatedly. Transformer tiling addresses both operand supply and reuse, whereas streaming a growing KV cache can still limit inference when its required traffic exceeds the available bandwidth.

11.5.4.1 On-chip memory

The **on-chip memory** subsystem integrates registers, Static Random-Access Memory (SRAM) caches, and software-managed scratchpads directly on the accelerator die. Register files provide the immediate operand and accumulator storage needed by execution pipelines. Modern accelerators provision substantial register capacity in aggregate across the chip, but partition this pool dynamically among concurrent threads or warps. Increasing per-thread register allocation allows individual threads to retain larger matrix tiles, but limits the total number of concurrently scheduled warps, reducing the hardware's ability to hide memory latency. Conversely, underallocating registers forces the compiler to spill values to off-chip memory, converting single-cycle register accesses into multi-cycle stalls. Efficient kernel design balances register allocation to maintain maximum occupancy while keeping hot operands resident.

Hardware-managed L1 and L2 caches provide fast buffering for frequently referenced data using automatic tag checking and hardware eviction policies. However, automatic caches are poorly suited for streaming multi-gigabyte tensor operations: scanning through large activation tensors can evict reusable weight tiles before arithmetic units can reuse them. Furthermore, specialized execution pipelines impose strict layout and data-type constraints that general caches cannot satisfy automatically. As illustrated in example 11.3, hardware matrix engines require specific alignments and memory contracts; a mismatch may require padding, conversion, or a different kernel whose efficiency must be checked.

Example 11.3: The Tensor Core contract

Scenario: An engineering team migrates a transformer workload to NVIDIA A100 GPUs expecting automatic Tensor Core acceleration, but profiling reveals Tensor Cores remain idle (NVIDIA Corporation 2020a).

Diagnosis: The workload used shapes and data types that selected less efficient kernels rather than the intended Tensor Core path.

Systems lesson: Specialized paths have operator, precision, layout, and shape requirements. Libraries can often handle nonaligned dimensions through padding or alternative kernels, but reaching high utilization requires verifying which path was actually selected.

To avoid the eviction overhead and unpredictability of automatic caches, AI accelerators incorporate scratchpad memory. Unlike hardware caches, scratchpads are software-managed SRAM structures mapped directly into the core's address space without tag directories or hardware replacement logic. The compiler or programmer explicitly controls when tensor tiles are loaded from

external DRAM, how long they reside on-chip, and when updated values are written back. In convolutional models, scratchpads retain input feature maps and weight kernels across sliding receptive windows, eliminating redundant DRAM reads (Chen et al. 2017). On NVIDIA GPUs, scratchpad memory is exposed as *shared memory*: a fast, programmer-controlled SRAM bank shared across all threads within a thread block. Custom kernels written in CUDA or Triton manage shared memory explicitly. FlashAttention demonstrates the power of this mechanism: rather than materializing the full $S \times S$ attention score matrix in off-chip HBM, the kernel tiles queries, keys, and values through shared memory and updates softmax statistics online (T. Dao et al. 2022). The speedup stems entirely from eliminating high-latency HBM round trips rather than reducing arithmetic operations; in fact, the algorithm recomputes intermediate attention values during the backward pass to avoid storing them to off-chip memory.

↩ FlashAttention as an IO-aware attention kernel for training is established in section 8.7.4.

11.5.4.2 Off-chip memory

When tensor working sets exceed on-chip SRAM capacity, operands and parameters reside in device-local DRAM. Rather than sequential hierarchy levels, HBM, Graphics DDR (GDDR), and Low-Power DDR (LPDDR) represent alternative memory architectures engineered for distinct cost, bandwidth, and power envelopes. High-performance accelerators use HBM, which stacks DRAM dies vertically using through-silicon vias (TSVs) and interfaces with the accelerator die across a wide silicon interposer (e.g., 1,024 to 8,192 bits). This wide bus delivers multi-terabyte-per-second bandwidth at lower clock frequencies and lower energy per bit than conventional DRAM, but requires complex 2.5D packaging. Where silicon interposers are cost-prohibitive, accelerators adopt GDDR, which uses narrower buses (e.g., 256 to 384 bits) driven at high pin speeds across standard circuit boards. Mobile and embedded accelerators use LPDDR, trading raw bandwidth for low operating voltage and reduced standby power.

When parameter and activation footprints exceed device-local DRAM capacity, the system must offload state to host DRAM (typically standard DDR). Accessing host memory requires crossing an I/O interconnect such as PCIe or a coherent fabric like NVLink or Compute Express Link (CXL). Because PCIe bandwidth (tens of gigabytes per second) is one to two orders of magnitude lower than accelerator DRAM bandwidth, unmanaged offloading stalls arithmetic execution. Efficient runtimes mitigate this penalty by overlapping asynchronous direct memory access (DMA) copies with kernel execution, prefetching upcoming layer parameters while current layers compute.

At the base of the hierarchy, nonvolatile flash storage and solid-state drives (SSDs) provide persistent, multi-terabyte capacity for datasets and training checkpoints (Narayanan et al. 2021). With read latencies measured in tens of microseconds and bus bandwidths capped at single-digit gigabytes per second, flash devices cannot participate directly in kernel execution. Checkpointing and data-loading pipelines treat flash storage as an asynchronous streaming source, staging batches through host DRAM into accelerator memory before computation begins.

The physical tiers of the memory hierarchy establish the latency floors and capacity ceilings of the accelerator. How closely an application approaches hardware peak depends on whether software can sustain high utilization across each tier: advertised memory bandwidth represents only a theoretical limit, while achievable throughput is determined by access regularity, tensor layout, and reuse locality.

11.5.5 Memory bandwidth and architectural trade-offs

Advertised memory bandwidth is only a ceiling; achievable bandwidth depends on access pattern, batching, locality, and the host interface that feeds the accelerator. Representative data center accelerators provide memory bandwidth on the order of a few TB/s, paired with tens of GB of high-bandwidth memory. Peak figures assume continuous, stride-1 coalesced access across all memory channels. Real neural network workloads deviate substantially from this ideal: transformer attention, convolution, and fully connected layers realize distinct fractions of peak bandwidth because their memory access strides, tiling dimensions, and operational reuse differ. A sufficiently large, coalesced weight stream can approach peak bandwidth even at small batch size. Batching instead reuses each loaded weight across more samples, raising arithmetic intensity as analyzed under the roofline model in section 11.6. Small or irregular kernels may lack enough concurrent accesses to approach the bandwidth ceiling, making memory latency and available parallelism part of the diagnosis.

Bandwidth also carries a steep thermal penalty: the reference DRAM access costs substantially more energy than the reference SRAM access of the same width (figure 11.8), with the ratio depending on storage tier and implementation. When off-chip accesses are frequent, driving those interfaces can dominate movement energy, motivating accelerators to structure their pipelines around spatial data reuse rather than repeatedly round-tripping tensors through external DRAM. Which operand an array keeps local (weights, inputs, or partial sums) is the stationarity choice that section 11.8.1.1 develops.

Physical constraints at the silicon and packaging boundaries govern how memory bandwidth scales across accelerator classes. The interposer-based HBM stacks described under off-chip memory reach on the order of 1 TB/s in mainstream products and a few TB/s in high-end systems, at substantially higher packaging cost and thermal density. TPU-class designs pair high-bandwidth memory with large on-chip buffers and 2D systolic arrays, trading general-purpose memory flexibility for tight spatial reuse on dense matrix multiplications. At the opposite extreme, mobile SoC designs operate within a few-watt power envelope and cannot accommodate the cost or thermal dissipation of interposer-based stacks. These edge processors rely on unified commodity LPDDR architectures, delivering on the order of hundreds of GB/s of shared bandwidth that must be strictly scheduled across CPU, GPU, and neural accelerator cores to prevent memory bus contention and thermal throttling.

Bandwidth limitations extend beyond the accelerator package to the host-device interface. When an input must cross a 64 GB/s PCIe link before using an accelerator with 2 TB/s of HBM bandwidth, the interface has roughly 30× less bandwidth and can dominate small, frequent transfers (NVIDIA Corporation 2020a, 2020c). For low-latency inference queries or fine-grained kernel offloading, transfer serialization across the host bus can easily outlast kernel computation. Mitigating this interface bottleneck requires keeping model parameters and intermediate activation states resident in accelerator memory across execution steps, or bypassing the host link entirely through direct peer-to-peer storage and network data paths.

11.5.6 Host-accelerator communication

In discrete accelerator systems, the host CPU and the accelerator operate as separate computational domains joined by a peripheral bus. The host manages storage I/O, network streams, and runtime orchestration, while the accelerator executes dense mathematical kernels over high-bandwidth device memory. Because CPU memory (system DRAM) and accelerator memory (HBM or GDDR) occupy distinct physical address spaces, computation cannot begin until tensors traverse the host-accelerator interconnect. Host-accelerator data movement therefore establishes an operational boundary: if transfers over the peripheral bus cannot feed the accelerator's compute cores at line rate, the device idles regardless of its raw FLOP/s capacity.

Figure 11.12 sequences the four-stage execution trace across host and device lifelines: copying input data across PCIe to device memory via DMA[21] (step 1), dispatching kernel instructions from the CPU (step 2), executing parallel core compute with local writeback (step 3), and copying results back to host memory (step 4). In a naive synchronous execution model, every transfer step sits directly on the critical path, stalling either the host or the accelerator until each memory transaction finishes.

The end-to-end latency of a kernel launch decomposes into transfer time, queue dispatch overhead, and arithmetic execution time. When tensor transfers are small or frequent, the fixed overhead of driver submission and bus round trips dominates runtime. When transfers are large, the low bandwidth of PCIe relative to on-package HBM creates a sustained I/O bottleneck. In D·A·M terms, the Machine boundary between host and accelerator forces the Algorithm to maximize arithmetic reuse on the device once tensors arrive.

[21] **DMA (direct memory access)**: A dedicated hardware unit that manages the data copy (step 1) without direct CPU management, freeing the CPU to immediately issue computation commands (step 2). This concurrency is critical: without it, the accelerator can idle between compute batches, especially when host-to-device movement is on the critical path.

11.5.6.1 Node-level interconnect topology

Within a single machine, how bits move between GPUs and the CPU sets the next ceiling. In the 1–8 GPU regime, scaling is achieved through high-speed intra-node interconnects such as NVLink and host-to-device PCIe transfers that mitigate the memory wall. These links form a **bandwidth taper**: data-movement speed falls at each step away from the compute units, from on-package HBM through the GPU-to-GPU NVLink bridge down to the host PCIe link. The PCIe step is much slower

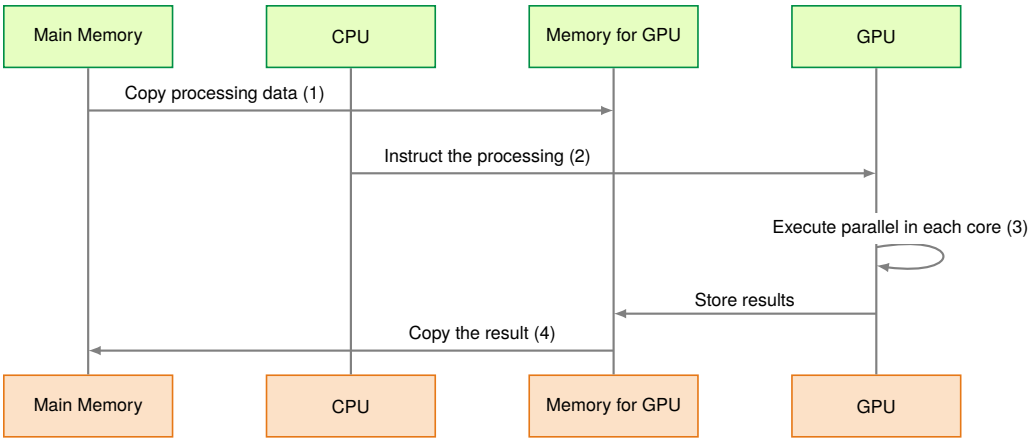


Figure 11.12: Host-Accelerator Data Transfer Sequence: Four-step execution flow between CPU host and GPU device: (1) copying input data over PCIe, (2) issuing kernel instructions, (3) GPU compute, and (4) transferring results back to host memory. Low PCIe bandwidth and host dispatch latencies can easily dominate runtime unless kernel compute time exceeds data transfer overhead.

than the accelerator-local memory and inter-GPU fabric, so any data path that touches the CPU can become a performance hazard, and keeping accelerator-to-accelerator traffic off that link is the reason NVLink exists (NVIDIA Corporation 2020c). A typical AI server is therefore not a flat mesh of connected devices, and optimizing data movement starts from its physical topology.

Within a multi-accelerator server node, three interconnect tiers define this bandwidth taper:

1. **Device-device interconnect (NVLink/Infinity Fabric)**: Modern multi-GPU nodes use specialized high-speed bridges like NVLink[22] to connect accelerators directly, bypassing the host CPU. Bandwidth ranges from 600 GB/s to 900 GB/s per GPU (NVIDIA Corporation 2020c; Choquette 2023). This link matters whenever tensors must move between accelerators within one server, including model partitioning, activation exchange, and gradient synchronization during training.[23] The architectural principle is the boundary: traffic that stays on the accelerator fabric is far cheaper than traffic that falls back through the host.
2. **Host-device interconnect (PCIe)**: The link between the CPU and the accelerator. Bandwidth ranges from 32 to 64 GB/s (PCIe Gen4/Gen5). Host-staged training data can bottleneck here, although resident data, direct storage or network I/O, and coherent accelerator links can use other paths. Multi-GPU servers may also provide multiple PCIe links rather than a single shared 64 GB/s path.
3. **Network interface card**: The link from the host to the outside world, connecting to other nodes. Bandwidth ranges from 25 to 50 GB/s (200 Gb/s to 400 Gb/s Ethernet/InfiniBand[24]). This interconnect is the first step from single-node hardware reasoning into the next frontier: scale. Traversing the network moves traffic onto the narrowest, highest-latency path in the system.

These three levels produce a characteristic bandwidth taper:

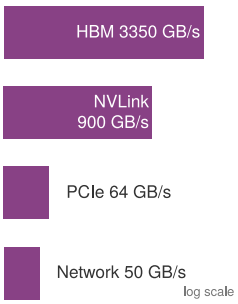
$$\text{HBM (3350 GB/s)} \gg \text{NVLink (900 GB/s)} \gg \text{PCIe (64 GB/s)} \gg \text{Network (50 GB/s)}$$

System efficiency depends on keeping tensors on the highest-bandwidth links available. Because PCIe and network links offer orders of magnitude less bandwidth and higher latency than on-package memory, placement and scheduling must minimize crossings across these boundaries.

11.5.6.2 Transfer optimization

Two complementary strategies mitigate transfer overheads across the host-accelerator interface: asynchronous data movement and unified memory abstraction.

In an asynchronous pipeline, software allocates pinned (page-locked) host memory and enqueues non-blocking memory copies onto dedicated DMA engines. Because copy engines operate indepen-



Bandwidth tapers steeply as data moves farther from the accelerator.

[22] **NVLink (NVIDIA Link)**: This direct GPU-to-GPU interconnect exists to keep accelerator-to-accelerator traffic off PCIe when tensors must move inside a server. Its 600–900 GB/s aggregate bidirectional bandwidth is roughly 300–450 GB/s per direction under balanced full-duplex traffic, several times the directional bandwidth of a standard PCIe link, so workloads with frequent cross-device tensor movement can remain in the fast part of the bandwidth taper (NVIDIA Corporation 2020c; Choquette 2023). Training algorithms determine how much tensor traffic must cross this link; the hardware fact needed here is the bandwidth gap.

dently of compute units, the next input batch can transfer across PCIe concurrently while the accelerator processes the current batch in local memory. This double buffering conceals transfer latency behind arithmetic runtime whenever computation time exceeds transfer time ($T_{\text{compute}} \geq T_{\text{transfer}}$). When compute duration is too short to amortize the transfer, execution remains bus-bound and compute units inevitably stall.

Unified Memory provides an alternative abstraction by presenting a single virtual address space spanning host DRAM and device HBM. Rather than requiring manual buffer allocation and explicit staging copies, the runtime driver migrates memory pages on demand via hardware page faults when either processor touches an address. While this eliminates boilerplate staging code, on-demand migration introduces severe performance unpredictability. Page faults trigger driver interrupts that stall execution pipelines, and scattered access patterns can cause page thrashing across the PCIe bus. Consequently, high-throughput production workloads rely on explicit, pipelined DMA transfers for deterministic performance, reserving Unified Memory for rapid prototyping or applications where programmer productivity outweighs sustained hardware utilization.

These interconnect and memory bounds dictate how neural network architectures map to physical silicon. Convolution exposes high spatial reuse that amortizes transfer costs through on-chip register and SRAM tiling. Dense self-attention exhibits regular matrix structure, yet quadratic sequence-length scaling creates intense pressure on package memory capacity and bandwidth. Sparse embeddings, mixture-of-experts routing, and autoregressive key-value caches present still other memory access patterns—shifting the binding constraint from arithmetic throughput to memory capacity, bus latency, or interconnect bandwidth.

11.5.7 Model memory pressure

Model architecture determines which memory term binds. Multilayer perceptrons (MLPs), CNNs, and transformers exhibit fundamentally different dataflow profiles: some are bound by weight capacity, others by intermediate activation footprint, device memory bandwidth, or host interconnect throughput. Consequently, each workload demands a distinct accelerator memory hierarchy and execution strategy.

The lighthouse models introduced in table 1.4 ground this analysis. ResNet-50 represents CNN workloads with high spatial reuse and heavy activation footprint, and GPT-2 and Llama exemplify transformer memory pressure spanning both parameter capacity and intermediate attention states; the subsections below follow these two families and the dense MLP layers that both contain. The other two lighthouses stress memory in ways this section does not cover in detail. DLRM's sparse embedding lookups stress random-access bandwidth rather than dense compute, and MobileNetV2's depthwise convolutions trade arithmetic intensity for a smaller parameter footprint; both reappear as memory-bound rows in the roofline operations table (table 11.11).

11.5.7.1 Multilayer perceptrons

An MLP layer is a dense matrix multiplication followed by optional bias, activation, and normalization. Its operational intensity depends directly on the batch size: at batch size $B = 1$, each parameter loaded from off-chip memory serves only a single multiply-accumulate operation, yielding low arithmetic intensity (often ≤ 1 FLOP/byte) that starves ALUs and leaves execution memory-bandwidth bound. Increasing the batch size reuses each loaded weight matrix across B independent input vectors, scaling arithmetic intensity until the matrix multiplication reaches the accelerator's compute-bound roofline ceiling.

Dense weight matrices regularly exceed the tens of megabytes available in on-chip SRAM, requiring parameters to reside in device-local DRAM (HBM or GDDR). The host interconnect (such as PCIe) enters the critical path only when streaming dynamic inputs or when parameters and activations exceed device memory capacity and require host offloading.

Because MLP memory access patterns are regular and predictable, accelerators tile matrix operations into blocks that fit within on-chip SRAM and register files. Hardware double-buffering prefetches subsequent weight and activation tiles from DRAM into SRAM concurrently with arithmetic execution on current tiles. Delivered throughput therefore depends on whether the batch size provides sufficient arithmetic intensity and whether matrix dimensions align cleanly with hardware tile boundaries (Chen et al. 2017).

[23] **AllReduce**: A collective operation from MPI that aggregates values across processes (the "reduce") and distributes the result back to every process (the "all"). Here, the term identifies why accelerator-to-accelerator bandwidth matters for distributed training workloads. At scale, algorithms, topology choices, and runtime protocols determine how costly this synchronization becomes.

[24] **InfiniBand**: Its key feature for multi-node scaling is RDMA (Remote Direct Memory Access). With a GPU-aware path such as GPUDirect RDMA and registered GPU memory, a network adapter can transfer data directly between GPUs across nodes without staging it through host memory. This reduces host involvement and protocol overhead, so scaling is more often constrained by physical bandwidth, topology, and collective implementation rather than CPU-managed packet processing.

↩ The feeding problem, in which data delivery fails to keep pace with compute throughput, is established in section 4.2.3.

11.5.7.2 Convolutional neural networks

A convolutional layer processes input feature maps using small filter kernels that slide across spatial dimensions. This localized sliding-window structure yields high arithmetic intensity through three distinct forms of data reuse: convolutional reuse (neighboring spatial windows share input activations), filter reuse (the same kernel weights evaluate across every spatial position), and channel reuse (input channels are evaluated across multiple output filters). Unlike MLPs, where weight matrices dominate memory footprint, early CNN layers feature compact parameter tensors but produce massive intermediate activation maps.

Because intermediate activation tensors frequently exceed on-chip SRAM capacity, unoptimized layer-by-layer execution would continuously spill feature maps to device DRAM, bottlenecking the pipeline on memory bandwidth. Accelerators prevent these round-trips through spatial tiling: dividing activations and filters into tiles dimensioned to fit within on-chip SRAM buffers (Chen et al. 2017). Staging overlapping boundary pixels directly in local memory allows adjacent convolutions to consume activations without refetching them across the device DRAM bus.

Specialized CNN accelerators configure their hardware dataflow around these reuse patterns. The Eyeriss architecture introduced a row-stationary dataflow that maps one-dimensional rows of filter weights and activations to processing engine registers, keeping filter rows and accumulating partial sums stationary in local storage to minimize energy-intensive interconnect traffic (Y.-H. Chen et al. 2016). In the broader taxonomy of section 11.8, this design exemplifies a hybrid dataflow that exploits multidimensional convolutional reuse across registers, SRAM buffers, and execution units.

11.5.7.3 Transformer networks

The transformer architecture replaces local sliding windows with global self-attention[25] and dense feed-forward blocks, creating dual memory bottlenecks: quadratic intermediate attention matrices and massive parameter footprints.

Global token interaction produces distinct memory challenges during training and inference. In standard multi-head attention, computing pairwise token affinity generates an $S \times S$ score matrix for sequence length S. Materializing this intermediate matrix in HBM repeatedly reads and writes $\mathcal{O}(S^2)$ data across the memory bus, saturating bandwidth even though matrix multiplication arithmetic units remain idle. Tiled attention algorithms, such as FlashAttention (T. Dao et al. 2022), resolve this bottleneck by staging query, key, and value blocks in on-chip SRAM, computing online softmax reductions locally, and streaming outputs directly to HBM without materializing the quadratic intermediate matrix in device DRAM.

In autoregressive inference, memory pressure shifts to the key-value (KV) cache. Generating each subsequent token requires reading the accumulated historical KV states from HBM for a single vector-matrix operation, operating at very low arithmetic intensity and binding generation speed directly to memory bandwidth.

When a large language model's weights and runtime state exceed the chosen accelerator's HBM capacity (Brown et al. 2020), capacity becomes a placement constraint. Distributing these models across multiple accelerators requires partitioning weights and activations across GPUs. Within a server node, high-bandwidth interconnects like NVLink provide the interconnect throughput necessary to sustain all-reduce and all-gather collectives during tensor and sequence parallelism, where standard PCIe links would throttle execution. System runtimes schedule asynchronous transfers to overlap these cross-accelerator communication phases with local matrix multiplication arithmetic (Narayanan et al. 2021).

11.5.8 Accelerator design implications

Workload-specific accelerator design is dictated by the memory and data-flow requirements of target neural network architectures. Within the D·A·M taxonomy, the algorithmic formulation directly shapes data movement patterns across the physical memory hierarchy, making a single rigid hardware architecture inefficient across different model classes. Table 11.9 compares the resulting tensor dimensions, reuse profiles, access regularities, and potential memory pressures across foundational model families under the stated workload conditions.

[25] **Attention mechanism**: Bahdanau, Cho, and Bengio introduced attention for sequence-to-sequence models; Transformer self-attention later allowed each token to attend to every token in the sequence (Bahdanau et al. 2015; Vaswani et al. 2017); materializing full attention scores for a sequence of length S requires an $S \times S$ matrix, producing quadratic intermediate storage. FlashAttention tiles the computation to avoid materializing that full matrix in HBM (T. Dao et al. 2022). The inference KV cache is separate: its capacity grows linearly with sequence length per layer, although each decode step reads an increasing cache.

↩ The transformer architecture, which pairs self-attention with feed-forward blocks and drops recurrence, is established in section 6.8.

↪ The KV cache's growth with sequence length and its claim on accelerator memory are developed in section 13.10.4.

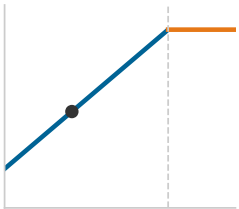
Table 11.9: ML Model Memory Access: Representative memory pressures vary with weight size, activation reuse, execution phase, batch size, and data movement; these rows are starting hypotheses rather than fixed bottleneck classifications. Standard dense transformers demand high bandwidth and capacity because large weights, KV caches, and attention traffic dominate memory pressure; sparse MoE or pruned variants add further routing and sparsity considerations. CNNs benefit from spatial locality and high activation reuse, reducing memory pressure.

Model type	Weight size	Activation reuse	Memory access pattern	Potential Memory Pressure
MLP (Dense)	Large, dense	Low	Regular, sequential (streamed)	Bandwidth at low batch
CNN	Small, reused	High	Spatial locality	Feature map movement
Transformer	Large, usually dense; sparse in MoE/pruned variants	Low-to-medium	Mostly regular GEMM plus KV-cache/attention traffic	Memory capacity + bandwidth

Each model architecture exercises the memory subsystem differently. Small-batch MLPs with weights streamed from device memory often face a bandwidth ceiling, as section 11.5.7.1 showed; batching turns matrix-vector products into matrix-matrix products with greater weight reuse. In CNNs, compact filter kernels slide over spatial feature maps, enabling high arithmetic reuse; accelerators exploit this spatial locality by staging feature tiles in on-chip static RAM (SRAM) or scratchpad memories to prevent redundant off-chip traffic. Transformers present distinct bottlenecks across execution phases: training must account for saved or recomputed activations and any inter-accelerator communication, whereas low-batch dense decoding often spends substantial time streaming weights and the expanding key-value (KV) cache from device memory.

Hardware architects address these divergent demands with multi-tier memory hierarchies. On-chip SRAM (caches and scratchpad memories) provides tens of terabytes per second of bandwidth at low latency, capturing high-reuse activations and tiled matrix blocks. HBM provides the capacity and throughput required to stage massive weight tensors and dynamic sequence states. When model scale exceeds the physical memory capacity of an individual accelerator, high-speed interconnects such as NVLink supply peer-to-peer inter-accelerator bandwidth for distributed tensor and pipeline parallelism, avoiding the narrower host PCIe bus.

Because a reference off-chip DRAM access costs far more energy per event than arithmetic (figure 11.8), reducing repeated external accesses can improve hardware efficiency. The workload's access and operation counts determine the total energy saving. Determining whether a given kernel execution is constrained by compute throughput or memory bandwidth requires an analytical boundary. The roofline model establishes this relationship by comparing operational intensity against hardware performance limits, so that optimization effort targets the resource that actually binds.



Low arithmetic intensity places the modeled throughput ceiling on the bandwidth slope.

11.6 Roofline Model

Every section so far has leaned on the roofline informally, calling a kernel compute bound or memory bound and pointing to the ridge point that separates the two. The compute units and memory systems that set its two ceilings are now in place, so the diagnosis can be made quantitative. Peak FLOP/s indicates what execution units can deliver when operands are already on chip, but sustained accelerator throughput is ultimately bounded by the rate at which the memory hierarchy delivers data. The Roofline Model[26] (Williams et al. 2009), the same model that sorted training kernels in section 8.4.4 and that section D.2.1 derives formally, plots attainable performance against arithmetic intensity and identifies the lower modeled ceiling, compute or memory bandwidth, for the chosen execution path. Rather than evaluating compute capacity in isolation, the model intersects hardware arithmetic throughput with memory bandwidth, bounding performance before profiling establishes which resource limits execution.

Performance is bounded by two ceilings, as equation 11.2 formalizes; it writes the bound of the arithmetic intensity law (principle 6) as an idealized equality that ignores launch and synchronization overheads. Here, attainable performance R_{attain} and peak compute R_{peak} are in FLOP/s (often reported as TFLOP/s), peak bandwidth BW is in bytes/s (often TB/s), and arithmetic intensity I is in FLOP/byte:

$$R_{\text{attain}} = \min(R_{\text{peak}}, \text{BW} \times I) \tag{11.2}$$

[26] **Roofline model**: Introduced by Williams et al. (2009) at UC Berkeley, building on earlier I/O complexity work from the 1980s. Their specific contribution was making the compute vs. bandwidth trade-off visual and actionable: the characteristic roofline plot immediately reveals whether a kernel is compute bound (hitting the flat ceiling) or memory bound (hitting the sloped bandwidth line) and quantifies the gap to hardware limits. A kernel operating at only 50 percent of its ceiling has 2× theoretical headroom, but instruction mix, latency, and occupancy may prevent it from closing that gap.

Definition 11.5: Arithmetic intensity

Arithmetic intensity is the ratio of operations to bytes transferred across the memory interface being modeled (FLOP/byte). Together with that level's bandwidth and the relevant compute peak, it determines which idealized roofline ceiling is lower.

1. **Significance**: The threshold between the two ceilings is the ridge point $R_{\text{peak}}/\text{BW}$, the hardware balance of section 11.5.2. For an A100 (312 TFLOP/s FP16/BF16, 2.04 TB/s), it is roughly 153 FLOP/byte. Under the stated traffic models, a large well-tiled 1024×1024 multiply reaches about 341.3 FLOP/byte, while standalone ReLU is about 0.125 FLOP/byte. They therefore meet different ceilings on the same hardware.
2. **Distinction**: Unlike total FLOPs (a count of operations), arithmetic intensity is a ratio that characterizes the shape of a workload's hardware demand. Two kernels with identical FLOPs but different memory access patterns can have different arithmetic intensities and face different modeled ceilings on the same hardware.
3. **Common pitfall**: A frequent misconception is that arithmetic intensity is a fixed property of an operation. In practice, it depends on implementation details: a naive matrix multiplication that reloads operands from DRAM for each output element has low arithmetic intensity; a blocked (tiled) implementation that reuses data from fast SRAM achieves high arithmetic intensity—the same mathematical operation, orders of magnitude apart in hardware efficiency.

With O as the operation count and D_{vol} as data volume in bytes, equation 11.3 makes the definition operational.

$$I = \frac{O}{D_{\text{vol}}} \tag{11.3}$$

The roofline visualization shows performance (TFLOP/s) on the vertical axis and arithmetic intensity (FLOP/byte) on the horizontal axis. At low arithmetic intensity, the upper bound increases linearly along the memory bandwidth slope ($\text{BW} \times I$). Above the ridge point, the upper bound is peak compute (R_{peak}). A kernel approaching the slope is bandwidth limited, while one approaching the flat roof is compute limited on the specified path. Merely lying below either line does not establish the cause; profiling explains that remaining gap. Section A.5.1 maps each regime to the optimizations that pay off and the ones that waste effort, so classifying a kernel tells the engineer whether more bandwidth, less traffic, or more arithmetic throughput is the investment to test first.

↪ The optimizations that pay off in each regime, and the ones that waste effort, are operationalized in section A.5.1.

11.6.1 Hardware ridge points

The ridge point, the hardware balance I_{ridge} established in section 11.5.2, is the arithmetic-intensity threshold at which a kernel crosses from memory bound to compute bound. Table 11.10 lists the peak rates and bandwidths behind three of the points in figure 11.10, so each ratio can be checked against its inputs:

Table 11.10: Hardware Ridge Points: Vendor-published FP16 peak throughput and memory bandwidth for three NVIDIA generations, and the ridge point each pair implies. The ridge rises from V100 to H100 because compute throughput grows faster than bandwidth; these values are hardware-specific rather than universal generation averages.

NVIDIA Accelerator	Peak FP16	Bandwidth	Ridge Point
V100 (2017)	125 TFLOP/s	0.9 TB/s	138.9 FLOP/byte
A100 (2020)	312 TFLOP/s	2.04 TB/s	153 FLOP/byte
H100 (2022)	989 TFLOP/s	3.35 TB/s	295.2 FLOP/byte

For a kernel whose reuse stays fixed, the rising ridge shows up as a ceiling on achievable utilization that falls with each generation.

Napkin Math 11.3: The utilization gap

Problem: Why is it harder to get 100 percent utilization on an H100 than a V100?

Metric: The ridge point $I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$ (FLOP/byte), the number of operations the hardware must perform for every byte loaded to keep its compute units busy.

Evolution: The ratios in table 11.10 climb from 138.9 FLOP/byte on the V100 through 153 FLOP/byte on the A100 to 295.2 FLOP/byte on the H100.

Result: The intensity needed to reach the tensor peak has increased. Under this simple HBM roofline, a kernel with $I = 200$ FLOP/byte lies above the A100 ridge but below the H100 ridge. If its intensity and implementation remain unchanged, its idealized gain is then closer to the 1.6× bandwidth ratio than the 3.2× peak-compute ratio.

Systems insight: A standalone ReLU under a one-read, one-write FP32 traffic model performs about 1 operation per 8 bytes, far below the H100 tensor ridge. A well-tiled 1024×1024 dense multiply reaches about 341.3 FLOP/byte under its idealized traffic model. Fusion is valuable for low-intensity chains because it can remove intermediate round-trips; it is not automatically the best optimization for every kernel.

Operations such as depthwise convolution, embedding lookup, LayerNorm, and softmax serve as low-intensity reference points that commonly fall below the HBM ridge on compute-dense accelerators under the stated traffic models. Table 11.11 maps common neural network operations to their representative roofline regimes and to the lighthouse workload that exercises each. Classification depends on the implementation, memory interface, and target path; profiling distinguishes a bandwidth ceiling from latency, limited parallelism, or other execution constraints.

Table 11.11: Operations on the Roofline: Neural network layers span a wide range of arithmetic intensities. Large, well-tiled convolutions and batched GEMMs can be compute-bound, while MobileNet depthwise layers, attention softmax, normalization, and DLRM embeddings are often memory-bound. To see how these intensity values translate into real performance predictions, a transformer layer provides a complete arithmetic-intensity calculation across its major sub-operations.

Operation	Arithmetic Intensity	**Classification**	**Lighthouse example**
Conv2D (Dense)	50–200 FLOP/byte	Straddles ridge; high-reuse cases compute-bound	ResNet-50
Dense MatMul (large batch, well-tiled)	~150–1,000+ FLOP/byte	Compute-bound once above the target's ridge	GPT-2 (batched projections)
Depthwise conv	10–20 FLOP/byte	Below modeled HBM ridge	MobileNet
Attention Softmax	Convention-dependent	Below modeled HBM ridge	GPT-2 (Generation)
LayerNorm	1–2 FLOP/byte	Below modeled HBM ridge	GPT-2/Llama
Embedding lookup	< 1 FLOP/byte	Below modeled HBM ridge	DLRM

Napkin Math 11.4: Transformer layer analysis

For a transformer with hidden dimension $d = 768$, batch size $B = 32$, sequence length $S = 512$, and $N_{\text{heads}} = 12$ attention heads:

Attention QKV projection:

- FLOPs: $2 \times 3 \times 32 \times 512 \times 768 \times 768 = 58$ GFLOP
- Bytes: (input + weights + output) = $(32 \times 512 \times 768 + 3 \times 768 \times 768 + 32 \times 512 \times 768 \times 3) \times 2 \approx 104.2$ MB
- $I = 58$ GFLOP / 104.2 MB = 556.4 FLOP/byte, which is compute bound on A100 (above the 153 FLOP/byte ridge point)

Softmax:

- FLOPs: $32 \times 12 \times 512 \times 512 \times 3 \approx 302$ MFLOP (exp, sum, div)

- Bytes: $32 \times 12 \times 512 \times 512 \times 2 \times 2 = 402.7$ MB
- $I = 302$ MFLOP / 402.7 MB = 0.75 FLOP/byte under this simple operation count, placing it below the reference tensor-path HBM ridge. Counting exponentials and division as multiple operations raises the numerical intensity, but those instructions use different execution units. The relevant vector, reduction, and special-function ceilings, together with profiling, determine whether bandwidth actually binds.

Systems insight: This analysis explains why FlashAttention focuses on reducing memory traffic in attention rather than reducing FLOPs.

The roofline supplies an initial resource bound; profiling determines which physical subsystem limits the kernel. When a kernel approaches its relevant off-chip bandwidth ceiling, optimization should focus on conserving memory traffic: fusing adjacent operators keeps intermediate activations in on-chip registers or shared memory (SRAM), lower-precision numeric formats reduce the physical byte count of memory transfers, and algorithmic refactorings like FlashAttention restructure access patterns to compute within SRAM tiles. When a kernel approaches the compute ceiling of its actual execution path, high data reuse keeps execution units supplied and off-chip bandwidth is no longer the binding resource. Performance here depends on saturating arithmetic execution pipelines: structuring matrix tiles to sustain Tensor Core throughput, hiding instruction latencies through software pipelining, and scaling batch sizes to distribute work across all available streaming multiprocessors.

↪ How a benchmark determines where a workload actually sits against its roofline ceiling is developed in section 12.3.2.

11.6.2 Calculating memory bandwidth bounds

The memory-bound region of the Roofline Model is governed by the accelerator's off-chip memory bandwidth. To sustain an operational throughput R_{ops} (FLOP/s) for a workload with arithmetic intensity I (FLOP/byte), the memory subsystem must transfer data at the rate defined by equation 11.4:

$$\text{BW}_{\text{req}} = \frac{R_{\text{ops}}}{I} \text{ bytes/s} \tag{11.4}$$

When this required bandwidth exceeds the hardware's peak memory bandwidth BW, memory transfers cap attainable throughput according to equation 11.5:

$$R_{\text{attain}} = \text{BW} \times I \tag{11.5}$$

Evaluating an operation against its execution path's ridge point identifies the lower modeled ceiling. Extensive data reuse can raise arithmetic intensity above that ridge, making compute the tighter bound; profiling must establish whether execution approaches it. A 2D convolution demonstrates this behavior.

Napkin Math 11.5: Convolutional layer analysis

Consider a Conv2D layer with input shape (batch = 32, channels = 128, height = 56, width = 56), output channels = 256, and kernel size 3×3 on an A100 GPU:

Computational requirements:

- Output size: $32 \times 256 \times 56 \times 56 = 25.7$M elements
- FLOPs per output: $128 \times 3 \times 3 \times 2 = 2{,}304$ (multiply-add)
- Total FLOPs: 25.7M × 2,304 = 59.2 GFLOP

Memory traffic analysis:

- Input: $32 \times 128 \times 56 \times 56 \times 2 = 25.7$ MB (FP16)
- Weights: $256 \times 128 \times 3 \times 3 \times 2 \approx 0.6$ MB (FP16)
- Output: $32 \times 256 \times 56 \times 56 \times 2 = 51.4$ MB (FP16)
- Total: 77.7 MB

Arithmetic intensity: I = 59.2 GFLOP / 77.7 MB = 762.2 FLOP/byte

Systems insight: This is well above A100's ridge point of 153 FLOP/byte, so the simple Roofline Model places the operation in the compute-limited region. Peak throughput of ~312 TFLOP/s (FP16 with Tensor Cores) is therefore the ceiling, but realized performance still depends on tiling, occupancy, instruction mix, and library efficiency.

The convolutional layer's high arithmetic intensity arises from spatial weight reuse: the same 3×3 kernel is applied across every spatial location in the feature map, amortizing the cost of loading weights from off-chip memory across millions of multiply-accumulate operations. Keeping active tiles in registers and shared memory reduces repeated off-chip transfers, making high arithmetic throughput possible. Whether a particular kernel then saturates the Tensor Cores depends on tiling, occupancy, and instruction mix, which the roofline does not model.

Not all neural network layers exhibit such extensive data reuse. Fully connected (dense) layers—such as classification heads, feed-forward networks, and linear projections in transformer attention blocks—lack spatial dimensions. When evaluated at small batch sizes, their arithmetic intensity drops substantially, shifting execution into the memory-bound regime:

Napkin Math 11.6: Dense layer analysis

Consider a fully connected layer: input (batch = 32, features = 2048) $\rightarrow$ output (batch = 32, features = 2048) on the same A100:

Computational requirements:

- Matrix multiply: $(32 \times 2048) \times (2048 \times 2048)$
- Total FLOPs: $2 \times 32 \times 2048 \times 2048$ = 268.4 MFLOP

Memory traffic analysis:

- Input: $32 \times 2048 \times 2$ = 131.1 KB (FP16)
- Weights: $2048 \times 2048 \times 2$ = 8.4 MB (FP16)
- Output: $32 \times 2048 \times 2$ = 131.1 KB (FP16)
- Total: 8.7 MB

Arithmetic intensity: I = 268.4 MFLOP / 8.7 MB = 31 FLOP/byte

This intensity sits below A100's ridge point of 153 FLOP/byte, making this operation memory bound under the stated traffic model. Attainable performance: R_{attain} = 2,039 GB/s $\times$ 31 FLOP/byte = 63.3 TFLOP/s

Systems insight: Attainable throughput is only 20.3 percent of peak compute capability, so at batch 32 the memory wall caps this layer long before the Tensor Cores do.

The dense layer's lower arithmetic intensity stems from limited parameter reuse. While a convolutional kernel is reused across every spatial location in an activation map, a dense weight matrix is reused only across the batch dimension. At a batch size of 32, each fetched weight contributes to 32 sample products in this operation. When the weights stream from HBM on each invocation, as modeled, the bandwidth ceiling caps throughput at 20.3 percent of peak compute capacity. The fraction is a bound rather than a measured stall count, and a layer whose weights stay cached between invocations would do better. The same arithmetic explains why transformer inference at low batch sizes is governed by memory bandwidth rather than arithmetic capability, a regime the throughput-ceiling notebook below quantifies for GPT-2.

While small-batch matrix multiplications suffer from restricted parameter reuse, elementwise and reduction operators represent an even more acute memory bottleneck. Operations such as layer normalization, activation functions, and residual additions perform only a few arithmetic operations per element loaded from memory, driving arithmetic intensity to its physical floor:

Napkin Math 11.7: LayerNorm analysis

LayerNorm with input shape (batch = 32, seq = 512, hidden = 768):

Computational requirements:

- Elements: $32 \times 512 \times 768 = 12.6\text{M}$
- Approximate operations per element: mean reduction (1 ADD), variance (2 ADD, 1 MUL), normalization and affine transform (2 ADD, 2 MUL) ≈ 8; reciprocal-square-root overhead is amortized across each hidden vector
- Total FLOPs: $12.6\text{M} \times 8 = 100.7$ MFLOP

Memory traffic:

- Input: $12.6\text{M} \times 2 = 25.2$ MB
- Parameters (scale, bias): $768 \times 2 \times 2 = 3.1$ KB (negligible)
- Output: $12.6\text{M} \times 2 = 25.2$ MB
- Total: 50.3 MB

Arithmetic intensity: $I = 100.7$ MFLOP / 50.3 MB = 2.0 FLOP/byte

This arithmetic intensity sits severely below the A100 ridge point (77× below). The HBM performance upper bound is: $R_{\text{attain}} = 2039$ GB/s $\times$ 2.0 FLOP/byte = 4.1 TFLOP/s

Systems insight: The simple HBM roofline bounds this standalone LayerNorm at about 1 percent of A100's tensor peak. That tensor peak is a reference, not the throughput available to LayerNorm's vector and reduction instructions. Actual latency also depends on their execution ceilings, fusion, launch overhead, and which memory level supplies the data.

Standalone normalization performs minimal arithmetic relative to the memory traffic it generates, as the LayerNorm analysis demonstrates. Under the modeled unfused HBM read/write boundary, the implementation performs approximately 2 FLOP/byte, giving a low bandwidth ceiling relative to the tensor peak. Reaching that ceiling still requires enough parallel work and an efficient reduction. Cache residency changes the physical HBM byte count and can shift the relevant bound to another memory interface. In the D·A·M framework, when the machine constraint is a memory bandwidth wall, the algorithmic response is kernel fusion, which collapses adjacent projections, bias additions, normalization, and activations into one kernel so that intermediate values stay in registers or SRAM. Fusion pays off when the grouping preserves numerical behavior and fits the local register and SRAM budget, and profiling confirms that the saved traffic outweighs any loss of occupancy.

11.6.3 Optimization by intensity regime

The roofline analysis directly informs optimization priorities, summarized in table 11.12.

Table 11.12: Optimization by Arithmetic Intensity Regime: Position relative to the target execution path's ridge guides compute and traffic optimization. Profiling confirms the bottleneck; fusion removes intermediate round-trips when dependencies and local storage permit it.

Intensity regime	Typical operations	Optimization priority	Common techniques	Expected impact
Above the target path's ridge	Large convolutions	Maximize compute utilization.	Tensor Cores, thread-block tuning, and high occupancy.	Raises sustained compute utilization when the kernel already clears the ridge.
Near the target path's ridge	Medium-sized dense layers	Balance compute and memory optimization.	Larger batches, register tiling, and fusion with adjacent operations.	Can move the binding bottleneck between memory and compute.
Below the target path's ridge	Small dense layers and element-wise operations	Reduce memory traffic.	Aggressive kernel fusion, reduced precision (FP16 → INT8), and algorithmic changes.	Reduces external-memory traffic when fusion or layout changes are legal.

Intensity regime	Typical operations	Optimization priority	Common techniques	Expected impact
Low-intensity operator chains	Normalization layers and activation functions	Eliminate memory round-trips.	Fuse with adjacent operations and use in-place computation where legal.	Removes intermediate round-trips when adjacent operations can be fused.

For low-intensity operations, kernel fusion is often the decisive optimization: LayerNorm combined with the Gaussian Error Linear Unit (GELU), for example, can become a single fused kernel. One of the most accessible levers for moving an operation up and right on the roofline is batching.

Napkin Math 11.8: Batch size and arithmetic intensity

Increasing batch size improves arithmetic intensity for matrix operations by amortizing weight loading. Equation 11.6 formalizes this relationship for an idealized FP16/BF16 dense layer $(B \times d_{\text{in}}) \times (d_{\text{in}} \times d_{\text{out}})$, counting one read each of the input and weights and one write of the output, where B is batch size and d_{in} and d_{out} are the layer's input and output widths:

$$I = \frac{2Bd_{\text{in}}d_{\text{out}}}{2Bd_{\text{in}} + 2d_{\text{in}}d_{\text{out}} + 2Bd_{\text{out}}} \approx \frac{2Bd_{\text{in}}d_{\text{out}}}{2d_{\text{in}}d_{\text{out}}} = B \quad (\text{when } 2d_{\text{in}}d_{\text{out}} \gg 2B(d_{\text{in}} + d_{\text{out}})) \tag{11.6}$$

Example: Dense layer with $d_{\text{in}} = d_{\text{out}} = 2048$ (FP16)

- Batch = 1: $I \approx 1$ FLOP/byte (memory bound)
- Batch = 32: $I \approx 31$ FLOP/byte (memory bound)
- Batch = 256: $I \approx 204.8$ FLOP/byte (compute bound on A100)

Systems insight: This explains why batching can produce large throughput improvements in production inference systems, as MLPerf Inference, the standardized benchmark suite, demonstrates by separating bulk-throughput runs from latency-constrained serving runs (Reddi et al. 2019).

The batch size analysis reveals why inference serving systems are designed around batching: it changes the arithmetic intensity regime of memory-bound workloads. However, batching introduces latency trade-offs, since requests must wait in a queue until a batch forms. This tension between throughput (favoring large batches) and latency (favoring small batches) is a central challenge in ML serving systems, explored in depth in section 13.7.4.

↪ The MLPerf Inference execution scenarios that separate throughput runs from latency-constrained runs are developed in section 12.8.4.2.

When service latency limits batching, batch-1 or small-batch LLM decode often retains low weight reuse and low arithmetic intensity. The next worked model quantifies that particular regime rather than treating all LLM inference as identical.

Napkin Math 11.9: The throughput ceiling

Problem: A batch-1 GPT-2 calculation makes this bandwidth ceiling concrete: what is the maximum possible utilization of an NVIDIA A100 when running GPT-2 inference (batch size 1)?

The hardware constraints (the denominators)

- **Peak compute**: 312 TFLOP/s (FP16 Tensor Core).
- **Peak bandwidth**: 2.04 TB/s (HBM2e).
- **Ridge point** ($R_{\textbf{peak}}/\textbf{BW}$): 312 TFLOP/s / 2.04 TB/s = 153 FLOP/byte (for FP16 Tensor Core).

  Interpretation: Saturating this chip at FP16 precision requires 153 FLOP/byte operations for every byte loaded. The ridge point varies by precision: FP32 operations (19.5 TFLOP/s peak) have a ridge point of only ~9.6 FLOP/byte.

The workload characteristics (the numerator)

- **Model**: GPT-2 XL (1.5 billion parameters).
- **Operation**: Autoregressive generation (1 token at a time).
- **Data movement**: Must load all weights (3 GB @ FP16) for every token.
- **Compute**: Vector-Matrix multiplication. Two FLOPs per parameter, $2P \approx 3$ GFLOP.
- **Arithmetic intensity**: 3 GFLOP / 3 GB = 1 FLOP/byte

The prediction (iron law)
Since the modeled intensity I (1) $\ll$ the ridge point I_{ridge} (153 FLOP/byte), batch-1 decode is memory bound.

- **Maximum throughput**: 1 FLOP/byte $\times$ 2.04 TB/s = 2.04 TFLOP/s.
- **Utilization ceiling**: 2.04 TFLOP/s (Bound) / 312 TFLOP/s (Peak) $\approx 0.7\%$

Systems insight: Without batching, a $15,000 GPU cannot exceed 1 percent of its compute peak in this weight-streaming decode scenario. Batching and weight quantization address that gap; key-value caching instead avoids recomputing prior attention states.

Through this derivation, the Roofline Model provides a diagnostic framework for identifying whether operations are compute bound or memory bound. Knowing that batch-1 decode is memory bound at a ceiling of 0.7 percent of peak is only the first step; the next challenge is translating this diagnosis into efficient execution plans that exploit accelerator architectures.

11.7 Hardware Mapping

Consider a 3×3 convolution running on an accelerator tile. The mathematical operation is fixed, but the execution plan is not. One schedule can keep a filter in local registers while many output pixels stream past it; another can advance pixel by pixel and reload the same filter values repeatedly from a slower memory tier. Both schedules compute the same tensor. Only one turns the reuse in the convolution into real bandwidth savings.

Definition 11.6: Mapping in AI acceleration

Mapping in AI acceleration is the accelerator-compiler process of binding the logical computation graph to the physical hardware topology by deciding which operations execute on which processing elements, which data resides in which memory tier, and in what temporal order.

1. **Significance**: Within the D·A·M taxonomy, mapping is a machine-axis decision that helps determine how closely an operation approaches the roofline bound $\min(R_{\text{peak}}, \text{BW} \times I)$. A poor tiling choice that forces unnecessary DRAM accesses can reduce effective arithmetic intensity enough to move a nominally compute-bound operation into the bandwidth-bound regime.
2. **Distinction**: Traditional compilation often emphasizes instruction selection, register allocation, and scheduling for general-purpose processors. Accelerator mapping gives equal prominence to spatial placement and explicit data movement across a dataflow architecture. The reason is physical: as quantified by the storage-tier energy ladder in figure 11.8, an off-chip DRAM access costs about 173× the energy of an FP32 multiply, making operand placement as critical to efficiency as the execution unit that evaluates it.
3. **Common pitfall**: A frequent misconception is that mapping is automatically handled by frameworks. For general GPU workloads, compilers such as Accelerated Linear Algebra (XLA) can find strong mappings for common kernels; for specialized accelerators (systolic arrays, custom ASICs), compiler-generated mappings may still lag hand-tuned schedules because the compiler's search space is limited by the time budget at compilation.

The convolution example exposes the three decisions that recur throughout accelerator compilation. Placement assigns the multiply-accumulate work to processing elements so parallelism does

not turn into idle time or interconnect congestion. Allocation keeps weights, activations, and partial sums in the memory tier where their next use will occur, rather than letting reuse spill back to DRAM. Scheduling orders loops and kernels so the chosen placement and allocation remain valid over time. A poor choice in any one dimension can collapse a high-arithmetic-intensity operation back into a bandwidth-bound execution. In practice, these choices are too coupled for developers to manage by hand at model scale, which is why systems such as XLA, TVM, and TensorRT lower high-level models and search or select execution plans within their compile-time and hardware budgets. Section 11.9 examines that compiler support in detail.

11.7.1 Placement and allocation

Translating a model's computational graph into efficient hardware execution requires solving two tightly coupled problems. Computation placement determines which operations run on which processing elements, balancing parallelism against communication costs. Memory allocation determines where data resides within the memory hierarchy, trading capacity against access latency. These two decisions interact: placing operations on distant processing elements increases the memory bandwidth required to shuttle data between them, while allocating data to fast but small on-chip memory limits which operations can execute concurrently. Getting either wrong leaves thousands of processing elements idle or starved for data.

11.7.1.1 Computation placement

Computation placement is the process of assigning operations to an accelerator's processing elements (PEs) to expose parallelism, limit idle time, and reduce unnecessary data movement. Modern accelerators contain many such resources: the NVIDIA H100 has more than 16,000 CUDA cores and more than 500 Tensor Cores (Choquette 2023), TPUs organize thousands of multiply-accumulate units into systolic arrays (Jouppi et al. 2017), and wafer-scale processors such as Cerebras' CS-2 integrate more than 850,000 cores (Systems 2021). At these scales, placement inefficiencies become measurable because idle cores and redundant transfers waste both time and energy.

The difficulty of placement depends on workload regularity. Convolutional networks and dense transformer operations possess regular access patterns that tile predictably, though transformer architectures mix operations with distinct tensor shapes and reuse profiles. Graph neural networks (GNNs) present greater difficulty: sparse, input-dependent neighborhood aggregations introduce load imbalance and non-uniform communication across processing elements. Table 11.13 summarizes these core placement challenges and their operational impact.

Table 11.13: Computation Placement Challenges: Effective neural network deployment requires strategic allocation of computations to processing elements, balancing workload distribution, data movement costs, and hardware constraints to maximize execution efficiency. These challenges guide the design of mapping strategies that optimize resource utilization and minimize communication overhead.

Challenge	Impact on execution	Key Considerations for Placement
Workload imbalance	Some processing elements finish early while others remain overloaded, leading to idle compute resources.	Distribute operations evenly to prevent stalls and ensure full utilization of PEs.
Irregular computation patterns	Sparse graphs, routing, and variable shapes can create nonuniform work that complicates static placement.	Use adaptive placement when workload characteristics vary at run time.
Excessive data movement	Frequent memory transfers introduce latency and increase power consumption.	Keep frequently used data close to the compute units and minimize off-chip memory accesses.
Limited interconnect bandwidth	Poorly placed operations can create congestion, slowing data movement between PEs.	Optimize spatial and temporal placement to reduce communication overhead.
Model-specific execution needs	CNNs, transformers, and GNNs require different execution patterns, making a single placement strategy ineffective.	Tailor placement strategies to match the computational structure of each model type.

Static spatial mappings suffice for regular dense operations whose shapes and loop bounds are known ahead of time. Workloads with dynamic shapes, sparsity, or mixture-of-experts routing instead require runtime scheduling to balance execution across processing elements. Yet even an optimal computation placement stalls if the required operands are not staged in time: compute units can execute only as fast as operands arrive from the memory hierarchy.

11.7.1.2 Memory allocation

While computation placement determines where operations execute, memory allocation defines where data resides and how it flows through the hierarchy. The goal is to keep reused data near the processing elements without exceeding faster-tier capacity. GPUs expose global memory, shared memory, caches, and registers that kernels coordinate through tiling (NVIDIA Corporation 2020a). TPUs use on-chip SRAM buffers to stage horizontal activations and vertical weights for the 2D systolic-array dataflow diagrammed in figure 11.7, while wafer-scale processors partition memory and computation to control interconnect traffic (Systems 2021). General-purpose processors and accelerators both use memory hierarchies, but many accelerators expose more placement decisions to compilers and kernels. When allocation fails to exploit locality, processing elements stall waiting for operands, driving down utilization and dissipating energy across the memory bus.

The severity of these penalties varies by workload. Convolutional networks rely on structured, localized access patterns and benefit from well-defined memory layouts that facilitate predictable reuse (Y.-H. Chen et al. 2016). Transformers require access to large parameter sets and intermediate activations, making them sensitive to capacity and bandwidth. Graph neural networks add irregular sparse structures that complicate allocation and prefetching.

Capacity is only the first allocation test. A tensor can fit in accelerator memory yet still dominate execution if each kernel rereads it from a distant tier; conversely, tiling can keep a larger working set efficient by preserving reuse locally. The allocation diagnostic is therefore traffic, not capacity alone: compilers must track how frequently values cross each memory boundary and keep high-reuse tensors in the fastest feasible tier. Systems combine static memory plans for known shapes with dynamic buffer management when shapes vary, while device capacity limits which models fit without partitioning or offload. Table 11.14 summarizes these allocation challenges across the hierarchy.

Table 11.14: Memory Allocation Challenges: Efficient memory management in AI accelerators balances data access speed with hardware constraints, mitigating performance bottlenecks caused by latency, bandwidth limitations, and irregular data patterns. Complex models such as transformers and graph networks impose variable and demanding memory requirements that amplify these challenges.

Challenge	Impact on Execution	**Key Considerations for Allocation**
High memory latency	Slow data access delays execution and reduces throughput.	Prioritize placing frequently accessed data in faster memory locations.
Limited on-chip storage	Small local memory constrains the amount of data available near compute units.	Allocate storage efficiently to maximize data availability without exceeding hardware limits.
High off-chip bandwidth demand	Frequent access to external memory increases delays and power consumption.	Reduce unnecessary memory transfers by carefully managing when and how data is moved.
Irregular memory access patterns	Some models require accessing data unpredictably, leading to inefficient memory usage.	Organize memory layout to align with access patterns and minimize unnecessary data movement.
Model-specific memory needs	Different models require different allocation strategies to optimize performance.	Tailor allocation decisions based on the structure and execution characteristics of the workload.

Example 11.4: Loop ordering in a small convolution

Consider a convolution applying 16 filters of size 3×3 to an 8×8 single-channel input. The computation can be expressed as five nested loops iterating over output rows (H_{out}), output columns (W_{out}), filter count (d_{out}), filter height (K_h), and filter width (K_w). The $5! = 120$ permutations are an unconstrained upper bound: legal orderings preserve the mathematical result, although floating-point reduction order can change rounding, and they generate different memory traffic.

Ordering A (weight-stationary): Place the filter loops (d_{out}, K_h, K_w) outermost and the spatial loops (H_{out}, W_{out}) innermost. Each 3×3 filter is loaded into registers once and then applied across all 36 output positions before the next filter is loaded. Total weight loads: $16 \times 9 = 144$ values, each loaded exactly once.

Ordering B (output-stationary): Place the spatial loops outermost and the filter loops innermost. For every output position, all 16 filters must be loaded, applied, and their partial sums accumulated before advancing to the next position. If the register file cannot hold all 16 filters simultaneously, filters are repeatedly fetched from cache or DRAM. In the worst case, each of the 36 output positions reloads all 144 filter weights, producing $36 \times 144 = 5{,}184$ weight reads.

Systems insight: Under these two limiting assumptions, Ordering A reduces modeled weight reads by $36\times$ relative to the worst case for Ordering B. Real kernels also account for activation and partial-sum traffic, register capacity, cache behavior, and vectorization. The example nevertheless shows why two mathematically equivalent loop orders can place very different demands on the memory system.

11.7.2 Combinatorial complexity

The small convolution example illustrates why hardware mapping becomes a combinatorial search problem. Keeping a filter resident in local storage improves reuse only if the assigned processing elements have sufficient storage and if the loop schedule revisits that filter before eviction. Similarly, parallelizing across more processing elements improves throughput only until synchronization overhead and interconnect contention erode the benefit. Because changing one decision alters the feasible and efficient choices for the others, compilers must evaluate placement, allocation, and scheduling jointly rather than independently. Table 11.15 summarizes these recurring trade-offs across the execution stack.

Table 11.15: Placement-Allocation-Scheduling Trade-Offs: AI accelerator performance depends on mapping computations to hardware, allocating data to memory tiers, and scheduling execution over time. Careful consideration of these interdependent factors is essential for maximizing throughput and minimizing energy consumption.

Dimension	**Placement considerations**	**Allocation and Scheduling Considerations**
Computational granularity	Fine-grained placement enables greater parallelism but increases synchronization overhead.	Coarse-grained scheduling reduces synchronization overhead but may limit flexibility.
Spatial vs. Temporal Mapping	Spatial placement enhances parallel execution but can lead to resource contention and memory congestion.	Temporal scheduling balances resource sharing but may reduce overall throughput.
Memory and Data Locality	Placing data closer to compute units minimizes latency but may reduce overall memory availability.	Allocating data across multiple memory levels increases capacity but introduces higher access costs.
Communication and synchronization	Co-locating compute units reduces communication latency but may introduce contention.	Scheduling synchronization mechanisms mitigates stalls but can introduce additional overhead.
Dataflow and Execution Ordering	Static placement simplifies execution but limits adaptability to workload variations.	Dynamic scheduling improves adaptability but adds scheduling complexity.

Evaluating these trade-offs requires navigating an unconstrained search space governed by three structural decisions: loop execution order, spatial parallelization across processing elements, and memory allocation across the hierarchy. To understand why exhaustive search is impractical, consider the combinatorial growth of each dimension independently.

11.7.2.1 Ordering computation and execution

Machine learning workloads are typically expressed as nested loops that iterate over tensor dimensions. For instance, a matrix multiplication kernel loops over batch size (B), input features (d_{in}), and output features (d_{out}). The loop execution order dictates which tensor elements remain resident in registers or SRAM between iterations, directly determining data locality and reuse.

Ignoring dependences and equivalent orderings, the number of ways to arrange n_{loops} loops has the toy upper bound:

$$N_{\text{order}} = n_{\text{loops}}!$$

which scales rapidly. A typical convolutional layer may involve up to seven loop dimensions, leading to:

$$7! = 5{,}040 \text{ possible execution orders.}$$

If each memory level could choose an ordering independently, the corresponding toy upper bound would expand as:

$$(n_{\text{loops}}!)^{N_{\text{mem}}}$$

where N_{mem} is the number of memory hierarchy levels. This rapid expansion shows why execution order optimization matters: suboptimal loop ordering evicts values prematurely and multiplies DRAM traffic, whereas an optimized ordering maximizes reuse in local SRAM and caches (Sze et al. 2017).

11.7.2.2 Parallelization across processing elements

Modern AI accelerators use thousands of processing elements to execute loop iterations concurrently. Parallelizing an inner reduction loop, however, requires cross-core synchronization and collective communication, whereas parallelizing outer independent loops (such as batch or output feature channels) allows each core to execute independently without interconnect traffic. Determining which loops to map spatially requires careful partitioning.

Before legality and capacity constraints are applied, the number of ordered ways to select loops for parallel execution has the upper bound:

$$\mathcal{P}_{\text{parallel}} = \frac{n_{\text{loops}}!}{(n_{\text{loops}} - k_{\text{parallel}})!}$$

where n_{loops} is the number of loops, and k_{parallel} is the number selected for parallel execution. For a six-loop computation where three loops are selected, this unconstrained count is:

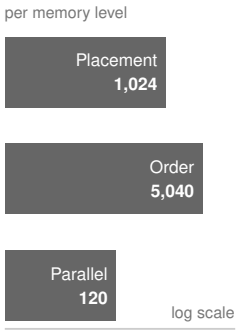
$$\frac{6!}{(6-3)!} = 120.$$

Even for a single layer, the candidate count reaches hundreds before invalid or equivalent strategies are removed. Each candidate schedule imposes different demands on barrier synchronization, interconnect bandwidth, and PE utilization.

11.7.2.3 Memory placement and data movement

The physical memory hierarchy forces compilers to determine which tensors reside in registers, shared memory, or global DRAM at each stage of execution. Because an off-chip DRAM fetch costs roughly 128× more energy than a read from an 8 KB SRAM array (figure 11.8), and more still than a small register-adjacent L1 access, data placement directly dictates the achieved arithmetic intensity.

Unlike loop ordering, memory placement cannot be chosen independently for each tensor dimension. Physical capacity sets a hard boundary: an operand tile can remain resident in fast on-chip SRAM only if its working set fits within the allocated buffer without spilling back to DRAM. Placing larger tiles in local storage increases data reuse and amortizes transfer costs across more arithmetic operations, but it also consumes scarce on-chip capacity and increases register pressure, potentially reducing thread concurrency or forcing spilling to slower memory tiers.



Per memory level, from the section's separate toy examples: $4^5 = 1{,}024$ placements, $7! = 5{,}040$ orders, $6!/3! = 120$ parallelizations. The product is raised to the number of levels, so three levels give 4^{15} placements alone.

11.7.2.4 Mapping search space

These structural choices—loop ordering, spatial parallelization across processing elements, tile sizing, and memory placement—are deeply coupled. Choosing an output-stationary schedule, for example, pins accumulators in local registers and dictates which loops can be mapped spatially across processing elements without requiring inter-core reduction networks. Conversely, parallelizing across more execution units reduces the per-core SRAM budget available for operand staging, forcing smaller tile dimensions and increasing off-chip memory traffic.

While the unconstrained Cartesian product of these decisions suggests a massive combinatorial space, hardware capacity limits, data dependencies, and symmetry prune the vast majority of candidates as either physically illegal or strictly dominated. Exhaustive search across arbitrary loop

permutations and placement choices is therefore both unnecessary and intractable. Production compilers and kernel developers manage this complexity by structuring their search around a small family of canonical dataflow patterns—spatial and temporal schedules engineered to exploit the specific reuse properties of neural network operators.

11.8 Dataflow Optimization

The loop-ordering case in example 11.4 showed two mathematically identical schedules that differ 36-fold in weight reads, and every avoided read that would have reached DRAM saves an access costing about 173× the energy of the FP32 multiply it feeds. While spatial and temporal mapping establish where operations execute and where operands reside, **dataflow optimization** governs the exact trajectories and schedules of data moving through processing elements and the memory hierarchy to maximize operand reuse. In a systolic array or tiled vector unit, the temporal schedule and spatial routing of weights, inputs, and partial sums dictate how frequently operands must be fetched from energy-expensive DRAM rather than reused from local registers or scratchpads. By controlling off-chip traffic, the dataflow establishes the effective operational intensity of the kernel, determining whether execution achieves the compute-bound plateau or stalls in the memory-bound regime of the Roofline model. Optimizing compilers (section 11.9) and runtime systems (section 11.10) therefore tailor dataflow patterns to match the reuse characteristics of each layer to the hardware constraints of the target accelerator.

Three recurring architectural decisions structure this design space:

1. **Locality**: Stationary operand selection (weight-stationary, output-stationary, or input-stationary) dictates which tensor remains pinned in local registers or processing-element scratchpads, trading off memory traffic against register pressure.
2. **Organization**: Physical tensor layouts (such as channels-last `NHWC` versus channels-first `NCHW`) govern whether memory accesses match hardware memory controller burst sizes and SIMD vector widths, preventing uncoalesced memory transactions.
3. **Combination**: Kernel fusion and loop tiling partition iteration spaces and merge adjacent operations to eliminate round-trips to off-chip memory for intermediate tensors.

Together, these patterns provide a compact vocabulary for navigating dataflow choices without exhaustive search across billions of candidate schedules.

11.8.1 Building blocks of mapping strategies

The three decisions are realized through four building blocks, because the third decision has two mechanisms. Locality is set by data movement patterns that fix specific operands near the arithmetic units (weight-stationary, output-stationary, and input-stationary). Organization is set by memory-efficient tensor layouts (channels-last versus channels-first). Combination is set by kernel fusion, which keeps intermediate activations in on-chip SRAM between operations, and by loop tiling, which bounds working set sizes to fit local memory capacity. Rather than rediscovering schedules from scratch, heuristic compilers and auto-tuning frameworks compose these building blocks to construct efficient execution plans across deep neural network layers.

11.8.1.1 Data movement patterns

Dataflow design exists to exploit the steep energy divergence isolated in figure 11.8, where fetching an operand from DRAM requires orders of magnitude more energy than executing an arithmetic operation on it (Y.-H. Chen et al. 2016). Whether a workload executes dense matrix operations that exceed on-chip cache capacity or irregular sparse lookups, data movement strategy dictates overall execution latency and total power consumption. If data cannot be staged and supplied to processing elements at the rate required by their arithmetic throughput, the execution units stall on memory latency, stranding theoretical compute capacity and inflating memory subsystem energy.

To prevent arithmetic pipelines from stalling, a data movement strategy must capture the inherent algorithmic reuse of tensor operations within local, low-energy storage. Listing 11.14 illustrates the canonical three-nested-loop implementation of matrix multiplication, exposing the operand reuse across weights, inputs, and partial sums that a hardware mapping must capture in local storage.

Listing 11.14: Naïve Matrix-Multiplication Dataflow: The loop nest revisits weight rows, input columns, and output accumulators, exposing the reuse that a mapping must capture in local memory.

```
# Matrix multiplication where:
# weights: [512x256] - model parameters
# input:   [256x32]  - batch of activations
# Z:       [512x32]  - output activations

# Computing each output element Z[i,j]:
for i in range(512):
    for j in range(32):
        for k in range(256):
            Z[i, j] += weights[i, k] * input[k, j]
```

Every operand in this loop nest is reused, but along a different loop. Each weight `weights[i, k]` is used once for each of the 32 output columns, each input `input[k, j]` is used once for each of the 512 output rows, and each output `Z[i, j]` accumulates 256 products before it is final. A processing element's local storage holds only a small tile at a time, so the mapping cannot capture all three reuses at once. Fetching from DRAM whichever operand is not held locally, on every use, is what turns a compute-bound kernel into a memory-bound one (Han, Liu, et al. 2016).

The central dataflow decision is therefore which operand to keep local. The loop order answers it: whatever the innermost loops touch repeatedly without changing stays resident, and everything else streams. Listing 11.15 reorders the same computation three ways, and the three orders are the three classic stationary dataflows.

Listing 11.15: Three Stationary Dataflows: The same matrix multiplication as the naïve loop, reordered three ways; the operand held fixed across the innermost loop is the one fetched once and reused from local storage, while the other two stream.

```
# Weight stationary: hold one weight, stream inputs and partial sums past it
for i in range(512):
    for k in range(256):
        w = weights[i, k]  # fetched once, reused 32 times
        for j in range(32):
            Z[i, j] += w * input[k, j]

# Output stationary: hold one partial sum, stream weights and inputs past it
for i in range(512):
    for j in range(32):
        acc = 0  # stays in a local accumulator
        for k in range(256):
            acc += weights[i, k] * input[k, j]
        Z[i, j] = acc  # written once

# Input stationary: hold one input, stream weights and partial sums past it
for k in range(256):
    for j in range(32):
        x = input[k, j]  # fetched once, reused 512 times
        for i in range(512):
            Z[i, j] += weights[i, k] * x
```

A weight-stationary dataflow keeps parameters on chip, so each weight crosses the expensive memory boundary once no matter how many inputs it touches. That pays off wherever one filter or weight tile is applied to many inputs, as in convolutions and batched matrix multiplication, and it is the canonical TPU dataflow traced cycle by cycle in section 11.4.7. The saved traffic reappears on the other operands: inputs must arrive fast enough to keep the resident weights busy, and partial sums must move between processing elements as they accumulate. The resident tile is also bounded by on-chip capacity, so a layer whose weights far exceed it must cycle weight tiles through, reloading the inputs for each one.

An output-stationary dataflow keeps each running sum in a local accumulator until all of its contributions have arrived, so every output is written to memory exactly once (Y.-H. Chen et al.

2016). It suits layers with wide fan-in, such as fully connected layers and attention projections, where writing a partial sum out after every step would cost far more than the reads it avoids. The price is two operand streams at once, since both weights and inputs must now be delivered from memory. The naïve loop in listing 11.14 already has this order, because its innermost loop runs over k while i and j stay fixed.

An input-stationary dataflow keeps an activation tile local while weights stream past it. It pays off when one activation feeds several computations before eviction, as when a transformer token's activation feeds the query, key, and value projections. Its costs mirror weight stationary's, with weights now streaming from memory and partial sums still moving as they accumulate.

The three strategies form a decision framework rather than a quality ranking. The 36-fold gap in the loop-ordering case of example 11.4 came from a register file too small to hold all 16 filters, not from output stationarity itself; with the filters resident, the same output-stationary order would load each weight once. The right choice is the operand with the highest reuse relative to the on-chip storage it would occupy, which depends on layer dimensions and batch size, so compilers select among them per layer. Convolution-specific designs add a fourth label, row-stationary (as in Eyeriss), a hybrid that keeps rows of inputs and partial sums local when that mapping yields higher reuse than fixing any single operand (section 11.5.7.2).

11.8.1.2 Memory-efficient tensor layouts

The dataflow strategies in section 11.8.1.1 determine which operands stay close to compute units; tensor layouts determine whether those operands can be accessed efficiently once they arrive. A well-chosen weight-stationary schedule still stalls if weights are laid out in a format that produces strided, non-coalesced memory fetches. Tensor layout represents a kernel contract: the physical ordering of multidimensional data in linear memory must match the memory access stride expected by the accelerator's execution units, or the processor pays in memory stalls, cache underutilization, and redundant bus transactions.

Hardware accelerators fetch data from device memory in fixed-size transactions—typically 32 to 128 bytes per transaction. When compute units request non-contiguous addresses, each transaction delivers only a fraction of useful payload, degrading effective memory bandwidth. Aligning tensor layout with the execution model lets concurrent threads or SIMD lanes access nearby aligned locations in fewer transactions; the count depends on access width and device granularity (NVIDIA Corporation 2021b).

Machine learning frameworks and compilers often manage this layout selection automatically. Optimization tools such as cuDNN for NVIDIA GPUs, XLA for TensorFlow and JAX, and MLIR-based compiler stacks transform tensor layouts as they lower operations to backend-specific kernels (NVIDIA Corporation 2021b; Google 2025; Lattner et al. 2020). In high-level frameworks, layout transformations are typically applied transparently, but developers authoring custom kernels or targeting low-level APIs such as CUDA, Metal, or OpenCL must manage tensor formats directly.

In PyTorch, for example, view operations such as `tensor.permute()` adjust tensor strides without moving data in memory, requiring an explicit `tensor.contiguous()` call to materialize a packed buffer when a downstream kernel requires contiguous storage (PyTorch Contributors 2026b). Conversely, TensorFlow defaults to channels-last (NHWC) for convolutional models on NVIDIA GPUs, warning that NCHW inputs introduce automatic transpose overhead when lowered to GPU kernels (TensorFlow Developers 2024b). Hardware-optimized libraries such as cuDNN on GPUs and oneDNN on CPUs implement tailored memory formats to maximize cache line utilization and SIMD throughput. The governing principle is to treat memory layout as an intrinsic property of the backend execution path: the fastest tensor format is the one that minimizes layout conversion overhead and makes memory accesses contiguous across hardware execution units.

Channels-last layout In an NHWC channels-last layout, the channel index varies fastest in a contiguous tensor, placing the channel values for each spatial coordinate adjacent in memory. The label names the order in which dimensions are laid out in memory, from slowest-varying to fastest-varying, so NCHW and NHWC hold the same values in different linear orders.

Consider an RGB image tensor of shape $(\text{Height}, \text{Width}, \text{Channels}) = (3,3,3)$. The elements $X(h,w,c)$ are laid out in linear memory as follows:

$$X(0,0,0), X(0,0,1), X(0,0,2), X(0,1,0), X(0,1,1),$$
$$X(0,1,2), X(0,2,0), X(0,2,1), X(0,2,2), \ldots$$

All channel values for a given pixel are adjacent, and memory addresses advance across width before advancing down height. This layout supports vectorized execution when a compute kernel processes multiple channels simultaneously: CPU SIMD registers and GPU Tensor Cores can load contiguous channel vectors directly without intra-register shuffling.

The advantage of channels-last storage depends on the target operator and hardware unit. TensorFlow uses NHWC by default, whereas PyTorch provides a dedicated channels-last memory format that decouples logical dimension ordering from physical storage. Modern Tensor Core kernels in cuDNN favor NHWC because matrix-multiply units pack channel dimensions directly into accumulator tiles. When kernels require channel-contiguous inputs, passing an incompatible format forces the framework runtime to insert an explicit memory transpose, incurring a memory bandwidth penalty before computation begins.

Channels-first layout In a contiguous NCHW channels-first layout, the spatial width index varies fastest, and all spatial values across an entire channel are stored contiguously before the next channel begins. Accelerators process data in parallel across thread warps or SIMD lanes; when consecutive threads in a warp access consecutive memory addresses, the memory controller coalesces those requests into fewer memory transactions (**memory coalescing**).[27]

Consider the same RGB image tensor with dimensions reordered in channels-first order as $(\text{Channels}, \text{Height}, \text{Width}) = (3,3,3)$. Retaining $X(h,w,c)$ as semantic coordinate notation, the data are stored in channels-first order as follows:

$$X(0,0,0), X(0,1,0), X(0,2,0), X(1,0,0), X(1,1,0), X(1,2,0), \ldots,$$
$$X(0,0,1), X(0,1,1), X(0,2,1), \ldots, X(0,0,2), X(0,1,2), X(0,2,2), \ldots$$

In this format, all red channel values for the entire image are stored first, followed by all green values, and then all blue values. This arrangement allows kernels that parallelize across spatial coordinates (H,W) to issue coalesced reads along image rows, which suited earlier GPU convolution algorithms and SIMD execution pipelines (Chetlur et al. 2014).

The relative advantage of NCHW versus NHWC depends on how the underlying kernel maps computation to hardware threads. Convolutional layers evaluate filters across spatial and channel dimensions simultaneously. In classic direct or FFT-based convolutions, channels-first ordering enabled efficient spatial sliding without stride jumps. However, modern GEMM-lowered convolutions and Tensor Core pipelines map channel reduction directly to matrix multiply accumulators, favoring channels-last structures. When a model operates in NCHW but invokes a Tensor Core-optimized backend, the runtime must either run an unoptimized fallback kernel or dynamically transpose the tensor into NHWC, burning memory bandwidth in both cases.[28]

Comparing channels-last and channels-first layouts Both channels-last (NHWC) and channels-first (NCHW) layouts address specific hardware access patterns, with their efficiency determined by whether a kernel parallelizes over spatial coordinates or vectorizes across channel dimensions. Layout choice directly governs cache line utilization, memory bus transaction count, and compute unit occupancy. Table 11.16 contrasts the performance trade-offs and hardware compatibility between these two approaches.

[27] **Memory coalescing**: The GPU hardware mechanism that combines memory requests from threads in a warp into transactions according to address locality, alignment, access width, and device granularity. Tensor layout affects coalescing, but the sign of the effect depends on the kernel: NCHW can be efficient for some convolution implementations, while NHWC/channels-last is often preferred for modern Tensor Core convolution and fusion paths. Poor layout choices can still create multi-fold performance gaps, but the correct fix is to match the layout to the backend rather than memorize one universal format.

[28] **NHWC vs. NCHW**: NHWC lists dimensions as batch, height, width, channel; NCHW lists batch, channel, height, width. Physical memory format determines whether adjacent threads read adjacent addresses, so the performance effect is backend-specific. Layout-to-hardware mismatch is not a micro-optimization; it can create multi-fold performance gaps, especially when a conversion prevents Tensor Core convolution or fusion paths from being used.

Table 11.16: Data Layout Strategies: Channels-last (NHWC) and channels-first (NCHW) layouts optimize memory access patterns for different backend kernels. NHWC often suits CPU vectorization and modern Tensor Core convolution and fusion paths, while NCHW remains the default PyTorch tensor shape and suits older FP32 GPU kernel paths. Choosing the appropriate layout directly impacts performance by maximizing cache utilization and memory bandwidth efficiency.

Feature	**Channels-Last (NHWC)**	**Channels-First (NCHW)**
Memory storage order	Pixels are stored row-by-row, channel interleaved	All values for a given channel are stored together first
Best for	CPU loops, element-wise operations, many channels-last kernels	Many legacy GPU convolution paths and channel-first model code
Cache efficiency	High cache locality for sequential row/channel-last access	Can improve coalescing for channel-first kernels
Convolution performance	Often preferred by modern Tensor Core convolution/fusion paths	Efficient for older FP32 cuDNN paths and direct convolutions
Memory fetching	Good when kernels vectorize over adjacent channel data	Good when kernels process channel-major tiles
Default in frameworks	Common TensorFlow convention; PyTorch channels-last option	Common PyTorch tensor shape convention

While compilers such as XLA, TensorRT, and PyTorch Inductor automate layout propagation and insert layout conversions when beneficial, format conversions are not free: transposing a tensor requires reading every element from memory and writing it back in a different order, consuming memory bandwidth without executing useful arithmetic. In production deployments, matching the model's memory layout to the accelerator's native kernel format eliminates these layout transformations entirely.

Even when tensor layout is perfectly aligned with the hardware, executing operations as isolated kernels introduces a separate bottleneck: intermediate values must be exposed through global-memory buffers between operations, even when caches absorb some physical traffic. Resolving that bottleneck requires changing how operations are scheduled rather than how tensors are laid out.

11.8.1.3 Kernel fusion

Kernel fusion[29] entered the book as a framework optimization that combines successive operations into a single kernel to avoid intermediate memory round trips. The LayerNorm analysis (notebook 11.7) shows why it matters on an accelerator. At 2.0 FLOP/byte, a standalone normalization kernel can reach at most 4.1 TFLOP/s, about 1 percent of A100 tensor peak, however its arithmetic is scheduled. The hardware questions are which intermediates fusion removes from the memory bus and what the fused kernel costs in registers and occupancy.

Intermediate memory write Neural network execution is frequently bound by memory bandwidth rather than arithmetic throughput. When operations execute as isolated kernels, each intermediate result is written to a global-memory buffer and subsequently read by the downstream operator; cache behavior determines how much of that traffic reaches DRAM or HBM. This round trip introduces memory latency, stalls compute units, and consumes bus bandwidth on data with zero reuse outside that immediate pair of operations.

This hardware reality provides the physical motivation for kernel fusion. While section 11.5.1 analyzed bandwidth limits in terms of low operational intensity, chained elementwise operations aggravate the memory wall by multiplying global memory transactions (NVIDIA Corporation 2017).

In a naïve execution model, each operation executes as a separate GPU kernel, forcing intermediate tensors to round-trip through device memory (listing 11.16). In a chain of three operations—such as ReLU, batch normalization, and affine scaling—the intermediate outputs $\mathbf{X}'$ and $\mathbf{X}''$ are materialized in global-memory buffers and immediately reread by the subsequent kernel; the naive traffic model assumes those transfers reach DRAM. For elementwise operators where arithmetic intensity is low ($I \ll 1$ FLOP/byte), the time spent moving data across the memory bus outweighs the computation time (Jia et al. 2019). Furthermore, holding these un-fused intermediate buffers in memory inflates the live-tensor footprint during execution (table 11.17).

[29] **Kernel fusion**: Separate GPU kernels expose intermediate results through device memory, although caches can absorb some traffic. A fused kernel can keep intermediates in registers or local storage and avoid corresponding external write/read cycles. The resulting speedup depends on cache behavior, occupancy, launch overhead, and whether memory traffic was the bottleneck; it is not necessarily proportional to the byte reduction.

↩ Kernel fusion (also called operator fusion), with its LayerNorm, dropout, and ReLU traffic example, is established in section 7.3.1.

↩ The traffic count for un-fused versus fused operator chains, $2\,n_{\text{ops}}M_{\text{tensor}}$ versus $2M_{\text{tensor}}$, is established in section 10.5.1.

Listing 11.16: Naïve Execution: Each step writes intermediate results to memory before processing the next, leading to increased bandwidth usage and reduced efficiency.

```
import torch
import torch.nn.functional as F

# Input tensor
X = torch.randn(1024, 1024).cuda()
running_mean = torch.zeros(1024, device=X.device)
running_var = torch.ones(1024, device=X.device)

# Step-by-step execution (naive approach)
X1 = torch.relu(X)  # Intermediate tensor stored in memory
X2 = F.batch_norm(
    X1, running_mean, running_var, training=False
)  # Frozen inference BatchNorm
Y = 2.0 * X2 + 1.0  # Final result
```

Kernel fusion for memory efficiency The two intermediate tensors consume memory capacity when they remain live and create external traffic when separate kernels materialize them. Kernel fusion can eliminate those intermediate writes and reloads (Jia et al. 2019) by propagating values through registers or local memory within one kernel.

Table 11.17: Intermediate Tensor Storage: If all four named tensors remain live in this naïve example, they occupy 16.8 MB; an optimizing runtime may reuse buffers when earlier values are no longer needed. Fusion primarily guarantees removal of the external write and read for each eliminated intermediate rather than a universal reduction in peak allocated memory.

Tensor	**Size (MB) for 1024×1024 Tensor**
X	4.2 MB
X′	4.2 MB
X′′	4.2 MB
Y	4.2 MB
Total Memory	**16.8 MB**

A machine learning inference sequence might apply ReLU, frozen batch normalization, and then an affine scaling with scale s and offset b. In a naïve implementation, each separate kernel generates an intermediate tensor that is written to memory and read back by the next kernel:

$$\begin{aligned}\mathbf{X}' &= \mathrm{ReLU}(\mathbf{X})\\ \mathbf{X}'' &= \mathrm{BatchNorm}(\mathbf{X}')\\ \mathbf{Y} &= s\cdot\mathbf{X}'' + b\end{aligned}$$

With kernel fusion, these operations are combined into a single computation step, allowing the entire transformation to occur without generating unnecessary intermediate tensors:

$$\mathbf{Y} = s\cdot\mathrm{BatchNorm}\big(\mathrm{ReLU}(\mathbf{X})\big) + b$$

Table 11.18 highlights the impact of kernel fusion on memory efficiency. By keeping intermediate results in registers or local memory rather than writing them to main memory, fusion reduces external traffic. For three separate pointwise inference kernels, the idealized naïve path reads and writes one tensor per operation, whereas the fused path reads the input once and writes the output once.

Table 11.18: Operation Fusion Benefits: In this idealized inference sequence, fusion reduces external tensor traffic from 25.2 MB to 8.4 MB, a 3× reduction. Actual traffic depends on cache behavior and the compiler's buffer reuse.

Execution Model	**External tensor payloads**	**Traffic (MB)**
Naïve execution	3 reads + 3 writes	25.2 MB

Execution Model	External tensor payloads	Traffic (MB)
Fused execution	1 read + 1 write	8.4 MB

Performance benefits and constraints Kernel fusion brings several key advantages that enhance memory efficiency and computation throughput. By reducing memory accesses, fused kernels ensure that intermediate values stay within registers instead of being repeatedly written to and read from memory. This significantly lowers memory traffic, which is one of the primary bottlenecks in machine learning workloads. GPUs and TPUs, in particular, benefit from kernel fusion because high-bandwidth memory is a scarce resource, and reducing memory transactions leads to better utilization of compute units (NVIDIA Corporation 2020a).

However, not all operations can be fused arbitrarily. Element-wise operations and frozen inference normalization are strong candidates because their computations do not require batch-wide reductions. Training-mode batch normalization computes batch statistics and is not purely element-wise. Matrix multiplications and convolutions constrain fusion because they involve reductions and large data movement; they are often fused with element-wise epilogues such as bias or activation, but cannot be freely fused with unrelated global operations.

Another major consideration is register pressure. Fusing multiple operations means all temporary values must be kept in registers rather than memory. While this eliminates redundant memory writes, it also increases register demand. If a fused kernel exceeds the available registers per thread, the compiler can spill excess values to per-thread local memory, which is backed by device memory and may be cached, introducing additional latency and potentially negating the benefits of fusion. On GPUs, where thread occupancy (the number of threads that can run in parallel) is limited by available registers, excessive fusion can reduce parallelism, leading to diminishing returns.

Different AI accelerators and compilers handle fusion in distinct ways. NVIDIA GPUs, for example, favor warp-level parallelism, where element-wise fusion is straightforward (NVIDIA Corporation 2020a). TPUs, on the other hand, prioritize systolic array execution for dense matrix operations (Jouppi et al. 2017). Compiler and inference stacks such as TVM, XLA, TensorRT, and MLIR apply graph rewrites, lowering passes, or engine-building heuristics to balance memory savings against execution constraints (Chen et al. 2018; Google 2025; NVIDIA 2024c; Lattner et al. 2020).

Despite its advantages, fusion is not always beneficial. Some AI frameworks allow developers to disable fusion selectively, especially when debugging performance issues or making frequent model modifications. The decision to fuse operations must consider trade-offs between memory efficiency, register usage, and hardware execution constraints to ensure that fusion leads to tangible performance improvements.

Checkpoint 11.3: Data movement and kernel fusion

The dataflow building blocks are now in place: data locality, tensor layout, and kernel fusion. These fusion decisions are ultimately about data locality, which ties together the chapter's core data movement strategies:

- ☐ **Locality choice**: given weight-stationary, output-stationary, and input-stationary dataflows, which tensor does each hold near the compute units, and how does that choice shift where the memory traffic lands?
- ☐ **Layout match**: a kernel runs on a backend expecting NHWC but receives an NCHW tensor. Predict what happens to its memory access pattern and to delivered throughput.
- ☐ **Fusion eligibility**: for a Conv2D, a BatchNorm, and a ReLU executed in sequence, decide which pairs are worth fusing and what determines whether fusing them pays off.
- ☐ **Remaining gap**: an operation still repeatedly fetches operands because its active blocks exceed fast-memory capacity. How could tiling from section 11.4.7 change this access pattern, and what capacity and reuse evidence would justify that choice?

11.8.1.4 Memory-efficient tiling strategies

Layout changes how values are arranged, and fusion can remove intermediate buffers. Tiling addresses another part of the locality problem by bounding the operand blocks actively reused at each memory tier. Section 11.4.7 introduced tiling as the bridge between a graph-level matrix and a fixed systolic array, including the fringe tax and the loop nest in algorithm 11.1. The same partitioning recurs at every tier of the memory hierarchy. Tiling[30] restructures a computation into blocks that fit within fast local memory (caches, shared memory, or registers), so that each block is reused before eviction rather than refetched from DRAM (Lam et al. 1991).

Matrix multiplication, widely used in AI models, demonstrates inefficient memory access when implemented naively. Listing 11.17 shows how, without tiling, repeated memory accesses for the same data lead to unnecessary bandwidth consumption.

[30] **Tiling (loop blocking)**: This restructuring partitions a computation into blocks sized for a fast local tier. A naive loop makes repeated references to the same elements; caches may capture some, while blocking deliberately concentrates reuse before eviction (Lam et al. 1991). This reduction in lower-tier traffic is a primary source of the gap between naive matrix multiplication and optimized GEMM routines.

Listing 11.17: Naïve Matrix Multiplication: Direct implementation makes $\mathcal{O}(N^3)$ array references for $N \times N$ matrices. Caches can serve some references, but the loop order does not deliberately block reuse for the memory hierarchy.

```
for i in range(N):
    for j in range(N):
        for k in range(N):
            C[i, j] += A[i, k] * B[k, j]  # Repeatedly fetching
            # A[i, k] and B[k, j]
```

The loop repeatedly references elements of **A** and **B**. Some references can hit in cache, but large matrices and an unfavorable traversal order can still cause excessive lower-tier traffic. Figure 11.13 diagrams the spatial partitioning of matrix multiplication ($\mathbf{C} = \mathbf{AB}$) across its three principal dimensions (M, K, and N). Each highlighted region represents the working set staged in fast memory at one moment: an $M_{\text{tile}} \times K_{\text{tile}}$ block of **A** (within the green row band) multiplies a $K_{\text{tile}} \times N_{\text{tile}}$ block of **B** (within the pink column band) to accumulate one $M_{\text{tile}} \times N_{\text{tile}}$ output tile ($\text{Block}_{m,n}$, shaded blue) of **C**. Processing all inner-product operations for an active tile before evicting it amortizes off-chip DRAM latency across $M_{\text{tile}} \cdot N_{\text{tile}} \cdot K_{\text{tile}}$ arithmetic operations.

Tiling fundamentals In listing 11.17, every iteration of the innermost loop fetches A_{ik} and B_{kj}, multiplies them, and updates C_{ij}. Because the matrices are large, the processor reloads values it used moments earlier, and each reload that misses on chip pays the DRAM penalty in latency and energy (Horowitz 2014; Sze et al. 2017).

Performance benefits of tiling Tiling processes one submatrix at a time, so that once a block is loaded it is reused many times before the loop moves on. Listing 11.18 demonstrates cache-friendly loop blocking: the loop bounds partition the matrices into tiles, and the hardware cache hierarchy keeps recently used tile data close to the compute units when access order has locality.

Listing 11.18: Cache-Blocked Matrix Multiplication: This high-level loop blocking approach divides matrices into smaller index ranges so hardware caches can reuse data within each tile, improving computational efficiency without explicit scratchpad loads.

```
TILE_SIZE = 32  # Choose a tile size based on hardware constraints

# Cache blocking: partition data via loop bounds.
# Loads are implicit through the hardware cache hierarchy.
for i in range(0, N, TILE_SIZE):
    for j in range(0, N, TILE_SIZE):
        for k in range(0, N, TILE_SIZE):
            # Each tile computed independently
            for ii in range(i, min(i + TILE_SIZE, N)):
                for jj in range(j, min(j + TILE_SIZE, N)):
                    for kk in range(k, min(k + TILE_SIZE, N)):
                        C[ii, jj] += A[ii, kk] * B[kk, jj]
```

The restructuring produces a chain of effects. A small tile is revisited while it is still likely to be cached, so most operand references hit in L1/L2 or shared memory instead of DRAM. DRAM

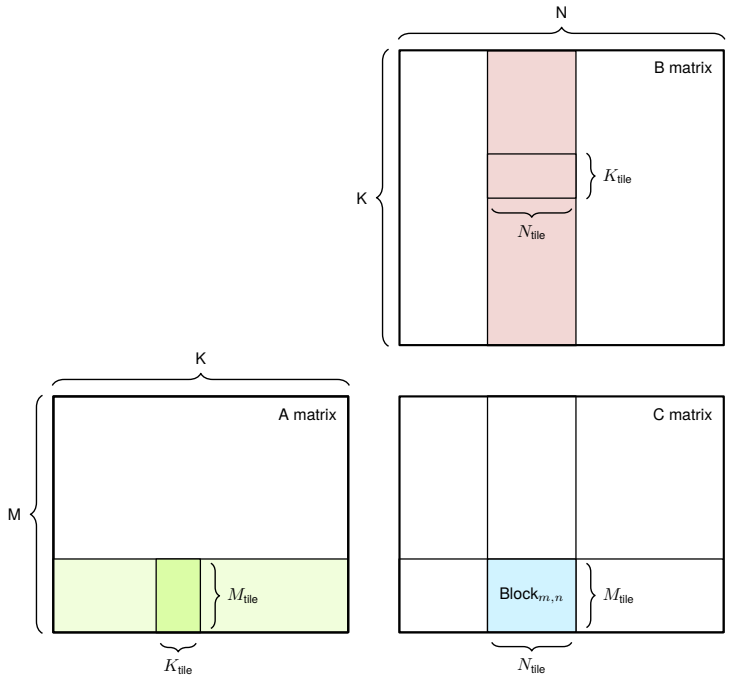


Figure 11.13: Matrix Tiling Mechanics: Partitioning large matrix multiplication ($\mathbf{C} = \mathbf{AB}$) into $M_{\text{tile}} \times K_{\text{tile}}$ and $K_{\text{tile}} \times N_{\text{tile}}$ blocks aligns memory footprints with fast on-chip SRAM/registers. Maximizing register reuse within each tile amortizes slow off-chip DRAM accesses and keeps compute engines operating at peak throughput.

traffic falls as a result, and the processing units spend less time waiting for operands. When an untiled access pattern repeatedly fetches operands from lower memory tiers, that traffic can impose a bandwidth ceiling below the arithmetic ceiling.

Tiling methods Tiling implementations differ along one axis, which is whether the hardware or the program decides what stays in fast memory. The two ends are implicit (cache) blocking and explicit (scratchpad) staging, and practical kernels often combine them.

Implicit (cache) blocking restructures only the loop bounds. The tiled matrix multiplication in listing 11.18 issues no explicit loads; its tile-shaped access pattern lets hardware caches reuse nearby values before they are evicted. It suits CPUs and GPUs with capable caches and large tensors that exceed fast-memory capacity.

Explicit (scratchpad) staging copies each tile into shared memory, a scratchpad, or registers and reorganizes the computation around the staged copy. It is the only option on accelerators whose fast memory is software managed, and it guarantees reuse that a cache might lose to eviction. Convolution is the classic case, because staging a filter once lets it serve many input regions without being refetched. Listing 11.19 loads blocks of **A** and **B** into temporary fast storage and reuses them across the inner loops.

Listing 11.19: Explicit Tile Staging Pseudocode: A backend maps the temporary tile objects to fast storage and reuses them across inner-loop operations.

```
# Explicit tile staging: load data into fast
# temporary storage before the inner loops.
for i in range(0, N, TILE_SIZE):
    for j in range(0, N, TILE_SIZE):
        for k in range(0, N, TILE_SIZE):
            # Pseudocode: map these tile objects to fast memory
            A_tile = A[i : i + TILE_SIZE, k : k + TILE_SIZE]
            B_tile = B[k : k + TILE_SIZE, j : j + TILE_SIZE]

            # Reuse loaded tiles for all inner iterations
            for ii in range(A_tile.shape[0]):
                for jj in range(B_tile.shape[1]):
                    for kk in range(A_tile.shape[1]):
                        C[i + ii, j + jj] += (
                            A_tile[ii, kk] * B_tile[kk, jj]
                        )
```

The pseudocode includes shortened boundary tiles when N is not divisible by `TILE_SIZE`, the software counterpart of the fringe tax that a fixed-size array pays in idle processing elements.

Tiling challenges and trade-offs Tiling improves performance only when the tile matches the locality budget of the hardware. If the tile is too small, memory fetches still dominate execution time because reuse is too limited. If the tile is too large, it exceeds fast memory and causes cache thrashing or scratchpad spills. Selecting the right tile size therefore directly determines computational efficiency and memory bandwidth usage.

The capacity test counts every buffer that is live at the same time. For the tile dimensions in algorithm 11.1, the operand blocks contain $M_{\text{tile}}K_{\text{tile}}$ and $K_{\text{tile}}N_{\text{tile}}$ elements, while the output accumulator contains $M_{\text{tile}}N_{\text{tile}}$ elements. Suppose a hypothetical accelerator retains all three in one software-managed local store, uses FP16 operands and an FP32 accumulator, and loads only the current operand pair. Element storage size s_{elem} is then 2 bytes for each operand and 4 bytes for each accumulator entry. After reserving space for other live state, the tile must satisfy

$$(2\text{ bytes})M_{\text{tile}}K_{\text{tile}} + (2\text{ bytes})K_{\text{tile}}N_{\text{tile}} + (4\text{ bytes})M_{\text{tile}}N_{\text{tile}} \leq \text{available local bytes}. \tag{11.7}$$

Keeping the output dimensions fixed while doubling the reduction tile doubles both operand buffers without changing the accumulator. Reject that larger candidate if the revised left-hand side of equation 11.7 exceeds the available capacity, even if its longer reduction amortizes accumulator handling over more products. Double buffering has the same operand-capacity effect at fixed tile dimensions: retaining the next **A** and **B** tiles while computing the current pair requires a second copy of each operand buffer, while the running accumulator remains single buffered.

Actual storage tiers must be budgeted separately. A GPU kernel may stage operand tiles in shared memory while keeping output accumulators in thread registers; unused register capacity cannot make an oversized shared-memory allocation fit. Its shared-memory test counts the staged operand copies and any other shared state, while its register test counts accumulators, operand fragments, and temporary values across the participating threads. Passing both tests establishes capacity feasibility. It does not establish speed: alignment, bank conflicts, occupancy, synchronization, and measured traffic still determine whether the tile improves execution.

The tile choice also controls load balance. In architectures such as GPUs and TPUs, computations execute in parallel across thousands of processing units. If tiles are not evenly distributed, some units remain idle while others are overloaded, leading to suboptimal utilization of computational resources. Effective tile scheduling keeps parallel execution balanced and efficient.

Data movement remains a cost after tiling, even when another resource becomes the throughput limit. Although tiling reduces the number of slow memory accesses, transferring tiles between hierarchy levels still incurs latency and energy cost, especially when data falls from cache or scratchpad back to DRAM. Efficient memory prefetching and scheduling strategies minimize this residual movement and ensure that data is available when needed.

Hybrid tiling combines implicit blocking and explicit staging when neither alone captures the workload's reuse, for example relying on caches for an outer tier while staging convolution weights explicitly in a scratchpad.

Register blocking, double buffering, and hierarchical tiling extend the same locality principle at smaller and larger memory tiers. AI compilers and runtime systems such as TensorFlow XLA, TVM, and MLIR automatically select these tiling strategies based on hardware constraints, enabling fine-tuned performance optimization without manual intervention. Table 11.19 compares implicit, explicit, and hybrid tiling.

Table 11.19: Tiling Strategies: Implicit, explicit, and hybrid tiling optimize memory access patterns for improved performance. Implicit (cache) blocking shapes loop bounds and lets caches decide residency, explicit (scratchpad) staging copies tiles into software-managed memory to guarantee reuse, and hybrid tiling combines both across memory tiers. AI compilers and runtime systems use these techniques to automatically optimize model execution on diverse hardware.

Aspect	**Implicit (Cache) Blocking**	**Explicit (Scratchpad) Staging**	**Hybrid tiling**
Who Decides Residency	Hardware cache replacement policy	The program, through explicit copies into software-managed memory	Caches for some tiers, explicit copies for others
Optimization Focus	Loop bounds shaped so accesses stay within cache-sized blocks	Staging tiles and reordering computation around the staged copy	Matching each memory tier to the mechanism it supports
Memory usage	Improves cache locality and reduces DRAM access	Keeps frequently used data in fast memory for multiple iterations	Minimizes data movement while ensuring high reuse
Common use cases	Matrix multiplications, CNNs, self-attention in transformers	Convolutions, recurrent neural networks (RNNs), iterative computations	AI accelerators with hierarchical memory, mixed workloads
Performance gains	Reduced memory bandwidth requirements, better cache utilization	Lower memory fetch latency, improved data locality	Maximized efficiency across multiple hardware types
Challenges	Reuse is not guaranteed; eviction can discard a tile before it is reused	Can increase register pressure, requires loop restructuring	Complexity in tuning tile size and execution order dynamically
Best when	Hardware caches are capable and the working set exceeds fast memory	Fast memory is software managed or reuse must be guaranteed	Different memory tiers favor different mechanisms

11.8.2 Applying mapping strategies to neural networks

While section 11.5.7 diagnosed how convolutional networks, transformers, and multilayer perceptrons stress memory subsystems, the mapping strategies above provide the systems remedies. Mapping is not a heuristic exercise in isolation; it is a systematic search over loop permutations, tiling factors, and stationarity policies to maximize data reuse under hard SRAM and register constraints.

Because each model family presents a distinct ratio of weights to activations and differing opportunities for temporal locality, optimal mapping policies diverge across workloads. Convolutional networks exploit 2D spatial weight reuse across receptive fields; transformers split between compute-bound matrix multiplications during prefill and bandwidth-bound KV-cache streaming during autoregressive decoding; and dense MLPs demand blocked GEMM execution across batch dimensions. Table 11.20 consolidates how dataflow, memory layout, kernel fusion, and tiling align with these three architectural archetypes.

11.8.2.1 Convolutional neural networks (ResNet-50)

For ResNet-50 and similar CNNs, a key mapping opportunity is spatial weight reuse. A small filter is applied across many spatial locations, so its weights participate in many multiply-accumulate operations. Weight-stationary mappings can capture that reuse by retaining filter tiles locally, while row-stationary or output-stationary mappings may better balance activation and partial-sum traffic on other architectures. Favorable convolution shapes can achieve high arithmetic intensity, which makes them well suited to matrix and systolic execution paths. The model family does not determine

one universal dataflow: layer shape, batch size, local-storage capacity, and the target's supported kernels decide which reuse pattern is worth retaining.

Table 11.20: Architecture-Specific Mapping Strategies: Each neural network architecture benefits from different optimization priorities based on its computational and memory characteristics.

Optimization technique	CNNs	Transformers	MLPs	Rationale
Dataflow strategy	Weight, row, or output stationary	Phase- and kernel-dependent	Weight or output stationary	CNNs expose filter and spatial reuse; transformer attention and MLP blocks impose different traffic; dense MLPs reuse weights across a batch and accumulate outputs.
Memory-aware tensor layouts	Backend-dependent (channels-last on Tensor Core paths, NCHW on older FP32 paths)	Backend-dependent (row-major activations typical)	Row-major typical	Layout choice depends on backend kernel path and precision mode (section 11.8.1.2 compares the layouts); the entries name common defaults, not universal prescriptions.
Kernel Fusion	Convolution + Activation	Fused Attention	GEMM Fusion	CNNs optimize convolution+activation fusion; Transformers fuse attention mechanisms; MLPs benefit from fused matrix multiplications.
Tiling for Memory Efficiency	Explicit staging of feature-map tiles	Explicit staging of attention blocks	Blocked GEMM	CNNs stage feature-map subregions in on-chip SRAM; transformers stage query and key/value blocks in fused attention; MLPs block large matrix multiplications, implicitly or explicitly.

This spatial regularity also enables aggressive fusion and tiling. For inference with frozen BatchNorm statistics, compilers can fold or fuse convolution, normalization, and activation to avoid unnecessary intermediate writes. Training-mode BatchNorm requires reductions over batch statistics and does not have the same element-wise fusion contract. Spatial tiling partitions the feature map into subregions sized to fit within on-chip SRAM, so the fused kernel processes each tile from fast memory before moving to the next.

11.8.2.2 Transformer architectures (GPT-2/Llama)

Transformer mapping depends on execution phase. During training and prefill, attention creates large intermediate matrices unless a tiled implementation avoids materializing them; during autoregressive decoding, attention reads the growing key-value (KV) cache. Each new query vector must access the stored key and value pairs across the preceding sequence. As sequences grow, the full KV cache usually lives in HBM or device DRAM, while attention kernels tile active blocks through SRAM, registers, or shared memory. This access pattern motivates staging active K/V tiles in SRAM so attention consumes them from fast local storage.

During training and prefill, the memory traffic created by standard attention implementations explains why fused attention kernels, such as FlashAttention (T. Dao et al. 2022), can deliver large performance gains: by fusing the query-key dot product, softmax normalization, and value-weighted summation into a single kernel that tiles along the sequence dimension, these implementations avoid materializing the full attention matrix in main memory. This explicit staging processes sequence blocks that fit within on-chip SRAM, substantially reducing HBM traffic while preserving the $\mathcal{O}(S^2)$ attention computation. Transformer mapping depends on the phase and shape: training and prefill GEMMs can be compute-bound, while low-batch autoregressive decode is commonly constrained by weight and KV-cache bandwidth.

11.8.2.3 Multilayer perceptrons and DLRM

MLPs present the most straightforward mapping problem because their computation reduces largely to dense GEMM.[31] Each fully connected layer multiplies an activation matrix by a weight matrix. The weight matrix is fixed across samples, so batching reuses each loaded weight across more activations and raises arithmetic intensity. A batch size of one provides little cross-sample weight reuse and often leaves wide matrix engines underused, while larger batches can move execution toward the compute-bound regime.

[31] **GEMM**: The operation $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$, with scalars α and β, is the dense linear-algebra primitive that many deep-learning layers lower to. Optimized GEMM libraries such as cuBLAS and oneDNN use register blocking, vectorization, and hierarchical tiling to approach hardware limits on favorable shapes (NVIDIA 2024a; Intel Corporation 2021b). Modern AI accelerators are heavily specialized for GEMM-like tiles: Tensor Cores, systolic arrays, and matrix extensions all exist to accelerate this primitive, which is why GEMM performance is an important predictor of end-to-end throughput across architectures.

↪ How GEMM arithmetic intensity scales with matrix dimension is developed in section C.1.4.

Because MLP layers are typically followed by activation functions and bias additions, GEMM fusion combines these steps into a single kernel, avoiding intermediate memory writes. Blocked tiling partitions the large matrix multiplications into sub-blocks sized for the accelerator's shared memory, ensuring high cache utilization throughout computation. The simplicity of the MLP mapping, dominated by a single primitive with predictable access patterns, is precisely why hardware vendors optimize GEMM libraries so aggressively: gains in GEMM performance translate directly to MLP throughput. DLRM is not purely an MLP workload, however. Its large sparse embedding tables and feature-interaction stage can dominate capacity and memory traffic, so the dense mapping described here applies to its bottom and top MLP towers rather than to the entire model (Naumov et al. 2019).

11.8.3 Hybrid mapping strategies

The preceding architectural subsections treat each architecture in isolation, but real models rarely consist of a single layer type. A vision transformer, for example, combines a patch embedding stage, self-attention layers, and MLP blocks (Dosovitskiy et al. 2021). Those layers create different reuse patterns: the embedding stage can benefit from weight-stationary mapping, attention emphasizes activation movement and tiling, and MLP blocks demand blocked GEMM tiling and fusion. No single dataflow strategy is optimal across all these layers, so hardware mapping becomes hybrid and layer-specific.

Hybrid mapping addresses this heterogeneity by allowing the accelerator to switch strategies at layer boundaries. Each layer presents a different balance of compute intensity, data reuse, and memory access pattern, and the optimal mapping must shift accordingly (Sze et al. 2017). Rather than committing to one dataflow for the entire model, hybrid approaches consider weight stationary execution when weights can be reused locally, input-stationary execution when activation reuse justifies its storage, and output stationary execution when partial-sum traffic is costly. Layer dimensions and available local memory determine which candidate pays off.

Modern accelerators provide the architectural features needed to realize hybrid mapping in practice. TPU-style systolic arrays, NVIDIA GPUs, and tile-based accelerator designs expose different combinations of local memory, tensor layouts, fusion, and scheduling controls, allowing compilers and runtimes to choose layer-specific strategies rather than one global dataflow for the whole model (Jouppi et al. 2023; NVIDIA Corporation 2020a; Chen et al. 2018). These implementations require programmable memory hierarchies, efficient interconnects, and specialized execution pipelines, reinforcing the hardware-software co-design principle.

However, hybrid mapping remains a design-time optimization. In production workloads, execution conditions change dynamically due to varying input sizes, memory contention, and hardware resource availability. Machine learning compilers and runtime systems extend these static mapping choices by introducing dynamic scheduling, memory optimizations, and automatic tuning, ensuring that deep learning workloads operate efficiently across diverse accelerators and deployment environments.

11.9 Compiler Support

A single convolution can be implemented with dozens of valid tiling strategies, kernel variants, and memory layouts, most of which perform poorly on a given target. Compilation first appeared from the framework's side, as the choice between eager and graph execution and as a way to cut dispatch overhead. From the hardware's side, **machine learning compilers** navigate this space by translating high-level dataflow representations into target-specific executable code, making the mapping and dataflow decisions of the preceding sections automatically. Compiling ResNet-50 for GPU inference illustrates the core compiler stages:

1. **Graph optimization** fuses repeated Conv2D-BatchNorm-ReLU patterns into single compound kernels, eliminating intermediate round-trips to HBM.
2. **Kernel selection** chooses specialized Tensor Core implementations for compatible matrix multiplications, exploiting the high arithmetic intensity of dense convolutions.
3. **Memory planning** determines activation lifetimes to safely overlap and reuse intermediate buffer allocations.

4. **Computation scheduling** pipelines memory copies with execution across independent hardware queues to hide data transfer latencies.

In this scenario, inference latency drops from 47 ms to 8 ms, a 5.9× improvement without altering the model's mathematical output. The latencies are illustrative rather than measured, but the mechanism is general. Graph rewrites, kernel selection, memory planning, and scheduling systematically apply kernel fusion (section 11.8.1.3) and memory-efficient tiling (section 11.8.1.4) to convert theoretical hardware FLOP/s into delivered throughput (Chen et al. 2018; NVIDIA 2024c).

↩ Graph compilation, kernel fusion, and framework dispatch overhead are established in section 7.3.

This pipeline embodies the hardware-software co-design principle introduced in section 11.1. Machine learning compilers bridge high-level model specifications and low-level execution engines by restructuring computations, selecting hardware-specialized kernels, and planning tensor storage lifetimes (Chen et al. 2018). They build on classical compiler foundations while introducing tensor- and graph-level transformations.

11.9.1 ML compiler design

Machine learning compilers specialize conventional compilation techniques for programs expressed as tensor graphs and operator libraries. General-purpose compilers optimize scalar control flow, vector instructions, register allocation, and cache locality; ML compiler stacks operate at a higher semantic level, optimizing graph topology, tensor memory layouts, shape specialization, accelerator kernel dispatch, and inter-device execution planning (Li et al. 2021).

Table 11.21 contrasts the emphasis of these two compiler layers. The boundary is complementary rather than disjoint: an ML compiler stack lowers high-level tensor operations into intermediate representations that feed into an LLVM, GPU, or vendor-specific backend.

Table 11.21: Compiler Optimization Priorities: ML compiler stacks add tensor- and graph-level decisions to conventional instruction- and loop-level compilation. In a production toolchain, the two layers commonly cooperate rather than replace one another.

Aspect	**Traditional compiler**	**Machine learning compiler**
Input representation	Source or intermediate-representation program	Tensor graph or tensor-level intermediate representation
Execution Model	Control flow, loops, threads, vectors, and tasks	Tensor operators, kernels, graphs, and accelerator streams
Optimization priorities	Instruction selection, loop transforms, vectorization, registers	Graph rewrites, fusion, layouts, shapes, and kernel selection
Memory management	Objects, stack/heap, caches, locality, and prefetching	Tensor lifetimes, buffer reuse, layouts, tiling, and device transfers
Target Hardware	CPUs, GPUs, and other programmable targets	CPUs, GPUs, TPUs, and custom accelerators
Compilation output	Machine code or lower-level intermediate representation	Kernels plus a hardware-specific graph or execution plan

The distinction in table 11.21 explains why compiler configuration dictates delivered performance even when the underlying neural network code remains unmodified. ML compilers manage the translation layer between high-level tensor operations and hardware execution units; when this mapping fails to expose reuse or efficient access, execution can be limited by memory traffic, latency, or insufficient parallel work.

> Systems Perspective 11.2: The hidden optimization layer
>
> Most practitioners never interact directly with ML compilers, yet compiler quality determines what fraction of peak accelerator throughput a model achieves. Enabling a supported just-in-time compilation backend through Keras `model.compile()`, calling `torch.compile()` in PyTorch, or building an engine through TensorRT can invoke optimization pipelines that apply four kinds of transformation:
>
> - **Fuse operations** never combined by the developer (such as merging Conv2D, BatchNorm with frozen inference statistics, and ReLU into a single kernel)

- **Reorder computations** to maximize data reuse in fast local memory (such as tiling matrix multiplications)
- **Select kernels** from libraries containing hundreds of hand-tuned, architecture-specific implementations
- **Transform tensor layouts** between what the framework API expects and what hardware memory controllers require

The same model definition runs at vastly different throughputs depending on the compilation backend, graph lowering choices, kernel selection, and runtime configuration. When execution latency exceeds service targets, compiler configuration and backend lowering are primary optimization levers that require no alterations to model architecture or training parameters.

11.9.2 ML compilation pipeline

Machine learning models defined in modern frameworks are represented as high-level dataflow graphs describing operations on multidimensional tensors. Lowering these representations into executable code for CPUs, GPUs, TPUs, or custom ASICs requires an ML compilation pipeline (Chen et al. 2018; Google 2025; Lattner et al. 2020).

A standard compilation pipeline adds a fifth stage, code generation, to the four that the ResNet-50 example illustrated:

1. Graph optimization restructures the computation to eliminate redundant work and reduce data traffic.
2. Kernel selection maps mathematical operations to optimized target-specific implementations.
3. Memory planning assigns tensor layouts, static buffer offsets, and lifetime bounds.
4. Computation scheduling orders kernels and memory transfers to maximize parallelism and hide transfer latencies.
5. Code generation emits target-specific machine instructions, assembly, or interpreted bytecode.

Across these stages, the compiler applies the dataflow strategies developed in section 11.8: kernel fusion, memory tiling, and compute placement. Hardware acceleration delivers its theoretical throughput only when these compiler-driven transformations expose sufficient arithmetic intensity and align memory access with the hardware hierarchy.

11.9.3 Graph optimization

Machine learning frameworks represent models as high-level computation graphs, where nodes represent mathematical operators (such as convolutions, matrix multiplications, and activations) and directed edges represent tensor data dependencies. Executing each operator as a separate kernel can incur intermediate-buffer traffic and repeated launches. An edge denotes a dependency, not necessarily a new allocation or an off-chip transfer, and asynchronous dependencies need not synchronize the host. When intermediate materialization and dispatch dominate execution, graph transformations can remove substantial overhead.

Graph optimization transforms this computation graph into an efficient execution plan before hardware code generation (Chen et al. 2018; Jia et al. 2019). Working primarily at a target-independent intermediate representation (IR) level before lowering (section 11.9.7), graph passes apply four core transformations:

Kernel fusion merges adjacent operations into a single compound kernel to eliminate off-chip memory round-trips and launch overheads. In convolutional networks, fusing Conv2D, batch normalization with frozen inference statistics, and ReLU into one kernel keeps activations in local registers or shared SRAM throughout the sequence (section 11.8.1.3). In transformer architectures, pattern matchers replace multi-operator self-attention subgraphs (score matrix multiplication, scaling, masking, softmax, and multiplication by the value matrix) with IO-aware attention kernels that keep active tiles and running reduction statistics in fast on-chip memory (T. Dao et al. 2022).

Redundant computation elimination and constant folding identify and remove unnecessary operations. During inference compilation, parameters from static layers—such as batch normalization

scale and offset vectors—are folded directly into the weights and biases of preceding linear or convolutional layers, eliminating normalization operators from runtime execution. Graph passes also prune dead subgraphs and merge identical parallel paths through common subexpression elimination.

Algebraic simplification and layout adjustments reorder operations where mathematically equivalent to minimize data movement. For example, transpositions can be pushed through elementwise operations or cancelled against inverse permutations, reducing runtime reshape overhead.

Memory-aware node reordering evaluates data dependencies across topological sorts of the graph to schedule independent operations in an order that minimizes peak live-tensor footprint. Reordering nodes ensures that intermediate activations are consumed and deallocated promptly, preventing memory pressure from forcing spills to host memory.

Compilers such as XLA, TVM, TensorRT, and MLIR execute these rewrites through pattern-matching rules. By eliminating unnecessary memory round-trips and exposing coarse-grained parallelism, graph optimization produces an execution graph that aligns with the accelerator's bandwidth constraints. Once the graph is restructured into optimized compound operations, the compiler must bind each node to a concrete, hardware-specific execution routine.

11.9.4 Kernel selection

Kernel selection binds each node in the optimized graph to a concrete hardware implementation. Accelerators provide dozens of valid kernel implementations for a single tensor operation, each parameterized by tile dimensions, thread block configurations, memory staging patterns, and target precision. Choosing the optimal kernel determines whether the execution engine achieves high compute utilization or stalls on uncoalesced memory accesses and pipeline bubbles (Chen et al. 2018; Zheng et al. 2020).

In most production pipelines, compilers draw from vendor-optimized kernel libraries rather than synthesizing kernels from scratch. NVIDIA's cuDNN and cuBLAS provide hand-tuned routines for GPUs, Intel's oneDNN targets x86 architectures, and comparable libraries serve Arm and other CPUs. These libraries encode microarchitectural details (such as register file capacity, cache hierarchy latency, and vector lane width), allowing the compiler to invoke pre-tuned implementations.

AI compilers employ three primary strategies to select among candidate kernels:

Rule-based selection relies on static heuristics[32] to map operators based on tensor shapes, data types, and hardware targets. For example, shape, alignment, precision, and target support can guide selection toward a Tensor Core-enabled GEMM. Heuristics avoid the cost of benchmarking a large candidate set, though they still incur selection work and can miss faster configurations on irregular shapes.

Cost model-based selection estimates kernel execution latency and memory traffic using analytical models of the hardware architecture. Frameworks such as MLIR provide infrastructure for cost models that predict register pressure, cache hit rates, and arithmetic intensity across parameter spaces, pruning unpromising candidates without running physical benchmarks (Lattner et al. 2020).

Profile-guided selection evaluates candidate kernels empirically on the target hardware. TVM's AutoTVM and Ansor systems measure execution latency directly on the accelerator, navigating tile sizes, loop unrolling factors, and thread configurations via statistical search to converge on the fastest implementation (Chen et al. 2018; Zheng et al. 2020). Similarly, TensorRT builds hardware-specific execution engines by benchmarking tactic candidates directly against target batch sizes and memory constraints.

Precision-aware selection integrates numerical requirements into the selection process. Training recipes choose input, accumulation, and state formats to preserve numerical behavior; inference can also use lower-precision paths after quality validation. Relative to FP32, FP16 halves value bytes and INT8 quarters them, while total traffic depends on the complete representation. Selecting an INT8 path without appropriate range handling can degrade accuracy, and selecting a higher-precision path unnecessarily can forgo available throughput.

Once kernels are selected, memory planning and computation scheduling determine exact buffer allocations, launch sequences, and stream concurrency.

[32] **Heuristic in kernel selection**: A practical rule that chooses a promising implementation without exhaustively benchmarking the full candidate space. Tile sizes, layouts, precision modes, fusion choices, and shape constraints can create many legal GEMM variants. Heuristics reduce tuning cost but may miss a faster candidate, which is why autotuners such as AutoTVM profile selected options on the target hardware.

11.9.5 Memory planning

Memory planning assigns physical storage locations, buffer offsets, and tensor lifetimes across the accelerator's memory hierarchy (Roesch et al. 2018; Chen et al. 2018). High-performance execution plans fail if memory management introduces uncoalesced memory transactions, bank conflicts, or unnecessary memory allocations that exhaust accelerator memory.

The compiler coordinates three core memory management mechanisms:

Tensor layout optimization determines the physical arrangement of tensor dimensions in memory. Different hardware architectures and compute engines require specific memory layouts to enable coalesced global memory access and vectorization (section 11.8.1.2). For example, NVIDIA Tensor Cores achieve peak throughput with the channels-last `NHWC` layout for FP16 and INT8 convolutions, where channel dimensions align with 16-byte memory transactions, whereas legacy FP32 CUDA cores historically favored channels-first `NCHW` (NVIDIA Corporation 2021b; Google 2025). Compilers perform layout assignment globally across the graph to prevent back-to-back layout transformation kernels that consume memory bandwidth.

Liveness analysis and buffer reuse prevent out-of-memory errors by sharing physical memory across non-overlapping tensors. Deep neural networks generate dozens of intermediate activation tensors during forward propagation. Allocating a unique memory buffer for every tensor would quickly exceed the capacity of device memory (such as 16 to 80 GB on typical accelerator GPUs). The compiler analyzes the graph to compute the liveness interval of each tensor—from its generation to the completion of its last consumer. By modeling memory allocation as an interval graph coloring or 2D bin-packing problem, the compiler assigns multiple non-overlapping intermediate tensors to the same physical memory offsets (Roesch et al. 2018). This static reuse reduces peak device memory requirements to the working set size of concurrently active tensors, avoiding runtime allocation overhead and fragmentation.

Memory hierarchy placement optimizes the staging of data between external memory (HBM or GDDR) and on-chip scratchpad memories (SRAM or shared memory). External memory bandwidth is a scarce resource; moving intermediate tiles back and forth across the memory bus creates memory-bound stalls. Compilers synthesize tiling parameters that guarantee intermediate blocks remain resident in on-chip SRAM until all local compute operations complete, minimizing round-trip traffic across the memory bus.

With memory layouts fixed and buffer addresses assigned, the compiler schedules operation execution across hardware compute resources.

11.9.6 Computation scheduling

Computation scheduling determines the execution timeline and hardware resource binding for operations in the compiled graph (Zheng et al. 2020). Modern accelerators feature deep hardware pipelines, multiple independent compute engines, and dedicated DMA copy engines. Effective scheduling coordinates these resources to maximize hardware occupancy, preserve data dependencies, and prevent pipeline stalls (Chen et al. 2018).

11.9.6.1 Implementation in AI compilers

Compilers implement scheduling across three distinct architectural dimensions:

Task partitioning maps computational subgraphs onto the target's physical execution units. On GPUs, computations are partitioned into grids of thread blocks dispatched across Streaming Multiprocessors (SMs). On TPUs, matrix multiplications are partitioned into tile sequences matched to the physical dimensions of systolic arrays (e.g., 128×128 or 256×256 processing elements) (Norrie et al. 2021). On CPUs, tasks are split into thread pools matching physical cores and vectorized to fill SIMD registers.

Execution ordering resolves dependencies while minimizing live memory pressure. Operations that depend on prior results must wait for producer kernels to complete, but independent branches—such as multi-head attention projections or parallel residual paths—can execute concurrently or interleave to maintain uniform compute utilization. Compilers analyze the dependency graph to schedule operations in an order that keeps compute pipelines full while keeping activation lifetimes short, preventing buffer bloat.

Asynchronous overlap hides memory latency by executing data transfers concurrently with compute kernels. Accelerators feature dedicated DMA engines that operate independently of arithmetic pipelines. Compilers exploit this architectural separation by scheduling memory transfers (such as host-to-device weights or inter-accelerator halo exchanges) in separate hardware queues or CUDA streams, overlapping transfer latency behind long-running compute operations (Chen et al. 2018; NVIDIA 2024c). When compute and transfer overlap completely, data movement costs are hidden behind arithmetic execution.

Poor scheduling introduces execution bubbles where arithmetic units sit idle waiting for kernel dispatches or serialized data copies. Synchronizing kernels via explicit compiler-generated events and stream schedules ensures the hardware sustains continuous execution without CPU driver intervention (NVIDIA 2024c; Zheng et al. 2020).

11.9.7 Code generation

With scheduling complete, the final compilation stage translates the scheduled execution plan into target-specific machine instructions. Code generation applies traditional backend compiler passes—instruction selection, register allocation, instruction scheduling, and peephole optimization—tailored to accelerator architectures.

Instruction selection for ML targets must emit instructions that directly engage specialized matrix ISA extensions. On NVIDIA GPUs, code generators emit PTX instructions such as `mma.sync.aligned` to invoke Tensor Cores directly, as shown in listing 11.13. On Intel CPUs with Advanced Matrix Extensions (AMX), the backend targets tile-multiply instructions operating on 2D register tiles. On Arm architectures with the Scalable Matrix Extension (SME), the target is outer-product accumulation across scalable vector tiles. A backend that emits generic floating-point vector instructions instead of these matrix extensions leaves the primary compute engines idle, reducing effective throughput by an order of magnitude.

For CPUs and GPUs, compilers emit machine code or optimized assembly; for TPUs, field-programmable gate arrays (FPGAs),[33] and proprietary NPUs, the output is often an optimized bytecode or binary execution package executed by a specialized hardware runtime.

[33] **Field-programmable gate array**: "Field-programmable" means the logic fabric is configurable after manufacturing, contrasting with fixed-function ASICs. FPGAs can improve performance for latency-sensitive data center services by implementing custom pipelines matched to a particular workload (Putnam et al. 2014). This reconfigurability makes FPGAs attractive for rapidly evolving ML architectures where committing to an ASIC risks obsolescence, but the requirement for hardware description languages (Verilog/VHDL) and compilation times measured in hours creates a productivity barrier that limits adoption to deployments where the efficiency benefit justifies the engineering cost.

11.9.8 From compilation to runtime

A compiler transforms a high-level model into an execution plan tailored to target hardware, but that plan embeds static assumptions about runtime conditions: tensor shapes, workspace allocation limits, device capabilities, and execution concurrency. Ahead-of-time (AOT) compilation optimizes graph structures, fusion boundaries, and memory layouts for fixed shapes and known hardware configurations. While just-in-time (JIT) compilation can compile specialized variants dynamically as new shapes arrive, each compiled kernel still executes a fixed execution plan.

Production AI serving environments, however, operate under dynamic constraints that differ from static compiler assumptions. Request arrival rates fluctuate, batch sizes vary dynamically, multiple inference workloads compete for shared accelerator memory, and thermal throttling can reduce clock frequencies below rated specifications. Serving runtimes manage these variations through dynamic request batching, multi-stream concurrency, admission control, and execution engine selection, rather than regenerating kernel fusion or tiling strategies on the fly. Dynamic batching, admission control, and service-level objectives are system-level engineering challenges of the serving layer; the immediate systems concern is how the accelerator runtime dispatches compiled execution plans efficiently once a batch reaches the hardware.

11.10 Runtime Support

An AI runtime bridges static graph compilation and the dynamic realities of accelerator execution. Compilers optimize and lower computational graphs under fixed structural assumptions, but production systems encounter variable batch sizes, fluctuating sequence lengths, and constrained memory budgets. The runtime executes compiled engines while managing input-dependent tensor shapes, device-memory buffers and workspace, asynchronous hardware streams, and precompiled execution profiles. In production inference stacks such as NVIDIA TensorRT, the offline builder optimizes the network and benchmarks candidate tactics to construct an execution engine, while the

runtime engine binds input buffers, selects valid optimization profiles, and dispatches kernels to the accelerator (NVIDIA Corporation 2026a, 2026b).

This execution lifecycle spans three core responsibilities. First, the runtime selects and dispatches precompiled kernel variants matched to runtime tensor dimensions and hardware capabilities. Second, it manages device memory through arena allocators and static buffer reuse, reducing repeated allocation and associated host-device synchronization overhead. Third, in distributed environments, framework runtimes coordinate cross-device execution boundaries, as demonstrated in pipeline-parallel frameworks such as GPipe (Huang et al. 2019) and automated device-placement systems (Mirhoseini et al. 2017). While serving schedulers oversee request batching and admission control across clusters, the accelerator runtime governs local execution on the device.

Comparing AI runtimes to traditional software execution environments clarifies why machine learning workloads demand specialized execution strategies.

11.10.1 ML runtime architecture

General-purpose runtimes manage threads, preemptive tasks, heap allocations, and asynchronous I/O across arbitrary control-flow paths. In contrast, an AI runtime operates over dataflow graphs composed of multidimensional tensors, pre-scheduled kernel grids, device memory arenas, and hardware command queues.

The architectural differences between traditional software runtimes and AI execution environments are contrasted in table 11.22.

Table 11.22: Runtime Execution Models: AI runtimes add tensor-, graph-, and accelerator-aware resource management to conventional runtime services. The exact boundary among framework, compiler runtime, device library, and serving scheduler varies by stack.

Aspect	**Traditional runtime**	**AI runtime**
Execution Model	Functions, threads, tasks, events, and asynchronous I/O	Tensor graphs, kernels, streams, and device events
Task scheduling	Threads and tasks across processor resources	Dependency-aware kernel and transfer dispatch
Memory Management	Objects, stacks, heaps, pools, and virtual memory	Tensor buffers, workspaces, device memory, and buffer reuse
Optimization Priorities	Responsiveness, throughput, locality, and resource sharing	Shape compatibility, transfer overlap, reuse, and utilization
Adaptability	Dynamic scheduling and allocation within program semantics	Runtime choices within compiled graphs and available kernels
Target Hardware	CPUs and heterogeneous devices	CPUs, GPUs, TPUs, and custom accelerators

Tensor memory management represents a primary architectural divergence. In traditional systems, general-purpose allocators manage variable-sized heap objects using runtime metadata. Traditional device allocation calls such as `cudaMalloc` can introduce costly driver work and synchronization. AI runtimes reduce repeated allocation overhead by combining lifetime analysis with pooled or arena allocation. For fixed-shape graphs, the compiler computes static memory offsets, mapping non-overlapping intermediate tensors to shared physical buffers. For variable-shape tensors, the runtime can carve device-memory buffers from preallocated pools, avoiding repeated driver allocation calls within the reserved capacity.

Hardware execution environments also impose physical operating boundaries that diverge from offline benchmarking setups.

Production environments introduce physical constraints that differ from offline benchmarks. For example, an NVIDIA A100 SXM accelerator operates with a maximum TDP of 400 W, whereas the A100 PCIe variant is constrained to a 300 W TDP envelope. Because sustained clock frequencies scale with available power and thermal dissipation, a compiled kernel tuned for an SXM module may experience thermal throttling or fail to meet latency bounds on a PCIe card. The runtime cannot re-synthesize loop structures or change thread-block dimensions dynamically in response to thermal constraints; it must execute within precompiled profiles while the serving system handles request batching and admission control.

11.10.2 Dynamic kernel execution

While static compilation generates high-performance machine code for fixed shapes, production inference requires executing requests with dynamic dimensions. In language models, input sequence length S and batch size B vary across requests. Recompiling an execution plan for each unexpected shape can violate a request deadline: exploring loop unrolling, shared-memory tiling, and register allocation requires seconds or minutes, while serving deadlines operate on millisecond timescales.

Profile-based inference stacks such as TensorRT address this mismatch using precompiled optimization profiles. An optimization profile establishes valid dimension bounds (minimum, optimum, and maximum shapes) for dynamic tensor axes. During engine construction, the builder selects tactics under workspace and shape constraints. Execution contexts require device-memory buffers and workspace appropriate to the selected profile; this workspace is distinct from a kernel's on-chip shared memory. At execution time:

- When incoming tensor shapes match an active profile, the runtime binds tensor pointers to preallocated buffers and launches the corresponding precompiled kernels without JIT overhead.
- When input dimensions fall outside all compiled profiles, the runtime cannot synthesize new tiling strategies on the fly; the serving system must either pad inputs to a valid dimension boundary, split the batch, or route the request to a slower, general-purpose fallback kernel.

↪ How serving runtimes form batches of varying size under latency deadlines is developed in section 13.7.3.

Dynamic request processing also strains the host-to-accelerator interconnect. If input transfers proceed synchronously, accelerator compute engines sit idle during data movement. To overlap communication with computation, the runtime employs asynchronous streams and double buffering. By staging input tensors in page-locked (pinned) host memory, dedicated DMA copy engines transfer batch $k{+}1$ across PCIe or NVLink concurrently with the execution of batch k on accelerator cores. This pipelining hides transfer latency provided the compute duration exceeds the transfer duration; if memory movement dominates, the accelerator pipeline stalls.

11.10.3 Runtime kernel selection

An operation may have multiple kernel implementations, termed tactics in TensorRT. The builder selects tactics while constructing an engine; other stacks may retain variants for runtime dispatch. The available choice depends on tensor dimensions, the compiled plan, and permitted device-memory workspace.

Operational batch size changes the available reuse and parallelism. At $B = 1$ in weight-streaming autoregressive decoding, GEMV kernels often target memory throughput. Larger prompt-processing batches can favor GEMM kernels using Tensor Core paths, register tiles, and shared memory. The transition depends on dimensions, precision, traffic, and hardware balance; no fixed batch threshold establishes compute-bound execution.

Device-memory workspace availability further constrains tactic selection. When a GEMM has a small output dimension but a large reduction dimension K, standard tiling leaves the accelerator underused because too few thread blocks are launched to occupy all Streaming Multiprocessors (SMs). In this regime, the builder provides a split-K tactic that partitions the reduction dimension across separate thread blocks running concurrently on different SMs. However, split-K execution requires an auxiliary workspace buffer in device memory (HBM) to store partial reduction sums before a final accumulation pass. If device memory is constrained, the execution plan must respect the workspace ceiling. A stack with validated variants may choose a non-split kernel, trading parallel occupancy for lower workspace; a fixed engine instead needs sufficient reserved workspace or a different prebuilt plan.

Runtime selection remains strictly bounded by the numerical formats established during compilation. In mixed-precision training and inference systems such as Megatron-LM, executing matrix multiplications in FP16 or BF16 on Tensor Cores with FP32 accumulation is a numerical contract of the chosen recipe, validated prior to deployment (Shoeybi et al. 2019). The runtime does not monitor loss values or dynamically alter precision; it only chooses among the validated tactics that implement the specified numerical precision.

11.10.4 Kernel scheduling and utilization

Kernel dispatch represents the final stage of runtime execution, coordinating host-side task queues with device-side execution.

The software runtime manages host-side command queues (such as CUDA streams or TPU command rings) and records hardware events or semaphores to enforce dataflow dependencies asynchronously. When independent operations—such as parallel multi-head attention projections—exhibit no mutual data dependencies, the runtime dispatches them onto separate streams to enable concurrent execution on the accelerator. Once submitted to the device queue, however, execution control transfers entirely to hardware schedulers. On a GPU, the hardware warp scheduler and work distribution engine assign thread blocks to SMs based on register file availability, shared memory capacity, and maximum warp occupancy; the software runtime cannot schedule individual threads or dictate L1/L2 cache replacement policies. Similarly, TPU runtimes submit macro-instructions to on-chip command queues, while hardware sequencers feed systolic matrix units and vector units (Jouppi et al. 2017).

Memory management directly reinforces accelerator utilization. Traditional device allocation calls during request execution can introduce synchronization and interrupt the pipeline. Preallocated device-memory arenas reduce this overhead. With a known execution schedule, intermediate tensors with nonoverlapping lifetimes can share physical addresses, avoiding repeated driver allocation on the inference hot path while preserving every live value.

A low achieved rate alone cannot distinguish runtime gaps from a memory bottleneck. Consider three supplied illustrative profiles of low-intensity kernels, not measurements from a particular device. In the first, the timeline shows continuous kernel execution and HBM traffic close to the sustainable bandwidth ceiling. Reducing transferred bytes or increasing reuse addresses that observed limit. In the second, kernels remain active but dependent loads produce long stalls while HBM traffic stays well below its ceiling. More bandwidth alone need not help: independent requests or a different access schedule must expose enough concurrency to hide the latency. In the third, short kernels alternate with device-idle gaps while the host submits work. Reducing dispatch and synchronization overhead addresses those gaps; changing the arithmetic instruction rate does not.

The discriminating evidence is the timeline together with traffic and stall observations. If added independent requests raise bandwidth and shorten load stalls, they support the latency hypothesis; if bandwidth is already sustained near its ceiling, traffic reduction is the more direct lever. The roofline bounds the resource opportunity, while the profile identifies which mechanism leaves the kernel below it.

Even with aggressive kernel fusion, optimal tactic selection, and zero-allocation memory reuse, the execution capacity of a single accelerator is physically bounded by its silicon area, thermal dissipation limit, and high-bandwidth memory capacity. Scaling beyond these physical boundaries requires distributing computation across multiple accelerators.

Consider the estimated 3.14×10^{23} floating-point operations associated with training GPT-3 (Brown et al. 2020). As a scale comparison, divide that historical operation count by a later H100's peak FP8 rate of 1.98 PFLOP/s. The arithmetic lower bound is about 5 years on one device, and 8.4–12.6 years under the illustrative 40–60 percent utilization range (Choquette 2023). This is not a reconstruction of GPT-3 training: the precision, hardware, communication, and workload are counterfactual. It simply shows why model-scale compute or service-scale request volume can exceed the useful capacity of one accelerator.

11.11 Multi-Chip Scaling

A single H100 has a large FP8 peak, yet a training-time or throughput target may still require many accelerators. The techniques covered earlier remain the foundation for multi-chip scaling; each device still needs an efficient dataflow. The new lesson is that communication adds further hierarchy boundaries. Moving data through on-chip storage, HBM, an intra-node link, and a cluster fabric generally increases latency and energy, so scaling is not merely a question of adding chips.

When single-accelerator capacity proves insufficient, the design problem shifts from feeding one chip to choosing which communication boundary the workload can tolerate. Practitioners encounter these boundaries in production even when most optimization work remains inside a single accelerator.

11.11.1 Multi-chip scaling approaches

Large AI systems scale beyond individual accelerators by moving the communication boundary outward, and each boundary changes the dominant trade-off. The sequence begins inside the package. Chiplet-based architectures partition large designs into smaller, modular dies interconnected within one package, bypassing manufacturing limits of monolithic chips while preserving relatively low communication latency. The next boundary is the node: multi-accelerator servers connect several chips through board- or server-level interconnects. Each accelerator has dedicated memory and compute resources, so workloads split through data parallelism (each accelerator processes different batches) or model parallelism (either pipeline parallelism, where different accelerators handle sequential network layers, or tensor parallelism, where individual matrix multiplications are sliced across devices within high-bandwidth interconnect domains). High-bandwidth intra-node interconnects can enable efficient gradient synchronization, though realized performance depends on topology and collective communication efficiency.

Beyond the node, cluster fabrics add another communication boundary with different topology and coordination costs. Wafer-scale integration takes the opposite approach by keeping more computation on one device. These designs extend the same locality trade-off; the decision within a single server is whether partitioning saves enough computation or device-memory capacity to justify intra-node transfers.

11.11.2 Why scaling introduces new constraints

The transition from single-chip to multi-chip architectures introduces communication overhead and other constraints that reshape optimization. Communication, synchronization, load balance, and parallel work per device can all limit scaling. Amdahl's Law[34] provides one first-order model for exposed work that does not shrink with additional devices. For hundred-billion-parameter models, an unsharded full-gradient payload can occupy hundreds of gigabytes per step, and an AllReduce collective must aggregate it; precision, sharding, compression, and the collective algorithm change the bytes each device transfers.

The first-order quantity is the gradient payload. For a model with P parameters, that payload is roughly the parameter count multiplied by the bytes stored for each gradient element before optimizer state, padding, and protocol overhead enter. Scaling therefore improves only when the saved compute time exceeds the time to move this payload through the chosen interconnect and collective algorithm.

This overhead explains why large accelerator counts can show diminishing returns unless the system reduces exchanged data, overlaps communication with useful computation, or chooses a better parallelization pattern. The memory model also changes: separate accelerator memories are commonly managed explicitly, and collective operations establish the required synchronization rather than presenting one fully coherent memory image across the cluster. Coherent links exist in some systems, but their scope and cost are architecture-dependent.

Within a multi-accelerator server, each partition must still preserve useful locality. Moving intermediate values or gradients across device boundaries adds traffic that a single-device roofline omits. The engineering test compares saved computation with exposed communication, accounting for overlap and the actual directional link bandwidth.

Data center scaling and edge deployment represent opposite ends of a deployment spectrum, yet they share the same core principles. Data center scaling coordinates many high-throughput accelerators, while edge scaling fits useful AI into a few constrained watts. Both cases require matching workload characteristics to hardware capabilities while minimizing data movement. The principles of compute specialization, memory hierarchy optimization, and workload mapping apply at both scales; only the constraints differ. Data centers optimize for aggregate throughput within power budgets measured in megawatts; edge devices optimize for responsiveness within tight battery and thermal envelopes. The same vision model that runs comfortably in a data center may need a radically different mapping strategy on a smartphone or microcontroller.

[34] **Amdahl's law (scaling limit)**: section D.2.3.1 formalizes the bound for a fixed nonaccelerated fraction. If 5 percent of a fixed workload remains exposed and unaffected by device count, that model caps speedup at 20×. Distributed training is more complicated because communication can overlap computation and its cost can itself change with scale, but the example explains why exposed synchronization must be reduced or hidden.

11.12 Heterogeneous SoC Design

Edge, mobile, and automotive deployments operate under strict physical constraints: sealed enclosures without active fans limit thermal dissipation to a few watts, while battery capacity caps total

energy consumption. To deliver machine learning capabilities within these boundaries, heterogeneous SoC architectures integrate general-purpose CPU cores, GPU shader arrays, digital signal processors (DSPs), and specialized neural processing units (NPUs) onto a single silicon die sharing a unified memory subsystem. The workload mapping problem introduced for discrete accelerators persists, but it expands to include hardware operator coverage, cross-engine transfer overheads, real-time deadlines, and dynamic thermal throttling.

11.12.1 Mobile SoC architecture evolution

Mobile system-on-chip architectures implement heterogeneous computing by coupling distinct execution engines—multicore CPUs, wide SIMD GPU shader cores, fixed-point DSPs, and dedicated neural processing units (NPUs)[35]—over an on-chip network connected to shared low-power DRAM (LPDDR). Workload partitioning routes sensory and perceptual tasks to the compute engine best aligned with their arithmetic structure: continuous audio feature extraction and keyword spotting run on low-power DSPs, dense convolution and matrix-multiplication layers execute on systolic or tensor-array NPUs, parallel graphics and image postprocessing map to GPUs, and sequential control flow runs on the CPU. Meeting real-time frame deadlines without triggering aggressive thermal throttling requires scheduling execution across these engines while respecting their differing power profiles.

35 **NPU (neural processing unit)**: The NPU's specialized matrix engines are optimized for dense tensor operations, providing the hardware basis for the workload distribution described. This specialization creates a critical constraint for the scheduler: any AI operator not mapped to the NPU's supported data paths must "fall back" to a GPU or CPU. This fallback can erase the NPU's energy-efficiency advantage, complicate real-time latency budgets, and contribute to thermal pressure on mobile devices.

> Example 11.5: Heterogeneous microcontrollers
>
> **Scenario**: An embedded smart doorbell must run periodic person detection within a tight memory, energy, and latency budget.
>
> **Diagnosis**: On a supported deployment, a general-purpose Cortex-M path misses the measured energy or latency target. A micro-NPU such as Arm Ethos-U can offload compatible convolution operators, while the CPU retains control flow and unsupported operations.
>
> **Systems lesson**: Micro-NPU specialization can improve the energy and latency of supported operators, but feasibility must be measured for the whole pipeline, including fallbacks and sensor processing.

Modern mobile SoCs use unified memory architectures where all processing units share a common physical address space in LPDDR. Unlike discrete accelerator architectures that incur explicit PCIe transfer latencies, unified memory allows zero-copy buffer passing between the image signal processor (ISP), CPU, GPU, and NPU. However, sharing physical DRAM does not guarantee zero software overhead. If an NPU engine lacks hardware cache snooping, passing an intermediate tensor from the CPU requires explicit cache flushes and invalidations. Furthermore, specialized NPUs often require hardware-specific tensor layouts—such as channels-last (NHWC) with 16- or 32-byte channel alignment to feed vector register lanes—forcing memory-bound transpositions when interfacing with CPU layers structured in channels-first (NCHW) formats.

SoC architects customize these compute blocks according to domain-specific physical constraints. High-end smartphone SoCs emphasize peak burst performance and low idle leakage for interactive tasks such as real-time computational photography and on-device text generation. In contrast, embedded and automotive SoCs prioritize deterministic memory access, hardware fault detection, and bounded worst-case latency to satisfy real-time control constraints.

11.12.2 Strategies for dynamic workload distribution

Partitioning neural network workloads across heterogeneous SoC cores requires matching the arithmetic intensity and memory regularity of each computational graph operator to the microarchitecture best equipped to execute it. Consider a real-time object detection pipeline executing on a mobile SoC with an integrated CPU, GPU, and NPU. The pipeline comprises three consecutive stages: a MobileNet backbone for feature extraction, nonmaximum suppression (NMS) for postprocessing candidate bounding boxes, and a display overlay to render final detections onto a camera feed. The convolutional backbone exhibits regular, deterministic memory access and dense arithmetic suitable for the NPU's fixed-function tensor engine, even though depthwise separable layers exhibit

lower arithmetic intensity than standard convolutions. NMS, by contrast, relies on score-based sorting and conditional intersection-over-union loops evaluated across variable-length candidate lists. This irregular, branch-heavy control flow maps poorly to an NPU's spatial array but executes efficiently on out-of-order CPU cores with branch prediction and hardware caching. Finally, the display overlay requires per-pixel rasterization and alpha blending across a high-resolution frame—an embarrassingly parallel, memory-bandwidth-bound task that maps directly to the GPU's rasterizers and texture units. Splitting the pipeline across the three engines—NPU for feature extraction, CPU for NMS, and GPU for display compositing—is a candidate mapping. Its compute savings must exceed transfer, layout, and synchronization costs before it improves end-to-end latency or energy over a single-engine implementation.

This pipeline partitioning reflects the broader relationship between algorithmic structures and machine architectures. Regular matrix contractions and convolutions map cleanly to systolic or vectorized NPU datapaths, whereas data-dependent control flow maps to CPU execution resources. Transformer attention layers span both paradigms: the projection GEMMs ($\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$) and attention score contractions ($\mathbf{QK}^T$, and the product of the attention weights with $\mathbf{V}$) execute efficiently on NPU matrix units, but intermediate softmax reductions, causal masking, and layer normalizations involve elementwise operations. If the NPU runtime cannot fuse these elementwise operations into on-chip SRAM scratchpads, intermediate tensors must spill to LPDDR, eroding the accelerator's throughput advantage.

Optimal processor assignment is dynamic rather than static. As device thermal conditions deteriorate or battery reserves drop, the runtime must adapt its scheduling policy. However, migrating an executing model across processor backends carries non-trivial overhead: weights must be re-bound in memory, intermediate buffers may require layout reformatting, and device drivers incur kernel dispatch latency. If the overhead of synchronizing state across processors exceeds the compute savings of offloading, retaining execution on a single suboptimal engine can yield lower total latency and energy than continuous cross-engine migration.

Shared memory subsystems introduce cross-engine interference during simultaneous execution. Because the CPU, GPU, NPU, camera ISP, and display engine share a single LPDDR memory bus, the SoC memory controller enforces Quality of Service (QoS) priorities to satisfy real-time hardware deadlines. The display engine and camera ISP must receive guaranteed memory bandwidth and bounded latency to prevent display flicker and sensor FIFO overflow; the memory controller consequently prioritizes display and camera transactions over NPU and GPU traffic. During memory-intensive phases—such as autoregressive language model decoding where model weights must be read from DRAM for every generated token—high-priority ISP or display traffic reduces effective NPU memory bandwidth, increasing token generation latency.

11.12.3 Power and thermal management

Fanless mobile devices operate within strict continuous power budgets and thermal envelopes, typically dissipating no more than 3 to 5 watts sustained across the entire chassis before touch temperatures exceed safety thresholds. Although transient bursts can draw 10 to 15 watts, continuous inference rapidly saturates the thermal capacitance of the device, heating the silicon junction and requiring active power management across the SoC.

Heterogeneous SoCs manage power across distinct silicon partitions using **dynamic voltage and frequency scaling (DVFS)**. Because dynamic power scales quadratically with supply voltage and linearly with operating frequency ($P_{\text{dynamic}} \propto V^2 f$), dropping the clock frequency allows lowering the core voltage, yielding substantial power reductions. On a heterogeneous SoC, CPU clusters, GPU shader cores, and NPU engines occupy separate power and voltage domains. Increasing the voltage and frequency of the NPU to meet a tight latency deadline consumes electrical current and thermal headroom that would otherwise be available to the CPU or GPU. System power governors continuously adjust these operating points based on workload queues, temperature sensor readings, and frame deadline targets.

When sustained computation exceeds the chassis dissipation rate, the platform initiates thermal throttling. Because specialized NPUs achieve higher energy efficiency (TOPS/W) for tensor math than general-purpose cores, offloading matrix workloads to a CPU or GPU during a thermal event often increases total energy dissipation, exacerbating thermal accumulation. Instead, runtimes man-

age thermal limits by capping clock frequencies, reducing inference duty cycles (such as throttling a perception model from 30 frames per second to 15 frames per second), or dynamically switching to a smaller, pruned model checkpoint. Task migration is applied selectively to move non-tensor preprocessing away from localized silicon hot spots on the die.

Battery state further shapes runtime scheduling. When battery charge drops below critical thresholds or the device enters a low-power mode, the system scheduler trades inferencing fidelity for battery life by selecting prevalidated lower-precision or sparse model variants, or by batching asynchronous inference requests to maximize the time cores spend in deep low-power sleep states.

11.12.4 Automotive heterogeneous AI systems

Automotive computing platforms expose a stricter form of the shared-resource constraint. A perception pipeline with a hard deadline may share the SoC and memory system with a cabin assistant or infotainment model. If the convenience workload streams weights from DRAM without a bandwidth limit, it can delay the perception pipeline even when the two use different compute engines. Such deployments require resource isolation and validation under concurrent load; average inference throughput alone cannot establish deadline compliance. The hardware lesson is the interference boundary: assigning work to separate engines does not isolate workloads that still share memory bandwidth.

11.12.5 Software stack challenges

Heterogeneity shifts the primary systems challenge from silicon design to software coordination. Standard programming models like OpenCL and Vulkan provide portable low-level execution abstractions across compute devices, but they do not eliminate the need for microarchitecture-specific kernel tuning. Machine learning inference runtimes partition computation graphs through hardware-specific delegates or backends targeting CPUs, GPUs, DSPs, and NPUs. When a model contains an operator unsupported by an NPU delegate—such as a custom activation, dynamic tensor shape, or non-standard reduction—the runtime splits the graph into subgraphs, inserting fallback paths to the CPU or GPU. Each boundary crossing requires synchronizing execution and translating data representations, transforming an algorithmic graph into a fragmented sequence of hardware dispatches.

Shared memory subsystems require explicit cache and memory management across non-coherent execution domains. While CPUs maintain cache coherence across their own cores via hardware snooping protocols, mobile NPUs and DSPs frequently interface with system memory as non-coherent DMA bus masters. When the CPU writes input tensors to cached DRAM and hands control to an NPU, software must explicitly flush the CPU's dirty cache lines to DRAM before NPU execution begins. Similarly, upon NPU completion, the CPU must invalidate its own cache lines over the destination memory range to prevent reading stale cached data. If unsupported operators cause an inference graph to ping-pong repeatedly between CPU and NPU, the latency of cache maintenance and intermediate DRAM round-trips can exceed the execution time of the operators themselves.

Runtimes navigate this scheduling complexity using ahead-of-time compilation, empirical profile databases, and dynamic dispatch heuristics. Device telemetry—monitoring frame completion times, core temperatures, and battery drain—informs runtime decisions to throttle clock frequencies or swap execution delegates. However, dynamic adaptation cannot replace deterministic graph lowering for latency-sensitive applications; deployment requires validating the compiled, lowered partition on physical target silicon under worst-case thermal and memory contention.

Heterogeneous acceleration demonstrates that system efficiency is fundamentally a coordination problem rather than an inventory of specialized execution blocks. In the D·A·M taxonomy, an SoC represents a composite Machine layer whose components exhibit divergent memory hierarchies, arithmetic strengths, and thermal envelopes. The iron law's data term and the energy-movement principle (7) together make data movement the governor of both runtime and energy: when software coordination fails to maintain tensor locality or induces frequent cross-engine fallbacks, energy dissipation is dominated by interconnect and DRAM traffic rather than arithmetic. Maximizing performance on heterogeneous SoCs requires minimizing data movement across domain boundaries while matching each algorithmic operation to the specialized datapath architected to accelerate it.

11.13 Hardware Sustainability

At fleet scale, energy per useful inference becomes an important hardware-selection criterion. Operational emissions depend on workload throughput, measured system power, utilization, cooling overhead, and the electricity mix; embodied emissions add another lifecycle cost. Peak performance per watt is therefore a screening metric, not a substitute for end-to-end energy per request. The following scenario shows how useful throughput and a common power boundary determine the fleet needed for the same inference service.

Napkin Math 11.10: An illustrative operational-energy comparison

Problem: An inference service must complete 1 billion requests per day. Which of two hypothetical server platforms uses less operational energy for that service?

Supplied observations: For the same model, input distribution, accuracy, and latency target, a CPU server sustains 100 requests/s, and an NPU-equipped server sustains 1,000 requests/s. Their whole-server wall powers are 100 W and 200 W, respectively, including hosts and attached accelerators. These are hypothetical observations for this exercise, not product benchmarks. Assume requests arrive steadily through the day. Both fleets operate at no more than 80 percent of the supplied throughput to reserve capacity for arrival-rate variation. Assume the listed wall power throughout the 24 h day, including spare capacity. Facility cooling and embodied emissions are outside this operational boundary.

Calculation:

1. **Size the fleets from useful work.** Multiplying useful throughput by the operating fraction and the day length gives 6,912,000 requests per CPU server per day and 69,120,000 per NPU-equipped server. Divide the daily target by each capacity and round up to whole servers. The service requires 145 CPU servers or 15 NPU-equipped servers.
2. **Account for the provisioned fleet.** Server count multiplied by wall power and day length yields 348 kWh per day for CPUs and 72 kWh for NPUs. Dividing each fleet's energy by the same completed-request target gives 1.2528 J per request and 0.2592 J per request. These values include capacity headroom and integer fleet rounding.
3. **Translate the energy difference.** At a common grid factor of 0.429 kg/kWh, the NPU-equipped fleet avoids approximately 43.2 t of operational CO_2 emissions per year at the stated system boundary.

Decision: Choose the NPU-equipped fleet if the objective is to minimize operational energy under these supplied conditions. Each server draws more power, but its greater useful throughput reduces the required fleet enough to lower joules per completed request. This conclusion depends on meeting the same quality and latency targets; a faster peak arithmetic rate alone would not establish the advantage. A procurement decision must additionally compare acquisition costs, software support, and embodied emissions.

The sustainability perspective reinforces a central theme: hardware selection is not determined by peak speed alone. Energy per useful result, carbon intensity, embodied impact, and total cost of ownership belong beside latency, throughput, and memory capacity. The remaining step is to identify the misconceptions that cause teams to choose the wrong hardware path.

11.14 Fallacies and Pitfalls

Hardware acceleration exhibits counterintuitive performance characteristics because headline specifications frequently mask workload-specific bottlenecks. The fallacies and pitfalls here capture selection, scaling, and optimization errors that leave expensive compute resources underused.

Fallacy: *More specialized hardware always provides better performance than general-purpose alternatives.*

Specialized accelerators achieve high efficiency only when workload access patterns match underlying execution datapaths: the core tenet of algorithm-machine co-design. The Roofline Model (section 11.6) compares bandwidth and compute ceilings for a specified memory interface and

execution path. The A100 reference tensor path has a ridge point of 153 FLOP/byte; below that ridge, increasing its 312 TFLOP/s arithmetic peak does not raise the modeled bandwidth ceiling. A transformer attention softmax with $I = 2$ FLOP/byte–5 FLOP/byte is capped at 4.1 TFLOP/s–10.2 TFLOP/s (3.3 percent of tensor peak) on an A100. On a CPU, whose ridge point sits around 10 FLOP/byte–20 FLOP/byte, the same traffic model places the kernel below the reference ridge, and its bandwidth ceiling represents 10 percent–50 percent of the CPU's compute capacity. These fractions are modeled bandwidth bounds, not measured utilization or a latency comparison between devices. Softmax also has vector, reduction, and special-function ceilings; neither device's tensor or general arithmetic peak alone diagnoses its execution. When workloads exhibit irregular memory access, tiny batch sizes, or dynamic control flow, massively parallel execution pipelines stall on memory latency and thread divergence. General-purpose CPUs with large cache hierarchies, out-of-order execution, and branch predictors often deliver superior latency and resource efficiency under these conditions. Hardware selection requires matching workload arithmetic intensity and dataflow regularity to architectural ridge points rather than assuming specialization guarantees speed.

Pitfall: *Ignoring memory bandwidth limitations when selecting acceleration strategies.*

Procurement decisions frequently target peak arithmetic throughput while overlooking memory interface constraints. Off-chip data movement imposes severe performance and energy penalties: fetching an operand from DRAM consumes roughly 640 pJ compared to 0.5 pJ for a small, register-adjacent L1 SRAM access, the cheapest tier noted in the caption of figure 11.8. Consider an accelerator providing 300 TFLOP/s of compute alongside 2 TB/s of memory bandwidth, setting a ridge point of 150 FLOP/byte. A normalization kernel such as LayerNorm with an arithmetic intensity of 1.5 FLOP/byte has a bandwidth ceiling of 3 TFLOP/s (1 percent of compute peak). Under the stated HBM streaming assumption, memory bandwidth supplies this upper bound; LayerNorm's reduction execution and available parallelism can impose a lower one. Adding compute units alone does not raise this bandwidth ceiling; measured traffic and execution determine whether more bandwidth or less movement would improve runtime.

Fallacy: *Hardware acceleration benefits scale linearly with additional accelerators.*

Distributed training need not deliver $8\times$ throughput on eight accelerators because exposed inter-chip communication can reduce parallel efficiency. Synchronizing gradients across ranks via ring AllReduce requires each rank to transfer $2(N_{\text{GPU}} - 1)/N_{\text{GPU}}$ times the model gradient volume for N_{GPU} accelerators, because the ring first reduce-scatters and then all-gathers, so each rank sends and receives about twice the payload: 1.75× payloads across eight devices. With an interconnect such as NVLink providing 600 GB/s bidirectional bandwidth (half that per direction), transmitting a 1 GB gradient payload consumes 5.83 ms. For a model with a 50 ms compute step, this data exchange adds 11.7 percent communication overhead before counting software collective latency. Without compute-communication pipelining, an eight-accelerator node reaches only 7.2× speedup (89.6 percent parallel efficiency). Stragglers, barrier synchronizations, and collective protocol overhead widen the gap between linear expectations and physical cluster scaling.

Pitfall: *Planning accelerator capacity from peak FLOP/s specifications.*

Peak FLOP/s measures arithmetic unit capacity under ideal instruction mixes and continuous operand availability, not delivered application throughput. Realized performance depends on kernel arithmetic intensity, matrix dimensions, memory coalescing, and runtime launch overhead. The Roofline Model (section 11.6) bounds the gap between peak compute and bandwidth-limited execution. On an A100 GPU with a 312 TFLOP/s peak, an illustrative dense-transformer scenario supplies a sustained range of 120 TFLOP/s–180 TFLOP/s (40 percent–60 percent model FLOPs utilization). A contrasting illustrative sparse-recommender scenario supplies 10 TFLOP/s–30 TFLOP/s (3.2 percent–9.6 percent utilization). These supplied ranges illustrate different useful-compute fractions; they are not measurements or predictions for every model. Sparse embedding lookups can limit throughput through irregular memory access, but a trace is needed to establish the actual cause. Sizing cluster capacity from peak FLOP/s creates severe under-provisioning; capacity planning must rely on measured sustained throughput or roofline-bounded kernel projections.

Fallacy: *Any FLOP/s rating can estimate a low-precision workload.*

Accelerators have separate datapaths for different precisions, and the peak throughput varies dramatically across them. An H100 delivers roughly 989 TFLOP/s in FP16 tensor operations but only about 67 TFLOP/s in FP32 CUDA-core operations: a roughly 15× gap within the same chip (Choquette 2023). Dividing achieved BF16 model FLOP/s by the smaller FP32 peak inflates the reported utilization, while using that FP32 peak to estimate ideal training time overestimates the time. Matching against the wrong roofline misclassifies kernels as compute-bound when they are memory-bound (or vice versa). Useful model FLOP utilization divides the model's useful operations per elapsed second by the matching supported precision and execution-path peak. Hardware busy time is a different metric: an active device can spend time on transfers, redundant work, or fallback execution. Quote the denominator and precision explicitly.

Pitfall: *Deploying small-batch inference workloads on high-compute accelerators.*

Small-batch dense inference can use only a small fraction of arithmetic peak because single-sample execution lacks cross-sample parameter reuse. For a dense linear projection with widths $d_{\text{in}} = d_{\text{out}} = 2048$, a single request ($B = 1$) is modeled as streaming the entire weight tensor from device memory to execute a matrix-vector product, yielding an arithmetic intensity of only 1 FLOP/byte. At batch size $B = 256$, weight reuse amortizes memory transfers across tokens, elevating arithmetic intensity to 204.8 FLOP/byte. At $B = 1$, memory bandwidth bounds performance at 2.04 TFLOP/s on an A100 and 0.32 TFLOP/s on a T4. Although the T4 provides a lower peak of 65 TFLOP/s with a ridge point of 203.1 FLOP/byte, the two batch-1 ceilings differ by 6.4×, which is exactly their memory-bandwidth ratio; the A100's 4.8× advantage in peak compute contributes nothing. The A100 costs 6× as much as the T4, so their modeled batch-1 throughput ceilings per dollar are nearly the same. Actual procurement requires measured latency, throughput, memory fit, and the intended request mix. System architects must align accelerator selection with operational batch sizes and latency objectives.

Fallacy: *Vendor-specific optimizations have no long-term portability cost.*

Optimizing exclusively for a single vendor runtime captures immediate kernel efficiency but introduces severe architectural lock-in (section 11.9). Codebases saturated with proprietary device intrinsics, vendor-specific assembly, and non-standard memory layouts require months of engineering labor to port when newer, more cost-effective accelerators emerge. This technical debt delays hardware transitions and prevents multi-vendor procurement strategies. To preserve flexibility, systems must isolate vendor-tuned micro-kernels behind hardware abstraction layers or target intermediate compiler representations (such as Triton or MLIR). Preserving portable execution paths allows infrastructure to track hardware innovation without sacrificing the optimization benefits of specialized execution units.

Checkpoint 11.4: Feasibility assessment: Can you run it?

Before procuring hardware, validate each physical constraint. These failure modes translate into a concrete feasibility test:

- ☐ **Memory capacity**: use $M_{\text{total}} = M_{\text{weights}} + M_{\text{KV}} + M_{\text{activations}}$ (weights, KV cache, and activation buffers) to list the inputs needed to test $M_{\text{total}} < M_{\text{device}}$ for a 7-billion-parameter Llama model with 14 GB FP16 weights on a 16 GB GPU. What sequence length, concurrency, model dimensions, and runtime workspace information is missing before a complete fit decision?
- ☐ **Bandwidth**: compute the weight-streaming lower bound $T_{\text{token}} = D_{\text{vol}}/\text{BW}$ for a 70-billion-parameter model (140 GB) on 1 TB/s memory bandwidth, then compare the result with a 50 ms latency target. Assume one dense FP16 weight read per generated token; what additional traffic and overhead would a measured latency include?
- ☐ **Compute**: identify the operation count, numerical path, and measured efficiency needed to use $T_{\text{compute}} = O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$. With serialized frame processing, 30 FPS leaves 33.3 ms

per frame for the whole pipeline. How does overlapped processing change the distinction between frame latency and sustained throughput?

- ☐ **Roofline placement**: for an accelerator at roughly 2 TFLOP/s peak and 200 GB/s bandwidth, estimate the ridge-point arithmetic intensity, then place a 10 FLOP/byte kernel against it: how do the two ceilings compare, and would increasing only the compute clock raise the overall bound? What measurements would test the actual bottleneck?

This checklist translates architectural principles into rigorous engineering practice. System evaluation demands a disciplined diagnostic habit: characterize the workload arithmetic intensity, identify the physical bottleneck metric, match execution to the appropriate datapath, and account for interconnect, scaling, and portability costs.

11.15 Summary

Hardware acceleration co-designs compute engines, memory hierarchies, numerical formats, and dataflows. For a chosen memory interface, the iron law compares compute time $O/(R_{\text{peak}}\eta_{\text{hw}})$ with movement time D_{vol}/BW; the roofline restates that competition as $\min(R_{\text{peak}}, \text{BW} \times I)$. End-to-end latency also includes launch, synchronization, communication, and fixed overheads. Accelerators improve efficiency only when supported paths capture reuse locally. Hardware selection therefore begins with the workload's measured or modeled bottleneck, not a vendor peak.

Key Takeaways: Moving data costs more than computing it

- **The Roofline Model bounds attainable performance**: Compare a kernel's arithmetic intensity with the ridge for its memory interface and supported execution path (153 FLOP/byte for the A100 reference tensor path). Below it, bandwidth supplies the tighter modeled ceiling; above it, compute does. Profiling establishes whether the kernel approaches that ceiling or is limited by dependencies, dispatch, or another execution resource.
- **Memory bandwidth constrains performance**: GPU compute capacity has grown faster than memory bandwidth. Many inference phases are memory bound, especially low-batch decoding and embedding lookup, while large-batch GEMMs can remain compute bound.
- **Hardware-software co-design compounds performance**: Matching algorithm patterns to architectural capabilities, such as dense GEMM on systolic or tensor units and supported sparsity on sparse paths, can produce large improvements.
- **Tensor Cores require a compatible path**: Precision, dimensions, layouts, and library support must match the architecture. Batch and shape affect reuse and utilization, but no single batch threshold guarantees peak performance.
- **End-to-end speedup stops at the next bottleneck**: Accelerating one kernel leaves launch, transfer, synchronization, communication, and unsupported work unchanged. Use Amdahl's law (principle 8) and profiling to decide whether further silicon, better mapping, or less movement will improve the complete workload.

An accelerator is not a uniformly faster computer. It provides efficient paths for particular operations and dataflows, and each of those paths is a way of moving fewer bytes per useful operation. Tiling, fusion, stationary dataflows, and systolic execution pay off only when the workload, numerical format, and software path can use them. The diagnostic habit the chapter leaves is therefore an ordering. Place the kernel on the relevant roofline to bound its performance, bound the end-to-end gain with Amdahl's law, and only then choose among more silicon, a better mapping, or less movement. Each step is a prediction that measurement must confirm.

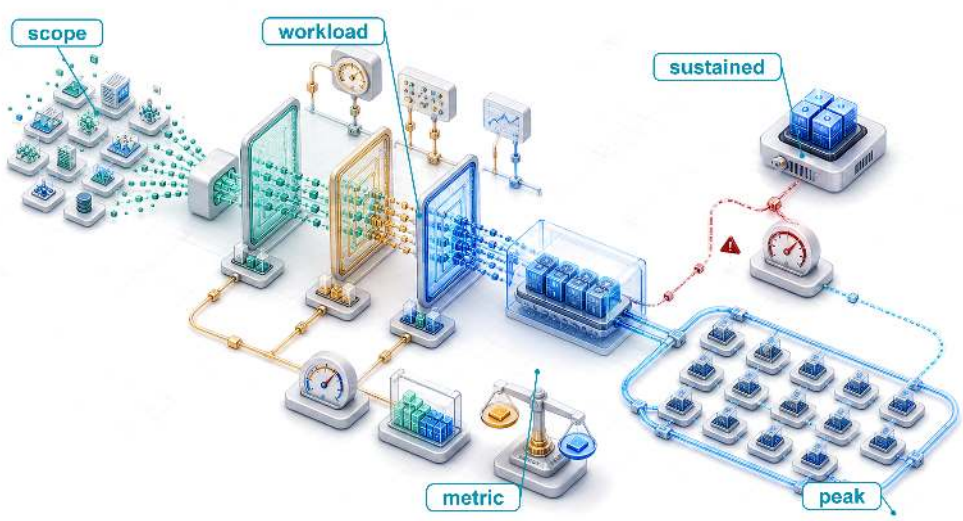
The same habit is what keeps the bitter lesson affordable. Scale keeps winning only while each additional operation gets cheaper in time and energy, and on modern accelerators the operation is rarely the expensive part; the bytes that feed it are.

What's Next: From optimization to validation

Data selection, model compression, and hardware acceleration each end with a favorable local number. Data selection reaches target accuracy on fewer examples, compression produces a smaller artifact, and the roofline places a kernel below a ceiling that the vendor's peak overstates. Each number is a prediction made for one stage in isolation, and the stages interact once they run together on a real device under real load. Chapter 12 takes up that test. It asks which local gains survive deployment-representative conditions, and it supplies the measurement scope, standardized suites, and statistical protocols that let different systems be compared on the same terms.

12

Benchmarking

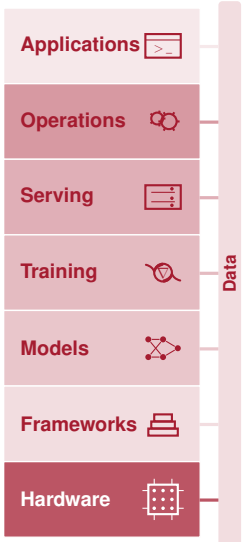


12.1 ML Benchmarking Framework
12.2 Historical Foundations
12.3 System Benchmarking Suites
12.4 Benchmarking Granularity
12.5 Benchmark Components
12.6 Training vs. Inference
12.7 Training Benchmarks
12.8 Inference Benchmarks
12.9 Power Measurement Techniques
12.10 Benchmarking Best Practices
12.11 Model and Data Evaluation
12.12 Production Considerations
12.13 Fallacies and Pitfalls
12.14 Summary

Purpose

How can ML systems be compared fairly when hardware, models, data, and deployment all interact?

Benchmarking brings together decisions already developed through data selection, model compression, and hardware acceleration, then tests whether their gains survive under deployment-representative conditions. Each decision targets one dimension (latency, accuracy, throughput, or energy), but an ML system is the product of all these dimensions simultaneously. A pruned model runs faster on one accelerator but slower on another. A larger batch size improves accelerator utilization but can violate a latency service-level agreement. An edge device may advertise peak throughput that thermal throttling sharply reduces within minutes of sustained load. A speedup measured on a warm, idle machine with synthetic inputs can vanish or reverse once real requests, cold caches, and competing jobs share the hardware. The challenge is not whether a local metric improves but whether the combined system improves under conditions that actually matter. Benchmarking makes such comparisons systematic rather than anecdotal. It requires defining what to measure, at what granularity (a single kernel, a full model, an end-to-end pipeline), and under which conditions (batch size, input distribution, thermal state, concurrent load). Without this structure, teams compare numbers that were never measured on the same terms, and decisions that looked sound in a spreadsheet collapse under production workloads. Choosing hardware, a model, or a deployment configuration on such numbers can commit an organization to millions of dollars of infrastructure that delivers a fraction of the capacity it was bought to provide. Benchmarking validates each optimization, bringing claims into contact with evidence and quantifying the gap between promise and delivery. In D·A·M terms, benchmarking holds co-design to account by revealing whether Data, Algorithm, and Machine were matched or merely assembled.



Learning Objectives

- Explain benchmarking as D·A·M validation that tests whether optimization claims hold under representative conditions
- Compare training and inference benchmarks using throughput, latency percentiles, energy, accuracy, and workload scope
- Select micro, macro, or end-to-end granularity based on the engineering decision being tested
- Apply standardized benchmark run rules to align datasets, metrics, hardware configuration, and reporting
- Design benchmark protocols that control power boundaries, input distributions, batch sizes, and statistical variance
- Evaluate model and data quality with calibration, robustness, representativeness, and slice-level metrics
- Diagnose benchmark-production gaps caused by drift, thermal throttling, dynamic load, and silent degradation

12.1 ML Benchmarking Framework

A model quantized to INT8 may benchmark 2× faster on a synthetic workload but show no improvement under real traffic patterns with variable input sizes and concurrent requests. A pruned model may maintain accuracy on the test set but fail on edge cases the benchmark never covered. Data selection promises more efficient training, model compression promises smaller and faster models, and hardware acceleration promises higher throughput, with the roofline and Amdahl's law predicting how much. Each of those predictions still has to be confirmed by measurement, and verifying that the claims hold in production is itself an engineering discipline.

Definition 12.1: Machine learning benchmarking

Machine learning benchmarking is the empirical measurement of ML workloads under specified conditions, used to test claims about components, models, or end-to-end systems against representative evidence rather than peak specifications alone.

1. **Significance**: The gap between peak and sustained performance can be large. An A100 GPU delivers 312 TFLOP/s (BF16) at peak, but in this illustrative 30 percent–50 percent model FLOPs utilization (MFU) scenario (achieved model FLOP/s divided by the hardware's peak FLOP/s), it sustains 93.6 TFLOP/s–156 TFLOP/s, about a 2–3.3× gap due to factors such as memory stalls, pipeline bubbles, and kernel launch overhead. Benchmarking quantifies the actual η_{hw} gap for a workload; vendor spec sheets do not.
2. **Distinction**: ML benchmarking spans multiple scopes. Micro-benchmarks isolate operations such as matrix multiplication, model-level benchmarks evaluate complete training or inference workloads, and end-to-end benchmarks include surrounding work such as data loading, preprocessing, optimization, checkpoint I/O, or serving infrastructure. The scope must match the engineering claim.
3. **Common pitfall**: A frequent misconception is that benchmark numbers are stable references. Both the workload (new model architectures) and the hardware (new GPU generations) evolve, so a result that leads a benchmark under one version often becomes the baseline under a later version, making year-over-year comparisons meaningful only when the benchmark version is held constant.

Benchmarking is where the physical laws established in earlier chapters face empirical reality. The **benchmark-production gap** measures the difference between modeled expectations and observed production behavior. Closing that gap requires measurements that convert theoretical claims into verified engineering knowledge.

↩ The iron law, whose utilization term η_{hw} benchmarks measure against peak, is established in section 1.7.

ML benchmarking operates across three interdependent dimensions that map directly to the components of any deployed system. **System benchmarking** measures whether the hardware delivers promised performance under realistic workloads or whether memory bandwidth saturation and software dispatch overhead erode the gains. **Model benchmarking** measures whether optimization techniques preserve model quality across the full input distribution, not just on curated test sets. **Data benchmarking** measures whether the model generalizes to real-world data with all its noise, bias, and distributional shift. Each dimension can independently reveal problems invisible to the others, and a system that passes all three provides far stronger deployment confidence than one evaluated along any single axis. The three dimensions do not receive equal space. System benchmarking carries most of the chapter, because its measurement problems (granularity, benchmark components, training and inference protocols, power) also shape how models and data are evaluated; model and data evaluation follow once that machinery is in place, and production considerations close the arc by asking whether all three survive live traffic.

Systems Perspective 12.1: Benchmarks as moving targets

In traditional systems (for example, SPEC CPU), the benchmark is a *rigid specification*. A sorting algorithm is correct if it sorts the list. Correctness is absolute and unchanging. In ML systems, the benchmark is a *soft specification*: correctness is defined by a finite set of examples (ImageNet), and the world moves. A model that scores highly on ImageNet can still underperform on user photos taken years after the benchmark was created.

In computer architecture, engineers design for the benchmark because the benchmark represents the workload. In ML engineering, designing solely for the benchmark is overfitting. Robustness comes from acknowledging that the benchmark is only a proxy for a shifting reality.

Validating the deployment of MobileNetV2 grounds this three-dimensional framework in physical hardware. It serves as the chapter's running example because it spans all three evaluation dimensions, illustrating how each reveals failure modes that the others miss. The model remains fixed across the chapter, but the target device varies across scenarios (an edge accelerator with a host CPU, a Raspberry Pi 4 camera trap, a smartphone) so that latencies are evaluated within a coherent deployment context rather than compared haphazardly across unrelated platforms.

↩ MobileNetV2, the lighthouse model for edge latency, is established in section 6.2.



Lighthouse 12.1: MobileNetV2 deployment validation

MobileNetV2 serves as the lighthouse example for validating the complete optimization pipeline. MobileNetV2 refines v1's depthwise separable design with inverted residuals and linear bottlenecks while maintaining a similar parameter scale (Sandler et al. 2018). It exemplifies the deployment challenges where benchmarking determines success or failure: compression can reduce model size, hardware acceleration can reduce inference latency, and only benchmarking can determine whether those gains survive the full deployment pipeline.

1. **Model compression** (chapter 10): INT8 quantization reduces this MobileNetV2 worked example from 14 MB to 3.5 MB (4× compression)
2. **Hardware acceleration** (chapter 11): the illustrative EdgeTPU scenario uses 2 ms inference vs. 15 ms on the board's host CPU
3. **Benchmarking validation**: Verify the pipeline delivers in practice

A systematic evaluation isolates EdgeTPU latency from preprocessing and data transfer overhead, confirms INT8 quantization preserves accuracy on edge cases such as unusual lighting, and verifies that throughput holds on real-world smartphone sensor streams rather than curated ImageNet test sets alone.

Each stage of that pipeline arrives with a favorable number from its own chapter, such as a smaller model file or a faster kernel. The benchmarking question is which of those local numbers still hold once the stages run together on the target device.

Systems Perspective 12.2: Benchmarking as cross-layer evidence

An ML benchmark is meaningful only when it identifies which layer of the system is being validated. Data selection metrics such as performance-per-data (PPD), area under the learning curve (AULC), and data compression ratio (DCR), all defined in section 9.6.1, measure whether fewer examples can preserve learning signal. Compression metrics measure whether fewer parameters or lower precision preserve quality. Hardware metrics such as roofline position and TOPS/W measure whether the machine executes the workload efficiently. The system benchmarks in section 12.3.2 through section 12.9.4 connect these layers by testing whether local improvements survive in an end-to-end workload.

Rigorous evaluation begins with the mindset that separates meaningful evidence from misleading metrics. Three principles distinguish effective practitioners.

First, *benchmarks are proxies, not truth.* Every benchmark measures specific conditions that may not match the target deployment. A system can achieve high sample throughput in Offline mode (bulk throughput with all inputs available) and much lower queries per second (QPS) in Server mode (latency-constrained requests arriving over time). The critical question is always what the benchmark does not measure.

Second, Goodhart's Law applies everywhere.[1] "When a measure becomes a target, it ceases to be a good measure." Teams that optimize for benchmark rankings often produce systems that excel in evaluation but fail in production. Benchmark-specific optimizations frequently degrade characteristics that matter for deployment: robustness, calibration, and efficiency.

Third, end-to-end beats component metrics. In this illustrative pipeline, a 3× inference speedup applied to a 10 ms model stage inside a 50 ms request yields only about a 1.2× end-to-end improvement, or worse if the optimization increases memory pressure. These principles reappear throughout the benchmarking methodology and are examined in depth in section 12.13.

Knowing *what* to measure, however, is only half the problem. Measuring incorrectly (with the wrong workloads, biased baselines, or uncontrolled variables) produces numbers that feel precise but mislead decisions. The history of computing benchmarking is littered with examples of technically sound metrics applied with flawed methodology, from compiler-gamed Whetstone scores to cherry-picked GPU benchmarks that predict nothing about sustained workloads. The historical foundations of benchmarking[2] matter because they expose the validation failures that still recur in ML: optimized metrics that stop predicting real workloads, hardware numbers that ignore sustained operating state, and model scores that miss deployment cost. The same validation sequence governs modern practice: first verify that hardware delivers promised performance, then verify that the model and data optimizations built atop that hardware deliver their promised gains.

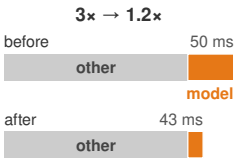
12.2 Historical Foundations

Each benchmark generation arose when existing metrics failed to predict end-to-end behavior, and those failures are direct precedents for why modern ML evaluation requires the D·A·M taxonomy (Data · Algorithm · Machine). The evolution from early synthetic tests to modern ML suites reveals three methodological shifts: from isolated kernels to representative application suites, from single-metric throughput to multi-objective energy profiles, and from generic compute metrics to domain-specific execution constraints.

12.2.1 Performance benchmarks

The earliest computing benchmarks revealed an enduring vulnerability in empirical evaluation: benchmark gaming. Whetstone (Curnow and Wichmann 1976) used a synthetic mix of scientific operations, while LINPACK[3] (Dongarra et al. 1979) measured dense linear-system solving. Because these workloads relied on predictable computational loops, compiler and hardware vendors could optimize specifically for the benchmark kernels—such as unrolling synthetic loops or tailoring cache tiling to exact matrix dimensions—without improving general application performance. SPEC CPU (1989) addressed this limitation by evaluating complete, portable software suites that stressed integer branching, irregular memory hierarchies, and floating-point pipelines (Dixit 1993). This

[1] **Goodhart's law**: Goodhart (1984) articulated the original 1975 Bank of England observation on monetary policy; Strathern (1997) generalized it into the form quoted above. The original context was macroeconomics: once a monetary aggregate became an official policy target, banks changed behavior to game the metric, destroying its predictive value. In ML, the same failure mode recurs structurally: BLEU rewards n-gram overlap (Papineni et al. 2002), ImageNet rewards performance on a fixed visual distribution (Deng et al. 2009; Recht et al. 2019), and benchmark leaderboards can incentivize test-set-specific tuning.



Component speedup rarely survives as end-to-end benchmark speedup.

[2] **Benchmark**: From surveying, where a "bench mark" was a horizontal cut in stone serving as a fixed elevation reference. The term entered computing in the 1970s to describe standardized comparison points, but the surveying metaphor carries a systems lesson: just as an elevation measurement is meaningless without a calibrated reference, an ML throughput number is meaningless without controlled workloads, thermal state, and precision settings.

progression directly informs ML systems: model compression claims cannot rely on isolated matrix multiplication kernels, but require validation on representative tasks like ResNet-50 and BERT across the full software runtime.

As computing expanded into interactive and mobile environments, single-metric evaluation proved insufficient. Graphics benchmarks paired frame rate with rendering quality, and mobile evaluations treated battery draw as co-equal with latency. This multi-objective trade-off mirrors ML serving, where optimizing latency in isolation can trigger catastrophic degradation in model accuracy or energy efficiency.

A third shift occurred when distributed computing demonstrated that component-level microbenchmarks fail to predict cluster-level throughput. Evaluating a CPU or accelerator in isolation cannot capture execution dynamics when network interconnect latency, serialization overheads, and collective communication dominate wall-clock time. Distributed ML training similarly depends on the tight interplay among accelerator matrix engines, host-to-device PCIe bandwidth, input data prefetching pipelines, and gradient synchronization across network fabrics.

To capture these interactions, DAWNBench (Coleman et al. 2019) pioneered time-to-accuracy evaluation, tying raw hardware throughput directly to algorithmic convergence. Maximizing samples processed per second offers no engineering value if numerical instability or suboptimal batch sizing prevents the model from reaching target validation accuracy. These empirical lessons culminated in MLPerf[4] (2018), which institutionalized full-system measurement, multi-objective constraints, and representative workloads spanning vision, language, and recommendation systems (Mattson et al. 2020; Reddi et al. 2019).

12.2.2 Energy benchmarks

The multi-objective evaluation paradigm extended to energy efficiency as computing diversified beyond mainframes into battery-limited mobile devices and megawatt-scale data centers. Because energy consumption directly dictates battery life at the edge and operating expenses in warehouse-scale clusters, energy became a first-class evaluation metric alongside performance. This shift produced standardized energy benchmarks such as SPEC Power[5] for servers and Green500[6] for supercomputers.

Evaluating energy in ML workloads introduces distinct physical measurement challenges. Accelerators exhibit sharp dynamic power swings: alternating between memory-bound activation layers and power-dense matrix multiplication units creates rapid current fluctuations (di/dt) that simple average-power meters fail to resolve. Furthermore, isolating accelerator consumption from host CPU activity, PCIe interconnects, and cooling fans requires rigorous measurement boundaries. MLPerf Power (MLCommons 2024b) addresses these demands by coupling standardized workloads with strict physical measurement methodologies and calibrated external power analyzers.

Energy benchmarking extends beyond hardware telemetry to evaluate algorithmic efficiency. Model compression techniques (pruning, quantization, knowledge distillation) reduce energy consumption by decreasing arithmetic operations and off-chip memory traffic, rather than relying solely on lower-power silicon. For instance, MobileNet architectures replace standard convolutions with depthwise separable convolutions, drastically reducing DRAM access and arithmetic operations relative to standard convolutional neural network (CNN) baselines such as ResNet (Howard et al. 2017; Sandler et al. 2018; He et al. 2016a). As quantified in section 10.4.1.1, reading an operand from off-chip DRAM consumes orders of magnitude more energy than executing an arithmetic operation, making algorithmic memory-traffic reduction a primary lever for energy-efficient computing.

12.2.3 Domain-specific benchmarks

As specialized accelerators emerged to bypass general-purpose CPU bottlenecks, generic benchmarks failed to capture domain-specific execution constraints. Three operational axes drove this specialization, defining how systems are evaluated under real-world conditions.

Deployment constraints dictate core metric priorities based on physical limits. Data center training clusters optimize for aggregate throughput within rack- and facility-level power envelopes. In contrast, mobile AI operates within strict thermal throttling thresholds (typically under 5 watts), and embedded microcontroller systems function within milliwatt power budgets. These physical

[3] **Whetstone and LINPACK**: Whetstone (Curnow and Wichmann 1976) was named after the English Electric facility in Whetstone, Leicestershire, where the original ALGOL compiler was built; LINPACK (Dongarra et al. 1979) was a package and benchmark for dense linear systems, later used by the Top500 list. Whetstone's fixed synthetic program mix and LINPACK's dense linear-algebra focus made each useful but narrower than a diverse application suite. ML benchmarking inherited the same vulnerability: model-specific kernel tuning can overfit a single workload, which is why MLPerf uses multiple workloads spanning vision, language, and recommendation (Mattson et al. 2020; Reddi et al. 2019).

[4] **MLPerf**: Launched in 2018 by a consortium of industry and academic institutions, MLPerf takes its name from "ML" combined with "Perf" (performance), echoing SPEC's benchmarking tradition. MLPerf's design principles—representative workloads, full-system measurement, and open submission—directly address the gaming that plagued Whetstone and LINPACK: vendors who could previously report peak kernel throughput on cherry-picked problem sizes must now report end-to-end system performance on standardized tasks (Mattson et al. 2020; Reddi et al. 2019).

[5] **SPEC Power**: Introduced in 2007, SPEC Power measures performance per watt across 11 load levels from idle (0 percent) through 100 percent in 10 percent increments (Lange 2009). This granularity matters for ML serving: inference workloads rarely sustain 100 percent load, and servers that are efficient at peak but wasteful at partial load inflate the energy cost of real-world deployment.

bounds determine whether a benchmark prioritizes throughput (samples per second) or energy efficiency (Joules per inference).

Application requirements impose functional constraints beyond raw speed. Clinical AI models require rigorous calibration and interpretability alongside classification accuracy; high-frequency financial trading systems enforce hard tail-latency cutoffs paired with deterministic execution; autonomous vehicles demand bounded worst-case latency and formal functional safety validation. These requirements broaden evaluation criteria beyond throughput.

Operational conditions determine real-world robustness. Edge devices must maintain inference deadlines across ambient temperature swings and noisy sensor streams; data-center serving infrastructure must absorb sudden query spikes and network packet loss without dropping requests; industrial IoT nodes must operate unattended for years. Hardware acceleration capabilities deliver production value only when sustained under these operational environments.

Machine learning exemplifies this domain-specific divergence. Because ML workloads trigger complex interactions between tensor execution units, memory hierarchy bandwidth, and distributed communication fabrics, no single metric or workload can characterize a system. MLPerf formalizes this specialization across distinct deployment suites: MLPerf Training evaluates multi-node scaling and high-bandwidth interconnects under time-to-quality targets (Mattson et al. 2020); MLPerf Inference measures throughput and latency percentiles under service level agreements (SLAs) across server and client systems (Reddi et al. 2019); MLPerf Tiny tests ultra-low-power microcontrollers operating with tight memory constraints (C. Banbury et al. 2021); and the cross-cutting MLPerf Power track evaluates energy efficiency across each environment. Reading table 12.1 down its constraint column illustrates how metrics adapt to physical limits: cluster-scale interconnect bandwidth gives way to latency percentiles at the edge, and ultimately to strict memory footprints on microcontrollers.

[6] **Green500**: Started in 2007 as a counterpart to the Top500, Green500 ranks systems by FLOP/s per watt rather than raw performance (Feng and Cameron 2007). Its lesson for ML systems is methodological: the most cost-effective training cluster is not necessarily the fastest one, but the system that delivers useful work per watt under the workload and measurement boundary that matter.

Table 12.1: MLPerf Benchmark Suite Variants: Each variant addresses a different deployment context, from data-center-scale training to ultra-constrained microcontroller inference, targeting specific operational constraints and measuring metrics relevant to its deployment scenario.

MLPerf Variant	Target Domain	Key Constraints	Primary Metrics
MLPerf Training	Data center	Multi-node scaling, high bandwidth interconnects	Time-to-quality, throughput (samples/sec)
MLPerf Inference	Server/Edge	Latency SLAs, throughput requirements	QPS, latency percentiles, accuracy preservation
MLPerf Tiny	MCU/IoT	Ultra-low-power inference, limited memory	Latency, accuracy, energy per inference
MLPerf Power	Cross-cutting	Energy budgets, thermal constraints	Performance/W, energy per query

Across all variants, MLPerf Power measures useful work per watt rather than raw throughput alone. Domain-specific suites ensure that architectural optimizations translate to deployment success rather than remaining confined to isolated synthetic benchmarks.

While ML benchmarking suites inherit these historical lessons, machine learning workloads also introduce statistical and data-dependent variability absent in traditional deterministic computing. Two runs of the same model on the same hardware can legitimately produce different numbers, so rigorous ML evaluation requires explicit statistical controls (multi-seed replication, variance reporting, and deterministic data-loading rules). Where that variability comes from is the first measurement challenge taken up below (section 12.3.1).

Standardization transforms these empirical lessons into a shared engineering methodology. When one team measures inference latency including host preprocessing while another begins timing only after tensors reside in accelerator device memory, or when power meters draw different physical boundaries around the power supply unit, the resulting numbers are incommensurable. Standardized suites replace ad-hoc testing with audited harnesses, uniform measurement boundaries, and verifiable submission rules, enabling rigorous hardware procurement and algorithmic comparison across the industry. Table 12.2 synthesizes this historical progression from early synthetic loops to multi-dimensional ML systems evaluation.

Table 12.2: Benchmark Evolution: Evolution of computing benchmarks from synthetic operations to ML-specific evaluation. Each generation addressed limitations of its predecessors, culminating in MLPerf's synthesis of representative workloads, multi-objective metrics, and integrated system measurement.

Benchmark	Year	Primary Focus	Key Metric(s)	Lesson for ML Benchmarking
Whetstone	1976	Synthetic floating-point operations	MWIPS	Gaming synthetic tests undermines evaluation validity
LINPACK	1979	Linear algebra (matrix operations)	FLOP/s	Isolated operations miss system-level complexity and bottlenecks
SPEC CPU	1989	Real application workloads	SPECrate, SPECspeed	Representative workloads reveal true deployment performance
SPEC Power	2007	Server energy efficiency	ssj_ops/W across load levels	Energy efficiency requires multi-load evaluation, not just peak performance
Green500	2007	HPC energy efficiency	GFLOP/s per watt	Efficiency rankings complement raw performance rankings
MLPerf	2018	ML systems (training + inference)	Time-to-quality, QPS, latency, accuracy	Synthesizes all lessons: representative workloads + multi-objective + system

12.3 System Benchmarking Suites

A team evaluating edge deployment hardware must compare five different system-on-chip (SoC) designs for a smart camera product. Vendor A reports 8 TOPS at INT8; Vendor B reports 15 TOPS at INT4; Vendor C reports inference latency on a proprietary model; Vendor D cites MLPerf scores from two generations ago; Vendor E provides only peak throughput at maximum batch size. None of these figures can be directly compared. The team cannot make an informed procurement decision because each vendor isolates a favorable metric under idiosyncratic conditions, obscuring true operating trade-offs. Benchmarking suites resolve this fragmentation by enforcing uniform measurement harnesses and standardized evaluation rules.

Three lessons from benchmark history—representative workloads, multi-objective evaluation, and integrated system measurement—converge on a requirement unique to machine learning: managing statistical approximation. Traditional computing benchmarks specify correctness checks, which may require exact outputs or permit stated numerical tolerances. In ML systems, low-level engineering choices routinely alter numerical output: aggressive weight quantization, kernel pruning, or reduced-precision accumulators trade accuracy for execution speed and lower memory bandwidth pressure. Consequently, an ML benchmark cannot evaluate execution time in isolation; output accuracy serves as an inviolable operational constraint. Modern benchmarking suites encode these constraints into standardized frameworks, making rigorous cross-platform comparisons possible.

Evaluating an ML system therefore requires assessing the coupled interactions across the Data, Algorithm, and Machine (D·A·M) co-design space rather than measuring raw compute throughput. Early benchmarks focused primarily on algorithmic accuracy on isolated datasets (LeCun et al. 1998). As models scaled, compute bottlenecks expanded benchmarking to hardware execution efficiency (Jouppi et al. 2017), and deployment failures demonstrated that data pipeline bottlenecks and input distribution shifts dictate real-world behavior (Gebru et al. 2021). Energy consumption cuts directly across all three dimensions: algorithmic structure dictates arithmetic intensity and computational complexity (Hernandez and Brown 2020), machine microarchitecture sets the energy cost per arithmetic operation and memory access, and data ingestion patterns determine bus utilization. A robust benchmarking suite tests these dimensions simultaneously, ensuring that claimed optimizations survive under production constraints. A suite earns that trust only by solving three problems in order. It must first produce valid measurements despite run-to-run noise, finite test sets, unrepresentative workloads, and misaligned metrics. It must then isolate what the hardware-software stack contributes, which is the job of a system benchmark and its roofline diagnostic. Finally, it must make results comparable across organizations, which is the job of community standardization.

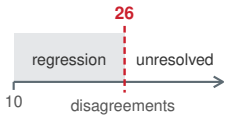
12.3.1 ML measurement challenges

ML systems combine several sources of measurement variability that classical software benchmarks were not designed to evaluate together. These fluctuations originate at both the machine and algorithmic layers. On the machine side, sustained execution pushes accelerators against thermal

and power envelopes, triggering dynamic voltage and frequency scaling (DVFS) that throttles processor clocks mid-run. Host operating-system activity (thread scheduling jitter, memory paging, and PCIe bus contention during batch staging) further introduces latency variance across runs. On the algorithmic side, floating-point addition is non-associative, so reduction orders that change across parallel execution threads shift results slightly, while mini-batch shuffling, pseudo-random augmentations, stochastic dropout masks, and weight initialization perturb the trajectory that stochastic gradient descent takes across a non-convex loss surface. Distinguishing a genuine systems optimization from background measurement noise requires isolating these overlapping sources of variance.

To isolate run-to-run variability, benchmark protocols require repeated experimental trials across controlled pseudo-random seeds. The number of runs must satisfy explicit statistical-power targets, specifying the probability of detecting a stated real effect, rather than opportunistic sampling. Reporting must extend beyond single best-of-k scores or isolated arithmetic means: reporting sample variance, standard deviations, or confidence intervals quantifies stability and establishes whether an observed speedup exceeds background system jitter. Without these controls, benchmarking yields misleading conclusions. In reinforcement learning, reported performance advantages frequently evaporate into the statistical noise of random weight initializations and environment dynamics (Henderson et al. 2018). Similarly, unstandardized evaluations of generative adversarial networks produce rank reversals across random seeds, demonstrating that uncharacterized variance undermines comparative claims (Lucic et al. 2018).

Even when run-to-run execution variance is fully controlled, benchmarking encounters a structural constraint in the evaluation dataset itself: test-set measurement capacity. A test set operates as a measurement instrument with a finite resolution limit. Because accuracy on a finite sample is a binomial proportion, empirical scores carry intrinsic sampling variance that scales inversely with sample size. When an optimization alters accuracy by one or two percentage points, the sample size and paired pattern of disagreements determine whether the test can distinguish a genuine algorithmic effect from sampling fluctuation. As analyzed in the accompanying notebook, marginal accuracy scores on a thousand-sample test set are insufficient to establish the significance of a fine-grained difference without examining the paired outcomes.



The disagreement count decides: at most 26 disagreements reveal a regression.

Napkin Math 12.1: The statistical confidence trap

Problem: A baseline classifier scores 95 percent, while a compressed version scores 94 percent on the same 1,000 images. Does this one-point drop establish a regression?

Math:

1. **Expected errors**: The scores correspond to 50 errors and 60 errors. Under the baseline binomial model, the error count has a standard deviation of about 7 errors.
2. **Difference interval** (95 percent): The same-image comparison requires paired outcomes, which the two marginal scores do not supply. For comparison, suppose the models instead achieved these scores on independent test sets sampled from the same population. Let $\hat{p}_1$ and $\hat{p}_2$ denote their accuracy estimates, and let N be the number of images evaluated for each model. Under that independent-set assumption, the standard error of their difference is

$$\mathrm{SE}(\hat{p}_1 - \hat{p}_2) = \sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{N} + \frac{\hat{p}_2(1-\hat{p}_2)}{N}}.$$

   The observed 1 percentage point difference has an approximate interval from -1 percentage point to 3 percentage points, which includes zero. Treated as independent samples, the one-point drop is indistinguishable from sampling noise. Pairing can change that verdict, because images that both models classify the same way cancel out of the comparison.
3. **Paired test on the same images**: The stated problem uses one test set, so the decisive evidence is where the two models disagree. Let n_{base} count images only the baseline

classifies correctly and n_{comp} count images only the compressed model classifies correctly. Images both models get right or both get wrong carry no information about which is better, and McNemar's test discards them. The scores fix the difference $n_{\text{base}} - n_{\text{comp}}$ at 10 (the compressed model's extra errors) but not the sum. The test statistic, without continuity correction, is

$$\chi^2 = \frac{(n_{\text{base}} - n_{\text{comp}})^2}{n_{\text{base}} + n_{\text{comp}}},$$

and the drop is significant at 95 percent when χ^2 exceeds 3.84. That happens only when the models disagree on at most 26 images.

Implication: The marginal scores cannot answer the question; the disagreement count decides it. If the compressed model's 10 extra errors are nearly its only disagreements with the baseline, the paired test supplies evidence of a regression at the stated significance threshold. If the models disagree on many more images, the test leaves the same one-point gap unresolved. Nonsignificance does not establish equivalence or compliance with a permitted quality-loss tolerance. A benchmark report must therefore keep per-image outcomes, not only the two accuracies; the interval-based decision rule in section 12.5.8 separates an unresolved difference from a supported improvement.

Systems insight: Small benchmarks exhibit what amounts to a laboratory fallacy. The test set, viewed as a measurement instrument, must be sized to match the precision of the change it is meant to detect. A single accuracy estimate alone needs about 1,825 images for ±1 percentage point precision, and that sample size does not by itself power a comparison between two models.

Beyond statistical power, benchmark validity depends on workload representativeness. Synthetic microbenchmarks generate uniform, statically sized tensors directly in accelerator high-bandwidth memory (HBM). By executing continuous, compute-bound general matrix multiplication (GEMM) operations in an isolated loop, microbenchmarks measure theoretical peak arithmetic throughput. However, they conceal the memory hierarchies and scheduling bottlenecks that dominate production deployments. Synthetic workloads bypass host-to-device PCIe data transfers, input decoding pipelines, dynamic memory allocation churn, and GPU cache evictions. In contrast, **trace-based benchmarking** replays timestamped request logs recorded from live serving clusters. Production traces capture Poisson and bursty request arrival distributions, variable sequence lengths, and concurrent query streams. Replaying live traces forces the serving infrastructure to manage dynamic batching (grouping requests that arrive within a short time window into one batch), key-value (KV) cache allocation under memory pressure, and queuing delays, stress-testing the system under operational conditions that synthetic steady-state loops obscure.

A representative workload can still mislead if the evaluation metric fails to reflect deployment constraints. This failure mode represents a breakdown of metric alignment. When an engineering team optimizes a single algorithmic figure of merit (such as an offline BLEU score, top-1 classification accuracy, or token perplexity) in isolation from system service-level objectives (SLOs), the metric ceases to measure genuine operational fitness. This divergence is the chapter's second principle, Goodhart's Law, operating inside a single team's choice of metric, and once the proxy becomes the sole optimization target it stops reporting overall system health. The accompanying example illustrates this pathology in neural machine translation, where an offline accuracy optimization creates a catastrophic latency penalty.

↩ The key-value (KV) cache, the per-request attention state that grows with every generated token, is established in section 6.8.4.

Example 12.1: Goodhart's Law in action

Scenario: An engineering team optimizes a neural machine translation model for offline BLEU score, increasing the beam search size (the number of candidate partial translations kept at each decoding step) from 1 to 10.

Diagnosis: The larger beam search achieves a 0.5-point BLEU gain on paper but increases candidate evaluations by 10×, increasing inference latency from 50 ms to 200 ms (4× slowdown), exceeding the serving deadline.

Systems lesson: Optimizing offline accuracy without latency constraints invites Goodhart's Law failures. Production benchmarks must enforce strict SLOs for serving latency (e.g., latency < 100 ms).

The translation trade-off exposes the fundamental tension across the Data, Algorithm, and Machine (D·A·M) co-design space: an algorithmic change (expanding the beam width tenfold) multiplies computational load and memory traffic, extracting a marginal output-quality improvement while violating hardware latency deadlines. Beyond metric misalignment, evaluating systems on static datasets introduces a deeper limitation: fixed benchmarks assess execution under an unvarying input distribution, concealing how systems degrade under production distribution shifts. Model and data evaluation must test those distribution limits. System comparisons address a separate question by isolating infrastructure execution efficiency from model architecture and dataset variation. Standardized suites like MLPerf establish reference workloads, datasets, quality targets, and permitted implementation changes, enabling reproducible comparisons of hardware-software execution efficiency within a specified benchmark version and division.

12.3.2 System benchmarks

System benchmarks measure the computational foundation that enables model capabilities, examining how hardware architectures, memory systems, and interconnects affect overall performance. This validation is critical because hardware specifications often describe theoretical peaks that application workloads do not sustain. The discrepancy is common enough to make peak-performance claims incomplete. System benchmarks reveal these gaps by running standardized ML workloads rather than relying on peak arithmetic rates alone.

Systems Perspective 12.3: The fallacy of peak performance

A long-standing computer architecture quip calls peak performance "the performance the manufacturer guarantees you will not exceed." For ML systems, the gap can be wide because of the memory wall: a memory-bound model leaves much of the advertised FLOP/s unreachable no matter how fast the arithmetic units run. Standardized benchmarks like MLPerf require systems to run specified models and data, revealing sustained performance at a documented operating point rather than a universal rate for every workload.

For memory-bound workloads, the peak-vs.-sustained gap follows from the memory wall; compute-bound workloads may approach peak. This distinction reframes vendor evaluation from guesswork into a checklist of concrete criteria.

Checkpoint 12.1: Decoding vendor benchmark claims

When evaluating hardware or software based on vendor-reported benchmarks, check whether the claim identifies the workload, measurement boundary, and operating conditions.

- ☐ Measured quantity: Does the latency number include preprocessing and postprocessing, not just model execution?
- ☐ Precision boundary: Can the precision behind a throughput claim, such as INT4 versus INT8 or FP16, be identified?
- ☐ Workload boundary: Are the model, batch size, and input distribution used to produce the comparison explicitly named?
- ☐ Excluded costs: Are memory transfers, model loading, initialization, sustained thermal behavior, and power accounted for at the claimed performance level?

- ☐ Claimed vs. achieved: Given a 1,000 TFLOP/s peak claim and a benchmark that sustains 350 TFLOP/s, estimate the MFU and state whether that utilization ratio alone identifies the bottleneck.

Engineers should reject any benchmark claim whose workload boundary, precision, and excluded costs cannot be reconstructed. A headline throughput or latency number becomes useful only after the engineer can map it to the actual model, batch shape, data movement, sustained operating point, and power envelope.

The underlying hardware—CPUs, GPUs, Tensor Processing Units (TPUs),[7] and application-specific integrated circuits (ASICs)[8]—determines ML-system speed, efficiency, and scale. System benchmarks provide standardized methods for comparing hardware across AI workloads by computational throughput, memory bandwidth, power efficiency, operator coverage, and scaling (Reddi et al. 2019; Mattson et al. 2020).

Table 12.3 translates common marketing phrases into the technical caveats behind each.

Table 12.3: Decoding Vendor Benchmark Claims: Four common marketing phrases and the technical caveats that determine whether a claim describes end-to-end performance, a narrow accelerator measurement, or an unsupported efficiency comparison.

Vendor Claim	What It Often Means
"Up to 10,000 images/sec"	Peak throughput at maximum batch size, INT8, without preprocessing
"Sub-millisecond latency"	Accelerator compute only, excluding data transfer
"5× more efficient"	Per-operation efficiency, not total system efficiency
"Optimized for AI"	May only accelerate specific operations or precisions

System benchmarks serve two functions. For practitioners, they enable informed hardware selection by providing comparative data across configurations. For manufacturers, they quantify generational improvements and guide accelerator development. Both functions require separating what the hardware contributed from what the model and data contributed, which is the separation the next definition formalizes.

Definition 12.2: Machine learning system benchmarks

Machine learning system benchmarks are standardized evaluation protocols that hold the workload and quality target constant while varying the hardware-software stack, measuring $\eta_{\text{hw}} = R_{\text{sustained}}/R_{\text{peak}}$ (the measured sustained rate $R_{\text{sustained}}$ over the peak rate) and L_{lat} to isolate infrastructure efficiency from algorithmic improvements.

1. **Significance**: The same ResNet-50 model can deliver very different throughput across hardware stacks, precision formats, batch sizes, and compiler configurations, yet still report the same ImageNet Top-1 accuracy. System benchmarks capture this implementation gap, which is invisible to algorithmic benchmarks that only report accuracy.
2. **Distinction**: Unlike algorithmic benchmarks (which vary model architectures and training procedures to improve convergence accuracy), system benchmarks hold the algorithm fixed and vary the implementation (kernel libraries, quantization formats, batch sizes, and hardware generations) to measure how efficiently the hardware-software stack executes the iron law's $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ term.
3. **Common pitfall**: A frequent misconception is that a system benchmark result generalizes across workloads. An accelerator that achieves high utilization on ResNet-50 (a compute-friendly vision workload) may achieve much lower utilization on a recommendation system (a memory-bandwidth-bound workload). System benchmarks are workload-specific; no single metric characterizes a hardware platform.

[7] **TPU (tensor processing unit)**: Google's custom ASIC for neural network workloads. A TPU v4 pod (4,096 chips) delivers 1.1 exaFLOP/s peak BF16 (Jouppi et al. 2023), but benchmarking TPUs requires caution: their systolic-array architecture favors regular tensor operations, so peak FLOP/s overstate performance on irregular workloads like sparse attention or dynamic control flow.

[8] **ASIC (application-specific integrated circuit)**: An ASIC's peak TOPS number applies only to the specific operators it was designed for. An unsupported layer may require a host fallback or a different implementation, adding transfer and coordination costs. This makes operator coverage an early question in any ASIC benchmark: workload compatibility can explain part of the gap between peak and achieved throughput.

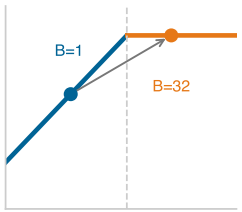
The roofline model of section 11.6, which plots the bound of the arithmetic intensity law (principle 6), is a prediction, and a benchmark is the measurement that tests it. The roofline bounds where a kernel can sit; a benchmark determines where it does sit, and the gap between the two is the diagnostic. A kernel far below its predicted ceiling is losing time to something the model omits (launch overhead, poor coalescing, synchronization), which is exactly what the profiling tools later in this chapter look for. Making the prediction requires the workload's arithmetic intensity at a named memory interface and the target's matching compute and bandwidth limits. For an NVIDIA A100 GPU with 312 TFLOP/s of FP16 Tensor Core performance (FP32 is 19.5 TFLOP/s)[9] and 2.04 TB/s memory bandwidth (SXM variant), the ridge point (peak compute divided by peak bandwidth, the intensity at which the two ceilings meet) falls at 153 FLOP/byte. Below it, bandwidth sets the lower ceiling; above it, compute does (section D.2.1 derives the threshold from first principles).

Roofline position[10] depends on the workload. In this worked A100 example, an illustrative high-intensity ResNet-50 forward-pass workload at large batch size uses an arithmetic intensity of ~300 FLOP/byte, above the A100 ridge, placing its modeled ceiling on the compute roof (He et al. 2016a; Choquette et al. 2021). Lower-intensity operations fall below the ridge into the memory-bound regime: a BERT inference at batch size one, counting only weight-loading traffic, reaches ~100 FLOP/byte arithmetic intensity and a lower performance ceiling than peak. Increasing the batch size raises weight reuse and can move the workload across the ridge, provided activation and output traffic do not keep total intensity below it (Pope et al. 2023). Section D.2.1.1 works the intensity-to-utilization calculation end to end on the A100, contrasting a compute-bound matrix multiplication kernel against a memory-bound element-wise kernel, so the steps generalize to any model-hardware pair.

A worked BERT inference estimate shows the prediction side concretely; the perspective 12.5 later maps each term of such a prediction to the profiler signal that confirms or refutes it.

9 **FLOP/s (floating-point operations per second)**: The gap between advertised peak FLOP/s and achieved FLOP/s is the central tension in hardware benchmarking. The A100 advertises 312 TFLOP/s FP16 Tensor Core, but real workloads achieve different fractions of peak depending on arithmetic intensity, memory access patterns, precision, and runtime overhead. Reporting peak FLOP/s without utilization context is the most common benchmarking distortion.

10 **Roofline model**: Williams et al. (2009) introduced the Berkeley model, named for the visual shape of its performance ceiling. Its ridge point (peak FLOP/s divided by peak bandwidth) separates bandwidth-limited from compute-limited ceilings, suggesting which interventions to test against measured execution.



Larger batches increase weight reuse; total traffic determines whether inference crosses the ridge.

↪ The A100 ridge point, with a compute-bound matrix multiplication and a memory-bound element-wise kernel placed on either side of it, is developed in section D.2.1.1.

Napkin Math 12.2: Roofline analysis for BERT inference

Problem: BERT-Base must be deployed for inference on an A100 GPU. Management expects high GPU utilization. What performance should be predicted, and how can it be improved?

Step 1: Hardware limits.

- Peak compute: 312 TFLOP/s (FP16 Tensor Core)
- Memory bandwidth: 2.04 TB/s
- Ridge point: 312 TFLOP/s ÷ 2.04 TB/s = 153 FLOP/byte, the A100 value above

Step 2: BERT-base characteristics.

- Parameters: 110M = 220 MB (FP16)
- FLOPs per inference: ~22 GFLOP (forward pass with sequence length $S = 128$)
- Data movement: ~220 MB (one weight read from the modeled memory interface, excluding activations and outputs)
- Arithmetic intensity: $(22 \times 10^9) \div (220 \times 10^6)$ = 100 FLOP/byte (weights-only model; see note in main text)

Step 3: Performance prediction. Since 100 FLOP/byte < 153 FLOP/byte, the weights-only bandwidth ceiling for BERT at batch = 1 is:

$$\text{Roofline bound} = 100 \text{ FLOP/byte} \times 2.04 \text{ TB/s} = 203.9 \text{ TFLOP/s}$$
$$\text{Fraction of peak} = 203.9 \text{ TFLOP/s} / 312 \text{ TFLOP/s} = 65.4\%$$

Step 4: Optimization via batching. Increase batch size to 32:

- Same 220 MB of weights, but 32× more compute
- New FLOPs: $22 \times 10^9 \times 32$ = 704 GFLOP
- New intensity: $(704 \times 10^9) \div (220 \times 10^6)$ = 3200 FLOP/byte

Since 3200 FLOP/byte > 153 FLOP/byte, the weights-only estimate for batch = 32 reaches the compute ceiling, so the roofline permits up to the full peak. How much of that peak the implementation achieves is a property the roofline does not predict. Assuming the complete implementation sustains 85 percent of peak:

$$\text{Assumed perf} \approx 85\% \times 312 \text{ TFLOP/s} = 265.2 \text{ TFLOP/s}$$
$$\text{Fraction of peak (assumed, not predicted)} = 85\%$$

Systems insight: Batching raises intensity by reusing weights across inputs. This weights-only estimate omits activation and output traffic, so crossing its ridge does not establish that complete inference is compute bound. The displayed 85 percent is an implementation-efficiency assumption, not a roofline prediction. Batching can increase request latency when the system waits to accumulate requests. This throughput-latency trade-off is what MLPerf's execution scenarios (section 12.8.4.2) separate, contrasting a latency-optimized scenario that issues one query at a time with a throughput-optimized one that batches as large as memory allows.

System benchmarks evaluate performance across scales, ranging from single-chip configurations to large distributed systems and covering both training and inference. The ImageNet challenge shows why separating the hardware's contribution from the model's is hard to do after the fact. Between 2010 and 2014, top-5 error (the blue line in figure 12.1) fell from 28.2 percent to 7.3 percent while GPU-accelerated entries (the green bars) rose from zero to over 100, with the steepest single drop at AlexNet's arrival in 2012 (Krizhevsky et al. 2012; Russakovsky et al. 2015; Gray 2015). These paired series plot contemporaneous empirical trends, not a causal attribution of how much accuracy gain stemmed from hardware throughput versus algorithmic design, training schedules, or dataset expansion. A system benchmark avoids that ambiguity by construction: it fixes the model and quality target so that any change in the result belongs to the hardware-software stack.

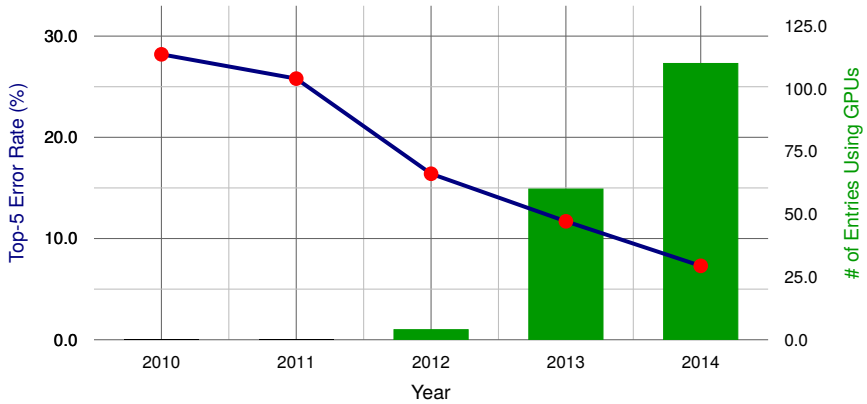


Figure 12.1: ImageNet Error and GPU Adoption: Sourced top-5 error rates are shown beside GPU use among challenge entries. The temporal association does not isolate hardware's causal contribution. Error-rate sources: (Russakovsky et al. 2015; Krizhevsky et al. 2012). Gray (2015) charts the per-year GPU entry counts.

While section 12.11.1 evaluates model compression against task accuracy baselines, system benchmarking isolates the execution efficiency of the hardware-software stack under a fixed model. Modern accelerators rarely sustain theoretical peak throughput because physical execution depends on the coupling between tensor shapes and memory hierarchies. As the preceding roofline analysis illustrated, GPU utilization shifts dramatically across operational regimes: BERT achieves an upper bound of only 65.4 percent under single-request inference due to memory bandwidth limits, whereas batching can raise intensity toward the compute plateau where 85 percent utilization is assumed. Parameter-heavy transformer decodes and activation-heavy convolutional passes stress entirely different levels of the memory subsystem, causing achievable throughput to diverge across precision formats.

Measuring sustained utilization rather than peak capability establishes two critical scope boundaries. First, energy consumption scales with execution efficiency: performance per watt varies widely across hardware platforms, and an underutilized accelerator draws substantial static power for minimal computational output. Second, multi-accelerator execution introduces communication boundaries. While multi-node cluster scaling involves network-topology effects beyond this book's single-node scope, intra-node benchmarking must account for interconnect bandwidth, memory sharing across accelerators, and gradient synchronization over NVLink or PCIe. Across every operational scale, a benchmark delivers engineering value only when its operating point reflects the constraints of live deployment rather than the unconstrained conditions of a datasheet.

12.3.3 Community-driven standardization

Hardware utilization metrics become meaningless for comparative systems evaluation without rigid, shared measurement boundaries. If one benchmark includes host-side data preprocessing and PCIe transfers while another measures only device kernel execution, the resulting latency figures reflect incompatible system boundaries rather than architectural efficiency. Similarly, measuring power consumption solely at the accelerator voltage rail omits the substantial energy drawn by host processors and chassis cooling fans. Independent organizations cannot resolve this discrepancy alone, as individual vendors face commercial incentives to tailor measurement boundaries to their specific hardware strengths.

Reliable benchmarking standards require broad multi-organizational consensus to prevent selective reporting. In mature engineering domains, this consensus formalizes through bodies such as IEEE working groups (IEEE Standards Association 2024) and ISO/IEC technical committees (ISO 2024), which establish rigorous measurement specifications such as IEEE 2416 (IEEE Standards Association 2019) for system power modeling. In machine learning systems, where hardware architectures and model structures evolve rapidly, consortia such as MLCommons provide the operational framework needed to maintain living benchmark suites. By providing open-source reference implementations paired with strict compliance verification rules, these consortia ensure that performance numbers reported across different laboratories represent comparable workloads under the same version, division, quality target, and scenario rules.

Standardizing system benchmarks requires formalizing the D·A·M taxonomy so that evaluations isolate the Machine without confounding changes to the Algorithm or Data. The MLPerf Training benchmark enforces this separation by measuring time-to-train to a specified quality target rather than raw floating-point throughput or epoch duration (Mattson et al. 2020). A hardware platform that achieves high TFLOP/s through aggressive low-precision scaling provides a time-to-quality benefit only if faster steps outweigh any extra work needed to reach the target; a run that misses the target is not a valid success. MLPerf splits submissions into two divisions. In the closed division, every submitter must run the reference model and reach the same quality target, so differences in time-to-train belong to the system; the open division permits model and algorithm changes, so its results compare whole solutions rather than machines. Within the closed division, the rules constrain the workload while permitting specified tunable hyperparameters and approved numerical formats. Quality and compliance checks establish which hardware, compiler, and training-configuration improvements constitute valid submissions.

The MLPerf Inference benchmark applies this discipline to production deployment constraints (Reddi et al. 2019). An accelerator cannot simply report peak throughput under infinite batching if interactive serving requires guaranteed responsiveness. The benchmark therefore fixes how queries arrive, through execution scenarios matched to physical operating regimes: one query at a time for a single interactive user, randomly arriving queries under a tail-latency bound for a shared service, and all inputs supplied upfront for bulk throughput. Section 12.8.4.2 defines each scenario and the latency percentile it scores. By standardizing these execution boundaries, community benchmarks account for queueing delays and host-to-device PCIe transfers, making the tested load and timing boundary explicit when interpreting a throughput claim.

Community standards establish reproducible measurement boundaries, but they do not dictate the physical scale at which evaluation occurs. A measurement can target an isolated matrix multiplication kernel or profile an end-to-end multi-accelerator training run, and each scope diagnoses

distinct physical bottlenecks. The depth of measurement determines which architectural limits can be isolated, establishing the role of benchmarking granularity.

12.4 Benchmarking Granularity

A GPU kernel that runs 3× faster in isolation may deliver zero end-to-end speedup if the data ingestion pipeline cannot keep the accelerator fed, the same dilution that turned the opening example's 3× model-stage speedup into about 1.2× end to end. This diagnostic mismatch makes evaluation granularity a fundamental systems design choice. Standardization specifies *how* measurement is kept consistent, while **benchmarking granularity** specifies *what* physical boundary is measured. System validation spans multiple architectural scales, from individual tensor operations to distributed production pipelines, with each granularity diagnosing distinct bottlenecks:

- Micro benchmarks isolate individual components: kernel execution time, memory bandwidth utilization, and single-layer numerical behavior. These diagnose *where* low-level hardware inefficiencies occur.
- Macro benchmarks evaluate composed model subsystems: full model training convergence, inference pipeline throughput, and dataset accuracy metrics. These reveal *what* architectural trade-offs exist across layer compositions.
- End-to-end benchmarks measure complete operational pipelines: client request-to-response latency including preprocessing, training time-to-accuracy including storage I/O, and model robustness on production data distributions. These show whether the combined system satisfies real-world service-level objectives.

Optimization techniques operate across distinct granularities: kernel fusion targets micro-level execution, structured pruning alters macro-level parameter structures, and data curation determines end-to-end generalization. System validation must align with these respective boundaries. An isolated micro-benchmark may show substantial kernel speedup, yet a macro-benchmark can reveal that increased activation memory footprints force smaller batch sizes that degrade overall throughput. Similarly, an end-to-end benchmark exposes storage or host deserialization stalls that remain completely invisible at the accelerator kernel level.

Measurement perimeters widen from left to right in figure 12.2, and the figure inserts one tier between macro and end-to-end. Micro-benchmarks isolate kernel layers (ML Layers) and macro-benchmarks evaluate composed models (ML Model). An application benchmark then adds the supporting host compute around the model (AI Task), and the end-to-end benchmark adds the non-AI compute nodes the request also traverses. The three granularities in this section fold the application tier into end-to-end, but the four-tier view shows where bottlenecks migrate as the boundary widens: out of the accelerator and into the host systems around it.

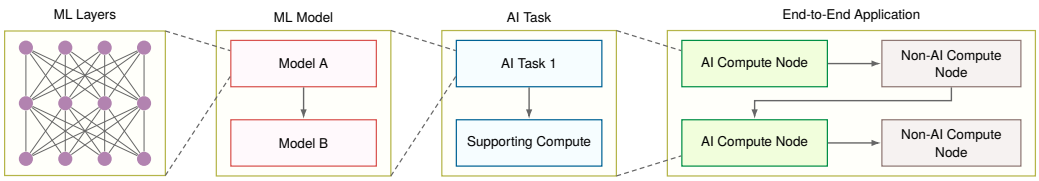


Figure 12.2: Benchmarking Granularity: Four-panel block diagram showing micro, macro, application, and end-to-end evaluation layers. Each panel maps a distinct scope of assessment, from isolated kernel operations through full-system deployment, enabling targeted optimization at every level of the ML stack.

12.4.1 Micro benchmarks

While end-to-end metrics govern service-level agreements, optimizing an accelerator pipeline requires isolating the specific mathematical operations that consume execution time and energy. Micro-benchmarks serve this diagnostic role by evaluating individual tensor operations in isolation, measuring the accelerator's hardware primitives directly. When profiling a slow inference service, macro-benchmarks report aggregate latency violations, but only micro-benchmarks reveal whether

↩ The compute primitives that micro-benchmarks isolate, vector and matrix operations and special function units, are established in section 11.3.

execution time is dominated by matrix multiplications, self-attention projections, memory copies across the host-accelerator interconnect, or elementwise activation kernels.

Tensor operations constitute the primary target of micro-benchmarking because matrix multiplications and convolutions dominate the arithmetic budget of deep neural networks. Highly optimized runtime libraries such as cuDNN[11] (Chetlur et al. 2014) provide hand-tuned kernel implementations tailored to specific microarchitectures. Micro-benchmarks also evaluate standalone activation functions (such as ReLU, Sigmoid, and GELU) and structural submodules (such as multi-head attention projections or transformer feed-forward blocks) across standardized input shapes. Benchmark suites such as Baidu's DeepBench (Baidu Research 2016) formalized this methodology by evaluating raw GEMM, convolution, and collective communication primitives across diverse hardware platforms, isolating silicon-level execution efficiency from higher-level framework dispatch overhead.

Measuring isolated kernels requires rigorous experimental control. Without disciplined measurement protocols, dynamic hardware frequency scaling, asynchronous execution queues, and cache state introduce transient artifacts that distort kernel runtime.

11 | **cuDNN (CUDA deep neural network library)**: Released by NVIDIA in 2014, cuDNN provides hand-tuned kernel implementations for convolutions, pooling, and normalization. The benchmarking implication: reported inference latencies depend heavily on which cuDNN version and algorithm autotuner settings were used, making cuDNN version a mandatory element of any reproducible benchmark specification.



Systems Perspective 12.4: Micro-benchmarking rules

Five rules keep a micro-benchmark measuring the kernel rather than a hardware artifact:

1. **The warm-up rule**: Do not measure cold-start iterations as steady-state performance. Modern hardware uses DVFS (dynamic voltage and frequency scaling) and dynamic frequency boosting; caches, kernels, and clocks need warm-up before the measured loop represents sustained behavior.
2. **The variance rule**: Report the coefficient of variation (CV) ($\text{CV} = \sigma_{\text{run}}/\mu_{\text{run}}$), where σ_{run} and μ_{run} are the standard deviation and mean across repeated runs. Investigate values above the protocol's workload-specific tolerance; common causes include background OS jitter, thermal throttling, and memory contention, and 5 percent is not a universal cutoff.
3. **The "speed of light" (SOL) check**: Compare the achieved throughput against the roofline. If a kernel achieves 10 TFLOP/s on an H100 (peak ~989 TFLOP/s FP16, or ~1,979 TFLOP/s FP8 dense), the diagnostic step is to identify the cause of low utilization (often kernel launch latency from too many small kernels) before optimizing the code itself.
4. **The cache-state rule**: Set cache state to match the claim. Flush or exceed the L2 cache when measuring cold dynamic random-access memory (DRAM) bandwidth; preserve a warmed cache when measuring steady-state reuse. Otherwise a nominal DRAM result may instead reflect cache bandwidth (~5 TB/s–10 TB/s) rather than DRAM bandwidth (~1 TB/s–2 TB/s).
5. **The synchronization barrier rule**: Accelerator kernel launches are asynchronous on the host CPU. Timing GPU operations using CPU clocks without an explicit device barrier (such as `torch.cuda.synchronize()`) measures host enqueue latency rather than kernel execution; always place synchronization barriers immediately before and after the timed region, or use GPU-side timestamp events (`cudaEventRecord`).

A profiler translates these measurement rules into physical diagnostics by decomposing kernel execution time into the three governing terms of the iron law: data movement, compute throughput, and latency overhead.



Systems Perspective 12.5: Measuring the iron law terms

Moving from theory to trace means mapping the iron law equation onto a profiler timeline (like Nsight Systems or PyTorch Profiler).

Measuring the data term $\left(\frac{D_{\text{vol}}}{\text{BW}}\right)$

- **Signal**: Look for the "Memory Throughput" or "DRAM Bandwidth" line.

- **Formula**: $\text{BW}_{\text{effective}} = \frac{D_{\text{vol}}}{T_{\text{kernel}}}$.
- **Diagnosis**: If $\text{BW}_{\text{effective}} \approx \text{BW}_{\text{peak}}$ (for example, close to the A100's 2.04 TB/s peak), bandwidth pressure is a strong candidate bottleneck. Test whether reducing traffic or increasing bandwidth improves time; a high bandwidth reading alone does not establish that changing compute (O) has no effect.

Measuring achieved compute throughput $(R_{\text{peak}} \cdot \eta_{\text{hw}})$

- **Signal**: Distinguish "SM Active," the fraction of time the GPU's Streaming Multiprocessors (SMs), its parallel execution cores, have work resident, from executed-operation throughput. A busy multiprocessor does not imply high useful-model FLOP/s.
- **Formula**: $R_{\text{peak}} \cdot \eta_{\text{hw}} = \frac{O}{T_{\text{kernel}}}$, reported in TFLOP/s.
- **Diagnosis**: If $\eta_{\text{hw}} \ll 1$ AND $\text{BW}_{\text{effective}} \ll \text{BW}_{\text{peak}}$, neither quoted peak is being approached. Kernels too small to amortize launch latency or too few threads to occupy the SMs are candidate explanations, but low aggregate rates alone do not exclude dependency stalls or an inefficient memory-access path. Separate inter-kernel gaps from in-kernel stalls, then inspect the corresponding dispatch, dependency, occupancy, or memory-access evidence before selecting an optimization.

Measuring the latency term (L_{lat})

- **Signal**: Look for gaps (empty space) between colored kernel bars on the timeline.
- **Formula**: Overhead Ratio $= \frac{T_{\text{gap}}}{T_{\text{kernel}} + T_{\text{gap}}}$.
- **Diagnosis**: A "Sawtooth" pattern (Compute, Gap, Compute, Gap) suggests exposed waiting; correlate CPU dispatch, dependencies, and transfers before attributing it to software overhead. If dispatch dominates, candidates include kernel fusion, covered in section 11.8.1.3, or CUDA Graphs, which capture a repeated sequence of GPU launches so the runtime can replay it with less CPU dispatch overhead.

While standardized benchmarks measure aggregate throughput, profilers reveal the physical mechanisms that cause execution to stall. System diagnosis uses two complementary levels of instrumentation: framework profilers and kernel profilers.

Framework profilers

Framework profilers, such as the PyTorch Profiler, capture the logical execution trace of a training or inference iteration. They track operator dispatch on the host CPU, determine whether host preparation overlaps asynchronously with accelerator execution, and identify whether the data loader pipeline starves compute queues. The primary diagnostic metric is the step-time breakdown partitioned across data loading, host launch latency, device compute, and inter-device communication. This breakdown pinpoints which subsystem bounds iteration latency before inspecting individual kernels.

↩ The feeding tax, the wall-clock time an accelerator loses waiting on input I/O, is established in section 4.2.3.

Kernel profilers

Kernel profilers, such as NVIDIA Nsight Systems and Nsight Compute, capture physical execution on accelerator silicon. Nsight Systems constructs a global timeline across CPU threads, CUDA API calls, memory transfers, and kernel durations to expose serialization bubbles. Nsight Compute conducts in-depth analysis of individual kernel invocations, measuring SM warp occupancy, instruction issue stalls, and memory coalescing efficiency against the hardware roofline. Kernel profilers extract these diagnostics directly from on-chip Performance Monitoring Units (PMUs), capturing hardware counters for HBM byte traffic, L2 cache hit rates, and pipeline stall reasons, but counter collection and kernel replay can perturb timing. Compare diagnostic traces with minimally instrumented measurements.

Interpreting these silicon metrics starts from the warp model of section 11.4.2, in which nearby, aligned addresses let a warp's loads use fewer memory transactions; transaction granularity depends

on the device and access path. Scattered or strided addresses force separate transactions that move bytes no thread uses; a 4-byte load from each 32-byte sector, for example, uses 4 of every 32 bytes fetched. Profilers report the ratio as memory coalescing efficiency, pinpointing why a kernel achieves only a fraction of peak bandwidth.

↩ The single instruction, multiple threads (SIMT) warp model, in which 32 threads issue one instruction and contiguous loads merge into a single coalesced transaction, is established in section 11.4.2.

A rigorous profiling workflow proceeds top-down. An engineer begins with a framework profiler to isolate the dominant layer in the execution timeline (for example, identifying that a multi-head attention block accounts for 65 percent of step latency). The engineer then applies a kernel profiler to analyze that layer down to the silicon (for example, verifying that the softmax kernel achieves only a small fraction of peak memory bandwidth due to non-coalesced DRAM accesses). This two-stage workflow prevents misdirected optimization by ensuring engineering effort targets the limiting physical bottleneck.

These granular measurements enable precise kernel optimization, but they cannot reveal how components interact when assembled into complete models. Macro-benchmarks address this gap.

12.4.2 Macro benchmarks

Micro-benchmarks verify that isolated operators achieve high compute or memory efficiency. Macro-benchmarks determine whether those gains persist when layers are composed into a complete model. Composing layers introduces physical interactions that isolated benchmarks cannot observe: intermediate activation tensors expand peak memory footprint, sequential kernel launches expose CPU driver dispatch overhead, and memory access streams from preceding operations evict shared cache lines before downstream kernels can reuse them.

Macro-benchmarks serve a central engineering objective: evaluating model architectures and runtime configurations under standardized workloads. Model-level evaluation captures the trade-offs that emerge from layer composition: generalization accuracy on unseen validation data, dynamic memory allocation across batch sizes and context lengths, and aggregate throughput across varied request arrival patterns. These dimensions interact directly: an architecture that achieves higher accuracy may require an activation footprint that forces a smaller batch size under device memory constraints, collapsing the computational throughput that justified its adoption.

Evaluating complete models requires standardized datasets, fixed reference tasks, and reproducible metric definitions. In computer vision, standard benchmarks evaluate models on ImageNet (Deng et al. 2024), measuring top-1 classification accuracy alongside inference latency across fixed batch sizes. In natural language processing, standardized tasks evaluate sequence-to-sequence translation and autoregressive generation, quantifying output quality against token generation rate across defined context lengths.

Several industry-standard suites establish reproducible model-level comparisons across platforms. The MLPerf suite standardizes evaluation across data centers, edge appliances, mobile devices, and microcontrollers, as detailed in section 12.8.4. For resource-constrained embedded targets, EEMBC's MLMark quantifies inference throughput and energy consumption under strict memory budgets, while the AI-Benchmark (Ignatov and Timofte 2024) suite evaluates mobile systems-on-chip across heterogeneous compute units.

12.4.3 End-to-end benchmarks

End-to-end benchmarks encompass the entire production pipeline, capturing every stage required to ingest raw data, generate model predictions, and deliver results to downstream clients. While micro- and macro-benchmarks focus on accelerator execution, real-world serving pipelines frequently find their primary bottlenecks outside the model itself.

Data preprocessing forms the initial phase of the pipeline, converting raw external inputs into model-ready tensor representations. In computer vision pipelines, image ingestion requires decoding compressed file formats (such as JPEG decompression), resizing, cropping, and color normalization. When executed on host CPUs without hardware acceleration, these preprocessing steps can take several times longer than model inference on a modern accelerator, leaving high-throughput tensor cores idle while waiting for host memory transfers. Similarly, in natural language pipelines, subword tokenization and dynamic sequence padding on the host CPU often bound serving throughput. Postprocessing introduces analogous latency bottlenecks: object detection pipelines require

non-maximum suppression (NMS) and bounding-box coordinate transformations, which create sequential execution stalls if not integrated efficiently into the execution graph.

Distributed infrastructure components introduce further performance constraints beyond the compute fabric. Remote storage systems bound training throughput when reading large multimodal datasets, and network interface card (NIC) saturation in distributed clusters throttles collective communication. End-to-end benchmarks must measure pipeline throughput under realistic network latencies, concurrent query arrival patterns, and actual storage I/O constraints to ensure reproducible evaluation of the deployment as an integrated system.

Public benchmark suites rarely measure storage, networking, and compute within a single unified protocol. While MLPerf Training and Inference evaluate end-to-end scenarios more closely than isolated operator tests, their closed divisions deliberately standardize input data loading to isolate accelerator performance from storage and network variability. Consequently, production end-to-end benchmarking remains primarily an internal engineering practice, evaluated through distributed tracing and production telemetry on live query distributions.

12.4.4 Granularity trade-offs and selection criteria

Table 12.4 contrasts the diagnostic focus, scope, and engineering trade-offs across the three benchmarking granularities. Micro-benchmarks isolate low-level operations to guide kernel optimization and hardware selection, macro-benchmarks evaluate complete models to guide architectural choices, and end-to-end benchmarks measure full pipelines to ensure production deployments satisfy real-world service requirements.

Table 12.4: Benchmarking Granularity Levels: Different benchmark scopes target distinct stages of ML system development. Micro-benchmarks isolate individual operations for low-level optimization, macro-benchmarks evaluate complete models to guide architectural choices, and end-to-end benchmarks assess full system performance in production environments.

Component	Micro Benchmarks	Macro Benchmarks	End-to-End Benchmarks
Focus	Individual operations	Complete models	Full system pipeline
Scope	Tensor ops, layers, activations	Model architecture, training, inference	Extract, transform, load; model; infrastructure
Example	Conv layer performance on cuDNN	ResNet-50 on ImageNet	Production recommendation system
Advantages	Precise bottleneck identification, Component optimization	Model architecture comparison, Standardized evaluation	Realistic performance assessment, System-wide insights
Challenges	May miss interaction effects	Limited infrastructure insights	Complex to standardize, Often proprietary
Typical Use	Hardware selection, Operation optimization	Model selection, Research comparison	Production system evaluation

Selecting a single granularity level is rarely sufficient because benchmarking presents an inherent trade-off between diagnostic isolation and production fidelity. Along the descending diagonal in figure 12.3, evaluation regimes trade diagnostic power for operational realism: micro-benchmarks (the red point at the top left) isolate kernel-level memory bandwidth and warp occupancy at the expense of end-to-end context, macro-benchmarks (the blue center point) capture composed model topologies, and end-to-end benchmarks (the green point at the bottom right) encompass live production traffic but obscure the root causes of latency spikes. Effective evaluation requires strategically combining all three tiers to verify that kernel speedups survive full-system deployment.

This trade-off maps directly onto the D·A·M taxonomy. Micro-benchmarks assess the physical capability of the Machine (M) executing a primitive Algorithm (A). Macro-benchmarks evaluate how the Algorithm (A) orchestrates tensor data flow across composed layers on the Machine (M). End-to-end benchmarks evaluate how raw Data (D) ingestion from external storage and network infrastructure interacts with both Algorithm and Machine under production load. Systematic performance engineering requires evaluating all three tiers to ensure local kernel optimizations translate into global system speedups.

Selecting the appropriate granularity defines the physical boundary of measurement, but establishing that boundary is only the first step. A complete benchmark also requires specifying the operational ingredients: the task definition, representative datasets, target models, and evaluation

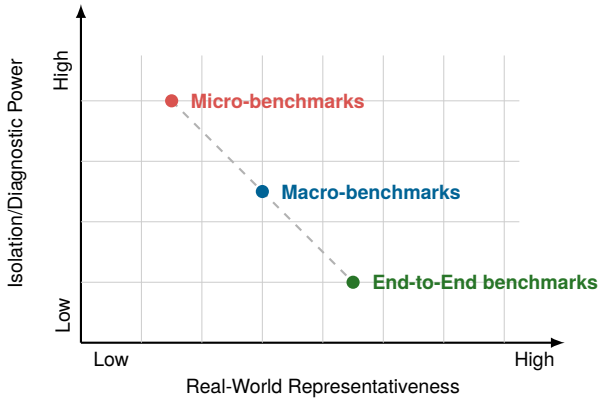


Figure 12.3: Isolation vs. Representativeness: The core trade-off in benchmarking granularity. Micro-benchmarks provide high diagnostic precision but limited real-world relevance, while end-to-end benchmarks capture realistic system behavior but offer less precise component-level insights. Effective ML system evaluation requires strategic combination of all three levels.

metrics. Without these concrete components, measurements at any granularity fail to produce actionable engineering conclusions.

12.5 Benchmark Components

Choosing between micro, macro, and end-to-end granularity determines what a benchmark can diagnose, but at every granularity the benchmark must also specify the task, data, model, metrics, harness, system context, and run rules that make its results interpretable. A synthetic matrix-multiplication loop, a ResNet-50 run over ImageNet, and a full serving pipeline all need these specifications, and none of the components can be selected in isolation, because each one fixes physical and algorithmic boundary conditions for the next.

These components interconnect to form an evaluation pipeline structured by the D·A·M taxonomy (Data, Algorithm, and Machine). The multi-stage pipeline in figure 12.4 traces an industrial audio anomaly detection benchmark across nine sequential phases, carrying the task from initial problem definition in the top chevrons through spectrogram extraction, autoencoder training, and INT8 quantization down to embedded deployment on an ARM mbed OS microcontroller. Each stage fixes boundary conditions for the next: the problem definition dictates the acoustic sampling rate and input representation (Data); the dataset properties and task objective determine feasible model architectures (Algorithm); and the microcontroller's hardware limits (SRAM capacity, energy envelope, and real-time latency deadlines) dictate INT8 quantization and memory-efficient kernel compilation (Machine). Anomaly detection highlights this full-stack coupling directly: a benchmark that measures only classification accuracy or only raw inference latency misses cross-stage interactions, where an engineering decision at any stage narrows the feasible design space downstream. This audio pipeline anchors the components that follow because one small embedded model touches every stage of the workflow; MobileNetV2 remains the running example for the deployment arc.

Evaluating an ML system requires coupling statistical task metrics with physical efficiency metrics. Quantization and pruning reduce parameter footprint and memory bus traffic, but they alter numerical representations and risk degrading task accuracy. A benchmark that records execution latency without statistical accuracy (such as area under the ROC curve (AUC) or classification error) rewards aggressive approximations that produce invalid outputs; conversely, reporting accuracy without latency or memory consumption conceals whether the model satisfies the target system's physical constraints. Interpreting these results requires placing the workload's arithmetic intensity against the machine's ridge point on the roofline, identifying whether an optimization relieves a memory-bandwidth bottleneck or a compute-throughput bottleneck.

The measurement harness and run rules provide the experimental controls that make these evaluations reproducible. The protocol follows the question being tested. A controlled steady-state experiment can use warm-up iterations, fixed processor clocks, and pinned host memory to isolate execution differences. A deployment benchmark instead retains the relevant dynamic voltage and

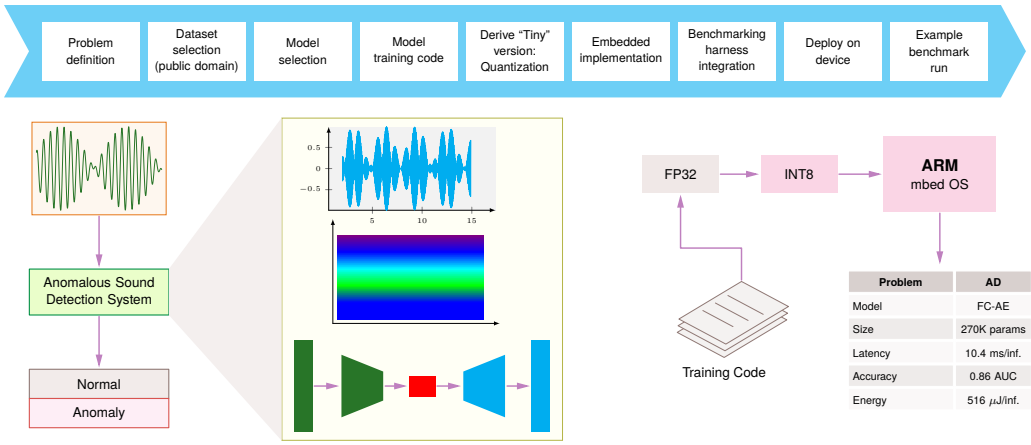


Figure 12.4: Benchmark Workflow Components: Read left to right: each stage fixes assumptions for the next, carrying an audio anomaly-detection task from problem definition to a reproducible embedded-device benchmark run.

frequency scaling (DVFS) policy, cold starts, and competing workloads, documenting their effects. Recording tail latency distributions (p95, p99) alongside the mean exposes delays hidden by averages. These operational choices define what the measurements support, establishing the foundation for examining each component in detail, beginning with the problem definition.

12.5.1 Problem definition

Every benchmark begins by fixing the computational and operational boundaries of the workload. Without an explicit task specification, hardware and software optimizations cannot be evaluated on equivalent terms: altering an input resolution, a numerical datatype, or a latency deadline fundamentally changes the physical work demanded of the underlying machine. In the anomalous sound detection system in figure 12.4, an application stages raw acoustic waveform signals into binary operational decisions ("Normal" versus "Anomaly"). Across domains—whether natural language processing tasks such as machine translation, question answering (Hirschberg and Manning 2015), and text classification, or computer vision tasks such as object detection and image segmentation (Everingham et al. 2009; Lin et al. 2014)—a complete problem definition establishes three concrete specifications: an input specification, an output specification, and a performance specification.

The input and output specifications establish the data movement contract and define the exact scope of timed execution. The input specification fixes tensor dimensions, batching behavior, numerical representations (such as FP16 or INT8), and ingestion rates. In the audio anomaly detection pipeline, streaming single-channel audio at 16 kHz in fixed one-second windows establishes a rigid ingestion rate and memory buffer requirement before feature extraction begins. Conversely, the output specification defines the termination boundary of the computation. Benchmarks frequently introduce measurement bias by timing only accelerator forward passes while ignoring the CPU post-processing required to turn raw tensor outputs into usable decisions. A robust output specification defines whether the system emits raw logits, intermediate embeddings, or final binary anomaly flags, accounting for any downstream thresholding or reduction kernels that consume host memory bandwidth.

The performance specification defines the operational envelope that governs whether an optimization is viable. Unconstrained throughput numbers are meaningless if statistical accuracy degrades below acceptable thresholds, and high average throughput cannot rescue a system that violates latency SLAs. For real-time anomaly detection, the performance specification couples a target quality metric (such as area under the ROC curve) with hardware execution limits: a specified tail-latency ceiling (such as a 99th-percentile response time p99 $\leq$ 10 ms), sustained throughput matching the data arrival rate, and a strict resident memory footprint to prevent out-of-memory faults on memory-constrained accelerators. By defining these three boundaries up front, the benchmark ensures that subsequent stages—from dataset construction to measurement harness configuration—evaluate system optimizations against the true operational constraints of deployment.

12.5.2 Standardized datasets

A task definition is only as rigorous as the data used to evaluate it. Standardized datasets enforce identical evaluation conditions across competing approaches, preventing teams from cherry-picking private slices that obscure comparative weaknesses. In computer vision, ImageNet (Deng et al. 2024, 2009), COCO (Lin et al. 2014), and CIFAR-10 (Krizhevsky 2009) establish reference tasks across classification and detection; in natural language processing, SQuAD[12] (Rajpurkar et al. 2016), GLUE[13] (A. Wang et al. 2018), and WikiText (Merity 2016; Merity et al. 2016) fulfill equivalent roles by standardizing comprehension, classification, and language modeling across diverse textual domains.

Dataset selection is the first place a benchmark can lose contact with deployment reality. At the dataset selection stage of figure 12.4 (the second chevron), the benchmark pipeline ingests representative audio waveform samples of normal machine operation alongside anomalous conditions before spectrogram feature extraction. Domain-specific collections cover different audio tasks: ToyADMOS[14] (Koizumi et al. 2019) supports controlled anomaly-detection research, while Google Speech Commands (Warden 2018) supports keyword recognition. Effective benchmark datasets must balance two competing demands: accurately representing real-world challenges while maintaining sufficient complexity to differentiate model performance. Simplified datasets like ToyADMOS are valuable for methodological development but may not capture the full complexity of production environments.

12.5.3 Model selection

With the task and dataset fixed, the benchmark must specify the model architecture and its baseline implementations. In systems benchmarking, the model defines the execution workload: its computational graph determines arithmetic intensity, memory footprint, and communication topology. Selecting the benchmark model determines whether measured performance variations reflect architectural efficiency, software runtime optimizations, or raw accelerator capacity. Evaluating an architectural modification requires holding the runtime and hardware target constant. Conversely, evaluating hardware platforms or software execution engines requires standardizing on a fixed, representative model graph so that throughput differences isolate execution efficiency rather than mathematical changes to the workload.

Baseline models establish the empirical reference point for a target domain. Rather than toy algorithms, systems benchmarks rely on canonical workloads whose computational profiles and memory access patterns are well-characterized. In language processing, BERT[15] serves as a standard baseline because its multi-head self-attention and dense feed-forward blocks exercise both compute-bound matrix multiplications and memory-bandwidth-bound reduction operations. However, a baseline specification cannot stop at the mathematical graph. Two implementations of the identical architecture across different frameworks or runtimes often exhibit divergent execution characteristics. Differences in kernel fusion (such as fusing normalization and activation functions directly into GEMM output epilogues to avoid round-trips to HBM), kernel launch overhead, and memory allocator behavior can exceed the performance impact of minor architectural variations. A comparison that isolates hardware must control the runtime implementation alongside the mathematical model; a runtime comparison instead names that implementation as the intervention and documents both stacks.

A benchmark must select the operational regime relevant to its engineering question. In training, memory capacity and interconnect bandwidth can be binding constraints: storing forward activations, backward gradients, and optimizer states across multiple accelerators limits batch size and demands high-bandwidth collective communication. Training benchmarks therefore measure time-to-accuracy under fixed convergence criteria rather than raw iteration speed alone. In inference, backward passes and optimizer states disappear. Evaluation instead emphasizes the serving objective, such as tail latency under SLAs, while low-batch autoregressive decode often faces bandwidth pressure. Transitioning a model to production frequently requires precision reduction—quantizing FP32 or BF16 weights and activations to INT8 or FP8. Moving eligible tensors from a 16-bit to an 8-bit format halves their raw storage and can reduce traffic or raise supported arithmetic throughput. Whole-model gains depend on the execution path, and validation must establish whether numerical error stays within the quality target. A development-to-deployment comparison may need both

[12] **SQuAD (Stanford question answering dataset)**: Introduced in 2016 with more than 100,000 question-answer pairs from Wikipedia (Rajpurkar et al. 2016). AI systems exceeded the SQuAD 1.1 human baseline of 91.2 percent F1 by 2018 (Devlin et al. 2019), but this "superhuman" result illustrates a benchmarking failure mode: the task's extractive format (answers are text spans within the passage) makes it easier than open-ended question answering, inflating perceived capability relative to production NLP systems.

[13] **GLUE (general language understanding evaluation)**: Introduced in 2018 as a broad language-understanding benchmark (A. Wang et al. 2018), GLUE was quickly saturated by systems such as BERT (Devlin et al. 2019). This is Goodhart's Law in action: once GLUE became a target, leaderboard optimization reduced its discriminating power. The pattern motivated harder follow-on evaluations such as SuperGLUE and BIG-bench.

[14] **ToyADMOS**: Developed by NTT Communications in 2019 for acoustic anomaly detection, containing audio recordings from toy car, toy conveyor, and related miniature-machine operating sounds (Koizumi et al. 2019). The "toy" prefix is intentional: the controlled environment enables reproducible benchmarking but can create a domain gap when models are moved to noisier industrial environments with different machines, sensors, vibration, and background sound.

[15] **BERT (bidirectional encoder representations from transformers)**: BERT-Large (340M parameters, a larger sibling of the BERT-Base used in the roofline notebook) is a language-processing workload in MLPerf Inference. The benchmark fixes the task, data, quality target, and applicable execution scenarios so systems can be compared on the same workload. BERT latency still depends on sequence length, batching, and implementation.

regimes because efficient training does not establish serving feasibility; a deployment decision for a fixed pretrained model can focus on inference.

12.5.4 Evaluation metrics

Evaluation metrics[16] translate raw model behavior into numbers that can be compared, ranked, and used to make engineering decisions. The challenge is choosing the right numbers: a metric that captures accuracy but ignores latency may declare the winner to be a model too slow for production; one that rewards throughput but ignores energy may optimize for a deployment budget that does not exist.

Table 12.5 categorizes evaluation measures by the failure mode each exposes and the deployment context it serves.

16 **Metric**: In mathematics, a metric is a distance function satisfying strict axioms including the triangle inequality. ML borrows the term loosely for quantitative measures such as BLEU and perplexity, which are scoring rules rather than mathematical metrics. Leaderboard rankings can change when the evaluation protocol, dataset slice, or metric weighting changes, making the choice of metric an engineering decision that shapes which system wins, not just how performance is measured.

Table 12.5: ML Benchmarking Metric Taxonomy: Metrics organized by evaluation category, unit, and primary use case. Accuracy metrics quantify model quality, throughput and latency metrics capture system speed, and efficiency metrics combine multiple dimensions. Selecting the right metric for the deployment context is often more important than optimizing any single metric to its maximum.

Category	Metric	Unit	Primary Use Case
Accuracy	Top-1/Top-5 Accuracy	Percentage	Classification
	mAP (mean Average Precision)	0–1 score	Object detection
	BLEU/ROUGE	0–100 score	NLP generation
	Perplexity	Score (lower = better)	Language modeling
Throughput	Samples/second	Samples/s	Batch inference
	Token throughput	tokens/s	LLM inference
	Time-to-train	Hours/days	Training benchmarks
Latency	p50 latency	Milliseconds	Median response time
	p99 latency	Milliseconds	Tail latency (SLA)
	First-token latency	Milliseconds	LLM responsiveness
Efficiency	Samples/second/watt	Samples/s/W	Energy efficiency
	Accuracy/FLOP	percent/PFLOP	Algorithmic efficiency
	TCO per inference	$/inference	Economic efficiency

The metrics in table 12.5 expose physical trade-offs governed by the accelerator architecture. Throughput measures aggregate processing capacity, whereas latency measures individual request completion time. These two objectives conflict directly at the hardware boundary. Maximizing throughput requires aggregating requests into larger batches to amortize weight-loading memory bandwidth across multiple inputs, raising arithmetic intensity on tensor units. That batching mechanism inflates per-sample latency: individual requests wait in arrival queues while a batch forms, and larger matrix operations take longer to complete. Furthermore, arithmetic mean latency masks tail behavior. Queuing jitter, memory allocation stalls, and accelerator thermal throttling cause execution times to diverge; a system with a 10 ms mean latency can exhibit a 500 ms 99th-percentile (p99) latency that violates service-level agreements. Benchmarks must therefore report distribution percentiles (p50, p95, p99) rather than averages. Compound metrics such as samples/second/watt compress multiple dimensions into a single scalar, facilitating high-level comparisons but obscuring where bottlenecks reside. Reporting atomic metrics (latency distributions, throughput, and power draw) alongside compound figures isolates which physical constraint limits the system.

Task-specific metrics evaluate the algorithmic dimension, quantifying whether the model solves the statistical problem. Classification workloads evaluate accuracy (the fraction of correct predictions), precision, recall, and the F1 score (Sokolova and Lapalme 2009). Precision and recall isolate true positive rates from false alarms, which is essential when class distributions are imbalanced. Continuous estimation tasks rely on Mean Squared Error (MSE) or Mean Absolute Error (MAE), while sequence generation uses specialized scoring rules such as BLEU[17] to measure modified n-gram precision against human reference translations (Papineni et al. 2002).

Task metrics evaluate algorithmic quality in isolation, but deployment viability depends on physical machine constraints. Model capacity, measured in parameter count and memory footprint, determines whether weights fit into on-chip SRAM or accelerator memory, or whether they spill

17 **BLEU (bilingual evaluation understudy)**: Introduced by IBM in 2002, BLEU measures translation quality through modified n-gram precision with a brevity penalty against reference translations (Papineni et al. 2002). BLEU is a canonical example of Goodhart's Law in ML: optimizing for n-gram matches can reward surface-level word overlap even when meaning, fluency, or deployment usefulness diverges from the target.

across high-latency interconnects. Per-inference execution latency dictates whether a system meets real-time deadlines, while energy consumption per inference (measured in joules or millijoules) bounds thermal dissipation and battery life.

Benchmarks must also account for measurement sensitivity across software runtimes. As established in chapter 8, different frameworks handle loss computation, kernel fusion, and gradient accumulation differently, altering reported benchmark numbers. Subtle implementation differences, such as whether batch normalization is configured to use stored running statistics or current mini-batch statistics during evaluation, or how non-associative floating-point additions are reordered in reduction kernels, shift measured accuracy and latency enough to invert benchmark rankings when candidate models differ by narrow margins.

This interdependence between statistical quality and hardware constraints is evident in anomaly detection. In industrial monitoring, extreme class imbalance renders raw accuracy deceptive: if anomalies constitute only 0.1 percent of operating samples, a trivial model that constantly predicts normal operation achieves 99.9 percent accuracy while failing to detect any faults. The benchmark must instead report AUC, which scores how well the model ranks anomalous samples above normal ones across every decision threshold, alongside physical resource metrics.

This multi-metric evaluation approach governs the anomaly detection pipeline, which balances four simultaneous constraints: a model size of 270K parameters whose stored representation and runtime buffers must fit the embedded SRAM budget, a processing latency of 10.4 ms per inference to maintain real-time sampling, a detection accuracy of 0.86 AUC, and an energy consumption of 516 µJ per inference within the target battery budget. Evaluating across all four axes prevents declaring a model superior when its accuracy gain stems from an architecture too large to fit in memory or too power-hungry to survive sustained deployment.

12.5.5 Benchmark harness

Metrics define *what* to measure; the benchmark harness determines *how* to measure it. The harness is the software and instrumentation scaffold that feeds inputs to the system under test, coordinates execution, collects timing and power traces, and enforces reproducibility. In an ML system, measuring execution down to the metal requires accounting for the asynchronous execution boundary between the host CPU and the accelerator. When host software initiates an inference or training step, the framework runtime merely enqueues commands onto an accelerator work queue (such as a CUDA stream or hardware command buffer) and returns control immediately. A naive timer wrapping the host-level call measures only this sub-microsecond queue dispatch latency rather than actual device execution. A rigorous harness inserts explicit device synchronization barriers (such as `torch.cuda.synchronize()`) or queries on-device timestamp events (such as `torch.cuda.Event`), ensuring that recorded timings reflect complete kernel execution and memory transfers across the bus rather than mere host enqueue overhead.

Harness architecture must align with the operational regime of the deployment target. In server environments, the harness generates concurrent request traffic using an open-loop load generator, often parameterizing inter-arrival times with a Poisson distribution.[18] An open-loop harness issues requests according to an independent arrival schedule regardless of server responsiveness. This decoupling exposes queuing delays and tail latency (such as p99 response times) when arrival bursts saturate available compute.

In contrast, a naive benchmarking client often employs a closed-loop pattern: sending a request, blocking until the response arrives, and only then issuing the next request. This design introduces the severe measurement distortion known as **coordinated omission**. Consider a server that experiences a transient 500 ms garbage collection pause or memory reallocation stall. In an open-loop production environment, independent users continue submitting queries, causing dozens of requests to accumulate in the server's ingress queue; once the pause ends, every queued query suffers inflated tail latency. In a closed-loop benchmark, however, the client blocks idly during the entire 500 ms pause, sending zero traffic while the server is struggling. The benchmark records a single slow query and zero queue backlog. By coordinating request generation with server responsiveness, the closed-loop harness artificially paces traffic, systematically erasing the very queue collapse and p99 latency spikes it was constructed to measure. Conversely, offline batch processing eliminates arrival

[18] **Poisson distribution**: Named after Siméon Denis Poisson, who formalized it in 1837 while modeling wrongful conviction rates in French courts. The distribution models independent events at a constant average rate (λ_{arr}) and is a common baseline for server request arrivals. Real ML serving traffic often violates its assumptions through burstiness, correlation, and time-varying rates. A Poisson harness can therefore misestimate tail latency unless traces or stress cases cover those production patterns.

scheduling entirely; the harness pre-stages large input batches in memory to saturate accelerator execution units, maximize memory bandwidth utilization, and measure peak sustained throughput.

In embedded and edge deployments, workloads are neither stochastic network streams nor unconstrained batch queues. Instead, inputs arrive sequentially from physical sensors (such as an audio analog-to-digital converter or an image sensor) governed by fixed hardware sampling clocks. The harness must inject these sensor streams through direct memory access into limited on-chip SRAM buffers, verifying that each inference completes within the inter-sample arrival deadline to prevent buffer overflow. Furthermore, edge harnesses must integrate physical instrumentation, such as shunt resistors or external power analyzers, to sample voltage and current across active execution cycles and low-power sleep states, capturing the true energy consumed per inference.

Reproducibility requires specifying which runtime and environmental conditions are controlled and which are part of the workload. The warm-up rule from micro-benchmarking (perspective 12.4) scales up here from one kernel to a whole run, because modern accelerators and runtime frameworks exhibit substantial cold-start transients. Initial inference passes trigger on-demand memory allocations, dynamic just-in-time (JIT) compilation, driver initialization, and weight transfers from secondary storage into accelerator memory. For steady-state measurements, the harness executes unmeasured warm-up iterations until caches populate, memory allocators stabilize, and the intended thermal operating state is reached. Controlled experiments may fix clocks and remove competing processes; deployment tests retain and report the relevant DVFS policy and contention. Cold-start measurements time initialization separately instead of discarding it. Finally, telemetry logging must minimize the probe effect by recording execution timestamps and performance counters into non-blocking, pre-allocated memory buffers. The remaining instrumentation overhead should be checked against a minimally instrumented run.

12.5.6 System specifications

Complementing the harness that controls execution conditions, system specifications record the computational environment: the physical hardware and software stack that establishes the Machine (M) boundaries of the system. Without complete specifications, a reported throughput or latency measurement cannot be interpreted. An execution metric reflects the interaction between an algorithm's arithmetic intensity and a machine's physical limits: peak arithmetic throughput, memory hierarchy bandwidth, and interconnect latency. A throughput figure reported without accelerator model, memory bandwidth, or operating clocks conceals whether an observed speedup stems from algorithmic efficiency or upgraded silicon.

Rigorous hardware specifications capture the constraints that dictate kernel execution and data movement. At the accelerator level, documentation must record the microarchitecture, compute capability, active core counts, base and boost clock frequencies, and power limits (thermal design power (TDP)), alongside the device memory hierarchy: HBM capacity, memory bus width, and peak memory bandwidth. For multi-accelerator configurations scaling from one to eight devices, specifications must delineate the host-to-device bus interface (such as PCIe lanes) and the inter-accelerator fabric—whether communication traverses point-to-point links (such as NVLink) or shared PCIe switches. Because distributed collective operations such as all-reduce are bounded by interconnect topology and bisection bandwidth, omitting interconnect specifications obscures the source of communication bottlenecks.

The software stack is equally decisive, dictating how mathematical operations map onto execution units. Specifications must record the operating system kernel, device drivers, low-level compute runtimes (such as CUDA or ROCm), and vendor math libraries (such as cuBLAS or CUTLASS). At the framework layer, tracking runtime versions (such as PyTorch or JAX), execution modes (eager interpretation versus graph-level compilation via `torch.compile`), and kernel fusion optimizations separates framework overhead from model performance. Finally, specifications must quantify operational power and energy metrics, including sustained board power, thermal throttling events, and energy consumed per token or sample. Documenting these operating envelopes ensures that measured speedups are not masked artifacts of unconstrained power draws or transient clock boosting.

12.5.7 Run rules

System specifications describe what the benchmark runs on; run rules govern how it runs. These procedural constraints make results interpretable and repeatable, which is harder than it sounds in a field where the stochastic processes catalogued in section 12.3.1 mean that two runs on identical hardware can produce different numbers. A protocol may fix seeds and data order when isolating system effects, or require repeated runs and quality thresholds when stochastic variation is part of the workload. In either case, the seed policy and known sources of nondeterminism must be explicit.

Hyperparameter documentation is equally critical. A learning-rate change can shift convergence and final accuracy, so a reproducible result records every configuration setting that can affect the outcome. Dataset versions, splits, and preprocessing must likewise be identified. When privacy or licensing prevents sharing data directly, the report must state that limitation and preserve enough provenance and transformation detail to judge comparability.

Code provenance completes the reproducibility chain. Strong benchmark protocols preserve the implementation version—spanning the model, preprocessing, training, and evaluation code—and disclose whether that code can be shared. Reference suites may distribute containerized environments that encapsulate dependencies and configurations, while experimental logs retain training metrics, checkpoints, and any mid-run adjustments. Together, these records turn a one-time measurement into evidence another team can inspect and, when access permits, reproduce.

12.5.8 Result interpretation

Producing raw benchmark numbers requires only execution; interpreting them requires understanding the conditions that produced them, the statistical confidence behind them, and the operational boundaries that determine whether the numbers translate to production behavior. A single reported metric—such as latency or accuracy—rarely reveals whether an optimization succeeds in deployment. Evaluating an edge vision model on a resource-constrained device illustrates how throughput, memory footprint, and model fidelity interact under strict hardware boundaries.

Example 12.2: Benchmarking a vision model for edge deployment

Scenario: An edge AI team evaluates MobileNetV2 for a wildlife camera trap on a Raspberry Pi 4 under strict latency and model size limits (Sandler et al. 2018).

Diagnosis: FP32 execution yields 120 ms latency (8.3 FPS) and 14 MB size. INT8 quantization speeds up inference by 3.4× to 35 ms (28.6 FPS) and reduces model size by 4×, at a cost of 0.9 percentage points Top-1 accuracy.

Systems lesson: Edge model benchmarking requires evaluating multi-dimensional trade-offs across latency, accuracy, and memory payload. Quantization enables high-throughput edge execution when modest accuracy drops meet product requirements.

Translating benchmark numbers into engineering decisions requires two methodological checks: a defined intervention and statistical rigor. First, state what changes and what remains comparable. ResNet-50 versus MobileNet can be a valid architecture comparison at matched task and quality requirements; FP32 versus INT8 can be a valid precision comparison on a supported execution path. Hold the other relevant conditions fixed or report their changes, including batch size, hardware, runtime, input distribution, and service targets. Second, distinguish steady-state execution from initialization, using the harness's warm-up protocol (section 12.5.5). A single run conflates cold-start overhead (kernel compilation, page faults, and cache warming) with sustained throughput, while a prolonged stress test may instead capture the thermal throttling examined for edge devices in section 12.8.5. Report cold-start and steady-state results according to the deployment question, record variation across repeated trials, and report both uncertainty in the comparison and the latency statistic required by the service objective.

For a timing comparison, suppose an engineer measures baseline and candidate mean latency under the same request mix, offered load, and timing boundary. Each experimental trial pairs a complete baseline run with a complete candidate run, following the same warm-up and thermal-

state protocol and randomizing their execution order to limit drift bias. The comparison assumes that separately repeated run pairs are independent, with persistent background load or thermal drift controlled or accounted for. Requests within one run can share queues, caches, and thermal conditions and therefore cannot automatically count as independent trials. The report states the interval method and confidence level and uses the run pairs as the repeated observations. A request-latency p99 describes the slow end of the observed request population. It is neither a confidence interval for mean latency nor a measure of uncertainty in the estimated p99.

Suppose the report supplies an uncertainty interval for the mean latency reduction, defined as baseline latency minus candidate latency. If the interval extends from a negative reduction to a positive reduction, it includes no change, and the experiment cannot resolve whether the candidate is faster. That conclusion does not establish that the implementations perform identically. If a subsequent adequately controlled comparison yields an interval entirely above zero, it supports a latency reduction under the tested conditions. Deployment still requires a prespecified minimum useful reduction, reflecting the cost of adopting the candidate. An interval entirely above zero but extending below that threshold supports a gain without establishing that the gain justifies adoption; an interval entirely above the threshold supports both claims, provided the quality and resource gates also pass. Applying the same demand for explicit conditions to an isolated throughput claim exposes another source of misleading evidence.



Systems Perspective 12.6: Interpreting a benchmark claim

A vendor claims "Our system achieves 10,000 inferences/second on ResNet-50." Taken alone, the number cannot support deployment planning because the conditions that determine its meaning are unstated.

Four unstated conditions determine what the claim means:

1. **Batch size**: Large batches amortize weight transfers and maximize compute utilization but inflate queuing delay; batch 1 minimizes latency but leaves execution units underutilized.
2. **Precision**: INT8 execution exploits higher-throughput integer pipelines and cuts memory traffic, but requires calibration that can compromise numerical fidelity.
3. **Measurement boundary**: The measurement window must state whether it isolates pure accelerator kernel execution or includes host-to-device transfers, JPEG decoding, and tensor format conversions.
4. **Accuracy**: The claim must state whether the optimized model retains baseline validation accuracy (such as 76.1 percent Top-1 on ImageNet) or trades task quality for raw execution speed.

Example: "10,000 inferences/second on ResNet-50 at batch size 32, INT8 precision, 76 percent Top-1 accuracy, including JPEG decoding, on NVIDIA H100 at 700 W TDP."

Systems insight: Evaluating whether a performance difference is meaningful requires both statistical rigor and contextual validation. A throughput figure lacking batch size, precision, measurement scope, and accuracy targets is a marketing metric rather than an engineering specification.

Beyond vendor claims, deployment constraints dictate which metrics govern system viability. In safety-critical applications such as medical imaging or autonomous driving, a 1 percent accuracy gain justifies substantial computational cost; in interactive search or speech transcription, tail latency within a tight service-level agreement dominates, rendering minor accuracy gains irrelevant if they breach latency budgets. Systems evaluation must also guard against *benchmark overfitting*, wherein an optimization targets the idiosyncratic computational patterns of a standardized suite at the expense of production workloads. For example, aggressive kernel fusion or operator tiling tuned exclusively for fixed sequence lengths in a benchmark harness can degrade throughput when deployed against dynamic, variable-length user requests. Robust interpretation requires validating performance across diverse input distributions, perturbed batch sizes, and representative operational duty cycles.

12.5.9 Example benchmark

The deployed anomaly detection workflow in figure 12.4 records the four measurements from section 12.5.4 in its terminal reporting table (lower right). For the fully connected autoencoder (FC-AE) on the ARM mbed OS target, each certifies something different. Parameter count and per-inference latency certify computational resources, the AUC certifies task effectiveness, and energy per inference certifies operational efficiency.

Which of these metrics governs viability depends entirely on the deployment environment and its physical constraints. Energy per inference dictates battery life on untethered edge devices and bounds power distribution limits and cooling costs in server racks. Model size determines whether weights reside entirely within fast on-chip SRAM or spill into off-chip DRAM, while in multi-tenant cloud accelerators, parameter volume dictates how many model replicas fit within HBM. Inference latency determines whether the system satisfies real-time deadlines on streaming audio without dropping frames, whereas batching trades per-sample latency for higher throughput by amortizing weight-transfer overhead across multiple inputs. These metrics expose direct engineering trade-offs: reducing model size below 270K parameters can reduce memory traffic and execution time on a supported path, but risks degrading the 0.86 AUC detection accuracy. Whether these measurements constitute a passing benchmark depends on the operational envelope; the benchmark specification standardizes the evaluation protocol, but application requirements dictate the acceptance thresholds.

The audio example fixes the task, data, model, and measurement boundary, but its acceptance criteria concern inference on an embedded device. A training experiment instead has to measure the work needed to learn a model that reaches the quality target. The same protocol components support both decisions, while the timed workload and acceptance criteria change.

12.6 Training vs. Inference

The same accelerator can fail in opposite operational regimes: a training run can stall when inter-accelerator gradient synchronization saturates the interconnect, while a serving deployment can violate its SLO when request bursts create queueing delays. Training and inference impose evaluation requirements so divergent that separate benchmarking suites exist for each: MLPerf Training and MLPerf Inference (Mattson et al. 2020; Reddi et al. 2019). Theoretical peak TFLOP/s provides little insight into either workload. Training iteratively refines parameters over billions of samples, demanding sustained high-power compute, large memory capacity for optimizer states, and efficient collective communication across accelerators. Inference evaluates fixed parameters against incoming queries under bounded response times, demanding bounded tail latency, minimal cold-start initialization latency, and high energy efficiency.

This division alters memory allocation and arithmetic intensity. In training, device memory must store model parameters, gradients, optimizer states (such as momentum and variance accumulators), and intermediate activations retained for backpropagation. These state requirements can greatly exceed weight-only footprints; the ratio depends on the optimizer, activation checkpointing, numerical precision, and batch dimensions. Mixed-precision arithmetic can alleviate capacity pressure, while gradient compression is a separate candidate for communication reduction whose effect on convergence must be tested. Inference discards the backward graph and optimizer states entirely and can free each activation after its last forward consumer; memory demand is dictated by static weights and dynamic execution buffers (such as key-value caches in autoregressive models). Serving workloads can therefore apply aggressive precision reduction (evaluated in section 12.8.2), post-training quantization, and weight pruning. Hardware utilization reflects this operational contrast: large-batch training can expose substantial arithmetic reuse, whereas low-concurrency inference may have much less reuse and greater sensitivity to launch latency. Offline or heavily batched inference can occupy a different regime, as illustrated by the roofline analysis in section 12.3.2. For a benchmark, the consequence is that a training result and an inference result on the same accelerator measure different resources and cannot stand in for each other.

↩ The training memory budget of weights, gradients, optimizer state, and retained activations is established in section 8.5.4.1.

Energy consumption follows contrasting accounting regimes across the two phases. Training energy represents a capital investment amortized across the operational lifetime of the model; a large training run can consume on the order of a thousand megawatt-hours (section 12.7.2.4) (Patterson et al. 2021). Inference energy accumulates continuously with request volume, dominating total lifecycle cost as query counts scale into millions. For an isolated inference window containing

one completed request, energy obeys the same identity $E_{\text{inf}} = P_{\text{active}} \times L_{\text{lat,infer}}$ developed for edge battery accounting in section 12.8.5.2, with P_{active} now the average accelerator power during that window and $L_{\text{lat,infer}}$ its duration. With batching or concurrent requests, integrate energy over a common measurement window and divide by completed requests; multiplying total device power by each request's overlapping serving latency would double-count energy. For example, if measured average accelerator power during the inference window is 300 W, a 10 ms inference uses $300\ \text{W} \times 0.01\ \text{s} = 3\ \text{J}$, or about 0.0008 Wh; at 100 ms, that becomes about 0.0083 Wh. An accelerator's TDP rating represents a cooling ceiling rather than dynamic power consumption, making empirical runtime power measurement essential for accurate benchmarking.

These divergent physical requirements shape benchmark methodology through the D·A·M lens. Training benchmarks evaluate whether algorithmic updates converge efficiently when scaled across distributed accelerator interconnects under continuous data ingestion, where the primary figure of merit is wall-clock time to a target accuracy. Inference benchmarks evaluate whether a fixed algorithmic artifact satisfies latency percentiles (such as p95 and p99 response times) and throughput targets under bursty query traffic on provisioned hardware. The following sections first examine time-to-quality during training, then characterize the fixed model artifact under serving requirements. Deployments using pretrained models can begin with the latter.

12.7 Training Benchmarks

In an illustrative procurement failure, a team purchases a larger GPU cluster expecting proportional training-speed gains, only to discover that communication overhead and memory bottlenecks limit the actual speedup. Training benchmarks exist to catch this kind of gap before procurement. Training benchmarks span six dimensions (table 12.7), anchored by three: convergence metrics that measure learning progress, throughput metrics that measure computational efficiency, and scalability metrics that measure distributed performance.

Definition 12.3: ML training benchmarks

ML training benchmarks are machine learning system benchmarks that measure the time to reach a target quality metric (for example, a specified validation accuracy or loss threshold) on a fixed dataset and model, quantifying the rate of convergence per unit of resource.

1. **Significance**: Training benchmarks reveal large gaps invisible to hardware specs. Holding the model and quality target fixed, the time to convergence can vary widely across hardware-software stacks because training performance depends on the full pipeline: data loading $(D_{\text{vol}}/\text{BW})$, compute utilization (η_{hw}), gradient synchronization $(\text{BW}, L_{\text{lat}})$, and fault recovery overhead. A peak FLOP/s spec sheet captures none of these interactions.
2. **Distinction**: Unlike inference benchmarks, which measure per-query latency and throughput under load, training benchmarks measure time-to-accuracy across the full optimization loop: data loading, forward pass, backward pass, gradient synchronization, and optimizer step. As scale increases, communication (BW) can displace compute (R_{peak}) as the binding constraint; profiling establishes the transition for a particular workload.
3. **Common pitfall**: A frequent misconception is that training benchmarks measure "how fast the GPU runs." At large scale, interconnect bandwidth (BW) for gradient synchronization and fault tolerance overhead (checkpoint I/O, straggler mitigation) often dominate the benchmark result more than peak FLOP/s.

Training benchmarks validate whether hardware acceleration delivers promised training throughput. Accelerators and multi-chip systems all claim dramatic speedups, and training benchmarks reveal which claims hold under realistic workloads. They evaluate how hardware configurations, data loading mechanisms, and distributed training strategies perform when training production-scale models. These benchmarks are vital because training represents the largest capital expenditure in ML systems, and only rigorous time-to-accuracy measurement reveals whether that capital deliv-

ers proportional value rather than dissipating into scaling inefficiencies, memory bottlenecks, or communication overhead.

For instance, large-scale models like OpenAI's GPT-3[19] (Brown et al. 2020), which consists of 175B parameters trained on approximately 570 GB of filtered CommonCrawl text (from a ~45 TB raw dataset, combined with other sources to form 300B training tokens), highlight the immense computational demands of modern training. Standardized ML training benchmarks provide systematic evaluation of the underlying systems to ensure that hardware and software configurations can meet these unprecedented demands efficiently. Cluster-scale examples such as GPT-3 appear in this section as context for why the metrics matter; the metrics themselves (time-to-accuracy, scaling efficiency, utilization, checkpoint overhead) are defined and worked at the scale of one machine with one to eight accelerators, the scale of the scaling-efficiency notebook below.

[19] **GPT-3 (Generative Pre-trained Transformer 3)**: OpenAI's 2020 language model (175B parameters, 300B training tokens) consumed an estimated 3,640 petaFLOP-days on 10,000 V100 GPUs (Patterson et al. 2021). This scale illustrates why training benchmarks are essential for predicting whether a planned training run is operationally viable before committing the compute.

12.7.1 Training benchmark motivation

MLPerf Training (Mattson et al. 2020; MLCommons 2024c) provides the standardized framework for this time-to-quality measurement. The longitudinal scaling curves in figure 12.5 track cumulative relative performance gains across nine model workloads between 2018 and 2024 against the orange Moore's Law baseline. Every workload curve outpaces cumulative transistor scaling (which reaches 6.6× by mid-2024), led by Mask R-CNN climbing to 48.8× and ResNet reaching 34× as distributed scaling and compiler optimizations compound hardware improvements (Tschand et al. 2024). This trajectory documents improvements in the full hardware-software system beyond transistor scaling alone; the plotted results do not isolate the contribution of competitive pressure from other causes.

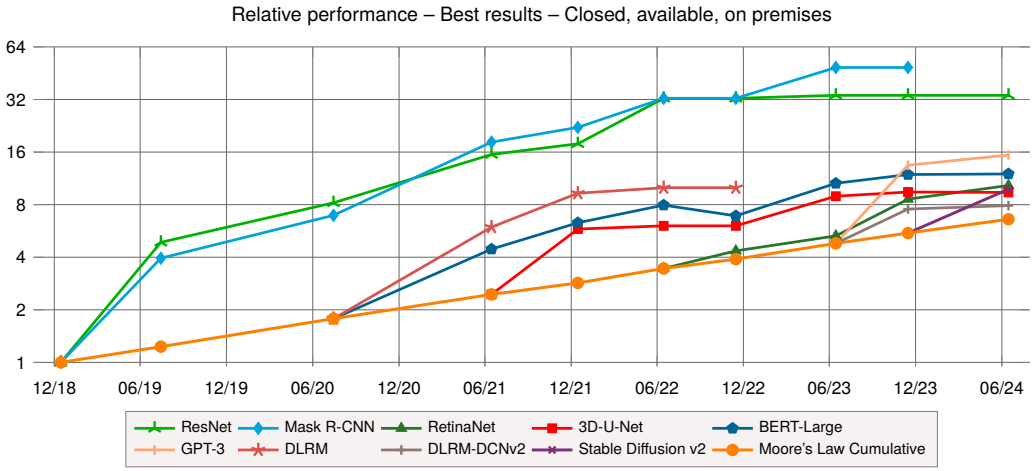


Figure 12.5: MLPerf Training Progress: Workload lines report cumulative relative performance across benchmark versions against the plotted cumulative Moore's Law baseline. Every workload's latest point exceeds the baseline at the same date, and Mask R-CNN reaches the largest plotted gain at 48.8×; the comparison captures combined hardware and software progress, not hardware scaling alone. Source: (Tschand et al. 2024).

Beyond charting that progress, training benchmarks uncover the inefficiencies that systematic evaluation makes visible: slow data loading, underutilized accelerators, excessive memory overhead, and communication bottlenecks that erode scaling efficiency. The theoretical hardware capabilities (for example, GPU TFLOP/s, TPU tensor throughput) only translate to actual training speedups when benchmarks verify them under realistic conditions.

Training benchmarks serve four interconnected functions. First, they enable hardware and software optimization by providing vendor-neutral comparisons across accelerator architectures and frameworks (TensorFlow, PyTorch) on standardized tasks, guiding hardware selection for data centers and cloud environments. Software optimizations including mixed-precision training[20] and memory-efficient data loading are similarly quantified. Second, they evaluate scalability: adding GPUs should reduce training time proportionally, but communication overhead, synchronization latency, and memory bottlenecks limit scaling efficiency in practice. Training benchmarks quantify

[20] **Mixed-precision training**: Uses lower precision for most arithmetic while preserving higher-precision accumulation where needed (Micikevicius et al. 2017). The benchmarking consequence: mixed-precision and full-precision runs are not directly comparable because reduced memory traffic and larger feasible batch sizes can change convergence dynamics. MLPerf addresses this by fixing the accuracy target, making time-to-accuracy the comparable quantity regardless of precision strategy.

these losses, revealing whether infrastructure investments deliver proportional returns. Third, they provide cost and energy accountability: with a single large-scale training run consuming energy on the scale of the GPT-3 estimate in section 12.7.2.4, benchmarks that track cost per training run and power consumption per unit of progress help organizations balance computational power with sustainability goals. Finally, standardized evaluation criteria, controlled randomness, submission rules, and audits make comparisons more reproducible and constrain implementation-specific shortcuts.

12.7.2 Training metrics

From a systems perspective, training benchmarks assess the rate of convergence toward a verified quality threshold rather than the raw processing speed of an execution loop. Metrics such as sample throughput and scaling efficiency become meaningful only when the workload reaches target convergence; without that operational constraint, raw throughput rewards numerical instability and divergence. MLPerf Training formalizes this requirement by coupling wall-clock time directly to task-specific validation targets across each standardized workload.

12.7.2.1 Time and throughput

One of the primary metrics for evaluating training efficiency is the time required to reach a predefined accuracy threshold. Training time (T_{train}) measures how long a model takes to converge to an acceptable performance level, reflecting the overall computational efficiency of the system. Let $\text{Accuracy}_{\text{val}}(t)$ be the model's validation accuracy at training time t, and let the target accuracy be the benchmark-specific threshold (for example, 75.9 percent top-1 accuracy for ResNet-50 on ImageNet in MLPerf). Equation 12.1 formally defines this metric, keeping the benchmark focused on how quickly a system achieves meaningful results:

$$T_{\text{train}} = \inf\{t \geq 0 : \text{Accuracy}_{\text{val}}(t) \geq \text{target accuracy}\} \tag{12.1}$$

A run that never reaches the target has no finite time-to-accuracy, preventing raw training speed from rewarding a system that fails the benchmark's quality criterion.

Throughput,[21] often expressed as the number of training samples processed per second, provides an additional measure of system performance. Let N_{samples} be the total number of training samples processed and T_{train} the training time from equation 12.1. Equation 12.2 makes the rate explicit by dividing processed samples by training time:

$$\text{Throughput} = \frac{N_{\text{samples}}}{T_{\text{train}}} \tag{12.2}$$

Throughput alone cannot certify system capability. A platform may sustain high sample rates using aggressive low-precision formats or enormous minibatches, yet fail to reach target accuracy because of gradient divergence or poor generalization. In MLPerf Training, for instance, a ResNet-50 benchmark on ImageNet is valid only if the model achieves the required 75.9 percent top-1 validation accuracy (Mattson et al. 2020; MLCommons 2024c). Evaluating throughput strictly as the rate of progress toward convergence ensures that reported gains reflect genuine learning productivity rather than wasted arithmetic cycles.

[21] **Throughput**: Originating in manufacturing to measure units produced per unit time, the term entered computing in the 1960s batch-processing era. The manufacturing origin carries a systems lesson: throughput and latency are inherently opposed, because batching increases throughput (more units per hour) at the cost of individual item wait time. In ML serving, this manifests as the batch-size trade-off: larger batches improve GPU utilization but increase per-request latency.

12.7.2.2 Scalability and parallelism

Scalability measures how effectively training performance improves as resources are added. Ideally, doubling GPU count should halve training time. In practice, communication overhead, memory bandwidth limits, and parallelization inefficiencies constrain scaling well below linear.

Napkin Math 12.3: Scaling efficiency calculation

Problem: A team trains ResNet-50 on ImageNet. Single-GPU training takes 24 hours. With 8 GPUs, training takes 4 hours. Is this good scaling? Where did the efficiency go?

Step 1: Define scaling efficiency. For strong scaling (fixed problem size, more processors), let $T(1)$ be the training time on a single GPU, and $T(N_{\text{GPU}})$ the training time on N_{GPU} GPUs. Equation 12.3 defines efficiency:

$$\text{Eff}_{\text{scaling}}(N_{\text{GPU}}) = \frac{T(1)}{N_{\text{GPU}} \times T(N_{\text{GPU}})} \times 100\% \tag{12.3}$$

Step 2: Calculate efficiency. $\text{Eff}_{\text{scaling}}(8) = \frac{24\,\text{hours}}{8 \times 4\,\text{hours}} \times 100\% = 24/32 = 75$ percent

With perfect scaling, 8 GPUs would complete in 3 hours (24 hours/8 GPUs). The actual 4 hours represents 75 percent efficiency.

Step 3: Investigate the efficiency loss. The two elapsed times establish the "missing" 25 percent, but do not attribute it. Table 12.6 lists illustrative contributions and profiling signals for gradient synchronization, memory copy, load imbalance, and batch-size effects. Strong-scaling comparisons hold total work fixed; time-to-quality comparisons must also account for any change in steps to convergence.

In multi-accelerator data-parallel training (such as 2, 4, or 8 GPUs connected over PCIe or NVLink within a host), each accelerator computes gradients on its own local slice of the minibatch, and the AllReduce collective of section 8.8.1.1 must sum them so every worker receives the identical global gradient average before the next step. Gradient tensors have one entry per trained parameter, but their byte payload equals weight storage only when their representations match. Moving them across interconnects adds communication whose cost depends on topology, the collective algorithm, and overlap. With fixed total work and ideal partitioning, compute time per step shrinks ($T_{\text{compute}} \propto 1/N_{\text{GPU}}$), while communication may shrink more slowly or grow. Exposed synchronization then occupies a larger fraction of the step; traces determine how much of the observed scaling loss it explains.

Table 12.6: Scaling Efficiency Loss Sources: Illustrative decomposition of the missing efficiency budget into measurable overhead categories and the corresponding measurement signal used to attribute each loss. Actual percentages depend on model, interconnect, storage, batch schedule, and framework implementation.

Source	Example Contribution	Measurement
Gradient synchronization	10–15%	AllReduce time per step
Memory copy (CPU↔GPU)	3–5%	Data transfer profiling
Load imbalance	2–5%	Per-GPU step time variance
Batch size effects	2–5%	Larger batches converge differently

Step 4: The systems insight. Scaling efficiency can decrease as N_{GPU} grows when exposed communication and other overheads fail to shrink with per-GPU compute. In this worked example, eight GPUs reach 75 percent efficiency; at larger scales, the same arithmetic makes clear why sophisticated communication and input-pipeline optimization become necessary.

MLPerf reports raw performance, while scaling efficiency can be calculated across matched submissions: a system achieving 2× throughput at 50 percent efficiency may be worse than 1.5× throughput at 90 percent efficiency, depending on cost constraints.

↩ The AllReduce collective that averages gradients across data-parallel workers is established in section 8.8.1.1.

When training large-scale models such as GPT-3, OpenAI employed a large cluster of NVIDIA V100 GPUs in a distributed training setup (Brown et al. 2020; Patterson et al. 2021). Google's TPU v4 systems demonstrate the same distributed-systems lesson at scale: adding computational resources provides more raw power, but performance and resiliency depend on communication efficiency, interconnect topology, and operational management (Jouppi et al. 2023; Zu et al. 2024). Benchmarks such as MLPerf quantify how well a system scales across multiple accelerators, providing insights into where inefficiencies arise during parallel execution.

Parallelism in training is categorized into data parallelism, model parallelism, and pipeline parallelism, each presenting distinct challenges. Data parallelism, the most commonly used strategy, involves splitting the training dataset across multiple accelerators (such as 2, 4, or 8 GPUs in a

single host). The efficiency of this approach depends on synchronization mechanisms and gradient communication overhead. In contrast, model parallelism partitions the neural network itself, requiring efficient coordination between processors. Benchmarks evaluate how well a system manages these parallelism strategies without degrading accuracy convergence. A key metric for evaluating parallelism is **scaling efficiency**, which quantifies how much of the added computational capacity translates into actual speedup. While strong scaling evaluates whether adding processors reduces training time on a fixed total workload, **weak scaling** scales the problem size proportionally with processor count to keep the per-device workload constant, measuring whether the system can train larger models or ingest larger datasets within a fixed time budget without communication bottlenecks dominating.

12.7.2.3 Resource utilization

Training efficiency depends on learning speed, cluster scalability, and how effectively the hardware resources are used. Compute utilization requires an explicit numerator. Model FLOPs utilization (MFU, definition 8.4) compares the useful model operation rate with the matching hardware peak; activity counters instead report whether units are busy. MFU contrasts with hardware FLOPs utilization (HFU), which counts all executed operations including activation recomputation; recomputation adds executed FLOPs without adding useful model FLOPs for the same step. Both utilization ratios also depend on elapsed time, so neither rate is fixed by the additional work alone. Comparing them separates useful model work from work expended to relieve memory capacity limits. Low utilization may indicate bottlenecks in data movement, memory access, or inefficient workload scheduling.

For instance, when training BERT on a TPU cluster, input-pipeline inefficiencies can limit overall throughput even when the accelerators have high raw compute power. If storage retrieval or preprocessing cannot keep up, the system fails to keep the TPUs fully busy. Profiling resource utilization identifies the bottleneck, and optimizations such as prefetching, caching, and more parallel input processing can improve sustained performance.

Memory bandwidth is another critical factor, as deep learning models require frequent access to large volumes of data during training. If memory bandwidth becomes a limiting factor, increasing compute power alone will not improve training speed. Benchmarks assess how well models use available memory, ensuring that data transfer rates between storage, main memory, and processing units do not become performance bottlenecks.

I/O performance also plays a direct role in training efficiency, particularly when working with large datasets that cannot fit entirely in memory. Benchmarks evaluate the efficiency of data loading pipelines, including preprocessing operations, caching mechanisms, and storage retrieval speeds. Systems that fail to optimize data loading can experience large slowdowns, regardless of computational power.

12.7.2.4 Energy efficiency and cost

Training large-scale machine learning models requires substantial computational resources, leading to considerable energy consumption and financial costs. Energy efficiency metrics quantify the power usage of training workloads, helping identify systems that optimize computational efficiency while minimizing energy waste. The increasing focus on sustainability has led to the inclusion of energy-based benchmarks, such as those in MLPerf Training, which measure power consumption per training run. The same power accounting governs inference, where precision is one potential energy lever; section 12.9 works through why INT8 quantization cuts per-inference energy by attacking both memory traffic and arithmetic cost.

Training GPT-3 was estimated to consume 1,287 MWh of electricity (Patterson et al. 2021). Fewer iterations reduce energy when their total energy cost is lower; a more expensive step can offset the reduced count. Measure energy over the complete run to the same quality target. Energy-aware benchmarks help guide the development of hardware and training strategies that optimize power efficiency while maintaining accuracy targets.

Cost considerations extend beyond electricity usage to include hardware expenses, cloud computing costs, and infrastructure maintenance. Training benchmarks provide insights into the cost-effectiveness of different hardware and software configurations by measuring training time in relation to resource expenditure. Organizations can use these benchmarks to balance performance and budget constraints when selecting training infrastructure.

12.7.2.5 Fault tolerance and robustness

Training workloads often run for extended periods, sometimes spanning days or weeks, making fault tolerance an essential consideration. A resilient system must handle unexpected failures (hardware malfunctions, network disruptions, and memory errors) without compromising accuracy convergence.

Production systems checkpoint for fault tolerance, periodically saving progress so a failure does not restart the run. On a single host, a benchmark records the save interval, the state written, and the pause each save exposes to training. Model weights and optimizer states consume storage bandwidth, and any pause extends wall-clock time to the quality target. Asynchronous saving can overlap I/O with computation, but its staging buffers and competing storage traffic remain part of the measured workload. Cluster recovery and parallel file systems extend this problem beyond the local machine; the local benchmarking lesson is to include the intended saving behavior in sustained performance measurements. MLPerf Training primarily measures time-to-quality under standardized workloads and does not benchmark failure recovery directly.

12.7.2.6 Reproducibility and standardization

Reproducibility studies have repeatedly shown that modest benchmark gains can disappear when random seeds, hardware, framework versions, or implementation details change (Henderson et al. 2018). This failure mode reflects a pervasive problem. The stochastic processes that section 12.3.1 identified as measurement noise interact, over a long training run, with hardware-specific behaviors (floating-point rounding, memory layout, compiler optimizations) to produce results that can vary meaningfully across environments.

A deeper layer of non-determinism comes from the parallel hardware itself. Operations such as parallel atomic additions, used during gradient accumulation for sparse embeddings in models like Graph Neural Networks, execute in non-deterministic order across threads when concurrent updates target the same memory location. The resulting floating-point summation order changes across runs, producing bit-for-bit different gradients even with identical inputs and seeds. Enforcing bit-exact reproducibility in these cases requires a deterministic reduction path, which can reduce training throughput—a direct trade-off between reproducibility and performance that benchmark protocols must explicitly address. Without explicit controls for all these sources of variability, benchmark numbers reflect a specific confluence of conditions rather than a system's genuine capability.

MLPerf Training addresses this through standardized data preprocessing, target-quality rules, and repeated accepted runs that characterize stochastic variation (Mattson et al. 2020). The point is not merely to produce a fast run, but to show that the reported performance reflects system capability rather than a favorable combination of stochastic factors.

For a training benchmark, reproducibility is therefore the full run envelope, not just the random seed. A credible report must preserve the model commit, dataset checksum, preprocessing pipeline, seed plan, framework and compiler versions, precision policy, batch schedule, hardware topology, thermal and power limits, and checkpoint behavior. It must also report the distribution of accepted runs rather than a single best run. Only then can the benchmark separate a real system improvement from a favorable interaction among software version, hardware state, and stochastic training path.

12.7.3 Training performance evaluation

A comprehensive training benchmark considers multiple dimensions of system behavior because each dimension identifies a different way hardware investment can fail to become convergence. Table 12.7 summarizes the core categories and associated metrics commonly used to benchmark system-level training performance, providing a framework for understanding how training systems behave under different workloads and configurations.

Table 12.7: Training Benchmark Dimensions: Key categories and metrics for evaluating machine learning training systems beyond simple speed, covering resource efficiency, reproducibility, and overall performance trade-offs across different training approaches and infrastructure configurations.

Category	Key Metrics	Example Benchmark Use
Training Time and Throughput	Time-to-accuracy (seconds, minutes, hours); Throughput (samples/sec)	Comparing training speed across different GPU architectures
Scalability and Parallelism	Scaling efficiency (percent of ideal speedup); Communication overhead (latency, bandwidth)	Analyzing distributed training performance for large models
Resource Utilization	Compute utilization (percent GPU/TPU usage); Memory bandwidth (GB/s); I/O efficiency (data loading speed)	Optimizing data pipelines to improve GPU utilization
Energy Efficiency and Cost	Energy consumption per run (MWh, kWh); Training throughput per watt (FLOP/s/W)	Evaluating energy-efficient training strategies
Fault Tolerance and Robustness	Checkpoint overhead (time per save); Recovery success rate (percent)	Assessing failure recovery in cloud-based training systems
Reproducibility and Standardization	Variance across runs (percent difference in accuracy, training time); Framework consistency (TensorFlow vs. PyTorch vs. JAX)	Ensuring consistency in benchmark results across hardware

The scaling notebook supplies a concrete interpretation of table 12.7. The measured completion times establish a speedup and a resource-efficiency cost, but they do not identify which overhead caused the difference. A procurement decision therefore uses the time-to-quality result first, then the utilization and communication measurements to decide whether additional hardware or a different execution configuration would address the limiting resource.

Across matched runs, GPU boost-clock[22] behavior and thermal state must follow the same protocol; otherwise a short fast run can be mistaken for a durable improvement. The repeated-run controls in section 12.3.1 determine whether the difference exceeds experimental uncertainty.

[22] **GPU boost clock**: Dynamic frequency scaling raises clocks above base when thermal and power headroom permit. Short benchmark runs can capture this burst performance, while sustained training may settle to a different operating point. The benchmark must document the measured thermal state and clock policy.

In the worked case, 8 GPUs reach 75 percent scaling efficiency. If the required quality target and completion deadline are met, the configuration clears the time constraint, but the remaining scaling overhead still matters for cost. If the deadline is missed, the efficiency ratio alone does not justify buying more accelerators. The next experiment must measure exposed synchronization, input stalls, and per-device work under the intended batch and checkpoint schedule before choosing an intervention. If convergence changes between configurations, compare complete time-to-quality runs rather than interpreting the difference as pure strong-scaling loss.

A training report can therefore support a configuration choice without claiming that every measured overhead has already been explained. Its acceptance gates concern learning; an efficiently trained model still needs separate evidence that it can serve requests under deployment constraints.

12.8 Inference Benchmarks

An autonomous vehicle perception pipeline must classify roadway obstacles within a strict 33-millisecond deadline to sustain a 30-frame-per-second control loop. If an unbenchmarked garbage collection pause, dynamic batching timeout, or kernel launch hiccup stretches the 99th-percentile (p99) latency to 120 milliseconds, the vehicle travels several meters without updated object tracks—rendering a high average throughput metric completely irrelevant to production safety. Training benchmarks measure how quickly a system learns; inference benchmarks measure how reliably it serves under real-world operational constraints. This shift changes nearly every aspect of evaluation: training tolerates variable iteration times as long as gradient descent converges, but inference enforces strict, non-negotiable service-level objectives where every delayed prediction directly compromises safety or user experience.

Inference is also where the optimization chapters converge, because the accelerated hardware runs the compressed models to deliver real-time predictions. Inference benchmarks reveal whether those theoretical speedups become actual latency reductions under realistic deployment conditions. Their central quantity is end-to-end request latency $L_{\text{lat,total}}$, the full time from a request's arrival to its response, including queueing, preprocessing, model execution, and postprocessing. It is distinct from the iron law's latency term L_{lat}, which counts only the fixed overhead (orchestration,

serialization, dispatch) that one execution pays regardless of how much data it moves or arithmetic it performs; queueing delay, which grows with load, belongs to $L_{\text{lat,total}}$ but not to L_{lat}.

Definition 12.4: ML inference benchmarks

ML inference benchmarks are machine learning system benchmarks that quantify a system's ability to meet latency constraints ($L_{\text{lat,total}}$) at specified throughput levels, measuring scenario-defined latency statistics, throughput (queries per second), and power efficiency across representative serving scenarios.

1. **Significance**: Inference benchmarks expose the workload- and system-specific gap between unconstrained throughput and throughput while meeting a service-level objective (SLO), such as a p99 latency target. Queuing delays can push tail latency above the target at high load, a gap that is invisible without a benchmark that enforces latency targets at each throughput level.
2. **Distinction**: Unlike training benchmarks, which measure time-to-accuracy over a fixed dataset, inference benchmarks measure per-query response time under realistic load patterns, capturing queuing effects, batching trade-offs, and cold-start overhead that determine real-world serving economics.
3. **Common pitfall**: A frequent misconception is that average latency is a sufficient benchmark. A system with low average latency but a long p99 tail can violate a percentile-based production SLO for the slowest 1 percent of requests; at high request rates, that small percentage becomes a large number of affected users. The benchmark must report the latency statistic named by the service objective rather than assuming the mean or p99 is universally decisive.

12.8.1 Inference benchmark motivation

Large-scale benchmark training commonly runs on dedicated data center hardware, whereas inference spans dramatically different deployment scenarios—from real-time applications like autonomous driving and conversational AI to mobile devices, IoT systems, and embedded processors. This diversity extends to hardware: while GPUs and TPUs dominate large-scale training, inference workloads often use specialized accelerators like NPUs, field-programmable gate arrays, and dedicated inference chips such as Google's Edge TPU.[23] Inference benchmarks evaluate how well hardware selection, model optimization, and data pipeline design work together across these deployment environments.

[23] **Edge TPU (tensor processing unit)**: Google's fixed-function edge AI accelerator. It illustrates a benchmarking constraint specific to fixed-function accelerators: its headline throughput applies only to quantized TensorFlow Lite models with supported operator types, so models requiring unsupported operators fall back to the host CPU or need graph rewrites before the accelerator result is meaningful.

Scaling inference workloads across cloud servers, edge platforms, mobile devices, and TinyML systems introduces additional complexity. The dumbbell plot in figure 12.6 isolates this operational disparity on a logarithmic power scale, plotting minimum (blue) and maximum (red) envelopes across four deployment tiers: TinyML systems (5.6 µW to 167 mW), edge hardware (3.9 W to 1.1 kW), data center inference nodes (267 W to 6.3 kW), and large-scale training clusters (5.5 kW to 498 kW). Spanning nearly eleven orders of magnitude from the 5.6 µW TinyML floor to the 498 kW training-cluster ceiling, this spectrum explains why no single metric serves all deployment contexts: a figure of merit optimized for multi-rack data center throughput becomes meaningless for battery-powered edge devices governed by milliwatt thermal envelopes. Inference benchmarks must therefore evaluate the trade-offs between latency, capital cost, and energy efficiency within each specific deployment scale.

These deployment differences create the practical motivation for inference benchmarks: they evaluate the bottlenecks that emerge when models transition from development to production serving. The motivating factors parallel those for training (hardware optimization, scalability, cost, fair comparison) but differ in specifics. Software optimization frameworks apply inference-specific techniques such as kernel fusion, precision calibration, and kernel tuning, whose impact on latency, throughput, and power efficiency must be measured under realistic conditions to confirm they deliver real improvements without degrading accuracy. Auto-tuning compilers add a hidden variable: the compiler itself can require hours of optimization per model-hardware pair, meaning benchmark

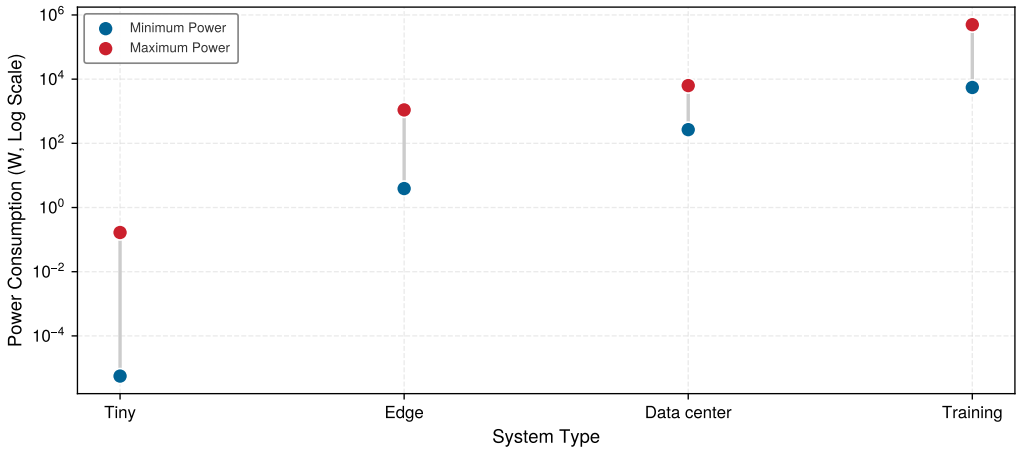


Figure 12.6: Power Consumption Differentials: Power usage spans over ten orders of magnitude across ML system types, from microwatts in TinyML devices through watts at the edge to kilowatts in data center inference and hundreds of kilowatts for training clusters. Ranges are representative and vary by hardware and workload.

results reflect the tuning budget as much as the hardware capability, and comparing results across submissions requires normalizing for compiler optimization time.

Scalability concerns also shift character. Training scales by adding GPUs to reduce time-to-accuracy on a fixed workload, whereas inference must scale dynamically in response to fluctuating user demand, handling traffic spikes without violating latency guarantees. Cold-start performance, the time required for a model to load and begin processing queries, makes initialization cost directly visible to inference users. Applications that load models on demand, such as serverless AI deployments, are particularly sensitive to this overhead.

The cost and energy profile of inference differs sharply from training. Training concentrates cost in discrete runs that may recur as data, objectives, or models change; inference cost accumulates continuously with production traffic. Running an inefficient model at scale can multiply cloud compute expenses, and on battery-powered devices, excessive computation directly affects usability. Benchmarks that measure cost per inference request and efficiency per watt help organizations optimize for both performance and sustainability across deployment platforms.

MLPerf Inference extends the standardized comparison principles established for training benchmarks to deployment scenarios, defining evaluation criteria for tasks such as image classification, object detection, and speech recognition across different hardware platforms. This ensures that inference performance comparisons remain meaningful and reproducible while accounting for deployment-specific constraints like latency requirements and energy efficiency (Reddi et al. 2019).

12.8.2 Inference metrics

For example, a voice assistant must respond quickly enough that users do not perceive lag, while a recommendation engine must score enough candidates to keep pace with user scrolling. These constraints (latency and throughput) define the performance envelope within which all serving optimizations must operate. Inference metrics formalize these real-world demands into measurable quantities, and they differ from training metrics in kind, not just degree, because the optimization target shifts from convergence rate during training to latency predictability during serving. Training optimizes for throughput and time-to-accuracy; inference optimizes for latency consistency, resource efficiency, and service-level predictability, spanning cloud data centers handling millions of requests to edge devices operating under strict power constraints.

12.8.2.1 Latency and tail latency

Latency measures the time for an inference system to process an input and produce a prediction. Average latency is useful, but it does not capture high-percentile delays that degrade reliability in high-demand scenarios.

To account for this, benchmarks often measure tail latency,[24] which summarizes the high-latency tail rather than the worst case. These values are commonly reported as the 95th percentile (p95) or 99th percentile (p99) latency, meaning that 95 percent or 99 percent of inferences are completed within a given time. Applications with hard deadlines require explicit deadline-miss or worst-case analysis beyond these percentiles.

These measurements form the basis for SLOs and SLAs, which formalize performance expectations.

[24] **Tail latency**: A high-percentile response time, such as p95 or p99, determines production SLO or SLA compliance only when that objective or agreement specifies the percentile. Dean and Barroso (2013) showed that in fan-out architectures (common in recommendation systems), even 1 percent slow responses compound: a request touching 100 backend shards has about a 63 percent chance that at least one shard hits its 1 percent tail. Benchmarks reporting only mean latency hide this failure mode.



Definition 12.5: SLOs and SLAs

SLOs and SLAs are performance commitment specifications for production ML serving systems: a service-level objective (SLO) is a target value or range for a service-level indicator, while an SLA is a contract that specifies service objectives and the consequences of failing to meet them.

1. **Significance**: A latency SLO bounds end-to-end request latency $L_{\text{lat,total}}$ by setting a target for a specified latency indicator, and the iron law's L_{lat} is only the fixed-overhead part of that budget. A representative production setup might set the internal SLO tighter than the external SLA, leaving operational headroom for transient spikes, maintenance windows, and cascading failures.
2. **Distinction**: An SLO has no contractual consequence by itself, while missing an SLA objective triggers the consequences specified in the agreement, which may include credits or penalties. Teams commonly set an internal SLO tighter than the external agreement to leave operational headroom, but this is a practice rather than a definitional requirement.
3. **Common pitfall**: A frequent misconception is that meeting average latency satisfies a tail-latency SLO. SLOs can use averages, percentiles, rates, or other indicators; when the objective is defined at p99 or p99.9, an excellent mean does not establish compliance.

The distinction matters in practice: engineering teams optimize toward SLOs while the business commits to SLAs. Choosing the wrong metric to optimize wastes engineering effort or violates customer guarantees.

A usable latency objective also fixes the request population and measurement window. Otherwise, two teams can report the same percentile threshold over different traffic mixes and reach incompatible conclusions about compliance.



Checkpoint 12.2: Metric selection

The metric shapes the optimization.

Apply three rules before finalizing metric selection:

- ☐ Throughput vs. latency: Name the objective to optimize and the quality, latency, and cost limits the benchmark must preserve.
- ☐ Tail latency: State what p99 exposes that mean latency hides.
- ☐ End-to-end: List the preprocessing, model execution, and postprocessing stages included in the reported latency.

Tail latency's connection to user experience at scale becomes critical in production systems serving millions of users. Even small p99 latency degradations create compounding effects across large request volumes: if 1 percent of requests experience 10× latency (for example, 1000 ms instead of 100 ms), this affects 10,000 requests per million, potentially leading to timeout errors, poor user experience, and customer churn. Search engines and recommendation systems demonstrate this sensitivity: Google's search-latency experiments found measurable reductions in daily searches per user after 100–400 ms server-side delays (Brutlag 2009), which is why interactive services often treat sub-100 ms response times as a practical design target.

Production SLOs should name the latency statistic relevant to the user experience. Interactive services often define percentile-based latency objectives because occasional slow responses have

disproportionate impact on user satisfaction. Large-scale systems may track even deeper tails, such as p99.9, when traffic spikes and infrastructure variation affect reliability.

The challenge of meeting these tail latency targets is that the source of the tail is often architectural, not algorithmic. A garbage-collected runtime, a shared kernel driver, or a priority-inversion bug in the serving stack can inject latency spikes that no model optimization will remove.

↪ Provisioning serving capacity so that p99 latency meets its SLO under peak load is developed in section 13.5.4.

War Story 12.1: Discord Read States rewrite (2019)

Context: Discord, a real-time chat platform supporting millions of concurrent users, originally implemented its Read States service in Go, tracking which channels and messages each user read (Howarth 2020).

Mechanism: Go's forced garbage collection passes scanned an LRU cache holding tens of millions of entries, causing periodic "stop-the-world" GC pauses every two minutes regardless of memory tuning.

Impact: Tail latency (p99) spiked dramatically every two minutes, degrading user experience across millions of active connections.

Fix: In 2019, Discord rewrote the Read States service in Rust, eliminating garbage collection pauses and dropping average response times to microseconds.

Systems lesson: Mean latency alone does not describe user experience when the service objective is defined at the tail. Language-runtime choices can set a floor on that tail: feature stores that serve real-time embeddings for recommendation models may use managed runtimes, and garbage-collection pauses can delay every downstream inference request waiting for retrieval. Model optimization cannot close a gap whose bottleneck lies in the retrieval path. The Discord incident documents the underlying mechanism outside ML and shows why the complete serving stack belongs inside a production latency benchmark.

12.8.2.2 End-to-end vs. component latency

A critical distinction in inference benchmarking is between component latency (time spent in model computation) and end-to-end latency (total time from request arrival to response delivery). Many benchmarks report only model inference time, obscuring the remaining overhead that determines actual user experience. The overhead is not marginal: serialization, network hops, and queue wait time can dominate total request time, making model-only optimizations yield diminishing returns.

Example 12.3: The JSON serialization trap

Scenario: Researchers evaluating the Clipper low-latency model serving system decomposed end-to-end request latency across network, CPU serialization, and model execution (Crankshaw et al. 2017).

Diagnosis: For lightweight models, CPU-side JSON deserialization and IPC data copying consume more wall-clock time than accelerator neural network execution.

Systems lesson: Reporting isolated model inference latency misses end-to-end serving bottlenecks. Production benchmarking must account for CPU data parsing, IPC serialization, and network transport overhead.

Table 12.8 gives an illustrative latency breakdown for an inference request. The model inference stage that vendors report as their "benchmark" number spans 5 to 100 ms, yet the queue wait time it sits behind ranges from 0 to over 1,000 ms: under load, the single component a benchmark measures is dwarfed by one it never sees, so the reported number can be a small slice of what the user actually experiences.

Table 12.8: Inference Latency Breakdown: Different pipeline components contribute to end-to-end latency, and model inference (the number vendors typically report) can be only one part of total request time. Queue wait time can dominate under load, making end-to-end measurement essential for realistic performance assessment. Ranges are illustrative rather than universal.

Component	Example Range	Notes
Network round-trip	10–100 ms	Varies by region
Request parsing	0.1–1 ms	JSON/protobuf
Input preprocessing	1–50 ms	Tokenization, image resize
Queue wait time	0–1000+ ms	Load-dependent
Model inference	5–100 ms	The "benchmark"
Output postprocessing	0.5–10 ms	Decoding, format
Response serialization	0.1–1 ms	JSON/protobuf

These component-level contributions explain why optimizing any single stage yields diminishing returns on end-to-end performance, an *optimization ceiling* formalized by Amdahl's Law (principle 8).

Napkin Math 12.4: Amdahl's Law: Optimization ceiling

Problem: A vision pipeline spends 8 ms on preprocessing (JPEG decode, resize, normalize) and 10 ms on inference. If inference alone is optimized by 5×, how much does end-to-end latency actually improve?

Math: Assume fixed work, sequential stages, and unchanged preprocessing, with no queueing feedback. Optimizing inference from 10 ms to 2 ms reduces total latency from 18 ms to only 10 ms, a 1.8× improvement rather than 5×. Amdahl's Law formalizes this ceiling: if inference, the part being accelerated, is fraction f_{parallel} of total latency, then even infinitely fast inference yields at most $1/(1-f_{\text{parallel}})$ speedup, because the preprocessing fraction $1-f_{\text{parallel}}$ remains. With preprocessing at 44.4 percent of total latency, the remaining fraction $1-f_{\text{parallel}}$ is 0.444, which yields a maximum achievable speedup $1/(1-f_{\text{parallel}})$ of 2.25×, regardless of model optimization.

Systems insight: Aggressive model optimization yields disappointing end-to-end results whenever the nonmodel fraction dominates. Any component speedup quoted in isolation shrinks once the untouched stages are measured alongside it, and the shortfall grows as those stages take a larger share of the pipeline. Comprehensive benchmarks must either include preprocessing in measurements or state explicitly that reported speedups apply only to the inference component.

Amdahl's ceiling highlights why rigorous benchmarking methodology matters. Comprehensive latency reporting requires specifying which components are included, measuring under realistic load conditions, and distinguishing component from end-to-end metrics. Before interpreting any benchmark result, verify that the measurement approach itself is sound.

Checkpoint 12.3: Benchmarking methodology

Bad benchmarks optimize the wrong things.

Three practices distinguish rigorous benchmarks from misleading ones:

- ☐ Representative data: Compare the benchmark input distribution against the production distribution.
- ☐ Warm-up: State whether the goal is steady-state or cold-start performance, and which initialization work is timed or discarded.
- ☐ Isolation: Document the machine, runtime, and competing workloads; explain which controls isolate the intervention and which conditions represent deployment.

Latency is only half of the serving envelope. Throughput and batch efficiency measure whether a serving system can use available hardware without violating latency constraints. Throughput counts how many inference requests a system processes per second, typically expressed as queries per second (QPS) or frames per second (FPS). Single-instance systems process each input independently on arrival; batch systems process multiple inputs in parallel, exploiting hardware parallelism for higher efficiency. For example, cloud-based services handling millions of queries per second benefit from batch inference, where large groups of inputs are processed together to maximize computational efficiency. In contrast, applications like robotics, interactive AI, and augmented reality require low-latency single-instance inference, where the system must respond immediately to each new input. Benchmarks must consider both single-instance and **batch throughput** to provide a comprehensive understanding of inference performance across different deployment scenarios.

Speed alone is insufficient because inference optimizations can change model behavior. Reducing numerical precision can accelerate computation while cutting memory and energy, as the simplified MobileNetV2 energy estimate worked out in the power section shows (section 12.9.1), but lower-precision calculations can introduce accuracy degradation. Inference benchmarks therefore evaluate how well models perform under different numerical settings, such as FP32, FP16, and INT8.[25] Many modern AI accelerators support mixed-precision inference, allowing systems to use numerical representations selected for workload requirements. Model compression techniques[26] further improve efficiency, but their impact on model accuracy varies depending on the task and dataset. Benchmarks help determine whether these optimizations are viable for deployment, ensuring that improvements in efficiency do not come at the cost of unacceptable accuracy loss.

Beyond speed and accuracy, table 12.9 lists the resource terms an inference benchmark must also report, and each exists because a system can pass the latency test and still fail to deploy. Memory footprint decides whether the model fits the device's budget at all, since inference rarely spans accelerators the way training can. Cold-start performance, the load time plus first-request latency, exposes initialization cost to the requesting user. It dominates in serverless AI environments[27] and on devices that evict models to free memory, so benchmarks must state whether initialization is included. Scalability records whether added accelerators or concurrent models raise throughput or collide on memory bandwidth, and energy per inference (joules per inference, or QPS/W in the cloud) sets battery life on mobile devices and operating cost in data centers.

12.8.3 Inference performance evaluation

The Raspberry Pi MobileNetV2 comparison in example 12.2 provides a candidate deployment change. Its INT8 latency and size improvements support further evaluation, but they do not establish that a serving contract has been met. The decision requires the request population, latency statistic, quality floor, and memory and energy budgets to be specified together.

Table 12.9 identifies the remaining evidence. A camera pipeline needs sustained completion times under its input cadence; an intermittent application also needs initialization and sleep-state costs. For either application, the candidate must retain the required model quality on representative inputs. The same model-level speedup can therefore support one deployment and fail another.

Table 12.9: Inference Performance Metrics: Latency, throughput, and resource usage metrics provide a quantitative basis for optimizing deployed machine learning systems and selecting appropriate hardware configurations, balancing speed, cost, and accuracy in production applications.

Category	Key Metrics	Example Benchmark Use
Latency and Tail Latency	Mean latency (ms/request); Tail latency (p95, p99, p99.9)	Evaluating interactive and soft real-time performance
Throughput and Efficiency	Queries per second (QPS); Frames per second (FPS); Batch throughput	Comparing large-scale cloud inference systems
Numerical Precision Impact	Accuracy degradation (FP32 vs. INT8); Speedup from reduced precision	Balancing accuracy vs. efficiency in optimized inference
Memory Footprint	Model size (MB/GB); RAM usage (MB); Memory bandwidth utilization	Assessing feasibility for edge and mobile deployments
Cold-Start and Load Time	Model load time (s); First inference latency (s)	Evaluating responsiveness in serverless AI

25 **INT8 (8-bit integer)**: INT8 sits at the aggressive end of the precision hierarchy (FP32 baseline, FP16 halves raw weight storage, INT8 quarters it), and each step demands increasing care to preserve accuracy. Post-training INT8 quantization normally uses a representative calibration dataset; accuracy depends on the model, operator coverage, quantization method, and how well calibration data represent deployment inputs. INT8 benchmarks must specify whether they use post-training quantization or quantization-aware training and document any calibration procedure.

26 **Model compression benchmarking**: Compression impact must be measured across four dimensions simultaneously: accuracy degradation, inference speedup, memory reduction, and energy savings. A technique achieving 10× size reduction with 1 percent accuracy loss may still be unsuitable if latency misses the deployment target; unstructured pruning, for example, leaves dense execution unchanged unless a compatible sparse representation and kernel exploit the zeros.

27 **Serverless AI**: Deployment paradigm where models scale from zero instances on demand. Cold-start time varies with the model, runtime, storage path, hardware allocation, and provider. Benchmarks for intermittent workloads must report whether initialization and model loading are included, because warm-instance latency alone can understate user-perceived latency.

Category	Key Metrics	Example Benchmark Use
Scalability	Efficiency under load; Multi-model serving performance	Measuring robustness for dynamic, high-demand systems
Power and Energy Efficiency	Power consumption (W); Performance per W (QPS/W)	Optimizing energy use for mobile and sustainable AI

The acceptance decision can reject a candidate on any binding constraint. A lower model-only latency does not establish an end-to-end gain if preprocessing dominates, and a smaller weight file does not establish that runtime buffers fit the device. The latency decomposition and Amdahl calculation in section 12.8.2.2 specify which additional measurements the MobileNetV2 comparison needs.

Because physical constraints differ across environments, no single metric ordering governs all systems. Benchmarking requires establishing a priority hierarchy derived directly from the deployment contract. Table 12.10 illustrates how metric priorities shift across five operational contexts, mapping hardware and operational constraints to optimization targets.

Table 12.10: Illustrative Performance Metric Priorities by Deployment Context: A sample priority ordering shows how operational constraints can change benchmark selection and result interpretation; actual thresholds must come from the deployment specification.

Deployment Context	Primary Priority	Secondary Priority	Tertiary Priority	Key Design Constraint
Real-Time Applications	Latency	Reliability	Memory Footprint	User experience demands immediate response
Cloud-Scale Services	Throughput (QPS)	Cost Efficiency	Average Latency	Business viability requires massive scale
Edge/Mobile Devices	Power Consumption	Memory Footprint	Latency	Battery life and resource limits dominate
Training Workloads	Training Time	GPU Utilization	Memory Efficiency	Research velocity enables faster experimentation
Scientific/Medical	Accuracy	Reliability	Explainability	Correctness cannot be compromised for performance

A metric that dictates architectural success in one deployment context may be irrelevant in another. Latency ranks first for autonomous robotics, where sensor-processing pipelines must complete within strict execution deadlines to maintain vehicle stability, whereas cloud batch services accept higher queuing latency in exchange for lower operational cost per query. Similarly, an edge-deployed model that boosts throughput while drawing additional power degrades overall system utility when battery capacity or thermal throttling is the binding constraint. In contrast, a medical diagnostic pipeline may rationally trade execution speed for higher verified accuracy. A 2× throughput gain represents substantial economic value in a hyperscale cloud cluster, but it provides minimal benefit on a battery-powered device where a 20 percent power reduction extends operational lifetime.

For the camera example, use the required deadline or percentile as the timing gate, the runtime peak allocation as the capacity gate, and the permitted quality loss as the model gate. If any gate lacks a measurement, the decision remains unresolved even when the reported inference speedup is large. A protocol should identify the missing evidence rather than treating the favorable metric as approval.

On-demand deployments add **cold-start latency**[28] to this decision. The benchmark must include provisioning, model transfer, runtime initialization, and the first response when those costs lie on the user request path; a warm-instance result answers a different question.

Beyond initialization delays, inference benchmarks become misleading when operational metrics are reported without functional validation.

Numerical precision optimization introduces a significant comparability trap across accelerator platforms. Accelerators provide substantially higher INT8 operation throughput[29] than baseline FP32 floating-point rates, but these peak figures are meaningful only when the benchmark verifies accuracy retention under quantization noise, confirms hardware support across all model operators, and enforces equivalent operation definitions across architectures.

[28] **Cold-start latency**: The initialization time from idle state includes storage reads, host-to-accelerator transfer, and framework setup. For a 7B-parameter model in FP16 (~14 GB) already resident in host memory, PCIe 4.0 transfer at 25 GB/s effective bandwidth alone takes ~560 ms; storage and initialization add to it. This lower bound makes cold-start mitigation (model caching, speculative loading) a systems design requirement.

[29] **TOPS (tera operations per second)**: A measure of raw computational throughput (trillions of operations/second). The H100 delivers 1979 TOPS INT8 vs. the Apple M2 Neural Engine at 15.8 TOPS and Edge TPU at 4 TOPS, but these numbers conflate different operation types—multiply-accumulate vs. accumulate vs. activation. TOPS comparisons across vendors are meaningful only when the operation definition, precision, and sparsity assumptions are identical, conditions rarely met in vendor specifications.

Once a candidate clears quality and capacity gates, varying offered load tests how far its timing result transfers. A runtime can sustain a favorable latency with one request in flight yet miss the target when queues and shared memory fill. The relevant outcome is the supported load under the declared latency bound, measured with the same input mix and power policy, rather than an isolated peak throughput.

Capturing these operational realities requires rigorous statistical protocols rather than isolated execution loops. Valid inference benchmarking demands sustained run durations, realistic query arrival distributions, and strict percentile latency accounting. Standardized benchmarking consortia formalize these requirements as the execution scenarios previewed in section 12.3.3, each of which fixes the arrival pattern and the latency statistic a valid result must respect.

12.8.4 MLPerf inference benchmarks

MLPerf Inference turns that balance among latency, throughput, memory, energy, and accuracy into scored scenarios, and it matters because deployment context changes what a result means. The benchmark, developed by MLCommons,[30] provides a standardized framework for evaluating machine learning inference performance across a range of deployment environments. MLPerf began with training benchmarks in 2018; MLPerf Inference was added later to standardize deployment-time evaluation across scenarios. As machine learning systems expanded into diverse applications, it became clear that a one-size-fits-all inference benchmark was insufficient. The resulting family of MLPerf inference benchmarks maps each benchmark to a deployment setting, so a score can be interpreted against the latency, throughput, memory, and power constraints the system will face.

12.8.4.1 MLPerf Inference

MLPerf Inference (Reddi et al. 2019) serves as the baseline inference benchmark, defining standardized scenarios for deployment-time evaluation across data-center and edge settings. Submissions are split into two tracks: the **closed division** constrains model structure and reference equivalence while permitting specified precision and quantization changes that meet quality and reproducibility rules, whereas the **open division** allows submitters to alter model architecture, quantization algorithms, or retrain models to demonstrate creative algorithmic co-design. It assesses performance across deep learning workloads such as image classification, object detection, natural language processing, and recommendation systems. This version of MLPerf is a widely used reference point for comparing AI accelerators, GPUs, TPUs, and CPUs when the submission rules and workload scenario match the intended deployment environment.

Major technology companies regularly reference MLPerf results for hardware procurement decisions. When evaluating hardware for recommendation systems infrastructure, MLPerf benchmark scores on DLRM[31] workloads can inform choices between different accelerator generations. Across generations, benchmark results often show substantial throughput improvements, although the magnitude depends on workload, software stack, and system configuration. This illustrates how standardized benchmarks can translate into consequential infrastructure decisions.

These standardized evaluations provide invaluable comparisons, but the cost of comprehensive benchmarking limits who can participate and how thoroughly systems are evaluated.

> Systems Perspective 12.7: The cost of comprehensive benchmarking
>
> Submitting to MLPerf can require dedicated engineering effort, hardware access, tuning, validation, and careful documentation across the relevant configurations. The cost depends heavily on workload and scope, but it is high enough that submissions are dominated by major technology companies and hardware vendors, while smaller organizations rely on published results rather than conducting their own comprehensive evaluations. That barrier motivates the need for more lightweight, internal benchmarking practices that organizations can use to make informed decisions without the expense of full-scale standardized benchmarking.

The rest of the MLPerf inference family narrows that baseline by deployment context. MLPerf Mobile (MLCommons 2024a) evaluates whether a model can remain responsive within smartphone power and memory limits (Janapa Reddi et al. 2022), measuring image classification, object detection,

30 **MLCommons**: Launched in 2020 as a nonprofit consortium evolving from the 2018 MLPerf effort, MLCommons includes members across industry and academia (MLCommons 2026a). Requiring published system specifications improves result comparability, but it does not prevent submitters from selecting which systems or workloads to enter. Published results reveal large performance differences between vendors on identical workloads, making MLCommons the closest the field has to SPEC-style apples-to-apples hardware comparison.

31 **DLRM (deep learning recommendation model)**: Facebook's 2019 recommendation architecture combines embedding tables for categorical features with multilayer perceptrons for continuous features (Naumov et al. 2019). DLRM stresses benchmarks differently than vision or language models: its embedding tables can be large enough that memory capacity and bandwidth dominate compute throughput. That makes DLRM a useful memory-bound recommendation workload in MLPerf-style inference evaluation, revealing hardware limitations invisible to compute-bound benchmarks (Reddi et al. 2019).

image segmentation, and language-processing workloads. MLPerf Client (MLCommons 2026b) addresses the local-computing decision: whether consumer devices can run AI workloads directly rather than relying on cloud inference. Its current emphasis on local generative-AI and large language model (LLM) workloads makes CPUs, discrete GPUs, and integrated NPUs part of the benchmarked system rather than incidental host hardware. MLPerf Tiny (C. Banbury et al. 2021) tests the extreme constraint case: embedded and ultra-low-power AI systems, such as IoT devices, wearables, and microcontrollers. These variants preserve the same benchmark discipline while changing the binding resource from data center throughput to client responsiveness, mobile power, or microcontroller memory.

12.8.4.2 MLPerf execution scenarios

The same hardware can report dramatically different benchmark numbers depending on *how* requests arrive—a fact that explains why vendor claims often fail to predict production performance. Classic MLPerf Inference defines four execution scenarios that characterize distinct traffic patterns, each requiring different optimization strategies (Reddi et al. 2019). Current client and generative-AI benchmark variants also include interactive measurements for latency-sensitive LLM workloads, where metrics such as time-to-first-token and time-per-output-token become central (MLCommons 2026b).

SingleStream SingleStream processes one request at a time, measuring latency for sequential inference. This scenario models mobile and embedded applications where a single user interacts with the device: a smartphone camera app classifying images, a voice assistant processing speech, or a wearable detecting gestures. The key metric is a tail percentile of per-query latency (the 90th percentile in MLPerf's rules), and inter-request batching is unavailable because the next query arrives after the previous one completes. Optimization focuses on preprocessing efficiency and power consumption rather than throughput.

MultiStream MultiStream processes multiple synchronized input streams simultaneously, modeling scenarios like autonomous vehicles with multiple cameras that must be processed together for spatial fusion. The application motivation is synchronized processing, but the benchmark contract is version-specific: current MultiStream rules issue a group of samples per query, wait for its completion before the next query, and report a specified latency percentile. Earlier versions used fixed arrival intervals. Grouped-query latency helps assess multi-input execution; it does not establish a hard-deadline guarantee for a vehicle planning loop.

Server Server generates requests following a Poisson distribution, simulating cloud API traffic where requests arrive independently and unpredictably. This scenario models web services handling millions of queries from different users. Unlike SingleStream's guaranteed sequential arrival, Server traffic creates queuing dynamics where multiple requests compete for resources. The key metrics are throughput (queries per second) and the workload's tail-latency bound, typically set at the 99th percentile, and dynamic batching can improve efficiency by grouping requests that arrive within a time window. Optimization balances throughput against latency SLOs.

Offline Offline provides all inputs upfront, measuring maximum throughput when latency constraints are removed. This scenario models batch processing pipelines: overnight data processing, scientific computing, or precomputing recommendations. With no per-query latency requirement, systems can tune batch size for sustained throughput within memory and execution limits. The key metric is pure throughput (samples per second), and optimization focuses entirely on hardware efficiency.

Table 12.11 maps the classic execution scenarios, plus the newer Interactive LLM-oriented case, to their deployment contexts and optimization strategies.

Table 12.11: MLPerf Execution Scenarios: Classic MLPerf Inference scenarios map to distinct deployment contexts, each requiring different optimization strategies, and newer Interactive scenarios extend the framework to latency-sensitive LLM workloads. SingleStream and MultiStream prioritize latency, Server balances throughput and latency, Offline maximizes throughput, and Interactive emphasizes token-level responsiveness. Matching the scenario to deployment context determines which benchmark results are relevant.

Scenario	Context	Strategy	Focus
SingleStream	Mobile apps, embedded devices	No batching (batch = 1)	Preprocessing, power efficiency
MultiStream	Autonomous driving, video analytics	Synchronized sensor fusion	Grouped-query latency percentile
Server	Cloud APIs, web services	Dynamic batching with timeout	Throughput-latency trade-off tuning
Offline	Batch processing, data pipelines	Maximum batch size	Throughput, hardware utilization
Interactive	Chat, agents, local generative AI	Token streaming, KV-cache management	Time-to-first-token, time-per-output-token

Each scenario acts as a workload contract by fixing which form of latency or throughput a valid result must preserve.

Lighthouse 12.2: MobileNetV2 on EdgeTPU

Extending lighthouse 12.1, the hardware-acceleration claim is evaluated using an illustrative SingleStream-style edge-inference scenario (Reddi et al. 2019; C. Banbury et al. 2021).

Hardware acceleration claim: In this illustrative edge-accelerator scenario, assume INT8 MobileNetV2 inference takes ~2 ms on the accelerator, approximately 7.5× faster than the same board's low-power host CPU (~15 ms). The device differs from the Raspberry Pi 4 of example 12.2, so the two scenarios' latencies are not comparable. Actual results depend on operator coverage, clock frequency, thermal state, and implementation.

Table 12.12 reports the illustrative SingleStream-style scenario assumptions across distinct measurement scopes.

Table 12.12: EdgeTPU vs. Host CPU MobileNetV2 Scenario: Illustrative SingleStream-style latency, power, and energy values show how measurement boundaries alter comparative ratios. While end-to-end latency captures host preprocessing and link transfer overhead, the power and energy rows reflect active component execution alone rather than whole-system draw; these are illustrative values rather than controlled laboratory measurements.

Metric	Measurement Scope	Host CPU	EdgeTPU	Comparative Ratio
Inference latency	Isolated model compute	~15 ms	~2 ms	7.5× faster (isolated headline)
End-to-end latency	Full request path (prep + link + exec)	~18 ms	~6 ms	~3× faster (system-level)
Active power	Component during inference execution	~120 mW	~500 mW	~4.2× higher (component active)
Active energy per inference	Component active energy ($P \times t_{\text{inference}}$)	~1.8 mJ	~1 mJ	~1.8× more efficient (component active)

What this reveals: Under these assumptions, the 7.5× inference speedup narrows to ~3× end to end because preprocessing runs on the host CPU in both cases. The accelerator path incurs additional latency copying each input across the host-accelerator link and retrieving the result, a transfer overhead the CPU-only path never pays.

Furthermore, the 1.8× energy advantage applies specifically to the active inference interval ($P \times t_{\text{inference}}$), measuring component-level efficiency rather than total system consumption. This metric excludes host CPU preprocessing energy, interconnect transmission power, and baseline idle draw during non-inference phases. Because end-to-end latency improves by only 3×, host baseline power consumed during these non-accelerated phases narrows the whole-system battery efficiency gain.

Workload arrival patterns determine whether component-level energy efficiency translates to extended battery life. For intermittent duty cycles, the lower active inference energy favors the accelerator, but total battery drain depends on sleep power, wake-up latency and energy overhead, host transfer costs, and any standby leakage the accelerator draws while awaiting work. For continuous streaming workloads such as video analytics, the accelerator's lower active energy per inference dominates cumulative energy consumption.

The SingleStream result illustrates why benchmarking requires matching the MLPerf scenario to the deployment context: SingleStream emphasizes latency, while Offline benchmarks would optimize for bulk throughput rather than latency bounds.

The scenarios explain why the same hardware can report dramatically different benchmark numbers. In an illustrative comparison, an accelerator with high Offline throughput can sustain much lower Server-mode throughput once p99 latency constraints and queuing overhead are enforced, because Server mode cannot always use maximum batch sizes. When evaluating hardware for a specific application, selecting the appropriate scenario makes the result relevant; production-representative validation tests whether it transfers. The MobileNetV2 result therefore makes scenario selection a deployment requirement rather than a reporting choice.

12.8.5 Mobile and edge benchmarks

Unlike data centers with kilowatt-scale power delivery and active cooling infrastructure, mobile and edge deployments operate within strict milliwatt- to watt-scale power envelopes and passive thermal dissipation limits. Benchmarking these systems requires navigating an interdependent triangle of power consumption, inference latency, and model accuracy, where optimizing any two typically degrades the third. Table 12.13 contrasts these operational constraints against cloud environments.

Table 12.13: Edge vs. Cloud Deployment Constraints: Typical cloud deployments treat power as an operating cost and latency as a service target, while edge deployments often face tighter energy and response-time limits that constrain achievable accuracy.

Constraint	Cloud Impact	Edge Impact
Power	Operational cost	Device energy budget
Latency	Service-level target	Local response deadline
Accuracy	Task-quality target	Balanced with power/latency

Consider a smartphone camera executing real-time object detection at 30 frames per second within a 3 W to 5 W thermal envelope. In this setting, an efficient architecture such as MobileNet serves as a realistic benchmark target where a larger ResNet model would violate thermal limits, despite the larger model's superior accuracy in cloud evaluations. An edge benchmark must therefore evaluate accuracy jointly with sustained power and thermal dissipation. The peak-versus-sustained gap established in section 12.1 can be especially acute at the edge: a passively cooled device has less capacity to shed continuous inference heat than an actively cooled server. Both environments must remain within their cooling limits. As junction temperatures rise, the dynamic thermal management (DTM) controller scales down operating voltage and clock frequency, causing sustained throughput to drop sharply below burst-mode peaks. That thermal mechanism makes the cooling configuration part of the benchmark contract.

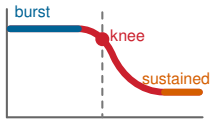
Example 12.4: Benchmarking the edge

Scenario: An edge engineering team benchmarks a smart doorbell chip with a vendor-advertised "1 W AI" power specification.

Diagnosis: Early burst-mode testing matches vendor claims, but continuous inference loop execution builds up junction heat, triggering thermal throttling and dropping steady-state throughput.

Systems lesson: Short burst benchmarks obscure long-term thermal throttling. Benchmarking edge ML workloads requires continuous, sustained execution testing to reflect true operational hardware limits.

Throttling is the most visible way a short measurement flatters edge hardware. Idle power, duty cycle, and the data movement around the accelerator can distort a headline number too, so each needs checking before a vendor claim informs a design decision.



Burst benchmark FPS collapses once thermal throttling sets in.

Systems Perspective 12.8: Edge benchmark reality check

When evaluating edge hardware claims, four factors determine whether vendor numbers translate to real-world performance:

1. **Peak vs. sustained**: A vendor may advertise 45 TOPS peak throughput while a sustained thermal run delivers closer to 20 TOPS. In this scenario, use a sustained run longer than 30 s and verify thermal stabilization; also test the deployment duty cycle.
2. **Power at idle vs. active**: In this scenario, a device consuming 50 mW idle and 2 W active could report active draw for marketing, but if the application runs inference 1 percent of the time, effective power draw is ~69.5 mW, not 2 W.
3. **Thermal envelope**: Edge devices often operate inside a narrow TDP envelope. Exceeding it triggers throttling, so benchmark reports omitting thermal conditions are incomplete.
4. **End-to-end vs. accelerator-only**: NPU benchmarks often exclude data transfer overhead. Moving image data from camera to NPU and back can exceed inference time for small models.

Because passive thermal dissipation caps sustained power draw, mobile SoC architects cannot rely on high-frequency monolithic cores. Instead, they divide execution across specialized compute blocks tailored to distinct arithmetic patterns. This hardware specialization shifts edge benchmarking from evaluating an isolated core to characterizing multi-processor coordination across the entire SoC.

12.8.5.1 Heterogeneous processor coordination

A modern mobile SoC packages CPUs, GPUs, digital signal processors (DSPs), and neural processing units (NPUs) onto a single die sharing system memory and a common thermal budget. Each processing unit targets a distinct compute pattern: CPUs handle irregular control flow, dynamic batching, and sequential post-processing; GPUs execute parallel floating-point operations and unquantized fallback layers; DSPs process low-power streaming vector mathematics for always-on sensing; and NPUs execute dense, quantized matrix multiplications (typically INT8 or INT4) with specialized systolic arrays or multiply-accumulate engines.

Benchmarking must evaluate pipeline orchestration rather than isolated kernel throughput. In an end-to-end voice assistant, for example, an ultra-low-power DSP continuously monitors microphone audio for a wake-word; upon detection, the system wakes the NPU to execute acoustic feature extraction and speech recognition, before passing token sequences to the CPU for language understanding and dialogue logic. Single-processor benchmarks evaluate only the isolated compute time of each stage, missing three dominant sources of system overhead: the latency of waking accelerators from deep sleep states, the bus contention when multiple engines access shared DRAM simultaneously, and the memory bandwidth consumed by transposing tensor layouts (such as converting NCHW CPU layouts into tiled NHWC formats required by the NPU). Benchmarks that report only compute time obscure these data movement and scheduling penalties.

12.8.5.2 Battery and thermal benchmarking

Battery drain depends on total energy consumed over time rather than peak instantaneous power. Computational photography bursts consume multiple watts for hundreds of milliseconds during image capture, whereas background sensor processing must remain within a milliwatt-scale budget

to deliver multi-day battery life. Characterizing battery impact requires measuring the total energy consumption $E_{\text{total}} = \int P_{\text{device}}(t)\,dt$ across representative operational profiles, where $P_{\text{device}}(t)$ is the instantaneous power the device draws at time t.

The dominant operational variable is the workload duty cycle α_{duty}, defined as the fraction of time the accelerator actively runs inference. For an active power draw P_{active} and standby power draw P_{idle}, the average power is:

$$P_{\text{avg}} = \alpha_{\text{duty}} P_{\text{active}} + (1 - \alpha_{\text{duty}}) P_{\text{idle}}$$

For intermittent workloads like a smart doorbell camera ($\alpha_{\text{duty}} \ll 0.01$), standby power P_{idle} and memory retention power dictate battery lifetime, making active inference efficiency secondary. For continuous pipelines like video analytics ($\alpha_{\text{duty}} \approx 1.0$), active inference energy $E_{\text{inf}} = P_{\text{active}} \cdot L_{\text{lat,infer}}$ dominates. Furthermore, elevated operating temperatures increase semiconductor leakage current, causing static power dissipation to climb as the device heats up. Benchmarking battery impact therefore requires measuring both active and standby power across the operational duty cycle while monitoring junction temperature over extended multi-minute runs.

12.8.5.3 Edge-cloud coordination

When edge devices offload computation to remote servers over 5G or Wi-Fi networks, benchmarking must evaluate the combined edge-cloud pipeline rather than local execution in isolation. While cellular standards such as URLLC[32] define aggressive radio-interface latency targets, real-world cellular and Wi-Fi channels introduce packet jitter, tail latency, and transient disconnections. Workload splitting benchmarks must therefore evaluate four system behaviors under emulated network impairment: the decision boundary for local versus remote execution, round-trip transmission overhead for intermediate activations versus raw sensor inputs, fallback mechanisms when network connectivity drops (such as degrading to a compact on-device model), and privacy policies restricting raw data egress. For safety-critical perception the decision boundary is fixed, because an automotive stack on a tens-of-milliseconds control loop cannot tolerate wide-area tail latency, so it runs locally and reserves the uplink for non-time-critical telemetry.

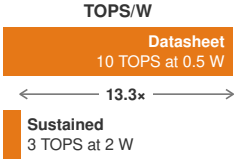
Training benchmarks measure learning speed; inference benchmarks measure serving speed. Measuring the energy cost of that speed requires power measurements matched to the same workload window and system boundary. A system that sets throughput records while consuming kilowatts of power may be economically unsustainable or physically impossible to deploy at the edge. Completing the evaluation picture requires power measurement: measuring the energy cost of performance.

[32] **URLLC (ultra-reliable low-latency communication)**: ITU-R defines a 1 ms radio-interface user-plane latency target and 99.999 percent successful delivery within 1 ms for a 32-byte packet under specified test conditions. These radio-interface targets are not a sub-1-ms end-to-end application guarantee. Edge-inference benchmarks should report radio or network latency and compute latency separately and together, alongside model quality.

12.9 Power Measurement Techniques

A chip vendor advertises "10 TOPS at 0.5 W," but under sustained inference load, thermal throttling drops actual throughput to 3 TOPS at 2 W. Without standardized power measurement, this 13.3× efficiency gap between the datasheet and reality goes undetected until deployment.

This efficiency dimension is critical because section 11.1 defines acceleration by its compute-density and performance-per-watt gains, making TOPS/W a primary design objective alongside raw TOPS. Power benchmarks validate whether efficiency-optimized accelerators deliver their promised energy savings. TOPS/W is particularly susceptible to gaming precisely because it is a ratio of two separately quotable peaks: a vendor can read the numerator (operations) at the batch size and precision that maximize throughput and the denominator (watts) at a near-idle operating point, so the advertised efficiency describes a state the chip never occupies under real load. Power benchmarks close that loophole by fixing the workload and the measurement window, forcing the numerator and denominator to be read at the same operating point.



Datasheet TOPS/W overstates sustained efficiency by 13.3×.

Measuring power differs fundamentally from measuring execution time. Latency is a single interval, but instantaneous power (supply voltage times current) fluctuates continuously across microsecond execution phases and drifts over minutes as silicon heats up, increasing temperature-dependent leakage current. Furthermore, ML deployments span an extreme physical range. As cataloged in table 12.14, representative systems span nearly eight orders of magnitude, from a 150 µW wake-word coprocessor to a 10 kW data center rack (Henderson et al. 2020). The span is narrower than the nearly eleven orders of figure 12.6 because the table lists typical devices rather than the

extreme floor and ceiling of each deployment tier; each order-of-magnitude count is meaningful only with its endpoints.

Table 12.14: Power Consumption Spectrum: The representative deployment points listed here span nearly eight orders of magnitude in power demands, from microwatt-scale TinyML devices (150 µW) to kilowatt-scale ML server racks (10 kW); the dumbbell chart in figure 12.6 extends this envelope down to 5.6 µW at the TinyML floor and up to roughly 498 kW at the training-cluster ceiling, covering nearly eleven orders of magnitude. This enormous range explains why no single measurement technique or efficiency metric applies universally: benchmarking a 150 µW neural processor requires fundamentally different instrumentation than measuring a 10 kW server rack.

Category	Device Type	Power Consumption
Tiny	Neural Decision Processor (NDP)	150 µW
Tiny	M7 Microcontroller	25 mW
Mobile	Raspberry Pi 4	3.5 W
Mobile	Smartphone	4 W
Edge	Smart Camera	10–15 W
Edge	Edge Server	65–95 W
Cloud	ML Server Node	300–500 W
Cloud	ML Server Rack	4–10 kW

Because power spans from microwatts to kilowatts, no single instrument or measurement protocol applies everywhere. Benchmarking an always-on neural sensor requires measuring microampere currents at millivolt rails, whereas benchmarking a server rack requires measuring high-voltage three-phase alternating current at the power distribution unit. This divergence forces benchmarking standards to establish precise physical boundaries for what hardware is included in the energy ledger.

12.9.1 Power measurement boundaries

Where an engineer draws the system boundary dictates what counts as efficient. If a benchmark measures only the accelerator core, it ignores the host CPU driving the PCIe bus, the off-chip DRAM holding weights, and the cooling fans dissipating heat. Figure 12.7 partitions these measurement perimeters across three deployment tiers (Tiny, Inference, and Training), where solid olive enclosures demarcate measured components and red dashed boxes isolate excluded infrastructure such as remote storage and data-center chillers (Tschand et al. 2024).

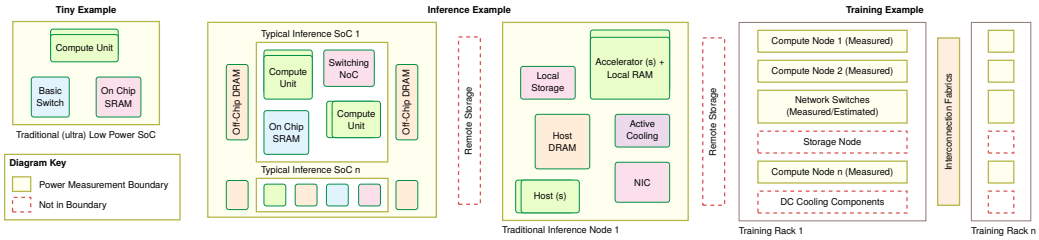


Figure 12.7: Power Measurement Boundaries: MLPerf defines system boundaries for power measurement, ranging from single-chip devices to full data center nodes, to enable fair comparisons of energy efficiency across diverse hardware platforms. These boundaries delineate which components' power consumption is included in reported metrics, impacting the interpretation of performance results. Source: (Tschand et al. 2024).

The three architectural tiers in figure 12.7—Tiny Example, Inference Example, and Training Example—reflect distinct physical instrumentation points. In TinyML deployments, the entire low-power SoC—compute cores, internal SRAM, and power management units—falls inside the boundary, measured directly at the DC power rail. Dedicated inference nodes expand this perimeter to include host CPUs, accelerator cards, PCIe buses, local storage, and on-chassis active cooling fans, while excluding remote storage networks. Multi-rack training deployments measure compute nodes and top-of-rack interconnect switches, but exclude centralized data center chillers, facility-level power distribution units (PDUs), and disaggregated storage clusters.

The measurement boundary determines which components are counted; workload composition determines where their energy is spent. On mobile devices running TensorFlow Mobile, data

movement accounts for 57.3 percent of total inference energy (Boroumand et al. 2018), exposing why operation counts alone fail to predict power consumption; under the energy-movement principle (7), total energy depends on how many events of each kind occur and what each one costs, not on arithmetic alone. The worked example in Notebook 12.5 decomposes MobileNetV2 inference energy to demonstrate how moving from FP32 to INT8 attacks both the memory-movement bottleneck and arithmetic switching costs.

Napkin Math 12.5: Why INT8 saves energy

Problem: Under the stated 45 nm component-energy model, how much does replacing FP32 with INT8 reduce MobileNetV2 weight-read and arithmetic energy?

Moving data costs far more energy than computing on it (principle 7, grounded in Horowitz's energy estimates (Horowitz 2014)). Understanding *why* quantization reduces energy consumption requires decomposing energy into its physical sources. Two dominant factors determine inference energy: compute operations and memory access.

Narrower datatypes generally require less switching and storage energy per operation, so table 12.15 reveals an 18× gap between FP32 and INT8 multiply cost:

Table 12.15: Per-Operation Energy by Precision: Energy cost of a single multiply at FP32, FP16, and INT8, with relative cost normalized to FP32, illustrating why narrower datatypes reduce arithmetic energy. Values follow the 45 nm estimates in Horowitz (2014) and section D.1.

Precision	Multiplier Energy	Relative Cost
FP32	~3.7 pJ/FLOP	1×
FP16	~1.1 pJ/FLOP	0.3×
INT8	~0.2 pJ/FLOP	0.05×

An 8-bit multiplier uses ~18× less energy than a 32-bit floating-point multiplier in this 45 nm component model because narrower arithmetic reduces switching and storage work (Horowitz 2014). Absolute values and ratios vary with circuit design and process technology.

Table 12.16 extends the picture to memory access, with energy cost per byte across each tier of the hierarchy:

Table 12.16: Memory Access Energy by Tier: Per-byte energy cost across the register-to-DRAM hierarchy, with relative cost normalized to a register read, exposing why off-chip memory traffic dominates inference energy budgets. Values follow the same canonical energy-estimate model used in Horowitz (2014) and section D.1.

Memory Level	Energy per Byte	Relative Cost
Register	~0.1 pJ/byte	1×
L1 Cache	~1 pJ/byte	10×
L2 Cache	~5 pJ/byte	50×
DRAM	~160 pJ/byte	1,600×

Memory access dominates: reading one byte from DRAM costs over 1,600× more energy than a register access.

Math: Component energy follows the energy corollary, with a load term $D_{\text{vol}} \times E_{\text{move}}$ (model bytes times DRAM energy per byte) and a compute term $O \times E_{\text{compute}}$ (operations times energy per operation). The FP32 terms sum as 2243 µJ + 2,220 µJ = 4,463 µJ; the INT8 terms sum as 561 µJ + 120 µJ = 681 µJ. Dividing the totals gives a 6.6× reduction under this simplified model.

Table 12.17 combines the two effects in a deliberately simplified MobileNetV2 model. It counts one DRAM read per weight and charges every cataloged FLOP at the listed multiplier energy, while excluding activation traffic, cache behavior, additions as a distinct cost, control overhead, and static power:

Table 12.17: MobileNetV2 INT8 Simplified Energy Estimate: One weight read plus a multiplier-energy charge for every cataloged FLOP under a 45 nm component-energy model. This is not a whole-device inference measurement.

Component	FP32 (14 MB)	INT8 (3.5 MB)	Savings
One weight read from DRAM	2243 µJ	561 µJ	4×
Compute (600 MFLOP)	2,220 µJ	120 µJ	18.5×
Total	**4,463 µJ**	**681 µJ**	**6.6×**

Systems insight: Within this simplified model, the weight read and the arithmetic contribute comparable FP32 energy (2243 µJ and 2,220 µJ), and INT8 reduces both, taking the total from 4,463 µJ to 681 µJ. This explains the physical mechanism behind potential savings, but battery-life or per-inference claims require controlled whole-device power measurements.

Beyond individual server nodes, large-scale deployments introduce shared infrastructure that complicates boundary definition. In data centers, cooling systems and power delivery account for 20 to 30 percent of total facility electricity consumption (Barroso et al. 2019). This facility-level overhead is captured by power usage effectiveness (PUE):

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}}$$

PUE values range from approximately 1.1 in hyperscale facilities using direct-to-chip liquid cooling to over 2.0 in legacy facilities relying on computer room air conditioning (Barroso et al. 2019). When an evaluation reports only server-plug energy, it excludes this facility multiplier. Furthermore, dynamic voltage and frequency scaling (DVFS) continuously modulates processor power and heat dissipation during execution (Kim et al. 2008), creating thermal feedback loops that alter the cooling overhead itself.

12.9.2 Computational efficiency vs. power consumption

Total power in complementary metal-oxide-semiconductor (CMOS) digital circuits is the sum of dynamic switching power and static leakage power:

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{leakage}} = \alpha C V^2 f + I_{\text{leak}} V$$

where α is the gate switching activity factor, C is physical load capacitance, V is supply voltage, f is clock frequency, and I_{leak} is temperature-dependent leakage current. Historically, computations per kilowatt-hour doubled approximately every 1.5 years—a long-run efficiency scaling trend known as Koomey's law (Koomey et al. 2011). Within any single hardware generation, however, dynamic power limits throughput scaling. Over an operating range where increasing clock frequency f requires supply voltage V to rise approximately proportionally, fixed activity and capacitance give the cubic approximation for dynamic power dissipation:

$$P_{\text{dynamic}} \propto f^3$$

Under that approximation, doubling clock frequency can incur an eightfold penalty in dynamic power (Le Sueur and Heiser 2010). This cubic penalty explains why modern ML accelerators favor massive parallel arrays of simple execution units clocked at moderate frequencies over narrow pipelines clocked at extreme frequencies.

Because frequency scaling quickly hits TDP ceilings, system designers optimize efficiency across the Data, Algorithm, and Machine axes. On the algorithm and data side, reduced-precision arithmetic (INT8 or FP8) directly reduces switching capacitance C in the ALUs and compresses memory traffic across interconnect buses, potentially cutting dynamic energy when the chosen format preserves the required model quality (Jacob et al. 2018; Wu et al. 2020; Gholami et al. 2021). On the machine side, the energy-delay product (EDP) measures one trade-off between energy and completion time. Its minimum need not coincide with minimum energy or maximum throughput per watt. Battery-constrained devices and power-limited data centers must choose operating points against their own latency, throughput, and energy requirements, measuring leakage and whole-system power alongside switching power.

12.9.3 Standardized power measurement

General-purpose power benchmarks such as SPECpower_ssj2008 measure server power across stepped load levels (0 to 100 percent in 10 percent increments) using steady-state transactional workloads (Lange 2009). In contrast, ML workloads exhibit extreme temporal volatility: instantaneous power swings rapidly between compute-bound matrix multiplications (where systolic arrays or Tensor Cores draw peak current) and memory-bound stalls or token transfers (where execution units stall waiting for DRAM or interconnect fetches).

Characterizing these fluctuations requires calibrated physical instrumentation. Software interfaces such as Intel RAPL and NVIDIA NVML expose counters or telemetry with device-specific accuracy, update rates, and polling overhead. The instrument must match the measurement boundary and required uncertainty. MLPerf power protocols use external calibrated analyzers for inference systems, while training and HPC measurements can use validated node or power-distribution telemetry; Tiny measurements use instrumentation suited to low-power devices (Tschand et al. 2024).

Resolving an individual transient requires instrument bandwidth and internal sampling appropriate to its duration; a kilohertz log cannot resolve arbitrary microsecond kernel events. Total run energy instead depends on accurate integration over the matched measurement window, which an analyzer may perform internally at a different rate from its exported readings:

$$E_{\text{total}} = \int_{t_{\text{start}}}^{t_{\text{end}}} P_{\text{device}}(t)\, dt$$

For a steady-state energy comparison, the harness separates initialization from the timed workload through three protocol stages:

1. A warm-up phase runs unmeasured inferences until caches, buffers, and temperature reach the intended operating state.
2. A sustained measurement window executes repeated inferences across minutes to average over microarchitectural phase shifts and thermal drift.
3. Hardware-to-software synchronization aligns power and workload timestamps so integrated energy and completed requests cover the same window. Cold-start or intermittent deployments require additional measurements that include initialization, sleep, and wake-up costs.

Workload structure further alters the power profile. In low-batch Transformer execution, decode often has less weight reuse than prefill and can face a lower bandwidth ceiling; their measured power profiles depend on shapes, kernels, and operating policy. DLRM embedding lookups can make memory traffic a substantial energy contributor, which must be measured alongside dense computation and static power. Batch sizing creates another trade-off: larger batches amortize parameter reads over more inputs and can lower energy per inference, while also changing activation traffic, execution time, and instantaneous power. At the edge, duty cycles govern the outcome through the duty-cycle average power of section 12.8.5.2: for an intermittent wake-word detector operating at a 1 percent duty cycle, baseline sleep and leakage power can consume most of the battery capacity, because the device spends 99 percent of its time asleep, rendering active inference efficiency secondary to low-power sleep state management.

12.9.4 MLPerf power case study

MLPerf Power (Tschand et al. 2024) translates raw electrical measurements into standardized efficiency metrics: completed inferences per second per watt under a rigorously defined system boundary. This protocol applies uniform measurement principles across data center, edge, and tiny inference scenarios, where the binding constraint shifts from rack cooling and operating cost to battery runtime and microwatt energy harvesting.

Standardizing the boundary prevents misleading comparisons across architectural families. By specifying instrumentation, measurement boundaries, matched logging windows, and the applicable workload constraints, MLPerf Power makes efficiency results interpretable within the tested scenario.

Over successive benchmark submission rounds from 2021 to 2024, standardized measurements capture how architectural improvements, compiler optimizations, and lower-precision arithmetic translate into production efficiency gains. Tracking normalized energy efficiency (samples per joule)

across eight submission cycles from v1.0 to v4.1, figure 12.8 charts diverging efficiency trajectories across seven distinct MLPerf Inference workloads.

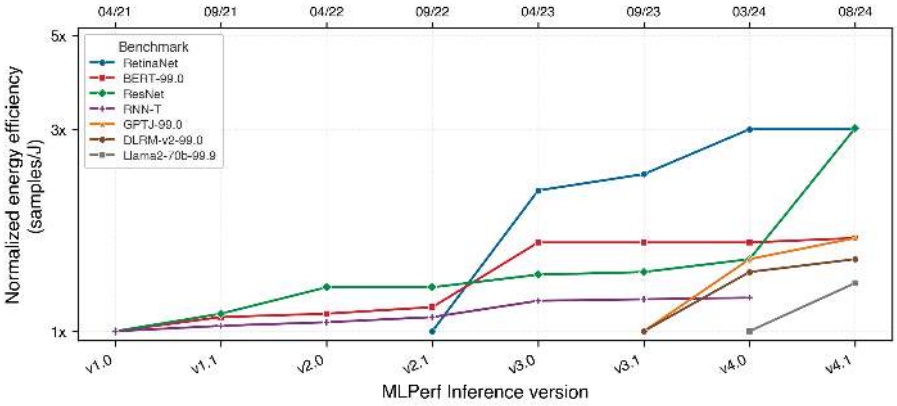


Figure 12.8: Data-Center Energy Efficiency Gains: Each series is normalized to its own first plotted version, so compare improvement within a workload rather than absolute efficiency across workloads. Source: (Tschand et al. 2024).

RetinaNet and ResNet show the largest plotted gains, each reaching 3× its initial normalized efficiency. BERT and GPT-J reach about 1.9×, while DLRM-v2 reaches about 1.7×, Llama 2 about 1.5×, and RNN-T about 1.3×. These trajectories combine changes in hardware, software, numerical formats, and submitted configurations. Static convolutional shapes can benefit from supported low-precision kernels and fusion; low-batch decoding and sparse embedding lookups often give memory improvements greater importance. The plotted ratios alone do not isolate how much each mechanism contributed.

Timing protocols and power instrumentation provide the raw data for benchmarking. Raw data alone, however, does not guarantee sound conclusions. Converting measurements into meaningful comparisons requires understanding the systematic sources of error, bias, and misalignment that can make even carefully collected benchmark numbers misleading.

12.10 Benchmarking Best Practices

An inference stack that passes a steady-state lab run can still miss latency targets when production traffic arrives in bursts, or when the input mix shifts toward expensive examples. Training throughput, inference latency, and power efficiency each have established measurement protocols validated through MLPerf, but knowing what to measure is insufficient without understanding what benchmarks cannot capture and why this gap causes production deployments to violate their service-level objectives.

Benchmarks make simplifying assumptions that enable standardized comparison but diverge from production reality. Training suites freeze datasets and tightly specify randomness; production data and retraining conditions evolve continuously. Inference suites emphasize a defined steady-state scenario; production traffic exhibits bursty arrival distributions and variable sequence lengths. Power benchmarks control thermal environments; deployed accelerators encounter variable ambient temperatures and sustained thermal loads. Four categories of limitations—statistical, deployment-related, system design, and organizational—determine whether benchmark results translate to deployment success.

12.10.1 Statistical and methodological issues

Benchmark results are only as reliable as the statistical apparatus that produces them. Three core methodological issues routinely undermine this reliability.

Incomplete problem coverage occurs when a benchmark exercises only a narrow operating point of a system. Controlled datasets such as CIFAR-10 (Krizhevsky 2009) constrain inputs to uniform, low-resolution images (32×32 pixels) with small per-image input footprints. This constrained input

shape may exercise memory differently from production images, but the complete working set still depends on the model and batch size, and allocation or kernel-launch overheads remain. A model that achieves high accuracy and throughput on a uniform, downscaled benchmark can stall in deployment when exposed to variable image resolutions, arbitrary aspect ratios, and sensor noise that force frequent kernel recompilations or fallback execution paths.

Statistical insignificance arises when evaluations rely on too few trials to distinguish algorithmic improvements from measurement noise, particularly when the evaluation medium introduces variance. Large language model evaluation exemplifies this vulnerability: whether scoring candidate generations against a reference via human preference ratings or an LLM-as-judge protocol, scores vary substantially based on prompt phrasing, judge model alignment, and response ordering. A reported two-point win can vanish under an alternative prompt template or judge configuration. Rigorous evaluation demands paired hypothesis tests and bootstrap confidence intervals over hundreds of independent prompt pairings. Omitting these confidence bounds obscures whether a reported margin reflects a genuine architectural advance or stochastic evaluation noise.

Reproducibility limits comparisons across software environments. A compiler or library revision can change kernel tiling, memory access, and reduction order, so an unexplained version change is a second intervention in the experiment. The run envelope established in section 12.7.2.6 must accompany the result; matching a model name and seed alone does not make two stacks equivalent.

12.10.2 Laboratory-to-deployment performance gaps

Statistical rigor ensures that benchmark measurements are precise, but precise measurements of the wrong operational regime still lead to production failures. Benchmark suites must align with the physical constraints of deployment, and the mismatch is sharpest where a standardized scenario meets live serving traffic. Section 12.12 returns to these gaps as a release problem once model and data evaluation are in place.

Misalignment with real-world goals persists even when a standardized benchmark is measured correctly. Offline throughput and Server throughput answer different questions, and the Server scenario still specifies only one arrival distribution. A production trace with correlated bursts or a different input mix can produce a different queue and memory footprint. The experiment must state which operating conditions its conclusion covers; production qualification tests those boundaries in section 12.12.

12.10.3 System design challenges

Statistical methodology and deployment alignment govern how performance is measured and targeted. A third category of limitations stems directly from the physical hardware executing the workload. Accelerator behavior fluctuates with thermal conditions, operating system resource contention, and underlying microarchitectural topology.

Environmental controls must match the claim being tested. Fixed clocks, CPU affinity, and isolated memory allocations can reduce noise in a comparison of implementations, while a deployment test must retain the relevant contention and thermal policy. The sustained-state mechanisms in section 12.8.5 apply to servers as well as edge devices. A result obtained under tighter isolation supports an implementation comparison, but does not by itself establish production capacity.

The hardware lottery[33] (Hooker 2021) compounds this physical variation by coupling algorithmic viability to specialized hardware support. Modern accelerators allocate the vast majority of their silicon area to dense matrix multiplication units (such as Tensor Cores or systolic arrays) backed by wide vector units and high-bandwidth memory hierarchies. Algorithmic architectures such as dense Transformers achieve high arithmetic intensity by reusing weights across batched tokens, sustaining a large fraction of peak theoretical FLOP/s. Conversely, architectures that rely on dynamic control flow, fine-grained sparsity, or graph message passing suffer from uncoalesced memory transactions and thread divergence, underutilizing available compute silicon. Algorithmic performance on a benchmark often reflects how naturally a model's computation graph maps onto dominant accelerator execution units rather than intrinsic mathematical superiority. The lottery is how the bitter lesson shows up on a leaderboard. General methods win by riding growth in computation, but the computation that grows is the dense form the dominant accelerators supply, so the winning methods are also the ones shaped to fit it.

[33] **Hardware lottery**: Coined by Hooker (2021) to describe how algorithmic success depends on alignment with available hardware and software. Transformer workloads benefit from dense matrix operations that map efficiently to modern accelerators, while graph neural networks and sparse mixture-of-experts models can be harder to evaluate when available silicon and software stacks favor dense kernels. Hardware-specific leaderboards therefore favor architectures aligned with the measured platform and may obscure alternatives that would fare differently under other hardware assumptions.

↩ The bitter lesson, that general methods which scale with computation outrun hand-built structure, is established in section 1.4.

Hardware compatibility dependence introduces systematic biases when evaluating models across heterogeneous execution targets. An architecture optimized for GPU execution may underperform on an embedded CPU, DSP, or custom edge accelerator with differing vector widths, cache hierarchies, and quantization support. Across five hardware targets on a Google Pixel 4, figure 12.9 charts how the latency–accuracy Pareto frontier reshuffles between CPU float, CPU uint8, GPU, EdgeTPU, and DSP execution. The diagonal purple arrow links MobileNetV3 Large at roughly 75 percent top-1 accuracy, from approximately 20 ms on CPU float to 4.5 ms on the DSP. Multi-MAX reaches roughly 75 percent accuracy at 2.2 ms on EdgeTPU and 3.4 ms on DSP; its points at higher accuracy carry higher latency. Comparisons must therefore match both the model variant and the accuracy operating point. The shifted frontiers show why model selection requires measurements on the target execution path rather than a latency ranking inherited from another processor.

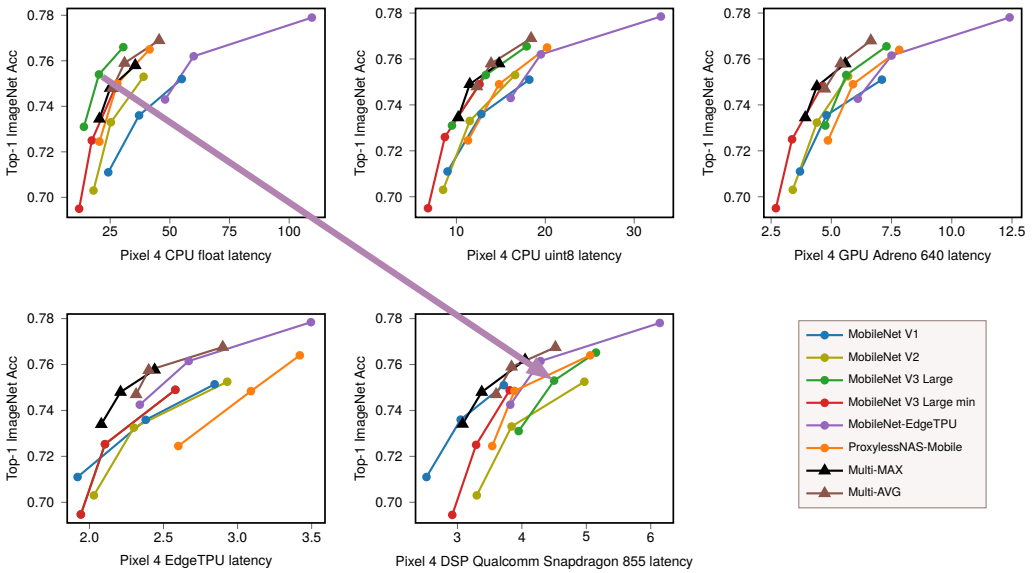


Figure 12.9: Hardware-Dependent Latency–Accuracy Trade-offs: Each panel holds top-1 ImageNet accuracy on the vertical axis while using a target-specific latency scale horizontally. The arrow links MobileNetV3 Large at roughly equal accuracy on CPU float and DSP, exposing the hardware-dependent latency shift. Source: (Chu et al. 2021).

Without benchmarking across heterogeneous hardware configurations, system designers risk selecting architectures solely because they conform to existing accelerator memory hierarchies and instruction sets. Evaluating models across diverse execution engines ensures that architectural decisions reflect fundamental computational efficiency rather than platform-specific co-design artifacts.

12.10.4 Organizational and strategic issues

Technical limitations—statistical noise, deployment misalignment, environmental variance, and hardware compatibility—address measurement mechanics. A distinct category of failure emerges from human incentives. Commercial procurement and academic leaderboard incentives create systematic biases in how benchmarks are executed and reported. Mitigating these organizational dynamics demands formal governance mechanisms and community audit standards to preserve benchmark integrity.

12.10.4.1 Benchmark engineering

While the hardware lottery is an unintended consequence of hardware specialization, benchmark engineering is the deliberate practice of tailoring models or systems to exploit the idiosyncrasies of specific benchmark harnesses. This practice yields inflated performance metrics that collapse under production conditions.

Benchmark engineering occurs when developers tune hyperparameters, preprocessing pipelines, or execution paths specifically to maximize benchmark scores rather than generalizable performance. Legitimate optimization respects the run rules and declared workload; misleading benchmark engineering exploits restricted test conditions while making broader claims that the evidence cannot support. For example, an inference pipeline might hardcode input tensor dimensions and pad batches specifically to match the memory stride of an accelerator's GEMM tile configuration, achieving peak throughput on fixed benchmark shapes while triggering costly kernel fallbacks and memory reallocations on arbitrary production inputs. Similarly, fine-tuning an evaluation pipeline to match the formatting biases of an automated evaluation model produces high leaderboard rankings that fail when interacting with unstructured human dialogue.

Procurement contracts and marketing claims create strong incentives to optimize directly for public benchmarks. When benchmark rankings dictate commercial adoption, engineering resources concentrate on optimizing for the test harness at the expense of general utility—the Goodhart's Law failure mode introduced in section 12.1 and illustrated with the BLEU-score example in section 12.3.1.

12.10.4.2 Bias and over-optimization

System engineers evaluating benchmark results must distinguish legitimate architectural gains from benchmark engineering. Several technical practices expose this gap. Transparency provides the initial defense: submissions that publish complete compiler flags, kernel configurations, and execution traces allow independent evaluators to separate generalized efficiency gains from harness-specific tuning. Reporting paired metrics from both controlled benchmarks and uncurated production traces exposes discrepancies in latency distributions and memory utilization. In addition, evaluating across multiple independently maintained benchmark suites raises the barrier against overfitting, as an optimization tailored to the synthetic quirks of one harness rarely transfers across diverse test suites.

Standardization and third-party verification enforce accountability. Independent audits verify whether reported speedups reproduce across identical hardware environments, as demonstrated in MLPerf's reference-vs-submission validation (section 12.10.5). Application-specific stress testing evaluates operational failure modes that static benchmarks omit, such as exposing vision models to adverse environmental conditions or subjecting inference engines to skewed request traffic. Finally, cross-hardware benchmarking validates whether performance advantages stem from genuine algorithmic superiority or narrow alignment with a single accelerator's memory hierarchy.

12.10.4.3 Benchmark evolution

Benchmarks are not static standards; they must evolve alongside model architectures and hardware capabilities. A benchmark that effectively separates system capabilities in one architectural generation becomes obsolete when models saturate its accuracy ceiling or when hardware innovations shift the primary system bottleneck. Stale benchmarks incentivize over-optimizing legacy computational kernels rather than advancing system efficiency.

This evolutionary cycle characterizes the history of machine learning benchmarks. Early deep learning benchmarks evaluated image classification and object detection on convolutional networks. As neural architectures expanded to large language models, recommendation systems, and generative AI, static vision benchmarks failed to capture long-context attention, autoregressive decoding, and sparse expert routing. In response, community benchmarks evolved to measure language understanding (A. Wang et al. 2018, 2019) and foundational model capabilities (Liang et al. 2022).

Benchmark evolution also expands the dimensions of measurement. While early benchmarks focused on throughput and accuracy, contemporary systems require multi-objective evaluation across energy efficiency, memory footprint, and tail latency under resource constraints. Scientific instruments provide a further constraint regime in figure 12.10. Its shaded "Fast ML for Science" envelope places high-throughput trigger pipelines outside typical mobile and IoT operating points (Duarte et al. 2022). The broader spectrum illustrates why a benchmark must match its application. Scientific trigger design extends the lesson beyond the single-machine protocols developed here.

Managing benchmark evolution introduces a fundamental trade-off between longitudinal stability and architectural relevance. Benchmarks must remain fixed long enough to allow rigorous cross-generation hardware and software comparisons. If benchmarks change too frequently, tracking

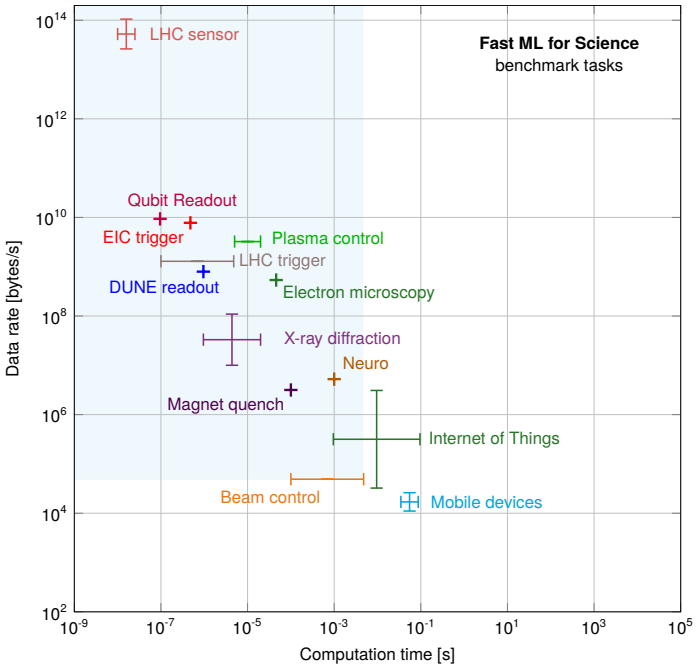


Figure 12.10: Performance Spectrum: Scientific and edge applications occupy widely separated operating points in data rate and available computation time, motivating benchmarks tied to each application's timing and data-movement constraints. Source: (Duarte et al. 2022).

historical progress becomes impossible. Conversely, freezing a benchmark indefinitely leads to metric stagnation and over-specialization. Modern benchmarking consortia balance this trade-off by operating on fixed release cycles that introduce new representative workloads while deprecating obsolete suites.

12.10.5 MLPerf synthesis and benchmark gaming

To prevent benchmark gaming from corrupting procurement and engineering evaluations, standardized suites enforce strict run rules backed by reference implementations, submission audits, and division boundaries. In quantitative computer architecture, benchmark rules strictly forbid three classes of non-generalizable shortcuts:

1. **Precision truncation without disclosure**: A compiler silently lowers numerical precision (for instance, executing intermediate activations in FP8 or INT8 instead of BF16) to inflate arithmetic throughput, without disclosing the reduced dynamic range or verifying whether the lower precision degrades numerical stability on production workloads.
2. **Work pruning and operator elimination**: An execution engine exploits fixed benchmark test sets to skip operations required by general production inputs, such as bypassing dynamic padding removal, masking out-of-bounds tokens, or skipping input boundary validation.
3. **Benchmark fingerprinting**: A runtime inspects input tensor dimensions, dataset hash fingerprints, or memory layout patterns to identify the benchmark suite, executing hardcoded, pre-tuned execution graphs and bespoke kernel configurations that are inaccessible to arbitrary user workloads.

MLPerf detects and deters these shortcuts through submission divisions, reference-equivalence checks, and peer-reviewed code audits. In the closed division, submissions must execute the identical reference workload and satisfy version-specific convergence and quality targets. Permitted changes include reproducible runtime optimizations, memory layout transformations (such as converting NCHW to NHWC), and hardware-generalizable kernel fusion. Submitters can use approved low-precision numerical formats provided they produce documented, verifiable output quality, but

non-generalizable shortcuts remain prohibited. Broader architectural alterations must be submitted under open-division rules, where submitters explicitly document modifications to compare entire solutions rather than isolated systems.

Even the most rigorous system benchmarks validate only one dimension of deployment readiness. Achieving record throughput and energy efficiency on an MLPerf workload confirms that the hardware and runtime stack can deliver high computational efficiency, but it cannot determine whether the deployed model generalizes to real-world distributions or whether the underlying dataset represents the operational population. In the D·A·M taxonomy, hardware throughput (Machine) is a necessary foundation, but algorithmic fidelity (Algorithm) and dataset integrity (Data) must hold simultaneously. Completing the validation stack requires evaluating across all three dimensions of the D·A·M taxonomy—ensuring that data quality, model capability, and hardware efficiency cohere under production constraints.

12.11 Model and Data Evaluation

A compressed model running on accelerated hardware can still fail if it was trained on biased data. System benchmarks can confirm that hardware delivers promised training throughput, inference latency, and power efficiency, but hardware validation alone cannot ensure deployment success. The optimization pipeline from Part III also included model compression and data selection, each requiring its own validation. The framework's other two dimensions address this gap: model benchmarks verify that compression preserved accuracy and critical model properties, while data benchmarks verify that training data enables robust generalization.

12.11.1 Model benchmarking

Model benchmarks validate whether compression preserved the properties that matter for deployment. This evaluation extends beyond top-line accuracy. A pruned model might maintain ImageNet accuracy while losing robustness to adversarial inputs. A quantized model might preserve average-case performance while degrading on rare but critical edge cases. A distilled model might match the teacher's accuracy while losing calibration. Historically, benchmarks focused almost exclusively on accuracy, but compression makes multi-dimensional evaluation essential.

↩ Pruning, quantization, and distillation, the compression techniques these benchmarks validate, are established in chapter 10.

Evaluating a compressed model requires comparing its behavior against an uncompressed reference baseline. In computer vision, standard network architectures establish the accuracy and resource baselines against which compression techniques are judged: a pruned or quantized model must demonstrate how much task accuracy it sacrifices for a given reduction in latency, memory footprint, or energy consumption. Canonical reference models such as ResNet-50[34] serve as standard MLPerf workloads precisely because their moderate scale (25.6M parameters) and well-understood compute profile (8.2 GFLOP per image) make them sensitive to both hardware and software optimizations without requiring distributed multi-node setups (He et al. 2016a; Mattson et al. 2020).

[34] **ResNet (residual network)**: Introduced by He et al. (2016a), skip connections enabled deep residual networks and achieved 3.57 percent top-5 ImageNet error (ensemble), surpassing the estimated human error rate in the ImageNet challenge context (Russakovsky et al. 2015). ResNet-50 became a standard MLPerf Training reference workload because its moderate size (25.6M parameters) and predictable computational load (8.2 GFLOP per image) provide sensitive instruments for measuring both software and hardware optimizations (Mattson et al. 2020).

12.11.1.1 Accuracy metrics and their blind spots

The most common model metrics (accuracy, precision, recall, F1) each reveal different aspects of model behavior while hiding others, and understanding their blind spots is essential for compression validation. Top-k accuracy measures whether the correct label appears in the model's top-k predictions. Top-1 accuracy is strict; top-5 is lenient. The gap between them reveals ranking behavior: a model with 75 percent top-1 but 95 percent top-5 accuracy often ranks the correct class among its leading candidates without placing it first. These rank statistics do not establish whether its confidence scores are calibrated. For deployment, the acceptable gap depends on whether downstream systems can use ranked predictions or require single answers.

Precision and recall matter when classes are imbalanced or errors have asymmetric costs (Sokolova and Lapalme 2009). A fraud detection model with 99 percent accuracy might have 10 percent recall on actual fraud (catching only one in 10 fraudulent transactions), an unacceptable outcome despite high accuracy. Precision (the fraction of predicted positives that are correct) and recall (the fraction of actual positives identified) expose failures that aggregate accuracy conceals.

Aggregate metrics also conceal subgroup failures. A model achieving 95 percent overall accuracy might achieve 60 percent on a critical demographic subgroup. The Gender Shades project (Buolamwini and Gebru 2018) revealed commercial gender-classification systems for facial analysis

performing substantially worse on darker-skinned women than on lighter-skinned men, a disparity invisible to aggregate benchmarks. Disaggregated evaluation across deployment-relevant subgroups is essential.

↪ Disaggregated evaluation as a release gate, with the slices, metrics, and thresholds that decide whether a model ships, is operationalized in section 15.3.2.

12.11.1.2 Calibration: When confidence scores matter

For many deployment scenarios, *how confident* the model is matters as much as *what* it predicts. A well-calibrated[35] model's confidence scores correspond to actual correctness probability: among 100 predictions made with 80 percent confidence, about 80 should be correct in expectation.

Calibration failures create severe operational hazards in automated pipelines. An overconfident model automates catastrophic errors that it should defer to human review (for example, reporting 95 percent confidence but failing 30 percent of the time). Conversely, an underconfident model floods manual review queues with decisions it could safely automate (reporting 60 percent confidence while being correct 98 percent of the time).

To quantify calibration across a dataset, predictions are partitioned into N_{bins} confidence bins $\text{bin}_1, \text{bin}_2, \ldots, \text{bin}_{N_{\text{bins}}}$ (such as ten intervals of width 0.1 from 0 to 1). For each bin_m, the evaluator computes the empirical accuracy $\text{acc}(\text{bin}_m)$ (the fraction of correct predictions in that bin) and the average predicted confidence $\text{conf}(\text{bin}_m)$. **Expected calibration error (ECE)** measures the weighted average difference between confidence and accuracy in equation 12.4:

$$\text{ECE} = \sum_{m=1}^{N_{\text{bins}}} \frac{|\text{bin}_m|}{N} \left|\text{acc}(\text{bin}_m) - \text{conf}(\text{bin}_m)\right| \tag{12.4}$$

where N is the total sample count and $|\text{bin}_m|$ is the sample count in bin m. A measured $\text{ECE} = 0$ means confidence and accuracy agree within the chosen finite-sample bins. Population calibration implies agreement in expectation; a zero estimate does not prove calibration at every confidence level or on another distribution.

Compression frequently disrupts calibration even when preserving aggregate accuracy, a critical concern when validating quantization techniques from section 10.4. When floating-point weights and activations are quantized to 8-bit integers, discrete rounding alters the numerical geometry and relative separation of final-layer logits. Often, headline top-1 accuracy barely moves, but the model becomes severely overconfident—assigning extreme probabilities to inputs where it is genuinely uncertain. Post-hoc techniques such as temperature scaling can improve calibration when fitted on separate calibration data and evaluated on an untouched representative holdout (Guo et al. 2017).

[35] **Calibration**: From Arabic *qalib* (a mold for casting metal) via Latin *calibrare*, originally describing the adjustment of measuring instruments against known standards. In ML, calibration ensures predicted probabilities match empirical frequencies; Guo et al. (2017) formalize this concern for modern neural networks and show that temperature scaling is a simple effective post-hoc correction. The etymology is apt: just as an uncalibrated instrument produces precise but inaccurate measurements, an uncalibrated model produces confident but unreliable predictions, causing downstream systems that threshold on confidence scores to make systematically wrong decisions.

12.11.1.3 Compression benchmarks

Neural network compression (pruning, quantization, knowledge distillation, and architecture optimization) requires specialized benchmarks because compression reshapes the trade-off landscape: every byte saved or operation eliminated must be weighed against potential accuracy loss and hardware compatibility. The most basic compression metric is raw size reduction: parameter count, memory footprint in bytes, and compressed storage requirements. Size alone, however, is misleading. On ImageNet, MobileNetV2 achieves approximately 72 percent top-1 accuracy with 3.5M parameters vs. ResNet-50's 76 percent accuracy with 25.6M parameters, about 7.3× fewer parameters at comparable accuracy, or roughly 6.9× more accuracy per parameter (Sandler et al. 2018; He et al. 2016a).

Pruning benchmarks must distinguish between structured and unstructured approaches, because they produce qualitatively different results on real hardware. Structured pruning removes entire neurons or filters, yielding smaller dense operations that conventional kernels can exploit (Li et al. 2017). Unstructured pruning eliminates individual weights and can produce very sparse models, but realizing actual speedups requires specialized sparse computation support—meaning benchmark protocols must specify hardware platform and software implementation (Han et al. 2015; Gale et al. 2019).

Quantization benchmarks evaluate precision reduction across data types. In the illustrative MobileNetV2 scenario, INT8 reduces raw weight storage by 4× and the assumed latency values yield the speedup in example 12.2; realized performance depends on the hardware and implementation. The precision-accuracy trade-off is worked in lighthouse 12.3 and the energy implications

in section 12.9.1. Mixed-precision approaches push further by applying different precision levels to different layers: critical layers retain FP16 while computation-heavy layers use INT8 or INT4, enabling fine-grained efficiency optimization. Knowledge distillation adds another dimension: a smaller student model can preserve much of a teacher's behavior while reducing size and inference cost, but benchmarking must verify that the student generalizes rather than merely memorizing the teacher's outputs (Hinton et al. 2015).

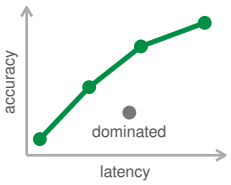
Acceleration factors vary across hardware platforms because sparse models, reduced-precision models, and efficient architectures need compatible kernels, memory layouts, and accelerator support. Current benchmark suites like MLPerf focus primarily on standardized reference models, while production deployments often use compressed or hardware-specific variants. A compression report must therefore name its representation and execution path before an efficiency-quality comparison can establish whether the variant is deployable.

12.11.1.4 Compression validation: The efficiency-quality frontier

A compression report identifies the representation and hardware path, but deployment acceptance requires deciding whether their measured gains justify changes in model behavior. Size, latency, and energy improvements must be evaluated against the quality properties and limits the application depends on.

Pareto frontier[36] evaluation (principle 5) determines whether a compressed model represents a good trade-off. Plotting accuracy against the target efficiency metric (latency, model size, energy) reveals the trade-off frontier. Models on the Pareto frontier *cannot* improve one metric without degrading the other; models below the frontier are dominated by better alternatives.

[36] **Pareto frontier**: Named after economist Pareto (1896), the frontier contains all solutions where improving one objective requires degrading another. In compression benchmarking, the trade-off between adjacent candidates must be interpreted using the named axes and units. A candidate is dominated when another is at least as good on every objective and strictly better on one; frontier membership still does not establish that deployment thresholds are met.



A dominated model is matched or beaten on both accuracy and latency by a frontier model.

Different compression techniques make different efficiency-quality trade-offs. Quantization reduces precision while often preserving aggregate accuracy (Jacob et al. 2018). Pruning induces sparsity while retaining varying levels of test performance (Han et al. 2015; Gale et al. 2019). Distillation transfers knowledge from a larger teacher or ensemble into a smaller model (Hinton et al. 2015). Because these aggregate results do not establish preserved calibration or tail behavior, validation must evaluate those properties directly (Guo et al. 2017).

Evaluating calibration during compression requires a standardized protocol. Expected calibration error (ECE) compares predicted confidence with empirical accuracy, but it has no universal thresholds. The metric depends on the binning scheme, bin count, sample size, and prediction distribution. A benchmark protocol must therefore freeze the estimator and evaluate degradation relative to the uncompressed baseline under a fixed task tolerance.

Pareto optimality identifies nondominated choices, but it does not establish that any choice clears the deployment's absolute quality floor. Acceptable degradation depends on deployment context: a 2 percent accuracy drop might be acceptable for a recommendation system where users tolerate imperfect suggestions, but unacceptable for medical diagnosis where every false negative carries severe consequences. Production acceptance gates must define absolute accuracy thresholds before compression, evaluating aggregate accuracy, calibration stability, and deployment-critical slices simultaneously. The MobileNetV2 lighthouse makes this multi-dimensional validation protocol concrete.



Lighthouse 12.3: MobileNetV2 INT8 compression

Returning to lighthouse 12.1, consider an illustrative validation protocol for INT8 quantization, grounded in MobileNetV2's architecture (Sandler et al. 2018) and post-training quantization practice (Jacob et al. 2018). The values in table 12.18 are assumed. For this example, acceptance permits less than one percentage point of loss on either top-one or top-five accuracy and requires no regression in ECE or critical edge-case accuracy under the same evaluation protocol:

Precompression baseline: MobileNetV2 achieves 71.8 percent top-1 accuracy on ImageNet at 3.5M parameters (14 MB FP32).

In table 12.18, aggregate accuracy barely changes after INT8 quantization to 3.5 MB, but calibration error and edge-case accuracy reveal substantial degradation. The INT8 model's

ECE rises from 0.031 to 0.089, failing this example's no-regression criterion under the fixed ECE protocol.

Table 12.18: MobileNetV2 INT8 Postcompression Metrics: Illustrative FP32 and INT8 values for top-1 and top-5 accuracy, calibration error, and edge-case accuracy, showing how aggregate accuracy can mask calibration and tail-case degradation.

Metric	FP32	INT8	Acceptable?
Top-1 accuracy	71.8%	70.9%	✓ (0.9 pp drop; below 1 percentage-point threshold)
Top-5 accuracy	91%	90.4%	✓ (below the stated accuracy-loss threshold)
Calibration ECE	0.031	0.089	No (degraded)
Edge-case accuracy	68.2%	61.4%	No (drop of 6.8 pp)

Edge-case definition: Images with $>$ 50 percent occlusion, $<$ 100 lux lighting, or $>$ 30° rotation from training distribution (approximately 5 percent of real-world inputs).

What this reveals: Under these assumptions, average-case accuracy looks acceptable (0.9 percentage-point drop), but calibration degrades and edge-case accuracy drops 6.8 percentage points. If the deployment uses confidence thresholds (for example, "only act if confidence > 85 percent") or encounters many edge cases (unusual lighting, partial occlusions), INT8 MobileNetV2 could fail despite passing aggregate benchmarks.

Fix: Evaluate temperature scaling as a post-hoc calibration candidate (Guo et al. 2017). Fit a positive scalar T_{cal} on separate calibration data to divide logits before softmax: $\text{softmax}(z_i/T_{\text{cal}})$. Validate on an untouched holdout. This rescaling preserves the top-one ranking, so edge-case classification errors require separate diagnosis of quantization and data coverage, followed by correction and testing on reserved edge-case examples.

The Lottery Ticket Hypothesis provides concrete benchmarking data illustrating what Pareto-efficient compression looks like. Through iterative pruning, Frankle and Carbin (2019) found sparse subnetworks ("winning tickets") in fully connected and convolutional networks that could match the original network's test accuracy when trained in isolation. These empirical findings demonstrate the nonlinear shape of compression trade-offs: aggressive pruning preserves accuracy up to a critical sparsity threshold, beyond which quality degrades sharply. Compression validation must therefore establish empirical trade-off curves for each specific model and architecture, identifying where the model sits on the Pareto frontier and whether further pruning yields hardware efficiency or merely degrades accuracy.

12.11.1.5 Large language model benchmarks

The compression evaluation framework applies cleanly when the task has a stable label, such as classification accuracy, detection mAP, or segmentation IoU. Large language models break that pattern. A team can choose a model because it scores well on a public benchmark, then discover in deployment that the model recognizes multiple-choice facts but cannot generate a grounded answer, responds too slowly for an interactive product, or produces confident unsafe text that the benchmark never stressed. LLM benchmarking therefore starts by naming the deployment failure that a score is meant to rule out.

The metric taxonomy in table 12.19 organizes evaluation around those failure modes rather than leaderboard rankings. Its rows use Massive Multitask Language Understanding (MMLU),[37] HELM (Holistic Evaluation of Language Models),[38] and perplexity[39] as examples of scores that answer distinct deployment questions:

[37] **MMLU (massive multitask language understanding)**: Introduced by Hendrycks et al. (2020) with 15,908 multiple-choice questions across fifty-seven subjects. MMLU's benchmarking limitation is its format: multiple-choice recognition is not the same task as open-ended generation, so an MMLU score should not be read as direct evidence that a model can produce grounded free-form answers in production.

[38] **HELM (holistic evaluation of language models)**: Stanford's 2022 evaluation framework tested a broad set of models across seven dimensions (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency) (Liang et al. 2022). HELM's contribution is methodological: by evaluating models that score similarly on accuracy but diverge on calibration or toxicity, it demonstrates that single-metric leaderboards systematically hide failure modes that matter for production deployment.

[39] **Perplexity**: From Latin *perplexus* (entangled); in information theory, $2^{H(p)}$ where H is entropy. A perplexity of 10 means the model is "10-way confused" on average. The systems consequence is interpretive rather than direct memory accounting: perplexity measures held-out next-token prediction on a corpus, while serving memory pressure is governed by context length, batch size, model shape, and decoding state; KV-cache management is a separate serving problem (Kwon et al. 2023).

Table 12.19: Failure-Oriented LLM Benchmark Taxonomy: LLM metrics are useful when each score is tied to the deployment failure it is meant to rule out. Recognition, holistic behavior, corpus prediction, and responsiveness expose different risks, so no single score establishes production readiness.

Deployment failure to rule out	Metric or benchmark family	What the score reveals	What the score cannot prove
The model recognizes facts poorly	MMLU (Massive Multitask Language Understanding)	Broad factual and disciplinary knowledge across fifty-seven subjects, with scores interpretable against chance-level multiple-choice performance	Whether the model can generate grounded open-ended answers rather than choose among multiple-choice options
The model is capable but unsafe	HELM (Holistic Evaluation of Language Models)	Accuracy alongside calibration, robustness, fairness, bias, toxicity, and efficiency	Whether one aggregate score captures the deployment risk; a model can be strong on accuracy and weak on calibration, safety, cost, or prompt stability
The model predicts its corpus well	Perplexity	Held-out next-token prediction on the same corpus; a perplexity of 10 means the model is "10-way confused" on average	Whether generated answers are helpful, safe, or grounded outside that corpus
The model feels slow in use	First-token latency, inter-token latency, and token throughput	Prompt-processing delay before generation starts and decode speed after generation begins	Whether a single throughput number hides poor interactive responsiveness, especially when batching improves throughput but worsens first-token latency

The responsiveness row requires a concrete timing anchor because LLM benchmarks frequently report a single aggregate throughput number even though users experience generation in distinct phases. A model can achieve high aggregate throughput in tokens per second while still feeling sluggish if the first token arrives late, or it can minimize first-token latency while generating subsequent text too slowly for interactive reading.

Token throughput translates these generation rates into user-perceived wall-clock time: for a response of 750 tokens, a generation rate of 25 tokens/s requires 30 seconds, whereas 100 tokens/s finishes in 7.5 seconds. Time-to-first-token and inter-token latency must therefore be reported together: one captures initial responsiveness, and the other captures sustained delivery speed. This latency split reflects contrasting computational regimes: time-to-first-token is governed by prompt prefill, which processes tokens in parallel and is typically compute-bound, whereas inter-token latency is governed by autoregressive decoding, where low-batch execution often streams weights and accesses a growing KV cache with limited reuse. The resulting bandwidth pressure depends on model residency, context length, batch size, and the execution path.

A final failure mode is that a benchmark score may reflect training-data memorization rather than general capability. Benchmark contamination is especially difficult to audit in LLMs because web-scale pretraining corpora may contain benchmark questions, inflating scores through memorization rather than skill (Xu et al. 2024). Leakage detection reframes this risk as something benchmark designers can test for rather than merely suspect. Temporal holdouts use content published after the training cutoff, dynamic benchmarks generate fresh instances continuously, and contamination tests check whether the model recalls verbatim benchmark phrasing. These techniques keep the benchmark aligned with the deployment question instead of rewarding pretraining exposure to the evaluation set.

12.11.2 Data benchmarking

Model benchmarks validate whether compression preserved model quality. Model quality, however, depends on the data alongside the architecture and training procedure, and this dependency creates the most insidious failure mode in ML deployment. A perfectly preserved model trained on biased or unrepresentative data can still fail in production. Data benchmarks validate whether the efficiency strategies from chapter 9 (active learning, curriculum design, data augmentation, and synthetic data generation) produced training sets that enable reliable deployment. This is often the last validation to fail and the hardest to diagnose: a model achieving excellent accuracy on held-out test data may collapse on production inputs that the training data never adequately represented.

↩ Active learning, curriculum learning, augmentation, and synthetic data generation, the strategies a data benchmark audits, are established in chapter 9.

Within the D·A·M taxonomy, optimizing the Algorithm and the Machine cannot compensate for structural deficiencies in the Data: an accelerator cluster operating at peak arithmetic intensity simply computes incorrect representations faster when training distributions are defective. A data benchmark therefore begins with an operational protocol rather than an aggregate score. The protocol defines the deployment slice the model must serve, reserves a leakage-resistant holdout, verifies duplicate and near-duplicate separation across partitions, sets minimum coverage for rare classes and subgroups, audits label quality, and establishes drift thresholds that determine when the benchmark no longer represents production. Only after those gates are explicit do aggregate metrics become interpretable.

Coverage metrics

The first question data benchmarking must answer is whether the training distribution covers the operational domain the model will encounter in production. Generalization to poorly represented input regimes is uncertain and requires direct evaluation; the mechanisms through which training coverage fails are often subtle.

Class balance is the most apparent failure mode. In a fraud detection dataset with 99 percent legitimate transactions and 1 percent fraud, a degenerate model that predicts the majority class unconditionally achieves 99 percent accuracy while failing its operational purpose. Mitigating severe class imbalance requires loss reweighting, targeted oversampling, or decision threshold calibration. More insidious is subgroup imbalance within classes: a dataset may exhibit balanced positive and negative classes in the aggregate, yet draw negative examples disproportionately from a specific demographic cohort or geographic region. Aggregate accuracy metrics mask these systematic localized failures. When systems interact with human populations, auditing representation across demographic dimensions (such as age, geography, or dialect) is essential, though missing or noisy demographic metadata frequently complicates detection.

Feature coverage extends beyond discrete class labels into the continuous space of operational variations. While class balance is verified directly from label statistics, feature coverage requires mapping the physical parameters of the deployment environment. A vision pipeline trained exclusively on daytime captures fails under low-light sensor noise; a voice processing pipeline trained on clean studio signals degrades under acoustic reverberation and ambient noise. These environmental variations—differing sensor hardware, lighting conditions, and rare operational edge cases—fall outside discrete label distributions. Benchmarking feature coverage therefore requires specifying concrete bounds on the input space alongside domain experts before evaluating model performance.

Quality metrics

Even when a training dataset spans the operational domain, annotation errors corrupt the supervisory signal. Label errors, which average about 3 percent across ten canonical test sets and approach 6 percent on ImageNet's validation set (Northcutt et al. 2021), do not act as benign noise. Supervised optimization treats incorrect labels as ground truth. When an evaluation set contains corrupted labels, a model that learns the true underlying distribution is penalized, while a model that memorizes the annotation artifacts receives an inflated score.

For small corpora, manual audits over stratified random samples provide baseline error estimates. At scale, confident learning identifies label errors algorithmically by estimating the joint distribution of noisy annotations and true latent labels using out-of-sample predicted probabilities. When a model consistently assigns high confidence to a class that conflicts with the assigned label, the discrepancy isolates candidate annotation errors from typical optimization uncertainty. Flagging these candidates enables targeted relabeling or filtering, circumventing the prohibitive cost of exhaustive human review.

Inter-annotator agreement measures consistency across annotators prior to model training. The statistics are the ones data engineering already uses to monitor label quality in production; a data benchmark applies them once, to certify the evaluation set itself. Cohen's kappa for two raters and Fleiss' kappa for multiple raters quantify agreement above chance expectation (Cohen 1960; Fleiss 1971). When agreement metrics fall below established thresholds on tasks with objective ground truth, the failure points to ambiguous labeling taxonomies, rater fatigue, or inconsistent guidelines. While Landis and Koch's qualitative kappa bands provide a historical reference (Landis and Koch

1977), production pipelines require agreement thresholds calibrated directly to application-specific risk tolerance.

↩ Cohen's kappa, its chance-agreement correction, and label quality drift are established in section 4.5.4.1.

The structural form of label noise determines its downstream impact. Random label noise dilutes gradient signals across classes, increasing sample complexity but rarely teaching coherent false representations. In contrast, systematic errors—such as consistently mislabeling an underrepresented subclass or coupling a label with an environmental artifact—inject structured bias into model weights. If images of wolves photographed against snow are consistently labeled as dogs, the model learns to associate snow with the target class. In the presence of systematic label error, collecting more training data under the same labeling protocol does not correct the bias; it merely reinforces the spurious association with higher parameter certainty.

Distribution alignment

Data benchmarking must ultimately evaluate whether a model generalizes from training conditions to deployment environments. Standard evaluation protocols assume that training, validation, and test splits are independent and identically distributed (i.i.d.). In production systems, this assumption fails routinely: temporal drift alters consumer behavior, sensor degradation shifts hardware characteristics, and geographic expansion introduces unmodeled subpopulations. When the i.i.d. assumption breaks down, held-out validation accuracy may become a poor predictor of operational performance. The direction and size of the change depend on the shifted population.

This train-to-production alignment gap is quantified systematically by the WILDS[40] benchmark (Koh et al. 2021), which evaluates models across distribution shifts spanning hospital systems, camera trap locations, and satellite passes. On Camelyon17-WILDS, the reported ERM baseline achieved 93.2 percent in-distribution average accuracy and 70.3 percent out-of-distribution average accuracy across unseen clinical sites. This performance drop occurs despite holding the architecture, hyperparameters, and training compute constant, demonstrating a distribution-dependent performance gap for that fixed model and training procedure. The comparison does not isolate which capacity, representation, or data changes would close it.

40 **WILDS**: Stanford's 2021 benchmark of ten datasets with real-world distribution shifts: hospital changes (Camelyon17), wildlife camera location shifts (iWildCam), and satellite imagery temporal drift (PovertyMap). For the reported Camelyon17-WILDS ERM baseline, average accuracy fell from 93.2 percent in-distribution to 70.3 percent out-of-distribution, demonstrating that standard held-out evaluation can overestimate performance when the i.i.d. assumption fails.

To detect changes that warrant investigation, serving pipelines can apply the statistical shift tests and the covariate, label, and concept shift taxonomy of section 4.5.4 to incoming inference streams; the benchmarking use is to decide when the evaluation set has stopped representing production. Two-sample hypothesis tests such as the Kolmogorov-Smirnov test (Berger and Zhou 2014) for univariate features and maximum mean discrepancy (Gretton et al. 2012) for multivariate embeddings detect changes in the marginal input distribution $p(x)$. Establishing covariate shift also requires the conditional target distribution $p(y \mid x)$ to remain stable; input-only tests cannot verify that condition without label evidence. Prediction entropy and confidence distributions provide additional warning signals, but shifted inputs can still produce confident predictions. Validate alert thresholds and fallback policies against labeled deployment samples; these signals alone do not guarantee detection before quality degrades.

↪ Live covariate shift detectors feed the statistical distance monitoring and automated drift alerts developed in section 25.

Distribution alignment challenges highlight a persistent tension in ML development between two paradigms: *fixing the data and iterating on models*, or *fixing the model and iterating on data*. The side-by-side schematics in figure 12.11 isolate this divergence through the placement of the padlock and feedback path. In the model-centric panel, the dataset is locked while the feedback loop iteratively refines model parameters on the CPU; in the data-centric panel, the model architecture is locked while the loop routes back to enhance training data quality, diversity, and labeling. As the bidirectional red arrow emphasizes, the two paradigms are complementary engineering strategies rather than mutually exclusive philosophies.

Data-centric AI reflects an important shift in understanding that challenges the "more data is always better" assumption: dataset composition matters alongside scale. Initiatives like DataPerf (Mazumder et al. 2023) and DataComp[41] have emerged to evaluate how dataset construction affects model performance systematically. In DataComp's compute-controlled setting, a baseline that retained the top 30 percent of the candidate pool by CLIP-based filtering outperformed the unfiltered-pool baseline on aggregate downstream evaluation (Gadre et al. 2023). The result establishes the value of curation for that protocol, not a universal optimal fraction.

41 **DataComp**: Introduced in 2023, DataComp fixes the model family, training code, and compute budget while participants vary dataset construction. Its filtering track isolates the effect of selecting examples from a common candidate pool, making curation strategies comparable without attributing every gain to additional training compute.

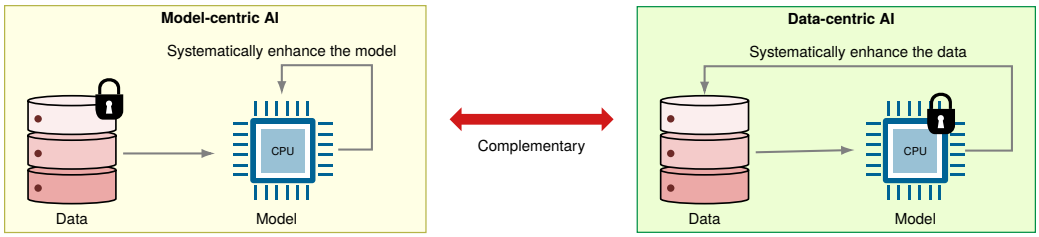


Figure 12.11: Development Paradigms: Model-centric AI iterates on architecture while holding the dataset fixed, whereas data-centric AI iterates on annotations, diversity, and bias while holding the architecture fixed. The double arrow marks the approaches as complementary.

Dataset saturation and dynamic benchmarks

Even when coverage, quality, and distribution alignment protocols are satisfied, static benchmarks encounter the problem of dataset saturation. As models approach empirical upper bounds on fixed test suites, leaderboard deltas compress, making it difficult to distinguish genuine capability advances from over-optimization to benchmark-specific artifacts. The capability trajectories in figure 12.12 plot this exhaustion over two decades, showing AI scores rising from normalized baselines (-100) to cross the zero line marking human performance (Maslej et al. 2024). While handwriting and speech recognition required over a decade to reach human parity (the green shaded region), reading comprehension and language understanding surged across the threshold in under two years between 2016 and 2020, rapidly exhausting the discriminating headroom of static suites.

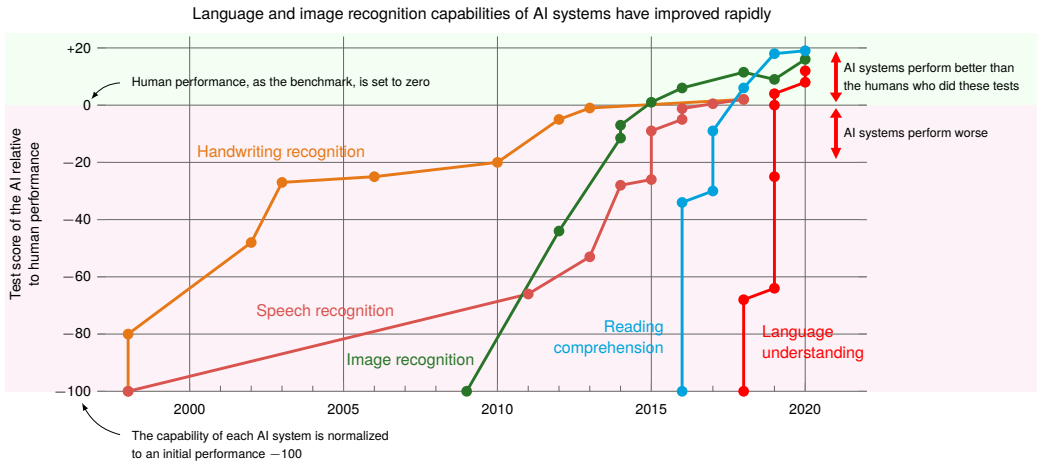


Figure 12.12: Dataset Saturation: The zero line denotes reported human performance; each curve's crossing marks a capability for which the static benchmark has less remaining discriminating headroom. The crossings do not distinguish genuine capability gains from optimization to the evaluation set. Sources: (Maslej et al. 2024; Kiela et al. 2021).

Once a static benchmark saturates, high leaderboard accuracy can mask brittle heuristics. MNIST digit recognition (LeCun et al. 1998) demonstrated early that models can exploit dataset-specific capture artifacts rather than invariant features, a concern that eventually prompted researchers to re-examine whether ImageNet had reached the end of its useful diagnostic lifetime (Beyer et al. 2020). When the evaluation distribution remains frozen, iterative model selection risks overfitting to idiosyncratic test artifacts rather than improving generalizable task capability.

Dynamic benchmarking frameworks such as Dynabench[42] (Kiela et al. 2021) address saturation by continuously evolving evaluation data. In this paradigm, human annotators generate adversarial inputs targeting the failure modes of current top-performing models, ensuring that the benchmark continues to exercise representational limits. However, dynamic benchmarks introduce a fundamental systems trade-off: because the evaluation distribution changes across iterations, longitudinal

[42] **Dynabench**: Facebook AI Research's 2021 platform for dynamic benchmark generation, where humans craft adversarial inputs that fool current best models. Dynabench addresses the saturation problem, where very high accuracy on static benchmarks may reflect test-set familiarity rather than robust capability, but introduces its own trade-off: dynamic benchmarks are harder to compare across time because the evaluation set changes. Static and dynamic benchmarks serve complementary diagnostic roles.

comparisons across model generations become difficult to calibrate. Static and dynamic benchmarks therefore serve complementary roles: static suites ensure longitudinal reproducibility on standardized tasks, while dynamic benchmarks identify newly emergent error boundaries.

12.11.3 Holistic system-model-data evaluation

Passing system, model, and data benchmarks independently is not enough. A system benchmark can validate hardware performance, a model benchmark can verify that compression preserved quality, and a data benchmark can assess training set representativeness, yet the deployed system can still fail because the three dimensions interact. Real-world AI performance emerges from that interaction, and optimizing one dimension can expose weaknesses in another.

Consider a concrete failure cascade: an engineering team achieves target throughput on MLPerf Inference by deploying an INT8-quantized vision model on a specialized accelerator. System benchmarks pass. However, because quantization scale factors and zero points were calibrated strictly against ImageNet validation samples, deploying the model onto a factory floor reveals severe accuracy degradation. Factory illumination introduces dynamic ranges that exceed the calibration clipping thresholds, truncating feature activations. A targeted model quality benchmark across varied exposure levels would have exposed this numerical sensitivity. Tracing the failure upstream reveals that the training pipeline contained no industrial imagery—a data distribution gap that no amount of accelerator optimization or runtime kernel tuning can resolve.

Cross-dimensional coupling implies that a passing benchmark in one domain can be invalidated by an unmeasured failure in another:

- **System success + model failure**: Hardware delivers promised throughput, but aggressive compression degrades accuracy below deployment thresholds.
- **System success + data failure**: Fast inference executes on benchmark inputs, but training data bias causes severe accuracy disparities across demographic subgroups.
- **Model success + system failure**: The model produces accurate predictions, but tail latency variance under concurrent load violates service-level agreements.
- **Model success + data failure**: Validation yields high accuracy on held-out test sets, but production distribution shift causes silent degradation.

These failure modes map directly onto the tripartite D·A·M taxonomy diagrammed in the Venn intersections of figure 1.4, though benchmark evaluation lenses do not align one-to-one with isolated physical vertices. Data benchmarks evaluate input distribution quality and coverage (D); model benchmarks assess learned algorithmic representations and numerical robustness (A); and system benchmarks expose hardware execution, kernel efficiency, and memory-interconnect traffic (M). Holistic evaluation validates the pairwise and central intersections where these domains meet—ensuring that optimizations across the Part III pipeline (data selection → model compression → hardware acceleration) do not introduce hidden cross-axis regressions.

↩ The D·A·M taxonomy and its pairwise intersections are established in section 1.5.

The D·A·M taxonomy provides a diagnostic framework for systematically identifying which axis limits performance. Table 12.20 formalizes this approach by crossing each D·A·M axis with compute, memory, and I/O constraints.

Table 12.20: D·A·M Bottleneck Diagnostic Matrix: Each cell suggests a constraint to investigate. Capacity limits determine whether the working set fits; bandwidth limits determine how quickly resident work can move. Activity counters alone do not identify the binding resource, and traces or controlled interventions must confirm a diagnosis.

Component	Compute Constraint	Memory Constraint	I/O Constraint
Data	Host preprocessing saturates compute	Capacity: working set exceeds RAM	Storage delivery cannot keep the accelerator fed
Algorithm	Required operations exceed the compute-time budget	Capacity: activations restrict feasible batch size	Exposed gradient synchronization limits step time
Machine	Useful operation rate approaches the compute roof	Bandwidth: traffic approaches the interface limit	Exposed transfers approach the interconnect limit

Applying this matrix organizes hypotheses when system telemetry diverges from theoretical hardware peaks. If an accelerator achieves only 30 percent GPU utilization during training, an

engineer might prematurely blame the model architecture (Algorithm row) and attempt model surgery. Profiling often reveals instead that host-side data workers saturating CPU cores on image augmentation or tokenization cannot feed batches fast enough across PCIe (Data row, Compute Constraint column). The accelerator stalls not from algorithmic inefficiency, but from input pipeline starvation. The matrix guides investigation; traces and controlled interventions establish which physical subsystem is binding.

Validation under controlled laboratory conditions differs from validation in production. A laboratory protocol may fix data, simplify arrivals, or isolate execution to answer a controlled question; other protocols deliberately vary those conditions. In production, all three assumptions can break simultaneously: data drifts, traffic spikes unpredictably, and concurrent workloads compete for shared hardware resources. The final dimension of benchmarking asks whether laboratory results hold under operational conditions.

↪ A production troubleshooting matrix that maps common failure symptoms to a candidate D·A·M axis and the measurement that confirms it is operationalized in section A.8.

12.12 Production Considerations

A candidate release can pass the hardware, model, and data tests yet still lack evidence for the workload it will serve. A service with bursty requests needs load validation, while a camera deployed under changing illumination needs labeled examples from those conditions. Production qualification assembles the measurements into release gates and establishes the baselines against which the live system will be checked.

12.12.1 From laboratory to production

Laboratory benchmarks establish behavior under specified conditions. A release decision must identify which conditions remain valid in the target deployment and test the assumptions that change. The candidate's latency result may come from an isolated warm runtime; its quality result may come from a curated holdout. Neither observation is discarded, but each needs a matched production check before it can support release.

The checks also need different evidence. Timing and memory telemetry can expose overload, while silent predictive degradation requires labels or validated quality proxies; successful execution is insufficient. When labels arrive late, the release protocol must name that delay and the resulting uncertainty. The deployment tests must exercise these concerns together, because capping batches to preserve latency can alter throughput and energy, while changing input shapes can alter both predictions and resource use.

12.12.2 Bridging benchmark to deployment

Before deployment, validate benchmarking conclusions against production-representative conditions. Table 12.21 names the benchmark assumption, the production reality that violates it, and the validation step that closes the gap; the checkpoint that follows turns those rows into release-readiness actions.

Table 12.21: Predeployment Benchmark Checklist: Each row pairs a simplifying benchmark assumption with a production condition that can violate it and the validation step that closes the gap. Each row should be independently verified against representative operational conditions.

Benchmark Assumption	Production Reality	Validation Approach
Uniform request arrival	Bursty traffic patterns	Load test with production trace replay
Clean, preprocessed inputs	Variable quality inputs	Evaluate on production data sample
Warm system state	Cold starts, cache misses	Measure cold-start performance
Isolated execution	Resource contention	Benchmark under realistic system load
Fixed model version	A/B testing, gradual rollout	Establish baseline for comparison

Checkpoint 12.4: Predeployment benchmark checklist

Before deploying a model based on benchmark results:

- ☐ Replay production traces: Use logged request patterns to validate throughput/latency under realistic conditions.
- ☐ Test with production data: Sample recent inputs with appropriate privacy controls and obtain reliable labels to check accuracy; state label delays or the limits of any proxy.
- ☐ Stress test edge cases: Identify worst-case inputs and verify graceful degradation.
- ☐ Establish monitoring baselines: Document expected metric ranges for anomaly detection.
- ☐ Define rollback criteria: Specify quantitative thresholds for reverting to the previous model version.

12.12.3 Production monitoring as continuous benchmarking

Production monitoring extends benchmarking from a one-time gate to a continuous process. The same principles apply (standardized metrics, reproducible measurement, statistical rigor), shifting the objective from offline qualification to continuous operational verification.

Once a model is live, benchmarking becomes a rolling comparison against the baselines just established. The immediate checks stay concrete: whether the input distribution remains close to the benchmark distribution, whether latency and throughput stay inside the measured envelope, and whether model quality moves outside the expected range. Answering those checks requires the same measurement discipline as the offline benchmark, but now the measurements arrive continuously and under live traffic.

Operational serving environments turn this measurement loop into release and recovery machinery: staged rollouts, shadow evaluation (running the new model beside production without serving its outputs), continuous validation, and automated rollback. Benchmarking defines the baseline performance envelope and failure thresholds; production monitoring continuously tests live telemetry against those bounds.

↪ Shadow, canary, and blue-green releases and the rollback tiers that act on these thresholds are operationalized in section 14.5.3.

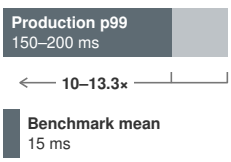
The gap between benchmark conditions and production reality explains why teams continue to fall into predictable traps. A set of recurring fallacies illustrates how sound metrics, misapplied, produce misleading conclusions.

12.13 Fallacies and Pitfalls

Benchmarking creates false confidence when standardized measurement obscures deployment realities. Teams assume controlled evaluations predict production performance, but real systems face variability, resource constraints, and multi-objective trade-offs that benchmarks cannot capture, wasting engineering effort on systems optimized for evaluation rather than deployment.

Fallacy: *Benchmark performance directly translates to real-world application performance.*

The apparent clarity of isolated benchmark numbers leads teams to assume that measuring a model's forward pass predicts user-facing response times. It does not. In research benchmarks, a model's raw inference pass might execute in 5–10 ms. However, when deployed inside a production serving microservice, Amdahl's Law governs end-to-end latency: request deserialization, tokenization, feature retrieval, and network transmission overhead expand the total response time to 50–100 ms. In an illustrative model-only test, the team reports mean latency under synthetic, evenly spaced inputs (15 ms). Under the scenario's production traffic, arrival bursts and queueing backlog raise p99 tail latency to 150–200 ms (10–13.3× gap from the model-only mean). This contrast changes the statistic, measurement boundary, and traffic conditions; quantifying a slowdown requires matched end-to-end statistics under stated workload conditions. In the same illustrative comparison, a model achieving 92 percent benchmark accuracy drops to 78–82 percent when confronted with real-world distribution shifts such as noisy text and uncurated sensor inputs. Evaluating an isolated neural network kernel cannot substitute for benchmarking the complete end-to-end serving pipeline.



In this scenario, the model-only mean understates the production tail by an order of magnitude.

Pitfall: *Optimizing exclusively for benchmark metrics without considering broader system requirements.*

Benchmark leaderboards incentivize aggressive optimization, but the optimizations that climb rankings often degrade the very characteristics production demands. This exemplifies Goodhart's Law (section 12.1), whose organizational form section 12.10.4 examined. When benchmark scores become optimization targets, they cease to be meaningful measures of system quality. In one illustrative scenario, a team reduces inference latency from 12 ms to 8 ms through aggressive quantization, improving MLPerf ranking by 15 positions while degrading calibration such that prediction confidence scores become unreliable for downstream decision-making. Another team improves ImageNet accuracy by 2.1 percent through extensive hyperparameter tuning but the optimized model consumes 40 percent more energy and exhibits 25 percent worse performance on out-of-distribution images from production cameras. Organizations rewarding benchmark rankings over deployment success systematically produce systems that excel in evaluation but fail in production.

Fallacy: *Single-metric evaluation provides sufficient insight into system performance.*

A single-number claim appears deceptively simple ("94 percent accurate" or "1,200 QPS fast"), yet production success requires balancing multiple competing objectives that any single metric obscures. Modern inference systems demand evaluation across accuracy, latency, throughput, energy, and robustness dimensions (section 12.8.2). In an illustrative trade-off, a recommendation model achieving 94 percent accuracy with 180 ms p99 latency fails service-level objectives requiring p99 < 100 ms despite excellent accuracy. Conversely, a system optimized for 1,200 QPS throughput achieves this rate while consuming 4.2 W vs. 1.8 W for a slightly slower system at 1,000 QPS (2.3× power difference). For battery-powered edge devices, the 17 percent throughput loss enables 2.3× longer operation time. Different stakeholders prioritize different metrics: ML engineers focus on accuracy, infrastructure teams on throughput and cost, product managers on latency percentiles. Single-metric optimization systematically produces systems that excel on one dimension while failing deployment requirements on others.

Pitfall: *Using outdated benchmarks that no longer reflect deployment challenges and requirements.*

Benchmarks have inertia: teams continue reporting on established benchmarks after the results cease to provide useful discrimination. Saturation occurs when multiple approaches achieve near-identical performance, eliminating useful comparison. ImageNet top-5 classification error decreased from 28.2 percent in 2010 (Russakovsky et al. 2015) to 3.57 percent by 2015 (He et al. 2016a), sharply compressing the headroom measured by that metric. As gaps narrow, the statistical confidence intervals discussed in section 12.10.1 and the result's deployment relevance matter more than leaderboard rank. Changing deployment contexts compound the problem: benchmarks designed for server hardware become misleading for edge devices with 10× less memory and 100× lower power budgets. Effective benchmarking requires retiring saturated benchmarks and developing evaluation frameworks matching target deployment realities.

Fallacy: *Research benchmarks predict production behavior under real traffic.*

A short, isolated laboratory run can miss the thermal and architectural constraints of sustained production execution. First, laboratory runs frequently suffer from initialization errors: omitting warmup runs conflates one-time JIT compilation, CUDA context initialization, and memory allocator page faults with steady-state execution. Second, short laboratory bursts (e.g., 30 seconds) capture transient boost clock frequencies. Under sustained production load, heat accumulates faster than cooling infrastructure can dissipate it; the hardware governor triggers dynamic voltage and frequency scaling (DVFS), downclocking silicon to protect junction temperatures. As a result, a system achieving 800 QPS throughput in short burst benchmarks sustains only 400–500 QPS under continuous load with 90 percent utilization (37.5–50 percent degradation). Finally, production systems must maintain high availability—99.9 percent availability allows only 43 minutes of downtime per month—demanding evaluation under thermal equilibrium, memory fragmentation, and fault-recovery conditions that short laboratory runs completely ignore.

Pitfall: *Using research benchmarks as production release gates.*

Teams sometimes promote a model because it passes the research benchmark, then discover only after launch that the benchmark never exercised the operational path. A release gate for a serving system must include load tests, tail-latency measurements, data-quality checks, failure drills, and rollback criteria. Research benchmarks remain useful for comparing algorithms, but production gates must measure the deployed system under the traffic, hardware, and failure conditions it will actually face.

12.14 Summary

Benchmarking completes Part III's optimization pipeline by validating whether the efficiency gains from data selection, model compression, and hardware acceleration deliver in practice. The three benchmark lenses examine system execution, model behavior, and data representativeness separately, then test whether their assumptions survive when the complete system runs under deployment conditions.

The lenses reveal different failure modes. System benchmarks expose underdelivered throughput, tail latency, and thermal behavior. Model benchmarks test accuracy, calibration, robustness, and other properties that an optimization may alter. Data benchmarks examine coverage, label quality, leakage, and alignment with deployment. Standardized suites such as MLPerf Training and Inference provide comparable system evidence; model and data protocols supply the quality and representativeness evidence that a hardware result cannot establish.

Grounding optimization claims requires measuring physical execution rather than relying on analytical proxies. Wall-clock timing captures DRAM bandwidth saturation, kernel launch serialization, and host-device transfer overhead that FLOP counts omit. Percentile profiling under concurrent load exposes queuing delays and thermal throttling that arithmetic means conceal. Evaluating workloads against production-representative data reveals dynamic shape variations and memory churn that uniform synthetic inputs never trigger.

Key Takeaways: Measuring what matters

- **Benchmarks validate co-design**: System, model, and data benchmarks expose hardware underdelivery, compression quality loss, and distribution mismatch. A system that passes only one axis can still fail when Data, Algorithm, and Machine constraints meet under production load.
- **Proxy numbers need boundaries**: Standardized run rules make comparisons honest, but fixed workloads are still proxies. Batch size, thermal state, input distribution, concurrency, and service-deadline windows decide whether a lab result survives the benchmark-production gap.
- **Granularity trades diagnosis for realism**: Micro-benchmarks isolate kernels, macro-benchmarks expose model-level costs, and end-to-end benchmarks capture user-visible behavior. Effective measurement stacks all three so teams can see both the symptom and the layer that caused it.
- **Tail latency is the benchmark**: Interactive services with percentile-based SLOs can violate their latency target while mean latency remains acceptable. Reporting percentile latency under representative load prevents a benchmark from approving a system whose mean passes while its worst-served requests violate the SLO.
- **Amdahl bounds a fixed serial pipeline**: In the chapter's vision pipeline, unchanged preprocessing at 44.4 percent of latency turns a 5× inference speedup into 1.8× end to end and caps even an infinitely fast model at 2.25×. Benchmark the whole request path before celebrating kernel speedups.
- **Efficiency still needs quality evidence**: INT8 may cut raw weight storage 4×; the simplified component model estimates an energy reduction of about 6.6×, but whole-device measurement, calibration, subgroup robustness, and edge-case behavior decide whether the compressed model is deployable.

Each chapter in this part promised fewer FLOPs, a smaller model, or higher throughput. Benchmarking tests those promises against the complete system. The gap between a claimed improvement and a measured one reveals where Data, Algorithm, and Machine were assembled rather than matched. Amdahl's Law shows why an infinitely fast model can still leave the pipeline bounded by everything outside it, while tail latency shows why an average can pass even as the worst-served requests fail. This is co-design held to account. An ML system is engineered, not asserted, and only measurement on the real workload can tell the two apart.



What's Next: From lab to live

The serving protocols in this chapter specified the arrival process through a load generator. Production supplies its own changing traffic, and the serving system must respond within its declared operating envelope. Part IV leaves the controlled benchmark environment, beginning with chapter 13. Engineering for arrivals no one schedules reorders the priorities that training optimized, putting per-request latency and spare headroom ahead of raw throughput.

IV

DEPLOYMENT AND OPERATIONS

Part IV

Deployment Principles

The code can be correct and the benchmarks excellent, and yet the system may fail. Part III ended with benchmarks whose load generator decides when each request arrives and whose test data stays fixed. Part IV moves from that controlled environment to production, where ML systems face the additional threat of silent decay. A machine learning system can continue to produce outputs that are confident, well-formatted, and wrong as the world drifts away from its training distribution. At deployment, the data environment escapes the engineer's control and can stress the trained algorithm and the serving machine in ways no test set anticipated. Reliability is therefore a continuous control loop of D·A·M co-design rather than a one-time release gate. The principles here define the requirements and diagnostic models for that reliability.

Principle 9: The verification gap

Policy: Statistical verification holds only over a defined deployment population p_{deploy}, task distance d_{task}, tolerance τ_{tol}, and target error ϵ, estimated by a finite-sample procedure at a stated confidence level:

$$\Pr_{(x,y)\sim p_{\text{deploy}}}[d_{\text{task}}(f(x),y) \leq \tau_{\text{tol}}] \geq 1-\epsilon$$

Implication: Deployment is not a one-way transfer; it is a control loop. Because no test suite can cover every possible real-world input, production systems must monitor their own uncertainty and fail gracefully when they drift outside their known performance envelope.

The verification gap means finite evaluation can only estimate behavior on a defined population with uncertainty. Those bounds may erode as production data diverges from the data used to set them.

Principle 10: The statistical drift diagnostic

Fitted model: Divergence between the production and baseline distributions does not by itself determine whether accuracy changes; over a measured deployment range, labeled outcomes can fit a local linear relationship:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

where P_t is the data distribution at time t after deployment, P_0 is the training (baseline) distribution, Accuracy_0 is the model's performance on P_0, $\mathcal{D}(P_t \| P_0)$ is the statistical divergence between the current and baseline distributions, and λ is fitted from labeled outcomes rather than inferred from divergence alone. Consider a credit scoring model trained on 2020 borrower behavior. Two years later, inflation rises, interest rates change, and lending policies shift. The system still produces scores, but the statistical relationship between inputs and outcomes may have changed; labeled outcome monitoring determines whether real accuracy declined while conventional error logs remained quiet. Unlike many traditional software failures, which are often surfaced by crashes, exceptions, or explicit service-health signals, ML systems can fail silently because the environment changes even when the code and infrastructure remain

unchanged. This first-order linearization applies only over the measured range; the relationship is model-dependent and may be nonlinear for large drift.

Implication: Observability must extend beyond system metrics (latency, errors) to combine statistical drift signals with labeled outcomes. A system can remain operational while prediction quality changes, but drift monitoring alone does not establish degradation.

External drift is not the only threat. Even when the world holds still, the serving pipeline itself can diverge from the model validated offline.

Principle 11: The training-serving skew diagnostic

Heuristic: Training and serving code paths can diverge on identical inputs, and that divergence is measurable without labels. For scalar model scores, define training-serving divergence as the expected absolute difference between the two functions over a stated comparison-input distribution:

$$S_{\text{skew}} = \mathbb{E}[|f_{\text{serve}}(x) - f_{\text{train}}(x)|]$$

The exact relationship between S_{skew} and accuracy depends on the loss function, decision boundary geometry, and production distribution. Unexplained divergence weakens the assumption that offline validation estimates production behavior and can cause silent accuracy loss, but S_{skew} is not a universal accuracy-loss equation. This divergence arises from inconsistent preprocessing logic, different library implementations, stale feature values, or environmental state changes between the two code paths.

Implication: Feature consistency is an architectural requirement, not merely a best practice. Feature stores are more than caches; they can reduce skew by centralizing feature definitions and retrieval. Teams still need validation for freshness, point-in-time correctness, preprocessing, model-runtime, and postprocessing parity. Even subtle differences (PIL vs. OpenCV resize, FP64 vs. FP32 normalization) can compound to produce silent accuracy degradation that standard monitoring may not detect.

Time is the nonnegotiable constraint beneath all these reliability concerns. A medical imaging system that detects tumors with 99 percent accuracy but takes 30 seconds per scan may miss its workflow deadline. An autonomous vehicle perception model that classifies obstacles correctly but responds in 200 ms instead of 50 ms may miss its braking deadline. A correct result may have no operational value if it arrives too late. Latency-sensitive deployed models operate under a deadline, and exceeding it can be functionally equivalent to returning no prediction at all.

Principle 12: The latency budget principle

Policy: Under serial request-path accounting, end-to-end latency is the sum of network, preprocessing, inference, postprocessing, and queueing delays, and a deployment meets its deadline only at a stated completion quantile:

$$L_{\text{lat,total}} = L_{\text{lat,network}} + L_{\text{lat,pre}} + L_{\text{lat,infer}} + L_{\text{lat,post}} + L_{\text{lat,queue}}, \qquad \Pr[L_{\text{lat,total}} \leq T_{\text{deadline}}] \geq q$$

Implication: In latency-sensitive serving, the product selects the deadline and the acceptable completion quantile, such as 0.95, 0.99, or 0.999, and throughput is optimized within that constraint. Serving systems must implement tail-latency controls (for example, bounded-delay dynamic batching or hedged requests). Serving systems must be willing to sacrifice overall throughput to meet the latency deadline of the oldest request in the queue.

A system can satisfy its latency SLOs, detect measured distributional shifts, and maintain training-serving consistency while still causing systematic harm. The previous principles address silent

failures in correctness and service quality; this one addresses failures that degrade equity through silent amplification.



Principle 13: The bias feedback model

Fitted model: When outputs influence future inputs, today's disparity becomes an input to the next training cycle, so disparity evolves as a dynamical system rather than staying fixed. Let $\Delta_g(k) \geq 0$ denote the disparity magnitude for group g relative to a specified reference group after k fixed feedback intervals. With no effective intervention and an approximately constant dimensionless feedback factor $\alpha_{\text{fb}} > 0$, disparity may evolve as:

$$\Delta_g(k) \approx \Delta_g(0) \cdot \alpha_{\text{fb}}^k$$

where $\Delta_g(0)$ is the initial disparity magnitude relative to that reference group and α_{fb} is fitted per feedback interval. Consider a loan approval model that denies credit at higher rates to applicants from historically underserved communities. Denied applicants cannot build credit history, which makes future applications weaker, which increases future denial rates. The model's accuracy on its training distribution remains stable, but the population it serves has been reshaped by its own decisions. When $\alpha_{\text{fb}} > 1$ under these assumptions, the feedback loop is self-reinforcing; when $\alpha_{\text{fb}} \leq 1$, the modeled dynamics are stable or damped. Real deployments may also be nonlinear, saturating, or changed by intervention.

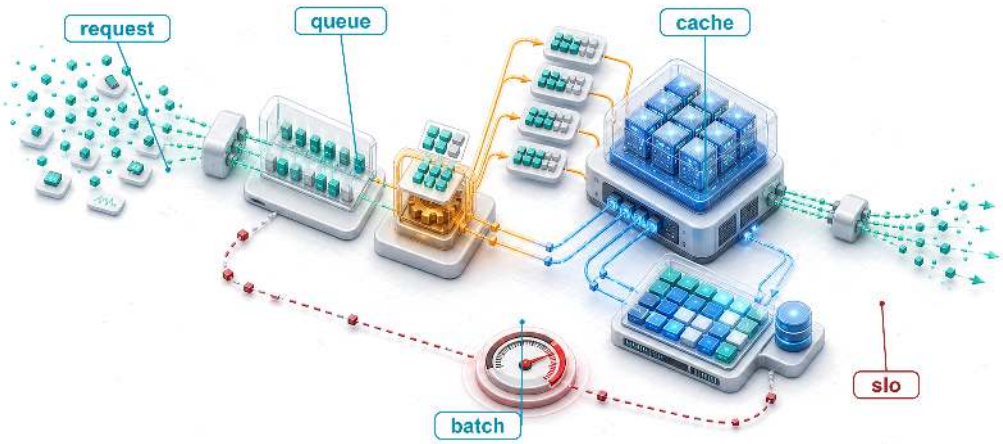
Implication: Fairness is not a postdeployment audit; it is a stability constraint on the deployment control loop. Where consequential impact warrants it and collecting or using group attributes is lawful and justified, systems must monitor disaggregated performance metrics across relevant demographic groups with the same rigor applied to latency percentiles, because a bias regression can be invisible to aggregate accuracy just as a tail-latency violation can be invisible to mean latency.

Together, the five principles define a deployment control loop that establishes what the evidence supports, observes the live system and environment, acts only when a measured failure warrants intervention, and re-evaluates the relevant service, predictive, or subgroup criterion. They separate diagnosis from response. A signal narrows the investigation but neither prescribes an automatic intervention nor proves that quality has already degraded.

Part IV makes that control loop concrete. Chapter 13 builds infrastructure that meets latency budgets under variable traffic and checks training-serving parity on the request path. Chapter 14 detects drift and skew over the model's operating lifetime while preserving the evidence needed for diagnosis and response. Chapter 15 treats fairness and feedback as measurable deployment dynamics rather than a one-time compliance check. Chapter 16 then closes the volume by gathering these production controls and the quantitative bounds and models of the preceding parts into one framework for reasoning across the whole stack.

13

Model Serving



13.1 Serving Paradigm
13.2 Serving Load, Latency, and Architecture
13.3 Serving System Architecture
13.4 Request Lifecycle
13.5 Queuing Theory
13.6 Model Lifecycle Management
13.7 Throughput Optimization
13.8 Inference Runtime Selection
13.9 Node-Level Optimization
13.10 LLM Serving
13.11 Economics and Planning
13.12 Fallacies and Pitfalls
13.13 Summary

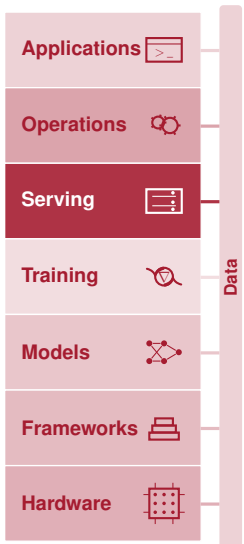
Purpose

Why does serving change the optimization priorities that made training successful?

Training and serving pursue fundamentally opposing operating objectives. Training maximizes aggregate throughput across homogeneous, scheduled batches and extended runs, whereas serving must evaluate individual, heterogeneous requests under strict latency deadlines. Training amortizes hardware initialization and weight streaming across millions of examples; serving pays a distinct tax on every incoming query, where microsecond-level inefficiencies accumulate into operational liabilities. A system processing one billion requests daily converts a single millisecond saved per transaction into more than eleven days of recovered machine time every twenty-four hours.

This operational divergence explains why architectures that train efficiently often falter in production. An online serving system operates within an uncompromising end-to-end latency budget. Delays in network ingress, serialization, input preprocessing, queueing, and response postprocessing directly diminish the execution window allocated to the model itself. When non-inference stages consume the bulk of the budget, aggressive kernel optimizations yield negligible improvement to the user. Furthermore, a prediction that arrives after an interactive client has timed out possesses zero utility, regardless of its mathematical accuracy.

Serving is therefore more than an isolated latency challenge; it is a systems capacity problem under stochastic, uncontrolled traffic. Request arrival rates oscillate between diurnal peaks and troughs, forcing systems to preserve operational headroom for sudden demand surges and hardware failures. Capacity provisioned solely for average load collapses under peak traffic, whereas capacity provisioned for peak demand sits idle and costly throughout off-peak hours. Evaluating an ML model ultimately moves beyond whether it computes correct outputs in isolation to whether it



sustains reliable, low-latency execution under production variability. In D·A·M terms, serving is where the trained algorithm meets unpredictable live data within the machine's strict latency budget, causing all three constraints to bind simultaneously on every incoming request.

Learning Objectives

- Explain serving inversion from training throughput to per-request latency, headroom, and tail behavior
- Decompose request latency across serialization, preprocessing, inference, queuing, post-processing, and network overhead
- Apply queueing laws and simple queue models to plan capacity against percentile latency targets
- Diagnose training-serving skew and cold starts from mismatched preprocessing, model loading, or cache behavior
- Select batching, load shedding, autoscaling, and runtime strategies for traffic patterns and latency budgets
- Evaluate LLM serving bottlenecks using token latency, KV-cache memory, and continuous batching constraints
- Calculate cost per inference from precision, hardware utilization, replica count, and runtime throughput

13.1 Serving Paradigm

In a benchmark run, a load generator decides when each request arrives. Production traffic has no such scheduler. Model **serving** must sustain the operating targets that benchmarking evaluated under specified conditions while live traffic tests that operating envelope continuously, so measurement remains part of the service's lifecycle. In offline training, systems maximize aggregate throughput by packing large batches to amortize weight transfers from high-bandwidth memory (HBM) and saturate matrix arithmetic units. In online serving, requests arrive stochastically from independent clients and must return within strict service-level objectives (SLOs). The primary engineering priority therefore inverts from long-run throughput to per-request latency. This **serving inversion** restructures the balance of terms in the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). When an application cannot delay requests to assemble large batches, it must process small batches or individual inputs ($B = 1$). For weight-dominated operations, small batches reduce arithmetic intensity and can shift execution from compute-bound matrix multiplications toward memory-bandwidth-bound transfers (D_{vol}/BW). Simultaneously, the latency term L_{lat} expands beyond microsecond accelerator kernel launches to include network transit, request serialization, and host orchestration, often becoming the binding constraint on end-to-end response time.

↩ The additive and pipelined formulations of the iron law of ML systems are established in section 1.7.

The D·A·M taxonomy clarifies how this inversion reshapes each layer of the system. In the data layer, the constraint shifts from total dataset volume to the arrival rate, payload freshness, and variable sequence dimensions of live queries. In the algorithm layer, execution becomes strictly forward-only: backpropagation state and optimizer memory vanish from the accelerator, but dynamic input shapes introduce runtime execution variability into previously static operator graphs. In the machine layer, the operating target shifts from maximum hardware utilization to **headroom**. While offline training treats an idle accelerator as wasted capital, online serving treats near-saturation as an operational hazard. Because request arrivals follow stochastic distributions, queue lengths and waiting times diverge nonlinearly as utilization approaches 100 percent. Maintaining adequate headroom preserves the spare capacity needed to absorb transient traffic spikes without violating tail-latency percentiles. Serving therefore optimizes completed requests under a strict latency deadline rather than steady-state hardware occupancy.

Meeting that latency deadline governs how engineering teams partition the end-to-end serving stack. A request's latency budget encompasses three sequential stages: host-side ingress and preprocessing (network termination, payload deserialization, and tokenization or feature transformation), accelerator kernel execution, and downstream postprocessing. Any millisecond consumed by unoptimized feature retrieval or queueing delays directly subtracts from the execution time allocated to the neural network itself. Consequently, the central engineering task in model serving is deciding which operations must remain on the synchronous request path, which tasks can be offloaded to

asynchronous background pipelines, and how much headroom the system must reserve before tail-latency guarantees collapse under load.

13.2 Serving Load, Latency, and Architecture

In production, serving infrastructure rarely experiences the smooth, uniform request streams assumed by synthetic benchmarks. Real-world traffic arrives in bursty, uncoordinated surges governed by external user behavior, scheduled background services, and diurnal cycles. Under steady-state conditions, an inference cluster operates comfortably within its provisioned latency envelope. However, when an unexpected arrival spike exhausts reserved headroom, queuing delays compound catastrophically rather than linearly. Because accelerator execution times cannot compress dynamically to accommodate surplus ingress, excess queries accumulate in host and hardware queues, causing tail latency to explode and triggering cascading client-side timeouts.

Example 13.1: The 'Black Friday' traffic spike

Scenario: An e-commerce recommendation serving system experiences a 10× traffic surge (10,000 QPS) during Black Friday sales events.

Diagnosis: A fleet sized for normal traffic now receives more requests per second than it can complete. Utilization pins at 100 percent, and every second of the surge adds the excess arrivals to a queue that the servers cannot drain, so waiting time grows for as long as the surge lasts. In this scenario, latency reaches 10 s and clients time out. Section 13.5 shows why the queue starts growing sharply well before utilization reaches 100 percent.

Systems lesson: High average throughput does not prevent queueing collapse under traffic surges. Depending on the service, preserving a tail-latency objective may require load shedding, graceful degradation, or autoscaling before utilization reaches the workload's queueing knee.

Figure 13.1 plots this divergence for an idealized single-server queue (the $M/M/1$ model that section 13.5 develops). Its horizontal axis is the serving utilization ρ_{serv}, the fraction of time the server is busy. Latency stays nearly flat at low utilization and then climbs without bound as the server approaches 100 percent busy, because each new arrival is increasingly likely to find work already waiting. The 99th percentile curve rises even faster in absolute terms, staying a fixed multiple above the mean that section 13.5.4 derives. The green zone below 50 percent utilization and the red zone above 70 percent illustrate a planning policy, not universal operating limits. This rapid compounding near saturation is why latency-sensitive systems reserve headroom rather than planning for permanently high accelerator occupancy. Section B.2.1 provides a mathematical treatment of long-tailed distributions and explains why high-percentile latency dictates capacity when an SLO covers the slowest requests.

↪ Why long-tailed latency distributions make high percentiles, rather than the mean, govern what users experience is developed in section B.2.1.

The physical limits of tail latency dictate the technical design of serving systems, but economic constraints dictate their operational scale. As specialized accelerators and runtime optimizations reduce the compute cycles required per forward pass, the marginal cost per inference declines.[1] Counterintuitively, reduced unit cost does not shrink aggregate serving expenditures. In accordance with Jevons paradox, lower marginal costs stimulate disproportionately higher query volumes, integrating model evaluation deeper into user-facing product loops. At hyperscale, recurring operational inference costs quickly eclipse one-time offline training investments, shifting the primary systems bottleneck toward sustained per-query efficiency.

[1] **Jevons paradox**: William Stanley Jevons observed in 1865 that efficiency improvements in coal-powered steam engines increased total coal consumption by making steam power economically viable for applications previously too costly (Jevons 1865); the same dynamic can apply to AI inference: each 10× cost reduction opens application classes that were economically infeasible at the previous price point, expanding aggregate demand by more than the efficiency gain. This is why cheaper inference can increase, not decrease, total GPU fleet demand. Efficiency and demand are often complements in AI, not substitutes.

Example 13.2: The inference tax at Facebook

Scenario: Production ML serving systems at Meta execute tens of trillions of inferences daily across global user-facing services (Hazelwood et al. 2018).

Diagnosis: Recurring inference serving compute and energy costs dwarf initial offline model training expenses. Live request arrival patterns and strict latency bounds constrain GPU batching.

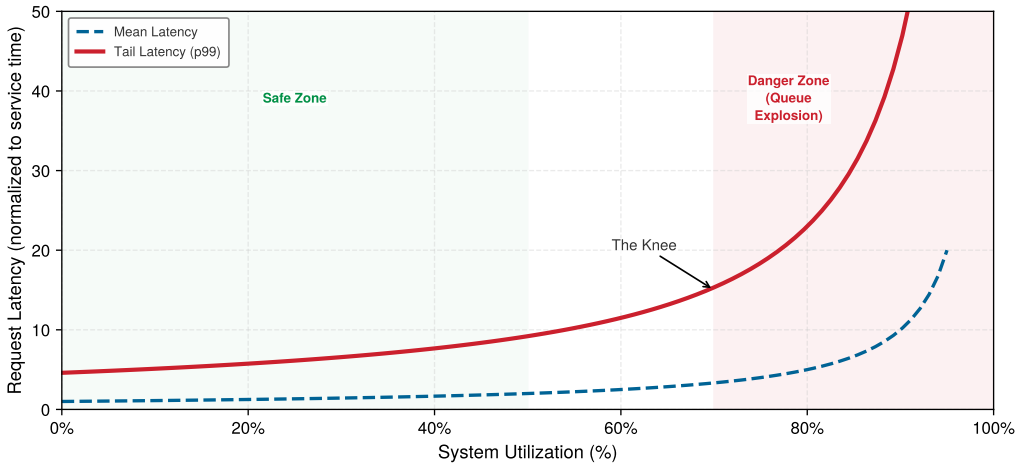


Figure 13.1: The Tail Latency Explosion: Request latency vs. serving utilization ρ_{serv}. Mean latency (blue) and the p99 approximation (red) both grow nonlinearly as utilization approaches saturation; the 70 percent marker is an illustrative planning knee rather than a model-derived breakpoint. In an $M/M/1$ queue, the 99th percentile latency scales as $-\ln(0.01) \approx 4.6\times$ mean latency, illustrating why tail delays compound violently near saturation.

Systems lesson: Model architectures cheap to train can prove unviable to serve at fleet scale. Production serving infrastructure requires hardware-software co-design to balance tail latency, batch size, and recurring operational total cost of ownership.

The same serving-economics pressure appears in public API prices. Figure 13.2 tracks this cost collapse across representative public API list-price snapshots from 2023 through 2025 on a logarithmic scale. The steep downward trend line isolates how prompt token pricing fell by multiple orders of magnitude, from thirty dollars per million tokens for original GPT-4 down to several cents for Gemini 1.5 Flash and DeepSeek-V3. Vendor prices change frequently, so these points should be read as historical provenance for the trend rather than as current purchasing guidance (OpenAI 2023b, 2023a, 2024; OpenAI Developer Community 2024; Anthropic 2024; Google Developers Blog 2024; DeepSeek 2024). Each order-of-magnitude drop changes which applications are economically viable, and a provider can sustain such prices only if its serving stack extracts more completed requests from each accelerator-hour. That conversion from engineering efficiency to unit cost is the subject of section 13.11; the sections before it build the latency, batching, and memory mechanisms that set the conversion rate.

Two pressures now frame the serving problem: tail latency that explodes once utilization passes the queueing knee, and per-inference economics that fall by orders of magnitude as efficiency improves. Together they force a formal definition of serving built around latency rather than throughput.

Definition 13.1: Model serving

Model serving is the operational phase that provides model predictions to end-users or downstream systems under latency, throughput, availability, and cost constraints.

1. **Significance**: Online serving turns training's priority on sustained hardware efficiency (η_{hw}) into a request-latency constraint ($L_{\text{lat,total}}$), requiring a stack designed around the percentile named by its SLO.
2. **Distinction**: Unlike model training, which commonly processes scheduled batches, serving may handle stochastic online requests, synchronized streams, or offline batches.

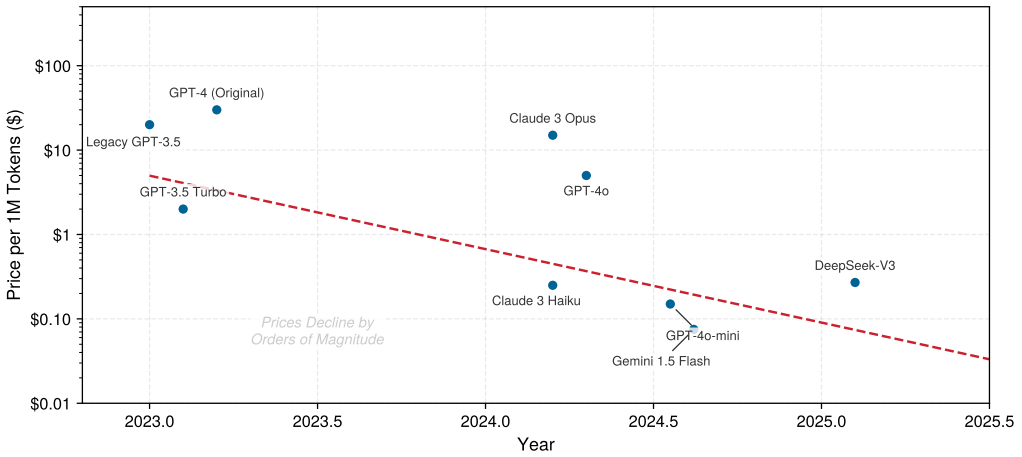


Figure 13.2: Intelligence Deflation: Representative public API input-token list prices per 1M tokens ($) over time (Log Scale). Eight model-version points are snapshots collected from the public pricing pages of OpenAI, Anthropic, Google, and DeepSeek between 2023 and 2025; the Legacy GPT-3.5 point is an illustrative baseline rather than a documented model-version snapshot. The series is intended as a market trend indicator, not a controlled or current-price comparison. API token-processing prices have collapsed by multiple orders of magnitude, transforming the economics of automated AI workflows.

3. **Common pitfall**: A frequent misconception is that serving is "just the forward pass." In reality, it is a systems problem: model execution is only one component of a stack that may include request routing, load balancing, and data transformation.

The SLO[2] defines the latency target that shapes every architectural decision in the serving stack, establishing the strict time budget within which all request handling must complete. When an inference server receives a request, model execution represents only one segment of an end-to-end pipeline: network ingress, payload deserialization, feature transformation, queue scheduling, and output serialization all consume time before and after the accelerator runs. Figure 13.3 diagrams this end-to-end request lifecycle, partitioning execution into three sequential stages: input preprocessing (traditional computing), neural network evaluation (deep learning), and output postprocessing with safety validation (traditional computing). Any of these stages can become the binding latency bottleneck. Section 13.4.1 quantifies exactly where time goes, revealing a counterintuitive result about which stages dominate.

[2] **Service level objective (SLO) vs. service level agreement (SLA)**: An SLO is an *internal* target (for example, "p99 latency under 50 ms") and an SLA is an *external* contractual commitment whose violation may carry remedies or penalties. Teams often set SLOs tighter than SLAs to preserve a safety margin. For ML serving, task quality and inference latency can both contribute to SLOs, creating multidimensional targets where improving one dimension (for example, deploying a larger model for accuracy) can violate another (latency).

↩ SLOs, service level agreements (SLAs), and the tail-latency percentiles they constrain are established in section 12.8.2.1.

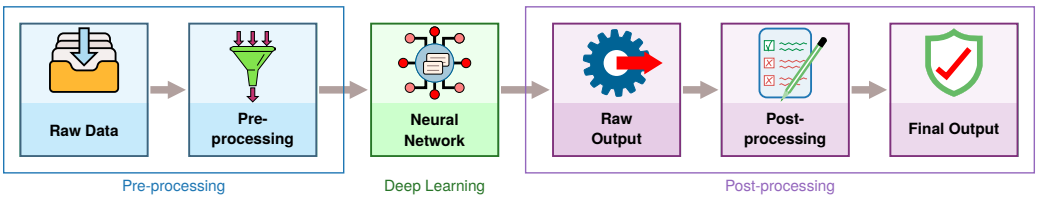


Figure 13.3: The Inference Pipeline: ML serving systems transform raw inputs into final outputs through sequential stages: preprocessing, neural network computation, and postprocessing. The neural network represents just one component; preprocessing and postprocessing make end-to-end latency depend on more than neural-network execution.

This pipeline turns serving into an orchestration problem: because every stage draws from the same latency budget, optimizing accelerator kernel execution yields diminishing returns if data transformation or queueing delays dominate. Before optimizing any individual stage, the system must decide whether predictions are computed ahead of time or on demand.

13.2.1 Static vs. dynamic inference

Deciding whether predictions are generated before or during a user request establishes the fundamental trade-off between offline hardware efficiency and online operational agility (Google 2024b). That choice determines whether an accelerator operates in an unconstrained throughput regime or under the strict tail-latency deadlines of an interactive serving loop.

13.2.1.1 Static inference

Static inference (also called offline or batch inference) precomputes predictions for anticipated inputs and stores them in a retrieval store. Moving model execution entirely out of the user request path alters the underlying hardware constraints: because batch jobs have no millisecond-level deadlines, accelerators can use large batches ($B \gg 1$) selected for throughput within memory limits. Reusing weights across samples raises arithmetic intensity for weight-dominated operations; a batch-size sweep identifies where throughput stops improving. When a client issues a request (for example, a user opening nightly generated recommendation rankings), the serving system performs a low-latency key-value lookup rather than executing a forward pass on an accelerator. The primary operational constraint shifts from accelerator latency to storage capacity and data freshness: precomputing predictions across an entire input domain scales combinatorially ($\mathcal{O}(N_{\text{users}} \times N_{\text{items}})$ for N_{users} users and N_{items} items in recommendation systems), and stale predictions persist until the next batch refresh.

13.2.1.2 Dynamic inference

Dynamic inference (also called online or real-time inference) evaluates inputs on demand as requests arrive. On-demand execution accommodates arbitrary, previously unseen inputs within the model schema and immediately reflects newly deployed model weights without requiring precomputation. However, executing the model directly within the user request path subjects inference to the strict end-to-end latency budget set by its SLO. Because user requests arrive stochastically, the serving runtime cannot delay execution indefinitely to accumulate large batches. Serving requests individually or in small batches ($B = 1$ to $B = 4$) can expose weight traffic or launch overhead that larger batches amortize. When these costs bind execution, the device completes fewer queries per unit time, increasing provisioned cost per query. This tension between interactive response time and hardware efficiency can make dynamic serving more expensive per query than offline batching (example 13.3).

Example 13.3: The cost of latency

Latency constraints directly dictate infrastructure costs. The figures below are round illustrative numbers; section 13.7.1 later measures the full batch-size sweep for one model. Consider a GPU server renting for $4/hour.

Scenario A (low latency): Batch size 1.

- Latency: 5 ms.
- Throughput: 200 req/s.
- Cost per million queries: $5.56.

Scenario B (high throughput): Batch size 8.

- Latency: 10 ms (doubled due to batching overhead).
- Throughput: 800 req/s (quadrupled due to parallel efficiency).
- Cost per million queries: $1.39.

Systems insight: Reducing latency from 10 ms to 5 ms increases the hardware bill by 300 percent. Engineers must quantify whether that speedup generates enough business value to justify the 4× cost increase.

Consider two contrasting deployment scenarios for a ResNet-50 image classifier. A static batch approach suits a photo management application that preclassifies every image in a user library overnight. With 10,000 photos requiring 5 ms of inference time each, batched execution completes in

approximately 50 s, allowing clients to experience instantaneous retrieval upon browsing. Conversely, a dynamic online approach suits a content moderation service that must evaluate user-uploaded media in real time, where each image traverses the complete preprocessing, accelerator execution, and postprocessing pipeline within a 100 ms service-level objective.

These two modes optimize for fundamentally different operating regimes. Static inference maximizes accelerator utilization and amortizes memory transfers across large, offline batches, trading persistent storage capacity for zero serving latency at query time. Dynamic inference minimizes tail latency under stochastic, concurrent traffic, accepting lower compute efficiency and higher marginal cost per query to evaluate arbitrary, unpredictable inputs. Production architectures frequently synthesize both paradigms into a tiered design: static precomputation populates key-value caches with predictions for high-frequency queries, such as catalog items or viral media, while dynamic serving clusters evaluate the long tail of novel or personalized requests.

Deciding when predictions are computed resolves only the temporal dimension of serving. Equally binding is the spatial dimension, where the model physically executes, because physical placement dictates memory hierarchy limits, network boundaries, and the hardware budget available to the serving stack.

13.2.2 The spectrum of serving architectures

Although "serving" often implies a networked server processing API requests, the architectural pattern varies by deployment environment. The four deployment paradigms (Cloud, Edge, Mobile, and TinyML) and the physical constraints (the light barrier, the power wall, and the memory wall) that give rise to them shape the serving stack. Architecturally, serving spans a continuum from **centralized networked microservices** (which pool accelerators to optimize throughput and cost-per-query at the expense of network round-trip overhead and cold-start latency) to **application-embedded edge serving** (which run models inside client processes via direct function calls, eliminating network latency and preserving privacy at the expense of strict device memory and power constraints) and out to **bare-metal firmware execution** on microcontrollers. Those constraints do not disappear at serving time; serving adds latency SLOs and cost pressure on top of hardware limits that training could absorb through patience. The same model may require different serving strategies depending on where it executes.

↩ The light barrier, power wall, and memory wall that give rise to the four deployment paradigms are established in section 2.2.

13.2.2.1 Networked serving (cloud/data center)

In networked serving, the model runs as a standalone service (microservice), matching the cloud deployment paradigm that trades latency for larger pooled compute. The primary interface is the network through protocols such as HTTP or gRPC, so network transfer and serialization join model execution as possible binding constraints. Data-center hardware such as NVIDIA GPUs (V100, A100, H100), Google Tensor Processing Units (TPUs), and AWS Inferentia supports high-throughput batching and concurrency, but cold start can still stretch from seconds to minutes because container startup, model loading, and warmup sit outside the steady-state inference path.

13.2.2.2 Application-embedded serving (mobile/edge)

In application-embedded serving, the model runs within the user application process (for example, a smartphone app using CoreML or TensorFlow Lite), following the embedded paradigm, which is chosen for its latency, privacy, and offline advantages. There is no "server." The interface is a function call, so optimization focuses on energy and responsiveness (the MLPerf SingleStream shape) rather than shared-server throughput.

↩ The MLPerf SingleStream, MultiStream, Server, and Offline scenarios are established in section 12.8.4.2.

Embedded execution enables **zero-copy inference**: when data moves through a system, each copy consumes CPU cycles and memory bandwidth. In cloud serving, separate physical memory spaces force repeated transfers: a camera frame passes from a network socket buffer to user space, through a preprocessing buffer into page-locked host DRAM, and across PCIe into accelerator memory. A mobile pipeline can eliminate these intermediate copies when its camera, image signal processor (ISP), CPU, and NPU exchange shared buffer descriptors in unified system memory. In unified-memory system-on-chip (SoC) architectures, such as Apple A- and M-series processors or Qualcomm Snapdragon, all compute engines share a single physical DRAM pool. When software passes buffer handles rather than duplicating bytes, inference latency drops and memory bus power decreases, although format conversions may still require CPU or ISP intervention.

Typical hardware includes mobile NPUs (Apple Neural Engine, Qualcomm Hexagon) and embedded GPUs (NVIDIA Jetson). When the model is already resident and compiled, startup can be measured in milliseconds; first inference takes longer if it triggers cold-page faults or just-in-time compilation. Mobile power and thermal envelopes are strictly constrained by passive cooling (1–10 W); sustained high-throughput inference saturates the device chassis thermal capacitance, forcing the operating system's thermal governor to throttle clock frequencies and degrade inference latency.

13.2.2.3 Bare-metal serving (TinyML)

In TinyML serving, the model is compiled directly into microcontroller firmware, reaching the extreme end of the deployment spectrum: ubiquitous sensing at microwatt power budgets. Many deployments lack a general-purpose operating system or dynamic memory allocator; serving executes as a tight bare-metal loop that samples sensors and invokes a static interpreter. Because static random-access memory (SRAM) is constrained to hundreds of kilobytes, optimization focuses on executing model weights in place from non-volatile flash memory while fitting dynamic activation buffers into a preallocated Tensor Arena. Request batching is infeasible on single-stream sensor feeds. Typical hardware includes ARM Cortex-M series, ESP32, and specialized TinyML accelerators. Startup latency is measured in microseconds when weights remain in memory-mapped flash and execution buffers are preallocated statically, operating within a microwatt-to-milliwatt power envelope that sustains battery life across months or years.

The deployment tiers differ first in their operating envelopes. Mobile and edge share a column in table 13.1 because, from the serving stack's point of view, both embed the model in the application process behind a function call; the hardware differences between a phone and an embedded board appear as the width of each range rather than as a different serving architecture. Table 13.1 compares the latency, batch, memory, power, update, failure, and monitoring constraints that follow from each environment.

Table 13.1: Serving Architecture Spectrum: Representative deployment envelopes for cloud, mobile, and TinyML serving. The latency, batch-size, memory, and power ranges are design guides rather than universal limits; update, failure, and monitoring rows summarize the operating pattern associated with each environment.

Characteristic	Cloud/Data center	Mobile/Edge	TinyML
Latency Target	10–100 ms	20–50 ms	1–100 ms
Batch Size	1–128 (dynamic)	1 (fixed)	1 (fixed)
Memory	16–80 GB VRAM	2–8 GB shared	256 KB–2 MB SRAM
Power	300–700 W	1–10 W	1–100 mW
Update Mechanism	Container deploy	App store update	Firmware over-the-air (OTA)
Failure Mode	Retry/failover	Graceful degradation	Silent or reset
Monitoring	Full telemetry	Limited analytics	Heartbeat only

Applying those envelopes to one image-classification workload, table 13.2 shows why cloud and mobile can adapt ResNet-50 while TinyML must substitute MobileNetV2. The cloud column runs the model through TensorRT, NVIDIA's ahead-of-time inference compiler, and the mobile column through the TensorFlow Lite runtime; the chapter uses these engines as given until section 13.8 examines what such runtimes do to a model.

Table 13.2: Image Classification Across the Serving Spectrum: Illustrative side-by-side comparison of cloud, mobile, and TinyML serving for the same image-classification task, showing how model format, latency, throughput, memory footprint, and energy budget differ across deployment contexts.

Dimension	Cloud	Mobile	TinyML
Model format	TensorRT FP16 engine (51.2 MB)	TensorFlow Lite INT8 (25.6 MB)	Not feasible (25.6 MB); alternative: MobileNetV2 INT8 (3.5 MB)
Inference (batch-1)	1.4 ms (batch-16: 14 ms)	12 ms (NPU), 45 ms (CPU)	120 ms
Throughput	1,143 img/s (batched)	~80 img/s (single-stream)	~8 img/s

Dimension	Cloud	Mobile	TinyML
Memory	2 GB VRAM (batch-32)	150 MB peak (shared with app)	320 KB arena (fits in 512 KB SRAM)
Energy/inference	—	0.8 mJ (NPU), 4.2 mJ (CPU)	12 mJ

Systems Perspective 13.1: ResNet-50 across the serving spectrum

Systems insight: It is tempting to read table 13.2 as one model deployed three ways. That reading is misleading: the cloud and mobile tiers share the ResNet-50 graph but use different precision and runtimes, while the TinyML tier cannot run ResNet-50 at all and substitutes an architecture designed for its constraints. Treating these as one model hides the work that makes each deployment possible.

13.2.3 The load balancer layer

When traffic exceeds what a single machine can handle, cloud and data center deployments that run multiple replicas of the same model require an additional infrastructure layer: the load balancer. Production serving systems place load balancers between clients and model servers.

A serving load balancer performs three functions. First, request distribution routes incoming traffic to available model replicas using algorithms such as round-robin or least-connections; routing away from slow or saturated replicas directly curbs tail latency. Second, health monitoring verifies replica readiness by inspecting both process liveness and model initialization state, confirming that weights are loaded and warmup passes are complete before admitting traffic. Third, deployment support enables progressive rollouts by shifting traffic gradually between model versions rather than cutting over abruptly.

↪ Shadow, canary, and blue-green release strategies that build on this traffic shifting are developed in section 14.5.3.

On a single machine running several model instances, such as several Open Neural Network Exchange (ONNX) Runtime sessions, the serving framework and the operating system queue requests without a separate balancer; a dedicated balancer becomes necessary once replicas span machines, and its distribution algorithms belong to that distributed setting. This chapter therefore plans capacity one node at a time. The unit of capacity is the sustained model throughput of one node, and the queueing analysis in section 13.5 determines the utilization at which that node can no longer meet its latency target and a second replica must be added behind the balancer.

Whether a deployment runs one replica or many, each replica must turn an irregular stream of requests into work its accelerator can execute efficiently. That conversion is the job of the inference server's internal architecture.

13.3 Serving System Architecture

Inside each replica, production serving confronts an impedance mismatch at the system boundary. User requests arrive stochastically, bursting milliseconds apart and then falling silent for seconds, whereas accelerator compute engines demand steady streams of uniform, dense batches to saturate memory bandwidth and functional units. Bridging this gap requires a specialized serving architecture that decouples network ingress from execution scheduling, absorbing arrival bursts and packing irregular inputs into dense batches without violating latency SLOs.

13.3.1 Internal architecture and request flow

An inference server[3] (such as NVIDIA Triton or TensorFlow Serving) functions as a specialized runtime scheduler that manages concurrency, device memory, and data movement between host CPUs and accelerator devices. Rather than invoking model execution synchronously upon request arrival, the server pipelines incoming requests through dedicated subsystems that isolate client network handling from accelerator compute. Every request traverses a multi-stage runtime pipeline designed to maximize hardware throughput while bounding queuing latency. Figure 13.4 diagrams the six stages of this pipeline, tracing how incoming client queries traverse network ingress and

[3] **Inference server**: Google's TensorFlow Serving (Olston et al. 2017) helped establish the separation of model logic from serving infrastructure; NVIDIA's Triton (NVIDIA 2024d) extends this pattern across multiple model frameworks and backends. The critical design insight is that a scheduler and dynamic batcher turn irregular single-request traffic into accelerator-friendly execution, improving utilization when latency budgets allow batching. Exact utilization gains depend on model, hardware, arrival rate, and the configured batching window.

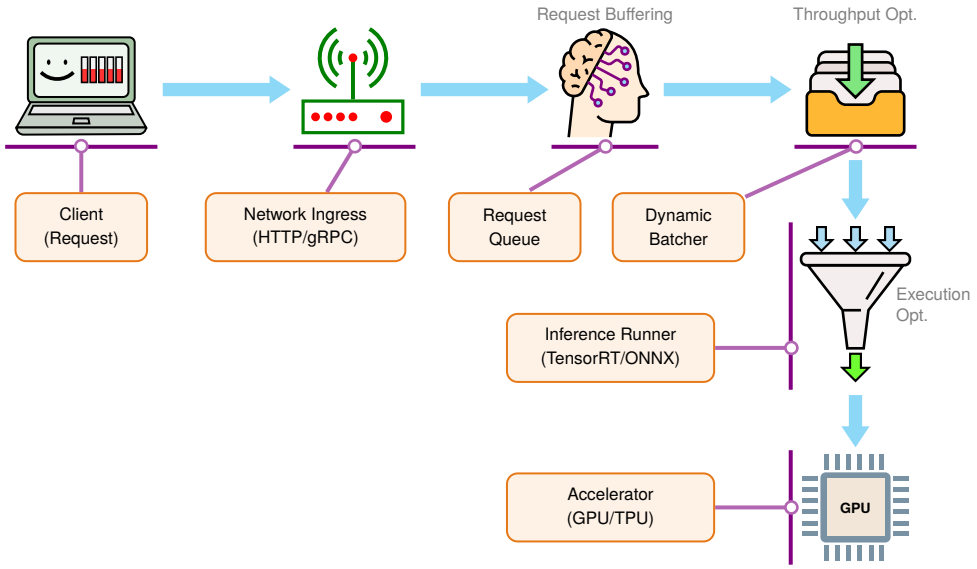


Figure 13.4: Inference Server Anatomy: A modern inference server decouples network handling from accelerator execution through a staged pipeline. Each stage isolates a concern, from absorbing bursty traffic to forming efficient batches, so the hardware accelerator stays highly utilized despite irregular arrival patterns.

request buffering queues before the dynamic batcher stages them downward to the inference runner and the hardware accelerator.

This architecture orchestrates four primary responsibilities: concurrency management, request transformation, model lifecycle management, and execution scheduling.

First, the server manages concurrency. Serving engines employ asynchronous event loops (such as `epoll` or `kqueue`) and worker thread pools to handle thousands of concurrent client TCP connections without stalling accelerator execution pipelines. Decoupling network I/O from compute ensures that slow client transfers or network stalls do not starve the accelerator of ready work.

Second, the server executes request transformation. It decodes serialized network payloads into contiguous, aligned memory buffers matching the tensor layout required by the underlying execution engine. For example, four-dimensional vision tensors typically adopt NCHW[4] or NHWC formats. On modern accelerators, matrix multiplication units (such as Tensor Cores) load 8- or 16-element channel vectors using coalesced memory transactions that favor channels-last (NHWC) layout. If the execution kernel requires a different physical layout, the server must materialize that layout. Calling `torch.permute` merely creates a view with reordered strides; an explicit memory copy or transpose kernel must physically reorder the data in memory. Because such materialization performs zero arithmetic ($I \approx 0$ FLOP/byte), memory traffic and launch overhead dominate its cost. A device-local conversion consumes device memory bandwidth, whereas conversions across the host-device boundary consume PCIe bandwidth.

Third, the server governs model lifecycle management. The runtime establishes the CUDA context before allocating device memory (VRAM) and streaming weights onto the accelerator. It pins host memory buffers for direct memory access (DMA) transfers and executes representative warmup passes to trigger deferred compilation and memory allocation before admitting live client traffic. While enterprise model registries, canary rollouts, and rollback policies belong to cluster-level operations (chapter 14), the serving engine governs local state: validating that active model instances are warmed, resident in accelerator memory, and ready for dispatch.

Fourth, the execution **scheduler** coordinates the transition between request buffering and accelerator execution by implementing dynamic batching (section 13.7). For weight-dominated inference at batch size $B = 1$, streaming parameters from HBM can limit execution before the arithmetic units reach peak throughput. Grouping B requests into a concurrent batch amortizes the byte transfers of reading model weights across all B inputs, increasing the arithmetic intensity ($I = O/D_{\text{vol}}$).

[4] **NCHW and NHWC (tensor memory layouts)**: These acronyms encode the dimension order of 4D image tensors: N (batch), C (channels), H (height), and W (width). In a contiguous NCHW tensor, each channel plane is contiguous; in NHWC, channels for one spatial position are adjacent. Optimized kernels may favor either layout, and a semantic mismatch can scramble values even when the element count is correct. Serving code should therefore make both dimension meaning and any physical-layout conversion explicit.

To balance individual request latency against batch compute efficiency, the scheduler tunes queue residency using a **batching window** (or timeout threshold). Setting the batching window to zero removes deliberate batch formation delay, but requests can still wait for an available worker. An immediately dispatched small batch may also expose too little work to use the accelerator efficiently. Conversely, setting a bounded window (such as 2 to 5 ms) allows the scheduler to buffer arriving requests until either the queue reaches a target batch size or the timeout expires. The window bounds deliberate formation delay; waiting for a busy worker is a separate resource-queue delay. Batching trades that formation time for greater weight reuse and potentially higher service capacity, while total latency includes both waits and execution.

13.3.2 Interface protocols and serialization

As accelerator kernels undergo aggressive compilation (via TensorRT, TorchDynamo, or custom kernels), device execution time often drops below a single millisecond, and the host-side cost of ingesting, transporting, and deserializing payloads becomes the dominant term in end-to-end latency (the serving tax itemized in section 13.4.2.1).

13.3.2.1 The serialization bottleneck

Machine learning payloads differ fundamentally from conventional web API payloads: they consist of multi-dimensional arrays of floating-point numbers (such as image tensors, dense embedding vectors, or token sequence logits). While text-based JSON is ubiquitous in web services, serializing dense numerical arrays into text incurs severe CPU and memory penalties. Serialization overhead manifests because JSON parsers must scan every byte sequentially, validate syntax, allocate heap nodes, and invoke string-to-float conversion routines (such as `strtod`). Each 32-bit float expands from 4 raw bytes into 7 to 15 ASCII characters including punctuation. Furthermore, raw binary data (such as compressed JPEG or raw image bytes) requires Base64 encoding to travel within JSON strings, imposing a 33 percent size inflation before parsing begins.

Binary serialization formats, such as Protocol Buffers[5] and FlatBuffers,[6] bypass this text translation tax. Float arrays transmit directly as packed IEEE 754 byte streams with zero ASCII conversion and zero Base64 framing. Furthermore, FlatBuffers organizes serialized records with internal relative offsets, enabling zero-copy deserialization: the serving application reads tensor dimensions, data types, and byte offsets directly out of the network receive buffer without allocating an intermediate object graph on the heap.

13.3.2.2 REST vs. gRPC

Transport protocol selection governs connection life cycles and socket multiplexing. Conventional REST APIs typically deploy HTTP/1.1 with JSON payloads. HTTP/1.1 clients commonly use multiple persistent TCP connections for concurrency. Establishing each connection incurs handshake and, for HTTPS, TLS costs, but keep-alive reuse amortizes setup across requests. HTTP/1.1 also suffers from request-level head-of-line blocking, because responses on a single connection must return in request order. While REST APIs can adopt HTTP/2 or binary encodings, the combination of HTTP/1.1 and JSON creates significant socket overhead and CPU parsing contention when serving high-throughput tensor workloads.

In contrast, gRPC (gRPC Remote Procedure Call)[7] pairs HTTP/2 transport with Protocol Buffers serialization. HTTP/2 establishes a single long-lived TCP connection and multiplexes hundreds of concurrent request and response streams simultaneously using binary framing. This architecture multiplexes concurrent streams over a persistent connection and removes HTTP-level response-order blocking (a lost TCP packet can still stall every stream on the connection, which HTTP/3 addresses by moving to QUIC), while Protobuf enforces strongly typed contracts and serializes binary tensor payloads efficiently. Consequently, gRPC is the standard protocol for internal microservice communication in model serving clusters, where low latency, compact wire payloads, and bidirectional token streaming are primary design criteria.

A concrete payload comparison quantifies how the serialization format alters wire footprint and host CPU parsing overhead.

↩ Clipper's decomposition of serving latency, in which serialization rivals model execution, is established in section 12.8.2.2.

[5] **Protocol Buffers (Protobuf)**: Protobuf uses a predefined schema (from a `.proto` file) to encode structured data into a compact binary format (Protocol Buffers Authors 2026). Because the schema carries field names and types, the wire payload need not repeat them as JSON does. Its wire format is still not identical to a C++ object's in-memory layout, so it requires a parsing step and does not provide the same direct zero-copy access pattern that FlatBuffers targets.

[6] **FlatBuffers**: The "flat" in the name describes the design: the binary buffer can serve as the serialized representation and the data structure being read, avoiding a separate parsing or unpacking phase for supported access patterns (FlatBuffers Authors 2026). For ML inference, this can enable zero-copy access to tensor metadata—the serving system reads tensor shapes and offsets directly from the buffer rather than allocating a second object representation.

[7] **gRPC (gRPC remote procedure call)**: The gRPC framework pairs HTTP/2 transport with an interface definition language and message format, most commonly Protocol Buffers (gRPC Authors 2026). The relevant serving advantage is the combination of typed contracts, persistent multiplexed connections, streaming support, and compact binary messages. The size and latency benefit over REST/JSON depends on payload shape, client/server implementation, and whether serialization is a meaningful share of the end-to-end latency budget.

Napkin Math 13.1: JSON vs. Protobuf serialization

Consider a request payload containing 1,000 floating point numbers (for example, an embedding vector).

- **JSON**: Uses ~9 KB on the wire. Requires ~50 µs to parse.
- **Protobuf**: Uses ~4 KB on the wire. Requires ~5 µs to parse.

Math: Switching one request to the binary payload saves 50 µs − 5 µs = 45 µs of parse time. For this illustrative system processing 10,000 requests per second, the savings compound to 10,000 × 45 µs = 0.45 s of CPU time reclaimed every wall-clock second, which is 45 percent of one core freed from serialization overhead alone.

Systems insight: This 10× scenario gain makes gRPC/Protobuf, FlatBuffers, or another binary protocol a strong candidate for high-throughput internal microservices when serialization is a visible part of the latency budget.

Architectural selection is governed by system constraints. REST over HTTP/1.1 remains practical for public-facing edge gateways where broad client compatibility and human-readable debugging take precedence over raw wire efficiency. Within cluster boundaries, gRPC over HTTP/2 with binary serialization can reduce overhead when high query volume or tensor serialization consumes a significant share of the host CPU budget; its streaming support also suits incremental token responses.

Protocol selection and process architecture establish the structural skeleton of an inference server. Evaluating actual operational performance, however, requires tracing how latency accumulates as individual requests traverse each stage of this runtime pipeline.

13.4 Request Lifecycle

A single HTTP request carrying a 224×224 JPEG image arrives at an inference server. Between the moment the first byte enters the network stack and the moment the classification payload departs, the request traverses a sequence of pipeline stages, grouped below into preprocessing, inference, and postprocessing, with data transfer between them. Each stage consumes a slice of the end-to-end latency budget, making granular phase decomposition essential for targeted performance tuning.

13.4.1 The latency budget

For dynamic inference systems, the serving inversion established in section 13.1 creates a latency budget that shapes system design (Gujarati et al. 2020). A serving system with second-scale per-request latency may miss many interactive SLOs, even if it achieves excellent throughput.

Relevant metrics expand from aggregate throughput and mean latency to the latency distribution. Mean and p50 describe the center, while p95 and p99 expose the slow tail. If mean latency is 50 ms but p99 is two seconds, the p99 threshold is 40× the mean, so about 1 percent of requests are at or beyond two seconds. Whether that tail matters depends on the service and its users, which is why percentile targets should come from the SLO rather than convention.[8]

Managing these percentile constraints requires decomposing the total allowed response time into a **latency budget** that allocates time across each processing phase, the serial accounting that the latency budget principle (12) states formally.

Definition 13.2: Latency budget

Latency budget is the time allocated to request stages under an end-to-end deadline; the SLO also specifies the required completion quantile.

1. **Significance**: It acts as a zero-sum constraint system where any milliseconds consumed by serialization or network overhead directly reduce the latency budget ($L_{\text{lat,total}}$) available for model inference.

[8] **Tail latency**: Unlike averages, percentile latencies reveal the performance impact of system outliers common in ML serving, such as model cache misses or garbage collection pauses. These rare, high-latency requests can disproportionately harm user satisfaction and business outcomes. Because the magnitude is workload-specific, services validate percentile targets (p95, p99) against their own users and SLOs.

2. **Distinction**: A latency **SLO** couples a deadline (for example, < 50 ms) with a completion quantile (p99 or p99.9). A service-level indicator (**SLI**) is the measured quantity used to evaluate that objective. Missing an internal SLO consumes reliability margin; crossing an external **SLA** threshold may trigger the remedies defined by the contract. Tail latency spikes can stem from queueing, batch formation, cache state, interference, or data movement.
3. **Common pitfall**: A frequent misconception is that the "model" has the entire budget. In the illustrative breakdown in table 13.3, the remainder is consumed by host preprocessing and data movement (JPEG decode, resize, normalization, and PCIe transfer), while upstream network and serialization taxes (table 13.5) consume further margin.

Every serving request decomposes into three phases that each consume part of the latency budget. Preprocessing transforms raw input such as image bytes or text strings into model-ready tensors. Inference executes the model computation. Postprocessing transforms model outputs into user-facing responses.



Inference is one slice of the latency budget; preprocessing rivals it.

Accelerating model execution alone yields diminishing returns when host-side preprocessing and response formatting consume the majority of the latency budget—an immediate manifestation of Amdahl's law. As hardware compilers compress accelerator kernel runtimes into sub-millisecond durations, host-device coordination and memory transfers emerge as the primary latency bottlenecks.

13.4.2 Latency distribution analysis

Dissecting where latency accumulates requires independent instrumentation across each pipeline boundary. In visual inference, for example, aggregate request duration conflates CPU-bound image decompression, host-to-device PCIe transfers, tensor execution across accelerator cores, and classification score deserialization. Isolating these distinct phases reveals the true bottleneck, preventing misdirected optimization efforts on kernels that account for only a fraction of total response time.

Example 13.4: ResNet-50: Latency budget breakdown

Table 13.3 details the per-phase latency breakdown for an illustrative ResNet-50 serving request.

Table 13.3: ResNet-50 latency budget: Illustrative per-phase breakdown of a single serving request, showing that preprocessing and data transfer together rival the cost of the ResNet-50 forward pass itself. The percentages expose where engineering effort pays off in this scenario. Per-phase percentages are rounded to the nearest whole number and may not sum to exactly 100.

Phase	Operation	Time	Percentage
Preprocessing	JPEG decode	3 ms	30%
Preprocessing	Resize to 224×224	1 ms	10%
Preprocessing	Normalize (mean/std)	0.5 ms	5%
Data Transfer	CPU→GPU copy	0.5 ms	5%
Inference	ResNet-50 forward pass	5 ms	50%
Postprocessing	Softmax + top-5	0.1 ms	~1%
Total		**10.1 ms**	**100%**

Systems insight: In this illustrative ResNet-50 serving budget, preprocessing consumes 44.6 percent of latency despite model inference being the computationally intensive phase. If TensorRT reduces inference to the assumed 2 ms, preprocessing would dominate at 63.4 percent.

The 5 ms batch-one forward pass in table 13.3 is the chapter's reference profile for ResNet-50 on a V100, and the later ResNet-50 examples derive from it. The capacity plan in section 13.5.4 uses it as the service time, and the batching sweep in section 13.7.1 starts from it at batch one. The runtime and precision comparisons in section 13.8 deliberately depart from it, assigning each engine and number format its own illustrative latency so that the effect of that one change is visible.

In the ResNet-50 scenario, dense convolutional arithmetic dominates the model stage's work. That does not make the stage compute bound at every batch size. At batch size one, the late layers, whose large weight tensors are reused over only a few spatial positions, lean toward memory bound, and some layers expose too little parallel work to fill the device or amortize per-kernel launch costs, so the stage runs far below peak arithmetic throughput; batching amortizes those costs and fills the machine (section 13.7.1). Changing the model architecture can move the dominant cost out of dense arithmetic entirely, and recommendation systems exhibit this shift.



Lighthouse 13.1: Lighthouse example: DLRM serving

Scenario: Serving DLRM with a 10 ms p99 latency budget.

Analysis: While ResNet-50's model stage is dominated by convolutional neural network compute, DLRM's dominant model-stage cost is embedding-table memory access. End-to-end serving bottlenecks still require measuring the full path: preprocessing, inference, postprocessing, and data movement. Table 13.4 breaks the recommendation request down by phase.

Table 13.4: DLRM serving latency: Illustrative per-phase breakdown of a recommendation request under a 10 ms p99 budget, contrasting DLRM's memory-bandwidth-bound embedding lookups against ResNet-50's convolution-dominated forward pass. Adding compute to the inference stage does not help once embedding-table bandwidth is the binding constraint.

Phase	Operation	Time	Bottleneck
Input Parsing	Request parsing	0.5 ms	CPU
Embedding Lookup	Fetch 100+ dense vectors	6 ms	memory bandwidth
Inference	MLP forward pass	1.5 ms	Compute
Postprocessing	Ranking & Filtering	1 ms	CPU
Total		**9 ms**	

Systems insight: In DLRM, the "Inference" multilayer perceptron stage is only ~17 percent of the latency. The majority of time is spent in embedding lookups, retrieving 128-dimensional embedding vectors from terabyte-scale tables. This is a memory-bandwidth and capacity-bound workload where adding more compute does not help unless the embedding tables can be served faster.

Together, table 13.3 and table 13.4 expose the same general failure mode: optimization effort goes where ML expertise applies (model quantization, pruning) while the binding constraint sits elsewhere (image decoding on CPU for ResNet-50, embedding-table memory bandwidth for DLRM). For an isolated serial request with other stage times held fixed, model-only optimization is bounded by the time spent outside the model, which is Amdahl's ceiling applied to a request path. Serving adds two twists to that familiar result. Accelerator progress tightens the ceiling, because each hardware generation shrinks the model term while the host stages stay where they were; the perspective below quantifies this for the reference request. Under load, a faster model also shortens the queue, an effect the fixed-fraction calculation cannot see and the first fallacy in section 13.12 develops.



Systems Perspective 13.2: The quantitative approach to serving

Analysis: Amdahl's law (principle 8) sets this ceiling (section D.2.3.1 provides the formal derivation). Preprocessing (4.5 ms) and data transfer (0.5 ms) consume 49.5 percent of total latency. Optimizing the model 10× faster (5 ms → 0.5 ms) yields only a 1.8× end-to-end speedup (reducing latency from 10.1 ms to 5.6 ms). Focusing exclusively on model optimizations like quantization or pruning yields diminishing returns when non-model stages dominate execution time.

Mechanism: Domain-specific accelerators (such as TPUs and Tensor Cores) replace complex control logic with dense multiply-accumulate arrays, which shrinks the model term. The

↩ Amdahl's law applied to an inference pipeline whose preprocessing caps the gain from a faster model is established in section 1.11.

host-side stages stay on general-purpose CPUs and I/O buses, so the faster the accelerator, the larger their share of end-to-end latency.

Systems insight: Profile before optimizing. If preprocessing dominates, GPU-accelerated pipelines (NVIDIA DALI) that move preprocessing next to the accelerator may outperform model quantization, although the end-to-end gain is pipeline-specific.

13.4.2.1 The serving tax bill

Beyond model forward-pass execution, every request pays an infrastructure "tax" to the host system and network stack. When an inference query arrives, it traverses the network interface, moves across kernel and user space, deserializes from wire protocols into memory buffers, queues for batch assembly, and copies across the PCIe interconnect to accelerator device memory. These host-side operations run on general-purpose CPUs and I/O buses, which is why their share grows as the model term shrinks. Table 13.5 itemizes the bill: a representative cost for each component, how it scales, and the design choice that reduces it.

Table 13.5: The Serving Tax Bill: A representative breakdown of noninference latency sources. While individual components like serialization can be below 1 ms, they compound. Under the upper ends of the displayed ranges, this tax can rival a 5 ms model invocation. The engineering goal is to measure the actual request path and reduce the binding costs through choices such as binary payloads, persistent connections, and fewer copies.

Tax Component	Typical Cost	Scaling Behavior	Tax Evasion Strategy
Network I/O	1–5 ms	Linear with payload	Compression, Region Colocation
Serialization	50–500 µs	Linear with payload	gRPC/Protobuf (vs. JSON)
Queuing	0.1–10 ms	Diverges near capacity	Dynamic Batching, Autoscaling
Dispatch	10–50 µs	Constant per batch	Kernel Fusion (reduce launches)
Data Copy	100–500 µs	Linear with tensor	Zero-Copy/Shared Memory

13.4.2.2 The killer microseconds problem

Barroso, Patterson, and colleagues identified a critical gap in how computer systems handle latency across time scales (Barroso et al. 2017). Nanosecond-scale stalls (such as cache misses) are hidden by hardware instruction-level parallelism, while millisecond-scale delays (such as disk I/O or network RPCs) are hidden by operating system thread context switching. Operations in the microsecond range (1–100 µs) fall into an awkward gap: they are too long to spin-wait on a CPU core without wasting compute cycles and power, yet too short for the operating system to context-switch to another thread without paying scheduling overheads that exceed the wait time itself.

Serving infrastructure operates directly in this killer microseconds regime. Using the representative ranges in table 13.5, serialization at 50–500 µs, dispatch at 10–50 µs, and data copy at 100–500 µs are each individually small, but for a 5 ms inference service, these named microsecond-scale overheads collectively consume about 3.2 percent to 21 percent of the latency budget before network and queuing delays are counted. No single overhead justifies optimization in isolation, yet together they determine whether the system meets its SLO.

Tackling this compound overhead demands systematic architectural intervention across the software-hardware interface. When microsecond delays accumulate across serialization, kernel dispatch, and memory copies, isolated optimizations to individual functions yield negligible improvements. Instead, high-performance serving runtimes eliminate boundaries altogether: employing zero-copy memory buffers between networking and execution runtimes, maintaining persistent CUDA streams to bypass driver dispatch latencies, and migrating preprocessing kernels directly onto the accelerator to avoid intermediate host-device transfers.

13.4.3 Resolution and input size trade-offs

Input resolution represents an operational control that couples host preprocessing overhead directly to accelerator inference latency. While model architectures are often evaluated at fixed resolutions, the serving runtime must ingest variable client inputs and rescale them before kernel execution. In convolution-dominated vision networks, floating-point operations scale quadratically with spatial dimensions ($\mathcal{O}(H \cdot W)$), whereas static costs—such as weight streaming from accelerator memory

and kernel launch overheads—remain invariant. Varying input resolution therefore shifts the operational arithmetic intensity across the roofline boundary: compute-bound stages slow down quadratically as pixel count expands, whereas memory-bound stages scale with activation memory footprint.

For convolution-dominated, compute-bound models, equation 13.1 gives a first-order approximation in which throughput scales inversely with resolution squared, comparing two input resolutions r_1 and r_2 (image side lengths in pixels):

$$\frac{\text{Throughput}(r_2)}{\text{Throughput}(r_1)} = \left(\frac{r_1}{r_2}\right)^2 \tag{13.1}$$

↩ The roofline boundary and arithmetic intensity ridge points that separate compute-bound from memory-bound stages are established in section 11.6.

Doubling resolution from 224 to 448 gives a 4× compute-scaling estimate; the illustrative 3.6× scenario includes fixed overhead. Higher resolution can raise convolution arithmetic intensity because weights are reused across more spatial positions, but whether the full model becomes compute-bound depends on the bytes moved by its kernels.

Table 13.6 shows the mechanism behind that gap. Down its last column, each step up in resolution reuses the same weights across more pixels, so the modeled arithmetic intensity rises, and the fixed weight traffic becomes a smaller share of the work. That amortization is why the illustrative slowdown falls short of the pure compute estimate. The estimate counts only weights and one pass over a scaled activation term, so it establishes the direction of the effect; whether the executed kernels are compute or memory bound at a given resolution is a question for a profiler.

Table 13.6: Resolution and Modeled Arithmetic Intensity: This estimate divides FLOPs by weight bytes plus one assumed read/write of a scaled activation term. It omits aggregate intermediate-tensor, cache, and workspace traffic, so crossing the V100 ridge point does not establish the executed-kernel bottleneck.

Resolution	Assumed Activation Term	Modeled Arith. Intensity
224×224	12.5 MB	64.4 FLOP/byte
384×384	36.7 MB	137 FLOP/byte
512×512	65.3 MB	183.9 FLOP/byte
640×640	102.0 MB	218.4 FLOP/byte

13.4.3.1 Resolution strategies in production

Different deployment contexts impose distinct resolution requirements shaped by their dominant constraints. A mobile application may accept lower resolution (224×224) when latency and battery life outweigh marginal accuracy gains. Some medical-imaging workloads use 512×512 or higher when diagnostic detail warrants the additional compute. A perception pipeline may use lower resolution over a wide field and high-resolution crops for fine-grained recognition. A public API can instead normalize arbitrary uploads to a fixed contract or select among validated resolution profiles. Adaptive resolution is useful only when the selection policy and every chosen profile preserve the task's quality requirements.

13.4.3.2 Adaptive resolution

Adaptive resolution lets production systems select resolution dynamically based on content. One approach runs a lightweight classifier at 128×128 to categorize content type, then selects task-appropriate resolution with documents at 512×512, landscapes at 224×224, and faces at 384×384. In this illustrative scenario, the policy provides 1.4× throughput improvement with 99.2 percent accuracy retention vs. fixed high resolution. This pattern trades preprocessing cost from running the lightweight classifier for inference savings on the main model.

Resolution sets the cost of the stages before the model. The stage after it is usually far cheaper, but it still crosses the device boundary and spends host CPU time, so it belongs on the same budget.

13.4.4 Postprocessing

The request lifecycle ends with postprocessing, which transforms raw accelerator output tensors into structured application responses. This stage straddles the physical boundary between the accelerator and the host CPU. Because client applications cannot consume device memory buffers

directly, output tensors must cross the interconnect (typically a PCIe bus) to reach host memory. How the serving engine partitions computation across this boundary dictates whether postprocessing completes in microseconds or stalls the entire serving pipeline.

13.4.4.1 From logits to predictions

Classification models emit unnormalized logits across the label space. Converting these numerical scores into actionable predictions requires reduction, filtering, and calibration. The simplest reduction is argmax selection, which extracts the index of the highest logit. Thresholding discards predictions below an empirical certainty cutoff, returning predictions only when the model meets a target precision. When applications require ranked alternatives, top-k extraction selects the k largest scores alongside their class indices. Probability calibration scales raw scores (for example, through temperature scaling) so that output confidence scores reflect empirical accuracy.

Executing reduction operations on the accelerator can reduce host-device transfer volume when the saved transfer cost outweighs the device execution and launch overhead. Performing softmax, argmax, or top-k selection in device memory prunes the payload before it crosses the PCIe bus. In ResNet-50 classification, for example, running top-k on the GPU reduces the transfer payload from 1,000 float32 logits (4,000 bytes) to five values and five integer indices. For ResNet-50 image classification, listing 13.1 traces this execution path from GPU tensor operations to host CPU label lookup and response formatting.

Listing 13.1: ResNet-50 Postprocessing: Normalizes raw logits into probabilities, extracts top-k predictions, and formats the API response.

```
# Normalize raw logits into probabilities
# Input: logits tensor of shape (1, 1000), one score per ImageNet
# class
probs = torch.softmax(logits, dim=-1)  # Normalize to sum=1 on GPU

# Extract top-5 predictions for multi-class response
# topk returns (values, indices) sorted by probability
top5_probs, top5_indices = probs.topk(5)  # GPU operation
top5_probs = top5_probs.squeeze(0).tolist()
top5_indices = top5_indices.squeeze(0).tolist()

# Map class indices to human-readable labels
# imagenet_labels: list of 1000 class names from synset mapping
labels = [imagenet_labels[i] for i in top5_indices]  # CPU lookup

# Format response with predictions and metadata for API contract
response = {
    "predictions": [
        {"label": label, "confidence": float(prob)}
        for label, prob in zip(labels, top5_probs)
    ],
    "model_version": "resnet50-v2.1",  # Client-side version tracking
    "inference_time_ms": 5.2,  # Observability for latency monitoring
}
```

In listing 13.1, total postprocessing time is approximately 0.1 ms, negligible compared with preprocessing and inference. The GPU handles the normalization and top-5 reduction, so the subsequent `.tolist()` call transfers only tens of bytes across PCIe. However, `.tolist()` forces a synchronous device-to-host copy, blocking the host thread until the GPU stream finishes. In high-throughput pipelined servers, such synchronization barriers must be scheduled carefully to avoid stalling the CPU thread while it feeds subsequent requests. Fitting a calibration mapping is an offline task; applying a simple fitted mapping online is usually inexpensive, but it is valuable only when downstream decisions require confidence scores whose empirical meaning has been validated.

13.4.4.2 Output formatting

Production systems package predictions to satisfy application-level API contracts. These contracts govern JSON serialization schemas, floating-point formatting, error codes, and fallback defaults when thresholding rejects all candidates. In Python-based runtimes, repetitive JSON serialization and string allocation can consume unexpected host CPU cycles under heavy concurrency, turning a microsecond-scale formatting step into a CPU bottleneck. High-throughput servers address this by delegating serialization to native C++ runtimes or binary protocols like Protocol Buffers. Response metadata, including model version and inference execution time, completes the payload to enable downstream observability.

Characterizing postprocessing closes the individual request's latency budget. However, analyzing these stages strictly in isolation assumes sequential evaluation, where each transaction completes its entire lifecycle before subsequent work begins.

13.4.5 Hardware utilization and request pipelining

Executing each request in lockstep isolation leaves specialized accelerators heavily underutilized. Because network termination, payload preprocessing, neural network evaluation, and response serialization bind on distinct physical subsystems—host network interfaces, CPU cores, PCIe interconnects, and accelerator matrix units—strict serial processing forces the accelerator into idle bubbles while the host prepares incoming tensors. The central systems objective is to maximize the accelerator's **duty cycle**—the proportion of wall-clock time spent actively executing arithmetic kernels—without violating request tail-latency deadlines. Serving runtimes achieve this through **request pipelining**, overlapping asynchronous host preprocessing of forthcoming queries with concurrent accelerator execution on previously staged batches.

13.4.5.1 Overlapping I/O and compute

↩ Asynchronous host-accelerator transfer and double buffering mechanisms are established in section 11.5.6.2.

Figure 13.5 sequences execution across CPU preprocessing and GPU inference to demonstrate how request pipelining recovers lost compute capacity. In serial execution (A), each request must complete its entire lifecycle (Network → CPU Preprocessing → GPU Inference → Postprocessing) before the next request begins, leaving the GPU idle for more than 50 percent of wall-clock time. Pipelining concurrent requests (B) overlaps CPU preprocessing of upcoming requests with active GPU inference on earlier inputs, eliminating the idle bubbles and maximizing accelerator duty cycle.

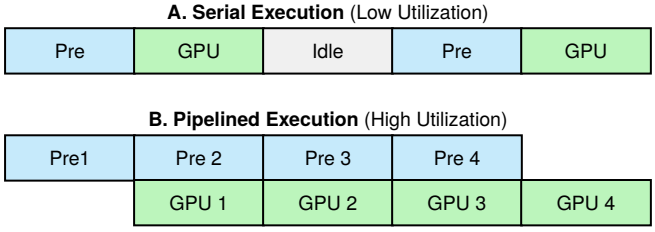


Figure 13.5: Request Pipelining: Pipelining hides latency by overlapping independent operations across different hardware resources. In pipelined execution (B), the CPU processes the next request's data while the GPU executes the current request's inference. This can increase accelerator duty cycle and throughput without changing the model.

Pipelining relies on non-blocking hardware queues and asynchronous execution streams. Instead of blocking until a GPU kernel completes, the host CPU thread enqueues memory transfers and kernel launches into an accelerator stream and immediately begins preprocessing the next incoming request. The accelerator's DMA engine and compute cores execute those queued operations independently, overlapping host preprocessing with device execution.

13.4.5.2 The systems metric: Hardware duty cycle

System efficiency here measures accelerator duty cycle rather than the fraction of peak arithmetic throughput or bandwidth achieved. In equation 13.2, compute time is the union of useful kernel-

active intervals on each resource, summed across resources; overlapping kernels on one GPU are not counted twice.

$$\text{System Efficiency} = \frac{\sum T_{\text{compute}}}{\text{Wall Clock Time} \times \text{Resource Count}} \tag{13.2}$$

If a ResNet-50 request takes 10 ms total (5 ms GPU, 5 ms CPU), a serial system achieves only 50 percent efficiency. With at least two requests in flight in a sustained stream, duty cycle can approach 100 percent after pipeline fill if the CPU keeps up with the GPU. A finite pair still pays pipeline fill and drain time, and stochastic service must retain enough capacity headroom to meet its tail-latency objective. If the CPU is too slow to feed the GPU, the system becomes CPU-bound, and further model optimization provides zero throughput gain; this is the Amdahl ceiling of section 13.4.2 seen from the throughput side. Hardware upgrades tighten it, because accelerator compute throughput has grown far faster than CPU single-thread performance across successive generations. A system that was compute-bound on an older accelerator may become CPU-bound after a hardware upgrade, not because preprocessing got slower but because the model got dramatically faster while the CPU did not.

Checkpoint 13.1: The latency budget

A request's latency is spent across every stage of its path, not only in the model. Use this checkpoint to locate where the budget goes before choosing an optimization.

- ☐ **Amdahl ceiling**: in the reference ResNet-50 request, why does a 10× faster model yield far less than a 10× faster response, and why does a faster accelerator make the ceiling tighter rather than looser?
- ☐ **Tail vs. mean**: can you explain why mean latency cannot demonstrate compliance with a p99 SLO?
- ☐ **Latency budget**: given a 50 ms request deadline and a combined upper bound of 18 ms for all other stages, including network transit, preprocessing, transfer, and postprocessing, how much remains for inference and queueing? Why would a measured 18 ms mean for those stages be insufficient to certify a p99 SLO?
- ☐ **Pipelining**: explain how overlapping CPU preprocessing with GPU inference raises accelerator duty cycle without changing the model, and what happens to throughput when the CPU stage is slower than the GPU stage.

Tracing an isolated request reveals the intrinsic execution time of each pipeline phase. In production, however, multiple requests arrive concurrently and contend for shared accelerators, CPU cores, and memory buses, introducing queuing delays that dwarf raw computation time.

13.5 Queuing Theory

When arrival streams fluctuate, queuing theory formalizes how contention for shared physical resources (accelerator execution queues, host-to-device PCIe transfers, and memory buses) degrades latency distributions. Its governing laws explain why seemingly well-provisioned inference services experience sharp, nonlinear latency explosions as utilization approaches capacity.

13.5.1 Little's Law

Capacity planning begins by determining system concurrency: given an expected arrival rate and measured mean response time, an engineer must calculate how many concurrent requests reside in the system at any instant. This calculation determines how much memory must be reserved for request buffers and activation states before deciding how many accelerators to provision. Little's Law resolves this concurrency question by relating mean in-system population to throughput, while queuing models predict how response times degrade as utilization nears saturation. Together, they provide the quantitative framework for capacity planning.

↪ The derivation of Little's Law from conservation of requests is developed in section D.2.4.

Systems Perspective 13.3: Notation alert: L vs. latency

Classical queueing texts write Little's law with single letters for the in-system count and the time in system; this book writes Q_{req} for the count and T_{lat} for the time, because the classical count letter would collide with the loss $\mathcal{L}$ and with the iron law's fixed-latency term L_{lat}. In queuing theory, T_{lat} denotes the *response time* or *time in system per request*; queue-only waiting time is W_q. Here, Q_{req} denotes the average in-system request count, λ_{arr} the arrival rate, μ the service rate of a server that handles one request at a time, and $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ the serving utilization. The subscripts distinguish queueing notation from the degradation equation's λ sensitivity parameter and keep serving utilization from occupying bare ρ. Section 13.7.2 adds the few symbols that batching needs.

Formulated by John D. C. Little, **Little's law**[9] establishes an exact relationship among three macroscopic metrics in any stable queuing system:

$$Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}} \tag{13.3}$$

where Q_{req} is the average number of requests residing in the system, λ_{arr} is the arrival rate (requests per second), and T_{lat} is the average time each request spends from arrival to completion.

Little's law holds regardless of the arrival distribution, service-time distribution, or scheduling discipline; it requires only that the system is stable and that requests are neither created nor destroyed internally. For an inference service targeting 1000 QPS with a mean response time of 50 ms, Little's law dictates the average in-flight request population that host memory buffers and worker threads must sustain.

Napkin Math 13.2: Little's Law capacity sizing

Problem: How many requests are in the system on average at 1,000 QPS?

Math: Little's Law gives $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$, so average population equals throughput multiplied by mean time in system.

Given:

- **Arrival rate** (λ_{arr}), equal to throughput in a stable system: 1,000 QPS.
- **Mean time in system** (T_{lat}): 50 ms (0.05 s).

Math: Q_{req} = 1,000 QPS $\times$ 0.05 s = 50 concurrent requests on average

Systems insight: The system holds 50 requests on average across batch and queue state. That population is a memory and concurrency requirement, not a limit on the accelerator. A server whose GPU executes at most 32 requests per batch can still sustain this load, because the requests not currently executing wait in host memory; service rate, queueing behavior, and the tail distribution, not the population itself, determine whether the load is feasible.

Because equation 13.3 binds these three variables, constraining any two dictates the third. If an inference service averages 10 ms per request ($T_{\text{lat}} = 0.01$ s) and holds 50 concurrent requests on average ($Q_{\text{req}} = 50$), the sustainable arrival rate must be $\lambda_{\text{arr}} = Q_{\text{req}}/T_{\text{lat}} = 5000$ requests per second. Conversely, if host memory limits concurrency to an average of 10 requests while average time in system remains 10 ms, the system cannot exceed a throughput of 1,000 requests per second without dropping traffic.

13.5.2 The utilization-latency relationship

Little's Law describes average system inventory under steady-state conditions, but it does not reveal how latency behaves as traffic approaches system capacity. The M/M/1 queuing model (in Kendall's notation: Markovian, that is Poisson, arrivals; Markovian, that is exponential, service times; and one server) quantifies the headroom required to prevent latency collapse (Harchol-Balter 2013).[10] For a

[9] **Little's law**: John D. C. Little (1961) proved that $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ holds for stable long-run averages over consistent system boundaries, regardless of arrival distribution, service distribution, or scheduling discipline; this universality anchors ML capacity planning: the formula requires no Poisson-arrival or exponential-service assumption. It does require well-defined long-run averages and consistent definitions of arrival rate, time in system, and population.

[10] **M/M/1 queue**: Queuing theory originated with Agner Krarup Erlang's 1909 analysis of the Copenhagen Telephone Exchange, where call arrivals were modeled as memoryless (Poisson). The M/M/1 model's exponential service-time assumption introduces more variance than many fixed-shape ML inference workloads exhibit. At the same utilization and mean service time, its mean queueing wait is twice the M/D/1 value; the ratio for total response time approaches two only as utilization becomes high. This comparison makes the model useful for planning against a deterministic-service baseline; it does not establish a bound for arbitrary production traffic.

system with Poisson arrivals and exponential service times, equation 13.4 gives the average time in system:

$$T_{\text{lat}} = \frac{1}{\mu - \lambda_{\text{arr}}} = \frac{\text{service time}}{1 - \rho_{\text{serv}}} \tag{13.4}$$

where λ_{arr} is the arrival rate, μ is the service rate (requests per second the server can handle), and $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ is the utilization (the fraction of time the server is busy).

Because the denominator contains $1 - \rho_{\text{serv}}$, latency grows nonlinearly as utilization approaches unity.[11] Small incremental increases in traffic near saturation trigger severe backpressure and queuing delays.

The standard M/M/1 formulation assumes exponentially distributed service times. For compiled neural networks processing fixed-size batches and static tensor dimensions, execution times are nearly constant, making an M/D/1 queue a tighter model of mean queuing delay. The analysis here retains the M/M/1 model because it yields closed-form solutions and predicts greater mean queueing delay than M/D/1 under matched arrival rate and mean service time. In production environments, variable input lengths, dynamic batch formation, CPU preprocessing, and key-value cache lookups introduce runtime variance that quickly breaks deterministic assumptions.[12]

Table 13.7 quantifies this relationship, demonstrating how average time in system multiplies rapidly as utilization approaches 100 percent.

Table 13.7: Utilization-Latency Relationship: Average time in system (wait + service) as a multiple of service time for an M/M/1 queue. At 50 percent utilization, time in system is 2× service time; at 90 percent, it reaches 10×. This nonlinear growth explains why systems that perform well at moderate load suddenly violate SLOs when traffic increases: moving from 80 percent to 90 percent utilization doubles latency.

Utilization (ρ_{serv})	Latency Multiple	Example (5 ms service)
50%	2×	10 ms
70%	3.3×	17 ms
80%	5×	25 ms
90%	10×	50 ms
95%	20×	100 ms

13.5.3 Multi-server considerations

A single serving node forms the atomic building block of inference infrastructure. Analyzing single-node queuing dynamics under $M/M/1$ assumptions exposes the fundamental headroom requirements and saturation thresholds of the hardware pipeline before deploying at scale. Rigorous node profiling determines whether service-time bottlenecks stem from accelerator compute saturation, dynamic batching window delays, host preprocessing overheads, or memory allocator locks. Once single-node performance and queueing knee boundaries are established, accommodating higher aggregate request volumes shifts the architectural focus to horizontal replication and traffic dispatch.

When ingress traffic exceeds single-node capacity, the architecture scales out horizontally by distributing incoming queries across a fleet of identical serving replicas positioned behind a load balancer. The $M/M/c$ queuing model extends single-server theory to c parallel workers, illustrating how centralized request pooling buffers stochastic arrival bursts more effectively than isolated single-server queues. In practice, achieving theoretical tail-latency gains across a replicated cluster depends critically on dispatch policies (such as least-loaded routing or power-of-two-choices), input-dependent service time variance, and per-replica headroom margins. Furthermore, horizontal replica scaling remains architecturally distinct from distributed model parallelism: horizontal replication duplicates complete model instances across independent nodes to process disjoint queries concurrently without inter-node synchronization, whereas model parallelism shards a single large model across accelerators linked by high-bandwidth fabrics (such as NVLink) to evaluate an individual query cooperatively.

[11] **Super-linear latency divergence**: The planning knee often appears well before full saturation; in the M/M/1 mean response-time equation, $T_{\text{lat}} = \frac{1/\mu}{1-\rho_{\text{serv}}}$, where $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ is utilization. The $(1-\rho_{\text{serv}})^{-1}$ term diverges as $\rho_{\text{serv}} \to 1$: at $\rho_{\text{serv}} = 0.7$, mean response time is already 3.3× the base service time; at $\rho_{\text{serv}} = 0.9$, it is 10×. The exact operating limit is a policy and workload choice, but trying to stretch a latency-sensitive queue toward saturation creates disproportionate latency growth.

[12] **Kendall notation**: Kendall's A/S/c (arrival/service/servers) shorthand is the one already used for $M/M/1$, where "M" signifies a Markovian (memoryless) process; the "D" in M/D/1 denotes deterministic service. M/M/1 is conservative relative to M/D/1 for mean queueing wait when service times are nearly deterministic, but neither model replaces measurement of a production workload's arrival process, service-time variance, or tail behavior.

13.5.4 Tail latency

Production SLOs typically specify percentile targets (p95, p99) rather than averages because tail latency determines user experience for the slowest requests (Dean and Barroso 2013).

The M/M/1 model that produced the mean in equation 13.4 also yields the full latency distribution, and the tail follows from it. Total request latency (queue wait plus service) is exponentially distributed with cumulative distribution function $F(t) = 1 - e^{-t/T_{\text{lat}}}$, where $T_{\text{lat}} = \frac{\text{service time}}{1-\rho_{\text{serv}}}$ is that mean response time. To derive the 99th percentile latency (p99), we solve for the time t_{99} where 99 percent of requests have completed ($F(t_{99}) = 0.99$):

$$1 - e^{-t_{99}/T_{\text{lat}}} = 0.99 \implies e^{-t_{99}/T_{\text{lat}}} = 0.01 \implies -\frac{t_{99}}{T_{\text{lat}}} = \ln(0.01) = -\ln(100) \approx -4.605$$

Solving for t_{99} and substituting the mean latency T_{lat} yields the governing expression for tail latency:

$$T_{\text{lat,p99}} \approx \frac{\text{service time}}{1-\rho_{\text{serv}}} \cdot \ln\left(\frac{1}{1-0.99}\right) \approx \frac{4.6 \cdot \text{service time}}{1-\rho_{\text{serv}}} \tag{13.5}$$

Mean latency masks tail contention: at 70 percent utilization, average latency is only 3.3 times service time, while p99 latency reaches 15.3 times service time $(4.6/0.3 \approx 15.3)$. For deterministic-service models such as M/D/1, tail values require numerical calculation rather than a simple universal multiplier, but the core systems reality remains: evaluating a serving cluster by average latency conceals severe tail degradation.

13.5.4.1 The tail at scale problem

Dean and Barroso's analysis reveals why tail latency becomes critical as systems scale beyond single machines (Dean and Barroso 2013). When requests fan out to m parallel backends (querying an ensemble of models, searching sharded embedding indices, or parallelizing pipeline stages), the overall user request completes only when the slowest sub-task finishes. If sub-task completion events are independent and each has a 99 percent probability of finishing within latency threshold $L_{\text{lat,SLO}}$, the probability that all m backends finish within $L_{\text{lat,SLO}}$ is $(0.99)^m$. For a fan-out of $m = 10$, only $0.99^{10} \approx 90.4\%$ of requests meet the target; at $m = 100$, that probability drops to $0.99^{100} \approx 36.6\%$, meaning nearly two-thirds of user requests experience a tail delay. This "tail at scale" effect makes controlling single-node tail variance essential for overall system performance.

Before exploring software mitigations like hedging, systems engineers must first provision sufficient accelerator capacity to satisfy tail-latency SLOs under peak load. Applying the M/M/1 tail-response-time model to a ResNet-50 deployment makes the provisioning workflow concrete.



Napkin Math 13.3: ResNet-50 capacity planning

Consider designing a ResNet-50 serving system with these requirements:

- **Queue-plus-GPU p99 subbudget**: 50 ms
- **Peak expected traffic**: 5,000 QPS
- **GPU service time** (batch one, the chapter's reference ResNet-50 profile): 5 ms

Step 0: Define the modeled request boundary. The worker model covers waiting for the GPU and the GPU forward pass only. This scenario allocates 50 ms to that portion of the request and separately assumes a combined 18 ms upper bound for network transit, serialization, preprocessing, transfer, and postprocessing. The end-to-end deadline must accommodate both allowances. A 50 ms end-to-end deadline would instead require subtracting the outside-worker allowance before applying the queue model; the arithmetic below would no longer apply. An outside-stage mean or p99 cannot substitute for the stipulated combined upper bound.

Step 1: Find safe utilization. From equation 13.5, $T_{\text{lat,p99}} \approx 4.6 \times$ service time / $(1 - \rho_{\text{serv}})$. Setting $T_{\text{lat,p99}} \leq 50$ ms with 5 ms service time gives $\rho_{\text{serv}} \leq 1 - (4.6 \times 5\,\text{ms})/50\,\text{ms} = 0.54$ (54 percent maximum utilization). This is an M/M/1 p99 estimate under the stated arrival and

service assumptions. The mean-wait comparison with M/D/1 does not establish a conservative bound on production p99; a load test must validate the tail.

Step 2: Calculate required service rate. $\mu_{\text{required}} = 5,000\,\text{QPS}/0.54 = 9259.3\,\text{req/s}$

Step 3: Determine GPU count. In this M/M/1 model, the single-GPU rate is 200 req/s.

GPUs needed = 9259.3 req/s / 200 req/s = 46.3 $\rightarrow$ 47 GPUs

This single-GPU rate is the batch-1 service rate. Each request holds the GPU for the full service time, which keeps the M/M/1 model closed-form. Dynamic batching (section 13.7) amortizes fixed overhead across requests and raises the per-GPU rate several-fold, so a batched deployment may need fewer GPUs after its own latency distribution is validated; section 13.11.3 compares the two regimes.

Step 4: Add headroom for variance. This scenario adds 30 percent headroom for traffic spikes and variance: final count = 47 $\times$ 1.3 = 61.1, rounded up to 62.

Step 5: Verify fault tolerance. The 30 percent headroom addresses traffic variance, but production systems also need fault tolerance. With 62 GPUs, losing one leaves 61 GPUs handling 5,000 QPS. The postfailure utilization is 5,000 QPS / (200 req/s $\times$ 61) = 41.0%.

This remains well below the 54 percent safe utilization threshold, confirming N+1 redundancy is satisfied. For stricter fault tolerance requirements, N+2 redundancy (tolerating two simultaneous failures) would require 49 GPUs under the same safe-utilization threshold, or about 64 GPUs if the 30 percent headroom must remain after two simultaneous failures.

Result: Under the batch-1 M/M/1 assumptions, 62 V100 GPUs supply the modeled queue-plus-GPU capacity for 5,000 QPS within the 50 ms p99 subbudget, including one failed replica. End-to-end acceptance still requires validating the tail under load and the separately reserved outside-worker allowance.

The queuing analysis explains the capacity planning approach detailed in section 13.11.3 and connects directly to the MLPerf Server scenario. Section 12.8.4.2 defines that scenario as Poisson arrivals scored by throughput against a tail-latency target. A Server result is therefore valid only when the run satisfies its percentile-latency constraint; otherwise, the target QPS must be reduced and the run repeated.

13.5.5 Tail-tolerant techniques

Eliminating all sources of latency variability is often impractical. Production systems instead employ techniques that tolerate variability while still meeting SLOs (Dean and Barroso 2013; Dean 2012). For one serving node, the first distinction is between an isolated straggler and overload. Racing an isolated slow request can reduce its latency when spare capacity exists; duplicating work on an overloaded accelerator deepens the queue. Admission control and graceful degradation instead limit how much work the node accepts.

When the node is overloaded, racing requests only exacerbates queue depth by injecting duplicate work. The system must instead protect the latency of admitted requests through graceful degradation or admission control. **Graceful degradation** returns lower-fidelity results to preserve throughput: classification pipelines fall back to cached approximations, generative models truncate decoding lengths, and model ensembles evaluate only a subset of members. Dynamically reducing active ensemble members shortens the service time $T_{\text{svc}}(B)$ of each batch of B requests, which increases the effective service rate $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$ and lowers utilization $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu_{\text{eff}}(B)$ toward a workload-appropriate operating point with more headroom, as illustrated by figure 13.1, trading a controlled accuracy reduction for SLO survival instead of an uncontrolled latency collapse. **Admission control** enforces an even stricter boundary: when the ingress queue exceeds a designated threshold, the server immediately rejects excess queries with HTTP 503 status codes rather than admitting work doomed to violate its latency deadline.

Setting this rejection threshold requires accounting for the work ahead of the arriving request. Consider a warmed, isolated worker that serves requests sequentially in first-in-first-out (FIFO) order, without batching or preemption. Assume known upper bounds on each request's service time and on every remaining stage outside the worker, including the response path. Subtract time already

spent since the request began from its end-to-end latency allowance, then subtract the remaining outside-worker bound and the arriving request's service bound. The result is its waiting allowance. The worker's outstanding-work bound is the sum of the current request's residual service bound and the service bounds of queued requests. Admission passes the deadline screen only when that sum fits within the waiting allowance; a negative allowance rules out admission even to an idle worker. A request-count limit approximates this comparison only when service costs are sufficiently uniform.

Suppose the outstanding work exceeds the full model's waiting allowance but fits the allowance obtained with a validated, faster model that is already resident and warmed. The server can admit the request through that fallback if its prediction quality meets the application's minimum requirement. Shortening the new request's service time frees waiting allowance, but does not remove work already queued ahead of it. If neither acceptable model fits, rejection avoids spending accelerator time on a result that would arrive too late. Rejection reduces availability, while fallback can reduce prediction fidelity; both outcomes must be counted alongside latency so that shedding difficult requests does not make the service appear successful merely by shrinking its measurement population.

The deadline screen is a deterministic bound only under the stated service, isolation, and scheduling assumptions. Mean service time or a per-stage p99 cannot replace those upper bounds. Variable input lengths, batch formation, and shared memory-bus contention require empirical calibration against the joint request-latency distribution. Completed-request tails must be reported together with rejection rates and fallback quality over offered traffic. Adaptive admission control can tighten the admissible backlog as measured tail latency approaches the SLO, but neither a fixed queue count nor the simplified screen certifies a production p99 target.

An optional extension beyond the local admission decision concerns races across replicas and protection against fan-out delays. For an isolated straggler replica, the system can race the slow path. Under hedging, when a request has not completed within the expected time, the system sends a duplicate request to another server.[13] The client uses whichever response arrives first and cancels the other. For ML serving, this means maintaining multiple model replicas and routing slow requests to alternative replicas. With a stable latency distribution, a threshold set at the historical 95th percentile triggers duplicates for about 5 percent of requests; correlated slowdowns can raise that fraction and make hedging amplify overload instead of reducing the tail.

Cancellation before dispatch prevents a losing copy from consuming accelerator time; cancellation after dispatch may only discard its result. A race therefore needs spare capacity and a duplicate-work budget, even when only a small fraction of requests trigger it.

Tied requests make the same race more aggressive by placing the request in multiple server queues simultaneously, with coordination that cancels the other copies once one server begins processing. This avoids waiting to detect a slow response before hedging and can reduce queuing delay when multiple ready replicas are available. It also creates duplicate queue entries, so the coordination mechanism must cancel losing copies before they consume scarce accelerator time.

Fan-out systems need a different intervention point because one slow backend can stall the entire distributed request. Canary requests first send the request to a small subset of one to two servers.[14] If these return within expected time, the system sends to the remainder. If the canary is slow, the system can retry elsewhere or use cached results before committing to the full fan-out. The technique turns a potential tail-latency amplification problem into an early warning signal.

The same duplicate-work constraint applies to retries across a fleet. A cascading failure occurs when all replicas experience simultaneous overload. If upstream load balancers retry rejected requests at other replicas that are also overloaded, retry traffic amplifies the overload. Coordinated load shedding addresses this by sharing load information across replicas, enabling system-wide decisions about which requests to accept. When global load exceeds capacity, replicas collectively reject the same fraction of requests rather than each rejecting independently and triggering retries.

Replicas or streams on one machine can also race requests, but sharing an accelerator or memory bus can correlate their slowdowns and exhaust the same headroom. The local admission decision remains the foundation; fleet coordination is an extension of it. Production frameworks may use shortest-job-first or deadline-aware scheduling rather than FIFO, changing which queued work lies ahead of an arrival and requiring a corresponding admission analysis (Harchol-Balter 2013).

13 **Hedging**: The term is borrowed from finance, where an offsetting bet reduces risk; here, the redundant request is a bet against a slow server. This is not free: for ML systems, the losing hedged request may still occupy accelerator time if inference has already launched, because ordinary GPU kernels are not cheaply canceled mid-execution. Thus, a hedging policy must budget duplicate work as well as the latency benefit.

14 **Canary**: Named for the coal mine practice (early 1900s–1980s) of using birds whose high metabolic rate made them sensitive to toxic gases before concentrations became lethal to humans. In ML serving, canary requests serve the same early-warning function for fan-out queries: by testing 1–2 backends before committing to the full fan-out, the system detects slow or failing replicas before a single straggler stalls the entire distributed inference request—a critical protection when fan-out width means tail latency grows with the maximum of all backend response times. Canary deployments borrow the name for a different mechanism, exposing a new model version to a small slice of traffic.

Checkpoint 13.2: Queuing and SLO headroom

Latency SLOs are not enforced by *fast inference* alone; they are enforced by *headroom*.

- ☐ **Little's Law**: at a fixed stable throughput, can you use $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ to explain why a higher average in-system population implies a higher mean response time, even if per-request compute time is unchanged?
- ☐ **Utilization cliff**: can you explain why latency grows nonlinearly as utilization ρ_{serv} approaches one, and why production systems target a conservative ρ_{serv} rather than "100 percent busy"?
- ☐ **Queue behavior**: use the mean and p99 curves in figure 13.1 to explain nonlinear latency growth near saturation. Why is the marked 70 percent knee an illustrative planning choice rather than a model-derived breakpoint?
- ☐ **Capacity planning**: can you explain why a throughput number is only "real" if requests still meet the percentile latency SLO under load?
- ☐ **Admission decision**: for the isolated FIFO worker, how does increasing the arriving request's service bound change its waiting allowance? If a faster fallback fits behind the outstanding work, what quality and availability conditions distinguish admitting it from rejecting the request? Why would substituting measured mean times invalidate the deterministic bound?
- ☐ **Headroom estimate**: a stable service handles 2,000 requests per second and measures a 10 ms mean response time. Use $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ to estimate its average in-flight population. Why does this result alone not establish compliance with a 50 ms p99 SLO?

Queuing theory and tail-tolerance policies govern the steady-state processing of requests. They presuppose, however, that serving instances are already initialized and that incoming data matches offline training assumptions. Both preconditions frequently fail during cold starts, auto-scaling surges, and silent feature corruption.

13.6 Model Lifecycle Management

Steady-state queuing optimizations offer little protection when a serving cluster fails to initialize properly or emits degraded predictions. In a representative deployment, a newly provisioned replica needs 35 seconds before it can answer its first request, most of them spent compiling a TensorRT engine, while a subtle PIL-to-OpenCV preprocessing discrepancy silently bleeds five percentage points of classification accuracy. The two failures sit at opposite ends of a replica's life. Skew corrupts every prediction a correctly running replica makes, so it is prevented by keeping training and serving transformations identical. Cold start delays the moment a new replica becomes useful, so it is controlled by how weights are loaded, cached, and shared across the models that occupy one machine.

13.6.1 Training-serving skew

A model that performed well during validation may silently degrade when deployed. This phenomenon, training-serving skew, represents one of the most subtle failure modes in production ML because it is invisible to latency monitoring and exception tracking (Sculley et al. 2015; Baylor et al. 2017). The consistency requirement belongs to the data pipeline; serving is where a violation surfaces. Because the divergence is measurable on identical inputs without labels (principle 11), it is also one of the most preventable.

↩ The consistency imperative, which requires equivalent transformation behavior and state in training and serving, is established in section 4.6.1.

Definition 13.3: Training-serving skew

Training-serving skew is the distributional divergence between the training and inference environments caused by inconsistent logic or state.

1. **Significance**: It violates the consistency imperative and can cause silent accuracy degradation when the transformation functions differ ($f_{\text{train}}(x) \neq f_{\text{serve}}(x)$).
2. **Distinction**: Unlike data drift (which is an external shift in the environment), training-serving skew is an internal failure of the engineering stack.
3. **Common pitfall**: A frequent misconception is that skew is "found" by looking for errors. In reality, it is invisible to exceptions: the system runs perfectly and the latency is low, but the predictions are statistically wrong.

Section 7 catalogs the skew categories and the method that detects each. The serving-specific manifestation of skew is **preprocessing divergence**: the real-time inference pipeline processes raw data differently than the batch training pipeline, a common failure mode when training uses Python/Pandas while serving uses C++/Java or optimized inference servers. Unlike data drift, this divergence is internally introduced and often preventable through shared transformations, parity tests, and careful engineering.

Example 13.5: ResNet-50: Image preprocessing skew

Scenario: ResNet-50 vision serving models experience silent prediction accuracy degradation between offline training and online serving pipelines.

Diagnosis: The serving pipeline implemented C++/OpenCV preprocessing (`cv2.INTER_LINEAR`, BGR order) while training used Python/PIL (`PIL.BILINEAR`, RGB order, different normalization constants).

Systems lesson: Preprocessing divergence causes silent training-serving skew without raising runtime exceptions. Standardizing data transformation pipelines across training and serving environments prevents input distribution corruption.

13.6.2 Cold start and initialization dynamics

Even with mathematical parity guaranteed between training and serving pipelines, a newly provisioned serving instance cannot process queries until its state is instantiated across hardware memory hierarchies (Romero et al. 2021). This initialization latency, known as cold start, dictates system responsiveness during deployments, autoscaling events, and failure recovery.

Definition 13.4: Cold start

Cold start is the initialization latency incurred when instantiating a new model replica.

1. **Significance**: It represents the fixed cost of state hydration (loading weights, compiling graphs), which can take seconds or minutes, effectively blocking the system's ability to scale elastically in response to traffic bursts.
2. **Distinction**: Unlike inference latency ($L_{\text{lat,infer}}$), which is a per-request cost, cold start is an initialization cost paid when a replica or model instance is created, including deployment, scale-out, recovery, and on-demand model loading.
3. **Common pitfall**: A frequent misconception is that cold start is "just loading weights." In reality, it includes graph compilation and memory allocation, which can often take longer than the bandwidth-limited data transfer itself (D_{vol}/BW).

Total cold-start latency compounds across distinct physical operations along the critical initialization path. First, weight loading transfers serialized parameters from persistent storage across the network or host I/O subsystem into host memory, and ultimately across the PCIe bus into GPU HBM. Graph compilation lowers the framework computational graph into device-specific machine code, executing kernel fusion and auto-tuning matrix multiplication kernels for the specific accelerator architecture. Memory allocation reserves device memory arenas for activation scratchpads and

intermediate buffers. Finally, warmup[15] execution issues synthetic inferences to populate kernel caches, prime framework memory allocators, and force lazy initialization before live traffic arrives.

Establishing a CUDA context[16] represents the first fixed cost on the device initialization timeline. Before launching any GPU kernel or allocating device memory, the runtime must instantiate a driver-level context: an internal control structure that manages virtual address spaces, loaded kernel binaries, and hardware event queues. Creating this context requires synchronous communication with the kernel-mode driver and allocates tens to hundreds of megabytes of device memory for bookkeeping. The 0.3–0.5 s range in table 13.8 reflects a typical data center environment; while CUDA lazy loading defers kernel module loading to shorten this initial pause, it merely defers that latency penalty to the first executed inference (NVIDIA 2026a).

In multi-process serving deployments where multiple worker processes share a single accelerator, replicating individual CUDA contexts wastes both memory capacity and initialization time. CUDA Multi-Process Service (MPS)[17] coordinates concurrent work from multiple client processes, reducing scheduling overhead when individual processes underuse the device (NVIDIA 2026d). Its isolation behavior depends on the architecture. Pre-Volta clients shared a GPU address space; Volta clients retain separate GPU address spaces and contexts while their kernels can execute concurrently. Separate address spaces do not provide hard fault containment: a fatal GPU fault can affect other clients sharing the affected GPU, so the serving design must budget for correlated replica failures.

Example 13.6: ResNet-50: Cold start timeline

Table 13.8 traces the per-phase duration of a ResNet-50 cold start:

Table 13.8: ResNet-50 cold start timeline: Illustrative per-phase durations for weight loading, CUDA context creation, TensorRT compilation, and warmup, with totals for the optimized local case and the first-deploy cloud case showing where the dominant cost lives.

Phase	Duration	Notes
Weight loading (SSD)	0.5 s	98 MB FP32 weights from local storage
Weight loading (S3)	3–5 s	Network latency dominates for cloud storage
CUDA context	0.3–0.5 s	GPU driver initialization and memory setup
TensorRT compilation	15–30 s	Converts PyTorch model to optimized engine
Warmup (10 inferences)	0.2 s	Triggers remaining lazy initialization
Runtime overhead	0.4 s	Process startup, framework hooks, and runtime setup
Total (local, optimized)	**~1.5 s**	With precompiled TensorRT engine, warm container
Total (cloud, first deploy)	**~35 s**	Including compilation from cold state

Systems insight: Precompiling models and storing the optimized engine eliminates the 30-second compilation phase on subsequent deployments.

Without warmup, the first live request absorbs deferred JIT compilation, driver memory allocation, and kernel launch overhead. A request with a steady-state execution time of 5 ms can stall for 500 ms during an unmitigated cold start, violating strict SLOs. Controlling this initialization overhead requires systematic selection among storage, memory, and runtime loading strategies.

13.6.3 Loading strategies

Different loading strategies trade off cold start duration against serving performance and memory efficiency. The simplest approach, full loading, reads the entire model into memory before serving begins. This maximizes inference speed since all weights are immediately available, but extends cold start duration and limits model size to available memory. The approach is appropriate when cold start latency is acceptable and models comfortably fit in memory.

Memory mapping offers an alternative by mapping model files into the process address space and faulting pages in on access. It can reduce eager copying and shorten apparent startup, but a dense model will eventually touch nearly all of its weights, so deferred page faults can move loading latency into early requests. Selective benefit is greater for sparsely accessed components or model

[15] **Warmup**: Borrowed from just-in-time (JIT) compilation, where initial executions compile hot paths into optimized machine code; for ML serving, warmup inferences trigger CUDA kernel compilation, cuDNN algorithm auto-tuning, and memory pool allocation that frameworks defer until first use. Without warmup, the first live request absorbs all of this setup and can be orders of magnitude slower than steady state. During autoscaling events, this means new replicas can violate SLOs for their first several seconds of traffic.

[16] **CUDA (compute unified device architecture)**: NVIDIA's parallel computing platform (Nickolls et al. 2008), named for its goal of unifying diverse GPU shader models into a single general-purpose architecture. Before CUDA, GPU programming required disguising computations as graphics operations. The CUDA context—the data structure tracking memory allocations, loaded kernels, and device state—is the runtime's per-process gateway to GPU resources; in serverless or rapidly scaling serving systems, context creation and lazy module loading can become visible parts of cold start latency (NVIDIA 2026a).

[17] **CUDA MPS (multi-process service)**: MPS creates a control daemon that allows CUDA work from different processes to overlap on the GPU, which can improve utilization and reduce context-switching overhead when processes individually underuse the accelerator (NVIDIA 2026d). For multi-model serving, MPS can help replicas share GPU streaming multiprocessors efficiently. The trade-off is fault isolation: clients share MPS-managed GPU state, so hardware partitioning with Multi-Instance GPU (MIG) provides stronger isolation at the cost of fixed partition granularity.

repositories that load subsets on demand; latency-sensitive services still prefault or warm the pages needed on the request path.

A third strategy, lazy initialization, defers compilation and allocation until first use. This minimizes startup time but shifts latency to the first request. Production systems often combine lazy initialization with synthetic warmup requests to trigger initialization before real traffic arrives.

13.6.4 Model caching infrastructure

Production systems cache model weights at the infrastructure level to reduce cold start for common deployment scenarios. One approach, container image embedding, bundles model weights directly in the container image. This produces a single deployment artifact and eliminates network fetches at startup, but creates large images (often 10–50 GB) that slow container pulls and consume registry storage. This approach works best for models that rarely update.

For organizations with many models and frequent updates, a shared filesystem (EFS, GCS FUSE) containing versioned model weights provides a more flexible alternative. Multiple replicas can read the same artifacts without baking each version into a container. Updates still require an explicit, atomic version switch and rollout; allowing files to change underneath live replicas would undermine reproducibility. The trade-off is that network latency affects cold start and filesystem availability becomes a critical dependency.

When cold start latency is critical for high-traffic models, node-local SSD caching prepopulates local SSDs on inference nodes with frequently-used models. This approach provides fast loading from Non-Volatile Memory Express (NVMe) drives at several GB/s without network dependency, but requires cache management to handle model updates and capacity limits. Infrequent updates favor container embedding, frequent updates favor shared filesystems, and performance-critical deployments benefit from local caching with background refresh.

13.6.5 Multi-model serving

Production systems often serve multiple models from a single machine, whether different model versions for A/B testing, ensemble components, or entirely different models sharing infrastructure. GPU memory becomes the limiting resource, requiring careful management strategies.

Three strategies address multi-model memory management. Time-multiplexing loads one model at a time and swaps based on request routing: this approach is simple but introduces swap latency. Memory sharing partitions GPU memory among models, limiting concurrent execution count but enabling more models to remain resident. Model virtualization, as implemented by frameworks like Triton, separates model lifecycle from application code through model repository and control APIs for loading, unloading, and versioning models (NVIDIA 2024d, 2026c, 2026b). The choice depends on request patterns: if models receive traffic evenly, concurrent loading works; if traffic is bursty and model-specific, time-multiplexing with explicit preloading reduces average latency while maximizing GPU utilization.

13.6.5.1 Partitioning one GPU across models

When multiple models or multiple instances of the same model must run concurrently on a single GPU, the hardware must partition resources between them. NVIDIA's Multi-Instance GPU[18] technology enables hardware-level isolation, dividing an A100 into up to seven independent GPU instances, each with dedicated memory and compute resources (NVIDIA 2026e). MIG is available on A100, A30 (up to four instances), H100, H200, and newer data center GPUs. For older GPUs such as V100 or T4, CUDA stream scheduling permits overlapping or time-multiplexed work without hardware partitioning. The choice depends on whether consistent latency with MIG or maximum utilization with shared streams is the priority.

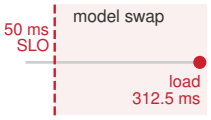
13.6.5.2 Model swapping and host memory

When the aggregate size of all models exceeds GPU memory capacity, the serving system must swap models between host dynamic random-access memory (DRAM) and device memory (VRAM) on demand. This introduces a new latency component determined by the PCIe bus bandwidth.

For a 10 GB model on PCIe Gen4 x16 (32 GB/s theoretical bandwidth), loading takes at least 312.5 ms before deserialization, graph setup, or warmup. This raw transfer latency dwarfs a tight

[18] **MIG (multi-instance GPU)**: Introduced with NVIDIA's A100 (NVIDIA Corporation 2020a) and documented across supported data-center GPUs (NVIDIA 2026e), MIG partitions a single physical GPU into independent instances, each with dedicated compute and memory resources; unlike software sharing (MPS or time-slicing), MIG provides hardware-level isolation between partitions. The trade-off is granularity—partitions must follow fixed profiles, so resources cannot be divided arbitrarily. For multi-model serving, MIG reduces noisy-neighbor risk on shared hardware, while per-model SLO guarantees still depend on scheduler policy, load, and the selected partition profile.

service-level objective such as a 50 ms deadline by more than 6×, rendering on-demand model swapping unviable on the interactive request path.



Model loading lands far outside a tight serving SLO.

To accelerate this transfer path, systems keep the weights in pinned memory (page-locked host RAM). The GPU's DMA copy engines require the source pages to stay at fixed physical addresses for the whole transfer, but ordinary heap memory is pageable, and the operating system may relocate or swap its pages at any time. A transfer from a pageable buffer therefore takes two stages: the CPU first copies the data into a pinned staging buffer, and the DMA engine then moves that buffer across PCIe. Allocating model weights with `cudaHostAlloc` removes the CPU staging copy; achieved bandwidth still depends on transfer size, memory locality, and contention. What serving adds is a capacity trade-off. Pinned memory cannot be paged out or reclaimed by the OS under memory pressure, so every pinned model holds its share of physical host RAM until it is released. High-throughput serving systems therefore pin active model weights and I/O ring buffers while leaving standby model checkpoints in pageable memory.

↩ Pinned memory and the DMA staging copy it avoids are established in section 7.5.1.3.

Once a model is loaded, warmed up, and producing predictions consistent with training, request grouping becomes the next optimization lever. Dynamic batching directly governs the balance between hardware efficiency and tail latency in queuing formulations.

13.7 Throughput Optimization

Consider a representative ResNet-50 classifier on a V100 GPU operating at batch size one. Each forward pass streams the model's weights from high-bandwidth memory to serve a single image and issues dozens of small kernels whose fixed launch cost is paid once per image, leaving neither weight traffic nor launch overhead shared across inputs. During each batch-1 forward pass, the GPU achieves only 15 percent compute execution efficiency (the fraction of streaming multiprocessor execution pipelines active during kernel execution), despite processing requests continuously at 200 images per second. Processing 32 images at once reaches 95 percent active compute efficiency and 1,280 images per second, a 6.4× throughput improvement on identical hardware, because the loaded weights are reused across inputs in on-chip memory, amortizing both memory bandwidth demands and kernel launch overheads. This in-kernel execution efficiency must not be confused with serving utilization: a single worker processing batch 1 at 200 requests per second is 100 percent busy in queueing terms ($\rho_{\text{serv}} = 1.0$) with zero spare serving capacity, yet uses its internal arithmetic pipelines at only 15 percent efficiency. **Batching**[19] is the primary lever for improving serving economics, but its operational constraints differ sharply between training and serving (Crankshaw et al. 2017). *Training* batches maximize throughput by grouping hundreds or thousands of samples together with no constraint on individual sample latency. *Serving* batches must balance accelerator saturation against individual request deadlines, ensuring that no early-arriving input waits excessively for a batch to form. An inference server resolves this tension through **dynamic batching**, adjusting batch size at runtime to match incoming request arrivals.

[19] **Batch**: From Old French *bache* (a quantity baked at one time), entering computing in the 1950s for jobs processed together without human interaction. The ML serving usage preserves the original trade-off: grouping requests amortizes fixed costs (kernel launch, weight movement) across multiple inputs, but an online request may wait for a batch to form. Training can use batches of hundreds or thousands when memory permits; serving batch sizes range from one to much larger values depending on the model, arrival rate, and latency SLO.

Definition 13.5: Dynamic batching

Dynamic batching is the ML serving optimization that trades latency for throughput under stochastic arrival patterns.

1. **Significance**: By buffering requests into a batching window, the scheduler amortizes fixed overheads (L_{lat}) across multiple inputs, pushing the system away from the memory-bound regime (BW) toward the compute-bound regime (R_{peak}).
2. **Distinction**: Unlike fixed-size serving batches, dynamic batching adjusts the realized batch size at inference time according to arriving traffic, a timeout, and a maximum-batch limit.
3. **Common pitfall**: A frequent misconception is that batching "always helps." In reality, there is a latency-throughput Pareto frontier: if the batching window is too large, the increased queuing delay may violate the system's SLO before the throughput gains are realized.

13.7.1 Why batching helps

For workloads that underuse the device at small batches, increasing batch size can improve efficiency (Shen et al. 2019). A single request can leave compute units idle when its operations expose too little parallel work or weight reuse; large spatial operations and smaller accelerators may already run efficiently at batch one. Batching amortizes fixed costs across multiple requests and enables parallel execution across the batch dimension.

Two fixed costs dominate at small batch sizes. Kernel launch overhead[20] is the time for the CPU to prepare and submit work to the GPU. The number of launches depends on graph lowering and fusion rather than the layer count: the CPU must assemble kernel parameters, copy them to GPU-accessible memory, and signal the GPU to begin execution. This overhead is typically 5–20 µs per kernel, independent of batch size. For an illustrative ResNet-50 execution with fifty kernel launches, this range contributes 250–1000 µs per inference. At batch size 1, this overhead may exceed the actual compute time; at batch size 32, the same overhead is amortized across thirty-two images. Weight loading reads model parameters from GPU memory (VRAM) to the compute units. In a weight-streaming approximation, batch size 1 reads the weights for one image; batch size 32 reuses that weight stream across thirty-two images, increasing weight-only operational intensity by 32×. Activation traffic, caches, and workspace determine the executed operation's full intensity. Measuring batching efficiency on a concrete model quantifies how these fixed costs amortize in practice.

Larger batches improve hardware efficiency but inflate per-request latency. The engineering objective is to identify the operating point that maximizes sustainable throughput while satisfying the request deadline. For the ResNet-50 benchmark quantified in table 13.9 below, serving batch size 8 with a 10 ms batching window requires a mean formation-plus-inference budget of approximately 14 ms: an arriving request waits on average half the window (5 ms, as derived in section 13.7.4.1) and requires roughly 9 ms of inference. If the service must meet a 20 ms SLO, the remaining 6 ms provides headroom for worker backlog and auxiliary pipeline stages, while delivering 4.4× higher service throughput than batch 1 on identical hardware. To illustrate why batch size cannot be increased indefinitely, figure 13.6 models this relationship across batch sizes on a logarithmic axis. To isolate the planning knee under worst-case scheduling conditions, the figure assumes a strictly serialized single worker that waits out the full batching window before executing each batch, without pipelined overlap (Latency $= T_{\text{window}} + T_{\text{svc}}(B)$). Under this unpipelined assumption, batch-1 throughput is 64 requests per second rather than the 200 images per second sustained when inference runs back-to-back. Throughput rises sharply as batch size approaches 32 while latency stays under 35 ms. Beyond that planning knee, incremental gains in arithmetic intensity diminish while batch execution time accelerates; at the largest batches, throughput actually declines while latency climbs steeply to 300 ms, demonstrating that oversized batches degrade both throughput and latency.

The vertical dashed marker at batch size 32 in figure 13.6 marks the point where increasing batch execution time outpaces batch-size growth. This illustrates a universal systems trade-off rather than an immutable operating setting. Under unpipelined serial dispatch, throughput is bounded by $B/(T_{\text{window}} + T_{\text{svc}}(B))$. In contrast, when the serving runtime pipelines batch formation with accelerator execution, saturated throughput depends solely on accelerator service time ($\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$), achieving the higher capacities documented in table 13.9.



Napkin Math 13.4: ResNet-50 batching efficiency

Table 13.9 illustrates the throughput-latency trade-off for a ResNet-50 workload on an NVIDIA V100 across batch sizes one through thirty-two. In this analysis, the utilization column measures *in-kernel compute efficiency*—the fraction of streaming multiprocessor pipelines actively executing arithmetic instructions during the inference burst. This execution metric must be distinguished from the server's overall queueing utilization ($\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$, section 13.5.2) and its wall-clock duty cycle (equation 13.2). For example, a dedicated worker executing batch size 1 continuously operates at 100 percent queueing utilization with zero idle time between requests, yet its

20 **Kernel (GPU)**: CUDA borrowed this term from operating systems circa 2007 because GPU functions represent the computational "core" of parallel algorithms. Unlike OS kernels that run continuously, GPU kernels are discrete units of parallel work launched by the CPU, so each one carries a host-side launch cost that serving pays per request unless batching or graph capture amortizes it.

↩ Per-kernel launch overhead, and why fusion and graph execution attack it, are established in section 7.3.4.2.

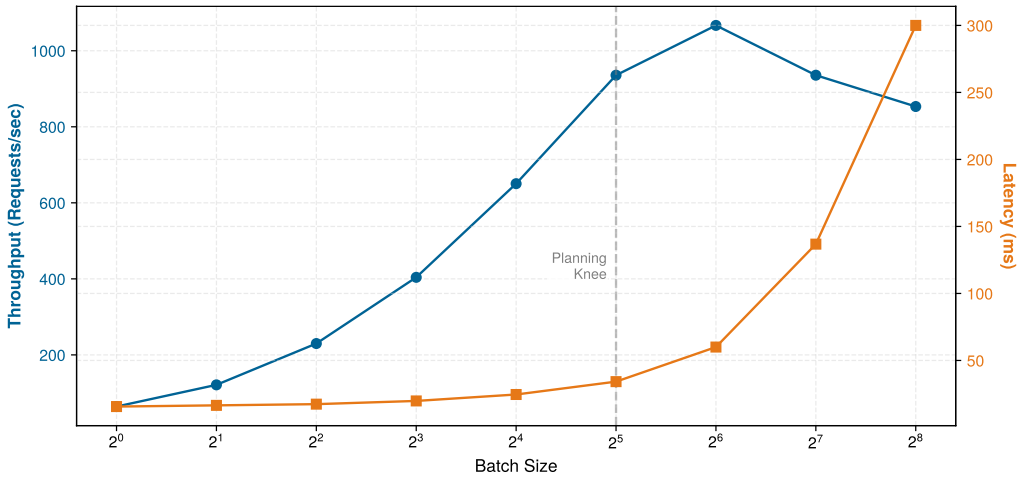


Figure 13.6: The Throughput-Latency Knee: Batch size vs. throughput (blue) and latency (orange). Throughput initially increases with batch size, then peaks and declines when batch latency grows faster than batch size. Latency rises sharply beyond the marked planning knee. Each throughput value is derived from batch size divided by the plotted formation-plus-service latency. Throughput assumes a single worker that forms and serves each batch serially; pipelined serving sustains higher throughput. Values are representative and depend on model/hardware.

internal hardware units achieve only 15 percent compute efficiency because small batches cannot saturate the GPU's parallel datapaths.

Math: For the batch-8 row, dividing batch latency by batch size yields per-image compute: 9.1 ms ÷ 8 images ≈ 1.1 ms per image. Dividing batch size by batch latency yields throughput: 8 images ÷ 9.1 ms ≈ 879 img/s. The same two relations apply to every derived row.

Table 13.9: ResNet-50 batching sweep: Illustrative per-image compute, throughput, and active GPU compute utilization (in-kernel SM efficiency) across batch sizes one through thirty-two on a V100. Throughput grows 6.4× from batch one to batch thirty-two as compute efficiency climbs from 15 percent to 95 percent, while pure inference time stretches from 5 ms to 25 ms. In-kernel efficiency measures hardware resource engagement during the forward pass and is distinct from server queueing utilization or wall-clock duty cycle.

Batch Size	Inference Time	Per-Image Compute	Throughput	GPU Util.
1	5 ms	5 ms	200 img/s	15%
4	7.2 ms	1.8 ms	556 img/s	42%
8	9.1 ms	1.1 ms	879 img/s	65%
16	14 ms	0.9 ms	1,143 img/s	85%
32	25 ms	0.8 ms	1,280 img/s	95%

The times shown are pure inference time, excluding queue wait; section 13.7.5 analyzes how user-perceived latency includes batching-window wait.

Systems insight: Batch size 32 achieves 6.4× higher throughput than batch size 1. However, user-perceived latency includes both queue wait and inference time. With a 10 ms batching window and 25 ms inference, formation plus inference reaches 35 ms, excluding resource backlog and other request stages, vs. 5 ms at batch size 1.

13.7.2 The batching tax: The latency-throughput frontier

The throughput gains demonstrated in table 13.9 incur a deliberate systems cost. Every request in a batch pays a **batching tax**: an engineered queueing delay introduced to elevate hardware utilization. The performance gain arises from parameter reuse across inputs. Evaluating B independent queries simultaneously converts memory-bound matrix-vector products into compute-dense general matrix

multiplications (GEMM), raising arithmetic intensity toward the hardware roofline ceiling (Principle 6). Conversely, the operational tax accumulates in the queue, because assembling B distinct queries forces earlier arrivals to sit idle while waiting for subsequent requests to appear.

Formalizing this trade-off requires extending the single-server queuing notation. Let $T_{\text{svc}}(B)$ denote the execution duration for a batch of size B. The unbatched service rate is $\mu = \mu_{\text{eff}}(1) = 1/T_{\text{svc}}(1)$, while batching B inputs yields an effective aggregate throughput of $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$. In a request-triggered fixed-batching system, end-to-end response time decomposes into two distinct penalties: **batch formation delay** ($L_{\text{lat,form}}$), the time earlier requests wait while the batch accumulates to capacity, and **inference inflation**, the modest increase in kernel execution duration $T_{\text{svc}}(B)$ as batch size expands from 1 to B. Under stationary Poisson arrivals at rate λ_{arr}, the first request in a batch waits an average of $(B-1)/\lambda_{\text{arr}}$ seconds while the final arrival waits zero, yielding an average formation delay of $(B-1)/(2\lambda_{\text{arr}})$. Assuming an execution worker is immediately available upon batch assembly, equation 13.6 models the resulting latency-throughput Pareto frontier:

$$L_{\text{lat,batch}} \approx \underbrace{\frac{B-1}{2\lambda_{\text{arr}}}}_{\text{Formation delay}} + \underbrace{T_{\text{svc}}(B)}_{\text{Inference time}} \tag{13.6}$$

Equation 13.6 formalizes the component trade-off between accelerator throughput and individual request latency, assuming an idle worker is immediately available upon batch assembly. Increasing B raises arithmetic intensity and amortizes memory bandwidth across inputs, but introduces formation delay for early arrivals. Evaluated as a formation-plus-service component comparison using the service times from table 13.9 at 500 QPS, moving from batch 1 to batch 32 increases formation wait from 0 ms to 31 ms, yielding a 11× growth in isolated stage latency (5 ms to 56 ms). However, batch-one service cannot sustain 500 QPS on a single worker whose peak service rate is only 200 requests per second; without batching, worker backlog diverges toward infinity, so 5 ms does not represent a viable steady-state response time at this arrival rate. Batching is what makes 500 QPS feasible on that hardware in the first place, demonstrating that grouping requests can lower loaded end-to-end response time even while adding formation delay. Sizing B is therefore an operational stability screen before it is an economic optimization: operators first enforce queue stability ($\lambda_{\text{arr}} < \mu_{\text{eff}}(B)$) and then select the batch size that maximizes throughput within the memory ceiling and latency SLO ($L_{\text{lat,SLO}}$).

Equation 13.6 assumes the batcher waits for exactly B requests. Under fluctuating traffic that assumption is itself the problem, because a slow period can leave early arrivals waiting indefinitely, and how serving systems relax it governs the choice of batching strategy.

13.7.3 Static vs. dynamic batching

Static batching fixes the batch dimension at compile time and holds execution until a queue accumulates exactly B requests. In offline batch processing and model training, static shapes allow compilers to allocate static memory buffers and emit optimized compute kernels that maximize weight reuse and arithmetic intensity. In interactive serving, however, client requests arrive stochastically. Under low or fluctuating arrival rates, static batching exposes requests to unbounded queuing delay while waiting for the batch to fill. Conversely, setting a small fixed batch size to curtail latency leaves accelerator compute units underutilized and fails to amortize weight transfers from high-bandwidth memory when traffic spikes.

Dynamic batching decouples batch formation from arrival determinism by buffering incoming requests behind a dual-trigger scheduler (Olston et al. 2017; NVIDIA 2024d). Rather than waiting indefinitely for a fixed tensor shape, the serving runtime makes a batch eligible for dispatch when either of two thresholds is satisfied: the queue accumulates a maximum batch size B_{max}, or a timeout window T_{window} expires relative to the oldest buffered request. When traffic bursts, the queue fills to B_{max} almost instantaneously, achieving high arithmetic intensity with minimal queuing delay. When arrival rates drop, the timeout window forces the dispatch of an undersized batch ($B < B_{\text{max}}$), bounding formation time at the expense of per-request compute efficiency. If the accelerator is busy, the ready batch can still wait for service. The timeout window T_{window} and maximum batch size B_{max} form the primary operational levers: shorter windows limit formation delay, whereas longer windows allow larger batches when enough requests arrive. A longer window is useful only when the resulting throughput gain remains within the latency and memory constraints.

13.7.4 Dynamic batching latency-throughput trade-offs

Dynamic batching introduces a quantifiable tension between throughput optimization and latency constraints. Under overload, the mechanism is queue growth rather than slower inference, which enables systematic configuration decisions instead of trial-and-error tuning.

> **Systems Perspective 13.4: Why latency spikes under load**
>
> Little's Law ($Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$) relates long-run averages only while the queue is stable (section 13.5.1). As the arrival rate approaches the maximum service rate μ, the stable queue's depth and latency grow rapidly. If λ_{arr} exceeds capacity for a sustained interval, no finite steady state exists: backlog accumulates until traffic falls, capacity grows, or work is rejected. This is why admission control can preserve latency during overload; scale-out or degraded service can also help when available.

Equation 13.7 decomposes the modeled batching-and-service portion of request latency into two components. Its total excludes network transit and any host stages outside the service interval:

$$L_{\text{lat,batch}} = L_{\text{lat,wait}} + T_{\text{svc}}(B) \tag{13.7}$$

where $L_{\text{lat,wait}}$ is the time spent waiting in the batching queue (corresponding to $L_{\text{lat,queue}}$ in the latency budget of principle 12) and $T_{\text{svc}}(B)$ is the service time for a batch of size B (encompassing $L_{\text{lat,infer}}$ plus portions of $L_{\text{lat,pre}}$ and $L_{\text{lat,post}}$). This wait includes formation and any subsequent wait for a worker. The batching window T_{window} bounds formation time alone; $L_{\text{lat,wait}} \leq T_{\text{window}}$ holds only when no additional resource backlog delays dispatch. Batch size affects service time through weight reuse, parallel work, and execution overhead.

13.7.4.1 Quantitative analysis of batching

Configuring a dynamic batching policy requires balancing batch formation delay against accelerator execution time and pending queue backlog. In a pure count-triggered batcher (equation 13.6), average formation delay $(B-1)/(2\lambda_{\text{arr}})$ contracts as arrival rate λ_{arr} grows, but diverges under light traffic. Conversely, a pure timer-triggered batcher dispatches on a periodic schedule of length T_{window}, decoupling formation delay from instantaneous arrival fluctuations.

To evaluate how time-based batching impacts latency budgets, consider a baseline fixed calendar window where batches dispatch periodically every T_{window} seconds and an idle execution worker is available at the boundary. Under stationary Poisson arrivals, query arrival times distribute uniformly across the interval $[0, T_{\text{window}}]$. A request arriving at offset t within the window experiences a formation wait of $T_{\text{window}} - t$. Integrating across the uniform distribution yields an expected waiting time equal to exactly half the batching window (equation 13.8):

$$\mathbb{E}[L_{\text{lat,wait}}] = \frac{T_{\text{window}}}{2} \tag{13.8}$$

A 20 ms batching window adds 10 ms average wait (up to 20 ms for the first arrival in a window; later arrivals wait less) regardless of batch size achieved. For a 50 ms mean latency SLO with 5 ms inference, the average wait consumes 20 percent of the latency budget before any computation begins; tail SLOs must budget the full window.

13.7.4.2 Batch size distribution

For fixed calendar windows, the number of requests collected during T_{window} follows a Poisson distribution with mean $\lambda_{\text{arr}} T_{\text{window}}$. Equation 13.9 formalizes this relationship:

$$\Pr(\text{batch size} = k) = \frac{(\lambda_{\text{arr}} T_{\text{window}})^k e^{-\lambda_{\text{arr}} T_{\text{window}}}}{k!} \tag{13.9}$$

Here k is a nonnegative integer count of arrivals in the window.

Table 13.10 quantifies this variability, showing how batch size fluctuates for different traffic levels with a fixed 10 ms window:

Table 13.10: Batch Size Variability: At low traffic, fixed calendar windows frequently contain zero requests, so no GPU batch is launched. At moderate traffic, batch sizes fluctuate significantly around the mean. High traffic provides more stable batching, and the probability of batches reaching at least twice the mean size decreases as traffic grows (from 39 percent at 50 QPS to 0.3 percent at 1000 QPS), reflecting the law of large numbers.

Arrival Rate	Mean Batch	Std Dev	$\Pr(\textbf{batch} = 0)$	$\Pr(\textbf{batch} \geq 2 \times \textbf{mean})$
50 QPS	0.5	0.7	61%	39%
200 QPS	2	1.4	14%	14%
500 QPS	5	2.2	0.7%	3%
1000 QPS	10	3.2	0.005%	0.3%

13.7.4.3 Throughput maximization strategy

Throughput optimization requires separating per-request latency from saturated service capacity. A request waits for batch formation, then pays the service time of the formed batch. Under sustained load, however, batch formation can overlap with the previous batch's execution, so capacity is governed by the ready-batch service time:

$$\mu_{\text{eff}}(B) \approx \frac{B}{T_{\text{svc}}(B)}, \qquad \text{throughput} = \min(\lambda_{\text{arr}}, \mu_{\text{eff}}(B)) \tag{13.10}$$

In equation 13.10, λ_{arr} is the offered arrival rate and $\mu_{\text{eff}}(B)$ is the saturated service capacity for batch size B. The numerator increases linearly with batch size while service time often increases sublinearly over a useful range because larger batches amortize fixed kernel-launch overheads and DRAM weight transfers, shifting execution from memory-bandwidth-bound matrix-vector operations toward compute-bound matrix-matrix multiplication. The batching window still appears in request latency and in low-traffic regimes, where the expected batch size is limited by arrivals during the window, roughly $\lambda_{\text{arr}} T_{\text{window}}$.

To analytically evaluate batching strategies across arbitrary batch sizes without interpolating discrete benchmark tables, ResNet-50 execution time on a V100 GPU can be modeled via the affine approximation $T_{\text{svc}}(B) = 5\ \text{ms} + (0.6\ \text{ms})\, B$ (5 ms fixed kernel launch and parameter-staging overhead plus 0.6 ms per image). This analytical service model unifies the queueing, window screening, and dispatch calculations throughout the remainder of this section. With a $T_{\text{window}} = 10$ ms batching window, table 13.11 uses this explicit service function to evaluate the trade-off between formation-plus-service latency and saturated capacity, where latency includes the full formation window plus execution but excludes worker queueing, and throughput reflects saturated pipelined execution:

Table 13.11: Batching Throughput Analysis: Scenario analysis for ResNet-50 throughput on V100 with 10 ms batching window. Throughput increases 7.4× from batch size one to 32 (179 img/s to 1322 img/s), but total latency more than doubles (15.6 ms to 34.2 ms). The optimal configuration depends on whether the latency SLO or throughput target is the binding constraint.

Batch Size	Service Time	Total Latency	Throughput	Efficiency
1	5.6 ms	15.6 ms	179 img/s	Low
4	7.4 ms	17.4 ms	541 img/s	Moderate
8	9.8 ms	19.8 ms	816 img/s	Good
16	14.6 ms	24.6 ms	1096 img/s	High
32	24.2 ms	34.2 ms	1322 img/s	Maximum

The throughput gains in table 13.11 trace directly back to the fixed-overhead term L_{lat} of the iron law (principle 3), which batching amortizes across requests.

Example 13.7: The iron law of batching efficiency

Context: In serving, batching improves throughput by amortizing fixed per-batch scheduling and kernel-launch work; it can also amortize weight traffic across requests, while queue wait remains a separate latency cost. Holding the data-movement term outside this illustrative two-term comparison, equation 13.11 isolates compute and fixed overhead:

$$T = \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \tag{13.11}$$

Analysis:

- **Case 1 (batch 1)**: Overhead (5 ms) $\gg$ Compute (0.6 ms). Efficiency $\approx$ 10 percent. The GPU is mostly waiting.
- **Case 2 (batch 32)**: Overhead (5 ms) $\ll$ Compute (19.2 ms). Efficiency $\approx$ 79 percent. The GPU is crunching numbers.

Systems insight: Within this two-term model, batching offers diminishing throughput gains once fixed overhead becomes negligible ($<$ 10 percent of total time). Treat that point as a candidate for measurement; stop earlier if memory capacity or the latency SLO binds, or if measured useful throughput declines. Overhead amortization alone does not identify the best serving configuration.

These results compose a calendar-window planning model: average formation wait is half the window, Poisson arrivals make the window count fluctuate around $\lambda_{\text{arr}} T_{\text{window}}$, and saturated service capacity $\mu_{\text{eff}}(B)$ grows as batching amortizes fixed costs. An oldest-request timer, a maximum-batch trigger, or a busy worker changes the observed wait and batch distributions. Mean-latency screening can narrow candidate configurations; a percentile SLO requires their tail behavior under the intended dispatch policy and load.

13.7.4.4 Latency-constrained optimization

When latency is the binding constraint, a mean-latency approximation provides an initial screen of candidate batch sizes. For the portion of a mean latency SLO allocated to batching and service, $L_{\text{lat,budget}}$, and average wait time $T_{\text{window}}/2$, equation 13.12 defines the maximum allowable batch size B_{SLO} using a first-order average latency approximation. Other request stages require separate budget allocations. This is a feasibility bound, not a throughput optimum, and it screens a mean budget rather than certifying a p95 or p99 target:

$$B_{\text{SLO}} = \max\left\{B : \frac{T_{\text{window}}}{2} + T_{\text{svc}}(B) \leq L_{\text{lat,budget}}\right\} \tag{13.12}$$

Applying equation 13.12 to the affine service model for a 50 ms target deadline illustrates how selecting candidate batching windows shapes the feasible configuration space. Table 13.12 contrasts two concrete design points: a conservative 5 ms window and an aggressive 25 ms window.

Table 13.12: Illustrative batching window trade-off: How two assumed batching configurations trade average wait, inference budget, batch-size cap, and saturated capacity.

Metric	Conservative (T_{window} = 5 ms)	Aggressive (T_{window} = 25 ms)
Average wait	2.5 ms (max wait = 5 ms for the first request in a window)	12.5 ms
Latency budget for inference	47.5 ms (mean-latency planning; tail SLOs should budget the full window)	37.5 ms
Batch size cap	32 images	48
Assumed saturated capacity	~1,140 img/s	~1,280 img/s

This comparison quantifies the diminishing returns of expanding the batching window. Widening the window by 20 ms delivers only 12.3 percent higher saturated capacity while increasing average

formation wait by 10 ms. However, this initial screen relies on mean formation wait and deterministic batch sizing. In production, stochastic arrivals introduce batch-size variance that can breach tail deadlines even when the mean latency appears well within the budget, requiring the percentile envelope analysis developed next.

13.7.4.5 SLO violation analysis

Batch size variability can cause SLO violations even when mean latency appears safe. A request in a fixed window sees itself plus the other arrivals, so $B_{\text{req}} = 1 + \text{Poisson}(\lambda_{\text{arr}} T_{\text{window}})$. With an available worker at dispatch, combining the full window with the request-seen p99 batch size gives the conservative formation-plus-service envelope in equation 13.13:

$$L_{\text{lat,envelope}} = T_{\text{window}} + T_{\text{svc}}(B_{\text{req,p99}}) \tag{13.13}$$

For λ_{arr} = 500 QPS and T_{window} = 10 ms, the mean request-seen batch size is 6 and its p99 is 12. The mean latency is 13.6 ms; the conservative envelope is 22.2 ms.

The envelope is 1.63× the mean, which is the lesson of this subsection: a configuration whose mean sits comfortably inside the SLO can still breach it when a full window coincides with an unusually large batch. Under the available-worker calendar-window assumptions, the envelope conservatively screens formation plus service because the longest window wait and largest batch need not hit the same request. A passing configuration clears only this screen; acceptance still requires evaluating worker backlog and the other request stages under the intended traffic and dispatch policy. A failing configuration can be rejected conservatively or investigated with a more faithful model and load test. Neither outcome alone establishes compliance with the end-to-end SLO.

> **Systems Perspective 13.5: The latency-throughput trade-off**
>
> A single "inference speed" number is undefined until the batch size is named, because batch size selects which regime, and which bottleneck, the system operates in.
>
> - **Batch-1 regime**: Latency-oriented. Launch overhead or memory traffic often limits the request path. This regime governs real-time interaction such as typing helpers and robotics.
> - **Batch-N regime**: Throughput-oriented. Larger batches amortize overhead and weight traffic and may shift the bottleneck toward compute. This regime governs offline processing and high-traffic services.
>
> The two regimes optimize opposite quantities, so a model that is "fast" at batch 1 may be far from peak throughput, and vice versa. Any latency or throughput figure must therefore specify whether it was measured at single-stream latency (batch 1) or maximum throughput (batch N).

13.7.4.6 Adaptive batching windows

Static batching windows impose a rigid trade-off that penalizes both traffic extremes. Under heavy traffic surges, a static timeout window holds early arrivals unnecessarily long after sufficient queries have already accumulated, squandering precious milliseconds of the latency budget. Conversely, during off-peak periods, a fixed window frequently dispatches tiny, inefficient batches, failing to exploit accelerator arithmetic intensity. To navigate this tension, serving runtimes employ **adaptive batching** policies that dynamically modulate window duration based on instantaneous queue backlog, observed arrival rates, and the residual latency margin before the service-level objective expires.

Listing 13.2 illustrates an adaptive batching heuristic. In a benchmark scenario where traffic oscillates between 200 and 1,000 QPS, a static 10 ms window yields an average latency of 15 ms at 650 img/s, whereas an adaptive window contracts during surges and expands during lulls, achieving 11 ms average latency at 680 img/s. Modulating the batching window dynamically expands the operational throughput envelope while strictly guarding tail-latency budgets.

Listing 13.2: Adaptive Batching Window: Heuristically adjusts batch timeout based on queue depth, arrival rate, and the remaining latency budget.

```
def adaptive_batching_window(
    queue_depth, arrival_rate, slo_ms, service_ms, fixed_overhead_ms
):
    """Compute a heuristic batching window.

    Based on current system state.
    """
    target_batch_size = 16  # Deployment-tuned target batch

    # Fast path: batch ready, close immediately to minimize latency
    if queue_depth >= target_batch_size:
        return 0

    # Compute maximum allowable wait from the remaining p99 budget.
    max_wait_ms = max(0, slo_ms - service_ms - fixed_overhead_ms)

    # Estimate time to accumulate target batch at current arrival
    # rate.
    # arrival_rate is requests/second, so convert seconds to
    # milliseconds.
    if arrival_rate > 0:
        requests_needed = target_batch_size - queue_depth
        estimated_wait_ms = requests_needed / arrival_rate * 1000.0
        # Return minimum of estimated wait and SLO-constrained maximum
        return min(estimated_wait_ms, max_wait_ms)

    return max_wait_ms  # Low traffic: use remaining budget to accumulate batch
```

The heuristic selects a candidate formation timer. Its inputs omit time already spent by the oldest request and work queued ahead of the batch, so the returned window does not establish a remaining deadline allowance. Deadline-aware dispatch must account for both quantities and validate the service and outside-worker budgets under load.

The nondominated batching configurations form a Pareto frontier where improving throughput requires accepting higher latency. Table 13.13 shows five illustrative configurations:

Table 13.13: Illustrative Batching Pareto Frontier: The five assumed configurations trade tail latency for throughput. From the first to the last, assumed throughput rises by 52 percent and assumed p99 latency rises to 5.4× its initial value.

Window (ms)	Max Batch	Avg Latency	p99 Latency	Throughput	Configuration
2	16	8 ms	18 ms	890 img/s	Ultra-low latency
5	32	10 ms	22 ms	1,140 img/s	Balanced
10	32	15 ms	35 ms	1,240 img/s	Moderate latency
20	64	23 ms	52 ms	1,310 img/s	Throughput-optimized
50	128	38 ms	98 ms	1,350 img/s	Maximum throughput

13.7.4.7 Practical configuration guidelines

The Pareto frontier in table 13.13 illustrates why configuration must work backward from the latency budget: past the knee, widening the window buys diminishing throughput for sharply rising tail latency. In the worked scenario, reserving 30 percent of the SLO for batching wait gives $T_{\text{window}} \leq 0.3 \times L_{\text{lat,SLO}}$; another workload may allocate a different fraction after accounting for network, preprocessing, inference, and postprocessing. The traffic estimate should include peaks and burst duration rather than rely only on the mean. GPU memory supplies a separate ceiling on batch size through activations and runtime workspace. Finally, monitoring the realized batch-size and wait-time distributions reveals whether the assumed arrival model holds and whether a fixed or adaptive window is appropriate.

For an illustrative ResNet-50 serving system with a 50 ms SLO and 500 QPS traffic, the calculation provides starting estimates for two configuration knobs: the batching window and maximum

batch size. Table 13.14 summarizes the resulting configuration and modeled operating point. The batch statistics and tail envelope use the calendar-window model without worker backlog; an oldest-request timer and maximum-batch trigger require a load test of their actual distributions. Its saturated-capacity row derates the ideal batch-32 rate by an assumed efficiency factor of 0.89, standing in for scheduling gaps and batches that dispatch before they fill.

Table 13.14: Practical Batching Configuration: A 30 percent allocation sets a 15 ms maximum window; this illustrative scenario selects a 12 ms window below that bound and assumes a batch-32 cap. The reported latency is a conservative envelope, not the request-latency p99.

Quantity	Value	Engineering role
Latency budget for batching	15 ms	Portion allocated to batch formation.
Maximum window	15 ms	Upper bound implied by the latency budget.
Expected request-seen batch	7	Average batch seen by an arriving request.
Request-seen p99 batch	13	Tail batch size under Poisson arrivals.
Assumed memory-limited batch	32	Scenario cap for accelerator memory.
Selected configuration	$T_{\text{window}} = 12$ ms, $B_{\max} = 32$	Candidate knob setting for load testing.
Conservative tail envelope	24.8 ms	Full window plus request-seen p99 batch service time.
Illustrative saturated capacity	1,176.9 img/s	Ideal batch rate derated by the efficiency factor.
Offered throughput	500 img/s	Offered load; acceptance requires load validation.



Example 13.8: From a batching window to a dispatch decision

Problem: In a hypothetical trace of the selected dual-trigger configuration, four requests arrive at 0, 2, 5, and 9 ms. No further requests arrive before dispatch. The worker becomes available at 18 ms, and the remaining response path takes 4 ms. Does the oldest request meet the 50 ms deadline under the planning service-time model?

Analysis: The batch remains below the maximum count, so the oldest-request timer makes it eligible after 12 ms. The worker is still busy, adding 6 ms of resource-queue wait. The four-request batch then takes 7.4 ms under the same linear service model used for the window analysis. Adding the remaining response path gives 29.4 ms from the oldest arrival to completion.

Result: Dispatching the four-request batch meets the oldest request's deadline with 20.6 ms of slack; the later arrivals finish with more slack. Waiting for the maximum batch would leave these requests stalled because the trace supplies no further arrivals. The timeout therefore determines eligibility, while worker availability determines actual dispatch.

Systems insight: The selected window passes this complete trace, including backlog and the response path. Establishing a percentile SLO requires repeating the accounting over the intended traffic, service-time variation, and input mix, with rejected requests and fallback quality reported alongside completed-request latency.

Working backward from the budget can also conclude that the right window is zero, because batching is not universally optimal. Schedulers and dynamic memory allocators introduce coordination complexity and synchronization overhead that can degrade performance in latency-critical or resource-constrained environments. In particular, three operational constraints favor single-request processing. First, under an ultra-low p99 target, even a short batching delay consumes an unacceptable fraction of the latency budget. Second, when serving runtimes lack support for ragged (variable-length) tensors, batching variable-length inputs incurs padding overhead to match the longest sequence in the batch, whereas single-request processing eliminates padding waste entirely. Finally, memory capacity binds when model weights already consume most of the accelerator's physical memory; under near-capacity conditions, concurrent batch activations and dynamic KV-cache allocations risk out-of-memory aborts, forcing the runtime to process requests individually.

13.7.5 Traffic patterns and batching strategy

The batching strategy depends critically on how requests arrive. Different deployment contexts exhibit different arrival patterns, and the MLPerf inference benchmark supplies four standardized scenarios that approximate several common cases.

13.7.5.1 Server traffic (Poisson arrivals)

The MLPerf Server scenario models cloud/API-like inference traffic with Poisson arrivals (Reddi et al. 2019).[21] Under that model, counts in disjoint intervals are independent and the average arrival rate is constant. Equation 13.14 expresses the expected count in a fixed calendar window with rate λ_{arr} and width T_{window}:

$$\mathbb{E}[\text{batch size}] = \lambda_{\text{arr}} \cdot T_{\text{window}} \tag{13.14}$$

For fixed calendar windows, variance equals the mean, so batch sizes fluctuate significantly at moderate traffic. With $\lambda_{\text{arr}} = 200$ requests/second and $T_{\text{window}} = 10$ ms, the expected count is two, but roughly 14 percent of windows have zero requests (and launch no GPU work) while others may have four or more.

Higher traffic can fill a desired batch sooner, permitting a shorter formation window. In the calendar-window model, the expected count follows equation 13.14; an oldest-request timer also includes the request that starts the window. Increasing the arrival rate therefore allows the same expected batch count with less deliberate waiting. The scheduler may instead target a larger batch if the latency and memory budgets permit it. Neither choice creates service capacity: the formed batches must complete fast enough to keep up with sustained arrivals, or resource backlog grows despite the shorter window. Window tuning must jointly check the arrival count, the service rate at the realized batch sizes, and the full request-latency distribution.

[21] **Poisson process**: Named after French mathematician Simeon Denis Poisson (1781–1840), this stochastic model describes events occurring continuously and independently at a constant average rate. In fixed calendar windows, variance equals the mean, so with $\lambda_{\text{arr}} = 200$ req/s and a 10 ms window, the expected count is two and roughly 14 percent of windows are empty, though an efficient scheduler launches no GPU work for them. A request-triggered timer instead begins with one request and counts only additional arrivals during the window.

13.7.5.2 Streaming traffic (correlated arrivals)

Autonomous vehicles, video analytics, and robotics systems receive inputs from multiple synchronized sensors. A six-camera example makes the synchronization deadline concrete.

Example 13.9: Multi-camera autonomous vehicle serving

Scenario: An autonomous vehicle vision system processes 6-camera video streams under a strict 33 ms real-time frame deadline.

Diagnosis: Sensor arrival jitter causes variable multi-camera batch assembly delays, risking frame deadline misses. A synchronization deadline at the latest batch start that still meets the frame deadline ($t_{\text{frame}} = 16$ ms in the timeline below), with a fallback that reuses a camera's previous frame on timeout, bounds latency.

Systems lesson: Synchronized multi-sensor streaming workloads enforce deterministic real-time deadlines. Serving pipelines must implement explicit synchronization timeout policies to prevent sensor jitter from breaching latency budgets.

Strict latency budgets make synchronized multi-camera frame processing challenging. Table 13.15 traces an event-by-event timeline for one synchronized frame set across six cameras operating at 30 FPS within a 33 ms frame deadline. Sensor capture and network arrival jitter consume 15 ms before inference can begin. Batching all six camera streams into a single GPU invocation completes in 10 ms (at $t_{\text{frame}} = 25$ ms), leaving 7 ms for downstream postprocessing, planning, and safety checks. If arrival jitter pushes batch assembly past 16 ms, the pipeline breaches its real-time deadline.

Table 13.15: Multi-camera frame timeline: Event-by-event timeline for one synchronized frame set across six cameras at 30 FPS. Frames arrive over a 7 ms spread, inference cannot start until the last one lands at 15 ms, and the result reaches the planning module at 32 ms, leaving 1 ms of slack inside the 33 ms deadline.

Time	Event
$t_{\text{frame}} = 0$ **ms**	Cameras begin capturing frame N

Time	Event
$t_{\text{frame}} = 8$ **ms**	Camera 1 frame arrives
$t_{\text{frame}} = 10$ **ms**	Cameras 2–5 frames arrive
$t_{\text{frame}} = 15$ **ms**	Camera 6 arrives (jitter)
$t_{\text{frame}} = 15$ **ms**	Batch inference begins (6 images)
$t_{\text{frame}} = 25$ **ms**	Inference complete
$t_{\text{frame}} = 32$ **ms**	Result ready for planning module

13.7.5.3 Single-user traffic (sequential arrivals)

Streaming traffic correlates arrivals through sensor synchronization, making batch size and deadline externally fixed. At the opposite end of the spectrum, interactive mobile and edge applications process one request stream at a time, leaving no opportunity to batch independent queries. The MLPerf SingleStream scenario captures this sequential-serving shape. Here, the optimization target shifts from a synchronization budget under a frame deadline to per-request latency and energy consumption under a rigid thermal envelope. For the smartphone vision pipeline modeled below, the primary engineering objective is not batch formation, but minimizing per-phase latency and energy dissipation.

13.7.5.4 Mobile serving constraints

Cloud services often emphasize fleet cost and latency; mobile serving adds three closely coupled constraints. The first is an energy budget that throughput alone does not capture, because each inference consumes battery. In the modeled pipeline, 2.98 mJ at 22 FPS draws about 66 mW for the inference path alone, before camera, display, ISP, and OS overhead add to that total in a full photo app. Thermal throttling can compound this limit when sustained system power reaches a device-specific thermal boundary. Memory constraints close the set because the model shares RAM with the application and operating system. Reload time depends on device storage, runtime, and model state; quantization and memory-mapped loading (section 13.6.3) can reduce footprint and startup work but do not eliminate those dependencies.

Example 13.10: ResNet-50: Mobile serving

Table 13.16 decomposes per-phase latency and energy for a single-user mobile vision inference:

Table 13.16: Mobile ResNet-50 pipeline: Illustrative per-phase latency and energy for a single-user mobile vision inference, showing that JPEG decode on the CPU dominates the energy budget even though the NPU inference stage carries the model's compute. Optimization targets shift from throughput to energy-per-inference at the edge.

Phase	Duration	Energy	Notes
Camera buffer read	8 ms	0.08 mJ	System API
JPEG decode (CPU)	15 ms	1.5 mJ	Single-threaded
Resize + Normalize	5 ms	0.4 mJ	CPU preprocessing
NPU inference	12 ms	0.8 mJ	82% utilization
Postprocess + UI	5 ms	0.2 mJ	Result rendering
Total	**45 ms**	**2.98 mJ**	22 FPS sustained

The mobile serving node is governed by four metrics:

- **Energy per inference**: 2.98 mJ enables ~12.1M inferences per 10 Wh battery (typical smartphone)
- **Modeled inference power**: 2.98 mJ / 45 ms = 66 mW, excluding the rest of the device and therefore not a guarantee of indefinite operation

- **NPU vs. CPU trade-off**: CPU fallback replaces the 12 ms, 0.8 mJ NPU inference stage with a 45 ms, 4.2 mJ CPU stage; the full pipeline would rise from 45 ms and 2.98 mJ to about 78 ms and 6.4 mJ before additional system overhead.
- **Memory footprint**: 150 MB peak (model + activations), competing with app memory

Systems insight: In this illustrative pipeline, the mobile NPU reaches 82 percent utilization at batch size one while the data-center example reaches 15 percent. The contrast is workload- and hardware-specific, but it shows why a mobile accelerator designed for single-stream execution can operate efficiently without the batches used to amortize a large server GPU.

These physical constraints make edge serving optimization fundamentally different from data-center optimization. Rather than maximizing aggregate batch throughput, the objective is to sustain real-time execution within a tight thermal and battery envelope without triggering hardware throttling or starving foreground applications of system memory.

13.7.5.5 Traffic pattern summary

Traffic-adaptive batching adjusts the batching window as queue depth and request rate change. Table 13.17 relates the four MLPerf scenarios to representative deployment contexts and common batching strategies.

These scenarios are benchmark abstractions rather than deployment prescriptions. The operative batching policy must still follow the service's current concurrency, deadline, and power constraints.

Table 13.17: Traffic Patterns and Batching Strategies: The four MLPerf inference scenarios approximate distinct deployment contexts. Server traffic (cloud APIs) uses dynamic batching with a timeout; MultiStream evaluates grouped queries under its version-specific latency rules, while a production sensor deadline requires separate validation; SingleStream processes requests individually; Offline uses large batches subject to memory capacity to maximize throughput.

Scenario	Context	Strategy	Focus
Server	Cloud APIs, web services	Dynamic batching with timeout	Window tuning, utilization-latency curve
MultiStream	Grouped sensor-like inputs	Joint query processing	Group latency under benchmark rules
SingleStream	Mobile apps, embedded devices	No batching ($B = 1$)	Preprocessing, power efficiency
Offline	Batch processing, data pipelines	Maximum batch size	Throughput, hardware utilization

The examples developed here (cloud API traffic under the Server scenario, grouped sensor inputs motivated by MultiStream, and throughput-dominated pipelines under Offline inference) all decide how requests share the accelerator. None of them decides how efficiently the accelerator executes each batch once it forms. The same batch of the same model can run several times faster or slower depending on the engine that compiles and schedules its kernels and on the number format it computes in, and those choices are the next lever.

Checkpoint 13.3: Batching and traffic patterns

Batching is the primary lever for serving economics, but the optimal strategy depends on context.

- ☐ **Throughput-latency trade-off**: Explain why batch size 32 achieves 6× higher throughput than batch size 1, yet a production system with a 20 ms SLO might still choose batch size 8.
- ☐ **Wait vs. compute**: Given an end-to-end latency budget, can you separate batch formation, waiting for an available worker, and batch service time $T_{\text{svc}}(B)$? Which wait does a batching timeout control?

- ☐ **Dynamic vs. static batching**: Describe why static batching fails under variable traffic arrival rates, and how dynamic batching with an adaptive time window resolves queue starvation.
- ☐ **Traffic pattern matching**: Map deployment scenarios (cloud API, autonomous vehicle, mobile edge) to corresponding MLPerf scenarios, identifying the batching strategy suited to each operational constraint.
- ☐ **Adaptive windows**: For a desired expected batch count, explain why higher traffic permits a shorter formation window. How would you check whether the realized batches can keep up with arrivals, and what must a load test verify before accepting the configuration?
- ☐ **When not to batch**: Name a latency target, runtime limitation, or memory condition under which a batching window of zero is the right configuration.

13.8 Inference Runtime Selection

Dynamic batching and request scheduling dictate how inferences are grouped and queued, but an underlying execution engine must evaluate the resulting tensor operations on physical hardware. The latency budgets established in section 13.4.1 and the batching gains of section 13.7 are achievable only if the runtime maps operations efficiently to accelerator execution units and memory hierarchies. The **inference runtime**, the software layer that orchestrates tensor operations and manages hardware resources, can materially change latency and throughput for an unchanged model architecture. Runtime engineering consists of two phases: selection chooses the execution engine, and configuration tunes that engine for the target model, hardware, input shapes, and latency distribution.

13.8.1 Runtime ecosystem and configuration

Engine selection begins with the binding system constraint rather than the framework used during training. When deployment velocity and framework compatibility dominate, PyTorch and TensorFlow models can serve directly through their native runtimes without an intermediate compilation pass. This preserves support for dynamic Python control flow and custom training operators, though Python interpreter overhead, Global Interpreter Lock (GIL) contention, and non-contiguous memory allocations can restrict production throughput. To alleviate interpreter overhead while maintaining framework compatibility, serialized graph representations—such as `torch.export` and TensorFlow SavedModel (with TorchScript supporting legacy deployments)—capture deterministic operator graphs for ahead-of-time graph optimization and multi-threaded C++ runtime execution. Nevertheless, framework-native runtimes prioritize developer expressiveness and broad operator coverage, whereas specialized inference engines trade that generality for targeted graph transformations and reduced execution overhead.

13.8.1.1 General-purpose optimization

When cross-hardware portability is the binding constraint, ONNX Runtime[22] provides an intermediate, hardware-agnostic execution layer (Microsoft 2024b). Models export to the ONNX format, after which ONNX Runtime applies target-independent graph optimizations and delegates subgraphs to hardware-specific execution providers. This architecture enables single-format deployment across CPUs, GPUs, and custom accelerator application-specific integrated circuits (ASICs).

Portability still demands profiling operator placement across execution boundaries. If an operation is unsupported by the preferred execution provider, placement depends on the configured provider order and available implementations. A CPU fallback can introduce host-accelerator transfers and synchronization that erode the expected latency advantage; if no provider supports the operation, execution may fail. Profiling must therefore verify provider assignment and operator coverage across the entire graph, not merely confirm that the exported artifact executes without errors.

13.8.1.2 Specialized inference engines

When latency budgets or serving costs bind more tightly than portability, specialized engines such as TensorRT[23] for NVIDIA GPUs and OpenVINO[24] for Intel hardware optimize graphs specifically for target microarchitectures (NVIDIA 2024c; Intel Corporation 2026b; Chen et al. 2018). Rather

22 **ONNX Runtime**: Microsoft's inference engine acts as a hardware abstraction layer: the same ONNX model can run on CPUs, NVIDIA GPUs, AMD GPUs, or custom accelerators through pluggable execution providers. ONNX Runtime applies graph optimizations such as constant folding, redundant-node elimination, and kernel fusion. Performance relative to a tuned target-specific engine such as TensorRT depends on the workload, execution provider, and hardware.

23 **TensorRT**: An NVIDIA inference compiler and runtime that optimizes models ahead of time for a specific microarchitecture (such as Hopper or Blackwell) (NVIDIA 2024c). Requiring a target-specific build phase allows the compiler to generate fused kernels, select microarchitecture-specific Tensor Core execution paths, and optimize memory layouts (such as NC/32HW32). The resulting nonportable engine reduces per-request execution latency and minimizes the accelerator count needed to satisfy a target throughput.

24 **OpenVINO (open visual inference and neural network optimization)**: An Intel-oriented inference toolkit that converts, optimizes, and runs models across Intel CPU, GPU, and NPU targets (Intel Corporation 2026b). Specializing for direct hardware execution departs from the portable execution guarantees of general-purpose frameworks, enabling the compiler to emit architecture-tuned vector instructions (such as AVX-512 or AMX) and exploit low-precision integer execution units. While speedups depend on the specific operator mix and target processor, this specialization can make CPU or edge serving economically viable for latency-sensitive models.

than maintaining portable abstractions, these compilers specialize kernels directly to the hardware's register files, cache hierarchies, and matrix execution units.

Layer fusion[25] is kernel fusion (also called operator fusion) applied when the engine is built. For the convolution → batch normalization → ReLU block fused in section 10.5.1, the serving compiler performs that folding once, at engine build time, because batch normalization is a fixed affine transform once training ends. It first folds the linear batch normalization parameters directly into the convolution weights and bias ($\mathbf{W}_{\text{fused}} = \frac{\gamma}{\sigma_{\mathcal{B}}}\mathbf{W}$ and $\mathbf{b}_{\text{fused}} = \frac{\gamma}{\sigma_{\mathcal{B}}}(\mathbf{b} - \mu_{\mathcal{B}}) + \beta$, where $\mu_{\mathcal{B}}$ is the per-channel running mean and $\sigma_{\mathcal{B}}$ denotes the square root of the sum of running variance and the numerical stabilizer used at inference and γ and β are the learned scale and shift), eliminating the normalization operation entirely. The compiler then fuses the convolution and the subsequent ReLU into a single kernel: thread blocks read the input feature map from HBM once, compute matrix multiply-accumulate operations in registers and shared memory, evaluate the ReLU activation directly in registers, and write only the final activated result back to HBM.

Kernel auto-tuning benchmarks and selects the optimal algorithm for each operation on the target microarchitecture. A single 2D convolution can be evaluated through direct convolution, GEMM transformation (im2col), Winograd minimal filtering (efficient for small 3×3 filters), or Fast Fourier Transforms (efficient for large spatial filters), alongside dozens of tile-size variations. Because the winning strategy depends on filter dimensions, batch size, and the accelerator's register and shared memory capacity, the compiler benchmarks candidate implementations against target hardware and caches the lowest-latency kernel, trading ahead-of-time compilation time for predictable serving latency.

These optimizations produce substantial speedups over an untuned framework-native baseline, but they require explicit graph compilation and may restrict dynamic tensor dimensions. The illustrative runtime comparison that follows quantifies one such optimization spectrum.

[25] **Layer fusion**: Analogous to loop fusion in compiler optimization, where adjacent loops over the same array are combined to reduce memory traffic. Kernel fusion applies the same principle to GPU operations: sequential kernels that write and re-read intermediate tensors from HBM are merged into a single kernel that can keep data in registers.

↩ Kernel fusion, which merges adjacent operations so intermediates stay in registers and on-chip memory instead of round-tripping through HBM, is established in section 7.3.1.



Example 13.11: ResNet-50: Runtime comparison

Table 13.18 gives an illustrative batch-one ResNet-50/V100 runtime scenario. Its per-engine latencies are separate assumptions chosen to isolate the effect of the engine; the chapter's 5 ms reference profile sits between the ONNX Runtime and TensorRT FP32 rows.

Table 13.18: Illustrative inference runtime comparison: Assumed latency and derived speedup for batch-one ResNet-50 across PyTorch eager, TorchScript, ONNX Runtime, and TensorRT on a V100.

Runtime	Latency	Speedup	Notes
PyTorch (eager)	8.5 ms	1×	Baseline, no optimization
TorchScript	6.2 ms	1.4×	JIT compilation
ONNX Runtime	5.1 ms	1.7×	Cross-platform
TensorRT FP32	2.8 ms	3×	NVIDIA-specific
TensorRT FP16	1.4 ms	6.1×	Tensor Core acceleration
TensorRT INT8	0.9 ms	9.4×	Requires calibration

Systems insight: In this illustrative comparison, TensorRT INT8 provides 9.4× speedup and requires quantization calibration data and NVIDIA-specific deployment.

This optimization-compatibility trade-off governs engine selection. Higher specialization can improve hardware efficiency for supported operations and shapes but increases deployment complexity, restricts dynamic control flow, and may introduce numerical deviations from the training baseline. The engineering choice depends on latency budgets, operational constraints, and how frequently model architectures evolve.

Once an execution engine is selected, configuration tunes its resource allocation to the hardware envelope. For CPU inference, thread pool sizing must match physical core topologies: undersubscription leaves execution units idle, while oversubscription induces CPU cache thrashing, OS scheduling latency, and lock contention across non-uniform memory access (NUMA) domains. For accelerators,

static memory preallocation creates fixed-size activation workspaces during initialization, eliminating runtime memory allocations (`cudaMalloc`) and page-table faults from the latency-critical request path. Execution provider priority determines which hardware backend handles each operator subgraph, while graph optimization levels balance build times against constant folding and algebraic simplification. Production deployments must benchmark these configurations across full latency distributions (p50, p99) rather than relying on framework defaults.

13.8.2 Precision selection for serving

On NVIDIA V100 GPUs, Tensor Cores accelerate FP16 matrix operations, whereas INT8 computations execute on standard integer ALUs (via `DP4A` vector instructions). In an illustrative serving scenario, transitioning a ResNet-50 model from FP32 to INT8 triples inference throughput while sacrificing less than 0.4 percentage points of top-1 accuracy. This dynamic establishes a direct coupling between numerical precision and infrastructure economics. At the serving layer, precision engineering introduces operational constraints: calibration data fidelity, layer sensitivity under live inputs, and dynamic precision dispatch.

13.8.2.1 Precision-throughput relationship

↩ The quantization techniques this selection draws on, from numerical formats through calibration and quantization-aware training, are established in section 10.4.

For memory-bandwidth-bound operations, reducing precision raises throughput by cutting the byte volume transferred across the memory bus. Equation 13.15 gives the bandwidth-only upper bound from precision reduction:

$$\frac{\text{Throughput}_{\text{INT8}}}{\text{Throughput}_{\text{FP32}}} = \frac{32}{8} = 4\times \text{ (theoretical maximum)} \tag{13.15}$$

Actual speedup depends on the compute-to-memory ratio, operator mix, hardware instruction throughput, and matrix alignment, falling below the bandwidth-only bound whenever arithmetic throughput or memory alignment binds. Low-precision kernels achieve peak arithmetic efficiency only when matrix dimensions align with the accelerator's warp-level execution tiles (for example, multiples of 8 or 16 elements). When dimensions do not align with hardware tiles, libraries such as cuBLAS and cuDNN pad tensors or fall back to slower execution paths. The precision trade-offs for a standard vision model illustrate how these theoretical limits manifest in practice.

↩ Tensor Core tile architectures and their matrix-dimension alignment constraints are established in section 11.4.3.

Example 13.12: ResNet-50: Precision trade-offs on V100

Table 13.19 gives illustrative latency, memory, and accuracy assumptions across FP32, FP16, and two INT8 paths for ResNet-50, together with Tensor Core utilization where supported:

Table 13.19: Illustrative precision trade-offs on V100: Assumed latency, memory footprint, and accuracy for ResNet-50 in FP32, FP16, and INT8 (PTQ and QAT), plus Tensor Core utilization where supported.

Precision	Latency	Memory	Accuracy	Tensor Core Util.	Calibration
FP32	2.8 ms	98 MB	76.13%	0%	None
FP16	1.4 ms	49 MB	76.13%	85%	None
INT8 (PTQ)	0.9 ms	25 MB	75.80%	N/A*	1,000 samples
INT8 (QAT)	0.9 ms	25 MB	76.05%	N/A*	Additional training

* **Note on V100 execution**: On the NVIDIA Volta (V100) microarchitecture, first-generation Tensor Cores exclusively support half-precision floating-point (FP16) arithmetic. INT8 operations achieve their speedup by using DP4A vector single instruction, multiple data (SIMD) instructions on the standard integer CUDA cores, hence Tensor Core utilization is marked N/A. Subsequent architectures (such as Turing T4 and Ampere A100) introduced native INT8 Tensor Core execution units.

Systems insight: In this illustrative scenario, INT8 provides 3.1× speedup and loses 0.33 percentage points of accuracy with PTQ. QAT recovers most of that loss but requires retraining.

13.8.2.2 Precision selection constraints

Precision selection is constrained by layer sensitivity, calibration data, and runtime policy. Not all layers tolerate reduced precision equally. The uniform quantizer of section 10.4 bounds the error each layer must absorb. For a scalar uniform b-bit quantizer over the calibrated range $[\alpha, \beta]$, the step size (scale) s and within-range rounding-error bound in equation 13.16 are

$$s = \frac{\beta - \alpha}{2^b - 1}, \qquad |\epsilon_{\text{quant}}| \leq \frac{s}{2} \tag{13.16}$$

for values that are not clipped. Values outside the calibrated range are clipped and incur additional saturation error. Layer-level output or task error depends on activation distributions and downstream sensitivity, so it must be measured rather than inferred from a universal norm law. This mechanism explains why first convolutional and final classification layers are commonly retained at FP16, while intermediate layers tolerate INT8. Initial layers process low-dimensional raw inputs (such as 3 RGB channels) before representations expand into overcomplete feature spaces that can absorb quantization noise, whereas final classification layers produce raw logits where small numerical perturbations directly shift decision boundaries and alter top-1 predictions.

Post-training quantization adds a data dependency. The calibration dataset determines the clipping thresholds $[\alpha, \beta]$ and scale factors s used for INT8 conversion, requiring it to match the statistical profile of live serving traffic rather than arbitrary training or validation splits. A calibration-serving distribution mismatch causes excessive clipping or underflow, degrading end-to-end task accuracy, an operational pitfall analyzed in section 13.12.

Production serving architectures can elevate precision from a static compilation setting into a dynamic runtime policy. When request queues are shallow and the system operates with ample latency headroom against its service level objective (SLO), the engine can dispatch requests at FP16 to maximize task accuracy. During traffic spikes that threaten queue saturation, the engine can dynamically downcast execution to INT8 to preserve latency bounds. Similarly, requests that yield low classification confidence under INT8 can be selectively re-evaluated at FP16. This dynamic dispatch operationalizes adaptive quality-latency trade-offs under fluctuating request arrival rates.

Precision selection carries direct infrastructure sizing consequences: under an illustrative $3\times$ saturated-throughput improvement, 10 GPUs at INT8 offer the same aggregate saturated capacity as 30 GPUs at FP32. Equal capacity does not establish equal queueing latency; service-time distributions, batching policy, and failure reserve must still satisfy the same SLO. Precision engineering is therefore an infrastructure sizing discipline, directly coupling numerical representation to cluster capital expense and energy consumption. Yet configuring the runtime and numerical bit width addresses only the model level; achieving low-latency serving in production requires optimizing the host-to-device boundaries within the physical server node itself.

13.9 Node-Level Optimization

Consider an image classification service where the compiled neural network forward pass takes only 0.8 milliseconds in a standalone benchmark, yet real-world request latency approaches 15 milliseconds at the network boundary. Profiling the physical host reveals that input JPEG decoding consumes 8 milliseconds of CPU time, serialized PCIe host-to-device transfers stall the accelerator queue for 4 milliseconds, and individual kernel launches suffer 2 milliseconds of driver synchronization bubbles. The isolated model benchmark masked every physical bottleneck on the host server.

Runtime selection and precision tuning operate at the model level, determining what computation runs and what numerical format it uses. Between the model graph and the silicon, however, lies an execution layer encompassing graph compilation, bus transfers, and CPU-accelerator coordination. Node-level optimization targets this complete end-to-end request path on concrete hardware, eliminating overhead between the network socket and accelerator execution units across four primary physical boundaries:

- **Graph-to-kernel boundary**: Translating computation graphs into numerous fine-grained kernel launches incurs driver serialization overhead and forces intermediate tensors through device HBM. Graph compilers fuse adjacent operations into composite kernels that preserve intermediate activations in on-chip SRAM or register files.

- **Host-accelerator boundary**: Accelerators sit idle when the host processor fails to enqueue work ahead of execution. Unpipelined copies across host-device interconnects (such as PCIe) and blocking synchronization calls stall accelerator compute pipelines.
- **CPU execution boundary**: Host preprocessing pipelines and CPU-based inference degrade when executed through interpreted scalar code rather than vectorized SIMD instruction units and NUMA-local memory allocations, and when operating-system noise preempts the threads that feed the accelerator.
- **Storage and deserialization boundary**: Initializing serving replicas requires streaming weight tensors from persistent storage into host memory. Formats requiring recursive object deserialization bottleneck CPU execution, creating cold-start latency spikes that impede autoscaling.

Each boundary represents a distinct physical constraint where execution stalls on memory bandwidth, bus transit, or control overhead rather than peak arithmetic throughput.

13.9.1 Runtime graph compilation

Specialized inference engines (section 13.8) outperform eager framework baselines because serving environments provide static execution topologies and bounded input-shape profiles. When operator sequences and tensor dimensions are fixed before incoming requests arrive, compilers shift optimization costs from runtime query processing to offline deployment preparation.

When the trace reveals intermediate-memory traffic or repeated launch overhead, a candidate intervention is kernel fusion, applied by the engines of section 13.8.1.2 across the whole graph. Its serving payoff is largest on chains of elementwise operations, such as the bias add and activation that follow a convolution. These operations perform $\mathcal{O}(1)$ arithmetic per byte transferred, so separate kernels can spend substantial time writing and reading global-memory intermediates instead of performing arithmetic; cache behavior determines the traffic that reaches HBM or GDDR. Fusion keeps those intermediates in registers or shared memory (SRAM), cutting memory traffic and amortizing kernel launch overheads over larger computational blocks.

Static execution graphs also enable compile-time constant folding. In model serving, model parameters are immutable. Compilers evaluate subexpressions composed entirely of static weights and constants prior to deployment. The batch-normalization folding shown above for the convolution block is the canonical case. It eliminates the normalization layer's kernel launches and associated memory traffic entirely, removing computational work from every live request without altering mathematical outputs.

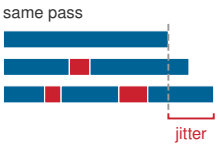
Memory planning applies structural optimization to memory layout rather than arithmetic. Reactive allocation during request processing triggers framework allocator locks, heap searches, and operating system page table modifications. In a static graph, tensor lifetimes are known deterministically from input to output. The runtime compiler calculates exact buffer offsets ahead of time, allowing intermediate tensors with nonoverlapping lifetimes to reuse the same physical memory arena. This precalculated buffer reuse bounds total memory consumption, prevents memory fragmentation, and eliminates runtime allocation stalls.

Beyond memory planning and kernel fusion, static execution paths can use **CUDA Graphs** to amortize host submission overhead across a captured sequence. In standard eager execution, launching an accelerator operation from Python or a serving runtime requires the host CPU to perform substantial bookkeeping: verifying tensor memory addresses, validating dimensions, selecting device kernel variants, and writing a hardware command descriptor into the driver's memory-mapped ring buffer. Each submission pays the 5–20 µs of host launch work quantified in section 13.7.1. Total submission cost depends on the executed kernel count, not the number of layers: a decode step launching 8 to 10 sub-kernels across each of 32 transformer layers can accumulate several milliseconds of host submission work. At small batch sizes, this dispatch latency can exceed the time the accelerator silicon takes to execute the actual tensor math, leaving compute units starved of work. CUDA Graphs circumvent this host bottleneck: during an initial warm-up execution, the driver captures the entire sequence of kernel launches, memory bindings, and inter-kernel dependencies into a static execution graph. During live inference, the host CPU dispatches the captured graph with a single driver invocation (illustratively $\sim 5\,\mu s$, depending on the graph,

driver, and hardware). The accelerator then schedules its constituent kernels while respecting their dependencies, reducing repeated host submission work without eliminating device execution costs.

> Systems Perspective 13.6: Compilation timing trade-off
>
> Just-in-time compilation defers graph optimization until the runtime engine observes concrete input shapes and execution paths. This runtime visibility enables aggressive kernel specialization for dynamic control flow and variable sequence lengths, but it injects compilation overhead directly into the serving path, generating multi-second latency spikes on early requests.
>
> Ahead-of-time compilation executes graph analysis, kernel fusion, and hardware-specific autotuning prior to deployment. Pre-building removes the covered compilation work from the request path, but weights, device contexts, allocations, and representative execution paths still require loading and warmup. The compiled engine remains constrained to its supported shape profiles and may require padding or multi-profile compilation to accommodate dynamic input sizes.

13.9.2 Deterministic latency and resource isolation



Noisy neighbors stretch identical forward passes unevenly, widening the tail.

Compilation fixes what the accelerator executes; it does not control how steadily the host feeds it. An inference server does not execute in a dedicated vacuum. On a single host, the operating system multiplexes hardware across background telemetry daemons, logging agents, and kernel worker threads. When these unisolated processes contend for execution resources, they intermittently preempt the inference pipeline. These noisy neighbors are a primary source of **latency jitter**, where the physical time required to execute identical forward passes fluctuates unpredictably. This contention inflates 99th percentile (p99) tail latency even when average hardware utilization remains low. Resource contention thus widens tail latency through service-time variance, a physical degradation distinct from the queuing-delay accumulation along the saturation knee plotted in figure 13.1.

Reducing performance variability requires controlling interference from the operating system's default time-sharing heuristics. Serving systems such as Clockwork demonstrate that deep neural network (DNN) inference execution times can be predictable down to sub-millisecond tolerances under their controlled workload and hardware conditions (Gujarati et al. 2020). CPU affinity (thread pinning) establishes the baseline compute boundary by binding inference worker threads to designated physical cores. Without pinning, the OS scheduler periodically migrates threads between cores, losing access to warm private L1 and L2 cache contents and requiring lines to be refetched from shared L3 cache or DRAM. On multi-socket architectures, thread migration across NUMA boundaries is even more penalizing: accesses to memory left on the previous socket cross inter-socket interconnects, adding tens of nanoseconds of latency per transaction. Pinning threads to specific cores and binding their memory allocations to local NUMA nodes reduces migration-related cache misses and remote-socket memory traffic.

Even with pinned CPU cores, memory access latency remains vulnerable to the virtual memory subsystem. Under host memory pressure, the kernel can page inactive address ranges out to disk. When the CPU touches a paged-out address, execution can stall for milliseconds while the page returns to DRAM. Host buffers used for PCIe DMA also need resident, registered backing pages during the transfer. Memory locking (`mlock`) instructs the kernel to pin specified virtual address ranges into physical host RAM, preventing page eviction for host-resident weights, input staging buffers, and offloaded key-value (KV) cache blocks. This removes eviction-related page faults at the cost of host memory flexibility; locked pages remain resident until unlocked, while cache misses and memory contention can still delay access. Because `mlock` operates strictly on host page tables, it does not apply to accelerator HBM, which is allocated directly by the accelerator driver.

Beyond scheduled processes, asynchronous hardware interrupts introduce another source of sub-millisecond preemption. When a network interface card (NIC) receives packet bursts, it triggers hardware interrupt requests (IRQs) that force the handling CPU core to immediately suspend user-space execution and process kernel interrupt handlers. If the OS routes these interrupts to cores running the inference engine's submission loop, the CPU pauses command dispatch to the accelerator. Because GPUs process short inference layers in microseconds, even brief CPU submission stalls

drain the accelerator's hardware command ring buffer, causing pipeline bubbles where compute units sit idle waiting for the next kernel descriptor. Interrupt shielding restricts IRQ affinities to dedicated housekeeping cores, reducing preemption of the GPU dispatch thread by those device interrupts.

Isolating physical cores, locking host memory, and shielding interrupt lines reduce specific sources of operating system noise. The resulting service-time distribution still needs measurement under the intended load. Isolation makes the host a predictable partner that feeds the accelerator; when deployment constraints or workload profiles preclude dedicated accelerators, the host becomes the executor itself, and vectorization and the memory hierarchy govern execution speed.

13.9.3 CPU inference optimization

Deploying inference on CPUs remains practical for lightweight models, sparse or bursty request streams that cannot saturate an accelerator, and applications where host-to-device transit latency dominates total computation time. Modern server CPUs[26] (such as Intel Xeon and AMD EPYC) integrate wide SIMD vector registers and matrix accelerators (including AVX-512 and AMX). General-purpose Python interpreters and scalar implementations leave these execution units idle. Specialized runtimes such as OpenVINO map neural network operators to optimized microkernels that tile matrix operations across L1/L2 caches and use packed fused multiply-add (FMA) instructions (Intel Corporation 2026b).

The NUMA boundaries that thread pinning respects matter more when the CPU itself runs the model. Memory controllers are physically distributed across CPU sockets and linked by coherent interconnects (such as Intel UPI or AMD Infinity Fabric). Accessing memory attached to a remote socket incurs substantially higher latency and lower bandwidth than local channel access. An inference server must maintain NUMA awareness: worker threads must be pinned to dedicated physical cores, and the weight tensors and request buffers those threads touch must reside in memory nodes local to that socket. Because model weights, at hundreds of megabytes to tens of gigabytes, routinely exceed per-socket L3 cache capacities, every forward pass streams weights from main RAM. Mismatched socket allocations force continuous weight streaming across the slower inter-socket interconnect, creating severe tail-latency degradation.

For lightweight models evaluated at batch size one, a CPU frequently outperforms a discrete GPU. When an accelerator processes an isolated request, the fixed overhead of transferring inputs across the host PCIe bus (several microseconds), driver queue serialization, and kernel launch overhead (5–20 µs per kernel) can easily surpass the arithmetic execution time. The CPU processes the request directly in local cache without bus transfers or driver queue delays. Determining the operational crossover requires empirical measurement across model architecture, arithmetic intensity, batch size, and latency constraints.

13.9.4 Model serialization and fast loading

Autoscaling systems operate as closed-loop controllers that provision or deprovision serving replicas in response to traffic demand. During demand surges, cold-start latency (section 13.6.2) governs how rapidly new replicas become ready to serve traffic. For multi-gigabyte models, cold-start duration is heavily dominated by streaming serialized model weights from persistent storage into host memory. The architectural choice of serialization format determines whether this stage is bounded by storage read bandwidth or by CPU parsing overhead.

Legacy PyTorch checkpoints loaded via `torch.load()` package weights within a zip archive containing pickle-encoded object metadata. Deserializing a pickle file forces the CPU to interpret a stream of serialization opcodes sequentially, allocating individual Python objects (dictionaries, metadata wrappers, and tensor headers) and potentially copying tensor bytes into newly allocated memory. Restricted loading modes reduce arbitrary code execution risks, while memory-mapped options can avoid some eager copies. Whether parsing or storage dominates depends on the checkpoint, loading path, and hardware.

A tensor-oriented format[27] bypasses interpreter-driven object construction. Safetensors organizes checkpoints with a lightweight header containing tensor metadata followed by contiguous, uncompressed tensor byte buffers aligned to memory boundaries. Because the on-disk binary layout

[26] **SIMD**: From Michael Flynn's 1966 taxonomy of computer architectures, SIMD enables one instruction to operate on multiple data elements simultaneously. Intel's AVX-512 can process sixteen FP32 values in a 512-bit vector, while AMX adds matrix tile operations. The attainable fraction of peak depends on data type, matrix shape, memory behavior, and implementation, but optimized vector kernels provide multiple-fold speedups over scalar execution.

[27] **Safetensors**: The name emphasizes safety: unlike unrestricted pickle deserialization, the format does not encode executable Python objects (Hugging Face 2026). It stores tensors as contiguous raw bytes with a small header and supports memory-mapped loading. In the local benchmark example, that path is 10× faster than the compared PyTorch checkpoint path; the result is workload- and storage-specific and does not include transfer to a discrete accelerator.

mirrors the in-memory array layout, the runtime uses memory mapping (`mmap(2)`, introduced in section 13.6.3) to map file blocks directly into the process's virtual address space. Virtual memory pages fault into host RAM on demand or are prefetched sequentially by the operating system, reducing Python object reconstruction and avoiding eager user-space tensor copies on the mapped path. On systems with discrete accelerators, transferring these tensors across PCIe or NVLink is still required before computation begins, but memory-mapped host loading can reduce CPU reconstruction costs without eliminating page faults, storage reads, or device initialization.

Example 13.13: Loading speed: Safetensors vs. Pickle

Scenario: A serverless inference replica loads a 5 GB model checkpoint during cold-start scaling events.

Diagnosis: PyTorch `torch.load()` (Pickle) requires CPU object reconstruction, taking 15 s to initialize. Safetensors memory-mapping bypasses CPU parsing, loading weights in 1.5 s (10× faster).

Systems lesson: Tensor-oriented, memory-mapped formats can reduce CPU reconstruction and eager-copy overhead during cold start. Storage throughput, page faults, runtime initialization, and accelerator transfer still remain in the scale-out budget.

13.9.5 Profiling the serving node

Optimization without empirical measurement degrades into speculative trial and error. The duty-cycle metric defined in equation 13.2 helps locate idle intervals, but active time alone does not measure arithmetic efficiency or prove adequate queueing headroom. The objective is useful throughput that meets the latency SLO. Timeline profilers, such as PyTorch Profiler and NVIDIA Nsight Systems (`nsys`), make this objective actionable by recording concurrent CPU threads, OS runtime calls, memory transfers, and GPU hardware queues along a synchronized timeline.

Effective trace analysis proceeds bottleneck-first from hardware utilization back toward application logic:

- **Idle compute bubbles**: White space on the GPU execution track indicates hardware starvation. Compute warps idle while waiting for the host CPU to complete request preprocessing (e.g., tokenization or image decoding), page faults during on-demand weight paging, or blocking inter-thread synchronization.
- **Kernel launch fragmentation**: A sequence of micro-kernels, each executing in under 5–10 µs, suggests launch overhead as a candidate. Host launch timing and gaps between device kernels reveal whether submission starves the GPU; if so, graph capture or legal fusion may help.
- **Interconnect transfer stalls**: Synchronous `MemcpyHtoD` blocks on the primary CUDA stream indicate unpipelined host-to-device data movement. Overcoming this stall requires allocating page-locked (pinned) host memory and executing asynchronous transfers on dedicated copy streams to overlap bus transit with concurrent accelerator execution on preceding requests.

Example 13.14: The profiling loop

Scenario: An inference serving node exhibits high p99 tail latency despite low average GPU compute utilization.

Diagnosis: Timeline profiling (Nsight Systems/PyTorch Profiler) reveals large idle gaps between tiny GPU kernels caused by CPU host-side preprocessing bottlenecks and kernel launch overhead.

Systems lesson: Serving node optimization requires empirical timeline profiling. Iteratively identifying trace bottlenecks (kernel launch overhead, host-device copy stalls) guides targeted optimizations like kernel fusion and pinned memory.

Table 13.20 summarizes representative node-level optimization techniques alongside their primary target metrics and engineering costs. The table serves as a decision matrix rather than an exhaustive

checklist: engineering effort should be directed exclusively to the row that addresses the binding bottleneck identified on the measured execution trace.

Table 13.20: Node-Level Optimization Impact: Illustrative ranges for matching techniques to measured bottlenecks. Realized gains and implementation cost depend on the model, runtime, hardware, and workload.

Technique	Target Metric	Illustrative Gain	Implement. Cost	Best For
Kernel Fusion	Latency & Throughput	2–5×	Medium (Compiler)	Memory-bound layers
INT8 Quantization	Throughput	3–4×	High (Calibration)	Inference-heavy nodes
Graph Compilation	Latency	1.5–3×	Low (One-line)	Static graph models
Zero-Copy Loading	Startup Time	10–50×	Low (File format)	Autoscaling/Cold Start
CPU Pinning	Tail Latency (p99)	20–50% reduction	Low (Config)	Latency-critical apps

This diagnostic hierarchy ensures engineering resources address structural bottlenecks along the entire request path rather than applying isolated micro-optimizations. A four-level audit checkpoint structures this prioritization from external service boundaries down to device kernels.

Checkpoint 13.4: The optimization hierarchy

Optimizing inference follows the request path from the outside in across four structural levels:

- ☐ **System budget**: Partition the end-to-end latency budget between network transit, host application processing, queueing delay, and kernel execution using distributed tracing tools to verify compliance.
- ☐ **Application level**: Determine whether the arrival stream and latency budget permit batching, and test whether the gain in service capacity outweighs formation and resource-queue waiting.
- ☐ **Model level**: If dispatch or operator placement binds the request path, evaluate a compatible compiled engine or graph configuration and recheck quality across supported input shapes.
- ☐ **Kernel level**: Where intermediate traffic or launch overhead is measured, assess legal fusion candidates and verify their numerical behavior and end-to-end benefit.

Systematically eliminating node-level bottlenecks lets every provisioned server operate near its physical efficiency limit. Every workload so far, however, has treated a request as one completed prediction. That assumption belongs to the service-time and batching models developed here, which it simplifies, rather than to the MLPerf scenarios, which do not require fixed-size outputs. The fastest-growing category of serving workloads violates this assumption entirely. Large language models generate outputs token by token, with each token depending on every previous one. A single request may produce hundreds or thousands of tokens over seconds of elapsed time, yet must feel responsive from the first token onward. The shift from *fixed-output* to *variable-length, streaming-output* serving reopens each tool developed so far, from the latency budget to the runtime.

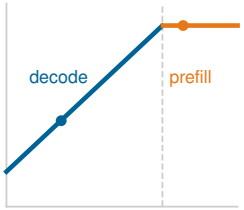
13.10 LLM Serving

Large language models (LLMs) make three properties far more prominent than in fixed-output serving: *autoregressive generation*[28] (each token depends on all previous tokens, making output inherently sequential), *variable-length output* (response length is unknown at request time, complicating fixed-batch assumptions), and *stateful memory* (the key-value cache grows with each generated token, creating dynamic memory pressure). Each property strains a different tool from the preceding sections: sequential generation splits the latency budget into phases, unknown output length breaks fixed-batch scheduling, and the growing cache makes memory capacity, not arithmetic, the limit on concurrency. The p50, p95, and p99 metrics used for classification serving still matter, but they apply separately to the initial prompt processing and to the subsequent token generation. The same principles of queuing theory, batching trade-offs, and latency budgets remain applicable; LLM serving adds domain-specific techniques atop them.

[28] **Autoregressive**: From Greek *auto-* (self) and Latin *regressus* (a going back)—the output "regresses" on itself; George Udny Yule introduced autoregressive models in 1927 for analyzing sunspot cycles. In language modeling, each output token conditions on all previously generated tokens, creating a serial dependency that prevents the parallelism exploited during training. Decode is often weight-bandwidth-bound at small batches; larger batches amortize the weight stream and can shift the bottleneck toward compute or KV-cache traffic (Pope et al. 2023).

↩ Prefill, small-batch autoregressive decode, and the KV cache that every decode step reads are established in section 6.8.4.2.

13.10.1 Performance metrics: TTFT and TPOT



Prefill and low-batch decode often face different resource limits.

Generative models produce a stream of tokens rather than a single output tensor. This streaming nature requires dedicated LLM performance metrics that reflect the transition from *prefill* (processing the prompt and populating its KV cache) to *decode* (generating output). Two key measures are **time to first token (TTFT)** and **time per output token (TPOT)**, which capture responsiveness and average generation pace. Per-gap **inter-token latency (ITL)** measures the distribution of pauses between individual streamed tokens; TPOT instead averages the post-first-token interval across an output.

Definition 13.6: LLM performance metrics

LLM performance metrics separate initial responsiveness from the cadence of streaming autoregressive generation.

1. **Significance**: They decompose user-perceived latency into time to first token (TTFT) (whose prefill work is often compute-bound) and time per output token (TPOT) (whose decode work is often bandwidth-bound at small batches).
2. **Distinction**: Unlike fixed-output metrics (for example, end-to-end latency), LLM metrics measure the fluidity of generation, acknowledging that the user experience depends on the rhythm of token arrival.
3. **Common pitfall**: A frequent misconception is that a low TTFT means a fast model. In reality, a model can have a fast TTFT but a sluggish TPOT (if the memory wall (BW) is the bottleneck), leading to a frustrating user experience where the answer starts quickly but "stutters" thereafter.

These metrics capture distinct aspects of the user experience, so interactive services often set separate targets for TTFT, TPOT or ITL, and aggregate throughput.

Systems Perspective 13.7: LLM serving latency targets

Interactive LLM services usually need separate SLOs for responsiveness, generation fluidity, and fleet throughput. The following values are illustrative targets rather than universal requirements:

- **TTFT**: $<$ 500 ms (for a 1000-token prompt)
- **TPOT**: $<$ 50 ms (equivalent to ~20 tokens/s, faster than human reading speed)
- **Throughput**: $>$ 1000 tokens/s aggregate across active serving replicas

A single "latency" number therefore hides the prefill/decode split. TTFT is the blank-screen budget, TPOT is the reading-flow budget, and aggregate service throughput, measured as tokens/s summed across active serving replicas, determines whether those targets hold under shared load.

13.10.2 Decoding strategies

Meeting per-token latency (TPOT) targets depends on more than raw accelerator memory bandwidth: the discrete sampling algorithm executed at each step governs both output distribution and host-accelerator synchronization. Generative inference relies on decoding algorithms that negotiate an explicit trade-off among token diversity, sequence quality, and runtime overhead.

The simplest approach, greedy decoding, selects the token with the highest predicted logit at each step, requiring a single forward pass per token. While computationally efficient, greedy selection frequently produces repetitive degeneration because it cannot recover from early suboptimal choices. Beam search preserves multiple candidate hypotheses across generation steps, pruning low-scoring sequences at each iteration; this exploration improves sequence-level coherence but scales computational work and KV-cache footprint linearly with beam width. Stochastic sampling incorporates parameterized entropy: temperature scaling modulates logit sharpness, while top-k and top-p nucleus sampling truncate the vocabulary distribution to restrict sampling to high-probability

subsets (Holtzman et al. 2020). The serving cost of any strategy depends on its kernel implementation and the sequence length distribution that continuous batching must absorb.

Interactive LLM services commonly return tokens as they are produced rather than waiting for complete generation. This streaming response transforms the user experience: a two-second total generation feels responsive when tokens stream continuously, but feels broken when users stare at a blank screen for two seconds. Streaming requires infrastructure support for incremental responses and client-side rendering. The latency profile shifts accordingly: TTFT determines when output starts appearing (responsiveness), while TPOT determines the perceived generation speed (fluidity). Once generation streams token by token, the serving unit changes from one fixed-output prediction to a stateful sequence whose length is unknown when it is admitted and whose memory footprint grows on every step. The unknown length breaks the fixed-batch scheduling developed for fixed-output models, and the growing state turns accelerator memory into the next constraint.

13.10.3 Continuous batching

An autoregressive request finishes only when it emits an end-of-sequence token, so its length is revealed one step at a time. Extending the request-level dynamic batching of section 13.7 to an entire generation cycle keeps batch membership fixed until every sequence finishes. When sequence lengths vary, this static commitment degrades accelerator efficiency. If one sequence in a batch of eight finishes after ten tokens while others require 100 tokens, 90 percent of the compute and memory bandwidth allocated to that sequence slot is wasted on padding tokens or masked dummy operations while awaiting the slowest sequence (Yu et al. 2022).

Continuous batching[29] (also called iteration-level batching) eliminates this idle-slot tax by rescheduling batch composition at each decoding iteration (Yu et al. 2022; Kwon et al. 2023). Rather than committing to a static batch that persists across hundreds of forward passes, the serving engine re-evaluates batch membership at every token-generation step. When a sequence emits its end-of-sequence token, the scheduler immediately reclaims that execution slot, admitting an arriving request into the batch for the subsequent forward pass. Active sequences continue decoding without interruption, maintaining high accelerator utilization across variable-length workloads.

Serving runtimes such as vLLM[30] and TensorRT-LLM improve throughput via two mechanisms: reclaiming wasted compute from completed sequences and shortening queue wait times for new arrivals (Kwon et al. 2023; NVIDIA 2026g). However, admitting new requests dynamically creates a systems collision between prompt prefill and token decoding. Long-prompt prefill often exposes high arithmetic intensity and can occupy the GPU for tens of milliseconds. Low-batch decoding often has less weight reuse and is memory-bandwidth bound, advancing one token per active sequence at a time. When an engine executes a prefill in the same iteration as active decodes, the ongoing decode streams stall, causing severe spikes in inter-token latency. Sarathi-Serve resolves this interference through chunked prefill and stall-free batching (Agrawal et al. 2024). By splitting long prompts into fixed-size chunks (e.g., 512 tokens) and co-scheduling a single chunk alongside active decode requests per step, the scheduler limits the prefill work admitted per iteration, reducing interference with active decodes; measured iteration times determine the resulting token cadence.

Continuous batching and chunked prefill keep decode batches full and their cadence steady, but at small batch each generated token still pays for one full weight stream. Speculative decoding[31] attacks that bound directly. A small draft model proposes several tokens, and the target model verifies the whole proposal in one parallel pass, so a single weight stream can yield more than one accepted token. Evaluated T5-XXL workloads achieved roughly 2–3× speedups (Leviathan et al. 2023), though the gain depends on how often the target model accepts the draft.

Keeping decode batches full, however, shifts the primary serving bottleneck to memory management. Because transformer generation appends attention states at every step, the KV cache for each active request grows dynamically in accelerator HBM. When sequences of varying lengths share memory, contiguous allocation produces severe external fragmentation. If a completed 100-token sequence releases its cache, an arriving 150-token request cannot occupy that freed memory because it requires a larger contiguous address range. Over time, physical memory fractures into small, noncontiguous gaps, preventing new sequences from launching even when total unallocated capacity appears sufficient. Together with the over-reservation of contiguous allocators, this memory

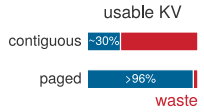
[29] **Continuous batching**: Also called "iteration-level batching" (Yu et al. 2022) and, in NVIDIA TensorRT-LLM, "in-flight batching" (NVIDIA 2026g); the distinction is scheduling granularity. Traditional batching commits to a fixed batch for an entire generation sequence (potentially hundreds of iterations), while continuous batching reschedules at every token-generation step—analogous to preemptive OS process scheduling vs. run-to-completion. This finer granularity reduces the waste from variable-length sequences, where a batch slot occupied by a completed sequence sits idle until all other sequences finish.

[30] **vLLM (virtual large language model (LLM))**: An open-source serving system that enables continuous batching via its PagedAttention algorithm (Kwon et al. 2023). Inspired by OS virtual memory, this technique reduces the severe KV-cache fragmentation and reservation waste that constrains static batching. By keeping KV-cache waste low, vLLM can serve larger effective batches on the same hardware.

[31] **Speculative decoding**: A small "draft" model generates γ candidate tokens autoregressively; the large target model then verifies the proposed block in parallel (Leviathan et al. 2023). Parallel verification is possible because the draft model guesses all γ candidate tokens in advance. Rather than generating sequentially, the target model evaluates the entire γ-token block in a single causal-attention forward pass—acting like a compute-bound mini-prefill—which replaces γ memory-bandwidth-bound matrix-vector operations with one parallel matrix-matrix evaluation. When the draft model's proposals are accepted at rate α, effective throughput can scale with the number of accepted tokens per verification step. This breaks the serial autoregressive bottleneck at the runtime layer, not the architecture layer.

fragmentation can waste 60 to 80 percent of KV-cache memory in naive implementations (Kwon et al. 2023), severely limiting the concurrent batch size that determines serving throughput.

13.10.3.1 PagedAttention



PagedAttention recovers the KV-cache waste of contiguous allocation.

PagedAttention,[32] introduced in vLLM, resolves this fragmentation by applying operating system virtual memory paging to accelerator memory (Kwon et al. 2023). In static contiguous allocation, reserving maximum sequence length memory up front causes internal waste, while freed regions may be too small or poorly placed to satisfy a new contiguous reservation. Instead of allocating one contiguous region per sequence, PagedAttention divides the KV cache into fixed-size blocks mapped through a block table. A sequence's cache can therefore occupy noncontiguous physical blocks. When a sequence completes, its blocks return to a free pool for reuse. Rather than copying noncontiguous blocks into a temporary contiguous buffer before computing attention, a specialized PagedAttention CUDA kernel traverses the block table directly, staging noncontiguous blocks into on-chip SRAM during query-key multiplication. By eliminating external fragmentation and bounding internal waste to each sequence's last, partially filled block, PagedAttention reduced KV-cache waste below 4 percent in the reported evaluation and enabled substantially higher batch concurrency on the same accelerator hardware (Kwon et al. 2023).

Paging decides where cache blocks live; it does not reduce how many blocks each sequence needs, and that number grows with every generated token.

13.10.4 Memory and KV cache

The KV cache[33] is a stateful memory structure that grows with sequence length. Prefill creates cache entries for every prompt token, and each decoded token adds another entry, so a request's footprint is set by its prompt length plus the number of tokens it has generated so far. Summed across concurrent variable-length sequences, this dynamic state can exceed the model weights and dominate available accelerator memory, which is why the remaining memory levers either reuse cache state across requests or move it off the accelerator.

13.10.4.1 Prefix caching and memory offloading

Many requests carry identical prefixes, and recomputing their cache entries wastes both prefill compute and memory. **Prefix caching** stores the KV states of common instruction prefixes (such as a 2,000-token system prompt or a shared retrieval-augmented generation (RAG) context), allowing independent requests to reuse the prefill result for an identical prefix. For n_{req} requests sharing a prefix of S_{prefix} tokens, the avoided prefill work is roughly $(n_{\text{req}}-1)S_{\text{prefix}}$ token steps, subject to cache hits, eviction, and implementation overhead. For multi-turn conversations, the same principle lets a subsequent turn process only the new suffix after reusing compatible prior state.

When the aggregate KV cache exceeds GPU VRAM, systems can employ **KV cache offloading**. This strategy spills inactive or low-priority context windows to host CPU RAM or NVMe SSD, freeing VRAM for active generation. The reload cost is bounded below by bytes moved divided by PCIe or NVMe bandwidth, before software overhead and queueing are added. Offloading therefore prevents OOM failures and enables larger context windows, but it also creates affinity, invalidation, and hot-decode latency risks that must be budgeted explicitly. Distributed parallelism, which splits one request across multiple devices or machines, is a further lever that belongs to large-scale systems.

13.10.5 Session affinity constraints

Prefix reuse and offloading both assume that the next turn of a conversation reaches the replica that holds its cache. Session affinity, routing successive requests from a client to the same physical replica, arises whenever serving state resides in local accelerator memory rather than being reconstructed on every invocation. In conversational models, this resident state is dominated by the KV cache. When turn i executes on a replica, that replica's HBM retains the projected keys and values across all prior prompt and generation tokens. Routing turn $i+1$ to a different replica discards that local memory footprint, forcing the new node to recompute the entire prefix sequence from scratch. This redundant prefill consumes memory bandwidth and tensor-core cycles that would otherwise advance active decode iterations, directly inflating time-to-first-token.

[32] **PagedAttention**: The name directly references OS virtual memory paging, first implemented on the Atlas computer at Manchester (1962) to solve the same class of allocation problem—programs needed more memory than physically available, and contiguous allocation wasted space (Kilburn et al. 1962). Introduced at SOSP 2023, PagedAttention applies this six-decade-old abstraction to GPU KV-cache memory: the evaluated earlier serving systems wasted 60 to 80 percent of KV-cache memory due to fragmentation and over-reservation. In that evaluation, PagedAttention reduced waste to under 4 percent, enabling 2–4× higher throughput on the same hardware (Kwon et al. 2023).

[33] **KV cache (key-value cache)**: To avoid redundant work, the system caches the Key and Value vectors from previous tokens, which remain valid throughout generation. This design choice causes the dynamic memory growth described in the text, because the cache grows linearly with every generated token, making memory management a primary constraint. For a 70-billion-parameter-class model with grouped-query attention, the FP16 KV cache is about 0.33 MB per token per sequence; grouped-query attention (GQA), used in Llama-family models such as Llama 3, shares key/value heads across multiple query heads, reducing the cache relative to full multi-head attention (Dubey et al. 2024). A batch of 32 requests at an 8,000-token context therefore requires roughly 80 GB just for KV cache, several times larger still without grouped-query or multi-query attention.

A parallel physical constraint governs personalized serving using fine-tuned Low-Rank Adaptation (LoRA) modules: small per-user adapters, each a pair of low-rank matrices whose product is added to a frozen base weight matrix. While base-model weights remain pinned in device HBM, individual user adapters must be dynamically staged into memory. If incoming requests for a user scatter across multiple replicas, each node must repeatedly transfer adapter tensors over the host-to-device PCIe bus or fetch them across the network, displacing other active working sets in memory. Stateful preprocessing pipelines, such as tokenizer vocabularies and per-session normalization state, impose the same penalty, requiring redundant CPU-side allocations and cache rebuilds whenever execution moves across node boundaries.

Fleet routing is an optional extension of this local state-reuse constraint. Strict affinity creates a direct conflict with dynamic batching. Continuous batch schedulers maximize accelerator throughput by packing concurrent requests into unified matrix multiplications, raising operational arithmetic intensity in proportion to batch size ($I \propto B$). When requests are pinned to designated replicas, the scheduler's pool of batchable candidates fragments across the cluster. A replica assigned to an idle or low-concurrency user session must either process underfilled batches, collapsing arithmetic intensity back into the memory-bandwidth-bound regime, or stall waiting for affinity-matched tokens. To prevent this collapse in utilization, production routers use soft affinity. Under soft affinity, the ingress gateway prioritizes the cache-holding replica when capacity allows, but sheds requests to alternate nodes when local queue delays exceed the physical latency required to transfer adapter weights or recompute prefix tokens.

↩ Products of two low-rank matrices, and the storage and memory traffic they save relative to a full matrix, are established in section 10.3.3.1.



Systems Perspective 13.8: LLM serving: Beyond the fundamentals

Each LLM serving technique answers one of two physical limits. The first is the decode bound, in which every generated token pays for a full weight stream at small batch: continuous batching keeps decode batches full so each stream serves many sequences, chunked prefill keeps prompt processing from stalling that stream, and speculative decoding extracts more than one accepted token per stream. The second is the KV cache, which grows with sequence length and batch size and often exceeds the weights themselves; PagedAttention, prefix caching, offloading, and session affinity decide where it lives and how much of it can be reused. Weight-only quantization (such as INT4 weights with FP16 activations), the precision lever of section 13.8.2 applied to the weight stream, addresses both at once, because it shrinks the bytes streamed per token and frees capacity for cache (Lin et al. 2024).

These LLM-specific optimizations build directly on core serving mechanics: queueing models govern scheduling timeouts, batching dynamics determine throughput-latency curves, and reduced precision bounds memory traffic. Advanced treatments extend these mechanics to multi-tenant prefix sharing across fleets and multi-node tensor-parallel execution.

Napkin Math 13.5: The energy cost of a chat

Problem: How much energy does an H100-backed chat service spend per generated token and for a response with 500 tokens?

As LLMs scale, joules per token becomes a first-class operational metric alongside latency. Assuming the H100 draws its 700 W thermal design power (TDP), this scenario estimate follows from throughput and power (Choquette 2023):

1. **Throughput**: 114 concurrent requests $\times$ 8 tokens/s per request $\approx$ 912 tokens/s.
2. **Power**: 700 W (GPU) + 300 W (Host/Overhead) = 1000 W.
3. **Energy per token**: 1000 W / 912 tokens/s $\approx$ 1.0965 J/token

Systems insight: Under these assumptions, a response of 500 tokens consumes $\approx$ 548.2 J.

- For comparison, charging a smartphone consumes $\approx$ 40000 J.
- Boiling a cup of water consumes $\approx$ 100000 J.

The primary way to reduce J/token is to increase hardware utilization and eliminate redundant compute. If the GPU sits at 10 percent utilization due to poor batching, it still draws ~300 W and the host adds 300 W, causing energy per token to rise to 6.6 J/token (approximately 6× the baseline). Architectural optimizations like prefix caching also skip the energy-intensive prefill phase for shared context, directly reducing the energy footprint of retrieval-augmented generation (RAG) and chat applications. The serving lesson is that efficiency is not only a latency or cost metric; it also determines how much energy each useful token consumes.

Every lever so far takes the number of tokens a request generates as given. Reasoning models make that number a choice.

Systems Perspective 13.9: Looking ahead: Spending more compute per query

Fixed-output serving minimizes latency ($L_{\text{lat,total}} \to 0$) for a fixed amount of work per request. Some inference-time-compute systems instead deliberately spend more computation per query to improve answer quality, generating intermediate reasoning or search tokens before the visible answer. The optimization target then expands to include answer quality, and every extra reasoning token pays the decode costs this section derived: one more step of decode work at the batch's cadence, one more KV-cache entry held until the response ends, and its share of the joules per token. Answer quality now trades directly against TPOT, cache capacity, and energy per query.

Checkpoint 13.5: LLM serving fundamentals

LLM serving introduces constraints absent from traditional model serving.

- ☐ **TTFT vs. TPOT**: Contrast the distinct user experiences captured by TTFT and TPOT, explain why long-prompt prefill often exposes more arithmetic intensity than low-batch decode, and convert a 50 ms TPOT target into its tokens-per-second equivalent.
- ☐ **Continuous batching and PagedAttention**: Contrast static batching waste under variable sequence lengths against iteration-level scheduling, and explain how PagedAttention eliminates external fragmentation in the KV cache and bounds internal waste to a single partially filled block per sequence.
- ☐ **Memory wall**: Explain why adding compute units yields negligible latency improvement when token generation is bandwidth-bound, and identify how higher-bandwidth memory, model quantization, or batching alters the operating regime.
- ☐ **Prefix caching and offloading**: Explain how caching key-value activations for shared prompt prefixes eliminates redundant prefill compute, and evaluate the latency trade-offs that govern offloading KV caches to host system memory.
- ☐ **Joules per token**: Explain why energy consumption scales with generated sequence length, and how low hardware utilization inflates the energy cost per useful token.

Every LLM serving optimization ultimately decides how many sequences share each weight stream and how much accelerator memory each sequence holds. The chat-energy calculation showed that the same sharing sets the joules spent per useful token: each decode step streams the weights once for the whole active batch, so a well-filled batch divides that energy across sequences. Unlike fixed-output inference, LLM energy also grows with response length. Carbon accounting additionally requires an emissions factor and a defined boundary. These per-token bounds describe one accelerator. Deciding how many accelerators a service needs, and what each generated token costs, requires carrying them, together with the fixed-output results of the earlier sections, into capacity planning and unit economics.

13.11 Economics and Planning

Batching, precision tuning, kernel fusion, graph compilation, and the LLM levers that decide how many sequences share each weight stream can reduce latency, increase capacity, or lower the cost of a completed inference. Production deployment, however, requires answering a fleet-level question: how many machines, of what type, and at what total cost. A team that has raised a V100's ResNet-50 throughput several-fold through batching still needs to know whether a few V100s or several times as many lower-cost T4s yield lower cost for its 5,000 QPS target while meeting the same SLO. Serving infrastructure cost grows with sustained request volume (Zhang et al. 2019), while training cost follows its own dataset, model, and optimization budget. The bitter lesson sharpens this asymmetry. The methods that win are the ones that scale, so successive models grow larger, and while training pays for that scale once, serving pays for it on every request. The multi-order-of-magnitude price collapse charted in figure 13.2 adds market pressure on top. As public API token prices plunge from thirty dollars per million tokens to several cents, infrastructure efficiency becomes the decisive factor in operational viability.

↩ The bitter lesson, that general methods which scale with computation keep outperforming hand-engineered ones, is established in section 1.4.

13.11.1 Cost per inference

The **cost per inference** divides the allocated serving cost by the number of completed inferences that satisfy the service objective. The numerator can include provisioned CPU or accelerator time, memory capacity, data transfer, storage, and orchestration. At high utilization, useful throughput spreads that provisioned cost across many requests. At low utilization, paid capacity sits idle, so the same hourly bill is divided among fewer successful inferences. Applying the framework to ResNet-50 shows how hourly price and sustained valid throughput combine into a unit cost.

13.11.2 GPU vs. CPU economics

Hardware selection for model serving presents an asymmetric trade-off between nominal hourly lease rates and parallel execution efficiency. While specialized GPU instances incur hourly costs that exceed commodity CPU instances by integer multiples, their massively parallel matrix units deliver throughput gains that can reduce the effective cost per completed query by an order of magnitude. The economic crossover point between general-purpose CPUs and dedicated accelerators is governed by arithmetic intensity, batchability, and the strictness of the latency deadline.

Host CPU execution is economically optimal for lightweight models with modest parameter counts, applications with relaxed latency thresholds (hundreds of milliseconds), and workloads characterized by low or unpredictable query density where dedicated accelerator capacity sits idle. Conversely, GPU acceleration dominates when models feature dense tensor operations, when service-level objectives require sub-tens-of-milliseconds response times, and when sustained traffic volumes enable dynamic batching to amortize kernel launch and memory streaming overheads.

Beyond steady-state costs, startup time affects scaling economics. Instance launch, image pull, driver initialization, model loading, compilation, and warmup can make accelerator replicas slower to become ready than CPU replicas, although exact times depend on the platform and deployment artifact. If traffic rises faster than useful capacity can come online, latency SLOs can fail despite sufficient eventual capacity.

When accelerator replicas have a long readiness delay, predictive scaling or a warm pool may be more reliable than purely reactive scaling. For a model that runs acceptably on both device types, a hybrid design can use always-on accelerator capacity for baseline load and CPU overflow for spikes, trading a different unit cost and latency profile for faster reserve capacity. Whether this GPU+CPU hybrid works depends on model compatibility and identical output semantics across runtimes.

Example 13.15: ResNet-50: Cost analysis

Table 13.21 gives an illustrative AWS-like scenario for hourly cost, throughput, and per-million-image cost when serving ResNet-50:

Table 13.21: Illustrative ResNet-50 cloud inference cost comparison: Assumed hourly cost and saturated throughput, with derived cost per one million images for CPU, T4, and V100 instances.

Instance Type	Cost/Hour	Throughput	Cost per 1M Images
c5.xlarge (CPU)	$0.17	50 img/s	$0.94
g4dn.xlarge (T4 GPU)	$0.53	400 img/s	$0.37
p3.2xlarge (V100 GPU)	$3.06	1,200 img/s	$0.71

Systems insight: The T4 GPU instance has the lowest unit cost under these assumptions. The V100 does not beat the T4 on unit cost at the stated full-utilization rates. Cloud pricing varies by region and changes over time; consult current pricing for production planning.

13.11.3 Capacity planning

The GPU vs. CPU decision establishes the cost per inference, but determining how much infrastructure to provision requires combining cost analysis with the queuing theory foundations from section 13.5. **Capacity planning** translates three inputs into infrastructure specifications: traffic patterns (peak request rate, daily/weekly cycles, growth projections), latency SLOs (p50, p95, p99 targets), and model characteristics (inference time distribution at various batch sizes) (Harchol-Balter 2013).

The worked example in section 13.5 demonstrates the complete workflow: starting from a 50 ms queue-plus-GPU p99 subbudget, a separately reserved outside-worker allowance, and a 5,000 QPS target, deriving the M/M/1 utilization threshold of 54 percent from equation 13.5, and determining GPU count with headroom of 62 V100s. That count assumes batch-1 service. For an illustrative capacity comparison, applying the same utilization factor to the saturated batched rate of 1,200 img/s in table 13.21 gives 8 V100s before headroom and 11 after; the batch-1 plan needed 47 and 62.

This arithmetic compares assumed capacities under different service models; it does not establish equal p99 latency. Batching changes service times, realized batch sizes, and waiting distributions, so the factor derived for unbatched M/M/1 service cannot by itself certify the batched plan. The throughput opportunity is the batching gain of section 13.7; its usable capacity requires a load test including formation delay and worker backlog. This scenario provisions peak load plus 30 percent headroom; production margins depend on the workload and policy. Autoscaling can reduce costs during low-traffic periods while meeting latency objectives during peaks; section 14.5.1 develops the operational policy layer around these serving calculations. Capacity must be sized for requests that meet the SLO, not for raw request volume.

↪ Autoscaling policies that adjust serving capacity to demand, including their cold-start constraints, are operationalized in section 14.5.1.

13.11.4 Production case study: Serving 8-billion-parameter Llama 3

In large-language-model serving, resident model weights establish only the static memory baseline; dynamic key-value (KV) cache memory dictates the physical serving envelope and maximum concurrency. To ground the memory and compute trade-offs of generative inference end to end, this case study analyzes an 8-billion-parameter Llama 3 deployment on a single accelerator. Before evaluating the 8-billion-parameter workload contract, figure 13.7 illustrates why KV-cache growth dominates capacity planning across the transformer architecture. At frontier 70-billion-parameter scale with grouped-query attention on an 80 GB A100 GPU, 4-bit weights consume roughly 35 GB, leaving 51 decimal GB for dynamic cache before runtime workspace. As context length grows, batch size 32 breaches that physical capacity ceiling at approximately 4,800 tokens, precipitating out-of-memory failure. The 8-billion-parameter model analyzed below obeys these identical physical laws but operates with substantial memory headroom on modern hardware, allowing an engineer to balance batching, prompt prefill, and autoregressive decoding on a single device without multi-node sharding.

13.11.4.1 Workload profile

The following fixed workload assumptions define the reference case used throughout the latency, memory, and economics calculations; together, they bound the analyses that follow:

- **Model**: 8-billion-parameter Llama 3 (quantized to 4-bit using activation-aware weight quantization (AWQ); see section 10.4 for quantization techniques) (Dubey et al. 2024; Lin et al. 2024).
- **Hardware**: 1× NVIDIA H100 SXM5 GPU (80 GB HBM3 device capacity, equivalent to 85.9 GB on the decimal basis used for tensor footprints, 3.35 TB/s bandwidth) (Choquette 2023).
- **Request characteristics**: 1,000-token input prompt (Prefill), 256-token generated response (Decode).
- **Target SLOs**: TTFT $<$ 200 ms, TPOT $<$ 20 ms.

Hardware datasheets report accelerator memory in binary gigabytes, whereas tensor footprint calculations use decimal gigabytes (GB). For the A100 memory baseline in figure 13.7, 85.9 GB expresses device capacity in decimal units, making weight and cache footprints directly comparable before reserving space for runtime workspace.

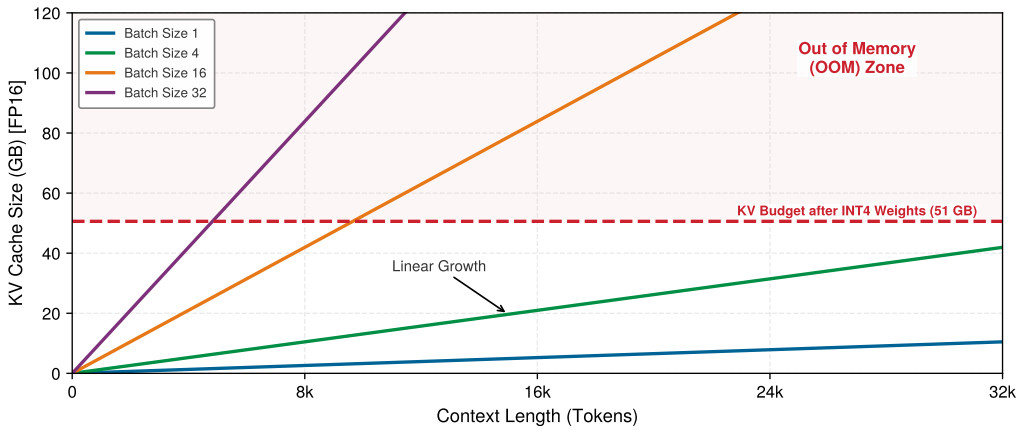


Figure 13.7: The KV-Cache Explosion: FP16 KV-cache usage vs. context length for a 70-billion-parameter-class model with GQA. On the modeled A100, 35.3 GB of INT4 weights leave 50.6 GB for KV cache before runtime overhead. Batch size 32 reaches that cache budget at roughly 4.8k tokens, forcing a trade-off between batch size and context length.

As demonstrated by figure 13.7, longer context lengths force serving engines to restrict concurrent batch sizes to avoid out-of-memory crashes, trading throughput efficiency for sequence length. In contrast, the case study's 8-billion-parameter workload (1,000-token prompt, 256-token decode) fits comfortably within the H100 memory budget, allowing analysis to isolate the two foundational serving bounds: the compute-bound prefill phase governing time to first token and the memory-bandwidth-bound decode phase governing time per output token.

13.11.4.2 Latency deconstruction

The end-to-end request latency is governed by the two-phase execution model of autoregressive transformers, applying the TTFT and TPOT metrics defined in section 13.10.1. Prefill determines whether the user sees a prompt response quickly; decode determines whether the generated stream keeps moving after it starts.

> Systems Perspective 13.10: The physics of token generation
>
> Recall the energy-movement principle quantified in table 4.1: moving a bit is 100–1,000× more expensive than computing on it. In the decode phase, this law determines the physical "cost per word."
>
> **Physics**: For batch-one decode with INT4 weights, the weight-dominated arithmetic intensity is approximately 2 FLOPs per parameter divided by 0.5 byte per parameter, or 4 FLOPs/byte. This is far below the H100 ridge point, so the modeled step is memory-bandwidth bound.

Larger decode batches reuse the weight stream across sequences and raise arithmetic intensity (Pope et al. 2023). The batch-one bandwidth floor is captured in equation 13.17:

$$T_{\text{token}} \approx \frac{D_{\text{vol}}}{\text{BW}_{\text{accelerator}}} \tag{13.17}$$

Implication: Every token generation pays an energy cost to move model weights from HBM into compute registers. For comparison with the case study's H100, on an A100 80 GB (2.04 TB/s HBM2e), the same model has a theoretical weight-streaming floor of approximately 1.97 ms per token. When decode remains bandwidth-bound, adding more compute cores yields little latency improvement; faster memory (Physics), smaller models (Algorithm), or better batching and cache management change the bound.

Prefill phase (time to first token) The model processes the 1,000-token prompt in parallel. Assuming 40 percent of peak compute efficiency during prefill, the model-side TTFT estimate is 41.9 ms, comfortably within the 200 ms SLO and leaving the remaining budget for network ingress, tokenization, and queueing.

Decode phase (time per output token) The model generates 256 tokens sequentially. At batch one, this phase is memory-bandwidth bound: each step streams the 4 GB weight tensor from VRAM. A batched step amortizes that stream across its active sequences.

Dividing the 4 GB weight tensor by the 3.35 TB/s bandwidth yields the theoretical floor: $T_{\text{token}} \approx$ 1.2 ms. Adding an assumed per-layer dispatch overhead gives a modeled TPOT of 1.57 ms. This dispatch overhead reflects the cumulative cost of launching 8–10 discrete GPU kernels (RMSNorm, RoPE, projections, attention, multilayer perceptron (MLP)) across each of the model's 32 layers. At batch size one, launching $ $250–300 small kernels adds several milliseconds of CPU-GPU submission and driver overhead, which serving engines mitigate by capturing the decode graph into a single CUDA Graph. Generating all 256 tokens therefore takes 256 tokens $\times$ 1.57 ms = 0.40 s.

13.11.4.3 Memory and throughput

With 4-bit weights occupying 4 GB, the remaining ~81.9 GB is available for the KV cache and runtime workspace. Each token requires approximately 0.131 MB of FP16 KV cache in the 8-billion-parameter Llama 3 configuration; while AWQ compresses weights to 4 bits, it leaves the KV cache in 16-bit precision. This per-token footprint derives directly from the architecture: storing 16-bit Key and Value tensors across $N_L = 32$ layers with $H_{\text{KV}} = 8$ grouped-query attention heads of dimension $d_{\text{head}} = 128$ requires $2 \times 32 \times 8 \times 128 \times 2$ bytes $= 131{,}072$ bytes ≈ 131 KB per token per sequence, cutting cache volume to one-fourth that of standard 32-head multi-head attention. Dividing the 77.3 GB reserve by that per-token cost yields capacity for ≈ 0.6 million tokens, so at 1,256 tokens the GPU can hold a concurrent batch size of ~469 requests.

13.11.4.4 Unit economics

Consider a representative H100 SXM5 rental cost of approximately $3/hour and an assumed isolated prefill rate of 10,000 prompt tokens/s. Dividing that rate by the 1,000-token prompt gives the prefill request capacity. For this calculation, each completed request counts both prompt and generated output tokens, totaling 1,256 tokens. Multiplying the limiting request rate by this token count and the seconds in an hour gives 45.2 million tokens/hour; dividing the hourly rental cost by that throughput gives $0.066/million tokens.

The comparison also converts KV residency into a request rate by dividing the resident request count by the modeled time to complete prefill and decode. It assumes that all those resident requests can advance at the modeled per-request pace, which is an optimistic concurrency assumption rather than a measured decode capacity. Under these assumptions, isolated prefill supplies the smaller rate and sets the displayed cost floor. Shared prefill/decode contention, batched decode timing, runtime workspace, and SLO-qualified throughput determine whether the service realizes that rate; memory fit alone does not establish it.

The resulting cost floor is a bound for this workload. A load experiment must supply the missing service capacity before the same arithmetic can price an accepted deployment configuration.

Checkpoint 13.6: Complete the LLM capacity decision

- ☐ **Define acceptance**: For the case study's 200 ms TTFT and 20 ms TPOT targets, state the required completion quantiles, availability requirement, and memory reserve before evaluating a configuration. Why do latency thresholds alone leave the SLO incomplete?
- ☐ **Measure the missing capacity**: Hold runtime, precision, hardware, and prompt/output lengths fixed. Increase offered request rate across several concurrency limits below the residency ceiling. For each warmed configuration, record TTFT and TPOT distributions, completed request rate, rejection rate, and peak allocated memory. Identify the configurations that satisfy all acceptance conditions.
- ☐ **Close the economic decision**: Recompute cost using each qualifying configuration's completed request rate and the case study's prompt-plus-output token convention. Select the lowest-cost qualifying setting for the required load. If none qualifies, identify whether lower offered load, a different batching/runtime policy, or different hardware is the next experiment. Explain why the KV-residency ceiling cannot supply the missing throughput observation.

For large language models, memory capacity (the size of the unallocated VRAM dedicated to the KV cache) determines maximum concurrent residency, while prefill compute and decode bandwidth determine realized token throughput and cost under a specific traffic mix. Memory bandwidth bounds low-batch decode latency; larger batches amortize parameter movement and shift the limit toward compute or KV-cache traffic.

Physical constraints govern serving economics across the entire stack: latency budgets partition into prefill and decode phases, queueing theory dictates batch sizing and capacity headroom, and memory bandwidth bounds decode throughput. Misinterpreting these interactions leads directly to the operational fallacies and pitfalls that follow.

13.12 Fallacies and Pitfalls

Serving inverts training priorities in ways that violate intuitions from batch processing. The nonlinear relationship between utilization and latency, the hidden costs of preprocessing, and the silent failure modes of training-serving skew cause violated SLOs, wasted optimization effort, and accuracy degradation invisible to standard monitoring.

Fallacy: *Reducing model inference latency proportionally reduces user-perceived latency.*

Engineers who optimize model inference expect proportional improvement in user-perceived latency, but serving systems introduce latency sources absent from offline benchmarks. Under load, queuing delay dominates: equation 13.4 shows that at 80 percent utilization with 5 ms service time, average wait time is 20 ms before inference even begins. Reducing inference from 5 ms to 2 ms changes service time but also shifts utilization from 80 percent to 32 percent, reducing queuing wait from 20 ms to 0.9 ms, a 21.2× queuing improvement that dwarfs the 3 ms inference gain.

This nonlinear interaction between inference speed and queuing behavior produces an apparent paradox: the system-level speedup (25 ms $\rightarrow$ 2.9 ms, or 8.5×) far exceeds the model-level speedup (5 ms $\rightarrow$ 2 ms, or 2.5×). At first glance, this superlinear improvement appears to contradict Amdahl's law, which states that total system speedup is strictly bounded by the enhanced execution fraction. However, the fixed-fraction Amdahl calculation holds other stage costs unchanged; serving is a dynamic queueing system. Average queue waiting time scales nonlinearly with server utilization: $W_q \propto \frac{\rho_{\text{serv}}}{1-\rho_{\text{serv}}}$, where $\rho_{\text{serv}} = \lambda_{\text{arr}} \cdot T_{\text{svc}}(1)$ for unbatched service. When kernel optimization cuts service time $T_{\text{svc}}(1)$, it does not merely accelerate execution; it simultaneously lowers server utilization ρ_{serv}. Pulling utilization away from saturation collapses queue waiting times nonlinearly. Kernel optimizations thus pay a compound dividend: shortening compute time and annihilating the queue. Conversely, profiling inference latency in isolation without modeling arrival load conceals the primary driver of user-perceived performance.

Pitfall: *Running serving infrastructure at high utilization to maximize cost efficiency.*

Teams target 90 percent utilization to minimize idle capacity. In production, latency degrades nonlinearly as utilization approaches capacity. Equation 13.4 shows that at 90 percent utilization, average time in system reaches 10× service time. Moving from 70 percent to 90 percent utilization cuts modeled infrastructure cost by 22.2 percent but triples average latency. For a 5 ms inference service, modeled p99 latency rises from ~76.7 ms to ~230 ms under the M/M/1 assumptions. In this scenario, operating at 60 to 70 percent utilization preserves substantially more latency headroom than 90 percent. A production target must instead be selected from the measured arrival process, service-time distribution, SLO, failure reserve, and cost objective.

Fallacy: *Training accuracy guarantees serving accuracy.*

Engineers assume identical model weights preserve validation-set performance. In production, preprocessing differences can shift inputs outside the training distribution. Section 13.6.1 explains how resize interpolation, numerical precision, and feature timing can create skew despite identical weights. Validation accuracy holds only over the population it sampled (principle 9), and a serving path that transforms inputs differently presents the model with a different population. The illustrative scenario here drops from 95 percent to 90 percent, a 5 percentage-point loss; these values are not a reported production incident. Production systems require consistent preprocessing and task-quality monitoring because latency and exception checks cannot detect this failure.

Pitfall: *Using average latency to evaluate serving system performance.*

Engineers monitor average latency because it trends smoothly and is simple to compute. An average, however, cannot show whether the percentile named by an SLO is failing. As section 13.5.4 demonstrates, at 70 percent utilization with 5 ms service time, average latency is 16.7 ms but the modeled p99 reaches 76.7 ms, a 4.6× gap invisible to mean-based monitoring. Services therefore report the relevant latency distribution and SLO percentile rather than the mean alone.

Fallacy: *Larger serving batches always improve throughput without affecting latency SLOs.*

Engineers maximize batch size assuming GPU saturation improves cost efficiency under production load. In serving systems, however, batching introduces a latency-throughput trade-off governed by queuing dynamics absent from offline benchmarks. Accumulating requests into larger batches increases wait time for early arrivals: an oldest-request window of 10 ms can add up to 10 ms of formation wait if the batch-size trigger does not fire first. Waiting for an available worker can add further delay; the combined distribution determines p99 latency. In the representative ResNet-50/V100 scenario in section 13.7.5, increasing batch size from 16 to 32 improves throughput only 12 percent but nearly doubles per-batch inference time from 14 ms to 25 ms, and variable input sizes within a batch can create padding overhead that wastes compute on padding tokens. Section 13.7.4 shows why, for tight p99 targets, larger batch sizes can violate SLOs when batch formation delay plus increased per-batch inference time exceeds the latency budget. Serving batch optimization requires jointly tuning batch size, batch timeout, and concurrency against latency SLOs under realistic traffic patterns, not maximizing throughput in isolation.

Pitfall: *Calibrating quantized models with unrepresentative data.*

Teams may calibrate with a convenient sample even when it does not represent the supported deployment distribution. Post-training quantization determines INT8 scale factors from activation ranges in calibration data, so representative coverage matters. The illustrative scenario here drops from 76.1 percent to 72.9 percent, a 3.2 percentage-point loss; it is not a reported production incident. Effective quantization requires calibration and validation on data representative of the inputs the service is expected to handle.

Fallacy: *Cold start latency only matters for the first request.*

Engineers optimize steady-state latency assuming most requests hit warm instances. Cold starts can compound during traffic spikes, deployments, and recovery. In this illustrative scenario, 10 instances each require 30 s of compilation, totaling 300 instance-seconds; parallel warming makes capacity useful after about 30 s. The assumed cold-request latency is 500 ms versus 5 ms at steady state.

Pitfall: *Scaling without a warm-pool or staged-loading budget.*
Autoscaling policies that count only steady-state replicas underestimate the capacity needed during traffic spikes and deployments. Serving systems need warm pools (pre-initialized spare replicas), staged model loading, or admission control so that new replicas become useful before user requests depend on them. The budget should include compilation, weight loading, cache initialization, and health-check time, because those steps determine whether scale-out adds capacity or adds another source of tail latency.

13.13 Summary

Serving moves a validated model into an operating workload. Online requests add a latency distribution, queueing, traffic variability, and failure reserve to the throughput concerns familiar from training. Little's Law relates stable long-run population, arrival rate, and response time; M/M/1 supplies an intentionally simple model of nonlinear latency growth near saturation. Neither replaces measurement, but together they turn headroom and capacity into explicit hypotheses that a load test can validate.

Effective serving optimization measures the complete request path rather than model inference alone. Protocol and serialization choices matter when transport is a visible fraction of the budget; preprocessing matters when it feeds an accelerator too slowly; batching matters only when arrivals and the SLO permit waiting. Training-serving skew adds a different failure mode: the service can remain fast and error-free while inconsistent transformations silently change its predictions.

The chosen deployment paradigm changes the feasible request path. A cloud API may use dynamic batching under a measured arrival process, synchronized sensors impose a frame deadline, and a single-stream mobile workload may choose batch one to protect responsiveness and energy. The MLPerf scenarios provide standardized approximations of these shapes; production design still begins with the service's own traffic, hardware, and objective.

Node-level optimization connects the model's operations to the machine that executes them. Profiling reveals whether compilation, kernel fusion, precision, locality, serialization, or loading actually binds the request path. Gains matter only when they survive production input shapes, accuracy checks, and latency variation. Token-streaming language models stretch the whole toolkit at once. Latency splits into time to first token and time per output token, batching must reschedule at every generated token, and the KV cache, not arithmetic, bounds how many requests can share an accelerator. Capacity planning converts that valid throughput into replica count and unit cost, while the Llama 3 case study in section 13.11.4 separates calculable capacity bounds from the load experiment needed to establish valid throughput. A faster kernel that neither reduces provisioned capacity nor improves the SLO has not produced an economic gain.

Key Takeaways: Inverting every training priority

- **Serving is latency economics**: Training rewards throughput over long runs, but serving spends a fixed per-request budget across serialization, preprocessing, queuing, inference, postprocessing, and the network. Optimizing only model latency misses the stages users actually wait on.
- **Utilization turns into waiting**: In the chapter's M/M/1 approximation, average time in system rises from 5× service time at 80 percent utilization to 10× at 90 percent. Headroom keeps modest traffic surges from becoming SLO failures.
- **Fast models reveal pipeline taxes**: Once inference is fast, image decode, tokenization, and other preprocessing can dominate total latency. The binding optimization becomes the request path, not the neural network kernel.
- **Batching follows traffic, not habit**: Poisson-modeled web arrivals can use dynamic batching, synchronized sensors need aligned batches, and single-stream mobile workloads may have little batching opportunity. A useful window converts available latency slack into throughput without violating the SLO.

- **Skew breaks accuracy without errors**: Resize methods, color-channel order, normalization constants, or feature definitions that differ between training and serving can change the model's effective input distribution. Shared transformations, parity tests, and production-slice monitoring reduce this risk.
- **Runtime choices become infrastructure bills**: Precision, graph compilation, kernel fusion, and serving runtime translate directly into replica count and cost per inference. Quantization and specialized runtimes can materially reduce required serving capacity when they preserve accuracy and fit the target hardware.
- **LLM serving is memory management**: Decode often reads weights from VRAM for every generated token, so token latency is bandwidth-bound unless batching changes the constraint. KV-cache layout, PagedAttention, continuous batching, precision, and runtime choice determine both concurrency and cost per token.

Training is judged by useful work completed; an online service is judged by whether requests meet their latency objective. Each stage spends against the same envelope: serialization, preprocessing, the queue, the model, and the response. Data determines input size and arrival shape, the algorithm determines compute and state, and the machine determines service rate and movement costs. Queueing couples all three nonlinearly, so a system within budget at moderate traffic can breach it after a small surge. Serving engineering therefore allocates the budget across the request path, preserves headroom for variability and failure, and verifies that the required fraction of requests finishes before the deadline, which is the latency budget principle (12) in operational form.

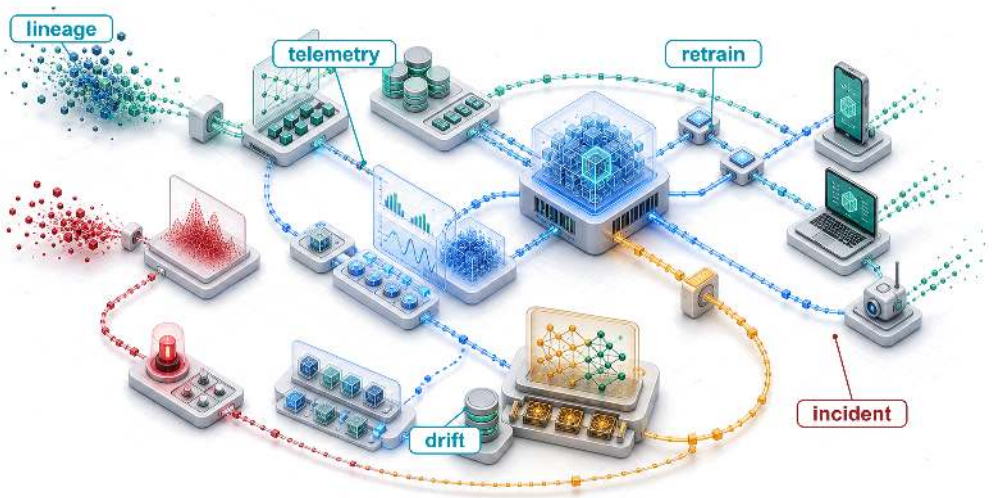


What's Next: From fast to still correct

This chapter evaluates the request path under a specified workload: latency within budget, headroom preserved, and useful throughput converted into cost. Those performance checks do not establish whether the answers are still right. Training-serving skew showed that a service can stay fast and error-free while its predictions change, and drift produces the same silence when the world moves rather than the pipeline. Chapter 14 widens the unit of concern from request-path performance to the model's operating lifetime. Monitoring tracks prediction quality alongside system health, feature stores and model registries keep serving aligned with training, and staged rollout with rollback lets a replacement model reach users without turning a bad release into an outage.

14

ML Operations

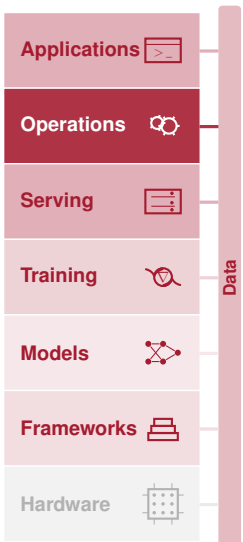


14.1 MLOps Overview
14.2 MLOps Principles
14.3 Technical Debt
14.4 Development Infrastructure
14.5 Deployment and Serving
14.6 Production Monitoring
14.7 Incident Response and Governance
14.8 Design and Maturity Framework
14.9 Case Studies
14.10 Fallacies and Pitfalls
14.11 Summary

Purpose

Why can an ML system be perfectly available and perfectly wrong at the same time?

Conventional monitoring often detects crashes, latency regressions, and availability failures quickly, but semantic errors can remain silent in any software system. ML systems add another silent failure mode. A model experiencing data drift can continue serving ordinary-looking predictions while accuracy degrades, triggering no alerts because every health check (latency, throughput, uptime) remains green. A fraud model can keep answering in milliseconds for months after the patterns it learned have moved elsewhere, and the first alarm may be the losses themselves. Serving infrastructure gets models into production; operations helps keep them correct once they are there. Model quality can degrade because the world changes. Customer behavior shifts, new product categories appear, seasonal patterns evolve, and the distribution the model learned from diverges from the one it now faces. Drift is not a certainty but a persistent risk for deployed models. Unlike latency or uptime, correctness may become observable only after labels or downstream outcomes arrive, so detecting degradation often requires imperfect signals and explicit investigation rather than a single threshold. Managing it requires continuous monitoring that tracks prediction quality alongside system health, drift signals that trigger investigation, retraining when outcome evidence supports it, and deployment strategies that validate new model versions against production traffic before full rollout. The gap between development and production is not a hurdle cleared once but a condition managed indefinitely, for as long as the model makes decisions that someone relies on. Machine learning operations exists because uptime without measured accuracy may still deliver wrong answers at scale. In D·A·M terms, operations makes co-design continuous, because the data environment can remain a moving target long after initial deployment.



Learning Objectives

- Explain why ML systems can remain available while prediction quality silently degrades under distribution shift
- Diagnose technical debt across data-model, model-infrastructure, and production-monitoring interface boundaries
- Design feature stores, registries, and CI/CD pipelines that preserve training-serving consistency and reproducible rollback
- Apply the retraining staleness model to choose cost-aware retraining triggers and intervals
- Design layered monitoring and an incident-response workflow that trace drift, skew, and degradation to a data, model, or infrastructure cause
- Compare canary, blue-green, shadow, and rollback strategies for production model release risk
- Evaluate operational maturity and investment using model criticality, operational risk, and organizational readiness

14.1 MLOps Overview

After a model is built, optimized, benchmarked, and served, the system still has to remain correct. A benchmark establishes performance at a point in time, and serving infrastructure verifies that the required fraction of requests finishes within the latency budget. Neither check asks whether the answers are still right. The team deploys to production, and week one looks excellent. The challenge begins in week two.

Data distributions shift, user behavior changes, and the world moves on from the conditions under which the model was trained. ML models that succeed in development can fail to achieve sustained production use when postdeployment ownership and monitoring are absent. The root cause is an operational mismatch: availability-focused monitoring tracks server uptime, request latency, and request success rates, while ML monitoring must also track statistical health, including accuracy over time, input-distribution shift, and per-segment prediction quality. A model can degrade substantially while throwing no exceptions, triggering no infrastructure alarms, and maintaining perfect uptime.

The discipline that makes these invisible failures visible is **machine learning operations (MLOps)**. MLOps synthesizes monitoring, automation, and governance into production architectures that detect degradation, trigger retraining, and maintain system health throughout a model's operational lifetime. It inherits the automation and operations lineage of DevOps (Kreuzberger et al. 2023), but addresses a different failure mode. Conventional services can often be tested against deterministic code paths, while ML systems depend on training data distributions, learned parameters, and environmental conditions that shift continuously.

The week-two problem takes concrete shape in an illustrative deployment scenario. Consider a demand prediction system for a ridesharing service. Initial measurements show 94 percent accuracy, 15 ms p99 latency, and strong performance across test segments. By week four, accuracy has dropped to 88 percent, but the infrastructure metrics show nothing wrong. By week eight, a product manager notices driver dispatch is inefficient; investigation reveals the model has not adapted to a competitor's new promotion that shifted user behavior. The model needed investigation six weeks ago, but no system was watching for this degradation. MLOps provides the framework to detect such changes, assess whether retraining addresses them, and validate replacement models while limiting the impact on users.

The operational mismatch connects directly to the book's analytical foundations. If benchmarking provides the *sensors* for the system, MLOps is the complete *control system*, and the degradation equation (equation 1.2) names the quantity it controls. Accuracy falls by a fitted sensitivity λ times the divergence between the serving and training distributions, so operations must measure that divergence continuously and estimate λ from paired drift and outcome data, because divergence alone does not prove that accuracy will fall. MLOps also formalizes interfaces and responsibilities across traditionally isolated domains (data science, machine learning engineering, and systems

operations (Amershi et al. 2019)) through continuous retraining, A/B evaluation, graduated rollout, and standardized artifact tracking that supports reproducibility and auditability.

Deploying, monitoring, and maintaining one production ML system defines the fundamental operational unit: the **ML node**, a complete system comprising data pipelines, feature computation, model training, serving infrastructure, and monitoring for a single machine learning application. Platform operations at larger scale (managing hundreds of models, cross-model dependencies, multi-region coordination, and organization-wide ML platform engineering) constitute advanced topics that build on these single-model foundations.

Keeping one ML node correct decomposes into three distinct interfaces. The **data-model interface** is the handoff between data infrastructure and model training; its goal is feature consistency, so training and serving pipelines compute features the same way. The **model-infrastructure interface** is the transition from trained weights to scalable service; its challenge is environment parity, because a model that works in a notebook may fail in production due to version, dependency, or runtime mismatches. The **production-monitoring interface** is the feedback loop that enables self-correction, returning statistical telemetry[1] from production to training because ML systems can degrade through drift without crashing.

[1] **Telemetry**: The feedback path that can make model degradation visible before it becomes a business failure; crashes and error codes expose some software failures, but semantic errors can remain silent in any application. ML systems add distribution shift, which can go undetected without statistical telemetry such as feature distributions, prediction confidence, and drift indicators.

These interfaces determine how infrastructure components partition responsibility. Feature stores stabilize feature computation at the data-model boundary. Model registries and deployment pipelines preserve the model-infrastructure handoff. Drift monitors, retraining triggers, and governance policies connect production evidence to a controlled response, shortening the window in which degradation goes unaddressed.

Operating this feedback path requires adding statistical controls to the release and reliability practices of DevOps. Technical debt accumulates wherever one of the three interface boundaries erodes. Feature stores, versioning, and CI/CD pipelines restore the reproducibility needed to isolate a fault at those boundaries, while monitoring, drift detection, and staged rollout protect predictive quality once inputs begin to shift. A small set of foundational principles decides which of these controls a given model needs and when each should act.

14.2 MLOps Principles

A production machine learning release extends far beyond an executable binary or code diff. Data distributions, learned parameters, slice-level evaluations, and runtime feedback loops all function as release artifacts that govern production behavior. Traditional DevOps automates software integration, test suites, and deployment manifests under the premise of deterministic code paths (Kreuzberger et al. 2023; Amershi et al. 2019). MLOps extends this discipline to statistical, data-dependent workflows spanning data acquisition, preprocessing, distributed training, evaluation, deployment, and continuous monitoring through an iterative loop connecting system design, model development, and live operations.

Definition 14.1: MLOps

Machine learning operations (MLOps) is the engineering practice of productionizing ML systems through CI/CD, workflow orchestration, reproducibility, versioning, collaboration, continuous training and evaluation, monitoring, and feedback loops (Kreuzberger et al. 2023).

1. **Significance**: The cost of leaving this loop open manifests as stale predictions, silent accuracy degradation, and prolonged recovery times. Drift thresholds, retraining triggers, and mean time to recovery (MTTR) targets are deployment-specific quantities calibrated from prediction value, label delay, validation risk, and retraining compute expense. The central engineering discipline is the closed control loop: measure distribution shift, estimate staleness cost, trigger retraining only when expected accuracy gains outweigh execution cost and rollout risk, and validate replacement models before promotion.
2. **Distinction**: DevOps monitors functional correctness, system throughput, and infrastructure availability. MLOps introduces statistical controls for *predictive correctness*, which can decay silently while server uptime, request latency, and HTTP status codes remain completely healthy.

3. **Common pitfall**: A frequent misconception is that retraining on new data automatically resolves distribution shift. Retraining without diagnosing which distribution shifted—input features ($p(x)$), label conditionals ($p(y \mid x)$), or both—preserves underlying failure modes or amplifies collection bias. Under covariate shift and common support, sample reweighting or stratified resampling can correct a mismatch in population weighting, but calibration still requires validation. Concept drift calls for new outcome evidence to evaluate whether the model needs adaptation.

The loop has three stages (design, model development, and operations), but its defining feature is the return path. Runtime telemetry and retraining triggers carry live performance signals from operations back into requirements and data engineering, so deployment feeds the next design cycle instead of ending the project (figure 14.1).

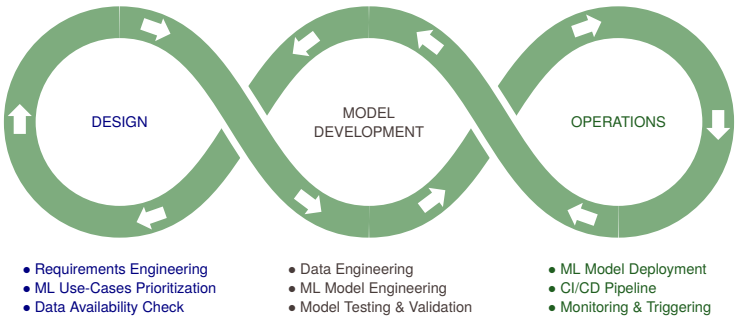


Figure 14.1: Iterative MLOps Loop: Monitoring and triggering feed production evidence back into design and model development, turning requirements, engineering, testing, and deployment into a continuous operating cycle.

Operating this feedback loop transforms postdeployment management from an ad hoc troubleshooting exercise into a continuous engineering discipline. When a model's predictive quality decays while its servers remain fully operational, five complementary principles organize the diagnostic and remediation response: reproducibility, separation of concerns, consistency, observable degradation, and cost-aware automation.

14.2.1 Foundational principles

These five principles dictate how an engineering team isolates faults across the operational lifecycle. Reproducibility reconstructs the exact state of a deployed artifact. Separation of concerns isolates whether an anomaly originates in data ingestion, model training, online serving, or production monitoring. Consistency enforces identical feature semantics across training and inference. Observable degradation generates the statistical telemetry required to detect silent quality decline. Cost-aware automation evaluates whether an operational intervention justifies its compute cost and rollout risk. Together, they establish the operational invariants required to govern production learning systems.

14.2.1.1 Reproducibility

Reproducibility requires every artifact[2] that influences model behavior to be versioned and traceable. This principle extends beyond code repositories to encompass input datasets, feature schemas, hyperparameter configurations, and execution environments. Equation 14.1 expresses this dependency formally:

$$\text{Model Output} = f(\text{Code}_v, \text{Data}_v, \text{Config}_v, \text{Environment}_v; \xi) \tag{14.1}$$

where v identifies an immutable version hash and ξ captures execution nondeterminism. Exact reproduction requires freezing all four versioned inputs alongside random seeds, dataloader sample ordering, deterministic GPU reduction kernels, and identical accelerator hardware architectures.

Fixing random seeds (such as `torch.manual_seed(42)`) still fails to guarantee bitwise reproducibility across training runs on accelerator hardware. The root cause is finite-precision arithmetic.

[2] **Artifact**: Model weights are realized training outputs; the inputs and randomness that produced them cannot generally be inferred from the parameters. Versioning only code is therefore insufficient, and even versioning code, data, configuration, and environment may not guarantee exact replay when kernels or training are nondeterministic.

In exact arithmetic, addition is associative, so $(a+b)+c = a+(b+c)$. Floating-point formats such as FP32, FP16, and BF16 break associativity because each intermediate sum is rounded to fit a finite mantissa width. On an accelerator, some gradient kernels have thousands of concurrent threads accumulate into shared tensors in global memory using hardware atomic instructions (`atomicAdd`). Because warp scheduling and memory-system contention vary from run to run, the order in which those updates reach global memory is not fixed. If Run 1 accumulates three updates A, B, and C as $(A+B)+C$ and Run 2 accumulates them as $A+(B+C)$, the accumulated gradients can differ in their least significant bits. Over thousands of optimization steps, these microscopic round-off variances compound through non-linear activation functions, causing two runs with identical seeds and identical code to diverge into measurably different model weights unless deterministic reduction algorithms (which enforce a fixed summation tree at the cost of parallel throughput) are explicitly enabled.

↩ The floating-point formats, their dynamic ranges, and where accumulated rounding error matters in training are established in section 8.7.3.

14.2.1.2 Separation of concerns

Separation of concerns decomposes MLOps systems into distinct functional layers that can evolve independently, as table 14.1 shows:

Table 14.1: MLOps Separation of Concerns: Each layer addresses a distinct responsibility and evolves at different rates across the data, training, serving, and monitoring layers. This separation enables independent scaling and updates, reducing blast radius when changes are required.

Layer	Responsibility	Stability
Data Layer	Feature computation, storage, serving	Changes with data schema evolution
Training Layer	Model development, hyperparameter optimization	Changes with algorithm research
Serving Layer	Inference, scaling, latency management	Changes with traffic patterns
Monitoring Layer	Drift detection, performance tracking	Changes with business requirements

14.2.1.3 Consistency imperative

The architectural boundaries in table 14.1 allow teams to upgrade serving runtimes without retraining models, adjust alert thresholds without redeployment, and modify data ingestion pipelines while preserving interface contracts. That architectural independence remains safe only when training and serving environments process features identically, establishing training-serving parity as the consistency imperative of section 4.6.1. A violation of that parity is training-serving skew, and the training-serving skew diagnostic (principle 11) detects it without waiting for labels by comparing what the training and serving code paths compute, from features through model scores, for the same inputs. The financial consequence of this discrepancy is captured in equation 14.2:

$$\text{Skew Cost} = \text{Rate}_{\text{skew}} \times Q_{\text{day}} \times C_{\text{error}} \tag{14.2}$$

where $\text{Rate}_{\text{skew}}$ is the fraction of queries with erroneous predictions caused by training-serving skew, Q_{day} is the daily query volume, and C_{error} is the average business cost incurred per erroneous prediction.

For a system serving 1,000,000 queries/day with 1 percent skew-induced errors costing $0.10 each, annual skew cost reaches $365,000. This quantifies why consistency mechanisms represent investments with measurable financial returns. Shared feature stores, unified C++ or Rust feature-extraction libraries invoked identically during offline training and online serving, and automated schema validation tests support consistency at the system boundary; matched-input tests must still verify feature values and semantics.

14.2.1.4 Observable degradation

Observable degradation requires ML systems to make silent failures visible through continuous measurement. Model accuracy degrades along a continuous statistical spectrum rather than failing as a discrete binary crash. An observed failure's time signature (an acute drop, gradual drift, or localized cohort decay) governs both the detection mechanism and the operational response. The ridesharing model's four-week slide from 94 to 88 percent was a gradual signature, the kind that

trend analysis on accuracy catches and that no uptime or latency alert can see. Making degradation observable therefore means pairing each failure signature with the telemetry that can see it and the countermeasure it calls for, as table 14.2 outlines.

Table 14.2: Degradation Detection Strategies: Four failure signatures map to detectors and operational responses. Sudden drops require immediate diagnosis and may call for rollback, while gradual or subgroup degradation calls for diagnosis before retraining or targeted data collection.

Degradation Type	Detection Mechanism	Response Strategy
Sudden accuracy drop	Threshold alerts	Diagnose; roll back if release-related
Gradual drift	Trend analysis	Diagnose, then retrain if warranted
Subgroup degradation	Cohort monitoring	Targeted data collection
Latency increase	Percentile tracking	Infrastructure scaling

14.2.1.5 Cost-aware automation

Cost-aware automation balances computational costs against accuracy improvements. Let ΔAccuracy be the expected gain in accuracy percentage points, Value per Point the monetary value of one percentage-point gain over the decision horizon, Training Cost the compute and labor cost of a retraining run, and Deployment Risk the expected cost of validation and rollout failure. Equation 14.3 models this trade-off:

$$\text{Retrain if: } \Delta\text{Accuracy} \times \text{Value per Point} > \text{Training Cost} + \text{Deployment Risk} \tag{14.3}$$

The inequality acts as an operational decision gate rather than an unconstrained trigger. An automated pipeline should initiate retraining only when empirical evidence indicates that expected accuracy gains surpass execution cost and rollout risk. This economic trade-off governs retraining triggers, validation thresholds, and rollout gates; section 14.4.2.2 formalizes this model to calculate optimal retraining cadences. The margin by which the left side of equation 14.3 exceeds the right is the *net retraining value*, the quantity that a retraining trigger should test.

Each foundational principle becomes operational only when it is anchored to a measurable quantity, from a complete artifact hash for reproducibility to net retraining value for automation (table 14.3).

Table 14.3: MLOps Principles Summary: Quick reference for the five foundational principles that guide all MLOps tooling and practice decisions.

Principle	Core Insight	Key Metric
Reproducibility	Version all artifacts	Complete artifact hash
Separation of concerns	Independent layer evolution	Layers touched per change
Consistency	Training equals serving	Feature skew rate
Observable degradation	Make failures visible	Time to detection
Cost-aware automation	Optimize total cost	Net retraining value

How these principles manifest in practice depends on the workload. A recommendation system drifts daily as user preferences shift; a TinyML model deployed on embedded hardware may run unchanged for months. The monitoring strategy must match the archetype.

Lighthouse 14.1: Monitoring strategy by archetype

The dominant failure modes and monitoring priorities differ across workload archetypes. Table 14.4 compares four representative archetypes by drift pattern, monitoring metric, and example retraining trigger:

Table 14.4: Monitoring Strategy by Workload Archetype: Illustrative starting points for monitoring strategy. Real thresholds must be calibrated to the deployment's label delay, traffic volume, business risk, and alert-fatigue budget.

Archetype	Dominant Drift Pattern	Primary Monitoring Metric	Example Retraining Trigger
ResNet-50 (Compute Beast)	Visual distribution shift (lighting, camera, new object classes)	Accuracy on holdout set (ground truth available)	Accuracy drops > 2% from baseline (∼monthly for stable domains)
GPT-2 (Bandwidth Hog)	Vocabulary drift, topic shift, emerging entities	Perplexity on live traffic (no ground truth needed)	Perplexity increases > 10%; new vocabulary detected (∼weekly for news domains)
DLRM (Sparse Scatter)	User behavior shift, item catalog churn, cold-start items	CTR/CVR delta vs. historical cohorts	Engagement drops > 5%; catalog refresh (∼daily for e-commerce)
DS-CNN (Tiny Constraint)	Acoustic environment change (noise floor shift)	Duty cycle (wakeups/hour) + false positive rate	False wake rate > 1%; battery drain exceeds spec (∼quarterly OTA update)

Systems insight: Ground-truth availability governs monitoring design. The roofline regime sets cost and capacity but does not determine which quality signal is observable. Vision tasks with labels (ResNet-50) track holdout accuracy; language and recommendation services (GPT-2, DLRM) rely on perplexity or implicit clicks; microcontroller models without labels (DS-CNN) monitor duty cycles, while estimating false-wake rates requires a reference signal or reviewed examples. Retraining cadence is constrained by label delay, deployment access, validation cost, and the rate of observed change.

These principles respond to three recurring operational failures: concept drift,[3] data-quality defects (Schelter et al. 2018), and silent postdeployment degradation. Responding to them changes what a team must version and who must coordinate, not only what it monitors.

MLOps coordinates a broader stakeholder ecosystem and introduces specialized practices such as data versioning,[4] model versioning, and continuous statistical monitoring that extend beyond traditional software engineering. By treating datasets and pipeline state as first-class versioned artifacts alongside source code, this expanded operational scope turns deployment from a linear release pipeline into a continuous, evidence-driven feedback loop.

Checkpoint 14.1: The MLOps loop

MLOps is not linear; it is circular.

The feedback cycle

- ☐ **Loop closure**: Explain how production telemetry returns evidence to the model lifecycle and why an alert does not by itself authorize retraining.
- ☐ **Retraining trigger**: Distinguish a distribution change from a measured quality regression, and explain why either requires diagnosis before selecting a remedy.

The artifacts

- ☐ **Artifact versioning**: Identify the data, code, configuration, weights, and environment versions needed to trace a deployed model, and explain why versioning alone does not guarantee exact replay.

[3] **Concept drift**: Concept-drift and data-stream research formalized the problem that a model's target relationship can change after deployment (Widmer and Kubat 1996; Gama et al. 2014). In adversarial domains such as spam, fraud, and abuse detection, the distribution can actively adapt in response to the model, making continuous monitoring and retraining a structural requirement rather than an operational luxury.

[4] **DVC (Data Version Control)**: DVC brings Git-like versioning to datasets and model artifacts (Iterative 2024), solving the artifact gap that equation 14.1 formalizes: without data versioning, the Data_v term is unrecoverable, and no combination of code commits can reconstruct the model that was deployed.

Each iteration through the operational loop introduces data dependencies, model interactions, and configuration state invisible to traditional compilation and testing. When left unmanaged, these hidden dependencies accumulate into systemic technical debt that erodes operational velocity and degrades predictive accuracy. Understanding the architectural patterns of this debt is the prerequisite for designing the infrastructure that pays it down.

14.3 Technical Debt

In traditional software systems, technical debt[5] represents the future rework created by choosing expedient shortcuts over disciplined architecture (Cunningham 1992). In machine learning systems, that debt compounds through statistical coupling and data dependencies that code review and unit tests cannot detect (Sculley et al. 2015). A codebase can remain pristine while the external world shifts, rendering trained parameters obsolete and destabilizing downstream consumers. Systematic evaluation rubrics, such as the ML Test Score (Breck et al. 2017), structure this operational challenge by quantifying debt across data, model, infrastructure, and monitoring boundaries.

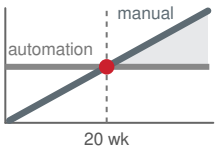
5 **Technical debt**: Ward Cunningham's 1992 WyCash experience report introduced the debt metaphor for expedient code and delayed consolidation (Cunningham 1992); in ML, debt can compound silently through data and model dependencies that code-focused unit tests and reviews may not detect. A pipeline can remain unchanged while the world it models changes. The ML Test Score rubric (Breck et al. 2017) makes this debt explicit through 28 production-readiness tests grouped into data, model, infrastructure, and monitoring sections.

Definition 14.2: Technical debt in ML

Technical debt in machine learning is the accumulating maintenance and change cost created by implicit data dependencies, entangled features, and undeclared consumers in ML systems, where the resulting liabilities may also appear as silent accuracy degradation.

1. **Significance**: Google's analysis of production ML systems argues that model code is only a small fraction of the surrounding system; the larger operational surface includes data collection, feature extraction, configuration, serving infrastructure, monitoring, and process management (Sculley et al. 2015). ML-specific debt drivers compound this: changing one input feature can silently shift the learned representation of every other feature (entanglement), a model trained to correct another model's errors creates a fragile dependency chain (correction cascades), and downstream systems consuming model outputs without explicit contracts become undeclared consumers that break silently when the model is updated.
2. **Distinction**: ML technical debt includes system- and data-level dependencies that code review and ordinary code-focused tests may miss. It can slow development and degrade prediction quality even while availability metrics remain healthy.
3. **Common pitfall**: A frequent misconception is that "better code" solves technical debt in ML. In reality, it is a systems architecture problem: the debt accumulates when the assumptions of the training distribution (feature ranges, label meanings, data freshness) are not enforced as runtime contracts at the system boundary.

A break-even calculation makes the cost dynamics of technical debt concrete. Teams often resist automation investment because manual processes seem faster in early development, but that advantage disappears once recurring manual interventions exceed the up-front automation cost.



Cumulative manual work overtakes the one-time automation investment near week 20.

Napkin Math 14.1: The cumulative cost of manual operations

Problem: Why build automated pipelines when manual retraining is faster?

Physics: Manual work accumulates over time.

- Manual retrain: 4 hours of engineering per week.
- Pipeline build: 80 engineering hours (one-time).

Math:

- **Result**: 80 engineering hours ÷ 4 hours per week = 20 weeks.
- **Assumption**: This simplified calculation sets pipeline upkeep to zero, so automation removes the weekly task entirely.

- **Upkeep**: Real pipelines need maintenance and new features. Each week then recovers only the difference between the manual load and the upkeep, which pushes break-even later; automation still pays back whenever its weekly upkeep stays below the manual work it replaces.
- **Consequence**: After 1 year, manual teams still spend 4 hours per week on maintenance, while pipeline teams spend only their upkeep (0 recurring hours under this calculation's assumption).

Context: A central law of systems engineering is that the cost of *maintaining* a system over its lifetime can dominate the cost of *building* it. In ML, technical debt is especially dangerous because it is often *data-driven* rather than *code-driven*: a perfect piece of code can still fail if the data it processes shifts. Measurement is the management boundary: without telemetry, the team cannot tell whether maintenance work is reducing debt or merely hiding it.

Systems insight: Automation addresses capacity limits, not speed alone, provided its upkeep stays below the manual load it removes. A manual team eventually reaches a point where maintaining existing models prevents it from deploying new ones. MLOps responds with systematic observability and automation. Without monitoring infrastructure to make silent failures visible, the team accumulates debt and builds a system that becomes increasingly difficult to manage.

The debt is larger than manual retraining alone, because modeling code constitutes only a small fraction of a real-world deployment. Figure 14.2 diagrams this disproportionate operational footprint, isolating core modeling code as just one peripheral satellite surrounding the central ML system, alongside nine supporting subsystems spanning data verification, feature extraction, configuration, and machine resource management. Their presence broadens the maintenance surface; the diagram does not measure each component's cost.



Figure 14.2: Hidden Infrastructure of ML Systems: ML code is one of ten components arranged around the ML system at the center, alongside data collection, data verification, feature extraction, configuration, machine resource management, serving infrastructure, monitoring, analysis tools, and process management tools. Modeling occupies one box and the remaining nine identify the operational surface a deployment must also carry; box counts do not measure code volume or maintenance cost. Source: (Sculley et al. 2015).

While automation addresses the recurring operational overhead of manual retraining, it cannot prevent architectural decay. Deployed systems accumulate hidden complexity through systemic failure modes that emerge from statistical coupling rather than deterministic logic. Figure 14.3 partitions hidden technical debt into six radiating spokes, linking core structural vulnerabilities—

such as parameter sprawl, fragile pipeline coupling, and undeclared consumer feedback loops—to the operational anti-patterns that erode production reliability.

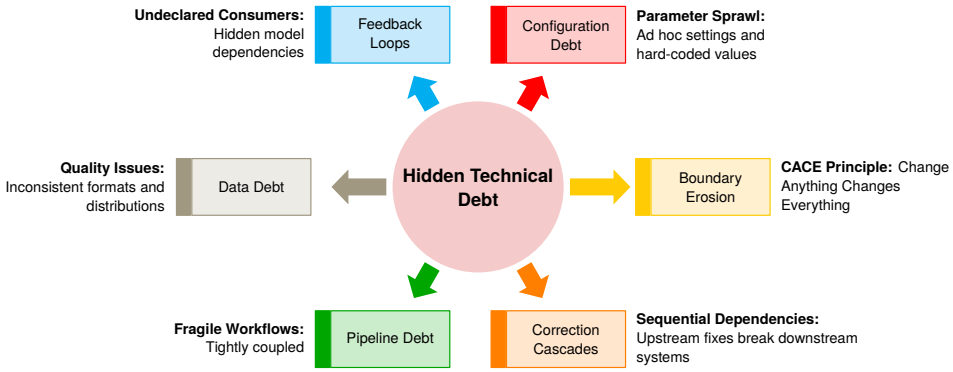


Figure 14.3: ML Technical Debt Taxonomy: Six debt patterns radiate from hidden technical debt: configuration debt, feedback loops, data debt, pipeline debt, correction cascades, and boundary erosion. The surrounding labels identify their associated failure patterns.

14.3.1 Boundary erosion

The dissolution of system boundaries represents the primary failure mode of statistical software. In conventional software, modular encapsulation and typed APIs make dependencies explicit, though changes can still violate behavioral contracts. Machine learning systems blur these boundaries because component behavior depends on the empirical distribution of incoming data rather than syntactic interface definitions. An upstream formatting modification or normalization change can satisfy every unit test while silently shifting feature distributions and degrading downstream model accuracy. This implicit coupling through data pipelines creates unmonitored dependencies among feature engineering, training pipelines, and serving endpoints.

This erosion produces **entanglement**, where dependencies between features and components become so intertwined that local modifications demand global coordination. The resulting failure mode reflects the CACE principle: *Change Anything Changes Everything*. When systems lack strict distributional boundaries, adjusting a feature encoding or data filtering threshold can change learned weights throughout the model during retraining. With frozen weights, the same change instead alters the inputs and potentially the predictions. For example, changing the binning strategy of a continuous feature shifts the feature's variance and covariance structure, causing a previously calibrated model to underperform and forcing downstream recalibrations that ripple across dependent services.

Defending against boundary erosion requires architectural modularity enforced at runtime. Components require explicit data contracts that govern statistical invariants, including allowed feature ranges, null tolerances, and drift thresholds. Enforcing separation between data ingestion, feature generation, and inference logic creates verification boundaries where data distributions can be validated before reaching model weights. Because boundary erosion remains silent during initial prototyping, establishing explicit schema contracts and invariant checks provides an explicit boundary for detecting and managing statistical coupling.

14.3.2 Correction cascades

When boundary erosion goes unaddressed, engineering interventions frequently trigger a **correction cascade**: fixing a specific model defect with an auxiliary patch introduces secondary errors elsewhere, requiring further compensatory fixes that destabilize the entire pipeline. In ML systems, these cascades are severe because changes propagate through statistical dependencies rather than deterministic code branches. Retraining a model to correct a specific false-positive cluster may degrade accuracy across previously stable cohorts. Adding an auxiliary model to override edge-case predictions introduces uncalibrated residuals that distort downstream consumers. Repeated

dependent patches can demand coordinated revalidation, with maintenance costs that may exceed those of revising the base model.

Figure 14.4 traces this failure propagation along a four-stage dependency chain (Models A through D), where each downstream component compensates for the residual errors of its predecessor. When an upstream patch alters Model A, dashed propagation arcs identify downstream corrections that require revalidation and may need retraining. The red arcs highlight how a local upstream fix propagates through subsequent stages, requiring coordinated checks of which learned corrections remain valid.

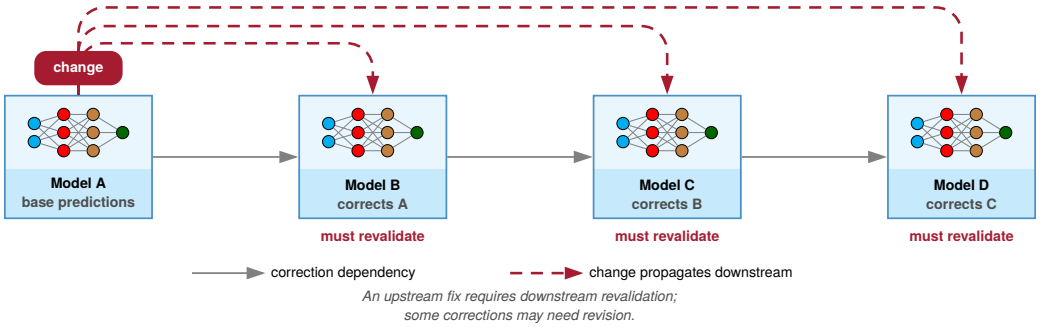


Figure 14.4: Correction Cascades: Each model is trained to correct the errors of the model before it, forming a chain of dependencies. Changing an upstream model can invalidate downstream corrections, turning one local fix into revalidation across the chain and potentially several retraining cycles.

This propagation turns localized model repairs into lifecycle-wide engineering dependencies. Dependent correction models are one source; fine-tuning or reusing an established model alone does not create a correction cascade. Reuse does inherit representations and assumptions that require evaluation on the new task. Over time, those inherited properties become rigid architectural constraints that complicate future updates.

For example, fine-tuning an existing customer churn model for a new product line inherits the base model's feature encodings and calibration thresholds. If the new product exhibits different user engagement patterns, patching prediction errors with auxiliary post-processing rules creates a fragile multi-stage pipeline. When accuracy degrades, engineering teams discover that the root failure lies several layers upstream in the original labeling conventions.

Mitigating correction cascades requires balancing component reuse against redesign. While fine-tuning reduces initial compute and data collection costs, training a replacement base model can remove dependencies on a chain of auxiliary corrections, while data and labeling assumptions still require validation. The architectural decision depends on task similarity, covariate alignment, and the ongoing maintenance cost of chained models. Where dependent models are unavoidable, systems must decouple them through version-pinned model artifacts and frozen intermediate representations.

14.3.3 Interface and dependency challenges

Boundary erosion and correction cascades both stem from interface dependencies that bypass formal software contracts. Traditional software dependencies are declared through import statements, package managers, and RPC schemas, allowing static analysis tools to map dependency graphs. In machine learning systems, dependencies hide inside data streams. When model A's output is logged to a data warehouse and later ingested as a training feature for model B, the coupling exists solely within the data pipeline, invisible to static code analysis.

Hidden coupling frequently manifests as **undeclared consumers**, where downstream services consume model predictions without formal registration or service-level agreements. When the upstream model is retrained or its output distribution shifts, these untracked consumers fail silently. For example, a credit risk score consumed by an automated marketing system can alter applicant pools, feeding shifted distributions back into future training runs and inducing runaway selection bias. This vulnerability is compounded by **data dependency debt**, which accumulates when ML pipelines ingest unversioned tables, unstable upstream features, or signals with volatile distributions.

Eliminating interface debt requires infrastructure-enforced boundaries: strict authentication and registration for model prediction endpoints, machine-readable schema contracts for feature stores, and automated data lineage tracking that registers consumers before granting read access.

14.3.4 System evolution challenges

Even well-designed ML systems with documented interfaces encounter evolving dependencies; learned feedback makes their statistical consequences especially important.

The most insidious of these dynamics is the feedback loop, where a deployed model influences the data distribution that trains its successors. Recommendation systems illustrate this pattern directly: suggested content dictates user engagement, clicks generate subsequent training sets, and the model reinforces its own historical selections while starving unselected content of exposure. In production, this failure mode often surfaces as a widening performance disparity across demographic cohorts: underrepresented groups receive suboptimal predictions, generating skewed engagement logs that further degrade cohort representation in subsequent retraining runs, the self-reinforcing loop that the bias feedback model (principle 13) describes. Detecting feedback loops requires cohort-stratified monitoring that tracks error distributions across user segments rather than relying on aggregate validation metrics.

↪ A worked example that fits the bias feedback factor from two audits and treats fairness as a stability property of the retraining loop is developed in section 15.3.4.

Over repeated deployment cycles, workflows accumulate **pipeline and configuration debt**, evolving into jungles of ad hoc glue scripts and divergent hyperparameter files. When pipelines lack modular interfaces, teams copy existing scripts rather than refactor brittle data loaders, producing redundant pipelines that process the same raw inputs with subtle inconsistencies. Compounding this, rapid prototyping frequently embeds business logic directly into feature preparation scripts. Controlling pipeline evolution requires workflow orchestration engines with immutable directed acyclic graph (DAG) definitions, strict parameter validation, and configuration versioning managed under the same source control as model code.

14.3.5 Code and architecture debt

Beyond statistical coupling and data flows, ML systems accumulate code-level technical debt that paralyzes infrastructure refactoring (Sculley et al. 2015).

At the implementation tier, **glue code** frequently dominates production repositories. Massive integration wrappers are written to shuttle tensors and records between general-purpose ML frameworks, specialized feature pipelines, and low-latency serving engines. Sculley et al. (2015) observe that a mature system can end up at most 5 percent machine learning code and at least 95 percent glue code. The proportion varies by system, but the asymmetry explains why integration code, not the model, dominates the maintenance surface. The resulting tight coupling to framework-specific APIs creates severe maintenance liabilities: minor upstream framework updates force extensive rewrites of the surrounding integration scaffolding. Isolating third-party libraries behind stable internal interfaces prevents external API churn from propagating throughout the serving stack.

Rapid experimentation further burdens codebases through **dead experimental codepaths**. Research iterations leave behind legacy branches, alternative feature transforms, and abandoned scoring logic. Unlike traditional dead code that static analyzers flag and compilers prune, experimental ML codepaths often remain executable at runtime because they are guarded by dynamic configuration flags. These dormant branches expand the test matrix, complicate regression verification, and obscure the precise execution graph operating in production. Enforcing deprecation schedules and pruning retired feature flags prevents experimental residue from destabilizing the serving runtime.

Underlying both patterns is **abstraction debt**: traditional software engineering relies on mature abstractions such as objects, interfaces, and modules, whereas ML engineering lacks universal abstractions for concepts such as a feature, a training dataset slice, or a model lifecycle state. Without standardized primitives, teams frequently reinvent bespoke abstractions or abandon encapsulation altogether. Infrastructure components such as feature stores, model registries, and standardized prediction services mitigate this debt by providing uniform abstractions for feature transformation, artifact lineage, and inference routing.

Beyond these structural patterns, Sculley et al. (2015) identify system smells that signal mounting debt: the *Plain-Old-Data Type Smell* (passing unvalidated raw floats and string arrays rather than domain types with invariant checks), the *Multiple-Language Smell* (stitching Python, SQL, C++, and

shell scripts across the training-serving boundary), and the *Prototype Smell* (promoting throwaway research notebooks directly into production serving without re-engineering). Treating debt paydown as a routine operational discipline rather than an emergency intervention prevents these code smells from degrading cluster efficiency.

14.3.6 Technical debt in practice

These debt patterns are not theoretical edge cases; they represent the dominant failure modes of production ML deployments at scale. When statistical dependencies and uncontracted data flows interact with real-world operating environments, unmanaged technical debt translates directly into systemic outages and commercial failure.

14.3.6.1 Production debt patterns

The first pair exposes coupling through model behavior. YouTube's recommendation system illustrates how large recommenders learn from noisy user-behavior signals, making ranking objectives, equal per-user weighting, and live A/B evaluation part of the system design rather than offline evaluation details (Covington et al. 2016). Because the recommender's own rankings shape what users watch, today's recommendations become tomorrow's training signal, a feedback loop that evaluation on fixed historical logs alone may miss. Zillow's iBuying venture exposes the decision-coupling version of a cascade, in which a forecast drives purchases that cannot be undone when the forecast is later corrected (war story 14.1).

War Story 14.1: The Zillow Offers wind-down (2021)

Context: Zillow, a real-estate marketplace, ran Zillow Offers, an iBuying business that bought homes directly on the strength of forecasts of home prices and resale economics (Zillow Group 2021).

Mechanism: The valuation forecast did not remain a prediction. It set purchase prices, purchases became inventory, and inventory carried the forecast's error onto the balance sheet. Zillow reported that it had bought homes at prices above its revised estimates of future selling prices, while resale-capacity constraints delayed some closings (Zillow Group 2021). Revising the forecast corrected the model's view of the market but could not reverse commitments made on the earlier view, so the correction propagated downstream as losses rather than as a repair.

Impact: In November 2021 Zillow announced that it would wind down Zillow Offers, reported a $304 million third-quarter inventory write-down, and cut about 25 percent of its workforce (Zillow Group 2021).

Response: A safer design caps the inventory exposure any one forecast can create, keeps manual override between prediction and purchase, and places circuit breakers that halt purchasing when forecast uncertainty exceeds the accepted risk budget.

Systems lesson: The case does not show a chain of residual-correction models, and retraining alone would not have removed the business risk. It shows the hazard that correction cascades share: a fix applied at one stage cannot undo the state that downstream stages already built on its earlier output. When predictions trigger irreversible actions, the dependency chain runs through the physical world, and the operational controls must sit on that chain.

The second pair exposes coupling through ownership and configuration. Safety-critical driving automation illustrates a related boundary failure: when automated-control outputs, driver expectations, and subsystem responsibilities are not specified clearly enough, operational failures can cross component boundaries rather than staying local (National Transportation Safety Board 2017). Facebook's News Feed iterations show the configuration version of the same governance problem. Rapid experimentation and ranking changes require traceable settings and explicit objectives; otherwise behavioral changes become hard to audit after deployment (Engineering 2016; Mosseri 2018).

↩ Zillow Offers as an example of predictions that drive irreversible actions is established in section 2.11.

Each case illustrates a different pattern that emerges when probabilistic decision systems are deployed without infrastructure that makes coupling visible: feedback loops at YouTube, irreversible

forecast-driven purchases at Zillow, unclear responsibility boundaries in driving automation, and configuration sprawl at Facebook.

The remedies are not isolated point solutions but structural boundaries. Mitigating technical debt requires end-to-end MLOps infrastructure: feature registries that enforce runtime schemas, metadata stores that track model-data lineage, automated CI/CD pipelines that validate statistical properties, and observability platforms that detect distribution shift before silent errors propagate downstream.

14.3.7 ML test score

A release-readiness assessment requires an explicit catalog of the structural debt patterns that make a model unsafe to serve even when offline validation metrics appear acceptable. Machine learning systems fail in production not only from algorithmic degradation, but from silent interface decay, untracked consumers, and unversioned pipeline dependencies. Table 14.5 categorizes these primary debt patterns alongside their root causes, failure symptoms, and mitigation strategies, establishing the failure modes that a production readiness rubric must detect and prevent.

Table 14.5: Technical Debt Patterns: Machine learning systems accumulate distinct forms of technical debt from data dependencies, model interactions, and evolving operational contexts. Primary debt patterns, their causes, symptoms, and recommended mitigation strategies guide practitioners in recognizing and addressing these challenges systematically.

Debt Pattern	Primary Cause	Key Symptoms	Mitigation Strategies
Boundary Erosion	Tightly coupled components, unclear interfaces	Changes cascade unpredictably, CACE principle violations	Enforce modular interfaces, design for encapsulation
Correction Cascades	Sequential model dependencies, inherited assumptions	Upstream fixes break downstream systems, escalating revisions	Careful reuse vs. redesign trade-offs, clear versioning
Undeclared Consumers	Informal output sharing, untracked dependencies	Silent breakage from model updates, hidden feedback loops	Strict access controls, formal interface contracts, usage monitoring
Data Dependency Debt	Unstable or underutilized data inputs	Model failures from data changes, brittle feature pipelines	Data versioning, lineage tracking, leave-one-out analysis
Feedback Loops	Model outputs influence future training data	Self-reinforcing behavior, hidden performance degradation	Cohort-based monitoring, canary deployments, architectural isolation
Pipeline Debt	Ad hoc workflows, lack of standard interfaces	Fragile execution, duplication, maintenance burden	Modular design, workflow orchestration tools, shared libraries
Configuration Debt	Fragmented settings, poor versioning	Irreproducible results, silent failures, tuning opacity	Version control, validation, structured formats, automation
Prototype Debt	Rapid prototyping shortcuts, tight code-logic coupling	Inflexibility as systems scale, difficult team collaboration	Flexible foundations, intentional debt tracking, planned refactoring

Qualitative awareness of failure modes does not prevent operational outages; deployment gates require a formal technical debt assessment rubric that converts subjective readiness debates into quantifiable engineering constraints. The ML Test Score (Breck et al. 2017) structures this evaluation across four operational categories: data tests, model tests, infrastructure tests, and monitoring tests. The rubric specifies 28 tests, seven per category. A documented manual test whose results are distributed receives 0.5 points; an automated test run repeatedly receives 1 point, and an unperformed test receives 0. Rather than averaging these evaluations into a single compensatory aggregate, the composite score follows the weakest-link principle:

$$\text{ML Test Score} = \min(\text{Score}_{\text{data}}, \text{Score}_{\text{model}}, \text{Score}_{\text{infra}}, \text{Score}_{\text{mon}})$$

where each category score $\text{Score}_{\text{cat}} \in [0, 7]$. A deployment with exhaustive offline model validation ($\text{Score}_{\text{model}} = 7$) but absent production telemetry ($\text{Score}_{\text{mon}} = 0$) receives an overall score of 0. This min-operator reflects the reality of machine learning systems: an unmonitored serving path can deliver corrupted predictions despite strong offline benchmark accuracy.

Three of the four suites partition production risk along the D·A·M axes, and the fourth watches all three. The data suite (D) verifies that feature distributions conform to declared schemas, privacy boundaries remain unviolated, and each feature justifies its inference latency and memory footprint. The model suite (A) confirms that architectures and hyperparameter configurations are version-controlled, staleness bounds are enforced, and offline benchmark improvements translate into live metric gains. The infrastructure suite (M) tests training reproducibility under documented execution conditions, serving-path parity between training and inference pipelines, and automated rollback mechanisms. The monitoring suite is the feedback layer across all three axes; it verifies that schema changes, invariant violations, training-serving skew, and stale weights generate actionable alerts that help limit downstream impact. Table 14.6 summarizes representative tests across each category:

Table 14.6: ML Test Score Checklist: Representative production-readiness tests from the ML Test Score rubric. The original rubric contains 28 tests grouped into four sections of seven tests each; section scores expose whether production risk comes from data validation, model validation, infrastructure, or monitoring rather than hiding the weakness in a single total. Based on Breck et al. (2017).

Category	Test	Implementation Example
Data Tests	Feature expectations are captured in schema	Great Expectations, TFX Data Validation
	All features are beneficial (no unused features)	Feature importance analysis, ablation studies
	No feature's cost exceeds its benefit	Latency/accuracy trade-off analysis
	Data pipeline has appropriate privacy controls	PII detection, access logging
Model Tests	Model spec is reviewed and checked into version control	Git-tracked model configs
	Offline and online metrics are correlated	A/B test validation of offline improvements
	All hyperparameters are tuned	Automated HPO with tracked results
	Model staleness is measured and bounded	Performance decay monitoring
Infrastructure Tests	Training is reproducible	Fixed seeds, versioned data, locked dependencies
	Model can be rolled back to previous version	Model registry with versioning
	Training and serving code paths are tested for consistency	Feature store integration tests
	Model quality is validated before serving	Automated validation gates in CI/CD
Monitoring Tests	Dependency changes result in alerts	Data schema monitoring
	Data invariants hold in training and serving	Distribution comparison tests
	Training and serving features are not skewed	Training-serving skew detection
	Model staleness is monitored	Alerts and evidence-based retraining review

Because the composite score is bounded by the minimum category, reliability audits must prioritize the bottleneck subsystem limiting the system score. Expanding a model test suite from 5 to 6 points while monitoring remains at 1 leaves the composite score unchanged, but can still improve actual coverage. The score highlights the weakest category without quantifying every reliability gain. Regular audits against this rubric reveal whether reliability risks stem from unvalidated input pipelines, offline-online distribution divergence, or uninstrumented serving paths, and so direct refactoring effort toward the debt that most threatens production. Each low category score also names the infrastructure that pays the debt down. A weak data score calls for schema contracts and shared feature definitions, a weak infrastructure score for versioned and reproducible pipelines, and a weak monitoring score for production telemetry.

14.4 Development Infrastructure

Development infrastructure turns the debt patterns diagnosed in section 14.3 into enforcement points. A feature schema that drifts upstream cannot be repaired by a dashboard alone; it needs a shared contract, a versioned artifact, and a deployment path that rejects incompatible changes before they reach production. Table 14.7 maps each component directly to a foundational principle (section 14.2.1) and a specific failure mode. CI/CD pipelines also carry versioned artifacts, but the table assigns them cost-aware automation because the pipeline is where the retraining and promotion gates of equation 14.3 actually execute, automating only the changes that pass them.

Table 14.7: MLOps Infrastructure as Debt Remediation: Each infrastructure component responds to a class of technical debt observed in production ML systems. Feature stores can reduce training-serving skew by reusing feature definitions and values; versioning systems support reproducibility; CI/CD pipelines automate rollout controls; monitoring systems can surface silent degradation before users do.

Infrastructure Component	Principle Implemented	Debt Pattern Addressed
Feature stores	Consistency	Data dependency debt, training-serving skew
Versioning systems	Reproducibility	Configuration debt, correction cascades
CI/CD pipelines	Cost-aware automation	Pipeline debt, boundary erosion
Monitoring systems	Observable degradation	Feedback loops, silent failures

These operational components span the boundary between high-level algorithmic abstractions and low-level machine execution, stratifying responsibilities across five tiers: ML models, frameworks, orchestration, infrastructure, and hardware. While models and frameworks express tensor computations mathematically, they remain unaware of cluster state, hardware faults, or input distribution drift. Model orchestration and infrastructure bridge this gap: orchestration manages data lineage, feature computation, and model serving, while infrastructure schedules accelerator resources, manages storage tiers, and collects runtime telemetry to enforce the debt-remediation contracts identified in table 14.7.

MLOps occupies the middle of that five-tier stack (figure 14.5). Model orchestration, which runs from data management to serving, and operational infrastructure, which runs from job scheduling to monitoring, together bridge mathematical frameworks and physical accelerator clusters. The tiers say *where* a component sits in the implementation stack, while the four layers of table 14.1 say *who* owns an operational responsibility; the rest of the chapter reasons with the layers and the three interfaces, not the tiers.

Inside the orchestration and infrastructure tiers, four architectural mechanisms execute these operational responsibilities. First, a feature storage and retrieval layer serves a unified schema across two disparate access patterns: high-throughput analytical scans over immutable columnar files for offline training, and low-latency point lookups against key-value stores for online serving (section 14.4.1.2). Second, an artifact and lineage ledger records content-addressed hashes for dataset snapshots, code commits, container images, and model weights, ensuring any live prediction can be audited back to its exact training inputs (section 14.4.1.3). Third, workflow orchestration sequences pipeline stages—from ingestion and validation through training and evaluation—as declared dependency graphs with retry semantics and automated triggers (section 14.4.2.1). Fourth, a compute runtime schedules accelerator hardware, host memory, and processor cores across training jobs and serving endpoints under strict resource isolation (section 14.4.2.2). Specific frameworks implement pieces of this decomposition, but the underlying mechanisms and interface boundaries outlast any particular tool.

14.4.1 Data infrastructure and preparation

In an operational machine learning system, data is not a static compilation input; it is an active physical dependency that flows continuously from external ingestion to accelerator execution. From initial ingestion to online inference, data infrastructure must guarantee quality, mathematical consistency, and end-to-end traceability across both offline training and real-time serving. Meeting these operational requirements demands systems that formalize feature transformation and artifact versioning across the entire machine learning lifecycle.

14.4.1.1 Data management

Machine learning technical debt stems largely from unmanaged data boundaries: unversioned datasets erode system boundaries, inconsistent feature computation triggers correction cascades, and undocumented data dependencies create hidden downstream consumers. In operations, the collection, preprocessing, and feature transformation built by data engineering become formalized operational contracts. Where data engineering optimizes single-pipeline correctness and ingestion throughput, MLOps data management governs cross-pipeline consistency, ensuring that offline training and online serving compute mathematically identical features. Data management extends beyond initial preparation to govern the continuous lifecycle of data artifacts.

↩ The collection, preprocessing, and feature-transformation pipelines that MLOps formalizes are established in chapter 4.

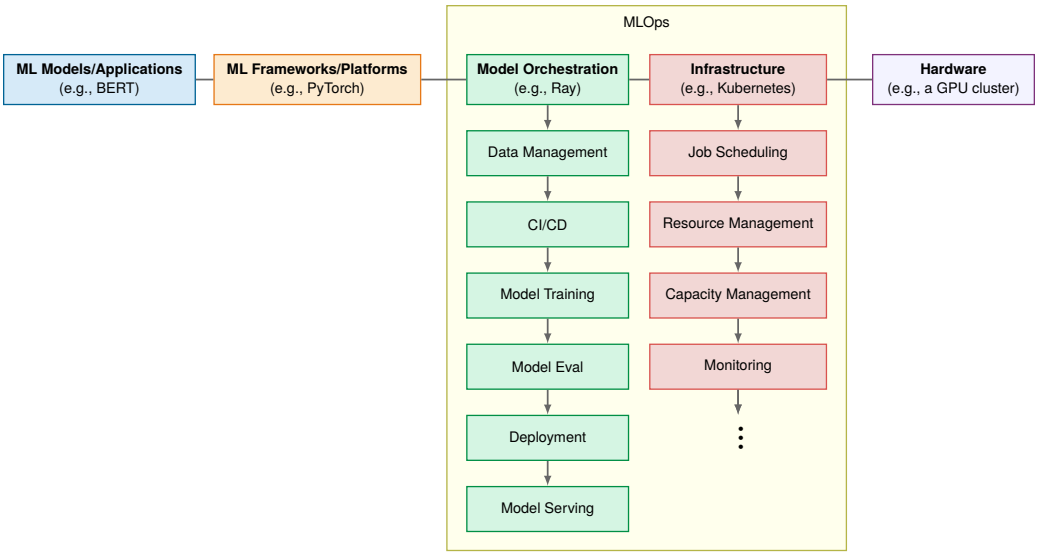


Figure 14.5: MLOps Stack Layers: Five columns organize the ML system stack: ML Models/Applications, ML Frameworks/Platforms, Model Orchestration, Infrastructure, and Hardware. MLOps spans orchestration tasks from data management through model serving and infrastructure tasks from job scheduling through monitoring, supporting automation, reproducibility, and scalable deployment.

Three principles organize the infrastructure that addresses these root causes: consistency, freshness, and quality. Each principle motivates specific tooling rather than the reverse.

The first principle, consistency, begins with reproducibility through versioned data. Every artifact influencing model behavior, from raw ingested records to engineered feature tables, must remain immutable and traceable. Without cryptographic versioning, teams cannot determine which dataset snapshot produced a given model checkpoint, making root-cause diagnosis and rollback impossible. Production implementations combine code repositories, content-addressed dataset versioning, and durable object storage. DVC (Data Version Control) (Iterative 2024), Git (Torvalds and Hamano 2024), Amazon S3 (Amazon Web Services 2024b), and Google Cloud Storage (Google Cloud 2024b) represent common tools for this pattern, but the systems invariant is that raw and processed artifacts must remain addressable by version (section 14.4.1.3). At the feature level, consistency demands unified transformation logic across offline training and online serving to prevent training-serving skew, an invariant formalized in the dedicated feature store architectures examined next (section 14.4.1.2).

Consistency alone is insufficient if the underlying data is stale. Data freshness ensures that models train and serve on current data rather than outdated snapshots. Automated data pipelines maintain freshness by continuously transforming raw data into analysis-ready formats through structured stages: ingestion, schema validation, deduplication, transformation, and loading. Workflow orchestrators such as Apache Airflow (Apache Software Foundation 2024) and Prefect (Prefect Technologies, Inc. 2024), together with the transformation framework dbt (dbt Labs 2024), make those stages explicit, schedulable, and reviewable as code. Once the pipeline is managed this way, data flows can evolve with model requirements without losing versioning, modularity, or CI/CD integration.

The third principle, data quality, governs whether the inputs reaching the model remain accurate, complete, and consistently labeled. In supervised learning systems, label errors and ambiguity can limit what training and evaluation establish, even with a larger model. Annotation platforms such as Label Studio (HumanSignal 2024) support team-based annotation with integrated audit trails and version histories, providing the verification needed when labeling ontologies evolve or require correction across multiple training cycles.

In an industrial predictive maintenance system, these three principles operate in concert. A continuous telemetry stream is ingested and joined with historical maintenance logs through a scheduled pipeline managed in Airflow (freshness). The resulting features—rolling window averages and vibration frequency aggregates—are stored in a feature store for both offline retraining and

low-latency online inference (consistency). Schema validation, sensor-range checks, missingness tests, and label audits reject corrupted telemetry records before they enter the training set (quality), while dataset versioning and model-registry integration maintain traceability from raw sensor logs to deployed model predictions. Structured around these three principles, data management establishes the operational foundation for model reproducibility and production reliability.

14.4.1.2 Feature stores

The data dependency debt and training-serving skew analyzed in section 14.3 share a common root cause: inconsistent feature computation across pipeline stages. Without a unified feature layer, feature definitions are typically implemented twice: a data scientist computes `user_session_length` in Python using batch aggregations over historical logs, while a systems engineer reimplements the calculation in Java or C++ to satisfy real-time serving latencies. Subtle differences emerge: one uses wall-clock time between raw events, the other tracks server processing time and filters idle timeouts. The model trains on one definition but serves with another, causing silent accuracy degradation in production. Feature stores[6] address this divergence by providing an abstraction layer between data engineering and machine learning, implementing the consistency imperative through a single source of truth for feature transformations. In pipelines lacking this abstraction, duplicated feature logic across environments introduces training-serving skew, temporal data leakage, and distribution drift.

Feature stores resolve a fundamental systems tension between batch throughput and online latency by maintaining two synchronized storage tiers behind a unified API. Offline training requires high-throughput sequential reads across terabytes or petabytes of historical logs, which is optimized for columnar storage formats (such as Parquet) hosted on distributed object storage or data warehouses. In contrast, online inference requires sub-millisecond point lookups keyed by entity identifier (such as `user_id`), bounded by strict tail-latency service-level objectives (SLOs). An online store satisfies this access pattern using in-memory or distributed key-value stores (such as Redis or Cassandra). While the underlying physical engines differ to match these access patterns, both read from the same versioned transformation logic.

Historical feature retrieval must strictly enforce point-in-time correctness (preventing temporal data leakage or time travel). A join to a current database record may include information unavailable when a historical prediction was made. Features that change over time therefore require *temporal* joins; static attributes do not require the same reconstruction. Suppose an engineer is training a fraud detection model to predict whether a credit card transaction occurring at 14:02 is fraudulent. At 14:05, the bank's automated fraud system flags the card and updates the user's account record in the operational database: `is_account_locked = True`. If the offline training pipeline performs a naive static SQL join between the transactions table and the current users table, the 14:02 transaction will be matched against the 14:05 record, seeing `is_account_locked = True`. The training algorithm quickly discovers that this feature is a near-perfect predictor of fraud. In live production, however, when a transaction arrives at 14:02, the account-lock event has not yet occurred. The model encounters `is_account_locked = False`, its learned decision boundary fails, and production accuracy collapses. This contamination is known as *lookahead bias* (or *temporal target leakage*).

A feature store prevents lookahead bias by executing an as-of join (the temporal join of section 4.8.5): for any historical observation timestamp t_{obs}, it scans an immutable, append-only feature change log and reconstructs the exact state of the world as it existed at $t_{\text{feat}} \leq t_{\text{obs}}$, excluding records unavailable at prediction time. This requires tracking availability as well as event time: an event that occurred earlier but arrived late must not leak into a historical feature. Correctly configured point-in-time joins preserve this boundary while keeping feature definitions unified across offline and online environments.

Beyond consistency, feature stores support versioning, metadata management, and feature reuse across models. A fraud detection model and a credit scoring model frequently depend on overlapping transaction features that can be centrally maintained, validated, and shared. Integration with data pipelines and model registries preserves lineage: when an upstream feature definition is updated or deprecated, downstream models can be automatically identified and scheduled for retraining.

[6] **Feature store**: Uber's Michelangelo platform described a centralized feature store for sharing and serving features across production models (Hermann and Del Balso 2017). At that scale, the consistency guarantee must hold under an online latency budget: what distinguishes a feature store from a shared library of feature code is that the shared feature path also has to serve fresh features fast enough for real-time inference.

↩ The formal definition of training-serving skew as a violation of the consistency imperative is established in section 13.6.1.

↩ The dual-storage architecture and temporal join semantics of feature stores are established in section 4.8.5.

Training-serving skew: Diagnosis and prevention Training-serving skew manifests operationally through feature store inconsistencies and pipeline divergence. Table 14.8 summarizes common causes and their detection methods:

Table 14.8: Training-Serving Skew Categories: Each category requires different detection and prevention strategies. Preprocessing and missing-data skew emerge from divergent transformation code and require parity validation, which shared feature logic, including a feature store where appropriate, can reduce. Time-dependent features need timestamp checks that both paths apply the same cutoff, and library version drift needs environment comparison because identical code can compute different values under different library versions.

Skew Type	Example	Detection Method
Feature preprocessing	Normalization uses different statistics	Same-input replay and value comparison
Missing data handling	Training fills NaN with mean; serving uses 0	Schema validation with explicit null handling
Time-dependent features	Features computed with different time cutoffs	Timestamp validation in feature pipelines
Library version drift	NumPy or Pandas version differences	Environment hash comparison

Training-serving skew case study A production recommendation system illustrates how feature divergence manifests in deployment. One month after rollout, the model exhibits an 8 percent accuracy degradation despite zero alterations to model code or weights. Comparing feature distributions reveals that `user_session_length` has an empirical mean of 45 minutes in the training set but only 12 minutes in online inference. That difference starts the investigation; different user populations could produce it even with identical feature logic. Replaying the same recorded sessions through the versioned training and serving transformations, with the same available events and cutoff, reveals different durations for the same requests. Inspection of those artifacts then identifies feature-definition skew: the offline training pipeline calculated wall-clock duration from the initial event to the final event in a session, whereas the online serving path counted only foreground-active time after filtering idle periods. The model learned decision boundaries against a feature distribution that the serving infrastructure never delivers.

Feature stores resolve this discrepancy by centralizing feature logic into a single versioned definition, extending upstream data pipelines. Listing 14.1 demonstrates the architecture: training materializes historical features with point-in-time correctness, serving performs real-time point lookups, and both pipelines bind to the identical versioned definition hash rather than independent code paths.

Listing 14.1: Feature Store Consistency: Unified retrieval reduces training-serving skew by sharing versioned feature definitions across both pipelines.

```
feature_definitions = registry.load(version="2026-06-01")

training_features = feature_definitions.materialize_historical(
    entities=training_entities,
    at_event_time=True,
    names=["user.session_length", "user.purchase_history"],
)

serving_features = feature_definitions.lookup_online(
    entities=[{"user_id": 12345}],
    names=["user.session_length", "user.purchase_history"],
)

assert training_features.schema == serving_features.schema
assert (
    training_features.definition_hash
    == serving_features.definition_hash
)
```

By centralizing the definition of `session_length`, both offline and online pipelines execute identical transformation logic, though automated integration tests must still verify value parity. Centralized

registries also record metadata and lineage, enabling rapid diagnosis when feature definitions evolve (Hermann and Del Balso 2017; Gojek and Google 2019).

The consistency imperative (section 14.2.1.3) priced this failure: a 1 percent skew error rate on 1,000,000 queries/day already costs $365,000 a year, while the `user_session_length` divergence above produced an 8 percent accuracy degradation. Centralized feature stores amortize the engineering cost of this infrastructure across multiple models and services, translating architectural consistency into measurable operational savings.

Example 14.1: Uber Michelangelo feature store

Scenario: Uber's Michelangelo platform supported production models such as estimated time of arrival and used shared feature infrastructure across offline training and online prediction (Hermann and Del Balso 2017). Separate implementations of the same feature logic would create training-serving skew.

Diagnosis: A shared feature contract reduces the chance that batch and online pipelines interpret a feature differently, while separate offline and online stores still require point-in-time and parity validation.

Systems lesson: Feature stores reduce one major source of training-serving skew by centralizing versioned feature definitions and access paths. They do not eliminate skew: point-in-time correctness, freshness, and online-offline parity still require explicit tests.

Skew detection in CI/CD Continuous integration pipelines must validate feature consistency before deployment. A parity test replays recorded inputs through the versioned training and serving transformations, aligns results by request identifier, and compares each feature using its declared semantics and numerical tolerance. Both paths must receive the same events available at the prediction cutoff; otherwise the test confounds implementation differences with differences in information. For a duration feature, the contract specifies the time unit and whether idle periods count. Categorical values and missingness must agree exactly; floating-point values may use a feature-specific tolerance.

A small counterexample shows why distributions cannot replace this check. Suppose the training transformation returns session durations of 12 and 45 minutes for requests A and B, respectively, while serving returns 45 and 12 minutes for those same requests. The marginal distributions are identical, so a KS comparison finds no difference, but request-keyed comparison detects both mismatches. Conversely, a new population of shorter sessions can shift the marginal distribution while both paths still agree for every replayed request. Point-in-time correctness remains a separate requirement; agreeing paths can both use information unavailable at prediction time.

Listing 14.2 supplies a complementary distribution screen using the two-sample Kolmogorov-Smirnov (KS) test that the data pipeline already uses to watch for drift (notebook 4.6). The KS statistic measures the maximum vertical distance between the empirical cumulative distribution functions (eCDFs) of the training and serving samples, F_{P_0} and F_{P_t}: $\mathcal{D}_{\text{KS}} = \max_{x_{\text{feat}}} |F_{P_0}(x_{\text{feat}}) - F_{P_t}(x_{\text{feat}})|$. Setting the threshold to 0.1 asserts that cumulative probabilities do not diverge by more than 10 percentage points at any value across the feature's support, providing a parameter-free divergence metric for continuous features without requiring empirical binning. In this illustrative release policy, exceeding the threshold pauses promotion for review rather than diagnosing skew. The threshold must be calibrated against sample size and feature sensitivity. Passing the screen establishes only limited marginal divergence at that threshold; it does not establish per-request parity.

Listing 14.2: Feature Distribution Screen: This function compares training and serving feature distributions using the Kolmogorov-Smirnov test, pausing promotion for review under an illustrative threshold policy.

```
def screen_feature_distributions(
    training_features, serving_features, threshold=0.1
):
    """Pause promotion for review if distributions diverge."""
    for feature in training_features.columns:
        ks_stat = ks_2samp(
            training_features[feature], serving_features[feature]
        )
        if ks_stat.statistic > threshold:
            raise DistributionReviewRequired(
                f"{feature}: KS={ks_stat.statistic:.3f}"
            )
```

14.4.1.3 Versioning and lineage

Lineage tracking and versioning implement the reproducibility principle formalized in section 14.2.1. Software artifact provenance already depends on source code, build tools, dependencies, and configuration. In machine learning systems, model behavior is the joint product of code, hyperparameter configuration, and the training dataset snapshot. Changing any single dependency produces an entirely distinct model artifact. MLOps infrastructure manages this multidimensional dependency space by enforcing cryptographic versioning across every pipeline component.

Data versioning addresses dataset mutability by retaining raw data snapshots and intermediate feature tables under content-addressable hashes, so the recorded inputs to historical runs remain retrievable. Hashes identify artifacts but cannot reconstruct missing content or remove execution nondeterminism. Model versioning registers trained weights as immutable artifacts alongside training hyperparameters, evaluation metrics, and environment dependency manifests. Model registries[7] manage lifecycle states (such as staging, production, or retired) for these artifacts and expose lineage graphs tracing the complete dependency chain from raw ingested records to deployed prediction endpoints (MLflow Project 2026; Google Cloud 2024c).

These mechanisms construct the lineage graph of the machine learning system. This graph establishes the audit trail required to diagnose production failures: determining whether an incoming request distribution diverged from training baselines, whether upstream feature calculations changed, or whether the serving environment loaded an unvalidated weight checkpoint. By formalizing provenance as an explicit architectural contract, systems engineering gives incident triage a verifiable record against which to test root-cause hypotheses.

14.4.2 Continuous pipelines and automation

Feature stores and versioning systems address data consistency statically: they ensure that features are computed correctly at a point in time. Automation enables these systems to evolve continuously, synchronizing data preprocessing, training, evaluation, and release into integrated workflows that respond to new data, shifting objectives, and operational constraints (Orr et al. 2021; Google Cloud 2026a).

14.4.2.1 CI/CD pipelines

CI/CD pipelines carry that consistency from the *data* side to the *process* side, ensuring that changes flow through validated stages rather than ad hoc deployments. ML CI/CD pipelines must handle complexity absent from traditional software: data dependencies, model training workflows, and artifact versioning that couple code changes to statistical behavior changes.

A typical ML CI/CD pipeline consists of coordinated stages: checking out updated code, preprocessing input data, training a candidate model, validating performance, packaging the model, and deploying to a serving environment. In some cases, pipelines also include triggers for automatic retraining based on data drift or performance degradation. By codifying these steps, CI/CD pipelines[8] reduce manual intervention, enforce quality checks, and support continuous improvement of deployed systems.

[7] **Model registry**: Prevents "registry bypass," the failure mode where the undocumented production model diverges from the trained artifact through different preprocessing, stale serialization formats, or manual hotfixes applied directly to the serving endpoint. Without a registry enforcing versioned, immutable artifacts with queryable metadata and state, rollbacks require locating the correct weights from an ad-hoc artifact store under incident pressure.

[8] **Idempotency**: This property ensures that retrying a pipeline stage does not create duplicate externally visible effects, conflicting registrations, or repeated deployment actions. Producing identical models from identical inputs is a separate property—determinism and reproducibility—and the `training` stage may violate it due to randomness such as weight initialization. Production systems therefore combine stable run identifiers and atomic publication for idempotency with fixed random seeds, deterministic kernels, fixed library versions, and controlled data ordering for reproducibility.

ML-focused CI/CD layers two tiers of tooling for one reason. A general-purpose CI/CD orchestrator (Jenkins, CircleCI (2024), or GitHub Actions (GitHub, Inc. 2024b)) manages version-control events and execution logic, but the ML layer must additionally version data, gate on model metrics, and trigger retraining. Teams therefore add an ML platform or workflow orchestrator (Kubeflow (Authors 2024), Metaflow (Netflix 2024), or Prefect (Prefect Technologies, Inc. 2024)) that supplies higher-level abstractions for those tasks.

Without this automation, model deployment degrades into a manual, error-prone process: an engineer retrains locally, copies artifacts to a staging server, and promotes to production with no guarantee that the data, code, or hyperparameters match what was validated. The cost of such ad hoc workflows compounds with team size and deployment frequency, producing configuration drift and silent regressions that surface only after the model has served incorrect predictions.

Figure 14.6 traces this continuous training pipeline across seven sequential stages—progressing from dataset ingestion, data validation, and data transformation through training, evaluation, validation, and registry promotion. Driven by dedicated processing, training, and evaluation engines beneath the pipeline, the workflow coordinates with surrounding dataset, model, and metadata repositories whenever an external trigger initiates retraining.

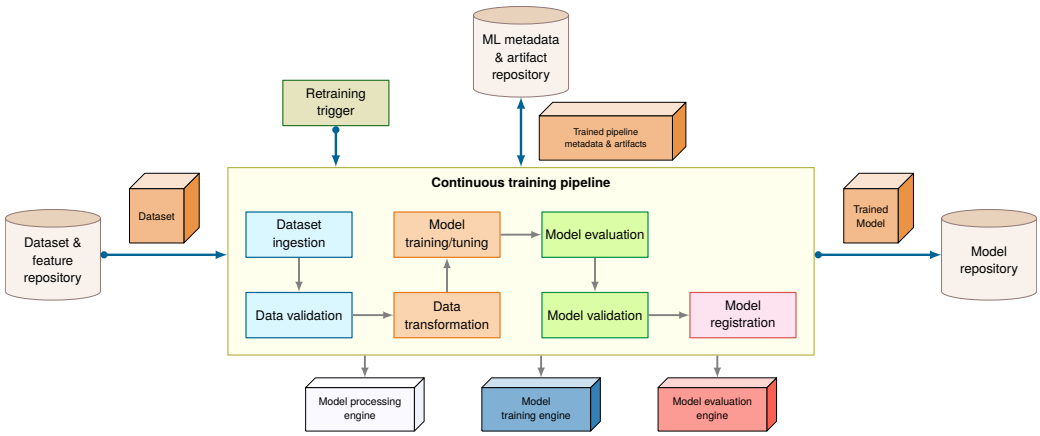


Figure 14.6: ML CI/CD Pipeline: Read the enclosed stages in execution order; the retraining trigger starts a new run, while connected repositories preserve the dataset, model, metadata, and artifacts needed to reproduce and govern it. Adapted from Google Cloud's MLOps continuous delivery and automation pipeline guidance (Google Cloud 2026a).

Consider an image classification model under active development. When a developer commits code changes to a GitHub (GitHub, Inc. 2024a) repository, a Jenkins pipeline triggers execution: it pulls the specified dataset version, performs feature preprocessing, and initiates model training. Experiments are tracked using MLflow (Databricks 2024), which logs loss curves and registers model artifacts. After passing automated offline validation, the model is containerized and deployed to a staging cluster managed by Kubernetes (Cloud Native Computing Foundation 2024a). If the candidate satisfies staging validation, the pipeline executes a canary rollout (detailed in section 14.4.2.3), gradually shifting production query traffic to the new model while monitoring inference latency and error rates. If a performance regression emerges, traffic automatically rolls back to the previous model checkpoint.

CI/CD becomes foundational when systems release models repeatedly or carry meaningful production risk, turning ad hoc experimentation into structured, repeatable deployment. Scaling beyond one pipeline requires reusable components and explicit artifact contracts.

Example 14.2: Google TFX production ML pipelines

Scenario: Google's production experience motivated TensorFlow Extended (TFX), a set of reusable components for production ML pipelines (Baylor et al. 2017). As pipelines multiply, ad hoc handoffs make validation, lineage, and reproducibility difficult.

Diagnosis: Each stage needs explicit validation and metadata so a deployed model can be traced to the data, transformations, code, and evaluation that produced it.

Systems lesson: Directed acyclic graph orchestration alone does not establish reproducibility. Standardized components and ML Metadata (MLMD), a store that records the inputs and outputs of every pipeline component run, preserve lineage through recorded inputs and transformations.

14.4.2.2 Training pipelines

CI/CD pipelines orchestrate the overall workflow, but training itself requires specialized compute infrastructure. Model training builds on the single-node parallel execution primitives that training systems already provide. Within MLOps, training activities become components of a reproducible pipeline supporting continual experimentation and production deployment.

Frameworks such as TensorFlow (Abadi et al. 2016), PyTorch (Paszke et al. 2019), and Keras (Chollet et al. 2024) supply modular components for model architectures and optimization routines, carrying framework-selection criteria into production. Operational discipline determines which exploratory training routines graduate into versioned, tested retraining jobs—and under what execution triggers.

↩ The single-node data- and model-parallel execution that training pipelines schedule is established in section 8.8.1.

Beyond scalability, operational integrity requires strict reproducibility. Training scripts and environment configurations are version-controlled using tools like Git (Torvalds and Hamano 2024) and hosted on platforms such as GitHub (GitHub, Inc. 2024a). While interactive environments like Jupyter (Project Jupyter 2024) notebooks accelerate initial prototyping by coupling code, inline visualizations, and execution state, running them unaltered in production pipelines introduces severe operational liabilities.

↩ The criteria for selecting a framework are established in section 7.9.2.

Notebooks in production Automated CI/CD pipelines assume deterministic and reproducible code execution, but Jupyter notebooks violate this assumption through persistent kernel state. In an interactive session, cells can be executed out of linear order, creating hidden memory dependencies that cannot be reproduced from a clean interpreter restart. A common production failure occurs when development relies on cells executed in sequence 1-3-2; a scheduled batch runner executing top-to-bottom (1-2-3) fails due to undefined variables or altered intermediate tensors.

Testing difficulties compound this state vulnerability. Standard unit-testing harnesses and mocking frameworks do not interface cleanly with interactive notebook documents. Cell-level assertions are rarely parameterized or wired into test runners, leaving notebooks significantly less tested than modular Python code.

Operationalizing notebooks requires specific tooling interventions. Papermill enables parameterization and headless execution of notebooks, injecting runtime configurations and executing cells top-to-bottom as discrete pipeline stages (Papermill Project 2026). Similarly, nbconvert converts validated notebooks into static executable Python scripts for production environments (Project Jupyter 2026), exposing exported code to linters and CI test runners. Teams must still select or refactor exploratory cells and verify clean, ordered execution.

Extracting code from notebooks into tested modules introduces refactoring overhead, but it prevents hidden state assumptions from causing training-serving skew in production, a silent failure whose cost grows with every day it goes undetected, as the silent-failure calculation in the monitoring discussion shows (notebook 14.5).

Modular training logic also allows a pipeline to schedule repeatable experiments within the node's resource budget. Concurrent runs compete for accelerator memory, host preprocessing capacity, and storage bandwidth; enforcing resource limits recorded alongside each run's configuration keeps experiments within the capacity reserved for them. If execution can be interrupted, checkpoint-and-resume preserves completed training work without requiring the entire run to restart.

Automated and reproducible pipelines still require an economically justified execution frequency. Each retraining run consumes compute and validation capacity, and sharing a node with serving adds an opportunity cost. A trigger must therefore establish that an update is worth those resources.

Retraining decision framework Automated training pipelines introduce a critical operational decision: execution cadence. Deciding when to retrain a model requires balancing accuracy degradation

against compute expenditure. Three common strategies exist, each presenting distinct systems trade-offs. Table 14.9 provides illustrative schedules across domains, from daily retraining for rapidly shifting ad click prediction to quarterly updates for stable medical imaging applications:

Table 14.9: Illustrative Retraining Schedules by Domain: These represent starting points; actual cadences depend on observed drift rates and business impact, and organizations typically calibrate them through operational experience.

Domain	Illustrative Schedule	Rationale
Ad click prediction	Daily	User interests shift rapidly
Fraud detection	Weekly	Attack patterns evolve continuously
Demand forecasting	Monthly	Seasonal patterns change slowly
Medical imaging	Quarterly	Disease presentations are stable

Those schedules are starting points, not invariant rules. In a *scheduled retraining* policy, pipeline execution runs on a fixed calendar cadence—daily, weekly, or monthly—regardless of incoming performance telemetry. This strategy is simple to automate and guarantees that freshly accumulated training data is integrated periodically. However, it burns accelerator hours during quiescent periods and reacts sluggishly when distribution shifts strike between scheduled intervals.

A *triggered retraining* policy initiates pipeline execution when monitoring systems detect statistical drift or performance degradation exceeding predefined thresholds. This approach can conserve compute by responding to evidence rather than a fixed schedule, but it relies heavily on low-latency label streams or accurate proxy metrics to prevent delayed responses or false-alarm retraining runs.

A *continuous retraining* policy updates model weights incrementally as labeled records arrive, using streaming micro-batches or online learning algorithms. While this strategy minimizes staleness lag, it dramatically amplifies validation risk: corrupted, unrepresentative, or adversarial data can quickly poison model parameters before automated anomaly detectors or engineering teams can intervene.

The operating choice among these strategies depends on four physical and operational constraints: retraining compute cost, validation pipeline latency, automated rollback capability, and label arrival delay. Re-optimizing a model on multi-accelerator nodes incurs tangible energy and cluster costs; triggered policies require statistically reliable proxy signals when true labels arrive with long delays; and every continuous deployment pipeline demands robust canary and rollback mechanisms. The retraining policy is an engineering optimization governed by operational trade-offs rather than arbitrary heuristics.

The economic and validation constraints become executable in listing 14.3. The decision function schedules a run only when the estimated recovered value exceeds the total cost of a run (compute, validation, deployment, and the expected cost of a bad rollout, the four terms that equation 14.6 formalizes below), and only when fresh validation data are available.

Listing 14.3: Triggered Retraining Decision: The decision combines quality loss, feature drift, prediction shift, and the cost of a run so automatic retraining fires only when the expected benefit exceeds that total cost.

```
quality_loss = baseline_accuracy - current_accuracy
feature_drift = max(population_stability_index(features))
prediction_shift = distribution_distance(
    baseline_predictions, live_predictions
)

benefit = estimate_value_recovered(
    quality_loss=quality_loss,
    feature_drift=feature_drift,
    prediction_shift=prediction_shift,
)
total_cost = (
    compute_cost + validation_cost + deployment_cost + rollout_risk
)

if benefit > total_cost and validation_data_is_fresh():
    schedule_retraining_run()
```

The gate separates detecting drift from authorizing retraining. Drift contributes to estimated benefit, while fresh validation data and positive net value determine whether automation acts. Turning that gate into a cadence requires pricing how fast an unretrained model loses value.

Quantitative retraining economics The decision to retrain a model balances the cost of accuracy decay against retraining expense. When outcome data support a locally exponential fit, the fitted decay rate gives a measurable timescale.[9] Its **half-life**,[10] the time for fitted accuracy to fall to half its initial value, states that timescale in intuitive units, although no production model is left to lose half its accuracy before retraining. The economically optimal interval is set by the price of staleness and the price of a run, and in the scenario worked below it is far shorter than the half-life. The derivation takes three steps: a staleness cost, a retraining cost, and the interval that minimizes their sum per unit time.

The staleness cost function Model accuracy can degrade over time, creating a staleness cost. For economic planning, a team may fit the observable impact over a stable local regime as an exponential decay process. Here γ is a temporal decay rate under the assumption that measured degradation accumulates steadily; it is not determined by distribution divergence alone. The exponential model is a simplification that enables closed-form economic analysis. Let $\text{Accuracy}(t)$ represent accuracy at time t since last training, and Accuracy_0 represent initial accuracy. Equation 14.4 captures this fitted scenario, where the rate γ depends on domain volatility and observed outcomes:

$$\text{Accuracy}(t) = \text{Accuracy}_0 \cdot e^{-\gamma t} \tag{14.4}$$

This temporal model is not a second theory of degradation. It is a time-domain summary of the degradation equation (equation 1.2), which predicts accuracy from divergence, $\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$. If divergence grows at a roughly steady rate over an interval, that equation makes accuracy fall linearly in time at rate λ times the growth rate of $\mathcal{D}(P_t \| P_0)$, while the exponential of equation 14.4 falls at rate $\text{Accuracy}_0 \gamma$ while γt is small. Matching the two slopes shows that γ is an empirical stand-in for how fast the input distribution moves, scaled by the fitted sensitivity λ and divided by Accuracy_0. The two forms also divide the chapter's tools between them. The retraining interval derived here uses the time-domain γ, because a schedule is set in days. The drift statistics of section 14.6.2 (the population stability index, the KS statistic, and KL divergence) estimate the divergence itself and alert on it directly. Steady divergence growth is an assumption rather than a law, so both forms must be checked against observed outcomes.

A longer-horizon fit would decay toward a floor, such as chance-level accuracy, rather than toward zero. The unshifted form in equation 14.4 suffices for the intervals that retraining exists to enforce, the early, small-loss regime ($\gamma t \ll 1$) on which the square-root approximation below also depends.

[9] **System entropy**: The fitted decay rate γ measures the system entropy defined in section 2.11, the quality decay that sets in while code and weights stay unchanged. It can differ substantially across deployments and must be estimated from outcome data. A shorter decay timescale demands more responsive monitoring, but it does not by itself require continuous training; label delay, validation risk, retraining cost, and rollback capacity determine the operational cadence.

[10] **Half-life** (from nuclear physics, where it measures the time for half of a radioactive sample to decay): In ML operations, the metaphor is a convenient approximation, not a universal law. For the exponential fit in equation 14.4, derived below, the half-life is $\ln(2)/\gamma$. Retraining cadence also depends on traffic, value, retraining cost, and risk; seasonal, abrupt, or adversarial change requires a richer drift process.

The cost of staleness accumulates based on the daily query volume Q_{day} and the value impact V of a unit change in accuracy fraction. Integrating the instantaneous accuracy loss $(\text{Accuracy}_0 - \text{Accuracy}(t))$ over the retraining interval T yields equation 14.5:

$$\begin{aligned}\text{Staleness Cost}(T) = \int_0^T Q_{\text{day}} \cdot V \cdot (\text{Accuracy}_0 - \text{Accuracy}(t))\, dt = \\ Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \left(T - \frac{1 - e^{-\gamma T}}{\gamma}\right)\end{aligned} \tag{14.5}$$

The integral accumulates cost over time t from 0 to T, and the closed form follows from substituting equation 14.4 for $\text{Accuracy}(t)$.

The retraining cost function Each retraining incurs fixed costs including compute, validation, and deployment overhead. Equation 14.6 decomposes these:

$$\text{Retraining Cost} = C_{\text{compute}} + C_{\text{validation}} + C_{\text{deployment}} + C_{\text{risk}} \tag{14.6}$$

where C_{compute} is the cost of the training run itself, $C_{\text{validation}}$ is the cost of evaluating the new model before release, $C_{\text{deployment}}$ is the cost of rolling it into production, and C_{risk} is the expected cost of potential regression from the new model. These are the four terms that `total_cost` sums in listing 14.3.

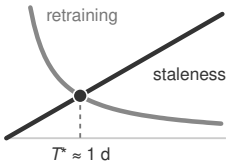
Optimal retraining interval The optimal retraining interval T^* minimizes total cost per unit time, as equation 14.7 shows:

$$T^* = \arg\min_T \frac{\text{Staleness Cost}(T) + \text{Retraining Cost}}{T} \tag{14.7}$$

Under the assumption $\gamma T \ll 1$, a Taylor expansion of the exponential yields a closed-form approximation. Specifically, the second-order Taylor approximation $e^{-\gamma T} \approx 1 - \gamma T + \frac{1}{2}(\gamma T)^2$ reduces the bracketed staleness term $\left(T - \frac{1-e^{-\gamma T}}{\gamma}\right)$ to $\frac{1}{2}\gamma T^2$. Dividing total costs by T yields an average cost rate of $\frac{1}{2} Q_{\text{day}} V \text{Accuracy}_0 \gamma T + \frac{C}{T}$, where C is the Retraining Cost of equation 14.6. Differentiating with respect to T and setting the derivative to zero balances marginal staleness loss against amortized setup expense and yields the square-root economic interval of equation 14.8:

$$T^* \approx \sqrt{\frac{2 \cdot C}{Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}} \tag{14.8}$$

An exact optimum requires minimizing equation 14.7 without the expansion. The interval grows with the cost of a run and shrinks with traffic, value, and decay rate, each under a square root.



The best interval sits where amortized retraining meets accrued staleness.

Napkin Math 14.2: The optimal retraining interval

Problem: How often should the team retrain the model to maximize profit?

Physics: In this fitted scenario, model accuracy $\text{Accuracy}(t)$ decays exponentially at rate γ. Each successful update is assumed to restore initial accuracy with negligible training and rollout delay; traffic and value remain constant.

- Q_{day}: Daily Query Volume (Traffic).
- V: Financial value per query for a unit change in accuracy fraction. With this convention, $V = \$0.50$ means 1 percentage point of accuracy is worth $\$0.005$ per query. The Value per Point of equation 14.3 is the same quantity priced over a decision horizon, $V/100$ per query multiplied by the number of queries in that horizon.
- C: Fixed cost of a retraining run, including compute and operational overhead.

Formula: Equation 14.8 gives the interval T^* that minimizes the sum of staleness losses and training costs per unit time.

Math: Consider an illustrative fraud model ($\text{Accuracy}_0 = 0.95$):

- **Traffic** (Q_{day}): 1,000,000 transactions/day.
- **Utility** (V): \$0.50/query for a unit accuracy change.
- **Retraining Cost** (C): \$5,000.
- **Drift Rate** (γ): 2 percent per day.

$$T^* \approx \sqrt{\frac{2 \times 5{,}000}{1{,}000{,}000 \times 0.50 \times 0.95 \times 0.02}} \approx \mathbf{1} \text{ day}$$

Systems insight: If traffic is high and accuracy is valuable, the team cannot afford to wait. This motivates evaluating automation against label availability, validation requirements, and achievable update delay. If T^* is less than the team's manual deployment time, the proposed cadence exceeds its operational capacity. The team must shorten the update path or reconsider the feasible policy and its assumptions.

With the parameters collected in table 14.10, the approximation places the economic optimum near one day. That puts this fraud system at the fast end of the illustrative schedules in table 14.9, which list weekly retraining for fraud detection. A 2 percent daily decay rate on 1,000,000 transactions a day accumulates staleness cost quickly enough that daily runs pay for themselves; a slower-moving fraud domain or a costlier run would push the same formula toward the table's weekly cadence.

Table 14.10: Retraining Decision Parameters: Example values for a fraud detection system processing 1,000,000 transactions daily.

Parameter	Value	Description
Q_{day}	1,000,000	Transactions per day
V	\$0.50/query	Value per query for a unit accuracy change
Accuracy_0	0.95	Initial accuracy
γ	0.02	Daily decay rate (2% per day)
C	\$5,000	Total retraining expense

Sensitivity analysis Under the square-root approximation, T^* scales with the square root of these parameters. Table 14.11 shows that a fourfold change in retraining cost, query volume, or decay rate moves the approximated interval twofold.

Table 14.11: Retraining Interval Sensitivity: Under the square-root approximation a fourfold change in an input moves the interval only twofold, and the directions differ. More expensive retraining lengthens the interval, while higher query volume or a faster decay rate shortens it.

Change	Effect on T^*
4× retraining cost	2× longer interval
4× query volume	2× shorter interval
4× decay rate	2× shorter interval

Model limitations This closed-form framework provides a first-order approximation for operational planning, but its utility depends on several simplifying assumptions. First, the exponential decay formulation presumes gradual drift at a steady rate γ; sudden covariate shifts or acute data outages require immediate statistical change-point detection rather than smooth decay models. Second, the formulation assumes a constant marginal value V per accuracy point, whereas real-world utility often exhibits nonlinear thresholds or asymmetric penalties. Third, the model treats retraining events as independent resets to initial accuracy with negligible delay, setting aside update failures, time spent training and validating, and transfer learning across cycles. Finally, infrastructure costs are modeled as fixed sums, whereas cloud spot pricing, queue contention, and elastic compute availability

introduce dynamic variations. Despite these boundaries, the derivation grounds retraining cadence in economic trade-offs rather than arbitrary calendar schedules. Parameters improve through empirical calibration against historical telemetry as operational experience accumulates. By making cost-benefit trade-offs explicit and quantifiable, this framework implements cost-aware automation (section 14.2.1), enabling justified infrastructure investments and monitoring thresholds grounded in measurable business impact.

14.4.2.3 Model validation

Training pipelines produce model candidates; model validation determines which candidates merit production deployment. Unlike research evaluation, where a model that beats a benchmark on a static test set may be considered successful, production validation must assess operational readiness under representative conditions and establish the monitoring and rollback controls needed after the distribution changes. This is where MLOps manages the verification gap (equation 1.1; principle 9) rather than closing it. No test suite covers the deployment population, so validation keeps re-estimating performance as that population changes.

The evaluation process begins with performance testing against a holdout test set sampled to represent the current production population and refreshed as that population changes. Core metrics such as accuracy, area under the curve (AUC), precision, recall, and F1 score (Rainio et al. 2024) are computed and tracked longitudinally to detect degradation from data drift (IBM 2024), the shift of the input distribution away from the one the model trained on. Figure 14.7 aligns three synchronized telemetry tiers over time to expose how covariate shift erodes model performance. Across the top panel, discrete request samples transition from predominantly green (online sales) to orange (in-store sales); the figure's "online store" and "offline store" labels name sales channels, not the feature-store tiers of section 14.4.1.2. The middle panel charts this shift as an expanding offline area in the sales_channel feature distribution, directly matching the steep decline of the model accuracy curve in the bottom panel. This temporal alignment motivates comparing input feature distributions with outcome metrics; their co-movement supplies a diagnostic hypothesis rather than establishing the cause by itself.

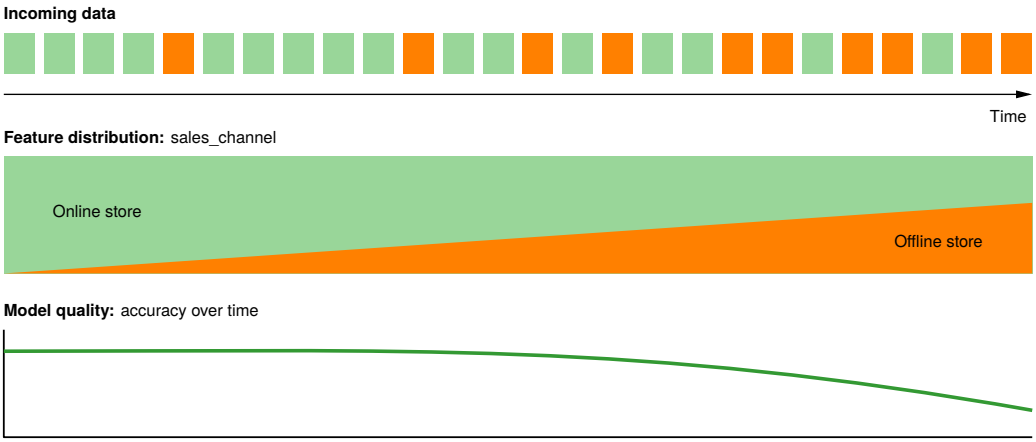


Figure 14.7: Data Drift Impact: In this illustrative scenario, incoming samples shift from predominantly online to increasingly offline sales while model accuracy declines over the same interval. Monitoring inputs and outcomes together helps determine whether the distribution shift is associated with the loss in accuracy.

Beyond static evaluation, production pipelines employ controlled deployment strategies that expose candidate weights to live serving environments while bounding blast radius. One widely adopted method is canary testing (Fowler 2014), the canary stage of the staged rollout in section 3.10.2, in which a new model receives a small fraction of production traffic. During this limited rollout, telemetry pipelines monitor inference latency, system stability, and prediction distributions. For instance, an e-commerce platform deploys a candidate recommendation model to 5 percent of live query traffic, monitoring click-through rates, p99 latency, and prediction drift against the baseline

model. Only after the candidate demonstrates statistical stability without latency regressions is it promoted to serve full production traffic.

Evaluating candidates under identical operational conditions is the prerequisite for a sound promotion decision; a candidate that shows apparent metric gains simply because it was evaluated against a differing traffic slice or temporal window confounds model quality with data variance. Cloud ML platforms support controlled evaluation through request replay, shadow serving, and synthetic load testing. The comparison must also be recorded: experiment trackers such as Weights & Biases (Weights & Biases, Inc. 2024) log the data slice, time window, configuration, and metrics used for each candidate, so a promotion decision can be audited and repeated under the same conditions.

While automated validation gates protect against regressions on primary aggregate metrics, production validation also requires auditing slices and long-tail behavior. Automated test suites often mask severe performance drops on underrepresented subpopulations or safety-critical edge cases. Production architectures combine automated gating with policy reviews, particularly for models deployed in regulated domains. This multi-stage validation discipline bridges offline evaluation with live production monitoring, ensuring that candidate models satisfy accuracy, latency, and safety invariants before receiving unconstrained serving traffic.

14.4.3 Infrastructure integration

Development infrastructure addresses two of the three critical system interfaces. Feature stores and data versioning reduce inconsistency at the data-model interface by centralizing and tracking feature access across training and serving. CI/CD pipelines, model registries, and validation gates stabilize the model-infrastructure interface by automating the transition from trained weights to containerized services with rollback capability.

These controls prepare an artifact for release, but deployment must still establish that the system installed at the serving site matches the validated configuration. Staged traffic exposure and recovery mechanisms limit the consequences of an integration failure. The third interface, production-monitoring, then closes the operational loop by tracking whether predictive quality holds as inputs and operating conditions change.

14.5 Deployment and Serving

A model that passes every offline gate has still not made a single production decision. Between the validated checkpoint and the first user-facing prediction lie packaging, conversion to a serving runtime, staged exposure to live traffic, and a path back if the new version misbehaves, and at each step the artifact users see can drift from the artifact that was evaluated. Deployment transforms a static checkpoint into a versioned, reproducible execution artifact; serving allocates accelerator memory, stages input batches, and schedules compute kernels to satisfy latency SLOs under dynamic request loads. Operations owns the seams between the two: proving that what ships is what was validated, limiting how much traffic a bad version can reach, and keeping rollback fast enough to use.

14.5.1 Infrastructure as code

Three failures show how an unmanaged environment drifts away from the one that was validated. A model works in staging but fails in production because someone manually provisioned a different GPU type. A training job crashes because a colleague's experiment consumed all available memory. An inference service cannot scale because its resource quotas were set through an informal message six months earlier. These failures share a root cause: infrastructure managed through manual processes rather than code.

Scalable, resilient infrastructure is foundational for operationalizing ML systems, and **infrastructure as code (IaC)** is the practice that makes it reliable. IaC treats infrastructure configuration as software (version-controlled, reviewed, tested, and automatically executed) rather than manually configured through graphical interfaces or command-line tools. This approach brings software engineering discipline to resource management: changes are tracked, configurations can be tested before deployment, and environments can be reliably reproduced.

The specific infrastructure tool matters less than the contract it enforces. Terraform (HashiCorp 2014), AWS CloudFormation (Amazon Web Services 2024e), and Ansible (Hatcher 2024) represent common ways to version infrastructure definitions alongside application code. In MLOps settings, that versioned definition is what lets a team reproduce the GPU type, network policy, storage permissions, and scaling limits used by a training or serving environment across AWS (Amazon Web Services 2024c), Google Cloud Platform (Google Cloud 2024a), Microsoft Azure (Microsoft 2024a), or on-premises infrastructure.

Infrastructure management spans the full ML lifecycle. During training, IaC scripts allocate compute instances with GPU or Tensor Processing Unit (TPU) accelerators, configure distributed storage, and deploy container clusters. Because infrastructure definitions are stored as code, they can be audited, reused, and integrated into CI/CD pipelines ensuring consistency across environments.

Containerization provides the same reproducibility boundary for runtime dependencies. Docker (Merkel 2014) packages the model, libraries, and serving code into an isolated unit, while orchestration systems such as Kubernetes (Cloud Native Computing Foundation 2024a) manage those units across clusters. The operational value is not the container name; it is the ability to deploy the same artifact repeatedly while resource allocation, scaling, and health management remain explicit.

To handle changes in workload intensity, including spikes during hyperparameter tuning and surges in prediction traffic, teams rely on cloud elasticity and autoscaling.[11] Cloud platforms support on-demand provisioning and horizontal scaling of infrastructure resources. Autoscaling mechanisms (Amazon Web Services 2024d) automatically adjust compute capacity based on usage metrics, enabling teams to optimize for both performance and cost-efficiency.

[11] **ML autoscaling**: Autoscaling adjusts capacity based on demand signals (Amazon Web Services 2024d), but ML serving adds constraints absent from stateless web services. Autoscaling decisions must account for model loading time (cold-start overhead), GPU memory fragmentation, and batching behavior in addition to CPU utilization. Scaling up too slowly violates latency SLOs; scaling down too aggressively forces repeated cold starts that degrade p99 latency.

Infrastructure in MLOps is not limited to the cloud. Many deployments span on-premises, cloud, and edge environments, depending on latency, privacy, or regulatory constraints. A robust infrastructure management strategy must accommodate this diversity by offering flexible deployment targets and consistent configuration management across environments.

Infrastructure as code addresses how to provision resources reproducibly, but operators must still decide when and how much capacity to allocate. ML workloads break the assumptions of conventional web provisioning. Training jobs demand high-concurrency bursts across distributed accelerator topologies, creating friction between cluster utilization efficiency and time-to-solution. Inference services require sustained baseline allocations with rapid elastic scale-out to absorb request spikes under strict tail-latency SLOs. The utilization targets in section 14.6.1.1 turn these competing demands into measurable provisioning goals.

14.5.2 Validating the serving artifact

The artifact that reaches the serving fleet is rarely the checkpoint that passed evaluation. Before deployment, the model is usually compiled from eager framework code into a static graph for a specific runtime and often lowered in precision. Each conversion buys speed with portability, and each can change what the model computes, so the operational obligation is to prove that the converted artifact still behaves like the model that was validated.

↩ Inference runtimes and the precision choices they support for serving are established in section 13.8.

That proof must target the failure modes that conversion introduces silently. Consider a team that deploys an INT8-quantized model after verifying only throughput improvement: classification accuracy drops on rare but high-value edge cases, and the degradation goes undetected for weeks because aggregate metrics remain within SLO bounds. The first validation layer is numerical equivalence, comparing optimized outputs against the original model on a representative test set with application-specific divergence thresholds. That check is necessary but insufficient, because a static calibration slice is not live traffic: rare inputs, out-of-distribution examples, and subgroup-specific cases can expose quantization artifacts that aggregate test metrics hide, so the comparison must be disaggregated by the same cohorts that production monitoring tracks (section 14.6).

The second layer is operational validation. Memory footprint must be measured at peak runtime utilization, including dynamic allocations during inference, since some optimizations trade increased runtime memory for computational speed. Warm-up behavior varies by runtime: Accelerated Linear Algebra (XLA) may JIT-compile kernels during initial execution, whereas TensorRT selects tactics and builds an engine before deployment; loading that engine can still add startup latency. Runtime version compatibility then closes the loop. Deployment configurations need explicit version pinning, because even a minor runtime upgrade can change kernel selection, and with it both latency and

numerical results, in an artifact whose weights never changed. An unpinned runtime is an untracked input to equation 14.1.

14.5.3 Model deployment

Consider a fraud detection model that achieves 99.2 percent precision in the development environment. An engineer exports the weights, copies them to a production server, and discovers the model predicts every transaction as legitimate: the production server runs a different version of the feature extraction library, producing inputs the model has never seen. This scenario illustrates why deployment is a systems engineering problem rather than a simple file transfer. Packaging, testing, and tracking ML models for reliable production deployment requires treating the model weights, runtime dependencies, and preprocessing configuration as a single deployable unit. Containerizing the model artifact and serving runtime[12] packages user-space dependencies, runtime libraries, and environment variables into an immutable image, enforcing environment parity between development and cluster nodes.

Production deployment requires frameworks that handle model packaging, versioning, and integration with serving infrastructure. Tools like MLflow and model registries manage these deployment artifacts (A. Chen et al. 2020), while serving-specific frameworks handle the runtime optimization and scaling requirements. Before full-scale rollout, teams deploy updated models to staging or QA environments[13] to rigorously test performance.

Three release strategies carry that staging validation into live traffic. **Shadow deployments**[14] duplicate live production traffic asynchronously to a candidate model without returning its inferences to users, enabling latency and output comparison without direct user exposure; accuracy still requires labels or another ground-truth reference. **Canary deployments**[15] are the canary test of section 14.4.2.3 carried through to release. The mechanism is the same small share of user traffic; what changes is that the share keeps ramping toward full traffic as long as no regression appears under live load. **Blue-green deployments**[16] maintain parallel environments for router-level cutover and rapid rollback. Each strategy reduces a different release risk, but none removes the need for tested rollback procedures and production monitoring.

> War Story 14.2: The Knight Capital error (2012)
>
> **Context**: Knight Capital Group was a major market maker in US equities handling over 17 percent of NYSE volume. In August 2012, an operational deployment updated SMARS, the firm's automated order-routing software, on seven of eight production servers but omitted the eighth (U.S. Securities and Exchange Commission 2013).
>
> **Mechanism**: The deployment repurposed a binary feature flag that, on the un-updated eighth server, activated Power Peg, an obsolete order-handling routine that had been left in the code. The stale server entered an un-throttled loop, transmitting parent order requests at $\sim 1{,}400$ orders/sec.
>
> **Impact**: In 45 minutes, the router executed over 4 million unauthorized transactions covering more than 397 million shares, incurring a $460 million loss. The firm survived only on emergency rescue financing raised days later, and lost its independence in an acquisition within months.
>
> **Response**: The incident motivates verifying that every production node runs the intended version before it receives traffic, explicit feature-flag lifecycle controls, and circuit breakers that limit the damage a faulty release can cause.
>
> **Systems lesson**: Automated deployment is a physical control problem where configuration drift across heterogeneous nodes causes system-wide service failure. ML deployments share this exact vulnerability: a model registry version mismatch, un-synchronized feature schemas, or partial canary routing can flood production with corrupted inference requests before telemetry alerts fire.

Knight Capital shows why every node must be verified to run the intended version before traffic reaches it. Staged rollouts add a complementary protection by limiting how much traffic a bad version

[12] **Containerization for ML deployment**: Docker (Merkel 2014) packages code with dependencies into portable units; Kubernetes (Burns et al. 2016) orchestrates those units across clusters. Containerization addresses the Environment$_v$ term in equation 14.1 partially by capturing user-space dependencies as a versioned artifact; host kernels, drivers, and hardware remain external dependencies.

[13] **Staging validation**: ML staging adds probabilistic adequacy checks to conventional functional and operational validation. A model can pass unit tests and still fail in production because the test data does not reflect the deployment distribution, so rollout gates must compare prediction statistics, evaluation slices, and business guardrails against calibrated thresholds rather than rely on unit tests alone.

[14] **Shadow deployment**: Economically justified when its expected reduction in rollout loss exceeds the cost of shadow infrastructure for duplicated inference without serving results.

[15] **Canary deployment**: The ML-specific challenge is that a "failure" is statistical degradation, not a deterministic crash: detecting a small accuracy difference with high confidence can require thousands of inferences, creating a tension between decision speed and statistical power that determines minimum canary duration.

[16] **Blue-green deployment**: Maintains two comparable production environments, "blue" (serving current traffic) and "green" (running the candidate), then switches traffic in a routing change once the green environment passes validation. Because rollback is a traffic flip rather than a staged drain, recovery can be faster than a gradual canary for stateless services. The trade-off is extra infrastructure during the switch, so blue-green wins over canary when brief duplicate capacity is acceptable and per-segment statistical validation is expensive.

↩ The shadow-then-canary staged rollout, and the evidence each stage contributes before full release, are established in section 3.10.2.

sees, but staged deployment also exposes load-dependent behavior. When canary deployments reveal anomalies at elevated traffic shares—such as memory exhaustion, queue spillover, or p99 tail latency inflation appearing at 30 percent traffic that remained undetectable at 5 percent—isolation requires systematic debugging across the stack. Effective diagnosis requires correlating multiple signals: performance benchmarking metrics, data distribution analysis to detect drift, and feature importance shifts that might explain degradation. Teams maintain debug toolkits including A/B test analysis frameworks, feature attribution tools, and data slice analyzers that identify which subpopulations are experiencing degraded performance.

That diagnosis loop must connect directly to the release pipeline. CI/CD integration automates deployment and rollback, but only when rollback is designed as part of the rollout mechanism rather than treated as an emergency script.

14.5.3.1 Rollback strategies and safety mechanisms

17 | **Rollback (from database transaction management)**: This "undo" action for deployments is complicated in ML by model-dependent state (for example, cached embeddings), which may be incompatible between model versions. This mismatch can prolong recovery and contribute to deployment aversion and slower iteration.

Rollback[17] capability is the safety net that enables confident deployment. Without reliable rollback, teams become deployment-averse and slow their iteration velocity. Effective rollback requires planning for three distinct scenarios:

The fastest tier, **immediate rollback**, addresses critical failures detected right after deployment: serving errors, latency spikes, or obvious prediction failures. It requires keeping the previous model version loaded and warm so traffic can switch without cold-start delay. **Rapid rollback** handles performance degradation detected through canary metrics soon after deployment, which requires model registry integration that keeps previous versions deployable with minimal configuration changes. **Delayed rollback** addresses subtle issues detected through business metrics or user feedback after full deployment, where rollback must account for model-dependent data such as personalization state or cached embeddings accumulated during the new model's operation.

Table 14.12 summarizes implementation patterns for each rollback type:

Table 14.12: Rollback Patterns by Scenario: Each rollback type requires different infrastructure support and state handling strategies. Immediate rollback demands always-warm standbys; delayed rollback may require data migration procedures.

Rollback Type	Trigger	Implementation	State Handling
Immediate	Serving errors, crashes	Hot standby with fast routing switch	Verify state compatibility
Rapid	Canary metric degradation	Registry-based redeployment	Clear caches, restart sessions
Delayed	Business metric decline	Full redeployment with migration	Migrate state, replay if needed

Rollback testing Rollback procedures that have never been tested may fail when needed, and the gap is often discovered during an active incident, when cognitive load and time pressure are highest. Failure can stem from configuration dependencies, incompatible state, or unclear ownership. Periodic rollback exercises expose these gaps before they matter. Automated rollback criteria can shorten reaction time, but thresholds must be calibrated to the service and include safeguards against flapping. The restored model must also produce consistent behavior rather than predictions corrupted by stale caches or incompatible feature state. Step-by-step runbooks ensure that the responder need not be the person who designed the deployment.

Stateful vs. stateless rollback ML systems vary in statefulness, which directly governs rollback complexity. For stateless workloads like tabular classification and static regression, rollback requires only swapping model weights or routing traffic back to the prior container, because individual inferences share no execution context. In contrast, stateful models—including sequential recommenders, session-based search rankers, and conversational systems—accumulate transient user state. Reverting such workloads can demand session resets, schema shims for cached context vectors, or explicit state migration; capturing versioned checkpoints at deployment boundaries mitigates this friction, but clean restoration remains an architectural property that requires empirical validation rather than assumption. For instance, two-tower recommendation and search services frequently precalculate and store dense user and item embeddings in low-latency caches; if Model v2 rotates the latent vector space or changes embedding dimensions, those cached vectors become mathematical nonsense to a rolled-back Model v1. A stateful rollback must therefore orchestrate atomic cache

invalidation or dual-version namespace switching alongside weight redeployment to prevent scoring queries against incompatible vector spaces. The most complex failure mode arises in systems with feedback loops: when bad predictions pollute downstream feature logs or clickstream training sets during the degraded window, reverting the serving binary leaves contaminated data in the retraining corpus that must be expunged or quarantined before previous behavior is restored.

14.5.3.2 A/B testing for model validation

A/B testing provides the statistical foundation for deployment decisions by comparing model versions under controlled conditions. Unlike canary deployments (which validate operational stability), A/B tests measure whether a new model improves business outcomes with statistical confidence.

Experiment setup and decision rules A valid A/B test establishes four controls to ensure deployment decisions remain statistically sound:

First, the **randomization unit** specifies whether traffic is split by user ID or individual request. User-level hashing preserves consistency across sessions at the expense of larger sample requirements, whereas request-level splitting maximizes sample throughput but risks exposing a single user to alternating model variants.

Second, the statistical power envelope determines the traffic volume required to resolve meaningful differences. Under a common-variance Normal approximation, let n be the required users per variant, δ_{MDE} the minimum detectable effect, σ the outcome standard deviation (for a conversion metric, each user's outcome is a yes-or-no event, so σ^2 is the baseline conversion rate times one minus that rate), and $z_{\alpha/2}$ and z_{β} the positive standard-Normal critical values for the chosen two-sided confidence and power. The worked scenario uses 95 percent confidence and 80 percent power. Equation 14.9 translates those design targets into required traffic before launch.

$$n = \frac{2(z_{\alpha/2} + z_{\beta})^2 \sigma^2}{\delta_{\text{MDE}}^2} \tag{14.9}$$

For a 2 percent relative lift on a 5 percent baseline conversion rate (5 percent to 5.1 percent) and 80 percent power, the experiment requires roughly 745,644 users per variant. If traffic limits the test to 25,000 users per variant instead, solving equation 14.9 for δ_{MDE} shows that the test can detect only a much larger lift, about 0.5 percentage points absolute.

Third, **guardrail metrics** define operational and behavioral boundaries that must not degrade even if the primary metric improves. A recommendation model improving click-through rate by 10 percent while increasing p99 response latency by 500 ms violates system guardrails and risks cascading timeouts across downstream services.

Fourth, a fixed evaluation horizon requires running the test until the preregistered sample size and minimum duration are reached, capturing cyclical weekly seasonality. Stopping an experiment the moment significance first appears inflates false positive rates through repeated peeking; early stopping requires a prespecified sequential design with calibrated alpha-spending boundaries.

Those controls establish the statistical envelope, but ML systems add failure modes that ordinary web experiments can hide. Conversion events may arrive days after prediction, creating delayed feedback: a recommendation shown Monday can drive a purchase Friday, so the attribution window must be part of the test design. Novelty effects can also inflate early performance as users engage with fresh recommendations, which is why mature experiments include a burn-in period before measurement.

Recommendation and ranking systems add interference effects because showing an item to one user can affect what remains available or salient for another, violating the independence assumption behind standard A/B analysis. Segment heterogeneity creates a second analysis problem: an overall neutral result may hide strong positive effects for one cohort and negative effects for another. These complications do not invalidate A/B testing, but they make guardrails, segment analysis, and preregistered decisions part of the experiment rather than after-the-fact interpretation.

Table 14.13 turns those constraints into a deployment decision:

Table 14.13: A/B Test Decision Matrix: Deployment decisions should consider both primary metrics and guardrails. Improvements that come at the cost of guardrail violations require careful trade-off analysis rather than automatic deployment.

Primary Metric	Guardrails	Decision
Significant improvement	All pass	Ship new model
Significant improvement	Some fail	Investigate trade-offs, may need model iteration
No significant change	All pass	Evidence inconclusive; retain current model unless a prespecified equivalence or noninferiority test supports the change
Significant degradation	N/A	Do not ship; investigate root cause

The decision matrix is reliable only when the evaluation methodology is preregistered before launch. Teams must establish expected effect sizes, the randomization unit, attribution windows, guardrail thresholds, and minimum runtimes prior to observing production traffic.

Interim evaluation demands equal rigor. Variance-reduction techniques such as CUPED (Controlled-experiment Using Pre-Experiment Data) reduce metric variance by subtracting the part of each user's outcome that the same user's pre-experiment behavior already predicts, accelerating decision velocity without sacrificing statistical power. Automated analysis pipelines protect the decision boundary from manual calculation errors, while archiving negative results preserves empirical evidence against repeating unproductive architectures.

An A/B deployment decision is actionable only when the release machinery can promote, hold, or roll back the exact artifact that was evaluated. The model registry (section 14.4.1.3) anchors that transition: it stores immutable version tags, links evaluation metrics to artifact hashes, and governs which checkpoint each serving tier runs (Google Cloud 2024c). This distinguishes a registry from an exploratory model catalog, which indexes candidate architectures for experimentation; a model enters the registry only when it becomes a versioned candidate for deployment. Experiment tracking frameworks such as MLflow[18] (Databricks 2024) and unified distributed runtimes such as Ray[19] record the surrounding execution state, training parameters, and lineage. Together, registry governance and lineage tracking turn experimental validation into controlled release: the tested candidate can be promoted to production, monitored under live traffic, and reversed without losing provenance.

[18] **MLflow**: An open-source MLOps framework that couples artifact storage (S3/GCS) with metadata DBs (PostgreSQL) to record hyperparameter runs, model binary hashes, and deployment stage transitions. The systems trade-off is centralizing lineage tracking vs. incurring database connection bottlenecks during massive parallel hyperparameter sweeps.

[19] **Ray**: A distributed computing framework from UC Berkeley (Moritz et al. 2018) that provides a unified task and actor interface, backed by a distributed scheduler and fault-tolerant store. The broader MLOps lesson is that fragmented infrastructure creates translation points where preprocessing logic, normalization constants, tokenizer versions, or artifact formats can diverge silently. Shared execution abstractions can reduce that fragmentation, but training-serving skew still requires explicit consistency checks across data, features, and serving code.

Checkpoint 14.2: Evidence for a release

- ☐ **Promotion evidence**: A shadow model has acceptable latency and similar output distributions, but outcome labels have not arrived. What has been established, and what quality evidence is still missing?
- ☐ **Recovery state**: A canary changes an embedding dimension. Which artifacts and cached state must a tested rollback restore together?
- ☐ **Converted artifact**: Explain why passing aggregate accuracy before quantization does not replace numerical, slice-level, and runtime checks after conversion.

14.5.4 Inference serving

Request scheduling, batching, and runtime selection belong to the serving system, and section 13.4.1 shows that model inference is only one slice of end-to-end latency, and often not the largest (principle 12). Operations inherits that decomposition as a diagnostic instrument: when p99 regresses, per-stage timings for network, feature retrieval, parsing, inference, and serialization tell the on-call engineer which stage moved, and therefore which fix and which team the incident needs. Without stage-level tracing, every latency regression looks like a model problem, and the reflexive responses of rolling back or quantizing further cannot recover time lost in a feature store or a network hop (principle 8).

The endpoint that exposes the validated model to live traffic contributes two of those stages, parsing and serialization, and its design is an operational choice. Lightweight services often use REST APIs over HTTP/1.1 for real-time scoring, while high-throughput systems adopt binary RPC

protocols such as gRPC over HTTP/2 to eliminate JSON serialization and reduce CPU parsing overhead. Capacity is the second choice. Dedicated accelerator instances keep the model resident and warm, whereas serverless inference[20] scales compute to zero when idle and pays for that saving with a cold-start stage that the latency budget must absorb.

Edge deployment keeps these operational goals (controlled rollout, fast rollback, stage-level telemetry) but changes the constraints under which they must be met, because the devices have limited power, memory, and connectivity.

14.5.5 Edge AI deployment

Consider a smoke detector with an ML model for distinguishing cooking smoke from fire. When this model degrades, an engineer cannot simply SSH into the device, roll back to a previous version, and restart. The device sits on someone's ceiling with intermittent Wi-Fi, a coin-cell battery, and 256 KB of memory. Every operational assumption from cloud MLOps (instant rollback, centralized logging, real-time monitoring) must be reimagined.

Edge AI represents this shift: machine learning inference occurs at or near the data source rather than in centralized cloud infrastructure (Reddi et al. 2019). Workloads governed by strict latency deadlines, bandwidth-constrained uplinks, data-privacy guarantees, or sub-watt power envelopes require executing inference directly on edge silicon. This operational paradigm introduces three interdependent systems constraints: resource limits, hierarchical tiered execution, and fail-safe over-the-air update mechanisms.

Resource constraints dominate edge deployment decisions. Edge devices require aggressive model optimization techniques (quantization, pruning, knowledge distillation) to meet the memory and power envelopes of microcontroller-class deployments (Warden and Situnayake 2020; David et al. 2021). Power budgets span four orders of magnitude, from milliwatts for IoT sensors to tens of watts in automotive systems, demanding power-aware inference scheduling and thermal management. Some hard real-time, safety-critical applications impose deterministic timing targets that require worst-case execution time (WCET) analysis under adverse conditions including thermal throttling and memory contention.

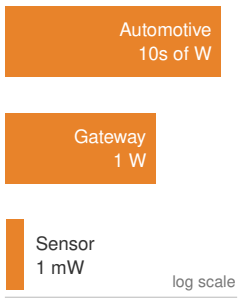
These constraints shape a natural deployment hierarchy across three tiers. Sensor-level processing handles immediate data filtering and feature extraction on microcontroller-class devices consuming 1–100 mW. Edge gateway processing performs intermediate inference on application processors with 1–10 W power budgets. Cloud coordination manages model distribution, aggregated learning, and complex reasoning requiring GPU-class resources. This hierarchy enables system-wide optimization, because computationally expensive operations migrate upward while latency-critical decisions remain local.

Two deployment contexts deserve specific attention. TinyML targets microcontroller-based inference under tight memory and milliwatt-class power constraints, requiring specialized engines such as TensorFlow Lite Micro and CMSIS-NN (David et al. 2021; Lai et al. 2018). Model architectures must be co-designed with hardware constraints, favoring compact operators, quantization, and pruning strategies whose aggressiveness depends on the device and accuracy target. Mobile AI extends edge deployment to smartphones with moderate compute, using neural processing units (NPUs) and GPU compute shaders to meet interactive latency and battery-life constraints through power-aware scheduling.

Updates and monitoring complete the edge operational picture. Over-the-air (OTA) model updates enable maintenance for physically inaccessible systems. OTA pipelines need secure model distribution, artifact verification, and rollback mechanisms; delta or differential updates can reduce transfer volume when devices need not receive a complete model artifact. One fail-safe OTA design on microcontroller and embedded targets uses dual-bank flash memory: the device keeps running from Bank A while it writes the candidate model and runtime to Bank B. After switching to Bank B, a hardware watchdog timer and a post-update self-test supervise the first inferences; if the new model triggers memory faults, hangs, or violates latency budgets, the bootloader automatically reverts execution to Bank A, preventing physically inaccessible devices from being bricked. Update scheduling must account for device connectivity patterns, power availability, and operational criticality.

20 **Serverless ML inference**: The cost-efficiency of this option stems from provisioning compute only upon request and scaling to zero when idle, eliminating the cost of a persistent endpoint. This creates a direct tension with performance targets, as the first request after an idle period incurs a "cold start" latency penalty while the model is loaded into memory. For large models, that delay can be long enough to violate real-time latency budgets unless the service keeps warm capacity or uses a runtime designed for fast loading.

↩ The quantization, pruning, and knowledge-distillation techniques that edge deployment relies on are established in chapter 10.



Edge power budgets span four orders of magnitude.

↩ The memory and power envelopes that separate the edge, mobile, and TinyML tiers are established in section 2.8.

Monitoring requires adaptation to resource-constrained environments: lightweight telemetry systems capture essential metrics (inference latency, power consumption, accuracy indicators) while minimizing overhead. Health monitoring tracks device-level conditions (thermal status, battery levels, connectivity quality) to predict maintenance needs. Edge-cloud coordination patterns enable adaptive offloading between tiers based on current load, network conditions, and latency requirements. Feature caching at edge gateways reduces redundant computation, while federated learning lets edge devices contribute model updates rather than raw training records. Parameter updates can still expose training information. Secure aggregation limits visibility into individual client updates, while differential privacy bounds information leakage under a specified privacy budget; the protections address different threats and require an explicit threat model (section 15.5.2 develops both).

Graceful degradation is the defining operational pattern for edge AI. When thermal limits or battery depletion constrain hardware capacity, edge runtimes must dynamically shed load—downsampling input sample rates, bypassing speculative execution branches, or falling back to shallow heuristic models—within a fallback policy validated for the application's safety and quality requirements.

Getting models into production is only half the operational challenge. A successfully deployed model can still degrade through distribution shift or corrupted upstream features without raising runtime errors. Robust monitoring, incident response, and on-call practices close this loop.

14.6 Production Monitoring

A deployed model has an operational half-life, measured rather than assumed. The retraining economics of section 14.4.2.2 needed a fitted decay rate, and only production monitoring can supply the evidence to fit it. Unlike ordinary software, an ML service also depends on the relationship between current data and learned parameters, so as live inputs change (the system entropy of section 2.11), predictive quality may decline. Monitoring is where the third critical interface, production-monitoring, makes that risk observable. Pairing staged rollout with input and outcome monitoring helps teams investigate failing versions and limit their impact, subject to the time needed to collect evidence and act.

14.6.1 Infrastructure telemetry and hardware utilization

Confirming whether predictive quality has changed, and by how much, requires ground-truth labels that often arrive with substantial delay, so monitoring cannot wait for outcomes alone. In production, serving failures span two fundamentally different physical regimes: immediate hardware execution bottlenecks and gradual distributional drift. Disentangling them requires two distinct telemetry layers: infrastructure metrics that reveal whether the serving system itself is compute-, memory-, or I/O-bound, and distribution and outcome metrics that connect data movement to model performance.

The first layer is infrastructure-level monitoring, which tracks indicators such as CPU and GPU utilization, memory bandwidth, disk consumption, network latency, and service availability. Measuring GPU utilization alone is incomplete; an accelerator reporting high execution time may still be stalled waiting for DRAM or PCIe transfers, while low utilization often signals host-side preprocessing bottlenecks. Correlating kernel execution times with memory bandwidth utilization and input pipeline queue depths distinguishes compute-, memory-, and I/O-bound behavior.

Power-efficiency metrics (such as inferences per joule or FLOP/s/W, depending on workload) add a cost-normalized view that enables mixed-workload scheduling. Thermal monitoring similarly informs operational scheduling, particularly under sustained high load where thermal throttling forces hardware clocks down, introducing tail-latency spikes that violate SLOs. An MLOps telemetry pipeline incorporates thermal headroom metrics to steer traffic across available accelerators before throttling occurs. Time-series engines such as Prometheus[21] (Cloud Native Computing Foundation 2024b), Grafana (Labs 2024), and Elastic (Elastic NV 2024) collect and visualize these operational metrics across the serving fleet.

Collecting these signals at production scale introduces severe network and storage constraints: capturing uncompressed telemetry per request can saturate the very networking fabric serving the

21 **Prometheus**: Prometheus periodically scrapes targets and stores time series for aggregation. Scrape and rule-evaluation intervals jointly affect alert latency and may miss short excursions. Monitoring per-accelerator thermals allows precise workload routing at higher data cost, while server-level aggregates can mask component-level throttling.

model. These overheads force a deliberate trade-off between monitoring granularity and infrastructure operating expense.

Napkin Math 14.3: The economics of observability

Trade-off: "Measure everything" is economically infeasible at scale. Let N_{series} be the expanded series count, f_{sample} the sampling frequency per series, s_{sample} the bytes per sample, and $T_{\text{retention}}$ the retention duration; $C_{\text{ingest/sample}}$ and $C_{\text{storage/byte-time}}$ are provider-specific ingestion and storage prices. Equation 14.10 combines these terms into an operating cost rate:

$$\text{Cost rate} \approx N_{\text{series}} f_{\text{sample}} \left(C_{\text{ingest/sample}} + s_{\text{sample}} T_{\text{retention}} C_{\text{storage/byte-time}} \right) \tag{14.10}$$

The product is an operating cost rate, not a one-time setup cost.

Both data volumes follow from the same two-step arithmetic. At full fidelity, every request's telemetry enters the stream: 1M req/s $\times$ 1 KB per request = 1 GB/s. Retaining one in 60 request traces reduces average ingest to 16.7 MB/s; batching sampled traces every 60 s changes delivery timing, not the average byte rate. Table 14.14 places the two regimes side by side with their cost impact.

Table 14.14: Trace Sampling vs. Observability Cost: Retaining one in 60 request traces yields a 60× data-volume difference at 1M req/s, assuming 1 KB per request. Batch intervals do not cause the reduction.

Sampling	Granularity	Data Volume (1M req/s)	Cost Impact
All traces	Micro-bursts	~1 GB/s	High (Requires dedicated cluster)
1-in-60 traces	Trends	~16.7 MB/s	Lower (Policy-dependent)

Systems insight: Retain 1 percent of successful requests and 100 percent of errors. Monitor aggregate counters every 1 s and high-cardinality sketches every 60 s. Section 14.6.3 turns these choices into a monthly budget for one node.

Alerting mechanisms convert these operational signals into proactive intervention before silent degradation propagates downstream. A sustained drop in model accuracy triggers drift investigation; infrastructure alerts signal memory saturation, thermal throttling, or degraded network throughput. The latency of these alerts governs the window of undetected failure. Alerts must record the model version, affected input subpopulation, and triggering aggregation window so responders can isolate and reproduce the failure state.

Example 14.3: Recommendation monitoring at scale

Scenario: Consider a high-throughput streaming recommendation service whose latency remained within its SLO while recommendation quality declined as user behavior changed.

Diagnosis: Traditional infrastructure metrics (CPU utilization, HTTP error rates) remained green while cohort-level CTR fell. The engagement decline motivates investigation of traffic, feature pipelines, and ranking behavior; it does not establish classification accuracy loss.

Systems lesson: Infrastructure telemetry alone cannot establish recommendation quality. Cohort-level outcome tracking exposes localized engagement changes, while matched comparisons and mature labels are needed to assess predictive-quality changes and their causes.

14.6.1.1 Hardware utilization patterns

Provisioning accelerator resources is only the first half of the problem; using them efficiently requires interpreting hardware metrics correctly rather than taking them at face value. Standard administrative GPU utilization metrics can mislead operators because reported percentages measure time with a kernel executing rather than the fraction of peak arithmetic throughput achieved.

Tools such as `nvidia-smi` report the fraction of the sample interval with at least one kernel executing. Off-chip high-bandwidth memory (HBM) sits several hundred clock cycles away from the compute units, so a kernel with poor memory locality or low arithmetic intensity (gathering non-contiguous embedding vectors, or executing unfused elementwise operations) can leave every resident warp stalled on memory transactions. A high utilization reading therefore remains possible while the arithmetic units spend much of the interval idle. The operations dashboard may display a reassuring green gauge while the underlying silicon is severely starved for data, making these metrics dangerously deceptive if interpreted as compute efficiency.

↩ The analytical model separating compute-bound execution from memory-bandwidth saturation across arithmetic intensities is established in section 11.6.

Distinguishing compute execution from memory or pipeline stalls requires evaluating streaming multiprocessor (SM) execution efficiency alongside memory bandwidth and I/O saturation. Table 14.15 gives illustrative screening patterns. Here GPU utilization is kernel-active time; measured FLOP rate, execution traces, and relevant stall counters are needed to test the proposed bottleneck:

Table 14.15: GPU Utilization Patterns: Illustrative utilization signatures suggest hypotheses, not unique diagnoses. High kernel-active time does not establish compute saturation. Confirm candidates with achieved arithmetic throughput, memory traffic, stall counters, and an execution trace before choosing an optimization.

Pattern	GPU Util	Memory bandwidth util.	Optimization Strategy
Possible compute limit	>85%	<70%	Check achieved FLOP rate; evaluate faster kernels or compute capacity
Possible bandwidth limit	50–85%	>85%	Check traffic and stalls; evaluate quantization or memory access
Possible input limit	<50%	<50%	Trace input starvation before changing prefetch or storage
Batch-starved	Variable (spiky)	Variable	Dynamic batching, request queuing on single server

Each candidate maps to a term of the iron law of ML systems (principle 3): the compute-bound signature to $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$, the memory-bound signature to D_{vol}/BW, and low utilization of both to the data pipeline, so the trace and counters guide which term to investigate when a latency SLO is violated.

Utilization targets by workload Representative utilization targets vary by workload characteristics, reflecting contrasting latency tolerances and cost sensitivities across operational modes. For batch training, an illustrative target is kernel-active time exceeding 80 percent; lower readings motivate checking input starvation, dispatch gaps, and undersized microbatches, while high readings still require throughput analysis. In contrast, **online inference** may target 50–70 percent of measured sustainable service capacity at median load, deliberately provisioning 30–50 percent capacity headroom to absorb unpredicted traffic bursts without violating tail-latency SLOs; running inference hardware near capacity cascades queuing delays into p99 timeouts. For **batch inference**, where queries tolerate substantial queuing and offline scheduling, operators may target above 85 percent kernel-active time while measuring useful throughput and amortized accelerator cost.

These figures serve as diagnostic starting points rather than universal thresholds: kernel-active time and demand relative to service capacity have different denominators. Validate online headroom against measured tail latency, and batch efficiency against useful throughput.

Memory hierarchy effects Model serving performance depends critically on GPU memory hierarchy utilization. Data must flow through multiple memory levels with vastly different bandwidths, as table 14.16 quantifies. L2 is a hardware-managed cache for a small active working set, not a placement tier for selected weights. Full model parameters normally reside in HBM; larger models may offload parameters or state to host memory or storage, with transfer latency often dominating inference. Section D.1 tabulates the current accelerator specifications and HBM bandwidths these serving numbers draw on, so the capacities and interface bandwidths in table 14.16 trace back to documented per-generation figures, while the on-die L2 cache bandwidth is an approximate value that vendors do not publish directly:

Table 14.16: GPU Memory Hierarchy and Bandwidth: Each level trades capacity for speed. L2 caches a small active working set, while full model parameters normally reside in HBM. The host link is a connection rather than a storage level, and it caps GPU access to system RAM and NVMe regardless of their own bandwidth. Host-memory or storage offload can support models that exceed GPU memory, but transfer latency may dominate inference time.

Memory Level	Bandwidth	Typical Contents
L2 Cache (40 MB on A100)	~3 TB/s	Cached working set
HBM2e GPU Memory (80 GB)	~2 TB/s	Model, activations
Host link (PCIe Gen4 x16)	~32 GB/s	Host device copies
System RAM (512 GB)	~200 GB/s	Batched inputs
NVMe SSD	~7 GB/s	Model swap

↪ The full latency hierarchy across the storage spectrum, from on-chip registers to persistent storage, is developed in section D.3.3.

For large language model (LLM) serving, the KV cache often becomes the per-node memory bottleneck; section 13.11.4 sizes it, and PagedAttention manages its allocation (Kwon et al. 2023). Operationally, monitoring KV-cache utilization on each serving node enables capacity planning. Near the memory limit, the scheduler may queue or reject requests, reduce batch size, or shorten the admissible context, each with a different latency or service-quality cost.

Utilization and memory telemetry also feed the number the business sees. Cost per inference, the hourly accelerator bill divided by sustained throughput, is defined and worked out in section 13.11.1; operationally it belongs on the same dashboard as the utilization signatures in table 14.15, because a rising cost per thousand inferences at constant pricing is often the first visible symptom of efficiency erosion (smaller effective batches, a shift in traffic mix, or a regression that stays inside the latency SLO).

14.6.2 Statistical monitoring and drift detection

Infrastructure telemetry shows how the serving hardware behaves; statistical monitoring shows whether the model's predictions remain correct. Unit tests can verify deterministic components but cannot by themselves establish population-level predictive performance, which must be estimated statistically. Monitoring implements observable degradation (section 14.2.1), transforming this theoretical limitation into operational practice. Once monitoring surfaces a symptom—a latency SLO miss, throughput below target, or memory creep—section A.5.1 maps that symptom to its dominant D·A·M term and tells the operator which optimizations will move the binding constraint and which will be wasted on serving infrastructure. Without continuous monitoring and the deeper **observability** it enables (the ability to infer internal state from outputs), teams lack the evidence needed to distinguish a healthy model from an unobserved regression.

Effective monitoring spans both infrastructure health and statistical model behavior. On the infrastructure side, metrics track hardware saturation, memory allocation, and latency distributions. On the model side, teams track evaluation metrics such as accuracy, precision, recall, and confusion matrix shifts (scikit-learn developers 2024) across live prediction streams.

Those quality metrics require joining predictions to outcomes under a fixed evaluation horizon. Consider a conversion model whose prediction log records a request identifier, prediction time, score, and the immutable version actually served. The outcome log records the same identifier, event time, and observation time. For an older request served by the previous version, a purchase event within the horizon supplies a positive label once observed; a completed observation window with no purchase supplies a negative label, provided outcome collection is complete. A recent request served by the candidate with no recorded purchase remains pending until its horizon and the permitted reporting delay have elapsed. The absence of a purchase record while that window is open is unknown, not a negative outcome. Joining by identifier and retaining the served version attributes each completed outcome to the model that made the prediction rather than to whichever model is deployed when the label arrives.

A quality comparison groups predictions by serving period and version, then evaluates cohorts whose outcome windows have matured under the same horizon and reporting policy, with comparable traffic conditions. Each metric and slice must report label coverage, the fraction of eligible predictions with resolved outcomes. An apparent drop caused by comparing the candidate's pending outcomes with the previous version's completed outcomes does not justify retraining; it calls

for completing the window or repairing outcome collection. Even a mature window can contain selective labels if outcomes are observed only for exposed recommendations or reviewed transactions. Coverage and collection-policy checks must accompany uncertainty estimates before a quality regression supports diagnosis and a retraining decision.

Statistical monitoring has to separate two axes of distribution shift,[22] because they differ in what a monitor can observe. Write the input distribution as $p(x)$ and the relationship between inputs and labels as $p(y \mid x)$. The training-time and serving-time distributions that the degradation equation compares, P_0 and P_t, are the same data distribution measured at two points in time. Concept drift occurs when the relationship between features and targets shifts ($p(y \mid x)$ changes), altering the decision boundary even if input features appear normal. Data drift[23] refers to a shift in the input feature distribution ($p(x)$ changes), such as seasonal lighting shifts or changing sensor characteristics in autonomous driving.

Data drift first appeared with the degradation equation in section 1.6. Operating a model requires a sharper version of that definition, one that separates the input shift a monitor can see from the quality decline it may or may not cause:

> **Definition 14.3: Data drift**
>
> ***Data drift*** is a change in the input distribution $p(x)$. Covariate shift is the special case in which $p(x)$ changes while $p(y \mid x)$ remains stable. The broader drift taxonomy from section 4.5.4 also includes concept drift, in which $p(y \mid x)$ changes; both dimensions can occur together.
>
> 1. **Significance**: It violates the identical-distribution assumption and can cause accuracy to erode as the distributional divergence ($\mathcal{D}(P_t \| P_0)$) grows. Under covariate shift and support coverage assumptions, importance weighting or retraining with current labeled data may recover performance.
> 2. **Distinction**: Model decay describes observed quality decline over time; the model code need not change. Data drift is one possible external cause, alongside concept drift and changes in the surrounding pipeline.
> 3. **Common pitfall**: Monitoring model outputs alone may miss input drift or confuse benign output changes with quality loss. Input feature statistics ($\mathcal{D}(P_t \| P_0)$ via PSI, KS tests, or the Wasserstein distance, also called Earth Mover's Distance (EMD), which measures how much probability mass must move, and how far, to turn one distribution into the other) can provide earlier warning than labeled performance because ground-truth feedback is often delayed.

A fundamental constraint in statistical monitoring is the **drift detection delay**: how quickly a statistical test can confirm that predictive quality has degraded. The speed of confirmation is governed by sample arrival rates and statistical power requirements. Low-traffic services may wait days or weeks to accumulate enough labeled outcomes to distinguish genuine performance erosion from stochastic noise. This latency gap is not an engineering deficiency that tooling can eliminate; it is a direct consequence of finite sample rates colliding with hypothesis testing power. Monitoring architectures must therefore keep the two axes apart. Data drift can be estimated from sufficient unlabeled samples, while concept drift generally requires labeled outcomes. Neither statistical estimate is instantaneous.

> **Napkin Math 14.4: The drift detection delay**
>
> **Problem**: A 95 percent-accurate model may fall by 5 percentage-point. The monitoring policy budgets 1,000 labeled examples. A power calculation of the same kind as equation 14.9 sets the minimum such a test needs; the budget is a policy choice above that floor, adding margin for label noise, dependence between outcomes, and per-slice checks. Calculate the collection duration required across high-rate and low-rate arrival scenarios.
>
> **Math**:



Drift detection speed is bounded by the sample rate.

[22] **Label-free vs. label-dependent detection**: A change in $p(x)$ can be estimated from sufficient input samples without labels, although detection is not immediate. A change in $p(y \mid x)$ generally requires labeled outcomes, which in high-stakes domains (medical diagnosis, fraud detection, legal decisions) can take days, weeks, or months. Proxy metrics such as prediction-confidence distributions and output entropy can provide imperfect early warnings that trade false-alarm rate for detection speed.

[23] **Covariate shift**: Importance-weighting corrections for covariate shift assume that the support of the training distribution covers the deployment distribution: every deployment input could have appeared in training, just with different probability. When deployment contains genuinely out-of-distribution inputs (new product categories, new demographics, adversarial inputs), the correction can fail and the model can produce confidently wrong outputs with no warning signal, making support coverage the hidden assumption that determines whether drift correction or full retraining may be required.

1. **Evidence budget**: 1,000 labeled examples.
2. **High-rate case**: At 1 labeled outcome per second, collection takes 1,000 seconds ≈ 16.7 minutes.
3. **Low-rate case**: At 100 labeled outcomes per day, collection takes 10 days.

Systems insight: Detection is bounded by labeled-outcome arrival, which may lag far behind traffic. Low-volume or delayed-label domains may need days or weeks, so high-stakes systems supplement statistical monitoring with proactive model audits.

Detection delay has a direct price. When a silent failure persists until the next review, every day of delay is lost revenue, which is the economic case for automated drift detection.

Napkin Math 14.5: The cost of silent failures

Problem: Is building an automated drift detection system worth the engineering effort?
Scenario: Consider a product recommendation engine generating $50M in annual revenue.
Failure mode: A deployment bug causes training-serving skew, dropping recommendation quality by 5 percent. This degrades conversion rate proportionally.
Assumptions: The incident begins immediately after a scheduled check, remains detectable at the next check, and ends with an effective repair immediately after detection. The calculation therefore compares nearly full review-interval delays, rather than expected detection times for incidents arriving throughout an interval. It excludes label delay and diagnosis or recovery time.
Cost analysis:

1. **Manual ops (monthly review)**:
   - **Detection time**: ~4 weeks (28 days).
   - **Revenue loss**: $50M × 0.05 × 28 days/365 days ≈ $191,780.8.
2. **Automated MLOps (daily checks)**:
   - **Detection time**: 1 day.
   - **Revenue loss**: $50M × 0.05 × 1 day/365 days ≈ $6,849.3.

Systems insight: Under these timing and recovery assumptions, a single silent failure costs $184,931.5 more with monthly review. Across this scenario's 4 incidents/year, daily checks avoid nearly $739,726 in annual loss. An operational budget must instead use the measured time from incident onset to effective recovery, including evidence collection, diagnosis, and mitigation. An earlier alert saves money only when it enables an earlier reduction in harm.

Divergence is what a label-free monitor can measure, and the degradation equation of equation 1.2 turns it into a forecast of accuracy when outcome evidence supports one:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

where λ is fitted per deployment from historical degradation data, as the statistical drift diagnostic (principle 10) requires. The decay rate γ of the retraining economics is the time-domain summary of this relation (section 14.4.2.2). While not a universal law, tracking distributional divergence $\mathcal{D}(P_t \| P_0)$ can provide an earlier warning than delayed outcomes; it does not establish the direction or size of a quality change.

Figure 14.8 charts this operational trade-off over a 365-day deployment lifecycle, contrasting two distinct retraining cadences against a red dashed drift threshold (0.88 accuracy). Under a scheduled policy (orange sawtooth), the model retrains at rigid 90-day intervals, repeatedly dipping below the quality floor before each reset. In contrast, a triggered policy (green sawtooth) initiates retraining the moment monitored accuracy touches the threshold, illustrating an idealized instantaneous reset;

in practice the trigger is often a divergence or proxy signal calibrated against accuracy. Both curves are drawn from the temporal decay model of equation 14.4. In deployment, sample collection, label arrival, training, validation, and rollout delays can carry accuracy below the trigger before recovery; the policy does not guarantee a performance floor.

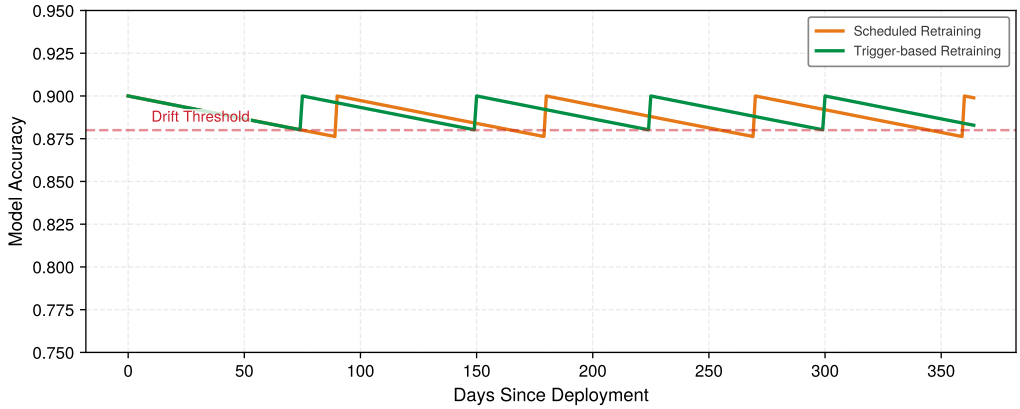


Figure 14.8: The Rotting Asset Curve: An illustrative exponential accuracy decay under two maintenance policies, both drawn as sawtooths. The scheduled policy resets every 90 days regardless of accuracy and dips below the drift threshold before each reset, while the triggered policy resets on crossing that threshold and holds a higher floor in this zero-delay model. Real response delays and update quality must be measured; neither cadence is universal.

The two curves turn drift from an abstract statistical problem into an operations policy choice. Scheduled retraining is easy to plan but can retrain too early or too late; trigger-based retraining requires stronger telemetry but aligns intervention with observed degradation.

Whichever policy fires, and whenever telemetry signals degradation, the registry lineage of section 14.4.1.3 is what makes diagnosis and rollback possible. Each registered version should link immutable digests for the training code, dataset snapshot, hyperparameters, and serialized weights, so a responder can return to a known-good checkpoint. As the reproducibility principle warned, that chain still cannot guarantee bitwise replay if driver versions, CUDA libraries, or nondeterministic reduction kernels vary across the fleet.

14.6.2.1 Data quality monitoring

Drift statistics reveal shifted inputs, while outcome metrics establish whether prediction quality has declined. By the time downstream business metrics degrade, upstream data corruption may have persisted for days or weeks. Data quality monitoring exposes these input defects before inference occurs, while output and outcome monitoring confirm whether distributional shifts impair prediction accuracy. The first input guardrail is executable validation: listing 14.4 illustrates schema expectations that reject malformed batches before inference, turning implicit data assumptions into enforceable software contracts.

Listing 14.4: Input Data Validation: Schema validation rules check column existence, data types, null values, and statistical bounds to catch data quality issues before they propagate to model inference.

```
schema.require_column("user_id")
schema.require_type("timestamp", "datetime")
schema.require_non_null("feature_a")

schema.require_range("age", min_value=0, max_value=120)
schema.require_mean_between(
    "purchase_amount", min_value=10, max_value=1000
)
```

Input data validation Schema validation[24] catches structural defects at ingest time. These checks enforce column existence, strict data typing, null-value limits, and numerical boundary invariants.

[24] **Schema validation**: The rules in listing 14.4 prevent silent data contract violations, such as a feature column changing from an integer to a float. Without this input-level guardrail, downstream model monitoring cannot distinguish a data quality error from a true performance regression, masking the root cause. A schema mismatch in a critical feature can invalidate an otherwise well-formed prediction batch.

Feature distribution monitoring Schema validation catches structural corruption (missing columns, wrong types, null values) but cannot detect the subtler failure mode where data arrives in the correct format but from a shifted distribution. A feature representing user age can pass every schema check while its mean silently migrates from 32 to 45 over three months as marketing attracts an older demographic. This distributional shift can change model accuracy without producing a structural anomaly; labeled outcomes are needed to determine its effect.

Gradual long-term degradation is particularly insidious because it evades coarse detection thresholds: small day-to-day changes in a quality metric compound into material degradation over a year without tripping monthly alerts. Seasonal patterns compound this complexity; a model trained in summer may perform well through autumn but fail in winter conditions it never observed. Detecting such gradual degradation requires multi-timescale monitoring: performance baselines across multiple time horizons (daily, weekly, quarterly), sliding window comparisons that detect slow trends, and seasonal profiles that account for cyclical variation.

Statistical distance measures quantify this divergence by comparing serving distributions against training baselines. Three are common: the population stability index (PSI) for categorical and binned features, the Kolmogorov-Smirnov (KS) statistic of the CI/CD distribution screen for continuous distributions, and Jensen-Shannon divergence for symmetric, bounded comparison of probability distributions. Each needs an alert threshold, and sample size makes that choice harder than it looks.

Feature-by-feature comparisons can miss changes in how inputs occur together. Consider two binary sensor flags. In the training population, equal groups have both flags off or both on; in the serving population, equal groups have one flag on and the other off. Each flag is still equally often on and off, so its individual histogram is unchanged, but the relationship between flags has reversed. Even perfect training-serving feature parity would not detect this population change. Monitoring selected feature combinations, relevant cohorts, or a joint-distribution statistic complements marginal screens; labeled outcomes are still needed to determine whether the change harms predictions.

In high-throughput production monitoring, statistical significance alone is an inadequate alert policy. With large samples, tests such as the two-sample KS test or the chi-square test (the standard test for differences in categorical frequencies) can detect small genuine differences that have little operational significance. The mechanism is the sample-size term in the test's critical value. The KS critical value c_{crit} used in notebook 4.6 scales with $\sqrt{(n_0+n_t)/(n_0 n_t)}$, so it shrinks roughly as $1/\sqrt{n}$, and a monitor that compares millions of serving requests against a large training sample rejects equality for shifts far smaller than the 0.1 gate of listing 14.2. These detections are not necessarily statistical false positives, but paging on each one creates unnecessary alerts. Conversely, small-sample edge telemetry may lack power to detect a meaningful change. Monitoring must consider effect size, sampling uncertainty, and the cost of missed change together.

Production monitoring therefore combines distribution-distance measures with uncertainty and operational thresholds. A common measure is PSI,[25] which quantifies distributional shift by measuring the binned relative entropy between expected (training) and actual (serving) frequencies across fixed bins (section B.2.2 develops the mathematical foundations of KL divergence, PSI, and information theory for systems monitoring). Here n_{bins} is the number of fixed bins, and expected_i and actual_i are aligned, strictly positive bin proportions that sum to one in each distribution. Equation 14.11 formalizes the metric:

$$\text{PSI} = \sum_{i=1}^{n_{\text{bins}}} (\text{actual}_i - \text{expected}_i) \times \ln\left(\frac{\text{actual}_i}{\text{expected}_i}\right) \tag{14.11}$$

Because $(\text{actual}_i - \text{expected}_i)$ and $\ln(\text{actual}_i/\text{expected}_i)$ always share the same algebraic sign, each bin's contribution is strictly non-negative: shifts in either direction penalize the aggregate metric. An illustrative policy using common scorecard-monitoring bands is:

- PSI < 0.1: Below this policy's warning threshold; continue monitoring outcomes and critical slices.
- $0.1 \leq$ PSI < 0.25: Warning range; investigate population changes and upstream pipeline behavior.
- PSI ≥ 0.25: Escalation range; notify the owning team under the service's calibrated severity policy. Diagnosis determines whether intervention is needed.

[25] **PSI (population stability index)**: PSI is widely used in credit-risk scorecard monitoring to compare expected and observed binned populations; Yurdakul and Naranjo analyze its statistical properties (Yurdakul and Naranjo 2020). The common 0.1 and 0.25 bands are useful operational conventions, not universal statistical laws. ML operations adopted PSI because it works on binned categorical or continuous features and provides an interpretable drift score that non-specialists can review.

Production monitors require a documented zero-bin policy, such as Laplace smoothing, because any empty bin in the denominator yields an undefined logarithm. Monitors must also preserve fixed bin boundaries across comparison periods; PSI is a drift signal, not an automatic retraining trigger, because divergence alone does not establish that accuracy has changed (principle 10).

For discrete or binned distributions, KL divergence provides another comparison. Let P_t be the monitored serving distribution and P_0 the training reference distribution, the same pair that the degradation equation compares. Equation 14.12 defines $\mathcal{D}_{\text{KL}}$, one concrete choice for the generic divergence $\mathcal{D}(P_t \| P_0)$, with log taken as the natural logarithm, as in PSI:

$$\mathcal{D}_{\text{KL}}(P_t \| P_0) = \sum_x P_t(x) \log \left(\frac{P_t(x)}{P_0(x)} \right) \tag{14.12}$$

Because KL divergence is asymmetric and becomes infinite when $P_t(x) > 0$ where $P_0(x) = 0$, operational monitors must preserve direction and apply an explicit support policy, such as smoothing previously unseen bins. Algebraically, PSI is identical to the symmetric Kullback-Leibler divergence (also known as Jeffreys divergence), expressing $\mathcal{D}_{\text{KL}}(\text{actual} \| \text{expected}) + \mathcal{D}_{\text{KL}}(\text{expected} \| \text{actual})$, which guarantees that probability shifts in either direction yield positive drift contributions. Similarly, Jensen-Shannon divergence avoids KL divergence's infinite explosion on non-overlapping distributions by evaluating divergence against the average distribution $\bar{P} = \frac{1}{2}(P_t + P_0)$, yielding a smooth, symmetric divergence bounded between 0 and 1 (using base-2 logarithms). A shift that is plainly visible on a histogram can still produce a small aggregate PSI, which is why thresholds must be calibrated per feature.

With all three measures defined, table 14.17 collects representative starting thresholds for each.

Table 14.17: Feature Distribution Thresholds: Starting points for drift detection, calibrated in practice to each feature's sensitivity and business impact. PSI thresholds such as 0.1 and 0.25 are common scorecard-monitoring conventions, while KS and JS thresholds must be calibrated to the feature, sample size, and cost of missed drift. Higher thresholds reduce alert fatigue but risk missing gradual drift.

Metric	Alert Threshold	Use Case
PSI	0.25 or higher	Categorical and binned features
Kolmogorov-Smirnov statistic	KS > 0.1	Continuous feature distributions
Jensen-Shannon divergence	JS > 0.1	Probability distributions

In a recommendation system monitoring user age, a shift toward older cohorts produces the bin-by-bin PSI decomposition shown in table 14.18:

Table 14.18: PSI Worked Example: User age distribution drift from training to serving, decomposed across six age bins. The total PSI is 0.029, well below the 0.1 warning threshold, even though several bins shifted by 3 percentage points. Aggregate PSI summarizes movement across bins; action depends on a calibrated threshold, sample size, and business cost.

Age Bin	Training	Serving	Difference	ln(Serving/Training)	Contribution
18–25	15%	12%	-0.03	-0.223	0.0067
26–35	25%	22%	-0.03	-0.128	0.0038
36–45	20%	18%	-0.02	-0.105	0.0021
46–55	18%	20%	+0.02	+0.105	0.0021
56–65	12%	15%	+0.03	+0.223	0.0067
66+	10%	13%	+0.03	+0.262	0.0079

Summing the six bin contributions yields a total PSI of 0.029. The training and serving columns are shown as percentages, while the difference column is expressed in proportion units, so a 3 percentage-point shift appears as ± 0.03. Even though specific bins shifted by 3 percentage points, the aggregate drift is well below the 0.1 warning threshold. Operational action depends on a calibrated threshold, sample size, and business cost.

↪ KL divergence, its asymmetry, and a worked drift example that computes both KL divergence and PSI are developed in section B.2.2.

Data freshness monitoring Feature stores and data pipelines can become stale without triggering obvious errors. Data freshness monitoring catches that failure mode, and listing 14.5 shows a configuration that monitors feature freshness and triggers fallback behavior when data becomes stale.

Listing 14.5: Data Freshness Alert Configuration: This configuration monitors the `user_purchase_history` feature for staleness, alerting operations teams via PagerDuty and Slack and falling back to default values when the feature exceeds the maximum allowed age.

```yaml
# Example freshness alert configuration
feature: user_purchase_history
max_staleness: 6h
alert_channels: [pagerduty, slack]
on_stale:
  action: fallback_to_default
  default_value: []
```

A freshness policy turns staleness from a silent data defect into an explicit fallback; the same detect-and-respond contract must cover every layer of the monitoring stack.

Checkpoint 14.3: The monitoring stack

ML monitoring is layered, not monolithic. The prompts in this checklist test whether each symptom can be traced to the responsible layer.

- ☐ **Infrastructure utilization**: Explain why a GPU reporting high kernel-active time can still be memory-bound, and which measurements (achieved FLOP rate, memory traffic, stall counters) would confirm it before choosing a remedy.
- ☐ **Telemetry sampling**: Explain what retaining one request trace in 60 preserves for trend monitoring, and what short-lived anomalies it can miss.
- ☐ **Data freshness**: Detect stale feature ingestion pipelines before plausible predictions mask obsolete real-world states.
- ☐ **Data drift**: Evaluate population stability index (PSI) or KL divergence metrics to detect when serving inputs diverge from the training baseline.
- ☐ **Model accuracy**: Account for delayed ground-truth labels when interpreting real-time task performance alerts.
- ☐ **Cohort bias**: Apply stratified cohort analysis to detect subgroup degradation hidden by aggregate top-level accuracy.

The same stack must monitor upstream dependencies feeding the ML system: database replication lag, feature-store ingestion watermarks, and pipeline completion status. In one representative incident, a recommendation service detected a material shift in its `user_lifetime_value` feature within two days, tracing the anomaly to a database migration that silently altered an aggregation window. Without input-layer distribution monitoring, such upstream corruption degrades model predictions for weeks before ground-truth outcome metrics expose the failure.

14.6.3 Monitoring cost model

Every layer of that stack costs money to run. Notebook 14.3 showed that sampling policy alone moves telemetry volume by 60×. A complete budget must also price storage, dashboard queries, and alert evaluation, and then weigh the total against the incidents that monitoring prevents.

Cost components

Monitoring costs break down into four categories, as equation 14.13 decomposes:

$$\text{Monitoring Cost} = C_{\text{ingest}} + C_{\text{storage}} + C_{\text{compute}} + C_{\text{alert}} \tag{14.13}$$

The four C_* terms are cost components over the same accounting window: data ingestion, retained storage, query or dashboard compute, and alert-rule evaluation. Separating them matters because each scales with a different control knob.

Table 14.19 provides representative unit-cost assumptions for each component. Translating these unit costs into a concrete budget estimate clarifies the real expense of monitoring even a single production model:

Table 14.19: Monitoring Cost Components: Illustrative scenario unit costs used for the worked example. Costs scale differently across components: metric ingestion scales with cardinality (number of unique metric series), storage scales with retention, and query costs scale with dashboard usage patterns.

Component	Illustrative Unit Cost	Scaling Factor
Metric Ingestion	\$0.10–0.50 per million data points	Number of metrics × sample rate
Log Storage	\$0.50–2.00 per GB/month	Log verbosity × retention period
Query Compute	\$0.01–0.05 per query	Dashboard refresh rate × users
Alert Evaluation	\$0.001–0.01 per evaluation	Number of alert rules × check frequency



Napkin Math 14.6: Single-model monitoring budget

Problem: Calculate the monthly metric ingestion, time-series storage, and query compute expenditure incurred to monitor a single ML serving node under production assumptions.

Variables:

- one model with 3 deployment variants (production, canary, staging), each emitting 50 metrics
- Metrics sampled every 15 seconds
- Retention requirement: 30 days
- 2 dashboards (model health, infrastructure), 3 team members, five-minute refresh

Metric ingestion:

- Data points per month: $3 \times 50 \times (4 \text{ samples/min} \times 60 \times 24 \times 30 \text{ days}) = 25.9\text{M}$
- Cost at \$0.30/million: \$7.8/month

Storage:

- At 8 bytes/point compressed: 25.9M × 8 bytes = 0.2 GB
- Cost at \$1/GB: \$0.21/month

Query compute:

- Queries per month: 2 dashboards × 3 users × (12 queries/hour × 8 hours/day × 22 days) = 12,672 queries/month
- Cost at \$0.02/query: \$253.4/month

Total: ~\$261.4/month for a single ML node. Alert evaluation and platform overhead are excluded.

Systems insight: Under these assumptions, cost scales linearly with identical nodes; at platform scale, query-cost optimization becomes increasingly important.

Cost optimization strategies

Each cost component in equation 14.13 maps to a distinct architectural mitigation. The primary driver of C_{ingest} and C_{storage} is metric cardinality: high-cardinality labels such as `user_id` or `request_id` cause combinatorial state explosion in time-series indexes, dwarfing raw compute costs. Dropping unnecessary labels or aggregating them into a bounded set of buckets reduces cardinality. Hashing each identifier to another distinct identifier alone does not reduce the number of series. The second

driver of storage cost is temporal resolution: retaining raw 15-second metrics across 30 days is rarely justified. A tiered retention policy (high-resolution telemetry for recent incident debugging, downsampled rollups for historical trend analysis) preserves diagnostic fidelity while slashing retained volume. For C_{compute}, dashboard query overhead accumulates through auto-refresh loops across dozens of unattended user sessions; setting conservative refresh intervals and auto-pausing inactive sessions bounds query load. Finally, C_{alert} scales with evaluation frequency: consolidating overlapping threshold checks into composite multi-condition rules can reduce redundant evaluation and notifications; thresholds, ownership, and actionability still determine alert fatigue.

Cost-benefit framework

Justify monitoring investments against incident costs using the monitoring benefit-cost ratio in equation 14.14:

$$\text{Monitoring Benefit/Cost} = \frac{\text{Incidents Prevented} \times \text{Avg Incident Cost}}{\text{Annual Monitoring Cost}} \tag{14.14}$$

If average incident costs \$50,000 (downtime + engineering time + reputation) and monitoring prevents 5 incidents annually at \$50,000 monitoring cost:

$$\text{Benefit/Cost} = \frac{5 \times \$50K}{\$50K} = 5\times$$

This framework helps justify monitoring investments and prioritize which metrics deserve fine-grained observation vs. coarse sampling. The monitoring systems themselves require resilience planning to prevent operational blind spots. When primary monitoring infrastructure fails (Prometheus experiencing downtime or Grafana becoming unavailable), teams risk operating blind during critical periods. Production-grade MLOps implementations therefore maintain redundant monitoring pathways: secondary metric collectors that activate during primary system failures, local logging that persists when centralized systems fail, and heartbeat checks that detect monitoring system outages.

Robust production deployments implement cross-monitoring where external watchdog infrastructure monitors the telemetry pipelines themselves, ensuring that observation failures trigger immediate alerts through alternative notification pathways. This defense-in-depth architecture reduces the risk of undetected simultaneous model and telemetry failures, provided the alternative paths do not share the failed dependency. Coordinating these safeguards across many replicated services, with consensus-based alerting and cross-region metric aggregation, is a fleet-scale concern that arises once a model is replicated across regions, beyond the single-node scope here.

Layered monitoring converts silent degradation into signals. Infrastructure telemetry shows whether the serving path is compute-, memory-, or I/O-bound, drift statistics show that inputs have moved, and delayed outcome metrics confirm whether the movement cost accuracy. A signal is not yet a response, however. Every alert must still be triaged, localized to data, model, or infrastructure, and acted on by someone with the authority and the runbook to act.

14.7 Incident Response and Governance

The ridesharing demand model from the opening of this chapter drew attention only in week eight, and a product manager noticed before any system did. Drift monitoring could have raised the alarm by week four, when accuracy had already fallen to 88 percent, but an alert only relocates the problem. Someone must then decide quickly whether the signal is real, which system caused it, and whether to roll back, and the organization must already have decided which models were allowed to ship and who owns them. Incident response supplies the first set of decisions, governance supplies the second, and the ML Test Score of section 14.3.7 audits whether both are in place before a failure tests them.

14.7.1 Incident response and operational practices

Incident response, debugging, and on-call rotation form the human half of the monitoring loop. They turn a statistical signal into an engineering action within a bounded time.

14.7.1.1 Incident response for ML systems

At 2:00 AM, an on-call engineer receives an alert: recommendation click-through rate has dropped 12 percent over the past hour. There is no stack trace, no error log, no crashed process, just a statistical signal that something has changed. The responder must distinguish among four candidate root causes: an upstream data-pipeline failure, model drift, a seasonal traffic pattern, or statistical noise. This ambiguity is common in ML incidents: symptoms may appear as degradation in outcome or predictive-quality metrics rather than explicit errors, so incident response must account for statistical uncertainty.

Severity classification provides the foundation for prioritizing response in this ambiguous landscape. Table 14.20 gives one recommendation service's illustrative policy, with response deadlines from 15 minutes to 24 hours. Priority depends on affected traffic, consequences, and the need for containment. Click-through rate measures engagement rather than classification accuracy, and a population stability index value alone establishes neither quality loss nor incident severity.

Table 14.20: Incident Severity Classification for ML Systems: Illustrative response deadlines belong to this service's policy. Impact, evidence, and containment needs determine priority; drift magnitude alone does not.

Level	Criteria	Response Time	Example
P0	Service unavailable across live traffic	15 minutes	Null predictions prevent recommendation delivery
P1	Broad harmful impact needing containment	1 hour	CTR falls across major cohorts; replay confirms a broken feature contract, while longer-term outcomes remain pending
P2	Bounded impact with a validated fallback	4 hours	A localized feature defect is contained; the owner investigates remaining exposure and pending outcomes
P3	No observed impact or immediate exposure	24 hours	Training is delayed while the current release remains within its measured operating limits

Once an incident is classified, responders execute a structured response workflow where urgent containment can proceed alongside evidence collection:

1. Detection identifies which monitoring signal, affected metric, and threshold triggered the alert.
2. Impact assessment quantifies the fraction of serving traffic, user cohorts, or downstream services exposed.
3. Change correlation audits recent deployments across model weights, serving runtimes, feature transforms, and upstream data pipelines.
4. Mitigation evaluates and executes containment options, including traffic shifting, fallback enablement, or rollback to a known-good checkpoint.
5. Root cause analysis isolates whether the defect originates in data pipelines, model representations, or serving infrastructure.

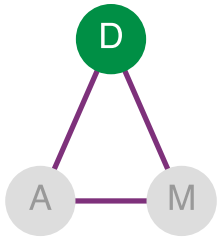
For P0 and P1 incidents, postmortem documentation is required. These postmortems analyze timelines, root causes, user impact, and preventive measures, specifically identifying which monitoring gap allowed the issue to reach production and what automated validation would have caught it earlier.

14.7.1.2 Model debugging: From detection to diagnosis

Incident response triages and mitigates; model debugging identifies root causes. Monitoring detects that something is wrong; debugging determines *why*. ML debugging must account for probabilistic and data-dependent failures in addition to conventional software defects. An incorrect prediction does not throw an exception or generate a stack trace, making systematic debugging essential for resolving ML incidents efficiently.

The debugging decision tree When model performance degrades, the following five stages provide a starting workflow; recent changes and incident severity can alter their priority. Section A.8 supplies a systematic diagnostic matrix that maps symptoms to D·A·M (Data · Algorithm · Machine) axes.

1. **Data pipeline integrity**: Inspect upstream data pipelines for schema changes, missing values, serialization errors, or input distribution shifts. Data integrity is an effective starting boundary because altered inputs, corrupted labels, and upstream feature changes can affect every downstream prediction.
2. **Training-serving skew**: Replay matched requests through the versioned offline and serving transformations and compare feature values, missingness, units, and time cutoffs. KS or PSI separately screen for population divergence; neither verifies that the two paths compute the same value for a given request.
3. **Subpopulation slicing**: Disaggregate performance across operational dimensions (such as geography, client hardware, or user cohorts). Localized degradation isolates data coverage gaps and slice-specific label noise that aggregate metrics conceal.
4. **Temporal degradation dynamics**: Evaluate performance trends across time. Abrupt drops prioritize recent release artifacts and upstream infrastructure outages, whereas gradual degradation motivates checking seasonality and changing predictive relationships.
5. **Model and representation attribution**: After verifying data pipeline integrity and ruling out serving skew, evaluate model-level failure modes through prediction confidence distributions, calibration curves, and feature attribution.



Production debugging starts on the data axis of D·A·M.

Slice analysis Slice analysis can guide pipeline and serving-skew investigations by testing whether the apparent system-wide problem is concentrated in a subpopulation. Performance metrics aggregated across all traffic can mask significant problems in subpopulations. **Slice analysis** exposes that masking by computing the same metric separately for each segment, and table 14.21 shows how a healthy overall accuracy can hide severe degradation in one of them.

Table 14.21: Slice Analysis Example: Overall accuracy of 91 percent appears acceptable, but tablet users (5 percent of traffic) experience 62 percent accuracy, a severe degradation masked by aggregation. Effective debugging requires systematic slice analysis across key dimensions.

User Segment	Traffic %	Accuracy	Impact
Desktop users	45%	94%	Nominal
Mobile (iOS)	30%	92%	Nominal
Mobile (Android)	20%	88%	Minor degradation
Tablet users	5%	62%	Severe—investigate
Overall	**100%**	**91%**	**Masks tablet problem**

Feature attribution for debugging When slice analysis identifies a problematic segment, feature attribution techniques help identify which features the model relies on within incorrect predictions. Listing 14.6 demonstrates a workflow that uses SHAP values, feature-attribution scores that estimate how much each input feature contributed to an individual prediction, to analyze mispredictions within a specific slice.

Listing 14.6: SHAP-Based Debugging Workflow: This code filters mispredicted tablet examples, computes SHAP values for their model outputs, and plots the selected examples' feature attributions.

```
# SHAP-based debugging workflow
import shap

# Select mispredicted examples from problematic slice
errors = predictions[
    (predictions.actual != predictions.predicted)
    & (predictions.device_type == "tablet")
]

# Compute SHAP values for error cases
explainer = shap.Explainer(model)
shap_values = explainer(errors[feature_columns])

# Plot feature attributions for the selected errors
shap.summary_plot(shap_values, errors[feature_columns])
```

The attribution plot identifies features the model relied on within the failing slice; it does not establish whether those features caused the errors. Diagnosing stale features, missing coverage, or semantic shift requires checking feature lineage and production distributions.

↪ Post-hoc feature attribution methods, Shapley value computational costs, and model interpretability boundaries are developed in section 15.3.3.1.

> **Systems Perspective 14.1: Zombie features**
>
> Long-lived production models often accumulate features whose original owners, semantics, or intended use have faded. A feature can be deprecated in application code while still flowing through a feature store, training dataset, or serialized model input contract. The model may learn to ignore it, split signal across a duplicate, or depend on a preprocessing artifact that nobody still owns. Removing such a feature is therefore no longer a local cleanup: it becomes a compatibility change that can affect retraining, serving, monitoring, and downstream analysis.
>
> Features in ML systems do not disappear just because code owners stop thinking about them. Unused feature payload volume D_{vol} inflates serialization overhead and consumes memory bandwidth BW without improving accuracy. Without explicit deprecation policies and feature-store governance, models accumulate "dead code" that degrades D_{vol}/BW and complicates debugging (Sculley et al. 2015).

Zombie features show that attribution can reveal what a model should no longer depend on. Counterfactual analysis asks the complementary question about one decision at a time.

Counterfactual analysis For individual mispredictions, counterfactual analysis identifies the minimal change that would flip the prediction. If a churn prediction would flip to "engaged" when `user_session_length` rose from 12 minutes to 45, that sensitivity suggests checking for the definition mismatch illustrated in the skew case study (section 7). Matched-input comparisons and feature lineage must establish whether the churn system has the same pipeline defect before the counterfactual can guide a repair. Counterfactuals reveal which feature boundaries drive decisions and whether those boundaries make semantic sense. Counterfactuals that require implausible changes ("user age would need to be -5 years") often indicate feature engineering problems.

These techniques (the debugging decision tree, slice analysis, feature attribution, and counterfactuals) form a debugging toolkit. To apply them consistently, teams codify the process.

Debugging checklist The diagnostic lenses identify places to investigate; a six-phase checklist organizes how to test a suspected cause. The ordering is deliberate because each phase narrows the search space for the next:

1. **Reproduce**: An ML failure that cannot be reproduced on held-out data is often data-dependent, an insight that redirects investigation toward the D·A·M taxonomy's data layer.
2. **Isolate**: Once reproduced, identifying the minimal input set that triggers the failure transforms a diffuse "the model is wrong" complaint into a specific, testable condition.

3. **Bisect**: If the failure correlates with a recent deployment, controlled comparisons of versioned model, feature-transform, and runtime artifacts help isolate which change introduced the regression.
4. **Attribute**: Feature attribution applies interpretability techniques to identify which input factors drive the erroneous behavior.
5. **Validate**: Confirming that the hypothesized root cause, when corrected, actually resolves the failure distinguishes genuine fixes from coincidental improvements.
6. **Prevent**: Converting each resolved incident into a monitoring rule or validation check systematically closes the gap between detection and recurrence. This cumulative hardening can reduce repeated failure modes over time because each incident strengthens the observability infrastructure.

The session-duration defect illustrates how these phases lead to a specific response. Suppose the CTR alert coincides with a serving-transform release on one recommendation host. The model weights, runtime, and offline feature definition are unchanged. Request logs place the new transform on the affected sessions, and replaying those exact inputs reproduces the wall-clock versus foreground-active mismatch. Representative traces show unchanged feature-retrieval and model-execution durations, weakening a latency-regression explanation without ruling out every infrastructure fault. Mature recommendation outcomes for the new release are still pending, so the responder has established a feature-contract violation, not its eventual effect on accuracy or revenue.

The owning engineer restores the prior serving transform and its compatible feature-cache namespace, first checking the rollback on replayed requests and then exposing it through a bounded canary. Matched values agree again, and the same-input scores return to the prior artifact's outputs within the declared tolerance. Those observations demonstrate recovery of the feature contract; later comparable, mature outcome cohorts must establish whether user outcomes recover. A weights-only rollback leaves the changed feature computation in place, and immediate retraining would teach against the defect rather than repair the contract. The prevention step adds the affected idle-session inputs to parity validation and records the transform, cutoff, and cache versions needed for future replay.

Debugging ML systems requires both systematic methodology and domain expertise. The most effective debugging often comes from engineers who understand both the model architecture and the business context of the predictions.

14.7.1.3 On-call practices for ML systems

The debugging techniques in section 14.7.1.2 work when an engineer is actively investigating an issue during business hours. Production systems, however, fail at 3:00 AM on weekends, and the person responding may not be the one who built the model. Debugging resolves individual incidents; on-call practices sustain operational health over time by ensuring that someone with appropriate expertise is always available and equipped to respond. On-call rotation for ML systems requires specialized practices beyond traditional software operations because ML incidents often manifest as gradual degradation rather than hard failures. A stack trace can localize some software failures, while silent semantic failures require broader investigation in any system. An ML engineer facing a 3 percent accuracy drop must determine whether the change represents statistical noise, a changed predictive relationship, or a failure requiring immediate mitigation. This distinction demands statistical context rather than simple log analysis.

This ambiguity compounds with delayed impact visibility. Unlike latency spikes that surface immediately in dashboards, ML degradation may take hours or days to manifest in business metrics. A recommendation model that began serving slightly worse suggestions on Monday might not produce measurable revenue impact until Friday, by which time the window for easy diagnosis has closed. Cross-system dependencies further complicate response: ML issues often originate in upstream data systems owned by different teams, requiring coordination across organizational boundaries during incident response. The deepest challenge is that effective response demands understanding model behavior, not infrastructure health alone. A database administrator can restart a crashed service without understanding its business logic, but an ML engineer cannot meaningfully debug accuracy degradation without understanding the model's feature dependencies and expected behavior patterns.

These challenges motivate tiered escalation structures that match expertise to incident complexity. Table 14.22 illustrates one possible structure, where primary responders handle routine issues using standardized runbooks while escalation paths connect to specialists capable of deeper investigation. A parallel data on-call role can reduce time to resolution when the root cause lies in an upstream data system.

Table 14.22: ML On-Call Structure: Tiered escalation with parallel data on-call enables efficient incident response. Tier 1 handles routine issues using runbooks; Tier 2 addresses complex debugging; Tier 3 manages critical incidents requiring architectural decisions.

Tier	Responder	Responsibility
Tier 1 (Primary)	ML Engineer	Initial triage, standard runbooks, escalation decisions
Tier 2 (Escalation)	Senior ML Engineer/Data Scientist	Complex debugging, cross-system investigation, model-specific issues
Tier 3 (Critical)	ML Platform Lead	Architecture decisions, major incidents, vendor escalation
Data On-Call (Parallel)	Data Engineer	Data pipeline issues, feature store problems, upstream dependencies

Effective on-call depends heavily on runbook quality. Every production ML model should have documentation covering the model's purpose, ownership, and business criticality alongside its normal operating parameters: expected latency, throughput, and accuracy ranges that define healthy behavior. Historical incidents and their resolutions provide templates for common failure patterns, while diagnostic commands enable rapid health assessment: how to check recent predictions, feature distributions, and model confidence scores. Critically, runbooks must specify escalation criteria (when to wake up Tier 2 vs. when to rollback without approval) and rollback procedures with step-by-step instructions and expected recovery times. Runbooks written during calm periods save critical minutes during 3:00 AM incidents.

Even well-designed monitoring can generate excessive alerts that erode on-call effectiveness. Alert fatigue, the tendency to ignore or dismiss alerts after experiencing too many false positives, represents a significant operational risk. Teams combat fatigue through consolidation, grouping related alerts so that multiple features drifting simultaneously generate a single notification rather than dozens. Adaptive thresholds that account for weekly and seasonal patterns prevent predictable variations from triggering unnecessary pages. Alerts that are rarely actionable should be retired or recalibrated. When temporary silencing is necessary, accountability mechanisms (requiring a follow-up ticket before snoozing) prevent alerts from being permanently ignored.

Shift handoffs represent another critical practice that distinguishes mature operations. Incoming on-call engineers need context about active incidents and their current status, recent deployments that might cause delayed issues, upcoming scheduled changes such as data migrations or model updates, and any alerts that were suppressed along with the reasoning. Without structured handoffs, context is lost between shifts, and incoming engineers waste time rediscovering information their predecessors already gathered.

Sustainable on-call practices must also address burnout. ML on-call carries particular stress due to incident ambiguity: the uncertainty of not knowing whether an alert represents a real problem demands constant vigilance. Organizations mitigate burnout by limiting consecutive on-call days, providing compensatory time off after high-severity incidents, conducting regular rotation reviews to balance load across team members, and investing in automation that reduces toil. The goal is to make on-call rotations sustainable over years of operation, not to staff them as an afterthought.

Technical monitoring capabilities alone do not ensure operational success. The most sophisticated dashboards fail if no one is responsible for acting on alerts, and the most detailed runbooks languish if team structures do not support their use. Production ML operations require organizational infrastructure paralleling the technical: clear governance, defined roles, and communication patterns that enable cross-functional coordination.

14.7.2 Governance and team coordination

On-call runbooks address real-time operational failures, but production ML systems also require proactive governance to enforce behavioral and regulatory invariants. Model governance establishes the verification criteria that models must satisfy before receiving live inference traffic. Unchecked models risk producing biased inferences or uninterpretable decisions that violate statutory regulations and organizational policies. Within an operational pipeline, governance enforces three concrete objectives: transparency through auditable training data lineage and verifiable model weights, fairness through evaluation against the application's selected and justified criteria, and compliance through adherence to privacy, retention, and auditability mandates. While chapter 15 details the statistical metrics and interpretability algorithms required for these properties, MLOps provides infrastructure to collect evidence and enforce specified checks across the deployment lifecycle, alongside accountable policy and domain review.

ML governance spans the full lifecycle of the artifact. During development, automated pipelines record feature schemas, training data provenance, and hyperparameter configurations. Prior to deployment, prerelease validation gates evaluate out-of-distribution robustness and the selected fairness criteria. Once in production, monitoring systems must track changes in the corresponding subgroup metrics alongside aggregate accuracy degradation. Encoding testable governance policies in CI/CD pipelines makes release checks repeatable; choices about acceptable risk, fairness criteria, and applicable obligations still require accountable review.

Concretely, a model registry promotion gate requires a signed feature contract, a recorded training-data lineage hash, subgroup metrics above policy thresholds, a canary SLO with automated rollback criteria, and a designated artifact owner before a model transitions from staging to production. That gate turns documented release requirements into assertions enforced by the deployment machinery. Passing those assertions establishes that the specified checks passed, not that every governance obligation has been satisfied.

Governance policies define the release criteria, but operational reliability depends on execution across organizational boundaries. Because machine learning systems span data engineering, statistical modeling, and distributed serving infrastructure, role handoffs represent the most failure-prone interfaces in the operational lifecycle. Versioned experiment stores and centralized model registries provide the shared substrate required for reproducible transitions between research and production engineering. At the data interface, formal schema specifications and lineage DAGs ensure that data scientists and platform engineers share identical assumptions regarding feature semantics, default null handling, and label arrival latency.

Titles and boundaries vary across organizations, but one common decomposition uses the five roles in table 14.23. The table maps each role to a primary responsibility without implying that every organization needs five separate positions:

Table 14.23: ML Team Roles Matrix: Clear role boundaries prevent gaps and overlaps. Data Scientists focus on model quality while ML Engineers handle productionization. Data Engineers own data pipelines while Platform Engineers own MLOps tooling. SREs ensure overall system reliability.

Role	Primary Focus	Key Deliverables	Collaboration Points
Data Scientist	Model development, experimentation, algorithm selection	Trained models, experiment results, performance benchmarks	Hands off to ML Engineer for productionization
ML Engineer	Production ML systems, training pipelines, serving infrastructure	Deployed models, training pipelines, serving systems	Receives from Data Scientist; works with Platform Engineer on infrastructure
Data Engineer	Data pipelines, feature engineering, data quality	Feature pipelines, data quality systems, feature stores	Provides data to Data Scientist; maintains feature store for ML Engineer
Platform Engineer	MLOps infrastructure, tooling, automation	CI/CD pipelines, monitoring systems, compute infrastructure	Enables ML Engineer; maintains shared infrastructure
DevOps/SRE	Reliability, incident response, system health	SLOs/SLAs, on-call procedures, runbooks	Supports all roles; owns production health

Clear role definitions matter most at handoff points, where artifacts transition between specialists. The most failure-prone transition occurs between Data Scientists and ML Engineers: an exploratory notebook script frequently fails in production environments due to undocumented data preprocessing steps, hardcoded local file paths, hidden state execution orders, or unpinned library dependencies. Similarly, the handoff from ML Engineers to SREs (Beyer et al. 2016) requires verified monitoring dashboards, active alerting rules, documented runbooks, and tested rollback procedures. Data Engineers interface with model builders through feature contracts—formal specifications of data schema, freshness SLOs, and distribution invariants that prevent upstream pipeline modifications from surfacing as silent downstream prediction failures weeks later. Teams eliminate these interface risks by requiring standardized container images, hermetic build manifests, and automated reproducibility assertions before any artifact advances to the next stage.

14.7.2.1 Stakeholder communication

While cross-functional handoffs govern internal engineering execution, stakeholder communication establishes the operational contract between technical teams and organizational decision-makers. Learned behavior adds dependence on the data and population, alongside latency-accuracy trade-offs and possible distribution drift. When stakeholders consider only service availability, misalignment can arise regarding expected predictive quality, latency budgets, and maintenance costs.

A frequent communication failure is the unconstrained request to "make the model more accurate." Model accuracy cannot be scaled independently of system costs: boosting validation accuracy by two percentage points often requires moving to a significantly larger model architecture, incurring higher FLOP counts per sample, increasing memory bandwidth demands, and violating strict inference latency SLOs. Grounding improvement discussions in the Pareto frontier (principle 5) forces explicit decisions among model accuracy, inference latency budgets, training compute costs, and dataset acquisition expenses.

Translating statistical metrics into operational impact requires mapping classification errors directly to economic or operational consequences. Conventional evaluation metrics such as accuracy, ROC-AUC, or F1 scores evaluate errors geometrically or symmetrically. In production, however, classification errors incur highly asymmetric costs: in an automated fraud detection pipeline, a false positive creates user friction during checkout, whereas a false negative allows direct, unrecoverable monetary theft.

Figure 14.9 plots this economic trade-off across classification thresholds from 0 to 1, summing a declining false-positive cost (dashed blue) with a climbing false-negative penalty (dashed red). Because missed fraud ($500 per incident) is five times more damaging than blocking a legitimate user ($100 per incident), the total cost curve (solid purple) reaches its global minimum well left of the symmetric 0.50 baseline, isolating the optimal operational decision threshold at 0.34 (green marker).

Operational incidents require structured communication protocols that convey technical severity without ambiguity. When monitoring systems detect distribution drift or an inference regression triggers an automated rollback, engineering status reports must state the empirical evidence directly: the statistical divergence metric, the specific demographic or tenant cohort affected, the canary rollback target, and the estimated recovery timeline. Dismissing anomalous output distributions as transient noise without statistical hypothesis testing allows silent semantic failures to propagate into production decisions.

Justifying hardware resource allocations similarly requires connecting hardware capabilities to development velocity and serving capacity. Requesting additional accelerator nodes must be framed in terms of throughput scaling and iteration speed: provisioning an 8-GPU node enables distributed data-parallel training runs to complete in hours rather than days, directly accelerating model iteration and validation cycles. Furthermore, engineering schedules must reflect real production bottlenecks: data extraction, feature engineering, and pipeline validation typically consume far more calendar time than hyperparameter optimization.

Consider a fraud detection team evaluating a proposed model upgrade. When stakeholders request improved capture rates, the engineering team frames the initiative as a concrete systems trade-off: increasing the fraction of fraud dollars captured from 92 percent to 94 percent requires integrating external feature pipelines, extending training duration by two weeks, and accepting 30 percent higher infrastructure costs. In return, the upgrade prevents an estimated $2 million in

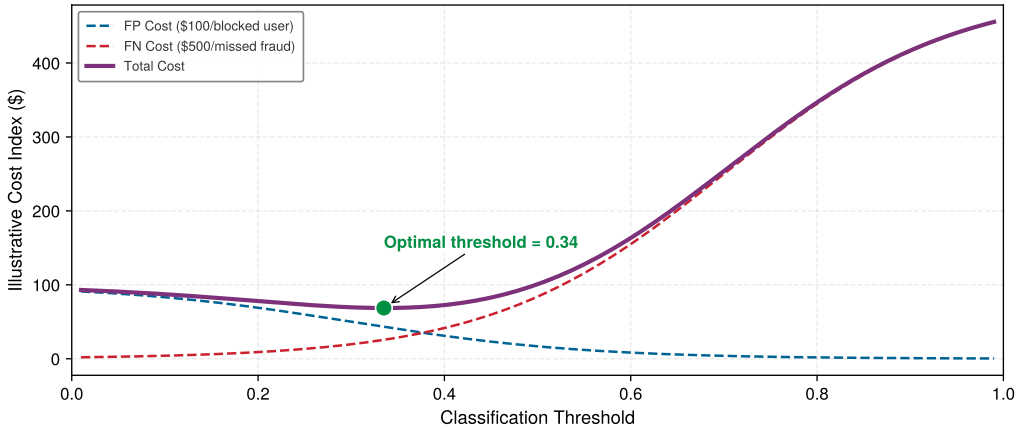


Figure 14.9: The Business Cost Curve: An equal-class-weighted illustrative cost index vs. classification threshold. Technical metrics like ROC curves hide the economic reality: errors have different costs. Here, a false negative (missed fraud) costs $500, while a false positive (blocked user) costs $100. The system flags a transaction as fraud when its score exceeds the threshold, so a lower threshold flags more transactions. Because missing fraud is $5\times$ more costly, the optimal threshold shifts left of center, making the system more aggressive at flagging suspicious transactions and accepting more false positives to minimize expensive misses. With equal costs the optimum would sit at threshold = 0.50; the asymmetry pulls it toward threshold ≈ 0.34. A population expected cost would also weight each term by class prevalence. MLOps tunes thresholds as costs and prevalence change.

annual fraud losses while, under a separate 20 percent reduction assumption, eliminating 50,000 false-positive alerts per month.

Framing model improvements around financial impact, latency budgets, and compute expenditure grounds operational governance in verifiable engineering trade-offs. By coupling automated CI/CD release invariants with structured communication of error costs, engineering teams ensure that production deployments remain both technically robust and economically sound.

Checkpoint 14.4: From alert to accountable action

- ☐ **Diagnosis**: A cohort's accuracy drops while its feature distribution shifts. What additional evidence would distinguish a feature-pipeline defect from a changed predictive relationship?
- ☐ **Release authority**: Name an automated promotion check and a policy decision that its passing result cannot make on the team's behalf.
- ☐ **Readiness score**: Explain how adding a useful model test can improve coverage while leaving the minimum-category ML Test Score unchanged.

14.8 Design and Maturity Framework

An organization deploying its initial ML model might rely on a hand-run Jupyter notebook, a scheduled cron job, and minimal monitoring. A mature enterprise runs thousands of models through automated pipelines with drift detection, canary deployments, and continuous validation. Both are doing "MLOps," yet the gap between them spans orders of magnitude in reliability, cost efficiency, and deployment velocity. Empirical surveys of production deployments reveal that systems failures cluster around data dependencies, pipeline boundaries, and monitoring blind spots rather than core model architectures (Paleyes et al. 2022). Operational maturity provides the systems framework for evaluating this progression: organizations evolve from ad hoc experimentation toward fully automated operations, and identifying where a system stands on this continuum governs resource allocation more effectively than adopting tooling indiscriminately. Placing a system on the continuum raises three questions in turn: what architecture the next level requires,

which organizational and failure-containment patterns keep that architecture working, and how much the move is worth paying for.

14.8.1 Maturity levels

While the ML Test Score assesses individual technical practices, **operational maturity** measures their systemic integration. Lifecycle tools such as MLflow address isolated pipeline stages (Zaharia et al. 2018), but maturity reflects how cohesively data, algorithms, and execution hardware interface across training and serving. Although operational maturity exists on a continuum, categorizing systems into distinct maturity levels clarifies how workflows transition from exploratory prototypes to production-grade infrastructure.

At the lowest level, workflows are ad hoc: training runs manually on local workstations, feature extraction is embedded in ephemeral scripts, and deployment relies on manual file transfers. As maturity increases, workflows become repeatable: teams introduce version-controlled pipelines, centralized artifact registries, and scheduled batch jobs. At the highest level, systems become scalable: declarative infrastructure-as-code, continuous validation pipelines, and real-time observability decouple model iteration from production serving.

Table 14.24 summarizes this architectural progression from disconnected execution toward unified lifecycle management.

Table 14.24: Maturity Progression: Machine learning operational practices evolve from manual, fragile workflows toward fully integrated, automated systems, impacting reproducibility and scalability. Key characteristics and outcomes at each maturity level emphasize architectural cohesion and lifecycle integration for building maintainable learning systems.

Maturity Level	System Characteristics	Typical Outcomes
Ad Hoc	Manual data processing, local training, no version control, unclear ownership	Fragile workflows, difficult to reproduce or debug
Repeatable	Automated training pipelines, basic CI/CD, centralized model storage, some monitoring	Improved reproducibility, limited scalability
Scalable	Fully automated workflows, integrated observability, infrastructure-as-code, governance	High reliability, rapid iteration, production-grade ML

Consider how a fraud detection system evolves across these maturity levels:

- **Ad hoc**: A data scientist trains a model in a Jupyter notebook, exports it as a pickle file, and hands it to an engineer who deploys it to a single server. When accuracy drops, the data scientist retrains manually by running the notebook again with fresh data. Debugging requires the original data scientist because no one else understands the preprocessing steps.
- **Repeatable**: The training script is version-controlled, with a scheduled Jenkins job that retrains monthly. Features are computed in a SQL script that engineering maintains separately. The model is deployed via container, with basic accuracy monitoring. When the feature SQL changes, the data scientist must manually verify the model still works.
- **Scalable**: Training and serving use the same feature store, reducing skew. A CI/CD pipeline investigates drift when PSI exceeds 0.25, retrains when evidence supports it, validates the new model against the baseline, and deploys via canary release. Monitoring tracks per-merchant accuracy, triggering investigation when specific segments degrade. The entire lineage from raw data to production prediction is auditable.

Transitioning between maturity levels requires substantial engineering investment, but the resulting reduction in mean time to recovery (MTTR) and silent failures justifies the cost for mission-critical services. Evaluating an organization against this spectrum isolates specific architectural bottlenecks—such as manual data transfers or unversioned preprocessing logic—before they compound into production outages.

14.8.2 System design implications

The repeatable fraud system above already automates training and deployment, yet a change to its separately maintained feature SQL still requires manual validation. More frequent retraining

would increase the manual validation burden without removing the mismatch risk. A versioned feature contract and matched-request parity tests address the failure boundary first; a shared feature store becomes useful when maintaining that contract across pipelines warrants the additional infrastructure. Maturity assessment therefore selects the next missing control, rather than requiring every system to acquire the same components.

Figure 14.10 provides a coverage map for that assessment. Service uptime is visible at the tip, while data- and model-health failures can remain hidden below the waterline. For each failure mode, the assessment must identify the evidence that exposes it and the action that contains it.

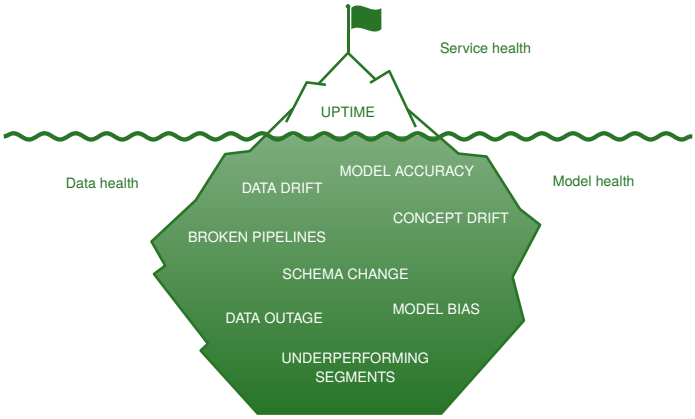


Figure 14.10: Uptime Dependency Stack: The waterline separates visible service uptime from data- and model-health failures that can remain hidden while the service stays available; the surrounding labels organize the monitoring surface into data, model, and service health.

A system that logs all three health domains can still lack a response path. If stale features are detected but no validated fallback exists, containment is the next investment. If quality outcomes are recorded without the served model version, attribution must be repaired before automatic promotion can rely on them. If a canary can exhaust the production worker's memory, resource isolation takes priority over another dashboard. Each finding identifies a specific capability to add and a failure against which to test it.

14.8.3 Design patterns and anti-patterns

Effective containment decisions require an owner with authority to change the affected system. A centralized MLOps team can maintain shared controls across models, while a federated model places engineers within product teams under shared architectural contracts. Either arrangement must resolve the same handoff: who validates a fallback, who can activate it, and who confirms recovery. Buying a feature store or alerting platform before settling that handoff is the tool-first anti-pattern.

Once ownership is explicit, the residual failure determines the technical protection. Broken inputs require a way to stop harmful predictions; competing model versions require resource isolation. The cost of these controls is justified by the exposure they contain, and their effectiveness must be tested against that specific failure.

A circuit breaker[26] automatically routes traffic away from a failing service when its error rate exceeds a threshold, and in ML serving it needs ML-specific trip conditions. Conventional circuit breakers trip when RPC timeouts or HTTP 5xx error rates exceed a set threshold. In an ML serving context, circuit breakers must also monitor invariants whose violation signals broken inputs rather than gradual drift: sudden shifts in feature bounds, missing-feature rates, or output distribution collapse. Gradual drift scores such as PSI open a diagnosis instead of tripping the breaker. When triggered, the circuit breaker bypasses the model to serve predictions from a deterministic heuristic fallback, a cached lookup table, or a simpler baseline model, preventing corrupted inferences from cascading into downstream transactional systems.

Similarly, bulkhead patterns[27] isolate candidate or canary models from primary serving paths.

[26] **Circuit breaker pattern**: Automatic failure detection that "opens" when error rates exceed configured thresholds, routing traffic away from failing services. In ML systems, the pattern requires a critical adaptation: prediction accuracy degradation demands different thresholds than service availability failures, because a model returning plausible but incorrect predictions triggers no error signal, leaving the circuit breaker blind to the most dangerous failure mode.

[27] **Bulkhead pattern**: This pattern partitions system resources to contain failures within isolated zones. In ML serving, a bulkhead allocates fixed accelerator memory, host RAM, and worker thread pools to experimental model versions, so a canary's resource exhaustion is less likely to terminate the primary serving path; the protection depends on the process and hardware isolation actually enforced.

On multi-tenant accelerator hardware, an unconstrained experimental model suffering an out-of-memory abort, memory leak, or thread-pool exhaustion can crash the shared serving runtime or stall the PCIe bus. Containerized memory limits, isolated worker queues, and supported hardware partitions can contain specific resource failures. Shared memory bandwidth, interconnects, and device-level faults still require explicit isolation analysis and testing.

Redundancy has a hard limit in ML serving. Replicating a model across workers or voting across ensemble members protects against crash faults, and agreement protocols can keep replicas consistent with one another, but agreement is not correctness. When every replica shares the same inductive biases or training-distribution blind spots, the replicas fail together and silently, and no amount of voting reveals the error before delayed ground truth arrives. Circuit breakers, bulkheads, and replication therefore strengthen an MLOps implementation only when each is applied to the failure class it can actually detect.

14.8.4 Contextualizing MLOps

Standard operational patterns must adapt to physical and organizational constraints. Every ML system operates within a specific context that shapes implementation: physical constraints (edge compute, power budgets), regulatory requirements (healthcare, finance), or organizational scale (team size, skill distribution). A standard CI/CD pipeline may be infeasible without direct host access; monitoring may require indirect signals or on-device anomaly detection; data collection may be limited by privacy regulations. These adaptations are expressions of maturity under constraint, not departures from foundational principles.

At the highest levels of operational maturity, single-model practices become building blocks for broader platform capabilities. Organizations operating many ML nodes simultaneously consolidate into platform architectures that provide shared infrastructure, centralized governance, and economies of scale. The transition from individual ML nodes to platform-scale infrastructure introduces qualitatively different challenges (cross-model resource allocation, system-level observability, and fault tolerance across interdependent pipelines) that extend beyond single-node operations. Sound ML node practices remain prerequisites for platform success because gaps in single-model monitoring, testing, or deployment multiply across the model portfolio.

14.8.5 MLOps investment economics

The operational benefits of MLOps become persuasive only when the investment matches the model's production value. This chapter has already priced each control separately. Training-serving skew cost a fixed fraction of daily queries (equation 14.2); automation paid back once manual hours overtook its build cost (notebook 14.1); retraining was worth a run only when recovered accuracy outweighed run cost and rollout risk (equation 14.3), with equation 14.8 setting its cadence; detection delay priced the gap between monthly and daily review (notebook 14.5); and monitoring earned its keep through the incidents it prevented (equation 14.14). An investment decision reconciles these separate ledgers within a single accounting boundary rather than naively summing them. Because an unmitigated skew event manifests simultaneously as lost prediction value, an extended silent failure, and a production incident, combining those ledgers directly would count the same underlying damage multiple times. Rigorous accounting establishes a common baseline and evaluation horizon, counts each prevented loss exactly once within a primary category, and separates routine deployment labor savings from emergency response hours already captured in incident costs. For a single ML node, the resulting ledger determines whether deployment speed, incident reduction, and monitoring coverage justify the operational spend; for a portfolio, the same discipline scales to justify platform investment.

14.8.5.1 Single-model MLOps investment

For a single production ML system, the first threshold is the annual cost of making the node observable, deployable, and recoverable. Table 14.25 summarizes the main cost categories:

Table 14.25: Single-Model MLOps Investment: Illustrative planning inputs for operationalizing one production ML system. Open-source tooling (MLflow, Feast) can reduce software costs; cloud-managed services trade higher unit costs for reduced engineering overhead.

Component	Illustrative Assumption	Justification
CI/CD pipeline setup	\$10–30K one-time	Reduces deployment time from days to hours
Monitoring and alerting	\$2–10K/year	Shortens detection and response delays
Feature store (basic)	\$5–20K/year	Reduces one source of training-serving skew
Model registry	\$0–5K/year	Enables rollback, audit trails
Engineering time	1–2 FTE-months setup	Initial automation and integration

14.8.5.2 Single-model ROI calculation

The return threshold then depends on model criticality: a revenue-facing model can justify more operational spend because avoided incidents and deployment-time savings have measurable value. Equation 14.15 formalizes that single-node calculation by extending the monitoring benefit-cost ratio of equation 14.14 with a second benefit term, the engineering labor that automation recovers:

$$\text{Annual Benefit/Cost} = \frac{\text{Incidents Avoided} \times \text{Avg Incident Cost} + \text{Time Savings} \times \text{Hourly Cost}}{\text{Annual MLOps Investment}} \tag{14.15}$$

where Incidents Avoided is the count of production failures the tooling prevents per year and Avg Incident Cost is the total loss per failure, so their product is the value of avoided incidents; Time Savings is the engineer-hours that automation reclaims per year from routine release and maintenance tasks (strictly distinct from emergency triage labor already subsumed under Avg Incident Cost), and Hourly Cost is the loaded labor rate, so their product is the value of recovered deployment labor; the denominator is the annualized investment, including recurring tooling and engineering costs plus setup costs amortized over the chosen planning horizon. The worked investment below applies this non-overlapping accounting boundary. The ratio expresses benefit per dollar invested, not net profit per dollar.

Consider a revenue-facing model with the following annual figures:

- 4 incidents/year avoided (at \$25K each) = \$100K saved
- 20 hours/month deployment time saved (at \$150/hr) = \$36K saved
- MLOps investment of \$30K/year

$$\text{Benefit/Cost} = \frac{\$100K + \$36K}{\$30K} = 4.53\times$$

14.8.5.3 When to invest more

The 4.53× benefit-cost ratio means the investment is not justified by tooling elegance; it is justified because the model is expensive enough that preventing incidents and shortening deployments outweigh the annual platform spend. The returns from single-model MLOps practices compound when teams add additional models. The transition from operating several independent ML nodes to building a centralized platform involves different economics entirely, including shared infrastructure amortization, platform team overhead, and cross-model coordination costs.

For single-model operations, invest in MLOps in proportion to model criticality. A model driving \$10M in annual revenue justifies more operational rigor than an internal analytics model. Monitoring, lineage, and repeatable deployment are often the first investments. Add a feature store when shared feature definitions and online-offline parity become material problems, and automate retraining only when evidence supports the trigger and the validation loop can contain a bad update.

The choice of operational controls also depends on what a deployment can measure and change. The architectural studies in section 14.9 contrast an edge design grounded in published wearable evaluation with a proposed clinical framework. Battery limits and clinical authority reshape how each system preserves evidence, validates updates, and responds to failure.

Checkpoint 14.5: Choosing the next investment

- ☐ **Accounting boundary**: Identify the setup, recurring tooling, and engineering costs needed to interpret an annual benefit-cost ratio.
- ☐ **Priority**: For a single model with unowned alerts and a tested deployment pipeline, justify the next operational investment before selecting additional tools.

14.9 Case Studies

Operational constraints in machine learning systems are governed by the physical envelope of the serving hardware and the failure consequences of the application. A battery-powered wearable operates under milliwatt power budgets, constrained on-device SRAM, and narrow telemetry bandwidth over Bluetooth Low Energy. In contrast, regulated AI-enabled medical devices require evidence and change controls appropriate to their intended use. FDA guidance for predetermined change control plans describes planned modifications, validation methods, and impact assessment (U.S. Food and Drug Administration 2025). These disparate environments dictate how engineering teams manage data pipelines, model artifacts, and hardware execution across the lifecycle. Edge architectures spend their engineering budgets bridging the hardware asymmetry between high-throughput cloud training clusters and microcontrollers, whereas clinical systems spend that budget on provenance tracking, cohort monitoring, and human-in-the-loop validation gates.

The Oura-inspired case study in section 14.9.1 models these edge constraints using published offline evaluation data, serving as an architectural study rather than an audit of proprietary production over-the-air pipelines. In this edge workflow, reference sleep labels derive from polysomnography (PSG), the laboratory-grade multichannel physiological recording used as the clinical reference for sleep staging. Despite their contrasting operating regimes, both edge and clinical deployments must satisfy the same core MLOps disciplines. Table 14.26 maps these foundational principles across the two environments, demonstrating how physical hardware limits and regulatory mandates alter implementation mechanisms without altering the underlying operational requirements.

Table 14.26: MLOps Principles by Case Study: Side-by-side mapping of the five foundational MLOps principles to a hypothetical Oura-inspired edge design and the ClinAIOps framework, showing how domain constraints reshape implementation without changing which principles apply.

Principle	Oura-inspired edge design	ClinAIOps
Reproducibility	Versioned synchronized wearable and PSG datasets	Audit trails, decision provenance
Separation of concerns	Independent data, training, and serving layers with edge-specific deployment pipeline	Distinct clinical validation and deployment stages with regulatory compliance isolation
Consistency	PSG-aligned preprocessing across training and on-device inference	Standardized clinical data pipelines ensuring training-serving parity
Observable degradation	On-device anomaly detection, limited telemetry	Cohort-specific monitoring, outcome tracking
Cost-aware automation	Battery-aware retraining triggers, CI/CD for edge balancing accuracy and resource cost	Automated model updates with human-in-the-loop gates balancing update cost and patient risk

14.9.1 Oura-inspired edge design

The Oura Ring provides the empirical foundation for this edge MLOps case study. The published clinical study validates data collection and multi-sensor sleep-stage evaluation (Altini and Kinnunen 2021), while the versioning, telemetry, over-the-air deployment, and lifecycle controls represent a hypothetical architecture rather than a reported account of Oura's proprietary production system. Operating within the milliwatt power envelope and strict memory limits of a wearable ring makes every operational trade-off visible in ways that cloud-scale systems obscure.

14.9.1.1 Context and motivation

As a miniature consumer wearable, the ring captures continuous physiological signals: accelerometry for movement, optical photoplethysmography (PPG) for pulse rate and heart-rate variability, and

negative temperature coefficient thermistors for skin temperature. The published study evaluates four sleep stages—wake, light, deep, and rapid eye movement (REM) sleep—from wearable signals. For the hypothetical operational design here, a miniature 15–22 mAh battery motivates limiting continuous Bluetooth Low Energy (BLE) streaming. The proposed architecture partitions computation across three tiers: on-ring sensor preprocessing and lightweight activity classification executing in microcontroller firmware, companion-phone inference executing the heavier multi-sensor sleep-stage sequence model during periodic synchronization, and cloud-scale historical aggregation and retraining.

The central objective was improving sleep-stage classification accuracy to align with PSG,[28] the clinical gold standard. Accelerometer-only models achieved 57 percent four-stage sleep classification accuracy because movement signals alone cannot separate quiet wakefulness from light or REM sleep. With expert human PSG scorers agreeing with each other at about 82 percent to 83 percent, the performance gap between single-modality motion sensing and clinical consensus exposed the physical limits of accelerometry. Incorporating autonomic nervous system signals (heart rate and heart-rate variability) alongside circadian temperature features lifted classification accuracy to 79 percent. The resulting 22 percentage-point gain closes roughly 84.6–88 percent of the baseline-to-human-agreement gap, although human inter-scorer agreement reflects label ambiguity rather than a strict theoretical ceiling against an adjudicated reference.

[28] **Polysomnography (PSG)**: A multi-parameter sleep study that provides the clinical ground truth data for this classification task. This "truth" is inherently noisy; expert human scorers interpreting the same PSG recordings agree with each other at about 82 percent–83 percent reliability (Altini and Kinnunen 2021). Inter-scorer agreement indicates label uncertainty but does not impose an accuracy ceiling against a fixed or adjudicated reference.

14.9.1.2 Model development and evaluation

Building this multi-sensor pipeline required grounding wearable telemetry in clinical standards through a study involving 106 participants from three continents (Altini and Kinnunen 2021). Each participant wore the Oura Ring while simultaneously undergoing laboratory PSG, yielding 440 nights of data and 3,444 hours of time-synchronized recordings that aligned raw wearable sensor streams with validated sleep annotations. The scale and demographic diversity of the collection captured physiological variation across age, sex, and baseline health, as well as behavioral differences essential for generalizing across a real-world user population.

Consolidating synchronized accelerometer, temperature, optical PPG, and clinical sleep records resolved temporal alignment and feature-engineering specifications. A production edge workflow requires strict dataset versioning and lineage tracking across sensor hardware revisions, sampling rates, and digital filtering routines to prevent training-serving skew between Python development pipelines, embedded C routines executing on the ring's microcontroller, and the phone-resident inference runtime.

Once synchronized clinical data was established, offline model exploration quantified the contribution of each additional sensing modality. Evaluating four feature configurations across 5-fold cross-validation confirmed that combining temperature, heart-rate variability, and circadian periodicity lifted four-stage classification accuracy to 79 percent, up from the 57 percent accelerometer-only baseline (Altini and Kinnunen 2021). While these offline gains demonstrate the value of multi-sensor fusion, deploying them across resource-constrained edge tiers—partitioning feature extraction on the ring and sequence inference on the companion phone—introduces operational requirements outside the original study: reproducible model quantization, numerical conversion validation across both execution environments, and strict runtime energy budgets.

14.9.1.3 Key operational insights

Following offline validation, the primary systems challenge shifts from statistical accuracy to deployment safety and runtime efficiency. In the proposed edge architecture, execution is partitioned across hardware boundaries: lightweight filtering and movement classification run continuously in microcontroller SRAM, while the heavier multi-stage sleep-staging sequence model executes during phone synchronization. To maintain pipeline integrity across thousands of physical devices, the MLOps toolchain requires automated quantization pipelines for phone-hosted models (such as converting float32 weights to int8 with calibrated scale factors), deterministic cross-compilation for ring-resident firmware routines, cryptographically signed release packages for each hardware target, and fault-tolerant OTA[29] delivery mechanisms.

Edge MLOps operates under physical constraints distinct from server-side pipelines: model accuracy must be co-designed with strict battery life, restricted radio bandwidth, on-device storage

[29] **Over-the-Air (OTA) Updates**: The mechanism used to deploy optimized models to devices already in the field without physical access. Minimizing artifact footprint through quantization and pruning directly reduces Bluetooth transfer energy and radio transmission time. Atomic update staging and rollback routines ensure that an interrupted wireless transfer or checksum mismatch cannot corrupt on-device flash or leave the device in an unrecoverable state.

bounds, and the near-total absence of real-time ground-truth labels. In the DS-CNN (Tiny Constraint) archetype from table 14.4, health monitoring relies on operational proxies—sensor duty cycles, inference latency anomalies, and classification confidence distributions—rather than verified outcomes. Sustaining the validated multisensor gain (from 57 percent accelerometer accuracy to 79 percent multi-sensor accuracy) in the field requires rigorous configuration management across execution targets: tracking ring-resident sensor firmware and DSP preprocessing routines alongside the quantized sequence model weights deployed to the companion phone.

These physical constraints map directly to the five foundational MLOps principles introduced in table 14.26. Versioned wearable and PSG datasets provide reproducibility by linking every deployed model artifact to the exact laboratory recordings and sensor firmware calibrations that trained it. Tiered software architecture enforces separation of concerns, isolating ring-resident sensor drivers and feature extractors from phone-hosted inference engines and cloud aggregators, so that microcontroller firmware, model topologies, and battery-saving fallbacks can be updated independently. Shared feature-extraction kernels reduce implementation divergence between offline training and microcontroller execution; parity tests must verify numerical agreement between Python prototypes, ring DSP routines, and the phone-resident inference engine across sensor firmware versions. Privacy-preserving telemetry maintains observable degradation by transmitting aggregate operational diagnostics—daily sensor duty cycle, battery drain, inference exception counts, and classification entropy—without streaming raw, sensitive biometrics off the user's device. Finally, over-the-air deployment defines the boundary for cost-aware automation: every firmware patch for the ring or model update for the companion phone must demonstrate that its accuracy gain outweighs the battery cost of radio reception, flash rewrite cycles, and the operational risk of altering software on a wearable operating continuously on the user.

The wearable edge illustrates how physical resource limits—milliwatts, kilobytes, and radio bandwidth—dictate MLOps design. When machine learning transitions from personal wellness into regulated healthcare, the primary engineering constraint shifts from hardware physics to patient safety, clinical validation, and algorithmic accountability—the operational domain of ClinAIOps.

14.9.2 ClinAIOps case study

On wearable consumer hardware, the engineering envelope is bounded by milliwatts, memory footprint, and radio bandwidth. When machine learning transitions from fitness tracking to regulated medical care, the governing constraint shifts from hardware physics to patient safety and clinical liability. In continuous therapeutic monitoring (CTM),[30] wearable sensors stream physiological telemetry to guide personalized medical interventions. General-purpose MLOps frameworks provide lifecycle automation—orchestrating pipelines, tracking artifacts, and monitoring availability. Clinical use adds intended-use-specific evidence, oversight, and authorization requirements because predictions may influence treatment. General-purpose MLOps can incorporate human gates; the clinical framework makes their responsibilities explicit.

[30] **Continuous therapeutic monitoring (CTM)**: Healthcare approach using wearable sensors for real-time physiological data collection and personalized treatment adjustments. CTM forces MLOps to confront constraints absent in typical deployments: feedback loops require oversight appropriate to the intended use, training labels need clinical validation, and model changes must follow applicable authorization and change-control requirements. These constraints reshape every MLOps principle, making CTM a stress test for operational maturity.

The proposed ClinAIOps framework (Chen et al. 2023) adapts MLOps principles to these constraints by formalizing the boundaries between algorithmic inference and clinical authority. Rather than treating human oversight as an external impediment, ClinAIOps integrates multi-stakeholder feedback loops directly into the serving architecture. The framework partitions operational control across three actors—patients, clinicians, and machine learning engineers—each operating on distinct timescales with explicit safety gates.

14.9.2.1 Feedback loops

Three interlocking feedback loops organize the proposed integration of machine learning into clinical practice; their effectiveness and safety require validation in the intended setting. Figure 14.11 models these three interlocking feedback loops as a triangular protocol among the patient, clinician, and AI developer. Pairwise bidirectional channels enforce distinct operational contracts: the patient streams wearable sensor telemetry and receives clinician-approved therapy alerts; the AI developer provides longitudinal monitoring summaries while accepting physician-defined dosing bounds; and the clinician directly governs patient therapy regimens while establishing safety boundaries for algorithmic actuation. The outer cycle connecting all three nodes forms the overarching clinical governance framework.

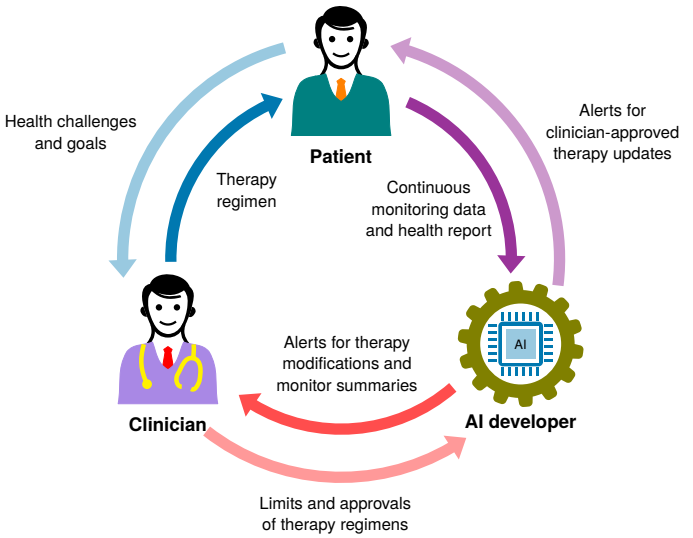


Figure 14.11: ClinAIOps Feedback Loops: The cyclical framework coordinates patients, clinicians, and AI developers to support continuous model improvement and safe clinical integration. Patients and clinicians use AI outputs in care workflows, while AI developers receive feedback from both groups to refine models and operations. Source: (Chen et al. 2023).

Each feedback loop operates on a distinct timescale to balance responsiveness against clinical verification. The **patient treatment loop** executes continuously in real time, capturing sensor telemetry and delivering bounded adjustments for patient self-management. At an asynchronous supervisory cadence, the **clinician oversight loop** reviews aggregated trends, sets operational boundaries, and validates treatment recommendations before interventions occur. Finally, the **developer feedback loop** aggregates telemetry and clinician override patterns back to engineering teams, driving offline retraining and interface calibration.

Patient treatment loop The patient treatment loop executes localized inference over high-frequency physiological streams captured by wearable devices, such as continuous glucose monitors or optical pulse sensors. The edge serving runtime evaluates these incoming feature vectors against recent patient history from electronic health records.

In this proposed design, recommendations follow a tiered execution model as one safety control. Minor adjustments that fall strictly within clinician-defined bounds are surfaced directly to the patient. Significant changes—or recommendations generated when model uncertainty exceeds an acceptable margin—are blocked from automated delivery and escalated to the clinician. This structure enables rapid adaptation to dynamic physiological states within validated, authorized operating limits; those limits and the escalation mechanism must be tested.

Clinician oversight loop The clinician oversight loop provides supervisory control over automated inference. Rather than burdening medical staff with continuous raw sensor telemetry, batch aggregation pipelines distill time-series streams into longitudinal summaries, such as circadian variance, multi-day trends, and signal-quality metrics.

When the system generates a therapy recommendation—such as adjusting an antihypertensive dosage for a patient with persistent hypotensive readings—the clinician evaluates the proposal alongside diagnostic factors unobserved by the wearable model. The clinician may approve, modify, or reject the recommendation. This decision is written to an audited event log, serving an operational dual role: it enforces clinical accountability at the point of care and records expert feedback for later review. Approval or override is not by itself a verified patient outcome; using it as training supervision requires a defined label interpretation and validation. Clinicians also update the operational thresholds that govern the patient treatment loop, recalibrating the boundary between automated guidance and mandatory review.

Developer feedback and patient-clinician coordination Offloading continuous data logging and low-level trend detection to the AI pipeline changes the patient-clinician relationship itself. Consultations shift from routine measurement gathering to targeted clinical evaluation and goal setting, an outcome the patient and clinician loops produce together. The third loop, developer feedback, links that clinical practice back to engineering infrastructure.

Operational telemetry generated during these interactions provides engineering teams with direct visibility into production behavior. Overrides, rejection frequencies, and reported adverse events motivate investigation of model behavior, workflow fit, patient mix, and sensor quality. They do not uniquely diagnose covariate shift. Engineers use the resulting evidence to evaluate targeted retraining, refine data validation schemas, and eliminate usability failure modes that induce alert fatigue in clinical staff.

14.9.2.2 Hypertension case example

Hypertension management illustrates how these three operational loops could coordinate care in the proposed framework. Blood pressure fluctuates continuously under exertion, stress, and circadian cycles, requiring individualized, long-term pharmacological adjustments that benefit from continuous therapeutic monitoring.

Data infrastructure Research systems estimate systolic blood pressure indirectly from electrocardiography (ECG), photoplethysmography (PPG),[31] pulse-transit time, and heart-rate features (Q. Zhang et al. 2017). In the illustrative workflow, these signals are augmented by accelerometer streams to identify motion artifacts and self-reported medication logs. Clinical use requires rigorous calibration and prospective validation for the intended task, together with any applicable regulatory authorization; authorization alone does not establish accuracy for every cohort. When properly validated for the target cohort, this multimodal data stream is merged with electronic health records to supply feature context for inference.

[31] **Photoplethysmography (PPG)**: Optical technique that infers blood-volume changes from variations in detected light after illuminating tissue. For ML operations, PPG introduces a data quality challenge absent in controlled environments: motion artifacts from wrist movement corrupt the signal, creating a data drift pattern where the same physiological state produces different input distributions depending on user activity. Models must either filter corrupted windows before inference or learn to be robust to motion noise, and monitoring must distinguish genuine physiological changes from artifact-induced distribution shift.

Loop implementation The hypertension workflow labels each of its three panels by the two parties in the exchange (figure 14.12), so the panels do not line up one-to-one with the three loops of figure 14.11. The patient-AI panel is the patient treatment loop, the clinical-AI panel is the clinician oversight loop, and the patient-clinical panel is the coordination those two loops produce together; the developer feedback loop operates out of band, as described in section 14.9.2.1, and does not appear. Across the top of the figure, the two AI-assisted loops operate concurrently: the patient-AI panel (upper left) captures continuous blood-pressure telemetry and issues recommendations for bounded dose titrations, while the clinical-AI panel (upper right) alerts the physician to severe hypertensive excursions and accepts updated titration boundaries. Below them, the patient-clinical panel captures the resulting clinical relationship, where routine telemetry offloaded to the AI loop frees consultation appointments for longitudinal trend review, adverse-event evaluation, and shared lifestyle goal setting.

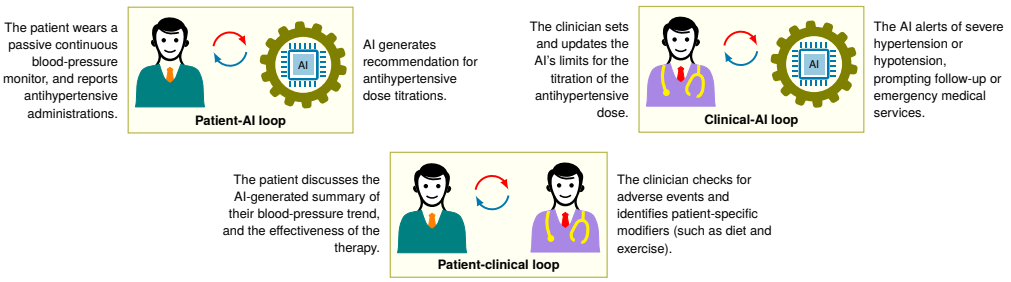


Figure 14.12: Hypertension Management Loops: The three panels, labeled patient-AI, clinical-AI, and patient-clinical, each carry a two-way exchange. In this proposed workflow, the AI issues bounded dose recommendations within validated, authorized clinician-set limits and escalates severe readings, which leaves the patient-clinical panel for trend review, adverse-event checks, and treatment decisions. The developer feedback loop of ClinAIOps is not among them and is discussed in the prose. Source: (Chen et al. 2023).

The three panels illustrate a proposed boundary for clinical automation, not an established treatment protocol: routine monitoring and bounded dose recommendations operate within vali-

dated, authorized clinician-defined thresholds, while acute escalations, adverse events, and regimen changes require explicit human authorization. That boundary is the point where ordinary MLOps practices require the additional clinical coordination summarized next.

14.9.2.3 MLOps vs. ClinAIOps comparison

The hypertension case illustrates the operational contrasts between standard software pipelines and healthcare deployments. General-purpose MLOps provides technical lifecycle controls—automated testing, continuous integration, model registries, and drift monitoring. ClinAIOps extends this foundation into regulated sociotechnical environments where outputs directly influence medical interventions. Table 14.27 compares the two approaches across eight operational dimensions.

Table 14.27: Clinical AI Operations: General-purpose MLOps supplies technical lifecycle controls; ClinAIOps extends them with clinical workflows, evidence, human oversight, and accountability.

	General-purpose MLOps emphasis	ClinAIOps extension
Focus	Technical model lifecycle	Human and AI decision-making
Stakeholders	ML, data, platform, and operations teams	Adds patients, clinicians, and clinical governance
Feedback loops	Monitoring, retraining, and release	Adds treatment, clinician, and developer feedback
Objective	Reliable, governed ML operations	Safe, effective, and accountable care support
Processes	Automated pipelines and operational controls	Integrates clinical workflows and human gates
Data considerations	Lineage, quality, freshness, and access control	Adds consent, clinical provenance, and protected data
Model validation	Predictive, operational, and slice-level metrics	Adds clinical utility, safety, and cohort outcomes
Implementation	Technical integration and operational ownership	Adds clinical accountability and stakeholder incentives

The table's central distinction is that clinical deployment alters risk ownership. Statistical accuracy is a prerequisite, but it does not protect against liability when an erroneous prediction drives a medical intervention. The ClinAIOps framework therefore changes the governing constraint from device efficiency to clinical accountability. The model participates in care delivery, but it cannot own clinical decisions. Every recommendation and subsequent clinician action must be traceable to the underlying sensor inputs, model version, and inference metadata; otherwise the system cannot support regulatory audit or retrospective incident analysis. Separation of concerns shifts from an architectural preference to a safety invariant: sensor ingestion, inference scoring, clinical diagnosis, and parameter updates must be decoupled by explicit software contracts and human approval gates.

The same accountability requirement transforms monitoring. Standardized clinical data pipelines support training-serving parity, but clinical validation requires evaluating model recommendations against standard-of-care outcomes and cohort-specific effects. Observable degradation extends beyond statistical loss and latency to clinical metrics: blood pressure control rates, adverse events, clinician overrides, and subgroup disparities. In this regime, multi-tier feedback loops are not unwanted operational latency; they are deliberate architectural controls that balance automated edge adaptation against legal and ethical accountability. Cost-aware automation operates strictly inside those gates: model refreshes occur only when validated clinical benefits outweigh validation costs and patient risks, and low-confidence predictions trigger deterministic fallback to human clinical review.

14.9.3 Case study synthesis

The Oura-inspired design and ClinAIOps cases separate stable MLOps principles from the deployment constraints that reshape their implementation. The hypothetical ring lifecycle is a resource-envelope case: the operational system must preserve reproducibility, consistency, and observable degradation while battery, telemetry, and weak ground truth limit what can be measured and updated on the device. ClinAIOps is an accountability-envelope case: the same principles apply, but validation, audit trails, and human gates dominate because the model influences clinical action.

The shared engineering lesson is that MLOps maturity is not tool accumulation. It is the ability to identify the governing constraint, choose the operational controls that match it, and preserve

evidence when the model changes. Production ML systems fail when teams apply code-focused operational intuitions without accounting for statistical behavior and shifting distributions, exposing common fallacies and operational pitfalls.

Checkpoint 14.6: Transferring the operational principles

- ☐ **Evidence and design**: Separate the wearable study's reported offline accuracy from the hypothetical design's deployment and monitoring controls.
- ☐ **Feedback quality**: Explain why human scorer agreement is not a strict accuracy ceiling and why a clinician override is not automatically a verified patient outcome.
- ☐ **Shared principle**: Choose one MLOps principle and explain how battery limits and clinical accountability lead to different implementations of it.

14.10 Fallacies and Pitfalls

Operating production machine learning systems under conventional software assumptions creates distinct failure modes. The following fallacies and pitfalls highlight where standard software engineering intuitions break down against the statistical realities of deployed models.

Fallacy: *MLOps is just applying traditional DevOps practices to machine learning models.*

Engineers often assume that standard CI/CD pipelines transfer directly to machine learning. Traditional DevOps automates code builds, testing, and artifact deployment with controls for dependencies and execution environments. In an ML system, code represents only one axis of the D·A·M triad. Model execution depends jointly on code, large evolving datasets, and accelerator compute budgets. As detailed in section 14.4.2.1, ML pipelines introduce stateful retraining, multi-metric validation gates, and immutable artifact registration. Standard CI/CD engines cannot natively track data lineage, orchestrate feature stores, or detect distribution drift. When teams apply DevOps without ML-specific adaptations, they maximize the computational availability of their serving cluster while leaving statistical behavior unmonitored. The system achieves high uptime and low latency, yet silently serves degraded predictions due to unvalidated data shifts and training-serving skew.

Pitfall: *Treating model deployment as a one-time event rather than an ongoing process.*

Treating deployment as a terminal milestone assumes that deployed weights remain permanently aligned with the production input distribution. Unlike compiled binaries whose execution semantics remain fixed, model quality can change as real-world distributions shift. Monitoring statistical indicators such as PSI (section 14.6.2.1) provides an early signal of feature drift, distinguishing benign variance from actionable distribution change. Crossing a warning threshold requires operational diagnosis rather than unvalidated retraining. As established in section 14.4.2.2, scheduling updates is an economic balancing problem: the optimal retraining interval $T^* \approx \sqrt{\frac{2C}{Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}}$ weighs the fixed compute and engineering cost C against the business losses accumulated from daily query volume Q_{day}, unit accuracy value V, and staleness decay rate γ. Operating a production model demands treating deployment as a continuous control loop that balances monitoring signals against retraining economics.

Fallacy: *Automated retraining ensures optimal model performance without human oversight.*

Automating pipeline execution does not eliminate the need for operational governance. Closed-loop retraining pipelines operate without semantic awareness; if upstream data ingestion suffers schema anomalies, corrupted labels, or sudden sensor drift, automated retraining bakes those errors directly into updated model weights. Furthermore, automated validation gates typically evaluate aggregate loss across a held-out test split, which easily conceals severe performance regressions on critical slices or low-frequency edge cases. For instance, retraining a ranking model on weekend traffic can degrade performance across latency-sensitive weekday workloads where user query distributions diverge. Resilient MLOps architectures decouple automated pipeline execution from automated

production promotion, enforcing canary deployments, slice-level regression gates, and human sign-off when validation metrics deviate from expected historical envelopes.

Pitfall: *Focusing on technical infrastructure while neglecting organizational and process alignment.*

Investing in MLOps tooling without defining organizational interfaces leads to fragmented operational ownership. Machine learning systems bridge two distinct engineering cultures: model developers optimizing offline accuracy metrics and platform engineers managing host-level latency, memory capacity, and throughput budgets. When organizations treat deployment as a handoff between isolated teams, root-cause triage stalls during production incidents. Infrastructure engineers inspect container health and GPU utilization without insight into model prediction distributions, while model authors inspect offline training curves without understanding runtime serving constraints. Tools such as feature stores and registries provide technical scaffolding, but reliability requires institutional alignment: unified SLOs spanning statistical accuracy and serving latency, joint on-call runbooks, and shared accountability for production regressions.

Fallacy: *Training and serving environments automatically remain consistent once pipelines are established.*

Establishing an initial end-to-end pipeline does not guarantee permanent parity between training and serving. Training-serving skew arises from fundamental architectural divergences between offline and online execution paths. Training typically runs as an offline batch workflow (such as distributed SQL queries or Spark pipelines) optimized for high-throughput scans over historical logs, whereas online serving runs in low-latency microservices under millisecond latency constraints. Maintaining dual implementations of feature transformations across these environments risks numerical disparities, differing missing-value semantics, or lookahead bias in historical joins. As demonstrated in section 14.4.1.2, feature stores alleviate this divergence by unifying transformation logic and synchronizing batch and point-lookup stores. However, software pipelines alone cannot guarantee consistency; operational verification requires request-keyed value comparisons and schema assertions between offline replays and online inference inputs, alongside separate distribution monitoring to catch divergence before it degrades downstream decisions.

Pitfall: *Assuming comprehensive monitoring prevents all production incidents.*

Dashboards and alert thresholds cannot prevent production outages caused by silent data failures. Monitoring supplies evidence; reducing harm also requires a response. Input-boundary checks can catch some defects before inference, while statistical quality estimates may arrive after affected predictions have been served. Furthermore, tracking output metrics alone leaves critical blind spots along the data pipeline. As shown in section 14.6.2.1, measuring service latency and prediction distributions fails to detect input anomalies such as feature staleness. An inference service can maintain nominal p99 response times and normal prediction ranges while consuming stale user embeddings caused by upstream database replication lag or cache eviction failures. By the time downstream business metrics degrade, corrupted predictions have already impacted users. Production reliability requires defensive boundary controls alongside passive observability: strict input schema enforcement, feature freshness assertions, and circuit breakers that fall back to deterministic heuristics when input quality slips below operational tolerances.

Fallacy: *Accuracy is the first production signal to monitor.*

Teams often configure accuracy dashboards as their primary operational tripwire, assuming degradation will register there first. When outcome labels arrive late, ground-truth accuracy is a lagging indicator. In production environments such as fraud detection, ad conversion, or credit underwriting, ground-truth labels arrive with substantial latency—often requiring weeks or months to accumulate through chargebacks or repayment histories. Relying solely on accuracy dashboards leaves the system operating blind throughout this label-delay window. Furthermore, aggregate accuracy can remain deceptively stable even as input distributions diverge across critical sub-populations. As detailed in section 14.6.2.1, monitoring input feature distributions with metrics like PSI or Kullback-Leibler (KL) divergence supplies indicators that can be estimated from sufficient input samples without labels. These statistical signals detect distribution shifts without waiting for labels, often enabling investigation before delayed outcome metrics are available, without guaranteeing that prediction errors have not already occurred.

Pitfall: *Routing leading-indicator alerts without a defined owner or response policy.*
Leaving leading-indicator alerts without an accountable response path undermines their operational value. When engineering teams route distribution drift and data freshness warnings to informational dashboards or low-priority channels without an owner, response deadline, or escalation rule, early warnings can be ignored. A lower-priority channel can be appropriate when severity is calibrated and responsibility remains explicit. Unowned alerts can accumulate unnoticed, leaving harmful changes unaddressed until an emergency response becomes necessary. A leading indicator provides operational utility only when tied to an active, owned response path with calibrated severity levels and explicit diagnostic runbooks. The operational objective is to use leading statistical signals to trigger root-cause diagnosis—verifying feature pipeline integrity and validating input schemas—ensuring that accuracy metrics serve to confirm operational health rather than announce an unmanaged failure.

Observability in production machine learning extends far beyond passive dashboard telemetry. It requires closed operational loops across the entire D·A·M architecture: enforcing input data integrity and feature stability, bounding model behavior through automated validation gates, and routing leading indicators directly to engineers empowered with diagnostic runbooks to investigate changes and limit harm.

14.11 Summary

MLOps extends software reliability with controls for learned behavior, data dependencies, and statistical uncertainty. A crashed server or network timeout turns availability dashboards red immediately, alerting operators to take corrective action. A degraded model introduces a silent statistical failure mode: the serving infrastructure returns 200 OK responses within latency deadlines, yet prediction quality can deteriorate undetected. Machine learning operations keeps that decay observable by treating model quality as a dynamic runtime variable rather than a static deployment artifact.

The five foundational principles (section 14.2.1) provide an evaluation framework that applies regardless of scale or domain. Reproducibility through versioning addresses the root cause of many production incidents: untracked artifacts (data versions, configuration shifts, and environment drift) that hinder reconstruction and make rollbacks unreliable. Separation of concerns contains the blast radius when changes occur, reducing the coupling risks through which boundary erosion and correction cascades turn local fixes into system-wide regressions. The consistency imperative targets training-serving skew, the silent accuracy erosion that occurs when feature computation diverges between offline training pipelines and online serving systems; feature stores centralize feature definitions and serving paths, though parity still requires continuous validation. Observable degradation transforms the abstract silent-failure problem into actionable alerts through layered monitoring that tracks data freshness, feature distributions, model outputs, and business metrics. Finally, cost-aware automation replaces arbitrary retraining schedules with a fitted staleness cost model ($T^* \approx \sqrt{\frac{2C}{Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}}$) whose practical application depends on its underlying assumptions.

Infrastructure components implement these principles across three critical operational boundaries. Feature stores and data versioning govern the data-model interface by reducing training-serving inconsistency. CI/CD pipelines and model registries govern the model-infrastructure interface by enforcing reproducibility and enabling rollback. Monitoring systems, incident response frameworks, and on-call practices govern the production-monitoring interface by making degradation observable and actionable. The retraining decision framework enables cost-aware automation by connecting measured degradation to economic thresholds. Domain constraints reshape *how* these principles are implemented without changing *which* principles matter: the Oura-inspired design uses the study's 57 percent to 79 percent offline accuracy improvement to motivate resource-aware validation and update controls rather than claiming an unverified production lifecycle. Similarly, ClinAIOps demonstrates why high-stakes clinical deployments require graceful-degradation and human-oversight controls, with three feedback loops functioning as architectural patterns rather than operational overhead.

Key Takeaways: Perfectly available, perfectly wrong

- **ML systems fail silently without outcome monitoring**: Model quality may degrade as distributional divergence $\mathcal{D}(P_t \| P_0)$ grows, although divergence alone does not determine accuracy loss. Because a serving node can maintain perfect uptime while predictions fail, tracking ground-truth outcomes is essential rather than relying on server health checks alone.
- **Training-serving skew corrupts inference without crashing**: Feature stores mitigate skew by unifying feature definitions and transformations across environments, but maintaining parity still requires continuous validation across online and offline data pipelines.
- **Retraining is an engineering optimization, not a guess**: The fitted staleness cost model $(T^* \approx \sqrt{2C/(Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma)})$ balances retraining compute against prediction quality decay, connecting update cadence to quantitative economics under its stated assumptions.
- **Deploy through graduated rollout with pretested rollback**: Canary, blue-green, and shadow deployments partition release risk across production traffic, backed by tiered rollback procedures that must be verified through regular operational drills.
- **Stage infrastructure investments by system risk**: Instrumentation and continuous deployment form the initial operational baseline. High-consequence systems justify greater operational rigor; feature stores and automated retraining should be added as skew and scale demand.
- **The five principles transfer across domains**: Reproducibility, separation of concerns, consistency, observable degradation, and cost-aware automation remain invariants across applications; domain constraints reshape only their concrete mechanisms and tool implementations.
- **Operational maturity is organizational as well as architectural**: Managing a single model differs qualitatively from governing a production fleet. While the core principles scale, operational complexity grows with fleet size, making unified incentives and shared on-call rotations as essential as tooling.

This operational discipline distinguishes production ML engineering from development experimentation. When a deployed model degrades, systematic instrumentation enables practitioners to isolate the failure mode: identifying data drift through distribution divergence checked against labeled outcomes (principle 10), training-serving skew through preprocessing parity audits (principle 11), configuration debt through artifact provenance, and feedback loops through temporal output tracking. Treating production ML as a "deploy and forget" exercise ignores these distinct dynamics.

This operational reality roots ML engineering in the D·A·M taxonomy. In an ML system, code, weights, and accelerator hardware can remain unchanged while the data axis moves beneath them, changing the predictive behavior that the system must validate. That exposure is the price of the bitter lesson. Behavior learned from data rather than written by hand is what lets these systems outperform hand-built rules, and it is also what lets their behavior change whenever the data does, with no line of code recording the change. The match between model and world is not an engineering state reached once at deployment, but an ongoing cost paid continuously through telemetry, validation gates, and cost-calibrated retraining. System reliability must therefore measure prediction outcomes rather than server uptime alone, because the most dangerous failure mode in production is a green dashboard masking a drifting model.

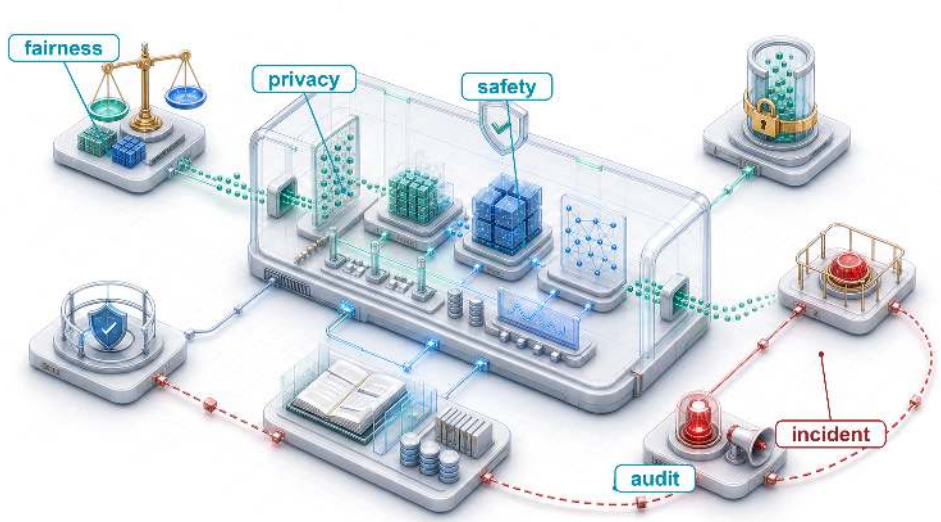
What's Next: From reliability to responsibility

What if the model never drifts and still does harm? Operations keeps a model faithful to its objective as the world shifts, but it cannot tell whether that objective was the right one. A system that tracks its training target perfectly can still reproduce historical bias, misallocate errors

across groups, or mishandle personal data, while every reliability check and aggregate accuracy metric stays green. Chapter 15 takes up that specification gap, extending the monitoring discipline of this chapter from predictive quality to the outcomes a system produces for the people it affects.

15

Responsible Engineering

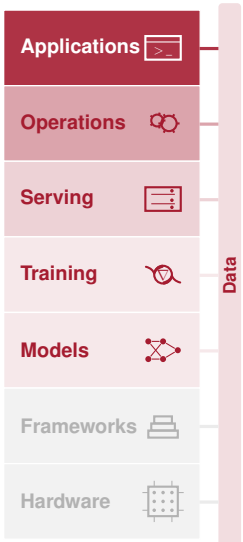


15.1 Responsibility as Systems Engineering
15.2 Engineering Responsibility Gap
15.3 Responsible Engineering Across the Lifecycle
15.4 Environmental and Cost Awareness
15.5 Data Governance and Compliance
15.6 Fallacies and Pitfalls
15.7 Summary

Purpose

Why can a system that does exactly what it was told to do still cause harm?

Operations targets low latency, high availability, and sustained predictive quality. Responsible engineering asks whether those targets describe the right behavior, whom the system serves, and which costs its specification leaves unmeasured. An ML system can satisfy latency, throughput, and aggregate accuracy requirements while reproducing historical discrimination, rewarding harmful engagement, consuming unjustified energy, or using data without adequate privacy and accountability controls. Such a system has not malfunctioned; it is efficiently optimizing an incomplete specification. The omitted constraints appear as uneven outcomes across populations, lifetime cost and emissions, decisions people cannot understand or contest, and evidence that cannot be reconstructed after harm occurs. If those consequences are not defined as requirements before deployment and monitored afterward, conventional health checks can remain green while harm accumulates. A credit model that treats groups unevenly can repeat that decision millions of times before anyone aggregates the outcomes, while its dashboards report healthy latency and accuracy throughout. Responsible engineering treats such outcomes as system failures to be diagnosed, measured, and mitigated with the same rigor as latency or accuracy regressions. That work requires translating broad obligations into testable bounds, documented assumptions, traceable data, and response paths that remain enforceable in production, so that a harm found after launch can be traced and corrected. In D·A·M terms, responsibility broadens the definition of correctness: data must be examined for encoded harms and lawful use, algorithms must be bounded by defensible outcome and efficiency requirements, and machine infrastructure must monitor, document, and enforce those boundaries throughout the system lifecycle.



Learning Objectives

- Explain how optimized ML systems can amplify harm through proxies, feedback loops, and distribution shift
- Apply data-algorithm-machine diagnosis to localize responsibility failures in data, algorithm objectives, or monitoring infrastructure
- Calculate fairness metrics from confusion matrices and compare trade-offs on the fairness-accuracy Pareto frontier
- Design disaggregated evaluation, stress testing, and monitoring to expose subgroup-specific failures before deployment
- Analyze total cost, inference dominance, and carbon impact as measurable responsibility constraints
- Construct model cards, datasheets, lineage records, and audit trails for accountability
- Evaluate privacy, access-control, and compliance designs against regulatory and human-review requirements

15.1 Responsibility as Systems Engineering

In 2014, Amazon built an AI recruiting tool[1] that penalized resumes containing the word "women's" (as in "women's chess club captain") and downgraded graduates of all-women's colleges. The system optimized faithfully for its stated objective: identify candidates similar to those previously hired. The system did not suffer an execution crash or optimization failure; historical hiring patterns encoded gender bias, which gradient descent systematically reproduced at scale.

If MLOps is the control loop for *reliability*, then **responsible engineering** is the control loop for *safety*. Where MLOps monitors runtime health and predictive quality to guide operational response, responsible engineering monitors outcome distributions to guide intervention when systems generate harm. In systems engineering terms, a model can pass *verification* (conforming exactly to its specified training objective) while failing *validation* (failing to satisfy operational, safety, or legal requirements) (National Aeronautics and Space Administration 2016). The verification gap (principle 9) concerns the first of these activities. It bounds how much any finite test can establish about conformance to a specification, whereas validation asks whether the specification was the right one. The failure stems from an incomplete specification rather than a fault in compiler execution or numerical optimization.

Traditional software engineering isolates defects within modules, though shared dependencies can still propagate faults. Machine learning systems introduce data-dependent coupling: data flows through shared representations, so corrupted or biased inputs alter the dense parameter matrices that serve every subsequent inference. A biased training dataset does not trigger a compile-time assertion or runtime crash; it silently shifts decision boundaries across the entire system. Read for responsibility rather than performance, the D·A·M taxonomy isolates where such failures originate: unrepresentative or biased inputs (Data), misaligned loss formulations (Algorithm), or telemetry blind spots and unjustified resource footprints in the serving infrastructure (Machine). This makes responsibility an architectural invariant rather than a postdeployment patch.

Responsible engineering expands the definition of correctness for machine learning systems. Traditional correctness (numerical stability, bounded latency, and high service availability) remains necessary, but insufficient. A production system must satisfy broader operational bounds: bounded error disparities across demographic slices, defensible energy expenditure, and verifiable data lineage. This expanded correctness applies engineering discipline to failure modes that aggregate performance metrics ignore. A p99 latency spike triggers automated alerts on an operational dashboard; a demographic fairness regression remains invisible to aggregate loss metrics until real-world harm occurs, and when the model's decisions shape its own future training data, the hidden disparity can grow with every retraining cycle (principle 13). Both demand rigorous telemetry and automated regression testing. The optimizations of the preceding chapters inherit the same blind spot, because each was judged by the cost terms its benchmark measured.

[1] **Amazon recruiting tool**: Developed starting in 2014 by Amazon's Edinburgh engineering team to rate applicants on a 1–5 scale, the system trained on approximately a decade of resumes, overwhelmingly from male applicants, reflecting the tech industry's gender ratio. By 2015 the gender bias was identified; by 2017 the project was abandoned after repeated remediation attempts (Dastin 2018). The engineering cost was not the compute but the opportunity cost: a multi-year recruiting project failed because the objective encoded historical bias, making it a documented specification failure in ML tooling.

↩ Continuous telemetry and automated retraining control loops for operational reliability are established in section 14.2.

↩ The D·A·M taxonomy, which classifies a system bottleneck along its data, algorithm, and machine axes, is established in section 1.5.

Systems Perspective 15.1: The full cost of the iron law

The iron law of ML systems (principle 3) decomposes execution time into data movement, computation, and fixed overhead. Compressing models, accelerating hardware, and automating operations each optimize these terms. Yet an optimization can create or shift costs that its benchmark omits.

A model quantized for edge deployment can consume less energy, but its accuracy loss need not fall evenly across demographic groups, so the compressed model must be revalidated on the same subgroup slices as the original. A recommendation system optimized for engagement maximizes a business metric, but may amplify harmful content. Responsible engineering extends this accounting to include broader impacts: the carbon cost of computation, the fairness cost of optimization choices, and the societal cost of deployment at scale. The iron law governs *how fast* systems run; responsible engineering governs *how well* they serve.

Enforcing expanded correctness requires hardware and data infrastructure capable of measuring what aggregate benchmarks obscure. The energy budgets and accelerator footprints quantified throughout this book contribute to carbon emissions through measured energy use, electricity sources, and hardware lifecycle impacts, establishing efficiency optimization as a physical responsibility constraint alongside serving latency. Data governance infrastructure (access control policies, data lineage tracking, and immutable audit logging) makes those operational boundaries enforceable in production. Yet before engineering these controls, systems architects must isolate where and why mathematical convergence diverges from system safety: the structural gap between scalar loss minimization and real-world outcomes.

15.2 Engineering Responsibility Gap

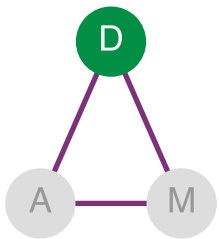
A loan model that approves 90 percent of qualified majority-group applicants but only 60 percent of equally qualified minority-group applicants can still report high aggregate accuracy, because the minority group contributes only a small share of the evaluated applicants. The worked example in section 15.3.2.1 computes exactly this model from its confusion matrices. The **responsibility gap** between this *technical correctness* and *responsible outcomes* represents a central challenge in machine learning systems engineering, one that existing testing methodologies were not designed to address. The gap manifests through concrete mechanisms: proxy variables, feedback loops, and distribution shift, each producing harm through a distinct pathway that conventional monitoring leaves invisible.

15.2.1 When optimization succeeds but systems fail

The Amazon recruiting tool from the chapter opening places this gap on the data axis of the D·A·M taxonomy (see the margin locator), because its failure stemmed from historical training signal rather than an algorithmic or software defect. A model trained on a decade of historical hiring data optimized faithfully for the objective it was given, but those historical patterns encoded gender bias that the system reproduced in candidate ratings (Dastin 2018).

Under empirical risk minimization, the model minimized classification loss by identifying token-level correlations across the resume corpus. Because historical technical hires were predominantly men, tokens associated with women's colleges, organizations, or activities exhibited negative statistical correlations with positive hiring labels. The optimization algorithm accurately extracted the empirical distribution of the training data, but that distribution reflected historical hiring practices rather than candidate competence. High-dimensional text representations encode and amplify these statistical regularities, embedding societal biases directly into downstream vector representations (Bolukbasi et al. 2016).

Amazon attempted remediation by removing explicit gender indicators and gendered terms from the training pipeline. This intervention failed to ensure unbiased recommendations because high-dimensional feature spaces contain **proxy variables** that retain demographic signal.[2] Scrubbing isolated attributes leaves correlated features intact: ZIP codes correlate with race through residential segregation, applicant names correlate with ethnicity, and healthcare utilization correlates with socioeconomic conditions. In Amazon's resume pipeline, even after explicit gender terms were



The Amazon failure turns on the data axis, not the algorithm.

[2] **Proxy variable**: Removing one proxy may have little effect when other correlated features retain similar signal. Conversely, correlation alone does not prove discrimination or identify the appropriate intervention. Protected-attribute removal, subgroup evaluation, counterfactual tests, causal analysis, and review of the full decision process provide complementary evidence; no single diagnostic is a complete defense.

excised, remaining vocabulary (attendance at specific colleges or participation in particular student organizations) continued to encode sex-correlated patterns (Dastin 2018). Removing protected attributes from training data is therefore insufficient to guarantee fairness.

Effective remediation requires architectural interventions across the full system lifecycle rather than a surface-level data filter. At the evaluation layer, disaggregated testing across demographic subgroups quantifies disparate scoring distributions before deployment. At the algorithm layer, training-time mitigations such as adversarial debiasing (developed with the loan worked example in section 15.3.2.1) can suppress protected-attribute signal in learned representations, though they still require continuous subgroup validation. At the operational layer, human review for borderline scores provides a safeguard only when reviewers are trained, empowered, and audited rather than used to rubber-stamp model outputs. Finally, at the monitoring layer, tracking downstream hiring outcomes over time detects disparate impact that offline loss metrics conceal. Because retrofitting these defenses onto an existing pipeline proved technically intractable, Amazon scrapped the project (Dastin 2018).

War Story 15.1: The COMPAS recidivism algorithm audit (2016)

Context: COMPAS is a risk assessment tool used in US courtrooms to predict re-offending for bail and sentencing decisions (Angwin et al. 2016).

Mechanism: The audited scores combined approximate calibration across racial groups (a given score corresponded to about the same observed re-offense rate in every group) with unequal error rates. These observations expose a policy choice about acceptable errors; they do not establish which objective the original optimizer used.

Impact: Black defendants who did not re-offend were incorrectly flagged as high-risk at nearly twice the rate of White defendants (44.9 percent vs. 23.5 percent), while White recidivists were far more often mislabeled low-risk.

Response: The audit exposed why calibration alone could not settle whether the error distribution was acceptable. Any jurisdiction using such scores needs an explicit policy for which fairness criteria matter, independent validation across groups, and review of how the score enters the decision workflow.

Systems lesson: The scores were approximately calibrated by race, but they violated equalized odds, the criterion that requires equal true positive and false positive rates across groups, because their error rates did not match. The loan worked example in section 15.3.2.1 computes error-rate criteria from confusion matrices and states calibration beside them. Formal fairness results show that calibration-style criteria and error-rate balance can conflict with unequal base rates and imperfect prediction, showing why engineering responsibility requires explicitly choosing which fairness constraint matters.

The Amazon recruiting tool and the COMPAS risk assessment system[3] illustrate the central thesis of this section: optimization success can coexist with catastrophic system failure. The reported disparities do not require a software bug or hardware malfunction to explain why a technically functioning model can fail its intended use. The architectural question is whether the problem specification captures acceptable outcomes. When the *technical objective* (such as minimizing empirical prediction error) diverges from the *system specification* (fair, defensible outcomes across user populations), improving the training objective can preserve or amplify harm.

Standard production telemetry cannot detect specification failures. Monitoring pipelines track operational invariants such as inference latency, memory utilization, throughput, and aggregate validation loss, all of which remain green while a model systematically misallocates errors across subgroups. Preventing silent specification failures requires making responsibility metrics part of the primary test harness: verifying whether the loss function acts as a defensible proxy for the true system goal, and enforcing the justified, application-specific fairness criterion before deployment.

[3] **COMPAS (Correctional Offender Management Profiling for Alternative Sanctions)**: In the analyzed data, COMPAS scores were approximately calibrated by race, meaning a given score corresponded to a similar observed re-offense probability across groups. Because recidivism prevalence differed between populations, calibration coexisted with disparate error rates (Chouldechova 2017; Kleinberg et al. 2017). Calibration testing alone therefore could not establish error-rate parity or determine whether the allocation of errors was acceptable.

Checkpoint 15.1: Responsible design

Responsibility is a system property, not a model property.

Failure modes

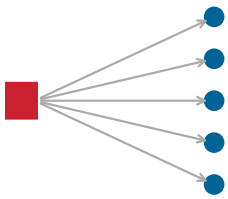
- ☐ **Alignment**: test whether the loss function is a defensible proxy for the system's true goal.
- ☐ **Disparate impact**: compare error rates across subgroups rather than relying on aggregate accuracy.

Check

- ☐ **Pre-mortem**: before deploying, ask: "If this system causes a scandal in six months, what likely went wrong?"

15.2.2 Silent failure modes

Consider a hospital sepsis model that begins recommending aggressive treatments for low-risk patients after an electronic health record (EHR) workflow change alters how vital signs are recorded. A single upstream modification propagates across the system's blast radius (the set of downstream predictions, and the patients behind them, that one upstream change reaches) without raising an alert. The model's confidence scores remain high, its inference latency stays within its service level agreement, and all system health checks pass green. The failure is silent: the input data distribution has shifted, but this hospital's pipeline monitors only service health and carries no drift monitors.



One upstream change; many silently affected.

↩ Drift monitors that quantify distributional divergence between serving and training data are established in section 14.6.2.

Definition 15.1: Distribution shift

Distribution shift occurs when the deployment distribution differs from the distribution used to develop or evaluate the model. Standard empirical-risk minimization commonly assumes matching training and deployment distributions. Covariate shift changes $p(x)$ while holding $p(y \mid x)$ stable; label shift changes $p(y)$ while holding $p(x \mid y)$ stable; concept drift changes $p(y \mid x)$.

1. **Significance**: Divergence statistics such as Jensen-Shannon divergence $\mathcal{D}_{\text{JS}}(P_t \| P_0)$ measure distributional change, not accuracy degradation. Useful alert thresholds must be calibrated empirically for each task, representation, label process, and deployment environment, then related to observed outcomes when labels are available. A $\mathcal{D}_{\text{JS}}$ value of 0.1 may be harmless for one feature space and severe for another, and input-divergence monitoring may miss concept drift.
2. **Distinction**: Distribution shift describes a change between development and deployment conditions. It does not imply that the learned mapping was correct at training time, nor does every shift reduce performance.
3. **Common pitfall**: Distribution shift is an umbrella term with several overlapping taxonomies. Monitoring only $p(x)$ can detect some covariate changes but cannot establish whether $p(y \mid x)$ remains stable; detecting concept drift generally requires labeled outcomes or justified proxies. Through the responsibility lens the pitfall sharpens, because shift often lands on one subgroup first, where a population-wide divergence statistic barely moves and concept drift on that slice stays invisible until its outcomes are labeled.

This sepsis scenario illustrates a class of failure that availability-focused monitoring cannot detect. Some software failures are loud: a null pointer exception terminates execution, a segmentation fault halts the process, or an exhausted socket pool returns a network timeout. Other software defects can silently produce incorrect results. ML systems introduce statistical failure modes in which corrupted or degraded outputs look indistinguishable from valid predictions. Floating-point tensors continue to flow through matrix multiplication kernels, accelerator execution completes within latency budgets, and downstream services receive structurally well-formed responses.

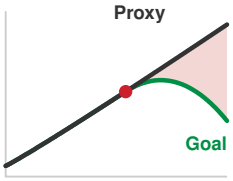
↩ Covariate, label, and concept drift, with the divergence statistics that detect them, are established in section 4.5.4.

Distribution shift represents the first major mechanism behind this silent degradation. The failure is environmental: the physical distribution of inputs changed after training, yet the model

has no internal telemetry to detect the discrepancy. Retraining on fresh data can partially restore accuracy under distribution shift. However, retraining cannot resolve a second, more fundamental mechanism of silent failure that persists even when the data distribution is perfectly stationary: metric misalignment.

Metric misalignment occurs when the objective function optimizes an observable proxy metric that diverges from the unobserved outcome the system is intended to deliver. Goodhart's Law, the benchmark trap in which a measure optimized as a target stops measuring the goal, applies with more force here than in benchmarking, because the training objective is the target and gradient descent searches it far more exhaustively than a team tuning against a benchmark. Up to a point the proxy and the goal improve together; past that point further optimization buys proxy gains at the goal's expense.

↩ Goodhart's law, under which a benchmark metric optimized as a target ceases to measure the goal, is established in section 12.1.



Past the knee, the proxy decouples from the goal.

Systems Perspective 15.2: The alignment gap

A model optimizes a proxy metric (Clicks) because the true metric (User Satisfaction) is unobservable, and the two can diverge sharply. In an illustrative system, $\text{Correlation}(\text{Clicks}, \text{Satisfaction})$ might begin at 0.8; once ranking maximizes clicks, it may find clickbait with high clicks but low satisfaction, and the correlation could fall to 0.2.

The falling correlation is the symptom. The quantity that matters is how far the proxy's movement overstates the goal's, which equation 15.1 captures for metrics normalized to a common scale:

$$\text{Gap} = \mathbb{E}[\text{Proxy}] - \mathbb{E}[\text{True}] \tag{15.1}$$

Suppose both metrics start at the same normalized value v, so the gap is zero. A ranking change that raises Clicks by 20 percent and lowers Satisfaction by 5 percent moves them to $1.20v$ and $0.95v$, opening a gap of $0.25v$. Every dashboard that reports Clicks shows a 20 percent win, while the goal the system exists to serve has moved the other way.

Systems insight: Engineers cannot directly optimize an unobserved goal. They need defensible proxy validation. A randomized holdout, a small random slice of users served without the new ranking, can estimate the ranking's causal effect on the gap, but only when the holdout's measured outcome reflects the goal or a justified surrogate for it.

The alignment gap illustrates a failure originating in the algorithm axis: the loss function optimizes a misspecified objective, so even a model that generalizes with zero test error drives outcomes that conflict with organizational or societal goals. Distribution shift, by contrast, originates in the data axis: the model logic remains intact, but the input distribution departs from the training distribution. Both failures can be silent, but they suggest different first interventions: objective redesign for a misspecified target, or pipeline investigation and possible adaptation for shifted inputs. These remedies can be complementary when both mechanisms are present. Conflating them wastes engineering cycles on ineffective remediations. Isolating the root cause is the job the D·A·M taxonomy already performs for performance bottlenecks, and perspective 15.3 reads the same three axes for responsibility failures.

↪ The performance-bottleneck form of the D·A·M diagnosis, from profiler symptom to binding axis, is developed in Appendix A.

Systems Perspective 15.3: Diagnosing responsibility failures with D·A·M

When a system causes harm, use the D·A·M taxonomy to identify the root cause. Responsibility failures are rarely "algorithm bugs"; they are structural flaws along one of the three axes:

- **Data (information)**: The training corpus reflects historical bias or unrepresentative sampling (such as Amazon's recruiting tool learning from biased historical applicant data). The failure lies in the *Fuel*.
- **Algorithm (logic)**: The objective function optimizes an unconstrained proxy metric that amplifies harm (such as engagement optimization driving polarization). The failure lies in the *Blueprint*.

- **Machine (physics)**: The serving and governance infrastructure cannot see or bound the harm, through telemetry that never slices outcomes by subgroup, missing lineage and audit records, or an energy and carbon footprint that exceeds the societal benefit of deployment (such as an overparameterized foundation model serving a low-complexity task). The failure lies in the *Engine*.

Locating the dominant failure in the taxonomy identifies the first remediation to test: better curation (Data), safer objectives (Algorithm), or better instrumented and more efficient infrastructure (Machine). Real failures can span axes, so the intervention must still be evaluated end to end.

While the D·A·M taxonomy isolates *where* failures originate, operational engineering requires identifying *when* and *how* different failure modes manifest in production telemetry. Table 15.1 categorizes ML system failures by detection latency, scope (how much of the served population the blast radius covers), and recovery interventions. Whereas traditional infrastructure crashes and throughput bottlenecks trigger immediate alerts, statistical failures operate silently, persisting for days or months while technical health checks remain green.

Table 15.1: Illustrative ML System Failure Mode Taxonomy: Detection time, scope, and remediation depend on the application; the entries show representative cases. Silent failures such as data quality issues, distribution shift, and fairness violations may require proactive monitoring because they do not necessarily trigger traditional alerts.

Failure Type	Illustrative Detection Time	Illustrative Scope	Possible Response	Example
Crash	Immediate	Complete	Restart after correcting cause	Out of memory error
Performance Degradation	Minutes	Complete	Relieve resource contention	Latency spike from resource contention
Data Quality	Hours–days	Partial	Data correction may be needed	Corrupted inputs from upstream system
Distribution Shift	Days–weeks	Partial or all	May require adaptation	Population change due to new user segment
Fairness Violation	Weeks–months	Subpopulation	May require redesign	Bias amplification in historical patterns

Beyond static distribution shifts and isolated proxy misalignments, closed-loop feedback introduces dynamic system degradation: model predictions actively reshape the user behavior that generates future training data. The YouTube recommendation feedback loop illustrates this dynamic at scale (M. H. Ribeiro et al. 2020).[4] M. H. Ribeiro et al. (2020) audited radicalization pathways on YouTube, finding migration from milder to more extreme channel categories and recommendation reachability between those categories. The broader systems lesson is that feedback loops can work exactly as designed while producing outcomes that conflict with societal values. Recommendation objectives must therefore be tested against downstream harms, not only against engagement proxies.

The Facebook News Feed overhaul provides an instructive contrast to the YouTube feedback loop: an engineering team deliberately sacrificing short-term engagement metrics to arrest proxy decoupling and protect long-term user retention (Mosseri 2018).

Example 15.1: News Feed proxy shifts

Scenario: Facebook shifted News Feed ranking objectives from short-term engagement proxies (clicks, watch time) to meaningful social interaction metrics (Mosseri 2018).

Diagnosis: Optimizing solely for short-term engagement proxies decoupled ranking outputs from long-term user welfare, amplifying clickbait and passive consumption.

Systems lesson: Engagement metrics are proxies for value, not value itself. Recommendation algorithms require multi-objective optimization with explicit safety constraints to prevent proxy decoupling.

[4] **Goodhart's law in feedback loops**: Recommendation feedback loops are the canonical ML manifestation of Goodhart's law. Gradient descent optimizes watch-time proxies at a speed no human curator can match, and the system's own outputs reshape the training distribution. Users who consume extreme content generate data that can reinforce extremity and can decouple the proxy from user welfare faster than manual editorial processes would.

↩ The YouTube feedback loop as a production technical-debt pattern is established in section 14.3.6.1.

When proxy misalignment intersects with demographic heterogeneity, silent failures manifest as structural discrimination. If the proxy itself reflects historical inequities, the training objective encodes those disparities into the model's decision boundaries, and the harm lands on the subgroup the proxy mismeasures. The outcome, a subgroup silently underserved, looks the same as the data-axis failure of Amazon's recruiting tool, but its origin lies on the algorithm axis, in the choice of target, so curating the data would not repair it. That algorithm-axis version appeared with life-threatening consequences in clinical care management.

War Story 15.2: The proxy variable trap (2019)

Context: In 2019, Ziad Obermeyer and colleagues at UC Berkeley audited a commercial Optum algorithm used by health systems to enroll patients with complex needs into high-risk care management programs, examining roughly fifty thousand patients across one large academic hospital (Obermeyer et al. 2019).

Mechanism: The model predicted "future healthcare cost" as a proxy for "future health need." Because the US healthcare system spends less on Black patients than on White patients with the same illness level, the algorithm learned this pattern and assigned lower risk scores to Black patients.

Impact: At any given risk score, Black patients carried substantially more chronic conditions than White patients, reducing the share of Black patients enrolled in specialized care.

Response: The study showed that replacing cost with a direct measure of health need would raise the share of Black patients identified for additional care from 17.7 percent to 46.5 percent at the analyzed threshold.

Systems lesson: Optimizing for a proxy inherits the biases of the system that generated the proxy. The proxy-target relationship must be audited across every demographic subgroup the system serves.

Silent failure modes fundamentally alter the systems contract between software engineering and operational verification. Traditional software verification relies on deterministic assertions: a unit test asserts that an output matches an exact value, memory bounds checkers assert that buffer allocations do not overflow, and watchdog timers ensure RPC latencies do not breach service level agreements. In an ML system, correctness cannot be verified by deterministic invariants alone, because, as the sepsis model showed, every such invariant can hold while the model serves biased, uncalibrated, or harmful predictions. What can be verified is the statistical bound of the verification gap (principle 9), which holds only over a named deployment population, task distance, and tolerance. A bound stated over the aggregate population says nothing about a subgroup that the aggregate averages away, so a responsible specification names each consequential subgroup as a population in its own right. Converting responsibility goals into structured engineering practice requires operationalizing statistical verification: continuously monitoring covariate shift across demographic slices, auditing proxy-to-target correlations, and bounding closed-loop feedback before silent errors propagate through production.

15.2.3 When responsible engineering succeeds

Each documented success shares the same structural move: a vague responsibility goal becomes an engineering constraint that can be specified, tested, communicated, and, when necessary, used to stop deployment. Following the findings of Gender Shades, a 2018 audit that exposed severe error-rate disparities in commercial gender classification (Buolamwini and Gebru 2018), Microsoft invested in improving gender-classification performance across demographic groups. Targeted data collection, model changes, and systematic disaggregated evaluation gave the team an explicit error-rate target, bringing audited error rates for darker-skinned subjects below 2 percent (Raji and Buolamwini 2019). Publishing these disaggregated benchmarks established a verifiable baseline and turned external audit findings into an operational release gate.

↩ Gender Shades, and the way aggregate benchmark metrics conceal subgroup disparities, are established in section 12.11.1.1.

Twitter's automatic image cropping system shows the same discipline under a different constraint. In 2020, users raised concerns about racial and gender bias in preview thumbnails. Twitter audited

the system, reported measured disparities and limitations, and shifted toward uncropped previews that gave users more control (Yee et al. 2021). In that case, responsible engineering changed the product design rather than relying on an incremental adjustment to an automated ranking threshold.

Differential privacy applies the same move to privacy. It turns a privacy requirement into a mathematical bound on how much any one record can change a system's outputs, a bound that can be budgeted and audited rather than asserted (Dwork 2008). Its mechanism and systems cost are developed with the privacy protections in section 15.5.2.

These successes demonstrate that responsible engineering functions only when high-level principles are translated into enforceable systems constraints across the D·A·M stack: curating training data distributions, bounding algorithmic loss functions, or tracking privacy budgets in execution runtimes. Enforcing these constraints requires the operational authority to halt deployment or retire components that violate specified bounds. Every successful intervention depends on continuous evaluation, yet verifying these statistical constraints differs fundamentally from traditional software verification.

15.2.4 The testing challenge

Traditional software testing evaluates behavior against unambiguous specifications: an arithmetic function must return the exact sum of its operands, and a relational database must preserve foreign-key integrity. Such deterministic properties map directly to executable assertions.

Responsible ML properties resist simple formalization. **Fairness** has multiple mathematical definitions that can conflict under conditions such as unequal base rates and imperfect prediction (Chouldechova 2017; Kleinberg et al. 2017). What counts as fair depends on context, values, and trade-offs that technical systems cannot resolve alone. Individual fairness requires that similar individuals receive similar treatment, while group fairness requires equitable outcomes across demographic categories. These criteria can conflict, and choosing between them requires value judgments beyond the scope of optimization.

Fairness constraints and predictive performance can trade off, but the relationship is application-specific. The Pareto frontier (principle 5) carries over unchanged, collecting the configurations for which one plotted objective cannot improve without degrading another. Figure 15.1 plots a hypothetical fairness-accuracy Pareto frontier against demographic disparity, measured as the difference between two groups' positive-prediction (approval) rates; the loan worked example in section 15.3.2.1 defines this demographic parity criterion formally. The disparity axis is inverted so that fairer configurations lie to the right. The figure contrasts an unconstrained maximum-accuracy operating point (A) against a balanced compromise (B) and a steep accuracy collapse under strict demographic equality (C). These annotated configurations illustrate candidate policy choices along the frontier; they do not imply that unconstrained training always maximizes accuracy at high disparity, that zero disparity must reduce accuracy, or that every application has a sweet spot.

The frontier tells engineers what trade-off they may need to choose, but it cannot be plotted until subgroup performance is measured. Responsible properties become testable when engineers work with stakeholders to define criteria appropriate for specific applications. The Gender Shades project[5] demonstrated how **disaggregated evaluation** across demographic categories reveals disparities invisible in aggregate metrics (Buolamwini and Gebru 2018), exposing the subgroup failures that responsibility monitoring must catch before deployment. Table 15.2 shows the error-rate differences that commercial gender-classification systems produced across demographic groups. Measuring such slices has its own cost, because small subgroups contribute few samples to a randomly drawn test set; napkin math 15.1 quantifies that representation cost.

Table 15.2: Gender Shades Face++ Error Rates: The dark-female error rate is 43.1× the light-male rate. Source: (Buolamwini and Gebru 2018).

Demographic Group	Error Rate (%)	Relative to Light-Skinned Males
Light-skinned males	0.8%	Baseline (1.0×)
Light-skinned females	9.8%	12.2×
Dark-skinned males	0.7%	0.9×

5 **Gender Shades**: A 2018 study by Joy Buolamwini (MIT Media Lab) and Timnit Gebru (Microsoft Research) that audited commercial gender-classification systems from Microsoft, IBM, and Face++ using the Fitzpatrick skin type scale, a dermatological classification that Thomas Fitzpatrick developed in 1975 for UV sensitivity and that was later validated for clinical use (Fitzpatrick 1988), repurposed here as a demographic benchmark for algorithmic auditing. The study demonstrated the value of disaggregated evaluation; in the Face++ results reproduced in table 15.2, the dark-female error rate is 43.1× the light-male rate. Microsoft's later reported reductions show how public audit results can motivate measurable remediation (Raji and Buolamwini 2019).

Demographic Group	Error Rate (%)	Relative to Light-Skinned Males
Dark-skinned females	34.5%	43.1×

As table 15.2 quantifies, disaggregated evaluation revealed what an aggregate score concealed. The audit measured Face++ error rates of 0.8 percent for light-skinned males and 34.5 percent for dark-skinned females (accuracies of 99.2 percent and 65.5 percent). The aggregate metric gave no indication that the dark-female error rate was 43.1× the light-male rate.

No universal threshold defines acceptable disparity, but teams should establish explicit, justified bounds before deployment. A team might adopt an error-rate ratio below 1.25× or a false-positive-rate difference under 5 percentage points as an application-specific release policy. In US employment law, the disparate impact doctrine[6] shapes regulatory oversight, while the four-fifths rule[7] provides a separate, non-dispositive selection-rate diagnostic. The key engineering discipline is defining criteria appropriate to the application and governing law rather than generalizing one domain's threshold.

Surfacing latent responsibility failures before deployment requires expanding testing beyond uniform validation sets across three distinct verification regimes.

Slice-based evaluation partitions evaluation data into demographic and operational cohorts, computing metrics independently for each slice. A model may achieve 95 percent accuracy overall while dropping to 78 percent on low-income applicants or rural cohorts, a disparity that aggregate reporting never shows.

Invariance testing verifies that model decisions remain stable across semantically irrelevant feature substitutions. Swapping candidate names such as "John" and "Jamal" in a loan application must not alter approval probabilities when demographic features are legally and technically irrelevant to creditworthiness. Behavioral testing frameworks such as CheckList operationalize this principle by organizing validation suites around behavioral capabilities and perturbation invariances rather than static validation loss (M. T. Ribeiro et al. 2020).

Boundary and stress testing probe input regimes where empirical validation data is sparse or unrepresentative. Boundary evaluation maps behavior at distributional margins (extreme age distributions, rare transaction amounts, or sparse geographic categories), where prediction variance explodes. Stress testing extends these margins into adversarial conditions: corrupted feature pipelines, sudden covariate shift, and synthetic edge cases designed to test safety margins under degradation. Stakeholder red-teaming complements automated suites by introducing adversarial domain probes from affected communities, surfacing failure modes that statistical benchmarks overlook.

Responsible testing strategies complement traditional software testing rather than replacing it. Each demands joint judgment to select, configure, and interpret. Legal specialists cannot alone specify which demographic slices matter for a healthcare algorithm, and product managers cannot alone determine appropriate invariance tests for a loan model. Domain experts, affected stakeholders, policy specialists, and engineers must define the relevant harms together; engineers then encode those requirements as measurable tests and production controls. Responsibility therefore needs clear ownership within engineering as well as independent oversight outside it.

15.2.5 Engineering leadership on responsibility

By the time an outside audit exposed the proxy target in the care-management algorithm (war story 15.2), correcting it meant changing what the model predicted, not tuning a threshold. Earlier design choices narrow the available fixes. Responsible AI engineering cannot be delegated exclusively to ethics boards or legal departments. These groups provide essential oversight but lack the technical access required to identify problems early in the development process.

Legal or ethics review can identify a problem near deployment, but it cannot recover design options the system has already foreclosed. If the team trained the model without fairness constraints, chose an architecture that cannot support interpretability requirements, or built a data pipeline without the demographic attributes needed for monitoring, review can only accept, reject, or demand expensive redesign. Engineers therefore occupy a critical position in the ML development lifecycle because their choices define the solution space for all subsequent interventions: architecture determines

[6] **Disparate impact**: In *Griggs v. Duke Power Co.* (1971), the US Supreme Court held under Title VII that an employment practice may be unlawful because of its operation even without discriminatory intent (Supreme Court of the United States 1971). Disparate impact is a legal claim with statute- and context-specific elements, not a synonym for any statistical disparity. Model outcomes and proxies can supply relevant evidence, but metrics alone do not establish liability.

[7] **Four-fifths rule**: The 1978 Uniform Guidelines on Employee Selection Procedures use the four-fifths rule as a practical employment-selection diagnostic (Equal Employment Opportunity Commission et al. 1978). A selection rate below 80 percent of the rate for the group with the highest rate is generally regarded as evidence of adverse impact, but the Guidelines also recognize that smaller differences may matter and that the ratio is not dispositive.

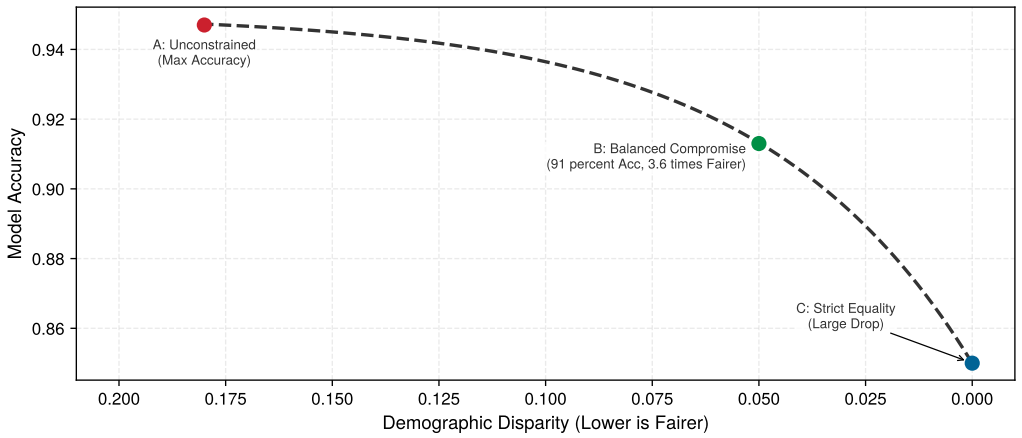


Figure 15.1: An Illustrative Fairness-Accuracy Pareto Frontier: A hypothetical relationship between model accuracy and demographic disparity. The x-axis is inverted so lower disparity lies to the right. Points A, B, and C illustrate three possible operating choices along this constructed frontier; B's label of 3.6 times fairer means its disparity (0.05) is 3.6 times smaller than A's (0.18). The curve is not an empirical or universal law.

which fairness constraints can apply, the optimization objective determines which patterns the system learns, and the data pipeline determines whether disaggregated evaluation is possible.

Definition 15.2: Responsible AI engineering

Responsible AI engineering is the engineering discipline of designing, deploying, and maintaining systems with probabilistic outputs by operationalizing societal and regulatory requirements as testable constraints on the D·A·M axes: permissible data contents, provenance, and composition; allowable model behavior and robustness properties; and infrastructure bounds such as latency, energy, compute budget, carbon emissions, and audit-log retention.

1. **Significance**: Each D·A·M axis acquires concrete governance constraints: the data axis is bounded by privacy regulations such as the General Data Protection Regulation (GDPR), which limits which records, fields, and features can be collected; the algorithm axis is bounded by fairness and robustness metrics (for example, positive prediction rates that differ across protected groups by no more than 5 percentage points, the demographic parity bound $\epsilon_{\text{par}} = 0.05$ defined with the loan worked example in section 15.3.2.1, or accuracy that degrades by less than 2 percent when an adversary may shift every normalized input feature by at most 0.01, written $\|\delta_{\text{adv}}\|_\infty \leq \epsilon_{\text{adv}} = 0.01$); and the machine axis is bounded by resource and infrastructure budgets such as latency, energy per inference, carbon emissions, and audit-log retention. Violating these bounds is a system failure, not a research shortcoming.
2. **Distinction**: Unlike AI ethics (which articulates aspirational values), responsible AI engineering translates those values into measurable, testable invariants that can be verified through automated testing and continuous monitoring, using the same lifecycle practices that enforce latency SLOs.
3. **Common pitfall**: A frequent misconception is that responsibility is "added" at the end of development. Constraints on what data may be collected affect what can be learned and audited, while infrastructure choices affect what evidence can be retained. Late-stage remediation may remain possible, but it is often narrower, slower, and more expensive.

An engineering-centered approach does not diminish the importance of diverse perspectives in identifying potential harms. Product managers, user researchers, affected communities, and policy experts contribute essential knowledge about how systems fail socially despite technical success.

Engineers translate these concerns into measurable requirements and testable properties that can be verified throughout the development lifecycle. Effective responsibility requires engineers who both listen to stakeholder concerns and possess the technical capability to implement appropriate safeguards.

Engineering teams do not operate in isolation. Figure 15.2 diagrams this multi-tier hierarchy as four nested operational rings, showing how team-level technical practices such as audit logging, bias testing, and explainable interfaces sit at the core of expanding organizational, industry, and regulatory structures. Requirements and legal mandates flow inward across these boundaries, while rigorous technical controls at the innermost layer provide the evidentiary substrate necessary for external compliance.

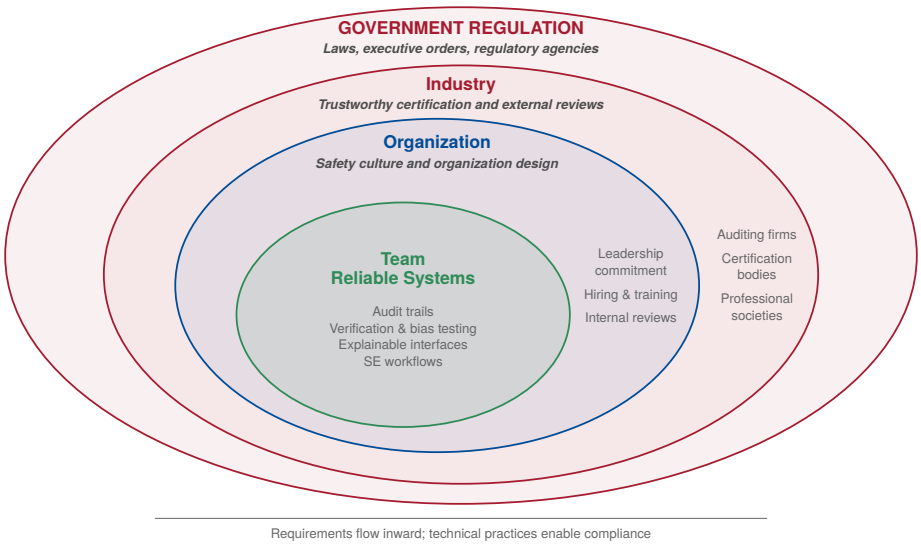


Figure 15.2: Responsible AI Governance Layers: Four nested ovals place team-level safeguards at the center, surrounded by organizational safety culture, industry certification and review, and government regulation. The nesting distinguishes implementation at the center from oversight outside it; the annotation states that requirements flow inward and technical practices enable compliance. Adapted from Shneiderman (2022).

Governance layers establish who must act, and organizations still need a reason to fund that action. Beyond ethical imperatives, responsible engineering can deliver business value through three reinforcing mechanisms. The most immediate is risk mitigation: ML system failures create legal and financial exposure that systematic responsibility practices can reduce. Organizations implementing disaggregated evaluation, documentation, and monitoring can reduce the probability of costly failures and preserve evidence of their engineering process if problems emerge.

A second mechanism is regulatory compliance, driven by legal requirements that vary by jurisdiction and application risk. The EU AI Act, for example, attaches risk-management, data-governance, transparency, and human-oversight requirements to high-risk applications (section 15.5.3). Organizations that build responsibility into engineering practice can demonstrate compliance through existing documentation and monitoring rather than expensive retrofitting; the engineering lesson is that proactive controls are usually cheaper than reconstructing evidence after deployment.

Competitive differentiation completes the business case. Trust can drive enterprise purchasing decisions for ML-powered services, and organizations that can demonstrate systematic responsibility practices through model cards, audit trails, and published evaluation results may qualify for deployments that competitors cannot. Microsoft's published remediation after the Gender Shades audit (Raji and Buolamwini 2019) shows the mechanism: disaggregated results that outsiders can check turn a claim of responsibility into evidence. Capturing any of these three forms of value requires repeatable workflows rather than good intentions: checklists, documentation standards, and testing protocols that embed responsibility constraints directly into the engineering lifecycle.

15.3 Responsible Engineering Across the Lifecycle

Amazon found its recruiting tool's bias internally but could not remediate it after the fact; COMPAS's error-rate disparity surfaced only through an external audit. In both cases the discovery came after the design options that could have addressed it were closed. The lesson concerns timing more than vigilance. Each responsibility failure in this chapter has a point in the ML lifecycle before which it is cheap to prevent and after which it becomes expensive or impossible to repair, so responsible engineering is organized around that lifecycle rather than around a list of concerns.

Six stages follow from that premise, and the first four form a chain in which each stage depends on the output of the one before it. *Specify* comes first because a system cannot be tested against a requirement nobody wrote down; the target, the populations that matter, the evaluation data those populations demand, and the boundary of human oversight must all be fixed before training forecloses the alternatives. *Measure* turns those written requirements into release gates, evaluating each named population separately and pricing the trade-offs among fairness criteria. Evidence that lives only in an engineer's notebook cannot be contested, audited, or reused, so *Explain and document* makes individual decisions legible to the people they affect and whole models legible to the teams that inherit them. The documented operating envelope then tells *Operate* what to watch after launch, when feedback loops and distribution shift erode a model that passed every gate. The last two stages span the whole lifecycle rather than following it, so each receives its own section after this one, while the four chained stages structure the subsections of this section. *Resource* accounts for the energy, carbon, and money that every stage consumes (section 15.4), and *Govern* supplies the regulatory obligations and the data infrastructure of access control, privacy, lineage, and audit that make each earlier stage provable to someone outside the team that built it (section 15.5).

15.3.1 Specify: Requirements before training

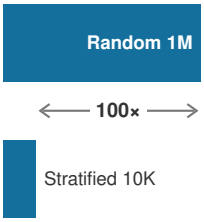
The proxy variable trap (war story 15.2) was a specification failure. The care-management model predicted exactly what it was asked to predict, and the request was wrong. Specification is the stage with the most leverage and the least visibility, because its errors surface only after the model has learned them. Before a loan approval model reaches production, a team must therefore determine the provenance of the training data, identify who is represented and who is missing, anticipate failure modes, and define recourse for affected users. Table 15.3 organizes these questions by pipeline phase, distinguishing critical-path blockers from high-priority items that can proceed with documented risk acceptance. Its rows follow the pipeline rather than the six stages because specification must anticipate every phase in advance. The Evaluation row fixes what *Measure* will gate, and the Deployment and Monitoring rows fix what *Operate* will watch and how it will respond.

Table 15.3: Predeployment Assessment Framework: Critical Path items block deployment until addressed. High Priority refinements may follow launch under documented risk acceptance; a specified objective and tested monitoring and response paths remain prerequisites. Systematic coverage of responsibility concerns throughout the ML lifecycle reduces the chance that risks are overlooked.

Phase	Priority	Key Questions	Documentation Required
Data	Critical Path	Where did this data come from? Who is represented? Who is missing? What historical biases might be encoded?	Data provenance records, demographic composition analysis, collection methodology documentation
Training	High Priority	What is the system optimizing for? What does the loss implicitly penalize? How do architecture choices affect outcomes?	Objective function specification, regularization choices, hyperparameter selection rationale
Evaluation	Critical Path	Does performance hold across different user groups? What edge cases exist? How were test sets constructed?	Disaggregated metrics by demographic group, edge case testing results, test set composition analysis
Deployment	Critical Path	Who will this system affect? What happens when it fails? What recourse do affected users have?	Impact assessment, stakeholder identification, rollback procedures, user notification protocols
Monitoring	High Priority	How are anomalies detected? Who reviews system behavior? What triggers intervention?	Monitoring dashboard specifications, alert thresholds, review schedules, escalation procedures

Critical Path items are deployment blockers: the system must not go to production until these questions are answered. The Training and Monitoring rows include both release prerequisites and later refinements: teams must specify the objective and test the minimum monitoring and response controls before launch. Remaining refinements may proceed with documented risk acceptance and a remediation timeline. The distinction enables teams to ship responsibly without requiring perfection on every dimension before initial deployment.

The Evaluation row in table 15.3 belongs to specification rather than to testing because naming the groups that matter also fixes a data budget. Showing that performance holds for each group requires a statistically valid test set per group, and the cost of that set grows with the group's rarity.



Random sampling needs 100× more images to reach a 1 percent subgroup.

Napkin Math 15.1: The statistics of representation

Problem: An engineering team needs to verify that a Face ID model works for a minority group representing 1 percent of the user base. The group's error rate must be known to within 1 percentage point at 95 percent confidence. The binomial standard error behind the statistical confidence trap (notebook 12.1) is largest at an error rate of 0.5, so the worst-case margin of error on n images is $1.96\sqrt{0.25/n}$, which falls to one percentage point near n = 10,000 images for this group.

Random sampling: To get 10,000 images of a 1 percent group in expectation via independent random sampling, the team must collect and label: $D_{\text{eval,total}}$ = 10,000 images / 0.01 = 1,000,000 images

Stratified sampling: Recruiting a representative, independently sampled evaluation set within this group requires 10,000 images.

Systems insight: Under these assumptions, natural-population sampling requires 100× as much total collection as direct subgroup recruitment, in expectation rather than as a guaranteed quota. Targeted recruitment must still represent the deployment subgroup; active selection of difficult cases cannot be treated as an unbiased evaluation sample without accounting for its sampling design. Fairness requires intentional data engineering rather than uncurated volume.

Intentional data engineering addresses what the model sees during evaluation, but even a perfectly representative dataset cannot prevent harm at deployment if the system lacks adequate human oversight. The representation cost derived in napkin math 15.1 settles what the evaluation must contain. The Deployment row of table 15.3 forces a second decision that cannot be deferred to launch: where people sit in the decision loop once the model is live, and what that arrangement must be designed to withstand.

War Story 15.3: The automation paradox (2018)

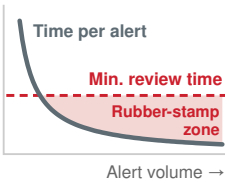
Context: Uber's Advanced Technologies Group (ATG) was testing self-driving cars in Arizona, relying on a human safety driver to intervene if the autonomous perception system failed (National Transportation Safety Board 2019).

Mechanism: The perception system detected a pedestrian crossing the road but repeatedly toggled its classification between vehicle, bicycle, and unknown, resetting its trajectory prediction while automatic emergency braking was intentionally disabled.

Impact: The safety driver was visually distracted by a personal phone and failed to take control, resulting in a fatal collision.

Fix: Uber retrofitted the fleet with driver-monitoring cameras, re-enabled automatic emergency braking, and established dual-operator staffing during autonomous road testing.

Systems lesson: Adding a human backup creates a new system with its own failure modes. High automation reliability can encourage complacency and reduce vigilance, so an effective fallback requires workload design, attention monitoring, and a safety case for the combined human-machine system.



If reviewer hours stay fixed, rising alert volume eventually leaves too little time per alert for real review.

For high-stakes applications, the specification should state where human oversight is required. Human-in-the-loop (HITL) systems route uncertain, high-consequence, or flagged decisions to human reviewers rather than acting autonomously; the clinician approval gates of ClinAIOps (section 14.9.2) are one proposed design. Effective HITL design must specify four requirements: the review scope (which decisions require human review), the confidence thresholds that trigger escalation, the training reviewers receive, and the mechanisms for monitoring reviewer performance. HITL is not a catch-all solution: human reviewers can rubber-stamp automated decisions, introduce their own biases, or become overwhelmed by alert volume. Effective HITL design requires calibrating the human-machine boundary to the specific application risks and reviewer capabilities.

The assessment parallels an aviation preflight checklist. It standardizes the questions a team must answer under schedule pressure, but answering them does not show that the system is safe. That evidence comes from measuring the model against the populations the specification named, which turns each written requirement into a gate the release must pass.

↩ The ClinAIOps clinician oversight loop, in which a clinician approves, modifies, or rejects each therapy recommendation, is established in section 14.9.2.1.

15.3.2 Measure: Testing across populations

The disaggregated evaluation that exposed the Gender Shades disparities (section 15.2.4) now becomes an operational release-gate task: selecting the slices, metrics, and thresholds that determine whether deployment is allowed.

> Systems Perspective 15.4: The flaw of averages
>
> A bridge that is "safe on average" but collapses under a heavy truck illustrates the **flaw of averages** (Savage 2009). An ML system that is "accurate on average" but fails for a specific demographic cohort presents the same design mistake. As with tail latency (p99), a release gate must test the conditions its aggregate metric conceals; disaggregated evaluation makes cohort-specific failures visible.

Selecting those conditions requires application knowledge. A vision classifier fails differently from a recommendation engine or an audio wake-word detector, so fairness metrics and evaluation slices must reflect the application domain. In healthcare diagnostics, demographic cohorts such as age and biological sex govern slice definitions. In content moderation, dialect and linguistic context dominate. In financial underwriting, credit regulations mandate auditing legally protected categories.

Operationalizing these evaluations requires four core capabilities in the testing infrastructure. First, stratified evaluation computes performance metrics separately for each cohort to expose error-rate disparities across populations. Second, intersectional analysis evaluates combinations of attributes, catching compounding failure modes that remain invisible in single-factor slices. Third, confidence intervals quantify statistical uncertainty when sparse subgroup sample sizes yield noisy point estimates. The binomial resolution limit that the statistical confidence trap (notebook 12.1) exposed for an aggregate test set binds hardest here, because each slice holds only a fraction of the evaluation data. Fourth, temporal monitoring tracks cohort performance continuously across deployment, detecting distribution drift that degrades minority slices before impacting aggregate health checks.

Tool selection matters only after the engineering team specifies the metrics, subgroup slices, and alerting thresholds. Open-source evaluation suites such as Fairlearn (Bird et al. 2020), AI Fairness 360 (Bellamy et al. 2019), and Google's What-If Tool (Wexler et al. 2020) simplify the computation of stratified and intersectional metrics, but software libraries cannot supply domain judgment. They compute whichever metric an engineer queries; they cannot decide which demographic boundaries matter, calibrate paging thresholds for on-call engineers, or select the appropriate legal fairness constraint.

An operational release gate must distinguish absence of evidence from evidence of parity. If a slice lacks sufficient labeled validation data to bound the error rate with statistical confidence, the CI/CD gate must report insufficient data rather than a false pass, which triggers targeted data acquisition or restricts deployment scope. Population testing thus enforces a tri-state operational contract: pass, fail, or insufficient evidence.

Lighthouse 15.1: Fairness concerns by archetype

The lighthouse workloads of section 2.3.2 were defined by the resource that binds each one, but their fairness risks come from somewhere else, from the data each learns from and the context each serves. Table 15.4 maps each lighthouse's data source and deployment context to a primary risk and metric.

Architecture alone does not determine fairness; each row identifies a deployment-specific risk to test first, not a universal property of the model family or its bottleneck.

Table 15.4: Fairness Risk by ML Archetype: Fairness risks vary by archetype's data source and deployment context.

Archetype	Primary Fairness Risk	Key Evaluation Metric	Real-World Example
ResNet-50 (Compute Beast)	Training data bias (underrepresentation of deployment-relevant groups)	Disaggregated accuracy by demographic group	Commercial face classifiers erred most on darker-skinned women (Gender Shades audited those systems, not ResNet-50)
GPT-2 (Bandwidth Hog)	Corpus bias (overrepresentation of majority viewpoints in web text)	Toxicity rate by demographic prompt context; stereotype score	LLMs produce more toxic completions for prompts mentioning minority groups
DLRM (Sparse Scatter)	Feedback-loop amplification (popular items get more data)	Share of recommendation impressions by item category and supplier or creator group	Filter bubbles: the system recommends similar content to similar users, reducing discovery of niche creators
DS-CNN (Tiny Constraint)	Deployment-context mismatch (trained on clean audio, deployed in noisy real-world environments)	False positive rate by acoustic environment and speaker accent	Voice assistants perform worse on accented speech; wake-word triggers on TV audio in some languages

Systems insight: Fairness evaluation must match each archetype's failure mode. Vision models require stratified demographic accuracy; large language models (LLMs) need toxicity and stereotype probes; recommenders need exposure audits; TinyML needs acoustic-environment tests. The keyword spotting (KWS) system that serves as the Tiny Constraint lighthouse faces exactly this challenge for its depthwise-separable convolutional neural network (DS-CNN). Trained on clean studio audio, it must perform equitably across accents, background noise levels, and speaker demographics in production homes, so its release gate needs accent and acoustic-environment slices with the per-slice confidence intervals that the testing infrastructure computes.

15.3.2.1 Worked example: Fairness analysis in loan approval

A loan approval model reports 85 percent accuracy on the majority group and 82.5 percent overall accuracy across the evaluated applicants, numbers that may satisfy a coarse aggregate dashboard. Evaluating the model separately across demographic cohorts exposes how aggregate metrics conceal severe disparities. This constructed evaluation assumes observed repayment outcomes for every applicant, independent of the model's approval predictions; those predictions are not the lending actions that produced the labels. The two cohorts' outcomes appear side by side, the majority cohort (Group A) in table 15.5 and the minority cohort (Group B) in table 15.6.

Table 15.5: Confusion Matrix for Group A (Majority): In this constructed scenario, loan approval outcomes for 10,000 applicants from the majority demographic group. The 90 percent true positive rate (4,500 approved of 5,000 qualified) and 20 percent false positive rate establish the baseline for fairness comparison.

	Approved (pred)	Rejected (pred)
Repaid (actual)	4,500 (TP)	500 (FN)
Defaulted (actual)	1,000 (FP)	4,000 (TN)

Table 15.6: Confusion Matrix for Group B (Minority): In the same constructed scenario, loan approval outcomes for 2,000 applicants from the minority demographic group. The 60 percent true positive rate (600 approved of 1,000 qualified) reveals a 30 percentage-point disparity compared with Group A; the confusion matrices alone do not identify its cause.

	Approved (pred)	Rejected (pred)
Repaid (actual)	600 (TP)	400 (FN)
Defaulted (actual)	200 (FP)	800 (TN)

Every rate in the calculations that follow can be reconstructed from the approved/rejected and repaid/defaulted cells of these two matrices. Three standard criteria for equitable treatment capture distinct operational objectives:[8]

Demographic parity requires equal positive selection rates across groups regardless of underlying qualification ($\Pr(\hat{Y}=1 \mid G=\mathrm{A}) = \Pr(\hat{Y}=1 \mid G=\mathrm{B})$). Group A receives approval at $(4{,}500+1{,}000)/10{,}000 = 55\%$, whereas Group B receives approval at $(600+200)/2{,}000 = 40\%$. The resulting 15 percentage-point disparity indicates unequal selection outcomes between cohorts.

Equal opportunity requires equal true positive rates among qualified applicants ($\Pr(\hat{Y}=1 \mid Y=1, G=\mathrm{A}) = \Pr(\hat{Y}=1 \mid Y=1, G=\mathrm{B})$). Group A achieves a TPR of $4{,}500/(4{,}500+500) = 90\%$, meaning 90 percent of creditworthy applicants secure loans. Group B achieves only $600/(600+400) = 60\%$. This 30 percentage-point disparity reveals that qualified applicants in Group B suffer disproportionate rejection relative to equally qualified peers in Group A.

Equalized odds tightens the constraint, demanding parity in both true positive and false positive rates ($\Pr(\hat{Y}=1 \mid Y=v, G=\mathrm{A}) = \Pr(\hat{Y}=1 \mid Y=v, G=\mathrm{B})$ for $v \in \{0,1\}$).[9] Neither group may then bear a disproportionate share of unjustified rejections (false negatives) or unjustified approvals (false positives). Group A yields an FPR of $1{,}000/(1{,}000+4{,}000) = 20\%$, matching Group B at $200/(200+800) = 20\%$. Although false positive rates align, the substantial true positive rate deficit violates equalized odds.

Both groups in this scenario have the same base rate (5,000 of 10,000 and 1,000 of 2,000 applicants repaid), so the impossibility results for unequal base rates do not bind here. Equal base rates leave remediation possible in principle, but these matrices do not identify the cause of the disparity or which intervention would close it. Those impossibility results involve a fourth criterion, calibration within groups, which applies the probability calibration that evaluation already checks to each group separately. Among applicants in group g who receive score s, the fraction who repay should be s, that is, $\Pr(Y=1 \mid \text{score}=s, G=g) = s$ for every group. Real populations often have unequal base rates, and with unequal base rates and imperfect prediction, calibration-style criteria and error-rate balance can conflict (Chouldechova 2017; Kleinberg et al. 2017). A score calibrated within each group then generally assigns the groups different false positive or false negative rates. No threshold or retraining choice satisfies every criterion at once, so the operational guarantee must be selected and defended explicitly for the application rather than deduced from aggregate accuracy or calibration.

These metrics disagree because they encode different policy choices about which error rates matter most. In this scenario, the model rejects qualified applicants from Group B at a much higher rate (40 percent false negative rate vs. 10 percent) while maintaining similar false positive rates. These confusion matrices establish an operational disparity, not whether it arose from classification thresholds, labeling bias, omitted features, sampling skew, or historical discrimination.

Where collection and use of protected attributes are lawful and justified, production systems can automate these calculations and trigger review when disparities exceed predefined thresholds. Listing 15.1 makes the failure-handling contract explicit: undefined group rates are reported as insufficient data rather than compared against a threshold. These denominator checks establish arithmetic validity only. A production release gate also needs minimum labeled sample counts and uncertainty checks for each relevant slice; a defined rate from one observation is not sufficient evidence.

8 **Fairness metric incompatibility**: The measured disparities in this worked example show how one set of confusion matrices can violate demographic parity, equal opportunity, and equalized odds. A separate impossibility result shows that, with unequal group base rates and imperfect prediction, several desirable criteria, including calibration-style predictive parity and error-rate balance, cannot generally be satisfied together (Chouldechova 2017). In those settings, optimizing one criterion can degrade another. A system designer must therefore make the trade-off explicit rather than assuming all guarantees can be achieved at once.

9 **Equalized odds**: Formalized by Hardt et al. (2016), requiring that both TPR and FPR be equal across protected groups; the weaker "equal opportunity" relaxes this to TPR alone. Equalized-odds postprocessing may require randomized, group-dependent decisions rather than deterministic thresholds alone. It also requires protected-group information at decision time and must be evaluated under applicable law; for example, US credit regulation generally restricts using protected bases to assess creditworthiness.

Listing 15.1: Automated Fairness Monitoring: The core pattern computes per-group metrics from confusion matrices and alerts when disparities exceed application-specific thresholds. Production use requires sufficient data and lawful, justified handling of group attributes.

```
def compute_fairness_metrics(confusion_matrix):
    tp, fp, tn, fn = (
        confusion_matrix[k] for k in ["TP", "FP", "TN", "FN"]
    )
    total = tp + fp + tn + fn
    return {
        # Demographic parity
        "approval_rate": (tp + fp) / total if total else None,
        # Equal opportunity
        "tpr": tp / (tp + fn) if (tp + fn) else None,
        # Equalized odds (with TPR)
        "fpr": fp / (fp + tn) if (fp + tn) else None,
    }


# Compare groups and flag disparities exceeding threshold
for metric in ["approval_rate", "tpr", "fpr"]:
    if metrics_a[metric] is None or metrics_b[metric] is None:
        report_insufficient_data(metric)
        continue
    disparity = abs(metrics_a[metric] - metrics_b[metric])
    # e.g., 0.05 for high-stakes applications
    if disparity > FAIRNESS_THRESHOLD:
        trigger_alert(metric, disparity)
```

Automated monitoring achieves what manual auditing cannot at scale: repeated tracking of fairness metrics and alerts when sufficient labeled evidence reveals a disparity. Label availability and the time needed to accumulate observations bound detection speed. The 30 percentage-point TPR disparity far exceeds this scenario's 5-percentage-point release threshold, indicating the model requires fairness intervention before deployment. Table 15.7 summarizes the computed metrics and resulting disparities across both cohorts.

Table 15.7: Fairness Metrics Summary: In this constructed loan scenario, approval rate and true positive rate differ across groups, while false positive rates match; the table establishes disparity but not its cause.

Metric	Group A	Group B	Disparity
Approval Rate	55%	40%	15 pp
True Positive Rate	90%	60%	30 pp
False Positive Rate	20%	20%	0 pp

Shared classification thresholds applied to differing score distributions illustrate how aggregate performance masks severe subgroup divergence (Barocas and Selbst 2016). Shifting between the candidate decision boundaries diagrammed in figure 15.3 lifts overall accuracy from 75 percent to 81.25 percent by achieving perfect separation in Subgroup A, yet simultaneously triples the misclassifications in Subgroup B. Because the two subgroups' scores are distributed differently, one shared boundary cannot sit at the best point for both, and the move that raises the global metric does so by trading away Subgroup B's outcomes. Aggregate accuracy alone cannot determine whether those outcomes are equitable for either cohort.

Mitigating these disparities means intervening before, during, or after training, and each stage imposes its own systems trade-offs:

Preprocessing via sample reweighting fixes a weight for every training instance before training begins; the training loss then scales each Group B instance's penalty by its weight, amplifying gradient signal for underrepresented cohorts without discarding data.[10] The trade-off appears across the broader loss surface, where reweighting can degrade accuracy on majority slices.

[10] **Reweighting**: A preprocessing technique rooted in importance sampling from statistics: samples from an underrepresented group receive higher loss weights during training, amplifying their influence on gradient updates without removing any data. Kamiran and Calders (2012) showed that appropriately chosen weights can reduce disparate impact from training data. The systems trade-off is application-specific: reweighting shifts the loss landscape and may reduce performance on other slices, so the cost must be evaluated against the Pareto frontier for the application.

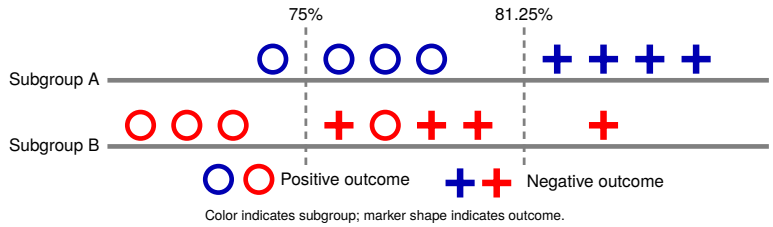


Figure 15.3: Threshold Effects on Subgroup Outcomes: Two candidate classification thresholds intersect different score distributions for Subgroups A and B. Circles mark positive outcomes (loan repayment), plus markers mark negative outcomes (default), and color distinguishes subgroup; each boundary approves the markers to its left. Each dashed line carries the overall accuracy that boundary achieves, not a score cutoff. Moving to the higher-accuracy boundary lifts overall accuracy from 75 percent to 81.25 percent, but the whole gain falls in Subgroup A, which becomes perfectly separated, while Subgroup B goes from one misclassification to three.

In-processing via adversarial debiasing turns the training loop into a zero-sum game between two neural networks. A primary model (encoder and predictor) transforms input features $\mathbf{x}$ (such as qualifications or income) into an intermediate latent representation $\mathbf{z}$, which a task head uses to predict the target label $\hat{y}$ under a loss $\mathcal{L}_{\text{task}}(\hat{y}, y)$ rewarding high predictive accuracy. Concurrently, an adversary head taps directly into the latent vector $\mathbf{z}$ and attempts to predict the protected attribute a (such as demographic group) by minimizing an adversarial loss $\mathcal{L}_{\text{adv}}(\hat{a}, a)$. During backpropagation, the adversary's gradient is negated as it flows into the shared feature encoder:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} - \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}$$

Here λ_{adv} weights the adversarial penalty against the task loss. The two players minimize different objectives. The adversary head minimizes $\mathcal{L}_{\text{adv}}$ alone, trying to recover a from $\mathbf{z}$, while the encoder and task head minimize $\mathcal{L}_{\text{total}}$. If the adversary easily decodes demographic identity from $\mathbf{z}$, $\mathcal{L}_{\text{adv}}$ is small, so little is subtracted and $\mathcal{L}_{\text{total}}$ stays high; the encoder lowers its objective only by making the adversary's task harder. To minimize total loss, the encoder is forced to purge latent features that correlate with the protected attribute, producing representations that remain informative for the task while suppressing, not eliminating, protected-attribute signal under the evaluated distribution.[11] The systems cost is real: a second (adversary) head to train, adversarial gradients that destabilize convergence, and a larger hyperparameter search.

Postprocessing via threshold adjustment decouples the decision boundary across groups, lowering the approval cutoff for Group B to equalize TPR. The operational trade-off is an increase in false positives within that cohort, admitting applicants who may subsequently default. The lever is also legally constrained. In the US, group-specific cutoffs are prohibited in employment selection and restricted in credit, so this intervention often exists only in principle, and teams move instead to reweighting, feature and label repair, or objective changes.

Selecting among these interventions requires stakeholder alignment on acceptable trade-offs for the specific deployment domain. Systems engineers make these trade-offs actionable by rendering them explicit and quantifiable.

> **Checkpoint 15.2: Fairness criteria**
>
> Fairness is not a single metric; it is a constrained design choice.
>
> - ☐ **Metric definitions**: can you distinguish demographic parity, equal opportunity, equalized odds, and calibration in terms of which rates must match across groups?
> - ☐ **Impossibility trade-off**: can you explain (at a high level) why base-rate differences make it impossible to satisfy all fairness criteria simultaneously?
> - ☐ **Systems interpretation**: given the confusion matrices in table 15.5 and table 15.6, can you identify which disparity matters operationally (TPR vs. FPR vs. approval rate) and what kind of harm it represents?

[11] **Adversarial debiasing**: The key differentiating property is representation pressure: the adversary discourages the primary model from encoding protected-attribute information, which can reduce protected-attribute leakage and help satisfy selected fairness criteria under the evaluated distribution (B. H. Zhang et al. 2018); it does not provide a general fairness guarantee under arbitrary deployment shift; guarantees depend on assumptions about invariance, labels, causal structure, and the type of shift. Postprocessing methods such as threshold adjustment may also be appropriate under different assumptions but must be revalidated when deployment demographics or label processes change. The cost is additional training, hyperparameter tuning, and slice-level validation rather than a universal percentage overhead.

☐ **Engineering decision**: for a concrete high-stakes domain (credit, hiring, criminal justice), can you justify which fairness constraint you would prioritize and why?

15.3.2.2 Quantifying the fairness-accuracy trade-off

The impossibility results of Kleinberg et al. (2017) and Chouldechova (2017) describe conflicts among particular fairness criteria under unequal base rates and imperfect prediction. A separate question is how a chosen engineering constraint changes predictive utility. The hypothetical frontier in figure 15.1 illustrates possible operating choices from A toward C; the deployment's measured candidate configurations, rather than that illustrative curve, determine its actual trade-offs. Quantifying this trade-off requires evaluating economic costs for the specific application. An automated hiring pipeline makes the resulting penalty concrete because each outcome carries an explicit value, a successful hire on one side and a bad hire on the other. The hiring model that follows is a separate scenario with its own rates, not the loan model of section 15.3.2.1.



Napkin Math 15.2: The price of fairness

Problem: Stakeholders demand elimination of a 20 percentage-point true positive rate (TPR) disparity in a hiring model. What is the "Price of Fairness" in terms of hiring quality?

Physics: TPRs can be equalized by adjusting the classification threshold (γ_{cls}) for the disadvantaged group.

- **Original state**: Group A (TPR $=$ 90 percent), Group B (TPR $=$ 70 percent, FPR $=$ 10 percent).
- **Intervention**: Lower $\gamma_{\text{cls},g=\text{B}}$ until $\text{TPR}_{g=\text{B}} = 90$ percent. (In US hiring, race- or sex-specific cutoffs are prohibited, so the calculation prices the trade-off rather than a deployable intervention.)
- **The cost**: Lowering the threshold increases false positives (hiring candidates who do not meet the bar).

Math:

1. Under the scenario's assumed threshold response, closing the 20 percentage-point TPR gap produces a 15 percentage-point increase in false positives for the disadvantaged group.
2. If the positive base rate is 20 percent, the value of a successful hire is \$100,000, and the cost of a bad hire is \$50,000, both sides of the intervention must be counted:
   - Group B baseline utility evaluates the original expected payoff: $0.70 \times 0.20 \times \$100{,}000 - 0.10 \times 0.80 \times \$50{,}000 = \$10{,}000$ per applicant.
   - Evaluating the intervention requires counting both marginal terms: $\Delta\text{Utility} = \Delta\text{TPR} \times \text{Base Rate} \times \text{Hire Value} - \Delta\text{FPR} \times (1 - \text{Base Rate}) \times \text{Bad Hire Cost}$.
   - The added true-positive value is \$4,000 per Group B applicant, while the added false-positive cost is \$6,000, producing a net loss of \$2,000.
   - Dividing this \$2,000 net loss by the $\$10{,}000$ baseline yields a 20 percent within-group utility loss. With Group B at 30 percent of applicants, the population-average loss is \$600 per applicant; calculating an aggregate percentage also requires the other group's baseline utility.

Systems insight: The "Price of Fairness" in this scenario is a 20 percent within-group utility loss, not a derivable aggregate percentage. The loss is not automatic: when a TPR gap reflects a miscalibrated threshold, closing it can raise net utility. It appears when the baseline threshold is near the utility optimum and marginal admissions skew unqualified. Engineering teams owe stakeholders both the Pareto frontier and the operational assumptions that produced it.

The calculation gives stakeholders a way to choose a point on the fairness-accuracy frontier, but it still does not explain any particular decision. When a loan applicant receives a rejection, stating that

"the model's true positive rate for this demographic group is 60 percent compared to 90 percent for other groups" provides no actionable information. The applicant needs to know *why* the application was rejected and *what* could be changed. These questions require explainability, which is the ability to articulate which input features drove specific predictions.

15.3.3 Explain and document: Decisions and models made legible

Explanation and documentation answer one demand at two timescales. An explanation makes a single decision legible, at the moment it is made, to the person it affects. Documentation makes the model legible across its lifetime to the engineers, auditors, and downstream teams who inherit it: what it was trained on, where it was validated, and what it must not be used for. Both exist for the same systems reason, since a guarantee that no one outside the team can inspect cannot be contested, audited, or safely reused.

15.3.3.1 Explainability requirements

A loan applicant denied credit by an algorithmic system may be entitled under applicable law to reasons or specific adverse-action factors, not merely aggregate statistics. **Explainability**[12] supports this capability: it enables human oversight of automated decisions, supports debugging when problems emerge, and can satisfy applicable requirements for decision transparency.

The level of explainability required varies by application context and regulatory environment, and the requirement constrains architecture rather than logging. When a rule requires specific reasons for an adverse action, a team can choose an inherently interpretable model, such as a sparse linear model, and accept whatever predictive capacity that structure gives up on the task. Alternatively, it can deploy a higher-capacity model with a post-hoc attribution method such as Shapley additive explanations (SHAP) or local interpretable model-agnostic explanations (LIME). The SHAP values that localized mispredictions during incident debugging now face an outside reader, which raises the bar. The explanation path adds serving latency, its fidelity to the model must be measured, and it yields a feature attribution rather than a causal account of the decision. Either way, the choice belongs in the design, because it determines which model family is trained and what the serving path must carry. Table 15.8 maps common deployment scenarios to their explainability needs.

Table 15.8: Explainability Considerations by Domain: Requirements vary by decision, actor, jurisdiction, and governing rules. The engineering challenge is matching explanation mechanisms to applicable requirements while managing risks such as gaming.

Application Domain	Explainability Level	Typical Requirements
Credit decisions	Specific reasons for covered actions	Principal reasons may have to be disclosed to the applicant
Medical diagnosis	Decision support	Support clinical review and applicable documentation
Content moderation	Context-dependent	Support notices or appeals where required
Recommendation	Context-dependent	Provide transparency appropriate to the use and governing rules
Fraud detection	Controlled disclosure	Balance applicable notice duties with adversarial-gaming risk

Selecting an explainability strategy requires balancing explanation fidelity, serving latency, and architectural flexibility across three distinct design paradigms:

Post-hoc explanation methods, including SHAP and LIME, generate feature importance scores for individual predictions without modifying model internals.[13] Model-agnostic explainers evaluate perturbed inputs or sampled feature subsets, introducing additional computation and approximation. Model-specific SHAP algorithms can exploit internal structure instead; the explanation method and where it runs determine latency and fidelity.

For the rejected loan application, a LIME fidelity check begins by defining a neighborhood of plausible applications, preserving valid feature ranges and dependencies such as the relation between income and debt-to-income ratio. The engineer fits the local surrogate on one set of perturbed applications and evaluates it on an independent set from the same neighborhood. Comparing its repayment scores with the original model's scores gives local prediction error; applying the same decision threshold to both gives approval/rejection agreement. Repeating the fit with different sampling seeds and defensible neighborhood widths tests whether the leading attributed features

[12] **Explainability and interpretability**: *Interpretability* usually describes how readily a person can understand a model or its behavior, often through a constrained structure such as a short rule list or sparse linear model. *Explainability* is broader and includes post-hoc methods such as LIME and SHAP. Neither property is automatic: a linear model with opaque features can be difficult to interpret, and a feature attribution is not a causal account of a decision. The systems implication is that intrinsic constraints affect model selection, while post-hoc methods add a computation and validation path whose cost depends on the method, model, and serving workflow. GDPR access and automated-decision provisions use the phrase "meaningful information about the logic involved" for covered processing (European Parliament and Council of the European Union 2016), leaving the appropriate technical approach dependent on the decision and governing law.

↩ SHAP values, which estimate how much each input feature contributed to one prediction, are established as a debugging tool in section 14.7.1.2.

and their signs remain stable. A surrogate that disagrees with the model near the applicant or changes its explanation sharply across these checks does not support a dependable reason for that decision. Acceptable error and stability depend on the application and must be specified before the check. This procedure evaluates a LIME surrogate; other attribution methods require checks appropriate to their own construction. Even a faithful account of model behavior does not establish causality or satisfy a legal explanation duty by itself.

Inherently interpretable models enforce transparency through structural constraints, using shallow decision trees, sparse linear models, or generalized additive models with inspectable feature effects. Interpretability still depends on model size and meaningful features; structural simplicity can limit expressive capacity on high-dimensional inputs. Internal representations such as attention weights do not constitute faithful post-hoc explanations of causal decision logic.

Concept-based explanations translate internal activations into human-interpretable semantic concepts rather than raw input coordinates. A concept vector is a direction in a layer's activation space learned from labeled examples of one concept. In a vision model, for instance, a "striped texture" direction separates the activations of striped and unstriped images. Projecting an image's latent vector onto that direction measures the representation's alignment with stripes. Establishing whether the prediction uses that component requires an additional check of how the output changes along the concept direction; the representation can encode stripes even when the output ignores them. The approach bridges low-level sensor features and high-level operational auditing, but it requires annotated concept sets to learn each direction.

13 **LIME (local interpretable model-agnostic explanations) and SHAP (SHapley additive explanations)**: LIME (Ribeiro et al. 2016) fits a local interpretable surrogate around each prediction; SHAP (Lundberg and Lee 2017) adapts Shapley values from cooperative game theory to compute feature contributions under a unified additive framework. Exact Shapley computation requires evaluating a feature's marginal contribution across all possible feature subsets, scaling exponentially with the feature count n_{feat} ($\mathcal{O}(2^{n_{\text{feat}}})$); practical production deployments therefore rely on polynomial-time model-specific algorithms (such as TreeSHAP for decision trees) or sampling approximations (such as KernelSHAP) to avoid unacceptable serving latency. The systems trade-off is that explanation fidelity, latency, and implementation complexity must be budgeted explicitly rather than treated as free.

Example 15.2: The hospital shortcut

Scenario: Pneumonia detection vision models trained at Mount Sinai achieved high internal accuracy but failed on external hospital datasets (Zech et al. 2018).

Diagnosis: The neural network learned shortcut features (hospital-specific scanner artifacts and text tags) rather than true biological lung pathology.

Systems lesson: Models exploit the path of least resistance in feature spaces. External validation is essential for detecting shortcut learning. Saliency maps, per-pixel heat maps of how strongly each input region influences the output, can provide supporting evidence but do not establish that the model learned the intended mechanism.

The hospital shortcut shows why interpretability is a systems requirement rather than presentation polish: teams need sufficient transparency to detect and isolate spurious correlations before deployment. Figure 15.4 partitions model families along a gradient continuum from intrinsic interpretability to post-hoc explanation. On the left, small trees and sparse linear or logistic models with meaningful features can expose decision logic to human inspection. Large trees or opaque engineered features can defeat that advantage. On the right, high-capacity representations, such as random forests, deep neural networks, and convolutional neural networks (CNNs) or transformers, often need supplementary explanation methods. LIME and SHAP require fidelity checks; attention maps alone do not establish faithful explanations. The spectrum organizes candidate approaches, not a categorical guarantee for any model family.

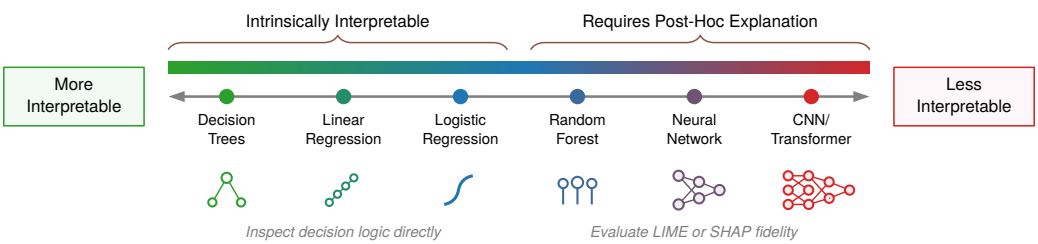


Figure 15.4: Model Interpretability Spectrum: A horizontal continuum groups decision trees, linear regression, and logistic regression as intrinsically interpretable, while random forests, neural networks, and CNNs/transformers require post-hoc explanation. The grouping is schematic: interpretability depends on model size and feature semantics, and any attribution method requires fidelity validation. It frames an auditability trade-off rather than a universal ranking of model quality.

The choice depends on the application's accountability requirements. Credit law requires specific reasons for covered decisions but does not mandate a particular model architecture, and other applications face different transparency, safety, or contestability duties; section 15.5.3 maps those duties to the capabilities they demand. The spectrum does not imply "simple is always better," because a highly interpretable model that makes inaccurate predictions may also cause harm. The engineering challenge is selecting a model and explanation process that meet the application's predictive and accountability requirements. An explanation, however, covers one decision at a time. The assumptions behind every decision the model makes need a durable record of their own, one that travels with the artifact.

15.3.3.2 Model documentation standards

Consider inheriting a production model without metadata: the model achieves 94 percent accuracy on a designated test set, but the origin of that test set, the composition of the training data, and the demographic or environmental slices evaluated are unrecorded. Without those baselines, updating or redeploying the model risks unmeasured regressions under distribution shift. **Model cards** provide a standardized interface specification for trained models[14] (Mitchell et al. 2019). Analogous to component datasheets in computer engineering, model cards document operational boundaries, evaluation conditions, and intended operating regimes alongside the model artifact.

A complete model card covers seven core sections that together formalize the model's operational envelope. It begins with technical details (architecture, training procedures, and hyperparameters) that enable reproducibility and auditing. It specifies *intended use* alongside explicit out-of-scope applications, preventing scope creep where models trained for consumer photo indexing get repurposed for high-stakes surveillance. The card then documents which *factors* (demographic groups, environmental conditions, or sensor variations) affect performance, guiding both evaluation slicing and production monitoring.

The remaining sections quantify and bound empirical behavior. Performance metrics must report disaggregated results across the operational factors identified in section 15.2.4, because aggregate accuracy conceals systematic subpopulation disparities. Documenting the training and evaluation datasets isolates potential sampling biases and defines the distribution over which the reported metrics hold. Finally, ethical considerations, caveats, and recommendations make deployment trade-offs explicit by recording known failure modes, unmodeled edge cases, and operational mitigations.

Applying this schema to MobileNetV2 illustrates how these categories translate into concrete deployment constraints. Table 15.9 fills five of the seven sections for an edge classification service, and its accuracy entries reuse the illustrative FP32 and INT8 values of the compression validation in section 12.11.1.4.

[14] **Model cards**: The primary failure mode model cards address is scope creep, the gradual expansion from "it worked for case A" to "try it for case B" without revalidating intended use. In practice, cards are often written after deployment decisions are made, documenting observed behavior rather than constraining it. The companion "Datasheets for Datasets" (Gebru et al. 2021) applies the same principle to training data. Without both, the card becomes a historical record rather than an engineering guardrail.

Table 15.9: Illustrative Model Card: MobileNetV2 for Edge Deployment: Abstract model card categories translate to practical documentation that guides responsible deployment decisions. The numerical entries are scenario assumptions, not reported benchmark results.

Section	Content
Model Details	MobileNetV2 architecture with 3.5M parameters, trained on ImageNet using depthwise separable convolutions. INT8 quantized for edge deployment.
Intended Use	Real-time image classification on mobile devices with less than 50 ms latency requirement. Suitable for consumer applications including photo organization and accessibility features.
Factors	Performance varies with image quality (blur, lighting), object size in frame, and categories outside ImageNet distribution.
Metrics	70.9% top-1 accuracy on ImageNet validation after INT8 quantization (full precision: 71.8%). Accuracy varies by category: 85% on common objects, 45% on fine-grained distinctions.
Ethical Considerations	Training data reflects ImageNet biases in geographic and demographic representation. Not validated for high-stakes applications (medical diagnosis, security screening). Performance may degrade on images from underrepresented regions.

Datasheets for datasets provide complementary documentation for the data tier (Gebru et al. 2021). In D·A·M terms, while a model card specifies the algorithmic and machine boundaries (including quantization precision and inference latency budgets), a datasheet captures the data

provenance, collection methodology, curation filters, and demographic composition. Together, these documentation standards establish the operational contract for the system. They state what the system is designed to tolerate and preserve the evidence that measurement produced, but both describe the system at the moment of release. Operation is the stage that keeps that description true once the model meets live traffic.

15.3.4 Operate: Monitoring and incident response

The Zillow Offers wind-down (war story 14.1) ended in a $304 million third-quarter 2021 inventory write-down[15] after Zillow's home-buying pipeline purchased houses at prices above revised estimates of their future selling prices (Zillow Group 2021). The forecasts set purchase prices, so by the time a forecast was revised, the commitments made on the earlier one could no longer be undone. Operational safety requires establishing incident response and monitoring before a model takes production traffic, rather than diagnosing failures after capital or social harm accumulates.

Table 15.10 extends the incident-response lifecycle to responsible deployment, where detection must surface fairness violations and demographic-slice degradation alongside service outages. The five components follow the standard incident-response lifecycle, but the operational safeguard lies in the predeployment verification column. The requirements column specifies what each component must do; the verification column defines what must be demonstrated before launch: an alert threshold tested against historical replay, an automated rollback path exercised under load, and an on-call rotation with documented escalation paths. A requirement without a verified control is an intention rather than a safeguard, making this verification gate the prerequisite for deployment.

15 **Zillow across the three axes**: Zillow's 2021 write-down is a useful systems case because the documented business failure combined forecast uncertainty with operational execution (Zillow Group 2021). A D·A·M diagnosis interprets the data axis as the mismatch between historical home-sale data and pandemic-era price volatility, the algorithm axis as the difficulty of pricing homes with reliable uncertainty estimates, and the machine axis as an automated iBuying pipeline that needed stronger capacity limits and circuit breakers. This is an engineering interpretation of Zillow's public disclosure, not a claim that Zillow identified one root technical cause.

↩ The ML incident-response sequence of severity classification, detection, impact assessment, mitigation, and root-cause analysis is established in section 14.7.1.1.

Table 15.10: Incident Response Framework: Systematic preparation for ML system failures requires five distinct components. Detection identifies anomalies through specialized monitoring; assessment evaluates scope using severity classifications; mitigation reduces harm through tested rollback procedures; communication notifies stakeholders through preapproved channels; remediation implements permanent fixes through root cause analysis. Each component requires both operational requirements and predeployment verification.

Component	Requirements	Predeployment Verification
Detection	Monitoring systems that identify anomalies, degraded performance, and fairness violations	Alert thresholds tested, on-call rotation established, escalation paths documented
Assessment	Procedures for evaluating incident scope and severity	Severity classification defined, impact assessment templates prepared
Mitigation	Technical capabilities to reduce harm while investigation proceeds	Rollback procedures tested, fallback systems operational, kill switches functional
Communication	Protocols for stakeholder notification	Contact lists current, message templates prepared, approval chains defined
Remediation	Processes for permanent fixes and system improvements	Root cause analysis procedures, change management integration

Service-health monitoring detects failures such as process crashes, out-of-memory errors, network timeouts, and HTTP 5xx responses. ML systems also accumulate technical debt that manifests as silent degradation (Sculley et al. 2015). An inference service can maintain sub-millisecond p99 latency with zero process exceptions while serving predictions computed over corrupted input schemas or uncalibrated weights. Furthermore, feedback loops can reinforce early erroneous predictions, amplifying small initial shifts into cascading errors across the user population. Incident response planning must account for these silent, ML-specific failure modes by deploying telemetry for input changes and outcome disparities, with response plans that account for label delay and the evidence needed to detect harm.

Production monitoring infrastructure already tracks performance stability, input distributions, and delayed outcomes. The responsibility lens adds two requirements to that telemetry surface. Subgroup parity becomes a first-class monitor, with false-negative and false-positive rates tracked per cohort over time, because aggregate accuracy masks localized regressions. User complaints, contested decisions, and appeals feed the same surface, because statistical aggregation over millions of queries smooths away the rare harms that affected people report first. Parity monitoring matters most where the system's decisions shape its own future training data. In that regime the bias

feedback model (principle 13) describes a disparity that evolves as $\Delta_g(k) \approx \Delta_g(0) \cdot \alpha_{\text{fb}}^k$ across feedback intervals rather than staying fixed. The loan example shows what that recurrence implies for a release gate.

↩ Production monitoring architectures for performance stability and distribution drift are established in section 14.6.

Napkin Math 15.3: Fairness as loop stability

Problem: After mitigation, the loan model from section 15.3.2.1 ships with a residual TPR gap of 3 percentage points, inside the scenario's 5 percentage-point release gate. Denied applicants generate no repayment labels in the routine training stream, so each quarterly retraining learns from a population the previous model selected. For this illustration, an independent audit sample provides observed repayment outcomes across both groups, including applicants not approved by the production policy. Without such outcome evidence or a justified missing-outcome correction, the all-qualified-applicant TPR gap would not be directly measurable. The next quarterly audit measures a gap of 3.3 percentage points. If the locally fitted growth factor persisted, when would the gap breach the release threshold?

Math:

1. **Fit the feedback factor**: $\alpha_{\text{fb}} = \Delta_g(1)/\Delta_g(0) = 1.10$ per quarter.
2. **Time to breach under the fitted model**: The projected gap exceeds the gate once $\Delta_g(0)\,\alpha_{\text{fb}}^k$ passes it, that is, for $k > \ln(\text{gate}/\Delta_g(0))/\ln\alpha_{\text{fb}}$. The model projects a breach after 6 quarterly retraining cycles, when the gap reaches 5.3 percentage points.
3. **Without mitigation**: The same factor would grow the original 30 percentage-point gap to 43.9 percentage points within 4 cycles, one year of retraining.
4. **Changing the loop**: If the team approves a small randomized sample of marginal applicants and observes their repayment, the loop again sees outcomes it previously censored. Suppose the refitted factor falls to 0.90; the same launch gap then decays to 1.6 percentage points over the same number of cycles.

Systems insight: A release gate passed at launch is a statement about $k = 0$. Under feedback, fairness is a stability property of the retraining loop. The projected breach time motivates a shorter audit interval, but two observations do not validate future dynamics. An intervention can change α_{fb} (through what data the loop collects) as well as $\Delta_g(0)$. The fitted factor is a local model; it must be refitted at every audit because intervention, saturation, or a shifting applicant pool changes the dynamics.

Telemetry infrastructure provides protection only when coupled with disciplined operational response. Automated dashboards require defined review cadences, clear on-call ownership, and explicit escalation procedures to ensure emerging anomalies trigger corrective engineering action rather than passive observation.

Checkpoint 15.3: Ethical deployment

Deployment is where safeguards must become operational.

Safety net

- ☐ **Rollback**: Verify that the previous model can be restored within the incident-response target.
- ☐ **Human-in-the-loop**: Verify that an operational routing path exists for human review of low-confidence predictions.

Monitoring plan

- ☐ **Silent failure**: Specify the subgroup metrics that will expose postdeployment bias.

Equitable performance across demographic cohorts addresses only one dimension of responsible engineering. The resource stage of the lifecycle addresses another, because operational expenditure and physical energy consumption constitute an equally critical constraint. Every training run, inference request, and monitoring pipeline draws electrical power that converts to carbon emissions and operating expenditure. An ML pipeline can exhibit demographic parity while consuming orders of magnitude more compute than the task warrants, imposing an unmonitored environmental footprint and inflating operating costs. Responsible engineering must govern both whom the system serves and the physical resources consumed in serving them.

15.4 Environmental and Cost Awareness

In 2019, researchers estimated that development-scale training and architecture search for a large natural language processing (NLP) model could emit as much carbon as five cars over their entire lifetimes (Strubell et al. 2019). Later analysis showed that the most quoted architecture-search estimate was highly sensitive to proxy-task, hardware, data-center efficiency, and grid carbon-intensity assumptions (Patterson et al. 2021). The correction sharpened the responsible-engineering point rather than weakening it: training runs consume megawatt-hours of electricity, inference at scale multiplies per-request inefficiencies into measurable environmental impact, and resource-intensive models exclude organizations that lack large compute budgets. The compression, acceleration, and measurement techniques that improve efficiency therefore serve double duty as instruments of responsible engineering, connecting computational efficiency to environmental sustainability, economic accessibility, and long-term scalability.

15.4.1 Efficiency as responsibility

Training a single large language model consumes thousands of GPU hours and energy measured in megawatt-hours. The bitter lesson predicts that this footprint will keep growing, because the methods that win are the ones that convert additional compute into capability. Much of this expense, however, is not intrinsic to the learning task but represents *accidental complexity*: training from scratch when fine-tuning would suffice, deploying larger architectures than tasks require, and running hyperparameter searches that explore redundant configurations. Computational cost depends on engineering choices as well as model physics. Green AI treats that efficiency as a primary metric rather than an afterthought.[16]

At the hardware level, environmental impact scales with total electrical energy draw across the machine lifecycle, mediated by accelerator utilization, data-center cooling overhead (power usage effectiveness), and local electrical grid carbon intensity. Within the accelerator package, data movement can dominate energy dissipation. In the 45 nm reference behind the energy-movement principle (7), a single off-chip DRAM access costs two to three orders of magnitude more energy than a 32-bit floating-point operation, although total energy also depends on access width, event counts, reuse, and the device. Algorithmic and runtime optimizations reduce environmental burden only when they reduce measured watt-hours for a verified quality target under production load, curbing this physical dissipation at the silicon level.

Physical resource demands also dictate deployment accessibility. Overparameterized models that require multi-accelerator nodes interconnected by high-bandwidth fabrics (such as NVLink) to host their memory footprints restrict operational control to centralized cloud facilities. Compressing models so their working sets fit within commodity DRAM or the unified memory of edge processors expands the deployment envelope. This local execution eliminates reliance on persistent, high-bandwidth network connectivity and recurring cloud API expenditures, enabling low-latency, privacy-preserving inference directly on client devices where data originates.

At production scale, where serving systems process millions of requests daily, per-query inefficiencies compound rapidly across accelerator fleets. Under the iron law of ML systems ($T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$), per-query execution latency dictates the request throughput that an individual accelerator can sustain before violating service-level latency objectives. Reducing execution time allows a smaller provisioned cluster to support the same aggregate traffic. Capacity planning can turn reduced service demand into lower provisioned-capacity costs by downsizing the active fleet. A fixed fleet may also save dynamic energy, although idle and baseline power remain. A valid comparison therefore measures total energy over the same traffic and quality target and distinguishes that result from savings on a capacity-based bill.

Optimization techniques achieve these goals by attacking specific terms in the execution and energy equations across the Data, Algorithm, and Machine layers. Quantization reduces numerical precision from 16-bit floating-point representations to INT8 or FP4, which can reduce memory traffic D_{vol} across the bus and increase arithmetic intensity when the representation is retained through supported kernels. The target hardware determines whether lower precision also improves arithmetic efficiency. Pruning can reduce the total operation count O and memory storage footprint when removed channels yield smaller dense tensors or a supported sparse format and kernels skip stored zeros. Knowledge distillation transfers the learned representations of an overparameterized

[16] **Green AI**: Schwartz et al. (2020) contrasted "Red AI" (performance at any cost) with "Green AI" (efficiency as primary metric). The compute-growth anchor comes from AI and Compute's 2012–2018 trend analysis (the fit that gives a 3.4-month doubling time), which reported a 300,000× increase in compute used in the largest AI training runs over that six-year window (Amodei and Hernandez 2018b); comparisons that extend to later frontier models span a longer window and report larger factors. The Green AI proposal (reporting FLOPs alongside accuracy for every published result) reframes efficiency from an engineering preference into a scientific reporting obligation, making the resource cost of marginal accuracy gains visible and comparable across research groups.

↩ The bitter lesson, the historical pattern in which methods that scale with compute overtake hand-built knowledge, is established in section 1.4.

↩ The physical energy disparity between off-chip memory access and arithmetic execution is established in section 11.2.3.

teacher into a compact student architecture, shrinking both operation count and memory footprint simultaneously. At the machine layer, supported hardware acceleration and legal kernel fusion can reduce intermediate-memory traffic, keeping selected activation tiles on chip to improve hardware efficiency η_{hw}.

Responsible engineering treats these optimizations as architectural design constraints established before training and deployment, rather than post-hoc remedies applied to an oversized model. System requirements specify explicit upper bounds on latency, memory capacity, and energy consumption alongside target predictive quality. The engineering task is finding the most compact architecture and execution strategy that satisfies those constraints.

15.4.2 Efficiency engineering in practice

Translating efficiency requirements into practice begins by establishing bounded machine envelopes (thermal design power, battery capacity, and peak latency) before choosing or training an architecture. Rather than deploying an oversized network and relying on post-hoc compression, responsible systems engineering treats hardware limits as primary architectural design constraints. Model selection prioritizes the most compact architecture that satisfies predictive quality, followed by targeted optimizations such as quantization or kernel fusion to minimize memory traffic and energy consumption.

Edge deployment scenarios make these physical limits immediate: passive thermal dissipation and battery capacity enforce boundaries that cannot be negotiated away. Consider an illustrative wearable device with a 500 mW peak power budget for its inference workload; sustained all-day operation on a watch-sized battery would demand an average draw far below that peak. Table 15.11 compares assumed constraints across four deployment contexts, from smartphones with 5 W budgets to IoT sensors operating at 100 mW.

Table 15.11: Illustrative Edge Deployment Constraints: The power and latency values are scenario assumptions for comparing deployment contexts; actual budgets depend on hardware, workload, and product requirements.

Deployment Context	Power Budget	Latency Requirement	Typical Use Cases
Smartphone	5 W	100 ms	Photo enhancement, voice assistants
IoT Sensor	100 mW	1 second	Anomaly detection, environmental monitoring
Embedded Camera	1 W	30 FPS (33 ms)	Real-time object detection, surveillance
Wearable Device	500 mW	500 ms	Health monitoring, activity recognition

Evaluating candidate architectures against these operating envelopes reveals whether an algorithm fits the target machine without exceeding its thermal dissipation or latency limits. Table 15.12 compares four illustrative model profiles under an assumed common hardware/runtime environment, using parameter counts, active inference power, and latency against the smartphone and IoT budgets defined in table 15.11. These power and latency values are scenario assumptions, not properties determined by model names. A small keyword-spotting model is not a substitute for a vision or health model merely because it fits the resource envelope; task quality and input compatibility remain prerequisites.

Table 15.12: Model Efficiency Comparison: Illustrative model profiles under the stated deployment assumptions. Model selection must account for accuracy, latency, power, memory, runtime support, and lifecycle impact; parameter count alone does not determine cost or environmental impact.

Model	Parameters	Inference Power	Latency	Fits Smartphone?	Fits IoT?
MobileNetV2	3.5M	1.2 W	40 ms	Yes	No
EfficientNet-B0	5.3M	1.8 W	65 ms	Yes	No
ResNet-50	25.6M	4.5 W	180 ms	No	No
TinyML Model	200K	50 mW	200 ms	No	Yes

For the wearable budget in table 15.11, the TinyML model leaves a 10× power margin, while MobileNetV2 exceeds the same power budget by 2.4× before accounting for sustained thermals. Fitting

the device envelope is necessary, but per-inference energy compounds into aggregate emissions and operational cost once deployed at scale. Techniques that lower measured energy per inference reduce lifecycle energy only when their savings outweigh any added training, preprocessing, or other workload costs, while cloud and cluster financial savings depend directly on provisioning and hardware utilization.

15.4.3 Total cost of ownership

A team spends $3,200 training a recommendation model and celebrates the modest cost. Six months later, they discover they are spending about $500,000 per year serving it. The surprise shows how total cost of ownership[17] can be dominated by recurring inference at high traffic. Other systems may be dominated by training, data, staffing, or idle capacity, so measurement determines where optimization should focus.

Consider a concrete example of a recommendation system serving 10M users daily. Training costs appear considerable: data preparation consumes 100 GPU-hours at approximately $4/hour ($400), hyperparameter search across multiple configurations requires 500 GPU-hours ($2,000), and the final training run uses 200 GPU-hours ($800). Total training cost reaches approximately $3,200.

Inference costs dominate in this scenario. With 10M users each receiving 20 recommendations per day, the system serves 200M inferences daily. Treating 10 milliseconds per inference as unbatched, non-overlapped dedicated GPU service demand yields approximately 23.1 GPUs running continuously. At $2.50/GPU-hour, the scenario's annual GPU cost reaches $506,944.

Over a three-year operational period, twelve total quarterly training cycles, including the initial cycle, produce training costs of approximately $38,400, while inference costs over the same period total $1.5M. The resulting 40:1 ratio is specific to these assumptions, but it dictates where optimization effort yields the highest return: inference latency and serving efficiency.

Per-query optimization becomes important when serving billions of requests. Shaving a fifth off the per-query accelerator service demand can reduce required hardware when other workload assumptions remain fixed. Hardware selection among CPUs, GPUs, and Tensor Processing Units changes costs and carbon footprint in workload- and deployment-dependent ways. Model compression through quantization and pruning can reduce high-volume inference costs when it lowers measured service demand without unacceptable quality loss.

Total cost of ownership (TCO) encompasses additional dimensions beyond computation. Operational costs include monitoring, maintenance, retraining, and incident response, all of which scale with system complexity and the rate of distribution shift in the application domain. Opportunity costs reflect that resources consumed by ML systems cannot be used for other purposes. Wasteful resource consumption in one project constrains what other projects can attempt.

Engineers should evaluate return on investment (ROI): whether the value an ML system delivers justifies its resource consumption. A recommendation system that increases engagement by 1 percent might not justify millions of dollars in computational costs, while a medical diagnosis system that saves lives does. Explicit trade-offs enable responsible resource allocation.[18]

15.4.3.1 TCO calculation methodology

Quantifying operational carbon impact requires measured or estimated energy use, facility overhead, and applicable grid carbon intensity. The ledgers that follow (table 15.13, table 15.14, and table 15.15) formalize the recommendation scenario just walked through, with the same traffic, service demand, and training budget, and add the two terms the walk-through left unquantified, operations and carbon. Engineers can estimate three-year total cost of ownership using a structured approach that separates training, inference, and operational costs into distinct accounting ledgers. Training is typically a periodic capital expense, inference recurs continuously with every user query, and operations accumulate through telemetry, retraining cycles, and incident response. The ledger methodology formalizes this accounting across training, serving, and operational infrastructure.

[17] **TCO (total cost of ownership)**: ML TCO includes labeling, monitoring, retraining, remediation, energy, audits, and compliance. At sufficient traffic, recurring inference can dominate a one-time training bill; utilization and update cadence determine the balance.



At production scale, recurring inference can dominate lifetime cost.

[18] **ML ROI (return on investment)**: Deployment-to-training cost ratios vary with traffic, utilization, hardware, update cadence, staffing, and incident burden. The appropriate model choice depends on measured lifecycle cost and task value rather than a fixed ratio.

Napkin Math 15.4: The carbon cost of compute

Problem: The TCO ledgers must convert GPU-only electrical energy into "kg CO_2eq." What carbon factor does one GPU-hour carry under the scenario baseline, excluding host, facility, and embodied impacts?

Variables:

- **Power**: 400 W per GPU (scenario baseline).
- **Intensity**: 0.4 kg/kWh CO_2eq (rounded grid baseline).

Math: Equation 15.2 captures the standard conversion:

$$\text{Carbon} = \text{Energy (kWh)} \times \text{Carbon Intensity (kg/kWh)} \tag{15.2}$$

Applying the baseline assumptions:

$$(0.4\ \text{kW} \times 1\ \text{hour}) \times 0.4\ \text{kg/kWh} = 0.16\ \text{kg}\ CO_2\text{eq per GPU-hour}$$

Systems insight: This GPU-only conversion factor lets the ledgers track a modeled accelerator-carbon subtotal alongside dollar cost. Whole-system and facility emissions require incorporating the host, facility overhead, and embodied energy boundaries.

Training costs Training costs include both initial development and ongoing retraining. Expanding hyperparameter search or retraining more often can improve robustness to distribution shift, but every added experiment or cycle repeats its full accelerator-hour cost. Table 15.13 assumes twelve total training cycles over three years: the initial cycle and eleven subsequent updates, each using the same data-preparation, search, and final-training budget.

Table 15.13: Training Cost Calculation: The modeled cycles include initial development and subsequent quarterly updates ($3,200 per cycle) over a three-year operational period. Data preparation, hyperparameter search, and final training consume GPU hours at $4/hour, totaling $38,400 across 12 training cycles. Despite appearing substantial, training represents only 1.9 percent of total cost of ownership.

Cost Component	Calculation	Financial Cost	Carbon (kg CO_2)
Initial data preparation	hours × rate	100 GPU-hr × $4 = $400	16 kg
Hyperparameter search	experiments × cost/experiment	50 × $40 = $2,000	80 kg
Final training	hours × rate	200 GPU-hr × $4 = $800	32 kg
Subtotal per training cycle		**$3,200**	**128 kg**
Total training cycles	cycles/year × years	4/year × 3 years = 12	same multiplier (12 cycles)
Total training cost	subtotal × cycles	**$38,400**	**1,536 kg**

Inference costs Inference costs scale directly with user traffic and model architecture. Techniques such as quantization, model distillation, and batching can reduce the accelerator time required per query, mitigating both financial costs and carbon emissions. Table 15.14 walks the conversion chain that turns traffic into physical impact: daily queries become GPU-seconds, GPU-seconds become GPU-hours, and GPU-hours convert into both dollars and carbon. Carbon attaches only once the workload is expressed in GPU-hours, so the query and GPU-second rows leave the carbon column blank by design rather than omitting data. Following the chain to the bottom row supplies the inference term for the total-cost comparison in table 15.16; dominance follows only when that term is compared with training and operations.

Table 15.14: Illustrative Inference Cost Calculation: In this unbatched, non-overlapped service model, 200M daily queries at 10 ms of dedicated accelerator demand require 556 GPU-hr daily, totaling $507K annually and $1.52M over three years. Real utilization, batching, and concurrency change this conversion.

Cost Component	Calculation	Financial Cost	Carbon (kg CO_2)
Daily queries	users × queries/user	10M × 20 = 200M	-
GPU-seconds/day	queries × service demand	200M × 0.01 s = 2M sec	-
GPU-hours/day	seconds ÷ SEC_PER_HOUR	556 GPU-hr	88.9 kg
Annual GPU cost	hours × 365 × rate	556 × 365 × $2.50 = $507K	32,444.4 kg
3-year inference cost	annual × 3	**$1.52M**	**97,333.3 kg**

Operational costs Operational costs encompass infrastructure, personnel, and incident response. ML systems add operational work around learned behavior: incident response includes investigating data drift, feature corruption, and distribution shifts, while monitoring must track model outcomes across demographic slices alongside the semantic and availability checks that other software also needs. Table 15.15 itemizes these ongoing expenses, which often surprise teams focused primarily on compute costs.

Table 15.15: Operational Cost Calculation: Illustrative operational costs include monitoring infrastructure ($50K/year), on-call engineering at 0.5 FTE ($100K/year), and incident response reserves ($20K/year). The $510K three-year total represents 24.6 percent of this scenario's TCO. Actual staffing and incident costs depend on system scale, risk, organizational structure, and support model.

Cost Component	Annual Estimate	3-Year Total
Monitoring infrastructure	$50K	$150K
On-call engineering (0.5 FTE)	$100K	$300K
Incident response (estimated)	$20K	$60K
Total operational		**$510K**

The stark breakdown in table 15.16 answers where the money goes: inference at 73.5 percent, operations at 24.6 percent, and training at only 1.9 percent.

Table 15.16: Total Cost of Ownership Summary: In this illustrative service-demand scenario, the inference-to-training cost ratio is 40:1. A 20 percent reduction in per-query accelerator demand saves about $304K and 19 t of modeled accelerator CO_2 under the same assumptions. The carbon subtotal excludes operations, facility overhead, and embodied impacts.

Category	3-Year Cost	Percentage	Modeled Accelerator Carbon
Training	$38K	1.9%	1.5 t
Inference	$1.52M	73.5%	97.3 t
Operations	$510K	24.6%	Not estimated
Total TCO	**$2.07M**	**100%**	**~98.9 t modeled subtotal**

Those proportions turn efficiency from a tuning preference into a responsibility check.

Checkpoint 15.4: Efficiency as responsibility

Total cost of ownership reveals where responsible optimization has the most leverage.

- ☐ **Inference dominance**: under this scenario's assumptions, can you compare a 20 percent reduction in the entire inference-cost ledger with a 50 percent reduction in the entire training-cost ledger, and explain why reducing only final-run time does not establish the latter saving?

- ☐ **Carbon accounting**: can you convert GPU-hours into kg CO_2eq using power draw and carbon intensity, and explain how workload design, hardware, scheduling, and region affect the result?
- ☐ **Sufficiency test**: for a given ML system, can you justify that the model size is appropriate for the task, or identify where a simpler model would deliver comparable accuracy at a fraction of the cost?

15.4.4 Environmental impact

The total-cost-of-ownership analysis in section 15.4.3 captures costs that appear on invoices, but computational resources carry costs that no invoice reflects. Operational emissions depend on measured energy, facility power usage effectiveness (PUE), and regional grid carbon intensity (gCO_2e/kWh); embodied manufacturing impacts require separate accounting. In power benchmarking, PUE separates server-plug energy from facility overhead; in carbon accounting, the same ratio becomes the multiplier that converts measured IT energy into the energy the facility actually draws:

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}}$$

An ideal PUE is 1.0, indicating that all facility energy reaches IT equipment. That denominator includes servers, storage, and networking, including server chassis fans already measured at the server plug. Facility overhead includes chillers, cooling towers, uninterruptible power supply (UPS) conversion losses, and power-distribution losses; counting server fans again as facility overhead would double-count their energy. Modern hyperscale facilities achieve optimized PUE values between 1.1 and 1.2, whereas older enterprise data centers frequently operate at 1.5 to 1.8, adding 50 to 80 percent to the measured IT-equipment energy boundary.

↩ The power measurement boundary that separates server-plug energy from facility overhead is established in section 12.9.1.

A first-order operational emissions estimate is $\text{Energy}_{\text{hardware}} \times \text{PUE} \times \text{Carbon Intensity}$, where the hardware term covers the full IT-equipment boundary over the accounting window, not GPU-only energy. Operational carbon includes electricity consumed by both active work and idle provisioned equipment. In contrast, embodied carbon encompasses the greenhouse gases emitted across the hardware supply chain (silicon wafer fabrication, packaging, transport, and facility construction) before a chip executes its first cycle. Embodied carbon can account for a large share of lifetime emissions, and on low-carbon grids a majority (Gupta et al. 2022), so prematurely replacing older servers for slight operational efficiency gains can paradoxically increase net lifetime emissions. Per-request energy is measured in joules per inference, whereas joules per FLOP characterizes hardware work efficiency. An optimization that reduces TCO may not reduce energy or emissions if it changes utilization, hardware, or workload placement. Data-center electricity use makes workload design, hardware, cloud region, and timing part of responsible engineering (Henderson et al. 2020).

Efficiency optimization and environmental responsibility align when measured lifecycle energy and emissions fall for the required workload and quality target. More granular carbon accounting methodologies build on this foundation: lifecycle assessment tracks impacts across the system's full life, scope 1/2/3 emissions separate direct emissions, purchased electricity, and supply-chain or use-phase emissions, and carbon-aware scheduling shifts work toward lower-carbon times or regions.

The same physical quantities that govern performance also affect responsibility. Data movement and computation contribute to chip-level energy, while data-center emissions additionally depend on utilization, facility overhead, grid intensity, and embodied impacts. Pareto analysis applies to fairness, latency, and carbon alike, and the first question is always whether the current design lies on the frontier, because a dominated design can improve several objectives at once. *Responsible engineering extends the constrained optimization problem this book has been teaching to objectives that include societal impact alongside throughput and latency.* The magnitude of the carbon term becomes clearer in a scale calculation for training a large foundation model; under the grid assumption of napkin math 15.5, one such run exceeds a hundred passenger-car years of emissions (see the margin).



This training scenario exceeds 100 passenger-car years of emissions.

Napkin Math 15.5: The carbon cost of scale

Problem: A foundation model is being trained at the scale of GPT-3, consuming 1,287 MWh of electricity (Patterson et al. 2021). What is the operational carbon impact under the stated grid-intensity assumption?

Math:

1. **Energy consumption**: 1,287 MWh = 1,287,000 kWh.
2. **Carbon intensity**: This scenario uses $\approx$ 429 g/kWh CO_2 (0.429 kg/kWh) (Patterson et al. 2021), a US grid-average figure slightly above the rounded 0.4 kg/kWh baseline of the TCO ledgers. Each calculation states its own grid assumption, and the difference changes the result by less than a tenth.
3. **Total emissions**: 1,287,000 kWh $\times$ 0.429 kg/kWh = 552,123 kg CO_2 (552 t).
4. **Comparison**: The scenario assumes 4.6 t CO_2 per passenger-car year. This per-year basis differs from the lifetime-car basis of the 2019 five-car estimate, so the two car comparisons are not directly comparable.

Systems insight: Under this training-energy scenario, the operational emissions equal the assumed annual emissions of 120 passenger cars. A 1 percent reduction in total training energy under the same grid mix would remove the equivalent of about 1.2 cars taken off the road for a year from the calculation.

Specification, measurement, documentation, and resource accounting define what a system must satisfy. The Govern stage of the lifecycle makes those guarantees provable to someone outside the team, which requires infrastructure that records validation results, preserves audit evidence, and enforces operational bounds in production.

15.5 Data Governance and Compliance

Recommendation and targeting models ingest user telemetry continuously at cluster scale, making a governance failure in the data pipeline an immediate failure in the ML system's ingestion and validation infrastructure. **Data governance**, the engineering discipline that turns regulatory and ethical obligations into verifiable technical controls, is the enforcement layer that makes accountability operational. Fairness metrics, model cards, and impact assessments have practical force only if the system can prove what data it ingested, which principals accessed it, which legal basis authorized processing, and whether applicable deletion or contestability requests were executed.

The Meta Ireland enforcement decisions make the governance constraint concrete: a system must establish a lawful basis for processing as well as protect the underlying data. In January 2023, the Irish Data Protection Commission issued separate EUR 210M and EUR 180M fines (totaling EUR 390M) against Meta Ireland. The decisions concerned unlawful reliance on contractual necessity for personalized advertising together with transparency and fairness violations, not a security breach (Data Protection Commission 2023).

Storage architectures also serve as governance enforcement mechanisms that determine which principals access data, how queries are tracked, and whether pipelines satisfy statutory requirements. Every architectural decision, from ingestion strategies through extract, transform, load (ETL) pipelines to persistent storage layout, carries governance implications that surface during regulatory audits, privacy investigations, or model recalls. Data governance translates abstract policy into concrete systems engineering through authenticated access controls on training partitions, immutable audit infrastructure that logs dataset access, privacy-preserving transformations during feature extraction, and deterministic lineage systems that trace training snapshots to deployed model checkpoints.

↩ The database, warehouse, and data-lake storage architectures, and the access patterns each one serves, are established in section 4.8.1.

Principle: Compliance as engineering constraint

Invariant: Data governance obligations are engineering requirements, not optional policy overlays.

Implication: Systems that process regulated data must map applicable duties to technical and organizational controls. Access control, erasure, contestability, audit, and lineage may be required or useful depending on the jurisdiction, data, actor, and use case. The General Data Protection Regulation (GDPR), California Consumer Privacy Act (CCPA), Brazil's General Data Protection Law (LGPD, *Lei Geral de Proteção de Dados*), and China's Personal Information Protection Law (PIPL) differ in scope and duties, so no single control checklist establishes compliance across them.

Data governance operationalizes security, privacy, compliance, and lineage as mutually dependent engineering constraints across four functional domains.

Security infrastructure protects raw features, training partitions, and model artifacts through mutual authentication, role-based authorization, encryption at rest and in transit, and hardware-rooted key management across the ingestion lifecycle.

Privacy mechanisms regulate information exposure even among authenticated consumers. Techniques such as differential privacy, k-anonymity, and automated feature redaction protect individual record subjects while preserving statistical signal for downstream model training.

Compliance frameworks translate jurisdiction-specific statutes (such as GDPR, CCPA, and LGPD) into hard architectural constraints that govern pipeline retention windows, consent verification, and cross-border data replication.

Lineage and audit systems construct the cryptographic trails that render the entire data lifecycle inspectable. Retained lineage linking deployed weights through training snapshots to authorized source records supports verification of security claims and applicable obligations; its required detail depends on the use case and evidence that must be reconstructed.

The KWS system introduced as the Tiny Constraint lighthouse illustrates how the fairness risks identified in table 15.4 intensify at the governance level. A deployed voice assistant captures ambient acoustic frames continuously in private spaces, maintains enrolled speaker profiles, and periodically syncs with cloud pipelines that train on population-wide audio corpora. These operational capabilities create governance obligations across consent management, data minimization, access auditing, and deletion rights.

In the KWS architecture, these four domains map directly to hardware and pipeline enforcement points. Security protects data across the edge-to-cloud boundary: hardware keystores on edge microcontrollers protect stored speaker templates, mutual TLS authenticates telemetry in transit, and role-based access controls gate access to centralized training audio in cloud storage. Privacy governs memory buffers and telemetry: circular SRAM buffers overwrite ambient audio continuously until an on-device keyword triggers detection, with an egress policy intended to retain or transmit audio only after activation. False wakes can still expose ambient speech, so the design must test false-activation rates and bound the audio window released, while differential privacy limits information leakage during fleet-wide acoustic model retraining. Compliance begins by determining the applicable legal basis and the scope of a revocation or erasure request. Authorized workflows then remove or restrict in-scope voice profiles and acoustic data according to documented retention and backup policies, with evidence of completion. Lineage and audit establish cryptographic provenance: content hashes and signed manifests link each deployed DS-CNN binary back through specific training splits and acoustic augmentation seeds to the original raw recordings.

Governance works as an operating model rather than a single control. The framework in figure 15.5 arranges eight elements, from organizational oversight and compliance policies to data catalogs, sourcing, quality metrics, runtime operations, and security, around a central governance hub. Technical mechanisms and organizational processes must function as mutually reinforcing subsystems: encrypted storage without identity-mapped access controls remains vulnerable to unauthorized exfiltration, and compliant storage without verifiable audit trails cannot survive regulatory inspection. Within the D·A·M taxonomy, data governance is the control plane of the Data

axis. It enforces lawful provenance, privacy bounds, and lifecycle integrity for the data that feeds the Algorithm, and it runs on Machine-axis infrastructure such as encrypted storage, access control, and audit logs.

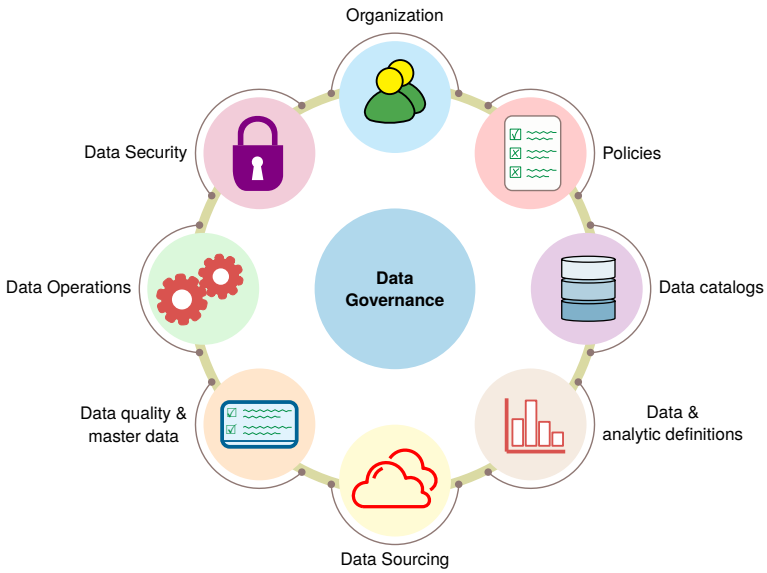


Figure 15.5: Data Governance Framework: Eight operating elements surround Data Governance as a central hub: policies, organization, security, operations, data quality and master data, sourcing, catalogs, and shared analytic definitions. The circular arrangement presents governance as a coordinated operating model rather than a single control, spanning organizational functions such as catalogs, sourcing, and analytic definitions alongside technical controls such as data security. Adapted from standard enterprise data governance frameworks.

15.5.1 Security and access control architecture

Consider a data scientist querying a feature store for training data. She can read aggregated voice features but cannot access the raw audio recordings from which they were derived. The serving pipeline can read online features for inference but cannot write to the training dataset. Neither can modify source data. The separation is intentional: it reflects a layered security architecture where governance requirements translate into enforceable technical controls at each pipeline stage. Feature stores enforce role-based access control (RBAC) by mapping organizational policies to authenticated identities and database permissions. These controls operate across distinct storage tiers: object storage enforces bucket-level access policies, data warehouses implement column-level security and dynamic masking for sensitive fields, and feature stores maintain isolated read and write paths that separate low-latency online inference key-value lookups from high-throughput offline batch training extracts.

Access control mechanisms remain incomplete without encryption, which protects data at rest and in transit but does not prevent misuse after decryption or authorized access. Training corpora stored in data lakes rely on server-side envelope encryption with keys managed through dedicated key management services (such as AWS KMS or Google Cloud KMS). Feature stores enforce symmetric encryption (such as AES-256) for tables at rest and TLS 1.3 for transport across services. Lighthouse KWS edge devices combine transport encryption for payload confidentiality with asymmetric code signing to verify model-update integrity; both controls require secure key management and update authorization.

↩ The dual-storage architecture of offline columnar stores and online key-value caches is established in section 4.8.5.

To protect data in use during training and inference, supported deployments can use **confidential computing** through hardware-based secure enclaves and trusted execution environments. Such a deployment isolates protected memory and execution from host software outside its trust boundary. Attestation provides evidence of the execution environment's identity and security state so that a client can check the approved configuration before releasing sensitive data or keys. Accelerator

execution and transfers require protection within the same declared boundary. The guarantee depends on the supported configuration, trusted application software, key management, and documented attacker boundary; it does not protect data from misuse by the application authorized to process it.

Serialized weights, ONNX (Open Neural Network Exchange) exports, and checkpoints require the same protection as training data. Deserializing unverified checkpoint files (such as Python `pickle` archives) introduces arbitrary code execution vulnerabilities on training nodes and inference workers. Restricted serialization formats (such as Safetensors) avoid the arbitrary object-execution mechanism of pickle, while write-protected promotion gates and cryptographic signatures (such as Ed25519) help verify artifact integrity and authorized provenance before loading. These controls do not establish that a model is free of backdoors or that every parser and runtime is secure.

Authorized loading also leaves the model's behavior to be evaluated. Poisoned training samples can induce backdoors, perturbed inference inputs can induce misclassification (evasion attacks), and systematic queries can extract model functionality (Tramèr et al. 2016). For privacy, the relevant exposure is membership inference: prediction APIs or raw weights can reveal whether a particular record was present in training (Shokri et al. 2017). Artifact integrity and limits on this information exposure require distinct controls.

Access control and cryptographic enclaves establish *who* can reach data and *how* it is protected across storage, transit, and execution. Controlling access is only half the problem: even authorized users can compromise individual privacy if the data itself is insufficiently protected.

15.5.2 Technical privacy protection methods

A data scientist with legitimate access to training data does not need, and should not see, individual user records when aggregate statistics suffice. Privacy-preserving techniques[19] address this gap by determining what information systems expose even to authorized users, adding a second layer of protection beyond access control. Differential privacy bounds changes in an exported output distribution under a specified neighboring-dataset relation.[20] One training mechanism is Differentially Private Stochastic Gradient Descent (DP-SGD). It clips each sample gradient to an ℓ_2 norm threshold C, then adds calibrated Gaussian noise with distribution $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ to the sum of clipped gradients, where σ is the noise multiplier. If the optimizer uses a mean, the noisy sum is divided by batch size B. A privacy accountant tracks the composed privacy bound across sampled training steps and checks the chosen epsilon budget.

Implementation determines the systems overhead. Naively materializing every sample's parameter gradient requires storage proportional to batch size times parameter count; small sequential microbatches can instead increase execution overhead. Fast or ghost clipping can obtain the required clipping factors without retaining a full batch of per-sample parameter gradients, and can retain ordinary batched operations where supported. There is no universal B-fold activation-memory penalty. Engineers must measure the chosen implementation's memory and throughput while preserving clipping, noise, sampling, and accounting assumptions.

Empirical validation methods such as membership inference attacks can supplement these formal checks by measuring whether an external observer can determine if a specific record was present in the training set, although an empirical attack cannot prove that an (ϵ, δ) bound holds mathematically.[21] Membership inference must be distinguished from model inversion attacks, where an adversary exploits continuous model confidence outputs to directly reconstruct the underlying private feature values or facial likeness of training subjects (Fredrikson et al. 2015). While membership inference determines binary presence (whether a specific record was included in the training corpus), model inversion exploits output probability distributions to reconstruct the underlying sensitive features. Reducing confidence-score precision or restricting outputs to top-k classes can reduce information exposed by some APIs, but neither is a complete inversion defense. Their value depends on the attacker's access and auxiliary information, so they must be evaluated against that threat model; differential privacy provides a separate bound under its stated mechanism and accounting assumptions.

While differential privacy bounds information leakage from model outputs, collecting continuous raw sensor streams creates an immediate vulnerability if raw data leaves the capture device. Always-listening edge architectures, such as the lighthouse KWS system, operate under severe privacy

[19] **Privacy-preserving techniques**: K-anonymity (Sweeney 2002) ensures each record is indistinguishable from at least $k_{\text{anon}} - 1$ others with respect to chosen quasi-identifiers, l-diversity adds attribute variety within equivalence classes (Machanavajjhala et al. 2007), and t-closeness bounds distribution distance (Li et al. 2007). These syntactic guarantees address different threats from differential privacy and do not by themselves protect against all ML-specific leakage: a model trained on de-identified data can still memorize examples or reveal membership signal under some conditions (Shokri et al. 2017). Differential privacy instead bounds the influence of one record under a specified mechanism and privacy budget, offering a semantic guarantee designed to remain meaningful in the presence of side information; the methods are therefore not a simple strength hierarchy, and the appropriate protection depends on the release, threat model, and utility requirements.

[20] **Differential privacy**: Introduced by Dwork et al. (2006), a randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ)-differential privacy if for all neighboring datasets X, X' differing by one record and output sets $\mathcal{S}$, $\Pr(\mathcal{M}(X) \in \mathcal{S}) \leq e^{\epsilon} \cdot \Pr(\mathcal{M}(X') \in \mathcal{S}) + \delta$. Here ϵ controls the multiplicative bound and δ is additive slack in the inequality; their acceptable values depend on the neighboring-dataset relation, release, and policy. The systems trade-off is utility rather than mere implementation complexity: stronger privacy often requires more noise or tighter sampling, and privacy loss composes across repeated queries or training steps. Engineers must therefore track cumulative privacy loss with a valid accountant and report the assumptions behind the chosen budget.

constraints because continuous ambient microphone sampling must coexist with strict minimization and retention boundaries. To reduce routine transmission exposure, this design confines acoustic feature extraction and wake-word detection to local low-power microcontrollers or digital signal processors (DSPs). Incoming audio samples stream through a small volatile circular buffer in on-chip SRAM, where older frames are continuously overwritten by newly sampled pulse-code modulation (PCM) data. The intended policy permits persistence or transmission only after an on-device trigger. Because detection can produce false wakes, validation must cover both the detector and the buffer and egress controls, including which pre-trigger audio becomes accessible after activation.

Confining raw audio to local edge memory enforces data minimization, but prevents centralized training pipelines from collecting diverse acoustic variations to improve model accuracy over time. Federated learning offers an alternative on training-capable clients, such as paired phones or adequately provisioned home hubs: they train locally and share model updates rather than pooling raw user audio.[22] In each training round, selected clients with sufficient memory, compute, and energy train on authorized local examples and transmit parameter deltas to a central coordinator. This is a different resource requirement from inference on the constrained KWS microcontroller. This architectural shift trades data centralization risks for severe physical systems bottlenecks. Uplink wireless bandwidth severely limits communication throughput when transmitting millions of parameters per round over cellular or Wi-Fi channels. Battery-powered client devices frequently drop offline mid-round as stragglers due to thermal limits or intermittent connectivity, and non-IID (not independent and identically distributed) data distributions across user devices cause local gradients to diverge, threatening model convergence. Furthermore, raw gradient updates remain susceptible to gradient inversion and reconstruction attacks (Zhu et al. 2019), so privacy controls must match the complete protocol and threat model. Compression and sparsification primarily reduce communication and do not by themselves provide a privacy guarantee.

Secure aggregation addresses direct exposure of client updates to the coordinator. Under the protocol's stated participation, dropout, and collusion assumptions, the coordinator learns an aggregate of participating clients' updates rather than inspecting each update separately (Bonawitz et al. 2017). Information revealed by that aggregate remains outside this protection, and neither the aggregate nor the final model becomes private merely because aggregation was secure. Differential privacy addresses this release boundary by bounding the effect of changing one record or one client's contribution, according to the chosen neighboring-dataset relation. The training mechanism and privacy accountant must support that relation and compose the releases the design actually exposes. A design that needs both protections must therefore account for what a curious coordinator can observe during training and what recipients can infer from released aggregates or model checkpoints.

When triggered audio recordings or aggregated model checkpoints must be retained on centralized infrastructure for verification or downstream training, privacy protection shifts to automated lifecycle management. Automated retention workflows apply time-to-live policies across primary stores, caches, training partitions, and backups. Cryptographic erasure destroys the keys needed to decrypt covered copies; physical overwrite is a distinct storage-sanitization mechanism. Neither removes every copy automatically. The workflow must inventory replicas, key copies, and backups, define applicable retention exceptions, and verify that in-scope data is no longer recoverable under the chosen threat model.

15.5.3 Regulation as architecture

Retention and erasure pipelines exist to satisfy legal obligations as much as engineering preferences. Regulation more broadly changes responsible engineering from a best-practice argument into architecture constraints. Imagine a credit model denies an applicant and the applicant asks why, contests the decision, and later requests access to the data used about her. The system must do more than report an accuracy score. It must produce an explanation tied to the specific decision, preserve the model and data lineage that led to that output, route the dispute to a substantive human review path, retain audit logs, and support deletion or access workflows where data rights apply. Responsible engineering now operates within explicit regulatory frameworks that turn transparency, oversight, and accountability into technical requirements.

Regulation first enters the architecture through risk classification. The EU AI Act establishes a comprehensive framework, classifying AI systems by risk level and mandating requirements

21 **Membership inference attack**: The attack uses model behavior to infer whether a record appeared in training; overfitting can increase the signal (Shokri et al. 2017). In practice, higher confidence on training samples can help distinguish members from nonmembers. Its measured success depends on model access, data distribution, and defense assumptions. A successful empirical attack reveals practical leakage, but it does not alone prove violation of a stated (ϵ, δ) bound. Formal differential privacy comes from the mechanism and its accounting.

22 **Federated learning**: McMahan et al. (2017) introduced Federated Averaging (FedAvg), in which each device trains locally and shares model updates rather than raw records. This architecture can limit routine transfer of raw data, but it is not a privacy guarantee: model updates can leak training information through reconstruction attacks (Zhu et al. 2019). Federated learning may therefore be combined with secure aggregation, differential privacy, and retention controls in a defense-in-depth design whose guarantees depend on the complete protocol.

accordingly.[23] The Act entered into force on August 1, 2024 and applies in phases: prohibited-practice rules began applying in 2025, while high-risk and other operator obligations phase in by system category and implementation guidance. Article 99 sets maximum fines of EUR 35 million or 7 percent of global turnover for prohibited AI practices, while many other operator obligations are capped at EUR 15 million or 3 percent (European Parliament and Council of the European Union 2024).

For systems engineers, compliance hinges on architectural capabilities rather than fine schedules. Covered high-risk systems[24] must implement risk management, data governance, technical documentation, transparency, human oversight, and accuracy, robustness, and security requirements. A covered credit-decision system therefore needs auditability from inception: model versions, training data provenance, validation evidence, human-oversight design, logging, and postdeployment monitoring must be part of the architecture rather than documents assembled after launch.

Contestability adds a second architectural requirement. GDPR moves the same applicant workflow into data-subject rights. Article 22 grants EU data subjects the right not to be subject to decisions based solely on automated processing that produce legal or similarly significant effects, subject to specified exceptions and safeguards.[25] Article 15(1)(h) separately gives data subjects access to meaningful information about the logic involved in automated decision-making referred to in Article 22. Engineering teams should determine which decisions are covered and design the required explanation and human-review capabilities accordingly. Where a substantive review path is required, it must be operationally staffed and supported by summaries, provenance, and audit tools. That staffing scales with decision volume. If 0.1 percent of 1M daily decisions are appealed or escalated, trained reviewers must work through 1,000 cases/day, a workload assumption rather than a model-error rate, and the review tooling must surface each case's explanation, lineage, and model version fast enough for that queue to clear.

US sectoral law reaches similar capabilities through domain-specific evidence requirements. These regulations are less unified than the EU AI Act, but they can impose related engineering duties. In the credit example, the Equal Credit Opportunity Act (ECOA) and its implementing Regulation B require specific principal reasons for covered adverse actions, and those reasons must accurately describe factors actually considered or scored (12 C.F.R. § 1002.9; official interpretation). If consumer-report information or credit scores influence the decision, the Fair Credit Reporting Act (FCRA) adds notice obligations; if the same scoring machinery is used for housing, the Fair Housing Act (FHA) adds a discrimination-prohibition constraint. The technical consequence is practical rather than abstract: covered systems need evidence and controls that support the applicable reasons, notices, review, and nondiscrimination duties.

Healthcare regulations, including the Health Insurance Portability and Accountability Act (HIPAA)[26] and Food and Drug Administration (FDA) guidance, impose the same pattern with different artifacts: protected-health-information controls, validation records, audit logs, and incident response. Employment systems likewise require evidence that automated screening does not reproduce discriminatory hiring practices. Across domains, the task is to translate each obligation into a concrete capability: explanation, human review, lineage, access control, deletion, monitoring, or incident response. These operational safeguards constitute a common production contract across regulatory regimes rather than a jurisdiction-specific checklist.

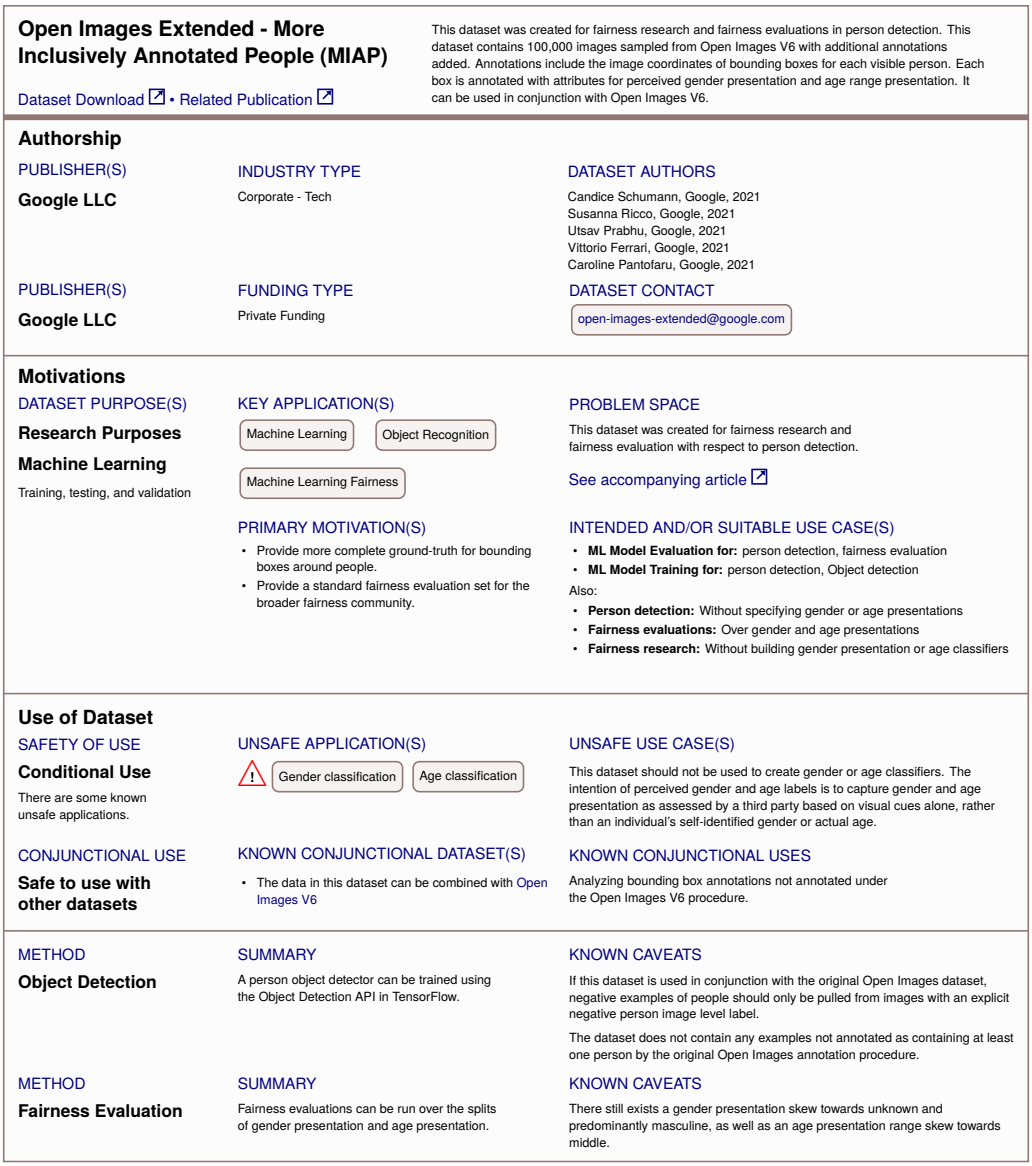
Data-protection law reaches deeper into the pipeline than decision-level duties do, because its obligations attach to the data itself rather than to a decision. When a European user invokes an applicable right to erasure under GDPR, the voice assistant must determine which personal data and downstream artifacts are in scope and execute the appropriate workflow within the regulation's response deadlines (European Parliament and Council of the European Union 2016). These obligations become architecture constraints in pipeline design, storage choices, and operational procedures. GDPR's data minimization principle requires limiting collection and retention to what is necessary for stated purposes. Article 15 access rights cover personal data undergoing processing and specified information about that processing, not every artifact merely associated with a user.

Voice assistants operating globally face overlapping regulatory regimes because requirements vary by jurisdiction and apply differently based on user age and data sensitivity. GDPR cross-border-transfer rules permit transfers through adequacy decisions, appropriate safeguards, or limited derogations rather than imposing categorical data localization. These rules can still drive regional

[23] **EU AI Act (Regulation 2024/1689)**: The first comprehensive AI legal framework, defining four risk tiers with penalties that vary by infringement category. The Act has extraterritorial reach: non-EU organizations may need to comply when they place systems on the EU market or when system outputs are used in the EU. Systems engineering implications are concrete: high-risk AI requires logging infrastructure for audit trails, human oversight mechanisms built into the architecture, and CE marking, all capabilities that must be designed in from inception, not retrofitted after deployment.

[24] **High-risk AI (EU AI Act Annex III)**: Annex III enumerates specified use cases in areas including biometrics, critical infrastructure, education, employment, essential services, law enforcement, migration, and justice. Classification depends on intended use and the Article 6 criteria, including exclusions for some systems that do not materially influence decisions or pose significant risk; profiling systems within Annex III remain high-risk. Model architecture alone does not determine classification.

[25] **GDPR (General Data Protection Regulation) articles 15 and 22**: Article 22 restricts certain solely automated decisions with legal or similarly significant effects and, for specified exceptions, requires safeguards including human intervention, the ability to express a point of view, and the ability to contest the decision; article 15(1)(h) contains the access right to meaningful information about the logic involved in such automated decision-making (European Parliament and Council of the European Union 2016). The European Data Protection Board's guidance emphasizes that required human oversight must be substantive and not merely a "rubber-stamping" exercise (European Data Protection Board 2018).



Open Images Extended - More Inclusively Annotated People (MIAP)

Dataset Download • Related Publication

This dataset was created for fairness research and fairness evaluations in person detection. This dataset contains 100,000 images sampled from Open Images V6 with additional annotations added. Annotations include the image coordinates of bounding boxes for each visible person. Each box is annotated with attributes for perceived gender presentation and age range presentation. It can be used in conjunction with Open Images V6.

Authorship

PUBLISHER(S): **Google LLC**

INDUSTRY TYPE: Corporate - Tech

DATASET AUTHORS: Candice Schumann, Google, 2021; Susanna Ricco, Google, 2021; Utsav Prabhu, Google, 2021; Vittorio Ferrari, Google, 2021; Caroline Pantofaru, Google, 2021

PUBLISHER(S): **Google LLC**

FUNDING TYPE: Private Funding

DATASET CONTACT: open-images-extended@google.com

Motivations

DATASET PURPOSE(S): **Research Purposes**; **Machine Learning**; Training, testing, and validation

KEY APPLICATION(S): Machine Learning; Object Recognition; Machine Learning Fairness

PROBLEM SPACE: This dataset was created for fairness research and fairness evaluation with respect to person detection.

See accompanying article

PRIMARY MOTIVATION(S):
- Provide more complete ground-truth for bounding boxes around people.
- Provide a standard fairness evaluation set for the broader fairness community.

INTENDED AND/OR SUITABLE USE CASE(S):
- **ML Model Evaluation for:** person detection, fairness evaluation
- **ML Model Training for:** person detection, Object detection

Also:
- **Person detection:** Without specifying gender or age presentations
- **Fairness evaluations:** Over gender and age presentations
- **Fairness research:** Without building gender presentation or age classifiers

Use of Dataset

SAFETY OF USE: **Conditional Use**; There are some known unsafe applications.

UNSAFE APPLICATION(S): Gender classification; Age classification

UNSAFE USE CASE(S): This dataset should not be used to create gender or age classifiers. The intention of perceived gender and age labels is to capture gender and age presentation as assessed by a third party based on visual cues alone, rather than an individual's self-identified gender or actual age.

CONJUNCTIONAL USE: **Safe to use with other datasets**

KNOWN CONJUNCTIONAL DATASET(S):
- The data in this dataset can be combined with Open Images V6

KNOWN CONJUNCTIONAL USES: Analyzing bounding box annotations not annotated under the Open Images V6 procedure.

METHOD: **Object Detection**

SUMMARY: A person object detector can be trained using the Object Detection API in TensorFlow.

KNOWN CAVEATS: If this dataset is used in conjunction with the original Open Images dataset, negative examples of people should only be pulled from images with an explicit negative person image level label.

The dataset does not contain any examples not annotated as containing at least one person by the original Open Images annotation procedure.

METHOD: **Fairness Evaluation**

SUMMARY: Fairness evaluations can be run over the splits of gender presentation and age presentation.

KNOWN CAVEATS: There still exists a gender presentation skew towards unknown and predominantly masculine, as well as an age presentation range skew towards middle.

Figure 15.6: Data Governance Documentation: The populated Open Images Extended—More Inclusively Annotated People card organizes authorship, motivation, intended and unsafe uses, conjunctional use, and method caveats in one artifact. Recording permitted and prohibited uses makes the card reviewable governance evidence, though it does not establish compliance by itself. Adapted from the Open Images Extended—MIAP Data Card in Pushkarna et al. (2022).

storage, replication, and processing choices. A structured data card turns these operational metadata requirements into reviewable contract fields. The card in figure 15.6, adapted from the Open Images Extended (MIAP) specification (Pushkarna et al. 2022), partitions authorship, primary motivations, explicit safety prohibitions (such as forbidding age or gender classification), and methodological caveats. Although documenting these boundaries does not alone guarantee statutory compliance, it provides the verifiable baseline necessary for pipeline ingestion checks.

The engineering response to these regulatory requirements is proactive architectural design. Teams that build documentation, monitoring, explainability, and human oversight into systems from inception demonstrate compliance efficiently. Teams that must retrofit these capabilities face expensive pipeline redesigns or deployment blocks. Nearly every regulatory obligation eventually

asks the infrastructure the same two questions: which data produced this model, and who touched that data under what authority. Lineage and audit systems answer them.

15.5.4 Building data lineage infrastructure

Translating the structured documentation tiers from figure 15.6 into executable pipeline gates converts static declarations into enforceable runtime invariants: declared authorship, permissible use boundaries, and safety prohibitions dictate which training jobs may consume specific feature partitions and which artifact dependencies must be audited. Compliance obligations are only as credible as the infrastructure that demonstrates them. When a regulator audits which training data produced a deployed model or a user invokes an applicable right to erasure, the organization must answer with engineering precision, not manual investigation. Data lineage provides this capability. The lineage graph built for reproducibility in section 4.6.4 and section 14.4.1.3, a directed acyclic graph (DAG) of artifact versions and transformation tasks, is the same structure an audit queries, which turns static documentation into queryable infrastructure. Lineage systems such as Apache Atlas and DataHub[27] integrate with pipeline orchestrators such as Airflow and Kubeflow to capture relationships automatically. When an Airflow DAG reads audio files from object storage and transforms them into spectrograms, the lineage system records each step and traces every feature tensor back to its source audio file. Lineage tracking also enables downstream impact analysis. When an erasure request arrives, traversing the lineage graph identifies candidate derived artifacts (cached spectrograms, feature embeddings, and checkpoint weights) for scope determination and possible invalidation, retraining, or legal review.

A production KWS system implements lineage tracking across the data engineering lifecycle. Source audio ingestion creates lineage records linking each audio file to its acquisition method, supporting verification of consent requirements. Processing pipeline execution extends lineage graphs as audio becomes features and embeddings, and each transformation records code versions and hyperparameters. Training jobs create lineage edges from feature collections to model artifacts, recording which data versions trained which model versions. When a voice assistant device downloads a model update, lineage tracking records the deployment, supporting recall if training data is later discovered to have quality or compliance issues. Lineage captures provenance, but accountable operation also requires access history: who touched the data, when, and under which authority.

15.5.5 Audit infrastructure and accountability

Accountability trails[28] record access events to satisfy statutory obligations such as HIPAA and the Sarbanes-Oxley Act (SOX). In production machine learning systems, however, capturing every access event creates an acute write-amplification bottleneck. For a fixed event schema, retained volume depends on recorded events per unit time, bytes per event, and the retention window. More accesses or instrumented stages can increase the event count, while logging full feature payloads and intermediate activation tensors greatly increases bytes per event. This workload can exceed ingestion or storage budgets even when the source dataset is unchanged. Separating audit metadata from payload data controls one major multiplier.

Production keyword spotting architectures resolve this trade-off through a tiered audit topology that reconciles capture fidelity with storage overhead and edge bandwidth limits.

At the edge tier, microcontrollers operating within strict SRAM budgets log local state transitions, wakeup timestamps, and power states to volatile circular ring buffers. To conserve radio energy and uplink bandwidth, devices batch telemetry over secure transport rather than streaming it continuously. Scheduled maintenance windows or charging cycles can provide upload opportunities only if the required evidence survives until then.

The retention requirement determines which events can use this volatile path. Dividing usable buffer capacity by the encoded telemetry byte rate gives the time to fill an empty buffer; the upload schedule must leave room for bursts and retries. Summaries may discard detail only when the retained fields still answer the specified audit questions. Events that must survive power loss require durable local storage until central storage acknowledges durable receipt, adding write energy and storage wear to the device budget. Sequence numbers and restart markers expose missing intervals after reconnection; an incomplete interval cannot establish that no incident occurred.

[26] **HIPAA (Health Insurance Portability and Accountability Act)**: Enacted in 1996, with Privacy Rule and Security Rule requirements establishing standards for protected health information (United States Congress 1996; U.S. Department of Health and Human Services 2003, 2005); PHI may be used for training under applicable authorization or another permitted pathway, such as an IRB or Privacy Board waiver; de-identified data and limited data sets with data-use agreements provide additional pathways. Model outputs may remain PHI when they identify or can reasonably identify individuals, and security-rule documentation retention must be reflected in audit and evidence design. Civil money penalties are tiered and inflation-adjusted by regulation (U.S. Department of Health and Human Services 2026).

[27] **Data lineage systems**: Apache Atlas and DataHub capture metadata about data flows from pipeline execution, creating graphs in which nodes are datasets and edges are transformations. GDPR Article 30 requires records of processing activities (European Parliament and Council of the European Union 2016), not automated model-level lineage specifically. Lineage nevertheless helps identify candidate derived artifacts for deletion, restriction, retraining, or legal review when an applicable request arrives.

↩ The lineage graph that traces raw records through versioned data, code, and model artifacts to deployed endpoints is established in section 14.4.1.3.

↩ Transformation lineage, which records the code version, parameters, and processing history behind each dataset, is established in section 4.6.4.

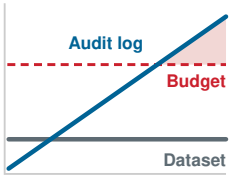
If retention requirements exceed the available storage or uplink budget, the design must reduce optional telemetry or provision additional capacity before deployment.

Once telemetry reaches centralized infrastructure, auditing shifts from bandwidth-constrained edge devices to high-throughput feature stores. At this feature tier, offline and online feature stores emit append-only audit events for every read and write transaction. Rather than duplicating high-dimensional feature vectors, these events record requesting service identities, authorized subject pseudonyms, feature group hashes, and read timestamps. This metadata-only logging strategy enables fine-grained access verification while keeping log volume proportional to access transactions rather than payload sizes.

From feature stores, data flows into training pipelines running across accelerator clusters. At this training tier, cluster schedulers record deterministic manifests mapping training jobs directly to immutable data partition hashes, git commit SHAs, and hyperparameter configurations. While these manifests support checks of pipeline integrity and reproducibility, they reveal an inherent systems boundary: access logs confirm which records were processed, but they cannot demonstrate whether deleted user data persists within learned model weights. If the applicable erasure obligation covers a deployed model artifact, teams must evaluate retraining, validated **machine unlearning** (removing a training record's influence from learned weights without retraining from scratch (Cao and Yang 2015; Bourtoule et al. 2021)), or other legally appropriate controls; database access logs alone cannot establish removal of the training contribution.

Regulatory requirements extend audit responsibility from data access to prediction-time inference behavior. Reconstructing why a specific applicant was denied a loan in section 15.3.2.1, or why an edge voice assistant triggered a false wake-word activation, requires auditing the exact operational state at inference time. Without inference-time logging, an audit trail may confirm database access without capturing which model checkpoint, decision threshold, and runtime features produced a given decision. Archiving raw input tensors for every inference request would impose unsustainable storage write burdens; production serving systems instead record compact, purpose-limited decision manifests. These manifests capture the active model version hash, runtime decision threshold, output confidence scores or reason codes, and cryptographic hashes of input features, supporting indexed retrieval of decision evidence. An input hash identifies content but cannot reconstruct it; replay also requires an appropriately retained source or an authorized recoverable representation, along with the relevant execution artifacts.

Together, the four governance domains (security, privacy, compliance, and lineage with audit) enforce Data-axis obligations through Machine-axis infrastructure. They provide the physical telemetry and boundary enforcement needed to verify algorithmic constraints, track data provenance, and preserve accountability across the model lifecycle. Comprehensive infrastructure still does not protect a team from its own intuitions, and the recurring ones are treating audit logs as passive compliance paperwork, assuming aggregate metrics guarantee subgroup fairness, and believing responsible constraints can be retrofitted after deployment.



Audit volume can outgrow the storage budget while the dataset stays flat.

[28] **Audit trail**: Audit integrity requires controls against unauthorized alteration, but retention and deletion duties vary by record type and law; append-only object storage with retention controls, or cryptographic hash chains, can provide tamper evidence without creating a universal rule that records are never deleted. A large platform may log billions of events daily; HIPAA's Security Rule also imposes multi-year documentation-retention obligations for required policies and procedures (U.S. Department of Health and Human Services 2005). Retention planning must distinguish records that must be preserved from personal data that must be minimized or deleted.

15.6 Fallacies and Pitfalls

Teams can still fail after assembling assessment frameworks, fairness metrics, explainability mechanisms, efficiency analyses, and governance infrastructure. A team may retrofit fairness after benchmark success, trust aggregate accuracy, or treat compliance evidence as paperwork rather than system behavior, drawing on intuitions from traditional software engineering where bugs are local and testing is deterministic. Recognizing these failure patterns early, before a fallacy shapes a design decision, is far cheaper than discovering it after deployment.

Fallacy: *Responsibility can be addressed after the system achieves technical objectives.*

Teams assume fairness constraints can be retrofitted once models demonstrate strong benchmark performance. In production, early architectural decisions constrain what interventions remain feasible. Amazon's recruiting tool, abandoned after remediation failed (section 15.2.1), is the case in point. Organizations deferring responsibility may face redesign, deployment with documented risks, or cancellation. Integrating fairness constraints early can be less costly than retrofitting them after data contracts, monitoring, and release gates are fixed.

Pitfall: *Relying on aggregate metrics to assess fairness.*

Engineers assume high overall accuracy indicates the system works well for all users. The flaw of averages (section 15.3.2) reveals this intuition fails: aggregate metrics can conceal large subgroup disparities (section 15.2.4). The loan approval analysis in section 15.3.2.1 showed a 30 percentage-point TPR gap, with qualified minority applicants rejected at 4× the majority-group rate. These disparities can persist undetected when standard monitoring tracks only aggregates. Production systems require disaggregated evaluation with application-specific thresholds, such as the 5-percentage-point TPR threshold used in the loan example.

Fallacy: *Removing sensitive attributes from training data eliminates bias.*

Teams remove gender, race, and protected attributes expecting this ensures fairness. Other features can act as proxy variables when they correlate with sensitive characteristics. ZIP codes, purchase patterns, browsing history, college names, and language choices can carry indirect demographic signal. Amazon's recruiting tool kept encoding sex through college and organization names after explicit gender terms were removed (section 15.2.1), and the care-management algorithm (war story 15.2) reproduced racial disparity through its cost target, a proxy that no attribute filter would remove. Removing protected attributes does not by itself establish fairness.

Pitfall: *Treating documentation as sufficient accountability.*

Teams invest effort in model cards, then consider responsibility requirements satisfied. Documentation provides transparency (section 15.3.3.2) but not enforcement. A model card specifying "not validated for high-stakes decisions" has no effect when the system is repurposed for loan approvals without technical restrictions. Accountability requires operational integration: monitoring dashboards, documented subgroup-disparity alert thresholds, incident response procedures, and access controls preventing deployment beyond validated use cases.

Fallacy: *Responsible AI is primarily a legal compliance issue.*

Teams treat responsibility as external oversight rather than engineering practice. Engineering decisions made months before legal review constrain the solution space more than any compliance assessment. Architecture selection determines what fairness interventions are feasible, while data pipeline design establishes whether disaggregated evaluation is even possible. As section 15.2.5 establishes, systems designed with responsibility as an engineering objective enable efficient validation; systems where responsibility is added at late-stage review face redesign or deployment with documented risks.

Pitfall: *Measuring the environmental impact of training but not inference.*

Public discourse focuses on training-run carbon, and engineers often follow this framing when assessing environmental responsibility. The illustrative TCO analysis in section 15.4.3 shows why this focus is incomplete: under its unbatched service-demand assumptions, the inference-to-training cost ratio is about 40:1. A model trained periodically but served millions of times daily can have a lifecycle footprint dominated by inference rather than training. For the recommendation system analyzed in table 15.16, training accounts for 1.9 percent of three-year costs while inference accounts for 73.5 percent. In this example, inference emits about 63× as much CO_2 as training. Engineers who optimize training efficiency while ignoring per-query inference demand can leave the larger term in this scenario unexamined. These values are scenario-dependent, but they show why lifecycle accounting must include both terms.

Fallacy: *Model weights are exempt from data governance and deletion requests.*

Teams often assume that once training data has been compiled into model weights, the data is gone and compliance obligations cannot reach the artifact. In reality, models can memorize training data, and membership inference attacks can reveal whether a sensitive record appeared in the training set (Shokri et al. 2017). Unlike a relational database where fulfilling an erasure request is an isolated delete query, a neural network diffuses each training point's statistical influence across millions or billions of parameters through gradient updates. Verifiably purging that influence through the machine unlearning introduced with audit infrastructure (section 15.5.5) forces stark systems trade-offs. Exact retraining from scratch on a properly cleaned dataset provides a reference for removing

the excluded records' contribution to that training run, but it re-expends the entire training budget (potentially thousands of accelerator-hours) along with its associated carbon emissions, while failing to erase other copies or facts inferable from retained data. Machine unlearning techniques aim to bound these computational costs, but verifying that an intervention eliminates statistical influence without degrading model utility remains an active systems challenge. Engineering teams must track data-to-model lineage and evaluate retraining, machine unlearning, or compensating controls with legal and privacy specialists whenever an erasure request affects deployed artifacts.

Across these failures, the recurring mistake is to treat responsibility as a document, metric, or late-stage review rather than a system property. Measurable constraints, continuous monitoring, and enforceable governance carry responsibility through the full lifecycle.

15.7 Summary

Responsible engineering is ML systems engineering done completely, not a separate discipline. The chapter traced a path from failure diagnosis through prevention to enforcement, beginning with the responsibility gap (the distance between technical performance and responsible outcomes) and demonstrating how proxy variables, feedback loops, and distribution shift can harm users while conventional metrics remain acceptable. The engineering response follows the model's lifecycle: specification that fixes targets, populations, and oversight before training forecloses them, fairness metrics that make disparities measurable, explanation and documentation selected for applicable stakeholder and regulatory requirements, and monitoring infrastructure that can surface silent failures and connect signals to response.

Translating responsibility concerns into measurable properties makes them tractable. A justified, application-specific bound on disparity is testable; "be fair" is not. This translation extends beyond fairness: in the chapter's illustrative TCO scenario, a 20 percent accelerator-service-demand reduction saves \$304K and is modeled to avoid 19 t of CO_2 under the stated assumptions. Documentation becomes model cards with explicit intended use and known limitations. Governance becomes access control, lineage tracking, and audit infrastructure that makes compliance evidence available rather than merely aspirational. Across these cases, abstract ethical obligations become concrete engineering requirements that can be specified, tested, monitored, and enforced.

These practices share one principle. A responsibility property can be enforced only at a lifecycle stage that still has the freedom to change it. Measurement can gate only the populations that specification named, documentation can record only what measurement produced, and governance can prove only what the pipeline retained. A system that defers fairness, efficiency, transparency, or governance to a late review is therefore incomplete in the same sense as a system shipped without a latency budget. The requirement exists, but nothing in the architecture can meet it.

Key Takeaways: Reliable for whom?

- **Aggregate correctness hides harm**: A model can look accurate in aggregate while one subgroup's error rate is 43.1× another subgroup's rate, as in the Face++ Gender Shades audit. Responsible evaluation therefore starts with disaggregated and intersectional slices, not aggregate accuracy alone.
- **Responsibility becomes testable through thresholds**: "Be fair" is not testable, but bounded disparity, documented intended use, and explainability requirements are. Translating values into measurable constraints lets teams review trade-offs among fairness, accuracy, latency, and cost.
- **Efficiency is a social constraint**: An efficient model can reduce energy and cost and broaden who can deploy it. In the chapter's illustrative scenario, the inference-to-training cost ratio is 40:1, making per-query optimization responsible engineering.
- **Monitoring must watch outcomes**: Bias and privacy failures can continue with green uptime dashboards because harmful predictions look operationally normal. Production monitoring must track subgroup outcomes, data lineage, feedback loops, and incident paths with the same rigor as latency regressions.

- **Governance has to be built in**: Model cards, datasheets, access controls, erasure workflows, human-review paths, and audit trails are technical infrastructure, not optional policy overlays. Regulations such as GDPR require capabilities that should be designed into the pipeline; later remediation may remain possible but can be narrower and costlier once the system is serving decisions.

A system that does exactly what it was told is dangerous precisely because the telling is never complete. Every objective a model is given is a specification with gaps, and an optimizer is a machine for finding them. It can reproduce the bias latent in its data, chase the proxy instead of the goal, and call the result success because nothing in the objective said otherwise. Responsible engineering is the discipline of writing back in what the specification left out. Constraints and monitoring reduce the risk that optimization amplifies harms encoded in the data. The constraint is the same kind the rest of the book imposed in latency and memory, except that here it protects people the objective never named, and a model that is fast and accurate while wrong about whom it serves has not failed less than one that crashes, only more quietly.



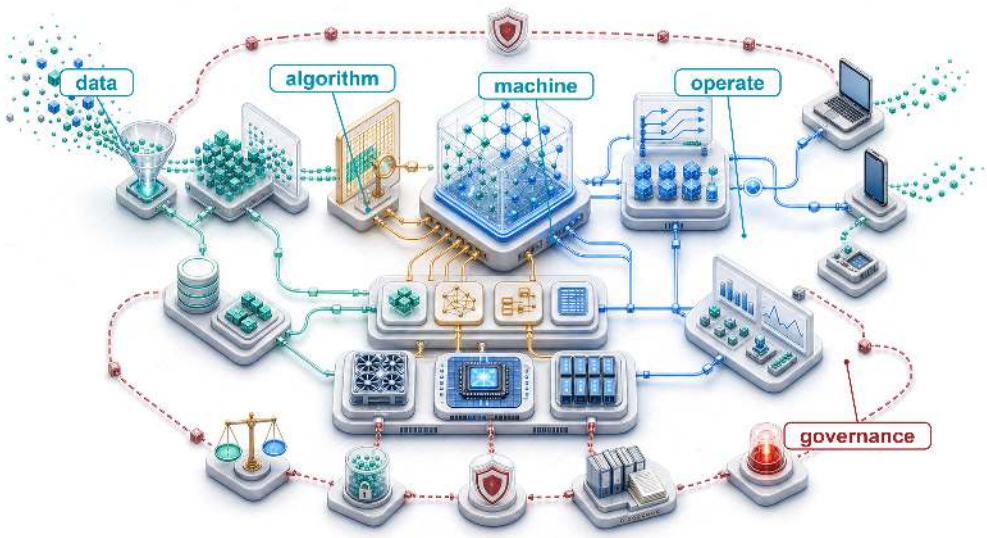
What's Next: From local targets to the whole system

Responsibility was the last constraint this book added, and it behaves like the ones before it. A disparity bound, a carbon budget, or a lineage requirement can be met only if every upstream stage preserved the freedom to meet it, exactly as a latency budget can. Every constraint so far has been taught one layer at a time, yet the costliest failures arise where layers interact, when each component meets its own target and the system still misses its goal. Chapter 16 opens with exactly such a failure in a mobile fleet, then gathers the quantitative principles of the preceding chapters, from the iron law (principle 3) onward, into one framework for reasoning across the whole stack.

SYNTHESIS

16

Conclusion



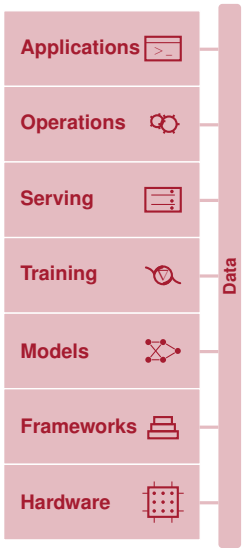
16.1 Synthesizing ML Systems
16.2 Lighthouse Models: Constraint Propagation
16.3 Thirteen Quantitative Principles
16.4 The Dual Mandate Revisited
16.5 Future Directions
16.6 Journey Forward
16.7 Fallacies and Pitfalls
16.8 Summary

Purpose

What does mastering the full stack enable that expertise in any single layer cannot?

A single production decision travels through the entire stack. A data pipeline decides which events count as training signal. That signal shapes the architecture, which in turn fixes memory footprint and arithmetic intensity. Those properties constrain hardware choice, quantization strategy, serving latency, drift monitoring, and governance obligations. Mastered individually, each layer represents a valuable skill; mastered together, they confer the ability to reason across boundaries that no single specialty sees in full. An engineer who understands only compression can shrink a model, but cannot predict whether the accuracy loss matters for the deployment context. An engineer who understands only serving can optimize latency, but cannot trace a performance regression to a data pipeline change three stages upstream. Many of the costliest production failures arise at these boundaries, where every component passes its local tests yet the system as a whole misses its target.

The discipline of ML systems engineering is the discipline of seeing these connections, where one team's optimization becomes another team's constraint. The principles governing these interactions—from the iron law and the silicon contract to drift and the latency budget—outlive any specific framework, hardware generation, or model family. Technologies change, but the underlying physical constraints and trade-offs endure. Frameworks and accelerators turn over every few years; the cost of moving data, the speed of light, and the gap between an assumed and a measured bottleneck do not. In D·A·M terms, what endures is the ability to analyze a system before it is built: reasoning about how data, algorithm, and machine constraints will interact, where bottlenecks will emerge, and which architectural commitments will prove irreversible. Thinking in systems rather than components separates an engineer who can build a part from one who can build the whole.



Learning Objectives

- Synthesize core ML systems principles into a framework for reasoning across Data, Algorithm, and Machine constraints
- Trace how data, architecture, compression, hardware, serving, operations, and governance decisions propagate constraints across an ML system
- Apply lighthouse-model reasoning to diagnose bottlenecks across cloud, mobile, edge, recommendation, and TinyML deployments
- Evaluate deployment trade-offs using latency budgets, memory and energy movement, drift, and responsibility constraints
- Justify an engineering approach for emerging contexts such as embodied, composed, and multi-node systems, using cross-layer evidence to identify where single-node reasoning must widen its boundary

16.1 Synthesizing ML Systems

Consider an illustrative deployment of an image classification model to a fleet of mobile devices, showing how cross-layer interactions can emerge in production. The architecture team chose depthwise separable convolutions to factorize spatial filtering from channel projection, reducing arithmetic operations. The compression team quantized weights and activations to INT8 to cut memory traffic and fit within on-chip SRAM. The serving team met a p99 latency target of 50 ms. Each team succeeded by its local metric, yet within weeks, accuracy dropped by four percentage points on specific firmware and device cohorts. The cause was an unanticipated interaction between dynamic range clipping in the quantization scheme and an interpolation routine in a firmware-specific image preprocessing path. No component failed in isolation; rather, the data pipeline, architecture, compression scheme, accelerator runtime, and monitoring infrastructure coupled in production. Nothing crashed. Users on the affected cohorts simply received worse predictions. It was the quiet failure that chapter 15 showed aggregate metrics can hide, one that the system's monitoring was never built to see.

Failures of this kind originate at the interfaces between components. Exposing those interfaces before they fail is the primary discipline of ML systems engineering. Traditional software engineering treats functional correctness as decoupled from the underlying hardware; in machine learning systems, statistical behavior is physically inseparable from the pipelines, memory hierarchies, and accelerators that train, serve, and monitor the model. The bitter lesson teaches that general methods scaling with computation outrun hand-encoded heuristics (section 1.4); the iron law of ML systems governs *how* to physically afford that scale.

↩ The bitter lesson, that general methods which scale with computation outrun hand-encoded expertise, is established in section 1.4.

At the center of this discipline sits the iron law of ML systems (principle 3). Its three terms (data movement, compute, and overhead) serve as the primary engineering levers for quantitative analysis. Building intelligence requires both statistical algorithms and adherence to the silicon contract (principle 4), the physical agreement between the model and the executing machine. Arithmetic intensity and roofline modeling convert qualitative performance intuitions into quantitative upper bounds (Williams et al. 2009).

System capabilities emerge from co-design across the Data, Algorithm, and Machine (D·A·M) triad rather than from isolated algorithmic breakthroughs. System success and system failure arise from the very same mechanism: cross-layer coupling. The transformer architecture introduced an attention-based model family (Vaswani et al. 2017), and large language models such as GPT-3 and Llama 2 demonstrate how that family scaled into modern workloads (Brown et al. 2020; Touvron, Martin, et al. 2023). Yet the mathematics of attention alone did not produce that scaling. It depended fundamentally on decisions across other layers: attention kernels tiled so that intermediate scores stay in on-chip SRAM instead of making round trips to off-chip memory, low-precision Tensor Core units that raise the peak arithmetic rate, and distributed partitioning strategies that split tensor state across accelerator fabrics. Coupling cuts both ways. The cross-layer interactions that allow an architecture to become an efficient physical system are the exact conduits through which an uncoordinated component choice can compromise the entire deployment.

↩ IO-aware attention tiling, which keeps attention scores in on-chip memory, is established in section 8.7.4.

Integration has concrete consequences. Software engineering often treats the "model" as a weights checkpoint, a single file of floating-point numbers. In a production environment, however, the weights are only one component of the true model, and often not the most fragile. A model that produces high benchmark accuracy fails if it ingests corrupted features, and a model that converges cleanly during training fails if it cannot satisfy deployment latency constraints. The true model is the sum of the data pipeline that dictates what the learner sees, the training infrastructure that governs optimization, the serving system that mediates inference, and the operational loop that monitors statistical drift. Optimize the system, and the model improves. Neglect the system, and the model degrades. Systems engineering is the physical realization of machine learning, which is why *the system is the model.*

↩ The finding that model code is a small fraction of a production ML system, whose surrounding pipelines, serving, and monitoring are larger and more enduring, is established in section 1.2.

Checkpoint 16.1: Systems thinking

The system's dependencies, feedback loops, and request path make its boundaries testable.

The integration

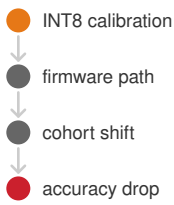
- ☐ **Dependencies**: can you trace a data-pipeline change through training and serving to identify its latency effect?
- ☐ **Feedback loops**: have you mapped how the model's predictions influence its future training data?

The holism

- ☐ **End-to-end**: can you trace a user request from the UI, through the network, preprocessing, model, postprocessing, and back to the UI?

Tracing a request end to end makes the same point structurally. Every layer of the stack enforces physical and statistical constraints on the next. Data curation and selection bound what the learner sees; architectures, training systems, and frameworks lower the resulting computation onto hardware; compression, acceleration, and benchmarking renegotiate the physical boundaries; and serving, operations, and responsible engineering hold the result to tail-latency budgets, drift, and subgroup outcomes in production.

Because these stages form a tightly coupled system rather than a sequence of isolated handoffs, engineering decisions never remain localized. Architectural choices, compression techniques, and accelerator lowering jointly dictate the available execution paths, while serving latency budgets and operational monitoring determine whether those paths satisfy production requirements. The opening fleet failure exemplifies this causal chain. INT8 quantization fixes its clipping ranges during calibration, permanently encoding the statistical distribution of the calibration dataset. When a device-specific firmware routine performs image interpolation differently, it presents an altered input distribution to the quantized model on that cohort. No individual team detected the flaw because each validated its component against standard inputs in isolation. Furthermore, while latency was evaluated end to end along the deployed request path, accuracy was measured only as an aggregate scalar over an offline evaluation set. The regression remained invisible until real users experienced degraded predictions in the field. Heterogeneous edge environments require disaggregated cohort monitoring from day one. Decisions propagate downstream along a continuous physical and statistical chain, as illustrated in the margin; an engineer who optimizes a single layer in isolation cannot predict how an intervention will ripple across the whole.



A calibration choice surfaces as a cohort accuracy drop three stages downstream.

Diagnosing these cascading interactions requires concrete constraint maps. Grounding the analysis in representative workloads reveals how physical bounds dictate architectural choices, runtime schedules, and operational trade-offs across the stack.

16.2 Lighthouse Models: Constraint Propagation

The five lighthouse models ground constraint propagation in concrete execution profiles, each exposing a distinct physical bottleneck:

- **ResNet-50**: Increasing batch size can reuse filter weights across inputs and raise arithmetic intensity. Whether execution moves from a bandwidth-limited regime toward the compute ceiling depends on layer shapes, reuse, kernels, and the hardware ridge point.
- **GPT-2/Llama**: Low-batch autoregressive decode stalls on memory bandwidth due to low weight reuse; batching, prefill scheduling, and model parallelism shift the binding constraint.
- **MobileNetV2**: Depthwise separable convolutions and INT8 quantization trade representational capacity for mobile NPU deployment under watt-scale thermal envelopes.
- **DLRM**: Embedding tables that reach terabytes in production make memory capacity a binding constraint alongside memory bandwidth, forcing systems to co-design around memory tiers and sparse access patterns.
- **Keyword spotting (KWS)**: Sub-megabyte microcontroller models with always-on inference under milliwatt power budgets leave little room for avoidable memory traffic or dispatch overhead.

Together, these five workloads span the deployment spectrum from warehouse-scale data centers to microcontrollers, exposing the bottlenecks that quantitative principles diagnose. Tracing these workloads across architecture, training, optimization, and serving establishes the integrated perspective that distinguishes ML systems engineering from isolated algorithm development.

↩ The five lighthouse models and their diagnostic execution regimes are established in section 1.7.2.

Table 16.1 traces this progression for MobileNetV2, illustrating how cross-layer constraints converge on a single engineering artifact. Across nine phases, from the deployment tier's envelope through architecture, training, data selection, compression, acceleration, benchmarking, serving, and runtime operations, each engineering decision narrows the options available downstream.

Table 16.1: The Lighthouse Journey (MobileNetV2): Tracing one model through nine phases of the systems stack shows how decisions in one domain (for example, architecture) propagate constraints and opportunities into downstream domains (for example, hardware acceleration and monitoring).

Journey Phase	System Lens	MobileNetV2 Implementation
Foundations (chapter 2)	Deployment Tier Constraints	Mobile tier sets a watt-scale power, thermal, and battery envelope that every later phase must respect
Architecture (chapter 6)	Algorithmic Efficiency	Depthwise Separable Convolutions: 8.7× fewer FLOPs for a representative 3-by-3, 256-output-channel layer and 13.7× fewer operations than ResNet-50 at ImageNet scale
Training (chapter 8)	Throughput vs. Latency	Trained at high-throughput batch sizes for a model that will serve single requests at low latency
Data Selection (chapter 9)	Capacity-Matched Augmentation	Milder augmentation matched to a compact network's limited capacity, validated against deployment invariances such as device and lighting variation, at no inference cost
Compression (chapter 10)	Navigating the Pareto Frontier	INT8 Quantization: FP32 uses 4× and FP16 uses 2× the per-value storage of INT8, with accuracy revalidated per deployment
Acceleration (chapter 11)	Honoring the Silicon Contract	Mapping kernels to Mobile NPUs (for example, Apple Neural Engine) to maximize hardware execution efficiency
Benchmarking (chapter 12)	Component vs. End-to-End	Confirming that compression and accelerator gains survive the full pipeline by separating accelerator latency from preprocessing and transfer, and rechecking INT8 accuracy on edge cases such as unusual lighting
Serving (chapter 13)	Respecting the Latency Budget	p99 < 50 ms constraint; optimizing preprocessing (resize/normalize) to avoid CPU bottlenecks
Operations (chapter 14)	Managing System Entropy	Drift Monitoring: Detecting distribution shifts and tracking cohort-level accuracy across heterogeneous device populations and lighting conditions

Decisions in one row constrain options in the next. Depthwise separable convolutions cut the operation count, but depthwise filtering reduces cross-channel reuse and often leaves these layers bandwidth bound on mobile hardware. When profiling confirms that regime, a further cut in operations buys little, and the compression row's reduction in bytes per value becomes the next

lever. That lever pays only if the acceleration row confirms that the mobile NPU executes those INT8 operators, and only if the benchmarking row shows the gain surviving preprocessing and transfer. This constraint propagation is universal across machine learning workloads. Formalizing these trade-offs requires quantitative principles.

16.3 Thirteen Quantitative Principles

When an engineer alters an embedding table layout, doubles batch size, or quantizes an attention layer from FP16 to INT8, the performance outcome is not random; it is governed by physical bounds, empirical trade-offs, and operational feedback. Reasoning quantitatively about this behavior across the entire stack requires rigorous analytical instruments. Table 16.2 summarizes these **thirteen quantitative principles** across four systems phases. They are not all the same kind of statement. Three are hard bounds (the Pareto frontier, the arithmetic intensity law, and Amdahl's Law), two are decompositions (the iron law and the energy-movement principle), two are fitted models (statistical drift and bias feedback), two are policies whose parameters a product decision sets (the verification gap and the latency budget), and four are heuristics (data-as-code, data gravity, the silicon contract, and training-serving skew). The heuristic label does not demote the silicon contract. It marks the contract as a wager about which resource an architecture saturates first, a wager that the arithmetic intensity bound then tests. Each principle box on the part pages names its kind, and each principle is reliable only within its stated assumptions.

Table 16.2: Thirteen Quantitative Principles: Each tool appears in the part where its governing constraint or diagnostic use first becomes visible. The collection combines bounds, decompositions, fitted models, policy requirements, and heuristics. Each row holds under its stated assumptions; together they provide a shared analytical vocabulary for system design, optimization, and deployment.

#	Principle	Part	Core Equation/Statement	What It Predicts
1	Data-as-Code Principle	I: Foundations	Learned behavior $\approx$ Train(training data, training procedure)	Data changes behavior; other inputs also matter
2	Data-Gravity Principle	I: Foundations	Move compute toward data when repeated transfer costs exceed placement costs	Depends on volume, reuse, network, and compute mobility
3	Iron Law of ML Systems	II: Development	$T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$; overlap ranges from max to sum	Stages may add or overlap
4	Silicon Contract	II: Development	Each architecture wagers on which resource (compute, bandwidth, or capacity) it saturates first	Design must target the resource saturated first
5	Pareto Frontier	III: Optimization	No feasible point is at least as good on every objective and strictly better on one	Along the frontier, improving one objective worsens another
6	Arithmetic Intensity Law	III: Optimization	$R_{\text{attain}} \leq \min(R_{\text{peak}},\ I \times \text{BW})$	More compute cannot raise a bandwidth ceiling
7	Energy-Movement Principle	III: Optimization	$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$; DRAM/FLOP cost ratio: 173–582×	Total energy depends on bytes moved, operations, and per-event costs
8	Amdahl's Law	III: Optimization	$\text{Speedup} = \frac{1}{(1-f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{G}}$	For fixed work, the serial fraction caps parallelism gains
9	Verification Gap	IV: Deployment	Deployment-population success probability must meet the specified target at the stated task-error tolerance	Specify distance, tolerance, population, and required success probability
10	Statistical Drift Diagnostic	IV: Deployment	$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$	A local fit; drift need not lower quality
11	Training-Serving Skew Diagnostic	IV: Deployment	$S_{\text{skew}} = \mathbb{E}[\lvert f_{\text{serve}}(x) - f_{\text{train}}(x)\rvert]$	Output mismatch signals risk, not accuracy loss
12	Latency Budget Principle	IV: Deployment	$\Pr[L_{\text{lat,total}} \leq T_{\text{deadline}}] \geq q$, where $L_{\text{lat,total}}$ sums the request-path delays	The product SLO selects q and the deadline T_{deadline}
13	Bias Feedback Model	IV: Deployment	$\Delta_g(k) \approx \Delta_g(0)\alpha_{\text{fb}}^k$ with fitted α_{fb}	Feedback may amplify harm; measure and intervene

Notation Guide for table 16.2: In deployment models, d_{task} is the task-specific distance metric, τ_{tol} is the acceptable error tolerance threshold, q is the target success probability for latency deadline

T_{deadline}, and α_{fb} is the amplification factor compounding disparity Δ_g at step k. The drift row restates the degradation equation (equation 1.2), in which λ is the sensitivity fitted from labeled outcomes and $\mathcal{D}(P_t \| P_0)$ measures how far the current input distribution has moved from the training distribution. S_{skew} is the expected absolute difference between serving-path and training-path outputs on the same input. In efficiency bounds, G is the speedup factor applied to the parallelizable fraction f_{parallel}. Finite tests estimate the deployment success probability with uncertainty; an evaluation must separately state the statistical confidence level of its interval or bound.

The thirteen principles are not independent axioms. The conservation-of-complexity heuristic[1] that opened Part I connects them into an integrated framework. In Part III, model compression demonstrated this trade-off at the scale of a single model: pruning sacrifices capacity, quantization sacrifices precision, and distillation expends training compute. At the scale of the entire system, the engineering discipline remains identical: following any local simplification, trace where the displaced complexity has migrated. The heuristic serves as a diagnostic rather than a physical conservation law. While thoughtful design can eliminate accidental complexity outright, systemic complexity frequently shifts into validation harnesses, state synchronization, orchestration protocols, or runtime operational burdens.

[1] **Conservation-of-complexity heuristic**: Tesler's Law, a design aphorism, says that an application's irreducible complexity must be handled somewhere in the interaction among user, application, and platform (Tesler 1984); extending it to all ML-system complexity is an analogy, not a physical law. Quantization may add validation burden, and abstraction may move implementation detail behind an interface, but good design can also remove accidental complexity outright. Large language model application pipelines illustrate a possible shift: simplifying the user-facing interface with shorter or vaguer prompts may move work into system prompts, retrieval, or output verification. Use the heuristic to search for displaced costs, not to assume that an equal compensating cost must exist.

Foundations: Where complexity originates (principles 1–2)

The data-as-code principle (1) and the data-gravity principle (2) establish data as both the logical program and the physical anchor of an ML system. As a program, the training dataset dictates learned behavior; a modification to data curation is a behavioral change requiring the same rigorous versioning, code review, and regression testing applied to source code. As an anchor, data volume dictates where computation must execute. In section 4.2.1, quantifying this gravitational pull showed that transferring 1 PB across cloud regions over a 10 Gbps link requires more than a week and incurs tens of thousands of dollars in egress fees—readily justifying moving compute resources to the data repository rather than the reverse.

The lighthouse models carry that physical anchor directly into execution. In DLRM, embedding tables that outgrow a single accelerator's memory make memory capacity a binding design constraint, requiring the execution system to be engineered around where the parameters physically reside. Compute-to-data placement recurs across deployment environments precisely because moving large, low-reuse datasets saturates interconnects.

↩ The transfer time and egress cost that make a large dataset the gravitational center of an architecture are established in section 4.2.1.

Development: How complexity becomes computation (principles 3–4)

The iron law (principle 3) and the silicon contract (principle 4) govern execution efficiency. The iron law's three-term decomposition isolates data movement, computation, and overhead; the silicon contract reveals which term limits performance on a given architecture-hardware pair. Representative execution regimes include sufficiently large batched ResNet-50 kernels approaching a compute ceiling, low-batch Llama decode limited by weight traffic, DLRM constrained by embedding capacity, and MobileNetV2 mapped to mobile NPU limits. Actual shapes, kernels, and runtime conditions determine which regime applies. Diagnosing whether an execution path is compute-bound, bandwidth-bound, or capacity-bound determines which optimizations yield speedup and which waste engineering effort. Training latency drops most when engineers target the binding term directly rather than distributing effort uniformly.

↪ The bottleneck diagnostic that pairs each binding iron-law term with the optimizations that work and those that are wasted is operationalized in section A.5.1.

Optimization: How constraints shape trade-offs (principles 5–8)

The four optimization principles form a tightly coupled diagnostic chain. The Pareto frontier (principle 5) identifies the designs that no alternative beats on every objective at once, and once a design lies on that frontier, improving one objective requires a trade-off against another within the same feasible design set. The arithmetic intensity law (principle 6) diagnoses whether compute or bandwidth sets the ideal ceiling. The energy-movement principle (principle 7) combines per-event costs with event counts. Under the reference energy constants, one DRAM access costs about 173–582× as much as one arithmetic operation. The lower end is the DRAM-versus-FP32 ratio that anchored hardware acceleration (chapter 11), and the upper end substitutes the cheaper FP16 operation. Which cost dominates a whole workload still depends on how many of each event occur.

↩ The compute-versus-bandwidth boundary and hardware balance (I_{ridge}) are established in section 11.5.2.

Amdahl's Law (principle 8) sets the ceiling on any parallelism gain, explaining why data loading and preprocessing can become bottlenecks in highly optimized systems.

MobileNetV2 balances all four principles simultaneously. Depthwise separable convolutions reshape the Pareto frontier (Sandler et al. 2018). INT8 quantization exploits the arithmetic intensity law by raising arithmetic intensity through reduced memory traffic (Jacob et al. 2018). Measured energy savings, where the reduced traffic and supported execution lower total consumption, align with the energy-movement principle, while Amdahl's Law bounds end-to-end speedup when host-side preprocessing remains unaccelerated. Microcontroller workloads such as KWS compress these trade-offs further, operating under sub-megabyte memories that leave little room for avoidable overhead.

Deployment: How reality defeats assumptions (principles 9–13)

The deployment principles address failures that bench testing cannot rule out: a system can work correctly on the bench yet fail silently in production. The verification gap (principle 9) specifies the required success probability over a stated deployment population and error tolerance. Testing estimates whether the system meets that requirement, with a separately stated statistical confidence level; it cannot establish correctness for every future input. The statistical drift diagnostic (principle 10) detects that the population has changed, and labeled outcomes fit how much that change costs in accuracy. The training-serving skew diagnostic (principle 11) flags training and serving paths that compute the same features differently. The latency budget (principle 12) holds serving to the tail quantile the product selects. The bias feedback model (principle 13) treats disparity as a quantity that compounds across retraining cycles at a fitted rate, which turns fairness into a stability question about the deployment loop.

The five deployment principles govern operational tooling: monitoring infrastructure, drift detection, feature stores, and disaggregated subgroup metrics exist to surface silent failures and trigger corrective action. A recommendation system that achieves high offline accuracy still requires parity checks when training-serving skew corrupts feature pipelines (principle 11) and outcome auditing when user behavior shifts seasonally (principle 10). Large language model serving must bound tail latency; continuous batching and speculative decoding are candidate mechanisms whose effects must be evaluated against the workload and product service-level objectives. Similarly, an automated credit model may satisfy throughput and convergence targets while systematically compounding disparities against underserved groups, requiring disaggregated monitoring to intercept feedback loops (principle 13).

↩ The monitoring infrastructure and drift quantification that surface these silent failures are established in section 14.6.

The integrated framework

The thirteen principles are not a checklist to apply sequentially. They form a web of mutual constraints, and a single decision activates several of them in turn.

Quantizing a model from FP16 to INT8 illustrates these mutual constraints in action. This single intervention navigates the Pareto frontier (principle 5), trading numerical precision for memory traffic. The consequences ripple across the stack: quantization alters the model's silicon contract (principle 4), shifting its operating point on the arithmetic intensity curve (principle 6) and transforming its energy profile (principle 7). In deployment, the latency budget (principle 12) dictates whether the resulting speedup meets the service-level objective (SLO), while the verification gap (principle 9) demands verifying that the lowered serving graph preserves the empirical accuracy measured during compression testing. The opening fleet failure escaped at exactly this step. Compression testing accepted the quantized model on images from the evaluation pipeline, while the firmware path presented a different population. The training-serving skew diagnostic (principle 11), comparing model outputs from the two preprocessing paths on the same captured inputs, is the measurement that would have exposed the mismatch before the affected cohort did. In that comparison the evaluation pipeline, which produced the calibration images, plays the role the diagnostic assigns to the training path, and the firmware path plays the serving path. The verification gap names why the failure escaped; the skew diagnostic names how to catch it.

The trace touched seven of the thirteen principles, and which seven depended on the decision. Had the same model moved off the device to a cloud endpoint, data placement (principle 2) would have entered, because every input image would then cross the network. Had preprocessing stayed

on the host CPU while the NPU ran the faster INT8 kernels, Amdahl's Law (principle 8) would have bounded how much of that speedup the request path could see. The principles activate in turn as a decision moves from model representation to hardware execution to production validation.

Evaluating a deployment proposal makes this web of constraints concrete.

Checkpoint 16.2: Applying the principles

A colleague proposes quantizing your model from FP16 to INT8 to reduce serving costs.

Trace the principles

- ☐ **Pareto frontier** (principle 5): What accuracy are you trading for the bandwidth gain?
- ☐ **Silicon contract** (principle 4): Does the target hardware efficiently accelerate the required INT8 operators?
- ☐ **Verification gap** (principle 9): Does the quantized serving artifact preserve the behavior accepted during compression testing?
- ☐ **Training-serving skew** (principle 11): Do the calibration images pass through the same preprocessing path that every serving device uses?
- ☐ **Latency budget** (principle 12): Does the speedup bring you within SLO, or create headroom for batching?

Grouped by the four parts, the principles form a cycle rather than a pipeline. The data principles (1–2) bound what model development can attempt, the iron law and silicon contract (3–4) set the execution terms that the optimization principles (5–8) then renegotiate, and deployment closes the loop. The deployment principles (9–13) turn runtime telemetry into evidence that sends work back to the data foundations, so each production lesson becomes the starting condition of the next design pass (figure 16.1). The conservation-of-complexity heuristic sits at the center of that cycle because every pass around it asks where the complexity displaced by the previous decision has landed.

That feedback loop is central to production engineering. Principles 9 through 13 expose signals that trigger targeted remediations: canary rollbacks, fallback heuristics, dataset augmentation, model retraining, or kernel retuning. The objective is not to catalog every future architecture, but to make feedback loops observable before operational failures compound.

Tracing a quantization proposal through the principles it touches is one diagnostic pass; the same habit applies when the bottleneck is not an optimization proposal but the cost of serving a single generated token.

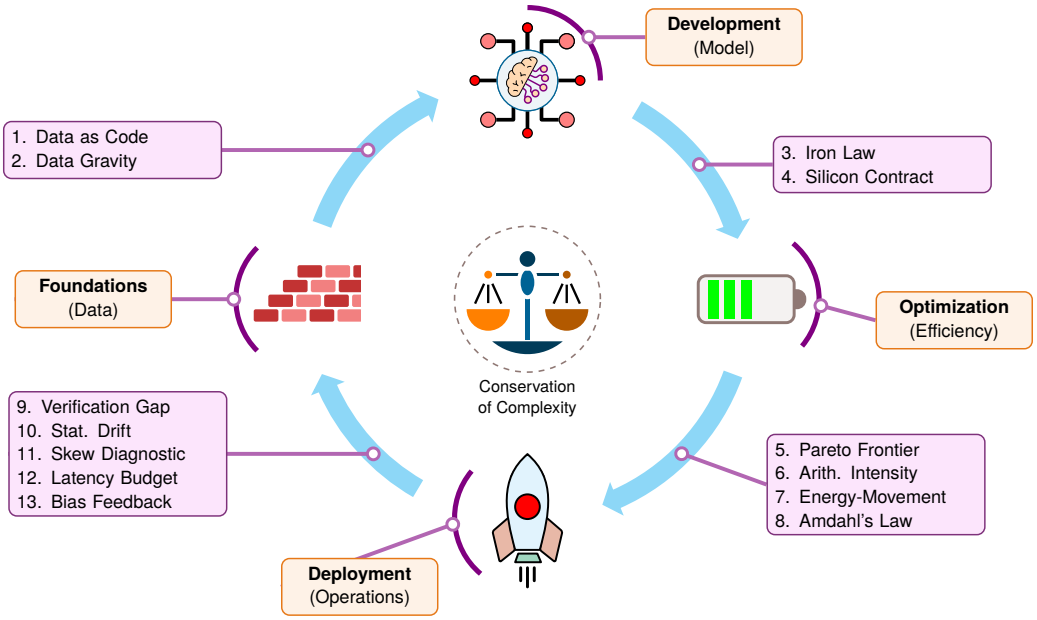


Figure 16.1: The Cycle of ML Systems (13 Principles): A four-phase systems engineering lifecycle organized around the conservation-of-complexity heuristic. The phases connect in a feedback cycle, and the thirteen principles are grouped along the transitions as bounds, decompositions, fitted models, policy requirements, and design heuristics whose assumptions must be checked. Icons mark the phases: bricks for the data foundations, a neural network for model development, a battery for the efficiency budget of optimization, and a rocket for launch into deployment.

Napkin Math 16.1: The cost of a token

Problem: Applying the iron law (principle 3) and the arithmetic intensity law (principle 6) to serving one token from a 70-billion-parameter model like Llama 2 70B, with its FP16 weights sharded evenly across two NVIDIA H100s, which idealized lower bound is larger: memory transfer or peak compute? Section D.3.2 supplies the H100 specifications. This calculation excludes KV-cache traffic, activations, context-dependent attention-over-KV FLOPs, interconnect communication, and dispatch overhead.

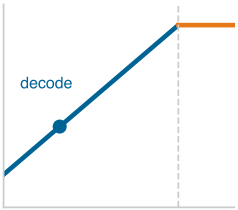
Physics:

- **Model-weight byte volume moved** (D_{vol}): 70 billion parameters $\times$ 2 bytes (FP16) = 140 GB.
- **Compute** (O): $O \approx 2 \times P = 140$ GFLOP per token, where P is the parameter count.
- **Hardware**: Two H100s with aggregate BW = 6.70 TB/s, $R_{\text{peak}} \approx 1978$ TFLOP/s FP16.

Math:

- **Time to move data**: $T_{\text{memory}} = \frac{140\ \text{GB}}{6700\ \text{GB/s}} \approx 20.9$ ms
- **Time to compute**: $T_{\text{compute}} = \frac{140 \times 10^9\ \text{FLOPs}}{1978 \times 10^{12}\ \text{FLOP/s}} \approx 0.07$ ms

Systems insight:
The idealized memory-transfer lower bound T_{memory} is 295.2× larger than the peak-compute lower bound T_{compute}. Under this batch-one model, decode is heavily bandwidth bound (arithmetic intensity ≈ 1 FLOP/byte), falling far to the left of the ridge point as shown in the margin. Because each 2-byte FP16 weight is fetched from high-bandwidth memory to perform a single multiply-accumulate (2 FLOPs) with the active token activation, the arithmetic intensity is $\frac{2\text{ FLOPs}}{2\text{ bytes}} = 1$ FLOP/byte, far below the H100 ridge point of ≈ 295 FLOP/byte. Sharding across two GPUs does not move that ridge, because it doubles peak rate and bandwidth together. Under this traffic model, attainable arithmetic throughput is less than 1 percent of the stated peak. That bound does not measure tensor-core busy or stalled cycles; profiling is needed to characterize realized execution. Batching can increase reuse, while quantization can reduce weight traffic. Optimizing only compute execution can reduce realized compute time toward the 0.07 ms lower bound, but it cannot reduce the 20.9 ms memory-transfer lower bound.



In this batch-one lower-bound model, decode sits left of the roofline ridge in the bandwidth-bound regime.

The roofline evaluation shows a physical bound acting as a diagnostic instrument rather than an abstract taxonomy. It also completes one half of the promise this book made at the outset.

16.4 The Dual Mandate Revisited

A recommendation model that ranks well on launch day can, six months later, be serving a population that its own rankings shaped, on hardware whose cost per query decides whether the service survives. The introduction named this pair of demands the dual mandate: learned behavior must remain trustworthy, and the machine must deliver that behavior within physical and economic limits. The thirteen principles are the book's instruments for both halves, and each half can be traced end to end on a single lighthouse.

↩ The dual mandate, trustworthy learned behavior delivered within the available time, memory, energy, and cost, is established in section 1.1.

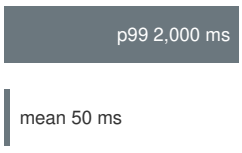
Trustworthy learned behavior

DLRM carries the first half, because several mechanisms can erode trust in its learned behavior. The degradation equation (equation 1.2) introduced a local fit relating distribution shift to observed accuracy change; the statistical drift diagnostic (principle 10) turns that idea into production telemetry, with divergence over user and item features as the early signal and delayed click and conversion outcomes as the confirmation. Training-serving skew (principle 11) attacks the same model from inside, when the online embedding lookup and the offline training pipeline compute a feature differently. The bias feedback model (principle 13) closes the loop, since items and creators that receive exposure generate the engagement data that earns them more exposure, and the resulting disparity can amplify or diminish across retraining cycles according to a fitted factor α_{fb}. The verification gap (principle 9) bounds what the launch evaluation ever promised. Together these principles make trustworthiness a monitored quantity with its own measurements, thresholds, and response paths, rather than a property certified once at launch.

Physical deliverability

The Llama 2 70B token analysis in notebook 16.1 carries the second half. There the iron law and the arithmetic intensity law locate the binding term. At about 1 FLOP/byte, batch-one decode sits far left of the ridge point, and its memory-transfer lower bound is 295.2× larger than the compute bound. The silicon contract (principle 4) exposes the architectural consequence: while hardware is provisioned and priced based on peak arithmetic throughput, this workload saturates memory bandwidth first, dictating that deliverable token generation speed is governed by bytes per second transferred from memory rather than peak FLOP/s. The energy-movement principle (7) highlights the same weight traffic. Because a reference DRAM access consumes approximately 173–582× as much energy as an arithmetic operation, and this workload performs only about one operation per byte fetched, moving weights dominates the energy expenditure of each token. One physical detail requires precision: the per-event ratio compares single operations to memory access events rather than isolated bytes. Because DRAM accesses occur in multi-byte burst widths (such as 32-byte or 64-byte transactions), an exact energy ledger scales by transaction width, with empirical hardware telemetry settling the aggregate energy profile.

Batching can amortize weight traffic across requests, and supported quantization can reduce bytes per weight. At the service level, the latency budget (principle 12) limits how far batching may go. The relevant delay now includes prompt processing and request waiting, not only token decoding. Consider an illustrative large language model (LLM) chat endpoint whose mean time to first token is 50 ms but whose p99, driven by long prompts and by queueing behind long generations, reaches 2,000 ms, a tail 40× the mean (illustrated in the margin). The M/M/1 model of section 13.5.4 puts the p99 of time in system near 4.6× its mean, because it assumes exponentially distributed service times. The illustrative endpoint instead assumes a broad mix of prompt and generation lengths and scheduling that can leave short requests waiting behind long generations. These mechanisms can stretch the tail, but the stated mean and p99 are scenario assumptions, not values derived from a queueing model. The actual tail depends on the request mix and scheduling policy and must be measured; the general importance of tail latency is established in Dean and Barroso (2013). The product commits to the tail quantile, so the batching that improves throughput must stop where it pushes that quantile past its deadline.



Mean latency hides the tail; the selected tail quantile governs the SLO.

The two traces use different principles, but they share one discipline: name the binding term, measure it, and budget for it. That discipline is the engineering answer to the bitter lesson. Scale works, but only a system that can afford it and can still be trusted continues to scale. The remaining question is where these constraints bind next, as ML systems spread across heterogeneous deployment targets, confront adversarial conditions, take on societal applications with public consequences, and grow into compositions of many components.

16.5 Future Directions

An autonomous vehicle navigating an unfamiliar intersection cannot wait for a cloud round-trip, and its sensor streams compete with the model for the same local bus; nor can a clinical assistant afford an uncalibrated diagnostic response under distribution shift. The framework is most useful when it forecasts where such physical and statistical constraints bind next. Three areas put the same physics under increasing pressure: deployment across diverse contexts, robustness under adversarial conditions, including deliberately crafted inputs (Goodfellow et al. 2015), and societal applications whose failures carry public consequences. A fourth horizon, systems that compose multiple models, tools, and verifiers or grow beyond one machine, extends the same lens rather than replacing it.

Applying principles to emerging deployment contexts

Deployment diversity tests whether one quantitative framework can explain systems with contrasting resource regimes. The cloud offers abundant power and centralized hardware, edge and mobile devices operate under latency and battery budgets, and TinyML and embedded systems compress the same design problem into kilobytes and milliwatts. Generative AI is not a fifth deployment environment; it is a workload class that stresses all four.

In the cloud regime, the binding decision is how to turn abundant hardware into useful throughput without letting data movement, capacity, or cost dominate. Dense workloads such as ResNet-50 maximize accelerator compute efficiency through kernel fusion and mixed-precision training, while DLRM-style recommendation systems must also manage embedding-table capacity, placement, and sparse access patterns. These execution-efficiency techniques, together with careful data placement, sustain high arithmetic intensity and balance operational costs at scale.

In contrast, mobile and edge devices operate under strict power, thermal, and memory bounds that necessitate hardware-software co-design. Efficient network architectures combined with model compression enable deployment on platforms where a reference mobile device has about 10× smaller memory capacity and about 140× smaller power envelope than an H100-class accelerator. Edge deployment is mandatory when latency, privacy, intermittent connectivity, or egress bandwidth make centralized cloud inference infeasible. In these regimes, efficiency dictates feasibility rather than minor cost optimization.

At the opposite extreme, TinyML and embedded systems, the domain of the keyword spotting lighthouse, operate within kilobyte SRAM budgets, milliwatt power envelopes, and multi-year operational lifecycles. Operating under these extremes requires uncompromised systems discipline: rigorous profiling isolates memory traffic bottlenecks, custom operator lowering reduces particular

dispatch and interpretation costs, and a fixed tensor arena, planned at build time from tensor lifetimes, replaces dynamic allocation so that peak memory is known before the device ships. Severe resource limitations historically catalyzed efficient architecture families such as MobileNets (Howard et al. 2017; Sandler et al. 2018) and EfficientNets (Tan and Le 2019), proving that physical machine constraints actively drive algorithmic innovation.

↩ The physical memory and latency envelopes that mandate compression for microcontroller and edge deployment are established in section 10.2.1.

A related frontier is embodied AI and robotics, where models execute inside closed-loop sensorimotor control cycles. In this regime, the deadline T_{deadline} of the latency budget (principle 12) becomes a hard real-time safety constraint, because a tail-latency spike can cause mechanical instability or collision, and continuous high-bandwidth sensor streaming competes directly with model weight traffic across constrained on-device memory buses.

Generative AI, a workload class rather than a deployment environment, stresses all of these regimes. Autoregressive models, illustrated by the GPT-2/Llama lighthouse family, concentrate that pressure at the memory hierarchy during token generation. Low-batch decode for dense autoregressive models stalls on memory bandwidth because each token step reuses model weights only once. Larger batch sizes elevate arithmetic intensity, whereas sufficiently large prompt-prefill matrix multiplications can approach the compute ceiling; short prompts, shapes, and implementation overhead can change that regime. Model partitioning across devices, through tensor and pipeline parallelism, redistributes memory footprint at the expense of collective communication. Speculative decoding trades surplus compute on a draft model to break the sequential latency bottleneck of the target model. Together, these mechanisms show the iron law adapting to complex workload structures.

The same physical bounds apply across these paradigms, while fitted statistical models and SLO policies remain deployment-specific. Success depends on checking those distinctions and applying the principles together rather than pursuing isolated optimizations. The more deployment contexts a system spans, the more failure surfaces it creates. Robustness, not coverage alone, therefore becomes a binding constraint at the next frontier.

Building robust AI systems

Machine learning systems can respond confidently and incorrectly while conventional infrastructure health checks remain green, allowing silent failures to persist undetected for weeks. Distribution shifts alter accuracy without code modifications, adversarial inputs (small, deliberately crafted perturbations that flip a prediction while leaving the input looking unchanged to a person) exploit vulnerabilities invisible to standard testing, and edge cases expose training-data boundaries that debugging alone cannot fix.

Finite testing estimates behavior on a defined population with uncertainty; it cannot prove correctness for every future input. Drift signals show when that population may have changed, while labeled outcomes determine whether quality changed. Together they motivate continuous monitoring as a design requirement, along with fallback policies and periodic revalidation, without claiming that every distribution shift causes degradation. The operational question is whether a failure will be detected and diagnosed before its impact spreads.

Robustness therefore demands designing for graceful degradation, not only prevention. At the single-system scale, that discipline appears as fallback paths, uncertainty thresholds, version-specific rollback policies, and monitoring hooks, with rollback reserved for a bad release rather than for external drift. At larger scale, the same logic extends to hardware redundancy and ensemble-style diversity. As AI systems assume increasingly autonomous roles in healthcare, transportation, and finance, the gap between "works in the lab" and "works in the world" becomes the critical engineering challenge. Robustness becomes more essential as systems add components, because each interface creates another timeout, stale input, inconsistent state, or recovery path to monitor.

↩ Incident response procedures, including the choice among rollback, fallback, and traffic reduction, are established in section 14.7.1.1.

AI for societal benefit

Robust systems are the prerequisite for deploying AI in domains where technical failures carry public consequences. The diabetic retinopathy screening case from the ML workflow shows why. A model that reaches expert-level performance on high-quality images from standardized research cameras can fail on blurry, poorly lit images from older clinic cameras, not because the algorithm is wrong but because the deployment population has moved outside the training envelope. In verification-gap

terms (principle 9), the launch evaluation made its promise over the wrong population. Each societal domain stresses a different governing constraint before it can deliver that value safely.

Scientific discovery workloads such as protein structure prediction, drug-interaction modeling, and materials screening can demand sustained training or inference throughput, so the iron law (principle 3) and the silicon contract (principle 4) decide how much of a search space a fixed compute budget can explore. Healthcare stresses the deployment principles instead. Calibrated uncertainty, external validation at each new site, clinical review, and continuous monitoring become safety requirements, because a diagnostic model trained on one hospital's population may degrade at another with different demographics, disease prevalence, or imaging equipment. Personalized education stresses data governance and the data-as-code principle (1), since the student-interaction records that drive personalization are both sensitive and, in effect, the program the model learns from; an interactive tutor must also meet its latency budget. A climate model that produces confident but uncalibrated projections carries the same verification problem into policy, where an overconfident forecast can steer decisions that are slow and costly to reverse.

↩ The lab-to-field gap in the diabetic retinopathy screening case, where field images fall outside the training distribution, is established in section 3.8.1.

In each of these domains the D·A·M taxonomy, the thirteen principles, and roofline analysis supply the systems questions, but the acceptable answers come from domain evidence such as clinical studies, educational outcome measurements, and physical validation, which no generic systems toolkit provides. These applications also share a property that keeps their constraints tractable: their task boundaries are fixed in advance. A diagnostic model classifies within a defined label set, and a structure predictor answers a defined kind of query. The next stress test removes that boundary.

System composition as a stress test

The most ambitious test of these principles is a system whose task boundaries are not fixed in advance. A task-general assistant or multi-component ML service may route one request through several components: retrieval, which fetches relevant documents from an index; a planner, a model that decides which steps to take; tool execution, which calls external programs such as a search engine or a calculator; generation of the answer; and a verifier, which checks the output against rules or with a second model. The governing challenge is systems engineering as much as algorithm design: the surrounding system must bound latency, reliability, cost, safety, and observability as work fans out across components.

System composition puts four of the principles under new load at once:

- **Iron law**: The computation that each component performs must be budgeted as work fans out across retrieval, planning, tool execution, generation, and verification.
- **Silicon contract**: Each component wagers on a different resource. An index lookup leans on memory capacity and bandwidth, generation on the weight traffic of decode, and a tool call on network round trips, so no single accelerator suits the whole request.
- **Pareto frontier**: The trade-off surface expands from two or three metrics, such as accuracy, latency, and memory, to a larger surface that also includes safety, fairness, factuality, privacy, and cost.
- **Statistical drift**: Drift applies not only to the final output, but also to retrieved documents, tool responses, and intermediate decisions.

A composed system cannot rely on a single model-quality claim; it needs interfaces whose behavior can be measured, because each interface is where one component's assumptions about another become testable (Lampson 1983). Composed systems trade monolithic simplicity for explicit coordination. Each step can be independently updated, monitored, and debugged, but each also creates another interface contract. The decomposition gives up latency and architectural simplicity in exchange for control and observability, a move along a Pareto frontier rather than a free gain.

The systems cost is visible in a single request. If an assistant fans out to retrieval, a planner, two tools, a generator, and a verifier, its realized latency follows the request's critical path. That path is the retrieval time, plus the planner's time, plus the slower of the two tool calls if they run in parallel, plus generation, verification, and the orchestration overhead between steps, and the latency budget (principle 12) must hold that sum at the selected quantile. Its reliability analysis must also account for each component: every additional component creates another timeout, schema mismatch, stale index, or verifier false negative to monitor. Conventional microservices already face runtime contract

failures across process boundaries. Composed ML systems add probabilistic interfaces in which a large language model planner may invent a tool name that does not exist or produce JSON that deviates from the declared schema (the agreed format of each call), requiring defensive parsing, retry logic, and output validation at each interface boundary. Capability can increase by adding system structure, but that structure must obey the same latency, reliability, and observability requirements as any production ML system.

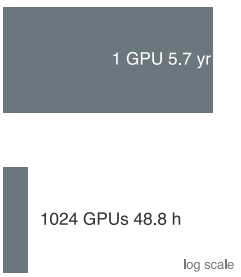
System composition aligns directly with fundamental systems engineering principles. Modular sub-components can be independently compressed and accelerated on targeted backends. Each component exposes an individual silicon contract (principle 4) and arithmetic intensity profile, opening avenues for heterogeneous specialization. The interfaces between components establish explicit observation points for detecting drift, skew, and numerical degradation. Orchestrating multiple models reliably, routing requests dynamically across specialized hardware, and keeping shared state consistent when several components read and update it, such as a conversation history or a retrieval index, require end-to-end co-design from data ingest through accelerator scheduling to operational feedback.

> **Systems Perspective 16.1: A new golden age**
>
> Hennessy and Patterson (2019) declared a "New Golden Age for Computer Architecture," driven by the end of Dennard scaling, the slowdown of Moore's Law, and new opportunities from domain-specific architectures, open instruction sets that any designer may implement without a license, and agile chip development with short, iterative hardware design cycles. Large-scale AI workloads are one domain where those pressures are visible. Building more capable AI services will not be a matter of writing a better loss function alone; it will be a systems engineering challenge involving energy efficiency, high-bandwidth interconnects, memory hierarchy design, and software stacks that keep heterogeneous hardware useful rather than idle. The thirteen principles provide a quantitative vocabulary for navigating that regime while preserving the distinction between physical bounds and deployment-specific assumptions.

The principles established here provide a coherent lens for individual ML systems. Larger systems do not invalidate that lens; they expose the same constraints at a different boundary.

A horizon note: From node to fleet

When workloads exceed a single node, the resource boundary moves outward, but bottleneck reasoning remains invariant. Local memory capacity constraints remain, while bisection bandwidth adds another limit; local exception handling expands into fleet-level fault tolerance, and training throughput becomes a distributed coordination problem. As illustrated in the margin ladder, under a model in which every GPU fails independently at the same constant rate, regardless of its age, and any GPU failure halts the pool, a reference component mean time to failure (MTTF) of 5.7 years collapses into a cluster mean time between failures (MTBF) of about 48.8 hours across a 1,024-GPU pool, before accounting for correlated network or power outages. Under this model, a multi-week training run is highly likely to encounter component failures. Co-designing a fault-tolerance strategy becomes necessary for reliable completion; checkpoint scheduling and recovery scope should follow the measured failure and recovery costs. Asynchronous checkpointing and localized recovery are options, not consequences uniquely determined by the failure-rate model. At fleet scale, the analysis does not abandon single-node principles; it widens their boundary: identify the binding constraint, quantify the governing cost terms in the iron law, and trace where displaced latency propagates. The boundary moves; the reasoning stays.



Fleet scale turns rare component failures into routine system events.

This discipline rewards engineers who use bounds to guide benchmarks and measurements to test the assumptions behind those bounds. The same habit governs the last question this book raises, which is what the engineer owes the people a system affects.

16.6 Journey Forward

Intelligence is a systems property. It emerges from co-designing data pipelines, model architectures, accelerator hardware, runtime software, and operational governance rather than from any isolated

algorithmic breakthrough. The governing lesson of systems engineering is not a fixed recipe for a single model family or hardware generation. It is the discipline of exposing hidden dependencies to measurement, quantifying trade-offs against physical hardware limits, and holding production deployments accountable to real-world operational constraints. Two appendices carry that discipline into practice: Appendix A supplies the diagnostic framework, and Appendix E provides the reference constants for rapid back-of-the-envelope calculations.

The engineering responsibility

The systems integration perspective demonstrates why ethical considerations cannot be separated from physical and algorithmic design. Hardware requirements govern access: a model demanding an array of high-end data-center accelerators for inference excludes organizations unable to procure specialized capital.[2] Upstream training data encodes historical sampling skews that distort downstream model predictions. Sustained accelerator power draw directly drives facility thermal dissipation and regional grid carbon emissions. System design choices, from arithmetic precision to data curation, distribute operational costs and societal consequences far beyond the engineering team.

Systems engineering extends beyond maximizing raw capability to operating deployments responsibly within societal and physical bounds. Production systems must achieve the arithmetic efficiency needed to run on accessible hardware, maintain fault tolerance against hardware anomalies, operate within strict thermal and power envelopes, and meet justified, application-specific fairness requirements across relevant demographic groups. Realizing applications such as planetary climate monitoring and clinical diagnostic assistants demands rigorous systems discipline that treats governance criteria as first-class physical and operational constraints.

That responsibility also demands continuous skepticism toward local wins. Optimizing a component in isolation is the most common way a system acquires failure modes that no team owns, and the benchmark win that does not survive production is its most familiar form. The recurring versions of that mistake are worth naming explicitly, because recognizing them early is far cheaper than a redeployment.

[2] **AI democratization**: Making AI accessible beyond a small number of well-resourced organizations through efficient systems engineering. Mobile-optimized models and cloud APIs can widen access, but doing so sustainably requires systematic optimization across hardware, algorithms, and infrastructure to maintain quality at scale.

↩ The lifecycle that turns responsibility into written requirements, release gates, documentation, and operational monitoring is established in section 15.3.

16.7 Fallacies and Pitfalls

Fallacies and pitfalls in ML systems arise from a common source: treating the system as decomposable into independent parts. Each fallacy assumes that optimizing one dimension, one metric, or one stage suffices; each pitfall shows the consequence when that assumption meets production reality.

Fallacy: *Systems engineering complexity disappears with better tools and abstractions.*

Tools abstract complexity; they do not eliminate physical constraints. A high-level framework that hides memory management still consumes memory. An AutoML system that tunes hyperparameters still faces the Pareto frontier. Simplifying one interface may shift burden to another, although good design can also remove accidental complexity. The engineer who believes tools eliminate fundamental constraints will be surprised when those constraints resurface at scale, often in forms harder to diagnose than the original problem.

Pitfall: *Optimizing one metric without tracing displaced costs.*

When an optimization reduces latency by 50 percent, systems analysis demands asking what changed elsewhere. Quantization may add validation burden. Caching may trade memory capacity for serving speed. Some simplifications remove accidental complexity; others displace cost. Engineers who celebrate gains in one metric without tracing cross-stack effects build systems that fail in unexpected ways. Rigorous profiling decides which occurred.

Fallacy: *Mastering individual components equals mastering the system.*

Component expertise is necessary but insufficient. An engineer who understands data pipelines, training, serving, and operations as isolated domains will still struggle with systems where a data schema change cascades through training, breaks quantization assumptions, and triggers silent accuracy degradation in production. Integration can create more complexity than the components

reveal in isolation because interfaces multiply failure modes. Systems thinking means understanding how components interact rather than merely how they operate in isolation.

Pitfall: *Scaling data collection without measuring marginal information value.*

The assumption that more data uniformly improves model quality breaks down in practice. As demonstrated in section 9.5, within a fitted scaling regime, additional data can yield diminishing improvements while storage, preprocessing, and labeling costs continue to grow. Coverage, data quality, model capacity, and distribution determine whether that fit remains useful for the next collection increment. The data-gravity principle (2) mandates measuring downstream operational costs, including whether repeatedly moving or scanning a larger dataset exceeds the cost of moving compute toward it. Scaling data volume without quantifying the marginal information value per sample optimizes an expensive proxy.

Fallacy: *A single accuracy metric captures model quality.*

A model evaluated solely on top-line accuracy ignores operational reality. Pareto analysis (principle 5) requires evaluating latency, throughput, memory footprint, energy, fairness, and serving cost simultaneously. Under a 100 ms tail-latency SLO, a 95 percent-accurate model executing in 500 ms is infeasible, whereas a 93 percent-accurate model with a measured 50 ms tail meets that latency requirement. It is a feasible deployment only if its accuracy, subgroup behavior, resource costs, and other acceptance criteria also pass. Furthermore, aggregate accuracy routinely conceals severe error disparities across subgroup cohorts, which disaggregated evaluation is designed to expose. Comprehensive evaluation spans the multi-dimensional Pareto surface rather than a single scalar.

↩ Disaggregated evaluation, which tests each relevant subgroup separately as a release gate, is established in section 15.3.2.

Pitfall: *Treating every drift alarm as an automated rollback trigger.*

Statistical drift detection should trigger a diagnosed response rather than an unconditional automated rollback. Rollback is appropriate when a regression is traced to a flawed binary or pipeline deployment. External covariate drift, however, is not repaired by redeploying an older, equally stale checkpoint. Safe operational responses include canary traffic shedding, fallback heuristics, prediction quarantine, or scheduled retraining once ground-truth labels confirm degradation. Operational monitoring must couple anomaly detection to cause-specific remediation, intervening according to the verified failure mechanism rather than a raw statistical divergence alarm.

Fallacy: *A single optimized pipeline stage makes the system fast.*

Amdahl's Law (principle 8) applies directly to end-to-end ML pipelines. Optimizing accelerator inference latency by 10× yields only 1.1× system speedup if CPU-bound preprocessing accounts for 90 percent of end-to-end latency. Serial fractions of that size can arise from host-side image augmentation, synchronous feature-store lookups, or tokenization that remains host bound in some implementations. The iron law of ML systems (principle 3) decomposes execution time into data movement, computation, and latency terms precisely so that engineers can identify the dominant term before investing optimization effort. The usual organizational cause is that teams profile the model kernel because it is visible, instrumented, and owned by the ML team, while the surrounding data path is split across storage, preprocessing, networking, and application code. End-to-end profiling keeps the target honest, because the stage to improve is the one that limits the system, not the one with the cleanest benchmark harness.

↩ The gap between component latency and end-to-end latency, which makes model-only optimizations yield diminishing returns, is established in section 12.8.2.2.

These fallacies and pitfalls share a common root: attempting to isolate components from the system that executes them. Whether optimizing a single metric, accelerating an isolated stage, or evaluating accuracy without latency constraints, local optimization fails when it ignores system coupling. Reasoning across physical and operational boundaries remains the central discipline of ML systems engineering.

16.8 Summary

↪ A diagnostic matrix that maps common production symptoms to candidate D·A·M axes and the measurements that confirm them is operationalized in section A.8.

ML systems engineering differs from isolated component optimization by reasoning across boundaries. Because decisions couple across data, algorithm, and machine, the deployed system rather than the weights checkpoint is the model, and its costliest failures live at interfaces that no component test exercises. The thirteen principles, the conservation-of-complexity heuristic, and the lighthouse traces that carried them through the stack are the tools for reasoning about that system as a whole.

Their stated assumptions distinguish exact bounds from fitted models, SLO policies, and design heuristics, allowing the tools to remain useful as frameworks, hardware generations, and model families change.

Key Takeaways: Reasoning across boundaries

- **Decisions couple across layers**: In the opening fleet scenario every team met its local metric while accuracy on specific device and firmware cohorts fell by four percentage points, because quantization calibration and a firmware preprocessing path met only in production. Isolated component tests miss such cross-layer interactions; cross-path parity checks, integration tests, and cohort-level outcome tracking test the governing assumptions at the interfaces.
- **Assumptions matter across implementations**: The thirteen principles turn framework-specific craft into measurable reasoning by combining physical bounds, decompositions, fitted models, requirements, and heuristics. Apply each only within its stated scope.
- **Trace displaced costs without assuming conservation**: Compression, batching, monitoring, and governance can relocate burdens across data, algorithm, and machine, while good design can remove accidental complexity outright. Measurement distinguishes the two cases.
- **Boundaries reveal the bottleneck**: In the idealized batch-one, two-H100 model, the memory-transfer lower bound for a Llama 2 70B token is about 295.2× the peak-compute lower bound, and in the illustrative LLM chat endpoint the p99 time to first token is 40× the mean. Systems thinking means measuring where physics, traffic, and users bind.
- **Both halves of the dual mandate are measurable**: Trustworthy behavior is tracked through drift, skew, and fitted feedback factors; physical deliverability through the iron law, the roofline, and energy per event. Neither is certified once at launch; both are budgets the system must keep meeting.

Hennessy and Patterson's *Computer Architecture: A Quantitative Approach* gave processor designers a shared analytical discipline by decomposing CPU time into instruction count, cycles per instruction (CPI), and clock period, so that any proposed improvement could be judged by which factor it changed (Hennessy and Patterson 2011; Hennessy and Patterson 2017). The iron law plays the same role for machine learning systems, and the thirteen principles around it extend that discipline from execution time to energy, trade-offs, and deployed behavior. Together they give a shared analytical baseline for evaluating trade-offs across data, algorithms, and machines.

What endures is the intellectual posture these principles embody: reasoning from empirical evidence and physical bounds rather than reacting to symptoms, quantifying trade-offs rather than following trends, and treating system design as constrained optimization. This is the engineering corollary of the bitter lesson drawn from seven decades of AI research. Because general methods that scale with computation repeatedly outrun hand-crafted expertise, the durable advantage belongs to systems engineering that can absorb that computation, not to any single clever architecture (Sutton 2019). Specific frameworks will rise and fall, hardware generations will turn over, and model architectures will be superseded. Disciplined reasoning about data, computation, and physical constraints will not.

The world is rushing to build AI systems. Our task is to engineer them.

Prof. Vijay Janapa Reddi, Harvard University

BACK MATTER

A

The D·A·M Taxonomy

A system that runs slowly rarely announces why. An idle accelerator can stem from a starved input pipeline (Data), an algorithm with too little arithmetic intensity (Algorithm), or a saturated memory bus (Machine). This appendix formalizes the book's single-node diagnostic taxonomy, mapping profiler symptoms and measurements directly to the axis that actually binds execution before optimization begins. Readers should arrive with the three axes from the opening chapters and the iron law that quantifies them.

How to Use This Appendix

Use the D·A·M taxonomy when a system misses a performance, cost, or quality target and the cause is not yet clear. Work through the framework in four passes:

1. **State the symptom and operating budget**: Record the missed end-to-end target, workload, and deployment context.
2. **Localize the candidate axis or boundary**: Use table A.1 for a first pass and table A.2 when the evidence crosses axes.
3. **Test the hypothesis**: Map the evidence to the iron law and arithmetic intensity, then gather confirming measurements with table A.6 and table A.7.
4. **Intervene and remeasure**: Choose a lever from table A.4 and repeat the end-to-end measurement. If the target does not improve, revise the hypothesis.

The symptom determines the most useful entry point. Low accelerator utilization or slow batch delivery should begin with section A.5 and section A.8. A missed service-level objective (SLO) should move from phase timing and queue depth in table A.5 to the corresponding iron-law term. A quality plateau should be checked against the Data/Algorithm boundary and section A.11 before allocating more compute. In every case, the axis label is a hypothesis, not a verdict: the axes overlap, and a useful diagnosis may name both a dominant axis and a boundary effect.

A.1 Diagnostic Summary

The taxonomy maps directly to the iron law of ML systems established in section 1.7. Table A.1 summarizes the role, primary physical constraint, and core optimization pathway for each axis.

Table A.1: D·A·M Axis Reference: Each axis maps to a constraint class and optimization strategy. Form a hypothesis, confirm it with measurements, then follow the chapter pointer.

Axis	Role	Physical Constraint	High-Leverage Optimization
Data (D)	Information (The Fuel)	Volume, quality, arrival rate	Data Selection (chapter 9)
Algorithm (A)	Logic (The Blueprint)	Operations and dependencies	Model Compression (chapter 10)
Machine (M)	Physics (The Engine)	Compute, memory, I/O	Hardware Acceleration (chapter 11)

This first-pass separation is useful, but production systems rarely suffer from a single axis-centered bottleneck. More often, the problem sits at the boundary between two axes—a data format choice that determines whether the GPU can be saturated, or a pruning strategy that changes the memory access pattern. Handling these cases requires mapping the intersections.

A.2 Intersection Landscape

While the conceptual Venn overlaps in figure 1.4 introduce the intersections of machine learning systems, figure A.1 diagrams these relationships as an operational diagnostic triad. Three circular vertices—Algorithm at the apex (neural network topology), Data at lower left (storage cylinders), and Machine at lower right (compute servers)—are linked by bidirectional coupling arrows. Diagnosing production bottlenecks requires traversing these connections: an engineer first identifies which axis currently binds the workload, then uses the incident edges to predict how an intervention will alter memory, compute, or I/O constraints on the other two.

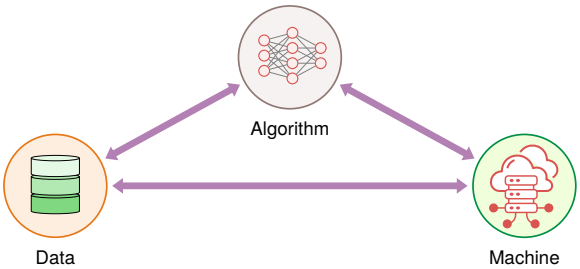


Figure A.1: The D·A·M Axis Triad: A compact diagnostic view of Data, Algorithm, and Machine as three interdependent axes. The bidirectional links emphasize that an intervention on one axis changes the constraints on the other two.

With the first-pass axis identified, table A.2 expands the pairwise and three-way intersections into concrete engineering techniques and their primary book chapters.

Table A.2: D·A·M Intersection Reference: Each zone maps specific techniques to the axes they span and the chapters that cover them. The pairwise intersections require reasoning about two domains simultaneously; the center requires all three.

Zone	Name	Key Techniques	Book Coverage
Data	Information	Storage formats, data quality, distributions	chapter 4
Algorithm	Logic	Loss functions, architectures, gradients	chapter 5, chapter 6
Data ∩ Algorithm	What to Learn From	Data selection, curriculum learning, compute-optimal scaling	chapter 9, chapter 8
Data ∩ Machine	How to Move Information	I/O bandwidth, prefetching, data formats	chapter 4, chapter 11
Algorithm ∩ Machine	How to Execute Efficiently	Quantization, pruning, kernel fusion, mixed precision	chapter 7, chapter 10
Machine	Physics	Silicon, memory hierarchy, peak FLOP/s	chapter 11
Data ∩ Algorithm ∩ Machine	ML Systems Engineering	Iron law, Roofline, training loops, serving	chapter 8, chapter 13, chapter 12

In the D·A·M acronym, Data, Algorithm, and Machine are taxonomy axes. They are not mathematical variables; formal quantities still follow the notation chapter, where D denotes dataset size or training tokens.

The axis-centered zones contain concepts that belong primarily to one axis: storage formats and distributions emphasize Data, loss functions and gradients emphasize Algorithm, and silicon physics and peak FLOP/s emphasize Machine. Single-domain expertise can introduce them, but deployed behavior may still depend on the other axes.

The pairwise intersections are where systems thinking begins. Data ∩ Algorithm (*What to Learn From*) encompasses data selection, curriculum learning, active learning, and scaling laws like Chinchilla ($D \approx 20P$ for the dense autoregressive language-model regime studied) (Hoffmann et al. 2022)—all requiring joint reasoning about information content and algorithmic capacity. Adding data without considering whether the model can learn from it wastes compute; choosing architectures without considering data availability wastes engineering time. Data ∩ Machine (*How to Move Information*) covers I/O bandwidth, prefetching strategies, data formats, and the energy-movement principle. This intersection is where data gravity manifests: the physical cost of moving bytes through the memory hierarchy determines whether the machine can be fed fast enough. Algorithm ∩ Machine (*How*

to Execute Efficiently) spans quantization, pruning, kernel fusion, mixed precision, and computational graph optimization. A pruning strategy that reduces FLOPs but destroys memory access patterns can slow down execution on real hardware.

The center—Data∩Algorithm∩Machine—is where all three axes converge. The iron law, the Roofline Model, end-to-end training loops, serving pipelines, and holistic benchmarking all require simultaneous reasoning about data flow, algorithmic complexity, and hardware utilization. This center is not a single technique; it is the discipline itself.

While the landscape classifies where a technique operates, isolating which axis actively throttles a workload requires translating these relationships into wall-clock execution time—the role of the iron law.

A.3 Iron Law Mapping

For serialized phases, the iron law maps work across the D·A·M axes to execution time. Here, T and L_{lat} are times in seconds; D_{vol} is bytes moved and BW is bytes per second; O is work in FLOPs, R_{peak} is FLOP/s, and η_{hw} is dimensionless hardware efficiency. These quantities map to the axes as follows:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Data/Machine}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Algorithm/Machine}} + \underbrace{L_{\text{lat}}}_{\text{Cross-axis overhead}}$$

Algorithm and Machine share the compute term, separated by which variable the engineer controls. Reducing the total operations (O) is an Algorithm lever, while improving the hardware's peak throughput (R_{peak}) or utilization (η_{hw}) is a Machine lever.

This equation transforms performance debugging from a qualitative guessing game into a quantitative engineering problem. The dominant measured cost appears in one of these terms. A slow system may move too much data (D_{vol}), lack bandwidth (BW), execute too many operations (O), fail to use the hardware's peak capability (η_{hw}), or pay cross-axis overhead (L_{lat}). The levers below map specific optimizations to the variables they improve.

A.3.1 Component levers

- **Data lever**: Reducing moved bytes (D_{vol}) through deduplication or selection, or increasing I/O bandwidth (BW).
- **Algorithm lever**: Reducing operations (O) through pruning or architectural refinement, or reducing the bytes each weight and activation occupies through lower-precision representation. Quantization shrinks D_{vol} and may raise hardware throughput; it does not generally reduce operation count.
- **Machine lever**: Increasing the denominator of the compute term by improving peak throughput (R_{peak}) or increasing the utilization factor (η_{hw}) via kernel fusion.

A.3.2 D·A·M coordination: From sum to max

The additive iron law represents sequential execution. Let $T_{\text{sequential}}$ denote time without overlap and $T_{\text{pipelined}}$ denote measured time with overlap. Overlap can move the sum toward a max, but dependencies, contention, and incomplete overlap keep the measured time above this ideal lower bound:

$$T_{\text{sequential}} = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \quad \xrightarrow{\text{overlap}} \quad T_{\text{pipelined}} \geq \max\left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}\right) + L_{\text{lat}}$$

The systems engineer's job is to make these components run in parallel, not in series. Table A.3 summarizes key D·A·M coordination techniques that overlap work across axes:

Table A.3: D·A·M Overlap Techniques: Each technique overlaps work across axes, moving sequentially added Data/Machine and Algorithm/Machine time toward their ideal lower bound: the larger of the two terms. Dependencies and contention limit realized overlap.

Technique	D·A·M Axes Overlapped	Implementation
Prefetching	D overlaps M	DataLoader with `prefetch_factor`, `pin_memory=True`
CUDA Streams	D overlaps M	Separate streams for H2D transfer and compute
Async Gradient Sync	M (communication) overlaps A	Overlap bucketed AllReduce with remaining backward computation
Double Buffering	D overlaps M	Fill buffer N+1 while computing on buffer N

Overlaps provide the greatest relative speedup when the D·A·M time terms are reasonably balanced. If one term dominates (for example, severely memory bound), overlapping the smaller term with the larger yields negligible gain—the max is still dominated by the same bottleneck. This balance occurs near $D_{\text{vol}}/\text{BW} \approx O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$. The latency term requires separate treatment.

> **Systems Perspective A.1: The overhead that cannot hide**
>
> The unhidden latency term L_{lat} (kernel launch, synchronization barriers, Python dispatch) contains serialization points that cannot be fully overlapped. Kernel fusion can reduce this term by combining operations, but it does not eliminate all launch, scheduling, or synchronization overhead.

While the iron law quantifies the execution time consumed by each stage, determining whether data movement or arithmetic throughput sets the active performance ceiling depends on their ratio—the domain of arithmetic intensity.

A.4 Arithmetic Intensity Boundary

The boundary between the Data/Machine memory term and the Algorithm/Machine compute term is not arbitrary; it is defined mathematically by arithmetic intensity[1] (I) of the workload.

The Roofline Model provides an upper performance bound when traffic and achieved rates are measured consistently. In the middle of a production incident, faster screening heuristics can point to the measurements to collect next.

[1] **Arithmetic intensity**: This book uses the term for floating-point operations per byte transferred. Williams et al. (2009) calls the measured DRAM quantity *operational intensity* in the Roofline Model. Comparing the workload ratio with the hardware ridge point ($R_{\text{peak}}/\text{BW}$) gives an upper-bound diagnosis of memory- versus compute-bound execution.

A.5 Rules of Thumb

In the heat of a production outage, there is rarely time to solve the full iron law equation. Veteran systems engineers instead rely on quantitative heuristics to quickly narrow the search space; the thresholds below are screening signals that should be checked against profiler traces and hardware counters.

- **Low accelerator utilization (< 80 percent)**: Screen for data, CPU, or launch starvation. Confirm with input-pipeline wait, host CPU saturation, memory-bandwidth counters, and trace gaps.
- **High accelerator utilization (> 95 percent)**: Treat this as machine bound only if compute units are busy and memory bandwidth is not saturated. If memory bandwidth is saturated, the bottleneck is still on the Data/Machine boundary.
- **Batch size of one**: Treat this as a warning that latency or launch overhead may matter; confirm with traces before labeling the algorithm itself the bottleneck.
- **Low arithmetic intensity (below ridge point)**: The workload is likely memory bound (Data/Machine boundary). The precise boundary is $R_{\text{peak}}/\text{BW}$ and depends on hardware and precision.
- **Development-production mismatch**: Treat data drift as one hypothesis alongside configuration, load, dependency, and runtime differences.

Common industry labels cross the axes. Memory-bound execution sits on the Data/Machine boundary because both bytes moved and available bandwidth matter. Compute-bound execution couples Algorithm work with Machine throughput. Latency-bound execution may reflect algorithmic serial depth, runtime dispatch, synchronization, or queueing.

A.5.1 Bottleneck diagnostic

Once profiling identifies the dominant term in the iron law, optimizations must target that specific axis. Table A.4 contrasts the interventions that yield end-to-end speedups against nonbinding optimizations that waste engineering effort:

Table A.4: What Works vs. What Is Wasted: Optimizing a nonbinding term yields little or no end-to-end improvement. A memory-bound large language model will not benefit from additional peak FLOP/s alone, though an accelerator with more memory bandwidth may help.

If the workload is…	Dominant Term	Optimization That Works	Optimization That is Wasted
Memory-Bound	D_{vol}/BW	Quantization; structured pruning; batching for weight reuse; fusion that removes intermediate traffic	Faster accelerator (more FLOP/s will not help)
Compute-Bound	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	Better kernels, Tensor Cores, faster accelerator, lower precision	More memory bandwidth (not the binding term)
Latency-Bound	L_{lat}	Reduce serial depth; fuse or dispatch asynchronously; batch within budget	More compute or bandwidth unless the trace shows it binding

Knowing what works also means recognizing what does not. In practice, teams under deadline pressure repeatedly fall into the same traps—optimizing the wrong axis with confidence. These failure modes are common enough to deserve their own names.

A.6 Anti-Patterns

Diagnosing systems is often a process of elimination. Before committing to complex kernel optimizations, watch for these common traps that waste engineering cycles.

- **The hardware crutch**: Buying faster accelerators (*Machine*) to fix a slow Python data loader (*Data*). The new hardware will just idle faster.
- **The model twiddle**: Changing neural architectures (*Algorithm*) when the bottleneck is actually network bandwidth or disk I/O.
- **The premature optimizer**: Writing custom CUDA kernels (*Machine*) before verifying if the Algorithm is simply doing too many unnecessary operations.

Each anti-pattern follows the same root cause: acting before diagnosing. The following case studies show what proper diagnosis looks like—starting from a confusing symptom and systematically narrowing to the dominant D·A·M axis.

A.7 D·A·M Case Studies

Theoretical constraints often manifest as confusing symptoms in production. These representative scenarios illustrate how to apply the taxonomy. Each case follows the same three diagnostic moves: symptom, diagnosis, and fix.

A.7.1 Case 1: The starving accelerator (Data)

A team provisions a large A100 GPU instance to speed up training, but training time hardly improves and `nvidia-smi` shows GPU utilization fluctuating between 10 percent and 40 percent. The pattern suggests accelerator starvation but does not identify its cause. If traces show input-wait gaps with storage or CPU decoding saturation, the data path is binding. The useful intervention is then upstream of the model. Optimize the extract, transform, load (ETL) path by moving from raw JPEGs with heavy CPU decoding to sequential formats such as TFRecords or WebDataset, increasing data-loader parallelism, and prefetching batches into accelerator memory.

A.7.2 Case 2: The latency cliff (cross-axis)

A real-time recommendation service fails to meet a 20 ms latency service level agreement (SLA), while accelerator utilization is low and the batch size is one. That combination suggests launch overhead, memory traffic, or serial model depth rather than a saturated chip. If a trace confirms that the sequential layer path dominates, adding more hardware will not remove it, so pruning or knowledge distillation can reduce model work. If weight traffic dominates instead, quantization can reduce moved bytes by using INT8 where accuracy allows. The measurements select the remedy.

A.7.3 Case 3: The compute wall (Machine)

Accelerator utilization is pinned at 99 percent, memory bandwidth remains unsaturated, and training is stable but takes three weeks. If profiler counters also show high compute-pipeline occupancy and model FLOPs utilization (MFU), the workload is compute bound. The data path is keeping the accelerator fed, so the next step must change the compute term in the iron law. The team can scale up from an A100 to an H100-class accelerator, scale out through data parallelism, or lower precision from FP32/TF32 to BF16 where numerically safe. On NVIDIA Tensor Core paths, BF16 peak throughput is typically about 2× TF32 peak, with realized speedup depending on kernels and bottlenecks (Choquette 2023). Taken together, the three cases show why the same utilization number can imply different next steps depending on loss behavior, batch size, and memory pressure.

Checkpoint A.1: D·A·M diagnosis check

1. A training job shows 95 percent accelerator utilization but loss has plateaued for two epochs. Which D·A·M axes could be responsible, and why is utilization alone insufficient to distinguish them?
2. Your colleague suggests adding more data loader workers to a job where `nvidia-smi` shows 98 percent GPU utilization. What trace evidence is needed before deciding whether this will help?
3. An inference server meets its latency SLO at batch size 1 but fails at batch size 16. Which iron-law terms and queueing effects may have changed, and which measurements identify the binding regime?

These three cases illustrate bottlenecks that cross axes. In live production incidents, symptoms frequently overlap and the dominant bottleneck can shift dynamically as optimizations land. Systematic elimination requires screening profiler signals against a structured diagnostic matrix.

A.8 Production Troubleshooting

Identifying the root cause of performance bottlenecks requires systematic elimination. Table A.5 provides a diagnostic matrix mapping common production failure modes to candidate axes and validating measurements.

Table A.5: D·A·M Diagnostic Matrix: First hypotheses and measurements for common failures. Confirm the binding component before intervening.

Symptom	Hypothesis	Confirm With	Act After Confirmation
Low Accelerator Utilization	Data/host starvation	Trace gaps; loader, CPU, storage, launch counters	Prefetch, parallelize, or cut launches.
High Latency (p99)	Algorithm/runtime/queue	Phase p99 and queue depth	Optimize the dominant phase.
High Training Cost	Low useful-work efficiency	MFU, communication, idle time	Reduce the largest loss or resize.
High Inter-GPU Overhead	Communication bound	Device trace; collective gaps	Bucket AllReduce; tune data-parallel overlap.
Out-of-Memory (OOM)	Algorithm/Machine fit	Peak weights, states, activations	Checkpoint, shard, quantize, or shrink batch.

A.9 Tooling Map

A diagnostic hypothesis requires concrete hardware and runtime counters before taking corrective action. Table A.6 connects theoretical D·A·M components to the specific Linux and Python profiling utilities that expose them.

Table A.6: D·A·M Tooling Map: Profiling utilities for diagnosing bottlenecks along each D·A·M axis. Start with the primary tool for quick triage; use secondary tools for deep-dive analysis when the primary tool's output is inconclusive.

Axis	Key Metric	Primary Tool	Secondary Tool
Data	Batch Load and Wait Time	Framework profiler trace	`iostat`, `pidstat` (I/O/CPU)
Algorithm	FLOPs, Model Depth	PyTorch Profiler	DeepSpeed Flops Profiler
Machine	Accelerator Utilization, SM Occupancy	Nsight Compute	`nvidia-smi`, Nsight Systems

Profiling tools generate raw metrics—utilization percentages, FLOP counts, and bandwidth measurements—that become actionable only when evaluated against workload constraints. The D·A·M Scorecard establishes screening signals rather than universal grades, isolating which measurements warrant immediate intervention.

A.10 D·A·M Scorecard

To move beyond qualitative guessing, the efficiency indicators in table A.7 help decide what to investigate next. This report-card view aids the first comparison, but no row provides a universal grade. MFU[2] is especially useful for large-model training.

Table A.7: D·A·M Screening Indicators: Context-dependent signals for investigation, not universal grades.

Axis	Metric	Definition	Investigate	Strong Signal
Data	I/O Overhead	$\frac{\text{Data Wait Time}}{\text{Total Step Time}}$	$>10\%$ (screen)	$<1\%$ (strong)
Algorithm	Avoidable Work	Profiler or ablation finds redundant operations	Redundant work found	End-to-end work falls
Machine	MFU	$\frac{\text{Achieved model FLOP/s}}{\text{Peak FLOP/s}}$	$<30\%$ (large models)	$>50\%$ (same regime)

The Scorecard and the Roofline Model both answer efficiency questions, but at different scales. The Scorecard screens the current system using context-dependent indicators. Scaling laws and the information-roofline metaphor ask what may happen as the system scales beyond its current size.

A.11 Scaling Laws vs. Roofline

Systems engineering requires distinguishing between growth trajectories and fundamental limits. While empirical scaling laws characterize how loss decreases as compute, data, or parameters expand, roofline bounds identify the hard physical ceilings imposed by memory bandwidth and arithmetic throughput.

A.11.1 Scaling laws (the journey)

Scaling laws[3] are empirical power laws that predict how held-out loss changes as resources increase within the fitted regime. Two landmark results are Kaplan scaling (Kaplan et al. 2020), which studied loss against parameter count (P), data (D), and total operations (O), and Chinchilla scaling (Hoffmann et al. 2022), which estimated a compute-optimal balance for the dense autoregressive

[2] **MFU (model FLOPs utilization)**: The ratio of achieved model FLOP/s to the hardware's theoretical peak FLOP/s, introduced in the PaLM paper (Chowdhery et al. 2022). Unlike raw accelerator utilization (which counts any work the accelerator performs), MFU measures model computation rate relative to peak hardware throughput, so non-model work and system overhead are not counted as achieved model FLOPs. Section 8.6 develops MFU as a training diagnostic.

[3] **Scaling laws**: Empirical relationships, typically power laws with a fitted negative exponent, that predict model loss as a function of dataset size, parameter count, or compute budget. Kaplan et al. (2020) studied these relationships for neural language models at OpenAI; Hoffmann et al. (2022) later estimated the compute-optimal training trade-off for its studied dense language-model regime. Section 9.6.2 develops the compute-optimal frontier and its data-starvation diagnostic in detail.

language-model regime studied, often summarized as roughly 20 training tokens per parameter ($D \approx 20P$).

As economic guides, these fits estimate how held-out loss changes with compute inside the observed regime. They do not guarantee a task error-rate reduction or extrapolate unchanged across architectures and data distributions.

A.11.2 Information roofline (the destination)

The **information roofline** is used here as a diagnostic metaphor rather than a standard quantitative law. Its destination image points to limits on what can be learned from the available data. For classification under a fixed data distribution and loss, the Bayes error rate[4] is an irreducible error floor, not a ceiling. Noise, label ambiguity, coverage gaps, and distribution shift can create data-quality limits, but they do not define one universal slope or breakpoint.

[4] **Bayes error rate**: The lowest achievable error rate under 0–1 loss for any classifier on a given data distribution, determined by class priors and the overlap between class-conditional distributions (Goodfellow et al. 2016). No amount of data, parameters, or compute can reduce error below this theoretical floor.

The diagnostic lesson is narrower. Scaling laws describe a fitted improvement trajectory, while the information-roofline metaphor prompts a search for data-quality limits. If a loss curve flattens relative to the fitted trend, investigate optimization, capacity, evaluation noise, and data quality; flattening alone does not identify which cause dominates. The Bayes floor describes the best achievable classifier for the specified distribution. Adding accelerators or parameters is futile only after measurements establish that data quality is binding. Whether debugging a training step, evaluating hardware utilization, or planning a scaling campaign, measure which axis is binding before choosing a lever.

A.12 Summary

These tools form a single diagnostic workflow. D·A·M localizes candidate causes across Data, Algorithm, and Machine. The iron law and arithmetic intensity translate those causes into time, operations, and bytes, while the case studies, anti-patterns, tooling map, and scorecard show how to test them in production. Scaling laws and the information roofline then separate observed improvement trajectories from data-quality limits. Use the framework as a loop rather than as a label.

Key Takeaways: Where to look first

- Begin with cross-axis hypotheses; a symptom rarely identifies its cause.
- Translate each hypothesis into the relevant iron-law term and a measurement that could falsify it.
- Use arithmetic intensity and the scorecard to identify the binding constraint, not to assign universal grades.
- Apply the corresponding lever and remeasure end to end; a local gain that leaves the binding term unchanged is not a system improvement.

B

Data Foundations

DATA problems rarely present as data problems; they surface as a training run that will not speed up, an accelerator sitting idle, or accuracy eroding while dashboards stay green. To keep chapter arguments focused on system design, this appendix gathers the back-of-the-envelope calculations and statistical tools that trace such symptoms back to measurable data bottlenecks: transfer times, storage-layout costs, drift diagnostics, and numerical-stability tools. The tools here lean on basic probability and on the data term of the iron law.

How to Use This Appendix

This appendix is designed as a reference. Reach for it when debugging "slow training," "low accelerator utilization," or "production accuracy drift" calls for the quickest path from symptom to measurement.

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW denotes bandwidth).

- **When data will not move**: Start with table B.1 and the transfer-time equation in section B.1.1.
- **When the accelerator is starving**: Use table B.2 and the layout discussion in section B.1.3.
- **When pipelines explode in cost**: Use the primitives in section B.1.4, especially join-induced shuffles.
- **When "average looks fine" but users complain**: Use section B.2.1 and session-level tail probability.
- **When accuracy drifts silently**: Use section B.2.2 to compare full distributions.

While production pipelines encounter an overwhelming variety of formats, encodings, and storage layers, this appendix focuses on the physical and statistical constants that govern them all.

With those reference points in place, the appendix begins with the physical constraints of data engineering and then moves to the statistical monitoring that keeps ML systems healthy. From storage formats that determine I/O throughput to drift metrics that detect silent failures, these foundations connect directly to the data pipelines in section 4.5 and the operational monitoring in section 14.6.2.

B.1 Data Engineering Foundations

Understanding hardware constraints is only the first requirement; systems engineering must also shape data to fit them. Data engineering applies the principles of the memory hierarchy to storage formats and pipeline design, ensuring that the accelerator never starves. This process begins with recognizing that data is physical—it has volume, it takes time to move, and it requires energy to parse.

B.1.1 Napkin math: The physics of data gravity

Data gravity[1] is a metaphor grounded in transfer-time calculations. Unlike compute, which gets faster every year, the speed of light is fixed and network bandwidth is a finite resource. When datasets grow large enough, their practical inertia grows—moving them can cost more time and energy than moving computation to where the data already lives.

The ideal line-rate transfer time is $T = D_{\text{vol}}/\text{BW}$ after converting data volume and bandwidth to compatible units (the calculations below convert decimal bytes and bits per second explicitly). For

[1] **Data gravity**: Coined by Dave McCrory in 2010 to describe how large datasets attract services and applications toward them, much as massive bodies attract smaller ones in physics (McCrory 2010). The analogy is apt: the "escape velocity" required to move a petabyte-scale dataset is often measured in weeks.

the large volumes here, latency is negligible; the full equation appears in section D.3.4. Table B.1 gives lower bounds; protocol overhead and endpoint limits increase them.

Table B.1: The Cost of Inertia: These transfer times explain why we often "ship compute to data" rather than "ship data to compute." Network columns are lower bounds at sustained line rate. The truck entry is an illustrative elapsed-time model for 1 PB: 8 hours to load, 32 hours in transit, and 8 hours to unload, excluding scheduling and provisioning overhead.

Data Volume	1 Gbps (Standard WAN)	10 Gbps (High-End WAN)	100 Gbps (Direct Connect)	Truck (Illustrative)
1 TB	2.2 hours	13.3 minutes	80 seconds	N/A
100 TB	9 days	22.2 hours	2.2 hours	N/A
1 PB	3 months	9 days	22.2 hours	2 days

B.1.2 The cost of serialization

Even after data arrives at the machine, the serialization tax remains a final hurdle.[2] As table B.2 shows, many engineers meticulously optimize their accelerator kernels while ignoring the CPU overhead of decoding data. Parsing text-based formats like JavaScript Object Notation (JSON) or CSV is extremely CPU-intensive, often leaving the accelerator idling while the CPU struggles to convert strings into floating-point numbers.

[2] **Serialization**: From Latin *serialis* (forming a series). The process of converting in-memory data structures into a byte stream for storage or transmission, and the reverse (*deserialization*). In ML pipelines, the choice of serialization format can dominate end-to-end training time; section 4.5.1 covers pipeline design strategies that minimize this overhead.

Table B.2: Serialization Overhead: These illustrative values show why binary columnar formats can outperform row-based text, but they are not portable benchmarks. Arrow is an in-memory/IPC format; Parquet is a compressed storage format.

Format	Decoding Speed (MB/s)	Relative CPU Decode Cost	Suitability
CSV/JSON	~100 MB/s	High	Inspection/interchange
Protobuf	~300 MB/s	Medium	RPC/Messages
Parquet	~1,000 MB/s	Low	Columnar storage

B.1.3 Row vs. columnar formats

The choice of file format determines the "physics" of how data traverses the storage hierarchy. Row-oriented formats such as CSV and JSON store data record-by-record, so reading the `age` field still requires parsing every attribute in each row. This layout suits append-only logs but incurs steep overhead for feature filtering. Parquet stores compressed column chunks on disk, while Arrow provides a contiguous columnar in-memory and IPC representation. Parquet readers exploit *projection pushdown* to fetch selected column chunks directly, and both formats support *vectorized processing*. Figure B.1 diagrams this structural contrast: where the row layout forces a wide scan across all fields (`ID`, `Age`, `Loc`), the columnar layout isolates the target `Age` chunk, avoiding data movement for irrelevant attributes.

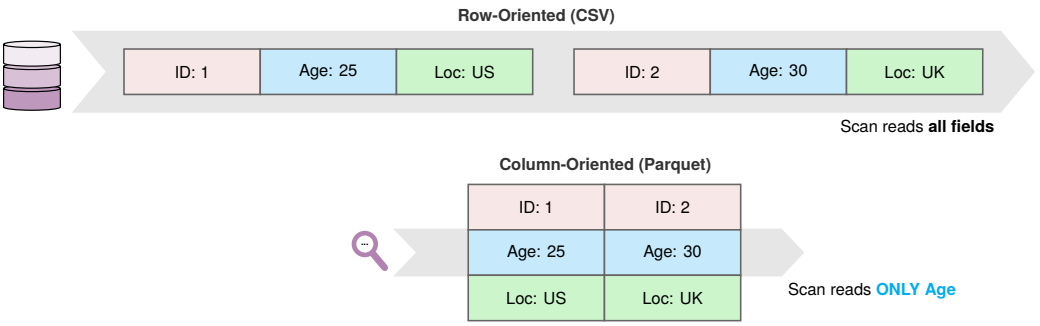


Figure B.1: Storage Layouts: Row-oriented formats group values by record, so the illustrated scan reads every field. Column-oriented formats group values by feature, so the illustrated scan reads only Age.

This physical partitioning in figure B.1 isolates how storage layout dictates bus traffic: extracting a single feature from a row store forces every intervening attribute into the host memory bus, whereas

columnar chunking allows the storage subsystem to skip unreferenced columns entirely before deserialization.

> **Systems Perspective B.1: The accelerator starvation problem**
>
> The choice of file format often determines whether a training pipeline is I/O bound or compute bound. Because row-oriented formats enforce full-record scans while columnar layouts enable projection pushdown, storage layout compounds the serialization tax whenever models consume a subset of tabular features. When ingest pipelines fail to filter at the storage layer, the data-movement term in the iron law can silently become the critical-path bottleneck, starving accelerators and leaving matrix cores idling.

B.1.4 The algebra of data

Feature engineering turns raw records into the columns a model can learn from, and that transformation is usually a dataflow problem before it is a modeling problem. A pipeline first narrows the population, then chooses the features, then attaches context from other tables. Those three moves correspond to three Structured Query Language (SQL) primitives, and their computational cost determines whether the feature job stays local, scans unnecessary bytes, or turns into a network shuffle.

1. **Selection (σ)**: Filtering rows (for example, `WHERE age > 30`).
   - **Cost**: For a table of N rows, $\mathcal{O}(\log N + K)$ if a B-tree index lookup returns K rows; $\mathcal{O}(N)$ for a full scan.
2. **Projection (π)**: Selecting columns (for example, `SELECT age`).
   - **Cost**: Selected chunks plus decoding and metadata in columnar formats. This row-format example reads 1 KB for a 4-byte integer, wasting 99.6 percent of the read.
3. **Join ($\bowtie$)**: Combining tables. The most expensive operation.
   - **Shuffle Join**: Both tables are partitioned by key and exchanged over the network.
     - **Cost**: Reshuffling both 1 TB tables moves about ~2 TB; partitioning and locality can reduce this.
   - **Broadcast Join**: One small table is sent to all workers.
     - **Cost**: $s_{\text{table}}(N_{\text{workers}} - 1)$ bytes; the s_{table}-byte table must fit on each of N_{workers} workers.

Understanding data formats, serialization costs, and algebraic primitives tells how to move data efficiently. Even a perfectly engineered pipeline, however, can silently fail if the data it carries changes character over time. Detecting that change—and quantifying how much it matters—requires a different set of tools: probability and statistics.

B.2 Probability and Statistics

Once data is flowing through our pipelines, we need mathematical tools to ensure its quality and consistency. Probability and statistics provide the language for monitoring system health, detecting the silent failures of data drift, and managing uncertainty.

B.2.1 Distributions and the long tail

In systems, the mean is often misleading. Latency distributions are often long-tailed, and user-visible services are commonly governed by high-percentile behavior rather than mean response time (Dean and Barroso 2013). A "p99" (99th percentile) latency of 500 ms means at most 1 percent of requests take longer than 500 ms; with many requests per session, the fraction of users who see at least one slow request can be much higher. At scale (1M users), even a 1 percent affected-user rate would affect 10,000 users.

> **Napkin Math B.1: The median experience**
>
> For $N_{\text{session}} = 100$ independent requests, each with a 0.01 chance of being slow, the probability of at least one p99-tail event is $\Pr(\text{Slow}) = 1-(0.99)^{100} \approx 1-0.366 = 63.4\%$.
>
> **Systems insight**: Under this model, a 100-request session is more likely than not to include an event beyond p99. Correlated slowdowns can change this probability (Dean and Barroso 2013).

B.2.2 Measuring drift (divergence)

Because distributions have long tails where the most dangerous failures hide, simple metrics like "mean shift" are insufficient. We need tools that compare the entire shape of the distribution. Detecting drift between the serving distribution (P_t) and training distribution (P_0) requires measuring the "distance" between distributions.

For discrete outcomes x, equation B.1 expresses KL divergence[3] as the expected extra coding cost of approximating P_t with P_0 (Kullback and Leibler 1951; Cover and Thomas 2006). The displayed form is finite only when $P_0(x) > 0$ wherever $P_t(x) > 0$; production implementations therefore need explicit smoothing or zero handling.

$$\mathcal{D}_{\text{KL}}(P_t \| P_0) = \sum_x P_t(x) \log \frac{P_t(x)}{P_0(x)} \tag{B.1}$$

[3] **KL divergence**: Named after Solomon Kullback and Richard Leibler, who introduced it in 1951. Also called *relative entropy*, it quantifies the expected extra information needed to encode samples from one distribution using a code optimized for another; for drift monitoring in this book, the canonical direction is $\mathcal{D}_{\text{KL}}(P_t \| P_0)$. With natural logarithms, as in the example above, the unit is nats; with base-2 logarithms, the unit is bits. In production ML, KL divergence is the theoretical backbone of many drift-detection metrics.

> **Napkin Math B.2: Worked example: KL divergence for drift detection**
>
> **Scenario**: A sentiment classifier was trained on data where 60 percent of reviews were positive, 30 percent negative, and 10 percent neutral. After deployment, the serving distribution shifts to 45 percent positive, 40 percent negative, and 15 percent neutral.
>
> **Math**:
>
> Training distribution P_0: [0.60, 0.30, 0.10]. Serving distribution P_t: [0.45, 0.40, 0.15].
>
> $$\begin{aligned}\mathcal{D}_{\text{KL}}(P_t \| P_0) &= 0.45 \log \frac{0.45}{0.60} + 0.40 \log \frac{0.40}{0.30} + 0.15 \log \frac{0.15}{0.10} \\ &= 0.45 \times (-0.2877) + 0.40 \times (0.2877) + 0.15 \times (0.4055) \\ &= -0.1295 + 0.1151 + 0.0608 = 0.0464 \text{ nats}\end{aligned}$$
>
> $$\begin{aligned}\mathcal{D}_{\text{KL}}(P_0 \| P_t) &= 0.60 \log \frac{0.60}{0.45} + 0.30 \log \frac{0.30}{0.40} + 0.10 \log \frac{0.10}{0.15} \\ &= 0.60 \times 0.2877 + 0.30 \times (-0.2877) + 0.10 \times (-0.4055) \\ &= 0.1726 + (-0.0863) + (-0.0405) = 0.0458 \text{ nats}\end{aligned}$$
>
> Notice the asymmetry: $\mathcal{D}_{\text{KL}}(P_0 \| P_t) \neq \mathcal{D}_{\text{KL}}(P_t \| P_0)$. The population stability index (PSI) symmetrizes this; the calculation below uses natural logarithms:
>
> $$\begin{aligned}\text{PSI} = \sum (P_{0,i} - P_{t,i}) \log \frac{P_{0,i}}{P_{t,i}} &= (0.15)(0.2877) + (-0.10)(-0.2877) + (-0.05)(-0.4055) \\ &= 0.04315 + 0.02877 + 0.02027 = 0.0922\end{aligned}$$
>
> **Systems insight**: Since PSI = 0.0922 < 0.1, this drift sits just below the conventional warning threshold and well short of the critical threshold of 0.25 used in chapter 14. The right response is to keep the feature on the monitoring dashboard and look for corroborating drift signals rather than retraining from this statistic alone.

The threshold remains illustrative: its meaning depends on sample size, binning, smoothing, and zero handling, and retraining requires model-quality evidence.

Both KL divergence and PSI are grounded in a deeper framework—information theory—which provides the units and bounds that make these metrics principled rather than *ad hoc*.

B.2.3 Information theory for systems

A training run can consume more examples, run longer, and still stop improving if the added data carries too little useful signal. At that point, the bottleneck is not only compute or bandwidth; it is the amount of information the data pipeline delivers to the learner. Section A.11.2 treats that data quality limit as a diagnostic constraint, and information theory provides the units for reasoning about it.

- **Entropy (H)**:[4] The average uncertainty in a distribution, defined as $H(X) = -\sum p(x) \log p(x)$. The log base sets the unit: natural logs give nats, while $\log_2$ gives bits. A uniform distribution has maximum entropy only over a fixed finite support. This connects directly to KL divergence above: $\mathcal{D}_{\text{KL}}$ measures the excess coding cost of using the wrong distribution.
- **Information density**: Used here as a book-level heuristic for useful learning signal per unit of storage, not as a standard information-theory quantity.
- **Signal-to-noise ratio (SNR)**: The ratio of signal power to noise power. In ML, the book uses low SNR qualitatively for data in which task-relevant structure is weak relative to noise; adding compute cannot compensate for missing task-relevant signal.

B.2.4 Logits and numerical stability

Neural-network classifiers commonly output a vector of *logits*[5] $\mathbf{z}$ (unnormalized scores). For class index j, softmax normalizes z_j over all class indices k to produce a probability:

$$\text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

The problem is that if z_j is large (for example, 100), the exponential e^{z_j} overflows common training and inference formats such as FP32, BF16, and FP16. FP64 can represent this specific value, but production ML kernels rarely use FP64 for softmax. The solution is to compute in log-space: the "Log-Sum-Exp" trick allows us to compute $\log\left(\sum_k e^{z_k}\right)$ without ever calculating the massive exponentials directly, preserving numerical precision.[6]

A small worked example shows why this trick matters in practice, even for large but realistic logit values:

> **Napkin Math B.3: Log-sum-exp in action**
>
> **Setup**: A three-class classifier outputs logits $\mathbf{z} = [100, 101, 102]$.
>
> **Without the trick (naive softmax)**:
>
> $$\exp(100) \approx 2.7 \times 10^{43},\ \exp(101) \approx 7.3 \times 10^{43},\ \exp(102) \approx 2.0 \times 10^{44}$$
>
> These numbers are representable in FP64 but overflow FP32 (max $\approx 3.4 \times 10^{38}$). With FP16 (max $\approx 65{,}504$), even e^{12} overflows. Raw logits rarely reach magnitude 100, but large intermediate values do arise (for example, unscaled attention scores or overconfident outputs), and because FP16 and BF16 are the standard training precisions, a naive softmax risks overflow often enough that the stable form is used by default.
>
> **With the trick**: Subtract $m = \max(\mathbf{z}) = 102$:
>
> Shifted logits: $\mathbf{z} - m = [-2, -1, 0]$
>
> Exponentials: $\exp(-2) \approx 0.135$, $\exp(-1) \approx 0.368$, $\exp(0) = 1.0$
>
> Sum = 1.503. LogSumExp = $102 + \log(1.503) = 102.408$.
>
> Softmax: $[0.135/1.503,\ 0.368/1.503,\ 1.0/1.503] = [0.090,\ 0.245,\ 0.665]$
>
> **Systems insight**: The exponentiated shifted values and the resulting softmax probabilities are in $[0, 1]$—no overflow risk, even in FP16.

Together, these worked examples connect the full chain, from physical data movement to distribution shift and numerical stability.

[4] **Entropy**: Shannon adopted the term in 1948 to quantify information content (Shannon 1948). In systems terms, Shannon entropy is a lower bound on average lossless code length. Practical prefix codes approach it, and block codes can approach it asymptotically. With $\log_2$, the unit is bits; natural logs give nats. These bounds inform compression ratios and data-pipeline sizing.

[5] **Logit**: Joseph Berkson coined the term in 1944 by analogy with *probit*. The logit function is the inverse of the logistic (sigmoid) function: $\text{logit}(p_{\text{sig}}) = \log(p_{\text{sig}}/(1 - p_{\text{sig}}))$. In deep learning, "logits" refers more loosely to the raw, unnormalized output of the final linear layer before any activation function is applied.

[6] **Log-sum-exp**: Implemented as `torch.logsumexp` in PyTorch and `scipy.special.logsumexp` in SciPy. It relies on the identity $\log(\sum e^{z_k}) = m + \log(\sum e^{z_k - m})$, where $m = \max(z_k)$. Shifted values $z_k - m$ are ≤ 0, ensuring exponentials never overflow.

B.3 Summary

The tools in this appendix—tail-aware metrics, drift divergences, information-theoretic bounds, and numerical stability techniques—support quantitative diagnosis of data-related failures rather than anecdotal explanations.

C

Algorithm Foundations

A profile showing low utilization is often misdiagnosed as a hardware limit when the cause is algorithmic: a matrix multiply shape that wastes Tensor Cores, a layout that breaks contiguity, or activation memory held for backpropagation. This appendix collects the computational complexity models and memory footprint derivations that the book's performance chapters lean on, enabling readers to evaluate algorithmic trade-offs before committing code to accelerators. It assumes familiarity with linear algebra and with the iron law introduced early in the book.

How to Use This Appendix

This appendix is designed as a reference. Reach for it when translating a profiler symptom ("slow matmul," "shape mismatch," "OOM during training") into a concrete computational or memory cause.

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW denotes bandwidth).

- **When general matrix multiply (GEMM) kernels are slow**: Use section C.1.4 and compare intensity to the hardware's ridge point.
- **When memory blows up in training**: Use section C.3.3 and the training memory decomposition.
- **When tensor code "should work" but does not**: Use section C.2.2 and section C.2.3.
- **When sparsity is proposed as a fix**: Use section C.1.5 to check density and metadata overhead.
- **When proving gradient compute bounds**: Use section C.4 for the Baur–Strassen reverse-mode automatic differentiation (AD) complexity theorem.
- **When analyzing sequence length bottlenecks**: Use section C.5 for the online softmax recurrence and FlashAttention high-bandwidth memory (HBM) I/O reduction proof.
- **When allocating compute between parameters and tokens**: Use section C.6 for the Kaplan and Chinchilla compute-optimal scaling law derivation.

Neural networks turn linear algebra into learned behavior: matrices transform activations, tensor layouts determine how data move through memory, and backpropagation carries error signals through the resulting computation. These foundations support the deep learning treatment in chapter 5, the framework internals in chapter 7, and the training strategies in chapter 8. Architectures change, but the underlying mathematical machinery remains constant.

C.1 Linear Algebra

Deep learning systems are, at their core, engines for transforming massive matrices. Frameworks like PyTorch abstract away the raw math, but performance engineering still depends on the underlying linear algebra. Building on how numbers are stored in section D.4, this section focuses on how they are manipulated.

> Systems Perspective C.1: Why this matters
>
> Many dense neural networks, especially transformers and large convolutional neural networks, spend most compute in matrix multiplication or GEMM-like kernels. A self-attention block

performs the Q, K, V, and output projection GEMMs plus two attention matrix multiplications: one for the attention scores and one for the attention-weighted values. The layer's feed-forward block adds more GEMMs. Understanding GEMM performance characteristics explains why batch size affects throughput, why certain layer dimensions are "better" than others, and how to interpret profiler output. Reasoning about matrix dimensions and arithmetic intensity predicts whether a dense workload is compute bound or memory bound before any profiler trace runs.

C.1.1 Tensor operations and notation

Einstein summation[1] notation makes complex operations explicit throughout this book (implemented as `torch.einsum` in PyTorch and `np.einsum` in NumPy). Matrix multiplication $\mathbf{C} = \mathbf{AB}$ becomes:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

In einsum notation, this is `ik,kj->ij`. The notation extends naturally to the multi-dimensional operations in attention mechanisms. For example, batched multi-head attention is `bhid,bhjd->bhij` (batch b, head h, query sequence i, key sequence j, and head dimension d).

[1] **Einstein summation convention**: Repeated indices in a product are implicitly summed over, eliminating explicit summation signs. ML frameworks adopted the convention because it concisely expresses arbitrary tensor contractions in a single string.

C.1.2 Memory layouts and performance

Data layout in memory (row-major vs. column-major) directly affects cache efficiency. When iterating over a matrix, accessing contiguous memory locations is dramatically faster than strided access.

A common optimization pattern is to materialize a transposed tensor once before repeated operations to ensure contiguous access in the hot loop. The one-time copy cost is amortized across many subsequent operations.

C.1.3 The dot product as similarity

The dot product $\mathbf{a} \cdot \mathbf{b} = \sum a_i v_i$ is geometrically equivalent to $\|\mathbf{a}\|\|\mathbf{b}\| \cos\phi$, which makes it a natural measure of similarity between two vectors: for nonzero vectors, its sign distinguishes acute, right, and obtuse angles, while its magnitude also depends on both vector norms.

This geometric interpretation is why dot products appear everywhere in modern architectures. In attention mechanisms, query (**Q**) and key (**K**) vectors are dot-produced to compute a similarity score that determines how much each token attends to every other token. The resulting attention weights are then used to form a weighted combination of value (**V**) vectors—making the dot product the foundation of the transformer's ability to model long-range dependencies.

C.1.4 General matrix multiply (GEMM)

GEMM[2] is the computational workhorse of deep learning. For matrices of size $M \times K$ and $K \times N$, GEMM performs $2MNK$ floating-point operations (multiply-accumulate counts as two operations).

The arithmetic intensity of GEMM scales linearly with matrix dimension. For square $N \times N$ matrices in FP16 (2 bytes/element), the ideal $\beta = 0$ bound reads **A** and **B** once and writes **C** once:

$$\text{Intensity} = \frac{O}{D_{\text{vol}}} = \frac{2N^3}{3N^2 \times 2} = \frac{N}{3} \text{ FLOP/byte}$$

Reading the old **C** when $\beta \neq 0$ changes the intensity to $N/4$ FLOP/byte. Three systems consequences govern GEMM execution:

- **Larger batches can improve efficiency**: Batching increases effective matrix dimensions when it forms larger GEMMs or enables reuse; merely queuing independent small GEMMs does not raise intensity.
- **Aligned dimensions help**: Hardware Tensor Cores are optimized for precision- and architecture-specific tile multiples. Dimensions that align with these multiples avoid padding overhead and improve kernel efficiency, but they do not need to be powers of two.
- **Small matrices are inefficient**: A square GEMM with $N = 64$ has intensity $64/3 \approx 21.3$ FLOP/byte, well below the ridge point (153.0 FLOP/byte). Its roofline cap is ~13.9 percent of peak; launch and tiling overhead can lower realized throughput.

[2] **General matrix multiply (GEMM)**: The BLAS (Basic Linear Algebra Subprograms) family began with vector routines standardized in 1979 (Lawson et al. 1979); GEMM was standardized later as a Level 3 routine (Dongarra et al. 1988). The "GE" prefix stands for "general," as opposed to symmetric or triangular forms. GEMM computes $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$ and is a performance-critical routine in deep learning. Section 8.4.1.2 explains how GEMM shapes determine training throughput.

C.1.5 Sparse matrix formats

When most elements in a matrix are zero, specialized storage formats avoid wasting memory on zeros and enable computations that skip them entirely. The **compressed sparse row (CSR)** format uses three arrays:

- `Values`: The nonzero elements, stored in row order
- `Col_Idx`: The column index of each nonzero element
- `Row_Ptr`: The starting position in `Values` for each row (length = num_rows + 1)

CSR is useful for sparse feature matrices in recommendation pipelines and for pruned model weights. For a matrix with n_{elem} elements, n_{rows} rows, and n_{nz} nonzeros, CSR uses $\mathcal{O}(n_{\text{nz}} + n_{\text{rows}})$ storage instead of $\mathcal{O}(n_{\text{elem}})$; when n_{nz} is large relative to n_{rows}, this is often summarized as $\mathcal{O}(n_{\text{nz}})$.

To see the trade-off concretely, consider a vocabulary embedding matrix with 100,000 rows and 10,000 columns (1B parameters):

- **Dense (FP32)**: 1B parameters $\times$ 4 bytes each = 4 GB.
- **Sparse** (1 percent density): CSR stores roughly 10M entries $\times$ (4-byte value + 4-byte column index), plus one row pointer per row and a final pointer, totaling $\approx$ 80.4 MB.
- **Result**: A 50$\times$ reduction in memory footprint, fitting a model that would otherwise trigger an out-of-memory (OOM) error.

Linear algebra tells us *what* to compute; the next question is *how* to express those computations in code. Tensor programming primitives—shapes, strides, and broadcasting—bridge the gap between mathematical notation and the array operations that actually execute on hardware.

C.2 Tensor Programming Primitives

A shape mismatch crash, a silently wrong broadcast, a kernel running at 5 percent of peak because of a noncontiguous tensor—these common ML engineering failures all trace back to the same layer of abstraction. *Tensor programming* translates the abstract math of linear algebra into concrete array manipulations that run on hardware.

C.2.1 Computational complexity cheat sheet

Table C.1 provides a quantitative reference for the most common building blocks. Use these formulas for back-of-the-envelope estimation of model size and compute requirements before hardware is provisioned. Given layer dimensions and input shapes, the formulas provide first-order parameter and compute estimates. The attention row is the key warning: sequence length contributes an S^2 term, while hidden dimension contributes d^2 projection work, so long-context models can shift the dominant cost without changing the parameter count.

Table C.1: Deep Learning Tensor Primitives: Summary of shapes, parameters, and FLOP counts. Note: B is batch size, S is sequence length, and K is kernel size. The attention FLOPs include QKV projections and score interactions. The S^2 term applies to full-sequence training and prefill, while KV-cached generation is linear per token in cached context; d^2 projection work dominates at large hidden dimensions.

Layer type	Output shape	Parameters (P)	FLOPs (per forward pass)
Linear	(B, d_{out})	$d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$ if bias is enabled	$2 \times B \times d_{\text{in}} \times d_{\text{out}}$
Conv2D	$(B, d_{\text{out}}, H_{\text{out}}, W_{\text{out}})$	$K^2 \times d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$ if bias is enabled	$2 \times B \times H_{\text{out}} \times W_{\text{out}} \times K^2 \times d_{\text{in}} \times d_{\text{out}}$
Multi-head self-attention	(B, S, d)	$4 \times d^2$, plus $4 \times d$ if projection biases are enabled	$B \times (4S^2d + 8Sd^2)$
LayerNorm	(B, S, d)	$2 \times d$ if affine scale and bias are enabled	$\mathcal{O}(B \times S \times d)$

C.2.2 Shapes and strides

A tensor is a view over an underlying storage buffer, described by shape, stride, dtype, and offset metadata. Its shape specifies the dimensions of the tensor (for example, `(3, 4)`), while its stride defines the number of elements to skip in memory to move to the next element along each dimension. Only contiguous tensors lay their logical elements out as one adjacent block.

Operations like `transpose()` or `view()` often just change the strides, not the data in memory. This is fast ($\mathcal{O}(1)$) but can lead to noncontiguous tensors that fail in kernels requiring contiguous data. In such cases, calling `contiguous()` on a tensor of N elements forces an $\mathcal{O}(N)$ memory copy that can dominate runtime if triggered repeatedly inside a loop.

C.2.3 Broadcasting

Broadcasting allows arithmetic operations on tensors of different shapes by comparing dimensions from right to left. Two dimensions are compatible when they are equal or when one of them is 1.

Figure C.1 diagrams this virtual expansion across complementary singleton dimensions, adding a $(3,1)$ column tensor (**A**) to a $(1,4)$ row tensor (**B**) to generate a $(3,4)$ matrix of pairwise sums $(A_i + b_j)$. Dashed cell boundaries mark result elements whose operands are read through stride-0 views, so neither input is copied; only the $(3,4)$ output is allocated, and hardware repeatedly accesses identical input buffer addresses.

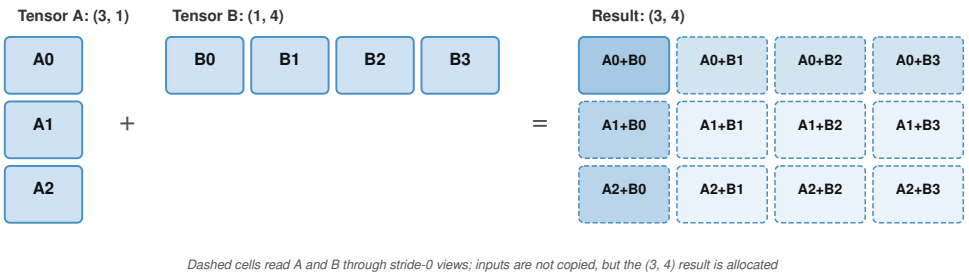


Figure C.1: Tensor Broadcasting Rules: A (3,1) column and a (1,4) row broadcast across complementary dimensions, producing the pairwise sums in a (3,4) result.

Consider a concrete case: tensor A has shape `(32, 1, 64)` and tensor B has shape `(1, 128, 64)`. Comparing dimensions right to left, 64 matches 64, then 1 stretches to 128, then 1 stretches to 32, yielding result shape `(32, 128, 64)`. Visualizing this expansion prevents silent logic bugs that accidentally allocate a large tensor (for example, a `(Batch, Batch)` matrix instead of an element-wise `(Batch)` vector).

Shapes, strides, and broadcasting govern how tensors flow through a model's forward pass. Training adds a second requirement: learning from errors. Backpropagation makes that learning possible and imposes the memory costs developed below.

C.3 Mechanics of Learning

Valid tensor programs make training loops possible. Backpropagation orchestrates these tensors to compute gradients, transforming a forward prediction into a backward learning signal.

> **Systems Perspective C.2: Why this matters**
>
> When training fails—loss goes to NaN, gradients explode, or memory runs out—understanding what backpropagation actually does is essential for diagnosing the problem. The backward graph supplies the mental model for reasoning about gradient flow and memory usage during training.

C.3.1 The chain rule and automatic differentiation

For a composed function $y = f(g(x))$, the derivative is $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$. In a neural network, f and g are layers, and the composition can be many levels deep. For a three-layer network $y = f_3(f_2(f_1(x)))$, the chain rule extends to:

$$\frac{\partial y}{\partial x} = \frac{\partial f_3}{\partial f_2} \cdot \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial x}$$

Each factor in this product is a local derivative—computed at one layer using only that layer's inputs and outputs. This locality is what makes the algorithm tractable: the entire network never needs to be differentiated as a monolithic function. Instead, each layer computes its own local derivative during the backward pass and multiplies it by the gradient flowing in from the layer above.

Modern frameworks use reverse-mode automatic differentiation, which computes gradients for all P parameters in a single backward pass. The key insight is that starting from the output and working backward (reverse mode) requires one pass regardless of the number of parameters, whereas starting from each input and working forward (forward mode) would require P passes—one per parameter. This is why a training step is a small constant multiple of inference, commonly about 2–3× a forward pass for dense networks, rather than P passes.

C.3.2 The backpropagation algorithm

Backpropagation[3] implements the chain rule efficiently through two passes: forward to compute outputs, backward to compute gradients. The gradient edges retrace forward dependencies, meaning each backward step requires intermediate activations retained from the corresponding forward operation. Figure C.2 traces these dual dataflow passes across a two-layer network: solid gray arrows stage forward activations ($\mathbf{x} \to \mathbf{h} \to \hat{\mathbf{y}}$) through weight parameters $\mathbf{W}_1$ and $\mathbf{W}_2$ to evaluate loss $\mathcal{L}$, while curved dashed red arrows propagate intermediate adjoints ($\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}}, \frac{\partial \mathcal{L}}{\partial \mathbf{h}}, \frac{\partial \mathcal{L}}{\partial \mathbf{x}}$) in reverse order.

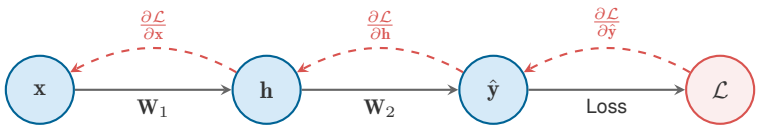


Figure C.2: Backpropagation Computational Graph: Solid gray arrows carry the forward computation from input $\mathbf{x}$ to loss $\mathcal{L}$, while dashed red arrows carry gradients backward along the same dependencies.

[3] **Backpropagation**: Short for "backward propagation of errors." The algorithm was independently discovered multiple times—by Werbos (1974) and Linnainmaa (1970) for reverse-mode differentiation and by Rumelhart et al. (1986) for neural network training. Its key insight is that computing gradients for all parameters requires one backward pass through the graph rather than one pass per parameter. Section C.4 derives the constant-factor work bound and a representative dense-layer cost.

Forward pass

Using row-vector activations, omitting biases, and retaining intermediate activations for backward execution, computation begins at input $\mathbf{x}$. Multiplying $\mathbf{x}$ by weight matrix $\mathbf{W}_1$ yields hidden activation $\mathbf{h}$, which the execution runtime caches for backward differentiation. Multiplying $\mathbf{h}$ by $\mathbf{W}_2$ produces network output $\hat{\mathbf{y}}$, which is similarly cached. Finally, comparing $\hat{\mathbf{y}}$ to the target label produces scalar loss $\mathcal{L}$.

At this point, forward evaluation is complete, and memory holds input $\mathbf{x}$, cached activation $\mathbf{h}$, cached prediction $\hat{\mathbf{y}}$, and loss $\mathcal{L}$. For large models, these retained activation tensors frequently dominate overall memory consumption.

Backward pass

Gradient propagation begins backward from scalar loss $\mathcal{L}$. The loss function evaluates $\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}}$, the error gradient with respect to network prediction, introducing the external learning signal.

Because $\hat{\mathbf{y}} = \mathbf{h} \cdot \mathbf{W}_2$, the matrix chain rule yields two gradients at this layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{h}^T \cdot \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \qquad \text{(weight gradient—used to update } \mathbf{W}_2\text{)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \cdot \mathbf{W}_2^T \qquad \text{(input gradient—passed backward to the previous layer)}$$

Computing $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2}$ requires cached activation $\mathbf{h}$ from the forward pass. Activations must remain resident in memory because every layer's weight gradient depends directly on that layer's input.

Continuing backward to the first layer, the linear relationship $\mathbf{h} = \mathbf{x} \cdot \mathbf{W}_1$ produces:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}}$$

Each backward step requires two operands: the gradient flowing in from downstream layers ($\frac{\partial \mathcal{L}}{\partial \mathbf{h}}$) and that layer's cached input ($\mathbf{x}$). Consequently, the backward pass requires roughly 2× the forward pass FLOPs: each layer executes two matrix multiplications (one for the parameter gradient, one for the upstream activation gradient) rather than one.

C.3.3 The true cost of training memory

A common mistake is to assume that training memory equals model size. This assumption leads to immediate OOM errors because weights are only one of four components. Training memory comprises weights, gradients, optimizer state, and retained activations:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$$

For a standard Adaptive Moment Estimation (Adam) optimizer (Kingma and Ba 2015) in mixed precision (Micikevicius et al. 2017):

- **Weights**: 2 bytes (FP16/BF16) or 4 bytes (FP32).
- **Gradients**: Same size as weights.
- **Optimizer State**: Adam moments require 8 bytes per parameter; an FP32 master copy adds another 4 bytes, giving 8–12 bytes per parameter.
- **Activations**: Activations typically dominate the dynamic memory footprint during training. A recomputation- or tiled-attention-friendly estimate scales as $\mathcal{O}(B \times S \times N_L \times d)$; a full no-recompute attention implementation also materializes attention-score tensors that scale with S^2.

To see how these components interact in practice, consider a concrete model.

> **Napkin Math C.1: Worked example: GPT-2 (1.5B) training memory**
>
> **The model**: GPT-2 XL has $P = 1.5 \times 10^9$ parameters, 48 layers, hidden dimension $d = 1600$.
>
> **Model state (fixed per step)**:
>
> - **Weights (BF16)**: $1.5 \times 10^9 \times 2$ bytes = 3 GB
> - **Gradients (BF16)**: $1.5 \times 10^9 \times 2$ bytes = 3 GB
> - **Optimizer (FP32 master + momentum + variance)**: $1.5 \times 10^9 \times 12$ bytes = 18 GB
> - **Total model state**: 24 GB—fits on an 80 GB-class A100/H100 (85.9 GB in decimal units), but leaves only 61.9 GB for activations.
>
> **Activations (scale with batch)**:
>
> Per-layer retained activations for this quick estimate are approximately $12 \times B \times S \times d$ BF16 elements, or $12 \times B \times S \times d \times 2$ bytes, where B is batch size and S is sequence length. The factor twelve accounts for the major intermediate tensors retained for backpropagation: input activations, QKV projections ($3d$), attention output, FFN intermediate ($4d$), and layer norm/dropout masks. This estimate assumes the implementation does not retain the full $B \times N_{\text{heads}} \times S^2$ attention-score tensor, as in recomputation- or tiled-attention-friendly accounting. With 48 layers, batch size 8, and sequence length 1024:
>
> $$48 \times 12 \times 8 \times 1024 \times 1600 \times 2 \text{ bytes} \approx 15.1 \text{ GB}$$
>
> **Systems insight**: Total memory reaches ~39.1 GB under this activation-accounting convention, which fits on one 85.9 GB accelerator. However, increase the batch to 64 and activations grow to ~120.8 GB, exceeding the remaining capacity. This is the threshold where gradient checkpointing or another memory-saving method becomes necessary; its savings and recomputation overhead depend on checkpoint placement and implementation.

C.3.3.1 Activation explosion

While weights are fixed ($\mathcal{O}(P)$), activations grow linearly with batch size and at least linearly with sequence length; implementations that materialize attention scores add an $\mathcal{O}(S^2)$ component. As

the worked example shows, activation memory can quickly exceed the room left after model state. Gradient checkpointing[4] reduces this pressure by storing only selected activations and recomputing the rest during backpropagation (T. Chen et al. 2016); techniques such as FlashAttention address related attention-memory bottlenecks by tiling attention to reduce memory round-trips (T. Dao et al. 2022). The same decomposition provides a quick feasibility check: sum the per-parameter model state, add the activation term implied by batch size and sequence length, and compare the total against the accelerator's capacity.

[4] **Gradient checkpointing**: Trades compute for memory. Instead of storing all activations, only a subset (checkpoints) is kept, and the missing ones are recomputed during the backward pass. This reduces memory usage from storing every layer activation to roughly the square root of the layer count at the cost of ~33 percent more compute (T. Chen et al. 2016).

C.3.4 Computational graphs and optimization

The dependency structure exposed by backpropagation also gives compilers something to optimize. ML compilers represent models as directed acyclic graphs (DAGs), and that representation enables hardware-independent transformations.

C.3.4.1 Static single assignment

Compilers transform graphs into static single-assignment (SSA) form, where each variable is assigned exactly once. This makes data dependencies explicit, enabling safe optimizations—most notably, operator fusion.

C.3.4.2 Operator fusion

Without fusion, each operation in a chain like `MatMul` $\rightarrow$ `Add (bias)` $\rightarrow$ `ReLU` produces an intermediate tensor that is written to HBM and then read back for the next operation. For elementwise operations like `Add` and `ReLU`, the compute is trivial (one FLOP per element) but the memory traffic is not (read the tensor, write it back). The arithmetic intensity of unfused elementwise operations is therefore close to zero—deeply memory bound.

Fusion combines consecutive operations into a single kernel that reads the input once, applies all operations in registers or shared memory, and writes the final result once. For a sequence of n_{ops} elementwise operations on a tensor of N elements of s_{elem} bytes each, fusion reduces memory traffic from $2n_{\text{ops}}Ns_{\text{elem}}$ bytes (each operation reads and writes) to $2Ns_{\text{elem}}$ bytes (one read, one write)—a $n_{\text{ops}}\times$ reduction.

FlashAttention is an I/O-aware tiled attention algorithm: it computes the score, softmax, and value-product stages in SRAM tiles without materializing the full attention matrix in HBM, reducing auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ for a fixed head dimension. HBM traffic also falls, with its reduction depending on head dimension and available SRAM capacity. The original work reports up to 3$\times$ wall-clock speedups on its evaluated workloads (T. Dao et al. 2022); realized gains depend on shape and hardware (see section C.5 for the step-by-step derivation). Together, the linear algebra foundations, tensor programming mechanics, and training memory model covered in this appendix form the algorithmic substrate on which all ML systems are built.

C.4 Baur–Strassen Reverse-Mode AD Complexity Theorem

Modern deep learning scales to hundreds of billions of parameters because evaluating the full gradient vector $\nabla f(\mathbf{x})$ of a scalar loss function $f : \mathbb{R}^N \rightarrow \mathbb{R}$ requires compute proportional to evaluating $f(\mathbf{x})$ itself, with a constant-factor bound independent of the input dimension N. Baur and Strassen proved this complexity result in 1983 (Baur and Strassen 1983).

C.4.1 Mathematical formulation and theorem statement

Let $f : \mathbb{R}^N \rightarrow \mathbb{R}$ be a multivariate scalar-valued function represented as a computational graph $G = (V, E)$ consisting of $W(f)$ elementary operations (additions, subtractions, multiplications, divisions, and unary functions such as $\exp, \ln, \sin, \sqrt{\cdot}$).

Theorem C.1: Baur–Strassen complexity theorem

If a scalar function $f : \mathbb{R}^N \to \mathbb{R}$ can be computed in $W(f)$ elementary operations, then the full gradient vector $\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_N}\right)^T \in \mathbb{R}^N$ can be evaluated in total work $\text{Work}_{\text{Reverse}}(f, \nabla f)$ satisfying:

$$\text{Work}_{\text{Reverse}}(f, \nabla f) \leq C \cdot W(f)$$

where $C \leq 5$ is a universal constant independent of the input dimension N. Furthermore, the backward pass alone requires work $\text{Work}_{\text{Backward}}(\nabla f) \leq 4 \cdot W(f)$.

C.4.2 Step-by-step chain rule adjoint graph propagation proof

Let the computational graph $G = (V, E)$ be topologically ordered with nodes $v_{1-N}, \ldots, v_0, v_1, v_2, \ldots, v_W$, where input variables are $x_m = v_{i-N}$ for $m \in \{1, \ldots, N\}$ and the final output is $f(\mathbf{x}) = v_W$. Each intermediate node v_j is evaluated via a primitive operation:

$$v_j = \phi_j(\{v_k \mid k \in \text{Parents}(j)\})$$

Define the *adjoint variable* $\bar{v}_i$ for every node v_i as the partial derivative of the scalar output v_W with respect to v_i:

$$\bar{v}_i \triangleq \frac{\partial f}{\partial v_i} = \frac{\partial v_W}{\partial v_i}$$

By the multivariate chain rule, the adjoint $\bar{v}_i$ is the sum of partial derivatives propagated from all successor nodes (children) that consume v_i:

$$\bar{v}_i = \sum_{j \in \text{Children}(i)} \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i}$$

Reverse-mode automatic differentiation initializes $\bar{v}_W = 1$ and $\bar{v}_i = 0$ for all $i < W$, then traverses the nodes in reverse topological order from v_W down to v_1. For each primitive node v_j, its fully accumulated adjoint $\bar{v}_j$ propagates error signals to its parent nodes v_k ($k \in \text{Parents}(j)$) via:

$$\bar{v}_k \leftarrow \bar{v}_k + \bar{v}_j \cdot \frac{\partial v_j}{\partial v_k}$$

We now perform operation-by-operation micro-cost accounting for all primitive nodes to bound the backward work:

1. **Addition/Subtraction ($v_j = v_i \pm v_k$)**:
   - Local derivatives: $\frac{\partial v_j}{\partial v_i} = 1$, $\frac{\partial v_j}{\partial v_k} = \pm 1$.
   - Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j$ and $\bar{v}_k \leftarrow \bar{v}_k \pm \bar{v}_j$.
   - Cost: 2 additions in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(2) = 3$ OPs $\leq 5 \cdot 1$.
2. **Multiplication ($v_j = v_i \cdot v_k$)**:
   - Local derivatives: $\frac{\partial v_j}{\partial v_i} = v_k$, $\frac{\partial v_j}{\partial v_k} = v_i$.
   - Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \cdot v_k$ and $\bar{v}_k \leftarrow \bar{v}_k + \bar{v}_j \cdot v_i$.
   - Cost: 2 multiplications + 2 additions = 4 operations in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(4) = 5$ OPs $\leq 5 \cdot 1$.
3. **Division ($v_j = v_i / v_k$)**:
   - Local derivatives: $\frac{\partial v_j}{\partial v_i} = \frac{1}{v_k}$, $\frac{\partial v_j}{\partial v_k} = -\frac{v_i}{v_k^2} = -\frac{v_j}{v_k}$.
   - Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \frac{\bar{v}_j}{v_k}$ and $\bar{v}_k \leftarrow \bar{v}_k - \frac{\bar{v}_j \cdot v_j}{v_k}$.

- Cost: Compute the quotient of the incoming adjoint and the denominator once, then reuse it in both updates: add it to the numerator adjoint, and multiply it by the cached forward output before subtracting from the denominator adjoint. This requires 2 multiplications/divisions + 2 additions = 4 operations in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(4) = 5$ OPs $\leq 5 \cdot 1$.

4. **Unary transcendental operations ($v_j = \phi(v_i)$)**:
   - Local derivative: $\frac{\partial v_j}{\partial v_i} = \phi'(v_i)$.
   - Backward update: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \cdot \phi'(v_i)$.
   - For $v_j = \exp(v_i)$, $\phi'(v_i) = v_j$, requiring 1 multiplication + 1 addition = 2 OPs.
   - For $v_j = \ln(v_i)$, $\phi'(v_i) = 1/v_i$, requiring 1 division + 1 addition = 2 OPs.

Summing across all primitive nodes in G, every forward operation generates at most 4 backward operations:

$$\text{Work}_{\text{Backward}}(\nabla f) \leq 4 \cdot W(f)$$

Adding the forward pass work $W(f)$ yields the total work bound:

$$\text{Work}_{\text{Reverse}}(f, \nabla f) = W(f) + \text{Work}_{\text{Backward}}(\nabla f) \leq 5 \cdot W(f) \qquad \blacksquare$$

Systems Perspective C.3: Why 175B-parameter large language models can be trained

The independence of $C \leq 5$ from input dimension N is the mathematical reason deep learning works. Consider training a transformer model with $P = 175 \times 10^9$ parameters:

- **Forward pass**: Matrix multiplications $\mathbf{Y} = \mathbf{XW}$ require $2P$ FLOPs per token.
- **Reverse-mode backward pass**: Computes both weight gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$ ($2P$ FLOPs) and input activation gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \mathbf{W}^T$ ($2P$ FLOPs), totaling $4P$ FLOPs per token.
- **Total training work**: $2P(\text{fwd}) + 4P(\text{bwd}) = 6P$ FLOPs per token. The backward compute factor is exactly $\frac{4P}{2P} = 2$, well within the Baur–Strassen theoretical bound of $C \leq 5$.

If we instead used forward-mode AD or finite differences to compute $\nabla f \in \mathbb{R}^P$, evaluating each partial derivative separately would require P forward passes: $\text{Work}_{\text{ForwardMode}}(\nabla f) = \mathcal{O}(P \cdot W) = \mathcal{O}(P^2)$ FLOPs. For $P = 175 \times 10^9$, that method would require $2 \times (175 \times 10^9)^2 \approx 6.1 \times 10^{22}$ FLOPs per token—P times the forward-pass work and therefore prohibitive at this scale. Reverse-mode AD collapses this $\mathcal{O}(P)$ factor into a constant multiplier of 2 for the dense-layer accounting above.

C.5 FlashAttention Tiling & Online Softmax Recurrence Proof

Standard self-attention in transformers requires materializing an $S \times S$ attention score matrix in HBM, creating an $\mathcal{O}(S^2)$ memory bandwidth bottleneck. FlashAttention (T. Dao et al. 2022) eliminates this bottleneck by tiling query, key, and value matrices and recomputing attention using an online softmax recurrence without materializing intermediate scores to HBM.

C.5.1 Standard softmax HBM I/O bottleneck

Given queries $\mathbf{Q} \in \mathbb{R}^{S \times d_{\text{head}}}$, keys $\mathbf{K} \in \mathbb{R}^{S \times d_{\text{head}}}$, and values $\mathbf{V} \in \mathbb{R}^{S \times d_{\text{head}}}$ for sequence length S and head dimension d_{head}, standard attention evaluates:

$$\mathbf{S} = \mathbf{QK}^T \in \mathbb{R}^{S \times S}, \qquad \mathbf{A} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{S \times S}, \qquad \mathbf{O} = \mathbf{AV} \in \mathbb{R}^{S \times d_{\text{head}}}$$

To prevent numerical overflow, row-wise softmax subtracts the row maximum $m_i = \max_{1 \leq j \leq S} S_{ij}$:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_{k=1}^{S} \exp(S_{ik} - m_i)}$$

Standard GPU implementations execute this in three discrete kernel launches:

1. Compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^T$ in SRAM, then write $\mathbf{S} \in \mathbb{R}^{S\times S}$ to HBM ($\Theta(S^2)$ writes).
2. Read $\mathbf{S}$ from HBM, compute row max m and row sum z, compute $\mathbf{A} = \text{softmax}(\mathbf{S})$, write $\mathbf{A} \in \mathbb{R}^{S\times S}$ to HBM ($\Theta(S^2)$ reads + writes).
3. Read $\mathbf{A}$ and $\mathbf{V}$ from HBM, compute $\mathbf{O} = \mathbf{A}\mathbf{V}$, write $\mathbf{O} \in \mathbb{R}^{S\times d_{\text{head}}}$ to HBM ($\Theta(S^2 + Sd_{\text{head}})$ reads + writes).

Total HBM memory traffic is $\Theta(S^2 + Sd_{\text{head}})$ elements. For $S = 8192$, $d_{\text{head}} = 128$ in FP16, $\mathbf{S}$ requires $8192^2 \times 2$ B $= 134$ MB per head per layer. Because modern GPUs feature massive compute throughput relative to HBM bandwidth, repeatedly reading and writing intermediate matrices $\mathbf{S}$ and $\mathbf{A}$ leaves GPU execution units memory-bound.

C.5.2 Online softmax recurrence proof

Tiling softmax is difficult because computing $z_i = \sum_{k=1}^{S} \exp(S_{ik} - m_i)$ requires the global maximum m_i across all S keys. The online softmax recurrence solves this by incrementally updating normalization statistics as sub-blocks arrive.

Theorem C.2: Online softmax recurrence theorem

Let a vector $\mathbf{x} \in \mathbb{R}^n$ be partitioned into two sub-vectors $\mathbf{x}^{(1)} \in \mathbb{R}^{n_1}$ and $\mathbf{x}^{(2)} \in \mathbb{R}^{n_2}$ with $n = n_1 + n_2$. Let the local statistics for block 1 be:

$$m^{(1)} = \max_{1\le j\le n_1} x_j^{(1)}, \qquad z^{(1)} = \sum_{j=1}^{n_1} \exp\left(x_j^{(1)} - m^{(1)}\right)$$

and for block 2 be:

$$m^{(2)} = \max_{1\le j\le n_2} x_j^{(2)}, \qquad z^{(2)} = \sum_{j=1}^{n_2} \exp\left(x_j^{(2)} - m^{(2)}\right)$$

Then the combined maximum m^{new}, normalization sum z^{new}, and accumulated output vector $\mathbf{O}^{\text{new}}$ satisfy the exact recurrence relations:

$$\begin{aligned} m^{\text{new}} &= \max\left(m^{(1)}, m^{(2)}\right), \\ z^{\text{new}} &= z^{(1)} e^{m^{(1)} - m^{\text{new}}} + z^{(2)} e^{m^{(2)} - m^{\text{new}}}, \\ \mathbf{O}^{\text{new}} &= \frac{z^{(1)} e^{m^{(1)} - m^{\text{new}}}}{z^{\text{new}}} \mathbf{O}^{(1)} + \frac{e^{m^{(2)} - m^{\text{new}}}}{z^{\text{new}}} \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)}, \end{aligned}$$

where $\mathbf{O}^{(1)} = \frac{1}{z^{(1)}} \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)}$ is the partial output accumulated from block 1.

Derivation of denominator recurrence

Expanding the combined sum $z^{\text{new}} = \sum_{j=1}^{n} e^{x_j - m^{\text{new}}}$:

$$\begin{aligned} z^{\text{new}} &= \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{\text{new}}} + \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{\text{new}}} \\ &= \sum_{j=1}^{n_1} e^{\left(x_j^{(1)} - m^{(1)}\right) + \left(m^{(1)} - m^{\text{new}}\right)} + \sum_{j=1}^{n_2} e^{\left(x_j^{(2)} - m^{(2)}\right) + \left(m^{(2)} - m^{\text{new}}\right)} \\ &= e^{m^{(1)} - m^{\text{new}}} \underbrace{\sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}}}_{z^{(1)}} + e^{m^{(2)} - m^{\text{new}}} \underbrace{\sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}}}_{z^{(2)}} \\ &= z^{(1)} e^{m^{(1)} - m^{\text{new}}} + z^{(2)} e^{m^{(2)} - m^{\text{new}}}. \quad \blacksquare \end{aligned}$$

Derivation of output rescaling recurrence

The true normalized output over the concatenated sequence is $\mathbf{O}^{\text{new}} = \frac{1}{z^{\text{new}}} \sum_{j=1}^{n} e^{x_j - m^{\text{new}}} \mathbf{V}_j$:

$$\begin{aligned}\mathbf{O}^{\text{new}} &= \frac{1}{z^{\text{new}}} \left[\sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{\text{new}}} \mathbf{V}_j^{(1)} + \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{\text{new}}} \mathbf{V}_j^{(2)} \right] \\ &= \frac{1}{z^{\text{new}}} \left[e^{m^{(1)} - m^{\text{new}}} \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)} + e^{m^{(2)} - m^{\text{new}}} \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)} \right]\end{aligned}$$

Noting that $z^{(1)}\mathbf{O}^{(1)} = \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)}$, we substitute this directly:

$$\mathbf{O}^{\text{new}} = \frac{z^{(1)} e^{m^{(1)} - m^{\text{new}}}}{z^{\text{new}}} \mathbf{O}^{(1)} + \frac{e^{m^{(2)} - m^{\text{new}}}}{z^{\text{new}}} \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)}. \quad \blacksquare$$

C.5.3 Formal HBM access complexity proof

Let M_{SRAM} be the fast SRAM capacity (in words). Following Algorithm 1 of T. Dao et al. (2022), FlashAttention sets $B_c = \lfloor M_{\text{SRAM}}/(4d_{\text{head}}) \rfloor$ and $B_r = \min(\lfloor M_{\text{SRAM}}/(4d_{\text{head}}) \rfloor, d_{\text{head}})$. The proof abstracts constant factors and buffer reuse, using $B_c = \Theta(M_{\text{SRAM}}/d_{\text{head}})$, $B_r = \Theta(\min(M_{\text{SRAM}}/d_{\text{head}}, d_{\text{head}}))$, and $B_r B_c = \mathcal{O}(M_{\text{SRAM}})$ for the on-chip working set.

The algorithm partitions $\mathbf{Q}$ into $T_r = \lceil S/B_r \rceil$ blocks of size $B_r \times d_{\text{head}}$, and partitions $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil S/B_c \rceil$ blocks of size $B_c \times d_{\text{head}}$. It executes two nested loops:

- **Outer loop** over $j = 1, \ldots, T_c$: Load $\mathbf{K}_j$ and $\mathbf{V}_j$ into SRAM.
- **Inner loop** over $i = 1, \ldots, T_r$: Load $\mathbf{Q}_i$, $\mathbf{O}_i$, and its softmax statistics; compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T$ in SRAM; then update and write $\mathbf{O}_i$ and the statistics without materializing $\mathbf{S}_{ij}$ in HBM.

This loop order is the source of the I/O reduction. A key-value tile remains resident while the algorithm visits every query tile, so each element of $\mathbf{K}$ and $\mathbf{V}$ crosses the HBM boundary once. The query and output tiles must be revisited for each key-value block, but the $B_r \times B_c$ score tile is consumed on chip and discarded. Standard attention instead writes and rereads the full $S \times S$ score and probability matrices.

Total memory access accounting

Total HBM accesses comprise two components: key and value reads, where each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded once (contributing $2Sd_{\text{head}}$ accesses); and query, output, and statistics passes, where each of the T_c column blocks reads all of $\mathbf{Q}$, reads and writes $\mathbf{O}$, and updates $\mathcal{O}(S)$ softmax statistics. The matrices dominate the statistics for head dimension $d_{\text{head}} \geq 1$, so each pass contributes $\Theta(Sd_{\text{head}})$ accesses and all passes contribute $\Theta(Sd_{\text{head}}T_c)$.

Because $T_c = \lceil S/B_c \rceil = \Theta(Sd_{\text{head}}/M_{\text{SRAM}})$, the total is

$$\text{HBM}_{\text{Flash}} = \Theta\left(Sd_{\text{head}} + Sd_{\text{head}}T_c\right) = \Theta\left(Sd_{\text{head}} + \frac{S^2 d_{\text{head}}^2}{M_{\text{SRAM}}}\right) = \Theta\left(\frac{S^2 d_{\text{head}}^2}{M_{\text{SRAM}}}\right),$$

where the final equality follows for $d_{\text{head}} \leq M_{\text{SRAM}} \leq Sd_{\text{head}}$.

The bound counts communication rather than arithmetic. FlashAttention still evaluates the same dense score blocks and may recompute intermediate quantities during backpropagation; its advantage comes from keeping those intermediates out of HBM.

I/O reduction ratio

Comparing standard attention I/O to FlashAttention I/O:

$$\frac{\text{HBM}_{\text{Standard}}}{\text{HBM}_{\text{Flash}}} = \frac{\Theta(S^2 + Sd_{\text{head}})}{\Theta\left(Sd_{\text{head}} + \frac{S^2 d_{\text{head}}^2}{M_{\text{SRAM}}}\right)} = \Theta\left(\frac{M_{\text{SRAM}}}{d_{\text{head}}^2}\right) \quad \text{for } S \gg \max\left(d_{\text{head}}, \frac{M_{\text{SRAM}}}{d_{\text{head}}}\right)$$

Systems Perspective C.4: A100 I/O reduction

The asymptotic ratio $M_{\text{SRAM}}/d_{\text{head}}^2$ predicts how greater SRAM capacity can reduce HBM traffic, but hidden constants and kernel details prevent it from serving as an exact transfer count. In an NVIDIA A100 measurement, T. Dao et al. (2022) report the effect for a forward-and-backward attention workload with sequence length 1,024, head dimension 64, 16 heads, batch size 64, a key-padding mask, and no dropout:

- **Standard attention**: 35.3 GB of HBM reads and writes; 35.1 ms runtime.
- **FlashAttention**: 4.4 GB of HBM reads and writes; 11.7 ms runtime.

The measurements show the systems consequence of the I/O bound: substantially less HBM traffic coincides with lower runtime even though FlashAttention performs additional recomputation. They do not imply that every configuration becomes compute bound; tile size, sequence length, and kernel implementation determine the realized regime.

C.6 Kaplan & Chinchilla Compute-Optimal Scaling Law Derivation

Determining the compute-optimal allocation between model parameters and training dataset tokens for a given floating-point budget O is a central decision in large language model system design. This section provides the rigorous mathematical derivation of compute-optimal scaling laws using Lagrange multipliers, contrasting the original findings of Kaplan et al. (2020) with the compute-optimal corrections of Hoffmann et al. (2022).

Notation note: This derivation writes model parameters as P, dataset size in tokens as D, and the training compute budget as O total FLOPs; Kaplan et al. (2020) and Hoffmann et al. (2022) write N for parameters and C for compute. The fitted constants E, A, B, α, and β and the Lagrange multiplier λ keep the papers' letters and are local to this section, so E is a loss rather than an energy and B is a fitted constant rather than batch size.

C.6.1 Parametric loss surface formulation

The cross-entropy evaluation loss $\mathcal{L}(P, D)$ of an autoregressive transformer language model trained with P non-embedding parameters on D tokens is modeled by the power-law formulation:

$$\mathcal{L}(P, D) = E + \frac{A}{P^\alpha} + \frac{B}{D^\beta}$$

where:

- E: Irreducible loss representing the entropy of natural language data.
- $\frac{A}{P^\alpha}$: Capacity bottleneck loss resulting from finite model parameters P.
- $\frac{B}{D^\beta}$: Dataset size bottleneck loss resulting from finite training tokens D.
- $A, B > 0$: Empirical scaling constants.
- $\alpha, \beta > 0$: Power-law scaling exponents for model size and dataset size, respectively.

C.6.2 Constrained optimization via Lagrange multipliers

The total floating-point operations required to train a transformer with P non-embedding parameters for D tokens is given by

$$O = 6PD \text{ FLOPs},$$

where $2PD$ accounts for the forward pass and $4PD$ accounts for the backward pass.

Given a fixed training compute budget O_0, we formulate the constrained optimization problem to minimize the reducible loss $\hat{\mathcal{L}}(P, D) = \frac{A}{P^\alpha} + \frac{B}{D^\beta}$:

$$\min_{P,D} \left(\frac{A}{P^\alpha} + \frac{B}{D^\beta} \right) \quad \text{subject to} \quad g(P, D) = 6PD - O_0 = 0$$

We construct the Lagrangian function $\mathcal{L}_{\text{Lagrange}}(P, D, \lambda)$:

$$\mathcal{L}_{\text{Lagrange}}(P, D, \lambda) = \frac{A}{P^\alpha} + \frac{B}{D^\beta} + \lambda(6PD - O_0)$$

Taking partial derivatives with respect to P, D, and λ and setting them to zero yields:

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial P} = -\alpha A P^{-\alpha-1} + 6\lambda D = 0 \implies \alpha A P^{-\alpha-1} = 6\lambda D, \tag{1}$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial D} = -\beta B D^{-\beta-1} + 6\lambda P = 0 \implies \beta B D^{-\beta-1} = 6\lambda P, \tag{2}$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial \lambda} = 6PD - O_0 = 0. \tag{3}$$

Multiplying Equation (1) by P and Equation (2) by D gives:

$$\alpha A P^{-\alpha} = 6\lambda PD,$$

$$\beta B D^{-\beta} = 6\lambda PD.$$

Equating these expressions yields the fundamental power-law optimality condition:

$$\alpha A P^{-\alpha} = \beta B D^{-\beta}.$$

This condition proves that at the compute-optimal allocation, the marginal loss reduction per fractional increase in parameter count must equal the marginal loss reduction per fractional increase in dataset tokens.

C.6.3 Analytical derivation of compute-optimal exponents

From $\alpha A P^{-\alpha} = \beta B D^{-\beta}$, solving for D in terms of P yields:

$$D^\beta = \left(\frac{\beta B}{\alpha A}\right) P^\alpha \implies D = \left(\frac{\beta B}{\alpha A}\right)^{1/\beta} P^{\alpha/\beta}.$$

Substituting D into the compute constraint $O = 6PD$ gives:

$$O = 6P\left(\frac{\beta B}{\alpha A}\right)^{1/\beta} P^{\alpha/\beta} = 6\left(\frac{\beta B}{\alpha A}\right)^{1/\beta} P^{\frac{\alpha+\beta}{\beta}}.$$

Solving explicitly for the compute-optimal parameter count $P^*(O)$:

$$P^*(O) = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{O}{6}\right)^{\frac{\beta}{\alpha+\beta}} \propto O^{\frac{\beta}{\alpha+\beta}}.$$

By symmetry, solving for the compute-optimal token count $D^*(O)$ yields:

$$D^*(O) = \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{O}{6}\right)^{\frac{\alpha}{\alpha+\beta}} \propto O^{\frac{\alpha}{\alpha+\beta}}.$$

Empirical exponents and the $D/P \approx 20$ rule

Fitting the parametric loss surface to over 400 experimental runs, Hoffmann et al. (2022) estimated $\alpha \approx 0.34$ and $\beta \approx 0.28$. Substituting these values gives exponents close to the paper's parametric-model result:

$$\frac{\beta}{\alpha+\beta} \approx 0.45, \qquad \frac{\alpha}{\alpha+\beta} \approx 0.55$$

Thus the parametric model predicts:

$$P^*(O) \propto O^{0.45}, \qquad D^*(O) \propto O^{0.55}$$

The paper's two direct empirical approaches report near-balanced exponents of $(0.50, 0.50)$ and $(0.49, 0.51)$, which motivate the common square-root shorthand. Under the parametric fit, however, D^*/P^* grows slowly with compute rather than remaining strictly constant. Across the regime studied, roughly 20 training tokens per parameter is a useful Chinchilla heuristic, not a universal invariant.

Systems Perspective C.5: Kaplan vs. Chinchilla scaling

The difference between Kaplan et al. (2020) and Hoffmann et al. (2022) highlights how hyperparameter choices in empirical benchmarking can alter system design conclusions:

- **Kaplan scaling model**: Kaplan et al. (2020) fit a different loss form with its own exponents and reported compute-optimal growth of $P \propto O^{0.73}$ and $D \propto O^{0.27}$; this split does not follow from substituting exponents into the two-term surface above. This led the field to scale model sizes aggressively while holding training dataset sizes relatively small (e.g., GPT-3 175B trained on 300B tokens, giving $D/P = 1.71$).
- **Methodological divergence**: Kaplan et al. (2020) used a fixed token budget and learning-rate schedule across models. Hoffmann et al. (2022) matched the schedule to each training horizon and found that the earlier setup understated the value of training smaller models on more data.
- **Chinchilla revision**: Hoffmann et al. (2022) found near-balanced parameter and token scaling: its three approaches report exponent pairs of $(0.50, 0.50)$, $(0.49, 0.51)$, and $(0.46, 0.54)$ rather than one exact universal ratio.
- **Practical impact**: Chinchilla (70B parameters trained on 1.4T tokens, $D/P = 20$) outperformed GPT-3 (175B parameters trained on 300B tokens) across downstream benchmarks while requiring $2.5\times$ less compute during inference and $2.5\times$ lower weight memory at the same precision (Hoffmann et al. 2022).

Summary

An algorithm becomes a systems workload through its shapes, operation graph, data dependencies, and numerical representation. Linear-algebra primitives, memory layouts, broadcasting, and sparse formats determine work, traffic, locality, and usable parallelism. Automatic differentiation and backpropagation add saved activations and backward computation; computational graphs then expose opportunities for fusion and recomputation. The Baur–Strassen result bounds reverse-mode overhead, while FlashAttention shows how tiling and an online recurrence can reduce HBM traffic without changing attention semantics.

The same accounting connects kernels to scaling decisions. Compute-optimal scaling turns a fixed budget into a model-data allocation, but the fitted exponents remain empirical rather than universal. Across these models, the reusable method is to expose operations, bytes, saved state, dependencies, and asymptotic constraints before selecting an optimization.

D

Machine Foundations

A datasheet reports peak throughput; a workload almost never reaches it. Closing that gap requires knowing which physical limit binds execution first: latency, memory bandwidth, arithmetic intensity, or energy. This appendix collects the durable hardware models and order-of-magnitude values that anchor single-node reasoning throughout the book, providing the mathematical baselines where chapter estimates can be audited or extended. Architecture essentials support readers encountering memory hierarchies and hardware cost models for the first time; familiarity with undergraduate computer architecture is helpful but not required.

How to Use This Appendix

This appendix is designed as a reference. When diagnosing performance issues, use this appendix to translate a vague symptom ("it's slow") into a specific constraint ("memory bound at batch size one") and then choose the lever that can actually move.

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW for bandwidth).

- **Establish the hardware vocabulary**: Start with the storage tiers in section D.3.3, then distinguish transfer delay from transfer rate in section D.3.4. The units check in section D.2.2 prepares those quantities for the roofline calculation.
- **Sanity-check feasibility**: Start with section D.1 for order-of-magnitude numbers.
- **Diagnose the dominant ceiling**: Use the roofline model in section D.2.1 to decide whether the workload is compute bound or memory bound.
- **Reason about scaling limits**: Use Amdahl's and Gustafson's laws in section D.2.3 to understand why adding accelerators may not reduce time-to-train.
- **Choose the right precision**: Use section D.4.1 to reason about FP32 vs. BF16/FP16 vs. INT8 as a systems trade-off.
- **Follow the chapter treatment**: For the full narrative, use chapter 11, chapter 8, and chapter 13.

Although modern hardware encompasses diverse accelerator architectures, interconnect topologies, and packaging options, the physical laws and bottleneck models detailed here govern them all.

D.1 Numbers to Know

Just as Jeff Dean's "Latency Numbers Every Programmer Should Know"[1] shaped a generation of systems engineers, these reference numbers provide the order-of-magnitude intuition essential for ML systems design. Although absolute values and ratios vary by technology and workload, the hierarchy is more durable. Memorize the relationships; use the specific numbers as sanity checks.

> Systems Perspective D.1: Three numbers that matter most
>
> - **Energy ratio**: In the cited 45 nm reference, a 32-bit DRAM read uses ~581× the energy of one FP16 multiply. This motivates arithmetic intensity.

[1] **Jeff Dean**: A Google Senior Fellow and one of the architects of Google's distributed systems infrastructure, including MapReduce, BigTable, and TensorFlow. His latency numbers, originally presented with Peter Norvig around 2010, became a canonical reference for systems engineers. The numbers have been updated over the years as hardware evolved, but the *hierarchy* of latencies remains remarkably stable; Colin Scott's interactive visualization shows the latency hierarchy across hardware generations (Scott 2012).

- **Training-state footprint**: Model weights (2 bytes FP16) + gradients (2 bytes FP16) + master weights (4 bytes FP32) + optimizer states for Adaptive Moment Estimation (Adam) at 8 bytes. That totals 16 bytes per parameter, so a 7B model needs 112 GB just to start training.
- **Fiber propagation limit**: Light travels about 200 km/ms in fiber. The table's ~40 ms cross-country round trip includes reducible overhead, but propagation cannot be optimized away.

Reference relationships

These relationships mix physical or arithmetic bounds with technology-specific measurements; only the bounds are invariant.

Speed of light tax

Each approximate round trip in table D.1 carries an irreducible propagation component.

Table D.1: Speed of Light Reference: Light in fiber travels ~200 km/ms. These approximate round trips include reducible overhead; only propagation is irreducible.

Distance	Round-Trip Latency	Implication
Same data center	~1 ms	Distributed training feasible
Cross-country (US)	~40 ms	Reduces the remaining request budget
Cross-Atlantic	~60 ms	Further reduces the request budget
Cross-Pacific	~100 ms	Data locality is critical

Energy hierarchy

Table D.2 quantifies the energy cost of data movement vs. computation—the fundamental reason why arithmetic intensity dominates ML performance optimization.[2]

[2] **Energy hierarchy sources**: The DRAM and arithmetic values come from Horowitz (2014) (45 nm). The L1 SRAM/register ratio is an illustrative hierarchy anchor, not a Horowitz measurement. The headline ratio compares a 32-bit DRAM access with a 16-bit floating-point multiply. Both the absolute values and the ratio vary with process, circuit, memory technology, and operand width; wire distance helps explain why movement remains expensive.

Table D.2: Energy Reference Ratios: The first three ratios use Horowitz's 45 nm measurements; the L1/register ratio is an illustrative on-chip hierarchy anchor.

Relationship	Reference Ratio	Source and Interpretation
DRAM access vs. FP16 multiply	~581×	Horowitz 45 nm; wire capacitance scales with distance
FP32 vs. INT8 energy	~18×	Horowitz 45 nm; bit width determines switching energy
FP32 vs. FP16 energy	~3.4×	Horowitz 45 nm; narrower arithmetic reduces switching and datapath energy
L1 SRAM vs. register	~5×	Illustrative on-chip hierarchy anchor; distance to the arithmetic unit

Memory hierarchy

Table D.3 shows why the memory hierarchy is uneven: nearby on-package hops can differ by only a few times, while off-chip, storage, and network tiers introduce orders-of-magnitude jumps.

Table D.3: The Latency Hierarchy: Selected latency relationships in the memory hierarchy. The largest gaps come from crossing physical boundaries: off-chip memory, peripheral links, storage, and network fabric.

Relationship	Ratio	Why It Persists
Accelerator high-bandwidth memory vs. register	~1000× slower	On-chip vs. off-chip
SSD vs. register	~300,000× slower	Electrical vs. mechanical/flash
Network vs. local memory	~16× slower	Speed of light + switching
Accelerator memory BW vs. CPU↔Accelerator link	~52× faster	Architectural investment priority

Scaling laws

Table D.4 collects the arithmetic relationships that govern memory and compute requirements for training and inference.[3]

[3] **Training memory (Adam)**: The 16 bytes/parameter rule assumes mixed-precision training with Adam. ZeRO optimization can reduce per-accelerator memory by sharding optimizer states across accelerators, but the total memory across all accelerators remains ~16× parameters.

Table D.4: Scaling Rules: These are arithmetic, not hardware-specific. Training memory includes FP16 weights (2 bytes), FP16 gradients (2 bytes), FP32 master weights (4 bytes), and Adam optimizer states (8 bytes for momentum + variance).

Rule	Formula	Example
Inference memory (FP16)	2 bytes× parameters	7B params → 14 GB
Inference memory (INT8)	1 byte× parameters	7B params → 7 GB
Training memory (Adam)	16 bytes× parameters	7B params → 112 GB
Inference FLOPs (transformer)	~2× parameters per token	7B model → ~14 GFLOPs/token
Training FLOPs	~6× parameters× tokens	7B on 1T tokens → 4×10^{22} FLOPs

Illustrative latency budgets

Latency budgets depend on application-specific safety requirements, user expectations, and service-level objectives. Table D.5 lists illustrative targets rather than universal limits.

Table D.5: Illustrative Latency Targets: Production requirements depend on the application and its service-level objectives.

Application	Budget	Constraint
Autonomous braking	<10 ms	At 100 km/h, 10 ms = 28 cm of travel
Voice assistant	<100 ms	Human perception of "instant"
Web search	<200 ms	User patience threshold
Video streaming	<1 s	Buffer tolerance
Batch training	hours–days	Throughput dominates latency

Current hardware reference (c. 2024)

These numbers reflect the current generation. Use them for back-of-envelope calculations and recheck them against current hardware.

Memory latency and bandwidth

Table D.6 provides representative latency and bandwidth anchors across the memory hierarchy.

Table D.6: Memory Hierarchy (c. 2024): Representative latency and bandwidth anchors across registers, caches, memory, interconnects, networks, and storage. Together they show why locality and data movement shape performance.

Level	Latency	Bandwidth
Register	~0.3 ns	—
L1 Cache	~1 ns	—
L2 Cache	~4 ns	—
GPU HBM3	~300 ns	3.4 TB/s
PCIe Gen5 (CPU↔GPU)	~1000 ns	64 GB/s
CPU DRAM	~100 ns	50 GB/s
InfiniBand (network)	~5000 ns	50 GB/s
NVMe SSD	~100000 ns	7 GB/s

Compute throughput

Table D.7 compares representative peak throughput and power for data-center GPUs and a mobile NPU.

Table D.7: Compute Reference (c. 2024): Representative peak throughput and power for data-center GPUs and a mobile NPU. Because the rows use different precision modes and operation conventions, compare them only within a matched workload and precision context.

Platform	FP16/BF16	INT8/FP8-class	Power
Data center GPU (H100)	989 TFLOP/s	1979 TFLOP/s (FP8 peak)	700 W
Data center GPU (A100)	312 TFLOP/s	624 TOPS	400 W
Mobile NPU	—	35 TOPS	3–5 W

Roofline ridge points

Table D.8 defines the arithmetic intensity thresholds that determine whether a workload is memory bound or compute bound.

Table D.8: Arithmetic Intensity Thresholds (c. 2024): Batch-1 weight-streaming inference is often memory bound; larger batches may cross the ridge.

Accelerator	Ridge Point	Implication
A100 (FP16)	153 FLOP/byte	Below → memory-bound
H100 (FP16)	295 FLOP/byte	Higher bar for compute-bound



Systems Perspective D.2: A note on terminology: GPUs and accelerators

Throughout this book, "accelerator" is the general term for hardware acceleration. The principles—roofline analysis, memory hierarchies, numerical precision, and performance modeling—apply to GPUs, Tensor Processing Units (TPUs), NPUs, application-specific integrated circuits, and other specialized AI accelerators. Vendor-specific features such as CUDA and NVLink are identified explicitly.

Knowing the numbers is only the first step. The real power comes from compact models that reveal *which* number matters for the bottleneck at hand. The Roofline Model begins that diagnosis by translating raw hardware specs into actionable performance ceilings.

D.2 Physics of Computing

Raw hardware specs—TFLOP/s, TB/s, watt budgets—are necessary but insufficient for performance reasoning. Without compact analytical models, an engineer cannot distinguish a compute-bound workload from a memory-bound one, or predict whether doubling GPUs will halve training time. The models in this section turn those distinctions into quantitative checks.

D.2.1 The Roofline model

The Roofline Model (Williams et al. 2009) bounds how fast a workload can run on a given hardware target. The answer depends on whether the workload runs out of compute or memory bandwidth first.

Every operation has an **arithmetic intensity**: the ratio of computations performed to bytes moved from memory. Matrix multiplication achieves high arithmetic intensity because each loaded element is reused across multiple multiply-accumulate steps. In contrast, element-wise operations like rectified linear unit (ReLU) exhibit low intensity because each element is fetched from memory, evaluated once, and written back. Figure D.1 charts this operational boundary by plotting achievable performance against arithmetic intensity: a diagonal memory-bandwidth ceiling rises linearly from

the origin until it intersects the horizontal peak-compute ceiling at the ridge point. A workload's operational intensity dictates which regime governs execution, revealing whether optimizations must target memory traffic or arithmetic throughput.

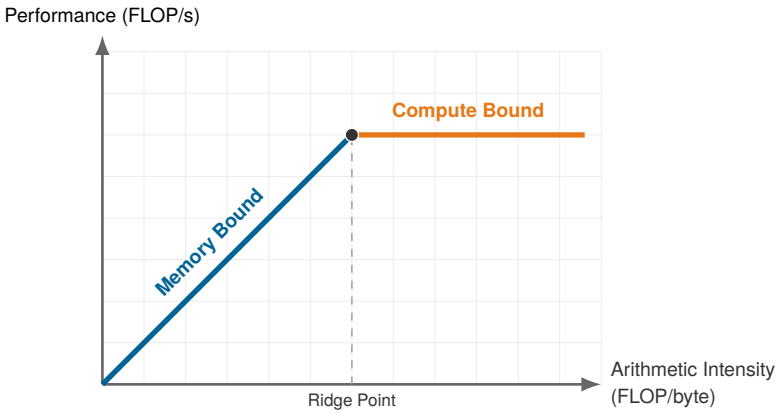


Figure D.1: The Roofline Model: The sloped memory-bandwidth ceiling and horizontal peak-compute ceiling meet at the ridge point. Plotting a workload by arithmetic intensity identifies which roofline ceiling applies.

The **ridge point** determines the hardware's balance. Formally, when computation and data movement execute under the overlapped iron law, idealized wall-clock time is bounded below by the critical path:

$$T \geq \max\left(\frac{O}{R_{\text{peak}}}, \frac{D_{\text{vol}}}{\text{BW}}\right)$$

Attainable throughput $R_{\text{attain}} = O/T$ is therefore bounded by:

$$R_{\text{attain}} \leq \frac{O}{\max\left(\frac{O}{R_{\text{peak}}}, \frac{D_{\text{vol}}}{\text{BW}}\right)} = \min\left(R_{\text{peak}}, \frac{O}{D_{\text{vol}}} \times \text{BW}\right) = \min(R_{\text{peak}}, I \times \text{BW})$$

where $I = O/D_{\text{vol}}$ is operational intensity at the DRAM interface. The ridge point I_{ridge} is the transition intensity where the compute and bandwidth ceilings meet:

$$R_{\text{peak}} = I_{\text{ridge}} \times \text{BW} \implies I_{\text{ridge}} = \frac{R_{\text{peak}}}{\text{BW}}$$

If a workload's operational intensity falls below I_{ridge}, it is memory bound; if above, it is compute bound.

> **Systems Perspective D.3: Batch size controls arithmetic intensity**
>
> For matrix multiplications, arithmetic intensity scales with the batch dimension. For $\mathbf{Y} = \mathbf{XW}$, where $\mathbf{X}$ is $(B \times d_{\text{in}})$ and $\mathbf{W}$ is $(d_{\text{in}} \times d_{\text{out}})$:
>
> - **FLOPs**: $2 \times B \times d_{\text{in}} \times d_{\text{out}}$ (multiply-adds).
> - **Bytes**: Moving $\mathbf{X}$, $\mathbf{W}$, and $\mathbf{Y}$ costs approximately $s_{\text{elem}}(Bd_{\text{in}} + d_{\text{in}}d_{\text{out}} + Bd_{\text{out}})$ bytes.
>
> Doubling B doubles FLOPs while weight traffic stays nearly constant when weights dominate, increasing arithmetic intensity. This is why batching often helps inference serving, although activation traffic, cache behavior, shape, precision, and the ridge point determine whether it reaches the compute ceiling.

D.2.1.1 A concrete example: The A100 analysis

Consider an NVIDIA A100 GPU with FP16 Tensor Core performance of 312 TFLOP/s and HBM2e bandwidth of 2.04 TB/s. The ridge point is 312 TFLOP/s/2.04 TB/s = 153 FLOP/byte (the Tera prefixes cancel, yielding FLOP/byte).

Two common operations fall on opposite sides of that ridge. General matrix multiply (GEMM) on two square matrices of size 4096 by 4096 has arithmetic intensity of approximately 1365 FLOP/byte, so 1365 FLOP/byte > 153 FLOP/byte and the operation is compute bound. An element-wise ReLU on an FP16 tensor performs 1 comparison while reading 2 bytes and writing 2 bytes (4 bytes total), yielding an arithmetic intensity of only $1/4 = 0.25$ FLOP/byte. Because 0.25 FLOP/byte $\ll$ 153 FLOP/byte, the operation is severely memory bound, achieving only about 0.16 percent of peak TFLOP/s. This contrast explains why modern frameworks fuse operations. Combining ReLU directly with the preceding MatMul kernel avoids writing intermediate results to memory, effectively preserving memory bandwidth.

D.2.2 Dimensional analysis

The Roofline Model helps diagnose *where* a bottleneck lies. Before applying any performance equation, however, it must be verified as physically meaningful. Dimensional analysis provides this sanity check: any valid equation must be **dimensionally homogeneous**—every term must resolve to the same units. If they do not, the equation contains an error.

For serialized phases, consider the iron law of ML systems (principle 3) introduced in section 1.7:

$$T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

Correctness is verified by confirming that every term resolves to time (seconds):

$$T[\text{s}] = \underbrace{\frac{D_{\text{vol}}[\text{bytes}]}{\text{BW}[\text{bytes/s}]}}_{\text{seconds}} + \underbrace{\frac{O[\text{FLOPs}]}{R_{\text{peak}}[\text{FLOP/s}] \cdot \eta_{\text{hw}}[1]}}_{\text{seconds}} + \underbrace{L_{\text{lat}}[\text{s}]}_{\text{seconds}}$$

- **Data term**: $\frac{\text{bytes}}{\text{bytes/s}} = \text{bytes} \times \frac{\text{s}}{\text{bytes}} = \text{s}$
- **Compute term**: $\frac{\text{FLOPs}}{\text{FLOP/s}} = \text{FLOPs} \times \frac{\text{s}}{\text{FLOP}} = \text{s}$
- **Latency term**: Already in seconds.

The equation is physically consistent. Apply this technique to any systems equation: if the dimensions do not match, the formula is wrong. "FLOPs" and "Bandwidth" cannot be traded directly because they have different units. Any such trade-off must convert through Time, which is precisely what the iron law quantifies.

Once an equation is dimensionally sound, the next question is how its time terms scale across devices. Section D.2.3 answers that question with two complementary bounds.

D.2.3 Amdahl's law and Gustafson's law

Parallelization is the primary tool for scaling ML, but its limits depend on *how* the workload scales. These two laws frame the fundamental tension in parallel computing. **Amdahl's law** is the pessimist's view, governing how much faster a *fixed* task can run (optimizing latency). **Gustafson's law** is the optimist's view, governing how much *more* work can be done in the same time (optimizing throughput).

D.2.3.1 Strong scaling (Amdahl's Law)

Strong scaling measures how much faster a fixed-size problem runs as processors are added.

Amdahl's law (Amdahl 1967) states that speedup is limited by the serial fraction of the task.[4] Let $T(1) = 1$ be normalized single-processor execution time, composed of serial fraction $1 - f_{\text{parallel}}$ and parallelizable fraction f_{parallel}. If N_{workers} processors execute the parallel fraction with linear scaling and zero communication overhead, execution time on N_{workers} processors is:

$$T(N_{\text{workers}}) = (1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{N_{\text{workers}}}$$

[4] **Gene Amdahl (1922–2015)**: A legendary computer architect at IBM, where he was the chief architect of the System/360. He later founded Amdahl Corporation to compete with IBM in the mainframe market.

The resulting theoretical speedup is:

$$\text{Speedup}(N_{\text{workers}}) = \frac{T(1)}{T(N_{\text{workers}})} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{N_{\text{workers}}}}$$

As $N_{\text{workers}} \to \infty$, the term $\frac{f_{\text{parallel}}}{N_{\text{workers}}} \to 0$, and the speedup converges to $1/(1 - f_{\text{parallel}})$.

To see Amdahl's Law in action, suppose 5 percent of a training step is serial overhead (for example, Python global interpreter lock (GIL), kernel launch latency) and 95 percent is parallelizable matrix math:

- With $N_{\text{workers}} = 1$, speedup is 1.
- With $N_{\text{workers}} = 8$, speedup is $1/(0.05 + 0.95/8) \approx 5.9\times$.
- With $N_{\text{workers}} \to \infty$, speedup is capped at $1/0.05 = 20\times$.

No matter how many accelerators are added, this fixed workload cannot run faster than 20×.

D.2.3.2 Weak scaling (Gustafson's Law)

Weak scaling measures how much larger a problem can become while holding runtime fixed as processors are added.[5]

This is the reality of Large Language Models. Rather than using 1,000 accelerators to train a model on a small dataset in milliseconds, they are used to train on a dataset 1,000× larger in reasonable time.

For N_{workers} processors, let $1 - f_{\text{parallel}}$ be the serial fraction of execution time measured on those processors for the scaled workload, and f_{parallel} the parallel fraction. Gustafson's law (Gustafson 1988) derives "scaled speedup" by asking how long that scaled problem would take on a single processor. On N_{workers} processors, normalized runtime is $T(N_{\text{workers}}) = (1 - f_{\text{parallel}}) + f_{\text{parallel}} = 1$. Executing the same total work on a single processor requires running the parallel fraction sequentially, taking $T(1) = (1 - f_{\text{parallel}}) + N_{\text{workers}} \cdot f_{\text{parallel}} = N_{\text{workers}} - (1 - f_{\text{parallel}})(N_{\text{workers}} - 1)$. The resulting scaled speedup is:

$$\text{Scaled Speedup}(N_{\text{workers}}) = \frac{T(1)}{T(N_{\text{workers}})} = N_{\text{workers}} - (1 - f_{\text{parallel}})(N_{\text{workers}} - 1)$$

Because the parallel part grows linearly with N_{workers} while the serial fraction $1 - f_{\text{parallel}}$ remains fixed, weak scaling can keep efficiency high as the problem grows.

Using the same 5 percent serial overhead ($1 - f_{\text{parallel}} = 0.05$), Gustafson's Law tells a very different story:

- With $N_{\text{workers}} = 1$, speedup is 1.
- With $N_{\text{workers}} = 8$, Scaled Speedup is $8 - 0.05 \times (7) = 8 - 0.35 = 7.65\times$.
- With $N_{\text{workers}} = 1000$, Scaled Speedup is $1000 - 0.05 \times (999) \approx 950\times$.

In weak scaling, efficiency remains high because the useful work (training the model) scales up to dwarf the fixed overheads.

The same scaling lens turns model size, data size, hardware count, and utilization into a single training-time estimate.

[5] **John Gustafson**: A computer scientist known for his work in parallel computing and for introducing the Unum (universal number) format. His law was a direct response to the perceived "limits" of Amdahl's Law when applied to massive scale.



Napkin Math D.1: The training time equation

Problem: How long does dense-transformer training take for a stated model size, token count, accelerator count, peak rate, and utilization?

Math:

$$T \approx \frac{6 \cdot P \cdot D}{N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}}}$$

Variables:

- **Training FLOP factor**: The factor 6 derives from $2P$ FLOP/token for the forward pass and $4P$ FLOP/token for the backward pass ($2P$ for activation gradients and $2P$ for weight gradients), yielding $6PD$ FLOPs total across D training tokens.
- P: Number of model parameters.
- D: Number of training tokens.
- N_{GPU}: Number of GPUs.
- R_{peak}: Peak FLOP/s of one accelerator.
- η_{hw}: Hardware utilization, typically 30 percent–50 percent in this training estimate.

Example: Training a 1B parameter model on 20B tokens using 1 A100 (312 TFLOP/s) at 40 percent utilization. Total FLOPs $= 6 \times 1 \times 10^9 \times 2 \times 10^{10} = 1.2 \times 10^{20}$ FLOPs. Throughput $= 1 \times (3.12 \times 10^{14}) \times 0.40 \approx 1.248 \times 10^{14}$ FLOP/s. $T = \frac{1.2 \times 10^{20}}{1.248 \times 10^{14}} \approx 961{,}538.5$ seconds $\approx 16{,}025.6$ minutes.

Systems insight: Under these assumptions, training takes 961,538.5 seconds (≈ 16025.6 minutes, or about 11.1 days); the estimate makes model scale, data scale, effective accelerator throughput, and utilization explicit levers.

D.2.4 Little's Law

For capacity planning in stable inference systems, **Little's law** (Little 1961) relates long-run mean concurrency (Q_{req}), mean arrival rate (λ_{arr}), and mean time in system (T_{lat}):[6]

$$Q_{\text{req}} = \lambda_{\text{arr}} \times T_{\text{lat}}$$

[6] **John Little**: An Institute Professor at MIT and a pioneer in the field of operations research. His law, proved in 1961, is fundamental to queuing theory and is used across fields from manufacturing to computer network analysis.

The law follows from operational conservation of requests over any observation interval $[0, T_{\text{obs}}]$. Let $A(t)$ denote cumulative arrivals and $C(t)$ cumulative completions, so the instantaneous in-flight queue occupancy is $Q(t) = A(t) - C(t)$. The average number of requests in the system over $[0, T_{\text{obs}}]$ is $\bar{Q} = \frac{1}{T_{\text{obs}}} \int_0^{T_{\text{obs}}} Q(t)\, dt$. The total accumulated request-time $\int_0^{T_{\text{obs}}} Q(t)\, dt$ equals the sum of residence times $\sum_{i=1}^{N(T_{\text{obs}})} W_i$ of all $N(T_{\text{obs}})$ requests completed by time T_{obs} (plus boundary terms that vanish as $T_{\text{obs}} \to \infty$). Dividing both sides by T_{obs}:

$$\bar{Q} = \frac{1}{T_{\text{obs}}} \int_0^{T_{\text{obs}}} Q(t)\, dt = \frac{N(T_{\text{obs}})}{T_{\text{obs}}} \cdot \frac{1}{N(T_{\text{obs}})} \sum_{i=1}^{N(T_{\text{obs}})} W_i$$

In the limit as $T_{\text{obs}} \to \infty$ for a stable queue, $\lim_{T_{\text{obs}} \to \infty} \frac{N(T_{\text{obs}})}{T_{\text{obs}}} = \lambda_{\text{arr}}$ and $\lim_{N \to \infty} \frac{1}{N} \sum_{i=1}^{N} W_i = T_{\text{lat}}$, yielding $Q_{\text{req}} = \lambda_{\text{arr}} \times T_{\text{lat}}$ regardless of arrival distribution or service-time distribution.

To see this in practice, consider sustaining 1,000 QPS with 50 ms average latency. The law dictates that the system must support 1000×0.05 s = 50 concurrent requests.

This sizes worker pools, but memory needs another assumption. If every in-system request simultaneously holds 1 GB of device state, the average population implies 50 GB; queued requests may instead stay in host memory or share batched state. If 24 device slots ($Q_{\text{req,max}}$, the most requests the device can hold at once) each remain occupied for 50 ms, their throughput proxy is $Q_{\text{req,max}}/T_{\text{lat}} = 24/0.05 = 480$ QPS; Little's Law alone sets no physical maximum.

These physics-based models—Roofline, Amdahl, Gustafson, and Little—diagnose *where* bottlenecks lie. Translating those diagnoses into actionable optimizations, however, requires understanding the concrete hardware structures that impose them: caches, memory buses, and interconnects.

D.3 Computer Architecture Essentials

A GPU advertises 1,000 TFLOP/s, yet a kernel achieves only 30 TFLOP/s. The gap may reflect data movement, insufficient parallelism, synchronization, launch overhead, or software inefficiency. While physics sets theoretical performance bounds, computer architecture defines the machinery that determines how close a real workload can get. The following discussion covers the latency, bandwidth, and energy trade-offs that shape system design.

D.3.1 Latencies every programmer should know

The first step in systems intuition is understanding the cost of distance. Table D.9 quantifies how long the processor waits for data from different levels of the memory hierarchy. If accessing a register is like picking up a nearby pencil, fetching from high-bandwidth memory (HBM) is walking across the office, and fetching from disk is flying to the moon.

Table D.9: The Latency Hierarchy: Representative access times for modern AI hardware; the approximate cycle counts use a 3 GHz reference clock. The jump from SRAM cache to HBM makes cache locality a major performance factor.

Component	Latency (ns)	Cycles (Approx)	Relative "Distance"
Register	~0.3 ns	1 cycle	10 seconds
L1 Cache	~1 ns	3–4 cycles	33.3 seconds
L2 Cache	~4 ns	12 cycles	2.2 minutes
HBM3 (GPU Memory)	~300 ns	1,000 cycles	2.8 hours
NVLink (GPU-GPU)	~500 ns	1,500 cycles	4.6 hours
PCIe (CPU-GPU)	~1000 ns	3,000 cycles	9.3 hours
InfiniBand (Network)	~5000 ns	15,000 cycles	1.9 days
SSD (NVMe)	~100000 ns	300,000 cycles	38.6 days

D.3.2 The AI hardware cheat sheet (modern reference)

Latency measures the wait for the *first* byte; bandwidth measures how many bytes follow. Table D.10 provides the constants for back-of-the-envelope "Roofline" calculations. These represent the "standard units of compute" for the current era of machine learning.

Table D.10: Reference Specs: Key constants for quantitative analysis. Always check specific datasheets; HBM capacity follows vendor units, and interconnect bandwidth is aggregate bidirectional, so topology and direction matter.

Spec	NVIDIA H100 (SXM)	Google TPU v5p	System Impact
BF16 Peak	989 TFLOP/s	459 TFLOP/s	The "Speed Limit" (R_{peak})
Memory Bandwidth	3.35 TB/s	2.76 TB/s	The "Width of the Pipe" (BW)
HBM Capacity	80 GB	95 GB	Max Model Size (P)/Batch Size (B)
On-chip SRAM	50 MB L2	128 MB VMEM/chip	Critical for Operator Fusion
Interconnect	900 GB/s (NVLink)	1200 GB/s (Inter-Chip Interconnect, ICI)	Determines Model Parallelism Scaling

D.3.3 The memory hierarchy

Computer systems rely on tiered storage architectures because no single physical medium simultaneously provides massive storage capacity and single-cycle access latency. Figure D.2 partitions these hardware technologies across a four-tier pyramid, contrasting the upward progression toward faster speed and lower latency (from base storage up through HBM/DRAM, on-chip cache, and registers) against the downward expansion in physical capacity and cost efficiency. When data movement sits on the critical execution path, staging tensors into upper tiers circumvents the orders-of-magnitude latency penalties imposed by off-chip DRAM and persistent disk.

The memory hierarchy is the fundamental physical constraint of machine learning systems. Table D.11 consolidates the physical properties—latency, bandwidth, and energy—across the entire stack.

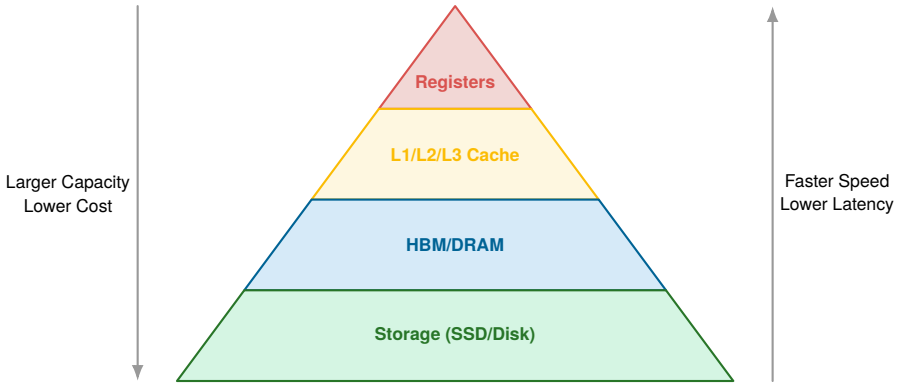


Figure D.2: The Memory Hierarchy: Performance depends on data proximity. Accessing HBM is roughly 1,000× slower than registers; accessing SSD is roughly 300,000× slower.

Table D.11: Physical Properties of the Memory Hierarchy (c. 2024): Representative latency and bandwidth anchors across the memory hierarchy. The hierarchy spans roughly five orders of magnitude in latency. NVLink is aggregate bidirectional; only the cited 45 nm DRAM energy is shown because other tiers are implementation-specific. Keeping data higher in the hierarchy can therefore deliver large performance gains.

Layer	Technology	Latency	Bandwidth	Energy Reference (per 32b)
Registers	Flip-Flops	~0.3 ns	—	—
L1 Cache	SRAM	~1 ns	—	—
L2 Cache	SRAM	~4 ns	—	—
Memory (Local)	HBM3	~300 ns	3350 GB/s	640 pJ
Interconnect	NVLink 4.0	~500 ns	900 GB/s	—
Host Link	PCIe Gen5	~1000 ns	64 GB/s	—
System RAM	DDR5	~100 ns	50 GB/s	—
Network (Fabric)	InfiniBand NDR	~5000 ns	50 GB/s	—
Storage (Local)	NVMe SSD	~100000 ns	7 GB/s	—

The hierarchy's energy costs reveal why data movement dominates modern system design.

> Systems Perspective D.4: The high cost of data movement
>
> In the cited 45 nm reference, fetching a 32-bit value from DRAM costs roughly 581× the energy of one FP16 multiply (~640 pJ vs. ~1.1 pJ). The ratio is platform-specific and does not alone determine workload energy, which also depends on access counts and reuse. It nevertheless explains why arithmetic intensity matters.

D.3.4 Bandwidth vs. latency

Bandwidth (throughput) and latency (delay) are distinct constraints. For data volume D_{vol} sent over effective bandwidth BW with fixed path latency L_{lat}, and assuming the latency and serialization costs do not overlap, total transfer time is:

$$T = L_{\text{lat}} + \frac{D_{\text{vol}}}{\text{BW}}$$

The crossover point is the transfer size where the two terms are equal:

$$n^* = L_{\text{lat}} \times \text{BW}$$

For transfers below n^* (for example, small request or control messages), latency dominates. For transfers above it (for example, loading weights), bandwidth dominates.

Consider sending data over a 10 Gb/s link with 10 ms base request/response latency. The crossover size is 12.5 MB, so the dominant term depends on which side of that threshold the transfer falls.

- **Latency-bound packet (1 KB)**: Transmission time is 1 KB $\times$ 8/10 Gb/s $\approx$ 0.8 µs. Total time is approximately 10 ms + 0.8 µs $\approx$ 10 ms. **Result**: Fixed path and protocol latency dominates; propagation is one component.
- **Bandwidth-bound checkpoint (1 GB)**: Transmission time is 1 GB $\times$ 8/10 Gbps $\approx$ 800 ms. Total time is $\approx$ 10 ms + 800 ms = 810 ms. **Result**: Base latency is negligible; bandwidth is the bottleneck.

Architecture determines how fast data can move, but numerical precision directly controls *how much* data must move. Halving precision from FP32 to FP16 halves bytes per parameter and can nearly double useful bandwidth when the hardware, kernels, and model accuracy support it. Understanding these trade-offs requires a closer look at how numbers are represented in hardware.

D.4 Numerical Representations

Statistics characterizes data distributions; numerical representations determine how the system stores those values. In ML systems, the choice of precision (FP32 vs. BF16 vs. INT8) is a direct trade-off between statistical fidelity and hardware throughput.

D.4.1 Floating-point format comparison

IEEE 754 standards and AI-specific numerical representations define fundamental trade-offs between dynamic range (the span of representable values) and precision (the granularity of values within that range). While table D.12 catalogs operational ranges and common use cases across precision levels, figure D.3 diagrams the underlying bit partitions for FP32, BF16, FP16, and INT8. Comparing these layouts highlights how BF16 retains FP32's full 8-bit exponent field—preserving identical dynamic range while truncating the mantissa to 7 bits—whereas FP16 widens the mantissa to 10 bits at the expense of a narrower 5-bit exponent.

Table D.12: Numerical Format Comparison: Each format trades off precision, dynamic range, memory footprint, and compute throughput. BF16 shares FP32's normal exponent range while using half the storage.

Format	Bits	Exponent	Mantissa	Dynamic Range	Typical Use Case
FP32	32	8	23	$\sim 10^{-38}$ to 10^{38}	Training (full precision), reference inference
FP16	16	5	10	Normal: 6.10×10^{-5}–6.55×10^{4}	Training often with loss scaling, inference
BF16	16	8	7	FP32 normal exponent range	Training, usually without loss scaling
FP8	8	4 or 5	3 or 2	Varies	Training/inference on supported hardware
INT8	8	N/A	N/A	-128 to 127	Inference after quantization

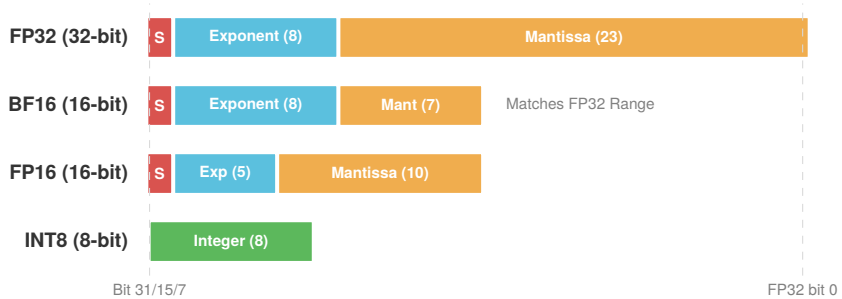


Figure D.3: Numerical Format Bit Layouts: A visual comparison of bit allocations. BF16 preserves FP32's 8-bit exponent and therefore its normal exponent range. FP16 trades range for precision and often uses loss scaling to reduce underflow.

Beyond bit width, the allocation of bits between exponent and mantissa determines what range of values each format can represent.

Among these formats, BF16[7] deserves special attention (Wang and Kanwar 2019; Kalamkar et al. 2019). It occupies the same 16-bit storage class as FP16 but behaves differently in training, so nominal width alone does not predict the operational burden. The following systems perspective connects bit allocation to memory traffic, numerical stability, and deployment efficiency.

[7] **BF16 (brain floating point 16)**: Originally introduced with Google TPUv2 and later adopted by Intel, Arm, and NVIDIA, BF16 became a common training format across accelerator ecosystems. Its adoption made format support in hardware, kernels, and distributed-training libraries a portability concern rather than a vendor-specific feature.



Systems Perspective D.5: The dynamic range wall

The choice of numerical format is a direct application of the iron law of ML systems (principle 3). Reducing precision from FP32 to BF16 or FP16 halves the data volume term, potentially doubling throughput on memory-bound workloads. However, the *type* of 16-bit format determines the engineering complexity:

- **Dynamic range (the exponent)**: BF16 preserves the eight-bit exponent of FP32 and therefore the same normal exponent range (Kalamkar et al. 2019).
- **Precision (the mantissa)**: FP16 has a larger 10-bit mantissa than BF16 (7 bits), offering higher precision for values within its range. Its five-bit exponent, however, raises underflow risk. FP16 training therefore often uses **loss scaling**, an operational step that multiplies gradients by a large constant to keep more values representable (Micikevicius et al. 2017).
- **Energy efficiency**: INT8 operations can be substantially more efficient than floating-point equivalents because they move less data and use simpler integer arithmetic paths. Moving to INT8 for inference is a primary lever for deploying neural networks under tight memory, latency, or power budgets (Jacob et al. 2018; Krishnamoorthi 2018).

D.4.2 Integer quantization

Quantization maps continuous floating-point values to discrete integers, typically INT8. The key challenge is choosing how to map the floating-point range to integers. Two approaches dominate.

For symmetric quantization, let $\alpha_{\text{sym}} = \max|x|$ over the calibrated range, assume x is clipped to $[-\alpha_{\text{sym}}, \alpha_{\text{sym}}]$, and set the scale $s = \alpha_{\text{sym}}/127$. The signed INT8 code is:

$$x_q = \text{round}\left(\frac{x}{s}\right)$$

This mapping works well for weight distributions centered around zero.

Asymmetric quantization handles distributions that are not centered (common after ReLU, which produces only nonnegative values) by shifting the range before scaling. Let α and β bound the calibrated range, clip x to $[\alpha, \beta]$, and set the scale $s = (\beta - \alpha)/255$. A common unsigned 8-bit mapping is:

$$x_q = \text{round}\left(\frac{x - \alpha}{s}\right)$$

The choice between symmetric and asymmetric quantization depends on the tensor's distribution and has measurable accuracy implications.

Summary

Machine-side diagnosis begins with units, ceilings, and working sets. Reference numbers establish orders of magnitude; the roofline model and the iron law identify whether operations, bytes, or latency bind; Amdahl's law, Gustafson's law, and Little's law test speedup, scaling, and concurrency claims; and dimensional analysis catches impossible results. The memory hierarchy and interconnect show how those limits appear as latency and bandwidth constraints, while numerical representations show how dynamic range, precision, and quantization change storage, energy, throughput, and accuracy.

Treat peak specifications as ceilings, not predictions. Compare a measurement with the relevant ceiling, then trace the remaining gap to utilization, data movement, runtime overhead, architecture,

or software. A back-of-envelope estimate is useful when it states its assumptions and units and makes the next measurement obvious.

E

System Assumptions

A napkin calculation is only as trustworthy as the inputs behind it, and those inputs are easy to lose track of: an accelerator's peak rate, a model's parameter count, an electricity price, an energy cost per operation. This appendix gathers those canonical values and unit conventions into a single reference sheet, allowing any quantitative estimate in the book to be audited, updated with newer figures, or checked across chapters. It also fixes the unit conventions those estimates depend on. The chapters can then argue from numbers without restating where each one came from.

How to Use This Appendix

When a chapter says an H100 delivers a certain ridge point, or that training a 7B model needs a certain amount of memory, the underlying bandwidths, capacities, and model sizes are listed here. A local estimate can use the relevant Value and Unit entries. The tables are grouped by topic: accelerators, reference models, energy per access, interconnect bandwidth, economics, production-scale anchors, and unit conventions. Assumptions come from vendor datasheet peaks, published studies or industry reports, illustrative market or grid statistics, and book conventions; section E.1 lists the provenance in one place.

The calculations below expose which assumptions dominate each estimate, so a changed hardware specification or price can propagate without changing the method.

The constants in this appendix support quick, reproducible calculations. The three examples below apply them to performance, capacity, and cost.

> **Napkin Math E.1: Classify a roofline regime**
>
> **Problem**: For the stated H100 and workload intensities, which operations are compute bound and which are memory bound?
>
> **Variables**: For an H100 at FP16/BF16 peak, use 989 TFLOP/s peak compute and 3.35 TB/s memory bandwidth.
>
> **Math**: Divide peak FLOP/s by memory bandwidth to get the roofline ridge point: 989 TFLOP/s/3.35 TB/s $\approx$ 295.2 FLOP/byte. A large general matrix multiply (GEMM) with $N = 4{,}096$ has intensity $N/3 \approx 1{,}365.3$ FLOP/byte.
>
> **Result**: Operations above 295.2 FLOP/byte are compute bound; operations below it are memory bound. The GEMM example is compute bound, while a single-token autoregressive decode with intensity ≈ 1 FLOP/byte is deeply memory bound.
>
> **Systems insight**: The same accelerator can be compute rich and memory constrained. Arithmetic intensity determines which resource the workload actually consumes.

Throughput is only one constraint. Training also requires enough capacity for weights, gradients, and optimizer state.

Napkin Math E.2: Estimate training-state memory

Problem: How much model-state memory is required to train a 7B model with mixed-precision Adaptive Moment Estimation (Adam)?

Variables: Mixed-precision Adam stores BF16 weights, gradients, FP32 master weights, momentum, and variance. The model-state budget is 16 bytes per parameter.

Math: For 7B parameters, model state is $7 \times 10^9 \times$ 16 bytes = 112 GB.

Result: An H100 has 80 GB of high-bandwidth memory (HBM), so the model state alone exceeds a single accelerator before accounting for activations.

Systems insight: Optimizer state, not weights alone, sets the floor for training memory. Capacity planning that starts from parameter bytes underestimates the real requirement.

Fitting the model in memory establishes feasibility, but not operating cost. Power and elapsed time provide the remaining quantities for a first electricity estimate.

Napkin Math E.3: Estimate accelerator electricity cost

Problem: What is the accelerator-only electricity cost of the GPT-3 training run?

Variables: Use 10,000 V100s, 300 W per accelerator, ~14.8 days wall-clock, and \$0.12/kWh electricity (Patterson et al. 2021).

Math: The run requires roughly 3.14×10^{23} FLOPs. The accelerator-only electricity-cost estimate is ~14.8 days wall-clock $\times$ 10,000 V100s $\times$ 24 h/day $\times$ 0.3 kW $\times$ \$0.12/kWh = ~\$127,872.

Result: The accelerator-only electricity cost is roughly \$128,000. This thermal design power (TDP)-only estimate excludes host CPUs, networking, storage, cooling, and facility overhead. The 1,066 MWh of accelerator energy behind it covers about 83 percent of the 1,287 MWh reported for the run, which also counts host power and data center overhead. The GPU count and duration are a published reconstruction from measured per-GPU throughput, so only their product is firm (Patterson et al. 2021).

Systems insight: Energy is measurable from power and time, but full cost accounting must also include capital utilization, networking, storage, staffing, and failed or repeated runs.

Accelerator Specifications

These are the **accelerator assumptions** used in the book's roofline, training-memory, energy, and cost examples: peak throughput, memory bandwidth, memory capacity, and TDP for each generation the chapters cite. Values are vendor datasheet peaks—ceilings for napkin math, not sustained utilization—drawn from NVIDIA product documentation and IEEE Micro architecture articles (NVIDIA Corporation 2017, 2020b, 2020a, 2021a, 2018, 2024; Choquette et al. 2021; Choquette 2023), AMD MI300X documentation (AMD 2023), Google Tensor Processing Unit (TPU) publications (Jouppi et al. 2020; Jouppi et al. 2023), and Google Cloud's current TPU v6e specification page (Google Cloud 2026b). Capacity rows preserve official capacity nameplates. Tables progress from table E.2 (Volta) and table E.3 (Turing) through current and forward-looking generations.

Master accelerator spec matrix

The master accelerator matrix in table E.1 synthesizes specifications across key GPU, TPU, Wafer-Scale, and domain-specific AI accelerators.

Table E.1: Master Accelerator Specification Matrix: Comprehensive reference matrix comparing production AI hardware across architectures, process nodes, TDP, multi-precision compute throughputs (FP64 to INT8), memory capacity and bandwidth, on-chip SRAM capacity, and FP16 Roofline Ridge Point (I_{ridge} = Peak FP16 TFLOP/s/Memory Bandwidth (TB/s)).

Architecture/Chip	Node (nm)	TDP (W)	Compute Peak TFLOP/s or TOPS	Memory Capacity & Bandwidth	On-Chip SRAM	I_{ridge} (FLOP/byte)
NVIDIA V100 (SXM2 32 GB)	12 nm (TSMC)	300	FP64: 7.8, FP32: 15.7, FP16: 125 (TC), INT8: 125 (TC)	32 GB HBM2 @ 0.90 TB/s	16 MB	138.9
NVIDIA A100 (80 GB SXM4)	7 nm N7 (TSMC)	400	FP64: 9.7/19.5, FP32: 19.5, TF32: 156, FP16/BF16: 312, INT8: 624	80 GB HBM2e @ 2.04 TB/s	40 MB	153.0
NVIDIA H100 (80 GB SXM5)	4 nm 4N (TSMC)	700	FP64: 34/67, FP32: 67, TF32: 494, FP16/BF16: 989, FP8: 1,978, INT8: 1,978	80 GB HBM3 @ 3.35 TB/s	50 MB	295.2
NVIDIA H200 (141 GB SXM5)	4 nm 4N (TSMC)	700	FP64: 34/67, FP32: 67, TF32: 494, FP16/BF16: 989, FP8: 1,978, INT8: 1,978	141 GB HBM3e @ 4.80 TB/s	50 MB	206.0
NVIDIA B200 (192 GB SXM)	4 nm 4NP (TSMC)	1000	FP64: 45/90, FP32: 90, TF32: 1,250, FP16/BF16: 2,250, FP8: 4,500, INT8: 4,500	192 GB HBM3e @ 8.00 TB/s	128 MB	281.3
NVIDIA L40S (48 GB PCIe)	4 nm 4N (TSMC)	350	FP64: 1.4, FP32: 91.6, TF32: 183, FP16/BF16: 366, FP8: 733, INT8: 733	48 GB GDDR6 @ 0.86 TB/s	96 MB	423.6
Google TPU v1	28 nm (TSMC)	75	INT8: 92 TOPS (FP formats N/A)	8 GB DDR3 @ 0.034 TB/s	28 MB	N/A (2,705.9 INT8)
Google TPU v2	16 nm (TSMC)	280	BF16/FP16: 46 TFLOP/s	16 GB HBM @ 0.70 TB/s	32 MB	65.7
Google TPU v3	16 nm (TSMC)	450	BF16/FP16: 123 TFLOP/s	32 GB HBM2 @ 0.90 TB/s	32 MB	136.7
Google TPU v4	7 nm N7 (TSMC)	170	BF16/FP16: 275, FP32: 275, INT8: 275	32 GB HBM2 @ 1.20 TB/s	48 MB	229.2
Google TPU v5e	7 nm N7 (TSMC)	140	BF16/FP16: 197, FP8: 394, INT8: 394	16 GB HBM2e @ 0.82 TB/s	32 MB	240.5
Google TPU v5p	4 nm 4N (TSMC)	450	BF16/FP16: 459, FP8: 918, INT8: 918	95 GB HBM3 @ 2.76 TB/s	96 MB	166.3
Google TPU v6e (Trillium)	4 nm 4N (TSMC)	400	BF16/FP16: 926, FP8: 1,852, INT8: 1,852	32 GB HBM3 @ 1.64 TB/s	64 MB	564.6
Cerebras WSE-3	5 nm N5 (TSMC)	23,000	FP16/BF16: 125,000, FP8: 250,000, INT8: 250,000	44 GB SRAM @ 21,000 TB/s	44,000 MB	5.95
AMD Instinct MI300X	5 nm + 6 nm 3D	750	FP64: 163.4, FP32: 163.4, TF32: 653.7, FP16/BF16: 1,307.4, FP8: 2,614.9	192 GB HBM3 @ 5.30 TB/s	256 MB	246.7
Intel Habana Gaudi 2	7 nm N7 (TSMC)	600	FP32: 38.5, TF32: 154, FP16/BF16: 307, FP8: 614, INT8: 614	96 GB HBM2e @ 2.45 TB/s	48 MB	125.3
Intel Habana Gaudi 3	5 nm N5 (TSMC)	900	FP32: 115, TF32: 460, FP16/BF16: 1,835, FP8: 3,670, INT8: 3,670	128 GB HBM3e @ 3.70 TB/s	96 MB	495.9

NVIDIA V100

Table E.2: NVIDIA V100 (Volta): Peak specs from NVIDIA's architecture whitepaper and product datasheet (NVIDIA Corporation 2017, 2020b). The V100 introduced Tensor Cores for mixed-precision training and anchors baseline-generation comparisons.

Assumption	Value	Unit
Peak FP16 tensor throughput (V100)	125	TFLOP/s
Peak FP32 throughput (V100)	15.7	TFLOP/s
HBM bandwidth (V100)	900	GB/s

Assumption	Value	Unit
HBM capacity (V100)	32	GB
TDP (V100)	300	W

NVIDIA T4

Low-power inference comparisons use the Turing-generation T4 as their baseline (table E.3).

Table E.3: NVIDIA T4 (Turing): Peak specs from NVIDIA Tesla T4 product documentation (NVIDIA Corporation 2018). The 70 W TDP makes this low-power inference accelerator a useful cost-per-inference anchor.

Assumption	Value	Unit
Peak FP16 tensor throughput (T4)	65	TFLOP/s
Peak INT8 throughput (T4)	130	TOPS
Memory bandwidth (T4)	320	GB/s
TDP (T4)	70	W

NVIDIA A100

The Ampere-generation specs (table E.4) anchor most training examples in the book.

Table E.4: NVIDIA A100 (Ampere): Peak specs from NVIDIA's architecture whitepaper, 80 GB datasheet, and architecture article (NVIDIA Corporation 2020a, 2021a; Choquette et al. 2021). The A100 is the most commonly cited accelerator in training examples; 80 GB HBM2e and TF32 Tensor Cores set memory-capacity and compute-intensity anchors.

Assumption	Value	Unit
Peak FP16 tensor throughput (A100)	312	TFLOP/s
Peak FP32 throughput (A100)	19.5	TFLOP/s
Peak INT8 throughput (A100)	624	TOPS
Peak TF32 throughput (A100)	156	TFLOP/s
HBM bandwidth (A100)	2039	GB/s
HBM capacity (A100)	80	GB
TDP (A100)	400	W

NVIDIA H100

For Hopper-era estimates, the reference values include FP8 Tensor Cores and the Transformer Engine (table E.5).

Table E.5: NVIDIA H100 (Hopper): Peak specs from the Hopper architecture overview (Choquette 2023). FP8 Tensor Cores and the Transformer Engine drive Hopper-generation training and serving estimates.

Assumption	Value	Unit
Peak FP16 tensor throughput (H100)	989	TFLOP/s
Peak FP32 CUDA throughput (H100)	67	TFLOP/s
Peak FP8 tensor throughput (H100)	1979	TFLOP/s
Peak INT8 throughput (H100)	1979	TOPS
Peak TF32 throughput (H100)	494	TFLOP/s
HBM bandwidth (H100)	3.35	TB/s
HBM capacity (H100)	80	GB
TDP (H100)	700	W

NVIDIA B200

Forward-looking capacity-planning examples rely on Blackwell-generation specs (table E.6).

Table E.6: NVIDIA B200 (Blackwell): Peak specs from NVIDIA Blackwell product documentation (NVIDIA Corporation 2024). Forward-looking capacity-planning examples use its HBM3e bandwidth and FP8 throughput.

Assumption	Value	Unit
Peak FP16 tensor throughput (B200)	2250	TFLOP/s
Peak FP8 tensor throughput (B200)	4500	TFLOP/s
Peak FP4 throughput (B200)	9000	TFLOP/s
HBM bandwidth (B200)	8	TB/s
HBM capacity (B200)	180	GB
TDP (B200)	1000	W

AMD Instinct MI300X

To compare with H100-class hardware used elsewhere, MI300X capacity, bandwidth, throughput, and power specifications provide a cross-vendor baseline (table E.7).

Table E.7: AMD Instinct MI300X: Peak specs from AMD Instinct MI300X product documentation (AMD 2023). High HBM capacity (192 GB) and bandwidth make it a common cross-vendor baseline for memory-bound workloads.

Assumption	Value	Unit
Peak FP16 tensor throughput (MI300X)	1307	TFLOP/s
HBM bandwidth (MI300X)	5.3	TB/s
HBM capacity (MI300X)	192	GB
TDP (MI300X)	750	W

Google TPU v4 and v6e

When comparing training economics across accelerator families, TPU figures offer an application-specific integrated circuit (ASIC)-based alternative (table E.8).

Table E.8: Google TPU v4 and v6e (Trillium): TPU v4 values come from Google TPU publications (Jouppi et al. 2023), and TPU v6e values come from Google Cloud's current Trillium specification page (Google Cloud 2026b).

Assumption	Value	Unit
Peak BF16 throughput (TPU v4)	275	TFLOP/s
Memory bandwidth (TPU v4)	1200	GB/s
Peak BF16 throughput (TPU v6e)	918	TFLOP/s
Memory bandwidth (TPU v6e)	1638	GB/s

CPU and mobile/edge processors

Edge and mobile ML examples rely on a different performance baseline (table E.9), contrasting starkly with data center throughput to illustrate why deployment target shapes every design decision.

Table E.9: CPU, Mobile NPU, and Edge Device Specs: Illustrative reference points for edge and mobile examples (not vendor peaks for a single SKU). The contrast with data-center accelerators shows why deployment target shapes every design decision.

Assumption	Value	Unit
Peak FP32 throughput (reference CPU)	1	TFLOP/s
DRAM bandwidth (reference server)	50	GB/s

Assumption	Value	Unit
Reference NPU throughput (iPhone 15 Pro)	35	TOPS
Memory bandwidth (iPhone 15 Pro)	51.2	GB/s
TDP (mobile device, reference)	5	W
Power (edge object detector, reference)	2	W
Battery capacity (phone, reference)	15	Wh

Model Specifications

These are the **reference model assumptions** behind training-cost, memory-footprint, and inference-workload examples: parameter counts, per-inference FLOP budgets, and published training-scale anchors. Published model sizes follow primary papers, model reports, and official model documentation (Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023). The inference FLOP budgets are book-derived values: they use sequence length 128 for BERT-Base, 224 $\times$ 224 images for ResNet-50 and MobileNetV2, and 640 $\times$ 640 images for YOLOv8-Nano; the vision models count two FLOPs per multiply-add. GPT-3 training FLOPs follow (Brown et al. 2020); the GPT-3 training duration and accelerator count follow the published carbon accounting of the run (Patterson et al. 2021). The GPT-4 parameter count and training GPU-days are public third-party mixture of experts (MoE) estimates (Patel and Wong 2023) because the GPT-4 technical report does not disclose architecture size. When a chapter estimates GPT-3-scale training time, it uses table E.10.

Table E.10: Reference Model Specifications: Parameter counts and FLOP budgets for worked examples. BERT, GPT-2, GPT-3, Llama, ResNet-50, MobileNetV2, and YOLOv8 rows use primary papers, model reports, or official model documentation (Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Patterson et al. 2021; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023). The GPT-3 duration and accelerator rows follow the run reconstruction in (Patterson et al. 2021). GPT-4 rows cite (OpenAI et al. 2023) for the official model family and (Patel and Wong 2023) for the public MoE parameter and GPU-day estimates used in this edition.

Assumption	Value	Unit
Inference FLOPs (BERT-Base)	2.2e+10	flop
Parameters (BERT-Base)	1.1e+08	param
Parameters (Llama 3 8B)	8.03e+09	param
Hidden dimension (GPT-2)	1600	-
Layers (GPT-2)	48	-
Parameters (GPT-2)	1.5e+09	param
Parameters (GPT-3)	1.75e+11	param
Reference training duration (GPT-3)	14.8	d
Reference training accelerators (GPT-3)	10,000	V100 GPUs
Reference training FLOPs (GPT-3)	3.14e+23	flop
Parameters (GPT-4, public MoE estimate)	1.76e+12	param
Reference training GPU-days (GPT-4)	2.5e+06	GPU-days
Inference FLOPs (ResNet-50)	8.2e+09	flop
Parameters (ResNet-50)	2.56e+07	param
Inference FLOPs (MobileNetV2)	6e+08	flop
Parameters (MobileNetV2)	3.50487e+06	param
Inference FLOPs (YOLOv8-Nano)	8.7e+09	flop

Training Memory Conventions

Training-memory napkin math assumes the mixed-precision Adam storage model used in the napkin-math callout and several training chapters: BF16 weights, BF16 gradients, and FP32 master weights plus Adam first- and second-moment buffers (16 bytes per parameter in total). This is a book convention, not a measured hardware constant. The BF16 data path follows established

BFLOAT16 mixed-precision practice (Kalamkar et al. 2019); the FP32-master pattern follows earlier FP16 work (Micikevicius et al. 2017; NVIDIA 2017), and Adam's first- and second-moment buffers follow (Kingma and Ba 2015). Table E.11 lists per-component byte widths; multiplying bytes per parameter (mixed-precision Adam) by the parameter count gives the nonactivation training-state footprint.

Table E.11: Training Memory Conventions: Per-parameter storage for mixed-precision Adam (2 + 2 + 12 = 16 bytes before activations). Book convention for napkin math; see (NVIDIA 2017) for mixed-precision training context.

Assumption	Value	Unit
Weight/gradient width (BF16)	2	bytes
Master weight width (FP32)	4	bytes
Master + Adam states per parameter (FP32)	12	bytes
Bytes per parameter (mixed-precision Adam)	16	bytes

Energy Constants

Hardware and model assumptions fix *what* runs where; **energy assumptions** fix whether the design is thermally and economically viable at the operation and memory-access level. Horowitz's 45 nm arithmetic and 32-bit DRAM estimates (Horowitz 2014), together with illustrative book anchors for register and SRAM access, MobileNetV2 inference, and 5G transfer, underpin the book's efficiency comparisons but are not process-independent. Table E.12 lists the access-energy hierarchy from registers through DRAM. That hierarchy quantifies why data reuse dominates kernel design.

Table E.12: Energy per Operation and Access: Arithmetic and 32-bit DRAM estimates use Horowitz's 45 nm model (Horowitz 2014), and the per-byte DRAM row is derived from that estimate. Register, SRAM, MobileNetV2, and 5G rows are illustrative book anchors. The roughly 6,400× gap between a 0.1 pJ register access and a 640 pJ DRAM access explains why data reuse dominates ML kernel optimization.

Assumption	Value	Unit
Register access energy	0.1	pJ
L1 SRAM access energy	0.5	pJ
L2 SRAM access energy	2	pJ
DRAM access energy (32-bit)	640	pJ
DRAM access energy (per byte)	160	pJ/byte
FP16 multiply energy (45 nm)	1.1	pJ/multiply
FP32 multiply energy (45 nm)	3.7	pJ/multiply
INT8 multiply energy (45 nm)	0.2	pJ/multiply
MobileNetV2 inference energy (reference)	0.1	mJ
5G transfer energy per MB	100	mJ/MB

Precision and arithmetic energy hierarchy

The arithmetic energy hierarchy in table E.13 compares the energy required per operation (E_{compute} in pJ/op) for additions and multiplications across floating-point, tensor, and integer precisions on modern 7 nm, 5 nm, and 3 nm silicon process nodes.

Table E.13: Precision and Arithmetic Energy Hierarchy: Energy per operation (E_{compute} in pJ/op) for Addition (Add) and Multiplication (Mul) across data formats (FP64, FP32, TF32, FP16, BF16, FP8 E4M3/E5M2, INT32, INT8, INT4) and semiconductor process nodes (7 nm, 5 nm, 3 nm). Energy scaling illustrates the quadratic $\mathcal{O}(b^2)$ multiplier vs linear $\mathcal{O}(b)$ adder bitwidth dependence (Horowitz 2014; Dally et al. 2021).

Precision/Data Format	7 nm Node (E_{compute} pJ)	5 nm Node (E_{compute} pJ)	3 nm Node (E_{compute} pJ)	Energy Ratio vs FP32 (3 nm)
FP64 (64-bit Float)	Add: 0.90/Mul: 2.50	Add: 0.58/Mul: 1.62	Add: 0.38/Mul: 1.05	2.23×

Precision/Data Format	7 nm Node (E_{compute} pJ)	5 nm Node (E_{compute} pJ)	3 nm Node (E_{compute} pJ)	Energy Ratio vs FP32 (3 nm)
FP32 (32-bit Float)	Add: 0.40/Mul: 1.10	Add: 0.26/Mul: 0.72	Add: 0.17/Mul: 0.47	1.00× (Ref)
TF32 (19-bit TensorFloat)	Add: 0.28/Mul: 0.65	Add: 0.18/Mul: 0.42	Add: 0.12/Mul: 0.27	0.57×
FP16 (16-bit IEEE Float)	Add: 0.20/Mul: 0.45	Add: 0.13/Mul: 0.29	Add: 0.08/Mul: 0.19	0.40×
BF16 (16-bit Bfloat16)	Add: 0.18/Mul: 0.40	Add: 0.12/Mul: 0.26	Add: 0.08/Mul: 0.17	0.36×
FP8 (E4M3/E5M2)	Add: 0.09/Mul: 0.20	Add: 0.06/Mul: 0.13	Add: 0.04/Mul: 0.08	0.17×
INT32 (32-bit Integer)	Add: 0.10/Mul: 0.80	Add: 0.06/Mul: 0.52	Add: 0.04/Mul: 0.34	0.72×
INT8 (8-bit Integer)	Add: 0.03/Mul: 0.20	Add: 0.02/Mul: 0.13	Add: 0.013/Mul: 0.085	0.18×
INT4 (4-bit Integer)	Add: 0.010/Mul: 0.060	Add: 0.006/Mul: 0.039	Add: 0.004/Mul: 0.025	0.053×

Interconnect and Network Bandwidth

Energy costs operate at the chip level, but real ML systems also move data across interconnects—between accelerators, across racks, and over wide-area networks. These **bandwidth assumptions** apply when chapters reason about gradient synchronization, pipeline bubbles, checkpoint I/O, or cross–data center latency. NVLink and PCIe rates follow accelerator product documentation (NVIDIA Corporation 2017, 2020a; Choquette 2023); the InfiniBand architecture specification anchors the protocol family (InfiniBand Trade Association 2000), while current high-speed product families and Ethernet roadmaps anchor modern link-rate examples (NVIDIA 2026f; Ethernet Alliance 2025); the speed-of-light-in-fiber floor is a physics identity. Table E.14 lists NVLink, InfiniBand, PCIe, Non-Volatile Memory Express (NVMe), and Ethernet rates together with the propagation speed of light in fiber.

Table E.14: Interconnect and Network Bandwidth: Link-family bandwidths use vendor specs, the InfiniBand architecture specification, and current product/roadmap documentation (InfiniBand Trade Association 2000; NVIDIA 2026f; Ethernet Alliance 2025). NVLink rates are aggregate bidirectional; other link rates are nominal per direction. Speed of light in fiber sets the cross–data-center latency floor.

Assumption	Value	Unit
NVLink bandwidth (V100)	300	GB/s
NVLink bandwidth (A100)	600	GB/s
NVLink bandwidth (H100)	900	GB/s
InfiniBand HDR link rate	200	Gb/s
InfiniBand NDR link rate	400	Gb/s
InfiniBand XDR link rate	800	Gb/s
PCIe Gen4 x16 rate	32	GB/s
PCIe Gen5 x16 rate	64	GB/s
NVMe sequential read bandwidth	7	GB/s
10 GbE link rate	10	Gb/s
100 GbE link rate	100	Gb/s
Speed of light in fiber	200000	km/s

Interconnect & network fabric physical spec sheet

The physical specification sheet in table E.15 details bandwidth, latency, framing, direct memory access (DMA) support, and energy per bit across host buses, accelerator interconnects, network fabrics, and cache-coherent interfaces.

Table E.15: Interconnect and Network Fabric Physical Spec Sheet: Comprehensive physical spec sheet for host buses, GPU interconnects, switches, network fabrics, and coherent interfaces (PCIe Gen3–Gen7, NVLink 1–5, NVSwitch 1–4, InfiniBand EDR–XDR, RoCE v2, CXL 1.1–3.1). Columns enumerate directional and aggregate bandwidth, PHY/link latency, line encoding/framing, DMA and GPUDirect capability, and physical layer energy cost (E_{link} in pJ/bit).

Bus/Interconnect Standard	Directional and Aggregate Bandwidth	Latency	Encoding/Framing	DMA/GPUDirect Support	Energy Cost (E_{link})
PCIe Gen3 (x16)	15.75 GB/s dir/31.5 GB/s agg	400–500 ns	128b/130b NRZ	PCIe P2P, GPUDirect RDMA	14.0 pJ/bit
PCIe Gen4 (x16)	31.50 GB/s dir/63.0 GB/s agg	250–400 ns	128b/130b NRZ	PCIe P2P, GPUDirect RDMA	10.0 pJ/bit
PCIe Gen5 (x16)	63.00 GB/s dir/126.0 GB/s agg	150–250 ns	128b/130b NRZ	PCIe P2P, GPUDirect RDMA, CXL 1.1/2.0	8.0 pJ/bit
PCIe Gen6 (x16)	126.00 GB/s dir/252.0 GB/s agg	100–150 ns	PAM4 (256 B flit)	PCIe P2P, GPUDirect RDMA, CXL 3.0/3.1	6.0 pJ/bit
PCIe Gen7 (x16)	252.00 GB/s dir/504.0 GB/s agg	< 100 ns	PAM4 (512 B flit)	PCIe P2P, GPUDirect RDMA, CXL 3.1+	4.5 pJ/bit
NVLink 1 (P100)	20 GB/s dir/40 GB/s agg (link)	200–300 ns	NRZ (20 Gb/s)	GPUDirect P2P/RDMA	10.0 pJ/bit
NVLink 2 (V100)	25 GB/s dir/50 GB/s agg (link)	150–200 ns	NRZ (25.78 Gb/s)	GPUDirect P2P/RDMA	8.0 pJ/bit
NVLink 3 (A100)	25 GB/s dir/50 GB/s agg (link)	100–150 ns	NRZ (50 Gb/s)	GPUDirect P2P/RDMA	6.5 pJ/bit
NVLink 4 (H100)	25 GB/s dir/50 GB/s agg (link)	80–100 ns	PAM4 (100 Gb/s)	GPUDirect P2P, SHARP Aggregation	4.5 pJ/bit
NVLink 5 (B200)	50 GB/s dir/100 GB/s agg (link)	60–80 ns	PAM4 (200 Gb/s)	GPUDirect P2P, NVLink Network Offload	3.5 pJ/bit
NVSwitch 1 (Volta)	900 GB/s aggregate per chip	~100 ns	NRZ	Hardware P2P Crossbar Routing	8.5 pJ/bit
NVSwitch 2 (Ampere)	2.4 TB/s aggregate per chip	~90 ns	NRZ	Hardware P2P Crossbar, SHARP v2	6.5 pJ/bit
NVSwitch 3 (Hopper)	3.2 TB/s aggregate per chip	~70 ns	PAM4	SHARP v3 In-Network Compute	4.5 pJ/bit
NVSwitch 4 (Blackwell)	14.4 TB/s aggregate per chip	~50 ns	PAM4	SHARP v4, FP8 Reduction Engine	3.2 pJ/bit
InfiniBand EDR	100 Gbps (12.5 GB/s dir)	~0.50 µs (500 ns)	64b/66b NRZ	GPUDirect RDMA (Verbs)	15.0 pJ/bit
InfiniBand HDR	200 Gbps (25.0 GB/s dir)	~0.60 µs (600 ns)	64b/66b PAM4/NRZ	GPUDirect RDMA, SHARP v2	10.0 pJ/bit
InfiniBand NDR	400 Gbps (50.0 GB/s dir)	~0.50 µs (500 ns)	256b/257b PAM4	GPUDirect RDMA, SHARP v3	7.0 pJ/bit
InfiniBand XDR	800 Gbps (100.0 GB/s dir)	~0.40 µs (400 ns)	PAM4 (200G/lane Flit)	GPUDirect RDMA, SHARP v4	5.0 pJ/bit
RoCE v2 Ethernet	100–800 Gbps (12.5–100 GB/s dir)	1.0–2.5 µs	NRZ/PAM4 (802.3)	GPUDirect RDMA (RoCEv2 UDP/IP)	8.0–12.0 pJ/bit
CXL 1.1	63 GB/s dir/126 GB/s agg (x16)	~200 ns	128b/130b NRZ	cxl.io, cxl.cache, cxl.mem	8.0 pJ/bit
CXL 2.0	63 GB/s dir/126 GB/s agg (x16)	~180 ns	128b/130b NRZ	SLD/MLD Memory Pooling	7.5 pJ/bit
CXL 3.0	126 GB/s dir/252 GB/s agg (x16)	~120 ns	256 B flit PAM4	P2P Memory Pooling & Fabric Switch	5.5 pJ/bit
CXL 3.1	126 GB/s dir/252 GB/s agg (x16)	~110 ns	256 B flit PAM4	Global Integrated Memory, TEE	5.0 pJ/bit

Economic Constants

These **pricing assumptions** underpin total cost of ownership (TCO) and energy-cost napkin math in table E.16. They are illustrative hyperscaler-order rates for ratio analysis (similar in spirit to carbon

accounting examples in (Patterson et al. 2021)), not quotes for a specific region or contract. Local values should replace these anchors when absolute price dominates.

Table E.16: Economic Assumptions: Illustrative cloud electricity and egress rates for TCO napkin math (2024–2025 order of magnitude). On-premise cost structures differ; relative magnitudes guide design trade-offs.

Assumption	Value	Unit
Cloud electricity price	0.12	dollar/kWh
Cloud egress price per GB	0.09	dollar/GB

Scale References

Economic constants set the price per unit of compute and data transfer, but they mean little without a sense of the volumes involved. Production ML systems handle millions to billions of requests per day—numbers large enough to be difficult to internalize without concrete reference points. These **scale assumptions** in table E.17 anchor practical orders of magnitude for capacity-planning examples, providing illustrative order-of-magnitude baselines for email and search volume, autonomous-vehicle sensor streams, and standard 1080p and 4K video formats. They are magnitude anchors, not audited statistics for a specific year.

Table E.17: Production Scale and Data Rate References: Illustrative workload and sensor-rate anchors, plus standard 1080p and UHD 4K video parameters.

Assumption	Value	Unit
Gmail emails per day	1.21e+11	-
Google searches per day	8.5e+09	-
Waymo sensor data rate (low)	1	TB/h
Waymo sensor data rate (high)	19	TB/h
1080p frame width	1920	-
1080p frame height	1080	-
4K frame width	3840	-
4K frame height	2160	-
Bytes per RGB pixel	3	bytes
Video frame rate (standard)	30	Hz

Unit Conventions

The assumptions above use shared unit conventions: decimal data prefixes, distinct FLOPs and FLOP/s quantities, and the aliases defined below. Table E.18 fixes the unit conventions used in every quantitative example in this book. Each row gives the multiplier k in `1 alias` $= k$ `base`. Data prefixes use decimal SI (KB $= 10^3$ bytes, not 1024). Binary IEC prefixes do not appear in the book's prose. Throughput quantities (FLOP/s, GB/s) combine these aliases with time; adding incompatible dimensions (bytes to FLOP/s) is a category error in napkin math, not a unit conversion.

Hardware capacity labels are the one deliberate exception. The reference tables retain vendor nameplate numbers and labels; the byte budgets used in calculations follow the device assumptions stored in the book's hardware registry. For example, the book models an 80 GB accelerator with 80×2^{30} bytes and assigns 512×2^{10} bytes to a microcontroller with 512 KB of SRAM. These are modeled capacity assumptions, not universal guarantees inferred from the labels. Every other data quantity, including model footprints, activation sizes, and transfer volumes, uses the decimal prefixes above. Where a worked example subtracts a computed footprint from a device capacity, the stored capacity byte budget is converted to decimal units alongside the footprint so that the arithmetic printed on the page is the arithmetic a reader can reproduce.

Table E.18: Unit Conventions: Decimal SI aliases and scale factors (book convention for napkin math). Throughput forms such as FLOP/s and GB/s divide work or data by time using the same conventions.

Alias	Multiplier	Base unit
`byte`	1	byte
`KB`	1000	byte
`MB`	1e+06	byte
`GB`	1e+09	byte
`TB`	1e+12	byte
`PB`	1e+15	byte
`flop`	1	flop
`GFLOPs`	1e+09	flop
`TFLOPs`	1e+12	flop
`ZFLOPs`	1e+21	flop
`param`	1	param
`Mparam`	1e+06	param
`Gbps`	1e+09	bit/s
`NS`	10^{-9}	second
`US`	10^{-6}	second
`MS`	10^{-3}	second
`second`	1	second
`hour`	3600	second
`day`	86400	second
`joule`	1	joule
`watt`	1	watt
`meter`	1	meter

E.1 Assumption Provenance

Each assumption has a provenance record that lets readers trace the number back to its primary source. Table E.19 groups the appendix's quantitative inputs by evidence class and identifies the primary references behind each group. Vendor specifications and standards define published limits, peer-reviewed measurements provide empirical anchors, and explicitly marked editorial scenarios supply illustrative values where no durable universal constant exists. This distinction lets readers judge the evidentiary weight of an input, update time-sensitive values, and reproduce an estimate with an appropriate local substitute.

Table E.19: Assumption Provenance Catalog: Quick map from appendix section to source class and bibliography.

Appendix section	Source type	Primary references
Accelerator specifications	Vendor datasheet peaks; editorial CPU/mobile anchors; Master Spec Matrix (table E.1)	(NVIDIA Corporation 2017, 2018, 2020a, 2024; Choquette et al. 2021; Choquette 2023; AMD 2023; Jouppi et al. 2023; Google Cloud 2026b); editorial for CPU/mobile rows
Model specifications	Published papers, model reports, and official docs; GPT-3 run reconstruction; GPT-4 size from public analysis	(Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Patterson et al. 2021; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023; OpenAI et al. 2023; Patel and Wong 2023)
Training memory conventions	Book convention (mixed-precision Adam layout)	(Kingma and Ba 2015; Kalamkar et al. 2019; Micikevicius et al. 2017; NVIDIA 2017)
Energy constants	Published 45 nm arithmetic/DRAM estimates; VLSI synthesis node scaling; Precision Energy Hierarchy (table E.13)	(Horowitz 2014; Dally et al. 2021) for arithmetic/DRAM; editorial for register/SRAM, MobileNetV2, and 5G rows

Appendix section	Source type	Primary references
Interconnect bandwidth	Vendor specs; InfiniBand standard; PCIe/NVLink/CXL standards; Physical Spec Sheet (table E.15)	(NVIDIA Corporation 2017, 2020a; Choquette 2023; InfiniBand Trade Association 2000; NVIDIA 2026f; Ethernet Alliance 2025)
Economic assumptions	Illustrative cloud/utility rates	(Patterson et al. 2021) (methodology context)
Scale references	Illustrative workload rates; standard video conventions	Editorial for workload rates; standard format definitions for video
Unit conventions	Decimal SI; book notation	Editorial

Summary

This appendix is the reference catalog for the quantitative inputs used throughout the book. It gathers accelerator and model specifications, training-memory conventions, energy constants, interconnect bandwidths, economic and scale references, unit conventions, and provenance. The values come from vendor peaks, published measurements, standards, and editorial scenarios, so they carry different evidentiary weight and age at different rates.

Use these tables as starting inputs, not universal constants. For each estimate, record the table row, unit convention, and provenance; replace time-sensitive peaks, prices, and rates with local values; distinguish a physical ceiling from a measured operating point and a scenario assumption; and report which assumptions dominate the result. This keeps napkin math reproducible and makes clear which input controls the conclusion.

Glossary

This glossary defines key terms used throughout this book and gathers them as a compact reference. Terms are organized alphabetically so they can be consulted alongside the chapters.

3

3DMark Graphics performance benchmark suite that evaluates real-time 3D rendering capabilities, measuring triangle throughput, texture fill rates, and modern features like ray tracing and DLSS performance.

A

A/B testing A controlled experimental method for comparing two versions of a system or model by randomly dividing users into groups and measuring performance differences between the variants.

ablation study An experiment that removes, disables, or isolates one component at a time to measure its contribution to model or system behavior.

activation-based pruning A pruning method that evaluates the average activation values of neurons or filters over a dataset to identify and remove neurons that consistently produce low activations and contribute little information to the network's decision process.

activation checkpointing A memory optimization technique that reduces memory usage during backpropagation by selectively discarding and recomputing activations instead of storing all intermediate results.

activation function A mathematical function applied to the weighted sum of inputs in a neural network neuron to introduce nonlinearity, enabling the network to learn complex patterns beyond simple linear combinations.

activation memory Memory used to store intermediate layer outputs needed for gradient computation during training, scaling with batch size, sequence length, and model depth.

active learning An approach that intelligently selects the most informative examples for human annotation based on model uncertainty, reducing the amount of labeled data needed for effective training.

Adam optimization An adaptive learning rate optimization algorithm that combines momentum and RMSprop by maintaining exponentially decaying averages of both gradients and squared gradients for each parameter.

AdamW A variant of Adam that decouples weight decay from the adaptive gradient update, improving regularization while keeping the optimizer's adaptive moments.

alerting Automated notification systems that inform teams when metrics exceed predefined thresholds or anomalies are detected in production ML systems.

AlexNet A landmark convolutional neural network architecture that won the 2012 ImageNet challenge, reducing error rates from 26 percent to 16 percent and sparking the deep learning renaissance.

AllReduce A collective communication operation in distributed computing where each process contributes data and all processes receive the combined result, commonly used for gradient aggregation in distributed training.

all-to-all communication A communication pattern where every device or process exchanges data with every other device, common in sharded embedding and expert-parallel workloads.

AlphaFold A landmark AI system developed by DeepMind that predicts the three-dimensional structure of many proteins from their amino acid sequences, achieving breakthrough accuracy without solving every folding problem and demonstrating how large-scale ML systems can accelerate scientific discovery. Its predictions still require confidence interpretation and, for many uses, experimental validation.

Amdahl's Law A speedup bound showing that overall improvement is limited by the portion of a workload or pipeline that remains serial or unoptimized.

AOT compilation Ahead-of-time compilation that converts a model, graph, or program into optimized executable code before deployment or execution, trading flexibility for lower runtime overhead.

Apache Kafka A distributed streaming platform that handles real-time data feeds using a publish-subscribe messaging system, commonly used for building ML data pipelines with high throughput and fault tolerance.

Apache Spark An open-source distributed computing framework that enables large-scale data processing across clusters of computers, revolutionizing ETL operations with in-memory computing capabilities.

application-specific integrated circuit (ASIC) Custom chips designed for specific computational tasks that offer superior performance and energy efficiency compared to general-purpose processors by abandoning general-purpose flexibility. Examples include Google's TPUs, Cerebras Wafer-Scale Engine, and Bitcoin mining ASICs.

arithmetic intensity The ratio of floating-point operations to bytes of memory accessed (FLOP/byte), used in roofline analysis to determine whether workloads are memory bound or compute bound and to guide optimization priorities.

artificial general intelligence AI systems intended to match human-level performance across a broad range of cognitive tasks; the term has no universally accepted capability threshold and implies no particular architecture.

artificial intelligence The field of computer science focused on creating systems that can perform tasks typically requiring human intelligence, such as perception, reasoning, learning, and decision-making.

artificial neural network A computational model inspired by biological neural networks, consisting of interconnected nodes (neurons) organized in layers that can learn patterns from data through adjustable weights and biases.

artificial neurons Basic computational units in neural networks that mimic biological neurons, taking multiple inputs, applying weights and biases, and producing an output signal through an activation function.

attention mechanism A neural network component that computes weighted connections between elements based on their content, allowing dynamic focus on relevant parts of the input rather than fixed architectural connections.

AUC Area under the ROC curve, a threshold-independent classification metric that compares true positive and false positive rates across decision thresholds.

audit trail An append-only record of who accessed or changed data, models, or system artifacts and when, supporting accountability, debugging, and compliance.

autograd tape A transient record of operations and saved values built during forward execution so reverse-mode automatic differentiation can compute gradients during the backward pass.

automatic differentiation A computational technique that automatically calculates exact derivatives of functions implemented as computer programs by systematically applying the chain rule at the elementary operation level, essential for training neural networks through gradient-based optimization.

automatic mixed precision A training technique that automatically manages the use of different numerical precisions (FP16, FP32) to optimize memory usage and computational speed while maintaining model accuracy.

AutoML Automated machine learning that automates model design decisions, including architecture search, hyperparameter optimization, and feature selection, to reduce manual effort.

autoscaling Dynamic adjustment of compute resources based on workload demand, auto-

matically scaling up during peak usage and scaling down during low usage to optimize costs and performance.

B

backpropagation An algorithm that computes gradients of the loss function with respect to network weights by propagating error signals backward through the network layers, enabling systematic weight updates during training.

backward pass The training phase that propagates gradients from the loss back through the network to compute parameter gradients, pairing with the forward pass.

bandwidth The maximum rate of data transfer across a communication channel or memory interface, typically measured in bytes per second and critical for optimizing data movement in AI accelerators.

batch inference The process of running inference on a large dataset in bulk, typically as a scheduled job, as opposed to real-time inference on individual requests. Batch inference enables higher throughput by amortizing overhead across many inputs.

batch ingestion A data processing pattern that collects and processes data in groups or batches at scheduled intervals, suitable for scenarios where real-time processing is not critical.

batch normalization A technique that normalizes inputs to each layer to have zero mean and unit variance, which stabilizes training and often allows for higher learning rates and faster convergence, and subsequently applies a learnable scale and shift to preserve representational power.

batch processing The technique of processing multiple data samples simultaneously to amortize computation and memory access costs, improving overall throughput in neural network training and inference.

batch size The number of training examples processed simultaneously during one iteration of neural network training, affecting both computational efficiency and gradient estimation quality.

batch throughput optimization Techniques for maximizing the number of samples processed per unit time when handling multiple inputs simultaneously, using parallelism and batching efficiencies.

batched operations Matrix computations that process multiple inputs simultaneously, converting matrix-vector operations into more efficient matrix-matrix operations to improve hardware utilization.

benchmark engineering The systematic design and development of performance evaluation frameworks, involving test harness creation, metric selection, and result interpretation methodologies.

benchmark harness Systematic infrastructure component that controls test execution, manages input delivery, and collects performance measurements under controlled conditions to ensure reproducible evaluations.

benchmarking Systematic evaluation of compute performance, algorithmic effectiveness, and data quality in machine learning systems to optimize performance across diverse workloads and ensure reproducibility.

BERT Bidirectional Encoder Representations from Transformers, a transformer-based language model introduced by Google in 2018 that revolutionized natural language processing through masked language modeling pretraining.

BF16 A 16-bit floating-point format developed by Google Brain that maintains the same dynamic range as FP32 but with reduced precision, making it particularly suitable for deep learning training.

bias A learnable parameter added to the weighted sum in each neuron that shifts the activation function, allowing neurons to activate even when all inputs are zero and providing additional flexibility for the network to fit complex patterns.

binarization An extreme quantization technique that reduces neural network weights, activations, or both to binary values (typically -1 and +1), achieving maximum compression but often requiring specialized training procedures and hardware support.

biological neuron A cell in the nervous system that receives, processes, and transmits information through electrical and chemical signals, serving as inspiration for artificial neural networks.

bitter lesson Richard Sutton's 2019 observation that general methods using computation consistently outperform approaches encoding human expertise, suggesting that systems engineering enabling computational scale is central to AI advancement.

black box A system whose inputs and outputs are observable but whose internal workings are inaccessible or opaque. This opacity is particularly problematic when the system makes consequential decisions without explaining its reasoning.

BLAS Basic Linear Algebra Subprograms, a specification for low-level routines that perform common linear algebra operations such as vector addition, scalar multiplication, dot products, and matrix operations, forming the computational foundation of modern ML frameworks.

block sparse matrix A sparse matrix whose nonzero values appear in dense blocks rather than isolated elements, preserving structure that hardware kernels can exploit efficiently.

blue-green deployment A deployment strategy using two production environments so traffic can switch to a validated new version while the old version remains available for rollback.

bounding box A rectangular annotation that identifies object locations in images by drawing a box around each object of interest, commonly used in computer vision training datasets.

brittleness The tendency of rule-based AI systems to fail completely when encountering inputs that fall outside their programmed scenarios, no matter how similar those inputs might be to what they were designed to handle.

C

caching A technique for storing frequently accessed data in high-speed storage systems to reduce retrieval latency and improve system performance in ML pipelines.

caching allocator A framework memory manager that reuses device-memory blocks instead of calling the hardware allocator for every tensor, reducing allocation latency and fragmentation.

calibration The process in post-training quantization of analyzing a representative dataset to determine optimal quantization parameters, including scale factors and zero points, that minimize accuracy loss when converting from high to low precision.

canary deployment A deployment strategy where new model versions receive a small percentage of production traffic to validate behavior before full rollout, enabling early detection of issues with minimal user impact.

CAP theorem A distributed systems principle stating that, during a network partition, a data store cannot guarantee both linearizable consistency and availability for every request.

carbon footprint The total greenhouse gas emissions, typically measured in CO_2 equivalent, produced directly and indirectly by training and operating an ML system.

carbon intensity The emissions produced per unit of energy, usually measured as kg CO_2e per kWh, used to convert ML energy use into carbon impact.

Cerebras Wafer-Scale Engine In the CS-2 generation, a single-wafer processor containing 2.6 trillion transistors and 850,000 cores, designed to eliminate inter-device communication bottlenecks in large-scale machine learning training.

chain rule The calculus rule enabling computation of derivatives for composite functions, fundamental to backpropagation.

channelwise quantization A quantization granularity approach where each channel in a layer uses its own set of quantization parameters, providing more precise representation than layerwise quantization while maintaining hardware efficiency.

CI/CD pipelines Continuous Integration and Continuous Delivery automated workflows that streamline model development by integrating testing, validation, and deployment processes.

circuit breaker pattern A resilience pattern that stops or redirects calls to an unhealthy service after repeated failures to prevent cascading failure and enable fallback behavior.

classification labels Simple categorical annotations that assign specific tags or categories to data examples, representing the most basic form of supervised learning annotation.

CloudSuite Benchmark suite developed at EPFL that addresses modern data center workloads including web search, data analytics, and media streaming, measuring end-to-end performance across network, storage, and compute dimensions.

cold-start performance Time required for a system to transition from idle state to active execution, particularly important in serverless environments where models are loaded on demand.

collaborative filtering A technique used in recommendation systems that predicts user preferences by identifying patterns in interactions from many users, using the collective behavior of the crowd rather than just item properties.

communication tax The performance penalty incurred in distributed systems due to the latency and bandwidth costs of synchronizing state between nodes.

compound AI systems AI architectures that combine multiple specialized models and components working together, rather than relying on a single monolithic model, enabling modularity, specialization, and improved interpretability.

compressed sparse row (CSR) A memory-efficient storage format for sparse matrices that uses three arrays (values, column indices, row pointers) to store only nonzero elements, reducing memory from $\mathcal{O}(n_{\text{rows}}^2)$ (for a square matrix with n_{rows} rows) to $\mathcal{O}(n_{\text{nz}} + n_{\text{rows}})$ where n_{nz} is the number of nonzeros.

computational graph A directed graph representing the sequence of operations in neural network computation, enabling automatic differentiation.

computer engineering An engineering discipline that emerged in the late 1960s to address the growing complexity of integrating hardware and software systems, combining expertise from electrical engineering and computer science to design and build complex computing systems.

concept drift The phenomenon where $p(y \mid x)$, the statistical relationship between inputs and targets, changes over time, distinct from data drift in $p(x)$; this can, but need not, degrade a fixed model's performance. Detecting it typically requires outcome labels because input drift alone does not establish a conditional change.

conditional computation A dynamic optimization technique where different parts of a neural network are selectively activated based on input characteristics, reducing computational load by skipping unnecessary computations for specific inputs.

consensus labeling A quality control approach that collects multiple annotations for the same data point to identify controversial cases and improve label reliability through inter-annotator agreement.

containerization Packaging applications and their dependencies into portable, isolated containers using tools like Docker to ensure consistent execution across different environments.

continuous batching An LLM serving technique that adds and removes requests from an inference batch between token-generation steps as sequences start and finish.

continuous integration A software development practice where code changes are automatically integrated, tested, and validated multiple times per day to detect issues early in the development cycle.

convolution A mathematical operation that slides a filter (kernel) across input data to extract features such as edges, textures, or patterns. Convolution is fundamental to convolutional neural networks and particularly effective for processing images and spatial data.

convolutional neural network A specialized neural network architecture designed for processing grid-like data such as images, using convolutional layers that apply filters to detect local features.

coreset A small subset of a dataset selected to preserve important statistical or training properties of the full dataset while reducing training or evaluation cost.

correction cascade An ML technical-debt pattern where fixing one component creates new failures or redesign needs elsewhere because data and model dependencies propagate through the system.

covariate shift A distribution shift where input feature distributions change while the relationship between features and labels remains stable.

CP decomposition CANDECOMP/PARAFAC decomposition that expresses a tensor as a sum of rank-one components, used to compress neural network layers by reducing the number of parameters while preserving computational functionality.

credit assignment problem The challenge of determining which weights in a multi-layer network contributed to prediction errors.

CRISP-DM Cross-Industry Standard Process for Data Mining, a structured methodology developed in 1996 that defines six phases for data projects: business understanding, data understanding, data preparation, modeling, evaluation, and deployment.

critical batch size The batch size beyond which larger batches provide diminishing optimization benefit and may reduce generalization, limiting simple linear learning-rate scaling.

cross-entropy loss A loss function commonly used in classification tasks that measures the difference between predicted probability distributions and true class labels, providing strong gradients for effective learning.

crowdsourcing A collaborative data collection approach that uses distributed individuals via the internet to perform annotation tasks, enabling scalable dataset creation through platforms like Amazon Mechanical Turk.

cuBLAS NVIDIA's CUDA Basic Linear Algebra Subprograms library that provides GPU-accelerated implementations of standard linear algebra operations, enabling high-performance matrix computations on NVIDIA graphics processing units.

CUDA (Compute Unified Device Architecture) NVIDIA's parallel computing platform and programming model that enables general-purpose computing on graphics processing units (GPUs), allowing machine learning frameworks to use massive parallelism for accelerated tensor operations.

CUDA stream An ordered queue of GPU work used to schedule kernels, memory transfers, and synchronization events so independent operations can overlap.

D

D·A·M taxonomy A diagnostic framework that analyzes ML system bottlenecks through three interacting axes: Data (information flow, bounded by bandwidth), Algorithm (mathematical logic, bounded by total operations), and Machine (physical execution, bounded by peak throughput).

Dartmouth conference The legendary 8-week workshop at Dartmouth College in 1956 where AI was officially born, organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon, following their 1955 proposal, which introduced the term artificial intelligence.

data as source code The Software 2.0 principle that training data defines ML system behavior, so dataset changes act like behavior-changing code changes.

data augmentation The process of artificially expanding training datasets by creating modified versions of existing data through transformations like rotation, scaling, or noise injection.

data cascades Systemic failures where data quality issues compound over time, creating downstream negative consequences such as model failures, costly rebuilding, or project termination.

data center A facility that houses computer systems and associated components such as telecommunications and storage systems, typically containing thousands of servers for cloud computing operations.

data-centric approach A machine learning paradigm that prioritizes improving data quality, diversity, and curation rather than solely focusing on model architecture improvements to achieve better performance.

data-centric computing Systems optimized for the efficient ingestion of data and iterative refinement of model parameters, where the programmer's job is to curate data.

data contracts Explicit agreements between data producers and consumers defining schema, quality expectations, and service levels to prevent downstream breakage.

data curation The process of selecting, organizing, and maintaining high-quality datasets by removing irrelevant information, correcting errors, and ensuring data meets specific standards for machine learning applications.

data debt Accumulated data quality, documentation, schema, lineage, or freshness liabilities that compound over time and degrade model behavior.

data drift The phenomenon where the input distribution $p(x)$ changes over time even though code remains unchanged; this may affect model performance but does not guarantee degradation. It can be monitored without labels, although labels are needed to measure its effect on quality.

data echoing A training input-pipeline technique that reuses batches, often with fresh augmentation, when data preparation is slower than accelerator consumption.

data governance The framework of policies, procedures, and technologies that ensure data security, privacy, compliance, and ethical use throughout the machine learning pipeline.

data gravity The architectural pull created by large data volumes, transfer time, bandwidth limits, and egress cost, which tends to draw computation toward the data.

data ingestion The process of collecting and importing raw data from various sources into a system where it can be stored, processed, and prepared for machine learning applications.

data lake A storage repository that holds structured, semi-structured, and unstructured data in its native format, using schema-on-read approaches for flexible data analysis.

data lakehouse A storage architecture that combines data lake flexibility with warehouse-style query semantics, schema management, and transactional guarantees.

data lineage The documentation and tracking of data flow through various transformations and processes, providing visibility into data origins and modifications for compliance and debugging.

data locality The principle that data and computation should be colocated when transfer latency, bandwidth, or cost dominates remote processing.

data parallelism A distributed training strategy that splits the dataset across multiple devices while each device maintains a complete copy of the model, enabling parallel computation of gradients.

data pipeline The infrastructure and workflows that automate the movement and transformation of data from sources through processing stages to final storage or consumption.

data quality The degree to which data meets requirements for accuracy, completeness, consistency, and timeliness, directly impacting machine learning model performance.

data quality multiplier The idea that improvements in data quality can amplify gains from model and system changes because all downstream stages depend on the data.

data validation The systematic verification that collected data meets quality standards, is properly formatted, and contains accurate information suitable for machine learning model training and evaluation.

data versioning The practice of tracking and managing different versions of datasets over time, similar to code versioning, to ensure reproducibility and enable rollback to previous data states when needed.

data warehouse A centralized repository optimized for analytical queries (OLAP) that stores integrated, structured data from multiple sources in a standardized schema.

dataflow architecture Specialized computing architecture where instruction execution is determined by data availability rather than a program counter, enabling highly parallel processing of neural network operations.

dataflow challenges Technical difficulties in managing data movement and dependencies in hardware accelerators, including memory bandwidth limitations and synchronization requirements.

datasheets for datasets Documentation for training data that captures provenance, collection methodology, demographic composition, and known limitations affecting model behavior.

dead letter queue A separate storage mechanism for data that fails processing, allowing for later analysis and potential reprocessing of problematic data without blocking the main pipeline.

deduplication The removal of exact, near-duplicate, or semantically redundant samples from a dataset to reduce storage, training compute, and leakage risk.

deep learning A subfield of machine learning that uses artificial neural networks with multiple layers to automatically learn hierarchical representations from data without explicit feature engineering.

demographic parity A fairness criterion requiring that the probability of receiving a positive prediction is independent of group membership across protected attributes.

Dennard scaling The observation that as transistors became smaller, their power density remained constant, enabling higher performance at the same power envelope. Its breakdown around 2005 ended the era of free frequency scaling.

dense layer A fully-connected neural network layer where each neuron receives input from all neurons in the previous layer, enabling comprehensive information integration across features.

deployment constraints Operational limitations such as hardware resources, network connectivity, regulatory requirements, and integration requirements that influence how machine learning models are implemented in production environments.

deployment paradigm A distinct approach to hosting and executing ML models characterized by specific resource constraints and operational properties, such as Cloud ML, Edge ML, Mobile ML, or TinyML.

depthwise separable convolution A convolution factorization that applies spatial filtering independently per channel and then mixes channels with a pointwise convolution to reduce computation.

DevOps Software development practice that combines development and operations teams to shorten development cycles and deliver high-quality software through automation and collaboration.

Dhrystone Integer-based benchmark introduced in 1984 that measures integer and string operations in DMIPS (Dhrystone MIPS), designed to complement floating-point benchmarks with typical programming constructs.

diabetic retinopathy A diabetes complication that damages blood vessels in the retina, serving as a leading cause of preventable blindness and a key application area for medical AI screening systems.

differential privacy A mathematical framework for quantifying and limiting privacy loss when releasing statistical information about datasets by bounding how much the output distribution can change when any one individual's data is added or removed.

direct memory access (DMA) A mechanism that lets a device move data to or from memory without CPU-managed copying for each transfer, enabling overlap and reducing host overhead.

disaggregated evaluation The practice of breaking down model performance metrics by demographic groups or other factors to reveal disparities that are hidden by aggregate measures.

dispatch tax Runtime overhead from launching and orchestrating many small operations through the host framework instead of executing fused or compiled graph regions.

distributed computing An approach that processes data across multiple machines or processors simultaneously, enabling scalable handling of large datasets through frameworks like Apache Spark.

distributed intelligence The placement of computational capabilities across multiple devices and locations rather than relying on a single centralized system, enabling local processing and decision-making.

distributed training A method of training machine learning models across multiple machines or devices to handle larger datasets and models that exceed single-device computational or memory capacity.

distribution shift A change in the statistical properties of data between training and deployment, or over time during deployment. Types include covariate shift (input distribution changes), label shift (output distribution changes), and concept drift (relationship between inputs and outputs changes).

DLRM Deep Learning Recommendation Model, an architecture developed by Meta that combines categorical embeddings with a bottom multi-layer perceptron (MLP) and top MLP to handle the massive scale and sparsity of recommendation system workloads.

domain-specific architecture Hardware designs tailored to optimize specific computational workloads, trading flexibility for improved performance and energy efficiency compared to general-purpose processors.

dropout A regularization technique that randomly sets a fraction of input units to zero during training to prevent overfitting and improve generalization.

dying ReLU problem A failure mode where ReLU neurons become permanently inactive and output zero for all inputs, preventing them from contributing to learning when weighted inputs consistently produce negative preactivations.

dynamic batching A serving technique that waits briefly to group independent requests into a batch, improving throughput while adding controlled batch-formation latency.

dynamic graph A computational graph that is built and modified during program execution, allowing for flexible model architectures and easier debugging but potentially limiting optimization opportunities compared to static graphs.

dynamic pruning A pruning approach that changes which parameters or units are inactive in response to input data or training dynamics, rather than relying only on a fixed pruning decision.

dynamic quantization The process of storing neural-network weights at lower precision and determining activation quantization parameters at runtime, reducing memory use while requiring supported low-precision kernels for speedup.

dynamic random-access memory (DRAM) A type of volatile memory that stores data in capacitors and requires periodic refresh cycles, commonly used as main memory in computer systems.

dynamic voltage and frequency scaling (DVFS) Power management technique that adjusts processor voltage and clock frequency based on workload demands to optimize energy consumption while maintaining performance.

E

eager execution An execution mode where operations are evaluated immediately as they are called in the code, providing intuitive debugging and development experience but potentially sacrificing some optimization opportunities available in graph-based execution.

early exit architectures Neural network designs that include multiple prediction heads at different depths, allowing samples to exit early when confident predictions can be made, reducing average computational cost per inference.

edge computing A distributed computing paradigm that brings computation and data storage closer to the sources of data, reducing latency and bandwidth usage.

edge deployment A deployment strategy where machine learning models run locally on devices at the network edge rather than in centralized cloud servers, reducing latency and enabling operation without constant internet connectivity.

efficiency frontier The optimal trade-off curve between accuracy and computational cost (latency, energy, or memory), where no model exists that is both more accurate and more efficient.

EfficientNet A family of neural network architectures discovered through neural architecture search that achieves better accuracy-efficiency trade-offs by using compound scaling to balance network depth, width, and input resolution.

EL2N (Error L2-Norm) A sample-scoring method that ranks examples by the L2 norm of prediction error early in training, often using a proxy model, to guide data selection.

ELIZA One of the first chatbots created by MIT's Joseph Weizenbaum in 1966 that could simulate human conversation through pattern matching and substitution, notable because people began forming emotional attachments to this simple program.

embedded systems Computer systems with dedicated functions within larger mechanical or electrical systems, typically designed for specific tasks with real-time computing constraints.

embedding sharding Splitting large embedding tables across devices or nodes so they fit in aggregate memory, while adding communication to gather embeddings for each batch.

embedding table A lookup table that maps discrete IDs or tokens to dense vectors, often dominating memory in recommendation models and large vocabularies.

emergent behaviors Unexpected system-wide patterns or characteristics that arise from the interaction of individual components, often becoming apparent only when systems operate at scale or in real-world conditions.

encoder-decoder An architectural pattern where an encoder processes input into an intermediate representation and a decoder

generates output from this representation, commonly used in sequence-to-sequence tasks.

end-to-end benchmarks Comprehensive evaluation methodology that assesses entire AI system pipelines including data processing, model execution, postprocessing, and infrastructure components.

energy efficiency The measure of computational work performed per unit of energy consumed, typically expressed as operations per joule and crucial for battery-powered and data center deployments.

energy per inference The total energy consumed to produce one model prediction, including model execution and relevant system overhead.

epoch One complete pass through the entire training dataset during neural network training, consisting of multiple batch iterations depending on dataset size and batch size.

equal opportunity A fairness criterion requiring equal true positive rates across different demographic groups.

equalized odds A fairness criterion requiring that both true positive and false positive rates are equal across different demographic groups.

expected calibration error (ECE) A metric measuring the gap between predicted confidence and observed accuracy across confidence bins, used to assess probability calibration.

experiment tracking The systematic recording and management of machine learning experiments, including hyperparameters, model versions, training data, and performance metrics, to enable comparison and reproducibility.

expert systems AI systems first developed in the mid-1960s that captured human expert knowledge in specific domains, exemplified by MYCIN for diagnosing blood infections, representing a shift from general AI to domain-specific applications.

extract, load, transform (ELT) A data processing paradigm that first loads raw data into the target system before applying transformations, providing flexibility for evolving analytical needs.

extract, transform, load (ETL) A traditional data processing paradigm that transforms data before loading it into a data warehouse, resulting in ready-to-query formatted data.

F

fairness laundering A failure mode where removing explicit protected attributes makes a system appear compliant while correlated proxy variables still reproduce discriminatory outcomes.

false positive rate (FPR) The proportion of actual negative cases incorrectly classified as positive, computed as false positives divided by false positives plus true negatives.

FarmBeats A Microsoft Research project that applies machine learning and IoT technologies to agriculture, using edge computing to collect real-time data on soil conditions and crop health while demonstrating distributed AI systems in challenging real-world environments.

feature engineering The process of manually designing and extracting relevant features from raw data to improve machine learning model performance, largely automated in deep learning systems.

feature map The output of a convolutional layer representing the response of learned filters to different spatial locations in the input, capturing detected features at various positions.

feature store A specialized data storage system that provides standardized, reusable features for machine learning, enabling feature sharing across multiple models and teams.

federated learning A machine learning approach that trains algorithms across decentralized edge devices or servers holding local data samples, without exchanging the raw data.

feedback loop A cyclical process where outputs influence future inputs. In ML, this includes both beneficial cycles (where model outputs inform system improvement) and harmful ones (where a model's predictions influence its own future training data, potentially reinforcing and amplifying initial biases over time).

feedforward network A neural network architecture where information flows in one direction from input to output layers without cycles, forming the foundation for many deep learning models.

field-programmable gate array (FPGA) A reconfigurable integrated circuit that can be programmed after manufacturing to implement custom digital circuits and specialized computations, offering flexibility between general-purpose processors and ASICs.

fine-tuning Adapting a pretrained model to a target task or domain by continuing training on task-specific data.

FlashAttention An IO-aware attention algorithm that tiles computation to avoid materializing the full attention matrix in high-bandwidth memory, reducing memory traffic and enabling longer sequences.

floating-point unit (FPU) A specialized processor component designed to perform arithmetic operations on floating-point numbers with high precision and efficiency.

FLOP/s Floating-Point Operations per Second, a measure of computational throughput that quantifies the number of floating-point operations a system can perform per second.

forgetting events Events where a model changes from classifying an example correctly to misclassifying it later in training, marking difficult or informative samples.

forward pass The process of computing neural network predictions by passing input data through successive layers, applying weights, biases, and activation functions at each stage to produce outputs. Also called *forward propagation*.

foundation model Large-scale machine learning models trained on broad data that can be adapted to a wide range of downstream tasks, serving as a base for specialized applications.

four pillars framework A data engineering framework organizing concerns into Quality, Reliability, Scalability, and Governance to manage the data lifecycle.

FP16 16-bit floating-point numerical representation that reduces memory usage and accelerates computation on modern hardware accelerators while maintaining acceptable precision for many machine learning applications.

FP32 32-bit floating-point numerical representation that provides standard precision for mathematical computations but requires more memory and computational resources than lower-precision formats.

FP32 to INT8 A common quantization transformation that converts 32-bit floating-point weights and activations to 8-bit integers, achieving roughly 4× memory reduction while maintaining acceptable accuracy for many models.

FP8 An 8-bit floating-point format family used on newer accelerators for high-throughput training and inference, requiring careful scaling because of limited precision and range.

framework decomposition The systematic breakdown of neural network frameworks into hardware-mappable components, enabling efficient distribution of operations across processing elements.

G

GEMM General Matrix Multiply operations that follow the pattern $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$, representing the fundamental computational kernel underlying most neural network operations including fully connected layers and convolutional layers.

GEMV General Matrix-Vector multiplication operations that compute the product of a matrix and a vector, commonly used in neural network computations and requiring careful optimization for memory access patterns.

generalization The ability of a machine learning model to perform well on unseen data that differs from the training set, often improved through diverse and high-quality training data.

generalization gap The difference between a model's performance on training data and its performance on held-out data. A large generalization gap can indicate overfitting to the training data.

generative AI A category of artificial intelligence systems capable of creating new content such as text, images, audio, or video based on learned patterns from training data.

Goodhart's Law The observation that when a measure becomes an optimization target, it can stop representing the underlying goal.

GPT-2 A 1.5-billion parameter autoregressive language model released by OpenAI in 2019, serving as a primary lighthouse model for analyzing memory bandwidth constraints in transformer inference.

graceful degradation A system design principle where services continue functioning with reduced capabilities when faced with partial failures or data unavailability.

gradient accumulation A technique that simulates larger batch sizes by accumulating gradients from multiple smaller batches before updating model parameters, enabling training with limited memory.

gradient-based pruning A pruning method that uses gradient information to rank parameters, neurons, or filters as candidates for removal.

gradient clipping An optimization technique that mitigates exploding gradients by limiting their magnitude during backpropagation, typically by scaling gradients when their norm exceeds a threshold.

gradient compression A technique used in distributed training to reduce the communication overhead by compressing gradient information exchanged between computing nodes.

gradient descent An optimization algorithm that iteratively adjusts neural network parameters in the direction that minimizes the loss function, using gradients to determine update directions and magnitudes.

gradient synchronization The process in distributed training where locally computed gradients are aggregated across devices to ensure all devices update their parameters consistently.

GraNd (Gradient Normed) A data selection score based on the norm of an example's gradient during early training, identifying samples with larger learning signal.

graph break A point where a compiler cannot capture part of a program and must fall back to eager execution, splitting optimized graph regions.

graphics processing unit (GPU) A specialized processor originally designed for rendering graphics that provides massive parallel computing capabilities essential for efficient neural network computation, training, and inference.

Green AI A movement in AI research and practice that prioritizes computational efficiency and energy consumption as primary metrics alongside traditional performance metrics like accuracy.

Green500 Ranking system that evaluates the world's most powerful supercomputers based on energy efficiency measured in FLOP/s per watt rather than raw computational performance.

ground truth The reference target treated as correct for training or evaluation, often derived from measurement or annotation and therefore not necessarily identical to objective reality.

GRU Gated Recurrent Unit, a simplified variant of LSTM that uses fewer gates while maintaining the ability to capture long-term dependencies in sequential data.

H

hardware abstraction The layer in ML frameworks that provides a unified interface to diverse computing hardware (CPUs, GPUs, TPUs, accelerators) while handling device-specific optimizations and memory management behind the scenes.

hardware acceleration The use of specialized computing hardware to perform certain operations faster and more efficiently than software running on general-purpose processors.

hardware accelerator Specialized computing hardware designed to efficiently execute specific types of computations, such as GPUs for parallel processing or TPUs for machine learning workloads.

hardware-aware design The practice of designing neural network architectures specifically optimized for target hardware platforms, considering factors like memory hierarchy, compute units, and data movement patterns to maximize efficiency.

hidden layer An intermediate layer in a neural network between input and output layers that learns abstract representations by transforming data through learned weights and activation functions.

hidden state The internal memory of recurrent neural networks that carries information from previous time steps, enabling the network to maintain context across sequential inputs.

hierarchical processing A multi-tier system architecture where data and intelligence flow between different levels of the computing stack, from sensors to edge devices to cloud systems.

high-bandwidth memory (HBM) An advanced memory technology that provides much higher bandwidth than traditional DRAM by using 3D stacking and wide interfaces, critical for data-intensive AI workloads.

horizontal scaling Increasing system capacity by adding more machines or instances rather than upgrading existing hardware, providing better fault tolerance and load distribution.

hybrid machine learning The integration of multiple ML paradigms such as cloud, edge, mobile, and TinyML to form unified distributed systems that combine complementary strengths.

hybrid parallelism A distributed training approach that combines data parallelism and model parallelism to use the benefits of both strategies for training very large models.

hyperparameter A configuration setting that controls the learning process but is not learned from data, such as learning rate, batch size, or network architecture choices.

hyperscale data center Large-scale data center facilities containing thousands of servers and covering extensive floor space, designed to efficiently support massive computing workloads.

I

I/O bottleneck A performance limit where storage, decoding, preprocessing, or data transfer cannot supply data fast enough to keep compute resources busy.

idempotent transformation A data transformation that has the same effect when applied repeatedly as when applied once, supporting safe retries and reprocessing.

im2col A convolution-lowering technique that unfolds image patches into matrix columns so convolution can execute as GEMM.

ImageNet A massive visual database containing over 14 million labeled images across 20,000+ categories, developed by a team led by Fei-Fei Li and introduced in 2009, whose annual challenge became instrumental in driving breakthrough advances in computer vision.

imperative programming A programming paradigm in which programs specify sequences of commands that update state and direct control flow; it is distinct from whether operations execute eagerly or are staged for later execution.

inductive bias A structural assumption built into a model or architecture that restricts the functions it can learn efficiently, such as locality in CNNs.

inference The operational phase where trained neural networks make predictions on new data using fixed parameters, without weight updates.

InfiniBand A high-throughput, low-latency networking technology commonly used for multi-node accelerator clusters and distributed training.

information-compute ratio (ICR) A metric quantifying the efficiency of data selection, defined as the ratio of model performance gain to the computational cost (FLOPs) required to achieve it. Maximizing ICR is the primary goal of data selection strategies.

infrastructure as code Practice of managing and provisioning computing infrastructure through machine-readable configuration files rather than manual processes, enabling version control and automation.

INT8 An 8-bit signed-integer representation used to quantize weights, activations, or both. Each INT8 value occupies one quarter the storage of an FP32 value, and supported hardware can accelerate applicable operations, with accuracy depending on the quantization scheme.

intermediate representation (IR) A compiler-internal representation between frontend model code and backend hardware code generation, used by ML frameworks for optimization and portability.

internet of things A network of physical objects embedded with sensors, software, and other technologies that connect and exchange data with other devices and systems over the internet.

intersectional analysis Evaluation that considers combinations of demographic attributes (for example, race and gender simultaneously) to detect concentrated harms not visible in single-factor analysis.

IOPS Input/Output Operations Per Second; a storage performance metric measuring the number of read/write operations a device can handle per second, critical for random access workloads like training data loading.

Iron Law of ML Systems A quantitative framework decomposing ML system performance into three terms: Data (limited by bandwidth), Compute (limited by FLOP/s), and Latency (limited by overhead). Formulated as $T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$.

iterative pruning A gradual pruning strategy that removes parameters in multiple stages with fine-tuning between each stage, allowing the model to adapt to reduced capacity and typically achieving better accuracy than one-shot pruning.

J

JAX A numerical computing library developed by Google Research that combines NumPy's API with functional programming transformations including automatic differentiation, just-in-time compilation, and automatic vectorization for high-performance machine learning research.

JIT compilation Just-In-Time compilation that analyzes and optimizes code at runtime, enabling frameworks to balance the flexibility of eager execution with the performance benefits of graph optimization by compiling frequently used functions.

K

k-anonymity A privacy technique that ensures each record in a dataset is indistinguishable from at least $k_{\text{anon}} - 1$ other records by generalizing quasi-identifiers.

kernel A small matrix of learnable weights used in convolutional layers to detect specific features through the convolution operation, also called a filter. In GPU programming, the term also denotes a function launched for parallel execution by a grid of threads.

kernel fusion An optimization technique that combines multiple computational operations into a single kernel to reduce memory transfers and improve performance on parallel processors.

keyword spotting (KWS) A technology that detects specific wake words or phrases in audio streams, typically used in voice-activated devices with constraints on power consumption and latency.

knowledge distillation A model compression technique where a smaller "student" network learns to mimic the behavior of a larger "teacher" network by training on the teacher's soft output probabilities rather than just hard labels.

Kolmogorov-Smirnov test (K-S test) A non-parametric test that measures the maximum distance between two cumulative distributions, often used to compare feature distributions for drift.

Kullback-Leibler divergence (KL divergence) An asymmetric information-theoretic measure of how one probability distribution differs from another, used in monitoring and drift analysis.

KV cache Stored key and value tensors from previous transformer tokens used during autoregressive inference to avoid recomputing attention history.

L

L0 norm The count of nonzero elements in a vector or tensor, used in pruning to express sparsity or a target budget for remaining weights.

label quality drift Degradation in the reliability or consistency of labels over time, even when the underlying input distribution remains stable.

label shift A distribution shift where label frequencies change while the feature patterns conditioned on labels remain stable.

LAPACK Linear Algebra Package that extends BLAS with higher-level linear algebra operations including matrix decompositions, eigenvalue problems, and linear system solutions, providing essential mathematical foundations for machine learning computations.

latency The time delay between a request or event and the corresponding response or completion, critical in real-time applications where prompt responses are required.

latency constraints Real-time requirements that limit the maximum acceptable delay for model inference, driving optimization decisions in deployment scenarios where response time is critical.

layer normalization A normalization technique that normalizes inputs across the features dimension for each sample, commonly used in transformer architectures to stabilize training.

layerwise quantization A quantization granularity where all parameters within a single layer share the same quantization parameters, providing computational efficiency but potentially limiting representational precision compared to finer-grained approaches.

learning rate A hyperparameter that determines the step size for weight updates during gradient descent optimization, critically affecting training stability and convergence speed.

learning rate scheduling The systematic adjustment of learning rates during training, using strategies like step decay, exponential decay, or cosine annealing to improve convergence and final model performance.

lifecycle coherence The principle that all stages of ML development should align with overall system objectives, maintaining consistency in data handling, model architecture, and evaluation criteria.

linear scaling failure The phenomenon where increasing computing resources (for example, doubling accelerators) results in sublinear performance gains due to communication overhead and synchronization costs.

LINPACK Benchmark developed at Argonne National Laboratory that measures system performance by solving dense systems of linear equations, famous for its use in TOP500 supercomputer rankings.

Little's Law A queuing relationship stating that average concurrency equals arrival rate times average time in the system, linking throughput, latency, and capacity.

Llama Large Language Model Meta AI, a family of open foundation models that popularized efficient architectural choices like RMSNorm and SwiGLU, serving as a modern reference point for large-scale transformer efficiency.

load balancing Distribution of incoming requests across multiple server instances to prevent bottlenecks, improve response times, and ensure high availability.

locality-sensitive hashing (LSH) A hashing family that maps similar items to the same buckets with high probability, enabling scalable approximate similarity search.

logits The raw, unnormalized scores output by the last layer of a neural network before the activation function (like softmax) converts them into probabilities.

loss function A mathematical function that measures how well model outputs satisfy a training objective, providing the objective minimized by the training algorithm.

loss scaling A technique used in mixed-precision training that multiplies the loss by a large factor before backpropagation to prevent gradient underflow in reduced precision formats.

Lottery Ticket Hypothesis The theory that large neural networks contain sparse subnetworks that, when trained in isolation from proper initialization, can achieve comparable accuracy to the full network while being significantly smaller.

low-rank factorization A matrix decomposition technique that approximates large weight matrices as products of smaller matrices, reducing the number of parameters and computational operations required for neural network layers.

LSTM Long Short-Term Memory, a type of recurrent neural network architecture designed to handle long-term dependencies through gating mechanisms that control information flow.

M

machine learning A subset of artificial intelligence that enables systems to automatically improve performance on tasks through experience and data rather than explicit programming.

machine learning framework A software platform that provides tools and abstractions for designing, training, and deploying machine learning models, bridging user applications with infrastructure through computational graphs, hardware optimization, and workflow orchestration. Examples include TensorFlow, PyTorch, and JAX.

machine learning lifecycle A structured, iterative process that encompasses all stages involved in developing, deploying, and maintaining machine learning systems, from problem definition through ongoing monitoring and improvement.

machine learning operations (MLOps) The engineering discipline and set of tools focused on operationalizing machine learning models through automation, monitoring, and management of the entire ML pipeline from development to production.

machine learning systems engineering The engineering discipline focused on building reliable, efficient, and scalable AI systems across computational platforms, spanning the entire AI lifecycle from data acquisition through deployment and operations with emphasis on resource-awareness and system-level optimization.

macro benchmarks Evaluation methodology that assesses complete machine learning models to understand how architectural choices and component interactions affect overall system behavior and performance.

magnitude-based pruning The most common pruning method that removes parameters with the smallest absolute values, based on the assumption that weights with smaller magnitudes contribute less to the model's output.

masking An anonymization technique that alters or obfuscates sensitive values so they cannot be directly traced back to the original data subject.

membership inference attack A privacy attack that tries to determine whether a specific record was included in a model's training data.

memory bandwidth Rate at which data can be read from or written to memory, measured in bytes per second, which often becomes a bottleneck in memory-intensive machine learning workloads.

memory hierarchy The organization of memory systems with different access speeds and capacities, from fast on-chip caches to slower off-chip main memory.

memory wall The widening performance gap between processor speed and memory bandwidth, where computational capacity outpaces the rate at which data can be delivered to the processor, becoming a primary bottleneck for large ML models.

metadata Descriptive information about datasets that includes details about data collection, quality metrics, validation status, and other contextual information essential for data management.

micro-batch A smaller slice of an effective batch processed sequentially for gradient accumulation or pipeline parallelism to reduce peak memory.

micro benchmarks Specialized evaluation tools that assess individual components or specific operations within machine learning systems, such as tensor operations or neural network layers.

microcontroller A small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals, commonly used in embedded systems.

MinHash A sketching method that approximates set similarity with compact signatures, commonly used for scalable near-duplicate detection.

mini-batch gradient descent A training approach that computes gradients and updates weights using a small subset of training examples simultaneously, balancing computational efficiency with gradient estimation quality.

mini-batch processing An optimization approach that computes gradients over small batches of examples, balancing the computational efficiency of batch processing with the memory constraints of stochastic methods.

mixed-precision computing A technique that uses different numerical precisions at various stages of computation, such as FP16 for matrix multiplications and FP32 for accumulations.

mixed-precision training A training methodology that combines different numerical precisions (typically FP16 and FP32) to optimize memory usage and computational speed while maintaining training stability.

ML node A single production ML application treated as an operational unit, including data pipelines, feature computation, model training, serving infrastructure, and monitoring.

ML systems Integrated computing systems comprising three core components: data that guides algorithmic behavior, learning algorithms that extract patterns from data, and

computing infrastructure that enables both training and inference processes.

ML systems spectrum The range of machine learning system deployments from cloud-based systems with abundant resources to tiny embedded devices with severe constraints, each requiring different optimization strategies and trade-offs.

ML Test Score A production-readiness rubric for ML systems that evaluates data, model, infrastructure, and monitoring tests.

MLCommons Organization that develops and maintains industry-standard benchmarks for machine learning systems, including the MLPerf suite for training and inference evaluation.

MLPerf Industry-standard benchmark suite that provides standardized tests for training and inference across various deep learning workloads, enabling fair comparisons of machine learning systems.

MLPerf execution scenarios Standard MLPerf inference traffic scenarios that model sequential, synchronized, server, batch, and interactive workloads with different latency and throughput requirements.

MLPerf Inference Benchmark framework that evaluates machine learning inference performance across different deployment environments, from cloud data centers to mobile devices and embedded systems.

MLPerf Mobile Specialized benchmark that extends MLPerf evaluation to smartphones and mobile devices, measuring latency and responsiveness under strict power and memory constraints.

MLPerf Power An MLPerf methodology for measuring power and energy efficiency of ML workloads alongside performance.

MLPerf Tiny Benchmark designed for embedded and ultra-low-power AI systems such as IoT devices, wearables, and microcontrollers operating with minimal processing capabilities.

MLPerf Training Standardized benchmark that evaluates machine learning training performance by measuring time-to-accuracy, throughput, and resource utilization across different hardware platforms.

mobile machine learning The execution of machine learning models directly on portable, battery-powered devices like smartphones and tablets, enabling personalized and responsive applications.

model cards A standardized format for documenting machine learning models, capturing information essential for responsible deployment, including intended use, performance factors, and ethical considerations.

model-centric AI A research paradigm where the dataset is treated as fixed and engineering effort focuses on optimizing model architecture.

model compression Techniques used to reduce the size and computational requirements of machine learning models while seeking to preserve accuracy, enabling deployment on resource-constrained devices.

model deployment The process of integrating trained machine learning models into production systems where they can make predictions on new data and provide value to end users.

model drift The degradation of machine learning model performance over time due to changes in data patterns, user behavior, or environmental conditions that differ from the original training conditions.

model evaluation The systematic assessment of machine learning model performance using various metrics and validation techniques to determine whether the model meets requirements and is ready for deployment.

model FLOPs utilization (MFU) The ratio of the model FLOPs per second implied by observed throughput to the hardware's theoretical peak FLOPs per second, measuring how effectively a workload uses available compute for model computation.

model optimization The systematic refinement of machine learning models to enhance their efficiency while maintaining effectiveness, balancing trade-offs between accuracy, computational cost, memory usage, latency, and energy efficiency.

model parallelism A distributed training strategy that splits a neural network model across multiple devices, with each device responsible for computing a portion of the network.

model quantization The process of reducing numerical precision in machine learning models, typically from FP32 to INT8, to decrease model size and memory traffic; speed increases only with efficient low-precision kernels.

model registry Centralized repository for storing, versioning, and managing trained machine learning models with associated metadata, facilitating model governance and deployment.

model serving Infrastructure and systems that expose deployed machine learning models through APIs to handle prediction requests at scale with appropriate latency and throughput.

model training The process of using machine learning algorithms to learn patterns from training data, adjusting model parameters to minimize prediction errors and create a functional predictive system.

model validation The process of testing machine learning models on independent datasets to assess their generalization ability and ensure they perform reliably on unseen data.

model versioning The systematic tracking and management of different versions of machine learning models, including their parameters, training data, and performance metrics, to enable comparison and rollback capabilities.

momentum An optimization technique that accumulates a velocity vector across iterations to help gradient descent navigate through local minima and accelerate convergence in consistent gradient directions.

monitoring The continuous observation and measurement of machine learning system performance, data quality, and operational metrics in production to detect issues and trigger maintenance actions.

multi-head attention An attention mechanism that uses multiple parallel attention heads, each focusing on different aspects of the input to capture diverse types of relationships simultaneously.

multilayer perceptron (MLP) A feedforward neural network with one or more hidden layers between input and output layers, capable of learning nonlinear mappings through dense connections and activation functions.

multiply-accumulate (MAC) The operation of multiplying two values and accumulating the result into a running sum, dominating linear layers and accelerator datapaths.

MYCIN One of the first large-scale expert systems developed at Stanford beginning in 1972 to diagnose blood infections, representing the shift toward capturing human expert knowledge in specific domains rather than pursuing general artificial intelligence.

N

N:M sparsity A structured sparsity pattern where exactly N of every M values are nonzero, giving hardware more regular zero patterns to exploit.

neural architecture search An automated approach that searches possible combinations of layers, connections, and architectural hyperparameters to find neural network architectures that perform well under specified objectives and constraints.

neural network A computational model consisting of interconnected nodes organized in layers that can learn to map inputs to outputs through adjustable connection weights.

neural processing unit (NPU) Specialized processors designed specifically for accelerating neural network operations and machine learning computations, optimized for parallel processing of AI workloads.

neuromorphic computing A computing approach that mimics the structure and function of biological neural networks, potentially offering more energy-efficient processing for AI applications.

NoSQL A category of database systems designed to handle large volumes of unstructured or semi-structured data with flexible schemas, often used in big data applications.

numerical precision optimization The dimension of model optimization that addresses how numerical values are represented and processed, including quantization techniques that map high-precision values to lower-bit representations.

NVLink NVIDIA's high-bandwidth GPU interconnect for intra-node communication, useful for gradient synchronization and model-parallel activation transfers.

O

observability Comprehensive monitoring approach that provides insight into system behavior through metrics, logs, and traces, enabling understanding of internal states from external outputs.

on-chip memory Fast memory integrated directly onto the processor chip, including caches and scratchpad memory, providing high bandwidth and low latency data access.

on-device learning The capability for machine learning models to adapt and learn directly on edge devices without requiring data transmission to external servers.

one-hot encoding A representation where categorical labels become vectors with a single 1 and remaining 0s.

one-shot pruning A pruning strategy where a large fraction of parameters is removed in a single step, typically followed by fine-tuning to recover accuracy, offering simplicity but potentially requiring more aggressive fine-tuning.

online analytical processing (OLAP) A database approach optimized for complex analytical queries across large datasets, typically used in data warehouses for business intelligence.

online inference Real-time prediction serving that processes individual requests with low latency, suitable for interactive applications requiring immediate responses.

online transaction processing (OLTP) A database approach optimized for frequent, short transactions and real-time processing, commonly used in operational applications.

ONNX Open Neural Network Exchange, a standardized format for representing machine

learning models that enables interoperability between different frameworks, allowing models trained in one framework to be deployed using another.

ONNX Runtime Cross-platform inference engine that optimizes machine learning models through techniques like operator fusion and kernel tuning to improve inference speed and reduce computational overhead.

optimizer An algorithm that adjusts model parameters during training to minimize the loss function, with common examples including SGD (Stochastic Gradient Descent), Adam, and RMSprop, each with different strategies for parameter updates.

optimizer state Auxiliary values maintained by an optimizer in addition to model parameters, such as Adam's momentum and variance tensors, often dominating training memory for large models.

orchestration Coordination and management of complex workflows and distributed computing tasks, often using platforms like Kubernetes for container management.

outlier detection The process of identifying data points that significantly deviate from normal patterns, which may represent errors, anomalies, or valuable rare events.

overfitting A phenomenon where a model learns specific details of training data so well that it fails to generalize to new, unseen examples, typically indicated by high training accuracy but poor validation performance.

P

padding A technique in convolutional networks that adds zeros or other values around the input borders to control the spatial dimensions of the output feature maps.

PagedAttention A KV-cache memory-management technique that stores attention context in fixed-size pages rather than contiguous per-request blocks to reduce fragmentation.

paradigm shift A fundamental change in scientific approach, like the shift from symbolic reasoning to statistical learning in AI during the 1990s, and from shallow to deep learning in the 2010s, requiring researchers to abandon established methods for radically different approaches.

parallelism The simultaneous execution of multiple computational tasks or operations, fundamental to achieving high performance in neural network processing.

parameter A learnable component of a neural network, including weights and biases, that gets adjusted during training to minimize the loss function.

Pareto frontier The set of configurations where improving one objective requires worsening another, used to reason about accuracy, latency, memory, and energy trade-offs.

partitioning A database technique that divides large datasets into smaller, manageable segments based on specific criteria to improve query performance and system scalability.

perceptron The fundamental building block of neural networks, consisting of weighted inputs, a bias term, and an activation function that produces a single output.

performance insights Analytical observations derived from monitoring production machine learning systems that reveal opportunities for improvement in model accuracy, system efficiency, or user experience.

performance-per-data (PPD) A metric measuring the accuracy gain per training sample, used to evaluate the quality of a dataset or selection strategy. High PPD indicates a dataset rich in information and low in redundancy.

pinned memory Page-locked host memory that enables direct, asynchronous transfers between CPU and GPU, improving data-loading throughput at the cost of reserving system memory.

pipeline jungle Anti-pattern where complex, interdependent data processing pipelines become difficult to maintain, debug, and modify, leading to technical debt and operational complexity.

pipeline parallelism A form of model parallelism where different layers of a model are placed on different devices and data flows through them in a pipeline fashion, allowing multiple batches to be processed simultaneously.

point-in-time correctness The guarantee that training examples use only feature values that would have been available at the historical prediction time, preventing leakage.

pooling A downsampling operation in convolutional networks that reduces spatial dimensions while retaining important features, commonly using max or average operations over local regions.

population stability index (PSI) A binned distribution-shift metric that compares current feature distributions against a baseline to detect population drift.

positional encoding A method used in transformer architectures to inject information about the position of tokens in a sequence, since transformers lack inherent sequential processing.

post-training quantization A quantization approach applied to already-trained models without modifying the training process, typically involving calibration on representative data to determine optimal quantization parameters.

power usage effectiveness (PUE) Metric used in data centers to measure energy efficiency, calculated as the ratio of total facility power consumption to IT equipment power consumption.

power wall The technological barrier reached around 2005 where increasing processor frequency no longer yielded performance gains without unsustainable increases in power density and heat generation, forcing a shift to parallel and specialized architectures.

precision In numerical computing, the number of significant bits or digits used to represent values, affecting computational accuracy and resource requirements in machine learning systems.

prefetching A system optimization technique that loads data into memory before it is needed, overlapping data loading with computation to reduce idle time and improve training throughput.

problem definition The initial stage of machine learning development that involves clearly specifying objectives, constraints, success metrics, and operational requirements to guide all subsequent development decisions.

processing element (PE) A repeated accelerator building block containing compute units and local storage, with arrays of PEs providing parallel tensor execution.

programmable logic controller (PLC) Industrial control systems used in manufacturing and IoT environments that can be integrated with ML models for automated decision-making in operational technology contexts.

protein folding problem The scientific challenge of predicting the three-dimensional structure and dynamics of proteins from their amino acid sequences and environments, a longstanding problem on which systems such as AlphaFold achieved breakthrough accuracy without resolving every folding question.

proxy variable A feature that indirectly carries signal about a protected or sensitive attribute even when that attribute is removed.

pseudonymization A privacy technique that replaces direct identifiers with artificial identifiers while maintaining the ability to trace records for analysis purposes.

PyTorch A deep learning framework developed by Facebook's AI Research lab that emphasizes dynamic computational graphs, eager execution, and intuitive Python integration, particularly popular for research and experimentation.

Q

quantization A model-compression technique that represents parameters or activations at lower precision, such as FP32 to INT8, reducing storage and data movement without reducing mathematical operation count; speedups require supported hardware and kernels.

quantization-aware training A training approach where quantization effects are simulated during the training process, allowing the model to adapt to reduced precision and typically achieving better accuracy than post-training quantization.

quantization granularity The level at which quantization parameters are applied, ranging from per-tensor (coarsest) to per-channel or per-group (finer), with finer granularity typically preserving more accuracy but requiring more storage.

queries per second (QPS) Performance metric that measures how many inference requests a system can process in one second, commonly used to evaluate throughput in production deployments.

query-key-value The three components of attention mechanisms where queries determine what to look for, keys represent what is available, and values contain the actual information to be weighted and combined.

R

real-time processing Processing data as it becomes available under application timing constraints; hard real-time systems guarantee worst-case deadlines, while soft real-time systems tolerate occasional misses.

receptive field The region of the input that influences a particular neuron's output, determining the spatial extent of patterns that can be detected by that neuron.

recurrent neural network A type of neural network designed for sequential data processing, featuring connections that create loops allowing information to persist across time steps.

Red AI AI research and development that prioritizes maximizing accuracy or performance without regard for the increasing computational and environmental costs required.

regularization Techniques used to prevent overfitting in neural networks by adding constraints or penalties, including methods like dropout, weight decay, and data augmentation.

ReLU Rectified Linear Unit activation function defined as $f(x) = \max(0, x)$ that introduces nonlinearity while maintaining computational efficiency and helping mitigate vanishing gradients for positive inputs.

residual connection A skip connection that adds the input of a layer to its output, enabling the training of very deep networks by mitigating the vanishing gradient problem.

ResNet Residual Network, a deep convolutional architecture built from residual blocks with skip connections, enabling the training of networks with hundreds of layers and achieving breakthrough performance.

ResNet-50 A specific 50-layer variant of the Residual Network architecture that serves as the canonical lighthouse model for compute-bound workloads, balancing depth and computational cost for vision benchmarks.

retinal fundus photographs Medical images of the interior surface of the eye, including the retina, optic disc, and blood vessels, commonly used for diagnosing eye diseases and training medical AI systems.

reverse-mode differentiation An automatic differentiation technique that computes gradients by traversing the computational graph in reverse order, highly efficient for functions with many inputs and few outputs, making it ideal for neural network training.

ridge point The arithmetic intensity at which a workload transitions from memory-bound to compute bound on a given hardware platform, calculated as peak FLOP/s divided by memory bandwidth. The inflection point on the Roofline Model diagram.

Ring AllReduce A bandwidth-efficient AllReduce algorithm where devices pass chunks around a ring and accumulate results for distributed gradient synchronization.

RMSprop An adaptive learning rate optimization algorithm that maintains a moving average of squared gradients to automatically adjust learning rates for each parameter during training.

role-based access control (RBAC) An access-control model that grants permissions according to user, service, or team roles rather than ad hoc individual grants.

rollback Process of reverting to a previous stable version of a model or system when issues are detected in production, ensuring service continuity.

roofline analysis A performance modeling technique that plots attainable performance against operational intensity, with peak compute and bandwidth forming the ceilings, to identify whether a system is memory bound or compute bound.

S

scalability The ability of machine learning systems to handle increasing amounts of data, users, or computational demands without significant degradation in performance or user experience.

scale and zero-point Quantization parameters that map real-valued tensors to integer values and back, where scale controls step size and zero-point maps real zero into the quantized range.

schema The structure and format definition of data that specifies data types, field names, and relationships, essential for data validation and processing consistency.

schema evolution The process of modifying data schemas over time while maintaining backward compatibility and ensuring continued functionality of dependent systems and applications.

schema-on-read An approach used in data lakes where data structure is defined and enforced at the time of reading rather than when storing, providing flexibility for diverse data types.

scratchpad memory Fast software-managed local memory used by accelerators to stage predictable data movement under compiler or runtime control.

segmentation maps Detailed annotations that classify objects at the pixel level, providing the most granular labeling information but requiring significantly more storage and processing resources.

self-attention An attention mechanism where queries, keys, and values all come from the same sequence, allowing each position to attend to all positions including itself.

semi-supervised learning A machine learning approach that uses both labeled and unlabeled data for training, using structural assumptions to improve model performance with limited labels.

sensitivity The proportion of actual positive cases correctly identified by a classification model, also known as true positive rate or recall; critical in medical applications where missing positive cases has severe consequences.

serverless Cloud computing model where infrastructure is automatically managed by the provider, allowing code execution without server management concerns.

service level agreement (SLA) A formal external contract specifying service commitments, often through one or more SLOs, and the consequences or penalties for noncompliance.

service-level objective (SLO) An internal measurable target for a service-level indicator such as latency, error rate, or availability that guides operations and may underpin an external SLA.

shadow deployment A deployment strategy where a new model runs in parallel with the production model, making predictions that are logged but not served to users, enabling validation without user impact.

shallow learning Machine learning approaches that use algorithms with limited complexity, such as support vector machines and decision trees, which require carefully engineered features but cannot automatically discover hierarchical representations like deep learning methods.

SHAP values Feature-attribution scores based on Shapley values that estimate each feature's contribution to a model prediction.

sigmoid An activation function that maps input values to a range between 0 and 1, historically popular but prone to vanishing gradient problems in deep networks.

silent bias Model unfairness that produces valid-looking but discriminatory outputs, evading traditional error monitoring and requiring disaggregated evaluation to detect.

silent degradation The gradual decline in ML system performance that occurs without triggering errors, exceptions, or alerts. An ML system can continue operating while producing increasingly inaccurate predictions.

silent failure A system failure mode where an ML model continues to produce plausible-looking outputs that are gradually less accurate or contextually relevant without triggering conventional error alerts.

SIMD (Single Instruction, Multiple Data) A parallel computing architecture that applies the same operation to multiple data elements simultaneously, effective for regular data-parallel computations.

SIMT (Single Instruction, Multiple Threads) An extension of SIMD that enables parallel execution across multiple independent threads, each maintaining its own state and program counter.

singular value decomposition A matrix factorization technique that decomposes a matrix into the product of three matrices, commonly used in low-rank approximations to compress neural network layers by retaining only the most significant singular values.

skip connection A direct connection that bypasses one or more layers, allowing gradients to flow more easily through deep networks and enabling better training of very deep architectures.

soft targets Teacher-model probability distributions used as supervision in knowledge distillation, carrying class-similarity information beyond hard labels.

softmax An activation function that converts logits into a probability distribution where outputs sum to 1, used in multi-class classification.

sparse Tensor Core A Tensor Core variant that accelerates supported structured sparsity patterns such as 2:4 sparsity by skipping constrained zero values.

sparsity The property of tensors in which many values are exactly zero, which can improve computational efficiency when the zeros have suitable structure and specialized sparse kernels or hardware support them; near-zero values count only after pruning.

SPEC CPU Standardized benchmark suite developed by the System Performance Evaluation Cooperative that measures processor performance using real-world applications rather than synthetic tests.

special function unit (SFU) Dedicated hardware for specialized operations such as exponentials, reciprocals, roots, and some activation-function primitives that do not map cleanly to ordinary arithmetic or matrix multiply units.

specificity The proportion of actual negative cases correctly identified by a classification model, also known as true negative rate; important for avoiding overwhelming referral systems with false positives.

SPECpower Benchmark methodology that measures server energy efficiency across varying workload levels, enabling direct comparisons of power-performance trade-offs in computing systems.

speculative decoding An optimization technique for autoregressive language models where a smaller model generates draft tokens that are then verified by a larger model, accelerating inference while maintaining quality.

speed of light (latency) The physical speed limit of information transmission (approx. 200,000 km/s in fiber), creating an irreducible lower bound on network latency that necessitates edge computing for applications requiring sub-10 ms response times over long distances.

state dict A framework serialization mapping from module or optimizer names to tensors, used to save, load, and checkpoint models and optimizer state.

static graph A computational graph that is defined completely before execution begins, enabling comprehensive optimization and efficient deployment but requiring all operations to be specified upfront, limiting runtime flexibility.

static quantization A quantization approach where quantization parameters are determined once during calibration and remain fixed during inference, providing computational efficiency but less adaptability than dynamic approaches.

statistical learning The era of machine learning that emerged in the 1990s, shifting focus from rule-based symbolic AI to algorithms

that could learn patterns from data, laying the groundwork for modern data-driven approaches to artificial intelligence.

stochastic gradient descent A variant of gradient descent that estimates gradients using individual training examples or small batches rather than the entire dataset, reducing memory requirements and enabling online learning.

Straight-Through Estimator (STE) A gradient approximation that lets training pass gradients through nondifferentiable operations such as rounding, often used in quantization-aware training.

stream ingestion A data processing pattern that handles data in real-time as it arrives, essential for applications requiring immediate processing and low-latency responses.

stream processing Real-time data processing approach that handles continuous flows of data as it arrives, enabling immediate responses to events and pattern detection.

stride The step size by which a convolutional filter moves across the input, controlling the spatial dimensions of the output and the degree of overlap between filter applications.

structured pruning A pruning approach that removes entire computational units such as neurons, channels, or layers, producing smaller dense models that are more hardware-friendly than the sparse matrices created by unstructured pruning.

supervised learning A machine learning approach where models learn from labeled training examples to make predictions on new, unlabeled data.

symbolic AI An approach to artificial intelligence that uses high-level symbolic representations of problems, logic, and search algorithms, dominant before the deep learning revolution and characterized by expert systems and rule-based reasoning.

synthetic benchmark Artificial test program designed to measure specific aspects of system performance, as opposed to benchmarks based on real-world applications and workloads.

synthetic data Artificially generated data created using algorithms, simulations, or generative models to supplement real-world datasets, addressing limitations in data availability or privacy concerns.

system entropy The tendency of ML systems to degrade over time as the world changes (drift) or as hidden dependencies accumulate (technical debt), requiring active energy (ops) to maintain order.

System-on-Chip (SoC) An integrated circuit that incorporates most or all components of a computer or electronic system, including CPU, GPU, memory, and specialized processors on a single chip. Commonly used in mobile devices and embedded systems for space and power efficiency.

systems co-design Joint design of algorithms, software, and hardware so model structure and execution fit physical capabilities and constraints.

systems integration The process of combining various components and subsystems into a unified, functional system that operates efficiently and reliably as a whole.

systems thinking An approach to understanding complex systems by considering how individual components interact and affect the whole system, particularly important in ML where data, algorithms, hardware, and deployment environments must work together effectively.

systolic array A specialized hardware architecture that efficiently performs matrix operations by streaming data through a grid of processing elements, minimizing data movement and energy consumption.

T

tail latency High-percentile response time in a system, typically measured as p95 or p99 latency, important for characterizing slow requests under peak load but not equivalent to the absolute worst case. It complements median and mean latency rather than replacing them.

tailored inference benchmarks Specialized performance tests designed for specific deployment environments or use cases, accounting for unique constraints and optimization requirements.

tanh Hyperbolic tangent activation function that maps inputs to (-1, 1), providing zero-centered outputs.

technical debt Long-term maintenance cost accumulated from expedient design decisions during development, particularly problematic in ML systems due to data dependencies and model complexity.

telemetry Automated collection and transmission of performance data and metrics from distributed systems, enabling remote monitoring and analysis.

tensor A multi-dimensional array used to represent data in neural networks, generalizing scalars (0D), vectors (1D), and matrices (2D) to higher dimensions.

Tensor Core A specialized GPU matrix unit for accelerated mixed-precision multiply-accumulate operations on tensor tiles.

tensor decomposition The extension of matrix factorization to higher-order tensors, used to compress neural network layers by representing weight tensors as combinations of smaller tensors with fewer parameters.

tensor parallelism A model parallelism strategy where individual tensor operations (like matrix multiplication) are split across multiple devices, reducing memory per device and potentially reducing large-layer latency when the reduction in computation time exceeds the added communication time.

tensor processing unit (TPU) Google's custom application-specific integrated circuit designed specifically for machine learning workloads, optimized for matrix operations and featuring systolic array architecture.

TensorFlow A comprehensive machine learning framework developed by Google that provides tools for the entire ML pipeline from research to production, featuring both eager execution and graph-based computation with extensive ecosystem support.

TensorRT NVIDIA's inference optimization library that applies techniques like operator fusion and precision reduction to accelerate deep learning inference on GPU hardware.

ternarization An extreme quantization technique that constrains weights to three values (typically -1, 0, +1), providing significant compression while maintaining more representational capacity than binary quantization.

thermal design power (TDP) The sustained power and heat level a device or cooling system is designed to dissipate under typical maximum workload.

thermal throttling A protective mechanism in mobile and embedded devices that reduces processor clock speed and performance to prevent overheating, often limiting the sustained performance of on-device ML inference.

throughput The rate at which a system can process data or complete operations, typically measured in operations per second and crucial for training large models.

time per output token (TPOT) The per-token latency during autoregressive decode after the first token, reflecting decode throughput and memory-bandwidth limits.

time-to-accuracy The wall-clock time required to train a model to a specified validation accuracy.

time to first token (TTFT) The time from request arrival to the first generated token in streaming LLM serving, including prefill, scheduling, and initial response latency.

TinyML A field focused on deploying machine learning models on ultra-constrained embedded devices such as microcontrollers and sensors, operating in the milliwatt to subwatt power range, with severe limitations on memory, power, and computational capacity. Also called *tiny machine learning*.

TOPS Tera Operations Per Second, an operation-rate unit indicating how many trillion operations a system can execute in one second. It is commonly reported for low-precision integer workloads, so the operation type and precision must be specified.

total cost of ownership (TCO) A comprehensive financial metric for ML systems encompassing training, inference, and operational costs over the system's entire lifecycle.

train-serve split A hybrid architecture pattern where computationally intensive model training occurs on powerful cloud infrastructure, while the trained model is optimized and deployed for inference on resource-constrained edge or mobile devices.

training The process of adjusting neural network parameters using data or interaction experience and an optimization procedure to minimize a loss or maximize a reward.

training-serving skew A mismatch between how features or data are computed during model training vs. serving in production, which can degrade model performance. Common causes include different preprocessing pipelines, feature computation timing, or data sources between training and inference.

transfer learning A machine learning technique that uses knowledge gained from pretrained models on related tasks, allowing faster training and better performance on new tasks with limited data by reusing learned features and representations.

transformer A neural network architecture built from attention, feed-forward blocks, residual connections, and normalization; the original design eliminates recurrence and convolution, though later variants may incorporate them.

translation equivariance A property where shifting the input causes a corresponding shift in the output feature map, distinct from invariance which discards position.

translation invariance The property that a model's output remains unchanged when the input shifts. Convolution is translation-equivariant; pooling or global aggregation can produce approximate invariance.

Tucker decomposition A tensor decomposition method that generalizes singular value decomposition to higher-order tensors using a core tensor and factor matrices, commonly used for compressing convolutional neural network layers.

TV white spaces Unused broadcasting frequencies that can be repurposed for internet connectivity, as employed by systems like FarmBeats to extend network access to remote agricultural sensors and IoT devices.

U

uniform quantization A quantization approach where the range of values is divided into evenly spaced intervals, providing simple implementation but potentially suboptimal for nonuniform value distributions.

universal approximation theorem A theoretical result proving that neural networks with sufficient width and a suitable nonlinear activation function can approximate any continuous function on a compact domain to arbitrary accuracy.

unreasonable effectiveness of data The empirical observation that for many problems, adding more data is more effective than improving algorithms, driving the data-centric AI paradigm.

unstructured pruning A pruning approach that removes individual weights while preserving the overall network architecture, creating sparse weight matrices that require specialized hardware support to realize computational benefits.

unstructured sparsity A form of model sparsity where individual weights are set to zero without following any particular pattern, creating irregular sparsity patterns that require specialized hardware support to realize computational benefits.

V

validation issues Problems identified during model testing that indicate poor performance, overfitting, data quality problems, or other issues that must be resolved before deployment.

vanishing gradient problem A problem in deep neural networks where gradients become exponentially smaller as they propagate backward through layers, making it difficult for early layers to learn effectively.

vector operations Computational operations that process multiple data elements simultaneously, enabling efficient parallel execution of element-wise transformations in neural networks.

verification gap The mismatch between finite test-set coverage and the much larger input space an ML system may encounter in production.

versioning The practice of tracking changes to datasets, models, and pipelines over time, enabling reproducibility, rollback capabilities, and audit trails in ML systems.

virtuous cycle The self-reinforcing process in deep learning where improvements in data availability, algorithms, and computing power each enable further advances in the other areas, accelerating overall progress.

Von Neumann bottleneck The performance limitation caused by the shared bus between processor and memory in traditional computer architectures, where data movement becomes more expensive than computation.

W

warehouse-scale computer (WSC) A data center operated as a single programmable computer, treating compute, storage, networking, and failures as system-level resources.

warp A group of 32 threads on NVIDIA GPUs whose active threads execute one common instruction at a time; the fundamental unit of thread scheduling on NVIDIA GPUs.

Waymo A subsidiary of Alphabet Inc. that represents a leading deployment of machine learning systems in autonomous vehicle technology, demonstrating how ML systems can span from embedded systems to cloud infrastructure in safety-critical environments.

weak supervision An approach that uses lower-quality labels obtained more efficiently through heuristics, distant supervision, or programmatic methods rather than manual expert annotation.

web scraping An automated technique for extracting data from websites to build custom datasets, requiring careful consideration of legal, ethical, and technical constraints.

weight A learnable parameter that determines the strength of connection between neurons in different layers, adjusted during training to minimize the loss function.

weight matrix An organized collection of weights connecting one layer to another in a neural network, enabling efficient computation through matrix operations.

weight sharing The practice of using the same parameters across different spatial locations, as in convolutional networks, reducing the number of parameters while maintaining pattern detection capabilities.

Whetstone Early benchmark developed at the UK National Physical Laboratory by Curnow and Wichmann, first published as an ALGOL 60 program in 1972 and later as the canonical FORTRAN version in 1976. It measured floating-point arithmetic performance in MWIPS (millions of Whetstone instructions per second), becoming one of the first widely-adopted standardized performance tests.

workflow orchestration Automated coordination and management of complex ML pipeline sequences, ensuring proper execution order, dependency management, and error handling across distributed systems.

X

XLA Accelerated Linear Algebra, a domain-specific compiler for linear algebra operations that optimizes TensorFlow and JAX computations by generating efficient code for various hardware platforms including CPUs, GPUs, and TPUs.

Y

YOLO You Only Look Once, a family of single-stage object detection models that reframes detection as a spatial regression problem, predicting bounding boxes and class probabilities in a single forward pass to achieve high throughput on edge accelerators.

Z

zero-copy memory A memory allocation pattern that allows accelerator execution units to directly read or write host pinned memory across the system bus without an explicit staging transfer, trading bus bandwidth for reduced memory allocation footprint.

ZeRO Zero Redundancy Optimizer, a memory-efficient distributed training paradigm that partitions optimizer states, gradients, and model parameters across data-parallel ranks to eliminate memory redundancy while maintaining communication efficiency.

zero-shot learning A deployment paradigm where a pretrained model performs inference on tasks or classes unseen during supervised training, relying on learned semantic representations without fine-tuning.

References

Abadi, Martín, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2016. "TensorFlow: A System for Large-Scale Machine Learning." *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 265–83.

Agarwal, Pankaj K., Sariel Har-Peled, and Kasturi R. Varadarajan. 2005. "Geometric Approximation via Coresets." In *Combinatorial and Computational Geometry*, edited by Jacob E. Goodman, János Pach, and Emo Welzl, vol. 52. MSRI Publications. Cambridge University Press. https://doi.org/10.1017/9781009701259.002.

Agrawal, Amey, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve." *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–34.

Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895–901. https://doi.org/10.18653/v1/2023.emnlp-main.298.

Akidau, Tyler, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, et al. 2015. "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing." *Proceedings of the VLDB Endowment* 8 (12): 1792–803. https://doi.org/10.14778/2824032.2824076.

Altini, Marco, and Hannu Kinnunen. 2021. "The Promise of Sleep: A Multi-Sensor Approach for Accurate Sleep Stage Detection Using the Oura Ring." *Sensors* 21 (13): 4302. https://doi.org/10.3390/s21134302.

Amazon Web Services. 2024a. *Amazon Mechanical Turk*.

Amazon Web Services. 2024b. *Amazon Simple Storage Service (S3)*.

Amazon Web Services. 2024c. *Amazon Web Services (AWS)*.

Amazon Web Services. 2024d. *AWS Auto Scaling*.

Amazon Web Services. 2024e. *AWS CloudFormation*.

AMD. 2023. *AMD Instinct MI300X Accelerators*. AMD product documentation.

Amdahl, Gene M. 1967. "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities." *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference on - AFIPS '67 (Spring)*, AFIPS '67 (spring), 483–85. https://doi.org/10.1145/1465482.1465560.

Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. "Software Engineering for Machine Learning: A Case Study." *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. https://doi.org/10.1109/icse-seip.2019.00042.

Amodei, Dario, and Danny Hernandez. 2018a. "AI and Compute." *OpenAI Blog* 2.

Amodei, Dario, and Danny Hernandez. 2018b. "AI and Compute." *OpenAI Blog* 6.

Angwin, Julia, Jeff Larson, Surya Mattu, and Lauren Kirchner. 2016. *Machine Bias*. ProPublica investigation.

Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, et al. 2024. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation." *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 929–47. https://doi.org/10.1145/3620665.3640366.

Anthropic. 2023. *Introducing Claude 2.1*.

Anthropic. 2024. *Introducing the Next Generation of Claude*.

AnyLogic. 2024. *Synthetic Data for Artificial Intelligence*.

Apache Software Foundation. 2024. *Apache Airflow*.

Apple. 2024a. *Facial Matching Security*. Apple Platform Security.

Apple. 2024b. *The Secure Enclave*. Apple Platform Security.

Ardila, Rosana, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. 2020. "Common Voice: A Massively-Multilingual Speech Corpus." *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 4218–22.

Armbrust, Michael, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, et al. 2020. "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores." *Proceedings of the VLDB Endowment* 13 (12): 3411–24. https://doi.org/10.14778/3415478.3415560.

Authors, Kubeflow. 2024. *Kubeflow*.

Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. "Layer Normalization." *arXiv Preprint arXiv:1607.06450*.

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. "Neural Machine Translation by Jointly Learning to Align and Translate." *International Conference on Learning Representations (ICLR)*.

Bai, Zhaojun, James Demmel, Jack Dongarra, Julien Langou, and Jenny Wang. 2006. "LAPACK." In *Handbook of Linear Algebra*. Chapman; Hall/CRC. https://doi.org/10.1201/9781420010572-75.

Baidu Research. 2016. *DeepBench: Benchmarking Deep Learning Operations on Different Hardware*.

Bakheet, Samy, and Ayoub Al-Hamadi. 2021. "A Framework for Instantaneous Driver Drowsiness Detection Based on Improved HOG Features and Naïve Bayesian Classification." *Brain Sciences* 11 (2): 240. https://doi.org/10.3390/brainsci11020240.

Banbury, Colby R., Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, et al. 2020. "Benchmarking TinyML Systems: Challenges and Direction." *arXiv Preprint arXiv:2003.04821*.

Banbury, Colby R, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul N Whatmough. 2021. "MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers." *Proceedings of Machine Learning and Systems* 3: 517–32.

Banbury, Colby, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, et al. 2021. "MLPerf Tiny Benchmark." *arXiv Preprint*.

Barocas, Solon, and Andrew D. Selbst. 2016. "Big Data's Disparate Impact." *California Law Review* 104: 671–732. https://doi.org/10.2139/ssrn.2477899.

Barroso, Luiz André, Urs Hölzle, and Parthasarathy Ranganathan. 2019. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*. Synthesis Lectures on Computer Architecture. Springer International Publishing. https://doi.org/10.1007/978-3-031-01761-2.

Barroso, Luiz, Mike Marty, David Patterson, and Parthasarathy Ranganathan. 2017. "Attack of the Killer Microseconds." *Communications of the ACM* 60 (4): 48–54. https://doi.org/10.1145/3015146.

Baur, Walter, and Volker Strassen. 1983. "The Complexity of Partial Derivatives." *Theoretical Computer Science* 22 (3): 317–30. https://doi.org/10.1016/0304-3975(83)90110-x.

Baydin, Atilim Gunes, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. 2018. "Automatic Differentiation in Machine Learning: A Survey." *Journal of Machine Learning Research* 18 (153): 1–43.

Baylor, Denis, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, et al. 2017. "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform." *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1387–95. https://doi.org/10.1145/3097983.3098021.

Bedford Taylor, Michael. 2017. "The Evolution of Bitcoin Hardware." *Computer* 50 (9): 58–66. https://doi.org/10.1109/mc.2017.3571056.

Beede, Emma, Elizabeth Baylor, Fred Hersch, Anna Iurchenko, Lauren Wilcox, Paisan Ruamviboonsuk, and Laura M. Vardoulakis. 2020. "A Human-Centered Evaluation of a Deep Learning System Deployed in Clinics for the Detection of Diabetic Retinopathy." *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, 1–12. https://doi.org/10.1145/3313831.3376718.

Bellamy, Rachel K. E., Kuntal Dey, Michael Hind, Seung Hoffman, Sylvain Houde, Karthikeyan Kannan, Pradeep Lohia, et al. 2019. "AI Fairness 360: An Extensible Toolkit for Detecting and Mitigating Algorithmic Bias." *IBM Journal of Research and Development* 63 (4/5): 4:1–15. https://doi.org/10.1147/jrd.2019.2942287.

Bender, Emily M., and Batya Friedman. 2018. "Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science." *Transactions of the Association for Computational Linguistics* 6: 587–604. https://doi.org/10.1162/tacl_a_00041.

Bengio, Emmanuel, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. 2015. "Conditional Computation in Neural Networks for Faster Models." *arXiv Preprint arXiv:1511.06297*.

Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2013. "Representation Learning: A Review and New Perspectives." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8): 1798–828. https://doi.org/10.1109/tpami.2013.50.

Bengio, Yoshua, Nicholas Léonard, and Aaron Courville. 2013. "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation." *arXiv Preprint arXiv:1308.3432*.

Bengio, Yoshua, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. "Curriculum Learning." *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48. https://doi.org/10.1145/1553374.1553380.

Berger, Vance W., and YanYan Zhou. 2014. "Kolmogorov–Smirnov Test: Overview." In *Wiley StatsRef: Statistics Reference Online*. Wiley. https://doi.org/10.1002/9781118445112.stat06558.

Bergstra, James, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. 2010. "Theano: A CPU and GPU Math Compiler in Python." *Proceedings of the Python in Science Conference* 4: 18–24. https://doi.org/10.25080/majora-92bf1922-003.

Beyer, Betsy, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.

Beyer, Lucas, Olivier J. Hénaff, Alexander Kolesnikov, Xiaohua Zhai, and Aäron van den Oord. 2020. "Are We Done with ImageNet?" *arXiv Preprint arXiv:2006.07159*.

Bird, Sarah, Miro Dudík, Richard Edgar, Brandon Horn, Roman Lutz, Vanessa Milan, Mehrnoosh Sameki, Hanna Wallach, and Kathleen Walker. 2020. "Fairlearn: A Toolkit for Assessing and Improving Fairness in AI." *Microsoft Technical Report MSR-TR-2020-32*.

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Springer.

Blalock, Davis, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. 2020. "What Is the State of Neural Network Pruning?" *Proceedings of Machine Learning and Systems (MLSys)* 2: 129–46.

Bobrow, Daniel G. 1964. "Natural Language Input for a Computer Problem Solving System." PhD thesis, Massachusetts Institute of Technology.

Boehm, Barry W. 1981. *Software Engineering Economics*. Prentice-Hall.

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam Tauman Kalai. 2016. "Man Is to Computer Programmer as Woman Is to Homemaker? Debiasing Word Embeddings." *Advances in Neural Information Processing Systems (NeurIPS)*, 4349–57.

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, et al. 2021. "On the Opportunities and Risks of Foundation Models." *arXiv Preprint arXiv:2108.07258*.

Bonawitz, Keith, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. "Practical Secure Aggregation for Privacy-Preserving Machine Learning." *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1175–91. https://doi.org/10.1145/3133956.3133982.

Boroumand, Amirali, Saugata Ghose, Youngsok Kim, Rachata Ausavarungnirun, Eric Shiu, Rahul Thakur, Daehyun Kim, et al. 2018. "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '18, 316–31. https://doi.org/10.1145/3173162.3173177.

Bourtoule, Lucas, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. "Machine Unlearning." *2021 IEEE Symposium on Security and Privacy (SP)*, 141–59. https://doi.org/10.1109/sp40001.2021.00019.

Bradbury, James, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*.

Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. 2017. "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction." *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data)*, 1123–32. https://doi.org/10.1109/bigdata.2017.8258038.

Breck, Eric, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2019. "Data Validation for Machine Learning." *Proceedings of the 2nd SysML Conference* 1: 334–47.

Brewer, Eric A. 2000. "Towards Robust Distributed Systems (Abstract)." *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*, 7. https://doi.org/10.1145/343477.343502.

Broder, A. Z. 1997. "On the Resemblance and Containment of Documents." *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No.97TB100171)*, 21–29. https://doi.org/10.1109/sequen.1997.666900.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. "Language Models Are Few-Shot Learners." *Advances in Neural Information Processing Systems* 33: 1877–901. https://doi.org/10.48550/arxiv.2005.14165.

Brutlag, Jake. 2009. *Speed Matters for Google Web Search*. Google Research Blog.

Buolamwini, Joy, and Timnit Gebru. 2018. "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification." *Conference on Fairness, Accountability and Transparency*, 77–91.

Burns, Brendan, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. "Borg, Omega, and Kubernetes." *Communications of the ACM* 59 (5): 50–57. https://doi.org/10.1145/2890784.

Cai, Han, Chuang Gan, Ligeng Zhu, and Song Han. 2020. "TinyTL: Reduce Activations, Not Trainable Parameters for Efficient on-Device Learning." *Advances in Neural Information Processing Systems* 33: 11285–97.

Campbell, Murray, Jr. Hoane A.Joseph, and Feng-hsiung Hsu. 2002. "Deep Blue." *Artificial Intelligence* 134 (1-2): 57–83. https://doi.org/10.1016/s0004-3702(01)00129-1.

Cao, Yinzhi, and Junfeng Yang. 2015. "Towards Making Systems Forget with Machine Unlearning." *2015 IEEE Symposium on Security and Privacy*, 463–80. https://doi.org/10.1109/sp.2015.35.

Chapman, Pete, Julian Clinton, Randy Kerber, Thomas Khabaza, Thomas Reinartz, Colin Shearer, and Rudiger Wirth. 2000. "CRISP-DM 1.0: Step-by-Step Data Mining Guide." *SPSS Inc* 1: 78.

Chen, Andrew, Andy Chow, Aaron Davidson, Arjun DCunha, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, et al. 2020. "Developments in MLflow: A System to Accelerate the Machine Learning Lifecycle." *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, 1–4. https://doi.org/10.1145/3399579.3399867.

Chen, Emma, Shvetank Prakash, Vijay Janapa Reddi, David Kim, and Pranav Rajpurkar. 2023. "A Framework for Integrating Artificial Intelligence for Clinical Care with Continuous Therapeutic Monitoring." *Nature Biomedical Engineering* 9 (4): 445–54. https://doi.org/10.1038/s41551-023-01115-0.

Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. 2021. "Evaluating Large Language Models Trained on Code." *arXiv Preprint arXiv:2107.03374*.

Chen, Tianqi, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, et al. 2018. "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning." *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, 578–94.

Chen, Tianqi, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. "Training Deep Nets with Sublinear Memory Cost." *arXiv Preprint arXiv:1604.06174*.

Chen, Tianshi, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. "DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning." *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14)*, 269–84. https://doi.org/10.1145/2541940.2541967.

Chen, Ting, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. "A Simple Framework for Contrastive Learning of Visual Representations." *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, vol. 119: 1597–607.

Chen, Yanxi, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. "EE-LLM: Large-Scale Training and Inference of Early-Exit Large Language Models with 3D Parallelism." *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Chen, Yu-Hsin, Joel S. Emer, and Vivienne Sze. 2016. "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks." *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 367–79. https://doi.org/10.1109/ISCA.2016.40.

Chen, Yu-Hsin, Joel Emer, and Vivienne Sze. 2017. "Using Dataflow to Optimize Energy Efficiency of Deep Neural Network Accelerators." *IEEE Micro* 37 (3): 12–21. https://doi.org/10.1109/mm.2017.54.

Chetlur, Sharan, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. "cuDNN: Efficient Primitives for Deep Learning." *arXiv Preprint arXiv:1410.0759*.

Cho, Aeree, Grace C. Kim, Alexander Karpekov, Alec Helbling, Zijie J. Wang, Seongmin Lee, Benjamin Hoover, and Duen Horng (Polo) Chau. 2025. "TRANSFORMER EXPLAINER: Interactive Learning of Text-Generative Models." *Proceedings of the AAAI Conference on Artificial Intelligence* 39 (28): 29625–27. https://doi.org/10.1609/aaai.v39i28.35347.

Cho, Kyunghyun, Bart van Merrienboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. "On the Properties of Neural Machine Translation: Encoder-Decoder Approaches." *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, 103–11. https://doi.org/10.3115/v1/w14-4012.

Choi, Dami, Alexandre Passos, Christopher J. Shallue, and George E. Dahl. 2019. "Faster Neural Network Training with Data Echoing." *arXiv Preprint*, ahead of print. https://doi.org/10.48550/arXiv.1907.05550.

Chollet, François. 2018. *Deep Learning with Python*. Manning Publications.

Chollet, François et al. 2024. *Keras*.

Choquette, Jack. 2023. "NVIDIA Hopper H100 GPU: Scaling Performance." *IEEE Micro* 43 (3): 9–17. https://doi.org/10.1109/mm.2023.3256796.

Choquette, Jack, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. "NVIDIA A100 Tensor Core GPU: Performance and Innovation." *IEEE Micro* 41 (2): 29–35. https://doi.org/10.1109/mm.2021.3061394.

Choudhary, Tejalal, Vipul Mishra, Anurag Goswami, and Jagannathan Sarangapani. 2020. "A Comprehensive Survey on Model Compression and Acceleration." *Artificial Intelligence Review* 53 (7): 5113–55. https://doi.org/10.1007/s10462-020-09816-7.

Choukroun, Yoni, Eli Kravchik, Fan Yang, and Pavel Kisilev. 2019. "Low-Bit Quantization of Neural Networks for Efficient Inference." *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, 3009–18. https://doi.org/10.1109/iccvw.2019.00363.

Chouldechova, Alexandra. 2017. "Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments." *Big Data* 5 (2): 153–63. https://doi.org/10.1089/big.2016.0047.

Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, et al. 2022. "PaLM: Scaling Language Modeling with Pathways." *arXiv Preprint arXiv:2204.02311*.

Chu, Grace, Okan Arikan, Gabriel Bender, Weijun Wang, Achille Brighton, Pieter-Jan Kindermans, Hanxiao Liu, Berkin Akin, Suyog Gupta, and Andrew Howard. 2021. "Discovering Multi-Hardware Mobile Models via Architecture Search." *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3016–25. https://doi.org/10.1109/cvprw53098.2021.00337.

CircleCI. 2024. *CircleCI: Continuous Integration and Delivery Platform*.

Clark, Kevin, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019. "What Does BERT Look at? An Analysis of BERT's Attention." *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 276–86. https://doi.org/10.18653/v1/w19-4828.

Cloud Native Computing Foundation. 2024a. *Kubernetes: Production-Grade Container Orchestration*.

Cloud Native Computing Foundation. 2024b. *Prometheus: Monitoring System and Time Series Database*.

Cohen, Jacob. 1960. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement* 20 (1): 37–46. https://doi.org/10.1177/001316446002000104.

Cohen, Taco, and Max Welling. 2016. "Group Equivariant Convolutional Networks." *Proceedings of the 33rd International Conference on Machine Learning (ICML)* 48: 2990–99.

Coleman, Cody, Edward Chou, Julian Katz-Samuels, Sean Culatana, Peter Bailis, Alexander C. Berg, Robert Nowak, Roshan Sumbaly, Matei Zaharia, and I. Zeki Yalniz. 2022. "Similarity Search for Efficient Active Learning and Search of Rare Concepts." *Proceedings of the AAAI Conference on Artificial Intelligence* 36 (6): 6402–10. https://doi.org/10.1609/aaai.v36i6.20591.

Coleman, Cody, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. 2019. "Analysis of DAWNBench, a Time-to-Accuracy Machine Learning Performance Benchmark." *ACM SIGOPS Operating Systems Review* 53 (1): 14–25. https://doi.org/10.1145/3352020.3352024.

Courbariaux, Matthieu, Yoshua Bengio, and Jean-Pierre David. 2015. "BinaryConnect: Training Deep Neural Networks with Binary Weights During Propagations." *Advances in Neural Information Processing Systems (NeurIPS)* 28: 3123–31.

Cover, Thomas M., and Joy A. Thomas. 2006. *Elements of Information Theory*. 2nd ed. Wiley. https://doi.org/10.1002/047174882X.

Covington, Paul, Jay Adams, and Emre Sargin. 2016. "Deep Neural Networks for YouTube Recommendations." *Proceedings of the 10th ACM Conference on Recommender Systems*, 191–98. https://doi.org/10.1145/2959100.2959190.

Crankshaw, Daniel, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. "Clipper: A Low-Latency Online Prediction Serving System." *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 613–27.

CrowdFlower. 2016. *Data Science Report 2016*.

Cubuk, Ekin D., Barret Zoph, Dandelion Mané, Vijay Vasudevan, and Quoc V. Le. 2019. "AutoAugment: Learning Augmentation Strategies from Data." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 113–23. https://doi.org/10.1109/cvpr.2019.00020.

Cubuk, Ekin D., Barret Zoph, Jonathon Shlens, and Quoc V. Le. 2020. "Randaugment: Practical Automated Data Augmentation with a Reduced Search Space." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3008–17. https://doi.org/10.1109/cvprw50498.2020.00359.

Cunningham, Ward. 1992. "The WyCash Portfolio Management System." *ACM SIGPLAN OOPS Messenger* 4 (2): 29–30. https://doi.org/10.1145/157710.157715.

Curnow, H. J., and B. A. Wichmann. 1976. "A Synthetic Benchmark." *The Computer Journal* 19 (1): 43–49. https://doi.org/10.1093/comjnl/19.1.43.

Cybenko, George. 1989. "Approximation by Superpositions of a Sigmoidal Function." *Mathematics of Control, Signals, and Systems* 2 (4): 303–14. https://doi.org/10.1007/bf02551274.

Dalal, Navneet, and Bill Triggs. 2005. "Histograms of Oriented Gradients for Human Detection." *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)* 1: 886–93. https://doi.org/10.1109/cvpr.2005.177.

Dally, Bill. 2023. "Hardware for Deep Learning." *2023 IEEE Hot Chips 35 Symposium (HCS)*, 1–58. https://doi.org/10.1109/hcs59251.2023.10254716.

Dally, William J., Stephen W. Keckler, and David B. Kirk. 2021. "Evolution of the Graphics Processing Unit (GPU)." *IEEE Micro* 41 (6): 42–51. https://doi.org/10.1109/mm.2021.3113475.

Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. 2022. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 16344–59. https://doi.org/10.52202/068431-1189.

Dao, Tri. 2023. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." *arXiv Preprint arXiv:2307.08691*.

Dao, Tri, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. 2022. "Monarch: Expressive Structured Matrices for Efficient and Accurate Training." *arXiv Preprint*.

Dastin, Jeffrey. 2018. "Amazon Scraps Secret AI Recruiting Tool That Showed Bias Against Women." *Reuters*.

Data Protection Commission. 2023. *Data Protection Commission Announces Conclusion of Two Inquiries into Meta Ireland*. Regulatory press release.

Databricks. 2024. *MLflow: An Open Source Platform for the Machine Learning Lifecycle*.

David, Robert, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, et al. 2021. "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems." *Proceedings of Machine Learning and Systems* 3: 800–811.

dbt Labs. 2024. *Dbt (Data Build Tool)*.

Dean, Jeff, David Patterson, and Cliff Young. 2018. "A New Golden Age in Computer Architecture: Empowering the Machine-Learning Revolution." *IEEE Micro* 38 (2): 21–29. https://doi.org/10.1109/mm.2018.112130030.

Dean, Jeffrey. 2012. *Achieving Rapid Response Times in Large Online Services*. Berkeley AMPLab Cloud Seminar.

Dean, Jeffrey, and Luiz André Barroso. 2013. "The Tail at Scale." *Communications of the ACM* 56 (2): 74–80. https://doi.org/10.1145/2408776.2408794.

Dean, Jeffrey, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, et al. 2012. "Large Scale Distributed Deep Networks." In *Advances in Neural Information Processing Systems (NeurIPS)*, edited by Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger, vol. 25. Curran Associates.

Dean, Jeffrey, and Sanjay Ghemawat. 2004. "MapReduce: Simplified Data Processing on Large Clusters." *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 137–50.

DeepSeek. 2024. *Introducing DeepSeek-V3*.

Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. "ImageNet: A Large-Scale Hierarchical Image Database." *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–55. https://doi.org/10.1109/cvpr.2009.5206848.

Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2024. *ImageNet*.

Dennard, Robert H., Frank H. Gaensslen, Hwa-Nien Yu, Victor L. Rideout, Elias Bassous, and Antoine R. LeBlanc. 1974. "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions." *IEEE J. Solid-State Circuits* 9 (5): 256–68. https://doi.org/10.1109/jssc.1974.1050511.

Denning, Peter J. 1968. "The Working Set Model for Program Behavior." *Communications of the ACM* 11 (5): 323–33. https://doi.org/10.1145/363095.363141.

Dettmers, Tim, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. "8-Bit Optimizers via Block-Wise Quantization." *International Conference on Learning Representations (ICLR)*.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171–86. https://doi.org/10.18653/v1/n19-1423.

DeVries, Terrance, and Graham W. Taylor. 2017. "Improved Regularization of Convolutional Neural Networks with Cutout." *arXiv Preprint arXiv:1708.04552*.

Dixit, Kaivalya M. 1993. "Overview of the SPEC Benchmarks." In *The Benchmark Handbook for Database and Transaction Systems*, 2nd ed., edited by Jim Gray. Morgan Kaufmann.

Dongarra, J. J., J. R. Bunch, C. B. Moler, and G. W. Stewart. 1979. *LINPACK Users' Guide*. SIAM. https://doi.org/10.1137/1.9781611971811.

Dongarra, Jack J., Jeremy Du Croz, Sven Hammarling, and Richard J. Hanson. 1988. "An Extended Set of FORTRAN Basic Linear Algebra Subprograms." *ACM Transactions on Mathematical Software* 14 (1): 1–17. https://doi.org/10.1145/42288.42291.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, et al. 2021. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *International Conference on Learning Representations (ICLR)*.

Dua, Dheeru, and Casey Graff. 2024. *UCI Machine Learning Repository*.

Duarte, Javier, Nhan Tran, Ben Hawks, Christian Herwig, Jules Muhizi, Shvetank Prakash, and Vijay Janapa Reddi. 2022. "FastML Science Benchmarks: Accelerating Real-Time Scientific Edge Machine Learning." *arXiv Preprint arXiv:2207.07958*.

Dubey, Abhimanyu, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. *The Llama 3 Herd of Models*. arXiv preprint arXiv:2407.21783.

Dwork, Cynthia. 2008. "Differential Privacy: A Survey of Results." In *Theory and Applications of Models of Computation*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-79228-4_1.

Dwork, Cynthia, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. "Calibrating Noise to Sensitivity in Private Data Analysis." In *Theory of Cryptography Conference (TCC)*, edited by Shai Halevi and Tal Rabin, vol. 3876. Lecture Notes in Computer Science. Springer Berlin Heidelberg. https://doi.org/10.1007/11681878_14.

Elastic NV. 2024. *Elasticsearch: Distributed Search and Analytics Engine*.

Elman, Jeffrey L. 1990. "Finding Structure in Time." *Cognitive Science* 14 (2): 179–211. https://doi.org/10.1207/s15516709cog1402_1.

Elsken, Thomas, Jan Hendrik Metzen, and Frank Hutter. 2019. "Neural Architecture Search." In *The Springer Series on Challenges in Machine Learning*, vol. 20. Springer International Publishing. https://doi.org/10.1007/978-3-030-05318-5_3.

Engineering, M. 2016. *Introducing FBLearner Flow: Facebook's AI Backbone*. Engineering at Meta Blog.

Epoch AI. 2024. *Machine Learning Trends*. Epoch AI Research Database.

Equal Employment Opportunity Commission, Civil Service Commission, Department of Labor, and Department of Justice. 1978. *Uniform Guidelines on Employee Selection Procedures*. Code of Federal Regulations.

Esmaeilzadeh, Hadi, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. 2011. "Dark Silicon and the End of Multicore Scaling." *Proceedings of the 38th Annual International Symposium on Computer Architecture*, 365–76. https://doi.org/10.1145/2000064.2000108.

Ethernet Alliance. 2025. *2025 Ethernet Roadmap*. Ethernet Alliance roadmap.

European Data Protection Board. 2018. *Guidelines on Automated Individual Decision-Making and Profiling for the Purposes of Regulation 2016/679*.

European Parliament and Council of the European Union. 2024. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence (AI Act)*. Official Journal of the European Union, L 2024/1689.

European Parliament, and Council of the European Union. 2016. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016*. Official Journal of the European Union.

European Space Agency. 1996. *Ariane 501: Presentation of Inquiry Board Report*. European Space Agency press release.

Everingham, Mark, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. 2009. "The Pascal Visual Object Classes (VOC) Challenge." *International Journal of Computer Vision* 88 (2): 303–38. https://doi.org/10.1007/s11263-009-0275-4.

Fedus, William, Barret Zoph, and Noam Shazeer. 2022. "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity." *Journal of Machine Learning Research* 23 (120): 1–39.

Feigenbaum, Edward A. 1984. "Knowledge Engineering: The Applied Side of Artificial Intelligence." *Annals of the New York Academy of Sciences* 426: 91–107. https://doi.org/10.1111/j.1749-6632.1984.tb16513.x.

Feng, Wu-chun, and Kirk W. Cameron. 2007. "The Green500 List: Encouraging Sustainable Supercomputing." *Computer* 40 (12): 50–55. https://doi.org/10.1109/MC.2007.445.

Fitzpatrick, Thomas B. 1988. "The Validity and Practicality of Sun-Reactive Skin Types i Through VI." *Archives of Dermatology* 124 (6): 869. https://doi.org/10.1001/archderm.1988.01670060015008.

FlatBuffers Authors. 2026. *FlatBuffers Documentation*.

Fleiss, Joseph L. 1971. "Measuring Nominal Scale Agreement Among Many Raters." *Psychological Bulletin* 76 (5): 378–82. https://doi.org/10.1037/h0031619.

Flynn, M. J. 1966. "Very High-Speed Computing Systems." *Proceedings of the IEEE* 54 (12): 1901–9. https://doi.org/10.1109/proc.1966.5273.

Fowler, Martin. 2014. *Canary Release*. Martin Fowler's Blog.

Frankle, Jonathan, and Michael Carbin. 2019. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks." *International Conference on Learning Representations (ICLR)*.

Fredrikson, Matt, Somesh Jha, and Thomas Ristenpart. 2015. "Model Inversion Attacks That Exploit Confidence Information and Basic Countermeasures." *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 1322–33. https://doi.org/10.1145/2810103.2813677.

Frostig, Roy, Matthew James Johnson, and Chris Leary. 2018. "Compiling Machine Learning Programs via High-Level Tracing." *Systems for Machine Learning*.

Gabor, D. 1946. "Theory of Communication. Part 1: The Analysis of Information." *Journal of the Institution of Electrical Engineers - Part III: Radio and Communication Engineering* 93 (26): 429–41. https://doi.org/10.1049/ji-3-2.1946.0074.

Gadre, Samir Yitzhak, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, et al. 2023. "DataComp: In Search of the Next Generation of Multimodal Datasets." *Advances in Neural Information Processing Systems (NeurIPS)* 36: 27092–112. https://doi.org/10.52202/075280-1179.

Gale, Trevor, Erich Elsen, and Sara Hooker. 2019. "The State of Sparsity in Deep Neural Networks." *arXiv Preprint arXiv:1902.09574*.

Gale, Trevor, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2022. "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts." *arXiv Preprint*.

Gale, Trevor, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. "Sparse GPU Kernels for Deep Learning." *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* 2020: 8955–67. https://doi.org/10.1109/sc41405.2020.00021.

Gama, João, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. "A Survey on Concept Drift Adaptation." *ACM Computing Surveys* 46 (4): 1–37. https://doi.org/10.1145/2523813.

Gao, Leo, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, et al. 2020. "The Pile: An 800GB Dataset of Diverse Text for Language Modeling." *arXiv Preprint arXiv:2101.00027*.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86–92. https://doi.org/10.1145/3458723.

Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. "A Survey of Quantization Methods for Efficient Neural Network Inference." *arXiv Preprint arXiv:2103.13630* abs/2103.13630.

Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2022. "A Survey of Quantization Methods for Efficient Neural Network Inference." In *Low-Power Computer Vision: Improve the Efficiency of Artificial Intelligence*. Chapman; Hall/CRC. https://doi.org/10.1201/9781003162810-13.

Gholami, Amir, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. "AI and Memory Wall." *IEEE Micro* 44 (3): 33–39. https://doi.org/10.1109/mm.2024.3373763.

Gilbert, Seth, and Nancy Lynch. 2002. "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services." *ACM SIGACT News* 33 (2): 51–59. https://doi.org/10.1145/564585.564601.

Ginsberg, Jeremy, Matthew H. Mohebbi, Rajan S. Patel, Lynnette Brammer, Mark S. Smolinski, and Larry Brilliant. 2009. "Detecting Influenza Epidemics Using Search Engine Query Data." *Nature* 457 (7232): 1012–14. https://doi.org/10.1038/nature07634.

GitHub, Inc. 2024a. *GitHub*.

GitHub, Inc. 2024b. *GitHub Actions*.

Glorot, Xavier, and Yoshua Bengio. 2010. "Understanding the Difficulty of Training Deep Feedforward Neural Networks." *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* 9: 249–56.

Gojek, and Google. 2019. *Feast: An Open Source Feature Store for Machine Learning*. Google Cloud Blog.

Goodfellow, I. J., J. Shlens, and C. Szegedy. 2015. "Explaining and Harnessing Adversarial Examples." *International Conference on Learning Representations (ICLR)*.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

Goodhart, Charles A. E. 1984. "Problems of Monetary Management: The UK Experience." In *Monetary Theory and Practice*. Palgrave. https://doi.org/10.1007/978-1-349-17295-5_4.

Google. 2024a. *Our Next-Generation Model: Gemini 1.5*.

Google. 2024b. *Static Vs. Dynamic Inference*. Google Machine Learning Crash Course.

Google. 2025. *XLA: Optimizing Compiler for Machine Learning*.

Google Cloud. 2024a. *Google Cloud Platform Documentation*. Https://cloud.google.com/docs.

Google Cloud. 2024b. *Google Cloud Storage*.

Google Cloud. 2024c. *Vertex AI Model Registry*.

Google Cloud. 2026a. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.

Google Cloud. 2026b. *TPU v6e*.

Google Developers Blog. 2024. *Gemini 1.5 Flash Price Drop with Tuning Rollout Complete, and More*.

Gordon, Mitchell, Kevin Duh, and Nicholas Andrews. 2020. "Compressing BERT: Studying the Effects of Weight Pruning on Transfer Learning." *Proceedings of the 5th Workshop on Representation Learning for NLP*, 143–55. https://doi.org/10.18653/v1/2020.repl4nlp-1.18.

Goto, Kazushige, and Robert A. van de Geijn. 2008. "Anatomy of High-Performance Matrix Multiplication." *ACM Transactions on Mathematical Software* 34 (3): 1–25. https://doi.org/10.1145/1356052.1356053.

Gou, Jianping, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. 2021. "Knowledge Distillation: A Survey." *International Journal of Computer Vision* 129 (6): 1789–819. https://doi.org/10.1007/s11263-021-01453-z.

Goyal, Priya, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour." *arXiv Preprint arXiv:1706.02677* abs/1706.02677.

Graphcore. 2020. *The Colossus MK2 IPU Processor*.

Gray, Allison. 2015. *NVIDIA and IBM Cloud Support ImageNet Large Scale Visual Recognition Challenge*. NVIDIA Technical Blog.

Gretton, Arthur, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. 2012. "A Kernel Two-Sample Test." *Journal of Machine Learning Research* 13 (25): 723–73.

Groeneveld, Dirk, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, et al. 2024. "OLMo: Accelerating the Science of Language Models." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15789–809. https://doi.org/10.18653/v1/2024.acl-long.841.

gRPC Authors. 2026. *Introduction to gRPC*.

Gudivada, Venkat N., Amy Apon, and Junhua Ding. 2017. "Data Quality Considerations for Big Data and Machine Learning: Going Beyond Data Cleaning and Transformations." *International Journal on Advances in Software* 10 (1–2): 1–20.

Gujarati, Arpan, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. "Clockwork: Predictable and Scalable DNN Inference in the Cloud." *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 443–62.

Gulshan, Varun, Lily Peng, Marc Coram, Martin C. Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, et al. 2016. "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs." *JAMA* 316 (22): 2402. https://doi.org/10.1001/jama.2016.17216.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. "On Calibration of Modern Neural Networks." *International Conference on Machine Learning (ICML)* 70: 1321–30.

Gupta, Suyog, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. "Deep Learning with Limited Numerical Precision." *International Conference on Machine Learning (ICML)*, 1737–46.

Gupta, Udit, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2022. "Chasing Carbon: The Elusive Environmental Footprint of Computing." *IEEE Micro* 42 (4): 37–47. https://doi.org/10.1109/MM.2022.3163226.

Gupta, Udit, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, et al. 2020. "The Architectural Implications of Facebook's DNN-Based Personalized Recommendation." *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 488–501. https://doi.org/10.1109/HPCA47549.2020.00047.

Gustafson, John L. 1988. "Reevaluating Amdahl's Law." *Communications of the ACM* 31 (5): 532–33. https://doi.org/10.1145/42411.42415.

Halevy, Alon, Peter Norvig, and Fernando Pereira. 2009. "The Unreasonable Effectiveness of Data." *IEEE Intelligent Systems* 24 (2): 8–12. https://doi.org/10.1109/mis.2009.36.

Han, Song, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. "EIE: Efficient Inference Engine on Compressed Deep Neural Network." *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 243–54. https://doi.org/10.1109/isca.2016.30.

Han, Song, Huizi Mao, and William J. Dally. 2016. "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding." *arXiv Preprint*.

Han, Song, Jeff Pool, John Tran, and William J. Dally. 2015. "Learning Both Weights and Connections for Efficient Neural Networks." *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, 1135–43.

Harchol-Balter, Mor. 2013. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press. https://doi.org/10.1017/cbo9781139226424.

Hardt, Moritz, Eric Price, and Nathan Srebro. 2016. "Equality of Opportunity in Supervised Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 29: 3315–23.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585 (7825): 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Harvard Law School. 2024. *Does ChatGPT Violate New York Times Copyrights?*

HashiCorp. 2014. *Terraform: Infrastructure as Code*. Software available from https://www.terraform.io/.

Hatcher, Blake Douglas. 2024. "Automating Server Deployments with Ansible: Utilizing Automation in DevOps." *Journal of Computing Sciences in Colleges* 40 (3): 42–43.

Hazelwood, Kim, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, et al. 2018. "Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective." *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 620–29. https://doi.org/10.1109/hpca.2018.00059.

He, Kaiming, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. "Momentum Contrast for Unsupervised Visual Representation Learning." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9726–35. https://doi.org/10.1109/cvpr42600.2020.00975.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." *2015 IEEE International Conference on Computer Vision (ICCV)*, 1026–34. https://doi.org/10.1109/iccv.2015.123.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016a. "Deep Residual Learning for Image Recognition." *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–78. https://doi.org/10.1109/cvpr.2016.90.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016b. "Identity Mappings in Deep Residual Networks." *Computer Vision – ECCV 2016*, 630–45. https://doi.org/10.1007/978-3-319-46493-0_38.

Henderson, Peter, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. 2020. "Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning." *CoRR* abs/2002.05651 (248): 1–43. https://doi.org/10.48550/arxiv.2002.05651.

Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. "Deep Reinforcement Learning That Matters." *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (1): 3207–14. https://doi.org/10.1609/aaai.v32i1.11694.

Hendler, James A. 2008. "Avoiding Another AI Winter." *IEEE Intelligent Systems* 23 (2): 2–4. https://doi.org/10.1109/MIS.2008.20.

Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. "Measuring Massive Multitask Language Understanding." *arXiv Preprint*.

Hendrycks, Dan, and Kevin Gimpel. 2016. "Gaussian Error Linear Units (GELUs)." *arXiv Preprint arXiv:1606.08415*.

Hennessy, J. L., and D. A. Patterson. 2011. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.

Hennessy, John L., and David A. Patterson. 2017. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann.

Hennessy, John L., and David A. Patterson. 2019. "A New Golden Age for Computer Architecture." *Communications of the ACM* 62 (2): 48–60. https://doi.org/10.1145/3282307.

Hermann, Jeremy, and Mike Del Balso. 2017. *Meet Michelangelo: Uber's Machine Learning Platform*. Uber Engineering Blog.

Hernandez, Danny, and Tom B. Brown. 2020. "Measuring the Algorithmic Efficiency of Neural Networks." *arXiv Preprint arXiv:2005.04305*, ahead of print. https://doi.org/10.48550/arxiv.2005.04305.

Hestness, Joel, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. 2017. "Deep Learning Scaling Is Predictable, Empirically." *arXiv Preprint arXiv:1712.00409*.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. "Distilling the Knowledge in a Neural Network." *arXiv Preprint*.

Hirschberg, Julia, and Christopher D. Manning. 2015. "Advances in Natural Language Processing." *Science* 349 (6245): 261–66. https://doi.org/10.1126/science.aaa8685.

Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. "Long Short-Term Memory." *Neural Computation* 9 (8): 1735–80. https://doi.org/10.1162/neco.1997.9.8.1735.

Hoefler, Torsten, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. 2021. "Sparsity in Deep Learning: Pruning and Growth for Efficient Inference and Training in Neural Networks." *Journal of Machine Learning Research* 22 (241): 1–124.

Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, et al. 2022. "Training Compute-Optimal Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 30016–30. https://doi.org/10.52202/068431-2176.

Holtzman, Ari, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. "The Curious Case of Neural Text Degeneration." *Proceedings of the 8th International Conference on Learning Representations (ICLR 2020)*.

Hooker, Sara. 2021. "The Hardware Lottery." *Communications of the ACM* 64 (12): 58–65. https://doi.org/10.1145/3467017.

Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer Feedforward Networks Are Universal Approximators." *Neural Networks* 2 (5): 359–66. https://doi.org/10.1016/0893-6080(89)90020-8.

Horowitz, Mark. 2014. "1.1 Computing's Energy Problem (and What We Can Do about It)." *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 10–14. https://doi.org/10.1109/isscc.2014.6757323.

Howard, A. G., M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. 2017. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." *CoRR* abs/1704.04861.

Howard, Andrew, Mark Sandler, Bo Chen, Weijun Wang, Liang-Chieh Chen, Mingxing Tan, Grace Chu, et al. 2019. "Searching for MobileNetV3." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 1314–24. https://doi.org/10.1109/iccv.2019.00140.

Howarth, Jesse. 2020. "Why Discord Is Switching from Go to Rust."

Hu, Jie, Peng Lin, Huajun Zhang, Zining Lan, Wenxin Chen, Kailiang Xie, Siyun Chen, Hao Wang, and Sheng Chang. 2023. "A Dynamic Pruning Method on Multiple Sparse Structures in Deep Neural Networks." *IEEE Access* 11: 38448–57. https://doi.org/10.1109/access.2023.3267469.

Hu, Ting-Kuei, Tianlong Chen, Haotao Wang, and Zhangyang Wang. 2020. "Triple Wins: Boosting Accuracy, Robustness and Efficiency Together by Enabling Input-Adaptive Inference." *International Conference on Learning Representations (ICLR)*.

Huang, Gao, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. 2017. "Densely Connected Convolutional Networks." *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2261–69. https://doi.org/10.1109/cvpr.2017.243.

Huang, Y., Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, et al. 2019. "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 103–12.

Hubara, Itay, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2018. "Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations." *Journal of Machine Learning Research* 18 (187): 1–30.

Hugging Face. 2026. *Safetensors Documentation*.

HumanSignal. 2024. *Label Studio: Open Source Data Labeling Platform*.

Hutter, Frank, Lars Kotthoff, and Joaquin Vanschoren. 2019. *Automated Machine Learning: Methods, Systems, Challenges*. In *Automated Machine Learning*. The Springer Series on Challenges in Machine Learning. Springer International Publishing. https://doi.org/10.1007/978-3-030-05318-5.

Hwu, Wen-mei W. 2011. "Introduction." In *GPU Computing Gems Emerald Edition*. Elsevier. https://doi.org/10.1016/b978-0-12-384988-5.00064-4.

Iandola, Forrest N., Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. "SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <0.5MB Model Size." *ArXiv Preprint* abs/1602.07360.

IBM. 2024. *IBM Watson OpenScale: Data Drift Detection*.

IEEE Standards Association. 2019. *IEEE 2416-2019: Standard for Power Modeling to Enable System-Level Analysis*.

IEEE Standards Association. 2024. *IEEE Standards Association: Working Groups for AI and ML*.

Ignatov, Andrey, and Radu Timofte. 2024. *AI Benchmark*.

InfiniBand Trade Association. 2000. *InfiniBand Architecture Specification Volume 1*. InfiniBand Trade Association.

Intel Corporation. 2021a. "Intel Advanced Matrix Extensions (Intel AMX)." *Intel Architecture Instruction Set Extensions Programming Reference*.

Intel Corporation. 2021b. *oneDNN: Intel's Deep Learning Neural Network Library*.

Intel Corporation. 2026a. *Intel oneAPI Math Kernel Library*.

Intel Corporation. 2026b. *OpenVINO Documentation*.

Ioffe, Sergey, and Christian Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." *Proceedings of the 32nd International Conference on Machine Learning (ICML)* 37: 448–56.

ISO. 2024. *ISO/IEC JTC 1/SC 42 Artificial Intelligence*.

Iterative. 2024. *Data Version Control (DVC)*.

Jacob, Benoit, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–13. https://doi.org/10.1109/cvpr.2018.00286.

Janapa Reddi, Vijay et al. 2022. "MLPerf Mobile V2. 0: An Industry-Standard Benchmark Suite for Mobile Machine Learning." *Proceedings of Machine Learning and Systems (MLSys)* 4: 806–23.

Janapa Reddi, Vijay, Brian Plancher, Susan Kennedy, Laurence Moroney, Pete Warden, Lara Suzuki, Anant Agarwal, et al. 2022. "Widening Access to Applied Machine Learning with TinyML." *Harvard Data Science Review* 4 (1). https://doi.org/10.1162/99608f92.762d171a.

Jevons, William Stanley. 1865. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal Mines*. Macmillan; Co.

Jia, Yangqing, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. "Caffe: Convolutional Architecture for Fast Feature Embedding." *Proceedings of the 22nd ACM International Conference on Multimedia*, 675–78. https://doi.org/10.1145/2647868.2654889.

Jia, Zhihao, James Thomas, Todd Warszawski, Mingyu Gao, Matei Zaharia, and Alex Aiken. 2019. "Optimizing DNN Computation with Relaxed Graph Substitutions." *Proceedings of Machine Learning and Systems (MLSys)*.

Jiang, A. Q., A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, et al. 2024. "Mixtral of Experts." *arXiv Preprint arXiv:2401.04088*.

Jiao, Xiaoqi, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. "TinyBERT: Distilling BERT for Natural Language Understanding." *Findings of the Association for Computational Linguistics: EMNLP 2020*, 4163–74. https://doi.org/10.18653/v1/2020.findings-emnlp.372.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2019. "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data* 7 (3): 535–47. https://doi.org/10.1109/tbdata.2019.2921572.

Johnston, Casey. 2012. "Netflix Never Used Its $1 Million Algorithm Due to Engineering Costs." *Ars Technica*, April.

Jordan, T. L. 1982. "A Guide to Parallel Computation and Some Cray-1 Experiences." In *Parallel Computations*. Elsevier. https://doi.org/10.1016/b978-0-12-592101-5.50006-3.

Jouppi, Norman P., Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, et al. 2021. "Ten Lessons from Three Generations Shaped Google's TPUv4i: Industrial Product." *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 1–14. https://doi.org/10.1109/isca52012.2021.00010.

Jouppi, Norman P., Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. "A Domain-Specific Supercomputer for Training Deep Neural Networks." *Communications of the ACM* 63 (7): 67–78. https://doi.org/10.1145/3360307.

Jouppi, Norman P., Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, et al. 2017. "In-Datacenter Performance Analysis of a Tensor Processing Unit." *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ISCA '17, 1–12. https://doi.org/10.1145/3079856.3080246.

Jouppi, Norm, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, et al. 2023. "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 1–14. https://doi.org/10.1145/3579371.3589350.

Jumper, John, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, et al. 2021. "Highly Accurate Protein Structure Prediction with AlphaFold." *Nature* 596 (7873): 583–89. https://doi.org/10.1038/s41586-021-03819-2.

Kalamkar, Dhiraj, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, et al. 2019. *A Study of BFLOAT16 for Deep Learning Training*.

Kamiran, Faisal, and Toon Calders. 2012. "Data Preprocessing Techniques for Classification Without Discrimination." *Knowledge and Information Systems* 33 (1): 1–33. https://doi.org/10.1007/s10115-011-0463-8.

Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. "Scaling Laws for Neural Language Models." *ArXiv Preprint* abs/2001.08361.

Karpathy, Andrej. 2017. *Software 2.0*. Medium.

Kiela, Douwe, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, et al. 2021. "Dynabench: Rethinking Benchmarking in NLP." *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* 9: 4110–24. https://doi.org/10.18653/v1/2021.naacl-main.324.

Kilburn, Tom, David B. G. Edwards, Michael J. Lanigan, and Frank H. Sumner. 1962. "One-Level Storage System." *IRE Transactions on Electronic Computers* EC-11 (2): 223–35. https://doi.org/10.1109/TEC.1962.5219356.

Kim, Wonyoung, Meeta S. Gupta, Gu-Yeon Wei, and David Brooks. 2008. "System Level Analysis of Fast, Per-Core DVFS Using on-Chip Switching Regulators." *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, HPCA '08, 123–34. https://doi.org/10.1109/hpca.2008.4658633.

Kingma, Diederik P., and Jimmy Ba. 2015. "Adam: A Method for Stochastic Optimization." *3rd International Conference on Learning Representations (ICLR)*.

Kipf, Thomas N, and Max Welling. 2017. "Semi-Supervised Classification with Graph Convolutional Networks." *International Conference on Learning Representations (ICLR)*.

Kleinberg, Jon, Sendhil Mullainathan, and Manish Raghavan. 2017. "Inherent Trade-Offs in the Fair Determination of Risk Scores." *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, Leibniz international proceedings in informatics (LIPIcs), vol. 67: 43:1–23. https://doi.org/10.4230/LIPIcs.ITCS.2017.43.

Kleppmann, Martin. 2016. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Knight, Will. 2023. *OpenAI's CEO Says the Age of Giant AI Models Is Already over*. WIRED.

Koh, Pang Wei, and Percy Liang. 2017. "Understanding Black-Box Predictions via Influence Functions." *International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 70: 1885–94.

Koh, Pang Wei, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, et al. 2021. "WILDS: A Benchmark of in-the-Wild Distribution Shifts." In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, edited by Marina Meila and Tong Zhang, vol. 139. Proceedings of Machine Learning Research. PMLR.

Koizumi, Yuma, Shoichiro Saito, Hisashi Uematsu, Noboru Harada, and Keisuke Imoto. 2019. "ToyADMOS: A Dataset of Miniature-Machine Operating Sounds for Anomalous Sound Detection." *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 313–17. https://doi.org/10.1109/waspaa.2019.8937164.

Koomey, Jonathan, Stephen Berard, Marla Sanchez, and Henry Wong. 2011. "Implications of Historical Trends in the Electrical Efficiency of Computing." *IEEE Annals of the History of Computing* 33 (3): 46–54. https://doi.org/10.1109/mahc.2010.28.

Kreuzberger, Dominik, Niklas Kühl, and Sebastian Hirschl. 2023. "Machine Learning Operations (MLOps): Overview, Definition, and Architecture." *IEEE Access* 11: 31866–79. https://doi.org/10.1109/access.2023.3262138.

Krishnamoorthi, Raghuraman. 2018. "Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper." *arXiv Preprint arXiv:1806.08342* abs/1806.08342.

Krishnan, Rayan, Pranav Rajpurkar, and Eric J. Topol. 2022. "Self-Supervised Learning in Medicine and Healthcare." *Nature Biomedical Engineering* 6 (12): 1346–52. https://doi.org/10.1038/s41551-022-00914-1.

Krizhevsky, Alex. 2009. *Learning Multiple Layers of Features from Tiny Images*. University of Toronto.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 25.

Kuhn, Max, and Kjell Johnson. 2013. *Applied Predictive Modeling*. Springer New York. https://doi.org/10.1007/978-1-4614-6849-3.

Kullback, Solomon, and Richard A. Leibler. 1951. "On Information and Sufficiency." *The Annals of Mathematical Statistics* 22 (1): 79–86. https://doi.org/10.1214/aoms/1177729694.

Kung, H. T. 1982. "Why Systolic Architectures?" *Computer* 15 (1): 37–46. https://doi.org/10.1109/mc.1982.1653825.

Kung, Hsiang Tsung, and Charles E Leiserson. 1979. "Systolic Arrays (for VLSI)." *Sparse Matrix Proceedings 1978* 1: 256–82.

Kuzmin, Andrey, Mart Van Baalen, Yuwei Ren, Markus Nagel, Jorn Peters, and Tijmen Blankevoort. 2022. "FP8 Quantization: The Power of the Exponent." *Advances in Neural Information Processing Systems (NeurIPS)*, 14651–62. https://doi.org/10.52202/068431-1065.

Kuznetsova, Alina, Hassan Rom, Neil Alldrin, Jasper Uijlings, Ivan Krasin, Jordi Pont-Tuset, Shahab Kamali, et al. 2020. "The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale." *International Journal of Computer Vision* 128 (7): 1956–81. https://doi.org/10.1007/s11263-020-01316-z.

Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–26. https://doi.org/10.1145/3600006.3613165.

Kwon, Youngeun, and Minsoo Rhu. 2018. "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning." *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 740–53. https://doi.org/10.1145/3352460.3358284.

Labelbox, Inc. 2024. *Labelbox Consensus Documentation*.

Labs, Grafana. 2024. *Grafana*.

Lai, Liangzhen, Naveen Suda, and Vikas Chandra. 2018. "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs." *ArXiv Preprint* abs/1801.06601.

Lam, Monica D., Edward E. Rothberg, and Michael E. Wolf. 1991. "The Cache Performance and Optimizations of Blocked Algorithms." *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS-IV*, 63–74. https://doi.org/10.1145/106972.106981.

Lampson, Butler W. 1983. "Hints for Computer System Design." *Proceedings of the Ninth ACM Symposium on Operating Systems Principles*, 33–48. https://doi.org/10.1145/800217.806614.

Landis, J. Richard, and Gary G. Koch. 1977. "The Measurement of Observer Agreement for Categorical Data." *Biometrics* 33 (1): 159–74. https://doi.org/10.2307/2529310.

Lange, Klaus-Dieter. 2009. "Identifying Shades of Green: The SPECpower Benchmarks." *IEEE Computer* 42 (3): 95–97. https://doi.org/10.1109/mc.2009.84.

Lattner, Chris, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation." *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2–14. https://doi.org/10.1109/cgo51591.2021.9370308.

Lawson, Charles L., Richard J. Hanson, David R. Kincaid, and Fred T. Krogh. 1979. "Basic Linear Algebra Subprograms for Fortran Usage." *ACM Transactions on Mathematical Software* 5 (3): 308–23. https://doi.org/10.1145/355841.355847.

Lazer, David, Ryan Kennedy, Gary King, and Alessandro Vespignani. 2014. "The Parable of Google Flu: Traps in Big Data Analysis." *Science* 343 (6176): 1203–5. https://doi.org/10.1126/science.1248506.

Le Sueur, Etienne, and Gernot Heiser. 2010. "Dynamic Voltage and Frequency Scaling: The Laws of Diminishing Returns." *Proceedings of the 2010 International Conference on Power Aware Computing and Systems*, 1–8.

Lebedev, Vadim, Yaroslav Ganin, Maksim Rakhuba, Ivan Oseledets, and Victor Lempitsky. 2015. "Speeding up Convolutional Neural Networks Using Fine-Tuned CP-Decomposition." *3rd International Conference on Learning Representations (ICLR)*.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. "Deep Learning." *Nature* 521 (7553): 436–44. https://doi.org/10.1038/nature14539.

LeCun, Yann, Bernhard Boser, John S. Denker, Donald Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. 1989. "Backpropagation Applied to Handwritten Zip Code Recognition." *Neural Computation* 1 (4): 541–51. https://doi.org/10.1162/neco.1989.1.4.541.

LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE* 86 (11): 2278–324. https://doi.org/10.1109/5.726791.

LeCun, Yann, Leon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. 2012. "Efficient BackProp." In *Neural Networks: Tricks of the Trade*, vol. 7700. Lecture Notes in Computer Science. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-35289-8_3.

LeCun, Yann, John S. Denker, and Sara A. Solla. 1989. "Optimal Brain Damage." *Advances in Neural Information Processing Systems 2 (NIPS 1989)*, 598–605.

Lee, Katherine, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. "Deduplicating Training Data Makes Language Models Better." *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8424–45. https://doi.org/10.18653/v1/2022.acl-long.577.

Lee, Peter. 2016. *Learning from Tay's Introduction*. The Official Microsoft Blog.

Lepikhin, Dmitry, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding." *International Conference on Learning Representations (ICLR)*.

Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2023. "Fast Inference from Transformers via Speculative Decoding." *International Conference on Machine Learning (ICML)*, 19274–86.

Li, Hao, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2017. "Pruning Filters for Efficient ConvNets." *International Conference on Learning Representations (ICLR)*.

Li, Mingzhen, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. "The Deep Learning Compiler: A Comprehensive Survey." *IEEE Transactions on Parallel and Distributed Systems* 32 (3): 708–27. https://doi.org/10.1109/tpds.2020.3030548.

Li, Ninghui, Tiancheng Li, and Suresh Venkatasubramanian. 2007. "t-Closeness: Privacy Beyond k-Anonymity and l-Diversity." *2007 IEEE 23rd International Conference on Data Engineering*, 106–15. https://doi.org/10.1109/ICDE.2007.367856.

Liang, Percy, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, et al. 2022. "Holistic Evaluation of Language Models." *arXiv Preprint arXiv:2211.09110*.

Lighthill, James. 1973. "Artificial Intelligence: A General Survey." In *Artificial Intelligence: A Paper Symposium*. Science Research Council.

Lin, Ji, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. "MCUNet: Tiny Deep Learning on IoT Devices." In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, Virtual*, edited by Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin. Curran Associates.

Lin, Ji, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. "AWQ: Activation-Aware Weight Quantization for on-Device LLM Compression and Acceleration." *Proceedings of Machine Learning and Systems* 6: 87–100.

Lin, Tsung-Yi, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. "Microsoft COCO: Common Objects in Context." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-319-10602-1_48.

Lindholm, Erik, John Nickolls, Stuart Oberman, and John Montrym. 2008. "NVIDIA Tesla: A Unified Graphics and Computing Architecture." *IEEE Micro* 28 (2): 39–55. https://doi.org/10.1109/mm.2008.31.

Linnainmaa, Seppo. 1970. "The Representation of the Cumulative Rounding Error of an Algorithm as a Taylor Expansion of the Local Rounding Errors." Master's thesis, University of Helsinki.

Little, John D. C. 1961. "A Proof for the Queuing Formula: $L = \lambda W$." *Operations Research* 9 (3): 383–87. https://doi.org/10.1287/opre.9.3.383.

Liu, Hanxiao, Karen Simonyan, and Yiming Yang. 2019. "DARTS: Differentiable Architecture Search." *International Conference on Learning Representations (ICLR)*.

Liu, Ze, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows." *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 9992–10002. https://doi.org/10.1109/ICCV48922.2021.00986.

Liu, Zhuang, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. 2022. "A ConvNet for the 2020s." *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11966–76. https://doi.org/10.1109/CVPR52688.2022.01167.

Loshchilov, Ilya, and Frank Hutter. 2019. "Decoupled Weight Decay Regularization." *Proceedings of the International Conference on Learning Representations (ICLR)*.

Lowe, David G. 1999. "Object Recognition from Local Scale-Invariant Features." *Proceedings of the Seventh IEEE International Conference on Computer Vision* 2: 1150–1157 vol.2. https://doi.org/10.1109/iccv.1999.790410.

Lu, Yucheng, Shivani Agrawal, Suvinay Subramanian, Oleg Rybakov, Christopher De Sa, and Amir Yazdanbakhsh. 2023. "STEP: Learning n:m Structured Sparsity Masks from Scratch with Precondition." *arXiv Preprint*.

Lucic, Mario, Karol Kurach, Marcin Michalski, Sylvain Gelly, and Olivier Bousquet. 2018. "Are GANs Created Equal? A Large-Scale Study." *Advances in Neural Information Processing Systems (NeurIPS)* 31.

Lundberg, Scott M., and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems (NeurIPS)* 30: 4765–74.

Lyons, Richard G. 2011. *Understanding Digital Signal Processing*. 3rd ed. Prentice Hall.

Machanavajjhala, Ashwin, Daniel Kifer, Johannes Gehrke, and Muthu Venkitasubramaniam. 2007. "l-Diversity: Privacy Beyond k-Anonymity." *ACM Transactions on Knowledge Discovery from Data* 1 (3): 3. https://doi.org/10.1145/1217299.1217302.

Mars Climate Orbiter Mishap Investigation Board. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase i Report*. National Aeronautics; Space Administration.

Maslej, Nestor, Loredana Fattorini, C. Raymond Perrault, Vanessa Parli, Anka Reuel, Erik Brynjolfsson, John Etchemendy, et al. 2024. "Artificial Intelligence Index Report 2024." In *CoRR*, abs/2405.19522. https://doi.org/10.48550/ARXIV.2405.19522.

Mattson, Peter, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, et al. 2020. "MLPerf Training Benchmark." *Proceedings of Machine Learning and Systems (MLSys)* 2: 336–49.

Mazumder, Mark. 2021. *TinyML: Multilingual Spoken Words Corpus*. tinyML Talks.

Mazumder, Mark, Colby Banbury, Xiaozhe Yao, Bojan Karlaš, et al. 2023. "DataPerf: Benchmarks for Data-Centric AI Development." *Advances in Neural Information Processing Systems* 36.

Mazumder, Mark, Sharad Chitlangia, Colby Banbury, Yiping Kang, Juan Manuel Ciro, Keith Achorn, Daniel Galvez, et al. 2021. "Multilingual Spoken Words Corpus." *Advances in Neural Information Processing Systems 35 (NeurIPS 2021) Datasets and Benchmarks Track*.

McCarthy, John, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. 1955. *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955*. No. 4. Vol. 27. Dartmouth College. https://doi.org/10.1609/aimag.v27i4.1904.

McCrory, Dave. 2010. *Data Gravity – in the Clouds*.

McCulloch, Warren S., and Walter Pitts. 1943. "A Logical Calculus of the Ideas Immanent in Nervous Activity." *The Bulletin of Mathematical Biophysics* 5 (4): 115–33. https://doi.org/10.1007/bf02478259.

McKinsey Global Institute. 2021. *The Internet of Things: Catching up to an Accelerating Opportunity*. McKinsey & Company.

McMahan, Brendan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. "Communication-Efficient Learning of Deep Networks from Decentralized Data." *International Conference on Artificial Intelligence and Statistics (AISTATS)*, Proceedings of machine learning research, vol. 54: 1273–82.

Mellempudi, Naveen, Sudarshan Srinivasan, Dipankar Das, and Bharat Kaul. 2019. "Mixed Precision Training with 8-Bit Floating Point." *arXiv Preprint arXiv:1905.12334*.

Merity, Stephen. 2016. *The WikiText Long Term Dependency Language Modeling Dataset*.

Merity, Stephen, Caiming Xiong, James Bradbury, and Richard Socher. 2016. "Pointer Sentinel Mixture Models." *arXiv Preprint arXiv:1609.07843*.

Merkel, Dirk. 2014. "Docker: Lightweight Linux Containers for Consistent Development and Deployment." *Linux Journal* 2014 (239).

Michel, Jean-Baptiste, Yuan Kui Shen, Aviva Presser Aiden, Adrian Veres, Matthew K. Gray, William Brockman, Joseph P. Pickett, et al. 2011. "Quantitative Analysis of Culture Using Millions of Digitized Books." *Science* 331 (6014): 176–82. https://doi.org/10.1126/science.1199644.

Michel, Paul, Omer Levy, and Graham Neumann. 2019. "Are Sixteen Heads Really Better Than One?" *Advances in Neural Information Processing Systems (NeurIPS)*.

Micikevicius, Paulius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, et al. 2017. "Mixed Precision Training." *arXiv Preprint arXiv:1710.03740*.

Micikevicius, Paulius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, et al. 2022. "FP8 Formats for Deep Learning." *arXiv Preprint arXiv:2209.05433*.

Microsoft. 2024a. *Microsoft Azure*.

Microsoft. 2024b. *ONNX Runtime: Cross-Platform Inference and Training Machine-Learning Accelerator*. GitHub.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Efficient Estimation of Word Representations in Vector Space." *ICLR* abs/1301.3781.

Mila. 2026. *Open Source Software*.

Minsky, Marvin, and Seymour A. Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press.

Mirhoseini, Azalia, Hieu Pham, Quoc V. Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. 2017. "Device Placement Optimization with Reinforcement Learning." *International Conference on Machine Learning (ICML)* 70: 2430–39.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. "Model Cards for Model Reporting." *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–29. https://doi.org/10.1145/3287560.3287596.

Mitchell, Tom M. 1980. *The Need for Biases in Learning Generalizations*. CBM-TR-117. Rutgers University, Department of Computer Science.

MLCommons. 2024a. *MLPerf Mobile Benchmark*.

MLCommons. 2024b. *MLPerf Power Measurement*.

MLCommons. 2024c. *MLPerf Training Benchmark*.

MLCommons. 2026a. *MLCommons: About Us*.

MLCommons. 2026b. *MLPerf Client Benchmark*.

MLflow Project. 2026. *MLflow Model Registry*.

Moore, G. E. 1998. "Cramming More Components onto Integrated Circuits." *Proceedings of the IEEE* 86 (1): 82–85. https://doi.org/10.1109/jproc.1998.658762.

Moravec, Hans. 1988. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press.

Moritz, Philipp, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, et al. 2018. "Ray: A Distributed Framework for Emerging AI Applications." *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 561–77.

Mosseri, Adam. 2018. *Bringing People Closer Together*. Meta Newsroom.

Murray, Derek G., Jiří Šimša, Ana Klimovic, and Ihor Indyk. 2021. "Tf.data: A Machine Learning Data Processing Framework." *Proceedings of the VLDB Endowment* 14 (12): 2945–58. https://doi.org/10.14778/3476311.3476374.

Nair, Vinod, and Geoffrey E. Hinton. 2010. "Rectified Linear Units Improve Restricted Boltzmann Machines." *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 807–14.

Narayanan, Deepak, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. "PipeDream: Generalized Pipeline Parallelism for DNN Training." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 1–15. https://doi.org/10.1145/3341301.3359646.

Narayanan, Deepak, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, et al. 2021. "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15. https://doi.org/10.1145/3458817.3476209.

National Aeronautics and Space Administration. 2016. *NASA Systems Engineering Handbook*. NASA/SP-2016-6105 Rev2. National Aeronautics; Space Administration.

National Transportation Safety Board. 2017. *Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. HAR-17/02. National Transportation Safety Board.

National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. HAR-19/03. National Transportation Safety Board.

Naumov, Maxim, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, et al. 2019. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." *arXiv Preprint arXiv:1906.00091*.

Naur, Peter, and Brian Randell, eds. 1969. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968*. Scientific Affairs Division, NATO.

Nayak, Prateeth, Takuya Higuchi, Anmol Gupta, Shivesh Ranjan, Stephen Shum, Siddharth Sigtia, Erik Marchi, et al. 2022. "Improving Voice Trigger Detection with Metric Learning." *Interspeech 2022*, 1896–900. https://doi.org/10.21437/interspeech.2022-11160.

Netflix. 2024. *Metaflow*.

Newell, Allen, and Herbert A. Simon. 1976. "Computer Science as Empirical Inquiry: Symbols and Search." *Communications of the ACM* 19 (3): 113–26. https://doi.org/10.1145/360018.360022.

Neyshabur, Behnam, Srinadh Bhojanapalli, David McAllester, and Nati Srebro. 2017. "Exploring Generalization in Deep Learning." *Advances in Neural Information Processing Systems* 30.

Ng, Andrew. 2021. *MLOps: From Model-Centric to Data-Centric AI*. DeepLearning.AI.

Nickolls, John, Ian Buck, Michael Garland, and Kevin Skadron. 2008. "Scalable Parallel Programming with CUDA: Is CUDA the Parallel Programming Model That Application Developers Have Been Waiting For?" *Queue* 6 (2): 40–53. https://doi.org/10.1145/1365490.1365500.

Norrie, Thomas, Nishant Patil, Doe Hyun Yoon, George Kurian, Sheng Li, James Laudon, Cliff Young, Norman Jouppi, and David Patterson. 2021. "The Design Process for Google's Training Chips: TPUv2 and TPUv3." *IEEE Micro* 41 (2): 56–63. https://doi.org/10.1109/mm.2021.3058217.

Northcutt, Curtis G., Anish Athalye, and Jonas Mueller. 2021. “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks.” *arXiv Preprint arXiv:2103.14749* 34: 19075–90. https://doi.org/10.48550/arxiv.2103.14749.

NVIDIA. 2017. *Training with Mixed Precision*.

NVIDIA. 2020a. *Accelerating Matrix Multiplication with Block Sparse Format and NVIDIA Tensor Cores*.

NVIDIA. 2020b. *TensorFloat-32 in the A100 GPU Accelerates AI Training, HPC up to 20x*.

NVIDIA. 2024a. *cuBLAS: CUDA Basic Linear Algebra Subprograms*.

NVIDIA. 2024b. *NVIDIA Omniverse and Simulation*.

NVIDIA. 2024c. *NVIDIA TensorRT: Programmable Inference Accelerator*.

NVIDIA. 2024d. *NVIDIA Triton Inference Server*.

NVIDIA. 2026a. *Lazy Loading: CUDA Programming Guide*.

NVIDIA. 2026b. *Model Management: NVIDIA Triton Inference Server*.

NVIDIA. 2026c. *Model Repository: NVIDIA Triton Inference Server*.

NVIDIA. 2026d. *NVIDIA CUDA Multi-Process Service*.

NVIDIA. 2026e. *NVIDIA Multi-Instance GPU User Guide*.

NVIDIA. 2026f. *NVIDIA Quantum-X800 InfiniBand Platform*. NVIDIA product documentation.

NVIDIA. 2026g. *NVIDIA TensorRT-LLM*.

NVIDIA Corporation. 2017. *NVIDIA Tesla V100 GPU Architecture*. NVIDIA Whitepaper.

NVIDIA Corporation. 2018. *NVIDIA Tesla T4 Tensor Core GPU*. NVIDIA product documentation.

NVIDIA Corporation. 2020a. *NVIDIA A100 Tensor Core GPU Architecture*. NVIDIA Whitepaper, V1.0.

NVIDIA Corporation. 2020b. *NVIDIA V100 Tensor Core GPU Datasheet*. NVIDIA product documentation.

NVIDIA Corporation. 2020c. “NVLink: Scalable High-Performance Interconnect.” *NVIDIA Technical Report* 2020.

NVIDIA Corporation. 2021a. *NVIDIA A100 Tensor Core GPU Datasheet*. NVIDIA product documentation.

NVIDIA Corporation. 2021b. *NVIDIA cuDNN Developer Guide*.

NVIDIA Corporation. 2023. *NVIDIA cuDNN Developer Guide, Version 8.9.6*.

NVIDIA Corporation. 2024. *NVIDIA Blackwell Architecture*. NVIDIA product documentation.

NVIDIA Corporation. 2026a. *How TensorRT Works*.

NVIDIA Corporation. 2026b. *Working with Dynamic Shapes*.

Oakden-Rayner, Luke, Jared Dunnmon, Gustavo Carneiro, and Christopher Re. 2020. “Hidden Stratification Causes Clinically Meaningful Failures in Machine Learning for Medical Imaging.” *Proceedings of the ACM Conference on Health, Inference, and Learning*, 151–59. https://doi.org/10.1145/3368555.3384468.

Obermeyer, Ziad, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. 2019. “Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations.” *Science* 366 (6464): 447–53. https://doi.org/10.1126/science.aax2342.

Olston, Christopher, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. “TensorFlow-Serving: Flexible, High-Performance ML Serving.” *CoRR* abs/1712.06139. https://doi.org/10.48550/arXiv.1712.06139.

ONNX Contributors. 2019. *ONNX: Open Neural Network Exchange*.

OpenAI. 2023a. *GPT-4*.

OpenAI. 2023b. *Introducing ChatGPT and Whisper APIs*.

OpenAI. 2023c. *New Models and Developer Products Announced at DevDay*.

OpenAI. 2024. *GPT-4o Mini: Advancing Cost-Efficient Intelligence*.

OpenAI Developer Community. 2024. *Announcing GPT-4o in the API*.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, et al. 2023. "GPT-4 Technical Report." *arXiv Preprint arXiv:2303.08774*, ahead of print. https://doi.org/10.48550/arXiv.2303.08774.

OpenBLAS Project. 2026. *OpenBLAS: An Optimized BLAS Library*.

Orr, Laurel, Atindriyo Sanyal, Xiao Ling, Karan Goel, and Megan Leszczynski. 2021. "Managing ML Pipelines: Feature Stores and the Coming Wave of Embedding Ecosystems." *Proceedings of the VLDB Endowment* 14 (12): 3178–81. https://doi.org/10.14778/3476311.3476402.

Owens, John D., Mike Houston, David Luebke, Simon Green, John E. Stone, and James C. Phillips. 2008. "GPU Computing." *Proceedings of the IEEE* 96 (5): 879–99. https://doi.org/10.1109/jproc.2008.917757.

Paleyes, Andrei, Raoul-Gabriel Urma, and Neil D. Lawrence. 2022. "Challenges in Deploying Machine Learning: A Survey of Case Studies." *ACM Computing Surveys* 55 (6): 1–29. https://doi.org/10.1145/3533378.

Palmer, John F. 1980. "The Intel 8087 Numeric Data Processor." *Proceedings of the 1980 National Computer Conference (AFIPS)*, 887–93. https://doi.org/10.1145/1500518.1500674.

Papermill Project. 2026. *Papermill Documentation*.

Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. "Bleu: A Method for Automatic Evaluation of Machine Translation." *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, 311–18. https://doi.org/10.3115/1073083.1073135.

Pareto, Vilfredo. 1896. *Cours d'économie Politique*. F. Rouge.

Park, Daniel S., William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. 2019. "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition." *Interspeech 2019*, 2613–17. https://doi.org/10.21437/interspeech.2019-2680.

Park, Intaik, Ehsan Ardestani, Damian Reeves, Sarunya Pumma, Henry Tsang, Levy Zhao, Jian He, et al. 2024. "Toward 100TB Recommendation Models with Embedding Offloading." *18th ACM Conference on Recommender Systems*, 841–43. https://doi.org/10.1145/3640457.3688037.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, et al. 2019. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 8024–35.

Patel, Dylan, and Gerald Wong. 2023. *GPT-4 Architecture, Infrastructure, Training Dataset, Costs, Vision, MoE*. SemiAnalysis Blog.

Patterson, David, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. "Carbon Emissions and Large Neural Network Training." *arXiv Preprint arXiv:2104.10350*.

Paul, Mansheej, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. "Deep Learning on a Data Diet: Finding Important Examples Early in Training." *Advances in Neural Information Processing Systems (NeurIPS)* 34: 20596–607.

Pham, Hieu, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. 2018. "Efficient Neural Architecture Search via Parameter Sharing." *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 80: 4095–104.

Pichai, Sundar. 2023. *Introducing Gemini: Our Largest and Most Capable AI Model*.

Pineau, Joelle, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché-Buc, Emily Fox, and Hugo Larochelle. 2021. "Improving Reproducibility in Machine Learning Research (a Report from the Neurips 2019 Reproducibility Program)." *Journal of Machine Learning Research* 22 (164): 1–20.

Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. "Efficiently Scaling Transformer Inference." *Proceedings of Machine Learning and Systems (MLSys)* 5: 606–24.

Prefect Technologies, Inc. 2024. *Prefect: Workflow Orchestration Framework for Python*.

Project Jupyter. 2024. *Project Jupyter*.

Project Jupyter. 2026. *nbconvert: Convert Notebooks to Other Formats*.

Protocol Buffers Authors. 2026. *Overview: Protocol Buffers*.

Pushkarna, Mahima, Andrew Zaldivar, and Oddur Kjartansson. 2022. "Data Cards: Purposeful and Transparent Dataset Documentation for Responsible AI." *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 1776–826. https://doi.org/10.1145/3531146.3533231.

Putnam, Andrew, Adrian M. Caulfield, Eric S. Chung, Derek Chiou, Kypros Constantinides, John Demme, Hadi Esmaeilzadeh, et al. 2014. "A Reconfigurable Fabric for Accelerating Large-Scale Datacenter Services." *ACM SIGARCH Computer Architecture News* 42 (3): 13–24. https://doi.org/10.1145/2678373.2665678.

PyTorch. 2022. *Accelerating Neural Network Training with Sparse Tensors*.

PyTorch Contributors. 2026a. *PyTorch Autograd Mechanics*.

PyTorch Contributors. 2026b. *Tensor Views*.

PyTorch Contributors. 2026c. *torch.export*.

PyTorch Contributors. 2026d. *TorchScript*.

Qi, Chen, Shibo Shen, Rongpeng Li, Zhifeng Zhao, Qing Liu, Jing Liang, and Honggang Zhang. 2021. "An Efficient Pruning Scheme of Deep Neural Networks for Internet of Things Applications." *EURASIP Journal on Advances in Signal Processing* 2021 (1): 31. https://doi.org/10.1186/s13634-021-00744-4.

Quiñonero-Candela, Joaquin, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence, eds. 2009. *Dataset Shift in Machine Learning*. Neural Information Processing Series. The MIT Press.

Rachwan, John, Daniel Zügner, Bertrand Charpentier, Simon Geisler, Morgane Ayle, and Stephan Günnemann. 2022. "Winning the Lottery Ahead of Time: Efficient Early Network Pruning." *International Conference on Machine Learning (ICML)*, 18293–309.

Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." *International Conference on Machine Learning (ICML)*, 8748–63.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models Are Unsupervised Multitask Learners*. OpenAI.

Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. 2020. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." *Journal of Machine Learning Research* 21 (140): 1–67.

Rainio, Oona, Jarmo Teuho, and Riku Klén. 2024. "Evaluation Metrics and Statistical Tests for Machine Learning." *Scientific Reports* 14 (1): 6086. https://doi.org/10.1038/s41598-024-56706-x.

Raji, Inioluwa Deborah, and Joy Buolamwini. 2019. "Actionable Auditing: Investigating the Impact of Publicly Naming Biased Performance Results of Commercial AI Products." *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, 429–35. https://doi.org/10.1145/3306618.3314244.

Rajpurkar, Pranav, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. "SQuAD: 100,000+ Questions for Machine Comprehension of Text." *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2383–92. https://doi.org/10.18653/v1/d16-1264.

Rastegari, Mohammad, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. "XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-319-46493-0_32.

Ratner, A., S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré. 2017. "Snorkel: Rapid Training Data Creation with Weak Supervision." *Proceedings of the VLDB Endowment* 11 (3): 269–82. https://doi.org/10.14778/3157794.3157797.

Ratner, Alex, Braden Hancock, Jared Dunnmon, Roger Goldman, and Christopher Ré. 2018. "Snorkel MeTaL: Weak Supervision for Multi-Task Learning." *Proceedings of the Second Workshop on Data Management for End-to-End Machine Learning*, 1–4. https://doi.org/10.1145/3209889.3209898.

Ratner, Alex, Paroma Varma, Braden Hancock, and Christopher Ré. 2019. *Weak Supervision: A New Programming Paradigm for Machine Learning*. Stanford AI Lab Blog.

Reagen, Brandon, Robert Adolf, Paul Whatmough, Gu-Yeon Wei, and David Brooks. 2017. *Deep Learning for Computer Architects*. Synthesis Lectures on Computer Architecture. Springer International Publishing. https://doi.org/10.1007/978-3-031-01756-8.

Real, Esteban, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. "Regularized Evolution for Image Classifier Architecture Search." *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01): 4780–89. https://doi.org/10.1609/aaai.v33i01.33014780.

Recht, Benjamin, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. 2019. "Do ImageNet Classifiers Generalize to ImageNet?" *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 5389–400.

Reddi, Vijay Janapa, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, et al. 2019. "MLPerf Inference Benchmark." *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 446–59. https://doi.org/10.1109/isca45697.2020.00045.

Ren, Pengzhen, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xin Wang. 2021. "A Survey of Deep Active Learning." *ACM Computing Surveys* 54 (9): 1–40. https://doi.org/10.1145/3472291.

Ribeiro, M. H., R. Ottoni, R. West, V. A. F. Almeida, and Jr. Meira Wagner. 2020. "Auditing Radicalization Pathways on YouTube." *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 131–41. https://doi.org/10.1145/3351095.3372879.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?: Explaining the Predictions of Any Classifier." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–44. https://doi.org/10.1145/2939672.2939778.

Ribeiro, Marco Tulio, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList." *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–12. https://doi.org/10.18653/v1/2020.acl-main.442.

Roesch, Jared, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. "Relay: A New IR for Machine Learning Frameworks." *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 58–68. https://doi.org/10.1145/3211346.3211348.

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. 2022. "High-Resolution Image Synthesis with Latent Diffusion Models." *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10674–85. https://doi.org/10.1109/cvpr52688.2022.01042.

Romero, Francisco, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. "INFaaS: Automated Model-Less Inference Serving." *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 397–411.

Rosenblatt, Frank. 1957. *The Perceptron: A Perceiving and Recognizing Automaton*. Report Nos. 85-460-1. Cornell Aeronautical Laboratory.

Rosenblatt, Frank. 1958. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review* 65 (6): 386–408. https://doi.org/10.1037/h0042519.

Royce, Winston W. 1970. "Managing the Development of Large Software Systems." *Proceedings of IEEE WESCON* 26: 328–88.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. "Learning Representations by Back-Propagating Errors." *Nature* 323 (6088): 533–36. https://doi.org/10.1038/323533a0.

Russakovsky, Olga, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, et al. 2015. "ImageNet Large Scale Visual Recognition Challenge." *International Journal of Computer Vision* 115 (3): 211–52. https://doi.org/10.1007/s11263-015-0816-y.

Sambasivan, Nithya, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M Aroyo. 2021. "'Everyone Wants to Do the Model Work, Not the Data Work': Data Cascades in High-Stakes AI." *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–15. https://doi.org/10.1145/3411764.3445518.

Samuel, A. L. 1959. "Some Studies in Machine Learning Using the Game of Checkers." *IBM Journal of Research and Development* 3 (3): 210–29. https://doi.org/10.1147/rd.33.0210.

Sandler, Mark, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4510–20. https://doi.org/10.1109/cvpr.2018.00474.

Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter." *arXiv Preprint arXiv:1910.01108*, ahead of print. https://doi.org/10.48550/arXiv.1910.01108.

Savage, Sam L. 2009. *The Flaw of Averages: Why We Underestimate Risk in the Face of Uncertainty*. John Wiley & Sons.

Scale AI, Inc. 2024. *Scale AI Advanced Settings*.

Schelter, Sebastian, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. "Automating Large-Scale Data Quality Verification." *Proceedings of the VLDB Endowment* 11 (12): 1781–94. https://doi.org/10.14778/3229863.3229867.

Schwartz, Roy, Jesse Dodge, Noah A. Smith, and Oren Etzioni. 2020. "Green AI." *Communications of the ACM* 63 (12): 54–63. https://doi.org/10.1145/3381831.

scikit-learn developers. 2024. *Sklearn.metrics.confusion_matrix — Scikit-Learn Documentation*. Scikit-learn Documentation.

Scott, Colin. 2012. "Numbers Every Programmer Should Know by Year."

Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. 2015. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems (NeurIPS)* 28: 2503–11.

Sener, Ozan, and Silvio Savarese. 2018. "Active Learning for Convolutional Neural Networks: A Core-Set Approach." *International Conference on Learning Representations (ICLR)*.

Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. "Improving Neural Machine Translation Models with Monolingual Data." *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 86–96. https://doi.org/10.18653/v1/P16-1009.

Sergeev, Alexander, and Mike Del Balso. 2018. "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow." *CoRR* abs/1802.05799.

Settles, Burr. 2009. *Active Learning Literature Survey*. Computer Sciences Technical Report 1648. University of Wisconsin-Madison.

Settles, Burr. 2012. *Active Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Springer Cham. https://doi.org/10.1007/978-3-031-01560-1.

Sevilla, Jaime, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. 2022. "Compute Trends Across Three Eras of Machine Learning." *2022 International Joint Conference on Neural Networks (IJCNN)*, 1–8. https://doi.org/10.1109/ijcnn55064.2022.9891914.

Shah, Jay, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision." *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 68658–85. https://doi.org/10.52202/079017-2193.

Shannon, Claude E. 1948. "A Mathematical Theory of Communication." *Bell System Technical Journal* 27 (3): 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.

Shazeer, Noam, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer." *International Conference on Learning Representations (ICLR)*.

Shazeer, Noam, and Mitchell Stern. 2018. "Adafactor: Adaptive Learning Rates with Sublinear Memory Cost." *International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 80: 4596–604.

Shen, Haichen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. "Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 322–37. https://doi.org/10.1145/3341301.3359658.

Shen, Sheng, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. 2020. "Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT." *Proceedings of the AAAI Conference on Artificial Intelligence* 34 (05): 8815–21. https://doi.org/10.1609/aaai.v34i05.6409.

Sheng, Victor S., and Jing Zhang. 2019. "Machine Learning with Crowdsourcing: A Brief Summary of the Past Research and Future Directions." *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01): 9837–43. https://doi.org/10.1609/aaai.v33i01.33019837.

Shi, Weisong, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. "Edge Computing: Vision and Challenges." *IEEE Internet of Things Journal* 3 (5): 637–46. https://doi.org/10.1109/jiot.2016.2579198.

Shneiderman, B. 2022. *Human-Centered AI*. Oxford University Press. https://doi.org/10.1093/oso/9780192845290.001.0001.

Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." *arXiv Preprint arXiv:1909.08053*.

Shokri, Reza, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. "Membership Inference Attacks Against Machine Learning Models." *2017 IEEE Symposium on Security and Privacy (SP)*, 3–18. https://doi.org/10.1109/SP.2017.41.

Shorten, Connor, and Taghi M. Khoshgoftaar. 2019. "A Survey on Image Data Augmentation for Deep Learning." *Journal of Big Data* 6 (1): 1–48. https://doi.org/10.1186/s40537-019-0197-0.

Shortliffe, Edward H., Randall Davis, Stanton G. Axline, Bruce G. Buchanan, C.Cordell Green, and Stanley N. Cohen. 1975. "Computer-Based Consultations in Clinical Therapeutics: Explanation and Rule Acquisition Capabilities of the MYCIN System." *Computers and Biomedical Research* 8 (4): 303–20. https://doi.org/10.1016/0010-4809(75)90009-9.

Shumailov, Ilia, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. 2024. "AI Models Collapse When Trained on Recursively Generated Data." *Nature* 631 (8022): 755–59. https://doi.org/10.1038/s41586-024-07566-y.

Sifre, Laurent, and Stéphane Mallat. 2014. "Rigid-Motion Scattering for Image Classification." *arXiv Preprint arXiv:1403.1687*, ahead of print. https://doi.org/10.48550/arXiv.1403.1687.

Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, et al. 2016. "Mastering the Game of Go with Deep Neural Networks and Tree Search." *Nature* 529 (7587): 484–89. https://doi.org/10.1038/nature16961.

Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, et al. 2017. "Mastering the Game of Go Without Human Knowledge." *Nature* 550 (7676): 354–59. https://doi.org/10.1038/nature24270.

Simonyan, Karen, and Andrew Zisserman. 2015. "Very Deep Convolutional Networks for Large-Scale Image Recognition." *arXiv Preprint*.

Smith, Steven W. 1997. "Digital Signal Processing." In *Digital Signal Processing Demystified*. Elsevier. https://doi.org/10.1016/b978-187870716-1/50004-4.

Snow, Rion, Brendan O'Connor, Daniel Jurafsky, and Andrew Y. Ng. 2008. "Cheap and Fast—but Is It Good? Evaluating Non-Expert Annotations for Natural Language Tasks." *Proceedings of the Conference on Empirical Methods in Natural Language Processing - EMNLP '08*, 254. https://doi.org/10.3115/1613715.1613751.

Sohn, Kihyuk, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. 2020. "FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 596–608.

Sokolova, Marina, and Guy Lapalme. 2009. "A Systematic Analysis of Performance Measures for Classification Tasks." *Information Processing &Amp; Management* 45 (4): 427–37. https://doi.org/10.1016/j.ipm.2009.03.002.

Sorscher, Ben, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. 2022. "Beyond Neural Scaling Laws: Beating Power Law Scaling via Data Pruning." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 19523–36. https://doi.org/10.52202/068431-1419.

Soviany, Petru, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. 2022. "Curriculum Learning: A Survey." *International Journal of Computer Vision* 130 (6): 1526–65. https://doi.org/10.1007/s11263-022-01611-x.

Statista Research Department. 2024. *Number of Sent and Received e-Mails Per Day Worldwide from 2017 to 2027*. Statista.

Stephens, Nigel, Stuart Biles, Matthias Boettcher, Jacob Eapen, Mbou Eyole, Giacomo Gabrielli, Matt Horsnell, et al. 2017. "The ARM Scalable Vector Extension." *IEEE Micro* 37 (2): 26–39. https://doi.org/10.1109/mm.2017.35.

Stonebraker, Mike, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, et al. 2018. "C-Store: A Column-Oriented DBMS." In *Making Databases Work: The Pragmatic Wisdom of Michael Stonebraker*. Association for Computing Machinery. https://doi.org/10.1145/3226595.3226638.

Strathern, Marilyn. 1997. "'Improving Ratings': Audit in the British University System." *European Review* 5 (3): 305–21. https://doi.org/10.1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4.

Stromberg, Joseph. 2015. *When Was the First Traffic Light Installed? Today in 1914*. Vox.

Strubell, Emma, Ananya Ganesh, and Andrew McCallum. 2019. "Energy and Policy Considerations for Deep Learning in NLP." *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3645–50. https://doi.org/10.18653/v1/p19-1355.

Sullivan, Gary J., Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. 2012. "Overview of the High Efficiency Video Coding (HEVC) Standard." *IEEE Transactions on Circuits and Systems for Video Technology* 22 (12): 1649–68. https://doi.org/10.1109/tcsvt.2012.2221191.

Sun, Pei, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, et al. 2020. "Scalability in Perception for Autonomous Driving: Waymo Open Dataset." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2446–54. https://doi.org/10.1109/CVPR42600.2020.00252.

Supreme Court of the United States. 1971. *Griggs v. Duke Power Co., 401 U.S. 424*. Legal decision.

Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. "Sequence to Sequence Learning with Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 27: 3104–12.

Sutton, Richard S. 2019. "The Bitter Lesson." *Incompleteideas.net* 43.

Sweeney, Latanya. 2002. "K-ANONYMITY: A MODEL for PROTECTING PRIVACY." *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 10 (05): 557–70. https://doi.org/10.1142/s0218488502001648.

Systems, Cerebras. 2021. "Wafer-Scale Deep Learning Acceleration with the Cerebras CS-2." *Cerebras Technical Paper* 2021.

Sze, Vivienne, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2017. "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." *Proceedings of the IEEE* 105 (12): 2295–329. https://doi.org/10.1109/jproc.2017.2761740.

Szegedy, Christian, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. "Going Deeper with Convolutions." *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1–9. https://doi.org/10.1109/cvpr.2015.7298594.

Tan, Mingxing, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le. 2019. "MnasNet: Platform-Aware Neural Architecture Search for Mobile." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2815–23. https://doi.org/10.1109/cvpr.2019.00293.

Tan, Mingxing, and Quoc V Le. 2019. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *International Conference on Machine Learning (ICML)*, 6105–14.

Team, The Theano Development, Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermueller, Dzmitry Bahdanau, Nicolas Ballas, et al. 2016. "Theano: A Python Framework for Fast Computation of Mathematical Expressions." *arXiv Preprint*.

Teerapittayanon, Surat, Bradley McDanel, and H. T. Kung. 2017. "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks." *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2464–69. https://doi.org/10.1109/icpr.2016.7900006.

Telgarsky, Matus. 2016. "Benefits of Depth in Neural Networks." *Proceedings of the 29th Annual Conference on Learning Theory*, 1517–39.

TensorFlow Developers. 2024a. *Better Performance with tf.function*.

TensorFlow Developers. 2024b. *Optimize TensorFlow Performance Using the Profiler*.

Tesler, Larry. 1984. *The Law of Conservation of Complexity*. Web page.

Thyagarajan, Aditya, Elías Snorrason, Curtis G. Northcutt, and Jonas Mueller. 2022. "Identifying Incorrect Annotations in Multi-Label Classification Data." *CoRR* abs/2211.13895. https://doi.org/10.48550/ARXIV.2211.13895.

Tibshirani, Robert. 1996. "Regression Shrinkage and Selection via the Lasso." *Journal of the Royal Statistical Society: Series B (Methodological)* 58 (1): 267–88. https://doi.org/10.1111/j.2517-6161.1996.tb02080.x.

Tillet, Philippe, H. T. Kung, and David Cox. 2019. "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations." *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 10–19. https://doi.org/10.1145/3315508.3329973.

Toneva, Mariya, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J Gordon. 2019. "An Empirical Study of Example Forgetting During Deep Neural Network Learning." *International Conference on Learning Representations (ICLR)*.

Torvalds, Linus, and Junio Hamano. 2024. *Git*.

Touvron, Hugo, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, et al. 2023. "LLaMA: Open and Efficient Foundation Language Models." *arXiv Preprint arXiv:2302.13971*.

Touvron, Hugo, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. 2023. "Llama 2: Open Foundation and Fine-Tuned Chat Models." *arXiv Preprint arXiv:2307.09288*.

Tramèr, Florian, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. "Stealing Machine Learning Models via Prediction APIs." *25th USENIX Security Symposium (USENIX Security 16)*, 601–18.

Tschand, Arya, Arun Tejusve Raghunath Rajan, Sachin Idgunji, Anirban Ghosh, Jeremy Holleman, Csaba Kiraly, Pawan Ambalkar, et al. 2024. "MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μWatts to MWatts for Sustainable AI." *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 1201–16. https://doi.org/10.1109/hpca61900.2025.00092.

Turakhia, Yatish, Gill Bejerano, and William J. Dally. 2018. "Darwin: A Genomics Co-Processor Provides up to 15,000X Acceleration on Long Read Assembly." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 199–213. https://doi.org/10.1145/3173162.3173193.

Turing, Alan M. 1950. "I.—Computing Machinery and Intelligence." *Mind* LIX (236): 433–60. https://doi.org/10.1093/mind/lix.236.433.

Ultralytics. 2023. *YOLOv8*.

Umuroglu, Yaman, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. "Finn: A Framework for Fast, Scalable Binarized Neural Network Inference." *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 65–74. https://doi.org/10.1145/3020078.3021744.

United States Congress. 1996. *Health Insurance Portability and Accountability Act of 1996*. Public Law 104-191.

U.S. Department of Health and Human Services. 2003. "Summary of the HIPAA Privacy Rule."

U.S. Department of Health and Human Services. 2005. "Summary of the HIPAA Security Rule."

U.S. Department of Health and Human Services. 2026. *Annual Civil Monetary Penalties Inflation Adjustment*. Federal Register.

U.S. Food and Drug Administration. 2021. *Artificial Intelligence/Machine Learning (AI/ML)-Based Software as a Medical Device (SaMD) Action Plan*. U.S. Department of Health; Human Services.

U.S. Food and Drug Administration. 2025. *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*. Guidance for Industry and Food and Drug Administration Staff. U.S. Department of Health; Human Services.

U.S. Securities and Exchange Commission. 2013. "Securities Exchange Act of 1934, Release No. 70694: Knight Capital Americas LLC."

Vasisht, Deepak, Zerina Kapetanovic, Jongho Won, Xinxin Jin, Ranveer Chandra, Sudipta N. Sinha, Ashish Kapoor, Madhusudhan Sudarshan, and Sean Stratman. 2017. "FarmBeats: An IoT Platform for Data-Driven Agriculture." *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 515–29.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. "Attention Is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)* 30: 5998–6008.

Villalobos, Pablo, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. 2022. "Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data." *arXiv Preprint arXiv:2211.04325*.

Viola, Paul, and Michael J. Jones. 2001. "Rapid Object Detection Using a Boosted Cascade of Simple Features." *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001* 1: I-511-I-518. https://doi.org/10.1109/cvpr.2001.990517.

Wang, Alex, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems." *arXiv Preprint arXiv:1905.00537*.

Wang, Alex, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding." *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 353–55. https://doi.org/10.18653/v1/w18-5446.

Wang, Linnan, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. "SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks." *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 41–53. https://doi.org/10.1145/3178487.3178491.

Wang, Shibo, and Pankaj Kanwar. 2019. *BFloat16: The Secret to High Performance on Cloud TPUs*.

Wang, Tianlu, Jieyu Zhao, Mark Yatskar, Kai-Wei Chang, and Vicente Ordonez. 2019. "Balanced Datasets Are Not Enough: Estimating and Mitigating Gender Bias in Deep Image Representations." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 5309–18. https://doi.org/10.1109/iccv.2019.00541.

Wang, Xin, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. 2018. "SkipNet: Learning Dynamic Routing in Convolutional Networks." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-030-01261-8_25.

Wang, Yu Emma, Gu-Yeon Wei, and David Brooks. 2019. "Benchmarking TPU, GPU, and CPU Platforms for Deep Learning." *arXiv Preprint arXiv:1907.10701*.

Wang, Zijie J., Robert Turko, Omar Shaikh, Haekyu Park, Nilaksh Das, Fred Hohman, Minsuk Kahng, and Duen Horng Polo Chau. 2021. "CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization." *IEEE Transactions on Visualization and Computer Graphics* 27 (2): 1396–406. https://doi.org/10.1109/tvcg.2020.3030418.

Warden, Pete. 2018. "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition." *arXiv Preprint arXiv:1804.03209*, ahead of print. https://doi.org/10.48550/arXiv.1804.03209.

Warden, Pete, and Daniel Situnayake. 2020. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.

Waterman, Andrew, Yunsup Lee, Rimas Avizienis, Henry Cook, David Patterson, and Krste Asanovic. 2013. "The RISC-V Instruction Set." *2013 IEEE Hot Chips 25 Symposium (HCS)*, 1–1. https://doi.org/10.1109/hotchips.2013.7478332.

Weights & Biases, Inc. 2024. *Weights & Biases: The AI Developer Platform*.

Weiser, Mark. 1991. "The Computer for the 21st Century." *Scientific American* 265 (3): 94–104. https://doi.org/10.1038/scientificamerican0991-94.

Weizenbaum, Joseph. 1966. "ELIZA–a Computer Program for the Study of Natural Language Communication Between Man and Machine." *Communications of the ACM* 9 (1): 36–45. https://doi.org/10.1145/365153.365168.

Welling, Max. 2009. "Herding Dynamical Weights to Learn." *Proceedings of the 26th Annual International Conference on Machine Learning*, 1121–28. https://doi.org/10.1145/1553374.1553517.

Wengert, Ralph E. 1964. "A Simple Automatic Derivative Evaluation Program." *Communications of the ACM* 7 (8): 463–64. https://doi.org/10.1145/355586.364791.

Werbos, Paul. 1974. "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences." PhD thesis, Harvard University.

Werchniak, Andrew, Roberto Barra Chicote, Yuriy Mishchenko, Jasha Droppo, Jeff Condal, Peng Liu, and Anish Shah. 2021. "Exploring the Application of Synthetic Audio in Training Keyword Spotters." *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 7993–96. https://doi.org/10.1109/icassp39728.2021.9413448.

Wexler, James, Mahima Pushkarna, Tolga Bolukbasi, Martin Wattenberg, Fernanda Viégas, and Jimbo Wilson. 2020. "The What-If Tool: Interactive Probing of Machine Learning Models." *IEEE Transactions on Visualization and Computer Graphics* 26 (1): 56–65. https://doi.org/10.1109/TVCG.2019.2934619.

Widmer, Gerhard, and Miroslav Kubat. 1996. "Learning in the Presence of Concept Drift and Hidden Contexts." *Machine Learning* 23 (1): 69–101. https://doi.org/10.1023/a:1018046501280.

Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76. https://doi.org/10.1145/1498765.1498785.

Wolpert, D. H., and W. G. Macready. 1997. "No Free Lunch Theorems for Optimization." *IEEE Transactions on Evolutionary Computation* 1 (1): 67–82. https://doi.org/10.1109/4235.585893.

Wu, Bichen, Kurt Keutzer, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, and Yangqing Jia. 2019. "FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10726–34. https://doi.org/10.1109/cvpr.2019.01099.

Wu, Hao, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. "Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation." *arXiv Preprint arXiv:2004.09602* abs/2004.09602.

Wu, Zuxuan, Tushar Nagarajan, Abhishek Kumar, Steven Rennie, Larry S. Davis, and Rogerio Feris. 2018. "BlockDrop: Dynamic Inference Paths in Residual Networks." *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 8817–26. https://doi.org/10.1109/cvpr.2018.00919.

Wulf, Wm. A., and Sally A. McKee. 1995. "Hitting the Memory Wall: Implications of the Obvious." *ACM SIGARCH Computer Architecture News* 23 (1): 20–24. https://doi.org/10.1145/216585.216588.

Xin, Ji, Raphael Tang, Yaoliang Yu, and Jimmy Lin. 2021. "BERxiT: Early Exiting for BERT with Better Fine-Tuning and Extension to Regression." In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, edited by Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.eacl-main.8.

Xu, Ruijie, Zengzhi Wang, Run-Ze Fan, and Pengfei Liu. 2024. "Benchmarking Benchmark Leakage in Large Language Models." *arXiv Preprint arXiv:2404.18824*.

Yang, Le, Yizeng Han, Xi Chen, Shiji Song, Jifeng Dai, and Gao Huang. 2020. "Resolution Adaptive Networks for Efficient Inference." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2366–75. https://doi.org/10.1109/cvpr42600.2020.00244.

Yao, Zhewei, Amir Gholami, Sheng Shen, Kurt Keutzer, and Michael W. Mahoney. 2021. "HAWQ-V3: Dyadic Neural Network Quantization." *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 11875–86.

Yee, Kyra, Uthaipon Tantipongpipat, and Shubhanshu Mishra. 2021. "Image Cropping on Twitter: Fairness Metrics, Their Limitations, and the Importance of Representation, Design, and Agency." *Proceedings of the ACM on Human-Computer Interaction* 5 (CSCW2): 1–24. https://doi.org/10.1145/3479594.

Yosinski, Jason, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. "How Transferable Are Features in Deep Neural Networks?" *Advances in Neural Information Processing Systems* 27.

Yu, Gyeong-In, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. "Orca: A Distributed Serving System for Transformer-Based Generative Models." *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–38.

Yun, Sangdoo, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. 2019. "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 6023–32. https://doi.org/10.1109/ICCV.2019.00612.

Yurdakul, Bilal, and Joshua Naranjo. 2020. "Statistical Properties of the Population Stability Index." *The Journal of Risk Model Validation* 52. https://doi.org/10.21314/jrmv.2020.227.

Zafrir, Ofir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. "Q8BERT: Quantized 8Bit BERT." *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS)*, 36–39. https://doi.org/10.1109/emc2-nips53020.2019.00016.

Zaharia, Matei, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, et al. 2018. "Accelerating the Machine Learning Lifecycle with MLflow." *IEEE Data Engineering Bulletin* 41 (4): 39–45.

Zaharia, Matei, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2010. "Spark: Cluster Computing with Working Sets." *HotCloud* 10: 10–10.

Zaharia, Matei, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. "Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics." *Proceedings of CIDR* 8.

Zech, John R., Marcus A. Badgeley, Manway Liu, Anthony B. Costa, Joseph J. Titano, and Eric Karl Oermann. 2018. "Variable Generalization Performance of a Deep Learning Model to Detect Pneumonia in Chest Radiographs: A Cross-Sectional Study." *PLOS Medicine* 15 (11): e1002683. https://doi.org/10.1371/journal.pmed.1002683.

Zhang, Biao, and Rico Sennrich. 2019. "Root Mean Square Layer Normalization." *Advances in Neural Information Processing Systems (NeurIPS)*.

Zhang, Brian Hu, Blake Lemoine, and Margaret Mitchell. 2018. "Mitigating Unwanted Biases with Adversarial Learning." *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, 335–40. https://doi.org/10.1145/3278721.3278779.

Zhang, Chengliang, Minchen Yu, Wei Wang, and Feng Yan. 2019. "MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving." *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 1049–62.

Zhang, Chiyuan, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. 2017. "Understanding Deep Learning Requires Rethinking Generalization." *International Conference on Learning Representations (ICLR)*.

Zhang, Hongyi, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. *mixup: Beyond Empirical Risk Minimization*. OpenReview.net.

Zhang, Li Lyna, Yuqing Yang, Yuhang Jiang, Wenwu Zhu, and Yunxin Liu. 2020. "Fast Hardware-Aware Neural Architecture Search." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. https://doi.org/10.1109/cvprw50498.2020.00354.

Zhang, Qingxue, Dian Zhou, and Xuan Zeng. 2017. "Highly Wearable Cuff-Less Blood Pressure and Heart Rate Monitoring with Single-Arm Electrocardiogram and Photoplethysmogram Signals." *BioMedical Engineering OnLine* 16 (1): 23. https://doi.org/10.1186/s12938-017-0317-z.

Zhang, Yundong, Naveen Suda, Liangzhen Lai, and Vikas Chandra. 2017. *Hello Edge: Keyword Spotting on Microcontrollers*.

Zhao, Jiawei, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection." *arXiv Preprint*.

Zheng, Lianmin, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, et al. 2020. "Ansor: Generating High-Performance Tensor Programs for Deep Learning." *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 863–79.

Zhou, Aojun, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. 2021. "Learning n:m Fine-Grained Structured Sparse Neural Networks from Scratch." *arXiv Preprint*.

Zhu, Chenzhuo, Song Han, Huizi Mao, and William J. Dally. 2017. "Trained Ternary Quantization." *International Conference on Learning Representations (ICLR)* 4 (4): 1–16.

Zhu, Ligeng, Zhijian Liu, and Song Han. 2019. "Deep Leakage from Gradients." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 14774–84.

Zillow Group. 2021. *Zillow Group Reports Third-Quarter 2021 Financial Results and Shares Plan to Wind down Zillow Offers Operations*. Investor Relations Press Release.

Zoph, Barret, and Quoc V. Le. 2016. "Neural Architecture Search with Reinforcement Learning." *International Conference on Learning Representations* 3.

Zu, Y., A. Ghaffarkhah, H.-V. Dang, B. Towles, S. Hand, S. Huda, A. Bello, et al. 2024. "Resiliency at Scale: Managing Google's TPUv4 Machine Learning Supercomputer." *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 761–74.

Index

A/B testing
  delayed feedback, A/B testing challenges, 831
  ML deployment, 100
  randomization unit, 831
  statistical model validation, 831
A100, 407
  data-loading requirements, 132
  HBM capacity, 354
  MIG partitioning, 761
  roofline analysis, 666, 968
  Tensor Core throughput, 338
Ablation study
  component isolation, 100
  definition, 99
Abstraction problem
  definition, 327
Accelerator
  bubbles, pipeline idleness, 363
  design, workload-specific memory, 610
  memory bandwidth limitations, 384
  ML, 575
  specifications, roofline analysis example, 666
Accelerator gain
  definition, 568
Accelerator-day
  definition, 12
Access pattern
  definition, 603
Activation checkpointing
  definition, 319
  memory-compute trade-off, 319, 363, 415
  optimal placement, 415
  recomputation strategy, 321
  recomputation trade-off, 394
Activation function
  definition, 195
  nonlinear transformation, 193
  softmax, 197
  training efficiency, 372
Activation memory
  definition, 215
Activation reuse
  definition, 611
Activation unit
  definition, 585
Activation-based pruning
  definition, 501
Active learning
  definition, 459
  label prioritization, 157
  labeling cost reduction, 446, 459
Adam optimization, *see* Adam optimizer
Adam optimizer
  convergence properties, 377
  Kingma and Ba, 377
  memory overhead, 220, 377, 378
  momentum and velocity, 220
  state buffers, 219
AdamW
  decoupled weight decay, 377
Adaptability
  definition, 642
Adaptive computation
  early exit, 518
Adaptive inference
  definition, 518
Adaptive resolution
  content-based selection, 749
Admission control
  queue depth threshold, 756
Adversarial debiasing
  fairness constraints, 887
  training cost, 887
Ahead-of-time compilation, 308
  predeployment, 780
Ahead-of-time optimization, 305
AI compute primitive
  definition, 577
AI democratization, 929
AI engineering
  definition, 25
AI memory systems
  bandwidth bottleneck, 597
AI Memory Wall
  definition, 598
AI runtime
  definition, 642
  dynamic execution management, 641
  vs. traditional runtime, 642
AI winters
  systems failure, 8
AI-assisted labeling
  preannotation, 156
  scalability, 155
Alert fatigue
  operational risk, 850
AlexNet
  GPU deep learning era, 572
  ImageNet breakthrough, 7, 186
  ImageNet revolution, 251
  memory constraint, 186
  multi-GPU origin, 424
  parameter growth, 162
Algorithm-hardware adoption lag, 189
Algorithmic efficiency
  definition, 23
Algorithmic fairness
  healthcare ML, 94
Alignment gap
  proxy metric divergence, 874
AllReduce
  collective communication, 337
  definition, 609
  gradient synchronization, 428, 686
  gradient synchronization cost, 609
  network wall, 428
  ring topology, 428
AlphaFold
  cloud deployment, 27
AlphaGo, 13
AlphaGo Zero
  self-play training, 13
Always-on sensing
  power constraints, 43
Amazon recruiting
  specification failure, 870
Amdahl's law
  acceleration ceiling, 568
  appendix refresher, 968
  data pipeline ceiling, 150
  diagnostic analogy, 19
  diminishing returns, 568
  distributed training limit, 645
  multi-accelerator scaling, 645
  optimization ceiling, 694
  parallel fraction, 568
  pipeline bottleneck, 77
  preprocessing bottleneck, 746, 921
  speedup limits, 77
  system optimization, 79
Amdahl's reality
  definition, 569
Amdahl, Gene, *see also* Amdahl's law
Anti-curriculum
  hard examples first, 459
AOT, *see* Ahead-of-time optimization
Apache Airflow
  lineage capture, 908
  pipeline orchestration, 815
Apache Arrow
  in-memory columnar format, 163
Apache Atlas
  data lineage, 908
Apache Kafka
  event streaming, 135
  ordering guarantee, 135
Apache Pulsar
  geo-replication, 135
Apache TVM
  accelerator compilation, 620
  ML compiler, 350
Application-specific integrated circuit, *see* ASIC
Architectural Integration
  execution unit organization, 595
Architecture
  building blocks, 282
  computational graph, 240
  compute vs. memory trade-off, 244
  data-to-architecture mapping, 287
  encoder-decoder, 286
  parameter efficiency, 246
  representational power vs. efficiency, 241
  selection as bias matching, 246
  systems engineering, 240
Area under the learning curve
  definition, 449
Area under the ROC curve
  threshold independence, 99
Argmax
  prediction selection, 750
Arithmetic intensity, 46
  appendix refresher, 938, 950, 966
  attention layer, 385
  bottleneck diagnosis, 920
  definition, 612
  FLOP/byte ratio, 243
  GPU utilization, 236
  high intensity workloads, 43

roofline diagnostic, 571
threshold for compute vs. memory bound, 666
training operations, 384
Artifact
interdependence, 101
lineage tracking, 101
reproducibility, 802
versioning, reproducibility requirement, 802
Artificial neuron
computing primitive, 192
As-of join
definition, 167
point-in-time correctness, 167
ASIC
benchmarking trade-off, 665
flexibility trade-off, 573
Assumptions
accelerator, 978
bandwidth, 984
energy, 983
nameplate capacity, 986
pricing, 985
reference models, 982
scale, 986
training memory, 982
unit conventions, 986
Asynchronous execution
hiding transfer latency, 332
Attention
memory bottleneck, 409
quadratic complexity, 409
Attention head
specialization, 272
Attention matrix, 272
Attention Mechanism
global token interaction, 610
memory-bound softmax, 614
Attention mechanism
Bahdanau, 265
content-addressable memory, 273
definition, 265
fused kernel, 300
information bottleneck, 265
memory pressure, 610
multi-head, 272
quadratic bottleneck, 270
query-key-value, 267
scaled dot-product, 273
self-attention, 272
similarity computation, 392
AUC
anomaly detection performance, 678
Audit trail
accountability logging, 908
inference-time logging, 909
multi-tier logging, 908
storage scale, 909
AutoAugment
search cost, 466
autocast
mixed-precision context, 320
Autograd
automatic differentiation, 218
engine, internals, 320
execution trade-off, 218
reverse-linked graph, 320
Autograd tape
definition, 302
dynamic graph construction, 302
memory cost, 302
reverse traversal, 355
Automatic differentiation
appendix refresher, 953
computational graph, 218, 380
definition, 314
forward mode, 315
forward vs. reverse mode, 315
forward-mode dual numbers, 316
framework role, 296
reverse mode, 215, 314, 316
reverse-mode backpropagation, 316
tape-based vs. transform-based, 326
Automatic mixed precision
framework support, 404
Automotive AI
real-time safety requirements, 648
Autonomous Vehicle
latency requirements, 61
Autoregressive generation, 243
bandwidth bottleneck, 243
memory-bound, 43
Autoregressive model
autoregressive, serial bottleneck, 783
token generation, 785
Autoscaling
ML cold-start trade-off, 828
ML workloads, 828
startup latency, 790

Back-translation
pipeline latency, 465
text augmentation, 465
Backpressure
streaming systems, 134
Backpropagation
appendix refresher, 953
computational asymmetry, 316
computational cost, 380
definition, 213
gradient computation, 213
historical development, 189
historical introduction, 370
memory cost, 189
memory requirements, 380, 575
provenance, 370
through time (BPTT), 263
Backward pass
gradient computation, 394
memory operations, 394
triggering gradient computation, 355
Bandwidth
bottleneck, video streaming, 56
latency trade-off, 58, 972
PCIe, model swapping, 761
Bandwidth hog
definition, 43
Bandwidth taper
interconnect hierarchy, 607
Baseline model
reference point selection, 676
Batch
loss averaging, 210
training iteration, 205
Batch inference
definition, 836
Batch ingestion
definition, 133
Batch normalization, 285
hardware implementation, 583
mode-dependent behavior, 341
training speedup, 285
training stability, 196
training-serving skew, 285
Batch processing
definition, 375
hardware utilization, 205
Batch Size
arithmetic intensity impact, 617
Batch size, 249
arithmetic intensity effect, 386
gradient stability, 213
hardware alignment, 391
inference trade-off, 227
loss computation, 210
Batch throughput
definition, 695
Batch-1 regime
definition, 769
Batch-N regime
definition, 769
Batching
batch formation delay, definition, 765
batch, small, CPU advantage, 781
etymology, 762
fallacy of larger batches, 795
hardware utilization, 372
tax, latency penalty, 764
training vs. serving, 762
window, latency-throughput trade-off, 744
Battery
coin-cell deployment, 65
Battery life
ML impact, 62
Battery tax
always-on constraints, 62
Bayes error rate
appendix refresher, 942
Beam search
decoding strategy, 784
Benchmark
etymology, 658
Benchmark contamination
LLM memorization risk, 716
Benchmark coverage
incomplete problem representation, 707
Benchmark engineering
intentional optimization bias, 709
Benchmark gaming
distorting performance metrics, 711
vendor optimization for tests, 658
Benchmark harness
test infrastructure, 678
Benchmark saturation
outdated benchmarks pitfall, 723
Benchmark-production gap, 656
Benchmarking
battery, duty cycle specification, 701
benchmarks as proxies, 658
definition, 656
end-to-end vs. component metrics, 658
energy, first-class metric, 659
fallacy of direct performance translation, 722
historical evolution, 658
probabilistic variability, 660
system, definition, 657
Benchmarking granularity
definition, 669
Bengio, Yoshua
Turing Award, 183
Berkson, Joseph, *see also* Logits
BERT
benchmarking baseline, 676
compression techniques, 506
inference deployment prediction, 666
roofline analysis at batch 1, 666
Best when
definition, 634
BF16
appendix refresher, 974
dynamic range preservation, 407
exponent range preservation, 403
hardware support, 407
reduced-precision format, 227
Bias
activation threshold, 199
attribute removal fallacy, 910
gender discrimination, 871
geographic, training data, 130
historical data encoding, 871
mitigation strategies, 876
parameter overhead, 199
proxy variables, 871
signal aggregation, 193
Bias feedback model
disparity amplification, 921
Bias-variance tradeoff
classical theory, 224

double descent, 224
overparameterization, 224
Binarization
definition, 539
Biological Neuron
definition, 192
Bitter lesson
cloud scale, 49
definition, 12
feature learning, 188
bitter lesson, The, 12, 13
BLAS
historical foundation, 298
matrix computation foundation, 370
neural networks, 249
performance foundation, 298
training compute hierarchy, 370
utilization, 249
vector and matrix operations, 339
BLEU
Goodhart's law example, 677
Blue-green deployment
duplicate capacity, 829
zero-downtime updates, 829
Bobrow, Daniel
STUDENT, 8
Bottleneck principle
definition, 42
Boundary erosion
dissolution of modularity, 808
Bounding box
annotation, 153
definition, 153
Break-even analysis
edge vs. cloud utilization, 52
Break-even point
data selection ROI, 476
Brittleness
definition, 12
Broadcasting
definition, 952
Bulkhead pattern
failure containment, 855
resource isolation, 855
Byte pair encoding
tokenization, 389

C4
quality filtering, 452
CACE principle
change propagation, 808
Cache efficiency
definition, 628
Cache locality
definition, 603
Calibration
clinical trust, 102
dataset, representative traffic, 778
definition, 102
definition and etymology, 713
INT8 quantization, 778
production traffic mismatch, 795
Canary
etymology, 757
Canary deployment
risk-controlled rollout, 829
statistical validation, 829
Canary request
fan-out protection, 757
Canonical workload, 242
CAP theorem
feature store trade-off, 135
streaming systems, 135
Capacity planning
infrastructure sizing, 791
Carbon footprint
energy to emissions conversion, 897
Carbon intensity
definition, 901
grid emissions, 900
Carbon-aware scheduling
lower-carbon times and regions, 900
Categorical Encoding
one-hot encoding, 147
Categorical feature, 277
Causal mask, 272
Central processing unit, *see* CPU
Chain Rule
gradient decomposition, 214
Chain rule
appendix refresher, 952
backpropagation foundation, 380
depth constraint, 215
Challenge
definition, 620
Channelwise quantization, *see* Quantization
CheckList, 878
Checkout-free retail
edge processing, 61
Checkpointing
I/O cost, 225
Chinchilla
compute-optimal scaling, 450
compute-optimal training, 450
tokens-per-parameter diagnostic, 451
Chiplet
modular die interconnect, 645
CI/CD pipeline
ML-specific adaptation, 819
CIFAR-10
classification reference dataset, 676
Circuit breaker
cascade prevention, 145
Circuit breaker pattern
cascade failure prevention, 855
ML adaptation, 855
Class balance
coverage metric, 717
Class imbalance
coverage failure mode, 717
minority-class performance, 236
Classical machine learning
feature engineering, 185
Classification
definition, 613
Classification label
storage requirements, 153
Cleanlab
label error detection, 454
ClinAIOps
healthcare ML operations, 860
MLOps comparison, 863
Clinician oversight loop, 861
CLIP, *see* Contrastive Language-Image Pretraining
Closed division
MLPerf submission, 697
MLPerf training, 668
Cloud economics
elastic pricing, 51
Cloud infrastructure
utility model, 49
Cloud ML, 26
accelerator infrastructure, 50
characteristics, 39
consumer-scale applications, 53
data center scale, 49
definition, 49
latency limitations, 51
privacy concerns, 51
scaling limits, 54
workload archetypes, 49
Cloud pricing
cost-latency trade-off, 128
CNN
definition, 250
spatial data reuse, 610
COCO
object detection dataset, 676
Code generation
hardware-specific instructions, 641
Cohen's kappa
inter-annotator agreement, 717
label quality, 143
Coin-cell battery
deployment longevity, 64
Cold start
anatomy, 759
bursty traffic impact, 795
cloud serving, 740
definition, 759
inference concern, 696
scaling events, 759
Cold start latency
physical lower bound, 696
serverless AI impact, 695
Collation
batch assembly, 336
Columnar storage
Parquet, 164
Common techniques
definition, 616, 617
Common use cases
definition, 634
Communication and synchronization
definition, 622
Communication tax
definition, 424
COMPAS
calibration and error rates, 872
recidivism algorithm audit, 872
Compilation continuum
principle, 309
Compilation output
definition, 637
Complexity tax
ML vs. heuristics, 68
Compositionality
pattern decomposition, 203
Compressed sparse row
definition, 951
Compression
algorithm selection, 164
Computation graph
dynamic (eager execution), 218
static (deferred execution), 218
Computation Placement
processing element assignment, 620
Computation scheduling
definition, 637
parallel execution, 640
Computational granularity
definition, 622
Computational graph
definition (DAG), 300
framework optimization, 296
graph execution, 296
static, 304
Computational photography
pipeline constraint, 61
Compute beast
definition, 43
Compute efficiency
definition, 23
Compute-Bound
roofline classification, 611
Compute-bound
operation classification, 300
resolution scaling, 749
Compute-Bound vs. Memory-Bound
training vs. inference, 46
Compute-bound vs. memory-bound
definition, 42
memory-bound optimization, 43
Compute-optimal frontier
Chinchilla scaling laws, 450
Computer engineering
hardware-software integration, 25
Concept drift
adversarial evolution, 805
changing relationships, 838
definition, 143
responsibility lens, 873
validation, 104
Concept-based explanation
human-understandable feature, 890
Conditional computation

definition, 518
Confidence interval
benchmark reporting, 662
Confidence threshold
automation trade-off, 231
decision making, 232
Confident learning
mislabeled example detection, 717
Confidential computing
hardware secure enclaves, 903
Configuration debt
Facebook case study, 811
parameter sprawl, 810
Confusion matrix
fairness analysis, 885
Consensus labeling
quality control, 155
Conservation of complexity
heuristic limits, 920
meta-principle, 920
structural optimization, 496
Consistency Imperative
definition, 146
Consistency imperative
training-serving parity, 146, 803
Consistency regularization
compute trade-off, 462
semi-supervised technique, 462
Constant folding, 305
compile-time optimization, 779
Constrained hardware
inference deployment, 226
Constraint layer
statistical physical operational, 93
Constraint propagation principle
definition, 88
Containerization
environment parity, 829
ML deployment, 828
reproducible environments, 152
Continuous batching
LLM serving, 785
scheduling granularity, 785
Continuous therapeutic monitoring
healthcare AI, 860
human oversight, 860
Contrastive Language-Image Pretraining
semantic deduplication, 454
Contrastive Learning
batch-based methods, 464
pretext task, 463
Convergence
monitoring, 224
rate, clean vs. noisy data, 453
time-memory trade-off, 377
Convolution
computational cost, 392
data reuse, 250
depthwise, 259
depthwise separable, 259
etymology, 250
hardware optimization, 260
kernel, 252
matrix multiplication equivalence, 581
padding, 254
pointwise, 259
stride, 254
Convolution performance
definition, 628
Convolutional neural network, *see* CNN
Coordinated omission
closed-loop measurement trap, 678
Coordination Tax
distributed data selection, 473
Coordination tax
distributed processing, 151
Coreset
data efficiency, 481
definition, 455
etymology, 454
geometry origins, 454
ICR example, 447
Correction cascade
sequential dependencies, 808
Zillow case study, 811
Cosine annealing
learning rate schedule, 380
Cost amortization
self-supervised pretraining, 477
Cost modeling
data selection economics, 474
Cost per inference
operational tracking, 837
serving economics, 790
Cost-aware automation
retraining decisions, 804
Cost-Performance Analysis
accelerator economics, 596
Counterfactual analysis
prediction debugging, 848
Covariate shift
definition, 143
input distribution change, 718
responsibility lens, 873
CP decomposition
definition, 511
CPU
inference optimization, 781
neural network execution, 209
CPU affinity
latency reduction, 780
CPU backend
optimized BLAS libraries, 357
Cray-1
AI accelerator lineage, 580
Credit assignment problem
weight responsibility, 213
Critical batch size
throughput-convergence limit, 375
Critical path
composed request, 927
optimization, 46
Cross-entropy loss
classification loss, 210
definition, 210
information theory, 210
Cross-industry standard process for data mining
lifecycle influence, 83
Cross-layer interaction
cross-optimization effects, 482
Crowdsourcing
annotation, 129
data labeling, 129
cuBLAS
hardware-accelerated linear algebra, 371
kernel selection, 338
NVIDIA linear algebra, 639
CUDA
context, initialization overhead, 760
cores, streaming processors, 620
dispatch overhead, 308
ecosystem lock-in, 587
runtime, OS layer, 331
serving cold starts, 760
stream definition, 332
stream overlap, 332
CUDA graphs
kernel launch elimination, 779
kernel launch replay, 671
CUDA MPS
context sharing, 760
multi-model serving, 760
cuDNN
benchmarking dependency, 670
NVIDIA deep learning library, 639
optimized implementations, 392
Curriculum learning
difficulty scorer and pacing function, 458
etymology, 458
optimization landscape, 458
Custom autograd function
backward implementation, 322
Custom differentiation rule, 321
CutMix
image augmentation, 465
label mixing, 465
Cutout
augmentation efficiency, 465
image augmentation, 465

D·A·M Taxonomy
bottleneck classification, 397
DAM taxonomy, *see* D·A·M taxonomy
D·A·M taxonomy
alignment of components, 234
appendix refresher, 935
axis intersections, 936
definition, 15
diagnostic framework, 720
lifecycle stages, 89
ML test score, 813
responsibility diagnosis, 874
scorecard, 941
training vs. inference, 46
workload classification, 43
Dark knowledge
class correlations, 507
Dark Silicon
specialization driver, 572
Dartmouth Conference, 8
systems constraints, 8
Data
as source code, 15
coverage, requirements, 130
partitioning, query optimization, 164
physics of, 115
transformation, techniques, 147
Data acquisition
scalability, 127
scale vs. quality, 127
source evaluation, 126
strategies, 126
Data augmentation
definition, 464
synthetic data, 129
training diversity, 231
Data benchmarking
definition, 657
training set validation, 716
Data card
compliance documentation, 907
Data cascade
definition, 89, 118
silent failure, 118
Data center
energy consumption, 900
ML infrastructure, 49
Data cleaning
error correction, 146
inconsistency removal, 146
signal-to-noise engineering, 117
Data collection
iron law data term, 94
Data compression ratio
definition, 450
Data curation
end-to-end generalization, 669
Data debt
definition, 169
hidden liability, 168
strategic vs. unconscious, 171
Data drift
definition, 17
degradation equation, 147
detection, 142
operational definition, 838
silent model degradation, 107
Data echoing
amortizing I/O costs, 470
GPU utilization, 470
Data engineering
definition, 114
Data freshness
SLA, 134
staleness detection, 843
Data governance
definition, 901

enforcement mechanisms, 901
model artifact compliance, 910
Data gravity
appendix refresher, 943
definition, 115
Data gravity principle
cloud limitations, 51
physical anchor, 920
Data ingestion
pipeline boundary, 131
Data labeling
clinical economics, 461
systems engineering, 153
Data lake
schema flexibility, 160
unstructured storage, 160
Data lakehouse
data gravity, 115
Data lineage
audit trail, 108
debugging and compliance, 152
GDPR compliance, 908
processing history, 152
regulatory compliance, 108
transformation tracking, 152
Data loading
shard-based sequential I/O, 469
Data Locality
spatial and temporal, 591
Data locality
code-to-data, 116
data-to-code, 116
Data locality invariant
definition, 58
Data localization
cross-border transfer, 906
Data management
MLOps lifecycle, 814
Data minimization
privacy by design, 906
Data movement
broadcast, 282
energy dominance, 41, 188, 624
gather, 282
optimization strategies, 624
reduction, 282
scatter, 282
Data Parallelism
training strategy, 686
Data parallelism
communication overhead, 369
framework implementation, 425
gradient averaging, 425
gradient synchronization, 425
multi-accelerator scaling, 645
Data path
customization, 573
Data pipeline
architecture, 131
automated workflows, 815
ingestion and preprocessing, 386
storage throughput, 388
throughput optimization, 334
Data poisoning
training pipeline security, 904
Data prefetching
accelerator utilization, 401
buffer management, 401
Data quality
as code, 141
container, 141
content, 141
expectation tests, 141
mechanical vs. semantic, 141
monitoring, 840
noise penalty on convergence, 453
statistical monitoring, 139
vs. data quantity, 77
Data quality multiplier
label noise penalty, 453
Data redundancy
empirical evidence, 452
Data reuse
accelerator optimization, 625
definition, 603
energy efficiency, 188
Data science
vs. data engineering, 114
Data selection
definition, 23, 442, 444
optimization ordering, 443
systems vs. ML framing, 444
Data shuffling
random access penalty, 388
Data supply rate
bandwidth constraint, 163
Data swamp
governance failure, 160
Data tax
definition, 447
Data validation
monitoring, 139
Data versioning
DVC tool, 815
model reproducibility, 165
reproducibility, 88, 819
Data wall
definition, 442
scaling constraint, 442
zero learning signal, 447
Data warehouse
analytical queries, 160
OLAP, 160
Data-as-code principle
definition, 7
logical program, 920
Data-centric AI, 7
dataset enhancement paradigm, 718
Data-model interface
feature consistency, 801
Database
OLTP for ML, 159
DataComp
data-centric benchmark, 718
data-centric benchmarking, 718
Dataflow
optimization strategies, 625
Dataflow Architecture
definition, 619
Dataflow optimization
data movement minimization, 624
Dataflow strategy
definition, 635
DataHub
data lineage, 908
DataLoader
configuration parameters, 401
throughput optimization, 335
DataPerf
data quality benchmark, 718
Dataset
iterable-style, 335
map-style, 335
Dataset compilation
definition, 114
Dataset saturation
performance capability illusion, 719
Datasheets for datasets
training data documentation, 891
DAWNBench
time-to-accuracy evaluation, 659
Dead code elimination, 305
graph optimization, 305
Dead neuron, 284
Dean, Jeff
latency numbers, 161, 963
Debugging
data pipelines, 171
Debugging checklist
runbook steps, 848
Decision framework
paradigm selection, 68
Decode phase
memory-bandwidth bound, 793
sequential generation, 793
Deduplication
data selection optimization, 453
definition, 115
embedding-based, 454
hash-based, exact matching, 453
storage and I/O cost reduction, 446
Deep Blue, 13
custom chess hardware, 13
Deep learning
automatic feature learning, 186
definition, 11, 183
network depth, 203
Deep learning framework
automatic differentiation, 298
Deep Learning Recommendation Model
workflow constraints, 90
DeepBench
operator-level testing, 670
Default in frameworks
definition, 628
Deferred execution
graph construction, 296
Degradation equation
definition, 17
distribution shift, 75
operational implementation, 800
Delta Lake
time travel, 166
Demographic drift
definition, 94
Demographic parity
definition, 885
Dennard Scaling
end of, 572
multi-core revolution, 573
Dennard scaling
origin and breakdown, 41
Dense connectivity, 247
parameter scaling, 202
Dense Layer
roofline analysis, 615
Deployment
compute-intensive cloud deployment, scientific computing, 27
controlled, risk mitigation, 826
high-stakes hybrid deployment, autonomous vehicles, 27
iron law overhead minimization, 104
resource-constrained edge, precision agriculture, 27
Deployment paradigm
definition, 26, 40
Deployment target
cloud to edge spectrum, 349
Depth
exponential advantage, 204
Depthwise conv
definition, 613
Depthwise Convolution
low arithmetic intensity, 613
Depthwise separable convolution, 259
mobile deployment, 90
mobile efficiency, 513
MobileNet, 243
Deterministic Transformation
reproducibility, 149
Developer feedback loop, 861
Device management
CPU-GPU transfers, 330
DevOps
MLOps extension, 801
Diabetic retinopathy
case study introduction, 85
deployment scale, 84
Differential privacy
epsilon budget, 904
epsilon trade-off, 904
formal guarantees, 904
mathematical guarantees, 877
noise injection, 904
Differentiation problem
definition, 314
Diffusion model
text-to-image synthesis, 465
Digital signal processing
multiply-accumulate units, 572

Dimensional homogeneity
  appendix refresher, 968
Direct memory access, *see* DMA
Disaggregated evaluation
  definition, 877
  demographic stratification, 877
  fairness testing, 883
Disparate impact
  legal doctrine, 878
Dispatch overhead
  law, 312
Dispatch overhead reduction
  kernel launch batching, 671
Dispatch tax
  framework overhead, 304
Display overlay
  GPU mapping, 646
Distance penalty
  cloud latency, 50
Distributed computing
  system-level performance, 659
Distributed data
  clinic coordination cost, 96
Distributed Processing
  scaling, 150
Distributed Selection
  network topology constraints, 474
Distributed training
  communication overhead, 427
  communication tax, 427
  data parallelism, 369
  data selection challenges, 472
  execution contexts, 337
  model parallelism, 369
  scaling efficiency, 426
Distribution Shift
  responsibility lens, 873
Distribution shift
  causes of model degradation, 873
  label shift, 143
  monitoring, 142
  train-to-production alignment, 718
Diversity Sampling
  active learning query strategy, 459
DLRM, 278
  capacity-bound workload, 918
  embedding bottleneck, 369
  I/O-bound serving, 747
  interaction layer, 278
  memory capacity bound, 22, 44
  memory-bound benchmark, 697
  memory-bound constraint, 55
  recommendation model benchmark, 697
  scalability challenge, 121
DMA
  computation overlap, 607
  PCIe data movement interface, 331
Documentation debt
  data provenance, 169
Domain adaptation
  feature alignment, 466
Domain gap
  training-serving skew, 466
Domain randomization
  bridging synthetic-real gap, 466
  rendering trade-off, 466
Domain-Specific Architecture, *see* DSA
Dot product
  appendix refresher, 950
  definition, 194
Double buffering
  latency hiding, 643
DQN
  real-time inference, 184
DRAM
  energy cost, 576
Drift
  covariate shift, 838
Drift detection delay
  label latency, 838
  statistical monitoring, 838
Dropout
  definition, 224
  training vs. inference, 341
DS-CNN
  monitoring strategy, 860
DSA
  definition, 573
  efficiency trade-off, 573
  scaling law breakdown, 573
Dual mandate, 5
Dual number
  forward mode AD, 316
Duty cycle
  accelerator utilization, 751
DVC
  data versioning, 805
  dataset versioning, 815
  git semantics, 165
DVFS
  dynamic voltage and frequency scaling, 705
  power-performance envelope, 647
  thermal throttling, 723
  warm-up measurement artifact, 670
Dying ReLU problem
  definition, 196
Dynabench
  dynamic evaluation, 719
Dynamic batching
  definition, 762
  time window, 765
Dynamic computation, 286
Dynamic graph
  definition, 302
  eager execution, 302
  runtime construction, 302
Dynamic inference
  real-time prediction, 739
Dynamic kernel execution
  runtime adaptation, 643
Dynamic quantization
  definition, 536
Dynamic random access memory, *see* DRAM
Dynamic selection
  active learning, 459
  curriculum learning, 458
  definition, 443
  training-time optimization, 458
Dynamic voltage and frequency scaling, *see* DVFS

Eager Execution
  optimization limitations, 300
Eager execution
  define-by-run model, 302
  definition, 296
  optimization trade-off, 296
Earth mover's distance
  multi-dimensional drift, 838
Edge accelerator
  deployment selection, 60
Edge AI
  deployment patterns, 833
  Oura Ring case study, 858
Edge deployment
  bandwidth constraints, 106
  constraint triangle, 700
  power constraints, 896
  storage, 165
  TinyML frameworks, 313
Edge hardware
  resource limits, 59
Edge inference
  power tiers, 226
Edge ML
  autonomous vehicles, 61
  bandwidth reduction, 56
  data sovereignty, 55
  definition, 55
  deployment challenges, 59
  distributed processing, 59
  industrial IoT, 61
  latency and bandwidth, 26
  latency benefits, 39
  privacy benefits, 58
  smart retail, 61
Edge orchestration
  fleet challenges, 59
Edge serving
  embedded function calls, 740
Edge summary
  uplink reduction, 95
Edge TPU
  inference efficiency, 226
  inference latency, 657
  operator coverage, 690
Efficiency
  Jevons paradox, 24
EfficientNet
  algorithmic efficiency, 24
Einstein summation convention
  definition, 950
EL2N
  definition, 455
  proxy transferability, 455
Element-wise operation
  memory bandwidth, 339
ELIZA
  brittleness, 8
ELT
  iteration speed, 135
  load first, 135
Embedded GPU accelerator
  edge deployment, 60
Embedding
  definition, 277
  etymology, 277
  memory access pattern, 277
  table, 278
Embedding Lookup
  memory-bound operation, 613
Embedding pruning, 454
Embedding table
  capacity bottleneck, 369
  memory capacity bound, 43
Embodied carbon
  hardware manufacturing, 900
Emergent behavior
  definition, 109
Encoder-decoder
  attention as routing, 286
  definition, 265
Encryption
  data lifecycle protection, 903
End-to-end benchmark
  complete workflow measurement, 669
Energy
  budget, mobile inference, 773
  data movement cost, 574
  training scale, 190
Energy consequence
  definition, 603
Energy consumption
  hardware platforms, 209
  training costs, 895
Energy dividend
  quantization efficiency, 524
Energy efficiency
  TFLOPS per watt, 649
  training cost accounting, 687
Energy harvesting
  milliwatt threshold, 64
Energy hierarchy
  reference sources, 964
Energy of transmission
  local vs. cloud, 56
Energy per inference
  binding constraint, 43
  paradigm comparison, 58
Energy tax
  definition, 20
Energy wall
  battery constraints, 56
Energy-movement principle, 118
  data locality, 920
  decode phase, 792
  distributed training, 427
Ensemble learning

accuracy vs. deployment trade-off, 99
deployment cost, 99
Entanglement
ML system coupling, 808
Entropy
appendix refresher, 947
definition, 947
Environment parity
production deployment, 801
Environmental impact
carbon accounting, 900
ML systems, 895
Epoch
dataset iteration, 222
training cycle, 388
training cycles, 222
Equal opportunity
definition, 885
Equalized odds
definition, 885
postprocessing, 885
Equivariance
translation, 253
Error Signal
backward propagation, 215
ETL
reprocessing cost, 135
EU AI Act
risk classification, 905
systems requirements, 906
Evaluation pipeline
validation metrics, 386
Evasion attack
inference adversarial perturbation, 904
Excessive data movement
definition, 620
Execution problem
definition, 300
Expected calibration error
confidence-accuracy gap, 713
Expected impact
definition, 616, 617
Expected Model Change
active learning query strategy, 459
Experiment tracking
tools and practices, 100
Expert systems, 10
knowledge engineering, 185
maintenance cost, 185
Explainability
architecture trade-off, 889
definition and purposes, 889
post-hoc methods (SHAP, LIME), 889
Exploding gradient problem, 284
training instability, 215
Exponential unit
definition, 585
External tensor payloads
definition, 629, 630
Eyeriss
row-stationary dataflow, 610

F1 score
precision-recall harmonic mean, 677
Face ID
privacy architecture, 63
Facebook AI Similarity Search
selection infrastructure, 469
Fairness
accuracy trade-off, 877
governance objective, 851
individual vs. group, 877
mathematical definitions, 877
Fairness metrics
demographic parity, 885
equal opportunity, 885
equalized odds, 885
impossibility theorem, 885
loan approval case study, 884
FAISS, *see* Facebook AI Similarity Search
Fallacy of peak performance
GPU utilization gap, 664
False Accepts per Hour, 122
False negative rate
fairness impact, 885
False positive rate
always-on systems, 122
FarmBeats
connectivity constraint, 27
Fault tolerance
training checkpoint strategy, 688, 928
Feature attribution
debugging techniques, 847
Feature Computation
hybrid patterns, 138
pipeline vs. loader, 138
Feature computation
placement, 138
Feature consistency
training-serving alignment, 801
Feature contract
schema specification, 852
Feature coverage
deployment variation, 717
Feature Engineering
domain knowledge, 147
Feature engineering
classical ML, 185
definition, 147
Feature engineering bottleneck
definition, 11
Feature extraction
hierarchical, 255
Feature importance
prediction attribution, 889
Feature learning
hierarchical, 183
Feature map, 252
Feature store
consistency, 147
consistency guarantee, 147
definition, 167
offline vs. online stores, 167
point-in-time retrieval, 166
scale, 816
streaming updates, 167
training-serving bridge, 166
training-serving consistency, 816
Federated averaging
data minimization, 905
Federated learning
data minimization, 905
privacy preservation, 905
Feedback Loop
multi-scale temporal structure, 109
Feedback loop
problem amplification, 892
recommendation amplification, 875
self-reinforcing bias, 810
YouTube case study, 811
Feedforward network
architecture, 201
Feeding problem
definition, 116
Feeding tax
definition, 116
Field-programmable gate array, *see* FPGA
FixMatch
label-compute trade-off, 462
FlashAttention
2, improved parallelism, 412
3, FP8 tensor cores, 412
appendix refresher, 955
framework fusion example, 300
I/O-aware algorithm, 409
I/O-aware design, 386
IO complexity reduction, 410
memory bandwidth optimization, 409
memory optimization, 614
origin and engineering insight, 409
SRAM tiling, 542
tiled attention implementation, 635
tiling strategy, 410
FlatBuffers
etymology, 744
zero-copy serialization, 744
Flaw of averages, 883
Flax
JAX neural network library, 341
Fleiss' kappa
inter-annotator agreement, 155
Floating point
FP32 precision, 227
Floating-point operations per second, *see* FLOP/s
Floating-point unit
history, 572
precision bottleneck, 572
FLOP/s
peak vs. achieved, 666
Flow rate
definition, 116
Food and Drug Administration
SaMD constraint, 104
Forced alignment
automated labeling, 158
speech labeling, 158
Forgetting events
coreset selection, 455
pruning trade-off, 455
Forward pass
compute operations, 391
kernel dispatch, 354
memory management, 391
Forward propagation
computation process, 205
layer computation, 370
matrix operations, 206
Foundation model
definition, 464
homogenization risk, 464
Four pillars framework
definition, 118
Four-fifths rule
adverse impact threshold, 878
FP16
appendix refresher, 974
dynamic range limitation, 403, 404
half precision, 227
subnormal flushing, 407
training precision, 575
FP32
gradient computation, 575
numerical precision, 227
FP32 master weights
mixed-precision training, 363
optimizer stability, 406
FP8
E4M3 and E5M2 formats, 407
hardware acceleration, 407
FPGA
reconfigurability trade-off, 641
Framework profiler
performance analysis tool, 671
timeline analysis, 782
Framework selection
constrained optimization, 351
trade-off analysis, 351
Fraud Detection
ML vs. rule-based, 87
Frequency boosting
warm-up measurement artifact, 670
Freshness debt
distribution divergence, 169
Fully-connected layer, 247
dense connectivity, 199
Function unit
definition, 585
Fuse operations
definition, 637
Fused execution
definition, 630

Gabor filters

learned equivalents, 186
GAN
synthetic data generation, 465
Gather
definition, 580
Gating mechanism, 286
sequence models, 286
GDDR
off-chip DRAM variant, 606
GDPR
Article 22 automated decisions, 906
Article 22 requirements, 906
compliance, cloud deployment, 51
data collection requirements, 131
data minimization principle, 906
ML erasure constraint, 51
GELU, 373
GEMM
appendix refresher, 950
arithmetic intensity, 339, 968
hardware utilization, 298
matrix multiplication, 338
ML performance predictor, 635
Gender classification
demographic disparities, 876
Gender Shades
algorithmic audit methodology, 877
demographic fairness study, 712
disaggregated evaluation, 877
Generalization
vs. overfitting, 223
Generative adversarial network, *see* GAN
Generative AI
benchmark evolution, 710
GLUE
benchmark saturation, 676
language understanding benchmark, 676
Glue code
integration overhead, 810
Goodhart's law
etymology, 658
metric optimization, 874
metric trap, 658
proxy decoupling, 875
Google Flu Trends
proxy drift, 6
Google TPUv4
definition, 596
GPipe
pipeline parallelism, 428
GPT-2, 243
arithmetic intensity, 244
autoregressive bottleneck, 44
bandwidth-bound decode, 22
data selection, 444
memory bandwidth, 918
training lighthouse, 366
GPT-3, 901
training compute, 366
training cost, 684
training energy, 13
training scale, 11, 49
weight storage, 162
GPU, 895
batching utilization, 762
etymology, 5
history, 572
memory hierarchy, 836
parallel computation, 209
training, power budget, 226
utilization monitoring, 835
GPU backend
CUDA implementation, 357
GPU boost clock
benchmark variability, 689
GPU Interconnect
switching fabric, 608
GPU memory
training constraints, 392
GPU profiler
kernel-level analysis, 397
system-level analysis, 397
timeline analysis, 782
GPU utilization
batch size dependency, 763
GPU vs. CPU
economics, 790
Graceful degradation
data pipelines, 144
overload handling, 756
triggers, 926
Gradient
backpropagation, 213
bias gradients, 216
chain rule, 214
error signal flow, 215
flow in residual networks, 284
memory footprint, 314
partial derivatives, 216
vanishing and exploding, 215
weight gradients, 216
Gradient accumulation
definition, 213
framework behavior, 379
memory efficiency, 413
memory trade-off, 318
memory-compute trade-off, 413
Gradient checkpointing
appendix refresher, 955
memory reduction, 381
Gradient compression
definition, 413
Gradient descent
definition, 212
etymology, 374
full-batch computation, 374
gradient noise, 375
parameter optimization, 374
training foundation, 374
Gradient flow
tensor detachment, 321
Gradient function node
computation graph, 320
Gradient hook
inspection and modification, 321
Gradient inspection, 321
Gradient instabilities
training failure, 183
training failure mode, 183
Gradient-based pruning
definition, 502
GraNd
gradient-based coreset scoring, 455
Graph break
causes and detection, 306
Graph Compilation
JIT overhead, 759
Graph compilation
eager-mode capture, 307
optimization, 778
Graph Execution
optimization advantages, 300
Graph Neural Network
irregular computation, 620
Graph optimization
computation graph restructuring, 638
definition, 636
Graph retention
multiple backward passes, 321
Graphcore
IPU, 590
Graphics processing unit, *see* GPU
Great Expectations
data validation, 141
Greedy decoding
LLM generation, 784
Green AI
compute reporting, 895
efficiency as metric, 895
Green500
energy efficiency ranking, 660
HPC energy efficiency ranking, 659
Ground truth
definition, 153
proxy limitation, 153
Group-equivariant CNN, 254
gRPC
inference protocol, 744
GRU
efficiency, 286
parameter efficiency, 286
GTX 580
AlexNet training, 11
Guardrail metric
A/B test constraint, 831
Gustafson's law
appendix refresher, 968
Gustafson, John, *see also* Gustafson's law
Gzip
decompression speed, 164

H100, 407
Half-life
model decay, 823
Hardware abstraction
framework design, 327
two dimensions (data, execution), 327
Hardware abstraction layer
portability, 651
Hardware acceleration
definition, 566
Hardware accelerator, *see* Accelerator
Hardware balance, 599
Hardware cost
deployment spectrum, 48
Hardware implementation
theory-practice gap, 540
Hardware lottery
architectural bias, 708
benchmark bias, 708
Hardware Mapping
combinatorial search space, 622
definition, 619
Hardware mapping
hybrid, layer-specific strategy, 636
Hardware selection
workload matching, 650
Hardware specialization
evolution, 571
Hardware spectrum
deployment platforms, 48
resource progression, 48
Hardware sustainability
carbon footprint, 649
Hardware-Software Co-design
definition, 567
Hardware-software co-design
edge deployment, 925
HBM
3D die stacking, 606
bandwidth-cost trade-off, 576
capacity constraint, 354
memory bandwidth, 409, 794
streaming weights, 281
throughput requirements, 576
Headroom
tail latency, 735
Healthcare AI
deployment gap, 85
Hedging
tail tolerance, 757
HELM
multi-dimensional evaluation, 715
Herding
coreset selection, 455
Heterogeneous computing
mobile SoC, 646
Heuristic
kernel selection trade-off, 639
pruning, 497
Hidden state
definition, 262
Hierarchical processing
data flow, 71
Hierarchical representation
depth advantage, 203
Hierarchical selection
distributed coreset, 472

High bandwidth memory, *see* HBM
High memory latency
  definition, 621
High off-chip bandwidth demand
  definition, 621
High-risk AI
  classification criteria, 906
  EU classification, 880
Hinton, Geoffrey
  backpropagation, 189
  Turing Award, 183
HIPAA
  audit requirements, 908
  compliance, cloud deployment, 51
  ML system constraints, 908
  security safeguards, 51
Hochreiter, Sepp, 286
HOG
  fixed vs. learned features, 185
Holdout test set
  performance evaluation, 826
Host-Accelerator Communication
  transfer bottleneck, 607
Human-in-the-loop
  confidence-based routing, 106
  decision oversight, 883
Hybrid architecture
  combining paradigms, 71
  voice assistant example, 79
  wake-word detection, 53
Hybrid machine learning, *see* Hybrid ML
Hybrid ML
  definition, 71
  hierarchical processing, 71
  integration strategies, 71
  progressive deployment, 71
  train-serve split, 71
Hybrid tiling
  definition, 634
Hyperparameter
  learning rate, 212
  search cost, 98
  tuning, 224
Hypertension
  ClinAIOps case study, 862

i.i.d. assumption
  held-out evaluation limitation, 718
I/O bottleneck
  data loading, 131
I/O-aware design
  algorithm optimization, 413
IB, *see* InfiniBand
Idempotency
  definition, 148
  ML pipelines, 819
  retry safety, 148
IIoT
  latency constraint, 55
Illustrative latency
  definition, 585
im2col, 256
  convolution to matrix, 581
  data center vs. mobile, 256
  memory trade-off, 256
  memory-compute trade-off, 581
Image cropping
  racial bias, 876
ImageNet
  benchmark dataset, 126
  competition progress, 186
  data scale, 126
  macro benchmark reference, 672
  scale, 22, 251
  standardized dataset, 676
  systems consequence, 251
Immutable artifact
  model registration, 819
Immutable data
  JAX, 346
Impact on execution
  definition, 620
Imperative programming
  framework heritage, 346
Implementation strategy
  definition, 585
Importance sampling
  gradient variance, 485
In-memory database
  feature serving, 165
Incident response
  ML system failures, 892
  ML-specific challenges, 846
  structured processes, 846
Inductive bias
  definition, 240
  etymology, 240
  fully connected vs. convolutional, 201
  hierarchy, 287
  systems consequence, 240
  unifying concept, 240
Industry 4.0
  control loop latency, 61
Inference
  batch size selection, 227
  compiler auto-tuning, 690
  compiler, graph optimization framework, 690
  deployment, 17
  deployment phase, 225
  engine, specialized, 775
  etymology, 225
  forward pass only, 210, 226
  latency optimization, 46
  latency requirements, 232
  optimization techniques, 227
  pipeline phase, 746
  real-time, storage requirements, 165
  serverless trade-off, 833
  zero-copy, mobile optimization, 740
Inference attack
  privacy risk, 27
Inference inflation
  definition, 765
Inference runtime
  execution engine, 775
Inference server
  architecture, 742
  serving architecture, 742
Inference-time compute
  reasoning models, 789
InfiniBand
  bandwidth tiers, 608
  RDMA for ML training, 609
Infinity Fabric
  AMD interconnect, 608
Information density
  definition, 947
Information roofline
  definition, 942
Information theory, 947
Information-compute ratio
  accuracy-cost frontier, 446
  definition, 446
  frontier, 447
Infrastructure as code
  ML systems, 827
Infrastructure bottleneck
  definition, 11
Initialization
  glorot, variance scaling, 203
  He initialization, 203
  lazy, deferred compilation, 761
  Xavier/Glorot, 203
Input representation
  definition, 637
Input-stationary
  dataflow strategy, 626
INT8
  appendix refresher, 974
  calibration dependency, 695
INT8 quantization
  bandwidth trade-off, 921
Integration bottleneck
  data movement cost, 576
Intel
  Sapphire Rapids, 596
Intel 8087
  floating-point coprocessor, 572
Intel Gaudi 2
  definition, 597
Intensity regime
  definition, 616, 617
Intentional data engineering
  fairness evaluation, 882
Inter-annotator agreement, 154
Inter-token latency
  throughput trade-off, 716
Interaction deduplication, 454
Interface dependency
  ML system challenge, 809
Intermediate representation
  compiler pattern, 306
Intermittent computing
  power management, 65
Interpretability
  activation visualization, 235
Interpretable model
  transparency by design, 890
Interrupt shielding
  latency isolation, 781
Intersectional analysis
  combined attribute testing, 883
Invariance testing
  fairness verification, 878
IOPS
  storage performance, 159
IoT
  data wall, 56
  deployment scale, 61
  power budget constraints, 896
  power-constrained environments, 698
Iron law of ML systems
  appendix refresher, 937, 968
  definition, 18
  deployment analysis, 42
  equation, 18
  hardware acceleration, 568
  multiplicative savings from data selection, 446
  production monitoring, 836
  sequential execution, 937
  synthesis, 916
  total operations reduction via data selection, 442
  workflow mapping, 90
Irregular computation patterns
  definition, 620
Irregular memory access patterns
  definition, 621
Irregular ML components
  definition, 603
Iteration tax
  definition, 87
Iterative pruning
  definition, 498

Jaccard similarity
  deduplication, 453
JAX
  automatic vectorization, 346
  composable program transformations, 346
  composable transformations, 326
  device-parallel mapping, 347
  functional transformations, 346
  gradient transformation, 346
  JIT compilation, 346
  pure functions requirement, 346
  transform-based AD, 326
  XLA compilation, 347
jax.grad
  composable transformation, 346
Jensen-Shannon divergence
  distribution shift, 841
Jetson AGX Orin
  edge inference, 226
Jevons paradox
  inference demand, 736
JIT compilation
  fidelity vs. generality, 306
  runtime optimization, 780

scripting, 306
shape specialization, 348
tracing, 306
JSON
serialization overhead, 745
storage overhead, 138
Jupyter Notebook
production risk, 821
Just-in-time compilation, 308

k-anonymity
privacy technique, 904
k-center
coreset selection, 455
coverage guarantee, 455
Karpathy, Andrej
Software 2.0, 5
Kendall notation
queuing models, 754
Kernel auto-tuning
algorithm selection, 776
Kernel dispatch
BLAS kernel selection, 354
hardware selection, 338
Kernel Fusion
hardware data movement, 628
Kernel fusion
backward-pass interaction, 319
compression-aware, 541
definition, 300
eliminating intermediate allocations, 319
graph compilation, 779
inference memory traffic, 228
inference optimization, 78
layer fusion, 776
memory efficiency, 629
memory traffic reduction, 628
memory-bound optimization, 617
optimizing memory-bound ops, 300
reducing memory writes, 624
serving compilers, 776
static graph optimization, 304, 305
Kernel launch
fixed overhead, 763
serving overhead, 763
Kernel Launch Latency
serial overhead, 568
Kernel launch latency
driver overhead reduction, 779
Kernel profiler
hardware-level analysis, 671
Kernel scheduling
hardware utilization, 644
Kernel selection
definition, 636
hardware-specific implementation, 639
Key characteristic
definition, 582
Key management
encryption infrastructure, 903
Keyword spotting, 243
automated labeling, 158
definition, 22
energy constraint, 90
sub-megabyte models, 918
TinyML archetype, 44
Killer microseconds
latency gap, 748
Knowledge acquisition bottleneck
definition, 10
human throughput, 10
Knowledge distillation
benchmark evaluation, 714
definition, 506
efficiency gains, 895
etymology, 506
information density, 467
overview, 506
student model, 506
teacher model, 506
teacher-generated targets, 467
temperature parameter, 467, 507
Kolmogorov-Smirnov test
continuous drift, 838
covariate shift detection, 718
drift detection, 107, 139
feature distribution, 818
Kubernetes
ML orchestration, 828
Kullback, Solomon, *see also* Kullback-Leibler divergence
Kullback-Leibler divergence
appendix refresher, 946
definition, 143
distillation loss, 508
distribution comparison, 144, 842
Kung, H.T.
systolic array inventor, 591
KV cache
definition, 265
linear growth, 276
LLM memory, 786
LLM serving bottleneck, 837
memory capacity, 794
memory scaling, 276, 786
multi-turn conversations, 786
offloading, 786
sequence length scaling, 786
session reuse, 786
transformer memory pressure, 635

l-diversity
privacy technique, 904
L0 norm
parameter count, 497
Lab-to-clinic gap
monitoring requirement, 107
Lab-to-field gap
data distribution shift, 95
Label propagation
graph-based semi-supervised, 462
Label quality
consensus mechanisms, 154
systematic checks, 154
Label quality drift
detection, 143
silent corruption, 143
Label shift
definition, 143
Label Studio
data quality, 815
Labelbox, 155
Laboratory-to-deployment gap
performance misalignment, 708
Ladder of abstraction, 298
LAPACK
framework foundation, 298
higher-level numerical routines, 298
Large language model, *see* LLM
Late correction cost
definition, 87
Latency
bandwidth trade-off, 972
decision thresholds, 46
deployment constraint, 41
deterministic, 780
edge deployment, 26
first-token, LLM responsiveness, 716
inference real-time requirement, 691
inference vs. user-perceived, 794
mean, monitoring pitfall, 795
p50, median response time, 745
p95, percentile target, 745
percentile, p50 p95 p99 reporting, 677
percentiles (p50, p95, p99), 745
real-time constraints, 232
responsiveness constraint, 41
selection, dynamic selection bottleneck, 468
serving constraint, 735
throughput trade-off, 795
Latency budget
definition, 745
end-to-end analysis, 832
optimization objectives, 745
request lifecycle breakdown, 745
tail latency constraint, 921
Latency jitter
resource contention, 780
Latency vs. throughput
paradigm trade-offs, 67
trade-offs, 46
Layer
hidden, 198
input, 198
organization, 198
output, 198
Layer fusion, *see* Kernel fusion
Layer normalization, 285
Transformer, 273
Layer sensitivity
precision tolerance, 778
LayerNorm
kernel fusion target, 617
memory-bound operation, 613
Layerwise quantization, *see* Quantization
Lazy Evaluation
data processing, 150
Learnability, 246
Learnability gap, 246
Learning rate
batch size coupling, 212
cosine decay, 380
etymology, 212
scheduling, 380
selection criteria, 224
update magnitude, 212
warmup, 380
Learning rate scheduling
definition, 380
LeCun, Yann, 251
commercial deployment, 251
LeNet, 251
Turing Award, 183
Leibler, Richard, *see also* Kullback-Leibler divergence
Leiserson, Charles E.
systolic array inventor, 591
LeNet
architecture comparison, 232
Li, Fei-Fei
ImageNet creation, 126
Light barrier
latency floor, 41
safety-critical systems, 50
Lighthouse example
definition, 613
Lighthouse model
bottleneck probe, 918
definition, 21
reference workloads, 43
LIME
local explanations, 889
Limited interconnect bandwidth
definition, 620
Limited on-chip storage
definition, 621
Lineage tracking
model provenance, 819
model registry, 840
Linear scaling rule
large batch training, 395
Linear transformation
layer computation, 206
Linnainmaa, Seppo
automatic differentiation, 189
LINPACK
matrix operations benchmark, 658
narrow benchmark, 659
List price
definition, 597
Little's law
appendix refresher, 970
capacity planning, 753

concurrency calculation, 753
Little, John, *see also* Little's law
Llama
bandwidth-bound decode, 22
data selection, 444
memory-bound inference, 44
Llama 2
70-billion-parameter memory-bound workload, 923
Llama 3
8-billion-parameter serving case study, 791
LLM, 243, 444, 895
KV-cache serving, 837
labeling assistance, 156
training scale, 49
LLM performance metrics
definition, 784
LLM serving
8-billion-parameter case study, 791
production case study, 791
token generation, 783
Load balancer
serving infrastructure, 742
Load shedding
retry coordination, 757
Locality crossover
worked example, 59
Locality of reference
ML workload violation, 98
Locality-sensitive hashing
near-duplicate detection, 453
probabilistic bucketing, 453
Log-sum-exp trick
appendix refresher, 947
numerical stability, 211
Logic unit cost
activation functions, 197
Logits
appendix refresher, 947
classification output, 750
inference optimization, 197
logit function, 947
raw network scores, 197
Loop Ordering
computational efficiency, 622
Loss Function
error measurement, 210
optimization target, 210
Loss function
cross-entropy, 210
etymology, 205
landscape geometry, 205
Loss landscape
nonconvex optimization, 213
Loss Scaling
gradient underflow prevention, 323
Loss scaling
appendix refresher, 974
gradient underflow prevention, 404
Lottery ticket hypothesis
compression trade-off, 715
etymology, 505
Low-rank factorization
definition, 509
LPDDR
low-power DRAM variant, 606
LSTM
compute overhead, 286
gating mechanism, 286

M/D/1 queue
deterministic service, 754
M/M/1 queue
Erlang origin, 753
wait time formula, 753
M/M/c queue
multi-server, 754
MAC
definition, 248
energy cost, 248
multiply-accumulate operation, 193
neural computation, 193
Machine learning
definition, 8
Machine learning compiler
definition, 637
Machine learning framework, *see* ML framework
Machine learning lifecycle, 83
definition, 83
iteration cycles, 86
storage requirements, 165
time allocation across stages, 86
Machine learning systems
definition, 14
Machine unlearning
definition, 909
systems trade-offs, 910
Macro benchmark
subsystem evaluation, 669
Magnitude-based pruning
definition, 497
Manifold hypothesis, 245
learnability, 245
MapReduce
data locality, 151
Masked Language Modeling
pretext task, 463
Masked operations
definition, 580
Materialized view
training-serving skew, 141
Matrix engine
definition, 596
Matrix multiplication
dense operations, 391
forward propagation, 206
GEMM optimization, 207
neural network workhorse, 580
training operations, 371
Matrix operation
hardware acceleration, 581
hierarchical decomposition, 580
neural network workloads, 587
Matrix tiling, 282
Maturity level
evolutionary stage, 854
Maximum mean discrepancy
covariate shift detection, 718
McCrory, Dave, *see also* Data gravity
McCulloch-Pitts neuron, 245
Mean time between failures
fleet-scale reliability, 928
Mean time to failure
component reliability, 928
Measurable property
responsibility requirements, 911
Measurement framework
data selection metrics, 449
Measurement variability
sources in ML systems, 661
Mechanical Turk
labeling scale, 129
Medical AI
performance metrics, 99
Membership inference attack
privacy testing, 904
privacy validation, 905
Memory
capacity, LLM throughput, 794
capacity-bound workload, 278
constraints, mobile RAM, 773
locking, mlock, 780
reuse, activation buffers, 228
sharing, GPU multi-model, 761
Memory Access
irregular patterns, 602
Memory access
random, 281
sequential, 281
strided, 281
Memory access pattern
definition, 611
Memory Bandwidth
architectural trade-offs, 606
scaling trends, 598
Memory bandwidth
activation function bottleneck, 374
bottleneck, 13
bottleneck mitigation, 413
bound workload, 277
LLM bottleneck, 793
LLM latency, 794
memory wall, 41
Memory coalescing
bus transaction efficiency, 671
GPU optimization, 627
tensor layout impact, 627
Memory fetching
definition, 628
Memory footprint
computational requirements, 203
training overhead, 215
Memory fragmentation
KV cache, 786
Memory hierarchy
appendix refresher, 971
domain-specific, 573
ML training tiers, 390
speed-capacity trade-off, 604
Memory layout
NCHW vs. NHWC, 338
row-major vs. column-major, 329
stride patterns, 329
Memory management
caching allocators, 325
definition, 637
device placement, 330
inference efficiency, 228
VRAM requirements, 392
Memory mapping
on-demand model loading, 760
Memory planning
compiler phase, 640
definition, 636
tensor allocation, 779
Memory storage order
definition, 628
Memory usage
definition, 634
Memory wall
bandwidth divergence, 41
compute-memory gap, 41
data movement bottleneck, 188
execution strategy impact, 300
framework execution, 300
impact on GPU utilization, 664
Memory-aware tensor layouts
definition, 635
Memory-bandwidth bound
precision impact, 777
Memory-Bound
roofline classification, 611
Memory-bound
activation tensors, 749
operation classification, 300
Memory-computation tradeoff
training systems, 383
Metric
etymology, 677
intransitivity, 677
MFCC
audio features, 123, 148
dimensionality reduction, 148
Micro benchmark
component isolation, 669
diagnostic purpose, 669
Micro-benchmark suite
operator-level testing, 670
Microcontroller
memory ceiling, 65
ML deployment, 66
TinyML substrate, 26
Microservice
cloud model serving, 740
model serving, 740
MIG
hardware isolation, 761
multi-model serving, 761
MinHash

distributed, cross-shard deduplication, 473
near-duplicate detection, 453
signature compression, 453
Mini-batch
gradient descent, 213
Mini-batch gradient descent, *see* Stochastic gradient descent
Mini-batch processing
accelerator efficiency, 375
Mixed-precision
layer-wise precision assignment, 714
Mixed-precision Adam
storage model, 432, 982
Mixed-Precision Computing
training and inference, 594
Mixed-precision training
benchmarking comparability, 684
definition, 363
FP16 and FP32, 684
FP16 vs. FP32 trade-offs, 322
FP16/BF16, 385, 404
loss scaling, 404
memory savings, 402
trade-off, 363
Mixture-of-experts, *see* MoE
MixUp
image augmentation, 465
ML Accelerator
definition, 575
ML compiler
compilation pipeline, 638
definition, 636
vs. traditional compiler, 637
ML development lifecycle
vs. traditional software, 25
ML failure
data dominance, 171
ML framework
compiler analogy, 296
compiler for the silicon contract, 297
compiler role, 45
definition, 297
differentiation problem, 296
dynamic memory management, 384
execution problem, 296
hardware abstraction problem, 296
historical evolution, 298
on-device deployment, 63
platform comparison, 344
ML inference benchmark
definition, 690
ML node
definition, 801
ML system benchmark
definition, 665
ML test score
production readiness, 812
ML training benchmark
convergence throughput scalability, 683
definition, 683
ML workflow
systematic framework, 83
MLCommons
nonprofit benchmark consortium, 697
open submission, 697
MLflow
experiment tracking, 832
MLOps
anti-patterns, organizational failures, 855
centralized mlops team, organizational pattern, 855
definition, 800, 801
DevOps comparison, 801
emergence, 800
federated, embedded engineers, 855
production deployment challenges, 800
MLOps fallacies
automated retraining sufficiency, 864
automatic consistency, 865
DevOps equivalence, 864
MLOps pitfalls
alert routing, 866
monitoring overconfidence, 865
one-time deployment, 864
organizational neglect, 865
MLP
definition, 245
GEMM-dominated computation, 635
MLPerf
founding, 659
founding and purpose, 659
inference benchmarking, 617
power, ML energy measurement, 659
MLPerf client
local generative AI workloads, 698
MLPerf Inference
benchmark family evolution, 697
MultiStream scenario, 774
offline scenario, 774
server scenario, 772
server to edge evaluation, 660
serving scenarios, 772
SingleStream scenario, 773
MLPerf Mobile
on-device inference, 697
MLPerf Power
energy efficiency measurement, 660
MLPerf Tiny
embedded AI benchmark, 698
microcontroller benchmarks, 660
MLPerf Training
data center multi-node scaling, 660
standardized framework, 684
mmap
model loading, 760
MMLU
format limitation, 715
multitask language understanding, 715
MNIST, 246
dataset saturation example, 719
digit recognition, 201
Mobile machine learning, *see* Mobile ML
Mobile ML
battery constraints, 61
computational photography, 63
definition, 61
energy budget, 62
energy constraints, 39
health monitoring, 64
memory bandwidth limits, 63
NPU inference, 63
resource constraints, 26
thermal envelope, 61
voice recognition, 64
Mobile SoC
architecture evolution, 646
MobileNet, 243
MobileNetV2, 891
definition, 22
deployment validation example, 657
depthwise separable convolutions, 44
efficiency gains, 44
efficiency under constraint, 918
latency-constrained deployment, 259
mobile power reduction, 61
workflow constraints, 89
MoCo
queue-based dictionary, 464
Model audit
high-stakes systems, 839
Model benchmarking
definition, 657
multi-dimensional evaluation, 712
Model caching
container embedding, 761
Model card
documentation standard, 891
standardized documentation, 891
Model collapse
recursive synthetic training, 467
tail distribution compression, 467
Model compression
accuracy-efficiency trade-off, 495
definition, 27, 492
efficient model representation, 496
iterative workflow, 100
mobile vision, 61
multi-dimensional benchmarking, 695
silicon contract, 491
Model debugging
decision tree, 847
probabilistic failures, 846
root cause analysis, 846
Model decay
definition, 838
Model deployment
packaging and serving, 829
Model extraction
model artifact security, 904
Model FLOPs utilization
appendix refresher, 941
definition, 396
origin, 396
PaLM benchmark, 396
Model governance
compliance objective, 851
proactive policies, 851
transparency and compliance, 851
Model inversion attack
feature reconstruction, 904
Model loading
full loading, 760
Model Memory Pressure
architecture-specific, 609
Model monitoring
drift detection, 837
performance tracking, 837
Model Optimization
conversion validation, 828
Model Parallelism
training strategy, 686
Model parallelism
layer-wise partitioning, 426
memory constraints, 426
multi-accelerator scaling, 645
pipeline bubbles, 369
Model parameter
storage requirements, 162
Model performance metrics
definition, 107
Model quantization, *see* Quantization
Model registry
centralized artifact management, 832
provenance tracking, 166
version management, 819
version promotion, 819
Model serialization
state dictionary pattern, 342
Model serving
definition, 737
paradigm shift, 735
Model size
parameter count, 203
Model type
definition, 611
Model validation
candidate selection, 826
definition, 101
predeployment assessment, 826

Model versioning
  artifact tracking, 819
  requirements, 162
Model virtualization
  lifecycle management, 761
Model weights
  data governance obligations, 910
  definition, 7
Model-centric AI
  architectural innovation, 718
Model-infrastructure interface
  environment parity, 801
Model-specific execution needs
  definition, 620
Model-specific memory needs
  definition, 621
Module abstraction
  hierarchical composition, 340
  PyTorch nn.Module, 340
Module hook
  forward and backward, 343
MoE
  definition, 519
  dynamic computation paths, 603
Momentum
  memory overhead, 376
  optimization physics, 376
Monitoring
  health, replica readiness, 742
  responsible operations, 892
Monitoring ROI
  calculation, 845
Moore's law
  ML systems consequence, 572
  slowdown, 572
  vs. AI growth, 23
Moravec's paradox
  compute implications, 9
Multi-chip scaling
  beyond single accelerator, 644
  communication overhead, 645
Multi-head attention, 272
Multi-instance GPU, *see* MIG
Multi-model serving
  GPU memory management, 761
Multilayer perceptron, *see* MLP
Multiply-Accumulate
  MAC unit, 591
MultiStream
  synchronized sensor fusion, 698
MYCIN
  expert system, 10

NAS
  bi-level optimization, 515
  bi-level optimization cost, 515
  definition, 515
  hardware-aware, 515
  search cost, 492
Naïve execution
  definition, 629
NCHW
  channels-first layout, 624, 627
Network depth
  training difficulty, 283
Network interface card
  node interconnect, 608
Network topology
  layer organization, 201
Neural architecture search, *see* NAS
Neural Network
  artificial neuron, 192
  depth vs. width trade-off, 204
Neural network
  capacity vs. cost trade-off, 193
  feedforward, 201
  layers, 198
  perceptron, 194
Neural network application
  definition, 580
Neural processing unit
  energy efficiency, 63
  mobile devices, 63
Neuron
  McCulloch-Pitts model, 192
Next-Token Prediction
  pretext task, 463
NHWC
  channels-last layout, 624, 627
NHWC vs. NCHW
  performance impact, 627
NN
  dominant MAC pattern, 577
nn.Module
  hierarchical composition, 342
  parameter discovery, 340
  parameter management, 340
  registered buffer, 340
  train vs. eval modes, 341
No free lunch theorem, 246
  architecture selection, 246
Non-maximum suppression
  irregular memory access, 646
Non-uniform memory access, *see* NUMA
Nonlinearity
  XOR problem, 201
Normalization
  batch, 285
  feature scaling, 147
  layer, 285
  parameter persistence, 147
  RMSNorm, 285
Notebook to production
  handoff challenges, 852
Novelty Effect
  A/B testing bias, 831
NPU
  efficiency trade-off, 646
  inference benchmarking considerations, 690
  mobile SoC, 646
  neural network acceleration, 701
  on-device strategy, 588
Nsight Compute
  profiling tool, 334
Nsight Systems
  profiling tool, 334
Nucleus sampling
  top-p, 784
NUMA
  inference latency, 781
  memory locality, 781
Numerical precision optimization, *see* Quantization
NumPy
  framework foundation, 298
  framework lineage, 299
NVIDIA
  A100, 597
  Ampere architecture, 595
  H100, 575
  V100, 597
NVIDIA A100
  definition, 596
NVIDIA Accelerator
  definition, 612
NVIDIA GTX 580
  AlexNet memory constraint, 186
NVIDIA H100
  definition, 597
NVIDIA Jetson
  edge trade-off, 95
NVIDIA Nsight
  systems and compute profiling, 671
NVIDIA Nsight Systems, 399
NVLink
  bandwidth hierarchy, 331
  definition, 608
  GPU interconnect, 427, 608
  gradient synchronization, 427
  intra-node scaling, 607
  PCIe bottleneck, 608
  training interconnect, 427
  training scaling, 608
NVMe
  data pipeline, 163
  GPU utilization, 161
  storage bandwidth, 391

Object detection
  bounding boxes, 153
Observability
  system state inference, 837
Observable degradation
  continuous measurement, 803
Offline capability
  paradigm comparison, 67
Offline scenario
  maximum throughput mode, 698
OLAP, 160
OLTP, 159
On-call practice
  ML systems, 849
On-call rotation
  ML-specific requirements, 849
On-chip memory
  definition, 605
One-hot encoding
  classification labels, 210
  sparsity scaling, 210
One-shot pruning
  definition, 503
oneDNN
  Intel optimization library, 639
Online inference
  definition, 836
Online softmax
  incremental computation, 410
ONNX
  cross-framework portability, 350
  definition, 350
  framework portability, 350
  hub-and-spoke interoperability, 350
ONNX Runtime
  cross-platform serving, 775
Open division
  MLPerf submission, 697
OpenCL
  cross-processor execution, 648
OpenVINO
  Intel inference, 775
Operating system
  runtime role, 45
Operation type
  definition, 582
Operational maturity
  system integration, 854
Operational mismatch
  ML vs. traditional software, 800
Operator fusion
  appendix refresher, 955
  latency gap elimination, 671
Operator kernel
  dispatch hierarchy, 337
  GPU definition, 300, 763
Optimization
  Amdahl's law, 77
  compile-time, constant folding, 779
  hardware, architecture-specific, 228
  inference, per-query cost, 897
  performance, inference, 227
Optimization difficulty, 246
Optimization priorities
  definition, 637
Optimization priority
  definition, 616, 617
Optimization stack
  multiplicative effects, 483
Optimization technique
  definition, 635
Optimizer
  Adam, 220
  adaptive learning rate, 376
  momentum-based methods, 376
Optimizer state
  appendix refresher, 978
  memory overhead, 363
Optum, 876
ORC
  columnar format, 164
Oura Ring
  wearable ML case study, 858
Out-of-memory error
  training constraint, 409

Outlier Detection
  data cleaning, 146
Output-stationary
  partial sum accumulation, 625
Over-the-air update
  edge deployment, 859
  model distribution, 833
Overfitting
  benchmark, over-optimization to test sets, 710
  definition, 223
  training challenge, 224
Overhead-bound
  small model training, 356
Overparameterization
  compression rationale, 496
  generalization benefit, 224

Pacing function
  linear warmup, 458
Padding
  batching overhead, 795
PagedAttention
  memory efficiency, 786
  memory fragmentation solution, 786
Pandera
  schema validation, 141
Papermill
  notebook parameterization, 821
Parallel Computing
  Amdahl's Law, 568
Parallel fraction
  accelerated path, 568
Parallel processing
  neural networks, 209
Parameter
  memory multiplier, 198
  registration, automatic tracking, 340
  reuse, hierarchical learning, 204
Parameter freezing
  transfer learning, 342
Parameter tracking
  transformation lineage, 152
Parameter update
  optimizer step, 394
Pareto Frontier
  serving trade-offs, 765
Pareto frontier, 241
  accuracy-efficiency trade-off, 714
  compression trade-off, 496, 714
  fairness-accuracy trade-off, 877
  multi-objective optimization, 920
  throughput-latency, 770
Parquet
  columnar format, 163, 164
  I/O reduction, 150
Partial derivative
  gradient computation, 216
Patient treatment loop, 861
PCIe
  bandwidth generations, 608
  host-device interconnect, 331, 608
  host-to-device transfers, 607
Peak FLOPS
  misleading metric, 650
Peak FP16
  definition, 612
Perceptron
  computational primitive, 245
  early neural computation, 8
  etymology, 245
  linear separability limit, 194
  Rosenblatt, 194
Perceptual hashing
  image deduplication, 454
Performance gains
  definition, 634
Performance per watt
  efficiency metric, 649
Performance-per-data
  metric definition, 449
Periodic score refresh
  active learning, 473
Peripheral component interconnect express, *see* PCIe
Perplexity
  etymology, 715
  filtering, low-information text removal, 454
  language model prediction measure, 715
  serving interpretation, 715
Personalized model
  user adapters, 787
Photoplethysmography
  data quality challenge, 862
  wearable sensing, 862
Physical AI
  real-time control constraints, 926
Physical constraints
  deployment implications, 39
  memory signaling, 40
  permanent boundaries, 78
  speed of light, 40
  thermodynamics, 40
Pickle
  loading overhead, 781
Pile, The
  data diversity, 452
Pinned memory
  definition, 331
  DMA transfer, 332, 762
PipeDream
  pipeline parallelism, 369
Pipeline
  cascading failures, 118
  debt, workflow fragmentation, 810
  failure diagnosis, 171
  failure modes, 139
  overlapping, latency hiding, 401
Pipeline Parallelism
  training strategy, 686
Pipeline parallelism
  micro-batch scheduling, 428
  micro-batching, 428
Pipelined execution
  throughput analysis, 42
Placement considerations
  definition, 622
Platt scaling
  calibration, 102
Point-in-time correctness, 166
  time travel, 167
Poisson distribution
  traffic modeling, 678
Poisson process
  queuing model, 754
  serving arrivals, 772
Polysomnography
  label uncertainty, 859
Pooling
  global average, 253
Population stability index
  appendix refresher, 946
  credit risk origin, 841
  definition, 143
  drift detection, 107, 144, 838, 842
  drift quantification, 841
Portrait Mode
  pipeline latency, 64
Positive Predictive Value
  prevalence dependence, 99
Post-training quantization
  calibration, 778
  definition, 522, 531
Postmortem documentation
  incident analysis, 846
Postprocessing
  confidence thresholds, 232
  logits to predictions, 749
  response formatting, 746
Power budget
  edge devices, 896
Power management
  mobile AI, 647
Power measurement
  evaluation triad completion, 702
Power usage effectiveness
  data center cooling metric, 705
Power wall
  frequency scaling, 41
  mobile implications, 63
  thermal limits, 41
Pre-annotation
  workflow, 156
Precision
  dynamic, adaptive quality, 778
  hardware design parameter, 594
  numerical format trade-offs, 973
  numerical, inference optimization, 228
  positive prediction accuracy, 677
  reduced, architectural trade-off, 585
  throughput trade-off, 777
Predictive entropy, 144
Predictive maintenance
  edge deployment, 61
  edge duty cycle, 61
Prefetching
  accelerator optimization, 573
  data loader pipeline, 335
  double-buffering, 163
Prefill phase
  compute-bound, 793
  parallel processing, 793
Prefix caching
  KV cache reuse, 786
Preprocessing
  digit segmentation, 232
  divergence, training vs. serving, 759
  GPU, accelerated pipelines, 748
  image normalization, 232
  latency impact, 746
Pretext Task
  self-supervised learning, 463
Price of fairness
  utility trade-off, 888
Price/performance
  definition, 597
Primary pressure
  definition, 603
Primary workloads
  definition, 596
Privacy
  always-listening devices, 905
  technical protection methods, 904
  technique hierarchy, 904
Probabilistic engineering, 7
Problem definition
  ML vs. traditional, 93
  multi-constraint optimization, 93
ProcessGroup
  distributed execution, 337
Processing element
  accelerator building block, 576
Production monitoring
  continuous benchmarking, 722
Production-monitoring interface
  feedback loop, 801
Progressive deployment
  model compression, 72
Projection pushdown, 944
Prometheus
  monitoring trade-off, 834
Protocol Buffers
  serialization, 744
Proxy metrics
  definition, 107
Proxy model
  practical workflow, 456
Proxy variable
  bias detection, 871
  bias reconstruction, 910
  definition, 871
Pruning
  carbon reduction, 895
  channel, 499
  definition, 497
  dynamic, 502
  layer, 499
  low-information filtering, 454

neuron, 498
Optimal Brain Damage, 497
outlier removal, 454
quality, label error detection, 454
static, 502
structured, 500
unstructured, 500
Pseudo-labeling
confirmation bias, 462
semi-supervised learning, 462
semi-supervised technique, 462
PUE
definition, 48
efficiency overhead, 48
power usage effectiveness, 900
Pure function
JAX, 346
JIT requirement, 348
Python dispatch
overhead, 354
PyTorch
autograd implementation, 326
autograd system, 320
eager execution default, 345
preprocessing parameters, 152
profiler, 399
research-first design, 345
tensor layout, 626
PyTorch profiler
performance analysis, 671

QKV
KV cache, 267
Quality control
edge processing, 61
Quality debt
uncorrected errors, 169
Quantization
activation-aware weight (AWQ), 535
affine, 534
appendix refresher, 974
asymmetric, 533, 974
AWQ bandwidth reduction, 535
channelwise, 535
definition, 521
energy impact, 530
environmental benefits, 895
extreme (INT4, binary), 522
groupwise, 535
inference precision, 226, 330
information-theoretic origin, 522
INT8 co-design, 567
INT8 compression ratio, 657
INT8 energy savings, 704
layer sensitivity, 778
layerwise, 535
reduced precision tolerance, 576
scale, 534
serving precision, 777
sub-channelwise, 535
symmetric, 533, 974
TinyML limits, 77
zero-point, 534
Quantization calibration
entropy method, 533
max method, 533
percentile method, 533
Quantization granularity, *see* Quantization
Quantization-aware training
accuracy preservation, 538
definition, 522
Queries per second
inference throughput metric, 695
Query key value, *see* Query-key-value
Query-by-committee
active learning strategy, 459
Query-key-value, 267
Queuing Theory
serving foundations, 796
Queuing theory
serving systems, 752
utilization divergence, 754

RandAugment
hyperparameter simplicity, 466
Random sampling
minority representation cost, 882
Random seed
benchmark protocol requirement, 662
Ray
distributed ML, 832
RDMA
multi-node scaling, 609
Reasoning model
inference-time compute, 789
Recall
positive case detection rate, 677
Receptive field, 252
depth trade-off, 252
Recommendation system
DLRM, 44
scalability challenge, 121
Reconstruction attack
federated learning vulnerability, 905
Recurrent neural network, *see* RNN
Red AI
performance at any cost, 895
Red teaming
stakeholder engagement, 878
Redis
online feature serving, 165
Reduction
definition, 580
Register
fastest memory, 604
Regular dense kernels
definition, 603
Regularization
definition, 223
preventing overfitting, 223
Regulatory compliance
AI governance, 905
architecture constraints, 906
business value, 880
EU AI Act, 905
Relative capacity
definition, 604
Relative latency
definition, 604
Reliability
circuit breakers, 145
dead letter queues, 144
error handling, 144
idempotency, 148
ReLU, 284
arithmetic intensity, 968
computational efficiency, 196
conditional operation overhead, 583
dead neuron, 284
dying ReLU problem, 196
etymology, 196
hardware efficiency, 196, 372
Remote direct memory access, *see* RDMA
Reorder computations
definition, 638
Representation capacity, 245
Reproducibility
benchmark requirement, 678
cross-platform challenge, 708
experiment lineage, 100
lineage, 152
versioning principle, 802
Reproducible system artifact
definition, 101
Request pipelining
GPU utilization, 751
Request transformation
tensor formats, 743
Residual connection
Transformer, 273
Residual learning, 284
ResNet
human-level performance, 186
MLPerf reference model, 712
ResNet-50, 242
activation memory, 393
arithmetic intensity, 244
batching efficiency, 763, 918
cloud vs. mobile, 47
compute-bound workload, 569
data selection, 444
data-loading requirements, 132
iron-law workload, 21
latency budget, 746
module hierarchy, 340
roofline analysis, 666
systems characteristics, 44
training memory footprint, 321
weight reuse mapping, 634
workflow constraints, 90
Resource isolation
serving, 780
Resource utilization
compute and memory, 687
Responsibility gap
definition, 871
Responsible AI engineering
definition, 879
Responsible engineering
lifecycle methodology, 881
safety control loop, 870
Retraining, 17
continuous updates, 822
decision framework, 821
economics, cost-benefit optimization, 823
scheduled updates, 822
scheduled vs. triggered, 821
triggered updates, 822
Retry logic
exponential backoff, 144
Return on compute
definition, 21
Return on investment
amortized, 476
data selection framework, 475
ML systems, 897
ML systems, heuristic baseline, 897
Reweighting
bias mitigation, 886
importance sampling, 886
RFM Analysis
feature engineering, 147
Ridge Point
hardware balance, 599
hardware comparison, 612
Ridge point
appendix refresher, 967, 977
definition, 19
machine balance ratio, 236
precision dependence, 384
Right to erasure
data lineage requirement, 906
Ring AllReduce
bandwidth optimization, 428
RISC-V
AI customization, 579
vector extensions, 579
Risk mitigation
responsible engineering value, 880
RMSNorm, 285
efficiency, 285
LLM adoption, 285
RMSprop
adaptive step sizes, 376
origin and Adam inheritance, 376
RNN
definition, 261
sequential dependency bottleneck, 265
Role-based access control
data governance, 903
Rollback
delayed, 830
immediate, 830
ML state mismatch, 108
model state management, 830
rapid, 830

strategies, safety mechanisms, 830
Roofline Analysis
conv2d layer, 614
dense layer, 615
layer normalization, 616
Roofline Model
bottleneck analysis, 46
efficiency measurement, 611
Roofline model
appendix refresher, 966
compute vs. memory bound, 358
definition, 244
etymology and origin, 666
hardware diagnosis, 571
optimization diagnostic, 611
training bottlenecks, 384
Root/power unit
definition, 585
Rosenblatt, Frank, 245
Rotting asset curve
accuracy decay, 839
Row-oriented storage
CSV, 164
Rule-based programming
limitations, 184
Rumelhart, David
backpropagation, 189

Safetensors
secure model loading, 781
zero-copy loading, 781
Safety
human oversight requirements, 883
responsible engineering, 870
Sample complexity, 246
Sampled softmax
compute efficiency, 210
Sampling
temperature, top-k, top-p, 784
Samuel, Arthur
machine learning, 8
SavedModel
TensorFlow export format, 345
Scalability
deep learning, 186
distributed training metric, 685
paradigm comparison, 67
Scale AI, 155
Scaling
attention, numerical stability, 273
frequency, cubic power relationship, 705
hybrid, GPU+CPU, 790
infrastructure, voice assistants, 54
physical ceiling, 430
Scaling efficiency
strong scaling definition, 687
Scaling laws
appendix refresher, 941
data limitations, 77
data-compute asymmetry, 442
power-law origin, 442
Scatter
definition, 580
Scheduler
inference server, 743
Schema contract
data producers, 171
Schema debt
workarounds, 169
Schema evolution
failures, 144
silent failure, 144
Schema validation
data quality, 840
ML pipelines, 139
structural data checks, 840
Schema-on-read
flexibility, 160
governance trade-off, 160
Schema-on-write
traditional databases, 160
Schmidhuber, Jürgen, 286
Scope creep
model cards, 891
Scratchpad Memory
accelerator design, 576
software-managed storage, 605
Scratchpad memory
definition, 604
explicit control, 282
ML advantage, 576
Secure Enclave
biometric privacy, 63
Secure enclaves
trusted execution environment, 903
Security
access control architecture, 903
Select kernels
definition, 638
Selection dependency
distributed training, 472
Selection engineering
definition, 467
Selection inequality
definition, 468
Selection logic
definition, 576
Selective FP32 computation
mixed precision, 406
Selective gradient computation, 321
Self-attention, 272
Self-paced learning
adaptive difficulty, 459
Self-supervised learning
definition, 463
foundation model paradigm, 464
iron law restructuring, 463
Semantic segmentation
pixel-level labels, 153
Semi-supervised learning
consistency plus pseudo-labeling, 462
definition, 461
labeling efficiency, 156
Sensitivity
medical AI metrics, 98
Separation of concerns
MLOps layers, 803
Sequence length variability
batch waste, 785
Serial fraction
Amdahl bottleneck, 568
Serialization
definition, 944
overhead, 744
Server scenario
Poisson-distributed traffic, 698
Server traffic
Poisson process, 772
Serverless AI
cold-start benchmarking, 695
Serving
cloud/data center, 740
edge inference, embedded serving, 740
mobile/edge, 740
production deployment, 735
TinyML, 741
Serving inversion
throughput-to-latency shift, 735
Session affinity
sticky sessions, 787
Severity classification
incident prioritization, 846
Shadow deployment
inference cost, 829
production validation, 829
Shallow learning
definition, 183
Shannon, Claude
information theory, 210, 947
SHAP
feature attribution, 889
inference latency, 890
SHAP value
model debugging, 847
Shapley value
feature attribution, 890
Shift handoff
context transfer, 850
Shuffle buffer
approximate random sampling, 469
SIFT
variable output size, 186
Sigmoid
computational cost, 195
etymology, 195
Signal-to-noise engineering, 117
Signal-to-noise ratio
definition, 947
Silent degradation, 17
Silent failure
detection challenges, 873
gradual degradation, 803
Silicon contract
constraint propagation, 916
framework compilation, 297
hardware side, 46
iron law, 42
physical constraints, 40
SimCLR
batch requirements, 464
SIMD
CNN inference, 258
CPU inference, 781
CPU vectorization, 781
data-level parallelism, 578
definition, 186
edge deployment, 258
hardware execution, 586
vector parallelism, 228
SIMT
definition, 586
Single instruction multiple data, *see* SIMD
Single instruction multiple thread, *see* SIMT
Single-Machine Processing
scalability, 150
Single-metric evaluation
fallacy, 723
Single-node regime
definition, 28
Single-node stack
architecture layers, 45
Single-user traffic
mobile serving, 773
SingleStream
sequential inference scenario, 698
Singular value decomposition
rank-bandwidth trade-off, 509
Skip connection
cross-architecture portability, 283
gradient flow, 202, 242
SLA
contractual penalty, 746
data freshness, 134
definition, 692
SLI
definition, 746
Slice analysis
subpopulation debugging, 847
SLO
capacity planning, 752
definition, 692
percentile targets, 755
vs. SLA, 738
vs. SLI vs. SLA, 746
Snappy
decompression speed, 164
Snorkel
weak supervision, 156
SoC
always-on island, 123
Soft affinity
load balancing, 787
Softmax
appendix refresher, 947
etymology, 197
global normalization, 373
memory access pattern, 373

memory-bound operation, 613
numerical stability, 197
probability distribution, 197
Softmax function
quadratic bottleneck, 267
systems, 267
Software 1.0
definition, 5
Software 2.0
definition, 5
Sparse connectivity
connection patterns, 202
Sparse scatter
definition, 43
Sparse Tensor Core
2:4 constraint, 590
Sparsity
activation sparsity, 196
definition, 543
hardware gap, 543
hardware speedup requirement, 588
memory access irregularity, 603
N:M structured pattern, 588
structured, 2:4 hardware support, 590
SPEC CPU
real application workloads, 658
SPEC Power
energy efficiency, 659
server energy efficiency, 659
SpecAugment
audio augmentation, 129
Special Function Unit
activation functions, 576
SFU, 582
Specificity
medical AI metrics, 98
Spectrogram
audio features, 148
audio representation, 148
Speculative decoding
draft model verification, 785
latency reduction, 785
throughput, 785
Speech recognition
forced alignment, 158
Speed of light check
utilization diagnosis, 670
SQL
feature pipeline primitives, 945
SQuAD
reading comprehension dataset, 676
saturation, 676
SRAM
on-chip cache, 604
on-chip fast memory, 605
SSD cache
model loading, 761
Stable Diffusion
generation cost, 466
Stage interface specification
contracts, 87
Staged rollout
canary, 102
shadow mode, 102
workflow, 102
Staging validation
probabilistic adequacy, 829
Stakeholder communication
business alignment, 852
technical translation, 852
Staleness cost
accuracy decay model, 823
Standardization
feature scaling, 147
Static batching
fixed batch size, 765
Static graph
define-then-run model, 304
definition, 309
optimization benefits, 306
Static inference
precomputed predictions, 739
Static pruning
coreset selection, 455
definition, 443
pretraining filtration, 452
Static quantization, *see* Quantization
definition, 535
Static random access memory, *see* SRAM
Static single assignment
appendix refresher, 955
compiler IR, 306
Statistical decision theory
loss function origin, 205
Statistical drift risk
accuracy erosion, 921
degradation equation, 839
Statistical learning
definition, 10
Statistical telemetry
ML observability, 801
Statistical validation
sampling strategies, 141
Statistics unit
definition, 585
Stochastic gradient descent
etymology, 375
gradient estimation, 375
memory-utilization trade-off, 375
mini-batch procedure, 222
Robbins and Monro, 375
systems trade-off, 7
Storage
activation, backpropagation, 215
cold, archival, 166
columnar, ML training, 160
Store-and-forward architecture
intermittent connectivity, 96
Straight-through estimator
gradient approximation, 537
quantization-aware training, 537
Stratified evaluation
population-based testing, 883
Stratified sampling
fairness evaluation, 882
Stream ingestion
definition, 133
Streaming multiprocessor
GPU architecture, 586
occupancy, 586
Streaming response
LLM serving, 785
Streaming traffic
sensor synchronization, 772
Strong scaling
appendix refresher, 968
definition, 686
Structured approximation
definition, 509
Structured pruning
hardware alignment, 567
Structured query language
feature pipeline primitives, 945
Structured sparsity
N:M pattern, 501, 588
STUDENT
symbolic AI failure mode, 8
Subgroup imbalance
demographic disparity, 717
Subnormal numbers
gradient underflow, 215
Supervised learning
labeled examples, 204
Supported Tensor Core precisions
definition, 595
Sustainability
environmental responsibility, 895
Sutton, Richard
bitter lesson, 12
Switch Transformer
scaling trade-off, 519
Symbolic AI
definition, 8
Symmetry-aware architecture, 250
Synchronization
device vs. stream, 333
Synchronization event
fine-grained, 333
inter-stream, 333
Synchronous scoring
active learning, 473
Synthetic data
generation, 129
rare event coverage, 129
scalability, 129
simulation, 129
Synthetic data generation
definition, 443
three-stage pipeline, 464
Synthetic Microbenchmark
limitations, 663
System Bottleneck
identifying, 42
System bottleneck
dominant constraints, 48
System composition, 927
System efficiency
feeding tax, 116
hardware duty cycle, 751
System entropy
definition, 75
model decay, 75, 823
System is the model, The
definition, 917
System-on-chip
energy integration, 63
heterogeneous AI acceleration, 646
mobile architecture, 63
Systematic error
learned ground truth, 718
Systems co-design, 11
Systems thinking
principles in ML, 84
Systolic Array
data flow architecture, 591
dataflow design, 591
energy efficiency, 591
TPU configuration, 591
Systolic array
CNN data reuse, 283
definition, 573
etymology, 283
TPU, 283

t-closeness
privacy technique, 904
T4
serving economics, 791
Tail at scale
fan-out amplification, 755
Tail latency
appendix refresher, 945
fan-out amplification, 692
p99 significance, 692
revenue impact, 745
utilization threshold, 736
Tanh
etymology, 196
zero-centered advantage, 196
Task scheduling
definition, 642
Task-relevant signal
definition, 116
Technical debt
abstraction debt, missing ML interfaces, 810
assessment rubric, 812
data dependency debt, unstable inputs, 809
data engineering, 169
dead code, experimental ML paths, 810
definition, 806
ML compounding, 806
ML entanglement, 89
ML system complexity, 806
ML systems, 89, 892
Technology S-Curve
computing paradigms, 573
Telemetry
ML observability, 801
Temperature
softmax distillation, 507
Temperature scaling

calibration, 102
calibration restoration, 715
Tensor
appendix refresher, 951
data types (dtypes), 330
definition, 328
etymology, 328
hardware contract, 328
memory layout, 183
metadata, 329
n-dimensional arrays, 183, 328
operations, GEMM, 338
stride patterns, 329
Tensor arena
TinyML memory, 741
Tensor Core
dimension alignment, 587
dimension constraints, 249
energy efficiency, 587
hardware execution, 587
matrix multiplication, 576
matrix multiply-accumulate, 407
mixed precision, 249
mixed precision acceleration, 406
mixed-precision acceleration, 322
precision support, 588
reduced-precision matrix units, 226
throughput mechanism, 588
training throughput, 406
Tensor decomposition
definition, 510
Tensor-Train, 511
Tensor layout
hardware alignment, 626
memory-aware, 624
NCHW vs. NHWC, 743
Tensor memory layout
NCHW, 743
NHWC, 743
Tensor parallelism
intra-layer splitting, 428
operation splitting, 428
Tensor Processing Unit, *see* TPU
TensorFlow
full framework, 351
graph-first architecture, 344
preprocessing parameters, 152
profiler, 399
symbolic differentiation, 326
tensor layout, 626
TensorFlow Data Validation, 140
TensorFlow Lite
mobile deployment, 351
TensorFlow Lite Micro
fixed memory arena, 314
microcontroller deployment, 351
TensorFlow Serving
inference server, 742
production deployment, 345
TensorFlow.js
browser deployment, 345
TensorRT
accelerator compilation, 620
GPU optimization, 775
hardware utilization, 350
model optimization, 828
Ternarization
definition, 539
Terraform
infrastructure as code, 828
Tesler's law
conservation of complexity, 920
tf.data, 150
tf.function
graph compilation, 352
TF32
tensor float 32, 595
TFRecord
definition, 163
Theano
computational graph origin, 299
Thermal design power
mobile constraint, 61
Thermal management
mobile constraints, 647
Thermal throttling
impact on sustained perf, 670
intelligent migration, 647
mobile serving, 773
performance limits, 76
Thermal trip point
hardware safety, 63
Thermal wall
definition, 62
mobile constraints, 63
Thirteen quantitative principles
framework, 919
Thread pinning
CPU affinity, 781
Threshold adjustment
fairness mitigation, 887
Thresholding
confidence cutoff, 750
Throughput
accelerator parallelism, 209
definition, 685
etymology, 685
storage bandwidth, 159
training, compute vs. data, 163
vs. latency trade-off, 677
Throughput ceiling, 289
Tied request
latency reduction, 757
Tiered escalation
incident response, 850
Tiered retention
hot/warm/cold, 166
Tiered storage
edge deployment, 165
hot warm cold architecture, 96
ML I/O bottleneck, 96
Tiling
algorithms, matrix computation, 413
attention computation, 410
data reuse, 188
definition, 339
DRAM traffic reduction, 631
explicit staging, 632
implicit cache blocking, 632
memory optimization, 624
memory-efficient strategies, 631
principle, hardware-graph bridge, 592
Time per output token
decode phase, 784
definition, 784
Time to first token
definition, 784
prefill phase, 784
Time travel
feature stores, 167
Time-multiplexing
model swapping, 761
Time-to-accuracy
training benchmark primary metric, 685
Timeline profiling
serving optimization, 782
Tiny constraint
definition, 43
TinyML, 26
always-on sensing, 39
bare-metal serving, 740, 741
cost efficiency, 64
definition, 65
device form factor, 66
ecosystem fragmentation, 67
energy efficiency, 58
energy gap, 65
medical wearables, 67
memory planning, 313
micro-runtimes, 313
model compression, 66
operational constraints, 833
precision agriculture, 67
resource constraints, 66
training memory constraint, 66
ubiquitous sensing, 64
wake-word detection, 66
Token throughput
generation rate, 716
Tool-first anti-pattern
fragmented responsibilities, 855
Top-k
sampling, 784
TOPS
comparability limitation, 696
TOPS/W
primary design objective, 702
torch.compile
hybrid execution, 308
TorchScript
tracing and scripting, 306
Total cost of data
formula, 474
Total cost of ownership
cloud vs. edge, 51, 52
definition, 897
deployment decisions, 51, 52
deployment economics, 52, 53
inference dominance, 897
operational costs, 897
ToyADMOS
domain gap, 676
TPU, 897
benchmarking considerations, 665
data center integration, 575
design trade-off, 191
matrix unit design, 372
origin, 571
specialization trade-off, 50
systolic array scaling, 591
XLA compilation, 357
Trace-based benchmarking
production replay, 663
Traditional compiler
definition, 637
Traditional runtime
definition, 642
Traffic-adaptive batching
window tuning, 774
Train-serve split
cost asymmetry, 71
economics, 71
Training
bidirectional computation, 46
computational cost, 370
computational intensity, 363
cost asymmetry, 362
data throughput, 46
inference vs. training, 362
infrastructure evolution, 430
mathematical foundations, 370
memory decomposition, 954
memory equation, 381
memory exhaustion threshold, 429
memory management, 383
memory pressure, 363
multi-GPU configuration, 424
scaling strategies, 424
scaling threshold, 429
step, anatomy, 354
supervised learning, 204
systems, 362
time estimation, 969
training-time law, 364
wall-clock time limit, 430
weight optimization, 213
weight update, 394
Training bottleneck
compute-bound vs. memory-bound, 396
min-of-three relationship, 390
profiling, 396
Training cost
electricity, 978
scaling trajectory, 362
Training cost asymmetry
definition, 362
Training loop
iterative learning, 222
Training memory
Adam storage model, 965
Training optimization

technique selection, 398
Training pipeline
architecture, 386
automated ML workflows, 821
data flow, 386
staged system workflow, 363
Training profiler
bottleneck analysis, 397
timeline visualization, 397
Training systems
definition, 362
Training vs. Inference
accelerator design, 575
Training vs. inference
operational differences, 226
resource differences, 225
Training-Serving Consistency, 146
Training-serving skew
accuracy impact, 816, 921
causes, 141
definition, 17, 758
diagnosis and prevention, 817
serving accuracy fallacy, 795
silent accuracy degradation, 758, 817
Training-time law
definition, 364
utilization gap, 364
Transfer learning
compute efficiency, 98
definition, 98
labeling alternative, 156
Transform tensor layouts
definition, 638
Transformation versioning
reproducibility, 152
Transformer
architectural shift, 266
definition, 271
etymology, 266
KV-cache memory pressure, 635
recurrence removed, 271
training evolution, 366
Transistor tax
activation functions, 197
Translation equivariance, 251
systems trade-off, 251
Translation invariance, 246
Transparency
governance objective, 851
regulatory requirements, 889
Triton
inference server, 742
True positive rate
fairness computation, 885
Trust
competitive differentiation, 880
Tucker decomposition
definition, 511
Turing
definition, 595
Turing, Alan
Imitation Game, 8
Typical operations
definition, 616, 617

Ubiquitous computing
etymology, 64
TinyML realization, 64
Uncertainty sampling
active learning query strategy, 459
Undeclared consumer
hidden model dependency, 809
Tesla case study, 811
Unified Memory
single address space, 609
Unified memory
mobile SoC, 646
Uniform quantization
definition, 531
Universal approximation theorem, 198, 245
depth vs. width, 198
systems consequence, 245
Unreasonable effectiveness of data
systems impact, 10
Unstructured pruning
hardware limitations, 544
Update rule
gradient descent, 212
Upstream echoing
definition, 470
URLLC
edge inference constraint, 702
Utilization
high utilization pitfall, 795
latency relationship, 754

V100, 371
batching throughput, 762
Validation
accuracy plateau, 224
Vanishing gradient problem, 265
activation saturation, 196
exponential decay, 196
Vector execution
definition, 596
Vector index
embedding-based selection, 469
Vector Operation
hardware acceleration, 578
Vector operation
definition, 580
Vector Unit
element-wise operations, 576
Vector-scalar broadcast
definition, 580
Vectorization
automatic batching, 346
columnar processing, 944
Vendor lock-in
cloud deployment, 53
portability trade-off, 651
Verification gap
definition, 7
MLOps management, 826
statistical testing, 921
Version pinning
inference runtime, 828
Viola-Jones algorithm
compute efficiency, 11
Virtuous cycle
definition, 462
Vision transformer, 287
compute trade-off, 287
ViT, *see* Vision transformer
vLLM
virtual memory analogy, 785
Voice assistant
hybrid architecture, 53
Voice Match
enrollment pipeline, 121
Volta
definition, 595
Von Neumann bottleneck
ML accelerator constraint, 598
VRAM
device memory, 761

Wake-word detection
always-on power budget, 67
layered architecture, 53
Wald, Abraham
statistical decision theory, 205
Warehouse-scale computing
training evolution, 430
Warmup
cache population, 760
cold start mitigation, 760
Warmup runs
benchmarking methodology, 723
Warp
divergence penalty, 586
etymology and hardware constraint, 391
execution unit, 586
GPU execution unit, 391
SIMT execution bundle, 671
Waterfall model
contrast with ML, 87
Wave quantization
accelerator utilization, 391
tail effects, 391
Waymo
domain gaps, 27
Weak scaling
appendix refresher, 969
definition, 687
Weak supervision
cost trade-off, 156
programmatic labeling, 156
Web scraping
ML training data, 127
WebDataset
sequential sharding, 163
Weight decay
definition, 224
Weight loading
cold start, 759
memory efficiency, 763
Weight matrix
layer connections, 198
Weight Reuse
spatial, 634
Weight sharing, 250
Weight size
definition, 611
Weight update
optimization, 213
Weight-stationary
CNN optimization, 625
Weights
learnable parameters, 192
matrix organization, 198
Werbos, Paul
backpropagation, 189
Whetstone
etymology, 659
synthetic floating-point benchmark, 658
Wildlife conservation
TinyML monitoring, 67
WILDS
distribution shift benchmark, 718
Williams, Ronald J.
backpropagation, 189
Workflow
lab-deployment gap, 83
Workflow orchestration
pipeline automation, 815
Working set size, 281
Workload archetype
definition, 43
Workload distribution
heterogeneous processor, 646
Workload imbalance
definition, 620
Worst-Case Execution Time
edge AI, 833

XLA
accelerator compilation, 620
compilation speedup, 306
definition, 306
graph compilation, 306
warm-up compilation, 828
XOR problem
depth requirement, 201
nonlinearity requirement, 201

YOLOv8-nano
edge inference sizing, 59

Zillow
coupled decisions, 811
distribution shift failure, 892
Zillow Offers
forecast risk, 76